SPRINGER NATURE
Reference

Rafael Martí
Panos M. Pardalos
Mauricio G. C. Resende
Editors

Handbook of Heuristics

Second Edition

Springer

Handbook of Heuristics

Rafael Martí • Panos M. Pardalos •
Mauricio G. C. Resende
Editors

Handbook of Heuristics

Second Edition

With 371 Figures and 174 Tables

Springer

Editors

Rafael Martí
Operations Research and Statistics
Department, School of Mathematics
University of Valencia
Valencia, Spain

Panos M. Pardalos
Department of Industrial and Systems
Engineering, Center for Applied
Optimization
University of Florida
Gainesville, FL, USA

Mauricio G. C. Resende
Industrial and Systems Engineering
University of Washington
Seattle, WA, USA

ISBN 978-3-032-00384-3 ISBN 978-3-032-00385-0 (eBook)
https://doi.org/10.1007/978-3-032-00385-0

© Springer Nature Switzerland AG 2018, 2025

This work is subject to copyright. All rights are solely and exclusively licensed by the Publisher, whether the whole or part of the material is concerned, specifically the rights of translation, reprinting, reuse of illustrations, recitation, broadcasting, reproduction on microfilms or in any other physical way, and transmission or information storage and retrieval, electronic adaptation, computer software, or by similar or dissimilar methodology now known or hereafter developed.
The use of general descriptive names, registered names, trademarks, service marks, etc. in this publication does not imply, even in the absence of a specific statement, that such names are exempt from the relevant protective laws and regulations and therefore free for general use.
The publisher, the authors and the editors are safe to assume that the advice and information in this book are believed to be true and accurate at the date of publication. Neither the publisher nor the authors or the editors give a warranty, expressed or implied, with respect to the material contained herein or for any errors or omissions that may have been made. The publisher remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

This Springer imprint is published by the registered company Springer Nature Switzerland AG.
The registered company address is: Gewerbestrasse 11, 6330 Cham, Switzerland

If disposing of this product, please recycle the paper.

Preface

Heuristics have become a very popular family of solution methods for optimization problems because they are capable of finding *acceptable* solutions in a *reasonable* amount of time. The word heuristic means *serving to discover* or *find out*. It is an irregular formation from the Greek term *heuretikos*, which means *inventive*. It is related to the Greek word *heuriskein*, meaning *to find* and the word *Eureka*, which comes from the ancient Greek word *eurika*, which means *I have found it*.

In the last decades, algorithmic advances as well as hardware and software improvements have provided an excellent environment on which to build heuristic-based decision support systems based on new and effective methodologies. To the layman, heuristics may be thought of as *rules of thumb,* but despite its imprecision, the field of heuristics is very rich and refers to experience-based techniques for problem-solving, learning, and discovery. Any given heuristic solution is not guaranteed to be optimal, but heuristic methods are used to speed up the process of finding satisfactory solutions where optimal solutions are impractical. Despite the dynamic state of heuristic optimization, we feel that the time is ripe to bring together in one handbook the major algorithmic and methodological advances in this field. Leading experts in heuristic optimization have contributed to this handbook.

The main goal of this handbook is to provide the basic principles and fundamental ideas that will enable students and practitioners to create valuable applications based on heuristic technologies. Specifically, the *Handbook of Heuristics* is aimed at engineers, scientists, operations researchers, and other applications specialists who are looking for the most appropriate and recent optimization tools based on heuristic and metaheuristic methodologies to solve particular problems. The Handbook provides a broad spectrum of advances in heuristic optimization with a focus on its algorithmic and computational aspects.

The Handbook consists of five main parts: search strategies, local search, metaheuristics, analysis and implementations, and applications. In the part devoted to search strategies, we cover methodological aspects, from multi-objective and re-start mechanisms to *matheuristics,* the exciting field in which mathematical programming is combined with heuristics. As it is well known to practitioners of heuristic optimization, local search constitutes the core of many methodologies. For this reason, we devote an entire part of the Handbook to local search. In the metaheuristics part of the Handbook, 14 important methodologies in the major field of heuristic optimization are described. We also focus on the analysis and

implementation of these methods in this part of the book. Finally, the purpose of the part on applications is to provide the practitioner with a description of some relevant optimization issues in a number of specific application areas, such as scheduling, vehicle routing, network optimization, supply chain, diversity models, as well as some examples of applied optimization in specific areas.

The first edition of the *Handbook of Heuristics* was published in 2018 and contained 47 chapters. In this second edition, the authors revised 30 of them to include new developments in the area that appeared in the last few years. The inclusion of 14 new chapters makes this second edition even more comprehensive, totaling 61 chapters. In particular, the reader may find 14 chapters in search strategies, including a new chapter on deep learning, 4 in local search, 14 in metaheuristics, 5 in analysis and implementations, and 24 in implementations.

Finally, we would like to take the opportunity to thank Springer for inviting us to edit this Handbook, their staff for helping put the Handbook into production, the authors for their contributions, and the anonymous referees for their valuable comments and suggestions.

Valencia, Spain
Gainesville, FL, USA
Seattle, WA, USA
September 2025

Rafael Martí
Panos M. Pardalos
Mauricio G. C. Resende

Contents

Volume 1

Part I Search Strategies **1**

1 Adaptive and Multi-level Metaheuristics 3
Marc Sevaux, Kenneth Sörensen, and Nelishia Pillay

2 Biased Random-Key Genetic Programming 25
José Fernando Gonçalves and Mauricio G. C. Resende

3 Data Mining in Heuristics 41
Marcelo R. H. Maia, Isabel Rosseti, Simone de Lima Martins, and Alexandre Plastino

4 Deep Learning in Search Heuristics 71
Nayeli Gast Zepeda, André Hottung, and Kevin Tierney

5 Evolution Strategies 89
Michael Emmerich, Ofer M. Shir, and Hao Wang

6 Evolutionary Algorithms 125
David Corne and Michael A. Lones

7 Innovative Applications of Metaheuristics to Supervised Machine Learning 147
Pritam Paral, Amitava Chatterjee, and Patrick Siarry

8 Matheuristics by Examples 177
Martina Fischetti and Matteo Fischetti

9 Multi-start Methods 211
Rafael Martí, Jose A. Lozano, Alexander Mendiburu, and Leticia Hernando

10 Multiobjective Optimization 231
Carlos A. Coello Coello

11 Quantum-Inspired Heuristics 259
Seán McGarraghy and Milena Venkova

12 Restart Strategies 319
Oleg V. Shylo and Oleg A. Prokopyev

13 Simheuristics 335
Angel A. Juan, Majsa Ammouriova, Javier Faulin, Javier Panadero, and Daniele Ferone

14 The Hybrid Metaheuristic CMSA 363
Christian Blum and Jaume Reixach

Part II Local Search **387**

15 Constraint-Based Local Search 389
L. Michel and P. Van Hentenryck

16 Guided Local Search 427
Christos Voudouris, Abdullah Alsheddy, and Ahmad Alhindi

17 Theory of Local Search 469
W. Michiels, E. H. L. Aarts, and J. Korst

18 Variable Neighborhood Descent 511
Abraham Duarte, Nenad Mladenović, Jesús Sánchez-Oro, and Raca Todosijević

Part III Metaheuristics **541**

19 Ant Colony Optimization: A Component-Wise Overview 543
Manuel López-Ibáñez, Thomas Stützle, and Marco Dorigo

20 Estimation of Distribution Algorithms 583
Pedro Larrañaga and Concha Bielza

21 Fundamentals of Scatter Search 599
Rafael Martí, José Laguardia, and María Teresa León

22 Genetic Algorithms 627
Carlos García-Martínez, Francisco J. Rodriguez, and Manuel Lozano

23 GRASP 663
Daniele Ferone, Paola Festa, and Mauricio G. C. Resende

24 Hyper-heuristics 687
Michael G. Epitropakis and Edmund K. Burke

25 Iterated Greedy 745
Thomas Stützle and Rubén Ruiz

26 Iterated Local Search 779
Thomas Stützle and Rubén Ruiz

27 Memetic Algorithms 809
Carlos Cotta, Luke Mathieson, and Pablo Moscato

28 Particle Swarm Optimization 851
Dimitra G. Triantali and Konstantinos E. Parsopoulos

29 POPMUSIC 903
Éric D. Taillard and Stefan Voß

30 Random-Key Genetic Algorithms: Principles and Applications 921
Mariana A. Londe, Luciana S. Pessoa, Carlos E. Andrade, José Fernando Gonçalves, and Mauricio G. C. Resende

31 Tabu Search 941
Manuel Laguna

32 Variable Neighborhood Search 959
Pierre Hansen and Nenad Mladenović

Volume 2

Part IV Analysis and Implementation 989

33 A History of Metaheuristics 991
Kenneth Sörensen, Marc Sevaux, and Fred Glover

34 Alternate Optima in Combinatorial Optimization 1011
Marc Benito-Marimón, Anna Martínez-Gavara, and Rafael Martí

35 Heuristics in Design of Deep Neural Networks 1031
Ricardo Martins de Abreu Silva and Andersson Alves da Silva

36 Parallel Meta-heuristic Search 1087
Teodor Gabriel Crainic

37 Time Complexity Analysis of Stochastic Search Algorithms 1131
Pietro S. Oliveto

Part V Applications 1173

38 City Logistics 1175
Jaume Barceló, Hanna Grzybowska, and Jesús Arturo Orozco

39 Cutting and Packing 1219
Ramón Alvarez-Valdes, Maria Antónia Carravilla, and José Fernando Oliveira

40 Diversity Optimization Models 1269
Fernando Sandoya, Anna Martínez-Gavara, Ricardo Aceves, Abraham Duarte, Nilse Romero, Osvaldo Palma, and Rafael Martí

41 Evolutionary Algorithms for the Inverse Protein Folding Problem 1295
Sune S. Nielsen, Grégoire Danoy, Wiktor Jurkowski, Roland Krause, Reinhard Schneider, El-Ghazali Talbi, and Pascal Bouvry

42 Graph Layout Problems 1321
Sergio Cavero, Eduardo G. Pardo, Abraham Duarte, and Rafael Martí

43 Heuristics in Vehicle Routing 1353
Claudia Archetti, Kris Braekers, and Diego Cattaruzza

44 Last-Mile Delivery with Crowdshipping 1385
Tiago Monteiro, João Pedro Pedroso, and Ana Viana

45 Maritime Container Terminal Problems 1411
Christopher Expósito-Izquierdo, Eduardo Lalla-Ruiz, Jesica de Armas, Belén Melián-Batista, and J. Marcos Moreno-Vega

46 Metaheuristics for Medical Image Registration 1445
Andrea Valsecchi, Enrique Bermejo, Sergio Damas, and Oscar Cordón

47 Metaheuristics for Natural Gas Pipeline Network Optimization 1469
Roger Z. Ríos-Mercado

48 Metaheuristics for Problems in Sports Scheduling 1493
Celso C. Ribeiro, Sebastián Urrutia, and Dominique de Werra

49 Network Optimization 1523
Luciana S. Buriol

50 Optimization Problems, Models, and Heuristics in Wireless Sensor Networks 1541
Vinicius Morais, Fernanda S. H. Souza, and Geraldo R. Mateus

51 Particle Swarm Optimization for the Vehicle Routing Problem: A Survey and a Comparative Analysis 1563
Yannis Marinakis, Magdalene Marinaki, and Athanasios Migdalas

52 Scheduling Heuristics 1597
Rubén Ruiz

53 Search-Based Software Engineering Problems 1621
Javier Yuste, Eduardo G. Pardo, and Francisco Chicano

54 Selected String Problems 1651
Christian Blum and Paola Festa

55 Solving Large-Scale Vehicle Routing Problems 1673
Luca Accorsi and Daniele Vigo

56 Supply Chain Management 1697
Helena Ramalhinho Lourenço and Martín Gómez Ravetti

57 The Maximum Clique and Vertex Coloring 1715
Oleksandra Yezerska and Sergiy Butenko

58 The Multiplant Lot Sizing Problem with Multiple Periods and Items 1747
Mariá C. V. Nascimento, Horacio H. Yanasse, Desiree M. Carvalho, and Silvio Alexandre de Araujo

59 The OGA Company: The Role of Heuristics in Logistic Consulting Services 1771
Arturo López-Damas, Adolfo García, Reyes Maíllo, Alfredo G. Hernández-Díaz, Juan Carlos Rubio, and Rafael Martí

60 Trees and Forests 1793
Andréa Cynthia Santos, Christophe Duhamel, and Rafael Andrade

61 World's Best Universities and Personalized Rankings 1821
Mario Inostroza-Ponta, Natalie Jane de Vries, and Pablo Moscato

Index 1859

About the Editors

Rafael Martí
Operations Research and Statistics Department
School of Mathematics
University of Valencia
Valencia, Spain

Rafael Martí is a Professor of Statistics and Operations Research at the University of Valencia, Spain. He earned his PhD in Mathematics from the same university in 1994. His research focuses extensively on metaheuristics for complex optimization problems. He has published over 200 papers, most of them in indexed journals (JCR), including the *European Journal of Operational Research*, *INFORMS Journal on Computing*, *IIE Transactions*, *Journal of Global Optimization*, *Computers & Operations Research*, and *Discrete Applied Mathematics*. He is the co-author of several monographic books, the most recent being *Discrete Diversity and Dispersion Maximization* (Springer, 2023) and *Exact and Heuristic Methods in Combinatorial Optimization* (Springer-Verlag, 2022).

He serves as Area Editor for the *Journal of Heuristics* and as Associate Editor for the *European Journal of Operational Research* and *Mathematical Programming Computation*. He is also a senior research scientist at the Spanish logistics consulting firm OGA and holds a US patent. He has delivered around 50 invited and plenary talks and is a recurring visiting professor at the University of Colorado (USA), Molde University College (Norway), and University College Dublin (Ireland). He has participated in over 20 government-funded research projects, and his publications have received more than 15,000 citations.

Distinguished Emeritus Professor
Panos M. Pardalos
University of Florida
Gainesville, FL, USA

Panos Pardalos was born in Drosato (Mezilo) Argitheas GR in 1954 and graduated from Athens University (Department of Mathematics). He received his PhD (Computes and Information Sciences) from the University of Minnesota. He is an Emeritus Distinguished Professor in the Department of Industrial and Systems Engineering at the University of Florida, and an affiliated faculty of Biomedical Engineering and Computer Science & Information & Engineering departments. Since 2011 he has been the academic advisor at *LATNA, HSE*.

He is a world-renowned leader in Global Optimization, Mathematical Modeling, Energy Systems, Financial applications, and Data Sciences. He is a Fellow of *AAAS, AAIA, AIMBE, EUROPT*, and *INFORMS* and was awarded the 2013 Constantin Caratheodory Prize of the International Society of Global Optimization. In addition, He has been awarded the 2013 EURO Gold Medal prize bestowed by the Association for European Operational Research Societies. This medal is the preeminent European award given to Operations Research (OR) professionals for "scientific contributions that stand the test of time."

He has been awarded a prestigious Humboldt Research Award (2018–2019). The Humboldt Research Award is granted in recognition of a researcher's entire achievements to date – fundamental discoveries, new theories, insights that have had significant impact on their discipline.

He is also a member of several Academies of Sciences, and he holds several honorary PhD degrees and affiliations. He is the Founding Editor of *Optimization Letters* and *Energy Systems*, and Co-Founder of the *International Journal of Global Optimization, Computational Management Science*, and Springer Nature *Operations Research Forum*. He has published over 600 journal papers, and edited/authored over 200 books. He is one of the most cited authors and has graduated 71 PhD students so far. Details can be found in https://faculty.eng.ufl.edu/pardalos/publications/.

Mauricio G. C. Resende
University of Washington
Seattle, WA, USA

Mauricio G. C. Resende is an Affiliate Professor of the Graduate Program in Operations Research of the Aeronautical Institute of Technology (ITA) and the Federal University of São Paulo (UNIFESP) and a Permanent Member of the Center for Discrete Mathematics and Theoretical Computer Science (DIMACS). He is most known for his work with metaheuristics and interior point methods for linear programming and network flows. He has published 5 books and over 200 papers on optimization. He holds 15 US patents. With a successful career spanning over four decades, he has brought his expertise to various industrial research laboratories worldwide: electrical power systems at Furnas Centrais Elétricas in Brazil; semiconductor device manufacturing at Fairchild Semiconductor Corporation in Silicon Valley, California; telecommunications at AT&T Bell Labs Research and AT&T Labs Research in New Jersey; and retail logistics at Amazon in Seattle. He holds a PhD in operations research from the University of California, Berkeley. He is an INFORMS Fellow.

Contributors

E. H. L. Aarts Tilburg University, Tilburg, The Netherlands

Luca Accorsi Google, Munich, Germany

Ricardo Aceves Universidad Nacional Autónoma de México, México City, México

Ahmad Alhindi Department of Computer Science and Artificial Intelligence, College of Computers, Umm Al-Qura University, Makkah, Saudi Arabia

Abdullah Alsheddy College of Computer and Information Sciences (CCIS), Imam Mohammad Ibn Saud Islamic University (IMSIU), Riyadh, Saudi Arabia

Ramón Alvarez-Valdes Universitat de València, Burjassot, Spain

Majsa Ammouriova German Jordanian University, Amman, Jordan

Universitat Oberta de Catalunya, Barcelona, Spain

Carlos E. Andrade AT&T Labs Research, Bedminster, NJ, USA

Rafael Andrade Departamento de Estatística e Matemática Aplicada, Centro de Ciências, Universidade Federal do Ceará, Fortaleza, Brazil

Claudia Archetti Department of Economics and Management, University of Brescia, Brescia, Italy

Jaume Barceló Department of Statistics and Operations Research, Universitat Politècnica de Catalunya, Barcelona, Spain

Marc Benito-Marimón Universidad de Valencia, Valencia, Spain

Enrique Bermejo Department of Computer Science and Artificial Intelligence, Unviersity of Granada, Granada, Spain

Concha Bielza Departamento de Inteligencia Artificial, Universidad Politécnica de Madrid, Madrid, Spain

Christian Blum Artificial Intelligence Research Institute (IIIA-CSIC), Bellaterra, Spain

Pascal Bouvry Computer Science and Communications (CSC) Research Unit, FSTC, University of Luxembourg, Luxembourg City, Luxembourg

Kris Braekers Research Group Logistics, Hasselt University, Diepenbeek, Belgium

Luciana S. Buriol Instituto de Informática, Universidade Federal do Rio Grande do Sul, Porto Alegre, Brazil

Edmund K. Burke School of Electronic Engineering and Computer Science, Queen Mary University of London, London, UK

Sergiy Butenko Department of Industrial and Systems Engineering, Texas A&M University, College Station, TX, USA

Maria Antónia Carravilla INESC TEC and Faculty of Engineering, University of Porto, Porto, Portugal

Desiree M. Carvalho Instituto de Ciência e Tecnologia, Universidade Federal de São Paulo - UNIFESP, São José dos Campos, SP, Brazil

Diego Cattaruzza Univ. Lille, CNRS, Centrale Lille, Inria, UMR 9189 – CRIStAL Centre de Recherche an Informatique Signal et Automatique de Lille, Lille, France

Department of Mathematics, Computer Science and Physics, University of Udine, Udine, Italy

Sergio Cavero Departamento de Informática y Estadística, Universidad Rey Juan Carlos, Móstoles, Spain

Amitava Chatterjee Department of Electrical Engineering, Jadavpur University, Kolkata, India

Francisco Chicano ITIS Software, Universidad de Málaga, Málaga, Spain

Carlos A. Coello Coello Departmento de Computación, CINVESTAV-IPN, México D.F., México

Oscar Cordón Department of Computer Science and Artificial Intelligence, Unviersity of Granada, Granada, Spain

David Corne Heriot-Watt University, Edinburgh, UK

Carlos Cotta Instituto de Ingeniería y Tecnología del Software (ITIS), Universidad de Málaga, Málaga, Spain

Teodor Gabriel Crainic School of Management, Université du Québec à Montréal, Montréal, QC, Canada

CIRRELT – Interuniversity Research Centre on Enterprise Networks, Logistics and Transportation, Montréal, QC, Canada

Sergio Damas Department Software Engineering, University of Granada, Granada, Spain

Grégoire Danoy Computer Science and Communications (CSC) Research Unit, FSTC, University of Luxembourg, Luxembourg City, Luxembourg

Andersson Alves da Silva Federal University of Pernambuco, Recife, PE, Brazil

Ricardo Martins de Abreu Silva Federal University of Pernambuco, Recife, PE, Brazil

Silvio Alexandre de Araujo Universidade Estadual Paulista-UNESP, São José do Rio Preto, SP, Brazil

Jesica de Armas Department of Economics and Business, University Pompeu Fabra, Barcelona, Spain

Simone de Lima Martins Universidade Federal Fluminense, Niterói, Rio de Janeiro, Brazil

Natalie Jane de Vries School of Electrical Engineering and Computing, Faculty of Engineering and Built Environment, The University of Newcastle, Callaghan, NSW, Australia

Dominique de Werra Département de Mathématiques, Ecole Polytechnique Fédérale de Lausanne, Lausanne, Switzerland

Marco Dorigo IRIDIA, Université Libre de Bruxelles (ULB), Brussels, Belgium

Abraham Duarte Department of Computer Science and Statistics, Universidad Rey Juan Carlos, Mostoles, Spain

Christophe Duhamel LITIS, Le Havre Normandie University, UFR Sciences et Techniques, Le Havre Cedex, France

Michael Emmerich University of Jyväskylä, Jyväskylä, Finland

Michael G. Epitropakis Department of Management Science, Data Science Institute, Lancaster University Management School, Lancaster University, Lancaster, UK

Christopher Expósito-Izquierdo Department of Computer Engineering and Systems, Universidad de La Laguna, Santa Cruz de Tenerife, Spain

Javier Faulin Public University of Navarra, Pamplona, Spain

Daniele Ferone Department of Mathematics and Applications "R. Caccioppoli", University of Napoli FEDERICO II, Naples, Italy

Paola Festa Department of Mathematics and Applications "R. Caccioppoli", University of Napoli FEDERICO II, Naples, Italy

Martina Fischetti Department of Statistics and Operations Research, University of Seville, Seville, Spain

Matteo Fischetti DEI, University of Padova, Padova, Italy

Adolfo García OGA, Sevilla, Spain

Carlos García-Martínez Computer Science and Numerical Analysis, University of Córdoba, Córdoba, Spain

Nayeli Gast Zepeda Bielefeld University, Bielefeld, Germany

Fred Glover Entanglement, Inc., Coral Gables, FL, USA

José Fernando Gonçalves Faculdade de Economia da, Universidade do Porto, Porto, Portugal

Hanna Grzybowska School of Civil and Environmental Engineering, Research Centre for Integrated Transport Innovation, University of New South Wales, Sydney, NSW, Australia

Pierre Hansen GERAD and Ecole des Hautes Etudes Commerciales, Montréal, QC, Canada

Alfredo G. Hernández-Díaz Universidad Pablo de Olavide, Sevilla, Spain

Leticia Hernando Intelligent Systems Group, Department of Computer Science and Artificial Intelligence, University of the Basque Country, Donostia, Spain

André Hottung Bielefeld University, Bielefeld, Germany

Mario Inostroza-Ponta Departamento de Ingeniería Informática, Universidad de Santiago, Santiago, Chile

Angel A. Juan Universitat Politècnica de València, Valencia, Spain

Wiktor Jurkowski The Genome Analysis Centre (TGAC), Norwich Research Park, Norwich, UK

J. Korst Philips Research Laboratories, Eindhoven, The Netherlands

Roland Krause Luxembourg Centre for Systems Biomedicine (LCSB), University of Luxembourg, Luxembourg City, Luxembourg

José Laguardia Facultad de Ciencias y Tecnología, Universidad Teconológica de Panamá, Panamá, República de Panamá

Manuel Laguna Leeds School of Business, University of Colorado Boulder, Boulder, CO, USA

Eduardo Lalla-Ruiz Department of High-Tech Business and Entrepreneurship, University of Twente, Enschede, The Netherlands

Pedro Larrañaga Departamento de Inteligencia Artificial, Universidad Politécnica de Madrid, Madrid, Spain

Mariana A. Londe Pontifical Catholic University of Rio de Janeiro, Rio de Janeiro, RJ, Brazil

Michael A. Lones Heriot-Watt University, Edinburgh, UK

Arturo López-Damas OGA, Sevilla, Spain

Manuel López-Ibáñez Alliance Manchester Business School, University of Manchester, Manchester, UK

Helena Ramalhinho Lourenço Universitat Pompeu Fabra, Barcelona, Spain

Jose A. Lozano Intelligent Systems Group, Department of Computer Science and Artificial Intelligence, University of the Basque Country, Donostia, Spain

Manuel Lozano Department of Computer Science and Artificial Intelligence, University of Granada, Granada, Spain

Marcelo R. H. Maia Universidade Federal Fluminense, Niterói, Rio de Janeiro, Brazil

Instituto Brasileiro de Geografia e Estatística, Rio de Janeiro, Brazil

Reyes Maíllo OGA, Sevilla, Spain

Magdalene Marinaki School of Production Engineering and Management, Technical University of Crete, Chania, Greece

Yannis Marinakis School of Production Engineering and Management, Technical University of Crete, Chania, Greece

Rafael Martí Operations Research and Statistics Department, School of Mathematics, University of Valencia, Valencia, Spain

Anna Martínez-Gavara Universidad de Valencia, Valencia, Spain

Geraldo R. Mateus Departamento de Ciência da Computação, Universidade Federal de Minas Gerais, Belo Horizonte, Brazil

Luke Mathieson School of Computer Science, University of Technology Sydney, Sydney, NSW, Australia

Seán McGarraghy School of Business, University College Dublin, Dublin, Ireland

Belén Melián-Batista Department of Computer Engineering and Systems, Universidad de La Laguna, Santa Cruz de Tenerife, Spain

Alexander Mendiburu Intelligent Systems Group, Department of Computer Architecture and Technology, University of the Basque Country, Donostia, Spain

L. Michel University of Connecticut, Storrs, CT, USA

W. Michiels NXP, Eindhoven, The Netherlands

Eindhoven University of Technology, Eindhoven, The Netherlands

Athanasios Migdalas Industrial Logistics, Luleå Technical University, Luleå, Sweden

Department of Civil Engineering, Aristotle University of Thessalonike, Thessalonike, Greece

Nenad Mladenović GERAD and Ecole des Hautes Etudes Commerciales, Montréal, QC, Canada

LAMIH, Université de Valenciennes, Valenciennes, France

Mathematical Institute, SANU, Belgrade, Serbia

Tiago Monteiro INESC Technology and Science, Porto, Portugal

Faculty of Engineering, University of Porto, Porto, Portugal

Vinicius Morais Departamento de Ciência da Computação, Universidade Federal de Minas Gerais, Belo Horizonte, Brazil

J. Marcos Moreno-Vega Department of Computer Engineering and Systems, Universidad de La Laguna, Santa Cruz de Tenerife, Spain

Pablo Moscato School of Information and Physical Sciences, University of Newcastle, Newcastle, NSW, Australia

School of Electrical Engineering and Computing, Faculty of Engineering and Built Environment, The University of Newcastle, Callaghan, NSW, Australia

Mariá C. V. Nascimento Instituto Tecnológico de Aeronáutica (ITA), São José dos Campos, SP, Brazil

Sune S. Nielsen Computer Science and Communications (CSC) Research Unit, FSTC, University of Luxembourg, Luxembourg City, Luxembourg

José Fernando Oliveira INESC TEC and Faculty of Engineering, University of Porto, Porto, Portugal

Pietro S. Oliveto Department of Computer Science and Engineering, Southern University of Science and Technology, Shenzhen, China

Jesús Arturo Orozco Department of Operations Management, IPADE Business School, Mexico City, Mexico

Osvaldo Palma Universidad Nacional Autónoma de México, México City, México

Javier Panadero Universitat Autònoma de Barcelona, Barcelona, Spain

Pritam Paral Department of Electrical Engineering, IIEST, Shibpur, Howrah, India

Eduardo G. Pardo Departamento de Informática y Estadística, Universidad Rey Juan Carlos, Madrid, Spain

Konstantinos E. Parsopoulos Department of Computer Science and Engineering, University of Ioannina, Ioannina, Greece

João Pedro Pedroso CMUP and Department of Computer Science, Faculty of Sciences, University of Porto, Porto, Portugal

Luciana S. Pessoa Pontifical Catholic University of Rio de Janeiro, Rio de Janeiro, RJ, Brazil

Nelishia Pillay University of Pretoria, Pretoria, South Africa

Alexandre Plastino Universidade Federal Fluminense, Niterói, Rio de Janeiro, Brazil

Oleg A. Prokopyev University of Pittsburgh, Pittsburgh, PA, USA

Martín Gómez Ravetti Universidade Federal de Minas Gerais, Belo Horizonte, Brazil

Jaume Reixach Artificial Intelligence Research Institute (IIIA-CSIC), Bellaterra, Spain

Mauricio G. C. Resende Industrial and Systems Engineering, University of Washington, Seattle, WA, USA

Celso C. Ribeiro Institute of Computing, Universidade Federal Fluminense, Niterói, Brazil

Roger Z. Ríos-Mercado Graduate Program in Electrical Engineering, Universidad Autónoma de Nuevo León (UANL), San Nicolás de los Garza, NL, Mexico

Francisco J. Rodriguez Department of Computer Science and Artificial Intelligence, University of Granada, Granada, Spain

Nilse Romero Universidad Nacional Autónoma de México, México City, México

Isabel Rosseti Universidade Federal Fluminense, Niterói, Rio de Janeiro, Brazil

Juan Carlos Rubio OGA, Sevilla, Spain

Rubén Ruiz AMAZON, Amazon.com, Inc., Seattle, WA, USA

Universitat Politécnica de València, València, Spain

Jesús Sánchez-Oro Department of Computer Science and Statistics, Universidad Rey Juan Carlos, Mostoles, Spain

Fernando Sandoya Escuela Superior Politécnica del Litoral, Guayaquil, Ecuador

Universidad de Guayaquil, Guayaquil, Ecuador

Andréa Cynthia Santos LITIS, ISEL, Le Havre Normandie University, Le Havre Cedex, France

Reinhard Schneider Luxembourg Centre for Systems Biomedicine (LCSB), University of Luxembourg, Luxembourg City, Luxembourg

Marc Sevaux Lab-STICC, CNRS, Université Bretagne Sud, Lorient, France

Ofer M. Shir Computer Science Department, Tel-Hai College, Upper Galilee, Israel

The Galilee Research Institute – Migal, Upper Galilee, Israel

Oleg V. Shylo University of Tennessee, Knoxville, TN, USA

Patrick Siarry Université Paris-Est Créteil, Créteil, Cedex, France

Kenneth Sörensen University of Antwerp, Antwerp, Belgium

Fernanda S. H. Souza Departamento de Ciência da Computação, Universidade Federal de São João del-Rei, São João del-Reil, Brazil

Thomas Stützle IRIDIA, Université Libre de Bruxelles (ULB), Brussels, Belgium

Éric D. Taillard HEIG-VD, University of Applied Sciences and Arts of Western Switzerland, Yverdon-les-Bains, Switzerland

El-Ghazali Talbi Université des sciences et technologies de Lille, INRIA Lille Nord Europe, Villeneuve d'Ascq, France

María Teresa León Facultat de Ciències Matemàtiques, Departament d'Estadìstica i Investigació Operativa, University of Valencia, València, Spain

Kevin Tierney Bielefeld University, Bielefeld, Germany

Raca Todosijević Univ. Polytechnique Hauts-de-France, LAMIH, CNRS, Valenciennes, France

INSA Hauts-de-France, Valenciennes, France

Dimitra G. Triantali Department of Computer Science and Engineering, University of Ioannina, Ioannina, Greece

Sebastián Urrutia Faculty of Logistics, Molde University College, Molde, Norway

Andrea Valsecchi Department of Computer Science and Artificial Intelligence, Unviersity of Granada, Granada, Spain

P. Van Hentenryck School of Industrial and Systems Engineering, Georgia Institute of Technology, Atlanta, GA, USA

Milena Venkova School of Mathematics & Statistics, Technological University of Dublin, Dublin, Ireland

Ana Viana INESC Technology and Science, Porto, Portugal

ISEP, Polytechnic of Porto, Porto, Portugal

Daniele Vigo Department of Electrical, Electronic and Information Engineering "G. Marconi", University of Bologna, Bologna, Italy

CIRI ICT, University of Bologna, Bologna, Italy

Stefan Voß Faculty of Business Administration, Institute of Information Systems, University of Hamburg, Hamburg, Germany

Christos Voudouris AI Team, Department of Informatics and Telecommunications, National and Kapodistrian University of Athens, Athens, Greece

Hao Wang Leiden Institute of Advanced Computer Science, Leiden University, Leiden, The Netherlands

Horacio H. Yanasse Instituto de Ciência e Tecnologia, Universidade Federal de São Paulo - UNIFESP, São José dos Campos, SP, Brazil

Oleksandra Yezerska Department of Industrial and Systems Engineering, Texas A&M University, College Station, TX, USA

Javier Yuste Universidad Rey Juan Carlos, Madrid, Spain

Part I
Search Strategies

Adaptive and Multi-level Metaheuristics

1

Marc Sevaux, Kenneth Sörensen, and Nelishia Pillay

Contents

Introduction 4
Definitions 5
Configuring a Metaheuristic 8
Adaptive Metaheuristics 10
  Simple Adaptive Mechanisms 11
  Reactive Search 11
  Greedy Randomized Adaptive Search Procedure 11
  Adaptive Large Neighborhood Search 12
Multi-Level Metaheuristics and Hyper-Heuristics 14
  Hyper-Heuristics 14
  Hyper-Heuristics for Metaheuristic Configuration 16
  Discussion 18
Conclusion 19
Perspectives 20
Cross-References 21
References 21

M. Sevaux (✉)
Lab-STICC, CNRS, Université Bretagne Sud, Lorient, France
e-mail: marc.sevaux@univ-ubs.fr

K. Sörensen
University of Antwerp, Antwerp, Belgium
e-mail: kenneth.sorensen@uantwerpen.be

N. Pillay
University of Pretoria, Pretoria, South Africa
e-mail: nelishia.pillay@up.ac.za

© Springer Nature Switzerland AG 2025

R. Martí et al. (eds.), *Handbook of Heuristics*,
https://doi.org/10.1007/978-3-032-00385-0_16

Abstract

In recent decades, metaheuristics have become increasingly popular as a tool for solving a large class of difficult optimization problems. However, determining the best configuration of a metaheuristic, which includes the program flow and the parameter settings, remains a difficult task. Adaptive metaheuristics (that change their configuration during the search) and multi-level metaheuristics (that change their configuration during the search by means of a metaheuristic) can be a solution for this. This chapter intends to give a quick review of the latest trends in adaptive metaheuristics and multi-level metaheuristics.

Keywords

Metaheuristics · Multi-level · Adaptive · Configuration · Hyper-heuristics

Introduction

Metaheuristics are flexible frameworks that can be used to design heuristics for virtually any combinatorial optimization problem. This flexibility also comes at a cost: many researchers in metaheuristics spend a large amount of time to properly design and tune their algorithm in a trial-and-error fashion. As mentioned and observed in many published papers, designing an efficient metaheuristic is an art, requiring a lot of intuition on the part of the metaheuristic designer. However, there is no doubt that the parameters and structure of the metaheuristic may influence the performance of the solution approach and the quality of the results in the end.

In the design of a metaheuristic, a large fraction of the time is usually spent on determining the *control flow*, i.e., the order in which the different components of the metaheuristic are used, and the optimal levels of the various parameters of the metaheuristic. A more structured approach than the one commonly used is needed. The purpose of this chapter is to discuss one of the ways to alleviate this problem, through adaptive and multi-level metaheuristics.

Our overview is necessarily very short and cannot replace the many years of research and the large number of books and papers that have tried to clarify the topic. We encourage the reader to address the existing work that we have pointed out and to explore the references that we may have missed here. This chapter is not a complete review of all papers in the field of adaptive or multi-level metaheuristics but rather a set of good practices that can be done when designing metaheuristics.

In the first section, we will try to clearly define some concepts to arrive at a definition of the terms *adaptive* and *multi-level* (section "Definitions"). We will also discuss the impact of heuristic parameters on the behavior of a metaheuristic algorithm (section "Configuring a Metaheuristic") and how we can reduce the number of parameters or how we can automatically tune some parameters. Adaptive

metaheuristics are discussed in section "Adaptive Metaheuristics" and multi-level metaheuristics and hyper-heuristics in section "Multi-Level Metaheuristics and Hyper-Heuristics." Section "Conclusion" will conclude the chapter.

Definitions

The terms *adaptive* and *multi-level* do not have a single, generally accepted definition within the metaheuristics community. The aim of this section is to develop clear definitions for both. Both *adaptive* and *multi-level* refer to the evolution of the *configuration* of a metaheuristic algorithm during the optimization process. We must therefore first define the term *configuration*. Our definitions will be intuitive rather than formal.

Each metaheuristic algorithm consists of several *components*, i.e., parts that form a more or less logical and atomic unit. Examples are a local search operator in a variable search algorithm, a tabu list in a tabu search algorithm, a crossover operator, or a selection operator in an evolutionary algorithm. Each of these components can exist more or less independently of the rest of the metaheuristic algorithm, which includes their use in a different metaheuristic algorithm for the same problem. Within the same metaheuristics, components can often be rearranged in the overall structure of the algorithm. For example, local search operators in a variable neighborhood search algorithm can be executed in a specific order; an evolutionary algorithm may use its selection operator before or after the crossover operator (or both), etc. In programming, this is called the *control flow*, a term we will adopt here. Clearly, not every control flow makes sense, but generally speaking, a sizeable number of possibilities exist. Determining the control flow of the algorithm, i.e., the order in which the components are executed, is a task for the algorithm designer.

Each component may have one or more parameters that determine its functioning. Such parameters may be numerical, such as the tabu tenure of a tabu list, the number of iterations without improvement before a perturbation move is used, etc. Other parameters may be nonnumerical, like the choice of a discrete set of move strategies to use in a local search operator (steepest descent, mildest descent, random improving, etc.). The algorithm designer usually defines a finite set of potential (or sensible) values, e.g., the restricted candidate list of a GRASP algorithm is defined to be an integer number between 5 and 20. In some situations, a parameter might also be a real number.

Using these concepts, we may now define the term *configuration*.

Definition 1. Given the set of components of a metaheuristic algorithm together with the set of all potential control flow alternatives, as well as its parameters and their potential values, the **configuration** of a metaheuristic algorithm defines the specific control flow and the specific set of parameter values it uses.

For example, a local search-based metaheuristic like the most basic tabu search method depicted in Algorithm 1 has one parameter, the tabu tenure (the length of the tabu list), and needs one type of neighborhood. It also needs several functions such as *initialize* and *update memory*. One specific tabu tenure, one type of neighborhood, and the set of necessary functions will be the configuration of the current search method.

Algorithm 1: Basic tabu search

```
1 initialize: find an initial solution x
2 repeat
3 |   neighborhood search: find a solution x' ∈ N*(x)
4 |   update memory: tabu list, frequency-based memory, aspiration level, ...
5 |   move x ← x'
6 until stopping criterion satisfied
```

For a simple variable neighborhood search heuristic presented in Algorithm 2, the order in which neighborhoods are explored (or whether they are explored at all), the way in which the starting solution in the current neighborhood is generated, and the local search method that is applied to improve the solution with the current neighborhood, as well as the total number of iterations k_{max}, define the configuration of the VNS.

Algorithm 2: Basic variable neighborhood search

```
1 initialize: find an initial solution x, k ← 1
2 repeat
3 |   shake: generate a point x' at random from the neighborhood N_k(x)
4 |   local search: apply a local search procedure starting from the solution x' to find a
  |   solution x''
5 |   if x'' is better than x then
6 |   |   x ← x'' and k ← 1 (center the search around x'' and search again with
  |   |   neighborhood 1)
7 |   else
8 |   |   k ← k + 1 (enlarge the neighborhood)
9 until k = k_max
```

We can now define the terms *adaptive* and *multi-level* in the context of metaheuristic algorithms.

Definition 2. A metaheuristic is **adaptive** when it includes a mechanism to modify its configuration *during* its execution.

In other words, an adaptive heuristic includes a mechanism to modify either the control flow or the parameter values (or both) of a heuristic, and, by doing so modify

the behavior of the metaheuristic. As an example, consider the basic tabu search shown in Algorithm 1. A common adaptive mechanism might make some changes to the *update memory*: the length of the tabu list, the aspiration level, etc.

Even though the definition has been conceived to be as watertight as possible, there is always room for interpretation. For example, considering the VNS heuristic in Algorithm 2, the basic design of the heuristic is such that the order in which the neighborhoods are searched depends on the instance being solved. Since the heuristic will move to the next neighborhood once a local optimum has been reached (and local optima are different for every instance), the control flow of the algorithm will be different for every instance. Yet, most researchers would not call a simple VNS heuristic adaptive.

There is a potential for recursion in the definition as well. A mechanism to determine the control flow or parameter values, by definition, becomes a component of the metaheuristic. This component must be integrated into the control flow itself and may also introduce its own parameters. This setup could potentially lead to a new level of adaptiveness, where an even higher-level component is introduced to control the parameters of the control mechanism, and so on. However, most adaptive metaheuristics include only one additional control component. In these cases, the parameter values are either self-defined or determined externally, so this recursion does not pose a problem.

The mechanism that does the actual adaptation can range from very simple to complex. Essentially, the aim of the mechanism is to search for the best configuration of the metaheuristic algorithm. This search can itself be seen as a combinatorial optimization problem, and an optimization algorithm may be used to solve it. When the adaptation mechanism itself is a metaheuristic algorithm, we call the overall result a *multi-level metaheuristic*.

Definition 3. A **multi-level** metaheuristic algorithm is a metaheuristic algorithm for which the configuration is altered by another metaheuristic algorithm.

Note that, since the algorithm doing the adapting is itself a heuristic, with its own components and parameters, the road is paved (again) for a recursive structure in which the configuration of the lowest-level heuristic is adapted by a higher-level metaheuristic, the configuration of which is adapted by an even higher-level heuristic, the configuration of which is adapted by ..., ad infinitum. However, the complexity added by implementing a higher-level metaheuristic algorithm to adapt the configuration of a lower-level metaheuristic is usually considerable, which precludes the design of a multi-level heuristic having many levels.

For reasons of clarity, a hybrid metaheuristic, i.e., a metaheuristic algorithm that combines ideas from several metaheuristic frameworks, for which the configuration remains unchanged throughout the search, is not considered a multi-level metaheuristic in this chapter. Also, the term multi-level is often used to denote optimization problems that can be decomposed into several (simpler) problems (e.g., a location-routing problem can often be decomposed into a location problem and a routing problem). Each of the problems may be separately solved by a metaheuristic

algorithm. In this chapter, we do not use the term multi-level metaheuristics to describe such approaches.

Configuring a Metaheuristic

An optimization method and especially a metaheuristic has several (potentially hundreds) possible configurations. Among all of them, only a few will allow the search to reach the optimal solution or the best possible solution, but not for all instances and not at all time. The configuration may have a great influence on the quality of the final solution or the effectiveness of the search method. This is the reason why properly tuning the parameters, or choosing the right configuration, is a very important task. Because of the "no free lunch theorem" [42], we know that there is no optimal configuration of a metaheuristic that will outperform all others on all problems and all instances.

Despite this fact, it is necessary to find an initial configuration, suitable to solve the problem at hand. This initial part will be described in this section. If the configuration is not satisfying, instead of starting again the resolution with a new configuration, one can change the configuration during the search (see section "Adaptive Metaheuristics").

As noted by several researchers [e.g., 16], the first step of setting up a metaheuristic has to go through the configuration phase during which the control flow is established and the parameters are tuned. This phase is sometimes called "offline parameter initialization." This is long and fastidious and usually done with a trial-and-error method. Moreover, even when this step is completed, its efficiency is often effective only on a subset of instances (usually close to the instances on which the parameters have been calibrated). In addition, as already mentioned for metaheuristics, the parameters are not only numerical values, but can be search components, updating function, etc. [38].

As a general observation, tuning these parameters is often so difficult that the designers change them one by one until they get the right configuration. And the value of these parameters is obtained empirically. Hence, the final combination of the parameters deduced from a sequential empirical adaptation of the parameters cannot guarantee that the final configuration is optimal. Furthermore, by changing the parameters one by one, it is impossible (or too difficult) to detect the possible interactions between these parameters. Moreover, the parameters and the control flow of a heuristic generally heavily influence each other, which makes the process of determining both even more difficult.

Eiben et al. [19] clearly define the parameter tuning for evolutionary algorithms: *By parameter tuning we mean the commonly practiced approach that amounts to finding good values for the parameters before the run of the algorithm and then running the algorithm using these values, which remain fixed during the run.*

Once this initial tuning phase is completed, most metaheuristics designers keep the configuration as it is to run their solution approach on the set of instances studied.

This configuration remains the same (as cited above) until the designer believes that it is not adapted anymore and should be reconfigured again with the same process.

Among the potential methods for tuning the parameters before solving, one can list:

- Manual tuning (usually from the experience of a metaheuristic designer),
- Parameter tuning on a subset of representative instances,
- Automatic parameter tuning by the use of an external method.

An experienced metaheuristic designer is often able to decide the value of a large number of parameters beforehand. The rules of thumb prevail on every other considerations. The reason for this is that after several years of practicing, one can know that some parameters need to have certain values, and the values that are close to it will not make a big difference on the final effectiveness of the results.

The population of a genetic algorithm is a rather good example. A small population will have a premature convergence because too many individuals will be the same (clones), and to avoid this without a specific mechanism, it is important to have a population of a large size. Many papers on genetic algorithms have a population of 100 individuals, but none mentioned how they have obtained this value, or the motivations to set it to this value. No analysis is done to see if 95 or 105 will give better results.

Only a few researchers report the difficulty of finding the right parameter settings and the limitations of this kind of approach [41]. Moreover, the manual tuning, without post-analysis experiments, has the drawback of not being applicable to different instances than the one presented in the paper. This is even more the case for transferring the method to any similar industrial application.

Whatever the technique used for tuning these parameters or configurations beforehand, it is important to keep in mind that every metaheuristic should be well balanced between *intensification* and *diversification* [37]. Hence, the tuning of the parameters should take this into account for ensuring that the metaheuristic is not converging too fast (too much intensification or exploitation) or is wandering in the search space without converging (too much diversification or exploration). Of course, for being able to detect this, one has to set up some indicators showing the speed of convergence, the evolution of the solution quality, the evolution of the solutions themselves, etc.

The best practice is to report the results as Prins [31] has done in his paper on the vehicle routing problem. In that paper, the tables present at the same time the results obtained on a set of instances with some "standard parameter settings" and the results obtained with the "best parameter settings." This is a fair comparison to existing work. The only drawback is the missing information on the time needed to set up the standard parameter settings as well as the best parameter settings. This can be a long and fastidious task and the total computational time can be high.

Setting up the parameters of a metaheuristic based on preliminary experiments is probably the most common technique used in designing metaheuristics. A subset of

representative instances is selected, and the parameters are tested on these instances until they converge to stable results. Hence, they are applied to the whole set of instances and the results are reported. The subset of instances should be carefully selected to be representative of the future experiments.

Usually, the designer selects one parameter at a time, adjust its value to the best one, and reiterate with the next parameter. Only a few reports that they practice a full factorial design as stated by Hooker [22]. With such a design, authors may try to understand the relative contribution of each parameter to the global effectiveness of the metaheuristic and the possible influence of the parameters between them. One of the best examples of the application of this type of parameter design is presented by Xu and Kelly [45] on a tabu search algorithm. In their paper, they have selected a small subset of seven instances to tune five components of the tabu search. A more general approach is presented by Xu et al. [46] but still based on the same factorial analysis.

Another technique to find the right parameter settings is to let an external method to tune the parameters for the metaheuristic designers. Very few studies exist on that issue if they are not used intrinsically during the metaheuristic search. Dean and Voss [12] in their book present a technique known as the *response surface methodology* in statistics. This method has been effectively used in [1]. This technique consists in running a local search method in the space of the parameters. A specific metric measuring the distance between each pair of parameters is calculated by running the metaheuristic. For a fixed setting of parameters (or a point in the search space of the parameters), the neighbors are also explored. If no better neighbor can be found, the value for the parameters is fixed and the search stops; otherwise, the search continues with the best neighbor and the new parameter settings.

Of course, one cannot guaranty the optimality for all the parameters at once and even at the end of the search. But usually, this technique is able to discover good parameter settings. One important drawback is the definition of the metric that is very sensitive, especially if an order cannot be defined on the variables.

For a more elaborate discussion on this topic, we refer the reader to [5] and to section "Hyper-Heuristics for Metaheuristic Configuration" in this chapter.

Adaptive Metaheuristics

As stated in the previous section, once a designer has the best parameter settings for its metaheuristics, he is able to run it confidently on the set of targeted instances and produce results. The question is: "Can it go further?" And the answer is yes. Yes, there is always some space for improvement. A parameter setting that works very well on one instance might work poorly on another one.

To overcome this difficulty, it is always possible to analyze the behavior of the metaheuristic during the search and adapt it to obtain better results. This phase can be named "online parameter tuning." Based on indicators (e.g., convergence, solution quality, similarity of explored solutions), the configuration of the metaheuristic

is changed. This technique is particularly appealing when one has to solve only one large instance and the tuning of parameters cannot be done beforehand.

Simple Adaptive Mechanisms

Detecting why a metaheuristic is not giving satisfying results is not an easy task. It largely depends on the type of metaheuristic itself. For example, identical individuals (clones) in a genetic or memetic algorithm are one of the known consequences of premature convergence. In a local search method, cycling in the objective space, or in the solution space, is also a situation that needs to be avoided.

The online parameter tuning can be simple as in [36] where a tabu search procedure is having a *cycle detection mechanism* and increases the tabu tenure along the search when cycles are detected. To really improve the behavior of a metaheuristic algorithm, however, more elaborate methods can be used during the search and exploited for a better efficiency.

One such mechanism includes Boutillon et al. [7] which can be activated during the search like *retroactive loops* where a simulated annealing temperature parameter is controlled during the search to follow a predefined probability acceptance decreasing scheme.

Reactive Search

Among all existing methods, the work of Battiti [2] had traced the path a long time ago. In the most simple reactive search, the past history of the search is intensely used for *feedback-based parameter tuning* and for *automated balance of diversification and intensification*. In the former, the tuning of parameters is automated, and decisions on the new values of parameters are made based on the past events of the search. In the latter, the concept of balancing exploration *vs.* exploitation is used to guide the search.

One of the simplest forms of reactive search is reactive tabu search. The main idea is to change the length of the tabu list (i.e., the tabu tenure) based on the search trajectory. Essentially, the tabu list is made longer if the search is not finding better solutions, and shorter if it does.

Greedy Randomized Adaptive Search Procedure

GRASP (greedy randomized adaptive search procedure) is a constructive metaheuristic, the main idea of which is to balance greediness and randomness. Many constructive heuristics are greedy, which means that, at every iteration, they pick the best element from the set of potential solution elements. An example is the nearest-neighbor heuristic for the TSP which starts from a given city and moves to the closest unvisited city at every iteration. The drawback of a fully greedy

heuristic is that it only generates a single solution, which is most likely suboptimal. A wrong decision early on in the constructive procedure may lead to bad solutions in the end.

GRASP attempts to overcome this drawback by introducing randomness into the solution construction process. Instead of picking the best element at each iteration, GRASP creates a *restricted candidate list*, i.e., a list of the α best elements and picks one element from this list *at random* (α represents a number of elements). By doing this, GRASP generates a different solution at each iteration. After several iterations, some solutions will likely have been found that are better than the one found by a purely greedy heuristic.

In *reactive* GRASP, introduced by Prais and Ribeiro [30], the parameter α is randomly chosen from a set of discrete value. Initially, each possible α_i has the same probability of being chosen. The search remembers the quality of the solutions found for each possible value of α_i. After some iterations, the probabilities of selecting α_i are updated to reflect the quality of the solutions it produced. The probabilities corresponding to α_i's that have resulted in good solutions are increased, the others are decreased.

The reactive GRASP by Delorme et al. [13], e.g., works as follows. A *value* λ_i is defined for each α_i. The probability of selecting α_i, p_i is calculated as follows:

$$p_i = \frac{\lambda_i}{\sum_{k=1}^{n} \lambda_k},$$

supposing that n different α_i's have been defined.

Whenever a good solution is found using a certain α_i, this solution is added to the *pool* P_i for this α_i. Periodically, the values of λ_i (and hence p_i) are updated according to the following formula:

$$\lambda_i = \left(\frac{\text{mean}_{x \in P_i}[f(x) - f(\underline{x})]}{f(\overline{x}) - f(\underline{x})} \right)^{\delta},$$

where $\underline{x}$ and $\overline{x}$ are the worst and best solutions found so far and δ is a parameter introduced to attenuate the update of the probabilities p_i.

Adaptive Large Neighborhood Search

Large neighborhood search (LNS) is a constructive metaheuristic that works by building solutions from their constituting elements. For this purpose, it relies on a set of simple constructive procedures to build solutions, and on a set of destructive procedures to partially destroy these solutions so they can be rebuilt. For this reason, LNS has also been called ruin-and-recreate. At each iteration, LNS selects a pair consisting of one destructive heuristic and one constructive heuristic. Using this pair, a new solution is obtained. Most LNS implementations use a probabilistic

mechanism to select both the destructive and the constructive heuristic at each iteration. A (nonadaptive) LNS algorithm could, e.g., assign equal probabilities to each constructive heuristic and to each destructive heuristic.

Adaptive large neighborhood search goes a step further by selecting the heuristic pair with a probability determined by the previous performance of both the constructive and the destructive heuristic. As a result, constructive and destructive heuristics that perform well will have a higher probability of being selected, whereas those that will not have a lower probability. Usually, however, the probabilities are bounded by some values that ensure all heuristics have at least a tiny chance of being selected.

An example of an ALNS adaptive constructive/destructive heuristic selection mechanism is the following. Suppose an ALNS heuristic has n constructive and m destructive heuristics. Initially, each constructive heuristic i (and each destructive heuristic j) is assigned a *value* $\lambda_i^c = 1$ ($\lambda_j^d = 1$). Then, a constructive heuristic is selected from the set of constructive heuristics with a probability p proportional to its value:

$$p_i = \frac{\lambda_i}{\sum_{k=1}^{n} \lambda_k}$$

Similarly, a destructive heuristic is selected according to an equivalent rule.

Using the pair of destructive and constructive heuristic, a new solution is generated. The quality of the new solution is evaluated, and the values of the selected constructive and destructive heuristic are updated.

$$\alpha = \begin{cases} 0.5 & \text{if the new solution is worse than the current solution} \\ 1.5 & \text{if the new solution is better than the current solution} \\ 2 & \text{if the new solution is better than the global best solution} \end{cases}$$

$$\lambda_{i,\text{new}}^c = \gamma \lambda_i^c + (1 - \gamma)\alpha$$

$$\lambda_{j,\text{new}}^d = \gamma \lambda_j^d + (1 - \gamma)\alpha$$

where γ is a parameter between 0 and 1.

Using the formulas above, the probabilities of selection for each constructive and destructive heuristic will adapt to the problem at hand. Moreover, using the formulas above, the values can never increase above 2 and never drop below 0.5. In other words, all constructive and destructive heuristics will keep having a positive probability of being selected, even if they consistently fail to find improving solutions.

A recent meta-analysis by Turkeš et al. [40] critically examines the effectiveness of the adaptive layer in adaptive large neighborhood search (ALNS) algorithms. By systematically reviewing existing studies, the authors assess whether adaptiveness—

dynamically adjusting heuristic selection probabilities based on performance—significantly enhances ALNS performance. Their findings indicate that, on average, the adaptive layer contributes to only a marginal improvement of approximately 0.14% in solution quality. This suggests that the complexity introduced by adaptiveness may not yield substantial benefits, prompting a reevaluation of its necessity in ALNS implementations.

While there are many more adaptive mechanisms than the one described here, the study by Turkeš et al. [40] demonstrates the importance of evaluating whether making a metaheuristic's adaptiveness genuinely improves its performance. One of the study's main conclusions is that adaptivity may be advantageous in scenarios where the heuristic needs to address a wide variety of fundamentally different problems or problem instances with substantial internal diversity. In such cases, incorporating an adaptive mechanism could offer tangible benefits.

Multi-Level Metaheuristics and Hyper-Heuristics

In the previous section, we have examined different techniques for adapting the configuration of metaheuristics. In this section we look at multi-level metaheuristics, i.e., metaheuristic algorithms for evolving the configuration of a metaheuristic. Much of the research in this area has used evolutionary algorithms to configure metaheuristics with one of the earliest studies being that conducted by Bölte and Thonemann [6] which uses genetic programming for generating annealing schedules, which were previously manually created, in a simulated annealing algorithm to solve the quadratic assignment problem. Various evolutionary algorithms, namely, genetic algorithms, evolutionary strategies, and estimation of distribution algorithms, have been used for parameter tuning of evolutionary algorithms [18]. These are referred to as meta-EAs and operate on the design level, while the EA solving the problem at hand is considered to form the algorithm layer. Hyper-heuristics are proving to be effective for the automatic configuration of metaheuristics, and we provide an overview of the use of hyper-heuristics for this purpose.

Hyper-Heuristics

Hyper-heuristics were initially introduced as "heuristics to choose heuristics" Burke et al. [8], Ross [33]. Hyper-heuristics aim at providing a more generalized solution for a problem domain rather than producing the best solution for certain problem instances. This is achieved by exploring a space of low-level heuristics rather than a solution space directly. The low-level heuristics can be constructive or perturbative. Constructive low-level heuristics are used to create an initial solution, while perturbative heuristics are used to improve an existing candidate solution. The first generation of hyper-heuristics was essentially selection hyper-heuristics which chose which constructive or perturbative heuristics to use at a particular point in constructing or improving a candidate solution respectively.

Selection constructive hyper-heuristics are used to select the low-level construction heuristic at each stage in constructing a solution. Similarly, in the case of selection perturbative hyper-heuristics, the hyper-heuristic chooses a perturbative low-level heuristic at each stage of the improvement process. We use an application of hyper-heuristics to the domain of examination timetabling to illustrate these concepts. Low-level construction heuristics generally used to solve examination timetabling problems are the graph coloring heuristics, namely, largest degree, largest weighted degree, largest color degree, largest enrolment, and saturation degree [32]. Each of these heuristics assesses the difficulty associated with allocating an examination to the timetable. For example, the saturation degree heuristic is the number of timetable slots; given the current state of the timetable at the particular point in construction, an examination can be allocated to without causing hard constraint violations such as a student being scheduled to write more than one examination at the same time. A selection hyper-heuristic chooses which of the low-level heuristics to use to schedule each of the examinations. This has proven to be effective as different low-level heuristics work well for different problem instances and more importantly different low-level heuristics are more effective at different points of solution construction. Metaheuristics such as simulated annealing, tabu search, variable neighborhood search, and genetic algorithms have generally been used to search the heuristic space [8, 11].

Examples of low-level perturbative heuristics for the examination timetabling domain include swapping examinations, swapping rows of the timetable, de-allocating examinations, and allocating examinations. Selection perturbative hyper-heuristics can perform a single-point search or a multipoint search. In the case of the former, the hyper-heuristic is comprised of a heuristic selection and move acceptance component [11]. Different techniques are employed to for heuristic selection and move acceptance. This technique can be as simple as randomly selecting a low-level heuristic and accepting on moves that results in improvement. Metaheuristics can also be employed for heuristic selection and move acceptance, e.g., simulated annealing and tabu search have previously been employed for this purpose. Multipoint search selection perturbation hyper-heuristics employ population-based methods such as evolutionary algorithms and particle swarm optimization to explore the heuristic space, with the population-based approach, by its nature, performing both the heuristic selection and move acceptance.

As the field developed, the idea of creating new low-level heuristics emerged resulting in a second category of hyper-heuristics, namely, generation hyper-heuristics. Generation hyper-heuristics generate low-level constructive or perturbative heuristics. Genetic programming has primarily been employed by generation hyper-heuristics to create low-level heuristics [10, 11]. An example of a generation hyper-heuristic is that implemented by Burke et al. [9] for the one-dimensional bin-packing problem. Construction low-level heuristics, e.g., first-fit, best-first, and next-fit, are used to choose which bin to allocate an item to. In this study, genetic programming is used to evolve low-level heuristics to decide which bin to allocate an item to. In the study conducted by Sabar et al. [34], grammatical evolution, a variation of genetic programming, is used to evolve new low-level perturbative

heuristics by combining mechanisms for heuristic selection and move acceptance. Generated low-level heuristics can be reusable or disposable. In the case of reusable heuristics, the generated low-level heuristic created to solve the problem for one instance can be used to solve other problem instances without any regeneration. Disposable low-level heuristics are generated for the specific problem instance and cannot be reused.

Selection and generation hyper-heuristics have generally been used for the automatic configuration of metaheuristic algorithms. In this case, the low-level heuristics represent parameters or operators of the metaheuristic and are essentially perturbative. The hyper-heuristic employs a metaheuristic to explore the space of low-level heuristics. Selection hyper-heuristics are used to determine control flow and for parameter tuning. In this case the hyper-heuristic selects a component at each point in the application of the algorithm to solve the problem. Furthermore, generation hyper-heuristics are used to create new low-level heuristics; in this context these represent the components of the metaheuristic.

Hyper-Heuristics for Metaheuristic Configuration

Control flow is achieved by producing a combination of low-level heuristics, each a component of the metaheuristic algorithms, i.e., the hyper-heuristic selects which low-level heuristic to apply at each point in a metaheuristic algorithm. The low-level heuristics are components of the metaheuristic algorithm. Lourenço et al. [25] use grammatical evolution to evolve evolutionary algorithms for the Royal Road Functions. The aim is for the evolutionary algorithm to adapt itself during the evolutionary process. Evaluating the evolutionary algorithm proved to be a computationally intensive task, and hence a limited number of runs were performed. Grammatical evolution combines the different evolutionary algorithm components, namely, mutation, crossover, and selection components and parameter values for these components. The evolved evolutionary algorithms are applied to a seeded initial population. One instance was used for training, and the evolved evolutionary algorithms were tested on the remaining four instances. Performance was found to be similar to that of the standard evolutionary algorithm. The evolved EA that performed better than the standard EA did not follow the standard structure and contained two types of crossover. There are a number of early initiatives that can be categorized as hyper-heuristics for inducing evolutionary algorithms, although this is not explicitly stated in the papers. Oltean and Groşan [28] use multigene expression programming to evolve an evolutionary algorithm to induce Griewang's function. Algorithms comprised of initialization, mutation, and crossover components are evolved. The number of crossover, initialization, and mutation components in the best individual was found to increase as the evolution progressed. Linear genetic programming has also been used for purposes of evolutionary algorithm induction [27]. Each algorithm evolved is a generational evolutionary algorithm composed of selection, crossover, and mutation components and is applied to an initial population of randomly generated elements. Evolutionary

algorithms are evolved for function optimization, the traveling salesman problem, and quadratic assignment problem. For all three problems, the evolved algorithms are trained on a problem instance and are able to generalize and solve other problem instances. In later work, Dioşan and Oltean [15] use genetic algorithms to evolve evolutionary algorithms for function optimization. Each chromosome is comprised of a combination of selection, mutation, and crossover operations as well as population altering strategies to place the newly created offspring into the population. A different evolutionary algorithm was evolved to induce each of the ten functions. As in previous studies, the crossover operator was the most prevalent in evolved evolutionary algorithms producing the best results.

In some studies the hyper-heuristic achieves both control flow and parameter tuning. In this case the hyper-heuristic selects the component of the metaheuristic and the parameter value. Tavares and Pereira [39] employ grammatical evolution to automatically configure ant colonization algorithms for solving the traveling salesman problem. The architecture of the ant colonization algorithm including the components of the algorithm, e.g., method for evaporation, and parameter values are evolved. The evolved architecture was found to be effective when applied to problem instances different from those used for training during evolution. The evolved architectures producing the best results were found to be different from those of the standard ant colonization algorithms. Lourenço et al. [26] use grammatical evolution to design evolutionary algorithms to solve the knapsack problem. The evolved evolutionary algorithms are applied to unseen instances. Each evolved evolutionary algorithm specifies the type of selection, type of crossover, type of mutation, and parameter values. Evolved evolutionary algorithms using binary swap mutation and uniform crossover performed the best.

Generation hyper-heuristics go a step further, and instead of choosing a component of a metaheuristic to decide the control flow, these hyper-heuristics create a new component. In the study conducted by Hong et al. [21], a generative hyper-heuristic, employing genetic programming, is used to evolve mutation operators for evolutionary programming. The terminal set is comprised of random numbers produced by a Gaussian random number generator, and the function set is comprised of arithmetic operators. The approach was used to evolve ten function classes, seven unimodal, and three multimodal, and was found to require less processor time than the man hours needed to design new mutation operators. A different mutation operator was evolved for each function class using one instance of the class. Woodward and Swan [44] use local search, namely, hill-climbing, to evolve mutation operators for genetic algorithms. These mutation operators were found to outperform human-created mutation operators. Register machines are used to simulate the behavior of mutation operators. Seven function classes were used to test the effectiveness of the evolved mutation operators. Different mutation operators were created for each function class. Woodward and Swan [43] evaluate a similar approach which uses random search to generate search heuristics for a genetic algorithm. As in the previous study, register machines are used to emulate the selection process. Selection is based on the fitness or rank of a bit string. The evolved selection heuristics were tested for the mimicry problem set and

were found to perform better than the human-designed selection heuristics. In the study conducted by Dioşan and Oltean [14], genetic programming induces crossover operators for a genetic algorithm for function optimization. A different crossover operator is evolved for each of the 11 function classes. The performance of the evolved operators was found to be comparative to human-designed crossover operators. Drake et al. [17] also employ grammatical evolution to design the construction heuristic and move operators used by variable neighborhood search to solve the vehicle routing problem. The move operators generated are ruin and insertion heuristics which are used to perform the shaking process in the variable neighborhood search. The variable neighborhood search produced results close to the global optimum for all problem instances. Løkketangen and Olsson [24] use the ADATE system to generate the move selection, tabu tenure, and the aspiration criteria in a tabu search for solving the Boolean optimization problem (BOOP). The authors describe ADATE as a generation hyper-heuristic that performs offline learning. ADATE is an automatic programming system that produces functional programs in metalanguage ML. The generated components were found to perform better than manually designed components. One of the generated components producing good results was found to give good moves a longer tabu tenure which is not typical of human-designed tabu searches. This again emphasizes the advantage of automatic generation of metaheuristic components.

Discussion

Multi-level metaheuristics can be categorized as selection or generation perturbative hyper-heuristics. As previously outlined designing a metaheuristic involves making decisions regarding what parameter values to use, what operators to use, and the control flow of the overall algorithm. From the survey presented above, it is evident that hyper-heuristics have been fairly effective in making these design decisions. Selection perturbative hyper-heuristics are used to select numerical and discrete parameter values. Hence, the hyper-heuristic explores the space of parameter values. Evolutionary algorithms have chiefly been used for this purpose. The parameter values can be either coevolved while solving the problem at hand or optimized separately. In the latter case, the parameters can be determined offline during the training phase. Selection perturbative hyper-heuristics can also be used to make control flow decisions. This essentially involves selecting different operators at different stages of solving the problem. Hence, the decision of which operator to use can be made as part of the control flow decision. Evolutionary algorithms, including linear genetic programming, genetic algorithms, multigene expression programming, and grammatical evolution, have been used for control flow design. The decision regarding which operator to use may not be a matter of selecting an existing operator but creating a new operator. Generation hyper-heuristics can be used to generate new operators. This research has focused to a large extent on selection, mutation, and crossover operators in evolutionary algorithms. Genetic programming has primarily been used to generate new operators. Grammatical

evolution, local search, and random search have also been used for this purpose. A hyper-heuristic can be used to make all three design decisions simultaneously. This can be seen in the studies conducted by Lourenço et al. [25, 26] and Tavares and Pereira [39] where grammatical evolution is used to make this decision for the induction of an evolutionary and an ant colonization algorithm simultaneously.

In the case of all three design decisions, the parameter values, operators, and algorithms induced by the hyper-heuristic can be reusable or disposable. In the case of reusability, two phases are performed, a training phase and a testing phase. During the training phase, one or a subset of problem instances are used. The induced parameter values, operators, and algorithms are then used to solve unseen instances. Disposable parameter values, operators, and algorithms are induced for the particular problem instance, and hence a training phase is not required. Reusability has the advantage of the time required for design being reduced as redesign is not needed for every new instance; however, there may be a limited number of instances for which the generated design is applicable.

One of the challenges associated with using hyper-heuristics for the design of metaheuristics is the processing time needed. Given the advances made in multicore architectures and the availability of multicore architectures on a standard desktop machine, distributed architectures can be designed for the implementation of these hyper-heuristics. Most of the research conducted this far has focused on the configuration of evolutionary algorithms. These ideas can be transferred to the design of other metaheuristics. Evolutionary algorithms have primarily been used for design purposes. These have ranged from genetic algorithms through to grammatical evolution. A comparative study into the performance of the different types of evolutionary algorithms and their contribution to the design process would be interesting.

In most research using hyper-heuristics for parameter configuration in metaheuristics, this is done offline. However, more recently the effectiveness of online parameter configuration of metaheuristics has been established. For example, in the study conducted in [29], the effectiveness of combining online and offline parameter configuration of metaheuristics is shown. Thus, a current recent trend in the field is using hyper-heuristics for online parameter configuration of metaheuristics. This also helps alleviate the computational cost of offline parameter configuration in some instances.

Conclusion

A metaheuristic algorithm's configuration is the combination of its control flow and its parameter settings. Determining the best possible configuration for a metaheuristic is a difficult task, which is commonly done by trial and error and based on the experience of the algorithm designer. For this reason, metaheuristics have been developed that are able to adapt their configuration during the search (adaptive metaheuristics), potentially using a higher-level metaheuristic (multi-level metaheuristics). In this chapter, we have surveyed the literature on this topic. The

chapter has also highlighted the effectiveness of hyper-heuristics as multi-level metaheuristics. This serves as a starting point for researchers wanting to use hyper-heuristics for the automated design of metaheuristics. Hyper-heuristics have been fairly effective for the purpose of design, and in most cases the generated designs have produced better results than the manually designed metaheuristic, in some experiments producing designs that have not previously been thought of. This overview has also brought to light certain research questions and hence areas for future research. Research thus far has highlighted the potential of hyper-heuristics in the automated configuration of metaheuristics. This has now set the foundation for wider application, including more complex problems.

Perspectives

Recent advances in artificial intelligence (AI) have transformed how metaheuristics can be designed, tuned, and adapted. This chapter would not offer enough space to describe this new trend, but instead, we will provide a few insights in this section.

AI techniques, particularly machine learning (ML), offers the potential to automate and enhance the configuration of metaheuristics, minimizing the need for human intervention and improving performance across diverse problem instances.

AI-Driven Parameter Tuning One of the critical challenges in metaheuristics is determining optimal parameter settings. AI addresses this challenge through techniques like reinforcement learning (RL) and Bayesian optimization [4], which model the configuration task as a learning problem. RL-based methods, for example, can adapt parameter values dynamically during the search process, optimizing the balance between exploration and exploitation based on feedback from the solution space.

Meta-Learning and Transferability Inspired by the meta-learning paradigm proposed by Finn et al. [20], AI systems can generalize configuration strategies across problem domains. By analyzing a repository of solved instances, AI models can learn patterns in parameter effectiveness, enabling them to suggest configurations for unseen problems. This transferability reduces the computational overhead of parameter tuning and improves scalability.

Evolving Control Flow with AI Control flow, or the sequence in which metaheuristic components are executed, significantly impacts algorithm performance. AI-based approaches, such as neural architecture search (NAS) and genetic programming [47], allow for the automatic design of metaheuristic control flows. These techniques evolve control flow structures to maximize performance metrics, uncovering novel algorithmic combinations that human designers may overlook.

Adaptive Metaheuristics Through AI AI enhances adaptivity in metaheuristics by integrating predictive models that anticipate performance shifts and adjust configurations accordingly. For instance, deep neural networks can predict convergence

trends and suggest changes to the algorithm's strategy or parameters [35]. This predictive capability ensures that the metaheuristic remains effective even as the problem landscape evolves.

AI-Enabled Multi-Level Metaheuristics The recursive nature of multi-level metaheuristics aligns well with AI's hierarchical learning capabilities. AI can operate at multiple levels of the metaheuristic design process, optimizing both high-level strategies and low-level components [3]. For example, at the higher level, AI might determine whether to use an evolutionary algorithm or a simulated annealing approach, while at the lower level, it could fine-tune specific operators or hyperparameters.

Despite its promise, the integration of AI in metaheuristic configuration faces several challenges. Computational cost is a significant barrier, particularly for methods requiring extensive training data or repeated simulations [23]. Interpretability is another concern; AI-designed configurations may outperform traditional methods but lack the transparency needed for scientific understanding.

Future research should focus on hybrid approaches that combine the strengths of traditional metaheuristics with AI-driven enhancements. Developing lightweight, interpretable AI models tailored for metaheuristics could democratize access to these techniques, enabling broader application in real-world optimization problems.

Cross-References

- ▸ Hyper-heuristics
- ▸ Restart Strategies
- ▸ Time Complexity Analysis of Stochastic Search Algorithms

Competing Interest Declaration The author(s) has no competing interests to declare that are relevant to the content of this manuscript.

References

1. Adenso-Díaz B, Laguna M (2006) Fine-tuning of algorithms using fractional experimental designs and local search. Oper Res 54(1):99–114. https://doi.org/10.1287/opre.1050.0243
2. Battiti R (1996) Reactive search: toward self-tuning heuristics. In: Modern heuristic search methods. Wiley, Chichester, pp 61–83
3. Bengio Y, Ducharme R, Vincent P, Jauvin C (2003) A neural probabilistic language model. J Mach Learn Res 3:1137–1155
4. Bengio Y, Lodi A, Prouvost A (2021) Machine learning for combinatorial optimization: a methodological tour d'horizon. Eur J Oper Res 290(2):405–421. https://doi.org/10.1016/j.ejor.2020.07.063
5. Birattari M (2009) Tuning metaheuristics. Springer, Berlin/Heidelberg. https://doi.org/10.1007/978-3-642-00483-4

6. Bölte A, Thonemann UW (1996) Optimizing simulated annealing schedules with genetic programming. Eur J Oper Res 92(2):402–416. https://doi.org/10.1016/0377-2217(94)00350-5
7. Boutillon E, Roland C, Sevaux M (2008) Probability-driven simulated annealing for optimizing digital FIR filters. In: Studies in computational intelligence. Springer Science & Business Media, pp 77–93. https://doi.org/10.1007/978-3-540-79438-7_4
8. Burke EK, Kendall G, Newall J, Hart E, Ross P, Schulenburg S (2003) Hyper-heuristics: an emerging direction in modern search technology. In: Glover F, Kochenberger GA (eds) Handbook of metaheuristics. International series in operations research & management science, vol 57. Springer, pp 457–474. https://doi.org/10.1007/0-306-48056-5_16
9. Burke EK, Hyde MR, Kendall G, Woodward J (2007) Automatic heuristic generation with genetic programming. In: Proceedings of the 9th annual conference on Genetic and evolutionary computation—GECCO'07. Association for Computing Machinery (ACM). https://doi.org/10.1145/1276958.1277273
10. Burke EK, Hyde MR, Kendall G, Ochoa G, Ozcan E, Woodward JR (2009) Exploring hyper-heuristic methodologies with genetic programming. In: Intelligent systems reference library. Springer Science & Business Media, pp 177–201. https://doi.org/10.1007/978-3-642-01799-5_6
11. Burke EK, Gendreau M, Hyde MR, Kendall G, Ochoa G, Özcan E, Qu R (2013) Hyper-heuristics: a survey of the state of the art. J Oper Res Soc 64(12):1695–1724. https://doi.org/10.1057/jors.2013.71
12. Dean A, Voss D (eds) (1999) Design and analysis of experiments. Springer, Berlin. https://doi.org/10.1007/b97673
13. Delorme X, Gandibleux X, Rodriguez J (2004) GRASP for set packing problems. Eur J Oper Res 153(3):564–580. https://doi.org/10.1016/s0377-2217(03)00263-7
14. Dioşan L, Oltean M (2006) Evolving crossover operators for function optimization. In: Genetic programming. Springer Science & Business Media, pp 97–108. https://doi.org/10.1007/11729976_9
15. Dioşan L, Oltean M (2009) Evolutionary design of evolutionary algorithms. Gen Program Evol Mach 10(3):263–306. https://doi.org/10.1007/s10710-009-9081-6
16. Dobslaw F (2010) A parameter tuning framework for metaheuristics based on design of experiments and artificial neural networks. In: Proceedings of the international conference on computer mathematics and natural computing 2010. WASET
17. Drake JH, Kililis N, Ozcan E (2013) Generation of VNS components with grammatical evolution for vehicle routing. In: Genetic programming. Springer Science & Business Media, pp 25–36. https://doi.org/10.1007/978-3-642-37207-0_3
18. Eiben AE, Smit SK (2011) Evolutionary algorithm parameters and methods to tune them. In: Autonomous search. Springer, Berlin/Heidelberg, pp 15–36. https://doi.org/10.1007/978-3-642-21434-9_2
19. Eiben AE, Hinterding R, Michalewicz Z (1999) Parameter control in evolutionary algorithms. IEEE Trans Evol Comput 3(2):124–141. https://doi.org/10.1109/4235.771166
20. Finn C, Abbeel P, Levine S (2017) Model-agnostic meta-learning for fast adaptation of deep networks. arXiv preprint arXiv:1703.03400
21. Hong L, Woodward J, Li J, Ozcan E (2013) Automated design of probability distributions as mutation operators for evolutionary programming using genetic programming. In: Proceedings of the 16th European conference on genetic programming—EuroGP 2013, vol 7831, pp 85–96
22. Hooker JN (1995) Testing heuristics: we have it all wrong. J Heurist 1(1):33–42. https://doi.org/10.1007/bf02430364
23. Lipton ZC (2016) The mythos of model interpretability. arXiv preprint arXiv:1606.03490
24. Løkketangen A, Olsson R (2009) Generating meta-heuristic optimization code using ADATE. J Heurist 16(6):911–930. https://doi.org/10.1007/s10732-009-9119-1
25. Lourenço N, Pereira FB, Costa E (2012) Evolving evolutionary algorithms. In: Proceedings of the fourteenth international conference on Genetic and evolutionary computation conference companion—GECCO 2012. ACM Press. https://doi.org/10.1145/2330784.2330794

26. Lourenço N, Pereira FB, Costa E (2013) The importance of the learning conditions in hyper-heuristics. In: Proceedings of the fifteenth annual conference on Genetic and evolutionary computation conference—GECCO 2013. ACM Press. https://doi.org/10.1145/2463372.2463558
27. Oltean M (2005) Evolving evolutionary algorithms using linear genetic programming. Evol Comput 13(3):387–410. https://doi.org/10.1162/1063656054794815
28. Oltean M, Groşan C (2003) Evolving evolutionary algorithms using multi expression programming. In: Advances in artificial life. Springer Science & Business Media, pp 651–658. https://doi.org/10.1007/978-3-540-39432-7_70
29. Pei J, Liu J, Mei Y (2024) Learning from offline and online experiences: a hybrid adaptive operator selection framework. In: GECCO '24. Association for Computing Machinery, New York, NY, USA, pp 1017–1025. ISBN 9798400704949. https://doi.org/10.1145/3638529.3654062
30. Prais M, Ribeiro CC (2000) Reactive GRASP: an application to a matrix decomposition problem in TDMA traffic assignment. INFORMS J Comput 12(3):164–176. https://doi.org/10.1287/ijoc.12.3.164.12639
31. Prins C (2004) A simple and effective evolutionary algorithm for the vehicle routing problem. Comput Oper Res 31(12):1985–2002. https://doi.org/10.1016/s0305-0548(03)00158-8
32. Qu R, Burke EK, McCollum B, Merlot LTG, Lee SY (2008) A survey of search methodologies and automated system development for examination timetabling. J Sched 12(1):55–89. https://doi.org/10.1007/s10951-008-0077-5
33. Ross P (2005) Hyper-heuristics. In: Search methodologies. Springer Science & Business Media, pp 529–556. https://doi.org/10.1007/0-387-28356-0_17
34. Sabar NR, Ayob M, Kendall G, Qu R (2013) Grammatical evolution hyper-heuristic for combinatorial optimization problems. IEEE Trans Evol Comput 17(6):840–861. https://doi.org/10.1109/tevc.2013.2281527
35. Sankarananth S, Karthiga M, Suganya E, Sountharrajan S, Bavirisetti DP (2023) Ai-enabled metaheuristic optimization for predictive management of renewable energy production in smart grids. Energy Rep 10:1299–1312. ISSN 2352-4847. https://doi.org/10.1016/j.egyr.2023.08.005
36. Sevaux M, Thomin P (2001) Heuristics and metaheuristics for parallel machine scheduling: a computational evaluation. In: Proceedings of 4th metaheuristics international conference, MIC 2001, Porto, pp 411–415
37. Sörensen K, Sevaux M (2006) MA|PM: memetic algorithms with population management. Comput Oper Res 33(5):1214–1225. https://doi.org/10.1016/j.cor.2004.09.011
38. Talbi E-G (2009) Metaheuristics: from design to implementation. Wileys, Hoboken. ISBN 978-0-470-27858-1
39. Tavares J, Pereira FB (2012) Automatic design of ant algorithms with grammatical evolution. In: Genet Program. Springer Science & Business Media, pp 206–217. https://doi.org/10.1007/978-3-642-29139-5_18
40. Turkeš R, Sörensen K, Hvattum LM (2021) Meta-analysis of metaheuristics: quantifying the effect of adaptiveness in adaptive large neighborhood search. Eur J Oper Res 292(2):423–442
41. Van Breedam A (1995) Improvement heuristics for the vehicle routing problem based on simulated annealing. Eur J Oper Res 86(3):480–490. https://doi.org/10.1016/0377-2217(94)00064-J
42. Wolpert DH, Macready WG (1997) No free lunch theorems for optimization. IEEE Trans Evol Computa 1(1):67–82. https://doi.org/10.1109/4235.585893
43. Woodward JR, Swan J (2011) Automatically designing selection heuristics. In: Proceedings of the 13th annual conference companion on Genetic and evolutionary computation—GECCO 2011. ACM Press. https://doi.org/10.1145/2001858.2002052
44. Woodward JR, Swan J (2012) The automatic generation of mutation operators for genetic algorithms. In: Proceedings of the fourteenth international conference on Genetic and evolutionary computation conference companion—GECCO 2012. ACM Press. https://doi.org/10.1145/2330784.2330796

45. Xu J, Kelly JP (1996) A network flow-based tabu search heuristic for the vehicle routing problem. Transp Sci 30(4):379–393. https://doi.org/10.1287/trsc.30.4.379
46. Xu J, Chiu SY, Glover F (1998) Fine-tuning a tabu search algorithm with statistical tests. Int Trans Oper Res 5(3):233–244. https://doi.org/10.1111/j.1475-3995.1998.tb00117.x
47. Zoph B, Le QV (2017) Neural architecture search with reinforcement learning. https://arxiv.org/abs/1611.01578

Biased Random-Key Genetic Programming

2

José Fernando Gonçalves and Mauricio G. C. Resende

Contents

Introduction . . . 26
Program Representation . . . 27
Biased Random-Key Genetic Programming . . . 28
  Overview . . . 29
  Biased Random-Key Genetic Algorithms . . . 29
  Encoding and Chromosome Structure . . . 31
  Decoding a Chromosome into a *PE* . . . 32
  Fitness Function . . . 35
Examples . . . 36
  SR-Example 1 . . . 36
  SR-Example 2 . . . 37
Conclusions . . . 38
Cross-References . . . 38
References . . . 39

Abstract

This chapter introduces biased random-key genetic programming, a new metaheuristic for evolving programs. Each solution program is encoded as a vector of random keys, where a random key is a real number randomly generated in

J. F. Gonçalves (✉)
Faculdade de Economia da, Universidade do Porto, Porto, Portugal
e-mail: jfgoncal@fep.up.pt

M. G. C. Resende
Industrial and Systems Engineering, University of Washington, Seattle, WA, USA
e-mail: resendem@amazon.com

© Springer Nature Switzerland AG 2025

R. Martí et al. (eds.), *Handbook of Heuristics*,
https://doi.org/10.1007/978-3-032-00385-0_25

the continuous interval [0, 1]. A decoder maps each vector of random keys to a solution program and assigns it a measure of quality. A Program-Expression is encoded in the chromosome using a head-tail representation which is later transformed into a syntax tree using a prefix notation rule. The artificial simulated evolution of the programs is accomplished with a biased random-key genetic algorithm. Examples of the application of this approach to symbolic regression are presented.

Keywords

Genetic programming · Biased random-key genetic algorithms · Head-tail representation · Prefix notation

Introduction

Genetic programming (GP) is an evolutionary metaheuristic inspired by biological evolution to find computer programs that perform a predefined user computational task.

GP has its origins around 1954 with the evolutionary algorithms applied by Nils Aall Barricelli to evolutionary simulations [2]. One of the earliest practitioners of the GP methodology was Lawrence J. Fogel who, in 1964, applied evolutionary algorithms to the problem of discovering finite-state automata [9]. The first paper on tree-based genetic programming was presented in [5] and was later expanded by John R. Koza, who pioneered the application of genetic programming in various complex optimization and search problems [20–23].

Traditionally, GP has favored the use of programming languages that naturally embody tree structures [1], but several new non-tree representations were suggested and successfully implemented, such as linear genetic programming (LGP) [4], gene expression programming (GEP) [6], and Parallel Distributed Graphical Programming (PDGP) [24].

Genetic programming is still a young field of research. It attracts a growing research community, and there are many avenues of research yet to be explored. In this chapter, we introduce biased random-key genetic programming (*BRKGP*), a novel metaheuristic for computer program evolution.

The remainder of the chapter is organized as follows. In section "Program Representation" we describe the program representation. In section "Biased Random-Key Genetic Programming" we introduce the new approach, describing in detail the BRKGA, the chromosome structure and the decoding procedure, and the fitness function. Finally, in section "Examples" we illustrate the application of the approach to two cases in the area of symbolic regression, and in section "Conclusions" we make concluding remarks.

Program Representation

In genetic programming programs are usually expressed by syntax trees (ST). Figure 1 shows the tree representation of the program:

$$\max\left(2.2 - \frac{X}{11}, 7 \times \cos(Y)\right).$$

The variables and constants in the program (X, Y, *2.2, 11*, and *7*) are leaves of the tree. In GP the nodes that correspond to the leaves are called *terminals*, and the internal nodes represent the operations ($-$, $\div$, $\times$, cos, and max) and are called *functions*. The sets of allowed functions ($\mathcal{F}$) and terminals ($\mathcal{T}$) together form what is called the *primitive set* ($\mathcal{P}$) of a GP system.

In this chapter a linear representation of syntax trees is used. This indirect representation, called Program-Expression (*PE*), encodes a syntax tree as a sequence of elements of the primitive set which will be later translated, according to some predefined rules, into a syntax tree.

The set of rules used to translate a *PE* into a *ST* follows a prefix notation convention (*P-rule*). The *P-rule* translates a *PE* into a *ST* by reading sequentially, from left to right, each element in the *PE* and placing it in the bottom-most (first-criteria) and left-most (second-criteria) available node in the partial *ST* being constructed. Note that if there are no nodes available, in the partially built syntax tree, in which to place an element in the *PE,* then the process stops and the remaining

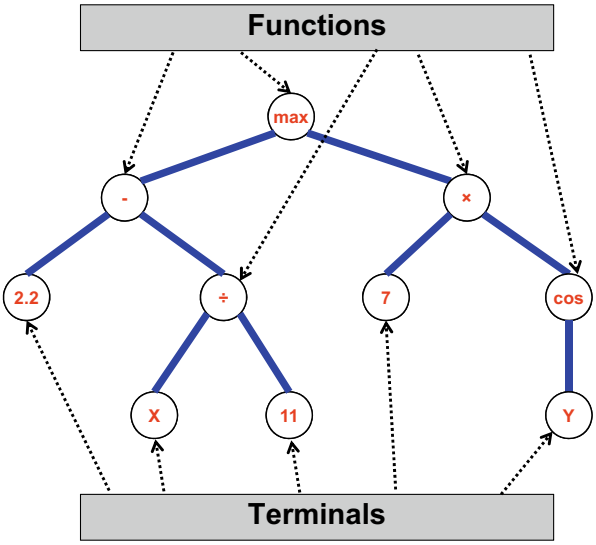


Fig. 1 Syntax tree of program $\max\left(2.2 - \frac{X}{11}, 7 \times \cos(Y)\right)$

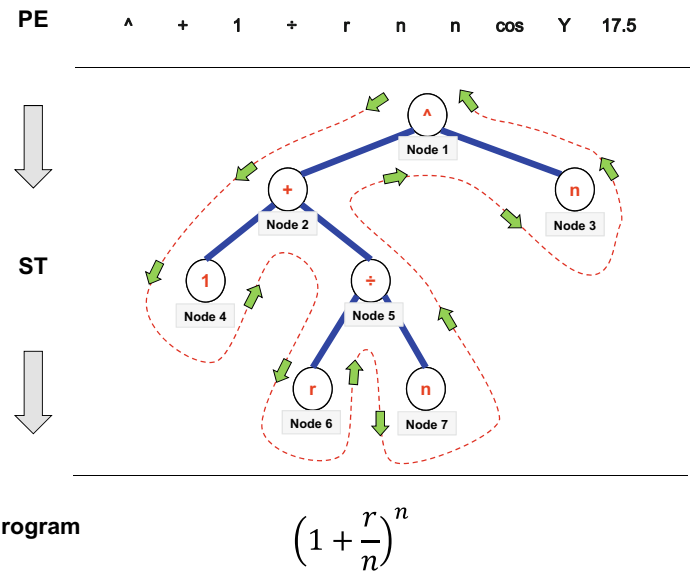


Fig. 2 Translation of a *PE* into a program using the *P-rule*

primitive elements in the *PE* are discarded and are called noncoding elements. Figure 2 demonstrates how the *PE*

$$\wedge + 1 \div r \;\; n \;\; n \;\; \cos \;\; Y \;\; 17.5$$

can be translated into a *ST* following the *P-rule*. The first primitive element in the *PE* is $|\wedge|$ and is placed in node 1 (the root of the tree). The second element in the *PE* is $|+|$ and is placed in node 2. The third primitive element in the *PE* is $|1|$ and is placed in node 4. The fourth primitive element in the *PE* is $|\div|$ and is placed in node 5. This process will be repeated until all elements in the *PE* are placed or there are no nodes available in the tree in which to place an element.

Note that the last three elements of the *PE,* $|\cos|$, $|Y|$, and $|17.5|$, are not included in the *ST*. The program that corresponds to the final *ST* is

$$\left(1+\frac{r}{n}\right)^n.$$

Biased Random-Key Genetic Programming

This section begins with an overview of the proposed biased random-key genetic programming *(BRKGP)* methodology. This is followed by a discussion of the biased random-key genetic programming algorithm, including detailed descriptions of the program encoding and decoding and the fitness measure.

Overview

The *BRKGP* metaheuristic presented in this chapter is based on four main components: a biased random-key genetic algorithm (*BRKGA*) [13], chromosomes, a decoding procedure to translate a chromosome into a syntax tree, and a fitness measure to assess the quality of the resulting programs.

The role of the *BRKGA* is to supply and evolve the chromosomes. A chromosome (genotype) indirectly represents a syntax tree (phenotype) that corresponds to a program. The decoding procedure receives as input a chromosome and in a first phase translates it into a Program-Expression (*PE*). Then, in a second phase the *PE*, obtained in the first phase, is translated into a syntax tree using the *P-rule* described in section "Program Representation."

Figure 3 illustrates the sequence of steps applied by the *BRKGP* to each chromosome.

The remainder of this section describes in detail the biased random-key genetic algorithm, the chromosome structure, the decoding procedure that maps a chromosome into a syntax tree, and the fitness measures.

Biased Random-Key Genetic Algorithms

Genetic algorithms with random keys, or *random-key genetic algorithms* (*RKGA*), were introduced by [3] for solving sequencing problems. In an *RKGA*, chromosomes

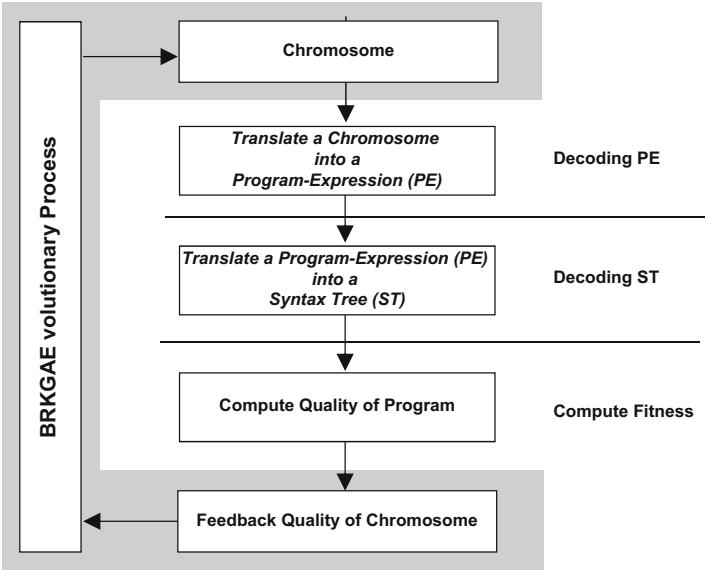


Fig. 3 Architecture of the *BRKGP*

are represented as vectors of randomly generated real numbers in the interval [0, 1]. A *decoder* is a deterministic algorithm that takes as input a chromosome and associates it with a solution of the combinatorial optimization problem for which an objective value or *fitness* can be computed.

An *RKGA* evolves a *population* of random-key vectors over a number of *generations* (iterations). The initial population is made up of p vectors each with r random keys. Each component of the solution vector, or random key, is randomly generated, independently, in the real interval [0, 1]. After the fitness of each individual is computed by the decoder in generation g, the population is partitioned into two groups of individuals: a small group of p_e *elite* individuals, i.e., those with the best fitness values, and the remaining set of $p - p_e$ *nonelite* individuals. To evolve the population of generation g, a new generation $(g + 1)$ of individuals is produced. All elite individuals of the population of generation g are copied without modification to the population of generation $g + 1$. *RKGAs* implement mutation by introducing *mutants* into the population. A mutant is a vector of random keys generated in the same way that an element of the initial population is generated. Its role is similar to that of mutation in other genetic algorithms [11], i.e., to introduce noise into the population and avoid convergence of the entire population to a local optimum. At each generation, a small number p_m of mutants is introduced into the population. With $p_e + p_m$ individuals accounted for in population $g + 1, p - p_e - p_m$ additional individuals need to be generated to complete the p individuals that make up population $g + 1$. This is done by producing $p - p_e - p_m$ offspring solutions through the process of *mating* or *crossover*.

A *biased random-key genetic algorithm* [13], *or simply BRKGA*, differs from an *RKGA* in the way parents are selected for mating. While in the *RKGA* of [3] both parents are selected at random from the entire current population, in a *BRKGA* each element is generated combining a parent selected at random from the elite partition in the current population and one from the rest of the population. Repetition in the selection of a mate is allowed, and therefore an individual can produce more than one offspring in the same generation. As in a *RKGA*, *parametrized uniform crossover* [25] is used to implement mating in a *BRKGA*. Let ρ_e be the probability that an offspring inherits the vector component of its elite parent. Recall that r denotes the number of components in the solution vector of an individual. For $i = 1, \ldots, r$, the i-th component $c[i]$ of the offspring vector c takes on the value of the i-th component $e[i]$ of the elite parent e with probability ρ_e and the value of the i-th component $\overline{e}\,[i\,]$ of the nonelite parent $\overline{e}$ with probability $1 - \rho_e$.

When the next population is complete, i.e., when it has p individuals, fitness values are computed for all of the newly created random-key vectors, and the population is partitioned into elite and nonelite individuals to start a new generation. Figure 4 depicts the transitional process between two consecutive generations.

A *BRKGA* searches the solution space of the combinatorial optimization problem indirectly by searching the continuous r-dimensional unit hypercube, using the

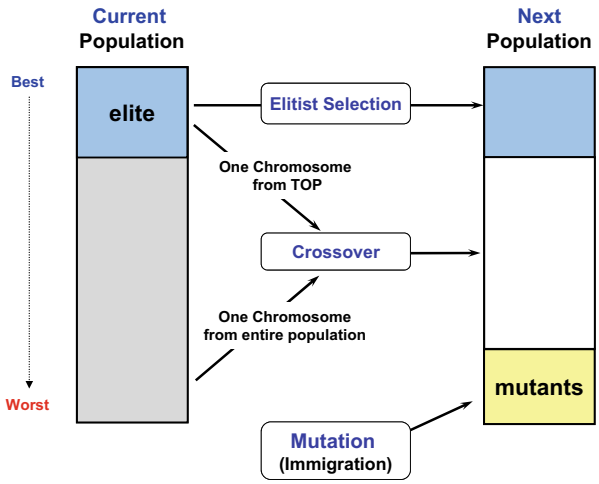


Fig. 4 Transitional process between consecutive generations

decoder to map solutions in the hypercube to solutions in the solution space of the combinatorial optimization problem where the fitness is evaluated.

*BRKGA*s have been applied with success to solve many types of combinatorial problems (see [10, 12, 14–19]).

A biased random-key genetic algorithm is specified by the parameters, the encoding and decoding of the solutions, and by the fitness measure. In the next section the algorithm is specified by first showing how programs are encoded and then decoded and how their fitness evaluation is computed.

Encoding and Chromosome Structure

A chromosome represents a program and is made of two parts. The first part encodes a *PE* and the second part encodes the values of the constants that can be used in the *PE*. To encode a *PE* a head-tail representation is proposed as in [6–8]. The head, h, represents the maximum number of internal nodes that can be in the syntax tree (*ST*) and can contain both functions (elements from set $\mathcal{F}$) and terminals (elements from the set $\mathcal{T}$), whereas the tail, t, represents the number of leaves of the *ST* and can only contain terminals (elements from the set $\mathcal{T}$). For each problem, the length of the head, h, is chosen, whereas the length of the tail, t, depends on h and on the arity, a, of the function with the most arguments and is determined by the expression

$$t = h\,(a - 1) + 1. \tag{1}$$

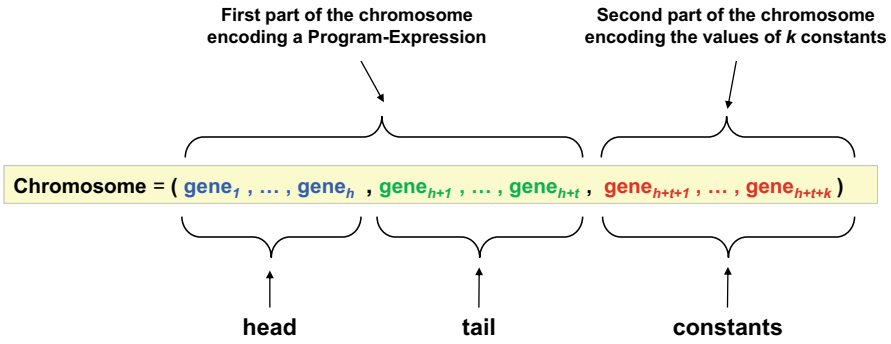


Fig. 5 Chromosome structure used to encode a *PE* with a vector of $h + t + k$ random keys

This way of determining the length of the tail guarantees that there will always be enough terminals to create syntactically valid *PE*s (assuming closure among the set of primitives).

Assuming that the terminal set includes k constants, then every chromosome supplied by the *BRKGA* encodes a Program-Expression (*PE*) as a vector of $(h + t + k)$ random keys, and Fig. 5 depicts the corresponding chromosome structure.

Decoding a Chromosome into a *PE*

The decoding of a chromosome into a Program-Expression (*PE*) has the following three steps:

- Decoding the head of the *PE*;
- Decoding the tail of the *PE*;
- Decoding the values of the constants.

Let the terminal set $\mathcal{T}$ be divided into the two mutually exclusive sets $\mathcal{V}$ and $\mathcal{K}$ which represent the set of variables and the set of constants, respectively, and let vF, vV, and vK represent vectors containing all the elements in the sets $\mathcal{F}$, $\mathcal{V}$, and $\mathcal{K}$, respectively. Additionally, let nF, nV, and nK represent the number of elements in the vectors vF, vV, and vK, respectively.

To better illustrate the various steps in the decoding process, an example based on the information presented in Table 1 is used.

According to Table 1 and the chromosome structure defined in the previous section, the chromosome will have 13 $(5 + 6 + 2)$ random keys. Furthermore, assume that the chromosome being decoded is

$$c = (0.25, 0.35, 0.93, 0.75, 0.05, 0.32, 0.67, 0.58, 0.15, 0.26, 0.86, 0.64, 0.43)\ . \tag{2}$$

Table 1 Information relative to the example

$\mathcal{F} = \{+, - \times \div, \surd\}$	$\Rightarrow$	$vF = \left(+, - \times \div, \surd\right)$ $nF = 5$
$\mathcal{T} = \{X, Y, Z, K_1, K_2\}$	$\Rightarrow$	$vV = \{X, Y, Z\}, nV = 3$ $vK = \{K_1; K_2\}$, $nK = 2$
$h = 5, a = 2$	$\Rightarrow$	$t = 5 \times (2 - 1) + 1 = 6$

```
procedure DecodeHead (vF, vV, vK, c, PE)
1   for r = 1, ..., h do
2       idx = ⌈c[r] × (nF + nV + nK)⌉
        // where ⌈x⌉ is the smallest integer greater or equal to x.
        // ** Decoding c[r] → PE[r] **
3       if (idx ≤ nF) then
            // it going to be a function
4           PE[r] = vF[idx];
5       else if (nF < idx ≤ nF + nV) then
            // it going to be a variable
6           PE[r] = vV[idx − nF];
7       else if (nF + nV < idx ≤ nF + nV + nK) ≤then
            // it going to be a constant
8           PE[r] = vK[idx − nF − nV];
9       endif
10  end for
end DecodeHead;
```

Fig. 6 Pseudo-code for the `DecodeHead` procedure

The decoding of the head of the *PE* is based on the fact that the head can be made of elements in any of the sets $\mathcal{F}$, $\mathcal{V}$, and $\mathcal{K}$. The mapping of the random key in position $r = 1, \ldots, h$ of the chromosome c, $c[r]$, into the primitive element in position r of the *PE*, $PE[r]$, is accomplished by the procedure DecodeHead whose pseudo-code is shown in Fig. 6.

Table 2 presents the results of the decoding of the head of the *PE* corresponding to the first five components of the chromosome given in expression (2).

At this point the first five components of the *PE* will be

$$\times \div \quad K_2 \quad Z + .$$

Table 2 Decoding the Head of the *PE*

r	$c[r]$	idx	Head of the *PE*
1	0.25	$\lceil 0.25 \times (5+3+2) \rceil = 3$	$vF[3] = \times$
2	0.35	$\lceil 0.35 \times (5+3+2) \rceil = 4$	$vF[5] = \div$
3	0.93	$\lceil 0.93 \times (5+3+2) \rceil = 10$	$vK[10-5-3] = K_2$
4	0.75	$\lceil 0.75 \times (5+3+2) \rceil = 8$	$vV[8-5] = Z$
5	0.05	$\lceil 0.05 \times (5+3+2) \rceil = 1$	$vF[1] = +$

```
procedure DecodeTail (vV, vK, c, PE)
1   for r = h + 1, ..., h + t do
2       idx = ⌈c[r] × (nV + nK)⌉
        // where ⌈x⌉ is the smallest integer greater or equal to x.
        // ** Decoding c[r] → PE[r] **
3       if (idx ≤ nV) then
            // it going to be a variable
4           PE[r] = vV[idx];
5       else if (nV < idx ≤ nV + nK) then
            // it going to be a constant
6           PE[r] = vK[idx − nV];
7       endif
8   end for
end DecodeTail;
```

Fig. 7 Pseudo-code for the `DecodeTail` procedure

The decoding of the tail of the *PE* is based on the fact that the tail can only consist of elements in sets $\mathcal{V}$ and $\mathcal{K}$. The mapping of the random key in position $r = h+1, \ldots, h+t$ of the chromosome c, $c[r]$, into the primitive element in position r of the *PE*, $PE[r]$, is accomplished by the procedure DecodeTail which has the pseudo-code presented in Fig. 7.

Table 3 presents the results of the decoding of the head of the *PE* corresponding to the chromosome given in expression (2).

At this point the components of the *PE* will be

$$\times \quad \div \quad K_2 \quad Z \quad + \quad Y \quad K_1 \quad Z \quad X \quad Y \quad K_2.$$

Table 3 Decoding the tail of the *PE*

r	$c[r]$	idx	Tail of the PE
6	0.32	$\lceil 0.32 \times (3+2) \rceil = 2$	$vV[2] = Y$
7	0.67	$\lceil 0.67 \times (3+2) \rceil = 4$	$vK[4-3] = K_1$
8	0.58	$\lceil 0.58 \times (3+2) \rceil = 3$	$vV[3] = Z$
9	0.15	$\lceil 0.15 \times (3+2) \rceil = 1$	$vV[2] = X$
10	0.26	$\lceil 0.26 \times (3+2) \rceil = 2$	$vV[4-3] = Y$
11	0.86	$\lceil 0.86 \times (3+2) \rceil = 5$	$vK[5-3] = K_2$

This *PE* is translated with the *P-Rule* into

$$\frac{K_2}{Z} \times (Y + K_1)\,. \tag{3}$$

Note that the last four components in the *PE* are noncoding elements.

The final step in the decoding process consists in decoding the values of the constants in the terminal set. That is accomplished by using the decoding expression

$$K_i = fk\left(c\left[h+t+i\right], p_1, p_2, \ldots, p_p\right) \quad i = 1, \ldots, nK,$$

where $fk()$ is a function that accepts as input a random key, $c[r]$, and a set of parameters $p_1, p_2, \ldots, p_p$ and outputs a real or integer value. The simplest case can be obtained when $fk(c[r]) = c[r]$, i.e., the value of the constant is equal to the value of the random-key input. Many functions can be used for $fk()$. However, the following functions were used:

1. **randReal**$(c[r], l, u) = l + c[r] \times (u - l)$ – which generates a real value between l and u;
2. **randInt**$(c[r], l, u) = \lfloor l + c[r] \times (u - l) \rfloor$ – which generates an integer value between the integers l and u.

For the example, assuming $fk(c[r]) = c[r]$, the values of the constants K_1 and K_2 are, respectively, $c[5+6+1] = 0.64$ and $c[5+6+2] = 0.43$. The final program can be obtained by replacing the constants K_1 and K_2 in expression (3) by 0.64 and 0.43, respectively, i.e.:

$$\frac{0.43}{Z} \times (Y + 0.64)\,. \tag{4}$$

Fitness Function

The objective of the fitness measure is to assign a quality value to each chromosome so that the evolutionary process differentiates the different chromosomes and directs

Table 4 Dataset for *SR*-example 1

x	y	x	y	x	y
1	10	11	62,810	21	806,610
2	98	12	88,428	22	969,958
3	426	13	121,186	23	1,156,946
4	1252	14	162,302	24	1,369,752
5	2930	15	213,090	25	1,610,650
6	5910	16	274,960	26	1,882,010
7	10,738	17	349,418	27	2,186,298
8	18,056	18	438,066	28	2,526,076
9	28,602	19	542,602	29	2,904,002
10	43,210	20	664,820	30	3,322,830

the search to better solutions (chromosomes). Depending on the problem being solved, different fitness measures can be used.

Suppose that a symbolic regression problem is to be solved where the best fitting function to a set of points is to be found. In this case the usual measures used in the linear or nonlinear regression can be used, least squares, mean squared error, etc. However, if, for example, a rule to decide when to buy or sell a certain stock in the NYSE is to be discovered, then the quality of rule could be evaluated by using historical data, and the profit obtained with the rule, over a certain number of historical days, could be considered the measure of fitness.

Examples

This section presents two examples in the area of symbolic regression (*SR*) to illustrate the application of *BRKGP*. The mean absolute error (MAE)

$$\text{MAE} = \frac{1}{ND} \sum_{d=1}^{d=ND} |f_d - y_d|$$

is used as fitness measure, where ND is the number of data points, y_d is the value of data point d, and f_d is the value obtained by the expression generated by the *BRKGP* for data point d.

SR-Example 1

Table 4 presents 30 data points that are sampled from the function $y = x + 2x^2 + 3x^3 + 4x^4$. *BRKGP* is used to try to discover the polynomial that best fits the data.

The *BRKGP* configuration used for this example is given in Table 5.

Table 5 *BRKGP* configuration for the *SR*-example 1

$\mathcal{F} = \{+, -, \times, \div\}$	$p = 500$
$\mathcal{T} = \{x\}$	$p_e = 100$
$h = 5, 10, 15$	$p_m = 100$
$a = 2$	$\rho_e = 0.85$
$t = h \times (a - 1) + 1$	Fitness = mean absolute error (MAE)
	Stopping criterion = 100 generations

Table 6 Dataset for *SR*-example 2

R	*Area*	*R*	*Area*
10	314.1592654	21	1385.44236
11	380.1327111	22	1520.530844
12	452.3893421	23	1661.902514
13	530.9291585	24	1809.557368
14	615.7521601	25	1963.495408
15	706.8583471	26	2123.716634
16	804.2477193	27	2290.221044
17	907.9202769	28	2463.00864
18	1017.87602	29	2642.079422
19	20,1256.637061	30	2827.433388

Note that, since the best value of h to use is not known, three possibilities for h ($h = 5$, $h = 10$, $h = 15$) are tried. For $h = 5$ and $h = 10$ the *BRKGP* was not able to find the correct polynomial (i.e., MAE > 0). However, with $h = 15$ it was able to find the correct polynomial (i.e., MAE = 0) after 12 generations. The final expression was

$$(((((X + X) \times (X \times X)) + X) \times ((X + X) + (X \div X))) + ((X \times X) \times X))$$

which after being simplified gives the correct polynomial.

SR-Example 2

In this example *BRKGP* is used to try to discover the relation between the *Area* of the circle and its radius *R* that best fits the 20 data points presented in Table 6. The 20 data points were sampled from the function $Area = \pi R^2$.

The *BRKGP* configuration used for this example is given in Table 7. Note that, in this case, three constants K_1, K_2, K_3 are included that will be decoded using $fk() =$ **randInt**(1, 10,000).

For $h = 10$ *BRKGP* found, after 67 generations, the expression

$$((R + ((R \div 689) \div 689)) \times (((9151 \div 4273) \times R) + R))$$

Table 7 *BRKGP*configuration for the *SR*-example 2

$\mathcal{F} = \{+, -, \times, \div\}$	$p = 500$
$\mathcal{T} = \{R, K_1, K_2, K_3\}$	$p_e = 100$
$fk() =$ **randInt**(1, 10,000)	$p_m = 100$
$h = 10, 15$	$\rho_e = 0.85$
$a = 2$	Fitness = mean absolute error (MAE)
$t = h \times (a - 1) + 1$	Stopping criterion = 100 generations

which after being simplified is equivalent to $Area = 3.1415933\ R^2$ and has MAE = 0.00027997.

For $h = 15$ *BRKGP* found, after 82 generations, the expression

$$((R \times (((R + R) + R) \times ((476 + 9787) - (6699 \div 476)))) \div 9787)$$

which after being simplified is equivalent to $Area = 3.1415939\ R^2$ and has MAE = 0.00051594.

Note that, in both cases, the value of π was approximated to five decimals.

Conclusions

This chapter introduced biased random-key genetic programming, a novel metaheuristic for genetic programming. After introducing how programs are represented using a linear Program-Expression representation, the chapter presents the head-tail encoding and explains how the decoding of the chromosome can be accomplished. The chapter concludes by illustrating how *BRKGP* can be applied to solve problems using two symbolic regression examples.

Cross-References

▸ Evolutionary Algorithms
▸ Genetic Algorithms
▸ Random-Key Genetic Algorithms: Principles and Applications

Acknowledgments The first author was supported by project PTDC/EGE-GES/117692/2010 funded by the ERDF through the Programme COMPETE and by the Portuguese Government through FCT – Foundation for Science and Technology.

Competing Interest Declaration The author(s) has no competing interests to declare that are relevant to the content of this manuscript.

References

1. Banzhaf W, Nordin P, Keller RE, Francone FD (1998) Genetic programming: an introduction. Morgan Kaufmann, San Francisco
2. Barricelli NA et al (1954) Esempi numerici di processi di evoluzione. Methodos 6(21–22): 45–68
3. Bean JC (1994) Genetic algorithms and random keys for sequencing and optimization. ORSA J Comput 6:154–160
4. Brameier MF, Banzhaf W (2007) Linear genetic programming. Springer, New York
5. Cramer NL (1985) A representation for the adaptive generation of simple sequential programs. In: Proceedings of the first international conference on genetic algorithms, Pittsburg, pp 183–187
6. Ferreira C (2001) Gene expression programming: a new adaptive algorithm for solving problems. Complex Syst 13:87–129
7. Ferreira C (2006) Designing neural networks using gene expression programming. In: Abraham A (ed) Applied soft computing technologies: the challenge of complexity. Springer, Berlin, pp 517–535
8. Ferreira C (2006) Gene expression programming: mathematical modeling by an artificial intelligence. Studies in computational intelligence. Springer, New York
9. Fogel LJ (1964) On the organization of intellect. PhD thesis, UCLA
10. Fontes DBMM, Gonçalves JF (2013) A multi-population hybrid biased random key genetic algorithm for hop-constrained trees in nonlinear cost flow networks. Optim Lett 7(6): 1303–1324
11. Goldberg DE (1989) Genetic algorithms in search, optimization, and machine learning. Addison-Wesley, Reading
12. Gonçalves JF, Almeida J (2002) A hybrid genetic algorithm for assembly line balancing. J Heuristics 8:629–642
13. Gonçalves JF, Resende MGC (2011) Biased random-key genetic algorithms for combinatorial optimization. J Heuristics 17:487–525
14. Gonçalves JF, Resende MGC (2013) A biased random-key genetic algorithm for a 2D and 3D bin packing problem. Int J Prod Econ 145:500–510
15. Gonçalves JF, Resende MG (2014) An extended Akers graphical method with a biased random-key genetic algorithm for job-shop scheduling. Int Trans Oper Res 21(2):215–246
16. Gonçalves JF, Resende MGC (2015) A biased random-key genetic algorithm for the unequal area facility layout problem. Eur J Oper Res 246(1):86–107
17. Gonçalves J, Resende M, Toso R (2014) An experimental comparison of biased and unbiased random-key genetic algorithms. Pesquisa Operacional 34:143–164
18. Gonçalves JF, Resende MGC, Costa MD (2014) A biased random-key genetic algorithm for the minimization of open stacks problem. Int Trans Oper Res. Published online 2 July 2014
19. Gonçalves JF, de Magalhães Mendes JJ, Resende MG (2015) The basic multi-project scheduling problem. In: Schwindt C, Zimmermann J (eds) Handbook on project management and scheduling, vol 2. Springer, Berlin, pp 667–683
20. Koza JR (1992) Genetic programming: on the programming of computers by means of natural selection, vol 1. MIT, Cambridge
21. Koza JR (1994) Genetic programming II: automatic discovery of reusable subprograms. MIT, Cambridge
22. Koza JR, Bennett FH III, Andre D, Keane MA (1999) Genetic Programming III: Darwinian invention and problem solving. Morgan Kaufmann, San Francisco
23. Koza JR, Keane MA, Streeter MJ, Mydlowec W, Yu J, Lanza G (2003) Genetic programming IV: routine human-competitive machine intelligence. Kluwer, Norwell/Dordrecht

24. Poli R (1997) Evolution of graph-like programs with parallel distributed genetic programming. In: Proceedings of the 7th international conference on genetic algorithms (ICGA), East Lansing, pp 346–353
25. Spears WM, DeJong KA (1991) On the virtues of parameterized uniform crossover. In: Proceedings of the fourth international conference on genetic algorithms, San Diego, pp 230–236

Data Mining in Heuristics

3

Marcelo R. H. Maia, Isabel Rosseti, Simone de Lima Martins, and Alexandre Plastino

Contents

Introduction . . . 42
Strategies to Use Data Mining in Heuristics . . . 43
DM-MSH and MDM-MSH Strategies . . . 43
MineReduce Strategy . . . 46
Hybrid Heuristics with Data Mining: A Selection of Cases . . . 49
DM-Supported HGS for the Capacitated VRP . . . 49
DM-Supported MS-ILS for the Heterogeneous Fleet VRP . . . 55
DM-Supported MS-ILS for the Multisource Capacitated FLP with Customer Incompatibilities . . . 63
Conclusion . . . 68
Cross-References . . . 68
References . . . 69

Abstract

This chapter explores some heuristics incorporating data mining procedures. The basic idea of using data mining inside a heuristic is to obtain knowledge from previous iterations performed by a heuristic to guide the search in the subsequent iterations. Patterns extracted from good-quality solutions can be used to guide the

M. R. H. Maia
Universidade Federal Fluminense, Niterói, Rio de Janeiro, Brazil

Instituto Brasileiro de Geografia e Estatística, Rio de Janeiro, Brazil
e-mail: mmaia@ic.uff.br; marcelo.h.maia@ibge.gov.br

A. Plastino · I. Rosseti · S. de Lima Martins
Universidade Federal Fluminense, Niterói, Rio de Janeiro, Brazil
e-mail: rosseti@ic.uff.br; simone@ic.uff.br; plastino@ic.uff.br

© Springer Nature Switzerland AG 2025

R. Martí et al. (eds.), *Handbook of Heuristics*,
https://doi.org/10.1007/978-3-032-00385-0_11

search, leading to a more effective exploration of the solution space. This survey shows that heuristics may benefit from data mining by obtaining better solutions in shorter computational times.

Keywords

Data mining · Frequent patterns · Hybrid metaheuristics · Problem decomposition · Problem size reduction

Introduction

Many combinatorial optimization problems of practical relevance are classified as NP-hard, which generally means they cannot be solved efficiently. As a result, one approach to tackling these challenging problems is to accept suboptimal solutions rather than attempting to find the optimal ones. Heuristics serve as computational methods designed to generate and evaluate candidate solutions iteratively. Although the complexity of finding optimal solutions can grow exponentially with the size of NP-hard problem instances, the evaluation of candidate solutions can often be conducted much more efficiently, typically in polynomial time [5].

Metaheuristics have been developed to enhance the performance of heuristics in exploring solution spaces. These high-level procedures provide a framework that guides basic heuristics, improving their effectiveness. Well-known metaheuristics, such as Simulated Annealing, Tabu Search, GRASP, and Iterated Local Search (ILS), utilize stochastic local search (SLS) methods. These iterative processes usually yield feasible candidate solutions at each iteration and can be applied to many combinatorial optimization problems.

Data mining is crucial in extracting valuable knowledge from data [4, 15, 17, 19]. It involves automatically identifying patterns representing significant features and revealing concepts embedded in raw data. As a result, data mining becomes essential for informed decision-making in various contexts.

In [7], we presented a survey about incorporating data mining into hybrid metaheuristics to extract patterns from high-quality solution sets. The core idea is to use knowledge from previous heuristic iterations to guide future searches. It has been shown that heuristics can achieve a more effective exploration of the solution space by analyzing patterns derived from high-quality solutions.

This chapter updates the survey presented in [7] by highlighting recent advancements in applying data mining techniques within heuristics. In section "Strategies to Use Data Mining in Heuristics," we review strategies from the earlier survey that combine metaheuristics with data mining processes and introduce a new approach called MineReduce. Then, in section "Hybrid Heuristics with Data Mining: A Selection of Cases," we analyze various implementations across different applications and the corresponding computational results. Finally, section "Conclusion" offers concluding remarks and discusses potential future work in this evolving field.

Strategies to Use Data Mining in Heuristics

This section describes some strategies already presented in [7] to incorporate data mining in metaheuristics and a new approach called MineReduce.

DM-MSH and MDM-MSH Strategies

As described in [5], given a (combinatorial) problem Π, a stochastic local search algorithm for solving an arbitrary problem instance $\pi \in \Pi$ is defined by the following components:

- the search space $S(\pi)$ of instance π, which is a set of candidate solutions to π;
- a set of (feasible) solutions $S'(\pi) \subseteq S(\pi)$;
- a neighborhood relation on $S(\pi)$, $N(\pi) \subseteq S(\pi) \times S(\pi)$;
- an initialization procedure INIT_PROC(π) that draws an element from a probabilistic function $init(\pi)$, which specifies a probability distribution over initial search positions;
- a step procedure STEP_PROC(π, s) that draws an element from a probabilistic function $step(\pi, s)$, which maps each given position s onto a probability distribution over its neighboring positions, for specifying the local search steps;
- a termination procedure TERM_PROC(π, s) which uses a terminate predicate T that indicates the probability the search will be terminated upon reaching a specific point in the search space.

When using stochastic local search, the solutions obtained may differ in each algorithm run, so multistart SLS metaheuristics can find a different solution in each iteration.

So, the basic concept of incorporating a data mining process in multistart SLS metaheuristics is that patterns found in high-quality solutions obtained in earlier iterations can be used to conduct and improve the search process.

Two strategies to use data mining were presented in [7]. In the first, DM-MSH, the multistart heuristic (MSH) is divided into two phases. The first one is called *Elite Set Generation Phase*. It executes gen_elite_iter MSH iterations to obtain a set $\mathscr{S}$ of different solutions. The d best solutions from $\mathscr{S}$ compose an elite set $\mathscr{D}$.

Between the two phases, the data mining procedure is applied. It is responsible for extracting a set of patterns $\mathscr{P}$ from the elite set $\mathscr{D}$. The patterns to be mined are sets of elements frequently appearing in solutions in $\mathscr{D}$, which characterizes a frequent itemset mining application. A frequent itemset mined from $\mathscr{D}$ with support $s\%$ represents a set of elements that occur together in $s\%$ of the elite solutions.

Next, in the second phase, called *Hybrid Phase*, several slightly different MSH iterations are executed. The construction and local search heuristics may use a mined pattern $p \in \mathscr{P}$ in these iterations. For example, in constructive heuristics,

all elements of a pattern p may be inserted into the partial solution, from which a complete solution will be built by executing the standard construction procedure. This way, all constructed solutions will contain the elements of a pattern $p \in \mathscr{P}$. Likewise, local search procedures may use the elements of pattern p by requiring that only neighbors that contain elements from pattern p should be evaluated.

The pseudo-code of DM-MSH is illustrated in Fig. 1. In line 1, the elite set $\mathscr{D}$ is initialized with an empty set. The loop from line 3 to line 8 corresponds to the *Elite Set Generation Phase*, in which gen_elite_iter iterations of the multistart heuristic are performed. The original construction method is executed in line 4. Then, a local search method is performed in line 5. The elite set $\mathscr{D}$ is updated in line 6. In case of improvement, the best solution is updated in line 7. The data mining procedure extracts the set of patterns $\mathscr{P}$ from $\mathscr{D}$ in line 9. The loop from line 10 to line 15 corresponds to the *Hybrid Phase*. Each iteration is based on a pattern $p \in \mathscr{P}$, selected in line 11. The construction procedure that may use data mining is performed in line 12. In line 13, the local search that may use data mining is executed. In case of improvement, the best solution is updated in line 14. The best solution is returned in line 16.

The second strategy, called MDM-MSH, was developed to improve the results further. The main idea of this strategy is to mine more than once whenever the elite set is stable and good enough to be mined. The mining process will be executed: (a) as soon as the elite set first becomes stable, which means that no change in the elite set occurs throughout a given number of iterations, and (b) whenever the elite set has been changed and again has become stable.

Figure 2 shows the pseudo-code of MDM-MSH. The loop from line 4 to 10 corresponds to the first elite set generation phase, in which MSH iterations are performed until the elite set becomes ready to be mined or the total number of

Fig. 1 Pseudo-code of DM-MSH

```
begin DM_MSH(n_iter, π, f())
1  𝒟 ← ∅;
2  f(π, best_sol) ← ∞;
3  for it = 1,...,gen_elite_iter do
4      s ← CONSTRUCT_SOL(π, f);
5      s ← LOCAL_SEARCH(s, π, f);
6      UPDATE_ELITE(𝒟, s);
7      if f(π,s) < f(π, best_sol) then best_sol ← s;
8  end-for;
9  𝒫 ← MINE(𝒟);
10 for it = gen_elite_iter + 1,...,n_iter do
11     p ← SELECT_PATTERN(𝒫);
12     s ← DM_CONSTRUCT_SOL(p, π, f);
13     s ← DM_LOCAL_SEARCH(p, s, π, f);
14     if f(π,s) < f(π, best_sol) then best_sol ← s;
15 end-for;
16 return best_sol;
end DM_MSH.
```

Fig. 2 Pseudo-code of MDM-MSH

```
begin MDM_MSH(n_iter, π, f())
1   𝒟 ← ∅;
2   f(π, best_sol) ← ∞;
3   it ← 0;
4   repeat
5       s ← CONSTRUCT_SOL(π, f);
6       s ← LOCAL_SEARCH(s, π, f);
7       elite_set_is_ready ← UPDATE_ELITE(𝒟, s);
8       if f(π, s) < f(π, best_sol) then best_sol ← s;
9       it ← it + 1;
10  until elite_set_is_ready or it = n_iter;
11  while it < n_iter do
12      if elite_set_is_ready then 𝒫 ← MINE(𝒟);
13      p ← SELECT_PATTERN(𝒫);
14      s ← DM_CONSTRUCT_SOL(p, π, f);
15      s ← DM_LOCAL_SEARCH(p, s, π, f);
16      elite_set_is_ready ← UPDATE_ELITE(𝒟, s);
17      if f(π, s) < f(π, best_sol) then best_sol ← s;
18      it ← it + 1;
19  end-while;
20  return best_sol;
end MDM_MSH.
```

iterations is executed. Next, in the loop from line 11 to 19, the data mining procedure is performed in line 12 whenever the elite set is ready. In line 13, a pattern is selected. Then, the adapted construction is performed in line 14, and in line 15 the adapted local search is executed. The best solution is updated in line 17 if a better solution is found. After executing all iterations, the best solution is returned in line 20.

DM-MSH and MDM-MSH share a common approach. At some point, the algorithm benefits from knowledge accumulated in previous iterations. This approach relies on the assumption that there is a high probability that frequent patterns mined from a restricted set of high-quality solutions are part of other suboptimal and maybe even optimal solutions to the same problem instance. Thus, they may be inserted in initial solutions or required in neighbor solutions during a local search. The main difference between DM-MSH and MDM-MSH is when and how often patterns are mined.

According to Talbi's taxonomy for metaheuristics that incorporate machine learning (ML) [14], DM-MSH and MDM-MSH can be classified as *online low-level* ML-supported metaheuristics since they gather knowledge during the search while solving the problem (hence, online) and use that knowledge in processes that drive the initialization of solutions or local search (low-level components).

Experimental evidence shows that incorporating those strategies improved the effectiveness and efficiency of several heuristics for different combinatorial optimization problems, indicating that their underlying assumption holds.

MineReduce Strategy

Another approach named MineReduce was proposed based on the previous ideas. Since the mined patterns are assumed to likely be part of the best solutions to a problem instance, this approach temporarily fixes items from a pattern in the solution, equivalent to fixing (or at least constraining) values of decision variables related to those items. Therefore, the MineReduce approach applies a decomposition strategy that reduces the original problem instance to a subproblem by deleting or aggregating items based on the information obtained from a pattern. In essence, a pattern $p \in \mathscr{P}$ is selected for driving the following three-step process:

1. *Reduce*: A reduction operation is applied to the original problem instance $\pi \in \Pi$, removing or aggregating items from p (for being fixed in the solution), which results in a reduced instance $\pi' \in \Pi$;
2. *Optimize*: A solution $s' \in S'(\pi')$ – i.e., a feasible (and presumably good) solution to the reduced instance π' – is obtained through the application of some optimization method (e.g., SLS);
3. *Expand*: An expansion operation is applied to s', reinserting or disaggregating the items from p, which results in a solution $s \in S'(\pi)$ – i.e., a feasible solution to the original instance π.

To picture this process more concretely, consider the classical 0-1 knapsack problem (01-KP), where we have a knapsack and a set of items, each with a weight and a value, and the objective is to fill the knapsack with the most valuable subset of items that it can accommodate. Specifically, consider the following toy instance of the 01-KP:

$$\pi = (W = 90, I = \{a, b, c, d, e, f, g, h, i, j\})$$

The knapsack in instance π has a capacity $W = 90$. It can hold items from the set $I = \{a, b, c, d, e, f, g, h, i, j\}$, whose properties are in Table 1.

Table 1 Items' properties for the 01-KP toy instance

Item	Weight	Value
a	9	50
b	13	35
c	18	20
d	38	16
e	15	60
f	68	45
g	27	60
h	39	40
i	23	30
j	52	10

Imagine we have designed and applied a problem-specific MSH for the 01-KP to our toy instance π. Suppose we have kept the best five solutions found across several iterations in an elite set $\mathscr{D} = \{s_1, s_2, s_3, s_4, s_5\}$, which are in Table 2.

Now, consider we have mined patterns from $\mathscr{D}$ with a minimum support of 60%. That means extracting the maximal frequent itemsets from $\mathscr{D}$, considering an itemset frequent if it is present in at least 60% of the solutions of $\mathscr{D}$. In this example, that would produce a pattern set $\mathscr{P} = \{p_1, p_2\}$. Those mined patterns are described in Table 3.

Once patterns have been mined, we can select one to drive MineReduce's reduce-optimize-expand process. For example, consider p_1 is selected. In the *reduce* step, we would assume that the items from p_1 must be part of a solution to π. Therefore, we would temporarily fix them in the solution and remove them from I, obtaining the following reduced instance:

$$\pi'_1 = (W'_1 = 53, I'_1 = \{c, d, f, g, h, i, j\})$$

In contrast, if pattern p_2 is selected instead of p_1, then a different reduced instance is obtained:

$$\pi'_2 = (W'_2 = 39, I'_2 = \{b, c, d, f, h, i, j\})$$

Note that the knapsack capacity is considerably lower in the reduced instances (we subtract the pattern weights since its items are fixed in the solution), and so is the number of items one can pick to fill the knapsack. Hence, these instances have a much smaller solution space, which means they are much easier to solve than the original one.

Now, in the *optimize* step, consider that we applied an optimization method (e.g., SLS) to π'_1 and obtained the following solution: $s'_1 = \{g, i\}$ (with weight = 50 and value = 90). Finally, we must reverse the reduction operation in the *expand* step. That is, the items from p_1 (which had been fixed in the solution) must be inserted in s'_1, which will produce a new solution to the original instance π: $s = \{a, b, e, g, i\}$ (with weight = 87 and value = 235).

Table 2 Elite solutions for the 01-KP toy instance

Solution	Weight	Value
$s_1 = \{a, b, e, h\}$	76	185
$s_2 = \{a, d, e, g\}$	89	186
$s_3 = \{a, b, c, e, i\}$	78	195
$s_4 = \{a, e, g, h\}$	90	210
$s_5 = \{a, b, c, e, g\}$	82	225

Table 3 Patterns mined for the 01-KP toy instance

Pattern	s_1	s_2	s_3	s_4	s_5
$p_1 = \{a, b, e\}$	✓		✓		✓
$p_2 = \{a, e, g\}$		✓		✓	✓

Fig. 3 Pseudo-code of MR-MSH

```
begin MR_MSH(n_iter, π, f())
1   𝒟 ← ∅;
2   f(π, best_sol) ← ∞;
3   it ← 0;
4   repeat
5       s ← CONSTRUCT_SOL(π, f);
6       s ← LOCAL_SEARCH(s, π, f);
7       elite_set_is_ready ← UPDATE_ELITE(𝒟, s);
8       if f(π, s) < f(π, best_sol) then best_sol ← s;
9       it ← it + 1;
10  until elite_set_is_ready or it = n_iter;
11  while it < n_iter do
12      if elite_set_is_ready then 𝒫 ← MINE(𝒟);
13      p ← SELECT_PATTERN(𝒫);
14      π′ ← REDUCE(π, p);
15      s′ ← CONSTRUCT_SOL(π′, f);
16      s′ ← LOCAL_SEARCH(s′, π′, f);
17      s ← EXPAND(s′, p);
18      s ← LOCAL_SEARCH(s, π, f);
19      elite_set_is_ready ← UPDATE_ELITE(𝒟, s);
20      if f(π, s) < f(π, best_sol) then best_sol ← s;
21      it ← it + 1;
22  end-while;
23  return best_sol;
end MR_MSH.
```

Since MDM-MSH has generally performed better than DM-MSH, its adaptive multi-mining structure has been adopted in the MineReduce-based MSH. Figure 3 shows the pseudo-code of MR-MSH. It has essentially the same structure as MDM-MSH. The differences can be observed from line 14 to line 18. In line 14, the problem instance π is reduced to π' based on the selected pattern p. Lines 15 and 16 correspond to a typical implementation of the *Optimize* step from the MineReduce approach, consisting of one MSH iteration (construction and local search) on $S(\pi')$, which produces a locally optimal solution s' to the reduced instance π'. In line 17, s' is expanded into the corresponding solution s to instance π based on pattern p. Finally, in line 18, s is the initial solution for a local search on $S(\pi)$.

According to Talbi's taxonomy [14], like DM-MSH and MDM-MSH, MR-MSH can be classified as an *online low-level* ML-supported metaheuristic. However, it is additionally (and primarily) classified as a *problem-level* ML-supported metaheuristic since it uses the discovered knowledge for hierarchical problem decomposition (defining and solving smaller subproblems).

The following section presents several cases showing that incorporating data mining using the above strategies improved the base heuristics' solution quality and convergence speed.

Hybrid Heuristics with Data Mining: A Selection of Cases

This section presents a selection of cases where data mining has been incorporated into heuristics for different combinatorial optimization problems.

In [7], several examples show the integration of data mining with heuristics based on Greedy Randomized Adaptive Search Procedures (GRASP) applied to problems with different characteristics. The first was proposed to solve the server replication for reliable multicast problem. A solution to this problem is an unordered set of elements. The second GRASP was proposed to solve the traveling salesperson problem with pickup and delivery. This problem has solutions that are ordered sets of elements, which require a different approach. In the last two of these four cases, data mining was integrated with hybrid versions of GRASP that include a path-relinking strategy, one for the two-path network design problem and another for the p-median problem. Data mining was also integrated with heuristics based on Iterated Local Search (ILS) to solve the set covering with pairs problem in [7].

Section "DM-Supported HGS for the Capacitated VRP" shows the results obtained by incorporating the MDM-MSH strategy into a Hybrid Genetic Search (HGS) heuristic for the capacitated vehicle routing problem.

Then, in section "DM-Supported MS-ILS for the Heterogeneous Fleet VRP," three multistart versions of ILS using data mining strategies (respectively DM-MSH, MDM-MSH, and MR-MSH) are presented and evaluated for the heterogeneous fleet vehicle routing problem.

Finally, in section "DM-Supported MS-ILS for the Multisource Capacitated FLP with Customer Incompatibilities," the MR-MSH approach is combined with the ILS heuristic for the multisource capacitated facility location problem with customer incompatibilities.

The computational results from all these applications confirm that frequent patterns mined from restricted sets of high-quality solutions can improve the performance of heuristic methods regarding solution quality and convergence speed.

DM-Supported HGS for the Capacitated VRP

This section summarizes the results obtained by incorporating data mining into a multistart SLS heuristic for the capacitated vehicle routing problem (CVRP). An application of the MDM-MSH strategy was proposed for this problem [10] and ranked 2nd in the CVRP track of the 12th DIMACS Implementation Challenge (http://dimacs.rutgers.edu/programs/challenge/vrp/cvrp).

Problem Definition

The CVRP is described as follows. Let $G = (V, A)$ be a directed graph, where $V = \{0, 1, \ldots, n-1\}$ is a set composed of n vertices and $A = \{(i, j) : i, j \in V, i \neq j\}$ is the set of arcs. Vertex 0 is the depot from where the vehicles depart, whereas the

set $V' = V \backslash \{0\}$ consists of the remaining vertices representing $n - 1$ customers. Each customer $i \in V'$ is associated with a nonnegative quantity q_i that specifies the demand for some resource, and a vehicle can carry a maximum quantity Q of the resource. Each arc $(i, j) \in A$ has an associated cost d_{ij} representing the distance between vertices i and j. A route is defined by a sequence of vertices $R = (i_1, i_2, \ldots, i_{|R|})$, $i_1 = i_{|R|} = 0$, and $\{i_2, \ldots, i_{|R|-1}\} \subseteq V'$; that is, each route is a circuit in G that starts and ends at the depot. A route R is feasible if the sum of all customers' demands on R does not exceed the vehicle capacity Q. A route's cost is the sum of the costs (distances) associated with each traversed arc. The aim is to discover feasible routes such that each customer is visited precisely once, and the sum of all route costs is minimized. There are no restrictions on the number of routes in a solution.

Data Mining Application

The proposed method was built by incorporating data mining into a state-of-the-art Hybrid Genetic Search (HGS) metaheuristic for the CVRP [16]. HGS is an iterative population-based metaheuristic that restarts the population whenever it completes a number of consecutive iterations without improving the overall best solution.

In MDM-HGS, the elite set E is updated whenever a new feasible solution is inserted in the population if E is not full or the new solution is better than the worst solution in E. It is considered $stable$ when one of the following criteria is met: (i) E has not been mined yet, and its contents have not changed for a number of consecutive restarts greater than or equal to δR_{MAX}, where δ is a parameter and R_{MAX} is the maximum number of restarts, dynamically estimated based on the time limit, the elapsed time, and the number of completed restarts; or (ii) the contents of E have changed after it was last mined, but they have not changed for a number of consecutive restarts greater than or equal to δR_{MAX}.

The data mining method returns frequent patterns found in E. Each pattern is a set of paths connecting customers. Figure 4 illustrates this data mining process with four distinct solutions for a CVRP instance. Suppose these solutions compose an elite set for the corresponding instance and let the minimum support be 100%. In that case, the data mining method would extract a pattern composed of the paths that appear in all four solutions (thick lines in the figure).

A new solution generation method is introduced in MDM-HGS. It is based on a randomized version of the Clarke and Wright (CW) heuristic and uses the paths from a mined pattern to initialize routes. Its randomization mechanism is a roulette wheel selection as in the CW implementation discussed in [13]. The *savings list S* is built when the problem instance data is loaded. It contains all edges that connect two customers in the input graph, sorted in decreasing order of their corresponding savings values. A savings value represents the cost savings obtained by concatenating two routes, one where customer i is adjacent to the depot and another where customer j is adjacent to the depot, via the insertion of the edge $\langle i, j \rangle$ in the solution. It is given by $s_{ij} = d_{0i} + d_{0j} - d_{ij}$, where d_{ij} is the distance between vertices i and j, and vertex 0 is the depot.

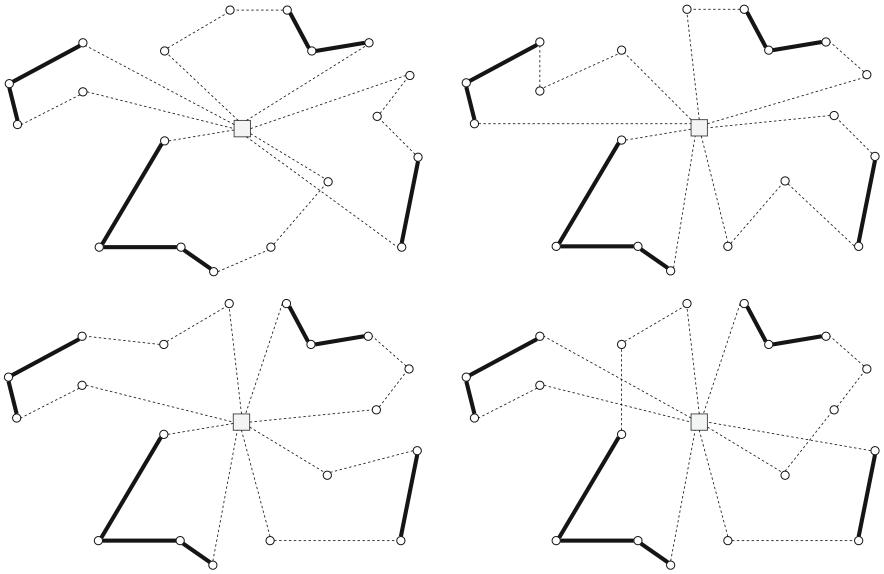


Fig. 4 Four distinct solutions for a CVRP instance and a pattern with 100% support. (Source: [10, p. 5])

The randomized CW solution generation based on a pattern p is described in Fig. 5. The solution is initialized with one route for each path in p (lines 1–3) and one for each customer not in p (lines 4–6). Then, the savings list S is iteratively processed (lines 7–13). At each step, an edge $\langle i, j \rangle$ is selected among the t (a random number between 2 and 6, inclusive) first unvisited entries in S using the roulette wheel method (line 9). Suppose customers i and j are adjacent to the depot on two different routes, and the vehicle capacity is sufficient for the total demand of both routes. In that case, these routes are concatenated by inserting $\langle i, j \rangle$ in the solution (lines 10–12). Finally, after S has been processed, a greedy strategy is used to avoid an excessive number of routes: for each route with only one customer i, i is moved to the end of a route with more than one customer. Each move is the cheapest, feasible if possible (line 16), infeasible otherwise (line 18).

Computational Results

Extensive computational experiments were conducted to compare MDM-HGS and the original method HGS-CVRP concerning the best solution found (within a given time-out) and the primal integral (PI) [1], which aims to measure the progress of the primal bound's convergence toward the optimal solution over the entire solving time. The smaller it is, the better the expected quality of the incumbent solution if the solving procedure is stopped at an arbitrary point in time. For each combination of instance and algorithm, the following values were registered: the average solution

```
begin DM_RCW(p)
1  for each path in p do
2      Initialize a route with the current path;
3  end-for;
4  for each customer not in p do
5      Initialize a route with the current customer;
6  end-for;
7  while S contains unvisited positive-valued entries do
8      t ← RAND(2,6);
9      ⟨i, j⟩ ← ROULETTE_WHEEL(S,t);
10     if two routes can be feasibly concatenated through ⟨i, j⟩ then
11         Concatenate the two routes through ⟨i, j⟩;
12     end-if;
13 end-while;
14 for each route with only one customer i do
15     if i can be feasibly moved to a route with more than one customer then
16         Make the cheapest feasible move of i to the end of another route;
17     else
18         Make the cheapest infeasible move of i to the end of another route;
19     end-if;
20 end-for;
21 return the generated solution;
end DM_RCW.
```

Fig. 5 Pseudo-code of the pattern-based randomized CW solution generation

cost (Avg Cost), the percentage gap of Avg Cost to the BKS (Gap), the best solution cost (Best Cost), and the average primal integral (Avg PI).

Table 4 summarizes the results, showing for each algorithm the following: the average Gap (Avg Gap); the global average PI; the number of wins in Avg Cost, Best Cost, and Avg PI; and the number of best-known solutions found. Improving values are highlighted in bold. This comparison shows that MDM-HGS obtains solutions of higher quality and better PI values than HGS-CVRP in most instances. The only exception is the DIMACS subset, where HGS-CVRP has a slightly lower PI, but that difference is not statistically significant. On the other hand, MDM-HGS has a statistically significant advantage in the average gap for the same subset.

The 12th DIMACS Implementation Challenge had 16 teams competing in the CVRP track. All competing algorithms were ranked according to their PI value for each instance. The best algorithm got 10 points, the second 8, then 6, 5, 4, 3, 2, and 1. The final ranking is based on the total point score obtained by each method in all instances. Table 5 presents the rankings from Phase 1 for all instances and the five subsets separately.

The Phase 1 results show that MDM-HGS was very competitive. It ranked first in 2 of the 5 instance subsets: X, the largest one with 100 instances, and Classic, for which MDM-HGS achieved an almost perfect score with 66 points of the 70 possible. Besides, despite ranking fourth in the Golden subset, that was a good performance, considering that the margin among the top-four algorithms in this

Table 4 Summarized comparison of HGS-CVRP to MDM-HGS

	All (141)		X (100)		Belgium (10)		Classic (7)		Golden (12)		DIMACS (12)	
	HGS	MDM	HGS	MDM	HGS	MDM	HGS	MDM	HGS	MDM	HGS	MDM
Avg Gap	0.40%	**0.33%**[a]	0.08%	**0.07%**[a]	4.74%	**3.75%**[a]	0.00%	0.00%	0.07%	0.07%	0.09%	**0.08%**[a]
Global Avg PI	0.4884	**0.3981**[a]	0.1176	**0.1145**	5.3981	**4.1644**[a]	0.0047	**0.0044**	0.1190	**0.1124**	**0.1384**	0.1387
Wins Avg Cost	34	**61**[a]	24	**40**[a]	1	**9**[a]	1	1	4	4	4	**7**
Wins Best Cost	21	**47**[a]	14	**32**[a]	1	**9**[a]	**1**	0	2	2	3	**4**
Wins Avg PI	53	**81**[a]	41	**54**	0	**10**[a]	2	**3**	5	**7**	5	**7**
Nb BKS	68	**72**	50	**53**	0	0	**7**	6	7	**8**	4	**5**

[a]Statistically significant advantage

Table 5 Rankings from the 12th DIMACS Implementation Challenge – CVRP Track – Phase 1

All (141)		X (100)		Belgium (10)		Classic (7)		Golden (12)		DIMACS (12)	
Solver	Score	Solver	Score	Solver	Score	Solver	Score	Solver	Score	Solver	Score
FHCSolver	931	**MDM-HGS**	**663**	FSP4D	100	**MDM-HGS**	**66**	FHCSolver	86	FHCSolver	83
MDM-HGS	**896**	FHCSolver	642	FHCSolver	80	POP-HGS	51	POP-HGS	83	FSP4D	75
POP-HGS	814	POP-HGS	605	AILS-II	56	FHCSolver	40	FSP4D	81	POP-HGS	74
FSP4D	737	HGSRR	480	HGSRR	44	FSP4D	34	**MDM-HGS**	**80**	HGSRR	73
HGSRR	678	FSP4D	447	LKHSP	36	HGSRR	28	HGSRR	53	**MDM-HGS**	**62**
AILS-II	558	AILS-II	416	**MDM-HGS**	**25**	MAESN	19	MAESN	35	AILS-II	52
MAESN	406	MAESN	352	optaplanner	19	ILS-SP	12	AILS-II	25	GA+TBD	29
GA+TBD	291	GA+TBD	222	ALNS++	17	AILS-II	9	GA+TBD	25	LKHSP	8
ILS-SP	70	ILS-SP	49	GA+TBD	7	GA+TBD	8	LKHSP	0	ALNS++	8
LKHSP	57	ALNS++	17	ILS-SP	5	LKHSP	6	ALNS++	0	ILS-SP	4
ALNS++	42	LKHSP	7	POP-HGS	1	optaplanner	0	ILS-SP	0	optaplanner	0
optaplanner	19	optaplanner	0	MAESN	0	ALNS++	0	optaplanner	0	Clowder	0
Clowder	0	Clowder	0	Clowder	0	Clowder	0	Clowder	0	MAESN	0
Resilience	0	Resilience	0	Resilience	0	OR-Tools	0	OR-Tools	0	Resilience	0
DDVRP	0	DDVRP	0	OR-Tools	0	Resilience	0	Resilience	0	OR-Tools	0
OR-Tools	0	OR-Tools	0	DDVRP	0	DDVRP	0	DDVRP	0	DDVRP	0

subset was very tight (only six points). The results obtained in the remaining two subsets were also above average, ranking fifth and sixth, respectively.

The top 8 competitors from Phase 1 qualified for the second and final phase, in which 100 new (undisclosed) instances were used. Table 6 presents the rankings from Phase 2 for all 100 instances and partitions concerning customer positioning and average route size.

Phase 2 results confirmed that MDM-HGS is very competitive. The instance partitioning reveals that it outperforms all competitors for two classes and ties in first for another. Regarding customer positioning, the instances are partitioned into three classes: "clustered," "random," and "random-clustered" (half of the customers are clustered, and the remaining customers are randomly positioned). MDM-HGS ranked first in the clustered instances. Naturally, these instances are more prone to the occurrence of patterns than random ones, favoring the pattern-based strategy applied in the MDM-MSH framework. Nevertheless, MDM-HGS also achieved high scores for the other partitions, tying for first place with FHCSolver in the random instances and ranking second in the random-clustered ones.

The average route size is defined as the number of customers divided by the minimum number of vehicles. Regarding this aspect, the instances are partitioned into "long-route" (the 50 with longest routes) and "short-route" (the 50 with shortest routes). MDM-HGS ranked first in the long-route instances. The longer the routes in solutions to an instance, the more likely long paths will occur in patterns mined from those solutions. Hence, long-route instances seem to favor the pattern-based strategy applied in the MDM approach more than short-route ones.

MDM-HGS was a close second to FHCSolver in the overall ranking. One interesting observation is that FHCSolver combines the original HGS-CVRP with an adapted version of another algorithm (FILO). Its strategy relies primarily on selecting which algorithm to use based on the instance's number of customers and average route size. Since the experimental results provide evidence that MDM-HGS outperforms HGS-CVRP, ultimately, FHCSolver's performance could probably be improved by simply replacing HGS-CVRP with MDM-HGS within its framework.

These results show that MDM-HGS is a competitive method in the CVRP state of the art, overcoming the original HGS-CVRP and many other novel algorithms. MDM-HGS was the first heuristic to apply this pattern-based approach in a population-based algorithm, which depicts an evolutionary framework, showing that data mining can improve performance in such a context.

DM-Supported MS-ILS for the Heterogeneous Fleet VRP

This section summarizes the results obtained by incorporating data mining into a multistart SLS heuristic for the heterogeneous fleet vehicle routing problem (HFVRP). Applications of the DM-MSH and MDM-MSH strategies were proposed for this problem in [8], and an application of the MR-MSH strategy was proposed in [9], the very first application of the MineReduce approach.

Table 6 Rankings from the 12th DIMACS Implementation Challenge – CVRP Track – Phase 2

All (100)		Clustered (32)		Random (34)		Rd-clustered (34)		Long-route (50)		Short-route (50)	
Solver	Score	Solver	Score	Solver	Score	Solver	Score	Solver	Score	Solver	Score
FHCSolver	643	**MDM-HGS**	**213**	FHCSolver	217	FHCSolver	236	**MDM-HGS**	**334**	FHCSolver	329
MDM-HGS	**622**	FHCSolver	190	**MDM-HGS**	**217**	**MDM-HGS**	**192**	FHCSolver	314	AILS-II	308
AILS-II	536	POP-HGS	184	AILS-II	184	AILS-II	179	FSP4D	274	**MDM-HGS**	**288**
POP-HGS	525	AILS-II	173	HGSRR	177	FSP4D	175	POP-HGS	272	POP-HGS	253
HGSRR	477	FSP4D	136	POP-HGS	175	HGSRR	166	HGSRR	236	HGSRR	241
FSP4D	458	HGSRR	134	FSP4D	147	POP-HGS	166	AILS-II	228	MAESN	203
MAESN	389	MAESN	132	MAESN	132	MAESN	125	MAESN	186	FSP4D	184
GA+TBD	250	GA+TBD	86	GA+TBD	77	GA+TBD	87	GA+TBD	106	GA+TBD	144

Problem Definition

The HFVRP is described as follows. Let $G = (V, A)$ be a directed graph, where $V = \{0, 1, \ldots, n\}$ is a set composed of $n + 1$ vertices and $A = \{(i, j) : i, j \in V, i \neq j\}$ is the set of arcs. Vertex 0 is the depot, where the vehicle fleet is located, whereas the set $V' = V \backslash \{0\}$ consists of the remaining vertices representing the n customers. Each customer $i \in V'$ is associated with a nonnegative demand q_i. The fleet comprises m distinct vehicle types, which compose a set $M = \{1, \ldots, m\}$. For each vehicle type $u \in M$, there are m_u vehicles available, each with a capacity Q_u and a fixed cost f_u. Finally, for each combination of an arc $(i, j) \in A$ and a vehicle type $u \in M$, there is an associated cost $c_{ij}^u = d_{ij} r_u$, where d_{ij} is the distance between vertices i and j, and r_u is the dependent (variable) cost per unit distance associated with the vehicle type u.

A route is defined by a pair (R, u), where $R = (i_1, i_2, \ldots, i_{|R|})$, $i_1 = i_{|R|} = 0$, and $\{i_2, \ldots, i_{|R|-1}\} \subseteq V'$; that is, each route is a circuit in G that starts and ends at the depot and is assigned to a vehicle of type $u \in M$. A route (R, u) is feasible if the sum of all customers' demands on R does not exceed the capacity Q_u of the vehicle assigned to it. A route's cost is the sum of the assigned vehicle's fixed cost and the dependent costs associated with each traversed arc and the vehicle type. Thus, the aim is to discover feasible routes such that each customer is visited precisely once, the total quantity of routes assigned to vehicles of each type $u \in M$ does not exceed m_u, and the sum of all route costs is minimized.

Data Mining Application

The proposed methods were built by incorporating data mining into a multistart ILS (MS-ILS) heuristic for the HFVRP [12], based on the DM-MSH, MDM-MSH, and MR-MSH strategies. They are called DM-MS-ILS, MDM-MS-ILS, and MR-MS-ILS, respectively.

In these proposals, solutions in the elite set are represented as sets of pairs containing an arc and a vehicle type. This representation allows the application of the data mining procedure to extract maximal frequent itemsets. For each route (R, u), the list R is decomposed into a set of arcs $\{(i_1, i_2), (i_2, i_3), \ldots, (i_{|R|-1}, i_{|R|})\}$. Then, the vehicle type u is assigned to each arc in the set, resulting in a set in which each element is a pair composed of an arc (i_r, i_{r+1}), $r = 1, 2, \ldots, |R| - 1$, in R and the vehicle type u, with the form $\{((i_1, i_2), u), ((i_2, i_3), u), \ldots, ((i_{|R|-1}, i_{|R|}), u)\}$. An arc must be present and associated with the same vehicle type in a minimum number of solutions in the elite set to belong to a pattern. Consequently, the patterns mined are formed of route segments, each with a vehicle type assigned.

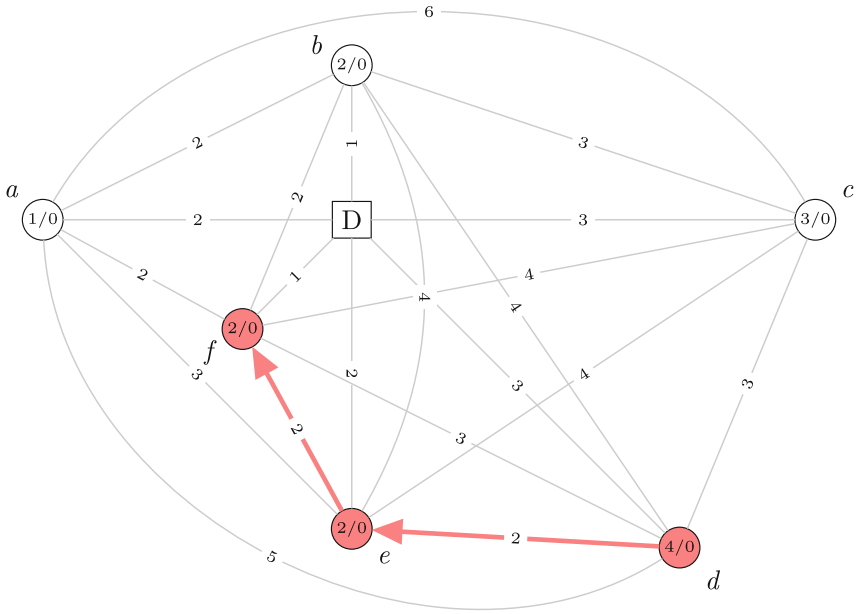
In MR-MS-ILS, the *Reduce* step of MineReduce relies on the use of patterns mined from the elite set to perform a vertex-based reduction procedure by merging customer vertices that appear consecutively (in the same route segment) in a pattern into one customer cluster vertex.

It is necessary to extend the HFVRP model to use this strategy. Since it must be possible to represent a route segment as a customer vertex, each customer $i \in V$ must have an associated l_i value corresponding to the inner length of the underlying

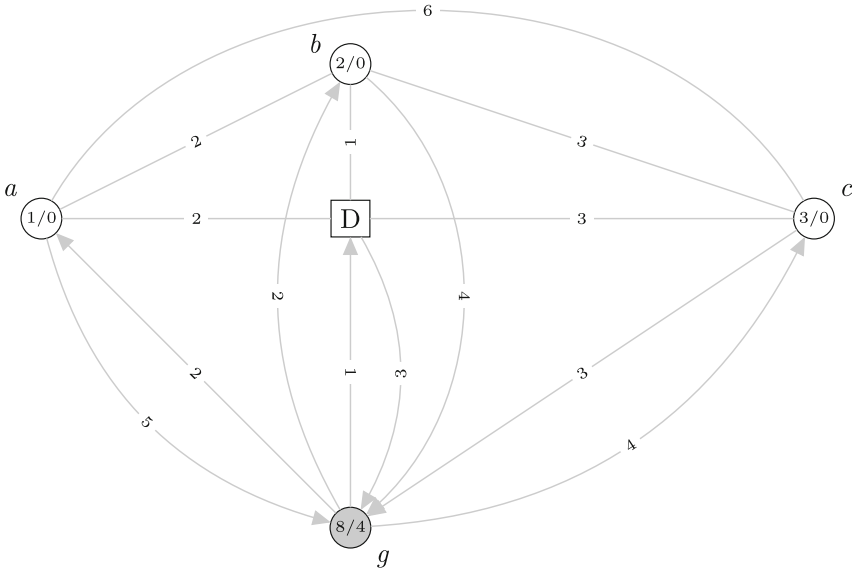
route segment. This value is used to calculate the cost $c_i^u = l_i r_u$, which represents the variable cost for a vehicle of type u to traverse the route segment represented by i, for each combination of a customer $i \in V$ and a vehicle type $u \in M$. Customer vertices of regular (non-reduced) instances and those of reduced instances that are not customer cluster vertices have an inner length of 0. The cost of a route becomes, in this extended model, the sum of the fixed cost of the vehicle associated with the route and the variable costs related to (i) the combination of the vehicle type and each traversed arc and (ii) the combination of the vehicle type and each visited customer vertex.

Let $G = (V, A)$ be a directed graph associated with an HFVRP instance and p a pattern consisting of a set of route segments in that instance, each segment defined by a pair (R', u), where $R' = (i_1, i_2, \ldots, i_{|R'|})$. Let $G^* = (V^*, A^*)$ be a directed graph associated with a reduced version of the instance associated with G based on p. Such a reduced version can be obtained as follows. Initially, G^* is defined as a copy of G. For each route segment $(R', u) \in p$, each of the customers in R' is removed from G^* – that is, the vertex corresponding to the customer is removed from V^* and the arcs that connect that vertex to the others are removed from A^*. Also, a customer cluster vertex $i_{R'}$ corresponding to the route segment is added to V^* and arcs connecting $i_{R'}$ to the other vertices in V^* are added to A^*. The demand for $i_{R'}$ is given by $q_{i_{R'}} = \sum_{r'=1}^{|R'|} q_{i_{r'}}$, that is, the sum of customer demands in R'. The distance from each vertex $i^* \in V^*$ to $i_{R'}$ is given by $d_{i^* i_{R'}} = d_{i^* i_1}$, that is, the distance from i^* to i_1 (the first customer in R'). The distance from $i_{R'}$ to each vertex $i^* \in V^*$ is given by $d_{i_{R'} i^*} = d_{i_{|R'|} i^*}$, that is, the distance from $i_{|R'|}$ (the last customer in R') to i^*. Finally, the inner length of $i_{R'}$ is given by $l_{i_{R'}} = \sum_{r=1}^{|R'|-1} d_{i_{r'} i_{r'+1}}$, that is, the sum of the distances between consecutive customers in R'.

Figure 6a presents a representation of a graph $G = (V, A)$ in the extended model described above. In this representation, the customer vertices present, in addition to the value of the demand, the value of the inner length. In this non-reduced instance, all customers have inner length 0. The pattern used in the presented reduction is highlighted in the graph. It consists of a single route segment: (R'_1, u_1), where $R'_1 = (d, e, f)$. The graph $G^* = (V^*, A^*)$ associated with the reduced version of the instance is presented in Fig. 6b. The customer vertices in R'_1 are replaced in the graph by a customer cluster represented by vertex g. This customer cluster's demand q_g is given by the sum of the customers' demands it replaces: $q_g = q_d + q_e + q_f = 4 + 2 + 2 = 8$. The distance from each of the other vertices to g is given by the distances from these vertices to d (first customer in R'_1): $d_{Dg} = d_{Dd} = 3$, $d_{ag} = d_{ad} = 5$, $d_{bg} = d_{bd} = 4$, and $d_{cg} = d_{cd} = 3$. The distance from g to each of the other vertices is given by the distances from f (last customer in R'_1) to these vertices: $d_{gD} = d_{fD} = 1$, $d_{ga} = d_{fa} = 2$, $d_{gb} = d_{fb} = 2$, and $d_{gc} = d_{fc} = 4$. Finally, the inner length l_g of g is given by the sum of the distances between consecutive customers in R'_1: $l_g = d_{de} + d_{ef} = 2 + 2 = 4$.



(a) Non-reduced instance (with a mined pattern highlighted)



(b) Reduced instance

Fig. 6 An example of reduction of an HFVRP instance based on a mined pattern. (Source: [9, p. 6]). (**a**) Non-reduced instance (with a mined pattern highlighted). (**b**) Reduced instance

Computational Results

Extensive computational experiments were conducted to compare the original MS-ILS and its hybrid versions with data mining. The first experimental comparison [8] showed that both DM-MS-ILS and MDM-MS-ILS significantly outperformed the original MS-ILS, with MDM-MS-ILS being the most effective and efficient of those two methods with data mining. The most recent computational experiments [9] compared MS-ILS, MDM-MS-ILS, and MR-MS-ILS.

The problem instances used in the computational experiments were the HFVRP benchmark instances proposed in [2], divided into 4 sets: Set 1, with 15 instances having fewer than 100 customers; Set 2, with 38 instances ranging from 100 to 150 customers; Set 3, with 31 instances ranging from 151 to 200 customers; and Set 4, with 12 instances having more than 200 customers. One instance from each set was reserved for parameter tuning and not used in the comparison.

Table 7 summarizes the results from this comparison, presenting the numbers of wins/ties (the number of instances for which a method achieved the best results) and the average percentage difference (APD) to the original MS-ILS for each instance set separately and for all instances (global).

The results show a clear superiority of MR-MS-ILS to the other two methods. It achieved better solution quality for most instances and much faster convergence for all of them, as it executes the same number of iterations in less computational time. MR-MS-ILS attained better average costs for 83% of the instances (65% with statistical significance) compared to the MS-ILS and better average costs for 76% of the instances (64% with statistical significance) in comparison with the MDM-MS-ILS. If we exclude the small instances (Set 1) where the three heuristics have shown similar performance, the superiority of MR-MS-ILS becomes more evident. It has better average costs for 91% of the instances (76% with statistical significance) compared to MS-ILS and better average costs for 86% of the instances (74% with statistical significance) compared to MDM-MS-ILS.

Additional analyses were performed to inspect the behavior of the methods further. Figure 7 evaluates the solution costs obtained throughout one execution of each method. The charts in the first row present, for each method, the solution costs obtained per iteration in the generation and local search phases, whereas the second row provides enlarged views focusing on the local search phase curve. The charts in Fig. 8 exhibit the computational time spent in the generation and local search phases per iteration. The dashed vertical lines indicate the iterations preceding a data mining method run.

In Fig. 7, the charts in the first row show that the reduction in the costs of the solutions generated by MR-MS-ILS after data mining is much more expressive than that observed for MDM-MS-ILS. The enlarged views in the second row show that the costs of generated solutions fall to a level even lower than that of the costs of solutions discovered through the local search in previous iterations, and the local search finds even better solutions after each run of the data mining procedure.

On the other hand, as shown in the last chart of Fig. 8, the computational time spent in the generation phase – which is close to zero in the initial iterations –

Table 7 Summarized results comparing MS-ILS, MDM-MS-ILS, and MR-MS-ILS

		MS-ILS			MDM-MS-ILS			MR-MS-ILS		
		Best Cost	Avg. Cost	Avg. Time	Best Cost	Avg. Cost	Avg. Time	Best Cost	Avg. Cost	Avg. Time
Set 1	Wins/Ties	10	**7**	–	**13**	**7**	–	**13**	6	**14**
	APD				− 0.01%	0.00%	− 14.28%	− 0.01%	0.05%	− 63.72%
Set 2	Wins/Ties	12	3	–	11	8	–	**29**	**26**	**37**
	APD				− 0.01%	− 0.04%	− 13.71%	− 0.15%	− 0.19%	− 65.09%
Set 3	Wins/Ties	1	–	–	1	–	–	**29**	**30**	**30**
	APD				− 0.06%	− 0.01%	− 13.83%	− 0.39%	− 0.46%	− 57.01%
Set 4	Wins/Ties	–	–	–	–	–	–	**11**	**11**	**11**
	APD				− 0.10%	− 0.03%	− 14.34%	− 0.56%	− 0.52%	− 53.24%
Global	Wins/Ties	23	10	–	25	15	–	**82**	**73**	**92**
	APD				− 0.04%	− 0.02%	− 13.91%	− 0.26%	− 0.28%	− 60.83%

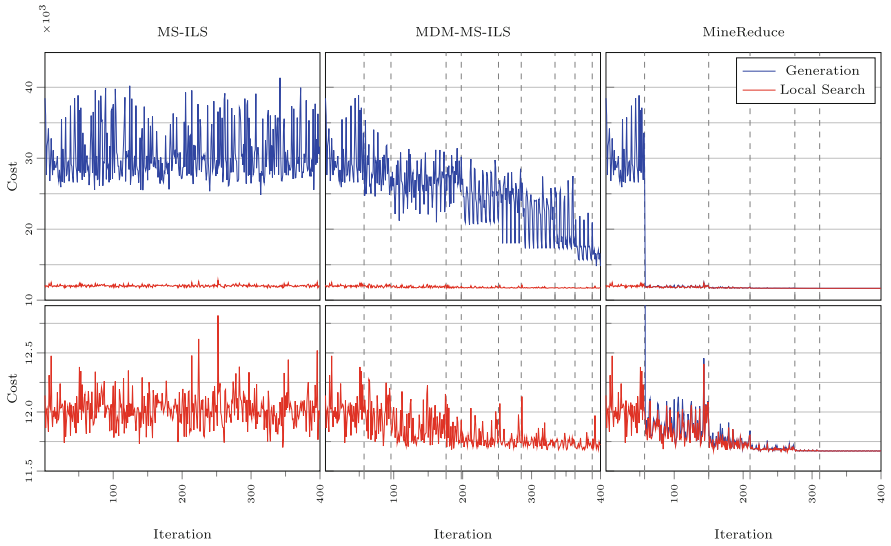


Fig. 7 Cost vs. iteration charts illustrating the behavior of MS-ILS, MDM-MS-ILS, and MR-MS-ILS (MineReduce). (Source: [9, p. 11])

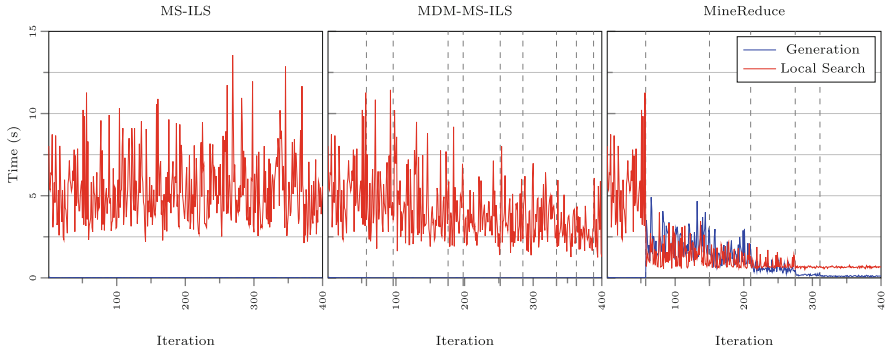


Fig. 8 Time vs. iteration charts illustrating the behavior of MS-ILS, MDM-MS-ILS, and MR-MS-ILS (MineReduce). (Source: [9, p. 11])

increases after the first run of the data mining procedure. This increase is due to the execution of the problem size reduction process, which includes a local search on a reduced version of the problem instance. However, the increase in time spent in the generation phase is compensated by an expressive reduction in time spent in the local search phase. After data mining, the combined time spent in the generation and local search phases per iteration is significantly reduced.

The overall reduction in computational time can be explained by the fact that MineReduce shrinks the problem instance and, consequently, the search space. Therefore, the first local search (embedded in MineReduce's generation phase over

the reduced instance) performs a much smaller number of movement evaluations. Afterward, the second local search (actual local search phase, over the original instance) starts from a higher-quality solution, converging faster.

In MR-MS-ILS, as the charts show, the reduction of solution costs and computational time is intensified after each execution of the data mining procedure, reaching very low levels in the last iterations, which explains the superiority demonstrated by the results of the experiments presented in Table 7.

One last experiment was conducted to assess convergence speed based on a time-to-target (TTT) plot. The target was a solution with a cost lower than or equal to the average cost obtained by MS-ILS in the experiment summarized in Table 7. The obtained chart is shown in Fig. 9.

The chart shows that MR-MS-ILS converges faster than the other methods. It is possible to observe that the probability that the target will be reached within 200 seconds, for example, is approximately 97% for MR-MS-ILS, 76% for MDM-MS-ILS, and 67% for MS-ILS.

The analyses of the charts presented in Figs. 7, 8, and 9 clearly illustrate the influence of the MineReduce approach in the behavior of metaheuristics. As expected, this behavior improves solution quality and convergence speedup.

DM-Supported MS-ILS for the Multisource Capacitated FLP with Customer Incompatibilities

This section summarizes the results obtained by incorporating data mining into a multistart SLS heuristic for the Multisource Capacitated Facility Location Problem with Customer Incompatibilities (MS-CFLP-CI). The MS-CFLP-CI is a recently

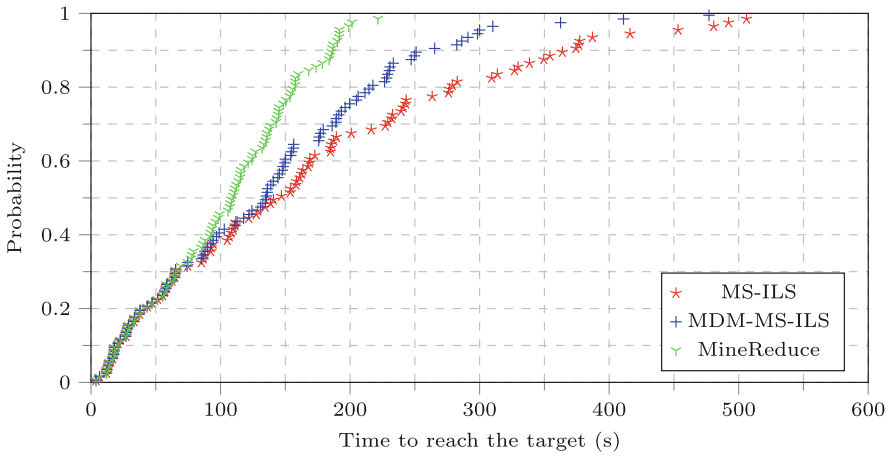


Fig. 9 TTT plot comparing MS-ILS, MDM-MS-ILS, and MR-MS-ILS (MineReduce). (Source: [9, p. 12])

introduced variant of the capacitated facility location problem, proposed in the MESS 2020+1 Metaheuristics Competition (https://www.ants-lab.it/mess2020/#competition). A MineReduced-based multistart ILS (MR-MS-ILS) heuristic was proposed for this problem and ranked first in the competition [11].

Problem Definition

Let $\mathscr{J} = \{1, \ldots, J\}$ be a set of facilities such that each facility $j \in \mathscr{J}$ has a capacity s_j and an opening cost f_j, and let $\mathscr{I} = \{1, \ldots, I\}$ be a set of customers such that each customer i has a demand of quantity of goods d_i to be completely satisfied by one or more facilities. The shipping cost c_{ij} is the cost per unit of transporting goods from facility j to customer i. Finally, a set Γ of pairs of incompatible customers is defined.

The problem consists of defining the number of goods moved from each facility $j \in \mathscr{J}$ to each customer $i \in \mathscr{I}$ to minimize the sum of the cost of opening facilities and the cost to ship the goods from the facilities to the customers. The constraints are the following:

- The total quantity of goods taken from an open facility cannot exceed its capacity;
- The total quantity of goods supplied to a customer must be exactly equal to its demand;
- The same facility cannot supply two incompatible customers.

Data Mining Application

In the proposed method, the mined patterns are frequent sets of pairs $\langle i, j \rangle$, i.e., sets of customer-facility assignments frequent in solutions of the elite set E. Since each decision variable x_{ij} in this problem represents an integer quantity of goods supplied to a customer i by a facility j, the search space reduction is achieved by constraining the decision variables' domains as a consequence of fixing a supplied quantity for each frequent assignment. Hence, after the frequent itemsets are mined, their assignments are filled with the corresponding minimum quantities in E, i.e., for each pair $\langle i, j \rangle$ in a frequent itemset, a supplied quantity q is set, which corresponds to the minimum positive supplied quantity for that pair among all solutions in E. Therefore, each final pattern is a set of assignments $\langle i, j, q \rangle$.

Patterns are used to reduce the size of the problem instance based on the assumption that their elements shall be part of the solution. In the particular case of reducing this problem, an extension to the problem formulation is needed, with the introduction of incompatibilities between customers and facilities, represented by a set Γ', such that for each $\langle i, j \rangle \in \Gamma'$, i cannot be supplied by j. Hence, an additional set of constraints is introduced:

- An incompatible facility cannot supply a customer.

The reduction process works as follows. Given a problem instance $\mathscr{P}$ and a pattern p, the reduced instance $\mathscr{P}'$ is initialized as a copy of $\mathscr{P}$. Then, for each

assignment $\langle i, j, q \rangle$ in p, the following changes are applied to $\mathscr{P}'$: (i) decrease both s_j and d_i by q (a supply of q goods to customer i by facility j is fixed); (ii) set $f_j = 0$, because since facility j is already open, there is no additional cost to use it; and (iii) for each customer i' incompatible with i (from Γ), add a pair $\langle i', j \rangle$ to Γ' (facility j cannot supply any customer that is incompatible with i).

Consequently, the domain of each x_{ij} variable will be reduced since the customer's demands and facilities capacities are reduced, leading to diminished maximum values for x_{ij}. Also, additional constraints will fix the values of many other decision variables, effectively reducing the problem instance size.

Computational Results

Four methods have been compared in [11]: the MineReduce-based Multi-Start ILS (MR-MS-ILS); a Greedy Randomized Adaptive Search Procedure (GRASP); a Permutation Coded Evolutionary Algorithm (PcEA); and a Multistart Greedy (MG) heuristic.

The methods have been tested on artificial instances created for the competition. Instances have been partitioned into two sets, a *training* set with 20 instances, which was publicly available since the competition started so the competitors could use it for tuning their algorithms, and a *validation* set with 10 instances, which was kept hidden during the competition as it was used for defining the final ranking.

The comparisons are based on the best solution found by each method in a given time-out (t). Two different time-outs are used based on the number of facilities J in the problem instance. The first is the one proposed for the competition, which grants $10\sqrt{J}$ seconds for solving each instance, whereas the second grants J seconds per instance.

Tables 8 and 9 summarize the results for the first and the second time-outs, respectively, presenting the numbers of wins (the number of instances for which a method achieved the best results) and the average percentage difference (APD) to PcEA for the training and validation sets separately and for all of them together (global).

Additionally, Table 10 shows the average ranks throughout all runs (10×4) upon all instances for each combination of dataset and time-out, which was the criteria adopted in the competition for ranking the methods. Note that if a method had been better than all others in all runs, it would have obtained ranks $1, 2, \ldots, 10$ for its ten runs, resulting in an average rank of 5.5 (the average of numbers $1, 2, \ldots, 10$).

The results show that MR-MS-ILS was indisputably superior to the other methods. It achieved better solution quality for the vast majority of the instances and far better average ranks.

During the development of the proposed method, experiments comparing MR-MS-ILS to a version of it with the MineReduce-based components disabled (MS-ILS) were conducted to measure the effect of the MineReduce approach – using the training set instances with $t = 10\sqrt{J}$. Table 11 summarizes the results obtained in this comparison, showing that the MineReduce approach significantly improves the baseline method's performance.

Table 8 Summarized results for the MS-CFLP-CI ($t = 10\sqrt{J}$)

		PcEA		GRASP		MG		MR-MS-ILS	
		Best Cost	Avg. Cost	Best Cost	Avg. Cost	Best Cost	Avg. Cost	Best Cost	Avg. Cost
Train. Set	Wins	–	–	3	3	2	2	**15**	**15**
	APD			− 76.79%	−78.09%	− 76.78%	−78.24%	− 80.21%	−81.58%
Valid. Set	Wins	–	–	1	1	3	3	**6**	**6**
	APD			− 76.77%	−77.64%	− 77.14%	−78.08%	− 80.34%	−81.24%
Global	Wins	–	–	4	4	5	5	**21**	**21**
	APD			− 76.79%	−77.94%	− 76.90%	−78.19%	− 80.25%	−81.46%

Table 9 Summarized results for the MS-CFLP-CI ($t = J$)

		PcEA		GRASP		MG		MR-MS-ILS	
		Best Cost	Avg. Cost	Best Cost	Avg. Cost	Best Cost	Avg. Cost	Best Cost	Avg. Cost
Train. Set	Wins	–	–	3	3	1	2	**16**	**15**
	APD			− 76.71%	− 77.82%	− 76.56%	− 77.82%	− 80.18%	−81.35%
Valid. Set	Wins	–	–	1	1	2	2	**7**	**7**
	APD			− 76.63%	− 77.69%	− 76.73%	− 77.92%	− 80.36%	−81.37%
Global	Wins	–	–	4	4	3	4	**23**	**22**
	APD			− 76.68%	− 77.78%	− 76.62%	− 77.86%	− 80.24%	−81.35%

Table 10 Average ranks obtained by the methods

	Train. Set		Valid. Set	
	$t = 10\sqrt{J}$	$t = J$	$t = 10\sqrt{J}$	$t = J$
MR-MS-ILS	**8.3**	**7.7**	**9.2**	**8.7**
MG	17.7	18.4	16.4	16.9
GRASP	20.5	20.4	20.9	20.8
PcEA	35.5	35.5	35.5	35.5

Table 11 Summarized results for the MS-CFLP-CI (MR-MS-ILS vs. MS-ILS)

	MS-ILS		MR-MS-ILS	
	Best Cost	Avg. Cost	Best Cost	Avg. Cost
Wins	–	–	**19**	**20**
APD			− 2.42%	− 2.46%

Conclusion

This chapter presents some ideas for incorporating data mining in local search heuristics. Some applications have been described, showing that heuristics can benefit from data mining by obtaining better solutions in shorter computational times.

These results motivate the use of data mining in other metaheuristics, where the patterns can also be used to guide the search. Data mining can also be used in exact algorithms like branch-and-bound, where the values of some variables could be set based on the patterns extracted from previously found solutions.

Analyzing the obtained results, it seems reasonable to expect that incorporating data mining techniques into SLS heuristics can be helpful in general. Wolpert and Macread [18] presented a series of No Free Lunch theorems which establish that for any algorithm, the high performance on a class of problems is offset by the low performance of another class. So, they conclude that all black-box optimization algorithms perform exactly equally well. However, some subsequent studies [3, 6] have shown that these theorems apply to some specific classes of problems and that is expected that real-world problems are not of this particular class. Therefore, it is reasonable to expect that the good results obtained using the data mining techniques in the presented problems may also be achieved in other real-world problems.

Cross-References

- ▸ Evolutionary Algorithms
- ▸ Genetic Algorithms
- ▸ Heuristics in Vehicle Routing
- ▸ Innovative Applications of Metaheuristics to Supervised Machine Learning
- ▸ Iterated Local Search
- ▸ Multi-start Methods
- ▸ Restart Strategies

Competing Interest Declaration The author(s) has no competing interests to declare that are relevant to the content of this manuscript.

References

1. Berthold T (2013) Measuring the impact of primal heuristics. Oper Res Lett 41(6):611–614. https://doi.org/10.1016/j.orl.2013.08.007
2. Duhamel C, Lacomme P, Prodhon C (2010) A GRASPxELS with depth first search split procedure for the HVRP. Technical report, Laboratoire d'Informatique, de Modélisation et d'Optimisation des Systèmes
3. García-Martínez C, Rodriguez FJ, Lozano M (2012) Arbitrary function optimisation with metaheuristics. Soft Comput 16(12):2115–2133. https://doi.org/10.1007/s00500-012-0881-x
4. Han J, Kamber M, Pei J (2012) Data mining: concepts and techniques, 3rd edn. Morgan Kaufmann. https://doi.org/10.1016/C2009-0-61819-5
5. Hoos HH, Stützle T (2005) Stochastic local search: foundations and applications. Elsevier, San Francisco. https://doi.org/10.1016/B978-1-55860-872-6.X5016-1
6. Igel C, Toussaint M (2003) On classes of functions for which no free lunch results hold. Inf Process Lett 86(6):317–321. https://doi.org/10.1016/S0020-0190(03)00222-9
7. de Lima Martins S, Rosseti I, Plastino A (2018) Data mining in stochastic local search. In: Martí R, Pardalos PM, Resende MGC (eds) Handbook of heuristics. Springer International Publishing, Cham, pp 39–87. https://doi.org/10.1007/978-3-319-07124-4_11
8. Maia MRH, Plastino A, Penna PHV (2018) Hybrid data mining heuristics for the heterogeneous fleet vehicle routing problem. RAIRO-Oper Res 52(3):661–690. https://doi.org/10.1051/ro/2017072
9. Maia MRH, Plastino A, Penna PHV (2020) MineReduce: an approach based on data mining for problem size reduction. Comput Oper Res 122:104995. https://doi.org/10.1016/j.cor.2020.104995
10. Maia MRH, Plastino A, Souza US (2024) An improved hybrid genetic search with data mining for the CVRP. Networks 83(4):692–711. https://doi.org/10.1002/net.22213
11. Maia MRH, Reula M, Parreño-Torres C, Vuppuluri PP, Plastino A, Souza US, Ceschia S, Pavone M, Schaerf A (2023) Metaheuristic techniques for the capacitated facility location problem with customer incompatibilities. Soft Comput 27(8):4685–4698. https://doi.org/10.1007/s00500-022-07600-z
12. Penna PHV, Subramanian A, Ochi LS (2013) An iterated local search heuristic for the heterogeneous fleet vehicle routing problem. J Heurist 19(2):201–232. https://doi.org/10.1007/s10732-011-9186-y
13. Sörensen K, Arnold F, Palhazi Cuervo D (2019) A critical analysis of the "improved Clarke and Wright savings algorithm". Int Trans Oper Res 26(1):54–63. https://doi.org/10.1111/itor.12443
14. Talbi EG (2021) Machine learning into metaheuristics: a survey and taxonomy. ACM Comput Surv 54(6). https://doi.org/10.1145/3459664
15. Tan PN, Karpatne A, Steinbach M, Kumar V (2019) Introduction to data mining, 2nd edn. Pearson, Deutschland
16. Vidal T (2022) Hybrid genetic search for the CVRP: open-source implementation and SWAP* neighborhood. Comput Oper Res 140:105643. https://doi.org/10.1016/j.cor.2021.105643
17. Witten IH, Frank E (2011) Data mining: practical machine learning tools and techniques, 3rd edn. Morgan Kaufmann. https://doi.org/10.1016/C2009-0-19715-5
18. Wolpert D, Macready W (1997) No free lunch theorems for optimization. IEEE Trans Evol Comput 1(1):67–82. https://doi.org/10.1109/4235.585893
19. Zaki MJ, Meira Jr W (2014) Data mining and analysis: fundamental concepts and algorithms. Cambridge University Press. https://doi.org/10.1017/CBO9780511810114

Deep Learning in Search Heuristics

4

Nayeli Gast Zepeda, André Hottung, and Kevin Tierney

Contents

Introduction . . . 72
Preliminaries . . . 73
Deep Neural Networks . . . 73
Graph Neural Networks . . . 74
Pointer Networks . . . 75
Transformer Architecture . . . 75
Deep Parameter Control . . . 76
Learning a Control Policy . . . 77
Dynamically Configurable Parameters . . . 78
Representing Search State . . . 78
Deep-Learned Construction and Improvement Policies . . . 79
Construction Methods . . . 79
Improvement Methods . . . 82
Transductive Methods . . . 84
Conclusion and Outlook . . . 85
References . . . 85

Abstract

The integration of deep learning techniques into search heuristics presents a transformative approach to solving combinatorial optimization problems (COPs) and has revitalized interest in using deep neural networks (DNNs) in the field of optimization. This chapter explores application of DNNs in two state-of-the-art learning scenarios, namely controlling parameter values in search techniques and directly controlling decision making in heuristics. Despite challenges such as

N. Gast Zepeda · A. Hottung · K. Tierney (✉)
Bielefeld University, Bielefeld, Germany
e-mail: nayeli.gast@uni-bielefeld.de; andre.hottung@uni-bielefeld.de; kevin.tierney@uni-bielefeld.de

© Springer Nature Switzerland AG 2025

R. Martí et al. (eds.), *Handbook of Heuristics*,
https://doi.org/10.1007/978-3-032-00385-0_64

their black-box nature and resource-intensive training requirements, DNN-based methods are showing significant progress versus traditional Operations Research methods. We present methods and results from recent literature showing the current abilities of these techniques and provide a critical assessment and outlook for future research.

Keywords

Deep neural networks · Neural combinatorial optimization · Search heuristics · Learning to optimize

Introduction

Artificial neural networks have a long history of being applied to heuristics [40, 55] for solving combinatorial optimization problems (COPs), starting with the Hopfield network [25] and elastic nets [15]. While early approaches effectively explored the bounds of what was possible with neural networks, they had a large performance gap to existing Operations Research (OR) methods. Recent advances in deep neural networks (DNNs) have renewed interest in using neural networks for solving COPs, as well as for creating DNN-heuristic hybrids. This has resulted in a wide range of highly competitive heuristic approaches, including providing high-level search guidance [28], dynamic parameter control [1], and learning policies for heuristic solution construction/improvement [34].

DNNs offer several advantages compared to other learning-based techniques in the field of heuristics, such as scoring mechanisms [18, 19], reactive search [6], and data mining techniques [44]. The first and more important advantage of DNNs is that they can perform automatic feature engineering, meaning users can input "raw" data, such as coordinates in a vehicle routing problem, and the network will synthesize this data to make predictions without the need for a data scientist to perform feature engineering, i.e., computing summary statistics about the input data. Second, DNNs allow optimization techniques to employ the GPU in addition to the CPU for solving problems, opening new possibilities for parallel heuristics. However, DNNs also come with some limitations, such as being black box (i.e., non-interpretable) models with many model parameters and associated complexity. Furthermore, training DNNs can be extremely time-consuming, both in terms of computational resources and human resources (finding a good architecture and parameter settings).

We attempt to highlight new, exciting areas of research on using DNNs to solve COPs, whether through hybridization or directly learning policies to solve COPs. Our goal is to offer a starting point for researchers and practitioners new to these areas. This chapter serves not only to highlight the unique advantages of DNNs versus other machine learning methods and strategies that have been integrated into heuristics over the years but also to point out limitations of DNN techniques.

This chapter is organized as follows. First, we provide preliminaries on DNNs, graph neural networks (GNNs), and transformer methods. We then investigate deep parameter control, which harnesses DNNs to dynamically adjust the parameters of a heuristic as the heuristic is solving a problem instance. We then examine deep-learned construction and improvement policies that directly learn to solve COPs using DNNs and GNNs. Finally, we conclude and discuss directions for future work.

Preliminaries

Deep Neural Networks

DNNs are a family of neural networks characterized by their multiple layers of perceptrons, i.e., interconnected nodes. The basic form of a DNN is the deep feedforward network, also called feedforward neural network (FNN), or multilayer perceptron (MLP). It defines a mapping $\boldsymbol{y} = f(\boldsymbol{x}; \boldsymbol{\theta})$, where $\boldsymbol{x}$ are the inputs, $\boldsymbol{y}$ the outputs, and $\boldsymbol{\theta}$ the parameters to be learned to best approximate the underlying function [22]. The concept of neural networks is loosely based on neuroscience, but we emphasize that this is just an inspirational metaphor and nothing more. As shown in Fig. 1, individual perceptrons (shown as nodes) are connected to layers, which are in turn interconnected. In the MLPs there are no feedback connections, i.e., information only flows forward from the inputs through the intermediate computations in the hidden units to the outputs. The error at the output is propagated back to compute gradients that guide weight adjustments in the network, minimizing future errors.

Convolutional neural networks (CNNs) [37] are specialized DNNs designed to process data with grid-like topology, such as images. Unlike MLPs, CNNs use convolutional layers that apply learnable filters across the input, exploiting sparse interactions, parameter sharing, and equivariant representations [22, p. 329 f.]. We note that, currently, CNNs are not commonly used in the optimization tasks

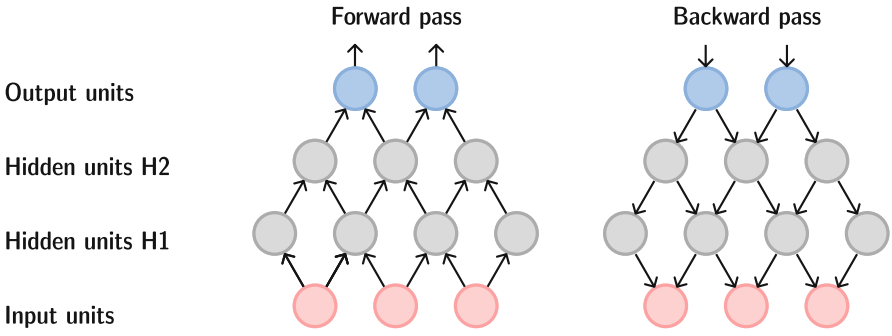


Fig. 1 Multilayer neural network forward and backward pass, simplified from [38]

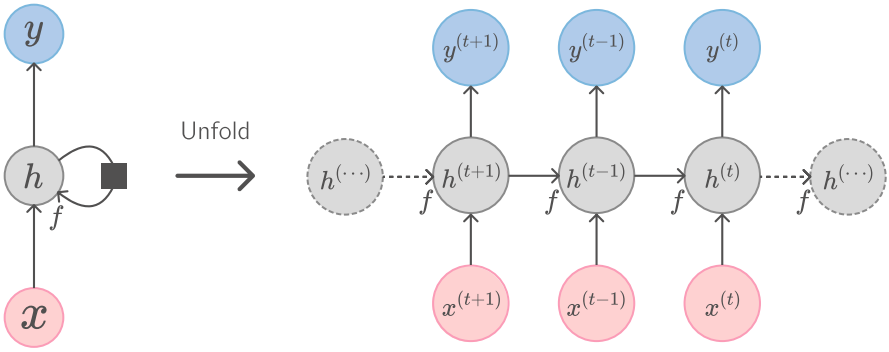


Fig. 2 Unfolding of a basic RNN: every node is now associated with a specific time step t. The square indicates a delay of a single time step. (Adapted from [22])

discussed in this chapter, although they have seen use on related applications, e.g., for algorithm selection portfolios [41].

Recurrent neural networks (RNNs) [46] extend DNNs to handle sequential data by maintaining an internal state (hidden state) that can capture temporal dependencies. At each time step t, an RNN processes input $\boldsymbol{x}^{(t)}$ along with the previous hidden state $\boldsymbol{h}^{(t-1)}$ to produce output $\boldsymbol{y}^{(t)}$ and update its hidden state to $\boldsymbol{h}^{(t)}$ (see Fig. 2). This recurrent connection allows information to persist across time steps. Variants like Long Short-Term Memory (LSTM) networks address the vanishing gradient problem that affects basic RNNs when processing long sequences [24]. The RNN concept is further generalized in Pollack [45] to support a computational graph structured like a deep tree as opposed to the chain-like structure of RNNs depicted in Fig. 2.

Graph Neural Networks

Graph neural networks (GNNs) were first introduced in [23] and later detailed more extensively in [48]. They extend RNNs by handling general graphs including cyclic, directed, and undirected graphs, although earlier approaches were limited to directed acyclic graphs (DAGs). The model also generalizes random walk approaches through learned parameters. Unlike traditional methods that require preprocessing graphs into vectors, GNNs can process graph-structured data directly [23, 48].

GNNs can be applied on both node-focused (e.g., object detection in an image) and graph-focused (e.g., image classification) problems. Both positional graphs, where the ordering of neighboring nodes matters, and non-positional graphs can be handled. The main idea is that individual nodes in the graph represent concepts, and the edges the relationships between them. A node n is associated with a state vector $x_n \in \mathbb{R}^s$ that aggregates information about the node's own labels, the labels of connected edges, and the states and labels of the neighboring nodes. The state vector

is iteratively updated based on these inputs until a stable fixed point is reached, allowing GNNs to naturally capture both local graph structures and global graph properties through the propagation of information across the graph [48].

Pointer Networks

The pointer network architecture (Ptr-Net), introduced by [59], supports variable-sized outputs that depend on the size of the input. For example, a standard MLP as shown before in Fig. 1 has a fixed input and output size that cannot be changed. In the context of combinatorial optimization, this is potentially problematic as it means problem instances of different sizes would require different network architectures. For example, imagine needing one network to solve a traveling salesperson problem (TSP) with 50 nodes and a different one for a TSP with 51 nodes.

Ptr-Nets allow for variable-sized inputs by harnessing the concept of *neural attention*. The attention mechanism was originally introduced in [5] in the context of machine translation. Summarized, attention allows a neural network to place weight on specific combinations of input values. We refer to Vaswani et al. [58] for the mathematical details. Bahdanau et al. [5] identify the use of fixed-sized vectors as a bottleneck to handle variable-sized data, particularly for long sentences, and instead propose to use a *sequence* of vectors and choose a *subset* of those vectors when decoding the output. This way the model can essentially *search* for parts of the input data that are relevant to predicting the target output [5].

Ptr-Nets use this neural attention as a pointer mechanism, i.e., they use attention to produce a softmax probability distribution over the input sequence positions and to select a member of the inputs for the output. This is particularly effective for combinatorial optimization problems where outputs are permutations of inputs, such as the TSP. The model demonstrates good generalization capabilities, often performing well on sequences longer than those seen during training [59].

Transformer Architecture

The transformer was first introduced in [58] and has since been leveraged to achieve new state-of-the-art performances in several fields [11, 14, 33]. The transformer mechanism also relies on attention as in the case of the Ptr-Net to model dependencies between model inputs and outputs. The transformer uses an encoder-decoder structure [57] with multi-head attention, meaning multiple attention mechanisms are stacked and aggregated, where both the encoder and the decoder are composed of "stacked self-attention and point-wise, fully connected layers" [58, p. 2]. The architecture is shown in Fig. 3. The encoder and decoder each are composed of a stack of N identical layers. For the encoder, each layer consists of two sublayers, a multi-head self-attention and a fully connected FNN. In the decoder, every layer has a third sublayer, a multi-head attention layer that processes the outputs of the encoder stack.

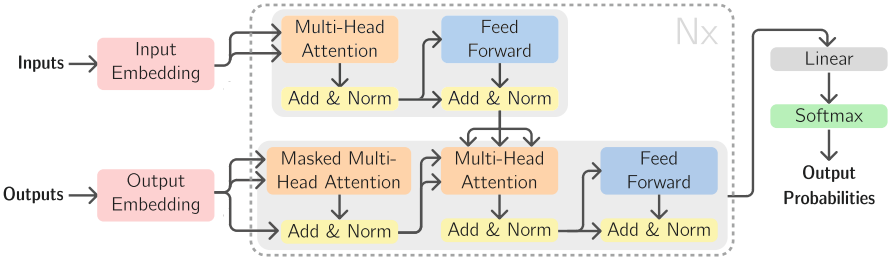


Fig. 3 The transformer architecture based on [58]

Relying on self-attention as opposed to recurrent architectures has three main advantages: (1) The computational complexity per layer is smaller for attention, as long as the sequence length n is smaller than the representation dimensionality d. (2) Computations can be parallelized compared to recurrent architectures, allowing for faster training, and (3) the number of operations needed to forward a signal is constant and thus independent of the path length between long-range dependencies.

Deep Parameter Control

Search heuristics often contain many parameters that directly influence their performance. These parameters have a direct impact on the exploitation/exploration trade-off, for example, the temperature reduction factor of simulated annealing [13] or the list size in tabu search [20]. Configuring these parameters to achieve high-quality solutions or fast runtimes is extremely time-consuming to do by hand. Hence, automated tools have been developed to assist in finding good values for parameters. See, e.g., Schede et al. [49] for an overview of automated algorithm configuration.

The metaheuristics literature has long sought to control parameters during search. Reactive search [6], for example, uses handcrafted strategies to adjust parameters based on the current search trajectory. Recent extensions of reactive search techniques use algorithm configuration tools to learn simple parameter policies offline [3, 4]. The evolutionary algorithms community has also investigated many strategies for controlling parameters during search [2, 39].

Deep learning offers opportunities for the parameter control setting, but also challenges. On the one hand, deep learning can accept a wide range of inputs (e.g., non-tabular inputs such as graphs) and offer powerful insights that simpler models (e.g., [3]) cannot provide. On the other hand, deep learning is computationally expensive and can slow down a solution procedure. Thus, the challenge of applying these methods lies in finding an acceptable balance between the efficiency of a heuristic (in terms of the number of solutions it can examine per second) and the quality of the parameters determined through the deep-learned model.

Learning a Control Policy

There are two main types of parameterized algorithms that can use a learned control policy that can be categorized as either *stepwise* or *non-stepwise* reconfigurable approaches. Stepwise reconfigurable algorithms are formally defined by [1] and involve optimization techniques that adjust their internal parameters at a predefined interval. Non-stepwise reconfigurable techniques, in contrast, update their parameters at undefined intervals, e.g., in the case of Ansótegui et al. [3]. In these approaches, whenever a parameter is used by a search algorithm, its value is queried from a model. The non-stepwise setting can potentially be more computationally efficient than the stepwise setting. However, due to the expense of querying DNNs, we are only aware of stepwise approaches involving these techniques and concentrate on them for the rest of this section.

Biedenkapp et al. [9] propose a general setting for learning control policies called the *dynamic* algorithm configuration (DAC) setting and refine it in Adriaensen et al. [1]. Figure 4 shows a general overview of the stepwise reconfigurable DAC setting for solving multiple instances in which a model is periodically queried for parameters that are inserted into the algorithm. Note that the length of a step is left to an algorithm designer: This could be a single iteration of a heuristic, or it could be an entire local search to improve a solution. The parameters for these models are generally learned in an reinforcement learning (RL) setting in which the reward for the learning is given by the best objective function value found by the algorithm.

The DAC literature has mostly applied standard deep reinforcement learning (DRL) techniques, e.g., double-deep $\mathcal{Q}$ networks [10, 53, 56], REINFORCE [21], and PPO [60]. The DACBench [16] also allows essentially any standard DRL framework to be used, enabling experimentation with multiple learning options. We note that while algorithm configuration techniques have been used to train DAC approaches for the non-stepwise setting without DNNs (e.g., [3]), we are unaware of any approaches combining such learning mechanisms with deep networks. Finally, the literature does not indicate that any of the standard DRL methods are necessarily better than any other, rather the critical ingredients to a DAC approach lie in the selection of parameters to configure and how to represent the search state.

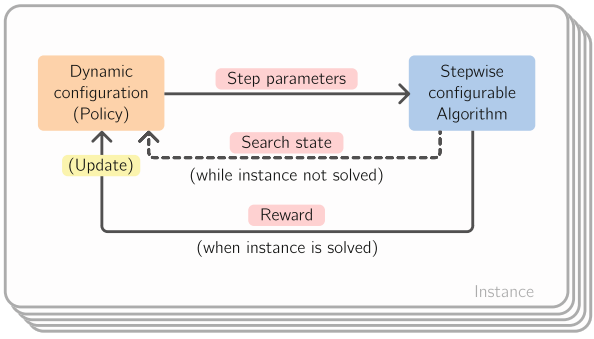


Fig. 4 An overview of DAC. (Adapted from [9])

Dynamically Configurable Parameters

Nearly every parameter that has an impact on the search behavior of a heuristic can be instrumented as a dynamically controlled parameter. Exceptions include the random seed provided to the algorithm and any parameters that, when adjusted, could result in infeasible solutions (e.g., floating point tolerances). The literature provides no clear guidance on how many parameters of an approach should be instrumented to be dynamic, and includes approaches that control as many parameters as possible (e.g., [3, 4]) as well as approaches controlling only a single parameter [43]. Commonly targeted parameters include those providing direct trade-offs between exploration and exploitation [43], step sizes [31, 52], and population adjustment mechanisms in evolutionary algorithms [53].

Parameters can be represented by the outputs of DNNs in two main ways. The network can either directly output the parameter value or output parameters for a distribution. The former is really only possible in the case of continuous parameters. In the latter case, the parameter is either sampled from the distribution or computed through the mode/mean of the resulting distribution. Discrete parameters can be determined through a softmax layer or through a beta distribution; see, e.g., [51]. In the case of softmax, a probability for each of the discrete values is directly generated by the deep learned model. Beta distributions are better suited to discrete parameters with large ranges, as they only require two output parameters (α and β), whereas a softmax requires output parameters for all discrete output values. The continuous value of the beta distribution must be discretized.

Representing Search State

The search state must be summarized in representative features that can be given to the learning approach to decide how to adjust its parameters. Features describing the search state can be partitioned into two sets: problem-independent and problem-dependent features. Problem-independent features can be further subdivided into algorithm-independent and algorithm-dependent features.

Problem-independent state features that are algorithm independent are mostly limited to descriptions of how long the search has been ongoing. A common choice for this is the number of elapsed iterations, e.g., in CMA-ES in the DACBench [16], or remaining budget, as in the SGD benchmark of DACBench. An alternative to iterations/budget is to use the elapsed CPU time such as in the MaxSAT solver from Ansótegui et al. [3]. The best known solution value can also be considered a problem and algorithm-independent feature, e.g., as used in Ma et al. [43]. However, we note that how this value is computed could be algorithm specific. In the case of Ma et al. [43], a global best solution from a population of an evolutionary algorithm is used, whereas in Ansótegui et al. [4], there is no population to select the global best solution. Statistics about the solution quality in a population are also used in Ma et al. [43] and provide a problem-independent view of search progress.

Even in the nascent literature on DAC, there is already a multitude of problem-dependent features. Thus, we provide a short overview of general ideas to inspire future work in this area and do not intend to be comprehensive. For the CMA-ES benchmark in DACBench, the search state is described through the current learning rate, current gradient, and the momentum of the algorithm. In contrast, the fast-downward benchmark of DACBench uses statistics about values found through its various planning heuristics. Historical observations can also be included in the current state, e.g., in Shala et al. [52], which tries to learn the step size of CMA-ES and includes the step size from previous iterations. Overall, the literature in this area so far indicates that while feature engineering on the search state will likely lead to better performance, many approaches do not report investing significant resources in creating/selecting features.

Deep-Learned Construction and Improvement Policies

Deep neural networks are frequently employed to learn *construction* or *improvement* strategies for various combinatorial optimization problems. Construction heuristics build solutions from scratch, starting from an empty solution, while improvement methods iteratively refine an existing solution by making small adjustments. For example, consider the TSP. A construction heuristic for the TSP builds a tour by starting from an initial city and adding one city at a time at each decision point until the tour is complete. In contrast, an improvement heuristic begins with a fully constructed tour and iteratively modifies it—such as by swapping edges—with the goal of enhancing the solution's quality.

In addition to construction and improvement methods, we introduce a third class of algorithms that combine aspects of construction and improvement methods, referred to as *transductive methods*. These methods iteratively build solutions from scratch over multiple iterations, progressively improving the quality of the solutions they produce. This improvement is achieved by updating the model parameters that guide the construction process after each iteration, based on the quality of the generated solutions. Like improvement heuristics, transductive methods seek better solutions through iteration, yet, similar to construction heuristics, they reconstruct the solution from the ground up at each iteration. In the following sections, we explore these three classes of algorithms in greater detail.

Construction Methods

Construction methods generate solutions for a problem instance from scratch. We categorize these methods into two main types: autoregressive and non-autoregressive. Autoregressive methods construct solutions by following a sequential decision-making process, where a DNN is queried at each step to determine the next decision, while taking into account all previous decisions. This process allows autoregressive methods to build solutions progressively, leveraging

each prior step to inform the subsequent one, which is particularly effective in problems with interdependent decisions. In contrast, non-autoregressive methods only query the DNN once. Instead of outputting a complete solution directly, the DNN in non-autoregressive methods produces an intermediate result that must undergo a post-processing step to be transformed into a complete solution. This approach can sometimes offer efficiency advantages since it requires only a single DNN call.

Autoregressive

Autoregressive construction methods excel at generating solutions through a step-by-step decision process and can be trained via RL. The first modern neural network-based autoregressive construction approach is introduced by Vinyals et al. [59] for the TSP. This pioneering method trains the model using supervised learning, establishing a foundation for neural network-based solution generation. Building on this approach, Bello et al. [7] propose training the network via reinforcement learning, showcasing its effectiveness in handling larger and more complex TSP instances without requiring labeled data for training. Later, Kool et al. [34] introduce a novel neural network architecture based on transformer networks. This architecture has been widely adopted, as it has proven capable of learning powerful construction strategies. With slight modifications, this architecture has been applied to generate solutions not only for routing problems, such as the vehicle routing problem (VRP), but also for scheduling problems, including the job shop scheduling problem, and other combinatorial optimization tasks [8]. Recognizing that many combinatorial optimization (CO) problems exhibit symmetries in their solution space, Kwon et al. [36] introduced POMO, a method that leverages these symmetries to enforce diverse solution rollouts, enhancing exploration during training. Building on this, Hottung et al. [29] proposed a novel construction method that trains a single model to learn multiple diverse construction strategies, enabling better search space exploration even in the absence of significant symmetries.

Figure 5 illustrates solution generation for a TSP instance using an encoder-decoder model as a construction method. The process begins with encoding the problem instance, where each node is transformed into a high-dimensional embedding that encapsulates essential features for solution construction. The encoder generates these embeddings, one for each node, capturing key information about the problem that will be used by the decoder to build the solution step by step.

During solution generation, the decoder sequentially constructs the tour. At each step, the decoder receives both the full set of node embeddings and context information specific to the current step—in the case of the TSP, this includes the previously selected node, indicating the current position. The model then outputs a probability distribution over possible next nodes, allowing the choice of the next visit either by sampling from this distribution or by selecting the node with the highest probability (greedy selection).

Autoregressive construction methods are particularly well-suited for training via RL. In contrast to supervised learning, RL approaches do not require reference solutions during training. As such, RL is ideal for discovering solution strategies

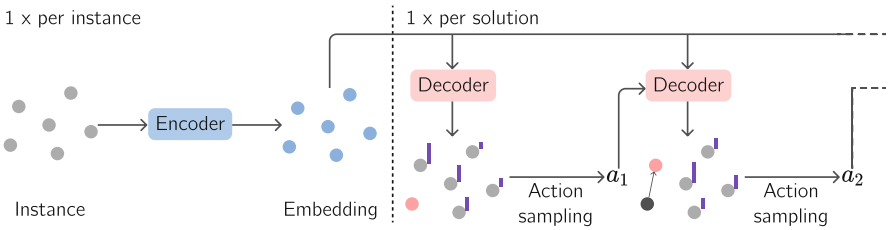


Fig. 5 Solution generation for the TSP with an encoder-decoder model, based on [28]

for novel problems where no existing methods can generate training solutions efficiently. However, this advantage comes with a computational cost: state-of-the-art RL approaches generally require the generation of millions of solutions during training to develop effective strategies. As a result, training often requires significant computational resources and is commonly parallelized on GPUs to meet practical time constraints.

To enable efficient RL training, it is essential to have a highly parallelizable implementation of the problem environment. For example, calculating objective function values should be optimized to run efficiently on a GPU. Additionally, state-of-the-art autoregressive methods limit the model's actions to ensure only feasible solutions are generated. For the TSP, for instance, the model is restricted to selecting nodes that have not yet been visited, a straightforward feasibility constraint that can be easily implemented on the GPU. However, for more complex optimization problems, determining feasible actions may not be as trivial, and difficult to implement using GPU-executable code.

Non-autoregressive

Non-autoregressive approaches generate solutions using a single model call, making them particularly efficient for large problem instances. In routing problems, these methods often use a DNN to produce a heatmap that reflects the likelihood of each edge being part of an optimal solution. This probability-based heatmap is then used in a post-processing step to construct a feasible solution. Figure 6 illustrates this two-step process for the TSP.

The concept of generating a heatmap as an intermediate solution representation was first introduced by Joshi et al. [32], where a graph convolutional model is trained via supervised learning to generate heatmaps based on instance-solution pairs. A beam search algorithm then utilizes this heatmap to assemble complete solutions by systematically selecting edges with the highest probabilities while ensuring feasibility.

Building on this heatmap concept, other methods introduce advanced post-processing techniques to improve solution quality. For instance, Kool et al. [35] employ dynamic programming to translate the generated heatmap into complete, high-quality solutions. Similarly, Ye et al. [61] use the heatmap to guide an ant colony optimization (ACO) algorithm, where the heatmap probabilities influence

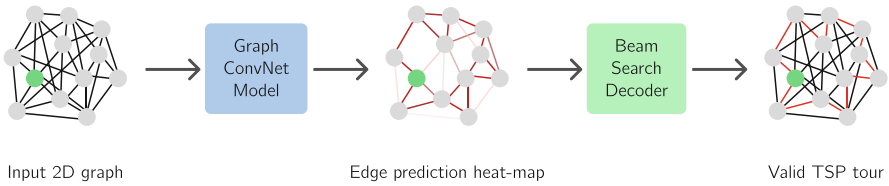


Fig. 6 Non-autoregressive solution generation, based on [32]

the ACO's exploration of edges, effectively combining learned model insights with traditional optimization heuristics.

The primary advantage of non-autoregressive approaches lies in their scalability: The number of model calls remains fixed regardless of the problem's size, making them suitable for larger instances where sequential decision processes could become computationally expensive. However, the effectiveness of these approaches often hinges on the strength of the post-processing strategies employed, as a single model output does not inherently guarantee solution feasibility or quality. Moreover, there is ongoing debate about how effectively the heatmap captures all the nuances needed for high-quality solution generation. While the heatmap successfully guides the post-processing strategies, recent studies suggest that it may not provide a significant advantage over simpler, distance-based measures, thus calling into question the usefulness of heatmap approaches [47].

Improvement Methods

Improvement methods aim to enhance a given initial solution through an iterative process. In each iteration, a (small) portion of the solution is modified, with the goal of producing a new solution that ideally has a better objective function value. These methods are particularly effective for problems where parts of the solution can be modified quickly without significantly affecting the rest of the solution, and where the change in the objective function resulting from the modification can be efficiently computed.

Improvement methods can be broadly categorized into two main types: local search methods and large neighborhood search methods. Local search methods focus on making small changes to a solution, guided by an explicitly defined neighborhood function. In contrast, large neighborhood search (LNS) methods implicitly define a solution's neighborhood and make more substantial modifications to the solution in each iteration. In the following sections, we will explore algorithms from both of these categories in greater detail.

Local Search Methods

We refer to improvement heuristics that only make small changes at each step as *local search* heuristics. Local search heuristics typically define a neighborhood function $N(s)$ that accepts a solution s and returns a set of similar (i.e., neighboring)

solutions to s. Starting with a given solution s, a local search method uses $N(s)$ to generate a set of neighboring solutions at each iteration. One solution s' is picked from $N(s)$ using a defined selection mechanism. For example, a simple selection mechanism is to select the neighboring solution with the lowest cost value. Finally, at the end of each local search iteration, s is set to s'. The search is usually continued until no improving solution in the neighborhood $N(s)$ exists, which means that a *local optimum* has been reached.

Large Neighborhood Search Methods

LNS methods generate new candidate solutions by removing certain components from a given solution and then rebuilding it. This fundamental approach has been widely adopted in various non-learning-based methods, including the rip-up and reroute technique by Dees and Smith [12], the original LNS by Shaw [54], and the ruin-and-recreate method by Schrimpf et al. [50]. In LNS, neighborhood functions are implicitly defined through destroy and repair operators: Destroy operators remove specific components, resulting in an incomplete and infeasible solution, while repair operators rebuild the solution to restore feasibility. Destroy operators typically remove a significant portion of the solution, enabling substantial modifications in a single iteration.

Thanks to its effectiveness and ease of use, the LNS framework has been widely adapted for learning-based methods addressing routing problems. Neural large neighborhood search (NLNS) methods, for instance, often replace the repair operators of LNS with a neural policy trained to learn the repair process. Repairing a destroyed solution closely resembles constructing a solution from scratch, making it possible to use similar, or even identical, learning setups and model architectures as those used in construction algorithms. Consequently, NLNS offers a straightforward way to extend a fast construction method into an approach that performs a more exhaustive search.

Neural large neighborhood search was first introduced by [26], where a neural network was trained to learn a repair policy for the capacitated vehicle routing problem (CVRP). This concept of learning a repair operator has since been applied to other problems, such as the capacitated team orienteering problem [27] and the vehicle routing problem with time windows [17]. While these early methods relied on reinforcement learning to train the repair policy, recent approaches have incorporated the LNS framework to enhance the performance of decoder-only models trained in a supervised manner. For example, [42] propose a method called random reconstruct, where a random subset of components from a given solution is removed and then restructured to create a new candidate solution. Their approach enables a more comprehensive search capability for their original construction method, leading to significant performance improvements. More recently, Hottung et al. [30] propose to learn the destroy operator instead of the repair operator via a deep neural network. They combine the learned destroy operator with a fast, handcrafted greedy insertion algorithm at test time. This approach demonstrates strong performance, matching state-of-the-art operations research methods across three variants of the vehicle routing problem.

Transductive Methods

Transductive methods construct solutions iteratively from scratch, refining model parameters after each iteration based on observed performance. In other words, transductive methods fine-tune a model to a specific instance (or set of instances) at test time. The fine-tuning process is typically implemented using the same reinforcement learning setup employed during the method's training. In these methods, the fine-tuning process itself acts as the search procedure, allowing the model to explore the solution space and discover high-quality solutions. Figure 7 illustrates the core components of transductive methods. Importantly, the fine-tuned model produced at the end of the search phase is not intended for reuse; rather, its purpose is solely to yield a high-quality solution for the current instance.

One major advantage of transductive search is its ease of implementation for many construction methods. Because the reinforcement learning framework and setup used during the search phase closely mirror those used during training, the method requires fewer additional components compared to other approaches. This alignment between training and test-time procedures simplifies the overall development.

The first transductive method, known as *active search*, is introduced by Bello et al. [7]. In active search, all model parameters are adjusted during the search phase, enabling a comprehensive exploration of the solution space. However, this approach requires substantial memory resources when solving multiple instances in parallel, as it updates every parameter independently for each instance. Hottung et al. [28] propose an enhanced variant called *efficient active search*, which updates only a subset of model parameters during the search, keeping the rest fixed. This selective update strategy reduces memory consumption significantly by allowing shared parameters across instances, resulting in a more stable search process. Additionally, efficient active search incorporates a modified loss function that directs the model toward the best solution encountered so far. Overall, efficient active search has demonstrated considerably improved performance and memory efficiency compared to the original active search approach.

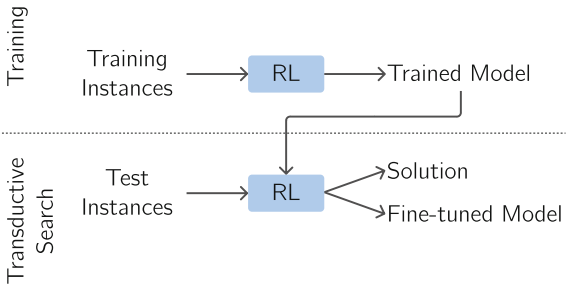


Fig. 7 Core components of transductive methods, e.g., active search [7]

Conclusion and Outlook

We have described several settings in which DNNs have been making an impact for solving OR problems. DNNs are not only capable of dynamically controlling the parameters of optimization approaches, but they can also generate solutions to a wide range of optimization problems in a multitude of ways. The approaches we present are not all competitive with state-of-the-art "traditional" OR alternatives, but the performance gap is rapidly narrowing on many applications, even in well-studied areas like vehicle routing.

We expect state-of-the-art OR methods to increasingly use DNNs in some part of their solution methodology. Hybrid methods in particular pose a promising way forward, in which the "intelligence" (high-quality heuristic abilities) of a DNN is used in conjunction with the speed of traditional OR techniques. Hybrids between the different areas of DNN usage listed in this chapter are also conceivable, e.g., solution approaches using DNNs to construct solutions with dynamically controlled parameters. Furthermore, for approaches requiring low domain knowledge (e.g., POMO), challenges remain in extending these techniques to real-world problems, especially those in which masking does not prevent infeasibility and those with complex side constraints that are not easily coded into a compact solution representation.

Acknowledgments Nayeli Gast Zepeda and André Hottung were supported by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) under Grant No. 521243122.

Competing Interest Declaration The author(s) has no competing interests to declare that are relevant to the content of this manuscript.

References

1. Adriaensen S, Biedenkapp A, Shala G, Awad N, Eimer T, Lindauer M, Hutter F (2022) Automated dynamic algorithm configuration. J Artif Intel Res 75:1633–1699
2. Aleti A, Moser I (2016) A systematic literature review of adaptive parameter control methods for evolutionary algorithms. ACM Comput Surv (CSUR) 49(3):1–35
3. Ansótegui C, Pon J, Sellmann M, Tierney K (2017) Reactive dialectic search portfolios for MaxSAT. In: AAAI, pp 765–772
4. Ansótegui C, Heymann B, Pon J, Sellmann M, Tierney K (2018) Hyper-reactive tabu search for MaxSAT. In: International conference on learning and intelligent optimization. Springer, pp 309–325
5. Bahdanau D, Cho K, Bengio Y (2014) Neural machine translation by jointly learning to align and translate. CoRR abs/1409.0473
6. Battiti R, Brunato M, Mascia F (2008) Reactive search and intelligent optimization, vol 45. Springer Science & Business Media, Springer New York
7. Bello I, Pham H, Le QV, Norouzi M, Bengio S (2016) Neural combinatorial optimization with reinforcement learning. ArXiv abs/1611.0
8. Berto F, Hua C, Park J, Luttmann L, Ma Y, Bu F, Wang J, Ye H, Kim M, Choi S, Zepeda NG, Hottung A, Zhou J, Bi J, Hu Y, Liu F, Kim H, Son J, Kim H, Angioni D, Kool W, Cao Z, Zhang

Q, Kim J, Zhang J, Shin K, Wu C, Ahn S, Song G, Kwon C, Tierney K, Xie L, Park J (2024) RL4CO: an extensive reinforcement learning for combinatorial optimization benchmark. arXiv preprint arXiv:230617100
9. Biedenkapp A, Bozkurt HF, Eimer T, Hutter F, Lindauer M (2020) Dynamic algorithm configuration: foundation of a new meta-algorithmic framework. In: ECAI 2020. IOS Press, pp 427–434
10. Biedenkapp A, Dang N, Krejca MS, Hutter F, Doerr C (2022) Theory-inspired parameter control benchmarks for dynamic algorithm configuration. In: Proceedings of the genetic and evolutionary computation conference, pp 766–775
11. Brown T, Mann B, Ryder N, Subbiah M, Kaplan JD, Dhariwal P, Neelakantan A, Shyam P, Sastry G, Askell A, Agarwal S, Herbert-Voss A, Krueger G, Henighan T, Child R, Ramesh A, Ziegler D, Wu J, Winter C, Hesse C, Chen M, Sigler E, Litwin M, Gray S, Chess B, Clark J, Berner C, McCandlish S, Radford A, Sutskever I, Amodei D (2020) Language models are few-shot learners. In: Larochelle H, Ranzato M, Hadsell R, Balcan M, Lin H (eds) Advances in neural information processing systems, vol 33. Curran Associates, Inc., Red Hook, pp 1877–1901
12. Dees WA, Smith RJ (1981) Performance of interconnection rip-up and reroute strategies. In: 18th design automation conference. IEEE, pp 382–390
13. Delahaye D, Chaimatanan S, Mongeau M (2019) Simulated annealing: from basics to applications. In: Handbook of metaheuristics. Springer, Cham, pp 1–35
14. Devlin J, Chang MW, Lee K, Toutanova K (2019) BERT: pre-training of deep bidirectional transformers for language understanding. In: Burstein J, Doran C, Solorio T (eds) Proceedings of the 2019 conference of the North American. Chapter of the Association for computational linguistics: human language technologies, Volume 1 (Long and Short Papers). Association for Computational Linguistics, Minneapolis, pp 4171–4186
15. Durbin R, Willshaw D (1987) An analogue approach to the travelling salesman problem using an elastic net method. Nature 326(6114):689–691
16. Eimer T, Biedenkapp A, Reimer M, Adriaensen S, Hutter F, Lindauer M (2021) DACBench: a benchmark library for dynamic algorithm configuration. In: Proceedings of the thirtieth international joint conference on artificial intelligence (IJCAI'21)
17. Falkner JK, Thyssens D, Schmidt-Thieme L (2022) Large neighborhood search based on neural construction heuristics. arXiv preprint arXiv:220500772
18. Glover F (1986) Future paths for integer programming and links to artificial intelligence. Comput Oper Res 13(5):533–549
19. Glover F, Greenberg HJ (1989) New approaches for heuristic search: a bilateral linkage with artificial intelligence. Eur J Oper Res 39(2):119–130
20. Glover FW, Kochenberger GA (2003) Handbook of metaheuristics, vol 57. Springer Science & Business Media, Springer New York
21. Gomoluch P, Alrajeh D, Russo A (2019) Learning classical planning strategies with policy gradient. In: Proceedings of the international conference on automated planning and scheduling, vol 29, pp 637–645
22. Goodfellow I, Bengio Y, Courville A (2016) Deep learning. MIT Press, http://www.deeplearningbook.org
23. Gori M, Monfardini G, Scarselli F (2005) A new model for learning in graph domains. In: Proceedings. 2005 IEEE international joint conference on neural networks, 2005, vol 2, pp 729–734
24. Hochreiter S, Schmidhuber J (1997) Long short-term memory. Neural Comput 9(8):1735–1780
25. Hopfield JJ (1982) Neural networks and physical systems with emergent collective computational abilities. Proc Natl Acad Sci U S A 79(8):2554–2558
26. Hottung A, Tierney K (2020) Neural large neighborhood search for the capacitated vehicle routing problem. In: Proceedings of the 24th European conference on artificial intelligence (ECAI-20)
27. Hottung A, Tierney K (2022) Neural large neighborhood search for routing problems. Artif Intell 313:103786

28. Hottung A, Kwon YD, Tierney K (2022) Efficient active search for combinatorial optimization problems. In: International conference on learning representations
29. Hottung A, Mahajan M, Tierney K (2024) Polynet: learning diverse solution strategies for neural combinatorial optimization. arXiv preprint arXiv:240214048
30. Hottung A, Wong-Chung P, Tierney K (2025) Neural deconstruction search for vehicle routing problems. arXiv preprint arXiv:250103715
31. Ichnowski J, Jain P, Stellato B, Banjac G, Luo M, Borrelli F, Gonzalez JE, Stoica I, Goldberg K (2021) Accelerating quadratic optimization with reinforcement learning. Adv Neural Inf Process Syst 34:21043–21055
32. Joshi CK, Laurent T, Bresson X (2019) An efficient graph convolutional network technique for the travelling salesman problem. arXiv preprint arXiv:190601227
33. Kolesnikov A, Dosovitskiy A, Weissenborn D, Heigold G, Uszkoreit J, Beyer L, Minderer M, Dehghani M, Houlsby N, Gelly S, Unterthiner T, Zhai X (2021) An image is worth 16x16 words: transformers for image recognition at scale
34. Kool W, van Hoof H, Welling M (2019) Attention, learn to solve routing problems! In: International conference on learning representations
35. Kool W, van Hoof H, Gromicho J, Welling M (2022) Deep policy dynamic programming for vehicle routing problems. In: Integration of constraint programming, artificial intelligence, and operations research
36. Kwon YD, Choo J, Kim B, Yoon I, Gwon Y, Min S (2020) POMO: policy optimization with multiple optima for reinforcement learning. In: Advances in neural information processing systems, vol 33, pp 21188–21198
37. LeCun Y (1989) Generalization and network design strategies. Connections in perspective
38. LeCun Y, Bengio Y, Hinton G (2015) Deep learning. Nature 521(7553):436–444
39. Li P, Hao J, Tang H, Fu X, Zheng Y, Tang K (2024) Bridging evolutionary algorithms and reinforcement learning: a comprehensive survey on hybrid algorithms. ArXiv:2401.11963 [cs]
40. Looi CK (1992) Neural network methods in combinatorial optimization. Comput Oper Res 19(3):191–208
41. Loreggia A, Malitsky Y, Samulowitz H, Saraswat V (2016) Deep learning for algorithm portfolios. In: Proceedings of the AAAI conference on artificial intelligence, vol 30
42. Luo F, Lin X, Liu F, Zhang Q, Wang Z (2023) Neural combinatorial optimization with heavy decoder: toward large scale generalization. Adv Neural Inf Process Syst 36:8845–8864
43. Ma Z, Chen J, Guo H, Ma Y, Gong YJ (2024) Auto-configuring exploration-exploitation tradeoff in evolutionary computation via deep reinforcement learning. In: Proceedings of the genetic and evolutionary computation conference, GECCO'24. Association for Computing Machinery, New York, pp 1497–1505
44. de Oliveira EC, de Lima Martins S, Plastino A, Rosseti I, Silva GC (2023) Data mining in heuristic search. Springer International Publishing, Cham, pp 301–321
45. Pollack JB (1990) Recursive distributed representations. Artif Intel 46(1):77–105
46. Rumelhart DE, Hinton GE, Williams RJ (1986) Learning representations by back-propagating errors. Nature 323(6088):533–536
47. Santana Í, Lodi A, Vidal T (2023) Neural networks for local search and crossover in vehicle routing: a possible overkill? In: International conference on integration of constraint programming, artificial intelligence, and operations research. Springer, Cham, pp 184–199
48. Scarselli F, Gori M, Tsoi AC, Hagenbuchner M, Monfardini G (2009) The graph neural network model. IEEE Trans Neural Netw 20(1):61–80
49. Schede E, Brandt J, Tornede A, Wever M, Bengs V, Hüllermeier E, Tierney K (2022) A survey of methods for automated algorithm configuration. J Artif Intel Res 75:425–487
50. Schrimpf G, Schneider J, Stamm-Wilbrandt H, Dueck G (2000) Record breaking optimization results using the ruin and recreate principle. J Comput Phys 159(2):139–171
51. Seiler MV, Rook J, Heins J, Preuß OL, Bossek J, Trautmann H (2023) Using reinforcement learning for per-instance algorithm configuration on the TSP. In: 2023 IEEE symposium series on computational intelligence (SSCI). IEEE, pp 361–368

52. Shala G, Biedenkapp A, Awad N, Adriaensen S, Lindauer M, Hutter F (2020) Learning step-size adaptation in CMA-ES. In: Parallel problem solving from nature–PPSN XVI: 16th international conference, PPSN 2020, Leiden, 5–9 Sep 2020, Proceedings, Part I 16. Springer, pp 691–706
53. Sharma M, Komninos A, López-Ibáñez M, Kazakov D (2019) Deep reinforcement learning based parameter control in differential evolution. In: Proceedings of the genetic and evolutionary computation conference, pp 709–717
54. Shaw P (1998) Using constraint programming and local search methods to solve vehicle routing problems. In: International conference on principles and practice of constraint programming. Springer, pp 417–431
55. Smith KA (1999) Neural networks for combinatorial optimization: a review of more than a decade of research. Informs J Comput 11(1):15–34
56. Speck D, Biedenkapp A, Hutter F, Mattmüller R, Lindauer M (2021) Learning heuristic selection with dynamic algorithm configuration. In: Proceedings of the international conference on automated planning and scheduling, vol 31, pp 597–605
57. Sutskever I, Vinyals O, Le QV (2014) Sequence to sequence learning with neural networks. In: Ghahramani Z, Welling M, Cortes C, Lawrence N, Weinberger K (eds) Advances in neural information processing systems, vol 27. Curran Associates, Inc., Red Hook,
58. Vaswani A, Shazeer N, Parmar N, Uszkoreit J, Jones L, Gomez AN, Kaiser Ł, Polosukhin I (2017) Attention is all you need. In: Guyon I, Luxburg UV, Bengio S, Wallach H, Fergus R, Vishwanathan S, Garnett R (eds) Advances in neural information processing systems, vol 30. Curran Associates, Inc., Red Hook
59. Vinyals O, Fortunato M, Jaitly N (2015) Pointer networks. In: Cortes C, Lawrence ND, Lee DD, Sugiyama M, Garnett R (eds) Advances in neural information processing systems, vol 28. Curran Associates, Inc., Red Hook pp 2692–2700
60. Xu Z, Dai AM, Kemp J, Metz L (2019) Learning an adaptive learning rate schedule. arXiv preprint arXiv:190909712
61. Ye H, Wang J, Cao Z, Liang H, Li Y (2024) DeepACO: neural-enhanced ant systems for combinatorial optimization. In: Advances in neural information processing systems, vol 36. Curran Associates, Inc., pp 43706–43728

Evolution Strategies

5

Michael Emmerich, Ofer M. Shir, and Hao Wang

Contents

Introduction . . . 90
Classical Evolution Strategies . . . 92
Derandomized Evolution Strategies . . . 98
  First Level of Derandomization . . . 99
  Second Level of Derandomization: CMA-ES . . . 101
Theoretical Results . . . 104
Nonstandard Evolution Strategies . . . 106
  Niching and Multi-population ESs . . . 107
  Noise Handling and Robust Optimization with ESs . . . 108
  Multi-criterion and Constraint-Handling ESs . . . 109
  Mirrored Orthogonal Sampling . . . 110
  Algorithm Configurators and Hyperparameter Optimization . . . 111
Benchmarks and Empirical Study . . . 111
  Experimental Settings . . . 112
  Results . . . 112
Conclusions . . . 117
Available Code Packages . . . 118
Cross-References . . . 118
References . . . 119

M. Emmerich (✉)
University of Jyväskylä, Jyväskylä, Finland
e-mail: michael.t.m.emmerich@jyu.fi

O. M. Shir
Computer Science Department, Tel-Hai College, Upper Galilee, Israel

The Galilee Research Institute – Migal, Upper Galilee, Israel
e-mail: ofersh@telhai.ac.il

H. Wang
Leiden Institute of Advanced Computer Science, Leiden University, Leiden, The Netherlands
e-mail: h.wang@liacs.leidenuniv.nl

© Springer Nature Switzerland AG 2025

R. Martí et al. (eds.), *Handbook of Heuristics*,
https://doi.org/10.1007/978-3-032-00385-0_13

Abstract

Evolution strategies (ESs) are classical variants of evolutionary algorithms which are frequently used to heuristically solve optimization problems, in particular, in continuous domains. In this chapter, a description of classical and contemporary ESs will be provided. The review includes remarks on the history of ESs and how they relate to other evolutionary algorithms. Furthermore, developments of ESs for nonstandard problems and search spaces will also be summarized, including multimodal, multi-criterion, and mixed-integer optimization. Finally, selected variants of ESs are compared on a representative set of continuous benchmark functions, revealing strengths and weaknesses of the different variants.

Keywords

Evolution strategy · Evolutionary algorithms · Self-adaptation · Covariance matrix adaptation · Step size adaptation

Introduction

Evolution strategies (ESs) are a class of metaheuristics for optimization by means of (computer) experiments. They belong to the broader class of evolutionary algorithms and like other heuristics from this class mimic adaptive processes in biological evolution. The search in ESs is characterized by the alternating application of variation and selection operators. Recombination and mutation operators are the variation operators and create new individuals. The selection operator selects individuals from a population based on their corresponding fitness value, which is obtained by computing the objective function value in ESs. The selected individuals form the next population and the process is repeated. A distinguishing feature of ESs as compared to most other evolutionary algorithms is self-adaptive mutation operators, which are capable of adapting the shape of the mutation distribution according to the local topology of the landscape and thereby help ESs achieve maximal progress rates.

Before discussing technical details of the ESs, it will be worthwhile to give a brief outline of their history: The idea of mimicking evolution in order to optimize technical systems arose in Germany, where it led to the development of ESs by Ingo Rechenberg [69] and Hans-Paul Schwefel [79], and in the USA, where it led to the development of genetic algorithms [34, 50] and evolutionary programming [31]. In the mid-1990s, these strategies were unified under the common umbrella of *evolutionary algorithms* (EAs) [8]. While all these heuristics share the idea to mimic evolution in computational algorithms, researchers in genetic algorithms and ESs emphasized on different aspects of algorithm design and problem domains.

Since their invention in the 1960s by Rechenberg and Schwefel at the Technical University of Berlin, ESs have been used to optimize real-world systems, typically in engineering design. The first application of an evolution strategy was the design of optimal shapes in engineering such as nozzles and wing shape using physical

experiments. Often the evolutionary design procedure discovered high-performing structures with surprising shapes that have never been considered by engineers before [10]. Starting from sequential stochastic hill climbing strategies, soon ESs advanced to more sophisticated problem solvers, and their main application domain became the treatment of black-box continuous optimization problems on the basis of computer models.

One important development introduced adaptive step sizes or mutation distributions. Although there were some precursors to the idea of step-size adaptation in stochastic search algorithms [78], the development of flexible and efficient adaptation schemes for mutation distributions became a major point of attention in ES research. This feature distinguished them from genetic algorithms which worked commonly with constant mutation strengths.

Different variants of the adaptation of the mutation parameters were developed, and the three mainstream variants are the control of a single step size by means of a so-called $1/5$th success rule, the mutative self-adaptation which most closely resembles natural evolution [20], and the derandomized self-adaptation schemes by Hansen, Ostermeier, and Gawelczyk [42]. Their efforts to find efficient ways to control the shape of the mutation distribution culminated in the covariance matrix adaptation evolution strategy (CMA-ES) [41]. Due to its invariance properties, it is seen today by many researchers as the state-of-the-art evolution strategy when it comes to practically solving ill-conditioned optimization problems (cf. [45]).

A parallel development was the introduction of population-based (or multimembered) ESs. Here, the idea is to create an evolutionary algorithm that performs, as Hans-Paul Schwefel called it, a *collective hill climbing* [80]. To categorize different population models, the notation of (μ, λ)- and $(\mu + \lambda)$-schemes was introduced, in which μ denotes the number of individuals in the parent population and λ the number of individuals in the offspring population. These multimembered strategies are able to exploit the positive effects of recombination (crossover) and are more reliable in global optimization settings and on noisy problems than the early single-membered variants. Moreover, population-based algorithms could be executed in parallel and later could more easily be extended to advanced ESs for solving multi-objective and multimodal optimization tasks.

Nowadays, ESs are mainly used for the simulation-based optimization, i.e., computerized models that require parametric optimization. Evolution strategies are suitable for the optimization of non-smooth functions because they do not require derivatives. In general, the algorithmic paradigms of ESs can be extended to general metric search spaces, and mainstream variants of ESs address continuous optimization problems, as opposed to genetic algorithms [34], which are more typically used on binary search spaces.

Contemporary ESs have shown to be competitive to other derivative-free optimization algorithms, both in theoretical studies [18, 88] and by benchmark comparisons on a large corpus of empirical problems [81]. Furthermore, their practical utility is underpinned by a large number of successful applications in engineering and systems optimization [11].

This chapter will give a brief introduction to classical and contemporary ESs with a focus on mainstream variants of ESs. Firstly, classical $(\mu, \lambda)-$ and $(\mu + \lambda)$

ESs will be described in section "Classical Evolution Strategies." Then, in section "Derandomized Evolution Strategies," derandomization techniques will be discussed, which is the distinguishing feature in the CMA-ES. Section "Theoretical Results" addresses theoretical findings on the convergence and reliability of ESs. An overview on new developments and nonstandard ESs is provided in section "Nonstandard Evolution Strategies." Moreover, this section covers adaptations of ESs that make them more suitable for multimodal optimization. Section "Benchmarks and Empirical Study" discusses empirical benchmarks used in this field and includes a comparative study of contemporary ESs. Section "Conclusions" summarizes the main characteristics of ESs and highlights future research directions.

Classical Evolution Strategies

The typical application field of ESs is (continuous) unconstrained optimization where the optimization problem is given by

$$f(\vec{x}) \to \min, \quad \vec{x} \in \mathbb{R}^d \tag{1}$$

In the context of ESs, the function f can be a black-box function, and usually function f is assumed to be nonlinear and have a minimum. Maximization problems can be brought into the standard form of Eq. 1 by simply flipping the sign of f. Standard implementations also allow to restrict the domain of decision variables to interval domains and to introduce constraints. For the sake of brevity, the extensions of ESs for constraint handling will be widely omitted in the following discussion.

Evolution strategies can be viewed as stochastic processes on a population of individuals from the space of individuals $\mathbb{I}$. An individual in ESs typically comprises the following information:

- A d-tuple of values for the decision variables $x_1, \ldots, x_d$, representing a candidate solution for the optimization problem (see Eq. 1).
- A tuple of strategy parameters. The strategy parameters can, for instance, be the standard deviations used to generate the perturbations of variables in the mutation (step sizes) or the components of a covariance matrix used in the mutation. Strategy parameters can be adapted during evolution.
- A fitness value. The fitness value is typically based on the objective function value, and it may be altered by a penalty for constraint violations.

In some variants of ESs, the so-called (μ, κ, λ)-ESs, the individual's age is also maintained, that is, the number of generations that an individual has survived.

Another basic data structure of an evolution strategy is a population. A population is a multi-set of individuals, i.e., of elements of $\mathbb{I}$. One distinguishes between the parent populations P_t consisting of μ individuals and the offspring populations consisting of λ individuals.

The basic algorithm of a (μ, λ)-evolution strategy and a $(\mu + \lambda)$-evolution strategy is outlined in Algorithm 1. The algorithm starts with initializing a population P_0 of μ parent individuals, for instance, by uniform random sampling in the feasible intervals for the objective variables $\vec{x}$. Then, the fitness values of P_0 are determined, and the best solution found in P_0 is identified and stored in the variables $\vec{x}_0^{\text{best}}$, f_0^{best}.

Then, the following *generational loop* is executed until a termination criterion is met. Common termination criteria are stagnation of the search process or the excess of a maximally allowed duration for searching. The search process in the generational loop is governed by two (stochastic) variation operators, *recombination* and *mutation*, and a deterministic *selection* operator. The recombination operator, namely $\texttt{Recombine} : \mathbb{I}^{\mu} \times \Omega \rightarrow \mathbb{I}^{\lambda}$, generates from the μ individuals in P_t an offspring population of λ individuals which are then mutated by the mutation operator, $\texttt{Mutate} : \mathbb{I}^{\mu} \times \Omega \rightarrow \mathbb{I}^{\mu}$. The mutated individuals are evaluated, and, if necessary, the best found solution $\vec{x}_t^{\text{best}}$, f_t^{best} gets updated. Then the parent population of the next round is determined by selecting the μ best solutions from:

- In case of a (μ, λ)-ES the λ offspring individuals
- In case of a $(\mu+\lambda)$-ES the μ parents of the current generation and the λ offspring individuals, and
- In case of a (μ, κ, λ)-ES from the μ parents who have not exceeded an age of κ and the λ offspring individuals

Finally, the generation counter is increased, and the loop is continued or terminated (if the termination condition is met). After completing the loop, the best attained solution constitutes the output.

It appears to be a "*chicken-and-egg*" dilemma whether it makes more sense to start the evolution strategy with the generation of a parent population (with μ individuals), as suggested in Algorithm 1, or to do so at some other stage of the evolution, i.e., starting with an offspring population. The chosen representation has the advantage that the process that generates the subsequent parent populations $P_0, P_1, P_2, \ldots$ can be viewed as a memoryless stochastic process or more precisely a Markov process:

$$P_{t+1} = \texttt{Select}\left(\texttt{Mutate}\left(\texttt{Recombine}\left(P_t\right)\right), P_t\right) \tag{2}$$

This means that given P_t for some $t \geq 1$, the information of P_{t-1} is irrelevant in order to determine P_{t+1}; alternatively, in the terminology of stochastic processes, the state of P_{t+1} is conditionally independent of the state of P_{t-1} given P_t. This so-called Markov property makes it easier to analyze the behavior of the evolution strategy. In addition, P_t can be viewed as a checkpoint of the algorithm, and if the process stops, e.g., because of a computer crash, the process may resume by starting the loop with the last saved state of P_t.

The main loop of the evolution strategy is inspired by the principles of evolutionary adaptation in nature that were discovered in parallel by the naturalists Alfred Russel Wallace (1823–1913) and Charles Darwin (1809–1882). In brief, a popula-

tion of individuals adapt to their environment by (random) variation and selection. The reason for variability in the population was unknown to these researchers. Only much later in the so-called *modern synthesis*, it was linked to the mutation and recombination of genes. The ES presented in Algorithm 1 does however by far not provide a complete model for evolution in nature. In fact, important driving forces of natural evolutionary processes such as the development of temporally stable species and coevolution cannot be modeled with this basic evolution strategy. On the other hand, by mimicking only the variation and selection process, one can already achieve a potent and robust optimization heuristic, and the theoretical analysis of ESs can provide new insights in the dynamics of natural evolution.

Algorithm 1: Evolution strategy

input: initial population P_0 with μ evaluated individuals
$t \leftarrow 0$ {Generation counter}
$(\vec{x}_1^{\text{best}}, f_1^{\text{best}}) = \texttt{Update}(P_0)$ {Updates best found solution to best solution found in P_0.}
while termination criterion not fulfilled **do**
  $t \leftarrow t + 1$
  $R_t \leftarrow \texttt{Recombine}(P_{t-1})$ {Creates λ offspring from P_t using recombination operator.}
  $M_t \leftarrow \texttt{Mutate}(R_t)$ {Creates a mutant for each individual in R_t.}
  $C_t \leftarrow \texttt{Evaluate}_f(M_t)$ {Evaluates fitness function values.}
  $(\vec{x}_{t+1}^{\text{best}}, f_{t+1}^{\text{best}}) = \texttt{Update}(C_t, \vec{x}_t^{\text{best}}, f_t^{\text{best}})$ {Updates best found solution.}
  $P_t \leftarrow \texttt{Select}(C_t, P_{t-1})$ {Selects the μ best individuals from $C_t \cup P_{t-1}$ in case of a $(\mu+\lambda)$-selection, and from C_t in case of a (μ, λ)-selection.}
end while
return $\vec{x}^{\text{best}}, f^{\text{best}}$

There are many options to instantiate the operators of an evolution strategy, and in the literature, a certain terminology is used to refer to standard choices. Next, the most common instantiations of operators will be discussed by following the structure of Algorithm 1. The first step is the *initialization* of P_0, where the starting population is set by the user, since this allows to resume the evolution from a checkpoint. However, it is also very common to view the initialization as an integral part of the evolution strategy. Initialization procedures vary, while common choices are either constant initialization by generating μ copies of a starting (seed) point, or random initialization, i.e., to initialize the decision variables randomly within their bounds. The initialization of strategy parameters can have a significant impact on the transient behavior of an evolution strategy. In case of step-size parameters, it is often recommended to set these to 5% of the search space size.

A more complex operator is the recombination operator. In the nomenclature, the number of individuals who participate in the creation of a single vector is called ρ. The notation $(\mu/\rho, \lambda)$-ES and, respectively, $(\mu/\rho + \lambda)$-ES makes this number explicit. The ρ individuals who participate in the recombination are drawn by independent uniformly random choices from the population.

Given ρ individuals, there are two common strategies to create an offspring vector—*intermediate* recombination and *dominant* recombination. The vector to be

determined is commonly the vector of decision variables $\vec{x}$, but it can also include the vector of strategy parameters:

- *Intermediate recombination* determines the offspring by averaging the components of the parent individual. It can be applied on the object parameters and on the strategy parameters. Given a ρ-tuple of parent vectors $(\vec{q}^{(1)}, \ldots, \vec{q}^{(\rho)}) \in (\mathbb{R}^d)^\rho$, it computes the resulting vector $\vec{r}$ by means of $r_j = \frac{1}{\rho}(\sum_{i=1}^{\rho} q_j^{(i)})$ for $i = 1, \ldots, d$.
- *Discrete (or dominant) recombination* sets the ith position of the offspring vector $\vec{r}$ randomly to one of values of the parents. By drawing d uniform random numbers u_j, $j = 1, \ldots, d$, from the set $\{1, \ldots, \rho\}$, the offspring individual is set to $r_j = q_{u_j}$.

The terminology, *intermediate* and *dominant*, is lent from the theory of inheritance of biological traits by the botanist Gregor Mendel (1822–1884).

The *mutation* operator is seen as a main driving force of the evolutionary progress in ESs. Mutation adds a small random perturbation to each component of $\vec{x}$. The scaling of this perturbation is based on the strategy variables. A common case is to use the individual so-called step-size parameters $\sigma_1 \in \mathbb{R}^+, \ldots, \sigma_d \in \mathbb{R}^+$. The mutation then performs as indicated in the following equation:

$$x_i' \leftarrow x_i + \sigma_i \cdot \mathcal{N}(0, 1), \quad i = 1, \ldots, d \tag{3}$$

Here, $\mathcal{N}(0, 1)$ denotes the generation of a standard normal distributed random number. The resulting distributions of random numbers have a standard deviation of σ_i, why the σ-variables are also termed as standard deviations of the mutation.

A distinguishing feature of ESs is the *self-adaptation* of the mutation's parameters. In case of d step sizes, the mutative self-adaptation lets the step sizes themselves undergo an evolutionary process. In continuous optimization, the parameters of the multivariate Gaussian distribution can be adapted. Three levels of adaptation can be devised: Firstly, it is possible to control only a single standard deviation that is used for all decision variables (possibly with a constant scaling factor). This is called isotropic self-adaptation. Then, in the so-called individual step-size adaptation for each decision variable, a different standard deviation for the mutation is maintained and adapted. Finally, it is also possible to learn the full covariance matrix of the multivariate Gaussian distribution that is used in the mutation. The different levels of mutation distribution adaptation are indicated in Fig. 1. As a rule of thumb, it can be stated that the more mutation parameters are to be adapted the longer it takes to reach an optimal convergence behavior for a given model.

There are different strategies for controlling or adapting the mutation parameters:

- Step-size control based on the success rate. It has been shown that for the $(1 + 1)$-ES and on two important benchmark functions—sum of squares and

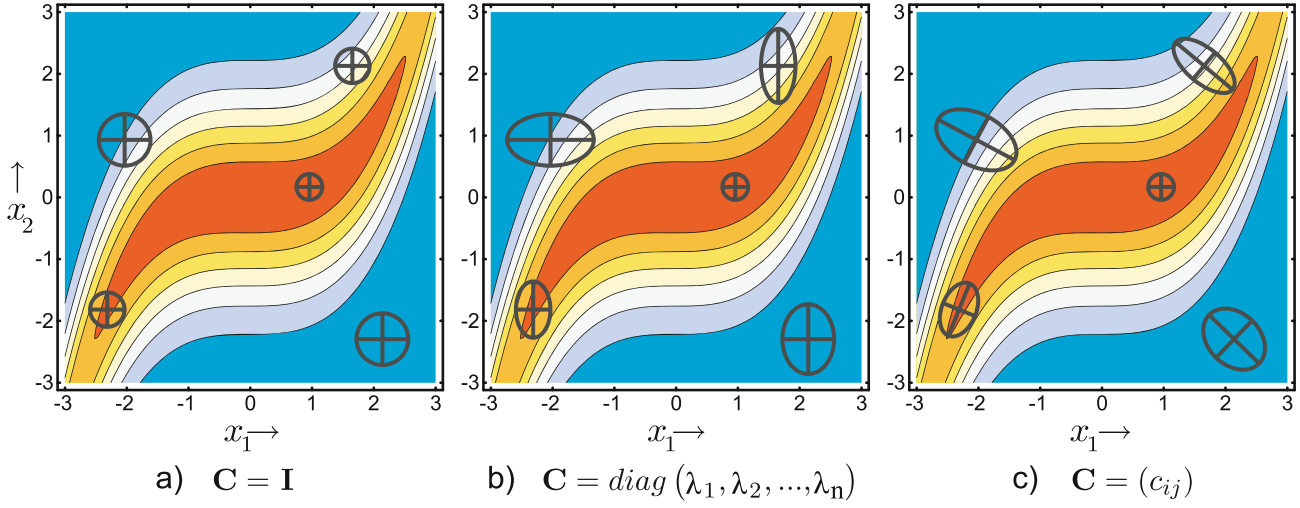


Fig. 1 Three levels of step-size control: isotropic (**a**), individual step sizes (**b**), and covariance matrix adaptation (**c**)

corridor model—among all isotropic Gaussian distributions, the optimal standard deviation is obtained at a step size that yields approximately a success probability of 1/5. Because the success probability can be assessed during execution, this allows for an effective step-size control of the $(1 + 1)$-ES.
- Mutative step-size control: The idea is to make the parameters of the mutation distribution part of the individual and let it undergo an evolutionary process itself. Details of this strategy will be elaborated in this section.
- Derandomized step-size control: Here a more efficient adaptation of the mutation distribution is derived based on cumulative information from previous successful mutation steps. Derandomized self-adaptation uses arithmetic procedures that can no longer be considered as biomimetic. For the price of losing flexibility and simplicity, they gain efficiency in particular for unconstrained continuous optimization and allow practicable schemes for adapting a full covariance matrix of a mutation distribution. The history and details of derandomized ESs will be elaborated on in the next section.

The classical self-adaptive mutation in ESs is nowadays called mutative step-size control. For adapting the standard deviations of the mutation distribution, it augments the individual by a step-size vector and mutates the standard deviations of the mutation before it mutates the decision variables using these standard deviations.

$$N_{\text{global}} \leftarrow \mathcal{N}(0, 1) \tag{4}$$

$$\sigma_i' \leftarrow \sigma_i \cdot \exp(\tau_{\text{local}}\mathcal{N}(0, 1) + \tau_{\text{global}}N_{\text{global}}), \quad i = 1, \ldots, d \tag{5}$$

$$x_i' \leftarrow x_i + \sigma_i \cdot \mathcal{N}(0, 1), \quad i = 1, \ldots, d \tag{6}$$

The parameters τ_{local} and τ_{global} are called local and global learning rates. The simplest form of the mutative step-size control exploits only a single step-size σ for all the coordinates. In this case, Eqs. 5 and 6 simplify to

$$\sigma' \leftarrow \sigma \cdot \exp(\tau_{\text{global}}N_{\text{global}}), \quad i = 1, \ldots, d \tag{7}$$

$$x_i' \leftarrow x_i + \sigma' \cdot \mathcal{N}(0, 1), \quad i = 1, \ldots, d \tag{8}$$

The utilization of mutative self-adaptation is valid when there are more offspring individuals than parent individuals. In this case, the step sizes with a greater value have a higher probability to be selected because they typically lead to better offspring individuals. Two commonly used default settings for parameters in the (μ, λ)-ES are: small population size: $\mu = 1$, $\lambda = 7$, $\tau_{\text{global}} = 1.2$, single step size, $\rho = 1$. Large population size: $\mu = 15$, $\lambda = 100$, $\tau_{\text{local}} = 1.1$, $\tau_{\text{global}} = 1.2$, $\rho = \mu$, and intermediate recombination of decision variables and step-size parameters. A larger population size is preferable if the optimization takes place on a rugged fitness function and a larger number of evaluations can be afforded.

Derandomized Evolution Strategies

Mutative step-size control tends to work well in the standard-ES for the adaptation of a single global step size but shows poor performance when it comes to the individual step sizes or arbitrary normal mutations. Schwefel claimed that the adaptation of the strategy parameters in those cases is impossible within small populations [80], and suggested larger populations as a solution to the problem. Later on, Rudolph questioned the effectiveness of ES learning upon discarding past information and claimed that ES learning would benefit from introducing memory to the individuals [71]. Indeed, due to the crucial role that the mutation operator plays within ES, its mutative step-size control was investigated intensively. Especially, the disruptive effects to which mutative step-size control is subject were studied at several levels [41, 66] and are reviewed here:

- **Indirect selection**. By definition, the goal of the mutation operator is to apply a stochastic variation to the decision variables, which will increase the individual's selection probability. The selection of the *strategy parameters* setting is indirect, i.e., the vector of a successful mutation is not utilized to adapt the step-size parameters, but rather the parameters of the distribution that led to this mutation vector.
- **Realization of parameter variation**. Due to the sampling from a random distribution, the *realization* of the parameter variation does not necessarily reflect the nature of the strategy parameters. Thus, the difference de facto between good and bad strategy settings of strategy parameters is only reflected in the difference between their probabilities to be selected—which can be rather small. Essentially, this means that the selection process of the strategy parameters is *strongly disturbed*.
- The *strategy parameter change rate* is defined as the difference between strategy parameters of two successive generations. Hansen and Ostermeier [41] argue that the change rate is an important factor, as it gives an indication concerning the adaptation speed, and thus it has a direct influence on the performance of the algorithm. The principal claim is that *this change rate basically vanishes* in the standard-ES. The *change rate* depends on the *mutation strength* to which the strategy parameters are subject. While aiming at attaining the maximal change rate, the latter is limited by an upper bound, due to the finite selection information that can be transferred between generations. Change rates that exceed the upper bound would lead to stochastic behavior. Moreover, the mutation strength that obtains an optimal change rate is typically smaller than the one that obtains good diversity among the mutants, a desired outcome of the mutation operator, often referred to as *selection difference*. Thus, the conflict between the objective of *optimal change rate* versus the objective of *optimal selection difference* cannot be resolved at the mutation strength level [66]. A possible solution to this conflict would be to *disconnect* the change rate from the mutation strength.

The so-called *derandomized mutative step-size control* aims to treat those disruptive effects, regardless of search space dimensionality, population size, or any other characteristic parameters.

The concept of derandomized ESs was originally introduced by scholars at the Technical University of Berlin at the beginning of the 1990s. It was followed by the release of a new generation of successful ES variants by Hansen, Ostermeier, and Gawelczyk [40, 42, 65, 67].

The first versions of *derandomized ES algorithms* introduced a controlled global step size to monitor the individual step sizes by decreasing the stochastic effects of the probabilistic sampling. The selection disturbance was completely removed with later versions by omitting the adaptation of strategy parameters by means of probabilistic sampling. This was combined with individual information from the last generation (the successful mutations, i.e., of selected offspring) and then adjusted to *correlated mutations*. Later, the concept of *adaptation by accumulated information* was introduced, aiming to use the information from preceding evaluations wisely for the purpose of step-size adaptation. Rather than using the information of the preceding generation alone, it was successfully generalized to a weighted average of the previous generations.

Note that the different derandomized ES variants strictly follow a $(1, \lambda)$ strategy, postponing the treatment of recombination or plus strategies for later stages. Moreover, the different variants hold different numbers of strategy parameters to be adapted, and this is an important factor in the complexity of the optimization routine and in its learning rate. The different algorithms hold a number of strategy parameters scaling either linearly ($\mathcal{O}(d)$ parameters responsible for individual step sizes) or quadratically ($\mathcal{O}(d^2)$ parameters responsible for arbitrary normal mutations) with the dimensionality d of the search space.

First Level of Derandomization

The so-called *first level of derandomization* aimed the following desired effects: (i) a degree of freedom with respect to the mutation strength of the strategy parameters, (ii) scalability of the *ratio* between the change rate and the mutation strength, and (iii) independence of population size with respect to the adaptation mechanism. The realization of the first level of derandomization can be reviewed through *three* particular derandomized ES variants:

DR1 The first derandomized attempt [65] coupled the successful mutations to the selection of decision parameters and learned the mutation step size as well as the scaling vector based on the successful variation. The mutation step is formulated for the k^{th} individual, $k = 1, \ldots, \lambda$:

$$\vec{x}^{(g+1)} = \vec{x}^{(g)} + \xi_k \delta^{(g)} \vec{\xi}^{k}_{\text{scal}} \vec{\delta}^{(g)}_{\text{scal}} \vec{z}_k \qquad \vec{z}_k \in \{-1, +1\}^d \tag{9}$$

Note that $\vec{z}_k$ is a random vector of ± 1, rather than a normally distributed random vector, while $\vec{\xi}^k_{\text{scal}} \sim \vec{\mathcal{N}}(0,1)^+$, i.e., distributed over the positive part of the normal distribution. The evaluation and selection are followed by the adaptation of the strategy parameters (subscripts *sel* refer to the selected individual):

$$\delta^{(g+1)} = \delta^{(g)} \cdot (\xi_{sel})^{\beta} \tag{10}$$

$$\vec{\delta}^{(g+1)}_{\text{scal}} = \vec{\delta}^{(g)}_{\text{scal}} \cdot \left(\vec{\xi}^{sel}_{\text{scal}} + b\right)^{\beta_{\text{scal}}} \tag{11}$$

DR2 The second derandomized ES variant [67] aimed to accumulate information about the correlation or anticorrelation of past mutation vectors in order to adapt the *global step size* as well as the *individual step sizes*—by introducing a *quasi-memory* vector. This accumulated information allowed omitting the stochastic element in the adaptation of the strategy parameters—updating them only by means of successful variations, rather than with random steps.

The mutation step for the k^{th} individual, $k = 1, \ldots, \lambda$, reads:

$$\vec{x}^{(g+1)} = \vec{x}^{(g)} + \delta^{(g)} \vec{\delta}^{(g)}_{\text{scal}} \vec{z}_k \qquad \vec{z}_k \sim \vec{\mathcal{N}}(0,1) \tag{12}$$

Introducing a quasi-memory vector $\vec{Z}$:

$$\vec{Z}^{(g)} = c\vec{z}_{sel} + (1-c)\,\vec{Z}^{(g-1)} \tag{13}$$

The adaptation of the strategy parameters according to the selected offspring:

$$\delta^{(g+1)} = \delta^{(g)} \cdot \left(\exp\left(\frac{\|\vec{Z}^{(g)}\|}{\sqrt{d}\sqrt{\frac{c}{2-c}}} - 1 + \frac{1}{5d} \right) \right)^{\beta} \tag{14}$$

$$\vec{\delta}^{(g+1)}_{\text{scal}} = \vec{\delta}^{(g)}_{\text{scal}} \cdot \left(\frac{\left|\vec{Z}^{(g)}\right|}{\sqrt{\frac{c}{2-c}}} + b \right)^{\beta_{\text{scal}}}, \qquad \left|\vec{Z}^{(g)}\right| = \left(|Z_1^{(g)}|, |Z_2^{(g)}|, ..., |Z_n^{(g)}| \right) \tag{15}$$

DR3 This third variant [42], usually referred to as the *generation set adaptation* (GSA), considered the derandomization of arbitrary normal mutations for the first time, aiming to achieve invariance with respect to the scaling of variables and the rotation of the coordinate system. This naturally came with the cost of a quasi-memory matrix, $\mathbf{B} \in \mathbb{R}^{r \times d}$, setting the dimension of the strategy parameters space to $d^2 \leq r \leq 2d^2$. The adaptation of the global step size is *mutative* with stochastic variations, just like in the **DR1**.

The mutation step is formulated for the kth individual, $k = 1, \ldots, \lambda$:

$$\vec{x}^{(g+1)} = \vec{x}^{(g)} + \delta^{(g)} \xi_k \vec{y}_k \tag{16}$$

$$\vec{y}_k = c_m \mathbf{B}^{(g)} \cdot \vec{z}_k \qquad \vec{z}_k \sim \vec{\mathcal{N}}(0, 1) \tag{17}$$

The update of the *memory matrix* is formulated as

$$\begin{aligned} &\mathbf{B}^{(g)} = \left(\vec{b}_1^{(g)}, \ldots, \vec{b}_r^{(g)}\right) \\ &\vec{b}_1^{(g+1)} = (1-c) \cdot \vec{b}_1^{(g)} + c \cdot (c_u \xi_{sel} \vec{y}_{sel}), \vec{b}_{i+1}^{(g+1)} = \vec{b}_i^{(g)} \end{aligned} \tag{18}$$

The step size is updated as follows:

$$\delta^{(g+1)} = \delta^{(g)} \, (\xi_{sel})^{\beta} \tag{19}$$

Second Level of Derandomization: CMA-ES

Following a series of successful derandomized ES variants addressing the first level of derandomization, and a continuous effort at the Technical University of Berlin, the so-called *Covariance Matrix Adaptation* (CMA) evolution strategy was released in 1996 [40], as a completely derandomized evolution strategy—the *fourth* generation of derandomized ES variants. The so-called *second level of derandomization* targeted the following effects: (i) The probability of regenerating the same mutation step is increased, (ii) *change rate* of the strategy parameters is subject to explicit control, and (iii) the strategy parameters are stationary when subject to random selection. The second level of derandomization was implemented by means of the CMA-ES.

The CMA-ES combines the robust mechanism of ES with powerful principles of *statistical learning*, and thus it is sometimes subject to informal criticism for not being a genuine biomimetic evolution strategy. In short, it aims to satisfy *maximum likelihood principle* by applying *principal components analysis* (PCA) [51] to the successful mutations and uses *cumulative global step-size adaptation*.

In the notation used here, the vector $\vec{m}$ represents the mean of the mutation distribution but is also associated with the current solution point, σ denotes the *global step size*, and the covariance matrix $\mathbf{C}$ determines the shape of the distribution ellipsoid:

$$\vec{x}^{NEW} \sim \mathcal{N}(\vec{m}, \sigma^2 \mathbf{C}) = \vec{m} + \sigma \cdot \mathcal{N}(\vec{0}, \mathbf{C}) = \vec{m} + \sigma \cdot \vec{z}$$

Two independent principles define the adaptation of the covariance matrix $\mathbf{C}$, versus the adaptation of the global step size σ:

- The mean $\vec{m}$ and the covariance matrix $\mathbf{C}$ of the normal distribution are updated according to *maximum likelihood principle*, so that good mutations are likely to appear again. $\vec{m}$ is updated such that

$$\mathcal{P}\left(\vec{x}_{sel}|\mathcal{N}\left(\vec{m}, \sigma^2\mathbf{C}\right)\right) \longrightarrow \max$$

  and $\mathbf{C}$ is updated such that

$$\mathcal{P}\left(\frac{\vec{x}_{sel} - \vec{m}_{old}}{\sigma}\middle|\mathcal{N}\left(\vec{0}, \mathbf{C}\right)\right) \longrightarrow \max$$

  considering the prior $\mathbf{C}$. This is implemented through the so-called *covariance matrix adaptation* (CMA) mechanism.
- σ is updated so that it is conjugate perpendicular to the consecutive steps of $\vec{m}$. This is implemented through the so-called *cumulative step-size adaptation* (CSA) mechanism.

Evolution Path A straightforward way to update the covariance matrix would be to construct a $d \times d$ *matrix analogue* to the **DR2** mechanism (see Eq. 13), with the *outer product* of the selected mutation vector $\vec{z}_{sel}$:

$$\mathbf{C} \longleftarrow (1 - c_{\text{cov}})\mathbf{C} + c_{\text{cov}}\vec{z}_{sel}\vec{z}_{sel}^T$$

However, to avoid discarding the sign information of $\vec{z}_{sel}$, the so-called *evolution path* is defined to accumulate the past information using an *exponentially weighted moving average*,

$$\vec{p}_c \propto \sum_{i=0}^{g}(1 - c_c)^{g-i}\vec{z}_{sel}^{(i)},$$

yielding the following update step for the covariance matrix:

$$\mathbf{C} \longleftarrow (1 - c_{\text{cov}})\mathbf{C} + c_{\text{cov}}\vec{p}_c\vec{p}_c^T$$

The Path Length Control The covariance matrix update is not likely to simultaneously increase the variance in all directions, and thus a global step-size control is much needed to operate in parallel. The basic idea of the so-called *path length control* is to measure the length of the evolution path, which also constitutes the consecutive steps of $\vec{m}$, and adapt the step size according to the following rationale: If *evolution path* is longer than expected, the steps are likely parallel, and thus the step size should be increased; alternatively, if it is shorter than expected, the steps are probably antiparallel, and the step size should be decreased accordingly. That

magnitude is defined as the expected length of a normally distributed random vector. This evaluation is explicitly carried out by the *conjugate* evolution path:

$$\vec{p}_\sigma \propto \sum_{i=0}^{g} (1 - c_\sigma)^{g-i} \mathbf{C}^{(i)\,-\frac{1}{2}}\, \vec{z}_{sel}^{\,(i)}$$

where the *eigendecomposition* of $\mathbf{C}$ is required in order to align all directions within the *rotated frame*. Then, the update of the step size depends on the comparison between $\|\vec{p}_\sigma\|$ and the expected length of a normally distributed random vector, $E\left[\|\mathcal{N}(0, \mathbf{I})\|\right]$:

$$\sigma \longleftarrow \sigma \cdot \exp\left(\frac{\|\vec{p}_\sigma\|}{E\left[\|\mathcal{N}(0, \mathbf{I})\|\right]} - 1\right)$$

The (μ_W, λ) Rank-μ CMA The Rank-μ covariance matrix adaptation [39] is an extension of the original update rule for larger population sizes. The idea is to use μ vectors ($\mu > 1$) to update the covariance matrix $\mathbf{C}$ in each generation, based on *weighted intermediate recombination*. Let $\vec{x}_{i:\lambda}$ denote the i^{th} ranked solution point, such that

$$f\left(\vec{x}_{1:\lambda}\right) \leq f\left(\vec{x}_{2:\lambda}\right) \leq \cdots \leq f\left(\vec{x}_{\lambda:\lambda}\right)$$

The updated *mean* is now defined as follows:

$$\vec{m} \leftarrow \sum_{i=1}^{\mu} w_i \vec{x}_{i:\lambda} = \vec{m} + \sigma \sum_{i=1}^{\mu} w_i \vec{z}_{i:\lambda} \equiv \langle \vec{x} \rangle_W$$

with a set of weights, $w_1 \geq w_2 \geq \cdots \geq w_\mu > 0$, $\sum_{i=1}^{\mu} w_i = 1$. The covariance matrix update is now formalized by means of rank-μ update, combined with the rank-one update:

$$\mathbf{C} \longleftarrow (1 - c_{\text{cov}})\mathbf{C} + \frac{c_{\text{cov}}}{\mu_{\text{cov}}} \vec{p}_c \vec{p}_c^{\,T} + c_{\text{cov}} \left(1 - \frac{1}{\mu_{\text{cov}}}\right) \sum_{i=1}^{\mu} w_i \vec{z}_{i:\lambda} \vec{z}_{i:\lambda}^{\,T}$$

The (μ_W, λ)-CMA-ES heuristic is summarized in Algorithm 2, with `initCMA()` referring to the parametric initialization procedure. It should be noted that a CMA-ES variant, which resembles DR2 and targets a vector of d individual step sizes (i.e., by means of a *diagonalized* covariance matrix), was released by the name of sep-CMA-ES [70]. Furthermore, the CMA-ES heuristic was simplified in the form of the so-called CMSA strategy [21] and was further improved for certain cases of global optimization [2].

Algorithm 2: (μ_W, λ)-CMA-ES

1: `initCMA()`
2: $g \leftarrow 0$
3: **repeat**
4: **for** $k = 1 \ldots \lambda$ **do**
5: $\vec{x}_k^{(g+1)} \leftarrow \langle \vec{x} \rangle_W^{(g)} + \sigma^{(g)} \cdot \vec{z}_k^{(g+1)}, \quad \vec{z}_k^{(g+1)} \sim \mathcal{N}\left(\vec{0}, \mathbf{C}^{(g)}\right)$
6: $f_k^{(g+1)} \leftarrow \texttt{Evaluate}\left(\vec{x}_k^{(g+1)}\right)$
7: **end for**
8: $\langle \vec{x} \rangle_W^{(g+1)} \leftarrow \texttt{Select}\left(\vec{f}^{(g+1)}, \vec{x}_{1\ldots\mu:\lambda}^{(g+1)}\right)$
9: $\vec{p}_c^{(g+1)} \leftarrow \texttt{UpdatePath}\left(\vec{p}_c^{(g)}, \vec{z}_{1\ldots\mu:\lambda}^{(g+1)}\right)$
10: $\mathbf{C}^{(g+1)} \leftarrow \texttt{UpdateCov}\left(\mathbf{C}^{(g)}, \vec{p}_c^{(g+1)}, \vec{z}_{1\ldots\mu:\lambda}^{(g+1)}\right)$
11: $\mathbf{C}^{(g+1)} := \mathbf{R}^{(g+1)} \mathbf{\Lambda}^{(g+1)} \left(\mathbf{R}^{(g+1)}\right)^T, \mathbf{\Lambda}^{(g+1)} = \text{diag}\left(\lambda_{\max}^{(g+1)}, \ldots, \lambda_{\min}^{(g+1)}\right)$
12: $\vec{p}_\sigma^{(g+1)} \leftarrow \texttt{UpdateSPath}\left(\vec{p}_\sigma^{(g)}, \vec{z}_{1\ldots\mu:\lambda}^{(g+1)}\right)$
13: $\sigma^{(g+1)} \leftarrow \texttt{UpdateStep}\left(\sigma^{(g)}, \vec{p}_\sigma^{(g)}\right)$
14: $g \leftarrow g + 1$
15: **until** stopping criterion is met
16: **return** $\langle \vec{x} \rangle_W^{(g)}$

Theoretical Results

Historically, the theory of ESs has primarily examined questions of convergence dynamics.

Under relatively mild conditions on the used mutation and recombination operators, complete global convergence for $t \to \infty$ can be proven for a certain probability [73]. Basically, for continuous objective functions, it is sufficient to ascertain that for every ϵ ball around the global minimizer, the probability that the mutation operator samples a point in this region is positive, regardless of the starting point. This is given, for instance, by bounding the standard deviations of the mutation from below by a small positive value. This can be easily generalized to ES for discrete [72] or even mixed-integer optimization problems [59].

Of more practical relevance are results on the convergence dynamics that is the speed of convergence to the optimum and on local progress rates. Different approaches for analysis have been used, based on dynamical systems theory [18], stochastic process theory [73], and techniques from the asymptotic analysis of randomized algorithms [49]. It is a well-established result that the most self-adaptive ES variants, if parameterized correctly, achieve a linear convergence rate in convex-quadratic problems, where the matrix condition number and the type of adaptation of step size determine the linear factor [20,62]. The same can be found for problems with proportional noise of fitness [4]. On the other hand, on sharp ridges, plateaus, and at the boundary of constraints, the classical step-size adaptation tends to fail

[54]. Also, for problems with additive noise, the accuracy of the results found will be limited by the standard deviation of the noise [55]. The theory of ESs also revealed some insights regarding the manner in which different parameters are correlated with each other and devised guidelines concerning optimal parameter settings. Most prominently, the $1/5$th success rule for step-size control in $(1+1)$-ES with isotropic mutation was developed based on theoretical studies on the sphere and the corridor model [69].

Also results on the effect of genetic drift and recombination in population-based ESs are available: Beyer highlighted the so-called genetic repair effect that recombination has in ESs. When using recombination, convergent behavior and optimal convergence rates can be achieved with higher mutation step sizes. This increases the robustness in global optimization settings as it is more likely to escape from local optima. The genetic repair effect is stronger when intermediate recombination is used, as compared to discrete recombination. Dynamical systems analysis and Markov chain analysis on simple search landscapes revealed that it is rather impossible to simultaneously explore different local optima by means of a single population [16]. Even if the recombination operator is disabled, when different attractor basins share exactly the same geometry, the population tends to quickly concentrate on a single attractor only [77]. For the global convergence on multimodal landscapes, analytical results have been derived for the progress rate and the success probability of hitting the global optimum [64, 75, 76]. Remarkably, on the Rastrigin function, it is shown that to maintain a high success probability, one must scale the population size μ with respect to the dimension $d \mid \mu \in \mathcal{O}(\sqrt{d} \ln d) \mid$ for both mutative step-size control and cumulative step-size adaptation [76]. These findings gave incentive to the development of niching [82] and restart methods [6, 26] that counteract this effect and prove to perform better on multimodal landscapes.

The Link to Statistical Learning and the Covariance-Hessian Relation The pragmatic motivation to obtain the landscape Hessian will become evident in the next section when we discuss noise handling and robust optimization (section "Noise Handling and Robust Optimization with ESs"). Here, as another important theoretical pillar, we elaborate on the inherent relation of the ESs' covariance matrix to the landscape Hessian. Attempts to formally characterize this relation originated in the release of the so-called natural ESs [94], and the consequent compilation of the information geometry optimization (IGO) philosophy [63] in parallel to the formulation of the natural gradient (NGD) algorithm [1]. In short, the building block of this class of algorithms is the *natural gradient method* [3], which features steepest ascent moves on the Riemannian manifold of the strategy distribution function using Fisher information. Importantly, in the context of landscape learning, modern ESs were proven to achieve such learning under certain adaptation mechanisms. Akimoto proved that the NGD algorithm adapts its covariance matrix so it becomes proportional to the inverse landscape Hessian of any monotonic convex-quadratic composite function in the limit of a large population size [1]. In a broader perspective, Beyer showed that upon implementing

the IGO philosophy in the same population-size limit, the self-adapted covariance matrix necessarily becomes proportional to the inverse Hessian [19]. Finally, a recent study proved a more fundamental result, stating that the covariance matrix over $(1, \lambda)$-selected decision vectors becomes proportional to the inverse of the landscape Hessian as the population size λ increases [85]. Explicitly, it showed that the statistically constructed covariance matrix over *single-winning* decision vectors converges to the inverse of the landscape Hessian, up to a scalar factor, when the population size tends to infinity for any quadratic objective function. Importantly, this kind of learning does not require the NGD mechanism or any derandomization (for adaptation) nor the IGO (as a proof tool)—i.e., remarkably, it is an inherent property of standard ESs which simply rely on Gaussian mutations and rank-based selection. This result constitutes a mathematical rigor to support a long-standing hypothesis on the behavior of ESs. The precise estimation of the local Hessian has been the topic of further, subsequent studies, such as [33], which propose an approach motivated by machine learning theory.

Nonstandard Evolution Strategies

The broad success of the family of ESs provided the motivation to devise extended heuristics for treating problem instances that are beyond the canonical unconstrained single-objective, unimodal optimization formulation. Indeed, ESs' extensions to mixed-integer search spaces [9, 58, 84, 95], uncertainty handling [43], multimodal domains [82] and to multi-objective Pareto optimization [47] were introduced in recent years. The goal of the current section is to provide an overview of those extensions.

ESs for Nonstandard Search Spaces Mixed-integer (MI) optimization [59] targets an objective function subject to constraints that include both continuous and discrete variables. MI optimization problems are NP-hard already in the linear case, and obtaining optimality in either white- or black-box scenarios is often challenging. MI ESs (MIESs) were originally investigated in the 1990s [9, 72] and have been enjoying a growing interest three decades later (see, e.g., [46, 90, 95]), including two successful MI variants of the CMA-ES [36, 61].

At a broader level, a general framework for ESs on nonstandard search spaces, termed metric-based evolutionary algorithms, was developed by Droste and Wiesmann [28]. They specified guidelines for instantiating of a mutation operator and recombination operators and exemplified the design method for the optimization of ordinary binary decision diagrams. By using similar guidelines, ESs for integer programming [72], mixed-integer programming [59], and graph-based optimization [29] were developed. Common guidelines for the design of ESs for new types of solution spaces are:

1. Causal Representation: Solutions should be represented in some metric search space such that relatively small changes with respect to the distance in the search

space result on average in only relatively small changes of the objective function value.
2. Unimodal Mutation Distributions: Small mutations should occur more likely than large mutations.
3. Scalability of Mutation: In order to implement self-adaptive mutation operators, it is essential that mutations can be scaled in terms of the average distance between parents and offspring.
4. Accessibility of Points by Mutation: By applying one (or a chain of many) mutation, it should be possible to reach every point in the search space, regardless of the starting point.
5. Unbiasedness of Mutation: Mutations should not introduce a bias in the search. They should be symmetric, and probability distributions with maximal entropy should be preferred.
6. Similarity to Parents in Recombination: The distance of an offspring to its parents should not exceed the distance of the parents to each other. Moreover, on average, the distance to all parents should be the same.

Following these design principles, one can expect generalized ESs to possess similar properties in comparison to standard ESs for continuous search spaces. However, a warning should be placed here regarding the functioning of step-size adaptation. In the theoretical derivation of optimal schemes of ESs, it is often assumed that differentiable problems locally resemble quadratic or linear functions. This property is lost when it comes to discrete optimization, and therefore, the generalization of such results requires some caution. On the other hand, it has been found that mutative self-adaptation of step sizes also works in nonstandard search spaces such as for the adaptation of mutation probabilities for binary vectors or for parameters of geometric distributions in MIESs.

Niching and Multi-population ESs

Given multimodal search landscapes with multiple basins of attraction that are of interest, targeting the simultaneous identification of several optima constitutes a challenge at both the theoretical and practical levels [60, 68, 82]. In such a scenario, the aim is to obtain the top optima of the landscape, yet in practical applications it may translate to locating a diverse set of high-quality solutions, optimal or suboptimal. Within the domain of evolutionary computation, this challenge is typically treated by extending a given search heuristic into subpopulations of trial solutions that evolve in parallel to various solutions of the problem. This idea stems from the evolutionary concept of *organic speciation*, and the so-called *niching* techniques are the extension of EAs to speciation forming multiple subpopulations. The computational challenge in niching may be formulated as achieving an effective interplay between partitioning the search space into niches occupied by stable subpopulations, by means of population diversity preservation, and exploiting the search in each niche by means of a highly efficient optimizer with local-search capabilities [82]. A niching framework utilizing derandomized ES

was introduced in [82], proposing the CMA-ES as a niching optimizer for the first time. The underpinning of that framework was the selection of a peak individual per subpopulation in each generation, followed by its sampling according to DES principles to produce the consecutive dispersion of search points. The *biological analogy* of this machinery is an *alpha-male winning all the imposed competitions and dominating thereafter its ecological niche*, which then obtains all the sexual resources therein to generate its offspring.

A common utility for defining the landscape subdomain of each subpopulation is a so-called niche radius. A radius-based framework for niching, which employs derandomized ES heuristics, has been formulated and investigated [83] and has shown a broad success in tackling both synthetic and real-world multimodal optimization problems. In practice, this framework holds multiple derandomized ES populations, which conduct heuristic search in their radii-defined subdomains and independently update their mutation distributions and step sizes. Since the partitioning is enforced per each generation according to the niche-radius parameter, and since no a priori knowledge is available on the global structure of the search landscape and the spatial distribution of its basins of attraction, an adaptive niche-radius approach was devised to remedy this so-called niche-radius presumption [86]. The main idea of ES niching with self-adaptive niche-shape approaches is to exploit learned landscape information, as reflected by the evolving mutation distribution, to define the niches in a more accurate manner. Especially, a Mahalanobis CMA-ES niching heuristic was formulated, which carries out the distance calculations among the individuals based upon the Mahalanobis distance metric, by utilizing the evolving covariance matrices of the CMA mechanism. Such heuristic operation resulted in successful niching on landscapes with unevenly shaped optima, on which the fixed-radii approaches performed poorly [86].

In scenarios where obtaining the *global* optimum is the paramount target in multimodal landscapes, niching may not necessarily be the most suitable approach to treat it. Rather, restart schemes can be effective in such scenarios. A multi-restart with increasing population-size approach was developed with the CMA algorithm, namely IPOP-CMA-ES [5]. This heuristic aims at attaining the global optimum, while possibly visiting local optima along the process and restarting the algorithm with a larger population size and a modified initial step size. The question of redundant restarts, when optimizing multimodal landscapes using ESs and particularly the CMA-ES, has been recently addressed [26]. That study proposed a metaheuristic to avoid such restarts while utilizing the CMA-ES.

Noise Handling and Robust Optimization with ESs

Robust optimization is concerned with identifying solvers that can also perform well when the input parameters and/or the values of the objective function are slightly perturbed [14]. Most variants of ESs are inherently suited to deal with noisy environments, and larger population size and the use of recombination were shown to be beneficial in noisy settings [4, 17]. Various techniques have

been shown to further improve the performance of noisy objective functions. One technique is the so-called thresholding operator, which considers search points as an improvement only when they introduce objective function improvements that exceed a certain threshold [13]. Moreover, it has been suggested to use the sample mean of multiple evaluations of the same individual (effective fitness) for evaluation. This increases the computation time, and therefore, more efficient sampling schemes have been developed subsequently. In [27], an adaptive sampling method is devised to determine the evaluation number based on the eigenspectrum of the covariance matrix, the step size, population size, and the local Lipschitz constant. Furthermore, a rank-stability scheme was suggested to treat noise, exploiting the fact that ESs rather require a correct ranking rather than a correct objective function value [43].

Knowledge of second-order Hessian information at the optimum is desirable not only as a measure of system robustness to noise in the decision variables but also as a means for dimensionality reduction and for landscape characterization. Experimental optimization of quantum systems motivated the compilation of an automated method to efficiently retrieve the Hessian matrix about the global optimum *without derivative evaluations* from experimental measurements [87]. The study designed a heuristic to learn the Hessian matrix based on the CMA-ES machinery, with necessary modifications, by exploiting an inherent relation between the covariance matrix and the inverse Hessian matrix. It then corroborated this newly proposed technique, entitled forced optimal covariance adaptive learning (FOCAL), on noisy simulation-based optimization as well as on laboratory experimental quantum systems. For a comprehensive overview of contemporary noise handling and robust optimization in ESs, the reader is referred to the PhD dissertation of Kruisselbrink, which was also complemented by an empirical study of the most common variants. In addition, theoretical limits in the precision of multievaluation schemes in the presence of additive noise are derived therein [55].

Multi-criterion and Constraint-Handling ESs

In practical settings, the scenario of unconstrained optimization is not very common. Rather, problems with multiple constraint functions and conflicting objective functions need to be solved.

In optimization with constraints, it is mandatory to use alternative schemes for step-size adaptation for reasons that are explained in detail by Kramer and Schwefel [54]. They also suggested alternative schemes that can better deal with the constraints.

Adaptations of the ESs are also required for optimization with multiple objective functions. In this case, it is common to search for a set of nondominated solutions, the so-called Pareto set. A first proposal on using ESs to approximate a Pareto front was made by Kursawe [56], a long time before the nowadays flourishing research field of evolutionary multi-criterion optimization was established. Today, three variants of the evolution strategy are used for optimization with multiple objectives:

- Pareto-archiving evolution strategy [53]: This classical multi-criterion optimization strategy uses an archive to maintain nondominated points. The archive is updated based on the nondominance and density of points.
- Predator-prey evolution strategy [35,57]: In this biomimetic strategy, individuals are distributed on a grid, and a population of predator individuals is performing a random walk on the grid and triggers local selection. The predators select their prey based on different objective functions or combination strategies.
- Multi-objective CMA-ES [47]: This strategy seeks to improve contributions of individuals to the hypervolume indicator, which measures the size of the Pareto-dominated space. Consequently, this strategy is well adapted to locate precise and regular representations of Pareto fronts for objective functions with complex shapes and correlated input variables. Another self-adaptation method, using local tournaments on hypervolume contributions, was suggested in [52] but so far received little attention.

Mirrored Orthogonal Sampling

The mirrored sampling technique is a derandomized mutation method. It was firstly introduced in [24] for non-elitist $(1, \lambda)$-ES and then extended to the (μ, λ)-ES [7]. The idea of mirrored sampling is to generate part (normally half) of the offspring population in a derandomized way. More specifically, a single mutation vector $\mathbf{z}$ is utilized to generate two offspring (rather than one in the standard ES)—one by adding $\mathbf{z}$ to the parent $\mathbf{x}$: $\mathbf{x} + \mathbf{z}$, and another by subtracting $\mathbf{z}$ from $\mathbf{x}$: $\mathbf{x} - \mathbf{z}$. The two offspring generated are *symmetric* or *mirrored* to the parental point. Mirrored sampling helps accelerating the convergence rate of ESs, which is theoretically proven in [24].

When applied in (μ, λ)-CMA-ES with cumulative step-size adaptation, the mirrored sampling leads to a reduction of recombined mutation variance. Consequently, the step size is more than desirably reduced and a premature convergence would occur. In order to solve this, the concept of pairwise selection is introduced [7], in which only the better offspring among the mirrored pair is allowed to possibly contribute to the weighted recombination. Then, it is assured that recombination will not use both elements of a mirrored pair at the same time. Mirrored sampling can be enhanced by incorporating orthogonal samples [93], where the generated samples form an orthogonal basis. This approach minimizes redundancy in sampling directions and further reduces sampling error, leading to more efficient exploration of the search space. For a comprehensive overview of classical and more advanced sampling strategies and variants of evolutionary strategies, the reader is referred to [12]. Studies such as Biedrzycki's work on CMA-ES involving bound constraints [22] address challenges in sampling strategies that occur when solving real-world problems but are underrepresented in the research of ES which mainly focused on unconstrained continuous optimization.

Algorithm Configurators and Hyperparameter Optimization

Hyperparameter optimization and automated algorithm configuration can be crucial to improving the performance of the covariance matrix adaptation evolution strategy (CMA-ES). This includes tuning parameters such as step size, learning rates, population size, and choosing sampling strategies and covariance matrix update algorithms. Van Rijn et al. [91] explored data mining approaches to identify links between algorithm configurations and performance, advancing automated parameter tuning. In a follow-up work, Van Rijn et al. [92] proposed a method to dynamically adjust CMA-ES configurations during runtime based on problem-specific data.

Recent advancements include the use of meta-optimization, where CMA-ES optimizes its own parameters. Thomaser et al. [89] demonstrated this approach, further advancing self-optimization strategies. In addition to these works that are specific to ES, it is also common to use general-purpose hyperparameter optimization methods in the context of ES tuning.

Benchmarks and Empirical Study

Besides a detailed description of ESs and their theoretical aspects, it would be intuitive and helpful to look at the empirical ability of ESs in solving the black-box problems. In the ESs benchmarking, two difficulties arise. On one hand, it is hard to design a set of test functions that captures the problem characteristics encountered in the real-world applications. On the other hand, summarizing ESs ability over a set of test functions would not be straightforward due to the fact that performance of ESs largely varies on problems with different characteristics (e.g., separability). The black-box optimization benchmark [44] (BBOB) is devised to tackle these difficulties. The noiseless BBOB encompasses 24 noise-free real-parameter single-objective functions that are either separable, ill-conditioned, or multimodal. All the test functions are defined over $\mathbb{R}^d$, while the global optima for all the test functions are initialized in $[-5, 5]^d$ [30]. In addition, BBOB also introduces a proper measure to represent the performance of ESs for global optimization—the empirical cumulative distribution function (ECDF). ECDFs can be summarized over multiple test functions and represented graphically, increasing the accessibility of the benchmark results.

As opposed to the earlier work, in this empirical study, ESs with different step-size adaptation strategies are considered, including the classical $1/5^{\text{th}}$ success rule and mutative self-adaptation. Moreover, the benchmark also covers a broad range of classical and contemporary strategies with derandomized self-adaptation. The ESs tested are listed in the following:

- $(1 + 1)$-ES: one plus one elitist evolution strategy with $1/5^{\text{th}}$ success rule
- $(15, 100)$-MSC-ES: mutative self-adaptation of individual step sizes

- (1, 7)-MSC-ES: mutative self-adaptation of individual step sizes
- DR2-ES: The derandomized evolution strategy using accumulated success mutation vector for step-size adaptation
- $(\mu/\mu_{\mathrm{w}}, \lambda)$-CMA-ES: covariance matrix adaptation evolution strategy with weighted intermediate recombination
- $(\mu/\mu_{\mathrm{w}}, \lambda_{\mathrm{m}})$-CMA-ES: CMA-ES with mirrored sampling and pairwise selection
- IPOP-CMA-ES: a restart CMA-ES with increasing population size
- $(1, \lambda)$-DR2-Niching: the niching approach based on the second derandomized ES variants
- $(1, \lambda)$-CMA-Niching: CMA-ES niching with fixed niche radius
- $(1 + \lambda)$-CMA-Niching: the elitist version
- $(1, \lambda)$-Mahalanobis-CMA-Niching: niche-shape adaptation using Mahalanobis distance

Experimental Settings

The BBOB parameter settings of the experiment are the same for all the tested ES variants. The initial global step size σ is set to 1. The maximum number of function evaluations is set to $10^4 \times d$. The initial solution vector (initial parent) is a uniformly distributed random vector restricted to the hyper-box $[-4, 4]^d$. The algorithms are tested for problems of a different number of input variables d. These are $d \in \{2, 3, 5, 10, 20\}$.

The parameter settings for CMA-ES variants follow the suggested values in [37]; the reader is referred to it for details. For all the niching ES variants, the setting $\lambda = 10$ is used throughout all the test. The fixed niche radius calculation can be found in [82].

Results

BBOB automatically records the history of the fitness values found by the tested algorithm. The time is measured in numbers of function evaluations. The benchmark provides a postprocessing procedure to statistically estimate the empirical cumulative distribution function (ECDF) of running length (function evaluations) to reach the global optimum from the data. The ECDF of running length describes the distribution of necessary function evaluations a specific optimization algorithm follows in order to reach the global optimum in an experiment. An ECDF curve that inclines to distribute running length over smaller values indicates good performance of its corresponding algorithm. Thus, ECDFs characterize the performance of optimization algorithms and are used to present the benchmark results.

In contrast to the standard way of generating ECDFs in BBOB, which is to aggregate ECDFs over function groups (separable, ill-conditioned, multimodal),

in each group, representative functions in BBOB are selected, which are listed as follows:

1. f_1 sphere function
2. f_2 ellipsoidal function
3. f_{10} rotated ellipsoidal function
4. f_8 Rosenbrock function
5. f_{13} sharp ridge function
6. f_7 step ellipsoidal function
7. $f_{15} - f_{19}$ multimodal function having weakly global structure
8. $f_{20} - f_{24}$ multimodal function having adequate global structure

By aggregating the selected functions in functions groups, it is possible to illustrate which ES variants perform better given a specific property (e.g., separable, isotropic, multimodal) by identifying functions with distinct properties. Rather than making a thorough competition, we aim for providing insights into the algorithms' strengths and weaknesses.

The results are presented in two parts. In the first part, all the ESs except IPOP-CMA-ES and the niching ES variants are compared on the aforementioned test functions are excluded from the comparison. The purpose is to compare all the ESs which are inclining to local search. The results are depicted in Figs. 2, 3, 4, and 5. In 5D, the comparisons on the first six functions show that the standard CMA-ES and its mirrored sampling variant outperform the others in most functions. In general, the mutative self-adaptation ESs perform worse than the derandomized ES variants. The reason is that the MSC-ES normally exploits a much larger population size in order to adapt the covariance and thus consumes much more function evaluations. On the simple sphere function, the winner is $(1 + 1)$-ES, as expected from theory. In addition, the DR2-ES is equally good as CMA-ES. On the separable ellipsoid function, DR2-ES even outperforms CMA-ES because it efficiently adapts the uncorrelated mutations. On the non-separable functions (ellipsoid, Rosenbrock, sharp ridge, and step ellipsoid), CMA-ES and mirrored sampling significantly outperform the other ES variants. This is because CMA-ES is capable of adapting arbitrary mutations by means of the covariance matrix, while the other ES variants exploit either isotropic or axis-parallel mutation distributions. In 20D, the comparison roughly shows the same results as in 5D. Note that on separable ellipsoid and step-ellipsoid function, $(15, 100)$-MSC-ES could catch the convergence speed of CMA-ES variants.

In the second part, all the niching ES variants are compared, IPOP-CMA-ES, restart $(1 + 1)$-ES and $(15, 100)$-MSC-ES only on the multimodal functions. These algorithms are grouped for this comparison because they are better equipped for global search. The results are depicted in Figs. 6 (5D) and 7 (20D). On the weakly structured multimodal functions ($f_{15} - f_{19}$), IPOP-CMA-ES outperforms all the niching ES variants, $(1 + 1)$-ES with restart and $(15, 100)$-MSC-ES. $(1+\lambda)$-CMA-Niching shows the best performance among all the niching ES variants

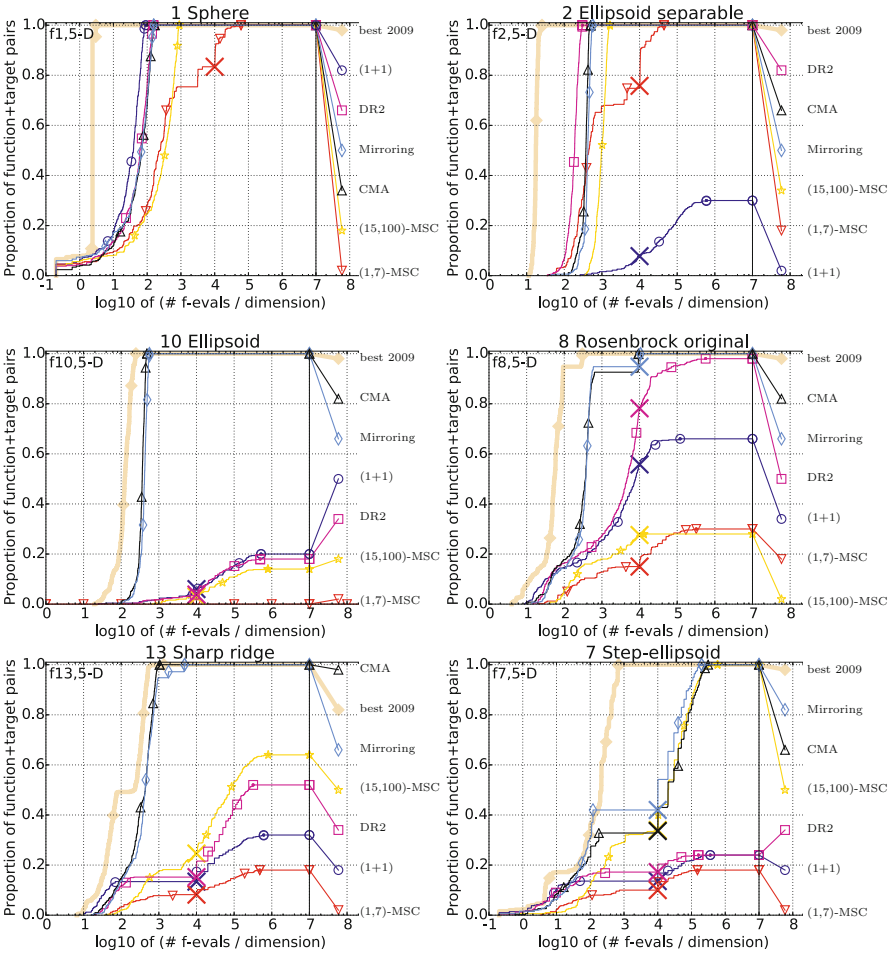


Fig. 2 Bootstrapped empirical cumulative distribution of the number of objective function evaluations divided by dimension (FEvals/DIM) for 50 targets in $10^{[-8..2]}$ for six selected functions 5-D. The "best 2009" line corresponds to the best ERT observed during BBOB 2009 for each single target. Legend: ○:(1+1), ▽:(1,7)-MSC, ⋆:(15,100)-MSC, □:DR2, △:CMA, ♢:Mirroring

tested. Surprisingly, (μ, λ)-MSC-ES performs well even compared to niching ES variants, which may be a consequence of its large population size. On the weakly structured multimodal functions, $(1 + \lambda)$-CMA-Niching could catch up with the performance of IPOP-CMA-ES in both 5D and 20D. (μ, λ)-MSC-ES performs quite poorly in this case. In addition, although it is a simple strategy, $(1 + 1)$-restart shows good results compared to IPOP-CMA-ES and niching ES. Evidently, the niching ES variants spend many function evaluations for maintaining the local optima and thus exhibit altogether poorer performance in terms of global convergence speed when compared to, e.g., IPOP-CMA-ES, which targets the

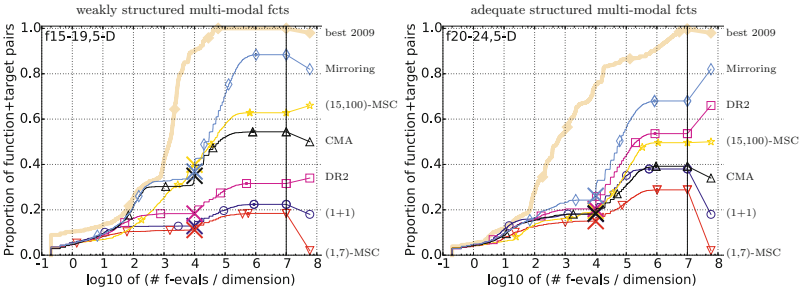


Fig. 3 Bootstrapped empirical cumulative distribution of the number of objective function evaluations divided by dimension (FEvals/DIM) for 50 targets in $10^{[-8..2]}$ for two groups of multimodal functions in 5-D. The "best 2009" line corresponds to the best ERT observed during BBOB 2009 for each single target. Legend: ○:(1+1), ▽:(1,7)-MSC, ⋆:(15,100)-MSC, □:DR2, △:CMA, ♢:Mirroring

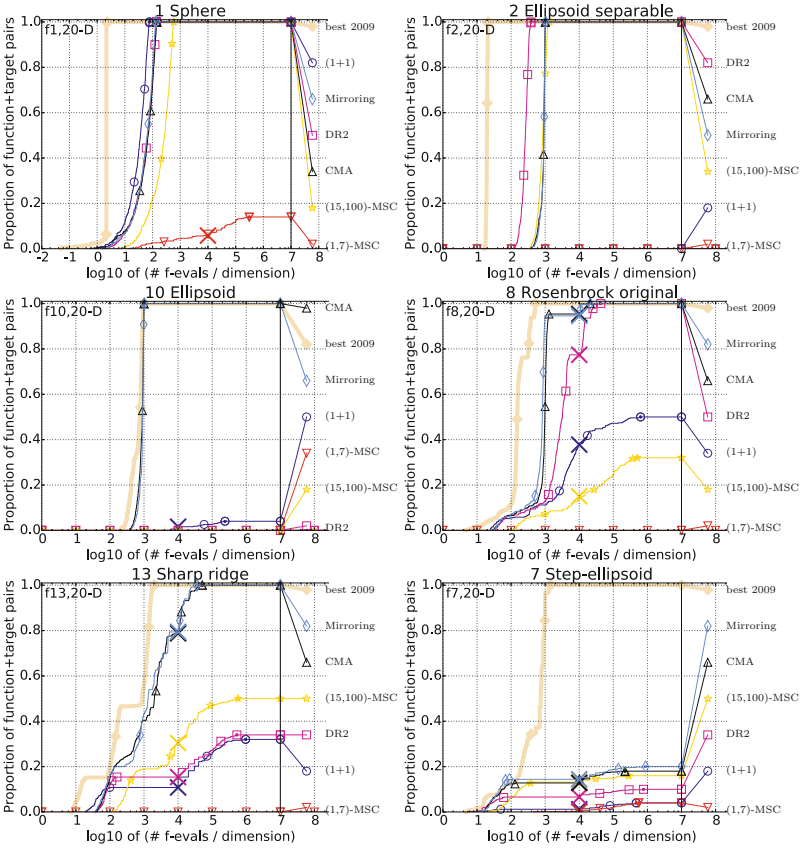


Fig. 4 Bootstrapped empirical cumulative distribution of the number of objective function evaluations divided by dimension (FEvals/DIM) for 50 targets in $10^{[-8..2]}$ for six selected functions 20-D. The "best 2009" line corresponds to the best ERT observed during BBOB 2009 for each single target. Legend: ○:(1+1), ▽:(1,7)-MSC, ⋆:(15,100)-MSC, □:DR2, △:CMA, ♢:Mirroring

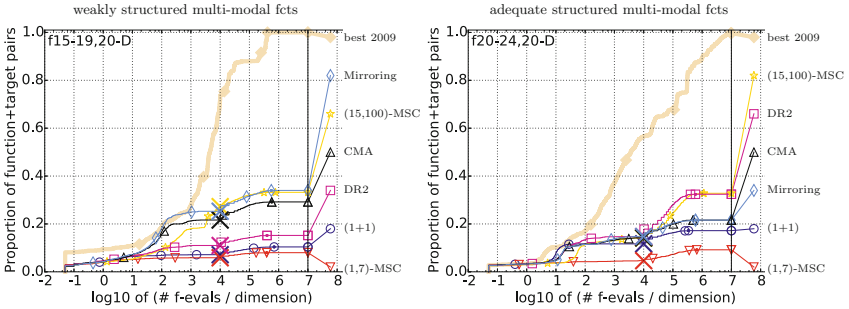


Fig. 5 Bootstrapped empirical cumulative distribution of the number of objective function evaluations divided by dimension (FEvals/DIM) for 50 targets in $10^{[-8..2]}$ for two groups of multimodal functions in 20-D. The "best 2009" line corresponds to the best ERT observed during BBOB 2009 for each single target. Legend: ○:(1+1), ▽:(1,7)-MSC, ⋆:(15,100)-MSC, □:DR2, △:CMA, ♢:Mirroring

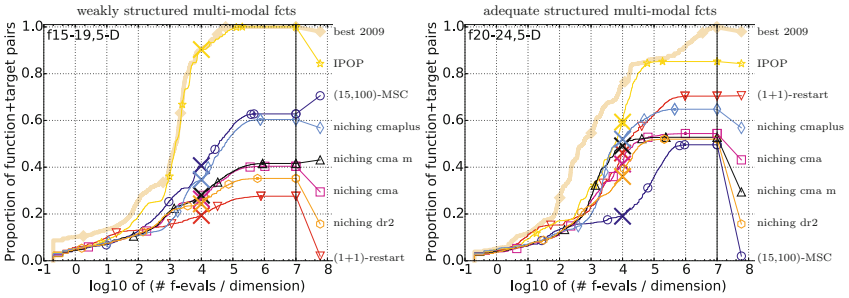


Fig. 6 Bootstrapped empirical cumulative distribution of the number of objective function evaluations divided by dimension (FEvals/DIM) for 50 targets in $10^{[-8..2]}$ for two groups of multimodal functions in 5-D. The "best 2009" line corresponds to the best ERT observed during BBOB 2009 for each single target. Legend: ○:(15,100)-MSC, ▽:(1+1)-restart, ⋆:IPOP, □:niching_cma, △:niching_cma_ma, ♢:niching_cmaplus, ⬡:niching_dr2

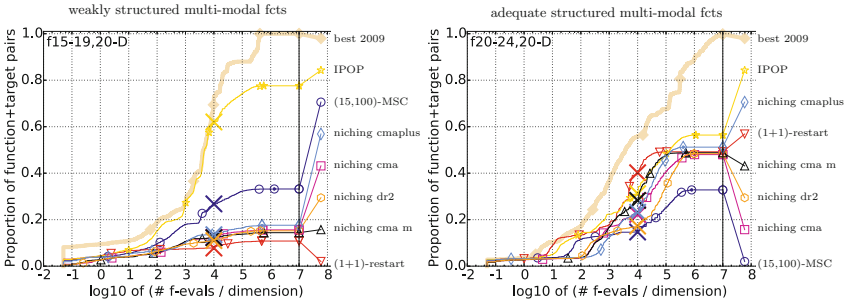


Fig. 7 Bootstrapped empirical cumulative distribution of the number of objective function evaluations divided by dimension (FEvals/DIM) for 50 targets in $10^{[-8..2]}$ for two groups of multimodal functions in 20-D. The "best 2009" line corresponds to the best ERT observed during BBOB 2009 for each single target. Legend: ○:(15,100)-MSC, ▽:(1+1)-restart, ⋆:IPOP, □:niching_cma, △:niching_cma_ma, ♢:niching_cmaplus, ⬡:niching_dr2

accurate approximation of a single optimum and exploits much of its resources to achieve this.

Conclusions

It has been shown in this chapter that ESs are a versatile class of stochastic search heuristics for optimization. There exists a rich body of theoretical results on ESs, including global convergence conditions, results showing linear convergence rates on high-dimensional functions, and findings on the stability of subpopulations and the impact of recombination on global convergence reliability. Moreover, ESs are rank-based (order-invariant) and invariant to changes of the coordinate system. The self-adaptation of the stochastic distribution is an important feature, too, as it frees the user from the burden of choosing the right parameters for mutation and it also makes highly precise approximation of optima possible.

This chapter highlighted mainstream variants of ESs for continuous optimization, including the $(1+1)$-ES with $1/5$th success rule, the $(\mu, \lambda)-$ES with mutative step-size adaptation and common variants of ES with different levels of derandomized step-size adaptation, namely DR1, DR2, DR3, and CMA-ES. Moreover, common concepts of ESs for multimodal optimization were discussed.

All these strategies have been compared on different categories of functions. The empirical studies confirmed the superiority of covariance matrix adaptation techniques on ill-conditioned problems with correlated variables. However, if these problems do not govern the search difficulty, other ESs can be highly competitive as well. Moreover, it was confirmed that multimodal optimization requires special adaptations to ESs in order to achieve maximal performance.

Our literature review has shown that the algorithmic techniques developed for ESs are not only fruitful in the domain of continuous optimization but can be applied to other problem classes as well. Here, the key lies in defining a metric representation of the search space and following a set of guidelines for the design of mutation and recombination operators—which have been reviewed herein rather informally. Importantly, ESs operating in the area of MI optimization have the potential to obtain high impact in comparison to mathematical programming solvers in either single- [95] or multi-objective [84] problem domains. Yet, the compliance of MIESs is still underexplored and calls for further research.

Some prevalent topics for future research will be the integration of multiple criteria and constraints, although some first promising results are already available in this direction. Moreover, for nonstandard ES, the theoretical analysis needs to be advanced, in particular the study of convergence dynamics when the available time is limited. Finally, looking back at the original biological inspiration of ESs, one might conjecture that nature still has many "tricks" in store that, when well understood, could lead to a further enhancement of ES-like search strategies. In this context, it will be interesting to follow recent trends in biological evolution theories [48], showing that a much broader set of mechanisms seems to govern organic evolution than those captured in modern synthesis.

Table 1 Source code packages with ES-related implementations

Package	Language	License	Reference
Pytorch ES	`Python`	MIT	[32]
Nevergrad (by Meta)	`Python`	MIT	[15]
Distributed ES (by OpenAI)	`Python`	MIT	[74]
modcma (ModularCMAES)	`Python`	MIT	[25]
ecr	`R`	GPL3 (GNU)	[23]
CMA-ES	Various	GNU	[38]

Available Code Packages

Table 1 provides an overview of various source code packages that implement ESs. The table includes implementations predominantly in `Python`, with one in `R`, reflecting the accessibility of ES frameworks across programming ecosystems. Each entry specifies the package name, programming language, license type, and reference to its source or documentation. Examples include well-known projects like *Nevergrad* by Meta and *Distributed ES* by OpenAI, both of which use the permissive MIT license. The table also features additional frameworks, such as *Pytorch ES* and *modcma* (ModularCMAES), which are customized for advanced ES experimentation, and allows to assemble different ES variants from modular components. An R-based package, *ecr*, is also listed, expanding the scope of users in statistical computing environments. Additionally, implementations of covariance matrix adaptation evolution strategy (CMA-ES) across a wide range of problems can be found on the homepage of Nikolaus Hansen. The homepage provides source code for CMA-ES in several programming languages, including C (ANSI), C++, Fortran, Java, Matlab and Octave, Python, R, and Scilab. It also contains practical hints and remarks about using the CMA-ES. Note that this list of references is merely a small selection of more recent implementations.

Cross-References

▸ Biased Random-Key Genetic Programming
▸ Evolutionary Algorithms
▸ Genetic Algorithms
▸ Random-Key Genetic Algorithms: Principles and Applications

Competing Interest Declaration The author(s) has no competing interests to declare that are relevant to the content of this manuscript.

References

1. Akimoto Y (2012) Analysis of a natural gradient algorithm on monotonic convex-quadratic-composite functions. In: Proceedings of the 14th annual conference on genetic and evolutionary computation, GECCO '12. ACM, New York, pp 1293–1300
2. Akimoto Y, Sakuma J, Ono I, Kobayashi S (2008) Functionally specialized CMA-ES: a modification of CMA-ES based on the specialization of the functions of covariance matrix adaptation and step size adaptation. In: GECCO '08: proceedings of the 10th annual conference on genetic and evolutionary computation. ACM, New York, pp 479–486
3. Amari S-I, Nagaoka H (2000) Methods of information geometry. Translations of mathematical monographs, vol 191. American Mathematical Society, Providence
4. Arnold DV (2002) Noisy optimization with evolution strategies, vol 8. Springer Science and Business Media, Cham
5. Auger A, Hansen N (2005) A restart CMA evolution strategy with increasing population size. In: Proceedings of the 2005 congress on evolutionary computation CEC-2005. IEEE Press, Piscataway, pp 1769–1776
6. Auger A, Hansen N (2005) A restart CMA evolution strategy with increasing population size. In: Proceedings of the IEEE congress on evolutionary computation (CEC 2005). IEEE Press, Piscataway, pp 1769–1776
7. Auger A, Brockhoff D, Hansen N (2011) Mirrored sampling in evolution strategies with weighted recombination. In: Proceedings of the 13th annual conference on genetic and evolutionary computation, GECCO '11. ACM, New York, pp 861–868
8. Bäck T (1996) Evolutionary algorithms in theory and practice. Oxford University Press, Oxford
9. Bäck T, Schütz M (1995) Evolution strategies for mixed integer optimization of optical multilayer systems. In: Evolutionary programming IV – proceedings of the fourth annual conference on evolutionary programming. The MIT Press, pp 33–51
10. Bäck T, Hammel U, Schwefel H-P (1997) Evolutionary computation: comments on the history and current state. IEEE Trans Evolut Comput 1(1):3–17 (1997)
11. Bäck T, Emmerich M, Shir OM (2008) Evolutionary algorithms for real world applications [application notes]. IEEE Comput Intell Mag 3(1):64–67
12. Bäck T, Foussette C, Krause P (2013) Contemporary evolution strategies, vol 86. Springer, Berlin
13. Bartz-Beielstein T (2005) Evolution strategies and threshold selection. In: Hybrid metaheuristics. Springer, Berlin, pp 104–115
14. Ben-Tal A, El Ghaoui L, Nemirovski A (2009) Robust optimization. Princeton series in applied mathematics. Princeton University Press, Princeton
15. Bennet P, Doerr C, Moreau A, Rapin J, Teytaud F, Teytaud O (2021) Nevergrad: black-box optimization platform. ACM SIGEVOlution 14(1):8–15
16. Beyer H-G (1999) On the dynamics of EAs without selection. Found Genet Algor 5:5–26
17. Beyer H-G (2000) Evolutionary algorithms in noisy environments: theoretical issues and guidelines for practice. Comput Methods Appl Mech Eng 186(2):239–267
18. Beyer H-G (2001) The theory of evolution strategies. Springer, Berlin
19. Beyer H-G (2014) Convergence analysis of evolutionary algorithms that are based on the paradigm of information geometry. Evolut Comput 22(4):679–709
20. Beyer H-G, Schwefel H-P Evolution strategies–a comprehensive introduction. Nat Comput 1(1):3–52
21. Beyer H-G, Sendhoff B (2008) Covariance matrix adaptation revisited – The CMSA evolution strategy. In: Parallel problem solving from nature - PPSN X. Lecture notes in computer science, vol 5199. Springer, pp 123–132

22. Biedrzycki R (2020) Handling bound constraints in CMA-ES: an experimental study. Swarm Evolut Comput 52:100627
23. Bossek J (2017) Ecr 2.0: a modular framework for evolutionary computation in R. In: Proceedings of the genetic and evolutionary computation conference (GECCO) companion, GECCO '18, Berlin. ACM, pp 1187–1193
24. Brockhoff D, Auger A, Hansen N, Arnold DV, Hohm T (2010) Mirrored sampling and sequential selection for evolution strategies. In: Schaefer R, Cotta C, Kolodziej J, Rudolph G (eds) Proceedings of the 11th international conference on parallel problem solving from nature: part I, PPSN'10. Springer, Berlin, pp 11–21
25. de Nobel J (2024) Modular CMA ES. Accessed: November 2024
26. de Nobel J, Vermetten D, Kononova AV, Shir OM, Bäck T (2024) Avoiding redundant restarts in multimodal global optimization. In: Affenzeller M, Winkler SM, Kononova AV, Trautmann H, Tušar T, Machado P, Bäck T (eds) Parallel problem solving from nature – PPSN XVIII. Springer, Cham, pp 268–283 Nature Switzerland
27. Dinu C-V, Patel YJ, Bonet-Monroig X, Wang H (2024) An adaptive re-evaluation method for evolution strategy under additive noise. arXiv preprint arXiv:2409.16757
28. Droste S, Wiesmann D (2003) On the design of problem-specific evolutionary algorithms. In: Advances in evolutionary computing. Springer, Berlin, pp 153–173
29. Emmerich M, Grötzner M, Schütz M (2001) Design of graph-based evolutionary algorithms: a case study for chemical process networks. Evolut Comput 9(3):329–354
30. Finck S, Hansen N, Ros R, Auger A (2010) Real-parameter black-box optimization benchmarking 2010: presentation of the noisy functions. Technical Report 2009/21, Research Center PPE
31. Fogel LJ (1999) Intelligence through simulated evolution: forty years of evolutionary programming. Wiley, Hoboken
32. Gambardella A (2024) Pytorch-es: PyTorch implementation of evolution strategies. https://github.com/atgambardella/pytorch-es. Accessed: 22 Nov 2024
33. Glasmachers T, Krause O (2020) The Hessian estimation evolution strategy. In: International conference on parallel problem solving from nature. Springer, pp 597–609
34. Goldberg D (1989) Genetic algorithms in search, optimization and machine learning. Addison-Wesley, Reading
35. Grimme C, Schmitt K (2006) Inside a predator-prey model for multi-objective optimization: a second study. In: Proceedings of the 8th annual conference on genetic and evolutionary computation. ACM, pp 707–714
36. Hamano R, Saito S, Nomura M, Shirakawa S (2022) CMA-ES with margin: lower-bounding marginal probability for mixed-integer black-box optimization. In: Proceedings of the genetic and evolutionary computation conference, GECCO '22, New York, NY. Association for Computing Machinery, pp 639–647
37. Hansen N (2005) The CMA evolution strategy: a tutorial
38. Hansen N (2024) CMA-ES source code. Accessed: November 2024
39. Hansen N, Kern S (1998) Evaluating the CMA evolution strategy on multimodal test functions. In: Parallel problem solving from nature - PPSN V. Lecture notes in computer science, vol 1498. Springer, Amsterdam, pp 282–291
40. Hansen N, Ostermeier A (1996) Adapting arbitrary normal mutation distributions in evolution strategies: the covariance matrix adaptation. In: Proceedings of the 1996 IEEE international conference on evolutionary computation. IEEE, Piscataway, pp 312–317
41. Hansen N, Ostermeier A (2001) Completely derandomized self-adaptation in evolution strategies. Evolut Comput 9(2):159–195
42. Hansen N, Ostermeier A, Gawelczyk A (1995) On the adaptation of arbitrary normal mutation distributions in evolution strategies: the generating set adaptation. In: Proceedings of the sixth international conference on genetic algorithms (ICGA6). Morgan Kaufmann, San Francisco, pp 57–64

43. Hansen N, Niederberger S, Guzzella L, Koumoutsakos P (2009) A method for handling uncertainty in evolutionary optimization with an application to feedback control of combustion. IEEE Trans Evolut Comput 13(1):180–197
44. Hansen N, Auger A, Finck S, Ros R (2010) Real-Parameter Black-Box Optimization Benchmarking 2010: Experimental Setup. Technical Report RR-7215, INRIA
45. Hansen N, Arnold DV, Auger A (2015) Evolution strategies. Springer, Berlin, pp 871–898
46. Hong Y, Arnold D (2023) Evolutionary mixed-integer optimization with explicit constraints. In: Proceedings of the genetic and evolutionary computation conference, GECCO '23. Association for Computing Machinery, New York, pp 822–830
47. Igel C, Hansen N, Roth S (2007) Covariance matrix adaptation for multi-objective optimization. Evolut Comput 15(1):1–28
48. Jablonka E, Lamb MJ (2005) Evolution in four dimensions. MIT, Cumberland
49. Jägersküpper J (2007) Algorithmic analysis of a basic evolutionary algorithm for continuous optimization. Theor Comput Sci 379(3):329–347
50. John H (1992) Holland, adaptation in natural and artificial systems. MIT Press, Cambridge
51. Jolliffe I (1992) Principal component analysis, 2nd edn. Springer, New York
52. Klinkenberg J-W, Emmerich MT, Deutz AH, Shir OM, Bäck T (2010) A reduced-cost SMS-EMOA using Kriging, self-adaptation, and parallelization. In: Multiple criteria decision making for sustainable energy and transportation systems. Springer, pp 301–311
53. Knowles JD, Corne DW (2000) Approximating the nondominated front using the Pareto archived evolution strategy. Evolut Comput 8(2):149–172
54. Kramer O, Schwefel H-P (2006) On three new approaches to handle constraints within evolution strategies. Natl Comput 5(4):363–385
55. Kruisselbrink J (2012) Evolution strategies for robust optimization. PhD Thesis, Leiden University, Leiden
56. Kursawe F (1991) A variant of evolution strategies for vector optimization. In: Parallel problem solving from nature. Springer, Berlin, pp 193–197
57. Laumanns M, Rudolph G, Schwefel H-P (1998) A spatial predator-prey approach to multi-objective optimization: a preliminary study. In: Parallel problem solving from nature - PPSN V. Springer, Berlin, pp 241–249
58. Li R (2009) Mixed-integer evolution strategies for parameter optimization and their applications to medical image analysis. PhD thesis, Leiden University, Leiden
59. Li R, Emmerich MT, Eggermont J, Bäck T, Schütz M, Dijkstra J, Reiber JH (2013) Mixed integer evolution strategies for parameter optimization. Evol Comput 21(1):29–64
60. Mahfoud S (1995) Niching methods for genetic algorithms. PhD Thesis, University of Illinois at Urbana Champaign
61. Marty T, Semet Y, Auger A, Héron S, Hansen N (2023) Benchmarking CMA-ES with basic integer handling on a mixed-integer test problem suite. In: Proceedings of the companion conference on genetic and evolutionary computation, GECCO '23 companion. Association for Computing Machinery, New York, pp 1628–1635
62. Meyer-Nieberg S, Beyer H-G (2007) Self-adaptation in evolutionary algorithms. In: Parameter setting in evolutionary algorithms. Springer, Berlin, pp 47–75
63. Ollivier Y, Arnold L, Auger A, Hansen N (2014) Information-geometric optimization algorithms: a unifying picture via invariance principles. arXiv:1106.3708v3
64. Omeradzic A, Beyer H (2023) Progress analysis of a multi-recombinative evolution strategy on the highly multimodal Rastrigin function. Theor Comput Sci 978:114179
65. Ostermeier A, Gawelczyk A, Hansen N (1993) A derandomized approach to self adaptation of evolution strategies. Technical Report TR-93-003, TU Berlin
66. Ostermeier A, Gawelczyk A, Hansen N (1994) A derandomized approach to self adaptation of evolution strategies. Evol Comput 2(4):369–380
67. Ostermeier A, Gawelczyk A, Hansen N (1994) Step-size adaptation based on non-local use of selection information. In: Parallel problem solving from nature - PPSN III. Lecture notes in computer science, vol 866. Springer, Berlin, pp 189–198

68. Preuss M (2015) Multimodal optimization by means of evolutionary algorithms. Natural computing series. Springer International Publishing, Chem
69. Rechenberg I (1973) Evolutionsstrategie: optimierung technischer systeme nach prinzipien der biologischen evolution. Friedrich Frommann Verlag, Stuttgart-Bad Cannstatt
70. Ros R, Hansen N (2008) A simple modification in CMA-ES achieving linear time and space complexity. In: Parallel problem solving from nature - PPSN X. Lecture notes in computer science, vol 5199. Springer, Berlin, pp 296–305
71. Rudolph G (1992) On correlated mutations in evolution strategies. In: Parallel problem solving from nature - PPSN II. Elsevier, Amsterdam, pp 105–114
72. Rudolph G (1994) An evolutionary algorithm for integer programming. In: Parallel problem solving from nature-PPSN III. Springer, Berlin, pp 139–148
73. Rudolph G (1997) Convergence properties of evolutionary algorithms. Kovac Hamburg, Hamburg
74. Salimans T, Ho J, Chen X, Sidor S, Sutskever I (2017) Evolution strategies as a scalable alternative to reinforcement learning
75. Schönenberger L, Beyer H (2024) Success rate of evolution strategies on the multimodal Griewank function. In: IEEE congress on evolutionary computation, CEC 2024, Yokohama, June 30–July 5, 2024. IEEE, pp 1–8
76. Schönenberger L, Beyer H-G (2024) On a population sizing model for evolution strategies in multimodal landscapes. IEEE Transactions on Evolutionary Computation, (online first), https://doi.org/10.1109/TEVC.2024.3419931
77. Schönemann L, Emmerich M, Preuß M (2004) On the extinction of evolutionary algorithm subpopulations on multimodal landscapes. Informatica (Slovenia) 28(4):345–351
78. Schumer M, Steiglitz K (1968) Adaptive step size random search. IEEE Trans Autom Control 13(3):270–276
79. Schwefel H-P (1965) Kybernetische Evolution als Strategie der experimentellen Forschung in der Strömungstechnik. Technische Universität, Berlin
80. Schwefel H-P (1987) Collective phenomena in evolutionary systems. In: Checkland P, Kiss I (eds) Problems of constancy and change – the complementarity of systems approaches to complexity, proceedings of the 31st annual meeting, vol 2. Int'l Soc. for General System Research, Budapest, pp 1025–1033
81. Schwefel H-PP (1993) Evolution and optimum seeking: the sixth generation. Wiley, Hoboken
82. Shir OM (2008) Niching in derandomized evolution strategies and its applications in quantum control. PhD thesis, Leiden University, Leiden
83. Shir OM, Bäck T (2009) Niching with derandomized evolution strategies in artificial and real-world landscapes. Nat Comput Int J 8(1):171–196
84. Shir OM, Emmerich M (2024) Multi-objective mixed-integer quadratic models: A study on mathematical programming and evolutionary computation. IEEE Trans Evolut Comput 29(3):661–675, https://doi.org/10.1109/TEVC.2024.3374519
85. Shir OM, Yehudayoff A (2020) On the covariance-Hessian relation in evolution strategies. Theor Comput Sci 801:157–174
86. Shir OM, Emmerich M, Bäck T (2010) Adaptive niche-radii and niche-shapes approaches for niching with the CMA-ES. Evol Comput 18(1):97–126
87. Shir OM, Roslund J, Whitley D, Rabitz H (2014) Efficient retrieval of landscape Hessian: Forced optimal covariance adaptive learning. Phys Rev E 89(6):063306
88. Teytaud O (2011) Lower bounds for evolution strategies. Theory Random Search Heurist 1:327–354
89. Thomaser A, Vogt M-E, Bäck T, Kononova AV, Stein NV, Marcelloni F, Lam H, Cottrell M, Filipe J (2023) Optimizing CMA-ES with CMA-ES. In: IJCCI, pp 214–221
90. Tušar T, Brockhoff D, Hansen N (2019) Mixed-integer benchmark problems for single- and bi-objective optimization. In: Proceedings of the genetic and evolutionary computation conference, GECCO '19. Association for Computing Machinery, New York, pp 718–726

91. van Rijn S, Wang H, van Stein B, Bäck T (2017) Algorithm configuration data mining for CMA evolution strategies. In: Proceedings of the genetic and evolutionary computation conference, pp 737–744
92. Van Rijn S, Doerr C, Bäck T (2018) Towards an adaptive cma-es configurator. In: International conference on parallel problem solving from nature. Springer, pp 54–65
93. Wang H, Emmerich M, Bäck T (2019) Mirrored orthogonal sampling for covariance matrix adaptation evolution strategies. Evol Comput 27(4):699–725
94. Wierstra D, Schaul T, Glasmachers T, Sun Y, Peters J, Schmidhuber J (2014) Natural evolution strategies. J Mach Learn Res 15(27):949–980
95. Zepko G, Shir OM (2024) Addressing unboundedness in quadratically-constrained mixed-integer problems. arXiv preprint arXiv:2405.05978

Evolutionary Algorithms

6

David Corne and Michael A. Lones

Contents

Introduction . . . 126
Principal Algorithms . . . 128
Genetic Algorithms . . . 128
Evolution Strategies and Evolutionary Programming . . . 130
Estimation of Distribution Algorithms . . . 131
Differential Evolution . . . 131
Performance Comparisons . . . 132
Common Variants . . . 132
Alternative Representations . . . 133
Hybridization with Local Search . . . 134
Multimodal Optimization . . . 135
Multi-objective Optimization . . . 135
Dynamic Optimization . . . 137
Coevolving Solutions . . . 137
Applying Evolutionary Algorithms . . . 138
Choosing a Methodology . . . 138
Choosing Parameters . . . 138
Software Tools . . . 139
Case Studies . . . 139
Evolutionary Algorithms at Large . . . 139
Using Niching and Coevolution to Understand Gene Regulation . . . 140
Evolving Classifiers for Parkinson's Disease Diagnosis . . . 142
Conclusion . . . 142
Cross-References . . . 143
References . . . 143

D. Corne (✉) · M. A. Lones
Heriot-Watt University, Edinburgh, UK
e-mail: d.w.corne@hw.ac.uk; mal1@hw.ac.uk

© Springer Nature Switzerland AG 2025

R. Martí et al. (eds.), *Handbook of Heuristics*,
https://doi.org/10.1007/978-3-032-00385-0_27

Abstract

Evolutionary algorithms (EAs) are population-based metaheuristics, originally inspired by aspects of natural evolution. Modern varieties incorporate a broad mixture of search mechanisms, and tend to blend inspiration from nature with pragmatic engineering concerns; however, all EAs essentially operate by maintaining a population of potential solutions and in some way artificially 'evolving' that population over time. Particularly well-known categories of EAs include genetic algorithms (GAs), Genetic Programming (GP), and Evolution Strategies (ES). EAs have proven very successful in practical applications, particularly those requiring solutions to combinatorial problems. EAs are highly flexible and can be configured to address any optimization task, without the requirements for reformulation and/or simplification that would be needed for other techniques. However, this flexibility goes hand in hand with a cost: the tailoring of an EA's configuration and parameters, so as to provide robust performance for a given class of tasks, is often a complex and time-consuming process. This tailoring process is one of the many ongoing research areas associated with EAs.

Keywords

Population-based · Genetic algorithms · Evolutionary programming · Evolution strategies

Introduction

Evolutionary algorithms (EAs) are population-based metaheuristics. Historically, the design of EAs was motivated by observations about natural evolution in biological populations. Recent varieties of EA tend to include a broad mixture of influences in their design, although biological terminology is still in common use. The term "EA" is also sometimes extended to algorithms that are motivated by population-based aspects of EAs but which are not directly descended from traditional EAs, such as scatter search. The term evolutionary computation is also used to refer to EAs but usually as a generic term that includes optimization algorithms motivated by other processes but which generally involve a population of potential solutions adapting and improving over time. This includes algorithms inspired by other natural processes, such as ant colony optimization (ACO—inspired by the collective problem-solving behavior of social insects) and artificial immune systems (AIS—inspired by the adaptive molecular processes involved in the human immune system's ability to recognize and destroy harmful agents). It also includes algorithms inspired by social behavior, such as particle swarm optimization (PSO); however, in recent years, the convention has shifted toward using the term "swarm intelligence" to describe PSO, ACO, and other algorithms inspired more by interaction than evolution. Although these algorithms often resemble EAs, this is

not always the case, and they will not generally be discussed in this chapter. For a discussion of their commonalities and differences, the reader is referred to [1].

Over the years, EAs have become an extremely rich and diverse field of study. In part this arises from their inherent design flexibility—there are innumerable ways to specify an algorithm that operates according to the core concepts of evolution. In part this also arises from the fact that their performance tends to be highly problem dependent—an EA that works well on one task may well perform poorly on another, even quite similar, task. Arising from these factors and others, the sheer number of publications in this area is challenging for people new to the field. To address this, this chapter aims to give a concise overview of EAs and their application, with an emphasis on contemporary rather than historical usage.

The main classes of EA in contemporary usage are (in order of popularity) genetic algorithms (GAs), evolution strategies (ESs), differential evolution (DE), and estimation of distribution algorithms (EDAs). Multi-objective evolutionary algorithms (MOEAs), which generalize EAs to the multiple objective case, and memetic algorithms (MAs), which hybridize EAs with local search, are also popular, particularly within applied work. Special-purpose EAs, such as genetic programming (GP) and learning classifier systems (LCS), are also widely used. These are all discussed in this chapter.

Although these algorithms differ from each other in a number of respects, they are all based around the same core process. Each of them maintains a population of search points (known variously as candidate solutions, individuals, chromosomes, or agents). These are typically generated at random and are then iteratively evolved over a series of generations by applying variation operators and selection. Variation operators generate changes to members of the population, i.e., they carry out moves through the search space. After each generation, the objective value (or *fitness*) of each search point is calculated. Selection then removes the search points with the lowest objective values, meaning that only the best search points are maintained, and new search points are always derived from these. It is this combination of maintaining a population of search points and carrying out selection between search points that differentiates EAs from most other metaheuristics.

Each EA uses its own distinctive set of variation operators, which are sometimes inspired by the mutative and recombinative processes that generate diversity in biological evolution. The mutation operator resembles the generation of "moves" in other optimization algorithms and involves sampling the neighborhood around an existing search point in some fashion. A typical approach would be to randomly change one component of a solution, though a particular EA may use more than one kind of mutation operator. The recombination (or *crossover*) operator explores the region between two or more search points, for example, by randomly reassembling the components that make up two existing solutions. This process of searching the region between existing search points is also a distinctive feature of EAs, though its practical utility depends upon the structure of the search space. Some EAs, particularly "evolutionary programming" and older varieties of evolution strategy, do not use recombination at all.

Principal Algorithms

All EAs share certain common features, including in particular the broad concepts of *variation* and *selection*, as introduced in section "Introduction", operating over a series of *generations*. More fundamentally, all EAs work by seeking the "best" (in some sense) solutions they can find to a given optimization task. Put in another way, an EA is almost always used to find the solution data-structure x which optimizes a given function $f(x)$. For example, the task at hand may be straightforward numerical function optimization, where the candidate solution data-structure is a binary string that is interpreted as a list of real-valued parameters, and $f(x)$ is a mathematical function over those parameters whose result is a scalar value. Alternatively, the task at hand may be to find an ideal schedule for a collection of manufacturing tasks at a factory; in this case, the candidate solution data-structure might be an ordered sequence of task identifiers, and $f(x)$ would be a program that simulates the schedule from the sequence, returning a vector of quality indicators including cost, time, and risk. Whatever the nature of the underlying data-structure being "evolved" and whatever the nature of $f(x)$, we can express a "canonical" EA in pseudocode as follows:

Preliminaries: determine a suitable way to represent solutions as data-structures, prepare the fitness function, and set $g = 0$.

Step 1:	set $g = 0$, and generate and evaluate an initial population Sg of candidate solutions;
Step 2:	apply *selection* operators to produce a set of "parent" solutions P;
Step 3:	apply variation operators to P to produce a set of "chil" solutions C, and evaluate the fitness of each one;
Step 4:	apply population update operations to the union of Sg and C to produce $S_{g+}1$, and increment g
Step 5:	if a termination criterion has not been reached, go to Step 2; otherwise, finish and return the fittest member of the population.

In the above pseudocode, steps 2–5 constitute a "generation," whose major components are selection, variation, and then the "population update" step that leads to renewal of the population, now ready to enter the next generation. Individual EAs vary greatly in each of these steps, including the "preliminaries" step. In the remainder of this section, we provide brief introductions to the principal classes of EA that are in current use and then discuss existing understanding of their performance and applicability.

Genetic Algorithms

Genetic algorithms, or GAs, are one of the earliest forms of EAs and remain widely used. Candidate solutions, often referred to as *chromosomes* in the GA literature, comprise a vector of decision variables. Nowadays, these variables tend to have a

direct mapping to an optimization domain, with each decision variable (or *gene*) in the GA chromosome representing a value (or *allele*) that is to be optimized. However, it should be noted that historically GAs worked with binary strings, with real values encoded by multiple binary symbols, and that this practice is still sometimes used. GA solution vectors are either fixed-length or variable-length, with the former the more common of the two.

Given their long history, genetic algorithm implementations vary considerably. However, it is fairly common to use a mutation operator that changes each decision variable with a certain probability (values of 4–8% are typical, depending upon the problem domain). When the solution vector is a binary string, the effect of the mutation operator is simply to flip the value. More generally, if the solution vector is a "$k-$ary" string, in which each position can take any of a discrete set of k possible values, then the mutation operator is usually designed to choose a random new value from the available alphabet. If the solution vector is a string of real-valued parameters within a set range, the new value may be sampled from a uniform distribution in that range, or it may be sampled from a nonuniform (e.g., Gaussian) probability distribution centered around the current value. The latter is generally the preferred approach, since it leads to less disruptive change on average. Recombination is typically implemented using two-point or uniform crossover. Recombination tends to be applied at a high rate (e.g., 0.7, typically called the *crossover rate*); this means, for example, that when a variation operator is to be applied, the chance of this operator being a crossover operator will be 0.7; otherwise (depending on the algorithm), the operator may be the application of mutation to a single parent or simply copying a single parent. Two-point crossover chooses two *parent* solutions and two *crossover points* within the solutions. The values of the decision variables lying between these two points are then swapped to form two *child* solutions. Historically, "one-point" crossover was popular, which produced the first (second) child by copying the first (second) parent up to a randomly chosen point and then copied the second (first) parent thereafter. However, though more convenient for theoretical analyses, one-point crossover is rarely used in practice these days. Meanwhile, in uniform crossover, crossover points are created at each decision variable with a given probability. Other forms of crossover have also been used in GAs. Examples include line crossover and multi-parent crossover. Other variation operators, such as inversion, have been found useful for some problems.

Various forms of selection are used with GAs. Rank-based or tournament selection are generally preferred, since they maintain exploration better than the more traditional fitness-proportionate selection (e.g., roulette-wheel selection). Note, however, that the latter is still widely used. Rank-based selection involves ranking the population in terms of objective value. Population members are then chosen to become parents with a probability proportional to their rank. In tournament selection, a small group of solutions (typically three or four) are uniformly sampled from the population, and those with the highest objective value(s) become the parent(s) of the next child solution that is created. Tournament selection allows selective pressure to be easily varied by adjusting the tournament size.

Evolution Strategies and Evolutionary Programming

Evolutionary programming (EP) and evolution strategies (ES) also have a long history, starting earlier than the development of GAs. First described in the 1960s and 1970s, respectively (see [2] for a comprehensive account of the historical development), early ES and EP used only single-parent operators in the variation step (i.e., mutation). Meanwhile, EP was singular in focusing on using finite-state machines as the evolving data-structures, while ES soon introduced a recombination operator and championed the exploration of so-called "$m + n$" schemes, whereby a population of m parents would generate n children and the best n of the combined parents and children becomes the next generation of parents. In current research and practice, modern formulations of EP focus on numerical optimization over real-valued parameters and still eschew multi-parent operators in favor of paying attention to careful design of the mutation operator(s). Of the two styles, however, ES is more widely researched and deployed in practice, and particular variants of ES have shown particular prowess in numerical function optimization. Modern ESs incorporate strategies that carefully guide how the mutation operator is applied to each decision variable. Unlike GAs, ESs mutate every decision variable at each application of the operator and do so according to a set of *strategy parameters* that determine the magnitude of these changes. Strategy parameters Strategy parameters usually control characteristics of probability distributions from which the new values of decision variables are drawn.

It is standard practice to adapt strategy parameters over the course of an ES run, the basic idea being that different types of move will be beneficial at different stages of search. Various techniques have been used to achieve this adaptation. Some of these involve applying a simple formula, e.g., the 1/5th rule, which involves increasing or decreasing the magnitude of changes based on the number of successful mutations that have recently been observed. Others are based around the idea of self-adaptation, which involves encoding the strategy parameters as additional decision variables and hence allowing evolution to come up with appropriate values. However, the most widely used contemporary approach is covariance matrix adaptation (CMA-ES), which uses a mechanism for estimating the directions of productive gradients within the search space and then applying moves in those directions. In this respect, CMA-ES has similarities with gradient-based optimization methods.

ESs use different recombination operators to GAs and often use more than two parents to create each child solution. For example, intermediate recombination gives a child solution the average values of each decision variable in each of the parent solutions. Weighted multi-recombination is similar but uses a weighted average, based on the fitness of each parent. Also unlike GAs, ESs tend to use deterministic rather than probabilistic selection mechanisms, whereby the best solutions in the population are always used as parents of the next generation.

Estimation of Distribution Algorithms

Like ESs, estimation of distribution algorithms [3], or EDAs, make use of probability distributions. However, rather than using them to describe a distribution of next moves, as an ES does, EDAs use them to describe a distribution of next sample solutions. The basic mechanism is quite simple. As for most EAs, the initial population of solutions is (typically) sampled from a uniform distribution. Selection is then used to remove the poorer members of this population, and a probability distribution is then constructed that attempts to model the statistics of the relatively high-fitness sample solutions that remain in the population. Importantly, this distribution is constructed in such a way that it "generalizes" the population members suitably well. The next generation of solutions is then constructed by sampling from this distribution. So, if the distribution was highly peaked around the existing samples, for example, the next generation would explore very little beyond the previous one. This pattern of building a distribution, sampling, and selection is then iterated in the usual generational fashion, with the hope that the final probability distribution will characterize solutions that are, or are close to, globally optimal.

While an EDA may be used with any kind of probability distribution, in practice, it is necessary to choose a distribution that induces an appropriate trade-off between efficiency and expressiveness. More expressive models, such as Bayesian networks and Markov models, can capture dependencies between decision variables but can be expensive to construct and sample from. Simple univariate distributions, by comparison, are cheap to build and sample from but are unable to capture dependencies between variables. This trade-off is reflected in the range of EDAs in common use. Population-based incremental learning (PBIL) and the compact genetic algorithm (CGA) are both examples of computationally efficient EDAs that build simple univariate models based on discrete variables. Because of their simplicity, they can be applied to large problem instances. Bayesian optimization algorithms (BOAs) lie at the other end of the spectrum: these can express dependencies between variables, and certain varieties can be applied to both discrete and continuous variables, but they are far more demanding of computational resources.

Differential Evolution

Differential evolution [4, 5], or DE, is a relatively recent EA formulation which uses a mechanism for adaptive search that does not make use of probability distributions. While its basic mechanism is similar to a GA, its mutation operator is quite different, using a geometric approach that is motivated by the moves performed in the Nelder-Mead simplex search method. This involves selecting two existing search points from the population, taking their vector difference, scaling this by a constant F, and then adding this to a third search point, again sampled randomly from the

population. Following mutation, DE's crossover operator recombines the mutated search point (the *mutant vector*) with another existing search point (the *target vector*), replacing it if the child solution (known as a *trial vector*) is of equal or greater objective value. There are two standard forms of crossover [6]: exponential crossover and binomial crossover, which closely resemble GA two-point crossover and uniform crossover, respectively. The comparisons between target vector and trial vector play the same role as the selection mechanism in a GA or ES. Since DE requires each existing solution to be used once as a target vector, the whole population is replaced in the course of applying crossover.

An advantage of using simplex-like mutations in DE is that the algorithm is largely self-adapting, with moves automatically becoming smaller in each dimension as the population converges. More generally, the authors of the method have claimed that this sort of self-adaptation means that the size and direction of moves are automatically matched to the search landscape, a phenomenon they term *contour matching*. When compared to CMA-ES, for example, this means that the algorithm has few parameters and is relatively easy to implement.

Performance Comparisons

Fair comparisons of optimization algorithms are inherently challenging [7] and arguably unachievable. Nevertheless, there have been some attempts to understand the comparative performance of different EAs, particularly within the domain of continuous optimization. In particular, a series of workshops held at two of the largest annual EA conferences, CEC and GECCO, have sought to define benchmark suites of real-valued function optimization problems suitable for comparing EAs (and other optimizers) [8–10]. Using these benchmarks, a number of authors have shown their algorithms to perform better than others, including variants of CMA-ES and DE (see [4]). It should be borne in mind that these are not exhaustive studies, either in terms of problems or approaches. The "no free lunch theorem" (NFLT) [11] may also be considered when attempting to generalize these results to a wider spectrum of problems, although, in itself, the NFLT does not apply in the case of comparisons based on the standard suites of test problems (since those suites are not closed under permutation [12]). A nice example of the perils of comparison study in this field is shown by a recent study that showed how quite different conclusions could be drawn from a comparative study by changing minimization problems into maximization problems [13].

Common Variants

The general purpose EAs introduced in the last section are applicable to a wide range of problems. However, over the course of EA history, algorithmic variants have been developed to deal with the characteristics of particular categories of problem. Some

of these categories are quite broad, for example, problems with multiple solutions. Others are more specific, such as discrete optimization problems. In this section, we discuss a number of these EA variants, focusing on those which are commonly used to solve real-world optimization problems.

Alternative Representations

In common with other optimization algorithms, most EAs are designed to work with and optimize vectors (or, equivalently, lists or arrays) of decision variables. Solutions to many kinds of problems can be represented, either directly or indirectly, in this form. However, EAs are not limited to working with vectors, and there are often advantages to working directly with representations that are more natural for the problem domain: for example, matrices [14], trees [15], graphs [16], rule sets, etc. The general approach is the same as for the EAs discussed in the previous section, except that specialized initialization routines and variation operators are used to randomly create, mutate, and recombine instances of the appropriate solution representation. These variants are typically based around GAs or ESs, since these two classes of EA can be readily adapted to use alternative solution representations. Nevertheless, DE [17] and EDA [18] have also been used successfully with other representations.

Genetic programming (GP) [15] is a well-known GA variant in which each candidate solution is a *program*; in early GP this was invariably achieved by encoding programs as tree-structures, in which each "node" in the tree corresponds to a function whose input parameters are the results returned by its child-nodes and which itself returns a result to its parent nodes (if any). GP is still mostly used to optimize computer programs or mathematical expressions, often expressed as tree-structures. However alternative ways to encode programs are now often explored in GP, such as so-called linear GP, in which programs are represented as a sequence of parameterized instructions interacting via registers. A particularly common current use of GP is *symbolic regression*, which involves finding a mathematical expression that fits a particular data set. Unlike standard mathematical approaches to regression, such as curve fitting, GP makes relatively few assumptions about the function that generated the data, allowing a wide exploration of the space of possible solutions. GP is also widely used for solving classification problems. More generally, the GP community is interested in automatic programming, i.e., finding computer programs that solve a particular task, and there are many variants of GP that use particular forms of program representation. See [19] and [20] for overviews.

Some EAs work with two different solution representations, using one of these when creating and manipulating search points, the other when evaluating search points, and a mapping process that converts the former into the latter [21]. Many of these approaches are motivated by biology, and hence this process is known as a *genotype-phenotype* mapping, with the representation used during search termed the *genotype* and the representation used for evaluation termed the

phenotype. This approach can be used when the natural representation for a domain is not well suited to being evolved, i.e., where mutation and recombination do not lead to productive solutions. This approach has also been widely used for generating complex structures, such as large neural networks [22], where a genotype representation can be chosen that compresses repetitive features such as symmetry and modularity. This is arguably an area in which EAs benefit from their relationship to biology, since biology provides a ready source of information on how to represent complex structures in an evolvable way.

Hybridization with Local Search

EAs are often considered to be global search algorithms, since they explore a relatively wide region of the search space and are relatively good at escaping local optima. However, their convergence to optimal solutions can be relatively slow when compared to local search algorithms. For this reason, EAs are often hybridized with local search, using it to locally optimize members of the population at regular intervals, hence speeding up convergence. Although the resulting hybrid algorithms are known by various names, the term *memetic algorithm* [23] (MA) has become popular in recent years. Memetic, in this case, refers to an analogy between the role of local search in these algorithms and the role of within-generation learning in biological systems, though the majority of memetic algorithms have no particular biological justification beyond this. In principle, these algorithms may involve hybridizing an EA with any or with multiple local search algorithms and consequently are very diverse. For a recent review, see [24].

Beyond hastening convergence, MAs are also seen as a means of introducing domain knowledge into EAs. This is done through the use of specialized local search operators that are relevant to a particular domain. For example, this approach underlies the success of MAs in the area of discrete optimization [25]. Related to the idea of problem specialization in memetic algorithms is the concept of hyper-heuristics in EAs, which has developed some traction in recent years [26]. Generally speaking, hyper-heuristics are applicable to domains in which a variety of so-called low-level heuristics exist (or can be invented) to build quick, good solutions. In the job shop scheduling problem, for example, "shortest-process-time" and "earliest-available-machine" are two examples of low-level "dispatch" heuristics that, when iterated, can build a single solution quickly. Hyper-heuristics are essentially mechanisms used to explore *combinations* of such lower-level heuristic strategies. The term hyper-heuristics is also used to describe the cases in which an EA (often GP) both creates anew and combines such low-level heuristics. In a nutshell, the broad idea of hyper-heuristics is to search a space of algorithms that can solve a class of problems, rather than search the space of solutions directly for a single problem instance.

Multimodal Optimization

An advantage of maintaining a population of search points is that EAs can be readily applied to multimodal optimization problems in which there is more than one solution of interest. However, effective multimodal optimization generally requires some modification to the EA's underlying behavior, since, although EAs explore diverse areas of the search space, they eventually converge to fairly small areas. This behavior can be mitigated, to an extent, by varying the global selection pressure used when choosing parents; for example, in the case of tournament selection, the tournament size can be made small, increasing the likelihood that less fit members of the population will contribute to the next generation of search points. While this increases exploration, it decreases exploitation: meaning that multiple solutions may be found, but they are less likely to be optimal. Since EAs are stochastic, and there is the potential for them to converge on different optima during different runs, another simple approach to finding multiple solutions is to run an EA multiple times. However, there is no guarantee that all optima will be explored, and algorithmic biases (such as the manner in which the initial population is generated) may favor some solutions over others.

A more effective approach is to use some kind of *niching* technique [27]. These aim to preserve global diversity in the population, but without lowering local selective pressure. Niching approaches are motivated by the biological concept of evolutionary niches, in which species compete within a niche but not between niches. In optimization terms, a niche is a local region within the search space that contains a solution of interest, and the aim is for the population to be distributed across all the relevant niches. Niching has been studied for some time in GAs, and techniques include crowding [28], fitness sharing [29], spatial segregation [30], and clustering [14]. For comparative studies, see [6] and [8]. A simple but effective example of niching is probabilistic crowding [28]. This works at the operator level and always replaces parent solutions with their children, meaning that search points are usually replaced with nearby search points and the population remains spread across the search space. Similar techniques have also been developed for use in DE. Niching is less commonly used in ESs, in part due to their use of smaller populations, though examples do exist [31]. It is also common to use multi-objective evolutionary algorithms (see below) to solve multimodal problems, since these algorithms often have effective mechanisms for preserving population diversity.

Multi-objective Optimization

Multi-objective EAs, or MOEAs, are used to solve problems which have multiple and often conflicting objectives. A central concept for MOEAs, and multi-objective optimization in general, is that of a non-dominated solution. This is a solution

which is no worse than any of the other solutions within the population when all objectives are taken into account, and the aim of an MOEA is to build and maintain a population of non-dominated solutions that cover all trade-offs between the objectives. This is known as the Pareto optimal front Pareto optimal front. Exactly how this is achieved varies between MOEAs. However, a well-known example is NSGA-II (non-dominating sorting genetic algorithm) [32]. Prior to selection, NSGA-II ranks all solutions in terms of dominance: those which are non-dominated are assigned rank 1, those which are only dominated by rank 1 solutions are assigned rank 2, etc. The population is then ordered by rank, and by a measure of crowding distance within ranks, and the first half of the ordered population is copied directly into the next generation. The remainder of the population is then filled by breeding, with parents selected from the higher ranks. Hence, non-dominated solutions are preserved between generations, and new solutions are explored via interbreeding, resulting in a diverse set of non-dominated solutions that approximate the Pareto optimal front.

The core challenge faced by multi-objective optimization (and absent from single-objective optimization) is how to rank candidate solutions in a way that leads to effective selection pressure, especially when the entire population (or most of it) may be non-dominated. Another way in which multi-objective optimization differs from single-objective optimization is in the nature of the "best-so-far solution." In single-objective optimization, the "best-so-far" solution is trivial to define and to keep track of; in multi-objective optimization, the situation is vastly different: the solution is, technically, the entire Pareto front, which is usually a set of solutions, whose cardinality may vary from one to the entire search space. For MOEAs, this leads to certain technical issues which are invariably addressed by maintaining an *archive* of non-dominated solutions; this archive simply keeps track of the "best-so-far" approximation to the Pareto front but is also often used as a reservoir for selection of parents. Approaches to the main challenge—how to apply effective selection pressure among the current population—are far more varied. While the approach taken by NSGA-II, as detailed above, is a common and quite successful one, many other styles of MOEA exist, which take different approaches to this central question. In PAES [33], for example, there is only a single "current" population member. Selection is consequently simplified; however, the challenge shifts to the question of whether or not to update the current solution with a newly generated one when the two are non-dominated; PAES makes this decision with the aid of its archive, preferring to explore new areas of the search space than to stay close to solutions already in the archive. Meanwhile, a different breed of MOEAs in this respect is represented by MOEA/D [34]; bypassing the need to distinguish between non-dominated solutions for selection purposes, MOEA/D "decomposes" a multi-objective problem into many single-objective simplifications of it, each involving a different weighting of the objectives. MOEA/D conducts these single-objective searches in parallel (typically using a local search mechanism) and organizes occasional communication between them, as well as bookkeeping activities that build and maintain the archive. Effectively, each of MOEA/D's single-objective searches explores a different area of the Pareto front. There are many other approaches, and MOEAs are becoming increasingly used as it becomes recognized

that real-world problems are almost invariably multi-objective in nature. Further discussion of the latter point, as well as a first introduction to MOEAs, may be found in [35], while an example of a fairly recent review of MOEAs may be found in [36].

Dynamic Optimization

So far, our discussion of optimization has only considered problems in which the search space remains fixed. In many real-world problems, this is not the case, and various EA approaches are used to handle these situations. Dynamic optimization is an area in which EAs might be expected to perform relatively well, since the natural diversity present in their populations provides a recovery mechanism that can respond to slow changes in the optimization landscape. This is especially the case when diversity maintenance techniques are implemented, such as those already discussed in the sections on multimodal and multi-objective optimization. However, this diversity may be insufficient when the optimization targets change rapidly or abruptly. A simple solution in this situation is to inject extra diversity into the population when a change is detected, for instance, by adapting the variation operators so that larger moves are made. Detection of change can be done by reevaluating a proportion of the population, looking for significant changes in fitness.

A variety of more elaborate approaches have been developed to handle dynamic optimization in EAs. An approach inspired by biological systems is to use redundancy in the encoding of a solution. Rather than replacing components of a solution when variation operators are applied, this allows old components to become recessive, i.e., to remain present within the solution but not be expressed during evaluation. Later in the evolutionary process, these components can become reactivated, in effect providing a mechanism to backtrack to previous search points. This is particularly useful when changes in the search space are cyclic. A well-known example is the use of *multiploidy* in GAs [37], where each solution has multiple chromosomes (only one of which is dominant) and variation operators are able to move information between chromosomes. Other approaches to handling dynamic search spaces include predicting change and using multiple populations; see [38] for a recent review.

Coevolving Solutions

Coevolutionary algorithms [39] are motivated by the interactions that occur between species during the course of biological evolution and the roles these interactions are thought to play in the evolution of complex organisms. Most coevolutionary algorithms use multiple populations, one per *species*. Coevolutionary relationships in biology can be cooperative or competitive. The latter class are particularly well known and are encapsulated in the idea of predator-prey patterns of evolution, where an arms race between two species can lead to the rapid emergence of complex

adaptations. Similar ideas have been explored in EAs, the classic example being the coevolution of sorting networks and sorting algorithms [40]: the discovery of harder problems (the sorting networks in the first population) leads to selective pressure to discover better solutions (the sorting algorithms in the second population), which leads to selective pressure to discover harder problems and so on. Competitive coevolution can be used to solve hard problems and is also useful in circumstances where a fitness function cannot be defined. However, competitive coevolution is known to be difficult to control, and pathological situations can lead to ineffective search. See [39] for a review.

Cooperative coevolution, by comparison, is seen as a useful mechanism for breaking down large problems into more tractable chunks [41]. The idea is that a solution to a problem is divided into sub-components. Each of these sub-components is then evolved in a separate population, with its objective value dependent upon how compatible it is with sub-components being evolved in other populations. Following this, the coadapted sub-components are then assembled to form a complete solution. In [42], for example, the authors describe how a cooperative coevolutionary variant of DE can be used to solve numerical optimization problems with up to 1000 variables. Cooperative coevolution can also take place within a single population. An example of this is a Michigan-style learning classifier system (LCS), a form of EA that coevolves a population of rules that can collectively solve difficult problems in classification and machine learning [43].

Applying Evolutionary Algorithms

Choosing a Methodology

It can be difficult to choose which EA to use for a particular task, since there are many different EAs in common use and relatively little in the way of objective comparative guidance. In practice, it may be necessary to try out different EAs to find out which is the best match to a problem, especially when the problem is poorly understood. However, given whatever is known about the problem at hand, it might be possible to leverage existing understanding of the strengths and weaknesses of particular algorithm frameworks. Some guidance on this matter is available in studies of comparative performance mentioned at the end of section "Introduction". It is hoped that section "Principal Algorithms" also provides useful pointers if the problem is multimodal, multi-objective, dynamic, or unusually large and complex. It is also notable that multi-objective and memetic algorithms, in particular, have become popular for solving difficult real-world problems.

Choosing Parameters

EAs invariably have many parameters, and once an algorithm has been selected, it is normal to carry out parameter tuning in order to obtain a better fit between the algorithm and the problem. It can be challenging to obtain optimal parameter

settings, since parameters are typically both numerous and not independent of one another. DE, for instance, is notable for having relatively few parameters, and this is often portrayed as a strength of the method. However, EAs are relatively forgiving, and good performance is likely to be possible with nonoptimal parameter settings. Nevertheless, guidance is available for choosing the settings of certain parameters [44], and a number of techniques have been developed for automating the choice of parameter settings [45].

Software Tools

Tools support is an important issue for many practitioners, and a particular EA methodology is likely to be more appealing if it has a mature supported implementation. Tools support is also important if it is necessary to handcraft a new algorithm to solve a particular problem, and in this situation the language used by the tool may also be a significant concern. Table 1 summarizes the features of some of the better known EA tools. GAs are widely supported by all of these. ES support is also widely available, though EvA2 stands out in this regard, with implementations of a wide range of ES variants. DE and EDAs are more recent algorithms, and this is reflected by fewer mature tools. However, EvA2 is again notable for having an implementation of BOA and other EDAs. GP support is offered by a number of these tools, with ECJ implementing a particularly wide range of GP variants. Most also offer support for multimodal and multi-objective approaches, though MOEA Framework stands out for the latter. All of these tools allow custom code to be written. Most use Java or C++, though DEAP is notable as a mature Python implementation and HeuristicLab is available for C# users.

Case Studies

Evolutionary Algorithms at Large

Now more than half a century since the first appearance of "EA"-style algorithms in the research literature (widely considered to be [46]), EAs have penetrated almost every area of science and industry and are regularly used in solving an immense range of optimization problems.

In terms of broad classes of problem, EAs have enabled practitioners and researchers to make particular headway in *combinatorial optimization* which, unlike *numerical optimization*, for example, was hitherto poorly served by classical algorithms. In combinatorial optimization, the task is typically to find an ideal permutation (or otherwise constrained arrangement) of a set of entities—such as a sequence of customer visits for a delivery vehicle or a sequence of production tasks for each of a set of machines in a factory. Before EAs, the primary approach used to solve such problems was integer programming and constraint programming (and variations thereon), which tend to require a complex (and often tortuous) problem reformulation step. EAs, however, provide a far more accessible and

Table 1 Open-source EA frameworks

Tool	Language	Summary
DEAP https://code.google.com/p/deap/	Python	Distributed Evolutionary Algorithms in Python offers good support for GAs and ESs. Also implements GP and MOEAs
ECJ http://cs.gmu.edu/~eclab/projects/ecj/	Java	Continuously developed since 1998, Evolutionary Computation in Java has particular strength in GP but also implements GAs, DE, and MOEAs
EO http://eodev.sourceforge.net	C++	Evolving Objects is an established general-purpose EA library with implementations of GAs, GP, ESs, and EDAs
EvA2 http://www.ra.cs.uni-tuebingen.de/software/JavaEvA/	Java	EvA2 is a general-purpose EA framework but has particular strengths in ESs and EDAs, including BOA
HeuristicLab http://dev.heuristiclab.com/	C#	HeuristicLab implements many of the common EA varieties and also has support for other population-based and local search metaheuristics
MOEA framework http://www.moeaframework.org/	Java	A relatively new EA framework with considerable strength in MOEAs and multi-objective variants of DE and GP
OpenBEAGLE https://code.google.com/p/beagle/	C++	A long established EA framework with good support for GAs and ESs. Also implements GP and NSGA-II

flexible approach to addressing such problems and are now commonly used in practice for combinatorial tasks such as vehicle routing, job shop scheduling, and facility allocation [47].

Meanwhile, beyond combinatorial optimization, the inherent flexibility of EAs has led to their use, to at least some extent, in every conceivable area of science, enterprise, and industry in which one or more important tasks can be formulated in terms of optimization and/or design. To name just a few of the areas in which EAs have had much impact, we can list aeronautical and automotive design [48], bioinformatics and biotechnology [49], chemical engineering [50], creative pursuits [51], finance and investment [52], manufacturing [53], and structural design [54]. To select a small number of case studies could not serve to characterize the true diversity of applied EAs. We therefore duck that challenge and take the liberty of concluding this chapter by providing two case studies from the authors' recent work, illustrating how some EA approaches are being applied to diverse and challenging optimization problems in just one corner of science.

Using Niching and Coevolution to Understand Gene Regulation

Understanding gene expression is fundamental to understanding living processes. The expression of each gene in an organism is determined by the binding of special proteins, called transcription factors, within a region of DNA upstream of its coding region. In higher organisms, such as humans, this regulatory region

typically contains around 5–10 transcription factor binding sites. Characterizing these bindings sites, both individually and in combination, is a fundamental part of reconstructing (and ultimately controlling) the genetic networks that underlie biological function.

Identifying binding sites is often reduced to an optimization problem that involves constructing a matrix model of the occurrence of each DNA base at each position within a short region of DNA. Candidate solutions to this problem can be evaluated by scanning them along the regulatory regions of groups of genes which are known to be expressed at a certain time or within a certain cell type, looking for matches to patterns embedded in the sequences. In most cases, this is a multimodal problem, since multiple binding sites are likely to be relevant to a particular regulatory context, and it is important to be able to identify these different optima. However, these binding sites can vary quite considerably in their degree of conservation, meaning that the objective values of different optima also vary considerably.

Identifying and preserving different modes within a multimodal search space is a challenging problem, especially when they have different relative finesses. In section "Common Variants", we discussed the idea of niching within the populations of EAs as a means of addressing multimodal problems. In [14] we used an EA furnished with a particular form of niching, termed population clustering, that uses a clustering algorithm to identify and preserve the different modes present within an evolving population of solutions. This allowed a number of different binding sites to be characterized and preserved during a single run. Compared to other forms of niching, it also had the benefit of explicitly identifying these different groups of solutions, allowing the progress of search to be visualized and for clustering parameters to be dynamically modified by an expert user. Even in the absence of dynamic modification, however, this was effective at identifying the clusters of binding sites within comparatively long regions of DNA.

Identifying binding sites is only one part of the problem. Another important aim is to understand the interactions between different binding sites during gene regulation. In [55], we used coevolution to explore solutions to this problem. This involved coevolving two populations, the first containing matrix models of binding sites and the second containing Boolean expressions describing their co-occurrence within binding regions. Members of the binding site population were used as leaves within the Boolean expressions. In essence, the problem of identifying binding sites and their co-expression was decomposed into two problems which were then solved in parallel, using coevolution to provide feedback between the populations. In comparison to a more traditional approach, which would involve sequentially learning binding sites and then learning their interactions, this allows search to be directed toward solutions that interact well with other solutions during the course of search. Such solutions, in turn, are more likely to be meaningful. This approach proved effective for reverse engineering the regulatory rules underlying differential gene expression within tissues. More surprisingly, it also provided a mechanism for solving harder instances of the single binding site optimization problem, with coevolution provided a means of implicitly decomposing the matrices associated with these harder problems.

Evolving Classifiers for Parkinson's Disease Diagnosis

This second case study concerns a problem in which the optimal representation for solutions is not clear in advance, requiring experimentation with different kinds of solution representation. As discussed earlier, EAs are entirely flexible in this regard. The problem involves building diagnostic classifiers for Parkinson's disease. These are required to reach their decision based on time series movement data recorded while patients and age-matched control were undergoing clinical assessments of motor function. Motor aspects of Parkinson's are incompletely understood, making it unclear what kind of features of the data are important.

To address this, we used EAs to explore two different relatively unconstrained classifier models. Initially we considered a GP-based approach, using it to discover mathematical expressions that describe overrepresented patterns of movement embedded in short segments of the time series data, i.e., a form of symbolic regression [56]. An advantage of this approach is that the resulting expressions were relatively interpretable, allowing us to gain insight into the basis of classifications and then pass this information on to our clinical partners. In particular, analysis of the evolved expressions identified specific aspects of the closing phase of a "finger tap" movement as highly discriminatory of Parkinson's disease patients versus control. These factors alone were indeed more discriminatory than standard metrics; however, overall classification performance was not quite as good as that of trained clinicians.

We then considered a more unusual method of representing programs, artificial biochemical networks [57]. These are abstract executable models of the networks of biochemical interactions that underlie the function of biological cells. In a nutshell, they attempt to capture the representation which biological evolution has selected to optimize complex behaviors, with the hypothesis that this makes them particularly suitable for use with EAs. Meanwhile, the use of an EA to search through the space of possible artificial biochemical network classifiers represents, in itself, a major and commonly understood strength of EAs: they can be tailored and deployed effectively with relative ease despite the complexity and diversity of the structure they are being used to evolve. By using this approach, we were now able to find classifiers that produced comparable performance to that of trained clinicians [58]; with accuracy at around 90% overall, accuracy was comparable to the diagnostic accuracies found in clinical diagnosis, and significantly higher than those found in primary and non-expert secondary care.

Conclusion

In this chapter we have provided a broad introduction to and overview of evolutionary algorithms, and the many varieties of them that appear in modern research and practice. We have seen that there are a handful of key EA 'families', such as 'genetic algorithms' and 'differential evolution'; meanwhile, to some extent cutting across

the principal types of EAs, there are several algorithm features and variants (such as co-evolution, or hybridisation with local search) that are often 'grafted' into an EA, in order to boost performance on a particular class of problems (or perhaps just for research purposes). Towards the end, we considered the real-world application of EAs, reporting that they are generally highly successful, and are growing to be common tools used for optimisation tasks in science and industry. However, for someone new to EAs, an early question might well be: "There are so many types and variants of EAs – which should I use?" In this chapter, we have effectively promised that EAs are likely to be a successful approach to any given optimization task—particularly if that task is not well served by other available schemes or heuristics. However, we have not necessarily made it easy for a novice to deliver on that promise. A common theme has been the very wide variety of EA designs that have been developed, and this great variety can be a barrier for those new to the field, who aim to understand the key aspects of EAs and how best to apply them to any given task. In closing, we attempt to provide some support for the novice via the following perspective: EAs should not be seen as a collection of algorithms; instead, they are far more fruitfully viewed as an approach to the engineering of problem-specific (or problem-class specific) optimization algorithms. The key lesson for practitioners, from half a century of research and practice in EAs, is, broadly speaking, that concepts from evolution (population, selection, variation) can be very powerful tools in optimization; but to achieve ideal performance on a given task, one must be pragmatic, and often creative, about the details.

Cross-References

- A History of Metaheuristics
- Evolution Strategies
- Evolutionary Algorithms for the Inverse Protein Folding Problem
- Genetic Algorithms
- Hyper-heuristics
- Memetic Algorithms
- Multiobjective Optimization

Competing Interest Declaration The author(s) has no competing interests to declare that are relevant to the content of this manuscript.

References

1. Lones MA (2014) Metaheuristics in nature-inspired algorithms. In: Proceedings of genetic and evolutionary computation conference (GECCO 2014), workshop on metaheuristic design patterns (MetaDeeP). ACM, pp 1419–1422
2. Fogel DB (1998) Evolutionary computation: the fossil record. Wiley-IEEE Press, Piscataway

3. Hauschild M, Pelikan M (2011) An introduction and survey of estimation of distribution algorithms. Swarm Evol Comput 1(3):111–128. https://doi.org/10.1016/j.swevo.2011.08.003. Available: http://www.sciencedirect.com/science/article/pii/S2210650211000435
4. Das S, Suganthan PN (2011) Differential evolution: a survey of the state-of-the-art. IEEE Trans Evol Comput 15(1):4–31. https://doi.org/10.1109/TEVC.2010.2059031. Available: http://ieeexplore.ieee.org/xpl/login.jsp?tp=&arnumber=5601760&url=http%3A%2F%2Fieeexplore.ieee.org%2Fxpls%2Fabs~all.jsp%3Farnumber%3D5601760
5. Storn R, Price K (1997) Differential evolution – a simple and efficient heuristic for global optimization over continuous spaces. J Glob Optim 11(4):341–359. https://doi.org/10.1023/A:1008202821328. Available: http://link.springer.com/article/10.1023/A%3A1008202821328%23page-1
6. Zaharie D (2009) Influence of crossover on the behavior of differential evolution algorithms. Appl Soft Comput 9(3):1126–1138. https://doi.org/10.1016/j.asoc.2009.02.012. Available: http://www.sciencedirect.com/science/article/pii/S1568494609000325
7. García S, Molina D, Lozano M, Herrera F (2009) A study on the use of non-parametric tests for analyzing the evolutionary algorithms' behaviour: a case study on the CEC'2005 special session on real parameter optimization. J Heuristics 15(6):617–644. https://doi.org/10.1007/s10732-008-9080-4. Available: http://link.springer.com/article/10.1007/s10732-008-9080-4
8. Hansen N, Auger A, Finck S, Ros R (2010) Real-parameter black-box optimization benchmarking 2010: experimental setup. INRIA research report No. 7215. INRIA
9. Liang J, Qu B, Suganthan P, Hernández-Díaz A (2013) Problem definitions and evaluation criteria for the CEC 2013 special session on real-parameter optimization. Technical report 201212. Computational Intelligence Laboratory, Zhengzhou University, Zhengzhou, China and Nanyang Technological University, Singapore, pp 3–18
10. Tang K, Li X, Suganthan PN, Yang Z, Weise T (2009) Benchmark functions for the CEC'2010 special session and competition on large-scale global optimization. Technical report. Nature Inspired Computation and Applications Laboratory, University of Science and Technology of China
11. Wolpert DH, Macready WG (1997) No free lunch theorems for optimization. IEEE Trans Evol Comput 1(1):67–82. https://doi.org/10.1109/4235.585893. Available: http://ieeexplore.ieee.org/xpls/abs~all.jsp?arnumber=585893
12. Igel C, Toussaint M (2003) On classes of functions for which no free lunch results hold. Inf Process Lett 86(6):317–321
13. Piotrowski AP (2015) Regarding the rankings of optimization heuristics based on artificially-constructed benchmark functions. Inf Sci 297:191–201. Available: http://www.sciencedirect.com/science/article/pii/S0020025514010937
14. Lones MA, Tyrrell AM (2007) Regulatory motif discovery using a population clustering evolutionary algorithm. IEEE/ACM Trans Comput Biol Bioinform 4(3):403–414. https://doi.org/10.1109/tcbb.2007.1044. Available: http://ieeexplore.ieee.org/lpdocs/epic03/wrapper.htm?arnumber=4288066
15. Koza J (1992) Genetic programming: on the programming of computers by means of natural selection. MIT Press, Cambridge
16. Miller JF (2011) Cartesian genetic programming. https://doi.org/10.1007/978-3-642-17310-3_2
17. Veenhuis CB (2009) Tree based differential evolution. Lect Notes Comput Sci 5481:208–219
18. Kim K, Shan Y, Nguyen X, McKay RI (2014) Probabilistic model building in genetic programming: a critical review. Genet Program Evolvable Mach 15(2):115–167. https://doi.org/10.1007/s10710-013-9205-x. Available: http://link.springer.com/article/10.1007/s10710-013-9205-x
19. Poli R, Langdon W, McPhee NF (2008) A field guide to genetic programming. Published via http://lulu.com
20. Luke S (2013) Essentials of metaheuristics. Published via http://lulu.com

21. Stanley KO, Miikkulainen R (2003) A taxonomy for artificial embryogeny. Artif Life 9(2):93–130. https://doi.org/10.1162/106454603322221487. Available: http://www.mitpressjournals.org/doi/abs/10.1162/106454603322221487 (pages 94 and 95)
22. Floreano D, Dürr P, Mattiussi C (2008) Neuroevolution: from architectures to learning. Evol Intell 1(1):47–62. https://doi.org/10.1007/s12065-007-0002-4. Available: http://link.springer.com/article/10.1007/s12065-007-0002-4
23. Moscato P (1989) On evolution, search, optimization, genetic algorithms and martial arts: towards memetic algorithms. Caltech concurrent computation program, C3P report 826
24. Neri F, Cotta C (2012) Memetic algorithms and memetic computing optimization: a literature review. Swarm Evol Comput 2:1–14. https://doi.org/10.1016/j.swevo.2011.11.003. Available: http://www.sciencedirect.com/science/article/pii/S2210650211000691
25. Hao J (2012) Memetic algorithms in discrete optimization. In: Neri F, Cotta C, Moscato P (eds) Handbook of memetic algorithms. Springer, Berlin/Heidelberg. https://doi.org/10.1007/978-3-642-23247-3_6
26. Ross P (2005) Hyper-heuristics. In: Search methodologies. Springer, Berlin, pp 529–556
27. Singh G, Deb K (2006) Comparison of multi-modal optimization algorithms based on evolutionary algorithms. ACM, New York. https://doi.org/10.1145/1143997.1144200
28. Mengshoel OJ, Goldberg DE (2008) The crowding approach to niching in genetic algorithms. Evol Comput 16(3):315–354. https://doi.org/10.1162/evco.2008.16.3.315. Available: http://www.mitpressjournals.org/doi/abs/10.1162/evco.2008.16.3.315
29. Sareni B, Krahenbuhl L (1998) Fitness sharing and niching methods revisited. IEEE Trans Evol Comput 2(3):97–106. https://doi.org/10.1109/4235.735432. Available: http://ieeexplore.ieee.org/xpl/login.jsp?tp=&arnumber=735432&url=http%3A%2F%2Fieeexplore.ieee.org%2Fiel4%2F4235%2F15834%2F00735432.pdf%3Farnumber%3D735432
30. Lim T (2014) Structured population genetic algorithms: a literature survey. Artif Intell Rev 41(3):385–399. https://doi.org/10.1007/s10462-012-9314-6. Available: http://link.springer.com/article/10.1007%2Fs10462-012-9314-6
31. Shir OM, Back T (2005) Dynamic niching in evolution strategies with covariance matrix adaptation. https://doi.org/10.1109/CEC.2005.1555018
32. Deb K, Pratap A, Agarwal S, Meyarivan TAMT (2002) A fast and elitist multiobjective genetic algorithm: NSGA-II. IEEE Trans Evol Comput 6(2):182–197
33. Knowles J, Corne D (1999) The pareto archived evolution strategy: a new baseline algorithm for paretomultiobjective optimisation. In: Proceedings of the 1999 congress on evolutionary computation (CEC'99), vol 1. IEEE
34. Zhang Q, Li H (2007) MOEA/D: a multiobjective evolutionary algorithm based on decomposition. IEEE Trans Evol Comput 11(6):712–731
35. Corne DW, Deb K, Fleming PJ, Knowles JD (2003) The good of the many outweighs the good of the one: evolutionary multi-objective optimization. IEEE Connect Newslett 1(1):9–13
36. Zhou A, Qu B, Li H, Zhao S, Suganthan PN, Zhang Q (2011) Multiobjective evolutionary algorithms: a survey of the state of the art. Swarm Evol Comput 1(1):32–49. https://doi.org/10.1016/j.swevo.2011.03.001. Available: http://www.sciencedirect.com/science/article/pii/S2210650211000058
37. Goldberg D, Smith R (1987) Nonstationary function optimization using genetic algorithm with dominance and diploidy. In: Proceedings of the second international conference on genetic algorithms and their application (ICGA). Laurence Erlbaum Associates, pp 59–68
38. Nguyen TT, Yang S, Branke J (2012) Evolutionary dynamic optimization: a survey of the state of the art. Swarm Evol Comput 6:1–24. https://doi.org/10.1016/j.swevo.2012.05.001. Available: http://www.sciencedirect.com/science/article/pii/S2210650212000363
39. Popovici E, Bucci A, Wiegand RP, De Jong ED (2012) Coevolutionary principles. In: Rozenberg G, Bäck T, Kok JN (eds) Handbook of natural computing. Springer, Heidelberg. https://doi.org/10.1007/978-3-540-92910-9_31
40. Hillis WD (1990) Co-evolving parasites improve simulated evolution as an optimization procedure. Physica D Nonlinear Phenom 42(1–3):228–234. https://doi.org/10.1016/0167-2789(90)90076-2. Available: http://www.sciencedirect.com/science/article/pii/0167278990900762

41. Potter MA, Jong KA (2000) Cooperative coevolution: an architecture for evolving coadapted subcomponents. Evol Comput 8(1):1–29. https://doi.org/10.1162/106365600568086. Available: http://www.mitpressjournals.org/doi/abs/10.1162/106365600568086
42. Yang Z, Tang K, Yao X (2008) Large scale evolutionary optimization using cooperative coevolution. Inf Sci 178(15):2985–2999. https://doi.org/10.1016/j.ins.2008.02.017. Available: http://www.sciencedirect.com/science/article/pii/S002002550800073X
43. Urbanowicz RJ, Moore JH (2009) Learning classifier systems: a complete introduction, review, and roadmap. J Artif Evol Appl 2009:1–25
44. Ochoa G, Harvey I, Buxton H (1999) On recombination and optimal mutation rates. In: Proceedings of genetic and evolutionary computation conference, vol 1, pp 488–495. Available: http://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.50.2369
45. Eiben AE, Smit SK (2011) Parameter tuning for configuring and analyzing evolutionary algorithms. Swarm Evol Comput 1(1):19–31. https://doi.org/10.1016/j.swevo.2011.02.001. Available: http://www.sciencedirect.com/science/article/pii/S2210650211000022
46. Fogel LJ (1962) Autonomous automata. Ind Res 4(2):14–19
47. Ochoa G, Blum C, Chicano F (2015) Evolutionary computation in combinatorial optimization. Springer International Publishing: Imprint: Springer, Cham
48. Bajpai RP (ed) (2014) Innovative design, analysis and development practices in aerospace and automotive engineering: I-Dad 2014, 22–24 Feb 2014. Springer Science & Business, Singapore
49. Gaurav A, Kumar V, Nigam D (2012) New applications of soft computing in bioinformatics: a review. J Pure Appl Sci Tech 11(1):12–22
50. Gupta SK, Ramteke M (2014) Applications of genetic algorithms in chemical engineering II: case studies. In: Applications of metaheuristics in process engineering. Springer, Cham, pp 61–87
51. Bentley P, Corne D (2002) Creative evolutionary systems. Morgan Kaufmann, San Francisco
52. Chen SH (ed) (2012) Genetic algorithms and genetic programming in computational finance. Springer Science & Business Media, New York
53. Gen M, Cheng R (1996) Genetic algorithms and manufacturing systems design, 1st edn. Wiley, New York
54. Adeli H, Sarma KC (2006) Cost optimization of structures: fuzzy logic, genetic algorithms, and parallel computing. Wiley, Chichester
55. Lones MA, Tyrrell AM (2007) A co-evolutionary framework for regulatory motif discovery. https://doi.org/10.1109/CEC.2007.4424978
56. Lones M, Alty JE, Lacy SE, Jamieson DR, Possin KL, Schuff N, Smith SL (2013) Evolving classifiers to inform clinical assessment of parkinson's disease. In: 2013 IEEE symposium on computational intelligence in healthcare and e-health (CICARE). IEEE, pp 76–82
57. Lones M, Turner AP, Caves LS, Stepney S, Smith SL, Tyrrell AM (2014) Artificial biochemical networks: evolving dynamical systems to control dynamical systems. IEEE Trans Evol Comput 18(2):145–166
58. Lones MA, Smith SL, Tyrrell AM, Alty JE, Jamieson DS (2013) Characterising neurological time series data using biologically motivated networks of coupled discrete maps. Biosystems 112(2):94–101

Innovative Applications of Metaheuristics to Supervised Machine Learning

7

Pritam Paral, Amitava Chatterjee, and Patrick Siarry

Contents

Introduction 149
Introduction to Supervised Machine Learning 149
Learning Aspects of the Supervised Methods and the Existing Challenges 151
Metaheuristic Optimization: A Potential Solution to the Local Minima Problem in Supervised ML 152
What Are Metaheuristics? 152
The Remainder of the Chapter 154
Black Widow-Optimized Convolutional Long Short-Term Memory Network for Hybrid Intrusion Detection 154
Background and Overview 154
Model and Methodology 156
Optimization of Hyperparameters Using BWO 158
Results and Discussions 160
Metaheuristic Optimization of Hyperparameters of GPR Model: Application to FEREBUS 163
Background and Overview 163
Fundamentals of GPR 165
Metaheuristic Optimizers Under Investigation 167
Details of Computation 168
Results and Discussions 170
Conclusion 173
References 174

P. Paral
Department of Electrical Engineering, IIEST, Shibpur, Howrah, India
e-mail: paral_pritam@ieee.org

A. Chatterjee
Department of Electrical Engineering, Jadavpur University, Kolkata, India
e-mail: amitava.chatterjee@ieee.org

P. Siarry (✉)
Université Paris-Est Créteil, Créteil, Cedex, France
e-mail: siarry@u-pec.fr; siarry@univ-paris12.fr

© Springer Nature Switzerland AG 2025

R. Martí et al. (eds.), *Handbook of Heuristics*,
https://doi.org/10.1007/978-3-032-00385-0_66

Abstract

The optimization of hyperparameters and the learning process associated with supervised machine learning models, including artificial neural networks and deep learning architectures, are viewed as among the most formidable challenges in machine learning. Several prior investigations have applied gradient-based backpropagation approaches in the training of deep learning architectures. However, gradient-based methods exhibit notable disadvantages, such as the tendency to become trapped in local minima when dealing with multi-objective cost functions, extensive calculations of gradient information with numerous iterations, and requiring the cost functions to maintain continuity. Given that the training process for a diverse array of supervised machine learning techniques is identified as an *NP-hard optimization* problem, there has been a considerable surge in the adoption of heuristic or metaheuristic algorithms aimed at optimizing the structures and parameters of these approaches. In the context of optimization, metaheuristic optimizers are designed to identify the optimal estimations for different components of supervised machine learning (e.g., weights, hyperparameters, the number of layers or neurons, and the learning rate in case of deep learning models, or hyperparameters for Gaussian processes in Gaussian process regression (GPR), etc.). Many researchers are inclined to extend innovative hybrid algorithms by integrating metaheuristic algorithms to optimize the hyperparameters of supervised machine learning models. The evolution of hybrid metaheuristics aids in boosting algorithm performance and is proficient in addressing complex optimization problems. In general, the optimal functioning of the metaheuristics ought to attain a favorable compromise between the exploration and exploitation characteristics. This chapter has comprehensively reviewed two recent advancements in the application of metaheuristic optimizers in supervised machine learning approaches, which are distinctly diverse in their characteristics. To begin with, this chapter offers a detailed review of a modern hybrid intrusion detection technique that employs a black widow-optimized convolutional long short-term memory neural network. Following this, the authors provide an in-depth analysis of a recent investigation focused on the metaheuristic optimization of hyperparameters for the GPR model associated with FEREBUS, a GPR engine embedded within the extensive framework of FFLUX, which is recognized as an innovative machine-learned force field. The integration of metaheuristics with supervised machine learning models is projected to expedite the training process in the forthcoming years. However, the concept of evolutionary hybrid architecture remains underutilized in the existing literature, with relevant publications being relatively uncommon.

Keywords

Nature-inspired metaheuristics · Supervised machine learning · Deep learning · Gaussian process regression · Hyperparameter optimization

Introduction

Introduction to Supervised Machine Learning

In the era of big data, sectors such as marketing, finance, health care, and telecommunications are persistently seeking innovative methods to derive valuable insights and facilitate informed decision-making. In this respect, a prominent technique that has garnered considerable attention in recent years is *data mining* [1]. Data mining is the *process* of extracting and discovering interesting patterns and knowledge from *large*-scale data collections. As a domain that prioritizes practical applications, data mining has assimilated a variety of methods from several other domains, including statistics, machine learning, pattern recognition, database and data warehousing systems, high-performance computing, etc. [1]. The interdisciplinary aspects of research and development in data mining play a crucial role in the overall success and diverse applications of data mining.

Therefore, *machine learning* represents one of the key disciplines that significantly impacts the advancement of data mining strategies. Despite their inherent differences, both these fields are closely associated and yield synergistic benefits when combined. Machine learning (ML) is considered a subdomain of *artificial intelligence* (AI) that enables computers to acquire knowledge and improve their performance autonomously, without the need for explicit programming [1]. One of the central areas of research is the ability of computer programs to learn independently to recognize complex patterns and to derive intelligent decisions from the data they process. For example, a standard ML problem within the field of medical imaging is to design a computer program that learns from a labeled dataset of medical images, where each image is accompanied by annotations indicating the relevant diagnosis or condition, enabling the algorithm to classify new *unseen* images based on the patterns it recognizes in the labeled examples. This technology is typically employed in tasks such as tumor detection, identification of abnormalities in imaging scans, or diagnosis of illnesses through medical imaging modalities like X-ray, magnetic resonance imaging (MRI), or computed tomography (CT) scan.

The name itself suggests that it endows the computer with a characteristic akin to that of humans: the capacity to learn. ML is currently being employed in a variety of applications, likely in more areas than one might anticipate. There are several varieties of ML, each with unique characteristics and applications. The following are a few of the primary categories of ML algorithms [1–4]: (i) supervised ML, (ii) unsupervised ML, (iii) semi-supervised ML, and (iv) reinforcement learning. The position of different types of ML techniques in the taxonomy of ML is depicted in Fig. 1.

Supervised ML is often more accurate than other ML methods, because it uses labeled training datasets. This provides the model with a foundational comprehension of what the appropriate output ought to be. These datasets are intended to train or "supervise" algorithms in accurately classifying data or predicting outcomes.

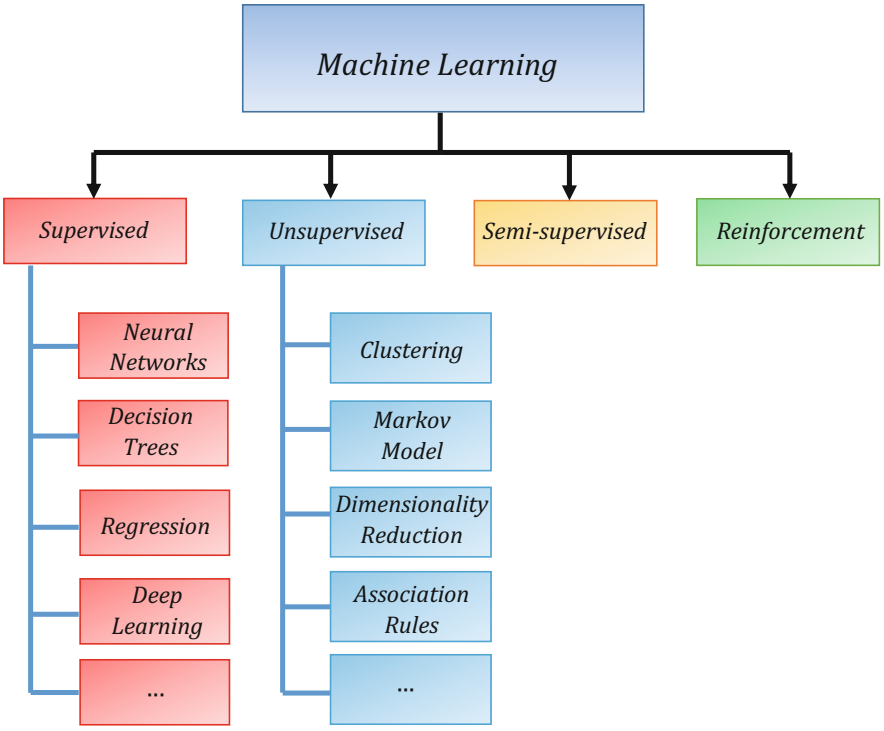


Fig. 1 Taxonomy of ML approaches

Through the application of labeled inputs and outputs, the model is capable of measuring its accuracy and evolving its learning process over time. In the context of data mining, supervised ML can be categorized into two distinct types of problems: *classification* and regression.

- *Classification* problems employ algorithms to effectively categorize test data into distinct groups, such as differentiating between apples and oranges. Or, in practical applications, supervised ML algorithms are employed to sort spam emails into a designated folder, thereby isolating them from your inbox. Commonly employed classification algorithms include linear classifiers, decision trees, support vector machines, random forest, etc.
- *Regression* is classified as another type of supervised ML approach that applies algorithms to elucidate the relationship between dependent and independent variables. Regression models serve as valuable tools for forecasting numerical outcomes by analyzing various data points, including projections of sales revenue for a specific enterprise. Linear regression, logistic regression, polynomial regression, GPR [5, 6], etc. are examples of popular regression algorithms.

Learning Aspects of the Supervised Methods and the Existing Challenges

The significance of employing supervised ML techniques across diverse applications underscores the necessity of recognizing their limitations and enhancing these algorithms, which remains a pressing concern in the field of ML. The process of learning different supervised ML techniques, including artificial neural networks (ANNs), deep learning (DL) architectures, and GPR, is viewed as one of the most formidable challenges in the field of ML. Throughout the previous two decades, one of the key areas of research has been the optimization of the structure and parameters associated with different supervised ML approaches. For example, optimization of ANNs and DL architectures, two of the most popular classes of supervised ML algorithms, is frequently approached from various perspectives, including the optimization of weights, biases, hyperparameters, activation nodes, network architecture, learning algorithms, learning parameters, learning environment, among others [7].

A variety of earlier research efforts have utilized gradient-based methods in the training of architectures. Nevertheless, the constraints associated with gradient-based algorithms have highlighted the importance of utilizing optimization algorithms. For instance, within the framework of the backpropagation (BP) learning algorithm, the primary objective of learning is to optimize the weights and thresholds of the network in order to reduce the cost function to its minimum. In gradient-based learning algorithms, it is essential for the cost function to be differentiable in order to utilize backpropagation. This aspect also highlights a significant drawback of gradient-based learning algorithms, because, in various cases, the activation and cost functions are non-derivable. In these algorithms, sigmoid activation functions are commonly employed. In the literature, a number of gradient-based techniques, including Levenberg Marquardt (LM) and backpropagation (BP) methods, have been established to facilitate the training of neural network-based systems. However, gradient-based methods exhibit several critical shortcomings as follows [7]:

- In the case of multi-objective cost functions, there is a possibility of being stuck at local minima.
- The time required to execute these algorithms is considerably costly, largely because of the extensive calculations of gradient information that involve thousands of iterations.
- In scenarios where multiple local minima exist within the problem search space, the learning algorithm attains an error of 0 at the first local minimum encountered. Consequently, the algorithm converges at this initial local minimum, preventing it from reaching the optimal solution.
- Gradient-based learning algorithms necessitate that the cost function be differentiable. Therefore, continuity of the cost function is a prerequisite. This characteristic highlights a significant drawback of gradient-based learning algorithms, particularly since, in various situations, the activation function may not be differentiable.

Initially, the Conjugate Gradient Algorithm, Stochastic Gradient Descent (SGD), Newton's Method, and Adaptive Moment Estimation (Adam) were introduced to enhance gradient-based learning algorithms, which exhibit superior generalizability and convergence compared to the BP algorithm. Nevertheless, the neural networks and DL architectures associated with these methods are often regarded as "black boxes," due to the fact that human intuition cannot interpret them [7].

Metaheuristic Optimization: A Potential Solution to the Local Minima Problem in Supervised ML

Given that the training of a broad spectrum of supervised ML techniques is classified as an *NP-hard optimization problem*, there has been a notable rise in the utilization of heuristic or metaheuristic algorithms for the optimization of the structures and parameters of those methods. As an optimization problem, metaheuristic optimizers aim to derive the optimal estimations for various supervised ML components (e.g., Gaussian process (GP) hyperparameters for GPR, or in case of DL models, weights, hyperparameters, the number of layers or neurons, the learning rate, etc.) [7]. The necessity for multi-objective metaheuristics has grown due to the existence of multiple objectives in the optimization of ANNs and DL models, such as minimizing errors, improving network generalization, and simplifying models. Recent literature indicates that the application of metaheuristic search (MHS) algorithms in training DL models enhances the overall learning process [7]. For example, evolutionary and swarm intelligence algorithms have contributed to the development of a generalized and optimal network [8–10]. This improves the model's accuracy and shortens the time required for its execution. The application of MHS algorithms to enhance supervised techniques, particularly in the realm of neural networks and DL architectures, remains a complex endeavor, necessitating further investigation.

What Are Metaheuristics?

Heuristic algorithm is a term that describes the implementation of systematic approaches which often compromise solution quality in favor of enhanced processing speed [11]. Many heuristics originate from straightforward, intuitive concepts aimed at problem-solving; however, they do not guarantee the attainment of an optimal solution. In certain cases, heuristics can yield solutions that are provably optimal. Heuristics can be described as a framework consisting of a series of permissible operations or transformations that convert an existing solution, labeled Q, into a new solution, Q*, which is usually an improved version. The domain encompassing all feasible solutions for a given problem is termed the search space, S, whereas the space of solutions that can be reached by applying a single transformation to an existing solution is known as the neighborhood space, N.

On the other hand, the term *metaheuristic* refers to the evolution of heuristic strategies that surpass local search (LS) methods in the pursuit of a global optimum [12, 13]. This concept is frequently drawn from analogies with natural processes, including both physical and biological phenomena. A metaheuristic is an advanced strategy or heuristic designed to identify, create, or choose a heuristic that can yield a sufficiently good solution to an optimization problem. These newer techniques were introduced aim to resolve challenges such as nonlinearity, multi-objective scenarios, uncertainty, and various other issues. Examples of metaheuristics include simulated annealing, tabu search, particle swarm optimization, genetic algorithms, artificial bee colony, etc. [12].

Metaheuristic optimization strategies can be classified into two categories—*population-based algorithms* and *trajectory-based algorithms*—based on the number of tentative solutions [12]. Trajectory-based algorithms, often referred to as single-solution metaheuristics, focus on a singular solution which they aim to enhance through iterative refinement employing a diverse array of stochastic operators. This pertains to the simulated annealing algorithm. The fundamental concept of simulated annealing involves creating a solution within the vicinity of the existing solution by making suitable moves in the search space. Subsequently, an acceptance probability is calculated to ascertain whether the new solution should be accepted or discarded. On the other hand, a set of agents is used by population-based MHS algorithms (e.g., particle swarm optimization, gravitational search algorithm, etc.) to examine the search space. The location of each agent is represented as a vector that matches the dimensions of the search space, thereby forming a potential solution. Population-based MHS algorithms facilitate the navigation of agents within the search space by enabling them to exchange information regarding their current locations and/or their best positions encountered thus far, utilizing well-structured communication rules.

The ability of population-based metaheuristic optimizers to deal with a particular optimization problem might differ greatly between techniques, since they use a wide variety of search strategies to update agents' positions. Generally speaking, metaheuristic-based optimization algorithms need to exhibit a decent balance between search space *exploration* and *exploitation* in order to produce the desired performance on a particular objective function [14]. It is commonly acknowledged that the optimizers that prioritize exploration (diversification) overexploitation (intensification) are more likely to explore search space regions where better solutions are located, leading to an overall improvement in performance. It has been demonstrated, meanwhile, that algorithms with comparable exploration-exploitation patterns can nevertheless behave completely differently. This finding implies that the specific ability of metaheuristic optimizers to produce promising solutions is just as important to their success as the trade-off between exploration and exploitation. The complexity of the search space affects an algorithm's ability to produce promising results, and we must remember that no MHS algorithm is intrinsically (by design) able to outperform all of its equivalents on *every* optimization task.

Therefore, comparing the effectiveness of several algorithms seems to be the most trustworthy approach of determining the best search mechanism for each unique optimization problem.

The Remainder of the Chapter

This study discusses two characteristically diverse verticals of application of metaheuristics in the realm of supervised ML in sections "Black Widow-Optimized Convolutional Long Short-Term Memory Network for Hybrid Intrusion Detection" and "Metaheuristic Optimization of Hyperparameters of GPR Model: Application to FEREBUS," respectively. In section "Black Widow-Optimized Convolutional Long Short-Term Memory Network for Hybrid Intrusion Detection," a recent approach to hybrid intrusion detection, which incorporates a black widow-optimized convolutional long short-term memory neural network, is reviewed in detail. Section "Metaheuristic Optimization of Hyperparameters of GPR Model: Application to FEREBUS" conducts a comprehensive review of a recent study that examines the metaheuristic optimization of hyperparameters for the GPR model linked to FEREBUS, a GPR engine integrated within the extensive framework of FFLUX, which is an innovative machine-learned force field. Section "Conclusion" concludes the study.

Black Widow-Optimized Convolutional Long Short-Term Memory Network for Hybrid Intrusion Detection

Background and Overview

Network traffic data has increased dramatically in the contemporary era due to developments in communication protocols and information technologies. Since practically every computer has an Internet connection, internet applications rely on different networks to provide services like social media, email, and surfing. But it has also left a security gap, opening the door for severe cyber threats including *network intrusions*. Unauthorized network traffic activities that are malevolent and damaging to computer networks are known as network intrusions. The results of these malicious invasions make a system susceptible to numerous attacks. It can cause system degradation or termination through identity theft attacks, *Denial of Service* (DoS) attacks, spam emails, or phishing attempts to gain sensitive user data by impersonating a trustworthy website. Additionally, it compromises user privacy in a shared environment [15] and raises security concerns with cloud storage. The intruders may initiate these attacks by executing a malware or malicious code to infect a host computer. This malware gives intruders the ability to take over infected hosts in order to obtain information or interfere with network services for illicit purposes.

Malevolent cyberattacks lead to serious security issues that need for an advanced, dependable, and adaptable *intrusion detection system* (IDS) [16]. An intelligent intrusion detection mechanism proactively identifies and classifies harmful activities or intrusions at both the system and host levels. IDS models are divided into *two* categories depending on the intrusion behaviors and architectures of the IDSs: *host-based IDS* (HIDS) and *network-based IDS* (NIDS). The HIDS models identify attacks using log files on the local host machines, whereas the NIDS do so based on network behavior [17]. To fully protect a network system and offer greater reliability and flexibility, both HIDS and NIDS are necessary. Therefore, a hybrid strategy that combines both HIDS and NIDS is preferred by the majority of organizations.

According to recent research, ML algorithms have been used in many studies to enhance intrusion detection in place of traditional technologies. The majority of ML algorithms, however, are not capable of analyzing all widely used benchmark datasets that are accessible in the public domain. The ML methods fail to adapt to modern datasets that include new attack classes having features incompatible with ML. Additionally, depending on the size of the dataset and optimization parameters, they offer sluggish training process, as well as high false-positive rates. To identify intrusions as fast as possible, computing performance must be enhanced as intrusion data quantity increases. Together with the ML classifiers for choosing the optimal features, statistical techniques and metaheuristic-based optimizers have been used to shorten the computation time. However, the limited learning and classification capabilities of the ML algorithms prevent them from identifying attacks, especially in case of varying feature dimensions or complex new feature characteristics of attacks. Therefore, intelligent assessment and the forecasting principles of high-dimensional learning with massive amounts of data are not compatible with ML algorithms.

The sluggish training problem is resolved by a class of ML techniques called DL. The advancement of DL algorithms has improved computing and processing power across a range of applications in different research fields. Convolutional neural networks (CNN), recurrent neural networks (RNN), long short-term memory (LSTM) neural networks, and other efficient DL techniques have served as the foundation for recent IDS models [18]. DL algorithms are more efficient and faster compared to ML for detecting intrusions in vast, diverse, and high-dimensional network traffic. Additionally, it makes it possible to train nonlinear models on big datasets in a scalable manner and accurately identify novel attack types. However, there are certain restrictions. Both the temporal and spatial features cannot be learnt simultaneously by CNN, RNN, LSTM, etc. CNN learns spatial features very well, whereas RNN and LSTM effectively deal with temporal information. While CNN has higher accuracy and a shorter training period, LSTM takes longer to train but produces fewer false positives than CNN. LSTM and CNN are both needed for developing a hybrid IDS model in order to learn all features and enhance overall performance. However, it will lead to increased processing and architectural complexity. Hybrid models complicate the integration of multiple classifiers by increasing computation time for each method. Additionally, the class imbalance

issue is more prevalent in the majority of publicly accessible IDS datasets and cannot be successfully addressed by CNN or LSTM alone. Modern solutions cannot keep up with the high-speed network traffic that is growing at an accelerated rate.

These difficulties inspired Kanna and Santhi to combine CNN and LSTM to develop a unified DL model that incorporates the advantages of both models while minimizing the effects of their drawbacks [19]. This fully integrated hybrid model called Conv-LSTM performed as a single powerful classifier. Its hyperparameters are optimized to cut down on computation time, because parameter tuning can be a useful in resolving this problem. However, rather than tuning the CNN and LSTM parameters separately, the parameters of the combined Conv-LSTM are learned by making use of black widow optimizer (BWO) [19], forming a new BWO-Conv-LSTM network-based IDS solution that significantly reduces the training time, as well as the computational cost [19]. BWO is chosen over other contemporary optimization methods due to its rapid convergence and ability to find global optimal solutions with minimal iterations. BWO-Conv-LSTM solves the class imbalance problem in network traffic datasets with simple oversampling, whereas the advanced DL classifier overcomes overfitting by utilizing *structural stabilization* strategy, which involves varying the values and number of weights with the BWO algorithm.

Model and Methodology

The IDS model based on BWO-Conv-LSTM aims at addressing the problem of intrusion classification in the big data context. The study [19] used four datasets, NSL-KDD, ISCX-IDS 2012, UNSW-NB15, and CSE-CIC-IDS 2018, which are mostly classified as big data on the basis of their volume aspect. These datasets are difficult to process, because the typical classifier models lack time or space to handle such problems. The processing of these datasets necessitates a massive and growing volume of resources, and additionally, advanced computing facilities and durable power sources are required to support learning and classification. These difficulties are lessened by the BWO-Conv-LSTM network model, which uses the characteristics of LSTM and CNN to extract the spatiotemporal features of network traffic data, resulting in accurate intrusion detection with reduced processing complexity. An overview of the design model for the hybrid IDS model developed in [19] is illustrated in Fig. 2.

The BWO-Conv-LSTM network-based intrusion detection system model consists of *four* major modules [19]:

(i) Oversampling,
(ii) Preprocessing,
(iii) Feature extraction and selection,
(iv) Classification

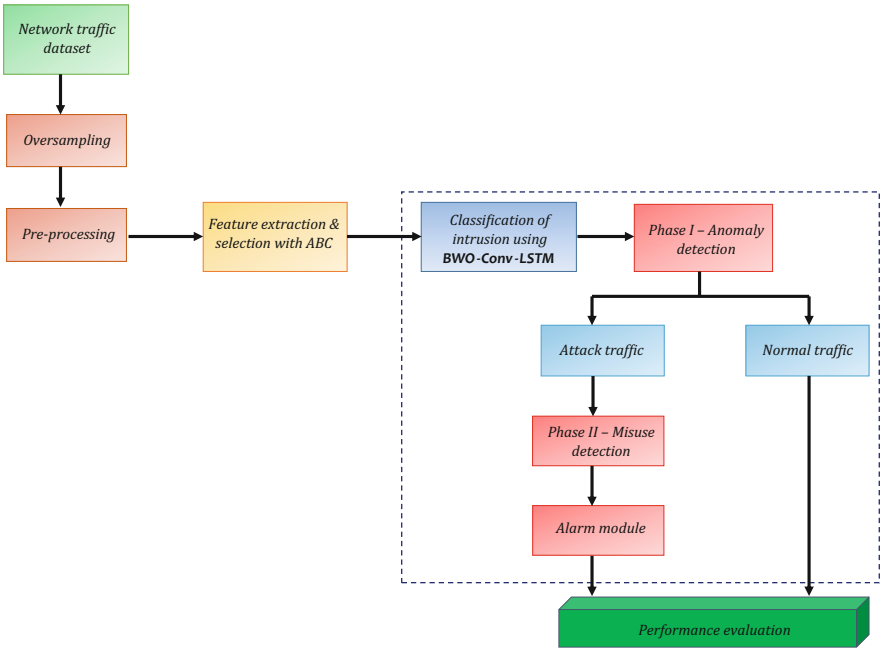


Fig. 2 A comprehensive outline of the hybrid IDS model presented in [19]

(i) **Oversampling**: Inadequate data distribution causes class imbalance problems in the input datasets; one class has the majority of the samples, whereas others have relatively less. Such imbalanced datasets will significantly impair the classifier performance. To address the imbalance issue, the majority of current approaches make use of statistical and ML techniques, such as data augmentation. However, those models solely use data samples from the majority class to fill the minority class samples, handling the minority class minimal. This leads to better data distribution across all minority classes without having regular minority class samples. High-dimensional imbalance problems with the evaluation datasets include potentially noisy, erroneous, and outlying cumulative data in addition to missing features and missing feature values. In order to overcome this problem, *oversampling* is carried out, which entails arbitrary replication of the minority class instances to expand the number of minority class samples. Despite the possibility of overfitting in the classifier, this method often produces no information loss and performs better than the other sampling and augmentation strategies. In order to prevent the potential overfitting problem, the BWO accomplishes the *structural stabilization* by changing the values and number of weights in the convolutional layers of the developed BWO-Conv-LSTM network. Since BWO finds these weights and their values in the best possible way, they can be changed within the optimal range without causing the classifier performance to deteriorate.

(ii) **Preprocessing**: The preprocessing phase carries out the label transformation, redundancy elimination, and data normalization operations following the oversampling procedure. Subsequently, the datasets are divided into 7 : 3 training and testing sets. For rapid evaluation, the training set was used for training, and the testing set was used to conduct the final assessment of the model.
(iii) **Feature Extraction and Selection**: After that, features are extracted from each dataset, and following the feature extraction, the artificial bee colony (ABC) algorithm [20] chooses the optimal feature subsets to enhance the performance of the classifier. Because ABC has proven to be incredibly effective at feature selection in large size datasets, it is used in this investigation.
(iv) **Classification**: The developed BWO-Conv-LSTM model-based classifier is incorporated in the final phase, which is *classification*. There are two components in the BWO-Conv-LSTM classifier: one for feature learning and one for classification. Two layers, namely, the *convolution* and *pooling* layers, make up the feature learner. While they pick up the spatial features, the convolutional layers miss the temporal information. Consequently, recurrent layers of LSTM are added after the CNN layers in order to more robustly capture both spatial and temporal features. The vanishing and exploding gradient problems will be stated properly by this procedure, improving the ability to capture temporal and spatial relationships and learn effectively. The CNN layers in the BWO-Conv-LSTM network handle the input mainly, while the LSTM layers process the output of the CNN layers to generate sequences at each time step. After passing through a fully connected layer, the sequence vector is then input into a Softmax layer to determine the probability distribution across the classes. In particular, this CNN and LSTM integration is carried out with the least amount of architectural complexity.

Optimization of Hyperparameters Using BWO

Using the BWO, the weights and biases are suitably learned, allowing for the optimal hyperparameter values. The life activities of black widow spiders are the source of BWO. It imitates mating behaviors, including procreation (reproduction), cannibalism, and mutation. These spiders reproduce by an unusual mating behavior, in which the females consume the conspecific males, and the young spiders undergo mutations after evading cannibalism by the mothers. The BWO algorithm was developed in light of these behaviors. The BWO begins by initializing the population of spiders. After that, these original spiders are coupled off in an attempt to reproduce a new generation. During mating, the female spiders consume the male. Following mating, the sperm is carried by the female and released toward eggs. Eleven days after the sperm-induced eggs are laid, the new baby spiders will emerge. On the maternal web, these young spiders live together, and the mothers cannibalize the weaker spiderlings to get rid of them and preserve the stronger spiders that are given priority. Young spiders undergo an enhanced mutation process as a result of this mechanism, and the winds subsequently carry the spiderlings to different regions. The methodology for implementing BWO in Conv-LSTM is outlined in Algorithm 1.

Algorithm 1: BWO algorithm [19]

Initialize population of spiders using a D-dimensional array, represented by *pop*.
Initialize Iteration = 0
Determine the count of reproductions (solutions nrp) by utilizing the procreation rate
Select the most effective solutions within the population and store them in *sub-pop-I*
/*Carry out procreation and cannibalism*/
for $i = 1$ to nrp **do**
    Selecting a pair of solutions (w_1, w_2) from *sub-pop-I* to serve as parents.
    Generate D offspring (z_1, z_2), which are new achieved solutions utilizing

$$\begin{cases} z_1 = \beta \times w_1 + (1 - \beta) \times w_2 \\ z_2 = \beta \times w_2 + (1 - \beta) \times w_1, \ \beta : \text{a } D\text{-dimensional array of random numbers from [0, 1]} \end{cases}$$

    Eliminate the paternal solution
    Evaluate the fitness function $f(z)$
    Annihilate several fragile children (new solutions) in light of the cannibalism rate
    Preserve the leftover solutions in *sub-pop-II*
end for
/*Carry out mutation*/
Calculate the count of mutated offspring (nmu) in accordance with the mutation rate
for $i = 1$ to nmu **do**
    Opt for a solution from *sub-pop-II*
    Implement a process of random mutation to create a new solution
    Save the new solution in *sub-pop-III*
end for
Update *pop* = *sub-pop-II*+*sub-pop-III*
Until maximum iteration
Return the most favorable solution from *pop*
end

This algorithm is used to tune the biases and weights of Conv-LSTM, resulting in the optimal values for the hyperparameters. The fitness function of Conv-LSTM is substituted for the original one, as $f(z) = f(\upsilon, \kappa)$, with υ and κ representing the sets of weights and biases of the optimized Conv-LSTM. This fitness function is associated with the convolution, pooling, fully connected layers, and activation function of the CNN, as well as the LSTM's learning rate and activation function. The optimal setup is achieved by tuning the parameters. The tuning procedure finds numerous configurations for the optimized Conv-LSTM, but BWO chooses the one with the *lowest error rate*. In this work, the initial values of three different parameters of BWO, i.e., procreation rate, cannibalism rate, and mutation rate are set as 0.6, 0.44, and 0.4, respectively.

Table 1 presents the best configuration for Conv-LSTM that is achieved with BWO. As can be observed in Table 1, the Conv-LSTM needed two LSTM layers and five layers of fully convoluted arrangement for optimal construction. With this setup, CNN can learn deep spatial features, and LSTM can learn temporal information. The convolution and pooling layers' sizes are determined by the results of prior experiments. There are three components that represent the dimensions, as shown in the table. For the input dimension (1, 256, 24), 1 represents the input

Table 1 Superior configuration featuring optimal hyperparameters achieved for Conv-LSTM through the application of BWO [19]

Layer	Name	Dimensions	Parameters
0	Input	(1, 256, 24)	–
1	Convolution	(16, 256, 24)	{16, 3, 3}
	Pooling	(16, 128, 12)	{2, 2}
	Activation (sigmoid)	–	–
2	Convolution	(32, 128, 12)	{32, 3, 3}
	Pooling	(32, 64, 6)	{2, 2}
	Activation (sigmoid)	–	–
3	Convolution	(64, 64, 6)	{64, 3, 3}
	Pooling	(64, 32, 3)	{2, 2}
	Activation (sigmoid)	–	–
4	Convolution	(128, 32, 3)	{128, 3, 3}
	Pooling	(128, 16, 1)	{2, 2}
	Activation (sigmoid)	–	–
5	Convolution	(256, 16, 1)	{256, 3, 3}
	Pooling	(256, 8, 1)	{2, 2}
	Activation (sigmoid)	–	–
6	Flatten	(1536,)	–
7	Fully connected	(278,)	{278}
8	LSTM	–	{200}
	Activation (tanh)	–	–
	Learning rate	–	{0.7}
9	LSTM	–	{200}
	Activation (tanh)	–	–
	Learning rate	–	{0.7}
10	Fully connected	(278,)	{278}

data channel, 256 the total number of data, and 24 the processing time steps. The convolution layer parameter (16, 3, 3) indicates that layer 1 has 16 filters of size (3, 3). Depending on the requirements of the dataset, the structural stabilization mechanism is implemented based on these parameter value ranges. To address the over-fitting issue, the nodes in the layers are either enhanced or decreased when the weights are readjusted.

As previously stated, the Adam is chosen as the Conv-LSTM optimizer, and the error rate is selected as the optimization function. Comparing the optimal setup presented in Table 1 to other potential choices, it has an effective architecture with the lowest error rate. Therefore, the BWO-Conv-LSTM is designed with a great deal of efficiency. This classifier's performance is assessed using four IDS datasets.

Results and Discussions

The performance evaluation of the BWO-Conv-LSTM with ABC-based feature selection (hereafter referred to as BWO-Conv-LSTM (ABC)) is carried out on the *four*

datasets based on *four* quantitative performance metrics, i.e., Accuracy, Precision, Recall, and F-Measure [19]. For performance comparison, *five* recent state-of-the-art models for IDS are implemented, namely, support vector machines (SVM) with improved binary grey wolf optimizer-based feature selection (referred to as IGWO-SVM) [21], ANN with correlation-based feature selection (referred to as ANN-CFS) [22], genetic algorithm (GA)-optimized CNN (referred to as GA-CNN) [23], hybrid convolutional RNN-based NIDS model (referred to as HCRNNIDS) [24], and optimized CNN and Hierarchical Multi-scale LSTM (referred to as OCNN-HMLSTM) previously developed by Kanna and Santhi [25]. These contemporary models are chosen for evaluation comparison, because they demonstrated superior performance, as pointed in the literature. The study [19] evaluated these established models under the same platform, despite the fact that they were actually tested in different experimental configurations in the literature.

Tables 2, 3, 4 and 5 report the comparison of the BWO-Conv-LSTM (ABC)-based IDS model against the five competing models over the four datasets, in terms of Accuracy, Precision, Recall, and F-Measure, respectively. The results in Tables 2 and 5, respectively, demonstrated that the BWO-Conv-LSTM (ABC) achieved higher accuracy values and F-scores than the other methods of comparison uniformly for all the four datasets. Table 3 reveals that BWO-Conv-LSTM (ABC) clearly outperforms its competitors in terms of precision over the NSL-KDD and CSE-CIC-IDS 2018 datasets, whereas all the six competing models achieved a precision of 100% for ISCX-IDS and UNSW-NB15 datasets. Likewise, as can be observed from Table 4, BWO-Conv-LSTM (ABC) achieved better recall results

Table 2 Performance comparison among BWO-Conv-LSTM (ABC) and five competing models on four datasets in terms of Accuracy (%) [19]

Models/datasets	NSL-KDD	ISCX-IDS	UNSW-NB15	CSE-CIC-IDS 2018
IGWO-SVM	85.76	87.32	89.66	89.55
ANN-CFS	81.66	92.42	94.71	94.51
GA-CNN	93.89	94.43	96.56	94.46
HCRNNIDS	95.35	94.58	95.48	97.48
OCNN-HMLSTM	90.67	95.33	96.34	97.27
BWO-Conv-LSTM (ABC)	**98.67**	**97.00**	**98.67**	**98.25**

Table 3 Performance comparison among BWO-Conv-LSTM (ABC) and five competing models on four datasets in terms of Precision (%) [19]

Models/datasets	NSL-KDD	ISCX-IDS	UNSW-NB15	CSE-CIC-IDS 2018
IGWO-SVM	90.62	**100**	**100**	82.60
ANN-CFS	69.89	**100**	**100**	79.54
GA-CNN	86.45	**100**	**100**	93.74
HCRNNIDS	96.67	**100**	**100**	96.33
OCNN-HMLSTM	89.71	**100**	**100**	94.91
BWO-Conv-LSTM (ABC)	**97.48**	**100**	**100**	**97.48**

Table 4 Performance comparison among BWO-Conv-LSTM (ABC) and five competing models on four datasets in terms of Recall (%) [19]

Models/datasets	NSL-KDD	ISCX-IDS	UNSW-NB15	CSE-CIC-IDS 2018
IGWO-SVM	**100**	80.29	89.03	93.62
ANN-CFS	85.43	89.42	91.91	90.15
GA-CNN	92.37	95.21	96.9	95.82
HCRNNIDS	**100**	93.65	96.25	97.12
OCNN-HMLSTM	95.19	94.77	95.87	97.79
BWO-Conv-LSTM (ABC)	**100**	**95.78**	**98.78**	**98.67**

Table 5 Performance comparison among BWO-Conv-LSTM (ABC) and five competing models on four datasets in terms of F-Measure (%) [19]

Models/datasets	NSL-KDD	ISCX-IDS	UNSW-NB15	CSE-CIC-IDS 2018
IGWO-SVM	85.36	81.11	91.36	96.80
ANN-CFS	74.32	94.7	96.14	94.14
GA-CNN	91.02	98.14	97.46	96.89
HCRNNIDS	96.5	97.98	97.87	97.60
OCNN-HMLSTM	91.46	97.61	98.13	97.09
BWO-Conv-LSTM (ABC)	**98.73**	**99.67**	**98.77**	**98.18**

Table 6 Average training and detection time comparison of the six competing IDS models over four datasets

Models/datasets	Training time (s)	Detection time (s)
IGWO-SVM	39,687.02	1006.71
ANN-CFS	44,991.95	898.86
GA-CNN	36,366.14	762.90
HCRNNIDS	29,488.27	677.93
OCNN-HMLSTM	29,082.29	669.60
BWO-Conv-LSTM (ABC)	**25,717.68**	**604.32**

compared to five competing models for the ISCX-IDS, UNSW-NB15, and CSE-CIC-IDS 2018 datasets, while BWO-Conv-LSTM (ABC), as well as IGWO-SVM and HCRNNIDS, produced a 100% recall for NSL-KDD dataset. In a nutshell, as evidenced by the results of Tables 2, 3, 4 and 5, BWO-Conv-LSTM (ABC) emerged as the overall superior to the five competing state-of-the-art models for IDS. These findings demonstrate how the BWO-Conv-LSTM (ABC) ensures better intrusion detection than the majority of current models through the use of more sophisticated DL of both temporal and spatial characteristics of network traffic data. The overall performance has also been greatly impacted by the use of ABC and BWO for feature selection and parameter optimization, respectively. The assessment of four datasets further shows that this model supports various types of network traffic data. Additionally, Table 6 reports the average training and detection time consumed by the six IDS models of comparison over the four datasets. It can be observed

from Table 6 that the training and detection time of BWO-Conv-LSTM (ABC) is noticeably less than the other models. This is because CNN requires less training time, which is why the overall detection and training time is justified.

Metaheuristic Optimization of Hyperparameters of GPR Model: Application to FEREBUS

Background and Overview

The cost-versus-accuracy trade-off has long plagued molecular dynamics (MD) simulations of macromolecules, encouraging low-cost approaches—more especially, classical force field (CFF) methodologies—despite their obvious inability to produce quantitatively reliable and consistent outcomes [26]. On the other hand, as the size of the system grows, *ab initio* quantum mechanics (QM) techniques, which offer a more plausible description of molecular systems by explicitly treating the electrons (neglected in CFFs), become unaffordable. The development of a new class of unconventional potentials known as machine learning force fields (MLFFs) is presently being pursued in an attempt to resolve this dilemma. The potential to combine the predictive effectiveness of ML regression models with the accuracy of QM observables is what drives MLFFs. The goal of MLFFs is to attain the accuracy of ab initio QM techniques at a significantly lower cost that is on par with CFFs.

FFLUX is an innovative yet well-established MLFF that was created from the scratch using proof-of-concept research [27]. It makes use of pretrained GPR models to forecast multipole moments and atomic energies, which are both necessary to estimate the physical interactions that occur within and between molecules during MD simulations. FFLUX does not define atom types or rely on any predetermined bonding arrangement like CFFs do. Rather, it works with quantum topological atoms, which are the identical space-filling atoms in molecules arising from a thorough division of molecular spaces in the framework of quantum chemical topology.

FEREBUS [28], a Fortran90 package, forms the core of FFLUX. It is in charge of constructing GPR models for multipole moments and atomic energies. FEREBUS builds as many models as quantum observables for each atom in the system. Every model associates the relevant input features, which encapsulate structural details about the system under consideration, with a specific target attribute (QM observable). Three major architectural changes have been made to the FEREBUS package in its current version (9.2): (i) multiple metaheuristic optimizers have been implemented; (ii) STELLA, a software module that can validate models on the fly, has been integrated; and (iii) a different, straightforward but effective protocol for optimizing model hyperparameters has been developed [29].

The performance of the latent GPR models determines the results of FFLUX simulations. Finding the factor that determines the quality of a GPR model is a valid question at this stage. The prediction ability of a GPR model is determined by the set of model hyperparameters, represented by $\boldsymbol{\phi}$, for a particular training dataset ($\mathbf{U}$, $\mathbf{v}$),

mean function $\mu(\mathbf{u})$, and covariance function $c(\mathbf{u}, \mathbf{u}')$. As $\boldsymbol{\phi}$ values are never known beforehand, the process of training a GPR model mostly involves determining the optimal combination of these hyperparameters. In general, the log marginal likelihood function $\mathcal{L}(\mathbf{v}|\mathbf{U}, \boldsymbol{\phi})$ of a GP is maximized to derive the hyperparameters of the GPR model; this approach is also known as the *Type-II maximum likelihood* (ML-II) method. One of its advantages is the computational ease provided by the analytical tractability of its gradient and the cost function, which enables the use of any standard gradient-based technique for model hyperparameter optimization. Nevertheless, the ML-II protocol has certain shortcomings in spite of its popularity. For example, it is well known that the existence of outliers in the dataset can significantly impair the performance of ML-II-based models. Several approaches have been put forth to address this problem, including (i) using an outlier-resistant observation model, like the student-t distribution, in place of the Gaussian-like likelihood of the observations, and (ii) applying a Gaussian mixture noise model to account for regular as well as irregular noises [29]. Sadly, neither solution maintains the intended analytical tractability of the marginal likelihood, therefore in order to access the model hyperparameters, the practitioner must either (i) rely on approximation techniques or (ii) just think of a different protocol.

Using the same justification, it was recently found that the propagation of numerical errors may have a significant impact on the $\mathcal{L}(\mathbf{v}|\mathbf{U}, \boldsymbol{\phi})$ cost function, potentially producing inconsistent findings. Thankfully, an alternative protocol for optimizing the hyperparameters of the GPR model is now incorporated in FEREBUS. In comparison to ML-II, this straightforward and user-friendly method known as *iterative hold-out cross-validation* (IHOCV) can significantly increase the quality of the trained models. Additionally, unlike the commonly used, yet occasionally contentious, k-fold cross-validation [30], IHOCV attempts to improve the same model by performing minimization of the predictive root-mean-square error (RMSE) RMSE($\boldsymbol{\phi}$) of intermediate models over a significantly large and consistent internal validation set rather than computing the average performance of multiple models constructed from different segments of the same dataset. Particularly in the noise-free regime, the ML-II technique cannot guarantee that the optimization process will continuously improve the performance of intermediary models, while the IHOCV protocol does.

Whichever training protocol—ML-II or IHOCV—an individual chooses, he/she still needs to select a method for optimizing the related cost functions. Finding the global optimum of the RMSE($\boldsymbol{\phi}$) and $\mathcal{L}(\mathbf{v}|\mathbf{U}, \boldsymbol{\phi})$ functions is extremely difficult because to their multimodal nature and complex landscape, particularly for large dimensional search spaces. This calls for a search method that can jump out of local optima. Gradient-based methods, which require the availability of the analytical gradient of the cost function, are often highly sensitive to initial guesses and converge to the closest optimum without escape mechanisms. On the contrary, the stochastic nature of MHS algorithms allows them to avoid repeated traps of local extrema, while they do not need any analytical gradient. Even though MHS algorithms are incredibly good at solving a wide range of complex optimization problems, they have not yet gotten the consideration they deserve in the context of

GPR-based atomistic potentials, where gradient-based approaches are prioritized for optimizing model hyperparameters. In contrast to this well-liked option, FEREBUS provides its users with a number of (population-based) MHS algorithms, all of which are able to optimize model hyperparameters globally.

In [29], Isamura and Popelier examined the performance of ten distinct population-based MHS algorithms to find the optimal set of hyperparameters of the GPR model following the two protocols (i.e., ML-II and IHOCV) as implemented in FEREBUS. The effectiveness of each combination of hyperparameters was assessed based on the corresponding model's predictive accuracy on data points that had not been seen before. Four of the most well-known families of population-based MHS algorithms, namely, (i) particle swarm optimization (PSO), (ii) bat algorithms (BA), (iii) firefly techniques (FFA), and (iv) gray wolf optimizer (GWO), were used to choose the optimizers investigated in [29]. All the GPR models were developed to estimate the total electronic energy of individual atoms across different conformations of capped alanine. It should be noted that the reference atomic energies were meticulously calculated using the Interacting Quantum Atoms (IQA) method [31]. As demonstrated by Isamura and Popelier, GWOs outperform the other three families. In particular, atomic models produced by the standard GWO have the lowest average prediction errors.

Fundamentals of GPR

Bayesian statistics serves as the foundation for GPR, often referred to as kriging, a nonparametric ML technique [32]. GPR has four key benefits over alternative regression techniques: (i) Being a nonparametric method, GPR does not specifically assume how the function to be learnt should be expressed analytically; (ii) even when trained on small datasets, GPR models can achieve high predictive accuracy, something that naturally "greedy" ANNs typically find difficult to do; (iii) there is less chance of overfitting; and (iv) alongside each point estimate (prediction), there is an estimate of the degree of uncertainty around the predicted (mean) value [29].

To train a GPR model, a dataset $\mathbf{D} = \{\mathbf{U}, \mathbf{v}\}$ comprising n observations is necessary, with $\mathbf{U}$ representing a rectangular matrix of input features and $\mathbf{v}$ denoting the vector of target values (which, in this context, are atomic energies). It is presumed that a function f exists such that for each observation $(\mathbf{u}_k, v_k)$, $v_k = f(\mathbf{u}_k) + \epsilon_k$, where ϵ_k indicates the noise corrupting the k-th measurement. The noise $\boldsymbol{\epsilon} = \{\epsilon_1, \epsilon_2, \ldots, \epsilon_n\}$ is assumed to follow a zero-mean Gaussian distribution with a standard deviation σ_n^2. The regression problem involves determining the most likely function f that associates each input vector $\mathbf{u}_k$ with the corresponding target value v_k. This task is performed in two steps from a Bayesian perspective: (i) defining a Gaussian process prior over the unknown distribution of functions f and (ii) updating the initial belief about f using the Bayes' theorem in light of the observations in $\mathbf{D}$. This results in a posterior distribution over f that provides a predictive Gaussian distribution for each new point, with its mean value being the point estimate of the prediction (expected value).

Assuming that a GPR model is already trained on **D**, the following formulas can be used to determine the expected value corresponding to any new point $\mathbf{u}^*$ [29]:

$$f(\mathbf{u}^*) = \mu(\mathbf{u}) + \mathbf{c}_*^T \mathbf{C}^{-1} \mathbf{v}_{\boldsymbol{\mu}} \tag{1}$$

$$\mathbf{c}_* = \begin{bmatrix} c(\mathbf{u}_1,\ \mathbf{u}^*|\boldsymbol{\phi_k}) \\ c(\mathbf{u}_2,\ \mathbf{u}^*|\boldsymbol{\phi_k}) \\ \vdots \\ c(\mathbf{u}_n,\ \mathbf{u}^*|\boldsymbol{\phi_k}) \end{bmatrix} \tag{2}$$

$$C_{ij} = cov(\mathbf{u}_i,\ \mathbf{u}_j) = c(\mathbf{u}_i,\ \mathbf{u}_j|\hat{\boldsymbol{\phi}}_k) + \sigma_n^2 \delta_{ij} \tag{3}$$

$$\mathbf{v}_{\boldsymbol{\mu}} = \mathbf{v} - \mathbf{1}\mu(\mathbf{u}) \tag{4}$$

where $\mu(\mathbf{u})$ and $c(\mathbf{u}_i,\ \mathbf{u}_j|\boldsymbol{\phi_k})$ are the prior mean and covariance functions, respectively, determining the GP prior over f, and **C** represents the covariance matrix. The superscript T represents the transposition of a column vector into a row vector. In addition to measuring the noise dispersion of the dataset, the parameter σ_n^2 also acts as a regularization factor meant to keep the matrix **C** from becoming ill-conditioned. Lastly, δ_{ij} represents the Kronecker delta.

There are several different approaches to define the prior mean function $\mu(\mathbf{u})$. However, this study chose to maintain it constant and have it match the arithmetic mean of the output attributes (atomic energies) in **D**. This decision guarantees that even in the atypical extrapolation regime where predictions converge toward $\mu(\mathbf{u})$, they maintain their physical significance. The covariance function $c(\mathbf{u}_i,\ \mathbf{u}_j|\boldsymbol{\phi_k})$, however, is often called as the *kernel*. This function evaluates the distance required between two points in the input space for them to be deemed uncorrelated. The strength of these correlations is governed by a specific set of parameters $\boldsymbol{\phi}_k$, which, when paired with σ_n^2, form the hyperparameters ($\boldsymbol{\phi}$) of the GP. The composite kernel described in Eq. (5) was used throughout the study [29], where N_{ft} represents the dimension of each feature (input) vector [29]. This kernel integrates periodic covariance functions with the radial basis function (RBF) and utilizes them across various dimensions of the input space.

$$c_{\text{RBF}}\left(\mathbf{u}, \mathbf{u}'|\phi_k = \{\sigma_f^2,\ \gamma_{d,1},\ \gamma_{d,2}, \ldots,\ \gamma_{d,N_{ft}}\}\right) = \sigma_f^2 \exp\left[-\sum_{d=1}^{N_{ft}} \Omega(\mathbf{u},\ \mathbf{u}')\right] \tag{5}$$

with

$$\Omega(\mathbf{u},\ \mathbf{u}') = \begin{cases} \dfrac{1 - \cos(\mathbf{u}_d - \mathbf{u}'_d)}{\gamma_d^2} & d > 3 \quad \text{and} \quad \text{mod}\ (d, 3) = 0 \\ \dfrac{(\mathbf{u}_d - \mathbf{u}'_d)^2}{2\gamma_d^2} & \text{otherwise} \end{cases}$$

It is important to mention that FEREBUS computes ϕ_d hyperparameters that are linked to the parameters of characteristic length scale γ_d via $\phi_d = 1/2\gamma_d^2$. It is assumed in Eqs. (1)–(4) that the GP hyperparameters ($\boldsymbol{\phi}$) are known. But these $\boldsymbol{\phi}$ values are never known in advance, and determining the best combination of them is the key to training a GPR model. In order to access the model hyperparameters, the study [29] took into consideration two distinct protocols: ML-II and IHOCV.

In the ML-II protocol, the hyperparameters $\boldsymbol{\phi}$ are derived by maximizing the LML function as described in Eq. (6)

$$\mathcal{L}(\mathbf{v}|\mathbf{U}, \boldsymbol{\phi}) = -\frac{1}{2}\sigma^2 - \frac{1}{2}\log|\mathbf{C}| - \frac{n}{2}\log(2\pi) \tag{6}$$

$$\sigma^2 = (\mathbf{v} - \boldsymbol{\mu}_\mathbf{v})^T \mathbf{C}^{-1}(\mathbf{v} - \boldsymbol{\mu}_\mathbf{v}) \tag{7}$$

$$\boldsymbol{\mu}_\mathbf{v} = \mathbf{1}\mu(\mathbf{u}) \tag{8}$$

where, in Eq. (6), the first two terms are, respectively, referred to as the *data-fit* and *complexity penalty components*.

The IHOCV protocol attains $\boldsymbol{\phi}$ values by minimizing the predictive RMSE of intermediary models across a sufficiently extensive internal validation dataset. The RMSE objective function is computed by making use of Eq. (9), with m representing the count of internal validation points and v_k^{pred} being the prediction corresponding to the kth validation point,

$$\text{RMSE}(\boldsymbol{\phi}) = \left(\frac{1}{m}\sum_{k=1}^{m}|v_k - v_k^{pred}|^2\right)^{1/2} \tag{9}$$

The tuning of model hyperparameters in the IHOCV protocol necessitates assessing the expected values for each unseen point in the internal validation dataset. Because FEREBUS uses MHS algorithms, the $\boldsymbol{\phi}$ vector in Eq. (9) actually represents a temporary combination of hyperparameters ($\boldsymbol{\phi}^{tmp}$) that corresponds to the best agent's location in the RMSE function landscape during the current iteration. To calculate v_p^{pred} for a new data point p that is not in $\mathbf{D}$, plug $\boldsymbol{\phi}^{tmp}$ in Eqs. (1)–(4)

Metaheuristic Optimizers Under Investigation

In a population-based MHS algorithm, all the search agents should gradually converge toward the same optimal solution under ideal conditions. That being said, this is not always the case. At times, due to a variety of factors, one or more agents may find it challenging to synchronize with the group's collective movement, leading them to diverge or exit the search space. This behavior can adversely affect the overall search process, and if not managed properly, it may result in unforeseen and potentially unfeasible solutions. To prevent this unfavorable scenario, FEREBUS employs a control strategy that involves reinitializing the position of any agent that exceeds the limits of the search space.

As previously mentioned, the optimizers investigated in the study [29] were chosen from four of the most widely recognized categories of population-based MHS algorithms: (i) PSO, (ii) GWO, (iii) BA, and (iv) FFA. Within the genre of PSO, a canonical version of PSO (i.e., CPSO) [33] and three noncanonical PSO algorithms, namely, the (i) global-best fully informed PSO (FIPSO-Gb) [34], (ii) randomized PSO (RPSO) [35], and (iii) bare bone PSO (BBPSO) [36], were considered. From the family of GWO, the standard GWO [37] and "random-walk" GWO (RW-GWO) [38] were selected, whereas the basic BA [39] and "directional" BA (dBA) [40] were chosen from the family of BA. In the realm of FFA, the standard FFA [41] and chaotic FFA (CFFA) [42] were chosen.

Details of Computation

Generation of Dataset

The datasets utilized in the research [29] to evaluate the effectiveness of metaheuristic optimizers across different optimization tasks were produced through a protocol executed in the proprietary task management software ICHOR. Consistent with the protocol, the potential energy surface (PES) of capped alanine was examined using classical molecular dynamics simulations, employing the GAFF2 [43] force field as featured in the AMBER18 software package [29]. This decision was motivated by the necessity to achieve a balanced compromise between computational time and the feasibility of the sampled conformations. The simulation was conducted over a duration of 1,000,000 steps, utilizing a timestep of 1 femtosecond, with the Langevin thermostat selected to regulate the temperature at approximately 300 K. Geometries were gathered at intervals of 100 steps, resulting in an initial population of 10,000 geometries. Each of the 10,000 geometries sampled underwent a single point calculation at the B3LYP/6–31+G(d,p) level to obtain the electron density, which is the sole physical quantity necessary for AIMAll [29] to conduct topological analyses. Gaussian16 was utilized to conduct single-point computations.

The atomic IQA energy for each atom across all sampled geometries was computed utilizing the AIMAll program, which employs the IQA methodology. Simultaneously, the structural information present in each geometry was represented through fixed-length vectors that comprised geometrical features specified within the *atomic local frame* (ALF) [27].

The datasets, comprising 22 files corresponding to each atom, were generated by concatenating the input features with the atomic energies. A filtering process was then applied to eliminate any geometries in which at least one atomic energy exhibited an absolute integration error exceeding 0.0004 a.u. It should be noted that the integration error $I(\delta)$, associated with the Laplacian of the electron density integrated over an atomic basin δ, serves as an indicator of the numerical accuracy of atomic integrations [29]. Typical values of 0.001 a.u. for elements in the second and third rows, and 0.0001 a.u. for hydrogen, are regarded as a reasonable compromise for medium-sized molecules, as they ensure the reconstruction of the total electronic energy within an accuracy of 1 kcal/mol. In the research [29], reducing the initial

population to $I(\delta) = 0.0004$ a.u. enabled the retention of approximately 74% of the geometries, specifically 7412 conformations, while attaining a satisfactory mean absolute recovery error of 4.2 kJ/mol, which is roughly equivalent to 1 kcal/mol. After filtering, the reference datasets were subsequently employed to sample the training, validation, and test sets.

ALF Feature Calculation

The input space was delineated through the concept of ALF [27], which is a molecular representation that integrates both *two-body* (noncyclic) and *three-body* (cyclic) interactions. Each local frame is defined by three atoms: the atom of interest (α_0) acting as the origin and two additional atoms arranged in descending order of importance in accordance with *Cahn-Ingold-Prelog* principles [44]. The atom given the highest priority, designated α_x, is positioned along the X axis, whereas the other atom α_y describes the XY plane. The Z axis is positioned at a right angle to the XY plane, thereby forming a right-handed (orthogonal) local coordinate system. Since every ALF is based around a particular atom, describing the entire system necessitates having as many local frames as there are atoms present.

For a system comprising M atoms, every input vector consists of $3M-6$ features. The initial three features of the input vector related to the local frame of α_0 include the two Euclidean distances from atom α_0 to the α_x and α_y atoms ($\Lambda^{\alpha x}$ and $\Lambda^{\alpha xy}$), in addition to the angle ψ^{α} formed by the three ALF atoms (refer to Eqs. (10)–(12)). Moreover, for each non-ALF atom α_n, where n is greater than or equal to 4, the input vector is augmented by incorporating three additional features that represent the spherical coordinates ($\Lambda^{\alpha n}$, $\omega^{\alpha n}$, $\varphi^{\alpha n}$) of atom α_n within the ALF. Equations (13)–(15) define these features.

$$\Lambda^{\alpha x} = \left[\left(\boldsymbol{\Lambda}^{\alpha x} - \boldsymbol{\Lambda}^{\alpha 0}\right)^2\right]^{1/2} \tag{10}$$

$$\Lambda^{\alpha xy} = \left[\left(\boldsymbol{\Lambda}^{\alpha xy} - \boldsymbol{\Lambda}^{\alpha 0}\right)^2\right]^{1/2} \tag{11}$$

$$\psi^{\alpha} = \cos^{-1}\left(\frac{\boldsymbol{\Lambda}^{\alpha x} \cdot \boldsymbol{\Lambda}^{\alpha xy}}{|\boldsymbol{\Lambda}^{\alpha x}||\boldsymbol{\Lambda}^{\alpha xy}|}\right) \tag{12}$$

$$\Lambda^{\alpha n} = \left[\left(\boldsymbol{\Lambda}^{\alpha n} - \boldsymbol{\Lambda}^{\alpha 0}\right)^2\right]^{1/2} \tag{13}$$

$$\omega^{\alpha n} = \cos^{-1}\left(\frac{\xi_3^{\alpha n}}{|\boldsymbol{\Lambda}^{\alpha n}|}\right) \tag{14}$$

$$\varphi^{\alpha n} = \tan^{-1}\left(\frac{\xi_2^{\alpha n}}{\xi_1^{\alpha n}}\right) \tag{15}$$

where $n \geq 4$. The term $\xi_i^{\alpha n}$ is defined as a Cartesian coordinate within the ALF, whereas $\boldsymbol{\Lambda}^{\alpha x}$, $\boldsymbol{\Lambda}^{\alpha xy}$, $\boldsymbol{\Lambda}^{\alpha 0}$, and $\boldsymbol{\Lambda}^{\alpha n}$ represent position vectors in the global frame.

The ALF representation is among a variety of molecular descriptors being utilized for the advancement of machine-learned potentials. Indeed, the features of ALF are (i) invariant to both translation and/or rotation of the system; (ii) distinctive in that a one-to-one correspondence can always be established between a specific atomic arrangement and the resulting features for all atoms; (iii) continuous, meaning that even the smallest alteration in a molecule's structure leads to corresponding changes in the features; (iv) compact, or ideally even minimalistic, ensuring that no extraneous features are included in the feature vector; and (v) feasible in terms of computational resources [29]. Additionally, the computation of ALF features is automated and does not necessitate human involvement. Nonetheless, because the ALF descriptor is characterized by its extensive nature (meaning that the number of features grows in direct proportion to the number of atoms), its practical application may be restricted to medium- and small-sized systems. Additionally, it appears reasonable to establish a local frame for each atom within a system and to utilize ML to realize the physical characteristics of a specific atom within its unique coordinate framework. However, this design decision neither promotes nor makes it easier to take into account ALF features in relation to (i) the representation of big systems, and (ii) the handling of nonlocal properties, including delocalization indices, molecular observables, etc.

Details of the Optimization Mechanisms

In [29], after being trained on 3000 points, all atomic models were assessed on 2000 different points without any prior standardization of ALF features. For IHOCV-based models, a 100-point internal validation set was used. The upper limit for iterations was established at a substantial figure of 10,000 to ensure that each optimization run consistently converged to solutions of equivalent quality across various randomly initialized successive runs (preliminary tests conducted with the CPSO algorithm confirmed this assumption). For the hyperparameters σ_f^2, ϕ_d, and σ_n^2, the search space was selected as [0, 10], [0, 3], and $[10^{-14}, 10^{-13}]$, respectively. The spectrum of σ_n^2 values may be considered as the noise-free domain. As highlighted by Isamura and Popelier, the earlier search space enabled them to achieve an effective equilibrium between the models' predictive accuracy and their anticipated capacity to replicate the training data. In relation to population size, groups consisting of 24 agents (each representing a potential solution) were assigned the responsibility of examining the search space. All optimization processes were conducted simultaneously utilizing eight cores on compute nodes equipped with Cascade Lake Xeon Gold 6230 CPUs, each operating at a frequency of 2.10 GHz.

Results and Discussions

This research primarily focuses on the performance of a chosen set of ten MHS algorithms in the training of GPR models for atomic energies. It is reasonable to assess the efficacy of MHS algorithms by considering the optimal or best value

obtained for the objective (cost) function. In the present context of regression, where the cost function optimization serves to inform the choice of the most suitable model, it is more logical to focus on the ability of the resulting models to generalize to previously unseen data points. Here, this is accomplished by calculating each trained model's predictive mean absolute error (MAE) across the identical external validation/test sets. The predictive MAE of the trained models will be used here to describe the performance of the selected MHS algorithms in the discussion that follows [29].

Unlike the majority of benchmark studies on MHS algorithms, which typically have a known exact solution to the optimization problem, the training of atomic GPR models involves a *blind search*. In this context, the analytical solution to the problem remains unknown and varies with the dataset. Consequently, it is impossible to ascertain the convergence rate of an algorithm, as it is not feasible to determine the distance of each solution provided by the algorithm from the exact solution. Additionally, because of the multimodal characteristics of the $\mathcal{L}(\mathbf{v}|\mathbf{U}, \boldsymbol{\phi})$ and RMSE($\boldsymbol{\phi}$) cost functions, it is common for various solutions (combinations of hyperparameters) to result in models that exhibit comparable or nearly indistinguishable performance levels. As per Rasmussen and Williams [32], this frequently indicates the presence of distinct yet comparable interpretations of the same dataset. In the interest of conciseness, the designation Pr-Al-El (such as ML-II-GWO-N) is adopted to signify the atomic models associated with an element El that has undergone training through an optimization algorithm Al, adhering to the protocol Pr. The atom numbering for capped alanine is illustrated in Fig. 3.

Optimization of the $\mathcal{L}(\mathbf{v}|\mathbf{U}, \boldsymbol{\phi})$ Cost Function

All atomic GPR models were initially trained in accordance with the ML-II protocol. Table 7 illustrates the average predictive MAEs per element over 22 at. models, achieved through the application of all ten optimizers.

Table 7 demonstrates that ML-II models display satisfactory, though not exceptional, predictive accuracies, with average predictive MAEs of 2.658, 3.754, 3.879, and 2.921 kJ/mol for the models corresponding to hydrogen (H), carbon (C),

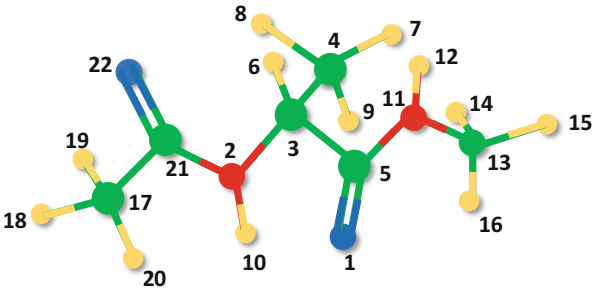


Fig. 3 Numbering of atoms of capped alanine. The yellowish, red, blue, and green spheres, respectively, represent the H, N, O, and C atoms [29]

Table 7 The average predictive MAEs in kJ/mol for atomic GPR models based on ML-II, which were trained utilizing ten distinct optimization algorithms. Each model underwent training using a dataset of 3000 points and was subsequently evaluated on a separate set of 2000 points. Each optimization run was allowed to proceed for a maximum of 10,000 iterations, a limit established to avoid premature convergence. The values highlighted in bold represent the algorithm that demonstrates the highest performance [29]

Optimizer/datasets	H	C	N	O
CPSO	3.123	4.967	7.005	4.661
BBPSO	2.671	5.276	5.890	3.662
RPSO	3.174	5.716	8.562	4.436
FIPSO	3.194	5.452	8.267	4.340
BA	3.196	5.309	9.343	5.322
dBA	3.372	5.636	9.218	5.136
FFA	3.210	5.446	7.748	4.492
CFFA	3.317	5.466	8.649	4.689
GWO	**2.658**	**3.754**	4.337	**2.921**
RW-GWO	2.831	3.665	**3.879**	3.172

nitrogen (N), and oxygen (O), respectively, as attained by the most effective optimizers. The generalization error associated with each of these ML-II-based GPR models could indeed have been minimized by increasing the size of the training dataset. In the context of the study [29], the data augmentation process could be structured to ensure that the supplementary points are gathered from areas of the PES that appear to be underrepresented in the initial training datasets. But expanding the training set's size isn't always feasible, particularly when the target values need to be derived from costly QM computations. It has been demonstrated in [29] that, with the same size of the training set, significantly improved models can be achieved by choosing the most suitable training protocol, i.e., the optimizer and cost function.

As Table 7 shows, the performance patterns of the ten MHS methods across the four elements are not consistent. This observation demonstrates that the training of atomic GPR models constitutes a distinct optimization challenge, tailored to individual atoms of an element, and corresponds with the reality that the cost function's landscape changes depending on the dataset. Understanding the function of an effective optimizer is already made easier by a close examination of the ML-II-RW-GWO-N and ML-II-BA-N models. Despite the fact that both were trained by maximizing the LML function (ML-II protocol), the accuracy of the first model is around 2.4 times higher than that of the second. The same finding holds true for further models, namely, ML-II-GWO-C and ML-II-dBA-C, which showed corresponding average predictive MAEs of 3.754 and 5.636 kJ/mol. The significance of selecting an appropriate cost function will be more evident in the following section, where the outcomes for models based on IHOCV will be presented.

Table 8 The average predictive MAEs in kJ/mol for atomic GPR models based on IHOCV, which were trained utilizing ten distinct optimization algorithms. Each model underwent training using a dataset of 3000 points and was subsequently evaluated on a separate set of 2000 points. Each optimization run was allowed to proceed for a maximum of 10,000 iterations, a limit established to avoid premature convergence. The values highlighted in bold represent the algorithm that demonstrates the highest performance [29]

Optimizer/datasets	H	C	N	O
CPSO	2.444	3.871	5.127	3.391
BBPSO	2.029	3.160	4.539	3.479
RPSO	2.424	3.771	5.039	3.303
FIPSO	2.180	3.301	4.187	2.695
BA	2.571	4.060	5.715	3.197
dBA	2.530	3.706	5.328	3.511
FFA	2.683	3.990	5.607	3.552
CFFA	2.521	3.807	5.574	3.581
GWO	**0.922**	**1.586**	1.975	1.422
RW-GWO	1.050	1.675	**1.962**	**1.145**

Optimization of RMSE(ϕ) Cost Function

The effectiveness of the selected metaheuristic optimizers was subsequently assessed in accordance with the IHOCV protocol. In this alternative protocol, the hyperparameters of the model are derived through the minimization of the predictive RMSE of intermediate models across an internal validation set. The aforementioned dataset should be sufficiently large to enable a dependable assessment of the generalization error of intermediate models. The internal validation set should also be kept at a reasonable size, as an excessively large set may compromise the model's flexibility and unnecessarily extend the training duration.

Table 8 provides a comprehensive list of average MAEs. The same table demonstrates that, similar to ML-II-based models, the performance of the ten optimizers across the four elements lacks consistency. IHOCV-based models demonstrate a significantly higher level of accuracy compared to their ML-II-based counterparts. For example, the average MAEs presented in Table 8 are calculated to be 0.922, 1.586, 1.962, and 1.145 kJ/mol for the models of H, C, N, and O, respectively, utilizing the most effective optimizers. Compared to the data gathered in Table 7 for ML-II-based models, these values are 1.5–2.3 times lower.

Conclusion

In this chapter, the authors have explored the most recent progressions in the implementation of MHS algorithms in the context of supervised ML techniques. Evidence indicates that the application of MHS algorithms in training supervised models significantly improves the learning process. This reportedly enhances the

accuracy of the algorithm while simultaneously decreasing its execution duration. The synergy between metaheuristics and supervised methods establishes a solid basis for the supervised ML process, thereby augmenting their performance. Nonetheless, there is still opportunity to utilize these hybrid techniques in diverse applications and assess their effectiveness against various challenging real-world datasets.

It is challenging to determine if supervised techniques (especially the likes of ANN or DL models) will maintain their position at the cutting edge of academia without the incorporation of metaheuristics. In the forthcoming years, it is anticipated that the integration of supervised ML with MHS algorithm will enhance the training process and sustain elevated performance levels. It has been observed that the evolutionary hybrid architecture remains somewhat restricted in its applicability within the existing literature and an analysis of the current studies reveals that the application of MHS algorithms for the optimization of supervised ML models continues to pose significant challenges, highlighting the need for additional research in this domain. There is an expectation that the application of metaheuristics will grow in the years ahead, contributing to the improvement of the performance of different supervised architectures. Yet, there is still a lack of sufficient relevant publications in this area.

Competing Interest Declaration The author(s) has no competing interests to declare that are relevant to the content of this manuscript.

References

1. Han J, Kamber M, Pei J (2011) Data mining: concepts and techniques. Morgan Kaufmann, Waltham, pp 471–47
2. Brahma D, Paral P, Chatterjee A, Rakshit A (2022) Color template matching based on fuzzy density clustering for vision sensor based shoe detection in human-robot coexisting environment. IEEE Sensors J 22(14):14320–14327
3. Ghosh S, Paral P, Chatterjee A, Munshi S (2023) Rough entropy-based fused granular features in 2-D locality preserving projections for high-dimensional vision sensor data. IEEE Sensors J 23(16):18374–18383
4. Paral P, Chatterjee A, Rakshit A (2021) Human position estimation based on filtered sonar scan matching: a novel localization approach using DENCLUE. IEEE Sensors J 21(6):8055–8064
5. Paral P, Ghosh S, Chatterjee A, Pal SK (2022) Automatic relevance determination kernel-embedded Gaussian process regression for sonar based human leg localization with a mobile robot. IEEE Sensors Lett 7(1):1–4
6. Paral P, Chatterjee A, Rakshit A, Pal SK (2023) Extended target tracking in human-robot coexisting environments via multisensor information fusion: a heteroscedastic Gaussian process regression-based approach. IEEE Trans Ind Inf 19(9):9877–9886
7. Kaveh M, Saadi Mesgari M (2023) Application of meta-heuristic algorithms for training neural networks and deep learning architectures: a comprehensive review. Neural Process Lett 55:4519–4622
8. Lan K, Liu L, Li T, Chen Y, Fong S, Marques JAL, Tang R (2020) Multi-view convolutional neural network with leader and long-tail particle swarm optimizer for enhancing heart disease and breast cancer detection. Neural Comput Appl 32:15469–15488

9. Kilicarslan S, Celik M, Sahin Ş (2021) Hybrid models based on genetic algorithm and deep learning algorithms for nutritional Anemia disease classification. Biomed Signal Proc Control 63:102231
10. Son NN, Chinh TM, Anh HPH (2020) Uncertain nonlinear system identification using Jaya-based adaptive neural network. Soft Comput 24:17123–17132
11. Müller-Merbach H (1981) Heuristics and their design: a survey. Eur J Oper Res 8(1):1–23
12. Boussaïd I, Lepagnot J, Siarry P (2013) A survey on optimization metaheuristics. Inf Sci 237:82–117
13. Paral P, Chatterjee A, Rakshit A, Pal SK (2022) Sonar-based human leg localization using chaos enhanced dynamic neighborhood learning-based GSA aided sNDT algorithm. IEEE Trans Instrument Measure 71:2520412
14. Cheng S, Shi Y, Qin Q, Zhang Q, Bai R (2015) Population diversity maintenance in brain storm optimization algorithm. J Artif Intell Soft Comput Res 4(2): 83–97
15. Pandiaraja P, Deepa N (2019) A novel data privacy-preserving protocol for multi- data users by using genetic algorithm. Soft Comput 23(18):8539–8553
16. Azam Z, Islam MM, Huda MN (2023) Comparative analysis of intrusion detection systems and machine learning-based model analysis through decision tree. IEEE Access 11:80348–80391
17. Liao HJ, Lin CHR, Lin YC, Tung KY (2013) Intrusion detection system: a comprehensive review. J Netw Comput Appl 36(1):16–24
18. Xu C, Shen J, Du X, Zhang F (2018) An intrusion detection system using a deep neural network with gated recurrent units. IEEE Access 6:48697–48707
19. Kanna R, Santhi P (2022) Hybrid intrusion detection using MapReduce based black widow optimized convolutional long short-term memory neural networks. Expert Syst Appl 194:116545
20. Karaboga D, Basturk B (2007) A powerful and efficient algorithm for numerical function optimization: artificial bee colony (ABC) algorithm. J Global Optim 39:459–471
21. Safaldin M, Otair M, Abualigah L (2020) Improved binary gray wolf optimizer and SVM for intrusion detection system in wireless sensor networks. J Ambient Intell Humaniz Comput 12:1559–1576
22. Sumaiya Thaseen I, Saira Banu J, Lavanya K, Rukunuddin Ghalib M, Abhishek K (2020) An integrated intrusion detection system using correlation-based attribute selection and artificial neural network. Trans Emerg Telecommun Technol 32(2):e4014
23. Nguyen MT, Kim K (2020) Genetic convolutional neural network for intrusion detection systems. Future Gener Comput Syst 113:418–427
24. Khan MA (2021) HCRNNIDS: hybrid convolutional recurrent neural network-based network intrusion detection system. Processes 9(5):834
25. Kanna R, Santhi P (2021) Unified deep learning approach for efficient intrusion detection system using integrated spatial-temporal features. Knowl-Based Syst 226:107132
26. Rauscher S, Gapsys V, Gajda MJ, Zweckstetter M, de Groot BL, Grubmüller H (2015) Structural ensembles of intrinsically disordered proteins depend strongly on force field: a comparison to experiment. J Chem Theory Comput 11:5513–5524
27. Burn MJ, Popelier PLA (2022) ICHOR: a modern pipeline for producing Gaussian process regression models for atomistic simulations. Mater Adv 3(23):8729-8739
28. Burn MJ, Popelier PLA (2022) FEREBUS: a high-performance modern Gaussian process regression engine. Digital Discovery 2:152–164
29. Isamura BK, Popelier PLA (2023) Metaheuristic optimisation of Gaussian process regression model hyperparameters: insights from FEREBUS. Artif Intell Chem 1(2):100021
30. Neunhoeffer M, Sternberg S (2019) How cross-validation can go wrong and what to do about it. Polit Analy 27(1):101–106
31. Guevara-Vela JM, Francisco E, Rocha-Rinza T, Martín Pendás Á (2020) Interacting quantum atoms–a review. Molecules 25(17):4028
32. Rasmussen CE, Williams CK (2006) Gaussian processes for machine learning. Springer, Berlin
33. Kennedy J, Eberhart R (1995) Particle swarm optimization. In: Proceedings of ICNN'95 - international conference on neural networks, IEEE, pp 1942–1948

34. Mendes R, Kennedy J, Neves, J (2004) The fully informed particle swarm: simpler, maybe better. IEEE Trans Evolut Comput 8(3):204–210
35. Liu W, Wang Z, Zeng N, Yuan Y, Alsaadi FE, Liu X (2020) A novel randomised particle swarm optimizer. Int J Mach Learn Cybern 12:529–540
36. Chen C-H (2013) A revised bare bone particle swarm optimizer and its variant. In: International conference on fuzzy theory and its applications (iFUZZY), IEEE, pp 488–493
37. Mirjalili S, Mirjalili SM, Lewis A (2014) Grey wolf optimizer. Adv Eng Softw 69:46–61
38. Gupta S, Deep K (2019) A novel random walk grey wolf optimizer. Swarm Evolut Comput 44:101–112
39. Yang X-S, Hossein Gandomi A (2012) Bat algorithm: a novel approach for global engineering optimization. Eng Comput 29:464–483
40. Chakri A, Khelif R, Benouaret M, Yang X-S (2017) New directional bat algorithm for continuous optimization problems. Expert Syst Appl 69:159–175
41. Yang X-S, He X (2013) Firefly algorithm: recent advances and applications. Int J Swarm Intell 1(1):36–50
42. Gandomi AH, Yang XS, Talatahari S, Alavi AH (2013) Firefly algorithm with chaos. Commun Nonlinear Sci Numer Simul 18(1):89–98
43. Wang J, Wang W, Kollman PA, Case DA (2006) Automatic atom type and bond type perception in molecular mechanical calculations. J Molecular Graph Modell 25(2):247–260
44. Mills MJL, Popelier PLA (2014) Electrostatic Forces: formulae for the first derivatives of a polarisable, anisotropic electrostatic potential energy function based on machine learning. J Chem Theory Comput 10:3840–3856

Matheuristics by Examples

8

Martina Fischetti and Matteo Fischetti

Contents

Introduction . . . 178
General-Purpose MIP-Based Heuristics . . . 179
Local Branching . . . 180
Relaxation-Induced Neighborhood Search . . . 181
Polishing a Feasible Solution . . . 181
Proximity Search . . . 182
Application 1: Wind Farm Layout Optimization . . . 182
Choice of the MIP Model . . . 183
Choice of Ad Hoc Heuristics . . . 186
The Overall Matheuristic . . . 187
Computational Results . . . 188
Application 2: Prepack Optimization . . . 189
Mathematical Model . . . 192
Matheuristics . . . 194
Computational Experiments . . . 196
Application 3: Vehicle Routing . . . 200
The ASSIGN Neighborhood for TSP . . . 200
From TSP to DCVRP . . . 202
Conclusion . . . 209
Cross-References . . . 209
References . . . 209

M. Fischetti (✉)
Department of Statistics and Operations Research, University of Seville, Seville, Spain
e-mail: mfischetti@us.es

M. Fischetti
DEI, University of Padova, Padova, Italy
e-mail: matteo.fischetti@unipd.it

© Springer Nature Switzerland AG 2025

R. Martí et al. (eds.), *Handbook of Heuristics*,
https://doi.org/10.1007/978-3-032-00385-0_14

Abstract

As its name suggests, a matheuristic is the hybridization of mathematical programming with metaheuristics. The hallmark of matheuristics is the central role played by the mathematical programming model, around which the overall heuristic is built. As such, matheuristic is not a rigid paradigm but rather a concept framework for the design of mathematically sound heuristics. The aim of this chapter is to introduce the main matheuristic ideas. Three specific applications in the fields of wind farm, packing, and vehicle routing optimization, respectively, are addressed and used to illustrate the main features of the method.

Keywords

Heuristics · Large scale neighborhood search · Local branching · Mathematical programming · Matheuristics

Introduction

The design of heuristics for difficult optimization problems is itself a heuristic process that often involves the following main steps.

After a clever analysis of the problem at hand and of the acceptable simplifications in its definition, one tries to set up an effective **mathematical programming** (MP) model and to solve it by a general-purpose piece of software—often a mixed-integer linear programming (MIP) solver. Due to the impressive improvement of general-purpose solvers in recent years, this approach can actually solve the instances of interest to proven optimality (or with an acceptable approximation) within a reasonable computing time, in which case of course no further effort is needed.

If this is not the case, one can insist on the MP approach and try to obtain better and better results by improving the model and/or by enhancing the solver with specialized features (cutting planes, branching, etc.). Or one can forget about MP and resort to ad hoc heuristics not based on the MP model. In this latter case, the MP model is completely disregarded or just used for illustrating the problem characteristics and/or for getting an offline indication of the typical approximation error on a set of sample instances.

A third approach is however possible that consists in using the MP solver as a basic tool *within* the heuristic framework. This hybridization of MP with *metaheuristics* leads to the *matheuristic* approach, where the heuristic is built around the MP model. Matheuristics became popular in recent years, as witnessed by the publication of dedicated volumes and journal special issues [8, 19, 24] and by the dedicated sessions on MP and metaheuristic conferences.

Designing an effective heuristic is an art that cannot be framed into strict rules. This is particularly true when addressing a matheuristic, which is not a

rigid paradigm but a concept framework for the design of mathematically sound heuristics. In this chapter, we will therefore try to illustrate some main matheuristic features with the help of different examples of application.

Section "General-Purpose MIP-Based Heuristics" describes powerful general-purpose MIP heuristics that can be used within the matheuristic framework. Interestingly, these heuristics can themselves be viewed as the first successful applications of the matheuristic idea of hybridizing MP and metaheuristics. Indeed, as noticed in [8], one of the very first illustrations of the power of the matheuristic idea is the general-purpose local branching [7] paradigm, where a black-box MIP solver is used to explore a solution neighborhood defined by invalid constraints added to the MIP model for the sake of easing its solution.

Section "Application 1: Wind Farm Layout Optimization" addresses the design of a matheuristic for wind farm optimization. This application is used to illustrate the importance of the choice of the MIP model: models that are weak in polyhedral terms can be preferred to tighter—but computationally much harder—models when heuristic (as opposed to exact) solutions are required.

Section "Application 2: Prepack Optimization" addresses a packing problem where the model is nonlinear, and the matheuristic is based on various ways to linearize it after a heuristic fixing of some variables.

Finally, section "Application 3: Vehicle Routing" is used to illustrate an advanced feature of matheuristics, namely, the solution of auxiliary MP models that describe a subproblem in the solution process. In particular, we address a vehicle routing problem and derive a matheuristic based on a set-partitioning MIP model asking for the reallocation of a subset of customer sequences subject to capacity and distance constraints.

The present chapter is based on previously published work; in particular, sections "General-Purpose MIP-Based Heuristics," "Application 1: Wind Farm Layout Optimization," "Application 2: Prepack Optimization," and "Application 3: Vehicle Routing" are based on [8, 11, 13, 15], respectively.

General-Purpose MIP-Based Heuristics

Heuristics for general-purpose MIP solvers form the basis of the matheuristic's toolkit. Their relevance to our chapter is twofold. On the one hand, they are invaluable tools for the solution of the subproblems tailored by the matheuristic when applied to a specific problem. On the other hand, they illustrate the benefits for a general-purpose MIP solver deriving from the use of metaheuristic concepts such as local search and evolutionary methods.

Modern MIP solvers exploit a rich arsenal of tools to attack hard problems. It is widely accepted that the solution of hard MIPs can take advantage from the solution of a series of auxiliary linear programs (LPs) intended to enhance the performance of the overall MIP solver. For example, auxiliary LPs may be solved to generate powerful disjunctive cuts or to implement a strong branching policy. On the other

hand, it is a common experience that finding good-quality heuristic MIP solutions often requires a computing time that is just comparable to that needed to solve the LP relaxation. So, it makes sense to think of exact/heuristic MIP solvers where auxiliary MIPs (as opposed to LPs) are heuristically solved on the fly, with the aim of bringing the MIP technology under the chest of the MIP solver itself. This leads to the idea of "translating into a MIP model" (*MIPping* in the jargon of [9]) some crucial decisions to be taken when designing a MIP-based algorithm.

We next describe the new generation of MIP heuristics that emerged in the late 1990s, which are based on the idea of systematically using a "black-box" external MIP solver to explore a solution neighborhood defined by invalid linear constraints. We address a generic MIP of the form

$$\text{(MIP)} \qquad \min c^T x \tag{1}$$

$$Ax \geq b, \tag{2}$$

$$x_j \in \{0, 1\}, \quad \forall j \in \mathcal{B}, \tag{3}$$

$$x_j \text{ integer}, \quad \forall j \in \mathcal{G}, \tag{4}$$

$$x_j \text{ continuous}, \quad \forall j \in \mathcal{C}, \tag{5}$$

where A is an $m \times n$ input matrix and b and c are input vectors of dimension m and n, respectively. Here, the variable index set $\mathcal{N} := \{1, \ldots, n\}$ is partitioned into $(\mathscr{B}, \mathcal{G}, \mathcal{C})$, where $\mathscr{B}$ is the index set of the 0–1 variables (if any), while sets $\mathcal{G}$ and $\mathcal{C}$ index the general integer and the continuous variables, respectively. Removing the integrality requirement on variables indexed by $\mathcal{I} := \mathcal{B} \cup \mathcal{G}$ leads to the so-called *LP relaxation*.

Local Branching

The *local branching* (LB) scheme of Fischetti and Lodi [7] appears to be one of the first general-purpose heuristics using a black-box MIP solver applied to sub-MIPs, and it can be viewed as a precursor of matheuristics. Given a *reference solution* $\overline{x}$ of a MIP with $\mathcal{B} \neq \emptyset$, one aims at finding an improved solution that is "not too far" from $\overline{x}$, in the sense that not too many binary variables need be flipped. To this end, one can define the k-opt neighborhood $\mathcal{N}(\overline{x}, k)$ of $\overline{x}$ as the set of the MIP solutions satisfying the invalid *local branching constraint*

$$\Delta(x, \overline{x}) := \sum_{j \in \mathcal{B}: \overline{x}_j = 0} x_j + \sum_{j \in \mathcal{B}: \overline{x}_j = 1} \left(1 - x_j\right) \leq k, \tag{6}$$

for a small neighborhood radius k—an integer parameter typically set to 10 or 20. The neighborhood is then explored (possibly heuristically, i.e., with some small node or time limit) by means of a black-box MIP solver. Experimental results [10] show that the introduction of the local branching constraint typically has the positive effect of driving to integrality many component of the optimal solution of the LP relaxation, improving the so-called relaxation grip and hence the capability of the MIP solver to find (almost) optimal integer solutions within short computing times. Of course, this effect is lost if parameter k is set to a large value—a mistake that would make local branching completely ineffective.

LB is in the spirit of local search metaheuristics and, in particular, of *large-neighborhood search* (LNS) [29], with the novelty that neighborhoods are obtained through "soft fixing," that is, through invalid cuts to be added to the original MIP model. Diversification cuts can be defined in a similar way, thus leading to a flexible toolkit for the definition of metaheuristics for general MIPs.

Relaxation-Induced Neighborhood Search

The *relaxation-induced neighborhood search* (RINS) heuristic of Danna, Rothberg, and Le Pape [4] also uses a black-box MIP solver to explore a neighborhood of a given solution $\overline{x}$ and was originally designed to be integrated in a branch-and-bound solution scheme. At specified nodes of the branch-and-bound tree, the current LP relaxation solution x^* and the incumbent $\overline{x}$ are compared, and all integer-constrained variables that agree in value are fixed. The resulting MIP is typically easy to solve, as fixing reduces its size considerably, and often provides improved solutions with respect to $\overline{x}$.

Polishing a Feasible Solution

The *polishing* algorithm of Rothberg [27] implements an evolutionary MIP heuristic which is invoked at selected nodes of a branch-and-bound tree and includes all classical ingredients of genetic computation, namely:

- *Population*: A fixed-size population of feasible solutions is maintained. These solutions are either obtained within the branch-and-bound tree (by other heuristics) or computed by the polishing algorithm itself.
- *Combination*: Two or more solutions (the parents) are combined with the aim of creating a new member of the population (the child) with improved characteristics. The RINS scheme is adopted, that is, all variables whose value coincides in the parents are fixed, and the reduced MIP is heuristically solved by a black-box MIP solver within a limited number of branch-and-bound nodes. This scheme is clearly much more time-consuming than a classical combination step in evolutionary algorithms, but it guarantees the feasibility of the child solution.

- *Mutation*: Diversification is obtained by performing a classical mutation step that (i) randomly selects a "seed" solution in the population, (ii) randomly fixes some of its variables, and (iii) heuristically solves the resulting reduced MIP.
- *Selection*: Selection of the two parents to be combined is performed by randomly picking a solution in the population and then choosing, again at random, the second parent among those solutions with a better objective value.

Proximity Search

Proximity search [10] is a "dual version" of local branching that tries to overcome the issues related to the choice of the neighborhood radius k. Instead of hard-fixing the radius, proximity search fixes the minimum improvement of the solution value and changes the objective function to favor the search of solutions at small Hamming distance with respect to the reference one.

The approach works in stages, each aimed at producing an improved feasible solution. As in LB or RINS, at each stage a reference solution $\overline{x}$ is given, and one aims at improving it. To this end, an explicit cutoff constraint

$$c^T x \leq c^T \overline{x} - \theta \tag{7}$$

is added to the original MIP, where $\theta > 0$ is a given tolerance that specifies the minimum improvement required. The objective function of the problem can then be replaced by the proximity function $\Delta(x, \overline{x})$ defined in (6), to be minimized. One then applies the MIP solver, as a black box, to the modified problem in the hope of finding a solution better than $\overline{x}$. Computational experience confirms that this approach is quite successful (at least, on some classes of problems), due to the action of the proximity objective function that improves the "relaxation grip" of the model.

A simple variant of the above scheme, called "proximity search with incumbent," is based on the idea of providing $\overline{x}$ to the MIP solver as a starting solution. To avoid $\overline{x}$ be rejected because of the cutoff constraint (7), the latter is weakened to its "soft" version:

$$c^T x \leq c^T \overline{x} - \theta\,(1 - \xi) \tag{8}$$

while minimizing $\Delta(x, \overline{x}) + M\xi$ instead of just $\Delta(x, \overline{x})$, where $\xi \geq 0$ is a continuous slack variable and $M \gg 0$ is a large penalty.

Application 1: Wind Farm Layout Optimization

Green energy became a topic of great interest in recent years, as environmental sustainability asks for a considerable reduction in the use of fossil fuels. The *wind*

farm layout optimization problem aims at finding an allocation of turbines in a given site so as to maximize power output. This strategic problem is extremely hard in practice, both for the size of the instances in real applications and for the presence of several nonlinearities to be taken into account. A typical nonlinear feature of this problem is the interaction among turbines, also known as wake effect. The wake effect is the interference phenomenon for which, if two turbines are located one close to another, the upwind one creates a shadow on the one behind. Interference is therefore of great importance in the design of the layout as it results into a loss of power production for the turbine downstream.

We next outline the main steps in the design of a sound matheuristic scheme for wind farm layout optimization that is able to address the large-size instances arising in practical applications.

Choice of the MIP Model

Different models have been proposed in the literature to describe interference. We will consider first a simplified model from the literature [5], where the overall interference is the sum of pairwise interferences between turbine pairs. The model addresses the following constraints:

(a) A minimum and maximum number of turbines that can be built is given;
(b) There should be a minimal separation distance between two turbines to ensure that the blades do not physically clash (turbine distance constraints);
(c) If two turbines are installed, their interference will cause a loss in the power production that depends on their relative position and on wind conditions.

Let V denote the set of possible positions for a turbine, called "sites" in what follows, and let:

- N_{MIN} and N_{MAX} be the minimum and maximum number of turbines that can be built, respectively.
- D_{MIN} be the minimum distance between two turbines.
- $\text{dist}(i, j)$ be the Euclidean distance between sites i and j.
- I_{ij} be the interference (loss of power) experienced by site j when a turbine is installed at site i, with $I_{jj} = 0$ for all $j \in V$.
- P_i be the power that a turbine would produce if built (alone) at site i.

In addition, let $G_I = (V, E_I)$ denote the incompatibility graph with

$$E_I = \{[i, j] \in V \times V:\ \text{dist}(i, j) < D_{\text{MIN}},\ i < j\} \text{ let } n := |V|$$

denote the total number of sites. Two sets of binary variables are defined:

$$x_i = \begin{cases} 1 & \text{if a turbine is built at site } i; \\ 0 & \text{otherwise} \end{cases} \qquad (i \in V)$$

$$z_{ij} = \begin{cases} 1 & \text{if two turbines are built at both sites } i \text{ and } j; \\ 0 & \text{otherwise} \end{cases} \qquad (i, j \in V, i < j)$$

The model then reads

$$\max \sum_{i \in V} P_i x_i - \sum_{i \in V} \sum_{j \in V, i < j} \left(I_{ij} + I_{ji} \right) z_{ij} \tag{9}$$

$$\text{s.t.} \qquad N_{\text{MIN}} \leq \sum_{i \in V} x_i \leq N_{\text{MAX}} \tag{10}$$

$$x_i + x_j \leq 1 \qquad \forall [i, j] \in E_I \tag{11}$$

$$x_i + x_j - 1 \leq z_{ij} \qquad \forall i, j \in V, i < j \tag{12}$$

$$x_i \in \{0, 1\} \qquad \forall i \in V \tag{13}$$

$$z_{ij} \in \{0, 1\} \qquad \forall i, j \in V, i < j \tag{14}$$

Objective function (9) maximizes the total power production by taking interference losses I_{ij} into account. Constraints (11) model pairwise site incompatibility. Constraints (12) force $z_{ij} = 1$ whenever $x_i = x_j = 1$; because of the objective function, this is in fact equivalent to imposing $z_{ij} = x_i x_j$.

The definition of the turbine power vector (P_i) and of interference matrix (I_{ij}) depends on the wind scenario considered, which greatly varies in time. Using statistical data, one can in fact collect a large number K of wind scenarios k, each associated with a pair (P^k, I^k) with a probability π_k, and define the average power and interference to be used in the model as:

$$P_i := \sum_{k=1}^{K} \pi_k P_i^k \qquad \forall i \in V \tag{15}$$

$$I_{ij} := \sum_{k=1}^{K} \pi_k I_{ij}^k \qquad \forall i, j \in V \tag{16}$$

While (9), (10), (11), (12), (13), and (14) turn out to be a reasonable model when just a few sites have to be considered (say $n \approx 100$), it becomes hopeless when

$n \geq 1000$ because of the huge number of variables and constraints involved, which grow quadratically with n. Therefore, when facing instances with several thousand sites, an alternative (possibly weaker) model is required, where interference can be handled by a number of variables and constraints that grow just linearly with n. The model below is a compact reformulation of model (9), (10), (11), (12), (13), and (14) that follows a recipe of Glover [17] that is widely used, for example, in the quadratic assignment problem [12, 32]. The original objective function (to be maximized), rewritten as:

$$\sum_{i \in V} P_i x_i - \sum_{i \in V} \left(\sum_{j \in V} I_{ij} x_j \right) x_i \tag{17}$$

is restated as:

$$\sum_{i \in V} (P_i x_i - w_i) \tag{18}$$

where

$$w_i := \left(\sum_{j \in V} I_{ij}\, x_j \right) x_i = \begin{cases} \sum_{j \in V} I_{ij} x_j & \text{if } x_i = 1 \\ 0 & \text{if } x_i = 0 \end{cases}$$

$$\max z = \sum_{i \in V} (P_i x_i - w_i) \tag{19}$$

$$\text{s.t.} \qquad N_{\text{MIN}} \leq \sum_{i \in V} x_i \leq N_{\text{MAX}} \tag{20}$$

$$x_i + x_j \leq 1 \qquad \forall [i, j] \in E_I \tag{21}$$

$$\sum_{j \in V} I_{ij}\, x_j \leq w_i + M_i\, (1 - x_i) \qquad \forall i \in V \tag{22}$$

$$x_i \in \{0, 1\} \qquad \forall i \in V \tag{23}$$

$$w_i \geq 0 \qquad \forall i \in V \tag{24}$$

where the big-M term $M_i = \sum_{j \in V : [i,j] \notin E_I} I_{ij}$ is used to deactivate constraint (22) in case $x_i = 0$, in which case $w_i = 0$ because of the objective function.

Choice of Ad Hoc Heuristics

A simple 1-opt heuristic can be designed along the following lines. At each step, we have an incumbent solution, say $\tilde{x}$, that describes the best-known turbine allocation ($\tilde{x}_i = 1$ if a turbine is built at site i, 0 otherwise), and a current solution x. Let

$$z = \sum_{i \in V} P_i x_i - \sum_{i \in V} \sum_{j \in V} I_{ij}\, x_i\, x_j$$

be the profit of the current solution, $\gamma = \sum_{i \in V} x_i$ be its cardinality, and define for each $j \in V$ the extra-profit δ_j incurred when flipping x_j, namely:

$$\delta_j = \begin{cases} P_j - \sum\limits_{i \in V : x_i = 1} \left(I_{ij} + I_{ji}\right) & \text{if } x_j = 0; \\ -P_j + \sum\limits_{i \in V : x_i = 1} \left(I_{ij} + I_{ji}\right) & \text{if } x_j = 1 \end{cases}$$

where we assume $I_{ij} = BIG$ for all incompatible pairs $[i,j] \in E_I$, and BIG $> \sum_{i \in V} P_i$ is a large penalty value, while $I_{ii} = 0$ as usual.

We start with $x = 0$, $z = 0$, and $\gamma = 0$ and initialize $\delta_j = P_j$ for all $j \in V$. Then, we iteratively improve x by a sequence of 1-opt moves, according to the following scheme. At each iteration, we look in $O(n)$ time for the site j with maximum $\delta_j + \text{FLIP}(j)$, where function $FLIP(j)$ takes cardinality constraints into account, namely

$$\text{FLIP}(j) = \left\{ \begin{array}{lll} -\text{HUGE} & \text{if } x_j = 0 \text{ and } \gamma \geq N_{\text{MAX}} \\ -\text{HUGE} & \text{if } x_j = 1 \text{ and } \gamma \leq N_{\text{MIN}} \\ +\text{HUGE} & \text{if } x_j = 0 \text{ and } \gamma < N_{\text{MIN}} \\ +\text{HUGE} & \text{if } x_j = 1 \text{ and } \gamma > N_{\text{MAX}} \\ 0 & \text{otherwise} \end{array} \right\}$$

Once the best j has been found, say j^*, if $\delta_{j^*} + FLIP(j^*) > 0$, we just flip x_{j^*}; update x, z, and γ in $O(1)$ time; update all δ_j's in $O(n)$ time (through the parametric technique described in [11]); and repeat. In this way, a sequence of improving solutions is obtained, until a local optimal solution that cannot be improved by just one flip is found. To escape local minima, a simple perturbation scheme can be implemented; see again [11] for details.

A 2-opt heuristic can similarly be implemented to allow a single turbine to move to a better site—a move that requires flipping two variables. Each 2-opt exchange requires $O(n^2)$ time as it amounts to trying n 1-opt exchanges and to apply the best one.

The Overall Matheuristic

Our final approach is a mixture of ad hoc (1- and 2-opt) and general MIP (proximity search with incumbent) heuristics and works as shown in Algorithm 1.

Algorithm 1: The Overall Matheuristic Framework
1: read input data and compute the overall interference matrix (I_{ij});
2: apply ad hoc heuristics (1- and 2-opt) to get a first incumbent $\tilde{x}$;
3: **while** time limit permits **do**
4: apply quick ad hoc refinement heuristics (few iterations of 1- and 2-opt) to possibly improve $\tilde{x}$;
5: if $n > 2000$, randomly remove points $i \in V$ with $\tilde{x}_i = 0$ so as to reduce the number of candidate sites to 2000;
6: build a MIP model for the resulting subproblem and apply proximity search to refine $\tilde{x}$ until the very first improved solution is found (or time limit is reached);
7: **end while**
8: **return** $\tilde{x}$

At Step 2, the heuristics of the section "Choice of Ad Hoc Heuristics" are applied in their "initial-solution" mode, where one starts with $\tilde{x} = x = 0$ and aborts execution when 1-opt is invoked 10,000 consecutive times without improving $\tilde{x}$. At Step 4, instead, a faster "cleanup" mode is applied. As we already have a hopefully good incumbent $\tilde{x}$ to refine, we initialize $x = \tilde{x}$ and repeat the procedure until we count 100 consecutive 1-opt calls with no improvement of $\tilde{x}$. As to time-consuming 2-opt exchanges, they are applied with a certain frequency and in any case just before the final $\tilde{x}$ is returned.

Two different MIP models are used to feed the proximity-search heuristic at Step 6. During the first part of the computation, we use a simplified MIP model obtained from (19), (20), (21), (22), (23), and (24) by removing all interference constraints (22), thus obtaining a much easier problem. A short time limit is imposed for each call of proximity search when this simplified model is solved. In this way, we aggressively drive the solution $\tilde{x}$ to increase the number of built turbines, without being bothered by interference considerations and only taking pairwise incompatibility (21) into account. This approach quickly finds better and better solutions (even in terms of the true profit), until either (i) no additional turbine can be built or (ii) the addition of new turbines does in fact *reduce* the true profit associated with the new solution because of the neglected interference. In this situation, we switch to the complete model (19), (20), (21), (22), (23), and (24) with all interference constraints, which is used in all next executions of Step 6. Note that the simplified model is only used at Step 6, while all other steps of the procedure always use the true objective function that takes interference into full account.

Computational Results

The following alternative solution approaches were implemented in C language, some of which using the commercial MIP-solver IBM ILOG Cplex 12.5.1 [21]; because of the big-Ms involved in the models, all Cplex's codes use zero as integrality tolerance (`CPX_PARAM_EPINT` $= 0.0$).

(a) `proxy`: The matheuristic outlined in the previous section, built on top of Cplex with the following aggressive parameter tuning: all cuts deactivated, `CPX_PARAM_RINSHEUR` $= 1$, `CPX_PARAM_POLISHAFTERTIME` $= 0.0$, `CPX_PARAM_INTSOLLIM` $= 2$;
(b) `cpx_def`: The application of IBM ILOG Cplex 12.5.1 in its default setting, starting from the same heuristic solution $\tilde{x}$ available right after the first execution of Step 2 of Algorithm 1;
(c) `cpx_heu`: Same as `cpx_def`, with the following internal tuning intended to improve Cplex's heuristic performance: all cuts deactivated, `CPX_PARAM_RINSHEUR` $= 100$, `CPX_PARAM_POLISHAFTERTIME` $= 20\%$ of the total time limit;
(d) `loc_sea`: A simple heuristic not based on any MIP solver, that just loops on Step 4 of Algorithm 1 and randomly removes installed turbines from the current best solution after 10,000 iterations without improvement of the incumbent.

For each algorithm, we recorded the best solution found within a given time limit.

In our view, `loc_sea` is representative of a clever but not oversophisticated metaheuristic, as typically implemented in practice, while `cpx_def` and `cpx_heu` represent a standard way of exploiting a MIP model once a good feasible solution is known.

Our test bed refers to an offshore 3000×3000 (m) square with $D_{\text{MIN}} = 400$ (m) minimum turbine separation, with no limit on the number of turbines to be built (i.e., $N_{\text{MIN}} = 0$ and $N_{\text{MAX}} = +\infty$). Turbines are all of Siemens SWT-2.3-93 type (rotor diameter 93 m), which produces a power of 0.0 MW for wind speed up to 3 m/s, of 2.3 MW for wind speed greater than or equal to 16 m/s, and intermediate values for winds in range 3–16 m/s according to a nonlinear power curve [30]. Pairwise interference (in MW) was computed using Jensen's model [22], by averaging 250,000+ real-world wind samples. Those samples were grouped into about 500 macro-scenarios to reduce the computational time spent for the definition of the interference matrix. A pairwise average interference of 0.01 MW or less was treated as zero. The reader is referred to [6] for details.

We generated five classes of medium-to-large problems with n ranging from 1000 to 20,000. For each class, ten instances have been considered by generating n uniformly random points in the 3000×3000 square. (Although in the offshore case turbine positions are typically sampled on a regular grid, we decided to randomly generate them to be able to compute meaningful statistics for each value of n.)

In what follows, reported computing times are in CPU seconds of an Intel Xeon E3-1220 V2 quad-core PC with 16 GB of RAM and do not take Step 1 of Algorithm 1 into account as the interference matrix is assumed to be precomputed and reused at each run.

Computational results on our instances are given in Table 1, where each entry refers to the performance of a given algorithm at a given time limit. In particular, the left part of the table reports, for each algorithm and time limit, the *number of wins*, that is, the number of instances for which a certain algorithm produced the best-known solution at the given time limit (ties allowed).

According to the table, `proxy` outperforms all competitors by a large amount for medium-to-large instances. As expected, `cpx_heu` performs better for instances with $n = 1000$ as it is allowed to explore a large number of enumeration nodes for the original model and objective function. Note that `loc_sea` has a good performance for short time limits and/or for large instances, thus confirming its effectiveness, whereas `cpx_heu` is significantly better than `loc_sea` only for small instances and large time limits.

A different performance measure is given in the right-hand side part of Table 1, where each entry gives the *average optimality ratio*, that is, the average value of the ratio between the solution produced by an algorithm (on a given instance at a given time limit) and the best solution known for that instance—the closer to one, the better. It should be observed that an improvement of just 1% has a very significant economical impact due to the very large profits involved in the wind farm context. The results show that `proxy` is always able to produce solutions that are quite close to the best one. As before, `loc_sea` is competitive for large instances when a very small computing time is allowed, whereas `cpx_def` and `cpx_heu` exhibit a good performance only for small instances and are dominated even by `loc_sea` for larger ones.

Application 2: Prepack Optimization

Packing problems play an important role in industrial applications. In these problems, a given set of *items* has to be packed into one or more containers (*bins*) so as to satisfy a number of constraints and to optimize some objective function.

Most of the contributions from the literature are devoted to the case where all the items have to be packed into a minimum number of bins so as to minimize, for example, transportation costs; within these settings, only loading costs are taken into account. The resulting problem is known as the *bin packing problem* and has been widely studied in the literature both in its one-dimensional version [25] and in its higher-dimensional variants [23].

We will next consider a different packing problem arising in inventory allocation applications, where the operational cost for packing the bins is comparable, or even higher, than the cost of the bins themselves. This is the case, for example, for

Table 1 Number of times each algorithm finds the best-known solution within the time limit (wins) and optimality ratio with respect to the best-known solution—the larger, the better

		Number of wins				Optimality ratio			
n	Time limit (s)	proxy	cpx_def	cpx_heu	loc_sea	proxy	cpx_def	cpx_heu	loc_sea
1000	60	6	1	3	0	0.994	0.983	0.987	0.916
	300	4	2	4	0	0.997	0.991	0.998	0.922
	600	7	3	7	0	0.997	0.992	0.997	0.932
	900	5	2	3	0	0.998	0.993	0.996	0.935
	1200	5	1	5	0	0.998	0.992	0.997	0.939
	1800	5	1	4	0	0.998	0.992	0.996	0.942
	3600	4	2	5	0	0.998	0.995	0.997	0.943
5000	60	9	6	6	5	0.909	0.901	0.901	0.904
	300	10	0	0	0	0.992	0.908	0.908	0.925
	600	10	0	10	0	0.994	0.908	0.994	0.935
	900	10	0	0	0	0.994	0.908	0.908	0.936
	1200	10	0	0	0	0.994	0.908	0.925	0.939
	1800	9	0	1	0	0.996	0.908	0.971	0.946
	3600	5	0	5	0	0.996	0.932	0.994	0.948
10,000	60	9	9	8	10	0.914	0.913	0.914	0.914
	300	10	2	2	2	0.967	0.927	0.927	0.936
	600	10	0	10	0	0.998	0.928	0.998	0.944
	900	10	0	0	0	1.000	0.928	0.928	0.948
	1200	10	0	0	0	1.000	0.928	0.928	0.951
	1800	10	0	0	0	1.000	0.928	0.928	0.957
	3600	9	0	0	1	1.000	0.928	0.928	0.964

15,000	60	9	10	9	9	0.909	0.912	0.911	0.909
	300	10	8	7	8	0.943	0.937	0.935	0.937
	600	10	0	10	0	0.992	0.939	0.992	0.942
	900	10	0	0	0	1.000	0.939	0.939	0.956
	1200	9	0	0	1	1.000	0.939	0.939	0.959
	1800	9	0	0	1	1.000	0.939	0.939	0.965
	3600	9	0	0	1	1.000	0.939	0.939	0.972
20,000	60	9	9	9	10	0.901	0.902	0.901	0.902
	300	10	8	10	10	0.933	0.933	0.933	0.933
	600	9	0	9	1	0.956	0.935	0.956	0.941
	900	10	0	0	0	0.978	0.935	0.935	0.945
	1200	10	0	0	0	0.991	0.935	0.935	0.950
	1800	10	0	0	0	0.999	0.935	0.935	0.963
	3600	10	0	0	0	1.000	0.935	0.935	0.971
ALL	60	42	35	35	34	0.925	0.922	0.922	0.909
	300	44	20	23	20	0.966	0.939	0.940	0.930
	600	46	3	46	1	0.987	0.941	0.987	0.938
	900	45	2	3	0	0.994	0.941	0.941	0.944
	1200	44	1	5	1	0.997	0.940	0.945	0.947
	1800	43	1	5	1	0.999	0.940	0.954	0.955
	3600	36	2	10	2	0.999	0.946	0.959	0.959

warehouses that have to manage a large number of different customers (e.g., stores), each requiring a given set of items. Assuming that automatic systems are available for packing, the required workforce is related to the number of different ways that are used to pack the bins to be sent to the customers. To limit this cost, a hard constraint can be imposed on the total number of different box configurations that are used.

Prepacking items into box configurations has obvious benefits in terms of easier and cheaper handling, as it reduces the amount of material handled by both the warehouse and the customers. However, the approach can considerably reduce the flexibility of the supply chain, leading to situations in which the set of items that are actually shipped to each customer may slightly differ from the required one—at the expense of some cost in the objective function. In addition, an upper bound on overstocking is usually imposed for each store.

The resulting problem, known as *prepack optimization problem* (POP), was recently addressed in [20], where a real-world application in the fashion industry is presented, and heuristic approaches are derived using both constraint programming (CP) and MIP techniques.

Mathematical Model

In this section, we briefly formalize POP and review the mathematical model introduced in [20]. We are given a set I of types of products and a set S of stores. Each store $s \in S$ requires an integer number r_{is} of products of type $i \in I$. Bins with different capacities are available for packing items: we denote by $K \subset Z_+$ the set of available bin capacities.

Bins must be completely filled and are available in an unlimited number for each type. A *box configuration* describes the packing of a bin, in terms of the number of products of each type that are packed into it. We denote by NB the maximum number of box configurations that can be used for packing all products and by $B = \{1, \ldots, NB\}$ the associated set.

Products' packing into boxes is described by integer variables y_{bi}: for each product type $i \in I$ and box configuration $b \in B$, the associated variable y_{bi} indicates the number of products of type i that are packed into the b-th box configuration. In addition, integer variables x_{bs} are used to denote the number of bins loaded according to box configuration b that have to be shipped to store $s \in S$.

Understocking and overstocking of product i at store s are expressed by decisional variables u_{is} and o_{is}, respectively. Positive costs α and β penalize each unit of under- and overstocking, respectively, whereas an upper bound δ_{is} on the maximum overstocking of each product at each store is also imposed.

Finally, for each box configuration $b \in B$ and capacity value $k \in K$, a binary variable t_{bk} is introduced that takes value 1 if box configuration b corresponds to a bin of capacity k.

Additional integer variables used in the model are $q_{bis} = x_{bs}y_{bi}$ (number of items of type i sent to store s through boxes loaded with configuration b); hence, $\sum_{b \in B} q_{bis}$ gives the total number of products of type i that are shipped to store s.

A mixed-integer nonlinear programming (MINLP) model then reads:

$$\min \sum_{s \in S} \sum_{i \in I} (\alpha u_{is} + \beta o_{is}) \tag{25}$$

$$q_{bis} = x_{bs} y_{bi} \quad (b \in B; i \in I; s \in S) \tag{26}$$

$$\sum_{b \in B} q_{bis} - o_{is} + u_{is} = r_{is} \quad (i \in I; s \in S) \tag{27}$$

$$\sum_{i \in I} y_{bi} = \sum_{k \in K} k\, t_{bk} \quad (b \in B) \tag{28}$$

$$\sum_{k \in K} t_{bk} = 1 \quad (b \in B) \tag{29}$$

$$o_{is} \leq \delta_{is} \quad (i \in I; s \in S) \tag{30}$$

$$t_{bk} \in \{0, 1\} \quad (b \in B; k \in K) \tag{31}$$

$$x_{bs} \geq 0 \ \text{ integer} \quad (b \in B; s \in S) \tag{32}$$

$$y_{bi} \geq 0 \ \text{ integer} \quad (b \in B; i \in I) \tag{33}$$

The model is of course nonlinear, as the bilinear constraints (26) involve the product of decision variables. To derive a linear MIP model, the following standard technique can be used. Each x_{bs} variable is decomposed into its binary expansion using binary variables v_{bsl} ($l = 0, \ldots, L$), where L is easily computed from an upper bound on x_{bs}. When these variables are multiplied by y_{bi}, the corresponding product $w_{bisl} = v_{bsl}\, y_{bi}$ are linearized with the addition of suitable constraints.

Our final MIP is therefore obtained from (25), (26), (27), (28), (29), (30), (31), (32), and (33) by adding

$$x_{bs} = \sum_{l=0}^{L} 2^l v_{bsl} \quad (b \in B; s \in S) \tag{34}$$

$$v_{bsl} \in \{0, 1\} \quad (b \in B; s \in S; l = 0, \ldots, L) \tag{35}$$

and by replacing each nonlinear Eq. (26) with the following set of new variables and constraints:

$$q_{bis} = \sum_{l=0}^{L} 2^l w_{bisl} \quad (b \in B; i \in I; s \in S) \tag{36}$$

$$w_{bisl} \leq \overline{Y} v_{bsl} \quad (b \in B; i \in I; s \in S; l = 0, \ldots, L) \tag{37}$$

$$w_{bisl} \leq y_{bi} \quad (b \in B; i \in I; s \in S; l = 0, \ldots, L) \tag{38}$$

$$w_{bisl} \geq y_{bi} - \overline{Y}(1 - v_{bsl}) \quad (b \in B; i \in I; s \in S; l = 0, \ldots, L) \tag{39}$$

$$w_{bisl} \geq 0 \quad (b \in B; i \in I; s \in S; l = 0, \ldots, L) \tag{40}$$

where $\overline{Y}$ denotes an upper bound on the y variables.

In case all capacities are even, the following constraint—though redundant—plays a very important role in improving the LP bound of our MIP model:

$$\sum_{i \in I} (u_{is} + o_{is}) \geq 1 \quad \left(s \in S : \sum_{i \in I} r_{is} \text{ is odd} \right) \tag{41}$$

Matheuristics

The MIP model of the previous subsection is by far too difficult to be addressed by standard solvers. As a matter of fact, for real-world cases even the LP relaxation at each node turns out to be very time consuming. So we designed ad hoc heuristic approaches to exploit the special structure of our MIP, following the matheuristic paradigm.

Each heuristic is based on the idea of iteratively solving a restricted problem obtained by fixing a subset of variables, so as to obtain a subproblem which is (reasonably) easy to solve by a commercial MIP solver, but still able to produce improved solutions.

Two kinds of heuristics can be implemented: constructive and refinement heuristics. Constructive heuristics are used to find a solution H starting from scratch. In a refinement heuristic, instead, we are given a heuristic solution $H = (x^H, y^H)$ that we would like to improve. We first fix some x and/or y variables to their value in H, thus defining a solution neighborhood $\mathcal{N}(H)$ of H. We then search $\mathcal{N}(H)$ by using a general-purpose MIP solver on the model resulting from fixing. If an improved solution is found within the given time limit, we update H and repeat; otherwise, a new neighborhood is defined in the attempt to escape the local optimum.

Fixing All *x* or *y* Variables

A first obvious observation is that our basic MINLP model (25), (26), (27), (28), (29), (30), (31), (32), and (33) reduces to a linear MIP if all the x (or all the y) variables are fixed, as constraints (26) trivially become linear. According to our experience, the resulting MIP (though nontrivial) is typically solved very quickly by a state-of-the-art solver, meaning that one can effectively solve a sequence of restricted MIPs where x and y are fixed, in turn, until no further improvement can be obtained.

Let $\mathcal{N}_y(H)$ and $\mathcal{N}_x(H)$ denote the solution neighborhoods of H obtained by leaving y or x free, that is, when imposing $x = x^H$ or $y = y^H$, respectively.

A basic tool that we use in our heuristics is function REOPT (S', y^H) that considers a store subset $S' \subseteq S$ and a starting y^H and returns the best solution H obtained by iteratively optimizing over the neighborhoods $\mathcal{N}_x(H)$, $\mathcal{N}_y(H)$, $\mathcal{N}_x(H)$, etc., after having removed all stores not in S', H being updated after each optimization.

Fixing *y* Variables for All But One Configuration

Another interesting neighborhood, say $\mathcal{N}_x\left(H, y_\beta\right)$, is obtained by leaving all x variables free and by fixing $y_{bi} = y^H_{bi}$ for all $i \in I$ and $b \in B\backslash\{\beta\}$ for a given $\beta \in B$. In other words, we allow for changing just one (out of NB) configuration in the current solution, and leave the solver the possibility to change the x variables as well.

In our implementation, we first define a random permutation $\{\beta_1, \ldots, \beta_{NB}\}$ of B. We then optimize, in a circular sequence, neighborhoods $\mathcal{N}_x\left(H, y_{\beta_t}\right)$ for $t = 1, \ldots, NB, 1, \ldots$ Each time an improved solution is found, we update H and further refine it through function REOPT (S, y^H). The procedure is stopped when there is no hope of finding an improved solution, that is, after NB consecutive optimizations that do not improve the current H.

A substantial speedup can be obtained by heuristically imposing a tight upper bound on the x variables, so as to reduce the number $L + 1$ of binary variables v_{bsl} in the binary expansion (34). An aggressive policy (e.g., imposing $x_{bs} \leq 1$) is however rather risky as the optimal solution could be cut off; hence, the artificial bounds must be relaxed if an improved solution cannot be found.

Working with a Subset of Stores

Our basic constructive heuristic is based on the observation that removing stores can produce a substantially easier model. Note that a solution $H' = (x', y')$ with a subset of stores can easily be converted into a solution $H = (x, y)$ of the whole problem by just invoking function REOPT (S, y').

In our implementation, we first define a store permutation $s_1, \ldots, s_{|S|}$ according to a certain criterion (to be discussed later). We then address, in sequence, the subproblem with store set $S_t = \{s_1, \ldots, s_t\}$ for $t = 0, \ldots, |S|$.

For $t = 0$, store set S_0 is empty and our MIP model just produces a y solution with random (possibly repeated) configurations.

For each subsequent t, we start with the best solution $H_{t-1} = (xt^{-1}, yt^{-1})$ of the previous iteration and convert it into a solution $H_t = (x^t, y^t)$ of the current subproblem through the refining function `REOPT` (S_t, y^{t-1}). Then we apply the refinement heuristics described in the previous subsection to $H\,t$, reoptimizing one configuration at a time in a circular vein. To reduce computing time, this latter step can be skipped with a certain frequency—except of course in the very last step when $S_t = S$.

Each time a solution $H_t = (x^t, y^t)$ is found, we quickly compute a solution $H = (x, y)$ of the overall problem through function `REOPT` (S, y^t) and update the overall incumbent where all stores are active.

As to store sequence $s_1, \ldots, s_{|S|}$, we have implemented three different strategies to define it. For each store pair a, b, let the dissimilarity index dist(a, b) be defined as the distance between the two demand vectors $(r_{ia} : i \in I)$ and $(r_{ib} : i \in I)$.

- `random`: The sequence is just a random permutation of the integers $1, \ldots, |S|$;
- `most_dissimilar`: We first compute the two most dissimilar stores (a, b), that is, such that $a < b$ and dist(a, b) is a maximum and initialize $s_1 = a$. Then, for $t = 2, \ldots, |S|$, we define $S' = \{s_1, \ldots, s_{t-1}\}$ and let

$$s_t = \operatorname{argmax}_{a \in S \setminus S'} \left\{ \min \left\{ \operatorname{dist}(a, b) : b \in S' \right\} \right\}$$

- `most_similar`: This is just the same procedure as in the previous item, with max and min operators reverted.

The rational of the `most_dissimilar` policy is to attach first a "core problem" defined by the pairwise most dissimilar stores (those at the beginning of the sequence). The assumption here is that our method performs better in its first iterations (small values of t) as the size of the subproblem is smaller, and we have plenty of configurations to accommodate the initial requests. The "similar stores" are therefore addressed only at a later time, in the hope that the found configurations will work well for them.

A risk with the above policy is that the core problem becomes soon too difficult for our simple refining heuristic, so the current solution is not updated after the very first iterations. In this respect, policy `most_similar` is more conservative: given for granted that we proceed by subsequently refining a previous solution with one less store, it seems reasonable not to inject too much innovation in a single iteration—as `most_dissimilar` does when it adds a new store with very different demands with respect to the previous ones.

Computational Experiments

The heuristics described in the previous section have been implemented in C language. IBM ILOG CPLEX 12.6 [21] was used as MIP solver. Reported computing

times are in CPU seconds of an Intel Xeon E3-1220 V2 quad-core PC with 16 GB of RAM. For each run, a time limit of 900 s (15 m) was imposed.

Four heuristic methods have been compared: the three construction heuristics `random`, `most_dissimilar`, and `most_similar` of section "Working with a Subset of Stores," plus

- `fast_heu`: The fast refinement heuristic of section "Fixing y Variables for All but One Configuration" applied starting from a null solution $x = 0$.

All heuristics are used in a multi-start mode; for example, after completion, they are just restarted from scratch until the time limit is exceeded. At each restart, the internal random seed is not reset; hence, all methods natively using a random permutation (namely, `fast_heu` and `random`) will follow a different search path at each run as the permutations will be different. As to `most_dissimilar` and `most_similar`, after each restart the sequence $s_1, \ldots, s_{|S|}$ is slightly perturbed by five random pair swaps. In addition, after each restart the CPLEX's random seed is changed so as to inject diversification among runs even within the MIP solver.

Due to their heuristic nature, our methods—though deterministic—exhibit a large dependency on the initial conditions, including the random seeds used both within our code and in CPLEX. We therefore repeated several times each experiment, starting with different (internal/CPLEX) random seeds at each run, and also report average figures.

In case all capacities are even (as it is the case in our test bed), we compute the following trivial lower bound based on constraint (42):

$$LB := \min\{\alpha, \beta\} \cdot \left| \left\{ s \in S : \sum_{i \in I} r_{is} \ \text{ is odd} \right\} \right| \tag{42}$$

and abort the execution as soon as we find a solution whose value meets the lower bound.

Test Bed

Our test bed coincides with the benchmark proposed in [20] and contains a number of subinstances of a real-world problem (named `AllColor58`) with 58 stores that require 24 ($= 6 \times 4$) different items: T-shirts available in six different sizes and four different colors (black, blue, red, and green). The available box capacities are 4, 6, 8, and 10. Finally, each item has a given overstock limit (0 or 1) for all stores but no understock limits, and the overstock and understock penalties are $\beta = 1$ and $\alpha = 10$, respectively.

Note that our testing environment is identical to that used in [20] (assuming the PC used are substantially equivalent), so our results can fairly be benchmarked against those therein reported.

Comparison Metric

To better compare the performance of our different heuristics, we also make use of an indicator recently proposed by [1, 2] and aimed at measuring the trade-off between the computational effort required to produce a solution and the quality of the solution itself. In particular, let $\tilde{z}_{\text{opt}}$ denote the optimal solution value and let $z(t)$ be the value of the best heuristic solution found at a time t. Then, a *primal gap function* p can be computed as

$$p(t) = \begin{cases} 1 & \text{if no incumbent found until time } t \\ \gamma\left(z(t)\right) & \text{otherwise} \end{cases} \tag{43}$$

where $\gamma(\cdot) \in [0, 1]$ is the *primal gap*, defined as follows

$$\gamma(z) = \begin{cases} 0 & \text{if } \ |\,\tilde{z}_{\text{opt}}\,| \;=\; |\,z\,| \;=\; 0, \\ 1 & \text{if } \ \tilde{z}_{\text{opt}} \;\cdot\; z < 0, \\ \frac{z - \tilde{z}_{\text{opt}}}{\max\left\{|\tilde{z}_{\text{opt}}|, |z|\right\}} & \text{otherwise.} \end{cases} \tag{44}$$

Finally, the *primal integral* of a run until time $t_{\max}$ is defined as

$$P\left(t_{\max}\right) = \frac{\int_0^{t_{\max}} p(t)\,\mathrm{d}t}{t_{\max}} \tag{45}$$

and is actually used to measure the quality of primal heuristics—the smaller $P(t_{\max})$, the better the expected quality of the incumbent solution if we stopped computation at an arbitrary time before $t_{\max}$.

Computational Results

We addressed the instances provided in [20], with the aim of benchmarking our matheuristics against the methods therein proposed. Results for the easiest cases involving only NB = 4 box configurations (namely, instances Black58, Blue58, Red58, and Green58) are not reported as the corresponding MIP model can be solved to proven optimality in less than one second by our solver—thus confirming the figures given in [20].

Tables 2 and 3 report the performance of various heuristics in terms of solution value and time and refer to a single run for each heuristic and for each instance.

Table 2 Performance of CPLEX and LNSheuristics from [20]. Single run for each instance. Times in CPU seconds (time limit of 900 s)

		CPLEX		LNS	
Instance	*NB*	Value	Time (s)	Value	Time (s)
BlackBlue10	7	66	7	21	16
BlackBlue58	7	525	43	174	74
AllColor10	14	202	49	89	293
AllColor58	14	1828	273	548	900

Table 3 Performance of matheuristics. Single run for each instance. Times in CPU seconds (time limit of 900 s)

						most		most	
		fast_heu		random		dissimilar		similar	
Instance	LB	Value	Time (s)	Value	Time (s)	Value	Time (s)	Value	Time (s)
BlackBlue10	10	10	1	10	2	10	1	10	1
BlackBlue58	58	58	4	58	3	58	2	58	4
AllColor10	6	6	29	17	82	17	734	17	379
AllColor58	42	141	71	273	722	614	105	53	66

Table 2 is taken from [20], where a two-phase hybrid metaheuristic was proposed. In the first phase, the approach uses a memetic algorithm to explore the solution space and builds a pool of interesting box configurations. In the second phase, a box-to-store assignment problem is solved to choose a subset of configurations from the pool—and to decide how many boxes of each configuration should be sent to each store. The box-to-store assignment problem is modeled as a (very hard in practice) MIP and heuristically solved either by a commercial solver (CPLEX) or by a sophisticated large-neighborhood search (LNS) approach.

Table 3 reports the performance of our four matheuristics, as well as the lower bound value *LB* computed through (43)—this latter value turned out to coincide with the optimal value for all instances under consideration in the present subsection.

Comparing Tables 2 and 3 shows that matheuristics outperform the LNS heuristics analyzed in [20]. In particular, `fast_heu` is able to find very good solutions (actually, an optimal one in 3 out of 4 cases) within very short computing times. For the largest instance (AllColor58), however, `most_similar` qualifies as the best heuristic both in terms of quality and speed.

To get more reliable information about the matheuristics' performance, we ran them 100 times for each instance, with different random seeds, and took detailed statistics on each run. Table 4 reports, for each instance and for each heuristic, the average completion time (time), the average time to find its best solution (time_best), the primal integral after 900 s (pint, the lower the better), and the number of provably optimal solutions found (#opt) out of the 100 runs. Note that, for all instances, a solution matching the simple lower bound (43) was eventually found by at least one of our heuristics.

The 100-run statistics confirm that `fast_heu` is very effective in all cases, though it is outperformed by `most_similar` for the largest instance AllColor58 with respect to the #opt criterion. The results suggest that a hybrid method running `fast_heu` and `most_similar` (possibly in parallel) qualifies a robust heuristic with a very good performance for all instances.

Table 4 Average performance (out of 100 runs) of our heuristics

Instance	Heuristic	Time (s)	Time_best (s)	Pint	#opt
BlackBlue10	`fast_heu`	1.08	1.08	0.34	100
	`Random`	1.44	1.44	0.27	100
	`most_dissimilar`	1.26	1.26	0.25	100
	`most_similar`	1.25	1.25	0.29	100
BlackBlue58	`fast_heu`	4.61	4.61	9.88	100
	`Random`	6.40	6.40	10.11	100
	`most_dissimilar`	2.76	2.76	9.13	100
	`most_similar`	5.62	5.62	15.42	100
AllColor10	`fast_heu`	71.82	71.82	3.81	100
	`Random`	600.29	304.06	18.63	36
	`most_dissimilar`	704.33	241.12	19.69	27
	`most_similar`	626.15	302.40	20.54	26
AllColor58	`fast_heu`	900.00	332.43	328.20	0
	`Random`	874.87	329.59	562.95	2
	`most_dissimilar`	893.48	323.93	545.47	1
	`most_similar`	859.86	287.50	404.29	1

Application 3: Vehicle Routing

In this section, we address the NP-hard ***distance-constrained capacitated vehicle routing problem*** (DCVRP) that can be defined as follows. We are given a central depot and a set of $n - 1$ customers, which are associated with the nodes of a complete undirected graph $G = (V, E)$ where $|V| = n$, node 1 representing the depot. Each edge $[i,j] \in E$ has an associated finite cost $c_{ij} \geq 0$. Each node $j \in V$ has a request $d_j \geq 0$ ($d_1 = 0$ for depot node 1). Customers need to be served by k cycles (*routes*) passing through the depot, where k is fixed in advance. Each route must have a total duration (computed as the sum of the edge costs in the route) not exceeding a given limit D and can visit a subset S of customers whose total request $\sum_{j \in s} dj$ does not exceed a given capacity C. The problem then consists of finding a feasible solution covering exactly once all the nodes $v \in V\backslash\{1\}$ and having a minimum overall cost; see, for example, [3, 31].

We will next outline the refinement matheuristic for DCVRP proposed in [15].

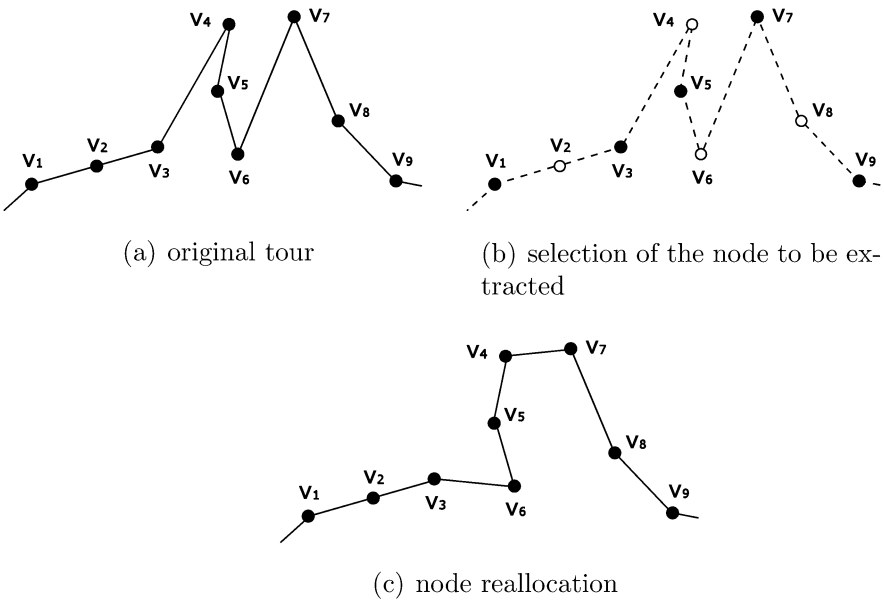
The ASSIGN Neighborhood for TSP

Sarvanov and Doroshko (SD) investigated in [28] the so-called ASSIGN neighborhood for the pure Traveling Salesman Problem (TSP), that is, for the problem of finding a min-cost Hamiltonian cycle in a graph. Given a certain TSP solution (viewed as node sequence $<v_1 = 1, v_2, \cdots, v_n>$), the neighborhood contains all the $\lfloor n/2 \rfloor !$ TSP solutions that can be obtained by permuting, in any possible

way, the nodes in even position in the original sequence. In other words, any solution $(\psi_1, \psi_2, \cdots, \psi_n)$ in the neighborhood is such that $\psi_i = v_i$ for all odd i. An interesting feature of the neighborhood is that it can be explored exactly in polynomial time, though it contains an exponential number of solutions. Indeed, for any given starting solution, the min-cost TSP solution in the corresponding ASSIGN neighborhood can be found efficiently by solving a min-cost assignment problem on a $\lfloor n/2 \rfloor \times \lfloor n/2 \rfloor !$ matrix; see, for example, [18]. Starting from a given solution, an improving heuristic then consists of exploring the ASSIGN neighborhood according to the following two phases:

- *node extraction*, during which the nodes in even position (w.r.t. the current solution) are removed from the tour, thus leaving an equal number of "free holes" in the sequence
- *node reinsertion*, during which the removed nodes are reallocated to the available holes in an optimal way by solving a min-sum assignment problem

The simple example in Fig. 1 gives an illustration of the kind of "improving moves" involved in the method. The figure draws a part of a tour, corresponding to the node sequence $\langle v_1, v_2, \cdots, v_9 \rangle$. In the node extraction phase, the nodes in even position v_2, v_4, v_6 e v_8 are removed from the sequence, whereas all the other nodes retain their position. In Fig. 1b the black nodes represent the fixed ones, while the holes left by the extracted nodes are represented as white circles. If we use symbol "–" to represent a free hole, the sequence corresponding to Fig. 1b is therefore



(a) original tour

(b) selection of the node to be extracted

(c) node reallocation

Fig. 1 A simple example of node extraction and reallocation

$\langle v_1, -, v_3, -, v_5, -, v_7, -, v_9 \rangle$. The second step of the procedure, that is, the optimal node reallocation, is illustrated in Fig. 1c, where nodes v_4 and v_6 swap their position, whereas v_2 and v_8 are reallocated as in the original sequence. This produces the improved part of tour $\langle v_1, v_2, v_3, v_6, v_5, v_4, v_7, v_8, v_9 \rangle$.

In the example, the same final tour could have been constructed by a simple 2-opt move. However, for more realistic cases, the number of possible reallocation is exponential in the number of extracted nodes; hence, the possible reallocation patterns are much more complex and allow, for example, for a controlled worsening of some parts of the solution which are compensated by large improvement in other parts.

From TSP to DCVRP

One can conjecture that the ASSIGN neighborhood would work well if applied to VRP problems. Indeed, due to the presence of several routes and of the associated route constraints, in VRP problems the node sequence is not the only issue to be considered when constructing a good solution: an equally important aspect of the optimization is to find a balanced distribution of the nodes between the routes. In this respect, heuristic refinement procedures involving complex patterns of node reallocations among the routes likely are quite effective in practice.

We can therefore extend the SD method to DCVRP so as to allow for more powerful move patterns, while generalizing its basic scheme so as to get rid of the too-simple min-sum assignment model for node reallocation in favor of a more flexible reallocation model based on the (heuristic) solution of a more complex MIP model. The resulting matheuristic will be introduced, step by step, with the help of some illustrative examples.

Let us consider Fig. 2, where a nonoptimal part of a VRP solution is depicted. It is clear that the position of node v_3 is not very clever, in that inserting v_3 between node v_1 and v_2 is likely to produce a better solution (assuming this new solution is not infeasible because of the route constraints). Even if v_3 were an even position node, however, this move would be beyond the possibility of the pure SD method, where the extracted nodes can only be assigned to a hole left free by the removal of another node—while no hole between v_1 e v_2 exists which could accommodate v_3. The example then suggests a first extension of the basic SD method, consisting of removing the 1–1 correspondence between extracted nodes and empty holes. We therefore consider the concept of *insertion point*: after having extracted the selected nodes, we construct a *restricted solution* through the remaining nodes, obtained from the original one by short-cutting the removed nodes. All the edges in the restricted solution are then viewed as potential insertion points for the extracted nodes. In the example, removing v_3 but not v_1 and v_2 would produce the restricted solution depicted in Fig. 3a, where all dashed edges are possible insertion points for the extracted nodes—this allows the method to produce the solution in Fig. 3b.

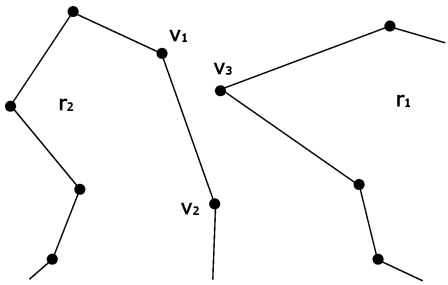
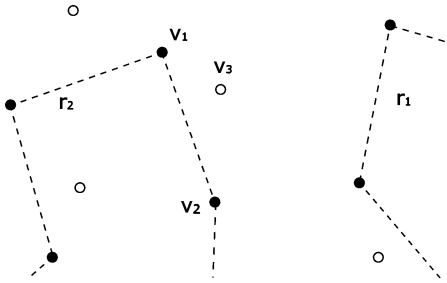
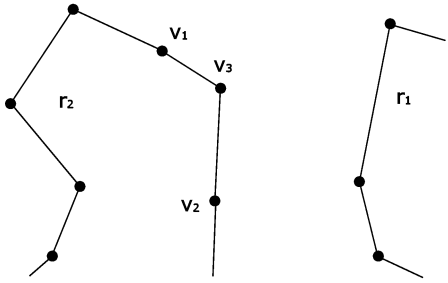


Fig. 2 The assignment of node v_3 to route r_1 is nonoptimal



(a) Restricted solution



(b) Final solution

Fig. 3 Improving the solution depicted in Fig. 2

A second important extension comes from a more flexible node extraction scheme that allows for the removal of a *sequence* of nodes; see Fig. 4 for an illustration. Once a sequence of nodes has been extracted, one can use a heuristic procedure to generate *new* sequences through the extracted nodes, to be allocated to different insertion points. To be more specific, starting from the extracted node sequences, one can create new *derived sequences* that combine the extracted nodes in a different way and consider the possibility of assigning each derived sequence to a different insertion point. Of course, one never knows in advance which are the best sequences to be used, so all the (original and derived) sequences should be available when solving the reallocation problem.

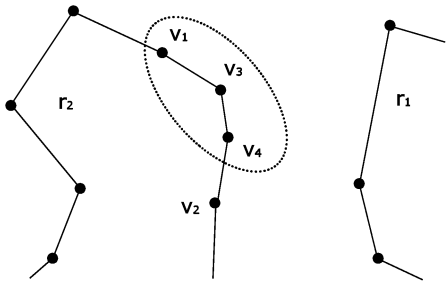


Fig. 4 Removing a sequence of nodes to better reallocate them (possibly in a different order)

The above considerations imply the use of a reallocation model which goes far beyond the scope of the original one, which is based on the solution of an easy min-cost assignment problem. Indeed, the new reallocation model becomes a MIP that receives as input the set of insertion points along with a (typically large) set of node sequences through the extracted nodes and provides an (almost) optimal allocation of at most one sequence to each insertion point, with the constraint that each extracted node has to belong to one of the allocated sequences, while fulfilling the additional constraints on the capacity and distance constraints on the routes. This model will be described in more detail in the next section.

The Overall Matheuristic

Here is a possible implementation of the ideas outlined in the previous section, leading to the so-called *selection, extraction, recombination, and reallocation* (SERR) matheuristic.

(i) (*Initialization*). Apply a fast heuristic method to find a first (possibly infeasible, see below) DCVRP solution.

(ii) (*Selection*). Apply one of the available criteria (to be described later) to determine the nodes to be extracted—the nodes need not be consecutive, and any node subset qualifies as a valid choice.

(iii) (*Extraction*). Extract the nodes selected in the previous step and construct the corresponding restricted DCVRP solution obtained by short-cutting the extracted nodes. All edges in the restricted solution are put in the list $\mathcal{I}$ of the available insertion points.

(iv) (*Recombination*). The node sequences extracted in the previous step (called *basic* in the sequel) are initially stored in a *sequence pool*. Thereafter, heuristic procedures (to be described later) are applied to derive new sequences through the extracted nodes, which are added to the sequence pool. During this phase, dual information derived from the LP relaxation of the reallocation model can be used to find new profitable sequences—the so-called *pricing* step. Each sequence s in the final pool is then associated with a (heuristically determined) subset I s of the available insertion points in $\mathcal{I}$. For all basic sequences s, we assume that $\mathcal{I}_s$ contains (among others) the *pivot* insertion point associated with s in the original tour, so as to make it feasible to retrieve the original solution by just reallocating each basic sequence to the associated pivot insertion point.

(v) (*Reallocation*). A suitable MIP (to be described later in greater details) is set up and solved heuristically through a general-purpose MIP solver. This model has a binary variable x_{si} for each pair (s, i), where s is a sequence in the pool and $i \in \mathcal{I}_s$, whose value 1 means that s has to be allocated to i. The constraints in the MIP stipulate that each extracted node has to be covered by exactly one of the selected sequences s, while each insertion point i can be associated with at most one sequence. Further constraints impose the capacity and distance constraints in each route. Once an (almost) optimal MIP solution has been found, the corresponding DCVRP solution is constructed and the current best solution is possibly updated (in which case each route in the new solution is processed by a 3-opt [26] exchange heuristic n the attempt of further improving it).

(vi) (*Termination*). If at least one improved DCVRP solution has been found in the last n iterations, we repeat from step (ii); otherwise the method terminates.

Finding a Starting Solution

Finding a DCVRP solution that is guaranteed to be feasible is an NP-hard problem; hence, we have to content ourselves with the construction of solutions that, in some hard cases, may be infeasible—typically because the total-distance-traveled constraint is violated for some routes. In this case, the overall infeasibility of the starting solution can hopefully be driven to zero by a modification of the SERR recombination model where the capacity and distance constraints are treated in a soft way through the introduction of highly penalized slack variables.

As customary in VRP problems, we assume that each node is assigned a coordinate pair (x, y) giving the geographical position of the corresponding customer/depot in a two-dimensional map.

One option for the initialization of the current solution required at step (i) of the SERR method is to apply the classical two-phase method of Fisher and Jaikumar (FJ) [14]. This method can be implemented in a very natural way in our context in that it is based on a (heuristic) solution of a MIP whose structure is close to that of the reallocation model.

According to computational experience, however, the solution provided by the FJ heuristic is sometimes "too balanced," in the sense that the routes are filled so tightly that leave not enough freedom to the subsequent steps of our SERR procedure. Better results are sometimes obtained starting from a less-optimized solution whose costs significantly exceed the optimal cost as, for example, the one obtained by using a simplified *SWEEP* method [16].

A second possibility is instead to start from an extremely good solution provided by highly effective (and time-consuming) heuristics or metaheuristics, in the attempt of improving this solution even further.

Node Selection Criteria

At each execution of step (ii), one of the following selection schemes is applied.

- Scheme RANDOM-ALTERNATE: This criterion is akin to the SD one and selects in some randomly selected routes all the nodes in even position, while in

the remaining routes the extracted nodes are those in odd position—the position parity being determined by visiting each route in a random direction.
- Scheme SCATTERED: Each node had a uniform probability of 50% of being extracted; this scheme allows for the removal of consecutive nodes, that is, of route subsequences.
- Scheme NEIGHBORHOOD: Here one concentrates on a seed node, say v^*, and removes the nodes v with a probability that is inversely proportional to the distance c_{vv^*} of v from v^*.

Schemes RANDOM-ALTERNATE and SCATTERED appear particularly suited to improve the first solutions, whereas the NEIGHBORHOOD scheme seems more appropriate to deal with the solutions available after the first iterations.

Reallocation Model

Given the sequences stored in the pool and the associated insertion points (defined through the heuristics outlined in the next subsection), our aim is to reallocate the sequences so as to find a feasible solution of improved cost (if any). To this end, we need to introduce some additional notation.

Let $\mathcal{F}$ denote the set of the extracted nodes, $\mathcal{S}$ the sequence pool, and $\mathcal{R}$ the set of routes r in the restricted solution. For any sequence $s \in \mathcal{S}$, let $c(s)$ be the sum of the costs of the edges in the sequence, and let $d(s)$ be the sum of the requests d_j associated with the *internal* nodes of s.

For each insertion point $i \in \mathcal{I}$, we define the extra-cost γ_{si} for assigning sequence s (in its best possible orientation) to the insertion point i. For each route $r \in \mathcal{R}$ in the restricted solution, let $\mathcal{I}(r)$ denote the set of the insertion points (i.e., edges) associated with r, while let $\tilde{d}(r)$ and $\tilde{c}(r)$ denote, respectively, the total request and distance computed for route r—still in the restricted tour. As already mentioned, our MIP model is based on the following decision variables:

$$x_{si} = \begin{cases} 1 \text{ if sequence s is allocated to the insertion point } i \in \mathcal{I}_s \\ 0 \text{ otherwise} \end{cases} \tag{46}$$

The model then reads:

$$\sum_{r \in \mathcal{R}} \tilde{c}(r) + \min \sum_{s \in \mathcal{S}} \sum_{i \in \mathcal{I}_s} \gamma_{si} x_{si} \tag{47}$$

subject to:

$$\sum_{s \in v} \sum_{i \in \mathcal{I}_s} x_{si} = 1 \ \forall v \in \mathcal{F} \tag{48}$$

$$\sum_{s \in \mathcal{S}: i \in \mathcal{I}_s} x_{si} \leq 1 \forall i \in \mathcal{I} \tag{49}$$

$$\tilde{d}(r) + \sum_{s \in \mathcal{S}} \sum_{i \in \mathcal{I}_s \cap \mathcal{I}(r)} d(s) x_{si} \leq C \quad \forall r \in \mathcal{R} \tag{50}$$

$$\tilde{c}(r) + \sum_{s \in \mathcal{S}} \sum_{i \in \mathcal{I}_s \cap \mathcal{I}(r)} \gamma_{si} x_{si} \leq D \quad \forall s \in \mathcal{S}, r \in \mathcal{R} \tag{51}$$

$$0 \leq x_{si} \leq 1 \text{integer} \quad \forall s \in \mathcal{S}, \quad i \in \mathcal{I}_s \tag{52}$$

The objective function, to be minimized, gives the cost of the final DCVRP solution. Indeed, each objective coefficient gives the MIP cost of an inserted sequence, including the linking cost, minus the cost of the "saved" edge in the restricted solution. Constraints (48) impose that each extracted node belongs to exactly one of the selected sequences, that is, that it is covered exactly once in the final solution. Note that, in the case of triangular costs, one could replace $=$ by $\geq$ in (48), thus obtaining a typically easier-to-solve MIP having the structure of a set-covering (instead of set-partitioning) problem with side constraints. Constraints (49) avoid a same insertion point be used to allocate two or more sequences. Finally, constraints (50) and (51) impose that each route in the final solution fulfills the capacity and distance restriction, respectively.

In order to avoid to overload the model by an excessive number of variables, a particular attention has to be paid to reduce the number of sequences and, for each sequence, the number of the associated insertion points.

Node Recombination and Construction of Derived Sequences

This is a very crucial step in the SERR method. It consists not just of generating new "good" sequences through the extracted nodes, but it also associates each sequence to a clever set of possible insertion points that can conveniently accommodate it. Therefore, one has two complementary approaches to attack this problem: (a) start from the insertion points and, for each insertion point, try to construct a reasonable number of new sequences which are likely to "fit well" or (b) start from the extracted nodes and try to construct new sequences of small cost, no matter the position of the insertion points. The following two-phase method turned out to be a good strategy in practice.

In the first phase, the sequence pool is initialized by means of the original (basic) sequences, and each of them is associated with its corresponding (pivot) insertion point. This choice guarantees that the current DCVRP solution can be reconstructed by simply selecting all the basic sequences and putting them back in their pivot insertion point. Moreover, when the NEIGHBORHOOD selection scheme is used, a further set of sequences is generated as follows. Let v^* be the extracted seed node, and let $N(v^*)$ contain v^* plus the, say, k closest extracted nodes ($k = 4$ in our implementation). A complete enumerative scheme is applied to generate all the sequences through $N(v^*)$ that are added to the pool. This choice is intended to increase the chances of locally improving the current solution, by exploiting appropriate sequences to reallocate the nodes in $N(v^*)$ in an optimal way.

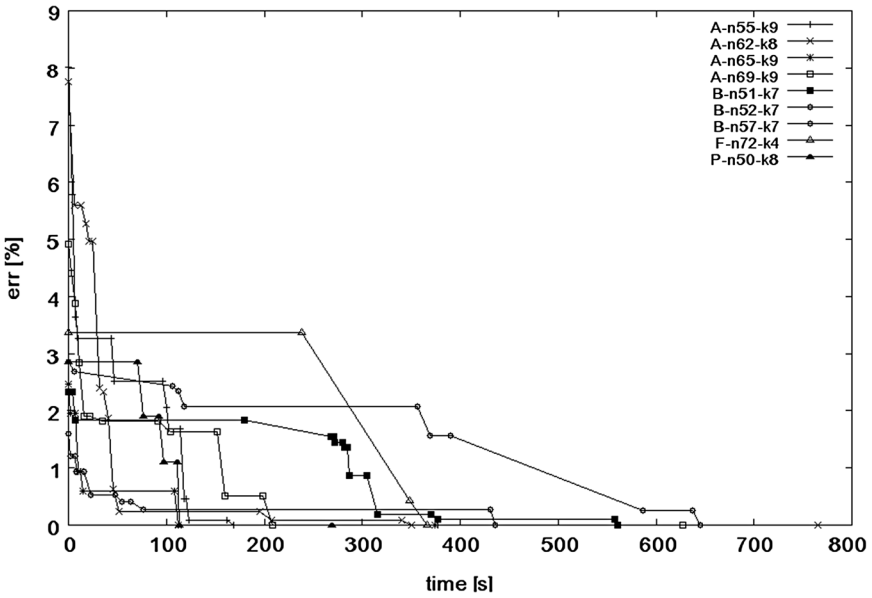


Fig. 5 Time evolution of the SERR solution for various CVRP instances, with FJ [14] initial solution

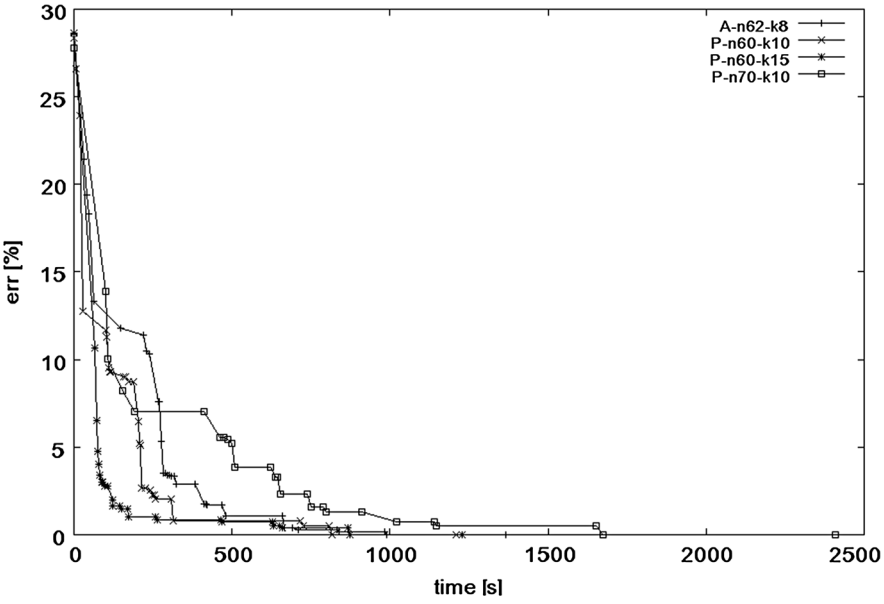


Fig. 6 Time evolution of the SERR solution for various CVRP instances, with SWEEP [16] initial solution

The second phase is only applied for the NEIGHBORHOOD and SCATTERED selection schemes and corresponds to a pricing loop based on the dual information available after having solved the LP relaxation of the current reallocation model. The reader is again referred to [15] for details.

Examples

Extensive computational results for SERR matheuristic have been reported [15] and are omitted because of space. We only report in Figs. 5 and 6 a plot of the incumbent SERR solution for some instances from the literature. Computing time is given in CPU seconds of an old AMD Athlon XP 2400+ PC with 1 GB of RAM, using ILOG Cplex 8.0 as MIP solver. The figures show that SERR is able to significantly improve the starting solution in the early part of its computation.

Conclusion

Contamination of metaheuristics with mathematical programming leads to the concept of "matheuristics." The result is a general approach to design mathematically sound heuristics. In this chapter, we presented the main ideas underlying matheuristics, and used some case studies to illustrate them. For each application, we described the specific problem at hand, the mathematical programming model that formalizes it, and the way the model—or a simplification of it—can be used to produce heuristic (as opposed to exact) solutions in an effective way.

Cross-References

- ▸ Adaptive and Multi-level Metaheuristics
- ▸ Constraint-Based Local Search
- ▸ Iterated Local Search

References

1. Achterberg T, Berthold T, Hendel G (2012) Rounding and propagation heuristics for mixed integer programming. In: Operations research proceedings 2011, Zurich, pp 71–76
2. Berthold T (2013) Measuring the impact of primal heuristics. Oper Res Lett 41(6):611–614
3. Christofides N, Mingozzi A, Toth P (1979) The vehicle routing problem. Combinatorial optimization. Wiley, New York
4. Danna E, Rothberg E, Le Pape C (2005) Exploring relaxation induced neighborhoods to improve MIP solutions. Math Program 102:71–90
5. Donovan S (2005) Wind farm optimization. In: Proceedings of the 40th annual ORSNZ conference, Wellington, pp 196–205
6. Fischetti M (2014) Mixed-integer models and algorithms for wind farm layout optimization. Master's thesis, University of Padova. http://tesi.cab.unipd.it/45458/1/tesi_Fischetti.pdf
7. Fischetti M, Lodi A (2003) Local branching. Math Program 98:23–47
8. Fischetti M, Lodi A (2011) Heuristics in mixed integer programming. In: Cochran JJ (ed) Wiley encyclopedia, Operations research and management science, vol 8. Wiley, Hoboken, pp 738–747

9. Fischetti M, Lodi A, Salvagnin D (2010) Just MIP it! In: Maniezzo V, Stützle T, Voß S (eds) Matheuristics, Annals of information systems, vol 10. Springer, Boston, pp 39–70
10. Fischetti M, Monaci M (2014) Proximity search for 0–1 mixed-integer convex programming. J Heuristics 6(20):709–731
11. Fischetti M, Monaci M (2016) Proximity search heuristics for wind farm optimal layout. J Heuristics 22(4):459–474
12. Fischetti M, Monaci M, Salvagnin D (2012) Three ideas for the quadratic assignment problem. Oper Res 60(4):954–964
13. Fischetti M, Monaci M, Salvagnin D (2015, to appear) Mixed-integer linear programming heuristics for the prepack optimization problem. Discret Optim, 22, 195
14. Fisher ML, Jaikumar R (1981) A generalized assignment heuristic for vehicle routing. Networks 11:109–124
15. De Franceschi R, Fischetti M, Toth P (2006) A new ILP-based refinement heuristic for vehicle routing problems. Math Program 105(2–3):471–499
16. Gillett BE, Miller LR (1974) A heuristic algorithm for the vehicle dispatch problem. Oper Res 22:340–349
17. Glover F (1975) Improved linear integer programming formulations of nonlinear integer problems. Manag Sci 22:455–460
18. Gutin G, Yeo A, Zverovitch A (2007) Exponential neighborhoods and domination analysis for the TSP. In: Gutin G, Punnen AP (eds) The traveling salesman problem and its variations, Combinatorial optimization, vol 12. Springer, Boston, pp 223–256
19. Hansen P, Maniezzo V, Voß S (2009) Special issue on mathematical contributions to metaheuristics editorial. J Heuristics 15(3):197–199
20. Hoskins M, Masson R, Melanon GG, Mendoza JE, Meyer C, Rousseau L-M (2014) The prepack optimization problem. In: Simonis H (ed) Integration of AI and OR techniques in constraint programming, Lecture notes in computer science, vol 8451. Springer, Berlin/Heidelberg, pp 136–143
21. IBM ILOG (2014) CPLEX user's manual
22. Jensen NO (1983) A note on wind generator interaction. Technical report, Riso-M-2411(EN), Riso National Laboratory, Roskilde
23. Lodi A, Martello S, Monaci M (2002) Two-dimensional packing problems: a survey. Eur J Oper Res 141:241–252
24. Maniezzo V, Stützle T, Voß S (eds) (2010) Matheuristics – hybridizing metaheuristics and mathematical programming, vol 10. Annals of information systems. Springer, Boston
25. Martello S, Toth P (1990) Knapsack problems: algorithms and computer implementations. Wiley, Chichester
26. Rego C, Glover F (2007) Local search and metaheuristics. In: Gutin G, Punnen A (eds) The traveling salesman problem and its variations, Combinatorial optimization, vol 12. Springer, Boston, pp 309–368
27. Rothberg E (2007) An evolutionary algorithm for polishing mixed integer programming solutions. INFORMS J Comput 19:534–541
28. Sarvanov VI, Doroshko NN (1981) The approximate solution of the travelling salesman problem by a local algorithm with scanning neighborhoods of factorial cardinality in cubic time (in Russian). In: Software: algorithms and programs. Mathematical Institute of the Belorussian Academy of Sciences, Minsk, pp 11–13
29. Shaw P (1998) Using constraint programming and local search methods to solve vehicle routing problems. In: Maher M, Puget J-F (eds) Principles and practice of constraint programming CP98, Lecture notes in computer science, vol 1520. Springer, Berlin/Heidelberg, pp 417–431
30. Siemens AG. SWT-2.3-93 turbine, Technical specifications http://www.energy.siemens.com
31. Toth P, Vigo D (2002) An overview of vehicle routing problems. In: The vehicle routing problem. SIAM monographs on discrete mathematics and applications. SIAM, Philadelphia
32. Xia Y, Yuan YX (2006) A new linearization method for quadratic assignment problem. Optim Methods Softw 21:803–816

Multi-start Methods

9

Rafael Martí, Jose A. Lozano, Alexander Mendiburu, and Leticia Hernando

Contents

Introduction . . . 212
Global Optimization . . . 214
Combinatorial Optimization . . . 215
Estimating the Number of Local Optima . . . 218
  Methods Proposed in the Metaheuristics Arena . . . 220
  Methods Proposed in the Field of Statistics . . . 220
Experiments . . . 221
  Experimental Design . . . 222
  Results . . . 223
Conclusions . . . 228
Cross-References . . . 228
References . . . 229

Abstract

Multi-start procedures were originally conceived as a way to exploit a local or neighborhood search procedure, by simply applying it from multiple random

R. Martí (✉)
Operations Research and Statistics Department, School of Mathematics, University of Valencia, Valencia, Spain
e-mail: rafael.marti@uv.es

J. A. Lozano · L. Hernando
Intelligent Systems Group, Department of Computer Science and Artificial Intelligence, University of the Basque Country, Donostia, Spain
e-mail: ja.lozano@ehu.es; leticia.hernando@ehu.es

A. Mendiburu
Intelligent Systems Group, Department of Computer Architecture and Technology, University of the Basque Country, Donostia, Spain
e-mail: alexander.mendiburu@ehu.es

© Springer Nature Switzerland AG 2025

R. Martí et al. (eds.), *Handbook of Heuristics*,
https://doi.org/10.1007/978-3-032-00385-0_1

initial solutions. Modern multi-start methods usually incorporate a powerful form of diversification in the generation of solutions to help overcome local optimality. Different metaheuristics, such as GRASP or tabu search, have been applied to this end. This survey briefly sketches historical developments that have motivated the field and then focuses on modern contributions that define the current state of the art. Two classical categories of multi-start methods are considered according to their domain of application: global optimization and combinatorial optimization. Additionally, several methods are reviewed to estimate the number of local optima in combinatorial problems. The estimation of this number can help to establish the complexity of a given instance, and also to choose the most convenient neighborhood, which is especially interesting in the context of multi-start methods. Experiments on three well-known combinatorial optimization problems are included to illustrate the local optima estimation techniques.

Keywords

Metaheuristics · Multi-start methods · Local optima estimation

Introduction

Heuristic search procedures that aspire to find globally optimal solutions to hard optimization problems usually require some type of diversification to overcome local optimality. One way to achieve diversification is to restart the procedure from a new solution once a region has been explored. This chapter describes the best known multi-start methods for solving optimization problems.

The first multi-start efforts described in the literature rely on methods used in statistics and calculus as instances of utilizing repeated constructions to produce a preferred candidate, although such methods were not used to solve optimization problems. In our context, early proposals can be found in the domains of heuristic scheduling [9, 34] and the traveling salesman problem [21]. Multi-start global optimization algorithms, on the other hand, were introduced in the 1980s for bound constraint optimization problems. The well-known Monte Carlo random restart approaches simply evaluate the objective function at randomly generated points [38]. The probability of success approaches one as the sample size tends to infinity under very mild assumptions about the objective function. Many algorithms have been proposed that combine the Monte Carlo method with local search procedures [26].

The restart mechanism inherent of a multi-start design has been superimposed on many different search methods. Once a new solution has been generated, a variety of options can be used to improve it, ranging from a simple greedy routine to a complex metaheuristic such as tabu search [18] or GRASP [37]. Note that the former is based on identifying and recording specific types of information (attributes) to exploit in

future constructions, while the latter is based on order statistics of sampling and on generating unconnected solutions. Martí et al. [30] presented a detailed description of these two methodologies within the multi-start framework, in the context of combinatorial optimization.

A basic multi-start procedure simply applies a procedure that can be called `ConstructSolution` multiple times, returning the best solution found over all starts. The constructed solution is typically improved with the `LocalSearch` procedure. These two procedures, also called phases, are alternated until a stopping criterion is satisfied. Then, each global iteration produces a solution (usually a local optima) and the best overall is the algorithm's output. This is illustrated in Algorithm 1 for a minimization problem.

Algorithm 1: Pseudo-code for multi-start algorithm

```
procedure MultiStart
f* ← ∞;
while stopping criterion not satisfied do
    Construct feasible solution:
        S ← ConstructSolution;
        S ← LocalSearch(S);
    if f(S) < f* then
        S* ← S;
        f* ← f(S);
return S*;
```

As it is well known in the heuristic community, the performance of the local search-based algorithms strongly depends on the properties that the neighborhood imposes on the search space. One of the most important properties is the number of local optima. Given an instance and a neighborhood, the estimation of the number of local optima can help to both measure the instance complexity and to choose the most efficient neighborhood. This chapter reviews and tests several methods to estimate the number of local optima in combinatorial optimization problems.

This chapter is focused on studying the different strategies and methods for generating solutions to launch a succession of local searches for global optimum in the context of two domains: global optimization and combinatorial optimization. It is organized as follows. Section "Global Optimization" describes the developments and strategies applied in the context of global (nonlinear) optimization. Section "Combinatorial Optimization" introduces notation for combinatorial optimization and provides descriptions for solution construction procedures and multi-start algorithms. The difficulty of finding the global optima of a combinatorial problem is evaluated through the estimation of its number of local optima in section

"Estimating the Number of Local Optima." Specifically, methods proposed in the literature are first reviewed, and then, the associated experiments are described in section "Experiments", where statistical tests are applied to draw significant conclusions. Concluding remarks are given in section "Conclusions."

Global Optimization

As mentioned above, many algorithms have been proposed in the 1980s that combine the Monte Carlo method with local search procedures [25], being the multi-level single linkage the most relevant. In general terms, the probability of success approaches one as the sample size tends to infinity under very mild assumptions about the objective function. The convergence for random restart methods is studied in [38], where the probability distribution used to choose the next starting point can depend on how the search evolves. Some extensions of these methods seek to reduce the number of complete local searches that are performed and increase the probability that they start from points close to the global optimum [31].

From a theoretical point of view, Hu et al. [24] study the combination of the gradient algorithm with random initializations to find a global optimum. Efficacy of parallel processing, choice of the restart probability distribution, and number of restarts are studied for both discrete and continuous models. The authors show that the uniform probability is a good choice for restarting procedures.

Hickernell and Yuan Hickernell and Yuan [23] present a multi-start algorithm for unconstrained global optimization based on *quasirandom samples*. Quasirandom samples are sets of deterministic points, as opposed to random points, that are evenly distributed over a set. The algorithm applies an inexpensive local search (steepest descent) on a set of quasirandom points to concentrate the sample. The sample is reduced replacing worse points with new quasirandom points. Any point that is retained for a certain number of iterations is used to start an efficient complete local search. The algorithm terminates when no new local minimum is found after several iterations. An experimental comparison shows that the method performs favorably with respect to other global optimization procedures.

Tu and Mayne [40] describe a multi-start with a clustering strategy for constrained optimization problems. It is based on the characteristics of nonlinear constrained global optimization problems and extends a strategy previously tested on unconstrained problems. In this study, variations of multi-start with clustering are considered including a simulated annealing procedure for sampling the design domain and a quadratic programming (QP) sub-problem for cluster formation. The strategies are evaluated by solving 18 nonlinear mathematical problems and six engineering design problems. Numerical results show that the solution of a one-step QP sub-problem helps predict possible regions of attraction of local minima and can enhance robustness and effectiveness in identifying local minima without sacrificing efficiency. In comparison with other multi-start techniques, the strategies of this study are superior in terms of the number of local searches performed, the number of minima found, and the number of function evaluations required.

Ugray et al. [41] propose OptQuest/Multi-start or OQMS, a heuristic designed to find global optima for pure and mixed integer nonlinear problems with many constraints and variables, where all problem functions are differentiable with respect to the continuous variables. It uses OptQuest, a commercial implementation of scatter search developed by OptTek Systems, Inc., to provide starting points for any gradient-based local NLP solver. This solver seeks a local solution from a subset of these points, holding discrete variables fixed. Computational results include 155 smooth NLP and MINLP problems, most with both linear and nonlinear constraints, coded in the GAMS modeling language. Some are quite large for global optimization, with over 100 variables and many constraints. Global solutions to almost all problems are found in a small number of local solver calls, often one or two. An improved version of OQMS is proposed in Ugray et al. [42] in terms of the filters to apply the NLP solver.

More recently, Kaucic [27] presents a multi-start particle swarm optimization algorithm for the global optimization of a function subject to bound constraints. The procedure consists of three main steps. In the initialization phase, an opposition learning strategy is performed to improve the search efficiency. Then, a variant of the adaptive velocity based on the differential operator enhances the optimization ability of the particles. Finally, a re-initialization strategy based on two diversity measures for the swarm is act in order to avoid premature convergence and stagnation. The algorithm is evaluated on a set of 100 global optimization test problems. Comparisons with other global optimization methods show its robustness and effectiveness.

Combinatorial Optimization

Boese et al. [4] analyze relationships among local minima from the perspective of the best local minimum, finding convex structures in the cost surfaces. Based on the results of that study, they propose a multi-start method where starting points for greedy descent are adaptively derived from the best previously found local minima. In the first step, adaptive multi-start (AMS) heuristics generate r random starting solutions and run a greedy descent method from each one to determine a set of corresponding random local minima. In the second step, *adaptive starting solutions* are constructed based on the local minima obtained so far and improved with a greedy descent method. This improvement is applied several times from each adaptive starting solution to yield corresponding *adaptive local minima*. The authors test this method for the traveling salesman problem and obtain significant speedups over previous multi-start implementations. Hagen and Kahng [20] apply this method for the iterative partitioning problem.

Moreno et al. [33] propose a stopping rule for the multi-start method based on a statistical study of the number of iterations needed to find the global optimum. The authors introduce two random variables that together provide a way of estimating the number of global iterations needed to find the global optima: the number of initial solutions generated and the number of objective function evaluations performed to

find the global optima. From these measures, the probability that the incumbent solution is the global optimum is evaluated via a normal approximation. Thus, at each global iteration, this value is computed, and if it is greater than a fixed threshold, the algorithm stops; otherwise, a new solution is generated. The authors illustrate the method using the median *p*-hub problem.

One of the most well-known multi-start methods is the greedy adaptive search procedure (GRASP), which was introduced by Feo and Resende [15]. It was first used to solve set covering problems [14]. Each GRASP iteration consists of constructing a trial solution and then applying a local search procedure to find a local optimum (i.e., the final solution for that iteration). The construction step is an adaptive and iterative process guided by a greedy evaluation function. It is iterative because the initial solution is built considering one element at a time. It is greedy because the addition of each element is guided by a greedy function. It is adaptive because the element chosen at any iteration in a construction is a function of previously chosen elements (i.e., the method is adaptive in the sense of updating relevant information from one construction step to the next). At each stage, the next element to be added to the solution is randomly selected from a candidate list of high-quality elements according to the evaluation function. Once a solution has been obtained, it is typically improved by a local search procedure. The improvement phase performs a sequence of moves toward a local optimum solution, which becomes the output of a complete GRASP iteration.

Hagen and Kahng [20] implement an adaptive multi-start method for a VLSI partitioning optimization problem where the objective is to minimize the number of signals sent between components. The method consists of two phases. It first generates a set of random starting points and performs the iterative (local search), thus determining a set of local minimum solutions. Then it constructs adaptive starting points derived from the best local minimum solutions found so far. The authors add a preprocessing cluster module to reduce the size of the problem. The resulting clustering adaptive multi-start (CAMS) method is fast and stable and improves upon previous partitioning results reported in the literature.

Fleurent and Glover [16] propose some adaptive memory search principles to enhance multi-start approaches. The authors introduce a template of a constructive version of tabu search based on both, a set of elite solutions and the intensification strategies based on identifying *strongly determined and consistent variables* according to the following definitions:

- *Strongly determined variables* are those whose values cannot be changed without significantly eroding the objective function value or disrupting the values of other variables.
- A *consistent variable* is defined as one that receives a particular value in a significant portion of good solutions.

The authors propose the inclusion of memory structures within the multi-start framework as it is done with tabu search. Computational experiments for the quadratic assignment problem show that these methods improve significantly over

previous multi-start methods like GRASP and random restart that do not incorporate memory-based strategies.

Patterson et al. [35] introduce a multi-start framework called *adaptive reasoning technique* (ART), based on memory structures. The authors implement the short-term and long-term memory functions, proposed in the tabu search framework, to solve the capacitated minimum spanning tree problem. ART is an iterative, constructive solution procedure that implements learning methodologies on top of memory structures. ART derives its success from being able to learn about and modify the behavior of a primary greedy heuristic. The greedy heuristic is executed repeatedly, and for each new execution, constraints that prohibit certain solution elements from being considered by the greedy heuristic are introduced in a probabilistic fashion. The active constraints are held in a short-term memory, while a long-term memory holds information regarding the constraints that were in the active memory for the best set of solutions.

Glover [17] approaches the multi-start framework from a different perspective. The author views multi-start methods as an extreme version of the *strategic oscillation* approach. Strategic oscillation is a mechanism used in tabu search to allow the process to visit solutions around a "critical boundary," by approaching such a boundary from both sides.

Braysy et al. [5] propose a multi-start local search heuristic for the vehicle routing problem with time windows. The objective in this problem is to design least cost routes for a fleet of identical capacitated vehicles to service geographically scattered customers within pre-specified service time windows. The suggested method uses a two-phase approach. In the first phase, a fast construction heuristic is used to generate several initial solutions. Then, injection trees, an extension of the well-known ejection chain approach [18], are used to reduce the number of routes. In the second phase, two new improvement heuristics, based on CROSS-exchanges [39], are applied for distance minimization. The best solution identified by the algorithm is post-optimized using a threshold accepting post-processor with both intra-route and inter-route improvement heuristics. The resulting hybrid method is shown to be fast, cost-effective, and highly competitive.

Mezmaz et al. [32] hybridize the multi-start framework with a model in which several evolutionary algorithms run simultaneously and cooperate to compute better solutions (called *island model*). They propose a solution method in the context of multi-objective optimization on a computational grid. The authors point out that although the combination of these two models usually provides very effective parallel algorithms, experiments on large-size problem instances are often stopped before convergence is achieved. The full exploitation of the cooperation model needs a large amount of computational resources and the management of the fault tolerance issue.

Under the template of a typical multi-start metaheuristic, [28] propose Meta-RaPS (metaheuristic for randomized priority search), in which several randomization methods and memory mechanisms are present. With the set covering problem (SCP) as the application problem, it is found that these randomization and memory-based methods work well for Meta-RaPS.

Beausoleil et al. [3] consider a multi-objective combinatorial optimization problem called extended knapsack problem. By applying multi-start search and path-relinking their solving method rapidly guides the search toward the most balanced zone of the Pareto-optimal front (the zone in which all the objectives are equally important).

Essafi et al. [13] propose a multi-start ant-based heuristics for a machine line balancing problem. The proposed procedure is a constructive algorithm that assigns operations sequentially to stations. The algorithm builds a feasible solution step by step according to a greedy function that computes the contribution of each unassigned operation to the partial solution under construction based on operational time and weights. The selection of the operation to be added is performed with a roulette wheel mechanism based on the typical ant probability distribution (pheromones of previous assignments). The proposed heuristic is applied to solve a real industry problem.

Dhouib et al. [10] propose a multi-start adaptive threshold accepting algorithm (MS-TA) to find multiple Pareto-optimal solutions for continuous optimization problems. Threshold accepting methods (TAs) are deterministic and faster variants of the well-known simulated annealing algorithms, in which every new move is accepted if it is not much worse than the old one. A multi-start technique is applied in this paper to the TA algorithm to allow more diversifications.

Villegas et al. [43] propose two hybrid algorithms for the single truck and trailer routing problem. The first one is based on GRASP and variable neighborhood descent (VND), while the second one is an evolutionary local search (ELS). In the first one, large tours are constructed with a randomized nearest neighbor method with a restricted candidate list that ignores capacity constraints and trailer-point selection. VND is applied to improve these initial solutions obtained with GRASP. In the second one, a multi-start evolutionary search is applied starting from an initial solution (giant tour). The best solution found is strongly perturbed to obtain different solutions from which the search is restarted. The perturbation is managed by a mutation operator. The results of the computational experiments on a set of 32 randomly generated instances also unveil the robustness of the proposed metaheuristics, all of them achieving gaps to best known solutions of less than 1% even in the worst case. Among the proposed methods, the multi-start evolutionary local search is more accurate and faster and scales better than the GRASP/VND.

Estimating the Number of Local Optima

As previously seen, multi-start algorithms depend on a neighborhood defined in the search space. Furthermore, the same multi-start algorithm can produce different results in the same instance when it is used with two different neighborhoods. Although there are several characteristics of a neighborhood that influence the behavior of a multi-start algorithm, probably, the most relevant is the number of local optima it generates.

The knowledge about the number of local optima that a neighborhood generates in an instance of a combinatorial optimization problem (COP) can have a high impact on the choice of the multi-start algorithm used to solve the instance. On the first hand, different neighborhoods can generate a dramatically different number of local optima. Of course, as a general rule, the higher the size of the neighborhood, the lower the number of local optima. However, given two neighborhoods of the same size complexity (number of local solutions in the same order of magnitude), the one that generates a lower number of local optima will always be preferred.

Given an instance of a COP and a neighborhood, the exact calculation of the number of local optima is impractical, except for extremely low dimensions ($n \leq 14$). Furthermore, this exact calculation requires, in most of the cases, the exhaustive inspection of every solution in the search space, which makes this approach useful. Therefore, statistical estimation methods are required to have an approximation of the number of local optima of a landscape. Note, however, that the representation of such number is in itself an additional challenge. When the dimension of a problem is high (for instance, $n \geq 200$) and the size of the search space is exponential in n, the number of local optima is usually so high that cannot be accurately represented in the computer. The alternative choice to represent the proportion of solutions that are local optima deals with similar issues as this number is too small to be accurately represented. Therefore, the estimation of the number of local optima of an instance of a COP is only plausible for moderate values of the problem size.

In spite of the useful information that the knowledge of the number of local optima can provide in order to choose the best algorithm for solving a COP instance, there have not been many proposals in the literature. This can probably be due to the difficulty of the estimation problem. These proposals are mainly divided into three groups (a good review and a comparison of several proposals can be found in [22]). A first group is based on the hypothesis that the instances have been generated uniformly at random [1, 2, 19]. The second group mainly includes proposals that have been developed in the metaheuristics community. Finally, a third group includes those that were not designed with the objective of estimating the number of local optima, but were adapted to this problem.

In order to estimate the number of local optima, all the methods share some steps of the process. All of them obtain uniformly at random a sample of size M from the search space. Departing from these solutions and applying a greedy local search algorithm, they finally obtain a sample of local optima (notice that in this sample some local optima could be repeated several times). In addition to that, the concept of basin of attraction is presented in all the estimation methods. Basically, a solution belongs to the basin of attraction of a local optimum if, after applying a greedy local search to that solution, it finishes in the corresponding local optimum. Notice that, in case of injective functions, the basins of attraction represent a partition of the search space.

Methods Proposed in the Metaheuristics Arena

Most of the methods proposed in the metaheuristics field are mainly due to [11, 12, 36]. They basically developed two kinds of methods: a first group where they use confidence intervals to provide lower bounds (with high probability) for the number of local optima and a second group using bias-correcting methods.

In the first group, by assuming that the sizes of the basins of attraction of all the local optima are the same, they manage to get lower bounds by means of several methods:

- *First repetition time*: A random solution is taken from the search space, and a greedy local search algorithm is applied to reach a local optimum. This process continues until a local optimum is repeated for the first time. This number of initial solutions needed until a local optimum is seen twice is the value used to make the estimation. Note that in this method the sample size is not fixed.
- *Maximal first repetition time*: This method is similar to the previous one, except that the sample size is fixed beforehand and the maximal first repetition time (the maximum number of solutions between the first reoccurrence of a local optimum) is used to carry out the estimation.
- *Schnabel census*: In order to make the estimation, the authors take into account in this case the probability that r different local optima are discovered from a sample of size M.

The main drawback of the previous methods is that they are strongly biased by the assumption that all the local optima have the same size of basin of attraction. In order to overcome this bias, a couple of very well-known methods in the statistical literature are proposed: Jackknife and Bootstrap. These are nonparametric methods that, departing from a biased estimation, try to correct the bias:

- *Jackknife method*: It consists in calculating M new estimations by leaving each time one of the current solutions out from the original sample. These new M estimations are used to modify the original estimation decreasing its bias.
- *Bootstrap method*: This resampling technique consists in obtaining several new samples from the original sample. Each sample has the same size as the original one and is obtained by uniformly at random sampling solutions with replacement from the original sample. On the contrary to the previous method, it assumes a probabilistic model in the basins of attraction to make a new estimation for each resampling.

Methods Proposed in the Field of Statistics

The community working in metaheuristics quickly realized that the methods used by biologists and ecologists to calculate the number of species in a population could be

easily adapted to the problem of calculating the number of local optima. Particularly four nonparametric methods [6–8] were adapted to estimate the number of local optima of instances of COPs [22]. Although they are nonparametric methods, they are based on particular sampling models. An important consideration is that all these methods assume an infinite population, in our case an infinite number of local solutions. This assumption does not impose a big constraint because of the large cardinality of the search space of the COPs:

- *Chao1984 method*: It is based on multinomial sampling. The estimator is the result of adding to the number of local optima obtained from the sample a term that depends only on the number of local optima seen once and twice in the sample.
- *Chao&Lee methods*: These two methods are also based on multinomial sampling. They include the idea of sample coverage (sum of the proportions of the basins of attraction of the local optima observed in the sample). They distinguish between the local optima observed many times in the sample and those observed a few number of times in the sample. So the estimators are the sum of the number of local optima observed many times and some terms that depend on the sample coverage and the number of local optima found a few times, trying to compensate for the local optima that have small basins of attraction.
- *Chao&Bunge method*: It is based on a mixed Poisson sampling model and is closely related to the previous estimators. This method bases the estimation of unobserved local optima on a formula that depends on the expected proportion of duplicates in the sample. It also makes the distinction between the local optima observed many times in the sample and those observed a few number of times in the sample.

Experiments

The accuracy of the different estimators presented in the previous section has been tested on instances of three different common problems: permutation flow-shop scheduling problem (PFSP), linear ordering problem (LOP), and quadratic assignment problem (QAP). Using these datasets, the methods can be tested over a wide set of instances with different characteristics. The number of local optima for different neighborhoods is already known for some of these instances, which allows to evaluate the accuracy of the different estimations. A comparison of the different methods is given, together with recommendations of the methods that provide the best estimations. Based on the results found in [22], the three first methods presented in the previous section are removed for the analysis, that is, first repetition time, maximal first repetition time, and Schnabel census.

Experimental Design

The instances used in the experiments are taken from three well-known benchmarks: 5 instances of the PFSP obtained from the Taillard's benchmark, 5 instances of the LOP taken from the LOLIB benchmark [29], and 5 instances of the QAP chosen from the QAPLIB.

The flowshop scheduling problem can be stated as follows: there are b jobs to be scheduled in c machines. A job consists of c operations and the j-th operation of each job must be processed on machine j for a specific processing time without interruption. Jobs are processed in the same order on different machines. The objective of the PFSP is to find the order in which the jobs have to be scheduled on the machines, minimizing the total flow time.

In the LOP, given a matrix $B = [b_{ij}]_{nxn}$ of numerical entries, a simultaneous permutation of the rows and columns of B has to be found, such that the sum of the entries above the main diagonal is maximized (or equivalently, the sum of the entries below the main diagonal is minimized).

The QAP is the problem of allocating a set of facilities to a set of locations, with a cost function associated to the distance and the flow between the facilities. The objective is to assign each facility to a location such that the total cost is minimized, given two $n \times n$ input matrices with real values $\mathbf{H} = [h_{ij}]$ and $\mathbf{D} = [d_{kl}]$, where h_{ij} is the flow between facility i and facility j and d_{kl} is the distance between location k and location l.

For the three problems, instances of permutation size $n = 13$ are considered, so the search space is of size $|\Omega| = 13! \approx 6.23 \cdot 10^9$. In the case of the PFSP, instances have 13 jobs and 5 machines. Notice that a first step consists of evaluating all the solutions of the search space to know in advance the exact number of local optima. Therefore, working with higher permutation sizes becomes impracticable.

The local optima of the instances have been calculated according two commonly used operators: the 2-exchange and the insert neighborhoods. Denoting by $\boldsymbol{\pi} = \pi_1\pi_2 \ldots \pi_n$ a solution (permutation of size n) of the search space, the 2-exchange neighborhood considers that two solutions are neighbors if one is generated by swapping two elements of the other:

$$N_S(\pi_1\pi_2 \ldots \pi_n) = \left\{ \left(\pi'_1\pi'_2 \ldots \pi'_n\right) \mid \pi'_k = \pi_k, \forall k \neq i, j, \pi'_i = \pi_j, \pi'_j = \pi_i, i \neq j \right\}.$$

Two solutions are neighbors under the insert neighborhood (N_I) if one is the result of moving an element of the other one to a different position:

$$N_I(\pi_1\pi_2 \ldots \pi_n) =$$

$$\left\{ \left(\pi'_1\pi'_2 \ldots \pi'_n\right) \mid \pi'_k = \pi_k, \forall k < i \text{and} \forall k > j, \pi'_k = \pi_{k+1}, \forall i \leq k < j, \pi'_j = \pi_i \right\}$$

$$\cup \left\{ \left(\pi'_1\pi'_2 \ldots \pi'_n\right) \mid \pi'_k = \pi_k, \forall k < i \text{and} \forall k > j, \pi'_i = \pi_j, \pi'_k = \pi_{k-1}, \forall i < k \leq j \right\}.$$

The different methods for estimating the number of local optima are applied to the 5 instances of each of the considered problems (PFSP, LOP, and QAP) using both neighborhoods (N_S and N_I). So, a total of 30 different landscapes are considered. These two neighborhoods create different situations for the estimations. As the insert neighborhood explores at each step more solutions than the 2-exchange neighborhood, the number of local optima obtained when using the first neighborhood is probabilistically lower than when assuming the second one. However, as will be shown, due to the intrinsic nature of the QAP, given an instance of this problem, the number of local optima when using the insert neighborhood is much higher than when considering the 2-exchange neighborhood.

The estimation methods *Jackknife*, *Bootstrap*, *Chao1984*, *Chao&Lee1*, *Chao&Lee2*, and *Chao&Bunge* are applied to each of the landscapes 10 times, and the average estimation value of the 10 repetitions is reported. Methods are applied with different sample sizes: $M = 100, 500, 1000, 5000$.

Results

Our aim is to compare the accuracy of the different methods and their relation to the sample size. Results are analyzed according to the type of problem and the neighborhood used, and the effect of the sample size on the methods using different sample sizes is studied. Table 1 reports the average estimations (of the 10 repetitions) obtained with the different methods and the different sample sizes for all the instances of the three problems when using both neighborhoods. It also indicates the real number of local optima of each instance. v denotes the real number of local optima of the instances with the corresponding neighborhood.

Table 1 shows that the method that provides the best results more times is *Chao&Lee2*. In fact, in general, the estimations, given by those methods that come from the field of statistics, provide better results than *Bootstrap* and *Jackknife*, above all, when the number of local optima is considerably high.

A statistical analysis was carried out to compare the estimations obtained with the different methods and three different scenarios. In the first one, the estimations are grouped in three sets according to the type of problem: PFSP, LOP, or QAP. The second scenario considers two different sets that contain the estimations of the instances when using the neighborhoods N_S or N_I. In the last scenario, the estimations are grouped in four sets, according to the parameter $M = 100, 500, 1000, 5000$. A nonparametric Friedman's test with level of significance $\alpha = 0.05$ is used to test if there are statistical significant differences among the estimations provided by the six methods in the different scenarios. It provides a ranking of the methods while also giving an average rank value for each method. Statistical differences are found in all the cases, and then a post hoc test is applied to perform all pairwise comparisons. Particularly, the Holm's procedure is applied fixing the level of significance to $\alpha = 0.05$.

Table 2 shows the ranking obtained for the methods with the Friedman's test in the first scenario, that is, for the instances of the PFSP (first pair of columns),

Table 1 Mean of the estimations obtained for all the instances of the PFSP, LOP, and QAP, under the 2-exchange and the insert neighborhoods

		Method	M = 100	M = 500	M = 1000	M = 5000	M = 100	M = 500	M = 1000	M = 5000	M = 100	M = 500	M = 1000	M = 5000	M = 100	M = 500	M = 1000	M = 5000	M = 100	M = 500	M = 1000	M = 5000
PFSP	**2-exchange**		Instance 1. $v = 2386$				Instance 2. $v = 8194$				Instance 3. $v = 1997$				Instance 4. $v = 5119$				Instance 5. $v = 192$			
		Jackknife	158.20	517.30	744.80	1371.20	185.50	709.50	1139.10	2733.60	142.70	397.50	590.40	1098.50	167.40	575.20	886.60	1920.50	77.10	131.80	153.00	**183.10**
		Bootstrap	155.90	562.80	862.00	1786.30	173.20	706.50	1199.00	3281.70	143.60	452.90	688.60	1428.00	161.40	609.20	982.40	2376.60	90.70	170.80	**208.80**	240.10
		Chao1984	400.80	692.80	874.70	1405.30	1148.60	1438.50	1751.30	3105.50	331.70	517.30	774.30	1153.30	544.30	843.40	1161.60	2122.60	94.00	136.20	151.60	178.30
		Chao&Lee1	397.30	725.70	932.00	1401.90	1196.50	1544.50	1884.70	3223.90	340.50	546.50	769.10	1146.60	510.90	889.20	1242.60	2194.20	90.10	133.30	147.70	175.00
		Chao&Lee2	**482.40**	**917.90**	1185.30	1565.30	**1302.50**	**2132.40**	**2605.80**	**4020.70**	**519.70**	**737.80**	1030.10	1301.30	587.60	1145.70	1679.00	2689.10	108.80	146.70	155.00	177.60
		Chao&Bunge	236.50	822.80	**2304.40**	**1901.50**	494.40	390.50	660.30	14054.20	204.60	3789.80	**2399.70**	**1695.90**	**793.20**	**2103.80**	**1887.90**	**6446.60**	**258.10**	**173.30**	162.10	178.60
	Insert		Instance 1. $v = 134$				Instance 2. $v = 923$				Instance 3. $v = 506$				Instance 4. $v = 190$				Instance 5. $v = 14$			
		Jackknife	54.30	95.40	104.00	173.40	125.10	317.30	435.90	**955.40**	87.50	208.80	265.10	**520.40**	72.10	127.80	150.60	300.30	12.70	13.30	**14.20**	**14.00**
		Bootstrap	66.60	**124.30**	**141.40**	231.20	129.00	374.60	535.90	1253.50	99.40	252.10	339.90	703.20	85.30	163.00	**199.70**	415.00	16.00	14.50	15.50	14.30
		Chao1984	62.40	100.10	103.20	170.60	256.60	389.90	482.80	997.40	138.40	253.80	277.20	536.60	94.60	138.10	154.10	290.90	11.60	12.90	13.10	**14.00**
		Chao&Lee1	65.80	95.00	100.20	**165.80**	247.70	408.90	488.20	978.10	128.30	263.50	285.70	522.30	83.50	133.90	147.10	**287.10**	12.30	13.40	**14.20**	**14.00**
		Chao&Lee2	**84.00**	103.90	104.60	169.60	**371.80**	545.50	583.90	1092.90	**178.10**	350.40	331.20	570.80	**101.40**	152.20	157.00	296.60	12.50	13.70	14.70	**14.00**
		Chao&Bunge	250.80	121.30	109.20	171.80	112.20	**1202.90**	**1108.50**	1341.40	74.70	**367.70**	**498.50**	652.70	281.00	**212.60**	169.80	303.80	**12.80**	**14.10**	15.10	**14.00**
LOP	**2-exchange**		Instance 1. $v = 9969$				Instance 2. $v = 3355$				Instance 3. $v = 4732$				Instance 4. $v = 3227$				Instance 5. $v = 6810$			
		Jackknife	183.00	732.10	1185.50	3256.30	176.20	642.40	980.70	1946.70	172.80	615.90	921.00	2068.10	136.30	394.10	587.80	1274.40	185.00	746.60	1248.20	2995.20
		Bootstrap	171.30	723.00	1237.20	3754.40	167.50	662.00	1073.30	2458.60	164.80	638.90	1019.00	2539.90	138.50	443.90	685.60	1607.50	172.70	734.40	1291.00	3578.20
		Chao1984	923.50	1633.00	1939.20	4073.00	810.70	1001.10	1225.40	2007.00	842.70	1013.50	1211.00	2321.60	259.90	562.90	765.70	1428.50	1884.60	1523.20	1970.00	3297.30
		Chao&Lee1	935.30	1612.50	2043.00	4580.20	740.40	1003.10	1319.30	2033.40	813.00	1070.80	1299.90	2369.00	257.40	615.10	792.10	1463.30	1757.50	1589.10	2157.70	3485.10
		Chao&Lee2	**987.30**	**2104.70**	**2860.80**	**6487.20**	**904.80**	1185.50	1677.20	2308.20	**1068.70**	**1450.40**	**1744.20**	**2914.70**	**328.00**	**935.00**	1088.80	1785.30	**1966.40**	**1960.20**	2868.70	**4263.30**
		Chao&Bunge	349.80	399.90	680.10	2040.20	144.30	**3188.10**	**2642.10**	**2950.40**	156.90	27501.80	558.90	7544.60	121.70	714.80	**2946.60**	**4654.60**	666.80	14991.50	**7635.90**	10894.20
	Insert		Instance 1. $v = 1113$				Instance 2. $v = 60$				Instance 3. $v = 12$				Instance 4. $v = 9$				Instance 5. $v = 67$			
		Jackknife	124.40	335.20	472.70	1093.50	51.90	**60.50**	61.50	137.10	9.60	12.40	12.60	19.30	7.80	9.40	**9.00**	20.60	49.10	63.80	**60.90**	108.90
		Bootstrap	128.80	391.80	578.00	1431.90	66.90	82.00	77.70	169.70	**11.80**	13.40	14.40	20.20	9.70	13.00	10.20	23.00	**64.00**	86.00	78.70	128.30
		Chao1984	224.00	396.30	528.70	**1121.70**	53.20	57.70	60.40	133.70	8.80	**11.70**	**12.10**	**18.20**	7.20	**9.00**	8.90	**20.20**	51.00	**65.40**	59.90	107.30
		Chao&Lee1	235.60	445.10	556.90	1137.30	52.30	57.60	59.60	**133.40**	9.10	13.20	12.60	20.10	**9.20**	9.20	8.90	20.30	47.80	60.40	58.70	**106.20**
		Chao&Lee2	351.70	595.80	**693.60**	1285.00	**57.30**	57.70	**59.70**	133.60	9.30	14.20	12.60	22.30	10.40	9.40	8.90	20.40	51.30	61.30	59.10	106.60
		Chao&Bunge	**1111.70**	**859.80**	8551.50	1630.70	67.70	57.70	**59.70**	133.60	9.20	14.80	12.40	19.00	8.20	9.40	8.90	20.50	57.00	61.80	59.10	106.80
QAP	**2-exchange**		Instance 1. $v = 18{,}720$				Instance 2. $v = 3472$				Instance 3. $v = 62$				Instance 4. $v = 173{,}568$				Instance 5. $v = 563$			
		Jackknife	197.10	958.80	1811.60	6594.90	171.70	598.50	910.60	1889.80	27.40	42.30	48.80	57.40	198.70	992.00	1965.20	9280.00	94.40	204.50	264.00	368.00
		Bootstrap	180.00	873.60	1678.60	6740.80	164.80	628.60	1009.20	2369.00	33.00	**54.30**	**66.00**	73.10	180.90	891.60	1771.50	8532.00	105.20	254.10	338.80	**494.60**
		Chao1984	2000.70	9847.80	8007.20	9818.00	609.40	870.40	1191.10	2009.20	39.90	48.70	47.50	**58.00**	580.10	48561.00	49112.70	51951.40	139.00	227.10	286.40	373.90
		Chao&Lee1	**2025.00**	9914.80	7973.80	9750.90	616.20	934.40	1247.50	2026.20	32.30	44.10	49.50	54.90	**585.00**	**48783.40**	**49601.10**	51848.80	157.80	224.70	278.60	351.90
		Chao&Lee2	**2025.00**	**10323.30**	**8296.00**	10683.60	**679.90**	1188.50	1652.60	2351.40	39.80	50.90	53.30	56.00	**585.00**	**48783.40**	**49601.10**	53432.60	257.00	266.60	320.90	365.80
		Chao&Bunge	1050.00	3752.20	128363.10	**17782.40**	260.10	**1330.50**	**4564.80**	**3386.60**	**55.20**	130.20	66.70	56.50	340.00	24529.10	25050.50	**174841.50**	**261.30**	**521.50**	**457.40**	378.10
	Insert		Instance 1. $v = 4{,}615{,}326$				Instance 2. $v = 1{,}501{,}175$				Instance 3. $v = 579{,}275$				Instance 4. $v = 6{,}712{,}090$				Instance 5. $v = 83{,}240$			
		Jackknife	198.70	995.70	1991.60	9821.40	198.70	995.30	1988.20	9795.60	**199.00**	987.00	1947.40	9108.50	**199.00**	997.70	1997.30	9955.30	193.70	884.40	1616.80	5935.70
		Bootstrap	180.80	892.70	1787.00	8851.40	180.80	892.50	1785.20	8837.20	181.00	889.90	1758.20	8408.30	181.00	894.50	1791.90	8929.10	179.00	829.50	1548.00	6141.30
		Chao1984	580.10	93351.20	210869.40	226925.80	580.10	45585.50	196680.90	199572.40	100.00	43655.70	33049.70	54343.30	100.00	25200.50	225000.60	938528.00	2902.10	3923.60	4892.70	10856.90
		Chao&Lee1	**585.00**	**93575.00**	**211316.70**	226706.30	**585.00**	41316.70	179342.60	199911.00	100.00	38924.50	32904.70	58780.20	100.00	**25275.00**	**225250.00**	933604.60	**2950.10**	3925.40	5521.50	12057.10
		Chao&Lee2	**585.00**	**93575.00**	**211316.70**	**245043.10**	**585.00**	**45980.20**	**198268.50**	**213030.90**	100.00	**45088.10**	**36647.70**	**83214.20**	100.00	**25275.00**	**225250.00**	**976729.70**	**2950.10**	**4803.90**	**8319.40**	**18835.30**
		Chao&Bunge	340.00	46925.00	105933.30	26271.90	340.00	16916.30	74087.70	34635.20	100.00	12960.90	8107.80	4677.70	100.00	12850.00	113000.00	304803.40	1499.90	458.70	857.40	3354.70

Table 2 Average rankings of the methods according to the type of problem

PFSP		LOP		QAP	
Method	Ranking	Method	Ranking	Method	Ranking
Chao&Lee2	4.85	*Chao&Lee2*	4.83	*Chao&Lee2*	4.77
Chao&Lee1	3.74	*Chao&Lee1*	4.08	*Chao&Lee1*	3.97
Chao1984	3.51	*Chao1984*	3.78	*Chao1984*	3.60
Bootstrap	3.37	*Chao&Bunge*	2.82	*Jackknife*	2.97
Chao&Bunge	3.05	*Jackknife*	2.78	*Bootstrap*	2.88
Jackknife	2.47	*Bootstrap*	2.69	*Chao&Bunge*	2.81

Table 3 Average rankings of the methods according to the neighborhood

N_S		N_I	
Method	Ranking	Method	Ranking
Chao&Lee2	5.22	*Chao&Lee2*	4.41
Chao&Lee1	3.94	*Chao&Lee1*	3.92
Chao1984	3.61	*Chao1984*	3.66
Bootstrap	3.12	*Jackknife*	3.26
Chao&Bunge	2.88	*Chao&Bunge*	2.91
Jackknife	2.23	*Bootstrap*	2.84

LOP (the pair of columns in the middle), and QAP (last pair of columns). The lower the rank, the worse the performance of the method is. So, the methods are ordered from best to worst. Therefore, the best methods in the three groups are *Chao&Lee2*, *Chao&Lee1*, and *Chao1984*. However, pairwise significant differences are not found between *Chao&Lee1* and *Chao1984*, *Chao1984* and *Bootstrap*, and *Bootstrap* and *Chao&Bunge*, for the PFSP instances, and between *Bootstrap* and *Chao&Bunge*, *Bootstrap* and *Jackknife*, and *Jackknife* and *Chao&Bunge*, for the LOP and the QAP instances.

In Table 3, the ranking for the methods is shown, but this time for the second scenario, that is, when grouping the estimations for the instances using the 2-exchange neighborhood N_S (the first pair of columns), and the insert neighborhood N_I (the last pair of columns). In the first case, the Holm's procedure states that significant differences exist among each pair of methods. Nevertheless, for the second neighborhood, significant differences are not found between *Chao&Lee1* and *Chao1984* and between *Chao&Bunge* and *Bootstrap*. It is observed in this table that, also in this scenario, the best estimations are provided by *Chao&Lee2*, *Chao&Lee1*, and *Chao1984*.

Finally, Table 4 shows the ranking obtained for the methods when the estimations are separated according to the sample size used by the methods $M = 100, 500, 1000, 5000$. Again, the three best methods are *Chao&Lee2*, *Chao&Lee1*, and *Chao1984*. Significant differences are not found between *Chao&Lee1* and *Chao1984* and between *Bootstrap* and *Jackknife*, when using $M = 100$. For $M = 500$ there are not significant differences among Bootstrap, *Chao&Bunge*, and *Jackknife*, while significant differences are not found between *Bootstrap* and *Chao&Bunge* when

Table 4 Average rankings of the methods according to the sample size

$M = 100$		$M = 500$		$M = 1000$		$M = 5000$	
Method	Ranking	Method	Ranking	Method	Ranking	Method	Ranking
Chao&Lee2	4.72	*Chao&Lee2*	5.01	*Chao&Lee2*	5.01	*Chao&Lee2*	4.54
Chao&Lee1	3.87	*Chao&Lee1*	4.06	*Chao&Lee1*	3.97	*Chao&Lee1*	3.83
Chao1984	3.65	*Chao1984*	3.63	*Chao1984*	3.57	*Chao1984*	3.67
Bootstrap	3.19	*Bootstrap*	2.93	*Bootstrap*	3.00	*Chao&Bunge*	3.48
Jackknife	3.12	*Chao&Bunge*	2.69	*Chao&Bunge*	2.93	*Bootstrap*	2.81
Chao&Bunge	2.46	*Jackknife*	2.67	*Jackknife*	2.52	*Jackknife*	2.67

Table 5 Average relative errors and standard deviations (in brackets) of the estimations provided by the different methods for the instances of the PFSP, according to the neighborhood and the sample size M

		$M = 100$	$M = 500$	$M = 1000$	$M = 5000$
2-exchange	*Jackknife*	0.88 (0.14)	0.74 (0.22)	0.66 (0.24)	0.44 (0.22)
	Bootstrap	0.87 (0.17)	0.69 (0.30)	0.61 (0.28)	0.38 (0.15)
	Chao1984	0.79 (0.15)	0.68 (0.21)	0.60 (0.22)	0.42 (0.20)
	Chao&Lee1	0.79 (0.14)	0.67 (0.20)	0.60 (0.20)	0.42 (0.19)
	Chao&Lee2	**0.74 (0.18)**	**0.60 (0.20)**	**0.51 (0.19)**	0.35 (0.16)
	Chao&Bunge	0.89 (0.26)	1.06 (2.23)	0.84 (1.12)	**0.28 (0.38)**
Insert	*Jackknife*	0.60 (0.28)	0.39 (0.22)	0.30 (0.18)	0.19 (0.23)
	Bootstrap	0.58 (0.26)	0.28 (0.23)	0.21 (0.16)	0.54 (0.40)
	Chao1984	0.53 (0.23)	0.34 (0.19)	0.28 (0.17)	0.19 (0.21)
	Chao&Lee1	0.54 (0.24)	0.34 (0.18)	0.29 (0.15)	**0.17 (0.19)**
	Chao&Lee2	**0.45 (0.22)**	**0.25 (0.15)**	0.24 (0.14)	0.23 (0.20)
	Chao&Bunge	0.92 (1.40)	0.50 (0.78)	**0.20 (0.18)**	0.32 (0.24)

$M = 1000$. Finally, when taking $M = 5000$, there are not significant differences among *Chao&Lee1*, *Chao1984*, and *Chao&Bunge* and between *Jackknife* and *Bootstrap*.

Although the statistical analysis gives a global picture of the performance of the methods, and *Chao&Lee2* provides the best solutions with significant differences between this method and the remaining methods, it is also relevant to consider some aspects that are not reflected in the hypothesis tests. The closeness of the estimations provided by the methods to the value to estimate is the most important factor. Obviously, there are methods that estimate better than others, but it does not mean that the estimations provided by the best methods are close enough to the real value. In order to check if the methods provide useful estimations, and so as to add more information to that deduced from Table 1, the average errors of the estimations with respect to the real number of local optima are calculated. Tables 5, 6, and 7 show the average relative errors and the standard deviations (in brackets) grouped by the neighborhood used and the sample size, for all the instances of the PFSP, LOP, and QAP, respectively.

Table 6 Average relative errors and standard deviations (in brackets) of the estimations provided by the different methods for the instances of the LOP, according to the neighborhood and the sample size M

		$M = 100$	$M = 500$	$M = 1000$	$M = 5000$
2-exchange	*Jackknife*	0.96 (0.01)	0.87 (0.04)	0.81 (0.06)	0.56 (0.08)
	Bootstrap	0.97 (0.01)	0.87 (0.04)	0.79 (0.06)	0.47 (0.12)
	Chao1984	0.83 (0.14)	0.79 (0.06)	0.73 (0.06)	0.52 (0.07)
	Chao&Lee1	0.84 (0.12)	0.78 (0.05)	0.71 (0.07)	0.49 (0.06)
	Chao&Lee2	**0.80 (0.15)**	**0.71 (0.07)**	**0.62 (0.09)**	**0.37 (0.05)**
	Chao&Bunge	0.95 (0.06)	2.38 (8.21)	1.12 (1.44)	0.51 (0.47)
Insert	*Jackknife*	0.33 (0.30)	0.18 (0.27)	0.16 (0.22)	0.77 (0.49)
	Bootstrap	**0.28 (0.32)**	0.37 (0.21)	0.26 (0.17)	1.05 (0.61)
	Chao1984	0.35 (0.25)	**0.18 (0.24)**	0.14 (0.20)	**0.72 (0.47)**
	Chao&Lee1	0.35 (0.28)	0.19 (0.23)	0.14 (0.19)	0.75 (0.48)
	Chao&Lee2	0.33 (0.33)	**0.18 (0.24)**	**0.12 (0.15)**	0.82 (0.51)
	Chao&Bunge	0.49 (0.74)	0.31 (0.70)	1.39 (4.95)	0.83 (0.37)

Table 7 Average relative errors and standard deviations (in brackets) of the estimations provided by the different methods for the instances of the QAP, according to the neighborhood and the sample size M

		$M = 100$	$M = 500$	$M = 1000$	$M = 5000$
2-exchange	*Jackknife*	0.87 (0.17)	0.75 (0.25)	0.67 (0.28)	0.50 (0.29)
	Bootstrap	0.84 (0.20)	0.69 (0.33)	0.62 (0.34)	0.44 (0.32)
	Chao1984	0.77 (0.24)	0.57 (0.21)	0.53 (0.19)	0.41 (0.19)
	Chao&Lee1	0.78 (0.20)	0.56 (0.21)	0.53 (0.19)	0.42 (0.19)
	Chao&Lee2	**0.73 (0.25)**	**0.52 (0.22)**	**0.49 (0.19)**	0.38 (0.19)
	Chao&Bunge	0.91 (0.40)	0.87 (1.41)	1.89 (5.45)	**0.27 (0.39)**
Insert	*Jackknife*	1.00 (0.00)	1.00 (0.00)	1.00 (0.01)	0.98 (0.03)
	Bootstrap	1.00 (0.00)	1.00 (0.00)	1.00 (0.01)	0.98 (0.03)
	Chao1984	**0.99 (0.02)**	**0.96 (0.04)**	**0.93 (0.06)**	0.89 (0.04)
	Chao&Lee1	**0.99 (0.02)**	0.97 (0.03)	0.94 (0.05)	0.89 (0.04)
	Chao&Lee2	**0.99 (0.02)**	**0.96 (0.04)**	**0.93 (0.06)**	**0.86 (0.06)**
	Chao&Bunge	1.00 (0.01)	0.99 (0.01)	0.98 (0.03)	0.98 (0.03)

A general conclusion drawn from Tables 5, 6, and 7 is that, in general terms, the estimations improve as the sample size grows, and they are worse as the number of local optima increases. Specifically, as shown in Table 1, the instances of the LOP with the insert neighborhood have a low number of local optima, and Table 6 shows that the average error is lower than in the rest of the cases. Moreover, the instances of the QAP with the insert neighborhood have a high number of local optima, and the errors obtained for those instances (in Table 7) are higher than for the rest. It is remarkable that, for sample size $M = 100$ and $M = 500$, the estimations are far from the real value. However, there is one method which behaves different and does not follow the general lines. This is the *Chao&Bunge* method. This estimation method

does not necessarily improve as the sample size grows, and the standard deviation presented in most of the cases is really high compared with other methods. It seems that *Chao&Bunge* is a very unstable method. The instability of this method is a consequence of the variability on the estimation of the parameter that represents the expected proportion of duplicates in the sample [22]. This estimation is unreliable when the sample has many local optima that are seen only once, but there is a small number of local optima seen a low (but higher than one) number of times. These particularities are commonly found when the sample size is small with respect to the number of local optima or even when the variance of the sizes of the attraction basins of the local optima is high.

As a conclusion from this analysis, we highly recommend the use of the *Chao&Lee2* method to estimate the number of local optima. According to the hypothesis test, this estimation method provides the best results. Although in the statistical analysis it is not reflected, the *Chao&Bunge* method gives also good estimations in a high number of occasions; however, its instability provokes that it is not trusted. Therefore, we recommend to execute both methods independently. If the results provided are close, *Chao&Bunge* is usually the choice; otherwise, select *Chao&Lee2*.

Conclusions

The objective of this study has been to extend and advance the knowledge associated to implementing multi-start procedures. Unlike other well-known methods, it has not yet become widely implemented and tested as a metaheuristic itself for solving complex optimization problems. Previous efforts have been classified according to its domain of applicability: global and combinatorial optimization.

In this chapter, different methods have been reviewed for estimating the number of local optima of instances of combinatorial optimization problems. Additionally, some methods in the optimization field have been compared with methods previously used for estimating the number of species in a population in the field of statistics. The methods have been applied to instances of three well-known problems in combinatorial optimization and two classical neighborhoods. The main conclusions are that, in general, the higher the sample used by the methods, the more precise the estimations, and the higher the number of local optima, the worse the estimations provided. Based on the results observed through the experiments, we recommend using the *Chao&Lee2* method or applying it together with the *ChaoBunge* method for a more accurate local optima estimation.

Cross-References

▸ GRASP
▸ Restart Strategies
▸ Tabu Search
▸ Theory of Local Search

Acknowledgments This work has been partially supported by the Spanish government with grants PID2021-125709OB-C21 and PID2024-160226OB-C21 funded by MCIN/AEI/10.13039/501100011033 and by ERDF A way of making Europe.

Competing Interest Declaration The author(s) has no competing interests to declare that are relevant to the content of this manuscript.

References

1. Albrecht A, Lane P, Steinhofel K (2008) Combinatorial landscape analysis for k-SAT instances. In: IEEE congress on evolutionary computation, CEC 2008, Hong Kong. IEEE World congress on computational intelligence, pp 2498–2504
2. Albrecht A, Lane P, Steinhofel K (2010) Analysis of local search landscapes for k-SAT instances. Math Comput Sci 3(4):465–488
3. Beausoleil R, Baldoquin G, Montejo R (2008) A multi-start and path relinking methods to deal with multiobjective knapsack problems. Ann Oper Res 157:105–133
4. Boese K, Kahng A, Muddu S (1994) A new adaptive multi-start technique for combinatorial global optimisation. Oper Res Lett 16:103–113
5. Braysy O, Hasle G, Dullaert W (2004) A multi-start local search algorithm for the vehicle routing problem with time windows. Eur J Oper Res 159:586–605
6. Chao A (1984) Nonparametric estimation of the number of classes in a population. Scand J Stat 11(4):265–270
7. Chao A, Bunge J (2002) Estimating the number of species in a stochastic abundance model. Biometrics 58(3):531–539
8. Chao A, Lee SM (1992) Estimating the number of classes via sample coverage. J Am Stat Assoc 87(417):210–217
9. Crowston WB, Glover F, Thompson GL, Trawick JD (1963) Probabilistic and parametric learning combinations of local job shop scheduling rules. Technical report 117. Carnegie-Mellon University, Pittsburgh
10. Dhouib S, Kharrat A, Chabchoub H (2010) A multi-start threshold accepting algorithm for multiple objective continuous optimization problems. Int J Numer Methods Eng 83:1498–1517
11. Eremeev AV, Reeves CR (2002) Non-parametric estimation of properties of combinatorial landscapes. In: Cagnoni S, Gottlieb J, Hart E, Middendorf M, Raidl G (eds) Applications of evolutionary computing. Lecture notes in computer science, vol 2279. Springer, Berlin/Heidelberg, pp 31–40
12. Eremeev AV, Reeves CR (2003) On confidence intervals for the number of local optima. In: Proceedings of EvoWorkshops 2003, Essex, pp 224–235
13. Essafi M, Delorme X, Dolgui A (2010) Balancing lines with CNC machines: a multi-start and based heuristic. CIRP J Manuf Sci Technol 2:176–182
14. Feo T, Resende M (1989) A probabilistic heuristic for a computationally difficult set covering problem. Oper Res Lett 8:67–71
15. Feo T, Resende M (1995) Greedy randomized adaptive search procedures. J Glob Optim 6:109–133
16. Fleurent C, Glover F (1999) Improved constructive multi-start strategies for the quadratic assignment problem using adaptive memory. INFORMS J Comput 11:198–204
17. Glover F (2000) Multi-start and strategic oscillation methods – principles to exploit adaptive memory. In: Laguna M, Gonzalez-Velarde J (eds) Computing tools for modeling optimization and simulation. Kluwer Academic, Boston, pp 1–25
18. Glover F, Laguna M (1997) Tabu search. Kluwer Academic, Boston
19. Grundel D, Krokhmal P, Oliveira C, Pardalos P (2007) On the number of local minima for the multidimensional assignment problem. J Combin Optim 13:1–18
20. Hagen L, Kahng A (1997) Combining problem reduction and adaptive multi-start: a new technique for superior iterative partitioning. IEEE Trans CAD 16:709–717

21. Held M, Karp R (1970) The traveling-salesman problem and minimum spanning trees. Oper Res 18:1138–1162
22. Hernando L, Mendiburu A, Lozano JA (2013) An evaluation of methods for estimating the number of local optima in combinatorial optimization problems. Evol Comput 21(4):625–658
23. Hickernell F, Yuan Y (1997) A simple multistart algorithm for global optimization. OR Trans 1:1–11
24. Hu X, Shonkwiler R, Spruill M (1994) Random restarts in global optimization. Ga Inst Technol 1:1–10
25. Kan AR, Timmer G (1987) Stochastic global optimization methods (Part II): multi level methods. Math Program 39:57–78
26. Kan AR, Timmer G (1998) Global optimization. In: Kan R, Todd MJ (eds) Handbooks in operations research and management science. North Holland, Amsterdam, pp 631–662
27. Kaucic M (2013) A multi-start opposition-based particle swarm optimization algorithm with adaptive velocity for bound constrained global optimization. J Glob Optim 55:165–188
28. Lan G, DePuy G (2006) On the effectiveness of incorporating randomness and memory into a multi-start metaheuristic with application to the set covering problem. Comput Ind Eng 51:362–374
29. Martí R, Reinelt G, Duarte A (2012) A benchmark library and a comparison of heuristic methods for the linear ordering problem. Comput Optim Appl 51(3):1297–1317
30. Martí R, Resende M, Ribeiro C (2013) Multi-start methods for combinatorial optimization. Eur J Oper Res 226(1):1–8
31. Mayne DQ, Meewella C (1988) A non-clustering multistart algorithm for global optimization. In: Bensoussan A, Lions J-L (eds) Analysis and optimization of systems. Lecture notes in control and information sciences. Springer, Berlin/New York, pp 111–117
32. Mezmaz M, Melab N, Talbi E (2006) Using the multi-start and Island models for parallel multi-objective optimization on the computational grid. In: Second IEEE international conference on e-science and grid computing, Amsterdam
33. Moreno J, Mladenovic N, Moreno-Vega J (1995) An statistical analysis of strategies for multistart heuristic searches for *p*-facility location-allocation problems. In: Eighth meeting of the EWG on locational analysis, Lambrecht
34. Muth JF, Thompson GL (1963) Industrial scheduling. Prentice-Hall, Englewood Cliffs
35. Patterson R, Pirkul H, Rolland E (1999) Adaptive reasoning technique for the capacitated minimum spanning tree problem. J Heuristics 5:159–180
36. Reeves C, Aupetit-Bélaidouni M (2004) Estimating the number of solutions for SAT problems. In: Yao X, Burke E, Lozano J, Smith J, Merelo-Guervós J, Bullinaria J, Rowe J, Tino P, Kabán A, Schwefel HP (eds) Parallel problem solving from nature – PPSN VIII. Lecture notes in computer science, vol 3242. Springer, Berlin/Heidelberg, pp 101–110
37. Resende M, Ribeiro C (2010) Greedy randomized adaptive search procedures: advances and applications. In: Gendreau M, Potvin JY (eds) Handbook of metaheuristics, 2nd edn. Springer, New York, pp 293–319
38. Solis F, Wets R (1981) Minimization by random search techniques. Math Oper Res 6:19–30
39. Taillard E, Badeau P, Gendreau M, Guertin F, Potvin J (1997) A tabu search heuristic of the vehicle routing problem with time windows. Transp Sci 31:170–186
40. Tu W, Mayne R (2002) An approach to multi-start clustering for global optimization with non-linear constraints. Int J Numer Methods Eng 53:2253–2269
41. Ugray Z, Lasdon L, Plummer J, Glover F, Kelly J, Martí R (2007) Scatter search and local NLP solvers: a multistart framework for global optimization. INFORMS J Comput 19(3):328–340
42. Ugray Z, Lasdon L, Plummer J, Bussieck M (2009) Dynamic filters and randomized drivers for the multi-start global optimization algorithm MSNLP. Optim Methods Softw 24:4–5
43. Villegas J, Prins C, Prodhon C, Medaglia A, Velasco N (2010) GRASP/VND and multi-start evolutionary local search for the single truck and trailer routing problem with satellite depots. Eng Appl Artif Intell 23:780–794

Multiobjective Optimization

10

Carlos A. Coello Coello

Contents

Introduction . . . 232
Basic Concepts . . . 233
Pareto Optimality . . . 233
Multiobjective Evolutionary Algorithms . . . 234
Multiobjective Evolutionary Algorithms . . . 236
Recent Algorithmic Trends . . . 239
Use of Other Metaheuristics . . . 240
Artificial Immune Systems (AIS) . . . 241
Particle Swarm Optimization (PSO) . . . 241
Ant Colony Optimization (ACO) . . . 242
Other Metaheuristics . . . 242
Some Applications . . . 242
Some Current Challenges . . . 243
Conclusions . . . 245
Cross-References . . . 246
References . . . 246

Abstract

This chapter provides a short overview of multiobjective optimization using metaheuristics. The chapter includes a description of some of the main metaheuristics that have been used for multiobjective optimization. Although special emphasis is made on evolutionary algorithms, other metaheuristics, such as particle swarm optimization, artificial immune systems, and ant colony

C. A. Coello Coello (✉)
Departmento de Computación, CINVESTAV-IPN, México D.F., México
e-mail: carlos.coellocoello@cinvestav.mx; ccoello@cs.cinvestav.mx

© Springer Nature Switzerland AG 2025

R. Martí et al. (eds.), *Handbook of Heuristics*,
https://doi.org/10.1007/978-3-032-00385-0_17

optimization, are also briefly discussed. Other topics such as applications and recent algorithmic trends are also included. Finally, some of the main research trends that are worth exploring in this area are briefly discussed.

Keywords

Multiobjective optimization · Metaheuristics · Optimization

Introduction

Metaheuristics have been widely used for solving different types of optimization problems (see, for example, [49, 73, 100]).

One particular class of optimization problems involves having two or more (often conflicting) objectives which we aim to optimize at the same time. In fact, such problems, which are called "multiobjective," are quite common in real-world applications, and their solution has triggered an important amount of work within Operations Research [129].

During the last 45 years, a large number of mathematical programming techniques have been developed to solve certain specific classes of multiobjective optimization problems. However, such techniques have a relatively limited applicability (e.g., some of them require the first or even the second derivative of the objective functions and the constraints, others can only deal with convex Pareto fronts, etc.). Such limitations have motivated the development of alternative optimization methods, from which metaheuristics have become a very popular alternative [26, 69].

From the many metaheuristics currently available, evolutionary algorithms have been, without doubt, the most popular choice for dealing with any sort of optimization problem [43] and multiobjective optimization is, by no means, an exception. Thus, in this chapter, we will focus our discussion mainly on the use of evolutionary algorithms for solving multiobjective optimization problems.

The use of evolutionary algorithms for solving multiobjective optimization problems was originally hinted in 1967 [150], but the first actual implementation of what is now called a "multiobjective evolutionary algorithm (MOEA)" was not produced until the mid-1980s [157, 158]. However, this area, which is now called "evolutionary multiobjective optimization," or EMO, has experienced a very important growth, mainly in the last 20 years [10, 14, 24]. The author maintains the EMOO repository, which, as of May 10th, 2024, contains over 13,650 bibliographic entries, as well as public-domain implementation of some of the most popular MOEAs. The EMOO repository is located at https://emoo.cs.cinvestav.mx/.

The remainder of this chapter is organized as follows. In section "Basic Concepts," we provide some basic concepts related to multiobjective optimization, which are required to make this chapter self-contained. The use of evolutionary algorithms in multiobjective optimization is motivated in section "Multiobjective Evolutionary Algorithms." The most popular MOEAs that have been developed

so far are briefly discussed in section "Multiobjective Evolutionary Algorithms." A short discussion on other bio-inspired metaheuristics that have also been used for multiobjective optimization is provided in section "Use of Other Metaheuristics." A set of sample applications of MOEAs is provided in section "Some Applications." Some of the main topics of research in the EMO field that currently attract a lot of attention are briefly discussed in section "Some Current Challenges." Finally, some conclusions are provided in section "Conclusions."

Basic Concepts

In this chapter, we focus on the solution of multiobjective optimization problems (MOPs) of the form:

$$\text{Minimize } \left[f_1(\vec{x}), f_2(\vec{x}), \ldots, f_k(\vec{x})\right] \tag{1}$$

subject to the m inequality constraints:

$$g_i(\vec{x}) \leq 0 \quad i = 1, 2, \ldots, m \tag{2}$$

and the p equality constraints:

$$h_i(\vec{x}) = 0 \quad i = 1, 2, \ldots, p \tag{3}$$

where k is the number of objective functions $f_i : \mathbb{R}^n \rightarrow \mathbb{R}$. We call $\vec{x} = [x_1, x_2, \ldots, x_n]^T$ the vector of decision variables. We wish to determine from among the set $\mathcal{F}$ of all vectors which satisfy (2) and (3) the particular set of values $x_1^*, x_2^*, \ldots, x_n^*$ which yield the optimum values of all the objective functions.

Pareto Optimality

It is rarely the case that there is a single point that simultaneously optimizes all the objective functions. In fact, this situation only arises when there is no conflict among the objectives, which would make unnecessary the development of special solution methods, since this single solution could be reached after the sequential optimization of all the objectives, considered separately. Therefore, we normally look for "trade-offs," rather than single solutions, when dealing with multiobjective optimization problems. The notion of "optimality" normally adopted in this case is the one originally proposed by Francis Ysidro Edgeworth [50] and later generalized by the Italian economist Vilfredo Pareto [137]. Although some authors call this notion *Edgeworth-Pareto optimality*, we will use the most commonly adopted term: *Pareto optimality*.

We say that a vector of decision variables $\vec{x}^* \in \mathcal{F}$ (i.e., a feasible solution) is *Pareto optimal* if there does not exist another $\vec{x} \in \mathcal{F}$ such that $f_i(\vec{x}) \leq f_i(\vec{x}^*)$ for all

$i = 1, \ldots, k$ and $f_j(\vec{x}) < f_j(\vec{x}^*)$ for at least one j (assuming that all the objectives are being minimized).

In words, this definition says that $\vec{x}^*$ is a Pareto optimal solution if there exists no feasible vector of decision variables $\vec{x} \in \mathcal{F}$ which would decrease some criterion without causing a simultaneous increase in at least one other criterion. Assuming the inherent conflict normally present among (at least some) objectives, the use of this concept normally produces several solutions. Such solutions constitute the so-called Pareto optimal set. The vectors $\vec{x}^*$ corresponding to the solutions included in the Pareto optimal set are called *nondominated*. The image of the Pareto optimal set under the objective functions (i.e., the objective function values corresponding to the decision variables contained in the Pareto optimal set) is called *Pareto front*.

Multiobjective Evolutionary Algorithms

The core ideas related to the development of search techniques that simulate the mechanism of natural selection (Darwin's survival of the fittest principle) can be traced back to the early 1930s [58]. However, the three main techniques based on this notion were developed during the 1960s: genetic algorithms [86], evolution strategies [162], and evolutionary programming [59]. These approaches, which are now generically denominated "evolutionary algorithms," have been found to be very effective for solving single-objective optimization problems [60, 70, 163].

The basic operation of an evolutionary algorithm (EA) is described next. First, a set of potential solutions (called "population") to the problem being solved is randomly generated. Each solution in the population (called "individual") encodes all the decision variables of the problem (i.e., each individual contains all the decision variables of the problem to be solved). The user needs to define a measure of performance for each of the solutions. Such a measure of performance is called "fitness function" and will allow us to know how good is a solution with respect to the others. Such a fitness function is normally a variation of the objective function of the problem that we wish to solve (e.g., the objective function that we aim to optimize). Then, a selection mechanism must be applied in order to decide which individuals will "mate." This selection process is generally stochastic and is normally based on the fitness contribution of each individual (i.e., the fittest individuals have a higher probability of being selected). Upon mating, a set of "offspring" (or children) are generated. Such offspring are "mutated" (this operator produces a small random change, with a low probability, on the contents of an individual) and constitute the new population to be evaluated at the next iteration (each iteration is called a "generation"). This process is repeated until reaching a stopping condition (normally, a maximum number of generations defined by the user) [51].

The main motivation for using EAs for solving multiobjective optimization problems relies on their population-based nature, which allows them to generate (if properly manipulated) several elements of the Pareto optimal set, in a single

run. In contrast, mathematical programming techniques normally generate a single element of the Pareto optimal set per run. Additionally, the so-called multiobjective evolutionary algorithms (MOEAs) are less susceptible to the shape and continuity of the Pareto front and require less specific domain information to operate [24].

MOEAs extend a traditional (single-objective) EA in two main aspects:

- **The selection mechanism**: In this case, the aim is to select nondominated solutions and to consider all the nondominated solutions in a population to be equally good (unless there is some specific preference from the user, all the elements of the Pareto optimal set are equally good).
- **A diversity maintenance mechanism**: Because of stochastic noise, EAs tend to converge to a single solution if run for a sufficiently long time [70]. In order to avoid this, it is necessary to block the selection mechanism in a MOEA, favoring the diversity of solutions, so that several elements of the Pareto optimal set can be generated in a single run.

Regarding selection, early MOEAs relied on the use of aggregating functions (mainly linear) [76] and relatively simple population-based approaches [157]. However, such approaches have evident drawbacks (i.e., the use of linear aggregating functions does not allow the generation of nonconvex portions of the Pareto front regardless of the weight combination that is adopted [34]). Towards the mid-1990s, MOEAs started to adopt variations of the so-called *Pareto ranking* selection mechanism. This approach was originally proposed by David E. Goldberg in his seminal book on genetic algorithms [70], and it consists of sorting the population of an EA based on Pareto optimality, such that all nondominated individuals are assigned the same rank (or importance). The aim is that all nondominated individuals get the same probability of being selected and that such probability is higher than the one corresponding to individuals which are dominated. Although conceptually simple, this sort of selection mechanism allows for a wide variety of possible implementations [24, 38].

Regarding diversity maintenance, a wide variety of methods have been proposed in the specialized literature to maintain diversity in a MOEA. Such approaches include fitness sharing and niching [39, 72], clustering [179, 205], geographically based schemes [103], the use of entropy [30, 102] and parallel coordinates [84], among others. In all cases, the core idea behind diversity maintenance mechanisms is to penalize solutions that are too close from each other in some space (i.e., decision variable or objective function space or even both). Most MOEAs penalize solutions that are too close from each other in objective function space, because it is normally aimed to have solutions well distributed along the Pareto front.

Additionally, some researchers have proposed the use of mating restriction schemes (which impose rules on the individuals that can be recombined) [93, 114, 202]. Furthermore, the use of relaxed forms of Pareto dominance became relatively popular some years ago, mainly as an archiving technique which encourages diversity while allowing the archive to regulate convergence (the most popular of

such mechanisms is, with no doubt, ϵ-dominance [107], which has been adopted in some approaches such as ϵ-MOEA [41]).

A third component of modern MOEAs is elitism, which normally consists of using an external archive (also called "secondary population" [88]) that may (or may not) interact in different ways with the main (or "primary") population of the MOEA during selection. The main purpose of this archive is to store *all* the nondominated solutions generated throughout the search process while removing those that become dominated later in the search (called local nondominated solutions). The approximation of the Pareto optimal set produced by a MOEA is thus the final contents of this archive. It is important to emphasize that the use of elitism is not only advisable (the lack of elitism could make us lose nondominated solutions generated during the search), but it is also required because of theoretical reasons (elitism is required in order to guarantee convergence of a MOEA to the Pareto optimal set as proved in [152]).

It is worth noticing that, in practice, external archives are normally bounded to a certain maximum number of solutions [104]. This was originally done in some MOEAs that used the external archive during the selection stage (see, for example, [205]). In such a case, allowing the size of the archive to grow too much dilutes the selection pressure, which has a negative effect on the performance of the MOEA. However, most modern MOEAs bound the size of the external archive, even if the archive is not used during the selection process, mainly because of practical reasons (this makes easier to compare results with respect to other MOEAs).

An important remark is that the use of a plus (+) selection is another possible elitist mechanism. Under this sort of selection scheme, the population of parents competes with the population of offspring (both populations are of the same size) and then we keep only the best half. This sort of selection scheme has been relatively popular in single-objective optimization and has been also adopted in some modern MOEAs (see, for example, [42]), but it is less popular than the use of external archives.

Multiobjective Evolutionary Algorithms

In spite of the very large number of publications related to MOEAs that can be found in the literature, there is only a handful of algorithms that are actually used by a significant number of researchers and/or practitioners around the world:

1. **Strength Pareto evolutionary algorithm 2** (SPEA2): This is an updated version of the strength Pareto evolutionary algorithm (SPEA) proposed in the late 1990s [205] whose main features are the following—it adopts an external archive (called the external nondominated set), which stores the nondominated solutions previously generated and participates in the selection process (together with the main population). For each individual in this archive, a *strength* value is computed. This strength value is proportional to the number of solutions that a

certain individual dominates. In SPEA, the fitness of each member of the current population is computed according to the strengths of all external nondominated solutions that dominate it. As the size of the external nondominated set grows too much, this significantly reduces the selection pressure and slows down the search. In order to avoid this, SPEA adopts a clustering technique that prunes the contents of the external nondominated set so that its size remains below a certain (predefined) threshold. SPEA standardized the use of external archives as the elitist mechanism of a MOEA, although this sort of mechanism had been used before by other researchers (see, for example, [91]). SPEA2 has three main differences with respect to the original SPEA [208]: (1) It incorporates a fine-grained fitness assignment strategy which takes into account, for each individual, both the number of individuals that dominate it and the number of individuals by which it is dominated, (2) it uses a nearest neighbor density estimation technique which guides the search more efficiently (i.e., a more efficient clustering algorithm is adopted), and (3) it uses an enhanced archive truncation method that guarantees the preservation of boundary solutions (this fixes a bug from the original SPEA).

2. **Pareto archived evolution strategy** (PAES): This is perhaps the most simple MOEA than can be possibly designed. It was proposed by Knowles and Corne [105], and it consists of a (1+1) evolution strategy (i.e., a single parent that generates a single offspring through the application of mutation to the parent) in combination with a historical archive that stores the nondominated solutions previously found. This archive is used as a reference set against which each mutated individual is being compared. Such (external) archive adopts a crowding procedure that divides objective function space in a recursive manner. Then, each solution is placed in a certain grid location based on the values of its objectives (which are used as its "coordinates" or "geographical location"). A map of such a grid is maintained, indicating the number of solutions that reside in each grid location. When a new nondominated solution is ready to be stored in the archive, but there is no room for it (the size of the external archive is bounded), a check is made on the grid location to which the solution would belong. If this grid location is less densely populated than the most densely populated grid location, then a solution (randomly chosen) from this heavily populated grid location is deleted to allow the storage of the newcomer. This aims to redistribute solutions, favoring the less densely populated regions of the Pareto front. Since the procedure is adaptive, no extra parameters are required (except for the number of divisions of the objective space).
3. **Nondominated sorting genetic algorithm II** (NSGA-II): This is a heavily revised version of the nondominated sorting genetic algorithm (NSGA), which was originally proposed in the mid-1990s [171] as a straightforward implementation of the Pareto ranking algorithm described by Goldberg in his book [70]. NSGA was, however, slow and produced poorer results than other non-elitist MOEAs available in the mid-1990s, such as MOGA [61] and NPGA [89]. NSGA-II adopts a more efficient ranking procedure than its predecessor. Additionally, it estimates the density of solutions surrounding a particular one

in the population by computing the average distance of two points on either side of this point along each of the objectives of the problem. This value is the so-called crowding distance. During selection, NSGA-II uses a crowded-comparison operator which takes into consideration both the nondomination rank of an individual in the population and its crowding distance (i.e., nondominated solutions are preferred over dominated solutions, but between two solutions with the same nondomination rank, the one that resides in the less crowded region is preferred). NSGA-II does not use an external archive as most of the modern MOEAs in current use. Instead, the elitist mechanism of NSGA-II consists of combining the best parents with the best offspring obtained (i.e., this is the $(\mu + \lambda)$-selection used in evolution strategies). Due to its clever mechanisms, NSGA-II is much more efficient (computationally speaking) than its predecessor, and its performance is so good that it became very popular, triggering a significant number of applications and becoming some sort of landmark against which new MOEAs were compared during more than 20 years, in order to merit publication. More recently, the nondominated sorting genetic algorithm III (NSGA-III) [40] was introduced. NSGA-III still adopts the same NSGA-II framework (i.e., it still performs a classification of the population in nondominated levels). However, its mechanism to maintain diversity is based on the use of a number of well-spread reference points which are adaptively updated. The NSGA-III does not require any additional parameters (other than those associated to the genetic algorithm that is used as its search engine), and it is designed to deal with many-objective optimization problems (i.e., problems having 4 or more objectives).

4. **Pareto envelope-based selection algorithm** (PESA): This algorithm was proposed by Corne et al. [28] and uses a small internal population and a larger external (or secondary) population. PESA adopts the same adaptive grid from PAES to maintain diversity. However, its selection mechanism is based on a crowding measure. This same crowding measure is used to decide what solutions to introduce into the external population (i.e., the archive of nondominated vectors found along the evolutionary process). Therefore, in PESA, the external memory plays a crucial role in the algorithm since it determines not only the diversity scheme, but also the selection performed by the method. There is also a revised version of this algorithm, called PESA-II [27], which is identical to PESA, except for the fact that region-based selection is used in this case. In region-based selection, the unit of selection is a hyperbox rather than an individual. The procedure consists of selecting (using any of the traditional selection techniques [71]) a hyperbox and then randomly selecting an individual within such hyperbox. The main motivation of this approach is to reduce the computational costs associated with traditional MOEAs (i.e., those based on Pareto ranking).
5. **Multiobjective evolutionary algorithm based on decomposition** (MOEA/D): This approach was proposed by Zhang and Li [201]—the main idea of this algorithm is to decompose a multiobjective optimization problem into several

scalar optimization sub-problems which are simultaneously optimized. The decomposition process requires the use of weights, but the authors provide a method to generate them. During the optimization of each sub-problem, only information from the neighboring sub-problems is used, which allows this algorithm to be effective and efficient. It is worth indicating that the decomposition process is based on the use of a scalarizing function [138], which plays a key role in the performance of the algorithm. MOEA/D is considered one of the most powerful MOEAs currently available, as has been evidenced by several comparative studies and many extensions of the algorithm are currently available (see, for example, [125, 180, 189]).

Recent Algorithmic Trends

Many other MOEAs have been proposed in the specialized literature based on concepts such as the use of very small population sizes [177, 178], the use of the maximin expression [124, 128], and a transformation of a multiobjective problem into a linear assignment problem [182], among many others. However, they will not be discussed here due to obvious space limitations. A more interesting issue, however, is to try to predict which sort of MOEA will become predominant in the next few years.

Efficiency is, for example, a concern nowadays, and several approaches have been developed in order to improve the efficiency of MOEAs (see, for example, [46, 95, 111]). Also, the use of fitness approximation, fitness inheritance, surrogates, and other similar techniques has become more common in recent years, which is a clear indication of the more frequent use of MOEAs for the solution of computationally expensive problems (see, for example, [20, 121, 122, 198]).

However, the most promising research line within algorithmic design seems to be the use of a performance measure in the selection mechanism of a MOEA. This research trend formally started with the indicator-based evolutionary algorithm (IBEA) [206], although this idea had been already formulated and used (in different ways) by other authors before (see, for example, [90, 103]). Nevertheless, the most representative MOEA within this family is perhaps the S metric selection evolutionary multiobjective algorithm (SMS-EMOA) [11, 52]. SMS-EMOA was originally proposed by Emmerich et al. [52] and is based on NSGA-II. SMS-EMOA creates an initial population and then it generates only one solution by iteration (i.e., it uses steady-state selection) using the operators (crossover and mutation) of the NSGA-II. After that, it applies nondominated sorting. When the last front has more than one solution, SMS-EMOA uses the hypervolume contribution to decide which solution will be removed. The **hypervolume** (also known as the S metric or the Lebesgue measure) of a set of solutions measures the size of the portion of objective space that is dominated by those solutions collectively. This is the only unary performance indicator which is known to be fully Pareto compliant [209].

Beume et al. [11] proposed not to use the contribution to the hypervolume indicator when in the nondominated sorting process we obtain more than one front. In that case, they proposed to use the number of solutions which dominate one solution (the solution that is dominated by more solutions is removed). The authors argued that the motivation for using the hypervolume indicator is to improve the distribution in the nondominated front and then it is not necessary in fronts different to the nondominated front.

The hypervolume indicator has attracted a lot of attention from researchers due to its interesting theoretical properties. For example, it has been proved that the maximization of the hypervolume is equivalent to finding the Pareto optimal set [57]. Empirical studies have shown that (for a certain number of points previously determined) the maximization of the hypervolume does indeed produce subsets of the Pareto front which are well distributed [52, 103].

However, there are also practical reasons for being interested in indicator-based selection. The main one is that MOEAs such as SMS-EMOA seem to continue working as usual as we increase the number of objectives, as opposed to Pareto-based selection mechanisms which are known to degrade quickly in problems having more than 3 objectives (this research area is known as *many-objective optimization*). Although the reasons for the poor scalability of Pareto-based MOEAs requires further study (see, for example, [161]), the need for scalable selection mechanisms has triggered an important amount of research around indicator-based MOEAs. The main drawback of adopting the hypervolume in the selection mechanism of a MOEA is its extremely high computational cost. One possible alternative for dealing with this high computational cost is to estimate the hypervolume contribution. However, MOEAs designed around this idea (e.g., HyPE [7]) seem to have a poor performance with respect to those that adopt exact hypervolume contributions. An alternative is to rely on other performance indicators. In this regard, several researchers have proposed the design of selection mechanisms based on performance measures such as Δ_p [149,160], and $R2$ [15,78,83,181]. The use of the maximin [8, 124, 127] is another intriguing alternative, as this expression seems to be equivalent to the use of the ϵ-indicator [207].

Use of Other Metaheuristics

A wide variety of other bio-inspired metaheuristics have become popular in the last few years for solving optimization problems [26]. Multiobjective extensions of many of these metaheuristics already exist [24], but few efforts have been made to actually exploit the main specific features of each of them. There are also few efforts in trying to understand the types of problems in which each of these metaheuristics can be more suitable.

Next, we briefly review three popular bio-inspired metaheuristics that are good candidates for being used as multiobjective optimizers, but many other choices also exist (see, for example, [24]).

Artificial Immune Systems (AIS)

Our natural immune system has provided a fascinating metaphor for developing a new bio-inspired metaheuristic. Indeed, from a computational point of view, our immune system can be considered as a highly parallel intelligent system that is able to learn and retrieve previously acquired knowledge (i.e., it has "memory"), when solving highly complex recognition and classification tasks. This motivated the development of the so-called artificial immune systems (AISs) during the early 1990s [35, 132].

AISs were identified in the early 1990s as a useful mechanism to maintain diversity in the context of multimodal optimization [62, 168]. Smith et al. [169] showed that fitness sharing can emerge when their emulation of the immune system is used. Furthermore, the approach that they proposed turns out to be more efficient (computationally speaking) than traditional fitness sharing [39], and it does not require additional information regarding the number of niches to be formed.

Over the years, a wide variety of multiobjective extensions of AISs have been proposed (see, for example, [23, 115, 116, 118, 140, 197]). However, most of the algorithmic trends in MOEAs have had a delayed arrival in multiobjective AISs. Also, the high potential of multiobjective AISs for pattern recognition and classification tasks has been scarcely exploited.

For more information on multiobjective AISs, the interested reader is referred to [16, 63].

Particle Swarm Optimization (PSO)

This metaheuristic is inspired on the movements of a flock of birds seeking food, and it was originally proposed in the mid-1990s [101]. In the particle swarm optimization algorithm, the behavior of each particle (i.e., individual) is affected by either the best local (within a certain neighborhood) or the best global (i.e., with respect to the entire swarm, or population) individual. PSO allows particles to benefit from their past experiences (a mechanism that does not exist in traditional evolutionary algorithms) and uses neighborhood structures that can regulate the behavior of the algorithm.

The similarity of PSO with EAs has made possible the quick development of an important number of multiobjective variants of this metaheuristic (see, for example, [106, 136, 146, 166]). Unlike AISs, most algorithmic trends adopted with MOEAs have quickly been incorporated into multiobjective particle swarm optimizers (MOPSOs). Nevertheless, few PSO models have been used for multiobjective optimization, and the study of the specific features that could make MOPSOs advantageous over other metaheuristics in some specific classes of problems is still a pending task.

For more information on MOPSOs, the interested reader is referred to [9, 22, 24, 48].

Ant Colony Optimization (ACO)

This metaheuristic was inspired on the behavior observed in colonies of real ants seeking for food. Ants deposit a chemical substance on the ground, called pheromone [45], which influences the behavior of the ants: They tend to take those paths in which there is a larger amount of pheromone. Therefore, pheromone trails can be seen as an indirect communication mechanism used by ants (which can be seen as agents that interact to solve complex tasks). This interesting behavior of ants gave rise to a metaheuristic called *ant system*, which was originally applied to the travelling salesperson problem. Nowadays, the several variations of this algorithm that have been developed over the years are collectively denominated *ant colony optimization* (ACO), and they have been applied to a wide variety of domains, including continuous optimization problems.

Several multiobjective versions of ACO are currently available (see, for example, [54, 147, 183, 203]). However, the algorithmic trends developed in MOEAs have had a slow delay in being incorporated into multiobjective ACO algorithms. Additionally, the use of alternative ACO models for multiobjective optimization has been relatively scarce.

For more information on multiobjective ACO, the interested reader is referred to [4, 24, 68].

Other Metaheuristics

Many other metaheuristics have been extended to deal with multiobjective optimization problems, including simulated annealing [77, 82, 165], differential evolution [47, 110, 191], tabu search [2, 36, 204], scatter search [85, 109, 155], and artificial bee colony [153, 193, 194], among many others. However, their discussion was omitted due to obvious space constraints.

Some Applications

Multiobjective metaheuristics have been extensively applied to a wide variety of domains. Next, we will provide a short list of sample applications classified in three large groups: (1) engineering, (2) industrial, and (3) scientific. Specific areas within each of these large groups are also identified.

By far, engineering applications are the most popular in the current literature on multiobjective metaheuristics. This is not surprising if we consider that engineering disciplines normally have problems with better understood mathematical models. A representative sample of engineering applications is the following:

- Electrical engineering [33, 96]
- Hydraulic engineering [64, 195]

- Structural engineering [94, 200]
- Aeronautical engineering [142, 173]
- Robotics [99, 196]
- Control [156, 172]
- Telecommunications [53, 170]
- Civil engineering [119, 126]
- Transport engineering [17, 174]

Industrial applications are the second most popular in the literature on multiobjective metaheuristics. A representative sample of industrial applications of multiobjective metaheuristics is the following:

- Design and manufacture [3, 74]
- Scheduling [31, 97]
- Management [6, 66]

Finally, there are several publications devoted to scientific applications. For obvious reasons, computer science applications are the most popular in the literature on multiobjective metaheuristics. A representative sample of scientific applications is the following:

- Chemistry [56, 144]
- Physics [154, 185]
- Medicine [110, 156]
- Computer science [151, 196]

This sample of applications should give at least a rough idea of the increasing interest of researchers for adopting multiobjective metaheuristics in practically all kinds of disciplines.

Some Current Challenges

The existence of challenging, but solvable problems is a key issue to preserve the interest in a research discipline. Although multiobjective optimization using metaheuristics is a discipline in which a very important amount of research has been conducted, mainly within the last fifteen years, several interesting problems still remain open. Additionally, the research conducted so far has also led to new and intriguing topics. The following is a small sample of open problems that currently attract a significant amount of research within this area:

- **Large-scale problems**: Although scalability in objective function space (the so-called many-objective problems [131, 184]) has been studied for more than 15 years, large-scale multiobjective optimization problems (i.e., problems having 100 or more decision variables) is a more recent research area (see, for example,

[87, 123, 176]). Initially, most of the proposed approaches relied on cooperative co-evolution [5] or on the analysis of the decision variables [18]. However, over the years, the use of approaches based on dimensionality reduction (this is also called problem reformulation) [81] and specialized mechanisms to generate offspring [80] became popular. Recent approaches contain mechanisms that allow solving problems with over one million decision variables, relying on clever offspring generation mechanisms (see [113]).

- **Incorporation of user's preferences**: It is normally the case that the user does not need the entire Pareto front of a problem, but only a certain portion of it. For example, solutions lying at the extreme parts of the Pareto front are normally unnecessary since they represent the best value for one objective, but the worst for the others. Thus, if the user has at least a rough idea of the sort of trade-offs that he/she aims to find, it is desirable to be able to explore in more detail only the nondominated solutions within the neighborhood of such trade-offs. This is possible, if we use, for example, biased versions of Pareto ranking [32] or some multi-criteria decision-making technique, from the many developed in Operations Research [13, 21, 134]. In spite of the importance of preference incorporation in real-world applications, the use of these schemes in multiobjective metaheuristics is still relatively scarce [112, 148].
- **Dealing with expensive problems**: There is an important number of real-world multiobjective problems for which a single evaluation of an objective function may take several minutes, hours, or even days [142]. Evidently, such problems require special techniques to be solvable using MOEAs [79]. The main approaches that have been developed in this area can be roughly divided into three main groups:

  1. **Use of parallelism**: This is the most obvious approach given the current access to low-cost parallel architectures (e.g., GPUs [92, 167]). It is worth noting, however, that in spite of the existence of interesting proposals (see, for example, [55]), the basic research in this area has remained scarce, since most publications involving parallel MOEAs focus on specific applications or on parallel extensions of specific MOEAs, but rarely involves basic research.
  2. **Surrogates**: In this case, knowledge of past evaluations of a MOEA is used to build an empirical model that approximates the fitness functions to be optimized. This approximation can then be used to predict promising new solutions at a lower evaluation cost than that of the original problem [122, 143]. Current functional approximation models include polynomials (response surface methodologies [164]), neural networks (e.g., multilayer perceptrons (MLPs) [136]), radial-basis function (RBF) networks [37], support vector machines (SVMs) [25], Gaussian processes [192], and Kriging [143] models. The use of statistical models developed in the mathematical programming literature in combination with MOEAs is also an interesting alternative (see, for example, the multiobjective P-algorithm [135, 188]).
  3. **Fitness approximation**: The idea of these approaches is to predict the fitness value of an individual without performing its actual evaluation [65, 141]. From

the several techniques available [20], fitness inheritance has been one of the most popular (in relative terms) in multiobjective optimization [145, 190] but the research in this area has been very scarce in recent years. However, other ideas, such as the use of transformation methods that allow to approximate the fitness landscape (see, for example, [19]) or incremental learning [120], are certainly worth pursuing.

- **Theoretical foundations**: Although an important effort has been made in recent years to provide a solid theoretical foundation to this field, a lot of work is still required and the work done in the Operations Research community can provide some useful hints (see, for example, [135, 187]). The most relevant theoretical work in this area includes topics such as convergence [186], archiving [1, 159], algorithm complexity [133], and runtime analysis [44, 108].
- **Constraint handling**: There has been a lot of research on constraint-handling techniques for MOEAs in recent years (see, for example, [29, 67, 98, 130]). However, there has been some criticism to state-of-the-art constraint-handling techniques developed for MOEAs, because there is evidence indicating that they cannot properly solve some real-world problems (e.g., in mechanical design [139]) in spite of the fact that such methods have a good performance in the currently available benchmarks. This has triggered an interesting discussion, since it is evident that the current benchmarks for constrained MOEAs are fairly limited. For example, these test problems are not properly characterized and, in general, the only information available about them is the number of decision variables, the number of constraints, the true Pareto front, and the feasibility rate. However, there are studies available which indicate that this information is insufficient [175].
- **Evolutionary multitasking**: Gupta et al. [75] proposed the so-called evolutionary multitasking as a new paradigm in evolutionary optimization in which the population of a sequential evolutionary algorithm is used to solve multiple optimization problems simultaneously. This idea is inspired on bio-cultural models of multifactorial inheritance. When introduced, its authors also proposed a cross-domain optimization platform that allows to solve diverse problems concurrently, and by allowing knowledge transfer, its use can speed up convergence in complex optimization problems. Multitasking has become relatively popular in multiobjective optimization in recent years (see, for example, [12, 117, 199]) and it is indeed a very promising research area.

Conclusions

In this chapter, we have provided some basic concepts related to multiobjective optimization using metaheuristics. This overview has included basic concepts related to multiobjective optimization in general, as well as some algorithmic details, with a particular emphasis on multiobjective evolutionary algorithms.

Some of the recent algorithmic trends have also been discussed, and some sample applications have been addressed. In general, breadth has been favored over depth, but a significant number of bibliographic references are provided for those interested in gaining an in-depth knowledge of any of the topics discussed herein.

The information provided in this chapter aims to serve as a general overview of the field and to motivate the interest of the reader for pursuing research in this area. As could be seen, several research opportunities are still available for newcomers.

Cross-References

- ▸ Evolutionary Algorithms
- ▸ Genetic Algorithms
- ▸ Time Complexity Analysis of Stochastic Search Algorithms

References

1. Hernández Castellanos CI, Schütze O (2022) A bounded archiver for Hausdorff approximations of the Pareto front for multi-objective evolutionary algorithms. Math Comput Appl 27(3). Article Number: 48
2. Abedi M, Chiong R, Athauda R, Seidgar H, Michalewicz Z, Sturt A (2019) A regional multi-objective tabu search algorithm for a green heterogeneous dial-a-ride problem. In: 2019 IEEE congress on evolutionary computation (CEC'2019), Wellington, 10 June–13 June 2019, pp 2082–2089. IEEE Press. ISBN 978-1-7281-2153-6
3. Ahn G, Hur S (2021) Multiobjective real-time scheduling of tasks in cloud manufacturing with genetic algorithm. Math Problems Eng. Article ID: 1305849
4. Angus D, Woodward C (2009) Multiple objective ant colony optimisation. Swarm Intell 3(1):69–85
5. Antonio LM, Coello Coello CA (2013) Use of cooperative coevolution for solving large scale multiobjective optimization problems. In: 2013 IEEE congress on evolutionary computation (CEC'2013), Cancún, 20–23 June 2013, pp 2758–2765. IEEE Press. ISBN 978-1-4799-0454-9
6. Awad M, Abouhawwash M, Agiza HN (2022) On NSGA-II and NSGA-III in portfolio management. Intell Autom Soft Comput 32(3):1893–1904
7. Bader J, Zitzler E (2011) HypE: an algorithm for fast hypervolume-based many-objective optimization. Evol Comput 19(1):45–76
8. Balling R, Wilson S (2001) The maximin fitness function for multi-objective evolutionary computation: application to city planning. In: Spector L, Goodman ED, Wu A, Langdon WB, Voigt H-M, Gen M, Sen S, Dorigo M, Pezeshk S, Garzon MH, Burke E (eds) Proceedings of the genetic and evolutionary computation conference (GECCO'2001), San Francisco, pp 1079–1084. Morgan Kaufmann Publishers
9. Banks A, Vincent J, Anyakoha C (2008) A review of particle swarm optimization. II: Hybridisation, combinatorial, multicriteria and constrained optimization, and indicative applications. Nat Comput Unconvent Comput 2006, Sel Pap 7(1):109–124
10. Bechikh S, Datta R, Gupta A (eds) (2017) Recent advances in evolutionary multi-objective optimization. Springer, Cham. ISBN 978-3-319-42977-9
11. Beume N, Naujoks B, Emmerich M (2007) SMS-EMOA: multiobjective selection based on dominated hypervolume. Eur J Oper Res 181(3):1653–1669

12. Bi Y, Xue B, Zhang M (2022) Learning and sharing: a multitask genetic programming approach to image feature learning. IEEE Trans Evol Comput 26(2):218–232
13. Branke J (2008) Consideration of partial user preferences in evolutionary multiobjective optimization. In: Branke J, Deb K, Miettinen K, Slowinski R (eds) Multiobjective optimization. Interactive and evolutionary approaches. Lecture notes in computer science, vol 5252. Springer, Berlin, pp 157–178
14. Brockhoff D, Emmerich M, Naujoks B, Purshouse R (eds) (2023) Many-criteria optimization and decision analysis. State-of-the-art, present challenges, and future perspectives. Springer, Cham. ISBN 978-3-031-25262-4
15. Brockhoff D, Wagner T, Trautmann H (2012) On the properties of the $R2$ indicator. In: 2012 genetic and evolutionary computation conference (GECCO'2012), Philadelphia, July 2012, pp 465–472. ACM Press. ISBN: 978-1-4503-1177-9
16. Campelo F, Guimar aes FG, Igarashi H (2007) Overview of artificial immune systems for multi-objective optimization. In: Obayashi S, Deb K, Poloni C, Hiroyasu T, Murata T (eds) Evolutionary multi-criterion optimization, 4th international conference, EMO 2007, Matshushima, Mar 2007. Lecture notes in computer science vol. 4403, pp 937–951. Springer
17. Chai R, Tsourdos A, Savvaris A, Chai S, Xia Y, Philip Chen CL (2021) Multiobjective overtaking maneuver planning for autonomous ground vehicles. IEE Trans Cybern 51(8):4035–4049
18. Chen H, Cheng R, Wen J, Li H, Weng J (2020) Solving large-scale many-objective optimization problems by covariance matrix adaptation evolution strategy with scalable small subpopulations. Inf Sci 509:457–469
19. Cheng R, Li M, Li K, Yao X (2018) Evolutionary multiobjective optimization-based multimodal optimization: fitness landscape approximation and peak detection. IEEE Trans Evol Comput 22(5):692–706
20. Chugh T, Sindhya K, Hakanen J, Miettinen K (2019) A survey on handling computationally expensive multiobjective optimization problems with evolutionary algorithms. Soft Comput 23(9):3137–3166
21. Coello Coello CA (2000) Constraint-handling using an evolutionary multiobjective optimization technique. Civil Eng Environ Syst 17:319–346
22. Coello Coello CA (2011) An introduction to multi-objective particle swarm optimizers. In: Gaspar-Cunha A, Takahashi R, Schaefer G, Costa L (eds) Soft computing in industrial applications. Advances in intelligent and soft computing series, vol 96, pp 3–12. Springer, Berlin. ISBN 978-3-642-20504-0
23. Coello Coello CA, Cruz Cortés N (2005) Solving multiobjective optimization problems using an artificial immune system. Gen Program Evol Mach 6(2):163–190
24. Coello Coello CA, Lamont GB, Van Veldhuizen DA (2007) Evolutionary algorithms for solving multi-objective problems, 2nd edn. Springer, New York. ISBN 978-0-387-33254-3
25. Coleto-Alcudia V, Vega-Rodriguez MA (2022) A multi-objective optimization approach for the identification of cancer biomarkers from RNA-seq data. Expert Syst Appl 193. Article Number: 116480
26. Corne D, Glover F, Dorigo M (eds) (1999) New ideas in optimization. McGraw-Hill, Berkshire. ISBN 007-709506-5
27. Corne DW, Jerram NR, Knowles JD, Oates MJ (2001) PESA-II: region-based selection in evolutionary multiobjective optimization. In: Spector L, Goodman ED, Wu A, Langdon WB, Voigt H-M, Gen M, Sen S, Dorigo M, Pezeshk S, Garzon MH, Burke E (eds) Proceedings of the genetic and evolutionary computation conference (GECCO'2001), San Francisco. Morgan Kaufmann Publishers, pp 283–290
28. Corne DW, Knowles JD, Oates MJ (2000) The Pareto envelope-based selection algorithm for multiobjective optimization. In: Schoenauer M, Deb K, Rudolph G, Yao X, Lutton E, Merelo JJ, Schwefel H-P (eds) Proceedings of the parallel problem solving from nature VI conference, Paris. Lecture notes in computer science, No. 1917. Springer, pp 839–848
29. Cuate O, Uribe L, Lara A, Schutze O (2020) A benchmark for equality constrained multiobjective optimization. Swarm Evol Comput 52. Article Number: UNSP 100619

30. Cui X, Li M, Fang T (2001) Study of population diversity of multiobjective evolutionary algorithm based on immune and entropy principles. In: Proceedings of the congress on evolutionary computation 2001 (CEC'2001), Piscataway, May 2001, vol 2. IEEE Service Center, pp 1316–1321
31. Cui Z, Zhao T, Wu L, Qin AK, Li J (2023) Multi-objective cloud task scheduling optimization based on evolutionary multi-factor algorithm. IEEE Trans Cloud Comput 11(4):3685–3699
32. Cvetković D, Parmee IC (2002) Preferences and their application in evolutionary multiobjective optimisation. IEEE Trans Evol Comput 6(1):42–57
33. da Rocha SP, Bambirra Pereira R, Machado-Coelho TM, lakovlevitch Ekel P, Luís Soares G (2018) Two-stage optimization combining PSO and TOPSIS for allocation of energy storage in electric power systems. In: 2018 IEEE congress on evolutionary computation (CEC'2018), Rio de Janeiro, 8–13 July 2018. IEEE Press, pp 1115–1122
34. Das I, Dennis J (1997) A closer look at drawbacks of minimizing weighted sums of objectives for Pareto set generation in multicriteria optimization problems. Struct Optim 14(1):63–69
35. Dasgupta D (ed) (1999) Artificial immune systems and their applications. Springer, Berlin
36. de Moraes MB, Coelho GP (2022) A diversity preservation method for expensive multiobjective combinatorial optimization problems using novel-first tabu search and MOEA/D. Expert Syst Appl 202. Article Number: 117251
37. de Winter R, van Stein B, Bäck T (2021) SAMO-COBRA: a fast surrogate assisted constrained multi-objective optimization algorithm. In: Ishibuchi H, Zhang Q, Cheng R, Li K, Li H, Wang H, Zhou A (eds) Evolutionary multi-criterion optimization, 11th international conference, EMO 2021. Lecture notes in computer science, Shenzhen, 28 Mar–31 Mar 2021, vol 12654. Springer, pp 270–282. ISBN 978-3-030-72061-2
38. Deb K (2001) Multi-objective optimization using evolutionary algorithms. Wiley, Chichester. ISBN 0-471-87339-X
39. Deb K, Goldberg DE (1989) An investigation of Niche and Species formation in genetic function optimization. In: David Schaffer J (ed) Proceedings of the third international conference on genetic algorithms, San Mateo, June 1989. George Mason University, Morgan Kaufmann Publishers, pp 42–50
40. Deb K, Jain H (2014) An evolutionary many-objective optimization algorithm using reference-point-based nondominated sorting approach, part I: solving problems with box constraints. IEEE Trans Evol Comput 18(4):577–601
41. Deb K, Mohan M, Mishra S (2005) Evaluating the ϵ-domination based multi-objective evolutionary algorithm for a quick computation of Pareto-optimal solutions. Evol Comput 13(4):501–525
42. Deb K, Pratap A, Agarwal S, Meyarivan T (2002) A fast and elitist multiobjective genetic algorithm: NSGA–II. IEEE Trans Evol Comput 6(2):182–197
43. Dey N (ed) (2023) Applied genetic algorithm and its variants. Case studies and new developments. Springer, Singapore. ISBN 978-981-99-3427-0
44. Doerr B, Qu Z (2022) A first runtime analysis of the NSGA-II on a multimodal problem. In: Rudolph G, Kononova AV, Aguirre H, Kerschke P, Ochoa G, Tušar T (eds) Parallel problem solving from nature – PPSN XVII, 17th international conference, PPSN 2022, proceedings, part II, Dortmund, 10 Sep–14 Sep 2022. Lecture notes in computer science, vol. 13399. Springer, pp 399–412. ISBN 978-3-031-14720-3
45. Dorigo M, Stützle T (2004) Ant colony optimization. The MIT Press. ISBN 0-262-04219-3
46. Drozdik M, Akimoto Y, Aguirre H, Tanaka K (2015) Computational cost reduction of nondominated sorting using the M-front. IEEE Trans Evol Comput 19(5):659–678
47. Duman S, Akbel M, Kahraman HT (2021) Development of the multi-objective adaptive guided differential evolution and optimization of the MO-ACOPF for wind/PV/tidal energy sources. Appl Soft Comput 112. Article Number: 107814
48. Durillo JJ, García-Nieto J, Nebro AJ, Coello Coello CA, Luna F, Alba E (2009) Multi-objective particle swarm optimizers: an experimental comparison. In: Ehrgott M, Fonseca CM, Gandibleux X, Hao J-K, Sevaux M (eds) Evolutionary multi-criterion optimization. 5th

international conference, EMO 2009, Nantes, Apr 2009. Lecture notes in computer science, vol. 5467. Springer, pp 495–509
49. Eddaly M, Jarboui B, Siarry P (eds) (2023) Metaheuristics for machine learning. New advances and tools. Springer, Singapore. ISBN 978-981-19-3890-0
50. Edgeworth FY (1881) Mathematical psychics. P. Keagan, London
51. Eiben AE, Smith JE (2003) Introduction to evolutionary computing. Springer, Berlin. ISBN 3-540-40184-9
52. Emmerich M, Beume N, Naujoks B (2005) An EMO algorithm using the hypervolume measure as selection criterion. In: Coello Coello CA, Hernández Aguirre A, Zitzler E (eds) Evolutionary multi-criterion optimization. third international conference, EMO 2005, Guanajuato, Mar 2005. Lecture notes in computer science, vol 3410. Springer, pp 62–76
53. Espinosa R, Jiménez F, Palma J (2022) Multiple objective optimisation for antenna diversity on airborne platforms. Futur Gen Comput Syst-Int J Sci 136:15–33
54. Falcon-Cardona JG, Coello Coello CA (2017) A new indicator-based many-objective ant colony optimizer for continuous search spaces. Swarm Intell 11(1):71–100
55. Falcon-Cardona JG, Gomez RH, Coello Coello CA, Castillo Tapia MG (2021) Parallel multi-objective evolutionary algorithms: a comprehensive survey. Swarm Evol Comput 67. Article Number: 100960
56. Fan Q, Wang W, Yan X (2017) Multi-objective differential evolution with performance-metric-based self-adaptive mutation operator for chemical and qbiochemical dynamic optimization problems. Appl Soft Comput 59:33–44
57. Fleischer M (2003) The measure of Pareto optima. Applications to multi-objective metaheuristics. In: Fonseca CM, Fleming PJ, Zitzler E, Deb K, Thiele L (eds) Evolutionary multi-criterion optimization. Second international conference, EMO 2003, Faro, Apr 2003. Lecture notes in computer science, vol 2632. Springer, pp 519–533
58. Fogel DB (1995) Evolutionary computation. Toward a new philosophy of machine intelligence. The Institute of Electrical and Electronic Engineers, New York
59. Fogel LJ (1966) Artificial intelligence through simulated evolution. Wiley, New York
60. Fogel LJ (1999) Artificial intelligence through simulated evolution. Forty years of evolutionary programming. Wiley, New York
61. Fonseca CM, Fleming PJ (1993) Genetic algorithms for multiobjective optimization: formulation, discussion and generalization. In: Forrest S (ed) Proceedings of the fifth international conference on genetic algorithms, San Mateo. University of Illinois at Urbana-Champaign, Morgan Kauffman Publishers, pp 416–423
62. Forrest S, Perelson AS (1991) Genetic algorithms and the immune system. In: Schwefel H-P, Männer R (eds) Parallel problem solving from nature. Lecture notes in computer science. Springer, Berlin, pp 320–325
63. Freschi F, Coello Coello CA, Repetto M (2009) Multiobjective optimization and artificial immune systems: a review. In: Mo H (ed) Handbook of research on artificial immune systems and natural computing: applying complex adaptive technologies. Medical information science reference. Hershey, New York, pp 1–21. ISBN 978-1-60566-310-4
64. Fu W, Lu QP (2020) Multiobjective optimal control of FOPID controller for hydraulic turbine governing systems based on reinforced multiobjective Harris Hawks optimization coupling with hybrid strategies. Complexity 2020. Article Number: 9274980
65. Gao G, Qiao L, Liu D, Chen S, Jiang H (2022) Surrogate-assisted multi-objective optimization for compiler optimization sequence selection. In: Rudolph G, Kononova AV, Aguirre H, Kerschke P, Ochoa G, Tušar T (eds) Parallel problem solving from nature – PPSN XVII, 17th international conference, PPSN 2022, proceedings, part II, Dortmund, 10 Sep–14 Sep 2022. Lecture notes in computer science, vol 13399. Springer, pp 382–395. ISBN 978-3-031-14720-3
66. Gao R, Tao J, Zhang J, Ma L, Xu M (2023) NSGA-III-SD based fuzzy energy management system optimization for lithium battery/supercapacitor HEV. Appl Soft Comput 142. Article Number: 110280

67. Llano Garcia JL, Monroy R, Hernandez VAS, Coello Coello CA (2021) COARSE-EMOA: an indicator-based evolutionary algorithm for solving equality constrained multi-objective optimization problems. Swarm Evol Comput 67. Article Number: 100983
68. García-Martínez C, Cordón O, Herrera F (2007) A taxonomy and an empirical analysis of multiple objective ant colony optimization algorithms for the bi-criteria TSP. Eur J Oper Res 180(1):116–148
69. Gendreau M, Potvin J-Y (eds) (2019) Handbook of metaheuristics. Springer, Cham. ISBN 978-3-319-91085-7
70. Goldberg DE (1989) Genetic algorithms in search, optimization and machine learning. Addison-Wesley Publishing Company, Reading
71. Goldberg, DE, Deb K (1991) A comparison of selection schemes used in genetic algorithms. In: Rawlins GJE (ed) Foundations of genetic algorithms. Morgan Kaufmann, San Mateo pp 69–93
72. Goldberg DE, Richardson J (1987) Genetic algorithm with sharing for multimodal function optimization. In: Grefenstette JJ (ed) Genetic algorithms and their applications: proceedings of the second international conference on genetic algorithms, Hillsdale. Lawrence Erlbaum, pp 41–49
73. Greco S, Pavone MF, Talbi E-G, Vigo D (eds) (2021) Metaheuristics for combinatorial optimization. Springer, Cham. ISBN 978-3-030-68519-5
74. Guo S, Dong M (2020) Order matching mechanism of the production intermediation internet platform between retailers and manufacturers. Int J Adv Manuf Technol 115(3):949–962
75. Gupta A, Ong Y-S, Feng L (2016) Multifactorial evolution: toward evolutionary multitasking. IEEE Trans Evol Comput 20(3):343–357
76. Hajela P, Lin CY (1992) Genetic search strategies in multicriterion optimal design. Struct Optim 4:99–107
77. Han X, Dong Y, Yue L, Xu Q, Xie G, Xu X (2021) State-transition simulated annealing algorithm for constrained and unconstrained multi-objective optimization problems. Appl Intell 51(2):775–787
78. Hansen MP (1998) Metaheuristics for multiple objective combinatorial optimization. PhD thesis, Institute of Mathematical Modelling, Technical University of Denmark, Mar 1998
79. Hao H, Zhou A, Qian H, Zhang H (2022) Expensive multiobjective optimization by relation learning and prediction. IEEE Trans Evol Comput 26(5):1157–1170
80. He C, Cheng R, Yazdani D (2022) Adaptive offspring generation for evolutionary large-scale multiobjective optimization. IEEE Trans Syst Man Cybern Syst 52(2):786–798
81. He C, Li L, Tian Y, Zhang X, Cheng R, Jin Y, Yao X (2019) Accelerating large-scale multiobjective optimization via problem reformulation. IEEE Trans Evol Comput 23(6): 949–961
82. Hemici M, Zouache D, Brahmi B, Got A, Drias H (2023) A decomposition-based multiobjective evolutionary algorithm using simulated annealing for the ambulance dispatching and relocation problem during COVID-19. Appl Soft Comput 141. Article Number: 110282
83. Hernández Gómez R, Coello Coello CA (2013) MOMBI: a new metaheuristic for many-objective optimization based on the $R2$ indicator. In: 2013 IEEE congress on evolutionary computation (CEC'2013), Cancún, 20–23 June 2013. IEEE Press, pp 2488–2495. ISBN 978-1-4799-0454-9
84. Hernández Gómez R, Coello Coello CA, Alba Torres E (2016) A multi-objective evolutionary algorithm based on parallel coordinates. In: 2016 genetic and evolutionary computation conference (GECCO'2016), Denver, 20–24 July 2016. ACM Press, pp 565–572. ISBN 978-1-4503-4206-3
85. Hernández-Ramírez L, Frausto-Solís J, Castilla-Valdez G, González-Barbosa J, Sánchez Hernández J-P (2022) Three hybrid scatter search algorithms for multi-objective job shop scheduling problem. Axioms 11(2). Article Number: 61
86. Holland JH (1962) Concerning efficient adaptive systems. In: Yovits MC, Jacobi GT, Goldstein GD (eds) Self-organizing systems—1962. Spartan Books, Washington, DC, pp 215–230

87. Hong W-J, Yang P, Tang K (2021) Evolutionary computation for large-scale multi-objective optimization: a decade of progresses. Int J Autom Comput 18(2):155–169
88. Horn J (1997) Multicriterion decision making. In: Bäck T, Fogel D, Michalewicz Z (eds) Handbook of evolutionary computation, vol 1. IOP Publishing Ltd./Oxford University Press, Boca Raton, Florida, USA, pp F1.9:1–F1.9:15
89. Horn J, Nafpliotis N, Goldberg DE (1994) A Niched Pareto genetic algorithm for multiobjective optimization. In: Proceedings of the first IEEE conference on evolutionary computation, IEEE world congress on computational intelligence, Piscataway, vol 1, June 1994. IEEE Service Center, pp 82–87
90. Huband S, Hingston P, White L, Barone L (2003) An evolution strategy with probabilistic mutation for multi-objective optimisation. In: Proceedings of the 2003 congress on evolutionary computation (CEC'2003), Canberra, Dec 2003, vol 3. IEEE Press, pp 2284–2291
91. Husbands P (1994) Distributed coevolutionary genetic algorithms for multi-criteria and multi-constraint optimisation. In: Fogarty TC (ed) Evolutionary computing. AIS workshop. Selected papers. Lecture notes in computer science, vol 865. Springer, pp 150–165
92. Iruela JRS, Ruiz LGB, Pegelajar MC, Capel MI (2020) A parallel solution with GPU technology to predict energy consumption in spatially distributed buildings using evolutionary optimization and artificial neural networks. Energy Conversion Manag 207. Article Number: 112535
93. Ishibuchi H, Shibata Y (2003) An empirical study on the effect of mating restriction on the search ability of EMO algorithms. In: Fonseca CM, Fleming PJ, Zitzler E, Deb K, Thiele L (eds) Evolutionary multi-criterion optimization. Second international conference, EMO 2003, Faro, Apr 2003. Lecture notes in computer science, vol 2632. Springer, pp 433–447
94. Jakirul Islam M, Li X, Deb K (2017) Multimodal truss structure design using bilevel and niching based evolutionary algorithms. In: 2017 genetic and evolutionary computation conference (GECCO'2017), Berlin, 15–19 July 2017. ACM Press, pp 274–281. ISBN 978-1-4503-4920-8
95. Jensen MT (2003) Reducing the run-time complexity of multiobjective EAs: the NSGA-II and other algorithms. IEEE Trans Evol Comput 7(5):503–515
96. Jensen WR, Pham TQ, Foster SN (2019) Comparison of multi-objective optimization methods applied to electrical machine design. In: Deb K, Goodman E, Coello Coello CA, Klamroth K, Miettinen K, Mostaghim S, Reed P (eds) Evolutionary multi-criterion optimization, 10th international conference, EMO 2019, East Lansing. Lecture notes in computer science, vol 11411, 10–13 Mar 2019. Springer, pp 719–730. ISBN 978-3-030-12597-4
97. Jiang X, Zhong M, Shi J, Li W, Sui Y, Dou Y (2022) Overall scheduling model for vessels scheduling and berth allocation for ports with restricted channels that considers carbon emissions. J Marine Sci Eng 10(11). Article Number: 1757
98. Jiao R, Zeng S, Li C, Yang S, Ong Y-S (2021) Handling constrained many-objective optimization. Problems via problem transformation. IEEE Trans Cybern 51(10):4834–4847
99. Jun Z, Ning Z, Tian L, Zhou Z, Wang P (2023) Robots' picking efficiency and pickers' energy expenditure: the item storage assignment policy in robotic mobile fulfillment system. Comput Ind Eng 176. Article Number: 108918
100. Kaveh A, Hamedani KB (2022) Advanced metaheuristic algorithms and their applications in structural optimization. Springer, Cham. ISBN 978-3-031-13428-9
101. Kennedy J, Eberhart RC (2001) Swarm intelligence. Morgan Kaufmann Publishers, San Francisco
102. Kita H, Yabumoto Y, Mori N, Nishikawa Y (1996) Multi-objective optimization by means of the thermodynamical genetic algorithm. In: Voigt H-M, Ebeling W, Rechenberg I, Schwefel H-P (eds) Parallel problem solving from nature—PPSN IV, Berlin, Sep 1996. Lecture notes in computer science. Springer, pp 504–512
103. Knowles J, Corne D (2003) Properties of an adaptive archiving algorithm for storing nondominated vectors. IEEE Trans Evol Comput 7(2):100–116

104. Knowles J, Corne D (2004) Bounded Pareto archiving: theory and practice. In: Gandibleux X, Sevaux M, Sörensen K, T'kindt V (eds) Metaheuristics for multiobjective optimisation. Lecture notes in economics and mathematical systems, vol 535. Springer, Berlin, pp 39–64
105. Knowles JD, Corne DW (2000) Approximating the nondominated front using the Pareto archived evolution strategy. Evol Comput 8(2):149–172
106. Kouka N, BenSaid F, Fdhila R, Fourati R, Hussain A, Alimi AM (2023) A novel approach of many-objective particle swarm optimization with cooperative agents based on an inverted generational distance indicator. Inf Sci 623:220–241
107. Laumanns M, Thiele L, Deb K, Zitzler E (2002) Combining convergence and diversity in evolutionary multi-objective optimization. Evol Comput 10(3):263–282
108. Laumanns M, Thiele L, Zitzler E (2004) Running time analysis of multiobjective evolutionary algorithms on pseudo-boolean functions. IEEE Trans Evol Comput 8(2):170–182
109. Le Q, Yang G, Hung WNN, Zhang X, Fan F (2014) A multiobjective scatter search algorithm for fault-tolerant NoC mapping optimisation. Int J Electron 101(8):1056–1073
110. Li H, He F, Chen Y, Pan Y (2021) MLFS-CCDE: multi-objective large-scale feature selection by cooperative coevolutionary differential evolution. Memetic Comput 13(1):1–18
111. Li K, Deb K, Zhang Q, Zhang Q (2017) Efficient nondomination level update method for steady-state evolutionary multiobjective optimization. IEEE Trans Cybern 47(9):2838–2849
112. Li K, Liao M, Deb K, Min G, Yao X (2020) Does preference always help? A holistic study on preference-based evolutionary multiobjective optimization using reference points. IEEE Trans Evol Comput 24(6):1078–1096
113. Lianghao Li, He C, Cheng R, Li H, Pan L, Jin Y (2022) A fast sampling based evolutionary algorithm for million-dimensional multiobjective optimization. Swarm Evol Comput 75. Article Number: 101181
114. Li L, He C, Cheng R, Pan L (2021) Manifold learning inspired mating restriction for evolutionary constrained multiobjective optimization. In: Ishibuchi H, Zhang Q, Cheng R, Li K, Li H, Wang H, Zhou A (eds) Evolutionary multi-criterion optimization, 11th international conference, EMO 2021, Shenzhen, 28 Mar–31 Mar 2021. Lecture notes in computer science, vol 12654. Springer, pp 296–307. ISBN 978-3-030-72061-2
115. Li L, Lin Q, Li K, Ming Z (2021) Vertical distance-based clonal selection mechanism for the multiobjective immune algorithm. Swarm Evol Comput 63. Article Number: 100886
116. Li L, Lin Q, Ming Z, Wong K-C, Gong M, Coello Coello CA (2022) An immune-inspired resource allocation strategy for many-objective optimization. IEEE Trans Syst Man Cybern-Syst 53:3284–3297
117. Li L, Xuan M, Lin Q, Jiang M, Ming Z, Tan KC (2023) An evolutionary multitasking algorithm with multiple filtering for high-dimensional feature selection. IEEE Trans Evol Comput 27:802–816
118. Li W, Chai Z, Tang Z (2021) A decomposition-based multi-objective immune algorithm for feature selection in learning to rank. Knowl-Based Syst 234. Article Number: 107577
119. Liang C, Xu X, Chen H, Wang W, Zheng K, Tan G, Gu Z, Zhang H (2021) Machine learning approach to develop a novel multi-objective optimization method for pavement material proportion. Appl Sci-Basel 11(2). Article Number: 835
120. Lin J, Liu H-L, Xue B, Zhang M, Gu F (2020) Multiobjective multitasking optimization based on incremental learning. IEEE Trans Evol Comput 24(5):824–838
121. Lin J, Zhang SX, Zheng SY (2024) A diverse/converged individual competition algorithm for computationally expensive many-objective optimization. Appl Intell 54:2564–2581
122. Lin Q, Wu X, Ma L, Li J, Gong M, Coello Coello CA (2022) An ensemble surrogate-based framework for expensive multiobjective evolutionary optimization. IEEE Trans Evol Comput 26(4):631–645
123. Liu S, Lin Q, Wong K-C, Li Q, Kay Tan C (2023) Evolutionary large-scale multiobjective optimization: benchmarks and algorithms. IEEE Trans Evol Comput 27(3):401–415
124. Ma J, Yang S, Shi G, Ma L (2022) A multiplicative maximin-based evaluation approach for evolutionary many-objective optimization. Appl Soft Comput 121. Article Number: 108760

125. Ma X, Yu Y, Li X, Qi Y, Zhu Z (2020) A survey of weight vector adjustment methods for decomposition-based multiobjective evolutionary algorithms. IEEE Trans Evol Comput 24(4):634–649
126. Majedi H, Fathian H, Nikbakht-Shahbazi A, Zohrabi N, Hassani F (2021) Multi-objective optimization of integrated surface and groundwater resources under the clean development mechanism. Water Res Manag 35(8):2685–2704
127. Menchaca-Mendez A, Coello Coello CA (2013) Selection operators based on maximin fitness function for multi-objective evolutionary algorithms. In: Purshouse RC, Fleming PJ, Fonseca CM, Greco S, Shaw J (eds) Evolutionary multi-criterion optimization, 7th international conference, EMO 2013, Sheffield, 19–22 Mar 2013. Lecture notes in computer science, vol 7811. Springer, pp 215–229
128. Menchaca-Mendez A, Coello Coello CA (2016) Selection mechanisms based on the maximin fitness function to solve multi-objective optimization problems. Inf Sci 332:131–152
129. Miettinen KM (1999) Nonlinear multiobjective optimization. Kluwer Academic Publishers, Boston
130. Ming F, Gong W, Wang L, Gao L (2023) A constraint-handling technique for decomposition-based constrained many-objective evolutionary algorithms. IEEE Trans Syst Man Cybern Syst 53(12):7783–7793
131. Mohammed RT, Yaakob R, Zaidan AA, Sharef NM, Abdullah RH, Zaidan BB, Dawood KA (2020) Review of the research landscape of multi-criteria evaluation and benchmarking processes for many-objective optimization methods: coherent taxonomy, challenges and recommended solution. Int J Inf Technol Decis Making 19(6):1619–1693
132. de Castro LN, Timmis J (2002) An introduction to artificial immune systems: a new computational intelligence paradigm. Springer, London. ISBN 1-85233-594-7
133. Neumann F (2012) Computational complexity analysis of multi-objective genetic programming. In: 2012 Genetic and evolutionary computation conference (GECCO'2012), Philadelphia, July 2012. ACM Press, pp 799–806. ISBN: 978-1-4503-1177-9
134. Pardalos PM, Siskos Y, Zopounidis C (eds) (1995) Advances in multiciteria analysis. Springer-Science+Business Media, B.V. ISBN 978-1-4419-4748-2
135. Pardalos PM, Žilinskas A, Žilinskas J (2017) Non-convex multi-objective optimization. Springer. ISBN 978-3-319-61005-4
136. Pareek CM, Tewari VK, Machavaram R (2023) Multi-objective optimization of seeding performance of a pneumatic precision seed metering device using integrated ANN-MOPSO approach. Eng Appl Artif Intell 117(A). Article Number: 105559
137. Pareto V (1896) Cours D'Economie Politique, vols I and II. F. Rouge, Lausanne
138. Pescador-Rojas M, Hernández Gómez R, Montero E, Rojas-Morales N, Riff M-C, Coello Coello CA (2017) An overview of weighted and unconstrained scalarizing functions. In: Trautmann H, Rudolph G, Klamroth K, Schütze O, Wiecek M, Jin Y, Grimme C (eds) Evolutionary multi-criterion optimization, 9th international conference, EMO 2017, Münster, 19–22 Mar 2017. Lecture notes in computer science, vol 10173. Springer, pp 499–513. ISBN 978-3-319-54156-3
139. Picard C, Schiffmann J (2021) Realistic constrained multiobjective optimization benchmark problems from design. IEEE Trans Evol Comput 25(2):234–246
140. Pierrard T, Coello Coello CA (2012) A multi-objective artificial immune system based on hypervolume. In: Coelo Coello CA, Greensmith J, Krasnogor N, Liò P, Nicosia G, Pavone M (eds) Artificial immune systems, 11th international conference, ICARIS 2012, Taormina, 28–31 Aug 2012. Lecture notes in computer science, vol 7597. Springer, pp 14–27. ISBN 978-3-642-33756-7
141. Pourbahman Z, Hamzeh A (2015) A fuzzy based approach for fitness approximation in multi-objective evolutionary algorithms. J Intell Fuzzy Syst 29(5):2111–2131
142. Arias-Montaño A, Coello Coello CA, Mezura-Montes E (2012) Multi-objective evolutionary algorithms in aeronautical and aerospace engineering. IEEE Trans Evol Comput 16(5): 662–694

143. Qin S, Sun C, Akhtar F, Xie G (2024) Expensive many-objective evolutionary optimization guided by two individual infill criteria. Mem Comput 16:55–69
144. Rangaiah GP, Bonilla-Petriciolet A (eds) (2013) Multi-objective optimization in chemical engineering. Developments and applications. Wiley. ISBN 978-1-118-34166-7
145. Reyes Sierra M, Coello Coello CA (2005) Fitness inheritance in multi-objective particle swarm optimization. In: 2005 IEEE swarm intelligence symposium (SIS'05), Pasadena, June 2005. IEEE Press, pp 116–123
146. Rezaei F, Safavi HR (2020) f-MOPSO/Div: an improved extreme-point-based multi-objective PSO algorithm applied to a socio-economic-environmental conjunctive water use problem. Environ Monitor Assessment 192(12). Article Number: 767
147. Rivera G, Cruz-Reyes L, Fernandez E, Gomez-Santillan C, Rangel-Valdez N (2023) An interactive ACO enriched with an eclectic multi-criteria ordinal classifier to address many-objective optimisation problems. Expert Syst Appl 232. Article Number: 120813
148. Rivera G, Coello Coello CA, Cruz-Reyes L, Fernandez ER, Gomez-Santillan C, Rangel-Valdez N (2022) Preference incorporation into many-objective optimization: an ant colony algorithm based on interval outranking. Swarm Evol Comput 69. Article Number 101024
149. Rodríguez Villalobos CA, Coello Coello CA (2012) A new multi-objective evolutionary algorithm based on a performance assessment indicator. In: 2012 genetic and evolutionary computation conference (GECCO'2012), Philadelphia, July 2012. ACM Press, pp 505–512. ISBN: 978-1-4503-1177-9
150. Rosenberg R (1967) Simulation of genetic populations with biochemical properties. PhD thesis, Department of Communication Sciences, University of Michigan, Ann Arbor, June 1967
151. Rosenfeld J, Van Assche D, De Smet Y (2021) Lexicographic constrained multicriteria ordered clustering. In: Ishibuchi H, Zhang Q, Cheng R, Li K, Li H, Wang H, Zhou A (eds) Evolutionary multi-criterion optimization, 11th international conference, EMO 2021, Shenzhen, 28 Mar–31 Mar 2021. Lecture notes in computer science, vol 12654. Springer, pp 453–464. ISBN 978-3-030-72061-2
152. Rudolph G, Agapie A (2000) Convergence properties of some multi-objective evolutionary algorithms. In: Proceedings of the 2000 conference on evolutionary computation, Piscataway, July 2000, vol 2. IEEE Press, pp 1010–1016
153. Saif U, Guan Z, Zhang L, Zhang F, Wang B, Mirza J (2019) Multi-objective artificial bee colony algorithm for order oriented simultaneous sequencing and balancing of multi-mixed model assembly line. J Intell Manuf 30(3):1195–1220
154. Saini N, Saha S (2021) Multi-objective optimization techniques: a survey of the state-of-the-art and applications multi-objective optimization techniques. Eur Phys J-Spec Top 230(10):2319–2335
155. Salazar-Aguilar MA, Rios-Mercado RZ, Gonzalez-Velarde JL, Molina J (2012) Multiobjective scatter search for a commercial territory design problem. Ann Oper Res 199(1): 343–360
156. Samy PG, Kanesan J, Tiu ZC (2023) Optimization of chemotherapy using hybrid optimal control and swarm intelligence. IEEE Access 11:28873–28886
157. Schaffer JD (1985) Multiple objective optimization with vector evaluated genetic algorithms. In: Genetic algorithms and their applications: proceedings of the first international conference on genetic algorithms. Lawrence Erlbaum, pp 93–100
158. Schaffer JD (1984) Multiple objective optimization with vector evaluated genetic algorithms. PhD thesis, Vanderbilt University, Nashville
159. Schuetze O, Laumanns M, Tantar E, Coello Coello CA, Talbi E-G (2010) Computing gap free Pareto front approximations with stochastic search algorithms. Evol Comput 18(1):65–96
160. Schütze O, Esquivel X, Lara A, Coello Coello CA (2012) Using the averaged Hausdorff distance as a performance measure in evolutionary multiobjective optimization. IEEE Trans Evol Comput 16(4):504–522
161. Schütze O, Lara A, Coello Coello CA (2011) On the influence of the number of objectives on the hardness of a multiobjective optimization problem. IEEE Trans Evol Comput 15(4): 444–455

162. Schwefel H-P (1965) Kybernetische evolution als strategie der experimentellen forschung in der strömungstechnik. Dipl.-Ing. thesis (in German)
163. Schwefel H-P (1981) Numerical optimization of computer models. Wiley, Chichester
164. Sellami A, Ben Ali M, Kairouani L (2020) Pareto-optimization of MSF-OT/TVC desalination plant using surface response methodology and genetic algorithm. Desalination Water Treatment 204:10–21
165. Sengupta R, Saha S (2018) Reference point based archived many objective simulated annealing. Inf Sci 467:725–749
166. Sharma D, Agarwal D, Kumar S (2023) Reference-lines steered guide assignment and update for Pareto-based many-objective particle swarm optimization. Evol Intell 16:89–114
167. Sharma D, Collet P (2013) Implementation techniques for massively parallel multi-objective optimization. In: Tsutsui S, Collet P (eds) Massively parallel evolutionary computation on GPGPUs. Springer, pp 267–286. ISBN 978-3-642-37958-1
168. Smith RE, Forrest S, Perelson AS (1992) Searching for diverse, cooperative populations with genetic algorithms. Technical Report TCGA No. 92002, University of Alabama, Tuscaloosa
169. Smith RE, Forrest S, Perelson AS (1993) Population diversity in an immune system model: implications for genetic search. In: Whitley LD (ed) Foundations of genetic algorithms 2. Morgan Kaufmann Publishers, San Mateo, pp 153–165
170. Song F, Xing H, Luo S, Zhan D, Dai P, Qu R (2020) A multiobjective computation offloading algorithm for mobile-edge computing. IEEE Int Things J 7(9):8780–8799
171. Srinivas N, Deb K (1994) Multiobjective optimization using nondominated sorting in genetic algorithms. Evol Comput 2(3):221–248
172. Su M, Hu J, Wang Y, He Z, Cong J, Han L (2022) A multiobjective incremental control allocation strategy for tailless aircraft. Int J Aerospace Eng 2022. Article Number: 6515234
173. Tahmassebi A, Gandomi AH, Meyer-Baese A (2018) A Pareto front based evolutionary model for airfoil self-noise prediction. In: 2018 IEEE congress on evolutionary computation (CEC'2018), Rio de Janeiro, 8–13 July 2018. IEEE Press, pp 909–916. ISBN: 978-1-5090-6017-7
174. Takadama K, Sato K, Sato H (2019) Evolving generalized solutions for robust multi-objective optimization: transportation analysis in disaster. In: Deb K, Goodman E, Coello Coello CA, Klamroth K, Miettinen K, Mostaghim S, Reed P (eds) Evolutionary multi-criterion optimization, 10th international conference, EMO 2019, East Lansing, 10–13 Mar 2019. Lecture notes in computer science, vol 11411. Springer, pp 491–503. ISBN 978-3-030-12597-4
175. Tanabe R, Oyama A (2017) A note on constrained multi-objective optimization benchmark problems. In: 2017 IEEE congress on evolutionary computation (CEC'2017), San Sebastián, 5–8 June 2017. IEEE Press, pp 1127–1134. ISBN 978-1-5090-4601-0
176. Tian Y, Si L, Zhang X, Cheng R, He C, Tan KC, Jin Y (2021) Evolutionary large-scale multi-objective optimization: a survey. ACM Comput Surv 54(147):1–34
177. Tiwari S, Koch P, Fadel G, Deb K (2008) AMGA: an archive-based micro genetic algorithm for multi-objective optimization. In: 2008 genetic and evolutionary computation conference (GECCO'2008), Atlanta, July 2008. ACM Press, pp 729–736. ISBN 978-1-60558-131-6
178. Toscano Pulido G, Coello Coello CA (2003) The micro genetic algorithm 2: towards online adaptation in evolutionary multiobjective optimization. In: Fonseca CM, Fleming PJ, Zitzler E, Deb K, Thiele L (eds) Evolutionary multi-criterion optimization. Second international conference, EMO 2003, Faro, Apr 2003. Lecture notes in computer science, vol 2632. Springer, pp 252–266
179. Toscano Pulido G, Coello Coello CA (2004) Using clustering techniques to improve the performance of a particle swarm optimizer. In: Deb K et al (ed) Genetic and evolutionary computation–GECCO 2004. Proceedings of the genetic and evolutionary computation conference. Part I, Seattle, June 2004. Lecture notes in computer science, vol 3102. Springer, pp 225–237
180. Trivedi A, Srinivasan D, Sanyal K, Ghosh A (2017) A survey of multiobjective evolutionary algorithms based on decomposition. IEEE Trans Evol Comput 21(3):440–462

181. Phan DH, Suzuki J (2013) R2-IBEA: R2 indicator based evolutionary algorithm for multiobjective optimization. In: 2013 IEEE congress on evolutionary computation (CEC'2013), Cancún, 20–23 June 2013. IEEE Press, pp 1836–1845. ISBN 978-1-4799-0454-9
182. Valencia-Rodríguez DC, Coello Coello CA (2022) Multi-objective evolutionary algorithm based on the linear assignment problem and the hypervolume approximation using polar coordinates (MOEA-LAPCO). In: Rudolph G, Kononova AV, Aguirre H, Kerschke P, Ochoa G, Tušar T (eds) Parallel problem solving from nature – PPSN XVII, 17th international conference, PPSN 2022, proceedings, Part II, Dortmund, 10 Sep–14 Sep 2022. Lecture notes in computer science, vol 13399. Springer, pp 221–233. ISBN 978-3-031-14720-3
183. Vanitha S, Balasubramanie P (2023) Improved ant colony optimization and machine learning based ensemble intrusion detection model. Intell Autom Soft Comput 36(1):849–864
184. Vargas-Martinez M, Rangel-Valdez N, Fernandez E, Gomez-Santillan C, Rivera G, Balderas F (2024) MOSA/D-O and MOSAD/D-O-II: performance analysis of decomposition-based algorithms in many objective problems. SoftwareX 25. Article Number: 101610
185. Vikram K, Ragavendran U, Kalita K, Ghadai RK, Gao X-Z (2021) Hybrid metamodel-NSGA-III-EDAS based optimal design of thin film coatings. CMC-Comput Mater Continua 66(2):1771–1784
186. Villalobos-Arias M, Coello Coello CA, Hernández-Lerma O (2006) Asymptotic convergence of metaheuristics for multiobjective optimization problems. Soft Comput 10(11):1001–1005
187. Žilinskas A (2013) On the worst-case optimal multi-objective global optimization. Optim Lett 7:1921–1928
188. Žilinskas A (2014) A statistical model-based algorithm for 'black-box' multi-objective optimisation. Int J Syst Sci 45(1):82–93
189. Wang J, Su Y, Lin Q, Ma L, Gong D, Li J, Ming Z (2020) A survey of decomposition approaches in multiobjective evolutionary algorithms. Neurocomputing 408:308–330
190. Wang T-C, Ting C-K (2018) Fitness inheritance assisted MOEA/D-CMAES for complex multi-objective optimization problems. In: 2018 IEEE congress on evolutionary computation (CEC'2018), Rio de Janeiro, 8–13 July 2018. IEEE Press, pp 1013–1020. ISBN: 978-1-5090-6017-7
191. Wang X, Dong Z, Tang L (2020) Multiobjective differential evolution with personal archive and biased self-adaptive mutation selection. IEEE Trans Syst Man Cybern-Syst 50(12): 5338–5350
192. Wu X, Zhang S, Gong Z, Ji J, Lin Q, Chen J (2020) Decomposition-based multiobjective evolutionary optimization with adaptive multiple Gaussian process models. Complexity 2020. Article Number: 9643273
193. Xiang Y, Zhou Y, Tang L, Chen Z (2019) A decomposition-based many-objective artificial bee colony algorithm. IEEE Trans Cybern 49(1):287–300
194. Xing H, Song F, Yan L, Pan W (2019) On multicast routing with network coding: a multiobjective artificial bee colony algorithm. China Commun 16(2):160–176
195. Xu G, Feng Z, Guo E, Cai C, Ding H (2022) Multi-objective optimization of hydraulic shovel using evolutionary algorithm. Autom Construct 142. Article Number: 104486
196. Xu M, Zhang Q, Wang J (2021) Discounted sampling policy gradient for robot multi-objective visual control. In: Ishibuchi H, Zhang Q, Cheng R, Li K, Li H, Wang H, Zhou A (eds) Evolutionary multi-criterion optimization, 11th international conference, EMO 2021, Shenzhen, 28 Mar–31 Mar 2021. Lecture notes in computer science, vol 12654. Springer, pp 441–452. ISBN 978-3-030-72061-2
197. Xu Z, Zhang K (2021) Multiobjective multifactorial immune algorithm for multiobjective multitask optimization problems. Appl Soft Comput 107. Article Number: 107399
198. Yang Z, Qiu H, Gao L, Chen L, Liu J (2023) Surrogate-assisted MOEA/D for expensive constrained multi-objective optimization. Inf Sci 639. Article Number: 119016
199. Yi J, Bai J, He H, Zhou W, Yao L (2020) A multifactorial evolutionary algorithm for multitasking under interval uncertainties. IEEE Trans Evol Comput 24(5):908–922
200. Zavala GR, Nebro AJ, Luna F, Coello Coello CA (2014) A survey of multi-objective metaheuristics applied to structural optimization. Struct Multidiscip Optim 49(4):537–558

201. Zhang Q, Li H (2007) MOEA/D: a multiobjective evolutionary algorithm based on decomposition. IEEE Trans Evol Comput 11(6):712–731
202. Zhang Y, Li Z, Zhang H, Yu Z, Lu T (2018) Fuzzy C-means clustering-based mating restriction for multiobjective optimization. Int J Mach Learn Cybern 9(10):1609–1621
203. Zhao H, Zhang C, Zheng X, Zhang C, Zhang B (2022) A decomposition-based many-objective ant colony optimization algorithm with adaptive solution construction and selection approaches. Swarm Evol Comput 68. Article Number: 100977
204. Zhou Y, Wang J, Wu Z, Wu K (2018) A multi-objective tabu search algorithm based on decomposition for multi-objective unconstrained binary quadratic programming problem. Knowl-Based Syst 141:18–30
205. Zitzler E, Deb K, Thiele L (1999) Comparison of multiobjective evolutionary algorithms on test functions of different difficulty. In: Wu AS (ed) Proceedings of the 1999 genetic and evolutionary computation conference. Workshop program, Orlando, July 1999, pp 121–122
206. Zitzler E, Künzli S (2004) Indicator-based selection in multiobjective search. In: Yao X et al (ed) Parallel problem solving from nature – PPSN VIII, Birmingham, Sep 2004. Lecture notes in computer science, vol 3242. Springer, pp 832–842
207. Zitzler E, Laumanns M, Bleuler S (2004) A tutorial on evolutionary multiobjective optimization. In: Gandibleux X, Sevaux M, Sörensen K, T'kindt V (eds) Metaheuristics for multiobjective optimisation. Lecture notes in economics and mathematical systems, vol 535. Springer, Berlin, pp 3–37
208. Zitzler E, Laumanns M, Thiele L (2002) SPEA2: improving the strength Pareto evolutionary algorithm. In: Giannakoglou K, Tsahalis D, Periaux J, Papailou P, Fogarty T (eds) EUROGEN 2001. Evolutionary methods for design, optimization and control with applications to industrial problems, Athens, pp 95–100
209. Zitzler E, Thiele L, Laumanns M, Fonseca CM, da Fonseca VG (2003) Performance assessment of multiobjective optimizers: an analysis and review. IEEE Trans Evol Comput 7(2):117–132

Quantum-Inspired Heuristics

11

Seán McGarraghy and Milena Venkova

Contents

Introduction . . . 261
A Brief Primer on Physics . . . 262
A High-Level Taxonomy of Modern Physics . . . 262
Classical Mechanics . . . 264
Energy and Momentum . . . 264
The Hamiltonian . . . 265
Thermodynamics . . . 267
Statistical Mechanics . . . 269
Ergodicity . . . 269
Quantum Mechanics . . . 269
Observation and Wavefunction Collapse . . . 270
Normalised Wavefunctions . . . 271
Entanglement and Decoherence . . . 271
Noncommuting Operators . . . 272
Tunnelling . . . 273
Quantum Statistical Mechanics . . . 274
Quantum Computing . . . 274
Two-State Systems and Qubits . . . 275
Digital Quantum Computers . . . 275
Quantum Advantage . . . 276
Quantum Information . . . 278
Digital Quantum Computer Hardware . . . 279
Adiabatic Quantum Computation . . . 281
Annealing and Spin Glasses . . . 283
Ising Spin Glasses . . . 284

S. McGarraghy (✉)
School of Business, University College Dublin, Dublin, Ireland
e-mail: sean.mcgarraghy@ucd.ie

M. Venkova
School of Mathematics & Statistics, Technological University of Dublin, Dublin, Ireland
e-mail: milena.venkova@tudublin.ie

© Springer Nature Switzerland AG 2025

R. Martí et al. (eds.), *Handbook of Heuristics*,
https://doi.org/10.1007/978-3-032-00385-0_68

Quantum Spin Glasses . . . 285
The Quantum Annealing Hamiltonian . . . 285
The Metropolis(–Hastings) Algorithm . . . 288
Simulated Annealing . . . 289
Search and Neighbourhoods . . . 291
Acceptance of "Bad" Moves . . . 291
Simulated Quantum Annealing . . . 292
Implementation of SQA . . . 293
Applications of SQA . . . 294
Quantum-Inspired Evolutionary Algorithms (QIEAs) . . . 296
Binary-Valued QIGA . . . 297
Real-Valued QIEA . . . 303
QIEAs and EDAs . . . 306
Applications of QIEAs . . . 307
Quantum Binary PSO . . . 307
The Algorithm . . . 308
Applications and Further Reading . . . 310
Applications . . . 310
Further Reading . . . 310
Conclusions . . . 311
Cross-References . . . 312
References . . . 312

Abstract

Physical processes can inspire the design of algorithms and (meta)heuristics. In recent years, a number of such algorithms, implemented on both quantum and classical computers, have been developed and applied. In this chapter, we provide an outline of the range of these algorithms, focusing on those inspired by quantum physics, and describe some applications, particularly to problems in the domain of operational research. The chapter begins with an overview of the most relevant concepts from physics, including digital quantum computing, annealing, and spin glasses. We give a high-level overview of the current state of quantum computing, including hardware, and discuss its potential and current limitations. The simulated annealing algorithm, as well as the related simulated quantum annealing algorithm, is introduced. The chapter concludes with evolutionary and swarm-based algorithms that derive inspiration from quantum mechanics. Because of the enormous scope of work done in quantum-inspired algorithms and quantum computing, the chapter is necessarily selective in the material covered.

Keywords

Algorithm · Metaheuristic · Heuristic · Quantum computing · Quantum annealing · Simulated annealing · Simulated quantum annealing · Quantum-inspired evolutionary algorithm · Quantum-inspired swarm algorithm

Introduction

Physical processes can inspire the design of algorithms and (meta)heuristics. (Many methods in this chapter are not strictly algorithms: they are not guaranteed to provide a true solution in a finite time and rarely have time estimates; but for many of them the term algorithm is most common in the literature they come from, so we mostly use it.) In recent years, a number of such algorithms, implemented on both quantum and classical computers, have been developed and applied. In this chapter, we provide an outline of the range of such algorithms, focussing on those inspired by quantum physics, and describe some applications, particularly to problems in the domain of operational research.

We begin with a short introduction to the major areas of physics in sections "A Brief Primer on Physics" and "Annealing and Spin Glasses," since many physical concepts are central in algorithms of interest. It is not possible to discuss in detail all of these areas in one chapter, so we focus on the aspects of each area that are most helpful in understanding the range of quantum-inspired algorithms that currently exist.

In subsequent sections, we describe a number of algorithms that are inspired by aspects of these physical phenomena (readers who are looking for a high-level overview of physically inspired algorithms may wish to initially skip to section "Simulated Annealing," and then return to earlier sections for discussion of the physical principles which underlie the algorithms). The degree of fidelity of these algorithms to the original natural process varies. Different algorithms could be devised depending on which aspect of the underlying phenomenon is chosen: an algorithm designer may choose the degree of fidelity to nature of an algorithm, to give an algorithm which is efficient on the problems to which it is applied, while not necessarily being perfectly true to nature. As described in section "Digital Quantum Computers," Feynman [36] showed that, because of entanglement, no classical computer can efficiently simulate quantum systems, as it would need exponential time or resources to do so. Thus, for algorithms run on a classical computer, the degree of inspiration that can be taken from quantum mechanics is perforce limited; all such algorithms in this chapter are at best partial approximations of nature. However, as our criterion of usefulness is not fidelity to nature but rather effectiveness in problem solving, we discuss them here: despite their lack of physical verisimilitude, they have led to a rich thread of literature and many applications.

The algorithms discussed begin with a brief overview of the simulated annealing (SA) algorithm in section "Simulated Annealing" and the simulated quantum annealing algorithm (SQA) in section "Simulated Quantum Annealing." Later sections introduce hybrid binary-valued and hybrid real-valued quantum-inspired evolutionary algorithms, as well as quantum binary particle swarm optimisation algorithms; all of whose workings draw inspiration from concepts of quantum mechanics.

To constrain the size of this chapter, which could otherwise become overweeningly large, we rule out of scope considerations such as computational experience or testing. However, the reader should be aware that many of the methods discussed here are in their infancy, particularly the pure quantum approaches on quantum hardware; while they are currently not competitive with the best classical approaches, this may change as technology advances. An example of their potential is the anecdotal evidence of harvesting of Internet financial transactions, as the encryption of these may be breakable in future with more advanced quantum computers.

A Brief Primer on Physics

This section provides a short overview of some major areas of physics and how they fit together. Sections "Classical Mechanics" and "Annealing and Spin Glasses" explore in a little more depth classical mechanics, thermodynamics, quantum mechanics, quantum computing, and models of annealing. A good further introduction to classical and modern physics, with a fair level of mathematical detail, is [84].

A High-Level Taxonomy of Modern Physics

Figure 1 provides a rough taxonomy of physics, with major areas shown as boxes with continuous outline. Continuous arrows show which areas come from others by relaxing assumptions or introducing new principles to existing areas. Dotted boxes show some of the best-known physically inspired algorithms, and dotted arrows show which area(s) of physics inspire them.

Classical Mechanics This is due to Galileo, Newton, and others. Particles and/or bodies are acted on by *forces* which instantaneously act at a distance. Lagrange, Hamilton, and others gave equivalent descriptions in terms of *energy*.

Thermodynamics This was developed by Carnot, Boltzmann, and others. It studies measurable macroscopic properties, e.g. temperature and pressure, of a system comprising microscopic components (e.g. a gas), and the ideas of thermodynamic equilibrium, thermodynamic processes, and entropy. It explains why physical systems seek the lowest state of free energy.

Statistical Mechanics This is a statistical view of thermodynamics, explaining measurable macroscopic properties in terms of statistics of distinguishable particles. The statistical mechanics Ising model of spin lattices describes spin-glass states of magnetic materials and inspires the simulated annealing and simulated quantum annealing algorithms.

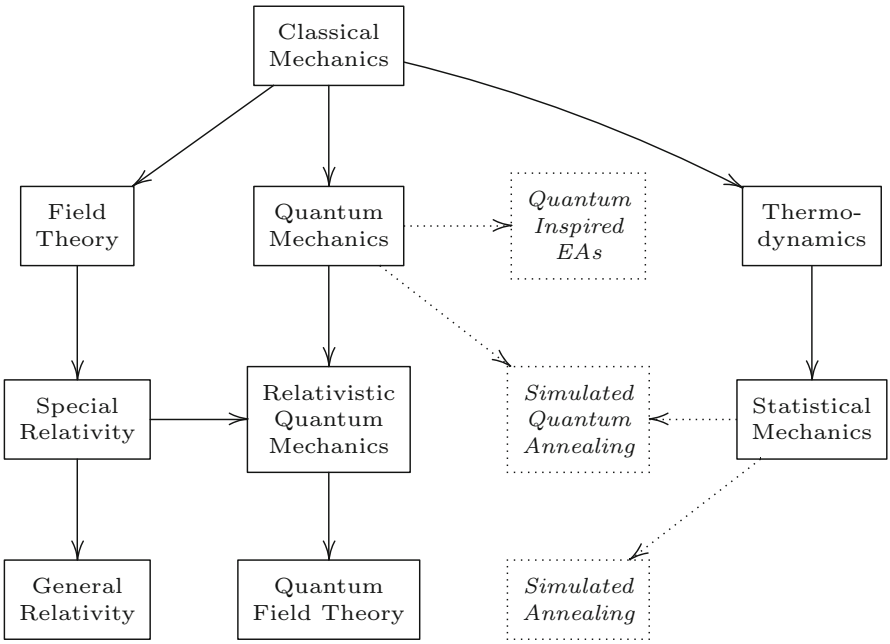


Fig. 1 Areas of physics (solid boxes) and connections among them (solid arrows); and major physically inspired algorithms (dotted boxes) and which areas of physics inspire them (dotted arrows)

Field Theory This was developed by Maxwell and others. An object may generate a scalar potential field V which gives rise to a vector field ∇V, e.g. the "classical" fields, gravitation, and electromagnetism. Other objects may be influenced by this field, e.g. a difference in electrical potential between two points can cause a flow of electrical current between them. A field can carry energy through space in the form of waves: this implies a finite signal propagation speed, dispensing with instantaneous "action at a distance." In particular, Maxwell's electromagnetism laws imply light has a constant finite speed c in a vacuum.

Special Theory of Relativity This is due to Einstein (and in part to Poincaré and Lorentz) who added the axiom of equivalence of inertial reference frames to the constant speed of light given by Maxwell's equations of electromagnetism. (Maxwell's equations are in fact relativistically invariant.)

General Theory of Relativity This was developed by Einstein from the Special Theory, by adding the *principle of equivalence* of inertial and gravitational mass: a uniform gravitational field is indistinguishable from acceleration.

Quantum Mechanics This is due to Planck, Bohr, Schrödinger, Heisenberg, Pauli, Einstein, and others. *Quantisation* means replacing physical quantities or *observables* (e.g. position, momentum, energy) by operators on a Hilbert space of *wavevectors* or *wavefunctions*.

Relativistic Quantum Mechanics Dirac reworked Schrödinger's equation for the electron to a relativistically invariant form in which the wavevectors have extra degrees of freedom, which capture the spin of the electron, and so predicted the existence of antimatter. However, to date, quantum mechanics and general relativity have not satisfactorily been combined.

Quantum Field Theory (QFT) This was developed by Feynman and others, building on Dirac's work. In QFT, the fields themselves, including the wavefunction, are quantised (called second quantisation): thus carriers of forces are particles. The first QFT, quantum electrodynamics (QED), was derived using a least action principle for path integrals and is the most precise current physical theory.

Classical Mechanics

In sections "Simulated Annealing" and "Simulated Quantum Annealing," we describe the simulated annealing and simulated quantum annealing algorithms. In these algorithms, "energy" is a core concept, so we first describe the ideas of energy and the Hamiltonian. When designing a physically inspired algorithm, constructing an artificial Hamiltonian to model the objective function of an optimisation problem is often a central step.

Energy and Momentum

Energy is the ability to perform work, e.g. move an object. (An object of negligible size is called a *particle*; otherwise – i.e. it has physical extension – it is called a *rigid body* or just *body*.) The total energy – or *Hamiltonian* – of a system may be viewed as comprised of two parts: the *kinetic* energy K, due to motion, and the *potential* energy, or *energy of interaction*, V, the energy an object has because of its position $x = (x_1, x_2, x_3)$ relative to other objects in space. Energy is a nonnegative scalar quantity. The *principle of conservation of energy* says that the total energy $K + V$ is a *conserved* or *invariant* quantity: it remains the same even if the internal configuration of the system changes (assuming no outside influences).

The time derivative of the position vector x of an object is called its *velocity* (*vector*) $v := \dot{x} := \frac{\mathrm{d}}{\mathrm{d}t}x$ (differentiation with respect to time t is commonly denoted by a dot over the variable, a notation introduced by Newton). A particle of mass m and velocity v has *momentum* $p := mv$. Momentum is a vector quantity, since velocity is a vector. The *total momentum* of a system is the vector sum of the momenta of individual objects in the system: it is a conserved quantity in that

it cannot change in the absence of influences from outside the system (Newton's first law). There are other conserved quantities in classical mechanics, e.g. angular momentum of rigid bodies.

In Newtonian dynamics, a *force* is an influence that can accelerate (change the velocity of) an object. Newton's second law, $F = ma$, relates the acceleration $a := \dot{v}$ of an object to the force vector F applied to it and the object's mass m. Because a force alters the velocity of an object, it alters its kinetic energy, and so does work. A *conservative* force is a force for which the amount of work it does (e.g. in moving a particle) is independent of the path taken: such processes are *reversible*. This is not true of nonconservative forces. Examples of conservative forces are gravity and electromagnetism.

Given a collection of particles, a conservative force can be represented by a scalar potential field $V = V(x_1, x_2, x_3)$ (i.e. a number defined at each point). Then its gradient $\nabla V := \left(\frac{\partial}{\partial x_1} V, \frac{\partial}{\partial x_2} V, \frac{\partial}{\partial x_3} V\right)$ is a vector field giving the resulting forces between particles; if there are no other forces, the particles will move in response to these forces, reducing the potential energy and converting the excess potential energy into kinetic energy, but keeping the sum $K + V$ constant. For example, if a ball is held above the surface of the earth and then released, gravitational forces will pull the earth and ball together, converting the potential energy into kinetic energy (which appears as heat energy of the atoms of the earth and ball).

The Hamiltonian

The *Hamiltonian*, or total energy of a system, is important in optimisation algorithms, both true quantum algorithms on quantum computers and physically inspired algorithms on classical computers, such as simulated annealing and simulated quantum annealing. In such physically inspired algorithms, the Hamiltonian (more precisely, an analogue of it) is constructed from a potential energy term, namely the objective function, and a kinetic energy term, representing some modification to the objective function.

Newton's laws were reformulated by Euler, Lagrange, Hamilton, and others in terms of energy, a scalar quantity, leading to unifying views of mechanics. For us, the most important of these views is the *Hamiltonian* approach; it is closely related to the *Lagrangian* view, which we discuss first.

Suppose that a system of particles and bodies is given. In both views, the *configuration space* $\mathcal{S}$ of the system is considered: each point of $\mathcal{S}$ represents an arrangement in space of all the particles and bodies, that is, all the locations of all parts of the system. $\mathcal{S}$ has a large dimension N. For example, for n particles, $\mathcal{S}$ has $N = 3n$ dimensions as it includes the three position coordinates of all n particles. As time passes, the single point q of $\mathcal{S}$ that represents the system moves around in $\mathcal{S}$ following some law that completely describes the classical behaviour of the system. This law derives from one scalar-valued function.

In the Lagrangian view, the function is known as the *Lagrangian* $\mathcal{L}$ and is defined on the $2N$-dimensional tangent bundle $T(\mathcal{S})$ of $\mathcal{S}$. The Lagrangian is the kinetic

energy minus the potential energy: $\mathcal{L} = K - V$. It is important for the following reason. As the system evolves, the point q representing it in $\mathcal{S}$ moves along some curve C; the (*Hamilton*) *principle of least action* says that the motion of q through $\mathcal{S}$ is such that the *action* (the integral of $\mathcal{L}$ along C) will be a minimum. That is, if the action is taken as a measure of curve length, C is a geodesic.

In the Hamiltonian view, the function describing the behaviour of the system is known as the *Hamiltonian* $\mathcal{H}$ and is defined on the *phase space* $\mathcal{P}$: another name for the $2N$-dimensional cotangent bundle $T^*(\mathcal{S})$ of $\mathcal{S}$. The Hamiltonian is the sum of kinetic and potential energy, that is, $\mathcal{H} = K + V$ is the total energy of the system. (Note: the cotangent bundle and tangent bundle are in general *manifolds*. Neither is a linear space, nor is there a single coordinate system for the whole space.)

Let the coordinates of a point x in $\mathcal{S}$ be $x_1, \ldots, x_N$, sometimes called *generalised position coordinates*. Let $p_1, \ldots, p_N$ be the corresponding *generalised momentum coordinates*. Here, if x_i is a position coordinate of a particle with mass m, and the particle has velocity component v_i in direction x_i, then $p_i = mv_i$. The point P in the phase space $\mathcal{P}$ corresponding to $x \in \mathcal{S}$ has coordinates $(x, p) = (x_1, \ldots, x_N, p_1, \ldots, p_N)$, and the value of the Hamiltonian at this point is $\mathcal{H}(x, p) = \mathcal{H}(x_1, \ldots, x_N, p_1, \ldots, p_N)$.

The Hamiltonian $\mathcal{H}$ leads to *Hamilton's dynamical equations*,

$$\frac{\mathrm{d}p_i}{\mathrm{d}t} = -\frac{\partial \mathcal{H}}{\partial x_i}, \qquad \frac{\mathrm{d}x_i}{\mathrm{d}t} = \frac{\partial \mathcal{H}}{\partial p_i}, \qquad i = 1, \ldots, N, \tag{1}$$

which describes the time evolution of the original system as the trajectory of the point P in the phase space.

For intuition, consider a single particle of mass m, in a field described by a potential $V = V(x_1, x_2, x_3, t)$ that depends on position and time. V defines the potential energy of the particle due to the field. The kinetic energy of a particle of mass m and velocity v is $\frac{1}{2}m\|v\|^2 = \frac{1}{2m}\|p\|^2$, which in this example gives $\frac{1}{2}m(\dot{x_1}^2 + \dot{x_2}^2 + \dot{x_3}^2) = \frac{1}{2m}(p_1^2 + p_2^2 + p_3^2)$. Therefore

$$\mathcal{H} = \frac{p_1^2 + p_2^2 + p_3^2}{2m} + V. \tag{2}$$

For example, if the particle is in the earth's gravitational field, with position coordinates x_1, x_2, x_3, where x_3 is its distance from the earth, we can take $V = mgx_3$ where $g \approx 9.8\ \mathrm{ms}^{-2}$ is the acceleration due to gravity. Equation (2) gives

$$\mathcal{H} = \frac{p_1^2 + p_2^2 + p_3^2}{2m} + mgx_3. \tag{3}$$

The phase space $\mathcal{P}$ has the important property that it is a symplectic manifold, equipped with a *Poisson bracket*, $\{\ ,\ \}$, which takes two scalar fields A and B defined on $\mathcal{P}$ and returns a third such scalar field, $\{A, B\}$. The Poisson bracket has useful properties. The time evolution of any scalar physical quantity $A = A(x, p)$ is given

by its Poisson bracket with the Hamiltonian: $\dot{A} = \{A, \mathcal{H}\}$. It can be shown that $\{\mathcal{H}, \mathcal{H}\} = 0$, the principle of conservation of energy in another form. The symplectic or *canonical* structure given by the Poisson bracket is generalised in the setup of quantum mechanics.

Thermodynamics

Annealing (see section "Simulated Annealing") may be viewed as minimising the (*Helmholtz*) *free energy* of a system, through a slow cooling; this is in the sense of the thermodynamic principle that *every system seeks to achieve a minimum of free energy*. It may also be viewed as maximising the entropy, since the system is relaxed to thermal equilibrium. We now give some intuition on these concepts.

A thermodynamic system is *macroscopic* or large scale: it is taken to consist of a large number of distinguishable particles, typically of the order of Avogadro's number, 6.022×10^{23}. In a gas or liquid, the particles are free to move; in a solid, they are largely fixed but can rotate or vibrate about their average positions. Thermal energy has two components, the kinetic energy due to random motion of particles and the potential energy of their relative positions. Each particle has several degrees of freedom that can hold energy: translational degrees (which usually dominate), namely, the velocity components in the three coordinate directions, as well as any rotation or vibration about its centre of mass that the particle may have. Measurable macroscopic properties of a system such as pressure or temperature are properly viewed as statistical averages derived from the energies in the degrees of freedom of the constituent particles (components) of the system.

We view the system at a given time as being in a state in phase space $\mathcal{P}$, often called a *microstate*. We "roughly" divide $\mathcal{P}$ into subsets called *macrostates*. If two (micro)states x and y lie in the same macrostate, this means that macroscopically we cannot tell apart x and y: the temperature, pressure, etc., of x and y may be indistinguishable to the best precision of our measuring instruments. This division into macrostates is somewhat arbitrary but suffices for our purposes.

The *entropy* S of a system microstate $x \in \mathcal{P}$ is $k_B \ln \Omega$, where Ω is the volume in $\mathcal{P}$ of the macrostate containing x and k_B is Boltzmann's constant. This definition of entropy is reasonably robust because of the slow growth of the log function, and the fact that the macrostates' volumes are generally very different. *Thermal equilibrium* (or *thermodynamic equilibrium*) is by far the most likely macrostate (its volume is almost equal to the volume of $\mathcal{P}$ itself) and so the state of maximum entropy. At thermal equilibrium, the principle of *equipartition of energy* holds, i.e. statistically, the system's energy is distributed equally among all of the system's degrees of freedom. (The Statistical Mechanics *Theorem of Equipartition of Energy*, which comes from Newton's Laws, says that *the total energy of a system of a large number of particles, exchanging energy among themselves through collisions, is, on average, shared equally among all the particles' degrees of freedom*.)

The *temperature* T of the system is a measure of the average energy per degree of freedom. If the system is not in thermal equilibrium, then the temperature differs

according to the location in the system (there is more energy at some locations), meaning there is a temperature potential and a resulting temperature gradient vector field. This leads to a flow of heat energy in the system and so it has the ability to provide useful work. Since only temperature gradients (differences) can perform useful work, a system in thermal equilibrium cannot perform any useful work, regardless of the actual quantity of (heat) energy in the system. Any disturbance to a system not already in thermal equilibrium is more likely than not to disperse energy more uniformly among degrees of freedom: this increases entropy and brings the system closer to thermal equilibrium.

Combinatorially, if units of energy are randomly allocated to degrees of freedom, this is much more likely to result in an *even* distribution of energy among degrees of freedom. Because of this, large-volume macrostates correspond to more uniformly spread energy, while smaller-volume macrostates correspond to nonuniformly distributed (peaked) distributions of energy among degrees of freedom. Thus, a low entropy means high energy (temperature) differences in the system, high temperature gradient, and high useful work, while high entropy means low energy differences in the system and low useful work. Thus, entropy may be viewed as a measure of the *unavailability* of a system's energy to do work. This allows us to interpret high entropy as low availability of "high-quality" energy and vice versa.

The *Helmholtz free energy* A is a thermodynamic measure of the useful work that can be obtained from a closed system at a constant temperature: $A = U - TS$, where U is the *internal* or total energy of the system (i.e. the Hamiltonian). It has a minimum at thermal equilibrium, that is, when entropy S is maximised. Hence, at nonzero temperatures, free energy is the quantity minimised in natural processes such as annealing.

The *first law of thermodynamics* is a restatement of the principle of conservation of energy, applied to heat energy.

The *second law of thermodynamics* says that, statistically, entropy increases over time; that is, the system's energy, which may originally have been distributed among relatively few degrees of freedom, tends to distribute itself among all possible degrees of freedom. (Thus, the second law is sometimes phrased as "Heat flows from a hotter to a colder place.") Hence, a system tends towards thermal equilibrium: differences in pressure, temperature, and density tend to even out; entropy measures how far this evening-out process has progressed. (An information-theoretic interpretation of entropy is that it is a measure of our uncertainty about a system. Thermal equilibrium maximises entropy because we lose all information about the initial conditions except for the summary statistics: temperature, etc. Maximising entropy maximises our ignorance about the details of the system: this is the sense in which entropy measures "disorder.") From the viewpoint of statistical mechanics, the second law arises because even distributions of energy across degrees of freedom are combinatorially much more likely (have bigger phase space volumes) than distributions with energy concentrated in relatively few degrees of freedom. The second law explains the effectively irreversible nature of physical processes involving large numbers of components.

Statistical Mechanics

Because of the large numbers of particles and degrees of freedom in any macroscopic physical system, measurable macroscopic quantities are described in terms of statistics of distinguishable particles.

For each microstate x of a classical thermodynamical system, we denote its total energy (Hamiltonian) by $\mathcal{H}_x$. Let $\beta = \frac{1}{k_B T}$, where $k_B \approx 1.38 \times 10^{-23}\,\mathrm{JK}^{-1}$ is Boltzmann's constant. (β is often called the *inverse temperature*, though its dimension is inverse energy.) The probability $P(x)$ that the system is in microstate x is proportional to the *Boltzmann factor* $\mathrm{e}^{-\beta\mathcal{H}_x}$. The proportionality constant is $Z := \sum_x \mathrm{e}^{-\beta\mathcal{H}_x}$ (the sum taken over all microstates), called the *partition function* of the system. In other words, the probability $P(x) = \frac{1}{Z}\mathrm{e}^{-\beta\mathcal{H}_x}$, so the energy of state x is proportional to the natural logarithm of its probability. The partition function is at the heart of statistical mechanics because all other thermodynamic properties of the system can be written in terms of Z or its derivatives. For example, the entropy is $S = \frac{\partial}{\partial T}(\frac{1}{\beta}\ln Z) = \frac{\partial}{\partial T}(k_B T \ln Z)$, and the Helmholtz free energy is $A = -\frac{1}{\beta}\ln Z$.

Ergodicity

We call a system's evolution or a thermodynamic process *ergodic* if, over a long time interval, the time spent by the system in a particular region of the phase space is proportional to the volume of the region.

To be more precise: a *weakly ergodic* evolution (*weak ergodicity*) means that the system forgets its initial state during the process of evolution; a *strongly ergodic* evolution (*strong ergodicity*) means that the system converges to a stationary distribution regardless of what its initial state was. For the design of algorithms, we prefer a search algorithm to be as ergodic as possible, and that (for a strongly ergodic evolution) the state to which the system converges is the state being sought (e.g. an optimum).

We remark later that certain physical systems such as magnets or spin glasses are not very ergodic, but that the kind of fluctuations caused by annealing or quantum annealing can make them more ergodic, so that they reach the ground (lowest) energy state more easily. Simulated annealing and simulated quantum annealing try to mimic this enhancement of ergodicity for objective functions.

Quantum Mechanics

Quantum mechanics is an extension of classical mechanics. It models behaviours of natural systems that are observed at very short time or distance scales. One example of such a natural system is a subatomic particle, e.g. a free electron or photon (a particle of light), but a quantum system may be composed of many particles.

Quantum mechanics comes about from classical mechanics by applying a formal *quantisation* procedure: variables representing observable physical quantities such as energy, position, momentum, spin, etc., are replaced by linear operators on a suitable vector space. The operators are called *observables*; examples include the position, momentum, and Hamiltonian of a particle. The vector space is actually a complete inner product space, that is, a *Hilbert space* over the complex numbers $\mathbb{C}$; the elements of this space are complex-valued (deterministic) functions of space and time coordinates.

This Hilbert "phase" space is analogous to a classical phase space. (In a classical phase space for n particles, we deal with real-valued functions of $6n$ dimensions, each particle having three spatial and three momentum coordinates. In quantum Hilbert phase space, we deal with complex-valued functions on a $3n$-dimensional space. However, much of the mathematical structure carries over.) An element ψ of the Hilbert space is associated with the system: ψ is called the system's *state vector* or *wavefunction* or *wavevector*, and it describes the *quantum state* the system is in. The only values that can be observed for a physical quantity are eigenvalues of the corresponding observable (a linear operator on the Hilbert space). When an eigenvalue is observed, the system's state vector ψ must be a corresponding eigenvector (also called an *eigenfunction* or *eigenstate*: it is viewed as a "classical state") of the operator. The operator's eigenvectors form a basis for the Hilbert space, so any wavefunction can be written as a linear combination of these eigenstates.

Observation and Wavefunction Collapse

The traditional "Copenhagen" interpretation of quantum mechanics is that to speak of the internal state of a quantum system before observation has no meaning, but that the wavefunction allows us to calculate probabilities of outcomes of experiments. The squared modulus $|\psi|^2$ of the wavefunction ψ is a probability density function (PDF). It describes the probability that an observation of, e.g. a particle will find the particle at a given time in a given region of space. The wavefunction ψ satisfies the linear *Schrödinger equation* (a differential equation):

$$\mathrm{i}\hbar \frac{\partial}{\partial t}\psi = \mathcal{H}\psi, \tag{4}$$

where $\mathrm{i} := \sqrt{-1}$, $\hbar := \frac{h}{2\pi}$ and $h \approx 6.62 \times 10^{-34}$ Js is Planck's constant. The Schrödinger equation is the basic (dynamical) equation of motion: it says that the quantum Hamiltonian of the wavefunction is equal to an imaginary constant $\mathrm{i}\hbar$ multiplied by the time derivative of the wavefunction. (The Hamiltonian contains all dynamical information: the wavefunction at time $t + \delta t$ (for δt small) is got by operating on the wavefunction at time t with $\frac{1}{\mathrm{i}\hbar}\mathcal{H}$.) This equation, a diffusion equation, describes the time evolution of the wavefunction, and thus the PDF $|\psi|^2$, at each point in space: as time proceeds, the PDF becomes more "spread

out" over space, and our knowledge of, e.g. the position of the particle becomes less precise. The spreading continues until an observation is carried out; then, the wavefunction *collapses* nonlinearly to a particular classical state (eigenstate), in this case a particular position, and the spreading out of the PDF starts all over again. (Wavefunction collapse upon observation is also called *state vector reduction*.)

We regard the system before observation as being in a linear combination of all possible classical states (called *superposition of states*); then the act of observing the system causes one classical state to be chosen, with probability as given by the PDF. This randomness in any measurement means that we cannot infer the initial state of the system from the measurement result.

Normalised Wavefunctions

The essentially complex nature of the wavefunction means that even if its amplitude is constant, it has a varying phase. (The *phase* of a wave is the argument or angle of a complex number: it is unrelated to phase space.) Thus, the wavefunction may interfere with itself, e.g. if a barrier with slits is placed in a particle's "path." The interference may be either constructive or destructive: the probability of detecting a particle in a given position may increase or decrease. Counterintuitive as it may seem, experimental evidence consistently agrees with this quantum-mechanical view of nature.

The probability of finding a particle is largest at locations where the modulus $|\psi|$ of the complex wavefunction ψ is largest; it is 0 if $|\psi| = 0$. Since the probability of finding a particle somewhere in space has to be 1, we generally restrict ourselves to *normalised* wavefunctions ψ, that is, those whose integral over all of space, $\int_{\mathbb{R}^3} |\psi(x)|^2 \mathrm{d}x = 1$. Thus, we restrict our attention to the *unit ball* in the Hilbert space: the set of all wavevectors of norm 1. Then the allowable operators on the Hilbert space are the *unitary* operators, that is the operators that preserve norms. (A linear operator A on a normed vector space V is called *unitary* if for all $v \in V$, it is the case that $\|Av\| = \|v\|$.)

Observables may be either continuous (e.g. position of a free particle) or discrete (e.g. energy of an electron in a bound state in an atom). Some discrete observables have only finitely many values, e.g. there are only two possible values for a given particle's spin in a given direction ("up" or "down"). (In Dirac's relativistic wave equation – an extension of the Schrödinger equation which is consistent with special relativity – spin is one of the arguments of the wavefunction.)

Entanglement and Decoherence

A key difference between classical and quantum physics is the phenomenon of quantum *entanglement*: the wavefunction of a system composed of many particles cannot be separated into independent wavefunctions, one for each particle. There is *one* wavefunction for the whole system, and this system wavefunction evolves

according to the Schrödinger equation until an observation is made. An observation made on one particle causes the collapse of the total system wavefunction and so produces an instantaneous nonlocal effect on other particles with which it is entangled, no matter how far apart they are: all components of the system are forced to immediately adopt a particular eigenstate. Entanglement causes an explosion in the dimension of the relevant Hilbert space. The correct Hilbert space for a quantum system of m particles, each of which can be in n distinct locations, is the m-fold tensor product, rather than the m-fold direct sum: its dimension is m^n, rather than mn as would happen classically. This is a crucial difference between quantum and classical physics and is one reason for quantum computers performing better than classical computers on certain problems [100]. Since the 1980s, physicists have come to view the nonlocal correlations of entangled quantum states as a new kind of nonclassical resource to be exploited, e.g. in quantum computers.

The issue of what exactly is an "observation" or "measurement" is a thorny one. A modern view is that the "quantumness" of a system is in the nonlocal entanglement effects among particles. Then, when a quantum system interacts with a macroscopic system (e.g. its environment, or a measuring device), the quantum nature of the system "leaks" into the environment in a thermodynamically irreversible way: the phase relations are spread so "thinly" among the many possible degrees of freedom that different elements of the combined wavefunction of system and environment can no longer effectively interfere with each other; we say they have *decohered*. This *decoherence* shows up as an apparent collapse of the wavefunction: decoherence causes superpositions of states to decay; and the larger the scale of the superposition, the faster the decay. In this view, quantum collapse is not an effect of a human observer but instead a consequence of the huge number of degrees of freedom (that is, the huge Hilbert space dimension) in any macroscopic system.

Noncommuting Operators

The symplectic or *canonical* structure given by the Poisson bracket in classical phase space is generalised in the construction of quantum mechanics: a *commutator* $[\,,\,]$ with similar properties is defined by $[A, B] := AB - BA$ for any two operators A and B. Then A and B commute (that is, the order in which they are applied does not matter) if and only if $[A, B] = 0$. The momentum operator associated with the kth coordinate direction x_k is defined as the differential operator:

$$p_k := \mathrm{i}\hbar \frac{\partial}{\partial x_k}. \tag{5}$$

(This is the *correspondence principle*: the key point of (canonical) quantisation.) The commutator $[x, p]$ of the momentum operator p and the position operator x can be shown to be nonzero (in fact $[x, p] = \mathrm{i}\hbar\mathbf{1}$, where $\mathbf{1}$ is the identity operator), telling us that the p and x operators do not commute: they are *complementary*, and each has eigenvalues which are not eigenvalues of the other. From this, via Fourier

analysis, we get Heisenberg's *uncertainty principle*, $\mathrm{d}p\,\mathrm{d}x \geq \hbar/2$: noncommuting observables cannot simultaneously be measured to arbitrary precision. (Classical mechanics is the limiting case of quantum mechanics as $h \to 0$. In this case, all commutators are 0 so all operators commute, and there is no uncertainty. Classical mechanics is also the limiting case of special relativity as $c \to \infty$, where c is the speed of light in vacuo.)

Using (5), and recalling (2) for the energy of a single particle, we can rewrite the (quantum) Hamiltonian of a particle as

$$\begin{aligned}\mathcal{H} &= \frac{p_1^2 + p_2^2 + p_3^2}{2m} + V \\ &= -\frac{\hbar^2}{2m}\Big[\Big(\frac{\partial}{\partial x_1}\Big)^2 + \Big(\frac{\partial}{\partial x_2}\Big)^2 + \Big(\frac{\partial}{\partial x_3}\Big)^2\Big] + V \\ &= -\frac{\hbar^2}{2m}\Big(\frac{\partial^2}{\partial {x_1}^2} + \frac{\partial^2}{\partial {x_2}^2} + \frac{\partial^2}{\partial {x_3}^2}\Big) + V \\ &= -\frac{\hbar^2}{2m}\nabla^2 + V. \end{aligned} \tag{6}$$

Here, multiplication of operators is just composition, so $(\frac{\partial}{\partial x_k})^2 = \frac{\partial}{\partial x_k}(\frac{\partial}{\partial x_k}) = \frac{\partial^2}{\partial {x_k}^2}$; and we have used the standard notation ∇^2 for the div operator $\frac{\partial^2}{\partial {x_1}^2} + \frac{\partial^2}{\partial {x_2}^2} + \frac{\partial^2}{\partial {x_3}^2}$. Then the particle's Schrödinger equation $\mathrm{i}\hbar\frac{\partial}{\partial t}\psi = \mathcal{H}\psi$ becomes

$$\mathrm{i}\hbar\frac{\partial}{\partial t}\psi = -\frac{\hbar^2}{2m}\nabla^2\psi + V\psi. \tag{7}$$

Apart from the imaginary factor i, this is similar to other equations in classical physics: the time rate of change of a quantity ψ is described by a diffusion term (the second spatial derivative) and a drift term (the potential V). If there was only a diffusion term, we would get the classical heat equation $\frac{\partial u}{\partial t} = k\frac{\partial^2 u}{\partial x^2}$.

Tunnelling

In *quantum tunnelling*, a particle can jump from one side of a physical or energy barrier to another, without having to occupy all intermediate positions, even if its energy is not enough to surmount the barrier. This can be explained by the wave nature of the particle. (The tunnelling effect is a property of any wave. It can be observed, e.g. when light waves hit, at a glancing angle, a boundary where the refractive index changes.) The particle's wavefunction is spread through space and, in particular, is nonzero on the opposite side of the barrier, giving a nonzero probability of observing the particle there. The narrower the barrier the greater the probability of tunnelling. From this, it follows that the expected time to tunnel

through a barrier depends not only on the height of the barrier, but also on its width. For a fixed height, the mean tunnelling time is shorter for a narrower barrier. This is seen when a particle is trapped in a potential well of finite height, for example, an electron in an atom. Another physical example of tunnelling is where a neutron may be spontaneously emitted from a Uranium ^{238}U nucleus in radioactive decay: here, the potential well is caused by the *strong nuclear* force among nucleons, which is repulsive at very short distance scales but becomes strongly attractive as nucleon separation increases. (*Nucleon* is a generic term for nuclear particles, namely, protons and neutrons.)

Quantum Statistical Mechanics

The idea of a partition function carries over to quantum physics, where $\mathcal{H}$ is now an operator: in this case, the partition function $Z = \sum_x \mathrm{e}^{-\beta \mathcal{H}_x}$ is often written as the basis-independent *trace*, $Z = \mathrm{Trace}(\mathrm{e}^{-\beta\mathcal{H}})$. The exponential of an operator is defined using the Taylor series $\mathrm{e}^t = 1 + t + \frac{1}{2}t^2 + \frac{1}{3!}t^3 + \cdots$ for the exponential function. Here, the product of operators means composition, i.e. AB is defined by $(AB)(\psi) := A(B(\psi))$. Thus, in a quantum setting,

$$Z = \mathrm{Trace}(\mathrm{e}^{-\beta\mathcal{H}}) = \mathrm{Trace}\left(\mathbf{1} - \beta\mathcal{H} + \frac{1}{2}\beta^2\mathcal{H}^2 - \frac{1}{3!}\beta^3\mathcal{H}^3 + \cdots\right). \tag{8}$$

When using quantum-inspired algorithms, a cost function to be minimised is typically mapped to a quantum Hamiltonian $\mathcal{H} = \mathcal{H}_{\mathrm{pot}} + \mathcal{H}_{\mathrm{kin}}$ made up of two parts, the "potential" objective function $\mathcal{H}_{\mathrm{pot}}$ and a "kinetic" perturbation term $\mathcal{H}_{\mathrm{kin}}$. Hence, the *Suzuki-Trotter formula* for the exponential of the sum of two operators $A + B$ is important for estimating the partition function:

$$\mathrm{e}^{A+B} = \lim_{M\to\infty} \left(\mathrm{e}^{A/M}\mathrm{e}^{B/M}\right)^M. \tag{9}$$

The Suzuki-Trotter formula can be used even if the commutator $[A, B] \neq 0$.

Quantum Computing

Quantum computers have been built based on ideas of quantum mechanics. These quantum computers are provably able to solve certain problems more efficiently than classical computers are. *Digital* quantum computers work with qubits, a quantum version of bits, while *adiabatic* quantum computers slowly evolve a quantum system to optimise some quantity such as energy, which models some real objective function.

We discuss quantum computers here for two reasons: firstly, because their workings are the inspiration for simulated quantum annealing (SQA: section "Simulated

Quantum Annealing"), quantum-inspired evolutionary algorithms (section "Quantum-Inspired Evolutionary Algorithms (QIEAs)"), and quantum-inspired particle swarm algorithms (section "Quantum Binary PSO"); and secondly, it is plausible that in the near- to medium-term future, true quantum computers of sufficient size, efficiency and error control may be available for use on complex problems [12].

Two-State Systems and Qubits

The spin of a quantum particle has precisely two possible values relative to a direction of measurement: "up" or "down." This is one example of a *two-state system*. In a two-state system, the quantum state ψ is a linear superposition of exactly two eigenstates, say $|0\rangle$ and $|1\rangle$, in the standard Dirac "bra-ket" notation; that is,

$$\psi = \alpha|0\rangle + \beta|1\rangle. \tag{10}$$

Here $|0\rangle$ and $|1\rangle$ (called "kets") are orthonormal basis vectors for a two-dimensional complex Hilbert space, and $\alpha, \beta \in \mathbb{C}$, with $|\alpha|^2 + |\beta|^2 = 1$ for ψ normalised (i.e. ψ is a unit vector). A two-state system where the states are orthogonal and normalised, as in this case, may be regarded as a *quantum bit* or *qubit*. (Geometrically, a qubit is a projective two-dimensional complex Hilbert space $P\mathbb{C}^2$: a compact two-dimensional complex manifold, the Riemann or Bloch sphere.) It is regarded as being in eigenstates $|0\rangle$ and $|1\rangle$ simultaneously, until a measurement projects ψ onto the basis $\{|0\rangle, |1\rangle\}$: then the state vector ψ collapses to $|0\rangle$ (with probability $|\alpha|^2$) or $|1\rangle$ (with probability $|\beta|^2$). The normalisation relation $\|\psi\|^2 = |\alpha|^2 + |\beta|^2 = 1$ captures the physical requirement that precisely one of $|0\rangle, |1\rangle$ must be observed, so their probabilities of observation must sum to 1.

Digital Quantum Computers

A *digital* (or *standard* or *circuit theory*) quantum computer is one which uses qubits instead of the bits used by classical computers. Paul Benioff [11] first considered a Turing machine which used a tape containing the equivalent of qubits. Richard Feynman [36], while studying the simulation of quantum systems on classical computers, developed examples of physical (quantum) computing systems that are not equivalent to the (deterministic) Turing machine. He showed that, because of entanglement, no classical computer (not even a probabilistic one) can efficiently simulate quantum systems, as it would need exponential time or resources to do so. An n-bit classical system is characterised by n times the amount of information needed to characterise a one-bit system. However, the amount of information needed to characterise a general entangled state of n qubits is $O(2^n)$: the Hilbert space has 2^n dimensions, so an entangled state is a superposition of 2^n n-qubit basis (eigen)states, and a complex coefficient is needed for each basis vector.

In a digital quantum computer, computations are carried out by applying a sequence of unitary operators, called quantum *gates* or quantum *evolutions*, to a system of qubits. A quantum gate transforms an input into an output, e.g. by a rotation, Hadamard or CNOT (controlled NOT) gate [100], and operates without measuring the value of the qubit(s); that is, no observation is performed until the end of the computation, when all gate transformations have been applied. (These ideas are described further in section "Binary-Valued QIGA," where they are used in new types of evolutionary algorithms.) A quantum gate effectively does a parallel computation on all qubits simultaneously and gives a new superposition. Because the Hilbert space of a system of n qubits has dimension 2^n, we can represent an operator on the space (e.g. a gate) by a $2^n \times 2^n$ matrix. The matrix must be unitary (and self-adjoint), so that the new coefficients for the ith qubit will still give valid probabilities.

The nature of qubits and entanglement means special algorithms must be developed for digital quantum computers. The details are beyond the scope of this chapter, but a good overview can be found in [12].

Quantum Advantage

The term *quantum advantage* (or *quantum supremacy*) is used to describe a problem that can be solved efficiently on a quantum system but cannot be solved on a classical machine in a reasonable time and at a reasonable energy cost. The term *quantum speedup* refers to the fact that for certain problems, the solving time on a quantum computer grows more slowly with the problem size than does the time on a classical computer.

Feynman [36] noted that quantum systems could efficently do computations that a classical computer could not (e.g. simulate themselves!). Deutsch [31] described the first universal quantum computer that could efficiently simulate any other quantum computer and gave the Deutsch algorithm, the first quantum algorithm that is provably faster than a classical algorithm.

In the 1990s, interest in qubit-based quantum computers was stimulated by the publication of algorithms that were provably faster than classical algorithms could be for the same problem: shor's algorithm [97, 99] for factorising a large integer of n bits into prime factors in $O(n^2)$ time – compared to $O(\exp(n^{1/3}))$ time classically – for which he received the Nevanlinna Prize in 1998; and Grover's unstructured database search algorithm [44, 45] which reduced the complexity from $O(n)$ on a classical computer to $O(\sqrt{n})$.

The problem of factorising a large integer has generated particular interest since important cryptography schemes such as RSA (used in everyday communications and commerce) depend on their security on such factorising being very difficult in practice. However, factorising an integer of practical significance on a current noisy quantum computer requires far more qubits (millions) than will be available in the near future. Note that other cryptography approaches can also be broken

efficiently by quantum computers [77, Table 2]. Quantum algorithms are successful in cryptography in part because there is "hidden problem structure that quantum computers can exploit in ways that classical computers cannot. Finding such hidden structure in other problems of practical interest remains an important open problem" [77].

In general, quantum computers need fewer operations, but each operation is slower. Whether quantum advantage exists then depends on multiple performance and cost measures [62]. However, Bova et al. [16] show that quantum computers – even if they have no quantum advantage – can still provide economic value compared to classical computers through lower cost and power consumption.

Other Applications of Quantum Computing

Other applications have been found for which quantum computers can provably outperform classical computers, such as computation of discrete logarithms [97], simulation of quantum systems [2, 56], solving certain systems of linear equations [49], the triangle finding problem in a graph [65], search and optimisation [77], sampling the output of a pseudo-random quantum circuit (using a 53-qubit quantum computer) [6], neural networks, and machine learning [71, 72] and finding one or more secret keys from a set of secret keys using a quantum oracle [101]. Some recent work [74] shows that the quantum singular value transformation (QSVT) encapsulates the three major quantum algorithms of factorisation, search, and simulation of quantum systems. Gomes et al. [43] give a wide-ranging overview of uses of quantum algorithms in operational research, with many references. Analogies of these, implemented on classical machines, form the basis of quantum-inspired evolutionary algorithms (QIEAs), described in section "Quantum-Inspired Evolutionary Algorithms (QIEAs)."

Complexity Classes for Quantum Computing

It is an active area of research to identify precisely which problems admit of a quantum advantage, and to try to define the complexity class of such problems. Because of the nature of wavefunction evolution and collapse, digital quantum computers are fundamentally probabilistic. Typically, an error probability of $\leq \frac{1}{3}$ is considered acceptable [97].

There are several complexity classes relevant to quantum computing [12, 62]. BPP (bounded-error probabilistic polynomial time) is the class of decision problems that can be solved in time polynomial in the input size by a probabilistic Turing machine with an error probability $\leq \frac{1}{3}$ for each instance. A fortiori, BPP contains the class P of all decision problems that can be solved in time polynomial in the input size by a deterministic Turing machine.

BQP (bounded-error quantum polynomial time) is the class of decision problems for which there exists a quantum algorithm that solves the decision problem with error probability of $\leq \frac{1}{3}$ and is guaranteed to run in time polynomial in the input size [100]. BQP is usually taken as the class of feasible problems for quantum computers. It contains the integer factorisation and discrete logarithm problems and some other

pure mathematics problems, e.g. the principal ideal problem. BQP is the quantum analogue to BPP and contains P and BPP. The integer factoring problem is in BQP but not in P.

NP is the class of decision problems for which the problem of verifying the correctness of a proposed solution is in P (equivalently, for which there exists an algorithm which solves the problem on a nondeterministic Turing machine in polynomial time). Although the problem of factorising integers is in both NP and BQP, quantum computers are not known to be able to solve all NP problems in polynomial time; in fact, it is widely believed that they are not able to. That is, BQP is not known to contain NP; in fact, it is conjectured that BQP does not contain NP. Thus, it is expected that quantum computers will not be able to solve NP problems quickly, in general. Proofs of inequality between BQP and these and other classical complexity classes are not yet available and are assumed to be difficult.

QMA (quantum Merlin Arthur) is the quantum equivalent of NP: a problem is in QMA if, given "yes" as the answer, the solution can be verified in a time polynomial in the problem input size.

Bharti et al. [12, Fig. 1] provide a diagram of inclusions (some proven, some conjectured) among these and other complexity classes.

Quantum Information

A key distinction between a qubit and a (classical) bit is that multiple qubits can be entangled. This means that a set of qubits can be highly correlated. Entanglement also allows many states to be acted on simultaneously, unlike bits that can only have one value at a time; thus, a quantum computer can extract global (nonlocal) information. This is sometimes called *quantum parallelism* [31] and partly explains the power of quantum computing: because the state of the quantum computer (i.e. the state of the whole system) can be a quantum superposition of many different classical computational states, these classical computations can all be carried out at the same time. For further information on quantum algorithms and entanglement, including possible use in cryptography, see [20].

Much of the art of designing quantum algorithms is in finding ways to make efficient use of nonlocal correlations. A quantum computer is usually taken to be *globally phase coherent*, that is having every qubit entangled with every other qubit (global phase coherence is, for example, assumed in Grover's proof of unstructured database search speedup [44, 45]).

However, quantum information has properties that make it more difficult to deal with, in some ways, than classical information:

- The uncertainty principle says that if A and B are complementary (noncommuting) observables, then measuring A will affect the result of any subsequent measurement of B. The act of acquiring information about a quantum system unavoidably disturbs the system: this is called the *disturbance principle*. This can lead to higher-than-desired error rates.

- Quantum information cannot be copied with perfect fidelity (this is the *no-cloning principle* [32, 114]); for, if it could be, then we could measure the copy without disturbing the original, so breaking the disturbance principle.
- John Bell [9] showed that quantum information is, because of entanglement, typically encoded in nonlocal correlations among different parts of the system: something that has no analogy in classical systems, and the main reason that we cannot efficiently simulate a quantum system with a classical computer.
- These nonlocal correlations are extremely unstable in practice, because of *decoherence*: the system interacts with its environment and its information becomes entangled with the information in the environment; since most of the information is in the nonlocal interactions, we lose the ability to recover the original system's information. Hence, long coherence times are crucial for practical quantum computers. There is a difficult balance to be struck between allowing qubit states to be controlled from outside the system and protecting them from decoherence. Shor [98] discovered quantum error correction (QEC) techniques to counteract the leakage from decoherence, essentially reducing the decoherence rate to acceptable levels. However, control of errors and noise remains a major issue [19, 53].

Digital Quantum Computer Hardware

Quantum computers are generally used together with classical computers in a hybrid approach. The classical computer controls the program flow and carries out preprocessing and postprocessing, only passing particular tasks to the quantum computer, namely, those for which there is quantum advantage. Such algorithms "variationally" update the variables of a parameterised quantum circuit and so are called *variational quantum algorithms* (VQAs) [12]. The objective function is the cost function to be variationally minimised, often a Hamiltonian or an expected value of a Hamiltonian, written as a linear combination of Pauli strings (tensor products of Pauli matrices $\hat{\sigma}_x$, $\hat{\sigma}_y$, $\hat{\sigma}_z$). For further details, see [12, Sect. II]. Converting a real-world objective into a VQA form is generally a nontrivial task, and approximations may be needed: see section "SQA Application to TSP-Type Problems" for an analogous case when applying simulated quantum annealing to the TSP (travelling salesperson problem).

Over the last two decades or so, standard quantum computers have been built. One of seven qubits, implemented using nuclear magnetic resonance (NMR) applied to seven chosen nuclei of 10^{18} identical molecules, used Shor's algorithm to factorise 15 as 3×5 [109]. The record size of an NMR digital quantum computer is 12 qubits [81]. However, other implementations of qubits have superseded the NMR approach, including two-state systems such as superconducting circuits (persistent circulating currents), trapped ions, neutral atoms, Rydberg atoms, photons in wells, nitrogen-vacancy centres, silicon quantum dots, photon polarisation, electron spins, or internal atom states [12, 62]. (Superconducting qubits are spin $\frac{1}{2}$ systems on silicon, made up of lithographically defined circuit elements: inductors, capacitors,

connections, Josephson tunnel junctions, etc. They are scalable, are controllable by electrical signals, have the possibility of high coherence, and have a superconducting digital logic: all of these factors make them attractive for the realisation of qubits [51].)

In recent years, there have been major advancements in trapped-ion devices including high-fidelity entangling gates and all-to-all connectivity [12]. By 2017, IBM had a 50-qubit quantum computer that could avoid decoherence for 90 microseconds. By 2019, Google AI Quantum had a 53-qubit "Sycamore" device in use, using transmon qubits (a kind of superconducting charge qubit), and showed a single-qubit gate fidelity of 99.85% and two-qubit gate fidelity of 99.64% on average in experiments with it [6]. By 2023, IBM had the 1,121-qubit Quantum Condor chip and Atom Computing had a 1,125-qubit machine. IBM will use its new Quantum Heron processors, with just 133 qubits each, for its "next-generation" system two quantum computer (which uses multiple simultaneous quantum processors) because of their superior error properties. At the time of writing (early 2025), there appear to be dozens of implementations of qubits, with different levels of error control, and it is not clear which – if any – will become the standard implementation.

The Current Situation: NISQ Computers

In 2018, Preskill [88] introduced the term *noisy intermediate-scale quantum* (NISQ) to describe current quantum computers. These contain of the order of a hundred (in 2018) or a thousand (at time of writing) noisy qubits, i.e. non-error-corrected qubits; thus, they carry out imperfect operations. They also have a shorter-than-desirable coherence time, typically of the order of several tens of microseconds, though some have reported almost a minute of coherence using neutral Ytterbium atoms with nuclear spin used as the qubit 0 or 1 [113]. However, an efficient universal fault-tolerant quantum computer would require millions of qubits [59] with long coherence times and low error rates. Such quantum computers are likely decades away [12].

Preskill [88] proposed reasons to believe that quantum advantage could become a reality. It is now an engineering problem to develop globally phase-coherent error-tolerant quantum computers of thousands to millions of qubits or more (which could break current encryption standards in an acceptable time); however, decoherence makes this problem very difficult. The most important quantum algorithms, e.g. Shor's factorisation algorithm [97, 99], Grover's search algorithm [44, 45], and the HHL algorithm for solving a system of linear equations [49], need error-corrected qubits to run. Currently, even the best current quantum error correction schemes on n qubits, which allow connectivity among all qubits and simultaneous execution of gates, suffer a slowdown of at least $O(\sqrt{n})$ [53].

However, some recent results [63] suggest that it may be possible to construct qubits with a much lower error rate, which could reduce the number of qubits needed for quantum advantage from millions to thousands or even hundreds.

Nevertheless, it is important to emphasise that at time of writing, quantum technology, although promising, does not have the same strength of optimisation capabilities as classical computing does, by a number of measures such as size

of problem that can be tackled, reliability of the solution procedure, and cost of the computing equipment. While it has obvious potential, it currently has strong limitations.

Adiabatic Quantum Computation

More generally, we can build quantum computation systems which are not made up of qubits. Such quantum computers are not necessarily "digital": they evolve with time and are closer in spirit to the original analogue computers, in that a physical system modelling the computation to be performed is set up, and then allowed to evolve with time. This approach, adiabatic quantum computing (AQC), began in solving optimisation problems and has become an alternative to the standard qubit quantum computing. It is universal in that it can efficiently simulate any quantum computer. It can be applied to any constraint satisfaction problem (CSP) where, given a set of constraints applied to the input bits, the problem is to output an assignment of values to the input bits that maximises the number of satisfied constraints. Many CSPs are NP-complete and of practical importance.

The *quantum adiabatic theorem* [15] describes the behaviour of a quantum system when its Hamiltonian (total energy) varies slowly in time:

Theorem 1 (Born, Fock) *A perturbed system remains in its instantaneous eigenstate if the perturbation is applied sufficiently slowly, and there is a gap between that state's eigenvalue and the Hamiltonian's other eigenvalues.*

This says that a quantum system subjected to sufficiently slowly varying external conditions can modify its functional form so as to remain in the same relative pure eigenstate of the system Hamiltonian at all times; however, if the conditions change rapidly, the functional form of the state does not have enough time to adapt, so it stays unchanged, ending in a linear superposition of states, i.e. *not* an eigenstate, of the final Hamiltonian. A *quantum adiabatic evolution* is one where the system is varied slowly enough that Theorem 1 applies, and the system stays in the same eigenstate of $\mathcal{H}$; often this is the ground state (the lowest energy state). These evolutions are the main idea of *adiabatic quantum computing* (AQC):

- The Hamiltonian is viewed as a multivariate cost function $\mathcal{H}_{\text{pot}}$ to be optimised: problem variables map to Ising spins (in, say, the z direction).
- If energy landscape barriers are high but very narrow, quantum-mechanical fluctuations in an observable because of its noncommuting with the Hamiltonian can help in tunnelling through the barriers: so a high strength transverse "tunnelling" field (in, say, the x direction) is applied which adds a noncommuting quantum kinetic term $\mathcal{H}_{\text{kin}}$.
- Begin with a quantum state that is the uniform superposition over all possible solutions to the optimisation problem; roughly speaking, this gives a very flat energy landscape for which the Hamiltonian easily reaches its ground state.

- Then slowly reduce the field strength to 0, allowing the Hamiltonian to find the ground state of the desired system (provided there is no crossing of energy levels with the ground state during the evolution).

We consider this in more detail in section "Quantum Spin Glasses" once we define Ising spins.

Because of its similarity to heat annealing of metals (section "Annealing and Spin Glasses"), AQC is also known as quantum annealing (QA). The adiabatic theorem guarantees that the result of the computation will be correct, so long as the computation (perturbation) is carried out sufficiently slowly. However, QA lacks general, rigorous worst-case upper bounds on its runtime: for certain instances of the constraint satisfaction problem on n bits, the time required for the adiabatic evolution can be exponential in n [77].

It has been shown [4] that an adiabatic quantum computer can perform any computation achievable by a "standard" qubit-based quantum computer: they are polynomially equivalent in power. As with classical analogue computers, devices based on AQC generally need to be tailored to the specific problem they are being applied to. For more details, see [5].

Applications of Adiabatic Quantum Computation

There have been a range of applications of AQC. Santoro and Tosatti [94] describe how QA/AQC may be used, by a transformation of the time variable, to address optimisation problems in classical physics. See also [102, 103]. The most important programmable quantum annealing platform is D-Wave annealers which have been used on a number of practical problems [14, 110]. However, this has some issues, e.g. requiring first solving the NP-hard minor embedding problem [110]; and there does not exist definitive evidence thus far that this form of AQC involves quantum effects or that it provides discernible speedup over classical computers [110]. This said, Choi [25] uses adiabatic quantum computers to successfully solve NP-hard quadratic unconstrained binary optimisation (QUBO) problems – involving minor embedding – on a graph G. QA has been applied to machine learning, including deep learning [10] and machine learning applied to computational biology [69]. Another application is that of [67], who use free-energy-based AQC applied to reinforcement learning.

Hardware Realisations

Experimental results [3, 18] on a TISG (a sample of $LiHo_{0.44}Y_{0.56}F_4$) show that the relaxation behaviour depends on the path taken. The sample was brought from a high-temperature paramagnetic phase to a low-temperature glassy phase along two paths from $(0.8, 0)$ to $(0.02, 7)$ in the T-Γ plane. Along the classical annealing path, the transverse field Γ was kept at 0, while T was slowly reduced; then Γ was switched on. Along the quantum annealing path, Γ was kept high, and then T was reduced, with Γ being reduced (slowly, i.e. adiabatically) only on reaching the final temperature. It was found that the quantum approach gave states with relaxation up to 30 times faster than thermal annealing. This indicates that quantum annealing is

much more effective at exploring the configuration space in the glassy "disordered" phase than thermal annealing. Crucially, it has been found [95] that even in the presence of all three ingredients of a spin glass – frustration, randomness, and long-range dipolar interactions – the spin-glass phase of $LiHo_xY_{1-x}F_4$ is destroyed by any nonzero transverse field, which demonstrates that quantum tunnelling can explore a rugged potential energy landscape with high barriers.

The current leading quantum annealers (e.g. D-Wave) use superconducting qubits and superconducting digital logic in part for control [51].

Current limitations of quantum annealing include: short coherence times (compared to digital quantum computers); limited connectivity (usually to four neighbouring Ising spins/qubits); only two-body interactions (thus only QUBO problems can be modelled, not higher degree polynomial unconstrained binary optimisation problems or PUBOs); limited ability to use different annealing schedules for different Ising spins/qubits. It is still unclear whether quantum annealers will provide quantum advantage for a general class of problems, and even whether overcoming the limitations listed will provide quantum advantage [51].

Annealing and Spin Glasses

In *annealing*, to achieve desirable physical characteristics, a material (e.g. a metal) is first heated and then slowly cooled. By adding energy to the material during heating, its atoms can take on random or *disordered* states. If the material is cooled slowly enough (by an adiabatic process), it remains approximately in thermodynamic equilibrium. (When thermodynamic equilibrium is maintained, the process is called *adiabatic*; hence the use by analogy of the word "adiabatic" in adiabatic quantum computation, where the system is slowly perturbed.) It will settle or *freeze* into a minimum energy crystalline structure with the individual atoms arranging themselves into a regular array.

If the cooling is performed too quickly (*quenching*), the metal will freeze into a local, rather than global, minimum energy state, and the atoms will not form a regular array. This can lead to a state with faults and defects in the metal, meaning the metal fractures if put under enough stress. This state or *phase* of the metal is analogous to the disordered but solid structure of glass. The annealed phase is analogous to a regular, ordered crystal. (This use of "phase" is not related to either phase space or the phase of a wave.)

A particular case of annealing, used as inspiration in simulated annealing (see section "Simulated Annealing"), is where a ferromagnetic alloy is slowly cooled to give a magnet. Physically, magnetism is caused by rotating particle charges (spins). Each such spin gives rise to a microscopic dipole which may be aligned either up or down along a given axis. Above a critical *Curie* temperature, T_c, the minimum of free energy $A = \mathcal{H} - TS$ occurs at states with no magnetisation: the spins are completely disordered. As the temperature is reduced below T_c, a phase change occurs. Ordered clusters start to appear and the material begins to show magnetism;

the minimum of A shifts towards states with higher order of magnetisation. Finally, at $T = 0$, the global minimum of free energy (and hence the global minimum of energy $\mathcal{H} = A + TS$) occurs at the highest order of magnetisation; the microscopic spins have all become aligned.

Spin glasses are a phase of magnetic alloys, so named because they bear the same relation to magnets as glasses do to crystals. In rapidly cooled or *quenched* spin glasses, different spin-spin interactions are randomly ferromagnetic or antiferromagnetic and frozen in time, called *frozen disorder*. This disorder causes competing interactions among the spins, called *frustration*; no spin state is able to satisfy all of the interactions. Because of this, the potential energy landscape becomes very rugged, with local and global minima separated by potential energy barriers. This disorder and frustration is reminiscent of the search landscapes occurring in applications, such as combinatorial optimisation problems; this motivates adapting features of the physics of spin glasses to search algorithms for these applications. In particular, the frustration effect is analogous to the conflicting constraints often encountered in optimisation applications. There is a large body of work on the physics of spin glasses, giving a good understanding of the structure of the spin-glass energy landscape, and of the slow (glassy) dynamics of many-body systems in the presence of frustration and disorder: thus, it can aid our understanding of certain computationally challenging problems. Research has found annealing approaches particularly useful for combinatorial optimisation problems: this is described in more detail in sections "Simulated Annealing" and "Simulated Quantum Annealing" and elsewhere in this book.

Kirkpatrick et al. [61] note that slow annealing of a material may avoid the frustrated spin interactions of a quenched spin glass, by allowing the material to take on a regular noncompeting pattern of spins, settling into a ferromagnet phase rather than a spin-glass phase. However, Hamiltonians encountered in applications may have a very stable spin-glass phase, even at low temperatures, with many degenerate near-to-ground states of nearly equal energy. Thus, even in the case of frustration, there will be many near-optimal solutions and a good search algorithm should find one or some of them.

Ising Spin Glasses

In statistical mechanics, a common model used for (among other things) spin glasses is the *Ising model*. It models a discrete collection of spins, each of which can take the value 1 or -1. Usually, the spins are the lattice points in a regular grid of dimensions 1, 2, or 3. The spins S_i are considered to interact in pairs, with an interaction (i.e. potential) energy J_{ij} which has one value when the two spins are the same (*aligned*), and another value when the spins are different (anti-aligned). The product of two aligned spins is 1, and the product of two anti-aligned spins is -1. The energy of a configuration of N such spins is modelled as

$$\mathcal{H} : \{-1, 1\}^N \longrightarrow \mathbb{R} \quad \text{with} \quad \mathcal{H} = -\sum_{i>j}^{N} J_{ij} S_i S_j. \tag{11}$$

The configuration space (of order 2^N) is a Cartesian product of N copies of $\{-1, 1\}$ and is in one-to-one correspondence to the search space of an evolutionary algorithm using a bit representation, with $\mathcal{H}$ playing the same rôle as a fitness or objective function.

Since the spins interact pairwise, they may be modelled as vertices of a weighted graph, with an edge (of weight J_{ij}) between vertices i and j if and only if $J_{ij} \neq 0$. This weighted graph completely characterises the Ising model.

In two or more dimensions, the Ising model undergoes a phase transition between an ordered and a disordered phase. This transition occurs at a critical temperature, T_c, above which the spins are disordered and below which is the ordered ("spin glass") phase. Below T_c, *freezing* occurs.

Quantum Spin Glasses

A quantum spin glass is a classical spin glass with an extra kinetic (quantum tunnelling) term added to the Hamiltonian. Since each Ising spin has two possible states, it can be viewed as a qubit. In quantum spin glasses, both thermal and quantum fluctuations can cause the order-disorder phase change. There are two kinds of quantum spin glasses: vector spin glasses, where the fluctuations cannot be affected by adjusting a laboratory field, and classical spin glasses perturbed by a quantum tunnelling term (the useful kind), where a transverse laboratory field Γ can affect the fluctuations. This transverse Ising spin-glass (TISG) model is the most studied [23]. The term *quantum annealing* is used for this "annealing" by reduction of the transverse field (by analogy with thermal annealing). As well as being able to tunnel through very high but narrow barriers (which makes the energy landscape more accessible to local moves), quantum annealing has the ability to "sense" the whole configuration space simultaneously via a delocalised wavefunction.

The Quantum Annealing Hamiltonian

The physical idea behind QA is to move the system from some initial Hamiltonian ground state (the "kinetic energy") to the ground state of the problem Hamiltonian (the objective: "potential energy"). In both SA and (S)QA, the kinetic term is gradually reduced to zero, according to an annealing schedule, as the algorithm progresses (respectively, by reducing temperature or the strength of a quantum field), so that by the end of the algorithm's run, the total energy that the algorithm has minimised is equal to the potential term, that is, the objective function.

In QA, to construct the kinetic term, we use noncommutative operators – a transverse magnetic field – to generate artificial quantum degrees of freedom. These induce quantum fluctuations, whose strength we control using the annealing schedule. This annealing is similar in spirit to SA, but rather than decreasing the temperature as in SA, we decrease the magnetic field strength.

Given a real-world problem, it must first be converted into a form appropriate for application of QA. A problem already in the quadratic unconstrained binary optimisation (QUBO) form is well suited, since its objective function can straightforwardly give a suitable Hamiltonian in terms of an Ising model. Many operational research combinatorial problems, including network flow problems, scheduling, sequencing and allocation problems, partitioning and clustering problems, and other graph problems can be described in QUBO terms [40]. A stream of work has been developed on applying quantum and quantum-inspired metaheuristic solvers to QUBO and the more general class of problems already in the polynomial unconstrained binary optimisation (PUBO) form; see, for example, [33, 34, 41, 42]. In particular, [33] explore the use of a special solver, Q-Card, belonging to the QUBO-Plus class, and [34] consider other quantum-inspired metaheuristic solvers that address QUBO and PUBO problems, including solvers such as the next-generation quantum (NGQ) solver for QUBO models and the NGQ-PUBO solver for PUBO models; these attain the performance of CPLEX and the D-wave quantum advantage solver.

For an n-vector of binary variables $x = (x_1, \ldots, x_n)^t$, a constant n-vector $c = (c_1, \ldots, c_n)^t$, and an $n \times n$ real matrix $A = (a_{ij})$ (the entries of c and A are derived from, and represent the constraints of, the particular problem), QUBO is defined as

$$\text{Find } x \text{ which minimises } x^t A x + c^t x. \tag{12}$$

QUBO problems can be represented by an interaction graph, where node i has node weight c_i (a bias) and edge ij has weight a_{ij} (a coupling coefficient). The goal is to find $x^* \in \{0, 1\}^n$ that encodes the optimal solution.

Suppose that we have a quantum Ising spin glass with spins aligned in the z direction. The binary variables $x_1, \ldots, x_n$ are associated to qubits $q_1, \ldots, q_n$: these qubits are the Ising spins, often represented by $\hat{\sigma}_z^i$, the Pauli matrix for Ising spin i (standing for the earlier S_i). Then the bias c_i captures the tendency for qubit q_i to end up in state 0 or 1, and two qubits q_i and q_j are pairwise coupled with an adjustable strength a_{ij}.

By mapping x_i to $\frac{1}{2}(1-\hat{\sigma}_z^i)$, we can convert this QUBO problem into the problem of finding a ground state of the diagonal n-qubit Ising Hamiltonian:

$$\mathcal{H}_{\text{QUBO}} = -\sum_{i,j} J_{i,j} \hat{\sigma}_z^i \hat{\sigma}_z^j - \sum_i h_i \hat{\sigma}_z^i, \tag{13}$$

where $J_{i,j} = -\frac{1}{4} a_{ij}$ and $h_i = \frac{1}{2}(-c_i + \sum_j a_{ij})$ [12]. This will be the basis of a potential energy term.

Now consider applying a strong transverse magnetic field in (say) the x direction: call this the base Hamiltonian $\mathcal{H}_0 = -\sum_i \hat{\sigma}_x^i$ (this will be the basis of a kinetic energy term). Assume $\mathcal{H}_0$ varies smoothly with time. Let time t run from 0 to T, where T is the time needed for an adiabatic evolution to complete.

We weight $\mathcal{H}_0$ by a real-valued function $A(t)$ where $A(0) = 1$ and $A(T) = 0$; $A(t)$ is the strength of the kinetic term $A(t)\mathcal{H}_0$ and is called the tunnelling strength or quantum field strength (sometimes denoted by $\Gamma(t)$). We weight $\mathcal{H}_{\text{QUBO}}$ by another real-valued function $B(t)$ where $B(0) = 0$ and $B(T) = 1$. Beginning with the ground state of $\mathcal{H}_0$ (at time $t = 0$, $A(t) = 1$, $B(t) = 0$), solving the QUBO problem on a quantum annealer means carrying out the annealing schedule as t goes from 0 to T for the combined Hamiltonian based on (11):

$$\mathcal{H}(t) = A(t)\mathcal{H}_0 + B(t)\mathcal{H}_{\text{QUBO}} = \mathcal{H}_{\text{kin}} + \mathcal{H}_{\text{pot}}. \tag{14}$$

Figure 2 (based on [62, Fig. 4]) illustrates the process of quantum annealing (of transverse field strength). For more details, see [12, 62], and D-wave's own documentation at https://docs.dwavesys.com/docs/latest/c_gs_2.html.

King et al. [60] "suggest that spin glasses can be brought into low-energy states faster by annealing quantum fluctuations than by conventional thermal annealing ... reproducing this phenomenon in a programmable system has remained a central challenge ... we achieve this goal by realising quantum critical spin-glass dynamics on thousands of qubits with a superconducting quantum annealer."

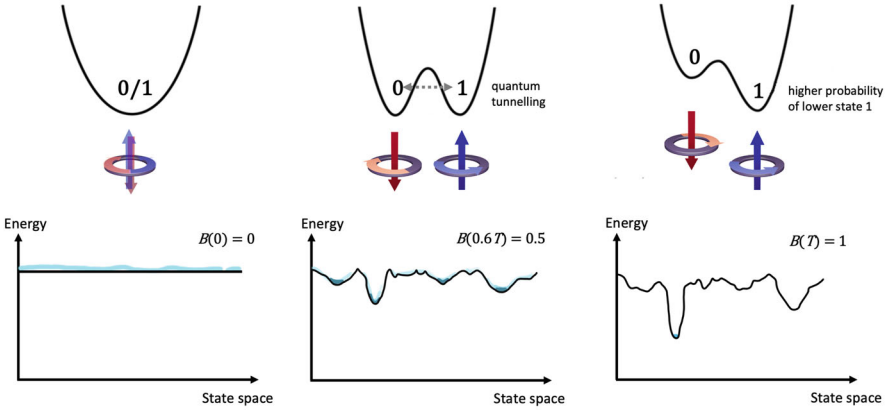


Fig. 2 The process of quantum annealing. Initially, a strong transverse magnetic field is applied to a superposition of binary quantum states (left); as the problem space is explored, the bias weight c_i gradually raises the energy barrier between the two possible states 0 and 1 of each qubit, but tunnelling means quantum fluctuations can penetrate through potential barriers (centre); during annealing, the transverse magnetic field – and so the quantum fluctuations – is reduced until when $B(t) = 0$ at time $t = T$ the system state is the optimum for this objective function (right)

The Metropolis(–Hastings) Algorithm

Simulated annealing and simulated quantum annealing (see sections "Simulated Annealing" and "Simulated Quantum Annealing," respectively) both use a Monte Carlo sampling approach due to Metropolis et al. and extended by Hastings [50,76], which incorporates controlled uphill steps, as well as downhill steps, in the search for a minimum. This Metropolis(-Hastings) algorithm (Algorithm 1) can draw samples from any probability distribution $P(x)$, and generates a Markov chain in which each state x_{t+1} depends only on the previous state x_t. It uses a *proposal density* $Q(x'|x_t)$, which depends on the current state x_t, to generate a new proposed sample x'. (The proposal density could be, e.g. a Gaussian centred on the current state x_t.) The new proposal x' is "accepted" as the next value x_{t+1} if a number a drawn (this is the "Monte Carlo" aspect of the algorithm) from the uniform distribution on [0, 1] satisfies (see [50])

$$a < \frac{P(x')Q(x_t|x')}{P(x_t)Q(x'|x_t)}. \tag{15}$$

Otherwise (the proposal is not accepted), the current value is kept: $x_{t+1} \leftarrow x_t$. In this and subsequent algorithms, the terminating condition can be based on some or all of a maximum number of iterations; a convergence tolerance test; or limits on time, memory, or other resources.

Algorithm 1: Metropolis(–Hastings) algorithm

```
1  begin MetropolisHastings(P, Q)
2      Choose starting x_0;
3      repeat
4          Generate proposed x' based on x_t;
5          Let α ← P(x')Q(x_t|x') / P(x_t)Q(x'|x_t);
6          if α > 1 then
7              x_{t+1} ← x';
8          else
9              x_{t+1} ← x' with probability α, otherwise x_{t+1} ← x_t;
10         end if
11         Increment t;
12     until terminating condition;
13     return x_t;
14 end MetropolisHastings.
```

Because P only occurs in the ratio $\frac{P(x')}{P(x_t)}$, we only require that a function proportional to the density can be calculated. In our case, the function is usually (though not necessarily) taken as the Boltzmann factor $e^{-\beta \mathcal{H}_x}$. (The proportionality constant is then the partition function Z, though the Metropolis algorithm does not need to compute it.) If Q is symmetric, that is $Q(x_t|x') = Q(x'|x_t)$, as in

Metropolis et al.'s original algorithm [76], then it suffices to compute $\frac{P(x')}{P(x_t)}$. If Q is not symmetric, then the more complicated formula $\frac{P(x')Q(x_t|x')}{P(x_t)Q(x'|x_t)}$ (as in [50]) must be used.

To apply this to Ising spin glasses, we note that the Ising model is statistical in nature, and, as in section "Statistical Mechanics," the probability of a given configuration x of spins is the normalised Boltzmann factor with inverse temperature β:

$$P(x) = \frac{1}{Z}\mathrm{e}^{-\beta\mathcal{H}}. \tag{16}$$

To generate configurations for simulation using this probability distribution, a variant of the Metropolis algorithm (Algorithm 2) may be used.

Algorithm 2: Metropolis algorithm for Ising spin models

```
1  begin MetropolisHastingsIsing(initial spin configuration)
2      repeat
3          Randomly pick a spin;
4          Calculate the contribution to the energy due to this spin;
5          Flip the value of the spin and calculate the new contribution;
6          Let Δℋ = new energy − previous energy;
7          if Δℋ < 0 then
8              Keep the flipped value;
9          else
10             Keep the flipped value with a probability e^{−βΔℋ}, otherwise keep the
                original value;
11         end if
12     until terminating condition;
13     return final configuration;
14 end MetropolisHastingsIsing.
```

The change in energy depends only on the value of the spin and its nearest neighbours in the Ising model graph. Thus, if the graph is sparse, the algorithm is fast. This process will eventually produce a choice from the distribution.

Simulated Annealing

Simulated annealing (SA) [21,22,61,85] is a global optimisation algorithm inspired by the physical process of thermal annealing of metals or glass (section "Annealing and Spin Glasses"). In the context of designing a search algorithm, the physical process of thermal annealing can be viewed as the search for a global minimum of an energy landscape. By analogy, the SA algorithm tries to find a global minimum of an objective function. This objective function could be defined on $\mathbb{R}^n$ or, more generally, on a space of "states" (or candidate solutions), e.g. valid tours of the network in a TSP. Simulated annealing is treated in detail elsewhere in

this book; in this chapter, we discuss it mainly in the context of similarities to and differences from simulated quantum annealing (SQA, section "Simulated Quantum Annealing").

A simulated annealing algorithm can be run using a single trial solution which is modified randomly during each iteration of the algorithm. Assuming the goal is to minimise the objective function, the search process usually proceeds "downhill," from a *higher* to a *lower* energy state. Modifications that improve the current trial solution are always accepted. However, occasional "wrong-way," or uphill, moves are accepted with a nonzero probability. This can allow the search process to escape a local minimum, which would entrap a standard hill climbing or other greedy approach.

The SA algorithm is based on the Metropolis algorithm 1 of statistical mechanics with the addition of a temperature schedule which controls the probability p of acceptance of uphill moves. This permits a gradual tuning of the exploration-exploitation balance away from exploration towards more intensive exploitation as the search progresses. A basic SA algorithm is given as Algorithm 3.

Algorithm 3: Simulated annealing algorithm

```
1  begin SimulatedAnnealing
2      Set initial temperature and determine the annealing schedule;
3      Generate an initial feasible solution and assess its cost;
4      repeat
5          Randomly generate new solution within a neighbourhood of the current solution;
6          Assess cost of new solution;
7          if cost of new solution < cost of previous solution then
8              Accept the new solution;
9          else
10             Accept the new solution with a probability p, otherwise keep the old
               solution;
11         end if
12         Adjust temperature according to the annealing schedule;
13     until terminating condition;
14     return current solution;
15 end SimulatedAnnealing.
```

This approach, with suitable parameter modification, allows for discrimination between the objective function's large-scale behaviour (at higher temperatures, near the start of the algorithm's run) and its finer-scale behaviour (at lower temperatures, later in the run). Earlier in the run, larger features of the system appear. The algorithm finds a broad area in the search space where a global optimum should exist, following the large-scale behaviour without regard to small-scale local optima found on the way. At lower temperatures, the finer details of this area emerge, and, with high probability, the algorithm will find a near-global optimum, or even the global optimum.

As noted earlier (section "Thermodynamics"), thermodynamic equilibrium corresponds to minimising (Helmholtz) free energy, $A = \mathcal{H} - ST$, with S being entropy and T being temperature. When $T = 0$, this reduces to the Hamiltonian. Since SA reduces a temperature parameter to 0, it may be thought of as minimising either (or both) the free energy or the Hamiltonian. Of course, in search algorithms such as SA or SQA, we may modify the Hamiltonian, e.g. by adding extra terms.

Search and Neighbourhoods

As with any search algorithm, the way in which moves are made from one state (or "point" in the search space) to another is crucial. In SA, as in SQA, a *neighbourhood structure* is defined. Let X denote the set of all possible states (assumed finite). For each state $x \in X$, there is a set N_x of states to which transitions from x may happen. N_x is called the *set of neighbours* or *neighbourhood* of x. We require that the neighbour relation be symmetric, i.e. $x \in N_y$ if and only if $y \in N_x$. In general, the neighbours of x (the possible moves) are specified by the user of the algorithm, in a way that depends on the particular problem. This neighbourhood structure defines a graph on the search space: the vertex set is the set of states X; and there exists an edge xy if and only if $x \in N_y$ (or, equivalently, $y \in N_x$); thus, the graph is undirected. The performance of the SA algorithm depends crucially on the choice of neighbourhood structure. Usually in SA (and in SQA), the neighbour of x is generated by making a small change (perturbation) to x: thus these algorithms are generally *perturbative* in nature.

Acceptance of "Bad" Moves

A key component of SA is the rule for accepting "wrong way" moves: moves which disimprove the solution. Traditionally (by analogy with the Boltzmann factor described in section "Quantum Statistical Mechanics," with Boltzmann's constant k_B scaled to 1) in SA, the acceptance probability for these cases is taken as

$$p_i = \mathrm{e}^{-\Delta f / T_i}, \tag{17}$$

where T_i is the *temperature* at iteration i and Δf is the difference in the values of the objective function at the current and previous trial solutions: $\Delta f > 0$ if the current solution is worse than the previous one.

The higher the value of T_i, the more likely that a poorer solution will be accepted. Thus, T is a tuning parameter. The value of T is varied during the algorithm's run according to an annealing or *cooling schedule*.

Simulated Quantum Annealing

The *simulated quantum annealing* (SQA) optimisation algorithm is inspired by the physical process of adiabatic quantum computing or quantum annealing (section "Quantum Spin Glasses"). (We call this algorithm the *simulated* quantum annealing algorithm, to avoid confusion with the physical quantum annealing process; however, particularly in the physics literature, the name "Quantum Annealing" seems to be used for both meanings.) It uses quantum fluctuations rather than the thermal fluctuations of SA to explore the search landscape. As in SA, a candidate solution is regarded as a system "state" and the objective function value is regarded as "energy," i.e. the Hamiltonian. Simulated quantum annealing was proposed by [37].

SQA is based on quantum tunnelling (section "Tunnelling"). In optimisation terms, this corresponds to making a jump on a landscape. This capability can permit a solution to "tunnel" through a (narrow) barrier on the landscape in one jump, rather than having to climb over the barrier by a succession of wrong-way moves as in SA. In some landscapes (Fig. 3), e.g. landscapes that often arise in combinatorial optimisation, thermal-annealing-inspired algorithms such as SA can have difficulties in escaping from poor-quality local minima, as they may be surrounded by high – though possibly narrow – barriers. The ability of a thermal fluctuation to escape a barrier depends only on the height of the barrier. Entropy is also a problem: the number of configurations grows exponentially with the number of variables; there are 2^n configurations for n Ising spins. A classical algorithm can only examine one configuration at a time; if there is no "guiding" gradient to roughly steer the algorithm from the start point towards the global optimum (which can happen, e.g., in deceptive landscapes), the algorithm must visit a significant proportion of the 2^n possible configurations. In this case, SA is little more effective than random search.

Let us use the term neighbourhood radius for the average distance between the current state x and the next x'. In SQA, as in SA, the current state x is replaced by a randomly selected neighbour state x' if x' has lower energy. The tunnelling field strength is a parameter that controls the radius of the neighbourhood explored by the

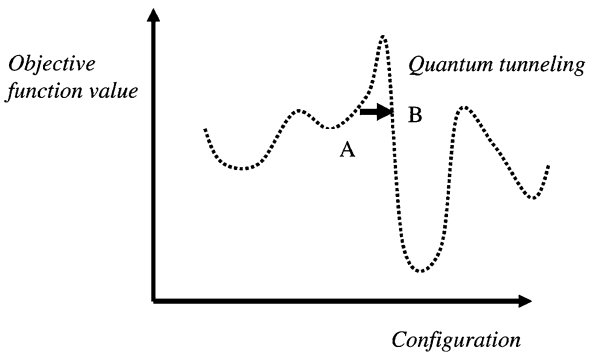


Fig. 3 Quantum tunnelling on a landscape. SA would require several "wrong way" moves to jump over the barrier between the basins of the local optimum (A) and the global optimum (B). With quantum tunnelling, this movement can potentially occur in one step

algorithm. The field strength starts at a high value: here, the neighbourhood covers the whole search space, which allows long jumps across the landscape, helping exploration. The field strength is slowly reduced during the algorithm's run, which facilitates local exploitation. Eventually the neighbourhood shrinks to include only a few states near to the current state.

Both SA and SQA are based on the physics of adiabatic processes, i.e., on systems that stay at or near thermodynamic equilibrium all through their evolution. Oliveira [35] gives more detail on this connection. Additionally, SA and SQA are both based on the Metropolis algorithm, with the SA temperature parameter playing a similar rôle to that of the SQA tunnelling strength. This means they are both stochastic local search algorithms and both generally use a perturbative approach to neighbour selection. However, in standard SA (as opposed to adaptive SA), the neighbourhood radius stays the same during the search, and the temperature controls the probability of moving to a higher energy state, whereas in SQA, it is the tunnelling field strength that controls the neighbourhood radius. The ability to tunnel through barriers can be shown to improve the "ergodicity" (section "Ergodicity") of SQA (compared to SA, which relies on thermal fluctuations) and so enhance landscape exploration [30]. The appropriate definition of the tunnelling field depends on the problem under consideration.

Implementation of SQA

To implement a quantum-inspired version (SQA) of quantum annealing (QA) on a classical machine, we must somehow simulate the quantum tunnelling effect classically. Feynman first pointed out (see [94]) that a quantum-mechanical system may be viewed as a classical system embedded in a world with one extra dimension, called imaginary time. The Suzuki-Trotter formalism maps a d-dimensional quantum Hamiltonian to an effective $(d+1)$-dimensional classical Hamiltonian, with the extra time-like dimension (called the *Trotter dimension*) discretised into M steps. At each index $k = 1, \ldots, M$ along this new dimension, there is a d-dimensional *Trotter slice*, and the Hamiltonian now represents a system of spins in a $(d+1)$-dimensional lattice. Now the Suzuki-Trotter formula (9) can be used to compute the partition function

$$Z = \mathrm{Trace}(\mathrm{e}^{-\mathcal{H}/T}) = \mathrm{Trace}(\mathrm{e}^{-(\mathcal{H}_{\mathrm{pot}}+\mathcal{H}_{\mathrm{kin}})/T}), \tag{18}$$

and this eventually gives the effective classical Hamiltonian in the Mth Trotter approximation as

$$\mathcal{H} = \sum_{i>j}^{N} \sum_{k=1}^{M} K_{ij} S_{ik} S_{jk} - K \sum_{i=1}^{N} \sum_{k=1}^{M} S_{ik} S_{ik+1}, \tag{19}$$

where $K_{ij} = J_{ik}/(MT)$, $K = \frac{1}{2} \ln \coth(\Gamma/(MT))$, k is the index in the extra Trotter dimension, i is the position in the original Ising model, and S_{ik} is the Ising spin on the lattice site (i, k). In theory, this equivalence of quantum Hamiltonian with a one-dimensional higher classical Hamiltonian only holds exactly as $M \to \infty$, but in practice there is an optimum M. In units chosen so that $\hbar = 1$, M should be of the order of $1/T$ (of course, $M \to \infty$ as $T \to 0$).

One can then study and optimise the equivalent classical Hamiltonian using Monte Carlo techniques, as in Algorithm 2, which leads to simulated quantum annealing (SQA). Path integral Monte Carlo (PIMC) and Green's function Monte Carlo (GFMC) have been used here [93]. In general, the implementation is quite technical.

One can study the thermodynamics of a d-dimensional classical system by reducing it to the study of the ground state(s) of a d-dimensional quantum system, the reverse of the above use of an equivalent classical Hamiltonian [94, 102]. SA itself can be "simulated" using a quantum algorithm, quantum simulated annealing or QSA (QSA is a *quantum* algorithm, very different from the *classical* algorithm, SQA.); when SA requires $1/\delta$ steps to get within a certain error bound of the optimum, then QSA will only require $1/\sqrt{\delta}$ steps [103].

Implementing SQA means using adjustable quantum fluctuations in the problem rather than thermal ones. For a given optimisation problem, the main tasks in developing an SQA approach are:

i. How to describe (and, usually, simplify) the Hilbert space for the problem. Real-world optimisation problems generally involve enormously large-dimension Hilbert spaces, so an alternative quantum Monte Carlo approach, often using path integrals, is required.
ii. How to formulate a quantum Hamiltonian $\mathcal{H} = \mathcal{H}_{\text{pot}} + \mathcal{H}_{\text{kin}}$, where the potential energy term $\mathcal{H}_{\text{pot}}$ is typically the cost or other objective function to be minimised, and the kinetic energy term is a perturbation term: a noncommuting operator analogous to the physical transverse field. It is useful to write $\mathcal{H} = \mathcal{H}_{\text{pot}} + \Gamma(t)\mathcal{H}_{\text{kin}}$, thus explicitly showing the time dependence of the tunnelling field strength: $\Gamma(t) \to 0$ as $t \to \infty$.

Thus SQA is more flexible than SA since not only can the "annealing schedule" be chosen, but there may be a range of choices for the kinetic term which is then weighted by $\Gamma(t)$.

Applications of SQA

SQA Application to TSP-Type Problems

An instance of the travelling salesperson problem (TSP) is defined on a (undirected) graph $G = (V, E)$ with $|V| = n$, together with a set of weights $\{d_{ij} : i, j = 1, \ldots, n\}$ (in general, some d_{ij} may be infinite, indicating that the edge ij is not present). The problem is to find a minimum length tour that starts and ends at the

same vertex and visits each vertex exactly once. This can be viewed as minimising an Ising Hamiltonian, as follows (see [73,93] for details).

A particular tour can be represented by an $n \times n$ matrix $T = (t_{ij})$ with $t_{ij} = 1$ if edge ij is on the tour and $t_{ij} = 0$ otherwise. Hence, a valid tour has a T matrix with precisely one 1 in each row and in each column, with all other entries being 0. For an undirected graph, a tour and its reverse have the same length, and we consider the undirected tour matrix $U = (u_{ij}) = T + T^t$, where T^t is the transpose of T and – since its i, j entry is t_{ji} – represents the reverse tour to T. U is a symmetric matrix with exactly two 1s in each row and column. The length of the tour is

$$\mathcal{H}_{\text{pot}} = \frac{1}{2} \sum_{i,j=1}^{n} d_{ij} u_{ij}. \tag{20}$$

Defining Ising spins by $S_{ij} = 2u_{ij} - 1$, we can rewrite this Hamiltonian in terms of S_{ij} as

$$\mathcal{H}_{\text{pot}} = \frac{1}{2} \sum_{i,j=1}^{n} d_{ij} \frac{1 + S_{ij}}{2}, \tag{21}$$

which is analogous to the Hamiltonian of noninteracting Ising spins on an $n \times n$ lattice, with fields d_{ij} on the lattice points. The frustration arises from the global constraints on the spin configurations that force them to conform to the structure of U.

$\mathcal{H}_{\text{kin}}$ is then chosen (with a high degree of arbitrariness) to induce fluctuations that generate the important elementary moves of the problem, namely, changing a valid tour into another valid tour. Deciding which configurations are to be neighbours of which others is crucial, since it controls the problem's effective landscape. The 2-opt perturbation or move can be used as it is guaranteed to transform a valid tour into another valid tour. A 2-opt move takes two edges ab and cd not incident with a common vertex and deletes them and replaces them in the tour by the edges ac and bd. The whole 2-opt perturbation can be represented (in U-matrix terms) by four spin-flip operators:

$$S^{+}_{\langle c,a\rangle} S^{+}_{\langle d,b\rangle} S^{-}_{\langle b,a\rangle} S^{-}_{\langle d,c\rangle}, \tag{22}$$

where by definition each $S^{\pm}_{\langle i,j\rangle}$ flips an Ising spin variable (defined as $S_{\langle i,j\rangle} = 2u_{ij} - 1 = \pm 1$) at position (i, j) and at the symmetric position (j, i); that is, $S^{\pm}_{\langle i,j\rangle} = S^{\pm}_{i,j} S^{\pm}_{j,i}$. From this comes a (time-dependent) TSP quantum Hamiltonian implementing the 2-opt moves:

$$\begin{aligned} \mathcal{H}_{\text{TSP}} &= \mathcal{H}_{\text{pot}}(U) + \mathcal{H}_{\text{kin}} \\ &= \sum_{\langle ij\rangle}^{n} d_{ij} \frac{1 + S_{ij}}{2} - \frac{1}{2} \sum_{\langle ij\rangle} \sum_{\langle i',j'\rangle} \Gamma(i, j, i', j'; t) S^{+}_{\langle i,i'\rangle} S^{+}_{\langle j,j'\rangle} S^{-}_{\langle j,i\rangle} S^{-}_{\langle j',i'\rangle}. \end{aligned} \tag{23}$$

Here Γ is a real positive weighting function that depends not only on the edges but also on time.

An actual Schrödinger annealing evolution is computationally impractical, because of the large dimension of the Hilbert space, so Metropolis-type Monte Carlo methods are used. The kinetic part of the quantum Hamiltonian above requires calculation of very many matrix entries of exponential operators in the Trotter-discretised system, which is computationally very expensive. Because of this, we replace it by a standard transverse Ising form, which is trivially Trotter-discretised:

$$\tilde{\mathcal{H}}_{\mathrm{TSP}} = \sum_{\langle ij \rangle}^{n} d_{ij} \frac{1 + S_{ij}}{2} - \Gamma(t) \sum_{\langle ij \rangle} [S^{+}_{\langle j,i \rangle} + \mathrm{H.c.}] \tag{24}$$

This simpler kinetic term is not guaranteed to move from a valid tour to another valid tour in the search space, but a new valid tour can be guaranteed by exclusively using 2-opt moves to generate new states in the Monte Carlo algorithm. It can be shown that this simplification still gives a working SQA algorithm.

SQAs have been applied to a variety of combinatorial problems. Battaglia and Stella [8], Martonak et al. [73] illustrate their application to the TSP. In testing on the `pr1002` instance of the standard benchmark TSPLIB [73, 93], SQA is found to anneal more efficiently than SA. However, SQA does not appear always to be superior to SA. SA outperforms SQA on the random Boolean satisfiabilty (3-SAT) problem [93]; and research [87] investigating extensions of the approach to the multivisit TSP and bottleneck TSP seems to show sensitivity of SQA to the particular flavour of TSP addressed, and to the network structure.

The conclusion seems to be that SQA has the potential to outperform SA, but may require domain knowledge in order to build the kinetic energy term. Further research is needed to clarify this.

Other Applications of SQA

For comprehensive discussions of SQA, including comparison to SA, along with several applications, see [29, 30]. Gomes et al. [43] give a wide-ranging overview of uses of both quantum and quantum-inspired algorithms in operational research.

Quantum-Inspired Evolutionary Algorithms (QIEAs)

In this and the following section, we describe a group of algorithms or metaheuristics whose workings draw inspiration from aspects of quantum mechanics, hybridised with ideas from natural computing: evolutionary algorithms and particle swarm algorithms. It should be noted that a range of metaheuristics have been hybridised with quantum-inspired approaches, but are largely omitted here for reasons of space. The interested reader may wish to refer to (among others): [1, 24, 54] for quantum-inspired approaches involving tabu search; to [64, 119] for quantum-inspired approaches hybridised with scatter search; and to [39, 105] for

adaptive memory programming and quantum-inspired variations; along with the references cited in these.

In the 1990s, Narayanan and Moore [79] first suggested the use of a quantum-inspired genetic algorithm. Since then, quantum-inspired ideas have been applied to a number of different families of natural computing algorithms, including evolutionary algorithms [46, 47, 116, 117], grammatical evolution [75], social computing [118], neurocomputing [38, 66, 108], and immunocomputing [55, 68]. There is now an enormous range of quantum-inspired metaheuristics; [90] and [39] give comprehensive overviews. Ross [90] also investigate "which quantum-inspired metaheuristics could be translated to be used in the existing quantum computers based on the circuit model programming paradigm." Li et al. [70] survey quantum-inspired genetic algorithms, quantum-inspired particle swarm optimisation, and other approaches. Sadeghi Hesar and Houshmand [92] study a memetic quantum-inspired genetic algorithm based on tabu search.

The most common quantum-inspired evolutionary algorithm (QIEA) is the quantum-inspired genetic algorithm (QIGA). We focus on two forms of QIGA, the first working with binary encodings and the second with real-valued encodings. The binary encoding version is more faithful to the quantum metaphor, though neither is completely faithful.

Quantum-inspired ideas have been applied by a number of authors to the area of evolutionary algorithms: one motivation is that because these algorithms use a quantum-inspired representation, they can keep a good balance between exploration and exploitation. Another motivation is that they can provide efficiencies in computation, since using a quantum representation can allow smaller population sizes than standard evolutionary algorithms.

We note that because a classical computer cannot efficiently simulate entanglement (section "Quantum Information"), the degree of fidelity of the algorithms of subsequent sections of this chapter to quantum computing algorithms is partial at best: they are not true quantum algorithms and cannot carry over all of the abilities of quantum computing, such as quantum parallelism.

However, recently, QIEAs have been implemented on a hybrid of classical hardware (e.g. GPUs) and IBM quantum devices, and their runs compared to a GPU-based quantum simulator and an IBM quantum simulator [91], showing interesting results. As NISQ matures, these algorithms should become more capable.

Binary-Valued QIGA

Qubit Representation on Classical Machines

Analogously to a chromosome in a genetic algorithm in evolutionary computation, a *quantum chromosome* can be defined as a system of m qubits (section "Two-State Systems and Qubits") where each qubit q_i is characterised by two scalars, α_i^2 and β_i, namely, the probability amplitudes of observing the eigenstates $|0\rangle$ and $|1\rangle$, respectively. The quantum chromosome can be written as a *qubit register*: a matrix with two rows, comprising one column for each qubit:

$$\begin{bmatrix} \alpha_1 & \alpha_2 & \dots & \alpha_m \\ \beta_1 & \beta_2 & \dots & \beta_m \end{bmatrix}. \tag{25}$$

As with a normalised system in quantum mechanics (section "Normalised Wavefunctions"), we require that for each i, the qubit norm $|\alpha_i|^2 + |\beta_i|^2 = 1$. When an observation is carried out on this register, each qubit collapses into either $|0\rangle$ or $|1\rangle$. The state of the whole system upon wavevector collapse can be depicted as a ket, for example, $|0110 \cdots 1101\rangle$. Because there is no entanglement in our classical model, the algorithm will have to choose either $|0\rangle$ or $|1\rangle$ for each qubit individually. Note that in quantum-inspired evolutionary algorithms to date, the values of α and β for each qubit are usually taken to be two real numbers whose squares sum to 1, whereas in physical qubit systems α and β are complex.

A key point about quantum systems is that they can compactly convey information on a large number of possible system states. In classical bit strings, a string of length m can represent 2^m possible states; but a quantum space of m qubits has 2^m *dimensions* (as a complex manifold). A single-qubit register of length m as in (25) can *simultaneously* represent *all* possible bit strings of length 2^m, e.g. an eight-qubit system can simultaneously encode 256 distinct strings.

From this comes the idea of modifying standard evolutionary algorithms to work with only a few quantum individuals (or even just one), each with a qubit register as its quantum chromosome, instead of using a large population of standard individuals, each of which represents a candidate solution. Because of its ability to represent multiple system states simultaneously, a qubit representation can also aid in maintaining diversity during the evolutionary algorithm's search process. Furthermore, using a qubit representation makes it possible to design hybrid quantum evolutionary algorithms that scale well.

There are several approaches to defining a quantum system to encode a set of binary strings (candidate solutions). As an example, consider the following three-qubit quantum system. Here, the quantum chromosome comprises three pairs (columns) of probability amplitudes:

$$\begin{bmatrix} \frac{\sqrt{3}}{2} & \frac{1}{\sqrt{2}} & \frac{1}{2} \\ \frac{1}{2} & \frac{1}{\sqrt{2}} & \frac{\sqrt{3}}{2} \end{bmatrix}. \tag{26}$$

The squared moduli of these numbers are the probabilities of observing a qubit in one particular eigenstate rather than another. For example, for the second qubit, observation of either of (eigen)states $|0\rangle$ or $|1\rangle$ is equally likely as both α_1 and β_1 have the same absolute value, but for the first qubit we are more likely to observe state $|0\rangle$ since its squared probability amplitude $\frac{\sqrt{3}}{2}$ is larger than that of $|1\rangle$.

The individual qubit probability amplitudes can be used to calculate the probability amplitudes of overall system eigenstates. For example, consider the system state $|000\rangle$. The probability amplitude for this state is obtained by multiplying the probability amplitudes in (26) of the $|0\rangle$ state for each of the three qubits:

$$\frac{\sqrt{3}}{2} \cdot \frac{1}{\sqrt{2}} \cdot \frac{1}{2} = \frac{\sqrt{3}}{4\sqrt{2}}.$$

Then the probability of observing this state is the square $\left(\frac{\sqrt{3}}{4\sqrt{2}}\right)^2 = \frac{3}{32}$.

Using this method, the associated probabilities of each of the individual states $|000\rangle$, $|001\rangle$, $|010\rangle$, $|011\rangle$, $|100\rangle$, $|101\rangle$, $|110\rangle$ and $|111\rangle$ are

$$\frac{\sqrt{3}}{4\sqrt{2}}, \frac{3}{4\sqrt{2}}, \frac{\sqrt{3}}{4\sqrt{2}}, \frac{3}{4\sqrt{2}}, \frac{1}{4\sqrt{2}}, \frac{\sqrt{3}}{4\sqrt{2}}, \frac{1}{4\sqrt{2}} \text{ and } \frac{\sqrt{3}}{4\sqrt{2}} \text{ respectively,}$$

giving associated probabilities

$$\frac{3}{32}, \frac{9}{32}, \frac{3}{32}, \frac{9}{32}, \frac{1}{32}, \frac{3}{32}, \frac{1}{32} \text{ and } \frac{3}{32} \text{ respectively.}$$

Then the (quantum) state of this three-qubit system is given by

$$\frac{\sqrt{3}}{4\sqrt{2}}|000\rangle + \frac{3}{4\sqrt{2}}|001\rangle + \frac{\sqrt{3}}{4\sqrt{2}}|010\rangle + \frac{3}{4\sqrt{2}}|011\rangle$$
$$+ \frac{1}{4\sqrt{2}}|100\rangle + \frac{\sqrt{3}}{4\sqrt{2}}|101\rangle + \frac{1}{4\sqrt{2}}|110\rangle + \frac{\sqrt{3}}{4\sqrt{2}}|111\rangle.$$

In the following, to simplify notation, we will often write $|0\rangle$ as 0 and $|1\rangle$ as 1.

Binary Quantum-Inspired Genetic Algorithm

Algorithm 4 is an example of a binary QIGA [46]. Many evolutionary algorithms run for a predetermined number of iterations or "generations" (of the population). We have used this as the terminating condition $t = t_{\max}$ here, but more complex termination conditions could also be used as mentioned for previous algorithms.

At the start of the algorithm's run, the population of quantum chromosomes is created as

$$Q(t) = \{q^1(t), q^2(t), \ldots, q^n(t)\},$$

where n is the population size (n could be equal to 1), and each quantum chromosome comprises m qubits (the superscript on each qubit is a label indicating which quantum chromosome it is part of, not an exponent). The α and β values for each qubit are each set to $1/\sqrt{2}$ to begin with, to ensure that for each qubit the states $|0\rangle$ and $|1\rangle$ are equally likely initially (since the probability of each state $|0\rangle$ or $|1\rangle$ is $(1/\sqrt{2})^2 = 1/2$). Alternatively, if domain knowledge indicates that some states are likely to lead to better results, this can be used to seed the initial quantum chromosome(s). During the run of the algorithm, the current population of quantum chromosomes at timestep t can be used to create a second population

$$P(t) = \{p^1(t), p^2(t), \ldots, p^n(t)\}$$

Algorithm 4: Binary quantum-inspired genetic algorithm

```
1  begin BinaryQIGA(n, t_max)
2      Set t ← 0;
3      Initialise Q(0), the population of quantum chromosome(s);
4      for j = 1 to n do
5          Create p^j(0) by carrying out an observation of q^j(0) ∈ Q(0);
6      end for
7      Set the population P(0) ← {p^1(0), ..., p^n(0)};
8      Evaluate P(0) and select the best solution from P(0);
9      Store the best solution in B(0);
10     while t < t_max do
11         t ← t + 1;
12         Create P(t) by carrying out observations of Q(t − 1);
13         Evaluate the population of solutions P(t);
14         Compare the best solution in P(t) with that of B(t − 1) and store the better of
             these in B(t);
15         Update Q(t) using B(t);
16     end while
17     return best solution found;
18 end BinaryQIGA.
```

of binary strings (candidate solution encodings) by carrying out "observations" on the quantum chromosomes. The observation step on qubit i of the jth quantum chromosome $q^j(t)$ can be carried out by drawing a random number $r \in [0, 1]$, e.g. from a uniform distribution on $[0, 1]$. If $r > |\alpha_i(t)|^2$, the corresponding bit $p_i^j(t)$ in the observed individual $p^j(t)$ is taken to be in state $|1\rangle$; otherwise, it is taken to be in state $|0\rangle$.

Since the observation step is probabilistic, the QIGA could be implemented using just a single quantum chromosome, with this chromosome being observed multiple times to create the population $P(t)$. Another option is to maintain a small population of quantum chromosomes and observe each quantum chromosome a fixed number of times to generate $P(t)$.

In the while loop of Algorithm 4, the population $Q(t)$ of quantum chromosomes is updated. This update step could be made in different ways, e.g. by using a suitable quantum gate (section "Diversity Generation in Binary QIEA") or by using "genetic" operators. The essence of the update step is to adjust the quantum chromosome to make it more likely that the best solution found so far will be generated in the next iteration. As the QIGA algorithm approaches the optimal solution, the values of each element of the quantum chromosome tend more closely towards either 0 or 1: this means there is a high probability that, when observed, the quantum chromosome will give a particular solution vector p^j. As an extreme example, the quantum chromosome $\begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \end{bmatrix}$ will generate the observed solution chromosome [0 0 0] with probability 1, no matter what value the parameter r has in the observation step.

Diversity Generation in Binary QIEA

The binary QIGA does not generally use distinct crossover and mutation operators, unlike the standard genetic algorithm (GA). Instead, it uses a single diversity-generation step. There are two common approaches to this, the first – quantum gates – being more inspired by quantum mechanics than the second – quantum mutation.

Quantum Gates

In the quantum gate approach, the QIGA simulates the working of a digital quantum computer gate (section "Digital Quantum Computers"). The idea of quantum gate has been used successfully in QIEAs, and a range of quantum gates is possible [115]. Currently, researchers appear to use real number probability amplitudes exclusively, rather than the complex probability amplitudes arising in quantum mechanics; a possible extension of such QIEAs would be to use complex probability amplitudes and take absolute values during the observation step.

Whereas the Hilbert space of a system of n physical qubits has dimension 2^n, our classical version of this in a QIEA has dimension $2n$. Rather than an operator on the space (e.g. a gate) being represented by a $2^n \times 2^n$ matrix (which would be physically more correct – see section "Digital Quantum Computers" – but prohibitively expensive computationally), a set of n 2×2 matrices is used, one for each classical qubit. Figure 4 shows an example of a quantum rotation gate which, as the name suggests, rotates the vector (α_i, β_i) through an angle. Each gate matrix must be unitary (and self-adjoint), so that the updated coefficients α'_i, β'_i for the ith qubit will still satisfy $|\alpha'_i|^2 + |\beta'_i|^2 = 1$.

In the mutation step, the idea is to alter the probability amplitudes on the quantum chromosome so that on the algorithm's next iteration, fitter individuals are more likely to be observed. We make this adjustment using information from the best-performing individual in the nonquantum population $P(t)$ and adjust the probability amplitudes using a rotation gate as in (27).

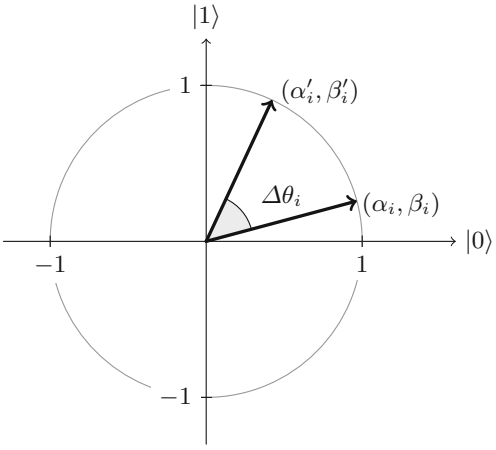


Fig. 4 Quantum rotation gate

$$\begin{bmatrix} \alpha_i' \\ \beta_i' \end{bmatrix} = \begin{bmatrix} \cos(\Delta\theta_i) & -\sin(\Delta\theta_i) \\ \sin(\Delta\theta_i) & \cos(\Delta\theta_i) \end{bmatrix} \begin{bmatrix} \alpha_i \\ \beta_i \end{bmatrix}. \tag{27}$$

A look-up table is used to find the most suitable value for $\Delta\theta$ that makes mutation tend towards the best current member $B(t)$ of the observed population $P(t)$. One straightforward way to carry this out is to generate a binary chromosome x from the current quantum chromosome q. Given

$$q = \begin{bmatrix} \alpha_1 & \alpha_2 & \dots & \alpha_m \\ \beta_1 & \beta_2 & \dots & \beta_m \end{bmatrix},$$

for each qubit q_i, if $\alpha_i^2 \leq 1/2$, then set $x_i = 0$; otherwise, set $x_i = 1$. Table 1 is the look-up table for $\Delta\theta$. The parameter a controls the mutation step size.

If a bit has the same value in both x_i and the best element b_i, then the quantum chromosome is not changed. As an example, suppose the quantum chromosome has a single qubit: $q = \begin{bmatrix} 0.7 \\ 0.3 \end{bmatrix}$. Since $\alpha_1^2 > 0.5$, we set $x_1 = 1$. Assume $b_1 = 1$. Then from the look-up table, we get $\Delta\theta = 0$. Applying the quantum rotation gate (27), the values of $\begin{bmatrix} \alpha_1 \\ \beta_1 \end{bmatrix}$ stay the same, because $\begin{bmatrix} 0.7 \\ 0.3 \end{bmatrix} = \begin{bmatrix} \cos(0) & -\sin(0) \\ \sin(0) & \cos(0) \end{bmatrix} \begin{bmatrix} 0.7 \\ 0.3 \end{bmatrix}$.

On the other hand, if $x_i > b_i$, the corresponding value of the quantum chromosome is reduced by $-a$, which reduces the probability of observing a $|1\rangle$ from the quantum chromosome in the future.

While Table 1 illustrates a simple quantum gate, it does not explicitly consider the quadrant that $\begin{bmatrix} \alpha_i \\ \beta_i \end{bmatrix}$ is in (since α_i and/or β_i could be negative). We can use a variation on the method for updating the ith qubit value $\begin{bmatrix} \alpha_i \\ \beta_i \end{bmatrix}$:

$$\begin{bmatrix} \alpha_i' \\ \beta_i' \end{bmatrix} = \begin{bmatrix} \cos(\xi(\Delta\theta_i)) & -\sin(\xi(\Delta\theta_i)) \\ \sin(\xi(\Delta\theta_i)) & \cos(\xi(\Delta\theta_i)) \end{bmatrix} \begin{bmatrix} \alpha_i \\ \beta_i \end{bmatrix}, \tag{28}$$

where $\xi(\Delta\theta_i) = s(\alpha_i, \beta_i) \times \Delta\theta_i$. The terms $s(\alpha_i, \beta_i)$ and $\Delta\theta_i$ are the direction of rotation and the angle, respectively. A number of variations of rotation schemes

Table 1 A look-up table for $\Delta\theta$

x_i	b_i	$\Delta\theta$
1	1	0
0	1	a
0	0	0
1	0	$-a$

exist, some quite complex: for further details, see [115, Sect. 3]. The step size for $\Delta\theta_i$ controls the speed of convergence of the algorithm and depends on the problem. We usually set the step size to a small value; e.g. [46,111] suggest $\Delta = 0.01\pi$. Trial and error or a more systematic grid search may be needed to find a good value.

Quantum Mutation

A simpler approach to diversity generation is quantum mutation [116, 117], which derives more inspiration from the standard GA mutation operator than from quantum physics. It is defined by

$$Q_{\text{pointer}}(t) = a \times B_{\text{best}}(t) + (1 - a) \times (1 - B_{\text{best}}(t)) \tag{29}$$

$$Q(t + 1) = Q_{\text{pointer}}(t) + b \times r, \tag{30}$$

where $B_{\text{best}}(t)$ is the best solution found in iteration t, $Q_{\text{pointer}}(t)$ is a temporary quantum chromosome that is used to guide the tailoring of $Q(t + 1)$ towards the form of B_{best}, and r is a random number drawn from a $N(0, 1)$ normal distribution. The parameters a and b determine the balance between exploration and exploitation: a controls the importance attached to $B_{\text{best}}(t)$, while b controls the degree of variance generation, centred on $Q_{\text{pointer}}(t)$. [116, 117] suggest choosing values of $a \in [0.1, 0.5]$ and $b \in [0.05, 0.15]$.

Another simple method for adapting the quantum chromosome [117] is

$$\alpha(t + 1) = [2 - B_{\text{best}}(t)] \times 0.5. \tag{31}$$

Using this approach, the bits in the current best individual will be more likely to be chosen during quantum chromosome observation in the next iteration.

Real-Valued QIEA

da Cruz et al. [27] proposed an alternative real-valued QIEA approach. As with the binary QIGA, the real-valued QIGA maintains two distinct populations: a quantum population and an observed population. In this approach, the observed population consists of real-valued m-dimensional solution vectors; the quantum population $Q(t)$ consists of n quantum chromosomes q_i with $i = 1, 2, \ldots, n$, where each individual q_i is composed of m genes q_{ij} with $j = 1, 2, \ldots, m$: q_{ij} is the jth gene of the ith quantum individual. Each gene is a pair of real values, $q_{ij} = (p_{ij}, \sigma_{ij})$, where p_{ij} and σ_{ij} are the mean and the width of a rectangular pulse, respectively. Representing a gene in this way corresponds loosely to quantum superposition of states, since a gene is a range of possible values, rather than a unique value; also, the act of observation (in this case, periodically sampling from an associated distribution) can be viewed as causing a wavefunction to collapse to an eigenstate. On observation, a chromosome gives a particular solution vector $\in \mathbb{R}^m$.

Algorithm 5 gives a version of the real-valued QIEA, which uses a single quantum chromosome. Here, $E(t)$ is a temporary population and $C(t)$ is a population of "best-so-far." The population $C(t)$ is maintained between iterations of the algorithm.

Algorithm 5: Real-valued quantum-inspired genetic algorithm

```
1  begin RealValuedQIGA(K, n, t_max)
2      Set t ← 0;
3      Initialise the quantum chromosome Q(t);
4      while t < t_max do
5          Create the PDFs and corresponding CDFs for each gene locus using the quantum
             individual;
6          Create a temporary population E(t) of K real-valued solution vectors through a
             series of observations of Q(t) via the CDFs;
7          if t = 0 then
8              C(t) ← E(t);
9          else
10             Let E(t) ← outcome of crossover between E(t − 1) and C(t − 1);
11             Evaluate E(t);
12             Let C(t) ← K best individuals from E(t) ∪ C(t − 1);
13             Evaluate C(t);
14         end if
15         With the n best individuals from C(t);
16         Q(t + 1) ← output of translate operation on Q(t);
17         Q(t + 1) ← output of resize operation on Q(t + 1);
18         t ← t + 1;
19     end while
20     return best solution found;
21 end RealValuedQIGA.
```

Initialising the Quantum Population

At the beginning of the algorithm's run, each quantum gene q_{ij} is initialised by randomly drawing a value for p_{ij} from the range of allowable values for that component j of the m-dimensional solution space $\mathbb{R}^m$. A gene's width value is set to the range of allowable values for the corresponding component of the solution vector. For example, if the allowable values for component j are $[-75, 75]$, then q_{ij} is initially found by randomly selecting a value from this range, (say) -50. The corresponding width value will be the width of the range, $75 + 75 = 150$. Hence, $q_{ij} = (-50, 150)$, a rectangular pulse of width 150 centred on -50.

The rectangular pulse need not be completely inside the allowable range for a component when initially created, since the algorithm will automatically adjust for this during its run. The height of the pulse of gene j in chromosome i is calculated using

$$h_{ij} = \frac{1/\sigma_{ij}}{n}, \tag{32}$$

where n is the number of chromosomes in the quantum population (n could be 1). The height assigned by (32) ensures that the rectangular pulse has area 1, so it is a valid probability density function (PDF) for generating the observed solution vectors (see below). Figure 5 illustrates the quantum gene $q_{ij} = (-50, 150)$ above.

Observing Quantum Chromosomes

To generate a population of real-valued solution vectors, we must make a series of observations of the population of quantum chromosomes (individuals). A pseudointerference process between the quantum individuals is simulated by summing up the rectangular pulse for each individual gene across all members of the quantum population. This generates a separate PDF for each gene, and applying (32) ensures that the area under this PDF is 1. Hence, the PDF for gene j on iteration t is

$$\mathrm{PDF}_j(t) = \sum_{i=1}^{n} q_{ij}. \tag{33}$$

To use this PDF to make an observation, its corresponding cumulative distribution function (CDF) is first obtained:

$$\mathrm{CDF}_j(x) = \int_{L_j}^{U_j} \mathrm{PDF}_j(x)dx, \tag{34}$$

where U_j and L_j are the upper and lower limits of the probability distribution. A separate PDF and CDF is calculated for each of the m gene positions.

By drawing a random number r uniformly from the interval (0,1), the CDF can be used to give an observation of a real number $x = \mathrm{CDF}^{-1}(r)$. If the "observed" value x is outside the allowable range for that component, then x is limited to the nearer allowable boundary value. The observation process is iterated to create a temporary population $E(t)$ with K members.

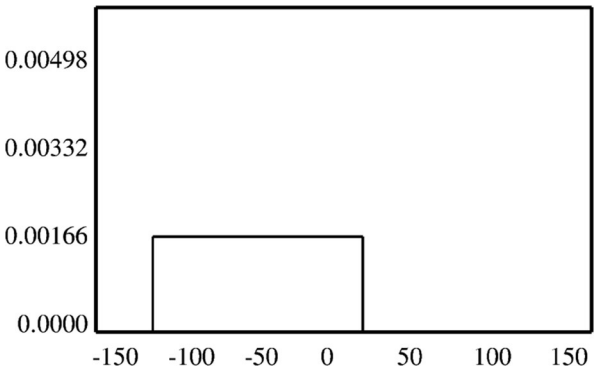


Fig. 5 A rectangular pulse, representing a quantum gene with a width of 150, centred on -50. The height of the pulse is $\frac{1}{150} = 0.001666\dot{6}$

Crossover Operation

Crossover takes place between $C(t)$ and the temporary population $E(t)$. This step can be implemented in a number of ways: da Cruz et al. [27] choose a variation of uniform crossover for real numbers. After the K crossover operations have been carried out, and the resulting offspring copied into $E(t)$, the best K individuals $\in C(t-1) \cup E(t)$ are copied into the current $C(t)$.

Updating the Quantum Chromosomes

After carrying out the crossover operation in Algorithm 5, the quantum population $Q(t)$ is updated using the "translate" and "resize" operations.

In "translate" the n quantum chromosomes are updated using the n best individuals from the current population $C(t)$. Each quantum gene's centre (mean value) is changed using

$$p_{ij} = c_{ij}. \tag{35}$$

Thus, the mean value of gene j of quantum chromosome i is set to the jth component of the ith ranked individual in population $C(t)$.

In the final "resize" step in Algorithm 5, the corresponding width value of gene j is updated. The purpose of this is to adjust the exploration-exploitation balance of the search algorithm, according to feedback from previous iterations. If the search process is still finding many new better solutions, then we should continue with exploration by keeping the pulse widths relatively broad. Conversely, if we are not finding many new better solutions, we reduce the pulse widths so as to encourage exploitation: finer-grained search around regions of the solution space already found to be good. This can be implemented in several ways: for example, da Cruz et al. [27] suggest the $\frac{1}{5}$ mutation rule from evolutionary strategies [89], where we adjust the width σ_{ij} of gene j of quantum chromosome i:

$$\sigma_{ij} = \begin{cases} \sigma_{ij} g & \text{if } \varphi < 1/5 \\ \sigma_{ij} / g & \text{if } \varphi > 1/5 \\ \sigma_{ij} & \text{if } \varphi = 1/5 \end{cases}. \tag{36}$$

Here, g is a constant in the range $[0, 1]$ and φ is the proportion of individuals in the new population that have improved their fitness.

QIEAs and EDAs

Although the real-valued QIEA has some similarity to the quantum ideas of superposition of states and wavevector collapse to a classical state when observed (in the QIEA, this is sampling from a gene's PDF), it takes less inspiration from quantum physics than does the binary QIEA, since the real-valued QIEA does not have an equivalent to the qubit representation. Also, it draws from uniform

distributions (unlike those which arise in physics) and only allows constructive interference (whereas destructive interference of wavevectors is experimentally observed in physical systems): in addition, this constructive interference is among wavevectors of *different* individuals, whereas physically interference is observed between a system's wavevector and *itself*.

An important difference between QIEAs and other nature-inspired algorithms such as the genetic algorithm (GA) and particle swarm optimisation (PSO) is that in QIEAs, a single probability vector (or a small population of probability vectors) is updated and used to guide search space exploration, rather than a population of individual solution vectors being manipulated.

The QIEA approach is similar to the approaches taken in binary and continuous estimation of distribution algorithms (EDAs). In particular, the design of real-valued QIEAs is very similar to that of univariate continuous EDAs; this suggests that a typical real-valued QIEA would struggle in situations where there are strong relationships among variables, given that EDAs have these issues. The binary-valued QIEA is also similar to binary EDAs such as population-based incremental learning (PBIL) [7] and the compact genetic algorithm (cGA) [48]; however, the binary QIEA diversity-generating mechanisms, especially the use of pseudoquantum gates, differ from the typical mechanisms used in EDAs to generate diversity. Platel et al. [86] discuss some of the similarities and differences between EDAs and QIEAs.

Applications of QIEAs

Early works on QIEAs include [46, 47, 79, 116, 117]. QIEAs have been widely applied; other applications include ordering problems [28], continuous optimisation (using a real-coded QIEA) [106], user pairing in non-orthogonal multiple access [80], neural architecture search [104], and the design of miniaturised reconfigurable Internet of Things antennas [13]. Montiel et al. [78] develop a QIEA (also partly inspired by ant colony optimisation) and find that it "outperforms existing outstanding quantum-inspired evolutionary algorithms when testing benchmark functions."

Quantum Binary PSO

The particle swarm optimisation algorithm (PSO) was introduced by Kennedy and Eberhart [57]. It can be applied in both continuous and discrete (including binary) settings. The best-known version of binary PSO, *BinPSO* [58] converts the continuous PSO algorithm into one which operates on binary representations. An alternative approach to applying PSO to binary-encoded problems, *angle-modulated PSO*, is outlined in [82]. For further discussion, see [17].

An alternative version of binary PSO, *Quantum Binary PSO*, proposed by Yang et al. [118], draws inspiration from quantum mechanics. The main quantum concepts used are the superposition of states and the collapse of the quantum state of a particle to a single eigenstate upon observation. Unlike many applications

of quantum concepts to evolutionary algorithms, two populations of the same size are maintained: one population of candidate solutions and a corresponding population of the same size of quantum particles. The population of quantum particles is updated from one algorithm iteration to the next by applying a variation of the PSO velocity update equation to the quantum population (replacing the original PSO two-step process of calculating velocity updates and then moving the population of particles). Each quantum particle is then observed in turn, to generate a corresponding binary particle whose fitness can be computed.

The Algorithm

In the algorithm, a population of N quantum particle vectors, each with m components, is created, defined as follows:

$$Q(t) = \{q^1(t), q^2(t), \ldots, q^N(t)\}, \tag{37}$$

where

$$q^j(t) = (q_1^j(t), q_2^j(t), \ldots, q_m^j(t)), \quad j = 1, 2, \ldots, N, \tag{38}$$

with $q_i^j(t)$ being the probability of the ith bit of the jth particle being 0 at time t. Thus, for each $q_i^j(t)$, the range of possible values is the closed interval $[0, 1]$. The population of binary particles has the same size and number of components (dimension) as the population of quantum particles; thus:

$$X(t) = \{x^1(t), x^2(t), \ldots, x^N(t)\}, \tag{39}$$

where

$$x^j(t) = (x_1^j(t), x_2^j(t), \ldots, x_m^j(t)), \tag{40}$$

and $x_i^j(t) \in \{0, 1\}$ is the binary value of particle j in position i derived from observing the quantum particle $q_i^j(t)$, for each $i = 1, 2, \ldots, m$ and each $j = 1, 2, \ldots, N$.

After the quantum population has been generated – or updated as the algorithm runs – a population of binary-encoded particles is generated by observing the corresponding quantum particle. That is, the first binary particle is created by observing the first quantum particle, and so on. The observation mechanism is relatively simple:

- For each $q_i^j(t)$, where $i = 1, 2, \ldots, m$, $j = 1, 2, \ldots, N$, generate a random number r uniformly in the range $[0, 1]$.
- If the random number is greater than $q_i^j(t)$, then set $x_i^j(t) = 1$; otherwise, set $x_i^j(t) = 0$.

A variation of the PSO velocity update step is applied to drive the search process of the quantum particle population from one iteration to the next. To begin, an m-dimensional quantum particle called $q_{g^{\text{best}}}(t)$ is determined using the global best binary particle $x_{g^{\text{best}}}(t)$,

$$q_{g^{\text{best}}}(t) = \alpha x_{g^{\text{best}}}(t) + \beta(1 - x_{g^{\text{best}}}(t)); \tag{41}$$

next, for each member of the quantum population, a quantum particle $q_{p^{\text{best}}}$ is calculated using the $x_{p^{\text{best}}}(t)$ of their associated binary vector:

$$q_{p^{\text{best}}}(t) = \alpha x_{p^{\text{best}}}(t) + \beta(1 - x_{p^{\text{best}}}(t)). \tag{42}$$

Here, α, β, where $\alpha + \beta = 1$, $0 < \alpha, \beta < 1$, are control parameters which control the speed of adaptation towards g^{best} and p^{best}. Finally, each quantum population member is updated using:

$$q(t+1) = wq(t) + c_1 q_{p^{\text{best}}}(t) + c_2 q_{g^{\text{best}}}(t), \tag{43}$$

where $w + c_1 + c_2 = 1$ and $0 < w, c_1, c_2 < 1$. Algorithm 6 lists the steps of quantum binary particle swarm.

Algorithm 6: Quantum binary particle swarm

```
1  begin QuantumBinaryPS(m, N)
2      Set t ← 0;
3      Initialise the populations Q(t) of quantum particles and X(t) of binary particles;
4      Evaluate the fitness of each (binary) individual in X(t);
5      Find the binary particle with highest fitness, and store as x_{g^best};
6      repeat
7          t ← t + 1;
8          Update all individuals in Q(t) using Eqs. (41)–(43);
9          Observe each individual in Q(t) in turn to get each individual in X(t);
10         for each i ∈ {1, ..., N}, each j ∈ {1, ..., m} do
11             Generate a random number r uniformly in the range [0, 1];
12             if r > q_i^j then
13                 x_i^j ← 1;
14             else
15                 x_i^j ← 0;
16             end if
17         end for
18         Evaluate fitness of each individual in X(t);
19         Update x_{g^best} for X(t) and individual x_{p^best}s if necessary;
20     until terminating condition;
21     return best solution found;
22 end QuantumBinaryPS.
```

Applications and Further Reading

Applications

The range of applications in section "Quantum Advantage" suggests that as and when sufficiently large and powerful quantum computers become available, they could productively be applied to difficult problems arising in decision-making, operational research (OR), and related areas. There are many interconnections and indeed synergies between operational research and quantum computing [83]. Recently, quantum computers have been increasingly applied to OR problems. For example, Creemers et al. [26] apply Grover's quantum database algorithm to the knapsack problem and Bochkarev et al. [14] apply a range of quantum approaches to a range of OR problems. However, it is generally necessary to reformulate the problem, especially the objective function, into qubit terms [14]. Klug [62, Fig. 1] provides a quantum landscape of optimisation algorithms in operational research, and Gomes et al. [43] provide an overarching view of the literature on applications of quantum and quantum-inspired algorithms to operational research, including a systematic classification of quantum and quantum-inspired algorithms in the context of possible operational research applications, with detailed analyses of 149 studies drawn from more than 2000 found. Klug [62] discusses operational research application areas of quantum algorithms in the contexts of routing, sequencing, scheduling, batching and loading, inventory management, and max-cut problems, as well as applications in finance. He emphasises that "From the point of view of practical OR applications, it is quite often more important to find a good instead of a perfect solution." Weinberg et al. [112] discuss applications of quantum computing approaches to supply chain management.

It should be noted that as of writing, there does not yet exist an exponential quantum advantage for optimisation problems [53]. Also, quantum algorithms can only sometimes achieve an exponential speed-up over classical algorithms but can in general attain a significant polynomial speed-up, e.g. quadratic; bigger improvements in speeding up solving require special structures in the problem [74]. These limited quantum speed-ups for particular tailored problems do not suggest a general quantum advantage for real decision problems in OR [62]. Also, which quantum technology is best is moot: bochkarev et al. [14] solve instances of three classical discrete optimisation problems – the traveling salesperson problem, weighted maximum cut, and maximum independent set – in a series of experiments on three types of quantum computer – a neutral atom machine (QuEra), a quantum annealing computer (D-Wave), and a gate-based device (IBM) – but find it difficult to assess which of the currently available systems is superior.

Further Reading

A number of papers are useful for the reader interested in acquiring further knowledge of quantum computing approaches, both on true quantum computers

and by simulations on classical computers. Good overviews of quantum computing, including more detail on the underlying physics than is given here, can be found in [12, 52, 62].

In particular, [12] is extensive and provides a detailed review of NISQ computers and QA computational paradigms and algorithms, along with a summary of recent quantum hardware and experiments. The authors note that scientists and engineers are also developing hybrid quantum computing platforms, but that such devices – built to solve particular problems – do not necessarily have universal quantum gate sets. They discuss a range of applications of NISQ algorithms, including machine learning, combinatorial optimisation, and numerical solvers. They point out that most current NISQ algorithms use quantum computers in a hybrid quantum-classical arrangement, where the algorithms perform the classically difficult part of a computation on a quantum computer and carry out the rest on a sufficiently powerful classical machine. These algorithms *variationally* update the variables of a parameterised quantum circuit and so are called variational quantum algorithms (VQA). Bharti et al. [12] concentrate mostly on VQA but consider quantum computing paradigms using different types of algorithms which are inspired by and hybridised with analogue approaches, such as quantum annealing. In addition, the article studies the methods developed to best utilise NISQ algorithms as well as theoretical and experimental challenges; they discuss the theoretical guarantees of some of these algorithms, as well as noisy quantum device error-mitigation techniques.

Ye and Lu [121] give an overview of research and current trends in the field of quantum science. Hauke et al. [51] provide a detailed introduction to quantum annealing and discuss challenges and future perspectives. Yulianti et al. [122] discuss implementations of quantum annealing and compare it to other common metaheuristics: simulated annealing, the genetic algorithm, particle swarm optimisation, and tabu search. Yarkoni et al. [120] review the literature to provide a centralised and condensed source on applications of quantum annealing.

Conclusions

This chapter has given an overview of the current state of quantum computing and quantum-inspired algorithms and metaheuristics. We began with a summary of the physics necessary to understand quantum computers and associated concepts, then related this to existing quantum hardware, and then to existing and potential quantum and quantum-inspired algorithms.

While quantum hardware is still in its infancy, and we are still in the NISQ phase of quantum computing, quantum and quantum-inspired algorithms have shown strengths; and quantum computing – together with ideas arising from it – has the potential to be a transformative technology.

Although quantum-inspired algorithms and metaheuristics predate working quantum computers, they have been found to have interesting properties in their

own right and are able to give useful results on classical computers; the wheel has turned, and some are now being implemented in limited ways on quantum hardware.

There is a range of applications across domains including operational research, and it is very likely that, in the near to medium term, quantum computing will have a major impact on optimisation and problem-solving in general.

Cross-References

- A History of Metaheuristics
- Adaptive and Multi-level Metaheuristics
- Cutting and Packing
- Evolutionary Algorithms
- Genetic Algorithms
- Iterated Local Search
- Metaheuristics for Medical Image Registration
- Metaheuristics for Natural Gas Pipeline Network Optimization
- Multiobjective Optimization
- Multi-start Methods
- Particle Swarm Optimization for the Vehicle Routing Problem: A Survey and a Comparative Analysis
- Random-Key Genetic Algorithms: Principles and Applications
- Selected String Problems
- The Maximum Clique and Vertex Coloring
- Variable Neighborhood Search

Competing Interest Declaration The author(s) has no competing interests to declare that are relevant to the content of this manuscript.

References

1. Abdullah Z, Tsimenidis CC, Johnston M (2018) Quantum-inspired tabu search algorithm for antenna selection in massive mimo systems. In: 2018 IEEE Wireless Communications and Networking Conference (WCNC), pp 1–6
2. Abrams DS, Lloyd S (1997) Simulation of many-body Fermi systems on a universal quantum computer. Phys Rev Lett 79:2586–2589
3. Aeppli G, Rosenbaum TF (2005) Experiments on quantum annealing. In: Das A, Chakrabarti B (eds) Quantum Annealing and Related Optimization Methods. Lecture Notes in Physics, vol 679. Springer, Berlin, pp 159–169
4. Aharonov D, van Dam W, Kempe J, Landau Z, Lloyd S, Regev O (2004) Adiabatic quantum computation is equivalent to standard quantum computation. In: 45th Annual IEEE Symposium on Foundations of Computer Science, pp 42–51
5. Albash T, Lidar DA (2018) Adiabatic quantum computation. Rev Mod Phys 90:015002
6. Arute F, Arya K, Babbush R et al (2019) Quantum supremacy using a programmable superconducting processor. Nature 574:505–510
7. Baluja S (1994) Population-based incremental learning: a method for integrating genetic search based function optimization and competitive learning, Technical Report, Carnegie Mellon University, Pittsburgh, PA, CMU-CS-94-163

8. Battaglia D, Stella L (2006) Optimization through quantum annealing: theory and some applications. Contemp Phys 47(4):195–208
9. Bell JS (1964) On the Einstein-Podolsky-Rosen paradox. Physics 1:195–200
10. Benedetti M, Realpe-Gómez J, Biswas R, Perdomo-Ortiz A (2016) Estimation of effective temperatures in quantum annealers for sampling applications: a case study with possible applications in deep learning. Phys Rev A 94:022308
11. Benioff P (1980) The computer as a physical system: a microscopic quantum mechanical Hamiltonian model of computers as represented by Turing machines. J Stat Phys 22:563–591
12. Bharti K, Cervera-Lierta A, Kyaw TH, Haug T, Alperin-Lea S, Anand A, Degroote M, Heimonen H, Kottmann JS, Menke T, Mok W-K, Sim S, Kwek L-C, Aspuru-Guzik A (2022) Noisy intermediate-scale quantum algorithms. Rev Mod Phys 94:015004
13. Bichara MR, Asadallah FAB, Awad M, Costantine J (2023) Quantum genetic algorithm for the design of miniaturized and reconfigurable iot antennas. IEEE Trans Anten Propag 71(5):3894–3904
14. Bochkarev A, Heese R, Jäger S, Schiewe P, Schöbel A (2024) Quantum computing for discrete optimization: a highlight of three technologies, Technical Report, Munich University of Applied Sciences, Department of Mathematics, RPTU Kaiserslautern-Landau, Kaiserslautern, 67663
15. Born M, Fock VA (1928) Beweis des Adiabatensatzes. Zeitschrift für Physik A Hadrons and Nuclei 51(3–4):165–180
16. Bova F, Goldfarb A, Melko RG (2023) Quantum economic advantage. Manag Sci 69(2):1116–1126
17. Brabazon A, O'Neill M, SGarraghy MC (2015) Natural Computing Algorithms, 1st edn. Springer, Berlin
18. Brooke J, Bitko D, Rosenbaum TF, Aeppli G (1999) Quantum annealing of a disordered magnet. Science 284(5415):779–781
19. Cai J-Y (2024) Shor's algorithm does not factor large integers in the presence of noise. Sci China Inf Sci 67:173501:1–173501:16
20. Calixto M (2009) Quantum computation and cryptography: an overview. Natural Comput 8:663–679
21. Cerny V (1982) A thermodynamical approach to the travelling salesman problem: an efficient simulation algorithm, Technical Report, Comenius University, Bratislava, Czechoslovakia
22. Cerny V (1985) Thermodynamical approach to the travelling salesman problem: an efficient simulation algorithm. J Optim Theory Appl 45:41–51
23. Chakrabarti BK, Dutta A, Sen P (1996) Quantum Ising Phases and Transitions in Transverse Ising Models. Springer, Berlin
24. Chiang H-P, Chou Y-H, Chiu C-H, Kuo S-Y, Huang Y-M (2014) A quantum-inspired tabu search algorithm for solving combinatorial optimization problems. Soft Comput 18:1771–1781
25. Choi V (2018) Minor-embedding in adiabatic quantum computation: i. the parameter setting problem. Quantum Inf Process 7:193–209
26. Creemers S, Armas LFP (2022) A quantum revolution (Part I), Technical Report, IÉSEG School of Management Lille Catholic University 3, rue de la Digue F-59000 Lille
27. da Cruz A., Vellasco M, Pacheco M (2006) Quantum-inspired evolutionary algorithm for numerical optimization. In: Proceedings of the 2006 IEEE Congress on Evolutionary Computation (CEC 2006) (Vancouver), IEEE, IEEE Press, 16–21 July 2006, pp 9180–9187
28. da Silveira LR, Tanscheit R, Vellasco MMBR (2017) Quantum inspired evolutionary algorithm for ordering problems. Expert Syst Appl 67:71–83
29. Das A, Chakrabarti B (eds.) (2005) In: Quantum Annealing and Related Optimization Methods. Lecture Notes in Physics, vol 679. Springer, Berlin
30. Das A, Chakrabarti BK (2008) Quantum annealing and analog quantum computation. Rev Modern Phys 80:1061
31. Deutsch D (1985) Quantum theory, the Church-Turing principle and the universal quantum computer. Proc R Soc Lon A400:97–117

32. Dieks D (1982) Communication by EPR devices. Phys Lett A 92:271–272
33. Du Y, Glover F, Kochenberger G, Hennig R, Wang H, Hulandageri A (2024) Solving the minimum sum coloring problem: alternative models, exact solvers, and metaheuristics. INFORMS J Comput 37(2):199–211
34. Du Y, Wang H, Hennig R, Hulandageri A, Kochenberger G, Glover F (2025) New advances for quantum-inspired optimization. Int Trans Oper Res 32(1):6–17
35. e Oliveira HA (2024) Homotopic quantum fuzzy adaptive simulated annealing [HQF ASA]. Inf Sci 669:120529
36. Feynman R (1982) Simulating physics with computers. Int J Theor Phys 21(6&7):467–488
37. Finnila A, Gomez M, Sebenik C, Stenson C, Doll J (1994) Quantum annealing: a new method for minimizing multidimensional functions. Chem Phys Lett 219:343–348
38. Garavaglia S (2002) A quantum-inspired self-organizing map (QISOM). In: Proceedings of 2002 International Joint Conference on Neural Networks (IJCNN 2002), IEEE, IEEE Press, 12–17 May 2002, pp 1779–1784
39. Gharehchopogh FS (2023) Quantum-inspired metaheuristic algorithms: comprehensive survey and classification. Artif Intell Rev 56(6):5479–5543
40. Glover F, Lewis M, Kochenberger G (2018) Logical and inequality implications for reducing the size and difficulty of quadratic unconstrained binary optimization problems. Eur J Oper Res 265(3):829–842
41. Glover F, Kochenberger G, Du Y (2022) Applications and computational advances for solving the QUBO model. In: The Quadratic Unconstrained Binary Optimization Problem: theory, Algorithms, and Applications Punnen AP (ed). Springer International Publishing, Cham, pp 39–56
42. Glover F, Kochenberger G, Hennig R, Du Y (2022) Quantum bridge analytics I: a tutorial on formulating and using QUBO models. Ann Oper Res 314:141–183
43. Gomes C, Fernandes JP, Falcao G, Kar S, Tayur S (2024) A systematic mapping study on quantum and quantum-inspired algorithms in operations research. ACM Comput Surv 57(3):66
44. Grover LK (1996) A fast quantum mechanical algorithm for database search. In: Proceedings of the 28th ACM Symposium on the Theory of Computing, pp 212–219
45. Grover LK (1997) Quantum computers can search arbitrarily large databases by a single query. Phys Rev Lett 79:4709–4712
46. Han K-H, Kim J-H (2002) Quantum-inspired evolutionary algorithm for a class of combinatorial optimization. IEEE Trans Evol Comput 6(6):580–593
47. Han K-H, Kim J-H (2004) Quantum-inspired evolutionary algorithms with a new termination criterion, H_ε gate and two-phase scheme. IEEE Trans Evol Comput 8(3):156–169
48. Harik G, Lobo F, Goldberg D (1998) The compact genetic algorithm. In: Proceedings of the International Conference on Evolutionary Computation (CEC 1998), IEEE, IEEE Press, New Jersey, pp 523–528
49. Harrow AW, Hassidim A, Lloyd S (2009) Quantum algorithm for solving linear systems of equations. Phys Rev Lett 103:150502
50. Hastings WK (1970) Monte Carlo sampling methods using Markov chains and their applications. Biometrika 57(1):97–109
51. Hauke P, Katzgraber HG, Lechner W, Nishimori H, Oliver WD (2020) Perspectives of quantum annealing: methods and implementations. Rep Progr Phys 83(5):054401
52. Hirvensalo M (2002) Quantum computing – Facts and Folklore. Natural Comput 1:135–155
53. Hoefler T, Häner T, Troyer M (2023) Disentangling hype from practicality: on realistically achieving quantum advantage. Commun ACM 66:82–87
54. Holliday J, Morgan B, Churchill H, Luu K (2024) Hybrid quantum tabu search for solving the vehicle routing problem. In: 2024 IEEE International Conference on Quantum Computing and Engineering (QCE)
55. Jiao L, Li Y (2005) Quantum-inspired immune clonal optimization. In: Proceedings of 2005 International Conference on Neural Networks and Brain (ICNN&B 2005), IEEE, IEEE Press, 13–15 October 2005, pp 461–466

56. Kassal I, Jordan SP, Love PJ, Mohseni M, Aspuru-Guzik A (2008) Polynomial-time quantum algorithm for the simulation of chemical dynamics. Proc Nat Acad Sci USA 105:18681–18686
57. Kennedy J, Eberhart R (1995) Particle swarm optimization. In: Proceedings of IEEE International Conference on Neural Networks, IEEE, IEEE Press, pp 1942–1948
58. Kennedy J, Eberhart R (1997) A discrete binary version of the particle swarm algorithm. In: Proceedings of IEEE International Conference on Systems, Man and Cybernetics, IEEE, IEEE Press, October 1997, pp 4104–4109
59. Kim J, Min D, Cho J, Jeong H, Byun I, Choi J, Hong J, Kim J (2024) A fault-tolerant million qubit-scale distributed quantum computer. In: Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (New York, NY), ASPLOS '24, Association for Computing Machinery, pp 1–19
60. King AD, Raymond J, Lanting T, Harris R, Zucca A, Altomare F, Berkley AJ, Boothby K, Ejtemaee S, Enderud C, Hoskinson E, Huang S, Ladizinsky E, MacDonald AJR, Marsden G, Molavi R, Oh T, Poulin-Lamarre G, Reis M, Rich C, Sato Y, Tsai N, Volkmann M, Whittaker JD, Yao J, Sandvik AW, Amin MH (2023) Quantum critical dynamics in a 5,000-qubit programmable spin glass. Nature 617(7959):61–66
61. Kirkpatrick S, Gelatt C, Vecchi M (1983) Optimization by simulated annealing. Science 220(4598):671–680
62. Klug F (2024) Quantum optimization algorithms in Operations Research: methods, applications, and implications, Technical Report, Munich University of Applied Sciences, 81243 Munich, Am Stadtpark 20
63. Lachance-Quirion D, Lemonde MA, Simoneau JO, St-Jean L, Lemieux P, Turcotte S, Wright W, Lacroix A, Fréchette-Viens J , Shillito R, Hopfmueller F, Tremblay M, Frattini NE, Lemyre JC, St-Jean P (2024) Autonomous quantum error correction of gottesman-kitaev-preskill states. Phys Rev Lett 132:150607
64. Layeb A (2010) Hybrid quantum scatter search algorithm for combinatorial optimization problems. Ann Comput Sci Ser 8:1–18
65. Le Gall F (2014) Improved quantum algorithm for triangle finding via combinatorial arguments. In: Proceedings of the 55th Annual Symposium on Foundations of Computer Science (FOCS 2014), IEEE, October 2014, pp 216–225
66. Lee C-D, Chen Y-J, Huang H-C, Hwang R-C, Yu G-R (2004) The non-stationary signal prediction by using quantum NN. In: Thissen et al (eds) Proceedings of 2004 IEEE International Conference on Systems, Man and Cybernetics, IEEE, IEEE Press, New Jersey, pp 3291–3295
67. Levit A, Crawford D, Ghadermarzy N, Oberoi JS, Zahedinejad E, Ronagh P (2017) Free energy-based reinforcement learning using a quantum processor. CoRR abs/1706.00074:1–8
68. Li Y, Zhang Y, Zhao R, Jiao L (2004) The immune quantum-inspired evolutionary algorithm. In: Thissen W et al (eds) Proceedings of 2004 IEEE International Conference on Systems, Man and Cybernetics, IEEE, IEEE Press, New Jersey, pp 3301–3305
69. Li RY, Felice RD, Rohs R, Lidar DA (2018) Quantum annealing versus classical machine learning applied to a simplified computational biology problem. npj Quantum Inf 4:14
70. Li Y, Tian M, Liu G, Peng C, Jiao L (2020) Quantum optimization and quantum learning: a survey. IEEE Access 8:23568–23593
71. Liu W-J, Gao P-P, Yu W-B, Qu Z-G, Yang C-N (2018) Quantum relief algorithm. Quant Inf Process 17(10):280
72. Mari A, Bromley TR, Izaac J, Schuld M, Killoran N (2020) Transfer learning in hybrid classical-quantum neural networks. Quantum 4:340
73. Martonak R, Santoro G, Tosatti E (2004) Quantum annealing of the traveling salesman problem. Phys Rev E 70(057701):1–4
74. Martyn JM, Rossi ZM, Tan AK, Chuang IL (2021) Grand unification of quantum algorithms. PRX Quantum 2:040203
75. McGarraghy S, Phelan M (2011) Generating supply chain ordering policies using quantum inspired genetic algorithms and grammatical evolution. In: Supply Chain Optimization,

Design & Management: advances & Intelligent methods, Minis I, Zeimpekis V, Dounias G, Ampazis N (eds). IGI Global, Hershey, pp 125–154
76. Metropolis N, Rosenbluth AW, Rosenbluth MN, Teller AH, Teller E (1953) Equations of state calculations by fast computing machines. J Chem Phys 21(6):1087–1092
77. Montanaro A (2016) Quantum algorithms: an overview. npj Quantum Inf 2:15023
78. Montiel O, Rubio Y, Olvera C (2019) Quantum-inspired Acromyrmex evolutionary algorithm. Sci Rep 9:12181
79. Narayanan A, Moore M (1996) Quantum-inspired genetic algorithms. In: Proceedings of IEEE International Conference on Evolutionary Computation, IEEE, IEEE Press, May 1996, pp 61–66
80. Narottama B, Hendraningrat DK, Shin SY (2022) Quantum-inspired evolutionary algorithms for NOMA user pairing. ICT Express 8(1):11–17
81. Negrevergne C, Mahesh TS, Ryan CA, Ditty M, Cyr-Racine F, Power W, Boulant N, Havel T, Cory DG, Laflamme R (2006) Benchmarking quantum control methods on a 12-qubit system. Phys Rev Lett 96(17):170501
82. Pampara G, Franken N, Engelbrecht A (2005) Combining particle swarm optimisation with angle modulation to solve binary problems. In: Proceedings of IEEE Congress on Evolutionary Computation 2005 (CEC 2005), vol. 1, IEEE, IEEE Press, New Jersey, 2–5 September 2005, pp 89–96
83. Parekh O (2023) Synergies between operations research and quantum information science. INFORMS J Comput 35(2):266–273
84. Penrose R (2004) The Road to Reality: a Complete Guide to the Physical Universe. Jonathon Cape, Chatham
85. Pincus M (1970) A Monte Carlo method for the approximate solution of certain types of constrained optimization problems. Oper Res 18:1225–1228
86. Platel M, Schliebs S, Kasabov N (2009) Quantum-inspired evolutionary algorithm: a multimodal EDA. IEEE Trans Evol Comput 13(6):1218–1232
87. Porowski J, Da M, McGarraghy S (2009) Quantum annealing in traveling salesman related problems, Technical Report, University College Dublin
88. Preskill J (2018) Quantum computing in the NISQ era and beyond. Quantum 2:79
89. Rechenberg I (1973) Evolutionsstrategie: optimierung technisher Systeme nach Prinzipien der biologischen Evolution. Fromman-Holzboog Verlag, Stuttgart
90. Ross OHM (2020) A review of quantum-inspired metaheuristics: going from classical computers to real quantum computers. IEEE Access 8:814–838
91. Rubio Y, Olvera C, Montiel O (2021) Quantum-inspired evolutionary algorithms on IBM quantum experience. Eng Lett 29(4):1573
92. Sadeghi Hesar A, Houshmand M (2024) A memetic quantum-inspired genetic algorithm based on tabu search. Evol Intell 17:1837–1853
93. Santoro GE, Tosatti E (2006) Optimization using quantum mechanics: quantum annealing through adiabatic evolution. J Phys A Math Gen 39:R393–R431
94. Santoro GE, Tosatti E (2007) Computing: quantum to classical and back. Nat Phys 3:593–594
95. Schechter M, Laflorencie N (2006) Quantum spin glass and the dipolar interaction. Phys Rev Lett 97(13):137204
96. Shi Y (ed) (2004) Proceedings of IEEE Congress on Evolutionary Computation 2004 (CEC 2004), vol 1, IEEE, IEEE Press, New Jersey, 19–23 June 2004
97. Shor PW (1994) Algorithms for quantum computation: discrete logarithms and factoring. In: Proceedings of the 28th ACM Symposium on the Theory of Computing (Piscataway, NJ), IEEE, IEEE Press, November 1994, pp 124–134
98. Shor PW (1995) Scheme for reducing decoherence in quantum computer memory. Phys Rev A 52:R2493–R2496
99. Shor PW (1997) Quantum computing. SIAM J Comput 26:1484
100. Shor PW (1998) Quantum computing. Documenta Mathematica (Extra Volume) ICM(I): 467–486

101. Shukla A, Vedula P (2023) A generalization of Bernstein–Vazirani algorithm with multiple secret keys and a probabilistic oracle. Quantum Inf Process 22(6):244
102. Somma RD, Batista CD, Ortiz G (2007) Quantum approach to classical statistical mechanics. Phys Rev Lett 99(3):030603. https://arxiv.org/quant-ph/0609216v2
103. Somma RD, Boixo S, Barnum H (2007) Quantum simulated annealing. Preprint at http://arxiv.org/quant-ph/0712.1008v1
104. Szwarcman D, Civitarese D, Vellasco M (2022) Quantum-inspired evolutionary algorithm applied to neural architecture search. Appl Soft Comput 120:108674
105. Taillard É, Gambardella LM, Gendreau M, Potvin J-Y (2001) Adaptive memory programming: a unified view of metaheuristics. Eur J Oper Res 135:1–16
106. Talbi H, Draa A (2017) A new real-coded quantum-inspired evolutionary algorithm for continuous optimization. Appl Soft Comput 61:765–791
107. Thissen W, Wieringa P, Ludema M (eds) (2004) Proceedings of 2004 IEEE International Conference on Systems, Man and Cybernetics, IEEE, IEEE Press, New Jersey, 10–13 October 2004
108. Tsai X-Y, Chen Y-J, Huang H-C, Chuang S-J, Hwang R-C (2005) Quantum NN vs NN in signal recognition. In: Proceedings of the Third International Conference on Information Technology and Applications (ICITA 05), IEEE, IEEE Press, 4–7 July 2005, pp 308–312
109. Vandersypen LMK, Steffen M, Breyta G, Yannoni CS, Sherwood MH, Chuang IL (2001) Experimental realization of Shor's quantum factoring algorithm using nuclear magnetic resonance. Nature 414:883–887
110. Wang C, Jonckheere E (2019) Simulated versus reduced noise quantum annealing in maximum independent set solution to wireless network scheduling. Quantum Inf Process 18:6
111. Wang Y, Feng XY, Huang YX, Pu DB, Zhou WG, Liang YC, Zhou CG (2007) A novel quantum swarm evolutionary algorithm and its applications. Neurocomputing 70(4–6): 633–640
112. Weinberg SJ, Sanches F, Ide T, Kamiya K, Correll R (2022) Supply chain logistics with quantum and classical annealing algorithms. Sci Rep 13:4770
113. Wilkins A (2023) Record-breaking quantum computer has more than 1000 qubits, New Scientist
114. Wootters WK, Zurek WH (1982) A single quantum cannot be cloned. Nature 299:802–803
115. Xiong H, Wu Z, Fan H, Li G, Jiang G (2018) Quantum rotation gate in quantum-inspired evolutionary algorithm: a review, analysis and comparison study. Swarm Evol Comput 42:43–57
116. Yang S, Wang M, Jiao L (2004) A genetic algorithm based on quantum chromosome. In: Proceedings of IEEE International Conference on Signal Processing (ICSP 04), IEEE, IEEE Press, 31 August–4 September 2004, pp 1622–1625
117. Yang S, Wang M, Jiao L (2004) A novel quantum evolutionary algorithm and its application. In: Shi Y (ed) Proceedings of IEEE Congress on Evolutionary Computation 2004 (CEC 2004), vol 1, IEEE, IEEE Press, New Jersey, pp 820–826
118. Yang S, Wang M, Jiao L (2004) A quantum particle swarm optimization. In: Shi Y (ed) Proceedings of IEEE Congress on Evolutionary Computation 2004 (CEC 2004), vol 1, IEEE, IEEE Press, New Jersey, pp 320–324
119. Yang W, Wang H, Du Y, Wang Y, Lü Z, Hao J-K (2025) Enhanced open-source scatter search algorithm for solving quadratic unconstrained binary optimization problems. Comput Oper Res 182:107137
120. Yarkoni S, Raponi E, Bäck T, Schmitt S (2022) Quantum annealing for industry applications: introduction and review. Rep Progress Phys 85(10):104001
121. Ye Z, Lu Y (2022) Quantum science: a review and current research trends. J Manag Analy 9(3):383–402
122. Yulianti LP, Surendro K (2022) implementation of quantum annealing: a systematic review. IEEE Access 10:73156–73177

Restart Strategies

12

Oleg V. Shylo and Oleg A. Prokopyev

Contents

Introduction 320
Basic Restart Strategies 321
Restart Distribution and Optimal Restart Strategies 323
Single Algorithm Portfolios of Restart Algorithms 326
Mixed Algorithm Portfolios of Restart Algorithms 331
Conclusions 332
Cross-References 333
References 333

Abstract

This chapter is focused on restart strategies in optimization, which often provide a substantial algorithmic acceleration for randomized optimization procedures. Theoretical models that describe optimal restart strategies are presented alongside with their relations to parallel computing implementations.

Keywords

Algorithm · Algorithm portoflio · Parallel optimization · Restart · Superlinear speedup

O. V. Shylo (✉)
University of Tennessee, Knoxville, TN, USA
e-mail: oshylo@utk.edu

O. A. Prokopyev
University of Pittsburgh, Pittsburgh, PA, USA
e-mail: droleg@pitt.edu

© Springer Nature Switzerland AG 2025

R. Martí et al. (eds.), *Handbook of Heuristics*,
https://doi.org/10.1007/978-3-032-00385-0_15

Introduction

When evaluating performance of an optimization method, the most commonly reported statistics are running time (or execution time) and solution quality. Often these statistics are sensitive to initial conditions and algorithm parameters, especially in randomized algorithms. To capture this variability when analyzing computational performance, it is common to report a sequence of run-times and the corresponding objective function values (or their averages and some measures of dispersion) for different problem instances, initial conditions, and parameter settings. In general, it is natural to consider a probability distribution of time that an algorithm requires to obtain a solution of a given quality (e.g., an optimal solution or a solution whose value is within factor α from optimal). The variability in run-times can often be attributed to randomized algorithmic steps, such as randomized branching rules and column selection in branch-and-price methods, arbitrarily resolved ties, stochastic initial solutions, and random local search moves in metaheuristics.

In some applications, the distribution of the algorithm's run-time has a large spread, and there is a relatively high probability of having a run-time that is far from the average run-time value [6]. Such peculiar distributions can be exploited to accelerate the search via multi-start strategy: each run has a limited duration, and the algorithm is repeatedly restarted with different initial conditions or random seeds. Importantly, such acceleration opportunity is not just a mathematical curiosity, but it is commonly found in many real-life applications providing significant acceleration for the state-of-the-art optimization algorithms; see, e.g., [6, 12, 16, 18].

If a probability distribution of an algorithm's run-time admits such acceleration, it is referred to as a *restart distribution*. In [18], the authors provide the formal definition of a restart distribution. The authors also establish the theoretical explanation of the efficient restart strategies for enumeration algorithms. A search tree model is described in [2] to provide a formal proof of heavy-tailed behavior in imbalanced search trees.

Incorporating variability into deterministic methods may lead to faster algorithms. Randomizing certain decision in the deterministic algorithms may also induce restart distribution of run-times. This type of acceleration has been reported for various applications of randomized backtracking search. In [6], random steps are embedded in a complete search algorithm for the constraint satisfaction problem and the propositional satisfiability problem, and it is shown that the proposed randomization accelerates deterministic methods.

Finally, multi-start is a popular framework in many metaheuristics (see, e.g., [3, 13, 14, 16]). However, the impact of restart strategies on the computational performance is often underestimated. One of the most successful metaheuristics—tabu search—and its run-time distributions in the context of restart strategies are discussed in [19].

The following example of a restart distribution is based on a randomly generated instance of the directed *maximum cut problem* [20] with 400 vertices, where every possible edge between vertices is included in the graph with probability $\frac{1}{2}$ (both

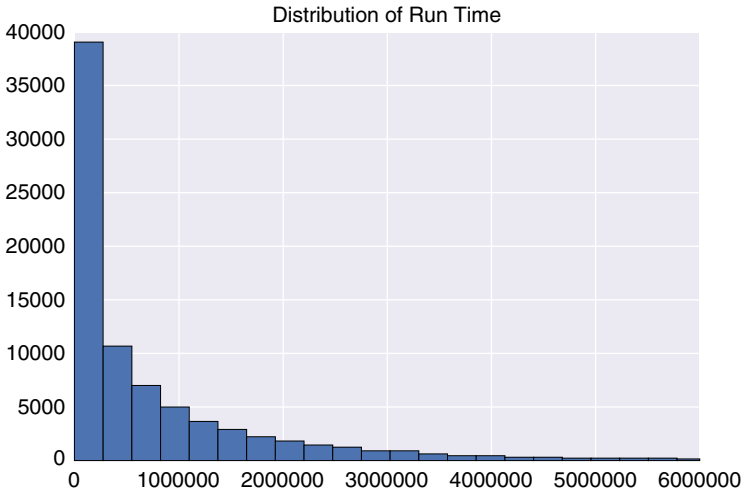


Fig. 1 The distribution of run-time of a tabu search algorithm for an instance of the maximum cut problem based on 80,000 runs

directions are considered separately). This random instance is repeatedly solved by a simple version of the *tabu search* method [4] starting from different initial solutions, terminating when the algorithm finds a solution whose objective is better than a predefined threshold value. The implementation of the tabu search in this example has a fixed tabu tenure of ten iterations, and it does not include any advanced features, except for the cycle detection [4].

The distribution of the run-times is presented in Fig. 1. Markedly, it can be observed that the probability of large computational times is rather high or, in other words, the run-time has a heavy-tailed distribution (the tails are not exponentially bounded). In order to accelerate the search, one can select a restart parameter value and repeatedly restart the algorithm after the number of iterations exceeds this predefined parameter. Figure 2 provides the average run times for the tabu algorithm as a function of the number of iterations between restarts. It is clear from the plot that certain values of the restart parameter guarantee superior algorithmic performance, when compared to the original algorithm that does not employ any restart strategy. In this context, an interesting question can be stated: What is the best restart value for algorithm acceleration? The following discussion provides an overview of the research that addresses this important question.

Basic Restart Strategies

Unlike Monte Carlo algorithms, which are guaranteed to provide approximate answers or solutions within a fixed time, the Las Vegas-type algorithms are defined as algorithms that always find "correct solutions," but the required computational

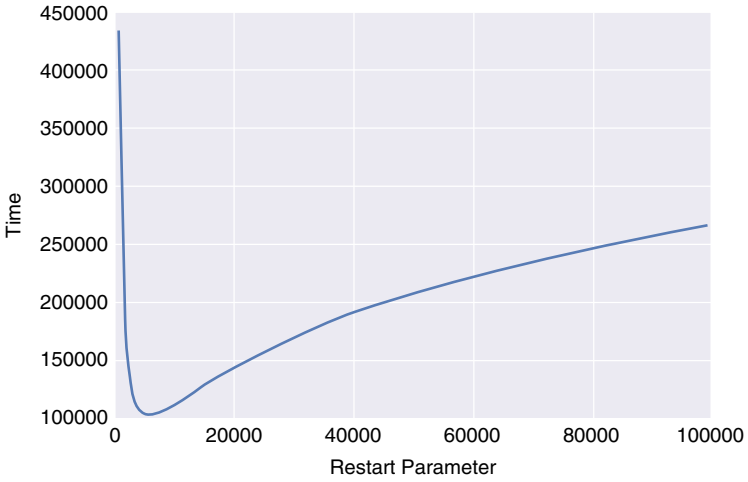


Fig. 2 The average run-time of a tabu search algorithm for an instance of the maximum cut problem as a function of the restart parameter (the same instance as in Fig. 1)

time is uncertain. To define the correctness of a solution, one can demand a proof of optimality or comparison of the solution objective function to a predefined threshold, in which case the "correct solution" is any solution with the objective function better than the threshold.

In general, the run-time of an algorithm can be measured either in iterations or in computational time. A *restart strategy* associated with a given algorithm is typically defined (see, e.g., [10, 22]) as a sequence, $S = \{t_1, t_2, \ldots\}$, that represents allocated run-times between restarts. According to the strategy S, the first run continues for t_1 time units (iterations) or until the correct answer is found. The second run lasts t_2 time units or until the correct answer is found and so on. The runs are independent from each other, as no information is passed between different computational runs. As briefly discussed in section "Introduction," different restart strategies might lead to different average run-times.

In [10], the authors consider integer run-times that represent the total number of iterations until a desired outcome is achieved. They consider a wide domain of restart strategies including strategies with random restart intervals and those that enable suspension of algorithm execution in favor of another run with an option to continue its execution later. It is shown that there always exists a *uniform restart strategy* (i.e., $t_1 = t_2 = \ldots = t^*$) that is optimal. It is easy to see that the same result holds if a run-time is determined using continuous time units instead of iterations—for example, consider rounding continuous times to the nearest integral values (number of seconds) and redefining the run-time distribution accordingly.

To find an optimal restart period t^*, a good estimate of the underlying run-time distribution is required, which might be problematic in practice. If this information is not available, the *universal restart strategy* introduced in [10] can provide a decent

performance guarantee. In this strategy, all run length are powers of 2 starting with a run of length 2^0 (this sequence may be rescaled). Considering the duration of the current run, if the number of previous runs with the same length is even, then the next run duration is double of the current run duration. Otherwise, the next run has the length equal to one. These rules define the following numerical sequence:

$$S^{\text{univ}} = \{1, 1, 2, 1, 1, 2, 4, 1, 1, 2, 1, 1, 2, 4, 8, 1, \dots\}$$

When comparing S^{univ} to the optimal restart strategy, its performance is within a logarithmic factor from optimality without any requirements on prior knowledge of the run-time distribution [10]. Formally, if T^{univ} denotes the expected run-time of the algorithm that uses universal restart strategy and T^{opt} denotes the expected run time under the optimal restart strategy, then

$$T^{\text{univ}} \leq 192 \cdot T^{\text{opt}} \left(\log_2 \left(T^{\text{opt}}\right) + 5\right).$$

Unfortunately, the difference between run-times of the universal and optimal strategies may be large in practice, which limits the applicability of this important theoretical result.

Restart strategies can be naturally extended to parallel optimization [9], where copies of the same algorithm are executed in parallel. Note that due to randomness, the search trajectories are in general different. As in the serial setting, a restart strategy defines maximum run-times for each copy of the algorithm, i.e.,

$$S = \begin{cases} t_1^1, t_2^1, t_3^1, \dots \\ t_1^2, t_2^2, t_3^2, \dots \\ \dots \\ t_1^n, t_2^n, t_3^n, \dots \end{cases} \tag{1}$$

In a uniform parallel restart strategy, each copy of the algorithm has the same restart schedule (i.e., $t_j^i = t^*$). This strategy is no longer optimal as it was in the serial case, but its performance is provably within a constant factor from optimality. An example of nonuniform optimal strategy can be found in [9].

Restart Distribution and Optimal Restart Strategies

Next, we overview some formal characterizations of run-time distribution that can be exploited to gain computational acceleration. Our discussion in this and the next sections is mostly based on the results from [22].

Consider a randomized algorithm A for solving a problem instance P. The execution time of A when solving P is a random variable, ξ. Based on the algorithm A, one can define its uniform restart version, A_R, which follows the same algorithmic steps as A, but is restarted after R iterations or time units if the correct solution is

not found. Similarly to A, algorithm A_R terminates as soon as the correct solution is obtained. The positive parameter R is referred to as a *restart period* (or a *restart parameter*) of A_R.

The number of restarts before the first successful run of A_R is a geometric random variable, and the expected run-time, $T(R)$, of A_R can be expressed as

$$T(R) = R \cdot \frac{1 - \Pr\{\xi \leq R\}}{\Pr\{\xi \leq R\}} + E[\xi | \xi \leq R] \tag{2}$$

The first part of (2) is an expected number of restarts multiplied by the duration of each run, while the second is an average duration of the run that produces the final solution.

Definition 1 A probability distribution of a random variable ξ, $Pr\{\xi \leq x\}$, is called a *restart distribution* if there exists $R > 0$, such that

$$R \cdot \frac{1 - \Pr\{\xi \leq R\}}{\Pr\{\xi \leq R\}} + E[\xi | \xi \leq R] < E[\xi] \tag{3}$$

From (3), if a run-time of an algorithm follows a restart distribution, then the algorithm has a restart version that outperforms the algorithm without restarts in terms of its average run-time. Clearly, the properties of the run-time distribution depend on the optimization problem and the algorithm itself. If, for example, an optimization algorithm uses an optimal restart policy, then its run-time will no longer follow a restart distribution.

A log-normal distribution is an example of a restart distribution. For example, consider three log-normal random variables with the same location parameter, $\mu = 1$. The scale parameters, σ, are 2, 2.5, and 1.5. The expected values of these random variables are 20.08, 61.86, and 8.37, respectively. If these variables represented run-times of some algorithms, we would be able to achieve acceleration by restarting after R units of run-time. Figure 3 illustrates this opportunity by showing the expected run-time given by (2) as a function of restart period R. It shows that restarting an algorithm allows us to achieve significant acceleration for these run-time distributions. Furthermore, we can see that the larger variability in run-times provides a better opportunity for acceleration by the restart mechanism.

The optimality conditions can be derived for continuous and differentiable (first and second derivative) cumulative distribution functions (CDF) of run-times (see [22] for more details). If this is not the case, then a continuous and differentiable approximation of original CDF can be used instead.

Proposition 1 *Let R^* be an optimal restart period for A_R. Then the expected run-time of A_{R^*} is*

$$T(R^*) = \frac{1 - \Pr\{\xi \leq R^*\}}{\left.\frac{dPr\{\xi \leq R\}}{dR}\right|_{R=R^*}} \tag{4}$$

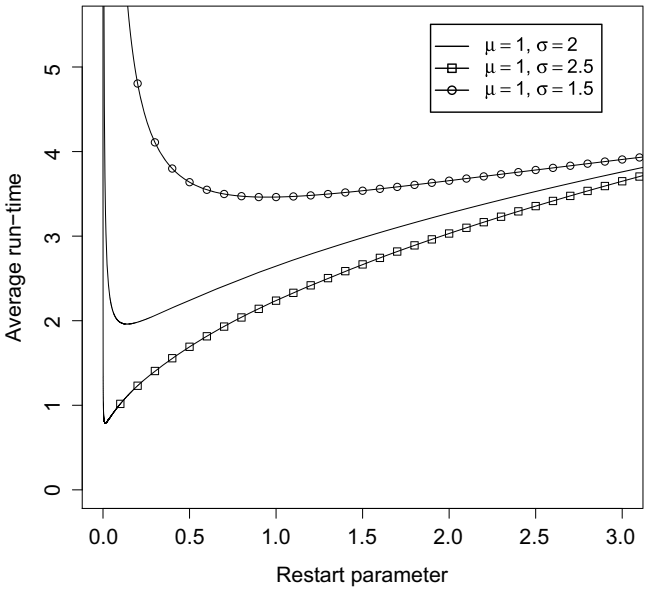


Fig. 3 The average run-time as a function of the restart parameter, R, for log-normally distributed run-times

Proof

$$\frac{dT(R)}{dR} = \frac{1 - \Pr\{\xi \leq R\}}{\Pr\{\xi \leq R\}} + R \cdot \left(\frac{-\frac{dPr\{\xi \leq R\}}{dR}}{\Pr\{\xi \leq R\}} - \frac{1 - \Pr\{\xi \leq R\}}{(\Pr\{\xi \leq R\})^2} \cdot \frac{dPr\{\xi \leq R\}}{dR} \right)$$

$$+R \cdot \frac{dPr\{\xi \leq R\}}{dR} \frac{1}{\Pr\{\xi \leq R\}} + \frac{-\frac{dPr\{\xi \leq R\}}{dR}}{(\Pr\{\xi \leq R\})^2} \int_0^R x f(x) dx$$

$$= \frac{1 - \Pr\{\xi \leq R\}}{\Pr\{\xi \leq R\}} - \frac{\frac{dPr\{\xi \leq R\}}{dR}}{\Pr\{\xi \leq R\}} \left(R \cdot \frac{1 - \Pr\{\xi \leq R\}}{\Pr\{\xi \leq R\}} + E[\xi | \xi \leq R] \right)$$

$$= \frac{1 - \Pr\{\xi \leq R\}}{\Pr\{\xi \leq R\}} - \frac{\frac{dPr\{\xi \leq R\}}{dR}}{\Pr\{\xi \leq R\}} \cdot T(R)$$

The reciprocal of the expression in the right-hand side of (4) is known as a *hazard rate function*, which is an important concept in reliability engineering [17]. The hazard rate function, or failure rate, can be used to describe general properties of restart distributions.

Proposition 2 *A hazard rate function of a restart distribution is nonincreasing on some interval containing an optimal restart period.*

This property allows us to rule out the distribution that has an increasing hazard rate functions [7]. The previous proposition shows, for example, that the following distributions are not restart distributions:

- Weibull distributions with the shape parameter $k > 1$,
- Gamma distribution with the shape parameter $k > 1$,
- Uniform distribution,
- Normal distribution.

Single Algorithm Portfolios of Restart Algorithms

Algorithmic acceleration can be achieved by combining algorithms in portfolios that are executed concurrently in a distributed manner. For example, a *single algorithm portfolio* consists of different copies of the same algorithm that are deployed on different processors. This simple parallelism can be extremely efficient in practical applications that involve randomized algorithms. There are numerous reports of superlinear speedup when using this type of parallelization (e.g., see [15]). By *superlinear speedup*, we imply that the algorithm utilizes up to n processors and on average is more than n time faster than the algorithm utilizing one processor.

The superlinear speedup can be related to the concept of restart distributions. Consider a single restart algorithm portfolio: a parallel algorithm A_R^n that consists of n independent copies of A_R running in parallel (no communication), where, as previously, A_R is the restart version A with parameter R. The algorithm A_R^n halts whenever one of n copies finds a correct solution. Let random variable $\xi_{\min}^n$ denote the run-time of A_R^n, while $T_n(R)$ denotes the expected run-time of A_R^n.

The expected run-time of the single algorithm portfolio with a uniform restart strategy is given by

$$T_n(R) = R \cdot \frac{(1 - \Pr\{\xi \leq R\})^n}{1 - (1 - \Pr\{\xi \leq R\})^n} + E\left[\xi_{\min}^n | \xi_{\min}^n \leq R\right] \tag{5}$$

An optimality condition similar to the serial case can be derived for parallel restart algorithms (see [22]).

Proposition 3 *Let R_n^* be the optimal restart period for A_R^n; then the expected running time of A_R^n is*

$$T_n\left(R_n^*\right) = \frac{\left(1 - \Pr\left\{\xi \leq R_n^*\right\}\right)}{n \cdot \left.\frac{dPr\{\xi \leq R\}}{dR}\right|_{R=R_n^*}} \tag{6}$$

As mentioned earlier, the log-normal distribution is a restart distribution. We use this distribution to illustrate some of the ideas, since we can easily calculate the exact values for the conditional expectations in (5) without resorting to statistical sampling. For example, we can use the log-normal run-time to provide an illustration of the superlinear speedup.

Let ξ be a random variable with a log-normal distribution with parameters $\sigma = 2$ and $\mu = 1$. We define a *parallel speedup* as a ratio between run-time of a serial algorithm and run-time of its parallel version. Firstly, we consider the speedup for the algorithm that does not apply any restart strategy. The data presented in Fig. 4 clearly indicates the superlinear average speedup relative to the run-time of the serial algorithm.

The following proposition from [22] states that if the superlinear parallel speedup is achieved by a single algorithm portfolio, then the underlying distribution of the run-time is a restart distribution.

Proposition 4 *If R^* and R_n^* are optimal restart periods for A_R and A_R^n, respectively, then* $\frac{T(R^*)}{T_n(R_n^*)} \leq n$.

Simply speaking, Proposition 4 indicates that the superlinear parallel speedup can be attributed to the inefficiencies of the serial algorithm, and these inefficiencies can be alleviated by adopting an appropriate restart strategy.

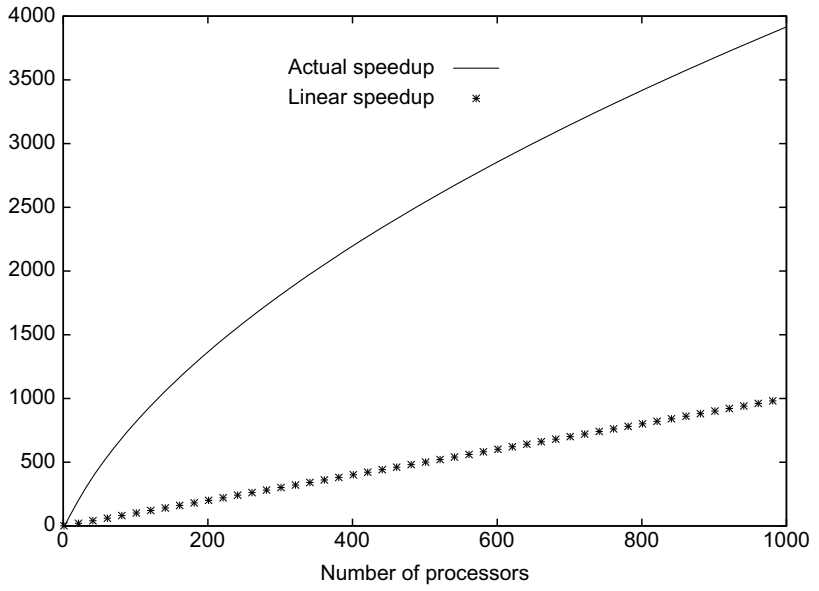


Fig. 4 Speedup is obtained by comparing the **serial no-restart** version with the **parallel no-restart** version [22]; ξ has log-normal distribution with $\mu = 1$ and $\sigma = 2$

Consider now the situation when both serial and parallel algorithms implement optimal restart strategies. It is important to note that the optimal values in parallel and serial cases are not necessarily the same. The following proposition relates the number of processors to the value of the optimal restart parameter.

Proposition 5 *If the hazard rate function of the run-time distribution is unimodal and* $\frac{T(R^*)}{T_n(R_n^*)} < n$, *then* $R^* < R_n^*$.

Intuitively, it might seem that the best serial algorithm is the best candidate for parallelizing. However, Proposition 5 shows that the optimal restart parameter for the serial algorithm is not necessarily optimal when considering the performance of the corresponding single algorithm portfolio. In other words, good average performance in serial setting can often provide suboptimal parallel performance.

Suppose that the run-time of a serial algorithm follows the same log-normal distribution as in the previous example. The optimality conditions (4) and (6) for the optimal restart parameters provide the corresponding optima. Figure 5 illustrates the computational speedup achieved by the parallel algorithms with optimal restart parameters and varying number of parallel processors compared to the serial algorithm (for the log-normal run-times). The restart parameters of the serial and parallel algorithms are different: in fact, it turns out that the optimal restart parameter is monotonically increasing as a function of the number of processors. The speedup is almost linear for the number of processors below 100. For large

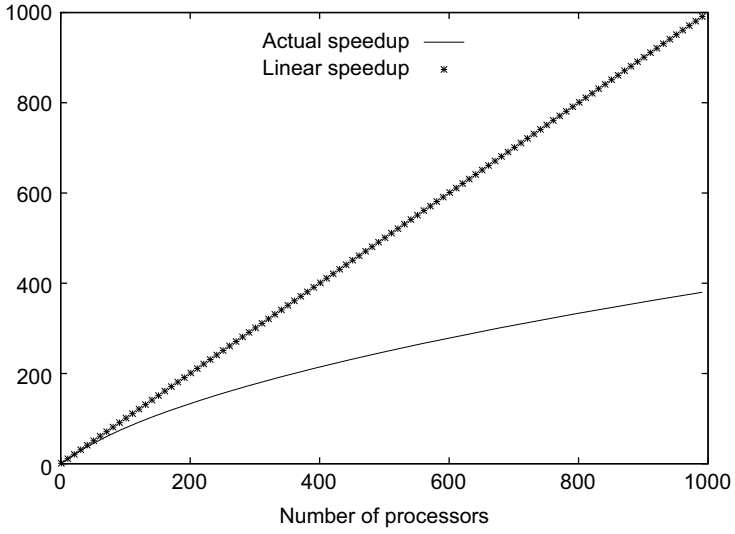


Fig. 5 Speedup is obtained by comparing the **optimal serial restart** version with the **optimal parallel restart** version [22]; ξ has log-normal distribution with $\mu = 1$ and $\sigma = 2$

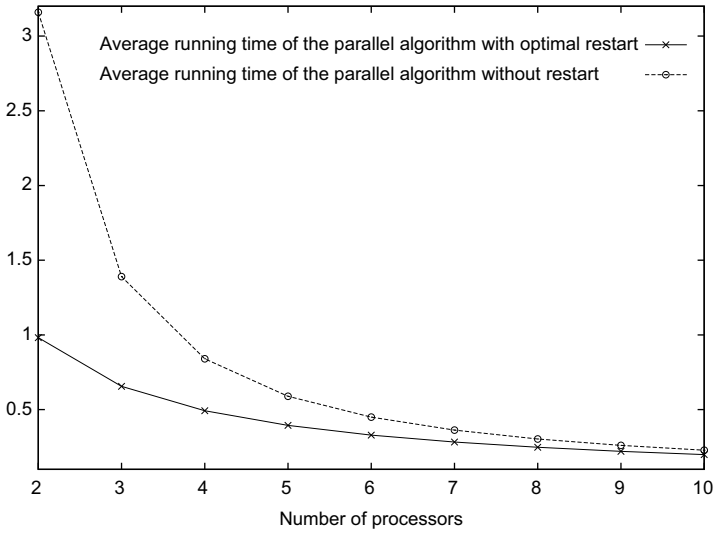


Fig. 6 Comparison of the parallel algorithms with and without implementation of the optimal restart strategy [22]; ξ has log-normal distribution with $\mu = 1$ and $\sigma = 2$

numbers of processors, the speedup quickly becomes sublinear, and the single algorithm portfolio parallelization becomes less effective.

Another interesting question that is important for practical applications is: what is the value of knowing the optimal restart parameter? Figure 6 shows the value of implementing optimal restart strategy with respect to the number of processors. As the number of processors increases, the benefit of knowing the exact value of optimal restart parameter is steadily decreasing. In some sense, the single algorithm portfolio framework takes advantage of restart-distribution properties without restarting the algorithm. The performance can be improved by applying an effective restart strategy explicitly; however, the value of such improvement quickly diminishes as the number of processes increases.

An optimal value of restart parameter is typically unknown in a realistic problem setting. Furthermore, as mentioned previously, the value of the optimal restart parameter may decrease as the number of processors grows. These observations suggest that the main effort should be concentrated on developing and identifying algorithms with run-times that follow restart distributions instead of focusing on the methods that find optimal restart values. The potential for acceleration of run-times that follow restart distributions can be automatically exploited via portfolio parallelization. Furthermore, any errors in estimating the exact value of the optimal restart sequence can degrade the performance with respect to adopting no-restart strategy.

We illustrate this idea by looking at different tenure parameters of tabu search. Again we consider a random instance of the maximum directed cut problem with 400 vertices. The computational experiment is based on a simple tabu search with

different values of tabu tenure that was mentioned in earlier examples. Table 1 shows the results for two tabu algorithms with different tenure parameters. The algorithm with tenure parameter 30 is significantly faster than the algorithm with tenure parameter 10 when deployed on a single processor. However, the situation changes when parallel implementation is considered: the parallel algorithm with tenure 10 is much faster than its serial version. This example (see [19]) shows that if one uses the average serial run-time as the main criterion for choosing an algorithm for parallel implementation, the better parameter choices (or even algorithmic approaches) can be dropped in favor of suboptimal choices.

Figure 7 highlights the value of knowing the optimal restart parameter. The tabu algorithm with tenure 10 without restarts quickly converges to the performance of the same tabu algorithm that uses an optimal restart strategy.

Table 1 Average number of iterations for different number of processors

Number of processors	Average number of iterations	
	Tabu tenure = 10	Tabu tenure = 30
1	192,216	26,073
2	19,950	13,264
4	1049	6996
8	434	3664
16	433	2003
32	325	1124

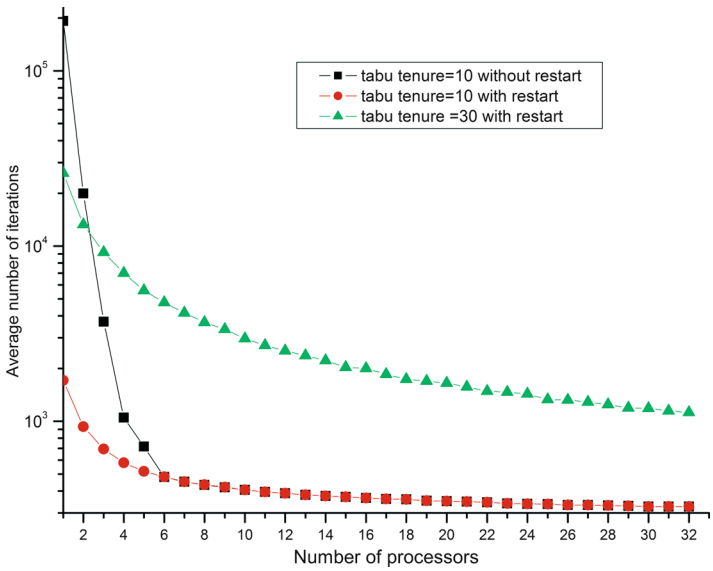


Fig. 7 Computational results for MAXDICUT problem using tabu search in a single algorithm portfolio [19]

Mixed Algorithm Portfolios of Restart Algorithms

Instead of focusing on a single algorithm portfolio, one may consider a combination of different algorithms. The diversity of mixed algorithms can improve the overall performance with respect to a single algorithm portfolio. Suppose that we have a set of available randomized algorithms and we want to select a subset of them to include into a mixed algorithm portfolio of a given size. If the available algorithms are simply different copies of the same algorithm, we call such selection a single algorithm portfolio; otherwise, we will refer to it as a *mixed algorithm portfolio*.

There are a number of examples in the literature of mixed algorithm portfolios that outperform single algorithm portfolios. In particular, the algorithm portfolio approach for constraint satisfaction and mixed integer programming is presented in [5]. The authors show that the mixed algorithm portfolio can outperform a single algorithm portfolio and discuss intuition behind such situations. An efficient algorithm portfolio approach using backtracking search for the graph-coloring problems is considered in [8]. Extensive computational experiments with restart strategies and algorithm portfolios for benchmark instances of network design problem are also investigated in [1].

The mathematical model of mixed portfolios of restart algorithms can be outlined as follows. Consider a set of m algorithms $A_1, \ldots, A_m$ with restart parameters $R_1, \ldots, R_m$ and random run-times $\xi_1, \ldots, \xi_m$, respectively. Additionally, there are N parallel processors that are available, and we need to select N algorithms to deploy on each processor. Each processor should be used by a single algorithm, and the same algorithm can be deployed on multiple processors. We assume that the run-time of each algorithm is an integer multiple of its restart parameter. In other words, even if an algorithm finds a solution in the beginning of the run, an actual run-time will be rounded up to the next restart period.

Using this setup, there are m single algorithm portfolios that can be formed. Let T_s denote the average run-time of the best single algorithm portfolio, which can be easily defined using the properties of the geometric distribution:

$$T_s\left(A_1, \ldots, A_m, N\right) = \min\left\{R_1 \frac{1}{1-p_1^N}, \ldots, R_m \frac{1}{1-p_m^N}\right\}.$$

Let $T_m(A_1, n_1, \ldots, A_m, n_m)$ denote the expected run-time of the mixed algorithm portfolio, which consists of n_1 copies of A_1, n_2 copies of A_2, and so on $\left(\sum_{i=1}^{m} n_i = N\right)$. The mixed algorithm portfolio terminates as soon as one of the algorithms finds the target solution (e.g., a solution with objective below a certain threshold).

To identify the computational benefit that can be achieved by mixing randomized algorithms with different properties, we define the *speedup ratio* S as

$$S = \frac{T_s\left(A_1, \ldots, A_m, n_1 + \cdots + n_m\right)}{T_m\left(A_1, n_1, \ldots, A_m, n_m\right)}. \tag{7}$$

Unlike the serial case, in [9] the authors demonstrate that the best uniform restart strategy repeated on every processor is not necessarily optimal; however, its performance is within a constant factor of the optimal strategy. Furthermore, in [21] and the subsequent work in [11], it is shown that the speedup ratio S of any mixed algorithm portfolio satisfies

$$S \leq \frac{1}{1 - e^{-1}} \approx 1.58. \tag{8}$$

Therefore, if one has a full knowledge of run-time distributions, the best mixed algorithm portfolio is less than two times faster than the best single algorithm portfolio. However, a mixed algorithm portfolio can be viewed as a strategy that can reduce risks associated with a nonoptimal algorithm selection for single algorithm portfolios. Recall that by the definition in (7), the value of S compares the performance of a mixed algorithm portfolio against the best possible algorithm portfolio.

Moreover, it is interesting to note that according to the theoretical approach described above (see derivation details of (8) in [11, 21]), the best performance of a mixed algorithm portfolio is achieved when it consists of N algorithms that can also be used as *candidates to form the best single algorithm portfolio*. Thus, the expected performance of these algorithms in the case of a single algorithm portfolio is the same. However, for the mixed algorithm portfolio, one should select $N - 1$ algorithms with a relatively short restart parameter and exactly one algorithm with a long restart parameter. As all of these algorithms have the same performance in the single portfolio setting, it also implies that each of the former $N - 1$ algorithms is relatively unreliable within its short restart period (thus, these algorithms have to be restarted a large number of times), while the remaining N-th algorithm, despite its large restart parameter, is very reliable (i.e., the corresponding p is close to one). Clearly, the existence of such algorithms is not necessarily guaranteed. Nevertheless, the above result provides an intuitive characterization of effective mixed algorithm portfolios. Namely, if there exists a trade-off that involves the restart parameter value and the algorithm reliability, then it can be exploited within a mixed algorithm portfolio.

Conclusions

In the discussion above, we only consider a setting with a single problem instance. However, the same framework can be easily extended to the setting with multiple problem instances. For example, optimization models of daily locomotive scheduling remain constant, i.e., one needs to find optimal routes using existing railroad network for a given demand. The particular demand patterns can vary substantially from day to day producing different problem instances. We can form a training set by selecting a set of problem instances from historical demands and use them for construction of optimal restart strategies and/or algorithm portfolios. This is also

a typical approach for computational experiments. The researchers often test their techniques on a small set of instances, tune the algorithms, and then conduct the final experiment on a larger set of problems.

Cross-References

- Multi-start Methods
- Tabu Search

Competing Interest Declaration The author(s) has no competing interests to declare that are relevant to the content of this manuscript.

References

1. Chabrier A, Danna E, Pape CL, Perron L (2004) Solving a network design problem. Ann Oper Res 130:217–239
2. Chen H, Gomes CP, Selman B (2001) Formal models of heavy-tailed behavior in combinatorial search. In: CP '01: proceedings of the 7th international conference on principles and practice of constraint programming. Springer, London, pp 408–421
3. D'apuzzo MM, Migdalas A, Pardalos PM, Toraldo G (2006) Parallel computing in global optimization. In: Kontoghiorghes E (ed) Handbook of parallel computing and statistics. Chapman & Hall/CRC, Boca Raton, pp 225–258
4. Glover F, Laguna M (1997) Tabu search. Kluwer Academic, Norwell
5. Gomes CP, Selman B (2001) Algorithm portfolios. Artif Intell 126(1–2):43–62
6. Gomes CP, Selman B, Kautz H (1998) Boosting combinatorial search through randomization. In: Proceedings of the 15th national conference on artificial intelligence. AAAI Press, Madison, pp 431–437
7. Gupta AK, Zeng WB, Wu Y, Gupta AK, Zeng WB, Wu Y (2010) Parametric families of lifetime distributions. In: Gupta AK, Zeng W-B, Wu Y (eds) Probability and statistical models. Birkhäuser, Boston, pp 71–86
8. Huberman BA, Lukose RM, Hogg T (1997) An economics approach to hard computational problems. Science 275(5296):51–54
9. Luby M, Ertel W (1994) Optimal parallelization of Las Vegas algorithms. In: Proceedings of the 11th annual symposium on theoretical aspects of computer science, STACS '94. Springer, London, pp 463–474
10. Luby M, Sinclair A, Zuckerman D (1993) Optimal speedup of Las Vegas algorithms. Inf Process Lett 47:173–180
11. Mostovyi O, Prokopyev OA, Shylo OV (2013) On maximum speedup ratio of restart algorithm portfolios. INFORMS J Comput 25(2):222–229. https://doi.org/10.1287/ijoc.1120.0497
12. Nowicki E, Smutnicki C (2005) An advanced tabu search algorithm for the job shop problem. J Sched 8(2):145–159. https://doi.org/10.1007/s10951-005-6364-5
13. Nowicki E, Smutnicki C (2005) Some new ideas in TS for job shop scheduling. In: Operations research/computer science interfaces series, vol 30, Part II. Springer, pp 165–190
14. Palubeckis G, Krivickiene V (2004) Application of multistart tabu search to the max-cut problem. Inf Technol Control 31(2):29–35
15. Pardalos PM, Rodgers GP (1992) A branch and bound algorithm for the maximum clique problem. Comput Oper Res 19:363–375
16. Resende MG, Ribeiro CC (2011) Restart strategies for grasp with path-relinking heuristics. Optim Lett 5(3):467–478

17. Ross SM (1996) Stochastic processes. Wiley, New York
18. Sergienko IV, Shilo VP, Roshchin VA (2000) Restart technology for solving discrete optimization problems. Cybern Syst Anal 36(5):659–666
19. Sergienko IV, Shilo VP, Roshchin VA (2004) Optimization parallelizing for discrete programming problems. Cybern Syst Anal 40(2):184–189
20. Shylo V, Shylo OV (2011) Path relinking scheme for the max-cut problem within global equilibrium search. IJSIR 2(2):42–51
21. Shylo O, Prokopyev O, Rajgopal J (2011) On algorithm portfolios and restart strategies. Oper Res Lett 39(1):49–52
22. Shylo OV, Middelkoop T, Pardalos PM (2011) Restart strategies in optimization: parallel and serial cases. Parallel Comput 37:60–68

Simheuristics

13

Angel A. Juan, Majsa Ammouriova, Javier Faulin, Javier Panadero, and Daniele Ferone

Contents

Introduction . . . 336
Stochastic Optimization Problems . . . 339
Simheuristic Methodology . . . 341
Simheuristic Extensions . . . 343
Recent Applications in Transportation . . . 345
Recent Applications in Logistics . . . 346
Recent Applications in Production . . . 348
Recent Applications in Health Care . . . 350
Recent Applications in Finance . . . 351
Other Recent Applications . . . 351
Overview of Computational Results . . . 352
Some Open Challenges . . . 354

A. A. Juan (✉)
Universitat Politècnica de València, Valencia, Spain
e-mail: ajuanp@upv.es

M. Ammouriova
German Jordanian University, Amman, Jordan

Universitat Oberta de Catalunya, Barcelona, Spain
e-mail: maysa.ammouri@gju.edu.jo

J. Faulin
Public University of Navarra, Pamplona, Spain
e-mail: javier.faulin@unavarra.es

J. Panadero
Universitat Autònoma de Barcelona, Barcelona, Spain
e-mail: javier.panadero@uab.cat

D. Ferone
Department of Mathematics and Applications "R.Caccioppoli", University of Napoli FEDERICO II, Naples, Italy
e-mail: daniele.ferone@unina.it

© Springer Nature Switzerland AG 2025

R. Martí et al. (eds.), *Handbook of Heuristics*,
https://doi.org/10.1007/978-3-032-00385-0_69

Conclusions 356
Cross-References 356
References 357

Abstract

Uncertainty affects many industries and sectors, including transportation and logistics, health care, production, smart cities, and finance. Simulation models are frequently employed to analyze complex systems under stochastic uncertainty. Many real-world systems are large scale and complex, requiring metaheuristic algorithms to generate high-quality solutions in reasonable computing times. However, metaheuristics are usually designed to solve optimization problems under deterministic scenarios. This chapter describes simheuristics, which merge simulation with metaheuristics to effectively tackle stochastic and complex optimization problems. It discusses the relevance of simheuristics today, reviews related research, outlines important methodological and computational aspects, and provides practical examples from various industries. Additionally, the chapter identifies open research areas and future trends in this emerging field.

Keywords

Simheuristics · Combinatorial optimization problems · Stochastic optimization problems · Metaheuristics · Simulation

Introduction

Since uncertainty is present in most real-world systems, simulation methods are often employed to analyze complex systems in sectors like manufacturing, production logistics, transportation, health care, finance, smart cities, telecommunications, etc. [14]. Typically, this uncertainty is modeled using either theoretical or empirical probability distributions. A logical model of the system is then constructed, and computer simulation is employed to understand system performance across various scenarios, each representing a possible system configuration [57]. As noted by Lucas et al. [50], advancements in computing power and simulation software have made simulation a primary method for analyzing complex systems under uncertainty.

However, simulation alone cannot optimize complex systems as it lacks mechanisms to efficiently explore the vast solution spaces found in combinatorial optimization problems. To solve stochastic optimization problems, simulation must be combined with an optimization engine [67]. Examples include finding the best distribution plan in a vehicle routing problem (VRP) with random demands, the optimal scheduling plan in a flow-shop problem with random processing times, or the best configuration of assets in a rich portfolio optimization problem with real-life constraints and random returns. These optimization problems are often NP-hard, limiting the effectiveness of exact solvers to small- and medium-sized instances.

Since real-life instances are usually large, metaheuristic algorithms are required as they can produce high-quality solutions in reasonable computing times, even for large instances, making them a preferred approach for real-world optimization problems [51].

A well-designed combination of simulation methods and metaheuristic algorithms can effectively solve many real-life optimization problems under uncertainty. Therefore, different authors have proposed integrating simulation and optimization methods to address such issues [19]. Simulation-optimization approaches incorporate various optimization methods, such as mathematical programming, metaheuristics, and reinforcement learning. Additionally, statistical and machine learning methods can be used to create surrogate models based on simulation output [4]. These models establish analytical relations among system variables and can provide quick estimates of simulation outputs. Figueira and Almada-Lobo [20] classified simulation-optimization approaches by their use of simulation, which can be: *(i)* evaluate an objective function or constraint in a stochastic optimization problem, *(ii)* generate solutions for an optimization problem, or *(iii)* enhance an analytical model. Comprehensive reviews and tutorials on simulation-optimization approaches are available in [3] and [19].

In recent years, the hybridization of simulation with metaheuristics, also known as simheuristics, has become a common approach for addressing stochastic optimization problems [41]. Early discussions on combining these methodologies can be found in [24, 25] and [6]. Simheuristic algorithms are designed to solve large-scale, NP-hard combinatorial optimization problems with stochastic elements, which can include stochastic objective functions or probabilistic constraints [13].

Figure 1 provides a visual comparison of optimization approaches and simulation, evaluating their performance (good, very good, or excellent) across four dimensions: *(i)* ability to generate optimal solutions (optimality); *(ii)* ability to model uncertainty (uncertainty); *(iii)* computing time required to deliver results (computing time); and *(iv)* capability to handle large-size instances (scalability). By combining the strengths of metaheuristics and simulation, simheuristics are able to enhance the performance of traditional metaheuristics in the "uncertainty" dimension. Additionally, they enhance the capabilities of pure simulation in the "optimization" dimension and can also outperform exact methods in several dimensions.

The term "simheuristics" first appeared in [39], although the underlying concept was already introduced in [38]. Moreover, the first description in detail of the concept "simheuristics" was depicted in [40]. Since then, the number of articles published in Scopus-indexed journals has grown significantly, from two articles in 2014 to around 21 in 2024, as displayed in Fig 2 (note that the value for 2024 is a forecasted estimate based on current data).

Figure 3 shows Scopus-indexed journals that have published at least two articles on simheuristics. Hence, "Journal of Simulation" includes eight articles on simheuristics, while "Simulation Modelling Practice and Theory" and "International Transactions in Operational Research" include seven each. Also, journals such as "Mathematics," "Algorithms," "SORT," and "Annals of Operations Research" have published five or more articles on simheuristics each. According to the Scopus database, articles on simheuristics have been published in up to 48 different journals.

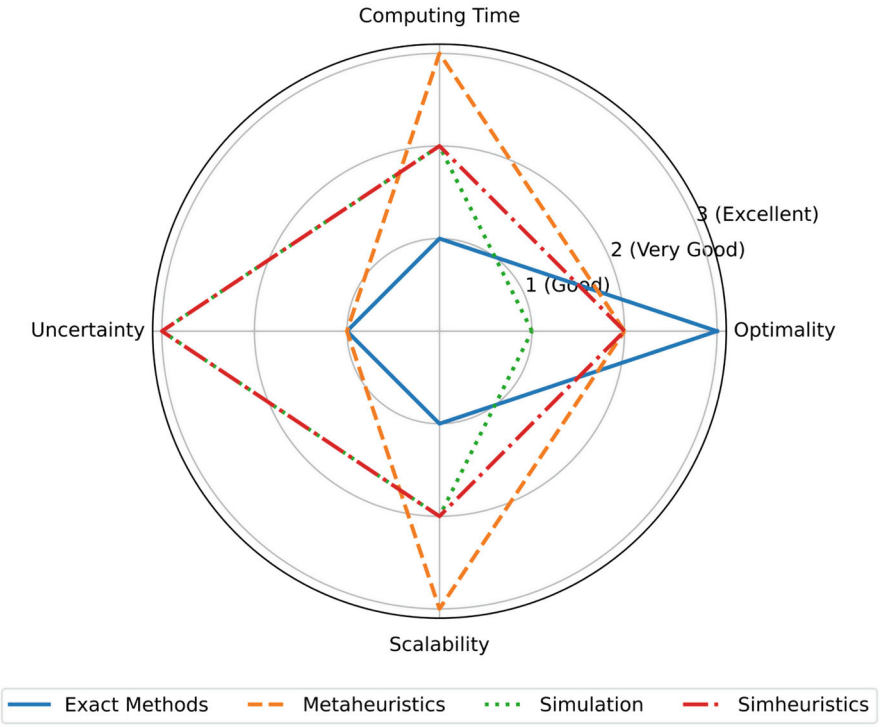


Fig. 1 Comparison of different optimization and simulation approaches across several dimensions

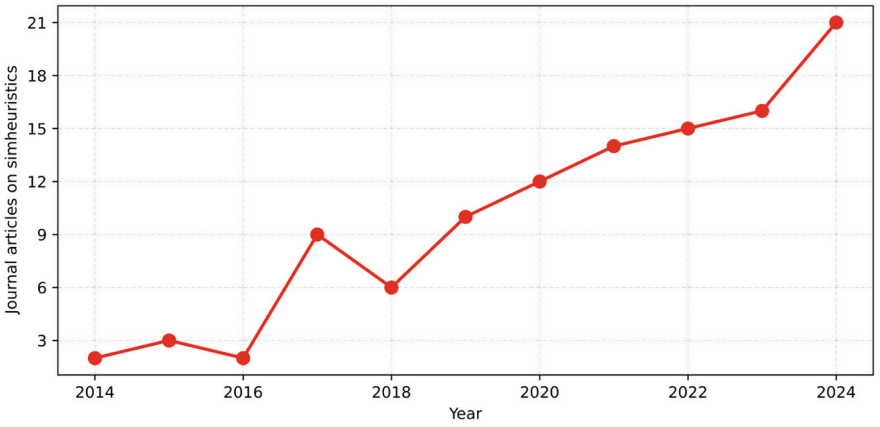


Fig. 2 Journal articles on simheuristics for the period 2014–2024

This chapter introduces the core concepts of simheuristics, along with recent applications and current research directions. The rest of the chapter is organized as follows: section "Stochastic Optimization Problems" describes the stochastic optimization problems that can be addressed with simheuristics.

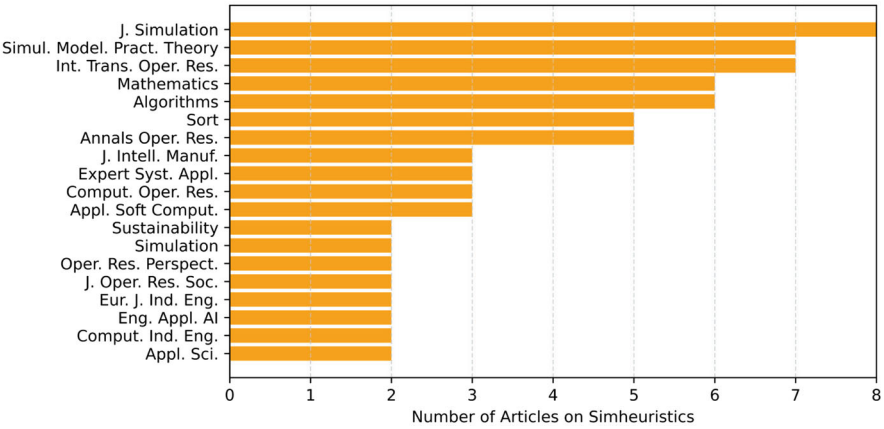


Fig. 3 Number of simheuristic articles by Scopus-indexed journal

Section "Simheuristic Methodology" explains the fundamental concepts of the methodology, while section "Simheuristic Extensions" discusses how simheuristics can be extended in order to incorporate fuzzy uncertainty as well as dynamic elements. Sections "Recent Applications in Transportation, Recent Applications in Logistics, Recent Applications in Production, Recent Applications in Health Care, Recent Applications in Finance, and Other Recent Applications" review recent simheuristic applications to fields such as transportation, logistics, production, health care, finance, and others, respectively. Section "Overview of Computational Results" provides an overview of the computational results that have made simheuristics an attractive approach methodology. Section "Some Open Challenges" highlights some trends and open challenges to be considered. Finally, section "Conclusions" summarizes the contents of the chapter.

Stochastic Optimization Problems

Reviewing the scientific literature on optimization reveals that many combinatorial optimization problems in industries such as transportation and logistics, production and manufacturing, energy, finance, etc. are still often formulated using deterministic models. However, deterministic models do not take into account the uncertainty and variability inherent in real-life systems, such as random travel and processing times, random customer demands, or random investment returns. At least in part, this oversimplification is due to the added complexity that stochastic components introduce into the problem modeling and solving processes. Many of these problems are NP-hard, making even their deterministic versions challenging and time-consuming to solve, particularly for realistic, large-scale instances. Consequently, numerous studies focus on finding an optimal solution to a deterministic version of the problem rather than a near-optimal solution to a more realistic version including stochastic elements. This approach results in an optimal solution for a model

that does not accurately represent the real-world system. When this deterministic optimal solution is applied to the actual system under uncertain conditions, it often turns out to be suboptimal. This can be easily demonstrated by running the deterministic optimal solution through a simulation model of the system.

Simheuristic algorithms aim to resolve the aforementioned discrepancy by using stochastic optimization models that more accurately reflect real-world operating conditions. These algorithms then try to find near-optimal solutions for these models. Typically, the stochastic optimization models used have the following form:

$$\text{optimize } f(s) = \Phi(s, X) \tag{1}$$

subject to:

$$Pr\Big(g_i(s, X) \leq l_i(X)\Big) \leq t_i \qquad \forall i \in I \tag{2}$$

$$Pr\Big(g_j(s, X) > l_j(X)\Big) \leq t_j \qquad \forall j \in J \tag{3}$$

$$g_k(s) \leq t_k \qquad \forall k \in K \tag{4}$$

$$s \in S \tag{5}$$

where

- As represented in Eq. (5), s denotes a potential solution (a configuration of decision variables) to the optimization problem, with S representing the space of all possible solutions.
- The term $\Phi(s, X)$ in Eq. (1) stands for a stochastic function that needs to be optimized (either minimized or maximized). Here, X is a vector of random variables, each following a specific probability distribution with a known mean. Note that $\Phi(s, X)$ does not have to be an expected value. On the contrary, it can be any statistic (standard deviation, median, percentile, etc.), or even a combination of them.
- Equations (2) and (3) describe probabilistic constraints related to the problem. For instance, the probability that the number of serviced customers in solution s, $g_i(s, X)$, falls below a certain lower bound $l_i(X)$ is less than a user-defined threshold t_i. Alternatively, the probability that the time delay in solution s, $g_j(s, X)$, exceeds a certain upper bound $l_j(X)$ is constrained by a user-defined threshold t_j.
- Lastly, Eq (4) represents the standard deterministic constraints found in combinatorial optimization problems.

There are other promising methods for addressing stochastic optimization problems, such as probabilistic programming [83], robust optimization [30], and stochastic programming [34]. However, as highlighted by Onggo et al. [60], it is important to notice that any given solution $s \in S$ exhibits probabilistic behavior when solving

stochastic optimization problems. Therefore, knowing as much as possible on the distribution of s could be valuable when making informed decisions. For example, in a stochastic VRP, while the primary objective may be to minimize the expected cost of a routing plan, other statistics such as the variance of s or its reliability (i.e., the probability of completing the route without failures) should also be evaluated. Similarly, in a stochastic portfolio optimization problem, minimizing risk may require understanding the probabilistic behavior of the random returns associated with a given solution s (portfolio configuration). In essence, optimization under uncertainty is more complex than optimization under deterministic conditions. Hence, experts should not only provide a solution s that optimizes an expected value but also offer insights into the statistical and probabilistic behavior of multiple alternative solutions. This is where simheuristic algorithms excel. They are effective in searching large solution spaces due to their metaheuristic component and can enhance the analysis of solutions with the statistical and probabilistic insights provided by their simulation component.

Simheuristic Methodology

When dealing with a deterministic optimization problem, it is reasonable to assume that its optimal and near-optimal solutions will likely perform with high quality even when moderate uncertainty is introduced into the problem. For example, in a VRP, routing plans with near-optimal performance under deterministic conditions are expected to maintain a high-quality performance even if some travel times have small variances. Similarly, in a deterministic portfolio optimization problem, low-risk asset configurations that guarantee a return of r_0 are likely to remain low risk under random returns with moderate variance, potentially achieving expected returns that exceed $r_0 - \epsilon$ with some probability, where $\epsilon > 0$.

For a given solution configuration s to a deterministic optimization problem with objective function $f(s)$, it is possible to consider its value under deterministic conditions, $determ(s) = f(s)$, as well as its expected value under stochastic conditions, $stoch(s) = E[f(s)]$. Under moderate uncertainty, it makes sense to assume a strong positive correlation between $determ(s)$ and $stoch(s)$, meaning that high-quality solutions for the deterministic version of the problem are likely to perform as high-quality solutions (on the average) for the stochastic version of the problem. However, the existence of such correlation does not necessarily imply that the best solution for the deterministic version of the problem will also be the best solution for the stochastic version. Hence, for example, solution s_1 could outperform solution s_2 in the deterministic version of the problem, while s_2 could perform better than s_1 in the stochastic version. In the VRP example, this could be the case if s_2 offers a more balanced distribution of routes than s_1, which allows it for supporting moderate variances in travel times or customer demands without suffering excessive delays or route failures.

Figure 4 illustrates the underlying logic of the simheuristic approach. It starts by considering the deterministic version of a stochastic optimization problem, which is achieved by substituting all random variables with their expected values, thus

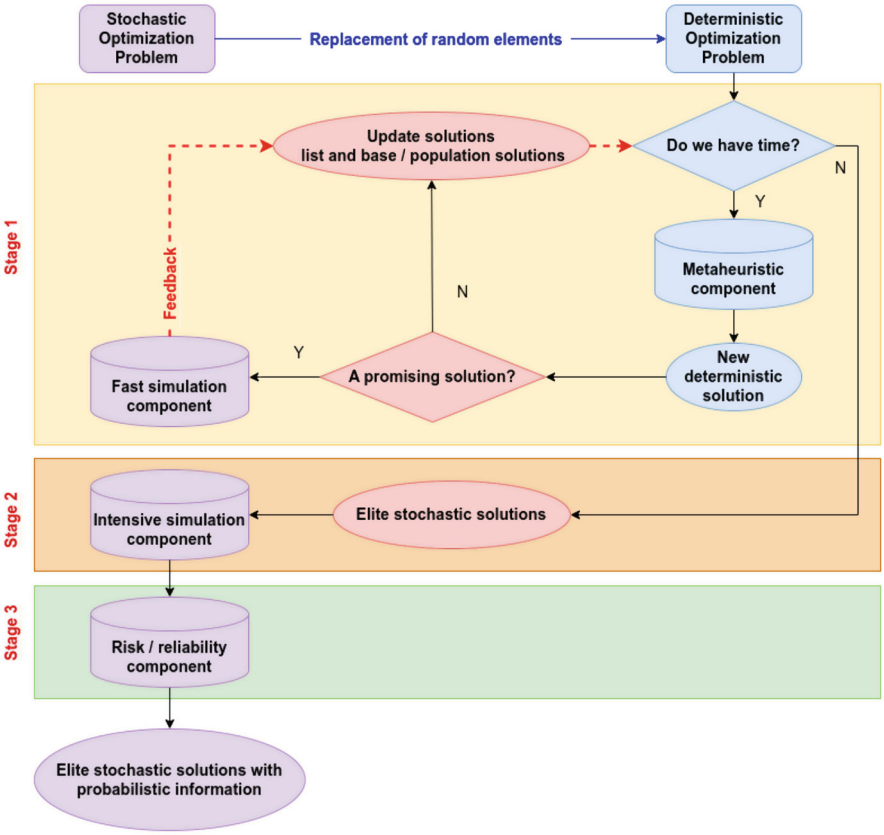


Fig. 4 The logic behind simheuristics

simplifying the problem by removing uncertainty. A metaheuristics is then applied to effectively explore the solution space, generating progressively better solutions for the deterministic problem. When the metaheuristics identifies a promising solution—one that is likely to perform well under uncertainty—it is evaluated through a simulation to determine its performance in the stochastic context. The simulation provides estimates of key statistics like the mean, variance, and percentiles, as well as probabilistic insights useful for risk or reliability analysis, along with data on various system variables. Given that simulations can be time-consuming, it is often advisable to limit the number of runs per evaluation to avoid slowing down the metaheuristics' convergence to near-optimal solutions. Suggestions for speeding up these computations are available in [70].

As explained in [13], the simulation feedback can also be employed to better guide the searching process carried out by the metaheuristics. This feedback can even be further processed by a machine learning component in order to better

identify promising solutions or to create a surrogate model of the simulation component that saves computational time. At the end of this first stage, a shortlist of elite solutions is generated. Each of these solutions is expected to perform well under uncertainty based on initial estimates. To improve the precision of these estimates, a more detailed simulation is run for each elite solution. Variance reduction techniques can be applied to speed up this process, with simulations potentially running in parallel on multiple processors. Finally, the results from these simulations can be used to conduct a risk or reliability analysis on the elite solutions, providing valuable probabilistic insights that enhance the decision-making process by detailing the stochastic behavior of each elite solution. This is one of the main strengths of simheuristics compared to other stochastic optimization techniques. Two additional key points should be noticed:

- As uncertainty increases, the correlation between deterministic and stochastic values is expected to weaken. In cases of high uncertainty (e.g., very large variances in random variables), it might be more effective to seek robust or reliable solutions rather than optimizing the expected value, as the latter could result in solutions with high variability.
- In optimization problems, the objective function's optimal value typically worsens as uncertainty increases, especially when the objective function is asymmetrical—where a favorable outcome from random variables does not offset the negative impact of an unfavorable outcome. If s^* is an optimal or near-optimal solution for the deterministic problem, and s^{**} is an optimal or near-optimal solution for the stochastic problem, the following inequalities should generally hold: *(i)* in a minimization problem, $determ(s^*) \leq stoch(s^{**}) \leq stoch(s^*)$; and *(ii)* in a maximization problem, $stoch(s^*) \leq stoch(s^{**}) \leq determ(s^*)$. In essence, the optimal or near-optimal solution of the deterministic version of a problem can be used to set lower and upper bounds for the near-optimal value of the stochastic problem.

Simheuristic Extensions

This section analyzes two emerging extensions of the simheuristic concept: fuzzy simheuristics and sim-learnheurisitics. As previously discussed, in many real-life applications, inputs may be subject to uncertainties, such as variations in travel times, customer demands, service times, or battery life for electric vehicles. When these inputs are modeled as random variables, simheuristic algorithms can be employed to tackle the resulting stochastic optimization problems. However, stochastic uncertainty is not the only type encountered in real-world problems. For non-probabilistic uncertainty, fuzzy techniques can be applied [12]. This type of uncertainty often arises from insufficient or scarce data, making it challenging to model with traditional probability distributions. It can also occur in scenarios requiring qualitative expert judgments. To tackle both types of uncertainty, [82] propose

a fuzzy simheuristic method, which integrates metaheuristics with simulation and fuzzy logic. This approach allows some inputs to be modeled as random variables while others follow fuzzy patterns. A fuzzy logic system computes outputs based on rules established by experts, accommodating scenarios where outputs have degrees of truth rather than binary values. Thus, for instance, [11] examine the permutation flow-shop problem incorporating both stochastic and fuzzy processing times. The primary objective is to minimize the expected makespan of job permutations while also considering additional solution characteristics due to uncertainty. The paper demonstrates how survival analysis can enhance the probabilistic information available to decision-makers. The study extends the simheuristic framework to accommodate fuzzy elements, integrating both stochastic and fuzzy uncertainties in the problem. Classical instances were modified to include either stochastic or fuzzy processing times for testing purposes.

Real-life applications frequently involve not just randomness but also dynamic elements that evolve over time or change according to some environmental conditions. For instance, travel times may vary with traffic conditions or weather, customer demands might shift based on environmental factors or the order in which each customer is delivered, processing times might vary according to the level of fatigue of the machine, and resource availability can fluctuate due to various real-time influences. In such cases, simheuristics alone may not suffice, because they are designed primarily to handle uncertainty rather than dynamic changes. To take into account these dynamic aspects, machine learning predictive models and in particular learnheuristics [9] can be highly beneficial. Learnheuristics combine machine learning models with metaheuristics in order to properly tackle optimization problem with dynamic inputs that need to be predicted as the environmental conditions vary. Integrating learnheuristics with simheuristics into what is called sim-learnheuristics can significantly enhance the capabilities of simheuristics to deal with dynamic elements. The machine learning component provides more accurate predictions of dynamic inputs, which are then used to inform the simheuristic algorithm. This combination allows for better anticipation of how these variables will evolve, leading to more robust and adaptive solutions. Peyman et al. [68] present a sim-learnheuristic approach tailored for the team orienteering problem, specifically applied to unmanned aerial vehicles. Unlike previous studies that focus on deterministic or stochastic versions of the problem, their method explores a more complex scenario that includes deterministic, stochastic, and dynamic elements. The approach becomes particularly intricate due to the uncertain travel times, which are influenced by dynamic factors like weather and traffic. For solving this problem, the authors propose a hybrid solution that combines a heuristic algorithm with Monte Carlo simulation and a multiple regression model. Similarly, [21] study a stochastic and dynamic variant of the capacitated dispersion problem. They propose a sim-learnheuristic algorithm that combines a biased-randomized metaheuristics for optimization, simulation to handle uncertainty, and machine learning to tackle dynamic behavior.

Recent Applications in Transportation

The following sections review journal articles on applications of simheuristics to different fields that have been published in 2020 or later. Previous articles, as well as conference articles, can be found in [13] and [10]. Transportation is one of the fields where simheuristics have been extensively employed. Panadero et al. [63] tackle the problem of routing unmanned aerial vehicles for surveillance, where the value of information varies across target locations, and not all locations need to be visited. The authors develop a simheuristic algorithm starting with a fast, original heuristic. This heuristic is then enhanced into a variable neighborhood search metaheuristics. Finally, simulation is integrated into the framework. Keenan et al. [43] tackle the stochastic time-capacitated arc routing problem, where the actual demand on each edge is a random variable, affecting service times as well. To solve this problem, the authors propose a strategic oscillation simheuristic algorithm. Latorre-Biel et al. [47] address a stochastic version of the vehicle routing problem, where customer demands are both uncertain and correlated. The authors propose a simheuristic approach combined with an adaptive demand predictor that employs Petri nets. Yazdani et al. [87] explore the challenges of managing construction and demolition waste. A hybrid genetic algorithm is used to optimize vehicle routing for transporting waste from construction sites to recycling facilities. The proposed method uses simulation to account for uncertainties in routing. Panadero et al. [61] explore both deterministic and stochastic versions of a team orienteering problem. The paper introduces a scenario where rewards depend on the order of visits, rewarding early customers and penalizing later ones, with travel times treated as random variables. Herrera et al. [36] explore a stochastic version of the team orienteering problem where random delays are considered. The goal is to find solutions that not only maximize the expected reward but also ensure high reliability, meaning there is a low probability of encountering delays beyond a specified threshold. Ramirez-Villamil et al. [74] focus on the two-echelon vehicle routing problem with stochastic travel times by proposing a simheuristic approach. Using real data from a French delivery company, their model also accounts for CO_2 and fine particle emissions. Ghorpade and Corlu [22] tackle the one-commodity pickup and delivery traveling salesman problem, which involves transporting a single type of good from supply locations to demand points while minimizing transportation costs. To address this, the paper introduces a new simheuristic algorithm, which combines a variable neighborhood descent metaheuristics with Monte Carlo simulation. Kızıloğlu and Sakallı [44] present a mathematical model that simultaneously optimizes flight planning, fleet assignment, and aircraft routing, while incorporating codesharing. While optimization software can handle smaller-scale problems effectively, it struggles with larger problems and multiple stochastic scenarios due to computational limitations. To address this, the study introduces two simheuristic algorithms. Rabet et al. [73] analyzes the integration of production, maintenance, and delivery operations within supply chains. Using a fertilizer producer case study with seasonal demand and outdated management practices,

the study proposes a mathematical model and two multi-objective simheuristics. Panadero et al. [64] discuss a team orienteering problem where travel times are uncertain and modeled as random variables, introducing reliability issues due to route-length constraints. The performance of their simheuristic is compared using the sample average approximation (SAA) method and a hybrid model that merges the simheuristic with SAA. Peyman et al. [68] present a new sim-learnheuristic method for solving the team orienteering problem. Their approach tackles a hybrid scenario incorporating deterministic, stochastic, and dynamic elements. Hence, travel times might be both uncertain and dynamically affected by factors like weather and traffic. Shone et al. [81] consider a runway scheduling problem where aircraft landings on a single runway are influenced by stochastic and dynamic factors. Due to the intractability of exact methods for this multistage optimization problem, the authors propose a simheuristic approach.

Recent Applications in Logistics

Similar to transportation, logistics is another natural application field for simheuristics. In effect, distribution of goods challenge logistics companies in urban areas, which defines problems related to last-mile delivery. Quite often, distribution problems are NP-hard and demand advanced optimization methods to be utilized. In real-life scenarios, problem parameters encounter uncertainty. Thus, various researchers have used simheuristics in solving such problems.

Gruler et al. [31] introduce a simheuristic approach for solving the multi-period inventory routing problem (IRP) with stochastic customer demands. The IRP involves both inventory management and delivery route planning, aiming to optimize refill policies for each customer over multiple periods to minimize the total expected costs, including both inventory and routing costs. Yazdani et al. [86] examine the challenge of evacuating populations in urban areas in response to extreme weather events, which are becoming more frequent and intense. The research proposes a hybrid simulation-optimization approach aimed at maximizing the number of evacuees transported from disaster zones to safe locations. Martins et al. [53] focus on the omnichannel delivery concept for the retailing industry, which addresses the replenishment of a set of retail stores and the direct shipment of the products to customers within an integrated vehicle routing formulation. Raba et al. [72] focus on optimizing a real-life supply chain for delivering animal feed to livestock farms. They consider a computer-aided system that manages orders, which can include various types of feed, from multiple depots using a fleet of diverse vehicles. The proposed method integrates biased-randomization techniques with a simheuristic framework, utilizing sensor data for optimization. Quintero-Araujo et al. [69] analyze the capacitated location routing problem with stochastic demands, where the exact demands are unknown until reaching each customer. The proposed iterated local search simheuristics aims to minimize the expected costs associated with facility operations, vehicle usage, routing, and corrective actions needed due to route failures from unexpected high demands. Martínez-Reyes et al. [54]

present a simheuristic approach to optimize the distribution of personal protective equipment (PPE) to intensive care units amid the COVID-19 pandemic. Examining the critical need for PPE, the approach uses the location-routing problem framework to consider uncertain demand, distribution costs, and solution reliability. de León et al. [16] addresses berth allocation at ports by proposing a proactive simulation-optimization approach using real-time data and a multi-objective method with a non-dominated sorting genetic algorithm. Rabe et al. [71] tackle the complexity of distribution in business-to-business materials trading, where decision-makers must balance conflicting goals like high service levels and low costs. It proposes two main strategies to enhance simheuristic processes. Firstly, it explores using reinforcement learning to reduce response times by learning from idle periods between decisions. Secondly, it incorporates domain-specific information to support metaheuristic algorithms, which helps in defining equivalent solutions and reducing the number of required simulation runs. Kabadayi and Dehghanimohammadabadi [42] explore the challenge of supplier selection in supply chain management by developing a decision support system (DSS) that integrates multi-criteria decision-making (MCDM) with simulation and optimization. The system includes an MCDM module to identify key criteria, a simulation model to assess supply chain performance with these criteria, and a multi-objective metaheuristic algorithm to optimize the criteria combinations for improved supply chain performance. Vieira et al. [84] study the challenge of sustainable supply chain network design by integrating risk management into the design process. It introduces a simulation-based decision support system applied to a Portuguese wine supply chain case study. Leon et al. [48] tackle the challenge of optimizing storage location assignments in warehouses, which is crucial for supply chain efficiency. Traditional methods often overlook the stochastic variability and interactions between warehouse activities. The study introduces a discrete-event simheuristic framework that integrates order sequence and picking routes to create robust solutions under real-life conditions. Gök et al. [23] present a novel simulation-optimization approach for scheduling aircraft turnarounds at airports, addressing the complexities and coordination challenges among various stakeholders such as airport operators, airlines, and ground service providers. The proposed method uses simheuristic techniques combined with an efficient search mechanism to enhance planning robustness. Bayliss and Panadero [7] examine the challenge of efficiently deploying temporary medical facilities during epidemic outbreaks, aiming to minimize costs, completion time, population travel time, and waiting time. It introduces a learnheuristic algorithm that integrates machine learning with a queuing model to evaluate facility location and population assignment decisions. The algorithm is trained on simulation data to rapidly assess facility performance and waiting times. Gomez et al. [27] focus on the capacitated dispersion problem with stochastic capacities, which aims to select facilities to maximize the minimum distance between any pair while meeting service levels under capacity uncertainties. The study introduces two models: one using probabilistic constraints to ensure service levels with a certain probability and the other incorporating soft constraints with penalty costs for capacity violations. A forward–backward simheuristic algorithm is proposed to solve these models

efficiently. Peidro et al. [66] introduce a simheuristic approach for solving the uncapacitated facility location problem with fluctuating service costs. The proposed method combines a tabu search metaheuristics with path-relinking and simulation techniques to consider random service costs. Ramirez-Villamil et al. [75] concentrate on a variant of the two-echelon capacitated vehicle routing problem used for urban logistics, focusing on a heterogeneous fleet with limited capacities and time windows. The study proposes a decomposition algorithm based on mixed integer linear programming for solving the deterministic version and a simheuristics for the stochastic version.

Recent Applications in Production

Similar to the applications of simheuristics in transportation and logistics, simheuristics has been utilized in solving problems related to production. Santos et al. [79] present a decision support system for optimizing the amount of active equipment in an ore crushing circuit to maximize production while accounting for variations in ore feed rate. The system uses a simheuristic approach, incorporating a simulated plant model to evaluate production rates. Lopes et al. [49] tackle the balancing optimization of mixed-model assembly lines under uncertainty, focusing on maximizing throughput in environments with stochastic product sequences and given workstation counts and buffers. The authors introduce a specialized cycle time simulator (CTS) and a novel simheuristic approach that employs CTS to evaluate cycle times and optimize line balancing. Gnanavelbabu et al. [26] investigate the flexible job shop scheduling problem with added complexities of worker flexibility and uncertain processing times. They introduce a simheuristic approach combining Monte Carlo simulation with a multi-objective modified backtracking search algorithm to minimize both the expected makespan and its standard deviation. The stochastic flexible job shop scheduling problem with uncertain operation processing times has been tackled by Caldeira and Gnanavelbabu [8]. The authors propose a simulation-optimization approach, and computational results demonstrate the algorithm's effectiveness across various levels of variability, with performance assessed using reliability-based methods. Villarinho et al. [85] discuss the permutation flow-shop problem with delivery dates and cumulative payoffs, where processing times are stochastic. The goal is to maximize the expected payoff by determining the optimal job sequence. A metaheuristic using a variable neighborhood descent framework is extended into a simheuristics by incorporating Monte Carlo simulation. Gonzalez-Neira et al. [29] present a hybrid simheuristic approach for solving a multicriteria stochastic permutation flow-shop problem with both stochastic processing times and sequence-dependent setup times. The proposed method integrates a greedy randomized adaptive search procedure, Monte Carlo simulation, Pareto archived evolution strategy, and analytic hierarchy process (AHP). The AHP is used to select the optimal sequence from the Pareto frontier that best aligns with decision-makers' objectives. Negri et al. [56] introduce a simheuristic framework designed for robust scheduling in the context of the

flow-shop scheduling problem, also utilizing digital technologies for real-time data analysis. The framework integrates genetic algorithms for schedule optimization with discrete event simulation, and it is enhanced by a digital twin that represents the production system to be scheduled, and it is equipped with an equipment prognostics and health management module that receives real-time data from the physical equipment and elaborates them to feed predictive information to the simulation model DT. Yazdani et al. [87] consider challenges in prefabricated construction, particularly focusing on on-time production of precast components under sequence-dependent due dates. They propose new scheduling methods to minimize total weighted earliness and tardiness. To manage uncertainties in production, the study introduces three simulation-optimization algorithms integrating simulation with metaheuristic frameworks. Castaneda et al. [11] tackle the permutation flow-shop problem under both stochastic and fuzzy uncertainties in processing times. The aim is to minimize the expected makespan while also considering additional solution characteristics due to uncertainty. To tackle this complex problem, the paper extends a simheuristic framework to incorporate both stochastic and fuzzy elements. Seiringer et al. [80] present a simheuristic approach designed to optimize material requirements planning parameters. The proposed method combines simulation annealing with an intelligent simulation budget management concept to efficiently minimize overall costs. This concept improves the efficiency by avoiding unnecessary replications of less promising iterations. Yu et al. [88] tackle the challenge of spare parts management by developing a joint optimization model for multi-item, multi-period inventory management and planned maintenance amid uncertain failures. They contrast independent and aggregate inventory policies and integrate predictive maintenance intervals to address uncertain failures. A novel two-stage genetic algorithm is introduced as a simheuristic approach to manage the model's nonlinearity, combinatorial complexity, and stochastic elements, suitable for large-scale instances. Meeß et al. [55] examine the challenge of designing product-service systems by proposing a simheuristic optimization approach to evaluate and select optimal business model configurations during the design phase. The study introduces a proprietary evolutionary algorithm integrated into a simheuristic framework for testing and evaluating different business models. Abu-Marrul et al. [1] focus on a parallel machine scheduling problem with non-anticipatory family setup times and batching, including stochastic processing times and release dates. They present an iterated greedy metaheuristics combined with Monte Carlo simulation to handle stochastic parameters. The study compares this simheuristic approach with a traditional iterated greedy metaheuristics. Grumbach et al. [32] introduce a simheuristic for solving dual resource constrained flexible job shop scheduling problems using a combination of memetic algorithms and deep reinforcement learning. Nessari et al. [58] present a hybrid algorithm for the flexible job shop scheduling problem, focusing on minimizing makespan and total weighted earliness and tardiness in the face of uncertainty. The algorithm integrates the equilibrium optimizer for initial solutions, the non-dominated sorting genetic algorithm for multi-objective optimization, and simulation techniques to handle stochastic variables. Ruiz-Rodríguez et al. [78] examine the effectiveness

of various maintenance scheduling techniques under different levels of uncertainty, such as unexpected machine failures and variable repair times. The study finds that a genetic algorithm simheuristic generally outperforms reinforcement learning and dispatching rules in total machine uptime, especially with high re-optimization frequencies. Zambrano-Rey et al. [89] tackle a stochastic job shop scheduling problem with sequence-dependent setup times, focusing on minimizing the expected maximum lateness. The stochastic elements are modeled through uncertain times between failures and times to repair. The proposed solution is a simheuristic based on tabu search. Rodríguez-Espinosa et al. [77] concentrate on a bi-objective flexible job shop scheduling problem with stochastic processing times. They aim to minimize both deterministic earliness plus tardiness and earliness plus tardiness risk, the latter focusing on robustness in uncertain environments. The proposed solution is a simheuristic combining the non-dominated sorting genetic algorithm with Monte Carlo simulation to generate the Pareto frontier for both objectives.

Recent Applications in Health Care

Recent applications of simheuristics can also be found in health care. Dehghanimohammadabadi and Kabadayi [17] introduce a combined simulation-optimization and multi-criteria decision-making approach to assess the impact of quality improvement initiatives in health care. It uses the analytic hierarchy process to determine the importance of various healthcare criteria from health providers. These weights are then used in a simulation-optimization framework to identify the most effective actions for enhancing quality of care. Datta et al. [15] aim to improve outpatient care delivery by proposing a multi-objective simulation-based decision support model that balances cost and patient dissatisfaction, with the latter addressing service fairness. The model, developed from field data, uses a queuing theory-based discrete event simulation and employs genetic algorithms for optimization. Pandhare et al. [65] introduce a prognostics and health management-enabled digital twin framework designed for robust production scheduling in flow-shop environments. This framework integrates genetic algorithm optimization with data-driven modeling techniques and discrete event simulation to address real-time equipment health. Dehghanimohammadabadi et al. [18] introduce the multi-objective patient appointment scheduling framework to enhance clinic operations and patient satisfaction. The framework integrates multi-objective particle swarm optimization, data exchange, and simulation. It aims to minimize total service time and maximize patient admissions without overtime. Hirpara et al. [37] studied on-scene injury assessment of trauma patients and their allocation to right hospitals by the emergency medical service providers. They utilized Monte Carlo simulation integrated in a genetic algorithm to solve multi-transportation capacitated facility location problem.

Recent Applications in Finance

Regarding recent applications of simheuristics in finance, [62] study the challenge of selecting the best portfolio of projects under limited financial resources, a problem that becomes NP-hard as constraints increase. The authors analyze a stochastic version of the project portfolio selection problem that considers uncertainties in cash flows and discount rates, aiming to maximize the expected net present value while adhering to risk constraints. Kizys et al. [45] explore the challenge of portfolio optimization, specifically minimizing risk for a given expected return, which becomes NP-hard as the number of assets and constraints increase. Traditional metaheuristics often fail to account for the uncertainty in financial markets, such as random returns and noisy covariances. To address this, the authors propose a simheuristic algorithm that combines a variable neighborhood search metaheuristics with Monte Carlo simulation to handle stochastic returns and noisy covariances. Nieto et al. [59] tackle the challenge of efficiently managing assets to cover a firm's liabilities over multiple periods, a task that is particularly complex under uncertainty. The authors propose a simheuristic approach that combines Monte Carlo simulation with a genetic algorithm to maximize expected terminal wealth. This approach is designed to produce effective solutions quickly and includes "smoothing" techniques to make the solutions more practical for real-world application.

Other Recent Applications

Finally, recent applications to other fields are reviewed in this section. Antoniadis et al. [5] focus on enhancing the self-healing capabilities of smart grids by addressing disturbances, such as potential overloads, and ensuring stability for a future planning horizon. The authors propose a simulation-based approach for evaluating the robustness of grid topology reconfigurations after disturbances. Lai et al. [46] explore the application of simheuristics to optimize the operating rule curve of the Klang Gate Dam (Malaysia) in response to future climate change impacts. The research involves selecting a set of general circulation models based on location and data availability, which is then downscaled and analyzed using artificial neural networks and support vector regression. Alvarez Fernandez et al. [2] examine the optimization of a hub-and-spoke telecommunication network where numerous nodes exchange real-time multimedia data, including audio, video, and images. In this scenario, the amount of data exchanged between nodes is modeled as a random variable. The objective is to identify the optimal locations for a fixed number of hubs to minimize the total expected transmission cost, considering the uncapacitated single-allocation p-hub median problem under uncertainty.

Overview of Computational Results

This section provides an overview of previously published results in the literature. These studies have been selected to demonstrate the efficiency of simheuristics in addressing well-known NP-hard and large-scale stochastic optimization problems across different fields of application. Table 1 summarizes the average results obtained for the selected optimization problems. Eleven different problems were chosen to represent a broad spectrum of real-life optimization problems. The first column lists the references from which the results were sourced, while the second column describes the specific type of problem addressed. The subsequent column

Table 1 Summary of different stochastic optimization problems

Publication	Problem	Type of problem	OBD	OBD-S	OBS
Panadero et al. [62]	Portfolio Optimisation Problem (SPOP-1)	Max.	3377.73	3250.20	3314.62
Kizys et al. [45]	Portfolio Optimisation Problem (SPOP-2)	Min.	0.0068630891	0.002396885	0.002720555
Panadero et al. [63]	Team Orienteering Problem (STOP)	Max.	528.28	359.11	468.80
Guimarans et al. [33]	2L Vehicle Routing Problem (2L-VRPST)	Min.	1549.28	1874.68	1825.65
González-Martín et al. [28]	Arc Routing Problem (ARPSD)	Min.	5412.75	6223.00	5669.25
Quintero-Araujo et al. [69]	Capacited Location and Routing Problem (CLRPSD)	Min.	98,587.08	111,545.99	111,246.35
Hatami et al. [35]	Distributed Parallel Flowshop Problem (DPFSP)	Min.	4822.96	4964.81	4944.81
Reyes-Rubiano et al. [76]	Electric Vehicle Routing Problem (EVRPST)	Min.	16,490.13	19,995.73	19,339.89
Peidro et al. [66]	Uncapacited Facility Location Problem (UFLP)	Min.	1,200,551.11	1,343,897.33	1,240,039.04
Gomez et al. [27]	Capacited Dispersion Problem (SCDP)	MaxMin	71.86	47.25	61.64
Panadero et al. [61]	Team Orienteering Problem with Position-Dependent rewards (STOP-PDR)	Max	189,640.74	212,649.83	197,589.52

specifies whether the objective function involves maximization or minimization. The following three columns present the results achieved under various scenarios. The OBD column displays the best solution found without accounting for stochasticity, representing the objective function value in a deterministic scenario, which serves as a lower bound reference in a perfect information scenario. The next column, OBD-S, indicates the expected cost associated with the OBD solution when it is assessed in a stochastic scenario, incorporating the associated level of uncertainty—a simulation is used to estimate the expected cost of the OBD solution under a stochastic scenario. Finally, the OBS column shows the expected cost achieved using a simheuristic approach for the stochastic version of the problem.

Figure 5 presents a graphical representation of the results from Table 1, where the vertical axis shows the percentage gap between OBD-S and OBS with respect to OBD (baseline at 0% gap). The results show that, for all the problems considered, the solutions provided by the simheuristic approach (OBS) outperform those of the deterministic version when the latter is applied under uncertainty conditions (OBD-S). This indicates that optimal or near-optimal solutions for deterministic versions of optimization problems will typically become suboptimal solutions when applied in stochastic scenarios. For instance, the solutions obtained for the arc routing problem with stochastic demands (ARPSD) perform near-optimally under deterministic conditions but become suboptimal in stochastic scenarios, exhibiting a large gap with respect to the OBS provided by the simheuristic. In this case,

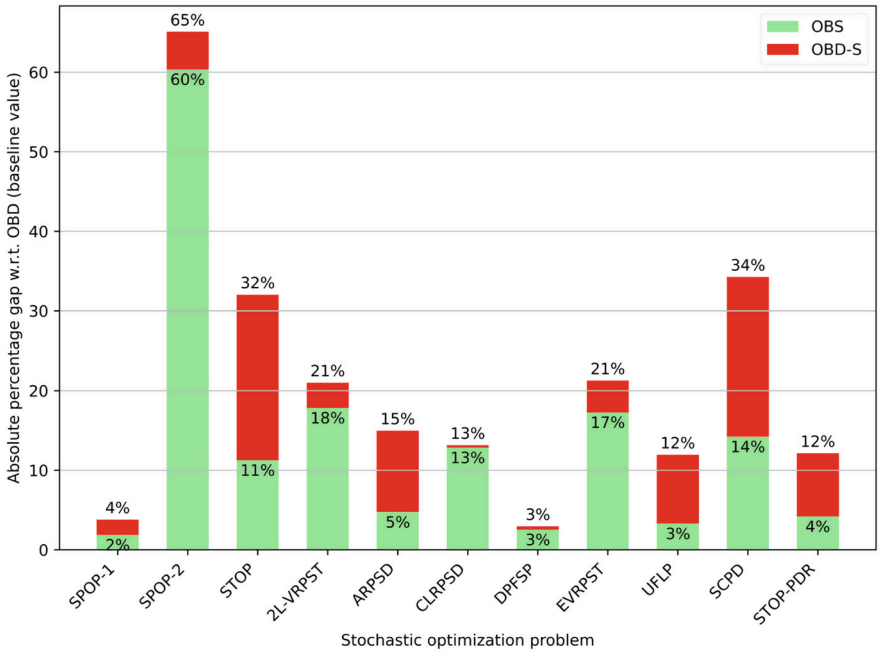


Fig. 5 Gaps of OBD-S and OBS solutions with respect to OBD solution (baseline at 0% gap)

Table 2 Summary of different optimization problems with fuzzy and dynamic components

Publication	Problem	Type of problem	Framework	OBD	OBD-H	OBH
Martin et al. [52]	Time Capacitated Arc Routing Problem (TCARP)	Min.	Fuzzy Sim.	144.80	176.00	169.70
Castaneda et al. [11]	Permutation Flow Shop Problem (PFSP)	Min.	Fuzzy Sim.	6317.67	7400.07	7325.24
Peyman et al. [68]	Team Orienteering Problem (TOP)	Max.	Learn Sim.	352.47	209.73	233.10
Ghorbani et al. [21]	Capacited Dispersion Problem (CDP)	MaxMin.	Learn Sim.	71.50	31.90	61.70

using the OBD-S solution could lead to inefficient decisions, resulting in additional costs. This gap can increase depending on the level of uncertainty in the problem's components, as observed in the SCPD problem, where the gap between the OBS and the OBD-S is quite large due to the high level of uncertainty. Conversely, when the level of uncertainty is low and has less influence on the elements of the problem, the OBD-S solution can still be competitive, providing results close to those of the OBS and even to those of the OBD. An example is the distributed parallel flow-shop problem (DPFSP), where, although the OBS solution outperforms the OBD-S, they are both quite close to the OBD.

Table 2 reports the obtained results for a set of problems solved using extensions of simheuristics, either fuzzy or sim-learnheuristics. Specifically, the first two problems were solved using fuzzy simheuristics to allow for different types of uncertainty (stochastic and fuzzy), while the others were solved using a sim-learnheuristics to handle both stochastic and dynamic elements. Similar to the previous table, this table shows the OBD, the OBD-H value (which indicates the expected cost when the best deterministic solution is assessed in a scenario with different types of uncertainty or dynamics), and the OBH, which indicates the expected cost using a fuzzy or sim-learnheuristic. The gap between the OBD-H and the OBS is depicted in Fig 6. As with the previous simheuristic results, the simheuristic approach outperforms the deterministic solution when applied in dynamic and uncertain scenarios for all tested problems.

Some Open Challenges

Despite being a relatively new approach, simheuristic algorithms have been successfully applied across a variety of fields. Methodologically, numerous research lines remain open that could enhance the efficiency and applicability of these algorithms. For instance, there is no universally accepted method or guidelines for

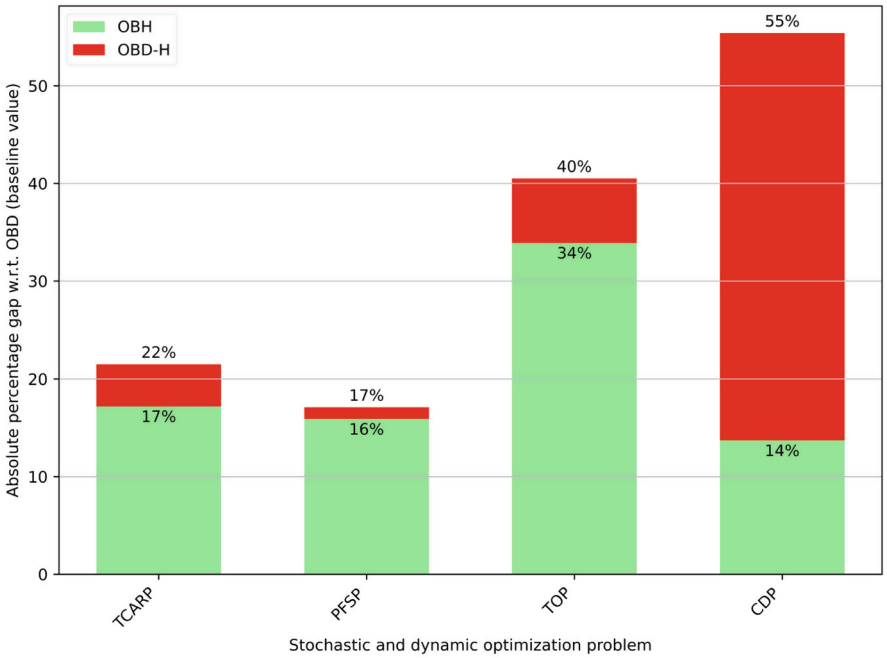


Fig. 6 Gaps of OBD-H and OBH solutions with respect to OBD solution (baseline at 0% gap)

determining the number of simulation executions to use. This decision involves balancing computing time and estimate accuracy. However, techniques like variance reduction can decrease the number of runs required without sacrificing accuracy. The use of parallelization techniques, distributed computing, and high-performance computing to speed up computations is also a promising field to explore in more detail. Another crucial aspect that has not been thoroughly studied is the criteria for labeling a solution as "promising" and thus warranting assessment by the simulation component. The proportion of such solutions depends on available computing time and the time needed to evaluate each solution. A preliminary exploratory study could help estimate the correlation between deterministic and stochastic measures, aiding in setting this proportion.

Integrating machine learning techniques into metaheuristics offers a natural solution for problems with dynamic inputs [9]. Thus, combining simheuristics and machine learning into what we called sim-learnheuristics is a promising approach to handle the increasing stochasticity and dynamism of real-world problems. Furthermore, machine learning techniques can enhance the efficiency of simheuristic algorithms, for example, by determining the number of scenarios or adaptively choosing when to send a solution to the simulation component based on learning from previously visited solutions. Even more, machine learning predictive models can constitute surrogate models for the simulation process, thus saving computational time.

Conclusions

This chapter has emphasized the advantages of combining simulation with metaheuristics to consider real-life optimization problems under uncertain conditions. Pure simulation models, while effective in modeling randomness, fall short in optimizing complex systems on their own. Similarly, deterministic optimization methods fall short in incorporating random inputs and probabilistic constraints commonly encountered in real-world decision-making. As systems in sectors like transportation, production logistics, supply chain management, telecommunication networks, energy, and finance grow more complex and large scale, using simheuristics and similar simulation-optimization methods becomes essential for accounting for uncertainty.

There are multiple ways to integrate simulation into a metaheuristic framework. This chapter has highlighted some best practices, both methodological and computational, that have proven useful in avoiding common pitfalls. Recommendations include allocating a balanced amount of computing time to the simulation component, using simulation results to better guide the metaheuristic search, and making more informed decisions on candidate solutions by conducting risk analyses using various statistics. Current challenges and open research areas were also identified, particularly *(i)* the fuzzy simheuristic extension, which can consider fuzzy uncertainty together with stochastic uncertainty; and *(ii)* the promising combination of simheuristics with learnheuristics for tackling optimization problems that involve both stochastic and dynamic elements, where the ability to adapt to changes and uncertainties in real time can significantly improve outcomes.

Cross-References

- ▸ City Logistics
- ▸ Heuristics in Vehicle Routing
- ▸ Multiobjective Optimization
- ▸ Scheduling Heuristics
- ▸ Supply Chain Management

Acknowledgments This work has been partially funded by the Spanish Ministry of Science and Innovation (PID2022-138860NB-I00, PID2022-140278NB-I00, RED2022-134703-T).

Competing Interest Declaration The author(s) has no competing interests to declare that are relevant to the content of this manuscript.

References

1. Abu-Marrul V, Martinelli R, Hamacher S, Gribkovskaia I (2023) Simheuristic algorithm for a stochastic parallel machine scheduling problem with periodic re-planning assessment. Ann Oper Res 320(2):547–572
2. Alvarez Fernandez S, Ferone D, Juan A, Tarchi D (2022) A simheuristic algorithm for video streaming flows optimisation with QoS threshold modelled as a stochastic single-allocation p-hub median problem. J Simul 16(5):480–493
3. Amaran S, Sahinidis NV, Sharda B, Bury SJ (2016) Simulation optimization: a review of algorithms and applications. Ann Oper Res 240:351–380
4. Angione C, Silverman E, Yaneske E (2022) Using machine learning as a surrogate model for agent-based simulations. Plos One 17(2):e0263150
5. Antoniadis N, Cordy M, Sifaleras A, Le Traon Y (2022) A variable neighborhood search simheuristic algorithm for reliability optimization of smart grids under uncertainty. Int Trans Oper Res 29(4):2172–2200
6. April, J., Glover F, Kelly JP, Laguna M (2003). Simulation-based optimization: practical introduction to simulation optimization. In: Proceedings of the winter simulation conference, pp 71–78
7. Bayliss C, Panadero J (2024) Simheuristic and learnheuristic algorithms for the temporary-facility location and queuing problem during population treatment or testing events. J Simul 18(4):626–645
8. Caldeira RH, Gnanavelbabu A (2021) A simheuristic approach for the flexible job shop scheduling problem with stochastic processing times. Simulation 97(3):215–236
9. Calvet L, Armas Jd, Masip D, Juan AA (2017) Learnheuristics: hybridizing metaheuristics with machine learning for optimization with dynamic inputs. Open Math 15(1):261–280
10. Castaneda J, Ghorbani E, Ammouriova M, Panadero J, Juan AA (2022) Optimizing transport logistics under uncertainty with simheuristics: concepts, review and trends. Logistics 6(3):42
11. Castaneda J, Martin XA, Ammouriova M, Panadero J, Juan AA (2022) A fuzzy simheuristic for the permutation flow shop problem under stochastic and fuzzy uncertainty. Mathematics 10(10):1760
12. Chandran S, Kandaswamy G (2016) A fuzzy approach to transport optimization problem. Optim Eng 17:965–980
13. Chica M, Juan AA, Bayliss C, Cordón O, Kelton WD (2020) Why simheuristics? benefits, limitations, and best practices when combining metaheuristics with simulation. SORT-Stat Oper Res Trans 44(2):311–334
14. Corlu CG, Akcay A, Xie W (2020) Stochastic simulation under input uncertainty: a review. Oper Res Perspect 7:100162
15. Datta S, Kapoor R, Mehta P (2023) A multi-objective optimization model for outpatient care delivery with service fairness. Busin Proc Manag J 29(3):630–652
16. de León AD, Lalla-Ruiz E, Melián-Batista B, Moreno-Vega JM (2021). A simulation–optimization framework for enhancing robustness in bulk berth scheduling. Eng Appl Artif Intell 103:104276
17. Dehghanimohammadabadi M, Kabadayi N (2020) A two-stage ahp multi-objective simulation optimization approach in healthcare. Int J Analy Hierarchy Proc 12(1):117–135
18. Dehghanimohammadabadi M, Rezaeiahari M, Seif J (2023) Multi-objective patient appointment scheduling framework (mo-pass): a data-table input simulation-optimization approach. Simulation 99(4):363–383
19. do Amaral JVS, Montevechi JAB, de Carvalho Miranda R, de Sousa Junior WT (2022) Metamodel-based simulation optimization: a systematic literature review. Simul Modell Pract Theory 114:102403

20. Figueira G, Almada-Lobo B (2014) Hybrid simulation-optimization methods: a taxonomy and discussion. Simul Modell Pract Theory 46:118–134
21. Ghorbani E, Gomez JF, Panadero J, Juan AA (2024) A sim-learnheuristic algorithm for solving a capacitated dispersion problem under stochastic and non-static conditions. AIMS Math 9(9):24247–24270
22. Ghorpade T, Corlu CG (2023) A simheuristic algorithm for the stochastic one-commodity pickup and delivery travelling salesman problem. J Simul 17(6):688–708
23. Gök YS, Padrón S, Tomasella M, Guimarans D, Ozturk C (2023) Constraint-based robust planning and scheduling of airport apron operations through simheuristics. Ann Oper Res 320(2):795–830
24. Glover F, Kelly JP, Laguna M (1996). New advances and applications of combining simulation and optimization. In: Proceedings of the winter simulation conference, pp 144–152
25. Glover F, Kelly JP, Laguna M (1999). New advances for wedding optimization and simulation. In: Proceedings of the winter simulation conference, pp 255–260
26. Gnanavelbabu A, Caldeira RH, Vaidyanathan T (2021) A simulation-based modified backtracking search algorithm for multi-objective stochastic flexible job shop scheduling problem with worker flexibility. Appl Soft Comput 113:107960
27. Gomez JF, Martínez-Gavara A, Panadero J, Juan AA, Martí R (2024) A forward-backward simheuristic for the stochastic capacitated dispersion problem. Mathematics 12(6):909
28. González-Martín S, Juan AA, Riera D, Castellà Q, Muñoz R, Pérez A (2012) Development and assessment of the sharp and randsharp algorithms for the arc routing problem. AI Commun 25(2):173–189
29. Gonzalez-Neira EM, Montoya-Torres JR, Jimenez J-F (2021) A multicriteria simheuristic approach for solving a stochastic permutation flow shop scheduling problem. Algorithms 14(7):210
30. Gorissen BL, Yanıkoğlu İ, Den Hertog D (2015) A practical guide to robust optimization. Omega 53:124–137
31. Gruler A, Panadero J, de Armas J, Pérez JAM, Juan AA (2020) A variable neighborhood search simheuristic for the multiperiod inventory routing problem with stochastic demands. Int Trans Oper Res 27(1):314–335
32. Grumbach F, Badr NEA, Reusch P, Trojahn S (2023) A memetic algorithm with reinforcement learning for sociotechnical production scheduling. IEEE Access 11:68760–68775
33. Guimarans D, Dominguez O, Panadero J, Juan AA (2018) A simheuristic approach for the two-dimensional vehicle routing problem with stochastic travel times. Simul Modell Pract Theory 89:1–14
34. Haneveld K, Van der Vlerk MH (2020) Stochastic programming. Springer, Berlin
35. Hatami S, Calvet L, Fernández-Viagas V, Framiñán JM, Juan AA (2018) A simheuristic algorithm to set up starting times in the stochastic parallel flowshop problem. Simul Modell Pract Theory 86:55–71
36. Herrera EM, Panadero J, Carracedo P, Juan AA, Perez-Bernabeu E (2022) Determining reliable solutions for the team orienteering problem with probabilistic delays. Mathematics 10(20):3788
37. Hirpara S, Parikh PJ, Kong N (2024) Impact of mistriages in on-scene injury assessment on trauma network and patient safety. IISE Trans Healthcare Syst Eng 14:1–18
38. Juan A, Faulin J, Grasman S, Riera D, Marull J, Mendez C (2011) Using safety stocks and simulation to solve the vehicle routing problem with stochastic demands. Transport Res Part C Emer Technol 19(5):751–765
39. Juan AA, Barrios BB, Vallada E, Riera D, Jorba J (2014) A simheuristic algorithm for solving the permutation flow shop problem with stochastic processing times. Simul Modell Pract Theory 46:101–117
40. Juan AA, Faulin J, Grasman SE, Rabe M, Figueira G (2015) A review of simheuristics: extending metaheuristics to deal with stochastic combinatorial optimization problems. Oper Res Perspect 2:62–72

41. Juan AA, Keenan P, Martí R, McGarraghy S, Panadero J, Carroll P, Oliva D (2023) A review of the role of heuristics in stochastic optimisation: from metaheuristics to learnheuristics. Ann Oper Res 320(2):831–861
42. Kabadayi N, Dehghanimohammadabadi M (2022) Multi-objective supplier selection process: a simulation-optimization framework integrated with MCDM. Ann Oper Res 319(2):1607–1629
43. Keenan P, Panadero J, Juan AA, Martí R, McGarraghy S (2021) A strategic oscillation simheuristic for the time capacitated arc routing problem with stochastic demands. Comput Oper Res, 133:105377
44. Kızıloğlu K, Sakallı ÜS (2023) Integrating flight scheduling, fleet assignment, and aircraft routing problems with codesharing agreements under stochastic environment. Aerospace 10(12):1031
45. Kizys R, Doering J, Juan AA, Polat O, Calvet L, Panadero J (2022) A simheuristic algorithm for the portfolio optimization problem with random returns and noisy covariances. Comput Oper Res 139:105631
46. Lai V, Huang Y, Koo C, Najah Ahmed A, El-Shafie A (2022). Conceptual sim-heuristic optimization algorithm to evaluate the climate impact on reservoir operations. J Hydrol 614:128530
47. Latorre-Biel JI, Ferone D, Juan AA, Faulin (2021) Combining simheuristics with petri nets for solving the stochastic vehicle routing problem with correlated demands. Expert Syst Appl 168:114240
48. Leon JF, Li Y, Peyman M, Calvet L, Juan AA (2023). A discrete-event simheuristic for solving a realistic storage location assignment problem. Mathematics 11(7):1577
49. Lopes TC, Michels AS, Lüders R, Magatão L (2020). A simheuristic approach for throughput maximization of asynchronous buffered stochastic mixed-model assembly lines. Comput Oper Res 115:104863
50. Lucas TW, Kelton WD, Sanchez PJ, Sanchez SM, Anderson BL (2015) Changing the paradigm: Simulation, now a method of first resort. Naval Res Logist 62(4):293–303
51. Martí R, Sevaux M, Sörensen K (2024). 50 years of metaheuristics. Eur J Oper Res 321:345–362
52. Martin XA, Panadero J, Peidro D, Perez-Bernabeu E, Juan AA (2023) Solving the time capacitated arc routing problem under fuzzy and stochastic travel and service times. Networks 82(4):318–335
53. Martins LDC, Bayliss C, Copado-Méndez PJ, Panadero J, Juan AA (2020) A simheuristic algorithm for solving the stochastic omnichannel vehicle routing problem with pick-up and delivery. Algorithms 13(9):237
54. Martínez-Reyes A, Quintero-Araújo CL, Solano-Charris EL (2021) Supplying personal protective equipment to intensive care units during the covid-19 outbreak in colombia. a simheuristic approach based on the location-routing problem. Sustainability (Switzerland) 13(14):7822
55. Meeß H, Herzog M, Alp E, Kuhlenkötter B (2023) Evolutionary algorithms for a simheuristic optimization of the product-service system design. J Intell Manuf 35:3235–3257
56. Negri E, Pandhare V, Cattaneo L, Singh J, Macchi M, Lee J (2021) Field-synchronized digital twin framework for production scheduling with uncertainty. J Intell Manuf 32(4):1207–1228
57. Nelson B et al (2021) Foundations and methods of stochastic simulation. Springer, Berlin
58. Nessari S, Tavakkoli-Moghaddam R, Bakhshi-Khaniki H, Bozorgi-Amiri A (2024) A hybrid simheuristic algorithm for solving bi-objective stochastic flexible job shop scheduling problems. Decision Analy J 11:100485
59. Nieto A, Serra M, Juan AA, Bayliss C (2023) A ga-simheuristic for the stochastic and multi-period portfolio optimisation problem with liabilities. J Simul 17(5):632–645
60. Onggo BS, Panadero J, Corlu CG, Juan AA (2019) Agri-food supply chains with stochastic demands: a multi-period inventory routing problem with perishable products. Simul Modell Practice Theory 97:101970
61. Panadero J, Barrena E, Juan AA, Canca D (2022) The stochastic team orienteering problem with position-dependent rewards. Mathematics 10(16):2856

62. Panadero J, Doering J, Kizys R, Juan AA, Fito A (2020) A variable neighborhood search simheuristic for project portfolio selection under uncertainty. J Heurist 26(3):353–375
63. Panadero J, Juan AA, Bayliss C, Currie C (2020) Maximising reward from a team of surveillance drones: a simheuristic approach to the stochastic team orienteering problem. Eur J Ind Eng 14(4):485–516
64. Panadero J, Juan AA, Ghorbani E, Faulin J, Pagès-Bernaus A (2024) Solving the stochastic team orienteering problem: comparing simheuristics with the sample average approximation method. Int Trans Oper Res 31(5):3036–3060
65. Pandhare V, Negri E, Ragazzini L, Cattaneo L, Macchi M, Lee J (2024) Digital twin-enabled robust production scheduling for equipment in degraded state. J Manuf Syst 74:841–857
66. Peidro D, Martin XA, Panadero J, Juan AA (2024) Solving the uncapacitated facility location problem under uncertainty: a hybrid tabu search with path-relinking simheuristic approach. Appl Intell 54(7):5617–5638
67. Peng Y, Chen C-H, Fu MC (2023). Simulation optimization in the new era of AI. In: Tutorials in operations research: advancing the frontiers of OR/MS: from methodologies to applications. INFORMS, pp 82–108
68. Peyman M, Martin XA, Panadero J, Juan AA (2024) A sim-learnheuristic for the team orienteering problem: applications to unmanned aerial vehicles. Algorithms 17(5):200
69. Quintero-Araujo CL, Guimarans D, Juan AA (2021) A simheuristic algorithm for the capacitated location routing problem with stochastic demands. J Simul 15(3):217–234
70. Rabe M, Deininger M, Juan AA (2020) Speeding up computational times in simheuristics combining genetic algorithms with discrete-event simulation. Simul Modell Pract Theory 103:102089
71. Rabe M, Ammouriova M, Schmitt D, Dross F (2021) Simheuristics approaches for efficient decision-making support in materials trading networks. Algorithms 14(1):23
72. Raba D, Tordecilla RD, Copado P, Juan AA, Mount D (2022) A digital twin for decision making on livestock feeding. INFORMS J Appl Analy 52(3):267–282
73. Rabet R, Ganji M, Fathi M (2024) A simheuristic approach towards supply chain scheduling: integrating production, maintenance and distribution. Appl Soft Comput 153:111264
74. Ramirez-Villamil A, Jaegler A, Montoya-Torres JR (2022) Sustainable local pickup and delivery: the case of Paris. Res Transport Business Manag 45:100692
75. Ramirez-Villamil A, Montoya-Torres JR, Jaegler A, Diaz Y (2024) Stochastic two-echelon vehicle routing problem with time windows: city logistics case study. Int J Shipping Transp Logist 18(2):223–248
76. Reyes-Rubiano L, Ferone D, Juan AA, Faulin J (2019) A simheuristic for routing electric vehicles with limited driving ranges and stochastic travel times. SORT 1:3–24
77. Rodríguez-Espinosa CA, González-Neira EM, Zambrano-Rey GM (2024) A simheuristic approach using the NSGA-II to solve a bi-objective stochastic flexible job shop problem. J Simul 18(4):646–670
78. Ruiz-Rodríguez ML, Kubler S, Robert J, Le Traon Y (2024). Dynamic maintenance scheduling approach under uncertainty: comparison between reinforcement learning, genetic algorithm simheuristic, dispatching rules. Expert Syst Appl 248:123404
79. Santos MS, Pinto TV, Júnior ÊL, Cota LP, Souza MJ, Euzébio TA (2020) Simheuristic-based decision support system for efficiency improvement of an iron ore crusher circuit. Eng Appl Artif Intell 94:103789
80. Seiringer W, Castaneda J, Altendorfer K, Panadero J, Juan AA (2022) Applying simheuristics to minimize overall costs of an mrp planned production system. Algorithms 15(2):40
81. Shone R, Glazebrook K, Zografos KG (2024) A new simheuristic approach for stochastic runway scheduling. Transport Sci 58(2):520–539
82. Tordecilla RD, Copado-Méndez PJ, Panadero J, Quintero-Araujo CL, Montoya-Torres JR, Juan AA (2021) Combining heuristics with simulation and fuzzy logic to solve a flexible-size location routing problem under uncertainty. Algorithms 14(2):45
83. Vajda S (2014) Probabilistic programming. Academic, Cambridge

84. Vieira AA, Figueira JR, Fragoso R (2023) A multi-objective simulation-based decision support tool for wine supply chain design and risk management under sustainability goals. Expert Syst Appl 232:120757
85. Villarinho PA, Panadero J, Pessoa LS, Juan AA, Oliveira FLC (2021) A simheuristic algorithm for the stochastic permutation flow-shop problem with delivery dates and cumulative payoffs. Int Trans Oper Res 28(2):716–737
86. Yazdani M, Mojtahedi M, Loosemore M (2020) Enhancing evacuation response to extreme weather disasters using public transportation systems: a novel simheuristic approach. J Comput Design Eng 7(2):195–210
87. Yazdani M, Kabirifar K, Frimpong BE, Shariati M, Mirmozaffari M, Boskabadi A (2021). Improving construction and demolition waste collection service in an Urban area using a simheuristic approach: a case study in Sydney, Australia. J Cleaner Prod 280:124138
88. Yu VF, Salsabila NY, Siswanto N, Kuo P-H (2022) A two-stage genetic algorithm for joint coordination of spare parts inventory and planned maintenance under uncertain failures. Appl Soft Comput, 130:109705
89. Zambrano-Rey GM, González-Neira EM, Forero-Ortiz GF, Ocampo-Monsalve MJ, Rivera-Torres A (2024) Minimizing the expected maximum lateness for a job shop subject to stochastic machine breakdowns. Ann Oper Res 338(1):801–833

The Hybrid Metaheuristic CMSA

14

Christian Blum and Jaume Reixach

Contents

Introduction 364
CMSA Variants 365
Self-Adaptive CMSA 367
CMSA Extended with Reinforcement Learning 368
Timeline of CMSA Applications 370
Application of CMSA to Maximum Independent Set 370
Definition of the Solution Components 372
Probabilistic Construction of MIS Solutions 372
Sub-instance Solving 374
Experimental Evaluation 374
Conclusion 384
Cross-References 384
References 384

Abstract

In this chapter, we describe the Construct, Merge, Solve & Adapt (CMSA) algorithm, a metaheuristic framework designed to address hard combinatorial optimization problems. CMSA combines the probabilistic construction of solutions within an adaptive, iterative process with solution merging and exact optimization techniques. The algorithm probabilistically constructs solutions through heuristic methods, which are then merged into a reduced subproblem. This subproblem is solved using exact optimization approaches, mostly integer programming. Based on the feedback from this solving phase, the algorithm adapts its construction and merging strategies in subsequent iterations,

C. Blum (✉) · J. Reixach
Artificial Intelligence Research Institute (IIIA-CSIC), Bellaterra, Spain
e-mail: christian.blum@iiia.csic.es; jaume.reixach@iiia.csic.es

© Springer Nature Switzerland AG 2025

R. Martí et al. (eds.), *Handbook of Heuristics*,
https://doi.org/10.1007/978-3-032-00385-0_79

progressively finding solutions of improving quality over time. CMSA's hybrid nature allows it to balance between the speed of heuristic construction and the accuracy of exact methods, making it particularly effective for large-scale problems. After a general description of standard CMSA, we outline a recent self-adaptive variant and a variant that uses reinforcement learning to improve the search process. All three algorithm variants are applied to the classical maximum independent set problem. Moreover, an experimental evaluation is provided to show their comparative behavior.

Keywords

Hybrid metaheuristic · Combinatorial optimization · Maximum independent set

Introduction

The "Construct, Merge, Solve & Adapt" (CMSA) algorithm [9, 15] is a metaheuristic framework designed to tackle combinatorial optimization problems, especially aiming for problem instances that are hard to solve exactly. It combines constructive heuristics, solution merging, and problem-solving techniques in an iterative process, adapting the search process based on the feedback from previous iterations. Each algorithm iteration consists of the following four steps, also shown in the general CMSA framework depicted in Fig 1. In the first step, solutions are constructed probabilistically using a heuristic approach. These solutions do not necessarily need to be of high quality. Instead, their components are expected to serve as building blocks of high-quality solutions. The heuristic approach may involve greedy algorithms, randomized methods, or other simple techniques that generate solutions quickly. After constructing solutions, the algorithm proceeds to merge

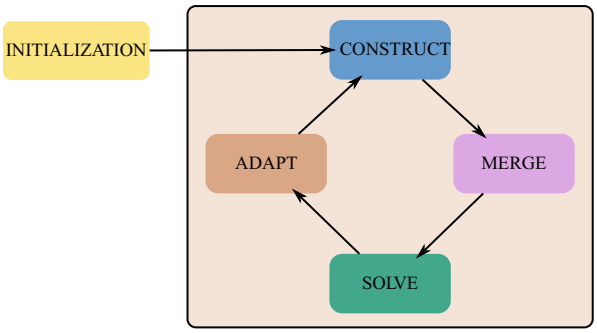


Fig. 1 General framework of CMSA

their components to generate a smaller subproblem, that is, a sub-instance of the original problem instance. In other words, the algorithm employs a bottom-up way of generating a sub-instance by merging components originating from different solutions. The incumbent sub-instance is solved in the third step using an appropriate solver. If the combinatorial optimization problem under consideration can be expressed as an integer linear program (ILP), for example, any ILP solver may be used for this purpose. The principal idea is that the sub-instance is more manageable, making it possible to solve it much more quickly than the original problem instance. Finally, the sub-instance is adapted based on the solution to the incumbent sub-instance returned by the solver. CMSA repeats this cycle, iterating between constructing, merging, solving, and adapting, progressively improving the quality of the obtained solutions. Over time, it generally converges toward a high-quality or near-optimal solution.

The key features of CMSA are as follows. First, as a hybrid approach, it combines the strengths of constructive heuristics (speed) and exact methods (precision). CMSA is particularly suited for large-scale, complex combinatorial optimization problems where exact methods may be too slow or even inviable. It can also be seen as a methodology for using exact techniques in the context of problem instances that are too large for their standalone application.

CMSA Variants

The first and original CMSA variant [15] works as follows. First, the complete set of solution components (C) must be defined. For example, when considering the traveling salesman problem (TSP), each edge between a pair of cities may be considered a solution component. For a second example, consider the one-dimensional bin packing problem where, intuitively, each item to be packed is a solution component. For providing a problem-independent description of CMSA, we henceforth consider a generic set $C = \{c_1, \ldots, c_n\}$ of solution components. We assume that any valid solution $S \in \mathcal{S}$, where $\mathcal{S}$ is the set of all feasible solutions, can be expressed as a subset of C, that is, $S \subseteq C$. Finally, let $f : \mathcal{S} \mapsto \mathbb{N}^+$ denote the objective function we aim to minimize.

Algorithm 1 contains the pseudo-code for this first and original CMSA variant. It begins by initializing the sub-instance (C') to the empty set. Moreover, the age values of the solution components are set to zero. The best-so-far solution S^{bsf} remains uninitialized. Each iteration of CMSA consists of the following four actions.

1. The first step of each iteration consists in the probabilistic construction of n_{a} (generally valid) solutions to the given problem. This is done by calling n_{a} times function ConstructSolutionProbabilistically(C, d_{rate}) in line 6 of Algorithm 1. Note that this action is problem-dependent. A randomized greedy heuristic, whose degree of randomness is determined by parameter d_{rate}, is typically employed for solution construction tailored to the considered optimization problem.

Algorithm 1: Standard CMSA

1: **input 1:** Complete set of solution components (C)
2: **input 2:** Values for n_a, d_{rate}, age_{max}, t_{SOLVER}
3: **initialization:** $C' := \emptyset$, $age[c] := 0\ \forall c \in C$, S^{bsf} is not initialized
4: **while** CPU time limit not reached **do**
5: **for** $i := 1, \ldots, n_a$ **do**
6: $S :=$ ConstructSolutionProbabilistically(C, d_{rate})
7: **if** S^{bsf} not initialized **or** $f(S) < f(S^{bsf})$ **then** $S^{bsf} := S$ **endif**
8: **for** all $c \in S$ **and** $c \notin C'$ **do**
9: $age[c] := 0$
10: $C' := C' \cup \{c\}$
11: **end for**
12: **end for**
13: $S^{SOLVER} :=$ SolveSubinstance(C', t_{SOLVER})
14: **if** $f(S^{SOLVER}) < f(S^{bsf})$ **then** $S^{bsf} := S^{SOLVER}$ **end if**
15: AdaptSubinstance(C', S^{SOLVER}, age_{max})
16: **end while**
17: **output:** S^{bsf}

2. Next, the solution components found in these n_a solutions are added to the incumbent sub-instance C'. Their age value is set to zero if they are not already present in C'. This is shown in lines 8–11 of Algorithm 1.
3. Third, an appropriate solver is applied to the updated sub-instance C' in function SolveSubinstance(C', t_{SOLVER}) (see line 13) to derive the best solution possible within the given time limit of t_{SOLVER} CPU seconds. There are no limitations concerning the nature of this solver. It may be a problem-specific exact or heuristic solver, or it may be, for example, an ILP solver. Note that the feasible search space of C' consists of those solutions to the original (complete) problem instance that only contains components present in C'. Henceforth, the solution returned by the employed solver is called S^{SOLVER}.
4. Finally, the incumbent sub-instance C' is adapted on the basis of solution S^{SOLVER} in function Adapt(C', S^{SOLVER}, age_{max}) at line 15. This is a problem-independent step that works as follows. After incrementing the age values of all solution components in $C' \setminus S^{SOLVER}$ by one, the age values of all components present in S^{SOLVER} are set to zero. In addition, to alleviate the work of the solver, all those solution components in C' whose age value has reached the user-determined upper limit of age_{max} are removed from C'. In this context, note that the age value of a solution component, at any given time, indicates the number of consecutive CMSA iterations in which this component has been part of the sub-instance but has not been included in the solver's solution to that sub-instance.

Note that CMSA is iterated until a predefined CPU time limit is reached. Upon termination, the algorithm returns the best-so-far solution S^{bsf} as output. The generic CMSA parameters include the number of solutions generated per iteration (n_a), the maximum allowable age of solution components (age_{max}), and the time limit for the

utilized solver at each iteration (t_{SOLVER}). However, additional parameters related, for example, to the sub-instance solver may be used.

Self-Adaptive CMSA

The standard CMSA algorithm was shown to be a well-working algorithm across various combinatorial optimization problems; however, certain applications have revealed its sensitivity to parameter settings. To address this issue, a self-adaptive variant of CMSA, named Adapt-CMSA, was proposed in [2]. A key feature of Adapt-CMSA is that it does not require intensive parameter tuning across subsets of the problem instances considered.

Adapt-CMSA (see Algorithm 2) differs from the standard CMSA primarily in how it manages the size of the sub-instance. In the standard CMSA, sub-instance size is controlled using age values in combination with a maximum age limit for solution components. In contrast, Adapt-CMSA does not rely on age values. Instead, after each iteration, all solution components, except those that are part of the best-so-far solution S^{bsf}, are removed from the sub-instance C' (see line 23).

Algorithm 2: Self-adaptive CMSA (Adapt-CMSA)

1: **input 1:** Complete set of solution components (C)
2: **input 2:** Values for Adapt-CMSA parameters t_{prop}, t_{SOLVER}
3: **input 3:** Values for solution construction parameters α^{LB}, α^{UB}, α_{red}
4: **initialization:** $n_{\text{a}} := 1$; $\alpha_{\text{bsf}} := \alpha^{\text{UB}}$; $C' := S^{\text{bsf}}$
5: $S^{\text{bsf}} :=$ GenerateGreedySolution(C)
6: **while** CPU time limit not reached **do**
7: **for** $i := 1, \ldots, n_{\text{a}}$ **do**
8: $S :=$ ConstructSolutionProbabilistically(C, S^{bsf}, α_{bsf})
9: $C' := C' \cup S$
10: **end for**
11: $(S^{\text{SOLVER}}, t_{\text{req}}) :=$ SolveSubinstance(C', t_{SOLVER}) ▷ This function returns two objects: (1) the obtained solution (S^{SOLVER}), (2) the required computation time (t_{req})
12: **if** $t_{\text{req}} < t_{\text{prop}} \cdot t_{\text{SOLVER}}$ and $\alpha_{\text{bsf}} > \alpha^{\text{LB}}$ **then** $\alpha_{\text{bsf}} := \alpha_{\text{bsf}} - \alpha_{\text{red}}$ **end if**
13: **if** $f(S^{\text{SOLVER}}) < f(S^{\text{bsf}})$ **then**
14: $S^{\text{bsf}} := S^{\text{SOLVER}}$
15: $n_{\text{a}} := 1$
16: **else**
17: **if** $f(S^{\text{SOLVER}}) > f(S^{\text{bsf}})$ **then**
18: **if** $n_a = 1$ **then** $\alpha_{\text{bsf}} := \min\{\alpha_{\text{bsf}} + \frac{\alpha_{\text{red}}}{10}, \alpha^{\text{UB}}\}$ **else** $n_a = 1$ **end if**
19: **else**
20: $n_{\text{a}} := n_{\text{a}} + 1$
21: **end if**
22: **end if**
23: $C' := S^{\text{bsf}}$ ▷ Only the solution components of S^{bsf} remain in C'
24: **end while**
25: **output:** S^{bsf}

Sub-instance size in Adapt-CMSA is partially controlled during the probabilistic construction of solutions at each iteration (see function ConstructSolutionProbabilistically(C, S^{bsf}, α_{bsf}) in line 8). This function receives as input not only the set of all possible solution components (C) but also the current best-so-far solution S^{bsf} and a parameter α_{bsf} (where $0 \leq \alpha_{\text{bsf}} < 1$). The parameter α_{bsf} influences the construction of new solutions by biasing them toward the best-so-far solution S^{bsf}; specifically, higher values of α_{bsf} lead to greater similarity between the generated solutions and S^{bsf}.

The self-adaptive nature of Adapt-CMSA is shown in the dynamic adjustment of α_{bsf} over time and depending on the search progress. In particular, the value of α_{bsf} can move between a lower bound α^{LB} and an upper bound α^{UB}, both algorithm parameters. At the beginning of the algorithm, α_{bsf} is initialized to its maximum α^{UB} (see line 4). In other words, at the start of the algorithm, the generated solutions will be most similar to the best-so-far solution S^{bsf}. If the incumbent sub-instance is solved within a computation time t_{req} that is less than a fraction t_{prop} of the maximum allowed computation time t_{SOLVER}, α_{bsf} is reduced by α_{red} (see line 12), where α_{red} is another algorithm parameter. The reasoning behind this adjustment is as follows: When the current sub-instance is solved easily, it suggests a relatively small search space. To expand the search space, the solutions generated in ConstructSolutionProbabilistically(C, S^{bsf}, α_{bsf}) should be more diverse relative to S^{bsf}. This is accomplished by reducing the α_{bsf} value.

Apart from α_{bsf}, the number of solution constructions per iteration (n_a) is also self-adaptive; see lines 13–22. At the start of Adapt-CMSA, n_a is set to 1 (see line 4). If the solution to the sub-instance provided by the solver (S^{SOLVER}) improves over the best-so-far solution S^{bsf}, n_a is reset to one (see line 15). Conversely, if S^{SOLVER} is strictly inferior to S^{bsf}, the corresponding sub-instance was too large (resp., difficult) for the solver to be solved within t_{SOLVER} seconds. If this case coincides with a setting of $n_a = 1$, the α_{bsf} value is marginally increased (by $\frac{\alpha_{\text{red}}}{10}$); otherwise, n_a is reset to one. In the event that $f(S^{\text{SOLVER}}) = f(S^{\text{bsf}})$, n_a is incremented by one (see line 20). This is done because solving the sub-instance failed to produce a solution better than S^{bsf}, even though the sub-instance was still solved to optimality within the allowed computation time of t_{SOLVER} seconds, suggesting that the size of the sub-instance should be increased.

CMSA Extended with Reinforcement Learning

Another potential limitation of standard CMSA is that the probabilistic construction of solutions in each iteration relies on a static, unchanging probability distribution. In other words, the probability of generating any particular valid solution remains constant throughout the algorithm's execution. In principle, this can be changed by adding a learning component to the solution construction phase. The first ones to do so were the authors of [28]. They intertwined CMSA iterations with the running of a population-based metaheuristic. At each iteration of their Learn-CMSA approach, the sub-instance of Learn-CMSA receives input from the incumbent population of

the metaheuristics and vice versa. A somewhat more straightforward approach was recently presented in [29], where the authors present a CMSA variant in which the construction of solutions is based on a simple reinforcement learning (RL) mechanism, with rewards determined on the basis of the solution returned by the solver at each iteration. This CMSA variant is called RL-CMSA and is presented in Algorithm 3.

RL-CMSA maintains a set $Q = \{q_1, \ldots, q_n\}$ of quality values which contains a q_i for each solution component $c_i \in C$. One of the main characteristics of RL-CMSA is that solutions are constructed by sampling solution components depending on these values. At the start of the algorithm, all $q_i \in Q$ are initialized to zero. Moreover, these values change during the algorithm due to cumulating rewards received after the adapt step depending on the solution S^{SOLVER} produced by the solver. In particular, those solution components from C' that form part of S^{SOLVER} receive a reward of one, while those from C' that the solver did not select receive a reward of minus one. In this way, the solver acts as a feedback mechanism for the solution construction process, and the values in Q keep track of the "quality" of the different solution components. The process of determining the rewards and adding them to the quality values is done in UpdateQ(q, C', S^{SOLVER}); see line 17.

Solutions are constructed in function ConstructSolutionProbabilistically(C, q, d_{rate}, β) in line 7 exclusively based on the quality values; that is, no heuristic information is used to bias this solution construction. The use of parameters d_{rate} and β will be outlined later. The only problem-dependent part is to be found

Algorithm 3: Reinforcement learning CMSA (RL-CMSA)

1: **input 1:** Complete set of solution components (C)
2: **input 2:** Values for RL-CMSA parameters n_{a}, $age_{\max}$, t_{SOLVER}, cf_{limit}, b_{reset}
3: **input 3:** Values for solution construction parameters d_{rate}, β
4: **initialization:** $C' := \emptyset$, $age[c] := 0$ and $q[c] := 0$ $\forall c \in C$, S^{bsf} is not initialized
5: **while** CPU time limit not reached **do**
6: **for** $i := 1, \ldots, n_{\text{a}}$ **do**
7: $S :=$ ConstructSolutionProbabilistically(C, q, d_{rate}, β)
8: **if** S^{bsf} not initialized **or** $f(S) < f(S^{\text{bsf}})$ **then** $S^{\text{bsf}} := S$ **endif**
9: **for** all $c \in S$ **and** $c \notin C'$ **do**
10: $age[c] := 0$
11: $C' := C' \cup \{c\}$
12: **end for**
13: **end for**
14: $S^{\text{SOLVER}} :=$ SolveSubinstance(C', t_{SOLVER})
15: **if** $f(S^{\text{SOLVER}}) < f(S^{\text{bsf}})$ **then** $S^{\text{bsf}} := S^{\text{SOLVER}}$ **end if**
16: AdaptSubinstance(C', S^{SOLVER}, $age_{\max}$)
17: UpdateQ(q, C', S^{SOLVER})
18: $cf :=$ ComputeConvergenceFactor(q)
19: **if** $cf > cf_{\text{limit}}$ **then**
20: $q[c] := 0$ $\forall c \in C$
21: **if** $b_{\text{reset}} = true$ **then** $C' := \emptyset$ **end if**
22: **end if**
23: **end while**
24: **output:** S^{bsf}

in determining those solution components that can be selected at each solution construction step.

As the reader might notice, if some quality values become much larger than the rest, the solution construction procedure could lead to highly similar solutions being constructed repeatedly. To avoid this, RL-CMSA implements a reset mechanism. This consists of computing a value cf denoted as the convergence factor, which gives a measure of the level of convergence, as follows. For every solution component c_i of the last solution constructed S, the probability z_i of selecting c_i over all the solution components not forming part of S is calculated. The convergence factor is then defined as $cf = \min_{c_i \in S} z_i$. Once the convergence factor is calculated, the algorithm checks if it is larger than a parameter $cf_{\text{limit}} \in [0, 1]$ known as the convergence factor limit. If this is the case, the algorithm is re-initialized, which consists of setting all quality values back to zero and emptying the sub-instance C' depending on a Boolean parameter b_{reset} (see lines 19-22). This last option allows completely erasing the information learned so far (if $b_{\text{reset}} = true$) or keeping some otherwise, in the form of the sub-instance C'.

Timeline of CMSA Applications

Table 1 provides a timeline of all applications of different CMSA variants found in the related literature to date. The first table column contains the name of the considered combinatorial optimization problem; the second column references the corresponding publication; the third column indicates the year of publication; and the last column provides information about the utilized (or introduced) CMSA variant. For easier visual differentiation between CMSA variants, colored dots are used in the last column, with a gray dot signifying a CMSA approach that does not align with any established CMSA variant. Any such exceptions are explained in the table.

Application of CMSA to Maximum Independent Set

In the final part of this chapter, we demonstrate the application of the three CMSA variants described before (standard CMSA, Adapt-CMSA, and RL-CMSA) to the NP-hard maximum independent set (MIS) problem. The MIS problem is a well-known combinatorial optimization problem in graph theory, which can be described as follows. Given is an undirected graph $G = (V, E)$. The set $N(v_i) \subset V$ denotes the set of neighbors of v_i in graph G, that is, for each $v_j \in N(v_i)$, there exists an edge $e = (v_i, v_j) \in E$. Moreover, $N[v_i]$ is the so-called closed neighborhood of v_i in G defined as follows: $N[v_i] := N(v_i) \cup \{v_i\}$. A set $S \subset V$ is a feasible solution to the MIS problem if for any pair $v_i \neq v_j \in S$ the edge set E of graph G does not contain the edge $e = (v_i, v_j)$. In other words, S is a feasible solution if no two nodes from S are neighbors in G. The optimization goal is to find a feasible solution that is as large as possible.

Table 1 Applications of CMSA until October 2024 (in alphabetical order)

Optimization problem	Publications	Year (first)	CMSA variant
Minimum covering arborescence	[15]	2016	Standard CMSA
Minimum common string partition	[8, 13, 15]	2016	Standard CMSA
Multi-dimensional knapsack	[13, 14, 24]	2016	Standard CMSA
Repetition-free longest common subsequence	[10, 11]	2016	Standard CMSA
Binary optimization	[12]	2019	Problem specific
Project scheduling	[35]	2019	Standard CMSA with ACO for solution construction
Minimum capacitated dominating set	[27]	2019	Standard CMSA
Routing of air-ground robots	[6]	2019	Standard CMSA
Maximum happy vertices problem	[21, 23, 34]	2019	Standard CMSA
Score-constrained packing	[22]	2020	Standard CMSA
Maintenance of nuclear power plants	[18]	2021	Standard CMSA
Test data generation in software product lines	[20]	2021	Standard CMSA
Minimum positive influence dominating set	[2]	2022	Adapt-CMSA
Bus driver scheduling	[30]	2022	Standard CMSA
Electric vehicle routing with time windows, simultaneous pickup and deliveries, and partial vehicle charging	[1]	2022	Adapt-CMSA
Unit disk cover	[5]	2022	Standard CMSA
Closest string problem	[16]	2023	Standard CMSA
Two-echelon electric vehicle routing with simultaneous pickup and deliveries	[3]	2023	Adapt-CMSA
Maximum disjoint dominating set	[31, 32]	2023	Standard CMSA with multiple solution constructors
Multi-way multi-dimensional number partitioning	[17]	2023	Adapt-CMSA
Rooted max tree coverage	[37]	2023	Standard CMSA
Diversity and equity optimization	[26]	2024	Standard CMSA
Minimum dominating set and far from most string problems	[29]	2024	RL-CMSA
Electric vehicle routing with time windows and simultaneous pickups and deliveries	[4]	2024	Standard CMSA
Variable-sized bin packing	[4]	2024	Adapt-CMSA
Far from most string problem	[28]	2024	Learn-CMSA

A typical ILP model for the MIS problem uses a binary variable x_i for each node $v_i \in V$. The formulation of the model is as follows:

$$\max \quad \sum_{v_i \in V} x_i \tag{1}$$

$$\text{subject to} \quad x_i + x_j \leq 1 \qquad \forall\, e = (v_i, v_j) \in V \tag{2}$$

$$x_i \in \{0, 1\} \qquad \forall\, v_i \in V$$

Constraints (2) make sure that any solution contains at most one endpoint of an edge in E.

Definition of the Solution Components

A key step in designing any CMSA algorithm is defining the set of solution components (C). For the MIS problem, we adopt a straightforward approach: The set C includes a component c_i for each node $v_i \in V$ of the input graph $G = (V, E)$. Therefore, it is important to keep in mind that, throughout the following discussion, both v_i and c_i refer to the same node v_i of the input graph G.

Probabilistic Construction of MIS Solutions

The process for constructing a valid MIS solution in standard CMSA is described in Algorithm 4. It begins with an empty solution, $V_{\text{sol}} = \emptyset$. At each step of the construction, $\tilde{V} \subseteq V$ is defined as the set of nodes that do not have a single neighbor included in V_{sol}. Exactly one node is selected from $\tilde{V}$ at each step using the function ChooseFrom($\tilde{V}$, d_{rate}). As mentioned before, the value of d_{rate} determines the randomness of the solution construction process. The selection process for a node is outlined as follows.

Algorithm 4: Function ConstructSolutionProbabilistically(C, d_{rate}) of standard CMSA

1: **input:** a graph $G = (V, E)$
2: $V_{\text{sol}} := \emptyset$
3: $\tilde{V} := V \setminus \bigcup_{v_i \in V_{\text{sol}}} N[v_i]$
4: **while** $\tilde{V} \neq \emptyset$ **do**
5: $\quad v_j :=$ ChooseFrom($\tilde{V}$, d_{rate})
6: $\quad V_{\text{sol}} := V_{\text{sol}} \cup \{v_j\}$
7: $\quad \tilde{V} := V \setminus \bigcup_{v_i \in V_{\text{sol}}} N[v_i]$
8: **end while**
9: **output:** a CMSA-solution S, corresponding to solution V_{sol}

Function ChooseFrom($\tilde{V}, d_{\text{rate}}$): A node $v_j \in \tilde{V}$ is chosen according to the greedy function $\mu(v_j) := |N(v_j)|$, which evaluates the degree of v_j. In this selection process, nodes with lower degrees are preferred. To choose a node v_j from $\tilde{V}$, a random number r is drawn uniformly from [0, 1]. If $r \leq d_{\text{rate}}$, node $v_j \in \tilde{V}$ is chosen such that

$$v_j := \text{argmin}\{\mu(v_i) \mid v_i \in \tilde{V}\} \ . \tag{3}$$

Alternatively, when $r > d_{\text{rate}}$, a set of $\min\{l_{\text{size}}, |\tilde{V}|\}$ nodes from $\tilde{V}$ is selected and added to a candidate set $L \subseteq \tilde{V}$ such that

$$\mu(v_i) \leq \mu(v_k) \quad \text{for all } v_i \in L, v_k \in \tilde{V} \setminus L \ . \tag{4}$$

A vertex $v_j \in L$ is then selected uniformly at random. It is important to highlight that once the construction of a solution V_{sol} is complete, it is converted into a corresponding CMSA-solution S. This conversion entails replacing each node $v_i \in V_{\text{sol}}$ with its corresponding solution component c_i. The obtained CMSA-solution S is then returned by Algorithm 4.

In Adapt-CMSA, solutions are constructed according to the same principle. Function GenerateGreedySolution(C) of line 4 of Algorithm 2, for example, is obtained from Algorithm 4 described above by using a setting of $d_{\text{rate}} = 1.0$. In other words, this function generates a solution deterministically. Next, ConstructSolutionProbabilistically(C, S^{bsf}, α_{bsf}) in line 8 of Algorithm 2 executes the solution construction from Algorithm 4 with the following change in function ChooseFrom($\tilde{V}, d_{\text{rate}}$). For being able to select a node from $\tilde{V}$, a probability $\mathbf{p}(v_k)$ is calculated for all $v_k \in \tilde{V}$ in the following way. For this purpose, first, a value $h(v_k)$ is determined for each vertex $v_k \in \tilde{V}$:

$$h(v_k) := \begin{cases} \frac{\alpha_{\text{bsf}}}{\mu(v_k)} & \text{if } c_k \in S^{\text{bsf}} \\ \frac{(1-\alpha_{\text{bsf}})}{\mu(v_k)} & \text{otherwise} \end{cases} \tag{5}$$

Second, the probabilities are defined as follows:

$$\mathbf{p}(v_k) := \frac{h(v_k)}{\sum_{v_l \in \tilde{V}} h(v_l)} \quad \forall\, v_k \in \tilde{V} \tag{6}$$

Subsequently, a node v_j is selected from $\tilde{V}$ based on these probabilities. Recall that the bias toward the best-so-far solution S^{bsf} is controlled by the parameter $\alpha_{\text{bsf}} \in [0, 1]$, where a higher α_{bsf} value increases this bias. Notably, this bias does not exist in standard CMSA.

Finally, in RL-CMSA, solutions are also constructed as shown in Algorithm 4. However, the choice of a node at each construction step (as done by function ChooseFrom($\tilde{V}, d_{\text{rate}}$)) is performed solely based on the quality values in Q (see the

description of RL-CMSA), in the following way. First, with a probability d_{rate}, this selection is done randomly among those nodes (resp., solution components) from $\tilde{V}$ with the highest quality values; otherwise, this is done probabilistically based on the following probabilities p_i for each $v_i \in \tilde{V}$:

$$p_i = \frac{e^{\beta q_i}}{\sum_{v_k \in \tilde{V}} e^{\beta q_k}}. \tag{7}$$

Hereby, $d_{\text{rate}} \in [0, 1]$ and $\beta \geq 0$ are parameters of the solution construction procedure, which balance between the exploration and exploitation of solution components.

Sub-instance Solving

Sub-instances C' are solved in the same way in all three CMSA variants considered in this chapter. To do so, the MIS ILP model is extended by adding the following set of constraints:

$$x_i = 0 \quad \text{for all } c_i \in C \setminus C' \tag{8}$$

Thus, only solution components (or nodes) that are part of the sub-instance C' can be chosen to appear in solutions. Note that the corresponding functions for sub-instance solving in all three CMSA variants return the ILP solver solution in terms of a set S^{SOLVER} of solution components. Moreover, the application of the ILP solver is subject to a time limit of t_{SOLVER} CPU seconds, which means that the returned solution (S^{SOLVER}) is not necessarily an optimal solution to sub-instance C'.

Experimental Evaluation

The experimental evaluation presented in this section includes the following algorithms:

1. Greedy: A greedy heuristic derived by applying the heuristic from Algorithm 4 in a deterministic manner.
2. CPLEX: The application of CPLEX 22.1 to each problem instance using the default parameter settings of CPLEX.
3. CMSA: The application of standard CMSA
4. Adapt-CMSA: The application of the self-adaptive CMSA variant described before.
5. RL-CMSA: The application of the CMSA variant supported by reinforcement learning.

CPLEX 22.1 is executed sequentially (one thread) in standalone mode and within the CMSA variants. For conducting the experiments, we used the IIIA-CSIC in-house high-performance computing cluster of machines equipped with Intel® Xeon® 5670 CPUs having 12 cores of 2.933 GHz and at least 32 GB of RAM.

Benchmark Set

To ensure a controlled experimentation environment, we chose to create a benchmark set featuring graphs of different sizes and densities, utilizing the following three network models:

1. **The Erdös-Rényi model [19]**. Named after mathematicians Paul Erdös and Alfréd Rényi, this model is one of the earliest and simplest models for generating random graphs. Introduced in 1959, this model is defined by two parameters: the number of nodes (n) and the probability (p) that an edge exists between any pair of nodes.
2. **The Watts-Strogatz model [36]**. This model generates small-world networks, that is, networks that feature a small average shortest path length between vertices while maintaining a high degree of local clustering. This model was introduced by Duncan J. Watts and Steven Strogatz in 1998.
3. **The Barabási-Albert model [7]**. Developed by Albert-László Barabási and Réka Albert in 1999, the main characteristic of this preferential attachment model is its ability to produce networks with a scale-free degree distribution. In such networks, the degrees of nodes follow a power-law distribution, where a few nodes have a very high degree while most nodes have much lower degrees.

Thirty graphs were generated using each of the 3 models for each combination of $|V| \in \{500, 1000, 1500, 2000\}$ and 4 distinct graph densities. In Erdős-Rényi graphs, density is controlled by the edge probability (p), in Watts-Strogatz graphs by the parameter (k) and in Barabási-Albert graphs by the parameter (m). Altogether, this benchmark set includes 480 graphs for each network model. We used the `igraph` library's implementations of these graph models.

ILP Solver Parameters

In addition to the CMSA-specific parameters of the three CMSA variants that will be applied to the MIS problem in this chapter, the following parameters related to the utilized ILP solver are used. Note that these parameters are explained in the context of CPLEX version 22.1 which was used for the implementations.

1. $\text{ilp}_{\text{emphasis}} \in \{\texttt{true}, \texttt{false}\}$ is a Boolean parameter indicating whether or not to utilize "heuristic emphasis." Hereby, $\text{ilp}_{\text{emphasis}} = \texttt{false}$ denotes CPLEX's default setting, while $\text{ilp}_{\text{emphasis}} = \texttt{true}$ refers to setting the emphasis parameter to a value of five, indicating the highest heuristic emphasis level in which the solver strongly favors the improvement of its best solution found over bound calculations.

2. $\text{ilp}_{\text{warmstart}} \in \{\texttt{true}, \texttt{false}\}$ is another Boolean parameter indicating whether or not to provide the solver with the best-so-far solution S^{bsf} of CMSA at the start of each application. This is called a "warm start" in the context of CPLEX.
3. $\text{ilp}_{\text{abort}} \in \{\texttt{true}, \texttt{false}\}$ is a third Boolean parameter indicating whether or not to abort CPLEX whenever a solution is found that is better than the current best-so-far solution S^{bsf} of CMSA.

Note that all three CPLEX-related parameters are used in standard CMSA and in RL-CMSA. However, using $\text{ilp}_{\text{warmstart}}$ and $\text{ilp}_{\text{abort}}$ would not make sense in Adapt-CMSA, as in this CMSA variant, it is important to know if a sub-instance can be solved to optimality within the allotted computation time.

Parameter Tuning

All three CMSA variants underwent parameter tuning with the `irace` tool [25]. Specifically, each CMSA variant was tuned once for the complete benchmark set, with an `irace` budget of 3000 algorithm runs. Additional problem instances were created to serve as tuning instances. Specifically, for each combination of $|V|$ and graph density, exactly one tuning instance was generated by each of the 3 network models, resulting in a total of 24 tuning instances. The computation time limit was set to 150 CPU seconds for all graphs with $|V| = 500$, 300 CPU seconds for graphs with $|V| = 1000$, 450 CPU seconds for graphs with $|V| = 1500$, and 600 CPU seconds for graphs with $|V| = 2000$.

The tuning results of the three CMSA variants are shown in Table 2. An interesting observation in examining these parameter settings is that standard CMSA performs best with the lowest possible determinism rate ($d_{\text{rate}} = 0.0$)

Table 2 Parameters, domains, and tuning results. "n.a." is the abbreviation for "not applicable"

Parameter	Domain	CMSA	Adapt-CMSA	RL-CMSA
n_{a}	$\{1, \ldots, 50\}$	17	n.a.	15
age_{max}	$\{1, \ldots, 10\}$	1	n.a.	4
d_{rate}	$[0.0, 0.99]$	0.0	n.a.	0.43
l_{size}	$\{3, \ldots, 100\}$	81	n.a.	n.a.
t_{SOLVER}	$\{1, \ldots, 20\}$	9	20	20
$\text{ilp}_{\text{emphasis}}$	{`true`, `false`}	`false`	`false`	`true`
$\text{ilp}_{\text{warmstart}}$	{`true`, `false`}	`false`	n.a.	`false`
$\text{ilp}_{\text{abort}}$	{`true`, `false`}	`false`	n.a.	`true`
α^{LB}	$[0.6, 0.99]$	n.a.	0.98	n.a.
α^{UB}	$[0.6, 0.99]$	n.a.	0.99	n.a.
α_{red}	$[0.01, 0.1]$	n.a.	0.04	n.a.
t_{prop}	$[0.1, 0.8]$	n.a.	0.54	n.a.
β	$[0.0, 2.0]$	n.a.	n.a.	0.16
b_{reset}	{`true`, `false`}	n.a.	n.a.	`false`
cf_{limit}	$[0.9, 1.0]$	n.a.	n.a.	0.9

for probabilistic solution construction. This sharply contrasts with the settings $\alpha^{\text{LB}} = 0.98$ and $\alpha^{\text{UB}} = 0.99$ in Adapt-CMSA, which cause newly constructed solutions to be heavily biased toward the best-so-far solution S^{bsf}. In contrast to the requirements of CMSA and Adapt-CMSA, RL-CMSA works best with an intermediate setting of $d_{\text{rate}} = 0.43$ concerning the determinism in constructing solutions. Another interesting observation concerns the setting of the ILP solver parameters. Whereas CMSA and Adapt-CMSA do not utilize any of the CPLEX features (such as heuristic emphasis, warm starts, or early termination upon improvement), RL-CMSA appears to gain an advantage from applying heuristic emphasis and the abort feature.

Results

The five algorithmic techniques (Greedy, CPLEX, CMSA, Adapt-CMSA, and RL-CMSA) were applied once to each problem instance from the benchmark set. The computation time limit for CPLEX and the three CMSA variants were the same as that used for tuning (described earlier). The results are presented as box plots in Figs 2, 3 and 4. Note that each network model has exactly one graphic (Erdös-Rényi, Watts-Strogatz, and Barabási-Albert). Each graphic features a 4×4 grid of box plots, where the rows show results (from top to bottom) for graphs of increasing size, and the columns (from left to right) show results for graphs of increasing density.

To facilitate the analysis of the results concerning statistical significance, critical difference (CD) plots are provided separately for the graphs of each of the three network models in Figs 5, 6 and 7. In short, a CD plot displays the average ranking of several algorithms across a set of problem instances, with the whiskers of two algorithms connected by a bold horizontal bar if their results are statistically equivalent. Each CD plot figure consists of five graphics. The first graphic (at the top) presents statistical information for the entire set of graphs corresponding to the respective network model. The remaining four graphics display statistical information for all graphs of a specific density.

When studying the results, the following main observations can be made:

1. First of all, the three CMSA variants clearly outperform Greedy, which is—as expected—for all network models the worst-performing technique.
2. The three CMSA variants also outperform CPLEX with statistical significance. This is with the exception of rather sparse graphs and, in general, Barabási-Albert graphs where CPLEX performs rather strongly. In fact, CPLEX outperforms standard CMSA with statistical significance on Barabási-Albert graphs (see Fig 7a). Remember that these graphs are scale-free; that is, their node degrees follow a power-law distribution. This means that some hubs are strongly connected to other nodes, while many others have only very few connections.
3. When comparing the three CMSA variants, it is clear that RL-CMSA is the best-performing approach across the graphs of all three network models (different sizes and densities). As shown by the CD plots in Figs 5, 6 and 7, there is only one subset of graphs (the densest Erdös-Rényi graphs) for which RL-CMSA is not the

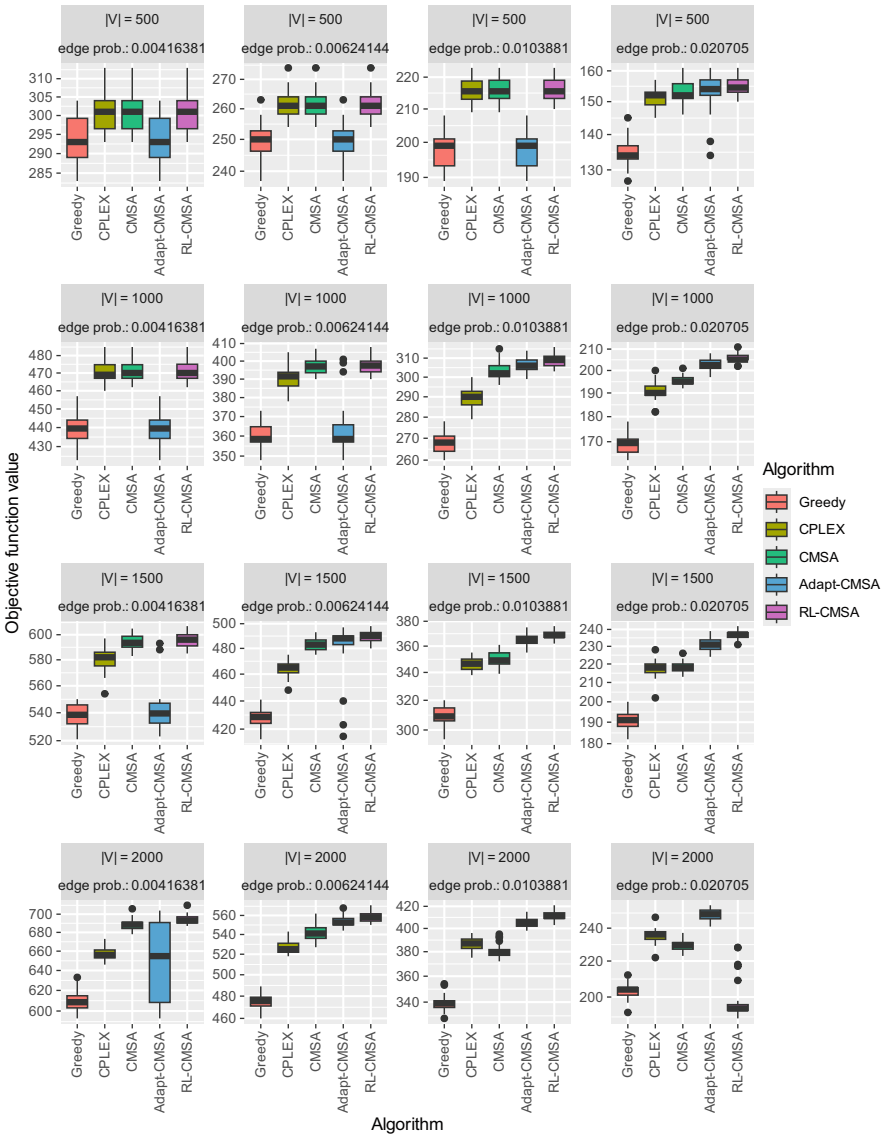


Fig. 2 Results for Erdös-Rényi graphs

best-performing method. This is because, for some reason, RL-CMSA does not work well for the largest and densest Erdös-Rényi graphs (see the bottom-right box plot in Fig 2.

4. Another interesting aspect is that Adapt-CMSA outperforms standard CMSA in the context of Watts-Strogatz and Barabási-Albert graphs. This is with statistical

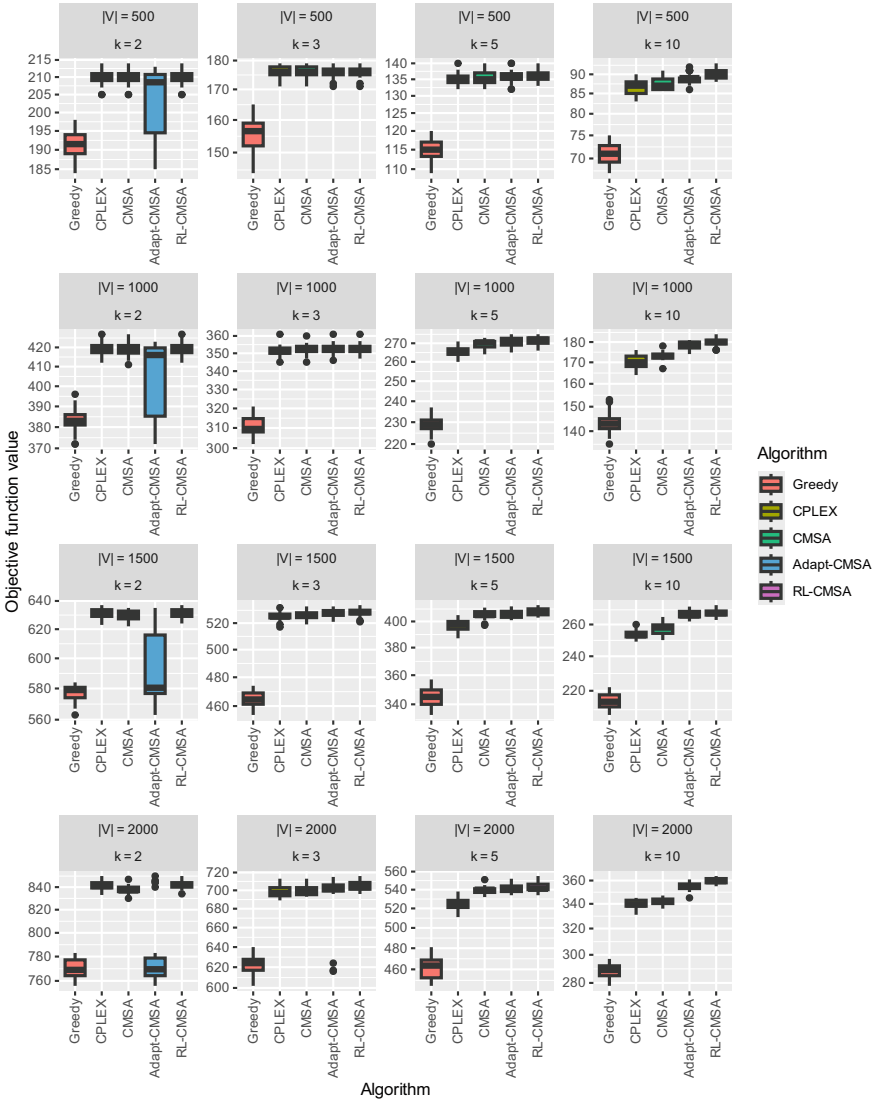


Fig. 3 Results for Watts-Strogatz graphs

significance as shown in Figs 6 and 7. However, in the case of Erdös-Rényi graphs, Adapt-CMSA—when considering all of these graphs—is outperformed by CMSA. When studying the box plots from Fig 2, it becomes clear that this is the case due to a very low performance of Adapt-CMSA for the sparsest graphs (see the boxplots in the first column of that graphic). Remember that the parameter setting of Adapt-CMSA chosen by `irace` was such that the construction of new solutions is very much biased toward the best-so-far solution.

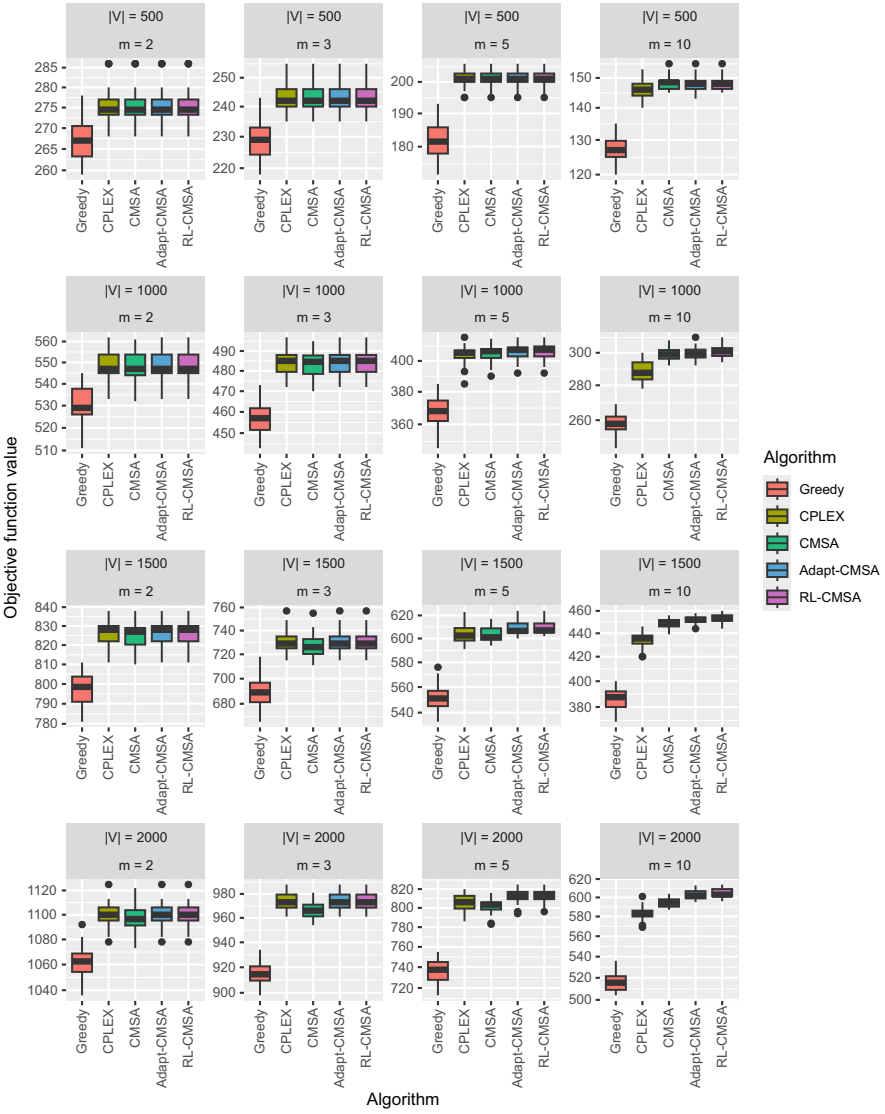


Fig. 4 Results for Barabási-Albert graphs

This setting, even though working very well for the three groups of graphs of higher density (regardless of the network model), does not seem to work well at all when sparse graphs (at least the ones produced by the Erdös-Rényi and the Watts-Strogatz models) are concerned. We assume that, with a rather greedy solution construction, it is very hard to escape from local minima in these cases.

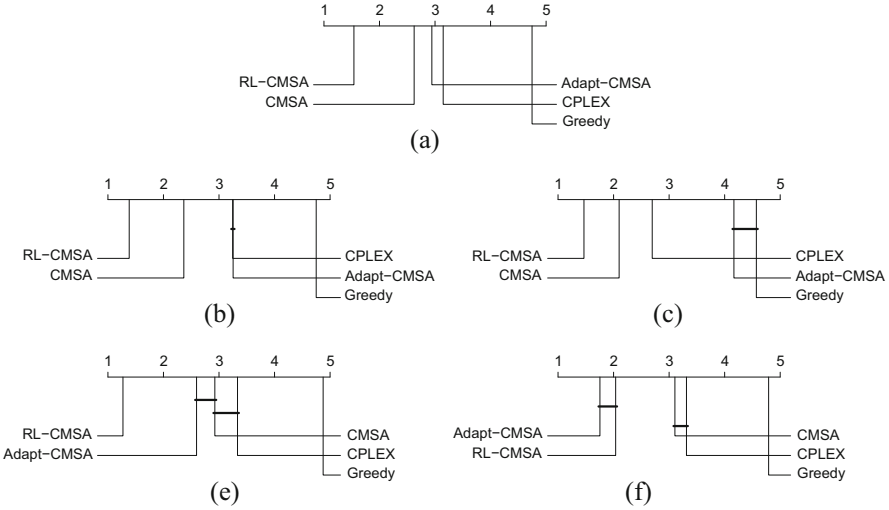


Fig. 5 Critical Difference (CD) plots concerning Erdös-Rényi graphs. (**a**) All graphs. (**b**) Density $p = 0.00624144$. (**c**) Density $p = 0.00416381$. (**d**) Density $p = 0.0103881$. (**e**) Density $p = 0.020705$

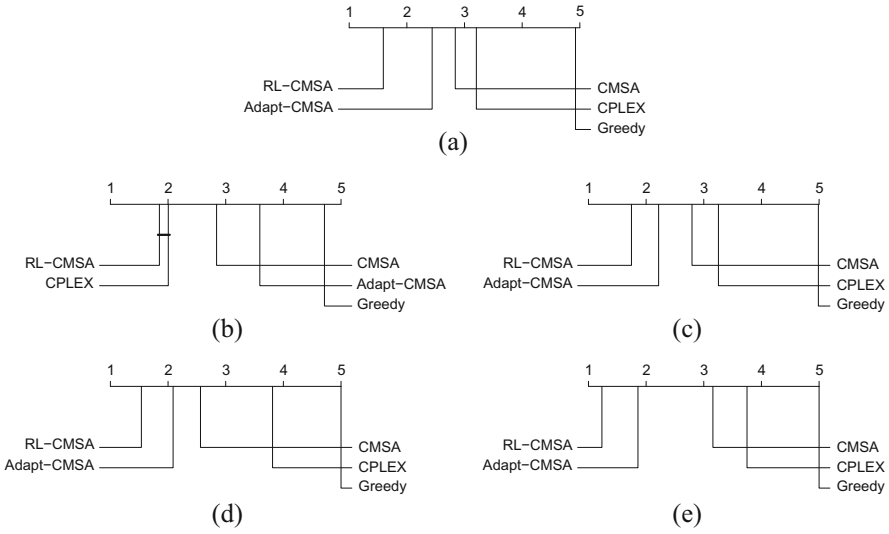


Fig. 6 Critical Difference (CD) plots concerning Watts-Strogatz graphs. (**a**) All graphs. (**b**) Density $k = 2$. (**c**) Density $k = 3$. (**d**) Density $k = 5$. (**e**) Density $k = 10$

Finally, the `STNWeb` tool [33] was used in an attempt to learn something about the changing behavior of the three CMSA variants when applied to graphs created with different network models. This tool produces graphics—called STN graphics—that visualize the trajectories of algorithms in the search space. We

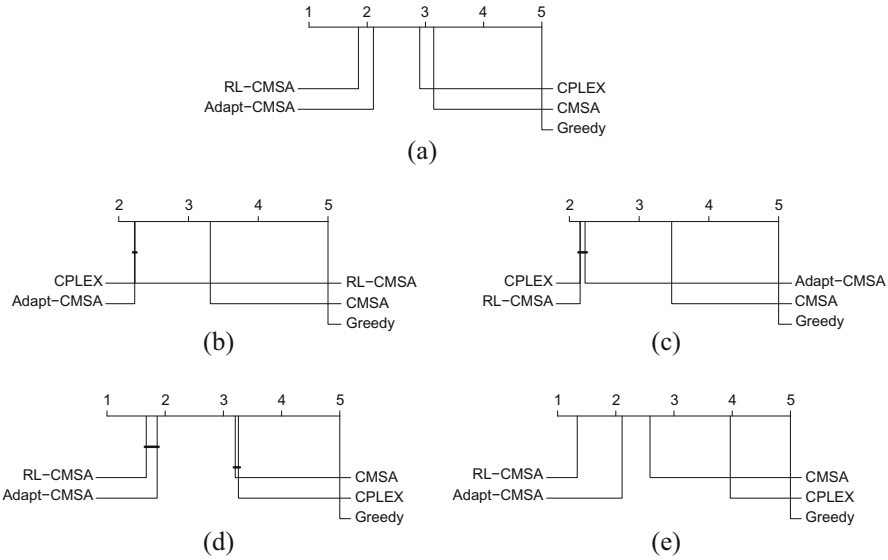


Fig. 7 Critical Difference (CD) plots concerning Barabási-Albert graphs. (**a**) All graphs. (**b**) Density $m = 2$. (**c**) Density $m = 3$. (**d**) Density $m = 5$. (**e**) Density $m = 10$

applied CMSA, Adapt-CMSA, and RL-CMSA ten times to all problem instances and recorded every new best-so-far solution to produce these trajectories. In Fig 8, we show the resulting STN graphics for the first (out of 30) graphs with 2000 nodes from the third density level as a representative example. The three graphics in this figure refer to the corresponding Erdös-Rényi graph (see Fig 8a), the Watts-Strogatz graph (see Fig 8b), and the Barabási-Albert graph (see Fig 8c). Before delving into the analysis, please note that the displayed graphics result from—in STN jargon—search space partitioning, meaning that each node (dot, square, triangle) in these graphics might represent a single solution or clusters of similar solutions. Firstly, it is evident that the STN graphics for the Erdös-Rényi and Watts-Strogatz graphs display far more similarities with each other than they do with the STN graphic for the Barabási-Albert graph. In particular, they show that both CMSA and Adapt-CMSA start their search trajectories in a common area of the search space. However, while some CMSA trajectories are attracted to the same search space region, this does not hold true for Adapt-CMSA. Moreover, they show that RL-CMSA trajectories start in different search space regions. These two observations make sense as CMSA and Adapt-CMSA, in contrast to RL-CMSA, are biased by the same greedy heuristic that favors low-degree nodes. In general, both graphics do not show any overlaps between the search trajectories of different algorithms. This suggests that, in Erdös-Rényi and Watts-Strogatz graphs, good solutions might be very different from each other. In contrast, in the context of the

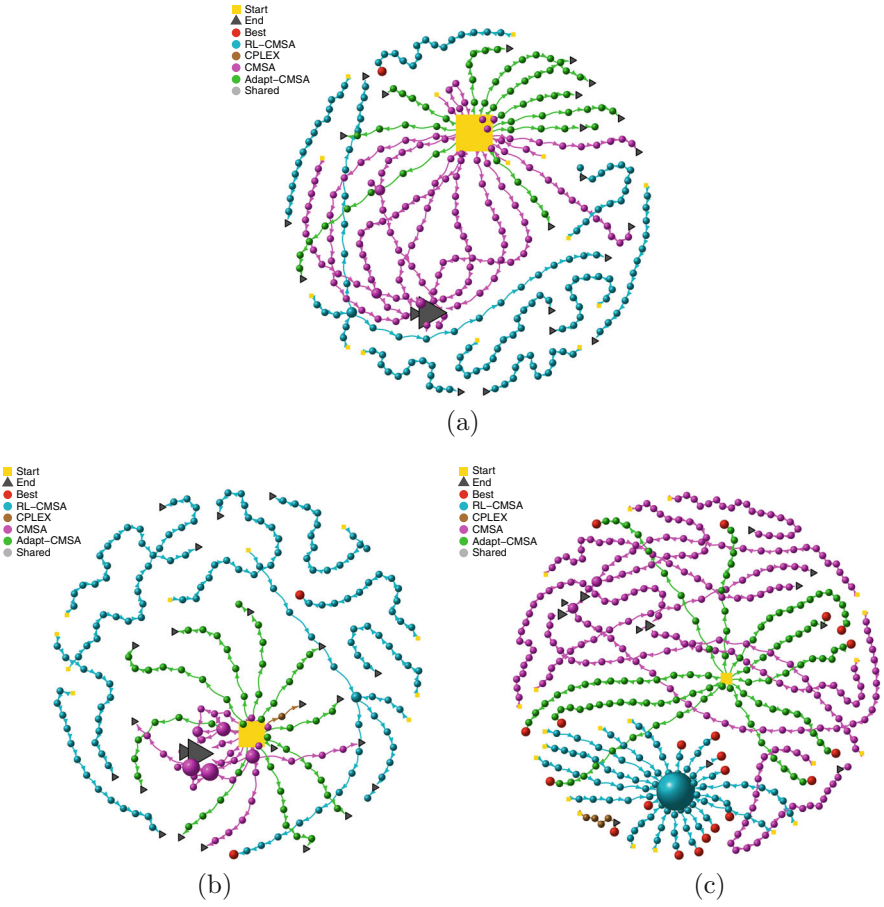


Fig. 8 STNWeb graphics. The three graphics show ten runs of all CMSA variants (in addition to the deterministic CPLEX run) for the first graph with 2000 nodes and the third density level. (**a**) shows results for the corresponding Erdös-Rényi graph, (**b**) for the Watts-Strogatz graph, and (**c**) for the Barabási-Albert graph

Barabási-Albert graph, the search trajectories of RL-CMSA seem to be attracted by a common area of the search space. After passing through this area, all of them find different solutions of the same quality (red dots). This behavior suggests that, in contrast to Erdös-Rényi and Watts-Strogatz graphs, in Barabási-Albert graphs, the MIS problem is characterized by a common backbone of nodes that must form part of high-quality solutions. As RL-CMSA is the only CMSA variant with a learning component, it is also the only one to learn to include the nodes from the backbone into solutions before finding the best solutions in each run.

Conclusion

This chapter introduced three variants of one of the most recent hybrid metaheuristics called "Construct, Merge, Solver & Adapt" (abbreviated as CMSA), which uses problem-specific solvers at each iteration to solve smaller sub-instances of the tackled problem instances. In addition to the standard CMSA variant, we described a self-adaptive CMSA variant called Adapt-CMSA and a CMSA variant called RL-CMSA that uses reinforcement learning. To showcase an application, these three CMSA variants were applied to solve the maximum independent set problem, a classical NP-hard problem in Computer Science. The RL-CMSA variant emerged as the best-performing technique, followed by the Adapt-CMSA variant. However, it was also evident that Adapt-CMSA might have problems escaping from local minima in case of an overly strong bias toward the best-so-far solution. In contrast, RL-CMSA showed a strong performance over the whole benchmark set of graphs produced by different network models.

We believe that one of the main research lines around CMSA in the near future will be concerned with strengthening the learning component of the algorithm. Recent applications of RL-CMSA [29] and Learn-CMSA [28] show the potential of this type of research. Another possibly fruitful research line concerns using specialized exact techniques (instead of black-box ILP solvers) for solving sub-instances.

Cross-References

▸ Matheuristics by Examples

Acknowledgments The authors were supported by grants TED2021-129319B-I00 and PID2022-136787NB-I00 funded by MCIN/AEI/10.13039/501100011033.

Competing Interest Declaration The author(s) has no competing interests to declare that are relevant to the content of this manuscript.

References

1. Akbay MA, Kalayci CB, Blum C (2022) Application of CMSA to the electric vehicle routing problem with time windows, simultaneous pickup and deliveries, and partial vehicle charging. In: Metaheuristics international conference, Springer, pp 1–16
2. Akbay MA, López Serrano A, Blum C (2022) A self-adaptive variant of CMSA: application to the minimum positive influence dominating set problem. Int J Comput Intell Syst 15(1):44
3. Akbay MA, Kalayci CB, Blum C (2023) Application of Adapt-CMSA to the two-echelon electric vehicle routing problem with simultaneous pickup and deliveries. In: European conference on evolutionary computation in combinatorial optimization (part of EvoStar), Springer, pp 16–33
4. Akbay MA, Blum C, Kalayci CB (2024) CMSA based on set covering models for packing and routing problems. Ann Oper Res 343(1):1–38

5. Alves Viana LG (2022) Uma meta-heurística híbrida para o problema de cobertura de discos ponderados. Bachelor's thesis, Universidade Federal de Alagoas, Instituto de Computação, Maceió, Brazil
6. Arora D, Maini P, Pinacho-Davidson P, Blum C (2019) Route planning for cooperative air-ground robots with fuel constraints: an approach based on CMSA. In: Proceedings of the genetic and evolutionary computation conference, pp 207–214
7. Barabási AL, Albert R (1999) Emergence of scaling in random networks. Science 286(5439):509–512
8. Blum C (2020) Minimum common string partition: on solving large-scale problem instances. Int Trans Oper Res 27(1):91–111
9. Blum C (2024) Construct, merge, solve & adapt: a hybrid metaheuristic for combinatorial optimization. Springer Nature, Berlin
10. Blum C, Blesa MJ (2016) Construct, merge, solve and adapt: application to the repetition-free longest common subsequence problem. In: Evolutionary computation in combinatorial optimization: 16th european conference, EvoCOP 2016, Porto, March 30–April 1, 2016, Proceedings 16, Springer, pp 46–57
11. Blum C, Blesa MJ (2018) A comprehensive comparison of metaheuristics for the repetition-free longest common subsequence problem. J Heurist 24(3):551–579
12. Blum C, Gambini Santos H (2019) Generic CP-supported CMSA for binary integer linear programs. In: Hybrid metaheuristics: 11th international workshop, HM 2019, Concepción, Chile, January 16–18, 2019, proceedings 11, Springer, pp 1–15
13. Blum C, Ochoa G (2021) A comparative analysis of two matheuristics by means of merged local optima networks. Eur J Oper Res 290(1):36–56
14. Blum C, Pereira J (2016) Extension of the CMSA algorithm: an LP-based way for reducing sub-instances. In: Proceedings of the genetic and evolutionary computation conference 2016, pp 285–292
15. Blum C, Pinacho P, López-Ibáñez M, Lozano JA (2016) Construct, merge, solve & adapt: a new general algorithm for combinatorial optimization. Comput Oper Res 68:75–88
16. de Oliveira EB, da Silva Batista M, Pinheiro RGS (2023) Uma abordagem híbrida CMSA para o problema da cadeia de caracteres mais próxima. In: Proceedings of the Simpósio Brasileiro de Pesquisa Operacional, vol 55
17. Djukanović M, Kartelj A, Blum C (2023) Self-adaptive CMSA for solving the multidimensional multi-way number partitioning problem. Expert Syst Appl 232:120762
18. Dupin N, Talbi EG (2021) Matheuristics to optimize refueling and maintenance planning of nuclear power plants. J Heurist 27(1–2):63–105
19. Erdös P, Rényi A (1959) On random graphs I. Publ math debrecen 6(290–297):18
20. Ferrer J, Chicano F, Ortega-Toro JA (2021) CMSA algorithm for solving the prioritized pairwise test data generation problem in software product lines. J Heurist 27:229–249
21. Ghirardi M, Salassa F (2022) A simple and effective algorithm for the maximum happy vertices problem. Top 30(1):181–193
22. Hawa A (2020) Exact and evolutionary algorithms for the score-constrained packing problem. PhD thesis, Cardiff University
23. Lewis R, Thiruvady D, Morgan K (2019) Finding happiness: an analysis of the maximum happy vertices problem. Comput Oper Res 103:265–276
24. Lizárraga E, Blesa MJ, Blum C (2017) Construct, merge, solve and adapt versus large neighborhood search for solving the multi-dimensional knapsack problem: which one works better when? In: Evolutionary computation in combinatorial optimization: 17th european conference, EvoCOP 2017, Amsterdam, April 19–21, 2017, Proceedings 17, Springer, pp 60–74
25. López-Ibáñez M, Dubois-Lacoste J, Cáceres LP, Birattari M, Stützle T (2016) The irace package: iterated racing for automatic algorithm configuration. Oper Res Perspect 3:43–58
26. Martí R, Parreño F, Mortes J (2024) Mathematical models and solving methods for diversity and equity optimization. J Heurist 30(5):291–323
27. Pinacho-Davidson P, Bouamama S, Blum C (2019) Application of CMSA to the minimum capacitated dominating set problem. In: Proceedings of the genetic and evolutionary computation conference, pp 321–328

28. Pinacho-Davidson P, Blum C, Pinninghoff MA, Contreras R (2024) Extension of CMSA with a learning mechanism: application to the far from most string problem. Int J Comput Intell Syst 17(1):109
29. Reixach J, Blum C (2024) How to improve "Construct, Merge, Solve and Adapt"? use reinforcement learning! Ann Oper Res In press
30. Rosati RM, Kletzander L, Blum C, Musliu N, Schaerf A (2022) Construct, merge, solve and adapt applied to a bus driver scheduling problem with complex break constraints. In: International conference of the Italian association for artificial intelligence, Springer, pp 254–267
31. Rosati RM, Bouamama S, Blum C (2023) Construct, merge, solve and adapt applied to the maximum disjoint dominating sets problem. In: Di Gaspero L, Festa P, Nakib A, Pavone M (eds) Metaheuristics. Springer International Publishing, Cham, pp 306–321
32. Rosati RM, Bouamama S, Blum C (2024) Multi-constructor CMSA for the maximum disjoint dominating sets problem. Comput Oper Res 161:106450
33. Sartori CC, Blum C, Ochoa G (2023) STNWeb: a new visualization tool for analyzing optimization algorithms. Softw Impacts 17:100558
34. Thiruvady D, Lewis R (2022) Recombinative approaches for the maximum happy vertices problem. Swarm Evolut Comput 75:101188
35. Thiruvady D, Blum C, Ernst AT (2019) Maximising the net present value of project schedules using CMSA and parallel ACO. In: Hybrid metaheuristics: 11th international workshop, HM 2019, Concepción, January 16–18, 2019, Proceedings 11, Springer, pp 16–30
36. Watts DJ, Strogatz SH (1998) Collective dynamics of 'small-world' networks. Nature 393(6684):440–442
37. Zhou J, Zhang P (2024) Simple heuristics for the rooted max tree coverage problem. In: Wu W, Guo J (eds) Combinatorial optimization and applications. Springer Nature Switzerland, Cham, pp 252–264

Part II
Local Search

Constraint-Based Local Search

15

L. Michel and P. Van Hentenryck

Contents

Introduction . . . 390
Getting Started . . . 392
Foundations . . . 395
  Models . . . 395
  Programs . . . 401
Case Studies . . . 406
  Progressive Party . . . 406
  Car Sequencing . . . 410
  Scene Allocation . . . 413
Implementation . . . 416
  Invariants . . . 416
  Differentiation . . . 420
Empirical Results . . . 423
Conclusion . . . 424
Cross-References . . . 425
References . . . 425

Abstract

Constraint-Based Local Search emerged in the last decade as a framework for declaratively expressing hard combinatorial optimization problems and solving them with local search techniques. It delivers tools to practitioners that

L. Michel (✉)
University of Connecticut, Storrs, CT, USA
e-mail: ldm@uconn.edu; ldm@engr.uconn.edu

P. Van Hentenryck
School of Industrial and Systems Engineering, Georgia Institute of Technology, Atlanta, GA, USA
e-mail: pvh@gatech.edu

© Springer Nature Switzerland AG 2025

R. Martí et al. (eds.), *Handbook of Heuristics*,
https://doi.org/10.1007/978-3-032-00385-0_7

enable them to quickly experiment with multiple models, heuristics, and meta-heuristics, focusing on their application rather than the delicate minutiae of producing a competitive implementation. At its heart, the declarative models are reminiscent of the modeling facilities familiar to constraint programming, while the underlying computational model heavily depends on incrementality. The net result is a platform capable of delivering competitive local search solutions at a fraction of the efforts needed with a conventional approach delivering model-and-run to local search users.

Keywords

Local Search · Model · Declarative · Incremental · Solver

Introduction

Complete techniques such as Integer Programming and Constraint Programming typically offer optimality guarantees on the results they deliver but do not always scale to large instances. This explains the appeal of local search methods that deliver a different trade-off in the algorithmic design space, favoring scalability at the expense of guarantees. Local search algorithms apply to diverse application domains such as routing, scheduling, resource allocation, or rostering to name just a few. In most cases, local search techniques scale to truly large problem instances that are often out of scope for complete techniques and are capable of producing streams of solutions of improving quality.

Yet, while modeling and high-level tools are ubiquitous for Mixed -Integer Programming and Constraint Programming, they have been relatively unexplored for local search until recently. This is primarily due to the fact that the separation of models and algorithms is not as simple in local search compared to MIP and CP which are declarative in nature. Indeed, the vast majority of papers discussing local search describe their solution in *algorithmic* terms rather than *declarative* terms like models, decision variables, and constraints. The 90s witnessed a shift in interest and the emergence of multiple tools expressly dedicated to local search techniques. GSAT [13, 15] and WalkSAT [14] offered the initial impetus behind formulating problems with a simple language (in clausal form) and using generic local search algorithms operating on that encoding. Integer Optimization by Local Search [25] generalized this line of work to the richer language of integer programming.

Localizer [8, 9] took a different approach, providing a first step to build a general-purpose modeling language for local search. It introduced the concept of *invariants* to express arbitrary one-way constraints that are automatically and incrementally updated under variable assignments. These invariants can then encode, in a declarative fashion, the incremental data structures typically needed in the implementation of meta-heuristics such as Tabu Search [5], Min-Conflict Search [10], Simulated Annealing [6], Scatter Search [7], or GRASP [4] to name just a few.

Constraint-Based Local Search is the culmination of this line of work. It complements the constraint programming efforts typically focused on complete

search techniques and delivers a "model and search" framework in which one uses decision variables and constraints to model a problem and relies on search procedures to explore the underlying search space. COMET is an optimization platform that embodies Constraint-Based Local Search and is a direct descendant of LOCALIZER. It is an object-oriented programming language with explicit support for modeling problems declaratively and solving them with local search techniques. The contributions underlying COMET span from the incremental computation model, the modeling abstractions, and the control mechanisms to specify and execute local search heuristics and meta-heuristics easily, efficiently, and compactly.

Foundations Constraint-Based Local Search aims at implementing the vision captured by the equation

$$LocalSearch = Model + Search$$

that expresses the belief that a local search algorithm is best viewed as the composition of a *declarative model* with a *search* component. This separation of concerns is central: It postulates the importance of expressing the structures of the problem being solved declaratively and compositionally and providing a search component which *exploits* those structures and guides the search toward high-quality local optima.

The Constraint-Based Local Search architecture delivers several key benefits:

Rich Language Constraints are declarative and capture the problem substructures. They range from simple arithmetic constraints, the indexing of arrays of variables with variables, meta-constraints (constraints on the truth value of other constraints) and logical constraints, to combinatorial constraints such as cardinality, sequence, or alldifferent constraints to name just a few. Constraints (and combinators) for local search were introduced in [23].

Rich Search Programming meta-heuristics is supported by a wealth of language combinators and control abstractions to automate the most tedious and error-prone aspects of an actual implementation. The abstractions foster the decoupling of neighborhood, heuristics, and meta-heuristics specifications while leveraging the incrementality exposed by the constraints present in the declarative model. Control abstractions were introduced in [20].

Separation The untangling of model and search promotes the independent design and evolution of these two components. With Constraint-Based Local Search, it is possible to explore alternative models independently of refinements to the search procedures.

Versatility The ability to specify meta-heuristics orthogonally to the model also enables a collection of *generic* search routines which are highly reusable and promote the experimental process to design an effective CBLS program. This versatility further promotes the reuse of highly generic "canned" search procedures.

Extensibility New constraints and objective functions can be added to the system library and used in conjunction with native constraints. Perhaps even more interestingly, the new constraints can be implemented directly in the host language (i.e., COMET), and the bulk of the implementation is often cast in terms of invariants. New heuristics and meta-heuristics can also be easily added to the system as the implementation of *any* heuristic or meta-heuristic relies on a few key concepts such as closures, continuations, selectors, and neighbors.

Performance Finally, the architecture heavily relies on incrementally maintaining the computational state over time. This capability is compositional and a direct by-product of the declarative model. The approach enables the platform to deliver constraint-based local search *programs* that are a fraction of the size and complexity of handcrafted code, yet deliver performance comparable, and sometime exceeding, manually crafted implementations.

The rest of this chapter starts with an illustration of Constraint-Based Local Search through the modeling and resolution of the classic n-queens problem in section "Getting Started." Section "Foundations" discusses the theoretical underpinnings of Constraint-Based Local Search starting with models and concluding with programs. Section "Case Studies" focuses on how to model applications with a rich language that goes beyond boolean formulas or linear equations. Section "Implementation" explores the implementation issues, starting with the support for incremental computation through invariants and proceeding with a discussion of differentiation. Section "Empirical Results" gives a brief survey of the type of performance that can be expected from Constraint-Based Local Search systems and section "Conclusion" concludes the chapter.

Getting Started

To introduce Constraint-Based Local Search, it is valuable to start with the modeling and resolution of a simple well-known problem. The examples presented in this chapter use the COMET platform.

The Problem The n-queens problem is to place n queens on a chess board of size $n \times n$ so that no two queens can attack each other. While the problem is polynomial, its simplicity is appealing to illustrate Constraint-Based Local Search. The COMET program is shown in Fig. 1 and features two clear components that are discussed next.

The Model The model definition spans lines 4–9, is declarative, and exclusively focuses on defining an array of decision variables $queen$ in line 6 and three combinatorial constraints on lines 7–9. Variable $queen_i$ models the row on which the queen in column i is to be placed. Each decision variable has a domain $Size$ representing the set of permissible values for that variable. Each combinatorial `alldifferent` constraint takes as input an array of expressions and requires that

```
1 import lssolver;
2 int n = 8;
3 range Size = 1..n;
4 Solver<LS> m();
5 ConstraintSystem<LS> S(m);
6 var{int} queen[Size](m,Size);
7 S.add(alldifferent(queen));
8 S.add(alldifferent(all(i in Size) (queen[i] + i)));
9 S.add(alldifferent(all(i in Size) (queen[i] - i)));
10
11 while(S.violations() > 0)
12   selectMin(i in Size,v in Size: queen[i]!=v)(S.getAssignDelta(queen[i],v))
13     queen[i] := v;
```

Fig. 1 A constraint-based local search model for the queens problem

the values of all entries in the array be pairwise distinct. For instance, line 7 states that no two queens can be assigned to the same row. The constraints on lines 8 and 9 play a similar role but for the upward and downward diagonals. Note how an operator `all` is used on line 8 to create an array of expressions

$$(queen_1 + 1, queen_2 + 2, queen_3 + 3, \cdots, queen_7 + 7, queen_8 + 8),$$

which is the input of the combinatorial constraint.

The Search A classic constraint-based local search starts from a tentative assignment of values to the decision variables and iteratively transforms it, moving from tentative assignments to tentative assignments using a *local* move operator. The local move used in this specific case is the assignment of a single variable to a new value. Namely, with n decision variables, each with a domain of size n, one could consider up to $\Theta(n^2)$ such moves. To illustrate the search process, consider the left board in Fig. 2a which depicts a tentative assignment. The bottom row of numbers conveys, for each column, the number of constraints violated by the queen placement in that column. For instance, the value in column 2 reports the number of constraints violated by the queen in that column (on row 4). Namely, the number of violations attributed to $queen_2$ is 2 since there is a queen on the same row (violating the row constraint) and a queen on the same up diagonal (in the next column). The numbers in the last column focus on $queen_4$ and therefore are replicas of column 4 of the delta matrix. Namely, they show how the total number of violations would change if $queen_4$ was to be reassigned to another row. Naturally, the fourth value in that column (from the bottom) is zero since it amounts to not moving the queen at all. The third value is -2 since $queen_4$ currently violates three constraints (one horizontal as well as one up and one down diagonal), and the move to row 3 would eliminate all three conflicts (-3) but introduce a new one with the queen in column 8 currently placed on row 3 as well for a net violation gain of -2. Finally, the 7 in

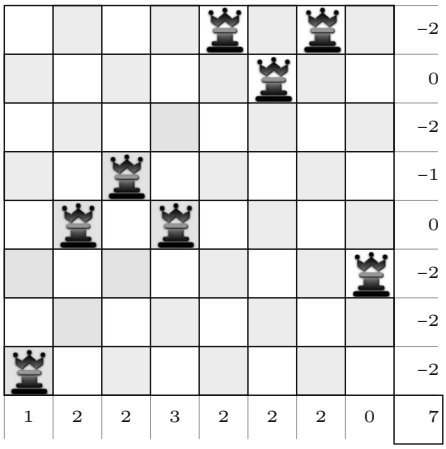


Fig. 2 The first step of the CBLS algorithm for the queens problem. (**a**) Starting point. (**b**) Delta matrix

the bottom-right corner is the overall violations for the entire board. Indeed, there are two rows with more than one queen, three upward diagonals with more than one queen and two downward diagonals ($7 = 2 + 3 + 2$). Here, a move consists of relocating a queen to a new row, and the algorithm chooses the best such move. The right side, Fig. 2b, shows the reduction in violations for each possible move. Therefore, a viable move is to relocate $queen_4$ to row 3, reducing the total number of conflicts (violations) from 7 to 5. The delta matrix shows that quite a few moves would deliver a -2 gain.

The search outlined above is a classic greedy search. The implementation spanning lines 11–14 obtains the set of constraints present in the model (line 11) and starts from a randomly initialized tentative assignment σ_0. It proceeds through a sequence of moves aimed at satisfying the three softened constraints present in S. The underlying neighborhood function N defined as

$$N(\sigma) = \{\sigma' \mid \exists j \in 1..8\ \forall i \in 1..8, i \neq j : \sigma'(queen_i) = \sigma(queen_i) \wedge \sigma'(queen_j) \in D(queen_j) \setminus \{\sigma(queen_j)\}\}$$

spells out a set of $\Theta(n^2)$ neighbors and is implemented with the `select` statement on line 12 that considers the reassignment of a single decision variable at a time. Overall, the search produces a sequence of tentative assignments

$$\sigma_0, \sigma_1 \in N(\sigma_0), \sigma_2 \in N(\sigma_1), \cdots, \sigma_k \in N(\sigma_{k-1})$$

delivering a solution σ_k that satisfies all the softened constraints (the number of violations of S is 0).

Foundations

This section reviews the foundations of Constraint-Based Local Search, covering both models and programs. It introduces the main concepts behind the declarative and operational models.

Models

From an end-user perspective, constraints and objective functions are at the heart of Constraint-Based Local Search. They provide the declarative bricks needed to construct models describing requirements and properties of the solution being sought. They also support the underlying computational model. This section first reviews the key concepts of constraints, and objective functions before finishing with a presentation of the underlying semantics.

Basics Constraint-Based Local Search is the process of looking for an assignment of values to decision variables that meets specific requirements. Decision variables are central to the modeling process as they characterize solutions. This chapter focuses on integer variables only for simplicity. Extensions to more complex variables (i.e., set, paths, and trees) have been proposed in [12, 22]. The chapter also assumes that Constraint-Based Local Search models are defined over a set X of decision variables.

Definition 1 (Assignment). An assignment σ is a mapping $X \rightarrow D(X)$ from variables to values in their domain. The value assigned to x in σ is denoted by $\sigma(x)$. The set of all possible assignments is denoted by Σ.

For convenience, the expression $\sigma[x/v]$ denotes a new assignment σ' which is similar to σ except that variable x is assigned to v, i.e.,

$$\forall y \in X \setminus \{x\} \ : \ \sigma'(y) = \sigma(y) \wedge \sigma'(x) = v.$$

Constraints are used to impose requirements on the decision variables. Constraints are naturally declarative, can be expressed in a variety of ways, and always capture a relation over a subset of variables from X.

Definition 2 (Constraint). A constraint $c(x_1, \cdots, x_n)$ is a n-ary relation over variables $x_1 \cdots x_n \in X$. The set $vars(c)$ is the set of n variables appearing in c, i.e., $vars(c) = \{x_1, \cdots, x_n\}$.

Constraints can be expressed through algebraic expressions, logical statements, or combinatorial structures.

Example 1. In the queens example, the requirement that any two variables cannot lay on the same row, i.e., `alldifferent(x)`, is semantically equivalent to the conjunction of constraints

$$\bigwedge_{i \in 1..8, j \in i+1..8} x_i \neq x_j.$$

Operationally however, the `alldifferent` maintains its state and violations more efficiently than the naive reformulation above.

Example 2. A constraint system S is a set of constraints, and its truth value is equivalent to the truth value of $\wedge_{c \in S}\ c$.

Evaluations and Violations The driving force behind Constraint-Based Local Search rests on the ability to assess how badly constraints are violated. This is captured by the concept of violation degrees (or violations for short).

Definition 3 (Violation Degree). The violation degree of $c(x_1, \cdots, x_n)$ is a function $v_c : \Sigma \to \mathbb{R}^+$ such that, for any assignment σ such that $\sigma(x_1) \in D(x_1)$, $\cdots, \sigma(x_n) \in D(x_n)$, it holds that

$$c(\sigma(x_1), \cdots, \sigma(x_n)) \equiv v_c(\sigma) = 0.$$

The violation degree definition depends critically on the structure conveyed by constraint c. The definition of violation is constraint-dependent but is derived systematically for algebraic and logical expressions. For completeness, consider the specification of expression evaluation.

Definition 4 (Expression Evaluation). Let $e \in \mathcal{E}$ be an arithmetic expression and $\sigma \in \Sigma$ be an assignment. The evaluation of e with respect to (wrt) σ is specified by the function $\mathbb{E}(\sigma, e) : \mathcal{E} \times \Sigma \to \mathbb{Z}$, which is defined inductively on the structure of e in Fig. 3.

Algebraic *constraints* are based on relations and logical combinators. To reason about the violations of relational and logical constraints, it is useful to derive an *expression* modeling the violations of constraint c from the structure of c.

Fig. 3 The evaluation of an expression with respect to an assignment

$$\begin{aligned}
\mathbb{E}(\sigma, v) &= v \Leftrightarrow v \in \mathbb{R} \\
\mathbb{E}(\sigma, x) &= \sigma(x)(x \in X) \\
\mathbb{E}(\sigma, e_1 + e_2) &= \mathbb{E}(\sigma, e_1) + \mathbb{E}(\sigma, e_2) \\
\mathbb{E}(\sigma, e_1 - e_2) &= \mathbb{E}(\sigma, e_1) - \mathbb{E}(\sigma, e_2) \\
\mathbb{E}(\sigma, e_1 \times e_2) &= \mathbb{E}(\sigma, e_1) \times \mathbb{E}(\sigma, e_2) \\
\mathbb{E}(\sigma, \textsc{Abs}(e)) &= \textsc{Abs}(\mathbb{E}(\sigma, e)) \\
\mathbb{E}(\sigma, \min(e_1, e_2)) &= \min(\mathbb{E}(\sigma, e_1), \mathbb{E}(\sigma, e_2))
\end{aligned}$$

Definition 5 (Constraint Conversion). Let c be a logical or relational constraint. The constraint conversion of c is an expression $\mathbb{V}(c)$ which can be evaluated to determine the violations of c with respect to some assignment. It is specified by the function $\mathbb{V}(c) : \mathcal{E} \to \mathcal{E}$ which is defined by induction on the structure of c as specified in Fig. 4.

The conversion for the conjunction of two constraints is none other than the sum of the converted relation violations.

Example 3. The conversion of $x = 5 \wedge y \neq 10$ where $x, y \in X$ is derived as follows:

$$\mathbb{V}(x = 5 \wedge y \neq 10) = \mathbb{V}(x = 5) + \mathbb{V}(y \neq 10) = \textsc{Abs}(x-5)+1-\min(1, \textsc{Abs}(y-10)).$$

It is now possible to compose both functions to obtain the actual violations of an algebraic constraint c with respect to an assignment σ.

Example 4. The violation of an arithmetic constraint $c \equiv l \geq r$ is given by

$$\mathbb{E}(\sigma, \mathbb{V}(l \geq r)).$$

The violation degree function for c is then given by

$$v_c(\sigma) = \mathbb{E}(\sigma, \max(r - l, 0)) = \max(\mathbb{E}(\sigma, r) - \mathbb{E}(\sigma, l), 0).$$

Example 5. The violation of a conjunction $c = c_1 \wedge c_2$ is defined as

$$v_c(\sigma) = \mathbb{E}(\sigma, \mathbb{V}(c_1 \wedge c_2)).$$

While the above mechanics are appropriate to obtain the violations of algebraic constraints, combinatorial constraints define their own notion of violations that capture the combinatorial substructure at hand. Consider the `alldifferent` constraint again.

Example 6. The violation function of $c = \texttt{alldifferent}(x_1, \cdots, x_n)$ simply counts the number of values used more than once by variables $x_1, \ldots, x_n$. More precisely, given an assignment σ and $vars(c) = \{x_1, \cdots, x_n\}$,

Fig. 4 The constraint conversion function

$$\begin{aligned}
\mathbb{V}(e_1 = e_2) &= \textsc{Abs}(e_1 - e_2) \\
\mathbb{V}(e_1 \leqslant e_2) &= \max(e_1 - e_2, 0) \\
\mathbb{V}(e_1 \geqslant e_2) &= \max(e_2 - e_1, 0) \\
\mathbb{V}(e_1 \neq e_2) &= 1 - \min(1, \textsc{Abs}(e_1 - e_2)) \\
\mathbb{V}(r_1 \wedge r_2) &= \mathbb{V}(r_1) + \mathbb{V}(r_2) \\
\mathbb{V}(r_1 \vee r_2) &= \min(\mathbb{V}(r_1), \mathbb{V}(r_2)) \\
\mathbb{V}(\neg r) &= 1 - \min(1, \mathbb{V}(r))
\end{aligned}$$

$$v_c(\sigma) = \sum_{i \in S} \max(0, |\{x_j \in vars(c) | \sigma(x_j) = i\}| - 1),$$

where $S = \bigcup_{i \in 1..n} D(x_i)$. Consider the `alldifferent` constraint on the rows for the board in Fig. 2a. The assignment is $\sigma = [1, 4, 5, 4, 8, 7, 8, 3]$: It has two variables using value 4 and two variables using value 8, giving a violation of 2 for the row constraints.

Differentiation The infrastructure covered so far enables the incremental assessment of the satisfaction, and the number of violations, of a constraint c. However, it does not specify how to assess the impact of a local move on the satisfiability or violations of constraints. *Gradients* are the cornerstone of this process.

Definition 6 (Gradient). Given an assignment σ and an arithmetic expression e, $\uparrow_x(\sigma, e)$ denotes the maximum increase for the evaluation of e over all possible values in $D(x)$ wrt σ, whereas $\downarrow_x(\sigma, e)$ denotes the largest decrease for the evaluation of e over all possible values in $D(x)$ wrt σ, i.e.,

$$\uparrow_x (\sigma, e) = \max_{v \in D(x)} \mathbb{E}(\sigma[x/v], e) - \mathbb{E}(\sigma, e)$$

$$\downarrow_x (\sigma, e) = \mathbb{E}(\sigma, e) - \min_{v \in D(x)} \mathbb{E}(\sigma[x/v], e).$$

Note that gradients are nonnegative in this specification. An efficient implementation can be derived inductively on the structure of expression e. Figure 5 gives an abridged version of such derivation. The definition for $\uparrow_x(\sigma, x)$ is an interesting

$$\begin{aligned}
\uparrow_x (\sigma, v) &= 0 \Leftrightarrow v \in \mathbb{R} \\
\uparrow_x (\sigma, y) &= 0 \Leftrightarrow y \in X \backslash \{x\} \\
\uparrow_x (\sigma, x) &= \max_{v \in D(x)} v - \sigma(x) \\
\uparrow_x (\sigma, e_1 + e_2) &= \uparrow_x (\sigma, e_1) + \uparrow_x (\sigma, e_2) \\
\uparrow_x (\sigma, e_1 - e_2) &= \uparrow_x (\sigma, e_1) + \downarrow_x (\sigma, e_2) \\
\uparrow_x (\sigma, \text{ABS}(e)) &= \max(\text{ABS}(\mathbb{E}(\sigma, e) + \uparrow_x (\sigma, e)), \text{ABS}(\mathbb{E}(\sigma, e) - \downarrow_x (\sigma, e))) - \mathbb{E}(\sigma, \text{ABS}(e)) \\
\uparrow_x (\sigma, \min(e_1, e_2)) &= \min(\mathbb{E}(\sigma, e_1) + \uparrow_x (\sigma, e_1), \mathbb{E}(\sigma, e_2) + \uparrow_x (\sigma, e_2)) - \mathbb{E}(\sigma, \min(e_1, e_2))
\end{aligned}$$

$$\begin{aligned}
\downarrow_x (\sigma, v) &= 0 \Leftrightarrow v \in \mathbb{R} \\
\downarrow_x (\sigma, y) &= 0 \Leftrightarrow y \in X \backslash \{x\} \\
\downarrow_x (\sigma, x) &= \sigma(x) - \min_{v \in D(x)} v \\
\downarrow_x (\sigma, e_1 + e_2) &= \downarrow_x (\sigma, e_1) + \downarrow_x (\sigma, e_2) \\
\downarrow_x (\sigma, e_1 - e_2) &= \downarrow_x (\sigma, e_1) + \uparrow_x (\sigma, e_2) \\
\downarrow_x (\sigma, \text{ABS}(e)) &= \texttt{if } \mathbb{E}(\sigma, e) \geqslant 0 \\
&\quad \texttt{then } \min(\mathbb{E}(\sigma, e), \downarrow_x (\sigma, e)) \\
&\quad \texttt{else } \min(-\mathbb{E}(\sigma, e), \uparrow_x (\sigma, e)) \\
\downarrow_x (\sigma, \min(e_1, e_2)) &= \mathbb{E}(\sigma, \min(e_1, e_2)) - \min(\mathbb{E}(\sigma, e_1) + \downarrow_x (\sigma, e_1), \mathbb{E}(\sigma, e_2) + \downarrow_x (\sigma, e_2))
\end{aligned}$$

Fig. 5 Expression gradients

base case. Indeed, $\uparrow_x(\sigma, x) = \max_{v \in D(x)} v - \sigma(x)$, which has the effect of picking up the largest increase as the distance between the value currently assigned to x in σ and the largest value of the domain. Similarly, note how $\uparrow_x(\sigma, e_1 - e_2)$ combines the largest increase in e_1 with the largest decrease in e_2.

The next concept, variable violation, is interesting: It captures how many violations can be attributed to a specific variable for a given assignment. Variable violations are specified in terms of gradients.

Definition 7 (Variable violations). Given a constraint c, the variable violations of c wrt $x \in vars(c)$ and assignment σ is specified by $\downarrow_x(\sigma, \mathbb{V}(c))$.

The concepts of violation degrees and variable violations are generic, and hence they enable the specification of a common API for all constraints. This API makes it possible to implement the slogan

$$LocalSearch = Model + Search$$

mentioned in the introduction. In particular, the API of a constraint is specified in Fig. 6. For instance, the method call `violations()` simply returns the evaluation of $v_c(\sigma)$, while the method call `violations(x)` returns $\downarrow_x(\sigma, \mathbb{V}(c))$, for the current variable assignment σ. Finally, the call to method `getAssignDelta(x,v)` returns the variation in violation degree when using the assignment $\sigma[x/v]$ instead of σ, i.e., it returns

$$v_c(\sigma) - v_c(\sigma[x/v]).$$

Combinatorial constraints like `alldifferent` also conform to this interface, and the implementation of the last two methods takes advantage of the constraint semantics to implement the specification incrementally.

Objective Functions Objective functions play an equally critical role within Constraint-Based Local Search. Objectives provide the necessary mechanics to express and exploit objective functions. Interestingly, once gradients are available, objectives do not add much complexity. Objective functions must conform to the interface depicted in Fig. 7.

```
1 interface Constraint<LS> {
2   bool holds();
3   var{int} violations();
4   var{int} violations(var{int} x);
5   int getAssignDelta(var{int} x,int v);
6 }
```

Fig. 6 The constraint interface

```
1 interface Objective<LS> {
2   var{int} evaluation();
3   var{int} increase(var{int} x);
4   var{int} decrease(var{int} x);
5   int getAssignDelta(var{int} x,int v);
6 }
```

Fig. 7 The objective interface

Given the current variable assignment σ, the call `evaluation()` returns $\mathbb{E}(\sigma, e)$, while the call `increase(x)` returns $\uparrow_x(\sigma, e)$ and the call `decrease(x)` returns $\downarrow_x(\sigma, e)$.

Models and Constraint Hardness From a purely pragmatic and operational standpoint, it is often convenient to partition the actual constraint set C in two

$$C = S \cup R \quad (S \cap R = \varnothing),$$

where R represents a set of *required* but easy-to-solve constraints and S represents a set of *softened* constraints that are typically much harder to satisfy. The intent is to handle both types of constraints differently. Intuitively, required constraints are always satisfied during the search, while softened constraints may be violated. In the n-queens examples introduced earlier, $R = \varnothing$ and all the constraints are softened in S. In general, however, R may contain some constraints that are not worth relaxing in S. The membership in S or R is clearly a design decision for the modeler to consider.

Definition 8 (Constraint Model). A constraint-based local search model M for a constraint optimization problem is a quintuplet $M = \langle X, D, F, R, S \rangle$, where:

- Every $x \in X$ is a decision variable taking its value from $D(x)$.
- F is, without loss of generality, a minimization function.
- R is a set of required constraints (easy to satisfy).
- S is a set of soft constraints (difficult to satisfy).

Declaratively, the semantics of a Constraint-Based Local Search model is given by the optimization problem

$$\min_{\sigma \in \Sigma} \mathbb{E}(\sigma, F) + \sum_{c \in S} v_c(\sigma)$$

subject to

$$v_c(\sigma) = 0 \quad : \forall c \in R.$$

Namely, the objective is to minimize the sum of the violations over the softened constraints and the original objective function subject to the required constraints. In

the case of constraint satisfaction, $\mathbb{E}(\sigma, F) = 0$ for every σ as the objective is absent. Note that constraints can be weighted, which makes it possible to balance the two terms of the objective. It is simple to write a combinator to weight a constraint in a generic way [22].

Definition 9 (Feasible Solution). A feasible solution σ to a Constraint-Based Local Search model $M = \langle X, D, F, R, S \rangle$ satisfies

$$\sum_{c \in S} \nu_c(\sigma) = 0.$$

$\mathcal{F}(M)$ denotes the set of feasible solutions to M.

Note that, by definition, every assignment σ also satisfies all the required constraints ($\nu_c(\sigma) = 0 \ : \ \forall c \in R$).

Definition 10 (Optimal Solution). An optimal solution σ^* to a Constraint-Based Local Search model $M = \langle X, D, F, R, S \rangle$ is a feasible solution $\sigma^* \in \mathcal{F}(M)$ such that

$$\forall \sigma \in \mathcal{F}(M) : \mathbb{E}(\sigma^*, F) \leq \mathbb{E}(\sigma, F).$$

Definition 11 (Search Procedure). A search procedure produces a sequence of assignments $\sigma_0, \cdots, \sigma_k$, where $\forall i \in 0..k \ : \ \nu_c(\sigma_i) = 0 \ (c \in R)$, and returns σ satisfying

$$\min_{\sigma \in \{\sigma_0, \cdots, \sigma_k\}} \mathbb{E}(\sigma, F) + \sum_{c \in S} \nu_c(\sigma).$$

A Constraint-Based Local Search procedure succeeds if $\sigma \in \mathcal{F}(M)$.

Programs

The model specifies the properties satisfied by assignments appearing in a trace $s_0, s_1, \cdots, s_k$; it does not dictate how the assignments in the trace are generated. This is the prerogative of concrete search procedures which are often viewed as the composition of a *neighborhood* function, a *legality* restriction function, and a candidate *selection* function.

Definition 12 (Neighborhood). The neighborhood of an assignment σ_k, denoted $N(\sigma_k)$, is the set of assignments reachable from σ_k via a local move.

Local moves can be *microscopic* changes to the candidate solution such as the reassignment of a single variable or the swap of the values associated with

two distinct variables. In specific domains, e.g., in scheduling, the move can be *macroscopic* and involve changing several variables to capture moves in more complex neighborhood, e.g., moving a task in a job sequence.

Example 7. In the 8-queens example outlined in section "Getting Started," the neighborhood is based on the reassignment of a single queen to a new row. The neighborhood consists of $\Theta(n^2)$ assignments. It could be further restricted to $\Theta(n)$ assignments by only considering the queen appearing in the largest number of conflicts and its possible reassignments. Given a model $M = \langle X, D, 0, \varnothing, S\rangle$, where S is the set consisting of three softened `alldifferent` constraints, the quadratic neighborhood function is

$$N(\sigma_k) = \{\sigma_k[x/v] \mid x \in X \wedge v \in D(x) \setminus \{\sigma_k(x)\}\},$$

while the linear neighborhood function is

$$N(\sigma_k) = \left\{\sigma_k[x/v] \mid x \in \arg\max_{y \in X} \downarrow_y (\sigma_k, \mathbb{V}(\bigwedge_{c \in S} c)) \wedge v \in D(x) \setminus \{\sigma_k(x)\}\right\}.$$

Definition 13 (Legal neighbors). The legal neighborhood of an assignment σ_k is a restriction of $N(\sigma_k)$, namely $L(N(\sigma_k), \sigma_k) \subseteq N(\sigma_k)$, and the legal subset is required to at least satisfy all the required constraints R.

Note that the legal subset of $N(\sigma_k)$ may even be more restrictive and reject neighbors that are feasible with respect to R but fail to exhibit other characteristics. The *full* characterization of the function L is part of the definition of a meta-heuristic. For instance, the tabu meta-heuristic excludes neighbors that were seen in the recent past.

Definition 14 (Neighbor Selection). The selection function S is responsible for choosing the next assignment among legal neighbors. Namely

$$\sigma_{k+1} = S(L(N(\sigma_k), \sigma_k), \sigma_k) \in L(N(\sigma_k), \sigma_k).$$

For instance, a greedy selection chooses the best neighbor, i.e.,

$$\sigma_{k+1} \in \arg\min_{\sigma \in N(\sigma_k)} \mathbb{E}(\sigma, \mathbb{V}(S)).$$

A heuristics or meta-heuristics specifies the three functions N, L, and S that parameterize the computation model.

Control Primitives To support these abstractions, Constraint-Based Local Search programs rely on a handful of control primitives designed to automate tedious and error-prone implementation details.

While the specification of a program relies on three distinct functions N, L, and S, any implementation concerned with performance produces code that fuses the three functions often degrading the readability and maintainability in the process. Key considerations such as randomization and tiebreaking add another layer of complexity to the code. The implementation of meta-heuristics induces its share of complexity by refining move legality and selection further.

COMET attempts to strike a delicate balance between efficiency, code readability, and ease of maintenance by decoupling these aspects as much as possible. The language introduces *selectors*, *neighbors*, and *randomized choosers* as control abstractions.

Neighborhood Selectors The concept of *neighborhood selector* is a cornerstone for the specification of complex local searches. A simplified version of its interface is presented below:

```
1 interface Neighborhood {
2   void insert(int q,Closure c);
3   boolean hasMove();
4   Closure getMove();
5 }
```

The key idea is that a neighbor is defined as a pair $\langle q, c\rangle$ consisting of a quality measure q and a closure c. The intuition is that, in general, the closure c defines the move which, when executed, will produce an objective value of q for the resulting assignment. Method `insert` adds a neighbor c of quality q (line 2), method `hasMove` checks whether the neighborhood is non-empty, and method `getMove` returns the selected move.

Concrete implementations of this interface commit to a specific selection policy. For instance, the concrete selector `MinNeighborSelector` implements the neighborhood interface and retains only a neighbor minimizing the quality measure. Namely, if the set of inserted neighbors is $N = \{\langle q_1, c_1\rangle, \cdots, \langle q_n, c_n\rangle\}$, the selector retains *one* neighbor

$$\langle q, c\rangle \in \underset{\langle q_j, c_j\rangle \in N}{\arg\min}\ q_j$$

and produces it as its selection when `getMove` is called. Alternative selector includes `KMinNeighborSelector` that returns one of the top-k best neighbors (k being a parameter of the selector).

The `neighbor` *Control Primitive* Neighborhood selectors work in conjunction with the `neighbor` control primitive. The primitive has the following syntax:

`neighbor(`$\langle expr\rangle, \langle N\rangle$`)` $\langle Body\rangle$

where $\langle expr \rangle$ refers to an arbitrary arithmetic expression, $\langle N \rangle$ is an expression referring to a selector, and $\langle Body \rangle$ is a statement (or block of statements). From a semantics standpoint, the control primitive creates a closure c of the body code responsible for producing the assignment σ_{k+1} from the current assignment σ_k. It then associates the closure with a quality measure q and submits the pair $\langle q, c \rangle$ to the selector denoted by N. What makes `neighbor` particularly attractive is the syntactic closeness between the code specifying the quality of the move and the code carrying out the move even if, in practice, there is a strong temporal disconnection between the time when the closure is recorded with the selector and the time it gets executed.

The search procedure in Fig. 8 illustrates an alternative implementation of the greedy search presented in Fig. 1. The loop on lines 3–4 scans all variables and values accumulating the reassignments in the selector N and associating with each one a quality measure based on the delta. The 1-instruction block `queen[i] := v;` on line 4 is automatically wrapped in a closure that is entrusted by `neighbor` to the selector N. The beauty in the construction lies in the ability to easily accumulate in the same selector the union of multiple neighborhoods, all defined with different move operators. This capability will be illustrated in the Progressive Party Problem use case in section "Progressive Party."

Randomized Choosers To support developers, COMET provides control abstractions to make greedy, semi-greedy, and randomized choices. The abstractions encapsulate the necessary state to deliver independent pseudorandom streams in each such instruction appearing in the program. For instance, the $\Theta(n)$-sized neighborhood that considers reassigning the queen with the most violations to a new row is easily modeled with instructions in lines 3–7 below.

```
1 MinNeighborSelector N();
2 while(S.violations()!=0) {
3    selectMax[2](i in Size)(S.violations(queen[i])) {
4       forall(v in queen[i].getDomain() : queen[i] != v)
5          neighbor(S.getAssignDelta(queen[i],v),N)
6             queen[i] := v;
7    }
8    if (N.hasMove()) call(N.getMove());
9 }
```

```
1 MinNeighborSelector N();
2 while(S.violations()!=0) {
3    forall(i in Size,v in queen[i].getDomain() : queen[i] != v)
4       neighbor(S.getAssignDelta(queen[i],v),N) queen[i] := v;
5    if (N.hasMove()) call(N.getMove());
6 }
```

Fig. 8 An alternative greedy search for n-queens

The bracketed 2 on line 3 simply requests the selector to retain any value i among the best two (according to the violation measure).

Discussion The Constraint-Based Local Search model appearing in Fig. 1 highlights the key features of the framework. It features a complete separation between a declarative model and a procedural search component. The declarative model relies on combinatorial constraints specifying the properties of solutions, while the search is exclusively devoted to the selection of a heuristic and meta-heuristic. In this model, the search remains problem specific. Yet, both the model and the search can evolve independently. One can add constraints without changing the search or choose a different search strategy without modifying the model. All these characteristics, when blended with the performance of an incremental computation, deliver an appealing architecture for producing local search solutions.

The program in Fig. 1 can still be improved. In particular, it is tempting to provide syntactic sugar to automatically handle the boiler plate code and support the notion of *constraint annotations* to partition the constraint set into soft and required constraints $S \cup R$. The resulting program is shown in Fig. 9. The model m is now attaching a *soft* annotation to each constraint to dictate its addition to S, while line 10 is used to extract the set of soft constraints from m. Perhaps even more interestingly, the adoption of an explicit first-class model concept enables the authoring of completely generic search procedures. Indeed, the code in lines 10–13 can be packaged as the reusable min-conflict procedure depicted in Fig. 10.

Naturally, lines 10–13 from Fig. 9 disappear from the model in favor of a single line calling the generic `minConflictSearch` on model m program, i.e.,

```
1 minConflictSearch(m);
```

This leaves us with a 10-line Constraint-Based Local Search implementation.

```
1  import lssolver;
2  int n = 8;
3  range Size = 1..n;
4  model m {
5     var{int} queen[Size](Size);
6     soft: alldifferent(queen);
7     soft: alldifferent(all(i in Size) (queen[i] + i));
8     soft: alldifferent(all(i in Size) (queen[i] - i));
9  }
10 ConstraintSystem<LS> S = m.getSoftConstraintSystem();
11 while(S.violations() > 0)
12   selectMin(i in Size,v in Size : queen[i]!=v)(S.getAssignDelta(queen[i],v))
13     queen[i] := v;
```

Fig. 9 A revised constraint-based local search model for n-queens

```
1 void function minConflictSearch(Model<LS> m) {
2    ConstraintSystem<LS> S = m.getSoftConstraintSystem();
3    var{int}[] X = S.getIntVariables();
4    while(S.violations() > 0)
5      selectMin(i in X.getRange(),
6                v in X[i].getDomain() : X[i]!=v)(S.getAssignDelta(X[i],v))
7        X[i] := v;
8 }
```

Fig. 10 A reusable Min-Conflict Procedure

Case Studies

To explore Constraint-Based Local Search, it is desirable to consider a few applications that are elegantly and effectively solved with COMET. The next three subsections consider the progressive party problem [16], car sequencing [3], and scene allocation [18] as each application illustrates a different aspect.

Progressive Party

The progressive party problem is a standard benchmark in combinatorial optimization, and it illustrates two important features. It shows a rich model with many combinatorial constraints as well as constraints on the truth value of relations. It also shows how soft constraints may be instrumental in obtaining a neighborhood. The goal in this problem is to assign guest parties to boats (the hosts) over multiple time periods. Each guest can visit the same boat only once and can meet every other guest at most once over the course of the party. Moreover, for each time period, the guest assignment must satisfy the capacity constraints of the boats.

Figure 11 depicts the declarative part of the model. The decision variable `boat[g,p]` specifies the boat visited by guest `g` in period `p`. Lines 8–9 specify the alldifferent constraints for each guest, lines 10–11 specify the capacity constraints, and lines 12–13 state that two guests meet at most once during the evening. The `soft(2)` annotations added on lines 9 and 11 are not only specifying that the constraint must be softened, but they also associate a fixed static weight of 2 with each constraint. The weights can be easily incorporated in the inductive definition of $\mathbb{V}(c)$ shown in Fig. 4

$$\mathbb{V}(\texttt{soft}(w) : c) = w \cdot \mathbb{V}(c)$$

to handle *statically weighted* soft constraints.

```
1 range Hosts = 1..13;
2 range Guests = 1..29;
3 range Periods = 1..up;
4 set{int} config[1..6];
5
6 model m {
7    var{int} boat[Guests,Periods](Hosts);
8    forall(g in Guests)
9       soft(2): alldifferent(all(p in Periods) boat[g,p]);
10   forall(p in Periods)
11      soft(2): knapsack(all(g in Guests) boat[g,p],crew,cap);
12   forall(i in Guests, j in Guests : j > i)
13      soft: atmost(1,all(p in Periods) boat[i,p] == boat[j,p]);
14 }
```

Fig. 11 The progressive party problem

The Neighborhood The first sensible neighborhood to consider focuses on reassigning a single variable `boat[g,p]` to a new value. This can follow the same template used for the queens problem. These moves impact the violations of *all* the constraints. However, these moves may also prove too to be restrictive. When an instance is near satisfaction, one can expect most of the bins in any given knapsack to be near full. A single variable reassignment amounts to moving an item from one bin into another and that may prove fruitless from a violation standpoint. A natural idea is to include a *second neighborhood* that considers the exchange of values between two variables appearing in the same constraint. While such swaps have no effects on the `alldifferent` constraints, they can more easily lead to violation improvements for the `knapsack` constraints. Formally, the neighborhood is therefore

$$\begin{aligned} N = \{\sigma[x/v] \quad & | \; x = \arg\max_{z \in X} \downarrow_z (\sigma, \mathbb{V}(S)) \wedge v \in D(x) \setminus \{\sigma(x)\}\} \bigcup \\ \{\sigma[x/c, y/d] \; & | \; x = \arg\max_{z \in X} \downarrow_z (\sigma, \mathbb{V}(S)) \wedge \\ & \quad y \in \bigcup_{c \in S \wedge x \in vars(c)} vars(c) \wedge c = \sigma(y) \wedge d = \sigma(x)\}, \end{aligned}$$

where S is the set of soft constraints.

The code in Fig. 12 implements that idea. The main loop (lines 7–30) seeks to improve the overall violations of the soft constraints. The selector on line 9 picks a variable `boat[g,p]` that induces the most violations. Once the variable is selected, the two *nested* selectors (lines 10–15 and lines 16–22) implement the two parts of the neighborhood structure. Lines 10–15 select the variable with the most violations and choose a new value that decreases its violations the most. The second selector is more interesting. It picks a second variable `boat[g1,p]` in the same

```
1  int tenure = 2;
2  int it = 0;
3  int tabu[Guests,Periods,Hosts] = 0;
4
5  ConstraintSystem<LS> S = m.getSoftConstraintSystem();
6  MinNeighborSelector N();
7  while (S.violations() > 0) {
8     int old = S.violations();
9     selectMax(g in Guests, p in Periods)(S.violations(boat[g,p])) {
10       forall(h in Hosts : tabu[g,p,h] <= it) {
11          neighbor(S.getAssignDelta(boat[g,p],h),N) {
12             tabu[g,p,boat[g,p]] = it + tenure;
13             boat[g,p] := h;
14          }
15       }
16       selectMin(g1 in Guests,d=S.getSwapDelta(boat[g,p],boat[g1,p]))(d) {
17          neighbor(d,N) {
18             tabu[g,p,boat[g1,p]] = it + tenure;
19             tabu[g1,p,boat[g,p]] = it + tenure;
20             boat[g,p] :=: boat[g1,p];
21          }
22       }
23    }
24    if (N.hasMove()) {
25       call(N.getMove());
26       if (violations < old && tenure > 2) tenure = tenure - 1;
27       if (violations >= old && tenure < 10) tenure = tenure + 1;
28    }
29    it = it + 1;
30 }
```

Fig. 12 The search procedure for the progressive party problem

period p as the first variable `boat[g,p]` in such a way that the swap between the two variables has the largest impact on the overall violations of S. Note that both variables appear in the knapsack constraint stated over period p. The remainder of the code (lines 24–28) executes the best move in N (if one exists) and updates the variable tabu tenure. Indeed, it marks assignment moves involving either $boat[g, p]$ or $boat[g1, p]$ and their current host assignment as tabu. (The tabu filter is used on line 10.) The swap operator used on line 20 (`:=:`) atomically exchanges the value of the two variables.

The 30 lines of code are compact and elegant: They benefit from the automation provided by the neighborhood selector, the selectors, and the neighbor construction. Yet, they still require some effort to analyze the model and produce a code template that follows a standard recipe. In addition, this code skeleton must still be updated with classic ideas like search intensification around high-quality local minima and diversification to escape from basins of attraction. However, the steps applied in deriving this search procedure are rather systematic: They rely on an analysis of the

model to recognize the presence of specific types of constraints that then suggest a particular neighborhood structure. That is the idea behind the synthesis of search procedures that is described next.

The Synthesized Search Given that models are first-class objects, COMET can manipulate and analyze them. Here, COMET recognizes the presence of knapsack constraints and generates a *composite* neighborhood consisting of the union of variable assignments and of the variable swaps appearing in violated knapsack constraints, an idea first articulated by Van Hentenryck [19]. The synthesis process per se was described in detail in [24].

The skeleton of a synthesized tabu search is depicted in Fig. 13. It uses the fact that all variables have the same domains. Lines 7–9 associate with each decision variable x_i the set of constraints mentioning x_i and susceptible to benefit from

```
1  function bool refersTo(var{int}[] av,var{int} x) {
2     return or(i in av.rng()) (av[i].getId() == x.getId());
3  }
4  function void minConflictWithSwap(Model<LS> m) {
5     ConstraintSystem<LS> S = m.getSoftConstraintSystem();
6     var{int}[] X = S.getVariables();
7     set{Constraint<LS>} cx[i in X.rng()] =
8                    collect(j in S.rng() : S[j].canUseSwap() &&
9                                 refersTo(S[j].getVariables(),X[i])) S[j];
10    MinNeighborSelector N();
11    int k = 0,tenure = 20;
12    int at[X.rng()] = 0;
13    int mat[X.rng(),X.rng()] = 0;
14    for(int k=0;S.violations() != 0;k++) {
15       selectMax(i in X.getRange())(S.violations(X[i])) {
16          forall(v in X[i].getDomain() : at[i] <= k)
17             neighbor(S.getAssignDelta(X[i],v),N) {
18                X[i] := v;
19                at[i] = k + tenure;
20             }
21          forall(c in cx[i]) {
22             var{int}[] Y = c.getVariables();
23             forall(j in Y.getRange() : mat[i,j] <= k)
24                neighbor(S.getSwapDelta(X[i],Y[j]),N) {
25                   X[i] :=: Y[j];
26                   mat[i,j] = mat[j,i] = k + tenure;
27                }
28          }
29       }
30       if (N.hasMove()) call(N.getMove());
31    }
32 }
```

Fig. 13 The synthesized search procedure for the progressive party problem

exchanging (swapping) the values of two of its variables. This initial step is a straightforward projection of the constraint array in S that only retains constraints that refer to x_i and *can use a swap*. Line 10 defines a neighborhood selector. Line 15 selects the variable x_i with the most violations among the softened constraints S. Lines 16–20 focus on the first part of the neighborhood and consider simple reassignments that lead to the largest violation decrease. All the potential neighbors are submitted to N. Lines 21–29 consider all the constraints referring to variable x_i and for which swaps can be of potential benefit. For each such constraint, the code retrieves the variables of the constraint and accumulates in the selector N all the closures modeling swaps (along with their impact on violations).

It is appealing to notice the similarities between the manual and synthetic implementation. In essence, they both capture the same idea. Yet, the generic code adapts to the model and considers exploiting all the constraints susceptible to profit from swaps. The COMET extension required to do so is a minimal extension to query the capabilities of the constraint in the model.

As shown later, the synthesized search outperforms all published results on this problem. Note that, if the set of required constraints R was not empty (it is empty in this application), the two neighborhoods would have to discard assignments and swaps that yield nonzero values for the calls to `getAssignDelta` and `getSwapDelta` to implement the legality requirement correctly. Finally, observe that this skeleton implementation contains a very simply tabu condition to further restrict the legal moves to those that were not recently attempted; this is achieved with two simple data structures (one per neighborhood) that record the last iteration number when a move was performed. Supporting generic intensification and diversification is equally easy.

Car Sequencing

Figure 14 presents a model for car sequencing. In this application, n cars must be sequenced on an assembly line of length n. The customer demands for car configurations are specified in an array $demand$ and the total demand is, of course, n. Each car configuration may require a different set of options, while capacity constraints on the production units restrict the possible car sequences. For a given option o, these constraints are of the form k out of m meaning that, out of m successive cars, at most k can require o. The model declares the decision variables specifying which type of car is assigned to each slot in the assembly line (line 12). It states a hard constraint specifying which cars must be produced (line 13) and then states the soft capacity constraints for each option (lines 14–15).

The handcrafted search procedure, illustrated in Fig. 15, is modeled after a conflict minimization structure. The initialization satisfies the cardinality constraint by using a random permutation of an array of values that already meet the cardinality requirement (lines 4–7). The main loop (lines 10–27) minimizes the number of violations by selecting the slot of the assembly line causing the most violations

```
// ... read parameters nbCars, nbConfigs, nbOptions
range Cars = 1..nbCars;
range Configs = 0..nbConfigs-1;
range Options = 1..nbOptions;
boolean requires[Configs,Options];
int demand[Configs];
int lb[Options],ub[Options];
// ... read the data ...

set{int} options[o in Options] = setof(c in Configs) requires[c,o];
model m {
   var{int} line[Cars](Configs);
   hard: atmost(demand,line);
   forall(o in Options)
      soft: sequence(line,options[o],lb[o],ub[o]);
}
```

Fig. 14 The car sequencing model

```
ConstraintSystem<LS> S = m.getSoftConstraintSystem();
Solver<LS> ls = m.getLocalSolver();
int best = System.getMAXINT();
int cars[Cars];
int nb = 0;
forall(c in Configs, n in 1..demand[c])
   cars[++nb] = c;
RandomPermutation perm(Cars);
forall(c in Cars) lines[c] := cars[perm.get()]; // Satisfy required constraint
while (S.violations() != 0) {
   selectMax(i in Cars)(S.violations(X[i])) {
      selectMin(j in Cars : line[i] != line[j] && t[i,j] <= it)
         (S.getSwapDelta(line[i],line[j])) {
         X[i] :=: X[j];
         t[i,j] = t[j,i] = it + tenure;
      }
   }
   if (S.violations() < best) { best = S.violations();stable = 0;}
   if (stable == 500) {
      with atomic(ls)
         forall(k in 1..5)
            select(a in Cars,b in Cars : line[a] != line[b]) line[a] :=: line[b];
      stable := 0;
      best = S.violations();
   } else stable++;
   it++;
}
```

Fig. 15 The handcrafted search procedure for car sequencing

and swapping its content with another slot that delivers the most improvements. The move itself appears on line 14. The search features a *diversification* component (lines 19–25) that randomly swaps a subset of slots in the assembly line when no improvement took place for a number of iterations. Each time an improving move is found, the stability counter is reset and the best value for this stage is recorded (line 18).

It is possible to recognize the combinatorial structure present in the model thanks to the presence of global `sequence` constraints and to automatically synthesize a search procedure that matches the ideas present in the handcrafted search procedure. The result is shown in Fig. 16. It depicts a generic tabu search procedure featuring several key components. Note how line 6 of the model retrieves the required

```
1  function void tabuSearch(Model<LS> m) {
2     ConstraintSystem<LS> S = m.getSoftConstraintSystem();
3     ConstraintSystem<LS> R = m.getHardConstraintSystem();
4     Solver<LS> ls = m.getLocalSolver();
5     var{int}[] X = R.getVariables();
6     forall(i in R.getRange()) with atomic(ls) R.getConstraint(i).initialize();
7     Counter it(ls) := 0;
8     Counter stable(ls) := 0;
9     int tenure = 20;
10    int t[X.rng(),X.rng()] = 0;
11    Integer best(System.getMAXINT());
12    whenever S.violations()@changes(int o,int n)
13       if (n < best) { best := n;stable := 0;}
14    whenever it@changes() stable++;
15    whenever stable@changes()
16       if (stable == 500) {
17          with atomic(ls)
18             forall(k in 1..5)
19                select(a in X.rng(),b in X.rng() : X[a] != X[b]) X[a] :=: X[b];
20          stable := 0;
21       }
22    while (S.violations() != 0) {
23       selectMax(i in X.rng())(S.violations(X[i])) {
24          selectMin(j in X.rng() : X[i] != X[j] && t[i,j] <= it)
25             (S.getSwapDelta(X[i],X[j])) {
26             X[i] :=: X[j];
27             t[i,j] = t[j,i] = it + tenure;
28          }
29       }
30       it++;
31    }
32 }
```

Fig. 16 The synthesized tabu search for car sequencing

constraints (line 3) and initializes the start assignment by delegating to each required constraint the task of picking a suitable assignment. A basic requirement for achieving this is the following independence property of the required constraints:

$$\forall c_i, c_j \in R \text{ s.t. } i \neq j : vars(c_i) \cap vars(c_j) = \varnothing.$$

In this case, $R = \{$`atmost(demand,line)`$\}$, and the sole cardinality constraint can create, in polynomial time, an array of values that meet the cardinality requirement and randomly permute it to produce the initial assignment to the variables in $vars(c)$. The algorithm for the cardinality constraint uses a (randomized) feasible flow algorithm to match values to variables. As all variables appear in the cardinality constraint, swapping two variables is feasibility-preserving (the number of cars of each type is unchanged). Moreover, the cardinality constraint is tight, meaning that there is a bijection from variable to value occurrences. The core of the search spans lines 22–31 and features the selection of the most conflicting variable (line 23) together with the variable that yields the largest decrease in violations through a swap (lines 24–25). The actual move is performed on line 26, and the move is marked tabu in line 27.

The implementation of the diversification is more interesting. To modularize the capability, it relies on *events*. In COMET, objects like `Counter`, `Integer`, or `var{int}` are capable of dispatching *notifications* when specific events occur. For instance, a counter issues a *change* event when its value is modified. COMET provides the ability to associate a code fragment with events: The code is then executed in response to these events. This is illustrated on line 14 that states that, each time the iteration counter `it` changes, the stability counter `stable` must increase. Similarly, lines 15–21 specify that, when the stability counter changes, one should check whether it has reached a critical value, 500 in this example, in which case a diversification step is undertaken. This architecture promotes the separation of the diversification logic from the main heuristic. Indeed, the code for the diversification simply dictates how to react to changes to the stability counter, and the COMET platform automatically *weaves* that code in the proper place. Finally, note how recording improvements in the violations is also done through an event hooked on the violations of the entire soft system S.

Scene Allocation

Figure 17 features a model for the scene allocation problem that is sometimes used to compare CP and MIP solvers since it is highly symmetric. The problem consists of assigning specific days for shooting scenes in a movie. There can be at most five scenes shot per day and all actors of a scene must be present. Each actor has a fee and is paid for each day she/he plays in a scene. The goal is to minimize the production cost. The decision variable *scene* (line 12) represents the day a scene is shot. The hard cardinality constraint (line 13) specifies the maximum number of scenes shot

```
1 int maxScene = ...; // read the data
2 int maxDay = ...; // read the data
3 range Scenes = 1..maxScene;
4 range Days = 1..maxDay;
5 enum Actor = ... ; // read the data
6 int pay[Actor] = ...; // read the data
7 set{Actor} appears[Scenes] = ...; // read the data
8 set{int} which[a in Actor] = setof(s in Scenes) member(a,appears[s]);
9 int occur[Days] = ...; // read the data
10
11 model m {
12    var{int} scene[Scenes](Days);
13    hard: atmost(occur,scene);
14    minimize: sum(a in Actor) pay[a]*
15                  (sum(d in Days) (or(s in which[a]) (scene[s] == d)));
16 }
17 generatedTabuSearch(m);
```

Fig. 17 Scene allocation model

on any one day. The objective function minimizes the sum of actor compensations, which is the actor fee times the number of days he/she appears in a scene shot on that day.

The model in Fig. 17 is now a constraint optimization model featuring an objective function that demands a different strategy to obtain a suitable search procedure. In general, it is necessary to juggle three considerations. First, one should maintain the feasibility of the required constraints through a suitable initialization and the selection of moves that never violate the constraints in R. Second, one may have to deal with softened difficult constraints (S) for whom one searches for a feasible solution by driving down the violations. Third, it is essential to drive down the value of the objective function. The latter two considerations (true objective and softened constraint) are naturally conflicting as it is easier to drive the objective function down if one violates difficult constraints and vice versa. The practical response is to rely on a statically or a dynamically weighted sum of the two objectives where the search shifts the emphasis on either considerations by altering the weights.

In this application, $S = \varnothing$ and $R = \{\texttt{atmost}(occur, scene)\}$; hence, the task is somewhat simpler as there is only one component to the objective function. In this case, the generated tabu search is driven by the sole required constraint, but with a significant difference. In the previous example, the soft constraint was tight, a fact detected by the model analysis. Indeed, the cardinality constraint in car sequencing is tight because the assembly line has as many slots as the number of cars to produce. This is not the case here: There are typically fewer scenes than the number of slots in which they can be scheduled, and thus the required constraint is not a bijection between variables and value occurrences. As a result, limiting the neighborhood

only to swaps would preserve the feasibility of the cardinality constraint at the expense of a significant decrease in solution quality. This is not surprising since the neighborhood would no longer be connected. The model analysis however recognizes that the `atmost` constraint is not tight and also considers feasibility-preserving assignments.

The generated skeleton is depicted in Fig. 18. As before, line 5 uses the required constraints to initialize the search to a feasible assignment (once again, the task is delegated to the required constraints and the model analysis ensures that $vars(c_i) \cap vars(c_j) = \varnothing \ \forall\, c_i, c_j \in R$ before generating code). The core of the search spans lines 11–24 and relies on the union of two neighborhoods. Line 12 starts by selecting a variable x_i that can lead to the largest decrease in the objective function. Lines 13–15 consider all the swaps that include x_i and lead to the largest decrease in the objective function (i.e., the most negative delta). The selector on line 13 is semi-greedy and will select, uniformly at random, one of the top-3 such moves. Lines 16–19 are devoted to the second neighborhood and collect the best value to reassign x_i. Line 17 shows the conjunct that eliminates assignments that are not feasible with respect to R. The skeleton search uses a vanilla tabu data structure and omits the diversification component for simplicity.

```
1  function void generatedTabuSearch(Model<LS> m) {
2     Function<LS> obj = m.getObjective();
3     ConstraintSystem<LS> R = m.getHardConstraintSystem();
4     Solver<LS> ls = m.getLocalSolver();
5     forall(i in R.getRange()) with atomic(ls) R.getConstraint(i).initialize();
6     int it = 0,tenure = 20,best = System.getMAXINT();
7     var{int}[] X = obj.getVariables();
8     int tm[X.rng(),X.rng()] = 0;
9     int t[X.rng()] = 0;
10    MinNeighborSelector N();
11    while (it < 10000) {
12       selectMax(i in X.rng())(obj.decrease(X[i])) {
13          selectMin[3](j in X.rng() : i != j && tm[i,j] <= it,
14                       d = obj.getSwapDelta(X[i],X[j]))(d)
15             neighbor(d,N) { X[i] :=: X[j];tm[i,j] = tm[j,i] = it + tenure;}
16          selectMin(v in X[i].getDomain() : t[i] <= it && v != X[i] &&
17                                            R.getAssignDelta(X[i],v)==0,
18                    d = obj.getAssignDelta(X[i],v))(d)
19             neighbor(d,N) { X[i] := v;t[i] = it + tenure;}
20       }
21       if (N.hasMove()) call(N.getMove());
22       if (obj.evaluation() < best) best = obj.evaluation();
23       it++;
24    }
25 }
```

Fig. 18 Generated search for scene allocation

Implementation

The implementation of a Constraint-Based Local Search system critically depends on incremental computation. Constraints and objective functions must respond to their APIs like `violations`, `increase`, `decrease`, or `getAssignDelta` extremely fast in order to consider large neighborhoods and long traces of assignment within an allotted time.

To deliver this performance, the implementation is presented in two distinct layers. The `invariant` layer is responsible for the basic incremental computation that occurs when an assignment is changed through a local move operator. The `differentiability` layer is responsible for implementing the response mechanism behind the constraints and objective function. Their implementation is primarily framed in terms of invariants. Both are highlighted in this section, starting with invariants (section "Invariants") and finishing with differentiation (section "Differentiation").

Invariants

Invariants provide a declarative concept that relieves programmers from the tedious task of maintaining complex data structures incrementally. By focusing on what to maintain, rather than how to maintain assignments under changes, programmers are relieved of an error-prone, yet critical, aspect of implementing constraints and objective functions. In essence, invariants capture the so-called *one-way constraints* [1,2,11,17], namely, they capture the value of an expression that must be maintained over time under changes to the value of its variables. An acyclic network of dependencies connects all the variables of the problems and is responsible for scheduling the evaluations. This subsection reviews examples and outlines the underlying implementation.

Definition 15 (Invariant). An invariant $\mathcal{I}$ is a one-way constraint

$$\langle x_1, \ldots, x_n \rangle \leftarrow f(y_1, \ldots, y_m),$$

where $x_1, \ldots, x_n$ are called invariant variables and $y_1, \ldots, y_m$ are either decision or invariant variables. The set $O = \{x_1, \ldots, x_n\}$ is the output variables of $\mathcal{I}$. The set $I = \{y_1, \ldots, y_m\}$ is the input variables of $\mathcal{I}$. We often abuse notation and rewrite the one-way constraints as

$$O \leftarrow f(I).$$

$\mathcal{I}_O$, $\mathcal{I}_I$, and $\mathcal{I}_f$ denote the output variables, the input variables, and the function of invariant $\mathcal{I}$.

Since invariants are one-way constraints, there are some necessary syntactic restrictions. One such restriction is that no two invariants have a common output variable. This ensures that every invariant variable is defined at most once.

The declarative semantics of an invariant specifies that the one-way constraints always hold.

Definition 16 (Declarative Semantics of an Invariant). Let σ be an assignment before or after any atomic instruction of a program for which the invariant $\mathcal{I}$ has been posted. Then, it follows that

$$\langle \sigma(x_1), \ldots, \sigma(x_n) \rangle \leftarrow f(\sigma(y_1), \ldots, \sigma(y_m)),$$

where $\mathcal{I}_O = \{x_1, \ldots, x_n\}$ and $\mathcal{I}_I = \{y_1, \ldots, y_m\}$. The narrative abuses notation and sometimes writes

$$\sigma(\mathcal{I}_o) \leftarrow \mathcal{I}_f(\sigma(\mathcal{I}_I)).$$

Example 8 (Expression Invariant). The numerical invariant

$$x \leftarrow y + 3 * z$$

stating that, at any point in time, the value of x in an assignment σ, i.e., $\sigma(x)$, should be the value $\sigma(y)$ plus three times the value of $\sigma(z)$.

Operationally, an invariant must maintain the link between its output and input variables under assignments to its input variables. The invariant maintains this link by keeping a local assignment of its input variables and by implementing an update function which updates the global and local assignments to reflect the change in one of its input variables.

Definition 17 (Operational Semantics of an Invariant). Let $\mathcal{I}$ be an invariant. Operationally, $\mathcal{I}$ maintains a local store $\mathcal{I}_\sigma$ over variables $\mathcal{I}_I$ and implements an update procedure $\mathcal{I}_u : \Sigma \times X$. Let $\sigma_l = \mathcal{I}_\sigma$ be the local store of $\mathcal{I}$, σ be an assignment, and $y \in \mathcal{I}_I$. The update procedure $\mathcal{I}_u(\sigma, y)$ performs the following assignments:

$$\begin{aligned} \sigma(\mathcal{I}_O) &:= \mathcal{I}_f(\sigma_l[\sigma(y)/y](\mathcal{I}_I)); \\ \sigma_l(y) &:= \sigma(y); \end{aligned}$$

To propagate a collection of invariants effectively, the implementation constructs an incremental graph.

Definition 18 (Incremental Graph). An incremental graph $G(X \cup I, A)$ is a directed acyclic graph whose vertices coincide with decision variables and invariants,

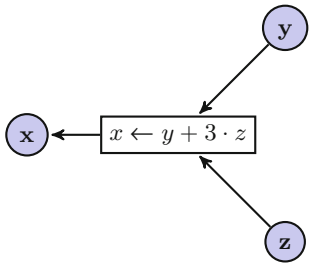


Fig. 19 The dependencies of an arithmetic invariant

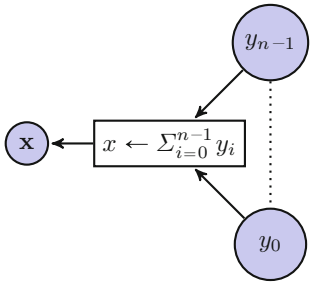
and whose arc set corresponds to the dependencies induced by the invariant. Each invariant $\mathcal{I}$ introduces a dependency $\mathcal{I} \leftarrow y$ for each $y \in \mathcal{I}_I$ and a dependency $x \leftarrow \mathcal{I}$ for each $x \in \mathcal{I}_O$.

Figure 19 depicts the dependencies of the arithmetic invariant presented earlier.

Example 9 (Summation Aggregate). A summation invariant captures the relation

$$x \leftarrow \sum_{i=0}^{n-1} y[i],$$

where n is a constant and y denotes an array of n variables. The dependencies are shown below:



The update function $\mathcal{I}_u(\sigma, y)$ implements the following code:

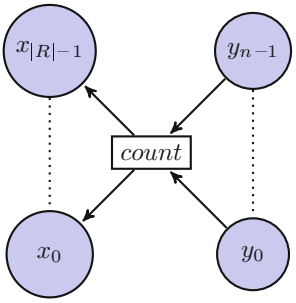
```
1 σ(x) := σ(x) + σ(y) − σ_l(y);
2 σ_l(y) := σ(y);
```

where $\sigma_l = \mathcal{I}_\sigma$.

Example 10 (Counting). A counting invariant $x \leftarrow count(y)$ defined over an array of variables y yields an array of variables x indexed by $R = \bigcup_{i=0}^{n-1} D(y_i)$ that maintains the relations

$$\forall\, v \in R \;:\; x_v = \sum_{i \,\in\, range(y)} (y_i = v)\,.$$

Namely, x_v counts the number of variables in y currently assigned to v. The dependencies are as follows:



and there are $|R| - 1 + n$ of them. The update function $\mathcal{I}_u(\sigma, y)$ implements the following code:

```
1 σ(x_{σ_l(y)}) := σ(x_{σ_l(y)}) − 1;
2 σ(x_{σ(y)}) := σ(x_{σ(y)}) + 1;
3 σ_l(y) := σ(y);
```

where $\sigma_l = \mathcal{I}_\sigma$.

Incremental computation Given $G(X \cup I, A)$, one can obtain a topological sort r of its vertices. Indeed, each dependency $y \leftarrow x$ imposes the constraint

$$1 + r(x) \leq r(y).$$

The partial ordering expressed in r drives the propagation algorithm that updates all the variables following an assignment of new values to decision variables.

Figure 20 shows the pseudo-code for the invariant propagation algorithm. The `propagate` algorithm is invoked with the incremental graph $G(X, A)$, an assignment σ, and the set of decision variables Y that have been updated. Line 3 initializes a priority queue PQ with all the invariants mentioning any member of Y as one of its sources. The priority associated with invariant $\mathcal{I}$ is its topological

```
1  propagate(G(X, A),σ,Y)
2  {
3     PQ = ⋃_{y∈Y} {⟨I ← y, r(I)⟩ ∈ A};
4     while (PQ ≠ ∅) {
5        σ_o := σ;
6        ⟨I ← y, r(I)⟩ := extractMin(PQ);
7        I_u(σ, y);
8        C = {x ∈ I_O : σ_o(x) ≠ σ(x)};
9        forall(x ∈ C)
10          PQ = PQ ∪ {⟨I ← x, r(I)⟩ ∈ A};
11    }
12 }
```

Fig. 20 The invariant propagation

number $r(\mathcal{I})$. The main loop spanning lines 6–11 considers the invariants in priority order (see Line 7). The update function of the selected invariant is executed in Line 8. Line 9 collects in C the modified output variables, and Line 10 enqueues the new invariant to reconsider.

The correctness of the `propagate` algorithm hinges on the facts that $G(X, A)$ is acyclic. The use of topological numbers guarantees that the invariant considered in iteration i is handled only after its sources have reached final values in σ. As long as u meets its specification, the assignment σ is guaranteed to satisfy all the invariants considered in iterations $1..i$. Computing the affected variables in C and scheduling any invariant depending on them cannot possibly schedule an earlier invariant since the graph is acyclic.

Differentiation

The implementation of constraints and objective functions relies on the foundation provided by invariants as first described in [21]. As indicated earlier, the implementation of the constraint API

```
1 interface Constraint<LS> {
2   bool holds();
3   var{int} violations();
4   var{int} violations(var{int} x);
5   int getAssignDelta(var{int} x,int v);
6 }
```

depends on an efficient, incremental evaluation of violations, variable violations, and gradients. The functions $\mathbb{E}(\sigma, e)$, $\mathbb{V}(e)$, $\uparrow_x(\sigma, e)$, $\downarrow_x(\sigma, e)$ are essential to the evaluation of expressions and the definition of violation and gradient expressions from algebraic definitions. Yet, none of them is incremental and therefore unsuitable

for *direct* use in, for instance, Example 5. Likewise, this is true for combinatorial constraints such as `alldifferent`. Invariants do provide the solution, and the subsection focuses on the implementation when constraints are expressed algebraically or combinatorially.

Algebraic and Logical Constraints The key insight to an incremental implementation is to forsake the evaluation function $\mathbb{E}$ and adopt instead a *compilation* approach, generating invariants that evaluate an expression incrementally. This is achieved by Function $\mathbb{I} : \mathcal{E} \rightarrow X \times 2^I$ which is partly shown in Fig. 21 and defined inductively on the structure of expressions. Specifically, a call $\mathbb{I}(e)$ on an expression e produces a decision variable holding the value of the expression and a set of invariants that define this variable.

Note how each line of the inductive definition obtains the variable and invariants supporting the operand and produces a fresh variable alongside an additional invariant based on the variables obtained from the inductive calls on the operands. For instance, the last line of Fig. 21 shows that the compilation of a summation expression inductively obtains a fresh decision variable i_k for each term e_k, as well as the invariants supporting i_k's definition in S_k. It then creates a new fresh variable i_Σ and the summation aggregate invariant that defines it. It finally adds all the invariants in S_k.

Relations simply give rise to arithmetic expressions through the function $\mathbb{V}$ whose definition is in Fig. 3. To obtain an incremental evaluation of the violations of an arbitrary relation $e_1 \diamond e_2$, one can simply obtain the violation expression and compile it with:

$$\langle i_{e_1 \diamond e_2}, S_{e_1 \diamond e_2} \rangle = \mathbb{I}(\mathbb{V}(e_1 \diamond e_2))$$

to retrieve a set of invariants (which it states) and an output variable $i_{e_1 \diamond e_2}$ whose value $\sigma(i_{e_1 \diamond e_2})$ denotes the violations of $e_1 \diamond e_2$ with respect to σ. At this point, the implementation of method `violation()` is straightforward and reduces to

Fig. 21 Compiling expression evaluations to invariants

$$
\begin{array}{ll}
\mathbb{I}(v) & = \langle i_v, \{i_v \leftarrow v \Leftrightarrow v \in \mathbb{R}\}\rangle \\
\mathbb{I}(x) & = \langle i_x, \{i_x \leftarrow x\}\rangle \\
\mathbb{I}(e_1 + e_2) & = \text{let } \langle i_1, S_1\rangle = \mathbb{I}(e_1), \langle i_2, S_2\rangle = \mathbb{I}(e_2) \\
 & \quad \text{in } \langle i_+, \{i_+ \leftarrow i_1 + i_2\} \cup S_1 \cup S_2\rangle \\
\mathbb{I}(e_1 - e_2) & = \text{let } \langle i_1, S_1\rangle = \mathbb{I}(e_1), \langle i_2, S_2\rangle = \mathbb{I}(e_2) \\
 & \quad \text{in } \langle i_-, \{i_- \leftarrow i_1 - i_2\} \cup S_1 \cup S_2\rangle \\
\mathbb{I}(e_1 \times e_2) & = \text{let } \langle i_1, S_1\rangle = \mathbb{I}(e_1), \langle i_2, S_2\rangle = \mathbb{I}(e_2) \\
 & \quad \text{in } \langle i_\times, \{i_\times \leftarrow i_1 \times i_2\} \cup S_1 \cup S_2\rangle \\
\mathbb{I}(\textsc{Abs}(e)) & = \text{let } \langle i_e, S_e\rangle = \mathbb{I}(e) \\
 & \quad \text{in } \langle i_{abs}, \{i_{abs} \leftarrow \textsc{Abs}(i_e)\} \cup S_e\rangle \\
\mathbb{I}(\min(e_1, e_2)) & = \text{let } \langle i_1, S_1\rangle = \mathbb{I}(e_1), \langle i_2, S_2\rangle = \mathbb{I}(e_2) \\
 & \quad \text{in } \langle i_{min}, \{i_{min} \leftarrow min(i_1, i_2)\} \cup S_1 \cup S_2\rangle \\
\mathbb{I}(\sum_{k=0}^{n} e_k) & = \text{let } \langle i_k, S_k\rangle = \mathbb{I}(e_k) \ \forall \ k \ \in 0..n \\
 & \quad \text{in } \langle i_\Sigma, \{i_\Sigma \leftarrow \sum_{k=0}^{n} i_k\} \cup \bigcup_{k=0}^{n} S_k\rangle
\end{array}
$$

returning $\sigma(i_{e_1 \diamond e_2})$. The incremental evaluation of gradients proceeds similarly with the generation of an expression modeling the gradient of e and its compilation with $\mathbb{I}$, i.e.,

$$\langle i_{\downarrow_x(e)}, S_{\downarrow_x(e)} \rangle = \mathbb{I}(\downarrow_x (e)),$$

and $\downarrow_x(e)$ is an expression (independent of σ) whose evaluation wrt σ would yield $\downarrow_x(\sigma, e)$. Similarly a method call `violations(x)` must simply return $\sigma(i_{\downarrow_x(e)})$. Finally, objective functions make a direct use of expressions as well as $\uparrow_x$ and $\downarrow_x$ and are therefore handled exactly like constraints.

Combinatorial Constraints While combinatorial constraints could be implemented in terms of expressions, it is often preferable to exploit the semantics of the constraints to directly produce an incremental implementation. To illustrate the idea, consider the `alldifferent(x)` constraint used in the introductory example and responsible for ensuring that no two entries in x have the same value.

Fundamentally, the constraint should maintain the cardinality of each value used in x and require that no two values have a cardinality larger than 1 to satisfy the constraint. The variable violation for x_i would, in this case, simply be the excess in the number of variables assigned to $\sigma(x_i)$. In essence, when the constraint `alldifferent(x)` is added on an array x with n variables where $\bigcup_{j=1}^{n} D(x_j) = V$, it is sufficient to state the following invariants:

$$\begin{aligned} c &\leftarrow count(x) \\ v_i &\leftarrow \max(0, c_i - 1) \ \forall i \in V \\ cv &\leftarrow \textstyle\sum_{k \in V} v_i \\ vv_j &\leftarrow v_{x_j} \qquad\qquad \forall j \in 1..n. \end{aligned}$$

The implementation of the constraint then reduces to

```
1  class alldifferent implements Constraint<LS> {
2    bool holds() { return σ(cv) == 0;}
3    var{int} violations() { return cv;}
4    var{int} violations(var{int} x) { return vv_id(x);}
5    int getAssignDelta(var{int} x,int v) {
6      if σ(x) == v
7        return 0;
8      else
9        return (σ(c_v) ≥ 1) − (σ(x) ≥ 2);
10   }
11 }
```

where the function `id` is used to identify the variable x by an integer. Note the simplicity of the method implementations that simply leverage the work done

by invariants. Additionally, the implementation does not have to provide any imperative code to handle the changes of decision variables as all of this logic is handled through the invariants. Finally, even the `getAssignDelta(x,v)` implementation remains straightforward. When the current value assignment to variable x is identical to the tentative assignment (v), the function returns 0. Otherwise, it returns the amount of change. Namely, if value v is already used once or more, the number of violations will increase by 1. Similarly, if $\sigma(x)$ is used twice or more, a violation is necessarily lost.

Empirical Results

Offering a comprehensive empirical evaluation of COMET is beyond the scope of this chapter. Yet, the monograph [22] contains an extensive empirical evaluation on many problems and discusses the impact of modeling techniques and search.

Instead, this section focuses on demonstrating the potential behind the synthesis of search procedures. In particular, it explores the performance of the search procedures shown in Figs. 13, 16, and 18 and contrasts them with the results obtained from purely synthesized search procedures in the spirit of [24]. In all cases, the results were obtained on a Core i7 machine clocked at 1.8 Ghz and running OSX 10.10, and the reported results are based on averages collected from 100 runs of each algorithm.

Progressive Party Two instances of the problem (5–7 and 6–7) were used in the evaluation in the following table.

Type	*Choice*	$\|P\|$	μ_{iter}	σ_{iter}	$\mu_T(ms)$	$\sigma_T(ms)$
Manual	5	7	2,360.9	1,386.1	723.9	377.3
Synthetic	5	7	2,282.2	1,495.9	842.5	476.1
Manual	6	7	6,847.8	5,714.7	1,683.0	1,324.4
Synthetic	6	7	4,532.2	4,724.5	1,338.1	1,281.6

The search procedures are extremely similar and only differ in the presence of an adaptive tabu list within the synthetic implementation. It is not surprising to note that both implementations are very close with a slight win for the synthetic search without having to invest any effort in parameter tuning.

Car Sequencing One instance (4–72) was used for car sequencing. In this case, the two implementations seem to exhibit significantly different behaviors as the number of iterations is almost 4 times as high (on average) for the manual implementation. This is most likely due to the difference in parameters and in the components of the meta-heuristics within the synthetic search. It also shows that, without exploring variants of the search procedure, it is not obvious to produce high-quality results.

Yet, the ability to easily exploit, without further ado, restarts, diversification, and intensification components is quite valuable. Lastly, note that the times per iteration of the two searches are quite close ($\frac{19.9}{162{,}944} \approx 1.2 \cdot 10^{-4}$ and $\frac{5.4}{49{,}598.8} \approx 1.1 \cdot 10^{-4}$ with σ_{iter} values of the same order of magnitude as the averages μ_{iter}) showing that the differences only come from the search heuristic and meta-heuristic, not the incremental computation.

Type	*Instance*	μ_{iter}	σ_{iter}	$\mu_T(s.)$	$\sigma_T(s.)$
Manual	4-72	162,944.3	164,853.9	19.9	20.5
Synthetic	4-72	49,598.8	57,815.4	5.4	6.3

Scene Allocation The scene allocation benchmark was used to compare three variants. The manual implementation is in Fig. 18, and two synthetic search procedures are with different parameters, namely, one of them uses 10,000 iterations and no restarts, while the other uses 10 restarts and 1000 iterations per restart. Here, the synthetic search relying on restarting is very close to the custom implementation. Its success rate is 99/100, just shy of a perfect 100 like the custom search. While the synthetic search uses more iterations (on average) to get to the optimum, the runtimes are very close. An examination of the synthesized search reveals that the root cause of the difference is the search heuristic. The equivalent of lines 13 and 17 in Fig. 18 in the synthesized search relies on a purely random selector rather than the more aggressive semi-greedy and greedy selectors used in the custom implementation. This difference explains the loss in greediness and explains the positive impact of restarts.

Type	μ_{iter^*}	σ_{iter^*}	μ_{f^*}	σ_{f^*}	$\#Opt$	$\mu_T(ms)$	$\sigma_T(ms)$
Manual	860	982	334,144	0	100	567.4	46.9
Synthetic (10,000 iters, 1 restart)	2,254	3,104	334,960	1,093	64	731.8	175.6
Synthetic (1000 iters, 10 restarts)	1,954	2,212	334,167	227	99	730.1	71.7

Conclusion

Constraint-based local search is an appealing framework for the design and implementation of local search models for any number of applications. It adopts the core practices of constraint programming through the support of separated components for expressing a declarative model and for programming the search. The declarative models rely on a rich language seamlessly blending algebraic, logical, and combinatorial constraints. The search component itself is exclusively devoted to the automation of the most tedious and error-prone activities that arise when implementing a variety of heuristics and meta-heuristics. Perhaps even more

crucially, search procedures can be written completely independently of the model, making them highly reusable and generic. The culmination of this effort is the availability of synthetic search procedures that take advantage of an analysis of the declarative model to produce sensible search procedures that often compete with tailored, hand-written implementations.

The entire framework competitiveness relies on incremental implementations that are constructed on top of invariants and differentiable abstractions such as constraints and objectives. The net result is a platform for practitioners who would to take advantage of the capabilities of local search techniques without the significant investment necessary to produce an efficient implementation.

Cross-References

- Fundamentals of Scatter Search
- Guided Local Search
- Iterated Greedy
- Restart Strategies
- Tabu Search
- Theory of Local Search
- Variable Neighborhood Descent
- Variable Neighborhood Search

References

1. Borning A (1981) The programming language aspects of ThingLab, a constraint-oriented simulation laboratory. ACM Trans Program Lang Syst 3(4):353–387
2. Borning A, Duisberg R (1986) Constraint-based tools for building user interfaces. ACM Trans Comput Graph 5(4):345–374
3. Dincbas M, Simonis H, Van Hentenryck P (1988) Solving the car sequencing problem in constraint logic programming. In: ECAI-88
4. Feo T, Resende M (1995) Greedy randomized adaptive search procedures. J Glob Optim 6:109–133
5. Glover F, Laguna M (1998) Tabu search. In: Du, DZ, Pardalos PM (eds) Handbook of combinatorial optimization. Springer, Boston, MA https://doi.org/10.1007/978-1-4613-0303-9_33
6. Kirkpatrick S, Gelatt C, Vecchi M (1983) Optimization by simulated annealing. Science 220:671–680
7. Laguna M (2002) Scatter search. In: Pardalos PM, Resende MGC (eds.) Handbook of applied optimization. Oxford University Press, New York, pp 183–193
8. Michel L (1998) Localizer: a modeling language for local search. PhD thesis, Brown University
9. Michel L, Van Hentenryck P (1997) Localizer: a modeling language for local search. In: Third international conference on the principles and practice of constraint programming (CP'97), Lintz
10. Minton S, Johnston MD, Philips AB (1990) Solving large-scale constraint satisfaction and scheduling problems using a heuristic repair method. In: AAAI-90

11. Myers B, Guise D, Dannenberg R, Vander Zanden B, Kosbie D, Pervin E, Mickish A, Marchal P (1990) GARNET: comprehensive support for graphical, highly interactive user interfaces. IEEE Comput 23(11):71–85
12. Pham Q-D, Deville Y, Van Hentenryck P (2012) Ls(graph): a constraint-based local search for constraint optimization on trees and paths. Constraints 17(4):357–408
13. Selman B, Kautz H (1993) An empirical study of greedy local search for satisfiability testing. In: AAAI-93, pp 46–51
14. Selman B, Kautz H, Cohen B (1996) Local search strategies for satisfiability testing. In: DIMACS series in discrete mathematics and theoretical computer science, vol 26. American Mathematical Society Publications. DIMACS
15. Selman B, Levesque H, Mitchell D (1992) A new method for solving hard satisfiability problems. In: AAAI-92, pp 440–446
16. Smith BM, Brailsford SC, Hubbard PM, Williams HP (1996) The progressive party problem: integer linear programming and constraint programming compared. Constraints 1:119–138
17. Sutherland IE (1963) SKETCHPAD: a man-machine graphical communication system. MIT Lincoln Labs, Cambridge
18. Van Hentenryck P (2002) Constraint and integer programming in OPL. Informs J Comput 14(4):345–372
19. Van Hentenryck P (2006) Constraint programming as declarative algorithmics. ACP Award for Research Excellence in Constraint Programming. Available at http://www.cs.brown.edu/people/pvh/acp.pdf
20. Van Hentenryck P, Michel L (2005) Control abstractions for local search. Constraints 10(2), 137–157. https://doi.org/10.1007/s10601-005-0553-x
21. Van Hentenryck P, Michel L (2006) Differentiable invariants. In: 12th international conference on principles and practice of constraint programming (CP'06). Lecture notes in computer science, Nantes
22. Van Hentenryck P, Michel L (2009) Constraint-based local search. The MIT Press, Cambridge, MA
23. Van Hentenryck, P, Michel L, Liu L (2005) Constraint-based combinators for local search. Constraints 10(3), 363–384. https://doi.org/10.1007/s10601-005-2811-3
24. Van Hentenryck P, Michel L (2007) Synthesis of constraint-based local search algorithms from high-level models. In: Proceedings of the 22nd national conference on artificial intelligence – volume 1 (AAAI'07). AAAI Press, pp 273–278
25. Walser J (1999) Integer optimization by local search: a domain-independent approach. Springer, Berlin, Heidelberg

Guided Local Search

16

Christos Voudouris, Abdullah Alsheddy, and Ahmad Alhindi

Contents

Introduction . . . 428
Guided Local Search . . . 430
Implementation Guideline . . . 431
Possible Features for Common Applications . . . 434
Guided Fast Local Search . . . 437
Implementation Guideline . . . 438
GLS Extensions, Hybrids, and Variations . . . 442
Extensions to GLS . . . 442
GLS Hybrids . . . 443
Variations of GLS . . . 444
GLS and Artificial Intelligence . . . 445
GLS for Multi-objective Optimization . . . 446
Pareto Local Search . . . 447
Guided Pareto Local Search . . . 447
Other Attempts . . . 449
GLS Implementation on the Traveling Salesman Problem . . . 450
Problem Description . . . 450
Local Search . . . 450

C. Voudouris (✉)
AI Team, Department of Informatics and Telecommunications, National and Kapodistrian University of Athens, Athens, Greece
e-mail: christos.voudouris@gmail.com

A. Alsheddy
College of Computer and Information Sciences (CCIS), Imam Mohammad Ibn Saud Islamic University (IMSIU), Riyadh, Saudi Arabia
e-mail: asheddy@imamu.edu.sa

A. Alhindi
Department of Computer Science and Artificial Intelligence, College of Computers, Umm Al-Qura University, Makkah, Saudi Arabia
e-mail: ahhindi@uqu.edu.sa

© Springer Nature Switzerland AG 2025

R. Martí et al. (eds.), *Handbook of Heuristics*,
https://doi.org/10.1007/978-3-032-00385-0_2

Guided Local Search . . . 451
Guided Fast Local Search . . . 453
GLS/GPLS Implementation on a Workforce Scheduling Problem . . . 453
Problem Description . . . 453
Local Search . . . 455
Feature Definition . . . 456
Overview of GLS Applications . . . 457
Routing/Scheduling Problems . . . 457
Assignment Problems . . . 459
Resource Allocation and Scheduling Problems . . . 460
Constrained Optimization Problem . . . 460
GLS in Commercial Packages . . . 461
Conclusion . . . 461
Cross-References . . . 462
References . . . 462

Abstract

Guided local search (GLS) is a meta-heuristic method proposed to solve combinatorial optimization problems. It is a high-level strategy that applies an efficient penalty-based approach to interact with the local improvement procedure. This interaction creates a process capable of escaping from local optima, which improves the efficiency and robustness of the underlying local search algorithms. Fast local search (FLS) is a way of reducing the size of the neighborhood to improve the efficiency of local search. GLS can be efficiently combined with FLS in the form of guided fast local search (GFLS). This chapter describes the principles of GLS and provides guidance for implementing and using GLS, FLS, and GFLS. It also surveys GLS' extensions, hybrids, and applications to optimization, especially in routing, including promising emerging approaches on combining GLS with AI.

Keywords

Heuristic search · Meta-heuristics · Penalty-based methods · Guided local search · Tabu search · Routing problems · Artificial intelligence

Introduction

Many practical problems are NP-hard in nature, which means complete, constructive search is unlikely to satisfy our computational demand. Many real-life problems cannot be realistically and reliably solved by complete search. This motivates the development of local search, or heuristic methods.

Local search (LS) is the basis of most heuristic search methods. It searches in the space of candidate solutions.The solution representation issue is significant, though it is not the subject of our discussion here. Starting from a (possibly randomly generated) candidate solution, LS moves to a "neighbor" that is "better" than the current candidate solution according to the objective function. LS stops when all neighbors are inferior to the current solution.

LS can find good solutions very quickly. However, it can be trapped in local optima – positions in the search space that are better than all their neighbors, but not necessarily representing the best possible solution (the global optimum). To improve the effectiveness of LS, various techniques have been introduced over the years. Simulated annealing (SA), Tabu search (TS), and Guided local search (GLS) all attempt to help LS escape local optimum. This chapter focuses on GLS [100], a general meta-heuristic algorithm and its applications. GLS sits on top of other heuristic methods with the aim to improve their efficiency or robustness. GLS has been applied to a nontrivial number of problems and found to be efficient and effective. It is relatively simple to implement and apply, with only a few parameters to tune.

GLS was inspired by GENET [30, 97], a neural network architecture which was developed for constraint satisfaction problems. GLS's and GENET's mechanism for escaping from local minima resembles model-free reinforcement learning [97]. GLS also relates to ideas from the area of search theory on how to distribute the search effort. In particular, incremental distribution of the search effort according to information in a probabilistic framework can be found in a class of methods deriving themselves from the optimal search theory of Koopman [54, 88].

The principles of GLS can be summarized as follows. As a meta-heuristic method, GLS sits on top of LS algorithms. To apply GLS, one defines a set of features for the candidate solutions. When LS is trapped in local optima, certain features are selected and penalized. LS searches the solution space using the objective function augmented by the accumulated penalties.

The novelty of GLS is in the way that it selects features to penalize. GLS effectively distributes the search effort in the search space, favoring promising areas. Penalty modifications regularize the solutions generated by local search to be in accordance to prior or gathered during search information. The approach taken by GLS is similar to that of regularization methods for "ill-posed" problems [95]. The idea behind regularization methods and GLS up to an extent is the use of prior information to help in solving an approximation problem. Prior information translates to constraints which further define our problem reducing so the number of candidate solutions to be considered.

The structure of the chapter is as follows. In section "Guided Local Search," we describe GLS with details about implementing its components. Similarly, the implementation of combining GLS with fast local search is discussed in section "Guided Fast Local Search." Next, in section "GLS Extensions, Hybrids, and Variations," we review other algorithms that extend GLS or hybridize it with other techniques. The section "GLS and Artificial Intelligence" examines emerging and promising approaches combining GLS with AI techniques. GLS's ability to handle multi-objective optimization problems is explored in section "GLS for Multi-objective Optimization." The applications of GLS to the traveling salesman problem and a workforce scheduling problem are explained in sections "GLS Implementation on the Traveling Salesman Problem" and "GLS/GPLS Implementation on a Workforce Scheduling Problem," respectively. We give comprehensive references to the applications of GLS and its variants in section "Overview of GLS Applications" with special focus on recent advances in tackling routing problems, and we conclude in section "Conclusion."

Guided Local Search

As mentioned earlier, GLS augments the given objective function with penalties. To apply GLS, one needs to define features for the problem. For example, in the traveling salesman problem [37], a feature could be *"whether the candidate tour travels immediately from city A to city B."* GLS associates a cost and a penalty with each feature. The costs can often be defined by taking the terms and their coefficients from the objective function. For example, in the traveling salesman problem, the cost of the above feature can simply be the distance between cities A and B. The penalties are initialized to 0 and will only be increased when the local search reaches a local optimum. Given an objective function g that maps every candidate solution s to a numerical value, GLS defines a function h that will be used by LS (replacing g):

$$h(s) = g(s) + \lambda \times \sum_{\text{i is a feature}} (p_i \times I_i(s)) \tag{1}$$

where s is a candidate solution, λ is a parameter of the GLS algorithm, i ranges over the features, p_i is the penalty for feature i (all p_i's are initialized to 0), and I_i is an indication of whether s exhibits feature i:

$$I_i(s) = 1 \text{ if } s \text{ exhibits feature } i; 0 \text{ otherwise.} \tag{2}$$

Sitting on top of local search algorithms, GLS helps them to escape local optima in the following way. Whenever the local search algorithm settles in a local optimum, GLS augments the cost function by adding penalties to selected features. The novelty of GLS is mainly in the way that it selects features to penalize. The intention is to penalize "unfavorable features" or features that "matter most" when a local search settles in a local optimum. A feature with high cost has more impact on the overall cost. Another factor that should be considered is the current penalty value of that feature. The utility of penalizing feature i, $util_i$, under a local optimum s_*, is defined as follows:

$$util_i(s_*) = I_i(s_*) \times \frac{c_i}{1 + p_i} \tag{3}$$

where c_i is the cost and p_i is the current penalty value of feature i. In other words, if a feature is not exhibited in the local optimum (indicated by I_i), then the utility of penalizing it is 0. The higher the cost of this feature (the greater c_i), the greater the utility of penalizing it. Besides, the larger the number of times it has been penalized (the greater p_i), the lower the utility of penalizing it again. In a local optimum, the feature with the greatest $util$ value will be penalized. When a feature is penalized, its penalty value is always increased by 1. The scaling of the penalty is adjusted by λ.

By taking the cost and current penalty into consideration in selecting the feature to penalize, GLS focuses its search effort on more promising areas of the search space: areas that contain candidate solutions that exhibit "good features,"

i.e., features involving lower cost. On the other hand, penalties help to prevent the search from directing all effort to any particular region of the search space.

Naturally the choice of the features, their costs, and the setting of λ may affect the efficiency of a search. Experience shows that the features and their costs normally come directly from the objective function. In many problems, the performance of GLS is not too sensitive to the value λ. It means that not too much effort is required to apply GLS to a new problem. In certain problems, one needs expertise in selecting the features and the λ parameter. Research aiming to reduce the sensitivity of the λ parameter in such cases is reported in [68].

Implementation Guideline

A local search procedure for the particular problem is required for the algorithm to be implemented. Guided local search is repeatedly using this procedure to optimize the augmented objective function of the problem. Each time the local search procedure reaches a local minimum, the augmented objective function is modified by increasing the penalties of one or more of the features present in the local minimum. These features are selected by using the utility function (3). The section below presents and explains the pseudo-code for implementing a guided local search method.

The pseudo-code for the guided local search procedure is depicted in Fig. 1, where p is the problem, g is the objective function, h is the augmented objective function, λ is a parameter, I_i is the indicator function of feature i, c_i is the cost of feature i, M is the number of features, p_i is the penalty of feature i, $CONSTRUCTIONMETHOD(p)$ is the method for constructing an initial solution for problem p, and $IMPROVEMENTMETHOD(s_k, h)$ is the method for improving solution s_k according to the augmented objective function h.

To understand the pseudo-code, let us first explain the methods for constructing and improving a solution, as they are both prerequisites for building a GLS algorithm.

Construction Method

As with other meta-heuristics, GLS requires a construction method to generate an initial (starting) solution for the problem. In the pseudo-code, this is denoted by $CONSTRUCTIONMETHOD$. This method can generate a random solution or a heuristic solution based on some known technique for constructing solutions for the particular problem. GLS is not very sensitive to the starting solution given that sufficient time is allocated to the algorithm to explore the search space of the problem.

Improvement Method

A method for improving the solution is also required. In the pseudo-code, this is denoted by $IMPROVEMENTMETHOD$. This method can be a simple local search algorithm or a more sophisticated one such as variable neighborhood search

```
begin GUIDEDLOCALSEARCH(p, g, λ, [I_1, ..., I_M], [c_1, ..., c_M], M)
1   k ← 0;
2   s_0 ← CONSTRUCTIONMETHOD(p);
3   for i = 1, ..., M do
4       p_i ← 0;
5   end-for;
6   h ← g + λ * ∑ p_i * I_i;
7   while stopping criterion not satisfied do
8       s_{k+1} ← IMPROVEMENTMETHOD(s_k, h);
9       for i = 1, ..., M do
10          util_i ← I_i(s_{k+1}) * c_i/(1 + p_i);
11      end-for;
12      forall i such that util_i is maximum do
13          p_i ← p_i + 1;
14      end-for;
15      k ← k + 1;
16  end-while;
17  s* ← best solution found with respect to objective function g;
18  return s*;
end GUIDEDLOCALSEARCH.
```

Fig. 1 The guided local search algorithm

[44], variable depth search [61], ejection chains [41], or combinations of local search methods with exact search algorithms [77].

It is not essential for the improvement method to generate high-quality local minima. Experiments with GLS and various local heuristics reported in [103] have shown that high-quality local minima take time to produce, resulting in less intervention by GLS in the overall allocated search time. This may sometimes lead to inferior results compared to a simple but more computationally efficient improvement method.

Note also that the improvement method is using the augmented objective function instead of the original one.

Indicator Functions and Feature Penalization

Given that a construction and an improvement method are available for the problem, the rest of the pseudo-code is straightforward to apply. The penalties of features are initialized to zero and they are incremented for features that maximize the utility formula, after each call to the improvement method.

The indicator functions I_i for the features rarely need to be implemented. Looking at the values of the decision variables can directly identify the features present in a local minimum. When this is not possible, data structures with constant time deletion/addition operations (e.g., based on double-linked lists) can incrementally maintain the set of features present in the working solution, thus avoiding the need for an expensive computation when GLS reaches a local minimum.

The selection of features to penalize can be efficiently implemented by using the same loop for computing the utility formula for features present in the local minimum (the other features can be ignored) and also placing features with maximum utility in a vector. With a second loop, the features with maximum utility contained in this vector have their penalties incremented by one.

Parameter λ

Parameter λ is the only parameter of the GLS method (at least in its basic version) and in general is instance dependent. Fortunately, for several problems, it has been observed that good values for λ can be found by dividing the value of the objective function of a local minimum with the number of features present in it. In these problems, λ is dynamically computed after the first local minimum and before penalties are applied to features for the first time. The user only provides parameter α, which is relatively instance independent. The recommended formula for λ as a function of α is the following:

$$\lambda = \alpha * g(x^*)/(\text{no. of features present in } x^*) \tag{4}$$

where g is the objective function of the problem and x^* is a local minimum. Tuning α can result in λ values, which work for many instances of a problem class. Another benefit from using α is that, once tuned, it can be fixed in industrialized versions of the software, resulting in ready-to-use GLS algorithms for the end user.

Augmented Objective Function and Move Evaluations

With regard to the objective function and the augmented objective function, the program should keep track of the actual objective value in all operations related to storing the best solution or finding a new best solution. Keeping track of the value of the augmented objective value (e.g., after adding the penalties) is not necessary since local search methods will be looking only at the differences in the augmented objective value when evaluating moves.

However, the move evaluation mechanism needs to be revised to work efficiently with the augmented objective function. Normally, the move evaluation mechanism is not directly evaluating the objective value of the new solution generated by the move. Instead, it calculates the difference Δg in the objective function. This difference should be combined with the difference in the amount of penalties. This can be easily done and has no significant impact on the time needed to evaluate a move. In particular, we have to take into account only features whose state changes (being deleted or added). The penalties of the features deleted are summed together. The same is done for the penalties of added features. The change in the amount of penalties due to the move is then simply given by the difference:

$$\sum_{\text{over all features j added}} p_j - \sum_{\text{over all features k deleted}} p_k \tag{5}$$

which then has to be multiplied by λ and added to Δg.

Another minor improvement is to monitor the actual objective value not only for the solutions accepted by the local search but also for those evaluated. Since local search is using the augmented objective function, a move that generates a new best solution may be missed. From our experience, this modification does not improve significantly the performance of the algorithm although it can be useful when GLS is used to find new best-known solutions to hard benchmark instances.

Stopping Criterion

There are many choices possible for the stopping criterion. Since GLS is not trapped in local minima, it is not clear when to stop the algorithm. Like other meta-heuristics, we usually resort to a measure related to the length of the search process. For example, we may choose to set a limit on the number of moves performed, the number of moves evaluated, or the CPU time spent by the algorithm. If a lower bound is known, we can utilize it as a stopping criterion by setting the gap to be achieved between the best-known solution and the lower bound. Criteria can also be combined to allow for a more flexible way to stop the GLS method. For practical applications of the algorithm such as the optimal routing of vehicles, the stopping criterion tends to be based on running times and the ability to generate near optimal solutions within seconds given the dynamic nature of these problems [86].

Possible Features for Common Applications

Applying guided local search to a problem requires identifying a *suitable* set of features to guide the search process. Features provide the heuristic search expert with quite a powerful tool since any solution property can be potentially captured and used to guide local search. Usually, we are looking for solution properties, which have a direct impact on the objective function. These can be modeled as features with feature costs equal or analogous to their contribution to the objective

function value. By applying penalties to features, GLS can guide the improvement method to avoid costly ("bad") properties, converging faster towards areas of the search space, which are of high quality.

Features are not necessarily specific to a particular problem and they can be used in several problems of similar structure. Real-world problems, which sometimes incorporate elements from several academic problems, can benefit from using more than one feature set to guide the local search in better optimizing the different terms of a complex objective function.

Below, we provide examples of useful features for several representative problems from various domains.

Routing/Scheduling Problems

In routing/scheduling problems, one is seeking to minimize the time required by a vehicle to travel between customers or for a resource to be set up from one activity to the next. Problems in this category include the traveling salesman problem, vehicle routing problem, machine scheduling with sequence-dependent setup times, and others.

Travel or setup times are modeled as edges in a path or graph structure commonly used to represent the solution of these problems. The objective function (or at least part of it) is given by the sum of lengths for the edges used in the solution.

Edges are ideal GLS features. A solution either contains an edge or not. Furthermore, each edge can be considered to have a feature cost equal to its length. We can define a feature for each possible edge and assign a cost to it equal to the edge length. GLS quickly identifies and penalizes long and costly edges guiding local search to high-quality solutions, which use as much as possible the short edges available.

Assignment Problems

In assignment problems, a set of items has to be assigned to another set of items (e.g., airplanes to flights, locations to facilities, people to work, etc.). Each assignment of item i to item j usually carries a cost, and depending on the problem, a number of constraints are required to be satisfied (e.g., capacity or compatibility constraints). The assignment of item i to item j can be seen as a solution property which is either present in the solution or not. Since each assignment also carries a cost, this is another good example of a feature to be used in a GLS implementation.

In some variations of the problem such as the quadratic assignment problem, the cost function is more complicated and assignments have an indirect impact on the cost. Even in these cases, we found that GLS can generate good results by assigning the same feature costs to all features (e.g., equal to 1 or some other arbitrary value). Although GLS is not guiding the improvement method to good solutions (since this information is difficult to extract from the objective function), it can still diversify the search because of the penalty memory incorporated and it produces results comparable to popular heuristic methods.

Resource Allocation Problems

Assignment problems can be used to model resource allocation applications. A special but important case in resource allocation is when the resources available are not sufficient to service all requests. Usually, the objective function will contain a sum of costs for the unallocated requests, which is to be minimized. The cost incurred when a request is unallocated will reflect the importance of the request or the revenue lost in the particular scenario.

A possible feature to consider for these problems is whether a request is unallocated or not. If the request is unallocated, then a cost is incurred in the objective function, which we can use as the feature cost to guide local search. The number of features in a problem is equal to the number of requests that may be left unallocated, one for each request. There may be hard constraints which state that certain requests should always be allocated a resource, in which case there is no need to define a feature for them. Problems in this category include the path assignment problem [10], maximum channel assignment problem [87], and workforce scheduling problem [13, 22]. Allocation of workloads to data centers in the context of cloud infrastructure using GLS, which represents a more complex real-world application, is discussed in [74].

Constrained Optimization Problems

Constraints are very important in capturing processes and systems in the real world. A number of combinatorial optimization problems deal with finding a solution, which satisfies a set of constraints or, if that is not possible, minimizes the number of constraint violations (relaxations). Constraint violations may have costs (weights) associated with them; in which case the sum of constraint violation costs is to be minimized.

Local search usually considers the number of constraint violations (or their weighted sum) as the objective function even in cases where the goal is to find a solution which satisfies all the constraints. Constraints by their nature can be easily used as features. They can be modeled by indicator functions and they also incur a cost (i.e., when violated/relaxed), which can be used as their feature cost. Problems which can benefit from this modeling include the constraint satisfaction and partial constraint satisfaction problem, the famous SAT and its MAX-SAT variant, graph coloring, various frequency assignment problems [1, 70], and others.

Packing and Placement Problems

GLS has been applied to packing and placement problems where the goal is generally to place or pack as many as possible two-dimensional or three-dimensional shapes or boxes into a set of surfaces or bins [33, 35, 36, 98]. The application of GLS to these problems is often taking a similar approach to constraint optimization problems where local search is attempting to minimize and ultimately eliminate the number of overlaps between shapes or objects (equivalent to constraint violations) by either altering their location, orientation, or allocation to the containing surfaces or bins.

The overall size of the surface or number of bins utilized is typically the overarching optimization criterion. GLS has been tested beyond classical problems in the context of practical applications such as strip packing of irregular shapes [33, 98] and also circuit module placement in VLSI design [36].

Features utilized represent the overlaps to be eliminated with feature cost reflecting the degree of overlap between objects and also the size of the objects where it is assumed that larger objects overlapping are less desirable than smaller ones [35]. In certain problems such as VLSI design, the rectilinear distance between shapes may also be used as feature to minimize the overall wiring required to connect the individual modules of an overall circuit [36].

Guided Fast Local Search

In this section, we look at the combination of guided local search with fast local search, a generalized algorithm for speeding up local search, resulting in the guided fast local search method. Guided fast local search addresses the issue of slow local search procedures and it is particularly useful when applying GLS to tackle large-scale problem instances.

One factor which affects the efficiency of a local search algorithm is the size of the neighborhood. If too many neighbors are considered, then the search could be very costly. This is especially true if the search takes many steps to reach a local optimum, and/or each evaluation of the objective function requires a significant amount of computation. Bentley presented in [19] the *approximate 2-opt* method to reduce the neighborhood of 2-opt in the TSP. We have generalized this method to a method called *fast local search* (FLS). The principle is to use heuristics to identify (and ignore) neighbors that are unlikely to lead to improving moves in order to enhance the efficiency of a search.

The neighborhood chosen for the problem is broken down into a number of small sub-neighborhoods and an activation bit is attached to each one of them. The idea is to scan continuously the sub-neighborhoods in a given order, searching only those with the activation bit set to 1. These sub-neighborhoods are called active sub-neighborhoods. Sub-neighborhoods with the bit set to 0 are called inactive sub-neighborhoods and they are not being searched. The neighborhood search process does not restart whenever we find a better solution but it continues with the next sub-neighborhood in the given order. This order may be static or dynamic (i.e., change because of the moves performed).

Initially, all sub-neighborhoods are active. If a sub-neighborhood is examined and does not contain any improving moves, then it becomes inactive. Otherwise, it remains active and the improving move found is performed. Depending on the move performed, a number of other sub-neighborhoods are also activated. In particular, we activate all the sub-neighborhoods where we expect other improving moves to occur as a result of the move just performed. As the solution improves, the process dies out with fewer and fewer sub-neighborhoods being active until all the sub-neighborhood

bits turn to 0. The solution formed up to that point is returned as an approximate local optimum.

The overall procedure could be many times faster than conventional local search. The bit setting scheme encourages chains of moves that improve specific parts of the overall solution. As the solution becomes locally better, the process is settling down, examining fewer moves and saving enormous amounts of time which would otherwise be spent on examining predominantly bad moves.

Although FLS procedures do not generally find very good solutions, when they are combined with GLS, they become very powerful optimization tools. Combining GLS with FLS is straightforward. The key idea is to associate features to sub-neighborhoods. The associations to be made are such that for each feature we know which sub-neighborhoods contain moves that have an immediate effect upon the state of the feature (i.e., moves that remove the feature from the solution).

By reducing the size of the neighborhood, one may significantly reduce the amount of computation involved in each local search iteration. The idea is to enable more local search iterations in a fixed amount of time. The danger of ignoring certain neighbors is that some improvements may be missed. The hope is that the gain in "search speed" outweighs the loss in "search quality."

Implementation Guideline

Guided fast local Search (GFLS) is more sophisticated than the basic GLS algorithm as it uses a number of sub-neighborhoods, which are enabled/disabled during the search process. The main advantage of GFLS lies in its ability to focus the search after the penalties of features are increased. This can dramatically shorten the time required by an improvement method to re-optimize the solution each time the augmented objective function is modified.

The following provides the pseudo-code for the method and also some suggestions on how to achieve an efficient implementation. We first look at the pseudo-code for fast local search, which is part of the overall guided fast local search algorithm.

Fast Local Search

The pseudo-code for fast local search is given in Fig. 2, where s is the solution, h is the augmented objective function, L is the number of sub-neighborhoods, bit_i is the activation bit for sub-neighborhood i, $MOVESFORSUBNEIGHBOURHOOD(i)$ is the method which returns the set of moves contained in sub-neighborhood i, and $SUBNEIGHBOURHOODSFORMOVE(m)$ is the method which returns the set of sub-neighborhoods to activate when move m is performed.

As explained earlier, the problem's neighborhood is broken down into a number of sub-neighborhoods and an activation bit is attached to each one of them. The idea is to examine sub-neighborhoods in a given order, searching only those with the activation bit set to 1. The neighborhood search process does not restart whenever we find a better solution but it continues with the next sub-neighborhood in the given

```
begin FASTLOCALSEARCH(s, h, [bit_1, ..., bit_L], L)
1   while ∃bit, bit = 1 do
2       for i = 1, ..., L do
3           if bit_i = 1 then
4               MOVES ← MOVESFORSUBNEIGHBOURHOOD(i);
5               forall move m in MOVES do
6                   s' ← m(s);
7                   if h(s') < h(s) then
8                       ACTIVATESET ← SUBNEIGHBOURHOODSFORMOVE(m);
9                       forall sub-neighbourhood j in ACTIVATESET do
10                          bit_j ← 1;
11                      end-for;
12                      s ← s';
13                      GOTO: IMPROVINGMOVEFOUND;
14                  end-if;
15              end-for;
16              bit_i ← 0;
17          end-if;
18      IMPROVINGMOVEFOUND: continue
19      end-for;
20  end-while;
21  return s;
end FASTLOCALSEARCH.
```

Fig. 2 The fast local search algorithm

order. The pseudo-code given above is flexible since it does not specify which bits are initially switched on or off, something which is an input to the procedure. This allows the procedure to be focused to certain sub-neighborhoods and not the whole neighborhood, which may be a large one.

The procedure has two points that need to be customized. The first is the breaking down of the neighborhood into sub-neighborhoods (*MOVESFORSUBNEIGHBOURHOOD* method in pseudo-code). The second is the mapping from moves to sub-neighborhoods for spreading activation (*SUBNEIGHBOURHOODSFORMOVE* method in pseudo-code). Both points are strongly dependent on the move operator used.

In general, the move operator depends on the solution representation. Fortunately, several problems share the same solution representation which is typically based on some well-known simple or composite combinatorial structure (e.g., selection, permutation, partition, composition, path, cyclic path, tree, graph, etc.). This allows us to use the same move operators for many different problems (e.g., 1-opt, 2-opt, swaps, insertions, etc.).

The method for mapping sub-neighborhoods to moves, which is denoted in the pseudo-code by *SUBNEIGHBOURHOODSFORMOVE*, can be defined by looking at the implementation of a typical local search procedure for the problem. This implementation, at its core, will usually contain a number of nested for-loops for generating all possible move combinations. The variable in the outermost loop in the move generation code can be used to define the sub-neighborhoods. The moves in each sub-neighborhood will be those generated by the inner loops for the particular sub-neighborhood index value at the outermost loop.

In general, the sub-neighborhoods can be overlapping. Fast local search is usually examining a limited number of moves compared to exhaustive neighborhood search methods, and therefore, duplication of moves is not a problem. Moreover, this can be desirable sometimes to give a greater range to each sub-neighborhood. Since not all sub-neighborhoods are active in the same iteration, if there is no overlapping, some improving moves may be missed.

The method for spreading activation, denoted by *SUBNEIGHBOURHOODSFORMOVE*, returns a set of sub-neighborhoods to activate after a move is performed. The lower bound for this set is the sub-neighborhood where the move originated. The upper bound (although not useful) is all the sub-neighborhoods in the problem.

A way to define this method is to look at the particular move operator used. Moves will affect part of the solution directly or indirectly while leaving other parts unaffected. If a sub-neighborhood contains affected parts, then it needs to be activated since an opportunity could arise there for an improving move as a result of the original move performed.

The fast local search loop is settling down in a local minimum when all the bits of sub-neighborhoods turn to zero (i.e., no improving move can be found in any of the sub-neighborhoods). Fast local search in that respect is similar to other local search procedures. The main differences are that the method can be focused to search particular parts of the overall neighborhood, and secondly, it is working in an opportunistic way looking at parts of the solution which are likely to contain improving moves rather than the whole solution. In the next section, we look at guided fast local search, which uses fast local search as its improvement method.

Guided Fast Local Search

The pseudo-code for guided fast local search is given in Fig. 3, where *FASTLOCALSEARCH* is the fast local search method as described in section "Fast Local Search," *SUBENEIGHBORHOODSFORFEATURE* is the method which returns the set of sub-neighborhoods to activate when feature i is penalized, and the rest of the

```
begin GUIDEDFASTLOCALSEARCH(p, g, λ, [I_1, ..., I_M], [c_1, ..., c_M], M, L)
1   k ← 0;
2   s_0 ← CONSTRUCTIONMETHOD(p);
3   for i = 1, ..., M do
4       p_i ← 0;
5   end-for;
6   for i = 1, ..., L do
7       bit_i ← 1;
8   end-for;
9   h ← g + λ * Σ p_i * I_i;
10  while stopping criterion not satisfied do
11      s_{k+1} ← FASTLOCALSEARCH(s_k, h, [bit_1, ..., bit_L], L);
12      for i = 1, ..., M do
13          util_i ← I_i(s_{k+1}) * c_i/(1 + p_i);
14      end-for;
15      forall i such that util_i is maximum do
16          p_i ← p_i + 1;
17          ACTIVATESET ← SUBNEIGHBORHOODSFORFEATURE(i);
18          forall sub-neighbourhood j in ACTIVATESET do
19              bit_j ← 1;
20          end-for;
21      end-for;
22      k ← k + 1;
23  end-while;
24  s* ← best solution found with respect to objective function g;
25  return s*;
end GUIDEDFASTLOCALSEARCH.
```

Fig. 3 The guided fast local search algorithm

definitions are the same than those used in the pseudo-code for GLS described in section "Implementation Guideline."

This pseudo-code is similar to that of guided local search explained in section "Implementation Guideline." All differences relate to the manipulation of activation

bits for the purpose of focusing fast local search. These bits are initialized to 1. As a result, the first call to fast local search is examining the whole neighborhood for improving moves. Subsequent calls to fast local search examine only part of the neighborhood and in particular all the sub-neighborhoods that relate to the features penalized by GLS.

Identifying the sub-neighborhoods that are related to a penalized feature is the task of the *SUBENEIGHBORHOODSFORFEATURE* method. The role of this method is similar to that of the *SUBNEIGHBOURHOODSFORMOVE* method in fast local search (see section "Fast Local Search"). The *SUBENEIGHBORHOODSFORFEATURE* method is usually defined based on an analysis of the move operator. After the application of penalties, we are looking for moves which remove or have the potential to remove penalized features from the solution. The sub-neighborhoods, which contain such moves, are prime candidates for activation. Specific examples will be given later in the chapter and in the context of GLS applications.

GLS Extensions, Hybrids, and Variations

GLS is closely related to other heuristic and meta-heuristic methods. In this section, we shall review the different variations, hybrids, and extensions of GLS and FLS that have been developed over the years.

Extensions to GLS

GLS is closely related to other heuristic and meta-heuristic methods. This motivates the adoption of ideas borrowed from other meta-heuristics in GLS. For example, taboo lists and aspiration ideas from tabu search have been used in versions of GLS. Resembling the tabu list idea, a limited number of penalties are used when GLS is applied to the radio link frequency assignment problem [70]. When the list is full, old penalties are overwritten [102]. The motive is that if too many penalties are built up during the search, the local search could be misguided. In another GLS work, aspiration is used to favor promising moves [68].

Tairan and Zhang in [91] studied how to enhance the performance of GLS through designing a cooperative mechanism based on the proximate optimality principle (POP). The idea is to run multiple agents of GLS and during the search the agents exchange their obtained information about the previous search to make their further search more rational. Based on POP, they suggested that common features that appear in many locally optimal solutions of GLS agents are more likely to be parts of the globally optimal solution. Thus, this property should be taken into consideration in the penalization stage during the search. The effectiveness of the proposed cooperative method was demonstrated through high-quality results obtained in the TSP. A similar approach, again in the context of the TSP, is presented in [2] where the agents exchange information utilizing the crossover operator to preserve the good components of the best solutions found by the agents during the search.

GLS Hybrids

Being simple and general, GLS ideas can easily be combined with other techniques. GLS has been hybridized with several meta-heuristics creating efficient frameworks which were successfully applied to several applications. Below, we review and comment on some of these hybrids of GLS.

As a meta-heuristic method, GLS can also sit on top of genetic algorithms (GA). This has been demonstrated in the guided genetic algorithm (GGA) [57–59]. GGA is a hybrid of GA and GLS. It is designed to extend the domain of both GA and GLS. One major objective is to further improve the robustness of GLS. It can be seen as a GA with GLS to bring it out of local optima: if no progress has been made after specific iterations (this number is a parameter of GGA), GLS modifies the fitness function (which is the objective function) by means of penalties, using the criteria defined in (Eq. 3). GA will then use the modified fitness function in future generations. The penalties are also used to bias crossover and mutation in GA – genes that contribute more to the penalties are more likely to be changed by these two GA operators. This allows GGA to be more focused in its search.

On the other hand, GGA can roughly be seen as a number of GLSs running in parallel from different starting points and exchanging material in a GA manner. The difference is that only one set of penalties is used in GGA, whereas parallel GLSs could have used one independent set of penalties per run. Besides, learning in GGA is more selective than parallel GLS: the updating of penalties is only based on the best chromosome found at the point of penalization.

GLS was hybridized with two major evolutionary computation (EC) techniques, namely, estimate distribution algorithm (EDA) and evolution strategy (ES). The hybrid of GLS with EDA was introduced by Zhang et al. [112]. They proposed a framework that incorporates GLS within EDA, in which GLS is applied to each solution in the population of EDA. The framework is successfully applied to the quadratic assignment problem. The results show the superiority of EDA/GLS over GLS alone.

The hybrid of GLS with ES was first studied by Mester and Braysy [65]. The resulting framework combines GLS and ES into an iterative two-stage procedure called active guided evolution strategies (AGES). GLS is used in both phases to improve the local search in the first stage and to regulate the objective function and the neighborhood of the modified ES in the second stage. The principle of FLS is also incorporated into the idea of penalty variable neighborhood in which the neighborhood considered by the local search is limited to a small set of the neighbors of the penalized feature.

GLS has also been hybridized with variable neighborhood search (VNS) and large neighborhood search (LNS). Kytojoki et al. [55] combine GLS with VNS in an efficient variable neighborhood search heuristic, named guided VNS (GVNS), which was applied to the vehicle routing problem. The addition to VNS is the use of GLS to escape local minima. The idea of threshold value borrowed from threshold accepting (TA) is used as a termination condition for every GLS stage. The hybrid of GLS with LNS is introduced in [106]. In the proposed framework, LNS is applied when the GLS cannot escape a local optimum after a number of

penalizations, with the aim of increasing the diversity and exploring more promising parts of the search space. The effectiveness of this hybrid was demonstrated through high-quality results obtained in a planning optimization problem.

Guided tabu search (GTS) is a hybrid meta-heuristic which combines GLS with TS proposed by Tarantilis et al. [93, 94] to solve the vehicle routing problem with heterogeneous fleet and then extended to solve another variant of the same general problem. The basic idea is to control the exploration of TS by a guiding mechanism, based on GLS, that continuously modifies the objective function of the problem. The authors propose a new arc (as a feature) selection strategy which considers the relative arc length according to the rest of the customers ($d_{i,j}/avg_{i,j}$ rather than $d_{i,j}$, where $avg_{i,j}$ is the average value of all outgoing arcs from i and j). They argue that this would lead to a more balanced arc selection, which should improve upon the most frequently employed strategy based on $d_{i,j}$ only. Experimental results confirm the effectiveness of GTS, producing new best results for several benchmarks. De Backer et al. [14] also proposed a guided tabu search hybrid in their work on the vehicle routing problem (VRP). Bai et al. [15] proposed a multi-start GLS variant assisted by tabu search for the freight service design problem.

GLS has been also successfully hybridized with ant colony optimization (ACO) by Hani et al. [43]. This hybrid algorithm was applied to the facility layout problem, a variant of the quadratic assignment problem (QAP). The basic idea is simple: GLS sits on top of the basic LS in the ACO.

The hybridization of GLS and constraint programming (CP) was introduced by Gomes et al. [42]. This method, named guided constraint search, borrows ideas from GLS to improve the efficiency of pure CP methods. The basic principle is to use a fitness function to choose at each iteration only the N most promising values of each variable's domain, defining a subspace for the CP method. The selection strategy is inspired from GLS; for each pair, a utility function, penalty parameter, and cost are defined. At each iteration, those features (variable/value pairs) which were considered but did not belong to new best solution are deemed bad features and they are penalized.

Variations of GLS

The success of GLS motivated researchers to invent new algorithms inspired from GLS, borrowing the ideas of features, penalties, and utilities. Below, we briefly describe such GLS-inspired algorithms.

Partially based on GLS, which is a centralized algorithm, Basharu et al. [18] introduce an improved version for solving distributed constraint satisfaction problems. The distributed guided local search (Dis-GLS) incorporates additional heuristics to enhance its efficiency in distributed scenarios. The algorithm has been successfully applied to the distributed version of the graph coloring problem producing promising results compared to other distributed search algorithms.

Hifi et al. [46] introduced a variant of GLS by proposing a new penalization strategy. The principle is to distinguish two phases in the search process, namely, the

penalty and normal phases. The search process switches between the two phases in order to either escape local optima or diversify the search to explore another feasible space. The computational results confirm the high quality of solutions obtained by the proposed variant.

Tamura et al. [92] propose an improved version of GLS, named the objective function adjustment (OA) algorithm which incorporates the idea of features (from GLS) alongside the concept of energy function.

Shi et al. [85] modified the penalization mechanism of GLS in the form of elite-based GLS (EB-GLS). EB-GLS records and updates an elite solution, which is the best solution found so far during the search. Features present in the recorded elite solution are more likely to be part of a global optimum; thus, EB-GLS reduces the probabilities of these features being penalized. This introduces an additional intensification mechanism which according to the authors of [85] may perform well for problems where high-quality solutions are closely clustered together.

Vansteenwegen et al. [99] introduced a diversification modification instead. In addition to penalizing features included in the current local optimum, features not included undergo an inverse version of the penalizing mechanism rewarding them instead.

In the GLS variant proposed by Zhong and Cole [113] for the vehicle routing problem, only features whose utilities are larger than a certain value are penalized. Furthermore, penalties have an upper bound introduced.

GLS and Artificial Intelligence

The GLS algorithm has its roots in artificial intelligence. It was inspired by GENET [30, 97] a neural network architecture for solving constrain satisfaction problems. Furthermore, both GLS and GENET adopt an approach to escaping local minima which is closing resembling reinforcement learning [97]. With the resurgence of AI and in particular developments in deep learning and large language models, there has been recent research on leveraging GLS in combination with AI approaches.

GLS features and associated costs can be the subject and outcome of a learning process with the view of optimizing algorithmic performance. More specifically, GNNGLS by Hudson et al. [49] and neural-GLS by Sui et al. [89] both adopt neural network models to learn the feature costs (referred to as "regret) ahead of applying GLS to random and classic TSP instances. In [49], a graph neural network is deployed for this task, while in [89] a graph convolutional network is considered alongside a more complicated strategy with better overall performance. Considering the reverse paradigm, GLS can be utilized to improve the training performance of neural networks. An example of such an approach is present in [45] in the context of a robotic-assisted intelligence system.

Variants of GLS based on different feature definitions, feature costs, construction heurists, and local search methods can be synthesized by LLMs evolving the structure of the algorithm and subsequently altering and optimizing further its performance. Liu et al. [62] proposed such a large language model framework

named AEL-GLS to maintain a population of GLS algorithms for the TSP. AEL-GLS utilizes a general evolutionary framework, comprising initialization, crossover, and mutation, to evolve new algorithms. The generation of novel GLS algorithms occurs during population initialization, as well as through crossover and mutation operations in the evolutionary process. The introduction of LLM enables such algorithm-level creations. The algorithm evolution process is entirely automated, without any human intervention.

In another example of this approach, ReEvo presented in [108] also maintains and evolves a population of GLS variants for the TSP, where insights and knowledge are verbalized as long-term reflections and accumulated throughout iterations to accelerate and improve heuristic design. The more elaborate approach of ReEvo compared to AEL-GLS claims to reduce running times from days in [62] to less than an hour in [108] with ability to generate a GLS variant that outperforms both human-designed and machine-designed variations of the method.

Another promising direction, not fully explored, is that of combining software agents and GLS. In a typical setup (e.g., see [16,91]), a collection of software agents using GLS cooperate to solve a problem by evolving a population of solutions. The approach has found application in the context of the dynamic VRP which requires periodic re-optimization of static subproblems, including requests, which have already arrived to the system as this is explained in [17].

We expect more research to emerge combining agents with deep learning and LLMs to evolve and apply GLS variants (or variants of other meta-heuristics for that matter) in the context of addressing large-scale and dynamic real-world problems. Individual agents could be seen applying deep learning and LLMs to improve the search heuristics they deploy as they gain experience from solving a series of problems within a specific domain while, in parallel, they also collaborate with other agents to share the search workload. This is essential so that rapid responses are achieved against even the largest and most dynamic real-world problems.

GLS for Multi-objective Optimization

Most real-world optimization problems are multi-objective in nature. The multi-objective optimization problem (MOP) concerns the optimization of two or more objectives simultaneously. Instead of searching for a global optimum solution as in single-objective optimization problems, the search in MOPs targets a set of solutions representing the optimum set of trade-offs between the objectives. This set is known interchangeably as the Pareto optimum set or the efficient solutions, and the objective values of these solutions are located at the Pareto front (PF). Efficient solutions are nondominant solutions in the sense that improving the value of any one of their objectives must be at the expense of degrading the quality of one or more of the other objectives. Thus, all efficient solutions are considered equivalent as long as there is no further information regarding the relative importance of each of the objectives.

Pareto Local Search

A simple, intuitive adaptation of local search to contain multiple objectives is to apply the Pareto domination as an acceptance criterion when comparing the current solution to the new one. An archive of nondominant solutions discovered during the search is maintained in order to produce an approximation to the PF. The algorithm stops when the neighborhood of all solutions in the archive have been explored, i.e., the archive is a Pareto local optimum set. Recently, this idea was termed Pareto local search (PLS) and has been developed and extended by several researchers such as in [73]. PLS can be outlined, from a high-level perspective, as follows:

1. It starts with a randomly or heuristically generated solution that is added to the $archive$.
2. A non-visited candidate solution in the $archive$ is randomly chosen, and its neighborhood is explored while applying a first-improvement strategy (i.e., any better solution found is accepted immediately, and its neighborhood is explored for further improvement). This is an iterative step which is applied every time a new better solution is found and accepted.
3. The $archive$ is updated with any non-dominated neighbor, i.e., the new neighbor is added to the $archive$ only if it is non-dominated by another solution in the $archive$, and then all existing solutions that are dominated by the new neighbor are removed from the $archive$.
4. The exploration of the neighborhood of the current solution stops when all of the neighbors are visited. When the exploration stops, the current solution is considered as a Pareto local optimum.
5. The current solution is marked as being visited.
6. The procedure continues at step 2, while there is a solution in the $archive$ that is not visited.

The PLS returns the $archive$, which is a *Pareto local optimal set*.

Guided Pareto Local Search

Guided Pareto local search (GPLS) [6, 7] extends the guidance approach in GLS to guide PLS to escape Pareto local optimum sets. The only change that GPLS makes to the underlying PLS is the replacement of each objective function g_i with an augmented objective function h_i during the evaluation of neighbor solutions. The augmented functions are not used in the updating procedure of the *archive*, which always depends only on the original objective functions.

In GPLS, the definition of features has to be derived from all objectives. Thus, a set (or multiple sets) of features has to be defined, which will be shared by all solutions. In multi-objective optimization problems, one should take into consideration two different scenarios:

1. All objectives have the same structure (e.g., putting an item in all knapsacks in the Knapsack problem, or an edge in the multi-objective TSP), and therefore, they share one defined feature set. However, the cost of a feature varies according to a particular objective. In order to define the cost of a feature in this case, the influence of the feature on each objective has to be considered and modelled into a single cost function. Such models include, for example, (weighted) aggregating approaches and other general functions such as min, max, or mean.
2. A distinct feature set needs to be defined for each objective, and a cost is associated with each feature to describe its influence on the corresponding objective. In this case, features from all feature sets should be considered at the penalization phase to pick one or more features to penalize.

The guidance strategy that GLS employs relies on penalizing some features exhibited by the recent local optimum. As described in section "Guided Local Search," the novelty of GLS is mainly in the way it selects features to penalize. The target is to penalize "bad features" when the local search settles in a local optimum. Two factors affect the utility of a feature, namely, its cost and the frequency of penalizing this feature in previous penalizations.

GPLS deals with a Pareto local optimum set instead of a local optimum. A straightforward penalization strategy is to evaluate the utility of all features exhibited by *any* nondominant solution in the *archive* and penalize features with maximum utility (Eq. 3). However, this simple strategy does not incorporate any information from the *archive* (i.e., the Pareto local optimum set). An example of important information that can be extracted from the *archive* is the number of nondominant solutions that exhibit a particular feature. This is another factor that can be incorporated into the utility function, such that the more Pareto local optima that exhibit a feature, the greater its utility of penalization. Recall that when a feature is penalized, only those Pareto local optima that exhibit the penalized feature will have the chance to escape. Therefore, increasing the utility of penalizing a feature that occurs in many Pareto local optima would enhance the chance of escaping a Pareto local optimum set by restarting from more solutions. It would also help to prevent the search from directing all its efforts towards any particular region of the PF, which therefore leads to a better spread over the PF. Bad features, in terms of their cost, are hoped to be removed either during the next calls of PLS or by future penalization.

The utility function, as stated in Eq. 3, is redefined to incorporate the number of solutions in the Pareto local optimum set that exhibits this feature (γ_i):

$$util_i(archive) = I_i(archive) \times \frac{c_i \times (\gamma_i / |archive|)}{1 + p_i} \tag{6}$$

where $archive$ is a Pareto local optimum set, $I_i(archive)$ indicates whether "at least" one solution in the *archive* exhibits feature i, c_i the feature's cost, p_i the

penalty, and $|archive|$ is the size of the Pareto local optimum set. The feature with the greatest $util$ value will be penalized. When a feature is penalized, its penalty value is always incremented by 1.

This utility function redefines the term "bad feature." If a feature is not exhibited in the Pareto local optimum set (indicated by I_i), then the utility of penalizing it is 0. The higher the cost of this feature (the greater c_i) and the more nondominant solutions exhibiting it (the greater γ_i), the greater the utility of penalizing it. Furthermore, the more times that it has been penalized (the greater p_i), the lower the utility of penalizing it again.

Having penalized a feature, all solutions in the $archive$ that exhibit this feature need to be marked as "non-visited," so as to be considered by the PLS method in the next iteration.

The lambda parameter λ is the only parameter to GLS which determines the scaling of the penalty. In multi-objective scenarios each objective ideally has its own lambda, which is calculated as a function of a local optimum with respect to the correspondent objective. Thus, GPLS requires a set of lambda parameters: $[\lambda_1, \cdots, \lambda_k]$, where λ_i is a parameter for the objective h_i. Recall that lambda can be dynamically computed after the first local optimum and before penalties are applied to features for the first time (Eq. 4).

GPLS proves its effectiveness to converge quite quickly, however, at the middle area of the Pareto front [7]. In [8], the speedy convergence property of GPLS is utilized, and the performance of GPLS is further enhanced by coupling it with an efficient initial solution set. Two GPLS-based frameworks are proposed, both of which require such an efficient initial solution set. The first framework applies the standard GPLS, after filling its archive with the initial set. The second framework is a parallel version of GPLS, where each independent GPLS run takes a solution from the initial set as a starting point. GPLS and its frameworks are successfully applied to the 0/1 multi-objective knapsack problem [7], the multi-objective TSP [6], and the empowerment-based workforce scheduling problem [6].

Other Attempts

Apart from GPLS, Alhindi et al. [4, 5] propose an idea of using GLS, as a heuristic local search procedure specific for single-objective optimization, in multi-objective evolutionary algorithms (MOEAs). In this work, GLS and multi-objective evolutionary algorithm based on decomposition (MOEA/D) [111] are combined to propose a highly efficient algorithm for solving MOPs. To this end, a combination of MOEA/D with GLS, called MOEA/D-GLS, is proposed. In MOEA/D-GLS, a MOP is decomposed into a number of single-objective subproblems. The subproblems are optimized in parallel by using neighborhood information and problem-specific knowledge. In the proposed work, GLS alternates between those subproblems to help them escape local Pareto optimal solutions. More specifically, for a subproblem

trapped in a local Pareto optimal solution, the GLS starts from its solution and constructs a transformation function (i.e., augmented function). Then, GLS optimizes the augmented function in order to identify a new better solution. The algorithm has been successfully applied to the multi-objective traveling salesman problems.

GLS Implementation on the Traveling Salesman Problem

For illustration, the implementation details of GLS on the TSP is discussed in this section. The TSP is chosen here as it is the most significant application of GLS. The full details about the application of GLS to the TSP is given in [103].

Problem Description

There are many variations of the traveling salesman problem (TSP). Here, we examine the classic symmetric TSP. The problem is defined by N cities and a symmetric distance matrix $D = [d_{ij}]$ which gives the distance between any two cities i and j. The goal is to find a tour (i.e., closed path), which visits each city exactly once and is of minimum length. A tour can be represented as a cyclic permutation π on the N cities if we interpret $\pi(i)$ to be the city visited after city i, $i = 1, \ldots, N$. The cost of a permutation is defined as:

$$g(\pi) = \sum_{i=1}^{N} d_{i\pi(i)} \tag{7}$$

and gives the cost function of the TSP.

Local Search

Solution Representation

The solution representation usually adopted for the TSP is that of a vector which contains the order of the cities in the tour. For example, the ith element of the vector will contain an identifier for the ith city to be visited. Since the solution of the TSP is a closed path, there is an edge implied from the last city in the vector to the first one in order to close the tour. The solution space of the problem is made of all possible permutations of the cities as represented by the vector.

Construction Method

A simple construction method is to generate a random tour. If the above solution representation is adopted, then all that is required is a simple procedure, which

generates a random permutation of the identifiers of the cities. More advanced TSP heuristics can be used if we require a higher-quality starting solution to be generated. This is useful in real-time/online applications where a good tour may be needed very early in the search process in case the user interrupts the algorithm. If there are no such concerns, then a random tour generator suffices since the GLS meta-heuristic tends to be relatively insensitive to the starting solution and capable of finding high-quality solutions even if it runs for a relatively short time.

Improvement Method

Most improvement methods for the TSP are based on the k-opt moves. Using k-opt moves, neighboring solutions can be obtained by deleting k edges from the current tour and reconnecting the resulting paths using k new edges. The k-opt moves are the basis of the three most famous local search heuristics for the TSP, namely, *2-opt* [27], *3-opt* [60], and *Lin-Kernighan (LK)* [61].

The reader can consider using the simple 2-opt method, which in addition to its simplicity is very effective when combined with GLS. With 2-opt, a neighboring solution is obtained from the current solution by deleting two edges, reversing one of the resulting paths and reconnecting the tour. In practical terms, this means reversing the order of the cities in a contiguous section of the vector or its remainder depending on which one is the shortest in length.

Computing incrementally the change in solution cost by a 2-opt move is relatively simple. Let us assume that edges e1 and e2 are removed and edges e3 and e4 are added with lengths d1, d2, d3, and d4, respectively. The change in cost is the following:

$$d3 + d4 - d1 - d2. \tag{8}$$

When we discuss the features used in the TSP, we will explain how this evaluation mechanism is revised to account for penalty changes in the augmented objective function.

Guided Local Search

For the TSP, a tour includes a number of edges and the solution cost (tour length) is given by the sum of the lengths of the edges in the tour (see Eq. (7)). As mentioned in section "Routing/Scheduling Problems," edges are ideal features for routing problems such as the TSP. First, a tour either includes an edge or not, and second, each edge incurs a cost in the objective function which is equal to the edge length, as given by the distance matrix $D = [d_{ij}]$ of the problem. A set of features can be defined by considering all possible undirected edges $e_{ij}(i = 1 \ldots N, j = i + 1 \ldots N, i \neq j)$ that may appear in a tour with feature costs given by the edge lengths d_{ij}. With each edge e_{ij} connecting cities i and j

is attached a penalty p_{ij} initially set to 0 which is increased by GLS during the search. When implementing the GLS algorithm for the TSP, the edge penalties can be arranged in a symmetric penalty matrix $P = [p_{ij}]$. As mentioned in section "Guided Local Search," penalties have to be combined with the problem's objective function to form the augmented objective function which is minimized by local search. We therefore need to consider the auxiliary distance matrix:

$$D' = D + \lambda \cdot P = [d_{ij} + \lambda \cdot p_{ij}]. \tag{9}$$

Local search must use D' instead of D in move evaluations. GLS modifies P and (through that) D' whenever the local search reaches a local minimum.

In order to implement this, we revise the incremental move evaluation formula (Eq. 8) to take into account the edge penalties and also parameter λ. If $p1$, $p2$, $p3$, and $p4$ are the penalties associated with edges $e1$, $e2$, $e3$, and $e4$, respectively, the revised version of (Eq. 8) is as follows:

$$(d3 + d4 - d1 - d2) + \lambda * (p3 + p4 - p1 - p2). \tag{10}$$

Similarly, we can implement GLS for higher-order k-opt moves.

The edges penalized in a local minimum are selected according to the utility function (Eq. 3), which for the TSP takes the form:

$$Util(tour, e_{ij}) = I_{e_{ij}}(tour) \cdot \frac{d_{ij}}{1 + p_{ij}}, \tag{11}$$

where

$$I_{e_{ij}}(tour) = \begin{cases} 1, & e_{ij} \in tour \\ 0, & e_{ij} \notin tour. \end{cases} \tag{12}$$

The only parameter of GLS that requires tuning is parameter λ. Alternatively, we can tune the parameter α parameter which is defined in section "Implementation Guideline" and is relatively instance independent. Experimenting with α on the TSP, we found that there is an inverse relation between α and local search effectiveness. Not so effective local search heuristics such as 2-opt require higher α values compared to more effective heuristics such as 3-opt and LK. This is probably because the amount of penalty needed to escape from local minima decreases as the effectiveness of the heuristic increases explaining why lower values for α (and consequently for λ which is a function of α) work better with 3-opt and LK. For 2-opt, the following range for α generates high-quality solutions for instances in the TSPLIB [79]:

$$1/8 \leq \alpha \leq 1/2. \tag{13}$$

The reader may refer to [103] for more details on the experimentation procedure and the full set of results.

Guided Fast Local Search

We can exploit the way local search works on the TSP to partition the neighborhood in sub-neighborhoods as required by guided fast local search. Each city in the problem may be seen as defining a sub-neighborhood, which contains all 2-opt edge exchanges removing one of the edges adjacent to the city. For a problem with N cities, the neighborhood is partitioned into N sub-neighborhoods, one for each city in the instance.

The sub-neighborhoods to be activated after a move is executed are those of the cities at the ends of the edges removed or added by the move.

Finally, the sub-neighborhoods activated after penalization are those defined by the cities at the ends of the edge(s) penalized. There is a good chance that these sub-neighborhoods will include moves that remove one or more of the penalized edges.

GLS/GPLS Implementation on a Workforce Scheduling Problem

The Workforce Scheduling Problem (WSP) is another application of GLS, which is chosen here as another illustrative example. It is chosen since both GLS and GPLS have been applied to two different versions of WSP. The implementation of GLS/GPLS components are provided here, and for full details the reader is referred to [6, 9, 96].

Problem Description

The WSP is basically the problem of allocating a set of technicians (resources): $R = \{r_1, r_2, \ldots, r_{|R|}\}$, to a set of tasks; $T = \{t_1, t_2, \ldots, t_{|T|}\}$. A task t is described by a 5-tuple:

$$< c_t, dur_t, reqSkill_t, [start_t, end_t], loc_t >$$

where c_t is a predefined priority which determines its importance to the company. The higher the value of c_t, the more important the task will be, and $c_t \in \Re$. dur_t is the expected duration a technician requires to finish this task. Each task requires a technician with a particular skill $reqSkill_t \in SkillSet$, where $SkillSet$ is the set of all skills: $SkillSet = \{skill_1, \ldots, skill_{|S|}\}$. A task t must be serviced within a predefined time window described by $[start_t, end_t]$. Tasks are geographically distributed, and the location of a task is denoted by loc_t.

Each technician $r \in R$ is described by the following triple:

$$< [start_r, end_r], skills_r, loc_r > .$$

Each technician has a limited shift hours where the beginning and end of the shift are expressed by $[br_r, er_r]$. $skills_r$ denotes the skill(s) a technician has, where $skills_r \subseteq SkillSet$. There is no single depot for technicians to start from, as they can start from home or from one of predefined depots. The location of a technician is denoted by loc_r.

There are two sets of decision variables in the WSP: the allocation variables $X = \{x_{rt} | r \in R; t \in T; x_{rt} \in \{0, 1\}\}$ and the service times $ServTime = \{st_t | t \in T\}$. A variable x_{rt} is set to 1 if the technician r is allocated to the task t, and 0 otherwise. A variable st_t denotes the start time of service for task t.

Having decided these variables, a set of routes $\pi = \{\pi_r | r \in R\}$ are defined. A route π_r is a sequence of tasks ($\subseteq T$) that are to be visited by technician r; $\pi_r = (\pi_{r1}, \cdots, \pi_{r\sigma_r}), 1 \leq \sigma_r \leq |T|$.

A main objective is to maximize the number of allocated tasks with respect to their priorities while satisfying all assignment and routing constraints.

The WSP can, then, be mathematically modelled as follows:

$$max \qquad \frac{\sum_{r \in R} \sum_{t \in T} c_t x_{rt}}{\sum_{t \in T} c_t} \tag{14}$$

subject to

$$\sum_{r \in R} x_{rt} \leqslant 1 \quad \forall t \in T \tag{15}$$

$$reqSkill_t \in skills_r \quad \forall x_{rt} = 1, t \in T, r \in R \tag{16}$$

$$start_t \leqslant st_t \quad \forall t \in T \tag{17}$$

$$st_t + dur_t \leqslant end_t \quad \forall t \in T \tag{18}$$

$$start_r \leqslant st_{\pi_{ri}} \quad \forall r \in R, i = \{1 \ldots \sigma_r\} \tag{19}$$

$$st_{\pi_{ri}} + dur_{\pi_{ri}} \leqslant end_r \quad \forall r \in R, i = \{1 \ldots \sigma_r\} \tag{20}$$

$$start_r + trvl_{loc_r loc_{\pi_{r1}}} \leqslant st_{\pi_{r1}} \forall r \in R \tag{21}$$

$$st_{t_i} + dur_{t_i} + trvl_{loc_{t_i} loc_{t_{i+1}}} \leqslant st_{t_{i+1}} \quad \forall t_i \in \pi_r, \pi_r \in \pi. \tag{22}$$

The objective function is represented by (14) which is normalized by dividing by the sum of all the costs associated with tasks. Constraint (15) imposes that each task is visited at most once. The skill constraint is expressed in (16). The time-window constraints of all tasks are assured by (17) and (18). (19) and (20) ensure that all tasks which are assigned to a technician must be within the technician's working time. Finally, constraints (21) and (22) ensure route validity by considering the traveling time between a technician's base location and the first task, as well as between subsequent tasks in technician's route.

Empowerment in Workforce Scheduling

Empowerment scheduling [6, 9] is a term introduced to refer to scheduling approaches that incorporate the empowerment concept in the scheduling system by involving technicians in the task-assignment decision. The hope of empowering human resources is to boost their contribution and commitment to the organization. In [6], an empowerment scheduling model is proposed for the workforce scheduling problem, named as the EmS-WSP, by enabling technicians to express their requests/preferences and submit plans for consideration in the allocation process. Naturally all plans can be expressed as a constraint relationship. These constraints will be considered to be satisfied in the scheduling process. Workforce scheduling problems are very complex, and thus, the promise of satisfying all the plans cannot be guaranteed. One direct way to handle employees' work plans is to treat them as soft constraints and associate a cost to each plan when it is not satisfied. In this case the task involves finding a schedule that satisfies the majority of constraints, rather than all the constraints. A key property of the empowerment scheduling model proposed in [9] is in the efficient strategy that dynamically determines the incurred cost of violating each plan; see [9] for more details.

Formally, every technician $r \in R$ is given an opportunity to provide a work plan (wp_r) every day. A plan is basically a constraint (ρ) by which a technician can describe the jobs he/she wants to undertake. For instance, a technician r might want to be allocated to jobs of a particular set of skills, or jobs in a particular region. Each plan is associated with a weight (ω); in our case this is equal to the EP value of r.

An indicator $y_r (\forall r \in R)$ is defined which is equal to 1 if the plan ρ_r is satisfied, and 0 otherwise.

The WSP is reformulated in the EmS-WSP to not only maximize the productivity in terms of the number of served jobs (Eq. 14), but also to satisfy the maximum number of working plans. The formulation of the additional objective is as follows:

$$wp_r = < \rho_r, \omega_r > \quad \forall r \in R, \quad 0 < \omega_r < 1 \tag{23}$$

$$max \qquad \frac{\sum_{r \in R} y_r \omega_r}{\sum_{r \in R} \omega_r}. \tag{24}$$

Local Search

Before going to the implementation details, it is worth mentioning that the two objectives in the EmS-WSP can be transformed to a single objective using an aggregation approach and then solved using a single-objective optimizer such as GLS. Otherwise, a multi-objective optimizer such as GPLS can be used to optimize the two objectives simultaneously. Since both GLS and GPLS are similar in the definition of their basic components, the following implementation details are for both algorithms.

Solution Representation

A candidate solution (i.e., a possible schedule) by a permutation of the jobs. Each permutation is mapped into a schedule using the deterministic algorithm described below:

Procedure **Evaluation** (input: one particular permutation of jobs)

1. For each job, order the qualified engineers in ascending order of the distances between their bases and the job (such orderings only need to be computed once and recorded for evaluating other permutations).
2. Process one job at a time, following their ordering in the input permutation. For each job x, try to allocate it to an engineer according to the ordered list of qualified engineers:
   (a) To check if engineer g can do job x, make x the first job of g; if that fails to satisfy any of the constraints, make it the second job of g, and so on.
   (b) If job x can fit into engineer g's current tour, then try to improve g's new tour (now with x in it): the improvement is done by a simple 2-opt algorithm (see section "GLS Implementation on the Traveling Salesman Problem"), modified in a such a way that only better tours which satisfy the relevant constraints will be accepted.
   (c) If job x cannot fit into engineer g's current tour, then consider the next engineer in the ordered list of qualified engineers for x; the job is unallocated if it cannot fit into any engineer's current tour.
3. The costs of the input permutation, which is the cost (as defined in Eq. 14 and 24) of the schedule thus created, are returned. These costs will be somehow aggregated in a single cost by GLS, or maintained simultaneously by GPLS.

Improvement Method

Given a permutation, the local search is performed in a simple way: the pairs of jobs are examined one at a time. Two jobs are swapped to generate a new permutation to be evaluated (using the evaluation procedure above) and compared to the current permutation. Note here that since the problem is also close to the vehicle routing problem (VRP), one may follow a totally different approach considering VRP move operators such as insertions, swaps, etc. In this case, the solution representation and construction methods need to be revised. The reader may refer to other works for more information on the application of GLS to the VRP (e.g., [14]) and also more advanced variants which are specific to this problem [11, 12].

Feature Definition

Since there are two objectives of different natures in the EmS-WSP, two sets of features need to be defined. As maximizing the total allocated tasks is an objective, each feature in the first set represents the inability to serving a task, so as to bias the local search to serve tasks of high importance. Thus, for each task t we define a feature $I_t(schedule)$ which is equal to 1 if t is unallocated in $schedule$, and 0 otherwise.

The cost of this feature is given by the task priority (c_t) which is equal to the cost incurred in the cost function (Eq. 14) when a task is unallocated.

The second objective concerns maximizing the total satisfied employees' plans, and therefore, each feature in the second feature set represents the inability to satisfy a work plan, so as to bias the (Pareto) local search to satisfy plans of high weight. Thus, for each technician r in the problem we define a feature $I_r(schedule) = y_r$, where y_r is equal to 1 if the work plan of r (ρ_r) is satisfied, and 0 otherwise. The cost of this feature is given by the working plan's weight (ω) which is equal to the cost incurred in the cost function (Eq. 24) when a plan is unsatisfied.

Overview of GLS Applications

GLS and its descendants have been applied to several nontrivial problems achieving state-of-the-art results.

Routing/Scheduling Problems

One of the most successful application of GLS and FLS is the *traveling salesman problem* (TSP). The Lin-Kernighan algorithm (LK) is a specialized algorithm for the TSP that has long been perceived as the champion of this problem [61, 64]. We tested GLS+FLS+2Opt against LK [103] on a set of benchmark problems from a public TSP library [79]. Given the same amount of time, GLS+FLS+2Opt found better results than LK on average. GLS+FLS+2Opt also outperformed simulated annealing [51], tabu search [53], and genetic algorithm [39] implementations for the TSP. One must be cautious when interpreting such empirical results as they could be affected by many factors, including implementation details. But given that the TSP is an extensively studied problem, it takes something special for an algorithm to outperform the champions under any reasonable measure ("find the best results within a given amount of time" must be a realistic requirement). It must be emphasized that LK is specialized for the TSP but GLS and FLS are much simpler general-purpose algorithms.

GLS extensions and hybrids have also been proposed for the TSP including a population-based GLS [91], the combination of GLS with memetic algorithms [47], and also with the dynamic-programming-based Dynasearch technique with encouraging preliminary results reported in [25].

Padron and Balaguer [72] have applied GLS to the related rural postman problem (RPP), Vansteenwegen et al. [99] applied GLS to the related team orienteering problem (TOP), and Mester et al. [66] applied the guided evolution strategy hybrid meta-heuristic to a genetic ordering problem (a unidimensional wandering salesperson problem [UWSP]).

In a vehicle routing problem, one is given a set of vehicles, each with its specific capacity and availability, and a set of customers to serve, each with specific weight and/or time demand on the vehicles. The vehicles are grouped at one or more depots.

Both the depots and the customers are geographically distributed. The task is to serve the customers using the vehicles, satisfying time, and capacity constraints. This is a practical problem which, like many practical problems, is NP-hard.

Kilby et al. applied GLS to vehicle routing problems and achieved outstanding results [52]. As a result, their work was incorporated in Dispatcher, a commercial package developed by IBM ILOG [14]. Over the years, the application of GLS and its hybrids to the VRP has been considerably extended to several variants of the problem. GLS has been applied to the vehicle routing problem with backhauls and time windows [113] and to the capacitated arc routing problem [20]. Guided tabu search has been applied to the VRP with time window [93, 94] and also extended to other variants of the VRP, namely, the VRP with two-dimensional loading constraints [109], the VRP with simultaneous pickup and delivery [110], and the VRP with replenishment facility [94]. GLS with VNS [55] as well as GLS with ES [65] hybrids has been proposed to solve large-scale VRPs. A new hybridization between GLS and GA has been devised to effectively tackle the transit network design problem in [78]. The deployment of fiber optics in the telecommunication industry has been successfully optimized using GLS in [26, 83].

More recently, we observed two directions with regards to GLS research on routing problems. One direction is towards creating advanced variants of GLS for tackling very-large-scale instances of routing problems. The other is towards utilizing open-source versions of the algorithm to address the needs of a variety of practical VRP-related applications.

One the first direction of tackling ever larger instances of the VRP, Arnold and Sorensen [12] introduced knowledge-guided local search (KGLS). KGLS is using some very powerful local search construction heuristics and move operators to explore the search space achieving high-quality local minima. This is followed by a "perturbation" phase applying up to 100 iterations of GLS. KGLS is selecting edges to penalize using a selection of different distance-related metrics which are alternating during different stages of the algorithm. Each perturbation phase is followed by another local search phase using the standard cost function without penalties and so forth.

KGLS is considered amongst the best algorithms for the VRP. It was applied in [11] against some of the largest instances of the problem ever addressed involving up to 30,000 visits, resembling real-world scenarios present in parcel distribution. Several of the AI-related GLS enhancements described earlier using LLMs have been applied to improve KGLS even further [62, 108].

On the second direction of using open-source versions of GLS to tackle practical routing problems, a landmark event was the release of Google's OR Tools [76]. Google's OR Tools is a publicly available software suite designed to tackle vehicle routing problems. This suite features one of the most proficient VRP solvers. It employs meta-heuristics like guided local search, tabu search, and simulated annealing to avoid local optima. To the words of the Google OR team referring to GLS, "this is generally the most efficient meta-heuristic for vehicle routing." The availability of an open-source version of GLS for the VRP provided by Google, in addition to the efficiency of the method as highlighted above, has encouraged

several research works using Google's GLS implementation as a benchmark or for addressing a diverse range of practical applications.

In [86], Silva et al. investigate the performance of GLS as implemented in Google OR tools to address dynamic delivery and pickup problems within constrained execution times of up to 10s. The application of GLS to solid waste collection in Saudi Arabia compared to SA and TS using the same underlying Google libraries is examined in [3]. Optimizing distribution routes for delivering medicines to pharmacies in Central Java using Google's GLS is described in [105]. Urban planning and transport network analysis for the city of New York using the same Google implementation is detailed in [107]. Published works probably represent only a small fraction of the actual uses of the algorithm in the real world since nowadays the wider developer community through open source can have access and put to use a highly efficient meta-heuristic algorithm for routing problems.

Assignment Problems

The generalized assignment problem is a generic scheduling problem in which the task is to assign agents to jobs. Each job can only be handled by one agent, and each agent has a finite resource capacity that limits the number of jobs that it can be assigned to. Assigning different agents to different jobs bear different utilities. On the other hand, different agents will consume different amounts of resources when doing the same job. In a set of benchmark problems, GGA found results as good as those produced by a state-of-the-art algorithm (which was also a GA algorithm) by Chu and Beasley [24], with improved robustness [59].

GLS hybrids have been proposed for the related QAP. Zhang et al. [112] proposed the GLS/EDA hybrid meta-heuristic. In addition, the hybrid of GLS with ACO (ACO_GLS) has been applied to a variation of the QAP [43]. This encourages Daoud et al. [28] to couple GLS with ACO, GA, and PSO into three frameworks which are successfully applied to the robotic assembly line balancing problem.

In the *radio link frequency assignment problem* (RLFAP), the task is to assign available frequencies to communication channels satisfying constraints that prevent interference [21]. In some RLFAPs, the goal is to minimize the number of frequencies used. Bouju et al. [21] is an early work that applied GENET to radio length frequency assignment. For the CALMA set of benchmark problems, which has been widely used, GLS+FLS reported the best results compared to all work published previously [101]. In the NATO Symposium on RLFAP in Denmark, 1998, GGA was shown to improve the robustness of GLS [58]. In the same symposium, new and significantly improved results by GLS were reported [102]. At the time, GLS and GGA held some of the best-known results in the CALMA set of benchmark problems.

We have experimented with GLS and FLS on a variety of other problems, including the maximum channel assignment problem, a bandwidth packing problem variant, graph coloring, and the car sequencing problem. Some of these works are largely undocumented due to lack of time during the original development phase of the algorithm.

GGA was also successfully applied to the processor configuration problem, with new best results than those found by other previously reported algorithms [57, 58].

GLS and FLS have been successfully applied to the 3-D bin packing problem and its variants [33, 34, 56] as mentioned earlier in this chapter and also to network planning problems [38, 106]. GLS has been applied to the natural language parsing problem [29], graph set T-coloring problem [23], query reformulation [69], and ligand docking problems via drug design [75]. Variations of GLS have been applied to graph coloring [18] and the multidimensional knapsack problem [46, 90]. The multi-objective 0/1 knapsack problem has been tackled using GPLS in [8].

Resource Allocation and Scheduling Problems

In the *workforce scheduling problem* (WSP) [13], the task is to assign technicians from various bases to serve the jobs, which may include customer requests and repairs, at various locations. Customer requirements and working hours restrict the service times at which certain jobs can be served by certain technicians. The objective is to minimize a function that takes into account the traveling cost, overtime cost, and unserved jobs. In the WSP, GLS+FLS holds the best-published results for the benchmark problem available to the authors [96]. GPLS has also been applied to a multi-objective WSP in [6]. Further applications of GLS to resource scheduling problems are described in [48, 71].

Constrained Optimization Problem

Given a set of propositions in conjunctive normal form, the satisfiability (SAT) problem is to determine whether the propositions can all be satisfied. The MAX-SAT problem is a SAT problem in which each clause is given a weight. The task is to minimize the total weight of the violated clauses. In other words, the weighted MAX-SAT problem is an optimization problem. Many researchers believe that many problems, including scheduling and planning, can be formulated as SAT and MAX-SAT problems; hence, these problems have received significant attention in recent years, e.g., see Gent et al. [40].

GLSSAT, an extension of GLS, was applied to both the SAT and weighted MAX-SAT problem [67]. On a set of SAT problems from DIMACS, GLSSAT produced more frequently better or comparable solutions than those produced by WalkSAT [81], a variation of GSAT [82], which was specifically designed for the SAT problem.

On a popular set of benchmark weighted MAX-SAT problems, GLSSAT produced better or comparable solutions, more frequently than state-of-the-art algorithms, such as DLM [84], WalkSAT [81], and GRASP [80]. In similar fashion, GLS has been utilized to very quickly produce feasible solutions to many hard MIP problem instances and proven to be highly effective on a range of problems from the MIPLIB 2017 benchmark set [63].

GLS in Commercial Packages

GLS and FLS have been incorporated into software packages, namely, iOpt, which is a software toolkit for heuristic search methods [104], and iSchedule [31], which is an extension of iOpt for planning and scheduling applications (e.g., for solving problems such as the VRP [32]). IBM ILOG Dispatcher, a commercial package developed by ILOG [14, 50], incorporates a version of GLS targeted at transport and logistics applications.

Conclusion

For many years, general heuristics for combinatorial optimization problems, with prominent examples such as simulated annealing and genetic algorithms, heavily relied on randomness to generate good approximate solutions to difficult NP-hard problems.

The introduction and acceptance of tabu search [41] by the Operations Research community initiated an important new era for heuristic methods where deterministic algorithms exploiting historical information started to appear and to be used in real-world applications.

Guided local search described in this chapter follows this trend. While tabu search is a class of algorithms (where a lot of freedom is given to the management of the tabu list), GLS is more prescriptive (the procedures are more concretely defined). GLS heavily exploits information (not only the search history) to distribute the search effort in the various regions of the search space. Important structural properties of solutions are captured by solution features. Solution features are assigned costs and local search is biased to spend its efforts according to these costs. Penalties on features are utilized for that purpose.

When local search settles in a local minimum, the penalties are increased for selected features present in the local minimum. By penalizing features appearing in local minima, GLS escapes the local minima visited (exploiting historical information) but also diversifies the choices, with regard to the various structural properties of solutions, as captured by the solution features. Features of high costs are penalized more often than features of low cost: the diversification process is directed and deterministic rather than undirected and random.

In general, several penalty cycles may be required before a move is executed out of a local minimum. This should not be viewed as an undesirable situation. It is caused by the uncertainty in the information as captured by the feature costs which forces the GLS to test its decisions against the landscape of the problem.

The penalization scheme of GLS is ideally combined with FLS which limits the neighborhood search to particular parts of the overall solution leading to the GFLS algorithm. GFLS significantly reduces the computation times required to explore the area around a local minimum to find the best escape route allowing many more penalty modification cycles to be performed in a given amount of running time.

Despite all the years of development, further research is possible to improve GLS and its related techniques further. The use of incentives implemented as negative penalties, which encourage the use of specific solution features, is one promising direction to be explored. Other interesting directions include probabilistic features matching specific solution patterns (perhaps clustered and identified by deep learning algorithms) with indicator functions returning real values in the [0, 1] interval, automated tuning of the λ or α parameters, definition of effective termination criteria, alternative utility functions for selecting the features to be penalized, and also studies about the convergence properties of GLS.

It is relatively easy to adapt GLS and GFLS to the different problems examined in this chapter. Although local search is problem dependent, the other structures of GLS and also GFLS are problem independent. Moreover, a mechanical, step-by-step procedure is usually followed when GLS or GFLS is applied to a new problem (i.e., implement a local search procedure, identify features, assign costs, define sub-neighborhoods, etc.). This makes GLS and GFLS easier to use by nonspecialist software engineers.

Cross-References

▸ Iterated Local Search
▸ Tabu Search
▸ Variable Neighborhood Search

References

1. Aardal K, Van Hoesel S, Koster A et al (2007) Models and solution techniques for frequency assignment problems. Ann Oper Res 153:79–129
2. Alalmaee Z, Tairan N (2022) A hybrid cooperative guided local search for solving the traveling salesman problem. In: 2022 IEEE 7th international conference for convergence in technology (I2CT), Mumbai, India, pp 1–6. https://doi.org/10.1109/I2CT54291.2022.9824785
3. Algethami H (2023) Local search-based metaheuristic methods for the solid waste collection problem. Appl Comput Intell Soft Comput 2023:Article ID 5398400. https://doi.org/10.1155/2023/5398400
4. Alhindi A (2015) Multiobjective evolutionary algorithm based on decomposition with advanced local search methods. Ph.D. thesis, Department of Computer Science, University of Essex, Colchester
5. Alhindi A, Alhindi A, Alhejali A et al (2019) MOEA/D-GLS: a multiobjective memetic algorithm using decomposition and guided local search. Soft Comput 23:9605–9615
6. Alsheddy A (2011) Empowerment scheduling: a multi-objective optimization approach using guided local search. Ph.D. thesis, Department of Computer Science, University of Essex, Colchester
7. Alsheddy A, Tsang EPK (2009) Guided Pareto local search and its application to the 0/1 multi-objective knapsack problems. In: Metaheuristics International Conference (MIC2009), Hamburg

8. Alsheddy A, Tsang EPK (2010) Guided pareto local search based frameworks for biobjective optimization. In: IEEE congress on evolutionary computation (CEC), Barcelona, pp 1–8
9. Alsheddy A, Tsang EPK (2011) Empowerment scheduling for a field workforce. J Sched 14(6):639–654
10. Anderson CA, Fraughnaugh K, Parker M et al (1993) Path assignment for call routing: an application of tabu search. Ann Oper Res 41:301–312
11. Arnold F, Gendreau M, Sörensen K (2019) Efficiently solving very large-scale routing problems. Comput Oper Res 107:32–42
12. Arnold F, Sörensen K (2019) Knowledge-guided local search for the vehicle routing problem. Comput Oper Res 105:32–46
13. Azarmi N, Abdul-Hameed W (1995) Workforce scheduling with constraint logic programming. BT Technol J 13(1):81–94
14. Backer BD, Furnon V, Shaw P et al (2000) Solving vehicle routing problems using constraint programming and metaheuristics. J Heuristics 6(4):501–523
15. Bai R, Kendall G, Qu R et al (2012) Tabu assisted guided local search approaches for freight service network design. Inf Sci 189:266–281
16. Barbucha D (2012) Agent-based guided local search. Exp Syst Appl 39:12032–12045
17. Barbucha D (2020) Multi-agent approach to the DVRP with GLS improvement procedure. In: Intelligent decision technologies: proceedings of the 12th KES international conference on intelligent decision technologies (KES-IDT 2020), pp 117–126
18. Basharu M, Arana I, Ahriz H (2005) Distributed guided local search for solving binary DisCSPs. In: Proceedings of FLAIRS 2005. AAAI Press, pp 660–665
19. Bentley JJ (1992) Fast algorithms for geometric traveling salesman problems. ORSA J Comput 4:387–411
20. Beullens P, Muyldermans L, Cattrysse D et al (2003) A guided local search heuristic for the capacitated arc routing problem. Eur J Oper Res 147(3):629–643
21. Bouju A, Boyce JF, Dimitropoulos CHD et al (1995) Intelligent search for the radio link frequency assignment problem. In: Proceedings of the international conference on digital signal processing, Cyprus
22. Castillo-Salazar A, Landa-Silva D, Qu R (2016) Workforce scheduling and routing problems: literature survey and computational study. Ann Oper Res 239(1):39–67
23. Chiarandini M, Stutzle T (2007) Stochastic local search algorithms for graph set T-colouring and frequency assignment. Constraints 12(3):371–403
24. Chu P, Beasley JE (1997) A genetic algorithm for the generalized assignment problem. Comput Oper Res 24:17–23
25. Congram RK, Potts CN (1999) Dynasearch algorithms for the traveling salesman problem. Presentation at the travelling salesman workshop, CORMSIS, University of Southampton
26. Cramer S, Kampouridis M (2015) Optimising the deployment of fibre optics using guided local search. In: Proceedings of the 2015 IEEE congress on evolutionary computation (CEC). IEEE Press, Sendai, pp 799–806
27. Croes A (1958) A method for solving traveling-salesman problems. Oper Res 5:791–812
28. Daoud S, Chehade H, Yalaoui F et al (2014) Solving a robotic assembly line balancing problem using efficient hybrid methods. J Heuristics 20(3):235–259
29. Daum M, Menzel W (2002) Parsing natural language using guided local search. In: Proceedings of the 15th European conference on artificial intelligence (ECAI-2002), pp 435–439
30. Davenport A, Tsang EPK, Wang CJ et al (1994) GENET: a connectionist architecture for solving constraint satisfaction problems by iterative improvement. In: Proceedings of the 12th national conference for artificial intelligence (AAAI), 325–330.
31. Dorne R, Voudouris C, Liret A et al (2003) iSchedule an optimisation tool-kit based on heuristic search to solve BT scheduling problems. BT Tech J 21(4):50–58
32. Dorne R, Mills P, Voudouris C (2007) Solving vehicle routing using iOpt. In: Doerner et al (eds) Metaheuristics: progress in complex systems optimization, operations research/computer science interfaces series, vol 39, pp 389–408

33. Egeblad J, Nielsen B, Odgaard A (2007) Fast neighbourhood search for two- and three-dimensional nesting problems. Eur J Oper Res 183(3):1249–1266
34. Faroe O, Pisinger D, Zachariasen M (2003) Guided local search for final placement in VLSI design. J Heuristics 9:269–295
35. Faroe O, Pisinger D, Zachariasen M (2003) Guided local search for the three-dimensional bin-packing problem. Informs J Comput 15:267–283
36. Faroe O, Pisinger D, Zachariasen M (2001) Local search for final placement in vlsi design. In: IEEE/ACM international conference on computer aided design. ICCAD 2001. IEEE/ACM digest of technical papers (Cat. No. 01CH37281), pp 565–572
37. Flood MM (1956) The traveling-salesman problem. Oper Res 4:61–75
38. Flores Lucio G, Reed M, Henning I (2007) Guided local search as a network planning algorithm that incorporates uncertain traffic demands. Comput Netw 51(11):3172–3196
39. Freisleben B, Merz P (1996) A genetic local search algorithm for solving symmetric and asymmetric travelling salesman problems. In: Proceedings of the 1996 IEEE international conference on evolutionary computation. IEEE Press, pp 616–621
40. Gent IP, van Maaren H, Walsh T (2000) SAT2000, Highlights of satisfiability research in the year 2000. In: Frontiers in artificial intelligence and applications. IOS Press, Amsterdam
41. Glover F, Laguna M (1997) Tabu search. Kluwer Academic Publishers, Boston
42. Gomes N, Vale Z, Ramos C (2003) Hybrid constraint algorithm for the maintenance scheduling of electric power units. In: Proceedings of the international conference on intelligent systems application to power systems (ISAP 2003), Lemnos, Greece
43. Hani Y, Amodeo L, Yalaoui F et al (2007) Ant colony optimization for solving an industrial layout problem. Eur J Oper Res 183(2):633–642
44. Hansen P, Mladenović N, Todosijević R (2016) Variable neighborhood search: basics and variants. EURO J Comput Optim 5:1–32
45. Hashem M, Mohammed M, Youssef A (2020) Improving the efficiency of dental implantation process using guided local search models and continuous time neural networks with robotic assistance. IEEE Access 8, pp 202755–202764. https://doi.org/10.1109/ACCESS.2020.3034689
46. Hifi M, Michrafy M, Sbihi A (2004) Heuristic algorithms for the multiple-choice multidimensional knapsack problem. J Oper Res Soc 55:1323–1332
47. Holstein D, Moscato P (1999) Memetic algorithms using guided local search: a case study. In: Corne D, Glover F, Dorigo M (eds) New ideas in optimisation. McGraw-Hill, London, pp 235–244
48. Hu S, Zhang Z, Wang S et al (2019) A project scheduling problem with spatial resource constraints and a corresponding guided local search algorithm. J Oper Res Soc 70:1349–1361
49. Hudson B, Li Q, Malencia M et al (2021) Graph neural network guided local search for the traveling salesperson problem. arXiv preprint arXiv:2110.05291
50. IBM ILOG Dispatcher (2024) https://public.dhe.ibm.com/software/websphere/ilog/docs/optimization/cp/refdispatcher.pdf. Accessed 12 May 2024
51. Johnson D (1990) Local optimization and the traveling salesman problem. In: Proceedings of the 17th colloquium on automata languages and programming. Lecture notes in computer science, vol 443. Springer, pp 446–461
52. Kilby P, Prosser P, Shaw P (1999) Guided local search for the vehicle routing problem with time windows. In: Voss S, Martello S, Osman IH et al (eds) Meta-heuristics: advances and trends in local search paradigms for optimization. Kluwer Academic Publishers, pp 473–486
53. Knox J (1994) Tabu search performance on the symmetric traveling salesman problem. Comput Ops Res 21(8):867–876
54. Koopman BO (1957) The theory of search, part III, the optimum distribution of search effort. Oper Res 5:613–626
55. Kytöjoki J, Nuortio T, Bräysy O et al (2007) An efficient variable neighbourhood search heuristic for very large scale vehicle routing problems. Comput Oper Res 34(9):2743–2757
56. Langer Y, Bay M, Crama Y et al (2005) Optimization of surface utilization using heuristic approaches. In: Proceedings of the international conference, COMPIT'05, pp 419–425

57. Lau TL (1999) Guided genetic algorithm. Ph.D. thesis, Department of Computer Science, University of Essex, Colchester
58. Lau TL, Tsang EPK (1998) Guided genetic algorithm and its application to the radio link frequency allocation problem. In: Proceedings of NATO symposium on frequency assignment, sharing and conservation in systems (AEROSPACE), AGARD, RTO-MP-13, paper No. 14b
59. Lau TL, Tsang EPK (1998) The guided genetic algorithm and its application to the general assignment problem. In: IEEE 10th international conference on tools with artificial intelligence (ICTAI'98), Taiwan, pp 336–343
60. Lin S (1965) Computer solutions of the traveling-salesman problem. Bell Syst Tech J 44:2245–2269
61. Lin S, Kernighan BW (1973) An effective heuristic algorithm for the traveling salesman problem. Oper Res 21:498–516
62. Liu F, Tong X, Yuan M et al (2024) An example of evolutionary computation + large language model beating human: design of efficient guided local search. arXiv preprint arXiv:2401.02051
63. Luteberget B, Sartor G (2023) Feasibility jump: an LP-free lagrangian MIP heuristic. Math Program Comput 15:365–388
64. Martin O, Otto SW (1966) Combining simulated annealing with local search heuristics. Ann Oper Res 63(1):57–75
65. Mester D, Bräysy O (2005) Active guided evolution strategies for large-scale vehicle routing problems with time windows. Comput Oper Res 32(6):1593–1614
66. Mester DI, Ronin YI, Nevo E et al (2004) Fast and high precision algorithms for optimization in large-scale genomic problems. Comput Biol Chem 28(4):281–290
67. Mills P, Tsang EPK (2000) Guided local search for solving SAT and weighted MAX-SAT problems. J Autom Reason 24:205–223
68. Mills P, Tsang E, Ford J (2003) Applying an extended guided local search to the quadratic assignment problem. Ann Oper Res 118(1–4):121–135
69. Moghrabi I (2006) Guided local search for query reformulation using weight propagation. Int J Appl Math Comput Sci 16(4):537–549
70. Murphey RA, Pardalos PM, Resende MGC (1999) Frequency assignment problems. In: Du D-Z, Pardalos P (eds) Handbook of combinatorial optimization, vol 4. Kluwer Academic Publishers
71. Nagata Y, Ono I (2018) A guided local search with iterative ejections of bottleneck operations for the job shop scheduling problem. Comput Oper Res 90:60–71
72. Padron V, Balaguer C (2000) New methodology to solve the RPP by means of isolated edge. In: Tuson A (ed) Cambridge conference tutorial papers. Young OR, vol 11. Operational Research Society, UK
73. Paquete L, Chiarandini M, Stützle T (2004) Pareto local optimum sets in the biobjective traveling salesman problem: an experimental study. Metaheuristics Multiobjective Optim 535:177–199
74. Parida S, Pati B, Nayak S et al (2022) PE-DCA: penalty elimination based data center allocation technique using guided local search for IAAS cloud. Comput Sci Inf Syst 19:679–707
75. Peh S, Hong J (2016) GLSDock – drug design using guided local search. In: Proceedings of the 2016 international conference on computational science and its applications, Beijing, pp 11–21
76. Perron L, Furnon V (2024) OR-Tools v9.9. https://developers.google.com/optimization/. Accessed 12 May 2024
77. Pesant G, Gendreau M (1999) A constraint programming framework for local search methods. J Heuristics 5(3):255–279
78. Rahman MK, Nayeem MA, Rahman MS (2015) Transit network design by hybrid guided genetic algorithm with elitism. In: Proceedings of the 2015 conference on advanced systems for public transport (CASPT), Rotterdam

79. Reinelt G (1991) A traveling salesman problem library. ORSA J Comput 3:376–384
80. Resende MGC, Feo TA (1996) A GRASP for satisfiability. In: Johnson DS, Trick MA (eds) Cliques, coloring, and satisfiability: second DIMACS implementation challenge. DIMACS series on Discrete mathematics and theoretical computer science, vol 26. American Mathematical Society, pp 499–520
81. Selman B, Kautz H (1993) Domain-independent extensions to GSAT: solving large structured satisfiability problems. In: Proceedings of the 13th international joint conference on AI, pp 290–295
82. Selman B, Levesque HJ, Mitchell DG (1992) A new method for solving hard satisfiability problems. Proc AAAI 92:440–446
83. Shaghaghi A, Glover T, Kampouridis M et al (2013) Guided local search for optimal GPON/FTTP network design. In: Chaki N et al (eds) Computer networks & communications (NetCom): proceedings of the fourth international conference on networks & communications. Springer, New York, pp 255–263
84. Shang Y, Wah BW (1998) A discrete lagrangian-based global-search method for solving satisfiability problems. J Glob Optim 12(1):61–99
85. Shi J, Zhang Q, Tsang E (2018) EB-GLS: an improved guided local search based on the big valley structure. Memet Comput 10:333–350
86. Silva AS, Lima J, Silva AMT et al (2024) Time-Dependency of Guided Local Search to Solve the Capacitated Vehicle Routing Problem with Time Windows. In: Pereira AI, Mendes A, Fernandes FP et al (eds) Optimization, learning algorithms and applications. OL2A 2023. Communications in computer and information science, vol 1981. Springer, Cham. https://doi.org/10.1007/978-3-031-53025-8_7
87. Simon HU (1989) Approximation algorithms for channel assignment in cellular radio networks. In: Proceedings of the 7th international symposium on fundamentals of computation theory. Lecture notes in computer science, vol 380. Springer, pp 405–416
88. Stone LD (1983) The process of search planning: current approaches and continuing problems. Oper Res 31:207–233
89. Sui J, Ding S, Xia B et al (2023) NeuralGLS: learning to guide local search with graph convolutional network for the traveling salesman problem. Neural Comput Appl 1–20. https://doi.org/10.1007/s00521-023-09042-6
90. Tairan N, Algarni A, Varghese J et al (2015) Population-based guided local search for multi-dimensional knapsack problem. In: Proceedings of the 2015 fourth international conference on future generation communication technology (FGCT), Luton, pp 1–5
91. Tairan N, Zhang Q (2013) P-GLS-II: an enhanced version of the population-based guided local search. In: Proceedings of the 13th annual conference on genetic and evolutionary computation (GECCO), 537–544
92. Tamura H, Zhang Z, Tang Z et al (2006) Objective function adjustment algorithm for combinatorial optimization problems. IEICE Trans Fundam Electron Commun Comput Sci E89–A:9:2441–2444
93. Tarantilis CD, Zachariadis EE, Kiranoudis CT (2008) A guided tabu search for the heterogeneous vehicle routeing problem. J Oper Res Soc 59(12):1659–1673
94. Tarantilis CD, Zachariadis EE, Kiranoudis CT (2008) A hybrid guided local search for the vehicle-routing problem with intermediate replenishment facilities. INFORMS J Comput 20(1):154–168
95. Tikhonov AN, Arsenin VY (1977) Solutions of ill-posed problems. Wiley, New York
96. Tsang EPK, Voudouris C (1997) Fast local search and guided local search and their application to British Telecom's workforce scheduling problem. Oper Res Lett 20(3):119–127
97. Tsang EPK, Wang CJ, Davenport A et al (1999) A family of stochastic methods for constraint satisfaction and optimisation. In: Proceedings of the first international conference on the practical application of constraint technologies and logic programming (PACLP), London, pp 359–383

98. Umetani S, Yagiura M, Imamichi T et al (2006) A guided local search based on a fast neighborhood search for the irregular strip packing problem. In: Proceedings of international symposium on scheduling 2006 (ISS2006), Tokyo, pp 126–131
99. Vansteenwegen P, Souffriau W, Berghe G et al (2009) A guided local search metaheuristic for the team orienteering problem. Eur J Oper Res 196(1):118–127
100. Voudouris C (1997) Guided local search for combinatorial optimisation problems. Ph.D. thesis, Department of Computer Science, University of Essex, Colchester
101. Voudouris C, Tsang EPK (1996) Partial constraint satisfaction problems and guided local search. In: Proceedings of PACT'96, London, pp 337–356
102. Voudouris C, Tsang EPK (1998) Solving the radio link frequency assignment problems using guided local search. In: Proceedings of NATO symposium on frequency assignment, sharing and conservation in systems (AEROSPACE), AGARD, RTO-MP-13, paper No. 14a
103. Voudouris C, Tsang EPK (1999) Guided local search and its application to the travelling salesman problem. Eur J Oper Res 113(2):469–499
104. Voudouris C, Dorne R, Lesaint D et al (2001) iOpt: a software toolkit for heuristic search methods. In: Walsh T (ed) Practice of constraint programming – CP 2001. Lecture notes in computer science, vol 2239, pp 716–729
105. Winadi F (2023) Completion of CVRP model using sweep algorithm and guided local search algorithm for route optimization (case study: PT. Sumber Berkah Farmasi). Unnes J Math 12(1) https://doi.org/10.15294/ujm.v12i1.65571
106. Xiaohu T, Haubrich H-J (2005) A hybrid metaheuristic method for the planning of medium-voltage distribution networks. In: Proceedings of 15th power systems computation conference (PSCC 2005), Liege, Belgium
107. Yang H, Landes H, Chow JYJ (2023) A large-scale analytical residential parcel delivery model evaluating greenhouse gas emissions, COVID-19 impact, and cargo bikes. Int J Transp Sci Technol https://doi.org/10.1016/j.ijtst.2023.08.002
108. Ye H, Wang J, Cao Z et al (2024) ReEvo: large language models as hyper-heuristics with reflective evolution. arXiv Preprint arXiv:2402.01145
109. Zachariadis E, Tarantilis C, Kiranoudis C (2009) A guided tabu search for the vehicle routing problem with two-dimensional loading constraints. Eur J Oper Res 195(3):729–743
110. Zachariadis E, Tarantilis C, Kiranoudis C (2009) A hybrid metaheuristic algorithm for the vehicle routing problem with simultaneous delivery and pick-up service. Expert Syst Appl 36(2):1070–1081
111. Zhang Q, Hui L (2007) MOEA/D: a multiobjective evolutionary algorithm based on decomposition. IEEE Tran Evol Comput 11:712–731
112. Zhang Q, Sun J, Tsang EPK et al (2003) Combination of guided local search and estimation of distribution algorithm for solving quadratic assignment problem. In: Bird of a feather workshops, genetic and evolutionary computation conference
113. Zhong Y, Cole MH (2005) A vehicle routing problem with backhauls and time windows: a guided local search solution. Transp Res Part E Logist Transp Rev 41(2):131–144

Theory of Local Search

17

W. Michiels, E. H. L. Aarts, and J. Korst

Contents

Introduction 470
Solution Quality 472
Exact Neighborhood Functions 472
Exact Neighborhood Function for TSP: An NP-Completeness Proof 473
Performance Ratios of Neighborhood Functions 476
Non-approximability Results 480
From Neighborhood Function to Polynomial-Time Algorithm 481
Exact Neighborhood Functions 483
Neighborhood Functions with Performance Bound 485
Time Complexity 488
Computational Complexity of Local Search Problems 489
Proving PLS-Completeness 493
Time Complexity of Iterative Improvement 499
Ad Hoc Strategies 501
General Strategy 503
Relevance of PLS Beyond Local Search 507
Conclusion 508
References 508

W. Michiels (✉)
NXP, Eindhoven, The Netherlands

Eindhoven University of Technology, Eindhoven, The Netherlands
e-mail: wil.michiels@nxp.com; w.p.a.j.michiels@tue.nl

E. H. L. Aarts
Tilburg University, Tilburg, The Netherlands
e-mail: e.h.l.aarts@uvt.nl

J. Korst
Philips Research Laboratories, Eindhoven, The Netherlands
e-mail: jan.korst@philips.com

© Springer Nature Switzerland AG 2025

R. Martí et al. (eds.), *Handbook of Heuristics*,
https://doi.org/10.1007/978-3-032-00385-0_6

Abstract

Local search is a widely used method to solve combinatorial optimization problems. As many relevant combinatorial optimization problems are NP-hard, we often may not expect to find an algorithm that is guaranteed to return an optimal solution in a reasonable amount of time, i.e., in polynomial time. Therefore, one often resorts to heuristic methods that return good, suboptimal solutions in reasonable running times. Local search is a heuristic method that is popular for its ability to trade solution quality against computation time. By spending more time, we will generally get better solutions. Well-known examples of local search approaches are iterative improvement, simulated annealing, and tabu search.

The performance of local search, in terms of quality or running time, may be investigated empirically, probabilistically, and from a worst-case perspective. In this chapter we focus on the last option. That is, we give provable results on the worst-case performance of local search algorithms.

Besides combinatorial optimization problems, the theory discussed in this chapter also finds its application in game theory and computational complexity.

Keywords

Performance ratio · Time complexity · PLS · Iterative improvement

Introduction

Local search is a widely used method to solve combinatorial optimization problems. An instance of a combinatorial optimization problem P can be characterized by a pair (S, f), where S denotes a finite set of solutions and $f : S \to \mathbb{R}$ a cost function that assigns to each solution a real value that represents the quality of this solution. The problem is to find an optimal solution s_* in S, i.e., a solution for which $f(s_*) \leq f(s)$ for each $s \in S$ in case P is a minimization problem or a solution for which $f(s_*) \geq f(s)$ for each $s \in S$ in case P is a maximization problem.

As many combinatorial optimization problems are NP-hard, we often may not expect to find polynomial-time solution approaches that always return provably optimal solutions. In practice, one often has to settle for heuristic methods that return good, suboptimal solutions in reasonable running times. Local search is a heuristic method that is popular for its ability to trade solution quality against computation time. By spending more time, we will generally get better solutions. Well-known examples of local search approaches are iterative improvement, simulated annealing, and tabu search.

Loosely defined, a local search approach starts at an (often) arbitrarily chosen solution $s_{\text{init}} \in S$ and iteratively jumps to a (potentially) better solution in the neighborhood, until it ends at a locally optimal solution s_{end} for which—hopefully—$f(s_{\text{end}})$ is close to $f(s^*)$.

Using local search amounts to defining a neighborhood function $N: S \to P(S)$, where the power set $P(S)$ denotes the set of all subsets of solutions in S. For each solution $s \in S$, $N(s)$ gives the neighbors of s, i.e., the solutions one can jump to from S in one iteration of the local search approach.

Given S and N, the execution of a local search algorithm can be considered as a path in the directed neighborhood graph $G = (V, E)$, where each node $v \in V$ corresponds to a solution in S and $(v, w) \in E$ if and only if w corresponds to a neighbor of v. A local search algorithm specifies how s_{init} is chosen, how iteratively for the current solution s a neighboring solution from $N(s)$ is chosen as the next solution, and how eventually the algorithm will terminate. For example, iterative improvement starts with an initial solution and repeatedly selects for the given current solution s one of its neighbors $s' \in N(s)$ using some pivoting rule. If s' has a better quality than s, then s' is adopted as the new current solution. Otherwise, s is retained as the current solution. This procedure is repeated until the algorithm arrives at a local optimum, i.e., a solution that is better than any of its neighbors. This local optimum is then returned as the final solution. For a given local search approach, the transition graph is defined to be the subgraph of the neighborhood graph containing only the cost-improving edges.

The performance of a local search algorithm is determined by two aspects, namely, (1) the quality of the solutions it finally produces and (2) the time it takes to find these solutions (time complexity). The performance of a local search algorithm can be investigated in three ways:

- *empirical analysis*: investigate the performance by applying it to a representative set of problem instances.
- *probabilistic analysis*: assume a probability distribution over the set of all problem instances and use this to derive probabilistic results on solution quality and running time.
- *worst-case analysis*: investigate the performance by assuming worst-case assumptions.

Empirical and probabilistic analyses have the disadvantage that additional assumptions are required on how representative or how probable problem instances will be for a given application at hand and the information to base these assumptions upon may not be available. In this chapter we restrict ourselves to worst-case analysis.

Many of the examples that we give in this chapter relate to the traveling salesman problem (TSP). An instance of this well-known combinatorial optimization problem is defined by a set of n cities and an $n \times n$ distance matrix d, where $d_{ij} \in \mathbb{N}^+$ defines the distance from city i to city j. The problem is to find a tour visiting all n cities with minimum tour length. The TSP has three variants that will be considered in this chapter, namely, Symmetric TSP, Metric TSP, and Euclidean TSP. For the first variant, the distance matrix is required to be symmetric, i.e., $d_{ij} = d_{ji}$ for all i, j. For the second, the distance matrix should satisfy the triangle inequality,

i.e., $d_{ik} \leq d_{ij} + d_{jk}$ for all i, j, k. And a TSP instance is said to belong to the third variant if the distance matrix d is defined by the Euclidean distance.

This chapter can be considered as a condensed version of two chapters from the monograph *Theoretical Aspects of Local Search* [1]. For a more extensive treatment of the subject, the reader is referred to this work. This chapter is organized in two parts. The first part considers solution quality, while the second part considers time complexity.

Solution Quality

In this section we study the guaranteed solution quality for iterative improvement. This corresponds to determining the quality of local optima. Ideally, we want iterative improvement to be guaranteed to end up in an optimal solution. This means that the neighborhood function N is exact. In the subsection below, we give examples of this. However, in most applications of local search, we do not have the luxury of a practical exact neighborhood function. There, we have to be satisfied with suboptimal solutions. In these cases, it is interesting to know how far the quality of the suboptimal solution can be from that of an optimal one. To this end, we say that a neighborhood function has a performance bound U if all its local optima are guaranteed to have a cost at most U times the optimal cost in case of a minimization problem and a cost at least $1/U$ times the optimal cost in case of a maximization problem. If bound U is tight, then U is called a performance ratio. We devote a subsection to analyzing the performance ratio of the k-change neighborhood function for Euclidean TSP. In the last two subsections, we finally elaborate on the relation between the formal hardness of finding an R-approximate solution and the existence of an efficient neighborhood function with a performance bound of R.

Exact Neighborhood Functions

A neighborhood function is said to be exact if each locally optimal solution is also globally optimal. In this section we present some examples of exact neighborhood functions. First of all, consider sorting. Sorting a multiset of n numbers $a_1, a_2, \ldots, a_n$ corresponds to finding a permutation π of $\{1, 2, \ldots, n\}$ that minimizes the cost function $f(\pi) = \sum_{i=1}^{n} i \cdot a_{\pi(i)}$ if the numbers have to be put in non-increasing order and that maximizes this cost function if they have to be put in non-decreasing order. A well-known algorithm for sorting is bubble sort. This algorithm starts with an arbitrary permutation π and then swaps repeatedly two adjacent numbers that are not in the right order. Bubble sort finds an optimal solution because a sequence of numbers is sorted if and only if any two adjacent numbers in the sequence are sorted correctly. Obviously, swapping two adjacent numbers improves the cost

of a permutation if and only if they are not in the right order. This means that bubble sort corresponds to iterative improvement with the neighborhood function that constructs neighbors by swapping two adjacent numbers. Because bubble sort solves sorting, this neighborhood function is exact.

In the minimum spanning tree (MST) problem, we are asked to find a spanning tree with minimum weight for an edge-weighted graph. This problem can also be solved by applying iterative improvement with the exact neighborhood function proposed in the following theorem, which is taken from Papadimitriou and Steiglitz [2].

Theorem 1. *Consider for MST the neighborhood function that generates a neighbor of a spanning tree T in the following way. First of all, we add an arbitrary edge to T. This produces a single cycle. From this cycle we delete an arbitrary edge, which gives a new spanning tree. This neighborhood function is exact.*

Proof Suppose we have a spanning tree T that is a local optimum, but not a global optimum. To prove the theorem, we show that this yields a contradiction. Let T^* be an optimal spanning tree that has a maximum number of edges in common with T. Furthermore, let e be an edge with minimum weight from the set of edges that are in T^* but not in T. As any spanning tree has $n - 1$ edges, such an edge exists. The removal of e splits T^* into two subtrees T_1^* and T_2^*. We now construct a neighbor T' of T as follows. First, we add edge e to T. This results in a cycle c. Besides e, this cycle must contain at least one other edge e' connecting a node from T_1^* with a node from T_2^*. We now construct T' by removing e' from c.

Because T is a local optimum, we have that the cost of T' is at least the cost of T. Hence, $w(e) \geq w(e')$, where $w(e)$ and $w(e')$ give the weights of edges e and e'. However, this implies that connecting T_1^* and T_2^* by edge e' results in a spanning tree that is at least as good as T_* and that has at least one edge more in common with T. This yields a contradiction. Hence, we conclude that the proposed neighborhood function is exact. □

Exact Neighborhood Function for TSP: An NP-Completeness Proof

Theoretically, all problems admit an exact neighborhood function. If we choose as neighborhood of a solution s the complete solution set, i.e., $N(s) = S$ for all $s \in S$, then this neighborhood function is exact. However, this neighborhood is not very practical since verifying whether a solution is locally optimal corresponds to solving the decision problem underlying the optimization problem. Mostly, we want to use local search for NP-hard problems, in which case this problem is NP-complete. In order to be practically useful, it makes sense to aim for a neighborhood function with the property that a single iteration of iterative improvement can be implemented to run in polynomial time. This is captured by the following definition.

Definition 1. A neighborhood function is said to be *polynomially searchable* if in polynomial time we can decide whether a solution is locally optimal and, if not, construct a better neighboring solution. □

The problem is how do we know whether or not an exact polynomially searchable neighborhood function exists for a problem? One way of settling this issue is to use complexity theory to prove for a particular problem that an exact neighborhood function does not exist. We now present the result from Papadimitriou and Steiglitz [2, 3], who show how this can be done for Metric TSP.

An outline of the proof is as follows. It is well known that Hamiltonian cycle (HC) is NP-complete, where HC is the problem of deciding for a graph whether it contains a Hamiltonian cycle. We first prove that the problem does not become easier if in addition to the graph we are also given a Hamiltonian path. Remember that a Hamiltonian cycle is a Hamiltonian path in which we return to the starting node. Hence, we prove that the following problem is NP-complete.

Definition 2 (Restricted Hamiltonian Cycle (RHC)). Given are a graph G and a Hamiltonian path in G. Is there a Hamiltonian cycle in G? □

Using this result, we next show that Metric TSP suboptimality is also NP-complete, which was our goal. □

Definition 3 (Metric TSP Suboptimality). Given are an instance I of Metric TSP and a tour τ for I. Is τ suboptimal? □

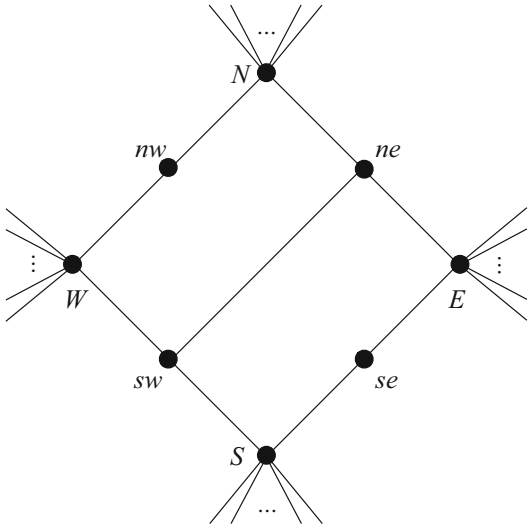
Lemma 1. *RHC is NP-complete.*

Proof Proving RHC $\in$ NP is trivial. Hence, to prove the lemma, it now suffices to show that RHC is polynomially reducible from HC. Let $G = (V, E)$ be an arbitrary graph defining a problem instance of HC. The construction of a corresponding instance of RHC is based on the special-purpose subgraph depicted in Fig. 1. This subgraph, which consists of eight nodes, is called a diamond.

In a diamond we identify four special nodes, called N (north), E (east), S (south), and W (west). We will use diamonds in such a way that if a graph G' contains a diamond D, then only these special nodes may be connected to other nodes in the graph. We now claim that if a graph G' contains a Hamiltonian cycle c, then for each diamond D in G' we have that its nodes are visited by c either in the way depicted in Fig. 2a or in the way depicted in Fig. 2b. The former case is referred to as north-south mode and the latter as east-west mode.

We now focus on proving this claim. Suppose that Hamiltonian cycle c enters diamond D in node N. We show that following any other path than the one depicted in Fig. 2a results in a contradiction. If node N is not succeeded by node *nw* (north-west) but by *ne*, then the only way for c to visit *nw* is via node W. However, this is not possible because, being in *nw*, we can neither go to N nor return to W as this would

Fig. 1 A diamond subgraph



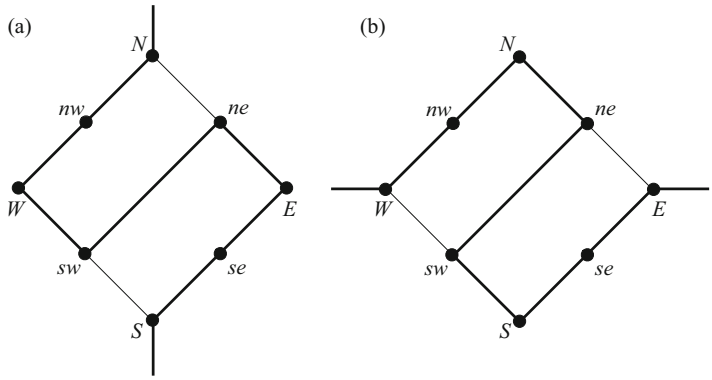


Fig. 2 (**a**) North-south mode. (**b**) East-west mode

imply visiting N or W twice. Hence, N is indeed succeeded by nw in c. Obviously, the only way to proceed from nw is to go to W. Once at node W, we cannot leave D because then the only way for c to visit sw and ne is by skipping se. From node sw we have to go to ne because if we chose S this would make it impossible to visit ne as well as se. From ne we can then only go to E, and to prevent skipping node se, we conclude the traversal of D by visiting se and S, successively. By a similar reasoning, it can be verified that if c enters a diamond D via the south, then D is also traversed as depicted in Fig. 2a, and if c enters D via the east or the west, then D is traversed as depicted in Fig. 2b. This proves our claim.

We now construct for our arbitrary graph $G = (V, E)$ specifying the arbitrary HC instance the following corresponding graph $G' = (V', E')$. Let $V = \{1, 2, \ldots, n\}$.

For each node $i \in V$, we introduce a diamond D_i with special nodes N_i, E_i, S_i, and W_i. Except for the last diamond D_n, we connect the south node of each diamond D_i with the north node of the next diamond D_{i+1}, i.e., we add the edges $\{S_i, N_{i+1}\}$ for all $1 \leq i < n$ to E'. Finally, we introduce for each edge $\{i, j\} \in E$ the two edges $\{W_i, E_j\}$ and $\{W_j, E_i\}$.

Obviously, the path that visits the diamonds in increasing order and traverses each diamond from north to south in the way depicted in Fig. 2a gives us a Hamiltonian path. Hence, to prove that we presented a valid polynomial-time reduction, it now suffices to show that G has a Hamiltonian cycle if and only if G' has one. We start with the "only if" part. Hence, suppose that G has a Hamiltonian cycle c. Then visiting the diamonds in G' in the order specified by c and the nodes inside a diamond in east-west mode yields a Hamiltonian cycle for G'. Next, assume that G' has a Hamiltonian cycle c. If c traverses one diamond in north-south mode, then it must traverse all diamonds in north-south mode as north and south nodes are not connected to east and west nodes. However, as only $n - 1$ north-south edges exist, no Hamiltonian cycle can exist that traverses each diamond in north-south mode. Hence, we can conclude that c must traverse each diamond in east-west mode. The order in which these diamonds are visited defines a Hamiltonian cycle for G. □

Theorem 2. *Metric TSP suboptimality is NP-complete.*

Proof Again, proving membership in NP is a trivial task. We now show that RHC is polynomially reducible to Metric TSP suboptimality. Let $G = (V, E)$ with Hamiltonian path $p = (v_1, v_2, \ldots, v_n)$ define an arbitrary problem instance I of RHC. If $\{v_1, v_n\} \in E$, then we can directly conclude that I is a yes-instance of RHC. Hence, without loss of generality, we assume that $\{v_1, v_n\} \notin E$. We transform problem instance I into the following problem instance of Metric TSP suboptimality. The set of cities is given by $C = V = \{1, 2, \ldots, n\}$, and the distance d_{ij} between cities i to j is defined by 1 if $\{i, j\} \in E$ and by 2 if $\{i, j\} \notin E$. Hamiltonian path p gives us a tour τ with cost $n + 1$ because $\{v_i, v_{i+1}\} \in E$ with $1 \leq i < n$ and $\{v_1, v_n\} \notin E$. Now, G has a Hamiltonian cycle if and only if tour τ is suboptimal. To see this, suppose that τ is suboptimal. A tour τ' then exists with length n. This tour defines a Hamiltonian cycle in G. Conversely, if G contains a Hamiltonian cycle, then this cycle determines a tour with length n, which implies that τ is suboptimal. This proves the theorem. □

Corollary 1. *Provided that $P \neq NP$, no polynomially searchable exact neighborhood function exists for Metric TSP.* □

Performance Ratios of Neighborhood Functions

In case a problem does not admit a polynomially searchable exact neighborhood function, we have to resort to a neighborhood function for which local optima can be suboptimal. To know how far we can be off from an optimal solution if we use

such a neighborhood function, we can try to derive bounds on the performance ratio of the neighborhood function. Observe that this performance ratio equals the performance ratio of iterative improvement where we select the initial solution randomly. If we select the initial solution more carefully, the performance ratio of iterative improvement may improve.

Consider Symmetric TSP. As the neighborhood size of k-change is polynomially bounded, a single iteration of k-Opt, which corresponds to a single step in the transition graph, can be implemented to run in polynomial time. Furthermore, finding a tour with length at most $2^{p(n)}$ times the optimal length is NP-hard for any fixed polynomial p [4]. Hence, if we were to prove that k-Opt is guaranteed to reach a locally optimal tour within a polynomial number of iterations, then we would obtain that, unless P = NP, k-Opt does not admit a performance ratio at most $2^{p(n)}$ for any fixed polynomial p. However, as we show in the next section, k-Opt may require an exponential number of iterations, which implies that the given reasoning is not applicable. The following theorem states that the lower bound of $2^{p(n)}$ on the performance ratio of k-Opt nevertheless holds. It even claims a stronger result: no function in n exists that gives an upper bound on the performance ratio of k-change. We note that this does not mean that it is impossible to bind the performance ratio. It can, for instance, be bounded trivially by the largest distance in the distance matrix d divided by the smallest distance in d.

Theorem 3. *For any fixed $k \geq 2$ and $n \geq 2\,k + 4$, the performance ratio of the k-change neighborhood function cannot be bounded by a constant for Symmetric TSP on n cities. This means that no function in n exists that gives a performance bound for k-change.*

Proof For any $k \geq 2$ and $\varepsilon > 0$, we define a problem instance of Symmetric TSP containing $n = 2\,k + 4$ cities. For this problem instance, we construct a locally optimal tour $\hat{\tau}$, which has a performance ratio of $1 + \frac{1}{n\varepsilon}$. It is easy to extend this problem instance and to modify the locally optimal tour, such that the problem instance contains any $n > 2\,k + 4$ cities, the tour $\hat{\tau}$ remains locally optimal, and the performance ratio of $\hat{\tau}$ is still $1 + \frac{1}{n\varepsilon}$. Since for $\varepsilon \to 0$ this performance ratio approaches ∞, this proves the theorem. □

We define the symmetric distance matrix d, such that all entries are given by $1 + \varepsilon$ except for the entries $d_{i,i+1}$ with $1 < i < n$, $d_{1,n}$, $d_{1,k+2+l}$ with $l = k \bmod 2$, and $d_{i,n-i+1}$ with $2 \leq i \leq k + 2$, which are all given by ε. In this definition we restrict ourselves without loss of generality to the entries d_{ij} with $i < j$.

Let $G = (V, E)$ be the graph obtained from the complete graph of n nodes by maintaining an edge $\{i, j\}$ if $d_{ij} = \varepsilon$ and by removing it if $d_{ij} = 1 + \varepsilon$; see Fig. 3. Obviously, a tour has length $n\varepsilon$ if it defines a Hamiltonian cycle in graph G and it has length at least $1 + n\varepsilon$ otherwise. Moreover, it can be proved that the tour $\tau^* = (1, n, n-1, 2, 3, n-2, n-3, \ldots)$ defines the only Hamiltonian cycle in G, where tours that only differ in their starting position or direction are considered to be equivalent. Hence, tour τ^* is the only optimal tour, and the length of any other tour τ is at least

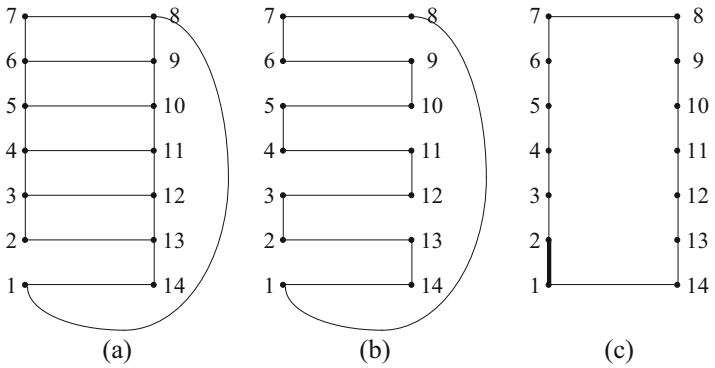


Fig. 3 For $k = 5$, the figure shows (**a**) graph G, (**b**) optimal tour τ^* of length $n\varepsilon$, and (**c**) locally optimal tour $\hat{\tau}$ of length $1 + n\varepsilon$

$1 + n\varepsilon$. From this it follows that if a tour $\hat{\tau}$ has length $1 + n\varepsilon$ and if it has $k + 1$ edges that are absent in τ^*, then $\hat{\tau}$ is a locally optimal tour with performance ratio $\frac{1+n\varepsilon}{n\varepsilon} = 1 + \frac{1}{n\varepsilon}$. It can be verified that such a tour is given by $\hat{\tau} = (1, 2, \ldots, n)$. This proves the theorem. □

In the remainder of this section, we focus on Metric TSP. Although solving this problem is as difficult as solving Symmetric TSP (they are both strongly NP-hard), it is easier to find an approximate solution for Metric TSP than for Symmetric TSP. Currently, Christofides' algorithm [5] is the polynomial-time algorithm with the best known worst-case performance. This constructive algorithm works as follows. First of all, it computes a minimum spanning tree T. Next, it converts this tree into an Eulerian graph, which is a graph in which each node has an even degree. This is done by deriving a perfect matching on the nodes with an odd degree. These edges are added to T. For an Eulerian graph, we can easily determine an Eulerian tour, which is a tour that traverses each edge exactly once. By introducing shortcuts, we can eliminate multiple visits to the same city in the tour. This results in a tour for which it can be proved that its length is at most $\frac{3}{2}$ times the optimal length.

For k-Opt Chandra, Karloff, and Tovey [6] have proved that, for infinitely many n, a lower bound on the performance ratio is given by $\frac{1}{4}\sqrt{n}$ for $k = 2$ and by $\frac{1}{4}n^{\frac{1}{2k}}$ for $k \geq 3$. Hence, from a worst-case perspective, the local search algorithm k-Opt cannot compete against Christofides' constructive algorithm. Nevertheless, k-Opt may still be preferred over Christofides' algorithm as in practice we may be more interested in a good average-case performance than in a good worst-case performance. Hence, it is still interesting to analyze the worst-case performance of k-Opt or, equivalently, k-change, not only to obtain a worst-case performance guarantee but also to enhance our understanding of the algorithm.

Chandra, Karloff, and Tovey [6] also proved that k-Opt has a performance bound, i.e., an upper bound on the performance ratio, of $4\sqrt{n}$ for any $k \geq 2$. By a tighter analysis, Levin and Yovel [7] improved this bound to $2\sqrt{2n}$. Below, we prove

this result. Note that this performance bound differs by a constant factor from the presented lower bound for $k = 2$ and by a factor that increases with n for any $k \geq 3$. In order to prove the bound, we need the following auxiliary result that states that a locally optimal tour cannot contain too many long edges as expressed as a fraction of the optimal tour length.

Lemma 2. *For an arbitrary problem instance of Metric TSP, let l^* be the optimal tour length and $\hat{\tau}$ a locally optimal tour with respect to the 2-change neighborhood function. For any i with $1 \leq i \leq n$, we have $|E_i| < i/2$, where set E_i contains the edges $\{j, j'\}$ from $\hat{\tau}$ with $d_{jj'} > \frac{2l^*}{\sqrt{i}}$.*

Proof We prove the lemma by contradiction. So, suppose we have $|E_i| \geq i/2$ for some i. Although the direction of $\hat{\tau}$ is irrelevant, we assume some orientation of the tour. As a result, the $r = |E_i|$ edges in E_i transform into r arcs (t_j, h_j) with $1 \leq j \leq r$. The cities t_j and h_j are called the tail and head of arc (t_j, h_j).

We first show that a tail t_u with $1 \leq u \leq r$ cannot be surrounded by too many other tails of arcs from E_i. Let V_u be the set of all tails of arcs from E_i that have a distance of at most $\frac{l^*}{\sqrt{i}}$ from t_u. Assume that $| V_u | \geq \sqrt{i}/2$. We prove that $| V_u | < \sqrt{i}/2$ by showing that this assumption yields a contradiction.

Let $t_j, t_{j'} \in V_u$. Because the triangle inequality holds and because, by definition, the distance from t_u to any of these two tails is at most $\frac{l^*}{\sqrt{i}}$, we obtain that the distance between t_j and $t_{j'}$ is at most $\frac{2l^*}{\sqrt{i}}$. This implies that the distance between the heads h_j and $h_{j'}$ is strictly larger than $\frac{2l^*}{\sqrt{i}}$. If this would not be the case, then replacing the edges $\{t_j, h_j\}$ and $\{t_{j'}, h_{j'}\}$ in $\hat{\tau}$ via a 2-change into $\{t_j, t_{j'}\}$ and $\{h_j, h_{j'}\}$ would result in a shorter tour because the length of each of the former two edges is strictly larger than $\frac{2l^*}{\sqrt{i}}$ by the definition of E_i. This would contradict the local optimality of $\hat{\tau}$.

Hence, $| V_u | \geq \sqrt{i}/2$ heads can be identified in E_i that all have a mutual distance of strictly more than $\frac{2l^*}{\sqrt{i}}$. This implies that the optimal tour on these heads is strictly more than l^*, which gives a contradiction because due to the triangle inequality it is not possible that an optimal tour over all cities is shorter than the optimal tour over a subset of all cities. This proves our claim $| V_u | < \sqrt{i}/2$. We use this result to derive a contradiction from the assumption $r \geq i/2$, which settles the proof of the lemma.

Consider the following labeling algorithm. Select an unlabeled tail t_u in E_i, and label all tails in E_i that have a distance of at most $\frac{l^*}{\sqrt{i}}$ from t_u (including t_u). Repeat this labeling step until all tails in E_i are labeled. As proved above, strictly fewer than $\sqrt{i}/2$ tails are labeled in each iteration. As $r \geq i/2$, this implies that the labeling procedure takes strictly more than $\sqrt{i}$ iterations. Obviously, the tails that are selected in these iterations all have a mutual distance strictly larger than $\frac{l^*}{\sqrt{i}}$. This implies that the optimal tour has length strictly larger than $\sqrt{i} \cdot \frac{l^*}{\sqrt{i}} = l^*$, which results in a contradiction. This proves the lemma. □

Using Lemma 2, we can now prove the claimed result.

Theorem 4. *The k-change neighborhood function has a performance bound of* $2\sqrt{2n}$ *for Metric TSP.*

Proof To prove the theorem, it suffices to show that a tour $\hat{\tau}$ has a performance ratio of at most $2\sqrt{2n}$ if it is a locally optimal tour with respect to the 2-change neighborhood function. Lemma 2 yields that the ith largest edge in $\hat{\tau}$ has a length of at most $\frac{\sqrt{2}l^*}{\sqrt{i}}$, where l^* is the optimal tour length. Hence, the length of $\hat{\tau}$ is at most

$$\sum_{i=1}^{n} \frac{\sqrt{2}l^*}{\sqrt{i}} = \sqrt{2}l^* \sum_{i=1}^{n} \frac{1}{\sqrt{i}}. \tag{1}$$

As $\frac{1}{\sqrt{x}}$ is a decreasing function in $x \in \mathbb{R}$, we have $\frac{1}{\sqrt{i}} \leq \int_{i-1}^{i} \frac{1}{\sqrt{x}}\,\mathrm{d}x$ for any positive integer i. Hence, the right-hand side of (1) is at most $\sqrt{2}l^* \int_0^n \frac{1}{\sqrt{x}}\mathrm{d}x$, which equals $2\sqrt{2n}l^*$. □

We note that, to get better performance guarantees, it sometimes helps to use an objective function that differs from the one defined in the optimization problem. An example of this is given by Angel et al. [8]. They refer to a result from [9] who show that the flip neighborhood function has a performance bound of $\frac{3}{2}$ for the non-weighted Max k-SAT problem. Khanna et al. [10] prove that this is the best you can achieve using any normal local search implementation. However, they also show that when using to a different objective function, the flip neighborhood function can achieve a performance ratio of $\frac{4}{3}$ for this problem.

Non-approximability Results

Above, we gave examples of performance ratio analyses for a particular neighborhood function. In this section, we show that we can also prove results on the performance ratio for a complete class of neighborhood functions for a problem.

Suppose that finding an R-approximate solution for Π is NP-hard for a given $R \geq 1$. Provided that P $\neq$ NP, this means that iterative improvement using neighborhood function N does not find a solution with performance ratio R in polynomial time. As it may take iterative improvement an exponential number of steps to reach a local optimum, this does not directly imply that N has a performance ratio larger than R. The following theorem from Yannakakis [11] states that the claim is nevertheless true. Observe that, by taking $R = 1$, the theorem yields that an NP-hard problem will not admit a polynomially searchable exact neighborhood function.

Theorem 5. *Let $\Pi \in$ NPO, and let N be a neighborhood function for Π that is polynomially searchable. If the problem of finding an R-approximate solution for Π is NP-hard for a given $R \geq 1$, then N does not have a performance bound of R, provided that NP $\neq$ co-NP.*

Proof We prove the result by contradiction. Suppose that approximating Π within a factor R is NP-hard and that N has a performance bound of R. We show that for any decision problem $\Pi_D \in$ NP the complementary problem Π_D^c obtained from Π_D by reversing the answers is also in NP. This yields a contradiction as it implies NP = co-NP.

To prove $\Pi_D^c \in$ NP, we need to show that each yes-instance of Π_D^c has a certificate that can be checked in polynomial time for validity, where the size of the certificate is polynomially bounded. As approximating Π within a factor R is NP-hard, a polynomial-time algorithm $\mathcal{A}$ exists that decides Π_D, where $\mathcal{A}$ may use an oracle that returns an R-approximate solution for any problem instance of Π. Hence, if for a problem instance I of Π_D we are given a sequence consisting of solutions that may be returned successively by the oracle during the execution of $\mathcal{A}$ on I, then we can determine in polynomial time whether I is a yes-instance or a no-instance of Π_D. This implies that as a certificate for yes-instances of Π_D^c we can use a sequence consisting of solutions that are locally optimal for the problem instances that are successively given to the oracle, i.e., the ith solution in the sequence is a local optimum for the ith problem instance given to the oracle. The certificate, which is of polynomial size, can be checked for validity in polynomial time by substituting each call of the oracle in $\mathcal{A}$ by a polynomial-time procedure that checks whether the corresponding solution in the certificate is a local optimum for the problem instance given to the oracle. By assumption this polynomial-time procedure exists. □

From the above theorem and the complexity result of Sahni and Gonzalez [4], it follows that if NP $\neq$ co-NP, then the performance ratio of the k-change neighborhood function cannot be bounded by $2^{p(n)}$ for some fixed polynomial p. Note that this result is weaker than the result of Theorem 3. Theorem 3 states that the performance ratio cannot even be bounded by 2^{2^n}, for instance, and that it also holds in the unlikely case that NP = co-NP.

From Neighborhood Function to Polynomial-Time Algorithm

Rewriting Theorem 5 gives that if a polynomially searchable neighborhood function N with performance bound R exists for some given combinatorial optimization problem $\Pi \in$ NPO, then the problem of finding an R-approximate solution for Π is not NP-hard, provided that NP $\neq$ co-NP. One question that arises is whether this result can be strengthened to the extent that the existence of such an N implies that the problem of finding an R-approximate solution for Π is easy, i.e.,

polynomially solvable. In this section we prove for a special type of combinatorial optimization problem that the answer is affirmative for $R = 1$ and that it is affirmative up to some arbitrary small error $\varepsilon > 0$ for $R > 1$. In other words, we show for a special type of combinatorial optimization problem how we can derive from a polynomially searchable exact neighborhood function a polynomial-time algorithm that solves the problem and how we can derive from a polynomially searchable neighborhood function with performance bound R a polynomial-time approximation algorithm with a performance bound of $R + \varepsilon$ for any precision $\varepsilon > 0$. Besides being polynomial in the input size, the $(R + \varepsilon)$-approximation algorithm is also polynomial in $\frac{1}{\varepsilon}$. Before proving these results, we introduce some terminology used in this section. The results are due to Schulz, Weismantel, and Ziegler [12] and Orlin, Punnen, and Schultz [13], respectively.

For many combinatorial optimization problems, it holds that a solution can be viewed as a subset of a ground set E and that the cost of a solution $s \subseteq E$ can be written as $\sum_{e \in s} c(e)$, where $c(e)$ is the cost of element $e \in E$. This is, for instance, the case for TSP: define E as the set of all pairs (i, j) of cities and $c(i, j)$ as the distance d_{ij} from city i to city j. We call such problems linear combinatorial optimization problems.

Definition 4. An *instance* of a linear combinatorial optimization problem is a pair (S, c), where each solution in the solution space $S \subseteq 2^E$ is a subset of a finite ground set $E = \{1, 2, \ldots, n\}$ and where $c : E \to \mathbb{Q}^+$ assigns a cost to each element $e \in E$. The cost of a solution $s \in S$ is defined by $f(s) = \sum_{e \in s} c(e)$. □

Definition 5. A *linear combinatorial optimization problem* Π is specified by a set of problem instances as defined in Definition 4, and the goal is to minimize for a problem instance the associated cost function f. Hence, for a problem instance (S, c), we have to find a solution s^* that satisfies $f(s^*) \leq f(s)$ for all $s \in S$. □

Definition 6. A linear combinatorial optimization problem Π is called *closed under scaling* whenever for each problem instance (S, c) of Π it holds that if we change function $c : E \to \mathbb{Q}^+$ to some other cost function $c' : E \to \mathbb{Q}^+$, then the problem instance remains in Π. □

In this section the focus is on linear combinatorial optimization problems Π that satisfy the following two properties.

- Π is closed under scaling.
- For each problem instance of Π, we can derive in polynomial time a feasible solution.

It can easily be verified that the linear combinatorial optimization problem TSP (the generic formulation) satisfies these two properties. However, this is not true, for instance, for Metric TSP because this problem is not closed under scaling.

If a problem is closed under scaling, then this need not necessarily mean that we can change the cost function of a problem instance in polynomial time. The reason for this is that an encoding scheme may be applied that does not give $c(e)$ explicitly for each $e \in E$. To avoid this problem, we focus on linear combinatorial optimization problems that are encoded by a cost-explicit encoding scheme, where a cost-explicit encoding scheme is defined as follows.

Definition 7. An encoding scheme of a linear combinatorial optimization problem is called *cost explicit* if the encoding $e_I \in \{0, 1\}^*$ of a problem instance $I = (S, c)$ with underlying ground set E is given by $e_I = e_I^{(1)} e_I^{(2)}$, where

- $e_I^{(1)}$ is an explicit encoding of the function $c : E \to \mathbb{Q}^+$, which means that it contains the encoding of $c(e)$ for each $e \in E$, and
- $e_I^{(2)}$ is an encoding, either implicit or explicit, of the solution space S.

Furthermore, $e_I^{(1)}$ only depends on S via its ground set E, and $e_I^{(2)}$ is independent of c. □

Exact Neighborhood Functions

Let Π be a linear combinatorial optimization problem from NPO that satisfies the two properties given above, and let a cost-explicit encoding scheme be used for Π. Furthermore, let N be an exact polynomially searchable neighborhood function for the problem. We will now present a polynomial-time algorithm, called $\mathcal{A}_{\text{opt}}(N)$, that solves Π.

Let $I = (S, c)$ be a problem instance of Π, where, without loss of generality, we assume that c is a function to the natural numbers instead of the rational numbers. $\mathcal{A}_{\text{opt}}(N)$ solves I in $K = \lceil \log_2(c_{\max} + 1) \rceil$ phases, where $c_{\max}$ is the maximum cost $c(e)$ of any element $e \in E$. In phase k an optimal solution is derived for the problem instance $I_k = (S, c_k)$ that is obtained from I by defining the cost $c_k(e)$ of an element $e \in E$ as the number represented by the k leading bits in the binary representation of $c(e)$. By adding leading zeros, the binary representation is made equally long for all $e \in E$, i.e., each binary representation consists of exactly K bits. We implement a phase by applying iterative improvement with the exact neighborhood function N. In the first phase, an arbitrary solution may be chosen as the starting solution, and in phase k with $k \geq 2$, the solution derived in phase $k - 1$ is chosen as the starting solution. Figure 4 gives the algorithm in pseudo-code, where f_k denotes the cost function related to problem instance I_k.

Theorem 6. *Let $\Pi \in$ NPO be a linear combinatorial optimization problem that is closed under scaling and for which we can derive a feasible solution in polynomial time. Furthermore, let N be a polynomially searchable exact neighborhood function for Π. Then the algorithm $\mathcal{A}_{\text{opt}}(N)$ given in Fig. 4 solves Π in polynomial time in the case that we use a cost-explicit encoding scheme for Π.*

algorithm $\mathcal{A}_{\mathrm{OPT}}(N)$

begin

$s :=$ some initial solution;

$K := \lceil \log_2(c_{\max} + 1) \rceil$;

for $k := 1$ **to** K **do**

begin

for all $e \in E$ **do** $c_k(e) := \lfloor c(e) \cdot 2^{k-K} \rfloor$;

repeat

generate an $s' \in N(s)$;

if $f_k(s') < f_k(s)$ **then** $s := s'$;

until $f_k(s') \geq f_k(s)$ for all $s' \in N(s)$;

end

end;

Fig. 4 Algorithm $\mathcal{A}_{\mathrm{opt}}(N)$ for a linear combinatorial optimization problem Π and an accompanying neighborhood function N. Π is closed under scaling, and (S, c) denotes an arbitrary problem instance of Π that is given as input to the algorithm. Cost function f_k relates to problem instance (S, c_k)

Proof By construction, the number of phases is bounded by $K = \lceil \log_2(c_{\max} + 1) \rceil$. Furthermore, because N is polynomially searchable, the individual iterations of the iterative improvement algorithm that makes up a single phase of $\mathcal{A}_{\mathrm{opt}}(N)$ can be implemented to run in polynomial time. Hence, to prove the theorem, we only have to show that in an arbitrary phase k the iterative improvement algorithm executed terminates within a polynomial number of iterations.

Let s_i be the solution derived in phase i of $\mathcal{A}_{\mathrm{opt}}(N)$, and let s_0 be the starting solution of the first phase. Hence, in phase k, iterative improvement starts with s_{k-1} and ends with s_k. Because the cost of the solution derived by iterative improvement decreases in each iteration, it suffices to prove that $f_k(s_{k-1}) - f_k(s_k)$ is polynomially bounded to show that in phase k iterative improvement terminates within a polynomial number of iterations. By defining $c^{(i)}(e)$ with $e \in E$ as the ith bit in the binary representation of $c(e)$, we obtain that for any solution s the cost $f_k(s)$ satisfies

$$f_k(s) = \begin{cases} \sum_{e \in s} c^{(1)}(e) & \text{if } k = 1 \\ 2 f_{k-1}(s) + \sum_{e \in s} c^{(k)}(e) & \text{if } k \geq 2. \end{cases}$$

Furthermore, we have $\sum_e \in {}_s c^{(k)}(e) \leq n$, where n denotes the number of elements in E. Using these two observations and the optimality of $s_k - {}_1$ with respect to cost function $f_k - {}_1$, we can derive for $k \geq 2$

$$f_k\,(s_{k-1}) - f_k\,(s_k) = 2\,(f_{k-1}\,(s_{k-1}) - f_{k-1}\,(s_k)) + \sum_{e \in s_{k-1}} c^{(k)}(e) - \sum_{e \in s_k} c^{(k)}(e) \leq n$$

and for $k = 1$

$$f_k\,(s_{k-1}) - f_k\,(s_k) = \sum_{e \in s_{k-1}} c^{(k)}(e) - \sum_{e \in s_k} c^{(k)}(e) \leq n.$$

This proves the theorem. □

Neighborhood Functions with Performance Bound

Theorem 6. Yields that if a polynomially searchable neighborhood function with performance bound $R = 1$ exists for a given linear combinatorial optimization problem Π that satisfies some properties, then the problem of finding an R-approximate solution for Π is polynomially solvable. We now show that for $R > 1$ this result holds up to some arbitrary small error $\varepsilon > 0$. More precisely, we proceed as follows.

Let Π be a linear combinatorial optimization problem from NPO that satisfies the same two properties as those given in Theorem 6 and for which we use a cost-explicit encoding scheme. Furthermore, let N be a polynomially searchable neighborhood function for Π with a performance bound of R. We now present an algorithm for which we will prove that for any precision $\varepsilon > 0$ it (*i*) has a running time that is polynomially bounded in both the input size and $\frac{1}{\varepsilon}$ and (*ii*) has a performance bound of $R + \varepsilon$.

Let $I = (S, c)$ be a problem instance of Π, and suppose that we perform iterative improvement on I, where we take an arbitrary $s \in S$ as a starting solution. By assumption, s can be derived in polynomial time. We turn iterative improvement into a polynomial-time algorithm in the following way: we replace the function c in I by c', such that in each iteration the cost of the solution derived by iterative improvement decreases by at least q and $f'(s)/q$ is polynomially bounded, where $f'(s)$ is the cost of solution s with respect to the new function $c' : E \to \mathbb{Q}^+$. The former property is achieved by defining for any $e \in E$

$$c'(e) = \left\lceil \frac{c(e)}{q} \right\rceil q.$$

The latter property is achieved by defining q as

$$q = \frac{f(s)\varepsilon}{2nR\,(R+\varepsilon)}$$

since we then have

$$\frac{f'(s)}{q} \leq \frac{f(s)+nq}{q} = 2nR\left(\frac{R}{\varepsilon}+1\right)+n,$$

which is polynomially bounded in $|I|$ and $\frac{1}{\varepsilon}$.

Using the polynomial-time algorithm described, we can now construct a polynomial-time $(R+\varepsilon)$-approximation algorithm $\mathcal{A}_\varepsilon(N)$ for Π. Algorithm $\mathcal{A}_\varepsilon(N)$ proceeds in phases. In each phase the algorithm described is executed, where the final solution of the previous phase is taken as a starting solution. If the algorithm arrives at a solution for which the cost is at most half the cost of the solution with which the phase started, then the algorithm is interrupted, and the next phase is initiated. If the algorithm terminates, i.e., if it arrives at a local optimum, then the overall algorithm $\mathcal{A}_\varepsilon(N)$ also terminates. In Fig. 5 algorithm $\mathcal{A}_\varepsilon(N)$ is given in pseudo-code.

Lemma 3. *Let $\Pi \in$ NPO be a linear combinatorial optimization problem that is closed under scaling and for which we can derive a feasible solution in polynomial time. Furthermore, let N be a polynomially searchable neighborhood function for Π. Then the algorithm $\mathcal{A}_\varepsilon(N)$ given in Fig. 5 runs in polynomial time in the case that we use a cost-explicit encoding scheme for Π.*

Proof The number of phases executed by $\mathcal{A}_\varepsilon(N)$ is bounded by $\log(nc_{\max})$, where $c_{\max}$ is the maximum cost $c(e)$ of any element $e \in E$. As a cost-explicit encoding scheme is assumed, this implies that the number of executed phases is polynomially bounded. Combined with the above-derived polynomial running time of each individual phase, this proves the polynomial-time running time of $\mathcal{A}_\varepsilon(N)$. □

The next lemma focuses on the quality of the solutions returned by $\mathcal{A}_\varepsilon(N)$.

Lemma 4. *Let Π be a linear combinatorial optimization problem that is closed under scaling, and let N be a neighborhood function for Π with performance bound R. Then the algorithm $\mathcal{A}_\varepsilon(N)$ given in Fig. 5 is an $(R+\varepsilon)$-approximation algorithm.*

Proof Let s_{init}, q, c', and f' denote the corresponding values at the moment $\mathcal{A}_\varepsilon(N)$ terminates, let s be the solution returned by the algorithm, and let s^* be an optimal solution to the problem instance $I = (S, c)$, which is the problem instance that is given as input to the algorithm.

algorithm $\mathcal{A}_\varepsilon(N)$

begin

$s :=$ some initial solution;

$stop :=$ FALSE;

while $\neg stop$ **do**

begin

$s_{\text{init}} := s$;

$q := f(s_{\text{init}})\varepsilon/(2nR(R+\varepsilon))$;

for all $e \in E$ **do** $c'(e) := \left\lceil \frac{c(e)}{q} \right\rceil q$;

repeat

if $f'(s') \geq f'(s)$ for all $s' \in N(s)$ **then** $stop :=$ TRUE;

else

begin

generate an $s' \in N(s)$;

if $f'(s') < f'(s)$ **then** $s := s'$;

end

until $stop =$ TRUE or $f(s) \leq f(s_{\text{init}})/2$;

end

end;

Fig. 5 Algorithm $\mathcal{A}_\varepsilon(N)$ for a linear combinatorial optimization problem Π that is closed under scaling, an accompanying neighborhood function N, and an arbitrary precision $\varepsilon > 0$. (S, c) denotes an arbitrary problem instance of Π that is given as input to the algorithm. Cost function f relates to problem instance (S, c), while cost function f' relates to problem instance (S, c')

According to the definition of function c', we have $f(s) \leq f'(s)$. Furthermore, as s is locally optimal for problem instance $I' = (S, c')$ (but not necessarily for I) and as the neighborhood function has a performance bound of R, by assumption, we have $f'(s) \leq R \cdot f'(s^*)$. Using this, we can derive

$$f(s) \leq R \cdot f'\left(s^*\right) = R \sum_{e \in s^*} \left\lceil \frac{c(e)}{q} \right\rceil q \leq R \sum_{e \in s^*} (c(e) + q) \leq R \cdot f\left(s^*\right) + Rnq. \tag{2}$$

Moreover, $f(s_{\text{init}})/2 < f(s)$ holds because otherwise the last phase of the algorithm would have been interrupted just before setting the variable *stop* to true. From this and the definition of q, it now follows that $q < f(s)\frac{\varepsilon}{nR(R+\varepsilon)}$. Combined with (2) this gives $f(s) \leq R \cdot f(s^*) + f(s)\frac{\varepsilon}{R+\varepsilon}$, which can be rewritten as $f(s) \leq (R + \varepsilon) f(s^*)$. □

Theorem 7. *Let $\Pi \in$ NPO be a linear combinatorial optimization problem that is closed under scaling and for which we can derive a feasible solution in polynomial time. Furthermore, let N be a polynomially searchable neighborhood function for Π with performance bound R. Then the algorithm $\mathcal{A}_\varepsilon(N)$ given in Fig. 5 is a polynomial-time $(R + \varepsilon)$-approximation algorithm in the case that we use a cost-explicit encoding scheme for Π.*

Proof The theorem follows directly from Lemmas 3 and 4. □

Time Complexity

Since optimal algorithms for NP-hard problems have exponential running time, we presented local search as an approach that trades off quality against performance. More precisely, it tries to reduce execution time by not solving the real optimization problem, but an associated local search problem, as defined below.

Definition 8. An *instance* of a local search problem is a triple (S, f, N), where (S, f) is an instance of a combinatorial optimization problem and $N: S \to 2^S$ is a neighborhood function on S. □

Definition 9. A *local search problem* Π_{LS} is specified by a set of problem instances, and it is either a minimization or a maximization problem. The problem is to find for a given instance (S, f, N) a locally optimal solution $\hat{s} \in S$. For a minimization problem, this means that $f\left(\hat{s}\right) \leq f(s)$ for all $s \in N(s)$, and for a maximization problem, it means that $f\left(\hat{s}\right) \geq f(s)$ for all $s \in N(s)$. □

For the time complexity of a local search algorithm, we can both be interested in the average case time complexity or in a guaranteed, i.e., worst-case time complexity. In this section we focus on the latter. First we study the computational complexity of local search problems. This means that we try to find an answer to the question of how hard the problem is that a local search algorithm tries to solve. This section is based on the paper of Yannakakis [11], who gives an

excellent presentation of the complexity theory for local search problems introduced by Johnson, Papadimitriou, and Yannakakis [14]. Second, we look at the time complexity of the basic local search algorithm: iterative improvement.

Computational Complexity of Local Search Problems

To distinguish easy *decision* problems from hard ones, the theory of NP-completeness has been developed. In a nutshell, the idea of this theory is as follows. A class NP of decision problems is defined that contains the decision variants of many apparently hard and interesting combinatorial optimization problems. To relate the complexity of two problems in NP, polynomial-time reductions are introduced. If a problem Π is polynomially reducible to problem Π', then Π' is at least as hard as Π, which means that a polynomial-time algorithm for Π' can only exist if it exists for Π. The problems in NP to which all problems in this class are polynomially reducible are called NP-complete. These problems are the hardest problems in NP since they only allow an efficient polynomial-time algorithm if such an algorithm exists for all problems in NP. This is very unlikely as no polynomial-time algorithm has been found for an NP-complete problem despite the enormous amount of effort that has been spent by many experts on finding one.

A similar strategy is used to identify hard local search problems for which it is very unlikely that they can be solved by a polynomial-time algorithm. First of all, we define a complexity class, called PLS, which contains local search problems that are specified by a combinatorial optimization problem from NPO and a "reasonable" corresponding neighborhood function. Next, we present a reduction to express that a local search problem is at least as hard as another given local search problem. Using this reduction, we can identify the hardest problems in PLS. Because no polynomial-time algorithm is known for any of these PLS-complete problems, as they are called, it is commonly believed that no such algorithm exists for these problems.

We now further elaborate on this theory. Let Π be a combinatorial optimization problem from NPO, and let N be a neighborhood function for it. A successful application of local search for this problem depends on whether it is easy to generate an initial solution and to check whether a solution is locally optimal and, if not, to derive a better neighbor. In terms of iterative improvement, this means that both the initialization and each single iteration of the algorithm can be implemented to run in polynomial time. The number of iterations, however, may be exponential. The class PLS contains all local search problems in which the properties described are satisfied.

Definition 10. Let Π_{LS} be a local search problem, and let Π be the underlying combinatorial optimization problem. Local search problem Π_{LS} is in the class *PLS (Polynomial-time Local Search)* if $\Pi \in$ NPO and if two polynomial-time algorithms A and B exist that satisfy the following properties.

- For a problem instance (S, f, N) of Π_{LS}, algorithm A returns a solution $s \in S$.
- For a problem instance (S, f, N) of Π_{LS} and a solution $s \in S$, algorithm B decides whether s is a local optimum and, if this is not the case, it returns a neighboring solution with better cost.

□

The following definition specifies a PLS-reduction by which we can relate the complexity of two local search problems. From the definition it follows directly that if local search problem Π_{LS} is PLS-reducible to local search problem Π'_{LS}, then the existence of a polynomial-time algorithm for Π'_{LS} implies the existence of such an algorithm for Π_{LS}. In other words, Π'_{LS} is at least as hard as Π_{LS}. Furthermore, it can be verified that PLS-reductions are transitive, which means that if Π_{LS} is PLS-reducible to Π'_{LS} and Π'_{LS} is PLS-reducible to Π''_{LS}, then Π_{LS} is also PLS-reducible to Π''_{LS}.

Definition 11. Local search problem Π_{LS} is *PLS-reducible* to local search problem Π'_{LS}, denoted by $\Pi_{\text{LS}} \propto \Pi'_{\text{LS}}$, if two polynomial-time algorithms φ_1 and φ_2 exist that satisfy the following properties; see also Fig. 6.

- Algorithm φ_1 transforms a problem instance I of Π_{LS} into a problem instance $\varphi_1(I)$ ofΠ'_{LS}.
- Algorithm φ_2 maps a problem instance $I = (S, f, N)$ of Π_{LS} and a solution $s' \in S'$ with $\varphi_1(I) = (S', f', N')$ to a solution $s \in S$.
- For a problem instance I of Π_{LS}, we have that if $s' \in S'$ is a local optimum for $\varphi_1(I) = (S', f', N')$, then $\varphi_2(I, s')$ is a local optimum for I.

The pair (φ_1, φ_2) is called a *PLS-reduction*. □

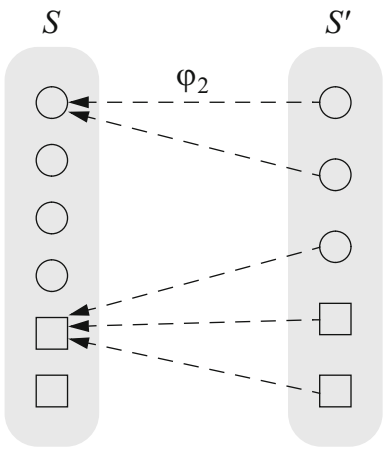


Fig. 6 Example of algorithm φ_2 for mapping solutions from S' to solutions from S. The squares indicate the local optima

We note that the definition of φ_2 can be weakened. For φ_2 it suffices that instead of all solutions in S', only the local optima are mapped to a solution in S. However, if we have $\Pi_{LS} \in$ PLS, then we can easily transform such a φ_2 into an algorithm that satisfies Definition 11 by mapping all solutions that are not in the domain of the restricted φ_2 to the solution returned by the polynomial-time algorithm A defined in Definition 10. Analogously to NP-completeness, we define the notion of PLS-completeness to indicate the hardest problems in PLS.

Definition 12. A local search problem in PLS is *PLS-complete* if each problem $\Pi_{LS} \in$ PLS is PLS-reducible to it. □

Let NPCO contain all NP-hard problems from NPO. When determining the complexity of solving some combinatorial optimization problem from NPO to optimality, one generally aims to decide whether the problem is in PO or in NPCO. It is therefore interesting to relate the class PLS and the subclass of PLS-complete problems to the classes PO, NPO, and NPCO. This is the focus of the remainder of this section.

First we show that PLS $\subseteq$ NPO. Let Π_{LS} be an arbitrary local search problem in PLS. Problem Π_{LS} can be formulated as a minimization problem $\Pi \in$ NPO by defining an instance $(S, f') \in \Pi$ for any instance $(S, f, N) \in \Pi_{LS}$, such that cost function $f': S \rightarrow \{0, 1\}$ satisfies $f'(s) = 0$ if s is locally optimal and $f'(s) = 1$ otherwise. The existence of algorithm B as defined in Definition 10 for Π_{LS} implies that f' is computable in polynomial time. From this it follows that Π is indeed a problem from NPO. Moreover, Π_{LS} and Π are clearly equivalent. As a result, we get PLS $\subseteq$ NPO.

We now show that PO $\subseteq$ PLS. Let $\Pi \in$ PO and let $\mathcal{A}$ be an optimal polynomial-time algorithm for Π. Without loss of generality, we assume that Π is a minimization problem. Then we define minimization problem $\Pi' \in$ PO such that any instance $I = (S, f)$ of Π corresponds to an instance (S, f') of Π', where $f'(s) = 0$ if solution s is optimal with respect to cost function f and $f'(s) = 1$ otherwise. If we define a neighborhood function for Π' in such a way that each solution has only one neighbor, namely, the solution returned by $\mathcal{A}$, then we obtain a local search problem that is equivalent to Π. Furthermore, the problem is in PLS. Algorithm A in Definition 10 can be defined as $\mathcal{A}$, and algorithm B in this definition can be realized by first comparing the cost of the solution given by $\mathcal{A}$ with the cost of a given solution s and by then returning the optimal solution derived by $\mathcal{A}$ if the latter cost is larger. Hence, we now obtain the following result.

Theorem 8. *The complexity classes PO, PLS, and NPO satisfy*

$$PO \subseteq PLS \subseteq NPO.$$

□

From the relation PLS $\subseteq$NPO, it follows that a PLS-complete problem is not harder than any NP-hard problem in NPO, as will now be shown. Suppose that a polynomial-time algorithm exists for an NP-hard problem in NPCO. This means that all problems in NPO are polynomially solvable, including those in PLS. To substantiate the intractability of PLS-complete problems, a more interesting question is whether PLS-complete problems are also at least as hard as the problems in NPCO. If the answer were to be affirmative, then the existence of a polynomial-time algorithm for a PLS-complete problem would be as unlikely as the existence of such an algorithm for a problem in NPCO. However, this is not the case because the following theorem states that if NP $\neq$ co-NP, then a PLS-complete problem is not NP-hard. Hence, in the unlikely case that PLS-complete problems turn out to be polynomially solvable, this still does not imply that the problems in NPCO are also solvable in polynomial time. Figure 7 depicts the relations between the different complexity classes.

Theorem 9. *If NP $\neq$ co-NP, then none of the problems in PLS are NP-hard.*

Proof The result can be proved in a similar way as Theorem 5. Note that Theorem 5 easily follows from this theorem. Suppose that a problem $\Pi_{\mathrm{LS}} \in$ PLS is NP-hard. We show that this contradicts NP $\neq$ co-NP by showing that it implies that, for any decision problem Π_D in NP, the complementary problem Π_D^c obtained by reversing the answers in Π_D is also in NP.

As Π_{LS} is NP-hard, a polynomial-time algorithm $\mathcal{A}$ exists that decides Π_D, where $\mathcal{A}$ may use an oracle that, for any problem instance of Π_{LS}, returns a local optimum. Hence, if for a problem instance I of Π_D we are given a sequence consisting of local optima that may be returned successively by the oracle during the execution of $\mathcal{A}$ on I, then we can determine in polynomial time whether I is a yes-instance or a no-instance of Π_D. As a result, we can use this sequence of locally optimal solutions as a certificate for yes-instances of Π_D^c. Furthermore, the certificate, which is of polynomial size, can be checked in polynomial time for validity by substituting each call of the oracle in $\mathcal{A}$ by a polynomial-time procedure that checks whether the corresponding solution in the certificate is a solution to the problem instance of Π_{LS} given to the oracle. The existence of polynomial-time

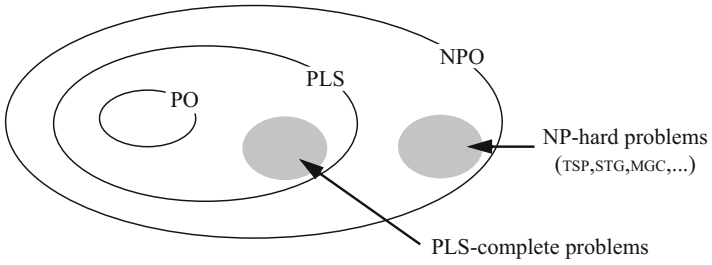


Fig. 7 Positioning of the classes PO, PLS, and NPO in the case that P $\neq$ NP, NP $\neq$ co-NP, and PO $\neq$ PLS

algorithm B in Definition 10 shows that this polynomial-time procedure exists. This proves $\Pi_D^c \in$ NP, which completes the proof of the theorem. □

Proving PLS-Completeness

As for the theory of NP-completeness, the theory of PLS-completeness only becomes useful once we have a first problem for which we can prove that it is among the hardest of the class PLS. After that, we can prove PLS-completeness of a problem by simply proving that a known PLS-complete problem is PLS-reducible to it. The first problem for which PLS-completeness has been proved is circuit/flip as defined below.

Suppose we want to apply local search to the problem of finding an integer s with $0 \leq s < 2^n$ for which a given function $f: \{0, 1, \ldots, 2^n - 1\} \to \{0, 1, \ldots, 2^m - 1\}$ is minimized. We can use the neighborhood function in which $s' \in \{0, 1, \ldots, 2^n - 1\}$ is a neighbor of s if s' can be obtained from s by flipping exactly one of the n bits in the binary representation of s. The corresponding local search problem, formulated in terms of Boolean circuits, is the first problem that has been proved to be PLS-complete.

Boolean circuits are theoretical counterparts of the digital circuits from which computers are made. They compute Boolean functions $f: \{0, 1\}^n \to \{0, 1\}^m$, and, conversely, each Boolean function is computed by a circuit.

Definition 13. A *Boolean circuit* is a directed acyclic graph $D = (V, A)$. The node set V consists of n input nodes and $|V| - n$ gates. The input nodes have indegree zero and are labeled by the binary variables $x_1, x_2, \ldots, x_n$. The labels of the gates are taken from the set $\{\wedge, \vee, \neg\}$ of Boolean functions. The gates with outdegree zero are called the output nodes, and they are additionally labeled by the binary variables $y_1, y_2, \ldots, y_m$. Boolean circuit D computes a Boolean function $f: \{0, 1\}^n \to \{0, 1\}^m$ by deriving for given values of the input variables $x_1, x_2, \ldots, x_n$ corresponding values for the output variables $y_1, y_2, \ldots, y_m$ in the following way.

Let l be the label of a gate g. If l is given by $\neg$, then the value of g is one minus the value of the node from which the only incoming edge of g is incident. Next, suppose that $l \in \{\wedge, \vee\}$. Then g has exactly two incoming edges. Let o_1, o_2 be the values of the nodes from which these two edges are incident. If g is labeled $\wedge$, then it is assigned the value 1 if $o_1 = o_2 = 1$, and it is assigned the value 0, otherwise. If g is labeled $\vee$, then it is assigned the value 0 if $o_1 = o_2 = 0$, and it is assigned the value 1 otherwise. The *size* of Boolean circuit $D = (V, A)$ is given by the number of nodes in V. □

By interpreting the input and output vectors as two numbers written in binary notation, i.e., by interpreting the input as $\sum_{i=1}^{n} 2^{i-1} x_i$ and the output as $\sum_{i=1}^{m} 2^{i-1} y_i$, a Boolean circuit can compute an integer function. Hence, the local search problem mentioned above can be formalized by the following definition. We note that the

Hamming distance of two strings s and s' of equal size, denoted by $H(s, s')$, is given by the number of positions in which s and s' differ.

Definition 14 (Circuit/flip). Given is a Boolean circuit D with n input nodes x_1, x_2, ..., x_n and m output nodes y_1, y_2, ..., y_m. The neighborhood function, called flip, is based on the solution space $\{0, 1\}^n$ containing all possible input vectors (x_1, x_2, ..., x_n) of the circuit. A solution s is a neighbor of solution s' if s and s' have a Hamming distance $H(s, s')$ of one. Find a solution $\hat{s}$ that is locally optimal with respect to the cost function

$$f\left(\hat{s}\right) = \sum_{i=1}^{m} 2^{i-1} y_i,$$

which gives the integer interpretation of the output of Boolean circuit D on input $\hat{s}$. In min-circuit/flip, it is our goal to minimize the cost function, while in max-circuit/flip it is our goal to maximize the cost function. □

Theorem 10. *Min-circuit/flip and max-circuit/flip are both PLS-complete.* □

Although very interesting, the proof of the above theorem is rather complex and therefore left out of this chapter. For details, we refer to Yannakakis [11], Johnson, Papadimitriou, and Yannakakis [14] or Michiels, Aarts, and Korst [1]. To focus on how the theory can be used, we now give an example of how it can be used to show that the uniform graph partitioning problem with the Kernighan-Lin neighborhood function is PLS-complete.

Definition 15 (Uniform Graph Partitioning (UGP)). Given is an edge-weighted graph $G = (V, E)$ with $|V| = 2n$ and a weight $w(e) \in \mathbb{N}$ for each edge $e \in E$. Find a partition of V into two subsets V_1 and V_2 with $|V_1| = |V_2| = n$, such that the sum of the weights of the edges that have one endpoint in V_1 and one endpoint in V_2 is minimal, i.e., such that

$$c\left(V_1, V_2\right) = \sum_{e \in E \cap (V_1 \times V_2)} w(e)$$

is minimal. □

An obvious neighborhood function for UGP is the swap neighborhood function in which the neighbors of a partition (V_1, V_2) are obtained by interchanging a node from V_1 with a node from V_2. Variable-depth search gives an effective approach for making the swap neighborhood function more powerful. The main idea is that we allow multiple swaps to be performed simultaneously. We call the resulting neighborhood function Kernighan-Lin, after its two inventors. For each i with $1 \leq i \leq n$, the Kernighan-Lin neighborhood of a partition (V_1, V_2) contains exactly one partition that is obtained from (V_1, V_2) by exchanging i nodes from V_1 with i nodes from V_2.

The n Kernighan-Lin neighbors of a partition (V_1, V_2), written as $\left(V_1^{(i)}, V_2^{(i)}\right)$ for $1 \leq i \leq n$, can be determined in n steps as follows. In Step i we derive partition $\left(V_1^{(i)}, V_2^{(i)}\right)$ by performing the best possible swap on (V_1, V_2) if $i = 1$ and on the partition $\left(V_1^{(i-1)}, V_2^{(i-1)}\right)$ derived in the previous step if $2 \leq i \leq n$. We are not allowed to select a node that has already been selected before for a swap. By the best possible swap, we mean the swap of the nodes from $V_1^{(i-1)}$ and $V_2^{(i-1)}$ that produces the maximum decrease in cost or, if such a swap does not exist, a minimum increase in cost. In the case of a tie, a tie-breaking rule is used to choose a unique pair of nodes that is swapped. The neighborhood consists of n partitions because, after n swaps, all nodes of the graph have been selected for an exchange and, by definition, each node may only be selected once. In other words, the neighbors are obtained by performing iterative improvement using the best-improvement pivoting rule and the swap neighborhood function. However, unlike the standard iterative improvement algorithm, we have the additional constraint that each node may only be selected once for a swap and that we accept a best cost-increasing neighbor if we are in a local optimum.

We prove that UGP with the Kernighan-Lin neighborhood function is PLS-complete by a reduction from another local search problem. Before defining this problem, we introduce some terminology. Let U be a set of binary variables. A truth assignment for U assigns to each variable in U either the value 0 (false) or 1 (true). A variable $u \in U$ is called a positive literal, and its negation $\overline{u} = 1 - u$ is called a negative literal. A clause is a set of literals and/or constants, where a constant has the value either 0 or 1.

Definition 16 (Positive Not-All-Equal Max-3Sat (POS NAE MAX-3SAT)). Given is a set U of binary variables, a set C of clauses over U, and a positive integer weight $w(c)$ for each clause $c \in C$. A clause does not contain negative literals, and the number of positive literals plus the number of constants in each clause is at most three. A clause is satisfied by some truth assignment if and only if it contains at least one literal/constant with value 1 and at least one literal/constant with value 0. The problem is to find a truth assignment that maximizes the sum of the weights of clauses that are satisfied. □

A possible neighborhood function for POS NAE MAX-3SAT is the flip neighborhood function in which truth assignment t' is a neighbor of truth assignment t if t' can be obtained from t by changing (flipping) the value of one variable. In much the same way as the Kernighan-Lin neighborhood function for UGP is obtained by applying variable-depth search to the swap neighborhood function, the Kernighan-Lin neighborhood function for POS NAE MAX-3SAT is obtained by applying variable-depth search to the flip neighborhood function. This means that a truth assignment t' is a neighbor of truth assignment t with respect to the Kernighan-Lin neighborhood function if it can be derived from t by performing a sequence of flips, where in each step the most profitable (least unprofitable) flip is chosen from the flips that change the value of a variable that has not been flipped before

in this sequence. An arbitrary rule may be used for breaking ties. We now state the following result without proof. For a proof, we again refer to Yannakakis [11], Johnson, Papadimitriou, and Yannakakis [14] or Michiels, Aarts, and Korst [1].

Theorem 11. *POS NAE MAX-3SAT with the Kernighan-Lin neighborhood function is PLS-complete.* □

Knowing that POS NAE MAX-3SAT/Kernighan-Lin is PLS-complete, we are now able to prove that UGP/Kernighan-Lin is PLS-complete.

Theorem 12. *UGP with the Kernighan-Lin neighborhood function is PLS-complete.*

Proof Let the cut of a partition (V_1, V_2) be defined as the set of edges that have one endpoint in V_1 and one in V_2.

UGP is equivalent to its maximization variant in which we are asked to partition the nodes of a graph into two sets V_1, V_2 of equal size, such that the weight $c(V_1, V_2)$ of the resulting cut is maximized instead of minimized. This can be seen as follows. Let $G = (V, E)$ be a weighted graph with maximum weight $W = \max_e \in_E w(e)$, and let K be the complete graph on the same node set V, in which the weight of an edge e is given by $W - w(e)$ if $e \in E$ and by W otherwise. The cost of a partition (V_1, V_2) in one variant of the problem is then equivalent to n^2W minus the cost of this partition in the other variant. This implies that both problems are indeed equivalent.

When we inspect the given relation between the two variants, it follows easily that the corresponding local search problems under the Kernighan-Lin neighborhood function are also equivalent. As a result, we can prove the theorem by giving a PLS-reduction (φ_1, φ_2) from POS NAE MAX-3SAT/Kernighan-Lin to the maximization version of UGP with the Kernighan-Lin neighborhood function, where we use Theorem 11, which states that the former problem is PLS-complete.

Definition PLS-reduction. Let I be an arbitrary problem instance of POS NAE MAX-3SAT/Kernighan-Lin, and let $x_1, x_2, \ldots, x_m$ be the binary variables of I. The problem instance $\varphi_1(I)$ is defined by the following graph $G = (V, E)$. Node set V contains two nodes x_i and x_i' for each variable x_i in I. The nodes x_i and x_i' represent the value of the variable x_i and its negation, respectively. To V we also add the nodes y and z, which represent the constant 1, and the nodes y' and z', which represent the constant 0. This concludes the definition of V.

The edge set E is partitioned into the two subsets E_1 and E_2. Set E_1 contains all edges that connect an unprimed node x_i with its primed variant. Furthermore, it contains an edge from each of the two 1-nodes y and z to each of the two 0-nodes y' and z'. All these $m + 4$ edges are assigned the same large weight of $W + 1$, where W is defined as the total weight of all clauses in I. The following two conditions are necessary and sufficient to guarantee that a cut contains all edges from E_1.

- Nodes y and z that represent the constant 1 are both assigned to the same subset, and nodes y' and z' that represent the constant 0 are both assigned to the other subset.
- The nodes x_i and x_i' are assigned to different subsets for all $1 \leq i \leq m$.

If these two conditions are satisfied by a partition, we say that the partition is feasible; see Fig. 8 for an example. We can assume the following one-to-one correspondence between a feasible partition and a truth assignment without violating the interpretation of the nodes x_i and x_i' as being the value of variable x_i and its negation. A variable x_i or its negation is assigned the value 1 if and only if the corresponding node is contained in the same subset as the variables y, z that represent the constant 1. This means that if a variable or its negation is assigned the value 0, then the corresponding node is contained in the same subset as the two variables y' and z' that represent the constant 0. We define P_t as the feasible partition corresponding to truth assignment t and R as the set of all feasible partitions, i.e., $R = \{P_t \mid t \text{ truth assignment}\}$.

To be able to derive the cost of a truth assignment t from the feasible partition P_t, we add additional edges to E via E_2. For each clause $c = \{a, b\}$, we add the edge $\{a, b\}$ to E_2 with the same weight as clause c, where for the constants 1 and 0 we use the nodes y and y', respectively. If such an edge contributes its weight to the cost of a feasible partition P_t, then a and b are assigned to different subsets, which imply that the clause $\{a, b\}$ is satisfied by truth assignment t.

For a clause $c = \{a, b, c\}$ of weight w, we include the three edges $\{a, b\}$, $\{b, c\}$, and $\{a, c\}$ in E_2, each with weight $w/2$. It can be verified that the cut of a potentially infeasible partition contains either two out of the three edges or none of the three edges. Moreover, for a feasible partition P_t, the former case implies that clause c is satisfied by t, and the latter case implies that c is not satisfied by t. As a result, the edges contribute w to the cost of a feasible partition P_t if c is satisfied by t and they do not contribute otherwise. We note that if a pair of nodes a, b occurs in several

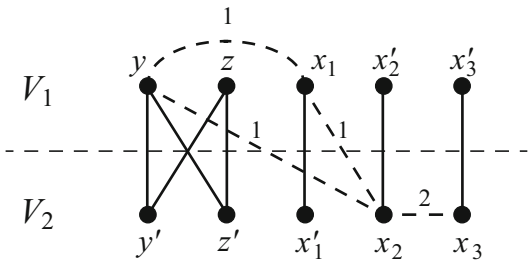


Fig. 8 Feasible partition $P_t = (V_1, V_2)$ for the graph $\varphi_1(I)$, where I is the problem instance of POS NAE MAX-3SAT/Kernighan-Lin consisting of the two clauses $\{x_1, x_2, 1\}$ and $\{x_2, x_3\}$. Both clauses have a weight of two. The undashed edges are from E_1 and the dashed edges are from E_2. Each edge from E_1 has weight $W + 1 = 5$, and the weights of the edges from E_2 are denoted in the figure. Feasible partition P_t corresponds to truth assignment t with $x_1 = 1$ and $x_2 = x_3 = 0$. This truth assignment satisfies the clause $\{x_1, x_2, 1\}$ but not the clause $\{x_2, x_3\}$

clauses, we assign to edge $\{a, b\}$ the weight obtained by summing up the weights that arise from all these clauses. This concludes the definition of G and therefore of problem instance $\varphi_1(I)$. To complete the definition of our PLS-reduction, we define algorithm φ_2 in such a way that it returns for a potentially infeasible partition P the truth assignment that assigns to a variable x_i the value 1 if and only if it is contained in the same subset as variable y. Note that this algorithm complies with the given relation between a truth assignment t and the feasible partition P_t.

Proof strategy. As the total weight W of the edges from E_2 that can be in the cut of a partition is strictly smaller than the weight $W + 1$ of a single edge in E_1 and as the cut of a feasible partition contains all edges from E_1, we can make the following observation.

Observation. The minimum cost over all feasible partitions is strictly larger than the maximum cost over all infeasible partitions.

This implies that, for any feasible partition P_t, the transition graph $T_{\varphi_1(I)}$ of $\varphi_1(I)$ only contains outgoing arcs to other feasible solutions. Consequently, to prove that (φ_1, φ_2) correctly defines a PLS-reduction, it suffices to prove that the transition graph satisfies the following two conditions.

- *Condition 1.* $T_{\varphi_1(I)}$ contains an arc $(P_t, P_{t'})$ if and only if T_I contains the arc (t, t'). See Fig. 9.
- *Condition 2.* For any partition $P \notin R$, transition graph $T_{\varphi_1(I)}$ contains a path to a feasible partition.

In the proof we assume that, to determine the Kernighan-Lin neighborhood of a partition, an arbitrary rule is used for breaking ties in the case that several pairs of nodes qualify for being selected for a swap. Given this rule, we may choose the tie-breaking rule used to determine the Kernighan-Lin neighborhood of a truth assignment as we like because the PLS-completeness result for POS NAE MAX-3SAT/Kernighan-Lin holds regardless of this rule.

Proof of Condition 1 As the cost of a feasible partition P_t is given by $|E_1|(W + 1)$ plus the cost of truth assignment t, the first condition holds if we can prove that

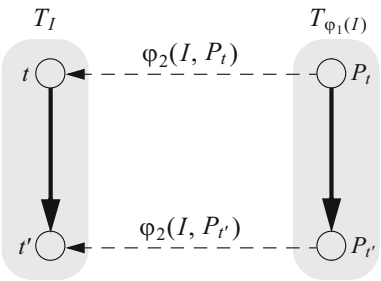


Fig. 9 Correspondence between arc in transition graph T_I and arc in transition graph $T_{\varphi_1(I)}$

for any truth assignment t, partition P_t and t have the same neighborhood. More precisely, it suffices to show that $P_{t'}$ is a neighbor of P_t for $t' \neq t$ if and only if t' is a neighbor of t. Until all pairs of nodes x_i, x_i' have been swapped and thus blocked, we only swap pairs x_i, x_i' of nodes when deriving the neighbors of P_t. This is true since swapping such a pair in a feasible partition results again in a high-quality, feasible partition, while swapping any other pair of nodes results in a low-quality, infeasible partition. After all pairs of nodes x_i, x_i' have been swapped, the nodes y, z and y', z' are swapped in two steps, where the intermediate partition is infeasible. This brings us back to partition P_t.

Swapping x_i and x_i' has the same effect on the cost of a feasible partition as flipping the value of variable x_i on the cost of the corresponding truth assignment. As a result, we obtain that, if the same rule is used for breaking ties in the derivation of the Kernighan-Lin neighborhood of t as is used in the derivation of the Kernighan-Lin neighborhood of P_t, then the feasible neighbors of P_t, excluding P_t, correspond to the truth assignments in the neighborhood of t. This proves the first claimed condition given above.

Proof of Condition 2 To prove the second condition, suppose that it does not hold, which means that an infeasible partition $\hat{P}$ exists that is locally optimal. We show that this gives rise to a contradiction. As $\hat{P} = (V_1, V_2)$ is infeasible, either V_1 contains a pair u, u' of corresponding nodes or the two 1-nodes y, z are split. First of all, consider the former case. Obviously, V_2 also contains a pair v, v' of corresponding nodes as $|V_1| = |V_2|$. Swapping u and v leads to a weight gain of at least $W + 1$ via the edges from E_1, and it leads to a weight reduction of at most W via the edges from E_2. This contradicts the assumption that $\hat{P}$ is locally optimal. Next, assume that the two 1-nodes are split but that V_1 does not contain a pair u, u' of corresponding nodes. The two 0-nodes then also have to be in different subsets. Swapping a 0-node from one subset with a 1-node in the other subset yields a cost improvement of $2(W + 1)$. Again, this contradicts the local optimality of $\hat{P}$. □

It follows from the definition of PLS that an improving solution can be found in polynomial time if it exists. This is true for k-Opt, where an improving tour can be found in $\mathcal{O}\left(n^k\right)$ time. However, this is only practical for very small k. An interesting question is whether we can do better. This is studied by Marx [15] and Guo et al. [16]. They prove for a number of TSP neighborhood functions that the answer to this question is negative. However, the complexity may be exponential in k.

Time Complexity of Iterative Improvement

While the focus of the previous section was on the computational complexity of local search problems, we now study the time complexity of the iterative improvement algorithm when it is used to solve such a problem. A local search problem can be formulated as finding a node in a transition graph with outdegree zero. Iterative improvement is the obvious algorithm for tackling this problem. It

traverses a transition graph, which is directed and acyclic, until it reaches a node without outgoing arcs. The precise path that the algorithm follows is determined by the pivoting rule. This rule specifies the neighboring solution that is selected if a solution has more than one neighbor with better cost. In terms of the transition graph, the pivoting rule prescribes which arc is to be selected in a given node if it has multiple outgoing arcs. Ideally, we would like to use a pivoting rule that takes the shortest path to a local optimum. However, computing this rule may be NP-hard, as, for instance, is the case for TSP with the 2-change neighborhood function [17].

As most of the neighborhood functions used in practice induce relatively small neighborhoods, which is reflected in our definition of the class PLS, it is generally easy to derive efficient implementations of commonly used (heuristic) pivoting rules as first improvement and best improvement. Hence, with regard to the time complexity of iterative improvement, we are particularly interested in the number of iterations required by iterative improvement. Obviously, the maximum number of iterations required by iterative improvement is bounded from above by the longest path that occurs in the transition graph. This upper bound holds regardless of the pivoting rule used and the solution with which the algorithm starts. Whether or not this bound is tight depends on the pivoting rule applied and on the algorithm we use for generating the initial solution. With regard to the starting solution of iterative improvement, we generally do not want to make any assumptions. A reason for this is that because the quality of a starting solution generally has a major effect on the quality of the final solution, we want to have the opportunity to tune the algorithm that derives a starting solution to the application we are considering. Another reason is that, in order to make multiple runs of iterative improvement successful, we like to initialize each run with a solution that is quite different from the starting solutions used in the other runs.

Suppose that we apply iterative improvement to solve a problem Π_{LS} in PLS and let c be the cost of the solution with which we initialize the algorithm. Because the cost function is assumed to be integral and because in each iteration of iterative improvement the cost of the solution improves, the number of iterations the algorithm needs for reaching a local optimum is bounded by $|c - f^*|$, where f^* is the cost of an optimal solution. From this, it follows that if in the combinatorial optimization problem underlying Π_{LS} the cost of a solution is polynomially bounded in the input size, then iterative improvement can be implemented to run in polynomial time for Π_{LS}.

Although not satisfying this strong constraint, the cost functions of many combinatorial optimization problems satisfy the weaker property that the cost of a solution is polynomially bounded in the input size and the magnitude of the largest number that occurs in the problem instance. In these cases iterative improvement has a pseudo-polynomial running time. Examples include the already discussed problems TSP and UGP. For TSP the cost of a solution is, for instance, bounded by n times the longest distance in the distance matrix. In the combinatorial optimization problem underlying circuit/flip, the cost function does not even satisfy this weaker property.

In the next two subsections, we describe two different strategies for analyzing the time complexity of iterative improvement. In the first subsection, we give an

example of the strategy where we analyze a specific implementation of iterative improvement. That is, the analysis assumes a certain pivoting rule. In the next subsection, we take a more generic strategy where we further exploit the theory of PLS-completeness to derive negative results that hold regardless of the pivoting rule used.

Ad Hoc Strategies

In the literature, a number of results have appeared on the maximum number of iterations that are required by an implementation of iterative improvement. Proving a negative result, i.e., that the number of iterations can be exponential, is easiest if we assume the pivoting rule that selects an improving neighbor randomly. In that case, we only need to show that there exists a path in the transition graph that has exponential length. An example of this is given in the result below from Luecker [18]. We note, however, that the result has been improved by Englert, Röglin and Vöcking [19] who prove that also for Euclidean TSP, the 2-change neighborhood function may result in an exponential number of iterations.

Theorem 13. *For Metric TSP with the 2-change neighborhood function, iterative improvement may require an exponential number of iterations to reach a local optimum if the pivoting rule is used that selects an improving move randomly.*

Proof Consider a problem instance I of Symmetric TSP instead of Metric TSP. By adding to each entry d_{ij} of the distance matrix d the maximum distance $d_{\max}$ occurring in d, we obtain a distance matrix that satisfies the triangle inequality. This modification does not affect a run of iterative improvement as for any tour it results in a length increase of exactly $n \cdot d_{\max}$. Hence, to prove the theorem, it suffices to show that it holds for Symmetric TSP.

We now proceed as follows. For any even n, we construct a problem instance I of Symmetric TSP consisting of n cities. For this problem we then derive a sequence of exponentially many tours that can be visited successively by the iterative improvement algorithm. Note that because iterative improvement runs in pseudo-polynomial time for Symmetric TSP, the distance matrix of I will have to contain distances of exponential length.

Because we are considering Symmetric TSP, we have $d_{ij} = d_{ji}$. Therefore, it suffices to define the distance d_{ij} for the case where $i \geq 2$ and either $j > i$ or $j = 1$. The definition of the distance matrix d is visualized in Fig. 10. Formally, for even $i \geq 2$, the distance d_{ij} is given by 2^{2i+3} if j is odd and either $j = 1$ or $j > i$, and d_{ij} is given by $d_{ij} = 2^{2i+4}$ if j is even and $j > i$. For odd $i \geq 2$, we apply the same case distinction: d_{ij} is given by 2^{2i} if j is odd and either $j = 1$ or $j > i$, and it is given by 2^{2i+3} if j is even and $j > i$.

We prove that iterative improvement can take an exponential number of iterations to transform the tour $(1, 2, \ldots, n)$ into the tour $(1, n-1, n-2, \ldots, 2, n)$ for the problem instance described. We prove this result by showing by induction on q that

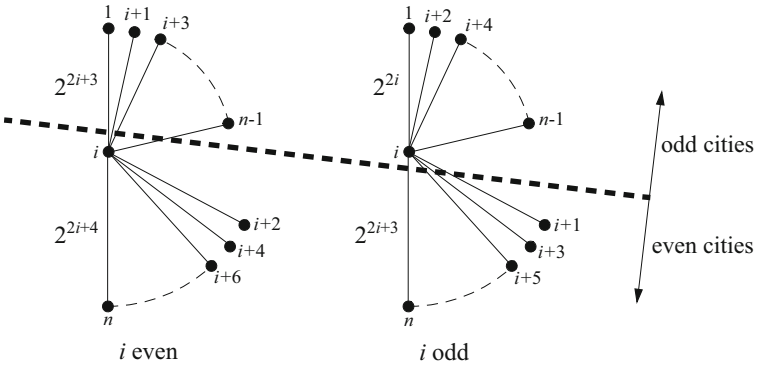


Fig. 10 Definition of the distance matrix d in the problem instance of Symmetric TSP introduced in the proof of Theorem 13, where $i \geq 2$ is an arbitrary city that is either even (left) or odd (right)

the following assertion holds. Let tour τ contain a subtour $x, 2, 3, \ldots, 2q+1, y$ for a $q \leq \frac{n-2}{2}$, where x and y are cities, possibly 1 and $2q+2$, such that for all $2 \leq i \leq 2q+1$ we have $d_{x,i} = d_{1,i}$ and $d_{i,y} = d_{i,2q+2}$. Then a sequence of at least 2^{q-1} improving 2-changes exists that transforms tour τ into τ_q, where tour τ_q is equivalent to τ, except that the subtour $x, 2, 3, \ldots, 2q+1, y$ is replaced by the subtour $x, 2q+1, 2q, \ldots, 2, y$, in which the order of the $2q$ middle cities has been reversed.

We first prove the basis case $q = 1$. Tour $\tau_q = \tau_1$ that contains $x, 3, 2, y$ is obtained from tour τ that contains $x, 2, 3, y$ by the single 2-change that replaces the edges $\{x, 2\}$ and $\{3, y\}$ by $\{x, 3\}$ and $\{2, y\}$. As $2^{q-1} = 1$, the basis case now follows if τ_1 is shorter than τ. This is true because for the cost improvement $\Delta(\tau, \tau_1)$ of τ_1 in comparison with τ, we can derive

$$\begin{aligned}\Delta(\tau, \tau_1) &= d_{1,2} + d_{3,4} - d_{1,3} - d_{2,4} \\ &= 2^7 + 2^9 - 2^6 - 2^8 \\ &> 0.\end{aligned}$$

Next, consider the case $q \geq 2$. We show how τ can be transformed into τ_q in at least 2^{q-1} improving 2-changes, where we distinguish five phases in the transformation. The final tour in phase i is denoted by $\tau^{(i)}$. Hence, $\tau^{(5)} = \tau_q$.

In the first phase, $\tau^{(1)} = \tau_{q-1}$ is derived from τ. By the induction hypothesis, this can be done in 2^{q-2} improving 2-changes. In Phase 2 we next change the order of the cities $2q$ and $2q+1$ in subtour $x, 2q-1, 2q-2, \ldots, 2, 2q, 2q+1, y$ of $\tau^{(1)}$ by performing a 2-change on the edges $\{2, 2q\}$ and $\{2q+1, y\}$. This results in a tour $\tau^{(2)}$ with shorter length than $\tau^{(1)}$ since

$$\begin{aligned}\Delta\left(\tau^{(1)}, \tau^{(2)}\right) &= d_{2,2q} + d_{2q+1,2q+2} - d_{2,2q+1} - d_{2q,2q+2} \\ &= 2^8 + 2^{4q+5} - 2^7 - 2^{4q+4} \\ &> 0.\end{aligned}$$

In Phase 3 the first and last edges $\{x, 2q-1\}$ and $\{2q, y\}$ occurring in subtour x, $2q-1, 2q-2 \ldots, 2, 2q+1, 2q, y$ of $\tau^{(2)}$ are replaced by the two edges $\{x, 2q\}$ and $\{2q-1, y\}$. This yields a cost improvement of

$$\begin{aligned}\Delta\left(\tau^{(2)}, \tau^{(3)}\right) &= d_{1,2q-1} + d_{2q,2q+2} - d_{1,2q} - d_{2q-1,2q+2} \\ &= 2^{4q-2} + 2^{4q+4} - 2^{4q+3} - 2^{4q+1} \\ &> 0.\end{aligned}$$

The resulting tour $\tau^{(3)}$ contains x, $2q$, $2q+1$, 2, 3, $\ldots$, $2q-1$, y. By the definition of the distance matrix, we have $d_{2q+1,i} = d_{1,i}$ and $d_{i,2q+2} = d_{i,2q}$ for all $2 \leq i \leq 2q$. As a result, we can apply the induction hypothesis to the subtour x', 2, 3, $\ldots$, $2q-1$, y' of $\tau^{(3)}$ with $x' = 2q+1$ and $y' = y$. This yields that $\tau^{(3)}$ can be transformed into $\tau^{(4)}$ by at least 2^{q-2} improving 2-changes, where $\tau^{(4)}$ is given by $\tau^{(3)}$ with the order of the cities 2, 3, $\ldots$, $2q-1$ reversed, i.e., $\tau^{(4)}$ contains x, $2q$, $2q+1$, $2q-1$, $2q-2$, $\ldots$, 2, y. In the final phase, we reverse the order of the cities $2q$ and $2q+1$ by performing a 2-change on $\{x, 2q\}$ and $\{2q+1, 2q-1\}$. This again results in an improvement of the tour as we have

$$\begin{aligned}\Delta\left(\tau^{(4)}, \tau^{(5)}\right) &= d_{1,2q} + d_{2q-1,2q+1} - d_{1,2q+1} - d_{2q-1,2q} \\ &= 2^{4q+3} + 2^{4q-2} - 2^{4q+2} - 2^{4q+1} \\ &> 0.\end{aligned}$$

In conclusion, we have transformed τ into $\tau^{(5)} = \tau_q$ by only performing improving 2-changes. The total number of 2-changes is at least 2^{q-2} in Phase 1, one in Phases 2 and 3, at least 2^{q-2} in Phase 4, and one in Phase 5. This makes a total of more than 2^{q-1} improving 2-changes. This proves the induction hypothesis and consequently it proves the theorem. □

The above result does not match the subquadratic number of iterations that one observes in practice [20]. To better understand the average case behavior, Chandra, Karloff, and Tovey [6] analyzed the expected length of the longest path in the transition graph in case the n cities are placed uniformly at random in the unit square. They proved an expected length of $\mathcal{O}\left(n^{10}\log(n)\right)$.

General Strategy

Suppose that, either empirically or via an analysis as given in the above subsection, we find out that iterative improvement takes an exponential number of steps. A question that arises is whether this bad worst-case behavior can be avoided by choosing a less naive pivoting rule or whether it is unavoidable due to the structure of the transition graph. This means that we are interested in the question of whether iterative improvement would still run in worst-case exponential time if for the pivoting rule we were to have an oracle at our disposal that gives us a neighbor that leads to a nearest local optimum. This problem, called the transition graph complexity problem, is formalized as follows.

Definition 17. Let T be a transition graph and let $\hat{V}$ be the nodes with outdegree zero (they represent the local optima). The potential of a node v is the minimum distance in T between v and a node in $\hat{V}$. The potential of T is the maximum potential over all its nodes.

Definition 18. For a given local search problem Π_{LS}, the *transition graph complexity problem* corresponds to deciding whether Π_{LS} can induce transition graphs with an exponentially large potential. □

In this section we study the transition graph complexity problem for PLS-complete problems. For PLS-complete problems, we already know that, unless PO = PLS, they cannot be solved in polynomial time. Solving the transition graph complexity problem gives us additional information on whether the unavoidable exponential running time is caused by the structure of the transition graph or is only caused by the intractability of computing an optimal pivoting rule.

The reason we introduced local search problems is that they are the problems that are actually solved by iterative improvement. Solving a local search problem is generally not a goal in itself. Therefore, considerably less attention has been paid by scientists to solving these problems efficiently than to finding efficient algorithms for NP-hard combinatorial optimization problems and NP-complete decision problems. As a result, we have stronger evidence for the inequality $\text{P} \neq \text{NP}$, which is equivalent to $\text{PO} \neq \text{NPO}$, than for $\text{PO} \neq \text{PLS}$. This brings us to the second relevant implication of knowing that the potential of a transition graph can be exponentially large for a given PLS-complete problem. It proves an exponential worst-case running time for iterative improvement even in the case that PO = PLS. We nevertheless emphasize that it is commonly believed that $\text{PO} \neq \text{PLS}$ holds.

To solve the transition graph complexity problem, we proceed as follows. We defined PLS-reductions such that if $\Pi_{\text{LS}} \propto \Pi'_{\text{LS}}$ holds and if Π_{LS} cannot be solved in polynomial time, then Π'_{LS} cannot be solved in polynomial time either. Below we refine the definition of a PLS-reduction such that, in addition to this behavior, we have that if $\Pi_{\text{LS}} \propto \Pi'_{\text{LS}}$ holds and if a transition graph for Π_{LS} can have an exponentially large potential, then a transition graph of Π'_{LS} can also have an exponentially large potential. This more powerful PLS-reduction is called a tight PLS-reduction. We show that the class PLS contains a local search problem for which the transition graph can have an exponentially large potential. Furthermore, most known PLS-reductions, including the ones for circuit/flip and Theorem 11, are tight [1, 11, 14]. We now elaborate a bit more on the details.

Let (φ_1, φ_2) define a PLS-reduction between two local search problems Π_{LS} and Π'_{LS}, and let $I = (S, f, N)$ be an arbitrary problem instance of Π_{LS} with image $\varphi_1(I) = (S', f', N')$. By definition, algorithm φ_2 maps any solution in S' to a solution in S, such that a local optimum in S' is also a local optimum in S. Figure 11a depicts possible transition graphs T_I and $T_{\varphi_1(I)}$ of I and $\varphi_1(I)$, respectively, and a possible choice for φ_2. For (φ_1, φ_2) to be a tight PLS-reduction, a subset R of S' has to

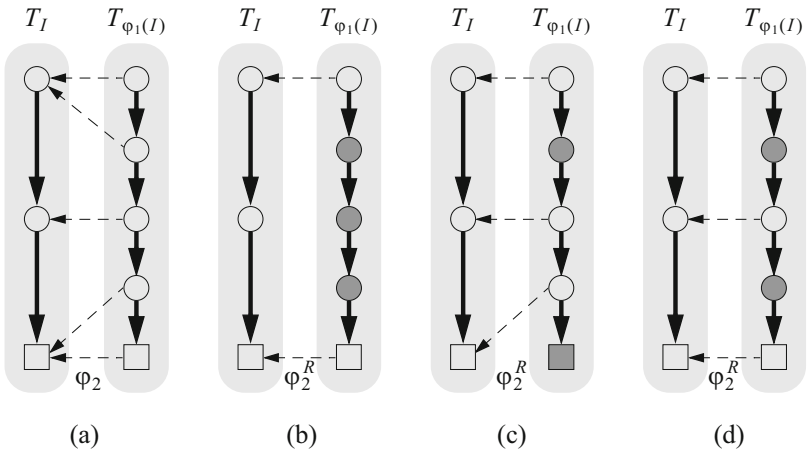


Fig. 11 (**a**) Transition graphs T_I of $I = (S, f, N)$ and $T_{\varphi_1(I)}$ of $\varphi_1(I) = (S', f', N')$ and algorithm φ_2 for some local search problem instance I and PLS-reduction (φ_1, φ_2). A dashed arc originates from an $s' \in S'$ and points to $s = \varphi_2(I, s')$. (**b**)–(**c**) Invalid choices of R, where R is given by S' minus the shaded solutions in transition graph $T_{\varphi_1(I)}$. φ_2^R denotes algorithm φ_2 for I with the solution set S' of $\varphi_1(I)$ restricted to R. (**d**) Valid choice of R

exist that satisfies the following three conditions. In the first place, φ_2 has to be surjective for I when restricting the solution set S' of $\varphi_1(I)$ to R. This means that for each solution $s \in S$, a solution $s' \in R$ must exist that is mapped to s by algorithm φ_2. Furthermore, solution s' must be computable in polynomial time. Note that for (φ_1, φ_2) to be a standard PLS-reduction it is not necessary that φ_2 is surjective for a given problem instance, as follows from Fig. 6. The second condition on R is that it contains all local optima of $\varphi_1(I)$. Figure 11b, c give examples of R that do not satisfy the first and second condition, respectively, and Fig. 11d gives an example of R that satisfies both conditions. The third and last condition that has to be satisfied is that if in $T_{\varphi_1(I)}$ a solution $s'_2 \in R$ can be reached from a solution $s'_1 \in R$ without visiting any other solution from R, then the distance between the corresponding solutions in S is not too large, i.e., it is polynomially bounded.

Definition 19. Local search problem Π_{LS} is *tightly PLS-reducible* to local search problem Π'_{LS}, denoted by $\Pi_{\mathrm{LS}} \propto_{\mathrm{tight}} \Pi'_{\mathrm{LS}}$, if a PLS-reduction (φ_1, φ_2) exists, such that for a polynomial p the following holds. For any problem instance $I = (S, f, N)$ of Π_{LS} with image instance $\varphi_1(I) = (S', f', N')$ of Π'_{LS}, we can choose a subset $R \subseteq S'$ that satisfies the following properties.

- For any $s \in S$ we can construct in polynomial time a solution $s' \in R$ with $\varphi_2(I, s') = s$.
- R contains all local optima in S'.

- If transition graph $T_{\varphi_1(I)}$ of $\varphi_1(I)$ contains a direct path from $s'_1 \in R$ to $s'_2 \in R$, then the distance from $s_1 = \varphi_2\left(I, s'_1\right)$ to $s_2 = \varphi_2\left(I, s'_2\right)$ in transition graph T_I of I is bounded from above by $p(|I|)$, where a path in $T_{\varphi_1(I)}$ is said to be direct if, except for the first and last solution, it only contains solutions outside R.

The pair (φ_1, φ_2) is called a *tight PLS-reduction*. □

Definition 20. A local search problem in PLS is *tightly PLS-complete* if each problem $\Pi_{\text{LS}} \in \text{PLS}$ is tightly PLS-reducible to it. □

This definition of a tight PLS-reduction preserves the property that a transition graph induced by a problem can be exponential, as shown in the following. By the last condition in Definition 19, we have that if the distance between any two solutions $s'_1 \in R$ and $s'_2 \in R$ in $T_{\varphi_1(I)}$ is $d_{s'_1,s'_2}$, then $d_{s'_1,s'_2} \cdot p\,(|I|)$ gives an upper bound on the distance between the corresponding solutions $s_1 = \varphi_2\left(I, s'_1\right)$ and $s_2 = \varphi_2\left(I, s'_2\right)$ in T_I. As R contains all local optima, this implies that if $s' \in R$ is within k steps of a local optimum $\hat{s}'$ in $T_{\varphi_1(I)}$, then solution $s = \varphi_2(I, s')$ is within $k \cdot p(|I|)$ steps of $\hat{s} = \varphi_2\left(I, \hat{s}'\right)$, which by the definition of a PLS-reduction is a local optimum in S. Hence, by the second condition in Definition 19, the potential of T_I is at most $p(|I|)$ larger than the potential of $T_{\varphi_1(I)}$. Because the input size of $\varphi_1(I)$ is bounded from above by a polynomial in the input size of I, this implies that if T_I has an exponentially large potential, then $T_{\varphi_1(I)}$ must have one as well. We thus arrive at the following result.

Lemma 5. *If (φ_1, φ_2) defines a tight PLS-reduction from local search problem Π_{LS} to local search problem Π'_{LS} and if the potential of Π_{LS} cannot be bounded by a polynomial, then neither can the potential of Π'_{LS}.* □

By Lemma 5, it now suffices to indicate one local search problem in PLS for which the transition graph can have an exponentially large potential to conclude that this result holds for any tightly PLS-complete problem.

Lemma 6. *PLS contains a local search problem for which the potential of the transition graph cannot be bounded by a polynomial.*

Proof Consider the trivial combinatorial optimization problem that, given an integer n, asks for the smallest integer i between 1 and n. This means that we want to minimize the cost function $f(i) = i$ with $1 \leq i \leq n$. For this problem, we consider the solution space consisting of the integers $1, 2, \ldots, n$, and we assume a neighborhood function in which integer 1 has no neighbors and integer $i \geq 2$ has only one neighbor, namely, $i - 1$. This yields a local search problem that is obviously in PLS. Furthermore, the transition graph is given by a single directed path that successively visits the solutions $n, n - 1, \ldots, 1$. The potential of this graph equals the distance from n to 1, which is $n - 1$. This is exponentially large as the input size is only $\log n$. □

Theorem 14. *For a tightly PLS -complete problem, iterative improvement requires an exponential number of iterations in the worst case to reach a local optimum, regardless of the pivoting rule used.* □

It can be verified from the definition that the PLS-reduction given in Theorem 12 is tight. Furthermore, as shown in [1, 11, 14], the problem POS NAE MAX-3SAT/Kernighan-Lin from which the reduction is made is tightly PLS-complete. This brings us at the following result.

Theorem 15. *The local search problem UGP/Kernighan-Lin is tightly PLS-complete.* □

Suppose that for a combinatorial optimization problem Π we have two neighborhood functions N and N', where N' is a generalization of N, i.e., $N(s) \subseteq N'(s)$ for any solution s. For TSP, the 2-change and 3-change neighborhood functions are examples of N and N', respectively. Then the local search problem Π_{LS} specified by Π and N is PLS-complete if this is the case for the local search problem Π'_{LS} specified by Π and N'. More generally, this means that to prove PLS-completeness results for one combinatorial optimization problem and multiple neighborhood functions it suffices to prove that the weakest neighborhood function induces a PLS-complete problem. However, if we want to know whether the induced local search problems are *tightly* PLS-complete, we can no longer restrict ourselves to the weakest neighborhood function. It may be the case that Π_{LS} is tightly PLS-complete, while this is not the case for Π'_{LS}. The reason for this is that the generalization can add arcs to the transition graph, which may decrease its potential to a polynomially bounded value.

We refer you to Michiels, Aarts, and Korst [1] and to Monien, Dumrauf, and Tscheuschner [21] for a set of local search problems that have been proven to be (tightly) PLS-complete.

Relevance of PLS Beyond Local Search

Although introduced to enhance the understanding of applying local search algorithms to combinatorial optimization problems, the class PLS has also found its way to other disciplines. First of all, the theory of PLS-completeness is used in game theory to prove the hardness of computing pure Nash equilibria. An overview of such results until 2010 is given in [21]. Furthermore, PLS is an interesting class of problem in between P and NP-hard. We motivate this using the argumentation given by the excellent paper of Papadimitriou [22]. Many search problems have proven either to be NP-complete or to belong to P. The complexity of NP-hard problems typically results from the difficulty of deriving whether a solution exists. As a result of this, most of the problems that have not been categorized as being either NP-complete or in P belong to the class of total search problems. For problems in this class, a solution is known to exist. The most well-known example of this

is factoring. It is known that any integer number can be factored into primes, but finding the factorization is considered to be difficult. This problem has not proven to be NP-complete, but as for NP-complete problems, many have tried to solve it in polynomial time but without success. Therefore, it is commonly assumed that this problem is also hard to solve. The security of the widely used cryptographic cipher RSA even relies on this assumption.

Different classes of total search problems are used to capture the possible different reasons why a total search problem can be hard to solve. PLS is such a class. Others are Polynomial Parity Argument (PPA), Polynomial Pigeonhole Principle (PPP), and Polynomial Parity Argument for Directed graphs (PPAD). Interesting papers in this area are [22–24].

Conclusion

Local search owes its popularity mainly to its good average-case performance. Its worst-case performance is typically less good compared to alternative solution approaches, like constructive algorithms. Nevertheless, studying the worst-case performance is still valuable since it both provides performance guarantees and enhances our understanding of the algorithm. In this chapter we discussed several worst-case local search results that have been scattered over the literature. We presented results on the worst-case performance ratio of local search implementations and on the time complexity of reaching a locally optimal solution. The latter results also find their applications outside the scope of local search for combinatorial optimization problems, viz., game theory and general computational complexity theory.

Competing Interest Declaration The author(s) has no competing interests to declare that are relevant to the content of this manuscript.

References

1. Michiels W, Aarts E, Korst J (2007) Theoretical aspects of local search. Springer, Berlin
2. Papadimitriou C, Steiglitz K (1982) Combinatorial optimization. Prentice-Hall, Englewood Cliffs
3. Papadimitriou C, Steiglitz K (1977) On the complexity of local search for the traveling salesman problem. SIAM J Comput 6:76–83
4. Sahni S, Gonzalez T (1976) P-complete approximation problems. J Assoc Comput Mach 23:555–565
5. Christofides N (1976) Worst-case analysis of a new heuristic for the traveling salesman problem. Technical report, 388. Graduate School of Industrial Administration, Carnegie-Mellon University, Pittsburgh
6. Chandra B, Karloff H, Tovey C (1999) New results on the old *k*-opt algorithm for the traveling salesman problem. SIAM J Comput 28:1998–2029
7. Levin A, Yovel U (2013) Nonoblivious 2-Opt heuristics for the traveling salesman problem. Networks 62(3):201–219

8. Angel E, Christopoulos P, Zissimopoulos V (2014) Local search: complexity and approximation. In: Paschos VT (ed) Paradigms of combinatorial optimization: problems and new approaches, vol 2. Wiley, Hoboken, pp 435–471
9. Hansen P, Jaumard B (1990) Algorithms for the maximum satisfiability problem. Computing 44:279–303
10. Khanna S, Motwani R, Sudan M, Vazirani U (1998) On syntactic versus computational views of approximability. SIAM J Comput 28:164–191
11. Yannakakis M (1997) Computational complexity. In: Aarts E, Lenstra J (eds) Local search in combinatorial optimization. Wiley, New York, pp 19–55
12. Schulz A, Weismantel R, Ziegler G (1995) 0/1-integer programming: optimization and augmentation are equivalent. In: Proceedings of the 3rd annual European symposium on algorithms, Corfu, pp 473–483
13. Orlin J, Punnen A, Schulz A (2004) Approximate local search in combinatorial optimization. SIAM J Comput 33:1201–1214
14. Johnson D, Papadimitriou C, Yannakakis M (1988) How easy is local search? J Comput Syst Sci 37:79–100
15. Marx D (2008) Searching the k-change neighborhood for TSP is $w[1]$-hard. Oper Res Lett 36:31–36
16. Guo J, Hartung S, Niedermeier R, Suchý O (2013) The parameterized complexity of local search for TSP, more refined. Algorithmica 67:89–110
17. Fischer S (1995) A note on the complexity of local search problems. Inf Process Lett 53:69–75
18. Luecker G (1976) unpublished manuscript. Department of Computer Science, Princeton University, Princeton
19. Englert M, Röglin H, Vöcking B (2007) Worst case and probabilistic analysis of the 2-opt algorithm for the TSP. In: Proceedings of the 18th ACM-SIAM symposium on discrete algorithm, New Orleans, pp 1295–1304
20. Johnson D, McGeoch L (1997) The traveling salesman problem: a case study. In: Aarts E, Lenstra J (eds) Local search in combinatorial optimization. Wiley, New York, pp 215–310
21. Monien B, Dumrauf D, Tscheuschner T (2010) Local search: simple, successful, but sometimes sluggish. In: Automata, languages and programming, pp 1–17
22. Papadimitriou C (2014) Algorithms, complexity, and the sciences. Proc Natl Acad Sci 111:15881–15887
23. Fearnley J, Savani R (2015) The complexity of the simplex method. In: Proceedings of the forty-seventh annual ACM symposium on theory of computing, pp. 201–208
24. Gordon S (2017) The complexity of continuous local search. Master thesis, University of Illinois

Variable Neighborhood Descent

18

Abraham Duarte, Nenad Mladenović, Jesús Sánchez-Oro, and Raca Todosijević

Contents

Introduction . . . 512
Neighborhoods . . . 514
  Neighborhoods for Continuous Optimization Problems . . . 515
  Neighborhoods for Binary Problems . . . 516
  Neighborhoods for Integer Problems . . . 518
  Neighborhoods for Permutation Problems . . . 520
Local Search Methods . . . 523
VND Variants . . . 526
  Sequential Variable Neighborhood Descent . . . 526
  Nested Variable Neighborhood Descent . . . 530
  Mixed Variable Neighborhood Descent . . . 533
Conclusions . . . 536
Cross-References . . . 536
References . . . 537

A. Duarte (✉) · J. Sánchez-Oro
Department of Computer Science and Statistics, Universidad Rey Juan Carlos, Mostoles, Spain
e-mail: abraham.duarte@urjc.es; jesus.sanchezoro@urjc.es

N. Mladenović
GERAD and Ecole des Hautes Etudes Commerciales, Montréal, QC, Canada

LAMIH, Université de Valenciennes, Valenciennes, France

Mathematical Institute, SANU, Belgrade, Serbia
e-mail: Nenad.Mladenovic@univ-valenciennes.fr

R. Todosijević
Univ. Polytechnique Hauts-de-France, LAMIH, CNRS, Valenciennes, France

INSA Hauts-de-France, Valenciennes, France

© Springer Nature Switzerland AG 2025

R. Martí et al. (eds.), *Handbook of Heuristics*,
https://doi.org/10.1007/978-3-032-00385-0_9

Abstract

Variable neighborhood descent (VND) is a local search-based heuristic which performs a deterministic exploration of a fixed set of neighborhood structures. Its success is based on the simple fact that different neighborhood structures usually have different local minimum. Thus, the local optima trap problem may be eventually solved by deterministic change of neighborhoods. VND can be used as a local search routine, and, therefore, it could be used within other metaheuristics. In this chapter we discuss typical problems that arise when designing VND: what neighborhood structures could be used, what would be their order, what rule of their change during the search would be used, etc. Comparative analysis of usual sequential VND variants is performed considering the traveling salesman problem.

Keywords

VND · Local search · Neighborhood · Sequential VND · Nested VND · Mixed VND

Introduction

Optimization is a key discipline in fields such as Computer Science, Artificial Intelligence, and Operations Research. Outside these scientific communities, the meaning of optimization becomes quite vague going to mean simply "do it as better as you can." In the context of this chapter, the concept of optimization is conceived as the process of trying to find the best possible solution to an optimization problem, usually in a limited time horizon.

In the simplest case, an optimization problem may be defined by a 2-tuple (X, f), where X represents the set of feasible solutions and f is an objective function that assigns a real number to each solution $x \in X$, which represents its quality or fitness. Then, the main objective of an optimization problem is to find a solution $x^\star \in X$ with the best objective function value among all solutions in the search space. Therefore, in a minimization problem, $x^\star \in X$ is a minimum point if $f(x^\star) \leq f(x)$, $\forall x \in X$. Notice that minimization of f is equivalent to maximization of $-f$.

In optimization problems there is usually either finite but a huge number or infinity number of solutions and a clear criterion for the comparison among them. Some well-known examples of optimization problems are the traveling salesman problem (TSP), the vehicle routing problem (VRP), the quadratic assignment problems (QAP), and scheduling problems, among others [3, 37]. The difficulty of solving these problems has been studied since the late 1970s [12]. These studies concluded that there is a subset of problems where it is possible to design an algorithm, which presents a polynomial computational complexity, i.e., the execution time of these algorithms polynomially grows with the problem size. Such problems belong to the class $\mathcal{P}$ and they are considered "easy to solve."

Examples of these problems are the shortest path problem (Dijkstra algorithm), the minimum spanning tree (Prim or Kruskal algorithms), and flows in networks (Ford-Fulkerson algorithm). However, computing optimal solutions is intractable for many optimization problems of industrial and scientific importance (i.e., there is no known algorithm with polynomial complexity to solve it optimally). This type of problems belongs to a class known as $\mathcal{NP}$ and they are considered "hard to solve."

In practice, we are usually satisfied with "good" solutions, which are obtained by heuristic algorithms. In particular, metaheuristics (MHs) represent a family of approximate [42] optimization techniques that gained a lot of popularity in the past two decades, becoming the most promising and successful techniques for solving hard problems. Unlike exact optimization algorithms, metaheuristics do not guarantee the optimality of the obtained solutions. Additionally, metaheuristics do not define how close the obtained solutions are from the optimal ones, in contrast with approximation algorithms. MHs provide "acceptable" solutions in a reasonable computing time for solving hard and complex problems in science and engineering.

The term metaheuristic was coined in 1986 [13] as a way of defining *a master process that guides and modifies other subordinate heuristics to explore solutions beyond simple local optimality*. MHs are among the most prominent and successful techniques to solve a large amount of complex and computationally hard combinatorial and numerical optimization problems arising in human activities, such as economics, industry, and engineering. MHs can be seen as general algorithmic frameworks that require relatively few modifications to be adapted to tackle a specific problem. They constitute a very diverse family of optimization algorithms including methods such as simulated annealing (SA), tabu search (TS), genetic algorithms (GA), ant colony optimization (ACO), and variable neighborhood search (VNS).

Metaheuristics are high-level strategies for exploring the search space using different methods. The search strategies are highly dependent on the philosophy of the metaheuristic itself. In particular, *trajectory-based metaheuristics* can be seen as intelligent extensions of traditional local search methods. The goal of this kind of MHs is to escape from a local optimum in order to proceed in the exploration of the search space and move on to find other hopefully better local optimum. Examples of these MHs are tabu search [13], simulated annealing [23], and variable neighborhood search [15], among others. *Population-based metaheuristics* deal with a set of solutions instead of dealing with only one solution. These techniques proved a natural and intrinsic way for the exploration of the search space. The final performance of these methods strongly depends on how the population is managed. Examples of population-based metaheuristics are genetic algorithms [19], scatter search [14], or memetic algorithms [33], among others. Some authors consider a third kind of metaheuristics called *constructive-based metaheuristics*, where the main effort is put in the intelligent construction of the solution. In other words, instead of starting the search from a random solution, these methods try to construct a high-quality initial solution. Examples of constructive-based metaheuristics are GRASP [10], ant colony optimization [5], and iterated greedy [41], among others.

The number of new proposed metaheuristics has amazingly increased in the last 25 years. Nowadays, the portfolio of MHs contains more than 50 variants, only considering the most stabilized ones (MHs successfully applied to a relatively large set of optimization problems). However, at the end, when designing a metaheuristic for an optimization problem, we face with two contradictory criteria: intensification (exploitation) and diversification (exploration). In fact, the performance of a metaheuristic basically relies on how it balances both criteria. The intensification of an algorithm describes its ability to thoroughly explore the promising regions in the hope to find better solutions. On the other hand, diversification describes the ability of the metaheuristic to explore non-visited regions of the search space in order to assure the evenly exploration of the search space and to avoid the confinement to the procedure to a reduced number of regions. Therefore, when tackling an optimization problem, it is necessary to search for the equilibrium between both criteria.

Variable neighborhood search (VNS) is a metaheuristic which was proposed in [32] as a general framework to solve hard optimization problems. This methodology is based on performing systematic changes of neighborhoods during the search space exploration. VNS has evolved in recent years, resulting in a large variety of strategies. Some of the most relevant variants are reduced VNS (RVNS), variable neighborhood descent (VND), basic VNS (BVNS), skewed VNS (SVNS), general VNS (GVNS), variable formulation search (VFS), and variable neighborhood decomposition search (VNDS), among others (see [18] for a recent survey on VNS). We refer the reader to [7, 8, 34, 35, 39] to some recent and successful applications of VNS to hard optimization problems.

This chapter is focused on variable neighborhood descent (VND), the deterministic variant of VNS. VND deserves separate attention since it is usually used in context of other metaheuristics as a local search routine. Once the set of neighborhood structures is selected, to be used in a deterministic manner, the VND is designed in three possible ways: (i) sequential VND, where neighborhoods are placed in the list with a given order and always explored in that order; (ii) nested or composite VND, where neighborhood operators are composed, i.e., $N_1(N_2(N_3(\ldots(x))))$ (neighborhood one of neighborhood two of neighborhood three, etc. of x); and (iii) mixed nested VND, where the two previous strategies are combined.

This chapter starts with some possible classification of neighborhood structures that are usually used in solving continuous and discrete optimization problems. Then, the pseudo-codes of sequential, composite, and mixed nested variants of VND are presented. Finally, variants of sequential VND are compared on the well-known traveling salesman problem.

Neighborhoods

The representation (encoding) of a solution in an optimization problem plays a relevant topic in the design of an algorithm. In fact, as it is well documented, this representation determines the difficulty of solving a problem [12] and also the complexity of some routines within algorithm. In addition, this encoding strongly

influences the way in which the neighborhoods of a given solution are defined. Therefore, it is not possible to separate the neighborhood of a solution from its corresponding representation in the computer memory.

Let us assume, without loss of generality, that each solution $x \in X$ is represented by a vector $x = (x_1, \ldots, x_n)$, being n the size of the problem. Depending on the values of each x_i, we can distinguish among different types of problems:

- Continuous $\rightarrow x_i \in \mathbb{R}$
- Binary $\rightarrow x_i \in \{0, 1\}$
- Integer $\rightarrow x_i \in \mathbb{N}$
- Permutations $\rightarrow x_i \in \mathbb{N}$, $1 \leq x_i \leq n$ and $x_i \neq x_j$ if $i \neq j$

We could also identify a fifth class of problems which encompasses mixed variables. In this section, we describe the most common neighborhoods defined in the related literature as well as some basic properties.

Neighborhoods for Continuous Optimization Problems

The continuous constrained nonlinear global optimization problem (GOP) in general form is given as follows:

$$(GOP) \quad \begin{cases} \min & f(x) \\ \text{s.t.} & g_i(x) \leq 0 \quad \forall i \in \{1, 2, \ldots, m\} \\ & h_i(x) = 0 \quad \forall i \in \{1, 2, \ldots, r\} \\ & a_j \leq x_j \leq b_j \ \forall j \in \{1, 2, \ldots, n\} \end{cases}$$

where $x \in \mathbb{R}^n$, $f : \mathbb{R}^n \rightarrow \mathbb{R}$, $g_i : \mathbb{R}^n \rightarrow \mathbb{R}$, $i = 1, 2, \ldots, m$, and $h_i : \mathbb{R}^n \rightarrow \mathbb{R}$, $i = 1, 2, \ldots, r$, are possibly nonlinear continuous functions, and $a, b \in \mathbb{R}^n$ are the variable bounds.

GOP naturally arises in many applications, e.g., in advanced engineering design, data analysis, financial planning, risk management, scientific modeling, etc. Most cases of practical interest are characterized by multiple local optima, and, therefore, in order to find the globally optimal solution, a global scope search effort is needed.

If the feasible set X is convex and objective function f is convex, then GOP is relatively easy to solve, i.e., the Karush-Kuhn-Tucker conditions may be applied. However, if X is not a convex set or f is not a convex function, we could have many local minima, and thus, the problem may not be solved by using classical techniques.

For solving GOP, neighborhood structures $\mathcal{N}_k(x)$ are usually induced from the ℓ_p metric:

$$\rho(x, y) = \left(\sum_{i=1}^{n} |x_i - y_i|^p \right)^{1/p} \quad (1 \leq p < \infty) \tag{1}$$

or

$$\rho(x, y) = \max_{1 \leq i \leq n} |x_i - y_i| \quad (p \to \infty). \tag{2}$$

The neighborhood $\mathcal{N}_k(x)$ denotes the set of solutions in the k–th neighborhood of x, and using the metric ρ, it is defined as

$$\mathcal{N}_k(x) = \{y \in X \mid \rho(x, y) \leq \rho_k\}, \tag{3}$$

or

$$\mathcal{N}_k(x) = \{y \in X \mid \rho_{k-1} \leq \rho(x, y) \leq \rho_k\}, \tag{4}$$

where ρ_k, known as the radius of $\mathcal{N}_k(x)$, is monotonically increasing with k.

Note that in some papers neighborhood structures in $\mathbb{R}^n$ are not induced from the ℓ_p metric (see, e.g., [1]).

Neighborhoods for Binary Problems

There are a large family of optimization problems, whose solution is usually represented as a binary array, where the presence or absence of an element is described by means of a binary variable. The Knapsack Problem [28], the Max-Cut Problem [29], and the Maximum Diversity Problem [6] are examples of these problems. For solving binary problems, neighborhood structures $\mathcal{N}_k(x)$ are usually induced from the Hamming metric:

$$d_H(x, y) = \sum_{i=1}^{n} |x_i - y_i| \tag{5}$$

More precisely, the k–th neighborhood of a solution x, i.e., $\mathcal{N}_k(x)$, relatively to Hamming metric, is defined as

$$\mathcal{N}_k^{Bin}(x) = \{y \in X \mid d_H(x, y) = k\}, \tag{6}$$

We will use the Max-Cut Problem to illustrate the most relevant characteristics of binary problems. Consider a graph $G = (V, E)$ with vertex set V and edge set E. Let w_{ij} be the weight associated with edge $(i, j) \in E$. A $cut(W, W')$ is a partition of V into two sets W, $W' = V \setminus W$, and its value is given by the expression:

$$cut(W, W') = \sum_{i \in W \wedge j \in W'} w_{ij}$$

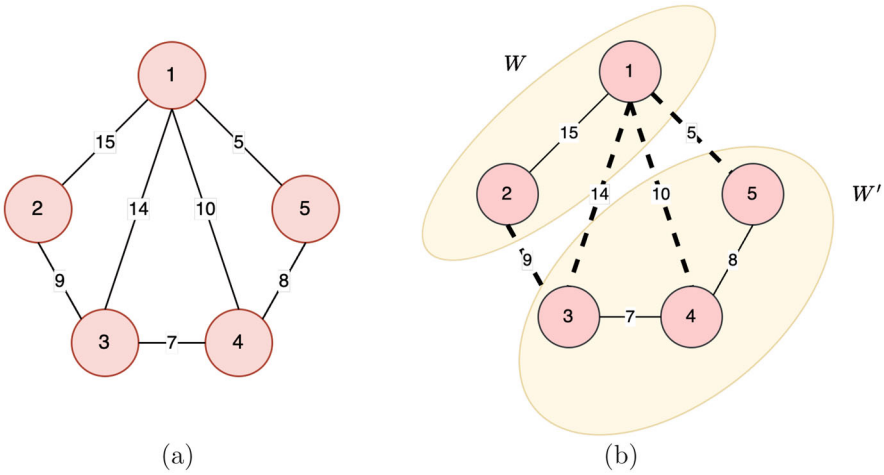


Fig. 1 Example of graph with five vertices and seven edges and a possible solution for the MCP. (**a**) Example of a graph. (**b**) Solution $x = \{1, 1, 0, 0, 0\}$

The Max-Cut Problem (MCP) consists of finding a cut in G with maximum value. Karp [22] showed that MCP is an $\mathcal{NP}$-hard problem. Figure 1a shows an example graph with five vertices and seven edges where the number close to each edge represents the corresponding weight. Figure 1b shows a possible solution, $x = (W, W')$, where $W = \{1, 2\}$ and $W' = \{3, 4, 5\}$. The value of this solution is $cut(W, W') = 9 + 14 + 10 + 5 = 38$, computed as the sum of the edges whose endpoints are in different sets (dashed edges). This solution can be represented as a binary vector $x = \{1, 1, 0, 0, 0\}$ where $x_i = 1$ indicates that the corresponding vertex is in W while $x_i = 0$ means that the vertex is in W'.

For a given solution x of a MCP, we define $\mathcal{N}^{Bin}_{drop}(x)$ neighborhood using the $drop(x, i)$ move operator. This operator is responsible of changing the value of a variable x_i from 1 to 0, producing a new solution x'. The associated neighborhood, $\mathcal{N}^{Bin}_{drop}(x)$, has size n (in the worst case) and is a subset of $\mathcal{N}^{Bin}_{1}(x)$. Formally, $\mathcal{N}^{Bin}_{drop}(x)$ is defined as

$$\mathcal{N}^{Bin}_{drop}(x) = \{x' \leftarrow drop(x, i) : x_i = 1 \wedge 1 \leq i \leq n\}$$

Figure 2a shows an example of this type of move. In particular, $drop(x, 2)$ considers the solution x (depicted in Fig. 1b), removes the vertex 2 from W, and includes it in W', producing a new solution $x^{drop} = \{1, 0, 0, 0, 0\}$. The Hamming distance between x and x' is 1 since there is only one vertex located in a different group.

Symmetrically, we define the $add(x, i)$ as the move operator responsible of changing the value of a variable x_i from 0 to 1, producing a new solution x^{add}. Considering the MCP, this move operator removes a vertex from W' and includes it

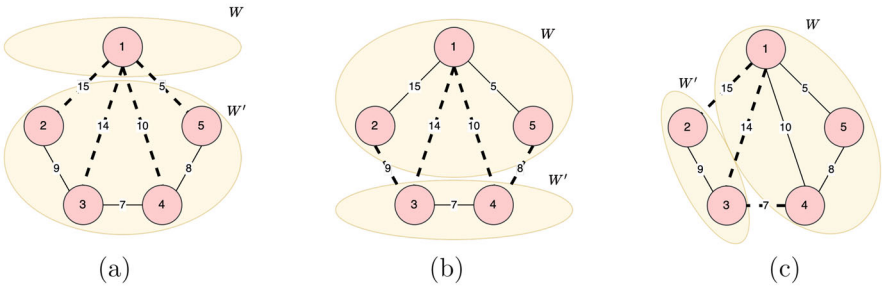


Fig. 2 Solutions generated when performing the different moves to the one depicted in Fig. 1b. (**a**) $x' \leftarrow drop(x, 2)$. (**b**) $x'' \leftarrow add(x, 5)$. (**c**) $x''' \leftarrow swap(x, 1, 3)$

in W. This move is represented as $x'' \leftarrow add(x, i)$ and the neighborhood, $\mathcal{N}^{Bin}_{add}(x)$, with size n in the worst case, is

$$\mathcal{N}^{Bin}_{add}(x) = \{x'' \leftarrow add(x, i) : x_i = 0 \wedge 1 \leq i \leq n\}$$

Figure 2b shows the move $add(x, 1)$. It considers again the solution x (depicted in Fig. 1b), removes the vertex 1 from W' including it in W. The Hamming distance between the new produced solution, $x'' = \{1, 1, 0, 0, 1\}$, and x is again 1.

Finally, we define $\mathcal{N}^{Bin}_{swap}(x)$ neighborhood as a subset of $\mathcal{N}^{Bin}_{2}(x)$ defined by *swap* move. *Swap* move is defined as an operation that changes the value of one variable x_i from 1 to 0 and simultaneously a different variable x_j change the value from 0 to 1. This move interchanges a vertex from W to W' and, simultaneously a different vertex from W' to W. This move is represented as $x''' \leftarrow swap(x, i, j)$ and the neighborhood, $\mathcal{N}^{Bin}_{swap}(x)$ (with size n^2 in the worst case), is

$$\mathcal{N}^{Bin}_{swap}(x) = \{x''' \leftarrow swap(x, i, j) : x_i \neq x_j \wedge 1 \leq i, j \leq n\}$$

Figure 2c shows the move $swap(x, 1, 3)$. It considers again the solution x (depicted in Fig. 1b) and interchanges the vertices 1 and 3 between W and W', respectively. The Hamming distance between the new produced solution, $x''' = \{0, 1, 1, 0, 0\}$, and x is in this case 2.

Neighborhoods for Integer Problems

There are some optimization problems where the solution x is represented as a vector of n variables, $x = (x_1, x_2, \ldots, x_n)$, such that $l \leq x_j \leq u$ for all j, assuming that all variables are integer and bounded within the interval $[l, u]$. For example, the solutions to the Set Covering Problem [9], the Capacitated Task Allocation Problem [24], or the Maximally Diverse Grouping Problems [11] can be represented

as a vector of integer values. For solving these problems, neighborhood structures $\mathcal{N}_k^{Int}(x)$ are usually induced from the metric defined as

$$\rho_{int}(x, y) = \sum_{i=1}^{n} |x_i - y_i| \tag{7}$$

or

$$\rho_{int}(x, y) = \max_{1 \le i \le n} |x_i - y_i| \tag{8}$$

The neighborhood $\mathcal{N}_k^{Int}(x)$ denotes the set of solutions in the k–th neighborhood of x, and it is defined as

$$\mathcal{N}_k^{Int}(x) = \{y \in X \mid \rho_{int}(x, y) \le k\}, \tag{9}$$

or

$$\mathcal{N}_k^{Int}(x) = \{y \in X \mid k - 1 \le \rho_{int}(x, y) \le k\}, \tag{10}$$

We use the Maximally Diverse Grouping Problem to illustrate neighborhoods for integer problems. It consists of grouping a set of M elements into G mutually disjoint groups in such a way that the diversity among the elements in each group is maximized. The diversity among the elements in a group is calculated as the sum of the individual distances between each pair of elements, where the notion of distance depends on the specific application context. The objective of the problem is to maximize the overall diversity, that is, the sum of the diversity of all groups. Figure 3 shows an example of a graph with ten vertices where we must select three groups of elements. For the sake of clarity, we only represent the distances among selected elements. In addition, we use different colors to differentiate the vertices on each group. In particular, group 1 (red) is $G_1 = \{1, 4, 9\}$ whose distances among edges are represented with dashed lines; group 2 (yellow) is $G_2 = \{3, 5, 8\}$ represented with dotted lines; and group 3 (blue) is $G_3 = \{2, 6, 7, 10\}$ represented with solid lines. Therefore, this solution can be represented as a vector $x = \{1, 3, 2, 1, 2, 3, 3, 2, 1, 3\}$ (i.e., each vertex i is located in group x_i).

With this representation, we define the following moves.

Exchange of values (Swap move): given a solution x, the exchange of the values of the variables x_i and x_j is denoted by $swap(x, i, j)$ and generates a new solution x' such that $x'_k = x_k, \forall k \neq i, j$ and $x'_i = x_j, x'_j = x_i$. Note that resulting solution x' belongs to the neighborhood:

$$\mathcal{N}_0^{Int}(x) = \{y \in X \mid \max_{1 \le i \le n} |x_i - y_i| = 0\}.$$

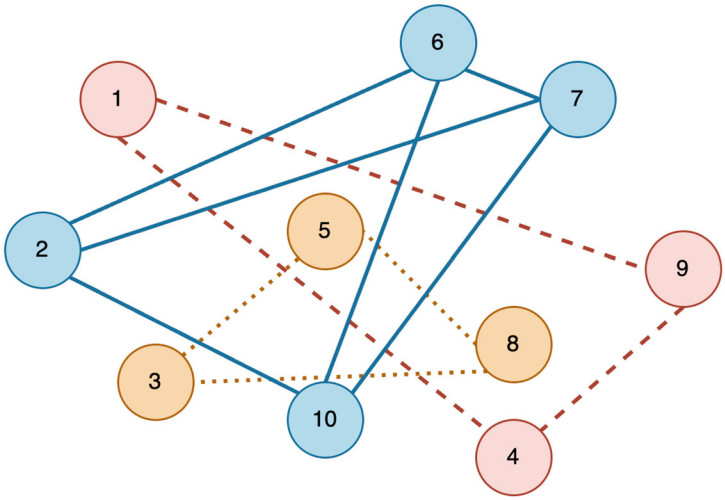


Fig. 3 Example of the Maximally Diverse Grouping Problem

Replacement of a single value (Replacement move): given a solution x, the replacement move denoted by $replace(x, j, i)$ creates a solution x' for which $x'_k = x_k, \forall k \neq j$ and $x'_j = i$. The move is such that $x_j \neq i$. In this case the resulting solution x' belongs to the neighborhood:

$$\mathcal{N}_m^{Int}(x) = \{y \in X \mid \sum_{1 \leq i \leq n} |x_i - y_i| \leq m\},$$

where $m = |x_j - i|$.

Given the solution $x = \{1, 3, 2, 1, 2, 3, 3, 2, 1, 3\}$, the move $swap(x, 6, 5)$ interchanges the corresponding group of vertices 6 and 5, producing the solution $x' = \{1, 3, 2, 1, 3, 2, 3, 2, 1, 3\}$ (see Fig. 4a). Similarly, the move $replace(x, 2, 1)$ includes the vertex 2 in group 1 (removing it from its original group). The final solution in this case is $x'' = \{1, 1, 2, 1, 2, 3, 3, 2, 1, 3\}$ (see Fig. 4b).

Neighborhoods for Permutation Problems

In permutation-based representations, a solution is typically expressed as a labeling (permutation), where each element receives a unique label from 1 to n, being n the size of the problem. The traveling salesman problem [38], the linear ordering problem [30], and the cutwidth minimization problem [36] are examples of this kind of problems. For solving these problems, neighborhood structures $\mathcal{N}_k^{Int}(x)$ may be induced using several metrics (see, e.g., [4] for possible metric). However, neighborhoods are usually induced from the Cayley distance:

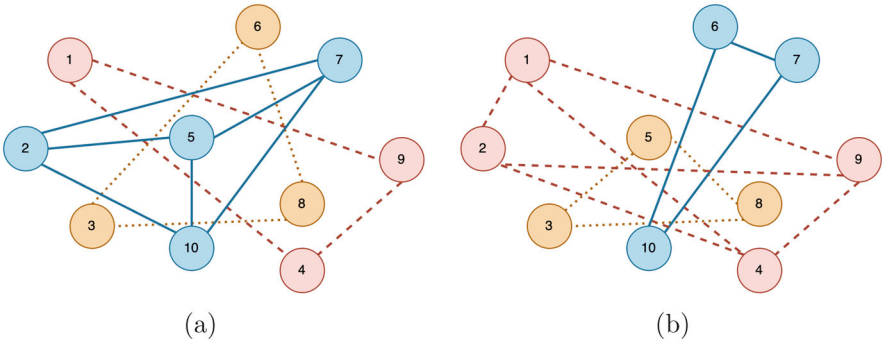


Fig. 4 Example of moves applied to a solution for the Maximally Diverse Grouping Problem. (**a**) $x' \leftarrow swap(x, 6, 5)$. (**b**) $x'' \leftarrow replace(x, 2, 1)$

$$\rho_{perm}(x, y) := \text{the minimum number of transpositions needed to obtain } y \text{ from } x \tag{11}$$

So, the neighborhood $\mathcal{N}_k^{Perm}(x)$ denotes the set of solutions in the k–th neighborhood of x and it is defined as

$$\mathcal{N}_k^{Perm}(x) = \{y \in X \mid \rho_{perm}(x, y) = k\}, \tag{12}$$

We use the cutwidth minimization problem (CMP) to illustrate some neighborhoods for permutation-based problem. It can be easily described in mathematical terms. Given a graph $G = (V, E)$ with $n = |V|$ and $m = |E|$, a labeling or linear arrangement x of G assigns the integers $\{1, 2, \ldots, n\}$ to the vertices in V, where each vertex receives a different label. The cutwidth of a vertex v with respect to x, $CW_x(v)$, is the number of edges $(u, w) \in E$ satisfying $x(u) \leq x(v) < x(w)$. In mathematical terms

$$CW_x(v) = |\{(u, w) \in E : x(u) \leq x(v) < x(w)\}|$$

The cutwidth of G with respect to x is defined as the maximum value of all $CW_x(v)$ for $v \in V$. More formally

$$CW_x(G) = \max_{v \in V} CW_x(v)$$

The optimum cutwidth of G is then defined as the minimum $CW_x(G)$ over all possible layouts of G. This optimization problem is $\mathcal{NP}$-hard even for graphs with a maximum degree of three [27]. Figure 5a shows an example of an undirected graph with six vertices and seven edges. Figure 5b shows a labeling, x, of the graph in Fig. 5a, setting the vertices in a line with the order of the labeling, as commonly represented in the CMP. We represent x with the ordering (6, 1, 3, 2, 4, 5) meaning that vertex 6 is located in the first position (label 1), vertex 1 is located in the second

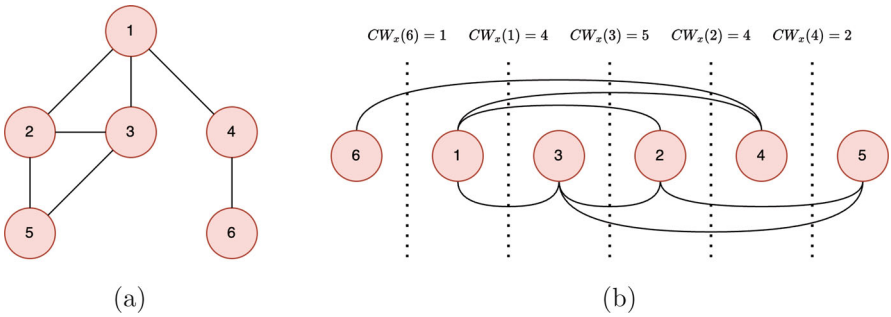


Fig. 5 Graph with six vertices and seven edges, and an example solution for the CWP. (**a**) Example graph. (**b**) Cutwidth of G for a labeling x

position (label 2), and so on. In Fig. 5b, the cutwidth of each vertex is represented as a dashed line with its corresponding value. For example, the cutwidth of vertex 6 is $CW_x(6) = 1$, because the edge $(6, 4)$ has and endpoint in 6 labeled with 1 and the other endpoint in a vertex labeled with a value larger than 1. In a similar way, we can compute the cutwidth of vertex 1, $CW_x(1) = 4$, by counting the appropriate number of edges: $(6, 4)$, $(1, 4)$, $(1, 2)$, and $(1, 3)$. Then, since the cutwidth of the graph G, $CW_x(G)$, is the maximum of the cutwidth of all vertices in V; in this particular example, we obtain $CW_x(G) = CW_x(3) = 5$.

The associated neighborhoods for this optimization problem are typically based on two different move operators. The first one is referred to as *exchange*. Given a solution $x = (v_1, \ldots, v_i, \ldots, v_j, \ldots, v_n)$, we define $exchange(x, i, j)$ as exchanging in x the vertex in position i (i.e., vertex v_i) with the vertex in position j (i.e., vertex v_j), producing a new solution $x' = (v_1, \ldots, v_{i-1}, v_j, v_{i+1}, \ldots, v_{j-1}, v_i, v_{j+1}, \ldots, v_n)$. So, *exchange* move represents a transposition. For the sake of simplicity, we denote $x' = exchange(x, i, j)$. The associated neighborhood $\mathcal{N}^{Perm}_{exchange}$, obtained applying all possible *exchange* moves, has size $n(n-1)/2$, and it is formally defined as

$$\mathcal{N}^{Perm}_{exchange}(x) = \{x' \leftarrow exchange(x, i, j) : i \neq j \wedge 1 \leq i, j \leq n\}$$

This neighborhood is actually $\mathcal{N}^{Perm}_1$ neighborhood.

The second move operator for CMP is known as *insert*. Specifically, given a solution x, we define $insert(x, j, v_i)$ as the move consisting of deleting v_i from its current position i and inserting it in position j. This operation results in the new solution x', as follows:

- If $i \geq j$, then $x = (\ldots, v_{j-1}, v_j, v_{j+1}, \ldots, v_{i-1}, v_i, v_{i+1}, \ldots)$ and the vertex v_i is inserted just before the vertex v_j, obtaining $x' = (\ldots, v_{j-1}, v_i, v_j, v_{j+1}, \ldots, v_{i-1}, v_{i+1}, \ldots)$.

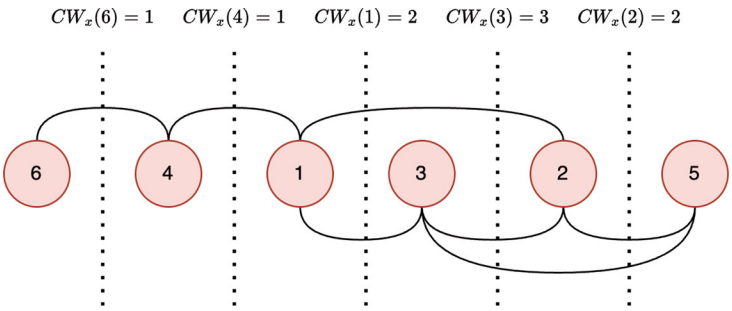


Fig. 6 Resulting solution after $insert(x, 2, 4)$ over solution depicted in Fig. 5b

- If $i < j$, then $x = (\ldots, v_{i-1}, v_i, v_{i+1}, \ldots, v_{j-1}, v_j, v_{j+1}, \ldots)$ and the vertex v_i is inserted just after the vertex v_j, obtaining $x' = (\ldots, v_{i-1}, v_{i+1}, \ldots, v_{j-1}, v_j, v_i, v_{j+1}, \ldots)$.

Thus, a move $insert(x, j, v_i)$ generates a solution belonging to the neighborhood $\mathcal{N}^{Perm}_{|j-i|}(x)$. The associated neighborhood $\mathcal{N}^{Perm}_{insert}$, obtained applying all possible *insert* moves, has size $n(n-1)/2$, and it is formally defined as

$$\mathcal{N}^{Perm}_{insert}(x) = \{x' \leftarrow insert(x, j, v_i) : i \neq j \wedge 1 \leq i, j \leq n\}$$

The following example illustrates how the *insert* is implemented. Let $x = (6, 1, 3, 2, 4, 5)$ be a solution of the cutwidth problem. Suppose that we perform $insert(x, 2, 4)$, obtaining solution $x' = (6, 4, 1, 3, 2, 5)$. Figure 6 graphically shows the solutions before and after the move.

As it can be seen in the resulting solution, the node 4 is now located at position 2, scrolling to the right the nodes located between the new and the original position. In this case, the maximum cutwidth value is obtained at node 3, with $CW_x(3) = 3$, thus reducing the objective function value in 2 units.

Local Search Methods

Local search methods are likely the oldest and simplest heuristic methods. Starting from a given feasible solution, these procedures explore a determined neighborhood in each iteration, replacing the current solution if a neighbor improves the objective function. The search ends when all neighbor solutions are worse (i.e., larger objective function value in a minimization problem) meaning that a local optimum is found.

There exist two typical strategies to explore the corresponding neighborhood: **best improvement** and **first improvement**. In the former (also known as **steepest descent**), the associated neighborhood is completely explored by a fully deterministic procedure, performing the best associated move. Therefore, no matter how the

neighborhood is scanned, since all neighborhood solutions are visited. Algorithm 1 shows the typical pseudo-code of this local search method for a minimization problem. The algorithm starts by initializing the control variable *improve* (step 1). Then, the best improvement strategy performs iterations until it finds a local optimum with respect to neighborhood $N(x)$ (steps 2–9). Given a solution x, the best neighbor solution x' is determined in step 4. This instruction explores the complete set of $|N(x)|$ solutions belonging to neighborhood $N(x)$. In steps 5–8 it is decided whether to perform a new iteration (by updating x to x') or not (abandoning the search).

Algorithm 1: Best-Improvement(x)

```
1: improve ← true
2: while (improve) do
3:     improve ← false
4:     x' ← arg min_{x∈N(x)} f(x)
5:     if f(x') < f(x) then
6:         x ← x'
7:         improve ← true
8:     end if
9: end while
10: return x
```

The first improvement strategy tries to avoid the time complexity of exploring the whole neighborhood by performing the first improving move encountered during the exploration of the corresponding neighborhood. In this kind of exploration, the order in which the neighbors are inspected can have a significant influence of the efficiency of the search. Instead of using a fixed ordering for scanning the neighbors, random orders are usually suggested, since the first scanning strategy always drives to the same local optimum, while the second one favors diversity, eventually reaching different local optima.

The pseudo-code of this strategy is equivalent to the one presented in Algorithm 1. The only difference is in step 4 where, instead of selecting the best neighbor, it is selected the first neighbor which improves the incumbent solution (in terms of objective function value). Additionally, if we follow a predefined order (e.g., a lexicographic order), the first positions in that order (e.g., $1, 2, \ldots$, in the lexicographic order) are favored, producing a kind of bias in the search.

Figure 7 shows the performance of two iterations of both strategies starting from the same solution, where the numbers over the arrows indicate the order of exploration of the solutions. Specifically, considering the MCP described in section "Neighborhoods for Binary Problems," the initial solution is $x_1 = [1, 1, 0, 0, 0]$, representing $W = [1, 2]$ and $W' = [3, 4, 5]$, with an objective function value of 38. The neighborhood selected to be explored is $\mathcal{N}_{add}^{Bin}(x_1)$, where each neighbor is generated by adding a new vertex to W (removing it from W'). In the first iteration, the best improvement strategy (Fig. 7a) generates all possible neighbor solutions for x_1 (i.e., x_2, x_3, x_4). The exploring order, as stated before, is irrelevant, since it is going to explore the whole neighborhood. Then, the method selects

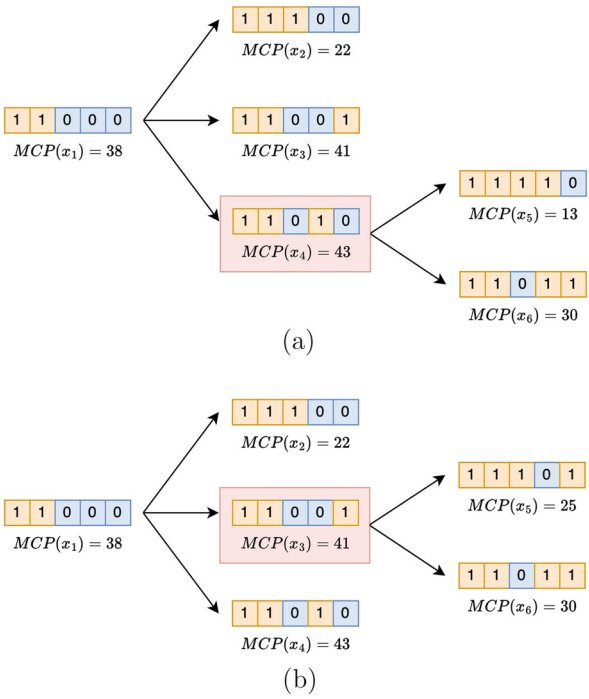


Fig. 7 Comparison of best and first improvement strategies when starting from the same solution for the MCP. (**a**) Best improvement. (**b**) First improvement

the best neighbor solution, which is x_4 in Fig. 7a, with a higher (better) objective function value of 43. Finally, in the next iteration, the method explores the entire x_4 neighborhood, stopping after the exploration, since there is no better neighbor solution.

Regarding the first improvement method (Fig. 7b), the exploration order (which is relevant in this case) is a lexicographical order, starting with solution x_2. As the objective function value of x_2 is lower (worse) than x_1, the method explores another neighbor, which is x_3. The objective function value for x_3, 41, is better than for x_1, 38, so in this case, the first improvement strategy stops the iteration, starting the next one from this new best solution x_3. Notice that although solution x_4 is better than x_3, it is not explored in this strategy, since the order selected has led the method to find a better solution before reaching x_4. Finally, in the next iteration, the method starts from x_4 and then explores, in this order, x_5 and x_6, stopping the search since there is no improvement in any neighbor.

In the context of large neighborhoods, there is a compromise between the number of iterations required to find a local optimum and the associated computing time. In general, iterations performed in the first improvement strategy are more efficient than those in the best improvement one, since the former only evaluates part of the neighborhood, while the latter explores it completely. On the other hand, the improvement obtained in the first improvement strategy is typically smaller than the one achieved by the best improvement strategy, requiring in general more iterations to obtain the local optimum. Additionally, the best improvement strategy

is usually more adequate to perform efficient catching and updating mechanisms, which allows the search to efficiently explore the neighborhood [20].

In [17], an empirical study on traveling salesman problem was conducted in order to compare the first and the best improvement strategy within 2-opt neighborhood structure. It appeared that the quality of the final solution depends on the quality of the initial solution: (i) if random initial solution is chosen, the better and faster is the first improvement strategy; (ii) the opposite holds if the greedy solution is taken as initial one.

VND Variants

The variable neighborhood search (VNS) is a metaheuristic proposed in [32] as a general framework to solve hard problems. It is based on a simple idea: systematical changes of neighborhood structures within the search procedure. Let $\mathcal{N} = \{\mathcal{N}_1, \ldots, \mathcal{N}_{k_{max}}\}$ be set of operators such that each operator $\mathcal{N}_k$, $1 \leq k \leq k_{max}$ maps a given solution x to a neighborhood structure $\mathcal{N}_k(x)$.

Note that the order of operators taken from the set $\mathcal{N}$ also defines the order of neighborhood structures of a given solution x examined. When solving an optimization problem by using different neighborhood structures, VNS methodology proposes to explore them in three different ways: (i) at random, (ii) deterministically, or (iii) mixed (combining both deterministic and random exploration).

Variable neighborhood descent (VND) is a variant of VNS that explores a predefined set of neighborhoods. In general, VND explores small neighborhoods until a local optimum is encountered. At that point the search process switches to a different (typically larger) neighborhood that might allow further progress. This approach is based on the fact that a local optimum is defined with respect to a neighborhood relation, such that if a candidate solution x is locally optimal in a neighborhood $\mathcal{N}_i(x)$, it is not necessarily a local optimum for another neighborhood $\mathcal{N}_j(x)$. Thus, VND explores the solution space using several neighborhood structures either in a (i) sequential, (ii) a nested (or composite), or (iii) mixed nested way [21,43].

Sequential Variable Neighborhood Descent

Most typical VND variants traverse the list of neighborhood structures in a sequential way. Let suppose that we have an ordered list of k_{max} neighborhoods, the current solution is x and new candidate solution x' is found in the neighborhood $k \leq k_{max}$. The main question that arises is by exploring which neighborhood structure we resume the search. The answer to this question is provided by the neighborhood change step. We may distinguish the neighborhood change steps that maintain a static list of neighborhoods and those with a dynamic list of neighborhoods. In the case of neighborhood change steps with the static order, the order of examining neighborhoods is predefined in advance, while in the case of a dynamic order, the order may evolve depending on the success/failure within some neighborhood structure.

All sequential VND procedures follow the steps given in Algorithm 2 and start from a given solution x. In each VND iteration, a local search procedure through a given neighborhood structure is applied (step 8), followed by a neighborhood change function (step 9). The neighborhood change function defines the neighborhood structure that will be examined in the next iteration. Each VND variant stops when there is no improvement with respect to any of the considered neighborhood structures. Thus, the solution obtained by any sequential VND is a local optimum with respect to all neighborhood structures.

Algorithm 2: SeqVND (x,$k_{\max}$,$\mathcal{N}$)

```
1: x ← x''
2: Stop ← False
3: while not Stop do
4:     x ← x''
5:     Stop ← True
6:     k ← 1
7:     while k ≤ k_max do
8:         x' ← arg min_{y∈N_k(x)} f(y)
9:         NeighborhoodChange(x, x', k)
10:        if f(x') < f(x'') then
11:            x'' ← x'
12:            Stop ← False
13:        end if
14:    end while
15: end while
16: return x''
```

VND with the static order of neighborhoods (Static VND) Within the variants with the static order of neighborhoods, the basic VND, the pipe, the cyclic VND, and the union VND emerge as the most representative search procedures. These variants differ in how they implement the neighborhood change procedures. Specifically, if an improvement has been detected in some neighborhood, how the search (after updating the incumbent solution) is continued:

- **Basic VND (B-VND)** – returns to the first neighborhood from the list.
- **Pipe VND (P-VND)** – continues the search in the same neighborhood.
- **Cyclic VND (C-VND)** – resumes the search in the next neighborhood from the list.
- **Union VND (U-VND)** (sometimes called multiple neighborhood search [44], where the single neighborhood is obtained as the union of all predefined neighborhoods) – continues the search in the same large neighborhood. U-VND is recently proposed in [2, 26, 44] and used within tabu search.

VND with dynamic order (Dynamic VND) Among VNDs with the dynamic order, the most typical ones are random VND and adaptive VND:

- **Random VND**. The random VND procedure as new neighborhood to be explored suggests one chosen at random from the list of available neighborhoods. This happens regardless we have the change of the incumbent solution or not.
- **Adaptive VND**. The simple adaptive VND changes neighborhoods according to "merits" of neighborhoods. At simplest, the merit needs to reflect the success/-capability of certain neighborhood to improve incumbent solution. Hence, the merit evolves during the search and accordingly the order of neighborhoods may change. One possibility is to have all the time neighborhoods that are ordered according to their merit and then favor that those with higher merit are explored at first. The merits may be maintained by using some adaptive mechanism or machine learning technique.

Empirical Study on Static VNDs

We show the performance of the aforementioned static VND variants, considering the example introduced in [31]. In particular, it shows an empirical study on different static VND variants used to solve traveling salesman problem. For testing purposes, 15,200 random test instances were generated in the way described in [17]. Within VND variants, three classical TSP neighborhood structures are considered: 2-opt (Fig. 8), Insertion-1 (Fig. 9), and Insertion-2 (Fig. 10).

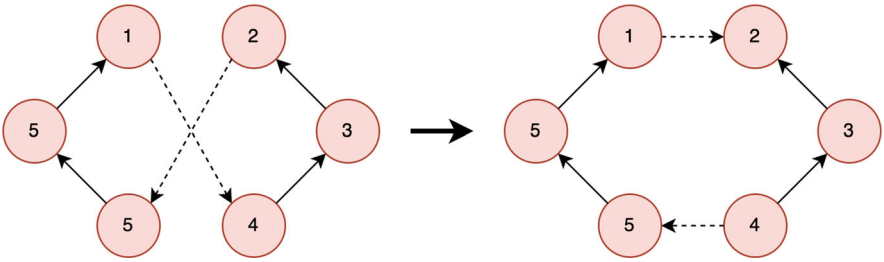


Fig. 8 Example of a 2-opt move involving vertices 1, 2, 4, and 5

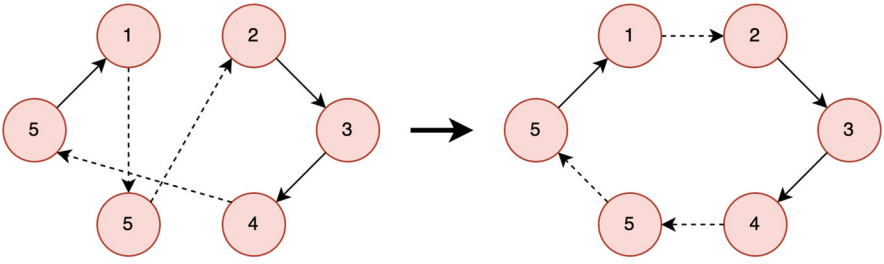


Fig. 9 Example of an Insertion-1 move where vertex 5 is inserted between vertices 1 and 2, and removed from its corresponding former position

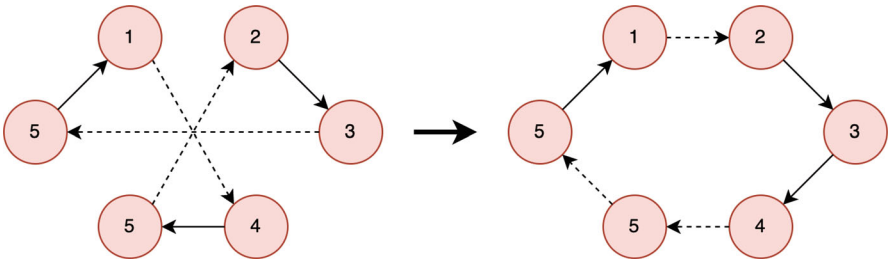


Fig. 10 Example of an Insertion-2 move where vertices 4 and 5 are inserted between vertices 1 and 2, removing them from its corresponding former position

Table 1 Comparison of VND variants

VND	Avg.	Time (s)
Basic VND	1198.24	0.16
Pipe VND	1198.52	0.12
Cyclic VND	1198.76	0.46
Union VND	1197.65	1.06

On each instance from this data set, each VND variant, except U-VND, is tested under 24 different settings. Each setting corresponds to choosing the following: (i) one out of two common ways for getting an initial solution, at *random* (solution generated as a random permutation of nodes) or *greedy*; (ii) one out of six possible neighborhood orders; and (iii) the *best* or the *first improvement search strategy*. This gives $2 \times 6 \times 2 = 24$ different search methods that use 2-opt, Insertion-1, and Insertion-2 neighborhoods.

On the other hand, U-VND is tested under only two different settings as U-VND that uses the best improvement search strategy and the greedy initial solution and U-VND that uses the best improvement search strategy and the random initial solution. Note that if the first improvement search strategy is used within U-VND, then U-VND is equivalent to basic VND.

Table 1 summarizes results considering the entire set of 15,200 instances as a test case. Namely, the average solution values (Column *Avg.*) and average CPU times (Column *Time (s)*) over all test instances in the data set, attained by VND variants under the best settings, are reported.

From the results presented in Table 1, the following conclusions may be drawn: (i) U-VND is slightly better than the other VNDs, regarding the best average values attained, but much slower than the others. This is explained by the fact that U-VND in each iteration performs exploration of large part of the solution space before deciding to re-center the search. Obviously, such principle is suitable for reaching good final solution, but requires a large CPU time; (ii) comparing VNDs that re-center the search in the inner loop (i.e., B-VND, P-VND, and C-VND), it follows that B-VND is able to provide the best solution values. Regarding average CPU times consumed by B-VND, P-VND and C-VND to find the best reported average solution value, their ranking is as follows: the fastest is P-VND, B-VND is ranked

as the second, while C-VND is the slowest one. However, since, B-VND consumed negligible more CPU time to find the best reported average solution value comparing to CPU time that P-VND consumed to do so, we may conclude that B-VND is the most appropriate sequential VND version.

Nested Variable Neighborhood Descent

A nested (composite) variable neighborhood descent procedure [21] explores a large neighborhood structure obtained as a composition of several neighborhoods. More precisely, let $\mathcal{N} = \{\mathcal{N}_1, \ldots \mathcal{N}_{k_{max}}\}$ again be set of move operators such that each one $\mathcal{N}_k$, $1 \leq k \leq k_{max}$ maps a given solution x to a predefined neighborhood structure $\mathcal{N}_k(x)$. Then, the neighborhood explored within a nested variable neighborhood procedure is defined with operator $\mathcal{N}_{nested} = \mathcal{N}_1 \circ \mathcal{N}_2 \circ \cdots \circ \mathcal{N}_{k_{max}}$. More precisely, the composite neighborhood of solution x is formed by first applying the move operator $\mathcal{N}_1$, obtaining $\mathcal{N}_1(x)$. Then, it is applied the move operator $\mathcal{N}_2$ to all solutions in $\mathcal{N}_1(x)$, forming the set $\mathcal{N}_1(\mathcal{N}_2(x))$, and so on. Obviously, the cardinality of a neighborhood structure $\mathcal{N}_{nested}x) = \mathcal{N}_1(\mathcal{N}_2(\ldots(\mathcal{N}_{k_{max}}(x))))$ of some solution x is less than or equal to the product of cardinalities of nested (composed) neighborhoods. More formally

$$|N(x)| \leq \prod_{k=1}^{k_{max}} |N_k(x)|$$

Such cardinality obviously increases chances to find an improvement in the neighborhood. The nested VND is illustrated in Algorithm 3. The neighborhood $\mathcal{N}(x)$ may be explored by using either the first or the best improvement search strategy. However, since its cardinality is usually very large, the first improvement is used more often [21,43].

Algorithm 3: *NestedVND*$(x, k_{\max}, \mathcal{N})$

1: $\mathcal{N}_{nested} = \mathcal{N}_1 \circ \mathcal{N}_2 \circ \cdots \circ \mathcal{N}_{k_{\max}}$
2: **repeat**
3: $x' \leftarrow x$
4: $x \leftarrow \arg\min_{y \in \mathcal{N}_{nested}(x')} f(y)$
5: **until** $f(x') \leq f(x)$
6: **return** x'

We use the Uncapacitated r-Allocation p-Hub Median Problem (r-p-HMP) to illustrate how this VND strategy works. Specifically, the r-p-HMP may be stated as follows. Given n nodes, this problem considers for each pair of nodes i and j, the distance d_{ij} and the amount of flow $t_{ij} \geq 0$ that needs to be transferred from i to j. It is generally assumed that the transportation between non-hub nodes i, j is

only possible via hub nodes h_i, h_j, to which nodes i, j are assigned, respectively. Transferring t_{ij} units of flow through path $i \rightarrow h_i \rightarrow h_j \rightarrow j$ induces a cost $c_{ij}(h_i, h_j)$, which is computed as

$$c_{ij}(h_i, h_j) = t_{ij}(\gamma d_{ih_i} + \alpha d_{h_i h_j} + \delta d_{h_j j}).$$

Parameters γ, α, and δ represent unit rates for collection (origin–hub), transfer (hub–hub), and distribution (hub–destination), respectively. Note that the hub nodes h_i and h_j may be equal.

We represent a solution of r-p-HMP by a set H containing p hubs and a matrix A, where each row i contains r hubs assigned to node i (i.e., i-throw coincides with the set H_i). Thus, our solution is represented as $x = (H, A)$. The initial solution is obtained using the greedy heuristic described in [40]. We consider two neighborhood structures of a given solution $x = (H, A)$. The first neighborhood structure, denoted by $\mathcal{N}_H$, is obtained by replacing one hub node from H by another non-hub node from $N \setminus H$ (see Fig. 11). More formally,

$$\mathcal{N}_H(x) = \{x' \mid x' = (H', A'), |H \cap H'| = p - 1\}.$$

The second neighborhood, denoted by $\mathcal{N}_A$, is obtained by replacing one hub assigned to some node with another hub, while the set H remains unchanged (see Fig. 12):

$$\mathcal{N}_A(x) = \{x' | x' = (H, A'), |A \setminus A'| = 1\}.$$

Unfortunately, evaluating the objective function value of a solution from $\mathcal{N}_H$ requires to solve an allocation problem, which is $\mathcal{NP}$-hard by itself [25]. Therefore, solving exactly the associated allocation problem will be quite time-consuming. In

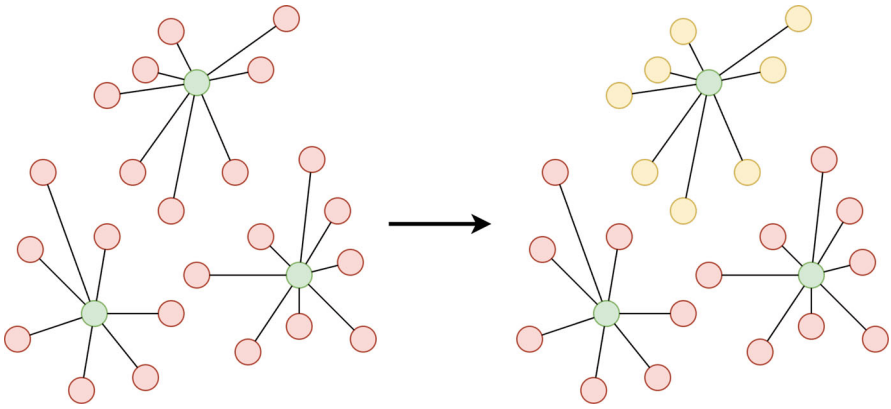


Fig. 11 Interchange neighborhood $\mathcal{N}_H$. Yellow dots are the possible new hub location for the one represented in green

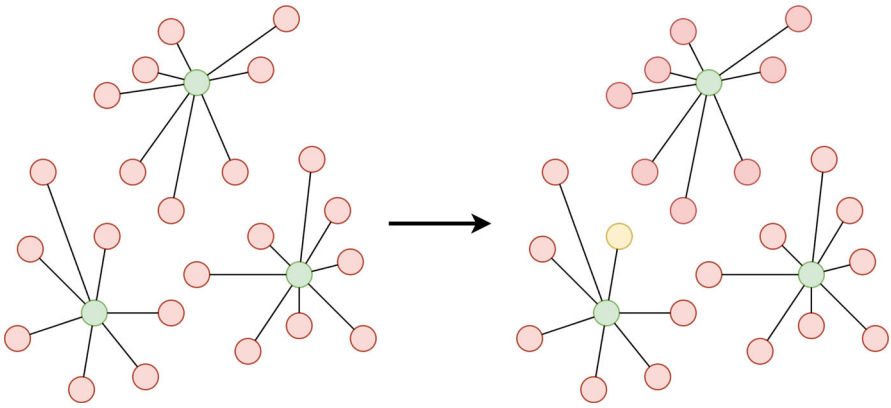


Fig. 12 Allocate neighborhood $\mathcal{N}_A$. Yellow dot represents the non-hub node which is reallocated to the another hub

order to deal with this drawback, we find near-optimal allocation A' of some solution $x'' \in \mathcal{N}_H$ as follows. For each node, we firstly determine node-to-hub allocation using the greedy procedure (see Algorithm 4).

Algorithm 4: *GreedyAllocation*(H, A)

for $i \in \mathcal{N}$ **do**
  for $j \in 1 \dots p$ **do**
    $value(j) = d_{ih_j} \sum_{k \in N} t_{ik} + \sum_{k \in N} t_{ik} d_{h_j k}$
  end for
  $Sort(value)$
  for $j \in 1 \dots r$ **do**
    $A[i][j] = h_{\pi(j)}$
  end for
end for

The solution $x'' = (H'', A'')$ obtained in this way is then improved by exploring the second neighborhood $\mathcal{N}_A(x'')$. In that way, the so-called nested variable neighborhood descent (Nest-VND) is defined (see Algorithm 5).

Algorithm 5: *NestedVND*(x)

for $x'' = (H'', A'') \in \mathcal{N}_H(x)$ **do**
  $GreedyAllocation(H'', A'')$
  $x' \leftarrow \arg\min_{y \in \mathcal{N}_A(x'')} f(y)$
  if $f(x') < f(x)$ **then**
    $x \leftarrow x'$
  end if
end for
return x

Table 2 Comparison of GRASP and GVNS on AP instances

	GRASP		GVNS_RP		
n	Avg.	Time (s)	Avg.	Time (s)	% Improve
60	122,348.90	4.59	121,829.27	3.73	0.42
65	123,001.53	6.66	122,689.74	5.87	0.25
70	123,931.76	10.51	123,604.38	5.75	0.26
75	124,776.42	11.11	124,650.73	5.93	0.10
80	125,148.22	14.40	124,844.76	9.36	0.24
85	125,566.58	19.48	125,378.23	13.10	0.15
90	124,934.99	22.95	124,734.55	12.32	0.16
95	125,121.18	24.27	124,926.55	25.45	0.16
100	125,805.04	4.81	125,588.19	10.39	0.17
150	126,728.85	21.42	126,307.10	24.70	0.33
200	129,144.44	58.86	128,788.66	98.67	0.28
Avg.	125,137.08	18.10	124,849.29	19.57	0.23

In [43], the above nested VNDs were used as a local search within a general variable neighborhood search (GVNS)-based heuristic. The performance of this heuristic, named `GVNS_RP`, was compared with the GRASP heuristic proposed in [40]. In Table 2, we provide a summarized result of comparison of `GVNS_RP` and GRASP on instances from AP data set (see [43]). The average value of best found solutions and average CPU times needed for finding these solutions over all instances with the same number of nodes are reported. The column headings are defined as follows. In the first column of Table 2, we report the number of nodes in the considered instances, whereas in the columns "Avg.," we provide the average of best solution values found by GRASP and `GVNS_RP`, respectively. In columns "Time (s)," the average time needed to reach best found solutions for instances with n nodes is given, while in Column "% Improve," we report the percentage improvement obtained by `GVNS_RP` compared with the current best known values. From the reported results, it follows that within each set of instances with the same number of nodes, there is at least one instance where the best known solution is improved by `GVNS_RP`. Moreover, the average improvement on AP data set achieved by GVNS variants is around 0.25%.

Mixed Variable Neighborhood Descent

Mixed variable neighborhood descent [21] combines ideas of sequential and nested variable neighborhood descent. Namely, it uses a set of move-operators $\mathcal{N} = \{\mathcal{N}_1, \ldots \mathcal{N}_b\}$ to define a nested neighborhood, and, after that, on each element in this nested neighborhood, it applies a sequential variable neighborhood descent variant defined by a set of move-operators $\mathcal{N}' = \{\mathcal{N}_{b+1}, \ldots \mathcal{N}_{k_{\max}}\}$. The cardinality of the set explored in one iteration of a mixed VND is bounded by

$$|\mathcal{N}_{mixed}(x)| \leq \prod_{\ell=1}^{b} |\mathcal{N}_\ell(x)| \times \sum_{\ell=b+1}^{k_{\max}} |\mathcal{N}_\ell(x)|, x \in S.$$

In Algorithm 6 we show the pseudo-code of a mixed VND. Note that if the set $\mathcal{N}$ is the empty set (i.e., $b = 0$), we get pure sequential VND. If $b = k_{\max}$, we get pure nested VND. Since nested VND intensifies the search in a deterministic way, boost parameter b may be seen as a way of balancing intensification and diversification in deterministic local search with several neighborhoods.

Algorithm 6: *MixedVND*$(x, b, k_{\max}, \mathcal{N}, \mathcal{N}')$

$\mathcal{N} \leftarrow \mathcal{N}_1 \circ \mathcal{N}_2 \circ \cdots \circ \mathcal{N}_b$
$x' \leftarrow x$
repeat
  stop $\leftarrow$ `True`
  $x \leftarrow x'$
  for $y \in \mathcal{N}(x)$ **do**
    $x'' \leftarrow SeqVND(y, k_{\max} - b, \mathcal{N}')$
    if $f(x'') < f(x')$ **then**
      stop $\leftarrow$ `False`
      $x' \leftarrow x''$
    end if
  end for
until *stop* = `True`
return x'

In [21], two mixed VND heuristics along with one basic sequential VND heuristic were proposed for solving the uncapacitated single allocation p-hub median problem (USApHMP). Neighborhood structures examined within these VND variants are cluster based. A cluster represents one hub with all locations assigned to it. In particular, the following neighborhood structures are distinguished:

- *Allocate*: change membership of a non-hub node by reallocating it to the another hub without changing the location of hubs.
- *Alternate*: change the location of the hub in one cluster.
- *Locate*: select one cluster C with hub h and a location node that is not in this cluster. The selected node becomes a hub, and all locations from the cluster C are assigned to the closest hub (including the new one).

The proposed basic sequential VND heuristic, named `Seq-VND`, examines neighborhood structures *Allocate*, *Alternate*, and *Locate* in that order. On the other hand, the first mixed VND heuristic, named `Mix-VND1`, takes several random points from *Locate* neighborhood and starting from each of them carries out search applying a basic sequential VND heuristic `Seq-VND1`. The used `Seq-VND1` explores *Allocate* and *Alternate* neighborhoods in that order. The second mixed VND heuristic, named `Mix-VND2`, performs more thoroughly exploration of the solution space than `Mix-VND1` applying `Seq-VND1` on each point from *Locate*

neighborhood. These three VND variants together with three local searches in the three defined neighborhood structures (Allocate, Alternate, and Locate) are experimentally compared on AP instance with $n = 100$ and $p = 15$ (see [21]). One thousand initial points are generated at random in the solution space, and a local search is conducted from each one in each type of neighborhood. The distance-to-target diagram [16] presented at Fig. 13 shows the distribution of local minima. Each point (x, y) plots the distance x and percentage deviation y of the local minimum from the best known solution. The results are summarized in Table 3.

Comparing the results in Table 3, we observe that (i) the search in first (*Allocate*) and second (*Alternate*) local neighborhoods are faster than the others, but they have the worse quality; (ii) `Seq-VND` gives better results than the third local search (*Locate*), and is considerable faster; (iii) all three multistart VND local searches

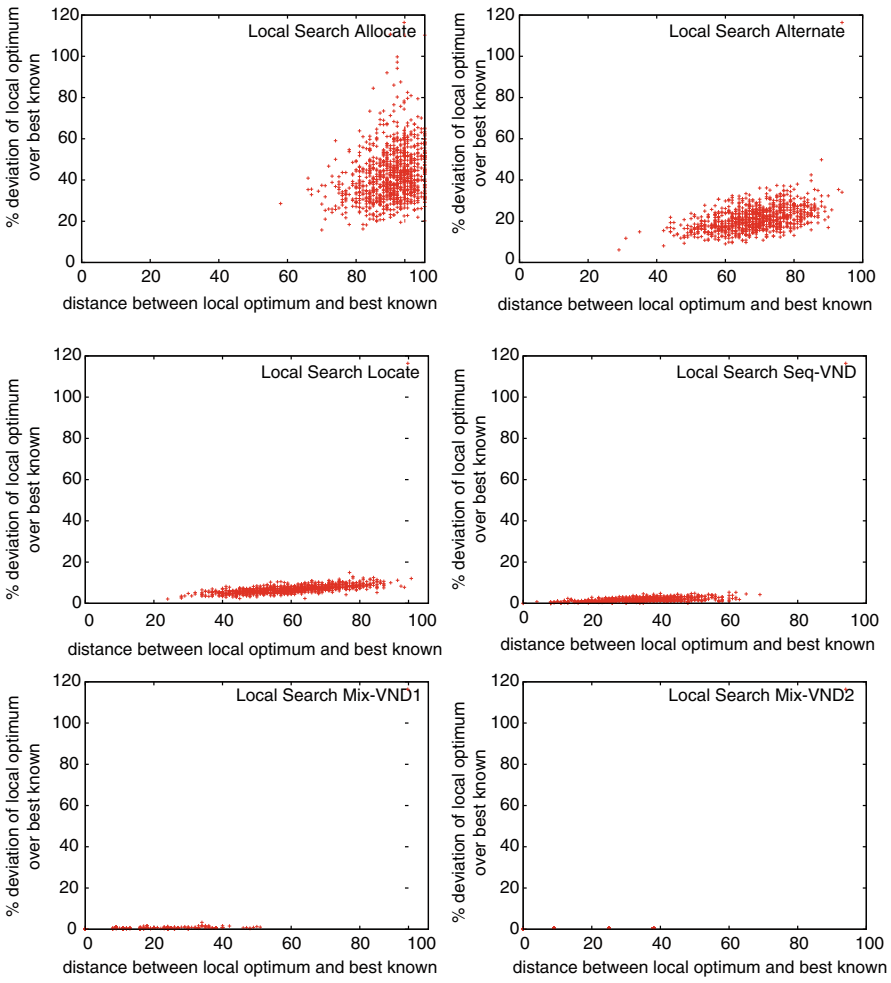


Fig. 13 Multistart on 1000 random initial solutions for AP instance with $n = 100$ and $p = 15$

Table 3 Comparison of 6 local searches on AP instance with $n = 100$ and $p = 15$

	Allocate	Alternate	Locate	Seq-VND	Mix-VND1	Mix-VND2
Avg. % Dev	41.28	21.17	7.41	1.62	0.35	0.16
Min. % Dev.	15.87	6.14	2.58	0.00	0.00	0.00
Max. % Dev.	110.63	49.84	14.6	5.37	3.41	0.68
Avg. Time (s)	0.002	0.004	0.15	0.06	3.36	27.15

found the best known solution in the AP instance; and (iv) the mixed VND versions give the best results, although, as expected, solution times are longer than solution time of Seq-VND.

We see in Fig. 13 that the number of local minima is reduced when VND is used. This is due to the larger neighborhood in VND. For example, the Seq-VND neighborhood clearly contains each of the individual neighborhoods, *Allocate*, *Alternate*, and *Locate*. It is remarkable that Mix-VND2, which utilizes the largest neighborhood, yields only four local minima for this problem instance.

Conclusions

Local search represents one of the most popular classical heuristic techniques that improves (locally) the current feasible solution of some continuous or discrete optimization problem. For that purposes, usually only one neighborhood structure is defined and explored to improve the incumbent solution. Searching for the best solution in such neighborhood is repeated until the local minimum with respect to that predefined neighborhood is reached.

Since the local minimum with respect to one neighborhood structure is not necessary local in another, one needs to construct deterministic local search in cases when more than one neighborhood is used. Such procedures are known as variable neighborhood descent (VND). In this VND survey, we first propose a possible classification of neighborhood structures in solving optimization problems and also provide some simple examples to clearly illustrate the basic ideas. Then, we discuss possible general ways of combining several neighborhoods in the deterministic fashion. Needless to say that the number of possible combination of VND local search variants is large, and they could include problem-specific knowledge in building heuristic for each particular problem. Therefore, VND area is an open avenue for the future research in the area of optimization.

Cross-References

▸ Guided Local Search
▸ Restart Strategies
▸ Theory of Local Search

Acknowledgments Work of Nenad Mladenović and Raca Todosijević is partly supported by the Ministry of Education and Science, Republic of Kazakhstan (Institute of Information and Computer Technologies), project number 0115PK00546, and also by the Ministry of Education, Science and Technological Development of Serbia, project number 174010. Work of Abraham Duarte and Jesús Sánchez-Oro is partly supported by the Spanish "Ministerio de Economía y Competitividad," with grants refs. PID2021-126605NB-I00 and PID2021-125709OA-C22.

Competing Interest Declaration The author(s) has no competing interests to declare that are relevant to the content of this manuscript.

References

1. Brimberg J, Hansen P, Mladenović N, Continuous optimization by variable neighborhood search. In: Encyclopedia of operations research and management science (EORMS), to appear
2. Carrasco R, Pham A, Gallego M, Gortázar F, Martí R, Duarte A (2015) Tabu search for the max–mean dispersion problem. Knowl-Based Syst 85:256–264
3. Cook W, Cunningham W, Pulleyblank W, Schrijver A (1997) Combinatorial optimization. Wiley, New York
4. Deza M, Huang T (1998) Metrics on permutations, a survey. J Comb Inf Syst Sci. Citeseer
5. Dorigo M, Gambardella LM (1997) Ant colony system: a cooperative learning approach to the traveling salesman problem. IEEE Trans Evol Comput 1(1):53–66. https://doi.org/10.1109/4235.585892
6. Duarte A, Martí R (2007) Tabu Search and GRASP for the maximum diversity problem. Eur J Oper Res 178(1):71–84
7. Duarte A, Sánchez A, Fernández F, Cabido R (2005) A low-level hybridization between memetic algorithm and VNS for the max-cut problem. In: ACM genetic and evolutionary computation conference
8. Duarte A, Escudero LF, Martí R, Mladenović N, Pantrigo J, Sánchez Oro J (2012) Variable neighborhood search for the Vertex Separation Problem. Comput Oper Res 39(12):3247–3255. https://doi.org/10.1016/j.cor.2012.04.017, http://www.sciencedirect.com/science/article/pii/S0305054812000974
9. Feige U (1998) A threshold of Ln N for approximating set cover. J ACM 45(4):634–652. https://doi.org/10.1145/285055.285059
10. Feo TA, Resende MGC (1995) Greedy randomized adaptive search procedures. J Glob Optim 6:109–133
11. Gallego M, Laguna M, Martí R, Duarte A (2013) Tabu search with strategic oscillation for the maximally diverse grouping problem. J Oper Res Soc 64(5):724–734. https://doi.org/10.1057/jors.2012.66
12. Garey MR, Johnson DS (1979) Computers and intractability: a guide to the theory of NP-completeness. W. H. Freeman & Co., New York
13. Glover F (1986) Future paths for integer programming and links to artificial intelligence. Comput Oper Res 13(5):533–549. https://doi.org/10.1016/0305-0548(86)90048-1
14. Glover F (1998) A template for scatter search and path relinking. In: Selected papers from the third European conference on artificial evolution, AE'97. Springer, London, pp 3–54. http://dl.acm.org/citation.cfm?id=647454.727571
15. Hansen P, Mladenović N (1999) An introduction to variable neighborhood search. In: Meta-heuristics. Springer, pp 433–458. https://doi.org/10.1007/978-1-4615-5775-3_30
16. Hansen P, Mladenović N (2003) Variable neighborhood search. In: Glover F, Kochenberger G (eds) Handbook of metaheuristics. Kluwer Academic Publisher, New York, pp 145–184
17. Hansen P, Mladenović N (2006) First vs. best improvement: an empirical study. Discret Appl Math 154(5):802–817

18. Hansen P, Mladenović N, Brimberg J, Pérez JAM (2019) Variable neighborhood search. Springer, Berlin
19. Holland J (1992) Adaptation in natural and artificial systems. MIT Press, Cambridge, MA
20. Hoos H, Süttzle T (2004) Stochastic local search: foundations and applications. Morgan Kaufmann Publishers Inc., San Francisco
21. Ilić A, Urošević D, Brimberg J, Mladenović N (2010) A general variable neighborhood search for solving the uncapacitated single allocation p-hub median problem. Eur J Oper Res 206(2):289–300
22. Karp R (1972) Reducibility among combinatorial problems. In: Complexity of computer computations. The IBM research symposia series. Springer, pp 85–103. https://doi.org/10.1007/978-1-4684-2001-2_9
23. Laarhoven PJM, Aarts EHL (1987) Simulated annealing: theory and applications. Kluwer Academic Publishers, Norwell
24. Laguna M, Gortázar F, Gallego M, Duarte A, Martí R (2014) A black-box scatter search for optimization problems with integer variables. J Glob Optim 58(3):497–516. https://doi.org/10.1007/s10898-013-0061-2
25. Love RF, Morris JG, Wesolowski GO (1988) Facilities location: models and methods. Elsevier Science Publishing Co., New York
26. Lü Z, Hao JK, Glover F (2011) Neighborhood analysis: a case study on curriculum-based course timetabling. J Heurist 17(2):97–118
27. Makedon FS, Papadimitriou CH, Sudborough IH (1985) Topological bandwidth. SIAM J Algebraic Discret Methods 6(3):418–444. https://doi.org/10.1137/0606044
28. Martello S, Toth P (1990) Knapsack problems: algorithms and computer implementations. Wiley, New York
29. Martí R, Duarte A, Laguna M (2009) Advanced scatter search for the max-cut problem. INFORMS J Comput 21(1):26–38. https://doi.org/10.1287/ijoc.1080.0275
30. Martí R, Reinelt G, Duarte A (2012) A benchmark library and a comparison of heuristic methods for the linear ordering problem. Comput Optim Appl 51(3):1297–1317. https://doi.org/10.1007/s10589-010-9384-9
31. Mjirda A, Todosijević R, Hanafi S, Hansen P, Mladenović N (2016) Sequential variable neighborhood descent variants: an empirical study on travelling salesman problem. Int Trans Oper Res. https://doi.org/10.1111/itor.12282
32. Mladenović N, Hansen P (1997) Variable neighborhood search. Comput Oper Res 24(11):1097–1100. https://doi.org/10.1016/S0305-0548(97)00031-2
33. Moscato P (1993) An introduction to population approaches for optimization and hierarchical objective functions: a discussion on the role of Tabu search. Ann Oper Res 41(1–4):85–121. http://dl.acm.org/citation.cfm?id=160231.160242
34. Sánchez Oro J, Mladenović N, Duarte A (2014) General Variable Neighborhood Search for computing graph separators. Optim Lett 1–21. https://doi.org/10.1007/s11590-014-0793-z
35. Sánchez Oro J, Pantrigo JJ, Duarte A (2014) Combining intensification and diversification strategies in VNS. An application to the Vertex Separation problem. Comput Oper Res 52, Part B(0):209–219. https://doi.org/10.1016/j.cor.2013.11.008, http://www.sciencedirect.com/science/article/pii/S0305054813003213, recent advances in Variable neighborhood search
36. Pantrigo J, Martí R, Duarte A, Pardo E (2012) Scatter search for the cutwidth minimization problem. Ann Oper Res 199(1):285–304. https://doi.org/10.1007/s10479-011-0907-2
37. Papadimitriou C, Steiglitz K (1998) Combinatorial optimization: algorithms and complexity. Dover
38. Papadimitriou CH (1977) The Euclidean travelling salesman problem is NP-complete. Theor Comput Sci 4(3):237–244
39. Pardo EG, Mladenović N, Pantrigo JJ, Duarte A (2013) Variable formulation search for the cutwidth minimization problem. Appl Soft Comput 13(5):2242–2252. https://doi.org/10.1016/j.asoc.2013.01.016, http://www.sciencedirect.com/science/article/pii/S1568494613000355
40. Peiró J, Corberán A, Martí R (2014) GRASP for the uncapacitated r-allocation p-hub median problem. Comput Oper Res 43:50–60

41. Ruiz R, Stützle T (2006) A simple and effective iterated greedy algorithm for the permutation flowshop scheduling problem. Eur J Oper Res 177:2033–2049
42. Talbi E (2009) Metaheuristics: from design to implementation. Wiley, New York
43. Todosijević R, Urošević D, Mladenović N, Hanafi S (2015) A general variable neighborhood search for solving the uncapacitated r-allocation p-hub median problem. Optim Lett. https://doi.org/10.1007/s11590-015-0867-6
44. Wu Q, Hao JK, Glover F (2012) Multi-neighborhood tabu search for the maximum weight clique problem. Ann Oper Res 196(1):611–634

Part III
Metaheuristics

Ant Colony Optimization: A Component-Wise Overview

19

Manuel López-Ibáñez, Thomas Stützle, and Marco Dorigo

Contents

Introduction . . . 544
Combinatorial Optimization Problems and Constructive Heuristics . . . 546
The ACO Algorithmic Framework . . . 548
Choice of Pheromone Trails and Heuristic Information . . . 549
Solution Construction . . . 551
Global Pheromone Update . . . 553
Pheromone Update Schedule . . . 556
Initialization of Pheromones . . . 557
Pheromone Reinitialization . . . 557
Local Pheromone Update . . . 557
Pheromone Limits . . . 558
Local Search . . . 559
ACO Algorithms as Instantiations of the ACO Metaheuristic . . . 559
ACOTSP/ACOQAP: A Unified Framework of ACO Algorithms for the TSP and QAP . . . 560
Finding a Better ACO Configuration for the TSP . . . 561
Finding a Better ACO Configuration for the QAP . . . 564
Applications of ACO to Other Problem Types . . . 566
Continuous Optimization Problems . . . 567
Multi-objective Problems . . . 567
Dynamic Problems . . . 568
Stochastic Problems . . . 569
ACO in Combination with Other Methods . . . 569
ACO and Tree Search Methods . . . 569
ACO and Exact Methods . . . 570

M. López-Ibáñez
Alliance Manchester Business School, University of Manchester, Manchester, UK
e-mail: manuel.lopez-ibanez@manchester.ac.uk

T. Stützle (✉) · M. Dorigo
IRIDIA, Université Libre de Bruxelles (ULB), Brussels, Belgium
e-mail: stuetzle@ulb.ac.be; mdorigo@ulb.ac.be

© Springer Nature Switzerland AG 2025

R. Martí et al. (eds.), *Handbook of Heuristics*,
https://doi.org/10.1007/978-3-032-00385-0_21

ACO and Surrogate Models . . . 571
Parameter Adaptation . . . 571
Conclusions . . . 572
Cross-References . . . 574
References . . . 574

Abstract

The indirect communication and foraging behavior of certain species of ants have inspired a number of optimization algorithms for NP-hard problems. These algorithms are nowadays collectively known as the ant colony optimization (ACO) metaheuristic. This chapter gives an overview of the history of ACO, explains in detail its algorithmic components, and summarizes its key characteristics. In addition, the chapter introduces a software framework that unifies the implementation of these ACO algorithms for two example problems, the traveling salesman problem and the quadratic assignment problem. By configuring the parameters of the framework, one can combine features from various ACO algorithms in novel ways. Examples on how to find a good configuration automatically are given in the chapter. The chapter closes with a review of combinations of ACO with other techniques and extensions of the ACO metaheuristic to other problem classes.

Keywords

Ant colony optimization · Automatic configuration · Combinatorial optimization · Metaheuristics

Introduction

Ant colony optimization (ACO) [35, 37, 39] is a metaheuristic that generates candidate solutions by repeated applications of a probabilistic solution construction procedure. In ACO, the probabilities that bias the solution construction are computed from two types of numerical information associated with construction decisions: *heuristic information*, which is derived from the problem instance being tackled, and *artificial pheromone trails*, which are adapted based on the search performance to bias the solution construction toward high-quality solutions. Pheromone trails are the result of a learning mechanism that tries to identify the solution components that, when appropriately combined, lead to high-quality solutions.

As the name suggests, ACO was inspired by entomological experiments on real ant colonies. These experiments demonstrated that some ant species can find the shortest path between their nest and a food source and showed how this mechanism is enabled by the ants' pheromone trail laying and following behavior [25]. The equations used in computer simulations that mimicked the probabilistic behavior of real ant colonies inspired computer scientists to define the way artificial ants

take decisions when solving combinatorial optimization problems [34, 40, 42]. Nonetheless, rather than faithfully simulating the behavior of natural ants, the focus of ACO research, since its early days, has been the computational performance of ACO algorithms and the quality of the solutions that can be generated. To this aim, additional algorithmic techniques have been quickly added to the first ACO algorithms to make them competitive with other metaheuristics. One example of such algorithmic techniques is the exploitation of heuristic information for guiding the construction of solutions [34, 40, 42]; another example is the use of local search algorithms for improving the solutions constructed by the ants [38, 39, 52, 137, 139].

The first ACO algorithm, *ant system*, was proposed by Dorigo et al. [34, 40–42]. After the publication of the first journal paper on ant system in 1996 [42], the research on ACO has gained momentum, and a large number of algorithmic improvements have been proposed. A first improvement over the basic form of ant system was elitist ant system (EAS) [34]. Among the most successful of these successors have been ant colony system (ACS) [38, 50] and max-min ant system (MMAS) [136, 137, 139]. Generally speaking, the main features of these improvements over the basic ant system include mechanisms to intensify the search around high-quality solutions and mechanisms to maintain a sufficient level of search space exploration.

When viewing algorithms from a component-wise perspective, that is, when considering that an algorithm is composed of algorithmic components that fulfill specific tasks for which alternative procedures may exist, there are similarities between various components of MMAS, ACS, and other ACO algorithms such as EAS [34, 42], rank-based ant system (RAS) [22], best-worst ant system (BWAS) [24], and others. In this chapter, we present the available ACO algorithms for combinatorial optimization from such a component-wise perspective. This perspective makes the contributions of each of the ACO algorithms more easily identifiable, which allows a more flexible approach toward their implementation. One additional contribution of this chapter is to provide such an implementation. The main advantage of this framework is that it enables the composition of new ACO variants. While one possible approach would be to manually explore different ACO algorithm designs and test their effectiveness, this exploration may also be done using automatic methods [70]. For illustration purposes, we demonstrate here how to find the best configuration of such a framework for the traveling salesman problem (TSP) and the quadratic assignment problem (QAP) using irace, an automatic algorithm configuration method [91].

This chapter is structured as follows. In section "Combinatorial Optimization Problems and Constructive Heuristics" we introduce some basic notions of combinatorial optimization and construction heuristics. Next, in section "The ACO Algorithmic Framework," we present the ACO metaheuristic as an algorithmic framework for single-objective combinatorial optimization problems, explaining ACO algorithms from a component-wise perspective. Section "ACOTSP/ACOQAP: A Unified Framework of ACO Algorithms for the TSP and QAP" describes how historical ACO algorithms are instantiations of this framework and how ACO

algorithms can be automatically configured. Section "Applications of ACO to Other Problem Types" describes recent developments in ACO applied to problems with particular characteristics, including continuous optimization and mixed-variable, multi-objective, and dynamic problems. Section "ACO in Combination with Other Methods" reviews hybridizations of ACO with other methods, such as tree search methods, exact methods, and surrogate models. We also briefly discuss recent experiments in dynamically changing the parameters of ACO algorithms during execution. Finally, section "Conclusions" concludes with some perspectives for future research in ACO.

Combinatorial Optimization Problems and Constructive Heuristics

Many problems of enormous practical and theoretical importance can be modeled as combinatorial optimization problems (COPs) [117]. Although practical problems usually have a large number of constraints to be satisfied and uncertainties or multiple objectives to be considered, even simpler COPs without such complicating factors may be very difficult to solve.

A COP can be defined as consisting of a set of instances [117]. An instance of a combinatorial optimization problem $(\mathscr{S}, \Omega, f)$ is defined by

- A search space $\mathscr{S}$ given by the possible value assignments to discrete decision variables x_i, with $i = 1, \ldots, n$;
- A set of constraints Ω among the decision variables;
- An objective function $f : \mathscr{S} \to \mathbb{R}$, which may have to be minimized or maximized.

The notion of *problem* refers to the general abstract task to be solved, while the *instance* is defined as a concrete instantiation of this task with fully specified data.

One of the most studied combinatorial optimization problems is the traveling salesman problem (TSP). In the TSP, a graph $G = (N, E)$ is given with $n = |N|$ nodes, a set E of edges fully connecting the nodes, and distances associated with the edges $d_{ij}, \forall (i, j) \in E$. The goal is to find a Hamiltonian tour of minimal length. Such a tour can be represented as a permutation $\pi = (\pi_1, \ldots, \pi_n)^t$ of the n nodes, where π_i is the node index at position i. Thus, $\mathscr{S}$ is the space of such permutations. The objective function in the TSP is

$$\min_{\pi \in \mathscr{S}} d_{\pi_n \pi_1} + \sum_{i=1}^{n-1} d_{\pi_i \pi_{i+1}} \tag{1}$$

It is important to note that the permutations in the TSP are cyclic, that is, for a TSP solution, the absolute position of a node in the permutation is not relevant.

Another well-known combinatorial problem is the quadratic assignment problem (QAP), where two matrices are given describing the distances between n locations and the flows (of persons, goods, etc.) between n facilities. The goal is to find an assignment of facilities to locations that minimize the sum of the products between distances and flows. More formally, given two $n \times n$ matrices $[d_{ij}]$ and $[f_{ij}]$ with $i, j = 1, \ldots, n$, a solution to the QAP is an assignment of facilities to locations (or, equivalently, of locations to facilities). Because the number of facilities is the same as the number of locations, such an assignment can be represented by a permutation π, where π_i is the facility assigned to location i. The objective function in the QAP is

$$\min_{\pi \in \mathscr{S}} \sum_{i=1}^{n} \sum_{j=1}^{n} d_{ij} \cdot f_{\pi_i \pi_j} \tag{2}$$

Solving a combinatorial optimization problem can often be described as choosing a subset from a finite set of *solution components* $\mathscr{C} = \{c_1, c_2, \ldots\}$ and determining their precise combination that results in a solution that satisfies the problem constraints, i.e., a *feasible* solution, and that optimizes the objective function, i.e., an optimal solution. (For what follows, we assume without loss of generality that the objective function is to be minimized.) In the TSP, for example, the solution components c_{ij} are typically taken to be the edges (i, j) of the given graph, whereas in the QAP, they are the possible individual assignments of a facility j to a location i. Commonly, problems are solved by either searching only in the feasible candidate solution space or, if this is deemed to be too difficult, by allowing the evaluation of infeasible candidate solutions but biasing the search in some way toward feasible ones. For both the TSP and the QAP, a search in the feasible space can be easily enforced, as any permutation is a feasible candidate solution.

Many COPs with practical applications belong to the class of NP-hard problems [54], and, hence, they are considered hard to solve. The NP-hardness of many of these problems, including the TSP and the QAP, implies that, in the worst case, the time needed to find the optimal solution for a given instance grows exponentially with instance size. Thus, instead of searching for an optimal solution and proving its optimality, which may require an infeasible amount of computation time, heuristic algorithms are used to generate good solutions within a reasonable time.

Perhaps the simplest heuristic methods are constructive heuristics. A constructive heuristic starts from an empty or partial candidate solution and then iteratively extends it by adding solution components to it. In many cases, a heuristic estimation of the quality of solution components is available and can be used to guide the choice of which solution components to add. Greedy constructive heuristics rank solution components according to their heuristic value, and, at each construction step, they add the best-ranked solution component to the current partial solution. If more than

one solution component has the best heuristic value, tiebreaking can be done either randomly or by a secondary heuristic.

An example of constructive heuristic for the TSP is the *nearest neighbor* heuristic. Starting from a random node i and an empty candidate solution $\pi = \langle\rangle$, it selects the solution component c_{ij}, $j \in N \backslash \{i\}$ with the smallest distance d_{ij} and adds it to π such that $\pi_1 = i$ and $\pi_2 = j$. The next solution component c_{jk} chosen is the one that minimizes the distance d_{jk}, $k \in N \backslash \{i, j\}$, and so on until a complete permutation π of the nodes in N is obtained.

A constructive heuristic for the QAP would be guided by the fact that facilities with a high total flow to other facilities should be placed at locations that are central and, thus, have a small sum of distances to other locations. Thus, one may precompute values $f = (f_1, \ldots, f_n)^t$, where $f_i = \sum_{j=1}^{n} f_{ij}$, and analogously $d = (d_1, \ldots, d_n)^t$, where $d_k = \sum_{l=1}^{n} d_{kl}$, and then assign the facility with the largest f_i to the locations with the smallest d_k and iterate these steps.

If the heuristic values of solution components remain constant during the solution construction, the construction heuristic is called *static*. Otherwise, when the heuristic values are a function of the partial solution generated so far, one talks of an *adaptive* construction heuristic. In the QAP case, one may obtain an adaptive heuristic by using the intuition that facilities with a high interaction with already assigned ones should be put on a location that is as close as possible to already chosen locations. Adaptive heuristics generally require higher computation times as the heuristic values need to be updated or even recomputed after each construction step; however, making decisions dependent on a partial solution may lead to higher-quality complete candidate solutions.

Constructive heuristics are basic ingredients of many metaheuristics. As an example, semi-greedy heuristics [69] are a randomized form of constructive heuristics; i.e., they make randomized decisions during the constructive search, thus generating many different solutions. The GRASP metaheuristic [47, 48] extends upon semi-greedy heuristics by combining randomized adaptive construction heuristics with local search algorithms. Another example is iterated greedy algorithms [142], which consist in the repeated application of constructive heuristics that start from partial solutions. A destruction mechanism generates these partial solutions by removing solution components from a complete candidate solution.

The ACO Algorithmic Framework

ACO algorithms can also be seen as an extension of construction heuristics. The main characteristics that distinguish an ACO algorithm from other metaheuristics and, in particular, other constructive metaheuristics are the following: (i) It is a population-based algorithm, where m solutions are generated at each iteration; (ii) solutions are generated by a probabilistic constructive mechanism that is biased by numerical information called pheromones (and possibly by heuristic information);

```
procedure ACO_Metaheuristic
    repeat
        for each ant do
            repeat
                ExtendPartialSolutionProbabilistically()
            until solution is complete
        for each ant ∈ SelectAntsForLocalSearch() do    // optional
            ApplyLocalSearch(ant)                       // optional
        EvaporatePheromones()
        DepositPheromones()
    until termination criteria met
end
```

Fig. 1 General pseudo-code of the ACO metaheuristic for NP-hard problems. First, each ant constructs a complete solution by probabilistically choosing solution components to extend their current partial solution (section "Solution Construction"). Some of these solutions are sometimes further improved by means of local search (section "Local Search"). Finally, pheromone values are updated according to the solutions constructed by the ants (section "Global Pheromone Update"). The update usually consists of two complementary steps: evaporation decreases pheromone values and deposit increases them

and (iii) the pheromones are updated during the run of the algorithm using the quality of generated solutions as feedback.

The general outline of an ACO algorithm is given in Fig. 1. After initializing data structures and parameters, an ACO algorithm generally iterates through two procedures: solution construction (procedure ConstructSolutions) and pheromone update (procedure UpdatePheromones). Additionally, a local search algorithm may be used to improve some or all the solutions constructed in the current iteration. Clearly, each of these procedures may in turn consist of different phases, and, depending on the particular ACO algorithm implemented, the particular choices taken within these phases may differ quite strongly. These differences, however, mainly concern specific building blocks or parameter settings that are used within the ACO algorithm phases. These building blocks and parameters are referred as algorithmic components.

In the following, we present the algorithmic components that have been proposed in typical ACO algorithms for combinatorial optimization and discuss alternative options proposed in the literature for implementing these components.

Choice of Pheromone Trails and Heuristic Information

In ACO, the solution construction is probabilistically biased by two types of information: pheromone trails and heuristic information.

The pheromone trails are a set $\mathcal{T}$ of numerical values associated, in principle, to all solution components, that is, $\forall c \in \mathcal{C}$, $\exists \tau_c \in \mathcal{T}$. Pheromone trails are updated during the run of an ACO algorithm according to the quality of previously generated solutions through the mechanisms of pheromone deposit and pheromone evaporation. During the solution construction procedure, pheromone trails bias the choice of solution components toward constructing high-quality solutions. A high value of τ_c represents a higher probability of adding solution component c to the solution being constructed. It is therefore important to appropriately define the set of solution components $\mathcal{C}$, and the corresponding pheromone trails $\mathcal{T}$, when applying an ACO algorithm to a particular problem.

For many problems, the appropriate definition of solution component and, thus, of pheromone trail is rather straightforward. In the TSP, the adjacency information is relevant, that is, which node is the direct successor or predecessor to another one, and therefore a solution component $c_{ij} = (i, j)$ typically refers to the edge connecting nodes i and j. Thus, τ_{ij} represents the pheromone trail associated with the edge. This definition of solution component is also related to the choices made in the nearest neighbor heuristic, which underlies the construction mechanism used in almost all ACO applications to the TSP. Differently, in the QAP, the individual assignments of facilities to locations are of crucial importance. Therefore, a solution component c_{ij} denotes that facility j is assigned to location i in a candidate solution π, that is, $\pi_i = j$ and the pheromone trails τ_{ij} then represent the desirability of assigning facility j to location i. When constructing solutions for the QAP, the order in which facilities are chosen for assigning them to locations may also have an impact on the solution that is eventually constructed. Such an order may also be determined during solution construction by the ants through pheromone trails that indicate which facility should be chosen next for an assignment to a location [134]. Hence, one could have two types of pheromones, the first type τ_{ij} would refer to the desirability of assigning facility j to location i and the second one τ'_{kl} would refer to the desirability of generating an assignment for facility l directly after having assigned facility k. However, no effective implementation of such a second type of pheromone trails seems to be available.

For more complex problems than the TSP and the QAP, alternative definitions of solution components, and the corresponding pheromone trails, may have to be considered [111]. How to define solution components and their associated pheromone trails is itself an ongoing research topic [20, 109]. However, a basic guideline would be to consider effective greedy constructive heuristics and then define the pheromone trails to bias the most important decisions done by this heuristic. Using this guideline, standard definitions of solution components and pheromone trails will be available for many known problems; however, for more complex problems, some research effort may be required to determine the most appropriate solution components and pheromone trails.

The heuristic information $\mathcal{H}$ is also a set of numerical values associated with solution components ($\forall c \in \mathcal{C}$, $\exists \eta_c \in \mathcal{H}$). However, in contrast to the pheromone trails, heuristic information is not updated during the run of the algorithm in

dependence of the quality of the solutions generated. Instead, η_c is a value that is either constant for each solution component, when static heuristic information is used, or a value that is a function of the partial solution generated so far, when adaptive heuristic information is used. The actual formulation of the heuristic information is specific to each problem. Even sometimes, recently, reinforcement learning has been used for generating heuristic information [116]. However, its computation should be fast as the value of η_c is used at every step of the solution construction by each ant. In some problems, there is no readily available heuristic information that effectively guides solution construction, and thus, no heuristic information is used. This is, for example, the case in most ACO applications to the QAP.

Solution Construction

Solution construction is the process by which a new set of solutions is generated at each iteration of an ACO algorithms. In ACO terms, a solution is constructed by an ant; in optimization terms, each ant corresponds to the execution of a probabilistic solution construction procedure. In particular, each ant constructs one solution by starting from an empty solution $s = \langle\rangle$ and adding, at each construction step i, one solution component c_j from a set of candidate components N_i. The probabilistic choice at each step of the solution construction can be modeled by a probability distribution that assigns to each solution component the probability with which it is chosen. Thus, for each c_j, there is a probability $Pr(c_j)$ of being chosen. This probability depends on the pheromone trails, the heuristic information, and the current partial candidate solution s, that is, $Pr(c_j|\mathscr{T}, \mathscr{H}, s)$. There are many different ways of computing this probability. Many ACO algorithms use the probabilistic rule that was introduced for ant system (AS) to choose one element from N_i as follows:

$$\Pr(c_j) = \frac{{\tau_j}^{\alpha} \cdot {\eta_j}^{\beta}}{\sum_{c_k \in N_i} {\tau_k}^{\alpha} \cdot {\eta_k}^{\beta}} \quad \forall\, c_j \in N_i, \tag{3}$$

where α and β are parameters that control the relative importance of pheromone trails and heuristic information on the decision probabilities. If α tends to zero, the solution construction becomes a probabilistic multi-start greedy algorithm; if β tends to zero, the solution construction is biased only by the pheromone trails, and heuristic information is neglected. This rule assigns a probability in a fashion similar to the well-known roulette wheel selection of evolutionary algorithms [56], where the value ${\tau_j}^{\alpha} \cdot {\eta_j}^{\beta}$ plays the role of the fitness assigned to a candidate. Clearly, many different ways can be used to define probability distributions for the solution components. For example, Maniezzo [92, 93] uses in his ANTS algorithm an additive way of combining pheromone trails and heuristic information:

$$\Pr(c_j) = \frac{\alpha \cdot \tau_j + (1-\alpha) \cdot \eta_j}{\sum_{c_k \in N_i} \alpha \cdot \tau_k + (1-\alpha) \cdot \eta_k} \quad \forall c_j \in N_i. \tag{4}$$

The above equation has the advantage over the most common one (Eq. 3) that the operations used (multiplication instead of exponentiation and sum instead of multiplication) are measurably faster in current CPUs. However, one needs to ensure that the range of values of the pheromone trails and of the heuristic information is similar to avoid undesired biases.

A method for making the construction more deterministic was proposed by Dorigo and Gambardella [38] with their ant colony system (ACS) algorithm. They introduced the *pseudorandom proportional rule*, which is controlled by a parameter q_0. At each construction step i, a random value q is drawn uniformly from [0, 1); if $q > q_0$, the probability of c_j is computed as in Eq. 3; otherwise (i.e., when $q \leq q_0$) the choice is made as

$$c_j = \arg\max_{c_k \in N_i} {\tau_k}^{\alpha} \cdot {\eta_k}^{\beta}, \tag{5}$$

that is, when $q \leq q_0$, the solution component with maximum probability is chosen deterministically. A larger value of q_0 is equivalent to a more "greedy" construction procedure, which usually results in faster convergence, whereas smaller values of q_0 lead to more varied solutions and, thus, to more exploration.

Solution construction is one of the critical components of ACO; thus, techniques have been proposed to improve its efficacy and efficiency. Efficacy is sometimes improved by using *lookahead* [106], that is, by considering more than one component at a time when choosing which component should be added to the current partial solution. Efficiency can be greatly improved, particularly in large problem instances, by restricting the choice of solution components to *candidate lists* [37, 38]. Candidate lists contain the most promising solution components, and their definition depends on the instance data and/or on the current partial solution. The most efficient candidate lists depend only on the instance data and are typically computed at the start of the algorithm. One example of candidate lists is the nearest neighbor lists in the TSP, which store the *cl* nearest neighbors to each city ordered according to nondecreasing distances. Even a small value of *cl*, such as 20, is sufficient to generate very-high-quality (or even optimal) solutions to the TSP. Such candidate lists are crucial when applying ACO algorithms to large TSP instances.

Candidate lists may also save memory if the algorithm only stores the pheromone values of those solution components that are part of the candidate lists. Another approach to save memory used in some ACO variants is to avoid explicitly storing the pheromone trails of all solution components by using, for example, a hash table [3] to store only the pheromone values associated with solution components that have appeared in previouslyconstructed solutions. This might be beneficial when the fraction of pheromone values explicitly stored is small enough (relative

to the total size of $\mathscr{T}$) to compensate for the fact that insertion, deletion, and lookup in the hash table require more than constant time.

Independent of the usage of candidate lists, solution construction may be speeded up when using static heuristic information by precomputing the values of ${\tau_j}^{\alpha} \cdot {\eta_j}^{\beta}$, $\forall c_j \in \mathscr{C}$, as all ants use the same total values in Eqs. 3 and 5, and each ant's partial solution only affects which precomputed values are considered at each construction step. If an adaptive heuristic information is used, the total values would depend on the partial solution of each ant; hence, they cannot be precomputed in this way.

Some ACO algorithms incorporate a type of negative learning. The first approaches can be traced back to Maniezzo [92], and Cordon et al. [24] used it to implement pheromone reduction for components that lead to lower-quality solutions during the solution construction process. Other variants of negative pheromone learning were developed in subsequent work [98, 114, 124, 154], which showed decent results compared to relying solely on the usual positive pheromone learning.

Most ACO algorithms construct solutions starting from an empty solution. However, a few proposals have studied the possibility of starting from partially constructed solutions with the goal of partially destroying a very good solution and reconstructing from it a, hopefully better, new one. This is the same concept applied by iterated greedy [?]. Examples of this kind of ACO algorithm are ACO algorithms using external memory [1], iterated ants [152], and cunning ants [151]. Considering the effectiveness of the solution construction, these methods are useful as they avoid the most expensive construction steps that happen when solutions are empty or almost empty. In the Enhanced ACO [52], solution construction is guided by the global-best solution, the best solution found since the start of a run of the algorithm, by adopting the choice done for that solution with a fixed probability at each construction step. This approach somehow introduces a guided perturbation into the best solution found so far and, thus, a straightforward way of increasing the exploitation of good solutions in ACO. It has also the advantage of resulting in faster solution construction as no probabilistic choices among several candidate components need to be done; thus, it is deemed to be useful particularly for tackling large instance sizes.

Typically, construction rules take into account the pheromone trails associated with only single solution components. However, for specific problems, it may be useful to include also the pheromone associated with other solution components. An example is scheduling problems, for which the *pheromone summation rule* was proposed [102]. Its goal is to avoid that jobs are scheduled in a position too far away from positions where they occur in high-quality solutions.

Global Pheromone Update

As mentioned above, the pheromone trails are modified during the run of an ACO algorithm in order to bias the construction of new solutions. Pheromone update usually consists of two complementary steps: pheromone evaporation and pheromone deposition. The general idea behind these two steps is to bias the

pheromone trails so as to favor the construction of high-quality solutions. This general idea can be implemented in a number of different ways.

Pheromone evaporation decreases the pheromone values by some factor with the goal of reducing the effect of previous pheromone depositions and, in this way, help to forget previous poor decisions. Pheromone evaporation itself can be applied, in principle, to some or all the pheromone trails. It can be described as

$$\mathscr{T}_{\text{new}} = \text{evaporation}(\mathscr{T}_{\text{old}}, \rho, S^{\text{eva}}), \tag{6}$$

where $\rho \in (0, 1)$ is a parameter called evaporation rate and S^{eva} denotes the set of solutions that are selected for evaporating the pheromone trails. Most of the existing ACO algorithms do not make use of the solutions in S^{eva} to implement the pheromone evaporation and simply reduce the pheromone trail value of all solution components by the same factor:

$$\tau_j = (1 - \rho) \cdot \tau_j \qquad \forall \tau_j \in \mathscr{T}. \tag{7}$$

A value of $\rho = 1$ would mean that pheromone trails are reset at each iteration of the algorithm, and, thus, no learning occurs. A value of $\rho = 0$ would mean that no evaporation occurs, which would result in an unlimited accumulation of pheromones. For intermediate values of ρ, the amount of pheromone decreases geometrically as a function of this parameter.

In some ACO algorithms, the global pheromone evaporation only applies to some specific solution components. For example, in ACS, pheromone evaporation affects only the solution for which pheromone is deposited in the current iteration:

$$\tau_j = (1 - \rho) \cdot \tau_j \qquad \forall \tau_j \mid c_j \in s^{\text{eva}}, \tag{8}$$

where $\forall \tau_j | c_j \in s^{\text{eva}}$ are all the pheromone trails associated with solution components that occur in the solution s^{eva} chosen for pheromone deposition (see also below). Equation 6 is also general enough to include the pheromone evaporation applied in population-based ACO [60], where the pheromone trails are defined as a function of the solution components occurring in a set of candidate solutions and where pheromone evaporation corresponds to the reduction of the amount of pheromone that is associated with components of the solutions that leave this set of candidate solutions.

Evaporation has the effect of slowing down the convergence of the algorithm as it reduces the probability of constructing again solutions previously constructed. However, while pheromone deposition selectively reinforces the pheromone trails associated with some solution components, pheromone evaporation has the effect of decreasing the probability of selecting those solution components less recently reinforced, thus allowing the algorithm to focus on the most recently found solutions and to "forget" previous pheromone depositions.

Pheromone deposition consists in increasing the pheromone values of a few selected solution components. These solution components belong to one or more

solutions previously constructed by the ACO algorithm. In a general form, the pheromone deposition can be written as

$$\tau_j = \tau_j + \sum_{s_k \in S^{\text{upd}} | c_j \in s_k} w_k \cdot F(s_k), \tag{9}$$

where $S^{\text{upd}} \subseteq S^{\text{eva}}$ is the set of solutions chosen to deposit pheromones, w_k is a weight that is associated with solution $s_k \in S^{\text{upd}}$, and $F(s_k)$ is a function that is nondecreasing with respect to the quality of the solution s_k—that is, whenever $f(s_i) < f(s_l)$ in the minimization case, then it follows that $F(s_i) \geq F(s_l)$. The amount $w_k \cdot F(s_k)$ therefore corresponds to the amount of pheromone that is deposited by a solution s_k.

The solutions used for updating the pheromone trails (S^{upd}) have a strong influence in the behavior of an ACO algorithm. Ant system (AS) selects all solutions constructed in the latest iteration. In contrast, there are alternative methods that consider one single solution for the pheromone deposition, e.g., the iteration-best update (ib-update), which uses the best solution from those generated in the most recent algorithm iteration; the global-best update (gb-update), which uses the best solution found since the start of the algorithm; and the restart-best update (rb-update), which uses the best solution found since the pheromones were reinitialized (see also below on pheromone reinitialization). Intermediate alternative methods would define a set of candidate solutions that may comprise a number of the best solutions of the latest algorithm iteration and solutions such as the global-best or the restart-best ones and use these to deposit pheromone. The particular set of solutions that is chosen to deposit pheromone has a direct effect on the speed of convergence and possible stagnation behavior of the algorithm, which occur when the pheromone trails have converged and the same solutions are constructed over and over again. The gb-update provides the fastest convergence but may more easily lead to such stagnation, while allowing all candidate solutions to deposit pheromones may delay the convergence of the algorithm [39].

In addition to the choice of solutions that form S^{upd}, the amount of pheromone deposited by these solutions also has a direct impact on the search behavior of ACO algorithms. A typical setting in ACO algorithms is to make the amount of pheromone deposited inversely proportional to the objective function value (in the minimization case), that is, to set $w_k \cdot F(s_k) = 1/f(s_k)$. This is the case, for example, for MMAS [139]. (Instead of making the amount of pheromone deposited a function of the quality of the solution, one may also deposit a constant amount of pheromone if the bias toward good solutions is ensured by choosing ib-update, gb-update, or similar biases toward the best candidate solutions.) Various other ACO algorithms add additional weighting factors that may be dependent or not on the quality of the solution relative to others. ACS uses a pheromone update by setting $w_k \cdot F(s_k) = \rho/f(s_k)$, where ρ is the evaporation rate. AS initially used a setting of $w_k \cdot F(s_k) = Q/f(s_k)$, where Q is a parameter; however, in many recent implementations, one simply uses $Q = 1$. Various ACO algorithms use an unequal

weighting for the amount of pheromone deposited. For example, RAS [22] makes the weight dependent on the rank of a solution in the current algorithm iteration by choosing $w_k \cdot F(s_k) = \max\{0, w - r\}/f(s_k)$, where $w = |S^{\text{upd}}|$ is the number of solutions that deposit pheromone after each iteration (this parameter is called *rasrank* in section "ACOTSP/ACOQAP: A Unified Framework of ACO Algorithms for the TSP and QAP") and r is a solutions' rank in the current iteration. The largest amount of pheromone ($w/f(s_k)$) in RAS is assigned to the global-best solution s_{gb}. This corresponds to a choice where S^{upd} comprises the global-best solution and the $w - 1$ best-quality solutions of the current iteration. In EAS [34, 40, 42], the usual AS deposition rule is followed, that is, all solutions generated in the current iteration deposit pheromone; in addition, the global-best solution deposits an amount of pheromone $w_{\text{gb}} \cdot F(s_{\text{gb}}) = Q \cdot m_{\text{elite}}/f(s_{\text{gb}})$, where Q is the same parameter as for AS and m_{elite} is the multiplicative factor that increases the weight of the global-best solution. In BWAS [24], not only the global-best solution deposits pheromone, but also further evaporation is applied, following Eq. 7, to the pheromone trails associated with the solution components that appear in the worst solution of the current iteration and that do not appear in the global-best one.

Pheromone Update Schedule

As said above, the choice of the solutions that deposit pheromone has a strong impact on the search behavior of ACO algorithms. The main goal of pheromone update schedules is to adapt the choice of the solutions that deposit pheromone during the ACO algorithm run. In the simplest case, this is done following a predefined schedule. The first algorithm to make such a predefined schedule explicit is MMAS in its application to the TSP [132, 139] (related ideas are also discussed in [100]). In particular, when local search is applied, the central idea is to shift from a frequent use of the ib-update, which is used by default, toward an increasing frequency of the gb-update (or an rb-update). This shift has the effect of changing the search behavior from a more explorative one toward a more exploitative one that searches for new, improving solutions around the best-so-far ones. When used after the reinitialization of the pheromone trails (see section "Pheromone Reinitialization"), the schedule can also switch in the pheromone deposit between ib-update, rb-update, and gb-update. In particular, if the restart-best solution is used every iteration and no new restart-best solution is found for a number of iterations, the schedule switches to gb-update. As a result, the pheromone trails will implicitly interpolate between the restart-best solution and the global-best solution in a way that resembles the ideas underlying path relinking [55].

The update schedule may have a critical impact on the performance of an ACO algorithm, because it determines the balance between speed of convergence and exploration capabilities. It is also very sensitive to the termination criteria: An update schedule that performs well for long computation times may converge too slowly if the algorithm is terminated much earlier.

Initialization of Pheromones

Two alternative ways of initializing the pheromone trails have been proposed: either using a very small initial value (ACS and BWAS) or using a rather large value (MMAS), where small and large are relative to the amount of pheromone deposited in the global pheromone update. A small value results in a rather quick bias toward the best solutions, while a large value results in a more explorative initial search phase (though depending also on other parameters such as the evaporation rate). Neither the original AS (nor EAS and RAS) specify how the pheromones are initialized and leave it open as a parameter τ_0 to be specified by the user.

Pheromone Reinitialization

It has been shown that resetting the pheromone values back to their initial value may help in long runs by increasing the exploration of the search space [139]. This procedure is often called *restart*, since it is equivalent to restarting the run, although the information about the global-best solution found is kept. While most ACO algorithms do not implement such restarts, restarting has been shown to be very effective for problems such as the TSP and the QAP where the use of strong local search algorithms leads to fast convergence.

MMAS was the first ACO algorithm to employ pheromone reinitialization to avoid stagnation. In particular, MMAS computes a measure, called branching factor, of the potential alternatives encoded in the pheromone trails. When the branching factor goes under a certain threshold value (close to 1), pheromone values are reset to their maximum value ($\tau_{\max}$).

Another ACO algorithm that uses pheromone reinitialization is best-worst ant system (BWAS) [24], which reinitializes the pheromone values to τ_0 whenever the distance, in the decision space, between the global-best solution and the worst solution found in the last iteration falls under a threshold value.

In order to avoid very frequent restarts, there is often a "grace period" when no restarts are allowed; for example, a certain number of iterations since the last restart-best solution was found.

Local Pheromone Update

Local pheromone update is a mechanism that works during solution construction. It modifies the amount of pheromone associated with solution components that have just been chosen by the ants. The first such mechanisms were studied during the design of AS in the so-called ant-density and the ant-quantity models [23,34,41] but were abandoned due to their poor performance. In later research, local pheromone update to diminish the pheromone trails associated with chosen solution components was explored in the design of the ACS algorithm [38]. In particular, ACS uses local pheromone update following

$$\tau_j = (1 - \xi) \cdot \tau_j + \xi \cdot \tau_0 \qquad \forall \tau_j \mid c_j, \tag{10}$$

where c_j is the solution component just selected by an ant during solution construction, $\xi \in (0, 1)$ is a parameter controlling the strength of the local update, and τ_0 is the initial pheromone value, which is set to a very small value, much lower than the expected amount of pheromone deposited in the global pheromone update. The effect of this update rule in ACS is to increase the search exploration during construction making used solution components less attractive.

Although the local pheromone update is very effective when combined with the other characteristics of ACS, it is difficult to incorporate it on its own to other ACO algorithms, because it relies on a strong gb-update and a lack of pheromone evaporation.

An important implementation detail is whether the solutions are constructed in *parallel*; that is, each ant chooses one solution component at a time, or *sequentially*, that is, each ant completes its own solution before the next one starts constructing their own. The different construction schemes may introduce differences in the effect of the local pheromone update.

Pheromone Limits

MMAS introduced the concept of explicit pheromone limits that restrict the possible values the pheromone strength can take to the range $[\tau_{\min}, \tau_{\max}]$. The goal is to prevent stagnation, which occurs when the pheromone values of some solution components are so low that there is no possibility the component will ever be selected (or alternatively, the pheromone values of some solution components are so high that there is no possibility that any other component will ever be selected).

The original proposal showed how to adapt the limits within a run of MMAS. In practice, however, the upper pheromone limit seems to be less important as the maximum amount of pheromone possible on any solution component is already limited by pheromone evaporation.

The definition of $\tau_{\min}$ and $\tau_{\max}$ is problem specific. In general, $\tau_{\max}$ should be an estimation of the pheromone added by the optimal solution. In the TSP, for instance, it corresponds to $\tau_{\max} = \frac{1}{\rho \cdot f(s_{\text{opt}})}$; however, since the optimal solution s_{opt} is not known, the global-best solution s_{gb} is used instead.

The main parameter controlling the pheromone limits is p_{dec}, which is the probability that an ant chooses exactly the sequence of solution components that reproduces the best solution found so far. The value $\tau_{\min}$ is given by

$$\tau_{\min} = \frac{\tau_{\max} \cdot (1 - \sqrt[n]{p_{\text{dec}}})}{n' \cdot \sqrt[n]{p_{\text{dec}}}} \tag{11}$$

where n' is an estimation of the number of solution components available to each ant at each construction step. This value often corresponds to $n/2$ (or to half the size

of the candidate lists if they are used). Nonetheless, in some implementations of MMAS, and specially when local search is used to improve the solutions constructed by the ants, the above computation may be simplified to $\tau_{\min} = \frac{\tau_{\max}}{2 \cdot n}$.

ACS also implicitly uses pheromone trail limits. Due to the way its local pheromone update rule is defined, the pheromone trails can never fall under the value τ_0, and they cannot grow above $1/F(s_{\text{gb}})$, thus limiting implicitly the pheromone trails to the range $[\tau_0, 1/F(s_{\text{gb}})]$.

Local Search

In many combinatorial problems, the use of local search algorithms is essential for obtaining competitive results. Local search algorithms start from an initial solution and iteratively apply small changes to it, as defined by a *neighborhood operator*, in order to find an improved solution. Two of the most basic local search algorithms are best improvement and first improvement, which replace the current solution with the best- and first-improving solution found in the neighborhood, respectively. These algorithms stop if, after examining the whole neighborhood of the current solution, no improved solution can be found, thus stopping at a local optimum. Other local search algorithms that do not necessarily stop at a local optima, such as tabu search or simulated annealing, have also been combined with ACO algorithms [90, 134].

When combined with an ACO algorithm, local search is typically applied to the solutions constructed by the ants. The decision about which ant applies local search to its solution depends on the effectiveness and efficiency of the specific local search [52]. If the local search is relatively fast and effective, it may be worth applying it to all ants' solutions. Otherwise, restricting it either to the iteration-best ant or to a candidate set of promising solutions may save computation time while still giving a chance to further improve the best solution found by the ants.

ACO Algorithms as Instantiations of the ACO Metaheuristic

Given the algorithmic components discussed above, then ACO algorithms from the literature can be described as combinations of specific components. (This list includes only a small subset of the ACO algorithms and algorithmic components proposed in the literature. A more comprehensive component view of ACO is left for further work.)

In particular, AS uses Eq. 3 for solution construction for all ants, Eq. 7 for pheromone evaporation, and Eq. 9 for the pheromone deposition with all ants using the same weight $w_1 = \cdots = w_m$, where m is the number of ants. EAS, RAS, BWAS, and MMAS all use the same solution construction rules and the same evaporation mechanism as AS, but they differ from AS in the way pheromone is deposited; also, they add new algorithmic components. In particular, the pheromone deposition of EAS uses the global-best solutions with a weight m_{elite}. Similarly, the only difference between AS and RAS is the way pheromone is deposited, which also

uses the global-best solution and weights defined according to a parameter called *rasrank*, already explained in section "Global Pheromone Update." The pheromone deposition of BWAS also uses the global-best solution and, in addition, applies further evaporation to pheromone trails associated with solution components that appear in the worst solution of each iteration and not in the global-best one. In addition, BWAS performs pheromone reinitialization depending on the average distance between the iteration-worst solution and the global-best one. MMAS adds several features: It enforces minimum and maximum limits to pheromone values and uses an update schedule to switch between iteration-best, global-best, and restart-best deposition and pheromone reinitialization according to branching factor, and pheromones are initialized to a high initial value. MMAS variants that use the pseudorandom proportional rule (Eq. 5) have also been considered in the literature [138]. Finally, ACS is the ACO algorithm that structurally differs the most from AS: It adds the pseudorandom proportional rule to AS solution construction, pheromone is deposited using the global-best solution, and evaporation is performed at the same time on those values updated during pheromone deposition (Eq. 8). Moreover, further evaporation is performed during solution construction (local pheromone update, section "Local Pheromone Update") by each ant.

ACOTSP/ACOQAP: A Unified Framework of ACO Algorithms for the TSP and QAP

We present in this section a software framework that unifies the implementation of several ACO algorithms found in the literature. The code is publicly available at http://iridia.ulb.ac.be/aco-tsp-qap/. The implementation aims at generality, by clearly separating the general components of the ACO metaheuristic from the problem-specific components that vary from one problem to another. In this chapter, we present an implementation for the TSP and the QAP. However, it is possible to extend the framework to other combinatorial optimization problems with far less effort than an implementation from scratch.

In its current form, the software implements AS, MMAS, EAS, RAS, ACS, and BWAS. In addition to the usual ACO parameters (α, β, ρ, m, ξ, etc), it also allows combining various algorithmic components by setting specific parameters. In particular:

- Pheromone limits as defined by MMAS can be enabled or disabled independently of the algorithm chosen (except for ACS).
- Pheromone reinitialization (restart) can also be enabled or disabled independently of the algorithm. In particular, when more than res_{it} iterations have passed since the last restart-best solution was found, a restart condition is tested. This restart condition may be the branching factor going under a threshold value (res_{bf}), as in MMAS, or the distance between the global-best and the iteration-worst ants going under a threshold value (res_{dist}). In addition, a setting of *always* (*never*) means that the restart condition is always (never) satisfied.

- Any algorithm may use the pseudorandom proportional rule of ACS (Eq. 5) by simply using a value $q_0 > 0$.

In the problem-specific part, the implementation for the TSP follows what is provided by the ACOTSP software [133], that is, heuristic information, candidate lists, and various types of local search, including three-opt first improvement with don't-look-back bits (*dlb-bits*) and nearest neighborhood lists (*nnls*). The part corresponding to the QAP does not implement heuristic information nor candidate lists, since these techniques have not proved useful for the QAP. It does implement, however, various local search algorithms, such as the fast two-exchange best- and first-improvement local searches proposed by Taillard [146] and the robust tabu search proposed in the same paper.

The update schedule (section "Pheromone Update Schedule") is implemented as specific to each problem and ACO algorithm; however, it would be straightforward to extend the update schedule to all ACO algorithms. In the case when MMAS is used together with local search, the user can control the frequency with which the restart-best (or global-best) solution, instead of the iteration-best one, is used to update the pheromone trails, by means of a parameter called *schedule length* (*slen*). Higher values mean longer emphasis on the iteration-best solution, thus possibly increasing exploration at the expense of faster convergence.

In the two following sections, we show how to find a good performing ACO algorithm for a specific class of instances of the TSP and the QAP by automatically configuring the proposed ACO framework. In particular, there is, for each problem, a set of training instances and another set of testing instances. The automatic configuration tool irace [91] is used here to find a parameter configuration given the set of training instances. Finally, this parameter configuration is compared with the default configuration for each problem by evaluating both configurations on the set of test instances.

The implementation of the proposed ACO framework assigns default values, consistent with the literature, to certain parameters depending on the particular ACO algorithm chosen. These default values are documented in Table 1 for completeness. For the purpose of automatic configuration, we consider a single default configuration based on MMAS. Table 2 summarizes, for each problem, the parameters of the proposed ACO framework, their domain, and default values considered in the automatic configuration experiments described below. Although some parameters are still exclusive to a particular ACO algorithm (e.g., ξ can only be used together with ACS), the framework allows using components from one ACO algorithm in others (e.g., the restart mechanism of BWAS can be enabled for any algorithm by setting parameter *restart* to the value "*distance*").

Finding a Better ACO Configuration for the TSP

In the case of the TSP, we consider random Euclidean TSP instances, in particular 50 instances of each size {1000, 1500, 2000, 2500, 3000} for training and other 50

Table 1 Default parameter settings for each ACO algorithm from the literature. Parameters that are available only for particular ACO algorithms are described under the table. The default parameter configuration used in the experiments is based on MMAS (see Table 2)

	m	ρ				
	TSP/QAP	TSP/QAP	q_0	*ph-limits*	*Restart*	Restart parameters
AS, EAS, RAS	25/5	0.5/0.2	0.0	No	Never	
BWAS	25/5	0.5/0.2	0.0	No	Distance	$res_{\text{dist}} = 0.05$
MMAS	25/5	0.2/0.2	0.0	Yes	Branch-factor	$res_{\text{bf}} = \begin{cases} 1.00001 & \text{(TSP)} \\ 1.1 & \text{(QAP)} \end{cases}$
ACS	10/5	0.1/0.2	0.98	No	Never	

$m_{\text{elite}} = n$ when using EAS, $rasranks = 6$ when using RAS, $\xi = 0.1$ when using ACS, $slen = 250$ (TSP) or 20 (QAP) when using MMAS

instances of each size for testing. A single run of irace has a maximum budget of 25,000 runs of the ACO software, and each run is stopped after 60 CPU seconds. Since the goal is to investigate whether it is possible at all to find a better ACO algorithm than the default one from the literature and in order to speed up the process, irace starts from the default configuration for the TSP (Table 2) as an initial solution. After irace finishes, the configuration found is evaluated on the test instances. In particular, we select two configurations found by irace that improve over the default (see Table 3): one is a tuned variant of MMAS and the other is a tuned variant of ACS. These two configurations use a higher value of β and a stronger evaporation ρ than the default configuration. Since $q_0 > 0$, this MMAS variant uses the pseudorandom proportional rule from ACS, which is a MMAS variant that has also been explored previously in the literature [138]. It is also interesting that the restart strategy is typically much more aggressive than the default, with a much larger threshold branching factor, which results in more frequent restarts, only limited by the grace period (res_{it}). Other parameters, such as the size of the candidate list (cl), remain mostly unchanged from the default configuration.

Figure 2 compares the results obtained by these two tuned configurations with those obtained using the default configuration on the test instances. We ran each configuration once on each test instance up to 60 CPU seconds, and we computed the relative percentage deviation (RPD), with respect to the optimal solution, of the best solution found throughout the run. Both plots show that the solutions obtained by the tuned configurations are, in most instances, better than those obtained by the default configuration, and the differences are specially large for the largest instances. These observations are further confirmed by comparing the mean and standard deviation of the values obtained by each configuration (Table 4). In particular, a 95% confidence interval around the mean difference between the default and the tuned configurations does not contain the value zero, which indicates that the observed differences are statistically significant.

Table 2 Domains and default values of the parameter settings of the ACO framework considered in the automatic configuration experiments for the TSP and the QAP. A value of n/a means that the parameter does not exist for the corresponding problem. A value of "–" means that the parameter has no value in the default configuration (because it depends on another parameter setting that is not enabled by default). The list of parameters enabled only for certain values of other parameters is given under the table

	TSP		QAP	
Parameter	Domain	Default	Domain	Default
Algorithm	{AS, EAS, RAS, ACS, MMAS, BWAS}	MMAS	{AS, EAS, RAS, ACS, MMAS, BWAS}	MMAS
m	[1, 500]	25	[1, 10]	5
α	(0.0, 5.0)	1.0	(0.0, 5.0)	1.0
β	(0.0, 10.0)	2.0	n/a	n/a
ρ	(0.01, 1.0)	0.2	(0.01, 1.0)	0.2
q_0	(0.0, 1.0)	0.0	(0.0, 1.0)	0.0
cl	[5, 50]	20	n/a	n/a
ξ	(0.01, 1.0)	–	(0.01, 1.0)	–
rasrank	[1, 100]	–	[1, 100]	–
m_{elite}	[1, 750]	–	[1, 750]	–
p_{dec}	(0.001, 0.5)	–	(0.001, 1)	0.005
ph-limits	{Yes, no}	Yes	{Yes, no}	Yes
slen	[20, 500]	250	[5, 250]	20
restart	{Never, branch-factor, distance, always}	Branch-factor	{Never, branch-factor, distance, always}	Branch-factor
res_{bf}	(1.0, 2.0)	1.00001	(1.0, 2.0)	1.1
res_{dist}	(0.01, 0.1)	–	(0.01, 0.1)	–
res_{it}	[1, 500]	250	[1, 50]	5
localsearch	{None, 2-opt, 2.5-opt, 3-opt}	3-opt	None, best-2-opt, short-tabu-search, long-tabu-search	Best-2-opt
dlb-bits	{Yes, no}	Yes	{Yes, no}	No
nnls	[5, 50]	20	n/a	n/a
rasrank	When *algo* = RAS			
m_{elite}	When *algo* = EAS			
ξ	When *algo* = ACS			
p_{dec}	when *ph-limits* = yes (and for the TSP only if also *restart* = never)			
ph-limits	When *algo* $\neq$ ACS			
slen	When *algo* = MMAS			
res_{bf}	When *restart* = branch-factor			
res_{dist}	When *restart* = distance			
res_{it}	When *restart* $\neq$ never			
dlb-bits, *nnls*	When *localsearch* $\neq$ *none*			

Table 3 Best configurations found by irace for the TSP

algo	*m*	α	β	ρ	q_0	ξ	*cl*	*nnls*	*ph-limits*	*slen*	*restart*	res_{it}
ACS	28	3.07	5.09	0.32	0.53	0.21	22	9	–	–	Branch-factor ($res_{\text{bf}} = 1.74$)	212
MMAS	40	0.94	4.11	0.72	0.14	–	18	12	Yes	191	Branch-factor ($res_{\text{bf}} = 1.91$)	367

Common parameters: *localsearch* = 3-opt + dlb-bits

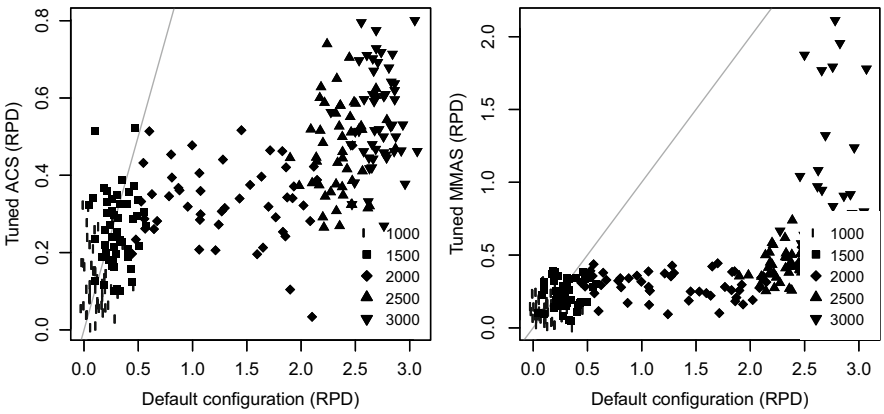


Fig. 2 Comparison of tuned vs. default configurations of ACOTSP on the test TSP instances. Tuned ACS configuration (*left*) and tuned MMAS configuration (*right*). Each point gives the relative percentage deviation from the optimal solution on one particular instance

Table 4 Results on the test TSP instances obtained by the default configuration and two tuned configurations found by irace. Objective function values are given as relative percentage deviation from the optimal solution. ΔCI denotes the 95% confidence interval around the mean difference between the default configuration minus the configuration in that column (thus, positive values would denote that the default configuration is worse)

	Default	Tuned ACS	Tuned MMAS
Mean	1.37	0.35	0.38
Sd	1.08	0.18	0.32
ΔCI		[0.904, 1.14]	[0.875, 1.1]

Finding a Better ACO Configuration for the QAP

In the case of the QAP, we consider two different instance sets: RR, where the distance matrix entries correspond to the Euclidean distances between points randomly generated in a square of side length 300 and the flow matrices are randomly generated according to a uniform distribution, and RS, where the distance matrix is generated as in the previous case and the flow matrix shows a structure

Table 5 Best configurations found by irace for the QAP

	algo	*m*	α	ρ	q_0	*dlb-bits*	*ph-limits*	*slen*	*restart*	res_{it}
RR	MMAS	6	0.324	0.29	0.062	Yes	No	153	Distance ($res_{\text{dist}} = 0.051$)	22
RS	MMAS	4	0.164	0.342	0.284	Yes	No	170	Branch-factor ($res_{\text{bf}} = 1.822$)	40

Common parameters: *localsearch* = best-2-opt

similar to that found in real-world QAP instances. Within each set, we consider 100 instances, and we use half of them for training and the other half for testing the ACO configurations generated. For tuning the ACO framework, we consider a similar setup as in the TSP case, that is, each run of irace has a maximum budget of 25,000 runs of the ACO software, and each run stops after 60 CPU seconds.

For each of the two training sets, RR and RS, Table 5 reports the best configuration found in three independent runs of irace. These two configurations are fairly similar but quite different from the default one. First of all, it is somehow surprising that none of them uses pheromone limits, despite this being one of the key characteristics of MMAS. This could be due to the stronger restart strategies, which do not allow pheromone values to reach the limits and, thus, enforcing the limits adds an overhead that never pays off. Also, the schedule-length (*slen*) value is more than five times higher than the default, which implies a much higher exploitation of the iteration-best solution.

The two configurations of Table 5 are run on each test instance for 60 CPU seconds. Since for these instances the optimal objective values are not known, the relative percentage deviation (RPD) is computed with respect to a reference solution, for each instance, obtained by running the default configuration ten times with a time limit of 600 CPU seconds and keeping the best solution found.

Figure 3 compares the above configurations with the default on the test instances. The improvements of the tuned configurations over the default are not so clear for the QAP as they were for the TSP. In general, most RPD values are extremely small, which indicate that the variations over the best-known solutions are very small for all configurations. In both cases, there are a few instances where the default configuration obtains better RPD values than the tuned one. The statistical analysis in Table 6 shows that the mean difference between the default and tuned configuration for the RS instances is very small but still statistically significant (since the confidence interval does not contain zero), whereas for the RR instances, the null hypothesis of zero difference cannot be rejected (since the corresponding CI contains the value zero). The latter result is likely caused by the relatively large RPD values obtained by the tuned configuration on three particular instances, as shown on the left side of Fig. 3. These results could provide the motivation for exposing additional algorithmic components to automatic configuration, such as

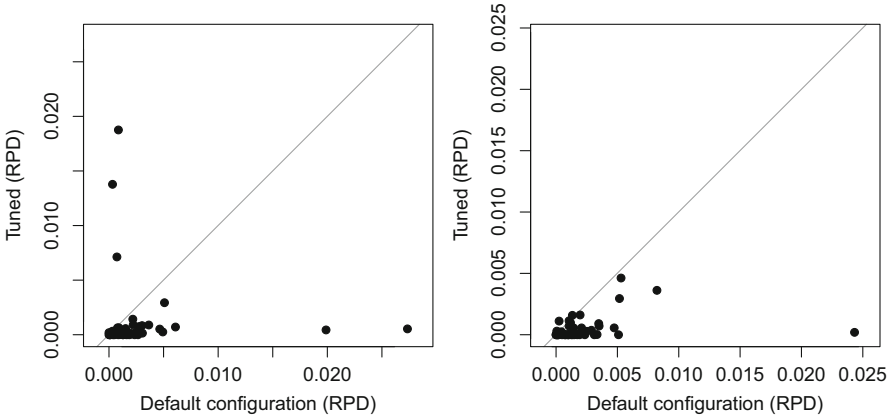


Fig. 3 Comparison of tuned vs. default configurations on the QAP test instances. RR instances (*left*) and RS instances (*right*). Each point gives the relative percentage deviation from the optimal solution on one particular instance

Table 6 Results on the QAP test instances obtained by the default configuration and the tuned configurations found by irace. CI denotes the 95% confidence interval around the mean difference between the default configuration minus the configuration in that column (thus, positive values would denote that the default configuration is worse)

RR instances		
	Default	Tuned ACO
Mean	0.00241	0.00109
Sd	0.00466	0.00335
ΔCI		[−0.000352, 0.00299]

RS instances		
	Default	Tuned ACO
Mean	0.00210	0.000485
Sd	0.00364	0.000941
ΔCI		[0.00062, 0.00261]

the precise update schedule of MMAS, which is known to be very sensitive to the instance characteristics [139]. Extending the proposed framework with new algorithmic components from other ACO algorithms may also further improve the results on the QAP.

Applications of ACO to Other Problem Types

Although the ACO metaheuristic was primarily designed for tackling single-objective combinatorial optimization problems, its main ideas have been extended to tackle other types of problems. In this section, we provide an overview of such problems and how ACO has been adapted to deal with their particular characteristics. Comprehensive reviews of ACO applications are available in the literature [39, 140].

Continuous Optimization Problems

In continuous optimization problems, the domain of the decision variables is the set of real numbers or a subset thereof. In some problems, the simplest approach is to discretize the real-valued domain, which would allow the direct application of ACO. This was, for example, the approach followed by Korb et al. [76] when applying ACO to the protein–ligand docking problem, where a discrete ACO algorithm was combined with a local search for continuous domains.

In many problems, this discretization approach is not feasible, and the problem must be tackled in the continuous domain. A first group of algorithms for continuous optimization is inspired by the behavior of some ant species [17,43,107]. However, these algorithms diverge from the basic framework of the ACO metaheuristic, for example, by requiring the direct communication among ants instead of a pheromone structure. Therefore, they should rather be considered a separate class of algorithms.

A second group of algorithms directly translates the ideas underlying the ACO metaheuristic to the continuous domain. For example, Socha and Dorigo [130] replace the discrete pheromone distributions by Gaussian kernel functions that take the role of the pheromone model that is updated by the ants. Other similar approaches are found in [128, 150]. More recently, Liao et al. [80, 82] proposed a unified model of various ACO algorithms for continuous optimization problems. Their model allows the automatic configuration of new ACO algorithms for continuous optimization very much in the line of what is presented in section "ACOTSP/ACOQAP: A Unified Framework of ACO Algorithms for the TSP and QAP." They show that their final configured continuous ACO algorithms are competitive to other state-of-the-art algorithms for continuous optimization.

These approaches have also been extended to mixed-variable (continuous and discrete) problems [81, 129] by using appropriate ways of representing pheromone trails when handling discrete and continuous variables.

Multi-objective Problems

In many real-world problems, a solution is evaluated according to multiple, conflicting criteria (objectives). If there is a priori information about the importance of each objective, the objectives can be aggregated according to a preference model, and solutions are compared in this way. In the absence of such preference model, one can only say that a solution is better than another if the former is not worse in all objectives and better in at least one of them. Thus, the goal becomes to find (an approximation of) the Pareto set, i.e., the set of solutions not dominated by any other feasible solution [44, 131].

There are several proposals in the literature on how to apply the ACO metaheuristic to multi-objective combinatorial optimization problems. A few of these proposals assume an order of importance among the objectives [51]; however, most proposals

attempt to approximate the Pareto set [5, 53, 88, 126]. As pointed out by López-Ibáñez and Stützle [87], many of these multi-objective ACO (MOACO) algorithms share similar algorithmic components combined in different ways. For example, Pareto ACO [31], BicriterionAnt [72], and COMPETants [30] use a different pheromone set $\mathscr{T}$ (a matrix, in their case) per objective, whereas MACS [12] and mACO-3 [2] use a single $\mathscr{T}$. Some papers have specifically compared both design choices [2, 90]. Moreover, there are basically two ways of updating the pheromone trails in MOACOs: either selecting the best solution for each objective, as done in COMPETants, Pareto ACO, and mACO-4 [2], or selecting (a subset of) all nondominated solutions, as done in BicriterionAnt, MACS, and mACO-3. Finally, a few MOACO algorithms make use of multiple colonies, where a colony is understood as a group of ants that construct solutions only according to the pheromone trails associated with their colony. The pheromone trails of each colony may correspond to multiple pheromone matrices (which are aggregated during solution construction), and colonies may exchange solutions [72]. Nonetheless, it is straightforward to define multi-colony variants of most MOACO algorithms [87].

Most of the MOACO algorithms were proposed for bi-objective problems. In the bi-objective case, the results presented in the literature indicate that the use of one pheromone matrix per objective and of multiple colonies, each of them with their own pheromone matrices, is necessary for MOACO algorithms to perform well on different regions of the objective space [53, 87, 88, 156, 157]. When applied to problems with more than two objectives, this approach quickly becomes too computationally expensive. To address this issue, MOACO algorithms based on population-based ACO have been proposed [4, 61]. More recently, Falcón-Cardona and Coello Coello [46] proposed a new indicator-based many-objective ant colony optimizer and Rivera et al. [125] proposed an ant colony algorithm based on interval outranking that also addresses many-objective problems.

Dynamic Problems

Dynamic problems are those in which some information about the problem changes or becomes available after the start of the algorithm. This is the typical case in network routing, where ACO algorithms specifically designed for these problems have achieved notable results [27, 28, 149]. These algorithms differ significantly from the classical ACO algorithms for combinatorial optimization problems: Ants are launched asynchronously in a distributed way, and no global pheromone update takes place. However, there are also in the literature dynamic variants of classical combinatorial problems, such as the dynamic TSP and the dynamic vehicle routing problem (VRP), where cities may appear or disappear and the distances between them may change. ACO algorithms for the dynamic TSP [26,45,58,99], the dynamic QAP [59], and the dynamic cVRP and variants of it [33, 108, 127] follow more closely the general outline of ACO algorithms discussed in section "The ACO Algorithmic Framework." In addition, they use specific routines to modify the pheromone trails after detecting changes in the problem data or structure. A real-

world application of ACO to the scheduling of hospital nurses in Austria is described by Gutjahr and Rauner [68]. More recently, Lissovoi and Witt [83] formally analyze which type of changes in the dynamic shortest path can be tracked by a constant number of ants using a simplified MMAS variant. An overview of ACO algorithms for dynamic optimization problems is given by these two papers of Leguizamón and Alba [79] and Rezvanian et al. [123]

Stochastic Problems

In some optimization problems, either the objective function, the decision variables, or the constraints are not deterministic but subject to uncertainty or noise, and they are specified only within certain bounds or in terms of a probability distribution. The first stochastic problem to which ACO algorithms have been applied is the probabilistic TSP (PTSP), where one is given for each city a probability that it requires a visit. The goal in the PTSP is to find a tour of minimal expected length over all the cities. The first ACO algorithm for the PTSP was proposed by Bianchi et al. [16], who were using ACS. This algorithm was later improved upon by Guntsch and Branke [57] and further by Balaprakash et al. [9]. The ACO algorithms developed in that latter paper were then shown to match or even surpass in some cases state-of-the-art performance for the PTSP [10], and extensions thereof are state-of-the-art for the vehicle routing problem with stochastic customers and demands [11]. Another early ACO proposal for problems under uncertainty is the S-ACO algorithm [64], which has been the basis of later applications, for example, to the selection of optimal screening policies for diabetic retinopathy [21]. The S-ACO algorithm was later extended by Birattari et al. [18] who integrated F-Race for determining the global-best solution in an ACO algorithm for stochastic optimization and have shown positive results for this latter integration. Another notable example is the application of ACO to the VRP with stochastic demands [14]. A survey of various metaheuristics, including ACO, for stochastic combinatorial optimization problems is provided by Bianchi et al. [15].

ACO in Combination with Other Methods

In this section, we briefly overview research on combining ACO with other methods.

ACO and Tree Search Methods

Ants in ACO perform a probabilistic solution construction that can be seen as a stochastic exploration of a search tree. Thus, a natural extension of ACO is to make use of tree search techniques such as branch and bound. An example is the approximate nondeterministic tree search (ANTS) algorithm by Maniezzo [92, 93], which uses lower bounds in three different ways. First, it incorporates a lower bound

estimate as the heuristic information. Second, it prunes feasible solution components during construction if the estimated solution cost is larger than a threshold (e.g., the cost of the best solution found so far). Third, the order of assignment of solution components during solution construction is influenced by lower bound computations.

A different alternative is to integrate concepts from ACO into tree search methods. An example of this approach is Beam-ACO [19], which incorporates the use of pheromone trails into beam search. Beam search is an incomplete tree search method that keeps a set of partial solutions (the beam) and, at each iteration, selects a number of potential extensions of each of them by adding an additional solution component. These potential extensions are selected based on heuristics [115]. The number of extensions is typically larger than the size of the beam; thus, only the best extensions, according to a lower bound estimate, are kept for the following iteration. Beam-ACO executes beam search several times, replacing the deterministic solution extension of beam search with the probabilistic construction of ACO. In this way, the extension of partial solutions is biased by pheromone trails that are updated with the best solutions found in previous executions of beam search [19]. If lower bound estimates are not reliable or computationally feasible for the problem at hand, an alternative is to perform *stochastic sampling* [86], that is, to complete partial solutions by means of the construction procedure of ACO and use the cost of the complete solution (or the best of several samples) as the cost estimate of the partial solution.

The integration of ACO into Monte Carlo tree search (MCTS) [75,145] is another type of combination between tree search algorithms and ant colony optimization with which good results have been obtained [6, 8].

ACO and Exact Methods

The combination of heuristic and exact methods, sometimes called *matheuristics*, is a promising trend in optimization. Roughly speaking, there are two types of matheuristics: those that use exact methods to solve simpler versions of a problem in order to improve the solutions generated by a heuristic or provide better estimates of solution quality and those that use heuristic methods to provide initial solutions or to constrain the search space of exact methods in order to make their application feasible to large problems. (The exploitation of tree search techniques such as branch and bound or beam search discussed before can actually also be seen as such a matheuristic approach.)

An example of the first type of matheuristic is the use of constraint programming (CP) techniques [94] to help ACO focus on feasible solutions. This is particularly useful in highly constrained problems, such as scheduling or timetabling, where a major challenge is to find feasible solutions among many infeasible ones. Classical ACO algorithms do not perform satisfactorily on such overly constrained problems. Meyer and Ernst [105] use CP techniques to identify in the ants' construction procedure whether specific solution components lead to infeasible solutions. An

example of the second type of matheuristic is the work by Massen et al. [96, 97]. Their pheromone-based column generation algorithm uses an ACO algorithm to generate a heuristic set of feasible routes, from which an optimal subset that is a solution to a vehicle routing problem is selected by an exact solver. The combined algorithm is still a heuristic, because the exact solver does not consider all possible routes, but it allows applying the exact solver to problems with many feasible routes and black box constraints. Recently, a slightly different approach has been developed for the column generation using ant colony optimization [153]. Their method reports better results on the bin packing problem with conflicts compared to various state-of-the-art methods and when integrated into a branch-and-price scheme it achieves significantly lower solution times.

ACO and Surrogate Models

When the evaluation of candidate solutions is very costly in terms of computation time, usually only a small number of candidates can actually be evaluated. Costly solution evaluation arises in the case of simulation-based optimization [7] and, frequently, in industrial settings [49, 147]. Applications with very small evaluation budgets have been rarely studied in the ACO literature, and preliminary results suggest that typical ACO parameter settings are not appropriate in such context [120]. In cases with very small evaluation budget, also the use of surrogate models during optimization may prove useful [73, 110, 155]. Surrogate modeling approaches build prediction models that estimate the quality of candidate solutions, such that only the most promising candidate solutions are actually evaluated. A first approach combining surrogate models and ACO algorithms for tackling combinatorial optimization problems was studied by Pérez Cáceres et al. [120]. More recently, methods that employ a surrogate model for expensive mixed-variable optimization problems that contains a multi-surrogate-assisted selection and a surrogate-assisted local search have been proposed Liu et al. [85]. The authors of [148] consider surrogate models for checking the inherent uncertainty in a mining supply chain problem and show that the use of a surrogate model increase the performance of an ant colony optimization algorithm.

Parameter Adaptation

Good parameter settings may not only be found "offline," as done in section "ACOTSP/ACOQAP: A Unified Framework of ACO Algorithms for the TSP and QAP," but modified or adapted "online," while the algorithm is running. Good parameter settings may also be predicted according to instance features [112]. In the context of ACO, several parameters have a strong effect on the balance between search space exploration and exploitation of the best found solutions, and their proper values may additionally depend strongly on how much computational time is available to the algorithm. Parameters β and ρ are the earliest and most frequently modified

parameters in the literature [100, 103]. A significant effort has been done to define adaptive parameter choices, where the algorithm parameters are modified as a predefined function of the ACO search behavior [78, 122], or to define self-adaptive parameter adaptation schemes, where the algorithm modifies the parameters itself during execution [74, 95, 121]. A relatively new technique is the integration of machine learning techniques in ant colony optimization. Promising results have been achieved by solution prediction via machine learning [144]. A paper [141] critically reviews the works that have applied parameter adaptation to ACO algorithms. The same paper also shows that the convergence speed of MMAS can be significantly improved, without leading to early stagnation, by means of very simple prescheduled parameter variations, such as starting with a very high value of β and reducing it to a much smaller value after a number of iterations. López-Ibáñez and Stützle [89] demonstrate how such prescheduled variations can be automatically tuned for the purpose of improving the convergence speed of ACO algorithms and show that fast increments in the number of ants, fast decrements of β, and, in particular, slow decrements of q_0 produce remarkable improvements on the TSP. (The software described in section "ACOTSP/ACOQAP: A Unified Framework of ACO Algorithms for the TSP and QAP" and published alongside this chapter is also able to replicate these parameter variation schemes.) Finally, Pellegrini et al. [119] empirically showed that parameter adaptation methods proposed in the literature (excluding prescheduled variations) may often worsen the performance of state-of-the-art ACO algorithms and only improve over static parameter settings when the latter produce very poor results and when only one parameter is carefully adapted.

Conclusions

In this chapter, we have reviewed ACO algorithms from a component-wise perspective, and we have provided an example implementation of ACO algorithms according to this perspective. We have also concisely reviewed trends in ACO research on applications to problems with challenging characteristics such as time-varying problem data, multiple objectives, and stochastic data as well as algorithmic developments that combine ACO algorithms with other techniques. We did not cover in this chapter research on the parallelization of ACO algorithms, which has the goal of either speeding up the execution of a single run of an ACO algorithm or of increasing the quality of the final solution obtained within the same wall-clock time (by evaluating more solutions than would be possible without parallelization) [118]. Another important area not covered in this chapter is the ongoing work on the theoretical understanding of ACO algorithms. Early theoretical work has focused on the convergence behavior of ACO algorithms [62, 63, 135]. Later work has analyzed the dynamics of ACO behavior [101] and its relationship to other algorithms [104]. An overview of early theoretical work on ACO is given by Dorigo and Blum [36]. More recent works theoretically analyze the convergence speed of ACO algorithms [13, 32, 65–67, 71, 77, 84, 113, 143].

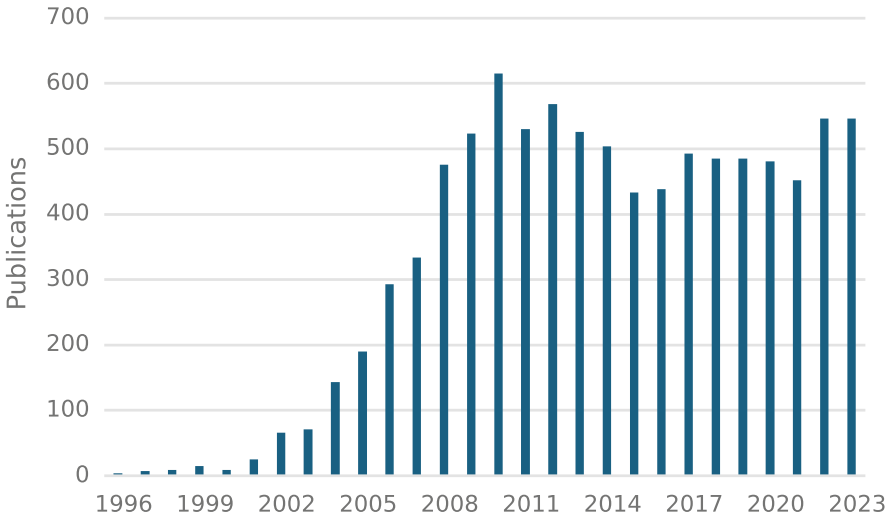


Fig. 4 Number of publications per year in the Scopus database whose titles contain the keywords "ant system," "ant colony system," "ant colony optimization," "ant colony optimisation"

These developments together with the large number of ACO algorithm variants that have been proposed in the literature and the large number of applications to a very diverse set of problems show that ACO has become a widely accepted and well-established metaheuristic. This fact is confirmed also by the number of publications that have as central topic some aspect related to ACO. Figure 4 shows the number of publications in the Scopus database that have one of the four terms "ant system," "ant colony system," "ant colony optimization," or "ant colony optimisation" in the article title. There is a very strong increase from the period 2000–2010, while after 2010, the number of publications remained at a high level. The success of ACO is also witnessed by a number of industrial applications. For example, AntOptima (www.antoptima.com) is a small company using the ACO methodology for tackling industrial problems in distribution and production management, while ArcelorMittal is using ACO algorithms in various areas relevant to steel production to improve operative production performance [29,49]. ACO is also a central topic of journals and conferences specializing in the area of swarm intelligence such as the ANTS conference series started in 1998 (http://iridia.ulb.ac.be/ants/) and the journal "*Swarm Intelligence*."

This success of ACO is due to (i) a truly original algorithmic idea inspired by a natural phenomenon, (ii) a strong versatility of the resulting algorithmic method, and (iii) a focus of ACO research on performance and a pragmatic approach trying to make it a useful technique. In the future, the ideas underlying the ACO metaheuristic promise to be of crucial importance when tackling challenging problems where aspects of constructive search, distributed information, and dynamic domains match well the inherent characteristics of ACO.

Cross-References

- GRASP
- Iterated Greedy
- Iterated Local Search
- Multiobjective Optimization

Acknowledgments The research leading to the results presented in this chapter has received funding from the European Research Council under the European Union's Seventh Framework Programme (FP7/2007-2013)/ERC Grant Agreement n°246939. Thomas Stützle and Marco Dorigo acknowledge support of the F.R.S.-FNRS of which they are research directors.

Competing Interest Declaration The author(s) has no competing interests to declare that are relevant to the content of this manuscript.

References

1. Acan A (2004) An external memory implementation in ant colony optimization. In: Dorigo M et al (eds) 4th international workshop on ant colony optimization and swarm intelligence (ANTS 2004). Lecture notes in computer science, vol 3172. Springer, Heidelberg, pp 73–84
2. Alaya I, Solnon C, Ghédira K (2007) Ant colony optimization for multi-objective optimization problems. In: 19th IEEE international conference on tools with artificial intelligence (ICTAI 2007), vol 1. IEEE Computer Society Press, Los Alamitos, pp 450–457
3. Alba E, Chicano F (2007) ACOhg: dealing with huge graphs. In: Thierens D et al (eds) Proceedings of the genetic and evolutionary computation conference (GECCO 2007). ACM Press, New York, pp 10–17
4. Angus D (2007) Population-based ant colony optimisation for multi-objective function optimisation. In: Randall M, Abbass HA, Wiles J (eds) Progress in artificial life (ACAL). Lecture notes in computer science, vol 4828. Springer, Heidelberg, pp 232–244
5. Angus D, Woodward C (2009) Multiple objective ant colony optimisation. Swarm Intell 3(1):69–85
6. Anh DT, Ha PB, Ha NT, Dzung DT, Bang BH, Hai PD (2023) A Monte Carlo tree search with ant colony optimization for inter-domain path computation problem. In: SOICT '23: proceedings of the 12th international symposium on information and communication technology. Association for Computing Machinery, New York, pp 671–678
7. April J, Glover F, Kelly JP, Laguna M (2003) Simulation-based optimization: practical introduction to simulation optimization. In: Chick SE, Sanchez PJ, Ferrin DM, Morrice DJ (eds) Proceedings of the 35th winter simulation conference: driving innovation, vol 1. ACM Press, New York, pp 71–78
8. Bae JH, Kim M, Wang BH, Lim JS (2022) Classification of lung sounds using ant colony optimization and Monte Carlo tree search hybrid algorithm. Available at SSRN: https://ssrn.com/abstract=4059536
9. Balaprakash P, Birattari M, Stützle T, Yuan Z, Dorigo M (2009) Estimation-based ant colony optimization algorithms for the probabilistic travelling salesman problem. Swarm Intell 3(3):223–242
10. Balaprakash P, Birattari M, Stützle T, Dorigo M (2010) Estimation-based metaheuristics for the probabilistic travelling salesman problem. Comput Oper Res 37(11):1939–1951
11. Balaprakash P, Birattari M, Stützle T, Dorigo M (2015) Estimation-based metaheuristics for the single vehicle routing problem with stochastic demands and customers. Comput Optim Appl 61(2):463–487

12. Barán B, Schaerer M (2003) A multiobjective ant colony system for vehicle routing problem with time windows. In: Proceedings of the twenty-first IASTED international conference on applied informatics, Insbruck, pp 97–102
13. Benbaki R, Benomar Z, Doerr B (2021) A rigorous runtime analysis of the 2-MMASib on jump functions: Ant colony optimizers can cope well with local optima. GECCO 2021. ACM Press, New York, pp 4–13
14. Bianchi L, Birattari M, Manfrin M, Mastrolilli M, Paquete L, Rossi-Doria O, Schiavinotto T (2006) Hybrid metaheuristics for the vehicle routing problem with stochastic demands. J Math Modell Algorithms 5(1):91–110
15. Bianchi L, Dorigo M, Gambardella LM, Gutjahr WJ (2009) A survey on metaheuristics for stochastic combinatorial optimization. Nat Comput 8(2):239–287
16. Bianchi L, Gambardella LM, Dorigo M (2002) An ant colony optimization approach to the probabilistic traveling salesman problem. In: Merelo JJ et al (eds) Parallel problem solving from nature, PPSN VII. Lecture notes in computer science, vol 2439. Springer, Heidelberg, pp 883–892
17. Bilchev G, Parmee IC (1995) The ant colony metaphor for searching continuous design spaces. In: Fogarty TC (ed) Evolutionary computing, AISB Workshop. Lecture notes in computer science, vol 993. Springer, Heidelberg, pp 25–39
18. Birattari M, Balaprakash P, Dorigo M (2006) The ACO/F-RACE algorithm for combinatorial optimization under uncertainty. In: Doerner KF, Gendreau M, Greistorfer P, Gutjahr WJ, Hartl RF, Reimann M (eds) Metaheuristics – progress in complex systems optimization. Operations research/computer science interfaces series, vol 39. Springer, New York, pp 189–203
19. Blum C (2005) Beam-ACO – hybridizing ant colony optimization with beam search: an application to open shop scheduling. Comput Oper Res 32(6):1565–1591
20. Blum C, Dorigo M (2005) Search bias in ant colony optimization: on the role of competition-balanced systems. IEEE Trans Evol Comput 9(2):159–174
21. Brailsford SC, Gutjahr WJ, Rauner MS, Zeppelzauer W (2006) Combined discrete-event simulation and ant colony optimisation approach for selecting optimal screening policies for diabetic retinopathy. Comput Manag Sci 4(1):59–83
22. Bullnheimer B, Hartl RF, Strauss C (1999) A new rank-based version of the ant system: a computational study. Cent Eur J Oper Res Econ 7(1):25–38
23. Colorni A, Dorigo M, Maniezzo V (1992) Distributed optimization by ant colonies. In: Varela FJ, Bourgine P (eds) Proceedings of the first European conference on artificial life. MIT Press, Cambridge, pp 134–142
24. Cordón O, de Viana IF, Herrera F, Moreno L (2000) A new ACO model integrating evolutionary computation concepts: the best-worst ant system. In: Dorigo M et al (eds) Abstract proceedings of ANTS 2000 – from ant colonies to artificial ants: second international workshop on ant algorithms. IRIDIA, Université Libre de Bruxelles, Belgium, pp 22–29
25. Deneubourg JL, Aron S, Goss S, Pasteels JM (1990) The self-organizing exploratory pattern of the Argentine ant. J Insect Behav 3(2):159–168
26. de Oliveira SM, Bezerra CT, Stützle T, Dorigo M, Wanner EF, de Souze SR (2021) A computational study on ant colony optimization for the traveling salesman problem with dynamic demands. Comput Oper Res 135:105359
27. Di Caro GA, Dorigo M (1998) AntNet: distributed stigmergetic control for communications networks. J Artif Intell Res 9:317–365
28. Di Caro GA, Ducatelle F, Gambardella LM (2005) AntHocNet: an adaptive nature-inspired algorithm for routing in mobile ad hoc networks. Eur Trans Telecommun 16(5):443–455
29. Díaz D, Valledor P, Areces P, Rodil J, Suárez M (2014) An ACO algorithm to solve an extended cutting stock problem for scrap minimization in a bar mill. In: Dorigo M et al (eds) Swarm intelligence, 9th international conference, ANTS 2014. Lecture notes in computer science, vol 8667. Springer, Heidelberg, pp 13–24
30. Doerner KF, Hartl RF, Reimann M (2003) Are COMPETants more competent for problem solving? The case of a multiple objective transportation problem. Cent Eur J Oper Res Econ 11(2):115–141

31. Doerner KF, Gutjahr WJ, Hartl RF, Strauss C, Stummer C (2004) Pareto ant colony optimization: a metaheuristic approach to multiobjective portfolio selection. Ann Oper Res 131:79–99
32. Doerr B, Neumann F, Sudholt D, Witt C (2011) Runtime analysis of the 1-ANT ant colony optimizer. Theor Comput Sci 412(1):1629–1644
33. Donati AV, Montemanni R, Casagrande N, Rizzoli AE, Gambardella LM (2008) Time dependent vehicle routing problem with a multi ant colony system. Eur J Oper Res 185(3):1174–1191
34. Dorigo M (1992) Optimization, learning and natural algorithms. PhD thesis, Dipartimento di Elettronica, Politecnico di Milano, Italy (in Italian)
35. Dorigo M (2007) Ant colony optimization. Scholarpedia 2(3):1461
36. Dorigo M, Blum C (2005) Ant colony optimization theory: a survey. Theor Comput Sci 344(2–3):243–278
37. Dorigo M, Di Caro GA (1999) The ant colony optimization meta-heuristic. In: Corne D, Dorigo M, Glover F (eds) New ideas in optimization. McGraw Hill, London, pp 11–32
38. Dorigo M, Gambardella LM (1997) Ant colony system: a cooperative learning approach to the traveling salesman problem. IEEE Trans Evol Comput 1(1):53–66
39. Dorigo M, Stützle T (2004) Ant colony optimization. MIT Press, Cambridge
40. Dorigo M, Maniezzo V, Colorni A (1991) The ant system: an autocatalytic optimizing process. Technical Report 91-016 Revised, Dipartimento di Elettronica, Politecnico di Milano, Italy
41. Dorigo M, Maniezzo V, Colorni A (1991) Positive feedback as a search strategy. Technical Report 91-016, Dipartimento di Elettronica, Politecnico di Milano, Italy
42. Dorigo M, Maniezzo V, Colorni A (1996) Ant system: optimization by a colony of cooperating agents. IEEE Trans Syst Man Cybern Part B 26(1):29–41
43. Dréo J, Siarry P (2004) Continuous interacting ant colony algorithm based on dense heterarchy. Future Gener Comput Syst 20(5):841–856
44. Ehrgott M (2000) Multicriteria optimization. Lecture notes in economics and mathematical systems, vol 491. Springer, Berlin
45. Eyckelhof CJ, Snoek M (2002) Ant systems for a dynamic TSP: ants caught in a traffic jam. In: Dorigo M et al (eds) Ant algorithms. Third international workshop, ANTS 2002. Lecture notes in computer science, vol 2463. Springer, Heidelberg, pp 88–99
46. Falcón-Cardona JG, Coello Coello CA (2017) A new indicator-based many-objective ant colony optimizer for continuous search spaces. Swarm Intell 11(1):71–100
47. Feo TA, Resende MGC (1989) A probabilistic heuristic for a computationally difficult set covering problem. Oper Res Lett 8(2):67–71
48. Feo TA, Resende MGC (1995) Greedy randomized adaptive search procedures. J Global Optim 6:109–113
49. Fernández S, Álvarez S, Díaz D, Iglesias M, Ena B (2014) Scheduling a galvanizing line by ant colony optimization. In: Dorigo M et al (eds) Swarm intelligence. 9th international conference, ANTS 2014. Lecture notes in computer science, vol 8667. Springer, Heidelberg, pp 146–157
50. Gambardella LM, Dorigo M (1996) Solving symmetric and asymmetric TSPs by ant colonies. In: Bäck T, Fukuda T, Michalewicz Z (eds) Proceedings of the 1996 IEEE international conference on evolutionary computation (ICEC'96). IEEE Press, Piscataway, pp 622–627
51. Gambardella LM, Taillard ÉD, Agazzi G (1999) MACS-VRPTW: a multiple ant colony system for vehicle routing problems with time windows. In: Corne D, Dorigo M, Glover F (eds) New ideas in optimization. McGraw Hill, London, pp 63–76
52. Gambardella LM, Montemanni R, Weyland D (2012) Coupling ant colony systems with strong local searches. Eur J Oper Res 220(3):831–843
53. García-Martínez C, Cordón O, Herrera F (2007) A taxonomy and an empirical analysis of multiple objective ant colony optimization algorithms for the bi-criteria TSP. Eur J Oper Res 180(1):116–148

54. Garey MR, Johnson DS (1979) Computers and intractability: a guide to the theory of NP-completeness. Freeman & Co, San Francisco
55. Glover F (1998) A template for scatter search and path relinking. In: Hao JK, Lutton E, Ronald EMA, Schoenauer M, Snyers D (eds) Artificial evolution. Lecture notes in computer science, vol 1363. Springer, Heidelberg, pp 1–51
56. Goldberg DE (1989) Genetic algorithms in search, optimization and machine learning. Addison-Wesley, Boston
57. Guntsch M, Branke J (2003) New ideas for applying ant colony optimization to the probabilistic TSP. In: Cagnoni S et al (eds) Applications of evolutionary computing. Proceedings of EvoWorkshops 2003. Lecture notes in computer science, vol 2611. Springer, Heidelberg, pp 165–175
58. Guntsch M, Middendorf M (2001) Pheromone modification strategies for ant algorithms applied to dynamic TSP. In: Boers EJW et al (eds) Applications of evolutionary computing. Proceedings of EvoWorkshops 2001. Lecture notes in computer science, vol 2037. Springer, Heidelberg, pp 213–222
59. Guntsch M, Middendorf M (2002) Applying population based ACO to dynamic optimization problems. In: Dorigo M et al (eds) Ant algorithms. Third international workshop, ANTS 2002. Lecture notes in computer science, vol 2463. Springer, Heidelberg, pp 111–122
60. Guntsch M, Middendorf M (2002) A population based approach for ACO. In: Cagnoni S et al (eds) Applications of evolutionary computing. Proceedings of EvoWorkshops 2002. Lecture notes in computer science, vol 2279. Springer, Heidelberg, pp 71–80
61. Guntsch M, Middendorf M (2003) Solving multi-objective permutation problems with population based ACO. In: Fonseca CM, Fleming PJ, Zitzler E, Deb K, Thiele L (eds) Evolutionary multi-criterion optimization, EMO 2003. Lecture notes in computer science, vol 2632. Springer, Heidelberg, pp 464–478
62. Gutjahr WJ (2000) A Graph-based ant system and its convergence. Future Gener Comput Syst 16(8):873–888
63. Gutjahr WJ (2002) ACO algorithms with guaranteed convergence to the optimal solution. Inf Process Lett 82(3):145–153
64. Gutjahr WJ (2004) S-ACO: an ant-based approach to combinatorial optimization under uncertainty. In: Dorigo M et al (eds) 4th international workshop on ant colony optimization and swarm intelligence (ANTS 2004). Lecture notes in computer science, vol 3172. Springer, Heidelberg, pp 238–249
65. Gutjahr WJ (2006) On the finite-time dynamics of ant colony optimization. Method Comput Appl Probab 8(1):105–133
66. Gutjahr WJ (2007) Mathematical runtime analysis of ACO algorithms: survey on an emerging issue. Swarm Intell 1(1):59–79
67. Gutjahr WJ (2008) First steps to the runtime complexity analysis of ant colony optimization. Comput Oper Res 35(9):2711–2727
68. Gutjahr WJ, Rauner MS (2007) An ACO algorithm for a dynamic regional nurse-scheduling problem in Austria. Comput Oper Res 34(3):642–666
69. Hart JP, Shogan AW (1987) Semi-greedy heuristics: an empirical study. Oper Res Lett 6(3):107–114
70. Hoos HH (2012) Programming by optimization. Commun ACM 55(2):70–80
71. Iacopino C, Palmer P (2012) The dynamics of ant colony optimization algorithms applied to binary chains. Swarm Intell 6(4):343–377
72. Iredi S, Merkle D, Middendorf M (2001) Bi-criterion optimization with multi colony ant algorithms. In: Zitzler E, Deb K, Thiele L, Coello Coello CA, Corne D (eds) Evolutionary multi-criterion optimization, EMO 2001. Lecture notes in computer science, vol 1993. Springer, Heidelberg, pp 359–372
73. Jones DR, Schonlau M, Welch WJ (1998) Efficient global optimization of expensive black-box functions. J Global Optim 13(4):455–492

74. Khichane M, Albert P, Solnon C (2009) An ACO-based reactive framework for ant colony optimization: first experiments on constraint satisfaction problems. In: Stützle T (ed) Learning and intelligent optimization. Third international conference, LION 3. Lecture notes in computer science, vol 5851. Springer, Heidelberg, pp 119–133
75. Kocsis L, Szepesvári C (2006) Bandit based Monte Carlo planning. In: Fürnkranz J, Scheffer T, Spiliopoulou M (eds) 17th European conference on machine learning, ECML 2006. Lecture notes in computer science, vol 4212. Springer, Heidelberg, pp 282–293
76. Korb O, Stützle T, Exner TE (2007) An ant colony optimization approach to flexible protein–ligand docking. Swarm Intell 1(2):115–134
77. Kötzing T, Neumann F, Röglin H, Witt C (2012) Theoretical analysis of two ACO approaches for the traveling salesman problem. Swarm Intell 6(1):1–21
78. Kovářík O, Skrbek M (2008) Ant colony optimization with castes. In: Kurkova-Pohlova V, Koutnik J (eds) ICANN'08: Proceedings of the 18th international conference on artificial neural networks, Part I. Lecture notes in computer science, vol 5163. Springer, Heidelberg, pp 435–442
79. Leguizamón G, Alba E (2013) Ant colony based algorithms for dynamic optimization problems. In: Alba E, Nakib A, Siarry P (eds) Metaheuristics for dynamic optimization, studies in computational intelligence, vol 433. Springer, Berlin, pp 189–210
80. Liao T, Montes de Oca MA, Aydın D, Stützle T, Dorigo M (2011) An incremental ant colony algorithm with local search for continuous optimization. In: Krasnogor N, Lanzi PL (eds) Proceedings of the genetic and evolutionary computation conference, GECCO 2011. ACM Press, New York, pp 125–132
81. Liao T, Socha K, Montes de Oca MA, Stützle T, Dorigo M (2014) Ant colony optimization for mixed-variable optimization problems. IEEE Trans Evol Comput 18(4):503–518
82. Liao T, Stützle T, Montes de Oca MA, Dorigo M (2014) A unified ant colony optimization algorithm for continuous optimization. Eur J Oper Res 234(3):597–609
83. Lissovoi A, Witt C (2015) Runtime analysis of ant colony optimization on dynamic shortest path problems. Theor Comput Sci 61(Part A):73–85
84. Lissovoi A, Witt C (2016) MMAS versus population-based EA on a family of dynamic fitness functions. Algorithmica 75:554–576
85. Liu J, Wang Y, Sun G, Pang T (2021) Multi-surrogate-assisted ant colony optimization for expensive optimization problems with continuous and categorical variables. IEEE Trans Cybern 52(11):11348–11361
86. López-Ibáñez M, Blum C (2010) Beam-ACO for the travelling salesman problem with time windows. Comput Oper Res 37(9):1570–1583
87. López-Ibáñez M, Stützle T (2012) The automatic design of multi-objective ant colony optimization algorithms. IEEE Trans Evol Comput 16(6):861–875
88. López-Ibáñez M, Stützle T (2012) An experimental analysis of design choices of multi-objective ant colony optimization algorithms. Swarm Intell 6(3):207–232
89. López-Ibáñez M, Stützle T (2014) Automatically improving the anytime behaviour of optimisation algorithms. Eur J Oper Res 235(3):569–582
90. López-Ibáñez M, Paquete L, Stützle T (2006) Hybrid population-based algorithms for the bi-objective quadratic assignment problem. J Math Modell Algorithms 5(1):111–137
91. López-Ibáñez M, Dubois-Lacoste J, Pérez Cáceres L, Stützle T, Birattari M (2016) The irace package: iterated racing for automatic algorithm configuration. Oper Res Perspect 3:43–58
92. Maniezzo V (1999) Exact and approximate nondeterministic tree-search procedures for the quadratic assignment problem. INFORMS J Comput 11(4):358–369
93. Maniezzo V, Carbonaro A (2000) An ANTS heuristic for the frequency assignment problem. Futur Gener Comput Syst 16(8):927–935
94. Marriott K, Stuckey P (1998) Programming with constraints. MIT Press, Cambridge
95. Martens D, Backer MD, Haesen R, Vanthienen J, Snoeck M, Baesens B (2007) Classification with ant colony optimization. IEEE Trans Evol Comput 11(5):651–665
96. Massen F, Deville Y, van Hentenryck P (2012) Pheromone-based heuristic column generation for vehicle routing problems with black box feasibility. In: Beldiceanu N, Jussien N, Pinson E (eds) Integration of AI and OR techniques in contraint programming for combinatorial

optimization problems. Lecture notes in computer science, vol 7298. Springer, Heidelberg, pp 260–274
97. Massen F, López-Ibáñez M, Stützle T, Deville Y (2013) Experimental analysis of pheromone-based heuristic column generation using irace. In: Blesa MJ, Blum C, Festa P, Roli A, Sampels M (eds) Hybrid metaheuristics. Lecture notes in computer science, vol 7919. Springer, Heidelberg, pp 92–106
98. Masukane T, Mizuno K (2018) Solving constraint satisfaction problems by cunning ants with multi-pheremones. Int J Mach Lear Comput 8(4):361-366
99. Mavrovouniotis M, Martins Müller F, Yang S (2017) Ant colony optimization with local search for dynamic traveling salesman problems IEEE Trans Cybern 47(7):1743-1756
100. Merkle D, Middendorf M (2001) Prospects for dynamic algorithm control: Lessons from the phase structure of ant scheduling algorithms. In: Heckendorn RB (ed) Proceedings of the 2001 genetic and evolutionary computation conference–workshop program. Workshop "The Next Ten Years of Scheduling Research". Morgan Kaufmann Publishers, San Francisco, pp 121–126
101. Merkle D, Middendorf M (2002) Modeling the dynamics of ant colony optimization. Evol Comput 10(3):235–262
102. Merkle D, Middendorf M (2003) Ant colony optimization with global pheromone evaluation for scheduling a single machine. Appl Intell 18(1):105–111
103. Merkle D, Middendorf M, Schmeck H (2002) Ant colony optimization for resource-constrained project scheduling. IEEE Trans Evol Comput 6(4):333–346
104. Meuleau N, Dorigo M (2002) Ant colony optimization and stochastic gradient descent. Artif Life 8(2):103–121
105. Meyer B, Ernst AT (2004) Integrating ACO and constraint propagation. In: Dorigo M et al (eds) Ant colony optimization and swarm intelligence. 4th international workshop, ANTS 2004. Lecture notes in computer science, vol 3172. Springer, Heidelberg, pp 166–177
106. Michel R, Middendorf M (1998) An island model based ant system with lookahead for the shortest supersequence problem. In: Eiben AE, Bäck T, Schoenauer M, Schwefel HP (eds) Parallel problem solving from nature, PPSN V. Lecture notes in computer science, vol 1498. Springer, Heidelberg, pp 692–701
107. Monmarché N, Venturini G, Slimane M (2000) On how *pachycondyla apicalis* ants suggest a new search algorithm. Futur Gener Comput Syst 16(8):937–946
108. Montemanni R, Gambardella LM, Rizzoli AE, Donati AV (2005) Ant colony system for a dynamic vehicle routing problem. J Comb Optim 10:327–343
109. Montgomery J, Randall M, Hendtlass T (2008) Solution bias in ant colony optimisation: lessons for selecting pheromone models. Comput Oper Res 35(9):2728–2749
110. Moraglio A, Kattan A (2011) Geometric generalisation of surrogate model based optimization to combinatorial spaces. In: Merz P, Hao JK (eds) Proceedings of EvoCOP 2011 – 11th European conference on evolutionary computation in combinatorial optimization. Lecture notes in computer science, vol 6622. Springer, Heidelberg, pp 142–154
111. Morin S, Gagné C, Gravel M (2009) Ant colony optimization with a specialized pheromone trail for the car-sequencing problem. Eur J Oper Res 197(3):1185–1191
112. Nallaperuma S, Wagner M, Neumann F (2014) Parameter prediction based on features of evolved instances for ant colony optimization and the traveling salesperson problem. In: Bartz-Beielstein T, Branke J, Filipič B, Smith J (eds) PPSN 2014. Lecture notes in computer science, vol 8672. Springer, Heidelberg, pp 100–109
113. Neumann F, Sudholt D, Witt C (2009) Analysis of different MMAS ACO algorithms on unimodal functions and plateaus. Swarm Intell 3(1):35–68
114. Nurcahyadi T, Blum C, and Manyà F (2022) Negative learning ant colony optimization for MaxSAT. Int J Comput Intell Syst 15:71
115. Ow PS, Morton TE (1988) Filtered beam search in scheduling. Int J Prod Res 26:297–307
116. Paniri M, Dowlatshahi MB, and Nezamabadi-pour H (2021) Ant-TD: Ant colony optimization plus temporal difference reinforcement learning for multi-label feature selection. Swarm Evol Comput 64:100892

117. Papadimitriou CH, Steiglitz K (1982) Combinatorial optimization – algorithms and complexity. Prentice Hall, Englewood Cliffs
118. Pedemonte M, Nesmachnow S, Cancela H (2011) A survey on parallel ant colony optimization. Appl Soft Comput 11(8):5181–5197
119. Pellegrini P, Birattari M, Stützle T (2012) A critical analysis of parameter adaptation in ant colony optimization. Swarm Intell 6(1):23–48
120. Pérez Cáceres L, López-Ibáñez M, Stützle T (2015) Ant colony optimization on a limited budget of evaluations. Swarm Intell 9(2–3):103–124
121. Randall M (2004) Near parameter free ant colony optimisation. In: Dorigo M et al (eds) 4th international workshop on Ant colony optimization and swarm intelligence (ANTS 2004). Lecture notes in computer science, vol 3172. Springer, Heidelberg, pp 374–381
122. Randall M, Montgomery J (2002) Candidate set strategies for ant colony optimisation. In: Dorigo M et al (eds) 3rd international workshop on Ant algorithms (ANTS 2002). Lecture notes in computer science, vol 2463. Springer, Heidelberg, pp 243–249
123. Rezvanian A, Mehdi Vahidipour S, Sadollah A (2023) An overview of ant colony optimization algorithms for dynamic optimization problems. In: Andriychuk M, Sadollah A C (eds) Optimization algorithms. IntechOpen, Rijeka
124. Rojas-Morales N, Riff-Rojas M-C, Coello Coello C, Montero E (2018) A cooperative opposite-inspired learning strategy for ant-based algorithms. In: Dorigo M et al (eds) Swarm intelligence, 11th international conference, ANTS 2018. Lecture notes in computer science, vol 11172. Springer, Heidelberg, pp 317-324
125. Rivera G, Coello Coello C, Cruz-Reyes L, Fernandez E, Gomez-Santillan C, Rangel-Valdez N (2022) Preference incorporation into many-objective optimization: an ant colony algorithm based on interval outranking. Swa Evol Com 69(2022):101024
126. Schilde M, Doerner KF, Hartl RF, Kiechle G (2009) Metaheuristics for the bi-objective orienteering problem. Swarm Intell 3(3):179–201
127. Schyns M (2015) An ant colony system for responsive dynamic vehicle routing. Eur J Oper Res 245(3):704–718
128. Socha K (2004) ACO for continuous and mixed-variable optimization. In: Dorigo M et al (eds) 4th international workshop on ant colony optimization and swarm intelligence (ANTS 2004). Lecture notes in computer science, vol 3172. Springer, Heidelberg, pp 25–36
129. Socha K, Dorigo M (2007) Ant colony optimization for mixed-variable optimization problems. Technical report TR/IRIDIA/2007-019, IRIDIA, Université Libre de Bruxelles
130. Socha K, Dorigo M (2008) Ant colony optimization for continuous domains. Eur J Oper Res 185(3):1155–1173
131. Steuer RE (1986) Multiple criteria optimization: theory, computation and application. Wiley series in probability and mathematical statistics. Wiley, New York
132. Stützle T (1998) Local search algorithms for combinatorial problems – analysis, improvements, and new applications. PhD thesis, FB Informatik, TU Darmstadt
133. Stützle T (2002) ACOTSP: a software package of various ant colony optimization algorithms applied to the symmetric traveling salesman problem. http://www.aco-metaheuristic.org/aco-code/
134. Stützle T, Dorigo M (1999) ACO algorithms for the quadratic assignment problem. In: Corne D, Dorigo M, Glover F (eds) New ideas in optimization. McGraw Hill, London, pp 33–50
135. Stützle T, Dorigo M (2002) A short convergence proof for a class of ACO algorithms. IEEE Trans Evol Comput 6(4):358–365
136. Stützle T, Hoos HH (1996) Improving the ant system: a detailed report on the MAX–MIN ant system. Technical report AIDA–96–12, FG Intellektik, FB Informatik, TU Darmstadt
137. Stützle T, Hoos HH (1997) The MAX–MIN ant system and local search for the traveling salesman problem. In: Bäck T, Michalewicz Z, Yao X (eds) Proceedings of the 1997 IEEE international conference on evolutionary computation (ICEC'97). IEEE Press, Piscataway, pp 309–314

138. Stützle T, Hoos HH (1999) MAX–MIN ant system and local search for combinatorial optimization problems. In: Voß S, Martello S, Osman IH, Roucairol C (eds) Meta-heuristics: advances and trends in local search paradigms for optimization. Kluwer Academic, Dordrecht, pp 137–154
139. Stützle T, Hoos HH (2000) MAX–MIN ant system. Future Gener Comput Syst 16(8):889–914
140. Stützle T, López-Ibáñez M, Dorigo M (2011) A concise overview of applications of ant colony optimization. In: Cochran JJ (ed) Wiley encyclopedia of operations research and management science, vol 2. Wiley, Hoboken, pp 896–911
141. Stützle T, López-Ibáñez M, Pellegrini P, Maur M, Montes de Oca MA, Birattari M, Dorigo M (2012) Parameter adaptation in ant colony optimization. In: Hamadi Y, Monfroy E, Saubion F (eds) Autonomous search. Springer, Berlin, pp 191–215
142. Stützle T, Ruiz R (2017) Iterated Greedy. In: Martí R, Pardalos PM, Resende MGC (eds) Handbook of Heuristics. Springer Cham, pp 547–577
143. Sudholt D, Witt C (2019) On the choice of the update strength in estimation-of-distribution algorithms and ant colony optimization. Algorithmica 81:1450–1489
144. Sun Y, Wang S, Shen Y, Li X, Ernst AT, and Kirley M (2022) Boosting ant colony optimization via solution prediction and machine learning. Comput Oper Res 143:105769
145. Świechowski M, Godlewski K, Sawicki B, Mańdziuk J (2023) Monte Carlo tree search: a review of recent modifications and applications. Artif Intell Rev (2023) 56:2497–2562
146. Taillard ÉD (1991) Robust taboo search for the quadratic assignment problem. Parallel Comput 17(4–5):443–455
147. Teixeira C, Covas J, Stützle T, Gaspar-Cunha A (2012) Multi-objective ant colony optimization for solving the twin-screw extrusion configuration problem. Eng Optim 44(3):351–371
148. Thiruvadya D, Nguyen S, Shiri F, Zaidi N, Li X (2022) Surrogate-assisted population based ACO for resource constrained job scheduling with uncertainty. Swarm Evol Comput 69:101029
149. Torres CE, Rossi LF, Keffer J, Li K, Shen CC (2010) Modeling, analysis and simulation of ant-based network routing protocols. Swarm Intell 4(3):221–244
150. Tsutsui S (2006) An enhanced aggregation pheromone system for real-parameter optimization in the ACO metaphor. In: Dorigo M et al (eds) 5th international workshop on Ant colony optimization and swarm intelligence (ANTS 2006). Lecture notes in computer science, vol 4150. Springer, Heidelberg, pp 60–71
151. Tsutsui S (2007) Ant colony optimization with cunning ants. Trans Jpn Soc Artifi Intell 22:29–36
152. Wiesemann W, Stützle T (2006) Iterated ants: an experimental study for the quadratic assignment problem. In: Dorigo M et al (eds) 5th international workshop on ant colony optimization and swarm intelligence (ANTS 2006). Lecture notes in computer science, vol 4150. Springer, Heidelberg, pp 179–190
153. Xu H, Shen Y, Sun Y, and Li X (2024) Machine learning-enhanced ant colony optimization for column generation. In: Li X, Handl J (eds) Proceedings of the genetic and evolutionary computation conference, GECCO 2024. ACM Press, New York, pp 1073–1081
154. Ye K, Zhang C, Ning J, and Liu X (2017) Ant colony algorithm with a strengthened negative-feedback mechanism for constraint satisfaction problems. Inf Sci 406:29–41
155. Zaefferer M, Stork J, Friese M, Fischbach A, Naujoks B, Bartz-Beielstein T (2014) Efficient global optimization for combinatorial problems. In: Igel C, Arnold DV (eds) Proceedings of the genetic and evolutionary computation conference, GECCO 2014. ACM Press, New York, pp 871–878
156. Zhang H, Zhang Q, Ma L, Zhang Z, Liu Y (2019) A hybrid ant colony optimization algorithm for a multi-objective vehicle routing problem with flexible time windows. Inf Sci 490:166-190
157. Zhou S, Zhan Z, Chen Z, Kwong S, Zhang J (2021) A multi-objective ant colony system algorithm for airline crew rostering problem with fairness and satisfaction. IEEE Trans Int Trans Syst 22(11):6784–6798

Estimation of Distribution Algorithms

20

Pedro Larrañaga and Concha Bielza

Contents

Introduction 584
An Illustrative Example 584
General Scheme of EDAs 587
Initializing the Population 588
Selection of Individuals 589
Learning the Joint Probability Distribution 589
Sampling of New Candidate Solutions 590
Practical Issues 591
Applications in Optimization 592
Applications in Machine Learning 593
Conclusions 595
Cross-References 595
References 595

Abstract

In this chapter, estimation of distribution algorithms will be described. These algorithms belong to the evolutionary computation field and are characterized by evolving a population of candidate individuals as solutions of the optimization problem estimating at each generation the joint probability distribution of the selected individuals and then sampling new individuals from that distribution. Thus how to model (and sample from) that joint distribution is an important issue that in this chapter will be analyzed. The specificities of these algorithms when dealing with multimodal, multiobjective, or dynamic optimization problems will also be overviewed. Then parallelization and hybridizations with other

P. Larrañaga (✉) · C. Bielza
Departamento de Inteligencia Artificial, Universidad Politécnica de Madrid, Madrid, Spain
e-mail: pedro.larranaga@fi.upm.es; mcbielza@fi.upm.es

© Springer Nature Switzerland AG 2025

R. Martí et al. (eds.), *Handbook of Heuristics*,
https://doi.org/10.1007/978-3-032-00385-0_34

optimization heuristics will be presented. Next, applications of the algorithms will come in the last two sections. On the one hand, the use of these algorithms to solve real-world optimization problems in biomedicine, bioinformatics, energy, vehicle routing, and scheduling will be shown. On the other hand, applications in different machine learning tasks, such as supervised classification, clustering, and Bayesian networks, will be discussed. Finally, the conclusions will round the chapter off.

Keywords

Estimation of distribution algorithms · Joint probability distribution · Sampling from a distribution · Bayesian network · Probabilistic dependence · Applications in optimization · Applications in machine learning

Introduction

Estimation of distribution algorithms (EDAs) [1–6] are a class of evolutionary algorithms that build probabilistic models of promising solutions to guide the search process in optimization. Unlike traditional genetic algorithms, EDAs do not use crossover and mutation but instead learn and then sample from a probability distribution that represents the best solutions found so far, gradually refining this distribution to converge to optimal or near-optimal solutions.

This method of evolving populations of candidate individuals toward the optimum of a function offers several advantages. Firstly, the evolutionary process is stochastic and grounded in robust probability theory, making it easier to escape from local optima. Secondly, there is no need to design ad hoc operators for each specific problem, as is often required in other evolutionary computation methods. Thirdly, interactions among the variables encoding each individual, represented through conditional (in)dependence, can be explicitly observed if a probabilistic graphical model is learned from the selected population at each generation. This feature enhances the interpretability of the search process. Fourthly, it is possible to incorporate domain expertise from the optimization process by embedding expert knowledge directly into the graphical model, such as by enforcing certain must-link or must-not-link constraints between variables within the graph.

An Illustrative Example

To enhance comprehension of the distinct components and stages of EDAs, the most straightforward version of this method is applied to a basic optimization scenario. Consider the objective of maximizing the LeadingOnes function, that is, a function that maps a bit string of length n to its number of 1s before the first 0 in the bit string, wherein the goal is to achieve the function's maximum output [7].

Table 1 The initial population D_0. Each individual is evaluated with the LEADINGONES function

Individual	X_1	X_2	X_3	X_4	X_5	X_6	$f(\mathbf{x})$
1	0	0	1	0	1	0	0
2	0	1	0	0	1	0	0
3	0	0	0	1	0	0	0
4	1	1	1	0	0	1	3
5	0	0	0	0	0	1	0
6	1	1	0	0	1	1	2
7	0	1	1	1	1	1	0
8	0	0	0	1	0	0	0
9	1	1	0	1	0	0	2
10	1	0	1	0	0	0	1
11	1	0	0	1	1	1	1
12	1	1	0	0	0	1	2
13	1	0	1	0	0	0	1
14	0	0	0	0	1	1	0
15	0	1	1	1	1	1	0
16	0	0	0	1	0	0	0
17	1	1	1	1	1	0	5
18	0	1	0	1	1	0	0
19	1	0	1	1	1	1	1
20	0	0	1	1	0	0	0

Formally, for $\mathbf{x} = (x_1, \ldots, x_n)$, $\text{LEADINGONES}(\mathbf{x}) = \sum_{i=1}^{n} \prod_{j=1}^{i} x_j$. Its maximum is achieved at point $\mathbf{x}^* = (1, \ldots, 1)$ and its function value is n.

Consider a six-dimensional space, that is, $n = 6$. The initial population can be generated randomly. Let D_0 denote the dataset containing 20 instances (see Table 1) resulting from this generation process.

In a subsequent phase, the most fitted individuals are selected from D_0. In this case, truncation selection is used, whereby half of the population is retained. Let D_0^{Se} denote the dataset comprising the selected individuals, where in this case 9 out of the 20 individuals have fitness values greater than 0 and for the rest this value is 0. Thus, from the 11 individuals of fitness 0, one individual is selected, which is usually done at random. Following the selection of these 10 individuals (see Table 2), their characteristics are aimed to be captured through a joint probability distribution. While it should ideally account for all dependencies among the variables, in this example, the simplest model to represent the joint probability distribution is employed, where each variable is considered independent of the others. Mathematically, this assumption is formulated as follows: $p_1(\mathbf{x}) = p_1(x_1, \ldots, x_6) = \prod_{i=1}^{6} p_1(x_i \mid D_0^{Se})$.

Table 2 The selected individuals D_0^{Se} from the initial population

Individual	X_1	X_2	X_3	X_4	X_5	X_6
1	0	0	1	0	1	0
4	1	1	1	0	0	1
6	1	1	0	0	1	1
9	1	1	0	1	0	0
10	1	0	1	0	0	0
11	1	0	0	1	1	1
12	1	1	0	0	0	1
13	1	0	1	0	0	0
17	1	1	1	1	1	0
19	1	0	1	1	1	1

Thus, the model specification requires only six parameters. Each parameter, $p(x_i \mid D_0^{Se})$ for $i = 1, \ldots, 6$, is estimated from D_0^{Se} based on the corresponding relative frequency:

$$\hat{p}_1(X_1 = 1 \mid D_0^{Se}) = 0.9 \quad \hat{p}_1(X_2 = 1 \mid D_0^{Se}) = 0.5 \quad \hat{p}_1(X_3 = 1 \mid D_0^{Se}) = 0.6$$
$$\hat{p}_1(X_4 = 1 \mid D_0^{Se}) = 0.4 \quad \hat{p}_1(X_5 = 1 \mid D_0^{Se}) = 0.5 \quad \hat{p}_1(X_6 = 1 \mid D_0^{Se}) = 0.5$$

By sampling from this joint probability distribution, $p_1(\mathbf{x})$, a new population D_1 of 20 individuals is generated. Table 3 presents these individuals evaluated according to $f(\mathbf{x})$. Subsequently, the top 10 individuals are selected from D_1, resulting in the dataset D_1^{Se}, as shown in Table 4. From D_1^{Se}, the estimations are:

$$\hat{p}_2(X_1 = 1 \mid D_1^{Se}) = 1 \quad \hat{p}_2(X_2 = 1 \mid D_1^{Se}) = 1 \quad \hat{p}_2(X_3 = 1 \mid D_1^{Se}) = 1$$
$$\hat{p}_2(X_4 = 1 \mid D_1^{Se}) = 0.6 \quad \hat{p}_2(X_5 = 1 \mid D_1^{Se}) = 0.6 \quad \hat{p}_2(X_6 = 1 \mid D_1^{Se}) = 0.5$$

With these first two generations of the EDA, observe that the mean of $f(x)$ in D_0^{Se} was 1.9, while it has now increased to 4.1 in D_1^{Se}.

These three steps: (i) selecting individuals from the population, (ii) learning the joint probability distribution of the selected individuals, and (iii) sampling from the learned distribution, are repeated until a predetermined stopping criterion is met.

The primary challenge with EDAs lies in the estimation of the probability distribution $p_l(\mathbf{x})$ at each generation l. Clearly, estimating all parameters necessary to specify this joint probability distribution is impractical. Consequently, various approximations have been proposed, wherein the joint probability distribution is assumed to factorize according to a chosen probabilistic model.

Table 3 The population of the first generation D_1. Each individual is evaluated with the LEADINGONES function

Individual	X_1	X_2	X_3	X_4	X_5	X_6	$f(\mathbf{x})$
1	1	1	1	1	1	0	5
2	1	1	1	0	0	1	3
3	1	0	0	0	1	1	1
4	1	1	1	1	0	0	4
5	1	0	0	0	0	0	1
6	1	1	1	0	1	1	3
7	1	1	1	0	0	1	3
8	1	0	0	1	1	0	1
9	1	1	1	1	0	0	4
10	1	0	1	0	0	0	1
11	1	0	0	1	1	1	1
12	1	1	0	0	0	1	2
13	1	0	1	0	0	0	1
14	1	0	0	0	1	1	1
15	1	1	1	0	1	1	3
16	1	0	0	0	0	0	1
17	1	1	1	1	1	0	5
18	1	1	1	1	1	0	5
19	1	1	1	1	1	1	6
20	1	0	0	0	0	1	1

Table 4 The selected individuals D_1^{Se} from the first generation D_1

Individual	X_1	X_2	X_3	X_4	X_5	X_6
1	1	1	1	1	1	0
2	1	1	1	0	0	1
4	1	1	1	1	0	0
6	1	1	1	0	1	1
7	1	1	1	0	0	1
9	1	1	1	1	0	0
15	1	1	1	0	1	1
17	1	1	1	1	1	0
18	1	1	1	1	1	0
19	1	1	1	1	1	1

General Scheme of EDAs

The probability distribution that governs the individuals selected in each generation based on their fitness is then sampled to generate a new population of candidate solutions. For a comprehensive recent review, see [8].

Algorithm 1 presents a general and unified pseudocode that encompasses all variants of EDAs introduced below. The process begins by generating an initial population of individuals.

Algorithm 1: Pseudocode for the EDA

1 $D_0 \leftarrow$ Generate M individuals (initial population)
2 **repeat** for $l = 1, 2, \ldots$
3 1. $D_{l-1}^{Se} \leftarrow$ Select $N \leq M$ individuals from D_{l-1} according to the selection method
4 2. $p_l(\mathbf{x}) = p(\mathbf{x}|D_{l-1}^{Se}) \leftarrow$ Estimate the probability distribution of an individual $\mathbf{x}$ being among the selected population
5 3. $D_l \leftarrow$ Sample M individuals (new population) from $p_l(\mathbf{x})$
6 **until** *the stopping criterion is met*

In the first phase, the M generated individuals are evaluated, and a subset of N individuals is chosen based on a predetermined selection criterion. This criterion can be deterministic (e.g., selecting the N individuals with the highest objective function values) or stochastic, where individuals are selected with a probability proportional to their evaluation.

The second phase involves estimating the joint probability distribution of the selected individuals under various assumptions on the interactions among the decision variables encoding each individual. In the EDA literature, three common scenarios are considered: (a) assuming that all variables are independent, (b) accounting for only bivariate dependencies, or (c) allowing for unrestricted dependencies among the variables. In most practical optimization problems, the independence assumption in (a) is unrealistic, whereas scenario (c) can be computationally prohibitive when dealing with a high number of variables.

The third phase uses the estimated probability distribution to sample a new population of individuals. In cases of constrained optimization problems, it is essential to ensure that the generated individuals satisfy the constraints during the sampling process.

These three steps—evaluation and selection of individuals, learning of the probabilistic model, and simulation—are repeated until a stopping criterion is met. The stopping condition could be based on the number of generations, the convergence toward a global optimum, or a predefined limit on execution time.

Initializing the Population

The simplest way to initialize the population is by sampling from a uniform distribution across each variable, thereby obtaining individuals that cover the entire search space in a balanced manner. If prior knowledge about the problem is available or even historical data, the initialization process could use them.

Selection of Individuals

The selection of individuals in each generation of an EDA can be carried out through common and straightforward procedures in evolutionary computation [9], such as elitist selection, roulette wheel selection, truncation selection, tournament selection, rank-based selection, or more sophisticated methods like Boltzmann selection or stochastic universal sampling.

Learning the Joint Probability Distribution

Discrete EDAs refer to a class of optimization algorithms grounded in the EDA framework, specifically tailored for addressing combinatorial optimization challenges. On the other hand, continuous EDAs are utilized to tackle optimization in continuous spaces. Both discrete [5] and continuous [10] EDAs are classified according to the probabilistic dependency structures allowed for $p_l(\mathbf{x})$ in Algorithm 1.

No dependencies The n-dimensional joint probability distribution of selected individuals in each generation is assumed to decompose as a product of n univariate probability distributions, one for each variable. In other words, $p_l(\mathbf{x}) = \prod_{i=1}^{n} p_l(x_i)$, where in continuous EDAs, densities replace p_l. Key algorithms in this category include: (a) the univariate marginal distribution algorithm (UMDA) [11] for discrete problems, the univariate marginal distribution algorithm for continuous domains (UMDA$_c^G$) where each variable assumes a univariate Gaussian distribution [10]; (b) population-based incremental learning (PBIL) [12], which updates the probability vector components using a Hebbian rule inspired by the selected individuals in each generation, and its continuous version [13], where Gaussianity is assumed for each marginal univariate distribution; and (c) the compact genetic algorithm (cGA) [14], which simulates only two individuals per generation from the current probability distribution, iteratively adjusting probabilities toward the more successful individual until convergence. A general framework for univariate discrete EDAs that encompasses UMDA, PBIL, and cGA is proposed in [15].

Bivariate dependencies The foundational works in this domain include: (a) mutual information maximization for input clustering (MIMIC), (b) combining optimizers with mutual information trees (COMIT), and (c) the bivariate marginal distribution algorithm (BMDA). In MIMIC [16], the best variable permutation is searched for in each generation to approximate the empirical distribution of selected individuals by minimizing the Kullback-Leibler divergence, where $p_l^{\pi}(\mathbf{x}) = p_l(x_{i_1}|x_{i_2})p_l(x_{i_2}|x_{i_3}) \cdots p_l(x_{i_{n-1}}|x_{i_n})p_l(x_{i_n})$, with $\pi = (i_1, i_2, \ldots, i_n)$ denoting a permutation of indices. MIMIC$_c^G$ [10] extends MIMIC for continuous problems by assuming Gaussianity for both marginal and conditional distributions. In COMIT [17], a tree structure

is learned using the maximum weight spanning tree algorithm in each generation. In [18] the dependency tree is based on bivariate copula functions. BMDA [19] factorizes the joint probability distribution according to an acyclic graph formed by a set of trees (forest), where dependencies between pairs of variables are considered based on a threshold. The search of the forest has also been carried out using normalized mutual information [20].

Multivariate dependencies EDAs in this category typically focus on learning a Bayesian network (or a Gaussian Bayesian network) [21] that best represents the selected individuals' distribution in each generation, followed by sampling from the learned model. Key examples include: (a) the estimation of Bayesian network algorithm (EBNA) [22,23], which explores the learning phase with both constraint-based and score-based approaches, with BIC and K2 being the prominent scores; EGNA [10], a variant for continuous problems that learns a Gaussian Bayesian network in each generation, and [24], which adapts EGNA for high-dimensional issues by controlling model complexity; (b) the Bayesian optimization algorithm (BOA) [25], which utilizes the so-called BDe score and a greedy search method starting from scratch in each generation; (c) the learning factorized distribution algorithm (LFDA) [26], which limits the Bayesian network's complexity via the BIC score and a restriction on the number of parent variables per node; (d) the estimation of multivariate normal algorithm (EMNA) [10], which assumes a Gaussian joint probability distribution and estimates the mean vector and covariance matrix using maximum likelihood; (e) regularization-based EDAs, which apply regularization during Bayesian network learning [27], or which introduce regularized Gaussian Bayesian networks to address high-dimensional continuous problems [28]; and (f) the extended compact genetic algorithm (EcGA) [29], which factorizes the joint probability distribution as a product of distributions of variable groups, without the need for learning a Bayesian network every generation. Recent works on continuous optimization include the use of denoising autoencoders [30], restricted Boltzmann machines using attention mechanisms [31], and semiparametric Bayesian networks [32].

Sampling of New Candidate Solutions

Sampling in EDAs is mainly based on stochastic simulation methods of Bayesian networks and Gaussian Bayesian networks. For the former, there are methods such as probabilistic logic sampling, rejection sampling, likelihood weighting, importance sampling, Gibbs sampling, Markov chain Monte Carlo or variational inference sampling, whereas for the latter, possible methods are the Cholesky decomposition, eigenvalue decomposition, principal component analysis sampling, or Box-Muller transformations.

Practical Issues

From a practical perspective, EDAs have been adapted to tackle special (although common) optimization settings. In problems involving the search for the best **permutation**, which naturally arises in the traveling salesman problem, vehicle routing, or scheduling, EDAs have fundamentally adopted four ways of representing individuals: (i) direct permutation representation, in which the individual directly represents a permutation; (ii) adjacency representation, where each position in the sequence indicates the next element in the permutation; (iii) position representation, where each element of the permutation is assigned a specific position in the sequence, rather than directly encoding the order of elements or their adjacency relationships; and (iv) a real-coded individual, where the sorted indices represent the order of items in the permutation. The probabilistic modeling of permutations of selected individuals in each generation (see a review in [33]) can be carried out using position-based models, where the probability distribution is defined over the position of each element in the permutation independently of the others. Alternatively, it can be done using probability distributions defined over the permutation space, such as the Mallows distribution, the generalized Mallows distribution, or the Plackett-Luce model. These special probability distributions for permutations capture dependencies across the entire permutation, making them better suited for more complex rankings and ordering scenarios, as they account for the relative positions of all elements in the permutation.

Multimodal optimization is the process of identifying multiple optimal solutions within a search space. This type of optimization is valuable for problems where diverse solutions are desired, allowing for a more comprehensive exploration of potential solutions rather than focusing solely on a single global optimum. Multimodal optimization problems can be very challenging for an EDA based on a single probability distribution. Therefore, the most common approach to such problems is to consider a finite mixture of probability distributions, each component of the mixture corresponding to one of the peaks of the objective function. Partition-based clustering is used in [34] for multimodal combinatorial optimization problems, while [35] applies it in continuous domains. The structural expectation-maximization (EM) algorithm for multimodal problems in combinatorial optimization (based on Bayesian networks), as well as in continuous domains (based on conditional Gaussian Bayesian networks), is proposed in [36].

Multiobjective optimization is the process of simultaneously optimizing two or more conflicting objectives, aiming to find solutions that balance trade-offs between them. Rather than a single optimal solution, a set of Pareto-optimal solutions is yielded, where improving one objective would worsen at least one another. Evolutionary computation in general and EDAs in particular are well-suited for multiobjective optimization because it naturally handles complex, conflicting objectives and efficiently explores diverse solutions within a population. There is a large number of references that use EDAs in multiobjective optimization. Among the most notable is the multiobjective hierarchical BOA [37] applied to multiob-

jective decomposable problems, and the proposal in [38] where a multidimensional Gaussian Bayesian network is used to learn a joint model of objectives and variables while at the same time differentiating their role in the network.

Dynamic optimization involves finding optimal solutions for problems where the decision variables and/or the objective function change over time. EDAs have been used in dynamic optimization for both single-objective problems [39], with a PBIL version based on multiple populations, and multiobjective problems [40], leveraging transfer learning.

The **parallelization** of EDAs is justified by the high computational cost associated with the three steps these algorithms comprise. Particularly costly is the learning of the joint probability distribution of the selected individuals when modeled through (Gaussian) Bayesian networks. While early parallel EDAs [41] focused on parallelizing only this step, later approaches extended parallelization to all three steps of EDAs [42].

While EDAs perform well in exploring the search space, they are not as effective in its exploitation [43]. This is the main reason why, in real-world problems, EDAs are often **hybridized** with other heuristic strategies. In the literature, there are some works on EDAs hybridized with local search, particle swarm optimization, genetic algorithms, variable neighborhood search, simulated annealing, ant colony optimization, scatter search, or tabu search. However, as we have checked in a literature review (unpublished) of EDA papers published in the Web of Science until 2024, the heuristic that appears most frequently in hybridization with EDAs is differential evolution [44].

Applications in Optimization

EDAs have been applied to solving real-world problems across a wide range of domains. In **biomedicine**, they have been used to predict the survival of cirrhotic patients who have undergone a transjugular intrahepatic portosystemic shunt, which is a procedure to create new connections between two blood vessels in the liver [45]. EDAs have also been applied to the treatment of epithelial ovarian cancer, the search for the optimal chemotherapy dose in cancer, variable selection in a severity classification system for Parkinson's patients based on non-motor symptoms, and a nurse shift scheduling problem. In **bioinformatics**, EDAs have been used to solve the problem of protein structure prediction in simplified models [46], along with other problems such as protein folding, side chain placement, motif discovery, and the search for genetic regulatory networks.

The determination of the best possible locations of depleted uranium bundles, which maximize fuel economy as well as safety, is a large-sized combinatorial optimization problem with constraints that have been approached with EDAs [47] in the **energy** domain. Other problems where EDAs have proposed accurate solutions in this domain are the optimal management of a large number of plug-in hybrid electric vehicles charging at a municipal parking station, the search for the optimal

model parameter estimation of solar and fuel cells, and the prediction of the possible gas emission quantity in coal mines.

In **vehicle routing**, EDAs have provided solutions to: the school bus routing problem with stops selection, an equitable risk routing problem for shipments of hazardous materials, truck and trailer routing problems, the vehicle routing problem with time windows, an agricultural route planning problem, trajectory optimization of space vehicle in rendezvous proximity operation, and signal optimization for large-scale traffic networks [48].

An artificial immune algorithm combined with an EDA for the **traveling salesman problem** (TSP) is proposed in [49]. EDAs have also been developed for some variants of the TSP: multiobjective multiple TSP or for the travelling thief problem [50].

Scheduling is the optimization problem for which the most EDA-based approaches have been developed. This includes some variants of the job shop scheduling problem, such as the job shop scheduling with sequence-dependent, the no-wait job shop scheduling, the flexible job shop scheduling and the multiobjective flexible job shop scheduling, as well as variants of the flow shop scheduling problem, such as the blocking flow shop scheduling, the hybrid flow shop scheduling, the hybrid flow shop scheduling with identical parallel machines, the hybrid flow shop scheduling with stochastic processing time, the reentrant hybrid flow shop scheduling, and the no-idle permutation flow shop scheduling.

Applications in Machine Learning

Machine learning is currently the area of artificial intelligence that is developing the most interesting methodological advancements and practical applications. A dataset serves as the input to an algorithm capable of inducing a model to make rational decisions. The search for the model (including its structure and parameters) that best fits the data or yields the best performance is conducted within a search space of such high dimensionality that an exhaustive search becomes unfeasible.

The exploration of high-dimensional search spaces is inherent to various machine learning tasks, as illustrated by the following examples:

- *Feature subset selection*, where the total number of possible feature subsets, denoted $f(n)$ for n features, is $f(n) = 2^n$
- *Partitional clustering*, where the number of possible partitions of N objects into K clusters is given by $S(N, K) = \frac{1}{K!}\sum_{i=0}^{K}(-1)^{K-i}\binom{K}{i}i^N$, for $K \in \mathbb{N}$, with initial conditions $S(0, 0) = 1$ and $S(N, 0) = S(0, N) = 0$
- *Learning a Bayesian network structure* with n nodes from data within the space of directed acyclic graphs (DAGs), where the number of possible structures is calculated using the recursive relation $f(n) = \sum_{i=1}^{n}(-1)^{i+1}\binom{n}{i}2^{i(n-i)}f(n-i)$ for $n > 2$, with initial conditions $f(0) = f(1) = 1$
- *Permutation-based problems*, such as optimal triangulation in the moral graph of a Bayesian network with n nodes, where the search space cardinality is $n!$

In each of these cases, the search is conducted in a discrete space, posing a highly complex combinatorial optimization problem. Additionally, determining optimal parameters for a machine learning model generally involves continuous optimization challenges, for instance, solving the likelihood equations in logistic regression which lacks a closed-form solution, or training an artificial neural network where its parameters are optimized using the backpropagation algorithm, a process often prone to local optima.

The complexity of this situation justifies the use of metaheuristics for searching the optimal machine learning model, leading to the field known as evolutionary machine learning when these heuristics are based on evolutionary computation. This section reviews several selected works in which EDAs are employed for this purpose. See [51] for a more complete discussion on this topic.

Supervised classification is one of the machine learning tasks where EDAs have been utilized. In feature subset selection for classifiers such as naive Bayes, the EBNA algorithm has been applied [52], as well as the UMDA$_c^G$ algorithm in logistic regression [53]. In artificial neural networks, the PBIL algorithm has been used as an alternative to backpropagation for finding optimal weights connecting nodes in consecutive layers of a multilayer perceptron [54], while the UMDA$_c^G$ has been applied to optimize weights in autoencoders [55]. Among Bayesian classifiers, a variant of naive Bayes that utilizes confidence intervals associated with each conditional probability in each predictor variable seeks the best combination of these parameters using the UMDA$_c^G$ algorithm [56]. Meanwhile, in semi-naive Bayes models, the UMDA algorithm has been applied to identify the model with the best performance [57]. In logistic regression, UMDA$_c^G$ has been used to search for parameter estimates that solve the system of $n+1$ equations and $n+1$ unknowns arising from the maximization of the conditional likelihood function [58].

The three most common types of **clustering**, namely hierarchical, partitional, and probabilistic, have been addressed using EDAs. In hierarchical clustering, UMDA has been applied to provide stochasticity to the subset merging process [59]. In partitional clustering, MIMIC, COMIT, and EBNA have been EDAs applied to the K-means algorithm [60], while UMDA and EBNA enhanced the search process for optimal exemplars in the affinity propagation algorithm [61]. Probabilistic clustering is based on the structural EM algorithm to search for probabilistic dependency relationships among variables. In [62], UMDA is incorporated to improve the greedy heuristic on which the structural EM typically relies.

In **Bayesian networks**, EDAs have been applied to optimization problems arising both in inference tasks and in learning the model from data. For inference, [63] uses a recursive EDA to search for the optimal triangulation of the moral graph associated with the Bayesian network, while [64] applies both UMDA and EBNA in counterfactual reasoning from Bayesian classifiers. Learning the structure and parameters of the Bayesian network in the space of DAGs has been carried out using UMDA and PBIL [65], while in the total ordering space, [66] has applied both discrete and continuous versions of UMDA and MIMIC.

Conclusions

EDAs are a type of evolutionary computation algorithm based on probability theory, enabling the evolution of a population of individuals through the learning of a probabilistic model that fits the selected individuals in each generation, followed by the simulation of this model to obtain the next generation of individuals. EDAs have been successfully applied to a wide range of real-world combinatorial optimization problems as well as in continuous domains and have also been utilized for providing the best model in various machine learning tasks.

In this chapter, the focus has been on EDAs where probabilistic (directed) graphical models like (Gaussian) Bayesian networks are learned from the selected individuals. Other probabilistic models that have supported EDA methodologies include (undirected) Markov networks [67].

EDAs can play a significant role in interpreting the process followed by the metaheuristic in reaching the optimal solution. In this regard, analyzing the evolution of (Gaussian) Bayesian network structures learned across generations, along with landscape analysis via search trajectory networks [68], can be highly beneficial.

Cross-References

- Ant Colony Optimization: A Component-Wise Overview
- Fundamentals of Scatter Search
- Genetic Algorithms
- Multiobjective Optimization
- Particle Swarm Optimization
- Tabu Search
- Variable Neighborhood Search

Competing Interest Declaration The author(s) has no competing interests to declare that are relevant to the content of this manuscript.

References

1. Larrañaga P, Lozano JA (eds) (2002) Estimation of distribution algorithms. A new tool for evolutionary computation. Kluwer Academic Publishers, Boston/Dordrecht/London
2. Pelikan M, Goldberg DE, Lobo F (2002) A survey of optimization by building and using probabilistic models. Comput Optim Appl 21(1):5–20
3. Lozano JA, Larrañaga P, Inza I, Bengoetxea E (eds) (2005) Towards an new evolutionary computation. Advances in estimation of distribution algorithms. Springer, Berlin/Heidelberg
4. Hauschild M, Pelikan M (2011) An introduction and survey of estimation of distribution algorithms. Swarm Evol Comput 1(3):111–128
5. Santana R, Lozano JA, Larrañaga P (2008) Research topics in discrete estimation of distribution algorithms. Mem Comput 1:35–54

6. Larrañaga P, Karshenas H, Bielza C, Santana R (2012) A review on probabilistic graphical models in evolutionary computation. J Heuristics 18(5):795–819
7. Rudolph G (1997) Convergence properties of evolutionary algorithms. Dr. Kovač
8. Ceberio J, Mendiburu A, Lozano JA (2022) A roadmap for solving optimization problems with estimation of distribution algorithms. Nat Comput 23(1):99–113
9. Zhang B-T, Kim J-J (2000) Comparison of selection methods for evolutionary optimization. Evol Optim 2(1):55–70
10. Larrañaga P, Etxeberria R, Lozano JA, Peña JM (2000) Optimization in continuous domains by learning and simulation of Gaussian networks. In: Proceedings of the 2000 genetic and evolutionary computation conference workshop program. ACM, pp 201–204
11. Mühlenbein H, Paaß G (1996) From recombination of genes to the estimation of distributions. I. Binary parameters. Lecture notes in computer science, vol 1411. Springer, pp 178–187
12. Baluja S (1994) Population-based incremental learning: a method for integrating genetic search based function optimization and competitive learning. Technical Report CMU-CS-94-163, Carnegie Mellon University
13. Sebag M, Ducoulombier A (1998) Extending population-based incremental learning to continuous spaces. In: Parallel problem solving from nature- PPSN V. Springer, pp 418–427
14. Harik G, Lobo FG, Goldberg DE (1999) The compact genetic algorithm. IEEE Trans Evol Comput 3(4):287–297
15. Doerr B, Dufay M (2022) General univariate estimation-of-distribution algorithms. In: Proceedings of the 17th international conference on parallel problem solving from nature. Springer, pp 470–484
16. De Bonet JS, Isbell CL, Viola P (1997) MIMIC: finding optima by estimating probability densities. Adv Neural Inf Process Syst 9:424–430
17. Baluja S, Davies S (1997) Combining multiple optimization runs with optimal dependency trees. Technical Report CMU-CS-97-157, Carnegie Mellon University
18. Salinas-Gutiérrez R, Hernández-Aguirre A, Villa-Diharce ER (2011) Dependence trees with copula selection for continuous estimation of distribution algorithms. In: Proceedings of the 13th annual conference on genetic and evolutionary computation. ACM, pp 585–592
19. Pelikan M, Mühlenbein H (1999) The bivariate marginal distribution algorithm. In: Advances in soft computing-engineering design and manufacturing. Springer, pp 521–535
20. Lin Z, Su Q, Xie G (2021) NMIEDA: estimation of distribution algorithm based on normalized mutual information. Concurr Comput Practi Exp 33(6):e6074
21. Koller D, Friedman N (2009) Probabilistic graphical models: principles and techniques. The MIT Press, Cambridge, Massachusetts
22. Etxeberria R, Larrañaga P (1999) Global optimization using Bayesian networks. In: Second international symposium on artificial intelligence, pp 332–339
23. Larrañaga P, Etxeberria R, Lozano JA, Peña JM (2000) Combinatorial optimization by learning and simulation of Bayesian networks. In: Proceedings of the 16th conference on uncertainty in artificial intelligence. Morgan Kaufmann Publishers, pp 343–352
24. Dong W, Chen T, Tiňo P, Yao X (2013) Scaling up estimation of distribution algorithms for continuous optimization. IEEE Trans Evol Comput 17(6):797–822
25. Pelikan M, Goldberg DE, Cantú-Paz E (1999) BOA: the Bayesian optimization algorithm. In: Proceedings of the 1st annual conference on genetic and evolutionary computation, vol 1, pp 525–532
26. Mühlenbein H, Mahning T (1999) FDA—a scalable evolutionary algorithm for the optimization of additively decomposed functions. Evol Comput 7(4):353–376
27. Xu H, Yang J, Jia P, Ding Y (2013) Effective structure learning for estimation of distribution algorithms via L1-regularized Bayesian networks. Int J Adv Robot Syst 10(1)
28. Karshenas H, Santana R, Bielza C, Larrañaga P (2013) Regularized continuous estimation of distribution algorithms. Appl Soft Comput 13(5):2412–2432
29. Harik G (1999) Linkage learning via probabilistic modelling in the ECGA. IlliGAL Report No. 99010, University of Illinois at Urbana-Champaign

30. Probst M, Rothlauf F (2020) Harmless overfitting: using denoising autoencoders in estimation of distribution algorithms. J Mach Learn Res 21(78):1–31
31. Bao L, Sun X, Gong D, Zhang Y, Xu B (2020) Enhanced interactive estimation of distribution algorithms with attention mechanism and restricted Boltzmann machine. In: 2020 IEEE congress on evolutionary computation, pp 1–9
32. Soloviev V, Bielza C, Larrañaga P (2024) Semiparametric estimation of distribution algorithms for continuous optimization. IEEE Trans Evol Comput 28(4):1069–1083
33. Ceberio J, Irurozki E, Mendiburu A, Lozano JA (2012) A review on estimation of distribution algorithms in permutation-based combinatorial optimization problems. Progress Artif Intell 1:103–117
34. Pelikan M, Goldberg DE (2000) Genetic algorithms, clustering, and the breaking of symmetry. In: Proceedings of parallel problem solving from nature IV. Springer, pp 385–394
35. Bosman PAN, Thierens D (2002) Multi-objective optimization with diversity preserving mixture-based iterated density estimation evolutionary algorithms. Int J Approx Reason 31(3):259–289
36. Peña JM, Lozano JA, Larrañaga P (2005) Globally multimodal problem optimization via an estimation of distribution algorithm based on unsupervised learning of Bayesian networks. Evol Comput 13(1):43–66
37. Pelikan M, Sastry K, Goldberg DE (2006) Multiobjective estimation of distribution algorithms. In: Scalable optimization via probabilistic modeling, vol 33. Springer, pp 223–248
38. Karshenas H, Santana R, Bielza C, Larrañaga P (2013) Multi-objective estimation of distribution algorithms based on joint modelling of objectives and variables. IEEE Trans Evol Comput 18(4):519–542
39. Uludağ G, Kiraz B, Uyar ASE, Özcan E (2013) A hybrid multi-population framework for dynamic environments combining online and offline learning. Soft Comput 17(12):2327–2348
40. Jiang M, Huang Z, Qiu L, Huang W, Yen GG (2018) Transfer learning-based dynamic multiobjective optimization algorithms. IEEE Trans Evol Comput 22(4):501–514
41. Lozano JA, Sagarna R, Larrañaga P (2002) Parallel estimation of distribution algorithms. In: Estimation of distribution algorithms. A new tool for evolutionary computation. Kluwer Academic Publishers, pp 129–145
42. Mendiburu A, Lozano JA, Miguel-Alonso J (2005) Parallel implementation of EDAs based on probabilistic graphical models. IEEE Trans Evol Comput 9(4):406–423
43. Zhou A, Sun J, Zhang Q (2005) An estimation of distribution algorithm with cheap and expensive local search methods. IEEE Trans Evol Comput 19(6):807–822
44. Sun J, Zhang Q, Tsang EPK (2005) DE/EDA: a new evolutionary algorithm for global optimization. Inf Sci 169(3–4):249–262
45. Blanco R, Inza I, Merino M, Quiroga J, Larrañaga P (2005) Feature selection in Bayesian classifiers for the prognosis of survival of cirrhotic patients treated with TIPS. J Biomed Inform 38:376–388
46. Santana R, Lozano JA, Larrañaga P (2008) Protein folding in simplified models with estimation of distribution algorithms. IEEE Trans Evol Comput 12(4):418–438
47. Mishra S, Modak RS, Ganesan S (2010) Optimization of depleted uranium bundle loading in fresh core of Indian PHWR by evolutionary algorithm. Ann Nucl Energy 37(2):208–217
48. Liang Y, Ren Z, Wang L, Liu H, Du W (2021) Surrogate-assisted cooperative signal optimization for large-scale traffic networks. Knowl-Based Syst 234:107542
49. Xu Z, Wang Y, Li S, Liu Y, Todo Y, Gao S (2016) Immune algorithm combined with estimation of distribution for traveling salesman problem. IEEJ Trans Electric Electron Eng 11(S1):142–154
50. Martins MSR, El Yafrani M, Delgado MRBS, Wagner M, Ahiod B, Lüders R (2017) HSEDA: a heuristic selection approach based on estimation of distribution algorithm for the travelling thief problem. In: Proceedings of the 2017 genetic and evolutionary conference. ACM, pp 361–368
51. Larrañaga P, Bielza C (2024) Estimation of distribution algorithms in machine learning: a survey. IEEE Trans Evol Comput 28(5):1301–1321

52. Inza I, Larrañaga P, Etxeberria R, Sierra B (2000) Feature subset selection by Bayesian network–based optimization. Artif Intell 123:157–184
53. Robles V, Bielza C, Larrañaga P, González S, Ohno-Machado L (2008) Optimizing logistic regression coefficients for discrimination and calibration using estimation of distribution algorithms. TOP 16(2):345–366
54. Baluja S (1995) An empirical comparison of seven iterative and evolutionary function optimization heuristics. Technical Report CMU-CS-95-193, Carnegie Mellon University
55. Xu Q, Liu A, Yuan X, Song Y, Zhang C, Li Y (2021) Random mask-based estimation of the distribution algorithm for stacked auto-encoder one-step pre-training. Comput Ind Eng 158:107400
56. Robles V, Larrañaga P, Peña JM, Menasalvas E, Pérez MS (2003) Interval estimation naïve Bayes. Lecture notes in computer science, vol 2810. Springer, pp 143–154
57. Robles V, Larrañaga P, Peña JM, Menasalvas E, Pérez MS, Herves V, Wasilewska A (2004) Bayesian networks multi-classifiers for protein secondary structure prediction. Artif Intell Med 31:117–136
58. Bielza C, Robles V, Larrañaga P (2011) Regularized logistic regression without a penalty term: an application to cancer classification with microarray data. Expert Syst Appl 38(5):5110–5118
59. Fan J (2019) OPE-HCA: an optimal probabilistic estimation approach for hierarchical clustering algorithm. Neural Comput Appl 31:2095–2105
60. Roure J, Larrañaga P, Sangüesa R (2002) An empirical comparison between K-means, GAs and EDAs in partitional clustering. In: Estimation of distribution algorithms. A new tool for evolutionary computation. Kluwer Academic Publishers, pp 343–360
61. Santana R, Bielza C, Larrañaga P (2011) Affinity propagation enhanced by estimation of distribution algorithms. In: Proceedings of the 2011 genetic and evolutionary conference. ACM, pp 331–338
62. Peña JM, Lozano JA, Larrañaga P (2004) Unsupervised learning of Bayesian networks via estimation of distribution algorithms: an application to gene expression data clustering. Int J Uncertain Fuzz Knowl-Based Syst 12:63–82
63. Romero T, Larrañaga P (2009) Triangulation of Bayesian networks with recursive estimation of distribution algorithms. Int J Approx Reason 50(3):472–484
64. Zaragoza D, Bielza C, Larrañaga P (2024) Multi-objective counterfactuals in Bayesian classifiers with estimation of distribution algorithms. In: Proceedings of the 12th international conference on probabilistic graphical models, PMLR, vol 246, pp 415–426
65. Blanco R, Inza I, Larrañaga P (2003) Learning Bayesian networks in the space of structures by estimation of distribution algorithms. Int J Intell Syst 18:205–220
66. Romero T, Larrañaga P, Sierra B (2004) Learning Bayesian networks in the space of orderings with estimation of distribution algorithms. Int J Pattern Recogn Artif Intell 18(4):607–625
67. Shakya S, Santana R (eds) (2012) Markov networks in evolutionary computation. Springer, Berlin/Heidelberg
68. Fyvie M, McCall JAW, Christie LA, Brownlee AEI, Singh M (2023) Towards explainable metaheuristics: feature extraction from trajectory mining. Expert Syst 42:e13494

Fundamentals of Scatter Search

21

Rafael Martí, José Laguardia, and María Teresa León

Contents

Introduction . . . 600
Past and Present . . . 602
The Five Main Components . . . 603
The Maximum Diversity Problem . . . 607
Scatter Search for the MDP . . . 609
The Diversification Generator Method . . . 609
Reference Set Initialization . . . 610
The Subset Generation Method . . . 611
The Solution Combination Method . . . 611
The Reference Set Update Method . . . 611
The Improvement Method . . . 612
Computational Experiments . . . 612
Search Methods . . . 613
Search Profiles . . . 616
Search Parameters . . . 618
Final Comparison . . . 620
Conclusions . . . 622
Cross-References . . . 622
References . . . 623

R. Martí (✉)
Operations Research and Statistics Department, School of Mathematics, University of Valencia, Valencia, Spain
e-mail: rafael.marti@uv.es

J. Laguardia
Facultad de Ciencias y Tecnología, Universidad Teconológica de Panamá, Panamá, República de Panamá
e-mail: jose.laguardia@utp.ac.pa

M. T. León
Facultat de Ciències Matemàtiques, Departament d'Estadística i Investigació Operativa, University of Valencia, València, Spain
e-mail: teresa.leon@uv.es

© Springer Nature Switzerland AG 2025

R. Martí et al. (eds.), *Handbook of Heuristics*,
https://doi.org/10.1007/978-3-032-00385-0_20

Abstract

Scatter search is a population-based metaheuristic that has been shown to yield high-quality outcomes for hard combinatorial optimization problems. It uses strategies for combining solution vectors, making limited use of randomization, that have proved effective in a variety of problem settings. The fundamental concepts and principles were first proposed in the 1960s and 1970s as an extension of mathematical relaxation techniques for combinatorial optimization problems. Its framework is flexible, allowing the development of implementations with varying degrees of sophistication.

This chapter provides a grounding in the scatter search methodology that will allow readers to create successful applications of their own. To illustrate this, we present a scatter search implementation for a $\mathcal{NP}$-hard variant of the classic p-hub median problem, for which we describe search elements, mechanisms, and strategies to generate, combine, and improve solutions.

Keywords

Scatter search · Metaheuristic · Combinatorial optimization · Maximum diversity problem

Introduction

Scatter search (SS) was conceived as an extension of a heuristic in the area of mathematical relaxation, which was designed for the solution of integer programming problems: surrogate constraint relaxation. The following three operations come from the area of mathematical relaxation, and they are the core of most evolutionary optimization methods including SS and genetic algorithms (GAs):

- Building, maintaining, and working with a population of elements (coded as vectors)
- Creating new elements by combining existing elements
- Determining which elements are retained based on a measure of quality

Two of the best-known mathematical relaxation procedures are Lagrangean relaxation [9] and surrogate constraint relaxation [11]. While Lagrangean approaches absorb "difficult" constraints into the objective function by creating linear combinations of them, surrogate constraint relaxations generate new constraints to replace those considered problematic. The generation of surrogate constraints also involves the combination of existing constraints using a vector of weights. In both cases, these relaxation procedures search for the best combination in an iterative manner.

Scatter search is more intimately related to surrogate relaxation procedures, because not only surrogate relaxation includes the three operations outlined above but also has the goal of generating information from the application of these operations. In the case of surrogate relaxation, the goal is to generate information that cannot be extracted from the original constraints. Scatter search takes on the same approach, by generating information through the combination of two or more solutions. It strategically explores the solution space of an optimization problem by evolving a set of *reference* points. These points define a set, known as *Reference Set* (*RefSet*), which consists of good solutions obtained by prior solving efforts.

SS and GAs were both introduced in the 70s. While Holland [19] introduced genetic algorithms and the notion of imitating nature and the "survival of the fittest" paradigm, Glover [12] introduced scatter search as a heuristic for integer programming that expanded on the concept of surrogate constraints as mentioned above. Both methods fall in the category of evolutionary optimization procedures, since they build, maintain, and evolve a set (population) of solutions throughout the search. Although the population-based approach makes SS and GAs part of the so-called evolutionary methods, there are fundamental differences between the two methodologies. Typical differences between the scatter search methodology and other evolutionary methods rely on the use of randomization and the size of the population of solutions. Specifically, SS mainly implements deterministic strategies to generate, combine, and improve solutions, while other evolutionary methods, such as GAs, generally introduce random elements in their strategies. On the other hand, in SS, the *RefSet* tends to be small, usually around 10 solutions, while in GAs the population tends to be much larger, usually around 100.

The following principles can be seen as the foundations of the scatter search methodology:

- Useful information about the characteristics of optimal solutions is typically contained in a suitable diverse collection of elite solutions.
- This information is exploited by the combination of the elite solutions.
- The purpose of the combinations is to incorporate to the elite set both diversity (in terms of solutions' attributes) and quality (in terms of objective value).
- Search mechanisms and strategies are not limited to combinations but include solution generation and improvement combination methods.

The scatter search framework is flexible, allowing the development of alternative implementations with varying degrees of sophistication. It is based on the idea of limiting the scope of the search to a selective group of combination types, as a mechanism for controlling the number of possible combinations in a given reference set. A comprehensive examination of this methodology can be found in the book by Laguna and Martí [33]. In the following subsection we review the most relevant recent applications to illustrate the impact of this methodology.

Past and Present

As mentioned before, scatter search was first introduced in 1977 [12] as a heuristic for integer programming. However, it seems that it was never applied or studied again until 1990, when it was presented at the EPFL Seminar on Operations Research and Artificial Intelligence Search Methods (Lausanne, Switzerland). An article based on this presentation was published in 1994 [13], in which the range of applications was expanded to nonlinear (continuous) optimization problems, binary and permutation problems. The algorithmic principles and the structure of the method were finally proposed in the so-called scatter search template [14]. In a way, this template was a simplification of the previous descriptions and served as the main reference for most of the scatter search implementations up to date. There are two important milestones in the scatter search publications since then. In 2002, a book on this methodology was published by Kluwer [33], that included implementations in C to help the reader to create his/her own applications. In 2006, a special issue of the *European Journal of Operational Research* [43] was devoted to successful scatter search applications.

The scatter search methodology has become the method of choice for the design of solution procedures for many $\mathcal{NP}$-hard combinatorial optimization problems. Its use has been steadily increasing as shown in Figs. 1 and 2. Those figures show the number of yearly publications from 1994 to 2024 on scatter search. Figure 1 shows the number of cites or references for the query "scatter search" on Google Scholar (http://scholar.google.com), while Fig. 2 shows the counts for the same query on the Science Citation Index database of Thompson Reuter's Web of Science, which corresponds to the publications in this topic (http://thomsonreuters.com/thomson-reuters-web-of-science/). To restrict our search, we looked up on the Scopus website (http://www.scopus.com) the number of publications with the query "scatter search"

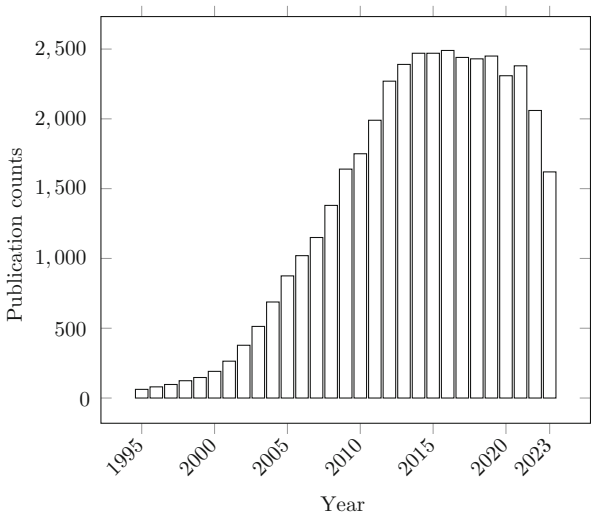


Fig. 1 Publication counts for the query "scatter search" on Google Scholar

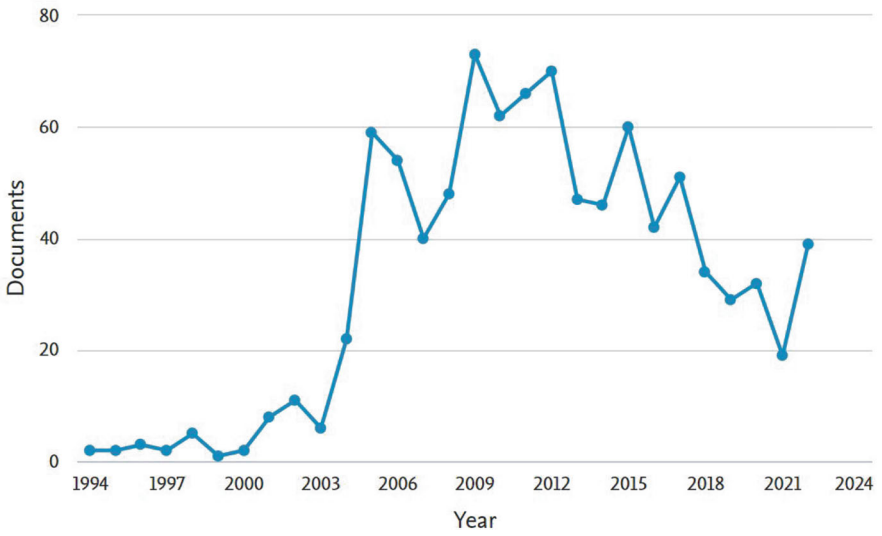


Fig. 2 "Scatter search" papers on the Web of Science

in the title and found that it was 383. We have shortlisted some of them (from 2012 to 2014) to have a restricted sample of journal papers with practical applications solved with scatter search. Table 1 summarizes the most relevant articles classified into six categories, according to the class of problem being solved. In particular, we consider classical problems such as TSP, VRP, and knapsack; scheduling and sequencing problems such as flowshop, jobshop, and project scheduling; manufacturing problems ranging from NoC mapping to coal and steel production; planning problems such as territory design and location problems; statistics and finance problems such as credit scoring and protein information prediction; and image registration problems such as image registration and segmentation, and craniofacial superimposition.

Although we can consider that the scatter search methodology is nowadays well established and that it has been used in many applications, if we compare SS with other metaheuristics, we can see that there is still significant room for improvement. In particular, a Scopus search with the query "genetic algorithm" returned almost 40,000 entries, while "tabu search" gave 2,273 hits. Consequently, one of this chapter's goal is to trigger the interest of researchers and practitioners to apply the scatter search methodology.

The Five Main Components

In scatter search, the search process is performed to capture information not contained separately in the original solutions. It takes advantage of auxiliary

Table 1 Journal papers published between 2012 and 2014

Classical problems	
TSP	[49]
VRP	[1,56,61,62]
Linear ordering	[21]
Knapsack	[38]
4-color mapping	[53]
Cutwidth minimization	[50]
Global optimization and black box	[22,34]
Scheduling and sequencing	
Flowshop	[18,48]
Assembly lines	[41]
Jobshop	[7]
Task scheduling	[26,55]
Project scheduling	[57]
Manufacturing	
NoC mapping	[35,36]
Aircraft conflict resolution	[39,58]
Cellular manufacturing systems	[24]
Grid resources selection	[3]
Coal production planning	[51]
Steel industry	[42]
Dissasembly sequence	[16]
SONET problems	[2]
Environmental—economic dispatch	[5]
Planning	
Layout	[25,29,30]
Transmission expansion planning	[17,47]
Commercial territory design	[54]
Location	[37,44]
Statistics and finance	
Credit scoring	[60]
Distribution fitting	[20]
Parameter determination	[40]
Protein information prediction	[27]
Image registration	
Intensity-based image registration	[59]
Image segmentation	[4]
Craniofacial superimposition	[23]

heuristic methods for selecting the elements to be combined and generating new solutions. The combination strategy is devised with the belief that this information can be better exploited when integrated rather than treated in isolation. In general, the decision rules created from such combination strategies produce better empirical outcomes than the standard applications of local decision rules.

As mentioned, scatter search operates on a set of solutions, *RefSet*, combining them to create new ones. The method basically performs iterations over the *RefSet* and can be summarized in three steps: Select the solutions, combine them, and update the *RefSet* with the resulting solutions. As in many other metaheuristics, an improvement method, or local search, can also be applied to improve the generated solutions. Here, the outcome of the combination method can be sent to a local search procedure to obtain a local optima.

Although the scatter search origins are linked with the mathematical programming area, in which convex combinations are very popular, convex and non-convex combinations are used in this method to generate new solutions. The non-convex combinations project new "centers" into regions of the solution space that are external to the original *RefSet* solutions, a strategy that is based on the principle of input diversity in the search, to avoid getting trapped in local optima.

The scatter search template provides a concise and direct description about how to implement the methodology. Figure 3 shows the scheme of our selective scatter search in which the improvement method is only applied at the end of the search. The methodology basically consists of five elements or methods and their associated strategies and may be sketched as follows:

- A diversification generation method (DGM):
  The DGM can be seen as the mechanism that creates the initial set of solutions P of an optimization problem. This is what we would call "a first generation" in the evolutionary terminology. If the attributes of this first generation are good, there is some confidence of getting better solutions in the following iterations after strategically combining these initial solutions. The DGM typically implements frequency-based memory to generate P and employs controlled randomization. As opposite to the GAs that usually rely on randomization to generate the initial population, the DGM samples the solution space in a systematic fashion. The size of P, $|P|$, is denoted as $Psize$.
- An improvement method (IM):
  The IM transforms a given solution of the problem, s, into an enhanced solution s'. Usually, both solutions are expected to be feasible, but this is not a requirement of the methodology. The IM generally relies on local search (LS) procedures (also known as *iterative improvement* procedures). As it is well known, given s, the local search tries to improve it by making "small changes" in its structure. These changes typically consist of adding elements to s, removing elements from s, changing the way in which elements are grouped in s, or changing their order, to mention a few. If an improvement is achieved in this way, then a new solution s' is obtained. This process of *small changes* is continued until no further improvement can be obtained or a time bound is elapsed. If no improvement of s was found, s would be the output.
- A reference set update method (RSUM):
  This method builds *RefSet* for the first time using a reference set creation subroutine (RSCsr) and updates *RefSet* during the search process. In the RSCsr, a given solution from P enters *RefSet* according to its quality or diversity. Many

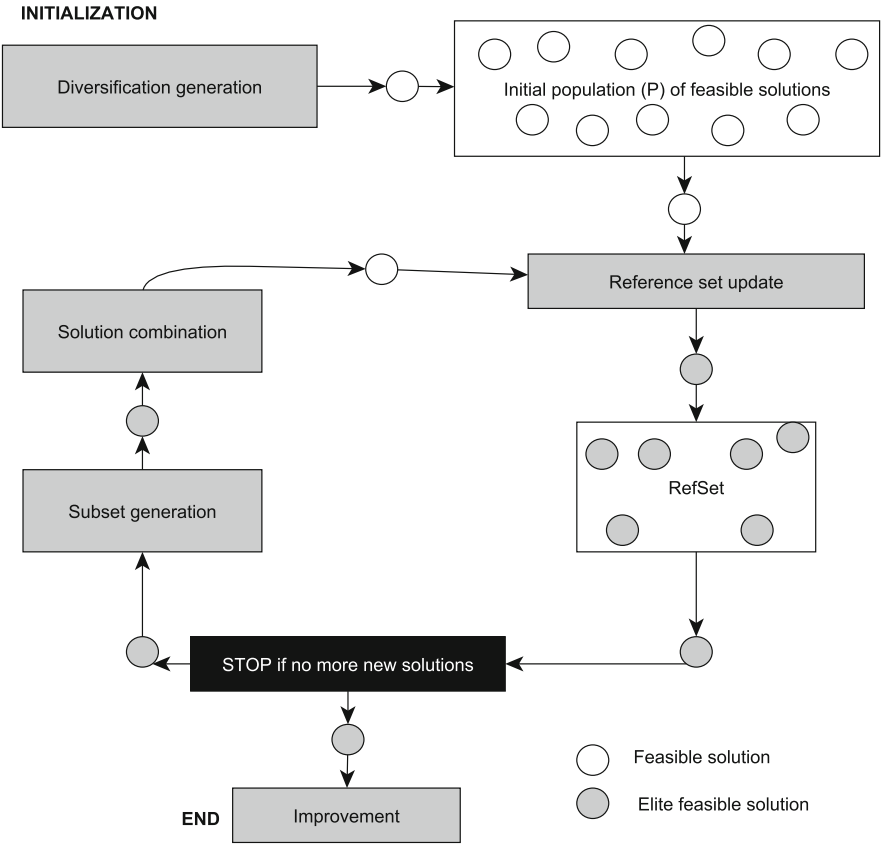


Fig. 3 Selective scatter search scheme

of the implementations of SS indicate that a good trade-off is to build 50% of *RefSet* by quality criterion, and the remaining 50% by a diversity criterion, but these proportions can be modified depending on the problem being solved.

A standard mechanism to build the initial *RefSet*, whose size will be denoted by b, follows. The construction starts with the selection of the best $b/2$ solutions from P. For each solution in $P \setminus RefSet$, the minimum of the distances to the solutions in *RefSet* is computed. Then, the solution that maximizes the minimal distances is selected. This solution is added to *RefSet* and deleted from P, and the minimal distances are updated. This process is repeated $b/2$ times. The resulting reference set has $b/2$ high-quality solutions and $b/2$ diverse solutions.

The update operation consists of maintaining a record of the b best solutions found, where the value of b is treated as a constant search parameter.

- A subset generation method (SGM):
  The SGM creates subsets of two or more solutions belonging to *RefSet*. These subsets will be subjected to the next SS process, the combination method. The general SS framework considers the generation of subsets with two, three, and four solutions but only generates a given subset if its solutions are being used to create this subset for the first time. This situation differs from those considered in the context of genetic algorithms, where the combinations are typically determined by the spin of a roulette wheel and are usually restricted to the combination of only two solutions.
- A solution combination method (SCM):
  The SCM creates new solutions from any subset of solutions previously created by the SGM. Although the process of creating new solutions that derive from other solutions can be achieved by a black-box procedure, it is generally more effective to design the SCM as a problem-specific mechanism, because it is directly related to the way we represent a solution in a problem. Depending on the specific form of the SCM, each subset can create one or more new solutions. This creation can be done in different ways, but the most common ones are systematic combinations of decision rules, path relinking, and randomization. We refer the reader to [15, 52] for some combination methods within SS.

In this second edition of the Handbook of Heuristics, we illustrate the SS methodology in this chapter by implementing its underlying framework to solve the Maximum Diversity Problem (MDP). Note that in the first edition [45], we implemented SS for the uncapacitated r-allocation p-hub median problem. Our results with these two problems show that the scatter search methodology is well suited for solving location problems. In line with our comment above, i.e., to encourage the reader to apply this methodology, here we can find a large family of problems in which the application of SS can result in a promising research area.

The Maximum Diversity Problem

The problem of maximizing diversity or dispersion deals with selecting a subset of elements from a given set in such a way that the distance among the selected elements is maximized. The definition of distance between elements is customized to specific applications, and the way that the overall diversity of the selected elements is computed results in different mathematical models. Maximizing diversity by means of combinatorial optimization models has gained prominence in Operations Research over the last two decades and constitutes nowadays an important area. In this handbook, there is a chapter specifically devoted to these problems in the Applications section. In this chapter, we consider the Maximum Diversity Problem (MDP) to illustrate the implementation of Scatter Search to solve an NP-hard optimization problem.

The MDP can be trivially formulated in mathematical terms as a quadratic binary problem, based on the pairwise distances d_{ij}, for elements i and j, where variable x_i takes the value 1 if element i is selected and 0 otherwise, $i = 1, \ldots, n$.

$$\begin{aligned} \text{Maximize} \quad & \sum_{i<j} d_{ij} x_i x_j \\ \text{subject to:} \quad & \sum_{i=1}^{n} x_i = m \\ & x_i \in \{0, 1\} \qquad i = 1, \ldots, n. \end{aligned} \tag{1}$$

Figure 4 shows a timeline diagram, which depicts the first contributions on diversity problems from an optimization perspective. In particular, it starts with the first publication on some discrete versions of dispersion problems in general graphs [31]. Kuby introduced the p-dispersion problem as locating p facilities on the nodes of a network, so that the minimum distance between any pair of facilities is maximized. The author proposed a linear integer formulation for this problem and applied it to a small example with 25 nodes. The author also extended the model to the MaxSum case, in which the objective is to maximize the sum of distances between all the pairs of selected facilities (nodes). These problems were later coined as the MaxMin Diversity Problem (MMDP) and MaxSum Diversity Problem (MDP), respectively.

Erkut and Neuman published in 1989 an invited review in the European Journal of Operational Research on location models for obnoxious facilities, where a function distance is maximized. Erkut also introduced a simple heuristic and a branch and bound exact method to solve small problems (with up to 40 elements) to optimality [8]. The construction is coupled with a local search method in the heuristic that scans the set of selected elements in search of the best exchange to replace a selected element with an unselected one. The method performs moves as long as the objective value increases, and stops when no improving exchange can be found. This local search has been applied to most of the algorithms proposed for both the MaxSum (MDP) and the MaxMin (MMDP). We will also apply it in our computational testing.

Kincaid proposed two heuristics for the MaxMin [28], also known as the discrete p-dispersion problem, based on exchanges: a simulated annealing (SA) heuristic

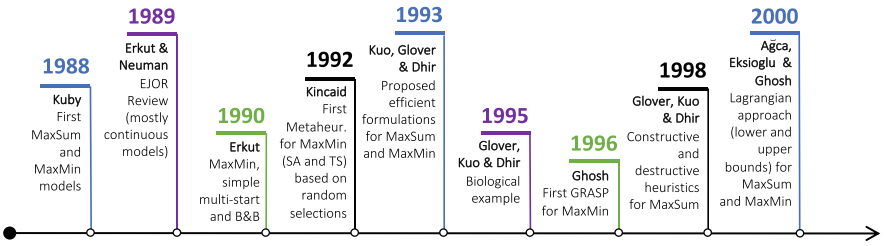


Fig. 4 Timeline of first diversity models and algorithms

and a tabu search (TS) heuristic. In a given iteration, these heuristics generate a random move (an exchange between a selected and an unselected element) and apply the standard acceptance rules of the methodology, the so-called temperature and cooling schedule in the SA, and the tabu status and aspiration criteria in tabu search. These methods were also adapted to the MaxSum problem and in the later years received a lot of attention, such as [10] and [32]. We refer the reader to the recent survey by Martí et al. [46] for a detailed description of the models and methods in diversity optimization.

Scatter Search for the MDP

As described in section "The Diversification Generator Method," and shown in Fig. 3, the SS method starts by generating a set P of diverse solutions. Then, b of them are selected to create the *RefSet*. In our implementation for the MDP, we follow the standard design of selecting the best $\frac{b}{2}$ solutions in P, and then the most diverse $\frac{b}{2}$ solutions with respect to the solutions already in *RefSet* (see section "Reference Set Initialization"). The main loop of the method consists of applying the combination method to all pairs of solutions in *RefSet* and updating this set with the new solutions obtained from these combinations. The method finishes when no new solutions are added to *RefSet*.

The Diversification Generator Method

A DGM for the MDP could consist of a random selection of m elements from all n elements in the problem, but we cannot expect such a random solution to have a good value. Although in the original design of some evolutionary methods the random generation of the initial population was a well-accepted strategy, Scatter Search generates solutions based on a principle that induces both quality and diversity in the original population. In the MDP, such principle should somehow reflect that we are searching for a subset of elements with a large sum of distances. In particular, we apply the GRASP metaheuristic to create the initial set of solutions.

Let $M \subseteq V$ be the set of selected elements. Initially, M is empty, and we add elements to M until it contains m elements (i.e., until $|M| = m$). For each element i, we define an evaluation function, $e(i)$, to measure its potential contribution to the partial solution under construction (i.e., the sum of its distances to the elements already in the solution).

$$e(i) = \sum_{j \in M} d_{ij}. \tag{2}$$

As described in the chapter on the GRASP methodology, at each iteration of the construction phase, GRASP maintains a candidate list of elements CL, which can

be added to the partial solution under construction, to obtain a complete solution. CL is simply computed in a given iteration as the list of elements not in the current partial solution. All candidate elements are evaluated to create a restricted candidate list RCL with the best elements in CL, i.e., those with the largest incremental contribution in a maximization problem such as the MDP.

Algorithm 1: Constructive randomized method

1: Let $G = (V, E)$ be a graph with distance matrix $D = (d_{ij})$
2: $i \leftarrow$ `SelectRandom`(V)
3: $M \leftarrow \{i\}$ and $f(M) = 0$
4: $CL = V \setminus M$
5: $e(j) = d_{ji}\ \forall j \in CL$
6: **while** $|M| < m$ **do**
7: $\quad e_{max} \leftarrow \max_{j \in \text{CL}} e(j)$
8: $\quad e_{min} \leftarrow \min_{j \in \text{CL}} e(j)$
9: $\quad RCL \leftarrow \{j \in CL \mid cont(j) \geq e_{max} - \alpha \cdot (e_{max} - e_{min})\}$
10: $\quad i \leftarrow$ `SelectRandom`(RCL)
11: $\quad M \leftarrow M \cup \{i\}$ and $f(M) = f(M) + e(i)$
12: $\quad$ CL $\leftarrow$ CL $\setminus \{i\}$
13: $\quad e(j) = e(j) + d_{ij}\ \forall j \in CL$
14: **end while**
15: **return** M, $f(M)$

Algorithm 1 shows a pseudo-code of the GRASP construction. The partial solution set M is initialized with a random element in step 3, and the candidate set of elements, CL, which contains all nonselected elements ($CL = V \setminus M$) is created in step 4. The initialization of the method finishes in step 5, in which the contribution of all the candidate elements is established as the sum of the distances to the element in M. The *While* loop in Algorithm 1 takes place from steps 6 to 15, reflecting the successive applications of this part of the code. In each one, the RCL is created, an element $i \in RCL$ is randomly selected, and the different elements of the algorithm are updated. We refer the reader to ▸ Chap. 23, "GRASP," for a detailed description on how this method works.

Reference Set Initialization

As mentioned above, to create the reference set, RefSet, the notion of best solutions is not limited to their quality, as measured by the objective function, since it also considers their diversity. In particular, our method first chooses $b/2$ solutions attending to their quality. We simply order the solutions in P according to their objective function value, and introduce them, one by one, in RefSet avoiding to repeat any solution. The rest of the solutions of the RefSet are then selected from P by a diversity criterion as follows.

The definition of distance between solutions is a key design issue in scatter search implementations. Distance is used to measure how diverse one solution is with respect to a set of solutions. Specifically, for the MDP, the distance between a trial solution x in P and the solutions in the RefSet is computed as the sum of the minimum distances from each element in x and the set formed with all the elements in the RefSet solutions. Note that if the element in x is in one or more solutions in the RefSet, then its corresponding minimum distance is 0. The trial element in P with the largest sum of minimum distances is selected for inclusion in the RefSet. Since the solutions are added one at a time, the distance calculations have to be updated before the next solution is selected.

The Subset Generation Method

The SGM that we propose for the MDP works by generating subsets defined by any two different solutions in *RefSet*. To avoid the repetition of solutions generated in previous iterations, each subset is generated only if at least one of its solutions was introduced in *RefSet* in the preceding iteration. Suppose that m subsets were not considered for this reason, then the number of resulting subsets at each iteration for which the combination method will be applied is $\frac{b^2-b}{2} - m$.

The Solution Combination Method

The combination method applies a restricted version of the constructive method based on a semi-greedy algorithm described in the Diversification Generation Method. In particular, given two solutions x and y, the combination method considers the set of elements formed with the union of the elements in x and y. We apply the constructive method to this set of candidate elements called $C(x, y)$. In other words, the candidate list CL with the elements that can be included in the solution M that we are creating as a combination of x and y is $CL = C(x, y)$, and therefore only the elements in CL can be selected to be included in M. The solution combination method applies the same steps of the constructive method, and it creates in each iteration the restricted candidate list, RCL, from which an element is randomly selected. The combination method stops when m elements have been selected from CL. Note that since the method makes random selections, it can be applied several times to the same pair of solutions and may result in different combined solutions (offspring in the evolutionary terminology).

The Reference Set Update Method

The *RefSet* update operation consists of maintaining a record of the b best solutions found so far by the procedure. The issues related to this updating function are straightforward: All the solutions in *RefSet* that are worse than those in the current

Pool will be replaced by these ones, with the aim of keeping in *RefSet* the b best and most different solutions found so far. Let $s \in Pool$ and $w \in RefSet$ such that the objective function value of s is less than that of w. In this case, s will replace w if s is different from all other solutions in *RefSet*. Note that the update method focuses on quality. In other words, although *RefSet* was created considering both quality and diversity, when updating it, we only consider the objective function value of the candidate solutions obtained from the combination method. In particular, once all the combinations have been performed and the new solutions have been moved to $Pool$, the new *RefSet* simply contains the best b solutions in the set formed with the previous $RefSet \cup Pool$. This strategy, based on quality, favors the convergence of the method.

The Improvement Method

Ghosh implemented in 1996 a simple local search method LS [10] in the context of GRASP. It scans the set of selected elements in search of the best exchange to replace a selected element with an unselected one. The method performs moves as long as the objective value increases, and it stops when no improving exchange can be found. Duarte and Martí [6] proposed in 2007 an improved variant of this classic local search. In particular, this variant selects the element $i*$ that provides the smallest contribution to the objective function value of the current solution. Then, it searches for an element j to be exchanged with element i*. The first element j that results in an improving move is selected, and the exchange is performed without examining the remaining unselected elements. If no improving move can be found to exchange element i*, then the selected element with the next smallest contribution is examined. This process continues until no improving exchange can be found. We implement this local search as the improvement method in scatter search.

Computational Experiments

This section describes the computational experiments performed first to find the parameter values of the scatter search method and then to study its performance. The procedure has been implemented in C using GCC 4.8.2, and the results reported in this section have been obtained with an Intel core i7–3770 at 3.40 GHz and 16 GB of RAM.

We have tested our algorithms on the three sets of instances previously reported. The benchmark instances for the MDP come from different sources that have been added over the years. The most used library is the MDPLIB available at www.uv.es/rmarti/paper/mdp.html with a mirror server in www.optsicom.es/mdp. It compiles a comprehensive set of benchmark instances representative of the collections used for computational experiments in the MDP. The library contains three sets of instances collected from different papers and named after their authors: GKD (Glover, Kuo,

and Dhir), MDG (Martí, Duarte, and Gallego), and SOM (Silva, Ochi, and Martins). We use these three sets in our experimentation.

The experiments are divided into three main blocks. The first block, described in section "Search Methods," is devoted to study the performance of the search methods, namely construction, improvement, and combination methods in scatter search. In the second subsection, Search profiles, we study the evolution of the best solution found in an attempt to disclose the contribution of the different steps of the algorithm to the final solution. Finally, in the third subsection, we determine the best values for the key search parameters. The paper finishes with a four subsection with a final comparison of the best SS variant and the reference GRASP algorithm. We employ two metrics to measure the performance of the algorithms tested. Dev is the average percentage deviation with respect to the best solution known (as reported in the MDPLIB), and CPU is the average computing time in seconds. We add into the final comparison a third metric reporting the number of best-known solutions that each method is able to match.

Search Methods

In our first experiment, we undertake to evaluate the contribution of the local search in the GRASP algorithm that we implement as the diversification generation method. Table 2 shows the Dev and CPU of the best solution obtained with the Constructive Method (CONST.) and the complete GRASP, in which the constructive is coupled with a local search. To do so, we generate 100 solutions with each method and report the value of the best solution across the 100. Additionally, this table also reports the best solution obtained with a pure random construction in which the m elements in the solution are randomly selected with a uniform distribution (RANDOM). This random method is repeated 100 times, as the other two in the comparison, and the best solution is returned as the output of the method.

As expected, results in Table 2 clearly show the important contribution of the local search method. In particular, it is able to reduce the deviation with respect to the best-known solution from 9.04 on average for the constructive method, to 0.04 when it is coupled with the local search. Note however that this reduction comes with a price in the computational time that increases from 2.36 for the construction

Table 2 Diversification generator method

	Dev. (%)			CPU		
Method	CONST.	GRASP	RANDOM	CONST.	GRASP	RANDOM
GKD	7.5	0.0	15.8	2.31	35.15	0.09
SOM	8.0	0.2	14.7	2.35	72.68	0.15
MDG	11.6	0.9	17.8	2.41	34.56	0.08
Average	9.04	0.4	16.1	2.36	47.47	0.11

to 47.47 s for the complete GRASP. This table also shows that the performance of the entirely random construction is poor, with a deviation of 16.1%.

We applied the nonparametric Friedman test for multiple correlated samples to the best solutions obtained by each of the three methods. This test computes, for each instance, the rank value of each method according to solution quality, and then it calculates the average rank values of each method across all the instances solved. If the averages differ greatly, the associated p-value or significance will be small. The resulting p-value of $2.2e-16$ obtained in this experiment clearly indicates that there are statistically significant differences among the three methods tested. We also compared GRASP and CONST. with the Wilcoxon nonparametric test for pairwise comparison. This test answers the question: Do the two samples (solutions obtained with the two methods in our case) represent two different populations? The resulting p-value of $5.7e-11$ indicates that the values are significantly different and confirms the superiority of GRASP as a diversification generation method.

This first experiment also permits us to identify the most challenging instances in the MDPLIB. We therefore do not include in our testbed those instances that can be considered easy to solve. Note that if the GRASP method that we used to populate the Reference Set is able to match the best-known solution in a given instance, then that instance could not help us to evaluate the contribution of the other elements in our scatter search algorithm. We therefore limit our testbed to 20 instances in the GKD-c set with $n = 500$ elements, 20 instances in the SOM-b set with n ranging from 100 to 500, and 20 instances in the MDG-a set with $n = 500$.

In our second experiment, we study the variability of the solutions generated with the constructive method and the complete GRASP applied in the first experiment. Table 3 shows the values of the best and the worst solutions across the 100 generated with each method. Specifically, the constructive method generates solutions that on average have a value ranging from 14,159.49 to 14,935.47, while the GRASP solutions range from 15,793.87 to 15,995.89, while the best-known value is on average in 16,028.32. This experiment not only confirms the important contribution of the local search, but also shows that there is room for improvement in the instances selected.

Figure 5 complements the information in Table 3 by showing a boxplot of the values obtained with each method on each instance. The distribution of the values in the figure clearly shows that GRASP is able to outperform the constructive method, since its box is in a higher position, and the random construction ranks the last one.

Table 3 Best and worst values of constructive method and GRASP

		CONST.		GRASP	
Method	Best known	Best	Worse	Best	Worse
GKD	19,502.55	18,042.80	16,985.07	19,502.55	19,489.46
SOM	20,811.30	19,896.95	19,078.45	20,781.25	20,459.05
MDG	7771.10	6866.64	6414.94	7703.88	7433.09
Average	16,028.32	14,935.47	14,159.49	15,995.89	15,793.87

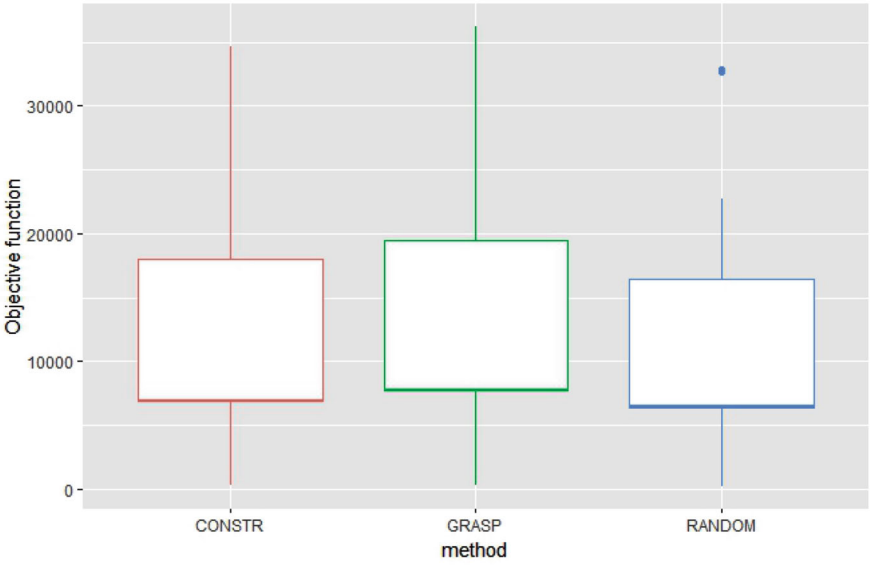


Fig. 5 Boxplot of solutions' values

The median value of GRASP is 9544, while the median values of CONST. and RANDOM are 8932 and 8248, respectively.

Figure 5 shows the superiority of GRASP with respect to the constructive method. However, in spite of removing from the figure two outlayers with values close to 100,000, the large magnitude of the units makes it difficult to estimate the relative improvement of GRASP over the constructive. Figure 6 represents this improvement for each type of instance. It shows that in the GKD and MDG instances the improvement is consistent, with values around 1,500 and 900, respectively, while in the SOM instances we can observe a large variability probably due to the variability in the data.

To evaluate the contribution of the combination method, our third experiment reports the solution of the entire Scatter Search, which includes the generation of initial solutions with GRASP and also the combination of the selected solutions in the RefSet. In this experiment, we report the best combined solution in the RefSet as the output of the method, and we do not consider its inclusion in the RefSet to perform further iterations. Table 4 clearly shows that when the entire SS method is applied, the deviation of the best solution with respect to the best known is very small. In particular, in the 20 instances of the GKD set, it is 0.00%, in the SOM 0.13%, and in the MDG 0.31%, with an overall average deviation of 0.15%, which compares favorably with the 0.4% obtained with GRASP, shown in Table 2. We can therefore conclude that the combination method applied to the elite solutions in the RefSet is able to improve the quality of the solutions in the initial population (those obtained with GRASP) in 0.25% (Table 4).

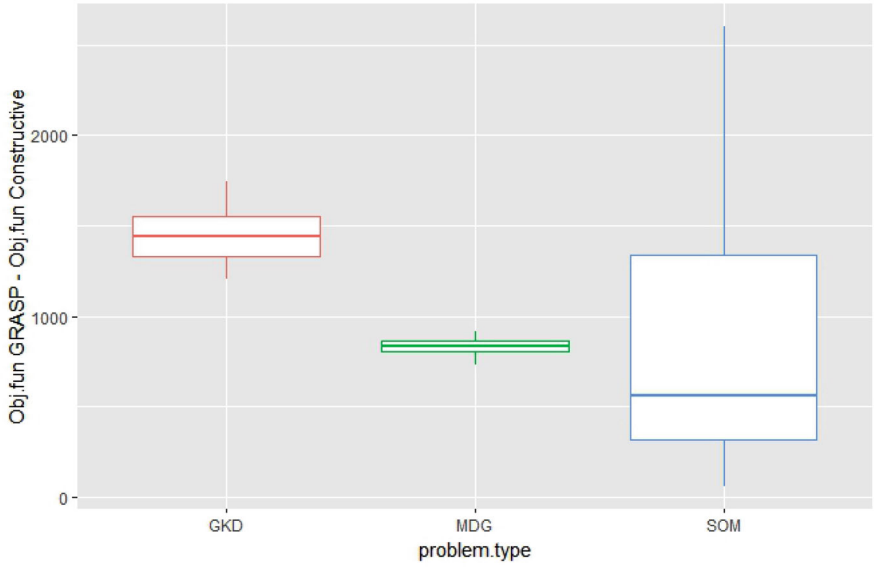


Fig. 6 Improvement of GRASP w.r.t. constructive

Table 4 Scatter search (one iteration)

	DEV. %	CPU
GKD	0.00	4.04
SOM	0.13	36.27
MDG	0.31	29.53
Average	0.15	23.28

Search Profiles

In this subsection, we apply the SS algorithm several iterations, which implies that we update the reference set with the solutions that result from the combination of the RefSet solutions in the previous iteration. Note that the update takes place when the combined solution qualifies to enter in the RefSet (i.e., if its value improves upon the worst value in the RefSet). In this way, we call a step to a complete iteration, and SS performs steps as long as the combined solutions qualify to enter in the RefSet. We consider four challenging instances in this experiment and report the best value obtained with SS in each step. We represent in a diagram the initial value obtained with the best value in the initial population (GRASP) and the best value in each step of the SS algorithm to visualize the search profile of the best value so far. The following four figures show respectively the search profile of the SOM-n400, SOM-n500, MDG-a2-n500, and MDG-a17-n500. Figure 11 shows the profile of the four instances in the same diagram for an easy comparison.

The figures with the individual search profiles (Figs. 7, 8, 9, and 10), together with the combined profiles in Fig. 11, show that in these four difficult instances,

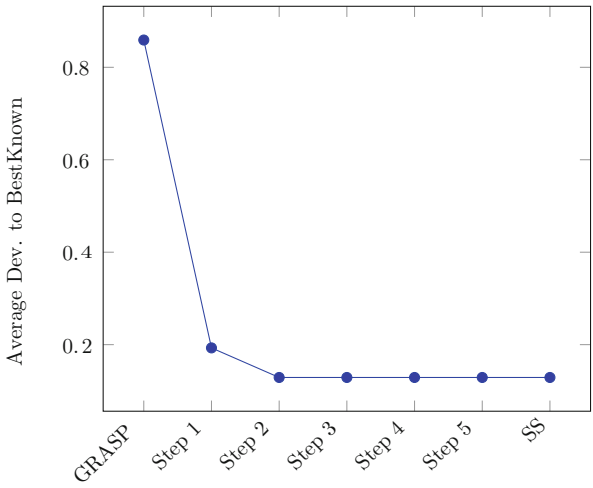


Fig. 7 Search profile of an SOM instance (n = 400, m = 40)

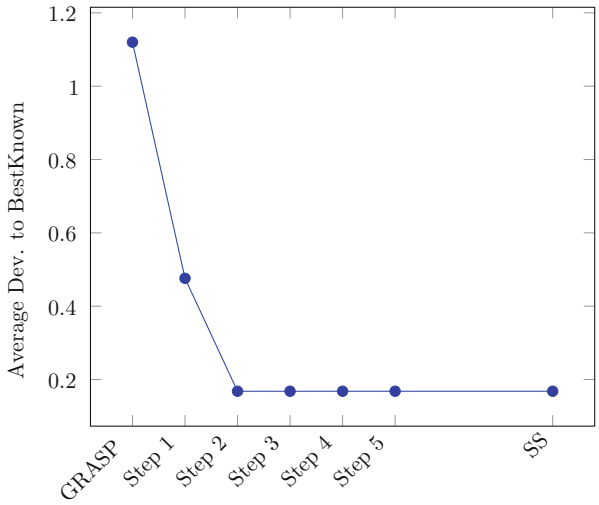


Fig. 8 Search profile of an SOM instance (n = 500, m = 50)

performing consecutive steps of the SS algorithm is able to marginally improve the best solution obtained in the first step of the method. As a matter of fact, after five steps, SS is able to match the best solution known for some instances (with a 0.0% deviation), while in other instances it exhibits a very small deviation value of 0.1%. In all the cases, after five complete iterations (steps), the method is not able to further improve the solutions, so this experiment also indicates the search horizon and termination criteria in terms of solution quality.

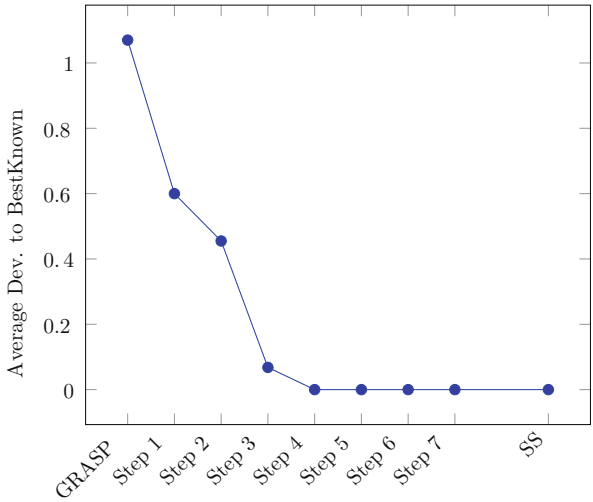


Fig. 9 Search profile of the MDG-a2 instance (n = 500, m = 50)

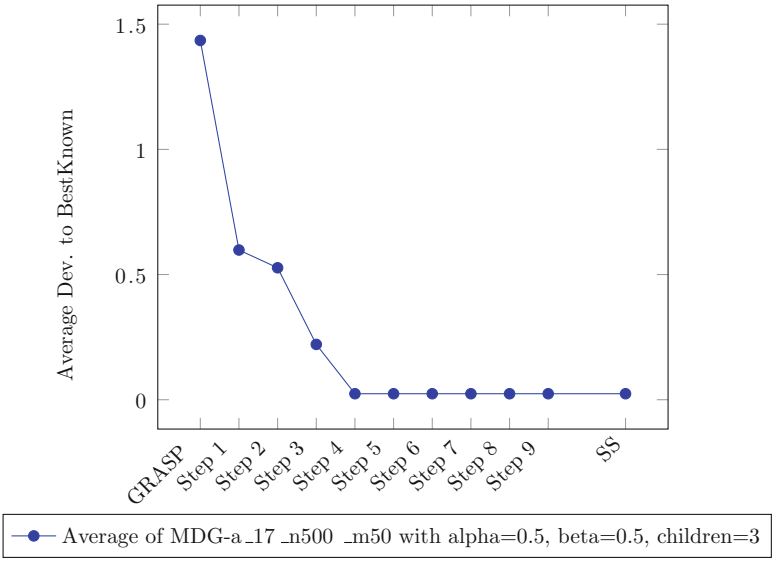


Fig. 10 Search profile of the MDG-a17 instance (n = 500, m = 50)

Search Parameters

In this subsection, we examine the impact of some parameters in the performance of the SS algorithm. In our first experiment here, we consider the so-called α value in GRASP, which determines the balance between randomization and greediness in the semi-greedy construction. Table 5 shows the results obtained with three different

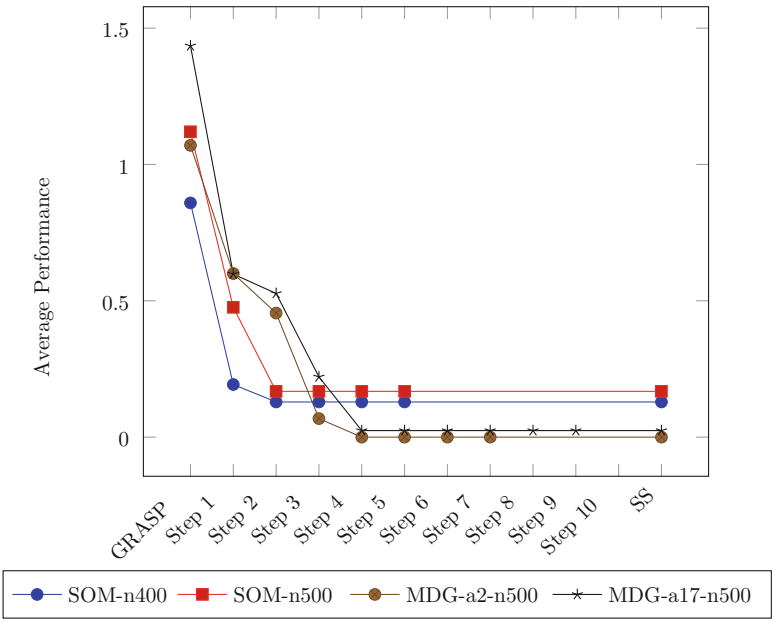


Fig. 11 Search profiles

Table 5 Construction parameter

	Dev. (%)			CPU		
α	0.3	0.5	0.7	0.3	0.5	0.7
GKD	0.00	0.00	0.00	5.28	5.41	5.25
SOM	0.07	0.07	0.04	69.46	63.00	73.67
MDG	0.09	0.08	0.13	66.00	61.65	63.60
Average	0.05	0.05	0.05	46.92	43.36	47.51

values of this parameter: 0.3, 0.5, and 0.7. This table shows that this parameter has no influence in the final solution's quality.

As explained in the algorithm's description, our combination method adapts the GRASP construction to the subset of elements in the union of the two solutions under combination. Then, it is also based on a search parameter to balance the RCL composition, as the α does in the constructive method. We call it β, and Table 6 reports the results of the SS algorithm with three values of this parameter: 0.3, 0.5, and 0.7. Results in this table show that $\beta = 0.7$ provides the best results.

We now consider the number of solutions obtained with the combination of two solutions in the reference set. As described above, the random component of the combination method permits to apply it repeatedly to the same pairs of solutions, thus obtaining more than one combined solution. We call Combined to this parameter. Table 7 shows the results of three values for this parameter: 1, 2, and 3. This table shows that the best results are obtained with Combined = 3.

Table 6 Combination parameter

	Dev. (%)			CPU		
β	0.3	0.5	0.7	0.3	0.5	0.7
GKD	0.00	0.00	0.00	3.78	5.41	5.84
SOM	0.03	0.07	0.02	50.37	63.00	85.49
MDG	0.08	0.08	0.07	40.60	61.65	76.67
Average	0.04	0.05	0.03	31.59	43.36	56.01

Table 7 Number of combined solutions

	Dev. (%)			CPU		
Combined	1	2	3	1	2	3
GKD	0.00	0.00	0.00	2.50	5.41	7.21
SOM	0.03	0.07	0.02	28.67	63.00	81.35
MDG	0.12	0.08	0.04	28.68	61.65	82.75
Average	0.05	0.05	0.02	19.96	43.36	57.11

Table 8 Parameters exploration

Variant	α	β	Combined
C1	0.5	0.5	2
C2	0.3	0.5	2
C3	0.7	0.5	2
C4	0.5	0.5	3
C5	0.5	0.5	1
C6	0.5	0.7	2
C7	0.5	0.3	2

We now complement the sequential design on the three parameters α, β, and *combined* shown above, in which we studied the value of one parameter, while the other two were kept fixed, with a more complete design. We consider seven variants of the scatter search algorithm according to the values shown in Table 8.

Figure 12 shows the boxplot obtained with the deviation of the solution values of each variant with respect to the best-known solutions. According to the boxplot represented in this figure, it seems that combination C4 with $\alpha = 0.5$, $\beta = 0.5$, and Combined = 3 is the one with the best performance. However, a Friedman test applied to the values depicted in this table resulted in a p-value of 0.509, so we cannot affirm that this variant is in general statistically better than the others, and just limit our conclusions to this set of data tested.

Final Comparison

Once we have established the contribution of each element and the best values of each search parameter in the SS algorithm, we present a final comparison with the previous GRASP method, considered a reference in the MDP. We run SS with $\alpha = 0.5$, $\beta = 0.5$, Combined = 3. We cannot present here the results of all previous heuristics for this problem because there are many, but instead, we consider the

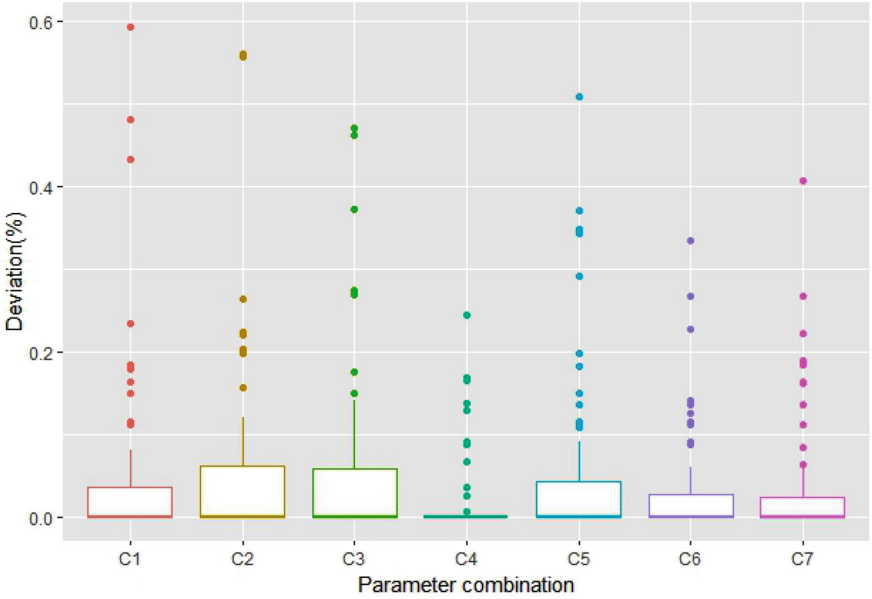


Fig. 12 Deviation of each SS variant w.r.t. best-known solutions

Table 9 Comparison between SS and GRASP

		DEV. (%)	CPU	NumBest			
Inst.	# Inst.	SS	GRASP	SS	GRASP	SS	GRASP
GKD	19	0.00	0.01	10.91	32.42	19	19
SOM	20	0.02	0.26	81.35	61.26	17	6
MDG	20	0.04	0.83	82.75	29.16	13	0
Summary	59	0.02	0.37	58.35	40.95	49	25

best-known values obtained with the most efficient, reported in the MDPLIB, which represent the state of the art in this problem. Table 9 reports the Average deviation (DEV.), computer time in seconds (CPU), and the number of best-known solutions (NumBest) that each method is able to find. These results clearly show the quality of the SS solutions in terms of both the previous GRASP and the reference best values.

To conclude our experimentation, we perform a nonparametric statistical test to compare the results of the two methods. The resulting p-value in the Friedman test of 5.5e-09 clearly indicates that there are significant differences between the methods. We represent in Fig. 13 a boxplot of the differences between the solutions' values of both methods divided by type of instance. The boxplot confirms the superiority of SS over GRASP. Additionally, it permits to identify that the GKD instances are solved by the two methods, while on the contrary in the MDG the differences are larger and can be considered more difficult to solve. Future studies should focus on these challenging instances.

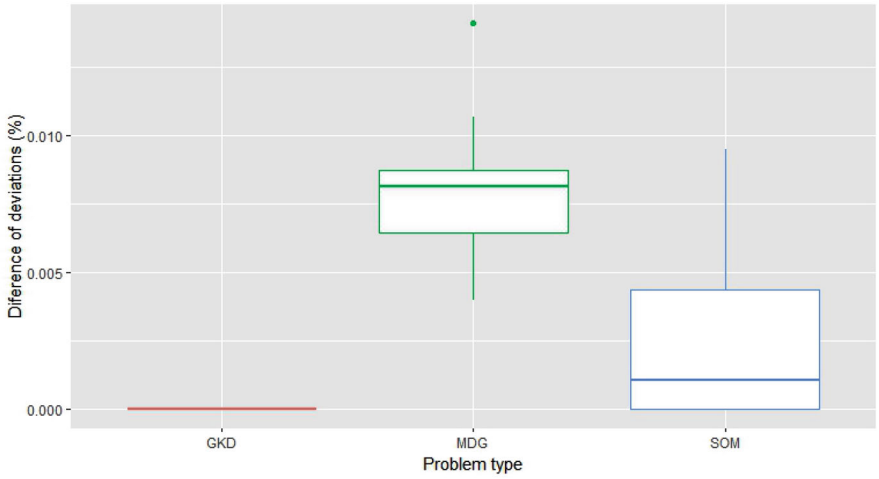


Fig. 13 Differences between SS and GRASP solution values by type of instance

Conclusions

In this chapter, we summarize our experiences on the subject of implementing scatter search. We have experimented with this methodology for a number of years, and as a result of implementing different search strategies, and performing extensive experimental testing, we have learned some valuable lessons condensed above. As it is well known, metaheuristic methodologies are based on principles and not necessarily on theory that can be spelled out with theorems and proofs. Hence, to illustrate and prove the efficiency of search mechanisms, strategies, and key elements in scatter search, we have shown how to solve an important and difficult combinatorial optimization problem: the maximum diversity problem. In particular, we describe in detail how to construct, improve, and combine solutions for this $\mathcal{NP}$-hard problem in an efficient way, i.e., with fast methods that are able to produce high-quality solutions. These research opportunities carry with them an emphasis on producing systematic and strategically designed rules, rather than following the policy of relegating decisions to random choices, as often is fashionable in evolutionary methods.

Cross-References

▸ Diversity Optimization Models
▸ Evolution Strategies
▸ Genetic Algorithms
▸ Network Optimization

Acknowledgments The authors have been supported by grant PID2021-125709OB-C21 and PID2024-160226OB-C21 funded by MCIN/AEI/ 10.13039/501100011033 and by ERDF, a way of making Europe. They also received financial support from the Generalitat Valenciana (code CIAICO/2021/224). We thank the European Union, the Spanish government (Ministerio de Ciencia e Innovación), and the Valencian government for their financial support.

Competing Interest Declaration The author(s) has no competing interests to declare that are relevant to the content of this manuscript.

References

1. Alkhazaleh HA, Ayob M, Ahmad Z (2013) Scatter search for solving team orienteering problem. Res J Appl Sci 8(3):181–190
2. Bernardino AM, Bernardino EM, Sánchez-Pérez JM, Gómez-Pulido JA, Vega-Rodríguez MA (2012) Solving SONET problems using a hybrid scatter search algorithm. In: Madani K, Dourado C, Rosa A, Filipe J (eds) Computational intelligence. Studies in computational intelligence, vol 399. Springer, Berlin/Heidelberg, pp 81–97
3. Botón-Fernández M, Vega-Rodríguez MA, Prieto-Castrillo F (2013) An efficient and self-adaptive model based on scatter search: solving the grid resources selection problem. In: Moreno-Díaz R, Pichler F, Quesada-Arencibia A (eds) Computer aided systems theory – EUROCAST 2013. Lecture notes in computer science, vol 8111. Springer, Berlin/Heidelberg, pp 396–403
4. Bova N, Ibáñez Ò, Cordón Ó (2013) Image segmentation using extended topological active nets optimized by scatter search. IEEE Comput Intel Mag 8(1):16–32
5. de Athayde Costa e Silva M, Klein CE, Mariani VC, Dos Santos Coelho L (2013) Multiobjective scatter search approach with new combination scheme applied to solve environmental/economic dispatch problem. Energy 53:14–21
6. Duarte A, Martí R (2007) Tabu search and grasp for the maximum diversity problem. Eur J Oper Res 178:71–84
7. Engin O, Ylmaz MK, Baysal ME, Sarucanl A (2013) Solving fuzzy job shop scheduling problems with availability constraints using a scatter search method. J Multiple-Valued Logic Soft Comput 21(3–4):317–334
8. Erkut E (1990) The discrete p-dispersion problem. Eur J Oper Res 46(1):48–60
9. Everett H (1963) Generalized Lagrangean multiplier method for solving problems of optimal allocation of resources. Oper Res 11:399–417
10. Ghosh JB (1996) Computational aspects of the maximum diversity problem. Oper Res Lett 19(4):175–181
11. Glover F (1965) A multiphase-dual algorithm for the zero-one integer programming problem. Oper Res 13:879–919
12. Glover F (1977) Heuristics for integer programming using surrogate constraints. Decis Sci 8(7):156–166
13. Glover F (1994) Tabu search for nonlinear and parametric optimization with links to genetic algorithms. Discret Appl Math 49(1–3):231–255
14. Glover F (1998) A template for scatter search and path relinking. Lecture notes in computer science, vol 1363, Springer, Berlin, Heidelberg
15. Glover F, Laguna M, Martí R (2000) Fundamentals of scatter search and path relinking. Control Cybern 29(3):652–684
16. Guo XW, Liu SX, Wang DZ (2012) Scatter search for solving multi-objective disassembly sequence optimization problems. J Northeastern Univ 33(1):56–59
17. Habibi MR, Rashidinejad M, Zeinaddini-Meymand M, Fadainejad R (2014) An efficient scatter search algorithm to solve transmission expansion planning problem using a new load shedding index. Int Trans Electric Energy Syst 24(2):153–165

18. Hariharan R, Golden Renjith Nimal RJ (2014) Solving flow shop scheduling problems using a hybrid genetic scatter search algorithm. Middle – East J Sci Res 20(3):328–333
19. Holland JH (1975) Adaptation in natural and artificial systems. University of Michigan Press, Ann Arbor
20. Hu L, Jiang Y, Zhu J, Chen Y (2013) Hybrid of the scatter search, improved adaptive genetic, and expectation maximization algorithms for phase-type distribution fitting. Appl Math Comput 219(10):5495–5515
21. Huacuja HJF, Valdez GC, Rangel RAP, Barbosa JG, Reyes LC, Valadez JMC, Soberanes HJP, Villanueva DT (2012) Scatter search with multiple improvement methods for the linear ordering problem. Malaysian J Comput Sci 25(2):76–89
22. Hvattum LM, Duarte A, Glover F, Martí R (2013) Designing effective improvement methods for scatter search: an experimental study on global optimization. Soft Comput 17(1):49–62
23. Ibáñez O, Cordón O, Damas S, Santamaría J (2012) An advanced scatter search design for skull-face overlay in craniofacial superimposition. Expert Syst Appl 39(1):1459–1473
24. Jabal-Ameli MS, Moshref-Javadi M (2014) Concurrent cell formation and layout design using scatter search. Int J Adv Manuf Technol 71(1–4):1–22
25. Jabal-Ameli MS, Moshref-Javadi M, Tabrizi BB, Mohammadi M (2013) Cell formation and layout design with alternative routing: a multi-objective scatter search approach. Int J Ind Syst Eng 14(3):269–295
26. Jaradat G, Ayob M, Ahmad Z (2014) On the performance of scatter search for post-enrolment course timetabling problems. J Comb Optim 27(3):417–439
27. Kashefi AH, Meshkin A, Zargoosh M, Zahiri J, Taheri M, Ashtiani S (2013) Scatter-search with support vector machine for prediction of relative solvent accessibility. EXCLI J 12:52–63
28. Kincaid RK (1992) Good solutions to discrete obnoxious location problems via metaheuristics. Ann Oper Res 40(1):265–281
29. Kothari R, Ghosh D (2014) A scatter search algorithm for the single row facility layout problem. J Heuristics 20(2):125–142
30. Krishnan M, Karthikeyan T, Chinnusamy TR, Venkatesh Raja K (2012) A novel hybrid metaheuristic scatter search-simulated annealing algorithm for solving flexible manufacturing system layout. Eur J Sci Res 73(1):52–61
31. Kuby MJ (1988) Programming models for facility dispersion: the p-dispersion and maxisum dispersion problems. Math Comput Model 10(10):792
32. Kuo C-C, Glover F, Dhir KS (1993) Analyzing and modeling the maximum diversity problem by zero-one programming. Decis Sci 24(6):1171–1185
33. Laguna M, Martí R (2002) Scatter search: methodology and implementations in C. Kluwer Academic Publishers, Boston
34. Laguna M, Gortázar F, Gallego M, Duarte A, Martí R (2014) A black-box scatter search for optimization problems with integer variables. J Glob Optim 58(3):497–516
35. Le Q, Yang G, Hung W, Guo W (2012) Reliable NoC mapping based on scatter search. In: Liu B, Ma M, Chang J (eds) Information computing and applications. Lecture notes in computer science, vol 7473. Springer, Berlin/Heidelberg, pp 640–647
36. Le Q, Yang G, Hung WNN, Zhang X, Fan F (2014) A multiobjective scatter search algorithm for fault-tolerant NoC mapping optimisation. Int J Electron 101(8):1056–1073
37. Li J, Prins C, Chu F (2012) A scatter search for a multi-type transshipment point location problem with multicommodity flow. J Intel Manuf 23(4):1103–1117
38. Li VC, Liang YC, Sun YY, Chen YS (2012) A scatter search method for the multidimensional knapsack problem with generalized upper bound constraints. J Chin Inst Ind Eng 29(8):559–571
39. Li ZZ, Song XY, Sun JZ, Huang ZT (2012) A scatter search methodology for the aircraft conflict resolution problem. In: Huang D, Jiang C, Bevilacqua V, Figueroa JC (eds) Intelligent computing technology. Lecture notes in computer science, vol 7389. Springer, Berlin/Heidelberg, pp 18–24
40. Lin SW, Chen SC (2012) Parameter determination and feature selection for c4.5 algorithm using scatter search approach. Soft Comput 16(1):63–75

41. Liu Q, Wang WX, Zhu KR, Zhang CY, Rao YQ (2014) Advanced scatter search approach and its application in a sequencing problem of mixed-model assembly lines in a case company. Eng Optim 46(11):1485–1500
42. Lv Y, Wang G, Tang L (2014) Scenario-based modeling approach and scatter search algorithm for the stochastic slab allocation problem in steel industry. ISIJ Int 54(6):1324–1333
43. Martí R (ed) (2006) Scatter search methods for optimization. Feature Cluster Eur J Oper Res 169
44. Martí R, Corberán Á, Peiró J (2015) Scatter search for an uncapacitated p-hub median problem. Comput Oper Res 58(0):53–66
45. Martí R, Corberán Á, Peiró J (2018) Scatter search. In: Martí R, Pardalos (eds) Handbook of heuristics. Springer Nature, Cham, pp 717–740
46. Martí R, Martínez-Gavara A, Pérez-Peló S, Sánchez-Oro J (2022) A review on discrete diversity and dispersion maximization from an OR perspective. Eur J Oper Res 299(3):795–813
47. Meymand MZ, Rashidinejad M, Khorasani H, Rahmani M, Mahmoudabadi A (2012) An implementation of modified scatter search algorithm to transmission expansion planning. Turkish J Electric Eng Comput Sci 20(SUPPL.1):1206–1219
48. Naderi B, Ruiz R (2014) A scatter search algorithm for the distributed permutation flowshop scheduling problem. Eur J Oper Res 239(2):323–334
49. Pantrigo JJ, Duarte A (2013) Low-level hybridization of scatter search and particle filter for dynamic TSP solving. In: Alba E, Nakib A, Siarry P (eds) Metaheuristics for dynamic optimization. Studies in computational intelligence, vol 433. Springer, Berlin/Heidelberg, pp 291–308
50. Pantrigo JJ, Martí R, Duarte A, Pardo EG (2012) Scatter search for the cutwidth minimization problem. Ann Oper Res 199(1):285–304
51. Pendharkar PC (2013) Scatter search based interactive multi-criteria optimization of fuzzy objectives for coal production planning. Eng Appl Artif Intel 26(5–6):1503–1511
52. Resende MGC, Ribeiro CC, Glover F, Martí R (2010) Scatter search and path-relinking: fundamentals, advances, and applications. In: Gendreau M, Potvin JY (eds) Handbook of metaheuristics. International series in operations research and management science, vol 146. Springer, Cham, pp 87–107
53. Sadiq AT, Sagheer AM, Ibrahim MS (2012) Improved scatter search for 4-colour mapping problem. Int J Reason-Based Intel Syst 4(4):221–226
54. Salazar-Aguilar MA, Ríos-Mercado RZ, González-Velarde JL, Molina J (2012) Multiobjective scatter search for a commercial territory design problem. Ann Oper Res 199(1):343–360
55. Shen Z, Zou H, Sun H (2012) Task scheduling for imaging reconnaissance satellites using multiobjective scatter search algorithm. In: Li Z, Li X, Liu Y, Cai Z (eds) Computational intelligence and intelligent systems. Communications in computer and information science. Springer, Berlin/Heidelberg, pp 240–249
56. Tan Y, Cheng TCE, Ji M (2014) A multi-objective scatter search for the ladle scheduling problem. Int J Prod Res 52(24):7513–7528
57. Tang J, Zhang G, Zhang B, Cen H (2012) Resource-constrained project scheduling using electromagnetism-based scatter search. J Comput Inf Syst 8(12):5219–5227
58. Tavakkoli-Moghaddam R, Ranjbar-Bourani M, Amin GR, Siadat A (2012) A cell formation problem considering machine utilization and alternative process routes by scatter search. J Intel Manuf 23(4):1127–1139
59. Valsecchi A, Damas S, Santamaría J, Marrakchi-Kacem L (2014) Intensity-based image registration using scatter search. Artif Intel Med 60(3):151–163
60. Wang J, Hedar AR, Wang S, Ma J (2012) Rough set and scatter search metaheuristic based feature selection for credit scoring. Expert Syst Appl 39(6):6123–6128
61. Xu Y, Qu R (2012) A hybrid scatter search meta-heuristic for delay-constrained multicast routing problems. Appl Intel 36(1):229–241
62. Zhang T, Chaovalitwongse WA, Zhang Y (2012) Scatter search for the stochastic travel-time vehicle routing problem with simultaneous pick-ups and deliveries. Comput Oper Res 39(10):2277–2290

Genetic Algorithms

22

Carlos García-Martínez, Francisco J. Rodriguez, and Manuel Lozano

Contents

Introduction . . . 628
GA Construction . . . 630
Representation . . . 631
Initial Population: Generating Candidate Solution from Scratch . . . 632
Evaluation . . . 633
Selection Operator . . . 634
Genetic Operators . . . 634
Evolution Model and Replacement Strategy . . . 638
Stop Condition . . . 640
Diversification Techniques for GAs . . . 641
Methods to Generate Raw Diversity . . . 641
Strategies to Maintain Diversity . . . 642
The Crossover Operator as Diversification Agent . . . 644
Diversification by Adapting GA Control Parameters . . . 645
Diversity Preservation Based on Spatial Separation . . . 646
Hybrid GAs . . . 648
Collaborative Teamwork Hybrid GAs . . . 649
Collaborative Relay Hybrid GAs . . . 650
Integrative Teamwork Hybrid GAs . . . 650
Integrative Relay Hybrid GAs . . . 651
GA Applications . . . 652
Conclusion . . . 655
Cross-References . . . 655
References . . . 656

C. García-Martínez (✉)
Computer Science and Numerical Analysis, University of Córdoba, Córdoba, Spain
e-mail: cgarcia@uco.es

F. J. Rodriguez · M. Lozano
Department of Computer Science and Artificial Intelligence, University of Granada, Granada, Spain
e-mail: fjrodriguez@decsai.ugr.es; lozano@decsai.ugr.es

© Springer Nature Switzerland AG 2025

R. Martí et al. (eds.), *Handbook of Heuristics*,
https://doi.org/10.1007/978-3-032-00385-0_28

Abstract

This chapter presents the fundamental concepts of genetic algorithms (GAs) that have become an essential tool for solving optimization problems in a wide variety of fields. The first part of this chapter is devoted to the revision of the basic components for the design of GAs. We illustrate this construction process through its application for solving three widely known optimization problems such as knapsack problem, traveling salesperson problem, and real-parameter optimization. The second part of the chapter focuses on the study of diversification techniques that represent a fundamental issue in order to achieve an effective search in GAs. In fact, analyzing its diversity has led to the presentation of numerous GA models in the literature. Similarly, the hybridization with other metaheuristics and optimization methods has become a very fruitful research area. The third part of the chapter is dedicated to the study of these hybrid methods. In closing, in the fourth part, we outline the wide spectrum of application areas that shows the level of maturity and the wide research community of the GA field.

Keywords

Genetic algorithms components · Evolutionary computation · Population diversity · Hybrid genetic algorithms · Genetic algorithms applications

Introduction

Genetic algorithms, or GAs, are nowadays an important field in artificial intelligence and operations research, with more than 3,500 publications per year in the last 15 years. Contrary to their most common type of application, GAs were not initially presented in the 1970s for problem-solving, but as an instrument for simulating the biological evolution of adaptive natural systems [69]. Nevertheless, it was precisely that simulated adaptive ability what promoted, in the subsequent years, the opportunity for tackling optimization problems successfully [27, 28, 50]. In this chapter, we focus on their utility for problem-solving, so GAs can operationally be described as iterative algorithms that sample candidate solutions for a given problem, with the aim of reaching the best configurations.

GAs take their inspiration from the biological evolution of species. Thus, the following ideas belong to their core foundations:

- Species evolve in populations of individuals that compete for survival within an environment.
- New individuals appear because of the recombination of previous individuals.
- Darwin's theory of *natural selection* establishes that the fittest individuals, those better adapted to the environment, get higher chances of surviving and breeding.

Thus, less fitted individual may probably perish at some point of the evolution process.
- Individuals' adaptation to the environment strongly depends on their phenotypic characteristics, which, in turn, are principally determined by their genotype.
- Individuals might eventually suffer mutations at their genotype level, which may affect the phenotype and adequacy to the environment.

Using these ideas, the following principles govern most of GAs for optimization:

- The environment is defined by the problem to be addressed.
- Individuals, also known as *chromosomes*, represent candidate solutions for the problem.
- Their *genotypes* encode the candidate solutions for the problem. The genotype-phenotype translation establishes how the chromosomes should be interpreted to get the actual candidate solutions.
- The *fitness* of individuals depends on their adequacy on the given problem, so fitter individuals are more likely to survive and breed.
- There is an evolving *population of individuals*, into which new individuals may enter and others may perish.
- New individuals are generated as the consequence of the *recombination* and/or the *mutation* of previous ones.

It should be pointed out that similar approaches were being developed since the 1960s, and thus, there are nowadays several optimization methods sharing a reasonable resemblance. All these approaches have been gathered up under the term *evolutionary algorithms* [33, 42], pointing out the fact that these strategies work with candidate solutions which evolve toward better configurations. However, a distinguishing feature of GAs at the time was the concept of recombination implemented as a crossover operator. This, in contrast with the neighborhood exploration of just one solution, carried out by the existing heuristics and by the operators of other algorithms, introduced the idea of combining the information from two candidate solutions to generate a new one. Years afterward, the idea of recombining two or more solutions was adopted by most representatives of evolutionary computation and also by many bioinspired methods [104].

The main aim of this chapter is to introduce the general and practical guidelines for designing GAs for given problems. Thus, we will gather up classical ideas from their origins and the latest tendencies without going into the specific and theoretical details. The interested reader is referred to Katoch et al. [77] for a survey of the works developed in this regard.

The chapter is organized as follows. The section "GA Construction" offers several hints to newcomers for the design and implementation of a GA. Three different and widely known classes of optimization problems are used for reviewing the basic GA components, and several classic alternatives are provided for each of them. The section "Diversification Techniques for GAs" analyzes the premature convergence problem of GAs produced by the lack of diversity and presents

different methods and operators that attempt to favor the diversity of the population in GAs. The section "Hybrid GAs" is devoted to another research area with an important number of algorithms reported in the last years and that is intended to exploit the complementary character of different optimization strategies. Finally, the section "GA Applications" outlines the application areas of GAs and the major challenges faced by GAs to deal with optimization problems in these areas.

GA Construction

In this section, we revise the basic ingredients for designing a GA. We comment some of the original and widely used strategies for each of these components without getting into a deep discussion whether they perform better or worse than other approaches.

Given an optimization problem with a set of decision variables arranged in a vector $\mathbf{x}$ and $f : X \rightarrow \Re$ a function that maps any configuration of the decision variables to a quality value, the aim is to find the combination of values for the decision variables that maximizes, or minimizes, function f. This function is usually known as the objective function. For the rest of this chapter, we will focus on maximizing the function f, noting that the required modifications for minimization problems are generally straightforward. In some cases, there are additional restrictions that solutions must satisfy, which are modeled by a set of inequality and equality constraints, $g_j(\mathbf{x}) \leq 0$ and $h_j(\mathbf{x}) = 0$ with $j = 1, \ldots, M$, respectively. We will use the following widely known optimization problems throughout the chapter to illustrate the application of GAs to problems with very different characteristics:

Knapsack Problem
In the classic knapsack problem [25] we are given a set of N objects with profits $p_i \geq 0$ and weights $w_i \geq 0$ and a container, the knapsack, with a maximum capacity C. The goal is to decide which objects should be included in the knapsack in such a way that the sum of the profits of the selected objects is maximized and the capacity C is not exceeded by the sum of their respective weights. Note that the decision variables of this problem should indicate which objects are included into the knapsack and which are not. Likewise, given a valid candidate solution that satisfies the capacity constraint ($g(\mathbf{x}) = -C + \sum_{i \in Knapsack} w_i$), the objective function f is the sum of the profits p_i of those objects in the knapsack.

Traveling Salesperson Problem
This problem [57] consists in the design of travel plans visiting a given set of cities or locations so that the traveled distance is minimized. The information given is the set of cities, all of them have to be visited, and the distances between each pair of cities. In this case, decision variables should represent the order in which the cities are visited, and the objective function is the sum of the distance between consecutive cities and the journey back to the first city from the last one.

Real-Parameter Optimization
Here, we are given an objective function whose decision variables take values from the continuous space $\Re$. In the context of optimization with metaheuristics, this objective function is treated as a *black box*, so analytical methods cannot be applied. This is often the case when the evaluation of candidate solutions involves the execution of an external simulation procedure, such as wind tunnels or driving simulators, whose internal processes are obscure or they are not subject of analysis. The distinguishing feature of these problems, with regard to combinatorial problem, is that decision variables take real values instead of binary or integer ones.

Whichever is the problem, the design of a GA goes throughout the following steps and not necessarily in this order:

Representation

The practitioner needs to specify how the decision variables of the problem are represented in the computer program to allow the GA to generate the different possible configurations. In particular, the GA community stresses the difference between the solution representation, *genotype*, from arrays of binary, integer (or elements from an alphabet), or floating-point variables to complex structures such as trees, graphs, or objects of specifically designed classes and the solution which is actually represented, *phenotype*. Consider the previous optimization problems:

Knapsack Problem: Representation
The classic representation for this problem is the *binary coding*, which is actually the traditional representation in GAs regardless of the problem. Solutions are represented by an array of N binary variables, as much as objects in the problem, where the value 1 at the i-th position indicates that the i-th object should be included into the knapsack, and 0 means the opposite. Therefore, the following example indicates that we should select the first, third, fourth, and seventh objects and discard the others:

$$[\,1\,0\,1\,1\,0\,0\,1\,]$$

Here, the array of binary variables is the genotype, and the interpretation of that genotype, the actual configuration of the knapsack, is the phenotype. However, note that we could have adopted a different representation such as a variable-length array or set with the integer indexes of the included objects ([1 3 4 7] in the case above). We assume from now on that binary coding is adopted for this problem.

Traveling Salesperson Problem: Representation
Given that the important information here is the order in which cities are visited, a natural way to represent solutions is by means of *permutations* [86], such as the following one:

[3 4 5 7 2 6 1]

These permutations are the genotype from which different interpretations could be developed, the phenotype. The most commonly used is the one that establishes traveling relations between consecutive values, i.e., in the example above the agent travels from the third city to the fourth one, from the fourth to the fifth, and so on; finally, the agent returns to the third city from the first one (the last value in the solution). As previously, other coding schemes could be adopted, such as the binary encoding of each of the integers in the example ([011 100 101 111 010 110 001]). Particularly, an indirect technique with relative success in the field is the *random key encoding* [55]. This represents a permutation as the sorting order of an array of numbers. Therefore, the array [3.0 4.1 0.2 0.3 −0.2 7 1] would be first translated into the permutation [5 3 4 7 1 2 6], from which the appropriate interpretation should be extracted. For the rest of the chapter, we adopt the former representation, which indicates the order by which cities are visited.

Real-Parameter Optimization: Representation
Initially, real (and integer) optimization problems were addressed with the binary coding scheme, considering either a natural or a gray transformation [17, 50, 69]. This latter intends to minimize the number of hamming differences between close final interpretations. However, real encoding is much widely used lately for real-parameter optimization problems, which avoids the differentiation between genotype and phenotype [66, 73, 101]:

[0.5 0.0001 0.2342 1.0 100.0 0.7 − 7.2]

Initial Population: Generating Candidate Solution from Scratch

At the beginning of the search process, GAs conform an initial population of candidate solutions. Usually, there is not much information to be exploited to generate these candidate solutions, and the following alternatives are commonly applied:

- *Random solutions*: When there is no other information apart from the considered representation (binary, integer, permutations, real, trees. . .), one of the easiest and most widely used methodologies is to generate the initial population at random. Thus, the task of generating the initial population becomes the one of producing arrays of random binary, integer or floating-point numbers, trees, graphs, or a mixture of them.
- *Diverse solutions*: As we will see later, the practitioner is often concerned with the presence of diverse solutions in the population. For the case of the initial population, many works recommend random procedures that incorporate some bias for generating diverse solutions and, thus, covering the search more uniformly [117]. One simple technique is a generalization of the Latin Hypercube [72] which works as follows. Suppose that N initial solutions should

Table 1 Latin Hypercube sampling with $N = 6$, $m = 3$, and $l = 7$. P_i are the generated permutations. Notice that each value, 1, 2, or 3, appears in two individuals at every gen

Individual	P_1	P_2	P_3	P_4	P_5	P_6	P_7	Genotype
I_1	2	5	5	6	2	3	2	[3 3 3 1 3 1 3]
I_2	5	2	2	4	1	6	6	[3 3 3 2 2 1 1]
I_3	6	3	6	2	5	4	4	[1 1 1 3 3 2 2]
I_4	4	1	1	3	4	2	5	[2 2 2 1 2 3 3]
I_5	1	4	4	5	6	5	1	[2 2 2 3 1 3 2]
I_6	3	6	3	1	3	1	3	[1 1 1 2 1 2 1]

be generated, whose genotype is an array with l variables that take values from an m-ary alphabet. First, l random permutations of $\{1, \ldots, N\}$ are composed. Then, solutions are generated concatenating the corresponding values of the permutations modulo m (plus 1 if needed), i.e., the first solution considers the first value of the permutations, and so on. See an example in Table 1. Note that N should be multiple of m to assure a uniform sample of the m values. In case of using a real encoding scheme, one possibility is to divide the domain of the variables in m disjoint intervals. The previous procedure is applied, and the variable finally takes a uniform random value from the selected interval.

- *Heuristic solutions*: In case you have some information of the problem at hand, you may exploit it to sample initial solutions better than those randomly generated. For example, in the knapsack problem, objects with better weight-profit ratios can be given higher probabilities of being included into the knapsack, or in the traveling salesperson problem, the nearest neighbor heuristic can be randomized to generate different solutions [150].

In Kazimipour et al. [79] different population initialization techniques are reviewed and categorized into a taxonomy.

Evaluation

Solutions are evaluated to decide whether they will, or will not, take part in the evolution process. This means assigning a fitness value to these candidate solutions. The most common practice is to assign a fitness value which is computed exactly as, or directly from, the objective function $f(\cdot)$. For example, the sum of the profits of the included objects can be used in the knapsack problem or the traveled distance, in the traveling salesperson problem. In some other occasions, you may be interested in another computation such as a scaling procedure to avoid the influence of the scale of the function, or including other information such as diversity maintenance or constraint violation, often addressed with penalty functions [127,153]. In case the calculation of the objective function requires excessive computational resources, it is very common to approximate it by means of a surrogate model [32,61,63], which is the one actually used by the GA.

A very different strategy is to consider a relative measure such as the ranking of the chromosomes in fitness order [122, 146]. Though the information loss, a clear advantage is that fitness values do not depend on the scale of the objective function.

Selection Operator

This operator is traditionally the only one in charge of assigning more reproduction possibilities to better solutions and, therefore, biasing the search process toward the pursued objective. GAs apply this operator to select those individuals allowed to breed and generate new candidate solutions.

The first selection operator applied is known as the *roulette-wheel* method. The idea is to assign selection probabilities, or roulette portions, to the solutions in the population, which are proportional to their fitness values. Thus, better individuals get higher probabilities of being selected. Then, the roulette is drawn as many times as a number of individuals are needed. An enhanced method is the *stochastic universal selection* of Baker [10]. In this case, the roulette has multiple equally spaced spinners, as much as individuals need to be selected, and is drawn just once. In contrast to roulette-wheel, Baker's method reduces the stochastic possibility of selecting the best individual too many or not enough times.

Tournament is another selection operator widely applied nowadays because of its simplicity [54]. This method is composed of a set of t_{size} random individuals from the population, and the best one is selected for breeding. Multiple tournaments are simulated to get the parent population. Note that parameter t_{size} controls the bias toward the best solutions, being $t_{size} = 2$ a classic setting, i.e., binary tournament.

Genetic Operators

Crossover and mutation operators are the GA components that generate new candidate solutions, which are afterward evaluated and considered for the subsequent evolution iteration. Between these two, the crossover has traditionally got a superior significance, and mutation often acts in the background, being applied less frequently. The goal of the former is to combine the information of two or more parent chromosomes from the current population, while the second usually modifies a chosen chromosome randomly and locally.

It is necessary to mention that, according to the *No Free Lunch* theorems for optimization [147], the performance of the GA will be determined by the ability of its components, particularly its genetic operators, to exploit useful knowledge of the concrete problem at hand or real-world problems in general [47]. It is for this reason that we can find a multitude of proposals for crossover and mutation operators particularly helpful for different problems or families of problems in the literature [97, 121, 139]. As an example, Dynastic Potential Crossover Operator [20] is an instance of optimal recombination operators [36], that exploiting the knowledge of the problem is able to produce or approximate the best offspring, from two parents, at a reduced computational cost.

There is not a clear methodology for the order in which these operators should be applied. Commonly, two GA parameters govern the frequency of crossover and mutation applications, namely crossover and mutation probabilities, and the process is often carried out as follows. The selected parent solutions firstly undergo crossover according to the crossover probability, and offspring is transferred to the mutation step; parents are otherwise transferred to the mutation phase. Subsequently, mutation is applied on the received solutions according to the mutation probability, and the result is temporarily stored for the next generation.

Given the strong interaction between the solution representation and genetic operators, we revise some classical proposals in the context of previous problems.

Crossover Operator

We show in this section some examples for combining the information from more than one parent through crossover application.

Knapsack Problem: Crossover Operator

The firstly proposed crossover operator took its inspiration from biology and is widely known as *one-point crossover* operator. Given two binary strings representing the corresponding parent solutions, this operator chooses a random position (4 in the example in Table 2) and combines the head of the first parent with the tail of the second one, generating a new candidate solution, and vice versa for a second candidate solution. However, other crossover operators such as uniform crossover [131] and half-uniform crossover [38] (see also section "The Crossover Operator as Diversification Agent") usually perform better [115]. Both create a new offspring by selecting randomly each of the gene values from the parents, but the latter assures that half of the number of differences are taken from one parent and the other half from the other parent. In the particular case of the knapsack problem, the resulting solutions may violate the capacity constraint, so penalty functions or repair operators would be needed.

Traveling Salesperson Problem: Crossover Operator

Problems using the permutation coding, such as this one, require crossover operators to accept permutations as input and to produce valid permutations as output. Note

Table 2 Example of binary crossover operators with parents $P_1 = [\,0\,1\,1\,0\,1\,0\,0\,]$ and $P_2 = [\,1\,1\,1\,1\,0\,0\,1\,]$

Name	Intermediate steps			Result
One-point	Cut point	Heads	Tails	Offspring
crossover	4	[0 1 1 0	1 0 0]	[0 1 1 0 0 0 1]
		[1 1 1 1	0 0 1]	[1 1 1 1 1 0 0]
Uniform		Genes from P_1	Genes from P_2	Offspring
crossover		1, 3, 5, 6	2, 4, 7	[0 1 1 1 1 0 1]
Half-uniform	Differences	Genes from P_1	Genes from P_2	Offspring
crossover	1, 4, 5, 6	4, 5	1, 6	[1 1 1 0 1 0 1]

Table 3 Example of crossover operators for permutations with parents $P_1 = [\ 1\ 2\ 3\ 4\ 5\ 6\ 7\ ]$ and $P_2 = [\ 2\ 4\ 5\ 7\ 1\ 6\ 3\ ]$

Name	Intermediate steps			Result
Partially	Cross. points	Comb. and rep. removal	Mappings	Offspring
mapped	2–4	[2 | 2 3 4 | 1 6 3]	2 → 4, 3 → 5, 4 → 7	[7 | 2 3 4 | 1 6 5]
crossover		[1 | 4 5 7 | 5 6 7]	4 → 2, 5 → 3, 7 → 4	[1 | 4 5 7 | 3 6 2]
Order	Cross. points	Inner segment	Valid values	Offspring
crossover	4–6	[− − − | 4 5 6 | −]	[2 4 | 5 7 1 6 | 3]	[2 7 1 | 4 5 6 | 3]
		[− − − | 7 1 6 | −]	[1 2 | 3 4 5 6 | 7]	[3 4 5 | 7 1 6 | 2]

that previous operators do not satisfy this necessity. Therefore, specific operators are proposed for this type of problems (see Table 3). The firstly proposed one was *partially mapped crossover* [52], which is aimed precisely at generating sequences of numbers without repetition. Its idea is to divide the two-parent permutations in three segments each according to two randomly chosen positions (2–4 in Table 3). Then, the inner segment of one parent is combined with the outer ones of the other, and repeated genes in the outer segments are changed according to the mapping rules defined in the inner segment. *Order crossover* [26] is another option which intends to exploit the relative order of the gene values instead of their absolute positions, as the previous operator does. Firstly sampling two random positions, this operator copies the inner segment into the offspring and continues copying those valid gene values from the secondly sampled position of the other parent (see Table 3). Given that different operators focus on different interpretations of the information transmitted, the practitioner should identify what is the one most appropriate for the adopted problem representation.

Real-Parameter Optimization: Crossover Operator

Similarly to previous cases, there are many proposals for the combination of the information in two real-coded parent solutions. In this case, researchers pay attention to the capacities of the crossover to converge, expand, or even correlate the changes on the decision variables [67]. An interesting characteristic of most real-parameter crossover operators is whether they tend to sample new solutions near the centroid of the parents or near the parents. BLX-α [39] is an example of centroid-centered operator, and PBX-α is its parent-centered version [45, 91]. For each offspring gene o_i, both operators create an interval $I = [a, b]$ from which the actual value is uniform randomly drawn. In both cases, the amplitude of the interval depends on the difference between the parent gene values, $I_i^a = abs(p_i^1 - p_i^2)$, and the domain of the variable, $[l_i, u_i]$. The difference is the location of that interval, biased toward the average of the previous parent gene values, in the case of BLX-α, or the value of the first parent, PBX-α. Table 4 shows an example of how both operators compute the intervals from where offspring gene values are sampled.

Table 4 Example of real-parameter crossover operators with parents $P_1 = [\ 0.3\ \ 7\ ]$ and $P_2 = [\ 0.4\ \ 1\ ]$, $\alpha = 0.5$, and domains $x_1 \in [0, 1]$ and $x_2 \in [-10, 9]$

Name	Intermediate steps			
BLX-α	I_i^a	Lower bound	Upper bound	Actual interval
	$abs(p_i^1 - p_i^2)$	$max\{l_i,\ min\{p_i^1, p_i^2\} - \alpha I_i^a\}$	$min\{u_i,\ max\{p_i^1, p_i^2\} + \alpha I_i^a\}$	
o_1	0.1	$max\{0,\ 0.3 - 0.05\}$	$min\{1,\ 0.4 + 0.05\}$	[0.25, 0.45]
o_2	6	$max\{-10,\ 1 - 3\}$	$min\{9,\ 7 + 3\}$	$[-2, 9]$
PBX-α	I_i^a	Lower bound	Upper bound	Actual interval
	$abs(p_i^1 - p_i^2)$	$max\{l_i,\ p_i^1 - \alpha I_i^a\}$	$min\{u_i,\ p_i^1 + \alpha I_i^a\}$	
o_1	0.1	$max\{0,\ 0.3 - 0.05\}$	$min\{1,\ 0.3 + 0.05\}$	[0.25, 0.35]
o_2	6	$max\{-10,\ 1 - 3\}$	$min\{9,\ 1 + 3\}$	$[-2, 4]$

Mutation Operator

As for the crossover operator, mutation is strongly connected with the adopted representation. Therefore, we will see some simple examples for any of the three problems considered through this chapter. Apart, we shall mention that the community usually distinguishes between applying mutation at individual or gene level. In the first case, the chromosome is mutated only once according to the mutation probability. One gene is randomly selected and modified, which could involve additional modifications in one or a few other genes. In the latter, all the genes of the chromosome are visited, and each one is mutated according to the mentioned probability, so chromosomes might suffer stronger alterations. In this case, the mutation probability is often much smaller.

Knapsack Problem: Mutation Operator

Flipping is the simplest mutation operator for binary strings. If mutation applies at the individual level, a random gene is selected, and its value is changed, from 0 to 1 or the other way around. If mutation is at the gene level, each variable is visited once and flipped according to the mutation probability. One should notice that other operators might be more appropriate for the problem at hand. For example in the case of the knapsack problem, flipping one bit from 1 to 0 might leave room for inserting another object in the knapsack, and changing one bit from 0 to 1 may make the solution unfeasible. Thus, flipping two bits with different values might be preferable.

Traveling Salesperson Problem: Mutation Operator

Assigning random values to one or several genes is often applied in integer-coded problems; however, this strategy is not appropriate for permutation codings. Instead, mutation operators usually apply swappings, insertions, reorderings, or sublist inversions [33, 126]. Table 5 shows the application of swap [12, 112] and sublist inversion [69]. In both cases, two positions are randomly drawn. Notice, however, that sublist inversion modifies less arcs of the solution path in symmetric graphs than swapping the selected positions. This is due to the fact that traveling direction

Table 5 Example of mutation operators for permutations with chromosome $S = [\ 2\ 4\ 5\ 7\ 1\ 6\ 3\ ]$

Swap	Mut. points	Result	Arcs removed	Added arcs
	2–5	[2 | 1 | 5 7 | 4 | 6 3]	$2 \to 4,\ 4 \to 5,$	$2 \to 1,\ 1 \to 5$
			$7 \to 1,\ 1 \to 6$	$7 \to 4,\ 4 \to 6$
Sublist	Mut. Points	Result	Arcs removed	Added arcs
Inversion	2–5	[2 | 1 7 5 4 | 6 3]	$2 \to 4,\ 1 \to 6$	$2 \to 1,\ 4 \to 6$

does not affect the solution cost in this kind of graphs. Thus, sublist inversion is often preferred for this problem.

Real-Parameter Optimization: Mutation Operator
Uniform and norm random variables, with a symmetric distribution around zero, are commonly used to perturb a given real-coded solution locally. In contrast to previous cases, it is usual to modify more than one and often all the variables of the solutions at the same time, by adding up a complete randomly drawn vector. Altering either one or all the variables, practitioners often pay attention at the global impact of the mutation so that the difference between the original and the mutated solution, under the Euclidean metric, for instance, is controlled. A widely used operator is the nonuniform mutation [73, 101], which reduces the intensity of mutation with the number of generations. Particularly, a variation vector $\Delta = [\ \delta_1\ \ldots\ \delta_i\ \ldots\ \delta_N\ ]$, computed as follows, is added up to the given solution $X = [\ x_1\ \ldots\ x_i\ \ldots\ x_L\ ]$:

$$\delta_i = \begin{cases} (u_i - x_i)(1 - z_i)^{\gamma}, & \text{with probability } 1/2 \\ (l_i - x_i)(1 - z_i)^{\gamma}, & \text{otherwise} \end{cases}$$

$$\gamma = \left(1 - \frac{t}{t_{max}}\right)^{\beta}$$

where z_i is a uniform random variable in $[0, 1]$, u_i and l_i are the box bounds of variable x_i, β is a parameter with $\beta > 0$, and t and t_{max} are, respectively, the current and maximal number of generations.

Evolution Model and Replacement Strategy

The traditional GA applied a *generational* evolution strategy. This means that evolution occurs at generations where a complete new population is generated from the current one, which, in turn, becomes the current population for the next generation. The pseudocode of this generational GA is shown in Fig. 1. A possible undesirable effect of this model is that the new population might be worse, in quality terms, than the current population. So usually, the best solution from the current population is artificially inserted in the new population, particularly when there is

Fig. 1 Basic scheme of a generational GA

```
begin GENERATIONAL-GA(NP, p_c, p_m)
1   for i = 1, ..., NP do
2       p_i ← GENERATE-SOLUTION;
3       p_i.fitness ← EVALUATE(p_i);
4       P ← P ∪ {p_i};
5   end-for;
6   repeat do
7       P_O ← ∅;
8       repeat do
9           parents ← SELECTION(P);
10          offspring ← CROSSOVER(parents, p_c);
11          offspring ← MUTATION(offspring, p_m);
12          offspring.fitness ← EVALUATE(offspring);
13          P_O ← P_O ∪ {offspring};
13      until |P_O| = NP;
14      P ← P_O;
15  until STOP-CONDITION;
16  return BEST-SOLUTION;
end GENERATIONAL-GA.
```

not any better solution in the new population. This evolution model is known as *generational with elitism* [11].

Another commonly applied model is the *steady-state GA* [29], which contrarily generates a reduced set of new chromosomes, usually just one solution, that compete to enter into the population (see pseudocode in Fig. 2). In steady-state GAs, the designer has to specify a replacement policy, which determines whether the new solution enters the population or it is discarded, and, which solution from the population is removed to make room for the new one, if this is accepted. A simple policy to introduce the new solution into the population is replacing the worst

Fig. 2 Skeleton of a steady-state GA

```
begin STEADY-STATE-GA(NP, p_c, p_m)
1   for i = 1, ..., NP do
2       p_i ← GENERATE-SOLUTION;
3       p_i.fitness ← EVALUATE(p_i);
4       P ← P ∪ {p_i};
5   end-for;
6   repeat do
7       parents ← SELECTION(P);
8       offspring ← CROSSOVER(parents, p_c);
9       offspring ← MUTATION(offspring, p_m);
10      offspring.fitness ← EVALUATE(offspring);
11      P ← REPLACEMENT(offspring, P);
12  until STOP-CONDITION;
13  return BEST-SOLUTION;
end STEADY-STATE-GA.
```

solution if the new one is better. Other strategies consider diversity measures, like replacing the most similar solution [62] (see also the section "Strategies to Maintain Diversity," crowding methods) or the worst parent.

Stop Condition

GAs evolve until a certain stopping criterion is met. Practitioners commonly consider a maximum number of generations, fitness evaluations, or a maximal processing time. These criteria are widely used when one is interested in comparing the performance of several algorithms, and the same computational resources have to be provided for a fair study.

On other occasions, especially when one is aimed at solving a particular problem, GAs are executed until a minimal solution quality is attained or the population

converges, so further progress is very difficult. In these cases, practitioners often need to be able to consult the progress of the search to decide at the moment, whether continuing the evolution or truncating it and applying a restart procedure (see also the section "Methods to Generate Raw Diversity").

Diversification Techniques for GAs

There are two primary factors in the search carried out by a GA [105, 142]: population diversity and selective pressure. In order to have an effective search there must be a search criterion (the fitness function) and a selection pressure that gives individuals with higher fitness, a higher chance of being selected for reproduction, mutation, and survival. Without selection pressure, the search process becomes random, and promising regions of the search space would not be favored over regions offering no promise. On the other hand, population diversity is crucial to a GA's ability to continue the fruitful exploration of the search space [70]. If the lack of population diversity takes place too early, a premature stagnation of the search is caused. Under these circumstances, the search is likely to be trapped in a region not containing the global optimum. This problem, called premature convergence, has long been recognized as a serious failure mode for GAs [64, 154]. In the literature, many approaches have been proposed to introduce new methods and operators that attempt to favor the diversity of the population to overcome this essential problem of genetic algorithms. Next, we present a quick overview of them.

Methods to Generate Raw Diversity

Premature convergence causes a drop in the GA's efficiency; the genetic operators do not produce the feasible diversity to tackle new search space zones, and thus the algorithm reiterates over the known zones producing a slowing down in the search process. Under these circumstances, resources may be wasted by the GA searching an area not containing a solution of sufficient quality, where any possible improvement in the solution quality is not justified by the resources used. Therefore, resources would be better utilized in restarting the search in a new area, with a new population. This is carried out by means of a restart operator [49, 51], which introduces chromosomes with high *raw diversity* to increase the average diversity level, thus to ensure the process can jump out the local optimum and to revolve again.

Eshelman's CHC algorithm [37] represents a GA with elitist selection and a highly disruptive recombination operator that restarts the search when the population diversity drops below a threshold level. The population is reinitialized by using the best individual found so far as a template for creating a new population. Each individual is created by flipping a fixed proportion (35%) of the bits of the template chosen at random without replacement. If several successive reinitializations fail to yield an improvement, the population is completely (100%) randomly reinitialized.

Another GA that utilizes population reinitialization is the micro GA [50]. In general, a micro GA is a small population GA which evolves for many generations. When after a number of generations the micro GA population converges, the evolutionary process is reinitialized by preserving the best individual and substituting the rest of the population with randomly generated individuals. The first implementation of a micro GA was reported by Krishnakumar [83], who used a population size of five individuals, tournament selection, single-point crossover with probability, elitism, and restart operator. The population was considered converged when less than 5% of the population bits were different from the bits of the best individual.

A GA model, called saw-tooth GA [82], manages a variable population size with periodic reinitialization following a saw-tooth scheme with a specific amplitude and period of variation. In each period, the population size decreases linearly, and at the beginning of the next period randomly generated individuals are appended to the population.

Strategies to Maintain Diversity

Pioneer works on the way diversity may be retained throughout the GA run focused on the design of alternative selection mechanisms. For example, in the linear ranking selection [9], the chromosomes are sorted in order of raw fitness, and then the selection probability of each chromosome is computed according to a linear function of its rank. With this selection mechanism, every individual receives an expected number of copies that depends on its rank, independent of the magnitude of its fitness. This may help prevent premature convergence by preventing super individuals from taking over the subpopulations within a few generations.

Disruptive selection [84] attempts to accomplish this objective as well. Unlike conventional selection mechanisms, this approach devotes more trials to both better and worse solutions than it does to moderate solutions. This is carried out by modifying the objective function of each chromosome, C, as follows: $f'(C) = |f(C) - \bar{f}|$, where $\bar{f}$ is the average value of the fitness function of the individuals in the population. A related selection method is the fitness uniform selection scheme (FUSS) [71]. FUSS generates selection pressure toward sparsely populated fitness regions, not necessarily toward higher fitness. It is defined as follows: If f_{min} and f_{max} are the lowest and highest fitness values in the current population, respectively, we select a fitness value uniformly in the interval $[f_{min}, f_{max}]$. Then, the individual in the population with fitness nearest to this value is selected. FUSS results in high selection pressure toward higher fitness if there are only a few fit individuals, and the selection pressure is automatically reduced when the number of fit individuals increases. In a typical FUSS population, there are many unfit and only a few fit individuals. Fit individuals are effectively favored until the population becomes fitness uniform. Occasionally, a new higher fitness level is discovered and occupied by a new individual, which then, again, is favored. Finally, another technique being worthy of mention is the repelling algorithm [148], which modifies the fitness function to increase the survival opportunity of chromosomes with rare alleles.

There are different replacement strategies for steady-state GAs that try to maintain population diversity, as well. In Lozano et al. [92], the authors propose a replacement strategy that considers two features of the individual to be included into the population: a measure of the contribution of diversity to the population and the fitness function. It tries to replace a chromosome in the population with worst values for these two features. In this way, the diversity of the population increases, and the quality of the solutions improves, simultaneously. The goal of this strategy is to protect those individuals that preserve the highest levels of useful diversity.

Other replacement methods to promote population diversity are the crowding methods [124]. They work as follows: New individuals are more likely to replace existing individuals in the parent population that are similar to themselves based on genotypic similarity. In this manner, the population does not build up an excess of similar solutions. Crowding methods promote the formation of stable subpopulations in the neighborhood of optimal solutions (niches), favoring the preservation of multiple local optima in multimodal problems. An effective crowding method is the restricted tournament selection (RTS) [62]. RTS initially selects two elements at random, A and B, from the population and performs crossover and mutation on these two elements resulting in a new element A'. Then, RTS scans ω (window size) more members of the population and picks the individual that most closely resembles A' from those ω elements. A' then competes with this element, and if A' wins, it is allowed to enter the population. Another type of crowding methods assumes that the parents would be those members of the population that are closer to the new elements. In this way, children compete with their parents to be included in the population, i.e., a family competition is held. These methods include deterministic crowding [94] and elitist recombination [135].

Some GA models were proposed in the literature which explicitly avoid the existence of duplicate individuals in the population, as a way to encourage diversity. In fact, early empirical studies confirmed that duplicate removal can enhance the performance of GA significantly [99]. For example, the non-revisiting GA [154] guarantees that no revisits ever occur in its fitness evaluations. It achieves this by interacting with a dynamically constructed binary space partitioning archive that is built up as a random tree for which its growth process reflects the evolution history of the GA and is a quick method to query whether there is a revisit. The entire previous search history is then used to guide the search to find the next unvisited position. A similar approach may be found in Gupta and Garg [59].

Finally, it is worth to mention that diploid GAs [81, 140] are evolutionary algorithms that manipulate a specific kind of diversity that becomes profitable to deal with dynamic optimization problems (in which some elements of the underlying model change over the course of the optimization). Most organisms in nature have a great number of genes in their chromosomes, and only some of the dominant genes are expressed in a particular environment. The repressed genes are considered as a means of storing additional information and providing a latent source of population diversity. Diploid GAs use diploid chromosomes which are different from natural ones in that the two strands of the diploid chromosomes are not complementary. Only some genes in a diploid chromosome are expressed and

used for fitness evaluation by some predetermined dominance rules. Unused genes remain in the diploid genotype until they may later become useful (*latent diversity*).

The Crossover Operator as Diversification Agent

The mating selection mechanism determines the way the chromosomes are mated by applying the crossover to them. In the conventional GA, no mating strategy is applied to the results of selection; that is, parents are approved without any further examination after they are chosen at random or just by fitness. However, mates can be selected so as to favor population diversity [37, 136]. A way to do this is the negative assortative mating mechanism. Assortative mating is the natural occurrence of mating between individuals of similar genotype more or less often than expected by chance. Mating between individuals with similar genotype more often is called positive assortative mating and less often is called negative assortative mating. Fernandes et al. [40] assume these ideas in order to implement a parent selection mechanism for the crossover operator. A first parent is selected by the roulette-wheel method, and n_{ass} chromosomes are selected with the same method. Then, the similarity between each of these chromosomes and the first parent is computed. If assortative mating is negative, then the one with less similarity is chosen. If it is positive, the genome that is most similar to the first parent is chosen to be the second parent. Clearly, the negative assortative mating mechanism increases genetic diversity in the population by mating dissimilar genomes with higher probability.

The crossover operator has always been regarded as one of the main search operators in GAs [80] because it exploits the available information in previous samples to influence future searches. This is why research has been focused on developing crossover operators with an active role as effective diversification agents. The half-uniform crossover used in the CHC algorithm [37] is a highly disruptive crossover that crosses over exactly half of the non-matching alleles (the bits to be exchanged are chosen at random without replacement). This way, it guarantees that the two offspring are always at the maximum Hamming distance from their two parents, thus proposing the introduction of a high diversity in the new population and lessening the risk of premature convergence. It is worth of mention that CHC applies this operator along with (1) a reproduction restriction that assures that selected pairs of chromosomes would not generate offspring unless their Hamming distance is above a certain threshold and (2) a conservative selection strategy with high selective pressure (it keeps the N best elements appearing so far). In fact, Kemenade et al. [141] suggest that higher selection pressures allow the application of more disruptive recombination operators.

A crossover operator that was specifically designed with the aim of diversifying the search process of the real-coded GAs is BLX-α [39] (see also the section "Crossover Operator" where its operation is described, and moreover, an illustrative example is presented). Nomura et al. [109] provide a formalization of this operator to analyze the relationship between the chromosome probability density

functions before and after its application, assuming an infinite population. They state that BLX-α spreads the distribution of the chromosomes when $\alpha > \frac{\sqrt{3}-1}{2}$ or otherwise reduces it. This property was verified through simulations. In particular, the authors observed that BLX-0.0 makes the variances of the distribution of the chromosomes decrease, reducing the distribution, whereas BLX-0.5 makes the variances of the distribution increase, spreading the distribution.

BLX-α has a self-adaptive nature in that it can generate offspring adaptively according to the distribution of parents without any adaptive parameter [13]. BLX-α uses probability distributions that are calculated according to the distance between the decision variables in the parents. If the parents are located closely to each other, the offspring generated by this operator might be distributed densely around the parents. On the other hand, if the parents are located far away from each other, then the offspring will be sparsely distributed around them. Therefore, it may fit their action range depending on the diversity of the population by using specific information held by the parents. In this way, depending on the current level of diversity in the population, it may favor the production of additional diversity (divergence) or the refinement of the solutions (convergence). This behavior is achieved without incurring into extra parameters or mechanisms to achieve the mentioned behavior.

Diversification by Adapting GA Control Parameters

Finding robust control parameter settings (such as mutation probability, crossover probability, and population size) is not a trivial task, since their interaction with GA performance is a complex relationship and the optimal ones are problem-dependent. Furthermore, different control parameter values may be necessary during the course of a run to induce an optimal exploration/exploitation balance. For these reasons, adaptive GAs [34, 65, 76, 128, 138] have been built to dynamically adjust selected control parameters or genetic operators during the course of evolving a problem solution. Their objective is to offer the most appropriate exploration and exploitation behavior [130].

Some adaptive techniques were presented to endow the GA with useful diversity. Specifically, the mutation probability (p_m) was considered as a key parameter to accomplish this task [114, 154]. Next, we describe different adaptive mechanisms presented in the GA literature to control this parameter.

- *Deterministic control of* p_m. A direction followed by GA research for the variation of p_m lies in the specification of an externally specified schedule which modifies it depending on the time, measured by the number of generations. One of the most considered schedules consists in decreasing p_m during the GA run [41]. For example, Hinterding et al. [68] suggested the equation $p_m = 0.1 - 0.09 \cdot \frac{g}{G}$, where g is the generation number from 1 to G. This schedule follows the heuristic *"to protect the exploration in the initial stages and the*

exploitation later," which has been considered to design other metaheuristics, such as simulated annealing.

- *Adaptive control of* p_m. In Srinivas and Patnaik [129], a technique for the adaptive control at individual level of p_m was proposed, in which p_m is varied depending on the fitness values of the solutions. Each chromosome C_i has its own associated p_m value, p_m^i, which is calculated as (maximization is assumed)

$$p_m^i = \frac{f_{max} - f_i}{f_{max} - \bar{f}} \text{ if } f_i \geq \bar{f}, \text{ and } p_m^i = 1 \text{ if } f_i < \bar{f}$$

where f_i is the chromosome's fitness, f_{max} is the population maximum fitness, and $\bar{f}$ is the mean fitness. In this way, high-fitness solutions are protected ($p_m^i = 0$), while solutions with subaverage fitnesses are totally disrupted ($p_m^i = 1$). This technique increases p_m when the population tends to get stuck at a local optimum and decreases it when the population is scattered in the solution space.

- *Self-adaptive control of* p_m. An extra gene, p_m^i, is added to the front of each chromosome, C_i, which represents the mutation probability for all the genes in this chromosome. This gene evolves with the solution [8, 138]. The values of p_m^i are allowed to vary from p_m^l to p_m^h. The following steps are considered to mutate the genes in a chromosome C_i:

  1. Apply a meta-mutation on p_m^i obtaining q_m^i. This is carried out by choosing a randomly chosen number from the interval $[p_m^i - d, p_m^i + d]$, where d is a control parameter.
  2. Mutate the genes in C_i according to the mutation probability q_m^i.
  3. Write the mutated genes (including q_m^i value) back to the chromosome.

  Crossover is presently applied only to the chromosome and has no impact on p_m^i. Each offspring resulting from crossover receives the p_m^i value of one of its parents. The initial p_m^i values are generated at random from $[p_m^l, p_m^h]$.

  The self-adaptive control of GA parameters attempts to exploit the indirect link between favorable control parameter values and fitness values, with the parameters being capable of adapting implicitly, according to the topology of the objective function [8].

Diversity Preservation Based on Spatial Separation

GA models based on the spatial separation of individuals were considered as an important way to research into mechanisms for dealing with the premature convergence problem. One of the most important representatives is the distributed GAs (DGAs) [6,35,64]. Their premise lies in partitioning the population into several subpopulations, each one of them being processed by a GA, independently of the

others. Furthermore, a migration mechanism produces a chromosome exchange between the subpopulations. DGAs attempt to overcome premature convergence by preserving diversity due to the semi-isolation of the subpopulations. Another important advantage is that they may be implemented easily on parallel hardware. Moreover, we should also point out that it is possible to improve the diversification in DGAs by designing specific migration policies [7].

Making distinctions between the subpopulations of a DGA through the application of GAs with different configurations (control parameters, genetic operators, codings, etc.), we obtain the so-called heterogeneous DGAs [64]. They are suitable tools for producing parallel multiresolution in the search space associated with the elements that differentiate the GAs applied to the subpopulations. This means that the search occurs in multiple exploration and exploitation levels. In this way, a distributed search and an effective local tuning may be obtained simultaneously, which may allow premature convergence to be avoided and approximate final solutions to be reached. An outstanding example is GAMAS [116], a DGA based on binary coding that uses four subpopulations, denoted as species I-IV. Initially, species II-IV are created. Species II is a subpopulation used for exploration. For this purpose, it uses a high mutation probability ($p_m = 0.05$). Species IV is a subpopulation used for exploitation. So, its mutation probability is low ($p_m = 0.003$). Species III is an exploration and exploitation subpopulation; the mutation probability falls between the other two ($p_m = 0.005$). GAMAS selects the best individuals from species II-IV and introduces them into species I whenever those are better than the elements in this subpopulation. The mission of species I is to preserve the best chromosomes appearing in the other species. At predetermined generations, its chromosomes are reintroduced into species IV by replacing all of the current elements in this species.

The cellular GA (cGA) [5] is another kind of decentralized GA in which the population is arranged in a toroidal grid, usually of dimension 2. The characteristic feature of cGAs is a neighborhood for each individual that is restricted to a certain subset of individuals in the immediate vicinity of its position. Individuals are allowed to interact only with other individuals belonging to their neighborhood. The overlapped small neighborhoods of cGAs help in exploring the search space because the induced slow diffusion of solutions through the population provides a kind of exploration (diversification). In contrast, the exploitation (intensification) is provided inside each neighborhood to improve the quality of solutions and could be controlled by the use of appropriate genetic operators. Several studies have been carried out in order to investigate mechanisms to dynamically control the exploration/exploitation trade-off kept by cGAs. This task may be achieved through tuning the relationship between the size and/or shape of the neighborhood and the grid [4] or by adjusting the local selection method [3]. It is worth remarking that since cGAs only require communication between few closely arranged individuals, they are very suitable for a parallel implementation, as well.

Interestingly, accelerating DGAs and cGAs on GPU (graphical processing unit) architectures has received significant attention from both practitioners and researchers ever since GPUs emerged [19, 156].

Hybrid GAs

Over the last few years, a large number of search algorithms were reported that do not simply follow the paradigm of one single classical metaheuristic, but they combine several components of different metaheuristics or even other kind of optimization methods. These methods are commonly known as *hybrid metaheuristics* [15,16,56,90,132]. The main motivation behind the development of hybrid metaheuristics is to take advantage of the complementary character from a set of metaheuristics and other optimization methods to produce a *profitable synergy* [123] from their combination.

In particular, the *hybridization of GAs*, and evolutionary algorithms in general, is becoming popular due to its ability to handle several problems involving complex features such as complexity, noise, imprecision, uncertainty, and vagueness [56,95, 118, 145]. We may also highlight that many different instantiations of hybrid GAs have been presented to solve real-world problems in many different fields such as protein structure prediction [134], image processing [14], job-shop scheduling [44], machine learning [111], and engineering design [48], to name but a few.

The flexibility offered by the GA paradigm allows specialized models to be obtained with the aim of providing intensification and/or diversification, i.e., GAs specializing in intensification and/or diversification. The outstanding role played by GAs at present along with the great interest raised by their hybridizations with other algorithms endorses the choice of their specialist approaches as suitable ingredients to build hybrid metaheuristics with other search techniques [90].

As mentioned above, these hybridizations encompass not only other metaheuristics but also other methods such as mathematical programming and simulation methods. Hybridization with mathematical programming methods is commonly known as *matheuristics* [96], and we can find in the recent literature several proposals to address real-world problems such as production and distribution planning in the supply chain [60], radiotherapy planning [110], and parallel machine scheduling [149]. *Simheuristics* [75] are a class of optimization techniques that combine elements of simulation and heuristics to tackle complex optimization problems. These techniques take advantage of the ability of simulations to model real-world systems characterized by uncertainty and heuristic strategies to explore and find promising solutions in large search spaces. Examples of integrating GAs with simulation methods can be found in several areas such as supply chain scheduling [119] and management of assets [107].

At this point, our objective is to illustrate the different strategies used successfully in the literature to hybridize GAs and classify them according to a taxonomy based on those proposed by Talbi [132] and Raidl [120] for hybrid metaheuristics.

We firstly have split the different instances of hybrid metaheuristics into two main groups according to the architecture of the algorithms:

- *Collaborative hybrid metaheuristics*. In this category, different self-contained metaheuristics interchange information between them running sequentially or in parallel. These hybrid metaheuristics can be considered as black boxes, and the

only cooperation takes place through the exchange of solutions, parameters, and so on from time to time.
- *Integrative hybrid metaheuristics*. In this case, one algorithm is in charge of the search process, whereas a subordinate method is embedded as a component of the master metaheuristic. This kind of hybridization addresses the functional composition of a single optimization method, replacing or improving a particular function within a metaheuristic with another metaheuristic.

At the same time, according to the way metaheuristics are executed, *collaborative* metaheuristics can be subdivided into two different categories:

- *Collaborative teamwork*. There are several metaheuristics that work in parallel and exchange some kind of information once in a while.
- *Collaborative relay*. Several metaheuristics are executed in a pipeline fashion. The output of each algorithm becomes the input of the next one.

Integrative hybrid metaheuristics can also be subdivided into teamwork or relay categories:

- *Integrative teamwork*. One metaheuristic (subordinate) becomes a component of another population-based metaheuristic (master) [132].
- *Integrative relay*. This kind of hybrid metaheuristics represents algorithms in which a given metaheuristic is embedded into a trajectory-based metaheuristic [132].

Collaborative Teamwork Hybrid GAs

In the case of *collaborative teamwork hybrid GAs*, we can highlight two schemes commonly used in the literature. In the first scheme, a single population is decentralized by partitioning it into several subpopulations (islands or demes), where island GAs are run performing sparse exchanges (migrations) of individuals. This kind of models are usually referred to as distributed GAs [64] (see also the section "Diversity Preservation Based on Spatial Separation"). Specifically, in this work several subpopulations are processed by GAs with different exploration or exploitation degrees. With this mechanism, refinement of the best solutions and expansion of the most promising zones are achieved in a parallel way. Communication between different GAs is done by sending the best solution of each population to the neighboring populations every five iterations.

In the second scheme, a set of heterogeneous metaheuristics including GAs is executed in parallel. For example, Talbi and Bachelet [133] combine a GA, tabu search, and a local search procedure that communicate through an adaptive memory that contains the search history. Multiple independent tabu search tasks run in parallel without any direct cooperation. However, the adaptive memory maintains a history of the search performed by these tabu search tasks. This information is used

by the GA to generate individuals in unexplored regions. At the same time, elite solutions from the adaptive memory are improved with the local search procedure.

Collaborative Relay Hybrid GAs

In the *collaborative relay hybrid GAs*, a GA is executed, in a pipeline fashion, with another GA or other type of metaheuristic so that the output of each algorithm is used as input of the next one. Most instances of collaborative relay GAs follow the principle of favor exploration in the initial stages and exploitation later, inspired by the design of classical metaheuristics such as simulated annealing [90]. In this line, Chelouah and Siarry [18] presented a hybrid GA that combines a GA specialized for diversification and a local search process that improves the best individuals found by the GA, whereas García-Martínez et al. [45] proposed a collaborative relay hybrid GA where the local refinement of solutions is performed by a specialized GA. They run a global real-coded GA during a determinate percentage of the available evaluations, and then they perform the local real-coded GA. The initial population for the local algorithm includes the best individuals of the global one.

Integrative Teamwork Hybrid GAs

In the case of *integrative teamwork hybrid GAs*, we can find two different approaches: *memetic algorithms* [106] and *GAs with metaheuristic-based components*. The classic scheme of memetic algorithms applies a local search method to solutions obtained by the GA that is in charge of the global search. The idea behind memetic algorithms is to provide an effective and efficient optimization method by achieving a trade-off between global exploration of GAs and local exploitation of local search procedures [103]. However, memetic algorithms also include combinations of GAs with problem-dependent heuristics, approximate algorithms, truncated exact methods, specialized recombination operators, etc. [106].

Many different instantiations of *memetic algorithms* have been presented to deal with a wide variety of application domains. However, the additional fitness function evaluations required for the local search method can increase significantly the computational cost of memetic algorithms. In order to mitigate this drawback, Molina et al. [103] proposed a memetic algorithm with local search chains that aims at focusing the local search action on promising areas of the search space. This is addressed by retaking the local search from solutions obtained in a previous application of the local search method, adopting also the former final strategy parameter values as its initial values.

In this line, we can find memetic algorithms in which the refinement procedure is performed by another GA instead of the classical local search procedure. With

this idea, there have been presented several memetic algorithms that use GAs with a small population and short evolution or micro GAs (μGA) [78, 91, 108] (see also the section "Diversification Techniques for GAs") to locally improve solutions. μGA models provide some advantages over classical local search methods. Most local search techniques find difficulties in following the right path to the optimum in complex search spaces. However, it was observed that μGAs are capable of following ridges of arbitrary direction in the search space regardless of their direction, width, or even, discontinuities [78]. For example, the μGA presented in Kazarlis et al. [78] is a GA with five individuals that encode perturbations. Aptitude values of these individuals depend on a solution given by the master GA. This feature ensures that search is focused on the neighborhood of the given solution, whereas low-sized population promotes high selection pressure levels.

With regard to *GAs with metaheuristic-based components*, we can find in the literature several proposals where the selection mechanism, the crossover, and mutation operators of GAs have been replaced or extended by other classical metaheuristic. The approach proposed in Adler [1] combines simulated annealing with a GA by extending the mutation and crossover operators with simulated annealing. Mutated and recombined solutions are accepted according to the standard simulated annealing acceptance condition. Kurahashi and Terano [85] proposed a tabu-based GA where after evaluating individuals in each iteration, they store the best individual of the generation into both long-term and short-term tabu lists. Then, the GA selection refers to the tabu lists in order to select individuals with dissimilar genotypes and consequently avoid premature convergence to local optima.

Integrative Relay Hybrid GAs

Finally, we undertake the study of *integrative relay hybrid GAs*. In this scheme, usually, a GA is used to perform one or more functions in the master metaheuristic. García-Martínez et al. [46] presented a GA designed specifically to play the role of the simulated annealing neighborhood operator. In particular, a steady-state GA creates one single candidate solution at each iteration, by crossing over the current solution of the master simulated annealing and another one from the population. Afterward, the master simulated annealing applies an acceptance mechanism to decide which solution becomes the new current solution, either the candidate solution or the current one. The other solution is inserted into the population by a replacement strategy. Another approach following this scheme was presented in Lozano and García-Martínez [90]. In this work, the authors present an iterated local search with a perturbation operator based on a μCHC algorithm. CHC provides high diversity by means of high selective pressure, allowing diverse and promising solutions to be maintained. This behavior is desirable for a GA assuming the work of a perturbation operator.

GA Applications

GAs have had a great measure of success in search and optimization problems. The reason for a great part of their success is their ability to exploit the information accumulated about an initially unknown search space in order to bias subsequent searches into useful subspaces, i.e., their adaptation. This is their key feature, particularly in large, complex, and poorly understood search spaces, where classical search tools (enumerative, heuristic, etc.) are inappropriate, offering a valid approach to problems requiring efficient and effective search techniques.

With the aim of showing the broad variety of GA applications, we have searched for papers from Scopus with the keyword "genetic algorithm." (The query was submitted in April 2024.) Table 6 outlines the number of documents with "genetic

Table 6 Research papers on GAs by subject areas

	Documents	
Subject Area	Title	Tit./Abs./Key.
Engineering	38,790	139,006
Computer Science	36,612	127,831
Mathematics	16,085	59,479
Physics and Astronomy	7134	25,797
Materials Science	5750	21,397
Energy	4917	22,762
Decision Sciences	3913	13,508
Environmental Science	2727	13,532
Earth and Planetary Sciences	2446	9704
Chemical Engineering	2444	10,119
Chemistry	2265	9769
Social Sciences	2154	8780
Business, Management and Accounting	1998	7447
Biochemistry, Genetics and Molecular Biology	1490	12,550
Medicine	1206	7400
Agricultural and Biological Sciences	1075	6004
Multidisciplinary	970	4193
Economics, Econometrics and Finance	444	1781
Neuroscience	419	2328
Arts and Humanities	246	1023
Pharmacology, Toxicology and Pharmaceutics	220	2157
Health Professions	179	707
Immunology and Microbiology	165	1379
Psychology	54	366
Nursing	21	106
Veterinary	7	72
Dentistry	5	51
Total	71,147	264,139

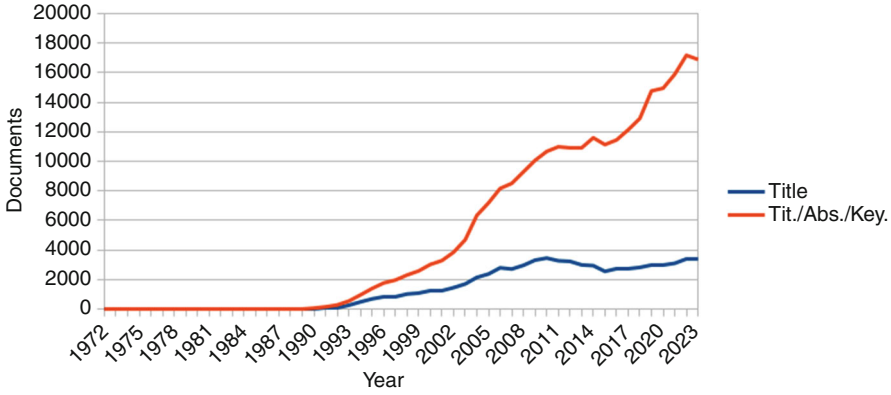


Fig. 3 GA publications per year

algorithm" in the title, abstract, or keywords, according to their subject areas. The huge amount of papers (71,147 and 264,139) and the wide spectrum of subject areas with GA applications allow us to say that the research on GAs has reached a stage of great maturity, and there is an active and vibrant worldwide community of researchers working on these algorithms, as it may be confirmed in Fig. 3, which illustrates the number of publications appeared in each year in the period 1970–2023. Specifically, we may see that from approximately the last 15 years, GAs involved more than 3500 publications, with the term in their titles, per year.

An additional remark from Table 6 concerns the outstanding activity coming from the engineering field, where challenging hard optimization problems arise, which are characterized by having multiple objectives, by being dynamic problems, by the high number of implied decision variables and the complex relationships among them, and, in many cases, by the strict feasibility constraints. To effectively face problems with such a diverse nature, the original GA scheme was extended in different ways:

- *Multiobjective GAs*. Multiobjective optimization problems require the simultaneous optimization (maximization or minimization) of several objectives that cannot be compared easily with each other. The goal of solving these problems is not to find an optimal solution, rather to find a set of nondominated solutions, the so-called Pareto set. Since the 1980s, the application of GAs in solving multiobjective optimization problems has been receiving a growing interest, and they are still one of the hottest research areas in the field of evolutionary computation [21, 30]. By evolving a population of solutions, these GA approaches are able to capture a set of nondominated solutions in a single run of the algorithm [31].
- *Evolutionary Multitasking* The work [58] stands out as a burgeoning subject within evolutionary computation. The objective is to tackle the solution of several optimization problems simultaneously by taking advantage of knowledge sharing between problems. The evolutionary paradigm is ideally suited for this

purpose given its intrinsically parallel nature through the concept of population of solutions. Within the field of multitask evolutionary algorithms, the most widely used are the *multifactorial evolutionary algorithms*. The latter facilitate the transfer of knowledge through crossover and mutation operators, leading to high-quality solutions in a more efficient manner compared to single-task evolutionary algorithms [89]. This paradigm has been successfully used for training reinforcement learning models [98] and permutation-based discrete optimization problems [113].

- *Constrained GAs*. In many science and engineering disciplines, it is common to encounter a large number of constrained optimization problems. Such problems involve an objective function that is subject to various equality and inequality constraints. The challenges in this optimization scenario arise from the various limits on the decision variables, the constraints involved, the interference among constraints, and the interrelationship between the constraints and the objective functions. Classical gradient-based optimization methods have difficulties in handling this kind of problems, as constrained optimization problems may usually lack an explicit mathematical formulation and have discrete definition domains. GAs are unconstrained search methods and lack of an explicit mechanism to bias the search in constrained search space. However, researchers have been able to tailor constraint handling techniques into GAs [22, 100, 143].
- *GAs for dynamic optimization problems*. In dynamic environments, the fitness landscape may change over time as a result of the changes of the optimization goal, problem instance, and/or some restrictions. Alternatives that were bad in the past can be good nowadays or vice versa; criteria that were important before become irrelevant now, etc., and moving in these dynamic scenarios is a challenge. For these cases, the goal of an optimization algorithm is no longer to find a satisfactory solution to a fixed problem, but to track the moving optimum in the search space as closely as possible. This poses great difficulties to standard GAs, because they cannot adapt well to the changing environment once converged. Over the past decade, an important number of GA approaches have been developed to face problems with these features, and readers are referred to Cruz et al. [24], Jin and Branke [74], Yang et al. [151] for a comprehensive overview.
- *Multimodal GAs*. Many real-world problems require an optimization algorithm that is able to explore multiple optima in their search space. However, given a problem with multiple solutions, a simple GA shall tend to converge to a single solution. As a result, various mechanisms have been proposed to stably maintain a diverse population throughout the search [87, 88, 152], thereby allowing GAs to identify multiple optima reliably in these multimodal function landscapes. Many of these methods work by encouraging artificial niche formation through sharing [53] and crowding [62, 124, 135] (see section "Strategies to Maintain Diversity").
- *Machine and Deep learning*. GAs have also been applied in a multitude of machine learning and data mining problems [144], such as feature and instance selection, rule-based classification, or frequent itemset analysis. In the realm of

deep learning neural networks, evolutionary algorithms have also been employed for tasks such as architecture search, hyperparameter optimization, and neural network optimization [2, 93, 155].

Finally, we remark that there are many open-source easy-to-use libraries for building GAs to solve real-world optimization problems. They are designed as general-purpose optimization libraries with the support of a wide range of parameters to give the user control over the GA life cycle. A variety of programming languages were employed to develop these libraries, such as *java* [137], *Python* [43], *R* [23, 125], and *C++* [102].

Conclusion

GAs have become a tool of choice since they offer practical advantages to researchers facing difficult optimization problems because they may locate high performance regions of vast and complex search spaces. Other advantages include the simplicity of the approach, their flexibility, and their robust response to changing circumstances. In the previous sections, we firstly provided a comprehensive guide for newcomers, revising the basic ingredients for designing a GA and presenting the classical GA approaches to solve three widely known optimization problems with very different characteristics. Then, we undertook the study of more complex GA instances through two fruitful research lines in this field such as the preservation of diversity in GAs and the hybridizations of GAs with other metaheuristics. With regard to the former, we analyzed the premature convergence problem in GAs and outlined different approaches that have been proposed to introduce new methods and operators that attempt to favor the diversity in GAs. Similarly, we described different schemes in the literature to hybridize GAs with other metaheuristics and presented different instantiations of these schemes that have been successfully applied. Finally, we reviewed the current activity in the GA field in terms of a number of publications and areas of interest, which shows the high level of interest in the field of GAs and predicts a promising evolution of this research area.

Cross-References

▸ Evolutionary Algorithms
▸ Memetic Algorithms
▸ Multiobjective Optimization
▸ Random-Key Genetic Algorithms: Principles and Applications

Competing Interest Declaration The author(s) has no competing interests to declare that are relevant to the content of this manuscript.

References

1. Adler D (1993) Genetic algorithm and simulated annealing: a marriage proposal. In: Proceedings of the IEEE international conference on neural network, pp 1104–1109
2. Akay B, Karaboga D, Akay R (2022) A comprehensive survey on optimizing deep learning models by metaheuristics. Artif Intell Rev 55(2):829–894
3. Al-Naqi A, Erdogan A, Arslan T (2013) Adaptive three-dimensional cellular genetic algorithm for balancing exploration and exploitation processes. Soft Comput 17(7):1145–1157
4. Alba E, Dorronsoro B (2005) The exploration/exploitation tradeoff in dynamic cellular genetic algorithms. Evol Comput IEEE Trans 9(2):126–142
5. Alba E, Tomassini M (2002) Parallelism and evolutionary algorithms. Evol Comput IEEE Trans 6(5):443–462
6. Alba E, Troya JM (2001) Analyzing synchronous and asynchronous parallel distributed genetic algorithms. Futur Gen Comput Syst 17(4):451–465
7. Araujo L, Merelo J (2011) Diversity through multiculturality: assessing migrant choice policies in an island model. Evol Comput IEEE Trans 15(4):456–469
8. Bäck T, Schütz M (1996) Intelligent mutation rate control in canonical genetic algorithms. In: Raś Z, Michalewicz M (eds) Foundations of intelligent systems. Lecture notes in computer science, vol 1079, Berlin, Heidelberg: Springer Berlin Heidelberg, pp 158–167
9. Baker J (1987) Adaptive selection methods for genetic algorithms. In: Grefenstette J (ed) International conference on genetic algorithms applications and their application. Erlbaum Associates, pp 14–21
10. Baker J (1987) Reducing bias and inefficiency in the selection algorithm. In: Proceedings of the international conference on genetic algorithms, pp 14–21
11. Baluja S, Caruanna R (1995) Removing the genetics from the standard genetic algorithm. In: Proceedings of the annual conference on machine learning, pp 38–46
12. Banzhaf W (1990) The molecular traveling salesman. Biol Cybern 64:7–14
13. Beyer H, Deb K (2001) On self-adaptive features in real-parameter evolutionary algorithms. Evol Comput IEEE Trans 5(3):250–270
14. Bhandarkar S, Zhang H (1999) Image segmentation using evolutionary computation. Evol Comput IEEE Trans 3(1):1–21
15. Blum C (2010) Hybrid metaheuristics – guest editorial. Comput Oper Res 37(3):430–431
16. Blum C, Puchinger J, Raidl G, Roli A (2011) Hybrid metaheuristics in combinatorial optimization: a survey. Appl Soft Comput 11:4135–4151
17. Caruana R, Schaffer J (1988) Representation and hidden bias: gray versus binary coding for genetic algorithms. In: Proceedings of the fifth international conference on machine learning, pp 153–162
18. Chelouah R, Siarry P (2003) Genetic and Nelder-Mead algorithms hybridized for a more accurate global optimization of continuous multiminima functions. Eur J Oper Res 148(2):335–348
19. Cheng JR, Gen M (2019) Accelerating genetic algorithms with GPU computing: a selective overview. Comput Ind Eng 128:514–525
20. Chicano F, Ochoa G, Whitley LD, Tinós R (2022) Dynastic potential crossover operator. Evol Comput 30(3):409–446
21. Coello C, Lamont G, Veldhuizen D (2006) Evolutionary algorithms for solving multi-objective problems. Springer, New York
22. Coello CAC (2002) Theoretical and numerical constraint-handling techniques used with evolutionary algorithms: a survey of the state of the art. Comput Methods Appl Mech Eng 191(11–12):1245–1287
23. Cortez P (2021) Modern optimization with R. Use R!. Springer International Publishing, New York: Springer
24. Cruz C, González JR, Pelta DA (2011) Optimization in dynamic environments: a survey on problems, methods and measures. Soft Comput 15(7):1427–1448

25. Dantzig G (1957) Discrete variable extremum problems. Oper Res 5:266–277
26. Davis L (1985) Adaptive algorithms to epistatic domains. In: Proceedings of the international conference on artificial intelligence, pp 162–164
27. De Jong K (1975) An analysis of the behavior of a class of genetic adaptive systems. PhD thesis, University of Michigan
28. De Jong K (1993) Genetic algorithms are NOT function optimizers. In: Foundations of genetic algorithms, vol 2. Morgan Kaufmann, San Mateo, CA, USA
29. De Jong K, Sarma J (1993) Generation gaps revisited. In: Foundations of genetic algorithms. Morgan Kaufmann, San Mateo, CA, USA pp 19–28
30. Deb K (2001) Multi-objective optimization using evolutionary algorithms. The Mathematical Gazette. Wiley, 87(509):409–410. https://doi.org/10.1017/S0025557200173498
31. Deb K (2008) Introduction to evolutionary multiobjective optimization. In: Branke J, Deb K, Miettinen K, Słowiński R (eds) Multiobjective optimization. Lecture notes in computer science, Berlin, Heidelberg: Springer Berlin Heidelberg, vol 5252, pp 59–96
32. Díaz-Manríquez A, Toscano G, Barron-Zambrano JH, Tello-Leal E (2016) A review of surrogate assisted multiobjective evolutionary algorithms. Computational intelligence and neuroscience 2016
33. Eiben A, Smith J (2015) Introduction to evolutionary computation. Natural computing series. Springer, Springer Berlin, Heidelberg
34. Eiben A, Hinterding R, Michalewicz Z (1999) Parameter control in evolutionary algorithms. Evol Comput IEEE Trans 3(2):124–141
35. Eklund SE (2004) A massively parallel architecture for distributed genetic algorithms. Parallel Comput 30(5–6):647–676
36. Eremeev AV, Kovalenko JV (2013) Optimal recombination in genetic algorithms. arXiv preprint arXiv:13075519
37. Eshelman L (1991) The CHC adaptive search algorithm: how to have safe search when engaging in nontraditional genetic recombination. Foundations of genetic algorithms, vol 1. Morgan Kaufmann, San Mateo, California, pp 265–283
38. Eshelman L, Schaffer J (1991) Preventing premature convergence in genetic algorithms be preventing incest. In: Proceedings of the international conference on genetic algorithms, pp 115–122
39. Eshelman L, Schaffer J (1993) Real-coded genetic algorithms and interval-schemata. In: Foundations of genetic algorithms, vol 2. Morgan Kaufmann, San Mateo, CA, pp 187–202
40. Fernandes C, Rosa A (2008) Self-adjusting the intensity of assortative mating in genetic algorithms. Soft Comput 12(10):955–979
41. Fogarty TC (1989) Varying the probability of mutation in the genetic algorithm. In: Proceedings of the third international conference on genetic algorithms, pp 104–109
42. Fogel D (1998) Evolutionary computation: The fossil record. IEEE Press, Hoboken, New Jersey
43. Gad AF (2023) PyGAD: an intuitive genetic algorithm python library. Multimedia tools and applications
44. Gao J, Sun L, Gen M (2008) A hybrid genetic and variable neighborhood descent algorithm for flexible job shop scheduling problems. Comput Oper Res 35(9):2892–2907
45. García-Martínez C, Lozano M, Herrera F, Molina D, Sánchez A (2008) Global and local real-coded genetic algorithms based on parent-centric crossover operators. Eur J Oper Res 185(3):1088–1113
46. García-Martínez C, Lozano M, Rodríguez-Díaz F (2012) A simulated annealing method based on a specialised evolutionary algorithm. Appl Soft Comput 12(2):573–588
47. García-Martínez C, Rodriguez FJ, Lozano M (2012) Arbitrary function optimisation with metaheuristics. Soft Comput 16(12):2115–2133
48. Garg H (2019) A hybrid GSA-GA algorithm for constrained optimization problems. Inf Sci 478:499–523
49. Ghannadian F, Alford C, Shonkwiler R (1996) Application of random restart to genetic algorithms. Inf Sci 95(1–2):81–102

50. Goldberg D (1989) Genetic algorithms in search, optimization, and machine learning. Addison-Wesley, New York
51. Goldberg D (1989) Sizing populations for serial and parallel genetic algorithms. In: Schaffer J (ed) International conference on genetic algorithms. Morgan Kaufmann, pp 70–79
52. Goldberg D, Lingle R (1985) Alleles, loci and the traveling salesman problem. In: Proceedings of the international conference on genetic algorithms, pp 154–159
53. Goldberg D, Richardson J (1987) Genetic algorithms with sharing for multimodal function optimization. In: Grefenstette J (ed) Proceedings of the international conference genetic algorithms. L. Erlbraum Associates, pp 41–49
54. Goldberg D, Korb B, Deb K (1990) Messy genetic algorithms: motivation, analysis, and first results. Complex Syst 3:493–530
55. Gonçalves JF, Resende MG (2011) Biased random-key genetic algorithms for combinatorial optimization. J Heuristics 17(5):487–525
56. Grosan C, Abraham A (2007) Hybrid evolutionary algorithms: methodologies, architectures, and reviews. In: Grosan C, Abraham A, Ishibuchi H (eds) Hybrid evolutionary algorithms. Springer, Berlin, Heidelberg: Springer Berlin Heidelberg, pp 1–17
57. Grötschel M, Padberg MM (1978) On the symmetric traveling salesman problem: theory and computations. In: Optimization and operations research. Lecture notes in economics and mathematical systems, vol 157. Springer, Springer Berlin, Heidelberg, pp 105–115
58. Gupta A, Ong YS, Feng L (2015) Multifactorial evolution: toward evolutionary multitasking. Evol Comput IEEE Trans 20(3):343–357
59. Gupta S, Garg ML (2013) Binary Trie coding scheme: an intelligent genetic algorithm avoiding premature convergence. Int J Comput Math 90(5):881–902
60. Guzman E, Poler R, Andres B (2023) A matheuristic approach combining genetic algorithm and mixed integer linear programming model for production and distribution planning in the supply chain. Adv Product Eng Manag 18(1):19–31
61. Han ZH, Zhang KS (2012) Surrogate-based optimization. In: Roeva O (ed) Real-world applications of genetic algorithms, chap 17. IntechOpen, Rijeka
62. Harik G (1995) Finding multimodal solutions using restricted tournament selection. In: Proceedings of the international conference on genetic algorithms. Morgan Kaufmann, pp 24–31
63. He C, Zhang Y, Gong D, Ji X (2023) A review of surrogate-assisted evolutionary algorithms for expensive optimization problems. Expert Syst Appl 217:119495
64. Herrera F, Lozano M (2000) Gradual distributed real-coded genetic algorithms. Evol Comput IEEE Trans 4(1):43–63
65. Herrera F, Lozano M (2003) Fuzzy adaptive genetic algorithms: design, taxonomy, and future directions. Soft Comput 7(8):545–562
66. Herrera F, Lozano M, Verdegay J (1998) Tackling real-coded genetic algorithms: operators and tools for behavioural analysis. Artif Intell Rev 12:265–319
67. Herrera F, Lozano M, Sánchez A (2003) A taxonomy for the crossover operator for real-coded genetic algorithms: an experimental study. Int J Intell Syst 18(3):309–338
68. Hinterding R, Michalewicz Z, Eiben A (1997) Adaptation in evolutionary computation: a survey. In: IEEE international conference on evolutionary computation, pp 65–69
69. Holland J (1975) Adaptation in natural and artificial systems. The University of Michigan Press, Ann Arbor, Michigan
70. Hussain A, Muhammad YS (2020) Trade-off between exploration and exploitation with genetic algorithm using a novel selection operator. Complex Intell Syst 6(1):1–14
71. Hutter M, Legg S (2006) Fitness uniform optimization. Evol Comput IEEE Trans 10(5):568–589
72. Iman R, Conover W (1982) A distribution-free approach to inducing rank correlation among input variables. Commun Stat Simul Comput 11(3):311–334
73. Janikow C, Michalewicz Z (1991) An experimental comparison of binary and floating point representation in genetic algorithms. In: Proceedings of the fourth international conference on genetic algorithms, pp 31–36

74. Jin Y, Branke J (2005) Evolutionary optimization in uncertain environments-a survey. Evol Comput IEEE Trans 9(3):303–317
75. Juan AA, Faulin J, Grasman SE, Rabe M, Figueira G (2015) A review of simheuristics: Extending metaheuristics to deal with stochastic combinatorial optimization problems. Oper Res Perspect 2:62–72
76. Karafotias G, Hoogendoorn M, Eiben A (2015) Parameter control in evolutionary algorithms: trends and challenges. Evol Comput IEEE Trans 19(2):167–187
77. Katoch S, Chauhan SS, Kumar V (2021) A review on genetic algorithm: past, present, and future. Multimedia Tools Appl 80:8091–8126
78. Kazarlis S, Papadakis S, Theocharis J, Petridis V (2001) Microgenetic algorithms as generalized hill-climbing operators for GA optimization. Evol Comput IEEE Trans 5(3): 204–217
79. Kazimipour B, Li X, Qin A (2014) A review of population initialization techniques for evolutionary algorithms. In: Proceedings of the IEEE congress on evolutionary computation, pp 2585–2592
80. Kita H (2001) A comparison study of self-adaptation in evolution strategies and real-coded genetic algorithms. Evol Comput 9(2):223–241
81. Kominami M, Hamagami T (2007) A new genetic algorithm with diploid chromosomes by using probability decoding for non-stationary function optimization. In: Systems, man and cybernetics, 2007. ISIC. IEEE international conference on, pp 1268–1273
82. Koumousis V, Katsaras C (2006) A saw-tooth genetic algorithm combining the effects of variable population size and reinitialization to enhance performance. Evol Comput IEEE Trans 10(1):19–28
83. Krishnakumar K (1989) Micro-genetic algorithms for stationary and non-stationary function optimization. In: Intelligent control and adaptive systems. Proceedings of the SPIE, vol 1196, pp 289–296
84. Kuo T, Hwang S (1996) A genetic algorithm with disruptive selection. Syst Man Cybern IEEE Trans 26(2):299–307
85. Kurahashi S, Terano T (2000) A genetic algorithm with tabu search for multimodal and multiobjective function optimization. In: Whitley LD, Goldberg DE, Cantú-Paz E, Spector L, Parmee IC, Beyer HG (eds) GECCO. Morgan Kaufmann, pp 291–298
86. Larrañaga P, Kuijpers C, Murga R, Inza I, Dizdarevic S (1999) Genetic algorithms for the travelling salesman problem: a review of representations and operators. Artif Intell Rev 13(2):129–170
87. Li JP, Balazs ME, Parks GT, Clarkson PJ (2002) A species conserving genetic algorithm for multimodal function optimization. Evol Comput 10(3):207–234
88. Liang Y, Leung KS (2011) Genetic algorithm with adaptive elitist-population strategies for multimodal function optimization. Appl Soft Comput 11(2):2017–2034
89. Liu Z, Li G, Zhang H, Liang Z, Zhu Z (2023) Multifactorial evolutionary algorithm based on diffusion gradient descent. IEEE Trans Cybern 54:1–13
90. Lozano M, García-Martínez C (2010) Hybrid metaheuristics with evolutionary algorithms specializing in intensification and diversification: overview and progress report. Comput Oper Res 37:481–497
91. Lozano M, Herrera F, Krasnogor N, Molina D (2004) Real-coded memetic algorithms with crossover hill-climbing. Evol Comput 12(3):273–302
92. Lozano M, Herrera F, Cano JR (2008) Replacement strategies to preserve useful diversity in steady-state genetic algorithms. Inf Sci 178(23):4421–4433
93. Ma Y, Xie Y (2022) Evolutionary neural networks for deep learning: a review. Int J Mach Learn Cybern 13(10):3001–3018
94. Mahfoud S (1992) Crowding and preselection revised. In: Männer R, Manderick B (eds) Parallel problem solving from nature, vol 2. Elsevier Science Publishers, New York, United States, pp 27–36
95. Mallipeddi R, Suganthan P (2010) Ensemble of constraint handling techniques. Evol Comput IEEE Trans 14(4):561–579

96. Maniezzo V, Boschetti M, Stützle T (2021) Matheuristics: algorithms and implementations. Springer International Publishing, New York, NY
97. Manzoni L, Mariot L, Tuba E (2020) Balanced crossover operators in genetic algorithms. Swarm Evol Comput 54:100646
98. Martinez AD, Del Ser J, Osaba E, Herrera F (2022) Adaptive multifactorial evolutionary optimization for multitask reinforcement learning. Evol Comput IEEE Trans 26(2):233–247
99. Mauldin M (1984) Maintaining diversity in genetic search. In: National conference on artificial intelligence, Austin, pp 247–250
100. Michalewicz A, Schoenauer M (1996) Evolutionary algorithms for constrained parameter optimization problems. Evol Comput 4(1):1–32
101. Michalewicz Z (1992) Genetic Algorithms + Data Structures = Evolution Programs. Springer, Heidelberg, Berlin, Germany
102. Mohammadi A, Asadi H, Mohamed S, Nelson K, Nahavandi S (2017) Openga, a C++ genetic algorithm library. In: 2017 IEEE international conference on systems, man, and cybernetics, SMC 2017, pp 2051–2056
103. Molina D, Lozano M, García-Martínez C, Herrera F (2010) Memetic algorithms for continuous optimization based on local search chains. Evol Comput 18(1):27–63
104. Molina D, Poyatos J, Ser JD, García S, Hussain A, Herrera F (2020) Comprehensive taxonomies of nature- and bio-inspired optimization: inspiration versus algorithmic behavior, critical analysis recommendations. Cogn Comput 12:897–939
105. Morales-Castañeda B, Zaldívar D, Cuevas E, Fausto F, Rodríguez A (2020) A better balance in metaheuristic algorithms: does it exist? Swarm Evol Comput 54:100671
106. Moscato P, Cotta C (2003) A gentle introduction to memetic algorithms. In: Handbook of metaheuristics. Kluwer Academic Publishers, Boston, MA: Springer US, pp 105–144
107. Nieto A, Serra M, Juan AA, Bayliss C (2023) A GA-simheuristic for the stochastic and multi-period portfolio optimisation problem with liabilities. J Simul 17(5):632–645
108. Noman N, Iba H (2008) Accelerating differential evolution using an adaptive local search. Evol Comput IEEE Trans 12(1):107–125
109. Nomura T, Shimohara K (2001) An analysis of two-parent recombinations for real-valued chromosomes in an infinite population. Evol Comput 9(3):283–308
110. Obal TM, Florentino HO, Gevert VG, Jones DF, Ouelhadj D, Volpi NMP, Wilhelm VE (2015) A matheuristic for the selection of beam directions and dose distribution in radiotherapy planning. In: Mini Euro conference on improving healthcare: new challenges, new approaches. Journal of physics conference series, vol 616
111. Oh IS, Lee JS, Moon BR (2004) Hybrid genetic algorithms for feature selection. Pattern Anal Mach Intell IEEE Trans 26(11):1424–1437
112. Oliver I, Smith D, Holland J (1987) A study of permutation crossover operators on the TSP. In: Proceedings of the international conference on genetic algorithms and their applications, pp 224–230
113. Osaba E, Martinez AD, Galvez A, Iglesias A, Ser JD (2020) dMFEA-II: an adaptive multifactorial evolutionary algorithm for permutation-based discrete optimization problems. In: Proceedings of the 2020 genetic and evolutionary computation conference companion, pp 1690–1696
114. Pereira A, de Andrade BB (2015) On the genetic algorithm with adaptive mutation rate and selected statistical applications. Comput Stat 30(1):131–150
115. Picek S, Golub M (2010) Comparison of a crossover operator in binary-coded genetic algorithms. WSEAS Trans Comput 9(9):1064–1073
116. Potts J, Giddens T, Yadav S (1994) The development and evaluation of an improved genetic algorithm based on migration and artificial selection. Syst Man Cybern IEEE Trans 24:73–86
117. Preechakul C, Kheawhom S (2009) Modified genetic algorithm with sampling techniques for chemical engineering optimization. J Ind Eng Chem 15:110–118
118. Preux P, Talbi E (1999) Towards hybrid evolutionary algorithms. Int Trans Oper Res 6(6):557–570

119. Rabet R, Ganji M, Fathi M (2024) A simheuristic approach towards supply chain scheduling: integrating production, maintenance and distribution. Appl Soft Comput 153:111264
120. Raidl G (2006) A unified view on hybrid metaheuristics. In: Almeida F, Aguilera MB, Blum C, Vega JM, Pérez MP, Roli A, Sampels M (eds) Hybrid metaheuristics. LNCS, vol 4030. Springer, Berlin, Heidelberg: Springer Berlin Heidelberg, pp 1–12
121. Ramos-Figueroa O, Quiroz-Castellanos M, Mezura-Montes E, Kharel R (2021) Variation operators for grouping genetic algorithms: a review. Swarm Evol Comput 60:100796
122. Reeves C (2010) Genetic algorithms. In: Handbook of metaheuristics, vol 146. Springer, New York: Springer, pp 109–139
123. Rodriguez F, Garcia-Martinez C, Lozano M (2012) Hybrid metaheuristics based on evolutionary algorithms and simulated annealing: taxonomy, comparison, and synergy test. Evol Comput IEEE Trans 16(6):787–800
124. Sareni B, Krahenbuhl L (1998) Fitness sharing and niching methods revisited. Evol Comput IEEE Trans 2(3):97–106
125. Scrucca L (2013) GA: a package for genetic algorithms in R. J Stat Softw 53(4):1–37
126. Serpell M, Smith J (2010) Self-adaption of mutation operator and probability for permutation representations in genetic algorithms. Evol Comput 18(3):491–514
127. Smith A, Coit D, Baeck T, Fogel D, Michalewicz Z (1997) Penalty functions. In: Handbook on evolutionary computation. Oxford University Press, Boca Raton, pp C5.2:1–C5.2:6
128. Smith JE, Fogarty TC (1997) Operator and parameter adaptation in genetic algorithms. Soft Comput 1(2):81–87
129. Srinivas M, Patnaik L (1994) Adaptive probabilities of crossover and mutation in genetic algorithms. Syst Man Cybern IEEE Trans 24(4):656–667
130. Sun N, Lu Y (2019) A self-adaptive genetic algorithm with improved mutation mode based on measurement of population diversity. Neural Comput Appl 31(5):1435–1443
131. Syswerda G (1989) Uniform crossover in genetic algorithms. In: Proceedings of the international conference on genetic algorithms, pp 2–9
132. Talbi E (2002) A taxonomy of hybrid metaheuristics. J Heuristics 8(5):541–564
133. Talbi EG, Bachelet V (2006) Cosearch: a parallel cooperative metaheuristic. J Math Model Algorithms 5(1):5–22
134. Tantar A, Melab N, Talbi E (2008) A grid-based genetic algorithm combined with an adaptive simulated annealing for protein structure prediction. Soft Comput 12(12):1185–1198
135. Thierens D (1998) Selection schemes, elitist recombination, and selection intensity. In: Proceedings of the 7th international conference on genetic algorithms. Morgan Kaufmann, pp 152–159
136. Ting CK, Li ST, Lee C (2003) On the harmonious mating strategy through tabu search. Inf Sci 156:189–214
137. Troiano L, De Pasquale D (2009) A Java library for genetic algorithms addressing memory and time issues. In: 2009 world congress on nature and biologically inspired computing, NABIC 2009 – proceedings, pp 642–647
138. Tuson A, Ross P (1998) Adapting operator settings in genetic algorithms. Evol Comput 6(2):161–184
139. Umbarkar AJ, Sheth PD (2015) Crossover operators in genetic algorithms: a review. ICTACT J Soft Comput 6(1):1083–1092
140. Uyar Ac, Harmanci AE (2005) A new population based adaptive domination change mechanism for diploid genetic algorithms in dynamic environments. Soft Comput 9(11):803–814
141. van Kemenade C, Kok J, Eiben AE (1995) Raising GA performance by simultaneous tuning of selective pressure and recombination disruptiveness. In: Proceedings of the 1995 IEEE congress on evolutionary computation (CEC 1995), pp 346–351
142. Črepinšek M, Liu SH, Mernik M (2013) Exploration and exploitation in evolutionary algorithms: a survey. ACM Comput Surv 45(3):35:1–35:33
143. Venkatraman S, Yen G (2005) A generic framework for constrained optimization using genetic algorithms. Evol Comput IEEE Trans 9(4):424–435

144. Ventura S, Luna JM et al (2016) Pattern mining with evolutionary algorithms, vol 16. Springer, Berlin
145. Vrugt J, Robinson B, Hyman J (2009) Self-adaptive multimethod search for global optimization in real-parameter spaces. Evol Comput IEEE Trans 13(2):243–259
146. Whitley D (1989) The GENITOR algorithm and selection pressure: why rank-based allocation of reproductive trials is best. In: Proceedings of the international conference on genetic algorithms. Morgan Kaufmann, pp 116–121
147. Wolpert DH, Macready WG (1997) No free lunch theorems for optimization. Evol Comput IEEE Trans 1(1):67–82
148. Wong YY, Lee KH, Leung KS, Ho CW (2003) A novel approach in parameter adaptation and diversity maintenance for genetic algorithms. Soft Comput 7(8):506–515
149. Woo YB, Kim BS (2018) Matheuristic approaches for parallel machine scheduling problem with time-dependent deterioration and multiple rate-modifying activities. Comput Oper Res 95:97–112
150. Yang CH, Nygard K (1993) Effects of initial population in genetic search for time constrained traveling salesman problems. In: Proceedings of the ACM computer science conference, pp 378–383
151. Yang S, Ong Y, Jin Y (eds) (2007) Evolutionary computation in dynamic and uncertain environments. Studies in computational intelligence, vol 51. Springer, Berlin, Heidelberg
152. Yao J, Kharma N, Grogono P (2010) Bi-objective multipopulation genetic algorithm for multimodal function optimization. Evol Comput IEEE Trans 14(1):80–102
153. Yeniay Ö (2005) Penalty function methods for constrained optimization with genetic algorithms. Math Comput Appl 10(1):45–56
154. Yuen SY, Chow CK (2009) A genetic algorithm that adaptively mutates and never revisits. Evol Comput IEEE Trans 13(2):454 –472
155. Zebari RR, Zeebaree SRM, Rashid ZN, Shukur HM, Alkhayyat A, Sadeeq MAM (2021) A review on automation artificial neural networks based on evolutionary algorithms. In: 2021 14th international conference on developments in eSystems engineering (DeSE), pp 235–240
156. Zhao Y, Chen L, Xie G, Zhao J, Ding J (2018) GPU implementation of a cellular genetic algorithm for scheduling dependent tasks of physical system simulation programs. J Comb Optim 35(1):293–317

GRASP

23

Daniele Ferone, Paola Festa, and Mauricio G. C. Resende

Contents

Introduction 664
Basic Components 665
Construction Phase 666
Local Search Phase 669
Enhancements 670
Reactive GRASP 670
Cost Perturbations 671
Bias Functions 672
Hybridizations 673
Continuous GRASP 676
Conclusions 678
Cross-References 678
References 679

Abstract

GRASP (greedy randomized adaptive search procedures) is a multi-start metaheuristic designed to find high-quality solutions for combinatorial optimization problems. Each iteration of GRASP typically consists of two phases: a construction phase, where a feasible solution is built, and a local search phase, which begins from the constructed solution and performs iterative improvements until a locally optimal solution is achieved. While the construction phase is

D. Ferone · P. Festa (✉)
Department of Mathematics and Applications "R. Caccioppoli", University of Napoli FEDERICO II, Naples, Italy
e-mail: daniele.ferone@unina.it; paola.festa@unina.it

M. G. C. Resende
Industrial and Systems Engineering, University of Washington, Seattle, WA, USA

© Springer Nature Switzerland AG 2025

R. Martí et al. (eds.), *Handbook of Heuristics*,
https://doi.org/10.1007/978-3-032-00385-0_23

usually based on a randomized greedy algorithm, other construction methods have also been explored. Repeated use of the construction procedure generates diverse starting solutions for the local search. This chapter provides an overview of GRASP, outlining its fundamental components and various enhancements to the basic approach, such as Reactive GRASP, intensification strategies, and random-key GRASP.

Keywords

GRASP · Continuous GRASP · Metaheuristics

Introduction

In a combinatorial optimization problem, given a finite (or countably infinite) solution set X and a real-valued objective function $f : X \to \mathbb{R}$, the goal is to find a solution $x^* \in X$ such that $f(x^*) \leq f(x)$ for all $x \in X$. While some of these problems can be solved in polynomial time, many are computationally intractable, as no exact polynomial-time algorithms are currently known to solve them [72]. Moreover, most real-world problems are either inherently computationally intractable or too large for exact algorithms to be practical. In these cases, metaheuristic methods are commonly used to find good, though not necessarily optimal, solutions. These methods typically alternate between phases of intensification and diversification. Intensification refers to focusing the search in a promising area of the solution space to refine and improve the current solution, often by applying more thorough local search techniques. On the other hand, diversification aims to explore new and potentially unexplored regions of the solution space, preventing the search from becoming stuck in local optima. Over time, various heuristic search techniques have been developed that effectively balance these two phases, significantly improving our ability to find high-quality solutions to complex combinatorial optimization problems. These techniques include simulated annealing [119], tabu search [73], genetic algorithms [101], biased random-key genetic algorithms [104], variable neighborhood search [85], adaptive large neighborhood search [151], and GRASP (greedy randomized adaptive search procedures) [52, 53].

GRASP is a multi-start metaheuristic designed to find good-quality solutions for combinatorial optimization problems. Each iteration consists of a construction phase, where a solution (which can be feasible or infeasible) is built, followed by a local search phase. The local search begins at the constructed solution and iteratively improves it until a locally optimal solution is reached. Although the construction phase is typically based on a randomized greedy algorithm, other construction procedures have been proposed. Repeated applications of the construction procedure generate diverse starting solutions for the local search, and the best local optimum found across all iterations is returned as the final solution.

This chapter overviews GRASP by describing its basic components along with some among the most fruitful proposed enhancements, including reactive GRASP,

Table 1 Applications of GRASP: operations research problems

Routing	[17, 20, 24, 32, 36, 95, 137]
Logic	[45, 50, 70, 121, 132, 136]
Covering and partition	[15, 16, 52, 74, 83, 118, 130]
Location	[1, 42–44, 60, 93, 150]
Minimum Steiner tree	[33, 108, 109, 141]
Optimization in graphs	[2, 19, 35, 55, 59, 99, 120, 133, 148]
Assignment	[5, 51, 71, 102, 112, 122, 123, 135, 145]
Timetabling and scheduling	[4, 7, 11, 12, 25, 29, 54, 56, 64, 81, 96, 105, 139, 142–144]

Table 2 Applications of GRASP: some industrial applications

Manufacturing	[6, 22, 23, 30, 94, 115]
Transportation	[17, 21, 51, 63, 147]
Telecommunications	[9, 10, 41, 93, 103, 125, 128, 149]
Graph and map drawing	[68, 69, 98, 107, 116, 133]
Power systems	[27, 28, 49]
Computational biology	[65, 79, 87]
VLSI	[14, 15]

intensification strategies, and hybridization with other metaheuristics. The chapter is organized as follows. Basic components, alternative construction mechanisms, and local search characteristics are described in the next section. Enhancements to the basic procedure, including reactive GRASP and intensification strategies, are discussed in section "**Enhancements**." Section "**Hybridizations**" describes several state-of-the-art hybridizations of GRASP with other metaheuristics, while section "**Continuous GRASP**" describes how GRASP has been extended to continuous optimization problems. Tables 1 and 2 report a number of GRASP implementations that have appeared in the literature, covering a wide range of applications in several and heterogenous fields. Moreover, the reader can refer to [66, 67, 134].

Basic Components

Given a finite solution set X and a real-valued objective function $f : X \to R$ to be minimized, a basic GRASP metaheuristic [52, 53] is a multi-start or iterative method, in which each iteration consists of two phases: construction of a solution and local search.

The construction phase builds a solution x, usually feasible, but optionally also infeasible. If x is not feasible, a repair procedure may be invoked to obtain feasibility. Once a solution x is obtained, its neighborhood is investigated by the

```
algorithm GRASP(f(·), g(·), MaxIterations, Seed)
1    x_best:=∅; f(x_best):=+∞;
2    for k = 1, 2, ...,MaxIterations→
3       x:=ConstructGreedyRandomizedSolution(Seed, g(·));
4       if (x not feasible) then
5          x:=repair(x);
6       endif
7       x:=LocalSearch(x, f(·));
8       if (f(x) < f(x_best)) then
9          x_best:=x;
10      endif
11   endfor;
12   return(x_best);
end GRASP
```

Fig. 1 Pseudo-code of a basic GRASP for a minimization problem

local search until a local minimum is found. The best overall solution is kept as the result. The pseudo-code in Fig. 1 illustrates the main blocks of a GRASP procedure for minimization, in which `MaxIterations` iterations are performed and `Seed` is used as the initial seed for the pseudorandom number generator.

Construction Phase

Starting from an empty solution, a complete solution is iteratively constructed in the construction phase, one element at a time (see Fig. 2). At each construction iteration, the choice of the next element to be added is determined by ordering all candidate elements (i.e., those that can be added to the solution) in a candidate list C with respect to a greedy function $g : C \rightarrow R$. This function measures the (myopic) benefit of selecting each element. The heuristic is adaptive because the benefits associated with every element are updated at each iteration of the construction phase to reflect the changes brought on by the selection of the previous element. The probabilistic component of a GRASP is characterized by randomly choosing one of the best candidates in the list, but not necessarily the top candidate. The

```
procedure ConstructGreedyRandomizedSolution(Seed, g(·))
1    x:=∅;
2    Sort the candidate elements i according to their incremental costs g(i);
3    while (x is not a complete solution)→
4       RCL:=MakeRCL();
5       v:=SelectIndex(RCL, Seed);
6       x := x ∪ {v};
7       Resort remaining candidate elements j according to g(j);
8    endwhile;
9    return(x);
end ConstructGreedyRandomizedSolution;
```

Fig. 2 Basic GRASP construction phase pseudo-code

list of best candidates is called the *restricted candidate list* (RCL). This choice technique allows for different solutions to be obtained at each GRASP iteration but does not necessarily compromise the power of the adaptive greedy component of the method.

The most used technique to build the RCL applies the min max–α percentage rules. At any GRASP iteration, let $g_{\min}$ and $g_{\max}$ be the smallest and largest incremental costs, respectively, i.e.,

$$g_{\min} = \min_{i \in C} g(i), \qquad g_{\max} = \max_{i \in C} g(i). \tag{1}$$

The RCL is made up of elements $i \in C$ with the best (i.e., the smallest) incremental costs $g(i)$. There are two main mechanisms to build this list: *cardinality-based* (CB) and *value-based* (VB) mechanisms. In the CB case, the RCL is made up of the k elements with the best incremental costs, where k is a parameter. In the VB case, the RCL is associated with a parameter $\alpha \in [0, 1]$ and a threshold value $\mu = g_{\min} + \alpha(g_{\max} - g_{\min})$. All candidate elements i whose incremental cost $g(i)$ is no greater than the threshold value are inserted into the RCL, i.e., $g(i) \in [g_{\min}, \mu]$. Note that the case $\alpha = 0$ corresponds to a pure greedy algorithm, while $\alpha = 1$ is equivalent to a random construction. The pseudo-code in Fig. 3 is a refinement of the greedy randomized construction pseudo-code shown in Fig. 2.

Prais and Ribeiro in [126,127] analyzed GRASP with different RCL construction mechanisms, varying the parameter α in three ways:

```
procedure ConstructGreedyRandomizedSolution(Seed, α, k, g(·))
1    x:=∅;
2    Initialize the candidate set C by all elements;
3    Evaluate the incremental cost g(i) for all i ∈ C;
4    while (|C| > 0)→
5        g_min := min_{i∈C} g(i); g_max := max_{i∈C} g(i);
6        if (CB RCL is used) then
7            Sort candidate elements i ∈ C according to g(i);
8            RCL:=C[1 ··· k];
9        else RCL:={i ∈ C | g(i) ≤ g_min + α(g_max − g_min)};
10       endif;
11       v:=SelectIndex(RCL, Seed);
12       x := x ∪ {v};
13       Update the candidate set C;
14       Reevaluate the incremental costs g(i) for all i ∈ C;
15   endwhile;
16   return(x);
end ConstructGreedyRandomizedSolution;
```

Fig. 3 Refined pseudo-code of the GRASP construction phase

(a) α randomly chosen from a uniform distribution
(b) α randomly chosen from a decreasing nonuniform distribution
(c) Fixed α, close to the greedy choice

They applied these strategies to four optimization problems, and they found that strategy (c) had the shortest computation times and less variability, while strategy (a) achieved more frequent hits, showing the effectiveness of varying α.

Additionally, they tested a self-adjusting version of GRASP, where α is modified based on solution quality. This extension is known as Reactive GRASP and will be discussed further in the next section.

Local Search Phase

As is the case for many deterministic methods, the solutions generated by a GRASP construction are not guaranteed to be locally optimal with respect to simple neighborhood definitions. Hence, it is almost always beneficial to apply a local search to attempt to improve each constructed solution. A local search algorithm works in an iterative fashion by successively replacing the current solution by a better solution in the neighborhood of the current solution. It terminates when no better solution is found in the neighborhood. The *neighborhood structure* N for a problem relates a solution s of the problem to a subset of solutions $N(s)$. A solution s is said to be *locally optimal* if there is no better solution in $N(s)$ with respect to the objective function value. The key to success for a local search algorithm consists of the suitable choice of a neighborhood structure, efficient neighborhood search techniques, and the starting solution. Figure 4 illustrates the pseudo-code of a generic local search procedure for a minimization problem.

In a GRASP framework, a local search starts from an initial solution $x_0 \in X$ and iteratively generates a sequence of improving solutions $x_1, \ldots, x_M$, where $M =$ `MaxIterations`. At the k-th iteration, $k = 1, \ldots, M$, x_k is locally optimal with respect to the neighborhood $N(x_{k-1})$ since $N(x_{k-1})$ is searched for an improving solution x_k such that $f(x_k) < f(x_{k-1})$. If such a solution is found, it is made the current solution. Otherwise, the search ends with x_{k-1} as a local optimum.

The effectiveness of local search depends on several factors, such as the neighborhood structure, the function to be minimized, and the starting solution. It has been experimentally shown that randomly generated solutions are of poor quality on average. On the other hand, greedy algorithms usually produce solutions of better quality than those of randomly generated solutions. Therefore, using

```
procedure LocalSearch(x, f(·))
1    Let N(x) be the neighborhood of x;
2    H:={y ∈ N(x) | f(y) < f(x)};
3    while (|H| > 0)→
4        x:=Select(H);
5        H:={y ∈ N(x) | f(y) < f(x)};
6    endwhile
7    return(x);
end LocalSearch
```

Fig. 4 Pseudo-code of a generic local search procedure

greedy solutions as starting points for local search in a multi-start procedure will usually lead to good, though, most often, suboptimal solutions. This is because the amount of variability in greedy solutions is small, and it is less likely that a greedy starting solution will be in the basin of attraction of a global optimum than a random solution. A greedy randomized construction as the one embedded in GRASP adds variability to the greedy algorithm.

Enhancements

To improve the performance of the basic GRASP framework, most efforts have focused on construction mechanisms. Since Mockus et al. [114] pointed out that GRASP with a fixed nonzero RCL parameter α is not asymptotically convergent to a global optimum (During construction, a fixed RCL parameter may rule out a candidate that is present in all optimal solutions.), several remedies have been proposed to get around this problem. They include reactive GRASP, cost perturbations in place of randomized selection, bias functions, memory and learning, and local search on partially constructed solutions.

Reactive GRASP

The results of the study conducted in [126, 128] involving variation of the value of the RCL parameter α motivated the proposal of the extension of the basic GRASP called reactive GRASP. Prais and Ribeiro in [128] have shown that using a single fixed value for the value of RCL parameter α very often hinders finding a high-quality solution, which eventually could be found if another value was used. Moreover, one drawback of the basic GRASP is the lack of *learning* from the history of solutions found in previous iterations. The basic algorithm discards information about any solution encountered that does not improve the incumbent. Instead, it is worth to use information gathered from good solutions leading to *memory-based* procedures. Information about the quality of previously generated solutions can influence the construction phase, by modifying the selection probabilities associated with each element of the RCL.

In this paragraph, we describe reactive GRASP, the first enhancement that incorporates a learning mechanism in the memoryless construction phase of the basic GRASP. In reactive GRASP, the value of the RCL parameter α is selected in each iteration from a discrete set of possible values with a probability that depends on the solution values found along the previous iterations. One way to accomplish this is to use the rule proposed in [128]. Let $\mathcal{A} = \{\alpha_1, \alpha_2, \ldots, \alpha_m\}$ be the set of possible values for α. At the first GRASP iteration, all m values have the same probability to be selected, i.e.,

$$p_i = \frac{1}{m}, \qquad i = 1, 2, \ldots, m. \tag{2}$$

At any subsequent iteration, let $\hat{z}$ be the incumbent solution and let A_i be the average value of all solutions found using $\alpha = \alpha_i$, $i = 1, \ldots, m$. The selection probabilities are periodically reevaluated as follows:

$$p_i = \frac{q_i}{\sum_{j=1}^{m} q_j}, \tag{3}$$

where $q_i = \hat{z}/A_i$, $i = 1, \ldots, m$. If values of $\alpha = \alpha_i$ ($i \in \{1, \ldots, m\}$) lead to the best solutions on average, then the value of q_i is increased and larger values of q_i correspond to more suitable values for α. The probabilities associated with these more appropriate values will then increase when they are reevaluated.

Due to greater diversification and less reliance on parameter tuning, reactive GRASP has been successfully applied to several problems and has led to improvements over the basic GRASP in terms of robustness and solution quality [8,26,146].

Cost Perturbations

Another step toward an improved and alternative solution construction mechanism is to allow *cost perturbations*. The idea is to introduce some "noise" in the original costs in a fashion that resembles the noising method of Charon and Hudry [38, 39]. Cost perturbations are effective in all cases when the construction algorithm is not very sensitive to randomization, as, for example, in the case of the Steiner problem in graphs [62].

Another situation where cost perturbations can be effective is when there is no greedy algorithm available for the problem to be solved, as, for example, in the case of the prize-collecting Steiner tree problem. To solve this problem, the hybrid GRASP procedure proposed by Canuto et al. in [34] used the primal-dual algorithm of Goemans and Williamson [80] to build initial solutions using perturbed costs. More in detail, in [34], at each iteration a new solution for the prize-collecting Steiner tree problem is built using node prizes updated by a perturbation function, according to the structure of the current solution. Two different prize perturbation schemes are used to enforce search diversification, as described in the following.

Perturbation by eliminations: The primal-dual algorithm used in the construction phase is driven to build a new solution without some of the nodes appearing in the solution constructed in the previous iteration. This is done by changing to zero the prizes of some persistent nodes, which appeared in the last solution built and remained at the end of the local search. A parameter μ controls the fraction of the persistent nodes whose prizes are temporarily set to zero.

Perturbation by prize changes: Similarly to the noising method of Charon and Hudry [38, 39], some noise is introduced into the node prizes, resulting in a change of the objective function as well. For each node i, a perturbation factor $\beta(i)$ is randomly generated in the interval $[1 - a, 1 + a]$, where a is

an implementation parameter. The original prize $\pi(i)$ associated with node i is temporarily changed to $\pi(i) = \pi(i) \cdot \beta(i)$.

Experimental results have shown that embedding a strategy of costs perturbation into a GRASP framework improves the best overall results. The hybrid GRASP proposed for the Steiner problem in graphs by Ribeiro et al. in [141] uses this cost perturbation strategy and is effective.

Bias Functions

Another construction mechanism was proposed by Bresina [31]. Once built the RCL, instead of choosing with equal probability one candidate among the RCL elements, Bresina introduced a family of probability distributions to bias the selection toward some particular candidates. A bias function is based on a rank $r(x)$ assigned to each candidate x according to its greedy function value and is evaluated only for the elements in RCL. Several different bias functions were introduced:

i. Random bias: $\text{bias}(r(x)) = 1$
ii. Linear bias: $\text{bias}(r(x)) = 1/r(x)$
iii. Log bias: $\text{bias}(r(x)) = \log^{-1}[r(x) + 1]$
iv. Exponential bias: $\text{bias}(r(x)) = e^{-r}$
v. Polynomial bias of order n: $\text{bias}(r(x)) = r^{-n}$

Let $\text{bias}(r(x))$ be one of the bias functions defined above. Once these values have been evaluated for all elements of the RCL, the probability p_x of selecting element x is

$$p_x = \frac{\text{bias}(r(x))}{\sum_{y \in \text{RCL}} \text{bias}(r(y))}. \tag{4}$$

A successful application of Bresina's bias function can be found in [29], where experimental results show that, although valid on all candidates, the evaluation of bias functions may be restricted only to the elements of the RCL.

Biased randomized GRASP (BR-GRASP) is a similar approach proposed in [61]. It applies a nonuniform, skewed probability distribution to the element selection process during the solution construction phase, but respect to Bresina [31], BR-GRASP allows all potential elements to be considered, with biased probabilities assigned to them. This change helps the algorithm converge more quickly to high-quality solutions, while keeping computational effort and the need for additional parameters minimal. Usually, the bias is given by a geometric distribution with a parameter that skews the probabilities.

Reactive GRASP has been the first and very simple attempt to enhance the basic GRASP in order to save and use history from previous iterations. Another very simple attempt is due to Fleurent and Glover [71] who proposed improved

constructive multi-start strategies that besides defining a special bias function also maintain a pool of *elite solutions* to be used in the construction phase. To become an elite solution, a solution must be either better than the best member of the pool, or better than its worst member and sufficiently different from the other solutions in the pool, in order to preserve not only solution quality but also the diversity of solutions. Fleurent and Glover defined: (1) a *strongly determined variable* as one that cannot be changed without eroding the objective or changing significantly other variables; (2) a *consistent variable* as one that receives a particular value in a large portion of the elite solution set, and (3) for each solution component i, a measure $I(i)$ of its strongly determined and consistent features that becomes larger as i appears more often in the pool of elite solutions. The intensity function $I(i)$ is used in the construction phase as follows. Recall that $g(i)$ is the greedy function, i.e., the incremental cost associated with the insertion of element i into the solution under construction. Let $K(i) = F(g(i), I(i))$ be a function of the greedy and the intensification functions. The idea of Fleurent and Glover is to define a special bias function that depends on $K(\cdot)$. In fact, the intensification scheme biases selection from the RCL to those elements i with a high value of $K(i)$ by setting its selection probability to be

$$p_i = \frac{K(i)}{\sum_{y \in \text{RCL}} K(y)}. \tag{5}$$

They suggested $K(i) = \lambda g(i) + I(i)$, with $K(i)$ varying with time by changing the value of λ, e.g., initially λ may be set to a large value that is decreased when diversification is called for. Rules and procedures for changing the value of λ are given by Fleurent and Glover [71] and Binato et al. [29].

Hybridizations

As enhancements to its basic framework, different hybridizations of GRASP with several other metaheuristics have been studied and proposed in the literature. In this section, some of them are surveyed and briefly described.

Laguna and Gonzalez-Velarde in 1991 [96] have first studied hybridization of GRASP with tabu search. Recently, in [131], GRASP is hybridized with tabu search to solve a variant of the multi-compartment vehicle routing problem (MCVRP). The hybrid approach uses GRASP to construct an initial solution and improves it by the tabu search algorithm, which intensifies the search by exploring the solution space more thoroughly. This combination enables the algorithm to balance exploration and exploitation, improving the overall solution quality by refining the initial GRASP solution while avoiding cycles and enhancing diversification. The proposed hybrid GRASP/TS method outperforms other heuristics in terms of solution quality and computational efficiency for the MCVRP.

GRASP has been used also in conjunction with genetic algorithms. Basically, the feasible solution found by using a GRASP construction phase has been used as

initial population by a genetic algorithm, as, for example, in [18] and in [3], where a greedy genetic algorithm is proposed for the quadratic assignment problem.

Another interesting hybridization of GRASP involves VNS (variable neighborhood search) and variable neighborhood descent (VND) proposed by Hansen and Mladenović [84, 113]. Almost all randomization effort in the basic GRASP algorithm involves the construction phase, while local search stops at the first local optimum. On the other hand, strategies such as VNS and VND rely almost entirely on the randomization of the local search to escape from local optima. With respect to this issue, probabilistic strategies such as GRASP and VNS may be considered as complementary and potentially capable of leading to effective hybrid methods.

A first attempt in the direction of integrating VNS into GRASP was done by Martins et al. [110]. The construction phase of their hybrid heuristic for the Steiner problem in graphs follows the greedy randomized strategy of GRASP, while the local search phase makes use of two different neighborhood structures as a VND strategy. Their heuristic was later improved by Ribeiro, Uchoa, and Werneck [140], one of the key components of the new algorithm being another strategy for the exploration of different neighborhoods. Ribeiro and Souza [138] also combined GRASP with VND in a hybrid heuristic for the degree-constrained minimum spanning tree problem. Festa et al. [69] studied different variants and combinations of GRASP and VNS for the MAX-CUT problem, finding and improving some of the solutions that at the time were the best known solutions for some open instances from the literature. In [57], the authors studied several hybridization of GRASP, including VNS, for the far from most string problem. Recently, Di Puglia Pugliese et al. [46] proposed a hybridization of GRASP with VND to solve a vehicle routing problem with occasional drivers. The VND has been used as local search phase, after a biased randomized greedy construction procedure.

At last, we devote the remainder of this section to the combination of the basic GRASP with path relinking. Path relinking was originally proposed by Glover [75] as an intensification strategy exploring trajectories connecting elite solutions obtained by tabu search or scatter search [76–78]. It can be traced back to the pioneering work of Kernighan and Lin [92]. Starting from one or more elite solutions, paths in the solution space leading toward other guiding elite solutions are generated and explored in the search for better solutions. This is accomplished by selecting moves that introduce attributes contained in the guiding solutions. At each iteration, all moves that incorporate attributes of the guiding solution are analyzed, and the move that best improves (or least deteriorates) the initial solution is chosen.

The first proposal of a hybrid GRASP with path relinking was in 1999 due to Laguna and Martí [97]. It was followed by several extensions, improvements, and successful applications [37, 47, 48, 58, 111, 124, 129]. Path relinking is applied to a pair of solutions $\mathbf{x}$ and $\mathbf{y}$, where one can be the solution obtained from the current GRASP iteration, and the other is a solution from an elite set of solutions. $\mathbf{x}$ is called the *initial solution*, and $\mathbf{y}$ the *guiding solution*. The set $\mathcal{E}$ of elite solutions has usually a fixed size that does not exceed `MaxElite`. Given the pair $\mathbf{x}, \mathbf{y}$, the procedure starts by computing the symmetric difference $\Delta(\mathbf{x}, \mathbf{y})$ between the two solutions, i.e., the set of moves needed to reach $\mathbf{y}$ (target solution) from $\mathbf{x}$ (initial

solution). A path of solutions is generated linking **x** and **y**. The best solution x^* in this path is returned by the algorithm. Since there is no guarantee that x^* is locally optimal, often local search is applied, starting from x^*, and the resulting locally optimal solution is returned.

Figure 5 illustrates the pseudo-code of the path-relinking procedure applied to the pair of solutions **x** (starting solution) and **y** (target solution). In line 1, an initial solution **x** is selected at random among the elite set of elements, and usually, it differs sufficiently from the guiding solution **y**. The loop in lines 6 through 14 computes a path of solutions $\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_{d(\mathbf{x},\mathbf{y})-2}$, local search is applied in line 15, and the solution x^* with the best objective function value is returned in line 16.

```
algorithm Path-relinking(f(·), x, E)
1   Choose, at random, a pool solution y ∈ E to relink with x;
2   Compute symmetric difference Δ(x, y);
3   f* := min{f(x), f(y)};
4   x* := arg min{f(x), f(y)};
5   x := x;
6   while (Δ(x, y) ≠ ∅) →
7       m* := arg min{f(x ⊕ m) | m ∈ Δ(x, y)};
8       Δ(x ⊕ m*, y) := Δ(x, y) \ {m*};
9       x := x ⊕ m*;
10      if (f(x) < f*) then
11          f* := f(x);
12          x* := x;
13      endif;
14  endwhile;
15  x* := LocalSearch(x*, f(·));
16  return (x*);
end Path-relinking
```

Fig. 5 Pseudo-code of a generic path relinking for a minimization problem

This is achieved by advancing one solution at a time in a greedy manner. At each iteration, the procedure examines all moves $m \in \Delta(x, \mathbf{y})$ from the current solution x and selects the one which results in the least cost solution (line 7), i.e., the one which minimizes $f(x \oplus m)$, where $x \oplus m$ is the solution resulting from applying move m to solution x. The best move m^* is made, producing solution $x \oplus m^*$ (line 9). The set of available moves is updated (line 8). If necessary, the best solution x^* is updated (lines 10–13). The procedure terminates when $\mathbf{y}$ is reached, i.e., when $\Delta(x, \mathbf{y}) = \emptyset$, returning the best solution found.

The procedure described above is the greedy PR, and it is just one way to apply path relinking. In the context of GRASP, path relinking (PR) has been applied in various forms, which enhance the search for better solutions. The main types of path relinking are greedy, random, and mixed. Randomized PR introduces a random element in selecting the moves, which helps explore diverse solutions and avoid local optima. Mixed PR combines both strategies, alternating between greedy and random moves during the path construction. This method aims to balance the exploration of the solution space and the exploitation of promising solutions. The choice between these strategies often depends on the specific problem and the desired balance between solution quality and computational effort.

Moreover, the concept of multiple parents introduces a novel strategy where several guiding solutions, rather than just one, are used to generate the path from the initiating solution. This approach creates a broader search trajectory, allowing the relinking process to combine attributes from multiple solutions, potentially leading to better solutions by exploring more diverse areas of the solution space. The path from the initiating solution to the guiding solutions is constructed by incorporating attributes from the guiding set based on their frequency of occurrence, which helps preserve beneficial solution characteristics. Additionally, interior and exterior PRs refer to different ways of managing the solutions during the relinking process. Interior PR focuses on incorporating attributes that are present in the guiding solution but not in the initiating solution, progressively modifying the initiating solution by adding the missing attributes. Exterior PR, in contrast, seeks to introduce elements that are not present in either the initiating or guiding solution, thereby creating more diversified intermediate solutions. This form of PR is particularly useful when diversification is required to avoid premature convergence to local optima. Both approaches aim to enrich the search either by focusing on attributes that are common in high-quality solutions or by exploring more distant solution spaces. Readers interested in further details can explore Laguna et al. [100].

Continuous GRASP

Continuous GRASP (C-GRASP) has been proposed in [86] and is an extension of the traditional GRASP algorithm, adapted for solving continuous global optimization problems. Unlike its discrete counterpart, which operates on combinatorial spaces, C-GRASP targets problems in continuous domains, specifically those constrained by box constraints. The algorithm follows a multi-start approach,

alternating between construction and local improvement phases, similar to the original GRASP method. During each iteration, C-GRASP constructs a solution using a greedy randomized approach, where each component of the solution is determined by evaluating the best possible outcome in its local search space. The local improvement phase refines the solution further by exploring the neighboring region for better solutions without relying on derivative information, making C-GRASP particularly useful for "black-box" problems where derivative-based optimization methods cannot be applied. C-GRASP's effectiveness lies in its ability to adapt the search resolution based on the progress of the solution, intensifying the search when improvements are slow. This makes it a robust and flexible tool for tackling complex global optimization problems in various fields.

In the construction phase, an initial solution is generated using a greedy randomized approach tailored for continuous optimization. The process begins by selecting a random point within the feasible region defined by the problem's constraints, such as the lower and upper bounds for each variable. Initially, all the coordinates of the solution are considered "unfixed," meaning that each dimension of the solution can vary. For each unfixed coordinate, a line search is conducted along that coordinate's direction while holding the other coordinates fixed. This line search aims to find the value that minimizes the objective function for that coordinate. After performing the line search for all unfixed coordinates, an RCL is created, which includes coordinates associated with the best objective function values. The RCL is determined by selecting those coordinates whose objective function values fall within a specific range, controlled by the parameter α. This parameter balances the level of greediness in the selection process, with higher values of α focusing more on the best options. Once the RCL is formed, a coordinate is randomly selected from it, ensuring some level of diversity in the generated solution. The selected coordinate is then fixed at the value determined from the line search. This process repeats for each coordinate until all dimensions of the solution are fixed, resulting in a complete solution.

The local search begins with the solution obtained from the construction phase. For a given solution, the algorithm explores possible moves by considering different directions from the current point in the solution space. These directions are formed by vectors with components set to values from the set $1, 0, -1$, indicating movement in the positive, zero, or negative direction along each dimension of the solution. To maintain tractability, a limit is set on the number of directions to explore. The number of directions is determined by a parameter, `MaxDirToTry`, which can either be fixed or based on the dimensionality of the problem. For each direction, the algorithm tests a new point by adjusting the current solution along that direction. If the new point improves the objective function (i.e., it yields a better solution), the search continues from this new point, and the process repeats with the updated solution. The local search continues iterating, modifying the solution along different directions, until no further improvements can be found in the selected directions. If a better solution is found, it becomes the new current best solution. If no improvements are possible after testing all allowed directions, the local search terminates, and the resulting solution is returned. This phase serves to fine-tune the

solution, progressively exploring the local neighborhood and refining the solution quality. The randomness in the direction selection helps avoid getting stuck in local minima by diversifying the search, while the use of multiple direction vectors increases the likelihood of discovering a better solution.

Hirsch et al. [90] proposed several techniques to speeding up C-GRASP, while de Araújo et al. [13] presented several direct search procedures that are used in a C-GRASP heuristic. Moreover, C-GRASP has been successfully applied to various problems, including solving nonlinear equations [89], matching projected 3D points and lines [91], economic dispatch of thermal units [117], and others [82, 88, 106].

An interesting variant of C-GRASP is the random-key GRASP (RK-GRASP) presented by Chaves et al. [40]. It integrates the random-key optimizer (RKO) paradigm to solve combinatorial optimization problems. In RK-GRASP, solutions are represented as vectors of random keys, which are real numbers in the interval [0, 1). The random-key vector encodes the solution, and a decoder is used to map this vector to a feasible solution in the problem space. The RK-GRASP procedure follows the typical two-phase structure of GRASP: the construction phase and the local search phase. In the construction phase, a semi-greedy approach is used to generate a solution. This phase starts by selecting a random-key vector, and then each key is perturbed using a line search technique, ensuring that the solutions evolve based on both greedy choices and some randomness. The algorithm builds the solution by iteratively selecting keys and updating the solution. The local search phase then refines the constructed solution by exploring its neighborhood. Three distinct local search strategies have been used: grid search, Nelder-Mead search, and random variable neighborhood descent (RVND). These strategies explore different neighborhoods of the solution space to improve the solution's quality. RK-GRASP is particularly attractive because it decouples the problem-specific components (the decoder) from the problem-independent solver, allowing for the application of the same algorithm to various combinatorial optimization problems by simply changing the decoder. This makes RK-GRASP a flexible and efficient approach for solving a wide range of optimization problems.

Conclusions

The goal of this chapter was to provide an overview of GRASP describing its basic components and enhancements to the basic procedure, including reactive GRASP, intensification strategies, and its hybridizations with different metaheuristics.

Cross-References

▸ Genetic Algorithms
▸ Multi-start Methods
▸ Variable Neighborhood Descent
▸ Variable Neighborhood Search

Competing Interest Declaration The author(s) has no competing interests to declare that are relevant to the content of this manuscript.

References

1. Abdinnour-Helm S, Hadley S (2000) Tabu search based heuristics for multi-floor facility layout. Int J Prod Res 38:365–383
2. Abello J, Pardalos P, Resende M (1999) On maximum clique problems in very large graphs. In: Abello J, Vitter J (eds) External memory algorithms and visualization, DIMACS series on discrete mathematics and theoretical computer science, vol 50. American Mathematical Society, Providence, pp 199–130
3. Ahuja R, Orlin J, Tiwari A (2000) A greedy genetic algorithm for the quadratic assignment problem. Comput Oper Res 27:917–934
4. Aiex R, Binato S, Resende M (2003) Parallel GRASP with path-relinking for job shop scheduling. Parallel Comput 29:393–430
5. Aiex R, Resende M, Pardalos P, Toraldo G (2005) GRASP with path relinking for three-index assignment. INFORMS J Comput 17(2):224–247
6. Alvarez-Valdes R, Parreño F, Tamarit J (2005) A GRASP algorithm for constrained two-dimensional non-guillotine cutting problems. J Oper Res Soc 56(4):414–425
7. Alvarez-Valdes R, Parreño F, Tamarit J (2008) Reactive GRASP for the strip-packing problem. Comput Oper Res 35(4):1065–1083
8. Alvarez-Valdes R, Parreño F, Tamarit J (2008) Reactive grasp for the strip-packing problem. Comput Oper Res 35(4):1065–1083. https://doi.org/10.1016/j.cor.2006.07.004
9. Amaldi E, Capone A, Malucelli F (2003) Planning UMTS base station location: optimization models with power control and algorithms. IEEE Trans Wirel Commun 2(5):939–952
10. Andrade D, Resende M (2006) A GRASP for PBX telephone migration scheduling. In: Eighth INFORMS telecommunication conference
11. Andrade D, Resende M (2007) GRASP with path-relinking for network migration scheduling. In: Proceedings of the international network optimization conference (INOC 2007)
12. Andres C, Miralles C, Pastor R (2008) Balancing and scheduling tasks in assembly lines with sequence-dependent setup times. Eur J Oper Res 187(3):1212–1223
13. de Araújo TMU, Andrade LMMS, Magno C, dos Anjos Formiga Cabral L, do Nascimento RQ, Meneses CN (2015) DC-GRASP: directing the search on continuous-GRASP. J Heurist 22(4):365–382. https://doi.org/10.1007/s10732-014-9278-6
14. Areibi S (1999) GRASP: An effective constructive technique for VLSI circuit partitioning. In: Proceedings of the IEEE canadian conference on electrical & computer engineering (CCECE'99)
15. Areibi S, Vannelli A (1997) A GRASP clustering technique for circuit partitioning. In: Gu J, Pardalos P (eds) Satisfiability problems, DIMACS series on discrete mathematics and theoretical computer science, vol 35, American Mathematical Society, Providence, pp 711–724
16. Argüello M, Feo T, Goldschmidt O (1996) Randomized methods for the number partitioning problem. Comput Oper Res 23(2):103–111
17. Argüello M, Bard J, Yu G (1997) A GRASP for aircraft routing in response to groundings and delays. J Comb Optim 1:211–228
18. Armony M, Klincewicz J, Luss H, Rosenwein M (2000) Design of stacked self-healing rings using a genetic algorithm. J Heurist 6:85–105
19. Arroyo J, Vieira P, Vianna D (2008) A GRASP algorithm for the multi-criteria minimum spanning tree problem. Ann Oper Res 159:125–133
20. Atkinson J (1998) A greedy randomised search heuristic for time-constrained vehicle scheduling and the incorporation of a learning strategy. J Oper Res Soc 49:700–708
21. Bard J (1997) An analysis of a rail car unloading area for a consumer products manufacturer. J Oper Res Soc 48:873–883

22. Bard J, Feo T (1989) Operations sequencing in discrete parts manufacturing. Manag Sci 35:249–255
23. Bard J, Feo T (1991) An algorithm for the manufacturing equipment selection problem. IIE Trans 23:83–92
24. Bard J, Huang L, Jaillet P, Dror M (1998) A decomposition approach to the inventory routing problem with satellite facilities. Transport Sci 32:189–203
25. Baykasoğlu A, Madenoğlu FS (2021) Greedy randomized adaptive search procedure for simultaneous scheduling of production and preventive maintenance activities in dynamic flexible job shops. Soft Comput 25(23):14893–14932. https://doi.org/10.1007/s00500-021-06053-0
26. Belkharroubi L, Yahyaoui K (2022) Solving the mixed-model assembly line balancing problem type-i using a hybrid reactive grasp. Prod Manufact Res 10(1):108–131. https://doi.org/10.1080/21693277.2022.2065380
27. Binato S, Oliveira G (2002) A Reactive GRASP for transmission network expansion planning. In: Ribeiro C, Hansen P (eds) Essays and surveys on metaheuristics. Kluwer Academic Publishers, Dordrecht, pp 81–100
28. Binato S, Oliveira G, Araújo J (2001) A greedy randomized adaptive search procedure for transmission expansion planning. IEEE Trans Power Syst 16:247–253
29. Binato S, Hery W, Loewenstern D, Resende M (2002) A greedy randomized adaptive search procedure for job shop scheduling. In: Ribeiro C, Hansen P (eds) Essays and surveys on metaheuristics. Kluwer Academic Publishers, Dordrecht, pp 58–79
30. Boudia M, Louly M, Prins C (2007) A reactive GRASP and path relinking for a combined production-distribution problem. Comput Oper Res 34:3402–3419
31. Bresina J (1996) Heuristic-biased stochastic sampling. In: Proceedings of the thirteenth national conference on artificial intelligence (AAAI-96), pp 271–278
32. Bruglieri M, Ferone D, Festa P, Pisacane O (2022) A GRASP with penalty objective function for the green vehicle routing problem with private capacitated stations. Comput Oper Res 143:105770. https://doi.org/10.1016/j.cor.2022.105770
33. Canuto S, Resende M, Ribeiro C (2001) Local search with perturbations for the prize-collecting Steiner tree problem in graphs. Networks 38:50–58
34. Canuto S, Resende M, Ribeiro C (2001) Local search with perturbations for the prize-collecting Steiner tree problem in graphs. Networks 38:50–58
35. Carrabs F, D'Ambrosio C, Ferone D, Festa P, Laureana F (2020) The constrained forward shortest path tour problem: mathematical modeling and GRASP approximate solutions. Networks 78(1):17–31. https://doi.org/10.1002/net.22010
36. Carreto C, Baker B (2002) A GRASP interactive approach to the vehicle routing problem with backhauls. In: Ribeiro C, Hansen P (eds) Essays and surveys on metaheuristics. Kluwer Academic Publishers, Dordrecht, pp 185–200
37. Chaovalitwongse WA, Oliveira CAS, Chiarini B, Pardalos PM, Resende MGC (2010) Revised GRASP with path-relinking for the linear ordering problem. J Comb Optim 22(4):572–593. https://doi.org/10.1007/s10878-010-9306-x
38. Charon I, Hudry O (1993) The noising method: a new method for combinatorial optimization. Oper Res Lett 14:133–137
39. Charon I, Hudry O (2002) The noising methods: a survey. In: Ribeiro C, Hansen P (eds) Essays and surveys on metaheuristics. Kluwer Academic Publishers, Dordrecht, pp 245–261
40. Chaves AA, Resende MGC, Silva RMA (2024) A random-key GRASP for combinatorial optimization. https://doi.org/10.48550/ARXIV.2405.18681
41. Commander C, Festa P, Oliveira C, Pardalos P, Resende M, Tsitselis M (2006) A greedy randomized algorithm for the cooperative communication problem on ad hoc networks. In: Eighth INFORMS telecommunications conference
42. Contreras I, Díaz J (2008) Scatter search for the single source capacitated facility location problem. Ann Oper Res 157:73–89
43. Cravo G, Ribeiro G, Lorena LN (2008) A greedy randomized adaptive search procedure for the point-feature cartographic label placement. Comput Geosci 34(4):373–386

44. Delmaire H, Díaz J, Fernández E, Ortega M (1999) Reactive GRASP and tabu search based heuristics for the single source capacitated plant location problem. INFOR 37:194–225
45. Deshpande A, Triantaphyllou E (1998) A greedy randomized adaptive search procedure (GRASP) for inferring logical clauses from examples in polynomial time and some extensions. Math Comput Modell 27:75–99
46. Di Puglia Pugliese L, Ferone D, Festa P, Guerriero F, Macrina G (2022) Solution approaches for the vehicle routing problem with occasional drivers and time windows. Optim Methods Softw 37(4):1384–1414. https://doi.org/10.1080/10556788.2021.2022142
47. Dong Y, Goldberg AV, Noe A, Parotsidis N, Resende MGC, Spaen Q (2024) A metaheuristic algorithm for large maximum weight independent set problems. Networks 85(1):91–112. https://doi.org/10.1002/net.22247
48. Duarte A, Sánchez-Oro J, Resende MGC, Glover F, Martí R (2015) Greedy randomized adaptive search procedure with exterior path relinking for differential dispersion minimization. Information Sciences 296:46–60. https://doi.org/10.1016/j.ins.2014.10.010
49. Faria H, Binato S, Resende M, Falcão D (2005) Power transmission network design by a greedy randomized adaptive path relinking approach. IEEE Trans Power Syst 20(1):43–49
50. Felici G, Ferone D, Festa P, Napoletano A, Pastore T (2017) A GRASP for the minimum cost SAT problem, Springer International Publishing, Cham, pp 64–78. https://doi.org/10.1007/978-3-319-69404-7_5
51. Feo T, González-Velarde J (1995) The intermodal trailer assignment problem: models, algorithms, and heuristics. Transport Sci 29:330–341
52. Feo T, Resende M (1989) A probabilistic heuristic for a computationally difficult set covering problem. Oper Res Lett 8:67–71
53. Feo T, Resende M (1995) Greedy randomized adaptive search procedures. J Global Optim 6:109–133
54. Feo T, Venkatraman K, Bard J (1991) A GRASP for a difficult single machine scheduling problem. Comput Oper Res 18:635–643
55. Feo T, Resende M, Smith S (1994) A greedy randomized adaptive search procedure for maximum independent set. Oper Res 42:860–878
56. Feo T, Sarathy K, McGahan J (1996) A GRASP for single machine scheduling with sequence dependent setup costs and linear delay penalties. Comput Oper Res 23:881–895
57. Ferone D, Festa P, Resende M (2013) Hybrid metaheuristics for the far from most string problem. In: Proceedings of 8th international workshop on hybrid metaheuristics. Lecture Notes in Computer Science, vol 7919, pp 174–188
58. Ferone D, Festa P, Resende MG (2015) Hybridizations of GRASP with path relinking for the far from most string problem. Int Trans Oper Res 23(3):481–506. https://doi.org/10.1111/itor.12167
59. Ferone D, Festa P, Guerriero F, Laganà D (2016) The constrained shortest path tour problem. Comput Oper Res 74:64–77. https://doi.org/10.1016/j.cor.2016.04.002
60. Ferone D, Festa P, Napoletano A, Resende MGC (2017) A new local search for the p-center problem based on the critical vertex concept. Springer International Publishing, Cham, pp 79–92. https://doi.org/10.1007/978-3-319-69404-7_6
61. Ferone D, Gruler A, Festa P, Juan AA (2018) Enhancing and extending the classical grasp framework with biased randomisation and simulation. J Oper Res Soc 70(8):1362–1375. https://doi.org/10.1080/01605682.2018.1494527
62. Ferone D, Festa P, Guerriero F (2022) The rainbow Steiner tree problem. Comput Oper Res 139:105621. https://doi.org/10.1016/j.cor.2021.105621
63. Ferone D, Festa P, Pastore T, Resende MGC (2023) Efficient GRASP solution approach for the prisoner transportation problem. Comput Oper Res 153:106161. https://doi.org/10.1016/j.cor.2023.106161
64. Ferone D, Festa P, Pastore T (2025) GRASP: an application to efficiently plan the low carbon emission distributed additive manufacturing. TOP. https://doi.org/10.1007/s11750-025-00696-0

65. Festa P (2007) On some optimization problems in molecular biology. Math Biosci 207(2):219–234
66. Festa P, Resende MGC (2009) An annotated bibliography of grasp – part I: algorithms. Int Trans Oper Res 16(1):1–24
67. Festa P, Resende MGC (2009) An annotated bibliography of grasp – part II: applications. Int Trans Oper Res 16(2):131–172
68. Festa P, Pardalos P, Resende M (2001) Algorithm 815: FORTRAN subroutines for computing approximate solution to feedback set problems using GRASP. ACM Trans Math Softw 27:456–464
69. Festa P, Pardalos P, Resende M, Ribeiro C (2002) Randomized heuristics for the MAX-CUT problem. Optim Methods Softw 7:1033–1058
70. Festa P, Pardalos P, Pitsoulis L, Resende M (2006) GRASP with path-relinking for the weighted MAXSAT problem. ACM J Exper Algor 11:1–16
71. Fleurent C, Glover F (1999) Improved constructive multistart strategies for the quadratic assignment problem using adaptive memory. INFORMS J Comput 11:198–204
72. Garey M, Johnson D (1979) Computers and intractability: a guide to the theory of NP-completeness. W.H. Freeman and Company, New York
73. Gendreau M, Potvin JY (2005) Tabu search. Springer US, Berlin, pp 165–186. https://doi.org/10.1007/0-387-28356-0_6
74. Ghosh J (1996) Computational aspects of the maximum diversity problem. Oper Res Lett 19:175–181
75. Glover F (1996) Tabu search and adaptive memory programing – Advances, applications and challenges. In: Barr R, Helgason R, Kennington J (eds) Interfaces in computer science and operations research. Kluwer, Dordrecht, pp 1–75
76. Glover F (2000) Multi-start and strategic oscillation methods – Principles to exploit adaptive memory. In: Laguna M, Gonzáles-Velarde J (eds) Computing tools for modeling, optimization and simulation: interfaces in computer science and operations research. Kluwer, Dordrecht, pp 1–24
77. Glover F, Laguna M (1997) Tabu search. Kluwer Academic Publishers, Dordrecht
78. Glover F, Laguna M, Martí R (2000) Fundamentals of scatter search and path relinking. Control Cyber 39:653–684
79. Goëffon A, Richer JM, Hao JK (2008) Progressive tree neighborhood applied to the maximum parsimony problem. IEEE/ACM Trans Comput Biol Bioinform 5(1):136–145
80. Goemans M, Williamson D (1996) The primal dual method for approximation algorithms and its application to network design problems. In: Hochbaum D (ed) Approximation algorithms for NP-hard problems. PWS Publishing, Worcester, pp 144–191
81. Gonzalez-Neira EM, Ferone D, Hatami S, Juan AA (2017) A biased-randomized simheuristic for the distributed assembly permutation flowshop problem with stochastic processing times. Simul Modell Pract Theory 79:23–36. https://doi.org/10.1016/j.simpat.2017.09.001
82. Guedes ADL, Neto FDM, Platt GM (2011) Double azeotropy: calculations with Newton-like methods and continuous GRASP (C-GRASP). Int J Math Modell Numer Optim 2(4):387. https://doi.org/10.1504/ijmmno.2011.043091
83. Hammer P, Rader, Jr D (2001) Maximally disjoint solutions of the set covering problem. J Heurist 7:131–144
84. Hansen P, Mladenović N (2002) Developments of variable neighborhood search. In: Ribeiro C, Hansen P (eds) Essays and surveys in metaheuristics. Kluwer Academic Publishers, Dordrecht, pp 415–439
85. Hansen P, Mladenović N, Brimberg J, Pérez JAM (2018) Variable neighborhood search. Springer International Publishing, Cham, pp 57–97. https://doi.org/10.1007/978-3-319-91086-4_3
86. Hirsch MJ, Meneses CN, Pardalos PM, Resende MGC (2006) Global optimization by continuous grasp. Optim Lett 1(2):201–212. https://doi.org/10.1007/s11590-006-0021-6

87. Hirsch M, Meneses C, Pardalos P, Ragle M, Resende M (2007) A continuous GRASP to determine the relationship between drugs and adverse reactions. In: Seref O, Kundakcioglu O, Pardalos P (eds) Data mining, systems analysis, and optimization in biomedicine, AIP conference proceedings, vol 953. Springer, Berlin, pp 106–121
88. Hirsch MJ, Meneses CN, Pardalos PM, Ragle M, Resende MGC, Seref O, Kundakcioglu OE, Pardalos P (2007) A continuous GRASP to determine the relationship between drugs and adverse reactions. In: AIP conference proceedings, AIP, vol 953, pp 106–121. https://doi.org/10.1063/1.2817334
89. Hirsch MJ, Pardalos PM, Resende MGC (2009) Solving systems of nonlinear equations with continuous GRASP. Nonlinear Analy Real World Appl 10(4):2000–2006. https://doi.org/10.1016/j.nonrwa.2008.03.006
90. Hirsch MJ, Pardalos PM, Resende MGC (2010) Speeding up continuous GRASP. Eur J Oper Res 205(3):507–521. https://doi.org/10.1016/j.ejor.2010.02.009
91. Hirsch MJ, Pardalos PM, Resende MGC (2011) Correspondence of projected 3-D points and lines using a continuous GRASP. Int Trans Oper Res 18(4):493–511. https://doi.org/10.1111/j.1475-3995.2011.00810.x
92. Kernighan B, Lin S (1970) An efficient heuristic procedure for partitioning problems. Bell Syst Tech J 49(2):291–307
93. Klincewicz J (1992) Avoiding local optima in the p-hub location problem using tabu search and GRASP. Ann Oper Res 40:283–302
94. Klincewicz J, Rajan A (1994) Using GRASP to solve the component grouping problem. Naval Res Logist 41:893–912
95. Kontoravdis G, Bard J (1995) A GRASP for the vehicle routing problem with time windows. ORSA J Comput 7:10–23
96. Laguna M, González-Velarde J (1991) A search heuristic for just-in-time scheduling in parallel machines. J Intell Manufact 2:253–260
97. Laguna M, Martí R (1999) GRASP and path relinking for 2-layer straight line crossing minimization. INFORMS J Comput 11:44–52
98. Laguna M, Martí R (2001) A GRASP for coloring sparse graphs. Comput Optim Appl 19:165–178
99. Laguna M, Feo T, Elrod H (1994) A greedy randomized adaptive search procedure for the two-partition problem. Oper Res 42:677–687
100. Laguna M, Martí R, Martínez-Gavara A, Pérez-Peló S, Resende MGC (2025) Greedy randomized adaptive search procedures with path relinking. an analytical review of designs and implementations. Eur J Oper Res. https://doi.org/10.1016/j.ejor.2025.02.022
101. Lambora A, Gupta K, Chopra K (2019) Genetic algorithm- a literature review. In: 2019 international conference on machine learning, big data, cloud and parallel computing (COMITCon), IEEE, pp 380–384. https://doi.org/10.1109/comitcon.2019.8862255
102. Li Y, Pardalos P, Resende M (1994) A greedy randomized adaptive search procedure for the quadratic assignment problem. In: Pardalos P, Wolkowicz H (eds) Quadratic assignment and related problems, DIMACS series on discrete mathematics and theoretical computer science, vol 16. American Mathematical Society, Providence, pp 237–261
103. Liu X, Pardalos P, Rajasekaran S, Resende M (2000) A GRASP for frequency assignment in mobile radio networks. In: Rajasekaran S, Pardalos P, Hsu F (eds) Mobile networks and computing, DIMACS series on discrete mathematics and theoretical computer science, vol 52. American Mathematical Society, Providence, pp 195–201
104. Londe MA, Pessoa LS, Andrade CE, Resende MG (2025) Biased random-key genetic algorithms: a review. Eur J Oper Res 321(1):1–22. https://doi.org/10.1016/j.ejor.2024.03.030
105. Lourenço HR, Paixão J, Portugal R (2001) Multiobjective metaheuristics for the bus-driver scheduling problem. Transport Sci 35:331–343
106. Macharet DG, Neto AA, da Camara Neto VF, Campos MFM (2011) Nonholonomic path planning optimization for Dubins' vehicles. In: 2011 IEEE international conference on robotics and automation, IEEE, pp 4208–4213. https://doi.org/10.1109/icra.2011.5980239

107. Martí R, Laguna M (2003) Heuristics and meta-heuristics for 2-layer straight line crossing minimization. Discrete Appl Math 127(3):665–678
108. Martins S, Ribeiro C, Souza M (1998) A parallel GRASP for the Steiner problem in graphs. In: Ferreira A, Rolim J (eds) Proceedings of IRREGULAR'98 – 5th international symposium on solving irregularly structured problems in parallel. Lecture notes in computer science, vol 1457. Springer, Berlin, pp 285–297
109. Martins S, Pardalos P, Resende M, Ribeiro C (1999) Greedy randomized adaptive search procedures for the Steiner problem in graphs. In: Pardalos P, Rajasekaran S, Rolim J (eds) Randomization methods in algorithmic design, DIMACS Series on discrete mathematics and theoretical computer science, vol 43. American Mathematical Society, Providence, pp 133–145
110. Martins S, Resende M, Ribeiro C, Pardalos P (2000) A parallel GRASP for the Steiner tree problem in graphs using a hybrid local search strategy. J Global Optim 17:267–283
111. Mateus GR, Resende MGC, Silva RMA (2010) Grasp with path-relinking for the generalized quadratic assignment problem. J Heurist 17(5):527–565. https://doi.org/10.1007/s10732-010-9144-0
112. Mavridou T, Pardalos P, Pitsoulis L, Resende M (1998) A GRASP for the biquadratic assignment problem. Eur J Oper Res 105:613–621
113. Mladenović N, Hansen P (1997) Variable neighborhood search. Comput Oper Res 24:1097–1100
114. Mockus J, Eddy E, Mockus A, Mockus L, Reklaitis G (1997) Bayesian discrete and global optimization. Kluwer Academic Publishers, Dordrecht
115. Monkman S, Morrice D, Bard J (2008) A production scheduling heuristic for an electronics manufacturer with sequence-dependent setup costs. Eur J Oper Res 187(3):1100–1114
116. Napoletano A, Martínez-Gavara A, Festa P, Pastore T, Martí R (2019) Heuristics for the constrained incremental graph drawing problem. Eur J Oper Res 274(2):710–729. https://doi.org/10.1016/j.ejor.2018.10.017
117. Neto JXV, Reynoso-Meza G, Ruppel TH, Mariani VC, Coelho LdS (2017) Solving non-smooth economic dispatch by a new combination of continuous GRASP algorithm and differential evolution. Int J Electr Power Energy Syst 84:13–24. https://doi.org/10.1016/j.ijepes.2016.04.012
118. Nie X, Zhang Q, Qiao Y, Qi Z, Zhang L, Niu D, Zhang H (2024) A greedy randomized adaptive search procedure (GRASP) for minimum 2-fold connected dominating set problem. Appl Soft Comput 165:112122. https://doi.org/10.1016/j.asoc.2024.112122
119. Nikolaev AG, Jacobson SH (2010) Simulated annealing. Springer US, Berlin, pp 1–39. https://doi.org/10.1007/978-1-4419-1665-5_1
120. Osman I, Al-Ayoubi B, Barake M (2003) A greedy random adaptive search procedure for the weighted maximal planar graph problem. Comput Ind Eng 45(4):635–651
121. Pardalos P, Pitsoulis L, Resende M (1996) A parallel GRASP for MAX-SAT problems. Lect Notes Comput Sci 1184:575–585
122. Pardalos P, Pitsoulis L, Resende M (1997) Algorithm 769: fortran subroutines for approximate solution of sparse quadratic assignment problems using GRASP. ACM Trans Math Softw 23:196–208
123. Pardalos P, Ramakrishnan K, Resende M, Li Y (1997) Implementation of a variance reduction based lower bound in a branch and bound algorithm for the quadratic assignment problem. SIAM J Optim 7:280–294
124. Parejo JA, Segura S, Fernandez P, Ruiz-Cortés A (2014) QoS-aware web services composition using grasp with path relinking. Expert Syst Appl 41(9):4211–4223. https://doi.org/10.1016/j.eswa.2013.12.036
125. Pinãna E, Plana I, Campos V, R Martì (2004) GRASP and path relinking for the matrix bandwidth minimization. Eur J Oper Res 153(1):200–210
126. Prais M, Ribeiro C (1999) Parameter variation in GRASP implementations. In: Extended abstracts of the third metaheuristics international conference, pp 375–380

127. Prais M, Ribeiro C (2000) Parameter variation in GRASP procedures. nvestigación Operativa 9:1–20
128. Prais M, Ribeiro C (2000) Reactive GRASP: an application to a matrix decomposition problem in TDMA traffic assignment. INFORMS J Comput 12:164–176
129. Pérez-Peló S, Sánchez-Oro J, Duarte A (2020) Finding weaknesses in networks using greedy randomized adaptive search procedure and path relinking. Expert Syst 37(6). https://doi.org/10.1111/exsy.12540
130. Pu G, Chong Z, Qiu Z, Lin Z, He J (2006) A hybrid heuristic algorithm for HW-SW partitioning within timed automata. In: Proceedings of knowledge-based intelligent information and engineering systems. Lecture notes in artificial intelligence, vol 4251. Springer, Berlin, pp 459–466
131. Póvoa CLR, Roboredo MC, Velasco AS, Pessoa AA, Paes FG (2025) A hybrid grasp and tabu-search heuristic and an exact method for a variant of the multi-compartment vehicle routing problem. Expert Syst Appl 259:125319. https://doi.org/10.1016/j.eswa.2024.125319
132. Resende M, Feo T (1996) A GRASP for satisfiability. In: Johnson D, Trick M (eds) Cliques, coloring, and satisfiability: the second DIMACS implementation challenge, DIMACS series on discrete mathematics and theoretical computer science, vol 26. American Mathematical Society, Providence, pp 499–520
133. Resende M, Ribeiro C (1997) A GRASP for graph planarization. Networks 29:173–189
134. Resende MG, Ribeiro CC (2016) Optimization by GRASP: greedy randomized adaptive search procedures. Springer, New York. https://doi.org/10.1007/978-1-4939-6530-4
135. Resende M, Pardalos P, Li Y (1996) Algorithm 754: Fortran subroutines for approximate solution of dense quadratic assignment problems using GRASP. ACM Trans Math Softw 22:104–118
136. Resende M, Pitsoulis L, Pardalos P (1997) Approximate solution of weighted MAX-SAT problems using GRASP. In: Gu J, Pardalos P (eds) Satisfiability problems, DIMACS series on discrete mathematics and theoretical computer science, vol 35. American Mathematical Society, Providence, pp 393–405
137. Reyes-Rubiano L, Ferone D, Juan AA, Faulin J (2019) A simheuristic for routing electric vehicles with limited driving ranges and stochastic travel times. SORT Stat Oper Res Trans 43:3–24. https://doi.org/10.2436/20.8080.02.77
138. Ribeiro C, Souza M (2002) Variable neighborhood search for the degree constrained minimum spanning tree problem. Discrete Appl Math 118:43–54
139. Ribeiro C, Urrutia S (2007) Heuristics for the mirrored traveling tournament problem. Eur J Oper Res 179:775–787
140. Ribeiro C, Uchoa E, Werneck R (2002) A hybrid GRASP with perturbations for the Steiner problem in graphs. INFORMS J Comput 14:228–246
141. Ribeiro C, Uchoa E, Werneck R (2002) A hybrid GRASP with perturbations for the Steiner problem in graphs. INFORMS J Comput 14:228–246
142. Ríos-Mercado R, Bard J (1998) Heuristics for the flow line problem with setup costs. Eur J Oper Res 110:76–98
143. Ríos-Mercado R, Bard J (1999) An enhanced TSP-based heuristic for makespan minimization in a flow shop with setup costs. J Heurist 5:57–74
144. Rivera L (1998) Evaluation of parallel implementations of heuristics for the course scheduling problem. Master's thesis, Instituto Tecnologico y de Estudios Superiores de Monterrey, Monterrey
145. Robertson A (2001) A set of greedy randomized adaptive local search procedure (GRASP) implementations for the multidimensional assignment problem. Comput Optim Appl 19:145–164
146. Ríos-Mercado RZ, Fernández E (2009) A reactive grasp for a commercial territory design problem with multiple balancing requirements. Comput Oper Res 36(3):755–776. https://doi.org/10.1016/j.cor.2007.10.024

147. Sosnowska D (2000) Optimization of a simplified fleet assignment problem with metaheuristics: Simulated annealing and GRASP. In: Pardalos P (ed) Approximation and complexity in numerical optimization. Kluwer Academic Publishers, Dordrecht
148. Souza Almeida L, Goerlandt F, Pelot R, Sörensen K (2022) A Greedy Randomized Adaptive Search Procedure (GRASP) for the multi-vehicle prize collecting arc routing for connectivity problem. Comput Oper Res 143:105804. https://doi.org/10.1016/j.cor.2022.105804
149. Srinivasan A, Ramakrishnan K, Kumaram K, Aravamudam M, Naqvi S (2000) Optimal design of signaling networks for internet telephony. In: IEEE INFOCOM 2000, vol 2, pp 707–716
150. Urban T (1998) Solution procedures for the dynamic facility layout problem. Ann Oper Res 76:323–342
151. Windras Mara ST, Norcahyo R, Jodiawan P, Lusiantoro L, Rifai AP (2022) A survey of adaptive large neighborhood search algorithms and applications. Comput Oper Res 146:105903. https://doi.org/10.1016/j.cor.2022.105903

Hyper-heuristics

24

Michael G. Epitropakis and Edmund K. Burke

Contents

Introduction . . . 688
Recent Advances in the Field of Hyper-heuristics . . . 691
Recent Advances on the Methodology of Hyper-heuristics . . . 692
Automatic Design of Algorithms . . . 695
Recent Theoretical Results in Hyper-heuristics . . . 696
Multi-objective Optimization . . . 697
Hyper-heuristics in Dynamic and Uncertain Environments . . . 698
Machine Learning Methodologies . . . 699
Automated Parameter Control and Tuning . . . 700
Hyper-heuristics for Scheduling Problems . . . 701
Recent Educational Timetabling Applications of Hyper-heuristics . . . 704
Games and Education . . . 705
Other Recent Developments . . . 706
Case Study: A Selection Hyper-heuristic-Based Iterated Local Search . . . 707
The HyFlex Framework . . . 707
Iterated Local Search with Adaptive Heuristic Selection . . . 708
Action Selection Models . . . 713
Computational Results . . . 716
Experimental Protocol . . . 716
Experimental Results . . . 718
Comparison to HyFlex State-of-the-Art Hyper-heuristics . . . 730
Conclusions . . . 732
Cross-References . . . 734
References . . . 735

M. G. Epitropakis (✉)
Department of Management Science, Data Science Institute, Lancaster University Management School, Lancaster University, Lancaster, UK
e-mail: m.epitropakis@lancaster.ac.uk

E. K. Burke
School of Electronic Engineering and Computer Science, Queen Mary University of London, London, UK
e-mail: vp-se@qmul.ac.uk

© Springer Nature Switzerland AG 2025

R. Martí et al. (eds.), *Handbook of Heuristics*,
https://doi.org/10.1007/978-3-032-00385-0_32

Abstract

This chapter presents a literature review of the main advances in the field of hyper-heuristics, since the publication of a survey paper in 2013. The chapter demonstrates the most recent advances in hyper-heuristic foundations, methodologies, theory, and application areas. In addition, a simple illustrative selection hyper-heuristic framework is developed as a case study. This is based on the well-known Iterated Local Search algorithm and is presented to provide a tutorial style introduction to some of the key basic issues. A brief discussion about the implementation process in addition to the decisions that had to be made during the implementation is presented. The framework implements an action selection model that operates on the perturbation stage of the Iterated Local Search algorithm to adaptively select among various low-level perturbation heuristics. The performance and efficiency of the developed framework is evaluated across six well-known real-world problem domains.

Keywords

Hyper-heuristics · Heuristics · Meta-heuristics · Evolutionary computation · Optimization · Search · Machine learning · Multi-objective optimization · Combinatorial optimization · Black box optimization · Dynamic optimization · Scheduling · Timetabling · Packing · Iterated local search

Introduction

Over the last 50 years or so, heuristic search methodologies have been successfully applied to address various real-world complex optimization problems. Computationally challenging optimization problems arise in various disciplines such as operational research, computer science, finance, bioinformatics, industrial informatics, engineering, and data sciences. Representative examples of optimization problems can be found in scheduling, timetabling, planning, resource and space allocation, cutting and packing, software design, hardware design, and engineering design problems. Such optimization problems can be addressed by exact or heuristic methodologies (or a hybridization). The main difficulty of solving such problems occurs from the often remarkably large and possibly heavily constrained search space. In such problems, or in noisy and dynamic real-world problem scenarios, finding optimal solutions by exact methods might be impossible. Therefore, in practice, heuristic search methodologies are often developed or applied that seek to find solutions of acceptable quality in reasonable time, but without having any guarantee of optimality.

Over the last few decades, a broad spectrum of different heuristics have been proposed and successfully applied to a wide variety of problems. However, heuristics are usually designed to solve specific optimization problems. Such heuristics are bespoke problem-specific methods that often demand significant time

to design and tune. They are also often expensive to implement and maintain. Hyper-heuristics have been motivated by the aim of raising the level of generality at which search algorithms can operate, as well as automating the design and tuning of heuristic algorithms [1–8]. The main characteristic of a hyper-heuristic method is that it operates on a space of heuristics rather than on a space of solutions. As such, a hyper-heuristic methodology attempts to find or generate the appropriate method or sequence of low-level heuristics instead of directly solving a particular problem. A hyper-heuristic can be characterized as a high-level methodology which automatically generates or combines low-level search components (heuristics) to effectively solve a given problem instance or class of problems.

The following definition has been provided in the literature:

> A hyper-heuristic is a search method or learning mechanism for selecting or generating heuristics to solve computational search problems [3, 4].

The use of the term of *hyper-heuristics* goes back to 2001 [9]. However, the idea of automating the design and/or selection of heuristics has existed in the literature for more than 50 years [10, 11]. It is possible to characterize hyper-heuristic methodologies by considering two dimensions [3]: the characteristics of the heuristics' search space and the feedback information from the search process. The heuristic search space can be further divided into heuristic selection and heuristic generation. The first category represents methodologies or models that select among a set of available heuristics, while the latter category covers methodologies that generate new heuristics from existing heuristic components. A further refinement in both categories can be made based on the characteristics of the utilized search heuristic, i.e., constructive and perturbative search paradigms [12].

Hyper-heuristics can be further distinguished by looking at how they handle feedback information from the search process. Two main categories can be currently observed: methodologies that learn from the feedback and the ones that do not learn from it. The first category can be further refined based on the source of learning: *online* or *off-line* learning. When considering online learning hyper-heuristics, the learning phase takes place during the search procedure. Thus, timely feedback information can be exploited by the learning mechanism to study the trends and behavior of the underlying components. In the case of off-line learning (off-line learning hyper-heuristics), all information about the process is known a priori. As such, the learning mechanism is able to gather knowledge and learn from a training set of problem instances and then apply its knowledge on a test set of unseen instances. The aim of such a procedure is to generalize from the training information to unseen problem instances in an expected way. Methodologies that do not learn from the feedback information usually simply discard it.

The aforementioned classification reflects the majority of the past and the current research trends that can be widely found in the literature. Nevertheless, interesting example methodologies can be found that represent an interplay between the categories of heuristics, i.e., combinations of selection and generation heuristic search procedures [13, 14].

Over the last few years, various introductory tutorials, review, and survey articles have been published on the area of hyper-heuristics. The first book chapter that appeared in 2003 [1] introduces the general idea of hyper-heuristics and provides a brief historical overview of the field at that time. In particular, the chapter focuses on one of the main objectives of hyper-heuristics, which is to increase the level of generality at which a search methodology can operate. The overview demonstrates the history as well as the developments and trends in the field at that time, through detailed descriptions of the representative applications and methodologies.

An introductory tutorial of the area was presented in [7] that effectively demonstrated the development process of a hyper-heuristic approach, through useful guidelines and characteristic examples. It additionally pinpoints potential application areas and discusses some issues and possible future research pointers in the area. A second-edition tutorial with enriched material and information was recently published in [8].

A classification of the field and a discussion of developments at that time were published in 2008 [5]. The chapter emphasizes the real-world applications that have been tackled by hyper-heuristic approaches and presents three criteria that characterize a hyper-heuristic methodology: operating at a higher level and managing low-level heuristics, searching the heuristic space rather than the solution space, and limited access to problem domain information, i.e., an increased level of generality. The authors identified the last criterion as a particularly important one for the general applicability and robustness of a hyper-heuristic approach.

An overview chapter was published in 2009 [2] that discusses hyper-heuristic methodologies that are able to generate new effective heuristics that are not known beforehand. The chapter addresses the concept of automatically designing heuristics and describes a methodology to generate new heuristics. Several representative case study examples are also provided. In addition, the presented methodology emphasizes the main algorithmic component of this class which is the genetic programming algorithm. Moreover, it briefly covers the related literature and discusses the characteristics and issues of this class of approaches.

A unified classification and definition of the field were presented in 2010 [3]. The chapter provides an overview of the previous categorizations of the area and determines a unified classification and definition of hyper-heuristics that captures the current and potential future directions of hyper-heuristics. Two main classes of hyper-heuristic approaches are recognized in the categorization, heuristic selection and generation approaches. Several characteristic examples are presented and discussed for both categories.

A recent survey of hyper-heuristics was published in 2013 [4]. The article presents the state of the art of the field up to 2012 by providing examples of the main and most representative methodologies and application areas. It also discusses the origins and intellectual roots of the field as well as provides a critical discussion of the literature. Various potential research trends are briefly discussed, and several future directions are highlighted. Finally, the most recent overview of hyper-heuristics was published in 2014 [6]. The article presents an overview

and critical analysis of hyper-heuristics specifically for educational timetabling problems.

Building upon the aforementioned overview articles that have been recently published, this chapter aims to provide a thorough literature review of the main very recent advances in the field of hyper-heuristics, since the publication of [4] in 2013. An additional objective is to provide a case study for the development process of a simple but highly efficient selection hyper-heuristic framework as a tutorial-style introduction to the field. The performance and efficiency of the developed framework is evaluated across six well-known real-world problem domains.

The remainder of the chapter is organized into three main parts. The first part will present a timely overview of the current developments in the field of hyper-heuristics (section "Recent Advances in the Field of Hyper-heuristics"). The second part presents the development of a simple selection hyper-heuristic framework as a case study (section "Case Study: A Selection Hyper-heuristic-Based Iterated Local Search"). The last part evaluates the generality and performance of the developed framework on six different problem domains (section "Computational Results"). The chapter concludes with a brief discussion that summarizes the presented work (section "Conclusions").

Recent Advances in the Field of Hyper-heuristics

This literature review covers and discusses the latest articles that have been published in the field. It aims to cover as many of the current advances and trends as possible, without presenting overlapping material with relatively recent overview and survey papers [1–8]. The main aim of this review is to cover the period since the publication of [4] in 2013.

The latest trends and directions regarding foundational and methodological aspects of hyper-heuristics are firstly presented in the literature review (section "Recent Advances on the Methodology of Hyper-heuristics"). Then it proceeds with discussion on the automatic design of algorithms (section "Automatic Design of Algorithms") and the latest theoretical works in the area (section "Recent Theoretical Results in Hyper-heuristics"). Hyper-heuristic developments in multi-objective and dynamic problem formulations are then reviewed (sections "Multi-objective Optimization" and "Hyper-heuristics in Dynamic and Uncertain Environments"). The intersection between machine learning and hyper-heuristic methodologies is studied in section "Machine Learning Methodologies", while section "Automated Parameter Control and Tuning" presents the recent advancements in hyper-heuristics for automatic parameter control. Finally, the most recent developments in application areas of hyper-heuristics are presented that include scheduling, timetabling, games, and various other areas (sections "Hyper-heuristics for Scheduling Problems", "Recent Educational Timetabling Applications of Hyper-heuristics", "Games and Education", and "Other Recent Developments").

Recent Advances on the Methodology of Hyper-heuristics

Starting from the foundations of hyper-heuristics and by studying the classifications of the area, the authors of [15] reformulate the mathematical definition of a hyper-heuristic. It is specifically exhibited that the new presented formulation naturally leads to a recursive definition of a hyper-heuristic. The authors draw parallels between the new hyper-heuristic formulation and a blackboard architecture from artificial intelligence, in which heuristics are able to annotate a shared workspace with information that can be exploited by other heuristics. In such a formulation, heuristics can share information at any level, which suggests that the domain barrier can be relaxed at any level required, without the loss of generality. A detailed description of the proposed formulation and its developments can be found in [15]. A concrete example application on the 3-SAT problem domain has been presented as a case study, which exhibits the improvements of the proposed idea over a wide set of problem instances. Other extensions of the existing formulations of hyper-heuristics have been proposed in [16]. This study advocates the need to reconsider the existing approaches to be able to produce self-organizing heuristics that will be able to automatically self-adapt to the environment when the underlying problem domain changes.

The need for more general and cross-domain efficient methodologies [4, 8, 17] has led to novel studies that try either to introduce unified representations or to embed common domain knowledge across various problem domains [18, 19].

In particular, the authors in [18] do not employ a high-level barrier across domains that are used in much of the hyper-heuristic literature. Instead, they propose an evolutionary memetic computing paradigm that is capable of embedding domain knowledge in memes across different but related problem domains. To improve search efficiency, the memetic paradigm is capable of learning and evolving knowledge memes on two different problem domains that can share a common representation, the capacitated vehicle and arc routing problems. Thorough analyses and experimental results show that the evolutionary search can benefit from different relevant problem domains. The knowledge that a meme has is crucial for the search procedure. It was observed that low discrepancy in the common feature space, between the problem domains, is able to enhance the evolutionary search procedure.

Furthermore, a general-purpose hyper-heuristic approach has been proposed that has the ability to solve combinatorial problems with minimal effort [19]. A unified representation and a set of domain-independent low-level heuristics are essentially proposed under a hyper-heuristic framework. To demonstrate the generality of the approach, the framework has been applied on different problem domains that share common characteristics but vastly differ as applications. An analysis reveals that the unified framework is very competitive in performance with state-of-the-art tailor-made heuristics for each problem domain.

The aforementioned studies suggest that both high and low levels of abstraction in the problem domain can enhance the search efficiency of hyper-heuristics. Such approaches demonstrate the real power of hyper-heuristic techniques. It is possible

to develop approaches that have enough generality and efficiency to enable them to produce competitive performance against tailor-made search algorithms for either a given problem class or across problem domains.

Various recent developments include novel hyper-heuristics that operate across the main categories of hyper-heuristics. Such an approach was introduced in [13]. It combines both heuristic selection and generation approaches. Specifically, it incorporates a dynamic multi-armed bandit with extreme credit values to online select the most appropriate low-level heuristic for the problem on hand. In parallel, instead of using a fixed acceptance criterion, it gradually generates new acceptance criteria for each problem instance through a gene expression programming framework. The effectiveness and generality of the approach are evaluated on exam timetabling instances from the 2007 International Timetabling Competition and on dynamic vehicle routing problem instances from the literature. Empirical results suggest that the introduced method shows competitive and general performance across both domains, compared with several state-of-the-art algorithms.

An automatic design framework was introduced in [20]. It automatically generates high-level heuristics for hyper-heuristic methodologies. The framework utilizes a gene expression programming algorithm to create high-level heuristics during the searching process. It captures information from the current state of the given problem and determines the low-level heuristic selection rule, as well as the solution acceptance criterion. As such, possibly new high-level rules can be produced for each problem instance. Experimental analyses on six problem domains exhibit its generality and competitive performance with state-of-the-art hyper-heuristic methodologies across all problem domains.

Techniques have been developed that are capable of learning and self-adapting their behavior based on the learned knowledge. More specifically, a hyper-heuristic system that embraces the concept of lifelong learning has been considered in [14, 21–23]. The system is based on a self-reliant network of interacting components inspired by artificial immune systems. It has the ability to continuously learn over time how to solve combinatorial optimization problems. It is capable of generating new heuristics continuously as well as sample problems from its environment. The core of the system is adaptable to its environment and has the intelligence to save and learn characteristic problem instances and heuristics from it. These characteristics result in an intelligent platform that exploits existing knowledge and quickly generates promising solutions of a given problem, as well as generalizing and adapting to new unseen problems with different properties. A rigorous experimental analysis on the bin packing domain demonstrates its excellent performance and generalization capabilities across a wide range of different problem instances. The dynamic and adaptive nature of the system makes it computationally efficient and scalable.

Various studies have been proposed that carefully investigate intrinsic characteristics of hyper-heuristics, such as diversity in the heuristic space, or understanding the behavior of the low-level heuristics. In particular, in [24], the impact and effect of a diverse set of available heuristic algorithms on the performance of a

selection hyper-heuristic is studied. The study firstly makes an attempt to define and measure the diversity of heuristic spaces and then proceeds to investigate the impact in performance of a hyper-heuristic that utilizes various strategies to manage the measured diversity. The diversity concept is studied with a hyper-heuristic that employs as low-level heuristic various black box optimization methods that operate on continuous spaces (see also [25]). The evaluation of the proposed method on a wide range of benchmark functions suggests that the heuristic space diversity significantly impacts upon the performance of the method.

The performance of low-level heuristics plays a crucial role in the performance of a hyper-heuristic methodology. In [26], the authors try to understand the performance efficiency of each low-level heuristic when it operates individually or in combination with other heuristics. As such, the concepts of competence and affinity (in terms of heuristics) have been defined and qualitatively estimated on a problem-specific hyper-heuristic approach. The resulting information from this process is exploited and used to improve the performance of problem-specific hyper-heuristics for the hybrid flow-shop scheduling problem. The tailor-made hyper-heuristics exhibit promising performance on the studied instances.

In parallel, novel search approaches have recently been incorporated within hyper-heuristic approaches. A novel diversification method is introduced in [27]. The method perturbs the problem instance under investigation to create justifiable new search directions. As such, various low-level heuristics that act upon this idea are proposed under a hyper-heuristic framework. The low-level heuristics are guided through an expressive grammar high-level strategy that has the ability to easily satisfy the problem constraints and thus easily produce feasible solutions. An experimental evaluation and analyses on the Ising spin glass and the p-median problem demonstrate its efficiency and its competitive performance.

When considering selection hyper-heuristics, each of the available search low-level heuristics (to be selected) is scored by a credit assignment mechanism. The credit assignment mechanism is responsible for assigning a reward, or credit value, to the applied heuristic, based on measurable feedback from the search procedure. The feedback may represent a measurement or qualitative characterization of the applied heuristic effects on the current state of the search process. The role of the credit value is crucial for the performance of the hyper-heuristic search algorithm, since it is the main information that guides the search procedure. Recently, different new types of feedback information have been introduced, based on landscape analysis techniques.

In [28], the evolvability of a search operator has been considered. The evolvability metric uses local characteristics of the fitness landscape to approximate the potential of a search component to generate improved solutions, from the current state of the search. An experimental study on benchmark problems reveals that evolvability feedback has significant potential to improve the performance of a hyper-heuristic, when compared against simple fitness improvement feedback.

Additionally, a novel selection hyper-heuristic methodology has been introduced in [29] to address the Capacitated Arc Routing problem. In the higher level, an online learning algorithm has been considered, the Dynamic Weighted Majority

method, to predict and determine the most appropriate search mechanism. The learning mechanism selects a crossover operator from an available set. Each of the available choices does not rely on its ability to improve the fitness of the generated solution. Instead, each operator can be scored based on four different fitness landscape analysis techniques. The analysis takes into account dispersion and neutrality as well as evolvability measures. Extensive comparisons on a set of benchmark problems verify the efficiency of the proposed methodology against state-of-the-art techniques.

Automatic Design of Algorithms

A comparison of the fields of meta-learning and hyper-heuristics has been recently proposed in [30]. The study draws parallels between methodologies and concepts from the two fields and advocates the common objective of automating the algorithm design process. A brief historical overview of automated algorithm design is presented along with a discussion on the similarities and differences between supervised machine learning and hyper-heuristics. Various issues are discussed, including the different levels of automation and generality, the problem and algorithm space, and various measures of performance.

The automatic design process was used to generate simple, general, modular, and efficient Iterated Local Search-based methodologies in [31, 32]. The generated methodologies are very competitive with the state-of-the-art methodologies in domain-independent meta-heuristic search [32]. Other hyper-heuristics based on Iterated Local Search that have been recently proposed include [33] in addition to the case study presented in the sections below. In parallel, a novel grammar representation has been introduced in [34] to be used for the automatic design of heuristic algorithms for hard combinatorial optimization problems. The proposed representation of the grammar includes a sequence of categorical, integer, and real-valued parameters and incorporates an off-line parameter configuration tool to search for algorithms for a given problem. Extensive experimentation on one-dimensional bin packing and on permutation flow-shop problems demonstrates the efficiency of both the grammar and the parameter configuration tools against an established grammatical evolution algorithm for automatic design. An extension of the methodology was able to build more complex algorithmic schemes from a composition of problem-dependent and problem-independent grammars [35].

A unified selection hyper-heuristic framework that handles one- and two-dimensional regular packing as well as two-dimensional irregular packing and cutting problems has been proposed in [36]. The framework utilizes several different stages to generate fast algorithms with competitive performance against the best problem-specific heuristic for a given problem. The framework generates different hyper-heuristic algorithms, by representing them as variable-length chromosomes of low-level heuristics that are being evolved by a messy genetic algorithm. A data mining-based feature selection procedure is used to determine the most relevant features in the problem-state representation, and an off-line analysis is employed to

discover the most representative set of well-performing heuristics. The framework has been evaluated on a large set of problem instances, and the experimental results show that it is able to generate very competitive hyper-heuristics for the given problem. In addition, the performance of the produced hyper-heuristics is comparable or even better than the application of the single low-level heuristics on their own. Moreover, an automatic design process has been proposed to efficiently solve two-dimensional rectangular blocks for packing [37].

Recently, the commonalities of certain areas of memetic computing and hyper-heuristics have been reviewed in [38]. This has led to the development of a new framework that combines a multi-meme approach with an adaptive operator selection hyper-heuristic to address continuous optimization problems. The framework rewards and adaptively selects the most suitable heuristics/memes during the search procedure. Although it has a simple structure, it exhibits high-performance gains against state-of-the-art search algorithms in a wide range of benchmark problems.

To avoid stagnation to local optima solutions, a multi-objective formulation has been proposed to formulate the single-objective two-dimensional packing problem [39]. A new memetic algorithm along with a parallel hyper-heuristic approach has been introduced to search the new formulation of the problem. The behaviors of the proposed methodologies are analyzed, and experimental results on two-dimensional packing problem instances suggest that the parallel hyper-heuristic is very competitive since it has found new best solutions for some problem instances.

The effectiveness and generality of a well-designed hyper-heuristic approach that outperforms human-designed problem-specific algorithms are demonstrated in [40]. Specifically, an Iterated Local Search-based hyper-heuristic methodology that learns from feedback of the problem at hand and dynamically selects among the available search operators is developed. An off-line learning approach is compared and contrasted with an online learning model on a large set of real-world problem instances from the first and second international timetabling competition. The online learning approach demonstrates superior performance over the off-line (static) model and is competitive with the state-of-the-art algorithms in the field.

Recent Theoretical Results in Hyper-heuristics

Although there is a significant growth of research in the field of hyper-heuristics, most studies are empirical. The theoretical foundations of hyper-heuristics are largely unexplored. Prominent examples of recent theoretical work in the field include mostly runtime analysis. A rigorous runtime analysis of hyper-heuristics has been presented in [41]. The analysis considers selection hyper-heuristics that operate on a predefined set of low-level heuristics. It reveals that the combination of heuristics can lead to exponentially faster search than deterministically chosen heuristics for a given problem. However, the analysis shows that an appropriate mixing distribution should be carefully selected.

A runtime analysis on some very common learning mechanisms has been performed in [42]. In particular, the analysis considers the simple random, random

gradient, greedy, and permutation learning mechanisms in the case of selection hyper-heuristics. The analysis shows that the learning mechanisms behave similarly, which suggests that they do not necessary improve the performance of the hyper-heuristics in the studied cases.

Multi-objective Optimization

A wide range of real-world problems require high-quality decisions for more than one objective. In general, an effective multi-objective algorithm should possess good search characteristics in terms of both solution diversity and convergence on the Pareto-front set. Several very effective multi-objective search algorithms have been proposed in the literature. They have different search dynamics, characteristics, advantages, and disadvantages. Hyper-heuristic approaches have also been explored within the context of multi-objective search (multi-objective hyper-heuristics).

The choice function approach has been used as a selection hyper-heuristic for a number of years. Interestingly, it still continues to be a promising and effective approach. The choice function along with the great deluge and late acceptance as nondeterministic move acceptance mechanisms has been investigated to address multi-objective optimization problems in [43]. The method utilizes the hyper-volume metric during the move acceptance process to effectively guide the search in promising areas and enhance both diversity and convergence characteristics of the method. The proposed method has been applied to traditional multi-objective benchmark problems in addition to the vehicle crashworthiness design problem [44]. Experimental analyses and results indicate that the proposed hyper-heuristic approach has the ability and potential to efficiently solve continuous multi-objective optimization problems.

Similarly, a hyper-heuristic method that combines the strengths of three well-known multi-objective algorithms has been introduced in [45]. The proposed approach implements an online learning choice function as a selection hyper-heuristic to combine the underlying algorithms. The choice function adaptively ranks the performance of the algorithms and determines which algorithm to apply at the next step. The introduced hyper-heuristic framework exhibits very effective performance on a wide range of multi-objective benchmark functions and on a real-world problem case, the vehicle crashworthiness design problem. The employed online learning mechanism greatly enhances the performance of the hyper-heuristic algorithm which is able to efficiently combine and exploit the advantages of the multiple low-level multi-objective algorithms.

In parallel, a multiple evolutionary algorithm portfolio has been developed in [46]. This approach utilizes simultaneously multiple algorithms for solving multi-objective optimization problems. The performance of each algorithm is evaluated by a score function, and the hyper-heuristic approach selects which to apply in the next step based on its score. The developed approach exhibits promising results. Another example of a recently proposed hyper-heuristic has been presented in [47]. This method includes diversity-based techniques to solve the patrol scheduling problem.

The concept of adaptive operator selection in multi-objective algorithms was introduced in [48]. This utilizes state-of-the-art models for adaptive operator selection to select the most suitable operator at each stage of the algorithm. The methodology extends the well-known MOEA/D algorithm, which decomposes the multi-objective problems to single-objective problems and optimizes them simultaneously. Thorough experimental analyses indicate that the proposed methodology is robust, its performance is significantly better than other state-of-the-art multi-objective algorithms, and the operator selection process works effectively. Likewise, a hyper-heuristic approach has been recently incorporated into the Multi-Objective Particle Swarm Optimization algorithm [49]. This is a particle swarm optimization variant for multi-objective problems. Its main characteristic is that it has to employ a leader selection strategy and an archiving mechanism.

In [50], a genetic programming-based hyper-heuristic is proposed to solve the bi-objective aspect of the water distribution network design problem. The goal here is to find the optimal pipe sizes required by a network to provide best service with minimal costs. The objective of the proposed hyper-heuristic methodology is to generate efficient search operators (mutation operators) for multi-objective algorithms applied to solve the bi-objective formulation of the problem. Interestingly, the generated mutation operators reveal useful information for the designer. For example, an evolved operator incorporates domain-specific knowledge (pipe smoothing), while others are able to reproduce well-known operators from the literature. The methodology exhibits significant potential on various problem instances, while the evolved operators are general and can be used as new algorithmic components for future multi-objective algorithms. A further analysis of the strengths of various search operators along with the features of the problem is provided in [51].

Hyper-heuristics in Dynamic and Uncertain Environments

In the last few years, hyper-heuristic methodologies have been employed to address applications that are inherently dynamic and uncertain. This poses a major challenge for the research community since the majority of the developed methodologies that operate on static environments cannot easily cope with the dynamic and uncertain aspects of some applications. Various recent hyper-heuristic methodologies embrace adaptivity and are able to cope with dynamically changing problem scenarios.

A hybrid hyper-heuristic has been introduced to consider dynamic optimization problems [52]. This combines a selection hyper-heuristic and a memory/search algorithm. The hyper-heuristic is able to select between the available low-level heuristics for the problem instance in hand. To evaluate its performance efficiency, the following two problems have been considered: the dynamic generalized assignment problem and the moving peaks benchmark. Experimental results indicate that the hyper-heuristic framework performs significantly better than the memory/search algorithm in the majority of the tested cases. From the considered selection heuristics, the choice function with the only improving acceptance criterion seems

to have the most promising performance, since it is able to track changes in the environment and it can rapidly react to different types of changes.

An additional recent study on hyper-heuristics in dynamic environments includes [53]. This approach analyzes the performance of a hyper-heuristic methodology that utilizes specialized meta-heuristics across different types of dynamic environment. An interesting real-world problem with uncertainty has recently been efficiently addressed by a hyper-heuristic approach [54]. This is the online path planning for autonomous unmanned aerial vehicles. Online path planning is a challenging problem that includes problem scenarios in uncertain or unknown environments. A hyper-heuristic methodology has been implemented to navigate these unmanned vehicles using a 3-D online path planning model with sensing uncertainty. An experimental study on various terrains with different characteristics demonstrates the efficiency of the methodology to create efficient navigation path plans.

Machine Learning Methodologies

Machine learning methodologies play an important role in the field. Various methodologies have been incorporated mostly into the higher level of hyper-heuristics to learn the behavior of the underlying heuristics and to make effective decisions to guide the search procedure toward the most promising areas of the search space. In parallel, the automatic design concept in hyper-heuristics is widely applicable in the machine learning field to generate novel and more efficient machine learning methodologies for specific classes of problems or tasks. Recent advances include a wide range of methodologies.

A new machine learning approach based on tensor analysis was introduced in [55, 56]. It utilizes tensor analysis techniques and in particular tensor factorization to reveal latent relationships between the low-level heuristics and the hyper-heuristic. Intuitively, it models the history trace of the considered hyper-heuristic method and tries to identify the low-level heuristics that exhibit promising performance. Tensor analysis and factorization are employed to identify promising interactions and features between low-level heuristics as the search method operates, by observing feedback from the search history and performance. The method then exploits the knowledge learned to improve its overall search ability.

Inspired by Inverse Reinforcement Learning theory, the authors in [57, 58] explored apprenticeship learning for generalizing experts' behavior. Apprenticeship learning or imitation learning or learning by demonstration has been widely applied in control and robotic applications. The main goal of apprenticeship learning is to learn by observing experts while they are in action. Within the context of hyper-heuristics, apprenticeship learning has been applied to learn expert behavior in vehicle routing [57] and online bin packing problem domains [58]. The learning procedure has been trained with diverse experts on small problem instances, and it has been tested on larger instances with different characteristics to test its generalization ability. The learning procedure successfully captures experts' different actions and generalizes them on unseen problem instances. Experimental results

and analyses suggest that the novel learning mechanism demonstrates outstanding performance gains in both problem domains.

Monte Carlo Tree search has been used in [59] to search the space of low-level heuristics for various problem domains. In particular, it models sequences of low-level heuristics as trees and searches for optimal sequences to be applied at the corresponding state of the search procedure. Experimental results on six problem domains demonstrate the generality of the method as well as its competitive performance.

An automated design of classification algorithms tailored to deal with both balanced and imbalanced data has been proposed in [60]. This aims to automatically design decision-tree induction algorithms. A thorough analysis is firstly performed to determine the impact of different kinds of fitness functions on the performance of the proposed methodology for both balanced and imbalanced datasets. The best-performing fitness function is then used for the automatic design process. The evolved decision-tree induction algorithm is compared against traditional decision-tree induction algorithms on a wide selection of datasets. The experimental analysis suggests that the proposed method significantly outperforms the other methods.

A grammatical evolution-based hyper-heuristic has recently been introduced for designing automatically split criteria in decision trees for data classification tasks [61]. A hyper-heuristic evolutionary algorithm to automatically build Bayesian Network Classifiers tailored to a specific dataset [62] represents another recent addition to the literature.

Automated Parameter Control and Tuning

Automated parameter control represents one of the major applications of hyper-heuristics. A recent overview of the trends and future directions of the area can be found in [63].

Two interesting parameter control methods based on hyper-heuristics and fuzzy logic have been recently proposed in [64, 65]. They investigate the impact of solving single-objective optimization problems with multi-objective approaches. The utilized methodology is a bi-objective approach that uses diversity as a second objective. A probabilistic selection hyper-heuristic and a fuzzy logic method have been utilized to control the parameters of the bi-objective methodology. Experimental analysis on a wide range of problems reveals the efficiency of both parameter control methods against several static parameter settings and self-adaptive rules. Overall, the resulting method is computationally efficient and is able to outperform the single-objective scheme in most of the tested cases. A similar approach has been applied to address the frequency assignment problem [66]. Additionally, fuzzy system methods have been investigated as selection hyper-heuristics to control parameters [67]. Off-line parameter tuning has been employed to fine-tune and successfully generate efficient heuristics for online bin packing [68].

A reinforcement learning-based parameter controller has recently been published in [69] to dynamically adapt the control parameters of evolutionary algorithms. The

method is a generic and parameter-independent controller that can be easily adapted on any evolutionary algorithm. Experimental analysis that incorporates the proposed parameter control method on various state-of-the-art evolutionary algorithms, to address several continuous optimization problems, suggests that the method is capable of good performance. The controller is able to improve the quality of the solutions found by the algorithms with minimal effort and additional computational resources. Similar approaches include entropy-based controllers [70]. Furthermore, a recent study investigates the impact in performance of various feedback mechanisms that have been used for adaptive parameter control in evolutionary algorithms [71]. The study considers indicative feedback measures for both single- and multi-objective optimization problem formulations.

Time series prediction methodologies have recently been used [72] as parameter controllers to accurately predict suitable parameter values during the search process. In particular, various prediction methods have been utilized to predict future parameter values based on historical data from the search procedure. Evaluation of the considered methodologies on evolutionary algorithms suggests that the simple linear regression performs quite well, but without having a significant difference against the studied methods. The impact in performance of predictive parameter control methods is evident only when the performance data complies with the required assumptions of the prediction model, resulting in significant performance differences when compared against state-of-the-art parameter controllers. Otherwise, the prediction method does not exhibit any evident impact on the performance of the considered algorithm.

Hyper-heuristics for Scheduling Problems

As widely identified in previously published overview and survey articles [1–5, 7, 8], scheduling problems have played a major role in the development of hyper-heuristic approaches.

The automatic design of dispatching rules has been the subject of recent research attention because of the time-consuming process of manually designing such rules. An analysis of the representation of dispatching rules has been recently undertaken in [73]. The representation of dispatching rules essentially determines the search space and its complexity. As such, efficient representations highly impact upon the search process. Three different kinds of representations have been discussed in [73]: a linear combination of attributes, artificial neural networks, and a tree-based representation. Empirical analysis on the suitability of each representation in dynamic stochastic job-shop scenarios suggests that the tree representation, evolved with a hyper-heuristic genetic programming method, is the most efficient. Artificial neural network representations operate well mostly on processes with a small computational budget, while linear representations operate competitively only on quite small computational budgets.

Dispatching rules for semiconductor manufacturing have been also automatically generated with genetic programming [74]. Grammatical evolution has been suc-

cessfully applied as a hyper-heuristic methodology to address capacitated vehicle routing problems [75, 76]. Comparisons between adaptive and grammar-based hyper-heuristic approaches have been recently performed for various problem domains [77]. Moreover, a genetic programming-based hyper-heuristic has been considered in [78] to tackle order acceptance and scheduling problems in make-to-order manufacturing systems. This automatically generates dispatching rules that have stochastic behavior in order to improve the search procedure. The evolved rules exhibit high-performance gains compared to rules produced by either tailor-made heuristics or a simple version of the considered genetic programming method.

Several novel multi-objective genetic programming-based hyper-heuristics have recently been proposed that are able to generate scheduling policies in job-shop environments [79]. This paper proposes an automatic design approach to generate dispatching and due-date assignment rules for a dynamic version of the multi-objective job-shop scheduling problem. Having identified the complexity and the labor needed to manually design an effective scheduling policy, four multi-objective algorithms are developed to automate the process. All approaches are capable of handling multiple scheduling decisions in parallel. Thorough experiments and analyses demonstrate that the evolved Pareto sets represent effective scheduling policies that dominate policies from combinations of existing dispatching rules. These policies also have promising performance on unseen scenarios with various shop configurations. Generally all proposed methodologies exhibit not only effective but also very meaningful scheduling policies that help the decision-maker to understand their characteristics.

A two-phase search approach has been introduced in [80] to tackle the Workover Rigs Scheduling problem for maintenance services in onshore oil wells. The problem can be characterized as a parallel heterogeneous machine scheduling problem derived from the maintenance planning of heterogeneous wells. This problem is to find schedules for a small number of work-over rigs that minimize the production loss associated with the large number of wells waiting for service. The first phase of the proposed approach is a selection hyper-heuristic that searches for a promising initial solution to warm-start the second stage, which utilizes an exact branch, price, and cut approach. The considered hyper-heuristic uses learning mechanisms to learn from feedback during search and select the most effective low-level heuristic to apply in the next step. Having applied the abovementioned methodology to a variety of real-world problem instances from Petrobras, the Brazilian National Petroleum Corporation, the authors were able to identify classes of heuristics, which seem to be more efficient for solving the problem. In general, it was observed that the learning mechanisms were very effective and able to guide the search to promising areas of the solution space. It is worth noting that the hyper-heuristic approach generated solutions that were very near to the optimal solutions found by the exact algorithm.

An interesting analysis has been performed in [81] that investigates the performance impact of heuristics while solving a problem with routing and rostering characteristics. The implemented hyper-heuristics act as analysis tools on the heuristic behavior. The aim is to analyze the behavior of the available heuristics

and determine the requirements that a heuristic should consider to solve such problems. The following three different scheduling and routing problems have been considered: home care scheduling, routing and rostering of security guards, and maintenance personnel scheduling. The analysis revealed that some specific features of the problems under investigation significantly affect the performance of the heuristics. These features are the number of activities, the number of resources, and the planning horizon. The observed information of such analysis can be exploited and used in future search approaches to efficiently solve similar problems that share common characteristics. As such, hyper-heuristics can be employed as analysis tools for a particular problem class under investigation and may offer novel information on the characteristics of the problem class.

Nurse rostering is one of the applications where hyper-heuristics find wide applicability, with recent works including research in selection hyper-heuristics [82, 83]. A generic cooperative agent-based search framework has recently been introduced [84] that is able to incorporate the search strengths of various meta-heuristics to efficiently solve nurse rostering problems. The framework promotes the asynchronous cooperation of effective agents using pattern matching and reinforcement learning techniques. This enables the exploration of different fairness objectives across several real-world rostering problem instances. A thorough experimental analysis demonstrates the efficiency of such frameworks to generate high-performance rostering solutions in minimal time.

Several hyper-heuristic methodologies draw upon swarm intelligence techniques, such as particle swarm and ant colony optimization. A particle swarm optimization-based hyper-heuristic approach has been introduced to address the resource-constrained project scheduling problem [85]. This operates on the heuristic space by representing its particles as ordered sequences of heuristics to apply for the given problem. Subsequently, a feasible solution is constructed by a serial scheduling construction mechanism, while the generated solution is improved by double justification local search. Several problem instances from the well-known PSPLIB library have been used to test and compare the hyper-heuristic algorithm with other algorithms. Computational results verify its efficiency and competitive performance.

The problem of intercell scheduling with single-processing machines and batch-processing machines has been tackled in [86] with an ant colony optimization hyper-heuristic search method. Generally, intercell transfers in cellular manufacturing systems are essential to substantially reduce production costs. The formulation of this scheduling problem considers the following three subproblems in parallel: an assignment, a sequencing, and a batch subproblem. An ant colony hyper-heuristic has been adapted to search among different assignment heuristic rules for both parts and machines simultaneously. Subsequently, the discovered rules are applied to generate schedules. The efficiency of the proposed method is evaluated on randomly generated problem instances. The experimental results indicate that the method is significantly more efficient than CPLEX and genetic algorithms. Channel scheduling in multicell networks has also been addressed recently with hyper-heuristic methodologies [87].

A novel hyper-heuristic methodology has been proposed in [88] to solve task scheduling in cloud computing systems. Usually, rule-based scheduling algorithm is used for task scheduling in such systems. The method presented here is able to learn from the performance of several available scheduling algorithms and select the most suitable algorithm to apply. It incorporates a diversity and fitness detection strategies to efficiently balance the diversification and intensification behavior of the search procedure. This approach significantly outperforms several state-of-the-art scheduling algorithms on both simulation and real system scenarios. It produces solutions that have a significantly better makespan compared against other scheduling algorithms.

The problem of resource provisioning-based scheduling tasks in large-scale distributed environments, such as grids, has been addressed by exploring a hyper-heuristic scheduling method [89]. The main goal of this methodology is to efficiently schedule jobs on the available resources in order to simultaneously minimize execution time and increase security and reliability. The proposed hyper-heuristic algorithm is a particle swarm optimization-based algorithm that has been specifically developed to efficiently schedule jobs on available resources while addressing security requirements and constraints. A thorough experimental analysis reveals that the hyper-heuristic algorithm outperforms the state-of-the-art approaches, in terms of minimizing time, cost, and the makespan of the scheduling process.

Hyper-heuristics also have wide applicability in transportation scheduling. Motivated by a real railway-based disaster transportation relief scenario in the area of China, a selection hyper-heuristic search method [90] has been proposed to tackle emergency railway transportation problems. The evolutionary hyper-heuristic utilizes various search strategies as low-level heuristics and rewards each search strategy by assessing its ability to generate successful movements in the search space. The method has been compared with various state-of-the-art evolutionary algorithms on several problem instances, which have been created from disaster rescue operations data in China. Experimental results show high-performance gains across all instances against the considered evolutionary algorithms. The method has also been successfully applied on a real emergency during the 2013 Dingxi earthquake in China, producing a very satisfactory solution.

Another recent application of hyper-heuristic methodologies is the dial-a-ride transportation problem with time windows [91]. This is concerned with scheduling a set of available vehicles to pick up customers from an origin location and deliver them to a destination. It considers a range of constraints. The proposed hyper-heuristic methodologies are efficient and result in competitive performance against state-of-the-art approaches from the literature. Moreover, cooperative hyper-heuristics have been successfully applied to bus driver scheduling problems [92].

Recent Educational Timetabling Applications of Hyper-heuristics

A recent overview and critical analysis of the literature on the hyper-heuristic methodologies applied to educational timetabling problems have been published in

[6]. The article focuses on the hyper-heuristic research in three general educational timetabling problems, the university examination, the university course, and the school timetabling problems, as well as proposes several future research directions on the subject.

An estimation of distribution algorithm has recently been introduced [93] as a general hyper-heuristic approach to address exam timetabling problem instances. The objective was to develop a hyper-heuristic with increased generality over different problem instances. The hyper-heuristic operates on sequences of low-level exam-selection heuristics that construct a timetable, based only on non-domain-specific knowledge. The resulting algorithm is capable of guiding the search to very promising areas. It generates efficient sequences of timetabling heuristics. Experimental results on various real-world exam timetabling and graph coloring problem instances suggest that the hyper-heuristic is generic across all problem instances, with competitive performance against other hyper-heuristic approaches. In addition, the developed approach is capable of learning the most promising heuristic for the problem at hand.

A two-stage hybrid hyper-heuristic approach has recently been proposed to solve timetabling problems [94]. The two-stage hybrid approach utilizes a Kempe chain move heuristic and the time-slot swapping heuristic as search operators. Firstly, it generates and automatically analyzes random heuristic sequences. The analysis keeps the repeated heuristics of the best sequence and empties the remaining positions to be processed in the next stage. The second stage randomly assigns sequences if heuristics are in the empty positions in order to search for high-quality sequences of heuristics. The constructed ones are used for the given problem. The hybrid method is evaluated on two challenging sets of problems, the Toronto benchmark and the exam timetabling set of problems from the second International Timetabling Competition. The method exhibited competitive performance against the state-of-the-art methodologies for both problem sets.

New selection hyper-heuristic approaches that are based on simple stochastic search methods have exhibited outstanding performance in the recent high school timetabling competition [95]. Indeed, such approaches have provided exceptional results and new best solutions to very challenging problem instances. A hybrid hyper-heuristic which utilized sequences of low-level heuristics has also been applied recently in examination timetabling problems [96].

Games and Education

Hyper-heuristic methodologies have recently been proposed in the areas of games and education. A recent study focuses on solving the Jawbreaker puzzle [97]. The evolutionary process operates on the low-level heuristic space and produces sequences of low-level heuristics that can be applied to solve the given puzzle. The resulting methodology is able to efficiently solve instances of the puzzle with various sizes and levels of difficulty, including very difficult levels.

An educational game-based software tool has been introduced in [98] to teach the intrinsic operational characteristics of hyper-heuristics to engineering students. The tool is based on the Bubble Breaker puzzle, and it represents a strong synergy with the concept of selection hyper-heuristics. In addition, a hyper-heuristic method is used, under a context-aware ubiquitous learning environment, to maximize the learning efficacy of students [99]. The main goal of the search method is to locate quality learning paths for students by considering real-world context and constraints. The proposed approach is evaluated on real-world scenarios.

Other Recent Developments

The general character and the wide applicability of hyper-heuristics enable them to efficiently address a wide variety of applications. Here, the most recent developments in various scientific fields are presented, from linear algebra to aircraft structural design applications. A genetic programming-based hyper-heuristic is introduced in [100] to evolve the structure of the Sloan algorithm that aims to find a reduced envelope size of matrices. The envelope is the sum of the distances between each element of the matrix and its main diagonal. Its size has a major impact on the performance of large linear systems solvers. The best evolved algorithm (with the addition of a local search method) exhibits substantial performance gains and significantly outperforms state-of-the-art algorithms in the literature on classic problem sets.

The magic square problem has been efficiently tackled by a wide range of selection hyper-heuristic approaches [101] that utilize perturbative low-level heuristics.

A modified version of the well-known and widely used choice function has been successfully hybridized with a late acceptance strategy hyper-heuristic to tackle the multidimensional 0–1 knapsack problem [102]. In addition, a stochastic hyper-heuristic version of the choice function has been utilized to address the winner determination problem in [103, 104].

Another interesting application of hyper-heuristics is the real-world warehouse storage location assignment problem [105], in which a genetic programming methodology is employed to generate matching functions that guide the selection of subsets of items for a given problem. The methodology is able to locate high-quality solutions on the training problem cases. It is also able to perform well on unseen scenarios in the testing phase. This indicates that the evolved heuristics are reusable and can be directly applied to similar problem scenarios. Furthermore, aircraft structural design optimization problems have recently been efficiently addressed by hyper-heuristic methodologies [106, 107].

The problem of distributing resources among multiple threads in a processor has been studied in [108]. A learning hyper-heuristic that predicts the best-performing heuristic between a set of available heuristics has successfully been applied to the problem. On average, its performance is comparable or significantly better than the state-of-the-art techniques in the field.

Case Study: A Selection Hyper-heuristic-Based Iterated Local Search

In this section, a simple selection hyper-heuristic framework is demonstrated to provide a tutorial-style introduction to hyper-heuristic development. Specifically, a hyper-heuristic framework is developed that is based on the well-known Iterated Local Search algorithm. The development process and the decisions that had to be made during implementation are briefly discussed (section "Iterated Local Search with Adaptive Heuristic Selection"). The framework implements an action selection model that operates on the Iterated Local Search algorithm to adaptively select among various search low-level perturbation heuristics (section "Action Selection Models"). It is based on the HyFlex platform which can be used to easily implement prototypes of hyper-heuristic search methods without having to be concerned about the implementation of problem domain low-level heuristics (which are available from the HyFlex site) (section "The HyFlex Framework").

The HyFlex Framework

HyFlex is a Java-based software framework that enables the easy development and testing of cross-domain hyper-heuristic search methodologies. The framework implements a common software interface to provide all the essential ingredients for easily developing a general-purpose search algorithm, without having to implement specific problem domain knowledge.

The framework implements six different combinatorial optimization problems, namely, maximum satisfiability (Max-SAT), one-dimensional bin packing, personnel scheduling, permutation flow-shop scheduling, the traveling salesman problem, and the vehicle routing with time windows problem. For each problem domain, there are 10–12 available problem instances, including both challenging benchmark and real-world industrial problem instances. The framework additionally exposes the user to a broad set of state-of-the-art low-level heuristic search operators with different search dynamics and characteristics. For all problem domains and all instances, appropriate evaluation functions have been implemented to easily evaluate generated solutions.

HyFlex currently has two main versions, 1.0 and 1.1. The first version includes the main functionality described above, while the second version adopts the concept of batch-mode hyper-heuristics, i.e., allowing the developed hyper-heuristic methodology to operate on a whole set of instances for each run, instead of using a single instance. A description of the APIs and documentation for both versions can be found in [109, 110] and in [111–113]. The HyFlex framework has been utilized in two cross-domain search challenges, CHeSC 2011 [111] and CHeSC 2014 [112].

In this chapter, the first version of the framework has been adopted to develop the hyper-heuristic presented in the next section.

Iterated Local Search with Adaptive Heuristic Selection

This section discusses the motivation (and provides a detailed description) for designing a simple and general applicable hyper-heuristic search algorithm. However, the main purpose of this section is not to design a hyper-heuristic that has the best performance obtained in the literature so far. Instead, the goal of this section is to demonstrate how a simple search algorithm can be seen as a general hyper-heuristic approach. Moreover, it aims to show how the search algorithm can easily be applied to different problem domains without considering many unusual and made-to-measure design choices. The resulting approach can be easily fine-tuned on the application domains at hand and produce state-of-the-art performance gains [31, 32, 34].

The developed approach is based on a simple, general, and highly successful local search algorithm, the Iterated Local Search algorithm [12, 114, 115]. Iterated Local Search (ILS) is a single-point local search method that belongs to the wide category of stochastic local search methods [12]. The main idea behind its structure is to "iteratively create a sequence of potential solutions produced by a given heuristic search method, that leads to much better solution quality, than performing repeated random executions of it" [12, 114, 115]. In general, ILS includes four main stages in its structure. Firstly, ILS generates an initial solution for the problem at hand and then iteratively improves the current solution by applying a diversification (perturbation) and an intensification (local search) operation. Diversification is accomplished by applying a perturbation heuristic on the current solution, while intensification is achieved by employing a local search method on the perturbed solution. Having performed these two search operations, ILS determines whether the new solution is accepted to the next iteration, via an acceptance criterion. As such, in order to create an effective ILS variant, one should carefully determine the perturbation heuristic, the local search, and the acceptance criteria. A vast amount of available choices for various problem domains can be found in the literature [12, 114, 115].

Notice that the strength of perturbation heuristics varies among the available heuristics and greatly influences the behavior of the algorithm. For example, strong perturbations vastly increase the diversification of the search and might lead to a random restart-like behavior, i.e., there is a very low probability to find better solutions. In contrast, perturbations with small strength might lead to very similar solutions that will not help the local search procedure find a better solution. Instead, the local search might revisit its previous solutions. Similarly, acceptance criteria can influence the balance between the intensification and diversification search abilities of the algorithm. For example, the acceptance of improving solutions will increase the intensification behavior of the algorithm, while accepting worsening moves will increase diversification.

Naturally, several ILS variants adapt their search strategies as well as their properties during the search procedure to react on the respective stage of the search. Representative reactive search principles can be found in [12, 114, 116].

Arguably this is the essence of a hyper-heuristic methodology [1–8]. Motivated by these findings, this approach is adopted to develop a simple and general hyper-heuristic search framework and apply it to various problem domains. In particular, a simple framework of Iterated Local Search-based hyper-heuristic (HHILS) is implemented to exhibit the development process and the design choices that an interested researcher, or practitioner, might be faced with during the development phase of the hyper-heuristic search methodology.

In general, the HHILS framework follows the basic algorithmic structure of an Iterated Local Search methodology and enhances its reactive search ability with an action selection and a credit assignment module. The action selection module is devoted to efficiently predict and select the most suitable action to apply in the next step, while the credit assignment module is responsible for assigning a reward, or a credit value to the applied action, based on feedback from the search procedure. Arguably, the action selection module can be applied on any level or combination of levels within the search procedure, i.e., it can select among different perturbation heuristics, local search methodologies, and acceptance criteria, or any possible combination of them. This design choice is based on the level of adaptivity or reactivity that the researcher/practitioner is willing to incorporate in the search algorithm. In many cases, this decision will have a major impact on the performance of the resulting search methodology, since the adaptive procedure may rapidly change the dynamics of the search procedure, i.e., the diversification and intensification levels of the search. Here, a simple case is adopted in which the action selection module will operate only in the perturbation phase of the algorithm, i.e., it will be able to select from a set of available perturbation low-level heuristics.

The selected and applied action is scored based on the credit assignment module. The main role of the credit assignment module is to assign a reward, or credit value, to the applied action, based on feedback from the search procedure. The feedback can be seen as one or more measurements (or qualitative characterizations) of the applied action effects on the current state of the search process. The most common measurement for a search action is the quality improvement of its state (e.g., the difference in fitness value between the previous and the current state). However, various other qualitative different characterizations may be used, such as ranking successful movements [117], and evolvability [28, 29] of the search procedure. Such feedback is able to change the search characteristics of the algorithm and thus guide the search based upon different objectives. For example, the resulting algorithm will have more diversification characteristics or will avoid stagnating during the search procedure. Arguably, the quality of the feedback has a major impact on the performance and the success of the action selection model. As such, the literature includes various different attempts to investigate the impact of different feedback characterizations, like fitness improvement [117–120], successful movements [121], rank based improvements [117], and evolvability [28, 29]. The selected feedback for the HHILS framework is based on the mean improvement of the applied perturbation heuristics and will be explained in detail below.

Algorithm 1: Pseudo Code of the HHILS Framework of ILS Based Hyper-heuristics

1: Initialize the action selection, and the credit assignment module as well as create all data structures required by HHILS.
2: $s_{cur} \leftarrow$ GenerateInitialSolution() /* **Generate Initial Solution:** Initialize or construct a solution for the problem instance at hand. */
3: **while** termination criteria do not hold **do**
4: selected$_{llh} \leftarrow$ ActionSelection(str) /* **Action Selection:** Select a low-level heuristic with the str-th action selection model. */
5: $s_{tmp} \leftarrow$ ApplyAction(s_{cur}, selected$_{llh}$) /* **Perturbation:** Perturb the current solution (s_{cur}) with the selected low-level heuristic (selected$_{llh}$). */
6: $s_{tmp} \leftarrow$ ApplyLocalSearch(s_{tmp}) /* **Local Search:** Apply a local search procedure on the temporary solution s_{tmp}. */
7: $s_{cur} \leftarrow$ AcceptanceCriterion(s_{cur}, s_{tmp}) /* **Acceptance:** Accept which solution, between s_{cur}, and s_{tmp}, will survive to the next iteration based on an Acceptance criterion. */
8: CreditAssignment(selected$_{llh}$) /* **Credit Assignment:** Score the used action (selected$_{llh}$) based on feedback from the problem at hand. */
9: **end while**
10: **Return:** the best solution found so far s_{cur}.

A family of different HHILS variants can be easily produced by adopting new action selection models and different credit assignment feedback functions. Algorithm 1 presents the general structure of the HHILS framework. Although some design choices have been made based on specific details related to the HyFlex framework (which was used for the implementation of HHILS), the algorithmic structure of the HHILS framework is relatively general and can be easily adapted to any other available framework.

HHILS firstly initializes all the necessary modules and data structures that are required by the algorithm to run properly (line 1 of Algorithm 1). This procedure includes the initialization of the credit assignment module with an empty or zero initial reward as well as the initialization of the action selection module that sets all actions to have equal chances to be selected. Next, HHILS initializes a solution (s_{cur}) for the problem instance at hand (line 2). The initial solution can be either generated randomly or constructed with a constructive heuristic. This depends on the problem domain and the available method to initialize, or construct, a feasible solution. HyFlex provides such functionality for all problem domains which is convenient for easily constructing an initial solution independently of the problem domain.

Having found an initial solution, each iteration of HHILS performs the following five main steps. Let set $\mathcal{S}$ define the κ available perturbation low-level search heuristics $\mathcal{S} = \{llh_1, llh_2, \ldots, llh_\kappa\}$, for the problem domain in hand. In HyFlex, the

developer has the option to select between greedy and non-greedy heuristics. Here, all available non-greedy heuristics for the current domain have been considered, which correspond mainly to *mutation*- and *ruin-and-recreate*-type-based heuristics. HHILS employs an action selection method (`ActionSelection`(*str*), line 4) to predict and select the most appropriate perturbation low-level search heuristics to apply at the next step (selected$_{llh} \in \mathcal{S}$). The `ActionSelection` method might include various models, such as uniform selection and proportionate selection based on the reward values. The user is able to define which model to use with the *str* variable $str \in \mathcal{M}$, where $\mathcal{M}$ is the set of the available m action selection models, $\mathcal{M} = \{\mu_1, \mu_2, \ldots, \mu_m\}$. Section "Action Selection Models" briefly describes the models used in this chapter.

After selecting the selected$_{llh}$ perturbation low-level heuristic, HHILS proceeds with its application on the current solution s_{cur} to perturb its current position and create a new solution, s_{tmp} (line 5). The new position (s_{tmp}) is being immediately refined by the `ApplyLocalSearch`(s_{tmp}) procedure, which applies a greedy local search heuristic. In HyFlex, greedy local search heuristics belong in the *local search* category and have the property that they are not able to produce a worse solution than the given one. To avoid making a poor choice for the current problem instance, all of the available local search methods are applied in an iterative way [32]. More specifically, given a list $\mathscr{L}$ of the available λ local search heuristics, $\mathscr{L} = \{l_1, l_2, \ldots, l_\lambda\}$, at each iteration, a local search method is selected in a uniform random way and is applied to the current solution (s_{tmp}). If the application of the selected local search method does not lead to a better position (fitness improvement), it is excluded from the active list, until another local search heuristic finds an improved solution. If all the local search methods in the active list do not lead to an improved solution, then a local optimum has been reached. In this case, the iterative process will finish with the best solution found so far (s_{tmp}). The algorithmic scheme of the iterative local search procedure is illustrated in Algorithm 2.

Having applied the aforementioned transformations on s_{cur}, a new temporary solution s_{tmp} has been created. In the next step, HHILS employs an acceptance criterion `AcceptanceCriterion`(s_{cur}, s_{tmp}) (line 7 of Algorithm 1) procedure that is responsible to decide which of the two solutions will be accepted for the next iteration. In the literature, there are various choices available for the acceptance criterion [12], where the most common ones are to accept only improved solutions or to accept worsening solutions based on a probability distribution. These criteria will affect the diversification and intensification abilities of the current search method, for the given problem. Their impact depends on the landscape structure of the problem at hand. HHILS utilizes a Metropolis-based acceptance condition that accepts a worse solution based on a probability distribution. Intuitively, the greater the fitness improvement, the greater the probability, or the greater the worsening fitness, the smaller the likelihood to accept the worse solution. A representative example of such an acceptance condition is the simulating annealing method [122].

Algorithm 2: Pseudo Code for the HHILS `ApplyLocalSearch(`s_{tmp}`)` Method

1: $\mathcal{L}_{active} \leftarrow \mathcal{L}$
2: **while** $\mathcal{L}_{active}$ is not empty **do**
3: $i \leftarrow$ uniformly random select a local search from $\mathcal{L}_{active}$
4: $s_{ls} \leftarrow l_i(s_{tmp})$: Apply l_i local search on s_{tmp}
5: **if** $f(s_{ls}) < f(s_{tmp})$ **then**
6: $s_{tmp} \leftarrow s_{ls}$
7: **else**
8: $\mathcal{L}_{active} = \mathcal{L}_{active} \backslash \{i\}$
9: **end if**
10: **end while**
11: **Return:** the best solution found so far s_{tmp}.

Specifically, the probability density function of acceptance is defined as $p(f(s_{cur}), p\left(f\left(s_{cur}\right), f\left(s_{tmp}\right), T, \mu_i\right) = e^{\frac{f(s_{cur})-f\left(s_{tmp}\right)}{T \cdot \mu_i}}$, where $f(s_{cur})$ and $f(s_{tmp})$ are the objective value of the s_{cur} and s_{tmp} solutions, respectively, T is a constant temperature value with $T \in \mathbb{R}$, and μ_i is the mean improvement of the improving iterations [32]. The role of μ_i is to normalize the objective value difference by a quantity that does not depend on the problem at hand. Notice that the value of the temperature parameter is problem dependent and has to be fine-tuned for the considered problem in order to obtain optimal performance. Here, the temperature is fixed in all experiments to $T = 2$. A short and interesting study on the temperature behavior can be found in [32].

By this stage, HHILS has finished with the searching operations, and the applied low-level perturbation heuristic can be scored based on its search behavior. The final step of HHILS, before proceeding to the next iteration, is to employ the credit assignment module `CreditAssignment(`$selected_{llh}$`)` to score the selected perturbation heuristic, $selected_{llh}$ (line 8 of Algorithm 1). As such, HHILS defines the reward of an action as the improvement moves performed by the action, normalized by the total time it spends on them. Such a reward is a representative score for the improvement rate of each action over the time spend for each improvement. Specifically, let $\mathcal{A} = \{a_1, a_2, \ldots, a_\alpha\}$ be the available set of α actions (note that in HHILS $\mathcal{A} = \mathcal{S}$) and t_{α_i} be the time consumed by action α_i on searching the solution space (initially $t_{\alpha_i} = 0, \forall \alpha_i \in \mathcal{A}$). Then at each iteration, the time spent for the applied action, i, is calculated as $t_i = t_i + t_i^{spent}$, where t_i^{spent} is the time of action i spent on the current iteration. The reward value (r_i) of the applied action (i) can be calculated according to $r_i = \frac{1+improvement_i}{t_i}$, where $improvement_i = f(s_{cur}) - f(s_{tmp})$. In general, various different reward approaches can be used at this point, such as the instantaneous latest reward or the average or a ranked-based reward [117]. However, some of them tend to be unstable and noisy

estimations of credit due to the stochastic nature of the search process. To alleviate this drawback, the empirical quality of an action is estimated by utilizing the average value of a sliding window of its latest w rewards. More specifically, let W_i be a set of the latest w improvement rewards (r_i) achieved by the action a_i during the time step t of the hyper-heuristic. The final credit assignment value $r_{a_i}(t)$ is the average reward of the sliding window W_{a_i}, which can be determined by

$$r_{a_i}(t) = \frac{\sum_{j=1}^{|W_{a_i}|} W_{a_i}(j)}{|W_{a_i}|} \tag{1}$$

where $|W_{a_i}|$ indicates the cardinality of the set W_{a_i}.

Finally, the algorithm will iterate until one or more termination criteria hold. Here, the maximum allowed CPU wall clock time is used as termination criterion, $t_{allowed}$, which the algorithm may consume to solve the problem under investigation. It is worth noting that the HHILS framework does not consider whether the algorithm gets trapped in a particular position of the solution space for a long period of time. This is a very common situation in which either the algorithm is in a local optimum or the search operators are not able to find a better solution for a long period of time. In such cases, a restarting procedure is essential to help the searching capabilities of the algorithm to diversify the current solution. However, a restarting procedure requires specific design choices that for the sake of simplicity are overlooked in this tutorial demonstration.

Action Selection Models

As discussed previously, HHILS is a general framework that has the ability to adopt any action selection model. The literature includes various action selection models from different scientific fields such as statistics, filter theory, artificial intelligence, and machine learning. Each model has different advantages and disadvantages that depend on the properties of the underlying reward distribution that it is trying to predict. Representative examples of such models that have been used for this purpose include probability matching [123, 124], adaptive pursuit [123, 124], statistical-based models like the multinomial distribution with history forgetting [118, 121], evolutionary meta-learning methods [125–128], and reinforcement learning approaches [117, 129, 130].

In this chapter, five different models have been utilized: the uniform selection model, the proportional selection model, the probability matching [123, 124], the adaptive pursuit [123, 124], and the upper confidence-bound multi-armed bandit methodology [117, 130].

The first two models are very simple and act as baseline models for the HHILS framework. The uniform selection model simply selects between the available actions randomly by following a uniform distribution. In this case, the scores

assigned to each action do not have any impact on the selection process. This is the baseline methodology that will provide insights into the usefulness and impact of applying an action selection model on HHILS. Notice that several studies have identified the usefulness of random variation in either action or parameter spaces [63, 131, 132]. The second model performs proportional selection among the available actions based on their assigned scores from the credit assignment module. Intuitively, this model provides proportional chances to each action based on their reward scores during the optimization phase, i.e., the most successful will have more chances to be selected again in the next step. The proportional selection is performed by applying a simple roulette wheel selection [133].

Probability matching (PM) [117, 123, 124] is a simple probabilistic model that updates the selection probability of each action proportionally to its empirical quality with respect to the other actions. Specifically, given a set $\mathcal{A} = \{a_1, a_2, \ldots, a_\alpha\}$ of the available α actions (here $\mathcal{A} = \mathcal{S}$), let $P(t) = \left\{p_{a_1}(t), p_{a_2}(t), \ldots, p_{a_\alpha}(t)\right\}$ be the selection probability vector of all actions, where initially $p_{a_i} = 1/\alpha, \forall a_i \in \mathcal{A}$. After the application of the perturbation, the local search, and the acceptance criterion stages, a reward $(r_{a_i}(t))$ is measured from the credit assignment module. Notice that in the general case, the reactive procedure operates on a nonstationary environment where the estimation of the empirical quality of an action might be more accurate and reliable if the more recent rewards influence the empirical quality more than the past ones. Therefore, in such cases, it is a common practice by various action selection models to estimate the empirical quality $q_{a_i}(t)$ of each action (a_i) in accordance with the following relaxation mechanism:

$$q_{a_i}(t+1) = q_{a_i}(t) + \gamma \left(r_{a_i}(t) - q_{a_i}(t)\right). \tag{2}$$

where $\gamma \in (0, 1]$ is the adaptation rate (here γ is fixed to 0.1) [117, 123, 124]. Having calculated the empirical quality of each action (a_i), PM adapts its selection probability (p_{a_i}) using the following rule:

$$p_{a_i}(t+1) = p_{\min} + (1 - \alpha \cdot p_{\min}) \frac{q_{a_i}(t+1)}{\sum_{i=1}^{\alpha} q_{a_i}(t+1)} \tag{3}$$

where $p_{\min} \in [0, 1]$ is the minimum probability value of each action. $p_{\min}$ is responsible for ensuring that each action will always have a small probability (here $p_{\min} = 0.05$) [123, 124]. Having estimated the probabilities of each action, PM utilizes a stochastic proportional selection mechanism, based on these probabilities, to select which action will be applied in the next step.

Notice that PM allows the inefficient actions to have at least $p_{\min}$ selection probability. Thus, the best action will be selected with probability $p_{\max} = (1 - (\alpha - 1) \cdot p_{\min})$ However, this hinders the efficiency and performance of PM, since all inefficient actions keep being selected. Therefore, the adaptive pursuit (AP) strategy [123, 124] addressed this drawback by embracing a winner-takes-all strategy. In detail, AP, instead of updating proportionally the action probabilities, increases the probability of the corresponding best action $(a_{i^\star})$, while in parallel, it

decreases the probabilities of the other actions according to the following equations:

$$a_{i^\star} = \arg \max_{i=1,2,\ldots,\alpha} \left\{ q_{a_i}(t+1) \right\} \tag{4}$$

$$p_{a_i}(t+1) = \begin{cases} p_{a_i}(t) + \beta \left(p_{\max} - p_{a_i}(t) \right), \text{ if } a_i = a_{i^\star} \\ p_{a_i}(t) + \beta \left(p_{\min} - p_{a_i}(t) \right), \text{ otherwise} \end{cases} \tag{5}$$

where $\beta \in [0, 1]$ is a learning rate that manipulates the greediness of the winner-takes-all strategy (here $\beta = 0.8$). Intuitively, AP first finds the best action of the current step and then updates the probabilities by being in favor of the best action.

The last action selection model used in this chapter formulates the action selection model as a multi-armed bandit (MAB) problem and utilizes the upper confidence-bound (UCB) algorithm to solve it. The MAB problem considers κ one-arm slot machines and a player that has to decide which arm to pull, how many times to pull it, and in which order to choose between the available arms, having in mind that each machine provides a random reward. Intuitively, the MAB problem faces the well-known trade-off of *exploration* versus *exploitation*, where each time the player has to decide either to "exploit" its current knowledge, i.e., to pull the arm that gives the highest expected payoff, or to "explore" the available choices, i.e., to acquire more information about the expected payoff of the other arms.

More specifically, let us have κ arms (one-arm slot machines), where each arm (i-th) has a fixed unknown reward probability $p_i \in [0, 1], \forall i \in \{1, 2, \ldots, \kappa\}$. Assume that the arms are independent and that the associated rewards with each arm are independently and identically distributed. At each time step (t), the player selects an arm (j) and obtains a reward $r_t = 1$ with probability p_j or $r = 0$ otherwise. At any stage of the procedure, the performance of the MAB algorithm is estimated either by calculating the cumulative reward at the current stage or by employing a regret measure, i.e., the difference in performance between the current and the best arm. As such, the main objective of the selection algorithm at any stage is to either maximize the cumulative reward of the procedure or minimize its regret. In such a context, an "optimal" algorithm will have the best possible performance if at each time step it is able to select the best (unknown) arm, i.e., the arm with the maximum probability of obtaining a reward.

The upper confidence-bound (UCB) algorithm is implemented to solve the MAB problem [117, 129, 134]. It is worth mentioning that UCB achieves an optimal regret rate on the aforementioned formulation. Specifically, let $q_{a_i}(t)$ be an estimation of the empirical quality of action a_i on time step t, and let c_i be a confidence interval associated with the empirical quality of action a_i that depends on the amount of times $n_i(t)$ action i has been tried until the current step t. Then, at each time step, UCB deterministically selects the next action ($selected_{llh}$) with the optimal upper bound of the confidence interval c_i. The selection process can be calculated according to

$$selected_{llh} = \arg \max_{i=1,2,\ldots,\alpha} \left(q_{a_i}(t) + \mathbf{C} \cdot \sqrt{\frac{2 \log \sum_k n_k(t)}{n_i(t)}} \right). \tag{6}$$

$$n_i\,(t+1) = n_i(t) + 1 \tag{7}$$

$$q_{a_i}\,(t+1) = \frac{(n_i(t)-1) \cdot q_{a_i}(t) + r_{a_i}(t)}{n_i(t)} \tag{8}$$

where $\mathbf{C}$ is a user-defined scaling factor that balance the trade-off between exploitation and exploration ability of the model (here $\mathbf{C} = 0.8$). Intuitively, the first term of Eq. 6 favors the action with the best empirical quality (exploitation), while the second term benefits the trial of the other actions (exploration). UCB thus selects mostly the action that potentially provides the best reward, while giving the opportunity for infrequently tried actions to be applied regularly.

An analysis of the behavior and adaptability of the applied models should provide useful insights into the behavior of each model and its impact on the respective algorithm that incorporates it. However, this issue is beyond the scope of this tutorial chapter.

Computational Results

This section demonstrates a thorough analysis of the performance of the developed hyper-heuristics by evaluating them on a wide range of problem domains with different characteristics. The experimental protocol used in the experiments is firstly defined, and then two qualitative different analyses are presented. The first study provides comprehensive experimental results and statistical analyses of the developed hyper-heuristics on each of the considered problem domains separately, while the second study follows the experimental protocol used in the CHeSC 2011 [1] competition and compares the developed hyper-heuristics with the state-of-the-art approaches of the competition.

Experimental Protocol

Five different hyper-heuristic approaches have been developed that are based on the HHILS framework described previously in section "Iterated Local Search with Adaptive Heuristic Selection". The difference between the hyper-heuristics mostly lies in the action selection mechanism used to select among the available low-level perturbation heuristics. As such, comparisons between the following five hyper-heuristics are conducted:

- HHILS: The simplest HHILS variant that selects the available perturbation low-level heuristics randomly following a uniform distribution.
- HHILS-SA: An HHILS variant that proportionally selects which perturbation low-level heuristic to apply, based on their reward value.
- HHILS-PM: An HHILS variant that utilizes the probability matching selection method to select the perturbation low-level heuristics.
- HHILS-AP: An HHILS variant that utilizes the adaptive pursuit selection method to select the perturbation low-level heuristics.
- HHILS-MAB: An HHILS variant that utilizes the upper confidence-bound multi-armed bandit methodology to select the perturbation low-level heuristics.

All HHILS variants have been implemented in the Java programming language, based on the functionality provided by the HyFlex framework. Although their implementation details are specific for the HyFlex framework, their algorithmic design is general and can be easily adapted and developed in any other framework. To motivate the usage and further development of the HHILS variants, their source code is available at https://github.com/mikeagn/hhils.

To maintain a fair and reliable comparison, the same parameter configuration is used for the common parameters of all algorithms. In addition, for the different action selection models that have been incorporated in the HHILS variants, the default parameter settings are used as proposed in the literature. Specifically, all HHILS variants employ a temperature value of $T = 2.0$, while the action selection models employ the default values used in the literature as described in the previous section. Notice that it has not been performed any fine-tuning process to obtain high-quality parameter configurations. However, such parameter configuration values can be found by performing either manual or automatic tuning and sensitivity analyses. This tuning process might lead to superior performance gains; nevertheless, this is out of the scope of the current tutorial demonstration. Prominent examples of successful off-line automatic tuning tools among others are the `irace` [135], the `SPOT` [136], and the `SMAC` [137] tools.

Six different problem domains have been implemented within the HyFlex framework, namely, the Max-SAT (SAT), the bin packing (BP), the personnel scheduling (PS), the Flow Shop (FS), the traveling salesman problem (TSP), and the vehicle routing with time windows problem (VRP). For each problem domain, the HyFlex framework provides 10–12 problem instances. A problem instance corresponds to either a real-world realization or a well-known and challenging benchmark case of the considered problem domain. More information about the available instances can be found online in [138]. To evaluate the performance of an algorithm on a given problem instance, the best objective value achieved by the algorithm within a prespecified available time budget is used. Problem domains have been modeled as minimization problems. As such, the lower objective value indicates a better performance. The first part of the experimental analyses provides extensive experimental results for all available instances per problem domain.

For fair comparisons, a benchmarking program is provided on the CHeSC 2011 website [111]. This determines the allowed time limit ($t_{allowed}$) of each run under the

specific hardware architecture $t_{allowed}$ which corresponds to 10 min of CPU time on the machine used during the competition. All experiments in this study have been conducted on an AMD Opteron CPU of 2.2 GHz machine with 32 GB of RAM running GNU/Linux operating system. For this machine, the allowed time limit is 624 s ($t_{allowed} = 624$ s).

Finally, the second part of the analyses strictly follows the rules of the CHeSC 2011 competition in order to provide an easy and fair comparison between all hyper-heuristic search methodologies. A thorough description of the experimental setup used in section "Comparison to HyFlex State-of-the-Art Hyper-heuristics" can be found in [110].

Experimental Results

In this section, the HHILS variants are evaluated on the six available problem domains provided by the HyFlex framework. Specifically, the performances of the five developed HHILS variants are compared on all available instances per problem domain. Firstly the performance gains obtained by each algorithm are described separately, for each problem domain. Then, their overall performance is summarized, and the observed behavior is verified by statistical analysis.

Tables 1, 2, 3, 4, 5, and 6 show statistics on the performance of each algorithm per problem domain, in terms of the best objective value obtained over 31 independent runs. For each algorithm and each problem instance (I), the following statistics are provided: the best (min), the median (m), the mean objective value (μ), and the standard deviation (σ) from the mean objective value obtained by the algorithm at hand. The cases where either the best, the median, or the mean objective value is the smallest (i.e., best performance) across all algorithms for a problem instance in hand are highlighted with **boldface**. In general, it can be observed that, although each algorithm behaves differently across the various problem domains, some algorithms exhibit robust performance across the majority of the domains, such as the HHILS-AP and the HHILS-SA.

More specifically, it can be easily observed that HHILS-AP shows superior performance against the other algorithms in three different domains, BP, FS, and TSP. As presented in Tables 1, 2, and 5, for at least 60% of the problem instances, HHILS-AP shows superior mean and median performance, while it is able to find the best objective value against the other algorithms in 6, 8, and 7 instances for the BP, FS, and TSP, respectively. Regarding the VRP problem domain, although there are three instances where the observed median and mean value are the smallest, in the majority of the remaining cases, it is not able to show competitive performance against the first-ranking algorithms in the domain (HHILS-SA, HHILS-PM).

The next most promising hyper-heuristic is HHILS-SA, which is able to achieve best performance, in terms of both mean and median objective values, in three problem domains FS, PS, and VRP. HHILS-SA also demonstrates the best performance in several problem instances across the majority of problem domains, i.e., 11, 6,

Table 1 Statistics on the best objective value achieved by the developed HHILS variants on all problem instances of the bin packing domain

	HHILS				HHILS-AP				HHILS-MAB				HHILS-PM				HHILS-SA			
I.	min	m	μ	σ	min	m	μ	σ	min	m	μ	σ	min	m	μ	σ	min	m	μ	σ
i0	0.007	0.017	0.017	0.005	0.007	**0.007**	**0.007**	0.000	**0.006**	0.007	0.008	0.001	0.007	0.011	0.010	0.002	0.007	0.007	0.008	0.002
i1	0.012	0.017	0.017	0.004	**0.007**	**0.008**	**0.008**	0.000	0.007	0.008	0.008	0.001	0.008	0.008	0.009	0.002	0.007	0.008	0.008	0.001
i2	0.022	0.024	0.024	0.001	**0.021**	**0.022**	**0.022**	0.000	0.022	0.024	0.024	0.002	0.022	0.023	0.023	0.001	0.023	0.023	0.023	0.000
i3	0.023	0.026	0.025	0.001	**0.021**	**0.022**	**0.022**	0.001	0.024	0.026	0.026	0.002	0.023	0.024	0.024	0.000	0.023	0.024	0.024	0.000
i4	0.007	0.007	0.007	0.001	0.005	0.005	0.005	0.000	**0.000**	**0.005**	**0.005**	0.001	0.005	0.007	0.006	0.001	0.005	0.006	0.006	0.001
i5	0.004	0.009	0.008	0.002	0.004	0.004	0.004	0.000	**0.003**	**0.003**	**0.003**	0.000	0.004	0.004	0.005	0.002	0.004	0.004	0.004	0.001
i6	0.084	0.089	0.088	0.003	**0.011**	0.037	0.045	0.032	0.025	0.032	0.033	0.003	0.022	0.032	0.032	0.008	0.011	**0.016**	**0.018**	0.005
i7	0.107	0.112	0.112	0.003	**0.028**	0.088	0.068	0.029	0.061	0.076	0.074	0.007	0.043	0.069	0.070	0.012	0.033	**0.051**	**0.051**	0.013
i8	0.049	0.052	0.052	0.001	**0.039**	**0.047**	**0.046**	0.005	0.057	0.066	0.065	0.003	0.045	0.049	0.049	0.002	0.046	0.049	0.049	0.001
i9	0.009	0.014	0.013	0.002	0.008	0.010	0.010	0.001	0.016	0.019	0.019	0.001	**0.007**	0.009	0.010	0.001	0.008	**0.009**	**0.009**	0.001
i10	0.110	0.111	0.111	0.001	0.109	0.109	0.109	0.000	**0.108**	**0.108**	**0.108**	0.000	0.110	0.110	0.110	0.000	0.109	0.110	0.110	0.000
i11	0.029	0.031	0.031	0.002	0.017	0.020	0.020	0.002	0.033	0.037	0.037	0.002	**0.012**	**0.017**	**0.016**	0.002	0.018	0.023	0.023	0.002

Table 2 Statistics on the best objective value achieved by the developed HHILS variants on all problem instances of the flow-shop domain

	HHILS				HHILS-AP				HHILS-MAB				HHILS-PM				HHILS-SA			
I.	min	m	μ	σ	min	m	μ	σ	min	m	μ	σ	min	m	μ	σ	min	m	μ	σ
i0	6328.0	6358.0	6356.1	12.8	**6308.0**	**6333.0**	**6331.0**	7.4	6327.0	6346.0	6346.5	9.4	6319.0	6343.0	6340.6	7.5	6324.0	6345.0	6344.2	6.1
i1	6282.0	6304.0	6302.4	8.4	6263.0	6288.0	**6285.6**	9.4	**6260.0**	6297.0	6295.7	9.0	6270.0	6294.0	6293.6	9.6	6270.0	**6287.0**	6288.0	6.4
i2	6370.0	6386.0	6384.9	7.3	6352.0	**6366.0**	**6363.8**	5.9	6365.0	6382.0	6382.5	7.8	6352.0	6372.0	6370.0	7.2	**6349.0**	6368.0	6367.3	7.5
i3	6361.0	6369.0	6369.2	3.3	**6323.0**	**6352.0**	**6352.4**	10.9	6350.0	6369.0	6367.2	6.0	6337.0	6367.0	6365.1	6.3	6330.0	6359.0	6357.7	11.0
i4	6424.0	6451.0	6447.1	10.7	**6402.0**	**6417.0**	**6416.3**	6.9	6409.0	6444.0	6442.0	11.9	6408.0	6426.0	6426.7	10.7	6407.0	6425.0	6424.2	7.2
i5	10,501.0	10,525.0	10,525.6	9.9	**10,495.0**	**10,501.0**	**10,500.9**	3.6	10,517.0	10,531.0	10,529.5	5.9	10,501.0	10,516.0	10,516.5	8.0	10,497.0	10,509.0	10,508.2	6.0
i6	10,948.0	10,959.0	10,960.4	4.4	**10,923.0**	**10,944.0**	**10,943.6**	10.2	10,938.0	10,960.0	10,959.7	6.6	10,934.0	10,957.0	10,955.4	6.8	**10,923.0**	10,956.0	10,950.7	10.2
i7	26,290.0	26,402.0	26,394.8	36.3	**26,249.0**	**26,293.0**	**26,293.0**	20.7	26,339.0	26,407.0	26,408.3	26.4	26,291.0	26,343.0	26,343.5	23.9	26,266.0	26,324.0	26,322.8	27.7
i8	26,859.0	26,886.0	26,895.1	28.1	26,765.0	26,831.0	26,823.6	24.7	26,852.0	26,896.0	26,895.5	21.2	26,806.0	26,866.0	26,865.3	28.8	**26,761.0**	**26,815.0**	**26,812.2**	20.4
i9	26,663.0	26,731.0	26,725.3	27.9	**26,580.0**	**26,645.0**	**26,641.6**	28.3	26,667.0	26,712.0	26,717.9	25.1	26,642.0	26,704.0	26,703.0	23.8	26,616.0	26,682.0	26,679.5	23.2
i10	11,420.0	11,473.0	11,471.4	20.5	**11,398.0**	**11,432.0**	**11,430.2**	14.7	11,442.0	11,464.0	11,466.9	11.8	11,431.0	11,458.0	11,457.5	12.0	11,404.0	11,445.0	11,444.4	12.4
i11	26,629.0	26,690.0	26,694.2	27.3	26,567.0	26,640.0	26,632.6	31.9	26,611.0	26,700.0	26,694.2	27.2	26,621.0	26,678.0	26,675.3	26.4	**26,548.0**	**26,638.0**	**26,631.0**	28.1

Table 3 Statistics on the best objective value achieved by the developed HHILS variants on all problem instances of the personnel scheduling domain

	HHILS				HHILS-AP				HHILS-MAB				HHILS-PM				HHILS-SA			
I.	min	m	μ	σ	min	m	μ	σ	min	m	μ	σ	min	m	μ	σ	min	m	μ	σ
i0	**3299.0**	**3317.0**	3319.6	12.7	**3299.0**	3318.0	**3318.5**	11.7	3312.0	3335.0	3335.9	14.9	3301.0	3325.0	3325.3	13.7	3301.0	3322.0	3321.5	11.8
i1	**1948.0**	2172.0	2155.8	101.3	2022.0	2175.0	2184.2	85.9	1950.0	2210.0	2208.5	88.3	1967.0	2137.0	2136.9	89.3	1955.0	**2103.0**	**2118.0**	98.1
i2	325.0	365.0	364.5	18.3	**305.0**	360.0	359.7	19.9	335.0	380.0	377.3	21.1	335.0	360.0	360.5	13.5	325.0	**355.0**	**354.8**	16.1
i3	13.0	18.0	18.3	2.1	13.0	19.0	19.0	2.4	17.0	21.0	21.4	2.6	**12.0**	**17.0**	**18.1**	3.5	**12.0**	18.0	**18.1**	3.0
i4	**14.0**	20.0	20.0	2.4	15.0	**19.0**	**18.9**	2.2	20.0	24.0	24.4	3.1	**14.0**	20.0	20.2	3.1	15.0	20.0	20.1	2.4
i5	16.0	21.0	21.3	3.0	**12.0**	**18.0**	**18.5**	2.6	17.0	25.0	24.8	3.2	14.0	20.0	20.6	2.8	16.0	21.0	22.0	3.1
i6	1108.0	1132.0	1148.4	40.2	1112.0	1132.0	1149.8	37.9	1108.0	1219.0	1213.6	66.2	**1103.0**	**1124.0**	**1132.2**	29.8	1107.0	1126.0	1141.1	38.6
i7	2170.0	2219.0	2230.1	38.1	2183.0	2262.0	2251.6	48.4	2195.0	2270.0	2262.6	51.7	2178.0	2225.0	2231.6	47.0	**2166.0**	**2211.0**	**2220.0**	34.0
i8	3159.0	3263.0	3263.3	65.5	3157.0	3256.0	3276.3	81.8	3158.0	3292.0	3308.6	76.8	3156.0	3238.0	3227.4	51.7	**3138.0**	**3181.0**	**3207.0**	59.2
i9	9420.0	9902.0	10,006.1	344.6	9429.0	9910.0	10,079.2	516.9	**9407.0**	**9678.0**	**9671.2**	177.8	9672.0	9904.0	10,090.1	444.0	9422.0	9958.0	9999.1	394.9
i10	**1405.0**	1630.0	1613.7	100.3	1445.0	1580.0	1603.7	84.3	1450.0	1623.0	1639.3	97.1	1462.0	1610.0	1622.2	94.4	1435.0	**1565.0**	**1574.3**	75.0
i11	305.0	**335.0**	335.8	12.6	320.0	**335.0**	337.3	12.0	320.0	350.0	348.3	15.5	315.0	**335.0**	334.9	10.5	**300.0**	**335.0**	**334.4**	15.0

Table 4 Statistics on the best objective value achieved by the developed HHILS variants on all problem instances of the SAT domain

	HHILS				HHILS-AP				HHILS-MAB				HHILS-PM				HHILS-SA			
I.	min	m	μ	σ	min	m	μ	σ	min	m	μ	σ	min	m	μ	σ	min	m	μ	σ
i0	2.00	**2.00**	2.00	0.00	2.00	**2.00**	2.00	0.00	2.00	**2.00**	2.00	0.00	2.00	**2.00**	2.00	0.00	**1.00**	**2.00**	**1.97**	0.18
i1	19.00	21.00	20.97	1.70	18.00	20.00	20.29	1.47	**17.00**	**19.00**	**18.97**	0.84	18.00	21.00	21.23	1.80	18.00	20.00	19.97	1.14
i2	**14.00**	16.00	16.52	1.61	**14.00**	16.00	16.10	1.04	**14.00**	16.00	15.68	1.11	**14.00**	16.00	16.03	1.28	**14.00**	**15.00**	**15.48**	1.03
i3	**0.00**	2.00	1.90	0.79	**0.00**	2.00	1.58	0.56	**0.00**	**1.00**	1.55	0.81	**0.00**	2.00	1.61	0.88	**0.00**	**1.00**	**1.52**	0.77
i4	1.00	2.00	1.71	0.78	1.00	**1.00**	1.32	0.48	**0.00**	**1.00**	1.55	0.81	**0.00**	**1.00**	1.42	0.56	**0.00**	**1.00**	**1.03**	0.55
i5	**1.00**	3.00	2.84	0.64	**1.00**	3.00	2.23	0.88	**1.00**	3.00	2.29	0.86	**1.00**	3.00	2.65	1.60	**1.00**	**2.00**	**1.97**	0.87
i6	**5.00**	**5.00**	**5.00**	0.00	**5.00**	**5.00**	5.19	0.48	**5.00**	6.00	5.81	0.87	**5.00**	**5.00**	5.03	0.18	**5.00**	**5.00**	5.45	0.62
i7	**5.00**	**5.00**	**5.06**	0.25	**5.00**	6.00	5.52	0.51	**5.00**	**5.00**	5.55	0.62	**5.00**	**5.00**	5.32	0.48	**5.00**	6.00	5.61	0.56
i8	**5.00**	**7.00**	**6.16**	1.19	**5.00**	8.00	7.13	1.31	**5.00**	8.00	7.45	1.21	**5.00**	8.00	7.29	1.22	**5.00**	8.00	7.26	1.37
i9	**209.00**	211.00	210.13	1.06	**209.00**	211.00	210.10	1.01	**209.00**	**209.00**	**209.26**	0.68	**209.00**	**209.00**	209.77	0.99	**209.00**	**209.00**	209.90	1.01
i10	**1.00**	2.00	2.16	1.19	**1.00**	**1.00**	1.42	0.76	**1.00**	**1.00**	1.32	0.83	**1.00**	**1.00**	1.16	0.37	**1.00**	**1.00**	**1.03**	0.18
i11	**7.00**	**7.00**	**7.39**	0.50	**7.00**	**7.00**	7.42	0.56	**7.00**	8.00	7.65	0.66	**7.00**	8.00	7.68	0.70	**7.00**	8.00	7.94	0.73

Table 5 Statistics on the best objective value achieved by the developed HHILS variants on all problem instances of the TSP domain

	HHILS				HHILS-AP				HHILS-MAB				HHILS-PM				HHILS-SA			
I.	min	m	μ	σ	min	m	μ	σ	min	m	μ	σ	min	m	μ	σ	min	m	μ	σ
i0	48,273.9	48,392.2	48,398.9	58.6	**48,194.9**	**48,214.9**	**48,221.7**	28.7	48,194.9	48,225.7	48,224.4	19.6	48,209.7	48,298.5	48,291.2	30.6	48,209.7	48,270.2	48,261.7	27.3
i1	108,616.5	109,392.8	109,618.3	735.8	**107,225.3**	109,349.4	108,800.4	962.6	108,217.7	109,167.8	109,126.8	498.6	107,442.3	108,459.1	**108,616.0**	817.0	107,259.6	**108,446.1**	108,740.2	948.8
i2	6894.6	6923.7	6921.9	12.8	**6823.4**	**6844.5**	**6843.9**	10.7	6888.0	6915.9	6914.9	11.4	6863.2	6897.2	6896.6	14.2	6862.2	6895.6	6893.7	11.8
i3	42,546.5	42,709.8	42,716.7	85.5	**42,173.0**	**42,279.8**	**42,274.5**	60.8	42,348.6	42,614.6	42,608.2	81.4	42,467.0	42,589.7	42,576.6	63.1	42,375.1	42,512.2	42,500.6	62.1
i4	9009.4	9048.3	9047.0	15.4	**8922.5**	**8943.0**	**8943.8**	11.2	8958.2	8973.4	8975.9	11.5	8985.8	9016.0	9012.9	14.4	8963.5	9004.1	9004.2	16.4
i5	58,192.9	58,612.5	58,592.8	156.2	**57,294.8**	**57,560.0**	**57,551.4**	141.8	58,207.2	58,766.1	58,743.6	234.4	57,975.3	58,184.4	58,197.8	137.1	57,779.9	57,977.5	58,006.1	153.2
i6	55,088.6	58,971.0	58,648.4	1749.7	53,077.3	54,605.6	54,902.8	1189.3	53,779.3	**54,030.7**	**54,031.2**	111.1	53,096.6	56,898.8	56,873.8	1882.7	**52,857.0**	54,295.0	54,257.8	781.0
i7	68,711.3	69,749.9	69,723.7	474.9	**65,789.9**	**66,301.9**	**66,296.2**	233.7	68,339.4	68,560.9	68,593.5	158.3	67,049.2	67,786.3	67,892.4	443.9	66,088.5	67,098.6	67,047.9	399.6
i8	**21,090,984.4**	21,380,811.8	21,396,672.8	187,926.6	21,145,557.5	21,375,242.0	21,394,858.1	165,005.6	21,164,043.9	**21,284,114.6**	**21,276,739.0**	53,894.4	21,145,557.5	21,374,787.3	21,400,271.2	160,545.3	21,210,701.9	21,364,765.7	21,377,212.8	144,263.6
i9	672,053.3	676,463.7	676,895.2	2362.8	673,105.2	676,356.6	677,073.4	2506.1	673,688.4	675,793.9	675,873.1	1241.5	673,583.9	676,599.4	677,243.7	2427.2	**671,431.6**	**675,118.2**	**675,141.7**	1863.9

Table 6 Statistics on the best objective value achieved by the developed HHILS variants on all problem instances of the VRP domain

	HHILS				HHILS-AP				HHILS-MAB				HHILS-PM				HHILS-SA			
I.	min	m	μ	σ	min	m	μ	σ	min	m	μ	σ	min	m	μ	σ	min	m	μ	σ
i0	5130.6	5165.7	5164.1	13.5	5115.0	5156.9	5155.6	13.9	**4250.7**	5156.7	**5124.0**	162.7	5114.0	**5156.5**	5158.0	14.4	5130.6	5165.4	5163.4	10.6
i1	20,655.0	20,657.4	20,657.8	1.7	20,652.2	**20,654.0**	**20,654.5**	3.1	20,652.6	20,656.5	20,721.0	247.1	20,652.5	20,655.2	20,655.0	0.6	**20,652.2**	20,654.7	20,654.5	0.6
i2	12,290.7	12,323.8	12,451.4	349.5	**12,274.4**	**12,294.6**	**12,293.2**	15.6	12,305.4	13,343.0	13,242.6	404.9	12,282.3	12,303.7	12,305.0	16.0	12,281.3	12,307.1	12,306.7	12.8
i3	5348.1	5373.7	**5374.6**	10.0	5351.4	6250.1	5939.5	435.2	**5321.4**	6245.4	6158.2	268.2	5326.7	**5366.1**	5453.2	277.0	5341.9	5367.3	5481.2	310.8
i4	13,281.3	13,293.6	13,356.8	244.9	**13,272.0**	**13,286.9**	13,695.2	493.3	13,291.5	14,280.2	14,217.5	355.9	13,277.3	13,288.1	**13,352.1**	245.8	13,275.1	13,287.5	13,383.9	294.6
i5	197,926.3	20,4256.8	204,910.1	3616.1	142,488.6	145,038.2	**144,824.7**	1402.2	210,603.3	219,067.4	219,536.2	3525.1	143,961.3	148,171.3	148,519.8	3054.0	**142,479.0**	**144,184.5**	144,854.3	1676.7
i6	64,490.5	67,479.9	67,417.3	1335.8	61,316.4	64,272.3	64,115.7	1719.2	71,101.8	72,849.9	73,027.1	1233.4	60,121.0	62,336.5	62,360.7	988.9	**59,730.3**	**61,651.6**	**61,552.1**	951.9
i7	173,238.4	177,164.0	176,870.5	1251.6	157,870.2	158,386.5	158,516.4	579.8	184,077.7	188,024.2	187,791.9	1622.3	157,881.5	158,794.6	158,839.0	545.1	**157,770.8**	**158,339.2**	**158,343.0**	360.4
i8	165,077.3	171,211.4	171,229.5	2639.7	143,085.2	146,652.3	147,012.3	1496.2	188,337.3	196,181.7	196,299.1	3476.2	143,584.5	148,048.2	148,395.3	2280.6	**142,928.5**	**144,561.6**	**144,347.1**	1222.1
i9	159,486.8	162,101.1	162,205.1	1293.7	144,086.8	145,586.4	145,595.4	668.4	165,525.7	168,651.6	168,616.4	1493.4	144,244.0	146,098.4	146,209.1	916.1	**143,565.7**	**144,827.5**	**144,974.6**	862.5

4, 4, and 2 in SAT, VRP, FS, PS, and TSP, respectively. The remaining HHILS variants perform similarly for the majority of the considered domains. HHILS and HHILS-MAB do not exhibit observable performance differences for the majority of the studied cases, while HHILS-PM shows slight improvement gains against them for the BP, FS, and TSP domains (in terms of mean and median objective values). However, in general, the performance of HHILS-PM is more robust when compared with both HHILS and HHILS-MAB, since the mean objective values and their standard deviations are lower for the majority of the considered benchmarks.

It is worth noting that in the SAT problem domain, most of the HHILS variants behave equally well, since for the majority of the problem instances, they exhibit similar mean and median performance values. In addition, for most of the problem instances, they successfully reach the best objective value achieved in this study. Notice that observable performance gains have been shown mostly by HHILS-SA which reached the smallest (min) objective value for all problem instances.

To facilitate easy comparisons of the performance of each algorithm across all problem domains, instances, and independent runs, their objective values are normalized in a common range of values. As such, for a given problem instance, the obtained objective values of all algorithms defined on a range $O = [O_{\min}, O_{\max}]$ are transformed linearly to the normalized range $N = [N_{\min}, N_{\max}]$, where $O_{\min}$ and $O_{\max}$ are the smallest and the largest objective value observed by the considered algorithms, and $N_{\min}$ *and* $N_{\max}$ are the lower and upper bound of the new normalized range. Here, the normalized range $N = [0, 1]$ has been used to keep calculations simple. Strictly speaking, let $y \in O \subset \mathbb{R}$ be an objective value obtained by an algorithm for a given instance, and let $f: O \rightarrow N$ be a linear function that transforms its input according to $\xi = f(y) = (y - O_{\min})/(O_{\max} - O_{\min})$. As such, the performance of each algorithm now can be computed by the set $Y_{\text{Alg}} = \{\xi_1, \xi_2, \ldots, \xi_n\}$ that consists of the normalized objective values across all problem domains, problem instances, and independent execution runs, where n indicates the total number of sample values and is equal to the number of problem instances for each domain, times the number of independent runs, summed up over all problem domains. Notice that this normalization enables a summarizing comparison of the algorithms across all problem domains and instances. Intuitively, the smallest normalized objective values indicate better performance.

To graphically demonstrate the distributions of the performance for each algorithm per problem domain, a summarizing illustration is also provided in Fig. 1. This demonstrates box-plot graphs of the normalized performance for the developed hyper-heuristics across all domains. In each box-plot graph, the mean value of the underlying distribution is additionally marked with a diamond.

Notice that the previously described behavior is clearly captured by the summarizing box plots illustrated in Fig. 1. On the whole, it can be easily identified that HHILS-AP exhibits great performance gains in three problem domains (BP, FS, and TSP), while HHILS-SA in two problem domains (VRP and PS). The next best-performing algorithm seems to be HHILS-PM which exhibits competitive performance in BP, PS, SAT, and VRP. HHILS and HHILS-MAB behave quite similarly, and their performance gains cannot be distinguished by the graphical

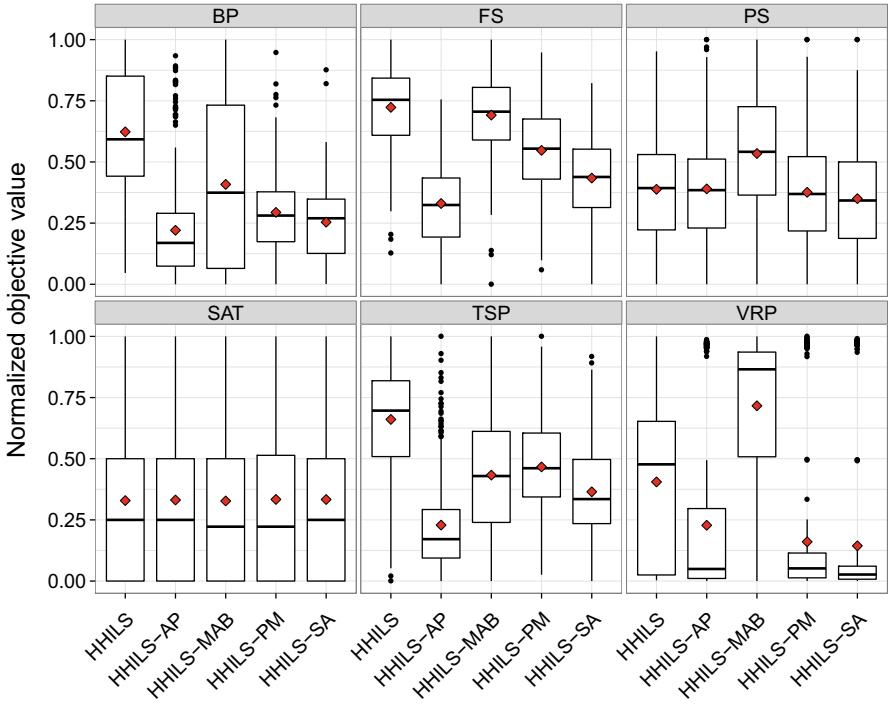


Fig. 1 Boxplot of the normalized objective values of all hyper-heuristics per problem domain

illustration. Interestingly, it can be additionally observed that the performance distribution of HHILS is the most robust against the remaining HHILS variants on the SAT domain.

To assess the existence of statistically significant differences, of the observed performance values, between at least two hyper-heuristics across all considered problem domains and instances, the Friedman rank sum test is applied [139]. For each algorithm, the distribution sample of its mean normalized performance per problem instance across all instances and all domains considered in this study is used. The null hypothesis of the Friedman test states that the distributions of all samples are the same, against the alternative hypothesis which states that at least two samples are not the same [139]. Given the existence of significant differences in performance, a post hoc analysis is employed to determine which two algorithms exhibit significant differences in performance. As such, pairwise comparisons are conducted, and for each comparison, the p-values (p_w) calculated by the nonparametric Wilcoxon-signed rank test are reported, since the underlying sample distributions do not follow a normal distribution (tested with the Shapiro-Wilk normality test [139]). Furthermore, to alleviate from the problem of having type I errors in multiple comparisons with a higher probability, the Bonferroni correction method is applied, and the adjusted p-values (p_{bonf}) are reported.

The Friedman test strongly suggests the existence of statistically significant differences in the performances between the studied hyper-heuristics ($p = 0.0000$, with $\chi^2 = 83.071$). Therefore, a post hoc statistical analysis is employed to determine which two algorithms significantly differ in performance. Table 7 presents summarizing statistics and p-values from pairwise statistical significance tests for all considered hyper-heuristics over all problem domains and instances. The left-hand side of Table 7 demonstrates the median (m_f), the mean (μ_f), and the standard deviation (σ_f) of the normalized objective values over all problem domains and instances, while the right-hand side presents the p-values obtained from the Wilcoxon-signed rank test (p_w) and the corresponding Bonferroni correction method (p_{bonf}). The presented statistics show that, on average, HHILS-SA and HHILS-AP are the most promising approaches. However, the statistical tests reveal that there are not statistically significant differences between their performances for the studied cases. Additionally, the pairwise comparisons indicate that HHILS and HHILS-MAB behave equally, while there exist statistically significant differences in performances for the remaining pairs of hyper-heuristics. This verifies the aforementioned reported performances observed in the analyses on each problem domain.

To further demonstrate the underlying performance distribution of the considered hyper-heuristics, Fig. 2 presents two graphical illustrations of their overall performances: a box-plot graph (left) and an empirical cumulative distribution function (ECDF) graph (right) on their normalized objective values over all the considered problem domains and instances. Such graphical illustrations are useful in displaying the characteristics of the distribution and the frequency of the observed performance values. Specifically, having measured a set of performance values (e.g., the normalized objective values) for an algorithm A, $M_A = \{\xi_1, \xi_2, \ldots, \xi_n\}$, the ECDF is defined as $F_n(\xi) = \frac{1}{n}\sum_{i=1}^{n} I_{(-\infty,\xi]}(\xi_i)$, where $I_{(-\infty,\xi]}(\cdot)$ is the indicator function which equals to one if $\xi_i \leq \xi$ and to zero otherwise. Intuitively, ECDF captures the empirical probability of observing a value that is less than or equal to ξ. The larger a ECDF value corresponding to a given value ξ, the higher is the empirical probability of observing ξ in Y_A.

The box-plot graph (on the left-hand side of Fig. 2) clearly illustrates the differences between the performance distributions of the considered algorithms. Based on the illustrated distributions as well as the previous mentioned results and statistical analyses, it can be safely concluded that HHILS-AP and HHILS-SA demonstrate the most promising performance across all domains. Although, from the analyses conducted per problem domain, HHILS-PM has not exhibited any superior performance gains, it can be observed that its average performance is very promising. Finally, the remaining two hyper-heuristics HHILS and HHILS-MAB perform on average quite similarly. Moreover, the ECDF graph interestingly reveals that almost all algorithms are able to reach best-performing results with quite similar frequency across all problem domains and instances. However, it has to be noticed that HHILS-AP, HHILS-SA, and HHILS-PM have more chance to locate a better solution for the problem under investigation. For example, HHILS-AP has a chance of more than 50% to achieve a quite competitive performance value

Table 7 Summarizing statistics of the normalized objective values of all hyper-heuristics over all problem domains and instances (left). Pairwise statistical significance tests of the normalized performances for all hyper-heuristics over all problem domains and instances (right)

Algorithm	m_f	μ_f	σ_f	HHILS		HHILS-AP		HHILS-MAB		HHILS-PM		
HHILS	0.542	0.521	0.293	p_w	p_{bonf}	p_w	p_{bonf}	p_w	p_{bonf}	p_w	p_{bonf}	
HHILS-AP	**0.222**	**0.292**	0.257	HHILS-AP	**0.000**	**0.000**	–	–	–	–	–	–
HHILS-MAB	0.533	0.515	0.309	HHILS-MAB	0.517	1.000	**0.000**	**0.000**	–	–	–	–
HHILS-PM	0.347	0.366	0.263	HHILS-PM	**0.000**	**0.000**	**0.001**	**0.005**	**0.000**	**0.001**	–	–
HHILS-SA	**0.297**	**0.317**	0.251	HHILS-SA	**0.000**	**0.000**	0.604	1.000	**0.000**	**0.000**	**0.000**	**0.000**

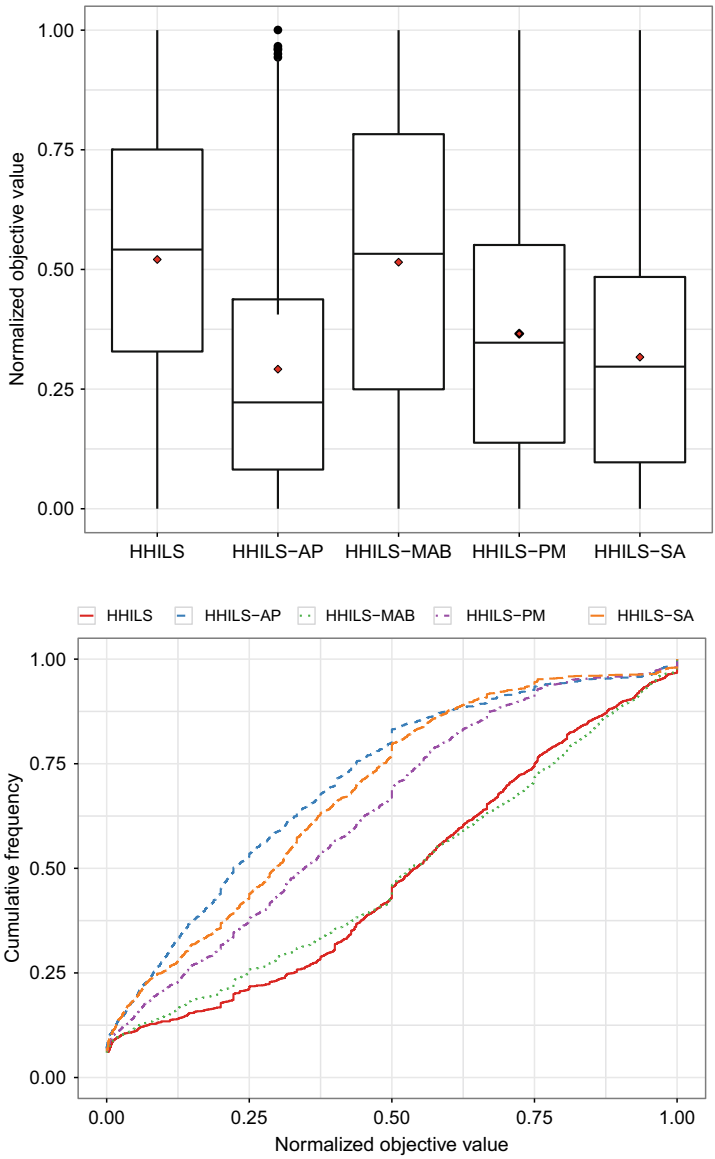


Fig. 2 Summarizing graphical illustrations: (Left) Box-plot graph, (Right) ECDF graph of the normalized objective values of all hyper-heuristics over all problem domains and instances

across all studied problem domains, i.e., there is a 50% chance that the obtained performance value will be in the first quartile of the observed performance values of all algorithms across all studied problem domains.

Comparison to HyFlex State-of-the-Art Hyper-heuristics

In this section, the performance of the HHILS variants is studied and compared against several state-of-the-art hyper-heuristics that participated in the CHeSC 2011 competition. To this end, the same experimental setup has been implemented as provided and described in the website of the CHeSC 2011 competition [111]. Specifically, five problem instances have been chosen (three tests and two hidden instances) for each of the six problem domains, resulting in total 30 different problem instances. Each hyper-heuristic is applied to solve the problem instance at hand for an allowed time period. To calculate the performance of each hyper-heuristic, 31 independent runs are conducted for each problem instance. The median objective value obtained out of these independent runs corresponds to the score of the hyper-heuristic for the specific problem instance at hand. As such, the competition entries are ranked based on their median objective value according to a Formula 1 pointing system. A thorough description of the methodology used for the CHeSC 2011 competition can be found in [110].

Table 8 lists the total and the per-problem domain ranking scores of the CHeSC 2011 competition entries along with the HHILS variants. Specifically, the rows of the table are sorted based on the total score (*total*) of each hyper-heuristic and rank them (*rank*) based on a descending order of their overall score. The remaining columns represent the obtained score of each hyper-heuristic across the different problem domains (for clarity, abbreviations SAT, BP, PS, FS, TSP, and VRP stand for Max-SAT, bin packing, personnel scheduling, flow shop, traveling salesman problem, and vehicle routing with time windows problem domain, respectively).

The scores in Table 8 suggest that the most competitive hyper-heuristics developed in this study, HHILS-SA and HHILS-AP, exhibit very competitive performance compared against CHeSC 2011 entries. HHILS-SA and HHILS-AP hold the second and third position, respectively, on the leaderboard. Notice that the score difference between HHILS-SA/HHILS-AP and the first ranked approach (AdaptHH) is small (9–11 units difference), while there is a large gap between them and the fourth entry (ML), (35–38 units difference). AdaptHH algorithm produces the best performance in two out of the six problem domains (BP and TSP), and it performs really well on the instances used for the FS domain. HHILS-SA exhibits superior performance in the PS and the VRP problem domains and high-performance gains in the SAT problem domain. Similarly, HHILS-AP shows superior performance in the SAT problem domain, while it shows quite competitive scores for the PS, the FS, and the VRP cases.

HHILS-PM is able to exhibit promising performance only in the SAT domain and high scores in the PS and VRP cases, while its contribution to the other domains is minimal. Notice that almost all HHILS variants show superior performance in the SAT domain, with HHILS-AP and HHILS-PM to share the first position in the ranking for this specific domain. In general, it can be observed that the HHILS

Table 8 Total and per problem domain ranking scores of the developed hyper-heuristics as competitors on the CHeSC 2011 competition

Rank	*Algorithm*	*Total*	SAT	BP	PS	FS	TSP	VRP
1	AdaptHH	137.25	8.50	**45.00**	3.50	33.00	**38.25**	9.00
2	HHILS-SA	128.35	31.85	12.00	**37.50**	11.00	4.00	**32.00**
3	HHILS-AP	125.95	**32.95**	8.00	25.50	21.50	10.00	28.00
4	ML	90.50	1.00	6.00	23.50	**35.00**	12.00	13.00
5	VNS-TW	86.75	11.00	2.00	24.50	31.00	16.25	2.00
6	HHILS-PM	78.45	**32.95**	3.00	24.50	0.00	0.00	18.00
7	EPH	68.75	0.00	6.00	3.00	16.50	34.25	9.00
8	PHUNTER	68.75	1.00	3.00	7.50	8.00	24.25	25.00
9	NAHH	63.60	6.60	19.00	0.00	22.00	12.00	4.00
10	HHILS	50.45	29.45	0.00	17.00	0.00	0.00	4.00
11	ISEA	48.00	0.00	28.00	11.00	0.00	9.00	0.00
12	HAEA	36.33	0.00	2.00	0.00	5.33	11.00	18.00
13	HHILS-MAB	35.10	23.10	8.00	0.00	0.00	3.00	1.00
14	ACO-HH	32.33	0.00	18.00	0.00	8.33	6.00	0.00
15	HAHA	19.33	7.00	0.00	7.00	0.33	0.00	5.00
16	KSATS-HH	19.00	3.00	8.00	0.00	0.00	0.00	8.00
17	DynILS	18.00	0.00	7.00	0.00	0.00	11.00	0.00
18	GenHive	16.00	0.00	9.00	0.00	3.00	1.00	3.00
19	XCJ	15.00	0.00	11.00	0.00	0.00	0.00	4.00
20	GISS	10.00	0.00	0.00	4.00	0.00	0.00	6.00
21	AVEG-Nep	9.60	6.60	0.00	0.00	0.00	0.00	3.00
22	SA-ILS	9.50	0.00	0.00	6.50	0.00	0.00	3.00
23	SelfSearch	3.00	0.00	0.00	0.00	0.00	3.00	0.00
24	Ant-Q	0.00	0.00	0.00	0.00	0.00	0.00	0.00
25	MCHH-S	0.00	0.00	0.00	0.00	0.00	0.00	0.00

variants were competitive entries for the SAT, PS, FS, and VRP domains. However, they were not successful in producing a competitive performance, against the other entries, in the BP and TSP cases.

It is worth noting that most of the CHeSC 2011 competitors have a very complex structure that demands a lot of time, effort, and experimentation in the development process, which is a natural process for participating in a competition. However, as suggested by the ranking scores, simple approaches, such as the tutorial demonstration hyper-heuristics in this chapter, might lead to quite efficient search methodologies with competitive performance gains. Notice that several approaches in the class of generation hyper-heuristics try to (semi-)automate the design algorithmic phase in order to alleviate the researcher from having to fine-tune the considered search methodology for a specific (or not) class of problems. Representative examples of automatic algorithmic design can be found in [2–4].

Conclusions

Hyper-heuristics range over a broad class of search methodologies. Many hyper-heuristic methods can be characterized by their general applicability and robustness across different application domains. The main feature of a hyper-heuristic is that it operates on a space of heuristics rather than on a solution space. Although there has been a significant level of research on hyper-heuristics during the last 15 years or so, several important and challenging research directions in the field are in their infancy and demand further exploration.

This chapter presents a timely and thorough literature review of the main advances in the field of hyper-heuristics since the publication of [4] in 2013. In addition, a simple hyper-heuristic framework has been developed and evaluated as a tutorial-style introduction to the field. The chapter has been divided into three parts. The first part reviews a wide spectrum of advances in the field of hyper-heuristics since 2012 and discusses the current trends and ideas that have been considered by the community. In particular, various studies have been identified that investigate the foundational ideas of the field, consider novel methodologies for automatic design of algorithms and parameter control, provide theoretical analysis, introduce novel hyper-heuristic methodologies and algorithms, develop hyper-heuristic frameworks in multi-objective and dynamic problem formulations, and successfully tackle real-world applications in various fields.

In the second and third parts, a simple but efficient selection hyper-heuristic framework is developed, as a tutorial-style introduction to the field, and evaluates its performance on six problem domains. The hyper-heuristic framework incorporates into the iterated local search algorithm an action selection model to adaptively choose the most promising perturbation heuristic based on feedback during the search process. Various state-of-the-art action selection models have been employed and evaluated. Experimental results and statistical analyses on six different problem domains verify its efficiency and performance. Comparisons with state-of-the-art hyper-heuristics in the field demonstrate its strength despite its relative simplicity.

The literature review highlights various interesting advances and limitations of the current state of the art, which can be summarized as follows.

The current formulation and classification of hyper-heuristics have stimulated the interest of the community. In particular, reformulations and extensions of the definition of hyper-heuristics have been proposed (see, e.g., [15]). A productive criticism on the level of the domain barrier revealed that both high and low levels of abstraction in the problem domain can enhance the search efficiency of hyper-heuristics. Problem domain information is sometimes able to strengthen the search dynamics of a hyper-heuristic without limiting its generality and robustness. Recent advances include unified representations as well as the embedding of mechanisms of common knowledge across different problem domains (see, e.g., [18, 19]).

From the methodological point of view, a variety of novel algorithms and frameworks have recently been published. Heuristic generation methodologies have been developed not only as hyper-visors to generate low-level heuristics but

also as mechanisms to generate high-level models [20]. Moreover, hyper-heuristic frameworks that are able to learn the feature space of a problem class and its representative search strategies have been proposed [14, 21–23]. These frameworks are capable of generating new search strategies and self-adapting their search characteristics based on new problem instances. Furthermore, various analyses have been published that study either the characteristics of the heuristic space, such as its diversity, or the feedback information that can be obtained through the search process (e.g., see [24–26, 28, 29]). It has also been identified that current developments in the field are mostly empirical, while the theoretical foundations and analyses of hyper-heuristics represent significant open research challenges. A few representative examples of recent theoretical work include the runtime analysis of hyper-heuristic algorithms [41, 42].

A comparison between the fields of meta-learning and hyper-heuristics has been performed that advocates their common objective of automating the algorithm design process [30]. Recent advances in the automatic design of algorithms include various simple, general, and efficient frameworks based on stochastic local search algorithms and novel gramma-based representations of heuristic algorithms among others (e.g., see [32, 35, 36]). Additionally, the commonalities in memetic computing and hyper-heuristics have been reviewed, and new frameworks that combine ideas from both fields have been presented [38]. Various other recent developments in the automated design of algorithms generate efficient algorithms for specific classes of problems, including (among others) packing and cutting problems [37, 39].

A significant number of real-world applications demand high-quality decisions. However, their formulations might include multiple objectives, uncertainty, or dynamically changing environments. Recent trends in the field promote research on both multi-objective and dynamic or uncertain problems. An effective multi-objective algorithm should possess good search characteristics in terms of both solution diversity and convergence on the Pareto-front set. Current developments include multi-objective hyper-heuristic approaches that operate on search strategies to efficiently guide search to diverse Pareto sets that converge to the Pareto front (e.g., see [43, 45, 46, 48–51]). Current methodologies mostly include selective mechanisms that combine the strengths of predefined search operators or algorithms, while heuristic generation approaches are in their infancy. Clearly, selection hyper-heuristics play a major role in this direction of research. However, the design of a multi-objective algorithm demands expert knowledge of the field, and it is a very demanding and time-consuming development process. Thus, automatic design processes that will efficiently address major issues will arguably have a tremendous impact on the multi-objective community. Hyper-heuristic methodologies for automatically designing and generating multi-objective algorithms are definitely a worthwhile (and growing) research direction.

Hyper-heuristic methodologies that employ adaptivity and cope with dynamically changing problem scenarios have been recently developed to address applications with uncertainty and dynamic changes in time. Most of the developed

approaches utilize heuristic selection mechanisms of specialized meta-heuristics that address dynamic problems across different types of dynamic environments [52–54]. Exploration of this area is only just beginning.

A major direction that draws upon the interdisciplinary scope of hyper-heuristics is the incorporation of machine learning methodologies to enhance hyper-heuristic efficiency and effectiveness. Machine learning methodologies have been incorporated mostly into the higher level of hyper-heuristics. The main aim of such methodologies is to learn the behavior of the underlying heuristics and be able to make effective decisions to guide the search toward the most promising areas of the search space. Recent advances include methodologies that consider tensor analysis, Inverse Reinforcement Learning theory, Monte Carlo Tree search, and grammatical evolution (among others, e.g., [55–62]). Moreover, hyper-heuristics and the automatic design of algorithms have influenced the machine learning community too. Various developed methodologies include the automatic design of classification algorithms, such as decision trees, to generate more general and robust methodologies for specific class of problems. Clearly, established methodologies in machine learning can potentially boost the strength of hyper-heuristic approaches. As foreseen in [3], it can be recognized that research at the interface of machine learning and hyper-heuristic methodologies has significant potential.

Hyper-heuristics have been successfully applied in a wide variety of diverse application areas. Complex applications that can be modeled as combinatorial optimization problems such as scheduling, timetabling, cutting, and packing problems represent the early adopted application areas of hyper-heuristic approaches. Recent developments not only continue to study these application domains but are also successfully applied to a wide variety of other applications, such as games, engineering, informatics, and parallel and/or distributed applications.

Hyper-heuristic research lies at the interface between operational research and computer science, and it has an impact on a broad spectrum of interdisciplinary application areas. The interdisciplinary character of the field promotes research that combines advanced knowledge from a variety of research areas with the common objective of successfully solving complex real-life optimization problems.

Cross-References

- Adaptive and Multi-level Metaheuristics
- Cutting and Packing
- Genetic Algorithms
- Iterated Local Search
- Matheuristics by Examples
- Memetic Algorithms
- Multiobjective Optimization
- Network Optimization
- Particle Swarm Optimization
- Restart Strategies

▸ Tabu Search
▸ Time Complexity Analysis of Stochastic Search Algorithms
▸ Variable Neighborhood Search

Acknowledgments Michael G. Epitropakis is supported by a grant from the Engineering and Physical Sciences Research Council (EPSRC Grant No. EP/J017515/1), by a Microsoft Azure Grant 2014, and by a Lancaster University Early Career Internal Grant (A100699). This support is gratefully acknowledged.

Competing Interest Declaration The author(s) has no competing interests to declare that are relevant to the content of this manuscript.

References

1. Burke EK, Kendall G, Newall J, Hart E, Ross P, Schulenburg S (2003) Hyper-heuristics: an emerging direction in modern search technology. In: Glover F, Kochenberger GA (eds) Handbook of metaheuristics. International series in operations research & management science, vol 57. Springer, Boston, pp 457–474
2. Burke EK, Hyde MR, Kendall G, Ochoa G, Özcan E, Woodward JR (2009) Exploring hyper-heuristic methodologies with genetic programming. In: Mumford CL, Jain LC (eds) Computational intelligence. Intelligent systems reference library, vol 1. Springer, Berlin/Heidelberg, pp 177–201
3. Burke EK, Hyde M, Kendall G, Ochoa G, Özcan E, Woodward JR (2010) A classification of hyper-heuristic approaches. In: Gendreau M, Potvin JY (eds) Handbook of metaheuristics. International series in operations research & management science, vol 146. Springer, Boston, pp 449–468
4. Burke EK, Gendreau M, Hyde M, Kendall G, Ochoa G, Özcan E, Qu R (2013) Hyper-heuristics: a survey of the state of the art. J Oper Res Soc 64(12):1695–1724. https://doi.org/10.1057/jors.2013.71
5. Chakhlevitch K, Cowling P (2008) Hyperheuristics: recent developments. In: Cotta C, Sevaux M, Sorensen K (eds) Adaptive and multilevel metaheuristics. Studies in computational intelligence, vol 136. Springer, Berlin/Heidelberg, pp 3–29
6. Pillay N (2014) A review of hyper-heuristics for educational timetabling. Ann Oper Res 239:3. https://doi.org/10.1007/s10479-014-1688-1
7. Ross P (2005) Hyper-heuristics. In: Burke EK, Kendall G (eds) Search methodologies, 1st edn. Springer, New York, pp 529–556
8. Ross P (2014) Hyper-heuristics. In: Burke EK, Kendall G (eds) Search methodologies, 2nd edn. Springer, New York, pp 611–638
9. Cowling P, Kendall G, Soubeiga E (2001) A hyperheuristic approach to scheduling a sales summit. In: Burke EK, Erben W (eds) Practice and theory of automated timetabling III. Lecture notes in computer science, vol 2079. Springer, Berlin/Heidelberg, pp 176–190
10. Crowston WBS (1963) Probabilistic and parametric learning combinations of local job shop scheduling rules. Carnegie Institute of Technology and Graduate School of Industrial Administration, Pittsburgh
11. Fisher H, Thompson GL (1963) Probabilistic learning combinations of local job-shop scheduling rules. In: Muth JF, Thompson GL (eds) Industrial scheduling. Prentice-Hall, Englewood Cliffs, pp 225–251
12. Hoos H, Stützle T (2004) Stochastic local search: foundations & applications. Morgan Kaufmann Publishers Inc., San Francisco, p 01275

13. Sabar NR, Ayob M, Kendall G, Qu R (2015) A dynamic multiarmed bandit-gene expression programming hyper-heuristic for combinatorial optimization problems. IEEE Trans Cybern 45(2):217–228. https://doi.org/10.1109/TCYB.2014.2323936
14. Sim K, Hart E, Paechter B (2014) A lifelong learning hyper-heuristic method for bin packing. Evol Comput 23:37. https://doi.org/10.1162/EVCO_a_00121
15. Swan J, Woodward J, Özcan E, Kendall G, Burke EK (2013) Searching the hyper-heuristic design space. Cogn Comput 6(1):66–73. https://doi.org/10.1007/s12559-013-9201-8, 00000
16. Ryser-Welch P, Miller JF (2014) Plug-and-play hyper-heuristics: an extended formulation. In: 2014 IEEE eighth international conference on self-adaptive and self-organizing systems (SASO), pp 179–180. https://doi.org/10.1109/SASO.2014.33, 00000
17. Poli R, Graff M (2009) There is a free lunch for hyper-heuristics, genetic programming and computer scientists. Springer, Berlin/Heidelberg, pp 195–207
18. Feng L, Ong Y, Lim M, Tsang I (2014) Memetic search with inter-domain learning: a realization between CVRP and CARP. IEEE Trans Evol Comput 19:644–658. https://doi.org/10.1109/TEVC.2014.2362558
19. Swiercz A, Burke EK, Cichenski M, Pawlak G, Petrovic S, Zurkowski T, Blazewicz J (2013) Unified encoding for hyper-heuristics with application to bioinformatics. CEJOR 22(3):567–589. https://doi.org/10.1007/s10100-013-0321-8, 00000
20. Sabar N, Ayob M, Kendall G, Qu R (2014) The automatic design of hyper-heuristic framework with gene expression programming for combinatorial optimization problems. IEEE Trans Evol Comput 99:309–325. https://doi.org/10.1109/TEVC.2014.2319051
21. Hart E, Sim K (2014) On the life-long learning capabilities of a NELLI*: a hyper-heuristic optimisation system. In: Bartz-Beielstein T, Branke J, Filipč B, Smith J (eds) Parallel problem solving from nature – PPSN XIII. Lecture notes in computer science, vol 8672. Springer International Publishing, Cham, pp 282–291
22. Sim K, Hart E (2014) An improved immune inspired hyper-heuristic for combinatorial optimisation problems. In: Proceedings of the 2014 conference on genetic and evolutionary computation (GECCO'14). ACM, New York, pp 121–128. https://doi.org/10.1145/2576768.2598241
23. Sim K, Hart E, Paechter B (2013) Learning to solve bin packing problems with an immune inspired hyper-heuristic. MIT Press, pp 856–863. https://doi.org/10.7551/978-0-262-31709-2-ch126
24. Grobler J, Engelbrecht AP, Kendall G, Yadavalli VSS (2015) Heuristic space diversity control for improved meta-hyper-heuristic performance. Inf Sci 300:49–62. https://doi.org/10.1016/j.ins.2014.11.012
25. Grobler J, Engelbrecht A, Kendall G, Yadavalli VSS (2014) The entity-to-algorithm allocation problem: extending the analysis. In: 2014 IEEE symposium on computational intelligence in ensemble learning (CIEL), pp 1–8. https://doi.org/10.1109/CIEL.2014.7015744
26. Salhi A, Rodríguez JAV (2013) Tailoring hyper-heuristics to specific instances of a scheduling problem using affinity and competence functions. Memetic Comput 6(2):77–84. https://doi.org/10.1007/s12293-013-0121-7, 00000
27. Ren Z, Jiang H, Xuan J, Hu Y, Luo Z (2014) New insights into diversification of hyper-heuristics. IEEE Trans Cybern 44(10):1747–1761. https://doi.org/10.1109/TCYB.2013.2294185, 00004
28. Soria Alcaraz JA, Ochoa G, Carpio M, Puga H (2014) Evolvability metrics in adaptive operator selection. In: Proceedings of the 2014 conference on genetic and evolutionary computation (GECCO'14). ACM, New York, pp 1327–1334. https://doi.org/10.1145/2576768.2598220, 00001
29. Consoli PA, Minku LL, Yao X (2014) Dynamic selection of evolutionary algorithm operators based on online learning and fitness landscape metrics. In: Dick G, Browne WN, Whigham P, Zhang M, Bui LT, Ishibuchi H, Jin Y, Li X, Shi Y, Singh P, Tan KC, Tang K (eds) Simulated evolution and learning. Lecture notes in computer science, vol 8886. Springer International Publishing, Cham, pp 359–370, 00000

30. Pappa GL, Ochoa G, Hyde MR, Freitas AA, Woodward J, Swan J (2013) Contrasting meta-learning and hyper-heuristic research: the role of evolutionary algorithms. Genet Program Evolvable Mach 15(1):3–35. https://doi.org/10.1007/s10710-013-9186-9, 00000
31. Adriaensen S, Brys T, Nowe A (2014) Designing reusable metaheuristic methods: a semi-automated approach. In: 2014 IEEE congress on evolutionary computation (CEC), pp 2969–2976. https://doi.org/10.1109/CEC.2014.6900575
32. Adriaensen S, Brys T, Nowé A (2014) Fair-share ILS: a simple state-of-the-art iterated local search hyperheuristic. In: Proceedings of the 2014 conference on genetic and evolutionary computation (GECCO'14). ACM, New York, pp 1303–1310. https://doi.org/10.1145/2576768.2598285
33. Ochoa G, Burke EK (2014) Hyperils: an effective iterated local search hyper-heuristic for combinatorial optimisation. In: 10th international conference on the practice and theory of automated timetabling (PATAT 2014)
34. Mascia F, López-Ibáñez M, Dubois-Lacoste J, Stützle T (2014) Grammar-based generation of stochastic local search heuristics through automatic algorithm configuration tools. Comput Oper Res 51:190–199. https://doi.org/10.1016/j.cor.2014.05.020
35. Marmion ME, Mascia F, López-Ibáñez M, Stützle T (2013) Automatic design of hybrid stochastic local search algorithms. In: Blesa MJ, Blum C, Festa P, Roli A, Sampels M (eds) Hybrid metaheuristics. Lecture notes in computer science, vol 7919. Springer, Berlin/Heidelberg, pp 144–158
36. López-Camacho E, Terashima-Marin H, Ross P, Ochoa G (2014) A unified hyper-heuristic framework for solving bin packing problems. Expert Syst Appl 41(15):6876–6889. https://doi.org/10.1016/j.eswa.2014.04.043, 00002
37. Thomas J, Chaudhari NS (2014) Design of efficient packing system using genetic algorithm based on hyper heuristic approach. Adv Eng Softw 73:45–52. https://doi.org/10.1016/j.advengsoft.2014.03.003, 00000
38. Epitropakis MG, Caraffini F, Neri F, Burke EK (2014) A Separability prototype for automatic memes with adaptive operator selection. In: 2014 IEEE symposium on foundations of computational intelligence (FOCI), pp 70–77. https://doi.org/10.1109/FOCI.2014.7007809
39. Segredo E, Segura C, León C (2013) Memetic algorithms and hyperheuristics applied to a multiobjectivised two-dimensional packing problem. J Glob Optim 58(4):769–794. https://doi.org/10.1007/s10898-013-0088-4, 00000
40. Soria-Alcaraz JA, Ochoa G, Swan J, Carpio M, Puga H, Burke EK (2014) Effective learning hyper-heuristics for the course timetabling problem. Eur J Oper Res 238(1):77–86. https://doi.org/10.1016/j.ejor.2014.03.046, 00006
41. Lehre PK, Özcan E (2013) A runtime analysis of simple hyper-heuristics: to mix or not to mix operators. In: Proceedings of the twelfth workshop on foundations of genetic algorithms XII (FOGA XII'13). ACM, New York, pp 97–104. https://doi.org/10.1145/2460239.2460249, 00008
42. Alanazi F, Lehre PK (2014) Runtime analysis of selection hyper-heuristics with classical learning mechanisms. In: 2014 IEEE congress on evolutionary computation (CEC), pp 2515–2523. https://doi.org/10.1109/CEC.2014.6900602, 00000
43. Maashi M, Kendall G, Özcan E (2015) Choice function based hyper-heuristics for multi-objective optimization. Appl Soft Comput 28:312–326. https://doi.org/10.1016/j.asoc.2014.12.012
44. Liao X, Li Q, Yang X, Zhang W, Li W (2007) Multiobjective optimization for crash safety design of vehicles using stepwise regression model. Struct Multidiscip Optim 35(6):561–569. https://doi.org/10.1007/s00158-007-0163-x
45. Maashi M, Özcan E, Kendall G (2014) A multi-objective hyper-heuristic based on choice function. Expert Syst Appl 41(9):4475–4493. https://doi.org/10.1016/j.eswa.2013.12.050, 00008
46. Yuen SY, Zhang X (2014) Multiobjective evolutionary algorithm portfolio: choosing suitable algorithm for multiobjective optimization problem. In: 2014 IEEE congress on evolutionary computation (CEC), pp 1967–1973. https://doi.org/10.1109/CEC.2014.6900470, 00000

47. Misir M, Lau HC (2014) Diversity-oriented bi-objective hyper-heuristics for patrol scheduling. In: 10th international conference on the practice and theory of automated timetabling (PATAT 2014)
48. Li K, Fialho A, Kwong S, Zhang Q (2014) Adaptive operator selection with bandits for a multiobjective evolutionary algorithm based on decomposition. IEEE Trans Evol Comput 18(1):114–130. https://doi.org/10.1109/TEVC.2013.2239648
49. Castro OR, Pozo A (2014) A MOPSO based on hyper-heuristic to optimize many-objective problems. In: 2014 IEEE symposium on swarm intelligence (SIS), pp 1–8. https://doi.org/10.1109/SIS.2014.7011803
50. McClymont K, Keedwell EC, Savić D, Randall-Smith M (2014) Automated construction of evolutionary algorithm operators for the bi-objective water distribution network design problem using a genetic programming based hyper-heuristic approach. J Hydroinf 16(2):302. https://doi.org/10.2166/hydro.2013.226, 00001
51. McClymont K, Keedwell E, Savic D (2015) An analysis of the interface between evolutionary algorithm operators and problem features for water resources problems. A case study in water distribution network design. Environ Model Softw 69:414. https://doi.org/10.1016/j.envsoft.2014.12.023
52. Topcuoglu HR, Ucar A, Altin L (2014) A hyper-heuristic based framework for dynamic optimization problems. Appl Soft Comput 19:236–251. https://doi.org/10.1016/j.asoc.2014.01.037, 00003
53. van der Stockt S, Engelbrecht AP (2014) Analysis of hyper-heuristic performance in different dynamic environments. In: 2014 IEEE symposium on computational intelligence in dynamic and uncertain environments (CIDUE), pp 1–8. https://doi.org/10.1109/CIDUE.2014.7007860
54. Akar E, Topcuoglu HR, Ermis M (2014) Hyper-heuristics for online UAV path planning under imperfect information. In: Esparcia-Alcázar AI, Mora AM (eds) Applications of evolutionary computation. Lecture notes in computer science. Springer, Berlin/Heidelberg, pp 741–752
55. Asta S, Özcan E (2014) A tensor-based approach to nurse rostering. In: 10th international conference on the practice and theory of automated timetabling (PATAT 2014), pp 442–445
56. Asta S, Özcan E (2015) A tensor-based selection hyper-heuristic for cross-domain heuristic search. Inf Sci 299:412–432. https://doi.org/10.1016/j.ins.2014.12.020
57. Asta S, Özcan E (2014) An apprenticeship learning hyper-heuristic for vehicle routing in HyFlex. In: 2014 IEEE symposium on evolving and autonomous learning systems (EALS), pp 65–72. https://doi.org/10.1109/EALS.2014.7009505
58. Asta S, Özcan E, Parkes AJ, Etaner-Uyar SA (2013) Generalizing hyper-heuristics via apprenticeship learning. In: Middendorf M, Blum C (eds) Evolutionary computation in combinatorial optimization. Lecture notes in computer science, vol 7832. Springer, Berlin/Heidelberg, pp 169–178
59. Sabar NR, Kendall G (2015) Population based Monte Carlo tree search hyper-heuristic for combinatorial optimization problems. Inf Sci 314:225. https://doi.org/10.1016/j.ins.2014.10.045
60. Barros RC, Basgalupp MP, Carvalho ACPLFd (2014) Investigating fitness functions for a hyper-heuristic evolutionary algorithm in the context of balanced and imbalanced data classification. Genet Program Evolvable Mach 1–41. https://doi.org/10.1007/s10710-014-9235-z
61. Basgalupp MP, Barros RC, Barabasz T (2014) A grammatical evolution based hyper-heuristic for the automatic design of split criteria. In: Proceedings of the 2014 conference on genetic and evolutionary computation (GECCO'14). ACM, New York, pp 1311–1318. https://doi.org/10.1145/2576768.2598327
62. Sá AGC, Pappa GL (2014) A hyper-heuristic evolutionary algorithm for learning Bayesian network classifiers. In: Bazzan ALC, Pichara K (eds) Advances in artificial intelligence – IBERAMIA 2014. Lecture notes in computer science. Springer International Publishing, Cham, pp 430–442
63. Karafotias G, Hoogendoorn M, Eiben A (2014) Parameter control in evolutionary algorithms: trends and challenges. IEEE Trans Evol Comput 19:167–187. https://doi.org/10.1109/TEVC.2014.2308294

64. Segredo E, Segura C, Leon C, Hart E (2014) A fuzzy logic controller applied to a diversity-based multi-objective evolutionary algorithm for single-objective optimisation. Soft Comput 19:2927. https://doi.org/10.1007/s00500-014-1454-y
65. Segredo E, Segura C, Leon C (2014) Control of numeric and symbolic parameters with a hybrid scheme based on fuzzy logic and hyper-heuristics. In: 2014 IEEE congress on evolutionary computation (CEC), pp 1890–1897. https://doi.org/10.1109/CEC.2014.6900538, 00000
66. Segredo E, Segura C, Leon C (2014) Fuzzy logic-controlled diversity-based multi-objective memetic algorithm applied to a frequency assignment problem. Eng Appl Artif Intell 30:199–212. https://doi.org/10.1016/j.engappai.2014.01.005
67. Jackson WG, Özcan E, John RI (2014) Fuzzy adaptive parameter control of a late acceptance hyper-heuristic. In: 2014 14th UK workshop on computational intelligence (UKCI), pp 1–8. https://doi.org/10.1109/UKCI.2014.6930167, 00000
68. Yarimcam A, Asta S, Özcan E, Parkes AJ (2014) Heuristic generation via parameter tuning for online bin packing. In: 2014 IEEE symposium on evolving and autonomous learning systems (EALS), pp 102–108. https://doi.org/10.1109/EALS.2014.7009510
69. Karafotias G, Eiben AE, Hoogendoorn M (2014) Generic parameter control with reinforcement learning. In: Proceedings of the 2014 conference on genetic and evolutionary computation (GECCO'14). ACM, New York, pp 1319–1326. https://doi.org/10.1145/2576768.2598360
70. Aleti A, Moser I (2013) Entropy-based adaptive range parameter control for evolutionary algorithms. In: Proceedings of the 15th annual conference on genetic and evolutionary computation (GECCO'13). ACM, New York, pp 1501–1508. https://doi.org/10.1145/2463372.2463560
71. Aleti A, Moser I (2013) Studying feedback mechanisms for adaptive parameter control in evolutionary algorithms. In: 2013 IEEE congress on evolutionary computation (CEC), pp 3117–3124. https://doi.org/10.1109/CEC.2013.6557950
72. Aleti A, Moser I, Meedeniya I, Grunske L (2013) Choosing the appropriate forecasting model for predictive parameter control. Evol Comput 22(2):319–349. https://doi.org/10.1162/EVCO_a_00113
73. Branke J, Hildebrandt T, Scholz-Reiter B (2014) Hyper-heuristic evolution of dispatching rules: a comparison of rule representations. Evol Comput 23:249. https://doi.org/10.1162/EVCO_a_00131
74. Hildebrandt T, Goswami D, Freitag M (2014) Large-scale simulation-based optimization of semiconductor dispatching rules. In: Proceedings of the 2014 winter simulation conference (WSC'14). IEEE Press, Piscataway, pp 2580–2590, 00000
75. Marshall RJ, Johnston M, Zhang M (2014) Developing a hyper-heuristic using grammatical evolution and the capacitated vehicle routing problem. In: Dick G, Browne WN, Whigham P, Zhang M, Bui LT, Ishibuchi H, Jin Y, Li X, Shi Y, Singh P, Tan KC, Tang K (eds) Simulated evolution and learning. Lecture notes in computer science, vol 8886. Springer International Publishing, Cham, pp 668–679
76. Marshall RJ, Johnston M, Zhang M (2014) Hyper-heuristics, grammatical evolution and the capacitated vehicle routing problem. In: Proceedings of the 2014 conference companion on genetic and evolutionary computation companion (GECCOComp'14). ACM, New York, pp 71–72. https://doi.org/10.1145/2598394.2598407
77. Marshall RJ, Johnston M, Zhang M (2014) A comparison between two evolutionary hyper-heuristics for combinatorial optimisation. In: Dick G, Browne WN, Whigham P, Zhang M, Bui LT, Ishibuchi H, Jin Y, Li X, Shi Y, Singh P, Tan KC, Tang K (eds) Simulated evolution and learning. Lecture notes in computer science, vol 8886. Springer International Publishing, Cham, pp 618–630
78. Park J, Nguyen S, Johnston M, Zhang M (2013) Evolving stochastic dispatching rules for order acceptance and scheduling via genetic programming. In: Cranefield S, Nayak A (eds) AI 2013: advances in artificial intelligence. Lecture notes in computer science, vol 8272. Springer International Publishing, Berlin, pp 478–489, 00001

79. Nguyen S, Zhang M, Johnston M, Tan KC (2014) Automatic design of scheduling policies for dynamic multi-objective job shop scheduling via cooperative coevolution genetic programming. IEEE Trans Evol Comput 18(2):193–208. https://doi.org/10.1109/TEVC.2013.2248159, 00013
80. Neamatian Monemi R, Danach K, Khalil W, Gelareh S, Lima FC Jr, Aloise DJ (2015) Solution methods for scheduling of heterogeneous parallel machines applied to the workover rig problem. Expert Syst Appl 42(9):4493–4505. https://doi.org/10.1016/j.eswa.2015.01.046
81. Misir M, Smet P, Vanden Berghe G (2014) An analysis of generalised heuristics for vehicle routing and personnel rostering problems. J Oper Res Soc 66:858. https://doi.org/10.1057/jors.2014.11
82. Anwar K, Awadallah M, Khader A, Al-betar M (2014) Hyper-heuristic approach for solving nurse rostering problem. In: 2014 IEEE symposium on computational intelligence in ensemble learning (CIEL), pp 1–6. https://doi.org/10.1109/CIEL.2014.7015743
83. Banerjea-Brodeur M (2013) Selection hyper-heuristics for healthcare scheduling. PhD thesis, University of Nottingham
84. Martin S, Ouelhadj D, Smet P, Vanden Berghe G, Özcan E (2013) Cooperative search for fair nurse rosters. Expert Syst Appl 40(16):6674–6683. https://doi.org/10.1016/j.eswa.2013.06.019
85. Koulinas G, Kotsikas L, Anagnostopoulos K (2014) A particle swarm optimization based hyper-heuristic algorithm for the classic resource constrained project scheduling problem. Inf Sci 277:680–693. https://doi.org/10.1016/j.ins.2014.02.155, 00008
86. Li D, Li M, Meng X, Tian Y (2015) A hyperheuristic approach for intercell scheduling with single processing machines and batch processing machines. IEEE Trans Syst Man Cybern Syst 45(2):315–325. https://doi.org/10.1109/TSMC.2014.2332443
87. Dong B, Jiao L, Wu J (2015) Graph-based hybrid hyper-heuristic channel scheduling algorithm in multicell networks. Trans Emerg Telecommun Tech 28:e2923. https://doi.org/10.1002/ett.2923
88. Tsai CW, Huang WC, Chiang MH, Chiang MC, Yang CS (2014) A hyper-heuristic scheduling algorithm for cloud. IEEE Trans Cloud Comput 2(2):236–250. https://doi.org/10.1109/TCC.2014.2315797, 00003
89. Aron R, Chana I, Abraham A (2015) A hyper-heuristic approach for resource provisioning-based scheduling in grid environment. J Supercomput:1–24. https://doi.org/10.1007/s11227-014-1373-9
90. Zheng YJ, Zhang MX, Ling HF, Chen SY (2015) Emergency railway transportation planning using a hyper-heuristic approach. IEEE Trans Intell Transp Syst 16(1):321–329. https://doi.org/10.1109/TITS.2014.2331239
91. Urra E, Cubillos C, Cabrera-Paniagua D (2015) A hyperheuristic for the dial-a-ride problem with time windows. Math Probl Eng 2015:e707056. https://doi.org/10.1155/2015/707056, 00000
92. Li S (2013) Hyper-heuristic cooperation based approach for bus driver scheduling. PhD thesis, Université de Technologie de Belfort-Montbeliard
93. Qu R, Pham N, Bai R, Kendall G (2014) Hybridising heuristics within an estimation distribution algorithm for examination timetabling. Appl Intell 42:679. https://doi.org/10.1007/s10489-014-0615-0
94. Burke EK, Qu R, Soghier A (2012) Adaptive selection of heuristics for improving exam timetables. Ann Oper Res 218(1):129–145. https://doi.org/10.1007/s10479-012-1140-3
95. Kheiri A, Özcan E, Parkes AJ (2014) A stochastic local search algorithm with adaptive acceptance for high-school timetabling. Ann Oper Res 239:135. https://doi.org/10.1007/s10479-014-1660-0
96. Anwar K, Khader AT, Al-Betar MA, Awadallah MA (2014) Development on harmony search hyper-heuristic framework for examination timetabling problem. In: Tan Y, Shi Y, Coello CAC (eds) Advances in swarm intelligence. Lecture notes in computer science, vol 8795. Springer International Publishing, Cham, pp 87–95

97. Salcedo-Sanz S, Matías-Román JM, Jiménez-Fernández S, Portilla-Figueras A, Cuadra L (2013) An evolutionary-based hyper-heuristic approach for the Jawbreaker puzzle. Appl Intell 40(3):404–414. https://doi.org/10.1007/s10489-013-0470-4, 00000
98. Salcedo-Sanz S, Jiménez-Fernández S, Matías-Román JM, Portilla-Figueras JA (2014) An educational software tool to teach hyper-heuristics to engineering students based on the Bubble breaker puzzle. Comput Appl Eng Educ 23:277. https://doi.org/10.1002/cae.21597, 00000
99. Yin PY, Chuang KH, Hwang GJ (2014) Developing a context-aware ubiquitous learning system based on a hyper-heuristic approach by taking real-world constraints into account. Univ Access Inf Soc 15:315. https://doi.org/10.1007/s10209-014-0390-z
100. Koohestani B, Poli R (2014) Evolving an improved algorithm for envelope reduction using a hyper-heuristic approach. IEEE Trans Evol Comput 18(4):543–558. https://doi.org/10.1109/TEVC.2013.2281512, 00000
101. Kheiri A, Özcan E (2014) Constructing constrained-version of magic squares using selection hyper-heuristics. Comput J 57(3):469–479. https://doi.org/10.1093/comjnl/bxt130, 00001
102. Drake JH, Özcan E, Burke EK (2015) Modified choice function heuristic selection for the multidimensional knapsack problem. In: Sun H, Yang CY, Lin CW, Pan JS, Snasel V, Abraham A (eds) Genetic and evolutionary computing. Advances in intelligent systems and computing, vol 329. Springer International Publishing, Cham, pp 225–234
103. Boughaci D, Lassouaoui M (2014) Stochastic hyper-heuristic for the winner determination problem in combinatorial auctions. In: Proceedings of the 6th international conference on management of emergent digital EcoSystems (MEDES'14). ACM, New York, pp 11:62–11:66. https://doi.org/10.1145/2668260.2668268
104. Lassouaoui M, Boughaci D (2014) A choice function hyper-heuristic for the winner determination problem. In: Terrazas G, FEB O, Masegosa AD (eds) Nature inspired cooperative strategies for optimization (NICSO 2013). Studies in computational intelligence, vol 512. Springer International Publishing, Cham, pp 303–314
105. Xie J, Mei Y, Ernst AT, Li X, Song A (2014) A genetic programming-based hyper-heuristic approach for storage location assignment problem. In: 2014 IEEE congress on evolutionary computation (CEC), pp 3000–3007. https://doi.org/10.1109/CEC.2014.6900604
106. Allen J (2014) A framework for hyper-heuristic optimisation of conceptual aircraft structural designs. Doctoral, Durham University
107. Allen JG, Coates G, Trevelyan J (2013) A hyper-heuristic approach to aircraft structural design optimization. Struct Multidiscip Optim 48(4):807–819. https://doi.org/10.1007/s00158-013-0928-3, 00001
108. Güney IA, Küçük G, Özcan E (2013) Hyper-heuristics for performance optimization of simultaneous multithreaded processors. In: Gelenbe E, Lent R (eds) Information sciences and systems 2013. Lecture notes in electrical engineering, vol 264. Springer International Publishing, Cham, pp 97–106, 00001
109. Asta S, Özcan E, Parkes AJ (2013) Batched mode hyper-heuristics. In: Nicosia G, Pardalos P (eds) Learning and intelligent optimization. Lecture notes in computer science. Springer, Berlin/Heidelberg, pp 404–409
110. Ochoa G, Hyde M, Curtois T, Vazquez-Rodriguez JA, Walker J, Gendreau M, Kendall G, McCollum B, Parkes AJ, Petrovic S, Burke EK (2012) HyFlex: a benchmark framework for cross-domain heuristic search. In: Hao JK, Middendorf M (eds) Evolutionary computation in combinatorial optimization. Lecture notes in computer science, vol 7245. Springer, Berlin/Heidelberg, pp 136–147
111. (2011) CHeSC 2011: cross-domain heuristic search challenge. http://www.asap.cs.nott.ac.uk/external/chesc2011/. Accessed 25 Mar 2015
112. (2014) CHeSC 2014: the second cross-domain heuristic search challenge. http://www.hyflex.org/chesc2014/. Accessed 25 Mar 2015
113. (2014) HyFlex API: hyper-heuristics flexible framework API. http://www.hyflex.org/. Accessed 25 Mar 2015

114. Battiti R, Brunato M, Mascia F (2009) Reactive search and intelligent optimization. Operations research/computer science interfaces series, vol 45. Springer, Boston, 00000
115. Lourenço HR, Martin O, Stützle T (2003) Iterated local search, handbook of meta-heuristics. Springer, Berlin/Heidelberg
116. Battiti R, Protasi M (2001) Reactive local search for the maximum clique problem. Algorithmica 29(4):610–637
117. Fialho A (2010) Adaptive operator selection for optimization. PhD thesis, Université Paris-Sud XI, Orsay
118. Epitropakis MG, Tasoulis DK, Pavlidis NG, Plagianakos VP, Vrahatis MN (2012) Tracking differential evolution algorithms: an adaptive approach through multinomial distribution tracking with exponential forgetting. In: Maglogiannis I, Plagianakos V, Vlahavas I (eds) Artificial intelligence: theories and applications. Lecture notes in computer science, vol 7297. Springer, Berlin/Heidelberg, pp 214–222
119. Gong W, Fialho A, Cai Z (2010) Adaptive strategy selection in differential evolution. In: Proceedings of the 12th annual conference on genetic and evolutionary computation (GECCO'10). ACM, New York, pp 409–416
120. Ong YS, Keane AJ (2004) Meta-Lamarckian learning in memetic algorithms. IEEE Trans Evol Comput 8(2):99–110. https://doi.org/10.1109/TEVC.2003.819944, 00460
121. Epitropakis MG, Tasoulis DK, Pavlidis NG, Plagianakos VP, Vrahatis MN (2012) Tracking particle swarm optimizers: an adaptive approach through multinomial distribution tracking with exponential forgetting. In: 2012 IEEE congress on evolutionary computation (CEC), pp 1–8
122. Kirkpatrick S, Gelatt CD, Vecchi MP (1983) Optimization by simulated annealing. Science 220(4598):671–680. https://doi.org/10.1126/science.220.4598.671, 00003
123. Thierens D (2005) An adaptive pursuit strategy for allocating operator probabilities. In: Proceedings of the 7th annual conference on genetic and evolutionary computation (GECCO'05). ACM, New York, pp 1539–1546
124. Thierens D (2007) Adaptive strategies for operator allocation. In: Lobo F, Lima C, Michalewicz Z (eds) Parameter setting in evolutionary algorithms. Studies in computational intelligence, vol 54. Springer, Berlin/Heidelberg, pp 77–90, 00042
125. Eiben AE, Smit SK (2011) Parameter tuning for configuring and analyzing evolutionary algorithms. Swarm Evol Comput 1(1):19–31
126. Epitropakis MG, Plagianakos VP, Vrahatis MN (2009) Evolutionary adaptation of the differential evolution control parameters. In: IEEE congress on evolutionary computation (CEC'09), pp 1359–1366
127. Lobo F, Lima C, Michalewicz Z (eds) (2007) Parameter setting in evolutionary algorithms. Studies in computational intelligence, vol 54. Springer, Berlin/Heidelberg
128. Smit SK, Eiben AE (2009) Comparing parameter tuning methods for evolutionary algorithms. In: IEEE congress on evolutionary computation (CEC'09), pp 399–406
129. Fialho A, Costa LD, Schoenauer M, Sebag M (2010) Analyzing bandit-based adaptive operator selection mechanisms. Ann Math Artif Intell 60(1–2):25–64. https://doi.org/10.1007/s10472-010-9213-y, 00032
130. Sutton RS, Barto AG (1998) Introduction to reinforcement learning, 1st edn. MIT Press, Cambridge, 02767
131. Karafotias G, Hoogendoorn M, Eiben AE (2013) Why parameter control mechanisms should be benchmarked against random variation. In: 2013 IEEE congress on evolutionary computation (CEC), pp 349–355. https://doi.org/10.1109/CEC.2013.6557590
132. Karafotias G, Eiben E, Hoogendoorn M (2014) Generic parameter control with reinforcement learning. In: Genetic and evolutionary computation conference (GECCO'14), Vancouver, 12–16 July 2014, pp 1319–1326
133. Bäck T, Fogel DB, Michalewicz Z (eds) (1997) Handbook of evolutionary computation. Oxford University Press, New York
134. Auer P, Cesa-Bianchi N, Fischer P (2002) Finite-time analysis of the multiarmed bandit problem. Mach Learn 47(2–3):235–256. https://doi.org/10.1023/A:1013689704352, 01559

135. López-Ibáñez M, Dubois-Lacoste J, Stützle T, Birattari M (2011) The irace package, iterated race for automatic algorithm configuration. Technical report. TR/IRIDIA/2011-004, IRIDIA, Université Libre de Bruxelles
136. Bartz-Beielstein T, Lasarczyk C, Preuss M (2010) The sequential parameter optimization toolbox. In: Bartz-Beielstein T, Chiarandini M, Paquete L, Preuss M (eds) Experimental methods for the analysis of optimization algorithms. Springer, Berlin/Heidelberg, pp 337–362, 00031
137. Hutter F, Hoos HH, Leyton-Brown K (2011) Sequential model-based optimization for general algorithm configuration. In: Coello CAC (ed) Learning and intelligent optimization. Lecture notes in computer science, vol 6683. Springer, Berlin/Heidelberg, pp 507–523, 00149
138. (2011) HyFlex competition instance summary. http://www.asap.cs.nott.ac.uk/external/chesc2011/reports/CHeSCInstanceSummary.pdf. Accessed 25 Mar 2015
139. Hollander M, Wolfe DA, Chicken E (2013) Nonparametric statistical methods, 3rd edn. Wiley, Hoboken

Iterated Greedy

25

Thomas Stützle and Rubén Ruiz

Contents

Introduction . . . 746
Iterated Greedy . . . 748
  Greedy Construction Heuristics . . . 748
  Iterated Greedy Framework . . . 749
Some Simple Examples of Iterated Greedy Algorithms . . . 753
  TSP Example . . . 753
  SCP Example . . . 754
  PFSP Example . . . 755
Case Study: Iterated Greedy for Flow Shop Scheduling . . . 756
  Results of the Simple Iterated Greedy Without Local Search . . . 759
  Results of the Simple Iterated Greedy with a Local Search Step . . . 760
IG Applications: Historical Development . . . 761
Relationship to Other Approaches . . . 764
  Repeated (Greedy) Construction Algorithms . . . 764
  (Perturbative) Local Search Techniques . . . 765
  Tree Search Algorithms . . . 766
Applications . . . 767
  Iterated Greedy for Scheduling Problems . . . 767
  Iterated Greedy for Routing Problems . . . 768
  Iterated Greedy for Other Problems . . . 768
Conclusions . . . 769
Cross-References . . . 770
References . . . 771

T. Stützle (✉)
IRIDIA, Université Libre de Bruxelles (ULB), Brussels, Belgium

R. Ruiz
Universitat Politécnica de València, València, Spain

AMAZON, Amazon.com, Inc., Seattle, WA, USA

© Springer Nature Switzerland AG 2025

R. Martí et al. (eds.), *Handbook of Heuristics*,
https://doi.org/10.1007/978-3-032-00385-0_10

Abstract

Iterated greedy is a search method that iterates through applications of construction heuristics using the repeated execution of two main phases, the partial destruction of a complete candidate solution and a subsequent reconstruction of a complete candidate solution. Iterated greedy is based on a simple principle, and methods based on this principle have been proposed and published several times in the literature under different names such as simulated annealing, iterative flattening, ruin-and-recreate, large neighborhood search, and others. Despite its simplicity, iterated greedy has led to rather high-performing algorithms. In combination with other heuristic optimization techniques such as a local search, it has given place to state-of-the-art algorithms for various problems. This chapter reviews the main principles of iterated greedy algorithms, relates the basic technique to the various proposals based on this principle, discusses its relationship with other optimization techniques, and gives an overview of problems to which iterated greedy has been successfully applied.

Keywords

Stochastic local search · Metaheuristics · Iterated greedy · Greedy methods · Local search · Constructive search

Introduction

Many effective algorithms for $\mathcal{NP}$-hard combinatorial optimization problems rely on the efficient, repeated execution of some simple, underlying mechanisms. A common example is perturbative search methods that iterate over neighborhood searches or iteratively apply underlying iterative improvement procedures. Examples of such methods include simulated annealing, tabu search, iterated local search, memetic algorithms, or dynamic local search [50,60]. In fact, simulated annealing, tabu search, or dynamic local search at each step explores the neighborhood of a current solution, while iterated local search and memetic algorithms can be seen as iterating across repeated applications of improvement algorithms. Less frequently, stochastic local search (SLS) methods make use of the iterative application of solution construction algorithms. Examples for these latter methods are ant colony optimization (ACO) [30, 31], greedy randomized adaptive search procedures (GRASP) [37, 38], or the pilot and rollout method [13, 34].

In this chapter, we review another SLS method that relies on the iterative application of solution construction procedures: iterated greedy. The method builds a sequence of solutions by iterating through phases of (partial) solution destruction and subsequent reconstruction of a complete candidate solution. A first complete solution in this sequence is generated by some constructive method. Then the following three steps are iteratively executed. First, components of a complete candidate solution s are removed, resulting in a partial solution s_p. Second, starting

from s_p, a complete candidate solution s' is rebuilt. Third, an acceptance criterion decides whether to continue this process from s or s'.

Iterated greedy has a clear underlying principle, and it is generally applicable to any problem for which constructive methods can be conceived. As such, iterated greedy is clearly a general-purpose method. Iterated greedy is a rather simple method that needs typically only short development times, especially if already a constructive heuristic is available. Iterated greedy provides also a rather simple way of improving over the single application of a constructive method, and for various problems, very high-quality solutions are generated. Additionally, basic versions of iterated greedy do only incur few main parameters, and their impact on the search process is rather intuitive to understand. All these reasons make iterated greedy a desirable technique for developers of heuristic algorithms.

Given the simple underlying principle, it is maybe also not surprising that the method that we here call iterated greedy has been (re-)discovered and applied a large number of times under different names by different authors (including ourselves). Algorithms that rely to a significant extent on the same underlying principle have been given names such as simulated annealing [62], evolutionary heuristic [77], iterative flattening [22], ruin-and-recreate [117], iterative construction heuristic [107], large neighborhood search [118], or, as here, iterated greedy [60, 114]. We will review these different developments and other related procedures in the section "IG Applications: Historical Development."

Despite the possible confusion that may arise for the reader due to the different names and the, in part, different views on the method, we want to stress that the really important aspect is the principle that underlies all these algorithms. In fact, in all these proposals, a repeated usage of constructive methods is made that starts from some intermediate, partial candidate solutions. This principle is a generic one of a potential large utility, and it should be understood as one of the basic principles that can be used to develop optimization algorithms. This basic principle may also be only one of the principles that is used in the development of a hybrid optimization algorithm that combines elements from different techniques. For example, several algorithms that make use of the iterated greedy principle include also a local search phase that may improve the solutions generated by the constructive mechanisms [114].

The structure of the chapter is as follows. In the section "Iterated Greedy," we review the basic principles of iterated greedy algorithms. Next, in the section "Some Simple Examples of Iterated Greedy Algorithms," we give some concrete examples of iterated greedy algorithms. In the section "Case Study: Iterated Greedy for Flow Shop Scheduling," we give some results of an experimental study that discusses the main trade-offs in the design of an iterated greedy algorithm for the permutation flow shop problem. We then present other algorithms that make use of the same principle as iterated greedy in the section "IG Applications: Historical Development." The relationship of iterated greedy to some other methods is discussed in the section "Relationship to Other Approaches." References to some noteworthy applications of iterated greedy are given in the section "Applications," and we conclude in section "Conclusions."

Iterated Greedy

Greedy Construction Heuristics

Constructive algorithms build candidate solutions to optimization or decision problems step by step, starting from an empty solution. At each step, they add a solution component to the current partial solution and repeat these steps until a complete candidate solution is obtained. Commonly, constructive algorithms use a heuristic function that estimates for each solution component the benefit of including it into a partial candidate solution. A baseline construction algorithm is formed by so-called greedy (constructive) algorithms that at each step add a solution component for which the value of the heuristic function is the best (see also Fig. 1). If more than one solution component has the same best heuristic value, a tiebreaking criterion is used to decide which solution component is actually added; in the simplest case, this tiebreaking is done uniformly at random, but it also may be done by a secondary heuristic function.

Greedy construction heuristics are frequently used when tackling combinatorial optimization problems due to a number of reasons. First, greedy construction heuristics are rather fast, and at the same time they generate solutions that are usually much better than those generated uniformly at random or by a randomized but heuristically biased construction. Second, these algorithms are often used to seed (perturbative) local search methods such as iterative improvement algorithms, more sophisticated SLS methods such as tabu search and simulated annealing, or population-based methods such as memetic algorithms. In the latter case, typically some members of the population are generated by greedy constructive methods, while others may be randomly generated. Seeding perturbative local search methods with solutions from greedy construction algorithms can incur advantages such as improved quality of local optima, faster identification of local optima, and a better trade-off between computation times and solution quality, that is, better anytime behavior [140]. Third, sometimes, one can prove guarantees on the quality of the solutions that are generated in the worst case, leading to so-called approximation algorithms. Often, the best provable guarantees that can be obtained even for more complex SLS algorithms are the guarantees that directly stem from those of the initial greedy construction. Fourth, for various polynomially solvable problems, greedy algorithms are also guaranteed to generate optimal solutions, the Kruskal algorithm for minimum spanning trees being a well-known example. However, for $\mathcal{NP}$-hard problems, this is not the case. Finally, they build the basis for a

Fig. 1 Algorithmic outline of a greedy heuristic

```
procedure Greedy Constructive Heuristic:
    s = ∅
    while s is not a complete solution do
        choose a best rated solution component c
        s = s + c
    end while
end
```

number of other methods such as GRASP [37, 38], ACO [30], or squeaky wheel optimization [64].

One straightforward way to improve over the generation of a single greedy solution is in some cases the repeated application of a greedy heuristic to generate a variety of different candidate solutions and then to choose the best one. Obviously, repetition in this sense is only reasonable if in the construction process different solutions can be generated. For example, for the well-known nearest neighbor heuristic for the traveling salesperson problem (TSP), n distinct nearest neighbor tours may result (assuming no random tiebreaking is done); for each of the n possible cities that may be chosen (randomly) as the initial city for the solution construction, a different tour may result. However, in other cases where the greedy construction is fully deterministic, additional randomization of the construction process, as proposed in the semi-greedy heuristics [57] and in GRASP [37, 104], may be required to generate different solutions.

Repeated construction of solutions also has inherent disadvantages. Constructing a full solution is relatively time-consuming as especially the initial construction steps require a large amount of computation when compared to later construction steps. Furthermore, no information is taken from one solution construction to another one, and thus such a repeated construction does not exploit knowledge gained from previous solutions. A method that alleviates these problems and that allows to invest, in principle, arbitrary computing times to generate different solutions by constructive heuristics is iterated greedy.

Iterated Greedy Framework

The main principle of iterated greedy is to iterate over (greedy) construction methods by first generating a complete candidate solution and then cycling through a main loop that consists of two main steps. In the first step, some solution components are removed from the current complete candidate solution s to result in some intermediate partial candidate solution s_p. We call this the *destruction step*. In the next step, starting from s_p, a construction heuristic is used to generate a new complete solution s'. We call this step the *construction step*. An acceptance test then decides from which of the two solutions, s or s', the next destruction step applies. While in the simplest case the acceptance test may accept only improved solutions, other choices may lead to more search diversification and, thus, to possibly better results when many iterations of the iterated greedy algorithm are done.

An algorithmic outline of an IG algorithm is given in Fig. 2. It starts by first generating an initial candidate solution using a procedure GenerateInitialSolution and then iterates through a main loop that consists of the application of the three procedures Destruction, Construction, and AcceptanceCriterion. Note that the construction procedures used in GenerateInitialSolution and Construction may be different and, hence, may also use different greedy heuristics. In the simplest case, however, they may be the same procedures.

Fig. 2 Algorithmic outline of iterated greedy (IG)

```
procedure Iterated Greedy
    s_0 = GenerateInitialSolution
    repeat
        s_p = Destruction(s*)
        s' = Construction(s_p)
        s* = AcceptanceCriterion(s*, s')
    until termination condition met
end
```

A simple default version of an iterated greedy algorithm could use the following choices. As constructive heuristic, one may take one that is either already available or implement a known state-of-the-art constructive heuristic. The solution destruction may delete some randomly chosen solution components, where the number d of solution components to be removed is a parameter of the algorithm. As acceptance criterion, one may force the cost to decrease by only accepting improved or equal quality candidate solutions.

The iterative process around construction heuristics gives IG some specific advantages when compared to the repeated construction of complete candidate solution from scratch. In fact, by starting from a partial solution, one may reduce significantly the time necessary to generate a new candidate solution as less constructive steps need to be done, and the time per construction decision is also reduced as the number of available solution components to choose from is smaller the larger the partial solutions. In addition, through the potential bias exerted by the acceptance criterion, the search process can more easily intensify around the best solutions found in the search process.

When trying to develop a more performing version of an iterated greedy algorithm, many different options for each of the specific operators may be taken into account. Some of the main relevant issues to be considered will be discussed in the following. Other ideas will be discussed also later when considering the relationship of iterated greedy to other methods or when discussing applications of iterated greedy algorithms.

Destruction There are a number of different possible choices for the solution destruction. A first consideration concerns the number of solution components that should be removed, as defined by a parameter d. The extreme settings would correspond to removing only a single component, that is, $d = 1$ or all of them. Even if one may argue that the resulting algorithms should be considered as iterated greedy algorithms, these parameter settings would not correspond to the main ideas underlying iterated greedy. The first case would be akin to a randomized local search while the latter to a repeated application of a construction heuristic. (We discuss these relationships in more depth in the section "Relationship to Other Approaches.") Intermediate values of d result in a trade-off between search intensification and diversification: removing a large number of solution components allows to jump to rather distant solutions in the construction phase, while removing few components leads to a more localized search.

Another choice is whether to leave the number of solution components to be removed fixed or variable during the algorithm run. In case d is left variable, a scheme of how to adapt its value is required. If the value of d is left fixed, it requires proper tuning. Possibilities for varying the value of d could be to modify this value randomly within some interval every few iterations, to choose the value according to a scheme similar as those introduced for variable neighborhood search [54], or to adapt the value of d at computation time, exploiting ideas of reactive search [11].

Once it is known how many components are to be removed, which ones should be chosen? There are a number of possible answers to this question. Intuitively, a randomized destruction is preferable over deterministic choices to reduce the danger of cycling. If a stochastic destruction is used, the simplest case is to choose components uniformly at random. More involved could be choices that take into account cost measures on the components or the partial solutions that remain, thus introducing a bias in the choice. In that case, solution components that have a strong contribution to the cost of a solution would then have a larger probability of being removed than low-cost components. Alternatively, one may use lower bounds on the partial solutions resulting after having removed a component: the smaller the lower bound, the higher the probability.

Construction One of the crucial ideas of IG is that restoring a fully specified solution is done by some form of (greedily biased) constructive mechanism. Hence, the usage of a constructive mechanism is essential. Using a random perturbation (corresponding to a move to a random solution in a large neighborhood) is more in the spirit of reduced variable neighborhood search than in the spirit of IG.

The naming iterated greedy suggests that the construction heuristics used in the algorithm are greedy construction heuristics and typically deterministic (modulo random tiebreaking). In fact, in many implementations of iterated greedy algorithms, this is also the case. However, we want to emphasize here that this need not necessarily be the case as, in principle, any suitable constructive mechanism that starting from some partial candidate solution s_p can generate a complete candidate solution—be it deterministically greedy, probabilistically greedy, or else—may be used as the underlying construction mechanism in an iterated greedy algorithm.

Among the construction rules, one may distinguish between *adaptive* heuristics and *static* ones. In the first case, the heuristic value assigned to a particular choice in the solution construction depends on the partial solution. Typically, adaptive construction heuristics will result in better-quality solutions than static ones; however, this improved solution quality is often reached at the cost of higher computation times.

In the simplest case, the solution construction is done following a deterministic construction heuristic—eterministic except of maybe random tiebreaking. Depending on the construction mechanism, it is possible to apply a deterministic rule: by applying stochastic destruction, either the order in which solution components are added is modified or, if adaptive construction heuristics are used, their heuristic evaluation.

Such basic considerations about solution construction in iterated greedy algorithms can be extended in many natural ways by adopting techniques that have been proposed in other methods. One may take inspiration from other constructive methods and use randomized selection in candidate lists built at each construction step as in GRASP algorithms, biased constructive decisions exploiting past experience such as the pheromone trails in ACO, or prohibitions by avoiding adding again solution components that have been removed in the solution destruction, taking ideas from tabu search. Another option may be to choose among different possible construction rules and use ideas from the hyperheuristics community for this task [19]. In fact, all kind of methods and techniques that are compatible with the idea of constructing solutions may be adopted within iterated greedy algorithms if this seems promising, making iterated greedy in this sense also a very flexible technique.

Acceptance criterion The acceptance criterion has a strong influence on the diversification/intensification behavior of an IG algorithm. On the extreme cases are the possibilities of accepting any new solution independent of its solution quality or to only accept solutions that improve over the previous one. There are many intermediate choices such as occasionally accepting worse solutions or allowing backtracks to previously seen solutions. A popular choice when accepting worse solutions is to use acceptance criteria from simulated annealing such as the Metropolis criterion: if a new solution is better or equal, it is accepted; otherwise, it is accepted with a probability $\exp\{(f(s) - f(s'))/T\}$, where T is a parameter called temperature. More elaborated approaches might probably make use of short-term memory as in tabu search. The acceptance criterion, whatever it might be, does not need to be applied at every iteration, i.e., a given incumbent solution can be destructed and reconstructed a number of times before deciding on its value. For the choice of the acceptance criterion, very much the same issues arise as in iterated local search [102], and an appropriate choice may be crucial to an IG algorithm's performance.

Hybridization with local search Iterated greedy algorithms can form directly the basis of hybrid algorithms that combine various search mechanisms. In fact, for constructive heuristics, a natural extension is to improve the generated solutions by the application of a (perturbative) local search method, in the simplest case this being an iterative improvement algorithm. Such an extension is also straightforward to be adopted in an iterated greedy algorithm and actually has been done in a number of such approaches. This extension results in an outline of iterated greedy as given in Fig. 3. With this additional local search phase, the IG algorithm also strongly resembles iterated local search (ILS) algorithms [102]. In fact, the destruction and construction phases implement a solution perturbation in the ILS sense. However, a minor difference is that ILS often makes use of perturbations that are randomly chosen from some large neighborhood, while in IG algorithms, an underlying constructive method is exploited. More importantly, IG can also reach very high performance when used without the local search phase, which is not necessarily

Fig. 3 Algorithmic outline of an IG with an additional local search step

procedure *Iterated Greedy with local search*
  $s_0 = \mathsf{GenerateInitialSolution}$
  $s^* = \mathsf{LocalSearch}(s_0)$
  repeat
    $s'_p = \mathsf{Destruction}(s^*)$
    $s' = \mathsf{Construction}(s'_p)$
    $s' = \mathsf{LocalSearch}(s')$
    $s^* = \mathsf{AcceptanceCriterion}(s^*, s')$
  until termination condition met
end

true for ILS algorithms. More information on relations of IG to other methods is given in the section "Relationship to Other Approaches."

In any case, for an algorithm to qualify as an iterated greedy algorithm, it is necessary that one can distinguish between a clearly available construction mechanism and a clear destruction mechanism that are repeatedly applied in an alternating order. An acceptance criterion may be used in an explicit way or in an implicit way; the latter is the case, for example, if every new solution is accepted and no mentioning of an acceptance criterion is done. In that case, the acceptance criterion would correspond to a "random walk-type" acceptance criterion used sometimes in ILS algorithms [102].

Some Simple Examples of Iterated Greedy Algorithms

Let us consider a few examples of constructive heuristics for basic combinatorial problems that will illustrate throughout the chapter various details of IG algorithms. We consider here three examples for the traveling salesman problem (TSP), the set covering problem (SCP), and the permutation flow shop problem (PFSP). We first shortly introduce the problem and then describe a basic constructive heuristic and basic IG algorithms.

TSP Example

Traveling salesman problem (TSP). The TSP is given a graph $G = (N, E)$ where N is the set of $n = |N|$ nodes and E is the set of edges that fully connects the nodes. To each edge (i, j) is associated with a distance d_{ij}. Here we assume that the distance matrix is symmetric, that is, we have $d_{ij} = d_{ji}$ for all $(i, j) \in E$; this type of TSP instances is called symmetric and among the most widely studied types of TSP instances. The objective in the TSP is to determine a Hamiltonian cycle of minimal length. Such a cycle can be represented by a permutation $\pi = \langle \pi(1), \ldots, \pi(n) \rangle$, where $\pi(i)$ is the node index at position i. The objective function to be minimized is

$$\min_{\pi \in \mathcal{S}} \; d_{\pi(n)\pi(1)} + \sum_{i=1}^{n-1} d_{\pi(i)\pi(i+1)} \tag{1}$$

where $\mathcal{S}$ is the search space consisting of the set of all permutations.

Probably the best known constructive heuristic for the TSP is the nearest neighbor heuristic. It chooses randomly a first node to start from to obtain a partial tour $\langle \pi(1) \rangle$. Then at each step, it appends to the partial tour $\langle \pi(1) \ldots \pi(l) \rangle$ a still unvisited node that has minimum distance to $\pi(l)$. Once all nodes are visited, the tour is completed by closing the tour and going back to $\pi(1)$. The nearest neighbor heuristic usually contains subtours that may be close to optimal ones but also contains long edges that are added often toward the end of the construction. Empirically, for Euclidean two-dimensional TSP instances, the nearest neighbor heuristic generates tours that are about 20–40% above optimal (see [63]).

One possibility to build an iterated greedy algorithm on top of the nearest neighbor heuristic would be to remove a subtour of d consecutive cities of a tour, resulting in one subtour of $n - d$ cities and to restart the nearest neighbor heuristic from it. Another possibility would be to remove randomly cities, reconnect the remaining subtours, and then restart the nearest neighbor heuristic. However, it is unclear what the performance of such an algorithm would be.

Another simple constructive heuristic for the TSP is the random insertion heuristic (RIH). It starts by choosing randomly two nodes that are ordered arbitrarily. At each construction step, a next node is chosen randomly and inserted into a position in the current path such that the cost increase is minimum. The RIH is one of simplest heuristics for the TSP; however, it is among the best performing constructive algorithms for the TSP [63].

IG can be used on top of the RIH as follows. The first solution is constructed by RIH. The procedure Destruct would remove l nodes that are chosen uniformly at random. Next, Construct adds the removed nodes using again the same rules as in the RIH. Finally, AcceptanceCriterion could be chosen to only accept improved solutions. This means that the solution destruction would be applied to the best tour found so far. Different choices for iterated greedy algorithms based on such steps have also been experimentally examined by [117].

SCP Example

Set covering problem (SCP). The set covering problem (SCP) is given a set $A = \{a_1, \ldots, a_n\}$ of items and a set $B = \{B_1, \ldots, B_m\}$ of subsets of elements of A that covers A; in other words, we have that for each $B_i \subseteq A$ and $\bigcup_{i=1}^{m} = A$. A set B_i covers an item a_j if $a_j \in B_i$. Each set B_i has a cost of c_i. The objective in the SCP is to find a subset C of the sets in B that covers each element in A and that is of minimal total cost.

For the SCP, constructive heuristics differ in the choice of the heuristic function that is used. A standard heuristic function is to compute the cover value of the sets in B, which is defined as $\gamma_i = c_i / b_i$, where b_i is the number of items that would be covered when adding subset B_i to a current partial cover. Note that when starting with an empty solution, that is, none of the subsets B_i is chosen to be in the cover C, we have that $b_i = |B_i|$. However, each time a subset is chosen, b_i needs to be updated, taking into account the items already covered in a partial cover C_p. The cover value is an example of an adaptive heuristic, where the heuristic values

depend on the partial solution already generated. Adaptive heuristics require higher computation times than their corresponding static heuristics, which do not update heuristic values in dependence of the partial solution, but typically lead also to better-quality solutions. Once a complete cover is obtained, some of the subsets may have become redundant if all the items they cover are also covered by other subsets; removing such redundant subsets then improves the solution cost of the generated cover. Note that the removal of redundant subsets is not yet a destruction step as through the removal of redundant subsets the candidate solution remains a complete cover. The destruction step then may remove a number of subsets from a complete solution, resulting in a partial cover C_p, from which again the construction heuristic starts.

A first IG algorithm for the SCP has been proposed by [62]. They construct the first solution using a simple greedy construction heuristic [9] that at each step first selects a random, not covered item and adds the least cost subset that covers that item. Once a complete cover is found, redundant columns are removed. The solution destruction removes $k_1 \cdot |C|$ subsets that are chosen uniformly at random; here $0 < k_1 < 1$ is a parameter, and $|C|$ is the number of subsets on the cover and the cover size. The solution construction uses the heuristic based on cover values explained above. (The iterated greedy algorithm by [62] is, hence, an example where the constructive heuristics for generating the initial solution and for extending partial solutions in the main loop differ.) However, the subsets to be considered are limited to a candidate set that comprises all those subsets that have a cost less than $k_2 \cdot \max\{c_i | i \in C\}$, where $k_2 > 0$ is a parameter that influences the size of the candidate set and C is the current cover. At each step of the construction, a subset with a minimum cover value is added to the current partial solution, breaking ties uniformly at random. Once a complete cover is obtained, redundant columns are removed as explained above. The acceptance criterion of a new cover C' is based on the Metropolis condition that is frequently used in simulated annealing algorithms. This simple IG algorithm for SCP obtained very high performance, improving over several earlier proposed SCP heuristics.

PFSP Example

Permutation flow shop scheduling problem (PFSP). In the PFSP, n jobs have to be scheduled on m machines. All jobs visit the machines in the same order, each job having an operation at each machine. Time p_{ij} denotes the non-negative, known, and deterministic processing time that job j needs on machine i. In the PFSP, the same processing sequence of the jobs is maintained throughout all machines, and hence the processing sequence is obtained as a permutation of the jobs. The standard objective is to minimize the completion time of the last job in this order, which is also known as makespan ($C_{\max}$). The PFSP arises in many practical situations as it is common to have production lines where machines are disposed in series. The PFSP is a thoroughly studied problem in the scheduling literature with literally hundreds of papers published each year. The minimization of the makespan is the most used criterion in the literature, but not so in practice [45]. Some reviews on the PFSP are [44, 59, 112].

A popular and high-performing constructive heuristic for the PFSP is the NEH heuristic [87], named after the initials of the last names of the paper's authors. NEH is an insertion heuristic, a kind of heuristics that iteratively extends a permutation by inserting at each step one new element into the current partial solution. The NEH heuristic first computes for each job j its sum of the processing time $P_j = \sum_{i=1}^{n} p_{ij}$ and then orders them according to non-increasing P_j values, resulting in a sequence $\phi(1) \ldots \phi(n)$. The first two jobs according to that order are taken, their two possible sequences $\langle \phi(1)\phi(2) \rangle$ and $\langle \phi(2)\phi(1) \rangle$ are evaluated, and the better of the two is adopted. Then at each step l, the job $\phi(l)$ is considered and tentatively inserted in all possible l positions in the current partial solution $\pi(1) \ldots \pi(l-1)$. Among these tentative insertions, the one resulting in the least increase of the makespan is taken. These steps are repeated until all jobs are inserted. Note that there might be two levels of ties, both at the ordering of the P_j values and at the insertion phase. The NEH heuristic has a computational complexity of $O(n^3m)$, which is lowered to $O(n^2m)$ using the efficient implementation of [120]. There is a rich literature of methods that propose variants of the NEH heuristic, mainly proposing mechanisms to break ties or reinsertions. As shown by [112] and more recently by [39], the NEH is a state-of-the-art constructive heuristic for the PFSP.

A simple IG algorithm for the PFSP has been proposed by [114]. It is based on insertion heuristics such as NEH. In fact, this iterated greedy algorithm uses NEH to construct an initial solution. At each destruction step, it removes a number of jobs that are chosen uniformly at random from the current permutation. In the construction step, these jobs are then reinserted in the same order in which they have been removed. For the insertion of the jobs, the same procedure as described for the NEH heuristic is followed. The acceptance criterion accepts a new candidate solution using the Metropolis condition known from simulated annealing algorithms but with the temperature T set to a constant value. The performance of this simple IG algorithm was very good, outperforming many metaheuristic algorithms for the PFSP. When combined with a local search phase, the proposed IG algorithm was shown to be a new state-of-the-art algorithm for the PFSP [114]. The IG implementation of [39] has given some further improvements of the above-presented iterated greedy algorithm. Currently, one top-performing iterated greedy algorithm for the PFSP is the variant of [33], which additionally adds a local search on the partial solution that is obtained after the destruction step. Other variants of the flow shop problem have been also studied in the papers [41, 42] with very good results and in the paper of Pagnozzi and Stützle [93], which applies automatic configuration to the problem and again obtains excellent results.

Case Study: Iterated Greedy for Flow Shop Scheduling

The development of an effective iterated greedy algorithm requires the algorithm designer to choose which of the various possible implementation choices to take. In this section, we exemplify the development of an iterated greedy algorithm for the PFSP and study the impact specific alternative choices have on iterated greedy

performance. Here we examine the impact of various alternative choices for the iterated greedy algorithm by [114], which we presented in the previous section.

In particular, we consider the following design choices in our experimental analysis.

Initial solution. It may seem advisable to start from an as good initial solution as possible. For the PFSP, this would be the NEH heuristic [39, 112] using the accelerations of [120]. However, at least for some instances, it is not clear whether a random or heuristic initialization of iterated greedy provides the best results. In fact, [101] study cases in which only for hard instances and in short CPU times it is advisable to use NEH (or extensions of the NEH) to obtain competitive results.

Destruction strength. The first and foremost decision to be taken is how many elements of a complete candidate solution should be removed. Here, we consider different fixed values for this parameter d, although it may also be interesting to consider schemes of how to vary the value of d at runtime.

Destruction type. The type of destruction determines which components of the incumbent solution are removed. In the simplest case, one may remove components of a solution uniformly at random. Alternatively, one may remove blocks of consecutive elements. In this case, a random block of d jobs could be removed from the sequence. Note that for the same solution π and a block-based destruction, there are only $n - d + 1$ possible choices for the destruction move. These two latter possibilities will be examined here.

However, one may consider a biased destruction, where components of the incumbent solution may be chosen randomly but in a biased way. For the PFSP, for example, jobs generating a large idle time at machines before or after their processing might be disrupting the sequence and are therefore likely to have been misplaced. We leave the study of such a destruction operator for future work.

Construction type. Similarly to the type of destruction move, different ways of how to construct a solution could be examined. The default choice in the iterated greedy algorithm is to insert jobs using the NEH heuristic. As an alternative, we consider a random insertion of the removed jobs, which could be used if no efficient greedy heuristic is available. However, in this case, the difference between iterated greedy and ILS becomes somewhat blurry, especially when additional local search is used.

Acceptance criterion. The acceptance criterion has a direct impact on the balance between intensification and diversification of the search. A simple idea is to accept only better-quality solutions, while alternatively, one may use the Metropolis condition that occasionally also accepts worse candidate solutions.

Acceptance iterations. Instead of applying the acceptance criterion at each iteration, one may apply an acceptance criterion only each l iterations. This would correspond to accepting for a few iterations every new candidate solution that is generated and only applying the acceptance test after each sequence of l steps.

Table 1 Summary of the factors and the levels studied in the experimental analysis

Factor	Abbreviation	Level 1	Level 2
Initial solution	*Initialization*	NEH	Random
Destruction strength	*Destruct*	4	6
Destruction type	*Destruction_T*	Random	Block
Construction	*Reconstruction_T*	NEH	Random
Acceptance criterion	*Acceptance_C*	SA	Descent
Acceptance iterations	*Iterations_Acc*	1	5
Local search	*LS*	No	Yes

Local search. Finally, it is well-known that local search can have a tremendous impact on the quality of the results achieved, even though iterated greedy algorithms may reach high-quality solutions even without local search. Hence, it may be worthwhile to test the impact of an additional local search phase.

In order to study the different design and implementation alternatives, we have carried out a design of experiments (DOE) approach [85] where the previous factors are analyzed. Seven factors are coded and tested at two levels each, as summarized in Table 1. As a result, a total of $2^7 = 128$ combinations are to be tested in a full factorial experimental design. However, we use a half fractional design, which has most of the power of a full factorial one but requires only half of the runs. We use a 2^{7-1}_{VII} design, which has a high-resolution VII: interactions between four factors are aliased (confounded) with interactions between three factors, interactions between five factors aliased with interactions between two factors, and so on. As a result, we can safely study the experimental data since it is highly unlikely that two high-level interactions might be significant. More elaborated techniques for the analysis and calibration of algorithms than the simple exploratory analysis we perform here have been published elsewhere [10]. Each studied combination has been tested on 30 of the hardest instances from Taillard's benchmark (122). This benchmark is composed of 12 groups of 10 instances, each ranging from 20 jobs and 5 machines to 500 jobs and 20 machines. The instances of one particular group are denoted as $n \times m$. For the tests, we pick three of the hardest groups, namely, 50×20, 100×20, and 200×20. We are interested in these instances since for most of the remaining problems, the optimum solution is already known, and today's state-of-the-art methods are capable of obtaining near optimum solutions.

Each combination is run five independent times (replicates) with each instance. We also control the number of jobs (n) as a blocking factor with three levels in the experiment. Therefore, the experimental design contains $2^{7-1} \cdot 3 = 192$ treatments, $192 \cdot 10 = 1920$ experimental units (for ten instances), and $1920 \cdot 5 = 9600$ experiments (for five replicates).

The basic iterated greedy algorithm along with all studied alternatives has been implemented in Delphi language and run during a predefined CPU time, which is fixed to 30, 60, and 120 s for the instances of 50, 100, and 200 jobs, respectively. The machine used for the tests is a Pentium IV PC/AT computer running at 3.2 GHz

with 2 GBytes of RAM memory. The performance measure and response variable of the experimental design are the so-called relative percentage deviation (*RPD*) over the optimum or best known solution (upper bound) for each tested instance:

$$\text{Relative percentage deviation } (RPD) = \frac{Heu_{\text{sol}} - Best_{\text{sol}}}{Best_{\text{sol}}} \times 100 \tag{2}$$

where Heu_{sol} is the solution given by any of the replicates of any iterated greedy variation for a given instance and $Best_{\text{sol}}$ is the optimum solution or the lowest known upper bound for that specific Taillard's instance as of November 2015. The results are analyzed by means of the parametric ANOVA technique. Notice that in this case, the three assumptions of this parametric test (normality, homoscedasticity, and independence of the residual) have to be satisfied. In our experiment, the factor *LS* results to be extremely significant, and it creates large differences in the response variable. As expected, applying the local search results in a statistically significant difference in the average *RPD* of 1.46 considering all results, compared with the average *RPD* of 3.73 without local search. Such a large difference generates normality problems. In order to avoid this situation, we study two separate ANOVAs, one for each level of the *LS* factor.

Results of the Simple Iterated Greedy Without Local Search

All remaining factors after fixing the local search have p-values very close to zero in the resulting ANOVA table. As a result, we focus on the F-Ratio, which is the ratio between the variance generated by a given factor and the residual variance in the studied two-level interaction linear model. The higher this ratio, the more significant the factor or interaction is. Figure 4 shows the means plot for the remaining six factors in order of importance.

From Fig. 4, we can observe how all six studied factors have levels and variants that result in statistically significant differences. Observed differences in the average performance of the iterated greedy when a factor has been set to a given level or

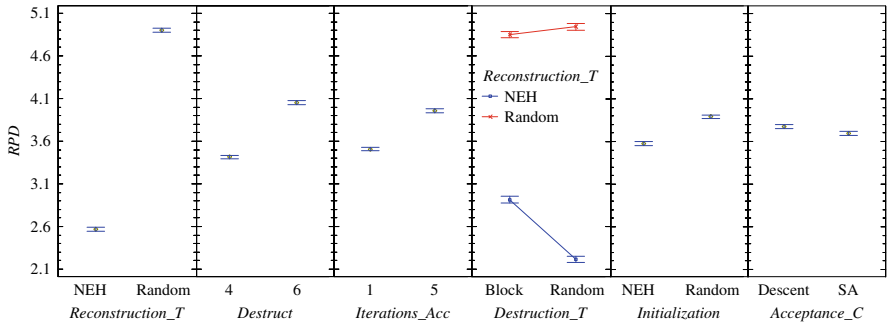


Fig. 4 Means plot of the average relative percentage deviation (*RPD*) and 99% Tukey HSD confidence intervals for the studied factors. Simple iterated greedy without local search

variant are depicted with non-overlapping confidence intervals in the plots. The importance of the F-Ratio is shown in Fig. 4 with the most significant factors to the left of the figure and the least significant to the right. As can be seen, without local search, it is clearly preferable to greedily reconstruct the solution as the difference in performance is extremely large. This is one of the main keys behind the iterated greedy algorithm. A random reconstruction of jobs means that first the jobs are randomly extracted and then randomly inserted. This is more or less a sequence of insertion moves. Too many moves (and no local search) result in an algorithm that is not performing well.

The second factor in importance is *Destruct* or d, the number of jobs to be removed in the destruction phase. Following the previous discussion, six jobs impose a rather large overall disruption, and removing only four jobs gives much better results. Notice that a similar conclusion was reached by [114] in their calibration of the iterated greedy for the PFSP. Although not shown here, removing four jobs is better for all the studied instances with 50, 100, and 200 jobs.

The third most important factor results to be *Iterations_Acc*. Among the two tested levels, applying the acceptance criterion at every iteration gives substantially better results. The analysis is continued until all factors have been fixed through the most important effects in the response variable. For example, the third most significant effect in the ANOVA is not a single factor but the interaction between factors *Destruction_T* and *Reconstruction_T*. It is shown in the fourth plot from the left in Fig. 4. We see that both factors interact greatly. When *Reconstruction_T* is set to random, the block destruction appears to be slightly better. However, when *Reconstruction_T* is set to NEH, as in the original NEH method, the destruction has to be done randomly as much better results are obtained. *Initialization* is best set at NEH, but we observe that the difference in performance with the random initialization is rather small. Finally, the acceptance criterion factor (*Acceptance_C*) is also slightly better at SA, but the difference with descent is very small, even though it is statistically significant. More specifically, the average *RPD* given by the simple iterated greedy across all factors with a SA-like acceptance criterion is 3.69%, whereas for the descent criterion, it is 3.77%. It is interesting to note that most results confirm the choices made in previous studies as in [114].

Results of the Simple Iterated Greedy with a Local Search Step

Now we proceed with the analysis of the experimental data after focusing on the results of the simple iterated greedy with the local search enabled. In this experiment, the first interesting result is that the observed differences as regards *RPD* among the levels of the factors are much smaller than in the previous one. It is safe to say that the local search is capturing much of the variability of the experiment. Figure 5 contains the means plot for the six factors, in order of importance according to their F-Ratios.

As can be seen, the relative importance of the studied factor is not the same when compared to the previous experiment without local search. The most important

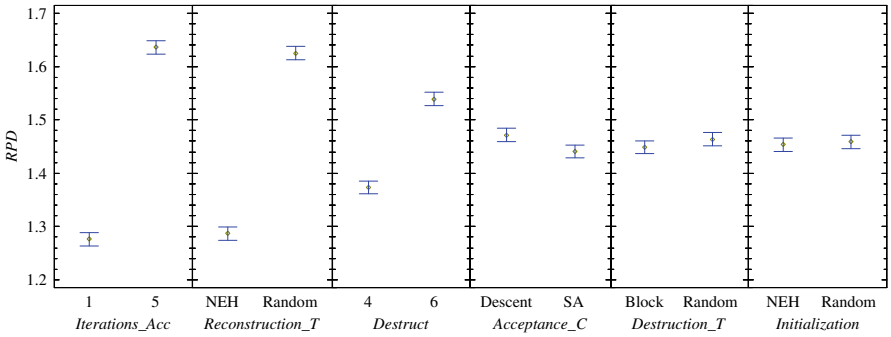


Fig. 5 Means plot of the average relative percentage deviation (*RPD*) and 99% Tukey HSD confidence intervals for the studied factors. Simple iterated greedy with local search enabled

factor is now *Iterations_Acc*. Similar to the previous experiment, applying the acceptance criterion at each iteration improves results significantly. The next factor in importance is *Reconstruction_T*. The plot is similar to the previous experiment and represents an important result. Considering that the algorithm already carries out a local search step after reconstruction, it could be argued that a random reconstruction might suffice. However, this is not the case. Carrying out a local search step from a randomly reconstructed solution yields, on average, much worse results than if the local search is carried out on a greedily (and, therefore, of higher quality) reconstructed solution. This supports the point that iterating over greedy heuristics gives very good results. The remaining factors per importance are *Destruct* and *Acceptance_C*. The last two remaining factors, *Destruction_T* and *Initialization*, are not statistically significant at a 99% confidence level as it can be seen by the fact that their corresponding means plots overlap. Hence, among the two tested types of destruction, it is not important how the jobs are removed. This is probably due to the fact that even though jobs are removed in blocks, they are reinserted one by one. As for the initialization, the advantage gained by an NEH initialization is nullified during the run of the iterated greedy algorithm, confirming the observation of [101].

As a conclusion from this study, the most significant factor is the local search and the NEH reconstruction. Most other factors have less importance. Iterated greedy can even do without a full NEH initialization for the hardest instances of Taillard in the PFSP problem.

IG Applications: Historical Development

Among the examples of iterated greedy algorithms in the section "Some Simple Examples of Iterated Greedy Algorithms," we had described an algorithm for the SCP by [62] who actually called their algorithm a *simulated annealing* algorithm. This naming is probably due to the use of the Metropolis condition as an acceptance

criterion, which was also used in the first proposals of simulated annealing [68]. This algorithm has later been extended by the same authors [18]. Somehow related is the more complex algorithm by Marchiori and Steenbeek [77] for the SCP; in this algorithm, a construction starts from a partial solution following the steps of the iterated greedy method; however, instead of obtaining the partial solution by choosing the subsets to be removed, they use a mechanism to choose the subsets of the best solution found so far that are maintained in the partial solution. Maybe surprisingly, in that paper, the proposed algorithm is viewed as an evolutionary algorithm. This algorithm is preceded by another algorithm for the unweighted SCP [76], which more directly follows the ideas of the iterated greedy method.

Interestingly, the abovementioned algorithms for the SCP are not the only ones that are directly based on the same destruction–construction process that is the basis of iterated greedy algorithms. In fact, a large number of other algorithms are based on the same principle. However, different names have been used for the underlying method or no specific name at all, which complicates the transfer of knowledge to other researchers.

Probably the earliest techniques that use some mechanism akin to the destructive–constructive moves of iterated greedy are found in the VLSI design automation community and, in particular, in the routing step, where components are connected by wires trying to obey design rules for integrated circuits. In the rip-up and reroute approach [26, 111], some routes related to bottlenecks are removed, and components are reconnected in a different order. The rip-up and reroute technique is commonly used in the design automation community, but the principle underlying this method seems not to have been generalized to the level of metaheuristic for tackling a wide variety of other optimization problems.

The possible use of destructive–constructive moves has also been mentioned earlier in the context of the type of techniques that have become known as strategic oscillation [47–49]. These types of approaches are often embedded into algorithms that make use of tabu search features to provide additional guidance to the search. Some examples of such implementations that have also a clear component related to the iterated greedy methodology are [51, 52, 71].

Among the first researchers to identify the potential of the principles underlying iterated greedy and to formulate these as a general-purpose SLS method are [117]. They called their method *ruin-and-recreate*, where the *ruin* stands for the solution destruction and the *recreate* for the reconstruction of a complete solution. (Ruin-and-recreate is protected by US patent *Optimization with ruin recreate* No. 6418398; see http://www.patentstorm.us/patents/6418398-fulltext.html.). In their seminal article, they applied ruin-and-recreate to the symmetric TSP, the vehicle routing problem with time windows, and to a network design problem, reporting good overall performance of the method. Following this work, a number of other papers have been published using the name ruin-and-recreate; however, not all papers that use the name ruin-and-recreate should be considered as iterated greedy algorithms; for example, [82, 83] essentially uses a random mutation of a current solution instead of a clearly separable destruction phase and greedy solution reconstruction.

Ahmadi and Osman [3] study the capacitated clustering $p-$median problem. Among different proposed methods and heuristics, the authors introduce a *periodic construction–deconstruction* procedure, which is basically a simple form of iterated greedy. After a given number of clusters have been built in the constructive method, some of them are randomly deconstructed, and the constructive method is reapplied.

Richmond and Beasley [107] have presented an *iterative construction heuristic* for a problem arising in ore selection. In this problem, processing options needs to be chosen for a number of mining blocks. The destruction operator deletes the processing option for some mining blocks; the size of the destruction is chosen randomly following a uniform distribution in some interval. After the reconstruction of a complete solution, the algorithm accepts only better-quality solutions as new incumbent ones. It should be clear from the description that the algorithm follows directly the very same principles of iterated greedy.

Cesta et al. [22] describe an algorithm for a capacitated scheduling problem that makes usage of the ideas underlying the concept of iterated greedy algorithms. They have developed a method that uses a greedy constructive heuristic to generate feasible schedules by introducing precedence constraints. In the destruction phase (called retraction in the chapter), they remove some of the introduced precedence relations to obtain a partial solution and then reconstruct a complete feasible one. They call their method *iterative flattening*, where flattening refers to the reduction of resource conflicts, which is an effect of the introduced precedence constraints. The term iterative flattening has gained some popularity, and several follow-up articles have been published on improvements of the algorithm [80], studies of combinations of basic algorithm components [89], or applications to other problems [90,91].

The authors of iterative flattening characterize the method in their initial paper as a local search method. Few years before, [118] has proposed the *large neighborhood search* method. The initial proposal consists in the removal of solution components and the reinsertion of solution components by using constraint programming techniques that exploit a tree search and constraint propagation techniques. Even though the method was proposed in a constraint programming framework, where a tree search is used to restore a complete solution instead of simple constructive heuristic, the latter is mentioned as one possible option. In fact, in many later articles on large neighborhood search, simple constructive heuristics are used [98], implementing, hence, directly an iterated greedy method.

Finally, the name *iterated greedy* has been used by [114] and also in [60] to denominate the type of SLS method we describe in this chapter. However, these are not the first publications that are using the name *iterated greedy*. A much earlier mentioning of the term iterated greedy is by Culberson [24]. His iterated greedy algorithm for graph coloring uses a greedy algorithm for generating a coloring, removing the color of all nodes, and then reapplying the greedy algorithm using a specific ordering of the nodes that guarantees the next generated coloring to not be worse than the previous one but potentially better. This algorithm may be seen as an extreme case of iterated greedy where the destruction operator destroys the complete solution. However, our view is rather that it is a specific algorithm that works in the way it is proposed mainly for graph coloring.

Relationship to Other Approaches

Iterated greedy is a general-purpose SLS method that has many links to other SLS methods. In the following, we discuss similarities and differences to a number of SLS methods considering other constructive methods, local search methods, and tree search algorithms. These relationships also open up many possibilities for combining techniques proposed for different methods.

Repeated (Greedy) Construction Algorithms

Iterated greedy has natural connections to other methods that repeatedly construct complete candidate solutions. A key difference between iterated greedy and few other repeated construction methods is that the solution construction in iterated greedy usually starts from a (non-empty) partial solution, while the other methods repeatedly generate new candidate solutions starting from *empty* candidate solutions.

A well-known simple such method is the greedy randomized adaptive search procedures (GRASP) that combines a greedily biased but randomized solution construction with a subsequent improvement of the generated candidate solutions by a local search procedure [37, 104]. GRASP in turn extends on some initial ideas of how to generate different solutions by a randomized greedy heuristic [57]. While GRASP does not start the construction from partial solutions, it is an example of a (simple) randomization scheme during the solution construction, which could be included directly in iterated greedy algorithms. An appealing feature of GRASP is the usage of adaptive constructive procedures, and the large number of articles on GRASP [104] may be a source for randomized greedy heuristics to be used in iterated greedy algorithms.

Many useful ideas that underlie other constructive methods may be adopted into iterated greedy algorithms. One such possibility is ways of how to improve the constructive mechanisms, for example, through look-ahead methods or, when taking look-ahead to an extreme, as advocated in the rollout [13] and pilot method [34]. In the rollout and pilot methods for each constructive decision to be taken, a full solution is constructed (or at least well approximated), and the construction decision that results in the best complete solution is taken as the next one. This process is repeated for each construction decision, resulting in a relatively time-consuming constructive approach. However, starting the rollout/pilot method from partial solutions with the resulting reduction in computation time may make this approach promising inside an iterated greedy algorithm.

Squeaky wheel optimization (SWO) [64] uses the idea of biasing the solution construction by information that was gained by analyzing complete candidate solutions. This is done by assigning priorities to specific solution components. The solution reconstruction in SWO is done, differently from iterated greedy, from empty initial solutions corresponding to a complete destruction of the current candidate solution. Within the SWO framework, Aicklin et al. [5] proposed the idea

of seeding the reconstruction of a full candidate solution by some partial solution that was obtained by removing some of the solution components of a complete candidate solution, making it a variant of iterated greedy.

A rather different way of biasing the solution construction is underlying the ant colony optimization (ACO) metaheuristic [30, 31]. In ACO, (artificial) ants implement stochastic solution construction heuristics that make their constructive decisions based on so-called pheromone information and heuristic information associated to specific solution components. The pheromone information tries to bias solution construction toward the best solutions that have been found so far. Again, differently from iterated greedy algorithms, the ants start their solution construction from empty candidate solutions. However, there have been a number of proposals in the ACO area that suggest starting the solution construction from partial solutions that are either stored [1, 2] or obtained by deconstructing complete solutions [126, 127, 134] in a way akin to iterated greedy algorithms. These approaches generally try to transfer the advantages of iterated greedy to ACO algorithms, and for several of these approaches, positive results are reported.

(Perturbative) Local Search Techniques

Iterated greedy has tight links to local search algorithms and, in particular, to very large-scale neighborhood searches [4]. In fact, as also mentioned in the section "IG Applications: Historical Development," one of the proposals of iterated greedy style algorithms called the method large neighborhood search Shaw [98]. The analogy stems from the fact that the removal of k solution components in the destruction step and the subsequent reconstruction of a complete candidate solution can be seen as a move in a large neighborhood that is implicitly defined by the number of components to be removed and/or added (Note that the number of solution components removed in a destruction step may be different from the number of solution components added in the construction step, and so we refrain from talking of k-exchange neighborhoods here. A common example where this happens is subset problems such as the SCP we discussed earlier.). The (greedy) reconstruction of a complete candidate solution typical for iterated greedy algorithms can then be seen as a heuristic examination of the neighborhood either in a fully greedy way or using randomization steps–details that simply depend on the particular design and implementation of the construction step in an iterated greedy algorithm. By varying the value of k, the size of the implicitly defined neighborhood is varied. Such changes may be done either (i) randomly within some limited range $[k_{\min}, k_{\max}]$ resembling known strategies in local search methods such as robust tabu search [121], where one tries to avoid overcommitment to a single value for a parameter, (ii) in a more systematic way such as advocated in the strategies defined for variable neighborhood search [54], or (iii) by using feedback from the search performance such as advocated in reactive search methods [11]. An interesting possibility for exploring the neighborhoods is used in the adaptive large neighborhood algorithms by [97, 110], who propose to consider different heuristics to be used in the solution

reconstruction. In their method, they additionally adapt the probability with which specific heuristics are chosen over the runtime of the algorithm based on the feedback of the search process.

An alternative to the heuristic exploration of neighborhoods provides methods that try to determine the best possible solution that can be reached from the current one, akin to best-improvement neighborhoods. In the case of large neighborhood searches, this corresponds to an exact examination of all the possible solutions that can be reached from a current partial candidate solution [4, 35]. The original proposal of large neighborhood search by Shaw [118] actually considered such an exact exploration of the resulting neighborhood by means of an constraint programming approach embedded within a branch-and-bound scheme. Due to the large variation in computation time that such a scheme however incurs, alternatives have been tested such as pure insertion-based reconstruction or limited enumeration schemes.

If we consider other local search-based SLS methods, the closest related one is certainly iterated local search [103], especially when iterated greedy algorithms exploit additional (perturbative) local search methods to improve candidate solutions, as outlined in Fig. 3. In that case, the destruction–construction cycle corresponds to what in ILS terms would be a solution perturbation. In general, it seems to be commendable in ILS to apply problem-specific perturbations whenever possible, and the destruction–construction cycle in iterated greedy introduces such problem-specific information through the use of heuristic information. Basic variable neighborhood search is an ILS-type algorithm that systematically varies the strength of the perturbation, and thus the link to iterated greedy is also immediate.

Besides the fact that complete solutions may be improved by an additional local search phase in iterated greedy algorithms, it could be worthwhile to also consider the local re-optimization of the partial solutions that are obtained during the destruction and reconstruction process. It is noteworthy that such occasional local re-optimization of partial candidate solutions has shown to be useful in a number of other contexts such as vehicle routing [21].

Tree Search Algorithms

Constructive mechanisms can be extended to an exhaustive search method by adding a simple backtracking mechanism; adding further bounding schemes, one quickly comes to methods such as branch-and-bound or, more in general, tree search techniques [60]. Hence, it is clear that iterated greedy algorithms also share some relationships to such methods.

Some links have already been mentioned in the previous section, when exact methods are exploited to generate the best possible completion of a partial candidate solution. Such a completion may be generated using tree search methods, and various of these ideas have been explored [118]. Even if tree search is used, the search need not be necessarily complete. For example, Shaw [118] has considered the usage of limited discrepancy search [58], which consists in examining a limited

number of alternative choices in the decision points during the tree generation. Any other variants such as depth-bounded discrepancy search [131] could also be interesting. Another alternative that does not seem to have been applied so far would be to apply beam search [92].

Yet another connection to tree search techniques could rely on the exploitation of lower-bound information to generate heuristic information or to prune choices that are guaranteed not to lead to improved solutions. So far, we are not aware of an exploitation of such ideas inside iterated greedy algorithms, although in other constructive SLS algorithms, such uses have been explored [14, 75].

Applications

In this section, we give a short overview of few applications of iterated greedy algorithms. Some applications have already been mentioned in the section "IG Applications: Historical Development" when discussing other methods that use the same principle as iterated greedy. Here, we focus mainly on applications that identified their algorithm as an iterated greedy algorithm. In fact, depending on the class of problems for which the respective methods have been proposed, the usage of names to refer to iterated greedy-type methods differs. For example, in the scheduling area, the excellent results of the iterated greedy algorithms for the permutation flow shop scheduling problem have spawned a lot of follow-up work on similar problems, while the name large neighborhood search is frequently found in application to vehicle routing problems, given the prominent role these algorithms have played in that domain.

Iterated Greedy for Scheduling Problems

As mentioned, iterated greedy, as defined in this chapter, was initially applied to the permutation flow shop scheduling problem by [114], and so many applications to other scheduling problems were published after that (Various applications of iterative flattening to scheduling problems have been referenced in the section "IG Applications: Historical Development."). Ruiz and Stützle [115] extended their iterated greedy algorithm to tackle the permutation flow shop scheduling problem when considering additional sequence-dependent setup times and other objectives than makespan, namely, the total tardiness. Other variants of flow shop problems have been studied for total tardiness minimization [43] and the flowtime minimization [79], blocking [106, 124], no-wait [29, 88, 95, 133], and for no-idle [105, 116, 123], and mixed no-idle [94] problems. Non-permutation flow shops were approached by [12, 17, 136]. Distributed flow shop scheduling problems have been solved with iterated greedy algorithms in [8, 40, 55, 56, 69, 74, 113]. Iterated greedy has also proven to be valuable in other optimization criteria apart from makespan, and multi-objective extensions of iterated greedy have proven to be worthwhile [23, 81, 119]. Dubois et al. [32] have embedded an iterated greedy algorithm into

the two-phase local search framework to tackle various bi-objective flow shop problems. Parallel machine problems were successfully tackled with iterated greedy algorithms [7, 36, 78, 109]. Iterated greedy algorithms are effective for various single-machine scheduling problems as shown in [28, 138]. Some other more complex problems have been studied, including job-shop scheduling with blocking constraints [100], job-shop scheduling with sequence-dependent setup times and job families [67], the optimization of manufacturing problems [137], hybrid flow shops [73, 84, 135], and real-life problems [27, 128–130].

Iterated Greedy for Routing Problems

As mentioned above, in the context of vehicle routing problems, several algorithms that follow the main structure of an iterated greedy algorithm have been applied but usually using branding under the name large neighborhood search. The article making these approaches popular has already been mentioned [118]. A major impact in that line of applications had the adaptive large neighborhood search approaches [97, 110], which led to a large number of follow-up work. Some overview is given in [98]. In fact, the usage of such iterated greedy-type methods as local searches inside SLS algorithms for vehicle routing problems is increasingly widespread and can by now be considered a standard in this area. Apart from vehicle routing, also iterated greedy approaches have been proposed to few other routing-type problems such as scheduling and routing problems of freight trains [139], in a study on a food ordering and delivery system [132], and the routing of hydrographic surveys [25]. Some few other works are in the context of TSP variants [6, 66] (Interestingly, in [6] they call their algorithm an iterated local search, but instead they do something that would be rather called an iterated greedy approach.).

Iterated Greedy for Other Problems

There have been a number of other applications where explicitly iterated greedy algorithms have been devised as the main solution techniques. Lozano et al. [70] presented an iterated greedy approach to the maximum diversity problem, where from a set of elements, a subset with maximum diversity has to be chosen. After proper tuning, the algorithm was shown to perform better than various competing algorithms and was established as a new state-of-the-art algorithm. Casado et al. [20] set up an iterated greedy to solve the minimum dominating set as obtain in graphs and derive an iterated greedy method with excellent performance within short computational times. García-Martínez et al. [46] have developed an iterated greedy algorithm enhanced by a short tabu list in the destructive phase for the quadratic multiple knapsack problem. Lozano et al. [71] have developed also another iterated greedy algorithm for the quadratic minimum spanning tree problem

obtaining excellent results on large instances. They embed the algorithm into a strategic oscillation approach by essentially extending it with tabu criteria, which led to further improvements on some problem instance classes. Early implementations of similar ideas for binary quadratic programming have been presented before [52], and Toyama et al. [125] apply iterated greedy to the same problem tackling also very large-scale instances successfully. Kang et al. [65] study the problem of allocating parallel tasks to processors in computing systems that are distributed and heterogeneous. Iterated greedy algorithms and population-based iterated greedy algorithms that exploit exact solutions to large neighborhood searches have been proposed for the maximal covering location problem [108], the goal of which is to cover clients such that the largest amount of demand possible is satisfied. Similar approaches based on population-based iterated greedy methods have been proposed by Bouamama et al [16] to maximize the sensor lifetime in a network and Lozano and Rodriguez-Tello [72] for an algorithm for the minimum S-labeling problem of a graph. A problem in market segmentation, where a company asks to partition a set of customers subject to some specific requirements related to homogeneity of customers and compactness of the areas is tackled by Huerta-Muñoz et al. [61]. The obnoxious p-median problem, where the objective is to find places to facilities that are out of some reason undesirable so that the sum of minimum distances between non-facility and the closest facility is maximized, has been tackled by two iterated greedy methods [53, 86]. Both at the time published the state-of-the-art algorithms; the later one is a variant with variable reconstruction size, which is a major advantage over the previous iterated greedy. A population-based iterated greedy algorithm has also been applied for delimiting and zoning rural settlements [99] and to the minimum weight vertex cover problem [15]. First applications of iterated greedy algorithms for machine learning tasks, in particular the generation of classification rules, have been explored [96].

Conclusions

The main principle of iterated greedy is to build a sequence of solutions by iterating over constructive algorithms through a loop of solution destructions and (re-)constructions. Deconstruction removes solution components resulting in partial solutions from which again full solutions are reconstructed. This loop may be extended by an additional local search phase, where the generated complete candidate solutions are further improved, and by many other techniques from other heuristic and exact search techniques, making iterated greedy a very flexible and malleable method.

We prefer to call this principle iterated greedy because (i) this name directly refers to what the principle implies, namely, making iterative use of construction methods, (ii) it does not obfuscate the name with natural or unnatural analogies, (iii) it is a short and punchy name describing the essence of the method, and (iv) it has

by now been used in a large number of publications. A bit unfortunate is that in the literature there have been proposed several methods under different names that make use of the same (or a very similar) principle including large neighborhood search [98, 118], simulated annealing [62], evolutionary heuristic [77], ruin-and-recreate [117], iterative construction search [107], or iterative flattening [22]. The multitude of different names for the same kind of approach can probably be explained in two ways. First, different researchers have a different perspective on the same method, and, thus, the iterated greedy principle can be viewed and proposed, for example, from the perspective of a (perturbative) neighborhood search leading to the heuristic or exact exploration of large neighborhoods or from the perspective of a (constructive) algorithm background, where the method is simply iterating through applications of constructive algorithms. Second, the method has been proposed in somewhat different communities such as ruin-and-recreate in a physics journal [117], large neighborhood search at a constraint programming conference [118], and iterative flattening at an artificial intelligence conference [22], and publications using other names have appeared in operations research journals [62, 107, 114]. It apparently has taken a significant amount of time that these ideas transpired from one community to another.

Even if this leads to some confusion and we maybe contribute to this confusion, we think that what is really important is (i) the principle underlying these methods, (ii) the fact that the iterated greedy principle can lead to very powerful algorithms, and (iii) the fact that iterated greedy is a very flexible method that can easily be combined with other techniques. By making the relationship among the different proposals clear, we hope to contribute also to a transfer of experience between these different algorithms. In any case, we hope that this review chapter will be useful for other researchers in stochastic local search methods by clearly identifying the potential of iterated greedy and in this way also contribute to its further development.

Cross-References

- ▸ GRASP
- ▸ Iterated Local Search
- ▸ Memetic Algorithms
- ▸ Tabu Search
- ▸ Variable Neighborhood Search

Acknowledgments Thomas Stützle acknowledges support from the Belgian F.R.S.-FNRS, of which he is a research director. Rubén Ruiz is partially supported by the Spanish Ministry of Economy and Competitiveness, under the project "SCHEYARD—Optimization of Scheduling Problems in Container Yards" (No. DPI2015-65895-R) financed by FEDER funds.

Competing Interest Declaration The author(s) has no competing interests to declare that are relevant to the content of this manuscript.

References

1. Acan A (2004) An external memory implementation in ant colony optimization. In: Dorigo M et al (eds) Ant colony optimization and swarm intelligence, 4th international workshop (ANTS 2004). Lecture notes in computer science, vol 3172. Springer, Heidelberg, pp 73–84
2. Acan A (2005) An external partial permutations memory for ant colony optimization. In: Raidl GR, Gottlieb J (eds) Proceedings of EvoCOP 2005—5th European conference on evolutionary computation in combinatorial optimization. Lecture notes in computer science, vol 3448. Springer, Heidelberg, pp 1–11
3. Ahmadi S, Osman IH (2004) Density based problem space search for the capacitated clustering p-median problem. Ann Oper Res 131:21–43
4. Ahuja RK, Ergun O, Orlin JB, Punnen AP (2002) A survey of very large-scale neighborhood search techniques. Discret Appl Math 123(1–3):75–102
5. Aickelin U, Burke EK, Li J (2006) Improved squeaky wheel optimisation for driver scheduling. In: Runarsson TP, Beyer HG, Burke EK, Merelo JJ, Whitley LD, Yao X (eds) (2006) Proceedings of PPSN-IX, ninth international conference on parallel problem solving from nature. Lecture notes in computer science, vol 4193. Springer, Heidelberg
6. Archetti C, Feillet D, Mor A et al (2018) An iterated local search for the traveling salesman problem with release dates and completion time minimization. Comput Oper Res 98:24–37
7. Arroyo J, Leung JT (2017) An effective iterated greedy algorithm for scheduling unrelated parallel batch machines with non-identical capacities and unequal ready times. Comput Ind Eng 105:84–100
8. Bao H, Pan Q, Ruiz R, Gao L (2023) A collaborative iterated greedy algorithm with reinforcement learning for energy-aware distributed blocking flow-shop scheduling. Swarm Evol Comput 83:101399
9. Balas E, Ho A (1980) Set covering algorithms using cutting planes, heuristics, and subgradient optimization: a computational study. Math Program Study 12:37–60
10. Bartz-Beielstein T, Chiarandini M, Paquete L, Preuss M (eds) (2010) Experimental methods for the analysis of optimization algorithms. Springer, Berlin
11. Battiti R, Brunato M, Mascia F (2008) Reactive search and intelligent optimization. Operations research/computer science interfaces, vol 45. Springer, New York
12. Benavides AJ, Ritt M (2015) Two simple and effective heuristics for minimizing the makespan in non-permutation flow shops. Comput Oper Res 66:160–169
13. Bertsekas DP, Tsitsiklis JN, Wu C (1997) Rollout algorithms for combinatorial optimization. J Heuristics 3(3):245–262
14. Blum C (2005) Beam-ACO—hybridizing ant colony optimization with beam search: an application to open shop scheduling. Comput Oper Res 32(6):1565–1591
15. Bouamama S, Blum C, Boukerram A (2012) A population-based iterated greedy algorithm for the minimum weight vertex cover problem. Appl Soft Comput 12(6):1632–1639
16. Bouamama S, Blum C, Pinacho-Davidson P (2022) A population-based iterated greedy algorithm for maximizing sensor network lifetime. Sensors 22:1804
17. Brum A, Ruiz R, Ritt M (2022) Automatic generation of iterated greedy algorithms for the non-permutation flow shop scheduling problem with total completion time minimization. Comput Ind Eng 163:107843
18. Brusco MJ, Jacobs LW, Thompson GM (1999) A morphing procedure to supplement a simulated annealing heuristic for cost- and coverage-correlated set covering problems. Ann Oper Res 86:611–627
19. Burke EK, Gendreau M, Hyde MR, Kendall G, Ochoa G, Özcan E, Qu R (2013) Hyper-heuristics: a survey of the state of the art. J Oper Res Soc 64(12):1695–1724
20. Casado A, Bermudo S, López-Sánchez AD et al (2023) An iterated greedy algorithm for finding the minimum dominating set in graphs. Math Comput Simul 207:41–58
21. Caseau Y, Laburthe F (1999) Heuristics for large constrained vehicle routing problems. J Heuristics 5(3):281–303

22. Cesta A, Oddi A, Smith SF (2000) Iterative flattening: a scalable method for solving multi-capacity scheduling problems. In: Proceedings of AAAI 2000—seventeenth national conference on artificial intelligence. AAAI Press/MIT Press, Menlo Park, pp 742–747
23. Ciavotta M, Minella G, Ruiz R (2013) Multi-objective sequence dependent setup times flowshop scheduling: a new algorithm and a comprehensive study. Eur J Oper Res 227(2):301–313
24. Culberson JC (1992) Iterated greedy graph coloring and the difficulty landscape. Technical Report 92-07, Department of Computing Science, The University of Alberta, Edmonton
25. Dao SD, Mallágol A, Meyer P et al (2022) A hybrid iterated greedy algorithm for hydrographic survey routing problem. Marine Geodesy 45(1):75–100
26. Dees WA Jr, Karger PG (1982) Automated rip-up and reroute techniques. In: Proceedings of the 19th design automation workshop (DAC'82). IEEE Press, pp 432–439
27. Demir Y (2024) An iterated greedy algorithm for the planning of yarn-dyeing boilers. Int Trans Oper Res 31:115–139
28. Deng G, Gu X (2014) An iterated greedy algorithm for the single-machine total weighted tardiness problem with sequence-dependent setup times. Int J Syst Sci 45(3):351–362
29. Ding JY, Song S, Gupta JND, Zhang R, Chiong R, Wu C (2015) An improved iterated greedy algorithm with a tabu-based reconstruction strategy for the no-wait flowshop scheduling problem. Appl Soft Comput 30:604–613
30. Dorigo M, Stützle T (2004) Ant colony optimization. MIT Press, Cambridge
31. Dorigo M, Birattari M, Stützle T (2006) Ant colony optimization: artificial ants as a computational intelligence technique. IEEE Comput Intell Mag 1(4):28–39
32. Dubois-Lacoste J, López-Ibáñez M, Stützle T (2011) A hybrid TP+PLS algorithm for bi-objective flow-shop scheduling problems. Comput Oper Res 38(8):1219–1236.
33. Dubois-Lacoste J, Pagnozzi F, Stützle T (2017) An iterated greedy algorithm with optimization of partial solutions for the makespan permutation flowshop problem. Comput Oper Res 81:160–166
34. Duin C, Voß S (1999) The pilot method: a strategy for heuristic repetition with application to the Steiner problem in graphs. Networks 34(3):181–191
35. Dumitrescu I, Stützle T (2009) Usage of exact algorithms to enhance stochastic local search algorithms. In: Maniezzo V, Stützle T, Voß S (eds) Matheuristics—hybridizing metaheuristics and mathematical programming. Annals of information systems, vol 10. Springer, New York, pp 103–134
36. Fanjul-Peyro L, Ruiz R (2010) Iterated greedy local search methods for unrelated parallel machine scheduling. Eur J Oper Res 207(1):55–69
37. Feo TA, Resende MGC (1989) A probabilistic heuristic for a computationally difficult set covering problem. Oper Res Lett 8(2):67–71
38. Feo TA, Resende MGC (1995) Greedy randomized adaptive search procedures. J Glob Optim 6:109–113
39. Fernandez-Viagas V, Framiñán JM (2014) On insertion tie-breaking rules in heuristics for the permutation flowshop scheduling problem. Comput Oper Res 45:60–67
40. Fernandez-Viagas V, Framinan JM (2015) A bounded-search iterated greedy algorithm for the distributed permutation flowshop scheduling problem. Int J Prod Res 53(4):1111–1123
41. Fernandez-Viagas V, Framinan JM (2019) A best-of-breed iterated greedy for the permutation flowshop scheduling problem with makespan objective. Comput Oper Res 112:104767
42. Fernandez-Viagas V, Valente JM, Framinan JM (2018) Iterated-greedy-based algorithms with beam search initialization for the permutation flowshop to minimise total tardiness. Expert Syst Appl 94:58–69
43. Framinan JM, Leisten R (2008) Total tardiness minimization in permutation flow shops: a simple approach based on a variable greedy algorithm. Int J Prod Res 46(22):6479–6498
44. Framiñán JM, Gupta JN, Leisten R (2004) A review and classification of heuristics for permutation flow-shop scheduling with makespan objective. J Oper Res Soc 55(12):1243–1255
45. Framiñán JM, Leisten R, Ruiz R (2014) Manufacturing scheduling systems: an integrated view on models, methods, and tools. Springer, New York

46. García-Martínez C, Rodriguez FJ, Lozano M (2014) Tabu-enhanced iterated greedy algorithm: a case study in the quadratic multiple knapsack problem. Eur J Oper Res 232(3): 454–463
47. Glover F (1977) Heuristics for integer programming using surrogate constraints. Decis Sci 8:156–166
48. Glover F (1986) Future paths for integer programming and links to artificial intelligence. Comput Oper Res 13(5):533–549
49. Glover F (1989) Tabu search—part I. INFORMS J Comput 1(3):190–206. https://doi.org/10.1287/ijoc.1.3.190
50. Glover F, Kochenberger G (eds) (2002) Handbook of metaheuristics. Kluwer Academic Publishers, Norwell
51. Glover F, Kochenberger GA (1996) Critical even tabu search for multidimensional knapsack problems. In: Osman IH, Kelly JP (eds) Metaheuristics: theory and applications. Kluwer Academic Publishers, Norwell, pp 407–427
52. Glover F, Kochenberger GA, Alidaee B (1998) Adaptive memory tabu search for binary quadratic programs. Manag Sci 44(3):336–345
53. Gokalp O (2020) An iterated greedy algorithm for the obnoxious p-median problem. Eng Appl Artif Intell 92:103674
54. Hansen P, Mladenović N (2001) Variable neighborhood search: principles and applications. Eur J Oper Res 130(3):449–467
55. Han B, Pan QK, Gao L (2024) A cooperative iterated greedy algorithm for the serial distributed permutation flowshop scheduling problem. Int J Prod Res 62(12):1–28
56. Han X, Han Y, Zhang B, Qin H, Li J, Liu Y, Gong D (2022) An effective iterative greedy algorithm for distributed blocking flowshop scheduling problem with balanced energy costs criterion. Appl Soft Comput 129:109502
57. Hart JP, Shogan AW (1987) Semi-greedy heuristics: an empirical study. Oper Res Lett 6(3):107–114
58. Harvey WD, Ginsberg ML (1995) Limited discrepancy search. In: Mellish CS (ed) Proceedings of the fourteenth international joint conference on artificial intelligence (IJCAI-95). Morgan Kaufmann Publishers, pp 607–615
59. Hejazi SR, Saghafian S (2005) Flowshop-scheduling problems with makespan criterion: a review. Int J Prod Res 43(14):2895–2929
60. Hoos HH, Stützle T (2005) Stochastic local search—foundations and applications. Morgan Kaufmann Publishers, San Francisco
61. Huerta-Muñoz DL, Ríos-Mercado RZ, Ruiz R (2017) An iterated greedy heuristic for a market segmentation problem with multiple attributes. Eur J Oper Res 261(1):75–87
62. Jacobs LW, Brusco MJ (1995) A local search heuristic for large set-covering problems. Nav Res Logist 42(7):1129–1140
63. Johnson DS, McGeoch LA (2002) Experimental analysis of heuristics for the STSP. In: Gutin G, Punnen A (eds) The traveling salesman problem and its variations. Kluwer Academic Publishers, Dordrecht, pp 369–443
64. Joslin DE, Clements DP (1999) Squeaky wheel optimization. J Artif Intell Res 10:353–373
65. Kang Q, He H, Wei J (2013) An effective iterated greedy algorithm for reliability-oriented task allocation in distributed computing systems. J Parallel Distrib Comput 73(8): 1106–1115
66. Karabulut K, Tasgetiren FM (2014) A variable iterated greedy algorithm for the traveling salesman problem with time windows. Inform Sci 279:383–395
67. Kim JS, Park JH, Lee DH (2017) Iterated greedy algorithms to minimize the total family flow time for job-shop scheduling with job families and sequence-dependent set-ups. Eng Optim 49(10):1719–1732
68. Kirkpatrick S, Gelatt CD, Vecchi MP (1983) Optimization by simulated annealing. Science 220:671–680
69. Lin SW, Ying KC, Huang CY (2013) Minimising makespan in distributed permutation flowshops using a modified iterated greedy algorithm. Int J Prod Res 51(16):5029–5038

70. Lozano M, Molina D, García-Martínez C (2011) Iterated greedy for the maximum diversity problem. Eur J Oper Res 214(1):31–38
71. Lozano M, Glover F, García-Martínez C, Rodríguez FJ, Martí R (2014) Tabu search with strategic oscillation for the quadratic minimum spanning tree. IIE Trans 46(4):414–428
72. Lozano M, Rodriguez-Tello E (2023) Population-based iterated greedy algorithm for the S-labeling problem. Comput Oper Res 155:106224
73. Lu Y, Tang Q, Pan Q, Zhao L, Zhu Y (2023) A heuristic-based adaptive iterated greedy algorithm for lot-streaming hybrid flow shop scheduling problem with consistent and intermingled sub-lots. Sensors 23(5):2808
74. Lu C, Zheng J, Yin L, Wang R (2024) An improved iterated greedy algorithm for the distributed hybrid flowshop scheduling problem. Eng Optim 56(5):792–810
75. Maniezzo V (1999) Exact and approximate nondeterministic tree-search procedures for the quadratic assignment problem. INFORMS J Comput 11(4):358–369
76. Marchiori E, Steenbeek AG (1998) An iterated heuristic algorithm for the set covering problem. In: Mehlhorn K (ed) Algorithm engineering, 2nd international workshop (WAE'92). Max-Planck-Institut für Informatik, Saarbrücken, pp 155–166
77. Marchiori E, Steenbeek AG (2000) An evolutionary algorithm for large scale set covering problems with application to airline crew scheduling. In: Cagnoni S et al (eds) Real-world applications of evolutionary computing, EvoWorkshops 2000. Lecture notes in computer science, vol 1803. Springer, Heidelberg, pp 367–381
78. Mecler D, Abu-Marrul V, Martinelli R et al (2022) Iterated greedy algorithms for a complex parallel machine scheduling problem. Eur J Oper Res 300:545–560
79. Mendoze-Gómez R, Ríos-Mercado RZ, Valenzuela-Ocaña KB (2020) An iterated greedy algorithm with variable neighborhood descent for the planning of specialized diagnostic services in a segmented healthcare system. J Ind Man Optim 16(2):857–885
80. Michel LD, van Hentenryck P (2004) Iterative relaxations for iterative flattening in cumulative scheduling. In: Zilberstein S, Koehler J, Koenig S (eds) Proceedings of the fourteenth international conference on automated planning and scheduling (ICAPS 2004). AAAI Press/ MIT Press, Menlo Park, pp 200–208
81. Minella G, Ruiz R, Ciavotta M (2011) Restarted iterated pareto greedy algorithm for multi-objective flowshop scheduling problems. Comput Oper Res 38(11):1521–1533
82. Misevičius A (2003) Genetic algorithm hybridized with ruin and recreate procedure: application to the quadratic assignment problem. Knowl Based Syst 16(5–6):261–268
83. Misevičius A (2003) Ruin and recreate principle based approach for the quadratic assignment problem. In: Cantú-Paz E et al (eds) Genetic and evolutionary computation—GECCO 2003, part I. Lecture notes in computer science, vol 2723. Springer, Heidelberg, pp 598–609
84. Missaoui A, Ruiz R (2022) A parameter-less iterated greedy method for the hybrid flowshop scheduling problem with setup times and due date windows. Eur J Oper Res 303:99–113
85. Montgomery DC (2012) Design and analysis of experiments, 8th edn. Wiley, New York
86. Mousavi S, Bhambar S, England M (2025) An iterated greedy algorithm with variable reconstruction size for the obnoxious p-median problem. Int Trans Oper Res 32:144–175
87. Nawaz M, Enscore E Jr, Ham I (1983) A heuristic algorithm for the m-machine, n-job flow-shop sequencing problem. Omega 11(1):91–95
88. Nagano MS, Siqueira de Almeida F, Miyata HH (2021) An iterated greedy algorithm for the no-wait flowshop scheduling problem to minimize makespan subject to total completion time. Eng Optim 53(8):1431–1449
89. Oddi A, Cesta A, Policella N, Smith SF (2008) Combining variants of iterative flattening search. Eng Appl Artif Intell 21(5):683–690
90. Oddi A, Cesta A, Policella N, Smith SF (2010) Iterative flattening search for resource constrained scheduling. J Intell Manuf 21(1):17–30
91. Oddi A, Rasconi R, Cesta A, Smith SF (2011) Iterative flattening search for the flexible job shop scheduling problem. In: Walsh T (ed) Proceedings of the twenty-second international joint conference on artificial intelligence (IJCAI-11). IJCAI/AAAI Press, Menlo Park, pp 1991–1996

92. Ow PS, Morton TE (1988) Filtered beam search in scheduling. Int J Prod Res 26:297–307
93. Pagnozzi F, Stützle T (2019) Automatic design of hybrid stochastic local search algorithms for permutation flowshop problems. Eur J Oper Res 276(2):409–421
94. Pan QK, Ruiz R (2014) An effective iterated greedy algorithm for the mixed no-idle flowshop scheduling problem. Omega 44(1):41–50
95. Pan QK, Wang L, Zhao BH (2008) An improved iterated greedy algorithm for the no-wait flow shop scheduling problem with makespan criterion. Int J Adv Manuf Tech 38(7–8):778–786
96. Pedraza JA, García-Martínez C, Cano A, Ventura S (2014) Classification rule mining with iterated greedy. In: Polycarpou MM, de Carvalho ACPLF, Pan J, Wozniak M, Quintián H, Corchado E (eds) Hybrid artificial intelligence systems—9th international conference (HAIS 2014), Salamanca, 11–13 June 2014. Proceedings. Lecture notes in computer science, vol 8480. Springer, Heidelberg, pp 585–596
97. Pisinger D, Ropke S (2007) A general heuristic for vehicle routing problems. Comput Oper Res 34(8):2403–2435
98. Pisinger D, Ropke S (2010) Large neighborhood search. In: Gendreau M, Potvin JY (eds) (2010) Handbook of metaheuristics. International series in operations research and management science, vol 146, 2nd edn. Springer, New York, pp 399–419
99. Porta J, Parapar J, Doallo R, Barbosa V, Santé I, Crecente R, Díaz C (2013) A population-based iterated greedy algorithm for the delimitation and zoning of rural settlements. Comput Environ Urban Syst 39:12–26
100. Pranzo M, Pacciarelli D (2016) An iterated greedy metaheuristic for the blocking job shop scheduling problem. J Heuristics 22(4):587–611
101. Rad SF, Ruiz R, Boroojerdian N (2009) New high performing heuristics for minimizing makespan in permutation flowshops. Omega 37(2):331–345
102. Ramalhinho Lourenço H, Martin O, Stützle T (2002) Iterated local search. In: Glover F, Kochenberger G (eds) (2002) Handbook of metaheuristics. Kluwer Academic Publishers, Norwell, pp 321–353
103. Ramalhinho Lourenço H, Martin O, Stützle T (2010) Iterated local search: framework and applications. In: Gendreau M, Potvin JY (eds) (2010) Handbook of metaheuristics. International series in operations research and management science, vol 146, 2nd edn. Springer, New York, chap 9, pp 363–397
104. Resende MGC, Ribeiro CC (2010) Greedy randomized adaptive search procedures: advances, hybridizations, and applications. In: Gendreau M, Potvin JY (eds) (2010) Handbook of metaheuristics. International series in operations research and management science, vol 146, 2nd edn. Springer, New York, pp 283–319
105. Riahi V, Chiong R, Zhang Y (2020) A new iterated greedy algorithm for no-idle permutation flowshop scheduling with the total tardiness criterion. Comput Oper Res 117:104839
106. Ribas I, Companys R, Tort-Martorell X (2011) An iterated greedy algorithm for the flowshop scheduling problem with blocking. Omega 39(3):293–301
107. Richmond AJ, Beasley JE (2004) An iterative construction heuristic for the ore selection problem. J Heuristics 10(2):153–167
108. Rodríguez FJ, Blum C, Lozano M, García-Martínez C (2012) Iterated greedy algorithms for the maximal covering location problem. In: Hao JK, Middendorf M (eds) Proceedings of EvoCOP 2012—12th European conference on evolutionary computation in combinatorial optimization. Lecture notes in computer science, vol 7245. Springer, Heidelberg, pp 172–181
109. Rodriguez FJ, Lozano M, Blum C, García-Martínez C (2013) An iterated greedy algorithm for the large-scale unrelated parallel machines scheduling problem. Comput Oper Res 40(7):1829–1841
110. Ropke S, Pisinger D (2006) An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows. Transp Sci 40(4):455–472
111. Rubin F (1974) An iterative technique for printed wire routing. In: Proceedings of the 11th design automation workshop (DAC'74). IEEE Press, pp 308–313

112. Ruiz R, Maroto C (2005) A comprehensive review and evaluation of permutation flowshop heuristics. Eur J Oper Res 165(2):479–494
113. Ruiz R, Pan QK, Naderi B (2019) Iterated greedy methods for the distributed permutation flowshop scheduling problem. Omega 83:213–222
114. Ruiz R, Stützle T (2007) A simple and effective iterated greedy algorithm for the permutation flowshop scheduling problem. Eur J Oper Res 177(3):2033–2049
115. Ruiz R, Stützle T (2008) An iterated greedy heuristic for the sequence dependent setup times flowshop problem with makespan and weighted tardiness objectives. Eur J Oper Res 187(3):1143–1159
116. Ruiz R, Vallada E, Fernández-Martínez C (2009) Scheduling in flowshops with no-idle machines. In: Chakraborty UK (ed) Computational intelligence in flow shop and job shop scheduling. Studies in computational intelligence, vol 230. Springer, Berlin, pp 21–51
117. Schrimpf G, Schneider J, Stamm-Wilbrandt H, Dueck G (2000) Record breaking optimization results using the ruin and recreate principle. J Comput Phys 159(2):139–171
118. Shaw P (1998) Using constraint programming and local search methods to solve vehicle routing problems. In: Maher M, Puget JF (eds) Principles and practice of constraint programming, CP98. Lecture notes in computer science, vol 1520. Springer, Heidelberg, pp 417–431
119. Siqueira de Almeida F, Seido Nagano M (2023) An efficient iterated greedy algorithm for a multi-objective no-wait flow shop problem with sequence dependent setup times. 4OR 22: 31–45
120. Taillard ÉD (1990) Some efficient heuristic methods for the flow shop sequencing problem. Eur J Oper Res 47(1):65–74
121. Taillard ÉD (1991) Robust taboo search for the quadratic assignment problem. Parallel Comput 17(4–5):443–455
122. Taillard ÉD (1993) Benchmarks for basic scheduling problems. Eur J Oper Res 64(2): 278–285
123. Tasgetiren FM, Pan QK, Suganthan PN, Buyukdagli O (2013) A variable iterated greedy algorithm with differential evolution for the no-idle permutation flowshop scheduling problem. Comput Oper Res 40(7):1729–1743
124. Tasgetiren MF, Kizilay D, Pan QK, Suganthan PN (2017) Iterated greedy algorithms for the blocking flowshop scheduling problem with makespan criterion. Comput Oper Res 77: 111–126
125. Toyama F, Shoji K, Mori H, Miyamichi J (2012) An iterated greedy algorithm for the binary quadratic programming problem. In: Joint 6th international conference on soft computing and intelligent systems (SCIS) and 13th international symposium on advanced intelligent systems (ISIS), 2012. IEEE Press, pp 2183–2188
126. Tsutsui S (2006) cAS: ant colony optimization with cunning ants. In: Runarsson TP, Beyer HG, Burke EK, Merelo JJ, Whitley LD, Yao X (eds) (2006) Proceedings of PPSN-IX, ninth international conference on parallel problem solving from nature. Lecture notes in computer science, vol 4193. Springer, Heidelberg, pp 162–171
127. Tsutsui S (2007) Ant colony optimization with cunning ants. Trans Jpn Soc Artif Intell 22: 29–36
128. Urlings T, Ruiz R (2007) Local search in complex scheduling problems. In: Stützle T, Birattari M, Hoos HH (eds) Engineering stochastic local search algorithms. Designing, implementing and analyzing effective heuristics. Lecture notes in computer science, vol 4638. Springer, Brussels, pp 202–206
129. Urlings T, Ruiz R, Sivrikaya-Şerifoğlu F (2010) Genetic algorithms for complex hybrid flexible flow line problems. Int J Metaheuristics 1(1):30–54
130. Urlings T, Ruiz R, Stützle T (2010) Shifting representation search for hybrid flexible flowline problems. Eur J Oper Res 207(2):1086–1095
131. Walsh T (1997) Depth-bounded discrepancy search. In: Pollack ME (ed) Proceedings of the fifteenth international joint conference on artificial intelligence (IJCAI-97). Morgan Kaufmann Publishers, pp 1388–1395

132. Wang X, Wang S, Wang L et al (2020) An effective iterated greedy algorithm for online route planning problem. In: 2020 IEEE congress on evolutionary computation (CEC)
133. Wang Y, Li X, Ruiz R et al (2018) An iterated greedy heuristic for mixed no-wait flowshop problems. IEEE Trans Cybern 48(5):1553–1566
134. Wiesemann W, Stützle T (2006) Iterated ants: an experimental study for the quadratic assignment problem. In: Dorigo M et al (eds) Ant colony optimization and swarm intelligence, 5th international workshop, ANTS 2006. Lecture notes in computer science, vol 4150. Springer, Heidelberg, pp 179–190
135. Ying KC (2008) An iterated greedy heuristic for multistage hybrid flowshop scheduling problems with multiprocessor tasks. IEEE Trans Evol Comput 60(6):810–817
136. Ying KC (2008) Solving non-permutation flowshop scheduling problems by an effective iterated greedy heuristic. Int J Adv Manuf Tech 38(3–4):348–354
137. Ying KC, Fruggiero F, Pourhejazy P, Lee BY (2022) Adjusted iterated greedy for the optimization of additive manufacturing scheduling problems. Expert Syst Appl 198:116908
138. Ying KC, Lin SW, Huang CY (2009) Sequencing single-machine tardiness problems with sequence dependent setup times using an iterated greedy heuristic. Expert Syst Appl 36(3):7087–7092
139. Yuan Z, Fügenschuh A, Homfeld H, Balaprakash P, Stützle T, Schoch M (2008) Iterated greedy algorithms for a real-world cyclic train scheduling problem. In: Blesa MJ, Blum C, Cotta C, Fernández AJ, Gallardo JE, Roli A, Sampels M (eds) Hybrid metaheuristics. Lecture notes in computer science, vol 5296. Springer, Heidelberg, pp 102–116
140. Zilberstein S (1996) Using anytime algorithms in intelligent systems. AI Mag 17(3):73–83

Iterated Local Search

26

Thomas Stützle and Rubén Ruiz

Contents

Introduction 780
Iterated Local Search 781
Local Search Heuristics 781
Iterated Local Search Framework 783
Some Examples of Iterated Local Search Algorithms 787
TSP Example 787
QAP Example 788
Permutation Flow-Shop Scheduling Example 789
Historical Development of Iterated Local Search 791
Relationship of ILS to 792
. . . Simple SLS Methods 793
. . . Hybrid SLS Methods 793
. . . Population-Based SLS Methods 795
Applications 795
Iterated Local Search for Routing Problems 796
Iterated Local Search for Scheduling Problems 797
Iterated Local Search for Other Problems 799
Conclusions 800
Cross-References 801
References 801

T. Stützle (✉)
IRIDIA, Université Libre de Bruxelles (ULB), Brussels, Belgium
e-mail: stuetzle@ulb.ac.be

R. Ruiz
Universitat Politécnica de València, València, Spain

AMAZON, Amazon.com, Inc., Seattle, WA, USA
e-mail: rruiz@eio.upv.es

© Springer Nature Switzerland AG 2025

R. Martí et al. (eds.), *Handbook of Heuristics*,
https://doi.org/10.1007/978-3-032-00385-0_8

Abstract

Iterated local search is a metaheuristic that embeds an improvement heuristic within an iterative process generating a chain of solutions. Often, the improvement method is some kind of local search algorithm and, hence, the name of the metaheuristic. The iterative process in iterated local search consists of a perturbation of the current solution, leading to some intermediate solution that is used as a new starting solution for the improvement method. An additional acceptance criterion decides which of the solutions to keep for continuing this process. This simple idea has led to some very powerful algorithms that have been successfully used to tackle hard combinatorial optimization problems. In this chapter, we review the main ideas of iterated local search, exemplify its application to combinatorial problems, discuss historical aspects of the development of the method, and give an overview of some successful applications.

Keywords

Stochastic local search · Metaheuristics · Iterated local search · Local search · Perturbation · Acceptance criterion

Introduction

One important rationale of a large number of metaheuristics is to overcome the limitations of local optimality incurred by iterative improvement methods [47]. Several concepts for this task have been considered in the prolific literature on metaheuristics that include the occasional acceptance of worsening moves within a local search process, an idea underlying methods such as simulated annealing [24, 71] or tabu search [47], the usage of populations as a simple means for increasing the exploration of the search space as in evolutionary algorithms [8] or ant colony optimization [34], or the introduction of penalties in the search process as in dynamic local search methods [60].

Certainly, one of the simplest ideas is to call iterative improvement methods several times with new starting solutions to sample possibly new and better local optima. Doing so by generating new starting solutions uniformly at random from the search space is known to be an ineffective approach, especially when instance size increases [114]. Iterated local search (ILS) instead exploits the idea of generating a chain of solutions by creating new starting solutions from a perturbation of some of the previously found solutions; in this way, it creates a biased sampling of starting solutions [81]. An acceptance criterion determines whether the new locally optimal solution is taken as the new incumbent or whether the previous solution is maintained. This basic idea underlying ILS has been implemented in a number of early papers on the method [11, 12, 67, 86, 87]. The method has been extended in several directions in follow-up research, and a number of new variants

and generalizations have been considered. In fact, a first generalization is that the underlying improvement heuristic need not be necessarily an iterative improvement algorithm but can be any method that takes some solution as input and returns a possibly improved solution. Other variants concern the introduction of rather elaborate perturbation mechanisms or the consideration of different acceptance criteria. Nevertheless, what characterizes an ILS algorithm is that a chain of solutions is built through a process that involves as main sub-procedures some form of solution perturbation, an improvement method of whatever type, and an acceptance criterion.

In this chapter, we review the main ideas underlying ILS and illustrate its main principles using some example applications to well-known combinatorial optimization problems in section "Some Examples of Iterated Local Search Algorithms." We then shortly highlight, in section "Historical Development of Iterated Local Search," the main developments on the method from a historical perspective, which also shows that ILS is among the oldest metaheuristic techniques. We discuss the most relevant links ILS has to other metaheuristics in section "Relationship of ILS to ...." We end with an overview of various noteworthy applications of ILS, which complements some earlier reviews on ILS [81,82], and some concluding remarks in section "Conclusions."

Iterated Local Search

Here, we first introduce basic notions of local search and then elaborate on the main mechanisms ILS uses to steer the search process.

Local Search Heuristics

Many heuristics for tackling hard optimization problems rely on (perturbative) local search [1,60]. The idea underlying local search is to replace the current candidate solution s within a search space S of complete candidate solutions by some neighboring candidate solution. The neighborhood $N(s)$ of a candidate solution $s \in S$ is defined by all candidate solutions that can be obtained by applying specific modifications or "movements" to s.

Neighborhoods are problem specific, but for many problems, standard neighborhood concepts can be applied. One of these standard neighborhood concepts is the so-called k-exchange neighborhoods, where two candidate solutions are neighbored if they differ in at most k solution components. For implementing such neighborhoods, one needs to identify, for a given problem, appropriate solution components. Intuitively, a solution component corresponds to an atomic relationship, assignment, or selection that characterizes a candidate solution. For example, in the well-known traveling salesman problem (TSP), solution components refer to the fact that a node j is visited directly after a node i in a tour, that is, to an edge (i, j); in the quadratic assignment problem (QAP), a solution component would correspond to

the assignment of a facility to a specific location, that is, an individual assignment. In a k-exchange move for the TSP, one would change a set of k edges by a different set of k edges, and for the QAP, one would change at most k individual assignments of facilities to locations.

Of critical importance is the size of neighborhoods: Large neighborhoods have the advantage that better-quality candidate solutions may be reached in one step of the algorithm, which comes, however, with the disadvantage of increasing the time it takes to actually search for such candidate solutions in large neighborhoods. To balance these two aspects, over the years a large set of techniques have been proposed that try to either define special-purpose neighborhood structures that allow to design algorithms for searching them efficiently or to heuristically explore very large neighborhoods trying to identify good candidate solutions there but without guarantee of finding the best one. For an overview of very large neighborhood search methods, we refer to Ahuja et al. [3].

The widest spread perturbative local search method is probably iterative improvement (also called iterated descent, hill climbing, etc., in the literature). It replaces a current candidate solution by an improving neighboring one and repeats these steps until a locally optimal candidate solution has been found, that is, a candidate solution without any improving neighbor. In the following, we will indicate by $s^* \in S^*$ a locally optimal candidate solution from the set of locally optimal candidate solutions S^*; more in general, we will refer to the set S^* as the possible output set of search methods that are not necessarily iterative improvement methods. Various variants of this basic algorithm exist differing in the order in which the neighborhood of a candidate solution is searched, which neighboring candidate solution replaces the current one, and so on. The most common mechanisms for these rules are the so-called best- and first-improvement rules, which, respectively, replace the current candidate solution by a most-improving candidate solution in $N(s)$ or by the first one encountered when scanning the neighborhood. The specific choice for these pivoting rules can have a significant impact on the quality of the local optima generated and the computational effort it takes to reach local optima. Similarly, for local search algorithms, speed-up techniques such as incremental move evaluations are of crucial importance for their efficiency [1, 60].

Local optima have an ambivalent importance in optimization. When compared to the candidate solutions in the full search space S, locally optimal solutions are of, on average, much better quality, and there are much less candidate solutions in S^* than in S. Hence, identifying local optima by an iterative improvement algorithm may seem a priori a good idea. However, iterative improvement algorithms are stuck in local optima, and even when several candidate solutions in S^* are sampled independently, the so sampled local optima may still be of rather poor quality [114]. Thus, other mechanisms than independent sampling in S^* are required to allow a highly performant search process. The search for such mechanisms led to a large number of proposals and studies of general mechanisms to pursue the local search beyond the traps of local optimality [47] or on the development and study of rather different search mechanisms, such as populations, which try to focus on a more global search behavior. These mechanisms are commonly referred to as

metaheuristics [45, 46] or general-purpose stochastic local search methods [60]. Within the context of ILS, of particular relevance are trajectory-based metaheuristics that at each step modify a single solution. This includes methods such as tabu search, simulated annealing, or guided local search. Such metaheuristics can be used in a straightforward way as an improvement method in ILS algorithms.

Iterated Local Search Framework

The main principle of ILS is to generate a sequence of candidate solutions that is obtained by iterating through solution perturbations and subsequent applications of an (local) improvement method. An additional acceptance criterion decides from which candidate solution seen during the run of the algorithm this sequence is continued. Typically, the improvement method is an iterative improvement method or a trajectory-following metaheuristic such as tabu search or simulated annealing. The goal of the perturbation is to introduce a modification into the current candidate solution that is larger than the modifications that are done in the local search phase, thus effectively allowing the search to escape local optima or specific search space regions.

An algorithmic outline of ILS is given in Fig. 1. After generating an initial candidate solution and improving it by a local search, the main loop is invoked. Within the main loop, first a (typically locally optimal) candidate solution $s^* \in S^*$ is perturbed leading to a candidate solution $s' \in S$. After improving this solution and obtaining a new solution $s^{*\prime} \in S^*$, an acceptance criterion decides whether to continue the search from s^* or $s^{*\prime}$. It is also possible to consider in the perturbation and the acceptance criterion aspects of the search history, for example, to adjust the perturbation strength or the choice done in the acceptance criterion; in that case, one would extend the corresponding procedures to Perturbation(s^*, *history*) and AcceptanceCriterion(s^*, $s^{*\prime}$, *history*).

An important advantage of ILS when compared to repeatedly starting the LocalSearch method from random initial candidate solutions is that (i) intermediate candidate solutions obtained through the perturbation are often of better quality and can lead to better locally optimal solutions, and (ii) in the case of iterative improvement methods, the subsequent local search may terminate more quickly than when starting from random initial candidate solutions, leading to the exploration of a larger number of local optima within a given computation time. Hence, more local

Fig. 1 Algorithmic outline of iterated local search (ILS)

```
procedure Iterated Local Search
    s_0 = GenerateInitialSolution
    s* = LocalSearch(s_0)      % optional
    repeat
        s' = Perturbation(s*)
        s*' = LocalSearch(s')
        s* = AcceptanceCriterion(s*, s*')
    until termination condition met
end
```

optima and local optima of a better average quality may be generated in an ILS algorithm, increasing the chance to find very high-quality solutions.

A further advantage of ILS is its ease of a first implementation, especially if an improvement method is already available. A first version of an ILS algorithm can be obtained by adding perturbations in some higher-order neighborhoods than the ones used in the local search and a reasonable first choice is to accept only better or equal quality solutions in the acceptance criterion. Such a version can be obtained by adding few lines of code to an existing improvement method and be the basis for an algorithm engineering effort to generate a high-performing ILS algorithm. In fact, ILS is very malleable and can range from very straightforward implementations to rather well-engineered algorithms when problem-specific details are taken into account.

Generally speaking, ILS is a modular approach, and the behavior of an ILS algorithm is determined by the four procedures GenerateInitialSolution, Perturbation, LocalSearch, and AcceptanceCriterion and their interaction. In what follows, we discuss the main issues arising for these procedures.

GenerateInitialSolution This procedure determines the starting point of the search, and it may have a significant impact during the initial search phase. Generally, candidate solutions generated by a good constructive heuristic should be preferable as often the better the quality of the initial candidate solution, the better is the quality of the local optima encountered. In addition, an iterative improvement algorithm requires usually less steps to reach a local optimum when starting from a better-quality solution. However, if the other procedures are well designed, the influence of the initial candidate solution is expected to be little especially for longer run times.

Perturbation The perturbation transforms one complete candidate solution into another complete candidate solution. Its task is to modify a current candidate solution and to generate a new, promising starting solution for the next local search application. To allow an effective escape from local optima, the perturbation should be larger than or at least different in nature from the modifications that are applied in the local search algorithm.

An important factor is the size of the perturbation, which can be measured by the number of solution components that are changed. If the perturbation is too small, it may quickly be undone by the next local search. If the perturbation is too strong, that is, too many solution components are modified, then much of the quality and the structure of the current candidate solution may be lost; in that case, the algorithm can become akin to a random restart algorithm, which is known to be usually of poor performance. Unfortunately, it depends typically on the problem and often on the specific problem instances how strong perturbations should be [81]. One possibility to adapt the perturbation strength may be either to do a proper parameter tuning, an adaptive scheme following ideas of reactive search [10], or to adapt the perturbation strength using simple schemes such as those of basic variable neighborhood search [52].

The simplest way of perturbing a candidate solution is by applying random perturbation moves. A disadvantage of choosing the perturbing moves uniformly at random is that one may lose significantly in solution quality. An alternative is to use biased perturbations and to introduce problem knowledge in this way. Biased perturbations may make use of heuristic information on the solution components that are changed. This can be done in various forms, for example, by removing preferentially solution components that have a high contribution to the cost of a candidate solution, or by introducing new solution components that have a low cost contribution. Closely related to ILS is the iterated greedy method that relies on a destruction and reconstruction cycle that is managed through constructive heuristics [110]; in fact, this cycle may be seen as a perturbation in the ILS sense. A different possibility is to introduce more complex perturbations by re-optimizing parts of a candidate solution [80] or applying data perturbations [25].

LocalSearch The local search procedure is of crucial importance for the performance of an ILS algorithm, which will benefit strongly from good choices and implementations for this procedure. A first crucial issue is the quality of the solutions that the local search algorithm produces. A priori, one may expect that the better the quality of the solutions that are generated by the local search, the better the results of the ILS algorithm. However, this reasoning would not take into account the execution time of the local search. In fact, a crucial aspect for the appropriate choice of a local search method is the trade-off between the quality of the solutions it generates and the computation time it takes to find these solutions. Given a fixed computation time, a fast local search may be applied more often and, thus, generate a larger number of candidate solutions that may be of less average quality than candidate solutions obtained by a more time-consuming local search procedure. Which choice is best depends on the particular problem or type of problem instances and has to be determined on an experimental basis.

Another important aspect is whether the local search is used as a black box or whether it is open and can be integrated in possibly more profitable ways into an ILS algorithm. One example is the exploitation of the don't look bit technique [15, 68]; we refer to section "TSP Example" for more details on this technique and how to integrate it with the perturbation. Another example requiring access to the local search is the adoption of techniques of tabu search by declaring solution components changed by the perturbation as tabu for a number of local search steps; this may help to avoid immediately undoing the perturbation.

The local search in an ILS algorithm is not limited to an improvement method, but any (even nonlocal search) method that takes as input a complete candidate solution and returns a potentially improved candidate solution upon completion could be used. Common examples include the usage of tabu search, simulated annealing, or dynamic local search algorithms as the local search. One can even think of using an ILS algorithm as the local search leading to a hierarchy of ILS algorithms [62, 81]. However, to allow a sufficient number of iterations of the ILS

algorithm, one needs to define a stopping criterion for the local search method, for example, by limiting the number of local search steps that are applied or the number of steps without improvement.

Finally, an advantage of a strong local search is that the sensitivity of the algorithm to the settings of numerical parameters is often reduced, making in this way the algorithm itself more robust. In this sense, ILS also benefits from the further development of local search techniques and the increased availability of powerful implementations based on very large neighborhoods [3].

AcceptanceCriterion One interpretation of the search behavior of ILS is that of a biased random walk in S^*, the space of local optima. The acceptance criterion has a crucial impact on the nature of this walk and, thus, the trade-off between exploration and exploitation. As the two extremes, one can have the acceptance criterion that only accepts improved solutions, leading to a search that can be characterized as a randomized iterative improvement algorithm in S^*. On the other extreme is the acceptance criterion that accepts any new solution independent of its quality, leading to a random walk in S^*. Clearly, there are many intermediate choices between these extremes, and various have been explored in the literature. One example is the Metropolis condition [95], which accepts a new, worse solution $s^{*\prime}$ with a probability $\exp\{f((s^*) - f(s^{*\prime}))/T\}$, where T is a parameter; a better or equal quality solution is accepted always. The parameter T may be left fixed throughout a run of ILS or be varied as done in simulated annealing.

Various other choices have been made such as combining short random walks with occasional backtracks to the best candidate solution found so far [26], resulting in a specific usage of the search history within the acceptance criterion. Another simple example for the usage of the search history in the acceptance criterion is the occasional restart of the search from a new initial candidate solution, which can lead to much improved algorithm behavior [117, 119].

If one is to reach high performance with an ILS algorithm, the interactions among the components need to be taken into account. The two main interactions are the following. First, the perturbation should complement the local search and ideally introduce moves that cannot be easily done by the local search. This results in two main advantages: (i) it is easier to avoid returning to the previous locally optimal candidate solution, and (ii) the structure of the candidate solution may be changed in a way that is not possible for the local search to do. Second, the acceptance criterion and the perturbation should be well balanced. In fact, both components can be biased toward a more explorative or more exploitative search behavior: large vs. small perturbations or non-strict (random walk) vs. strict (improvement) acceptance criteria. Finding the right balance between intensification and diversification of the search is one of the key issues in the design of any effective SLS algorithm, and one advantage of ILS is that the impact specific choices of the algorithm components have on this balance is rather easy to grasp. Thus, ILS is a method for which the SLS algorithm engineering process [122] can follow a well-guided direction.

Some Examples of Iterated Local Search Algorithms

We introduce example applications of ILS algorithms to well-known combinatorial optimization problems, the traveling salesman problem (TSP), the quadratic assignment problem (QAP), and the permutation flow-shop scheduling problem (PFSP). Example implementations for these ILS algorithms are available from http://iridia.ulb.ac.be/~stuetzle/Code.

TSP Example

Traveling salesman problem (TSP). The TSP is given as a graph $G = (N, E)$, where N is the set of $n = |N|$ nodes and E is the set of edges that fully connects the nodes. To each edge (i, j) is associated a distance d_{ij}. Here we assume that the distance matrix is symmetric, that is, we have $d_{ij} = d_{ji}$ for all $(i, j) \in E$; these types of TSP instances are called symmetric and are among the most widely studied types of TSP instances. The objective in the TSP is to determine a Hamiltonian cycle of minimal length. Such a cycle can be represented by a permutation $\pi = \langle \pi(1), \ldots, \pi(n) \rangle$, where $\pi(i)$ is the node index at position i. The objective function to be minimized is

$$\min_{\pi \in S} \; d_{\pi(n)\pi(1)} + \sum_{i=1}^{n-1} d_{\pi(i)\pi(i+1)} \tag{1}$$

where S is the search space consisting of the set of all permutations.

The TSP is one of the most famous examples of ILS applications, which is due to the important role it plays in the historical development of the method [11,12,67,86] and due to the very high performance ILS algorithms have reached for the TSP [4,5,56,57,68].

A basic ILS algorithm for the TSP could be the following. The initial candidate solution is generated by a constructive heuristic such as the nearest-neighbor heuristic or the greedy heuristic. For the local search, a first-improvement method in a k-edge-exchange neighborhood (typically, k equal two or three) is used. The perturbation is implemented by the so-called double-bridge move, which is illustrated in Fig. 2; the double-bridge move removes four edges and replaces them with four other edges so that in the schematic view of the move, the characteristic double bridge is formed. The acceptance criterion forces the cost to decrease, that is, only better-quality candidate solutions are accepted.

Such a basic ILS algorithm for the TSP reaches high-quality tours, but its performance can be improved by alternative choices for the components. ILS algorithms for the TSP usually benefit from a local search with better performance. For example, by replacing the local search algorithm in the two- or three-edge-exchange neighborhood with the Lin-Kernighan heuristic [77], better performance is usually achieved. It is interesting to note that the double-bridge move was considered within the Lin-Kernighan heuristic, but many implementations of it did not implement it. It is a type of move that cannot be easily achieved by a concatenation of moves in smaller neighborhoods or by the way complex moves in

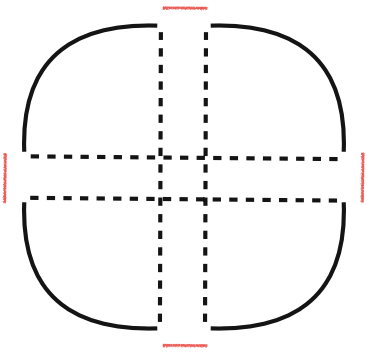


Fig. 2 Schematic view of the double-bridge move. Four edges (continuous, *red lines*) are removed from the current tour, and the four remaining tour segments are reconnected as shown by the *dashed line* in the schematic view of the move

the Lin-Kernighan heuristic are generated, and, thus, it is complementary to standard local search implementations. A main advantage of the double-bridge move is that it only leads to a small increase of the tour length as only four edges are replaced and the subsequent local search typically needs only few improvement steps to reach a new local optimum. In practice, the double-bridge move has found to be rather effective even for very large-sized TSP instances with many thousands or millions of nodes [5, 68].

For speeding up the search, direct access to the local search through the usage of the well-known don't look bit technique [15] was found to be useful. The don't look bit technique associates one such bit to each node in a TSP instance and starts a local search centered around a node for an improving move only if the don't look bit is set to one (turned on). If this local search is not successful, the don't look bit is turned off by setting it to zero. If a move improves a candidate solution, all end points of the edges involved will have set their don't look bit to one again. This technique leads to a particularly significant speed-up, when integrated with the perturbation by resetting only don't look bits to one for nodes that are directly affected by the perturbation [68] or additionally to some close-by ones [117].

An acceptance criterion that allows only moves to better or equal length tours leads to a strong intensification behavior of the search. While this results in very good performance for short and medium run times, for longer ILS runs, a stronger diversification through, say, additional restarts improves algorithm performance [117, 119].

QAP Example

Quadratic assignment problem (QAP). In the QAP we are given two matrices F and D corresponding to some kind of flow exchanged between items and a distance matrix between possible locations of the items, respectively. Let f_{ij} be the flow exchanged between items i and j and d_{kl} be the distance between locations k and l. The task in the QAP is to assign items to locations so that the cost given by the sum of flows times the associated distances is minimized. Assuming the number of items is equal to the number of locations, a candidate solution to the QAP can be represented by a permutation π, where $\pi(i)$ gives the location to which item i is assigned. The objective function to be minimized for the QAP can then be written as

$$f(\pi) = \sum_{i=1}^{n} \sum_{j=1}^{m} f_{ij} \cdot d_{\pi(i)\pi(j)} \tag{2}$$

Similar to the TSP, the QAP has played a central role in combinatorial optimization. Differently from the TSP, for which optimal solutions can be found for very large instances (the largest being solved optimally having 85,900 nodes [6]), QAP instances can only be solved optimally for relatively small instance sizes of around 30; there are only few exceptions for very particularly structured instances for which an instance of size 128 could be solved [41].

A basic ILS algorithm for the QAP can be implemented as follows. An initial candidate solution for the QAP can be generated uniformly at random, as no constructive algorithms generate generally very high-quality candidate solutions. As local search algorithm, an iterative improvement algorithm in the two-exchange neighborhood, where at each step the locations of two items are exchanged, is used. More formally, two candidate solutions π and π' are neighbored if for exactly one index pair (i, j) we have $\pi'(i) = \pi(j)$ and $\pi'(j) = \pi(i)$ and for all other indices $l \neq i, j$ we have $\pi'(l) = \pi(l)$. As a perturbation, a random move in a k-exchange neighborhood, $k > 2$, can be chosen and, as acceptance criterion, one accepts only better-quality candidate solutions.

Such a straightforward ILS version was tested by Stützle [117] and Lourenço et al. [81], and it was shown that its relative performance with respect to other choices of the main ILS components depended on the specific type of QAP instances. In fact, the specific structure of QAP instances as identified by different distributions of the entries of the flow or the distance matrices can induce widely different behavior [125]. For example, the benchmark instances that have been generated randomly assuming uniform distributions of the matrix entries result in landscapes with many local optima that are spread across the whole search space. Such a landscape requires strong search diversification, which can be achieved in ILS by using an acceptance criterion that accepts every new local optimum independently of its quality and helps to improve the performance on such instances even when using only very small perturbations [81, 118]. Differently, for randomly generated instances that resemble real-life instances, which exchange most flow among relatively few items, accepting only better-quality candidate solutions combined with some occasional restarts was found to result in high performance [81, 118]. Further studies on the QAP have shown that plain ILS algorithms reach very high performance for a variety of QAP instances and that population-based extensions of ILS algorithms are state of the art for tackling various classes of QAP instances [118].

Permutation Flow-Shop Scheduling Example

Permutation flow-shop scheduling problem (PFSP). The PFSP arises in many practical situations as it is common to have production lines where machines are disposed in series. In the PFSP are given n jobs $J_1, J_2, \ldots J_n$, each consisting of m operations that are to be executed on m distinct machines $M_1, M_2, \ldots, M_m$. Each job needs to be processed on

every machine, and all jobs pass through the machines in the same machine order, that is, first on machine M_1, next on machine M_2, and so on until machine M_m. The nonnegative, known, and deterministic processing time of job J_i on machine M_j is denoted by p_{ij}. The PFSP enforces that all jobs are processed on every machine in the same order; therefore, a permutation of the job indices is enough to define the processing sequence. In the basic PFSP, one assumes that jobs are available for processing at time zero, pre-emption is not allowed, each machine can process only one job at a time, and each job can be processed by at most one machine at a time, and that the capacity of buffers between machines is unlimited. A common objective is to minimize the completion time of the last job in the permutation, also known as makespan ($C_{\max}$).

The PFSP with the makespan objective is one of the most widely studied scheduling problems with hundreds of papers published; for reviews on it, see [40, 42, 55, 110]. Furthermore, it is the basis of many other variants and extensions.

A first ILS algorithm for this problem has been proposed by Stützle [116]. It used the NEH heuristic [102] to generate an initial candidate solution. The NEH heuristic is a state-of-the-art constructive method for the PFSP with some recent variants being proposed only in 2014 [38]. It is a heuristic that builds a schedule by inserting at each step a next job in a position of the partial schedule where it least increases the objective function value. Using the speed-up techniques proposed by Taillard [124], the NEH heuristic has a computational complexity of $O(n^2m)$. As an iterative improvement method in the ILS algorithm, the insert neighborhood was used, which exploits the same speed-ups as implemented for NEH. The insert neighborhood consists of all sequences that can be obtained by removing a job at position i and inserting it in all other possible positions; this results in a neighborhood of size $n \cdot (n-1)$. If one removes the job with index $\pi(i)$ from position i and inserts it at a position j, for $i < j$, one obtains $\pi' = (\pi(1), \ldots \pi(i-1), \pi(i+1), \ldots \pi(j), \pi(i), \pi(j+1), \ldots, \pi(n))$ and for $i > j$ one obtains $\pi' = (\pi(1), \ldots \pi(j-1), \pi(i), \pi(j), \ldots \pi(i-1), \pi(i+1), \ldots, \pi(n))$. The local search implemented in Stützle [116] used a kind of first-improvement neighborhood scan. It removes a job J_i from the sequence and checks all $n-1$ possible insertion positions; if several result in improvements, the position is taken that leads to the largest reduction in the makespan. One scan of the neighborhood consists of repetitions of this process for all possible positions i. The neighborhood scans are then repeated until a local optimum with respect to the insert neighborhood is found. As the perturbation, a mix of two contiguous swap moves, and one interchange move was proposed. In a contiguous swap move, a permutation $\pi = (\pi(1), \ldots, \pi(i), \pi(i+1), \ldots, \pi(n))$ is modified to $\pi' = (\pi(1), \ldots, \pi(i+1), \pi(i), \ldots, \pi(n))$; in an interchange move $\pi = (\pi(1), \ldots, \pi(i), \ldots, \pi(j), \ldots, \pi(n))$ is modified to $\pi' = (\pi(1), \ldots, \pi(j), \ldots, \pi(i), \ldots, \pi(n))$. A further restriction was imposed on the positions involved in the interchange move by enforcing $|i-j| \leq \max\{n/5, 30\}$ to avoid too strong disruptions. Finally, as the acceptance criterion, the Metropolis condition with a fixed temperature was suggested.

The performance of this ILS algorithm was found to be superior to the best methods for the PFSP published until then. The excellent performance was confirmed in the extensive study of heuristics for the PFSP under makespan minimization by Ruiz

and Maroto [110], where it was found to be the top-performing heuristic. However, the ILS is outperformed by the closely related iterated greedy method [111].

Historical Development of Iterated Local Search

ILS has a long history, and the ideas underlying the method can be traced back to several approaches. The most influential have been the early developments for tackling the TSP. Baum [11, 12] has proposed an algorithm he called *iterated descent*, which used an iterative improvement method based on two exchanges as the local search and random three exchanges as the perturbation and forced the tour length to decrease. His results were not very impressive, but inspired Martin et al. [86] to propose their *large-stop Markov chains* (LSMC) algorithm. The name of this algorithm stems from the acceptance criterion, which corresponds to the Metropolis condition. In first instantiations, LSMC used a three-exchange neighborhood in the local search but was later extended to use the Lin-Kernighan heuristic. One contribution of the LSMC algorithm was the introduction of the double-bridge move as the perturbation, which is used in many other ILS algorithms for the TSP. The results with the LSMC algorithm gave a major leap in the performance of TSP heuristics. Further refinements of this approach, in particular, by Johnson [67] and Johnson and McGeoch [68] on one side and Applegate et al. [4] on the other side, have defined for a long time the state-of-the-art TSP heuristics. Differences in these implementations concern, on the high-level ILS side, mainly the choices for the acceptance criteria and different choices of the perturbation operators, but decisive are also implementation details of the local search and the data structures used for tackling large instances. Noteworthy is the experimental study by Applegate et al. [5] who performed tests on very large TSP instances with up to 25 million cities and the more recent work of Merz and Huhse [94], which is particularly well suited for very large TSP instances under severe time constraints.

In a number of papers, specific choices for ILS algorithms for the TSP have been considered and analyzed. Codenotti et al. [25] study complex perturbation schemes based on data perturbation. Hong et al. [58] examined the impact the perturbation strength has on ILS performance, indicating that for some instances perturbations larger than those introduced by the double-bridge move are beneficial; they also studied population-based extensions of ILS algorithms for the TSP. Katayama and Narihisa [70] proposed to use candidate solutions different from the incumbent one to generate directed perturbations and showed that this idea can lead to very competitive results when compared to an iterated Lin-Kernighan algorithm. Stützle [117] and Stützle and Hoos [119] have analyzed the run-time behavior of ILS algorithms for the TSP, showing that the most common implementations show stagnation behavior for long runs. As a simple remedy to this behavior, they have proposed occasional restarts of the ILS algorithms [117,119]. Finally, there are some approaches for TSP solving that in their iterated version do not follow necessarily the main ILS steps but are, say, inspired from it. The most notable example is probably Helsgaun's iterated version of his Lin-Kernighan implementation [56].

It generates new starting candidate solution through a constructive mechanism, which is strongly biased toward generating a candidate solution that is close to the incumbent candidate solution.

The ILS principle has been discovered before its first applications to the TSP. Baxter [13] proposed an algorithm for a location problem that made repeated use of a data perturbation technique to create new starting solutions for a local search algorithm. However, it appears that these initial approaches have not been the main inspiring source for other ILS approaches and that this work has been overlooked for quite some time. In fact, most early ILS adaptations to problems other than the TSP seem to have been inspired by the proposals of ILS algorithms for the TSP.

Among the first problems other than TSP tackled by ILS is graph bipartitioning. Martin and Otto [84, 85] reported for specific types of graphs better performance with their ILS algorithm than for simulated annealing heuristics. Many other early applications of ILS algorithms have been for scheduling problems. Lourenço [80] presented an innovative idea that combined ILS with exact algorithms for job-shop scheduling. In her approach, the exact algorithm is used to solve exactly subproblems on one or two machines, keeping the sequence on other machines fixed; the so modified candidate solution is then used as the perturbed one.

It is important to stress that ILS algorithms have been published and made popular under a variety of different names, ranging from problem or algorithm specific names such as iterated Lin-Kernighan [67,68] or chained Lin-Kernighan [5] to generic names such as large-step Markov chains [86], chained local optimization [85], iterated descent [11, 12], iterated hill climbing [100], or others. In the local search template proposed by Vaessens et al. [129], ILS was referred to as a multilevel point-based local search. Even today, apparent variations of the ideas underlying ILS are published under new names such as breakout local search [14], creating possibly confusion with the earlier proposed breakout method [98], which relies on a different mechanism for escaping from local optima. A review and unification of the various papers published in the literature has been proposed by Lourenço et al. [81] in their 2002 book chapter on ILS. This chapter has led to a more coherent view of the method, has discussed the main ingredients of ILS algorithms, and has shown trade-offs in the design of effective ILS algorithms.

Relationship of ILS to . . .

ILS, being a relatively old method, has a number of links to other well-known methods, which we will shortly discuss in this section. We focus our discussion on relationships to simple, hybrid, and population-based metaheuristics (or stochastic local search methods), following the classification of Hoos and Stützle [60]. Within this classification, ILS belongs to hybrid SLS methods, which can be characterized as methods that interleave search steps in different neighborhoods or that combine different types of procedures in an interleaved fashion. As such, most links of ILS are within that group.

. . . Simple SLS Methods

Simple SLS methods typically move within one type of neighborhood. They can be seen as direct extensions of perturbative local searches that avoid local optima either by occasionally accepting moves to worse candidate solutions or modifications of the evaluation function during the search. Well-known examples for the former are simulated annealing [24, 71] and tabu search [47], while a well-known example for the latter is guided local search [132].

A first relation is that any of these methods may play the local search part within an ILS algorithm, replacing simpler iterative improvement methods. While iterative improvement methods do have a natural stopping condition, namely, hitting a local optimum, for simple local search methods, in principle, arbitrary computation times can be invested. This leads to the task of balancing the intensification obtained through the simple SLS method and the diversification given through perturbation steps and the acceptance criterion. A main task therefore for the algorithm designer is to determine an appropriate computation time for the SLS method to obtain a good trade-off between the computation time and solution quality.

Another relation is the usage of diversification measures in methods such as tabu search. For example, the random shake-up ideas [45] are directly linked to perturbations in ILS. The well-known reactive tabu search algorithms invoke occasional sequences of random moves to provide a means for search diversification [9], which can directly be seen as a random perturbation whose size is adjusted in dependence of features of the search process. Several ideas from the area of tabu search and, in particular, the memory usage explored there [47] may provide a source of inspiration of how to improve ILS algorithms or to define schemes for adapting parameters at run time [10].

. . . Hybrid SLS Methods

Hybrid SLS methods combine more than one basic search strategy into an overall method. For example, ILS combines typically one search strategy in the local search with a different type of search strategy in the perturbation. From a high-level perspective, one point ILS has in common with many other hybrid SLS methods is that it combines a local optimization with search steps oriented toward diversification. These diversifying steps may be based on constructive or perturbative search steps.

Considering hybrid SLS methods that obtain diversification through constructive procedures, the closest to ILS is the iterated greedy algorithm, if the latter uses a local search phase. In iterated greedy methods, a new candidate solution is obtained by removing from a complete candidate solution $s \in S$ some solution components, obtaining a partial solution s_p, from which in turn a solution construction process is performed. While such iterated greedy algorithms may result in reasonable performance without improving the newly constructed candidate solution by a local search phase, in many cases, an additional perturbative local search improves

performance. In that latter case, the destruction-construction cycle may be seen as a (directed) perturbation in the ILS sense. Another link between the two methods is to see both as creating an iterative process that in the ILS case iterates across local search applications and in the iterated greedy case iterates across applications of constructive algorithms. A different perspective on the mechanism of iterated greedy algorithms is taken by large neighborhood search techniques [115]. They consider also the removal of solution components, but then the subsequent completion of the resulting partial solution may be done with exact techniques, originally taken from constraint programming, or also constructive-type approaches. In any case, large neighborhood search interprets the completion of a partial solution as the exploration of a large neighborhood, hence the name of the method. In a sense, the application of one iteration of LNS would correspond in ILS terms to one (complex) perturbation that moves a solution $s \in S$ (typically a local optimum) to another complete candidate solution $s' \in S$. More details on iterated greedy and further related methods such as large neighborhood search can be found in the chapter on this topic by Stützle and Ruiz in this handbook. The use of constructive mechanisms to provide search diversification also underlies greedy randomized adaptive search procedures (GRASP) [36, 108], which in their basic form create many independent starting points for a local search through randomized greedy constructive searches. GRASP is clearly distinct from the basic principle underlying ILS, as it does not create a biased walk in the space of local optima. Later extensions of GRASP add various mechanisms with the goal of making it more performing. In fact, some extensions reuse parts of candidate solutions and, hence, move GRASP more toward mechanisms that underlie ILS; see Resende and Ribeiro [108] for an overview of GRASP.

The hybrid SLS method that shares most similarities with ILS is (basic) variable neighborhood search (VNS) [52, 53, 97]. VNS is based on the principle of changing the neighborhood during the search to avoid getting trapped in local minima with respect to one specific neighborhood. While the rationale of the main search mechanism underlying VNS and ILS is rather different, many instantiations of VNS, in particular those related to basic, skewed, or general VNS, can be seen directly as specific instantiations of ILS algorithms. For example, in basic VNS, an iterative improvement process is performed in the smallest neighborhood. Larger neighborhoods are explored randomly, and this random exploration is interleaved with the iterative improvement search in the smallest neighborhood. This loop corresponds to the perturbation and local search steps in ILS algorithms, where the main specificity of VNS is to modify the strength of the perturbation following some fixed scheme in which the neighborhoods are explored: The scheme could be in an increasing or decreasing order and additionally taking into account step sizes. In early instantiations of a basic VNS, only candidate solutions that improve on the incumbent candidate solution after the local search phase are accepted, while in skewed VNS worse candidate solutions are accepted depending on how far the new candidate solution is from the incumbent one. Finally, general VNS is a variation of the basic VNS schemes, where the underlying local search can be a variable neighborhood descent algorithm.

. . . Population-Based SLS Methods

Population-based SLS methods are characterized by the use of a population of candidate solutions to drive the search. As such, they are clearly distinct from ILS, which is based on modifying a single search point. In addition, several population-based search methods such as evolutionary algorithms [8] and ant colony optimization [34] do not necessarily make use of local search algorithms. However, several links exist. In fact, many population-based algorithms can profit from the introduction of a local search phase as it was clearly shown for ant colony optimization [34], for scatter search [48], and for many evolutionary algorithms resulting in methods such as genetic (or evolutionary) local search [100, 127] or memetic algorithms [99]. ILS as well as the population-based methods can in that case be seen as sampling candidate solutions in the space of local optima.

ILS and memetic algorithms are linked by taking an extreme parameterization of the latter, in particular, using memetic algorithms with a population size of one. In that case, the mutation operator takes over the role of the perturbation and the selection scheme used for population replacement the role of the acceptance criterion. The usage of only mutation to generate candidate solutions has been called parthenogenetic algorithm [67]. In fact, even when using a population size of more than one individual and excluding recombination operations, in the context of memetic algorithms, good performance results are reported for some problems [93]; mutation-only candidate solution modifications are also often used in evolution strategies.

Considering the usage of populations as a convenient means of providing search space exploration, several authors have explored population-based variants of ILS algorithms [58, 117, 118, 126]. The approaches proposed by Hong et al. [58] and Stützle [117] maintain the usual perturbation scheme, applying perturbation at each iteration to a single candidate solution. The usage of time-varying distance bounds among the candidate solutions in the population as a specific diversity mechanism was explored by Stützle [118], resulting in high solution quality for the quadratic assignment problem. Thierens [126] uses the population to (i) perturb only solution components in which pairs of candidate solutions differ, similar to what was done in the genetic transformations proposed by Katayama and Narihisa [70], and (ii) to reduce the neighborhood size during the local search.

Applications

This section summarizes some noteworthy applications of ILS in addition to those mentioned in sections "Some Examples of Iterated Local Search Algorithms" and "Historical Development of Iterated Local Search." As the number of applications of ILS has strongly increased over the recent years, we do not give here an exhaustive list but mainly refer to recent publications for illustrating the progress of ILS. Here, we mention only papers where the authors have identified their algorithms explicitly as ILS algorithms, even though a larger number of published

algorithms would fit the framework of ILS without this being made explicit in the respective papers.

Iterated Local Search for Routing Problems

Among the first explicit ILS algorithms for vehicle routing problems (VRPs) are those proposed by Ibaraki et al. [65] and Hashimoto et al. [54]. In the first paper, the authors tackle the VRP with time windows, where a dynamic programming approach minimizes the penalties for time window violations. The ILS algorithm, which used various neighborhoods in the local search, was shown to reach high performance on instances with up to 1000 customers. Hashimoto et al. [54] tackle a VRP with time windows and time-dependent travel times and costs. They introduce various speed-ups and neighborhood restrictions and show the effectiveness of their algorithm compared to previous proposals in the literature. Vaz Penna et al. [131] consider a variant where the fleet of vehicles is heterogeneous, that is, it consists of vehicles with different characteristics such as capacities. Brandão examines two variants of an open vehicle routing problem with time windows [17] and multi-depots [18]. In the first one, he applies ILS with ejection chains and the incorporation of elite solutions. On the second one, he also uses an history-based information to decrease the chance of cycling. He shows that he obtains good results on the two problems. Melo Silva et al. [92] consider a VRP where split deliveries are allowed, that is, a customer's demand may be satisfied by splitting deliveries across various vehicles or tours; they report very high performance for their algorithm improving a large number of benchmark instances. Palhazi Cuervo et al. [105] tackle the vehicle routing problems with backhauls, where in addition to the consumers who get delivered from the depot, there are also suppliers that send goods to the depot. The authors propose a structurally simple ILS algorithm, where the main ingredient is a local search algorithm that uses multiple neighborhoods and allows oscillations between feasible and infeasible candidate solutions. The problem of the paper Osorio-Mora et al. [104] is the multiple depot cumulative capacitated vehicle routing problem. The authors propose an ILS and show in an experimental analysis that it obtains state-of-the -art results. Michallet et al. [96] consider a periodic VRP with time windows under the consideration of malevolence acts, and the goal becomes spreading the repeated visits to customers across their time windows. A VRP with multiple, incompatible commodities and multiple trips per work day is considered by Cattaruzza et al. [23]. They propose an effective ILS algorithm for this problems, which is shown to perform better than previous approaches. Máximo and Nascimento [90] propose an adaptive ILS with a certain diversification control. They demonstrate that they perform better than current state-of-the-art techniques on large capacitated vehicle routing problems. In Máximo et al. [91], they further develop the adaptive ILS to surpass other algorithms for heterogeneous fleet vehicle routing problem. Nguyen et al. [103] consider a two-echelon location routing problem, where two types of trips arise. One type of trips serves from a main depot a number of subordinate depots, which have to be located appropriately, and a second type of

trips delivers goods to customers from the subordinate depots. Vansteenwegen and Mateo [130] solve a cyclic inventory routing problem for a single vehicle. The goal is to minimize the costs of the distribution and the inventory costs at the customers. An efficient ILS algorithm for this problem was shown to outperform previous approaches for this problem. Laurent and Hao [75] considered a multiple depot vehicle scheduling problem, which arises in public transport. Related to routing problems is the team orienteering problem, which arises commonly as a problem faced by tourists when planning their trips. In the paper of Fernandes Muritiba et al. [37], they tackle a generalization of the traveling repairman problem. The problem arises when maintenance is necessary when considering the travel times between cameras and the traffic flow; the objective is the minimization of the total weighted latency. They propose an ILS and a branch and cut algorithm and show in the experimental results that the heuristic algorithm reaches very good results in particular for large instances.

Iterated Local Search for Scheduling Problems

Scheduling problems have been among the first problems tackled by ILS algorithms, and by now a large number of high-performing ILS algorithms have been proposed for many variants of scheduling problems including single- and parallel-machine scheduling problems as well as scheduling problems of flow- and job-shop type.

The first applications of ILS to single- and parallel-machine scheduling problems are due to Brucker et al. [19, 20]; they have used two neighborhood structures that are nested, where the outer neighborhood essentially is used for perturbation. ILS algorithms have reached particularly excellent results for the well-known single-machine total weighted tardiness problem (SMTWTP), for which the iterated dynasearch algorithms by Congram et al. [26] and the extension thereof by Grosso et al. [50] are state-of-the-art algorithms. den Besten et al. [31] have applied an ILS with a variable neighborhood descent local search also to the SMTWTP. Later ILS algorithms have been applied to the SMTWTP with additional sequence-dependent setup times in two independent works, reaching high performance when compared to other heuristics [123, 134]. An application of ILS to the parallel-machine total weighted tardiness problem has been presented by Della Croce et al. [30]. They use generalized interchange moves, ideas from dynasearch, and new large-scale machine-based neighborhoods for the local search to improve over the current state of the art. In the paper of Queiroga et al. [107], they introduced an ILS for the batch scheduling on a single machine. A comparison of their algorithm shows that it is competitive to other algorithms for this problem.

ILS algorithms have been proposed to tackle a variety of flow-shop scheduling problems, starting with the best-known variant that considers the minimization of the makespan. The first ILS algorithm for this problem [116] has already been described in section "Permutation Flow-Shop Scheduling Example." Some issues in the design of ILS algorithms for the permutation flow-shop problem have been considered by Juan et al. [69]. ILS algorithms have been adapted by

other researchers to variants of the flow-shop scheduling problem and shown high performance for the flow-shop problem with flowtime objective [32]. Later, the authors of [32] further improve their ILS algorithm by including a multi-restart perturbation strategy, where the ILS is continued from the best of a set of perturbations obtained from a local optimum [33]. State-of-the-art results for the same flowtime objective are obtained by [106] with ILS variants, including versions that entail populations. ILS has been used to solve more complex variants of the flow-shop problem. Yang et al. [135] presented an ILS algorithm for a flow-shop variant with several stages in series, where at each stage a number of machines is available for processing the jobs. Ribas et al. [109] deal with the blocking flow-shop problem and propose an ILS procedure where the local search combines moves in different neighborhoods as does the perturbation step. M'Hallah [101] tackles the flow-shop scheduling problem with the objective of minimizing the earliness and tardiness, where due dates are distinct. An ILS algorithm is presented, which uses a variable neighborhood descent in the local search phase. Differently, the author of Ferone et al. [39] applies an ILS to a distributed assembly permutation flow-shop problem. Here, multiple production factories are concerned in the first stage, and in a subsequent second stage, they produce a final product. They consider the objective of the minimization of the makespan and propose an algorithm with a biased-randomization technique. Geiger [43] proposes the usage of an ILS algorithm for tackling the multi-objective flow-shop scheduling problem, obtaining promising solution quality despite the simplicity of the proposed approach. Finally, complex hybrid flexible flowline problems are studied by [128], where ILS approaches are intermingled with other schemes to obtain high-quality candidate solutions for scheduling problems that are very close to real settings found at production floors.

As mentioned before, the first scheduling problem tackled by ILS was the well-known job-shop scheduling problem [80]. The ILS algorithm by Kreipl [73] for the total weighted tardiness job-shop scheduling problem reached high performance, being surpassed only quite some time later by an evolutionary algorithm that integrated an ILS algorithm as the local search operator [35]. An ILS algorithm is underlying the generic approach proposed by Mati et al. [88], which tackles job-shop problems with regular objectives, that is, objectives whose values increase along with an increase of the completion times of the jobs. The proposed ILS algorithm, which uses a perturbation with a size chosen uniformly at random and an acceptance criterion that accepts every new candidate solution, was shown to reach very high performance for a number of criteria, including weighted tardiness and weighted completion time. Calmels [22] proposes an ILS for a complex scheduling problem that involves nonidentical parallel machines and tool switching. The approach uses a mixed-integer programming approach, construction heuristics, and iterated local searches, and in the experimental section he shows that the ILS is the best for large instances. The paper of Aguiar Santos et al. [2] shows an ILS search for tackling the minimization of the maximum completion time of jobs and the total delay time of the jobs in an unrelated parallel-machine scheduling problem with job deterioration. They compared their problem to two evolutionary algorithms, and they get a significantly improved performance with the multi-objective version of iterated local search.

Iterated Local Search for Other Problems

ILS algorithms have been applied to tackle many other problems, illustrating the wide range of possible applications of the ILS principle. Corte and Sörensen [29] use an ILS algorithm for designing a water distribution network, obtaining excellent computational results despite the fact that the proposed algorithm has a much simpler structure than many of the other proposed methods for the same task. Results with an ILS method for gravity-fed water distribution network design are also reported by Martinho et al. [89]. Their algorithm is shown to ameliorate on many state-of-the-art algorithms in shorter times. Grosso et al. [51] propose an ILS algorithm for the maximin Latin hypercube design problem, which consists in assigning positions to n points in a k-dimensional grid such that no two points have a same coordinate and the distance between the closest pair of points is maximized. A possibility to improve the state estimation for the monitoring of power systems is to insert power measure units into the network at appropriate positions. Hurtgen and Maun [61] propose an ILS algorithm for minimizing the size of the configuration to reach full observability of the network. While virtually all ILS applications have been to deterministic problems, Grasas et al. [49] consider adaptations for ILS to integrate simulation in order to be able to tackle stochastic combinatorial optimization problems. The approach is illustrated with some example application of the proposed SimILS framework. The integration of information from lower bounding techniques into an ILS algorithm is considered by Buson et al. [21] for solving the fixed-charge transportation problem, which extends the transportation problem by the consideration of fixed costs for sending a flow from some origin to a destination. The reduced costs from the lower bounding are used to guide a restart phase of the algorithm. Lai and Hao [74] apply ILS to obtain high-quality candidate solutions to the maximally diverse grouping problem, where the goal is to partition the vertices of an edge-weighted and undirected complete graph under some constraints on the group size. Wolf and Merz [133] develop an ILS algorithm to find a symmetric connectivity topology with minimum power consumption in wireless ad hoc networks. Benchmark results on large instances with up to 1000 nodes show that their algorithm outperforms other heuristics that were previously used to solve this problem. For the prize-collecting generalized minimum spanning tree problem with a two-level solution, the authors of Contreras-Bolton and Parada [27] introduced an ILS which proposes in the first level one vertex from each cluster and in the second level constructs a minimum spanning tree with the vertices from the first subproblem. ILS has been applied to image analysis tasks, and Cordón and Damas [28] applied it to image registration, obtaining promising initial results. Imamichi et al. [66] apply ILS as a method to improve the placement of irregular polygons in a rectangular surface with the goal of minimizing, for a fixed width, the length of the container so that no polygon overlaps with any other or protrudes the container. The proposed approach allows overlaps but tries to minimize them using nonlinear programming methods and swaps between polygons. In the paper by Schettini et al. [113], a capacitated demand-driven timetabling problem is

tackled, which also has a demand-driven nonperiodic timetabling problem for a two-directional metro line. The objective is to minimize the total passenger waiting time. They proposed an ILS and obtain excellent performance by three instance sets given by the metro of Beijing, Madrid, and Milan.

Few papers have considered the adoption of the ideas underlying ILS for tackling continuous optimization problems. Kramer [72] has explored a simple continuous optimization algorithm, which embedded Powell's methods into an ILS for continuous optimization. Liao and Stützle [76] use ILS as one of the component algorithms in their hybrid, competition-based approach for continuous optimization. This approach decides in a preliminary competition phase which of two continuous optimizers, an ILS algorithm or IPOP-CMAES [7] to execute and then, in a second phase, continues with the execution of the winning algorithm. This approach was a winner of the CEC 2013 benchmark competition for real-parameter optimization.

Conclusions

In the initial phases of metaheuristic research, many efforts were dedicated toward developing and refining new metaheuristic methods and on studying their performance and behavior. As such, the focus was on rather pure applications of the respective methods to tackle computationally hard problems. Iterated local search contributed by relying on a clear principle that is easy to identify and that leads usually to high-performing algorithms. In addition, ILS algorithms are relatively intuitive to design and at the same time malleable. Therefore, iterated local search served often as a basis for a larger algorithm engineering effort if high-performing heuristics are desired. This approach was very successful, as shown by the various problems for which ILS algorithms have been or still are state of the art. In later research efforts in the metaheuristics area, often hybrid methods were proposed that mix in one algorithm concepts from different principles to derive effective algorithms [16, 83]. Even if this trend somehow blurs the differences behind the methods, it is nevertheless important to have methods that rely on clear principles and to have knowledge of their particular strengths, which can be exploited in the design of hybrid methods.

The research in heuristic search methodologies and metaheuristics, in particular, has now reached a mature state, and fundamentally new ideas appear rarely. Within this context, ILS has a clear role as one of the main methods that offers an excellent trade-off between simplicity and flexibility. As future trends we expect on one side that the number of applications of ILS increases further, and that more complex problems and also non-combinatorial problems will be tackled by it. On the other side, as the development of effective heuristic algorithms is becoming increasingly more streamlined, we expect that the clear principles underlying ILS support well a sound algorithm engineering effort. While traditionally such an algorithm engineering effort relied in part on the manual configuration and tuning of the algorithms, over the recent years, the advent of automatic algorithm configuration

tools [59, 112, 120, 121] such as ParamILS [63], SMAC [64], or irace [78, 79] has helped to alleviate the algorithm designer from this task. In fact, we foresee that in the future, the development of effective ILS algorithms and, more in general, of any other metaheuristic algorithms will be strongly based on the exploitation of automatic algorithm configuration and design techniques.

Cross-References

- GRASP
- Iterated Greedy
- Memetic Algorithms
- Tabu Search
- Variable Neighborhood Search

Acknowledgments Thomas Stützle acknowledges support from the Belgian F.R.S.-FNRS, of which he is a research director. Rubén Ruiz is partially supported by the Spanish Ministry of Economy and Competitiveness, under the project "SCHEYARD—Optimization of Scheduling Problems in Container Yards" (No. DPI2015-65895-R) financed by FEDER funds.

Competing Interest Declaration The author(s) has no competing interests to declare that are relevant to the content of this manuscript.

References

1. Aarts EHL, Lenstra JK (eds) (1997) Local search in combinatorial optimization. Wiley, Chichester
2. Aguiar Santos VL, Mota Carvalho TF, Pereira de Assis L, Weiss-Cohen M, Gadelha Guimarães F (2022) Multi-objective iterated local search based on decomposition for job scheduling problems with machine deterioration effect. Eng Appl Artif Intell 112:104826
3. Ahuja RK, Ergun O, Orlin JB, Punnen AP (2002) A survey of very large-scale neighborhood search techniques. Discret Appl Math 123(1–3):75–102
4. Applegate D, Bixby RE, Chvátal V, Cook WJ (1999) Finding tours in the TSP. Technical report 99885, Forschungsinstitut für Diskrete Mathematik, University of Bonn
5. Applegate D, Cook WJ, Rohe A (2003) Chained Lin-Kernighan for large traveling salesman problems. INFORMS J Comput 15(1):82–92
6. Applegate D, Bixby RE, Chvátal V, Cook WJ, Espinoza D, Goycoolea M, Helsgaun K (2009) Certification of an optimal TSP tour through 85,900 cities. Oper Res Lett 37(1):11–15
7. Auger A, Hansen N (2005) A restart CMA evolution strategy with increasing population size. In: Proceedings of CEC 2005. IEEE Press, Piscataway, pp 1769–1776
8. Bäck T, Fogel DB, Michalewicz Z (1997) Handbook of evolutionary computation. IOP Publishing Ltd., Bristol
9. Battiti R, Tecchiolli G (1994) The reactive tabu search. ORSA J Comput 6(2):126–140
10. Battiti R, Brunato M, Mascia F (2008) Reactive search and intelligent optimization. Operations research/computer science interfaces, vol 45. Springer, New York
11. Baum EB (1986) Iterated descent: a better algorithm for local search in combinatorial optimization problems, manuscript

12. Baum EB (1986) Towards practical "neural" computation for combinatorial optimization problems. In: Neural networks for computing. AIP conference proceedings, pp 53–64
13. Baxter J (1981) Local optima avoidance in depot location. J Oper Res Soc 32(9):815–819
14. Benlic U, Hao JK (2013) Breakout local search for the quadratic assignment problem. Appl Math Comput 219(9):4800–4815
15. Bentley JL (1992) Fast algorithms for geometric traveling salesman problems. ORSA J Comput 4(4):387–411
16. Blum C, Puchinger J, Raidl GR, Roli A (2011) Hybrid metaheuristics in combinatorial optimization: a survey. Appl Soft Comput 11(6):4135–4151
17. Brandão J (2018) Iterated local search algorithm with ejection chains for the open vehicle routing problem with time windows. Comput Ind Eng 120:146–159
18. Brandão J (2020) A memory-based iterated local search algorithm for the multi-depot open vehicle routing problem. Eur J Oper Res 284:559–571
19. Brucker P, Hurink J, Werner F (1996) Improving local search heuristics for some scheduling problems—part I. Discret Appl Math 65(1–3):97–122
20. Brucker P, Hurink J, Werner F (1997) Improving local search heuristics for some scheduling problems—part II. Discret Appl Math 72(1–2):47–69
21. Buson E, Roberti R, Toth P (2014) A reduced-cost iterated local search heuristic for the fixed-charge transportation problem. Oper Res 62(5):1095–1106
22. Calmels D (2022) An iterated local search procedure for the job sequencing and tool switching problem with non-identical parallel machines. Eur J Oper Res 297:66–85
23. Cattaruzza D, Absi N, Feillet D, Vigo D (2014) An iterated local search for the multi-commodity multi-trip vehicle routing problem with time windows. Comput Oper Res 51:257–267
24. Cerný V (1985) A thermodynamical approach to the traveling salesman problem. J Optim Theory Appl 45(1):41–51
25. Codenotti B, Manzini G, Margara L, Resta G (1996) Perturbation: an efficient technique for the solution of very large instances of the Euclidean TSP. INFORMS J Comput 8(2):125–133
26. Congram RK, Potts CN, van de Velde S (2002) An iterated dynasearch algorithm for the single-machine total weighted tardiness scheduling problem. INFORMS J Comput 14(1):52–67
27. Contreras-Bolton C, Parada V (2021) An effective two-level solution approach for the prize-collecting generalized minimum spanning tree problem by iterated local search. Int Trans Oper Res 28:1190–1212
28. Cordón O, Damas S (2006) Image registration with iterated local search. J Heuristics 12(1–2):73–94
29. Corte AD, Sörensen K (2016) An iterated local search algorithm for water distribution network design optimization. Networks 67(3):187–198
30. Della Croce F, Garaix T, Grosso A (2012) Iterated local search and very large neighborhoods for the parallel-machines total tardiness problem. Comput Oper Res 39(6):1213–1217
31. den Besten ML, Stützle T, Dorigo M (2001) Design of iterated local search algorithms: an example application to the single machine total weighted tardiness problem. In: Boers EJW et al (eds) Applications of evolutionary computing, proceedings of EvoWorkshops 2001. LNCS, vol 2037. Springer, Heidelberg, pp 441–452
32. Dong X, Huang H, Chen P (2009) An iterated local search algorithm for the permutation flowshop problem with total flowtime criterion. Comput Oper Res 36(5):1664–1669
33. Dong X, Ping, Huang H, Nowak M (2013) A multi-restart iterated local search algorithm for the permutation flow shop problem minimizing total flow time. Comput Oper Res 40(2):627–632
34. Dorigo M, Stützle T (2004) Ant colony optimization. MIT Press, Cambridge, MA
35. Essafi I, Mati Y, Dauzère-Pèrés S (2008) A genetic local search algorithm for minimizing total weighted tardiness in the job-shop scheduling problem. Comput Oper Res 35(8):2599–2616
36. Feo TA, Resende MGC (1995) Greedy randomized adaptive search procedures. J Glob Optim 6:109–113

37. Fernandes Muritiba AE, Bonates T et al (2021) Branch-and-cut and iterated local search for the weighted k-traveling repairman problem: an application to the maintenance of speed cameras. Trans Sci 55(1):139–159
38. Fernandez-Viagas V, Framinan JM (2014) On insertion tie-breaking rules in heuristics for the permutation flowshop scheduling problem. Comput Oper Res 45:60–67
39. Ferone D, Hatami S, González-Neira EM, Juan AA, Festa P (2020) A biased-randomized iterated local search for the distributed assembly permutation flow-shop problem. Int Trans Oper Res 27:1368–1391
40. Fernandez-Viagas V, Ruiz R, Framiñan JM (2017) A new vision of approximate methods for the permutation flowshop to minimise makespan: state-of-the-art and computational evaluation. Eur J Oper Res 257(3):707–721. https://doi.org/10.1016/j.ejor.2016.09.055
41. Fischetti M, Monaci M, Salvagnin D (2012) Three ideas for the quadratic assignment problem. Oper Res 60(4):954–964
42. Framinan JM, Gupta JN, Leisten R (2004) A review and classification of heuristics for permutation flow-shop scheduling with makespan objective. J Oper Res Soc 55(12):1243–1255
43. Geiger MJ (2011) Decision support for multi-objective flow shop scheduling by the Pareto iterated local search methodology. Comput Ind Eng 61(3):805–812
44. Gendreau M, Potvin JY (eds) (2010) Handbook of metaheuristics. International series in operations research & management science, vol 146, 2nd edn. Springer, New York
45. Glover F (1986) Future paths for integer programming and links to artificial intelligence. Comput Oper Res 13(5):533–549
46. Glover F, Kochenberger G (eds) (2002) Handbook of metaheuristics. Kluwer Academic Publishers, Norwell
47. Glover F, Laguna M (1997) Tabu search. Kluwer Academic Publishers, Boston
48. Glover F, Laguna M, Martí R (2002) Scatter search and path relinking: advances and applications. In: [46], pp 1–35
49. Grasas A, Juan AA, Lourenço HR (2016) SimILS: a simulation-based extension of the iterated local search metaheuristic for stochastic combinatorial optimization. J Simul 10(1):69–77
50. Grosso A, Della Croce F, Tadei R (2004) An enhanced dynasearch neighborhood for the single-machine total weighted tardiness scheduling problem. Oper Res Lett 32(1):68–72
51. Grosso A, Jamali ARMJU, Locatelli M (2009) Finding maximin Latin hypercube designs by iterated local search heuristics. Eur J Oper Res 197(2):541–547
52. Hansen P, Mladenović N (2001) Variable neighborhood search: principles and applications. Eur J Oper Res 130(3):449–467
53. Hansen P, Mladenović N, Brimberg J, Pérez JAM (2010) Variable neighborhood search. In: [44], pp 61–86
54. Hashimoto H, Yagiura M, Ibaraki T (2008) An iterated local search algorithm for the time-dependent vehicle routing problem with time windows. Discret Optim 5(2):434–456
55. Hejazi SR, Saghafian S (2005) Flowshop-scheduling problems with makespan criterion: a review. Int J Prod Res 43(14):2895–2929
56. Helsgaun K (2000) An effective implementation of the Lin-Kernighan traveling salesman heuristic. Eur J Oper Res 126:106–130
57. Helsgaun K (2009) General *k*-opt submoves for the Lin-Kernighan TSP heuristic. Math Program Comput 1(2–3):119–163
58. Hong I, Kahng AB, Moon BR (1997) Improved large-step Markov chain variants for the symmetric TSP. J Heuristics 3(1):63–81
59. Hoos HH (2012) Automated algorithm configuration and parameter tuning. In: Hamadi Y, Monfroy E, Saubion F (eds) Autonomous search. Springer, Berlin, pp 37–71
60. Hoos HH, Stützle T (2005) Stochastic local search—foundations and applications. Morgan Kaufmann Publishers, San Francisco
61. Hurtgen M, Maun JC (2010) Optimal PMU placement using iterated local search. Int J Electr Power Energy Syst 32(8):857–860

62. Hussin MS, Stützle T (2009) Hierarchical iterated local search for the quadratic assignment problem. In: Blesa MJ et al (eds) Hybrid metaheuristics. LNCS, vol 5818. Springer, Heidelberg, pp 115–129
63. Hutter F, Hoos HH, Leyton-Brown K, Stützle T (2009) ParamILS: an automatic algorithm configuration framework. J Artif Intell Res 36:267–306
64. Hutter F, Hoos HH, Leyton-Brown K (2011) Sequential model-based optimization for general algorithm configuration. In: Coello Coello CA (ed) LION 5. LNCS, vol 6683. Springer, Heidelberg, pp 507–523
65. Ibaraki T, Imahori S, Nonobe K, Sobue K, Uno T, Yagiura M (2008) An iterated local search algorithm for the vehicle routing problem with convex time penalty functions. Discret Appl Math 156(11):2050–2069
66. Imamichi T, Yagiura M, Nagamochi H (2009) An iterated local search algorithm based on nonlinear programming for the irregular strip packing problem. Discret Optim 6(4):345–361
67. Johnson DS (1990) Local optimization and the traveling salesman problem. In: Paterson M (ed) 17th international colloquium on automata, languages and programming. LNCS, vol 443. Springer, Heidelberg, pp 446–461
68. Johnson DS, McGeoch LA (1997) The traveling salesman problem: a case study in local optimization. In: Aarts EHL, Lenstra JK (eds) Local search in combinatorial optimization. Wiley, Chichester, pp 215–310
69. Juan AA, Lourenço HR, Mateo M, Luo R, Castellà Q (2014) Using iterated local search for solving the flow-shop problem: parallelization, parametrization, and randomization issues. Int Trans Oper Res 21(1):103–126
70. Katayama K, Narihisa H (1999) Iterated local search approach using genetic transformation to the traveling salesman problem. In: Banzhaf W et al (eds) Proceedings of GECCO 1999, vol 1. Morgan Kaufmann Publishers, San Francisco, pp 321–328
71. Kirkpatrick S, Gelatt CD, Vecchi MP (1983) Optimization by simulated annealing. Science 220:671–680
72. Kramer O (2010) Iterated local search with Powell's method: a memetic algorithm for continuous global optimization. Mem Comput 2(1):69–83
73. Kreipl S (2000) A large step random walk for minimizing total weighted tardiness in a job shop. J Sched 3(3):125–138
74. Lai X, Hao JK (2016) Iterated maxima search for the maximally diverse grouping problem. Eur J Oper Res 254(3):780–800
75. Laurent B, Hao JK (2009) Iterated local search for the multiple depot vehicle scheduling problem. Comput Ind Eng 57(1):277–286
76. Liao T, Stützle T (2013) Benchmark results for a simple hybrid algorithm on the CEC 2013 benchmark set for real-parameter optimization. In: Proceedings of CEC 2013. IEEE Press, Piscataway, pp 1938–1944
77. Lin S, Kernighan B (1973) An effective heuristic algorithm for the traveling salesman problem. Oper Res 21(2):498–516
78. López-Ibáñez M, Dubois-Lacoste J, Stützle T, Birattari M (2011) The irace package, iterated race for automatic algorithm configuration. Technical report, TR/IRIDIA/2011-004, IRIDIA, Université Libre de Bruxelles. http://iridia.ulb.ac.be/IridiaTrSeries/IridiaTr2011-004.pdf
79. López-Ibáñez M, Dubois-Lacoste J, Pérez Cáceres L, Stützle T, Birattari M (2016) The irace package: iterated racing for automatic algorithm configuration. Oper Res Perspect 3:43–58
80. Lourenço HR (1995) Job-shop scheduling: computational study of local search and large-step optimization methods. Eur J Oper Res 83(2):347–364
81. Lourenço HR, Martin O, Stützle T (2002) Iterated local search. In: [46], pp 321–353
82. Lourenço HR, Martin O, Stützle T (2010) Iterated local search: framework and applications. In: [44], chap 9, pp 363–397
83. Maniezzo V, Stützle T, Voß S (eds) (2009) Matheuristics—hybridizing metaheuristics and mathematical programming. Annals of information systems, vol 10. Springer, New York
84. Martin O, Otto SW (1995) Partitioning of unstructured meshes for load balancing. Concurr Pract Exp 7(4):303–314

85. Martin O, Otto SW (1996) Combining simulated annealing with local search heuristics. Ann Oper Res 63:57–75
86. Martin O, Otto SW, Felten EW (1991) Large-step Markov chains for the traveling salesman problem. Complex Syst 5(3):299–326
87. Martin O, Otto SW, Felten EW (1992) Large-step Markov chains for the TSP incorporating local search heuristics. Oper Res Lett 11(4):219–224
88. Mati Y, Dauzère-Pérés S, Lahlou C (2011) A general approach for optimizing regular criteria in the job-shop scheduling problem. Eur J Oper Res 212(1):33–42
89. Martinho WC, Melo RA, Sörensen K (2021) An enhanced simulation-based iterated local search metaheuristic for gravity fed water distribution network design optimization. Comput Oper Res 135:105429
90. Máximo VR, Nascimento MCV (2021) A hybrid adaptive iterated local search with diversification control to the capacitated vehicle routing problem. Eur J Oper Res 294:1108–1119
91. Máximo VR, Cordeau JF, Nascimento MCV (2022) An adaptive iterated local search heuristic for the heterogeneous fleet vehicle routing problem. Comput Oper Res 148:105954
92. Melo Silva M, Subramanian A, Ochi LS (2015) An iterated local search heuristic for the split delivery vehicle routing problem. Comput Oper Res 53:234–249
93. Merz P, Freisleben B (2000) Fitness landscape analysis and memetic algorithms for the quadratic assignment problem. IEEE Trans Evol Comput 4(4):337–352
94. Merz P, Huhse J (2008) An iterated local search approach for finding provably good solutions for very large TSP instances. In: Rudolph G et al (eds) PPSN X. LNCS, vol 5199. Springer, Heidelberg, pp 929–939
95. Metropolis N, Rosenbluth AW, Rosenbluth MN, Teller A, Teller E (1953) Equation of state calculations by fast computing machines. J Chem Phys 21:1087–1092
96. Michallet J, Prins C, Yalaoui F, Vitry G (2014) Multi-start iterated local search for the periodic vehicle routing problem with time windows and time spread constraints on services. Comput Oper Res 41:196–207
97. Mladenović N, Hansen P (1997) Variable neighborhood search. Comput Oper Res 24(11):1097–1100
98. Morris P (1993) The breakout method for escaping from local minima. In: Proceedings of the 11th national conference on artificial intelligence. AAAI Press/MIT Press, Menlo Park, pp 40–45
99. Moscato P (1999) Memetic algorithms: a short introduction. In: Corne D, Dorigo M, Glover F (eds) New ideas in optimization. McGraw Hill, London, pp 219–234
100. Mühlenbein H (1991) Evolution in time and space—the parallel genetic algorithm. In: Rawlins G (ed) Foundations of genetic algorithms (FOGA). Morgan Kaufmann Publishers, San Mateo, pp 316–337
101. M'Hallah R (2014) An iterated local search variable neighborhood descent hybrid heuristic for the total earliness tardiness permutation flow shop. Int J Prod Res 52(13):3802–3819
102. Nawaz M, Enscore E Jr, Ham I (1983) A heuristic algorithm for the m-machine, n-job flow-shop sequencing problem. Omega 11(1):91–95
103. Nguyen VP, Prins C, Prodhon C (2012) A multi-start iterated local search with tabu list and path relinking for the two-echelon location-routing problem. Eng Appl Artif Intell 25(1):56–71
104. Osorio-Mora A, Willmer Escobar J, Toth P (2023) An iterated local search algorithm for latency vehicle routing problems with multiple depots. Comput Oper Res 158:106293
105. Palhazi Cuervo D, Goos P, Sörensen K, Arráiz E (2014) An iterated local search algorithm for the vehicle routing problem with Backhauls. Eur J Oper Res 237(2):454–464
106. Pan QK, Ruiz R (2012) Local search methods for the flowshop scheduling problem with flowtime minimization. Eur J Oper Res 222(1):31–43
107. Queiroga E, Pinheiro RGS, Christ Q, Subramanian A, Pessoa AA (2021) Iterated local search for single machine total weighted tardiness batch scheduling. J Heuristics 27:353–438
108. Resende MGC, Ribeiro CC (2010) Greedy randomized adaptive search procedures: advances, hybridizations, and applications. In: [44], pp 283–319

109. Ribas I, Companys R, Tort-Martorell X (2013) An efficient iterated local search algorithm for the total tardiness blocking flow shop problem. Int J Prod Res 51(17):5238–5252
110. Ruiz R, Maroto C (2005) A comprehensive review and evaluation of permutation flowshop heuristics. Eur J Oper Res 165(2):479–494
111. Ruiz R, Stützle T (2007) A simple and effective iterated greedy algorithm for the permutation flowshop scheduling problem. Eur J Oper Res 177(3):2033–2049
112. Schede E, Brandt J, Tornede A, Wever M, Bengs V, Hüllermeier E, Tierney K (2022) A survey of methods for automated algorithm configuration. J Artif Intell Res 75:425–487
113. Schettini T, Gendreau M, Jabali O, Malucelli F (2023) An iterated local search metaheuristic for the capacitated demand-driven timetabling problem. Trans Sci 57(5):1379–1401
114. Schreiber GR, Martin O (1999) Cut size statistics of graph bisection heuristics. SIAM J Optim 10(1):231–251
115. Shaw P (1998) Using constraint programming and local search methods to solve vehicle routing problems. In: Maher M, Puget JF (eds) CP'98. LNCS, vol 1520. Springer, Heidelberg, pp 417–431
116. Stützle T (1998) Applying iterated local search to the permutation flow shop problem. Technical report AIDA–98–04, FG Intellektik, FB Informatik, TU Darmstadt
117. Stützle T (1998) Local search algorithms for combinatorial problems—analysis, improvements, and new applications. PhD thesis, FB Informatik, TU Darmstadt
118. Stützle T (2006) Iterated local search for the quadratic assignment problem. Eur J Oper Res 174(3):1519–1539
119. Stützle T, Hoos HH (2001) Analysing the run-time behaviour of iterated local search for the travelling salesman problem. In: Hansen P, Ribeiro C (eds) Essays and surveys on metaheuristics. Kluwer Academic Publishers, Boston, pp 589–611
120. Stützle T, López-Ibáñez M (2015) Automatic (offline) configuration of algorithms. In: Laredo JLJ, Silva S, Esparcia-Alcázar AI (eds) GECCO (companion). ACM, New York, pp 681–702
121. Stützle T, López-Ibáñez M (2019) Automated design of metaheuristics algorithms. In: Gendreau M, Potvin Y (eds) Handbook of metaheuristics. International series in operations research & management science, vol 272, 3nd edn. Springer, New York, pp 541–579
122. Stützle T, Birattari M, Hoos HH (eds) (2007) SLS 2007. LNCS, vol 4638. Springer, Heidelberg
123. Subramanian A, Battarra M, Potts CN (2014) An iterated local search heuristic for the single machine total weighted tardiness scheduling problem with sequence-dependent setup times. Int J Prod Res 52(9):2729–2742
124. Taillard ÉD (1990) Some efficient heuristic methods for the flow shop sequencing problem. Eur J Oper Res 47(1):65–74
125. Taillard ÉD (1995) Comparison of iterative searches for the quadratic assignment problem. Locat Sci 3(2):87–105
126. Thierens D (2004) Population-based iterated local search: restricting the neighborhood search by crossover. In: Deb K et al (eds) GECCO 2004, part II. LNCS, vol 3103. Springer, Heidelberg, pp 234–245
127. Ulder NLJ, Aarts EHL, Bandelt HJ, van Laarhoven PJM, Pesch E (1991) Genetic local search algorithms for the travelling salesman problem. In: Schwefel HP, Männer R (eds) Proceedings of PPSN-I. LNCS, vol 496. Springer, Heidelberg, pp 109–116
128. Urlings T, Ruiz R, Stützle T (2010) Shifting representation search for hybrid flexible flowline problems. Eur J Oper Res 207(2):1086–1095
129. Vaessens RJM, Aarts EHL, Lenstra JK (1998) A local search template. Comput Oper Res 25(11):969–979
130. Vansteenwegen P, Mateo M (2014) An iterated local search algorithm for the single-vehicle cyclic inventory routing problem. Eur J Oper Res 237(3):802–813
131. Vaz Penna PH, Subramanian A, Ochi LS (2013) An iterated local search heuristic for the heterogeneous fleet vehicle routing problem. J Heuristics 19(2):201–232

132. Voudouris C, Tsang EPK (2002) Guided local search. In: [46], pp 185–218
133. Wolf S, Merz P (2009) Iterated local search for minimum power symmetric connectivity in wireless networks. In: Cotta C, Cowling P (eds) Proceedings of EvoCOP 2009. LNCS, vol 5482. Springer, Heidelberg, pp 192–203
134. Xu H, Lü Z, Cheng TCE (2014) Iterated local search for single-machine scheduling with sequence-dependent setup times to minimize total weighted tardiness. J Sched 17(3):271–287
135. Yang Y, Kreipl S, Pinedo M (2000) Heuristics for minimizing total weighted tardiness in flexible flow shops. J Sched 3(2):89–108

Memetic Algorithms

27

Carlos Cotta, Luke Mathieson, and Pablo Moscato

Contents

Introduction . . . 810
Structure of a Memetic Algorithm . . . 812
Skeleton of a Classical Memetic Algorithm . . . 813
Population-Based Components . . . 816
The Role of Local Search . . . 818
A Note on More Advanced Memetic Algorithms . . . 819
Memetic Algorithms in Practice . . . 820
An Example Memetic Algorithm for *Network Alignment* . . . 820
An Example Memetic Algorithm for *Weighted Constraint Satisfaction Problems* . . . 822
A Brief Survey of Recent Memetic Algorithm Applications . . . 824
Future-Generation Memetic Algorithms . . . 832
Conclusion . . . 834
Cross-References . . . 834
References . . . 835

C. Cotta (✉)
Instituto de Ingeniería y Tecnología del Software (ITIS), Universidad de Málaga, Málaga, Spain
e-mail: ccottap@lcc.uma.es

L. Mathieson
School of Computer Science, University of Technology Sydney, Sydney, NSW, Australia
e-mail: luke.mathieson@uts.edu.au

P. Moscato
School of Information and Physical Sciences, University of Newcastle, Newcastle, NSW, Australia
e-mail: pablo.moscato@newcastle.edu.au

© Springer Nature Switzerland AG 2025

R. Martí et al. (eds.), *Handbook of Heuristics*,
https://doi.org/10.1007/978-3-032-00385-0_29

Abstract

Memetic algorithms (MAs) provide a very effective and flexible metaheuristic approach for tackling hard optimization problems. MAs address the difficulty of developing high-performance universal heuristics by encouraging the exploitation of multiple heuristics acting in concert, making use of all available sources of information for a problem. This approach has resulted in a rich arsenal of heuristic algorithms and metaheuristic frameworks for many problems. In this chapter, we discuss the philosophy of the memetic paradigm, lay out the structure of an MA, develop several example algorithms, survey recent work in the field, and discuss the possible future directions of MAs.

Keywords

Evolutionary algorithms · Hybridization · Local search · Memetic computing · Metaheuristics

Introduction

The effectiveness and efficiency of (meta)heuristics—and we may view memetic algorithms as particularly good heuristics in this sense—rests upon their ability to explore the solution space thoroughly while avoiding exhaustive or near-exhaustive searching. If we take polynomial-time computability as an approximation of tractability, then we can view a polynomial-time algorithm as a very clever search procedure; in these cases we have a small search space or can reduce the search space drastically. However, in dealing with intractable problems, reducing the search space to a reasonable size is a much more difficult task. Most centrally, we face the problem of local versus global improvement; an improvement to a solution does not give us any guarantee that we are in fact moving toward the optimum. Actually, depending upon the problem under consideration, we may be moving away from it (hence the notion of deception in search algorithms [253]) or at the very least we may be getting closer to being trapped in a nonoptimal solution for which no simple modification can lead to an improvement (i.e., a local optimum).

Thus many (meta)heuristic methods include techniques for allowing non-improving alterations to a solution or for nonlocal moves across the search space (Of course, there is some notion of locality that would put any two given solutions to a problem nearby, but we shall slip quietly past this semantic wrangling.) in order to be able to escape from local optima [27]. Perhaps the archetypic example of such a metaheuristic is the genetic algorithm (GA) [99, 113]: Inspired by the principles of natural evolution, GAs maintain a population (i.e., a multiset) of solutions that are subject to successive phases of selection, reproduction (via recombination and mutation), and replacement. The use of a population of solutions provides a better chance of avoiding local optima than maintaining a single solution. On the one hand, the search is driven by operators that (1) allow the search to take non-

improving steps, most notably in the case of mutations, and (2) allow the search to move to significantly different portions of the search space, particularly by virtue of recombination. On the other hand, selection and replacement typically work on a global (population-wise) scale, meaning that non-improving solutions have a chance of persisting for a nontrivial amount of time, hence allowing escape from local optima. However, although this heuristic structure has proven quite effective, it relies almost entirely on recombination mechanisms to improve solution quality and evolutionary processes are slow. In particular, they are also less capable of fine-tuning solutions, that is, the progress toward a fully optimized solution once the algorithm has located its basin of attraction (i.e., the region of the search space from which a series of small—local—improvements can lead to a certain local optimum—see [131, 210])—is often sluggish. This is precisely in contrast to local search (single-solution or trajectory-based) techniques which can readily locate local optima (and hence are more sensitive to them).

To address this weakness, researchers began developing *hybridized* metaheuristics [28, 30], that is, metaheuristics that combine ideas from different search paradigms and/or different algorithms altogether. The underlying idea in such approaches is obviously trying to achieve some synergetic behavior whereby the deficiencies of a certain search technique are compensated by the combination with other techniques and their advantages are boosted due to this very same combination. This strict interpretation of the term *hybrid* has been broadened with time to encompass all forms of nonblind metaheuristics (i.e., not domain independent). Under this broad interpretation, hybridization is the process of augmenting a generic (problem-independent) metaheuristic with problem- (or problem-class, i.e., domain-) knowledge. Since this augmentation is often achieved via the blend of different metaheuristic components, both interpretations are equivalent in most situations. The broad interpretation has, in any case, the advantage of fitting better into theoretical results such as those of Hart and Belew [110] and—most conspicuously—those of Wolpert and Macready [254] in the so-called *No-Free-Lunch Theorem*, which state that search algorithms perform strictly in accordance with the amount and quality of the problem knowledge they incorporate. Although these results have provoked controversy in their time and have been refined [79, 80], the bottom line remains.

Memetic algorithms (MAs) championed this philosophy. The notion of *memetic algorithm* (The term "memetic" was developed from Dawkin's [71] notion of a "meme" (from the Ancient Greek μίμημα, meaning "imitated thing") as a unit of cultural inheritance (and hence cultural evolution)—the cultural analog of a gene.) was coined in [177] to characterize and codify these hybrid approaches. The use of the term meme was intended to capture the fact that information traits in human culture are subject to periods of lifetime learning, and therefore they are different when transmitted to what they were when first acquired. This bears a strong resemblance with the Larmarckian model of evolution, whereby traits acquired during the lifetime of an individual are transmitted to its offspring. Therefore, it is not surprising that MAs are sometimes disguised under other denominations featuring the terms "Lamarckian" or "hybrid."

Although the initial conception of memetic search did not include the idea of GAs or evolutionary algorithms (EAs) whatsoever [181], it turned out that these techniques were ideal recipients for exploiting the metaphor of MAs, namely having a collection of "agents" alternating periods of self-improvement with phases of cooperation and competition, cf. [180]. In fact, early MAs mixed GAs and EAs with simulated annealing and tabu search [178, 183], eventually developing the idea that MAs are EAs endowed with some kind of local search (LS) technique, leading to the restrictive definition MA = EA + LS [206]. However, note that the central concept of MAs is not to adhere to a particular heuristic approach or metaphor, but to provide a coherent structure to employ *several* heuristics (including exact methods [29]) that employ *complementary* heuristics that exploit *all available knowledge*. We can thus state that EA + LS $\subset$ MA as a consequence of this broader definition of MAs, cf. [58]. The next section will explore the structure of MAs in more detail, with particular emphasis on the classical characterization of this paradigm. Subsequently, we will describe two examples of application of MAs to distinct combinatorial optimization problems, detailing how to deploy MAs by leveraging problem-specific knowledge in each case. Following that, we will provide an overview of recent applications of memetic algorithms across various fields and discuss the most prevalent MA architectures. We conclude this work by providing information on current trends in the development of MAs.

Structure of a Memetic Algorithm

As mentioned above, the early definitions of MAs envisioned the paradigm as a pragmatic integration of ideas from different metaheuristics. These were orchestrated in terms of a collection of search agents performing individual explorations (i.e., lifetime learning) of the space of solutions and participating in periodic phases of cooperation and competition [193]. An abstract formulation of such an approach is provided in Fig. 1. This pseudocode matches the initial conception of MAs as an inherently parallel approach in which a collection of local searchers (simulated annealing in early developments [181]) run either concurrently or physically in parallel and establish synchronization points in which information was exchanged between them. However, this depiction of MAs is still generic enough to encompass most actual incarnations of the paradigm as shown later, as it captures the essential feature of MAs, namely the carefully crafted interplay between global (population-based) and local (individual-based) search (It must be noted that we are always using the terms *global* and *local* in connection to the mechanics of the search rather than to the ability of eventually (or asymptotically) finding the global optimum. It is certainly the case that many local search approaches (simulated annealing, tabu search, etc.) are capable of escaping from local optima and navigate the search space in order to find the global optimum. However, the distinctive feature of these techniques (as opposed to, e.g., genetic algorithms) is that they do this following a trajectory-based approach.).

Fig. 1 A generic memetic algorithm

```
begin GENERIC-MA(μ)
1   parfor j = 1,...,μ do
2       pop_i ← CREATE-SEARCH-AGENT();
3   end-parfor
4   repeat
5       parfor j = 1...μ do
6           pop_i.LEARN();
7       end-parfor
8       pop.COMPETE();
9       pop.COOPERATE();
10  until stopping criterion is satisfied;
end GENERIC-MA
```

Skeleton of a Classical Memetic Algorithm

Following early work in which the population-based aspect of MAs, namely the collection of agents and the synchronized stages of cooperation and competition, was captured by a genetic algorithm [183], the classical memetic model was cemented. The basic skeleton of such an MA is relatively straightforward, adding little additional complexity beyond that of a GA. Figure 2 provides a visual representation of the functioning of such a classical MA, and Fig. 3 gives a pseudocode sketch of the salient structure (The interested reader may find an actual implementation of this skeleton at https://github.com/Bio4Res/memetic-continuous.), using LOCAL-SEARCH as a placeholder for any particular individual improvement heuristic (see Table 1) including, for instance, a complete exact algorithm like Branch and Bound and others that guarantee the optimality of the final solution obtained when they stop. Although a small structural change to a typical GA, the inclusion of the individual improvement phase can dramatically alter its performance. This mix allows the metaheuristic to benefit from the diversity of solutions engendered by the evolutionary approach, but to avoid the lethargic pace of improvement via more directed optimization: Instead of relying on random processes subjected to fitness-based selection alone, each individual solution is optimized before the

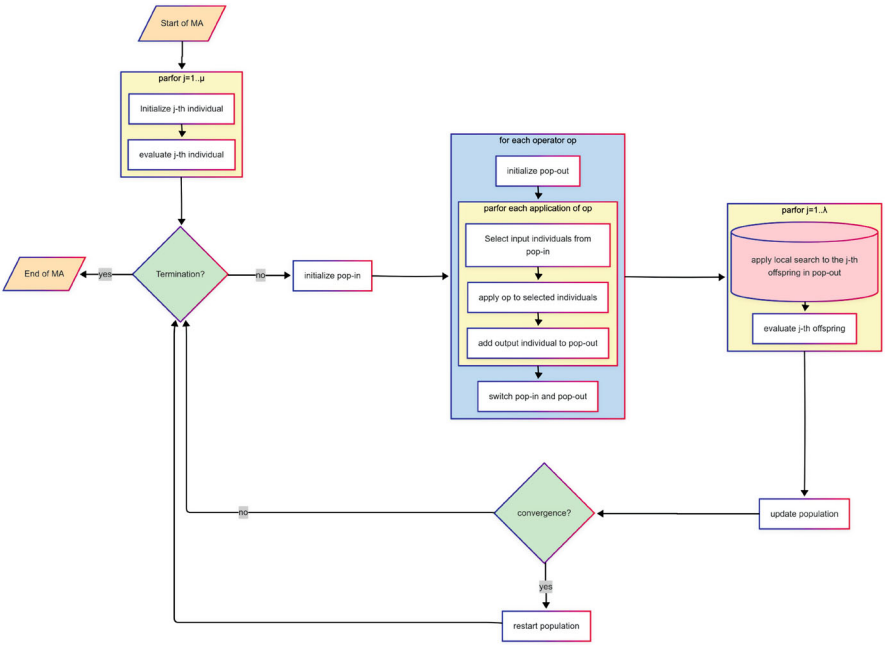


Fig. 2 A flowchart of the classical MA. Therein, μ is the population size and λ the number of offspring

evolutionary mechanism is applied (see Fig. 4b), significantly increasing the rate at which individual solutions converge to an optimum.

The structure of an MA is quite flexible, and the performance of the implementation, in terms of both solution quality and speed, can be affected by a number of factors. As an evolutionary population-based metaheuristic, the typical issues regarding choice and implementation of variation operators are inherited from the GA paradigm. Thus, it is very common to use recombination and mutation as the two primary elements to globally navigate the space of solutions (lines 8–14 in Fig. 3; see also Fig. 4). However, for those familiar with GAs, it should be readily apparent that the individual improvement phase is most likely to be the computational bottleneck—the improvement and subsequent evaluation of every individual is inherently expensive and, as shown in [170], it can easily take up to 95% of the computational cost of the algorithm. The trade-off is that, with a good choice of individual improvement heuristic, far fewer generations of mutation and recombination are required. The careful reader will also notice that the local search procedure (and, typically, any individual improvement heuristic) is highly amenable to parallelization. This helps to ameliorate the cost of individual improvement but more importantly lends the MA approach a high degree of scalability.

```
begin LOCAL-SEARCH-BASED-MA(parameters)
1  pop ← [];
2  parfor j = 1,...,parameters.μ do
3      pop_i ← INITIALIZE();
4  end-parfor
5  EVALUATE(pop);
6  repeat
7      pop-in ← pop;
8      for j = 1,...,parameters.#operators do
9          pop-out ← [];
10         parfor k = 1,...,parameters.op[j].times do
11             pop-out_k ← APPLY(parameters.op[j], SELECT(pop-in, parameters.op[j].arity));
12         end-parfor
13         pop-in ← pop-out;
14     end-for
15     parfor j = 1,...,parameters.λ do
16         pop-out_j ← LOCAL-SEARCH(pop-out_j, parameters);
17     end-parfor
18     EVALUATE(pop-out);
19     pop ← REPLACE(pop, pop-out);
20     if CONVERGED(pop) then
21         RESTART(pop, parameters);
22     end-if
23 until stopping criterion is satisfied;
end LOCAL-SEARCH-BASED-MA
```

Fig. 3 A Local Search-Based Memetic Algorithm. In addition to μ and λ (see Fig. 2), the algorithm parameters include the number of variation operators and for each of them the number of times they are applied and how many input solutions they require

Table 1 A list of some local search methods that can be used within an MA

Local search method	Description
Hill climbing	Simple neighborhood sampling (partial or complete) and selection of an improved neighbor (first one found or the best of the neighborhood explored)
Hooke-Jeeves	Continuous optimization. Derivative-free method. Pattern search
Local branching	Mixed-Integer Programming (MIP). Refinement via restricting the search space to a local neighborhood around the current solution
Lin-Kernighan Heuristic	TSP optimization. Tour reassembling
Nelder-Mead	Continuous optimization. Derivative-free method. Polytope search
Simulated annealing	Probabilistic acceptance of neighboring solutions using the Metropolis criterion, controlled by a temperature parameter. It can escape from local optima by allowing non-improving moves
Tabu search	Neighborhood exploration modulated by a tabu list of recent moves. It can escape from local optima by allowing non-improving moves
Variable neighborhood search	Systematic change of neighborhoods to conduct local search and use of perturbations to escape from local optima

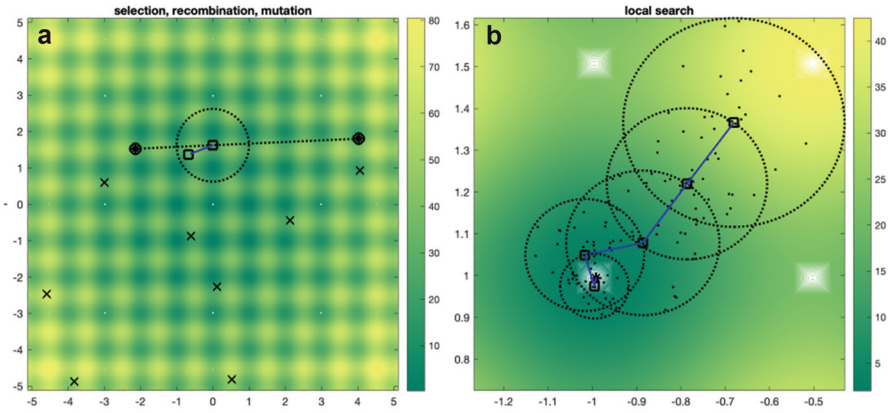


Fig. 4 Exploration done by the different components of the MA. (**a**) Population-based components. The crosses denote the current population, and the two small circles indicate selected individuals; along the line that connects them, a recombination is done (square mark), and the individual produced is mutated by selecting a random point within a certain neighborhood (represented as a dotted circle). (**b**) Local search. Focusing on the solution generated at the end of the previous subfigure, a sequence of steps is made: A number of neighbors of the current solution are selected (small dots), and the best of them is selected as the next current point (square marks). In both subfigures, the background is a contour plot of the 2D Rastrigin function. Note the different scale in either subplot

Population-Based Components

From the point of view of the different components into which a classical MA can be dissected, all of them are susceptible to encapsulating some portion of problem knowledge. Consider, for instance, recombination. This is the component that most appropriately captures the idea of agent cooperation. Such a cooperation is

typically established between a pair of agents but can in general involve an arbitrary number of *parents* [82] (notice nevertheless that in this case some forms of heuristic recombination can be very complex [60]). The generic idea of a knowledge-augmented recombination operator is to combine information pieces contained in the parents in an intelligent way. How these information pieces are defined is a problem-dependent issue that arises from the issue of representation in EAs. The underlying objective of an appropriate representation would be to have solutions described by some structured collection of objects whose values truly capture solution features of relevance (i.e., ultimately responsible for determining whether a solution is good or not). This is a substantial topic for which the interested reader is referred to, e.g., [214]. Focusing on the *smart* manipulation of these information units (however they are defined), that is, processing them in a problem-specific way instead of using domain-independent templates (such as those in [205]), the goal is picking the right combination of such units from either parent. Of course, this is easier said than done (and in fact, doing it is in general *provably* hard for arbitrary problems and/or definitions of *right* combination—see the discussion on the polynomial merger complexity class in [180, 182]), but there are numerous heuristics ideas in the literature that can be used to this end. In many cases—and following design advice already present in classical texts of hybridization pioneers, e.g., [70]—these ideas are based on the use of problem-specific heuristics such as greedy algorithms [132, 187], backtracking [55], dynamic programming [121], or branch and bound [62], just to mention a few.

Mutation is another classical operator, well known for its role of maintaining a continuous supply of genetic diversity that can be subsequently exploited by the remaining operators. Even more so, in certain EA models such as evolutionary programming [89], it actually bears sole responsibility for driving the search. Note that while this latter philosophy can also be used in an MA context—see Section "An Example Memetic Algorithm for *Weighted Constraint Satisfaction Problems*"—it is typically the case that MAs use sophisticated recombination operators such as those described before, and hence the criticism of recombination being just a disguised form of macromutation would not apply to them. Moreover, due to the presence of a local search stage in the main evolutionary cycle, one has to be careful to pick a mutation operator whose effects on solutions cannot be trivially undone by the local search, since that would defeat the very purpose of mutation. Following this line, in some cases there are MAs that even refrain from using mutation, e.g., [165, 246]. Although such a decision could be further supported by the fact that MAs usually feature a population restart procedure (see line 22 in Fig. 3), and therefore premature convergence is not so troublesome; this is not the most common course of action. An appropriate mutation operator (i.e., one that uses a neighborhood sufficiently different from that used by the local searcher) is often utilized. In fact, it is not unusual to have more than one such mutation operator, e.g., [147, 219], much like in metaheuristic approaches such as variable neighborhood search [108] (VNS). In some cases, these multiple mutation operators are used with the purpose of exerting different degrees of perturbation (i.e., *light* and *heavy* mutations) depending on the convergence of the population [90].

The restart procedure is another important element in an MA. The goal of this procedure is to perform a warm reinitialization of the population when the search is deemed stagnated (i.e., the population has converged to a suboptimal state). Of course, stagnation can be hindered by taking preventive measures such as the light/heavy mutation scheme mentioned before, the use of spatial structure in populations [238] (see also the next subsection), or some other diversity preservation policy [227]—see also [188]. However, a more drastic measure may eventually be required. For that purpose, a common approach is to keep a certain percentage of the current population and use the solution creation mechanism (the one used to create the initial population—line 3 in Fig. 3) to complete the new population. Regarding the former, they constitute a seed that allows one to keep a part of the search momentum without having to start from scratch. As to the latter, note that they need not be purely random solutions, but any available constructive heuristic can be used for this purpose, e.g., see [57, 76, 195].

The Role of Local Search

The use of local search components is one of the most distinctive elements of MAs. These components (one such component in most simple approaches, although some more advanced methods may feature multiple local search operators—see the next subsection) can take the form of any stand-alone method such as hill climbing, simulated annealing, tabu search, variable neighborhood search, etc. [194]. The choice of a particular technique must take into account two major issues, namely its parameterization and its interaction with the remaining components of the algorithm. Regarding the latter and in addition to the issues discussed above in connection with the mutation operator, one has to consider the interplay between the local searcher and the recombination operator. For example, a highly intensive local search procedure may be better suited to interact with a more diversification-oriented recombination operator; see, e.g., [91]. This heuristic recipe does not necessarily conflict with the use of a powerful recombination operator (see, e.g., [93]) but underlines that the knowledge embedded in either component, recombination operator, and local search heuristic must be complementary in terms of the effect they produce in the search dynamics (much as it was discussed for the mutation operator). An interesting analysis of these issues from the point of view of fitness landscapes is provided in [172]. Whatever the definition of the neighborhood (and note that it can be complex, even combining several simpler neighborhood schemes), it is often crucial for performance reasons to be able to evaluate solutions incrementally [109], that is, without having to resort to a full evaluation, but only by recomputing the fitness contribution of the solution components that were modified. This may require the use of appropriate data structures and is normally associated with discrete optimization (the high nonlinearity—and sometimes even the lack of a closed fitness function—often makes this complicated in continuous optimization).

The parameterization of the local search heuristic is another complex issue. We can include here both high-level algorithmic aspects such as when to apply

local search, to which solutions it should be applied, which local search operator to apply (if there are several of them), as well as low-level parameters such as the breadth (number of neighbors explored in each iteration of the local search heuristic) and depth (how many iterations of local search will be performed)—see [230]. Determining an adequate setting for these parameters is crucial for the performance of the algorithm, as it has been shown theoretically that small parameter changes can turn a problem from being polynomial time solvable with high probability to requiring super polynomial (even exponential) time [139, 229]. Unfortunately, a priori design guidelines to provably avoid this kind of behavior are ruled out by intractability results [230]. Thus, design by analogy and empirical testing appear to be the most helpful tools to approach this endeavor (although both self-parameterization and automatic tuning [164,228] are appealing alternatives that are increasingly gaining relevance). In this regard, it has been shown, for example, in several contexts that partial Lamarckism [115], that is, not applying a local search to every individual but just applying it with some probability p_{LS}, can produce notably better results than a fully Lamarckian approach [56, 124] although the best value of this parameter is problem dependent. In relation to the depth of the local search, it has also been proposed in the literature to save the state of the local search together with the solution to which it was applied, so as to resume the process from that point if necessary [175, 176].

A Note on More Advanced Memetic Algorithms

Although Figs. 1 and 3 set a basic MA framework, the structure can be made significantly more complex. As the central motivation of MAs is to exploit all available information, the restriction of any particular component would be antithetical. In addition to employing different heuristics, we can employ *multiple* heuristics in concert. We can easily combine different individual improvement heuristics, applying them to different individuals, different populations, in parallel, in sequence, or even in competition. Similarly, the population-based heuristic can employ multiple improvement techniques—such approaches are well known in the GA community.

Moscato and Tinetti [184] demonstrate a more advanced MA that uses several heuristics in concert and to achieve different goals within the algorithm. The algorithm employs *tree-structured population* where the population is divided into subpopulations of size 4, arranged in a ternary tree structure.

1. Each subpopulation divided into a *leader* node and three *supporters*. The supporters are stored one level below their leader.
2. The intermediate nodes in the tree hold an individual that is part of two populations; it is the leader of the three supporters lower in the tree and a support of its leader higher in the tree.
3. The number of subpopulation can be manipulated by adding levels to the tree.

Each individual can be optimized using a local search procedure that selects from a variety of local optimization moves; approximate 2-OPT, One-City Insertion, and Two-City Insertion [135, 150]. Genetic recombination occurs "normally" within each subpopulation with the leader individual representing the best tour in the subpopulation (note that overlap of subpopulations ensures that improvement propagates up the tree); however, the small size of the subpopulation can quickly lead to a lack of diversity, in which case the recombination mechanism switches to an external recombination procedure for the lowest level of the tree.

In this example, we can see a number of variations on the basic structure of an MA: multiple local search variants, a multipopulation variation of a GA which itself employs multiple recombination procedures. Another example of a more complex improvement strategy is given by Moscato [179], where a small population of individuals is maintained (only 16 individuals, each a binary vector), with a tabu search procedure for individual improvement. Again, in a small population, a loss of diversity is a potential drawback. To combat this, each individual notes the 16 best single-bit-flip moves available. When diversity falls below a given threshold, instead of following a simple tabu search approach, the individual i performs move i from its list of best moves. This deliberate (potentially) nonoptimal action has the effect of spreading the individuals further across the configuration space, increasing diversity. Once diversity is restored, normal tabu search optimization is restored.

The continuation of these ideas has led to the development of what are now called *self-adaptive* MAs, which allow the context-specific, dynamic application of different heuristics or tuning of search parameters by the algorithm itself. See Section "Future-Generation Memetic Algorithms".

Memetic Algorithms in Practice

In this section, we present two extended examples of MAs for specific problems—*Network Alignment* and *Weighted Constraint Satisfaction Problems*—and survey recent interesting applications of MAs in different fields.

An Example Memetic Algorithm for *Network Alignment*

The optimization version of the basic *Network Alignment* problem takes as input two networks G_1 and G_2 and asks for an injective partial mapping $f : V(G_1) \to V(G_2)$ between the vertices of the two networks that maximizes $\sum_{u,v \in V(G_1)} \tau_f(u, v)$ where

$$\tau_f = \begin{cases} 1 \text{ if } uv \in E(G_1) \text{ and } f(u)f(v) \in E(G_2) \\ 0 \text{ otherwise} \end{cases}$$

In fact, we may assume that the mapping is total and bijective by adding "dummy" vertices to the smaller network. Of course, τ_f is open to variation, as are the

precise details of f, leading to many variants of *Network Alignment*. The decision variant of *Network Alignment* is NP-complete [136] and W[1]-complete [169], suggesting, subject to standard complexity assumptions, that there is no suitably efficient exact algorithm for *Network Alignment*, making it a prime candidate for heuristic methods.

In developing an MA for this (and any) problem, we must (at a minimum) select an individual solution representation, mutation and recombination operators, and an individual improvement heuristic and its attending concerns.

For *Network Alignment*, the most direct individual representation is the mapping itself. Assuming that we have already added any necessary dummy vertices, we can represent the mapping by (for example) an array of size $|V(G_1)|$ storing a permutation of $V(G_2)$. For ease of representation, it is sufficient to assign each vertex a unique integer in the range $[0, |V(G_1)|]$. An alternative representation suitable for *Network Alignment* would be to store the alignment of the edges, making the computation of the basic fitness function simple, but the individual could be polynomially larger, affecting the efficiency of mutation, recombination, individual improvement, and even evaluation. In addition, care would need to be taken as to how to determine which vertices were aligned.

With this individual representation, a simple and reasonable mutation operator is that of a random shuffle, where each element of an individual is randomly swapped with another, randomly chosen element, with a given probability. A naïve recombination operator is, given two parents, to select a linear segment of the individual (i.e., a set of contiguous indices) and swap the mappings for those indices between the parents (with adjustment to handle the duplication of elements, etc.), producing two children. This recombination is commonly known as a Partially Matched Crossover (PMX) [100]. However, considering the problem we are trying to solve, it is easy to see that this choice may be somewhat inefficient. The *Network Alignment* problem, in essence, seeks to preserve as much topological structure (i.e., edge matchings) as possible—in this sense it is a relaxed *Graph Isomorphism* problem. Swapping a set of arbitrarily chosen indices is unlikely to preserve interesting structure, contrary to the goal of a recombination operator, which is to produce children of higher quality than their parents by mixing the better parts of the parents, aiming to place the child solution closer to the global optima. For *Network Alignment*, it is much more interesting to preserve vertices' neighborhoods in this regard. So, a better choice of recombination operator is to select a vertex and its 2-neighborhood (all vertices at distance at most 2) as the set of indices which will be swapped.

To complete the GA component, we employ a tournament selection process to choose the individuals included in the new generation and a restart mechanism, whereby the best solution is recorded and the population is restarted if no improvement has been observed after a given number of generations.

For individual improvement, we employ a local search heuristic where the neighborhood of each individual is the 2-swap neighborhood, the set of individuals obtained by swapping any two elements. The search is implemented by selecting an element in the individual and taking the optimal swap in the local neighborhood.

If this is not the identity mapping, we place the neighbors of the preimage of the swapped vertex into a list of vertices to swap. If no initial swap is found, we repeat the process with a new starting point until a swap is found or all vertices have been tested.

In combination with the skeletons given by Figs. 1 and 3, these assembled components constitute an MA for *Network Alignment*; however, the reader will notice that, even without considering more complicated approaches, there are several tunable parameters present, namely the probabilities and frequencies that control mutation and recombination (as in GAs), and more specifically for MAs, the frequency and application régime of the individual improvement step. We may choose to apply individual improvement, essentially at one extreme, regularly, to all individuals, or at the other extreme, only when evolutionary progress slows and to a select few individuals, or of course in some intermediate régime. As discussed in Section "A Note on More Advanced Memetic Algorithms", we could also take an adaptive approach, allowing the algorithm to adjust these parameters itself.

An Example Memetic Algorithm for *Weighted Constraint Satisfaction Problems*

Weighted Constraint Satisfaction Problems (WCSPs) are a general class of combinatorial problems in which (i) solutions are assignments of values to a collection of variables, each of them taken from a possibly different domain, (ii) there are hard constraints making some particular combinations of variable values infeasible, and (iii) there are some soft constraints that establish preferences between solutions. We can, for example, think of a school timetabling problem in which courses have to be fit into different time slots: No two courses can use the same time slot if they are taught by the same lecturer (a hard constraint) and lecturer preferences (e.g., teaching in the morning or in the afternoon) have to be respected if possible. In essence, both types of constraints can be represented by defining a collection of integer functions f_i, one for each constraint; these functions are used to weight the fulfillment/violation of the corresponding constraint, and therefore an objective function F (to be minimized, without loss of generality) can be built by summing them. Thus, it will typically be the case that hard constraints have a much larger weight (even infinite if violated) than soft constraints.

Formally, we can characterize a WCSP as a triplet $\langle \mathscr{X}, \mathscr{D}, \mathscr{F} \rangle$, where each $x_i \in \mathscr{X}$, $1 \leqslant i \leqslant n$, is a problem variable whose domain is $D_i \in \mathscr{D}$. Each function $f_j \in \mathscr{F}$, $1 \leqslant j \leqslant m$, has signature $f_j : V_j \to \mathbb{N}$, where $V_j \in 2^{\mathscr{X}}$ is the subset of variables involved in the jth constraint. With this formulation, a naïve evolutionary approach can be defined by using the Cartesian product $\mathscr{S} = D_1 \times \cdots \times D_n$ as search space, taking the fitness function to be $F(x) = \sum_j \hat{f}_j(x)$ (where $\hat{f}_i$ is a function that picks from its argument the variables in V_j and feeds them to f_j), and using standard operations for recombination and mutation. However, such an approach is going to perform poorly in general due to the lack of problem-specific

knowledge. A much more sensible approach can be built on the basis of (i) a smart recombination operator and (ii) a powerful local search technique.

Regarding recombination, it is very easy to define a greedy recombination mechanism for WCSPs: (1) start from a solution s with all variables unassigned, (2) sort constraints in some particular order (arbitrary or heuristically selected) $j_1, \cdots, j_m$, and (3) traverse this ordered list of constraints, checking for each j_k the variables in V_{j_k} that are still unassigned in s, constructing two (or as many as parents) candidate sets using the assigned values in s plus the values that the remaining variables in V_{j_k} take in either parent, and keeping the candidate set v minimizing $f_{j_k}(v)$ (which is subsequently used to expand the solution s). This procedure has been used, for example, in [64] for the construction of Golomb rulers and in [213] for the construction of balanced incomplete blocks, to cite only two examples.

It is possible to define a more intensive recombination approach by taking ideas from complete techniques [66]. More precisely, a complete technique can be used to explore the set of potential solutions that can be created using a given collection of parents, returning the best solution attainable. Different possibilities can be used for this purpose, such as branch and bound [62] or integer linear programming techniques [168]. A more WCSP-specific approach can be found in the use of bucket elimination (BE) [73]. BE can be regarded as a dynamic programming approach based on the successive elimination of variables and the use of an auxiliary table to store the best value of the fitness function for specific variable combinations. More precisely, BE considers some ordering of the variables (again, arbitrarily or heuristically selected—it must be noted that while the particular choice ordering is irrelevant for correction purposes, it can have a huge impact in the computational complexity of the algorithm though) $i_1, \cdots, i_n$. Then, it traverses this sequence, and for each variable x_{i_k} (1) determines the constraints $\mathscr{C} \subseteq \mathscr{F}$ in which x_{i_k} is involved, (2) computes the bucket

$$B_{i_k} = \left(\cup_{f_j \in \mathscr{C}} V_j\right) \setminus \{x_{i_k}\},$$

namely the collection of variables related to x_{i_k} in any constraint, (3) determines for each combination t of values for variables in B_{i_k} the value v_t^* for x_{i_k} such that $w = \sum_{f_j \in \mathscr{C}} \hat{f}_j(t \cdot (x_{i_k} = v))$ is minimal, and (4) removes $\mathscr{C}$ from $\mathscr{F}$ and adds a new constraint f' with domain $V' = B_{i_k}$ defined as $f'(t) = \sum_{f_j \in \mathscr{C}} \hat{f}_j(t \cdot (x_{i_k} = v_t^*))$. When all variables have been eliminated, the optimal cost w is found, and one only has to trace the process (using the auxiliary table) to determine the best assignment of variables [95]. This procedure has been used with great success in [93] to solve *Maximum Density Still Life Problem* in conduction with a local search procedure based on tabu search.

A potential drawback of recombination schemes such as those defined above is scalability: The use of an exact technique for recombination is less costly than using it to solve the problem completely from scratch, but its cost will nevertheless

grow with the problem size until becoming impractical at some point. To alleviate this problem, we can play with the granularity of the representation [61], that is, grouping variables into larger chunks which are subsequently used as basic units for the purposes of constructing solutions (hence reducing the number of potential solutions attainable and therefore the computational cost of the exact technique). In the context of the BE method described previously, this approach is termed mini-buckets [77] and can be readily applied to the recombination mechanism described above [95]. Another source of difficulties is the existence of symmetries or partial isomorphisms between solutions. This scenario is typical in many WCSPs in which variables or groups thereof can be relabeled without altering the solution. In such a situation, recombination can reduce to macromutation unless it is effectively capable of identifying correspondences between variables in different parents. This is, for example, done successfully in [173] in the context of clustering genomic data. Of course, it may be very complex, in general, to find a perfect match between variables in an arbitrary WCSP with symmetries. In problems for which this is deemed too complicated or time consuming, it must be noted that a recombination-less MA—essentially a population of local searchers subject to interleaved phases of self-improvement and competition via selection/replacement, much in the line of go-with-the-winner approaches [5]—can also provide acceptable results. This is, for example, the case of the *Social Golfer Problem*, a WCSP with a large degree of symmetry that was successfully attacked using a memetic evolutionary programming approach [63] (an MA propelled by selection, mutation, and local improvement).

A Brief Survey of Recent Memetic Algorithm Applications

In recent years, MAs have become a significant part of the optimization toolkit and have become particularly well used in recent years. As a rough gauge, the number of academic papers (Found via searching Scopus (https://www.scopus.com) and the Web of Science (https://www.webofscience.com) for relevant papers with the phrase "memetic algorithm" in their title, abstract, or keywords.) published is approximately 238 per year on average during the last decade, with thousands of academic publications in total since 1998 (between ∼3200 and ∼3900 depending on the source). These publications have attracted an interest that is well represented by the number of citations received (see Fig. 5). The total number of citations ranges between ∼77000 and ∼86000 depending on the source, and the field has a h-index between 113 (Scopus) and 118 (WoS), thus underpinning the traction of this area. Possibly the most interesting aspect of this growing interest in MAs is the diversity of techniques and application areas. This will be shown next.

Memetic Algorithms in the Wild

While many algorithms developed in the areas of Computer Science and Optimization are demonstrated via application to practical problems drawn from a

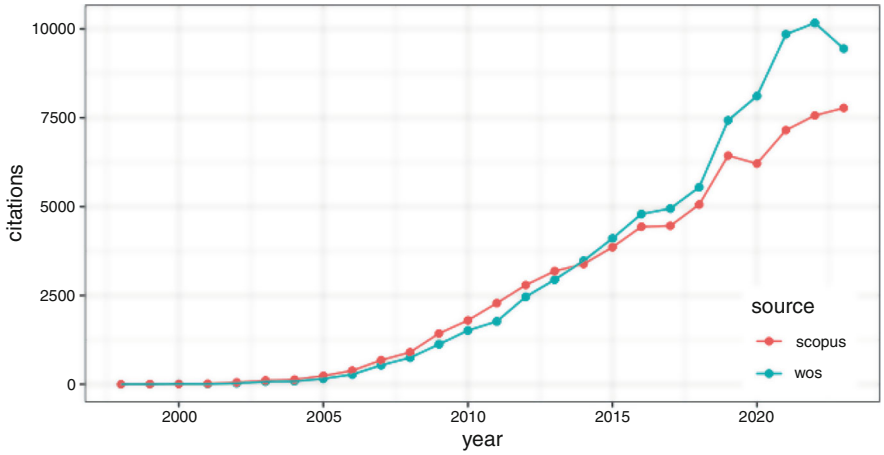


Fig. 5 Number of citations per year to academic papers dealing with MAs

variety of areas, a more reliable indicator of the effectiveness of a technique is the adoption of the technique as a tool within the communities from which the problems are drawn. Here, we briefly survey some of the areas in which MAs have been successfully applied. Table 2 gives an overview of the breadth of application areas for MAs, with recent references. Of course, this table is far from exhaustive, even within the application areas mentioned. In fact, in some areas the number of memetic applications has deserved individualized treatment in specialized surveys, e.g., scheduling and timetabling [58], engineering and design [41], bioinformatics [24], etc. The reader is also referred to application-oriented surveys in which MAs are also features, such as [225] (for feature selection), [127] (for flexible job shop), [49] (for IoT network topology), and [122] (for multiple sequence alignment), just to name a few. The breadth of the application areas suggests a significant generality and flexibility in the memetic paradigm.

Memetic Speciation

Along with a wide set of application areas, MAs have also embraced many forms, employing a wide variety of combinations of population-based heuristics and individual improvement heuristics. Table 3 lists some of the more prominent combinations, with examples of references for each (see also the annotated references in Table 2). Not only are different combinations of population-based heuristics and individual improvement heuristics extant, but also more exotic MAs that use heuristics of only one type or multiple heuristics of each type exist. The adaptability of MAs to parallel implementation also encourages the use of multiple different types of heuristics simultaneously—the exploitation of *all* available knowledge is, after all, the central idea of the memetic paradigm.

Table 2 Some recent publications reporting on MA applications by field of application and specific problem domain, annotated with a brief description of the algorithmic approach

Bioinformatics and biomedicine		
[87]	Differential evolution with problem-specific refinement method	Protein structure prediction
[134]	Niching MA with adaptive local search	Generative adversarial networks
[216]	MA with local search enhanced by domain-expert knowledge	Inference of gene regulatory networks
[8]	Multi-objective MA with simulated annealing	Vaccination allocation
Combinatorial optimization		
[241]	MA and iterated local search	Linear ordering problem
[102]	MA with multiple recombination and local search procedures	Graph coloring
[281]	MA with two complementary local searchers	Generalized minimum spanning tree problem
[54]	MA with heuristic chromosome enhancement and cluster-based local search	Generalized TSP
[143]	MA with k-means initialization, LKH, and ad hoc operators	Close-enough TSP
[126]	MA with guided initialization and adaptive parameter control	Euclidean TSP
[112]	MA with edge assembly crossover and multiple neighborhood local search	Hamiltonian p-median
[158]	MA with cluster-based recombination and two-phase local refinement	Set orienteering problem
Data analysis		
[245]	Binary-encoded MA with local likelihood score refinement	Time series analysis
[209]	MA with fuzzy fitness function and tabu search/hill climbing	Feature selection
[218]	MA with pruning refinement	Multilabel feature selection
[279]	MA with semantic crossover	Rank aggregation
[259]	MA with immune algorithm and evolutionary gradient search	Decision variable classification
[15]	Binary whale optimization algorithm with differential evolution-based local search	Feature selection
Engineering		
[43]	Mixed continuous-discrete MA	Reactive extrusion processes
[12]	Multi-objective MA with memeplexes	Composite structures
[201]	MA with Q-learning	Cogeneration systems
[271]	MA with hierarchical objectives and simulated annealing	Combined heat and power systems
[104]	MA based on 3D O-Tree representation	3D VLSI floor plan
[204]	MA with local greedy strategy	Topology reconfiguration in multicore processors
[69]	MA with multi-objective Jacobian-based optimization	Robot inverse kinematics

(continued)

Table 2 (continued)

Finances and investments		
[107]	Differential evolution MA with adaptive step-size local search	Portfolio optimization
Network design and optimization		
[125]	MA with LEACH protocol	Wireless sensor networks
[233]	MA with adaptive tabu search and greedy strategies	Influence maximization in complex networks
[36]	MA with guided initialization and ad hoc operators	"
[142]	MA with stochastic hill climbing	Community detection
[208]	MA with partial reinitialization of the population	Detection of dynamic communities in networks
[231]	MA with multilevel greedy local search	Structural balance in networks
[199]	MA with local search based on neighboring/global/random strategies	Influence in complex networks
Fleet management and vehicle routing problems (VRPs)		
[223]	Multi-population MA with VNS	Multicenter multi-periodic VRP
[280]	MA with heuristic initialization and repairing	Dynamic pickup and delivery problem
[203]	MA with an exact bi-objective decoder	Capacitated arc routing
[273]	Genetic algorithm with dynamic programming	Capacitated VRP
[98]	MA with heuristic recombination and greedy local search descent	VRP with time windows
[74]	Multi-objective MA with reinforcement learning and multiple local searchers	Multi-objective VRP with time windows
[277]	MA with guided initialization and self-adaptive operators	Two-echelon time-dependent VRP
[284]	MA with reinforcement learning guided variable neighborhood descent	Latency location routing
[274]	Self-adaptive MA with Q-learning	Automated guided vehicles
[276]	MA with reinforcement learning	Drone scheduling
[154]	MA with hill climbing and inbreeding prevention	"
[52]	MA with Q-learning	Unmanned aerial vehicles
[128]	MA based on PBIL and local search	"
[266]	MA with variable neighborhood search	Drone-truck delivery
[111]	Multilevel MA with tabu search	Clustered team orienteering problem
[256]	MA with ad hoc operators and tabu search with reinforcement learning	"
[20]	MA with variable-length encoding and local search based on Dijkstra's algorithm	Routing in multigraphs with time constraints
[19]	Multi-objective MA with multidirectional local search	Healthcare routing
[92]	Multi-objective artificial bee colony optimization with guided initialization and self-learning neighborhood	"

(continued)

Table 2 (continued)

[224]	MA with simulated annealing	"
[270]	MA with variable neighborhood search	Ship traffic in ports
[255]	Combination of discrete particle swarm optimization, adaptive differential evolution, and local search	Design of tourist routes
Manufacturing and supply chain optimization		
[258]	MA with heuristic initialization and knowledge-driven operators	Energy-efficient flowshop
[261, 267]	Multi-objective MA with deep Q-network	"
[160]	MA with guided initialization, multiple neighborhood structures, and Q-learning	Permutation flowshop
[265]	Q-learning guided metaheuristics	No-wait permutation flowshop
[248]	Differential evolution with three subpopulations and path-based local search	Distributed flowshop
[260]	Multi-objective MA with knowledge-based local search	Distributed heterogeneous flowshop
[235]	Coevolutionary memetic algorithm with specialized neighborhood structures	Hybrid flowshop
[264]	MA with heuristic initialization, ad hoc operators, and two neighborhoods for local search	"
[282]	MA with heuristic initialization and self-adaptive selection of ad hoc operators	"
[268]	MA with coevolving stratified subpopulations and variable neighborhood local search	"
[155]	MA with evolutionary transfer learning	Distributed hybrid flowshop
[275]	MA with dynamic rescheduling and ad hoc operators	"
[75]	MA with two-stage initialization and multiple collaborative operators	"
[263]	Multi-objective MA with decoding mechanism and guided initialization	"
[159]	MA with multiple heuristics for guided initialization	"
[161]	Multi-objective MA with adaptive neighborhood search	Job shop
[17]	MA with greedy decoding and diversity adaptive control	"
[243]	MA with various local search strategies	Distributed job shop
[249]	MA with tabu search and specialized neighborhoods	"
[101]	Multi-objective MA with ad hoc operators	Flexible job shop
[47]	MA with reinforcement learning	"
[34]	Multi-objective MA with greedy refinement strategy	"

(continued)

Table 2 (continued)

[118]	MA with hierarchical heuristic neighborhood search	"
[106]	Multi-objective MA with local search on aggregated fitness	"
[252]	Multi-objective MA with guided initialization and adaptive mutation	Flexible job shop inverse scheduling
[234]	MA with guided initialization and three local search methods	Distributed flexible job shop
[153]	MA with decoding mechanism and multiple neighborhoods	Parallel machine scheduling
[130]	Memetic genetic programming	Yard crane scheduling
[242]	Fast niching MA with linear programming	Double row layout problem
[1]	MA with mixed-integer linear programming	Maintenance scheduling
[200]	MA with insertion-based local search	Parallel row ordering problem
[145]	Simulation-based optimization with MAs	Supply chain disruptions
[269]	MA with Q-learning	Integrated production scheduling
[283]	MA with a decoder mechanism, guided initialization, and multiple ad hoc operators	"
[162]	MA with guided initialization and self-adaptive local search	Production scheduling
[14]	MA with decoding algorithm and steepest descent local search	Production and distribution scheduling
[262]	MA with Q-learning and variable neighborhood search	Human-robot collaboration in distributed welding shop
[157]	Multi-objective MA with variable neighborhood search	"
[166]	MA with multi-parent recombination	Product delivery
Real-world applications of optimization		
[185]	MA with Nelder-Mead local search	Nonlinear global optimization
[105]	Multi-population hierarchical memetic strategy	"
[217]	MA based on swarm intelligence methods for global and local search	"
[212]	Combination of genetic algorithms, differential evolution, and hill climbing	"
[163]	MA with continuation Newton method for population initialization	"
[38]	Multi-agent memetic scheme for multi-objective optimization	"
[152]	Many-objective surrogate-assisted memetic search	Many-objective optimization
[46]	Multi-objective MA with heuristic operators	Satellite observation scheduling
[226]	MA with heuristic initialization and reinforcement learning	Satellite range scheduling
[123]	MA with simulated annealing	Resource leveling

(continued)

Table 2 (continued)

[239]	MA with simulated annealing	Project scheduling
[51]	Multi-objective MA with variable insertion neighborhood search and ad hoc methods	"
[72]	Ant colony system with variable neighborhood descent	"
[22]	MA based on coral reef optimization and adaptive backpropagation	Neural network training and design
[37]	Multi-objective MA and federated learning	"
[114]	Multi-objective MA with portfolio-based selection and surrogate-assisted local search	Neural network pruning
[133]	MA with hill climbing and clustering algorithm	Software testing
[3]	MA with hill climbing and multi-agent system	Decision support in healthcare
[119]	Nondominated neighbor immune algorithm with two local searchers	Decision support for repair services
Telecommunications		
[97]	MA with domain-specific recombination and repairing and multi-objective archiving	Next-generation wireless communications
[39,40]	MA with iterative greedy search	Antenna design problems
[45]	MA with quasi-Newton method	Array antenna design
[236]	Parallelized MA with convolutional neural networks	IoT network management
[167]	Opposition-based MA	IoT applications in fog computing
[68]	Self-adaptive MA based on firefly algorithm and ensemble methods	IoT healthcare
[156]	Binary MA with elitism archive	Polar coding in 5G wireless systems
[11]	MA with constraint programming solver and path relinking	Physical cell identity assignment
Warehouse management		
[257]	MA with fuzzy-based population control	Joint order batching and picker routing problem
[211]	MA, differential evolution, and evolution strategy with reinforcement learning	Multi-echelon inventory optimization

If we examine some of the more recent trends, it is interesting to see that the use of several neighborhood structures is becoming increasingly popular. As already anticipated in [178,184] (where agents used different local search techniques for the TSP), this can enhance the final performance of an MA. Quite interestingly, some of them involve some sort of learning or adaptation procedure to guide the basic search mechanisms [48]. A special case of these is those that involve reinforcement learning and their variants [226,251,278,284]. In particular, Q-learning seems to be a very popular option in this regard [52,144,146,160,201,244,262,269,274].

We have also identified numerous recent combinations with mathematical programming techniques, such as linear programming [242], mixed integer linear

Table 3 Some varietal combinations of heuristics forming MAs

Population-based heuristic	Individual improvement heuristic	References
Ant colony optimization	Local search	[53, 81, 86, 149]
Bee colony optimization	—	[31, 33]
	Random optimization	[18]
	Nelder-Mead simplex	[88]
—	Nelder-Mead simplex with bidirectional random optimization	[2]
Binary differential evolution	Tabu search	[103]
Continuous differential evolution	Pool of strategies	[120, 237]
	Hooke-Jeeves-like	[202]
	Stochastic local search	[190]
Cross-entropy	Hill climbing, tabu search	[9]
Genetic algorithm	Local search	[13, 21]
	Tabu search	[26, 32, 94, 148, 151, 247, 250]
	Mathematical programming	[240]
Genetic algorithm with particle swarm optimization	—	[25, 116]
Particle swarm optimization	Local search	[16, 35, 78, 117, 272]
	Variable neighborhood search	[7, 157, 262, 270]
	Sequential quadratic programming	[207]
Particle swarm optimization with differential evolution	Nelder-Mead simplex with Rosenbrock algorithm	[42]

programming [1], constraint programming [11], dynamic programming [273], chance-constrained programming (in the form of an adaptive differential evolution algorithm with Q-learning) [144], or continuation Newton methods with deflation techniques for global optimization [163]. This is certainly the tip of the iceberg. Several techniques under the denomination of *matheuristics* can to some extent fall under the umbrella of the broad definition of MAs, since they are based on a population-based strategy with refinements using these mathematical programming methods. Other methods involve large neighborhood search which is explored via solving a mixed-integer programming subproblem (similarly, others use variable neighborhood search to provide exploration and rigorous local optimization of a population). Finally, there are several complete memetic algorithms [66] in which exact solvers are available to some of the agents in the population. In that way, one or more agents running branch and bound, or branch and cut and price method, for instance, are dedicated to solve large-scale integer programming problems, while other agents use independent search procedures guided by constructive heuristics or local search ones. We are confident that readers will recognize the memetic approach in some of these new procedures and reviewing all this material is a bit

out of the scope of this current review, but it shows a great promising new area of development for more powerful MAs now that computational resources allow these more complex search strategies.

Future-Generation Memetic Algorithms

Back in the days when MAs were just a nascent approach for optimization, different visions of what MAs would be in the future were foreseen. Among these, perhaps the one that has come closest to reality refers to the self-⋆ capabilities [23] of the paradigm and more precisely to self-generation properties. Early works envisioned that the algorithm could work on two timescales, one in which the solutions would be optimized and another in which the problem solving strategies used to optimize the solutions themselves would be optimized [180]. In essence, this has been a long standing goal in metaheuristics. It is widely acknowledged that designing an effective problem-solving technique is in itself a difficult task. Attempting to transfer a part of this design effort to the actual metaheuristic is just the logical course of action [65]—see, for example, the corpus of research in hyperheuristics [44, 67]. This latter approach is actually related to what has been termed meta-Lamarckian learning [196], a memetic approach in which a collection of local searchers is available and there is a decision-maker that decides which of them should be applied to specific solutions based on different criteria (e.g., the past performance of each local searcher, the adequacy of the current solution for being improved by a certain local searcher according to past experience, etc.). Multi-memetic or multi-meme algorithms provided a much more general approach [137, 138, 140]. In this approach, an encoding of a local searcher (ranging from the definition of the neighborhood or pivot rule used, up to a full algorithmic description of the procedure) is attached to each solution and evolves alongside it. Thus, the algorithm looks not only for improved solutions but also for algorithmic structures capable of improving the latter. The next natural step is to detach these memes from the genes and have them evolve in separate populations [220–222], paving the way for the emergence of complex structures of interacting memes [50]. An overview of adaptation in MAs is provided in [197]. This view of memes as explicit representations of problem-solving strategies that interact in a complex and dynamic way within an evolutionary context for optimization purposes leads to the notion of memetic computing [191, 198]—see [189] for a literature review on memetic computing.

Also related to the design of the problem solver, albeit not in an autonomic way, transfer learning and knowledge sharing are techniques that are being attempted, with varied results, to transfer knowledge between problem instances. Other more recent techniques suggest looking back at the transformation between problems, prototypical of the NP-completeness proofs, for instance, and using different transformations than the standard ones to speed up the convergence of memetic algorithms to optimal solutions. An interesting example can be found in [6] in which massive speedups have been obtained, thus allowing to solve very challenging

Hamiltonian cycle problems using a memetic algorithm for the TSP. In addition, other learning techniques are being used to predict suitable algorithm configurations or help design local search operators based on the identifying characteristics of the problem instance.

Another dimension along which some early ideas (far-fetched at their time) about MAs may become a reality is parallel computing. The deployment of metaheuristics in parallel and/or distributed environments is by no means new [4] and has been extensively used since the late 1980s; see, for example, [186, 232]. However, the continuous evolution of computational platforms is dragging these parallel modes along, forcing them to adapt to new scenarios. Thus, while early work often assumed dedicated local area networks, nowadays we have emerging computational environments such as peer-to-peer networks [174] and volunteer computing networks [215], which are much more pervasive, of a larger scale and inherently dynamic. The complex and dynamic structure of these computational substrates is undoubtedly a challenge. Fortunately, population-based metaheuristics have been shown to be intrinsically robust on a fine-grain scale [129] and can be endowed with appropriate churn-aware strategies if required [192]. They are therefore ripe for being deployed on these platforms to exploit the possibilities they offer. In this line—and connected to the previous discussion on meme evolution and interaction—some initial concepts revolving around "meme pools," that is, repositories of problem solving methods to be used synergistically, acquire a new scope more akin to service-oriented architectures [96]. Furthermore, to build on the idea of automated self-design of the MA requires the ability to keep or gather some sort of distributed knowledge about the state of the search and make design decisions on its basis. Some ideas from multi-agent systems and epistemic logic were proposed as potential tools for this purpose [59], but the concept is still largely unexplored.

There are a number of recent areas that we consider that are going to expand the types of MA we will see in the near future. One is the use of reinforcement learning within the memetic strategy. This could help to adaptively select or generate individual search operators for the agents based on a learning outcome from the previous search experience. This assumes that the strategy is not optimal and needs to be constantly varied over time. Another one is the use of either deep learning, symbolic regression, or continued fraction regression to approximate objective function (or guiding or fitness function) such that they can act as surrogate models in high-dimensional and computationally expensive instances of problems. Since the introduction of the CONCORDE algorithm for the TSP, it was clear that the use of dynamic programming algorithms can help optimally solve some tasks, such as finding the best possible offspring after a recombination step [62]. The exact optimization of these subproblems can give fantastic leaps of performance to memetic algorithms that also incorporate stochasticity in other parts of their design. There are also opportunities for the development of MAs (and GAs) on a small scale. Any use of recombination operators is naturally limited by the expectation that the recombination step will be performed many, many times during a run of the algorithm. This leads to the requirement that a recombination operator be able to be

implemented very efficiently. Traditionally, this would mean at most linear or close to linear time in the size of the individual (of course, ideally constant time). This immediately rules out the possibility of optimal recombination strategies for many problems, as such strategies are typically NP-hard. Parameterized complexity, for example, offers an opportunity to exploit the naturally arising parameters in many recombination strategies. If such parameters are small, or can be made small, then we may effectively reduce the complexity of optimal recombination to polynomial time [60, 83–85]. For further reading on the challenges raised by evolutionary approaches to optimization, many of the problems posed in [59] remain open.

In general, these trends discussed above lead to increasingly complex algorithmic architectures. In this context, and very much in the same vein as deep learning algorithms, which use multiple processing layers to learn data representations with multiple levels of abstraction [141], the notion of *deep memetic algorithm* [10] can be considered as an MA with multiple interconnected layers or components, each of them encapsulating the knowledge or tools required to tackle some computational issue or some problem specificity.

Conclusion

Since their primordial conception in the late 1980s, MAs have developed to become one of the most adaptable and flexible metaheuristic approaches available. Although many heuristic techniques perform well for some problems, the No-Free-Lunch Theorem [254] guarantees that their performance falters on the majority of problems. MAs, with their insistence on adaptability and utilitarianism (both on the part of the algorithm and the implementer), are free to exploit the performance of multiple approaches and choose the best suited for the problem at hand.

This adaptability, efficiency, and amenability to the current availability of large-scale parallelism, including traditional parallel architectures along with GPU computing, cloud, and peer-to-peer approaches, along with a tendency toward modularity in implementation, has led to their adoption across a broad range of fields with excellent results. Research on MAs has grown dramatically since its inception. With nearly 4000 academic papers published to date, the field is vibrant and dynamic. More importantly, MAs can be regarded as a methodological approach that can complement and cooperate with the many effective tools currently existing in the heuristic toolbox and can be helpful as a framework for articulating complex synergistic heuristics whenever the target problem thus requires.

Cross-References

▸ Adaptive and Multi-level Metaheuristics
▸ Deep Learning in Search Heuristics
▸ Evolutionary Algorithms

- Fundamentals of Scatter Search
- Genetic Algorithms
- Tabu Search
- Variable Neighborhood Search

Acknowledgments Carlos Cotta acknowledges the support of MICINN under the project Bio4Res (PID2021-125184NB-I00) and from the Universidad de Málaga, Campus de Excelencia Internacional Andalucía Tech.

Pablo Moscato acknowledges support from the Australian Research Council Future Fellowship FT120100060 and the Australian Research Council Discovery Projects DP120102576 and DP140104183. P.M. also acknowledges a generous donation from the Maitland Cancer Appeal. Profs. Moscato and Cotta also acknowledge joint support through the Australian Research Council's Discovery Projects funding scheme (project DP200102364).

Competing Interest Declaration The authors has no competing interests to declare that are relevant to the content of this manuscript.

References

1. Afifi S, Hrouga M, Mjirda A, Allaoui H (2024) A memetic based algorithm for simultaneous preventive maintenance scheduling and spare-parts inventory management for manufacturing systems. Appl. Soft Comput. 151:111161. https://doi.org/10.1016/j.asoc.2023.111161
2. Ahandani MA, Vakil-Baghmisheh MT, Talebi M (2014) Hybridizing local search algorithms for global optimization. Comput Optim Appl 59(3):725–748
3. Ajmi F, Ajmi F, Ben Othman S, Zgaya-Biau H, Dotoli M, Renard JM, Hammadi S (2024) Real time patient scheduling orchestration for improving key performance indicators in a hospital emergency department. J Comput Sci 82:102422. https://doi.org/10.1016/j.jocs.2024.102422
4. Alba E (2005) Parallel metaheuristics: a new class of algorithms. Wiley-Interscience, Hoboken
5. Aldous D, Vazirani U (1994) "go with the winners" algorithms. In: Proceedings of 35th IEEE symposium on foundations of computer science, IEEE Press, pp 492–501
6. Ali S, Moscato P (2024) A memetic algorithm to find a Hamiltonian cycle in a Hamiltonian graph. https://arxiv.org/abs/2403.07886. 2403.07886
7. Ali AF, Hassanien AE, Snasel V, Tolba MF (2014) A new hybrid particle swarm optimization with variable neighborhood search for solving unconstrained global optimization problems. In: Kromer P, Abraham A, Snasel V (eds) Fifth international conference on innovations in bio-inspired computing and applications, Springer Berlin, Ostrava, Czech Republic, Advances in intelligent systems and computing, vol 303, pp 151–160
8. Alkhamis AK, Hosny M (2023) A multi-objective simulated annealing local search algorithm in memetic CENSGA: application to vaccination allocation for influenza. Sustainability 15(21):15347. https://doi.org/10.3390/su152115347
9. Amaya JE, Cotta C, Fernández-Leiva AJ (2013) Cross entropy-based memetic algorithms: an application study over the tool switching problem. Int J Comput Intell Syst 6(3):559–584
10. Amaya JE, Cotta C, Fernández AJ, García-Sánchez P (2020) Deep memetic models for combinatorial optimization problems: application to the tool switching problem. Memetic Comput 12(1):3–22
11. Andrade CE, Pessoa LS, Stawiarski S (2024) The physical cell identity assignment problem: A practical optimization approach. IEEE Trans Evolut Comput 28(2):282–292. https://doi.org/10.1109/Tevc.2022.3185927

12. Antonio CC (2024) Memeplex-based memetic algorithm for the multi-objective optimal design of composite structures. Composite Struct 329:117789. https://doi.org/10.1016/j.compstruct.2023.117789
13. Arab A, Alfi A (2015) An adaptive gradient descent-based local search in memetic algorithm applied to optimal controller design. Inf Sci 299:117–142
14. Atasagun GC, Karaoglan I (2024) Integrated production and outbound distribution scheduling problem with multiple facilities/vehicles and perishable items. Appl Soft Comput 166:112144. https://doi.org/10.1016/j.asoc.2024.112144
15. Awais M, Ahmad R, Kausar N, Alzahrani AI, Alalwan N, Masood A (2024) ALL classification using neural ensemble and memetic deep feature optimization. Front Artif intell 7. https://doi.org/10.3389/frai.2024.1351942
16. Aziz M, Tayarani-N MH (2014) An adaptive memetic particle swarm optimization algorithm for finding large-scale Latin hypercube designs. Eng Appl Artif Intell 36:222–237
17. Ba Z, Yuan Y, Liu J (2024) A modified memetic algorithm with multi-operation precise joint movement neighbourhood structure for the assembly job shop scheduling problem. Int J Product Res 62(17):6292–6324. https://doi.org/10.1080/00207543.2024.2313087
18. Baghmisheh MTV, Ahandani MA, Talebi M (2008) Frequency modulation sound parameter identification using novel hybrid evolutionary algorithms. In: International symposium on telecommunications. IEEE, Tehran, pp 67–72
19. Bazirha M, Kadrani A, Benmansour R (2024) Pareto and decomposition based approaches for the multi-objective home health care routing and scheduling problem with lunch breaks. Eng Appl Artif Intell 128:107502. https://doi.org/10.1016/j.engappai.2023.107502
20. Beke L, Uribe L, Lara A, Coello CAC, Weiszer M, Burke EK, Chen J (2024) Routing and scheduling in multigraphs with time constraints-a memetic approach for airport ground movement. IEEE Trans Evolut Comput 28(2):474–488. https://doi.org/10.1109/Tevc.2023.3262743
21. Benlic U, Hao JK (2015) Memetic search for the quadratic assignment problem. Expert Sys Appl 42(1):584–595
22. Berchez-Moreno F, Duran-Rosal AM, Hervas Martinez C, Gutierrez PA, Fernandez JC (2024) A memetic dynamic coral reef optimisation algorithm for simultaneous training, design, and optimisation of artificial neural networks. Sci Rep 14(1):6961. https://doi.org/10.1038/s41598-024-57654-2
23. Berns A, Ghosh S (2009) Dissecting self-$\star$ properties. In: Third IEEE international conference on self-adaptive and self-organizing systems. IEEE Press, San Francisco, pp 10–19
24. Berretta R, Cotta C, Moscato P (2012) Memetic algorithms in bioinformatics. In: Neri F, Cotta C, Moscato P (eds) Handbook of memetic algorithms, studies in computational intelligence, vol 379. Springer, Berlin, pp 261–271
25. Biao S, Hua HC, Hua YX, Chuan H (2014) Mutation particle swarm optimization algorithm for solving the optimal operation model of thermal power plants. J Renew Sustain Energy 6(4):043118
26. Bilal N, Galinier P, Guibault F (2014) An iterated-tabu-search heuristic for a variant of the partial set covering problem. J Heurist 20(2):143–164
27. Blum C, Roli A (2003) Metaheuristics in combinatorial optimization: overview and conceptual comparison. ACM Comput Surv 35(3):268–308
28. Blum C, Blesa Aguilera MJ, Roli A, Sampels M (2008) Hybrid metaheuristics: an emerging approach to optimization, studies in computational intelligence, vol 144. Springer, Berlin
29. Blum C, Cotta C, Fernández AJ, Gallardo JE, Mastrolilli M (2008) Hybridizations of metaheuristics with branch & bound derivates. In: Blum C, Blesa Aguilera MJ, Roli A, Sampels M (eds) Hybrid metaheuristics: an emerging approach to optimization, studies in computational intelligence, vol 144. Springer, Berlin, pp 85–116
30. Blum C, Puchinger J, Raidl GR, Roli A (2011) Hybrid metaheuristics in combinatorial optimization: a survey. Appl Soft Comput 11(6):4135–4151
31. Bose D, Biswas S, Vasilakos AV, Laha S (2014) Optimal filter design using an improved artificial bee colony algorithm. Inf Sci 281:443–461

32. Boskovic B, Brglez F, Brest J (2014) Low-autocorrelation binary sequences: on the performance of memetic-tabu and self-avoiding walk solvers. CoRR abs/1406.5301
33. Bullinaria JA, AlYahya K (2014) Artificial bee colony training of neural networks: comparison with back-propagation. Memetic Comput 6(3):171–182
34. Burmeister SC, Guericke D, Schryen G (2024) A memetic NSGA-II for the multi-objective flexible job shop scheduling problem with real-time energy tariffs. Flexible Serv Manuf J 36:1571–1573. https://doi.org/10.1007/s10696-023-09530-w
35. Cai K, Zhang J, Zhou C, Cao X, Tang K (2012) Using computational intelligence for large scale air route networks design. Appl Soft Comput 12(9):2790–2800
36. Cai S, Wang S, Chen M (2024) Finding robust and influential nodes from networks under cascading failures using a memetic algorithm. Neurocomputing 589:127704. https://doi.org/10.1016/j.neucom.2024.127704
37. Cao B, Zhao J, Liu X, Li Y (2024) Adaptive 5G-and-beyond network-enabled interpretable federated learning enhanced by neuroevolution. Sci China-Inf Sci 67(7):170306. https://doi.org/10.1007/s11432-023-4011-4
38. Cao L, Wang M, Vasile M, Dai G (2024) Multi agent collaborative search algorithm with adaptive weights. Expert Syst 41:e13709. https://doi.org/10.1111/exsy.13709
39. Capek M, Jelinek L, Kadlec P, Gustafsson M (2023) Optimal inverse design based on memetic algorithms-part I: theory and implementation. IEEE Trans Antennas Propag 71(11):8806–8816. https://doi.org/10.1109/Tap.2023.3308587
40. Capek M, Jelinek L, Kadlec P, Gustafsson M (2023) Optimal inverse design based on memetic algorithms-part II: examples and properties. IEEE Trans Antennas Propag 71(11):8817–8829. https://doi.org/10.1109/Tap.2023.3308583
41. Caponio A, Neri F (2012) Memetic algorithms in engineering and design. In: Neri F, Cotta C, Moscato P (eds) (2012) Handbook of memetic algorithms, studies in computational intelligence, vol 379. Springer, Berlin, pp 241–260
42. Caponio A, Neri F, Tirronen V (2009) Super-fit control adaptation in memetic differential evolution frameworks. Soft Comput 13(8–9, SI):811–831
43. Cegla M, Engell S (2023) Optimal design and operation of reactive extrusion processes: modeling, model validation, and optimization algorithm. Polymer Eng Sci 63(12):4153–4173. https://doi.org/10.1002/pen.26515
44. Chakhlevitch K, Cowling P (2008) Hyperheuristics: recent developments. In: Cotta C, Sevaux M, Sörensen K (eds) Adaptive and multilevel metaheuristics, studies in computational intelligence, vol 136, Springer, Berlin, pp 3–29
45. Chakraborty A, Singh i, Bhattacharya S, Sharma KP, Namdev V, Shukla AK, Alkhayyat A, Shah SK (2023) Multiple pattern synthesis in four-dimensional antenna arrays using the quasi-Newton memetic optimization method. IETE J Res 70:6055–6068. https://doi.org/10.1080/03772063.2023.2294867
46. Chang Z, Zhou Z (2024) Three multi-objective memetic algorithms for observation scheduling problem of active-imaging agile earth observation satellites. Ann Oper Res 346:861–893. https://doi.org/10.1007/s10479-024-06156-5
47. Chang X, Jia X, Ren J (2024) A reinforcement learning enhanced memetic algorithm for multi-objective flexible job shop scheduling toward industry 5.0. Int J Prod Res 63:119–147. https://doi.org/10.1080/00207543.2024.2357740
48. Chang X, Jia X, Ren J (2025) A reinforcement learning enhanced memetic algorithm for multi-objective flexible job shop scheduling toward industry 5.0. Int J Prod Res 63(1):119–147. https://doi.org/10.1080/00207543.2024.2357740
49. Changazi SA, Bakhshi AD, Yousaf M, Mohsin SM, Akber SMA, Abazeed M, Ali M (2024) Optimization of network topology robustness in IoTs: a systematic review. Comput Netw 250:110568. https://doi.org/10.1016/j.comnet.2024.110568
50. Chen X, Ong YS (2012) A conceptual modeling of meme complexes in stochastic search. IEEE Trans Syst Man Cyber Part C 42(5):612–625
51. Chen L, Zhang J, Zhang X, Liu H (2024) A new low-carbon project scheduling problem with renewable and traditional energy: a comprehensive analysis and its solution. J Cleaner Prod 468:143089. https://doi.org/10.1016/j.jclepro.2024.143089

52. Chen TY, Miao ZH, Li WM, Pan QK (2024) A learning-based memetic algorithm for a cooperative task allocation problem of multiple unmanned aerial vehicles in smart agriculture. Swarm Evolut Comput 91:101694. https://doi.org/10.1016/j.swevo.2024.101694
53. Chen X, Ong YS, Feng L, Lim MH, Chen C, Ho CS (2013) Towards believable resource gathering behaviours in real-time strategy games with a memetic ant colony system. In: Cho S, Sato A, Kim K (eds) 17th Asia pacific symposium on intelligent and evolutionary systems, Elsevier, Seoul, South Korea, procedia computer science, vol 24, pp 143–151
54. Cosma O, Pop PC, Cosma L (2024) A novel memetic algorithm for solving the generalized traveling salesman problem. Logic J IGPL 32:576–588. https://doi.org/10.1093/jigpal/jzae019
55. Cotta C (2003) Protein structure prediction using evolutionary algorithms hybridized with backtracking. In: Mira J, Álvarez J (eds) Artificial neural nets problem solving methods. Lecture notes in computer science, vol 2687. Springer, Berlin, pp 321–328
56. Cotta C (2005) Memetic algorithms with partial Lamarckism for the shortest common supersequence problem. In: Mira J, Álvarez J (eds) Artificial intelligence and knowledge engineering applications: a bioinspired approach. Lecture notes in computer science, vol 3562. Springer, Berlin, pp 84–91
57. Cotta C (2025) Harnessing memetic algorithms: a practical guide. TOP https://doi.org/10.1007/s11750-024-00694-8
58. Cotta C, Fernández A (2007) Memetic algorithms in planning, scheduling, and timetabling. In: Dahal K, Tan K, Cowling P (eds) Evolutionary scheduling, studies in computational intelligence, vol 49. Springer, Berlin, pp 1–30
59. Cotta C, Moscato P (2004) Evolutionary computation: challenges and duties. In: Menon A (ed) Frontiers of evolutionary computation. Kluwer Academic Publishers, Boston, pp 53–72
60. Cotta C, Moscato P (2005) The parameterized complexity of multiparent recombination. In: Proceedings of the 6th metaheuristic international conference – MIC 2005, Universität Wien, Vienna, pp 237–242
61. Cotta C, Troya JM (2000) On the influence of the representation granularity in heuristic forma recombination. In: Carroll J, Damiani E, Haddad H, Oppenheim D (eds) Applied computing 2000. ACM Press, New York, pp 433–439
62. Cotta C, Troya JM (2003) Embedding branch and bound within evolutionary algorithms. Appl Intell 18(2):137–153
63. Cotta C, Dotú I, Fernández AJ, Van Hentenryck P (2006) Scheduling social golfers with memetic evolutionary programming. In: Almeida F, et al (eds) Hybrid metaheuristics – HM 2006. Lecture notes in computer science, vol 4030. Springer, Berlin, pp 150–161
64. Cotta C, Dotú I, Fernández AJ, Van Hentenryck P (2007) Local search-based hybrid algorithms for finding Golomb rulers. Constraints 12(3):263–291
65. Cotta C, Sevaux M, Sörensen K (2008) Adaptive and multilevel metaheuristics, studies in computational intelligence, vol 136. Springer, Berlin
66. Cotta C, Fernández Leiva AJ, Gallardo JE (2012) Memetic algorithms and complete techniques. In: Neri F, Cotta C, Moscato P (eds) (2012) Handbook of memetic algorithms, studies in computational intelligence, vol 379. Springer, Berlin, pp 189–200
67. Cowling P, Kendall G, Soubeiga E (2008) A hyperheuristic approach to schedule a sales submit. In: Burke E, Erben W (eds) PATAT 2000. Lecture notes in computer science, vol 2079. Springer, Berlin, pp 176–190
68. Dash PB, Senapati MR, Behera HS, Nayak J, Vimal S (2024) Self-adaptive memetic firefly algorithm and CatBoost-based security framework for IoT healthcare environment. J Eng Math 144(1):6. https://doi.org/10.1007/s10665-023-10309-z
69. Davila A, Colan J, Hasegawa Y (2024) Real-time inverse kinematics for robotic manipulation under remote center-of-motion constraint using memetic evolution. J Comput Design Eng 11(3):248–264. https://doi.org/10.1093/jcde/qwae047
70. Davis L (1991) Handbook of genetic algorithms. Van Nostrand Reinhold Computer Library, New York
71. Dawkins R (1976) The selfish gene. Clarendon Press, Oxford

72. de la Pisa MP, Molina JC, Eguia I (2024) Ant colony algorithms for minimizing costs in multi-mode resource constrained project scheduling problems with spatial constraints. Int J Ind Eng Comput 15(2):667–684. https://doi.org/10.5267/j.ijiec.2024.5.002
73. Dechter R (1999) Bucket elimination: a unifying framework for reasoning. Artif Intell 113(1–2):41–85
74. Deng J, Wang J, Wang X, Cai Y, Liu P (2024) Multi-task multi-objective evolutionary search based on deep reinforcement learning for multi-objective vehicle routing problems with time windows. Symmetry-Basel 16(8):1030. https://doi.org/10.3390/sym16081030
75. Deng L, Qiu Y, Gong W, Di Y, Li C (2024) A dynamic decision-driven memetic algorithm for fuzzy distributed hybrid flow shop rescheduling considering quality control. Expert Syst Appl 257:125002. https://doi.org/10.1016/j.eswa.2024.125002
76. Deng L, Qiu Y, Di Y, Zhang L (2025) A knowledge-driven memetic algorithm for distributed green flexible job shop scheduling considering the endurance of machines. Appl Soft Comput 170:112697. https://doi.org/10.1016/j.asoc.2025.112697
77. Detcher R, Rish I (2003) Mini-buckets: a general scheme for bounded inference. J ACM 50(2):107–153
78. Devi S, Jadhav DG, Pattnaik SS (2011) PSO based memetic algorithm for unimodal and multimodal function optimization. In: Panigrahi B, Suganthan P, Das S, Satapathy S (eds) 2nd swarm, evolutionary and memetic computing conference. Lecture notes in computer science, vol 7076. Springer, Berlin, pp 127–134
79. Droste S, Jansen T, Wegener I (1999) Perhaps not a free lunch but at least a free appetizer. In: Banzhaf W, Daida JM, Eiben AE, Garzon MH, Honavar V, Jakiela MJ, Smith RE (eds) Proceedings of the first genetic and evolutionary computation conference – GECCO 1999, Morgan Kaufmann, Orlando, FL, pp 833–839
80. Droste S, Jansen T, Wegener I (2002) Optimization with randomized search heuristics - the (A)NFL theorem, realistic scenarios, and difficult functions. Theoret Comput Sci 287(1): 131–144
81. Duan H, Yu X (2007) Hybrid ant colony optimization using memetic algorithm for traveling salesman problem. In: IEEE international symposium on approximate dynamic programming and reinforcement learning. IEEE, Honolulu, pp 92–95
82. Eiben AE, Raue PE, Ruttkay Z (1994) Genetic algorithms with multi-parent recombination. In: Davidor Y, Schwefel HP, Männer R (eds) Parallel problem solving from nature III. Lecture notes in computer science, vol 866, Springer, Berlin, pp 78–87
83. Eremeev AV (2008) On complexity of optimal recombination for binary representations of solutions. Evolut Comput 16(1):127–147
84. Eremeev AV (2011) On complexity of the optimal recombination for the travelling salesman problem. In: Proceedings of the 11th European conference on evolutionary computation in combinatorial optimization. Lecture notes in computer science, vol 6622. Springer, Berlin, pp 215–225
85. Eremeev AV, Kovalenko JV (2014) Optimal recombination in genetic algorithms, Part II. Yugoslav J Oper Res 24(2):165–186
86. Fathi M, Rodriguez V, Jesus Alvarez M (2014) A novel memetic ant colony optimization-based heuristic algorithm for solving the assembly line part feeding problem. Int J Adv Manuf Technol 75(1–4):629–643
87. Filgueiras JL, Santos J (2024) Refinement of protein structures with a memetic algorithm. examples with SARS-CoV-2 proteins. In: Vicente J, Calvo M, Adeli H (eds) Bioinspired systems for translational applications: from robotics to social engineering. Lecture notes in computer science, vol 14675, pp 129–139. https://doi.org/10.1007/978-3-031-61137-7_13
88. Fister I, Fister I Jr, Brest J, Zumer V (2012) Memetic artificial bee colony algorithm for large-scale global optimization. In: IEEE congress on evolutionary computation (CEC 2012), IEEE, Brisbane, IEEE congress on evolutionary computation

89. Fogel LJ, Owens AJ, Walsh MJ (1966) Artificial intelligence through simulated evolution. Wiley, New York
90. França PM, Gupta JND, Mendes AS, Moscato P, Veltnik KJ (2005) Evolutionary algorithms for scheduling a flowshop manufacturing cell with sequence dependent family setups. Comput Ind Eng 48:491–506
91. Freisleben B, Merz P (1996) A genetic local search algorithm for solving symmetric and asymmetric traveling salesman problems. In: 1996 IEEE international conference on evolutionary computation, Nagoya, IEEE Press, pp 616–621
92. Fu Y, Ma X, Gao K, Li Z, Dong H (2024) Multi-objective home health care routing and scheduling with sharing service via a problem-specific knowledge-based artificial bee colony algorithm. IEEE Trans Intell Transport Syst 25(2):1706–1719. https://doi.org/10.1109/Tits.2023.3315785
93. Gallardo JE, Cotta C, Fernández AJ (2006) A memetic algorithm with bucket elimination for the still life problem. In: Gottlieb J, Raidl G (eds) Evolutionary computation in combinatorial optimization. Lecture notes in computer science, vol 3906. Springer, Berlin, pp 73–85
94. Gallardo JE, Cotta C, Fernández AJ (2009) Finding low autocorrelation binary sequences with memetic algorithms. Appl Soft Comput 9(4):1252–1262
95. Gallardo JE, Cotta C, Fernández AJ (2009) Solving weighted constraint satisfaction problems with memetic/exact hybrid algorithms. J Artif Intell Res 35:533–555
96. García-Sánchez P, González J, Castillo P, Arenas M, Merelo-Guervós J (2013) Service oriented evolutionary algorithms. Soft Comput 17(6):1059–1075
97. Garza-Fabre M, Erazo-Agredo CC, Rubio-Loyola J (2024) A memetic algorithm for improved joint route selection and split-level management in next-generation wireless communications. Memetic Comput 16(3):315–336. https://doi.org/10.1007/s12293-024-00418-2
98. Gibbons E, Ombuki-Berman B (2024) A memetic algorithm for large-scale real-world vehicle routing problems with simultaneous pickup and delivery with time windows. In: Sevaux M, Olteanu A, Pardo E, Sifaleras A, Makboul S (eds) MIC 2024. Lecture notes in computer science, vol 14753, pp 78–92. https://doi.org/10.1007/978-3-031-62912-9_8
99. Goldberg DE (1989) Genetic algorithms in search, optimization and machine learning. Addison-Wesley Longman, Boston
100. Goldberg DE, Lingle Jr R (1985) Alleles, loci, and the traveling salesman problem. In: Grefenstette JJ (ed) Proceedings of the first international conference on genetic algorithms and their applications, Lawrence Erlbaum Associates, Publishers
101. Gong G, Tang J, Huang D, Luo Q, Zhu K, Peng N (2024) Energy-efficient flexible job shop scheduling problem considering discrete operation sequence flexibility. Swarm Evolut Comput 84:101421. https://doi.org/10.1016/j.swevo.2023.101421
102. Grelier C, Goudet O, Hao JK (2024) A memetic algorithm with adaptive operator selection for graph coloring. In: Stutzle T, Wagner M (eds) Evolutionary computation in combinatorial optimization. Lecture notes in computer science, vol 14632, pp 65–80. https://doi.org/10.1007/978-3-031-57712-3_5
103. Gu X, Li Y, Jia J (2015) Feature selection for transient stability assessment based on kernelized fuzzy rough sets and memetic algorithm. Int J Electr Power Energy Syst 64:664–670
104. Gupta R, Gill SS (2024) A new representation in 3D VLSI floorplan: 3D O-Tree. Genetic Programm Evolvable Mach 25(1). https://doi.org/10.1007/s10710-024-09485-3
105. Guzowski H, Smolka M, Pekai L (2024) Enhancing a hierarchical evolutionary strategy using the nearest-better clustering. In: Franco L, DeMulatier C, Paszynski M, Krzhizhanovskaya V, Dongarra J, Sloot P (eds) Computational science. Lecture notes in computer science, vol 14834, pp 423–437. https://doi.org/10.1007/978-3-031-63759-9_43
106. Hajibabaei M, Behnamian J (2024) Considering environmental impacts in flexible assembly job shop scheduling: non-dominated sorting memetic algorithm. Flex Serv Manuf J. https://doi.org/10.1007/s10696-024-09553-x
107. Han N, Chen Y, Ye L, Zhao X (2024) Stock portfolio optimization based on factor analysis and second-order memetic differential evolution algorithm. Memetic Comput 16(1):29–44. https://doi.org/10.1007/s12293-024-00405-7

108. Hansen P, Mladenović N (2001) Variable neighborhood search: Principles and applications. Eur J Oper Res 130(3):449–467
109. Hao J (2012) Memetic algorithms in discrete optimization. In: Neri F, Cotta C, Moscato P (eds) (2012) Handbook of memetic algorithms, studies in computational intelligence, vol 379. Springer, Berlin, pp 73–94
110. Hart W, Belew R (1991) Optimizing an arbitrary function is hard for the genetic algorithm. In: Belew R, Booker L (eds) Fourth international conference on genetic algorithms. Morgan Kaufmann, San Mateo, pp 190–195
111. He M, Wu Q, Benlic U, Lu Y, Chen Y (2024) An effective multi-level memetic search with neighborhood reduction for the clustered team orienteering problem. Eur J Oper Res 318(3):778–801. https://doi.org/10.1016/j.ejor.2024.06.015
112. He P, Hao JK, Wu Q (2024) A hybrid genetic algorithm for the Hamiltonian p-median problem. Networks 83(2):348–367. https://doi.org/10.1002/net.22197
113. Holland JH (1992) Adaptation in natural and artificial systems. MIT Press, Cambridge
114. Hong W, Li G, Liu S, Yang P, Tang K (2024) Multi-objective evolutionary optimization for hardware-aware neural network pruning. Fundamental Res 4(4):941–950. https://doi.org/10.1016/j.fmre.2022.07.013
115. Houck C, Joines J, Kay M, Wilson J (1997) Empirical investigation of the benefits of partial Lamarckianism. Evolut Comput 5(1):31–60
116. Hu M, Weir JD, Wu T (2014) An augmented multi-objective particle swarm optimizer for building cluster operation decisions. Appl Soft Comput 25:347–359
117. Hu Z, Bao Y, Xiong T (2014) Comprehensive learning particle swarm optimization based memetic algorithm for model selection in short-term load forecasting using support vector regression. Appl Soft Comput 25:15–25
118. Huang K, Gong W, Lu C (2024) An enhanced memetic algorithm with hierarchical heuristic neighborhood search for type-2 green fuzzy flexible job shop scheduling. Eng Appl Artif Intell 130:107762. https://doi.org/10.1016/j.engappai.2023.107762
119. Huo Z, Deng Q, Liu X, Miao B, Ren Y (2023) Integrated decision-making for repair order acceptance and service resource allocation that distinguishes between in-warranty and out-of-warranty. Comput Ind Eng 185:109691. https://doi.org/10.1016/j.cie.2023.109691
120. Iacca G, Neri F, Caraffini F, Suganthan PN (2014) A differential evolution framework with ensemble of parameters and strategies and pool of local search algorithms. In: Esparcia-Alcázar AI, Mora AM (eds) Applications of evolutionary computation. Lecture notes in computer science, vol 8602. Springer, Berlin, pp 615–626
121. Ibaraki T (1997) Combination with dynamic programming. In: Bäck T, Fogel D, Michalewicz Z (eds) Handbook of evolutionary computation. Oxford University Press, New York, pp D3.4:1–2
122. Ibrahim MK, Yusof UK, Eisa TAE, Nasser M (2024) Bioinspired algorithms for multiple sequence alignment: a systematic review and roadmap. Appl Sci-Basel 14(6):2433. https://doi.org/10.3390/app14062433
123. Iranagh M, Sonmez R, Atan T, Uysal F, Bettemir OH (2023) A memetic algorithm for the solution of the resource leveling problem. Buildings 13(11):2738. https://doi.org/10.3390/buildings13112738
124. Ishibuchi H, Yoshida T, Murata T (2003) Balance between genetic search and local search in memetic algorithms for multiobjective permutation flowshop scheduling. IEEE Trans Evolut Comput 7(2):204–223
125. Jain AK, Jain S, Mathur G (2024) Design and performance assessment of improved evolutionary computing based LEACH protocol for energy efficient and lifetime extension of wireless sensor network. Eng Res Express 6(2):025213. https://doi.org/10.1088/2631-8695/ad4aeb
126. Jian SJ, Hsieh SY (2023) A niching regression adaptive memetic algorithm for multimodal optimization of the Euclidean traveling salesman problem. IEEE Trans Evolut Comput 27(5):1413–1426. https://doi.org/10.1109/Tevc.2022.3211954

127. Jiang B, Ma Y, Chen L, Huang B, Huang Y, Guan L (2023) A review on intelligent scheduling and optimization for flexible job shop. Int J Control Autom Syst 21(10):3127–3150. https://doi.org/10.1007/s12555-023-0578-1
128. Jiao D, Wang L, Yang P, Yang W, Peng Y, Shang Z, Ren F (2024) Unmanned aerial vehicle-enabled grassland restoration with energy-sensitive of trajectory design and restoration areas allocation via a cooperative memetic algorithm. Eng Appl Artif Intell 133(A):108084. https://doi.org/10.1016/j.engappai.2024.108084
129. Jiménez Laredo JL, Bouvry P, Lombraña González D, Fernández de Vega F, García Arenas M, Merelo Guervós JJ, Fernandes CM (2014) Designing robust volunteer-based evolutionary algorithms. Genetic Programm Evolvable Mach 15(3):221–244
130. Jin C, Bai R, Zhou Y, Chen X, Tan L (2024) Enhancing online yard crane scheduling through a two-stage rollout memetic genetic programming. Memetic Comput 16(3):467–489. https://doi.org/10.1007/s12293-024-00424-4
131. Jones T (1995) Evolutionary algorithms, fitness landscapes and search. PhD Thesis, University of New Mexico
132. Julstrom BA (1995) Very greedy crossover in a genetic algorithm for the traveling salesman problem. In: Proceedings of the 1995 ACM symposium on applied computing. ACM, New York, pp 324–328
133. Kalaee A, Parsa S, Mansouri Z (2025) Enhancing logic-based testing with EvoDomain: a search-based domain-oriented test suite generation approach. Inf Softw Technol 177:107564. https://doi.org/10.1016/j.infsof.2024.107564
134. Karthika S, Durgadevi M (2024) IMDE-UGAN: improved memetic direction exploitation optimized U-Net generative adversarial network for classification of diabetic retinopathy. IETE J Res 70:6802–6818. https://doi.org/10.1080/03772063.2024.2310111
135. Kernighan B, Lin S (1972) An efficient heuristic procedure for partitioning graphs. Bell Syst J 49:291–307
136. Klau GW (2009) A new graph-based method for pairwise global network alignment. BMC Bioinf 10(Suppl 1):S59
137. Krasnogor N (2004) Self generating metaheuristics in bioinformatics: the proteins structure comparison case. Genetic Programm Evolvable Mach 5(2):181–201
138. Krasnogor N, Gustafson S (2004) A study on the use of "self-generation" in memetic algorithms. Nat Comput 3(1):53–76
139. Krasnogor N, Smith J (2008) Memetic algorithms: the polynomial local search complexity theory perspective. J Math Modell Algor 7(1):3–24
140. Krasnogor N, Blackburne B, Burke E, Hirst J (2002) Multimeme algorithms for protein structure prediction. In: Merelo JJ, et al (eds) Parallel problem solving from nature VII. Lecture notes in computer science, vol 2439. Springer, Berlin, pp 769–778
141. LeCun Y, Bengio Y, Hinton G (2015) Deep learning. Nature 521:436—444
142. Lee D, Kim J, Yoon Y (2024) Improving modularity score of community detection using memetic algorithms. AIMS Math 9(8):20516–20538. https://doi.org/10.3934/math.2024997
143. Lei Z, Hao JK (2024) An effective memetic algorithm for the close-enough traveling salesman problem. Appl Soft Comput 153:111266. https://doi.org/10.1016/j.asoc.2024.111266
144. Li R, Zhang X, Wang C, Cui J, Mu M (2024) A machine learning-based adaptive heuristic for vessel scheduling problem under uncertainty via chance-constrained programming. Comput Electr Eng 119(A):109523. https://doi.org/10.1016/j.compeleceng.2024.109523
145. Li ZY, Zhao PX, Wang CL, Mi YZ (2024) Research on recovery strategies of supply chain network under disruption propagation using memetic algorithm. Adv Prod Eng Manag 19(1):21–30. https://doi.org/10.14743/apem2024.1.490
146. Li L, Du Y, Yao F, Xu S, She Y (2025) Learning memetic algorithm based on variable population and neighborhood for multi-complex target scheduling of large-scale imaging satellites. Swarm Evolut Comput 92:101789. https://doi.org/10.1016/j.swevo.2024.101789
147. Liaw CF (2000) A hybrid genetic algorithm for the open shop scheduling problem. Eur J Oper Res 124:28–42
148. Liefooghe A, Verel S, Hao JK (2014) A hybrid metaheuristic for multiobjective unconstrained binary quadratic programming. Appl Soft Comput 16:10–19

149. Lim KK, Ong YS, Lim MH, Chen X, Agarwal A (2008) Hybrid ant colony algorithms for path planning in sparse graphs. Soft Comput 12(10):981–994
150. Lin S, Kernighan B (1973) An effective heuristic algorithm for the traveling salesman problem. Oper Res 21:498–516
151. Lin G, Zhu W, Ali MM (2014) A tabu search-based memetic algorithm for hardware/software partitioning. Math Probl Eng 2014: Article ID 103059
152. Lin J, Zhang SX, Zheng SY (2024) A diverse/converged individual competition algorithm for computationally expensive many-objective optimization. Appl Intell 54(3):2851–2866. https://doi.org/10.1007/s10489-024-05270-y
153. Lin X, Chen Y, Xue J, Zhang B, Chen Y, Chen C (2024) Parallel machine scheduling with job family, release time, and mold availability constraints: model and two solution approaches. Memetic Comput 16(3):355–371. https://doi.org/10.1007/s12293-024-00421-7
154. Liu C, Chen H, Li X, Zhou S, Jia Z (2024) Drone scheduling in a hexagon-based delivery network with multiple fulfillment centers. Int Trans Oper Res. https://doi.org/10.1111/itor.13476
155. Liu H, Zhao F, Wang L, Xu T, Dong C (2024) Evolutionary multitasking memetic algorithm for distributed hybrid flow-shop scheduling problem with deterioration effect. IEEE Trans Autom Sci Eng 22:1390–1404. https://doi.org/10.1109/Tase.2024.3365518
156. Liu L, Yuan W, Liang Z, Ma X, Zhu Z (2023) Construction of polar codes based on memetic algorithm. IEEE Trans Emerg Top Comput Intell 7(5):1539–1553. https://doi.org/10.1109/Tetci.2023.3234564
157. Lu C, Gao R, Yin L, Zhang B (2024) Human-robot collaborative scheduling in energy-efficient welding shop. IEEE Trans Ind Inf 20(1):963–971. https://doi.org/10.1109/Tii.2023.3271749
158. Lu Y, Benlic U, Wu Q (2024) An effective hybrid evolutionary algorithm for the set orienteering problem. Inf Sci 654:119813. https://doi.org/10.1016/j.ins.2023.119813
159. Luo Q, Deng Q, Guo X, Gong G, Zhao X, Chen L (2023) Modelling and optimization of distributed assembly hybrid flowshop scheduling problem with transportation resource scheduling. Comput Ind Eng 186:109717. https://doi.org/10.1016/j.cie.2023.109717
160. Luo C, Gong W, Ming F, Lu C (2024) A Q-learning memetic algorithm for energy-efficient heterogeneous distributed assembly permutation flowshop scheduling considering priorities. Swarm Evolut Comput 85:101497. https://doi.org/10.1016/j.swevo.2024.101497
161. Luo Q, Deng Q, Gong G, Guo X, Liu X (2024) A distributed flexible job shop scheduling problem considering worker arrangement using an improved memetic algorithm. Expert Syst Appl 239:120161. https://doi.org/10.1016/j.eswa.2023.120161
162. Luo Q, Deng Q, Zhuang H, Gong G, Fan Q, Liu X (2024) Collaborative scheduling of energy-saving spare parts manufacturing and equipment operation strategy using a self-adaptive two-stage memetic algorithm. Rob Comput-Integr Manuf 87:102707. https://doi.org/10.1016/j.rcim.2023.102707
163. Luo Xl, Xiao H, Zhang S (2024) Continuation Newton methods with deflation techniques for global optimization problems. Numer Algor 97:1715–1790. https://doi.org/10.1007/s11075-024-01768-1
164. López-Ibáñez M, Dubois-Lacoste J, Pérez Cáceres L, Birattari M, Stützle T (2016) The irace package: iterated racing for automatic algorithm configuration. Oper Res Perspect 3:43–58
165. Maheswaran R, Ponnambalam SG, Aranvidan C (2005) A meta-heuristic approach to single machine scheduling problems. Int J Adv Manuf Technol 25:772–776
166. Maji S, Maity S, Bsau S, Giri D, Maiti M (2024) Varied offspring memetic algorithm with three parents for a realistic synchronized goods delivery and service problem. Soft Comput 28(5):4235–4265. https://doi.org/10.1007/s00500-023-09574-y
167. Malathy N, Revathi T (2024) Opposition-based improved memetic algorithm for placement of concurrent internet of things applications in fog computing. Trans Emerg Telecommun Technol 35(2):e4941. https://doi.org/10.1002/ett.4941
168. Marino A, Prügel-Bennett A, Glass CA (1999) Improving graph colouring with linear programming and genetic algorithms. In: Proceedings of EUROGEN 99, Jyväskylä, Finland, pp 113–118

169. Mathieson L, de Vries NJ, Moscato P (2019) Using network alignment to identify conserved consumer behaviour modelling constructs. In: Moscato P, de Vries NJ (eds) Business and consumer analytics: new ideas. Springer, Berlin, pp 513–541
170. Mendes A, Cotta C, Garcia V, França P, Moscato P (2005) Gene ordering in microarray data using parallel memetic algorithms. In: Skie T, Yang CS (eds) Proceedings of the 2005 international conference on parallel processing workshops. IEEE Press, Oslo, pp 604–611
171. Merelo JJ, et al (eds) (2002) Parallel problem solving from nature VII. Lecture notes in computer science, vol 2439. Springer, Berlin
172. Merz P (2012) Memetic algorithms and fitness landscapes in combinatorial optimization. In: Neri F, Cotta C, Moscato P (eds) (2012) Handbook of memetic algorithms, studies in computational intelligence, vol 379. Springer, Berlin, pp 95–119
173. Merz P, Zell A (2002) Clustering gene expression profiles with memetic algorithms. In: Merelo JJ, et al (eds) Parallel problem solving from nature VII. Lecture notes in computer science, vol 2439. Springer, Berlin, pp 811–820
174. Milojičić DS, Kalogeraki V, Lukose R, Nagaraja K, Pruyne J, Richard B, Rollins S, Xu Z (2002) Peer-to-peer computing. Technical Report. HPL-2002-57, Hewlett-Packard Labs
175. Molina D, Lozano M, García-Martínez C, Herrera F (2010) Memetic algorithms for continuous optimisation based on local search chains. Evolut Comput 18(1):27–63
176. Molina D, Lozano M, Sánchez AM, Herrera F (2011) Memetic algorithms based on local search chains for large scale continuous optimisation problems: MA-SSW-chains. Soft Comput 15(11):2201–2220
177. Moscato P (1989) On evolution, search, optimization, genetic algorithms and martial arts. towards memetic algorithms. Technical Report. 826, California Institute of Technology, Pasadena, California
178. Moscato P (1989) On evolution, search, optimization, genetic algorithms and martial arts: towards memetic algorithms. Technical Report Caltech Concurrent Computation Program, Report. 826, California Institute of Technology, Pasadena, California
179. Moscato P (1993) An Introduction to population approaches for optimization and hierarchical objective functions: the role of tabu search. Ann Oper Res 41(1–4):85–121
180. Moscato P (1999) Memetic algorithms: a short introduction. In: Corne D, Dorigo M, Glover F (eds) New ideas in optimization. McGraw-Hill, New York, pp 219–234
181. Moscato P (2012) Memetic algorithms: the untold story. In: Neri F, Cotta C, Moscato P (eds) Handbook of memetic algorithms, studies in computational intelligence, vol 379. Springer, Berlin, pp 275–309
182. Moscato P, Cotta C (2003) A gentle introduction to memetic algorithms. In: Glover F, Kochenberger G (eds) Handbook of metaheuristics. Kluwer Academic Publishers, Boston, pp 105–144
183. Moscato P, Norman MG (1992) A memetic approach for the traveling salesman problem implementation of a computational ecology for combinatorial optimization on message-passing systems. In: Valero M, Onate E, Jane M, Larriba JL, Suarez B (eds) Parallel computing and transputer applications. IOS Press, Amsterdam, pp 177–186
184. Moscato P, Tinetti F (1992) Blending heuristics with a population-based approach: a memetic algorithm for the traveling salesman problem. Report 92-12, Universidad Nacional de La Plata
185. Moscato P, Haque MN, Moscato A (2023) Continued fractions and the Thomson problem. Sci Rep 13:7272
186. Mühlenbein H (1989) Parallel genetic algorithms, population genetics and combinatorial optimization. In: Schaffer (ed) 3rd International conference on genetic algorithms, Morgan Kaufmann, pp 416–421
187. Nagata Y, Kobayashi S (1997) Edge assembly crossover: a high-power genetic algorithm for the traveling salesman problem. In: Bäck T (ed) Seventh international conference on genetic algorithms, Morgan Kaufmann, San Mateo, pp 450–457
188. Neri F (2012) Diversity management in memetic algorithms. In: Neri F, Cotta C, Moscato P (eds) Handbook of memetic algorithms, studies in computational intelligence, vol 379. Springer, Berlin, pp 153–165

189. Neri F, Cotta C (2012) Memetic algorithms and memetic computing optimization: a literature review. Swarm Evolut Comput 2:1–14
190. Neri F, Mininno E (2010) Memetic compact differential evolution for Cartesian robot control. IEEE Comput Intell Mag 5(2):54–65
191. Neri F, Cotta C, Moscato P (eds) (2012) Handbook of memetic algorithms, studies in computational intelligence, vol 379. Springer, Berlin
192. Nogueras R, Cotta C (2015) Studying fault-tolerance in island-based evolutionary and multimemetic algorithms. J Grid Comput 13:351–374. https://doi.org/10.1007/s10723-014-9315-6
193. Norman M, Moscato P (1989) A competitive and cooperative approach to complex combinatorial search. Tech. Rep. Caltech Concurrent Computation Program, Report. 790, California Institute of Technology, Pasadena, California, USA, expanded version published at the 20th informatics and operations research meeting, Buenos Aires (20th JAIIO), Aug. 1991, pp. 3.15–3.29
194. Montes de Oca MA, Cotta C, Neri F (2012) Local search. In: Neri F, Cotta C, Moscato P (eds) Handbook of memetic algorithms, studies in computational intelligence, vol 379. Springer, Berlin, pp 29–41
195. Oliveira DF, Martins MS, Sousa JM, Vieira SM, Figueira JR (2025) Divide-and-conquer initialization and mutation operators for the large-scale mixed capacitated arc routing problem. Eur J Oper Res 321(2):383–396. https://doi.org/10.1016/j.ejor.2024.09.043
196. Ong YS, Keane A (2004) Meta-Lamarckian learning in memetic algorithms. IEEE Trans Evolut Comput 8(2):99–110
197. Ong YS, Lim MH, Zhu N, Wong KW (2006) Classification of adaptive memetic algorithms: a comparative study. IEEE Trans Syst Man Cyber Part B: Cyber 36(1):141–152
198. Ong YS, Lim MH, Chen X (2010) Memetic computation-past, present and future. IEEE Comput Intell Mag 5(2):24–31
199. Ou Z, Wang S (2024) Finding robust and influential nodes on directed networks using a memetic algorithm. Swarm Evolut Comput 87:101542. https://doi.org/10.1016/j.swevo.2024.101542
200. Palubeckis G (2025) A fast local search based memetic algorithm for the parallel row ordering problem. Appl Math Comput 486:129040. https://doi.org/10.1016/j.amc.2024.129040
201. Pang X, Wang Y, Yu Y, Liu W (2024) Optimal scheduling of a cogeneration system via Q-learning-based memetic algorithm considering demand-side response. Energy 300:131513. https://doi.org/10.1016/j.energy.2024.131513
202. Poikolainen I, Neri F (2013) Differential evolution with concurrent fitness based local search. In: Proceedings of the 2013 IEEE congress on evolutionary computation. IEEE Press, Cancun, pp 384–391
203. Porumbel D, Coelho IM, Talbi EG (2024) Using an exact bi-objective decoder in a memetic algorithm for ARC-routing (and other decoder-expressible) problems. Eur J Oper Res 313(1):25–43. https://doi.org/10.1016/j.ejor.2023.07.021
204. Qian J, Zhang C, Wu Z, Ding H, Li L (2024) Efficient topology reconfiguration for NoC-based multiprocessors: a greedy-memetic algorithm. J Parall Distrib Comput 190:104904. https://doi.org/10.1016/j.jpdc.2024.104904
205. Radcliffe N (1994) The algebra of genetic algorithms. Ann Math Artif Intell 10:339–384
206. Radcliffe NJ, Surry PD (1994) Formal memetic algorithms. In: Fogarty TC (ed) AISB Workshop on evolutionary computing. Lecture notes in computer science, vol 865. Springer, Berlin, pp 1–16
207. Raja MAZ, Ahmad SuI, Samar R (2014) Solution of the 2-dimensional Bratu problem using neural network, swarm intelligence and sequential quadratic programming. Neural Comput Appl 25(7–8):1723–1739
208. Ranjkesh S, Masoumi B, Hashemi SM (2024) A novel robust memetic algorithm for dynamic community structures detection in complex networks. World Wide Web-Int Web Inf Syst 27(1). https://doi.org/10.1007/s11280-024-01238-7

209. Reddy KG, Mishra D (2024) Enhancing feature selection in high-dimensional data with fuzzy fitness-integrated memetic algorithms. IEEE Access 12:130675–130692, https://doi.org/10.1109/Access.2024.3459390
210. Richter H, Engelbrecht A (2014) Recent advances in the theory and application of fitness landscapes, emergence, complexity and computation, vol 6. Springer, Berlin
211. Rizqi ZU, Chou SY (2024) Neuroevolution reinforcement learning for multi-echelon inventory optimization with delivery options and uncertain discount. Eng Appl Artif Intell 134:108670. https://doi.org/10.1016/j.engappai.2024.108670
212. Rodriguez-Esparza E, Morales-Castaneda B, Casas-Ordaz A, Oliva D, Navarro MA, Valdivia A, Houssein EH (2024) Handling the balance of operators in evolutionary algorithms through a weighted hill climbing approach. Knowl-Based Syst 294:111784. https://doi.org/10.1016/j.knosys.2024.111784
213. Rodríguez Rueda D, Cotta C, Fernández Leiva AJ (2011) A memetic algorithm for designing balanced incomplete blocks. Int J Comb Optim Probl Inf 2(1):14–22
214. Rothlauf F, Goldberg DE (2002) Representations for genetic and evolutionary algorithms. Physica-Verlag, Heidelberg
215. Sarmenta LF (1998) Bayanihan: web-based volunteer computing using java. In: Masunaga Y, Katayama T, Tsukamoto M (eds) Worldwide computing and its applications – WWCA'98. Lecture notes in computer science, vol 1368. Springer, Berlin, pp 444–461
216. Segura-Ortiz A, Garcia-Nieto J, Aldana-Montes JF (2024) Exploiting medical-expert knowledge via a novel memetic algorithm for the inference of gene regulatory networks. In: Franco L, DeMulatier C, Paszynski M, Krzhizhanovskaya V, Dongarra J, Sloot P (eds) Computational science. Lecture notes in computer science, vol 14835, pp 3–17. https://doi.org/10.1007/978-3-031-63772-8_1
217. Sehgal S, Ganesh A, Kamboj VK, Malik OP (2024) A memetic approach to multi-disciplinary design and numerical optimization problems using intensify slime mould optimizer. Appl Intell 54(2):2031–2083. https://doi.org/10.1007/s10489-023-05073-7
218. Seo W, Park J, Lee S, Moon AS, Kim DW, Lee J (2024) Memetic multilabel feature selection using pruned refinement process. J Big Data 11(1). https://doi.org/10.1186/s40537-024-00961-2
219. Sevaux M, Dauzère-Pérès S (2003) Genetic algorithms to minimize the weighted number of late jobs on a single machine. Eur J Oper Res 151:296–306
220. Smith JE (2007) Coevolving memetic algorithms: A review and progress report. IEEE Trans Syst Man Cyber Part B Cyber 37(1):6–17
221. Smith JE (2010) Meme fitness and memepool sizes in coevolutionary memetic algorithms. In: 2010 IEEE congress on evolutionary computation. IEEE Press, Barcelona, pp 1–8
222. Smith JE (2012) Self-adaptative and coevolving memetic algorithms. In: Neri F, Cotta C, Moscato P (eds) Handbook of memetic algorithms, studies in computational intelligence, vol 379. Springer, Berlin, pp 167–188
223. Snoussi C, El Fallahi A, Hicham S (2023) A memetic algorithm to solve the two-echelon collaborative multi-centre multi-periodic vehicle routing problem with specific constraints. Int J Adv Comput Sci Appl 14(12):951–960
224. Somar S, Urazel B, Sahin YB (2023) A modified metaheuristic algorithm for a home health care routing problem with health team skill levels. Appl Soft Comput 148:110912. https://doi.org/10.1016/j.asoc.2023.110912
225. Song X, Zhang Y, Zhang W, He C, Hu Y, Wang J, Gong D (2024) Evolutionary computation for feature selection in classification: a comprehensive survey of solutions, applications and challenges. Swarm Evolut Comput 90:101661. https://doi.org/10.1016/j.swevo.2024.101661
226. Song Y, Suganthan PN, Pedrycz W, Yan R, Fan D, Zhang Y (2024) Energy-efficient satellite range scheduling using a reinforcement learning-based memetic algorithm. IEEE Trans Aerospace Electr Syst 60(4):4073–4087. https://doi.org/10.1109/Taes.2024.3371964
227. Sörensen K, Sevaux M (2006) MA | PM: memetic algorithms with population management. ComputOR 33:1214–1225

228. Stützle T, López-Ibáñez M, Pérez-Cáceres L (2022) Automated algorithm configuration and design. In: Proceedings of the genetic and evolutionary computation conference companion, Association for Computing Machinery, New York, GECCO'22, pp 997–1019
229. Sudholt D (2009) The impact of parametrization in memetic evolutionary algorithms. Theoret Comput Sci 410(26):2511–2528
230. Sudholt D (2012) Parametrization and balancing local and global search. In: Neri F, Cotta C, Moscato P (eds) Handbook of memetic algorithms, studies in computational intelligence, vol 379. Springer, Berlin, pp 55–72
231. Sun Y, Liu Z, Jin Y, Sun X, Cao Y, Yang J (2024) Global and cluster structural balance via a priority strategy based memetic algorithm. IEEE Trans Emerg Top Comput Intell 9:874–888. https://doi.org/10.1109/Tetci.2024.3419723
232. Tanese R (1989) Distributed genetic algorithms. In: 3rd international conference on genetic algorithms. Morgan Kaufmann, San Francisco, pp 434–439
233. Tang J, Li Y (2024) A hybrid dynamic memetic algorithm for the influence maximization problem in social networks. Int J Modern Phys C 36. https://doi.org/10.1142/S0129183124501924
234. Tang J, Gong G, Peng N, Zhu K, Huang D, Luo Q (2024) An effective memetic algorithm for distributed flexible job shop scheduling problem considering integrated sequencing flexibility. Expert Syst Appl 242:122734. https://doi.org/10.1016/j.eswa.2023.122734
235. Teng Y, Li X, Gao L, Xie J, Li Y (2024) An improved co-evolutionary memetic algorithm based on novel schedule type and unconditional feasibility for hybrid flow-shop scheduling problem. Comput Ind Eng 193:110324. https://doi.org/10.1016/j.cie.2024.110324
236. Thangavelu A, Rajendran P (2024) Energy-efficient secure routing for a sustainable heterogeneous IoT network management. Sustainability 16(11):4756. https://doi.org/10.3390/su16114756
237. Tirronen V, Neri F, Kärkkäinen T, Majava K, Rossi T (2008) An enhanced memetic differential evolution in filter design for defect detection in paper production. Evolut Comput 16(4):529–555
238. Tomassini M (2005) Spatially structured evolutionary algorithms. Natural computing series. Springer, Berlin
239. Torba R, Dauzere-Peres S, Yugma C, Gallais C, Pouzet J (2024) Solving a real-life multi-skill resource-constrained multi-project scheduling problem. Ann Oper Res 338(1, Si):69–114. https://doi.org/10.1007/s10479-023-05784-7
240. Urselmann M, Engell S (2015) Design of memetic algorithms for the efficient optimization of chemical process synthesis problems with structural restrictions. Comput Chem Eng 72:87–108
241. Vieira MVC (2024) A linear ordering problem with weighted rank. J Comb Optim 47(2). https://doi.org/10.1007/s10878-024-01109-x
242. Wan X, Zuo X, Lu T, Chen G (2024) A hybrid niching memetic algorithm for multi-modal optimization of double row layout problem. Int J Prod Res. https://doi.org/10.1080/00207543.2024.2410467
243. Wang G, Wang P, Zhang H (2024) A self-adaptive memetic algorithm for distributed job shop scheduling problem. Mathematics 12(5):683. https://doi.org/10.3390/math12050683
244. Wang J, Han H (2025) A learning-based memetic algorithm for energy-efficient distributed flow-shop scheduling with preventive maintenance. Swarm Evolut Comput 92:101772. https://doi.org/10.1016/j.swevo.2024.101772
245. Wang L, Shen P (2024) Memetic segmentation based on variable lag aware for multivariate time series. Inf Sci 657:120003. https://doi.org/10.1016/j.ins.2023.120003
246. Wang L, Zheng DZ (2002) A modified genetic algorithm for job-shop scheduling. Int J Adv Manuf Technol 20:72–76
247. Wang Y, Hao JK, Glover F, Lu Z (2014) A tabu search based memetic algorithm for the maximum diversity problem. Eng Appl Artif Intell 27:103–114

248. Wang S, Feng Y, Li X, Zhang G (2024) Memetic discrete differential evolution with domain knowledge for blocking scheduling distributed flow shop with lot-streaming constraints. Expert Syst Appl 255(D):124874. https://doi.org/10.1016/j.eswa.2024.124874
249. Wang S, Li X, Gao L, Li J (2024) A multi-disjunctive-graph model-based memetic algorithm for the distributed job shop scheduling problem. Adv Eng Inf 60:102401. https://doi.org/10.1016/j.aei.2024.102401
250. Wang Y, Lu Z, Ding J, Su Z, Marti R (2024) A weighted vertex cover-based intensification tabu search for the capacitated dispersion problem. IEEE Trans Emerg Top Comput Intell 8:4225–4236. https://doi.org/10.1109/TETCI.2024.3389768
251. Wang Y, Han Y, Wang Y, Wang X, Liu Y, Gao K (2025) Reinforcement learning-assisted memetic algorithm for sustainability-oriented multiobjective distributed flow shop group scheduling. IEEE Trans Syst Man Cyber Syst 55:2399–2413. https://doi.org/10.1109/TSMC.2024.3518625
252. Wei S, Tang H, Li X, Lei D, Wang XV (2024) An improved memetic algorithm for multi-objective resource-constrained flexible job shop inverse scheduling problem: an application for machining workshop. J Manuf Syst 74:264–290. https://doi.org/10.1016/j.jmsy.2024.03.005
253. Whitley LD (1991) Fundamental principles of deception in genetic search. In: Rawlins G (ed) Foundations of genetic algorithms I. Morgan Kaufmann, San Mateo, pp 221–241
254. Wolpert D, Macready W (1997) No free lunch theorems for optimization. IEEE Trans Evolut Comput 1(1):67–82
255. Wu L, Wang Z, Liao Z, Xiao D, Han P, Li W, Chen Q (2024) Multi-day tourism recommendations for urban tourists considering hotel selection: a heuristic optimization approach. Omega-Int J Manag Sci 126:103048. https://doi.org/10.1016/j.omega.2024.103048
256. Wu Q, He M, Hao JK, Lu Y (2024) An effective hybrid evolutionary algorithm for the clustered orienteering problem. Eur J Oper Res 313(2):418–434. https://doi.org/10.1016/j.ejor.2023.08.006
257. Wu R, He J, Li X, Chen Z (2024) A memetic algorithm with fuzzy-based population control for the joint order batching and picker routing problem. Inf Sci 656:119913. https://doi.org/10.1016/j.ins.2023.119913
258. Xu Y, Jiang X, Li J, Xing L, Song Y (2024) A knowledge-driven memetic algorithm for the energy-efficient distributed homogeneous flow shop scheduling problem. Swarm Evolut Comput 89:101625. https://doi.org/10.1016/j.swevo.2024.101625
259. Xu Z, Xu J, Zhang K, Xu X, He J, Wu N (2024) Decision variable classification based multi-objective multifactorial memetic algorithm for multi-objective multi-task optimization problem. Appl Soft Comput 152:111232. https://doi.org/10.1016/j.asoc.2024.111232
260. Yan X, Zuo H, Hu C, Gong W, Gao L (2024) Distributed heterogeneous flow shop scheduling method for dual-carbon goals. IEEE Trans Autom Sci Eng 22:7409–7420. https://doi.org/10.1109/Tase.2024.3371940
261. Yao Y, Li X, Gao L (2024) A DQN-based memetic algorithm for energy-efficient job shop scheduling problem with integrated limited AGVs. Swarm Evolut Comput 87:101544. https://doi.org/10.1016/j.swevo.2024.101544
262. Yu F, Lu C, Yin L, Zhang B (2024) A self-learning memetic algorithm for human-robot collaboration scheduling in energy-efficient distributed mixed fuzzy welding shop. IEEE Trans Autom Sci Eng 22:6595–6607. https://doi.org/10.1109/Tase.2024.3448435
263. Yu F, Lu C, Yin L, Zhou J (2024) Modeling and optimization algorithm for energy-efficient distributed assembly hybrid flowshop scheduling problem considering worker resources. J Ind Inf Integr 40:100620. https://doi.org/10.1016/j.jii.2024.100620
264. Yu Y, Pan QK, Pang X, Tang X (2024) An attribution feature-based memetic algorithm for hybrid flowshop scheduling problem with operation skipping. IEEE Trans Autom Sci Eng 22:99–114. https://doi.org/10.1109/Tase.2023.3346446

265. Yuksel D, Kandiller L, Tasgetiren MF (2024) Q-learning guided algorithms for bi-criteria minimization of total flow time and makespan in no-wait permutation flowshops. Swarm Evolut Comput 89:101617. https://doi.org/10.1016/j.swevo.2024.101617
266. Zhai R, Mei Y, Guo T, Du W (2024) A collaborative drone-truck delivery system with memetic computing optimization. IEEE Trans Syst Man Cyber-Syst 54(6):3618–3630. https://doi.org/10.1109/Tsmc.2024.3371471
267. Zhang F, Li R, Gong W (2024) Deep reinforcement learning-based memetic algorithm for energy-aware flexible job shop scheduling with multi-AGV. Comput Ind Eng 189:109917. https://doi.org/10.1016/j.cie.2024.109917
268. Zhang G, Liu B, Wang L, Xing K (2024) Distributed heterogeneous co-evolutionary algorithm for scheduling a multistage fine-manufacturing system with setup constraints. IEEE Trans Cyber 54(3):1497–1510. https://doi.org/10.1109/Tcyb.2022.3217074
269. Zhang J, Deng Q, Luo Q, Wang Z, Zhuang H, Huang Y (2024) A double-layer Q-learning driven memetic algorithm for integrated scheduling of procurement, production and maintenance with distributed resources. Appl Soft Comput 165:112110. https://doi.org/10.1016/j.asoc.2024.112110
270. Zhang X, Li R, Wang C, Xue B, Guo W (2024) Robust optimization for a class of ship traffic scheduling problem with uncertain arrival and departure times. Eng Appl Artif Intell 133(C):108257. https://doi.org/10.1016/j.engappai.2024.108257
271. Zhao Y (2024) Optimal configuration of additional heat source for CHP system considering demand response based on comprehensive benefits. Int Trans Electr Energy Syst 2024. https://doi.org/10.1155/2024/5541642
272. Zhao F, Tang J, Wang J, Jonrinaldi (2014) An improved particle swarm optimization with decline disturbance index (DDPSO) for multi-objective job-shop scheduling problem. Comput Oper Res 45:38–50
273. Zhao J, Poon M, Tan VYF, Zhang Z (2024) A hybrid genetic search and dynamic programming-based split algorithm for the multi-trip time-dependent vehicle routing problem. Eur J Oper Res 317(3):921–935. https://doi.org/10.1016/j.ejor.2024.04.011
274. Zheng CZ, Sang HY, Xing LN, Zou WQ, Meng LL, Meng T (2024) A self-adaptive memetic algorithm with Q-learning for solving the multi-AGVs dispatching problem. Swarm Evolut Comput 90:101697. https://doi.org/10.1016/j.swevo.2024.101697
275. Zheng Y, Peng N, Qi H, Gong G, Huang D, Zhu K, Liu J, Liu G (2024) An improved memetic algorithm for distributed hybrid flow shop scheduling problem with operation inspection and reprocessing. Measure Control. https://doi.org/10.1177/00202940241245241
276. Zheng YJ, Xie XC, Zhang ZY, Shi JT (2024) Deep reinforcement learning assisted memetic scheduling of drones for railway catenary deicing. Swarm Evolut Comput 91:101719. https://doi.org/10.1016/j.swevo.2024.101719
277. Zhou G, Li D, Bian J, Zhang Y (2024) Two-echelon time-dependent vehicle routing problem with simultaneous pickup and delivery and satellite synchronization. Comput Oper Res 167:106600. https://doi.org/10.1016/j.cor.2024.106600
278. Zhou X, Wang F, Wu B, Li Y, Shen N (2025) Deep reinforcement learning-based memetic algorithm for solving dynamic distributed green flexible job shop scheduling problem with finite transportation resources. Swarm Evolut Comput 94:101885. https://doi.org/10.1016/j.swevo.2025.101885
279. Zhou Y, Hao JK, Li Z, Glover F (2024) Heuristic search for rank aggregation with application to label ranking. Inf J Comput 36(2). https://doi.org/10.1287/ijoc.2022.0019
280. Zhou Y, Kong L, Yan L, Liu Y, Wang H (2024) A memetic algorithm for a real-world dynamic pickup and delivery problem. Memetic Comput 16(2):203–217. https://doi.org/10.1007/s12293-024-00407-5
281. Zhu C, Lin Y, Zheng F, Lin J, Zhong Y (2024) An adaptive multi-meme memetic algorithm for the prize-collecting generalized minimum spanning tree problem. Swarm Evolut Comput 90:101664. https://doi.org/10.1016/j.swevo.2024.101664

282. Zhu Y, Tang Q, Cheng L, Zhao L, Jiang G, Lu Y (2024) Solving multi-objective hybrid flowshop lot-streaming scheduling with consistent and limited sub-lots via a knowledge-based memetic algorithm. J Manuf Syst 73:106–125. https://doi.org/10.1016/j.jmsy.2024.01.006
283. Zhuang H, Deng Q, Luo Q, Zhao Y, Zhang J (2024) Modelling and optimization for integrated scheduling problem considering spare parts production, batch transportation and equipment operation. Expert Syst Appl 252(B):124194. https://doi.org/10.1016/j.eswa.2024.124194
284. Zou Y, Hao JK, Wu Q (2024) A reinforcement learning guided hybrid evolutionary algorithm for the latency location routing problem. Comput Oper Res 170:106758. https://doi.org/10.1016/j.cor.2024.106758

Particle Swarm Optimization

28

Dimitra G. Triantali and Konstantinos E. Parsopoulos

Contents

Introduction . . . 852
Basic Model . . . 854
Convergence and Parameter Setting . . . 857
Early Precursors . . . 858
Velocity Clamping and Inertia Weight . . . 859
Stability Analysis . . . 860
Concept of Neighborhood . . . 863
Initialization, Stopping Conditions, and Boundary Violation . . . 865
Performance-Enhancing Techniques . . . 867
Enhanced and Specialized PSO Variants . . . 874
Binary PSO . . . 874
Guaranteed Convergence PSO . . . 874
Bare Bones PSO . . . 875
Fully Informed PSO . . . 876
Quantum PSO . . . 876
Unified PSO . . . 877
Cooperative PSO . . . 879
Comprehensive Learning PSO . . . 880
TRIBES . . . 881
Niching PSO . . . 882
Standard PSO . . . 883
Hybrid PSO . . . 884
Memetic PSO . . . 885
Opposition-Based PSO . . . 886
PSO in Noisy Environments . . . 887
Multiobjective PSO . . . 887

D. G. Triantali · K. E. Parsopoulos (✉)
Department of Computer Science and Engineering, University of Ioannina, Ioannina, Greece
e-mail: d.triantali@uoi.gr; kostasp@uoi.gr

© Springer Nature Switzerland AG 2025

R. Martí et al. (eds.), *Handbook of Heuristics*,
https://doi.org/10.1007/978-3-032-00385-0_22

Applications and Further Resources 889
Conclusions 889
Cross-References 890
References 891

Abstract

Particle swarm optimization has captivated the scientific community for three decades. Its effectiveness and efficiency have made it a significant metaheuristic approach in various scientific fields dealing with complex optimization problems. Its simplicity makes it accessible to nonexpert researchers, while its flexible operators and the ease of integrating new procedures allow its application to a wide range of problems with diverse characteristics. Additionally, its inherent decentralized nature facilitates easy parallelization, enabling it to leverage modern high-performance computing systems, effectively. The present work introduces the basic concepts of particle swarm optimization and presents several popular variants that have opened new research directions by incorporating novel ideas into the original algorithm. The focus is on conveying the essential information of these algorithms rather than delving into all the details. A comprehensive list of references and sources is provided for further inquiry, making this text a valuable starting point for researchers interested in the development and application of particle swarm optimization and its variants.

Keywords

Particle swarm optimization · Swarm intelligence · Metaheuristics · Nature-inspired algorithms · Stochastic search · Optimization · Computational intelligence

Introduction

Particle swarm optimization (PSO) was introduced in the pioneering works of Russell C. Eberhart and James Kennedy in 1995 [44, 80]. At that time, the widespread success of evolutionary algorithms (EAs) motivated researchers worldwide to develop and experiment with novel nature-inspired methods. Beyond the interest in genetic algorithm procedures that dominated EAs, new paradigms from nature became subjects of investigation. In this context, the hierarchically organized societies of simple organisms such as ants, bees, and fishes—known for their limited range of individual responses but fascinating collective behaviors—immediately attracted scientific interest.

Simulations of such organism populations exhibited traits of intelligent behavior and remarkable problem-solving capabilities [17]. The underlying mathematical models shared concepts with particle physics, enriched with elements from probability theory and stochastic processes. This theoretical background, along with

the notable potential for developing powerful optimization metaheuristics, led to the emergence of a new category of algorithms known as *swarm intelligence* (SI) [17, 45, 46].

PSO holds a prominent position among SI algorithms. It was inspired by simulators of social behavior that implemented rules such as neighbor velocity matching and acceleration by distance. These properties were sufficient to produce swarming behavior in groups of simple agents. The initial models were refined to fit the framework of optimization algorithms. The first PSO models introduced the innovation of using difference vectors among population members to sample new points in the search space. This approach diverged from the established procedures of EAs, which mostly relied on sampling new points from explicit probability distributions [163]. Nevertheless, it proved influential, as another algorithm with a similar structure, differential evolution [174], appeared in the literature almost concurrently with PSO. Additional advantages of PSO include the ease of adapting its operators and procedures to match the specific requirements of a problem, as well as its inherent decentralized structure that promotes parallelization.

PSO has rapidly gained wide recognition due to its effectiveness, efficiency, and easy implementation. This is evident from Fig. 1, which shows the number of scientific works including the term "particle swarm" in their titles from 1995 to 2024, as retrieved by the Scopus search engine in May 2024. The numbers encompass all sources indexed by Scopus, including journals, conferences, books, and other publications.

The present chapter aims to introduce the basic concepts of PSO and present several of its most influential variants, selected based on their novelty at the

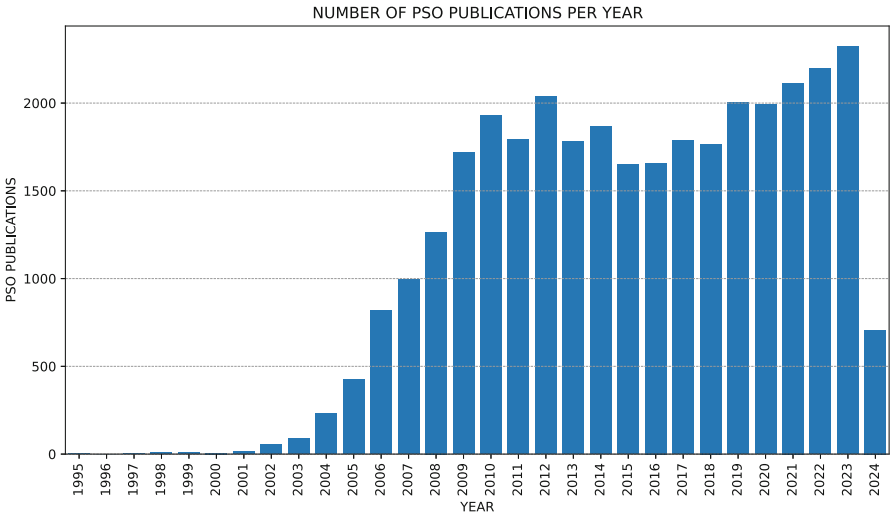


Fig. 1 Number of publications with the term "particle swarm" in the title. (Source: Scopus search engine, May 2024)

time of development, popularity, and potential to inspire new research directions. Naturally, the selection of variants cannot be exhaustive, nor can the presentation be comprehensive within the limited scope of a book chapter. Therefore, only basic information is provided here, while further details can be found in the cited references and sources. Thus, the present text is intended to serve as an "Ariadne's thread" for researchers interested in PSO.

The rest of the chapter is organized as follows: Section "Basic Model" introduces a basic PSO model that serves as a reference point for the variants discussed. Section "Convergence and Parameter Setting" outlines the theoretical properties of PSO, including convergence, parameter settings, special features, and performance-enhancing techniques. Section "Enhanced and Specialized PSO Variants" presents several essential PSO variants. Section "Applications and Further Resources" briefly comments on PSO applications. Finally, the chapter concludes in section "Conclusions." An appendix at the end of the text offers additional sources for further inquiry and experimentation with PSO.

Basic Model

PSO was primarily developed for optimization in real-valued search spaces. For this reason, the general d-dimensional bound-constrained optimization problem:

$$\min_{\vec{x} \in \mathcal{X}} f(\vec{x}), \tag{1}$$

where:

$$\mathcal{X} = \left[x_1^{\min}, x_1^{\max}\right] \times \cdots \times \left[x_d^{\min}, x_d^{\max}\right] \subset \mathbb{R}^d, \tag{2}$$

is henceforth considered as the reference problem for the presentation of the algorithm. In case of different search spaces, explicit definitions will be given. The only necessary assumption regarding the objective function is the availability of its value $f(\vec{x})$ for every point $\vec{x} \in \mathcal{X}$. Smoothness properties such as continuity and differentiability are not required.

The main search mechanism of PSO is based on a group of *search agents*, called *particles*, which iteratively move in the search space $\mathcal{X}$. The particles retain in an external memory the best positions they have ever visited in $\mathcal{X}$. Each particle's move is stochastically biased towards its own findings as well as promising regions of the search space discovered by the rest of the particles. This implicit information exchange scheme is responsible for the emergent convergence properties of the whole group, which is accordingly called a *swarm*.

Putting it formally, let A_i denote the ith particle (search agent) and let:

$$\mathcal{S} = \{A_1, A_2, \ldots, A_N\} \tag{3}$$

be a swarm of size N (the ordering of the particles is irrelevant). Also, let $I = \{1, 2, \ldots, N\}$ be the set of all particle indices, $D = \{1, 2, \ldots, d\}$ be the set of coordinate directions' indices, and t denote the algorithm's iteration number. Then, each particle can be defined by four essential elements:

$$A_i^{(t)} = \left\langle \vec{x}_i^{(t)}, \vec{v}_i^{(t)}, \vec{p}_i^{(t)}, NB_i^{(t)} \right\rangle, \qquad i \in I.$$

The first vector:

$$\vec{x}_i^{(t)} = \left(x_{i1}^{(t)}, x_{i2}^{(t)}, \ldots, x_{id}^{(t)} \right)^T \in \mathcal{X},$$

defines the *current position* of the particle in $\mathcal{X}$ at iteration t (with T denoting the transpose). Frequently, the term "particle" is used to define solely the current position $\vec{x}_i$. However, in our opinion, the use of this term to define the search agent with all its components promotes a more compact presentation, while it is aligned with the notation in similar SI algorithms (e.g., the term "ant" is similarly used to define search agents in ant colony optimization [17]).

The second vector:

$$\vec{v}_i^{(t)} = \left(v_{i1}^{(t)}, v_{i2}^{(t)}, \ldots, v_{id}^{(t)} \right)^T,$$

is an adaptable position shift, also called *velocity*, which is responsible for the particle's move. The third vector:

$$\vec{p}_i^{(t)} = \left(p_{i1}^{(t)}, p_{i2}^{(t)}, \ldots, p_{id}^{(t)} \right)^T \in \mathcal{X},$$

is the particle's *best position*, i.e., the best position it has ever visited in $\mathcal{X}$ up to iteration t. For the reference optimization problem of Eq. (1), the best position corresponds to the particle's previous position with the smallest objective value, i.e.:

$$\vec{p}_i^{(t)} = \arg \min_{\left\{ \vec{x}_i^{(k)},\ k \leqslant t \right\}} f\left(\vec{x}_i^{(k)} \right).$$

The best position plays the role of an attractor that biases the particle's velocity towards the promising regions it has discovered in $\mathcal{X}$. However, this sole information would render each particle an isolated search agent that produces trajectories solely by oscillating around its best visited points.

For this reason, in addition to its own discoveries, each particle has an implicit information-exchange mechanism with the members of a subset of the swarm, called its *neighborhood*. The neighborhood can be simply defined as a subset of the indices set I:

$$NB_i^{(t)} \subseteq I,$$

and its cardinality is usually called the *neighborhood size*. In most PSO variants, the best position in the neighborhood of the particle, i.e.:

$$\vec{p}_{g_i}^{(t)} = \arg \min_{\left\{\vec{p}_j^{(t)},\ j \in NB_i^{(t)}\right\}} f\left(\vec{p}_j^{(t)}\right),$$

is used as the second attractor for biasing the particle's search direction.

Based on the descriptions above, the basic update scheme for the particle current positions and velocities is componentwisely defined as follows:

$$v_{ij}^{(t+1)} = \chi\, v_{ij}^{(t)} + \mathcal{C}_1\left(p_{ij}^{(t)} - x_{ij}^{(t)}\right) + \mathcal{C}_2\left(p_{g_i j}^{(t)} - x_{ij}^{(t)}\right), \tag{4}$$

$$x_{ij}^{(t+1)} = x_{ij}^{(t)} + v_{ij}^{(t+1)}, \tag{5}$$

where $i \in I$ and $j \in D$. The scalar parameter χ is called the *inertia weight* or *constriction coefficient* and its role is discussed in section "Convergence and Parameter Setting." The stochastic terms $\mathcal{C}_1$ and $\mathcal{C}_2$ follow the uniform distribution:

$$\mathcal{C}_1 \sim \mathcal{U}(0, c_1), \qquad \mathcal{C}_2 \sim \mathcal{U}(0, c_2), \tag{6}$$

where c_1 and c_2 are predefined parameters, called the *acceleration constants*, which control the magnitude of attraction towards $\vec{p}_i^{(t)}$ and $\vec{p}_{g_i}^{(t)}$, respectively. The first difference term $\left(p_{ij}^{(t)} - x_{ij}^{(t)}\right)$ is called the *cognitive term* because it involves only the particle's own information. Correspondingly, the second difference term $\left(p_{g_i j}^{(t)} - x_{ij}^{(t)}\right)$ is called the *social term* since it involves information provided by other particles.

After updating all current positions, the best positions of the particles are also updated as follows:

$$\vec{p}_i^{(t+1)} = \begin{cases} \vec{x}_i^{(t+1)}, & \text{if } f\left(\vec{x}_i^{(t+1)}\right) \leqslant f\left(\vec{p}_i^{(t)}\right), \\ \vec{p}_i^{(t)}, & \text{otherwise,} \end{cases} \qquad i \in I. \tag{7}$$

The algorithm is executed until a predefined stopping condition is fulfilled. The presented basic PSO model is also known as the *canonical particle swarm* [79] and it is summarized in the pseudocode of Fig. 2. The discussion on initialization, search space boundary violation, and stopping conditions is postponed until the next section.

A number of issues arise regarding essential choices that need to be made prior to the application of the basic PSO algorithm. Specifically, the practitioner shall address the following questions:

1. How shall the parameters N, χ, c_1, c_2 be set?
2. How shall the neighborhoods be defined?

```
begin CanonicalPSO(𝒳, N, d)
  while stopping criterion not satisfied do
    for i = 1, ..., N do
      for j = 1, ..., d do
        Update v_ij by using Eq. (4)
        Check for velocity boundary violation and correct if needed
        Update x_ij using Eq. (5)
        Check for search space boundary violation and correct if needed
      end-for;
    end-for;
    for i = 1, ..., N do
      Update p⃗_i according to Eq. (7)
    end-for;
  end-while;
  return best solution;
end CanonicalPSO
```

Fig. 2 Pseudocode of the canonical PSO

3. How shall the boundary violations be handled?
4. How shall the swarm be initialized?
5. What stopping conditions shall be used?

Apparently, each decision can have significant impact on the algorithm's performance. For this reason, careful settings that take into consideration the potential impact on PSO dynamic are required.

A number of PSO variants have been developed on the basis of using alternative techniques for configuring the algorithm. Further discussion on these issues is provided in the next section. Canonical PSO has constituted the starting point for further developments and enhancements for more than two decades. Yet, it still remains a popular piece of the state-of-the-art due to its verified effectiveness and efficiency in numerous applications [140, 147, 164].

Convergence and Parameter Setting

Over the past decades, a large number of PSO variants have been proposed. In most cases, the motivation for further research stemmed from the necessity for more efficient variants that address the weaknesses of earlier versions. At the core of these enhancements are the answers to the fundamental questions posed in section "Basic Model" concerning the configuration and parameter settings of PSO.

In the following paragraphs, the most significant developments in PSO are highlighted, along with insights and suggestions for its configuration.

Early Precursors

Early precursors of PSO [44, 80] were not as sophisticated as the canonical PSO defined in section "Basic Model." In fact, they were mostly based on a simplified version of Eq. (4) with $\chi = 1$ and $NB_i = I$, i.e.:

$$v_{ij}^{(t+1)} = v_{ij}^{(t)} + \mathcal{C}_1 \left(p_{ij}^{(t)} - x_{ij}^{(t)} \right) + \mathcal{C}_2 \left(p_{gj}^{(t)} - x_{ij}^{(t)} \right), \tag{8}$$

$$x_{ij}^{(t+1)} = x_{ij}^{(t)} + v_{ij}^{(t+1)}, \tag{9}$$

with $i \in I$, $j \in D$, and $\vec{p}_g^{(t)}$ the best position (also called the *overall best*) discovered by any particle up to iteration t:

$$\vec{p}_g^{(t)} = \arg \min_{\left\{ \vec{p}_k^{(t)},\, k \in I \right\}} f\left(\vec{p}_k^{(t)} \right). \tag{10}$$

Obviously, the magnitude of the particles' position shifts depends on the values of the parameters c_1 and c_2 that determine the stochastic terms $\mathcal{C}_1$ and $\mathcal{C}_2$ in Eq. (6).

In [76] the trajectories of the particles were studied by further simplifying the system. The use of the previous velocity $\vec{v}_i^{(t)}$ in the update equation of the current velocity results in an oscillatory move of the particle around the weighted average of the two best positions:

$$\vec{\bar{p}}_i^{(t)} = \frac{1}{\mathcal{C}_1 + \mathcal{C}_2} \left(\mathcal{C}_1 \vec{p}_i^{(t)} + \mathcal{C}_2 \vec{p}_g^{(t)} \right). \tag{11}$$

The overall best particle, for which it holds that $\vec{p}_i^{(t)} = \vec{p}_g^{(t)}$, updates its velocity as follows:

$$v_{ij}^{(t+1)} = v_{ij}^{(t)} + \mathcal{C} \left(p_j^{(t)} - x_{ij}^{(t)} \right), \tag{12}$$

where $j \in D$, $\mathcal{C} = \mathcal{C}_1 + \mathcal{C}_2$, and $\vec{p}^{(t)} = \vec{p}_g^{(t)}$.

Considering the trivial case of a single particle and a constant best position $\vec{p}$, it was shown [76] that the particle's trajectory becomes highly dependent on the values of $\mathcal{C}$, as shown in Fig. 3. Values higher than 4.0 are detrimental for the algorithm's convergence, since the velocity can grow arbitrarily large (notice the scaling difference in the vertical axis for $\mathcal{C} = 4.0$ in Fig. 3). This problem was called the *swarm explosion effect* and it was verified also in [121, 122], emphasizing the necessity for a control mechanism to amplify the velocities.

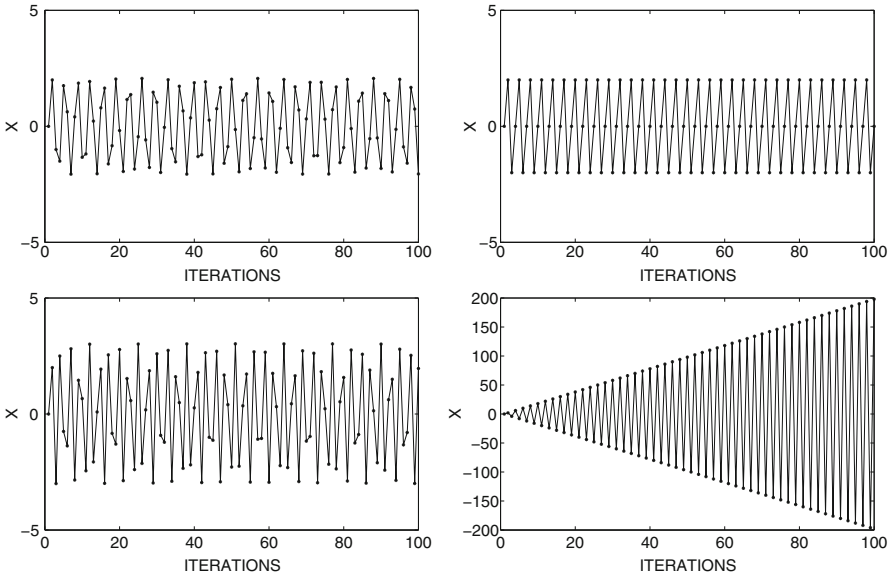


Fig. 3 Particle trajectory for fixed value $\mathcal{C} = 2.5$ (upper left), 3.0 (upper right), 3.5 (lower left), and 4.0 (lower right)

Velocity Clamping and Inertia Weight

The swarm explosion effect in early PSO variants led to the first significant improvements i.e., *velocity clamping* and the introduction of the *inertia weight*. Since the main problem was the arbitrary grow of the velocities, a straightforward solution was their explicit restriction in acceptable values. Specifically, a maximum value $v_j^{\max}$ was imposed on the absolute value of each velocity component:

$$-v_j^{\max} \leqslant v_{ij} \leqslant v_j^{\max}, \quad \text{for all } i \in I, j \in D.$$

If a velocity component v_{ij} escapes from the prescribed interval, it is set equal to the violated boundary value. This way, the particles are prohibited from taking large steps that may frequently send them out of the search space. The value $v_j^{\max}$ can be arbitrarily defined by the user or be set as a fraction of the search space along the jth coordinate direction:

$$v_j^{\max} = \lambda_j \left(x_j^{\max} - x_j^{\min} \right), \qquad j \in D, \ \lambda_j \in (0, 1). \tag{13}$$

Naturally, prior information regarding the number and density of the minimizers of the objective function facilitates the proper setting of the maximum velocity. For example, in cases of extremely large number of minimizers or very narrow regions of attraction around them, smaller velocities can offer better search accuracy [170]. Equation (13) is used also in modern PSO variants, usually assuming identical λ_j for all $j \in D$. Velocity clamping is applied in line 5 of the pseudocode of canonical PSO in Fig. 2.

Although velocity clamping has proved to be effective in hindering divergence, it can hardly produce convergent behavior of the particles. Obviously, convergence can be achieved by gradually diminishing the velocities. This way, the particles would perform decreasing oscillations around the attractors and, eventually, settle on a point. This was achieved by introducing an inertia weight w on the velocity update of Eq. (8) [168], as follows:

$$v_{ij}^{(t+1)} = w v_{ij}^{(t)} + \mathcal{C}_1 \left(p_{ij}^{(t)} - x_{ij}^{(t)} \right) + \mathcal{C}_2 \left(p_{gj}^{(t)} - x_{ij}^{(t)} \right), \tag{14}$$

$$x_{ij}^{(t+1)} = x_{ij}^{(t)} + v_{ij}^{(t+1)}. \tag{15}$$

The parameter w shall be properly set such that the impact of the previous velocity $\vec{v}_i^{(t)}$ declines during the execution of the algorithm. This can be achieved by setting either a fixed value $w \in (0, 1)$ or assuming a decreasing scheme between two extreme values $w_{\max}$ and $w_{\min}$, during the algorithm's run. For example, if $t_{\max}$ is a predefined maximum number of iterations, a linearly decreasing inertia weight may take values as follows:

$$w(t) = w_{\max} - \frac{t}{t_{\max}} \left(w_{\max} - w_{\min} \right).$$

Figure 4 illustrates swarm diversity in terms of the average standard deviations per coordinate direction of the particles, without inertia weight as well as with a linearly decreasing inertia weight, for a well-known optimization problem.

Velocity clamping and the introduction of inertia weight boosted PSO research due to the improved convergence properties of the derived variants. Both these improvements are still used in modern PSO approaches [140]. Further information and enhancements can be found in relevant works [21, 99, 156, 183, 207].

Stability Analysis

The empirical analysis of particle trajectories in [76, 121, 122] and their implications in PSO modeling and parameter setting motivated the systematic theoretical investigation of the algorithm. The first breakthrough appeared in 2002 due to M. Clerc and J. Kennedy who conducted a convergence and stability analysis of the algorithm in multidimensional search spaces [29].

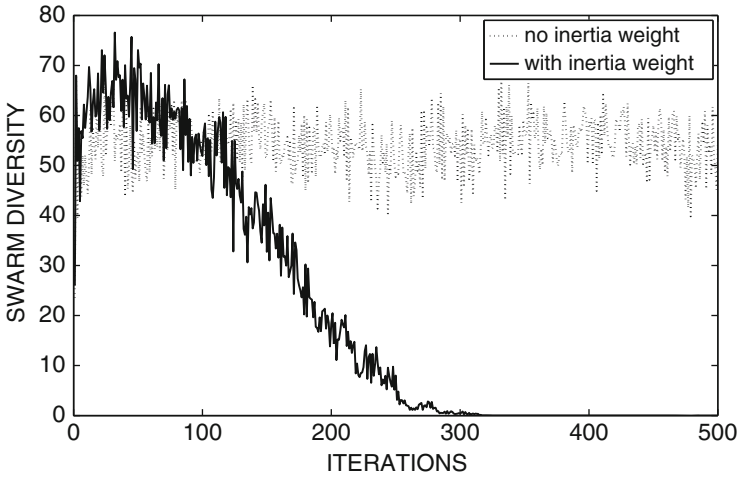


Fig. 4 Swarm diversity with (solid line) and without (dotted line) inertia weight while minimizing the 2-dimensional Rosenbrock function using 20 particles with $c_1 = c_2 = 2$, $v^{\max} = 50$, $w_{\max} = 1.2$, and $w_{\min} = 0.1$

The authors used as starting point the early model of Eqs. (8) and (9) with velocity clamping. The initial model was manipulated according to Eq. (12) and further simplified by assuming 1-dimensional particles without stochasticity, i.e., the two stochastic acceleration terms were assumed to be equal and fixed, $\mathcal{C}_1 = \mathcal{C}_2 = \mathcal{C}$. After dropping some indices and making proper algebraic manipulations, the 1-dimensional update rules can be rewritten in the form [29]:

$$v^{(t+1)} = v^{(t)} + \mathcal{C} y^{(t)}, \tag{16}$$

$$y^{(t+1)} = -v^{(t)} + (1 - \mathcal{C}) y^{(t)}, \tag{17}$$

where $y^{(t)} = \bar{p} - x^{(t)}$, with $\bar{p}$ being the (currently fixed) aggregate best position defined in Eq. (11) (note that all vectors currently collapse to scalars). This discrete system can be written also in matrix form:

$$P_{t+1} = M\, P_t = M^t\, P_0,$$

where:

$$P_t = \begin{pmatrix} v^{(t)} \\ y^{(t)} \end{pmatrix}, \qquad M = \begin{pmatrix} 1 & \mathcal{C} \\ -1 & 1 - \mathcal{C} \end{pmatrix}.$$

Then, the behavior of the system depends on the eigenvalues of M, which are given as:

$$\lambda_{1,2} = 1 - \frac{\mathcal{C}}{2} \pm \frac{\sqrt{\mathcal{C}^2 - 4\mathcal{C}}}{2}.$$

The previously identified critical value $\mathcal{C} = 4$ (recall Fig. 3) [76, 122] appears again as the limit between the case of two different real eigenvalues, one eigenvalue of multiplicity 2, and two complex conjugate eigenvalues $\lambda_{1,2}$.

For the case of $\mathcal{C} \in (0, 4)$, the eigenvalues become complex [29]:

$$\lambda_1^t = \cos(t\theta) + i\ \sin(t\theta), \qquad \lambda_2^t = \cos(t\theta) - i\ \sin(t\theta),$$

and the system exhibits cyclic behavior for $\theta = (2\kappa\pi)/t$. On the other hand, $\mathcal{C} > 4$ does not produce cyclic behavior. Instead, it is proved that P_t has monotonically increasing distance from the origin [29]. Finally, the limit case $\mathcal{C} = 4$ produces (*i*) oscillatory behavior $P_{t+1} = -P_t$ if P_0 is an eigenvector of M or (*ii*) linearly increasing or decreasing $\|P_t\|$ for $y_0 > 0$ or $y_0 < 0$, respectively.

The investigation was further extended to the continuous case by transforming the simplified model into the following recurrent equation [29]:

$$\vec{v}^{(t+2)} + (\mathcal{C} - 2)\vec{v}^{(t+1)} + \vec{v}^{(t)} = 0,$$

which in turn becomes a second-order differential equation:

$$\frac{\partial^2 \vec{v}}{\partial t^2} + \ln(\lambda_1\lambda_2)\frac{\partial \vec{v}}{\partial t} + \ln(\lambda_1)\ln(\lambda_2)\vec{v} = 0,$$

where λ_1 and λ_2 are the solutions of:

$$\lambda^2 + (\mathcal{C} - 2)\lambda + 1 = 0.$$

Thus, the quantities $\vec{v}^{(t)}$ and $\vec{y}^{(t)}$ take the following general form [29]:

$$\vec{v}^{(t)} = c_1\lambda_1^t + c_2\lambda_2^t,$$
$$\vec{y}^{(t)} = \left(c_1\lambda_1^t(\lambda_1 - 1) + c_2\lambda_2^t(\lambda_2 - 1)\right)/\mathcal{C}.$$

The parameters c_1 and c_2 depend on the initial vectors $\vec{v}^{(0)}$ and $\vec{y}^{(0)}$. Following a similar analysis as for the discrete case shows that the system explosion depends on whether the condition $\max\{|\lambda_1|, |\lambda_2|\} > 1$ holds or not [29].

Subsequently to the analysis above, the employed simplified model was generalized in order to approximate the actual model of PSO. A number of extended models were accordingly developed and studied in [29]. The outcome of the study was an effective model, namely, the canonical PSO model of Eqs. (4) and (5), accompanied with a proposed parameter setting [29]:

$$\chi = 0.729, \quad c_1 = c_2 = 1.49, \tag{18}$$

derived from closed-form formulae that retain the convergence behavior of the system. This parameter setting has become the most common choice in off-the-shelf PSO approaches in numerous applications [140].

The theoretical analysis of PSO in [29] was succeeded by new theoretical studies such as [67, 73, 148, 161, 190, 195]. These developments increased our understanding of the algorithm's dynamic. Also, they added merit that, along with its easy implementation and the reported excellent results in various applications, eventually placed PSO in a salient position among other established population-based optimization algorithms such as genetic algorithms and evolution strategies. Moreover, the motion dynamic of the canonical PSO model motivated approaches that spare function evaluations by taking advantage of the continuous particle movement [196].

Concept of Neighborhood

Information exchange among the particles is a concept of major importance in PSO. In the canonical PSO of Eqs. (4) and (5), the ith particle adopts the best position $\vec{p}_{g_i}$ of its neighborhood NB_i as an attractor for its move, besides its own best position. This way, the neighborhood structure determines communication channels among the particles and, consequently, the information flow in the swarm. Without communication, collective behavior cannot emerge and the swarm is downgraded to a group of isolated search agents with limited search capabilities.

Obviously, the characteristics of the neighborhood play a major role in PSO dynamics [77, 175]. The neighborhood structure defines the communication channels among the particles, while neighborhood size controls the influence of the swarm on each particle. Since the particles are attracted towards the best positions of their neighbors, it is easily comprehended that neighborhoods with a large number of particles and dense communication channels are more inclined towards intensification of search around the best detected positions in the search space. This is ascribed to the rapid diffusion of the discovered good solutions to the particles and their consequent attraction towards them.

However, this also renders the particles prone to get stuck in deep local minima that are possibly detected in early iterations of the algorithm's run. Such minima in the swarm are typically updated less frequently than shallow minima. Hence, they can impose stronger attraction on the particles. On the other hand, less crowded neighborhoods with sparse connections among the particles exhibit slower information diffusion in the swarm. In this case, the particles are gradually acquainted with good solutions, retaining higher diversity and exploration capability.

The discussion above suggests that a proper neighborhood configuration shall be carefully selected, as it is highly responsible for the diversification versus intensification (also known as exploration vs. exploitation) trade-off of the algorithm. To this end, prior knowledge on the studied problem can be valuable. For example, smooth functions with one or few minima may be properly handled through intensification-oriented neighborhoods, while rugged landscapes with a plethora of minimizers,

possibly dispersed at distant parts of the search space, may require exploration-oriented approaches.

In early PSO variants, each particle was assumed to be connected with all the rest, i.e.:

$$NB_i^{(t)} = I, \quad \text{for all } i \in I,$$

and the selected best position from the neighborhood was the swarm's overall best defined in Eq. (10). Also, the neighborhoods were time-invariant, i.e., they remained unchanged throughout the algorithm's run (hence we can neglect t in $NB_i^{(t)}$ notation). This PSO model was also called the *global PSO model* or simply the *gbest model*. The interconnection among the particles can be elegantly represented by undirected graphs where nodes denote the particles and edges denote communication channels. The corresponding structure is called the *neighborhood topology*. Apparently, the gbest model corresponds to a fully connected graph since all particles communicate with each other. This topology is illustrated in the left part of Fig. 5.

The tendency of the gbest model to rapidly converge towards the most promising detected solutions and easily get stuck in local minima led to further experimentation with sparsely connected models. The outcome of these efforts was the *local PSO model* or *lbest model*, which assumes neighborhoods that are proper subsets of I:

$$NB_i \subset I, \quad \text{for all } i \in I.$$

The most effective and easily implemented topology of this form is the *ring*, which is depicted in the right part of Fig. 5. In this scheme, the particles are assumed to lie on a ring according to their indices, i.e., neighboring particles have neighboring indices. Then, each particle is set to communicate only with its immediate neighbors on the ring. The number of neighbors for each direction (front and back) is called *neighborhood radius*. Thus, a neighborhood of radius $r < N$ of the ith particle is defined as the set:

$$NB_i = \{i - r, i - r + 1, \ldots, i - 1, i, i + 1, \ldots, i + r - 1, i + r\},$$

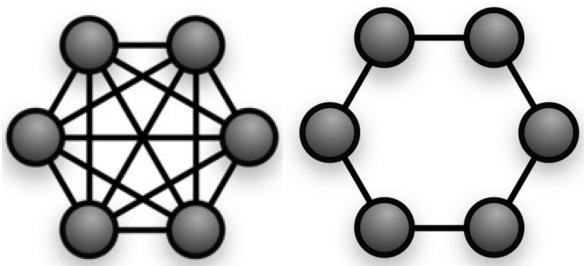


Fig. 5 Graphical representation of the fully connected (left) and the ring (right) neighborhood topology

where the indices are assumed to recycle after index N, i.e.:

$$j = \begin{cases} j, & \text{if } 1 \leqslant j \leqslant N, \\ j \bmod N, & \text{if } j > N, \\ N - |j|, & \text{if } j < 1, \end{cases} \qquad \text{for all } j \in NB_i.$$

Differences in diversity between the gbest model and the lbest with ring topology of radius $k = 1$ is indicatively depicted in Fig. 6 for a single run on a well-known optimization test problem.

The ring topology has become a standard for lbest PSO implementations due to its successful application in various problems [140]. There is also a multitude of alternative topologies that have drawn attention, such as random [175], hierarchical [63], and dynamic [24], but with limited number of applications. Also, the concept of neighborhood has offered motivation for the development of new PSO variants with sophisticated computational budget allocation based on the information carried by the neighborhoods rather than the particles [173].

Initialization, Stopping Conditions, and Boundary Violation

A basic requirement in the design and development of stochastic optimization algorithms such as PSO is their robustness under random initialization in the search space. In practice, this implies that mild perturbations on random initial positions of the particles shall correspond to similar performance profiles of the algorithm.

In benchmarking studies, it is commonly assumed that there is no information available regarding the problem at hand. This is usually called the *black-box optimization problem*. In such cases, there is no reason for purposely biasing particle initialization in specific areas of the search space. Consequently, random and uniform initialization of the particles is the typical procedure followed in most studies, i.e.:

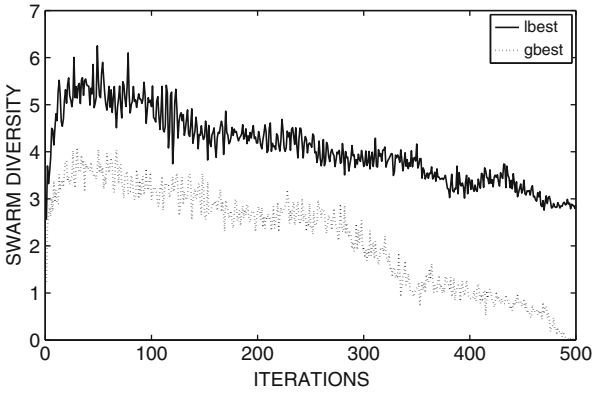


Fig. 6 Swarm diversity for the lbest (solid line) and gbest (dotted line) PSO model on the 10-dimensional Rastrigin function using 20 particles

$$x_{ij}^{(0)} = p_{ij}^{(0)} \sim \mathcal{U}\left(x_j^{\min}, x_j^{\max}\right), \qquad \text{for all } i \in I, j \in D.$$

Accordingly, the velocities are randomly initialized as follows:

$$v_{ij}^{(0)} \sim \mathcal{U}\left(-v_j^{\max}, v_j^{\max}\right), \qquad \text{for all } i \in I, j \in D.$$

Naturally, if there is prior information regarding promising regions of the search space, the distributions can be properly adapted to favor the sampling of particles in those regions.

The effect of initialization has been studied in a number of works for different PSO variants [33, 40, 49, 61, 134, 157, 181, 210], offering further insight and suggestions for specialized initialization in specific applications. Nevertheless, random initialization remains the standard approach also due to (*i*) its minor implementation effort and (*ii*) the availability of uniform random number generators in almost all programming languages and hardware platforms.

In contrast to initialization, which is based on problem characteristics, termination conditions are rather user-dependent and resource-dependent [140]. The most common termination criteria that can be found in the relevant literature are as follows:

1. Convergence in search space
2. Convergence in function values
3. Limitations in computational budget
4. Search stagnation

The first two stopping criteria entail information on the position of the global minimizer or its objective function value, respectively. Naturally, this information is not always available. However, in some cases relevant bounds can be estimated for the solution or its objective function value. Then, the user can set the algorithm to stop as soon as it reaches those bounds with a prespecified tolerance. The underlying stopping condition can have the following form:

$$\texttt{IF} \quad \left(\left\|\vec{p}_g^{(t)} - \vec{x^*}\right\| \leqslant \varepsilon_x \quad \texttt{OR} \quad \left\|f_g^{(t)} - f^*\right\| \leqslant \varepsilon_f\right) \texttt{ THEN STOP}$$

where $\vec{x}^*$ and f^* are the targeted solution vector and function value, respectively, and ε_x, ε_f, are the corresponding user-defined tolerances.

The last two stopping criteria in the above list are more common than the first two in PSO literature. Computational budget limitations are typically imposed by the maximum available time that can be spent for solving a problem. This quantity can be explicitly expressed either as wall-clock/CPU time or as the number of function evaluations performed by the algorithm. For benchmarking purposes, the latter is preferable since even the same algorithm executed on the same computer

at different time instances may result in different running times. This is due to irrelevant procedures that may be concurrently running on the same computer.

On the other hand, the search stagnation criterion can prematurely stop the algorithm even if the computational budget is not exceeded. Successful application of this criterion is based on the existence of a proper stagnation measure. Typically, the number of subsequent iterations without improvement of the best solution and/or the dispersion of the particles' current (or best) positions in the search space are used for this purpose.

The aforementioned criteria can be combined in joint stopping rules as follows:

$$\texttt{IF} \quad (t \geqslant t_{\max} \quad \texttt{OR} \quad t_{\text{fail}} \geqslant t_{\max\text{fail}} \quad \texttt{OR} \quad dv(t) < dv_{\min}) \ \texttt{THEN STOP}$$

where t stands for the iteration number, t_{fail} is the number of subsequent non-improving iterations, and $dv(t)$ is a measure of particle dispersion (for example, the average standard deviation per coordinate component). Overall, the user shall pay special attention to the selection of stopping criteria in order to avoid unintentional premature termination of the algorithm. Further developments on this issue can be found in [85].

Another topic of interest lies in handling boundary violations. More specifically, after the particle update of Eqs. (4) and (5), some components of the new position may violate the corresponding boundaries of the search space. The most popular approach to restrict the particle in the search space is to set the violated components equal to the value of the violated boundary:

$$x_{ij}^{(t)} = \begin{cases} x_j^{\max}, & \text{if } x_{ij}^{(t)} > x_j^{\max}, \\ x_j^{\min}, & \text{if } x_{ij}^{(t)} < x_j^{\min}, \\ x_{ij}^{(t)}, & \text{otherwise}, \end{cases} \tag{19}$$

while simultaneously setting the corresponding velocity component $v_{ij}^{(t)}$ equal to zero. Different boundary handling techniques such as absorbing, bouncing, and cyclic search spaces are also used although less frequently. A survey on such techniques can be found in [123].

Performance-Enhancing Techniques

The canonical PSO has offered satisfactory performance in various problems. However, it may exhibit declining performance in special problems such as the detection of multiple local/global minima, constrained, or discrete problems. In such cases, the user can either change the algorithm by introducing new, specialized operators or incorporate external techniques to tackle the problem's peculiarities.

There are established techniques that have been successfully used with canonical PSO. Transformations of the objective function have been used for the alleviation of local minima and the detection of multiple minimizers. Rounding has been used

for solving discrete optimization problems, while penalty functions can address constrained optimization problems. In the following paragraphs, basic techniques that have been combined with the canonical PSO model are presented. Specialized variants of the algorithm with ad hoc operators for similar purposes are presented in subsequent sections.

Alleviating Local Minimizers

In problems with a multitude of local and/or global minimizers, stochastic optimization algorithms such as PSO may approximate a different minimizer in each independent run. Frequently, the detected solutions are sub-optimal, while the user either needs more than one such solution or requires the globally best one. *Multistart techniques* where the algorithm is subsequently restarted from different initial conditions have been proposed for such cases and discussed in classical optimization texts [189]. However, these approaches cannot guarantee that an already detected minimizer will be avoided after restarting the algorithm.

An alternative approach consists of transforming the objective function into a new one that excludes the already detected solutions. Well-known examples of this type are the *filled functions* [51]. Similar techniques have been developed and successfully used with PSO [129, 134]. The *stretching* technique consists of a two-stage transformation of the objective function:

$$F_1(\vec{x}) = f(\vec{x}) + \gamma_1 \|\vec{x} - \vec{x}^*\| \left[1 + \text{sign}\left(f(\vec{x}) - f(\vec{x}^*)\right)\right], \tag{20}$$

$$F_2(\vec{x}) = F_1(\vec{x}) + \gamma_2 \frac{1 + \text{sign}\left(f(\vec{x}) - f(\vec{x}^*)\right)}{\tanh\left(\mu\left(F_1(\vec{x}) - F_1(\vec{x}^*)\right)\right)}, \tag{21}$$

where $f(\vec{x})$ is the original objective function, $\vec{x}^*$ is the best detected solution of the algorithm so far, and the sign function is defined as follows:

$$\text{sign}(\vec{z}) = \begin{cases} -1, & \text{if } \vec{z} < 0, \\ 0, & \text{if } \vec{z} = 0, \\ +1, & \text{if } \vec{z} > 0. \end{cases}$$

As soon as a local minimizer (or generally a sub-optimal solution) is detected, the first transformation stretches the objective function upwards, while the second one transforms the detected solution $\vec{x}^*$ into a local maximizer. Under proper parameter setting, stretching has the ability to remove higher local minima than the detected solution $\vec{x}^*$ while leaving unchanged all lower minima as well as the global one. This is illustrated in Fig. 7 for a 1-dimensional instance of a well-known test function. Thus, it can be very useful in problems with a multitude of local minima that can mislead the algorithm after its re-initialization. Of course, if a better solution is found in a subsequent application of the algorithm, it can simply replace $\vec{x}^*$ in Eqs. (20) and (21).

It shall be noted that the application of stretching on a global minimizer eliminates all other minimizers. For this reason, stretching is inappropriate for

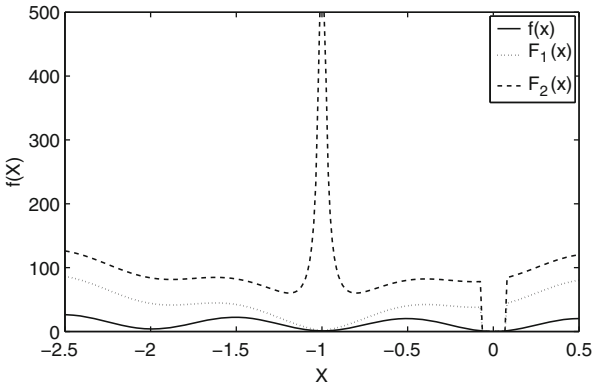


Fig. 7 The stretching technique applied on the 1-dimensional Rastrigin function for the local minimizer $\vec{x}^* = -1$ and parameters $\gamma_1 = \gamma_2 = 20$, $\mu = 0.1$

detecting multiple global minimizers (see next section for a relevant technique that tackles this problem). Also, similarly to its filled function predecessors, stretching may introduce new local minima around the point of application $\vec{x}^*$ [135]. This is also known as the *Mexican hat effect* and has been addressed in [204] through proper parameterization. Naturally, the application of stretching is not limited to the canonical PSO, as it can be incorporated to any PSO variant, even to different algorithms [57].

Detecting Multiple Minimizers

Deflection [106] is a technique that works similarly to stretching but it has only local effect on the objective function. Thus, it can be applied in cases where multiple solutions of the problem are needed. Deflection has been used with PSO for detecting many global and/or local minimizers with promising results [135].

Let $f(\vec{x})$ be the objective function and $\vec{x}_1^*, \ldots, \vec{x}_k^*$, be k previously detected solutions. Then, the objective function is transformed as follows:

$$F(\vec{x}) = \frac{f(\vec{x})}{\prod_{i=1}^{k} T_i\left(\vec{x}, \vec{x}_i^*, \lambda_i\right)}, \tag{22}$$

where T_i is defined as [135]:

$$T_i\left(\vec{x}, \vec{x}_i^*, \lambda_i\right) = \tanh\left(\lambda_i \|\vec{x} - \vec{x}_i^*\|\right), \tag{23}$$

and λ_i are relaxation parameters, $i = 1, 2, \ldots, k$. The idea behind this transformation is similar to stretching, i.e., the transformation of detected solutions into local maximizers. However, deflection changes the objective function only locally, around the point of application. The magnitude of change depends on the parameters λ_i that need proper tuning. Also, deflection requires strictly positive objective functions in order to achieve the desirable effect. In cases where the problem at hand is not

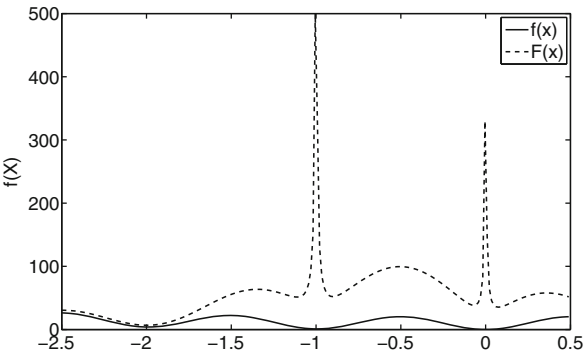


Fig. 8 The deflection technique applied on the 1-dimensional Rastrigin function for the local minimizer $\vec{x}_1^* = -1$ as well as the global minimizer $\vec{x}_g^* = 0$ and parameters $\lambda_1 = \lambda_g = 1$

strictly positive, it can be simply shifted to positive values by using $f(\vec{x}) + c$ instead of $f(\vec{x})$, with a sufficiently large bias $c > 0$.

Figure 8 illustrates deflection for the 1-dimensional instance of the Rastrigin test function. The Mexican hat effect appears also in deflection, although its impact can be controlled with proper selection of the parameters λ_i. Moreover, deflection can be combined with a *repulsion* technique that prevents the particles from visiting the neighborhoods of detected solutions and possibly get trapped in the artificially introduced local minima [135, 140]. Combined with PSO, deflection offered previously undetected solutions in computationally challenging problems [170].

Penalty Functions for Constrained Optimization Problems

In constrained optimization problems, the objective function is accompanied by a set of constraints that need to be satisfied at the final solution. In general, such a problem is defined as:

$$\min_{\vec{x} \in \mathcal{X}} f(\vec{x}) \quad \text{subject to} \quad C_i(\vec{x}) \leqslant 0, \quad i = 1, 2, \ldots, k, \tag{24}$$

where $C_i(\vec{x})$ are inequality constraints. Different constraint types can be transformed in the above form as follows:

$$C_i(\vec{x}) \geqslant 0 \;\Leftrightarrow\; -C_i(\vec{x}) \leqslant 0,$$
$$C_i(\vec{x}) = 0 \;\Leftrightarrow\; C_i(\vec{x}) \geqslant 0 \;\text{ and }\; C_i(\vec{x}) \leqslant 0.$$

Penalty functions have been widely used in constrained problems. The main goal is to penalize all solutions that lie in the infeasible region, such that the algorithm will be implicitly directed into the feasible search space.

The general form of a penalty function for the problem of Eq. (24) is defined as:

$$F(\vec{x}) = f(\vec{x}) + P(\vec{x}),$$

where:

$$P(\vec{x}) = \begin{cases} \alpha > 0, & \text{if } C_i(\vec{x}) > 0 \text{ for at least one } i, \\ 0, & \text{otherwise.} \end{cases}$$

In the simplest case, the penalty term $P(\vec{x})$ can be a constant for all infeasible solutions. This is not always a suitable choice since it neglects the magnitude of constraint violation. Also, it does not provide any information to the algorithm regarding the distance of the infeasible solutions from the feasible region. Thus, the penalty term is recommended to take into consideration the number of violated constraints as well as the degree of violation. Moreover, the magnitude of penalties can be time-varying, starting with mild penalties in early iterations and becoming strict in later ones.

The canonical PSO has been combined with a penalty function defined as follows [132, 212]:

$$F(\vec{x}) = f(\vec{x}) + h(t)H(\vec{x}), \tag{25}$$

where:

$$H(\vec{x}) = \sum_{i=1}^{k} \theta\left(q_i(\vec{x})\right) q_i(\vec{x})^{\gamma(q_i(\vec{x}))}, \tag{26}$$

and:

$$q_i(\vec{x}) = \max\left\{0, C_i(\vec{x})\right\}, \qquad i = 1, 2, \ldots, k.$$

The weight $h(t)$ controls the impact of the penalty term $H(\vec{x})$ with respect to iteration t. The degree of violation is accounted in $q_i(\vec{x})$ and manipulated with the power function $\gamma(q_i(\vec{x}))$ as well as with the multistage assignment function $\theta\left(q_i(\vec{x})\right)$. An alternative penalty function was proposed in [31]:

$$F(\vec{x}) = f(\vec{x}) + H(\vec{x}),$$

with:

$$H(\vec{x}) = w_1 H_{\text{NVC}}(\vec{x}) + w_2 H_{\text{SVC}}(\vec{x}),$$

where $H_{\text{NVC}}(\vec{x})$ is the number of violated constraints and:

$$H_{\text{SVC}}(\vec{x}) = \sum_{i=1}^{k} \max\left\{0, C_i(\vec{x})\right\},$$

is the sum of violations. The weights w_1 and w_2 can be either fixed or time-varying and they determine the importance of each penalty term.

The canonical PSO updates its best positions according to Eq. (7) by comparing objective values, solely. However, it is commonly desirable to allow only feasible best positions in order to avoid oscillations of the particles in the infeasible space. On top of that, the final solution shall be feasible and, thus, the particles shall eventually concentrate their search efforts in the feasible region.

In the penalty functions presented above, it is still possible that an infeasible solution has lower function value than a feasible one and, therefore, be preferable for inclusion in the best positions. In order to prevent such undesirable inclusions, a set of rules can be imposed in the selection procedure:

1. Between feasible solutions, the one with the smallest objective value is preferable.
2. Between a feasible and an infeasible solution, the feasible one is always preferable.
3. Between two infeasible solutions, the one with the smallest penalty term $H(\vec{x})$ is preferable.

These selection rules combined with the penalty function approach have been used with PSO in constrained optimization problems with promising results [115, 146].

Solving Discrete Problems

The canonical PSO was initially introduced as a continuous optimization method and its operators are designed to work on real-valued search spaces. In Mathematical Programming literature, integer optimization problems can also be solved through continuous algorithms by extending the discrete problem to a continuous one and rounding the solutions to the nearest integers. Such approaches have been used with branch and bound algorithms combined with quadratic programming solvers [87, 108].

A similar approach has been used also with PSO for solving integer problems [86]. Specifically, the particles are let to assume real vectors in their current positions, although they are rounded to the nearest integer vector when evaluated with the objective function. Also, the rounded integer vectors are preferable as best positions, since the final solution shall be integer. Putting it formally, if $\vec{x}_i = (x_{i1}, x_{i2}, \ldots, x_{id})^T$ is the ith particle, then its objective value is $f(\vec{x}_i) = f(\vec{z}_i)$, where the integer vector $\vec{z}_i = (z_{i1}, z_{i2}, \ldots, z_{id})^T$ is defined as:

$$z_{ij} = \lfloor x_{ij} + 0.5 \rfloor, \qquad i \in I, \ j \in D.$$

Rounding has effectively worked for various problems of integer and mixed integer type [86, 115, 126, 146]. Note that, in mixed integer problems, no additional representation effort is needed to distinguish between real and discrete coordinate

components. The algorithm can work by simply rounding the coordinate components that correspond to integer variables.

An issue that shall be taken into consideration when rounding is that the velocities may become rather small, resulting in search stagnation due to the inability of producing different integer components. This potential deficiency can be tackled by restarting the current positions of the particles (and possibly the best positions except the overall best) as soon as stagnation is identified. Alternatively, gradual truncation of the particles in less decimal digits can be applied as the number of iterations increases [86].

Another interesting category of problems with discrete search spaces comprises of *permutation problems*. In such problems, the main goal is the detection of an optimal permutation of fixed elements (numbers, items, indices), and they are frequently met in Combinatorics and Operations Research [16]. Such problems can be tackled through real-valued algorithms by using the *smallest position value* (SPV) representation scheme [184]. Specifically, let an ordered set:

$$\mathcal{Z} = \{Z_1, \ldots, Z_d\}$$

consisting of elements $Z_i, i \in D$. Then, the components of a real-valued particle are defined as numerical weights that denote the priority of the corresponding elements according to a predefined mapping. For example, the components of the ith particle $\vec{x}_i = (x_{i1}, x_{i2}, \ldots, x_{id})^T$ can be mapped as follows:

$$\begin{array}{lcccc} \text{Ordered list of elements:} & Z_1 & Z_2 & \cdots & Z_d \\ & \updownarrow & \updownarrow & & \updownarrow \\ \text{Particle components (weights):} & x_{i1} & x_{i2} & \cdots & x_{id} \end{array}$$

Then, the weights (particle components) are sorted in ascending order and the corresponding permutation is received by rearranging the elements, accordingly:

$$\begin{array}{rcccc} \text{Sorted particle components (weights):} & x_{ik_1} & x_{ik_2} & \cdots & x_{ik_d} \\ & \updownarrow & \updownarrow & & \updownarrow \\ \text{Corresponding permutation:} & Z_{k_1} & Z_{k_2} & \cdots & Z_{k_d} \end{array}$$

Thus, each particle corresponds to a permutation received after sorting its components. The goal of PSO is to find the appropriate weights that produce optimal or near-optimal permutations. A commonly used search space for the particles in such problems is $\mathcal{X} = [0, 1]^d$, i.e., the weights are all assumed to lie in the interval $[0, 1]$. Obviously, there is an infinite number of weight vectors that correspond to each permutation, since the produced permutation depends only on the relative ordering of the weights rather than their actual decimal values. SPV has been successfully combined with PSO in various permutation problems [83, 137, 185, 186].

Enhanced and Specialized PSO Variants

This section is devoted to PSO variants that stemmed from the canonical PSO as improvements or specialized modifications for specific problem types. The pluralism in literature renders a complete presentation impossible. For this reason, a selection was made based on the novelty introduced by each method as well as its influence for further developments. Chronological order of the methods' appearance was retained whenever possible.

Binary PSO

The first *binary PSO* (BPSO) variant was developed in 1997 [81]. Instead of using rounding for transforming the real-valued parameters into binary ones, the algorithm introduced a new interpretation of the velocities. More specifically, each component of the velocity vector is considered as the input of a sigmoid function that determines the state (0 or 1) of the corresponding coordinate component of the particle. Thus, the velocity $\vec{v}_i$ of the ith particle is updated according to Eq. (4), while its current position is updated as follows [81]:

$$x_{ij}^{(t)} = \begin{cases} 1, & \text{if } \mathcal{R} < S\left(v_{ij}^{(t)}\right), \\ 0, & \text{otherwise,} \end{cases} \tag{27}$$

for all $i \in I$, $j \in D$, where $\mathcal{R} \sim \mathcal{U}(0, 1)$ is a random variable. The sigmoid is defined as:

$$S(x) = \frac{1}{1 + \exp(-x)},$$

clamping the velocity in the range $[0, 1]$ so it can be translated to a probability value. The rest of the algorithm follows the basic rules of the canonical PSO.

The algorithm was demonstrated on a set of test problems with promising results. It also exhibits conceptual similarities with reinforcement learning approaches as it was shown in [93]. BPSO is still considered as the basis for the development of contemporary ad hoc binary PSO variants for specific applications [14, 19, 70, 119, 151, 211].

Guaranteed Convergence PSO

The *guaranteed convergence PSO* (GCPSO) was introduced in 2002 [193] based on the observation that small velocities can prohibit the convergence of the gbest PSO with inertia weight. This is ascribed to the cancellation of the attraction forces of

the overall best particle towards its best positions, along with small velocities that essentially immobilize it in the long run.

The problem was solved by modifying the position update of the overall best particle, denoted with the index g, as follows [193]:

$$x_{gj}^{(t+1)} = p_{gj}^{(t)} + \chi\, v_{gj}^{(t)} + \rho^{(t)}(1 - 2\mathcal{R}), \tag{28}$$

where $j \in D$, $\rho^{(t)}$ is a *scaling factor*, and $\mathcal{R} \sim \mathcal{U}(0, 1)$ is a random variable. The rest of the particles are updated with the standard rules of Eqs. (4) and (5). The scaling factor is dynamically adjusted according to the number of consecutive successes and failures in improving the overall best:

$$\rho^{(t+1)} = \begin{cases} 2\,\rho^{(t)}, & \text{if } m_{\text{suc}}^{(t)} > T_{\text{suc}}, \\ 0.5\,\rho^{(t)}, & \text{if } m_{\text{fail}}^{(t)} > T_{\text{fail}}, \\ \rho^{(t)}, & \text{otherwise}, \end{cases}$$

where $m_{\text{suc}}^{(t)}$ and $m_{\text{fail}}^{(t)}$ are counters of the consecutive successes and failures at iteration t, respectively, and T_{suc}, T_{fail} are the corresponding user-defined thresholds. The counters are reset whenever the row of consecutive successes or failures is interrupted by a failure or success, respectively.

Convergence of GCPSO was proved and the values $\rho^{(0)} = 1$, $T_{\text{suc}} = 15$, and $T_{\text{fail}} = 5$ were recommended in [193], where GCPSO was shown to be very promising on typical benchmark problems.

Bare Bones PSO

The *bare bones PSO* (BBPSO) was introduced in 2003 [78] as a simplification of the canonical PSO. Its main feature is the replacement of the velocity update of Eq. (4), with Gaussian sampling around the best positions. Specifically, the current position of each particle is updated by sampling the Gaussian distribution:

$$x_{ij}^{(t+1)} \sim \mathcal{N}\left(\mu_{ij}^{(t)}, \sigma_{ij}^{2\,(t)}\right), \tag{29}$$

where:

$$\mu_{ij}^{(t)} = \frac{1}{2}\left(p_{gj}^{(t)} + p_{ij}^{(t)}\right), \qquad \sigma_{ij}^{2\,(t)} = \left|p_{gj}^{(t)} - p_{ij}^{(t)}\right|,$$

and g stands for the index of the overall best particle. The rest of the canonical PSO procedures such as the best position update and boundary violations handling are retained.

BBPSO aimed at offering a very simple, parameter-free PSO variant. Its properties were studied in [124, 150] and it was extended for applications in various scientific fields [15, 20, 84, 192, 219–221].

Fully Informed PSO

The *fully informed PSO* (FIPS) was introduced in 2004 [111] as a variant of the canonical PSO that extends the concept of neighbor influence. In FIPS each particle is influenced by all its neighbors and not only the best one. The particle update scheme of Eqs. (4) and (5) is replaced by the following update rule:

$$x_{ij}^{(t+1)} = x_{ij}^{(t)} + \chi \left(x_{ij}^{(t)} - x_{ij}^{(t-1)} \right) + \sum_{k \in NB_i} \mathcal{C}_k \left(p_{kj}^{(t)} - x_{ij}^{(t)} \right), \tag{30}$$

where $i \in I$, $j \in D$, NB_i is the ith particle's neighborhood of size s_i, and $\mathcal{C}_k \sim \mathcal{U}(0, c/s_i)$ is a random variable. Applying the analysis of [29] in FIPS, the default parameter setting of Eq. (18) becomes $\chi = 0.729$, $c = 2.992$ [79].

FIPS introduced a novel point of view for the concept of neighborhood. The social influence was enhanced, providing additional attractors to the particles. The outcome is a stochastic move around a stochastic average of the best positions of all neighbors. However, this also implies the dependency of the algorithm's performance on the selected neighborhood topology. For example, in the gbest PSO model where the neighborhood of each particle is the whole swarm, the particle move tends to become nearly random [79].

A multitude of neighborhood structures (more than 1000) were tested in [111] and interesting conclusions were derived regarding the potential of FIPS to improve the canonical PSO. Potential superiority was also verified in the use of FIPS to address a clustering problem [109].

Quantum PSO

Quantum PSO (QPSO) was introduced in 2004 [177], placing the algorithm into a new framework. In QPSO, the swarm is considered as a quantum system where each particle possesses a quantum state while moving in a *delta potential well* (DPW) towards a position $\vec{p}$. The quantum state depends on the employed wave function.

Borrowing from previous PSO analyses such as [29], the aforementioned position $\vec{p}$ is defined as in Eq. (11). Depending on the considered potential, different variants of QPSO can be defined. Three established approaches are the following [112, 177, 179]:

1. Delta potential well:

$$\vec{x}_i^{(t+1)} = \vec{p}^{(t)} \pm \frac{\ln\left(\frac{1}{\mathcal{R}}\right)}{2\,q\,\ln(\sqrt{2})} \left\| \vec{x}_i^{(t)} - \vec{p}^{(t)} \right\|, \tag{31}$$

2. Harmonic oscillator:

$$\vec{x}_i^{(t+1)} = \vec{p}^{(t)} \pm \frac{\sqrt{\ln\left(\frac{1}{\mathcal{R}}\right)}}{0.47694\,q} \left\| \vec{x}_i^{(t)} - \vec{p}^{(t)} \right\|, \tag{32}$$

3. Square well:

$$\vec{x}_i^{(t+1)} = \vec{p}^{(t)} + \frac{0.6574}{\xi\,q} \cos^{-1}\left(\pm\sqrt{\mathcal{R}}\right) \left\| \vec{x}_i^{(t)} - \vec{p}^{(t)} \right\|, \tag{33}$$

where $i \in I, \mathcal{R} \sim \mathcal{U}(0, 1)$ is a random number, and ξ, q are user-defined parameters. Despite its identified parameter sensitivity, the algorithm has been embraced from the scientific community and also extended to a number of interesting applications [23, 30, 53, 62, 68, 101, 208, 218].

Unified PSO

The *unified PSO* (UPSO) was introduced in 2004 [136] as a PSO variant that harnesses the gbest and lbest PSO models in a unified scheme. The motivation behind its development lies in the inherent intensification properties of the gbest model and the corresponding diversification properties of the lbest model. Thus, their combination can form variants with different trade-offs of these two properties.

UPSO is based on the update equations of the canonical PSO with constriction coefficient. Specifically, let:

$$\mathcal{L}_{ij}^{(t+1)} = \chi\, v_{ij}^{(t)} + \mathcal{C}_1 \left(p_{ij}^{(t)} - x_{ij}^{(t)}\right) + \mathcal{C}_2 \left(p_{g_i j}^{(t)} - x_{ij}^{(t)}\right), \tag{34}$$

denote the velocity update of Eq. (4) for the lbest PSO model, where g_i is the index of the best neighbor of the ith particle and $j \in D$. Also, let:

$$\mathcal{G}_{ij}^{(t+1)} = \chi\, v_{ij}^{(t)} + \mathcal{C}_3 \left(p_{ij}^{(t)} - x_{ij}^{(t)}\right) + \mathcal{C}_4 \left(p_{gj}^{(t)} - x_{ij}^{(t)}\right), \tag{35}$$

be the corresponding velocity update for the gbest PSO model, i.e., g denotes the overall best particle. Then, the basic UPSO scheme is defined as [136]:

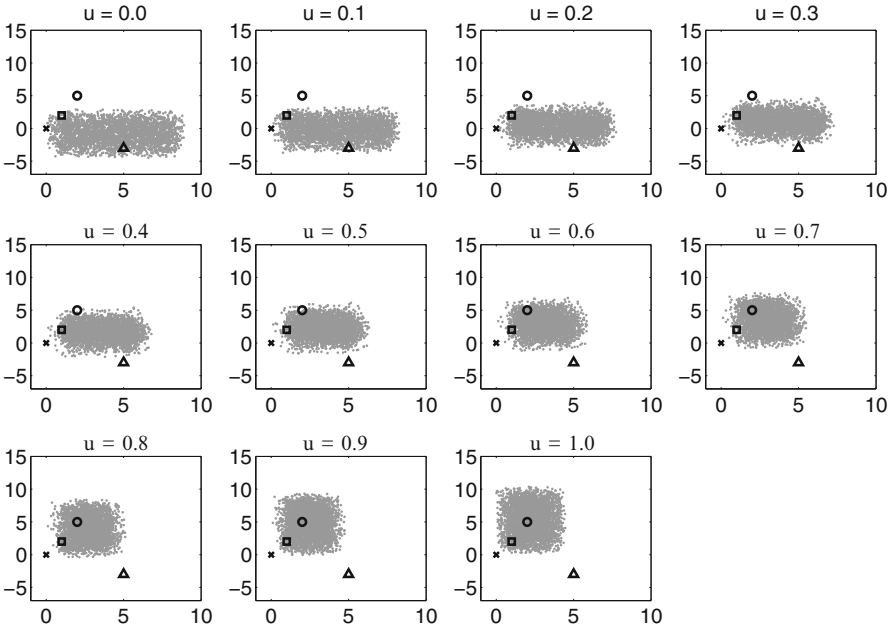


Fig. 9 The distribution of 3000 possible new positions (light grey points) of the 2-dimensional particle $\vec{x}_i = (0, 0)^\top$ (cross) with own best position $\vec{p}_i = (1, 2)^\top$ (square), neighborhood best position $\vec{p}_{g_i} = (5, -3)^\top$ (triangle), and overall best $\vec{p}_g = (2, 5)^\top$ (circle), for different values of $u \in [0, 1]$. For simplicity, the velocity vector is set to $\vec{v}_i = (0, 0)^\top$

$$v_{ij}^{(t+1)} = u\, \mathcal{G}_{ij}^{(t+1)} + (1 - u)\, \mathcal{L}_{ij}^{(t+1)}, \tag{36}$$

$$x_{ij}^{(t+1)} = x_{ij}^{(t)} + v_{ij}^{(t+1)}, \tag{37}$$

where $i \in I$, $j \in D$, and $u \in [0, 1]$ is a user-defined parameter called the *unification factor*, which controls the influence of the gbest and lbest term. Obviously, $u = 0$ corresponds to the lbest model, while $u = 1$ corresponds to the gbest model. All intermediate values define UPSO variants that combine the diversification/intensification properties of the two models. Figure 9 illustrates the distribution of new positions of a particle for different values of the unification factor.

In addition to the main UPSO model, an alternative with increased stochasticity was proposed in [136]. It came in two forms, namely:

$$v_{ij}^{(t+1)} = \mathcal{R}\, u\, \mathcal{G}_{ij}^{(t+1)} + (1 - u)\, \mathcal{L}_{ij}^{(t+1)}, \tag{38}$$

which is mostly based on the lbest model, and:

$$v_{ij}^{(t+1)} = u\, \mathcal{G}_{ij}^{(t+1)} + \mathcal{R}\,(1-u)\, \mathcal{L}_{ij}^{(t+1)}, \tag{39}$$

which is based on the gbest model [136]. The stochastic parameter $\mathcal{R} \sim \mathcal{N}(\mu, \sigma^2)$ is normally distributed and imitates the mutation operators in evolutionary algorithms.

Theoretical properties of UPSO were studied in [26, 136], while a thorough investigation of its parameter setting and adaptation was offered in [138]. Its performance has been studied on various problems with promising results [6, 52, 56, 83, 116, 117, 140, 191].

Cooperative PSO

Cooperative population-based algorithms [152] promote cooperation between population members towards a common goal. Cooperation can be either explicit through direct communication among them or implicit through a shared memory where information is deposited. Cooperation can be considered in a multi-population or single-population framework. In the first case, each population usually operates on a subspace of the original search space, e.g., on one coordinate direction of the solution vector. Its individuals carry partial solutions that are combined with those of the other populations, forming complete solutions. In the second case, the individuals usually carry complete solution information that is combined with the rest by using special recombination schemes [152].

In the context of PSO, one of the first notable attempts to design a *cooperative PSO* (CPSO) took place in 2004 [194]. This CPSO consists of d swarms (where d denotes the problem's dimension), each one comprising N_k particles, $k \in D$, i.e., according to Eq. (3):

$$\mathcal{S}^{[1]} = \left\{A_1^{[1]}, \ldots, A_{N_1}^{[1]}\right\}, \quad \ldots, \quad \mathcal{S}^{[d]} = \left\{A_1^{[d]}, \ldots, A_{N_d}^{[d]}\right\}.$$

Each swarm probes only one coordinate direction of the solution vector, applying any PSO variant (canonical PSO was used in the specific version). The evaluation of the particles with the objective function requires complete d-dimensional solutions. Thus, two main issues need to be addressed:

1. How shall particles from different swarms be selected to form complete solutions?
2. How shall particles be awarded or penalized for their contribution in solution quality?

These critical decisions have direct impact on the algorithm's performance and, thus, require special care.

In [194] two alternative schemes were proposed. The first scheme, denoted as CPSO-S$_k$, introduced a *context vector* $\vec{z}^*$ for the evaluation of the particles. This vector constitutes an external memory where each swarm $\mathcal{S}^{[k]}$ participates with its

1-dimensional overall best at the corresponding kth direction component:

$$\vec{z}^* = \left(\vec{p}_g^{[1]}, \vec{p}_g^{[2]}, \ldots, \vec{p}_g^{[d]}\right),$$

where $\vec{p}_g^{[k]}$ is the overall (1-dimensional) best of the kth swarm [194]. Then, the evaluation of the ith particle of the kth swarm is performed by replacing the kth swarm's best in $\vec{z}^*$ with its own information:

$$f\left(\vec{x}_i^{[k]}\right) = f\left(\vec{z}_{[i,k]}^*\right),$$

where:

$$\vec{z}_{[i,k]}^* = \left(\vec{p}_g^{[1]}, \ldots, \vec{p}_g^{[k-1]}, \vec{x}_i^{[k]}, \vec{p}_g^{[k+1]}, \ldots, \vec{p}_g^{[d]}\right),$$

where $\vec{x}_i^{[k]}$ is the current position of the particle under evaluation. Naturally, instead of the overall bests of the swarms, randomly selected best positions can be used in the context vector. Also, swarms of higher dimension can be used. However, both these alternatives can radically change the algorithm's performance. Obviously, the context vector $\vec{z}^*$ constitutes the best approximation of the problem's solution in CPSO-S$_k$.

The second variant from [194], denoted as CPSO-H$_k$, combines CPSO-S$_k$ with the canonical PSO and applies each algorithm alternatively at each iteration. In addition, information exchange between the two algorithms was considered by sharing half of the discovered solutions between them. The experimental assessment revealed that both CPSO-S$_k$ and CPSO-H$_k$ are promising, paving the way for further developments such as [66, 176, 180, 203, 222].

Comprehensive Learning PSO

The *comprehensive learning PSO* (CLPSO) [91] was proposed in 2006 as an alternative for alleviating premature convergence in gbest PSO that can be attributed to the use of the overall best solution in velocity update. In CLPSO, each particle can use the best position of any other particle to individually update the velocity vector components, according to a probabilistic scheme.

More specifically, the velocity update of Eq. (4) is replaced with the following [91]:

$$v_{ij}^{(t)} = \chi\, v_{ij}^{(t)} + \mathcal{C}\left(p_{q_{[i,j]}j} - x_{ij}^{(t)}\right), \tag{40}$$

where $j \in D$, $i \in I$, and $q_{[i,j]} \in I$ is the index of the particle that is used for the update of the jth velocity component of the ith particle. Naturally, this particle can be either the ith particle itself or another particle from the swarm. This decision is

probabilistically made according to predefined probabilities $\rho_1, \rho_2, \ldots, \rho_d$:

$$q_{[i,j]} = \begin{cases} i, & \text{if } \mathcal{R} \leqslant \rho_j, \\ \text{TOURN}\left(I'\right), & \text{otherwise,} \end{cases} \qquad \text{for all } j \in D,$$

where $\mathcal{R} \sim \mathcal{U}(0,1)$ is a uniformly distributed random variable, $I' = I \setminus \{i\}$, and $\text{TOURN}\left(I'\right)$ is an index selected from I' through *tournament selection* [91]. The latter requires the random selection of two particles from the set I'. The best between them, i.e., the one with smallest objective value, is the winner and participates in the update of v_{ij}.

In the case where $q_{[i,j]} = i$, for all $j \in D$, one of the components of v_{ij} is randomly selected and determined anew by using another particle. The indices $q_{[i,j]}$ are updated for each particle after a number of non-improving iterations. CLPSO has been extensively studied in [91]. A number of improvements, modifications, and applications in various fields have been proposed in [54,55,59,94,104,205,215,217].

TRIBES

The *TRIBES* algorithm was proposed in 2006 [27] as a novel PSO variant with self-adaptation capability. It is based on a special communication scheme between the neighborhoods, and it can admit the update rules of any PSO variant. In TRIBES, the ith particle is called *informant* of the jth particle if it shares its best position for the update of the latter. Accordingly, a *tribe* can be defined as a subset of the swarm, where each one of its members is informant of all the rest in the same tribe. Obviously, the swarm is the union set of all tribes.

Each tribe must have at least one communication channel with another tribe. In other words, between any two particles there shall be at least one path that connects them in the neighborhood's graph [27]. Also, the algorithm is self-adaptive, i.e., existing tribes can be deleted or new tribes can be generated. Hence, the communication channels between tribes change dynamically. The goodness criterion for the tribes is related to the performance of their members-particles. More specifically, they are characterized as *neutral* if they have not improved their best position in the last iteration, *good* if improvement was achieved in the last iteration, and *excellent* if improvement was achieved for more than one consecutive iterations. Accordingly, a tribe TR that contains s_{TR} particles is characterized as follows:

$$TR \text{ is } \begin{cases} \text{good}, & \text{if } N_{TR}^{\text{good}} > \mathcal{R}, \\ \text{bad}, & \text{otherwise,} \end{cases}$$

where N_{TR}^{good} is the number of good particles in tribe TR, and $\mathcal{R}$ is a randomly selected integer in $\{0, 1, \ldots, s_{TR}\}$. Moreover, additional rules are applied for the generation or elimination of particles and tribes:

1. The worst particle of the best tribe can be eliminated, inheriting its communication channels to the best particle of its tribe.
2. If a tribe consists of only one particle, it is eliminated if it has an informant with better performance.
3. Each tribe that was characterized as "bad" generates two new particles. The first one is randomly generated within the whole search space. The second one is randomly generated in the sphere centered on the best position of the best informant of the tribe's best particle, with radius equal to the distance between the latter and the sphere's center.

Adaptations are recommended to occur after a number of iterations so that the algorithm can deploy its dynamic [27]. Promising results were received with TRIBES under various settings [27]. Although TRIBES is a rather controversial PSO variant, it has contributed towards the development of self-adaptation mechanisms [33–35], while it was also enhanced for various applications [39, 171].

Niching PSO

Niching algorithms are applied on multimodal optimization problems where the main goal is the identification of multiple global/local minima. In such problems, the algorithms must be capable of identifying minima and retaining them until the end of the search. Although transformation techniques such as the one presented in section "Detecting Multiple Minimizers" can be used for this purpose, alternative algorithmic models that do not use external procedures have been developed.

Two efficient *niching PSO* approaches are the *speciation-based PSO* (SPSO) [128] and the *fitness Euclidean-distance ratio PSO* (FERPSO) [88]. In SPSO, the swarm is divided into subswarms, which are considered as species represented by the dominant (best) particle in the subswarm. A niche radius is also specified to define the size of species. Special procedures are applied for determining species and their seeds, while the global best particle is replaced by the species best or species seed. Also, all particles in the same species use the same neighborhood best at each iteration.

On the other hand, FERPSO is based on the lbest PSO model of Eqs. (4) and (5) [88], where the neighborhood's best $\vec{p}_{g_i}$ is taken as the particle that maximizes the *fitness Euclidean-distance ratio* (FER), defined as:

$$FER_{i,k} = \frac{\alpha \left[f(\vec{p}_i) - f(\vec{p}_k) \right]}{\| \vec{p}_i - \vec{p}_k \|}, \qquad k \in I,$$

where the scaling factor α is defined as:

$$\alpha = \frac{\sqrt{\sum_{l=1}^{d} \left(x_l^{\max} - x_l^{\min}\right)^2}}{f\left(\vec{p}_g\right) - f\left(\vec{p}_w\right)},$$

with $\vec{p}_g$ and $\vec{p}_w$ being the swarm's best and worst particles, respectively [88]. The effectiveness of both SPSO and FERPSO has led to further enhancements such as [89, 90, 100, 153, 162].

Standard PSO

The *standard PSO* (SPSO) was introduced in an attempt to define a baseline for the development and assessment of new PSO variants. Although there have been various versions (2006, 2007, and 2011), only SPSO-2011 [28] is presented here as it cures a number of weaknesses identified in the previous versions.

A feature of SPSO-2011 is the independence on the coordinate system. Let:

$$\mathcal{P}_{ij}^{(t)} = x_{ij}^{(t)} + \mathcal{C}_1 \left(p_{ij}^{(t)} - x_{ij}^{(t)}\right), \tag{41}$$

$$\mathcal{L}_{ij}^{(t)} = x_{ij}^{(t)} + \mathcal{C}_2 \left(p_{g_i j}^{(t)} - x_{ij}^{(t)}\right), \tag{42}$$

with $j \in D$, be two points for the ith particle that are a little "beyond" its own best position and its neighborhood best position, respectively, at iteration t. Then, the center of gravity between the two points and the current position of the particle is defined as:

$$\mathcal{G}_i^{(t)} = \frac{1}{3}\left(\vec{x}_i^{(t)} + \mathcal{P}_i^{(t)} + \mathcal{L}_i^{(t)}\right). \tag{43}$$

A new point $\vec{x'}_i^{(t)}$ is randomly (not necessarily uniformly) generated in the hypersphere $\mathcal{H}_i^{(t)}$ with center $\mathcal{G}_i^{(t)}$ and radius equal to its distance from the actual $\vec{x}_i^{(t)}$:

$$\vec{x'}_i^{(t)} \in \mathcal{H}_i^{(t)}\left(\mathcal{G}_i^{(t)}, \left\|\mathcal{G}_i^{(t)} - \vec{x}_i^{(t)}\right\|\right). \tag{44}$$

Then, the update equations of SPSO-2011 are given as follows [28]:

$$v_{ij}^{(t+1)} = \chi\, v_{ij}^{(t)} + x'^{(t)}_{ij} - x_i^{(t)}, \tag{45}$$

$$x_{ij}^{(t+1)} = x_{ij}^{(t)} + v_{ij}^{(t+1)}, \tag{46}$$

where $i \in I$ and $j \in D$. The rest of the algorithm follows the Canonical PSO approach. The particles are bounded in the search space according to Eq. (19). The

algorithm's parameters were studied in [216], while a thorough theoretical analysis was provided in [18].

Hybrid PSO

Hybrid optimization methods represent an advanced approach that integrates various techniques to leverage their respective strengths and mitigate their weaknesses. These methods are particularly beneficial for addressing complex and high-dimensional optimization challenges, as they enhance efficiency, accuracy, and robustness, yielding superior solutions compared to a singular method.

Although PSO is effective for many problems, its performance can be further enhanced by integrating different methods. A common approach involves hybridizing PSO with gradient-based methods, which utilize gradient information to enhance the search process and improve convergence rate. In the MEMPSODE software package [197], PSO was integrated with gradient-based optimization and direct search approaches.

Metaheuristics such as differential evolution (DE), simulated annealing (SA), ant colony optimization (ACO), variable neighborhood search (VNS), and genetic algorithms (GA) have been effectively integrated with PSO [7, 65, 107, 110, 172], enhancing its global search capability while preventing it from being trapped in local optima. The integration of different metaheuristics can be accomplished in various ways. For instance, in HGAPSO, a hybrid PSO-GA approach, the top half of the best-performing individuals (called elites) in the population is treated as a swarm, and it is improved through PSO at each iteration. The remaining half of the population is generated by applying crossover and mutation operations to the enhanced elites [72]. The method aims at deriving benefits from the search capability of PSO as well as the diversity introduced by genetic operators, resulting in a robust optimization strategy.

In recent years, integrating PSO with machine learning techniques has led to significant advancements in optimization. The relevant literature can be broadly divided into two categories, namely, (*i*) studies where a machine learning method enhances PSO and (*ii*) studies where PSO improves the performance of machine learning techniques. In the former category, machine learning methods are utilized to find promising areas for further exploration, dynamically adjust the parameters of PSO, or initialize the particles [92, 141, 169]. These methods result in more intelligent and adaptive optimization strategies, thereby significantly enhancing the efficiency and effectiveness of the search process.

In the latter category, PSO is utilized to enhance feature selection, optimize the hyperparameters of machine learning models, or fine-tune neural network architectures [3, 213]. The last case, referred to as neural architecture search (NAS), is considered to be a crucial advancement because, nowadays, there is ongoing need for more sophisticated network designs that can adapt to diverse and complex datasets while minimizing computational costs. Recently, PSO was applied to optimize a convolutional neural network (CNN) block to produce

specialized architectures for image classification [198]. The optimized block was repeated multiple times to enhance the capacity of the final network, resulting in promising outcomes in terms of classification accuracy, number of parameters, and computational cost. Furthermore, the transferability of the evolved block on two additional datasets was considered, demonstrating its successful adaptation to similar domains and promising results on different datasets.

Hybrid methods have offered a robust and adaptable approach for addressing complex optimization problems, leading to more efficient and successful solutions. A multitude of PSO hybridizations for diverse problems is reported in [2, 9, 107, 172, 206].

Memetic PSO

The term *memetic algorithm* [118] is used to describe hybrid algorithms that consist of population-based metaheuristics with additional local search or learning mechanisms. Early *memetic PSO* (MPSO) schemes appeared in 2005 [97], hybridizing PSO with the Solis and Wets local search approach. Later, different schemes were proposed using alternative local search algorithms such as the random walk with direction exploitation and the Hooke and Jeeves method [143, 144]. Further enhancements and applications can be found in [10, 60, 95, 102, 114, 167, 201].

Besides the choice of the appropriate local search algorithm, which is mostly problem-dependent, additional decisions shall be taken prior to the application of MPSO [140]:

1. When to apply local search?
2. Where to apply local search?
3. What computational budget shall be devoted to local search?

A straightforward choice is the application of local search on each new best position of the particles and/or on their current positions at each iteration until a local minimum is detected. Naturally, this approach would require excessive computational resources that are hardly available in practice.

For this reason, the following strategies were proposed in [143] after empirical evaluations on established test problems:

(S1) Application of local search on the overall best position $\vec{p}_g$, whenever it changes.
(S2) For each best position $\vec{p}_i$, $i \in I$, local search is applied with a predefined probability $\rho > 0$.
(S3) Local search is applied both on $\vec{p}_g$ and some randomly selected best positions $\vec{p}_i, i \in I$.
(S4) Local search is applied on $\vec{p}_g$ as well as on the best positions $\vec{p}_i$ that lie in adequate distance from $\vec{p}_g$, e.g., $\|\vec{p}_g - \vec{p}_i\| > \varepsilon$, where $\varepsilon > 0$ is a predefined distance usually defined as a fraction of the search space.

Shannon's information entropy, employed as a measure of information diversity of the swarm, was employed in [142] along with the above schemes in order to make swarm-level decisions on the application of local search. Further details and extensive results are reported in [140], where it was shown that MPSO outperforms the canonical PSO (as expected), but also its gbest model could outperform the lbest model of canonical PSO. The latter result suggests that local search applied as above can be beneficial for both PSO's intensification and diversification properties.

Opposition-Based PSO

The opposition-based algorithms are based on the concept of opposite point [155]. If $\vec{x} = (x_1, \ldots, x_d)^T$ is a point in the search space $\mathcal{X}$ defined as in Eq. (2), then its opposite is defined as a point $\vec{x'} = (x'_1, \ldots, x'_d)^T$ with $x'_j = x_j^{\min} + x_j^{\max} - x_j$, for all $j \in D$. Evaluating both points simultaneously and keeping the best one can accelerate the optimization procedure [155].

This scheme was adapted in the framework of PSO, producing the *generalized opposition-based PSO* (GOPSO) [202]. In GOPSO, there are two swarms, $\mathcal{S}$ and $\mathcal{S}'$, comprising the particles and their opposites, respectively. The initial positions $\vec{x}_i^{(0)}$, $i \in I$, of $\mathcal{S}$ are randomly initialized, while for $\mathcal{S}'$ the initial positions $\vec{x'}_i^{(0)}$ are obtained as follows [202]:

$$x'^{(0)}_{ij} = \mathcal{R}\left(\alpha_j^{(0)} + \beta_j^{(0)}\right) - x_{ij}, \tag{47}$$

where $i \in I$, $j \in D$, $\mathcal{R} \sim \mathcal{U}(0, 1)$, and $\alpha_j^{(0)} = x_j^{\min}$, $\beta_j^{(0)} = x_j^{\max}$. Subsequently, both swarms are evaluated and merged. The N best particles are then selected to form the initial swarm of GOPSO.

At each iteration, a probabilistic decision is taken. The algorithm, with a user-defined probability $\rho > 0$, either chooses to update the boundaries $\alpha_j^{(t)}$, $\beta_j^{(t)}$, as follows [202]:

$$\alpha_j^{(t)} = \min_i \left\{x_{ij}^{(t)}\right\}, \qquad \beta_j^{(t)} = \max_i \left\{x_{ij}^{(t)}\right\}, \tag{48}$$

or applies the update equations of the canonical PSO defined in Eqs. (4) and (5). The procedure continues with the best positions update of Eq. (7). The new overall best undergoes also a mutation procedure where its components are perturbed with random numbers following a Cauchy distribution [202].

Experimental results suggested that GOPSO can be competitive to other PSO variants [202]. A number of different opposition-based approaches have been proposed in various application fields [4, 37, 42, 74, 75, 103, 165, 188, 200, 223].

PSO in Noisy Environments

Noisy problems habitually arise in real-world engineering applications. The use of measurement instruments or approximations through inaccurate mathematical models impose uncertainty on the objective values. Thus, noise tolerance is a desirable property for metaheuristic optimization algorithms such as PSO. In [131] the gbest PSO model was studied on a number of test problems contaminated by Gaussian noise, exhibiting promising behavior. Various other studies followed in subsequent years [13, 125, 154].

A common technique for tackling noisy problems is the reevaluation of the objective function at each point. Specifically, if the objective function is given as:

$$f'(\vec{x}) = f(\vec{x}) + \mathcal{R},$$

where $\mathcal{R}$ is a random variable following some (usually Gaussian) distribution, then PSO evaluates the particles by using:

$$F(\vec{x}) = \frac{1}{M} \sum_{m=1}^{M} f'^{(m)}(\vec{x}),$$

where $f'^{(m)}(\vec{x})$ is the mth reevaluation of $\vec{x}$ using $f'(\vec{x})$. Reevaluation serves as a mean for approximating the expected value of the noisy objective function, i.e., $F(\vec{x}) \approx \mathbb{E}(f'(\vec{x}))$. Accuracy increases with the number M of reevaluations, although it also increases the computational cost.

Thus, the trade-off between better estimations of the objective values and the corresponding computational burden shall be tuned. In such cases, specialized techniques such as the *optimal computing budget allocation* (OCBA) [22] have been used to optimally allocate the reevaluation budget in order to provide reliable evaluation and identification of the promising particles [125]. Such techniques can be used along with parameter tuning [13] or learning strategies [145] for improved results. Also, they do not require modification of the algorithm. In different approaches, specialized operators have been developed with remarkable success [64, 182].

Multiobjective PSO

Multiobjective optimization (MO) problems consist of a number of objective functions that need to be simultaneously optimized. In contrast to the definition of Eq. (1), a multiobjective problem is defined as the minimization of a vector function [32]:

$$\vec{f}(\vec{x}) = (f_1(\vec{x}), f_2(\vec{x}), \ldots, f_K(\vec{x}))^\top,$$

possibly subject to constraints $C_i(\vec{x}) \leqslant 0$, $i = 1, 2, \ldots, m$. Typically, the objective functions $f_k(\vec{x})$ can be conflicting. Thus, it is highly improbable that a single solution that minimizes all of them can be found. For this reason, the main interest in such problems is concentrated on the detection of *Pareto optimal* solutions. These solutions are nondominated by any other point in the search space, i.e., they are at least as good as any other point for all the objectives $f_k(\vec{x})$. Formally, if $\vec{x}$, $\vec{y}$ are two points in the search space $\mathcal{X}$, then $\vec{f}(\vec{x})$ is said to dominate $\vec{f}(\vec{y})$, and we denote $\vec{f}(\vec{x}) \prec \vec{f}(\vec{y})$, if it holds that:

$$f_k(\vec{x}) \leqslant f_k(\vec{y}), \text{ for all } k = 1, 2, \ldots, K,$$

and:

$$f_{k'}(\vec{x}) < f_{k'}(\vec{y}), \text{ for at least one } k' \in \{1, 2, \ldots, K\}.$$

Thus, $\vec{x}^* \in \mathcal{X}$ is a Pareto optimal point if there is no other point $\vec{y} \in \mathcal{X}$ such that $\vec{f}(\vec{y}) \prec \vec{f}(\vec{x}^*)$. Obviously, a whole (even infinite) set $\{\vec{x}_1^*, \vec{x}_2^*, \ldots\}$ of Pareto optimal solutions may exist. The set of all the corresponding function value vectors $\{\vec{f}(\vec{x}_1^*), \vec{f}(\vec{x}_2^*), \ldots\}$ is called the *Pareto front*.

There are two main approaches for tackling MO problems. The first one aggregates the objectives into a single one and solves the problem with the typical methodologies for single-objective optimization. The second approach requires vector-evaluated operators and it is based on the concept of Pareto dominance. In the context of PSO, early aggregation-based approaches were developed in 2002 [133], where the canonical PSO was used for the minimization of a weighted aggregation of the objective functions:

$$F(\vec{x}) = \sum_{k=1}^{K} w_k \, f_k(\vec{x}), \qquad \sum_{k=1}^{K} w_k = 1.$$

Both a *conventional weighted aggregation* (CWA) approach with fixed weights and a *dynamic weighted aggregation* (DWA) approach [71] were investigated with promising results. Obviously, the detection of many Pareto optimal solutions through weighted aggregation requires multiple runs of PSO, as a single solution of $F(\vec{x})$ is obtained per run. From a computational point of view, this is a drawback since the swarms have the ability to simultaneously evolve many solutions. Nevertheless, it is still considered as a popular approach in various applications mostly due to its simplicity.

A *vector-evaluated PSO* (VEPSO) was also proposed in [133] and parallelized later in [130]. VEPSO uses a number of K swarms, one for each objective f_k. The kth swarm $\mathcal{S}_k$ is evaluated only with the corresponding objective f_k, $k = 1, 2, \ldots, K$. The swarms are updated according to the gbest model of the canonical PSO, although with a slight modification. Specifically, the overall best that is used for the velocity update of the particles in the kth swarm comes from another

swarm. Clearly, this is a migration scheme aiming at transferring information among swarms. The donator swarm can be either a neighbor of the kth swarm in a ring topology scheme as the one described in section "Concept of Neighborhood" or it can be randomly selected [133]. VEPSO was studied on standard MO benchmark problems with promising results [130].

There is a large number of developments and applications on multiobjective PSO approaches in the relevant literature [1, 5, 35, 36, 38, 41, 43, 69, 96, 98, 105, 209, 214, 224]. The interested reader can find comprehensive surveys in [139, 158].

Applications and Further Resources

It would be futile to even try to enumerate all published applications of PSO. From the year 2005 onwards, more than 400 papers on PSO applications appear each year, spanning various scientific and technological fields. Computer Science concentrates the majority of these works (29.6%), especially in the fields of machine learning, data mining, and robotics. Engineering follows closely (27.8%), with numerous works in intelligent systems, forecasting, and energy handling.

A fraction of these applications was cited in the previous sections. In addition, the interested reader can refer to devoted survey papers such as [147], which was probably the first notable attempt to collect and categorize PSO applications. An analytical survey was published in [164], where a huge number of PSO-based applications was categorized along with more than 100 PSO variants. Further applications are reported in relevant books such as [140]. The appendix at the end of the present chapter contains a number of resources for further inquiry on PSO-based developments.

Conclusions

PSO has been established as one of the most popular metaheuristic optimization algorithms. Its popularity stems from its nice performance and adequate simplicity that renders it usable even by nonexpert researchers. In the previous sections, a number of variants and enhancements was presented. This is only a small fraction of the existing PSO-related literature, which counts thousands of papers. Thus, a reasonable question would be how much room is left for further developments on PSO. In the author's opinion, there is still fertile ground for new developments.

Despite the numerous research contributions, there are open problems regarding the pursuit of intelligent behavior and self-adaptation. Moreover, the evolution of computer and web technologies always introduces new challenges on the design and development of algorithms that can take full advantage of their properties to solve problems of increasing complexity. For example, the GPU computing paradigm and modern multicore desktop systems can offer computational power comparable to small- and medium-size clusters. Cloud computing and ubiquitous computing environments are other examples. Also, new ad hoc operators and procedures for

specific problems are expected to appear, especially in fields related to artificial intelligence.

Closing this chapter, the authors would like to quote Albert Einstein's words to motivate new researchers towards unconventional thinking, which was the main ingredient for the development of PSO so far:

"You will never solve problems using the same thinking you created them with."

Appendix

A number of sources for further inquiry and experimentation with PSO are reported below:

Books
[27], [46], [45], [82], [113], [120], [127], [140], [178]

Survey papers
[8], [11], [12], [47], [48], [58], [147], [149], [159], [160], [164], [166], [187], [199]

Webpages
Particle Swarm Central
http://www.particleswarm.info/
M. Clerc's PSO page
http://clerc.maurice.free.fr/pso/

Software
PSO in C (code published in [197])
https://data.mendeley.com/datasets/s4thf738k7/1
PSO in Matlab
http://www.mathworks.com/matlabcentral/fileexchange/7506
http://psotoolbox.sourceforge.net/
PSO in Java
http://jswarm-pso.sourceforge.net/
PSO in Python
https://pyswarms.readthedocs.io/en/latest/index.html
PSO in R
https://www.rdocumentation.org/packages/pso/versions/1.0.4/topics/psoptim

Cross-References

▸ Adaptive and Multi-level Metaheuristics
▸ Ant Colony Optimization: A Component-Wise Overview
▸ Evolutionary Algorithms
▸ Genetic Algorithms
▸ Hyper-heuristics

- Memetic Algorithms
- Multiobjective Optimization
- Multi-start Methods
- Parallel Meta-heuristic Search
- Particle Swarm Optimization for the Vehicle Routing Problem: A Survey and a Comparative Analysis

References

1. Abido MA (2010) Multiobjective particle swarm optimization with nondominated local and global sets. Nat Comput 9(3):747–766
2. Abualigah L-M Khader A-T (2017) Unsupervised text feature selection technique based on hybrid particle swarm optimization algorithm with genetic operators for the text clustering. J Supercomput 73:4773–4795
3. Abualigah L-M, Khader A-T, Hanandeh E-S (2018) A new feature selection method to improve the document clustering using particle swarm optimization algorithm. J Comput Sci 25:456–466
4. Agarwal M, Srivastava G-M-S (2021) Opposition-based learning inspired particle swarm optimization (OPSO) scheme for task scheduling problem in cloud computing. J Ambient Intell Human Comput 12(10):9855–9875
5. Agrawal S, Panigrahi BK, Tiwari MK (2008) Multiobjective particle swarm algorithm with fuzzy clustering for electrical power dispatch. IEEE Trans Evol Comput 12(5):529–541
6. Ahmadi MA (2012) Neural network based unified particle swarm optimization for prediction of asphaltene precipitation. Fluid Phase Equilibria 314:46–51
7. Ajibade S-S-M, Ahmad N-B-B, Zainal A (2020) A hybrid chaotic particle swarm optimization with differential evolution for feature selection. In: 2020 IEEE symposium on industrial electronics & applications (ISIEA), pp 1–6
8. Aote AS, Raghuwanshi MM, Malik L (2013) A brief review on particle swarm optimization: limitations & future directions. Int J Comput Sci Eng 2(5):196–200
9. Aydilek I-B (2018) A hybrid firefly and particle swarm optimization algorithm for computationally expensive numerical problems. Appl Soft Comput 66:232–249
10. Aziz M, Tayarani-N M-H (2014) An adaptive memetic particle swarm optimization algorithm for finding large-scale Latin hypercube designs. Eng Appl Artif Intell 36:222–237
11. Banks A, Vincent J, Anyakoha C (2007) A review of particle swarm optimization. Part I: background and development. Nat Comput 6(4):467–484
12. Banks A, Vincent J, Anyakoha C (2008) A review of particle swarm optimization. Part II: hybridisation, combinatorial, multicriteria and constrained optimization, and indicative applications. Nat Comput 7(1):109–124
13. Bartz-Beielstein T, Blum D, Branke J (2007) Particle swarm optimization and sequential sampling in noisy environments. In: Doerner KF et al (eds) Metaheuristics: progress in complex systems optimization. Operations research/computer science interfaces series, vol 39, Springer US, Boston, pp 261–273
14. Bin W, Qinke P, Jing Z, Xiao C (2012) A binary particle swarm optimization algorithm inspired by multi-level organizational learning behavior. Eur J Oper Res 219(2):224–233
15. Blackwell T (2012) A study of collapse in bare bones particle swarm optimization. IEEE Trans Evol Comput 16(3):354–372
16. Blum C, Puchinger J, Raidl GR, Roli A (2011) Hybrid metaheuristics in combinatorial optimization: A survey. Appl Soft Comput J 11(6):4135–4151
17. Bonabeau E, Dorigo M, Théraulaz G (1999) Swarm intelligence: from natural to artificial systems. Oxford University Press, New York

18. Bonyadi MR, Michalewicz Z (2014) SPSO2011 – analysis of stability, local convergence, and rotation sensitivity. In: GECCO 2014 – proceedings of the 2014 genetic and evolutionary computation conference, pp 9–15
19. Camci F (2009) Comparison of genetic and binary particle swarm optimization algorithms on system maintenance scheduling using prognostics information. Eng Optim 41(2):119–136
20. Campos M, Krohling R-A (2016) Entropy-based bare bones particle swarm for dynamic constrained optimization. Knowl-Based Syst 97:203–223
21. Chauhan P, Deep K, Pant M (2013) Novel inertia weight strategies for particle swarm optimization. Mem Comput 5(3):229–251
22. Chen C-H, Lin J, Yücesan E, Chick SE (2000) Simulation budget allocation for further enhancing the efficiency of ordinal optimization. Discret Event Dyn Syst Theory Appl 10(3):251–270
23. Chen J, Yang D, Feng Z (2012) A novel quantum particle swarm optimizer with dynamic adaptation. J Comput Inf Syst 8(12):5203–5210
24. Chen Z, He Z, Zhang C (2010) Particle swarm optimizer with self-adjusting neighborhoods. In: Proceedings of the 12th annual genetic and evolutionary computation conference (GECCO 2010), Portland, pp 909–916
25. Chou J-S, Truong D-N, Le T-L (2020) Interval forecasting of financial time series by accelerated particle swarm-optimized multi-output machine learning system. IEEE Access 8:14798–14808
26. Cleghorn C-W, Engelbrecht A (2016) Unified particle swarm optimizer: Convergence analysis. In: 2016 IEEE congress on evolutionary computation (CEC). IEEE, pp 447–454
27. Clerc M (2006) Particle swarm optimization. ISTELtd, London
28. Clerc M (2012) Standard particle swarm optimization. Technical Report 2012, Particle Swarm Central
29. Clerc M, Kennedy J (2002) The particle swarm-explosion, stability, and convergence in a multidimensional complex space. IEEE Trans Evol Comput 6(1):58–73
30. Coelho LDS (2008) A quantum particle swarm optimizer with chaotic mutation operator. Chaos Solitons Fractals 37(5):1409–1418
31. Coello Coello CA (1999) Self-adaptive penalties for GA-based optimization. In: Proceedings of 1999 IEEE congress on evolutionary computation, Washington, DC, vol 1, pp 573–580
32. Coello Coello CA, Van Veldhuizen DA, Lamont GB (2002) Evolutionary algorithms for solving multi-objective problems. Kluwer, New York
33. Cooren Y, Clerc M, Siarry P (2008) Initialization and displacement of the particles in TRIBES, a parameter-free particle swarm optimization algorithm. Stud Comput Intell 136:199–219
34. Cooren Y, Clerc M, Siarry P (2009) Performance evaluation of TRIBES, an adaptive particle swarm optimization algorithm. Swarm Intell 3(2):149–178
35. Cooren Y, Clerc M, Siarry P (2011) MO-TRIBES, an adaptive multiobjective particle swarm optimization algorithm. Comput Optim Appl 49(2):379–400
36. Dai C, Wang Y, Ye M (2015) A new multi-objective particle swarm optimization algorithm based on decomposition. Inf Sci 325:541–557
37. Dai Y, Liu L, Feng S (2014) On the identification of coupled pitch and heave motions using opposition-based particle swarm optimization. Math Problems Eng 2014, 784049
38. Daneshyari M, Yen GG (2011) Cultural-based multiobjective particle swarm optimization. IEEE Trans Syst Man Cybern Part B Cybern 41(2):553–567
39. Daoudi M, Boukra A, Ahmed-Nacer M (2011) Adapting TRIBES algorithm for traveling salesman problem. In: Proceedings of the 10th international symposium on programming and systems, ISPS'2011, pp 163–168
40. Davarynejad M, Van Den Berg J, Rezaei J (2014) Evaluating center-seeking and initialization bias: the case of particle swarm and gravitational search algorithms. Inf Sci 278:802–821
41. Delgarm N, Sajadi B, Kowsary F, Delgarm S (2016) Multi-objective optimization of the building energy performance: a simulation-based approach by means of particle swarm optimization (PSO). Appl Energy 170:293–303

42. Dong W, Kang L, Zhang W (2017) Opposition-based particle swarm optimization with adaptive mutation strategy. Soft Comput 21:5081–5090
43. Dos Santos Coelho L, Ayala HVH, Alotto P (2010) A multiobjective gaussian particle swarm approach applied to electromagnetic optimization. IEEE Trans Magn 46(8):3289–3292
44. Eberhart RC, Kennedy J (1995) A new optimizer using particle swarm theory. In: Proceedings sixth symposium on micro machine and human science, Piscataway. IEEE Service Center, pp 39–43
45. Eberhart RC, Shi Y, Kennedy J (2001) Swarm intelligence. Morgan Kaufmann Publishers, USA
46. Engelbrecht AP (2006) Fundamentals of computational swarm intelligence. Wiley, USA
47. Eslami M, Shareef H, Khajehzadeh M, Mohamed A (2012) A survey of the state of the art in particle swarm optimization. Res J Appl Sci Eng Technol 4(9):1181–1197
48. Gad A-G (2022) Particle swarm optimization algorithm and its applications: a systematic review. Arch Comput Methods Eng 29(5):2531–2561
49. Gao W-F, Liu S-Y, Huang L-L (2012) Particle swarm optimization with chaotic opposition-based population initialization and stochastic search technique. Commun Nonlinear Sci Numer Simul 17(11):4316–4327
50. Garg H (2016) A hybrid PSO-GA algorithm for constrained optimization problems. Appl Math Comput 274:292–305
51. Ge RP, Qin YF (1987) A class of filled functions for finding global minimizers of a function of several variables. J Optim Theory Appl 54:241–252
52. Gholipour R, Khosravi A, Mojallali H (2013) Suppression of chaotic behavior in duffing-holmes system using backstepping controller optimized by unified particle swarm optimization algorithm. Int J Eng Trans B Appl 26(11):1299–1306
53. Gholizadeh S, Moghadas R (2014) Performance-based optimum design of steel frames by an improved quantum particle swarm optimization. Adv Struct Eng 17(2):143–156
54. Goudos SK, Moysiadou V, Samaras T, Siakavara K, Sahalos JN (2010) Application of a comprehensive learning particle swarm optimizer to unequally spaced linear array synthesis with sidelobe level suppression and null control. IEEE Antennas Wirel Propag Lett 9:125–129
55. Gülcü Ş, Kodaz H (2015) A novel parallel multi-swarm algorithm based on comprehensive learning particle swarm optimization. Eng Appl Artif Intell 45:33–45
56. He G, Wu B (2014) Unified particle swarm optimization with random ternary variables and its application to antenna array synthesis. J Electromagn Waves Appl 28(6):752–764
57. He J, Dai H, Song X (2014) The combination stretching function technique with simulated annealing algorithm for global optimization. Optim Methods Softw 29(3):629–645
58. Houssein E-H, Gad A-G, Hussain K, Suganthan P-N (2021) Major advances in particle swarm optimization: theory, analysis, and application. Swarm Evol Comput 63:100868
59. Hu Z, Bao Y, Xiong T (2014) Comprehensive learning particle swarm optimization based memetic algorithm for model selection in short-term load forecasting using support vector regression. Appl Soft Comput J 25:15–25
60. Huang K-W, Chen J-L, Yang C-S, Tsai C-W (2014) A memetic particle swarm optimization algorithm for solving the dna fragment assembly problem. Neural Comput Appl, in press
61. Huda R-K, Ramesh K, Banka H (2020) New efficient initialization and updating mechanisms in PSO for feature selection and classification. Neural Comput Appl 32(8):3283–3294
62. Jamalipour M, Gharib M, Sayareh R, Khoshahval F (2013) Pwr power distribution flattening using quantum particle swarm intelligence. Ann Nucl Energy 56:143–150
63. Janson S, Middendorf M (2004) A hierarchical particle swarm optimizer for dynamic optimization problems. In: Lecture notes in computer science, vol 3005. Springer, Berlin, pp 513–524
64. Janson S, Middendorf M (2006) A hierarchical particle swarm optimizer for noisy and dynamic environments. Gen Program Evol Mach 7(4):329–354
65. Javidrad F, Nazari M (2017) A new hybrid particle swarm and simulated annealing stochastic optimization method. Appl Soft Comput 60:634–654

66. Jiang B, Wang N (2014) Cooperative bare-bone particle swarm optimization for data clustering. Soft Comput 18(6):1079–1091
67. Jiang M, Luo YP, Yang SY (2007) Stochastic convergence analysis and parameter selection of the standard particle swarm optimization algorithm. Inf Process Lett 102:8–16
68. Jiao B, Yan S (2011) A cooperative co-evolutionary quantum particle swarm optimizer based on simulated annealing for job shop scheduling problem. Int J Artif Intell 7(11 A):232–247
69. Jin N, Rahmat-Samii Y (2007) Advances in particle swarm optimization for antenna designs: Real-number, binary, single-objective and multiobjective implementations. IEEE Trans Antennas Propag 55(3 I):556–567
70. Jin N, Rahmat-Samii Y (2010) Hybrid real-binary particle swarm optimization (HPSO) in engineering electromagnetics. IEEE Trans Antennas Propag 58(12):3786–3794
71. Jin Y, Olhofer M, Sendhoff B (2001) Evolutionary dynamic weighted aggregation for multiobjective optimization: why does it work and how? In: Proceedings GECCO 2001 conference, San Francisco, pp 1042–1049
72. Juang C-F (2004) A hybrid of genetic algorithm and particle swarm optimization for recurrent network design. IEEE Trans Syst Man Cybern Part B (Cybern) 34(2):997–1006
73. Kadirkamanathan V, Selvarajah K, Fleming PJ (2006) Stability analysis of the particle dynamics in particle swarm optimizer. IEEE Trans Evol Comput 10(3):245–255
74. Kang L, Chen R-S, Cao W, Chen Y-C (2020) Non-inertial opposition-based particle swarm optimization and its theoretical analysis for deep learning applications. Appl Soft Comput 88:106038
75. Kaucic M (2013) A multi-start opposition-based particle swarm optimization algorithm with adaptive velocity for bound constrained global optimization. J Glob Optim 55(1):165–188
76. Kennedy J (1998) The behavior of particles. In: Porto VW, Saravanan N, Waagen D, Eiben AE (eds) Evolutionary programming, vol VII. Springer, Berlin, pp 581–590
77. Kennedy J (1999) Small worlds and mega-minds: effects of neighborhood topology on particle swarm performance. In: Proceedings of IEEE Congress on Evolutionary Computation, Washington, DC. IEEE Press, pp 1931–1938
78. Kennedy J (2003) Bare bones particle swarms. In: Proceedings of IEEE swarm intelligence symposium, Indianapolis. IEEE Press, pp 80–87
79. Kennedy J (2010) Particle swarm optimization. In: Sammut C, Webb G (eds) Encyclopedia of machine learning. Springer, New York, pp 760–766
80. Kennedy J, Eberhart RC (1995) Particle swarm optimization. In: Proceedings of IEEE international conference on neural networks, Piscataway, vol IV. IEEE Service Center, pp 1942–1948
81. Kennedy J, Eberhart RC (1997) A discrete binary version of the particle swarm algorithm. In: Proceedings of conference on systems, man and cybernetics, pp 4104–4109
82. Kiranyaz S, Ince T, Gabbouj M (2014) Multidimensional particle swarm optimization for machine learning and pattern recognition. Springer, Berlin
83. Kotsireas IS, Koukouvinos C, Parsopoulos KE, Vrahatis MN (2006) Unified particle swarm optimization for Hadamard matrices of Williamson type. In: Proceedings of the 1st international conference on mathematical aspects of computer and information sciences (MACIS 2006), Beijing, pp 113–121
84. Krohling RA, Campos M, Borges P (2010) Bare bones particle swarm applied to parameter estimation of mixed weibull distribution. Adv Intell Soft Comput 75:53–60
85. Kwok NM, Ha QP, Liu DK, Fang G, Tan KC (2007) Efficient particle swarm optimization: a termination condition based on the decision-making approach. In: Proceedings of the 2007 IEEE congress on evolutionary computation (CEC 2007), pp 3353–3360
86. Laskari EC, Parsopoulos KE, Vrahatis MN (2002) Particle swarm optimization for integer programming. In: Proceedings of the IEEE 2002 congress on evolutionary computation (IEEE CEC 2002), Honolulu. IEEE Press, pp 1582–1587
87. Lawler EL, Wood DW (1966) Branch and bound methods: a survey. Oper Res 14:699–719

88. Li X (2007) A multimodal particle swarm optimizer based on fitness Euclidean-distance ratio, pp 78–85
89. Li X (2010) Niching without niching parameters: particle swarm optimization using a ring topology. IEEE Trans Evol Comput 14(1):150–169
90. Li Y, Chen Y, Zhong J, Huang Z (2019) Niching particle swarm optimization with equilibrium factor for multi-modal optimization. Inf Sci 494:233–246
91. Liang JJ, Qin AK, Suganthan PN, Baskar S (2006) Comprehensive learning particle swarm optimizer for global optimization of multimodal functions. IEEE Trans Evol Comput 10(3):281–295
92. Liang X, Li W, Zhang Y, Zhou M (2015) An adaptive particle swarm optimization method based on clustering. Soft Comput 19:431–448
93. Likas A, Blekas K, Stafylopatis A (1996) Parallel recombinative reinforcement learning: a genetic approach. J Intell Syst 6(2):145–169
94. Lin A, Sun W, Yu H, Wu G, Tang H (2019) Adaptive comprehensive learning particle swarm optimization with cooperative archive. Appl Soft Comput 77:533–546
95. Lin J-T, Chiu C-C (2018) A hybrid particle swarm optimization with local search for stochastic resource allocation problem. J Intell Manuf 29(3):481–495
96. Lin Q, Li J, Du Z, Chen J, Ming Z (2015) A novel multi-objective particle swarm optimization with multiple search strategies. Eur J Oper Res 247(3):732–744
97. Liu B-F, Chen H-M, Chen J-H, Hwang S-F, Ho S-Y (2005) MeSwarm: memetic particle swarm optimization, pp 267–268
98. Liu DS, Tan KC, Huang SY, Goh CK, Ho WK (2008) On solving multiobjective bin packing problems using evolutionary particle swarm optimization. Eur J Oper Res 190(2):357–382
99. Liu J, Mei Y, Li X (2015) An analysis of the inertia weight parameter for binary particle swarm optimization. IEEE Trans Evol Comput 20(5):666–681
100. Liu Q, Du S, Van Wyk B-J, Sun Y (2020) Niching particle swarm optimization based on Euclidean distance and hierarchical clustering for multimodal optimization. Nonlinear Dyn 99:2459–2477
101. Liu R, Zhang P, Jiao L (2014) Quantum particle swarm optimization classification algorithm and its applications. Int J Pattern Recogn Artif Intell 28(2), 1459004
102. Liu Z, Qin Z, Zhu P, Li H (2020) An adaptive switchover hybrid particle swarm optimization algorithm with local search strategy for constrained optimization problems. Eng Appl Artif Intell 95:103771
103. Lv L, Wang H, Li X, Xiao X, Zhang L (2014) Multi-swarm particle swarm optimization using opposition-based learning and application in coverage optimization of wireless sensor network. Sensor Lett 12(2):386–391
104. Lynn N, Suganthan P-N (2015) Heterogeneous comprehensive learning particle swarm optimization with enhanced exploration and exploitation. Swarm Evol Comput 24:11–24
105. Mac T-T, Copot C, Tran D-T, De Keyser R (2017) A hierarchical global path planning approach for mobile robots based on multi-objective particle swarm optimization. Appl Soft Comput 59:68–76
106. Magoulas GD, Vrahatis MN, Androulakis GS (1997) On the alleviation of local minima in backpropagation. Nonlinear Anal Theory Methods Appl 30(7):4545–4550
107. Mahi M, Baykan Ö-K, Kodaz H (2015) A new hybrid method based on particle swarm optimization, ant colony optimization and 3-opt algorithms for traveling salesman problem. Appl Soft Comput 30:484–490
108. Manquinho VM, Marques Silva JP, Oliveira AL, Sakallah KA (1997) Branch and bound algorithms for highly constrained integer programs. Technical report, Cadence European Laboratories, Portugal
109. Mansour E-M, Ahmadi A (2019) A novel clustering algorithm based on fully-informed particle swarm. In: 2019 IEEE congress on evolutionary computation (CEC), pp 713–720

110. Marinakis Y, Migdalas A, Sifaleras A (2017) A hybrid particle swarm optimization–variable neighborhood search algorithm for constrained shortest path problems. Eur J Oper Res 261(3):819–834
111. Mendes R, Kennedy J, Neves J (2004) The fully informed particle swarm: Simpler, maybe better. IEEE Trans Evol Comput 8(3):204–210
112. Mikki SM, Kishk AA (2006) Quantum particle swarm optimization for electromagnetics. IEEE Trans Antennas Propag 54(10):2764–2775
113. Mikki SM, Kishk AA (2008) Particle swarm optimization: a physics-based approach. Morgan and Claypool Publishers, USA
114. Moradi P, Gholampour M (2016) A hybrid particle swarm optimization for feature subset selection by integrating a novel local search strategy. Appl Soft Comput 43:117–130
115. Moustaki E, Parsopoulos KE, Konstantaras I, Skouri K, Ganas I (2013) A first study of particle swarm optimization on the dynamic lot sizing problem with product returns. In: XI Balkan conference on operational research (BALCOR 2013), Belgrade, pp 348–356
116. Nanda B, Maity D, Maiti DK (2014) Crack assessment in frame structures using modal data and unified particle swarm optimization technique. Adv Struct Eng 17(5):747–766
117. Nanda B, Maity D, Maiti DK (2014) Modal parameter based inverse approach for structural joint damage assessment using unified particle swarm optimization. Appl Math Comput 242:407–422
118. Neri F, Cotta C (2012) Memetic algorithms and memetic computing optimization: a literature review. Swarm Evol Comput 2:1–14
119. Nguyen B-H, Xue B, Andreae P, Zhang M (2019) A new binary particle swarm optimization approach: momentum and dynamic balance between exploration and exploitation. IEEE Trans Cybern 51(2):589–603
120. Olsson AE (ed) (2011) Particle swarm optimization: theory, techniques and applications. Nova Science Pub Inc., USA
121. Ozcan E, Mohan CK (1998) Analysis of a simple particle swarm optimization. In: Intelligent engineering systems through artificial neural networks, vol 8. ASME Press, USA, pp 253–258
122. Ozcan E, Mohan CK (1999) Particle swarm optimization: surfing the waves. In: Proceedings of 1999 IEEE international congress on evolutionary computation, Washington, pp 1939–1944
123. Padhye N, Deb K, Mittal P (2013) Boundary handling approaches in particle swarm optimization. In: Proceedings of seventh international conference on bio-inspired computing: theories and applications (BIC-TA 2012), vol 201, pp 287–298
124. Pan F, Hu X, Eberhart R, Chen Y (2008) An analysis of bare bones particle swarm
125. Pan H, Wang L, Liu B (2006) Particle swarm optimization for function optimization in noisy environment. Appl Math Comput 181(2):908–919
126. Pandremmenou K, Kondi LP, Parsopoulos KE, Bentley ES (2014) Game-theoretic solutions through intelligent optimization for efficient resource management in wireless visual sensor networks. Sig Process Image Commun 29(4):472–493
127. Parasuraman D (2012) Handbook of particle swarm optimization: concepts, principles & applications. Auris Reference, Nottingham
128. Parrott D, Li X (2006) Locating and tracking multiple dynamic optima by a particle swarm model using speciation. IEEE Trans Evol Comput 10(4):440–458
129. Parsopoulos KE, Plagianakos VP, Magoulas GD, Vrahatis MN (2001) Objective function "stretching" to alleviate convergence to local minima. Nonlinear Anal Theory Methods Appl 47(5):3419–3424
130. Parsopoulos KE, Tasoulis DK, Vrahatis MN (2004) Multiobjective optimization using parallel vector evaluated particle swarm optimization. In: Hamza MH (ed) Proceedings of the IASTED 2004 international conference on artificial intelligence and applications (AIA 2004), Innsbruck, vol 2. IASTED/ACTA Press, pp 823–828
131. Parsopoulos KE, Vrahatis MN (2001) Particle swarm optimizer in noisy and continuously changing environments. In: Hamza MH (ed) Artificial intelligence and soft computing. IASTED/ACTA Press, Anaheim, pp 289–294

132. Parsopoulos KE, Vrahatis MN (2002) Particle swarm optimization method for constrained optimization problems. In: Sincak P, Vascak J, Kvasnicka V, Pospichal J (eds) Intelligent technologies-theory and application: new trends in intelligent technologies. Frontiers in artificial intelligence and applications, vol 76. IOS Press, Netherlands, pp 214–220
133. Parsopoulos KE, Vrahatis MN (2002) Particle swarm optimization method in multiobjective problems. In: Proceedins of the ACM 2002 symposium on applied computing (SAC 2002), Madrid. ACM Press, pp 603–607
134. Parsopoulos KE, Vrahatis MN (2002) Recent approaches to global optimization problems through particle swarm optimization. Nat Comput 1(2–3):235–306
135. Parsopoulos KE, Vrahatis MN (2004) On the computation of all global minimizers through particle swarm optimization. IEEE Trans Evol Comput 8(3):211–224
136. Parsopoulos KE, Vrahatis MN (2004) UPSO: a unified particle swarm optimization scheme. In: Proceedings of the international conference of computational methods in sciences and engineering (ICCMSE 2004). Lecture series on computer and computational sciences, vol 1. VSP International Science Publishers, Zeist, pp 868–873
137. Parsopoulos KE, Vrahatis MN (2006) Studying the performance of unified particle swarm optimization on the single machine total weighted tardiness problem. In: Sattar A, Kang BH (eds) Lecture notes in artificial intelligence (LNAI), vol 4304. Springer, Berlin, pp 760–769
138. Parsopoulos KE, Vrahatis MN (2007) Parameter selection and adaptation in unified particle swarm optimization. Math Comput Model 46(1–2):198–213
139. Parsopoulos KE, Vrahatis MN (2008) Multi-objective particles swarm optimization approaches. In: Bui LT, Alam S (eds) Multi-objective optimization in computational intelligence: theory and practice. Premier reference source, chapter 2. Information Science Reference (IGI Global), Hershey, pp 20–42
140. Parsopoulos KE, Vrahatis MN (2010) Particle swarm optimization and intelligence: advances and applications. Information Science Reference (IGI Global), USA
141. Pawan Y-N, Prakash K-B, Chowdhury S, Chen Y-C (2022) Particle swarm optimization performance improvement using deep learning techniques. Multimedia Tools Appl 81(19):27949–27968
142. Petalas YG, Parsopoulos KE, Vrahatis MN (2007) Entropy-based memetic particle swarm optimization for computing periodic orbits of nonlinear mappings. In: IEEE 2007 congress on evolutionary computation (IEEE CEC 2007), Singapore. IEEE Press, pp 2040–2047
143. Petalas YG, Parsopoulos KE, Vrahatis MN (2007) Memetic particle swarm optimization. Ann Oper Res 156(1):99–127
144. Petalas YG, Parsopoulos KE, Vrahatis MN (2009) Improving fuzzy cognitive maps learning through memetic particle swarm optimization. Soft Comput 13(1):77–94
145. Piperagkas GS, Georgoulas G, Parsopoulos KE, Stylios CD, Likas AC (2012) Integrating particle swarm optimization with reinforcement learning in noisy problems. In: Genetic and evolutionary computation conference 2012 (GECCO 2012), Philadelphia. ACM, pp 65–72
146. Piperagkas GS, Konstantaras I, Skouri K, Parsopoulos KE (2012) Solving the stochastic dynamic lot-sizing problem through nature-inspired heuristics. Comput Oper Res 39(7):1555–1565
147. Poli R (2008) Analysis of the publications on the applications of particle swarm optimisation. J Artif Evol Appl, 2018: 685175
148. Poli R (2008) Dynamic and stability of the sampling distribution of particle swarm optimisers via moment analysis. J Artif Evol Appl 2008: 761459
149. Poli R, Kennedy J, Blackwell T (2007) Particle swarm optimization. Swarm Intell 1(1):33–57
150. Poli R, Langdon WB (2007) Markov chain models of bare-bones particle swarm optimizers, pp 142–149
151. Pookpunt S, Ongsakul W (2013) Optimal placement of wind turbines within wind farm using binary particle swarm optimization with time-varying acceleration coefficients. Renew Energy 55:266–276
152. Potter MA, De Jong K (2000) Cooperative coevolution: an architecture for evolving coadapted subcomponents. Evol Comput 8(1):1–29

153. Qu BYAB, Liang JJA, Suganthan PNB (2012) Niching particle swarm optimization with local search for multi-modal optimization. Inf Sci 197:131–143
154. Rada-Vilela J, Johnston M, Zhang M (2014) Population statistics for particle swarm optimization: resampling methods in noisy optimization problems. Swarm Evol Comput 17:37–59
155. Rahnamayan RS, Tizhoosh HR, Salama MMA (2008) Opposition-based differential evolution. IEEE Trans Evol Comput 12(1):64–79
156. Rathore A, Sharma H (2017) Review on inertia weight strategies for particle swarm optimization. In: Proceedings of the sixth international conference on soft computing for problem solving: SocProS 2016, vol 2. Springer, pp 76–86
157. Rauf H-T, Bangyal W-H, Ahmad J, Bangyal S-A (2018) Training of artificial neural network using PSO with novel initialization technique. In: 2018 international conference on innovation and intelligence for informatics, computing, and technologies (3ICT), pp 1–8
158. Reyes-Sierra M, Coello Coello CA (2006) Multi-objective particle swarm optimizers: a survey of the state-of-the-art. Int J Comput Intell Res 2(3):287–308
159. Rezaee Jordehi A, Jasni J (2013) Parameter selection in particle swarm optimisation: a survey. J Exp Theor Artif Intell 25(4):527–542
160. Rini DP, Shamsuddin SM, Yuhaniz SS (2014) Particle swarm optimization: technique, system and challenges. Int J Comput Appl 14(1):19–27
161. Schmitt M, Wanka R (2014) Particle swarm optimization almost surely finds local optima. Theor Comput Sci, in press
162. Schoeman IL, Engelbrecht AP (2010) A novel particle swarm niching technique based on extensive vector operations. Nat Comput 9(3):683–701
163. Schwefel H-P (1995) Evolution and optimum seeking. Wiley, New York
164. Sedighizadeh D, Masehian E (2009) Particle swarm optimization methods, taxonomy and applications. Int J Comput Theory Eng 1(5):486–502
165. Shahzad F, Masood S, Khan N-K (2014) Probabilistic opposition-based particle swarm optimization with velocity clamping. Knowl Inf Syst 39:703–737
166. Shami T-M, El-Saleh A-A, Alswaitti M, Al-Tashi Q, Summakieh M-A, Mirjalili S (2022) Particle swarm optimization: a comprehensive survey. IEEE Access 10:10031–10061
167. Sharifi A, Kordestani J-K, Mahdaviani M, Meybodi M-R (2015) A novel hybrid adaptive collaborative approach based on particle swarm optimization and local search for dynamic optimization problems. Appl Soft Comput 32:432–448
168. Shi Y, Eberhart RC (1998) A modified particle swarm optimizer. In: Proceedings IEEE conference on evolutionary computation, anchorage. IEEE Service Center, pp 69–73
169. Silva Filho T-M, Pimentel B-A, Souza R-M, Oliveira A-L (2015) Hybrid methods for fuzzy clustering based on fuzzy c-means and improved particle swarm optimization. Expert Syst Appl 42(17–18):6315–6328
170. Skokos C, Parsopoulos KE, Patsis PA, Vrahatis MN (2005) Particle swarm optimization: an efficient method for tracing periodic orbits in 3D galactic potentials. Mon Not R Astron Soc 359:251–260
171. Smairi N, Siarry P, Ghedira K (2016) A hybrid particle swarm approach based on Tribes and tabu search for multi-objective optimization. Optim Methods Softw 31(1):204–231
172. Soleimani H, Kannan G (2015) A hybrid particle swarm optimization and genetic algorithm for closed-loop supply chain network design in large-scale networks. Appl Math Model 39(14):3990–4012
173. Souravlias D, Parsopoulos KE (2015) Particle swarm optimization with neighborhood-based budget allocation. Int J Mach Learn Cybern, in press
174. Storn R, Price K (1997) Differential evolution-a simple and efficient heuristic for global optimization over continuous spaces. J Glob Optim 11:341–359
175. Suganthan PN (1999) Particle swarm optimizer with neighborhood operator. In: Proceedings of IEEE congress on evolutionary computation, Washington, DC, pp 1958–1961

176. Sun C, Jin Y, Cheng R, Ding J, Zeng J (2017) Surrogate-assisted cooperative swarm optimization of high-dimensional expensive problems. IEEE Trans Evol Comput 21(4): 644–660
177. Sun J, Feng B, Xu W (2004) Particle swarm optimization with particles having quantum behavior. In: Proceedings of IEEE congress on evolutionary computation 2004 (IEEE CEC'04), Portland, pp 325–331
178. Sun J, Lai C-H, Wu X-J (2011) Particle swarm optimisation: classical and quantum perspectives. CRC Press
179. Sun J, Xu W, Feng B (2004) A global search strategy for quantum-behaved particle swarm optimization. In: Proceedings of 2004 IEEE conference on cybernetics and intelligent systems, Singapore, pp 111–116
180. Sun S, Li J (2014) A two-swarm cooperative particle swarms optimization. Swarm Evol Comput 15:1–18
181. Sutton AM, Whitley D, Lunacek M, Howe A (2006) PSO and multi-funnel landscapes: How cooperation might limit exploration. In: Proceedings of the 8th annual conference on genetic and evolutionary computation (GECCO'06), Seattle, pp 75–82
182. Taghiyeh S, Xu J (2016) A new particle swarm optimization algorithm for noisy optimization problems. Swarm Intell 10:161–192
183. Taherkhani M, Safabakhsh R (2016) A novel stability-based adaptive inertia weight for particle swarm optimization. Appl Soft Comput 38:281–295
184. Tasgetiren F, Chen A, Gencyilmaz G, Gattoufi S (2009) Smallest position value approach. Stud Comput Intell 175:121–138
185. Tasgetiren MF, Liang Y-C, Sevkli M, Gencyilmaz G (2006) Particle swarm optimization and differential evolution for the single machine total weighted tardiness problem. Int J Prod Res 44(22):4737–4754
186. Tasgetiren MF, Liang Y-C, Sevkli M, Gencyilmaz G (2007) A particle swarm optimization algorithm for makespan and total flowtime minimization in the permutation flowshop sequencing problem. Eur J Oper Res 177(3):1930–1947
187. Thangaraj R, Pant M, Abraham A, Bouvry P (2011) Particle swarm optimization: Hybridization perspectives and experimental illustrations. Appl Math Comput 217(12):5208–5226
188. Too J, Sadiq A-S, Mirjalili S-M (2022) A conditional opposition-based particle swarm optimisation for feature selection. Connect Sci 34(1):339–361
189. Törn A, Žilinskas A (1989) Global optimization. Springer, Berlin
190. Trelea IC (2003) The particle swarm optimization algorithm: Convergence analysis and parameter selection. Inf Process Lett 85:317–325
191. Tsai H-C (2010) Predicting strengths of concrete-type specimens using hybrid multilayer perceptrons with center-unified particle swarm optimization. Expert Syst Appl 37(2): 1104–1112
192. Vafashoar R, Meybodi M-R (2016) Multi swarm bare bones particle swarm optimization with distribution adaption. Appl Soft Comput 47:534–552
193. Van den Bergh F, Engelbrecht AP (2002) A new locally convergent particle swarm optimiser. vol 3, pp 94–99
194. Van den Bergh F, Engelbrecht AP (2004) A cooperative approach to particle swarm optimization. IEEE Trans Evol Comput 8(3):225–239
195. Van den Bergh F, Engelbrecht AP (2006) A study of particle swarm optimization particle trajectories. Inf Sci 176:937–971
196. Voglis C, Parsopoulos KE, Lagaris IE (2012) Particle swarm optimization with deliberate loss of information. Soft Comput 16(8):1373–1392
197. Voglis C, Parsopoulos KE, Papageorgiou DG, Lagaris IE, Vrahatis MN (2012) MEMPSODE: a global optimization software based on hybridization of population-based algorithms and local searches. Comput Phys Commun 183(5):1139–1154

198. Wang B, Xue B, Zhang M (2020) Particle Swarm optimisation for evolving deep neural networks for image classification by evolving and stacking transferable blocks. In: 2020 IEEE congress on evolutionary computation (CEC), pp 1–8
199. Wang D, Tan D, Liu L (2018) Particle swarm optimization algorithm: an overview. Soft Comput 22:387–408
200. Wang D, Qiu H, Wu C-C, Lin W-C, Lai K, Cheng S-R (2019) Dominance rule and opposition-based particle swarm optimization for two-stage assembly scheduling with time cumulated learning effect. Soft Comput 23:9617–9628
201. Wang H, Moon I, Yang S, Wang D (2012) A memetic particle swarm optimization algorithm for multimodal optimization problems. Inf Sci 197:38–52
202. Wang H, Wu Z, Rahnamayan S, Liu Y, Ventresca M (2011) Enhancing particle swarm optimization using generalized opposition-based learning. Inf Sci 181(20):4699–4714
203. Wang H, Zhao X, Wang K, Xia K, Tu X (2014) Cooperative velocity updating model based particle swarm optimization. Appl Intell 40(2):322–342
204. Wang Y-J, Zhang J-S (2008) A new constructing auxiliary function method for global optimization. Math Comput Model 47(11–12):1396–1410
205. Wu H, Geng J, Jin R, Qiu J, Liu W, Chen J, Liu S (2009) An improved comprehensive learning particle swarm optimization and its application to the semiautomatic design of antennas. IEEE Trans Antennas Propag 57(10 PART 2):3018–3028
206. Wu J, Long J, Liu M (2015) Evolving RBF neural networks for rainfall prediction using hybrid particle swarm optimization and genetic algorithm. Neurocomputing 148:136–142
207. Xianfeng Y, Li LS (2014) Dynamic adjustment strategies of inertia weight in particle swarm optimization algorithm. Int J Control Autom 7(5):353–364
208. Xin-gang Z, Ze-qi Z, Yi-min X, Jin M (2020) Economic-environmental dispatch of microgrid based on improved quantum particle swarm optimization. Energy 195:117014
209. Xu W, Duan BY, Li P, Hu N, Qiu Y (2014) Multiobjective particle swarm optimization of boresight error and transmission loss for airborne radomes. IEEE Trans Antennas Propag 62(11):5880–5885
210. Xue B, Zhang M, Browne WN (2014) Particle swarm optimisation for feature selection in classification: novel initialisation and updating mechanisms. Appl Soft Comput J 18:261–276
211. Yang J, Zhang H, Ling Y, Pan C, Sun W (2014) Task allocation for wireless sensor network using modified binary particle swarm optimization. IEEE Sens J 14(3):882–892
212. Yang J-M, Chen Y-P, Horng J-T, Kao C-Y (1997) Applying family competition to evolution strategies for constrained optimization. In: Lecture notes in mathematics, vol 1213. Springer, Berlin, pp 201–211
213. Yang L, Shami A (2020) On hyperparameter optimization of machine learning algorithms: theory and practice. Neurocomputing 415:295–316
214. Yen GG, Leong WF (2009) Dynamic multiple swarms in multiobjective particle swarm optimization. IEEE Trans Syst Man Cybern Part A:Syst Hum 39(4):890–911
215. Yu X, Zhang X (2014) Enhanced comprehensive learning particle swarm optimization. Appl Math Comput 242:265–276
216. Zambrano-Bigiarini M, Clerc M, Rojas R (2013) Standard particle swarm optimisation 2011 at CEC-2013: a baseline for future PSO improvements, pp 2337–2344
217. Zhang K, Huang Q, Zhang Y (2019) Enhancing comprehensive learning particle swarm optimization with local optima topology. Inf Sci 471:1–18
218. Zhang Q, Wang Z, Tao F, Sarker BR, Cheng L (2014) Design of optimal attack-angle for RLV reentry based on quantum particle swarm optimization. Adv Mech Eng 2014, 352983
219. Zhang Y, Gong D, Hu Y, Zhang W (2015) Feature selection algorithm based on bare bones particle swarm optimization. Neurocomputing 148:150–157
220. Zhang Y, Gong D-W, Ding Z (2012) A bare-bones multi-objective particle swarm optimization algorithm for environmental/economic dispatch. Inf Sci 192:213–227

221. Zhang Y, Gong D-W, Sun X-Y, Geng N (2014) Adaptive bare-bones particle swarm optimization algorithm and its convergence analysis. Soft Comput 18(7):1337–1352
222. Zhao F, Li G, Yang C, Abraham A, Liu H (2014) A human-computer cooperative particle swarm optimization based immune algorithm for layout design. Neurocomputing 132:68–78
223. Zhao J, Lv L, Fan T, Wang H, Li C, Fu P (2014) Particle swarm optimization using elite opposition-based learning and application in wireless sensor network. Sens Lett 12(2): 404–408
224. Zheng Y-J, Ling H-F, Xue J-Y, Chen S-Y (2014) Population classification in fire evacuation: a multiobjective particle swarm optimization approach. IEEE Trans Evol Comput 18(1):70–81

POPMUSIC

29

Éric D. Taillard and Stefan Voß

Contents

Introduction . . . 904
POPMUSIC Template . . . 904
POPMUSIC Adaptation for Location Routing . . . 907
  Initial Solution . . . 907
  Part and Subproblem Definitions . . . 909
  Seed-Part Selection . . . 909
  Empirical Complexity . . . 909
POPMUSIC Applications . . . 910
  Application of POPMUSIC as Matheuristic for the CVRP . . . 910
  Application of POPMUSIC to the TSP . . . 911
  Application of POPMUSIC to Clustering . . . 911
  Application of POPMUSIC to Max Weight Stable Set Problems . . . 912
  Application of POPMUSIC to Berth Allocation Problems . . . 913
Related Approaches . . . 915
Conclusion and Future Works . . . 917
Cross-References . . . 917
References . . . 917

Abstract

This chapter presents POPMUSIC, a general decomposition-based framework within the realm of matheuristics that has been successfully applied to various combinatorial optimization problems. POPMUSIC is especially useful

É. D. Taillard (✉)
HEIG-VD, University of Applied Sciences and Arts of Western Switzerland, Yverdon-les-Bains, Switzerland
e-mail: eric.taillard@heig-vd.ch

S. Voß
Faculty of Business Administration, Institute of Information Systems, University of Hamburg, Hamburg, Germany
e-mail: stefan.voss@uni-hamburg.de

© Springer Nature Switzerland AG 2025

R. Martí et al. (eds.), *Handbook of Heuristics*,
https://doi.org/10.1007/978-3-032-00385-0_31

for designing heuristic methods for large combinatorial problems that can be partially optimized. The basic idea is to optimize subparts of solutions until a local optimum is reached. Implementations of the technique to various problems show its broad applicability and efficiency for tackling especially large-size instances.

Keywords

Decomposition method · Matheuristics · Large neighborhood search · Large scale optimization

Introduction

A natural way to solve large optimization problems is to decompose them into independent subproblems that are solved with an appropriate method. However, such approaches may lead to solutions of moderate quality since the subproblems might have been created in a somewhat arbitrary fashion. While it is not necessarily easy to find an appropriate way to decompose a problem a priori, the basic idea of POPMUSIC is to locally optimize subparts of a solution, a posteriori, once a solution to the problem is available. These local optimizations are repeated until a local optimum is found (and possibly beyond). POPMUSIC stands for "Partial OPtimization Metaheuristic Under Special Intensification Conditions" [33]. It is a template, or framework, for creating heuristics repeating partial optimization on a solution. To apply this template, special conditions must be fulfilled. Among these conditions, it is supposed that a solution is composed of parts that can be optimized almost independently. Thus, POPMUSIC may be seen as a "general purpose" framework which does not need too many problem-specific adaptations if a reasonable way of decomposing a problem, or a given incumbent solution, is at hand so that it can be readily applied to a wide spectrum of problem classes.

POPMUSIC typically applies to large-size problem instances, but the template was also applied successfully to smaller instances. Typical real-life problems usually contain a very large number of elements. The POPMUSIC template proposes a solution for dealing with problem instances of that size.

Next we describe the POPMUSIC template followed by a survey of several successful applications as well as an overview on a few related approaches.

POPMUSIC Template

The idea of decomposing problems of large size into smaller subproblems easier to solve is certainly very old [8]. In the context of metaheuristics and matheuristics, this can be accomplished, for instance, by using given incumbent solutions and

optimizing parts of them. In general terms, one may distinguish soft fixing and hard fixing to attain problems of reduced size that may be solved separately. While in *hard fixing* some of the decision variables of a given problem are directly excluded (e.g., by fixing them to a certain value), *soft fixing* refers to adding, say linear, constraints to the model to cut out all solutions from a problem that are beyond a certain distance from a given solution. In that way, the search can be intensified around those parts of a solution space that are not cut out.

To exemplify the POPMUSIC template in detail, we resort to the capacitated vehicle-routing problem (CVRP) as one of the first specific applications of the template [29]. We emphasize this as POPMUSIC can be well illustrated on the CVRP. In its simplest version, it consists of finding a set of vehicle tours, starting from a depot, servicing customers asking given quantities of goods and coming back to the depot. Knowing the vehicle capacity and all distances between clients, a set of tours, visiting each customer exactly once, is searched in such a way that the total distance of the tours is minimized.

If a solution is available, a tour can be considered as a part of the solution. Each tour can be interpreted as being independent from the others and can be optimized independently. Optimizing a tour consists of finding an optimal trip (or tour) of a traveling salesman problem (TSP). Since the number of customers on a tour of a CVRP is limited and exact codes for the traveling salesman problem are available and efficient up to few dozens of cities and beyond [9], the optimization of single CVRP tours can be considered as a simple task.

Now, if we consider the subset of customers delivered by a subset of tours of a given solution, it is also possible to optimize independently the delivery of this subset of customers. So, a subset of parts of a solution creates a subproblem that can be optimized independently from the remaining of the problem. For a CVRP, such a subproblem is a CVRP of reduced size.

If an optimization method is available that can treat CVRPs with up to r tours efficiently, larger instances can be tackled by repeating optimizations on subproblems containing up to r tours. The POPMUSIC template suggests a tactic for building subproblems that can be potentially improved. Instead of generating any subproblem with r parts, as done by matheuristics like large neighborhood search (LNS), POPMUSIC introduces a notion of proximity between parts. Subproblems are built by first selecting a part called a *seed part* and r of its closest parts. Figure 1 illustrates the principle of the generation of a subproblem containing six tours.

Given a solution composed of p parts, there are only p subproblems composed of $r < p$ parts that can be so built, instead of the $\frac{p!}{(p-r)!\cdot r!}$ possible ones. In contrast to the LNS template where the user must choose at each step how to build a subproblem and when to stop the optimization process, POPMUSIC-based approaches have a natural stopping criterion: When the p subproblems are solved optimally or with satisfaction, the process can be stopped.

To be specific, in the POPMUSIC template, it is supposed that a solution s can be decomposed into p parts $s_1, \ldots, s_p$ and that a proximity measure between two parts has been defined. Let U with $|U| =: p$ be the set of parts that have not been used as seed part for building a subproblem. Initially, U contains all p parts and U

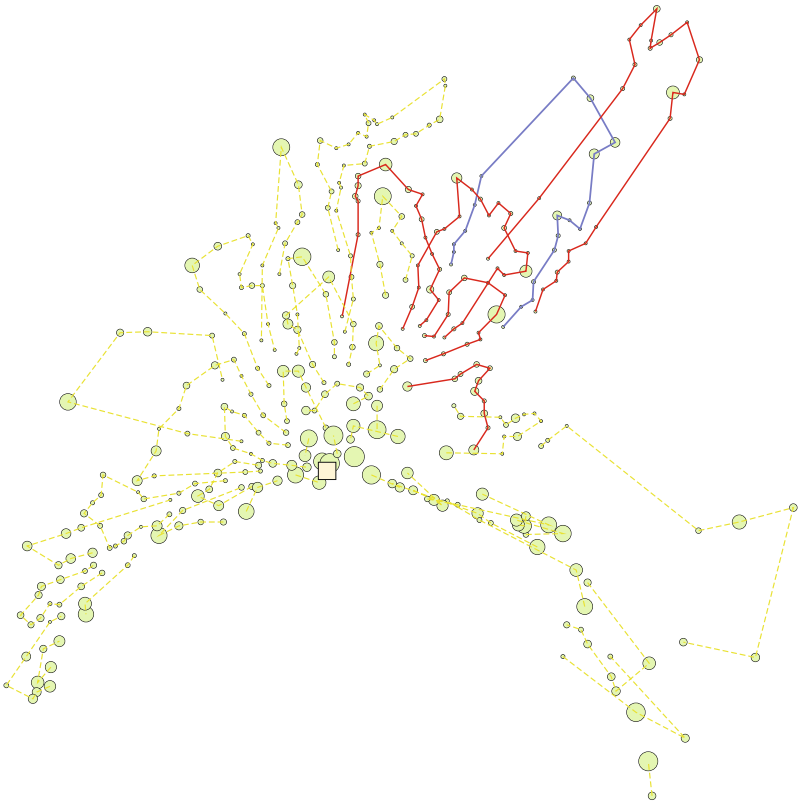


Fig. 1 For the vehicle-routing problem, the definition of a part can be a vehicle tour. In this figure, the size of the disks represents the customers' demand, and the trips from and to the depot (square) are not drawn. Here, the proximity between parts is measured by the distance of their center of gravity. A subproblem is a smaller VRP composed of the customers serviced by six tours

is empty at the end. Let $m(R)$ be an optimization method that allows to optimize a subproblem R composed of r parts. With these hypothesis, the POPMUSIC template can be expressed by Algorithm 1.

POPMUSIC can be seen from different perspectives: As mentioned above, it can be considered as a large neighborhood search [26] or as a soft fixing approach. In this case, the neighborhood considered consists of finding the best way to reshape the elements of r parts. The neighborhood is qualified as large, in contrast to local search that modifies only few elements of a solution at each step, allowing explicitly a complete examination of the neighborhood. When all the elements of r parts must be optimized, it is not possible to proceed to an explicit examination of the whole neighborhood, since its size is much too large. So, either an exact method is used, making POPMUSIC a matheuristic (see [22]), or an approximate algorithm is used, typically based on a metaheuristic.

Algorithm 1: POPMUSIC template

Input: Initial solution s composed of p parts $s_1, \ldots, s_p$, sub-problem optimization method m
Output: Improved solution s
$U = \{s_1, \ldots, s_p\}$
while $U \neq \emptyset$ **do**
    Select $s_g \in U$ `//` s_g`: seed-part`
    Build sub-problem R with the r parts of s that are the closest to s_g
    Optimize R with method m
    if R *improved* **then**
        Update solution s
        Remove from U the parts that do not belong to s anymore
        Insert in U the parts of optimized sub-problem R
    else `//` R `not improved`
        Remove s_g of U
    end
end

Deriving a specific POPMUSIC implementation is described by means of a specific application in the next section.

POPMUSIC Adaptation for Location Routing

The POPMUSIC template requires the programmer to define five components: the *initial solution*, a *part*, the *proximity* between parts, the *seed-part* choice, and an *optimization method*. This section illustrates how these components can be implemented for a location-routing problem (LRP). Very briefly, an LRP is a vehicle-routing problem with several depots where the location of the depots must be decided in addition to the building of vehicle tours.

Initial Solution

A key point for the treatment of large problem instances is to analyze the algorithmic complexity of each step of the solution method. Examining the POPMUSIC template, the first question is about the number of times the `while` loop at line 1 is repeated. Numerical experiments on different problems—unsupervised clustering [30], map labeling [3], location routing [4], berth allocation [17, 20]—have shown that this loop is repeated a number of times that grows quasi-linearly with the problem size. This behavior is typical for local searches based on standard neighborhoods where the theoretical exponential complexity is not observed. To lower the algorithmic complexity, the other steps of the template must be carefully designed.

The step that looks most complex is naturally the optimization of subproblems at line 1. Indeed, the computational time for optimizing a subproblem with r parts grows very fast with r, for instance, exponentially if an exact method is used.

The user can control the computational time by modifying the unique parameter r of POPMUSIC template. Let us suppose that the user has fixed the value of r to a value for which the subproblems are sufficiently large for getting solutions of good quality while limiting the computational time of the optimization method. If r is fixed, solving a subproblem takes a constant time—maybe large, but constant. So, if the use of an efficient optimization method is a key point of the POPMUSIC template, this is not an issue for limiting the algorithmic complexity.

The choice of the seed part can be implemented in constant time, for instance, by making a random choice or by storing set U as a stack or a queue. So, the key points for limiting the algorithmic complexity of a POPMUSIC implementation is to use an appropriate technique for producing the initial solution and for building subproblems from a seed part. Indeed, without an appropriate data structure, building a subproblem can take a time proportional to the problem size, leading to a global complexity being at least quadratic.

Alvim and Taillard [4] have presented how to build in $O(n^{3/2})$ a solution to a location-routing problem while building a data structure allowing to build a subproblem in $O(r)$. The idea of the construction is the following: Let us suppose that it is possible to advise a distance measure between the entities of the problem. For the LRP, this distance is part of the problem data: It is simply the distance between customers. The entities of the problem are grouped into $\sqrt{n}$ clusters by means of a heuristic method for the p-median problem. Although NP-hard, the p-median problem can be approximately solved with an empirical complexity of $\overline{O}((p \cdot n + (\frac{n}{p})^2)$, where p is the number of clusters desired. In this problem, the clusters are built by choosing p central elements and assigning all remaining elements to the closest center. By choosing $p = \sqrt{n}$, it is possible to cut a problem of size n into $\sqrt{n}$ clusters containing approximately $\sqrt{n}$ elements in $\overline{O}((n^{3/2})$. This method is illustrated in Fig. 2.

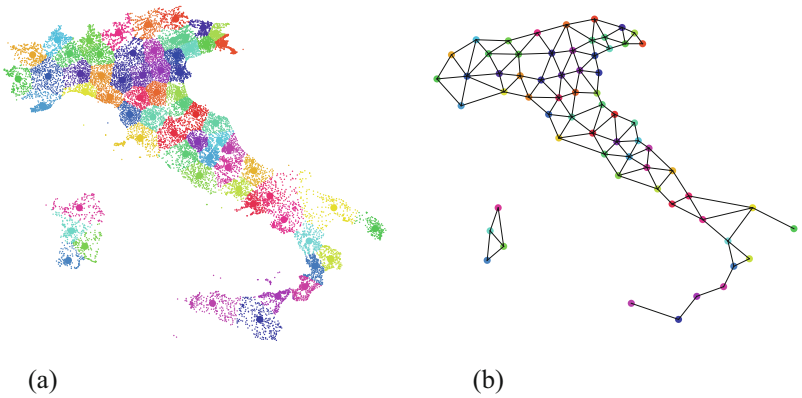


Fig. 2 Illustration of the decomposition of a problem into $\sqrt{n}$ clusters and building a proximity network between the clusters. (**a**) Problem decomposition into clusters. (**b**) Building proximity network

Then, all centers can be compared 2 by 2 to create proximity relations. Figure 2b illustrates the proximity relation obtained by considering that clusters a and b are neighbors if an entity of one of these clusters has the center of the other one as second closest center. Each cluster can then be decomposed into $O(\sqrt{n})$ clusters to create a total of $O(n)$ small clusters containing a constant number of elements. So, it is possible to create proximity relations between every entity of a problem of size n in $O(n^{3/2})$. In the context of the LRP, a vehicle tour can be assigned to the entities of a small cluster. The proximity relations can be exploited in POPMUSIC for finding efficiently the r parts that are the closest from a seed part.

However, if no versatile method for finding incumbent solutions is at hand, any type of heuristic or even a randomized approach may be used to derive them.

Part and Subproblem Definitions

The definition of a part strongly depends on the problem under consideration and on the procedure available for optimizing subproblems. For vehicle-routing problems, [4, 24, 29] have defined the customers serviced by a vehicle tour as a part. This definition is justified by the fact that the subproblems built around a seed part are smaller VRPs or multi-depot VRPs that can be efficiently optimized with a metaheuristic-based algorithm, like a basic tabu search or genetic algorithm hybridized with a variable neighborhood search. Several definitions have been adopted for the proximity between parts: For Euclidean instances, the distance between the center of gravity of tours can be used; generally, the technique presented in section "Initial Solution" can be used for defining the proximity.

Seed-Part Selection

Very few works have studied the impact of the seed-part selection procedure. In many applications, subproblems are generated around a seed part arbitrarily chosen in U (next available, random one, ...). Preliminary results in [4] have shown that a set U managed as a stack (last part entered in U is selected) seems to be better than an arbitrary selection.

Empirical Complexity

In [4], problem instances 10000 times larger than those usually treated in the literature have been tackled. As mentioned above, the most complex part is the generation of an initial solution in $O(n^{3/2})$. The subproblem optimization is less complex, almost linear. Figure 3 shows the evolution of the computational effort as a function of problem size for different POPMUSIC phases. The increase of computational time is higher for the steps of generating the initial solution (solving a p-median problem with two levels, as presented in section "Initial Solution") than

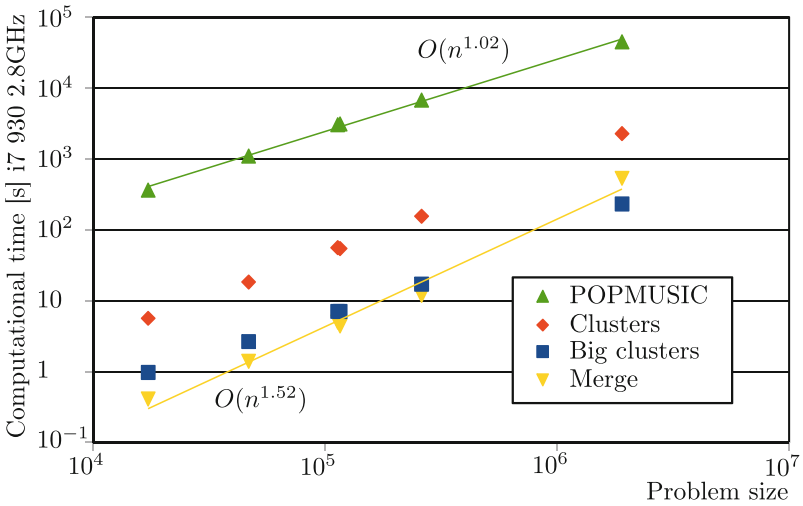


Fig. 3 Evolution of the computational effort as a function of problem size for various parts of the algorithm: building large clusters, decomposing large clusters into small ones containing subsets of customers that can be delivered by the same vehicle, finding positions for the depots and optimizing subproblems

for the subproblem optimization. However, this last phase still takes most of the computational effort, even with instances with two millions of entities.

POPMUSIC Applications

While we have used the CVRP and a location-routing problem to clarify concepts, in this section, we summarize a few additional successful applications of POPMUSIC. Other applications not specifically mentioned refer, e.g., to the turbine runner balancing problem [33] or the p-cable trench problem [19].

Application of POPMUSIC as Matheuristic for the CVRP

Recently, [27] precisely adapted POPMUSIC for the capacitated vehicle-routing problem (CVRP) using a branch-cut-and-price algorithm, which combines column and cut generation with several additional mechanisms to optimize the subproblems. These authors propose a variant for constructing subproblems:

Each customer takes turns playing the role of a seed. Then, all other customers are listed in ascending order based on their distance from the seed customer. As long as the subproblem contains a number of customers not exceeding a threshold α, the route containing the next closest customer to the seed is added to the subproblem—provided it is not already included. Thus, the parameter r, which fixes the number

of routes, is replaced by α, which sets the maximum number of customers in a subproblem.

To avoid unnecessary optimization attempts, the set of previously optimized subproblems is stored. Additionally, a variant of the POPMUSIC framework is proposed in this chapter, with the idea of increasing α by a quantity δ once each customer has played the role of a seed.

POPMUSIC has also been adapted to a variant of the CVRP, namely the multi-depot cumulative capacitated vehicle-routing problem, by Lalla-Ruiz and Voß [18].

Application of POPMUSIC to the TSP

A very simple way to apply the POPMUSIC framework to the traveling salesman problem (TSP) is to consider each city as a part and measure proximity between two cities by the number of edges that separate them in the tour to be optimized.

With this definition, a subproblem is a path that can be optimized while keeping its two endpoint cities fixed. This approach was studied in [32].

This proximity definition allows for the re-optimization of a tour with a linear empirical computational effort. However, its drawback is that it cannot "untangle" crossings between edges separated by a number of cities greater than the subproblem size.

The biggest challenge in applying POPMUSIC to the TSP is, therefore, obtaining an initial solution that lends itself well to further optimization. Taillard [31] proposed an $O(n \log n)$ approach that generates solutions for gigantic instances (with billions of cities). While the solution quality is relatively modest, it is significantly better than that of the nearest-neighbor heuristic, for example.

The main advantage of this method is its ability to quickly identify a very limited subset of edges that include those of excellent solutions, even for non-Euclidean instances. This property is leveraged in the latest versions of the LKH software [14, 15], which are currently among the best freely available solvers for this class of problems.

Application of POPMUSIC to Clustering

The POPMUSIC template is particularly well adapted to solve large clustering problem instances. The goal of clustering is to create groups of entities as well separated as possible while containing entities as homogeneous as possible. This means that metrics are available for measuring the similarity or the dissimilarity between elements. If the number of clusters is relatively large, using the POPMUSIC template is interesting. In this case, a part of a solution can be constituted as the set of entities belonging to the same cluster. The measure of the separation between groups can be used for defining the proximity between parts.

This technique was successfully applied to unsupervised clustering with centroids (sum-of-squares clustering, p-median, multisource Weber) in [30]. Figure 4 illustrates the construction of a subproblem from a solution to a p-median problem.

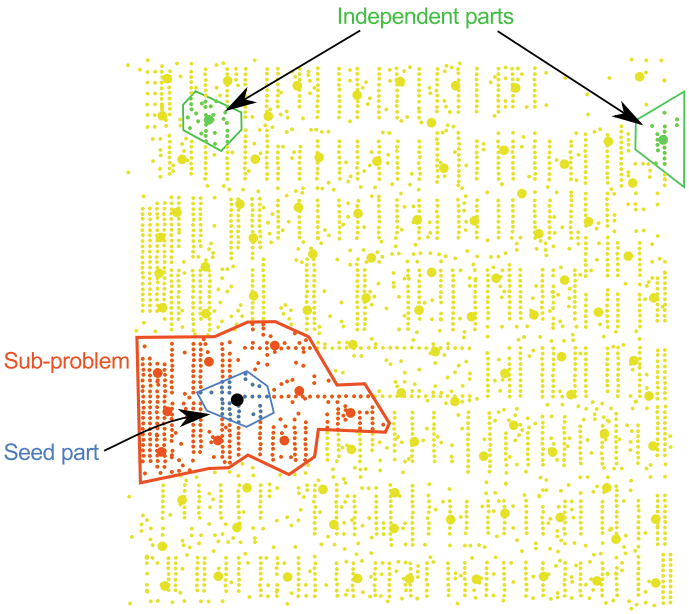


Fig. 4 Building a subproblem from a seed part in the case of a p-median problem. Clusters that are well separated can be considered as independent and should not be involved in the same subproblem

Application of POPMUSIC to Max Weight Stable Set Problems

The POPMUSIC template is also well adapted to the search of a stable set with maximum weight in a graph. Several practical problems can be modeled under this form. For instance, the map labeling problem can be formulated as a stable set with maximum weight. The problem is the following: Given objects on the Euclidean plane, one wants to place a label around them to identify them. For instance, when drawing a map, an object can be the top of a mountain and the label the name of the mountain. The label can be placed at several predefined positions. The problem consists of finding a position for each label in such a way that no label partially covers another one. If it is not possible to label all objects without superposition, it is searched to maximize the number of labels correctly placed. For typographic reasons, a different weight is associated to each label position, reflecting the positioning preferences or the importance of the object.

The problem can be formulated as the search of a stable set with maximum weight in a graph. The graph is built as follows: A node is associated with every label position, with a weight corresponding to the preference. An edge connects two nodes if the corresponding labels are incompatible: Either they are associated with the same object (every object must be labeled at most once) or they overlap. Figure 5 illustrates the construction of a graph for labeling three objects with four possible label positions for each of them.

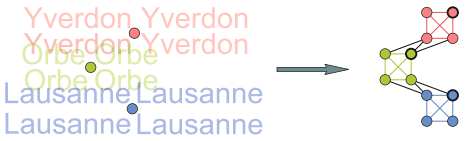


Fig. 5 Formulation of the problem of map labeling under a stable set

Applying the POPMUSIC template to this problem can be done as follows: A part is an object to label, so the set of all nodes representing the different possible label positions for this object. The distance between two objects (interpreted as parts) is the minimum number of edges needed to connect two nodes associated to labels of the objects they identify. A subproblem is constituted by r objects for which the label position can be modified. A subproblem must also consider other labels of the current solution that may overlap, but without searching to modify the position of these labels.

The optimization procedure is a re-implementation of a tabu search due to [35] for which the parameters were tuned for solving at best problem instances with few dozens of objects. Alvim and Taillard [2] have shown that excellent solutions can be obtained for problem instances with one thousand objects and that the method can still be used for instances with millions of objects.

This work was continued by Laurent et al. [21] for allowing the labeling of cartographic maps containing non-punctual objects like lines (streets, rivers) or polygons (states, countries). The subproblem optimization used in this reference is based on ejection chains. The algorithms developed by Laurent et al. [21] have been integrated in the QGIS open-source software (http://www.qgis.org/en/site/).

Various problems in the domain of transportation can be formulated as a map labeling (or a maximum weight stable set). The assignment of cruising levels of commercial aircraft is one of them. Knowing the departure hour and horizontal trajectory of every aircraft at a continent level, it is required to assign each of them a cruising level in order to avoid the collision between aircraft. The label shape is determined in this case by the possible position of the aircraft for a given time interval. The exact position of the aircraft cannot be determined, since the departure hour may fluctuate (traffic delay in the airport) and the weather conditions may affect the speed and position of the aircraft (wind, clouds, storm, ...). For a time interval, it is required to find a level for each label in such a way that no label covers another one.

Application of POPMUSIC to Berth Allocation Problems

The berth allocation problem (BAP) is a well-known optimization problem within maritime shipping. It aims at assigning and scheduling incoming vessels to berthing positions along the quay of a container terminal. Lalla-Ruiz and Voß[17] propose two POPMUSIC approaches that incorporate an existing mathematical program-

ming formulation based on modeling the problem as a set partitioning problem. The computational experiments reveal state-of-the-art results outperforming all previous approaches regarding solution quality.

To be specific, [17] study the application of POPMUSIC for solving the discrete dynamic berth allocation problem (DBAP). In the DBAP, one is given a set of incoming container vessels, N, and a set of berths, M. Each container vessel, $i \in N$, must be assigned to an empty berth, $k \in M$, within its time window, $[t_i, t_i']$, and the assigned berth time window, $[s_k, e_k]$. A simplified assumption is that each berth can handle at most one vessel at a time. For each container vessel, $i \in N$, its handling time, ρ_{ik}, depends on the berth $k \in M$ where it is assigned to. That is, the service time of a given vessel differs from one berth to another. Moreover, some vessels may have forbidden berths in order to model water-depth or maintenance constraints. Finally, each vessel $i \in N$ has a given service priority, denoted as v_i, according to its contractual agreement with the terminal.

A natural way to define parts is by means of the berths themselves. If a restricted number of berths (and assigned vessels) can be handled with an exact approach, this allows for efficient subproblem solving. Moreover, it is also easy to define the proximity between berths, taking real distances.

An interesting observation deduced from solving the DBAP and extensions by means of POPMUSIC also reveals some general lessons learned regarding the definition of parts. Let us assume a problem extension as the berth allocation problem under time-dependent limitations (BAP-TL) [20]. In this problem, in order to assign a vessel to a berth, terminal managers might have to take into account not only the berthing place and vessel draft, but also the arrival and berthing time of an incoming vessel while observing changing environments due to tidal changes. Note that in the DBAP the decisions about to which berth the vessel should be assigned to is relevant since there are different handling times depending on the berth. This is relaxed in the BAP-TL, i.e., all the berths provide the same handling time, having also an important implication on the proper definition of parts. Note also that in the BAP-TL the berthing time is important due to the tidal constraints. Thus, it makes sense to define the parts not necessarily following the berths delimitation, but defining the parts as intervals of time. Numerical results are given in [20].

To end this subsection, we should point to an interesting analogy which may be drawn between the pure (and possibly simplified) berth allocation problem and the map labeling problem. In this case, the width of a label corresponds to the time interval during which the vessel must be at the berth for being loaded/unloaded. The height of the label is the vessel length. A label can be placed at different positions along the berth. Figure 6 provides an example of the transformation of a small problem with four vessels arriving at different times and that can be placed at few different positions along a berth (considering vessel length and time windows, TW).

The POPMUSIC template was used in [17, 20] for the dynamic berth allocation problem without and with the consideration of tidal constraints. In these references, the optimization procedure is an exact method.

The use of the POPMUSIC framework in multi-objective optimization for an integrated berth allocation and quay crane assignment problem is described in [36].

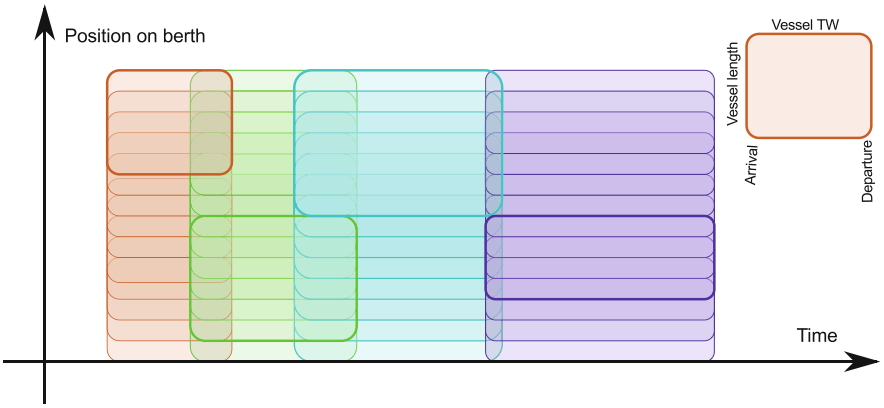


Fig. 6 Transformation of berth allocation problem to a map labeling

An integrative algorithm framework is developed, composed of POPMUSIC, the Strength Pareto Evolutionary Algorithm 2, and k-means clustering. An extension, again applying the POPMUSIC framework in multi-objective optimization, toward the adoption of different green technologies is described in [37].

Related Approaches

For large optimization problems, it is often possible to interpret a solution as composed of parts or chunks [34] as can also be found under the term vocabulary building. Suppose that a solution can be represented as a set of parts as seen above, some parts are more in relation with some other parts so that a corresponding heuristic measure can be defined between two parts.

The *corridor method* (CM) has been presented by Sniedovich and Voß [28] as a hybrid metaheuristic, linking mathematical programming techniques with heuristic schemes. The basic idea of the CM relies on the use of an exact method over restricted portions of the solution space of a given problem. Given an optimization problem, the basic ingredients of the method are a very large feasible space, and an exact method that could easily solve the problem if the feasible space were not large.

The basic concept of a *corridor* is introduced to delimit a portion of the solution space around the incumbent solution. The optimization method will then be applied within the neighborhood defined by the corridor with the aim of finding an improved solution. Consequently, the CM defines method-based neighborhoods, in which a neighborhood is built taking into account the method used to explore it.

As mentioned before, POPMUSIC may be seen as a soft fixing approach as does the CM. Food for thought might be considered putting POPMUSIC into perspective regarding other methods like large neighborhood search, adaptive randomized decomposition, kernel search, etc. An older research which may serve as motivation

relates to solving the job shop problem by means of the shifting bottleneck heuristic [1]. The idea of developing heuristics identifying a small/moderate size subset of variables in order to intensify the search in a promising region of the solution space has been used in other contexts. In the knapsack problem family, for instance, [7] propose the idea of selecting a small subset of items (called the core) and solving exactly a restricted problem on that subset. The use of an expanding method to modify the size of the core during the algorithm execution is proposed by Pisinger [25]. A heuristic framework called kernel search has been proposed for the solution of mixed-integer linear problems in [5, 6]. The kernel search framework is based on the idea of exhaustively exploring promising portions of the solution space. Kernel search is similar to POPMUSIC since "buckets," analogous to parts, are defined at the beginning of the search. During the execution, the algorithm revises the definition of the core problem, called "kernel," by adding/removing buckets to/from the current problem. Once the current kernel is defined, an exact method is applied to the restricted problem.

Local branching [10] is another soft fixing technique, in which the introduction of linear constraints is used to cut solutions from the feasible space. The feasible space of the constrained problem includes only solutions that are not too far away from the incumbent. When presented, local branching was used within a branch and bound framework. A generalized local branching concept has been proposed by Hill and Voß [16].

Variable neighborhood decomposition search is in line with the strategy employed by POPMUSIC. Neighborhoods around an incumbent are defined using a distance metric and explored via any method. If a local optimum better than the incumbent is found in the current neighborhood, the neighborhood is re-centered around the new incumbent, and the mechanism moves on to the exploration of the new neighborhood. If no improvement occurs, the search moves to the next neighborhood defined around the same incumbent. An interesting modification of variable neighborhood decomposition is provided in [13], where variable neighborhood search is coupled with local branching. Neighborhoods are defined imposing linear constraints on a mixed-integer linear programming model, as done in local branching, and then explored using a general-purpose solver as black box.

Fischetti et al. [11] propose a similar method called diversification, refining, and tight refining (DRT). It is aimed at solving problems with two-level variables, in which fixing the value of the first-level variables leads to an easier to solve, but still hard, subproblem. Finally, another way to see POPMUSIC is to exploit the proximate optimality principle (POP, see, e.g., [12]). In tabu search, the POP notion is exploited by performing a number of iterations at a given level before restoring the best solution found to initiate the search at the next level. In that context, a level corresponds to the optimization of a subproblem in the POPMUSIC terminology.

POPMUSIC is also applied toward various combinatorial optimization problems as part of an integrated approach combining various metaheuristics. As an example, we mention [23] for a matheuristic approach hybridizing particle swarm optimization, simulated annealing, and mathematical programming based on the POPMUSIC concept.

Conclusion and Future Works

The main strength of POPMUSIC is its simplicity, the fact that it has a unique parameter and its ability to solve large problem instances. So, the main effort when implementing a POPMUSIC-based procedure is not devoted to parameter tuning as it can be the case for other metaheuristics. But there is no free lunch. The weakness of POPMUSIC is that an initial solution with a structure adapted to the template must be available as well as an optimization procedure for the subproblems. These points make POPMUSIC less general and more problem-specific than other metaheuristics. Adapting POPMUSIC to various hard problems which are not easy to decompose is a first research avenue.

Up to now, very few works have been devoted to study the influence of POPMUSIC options, such as the way the seed part is chosen or the procedure used for optimizing subproblems. These are other promising research topics for the future. It can also be interesting to revisit existing methods that are somewhat based on decomposition principles under the form of the POPMUSIC template. This could lead to simplifications and/or efficiency improvements.

The POPMUSIC template can be adapted to parallel implementations. Our first works on the vehicle-routing problem were primarily devoted to parallel implementations of metaheuristics. A relatively easy issue is to use the few cores of CPU in parallel. However, implementing a POPMUSIC with a number of processors dependent on problem size is less trivial. Another research avenue is to see how to exploit the large number of processors of graphic process units in the context of POPMUSIC.

Cross-References

▸ Iterated Local Search
▸ Iterated Greedy
▸ Tabu Search

Competing Interest Declaration The author(s) has no competing interests to declare that are relevant to the content of this manuscript.

References

1. Adams J, Balas E, Zawack D (1988) The shifting bottleneck procedure for job shop scheduling. Manag Sci 34:391–401
2. Alvim ACF, Taillard ÉD (2007) An efficient POPMUSIC based approach to the point feature label placement problem. In: Metaheuristic international conference (MIC'07) proceedings
3. Alvim ACF, Taillard ÉD (2009) POPMUSIC for the point feature label placement problem. Eur J Oper Res 192(2):396–413. https://doi.org/10.1016/j.ejor.2007.10.002
4. Alvim ACF, Taillard ÉD (2013) POPMUSIC for the world location routing problem. EURO J Transport Logist 2:231–254

5. Angelelli E, Mansini R, Speranza MG (2010) Kernel search: a general heuristic for the multi-dimensional knapsack problem. Comput Oper Res 37(11):2017–2026
6. Angelelli E, Mansini R, Speranza MG (2012) Kernel search: a new heuristic framework for portfolio selection. Comput Optim Appl 51(1):345–361
7. Balas E, Zemel E (1980) An algorithm for large zero-one knapsack problems. Oper Res 28(5):1130–1154
8. Ball MO (2011) Heuristics based on mathematical programming. Surveys Oper Res Manag Sci 16(1):21–38. https://doi.org/10.1016/j.sorms.2010.07.001
9. Concorde (2015) Concorde TSP solver. http://www.math.uwaterloo.ca/tsp/concorde/index.html
10. Fischetti M, Lodi A (2003) Local branching. Math Programm B 98:23–47
11. Fischetti M, Polo C, Scantamburlo M (2004) A local branching heuristic for mixed-integer programs with 2-level variables, with an application to a telecommunication network design problem. Networks 44(2):61–72
12. Glover F, Laguna M (1997) Tabu search. Kluwer, Dordrecht
13. Hansen P, Mladenović N, Urosević D (2006) Variable neighborhood search and local branching. Comput Oper Res 33(10):3034–3045
14. Helsgaun K (2022). Helsgaun's implementation of Lin-Kernighan. http://webhotel4.ruc.dk/~keld/research/LKH/. Version LKH-2.0.10
15. Helsgaun K (2024) Extension of Helsgaun's implementation of Lin-Kernighan for constrained TSP and VRPs'. http://webhotel4.ruc.dk/~keld/research/LKH-3/. Version LKH-3.0.10
16. Hill A, Voß S (2018) Generalized local branching heuristics and the capacitated ring tree problem. Discrete Appl Math 242:34–52. https://doi.org/10.1016/j.dam.2017.09.010
17. Lalla-Ruiz E, Voß S (2016) POPMUSIC as a matheuristic for the berth allocation problem. Ann Math Artif Intell 76:173–189
18. Lalla-Ruiz E, Voß S (2020) A POPMUSIC approach for the multi-depot cumulative capacitated vehicle routing problem. Optim Lett 14(3):671–691. https://doi.org/10.1007/s11590-018-1376-1
19. Lalla-Ruiz E, Schwarze S, Voß S (2016) A matheuristic approach for the p-cable trench problem. Lect Notes Comput Sci 10079:247–252
20. Lalla-Ruiz E, Voß S, Exposito-Izquierdo C, Melian-Batista B, Moreno-Vega JM (2017) A POPMUSIC-based approach for the berth allocation problem under time-dependent limitations. Ann Oper Res 253:871–897
21. Laurent M, Taillard ÉD, Ertz O, Grin F, Rappo D, Roh S (2009) From point feature label placement to map labelling. In: Metaheuristic international conference (MIC'09) Proceedings
22. Maniezzo V, Stützle T, Voß S (eds) (2009) Matheuristics: hybridizing metaheuristics and mathematical programming. Springer, Berlin
23. Nourmohammadzadeh A, Voß S (2023) An effective matheuristic approach for robust bus driver rostering with uncertain daily working hours. Lect Notes Comput Sci 14239:365–380
24. Ostertag A, Doerner KF, Hartl RF, Taillard ÉD, Waelti P (2009) POPMUSIC for a real-world large-scale vehicle routing problem with time windows. J Oper Res Soc 60(7):934–943. https://doi.org/10.1057/palgrave.jors.2602633
25. Pisinger D (1999) Core problems in knapsack algorithms. Oper Res 47(4):570–575
26. Pisinger D, Ropke S (2007) A general heuristic for vehicle routing problems. Comput Oper Res 34(8):2403–2435. https://doi.org/10.1016/j.cor.2005.09.012
27. Queiroga E, Sadykov R, Uchoa E (2021) A POPMUSIC matheuristic for the capacitated vehicle routing problem. Comput Oper Res 136:105475. https://doi.org/10.1016/j.cor.2021.105475
28. Sniedovich M, Voß S (2006) The corridor method: a dynamic programming inspired metaheuristic. Control Cyber 35:551–578
29. Taillard ÉD (1993) Parallel iterative search methods for vehicle routing problems. Networks 23(8):661–673. https://doi.org/10.1002/net.3230230804
30. Taillard ÉD (2003) Heuristic methods for large centroid clustering problems. J Heurist 9(1):51–73

31. Taillard ÉD (2022) A linearithmic heuristic for the travelling salesman problem. Eur J Oper Res 297(2):442–450. https://doi.org/10.1016/j.ejor.2021.05.034
32. Taillard ÉD, Helsgaun K (2019) POPMUSIC for the travelling salesman problem. EURO J Oper Res 272(2):420–429. https://doi.org/10.1016/j.ejor.2018.06.039
33. Taillard E, Voß S (2002) POPMUSIC — partial optimization metaheuristic under special intensification conditions. In: Ribeiro CC, Hansen P (eds) Essays and surveys in metaheuristics. Kluwer, Boston, pp 613–629. https://doi.org/10.1007/978-1-4615-1507-4_27
34. Woodruff DL (1998) Proposals for chunking and tabu search. Eur J Oper Res 106:585–598
35. Yamamoto M, Camara G, Lorena LAN (2002) Tabu search heuristic for point-feature cartographic label placement. GeoInformatica 6:77–90
36. Yu J, Voß S, Song X (2022) Multi-objective optimization of daily use of shore side electricity integrated with quayside operation. J Cleaner Prod 351:131406. https://doi.org/10.1016/j.jclepro.2022.131406
37. Yu J, Tang G, Voß S, Song X (2023) Berth allocation and quay crane assignment considering the adoption of different green technologies. Transport Res Part E Logist Transport Rev 176:103185. https://doi.org/10.1016/j.tre.2023.103185

Random-Key Genetic Algorithms: Principles and Applications

30

Mariana A. Londe, Luciana S. Pessoa, Carlos E. Andrade, José Fernando Gonçalves, and Mauricio G. C. Resende

Contents

Introduction . . . 922
Biased Random-Key Genetic Algorithms . . . 923
A Model for the Implementation of a BRKGA . . . 924
Optional Procedures of the Framework . . . 926
Hybridizing a Random-Key Genetic Algorithm . . . 928
Customizing a RKGA . . . 928
Tuning an RKGA . . . 929
APIs for BRKGA . . . 931
Conclusion . . . 932
Cross-References . . . 932
References . . . 933

Abstract

A random-key genetic algorithm is an evolutionary metaheuristic for discrete and global optimization. Each solution is encoded as a vector of n random keys, where a random key is a real number randomly generated in the continuous

M. A. Londe · L. S. Pessoa
Pontifical Catholic University of Rio de Janeiro, Rio de Janeiro, RJ, Brazil
e-mail: mlonde@aluno.puc-rio.br; lucianapessoa@puc-rio.br

C. E. Andrade
AT&T Labs Research, Bedminster, NJ, USA
e-mail: cea@research.att.com

J. F. Gonçalves
Faculdade de Economia da, Universidade do Porto, Porto, Portugal

M. G. C. Resende (✉)
Industrial and Systems Engineering, University of Washington, Seattle, WA, USA
e-mail: mgcr@berkeley.edu

© Springer Nature Switzerland AG 2025

R. Martí et al. (eds.), *Handbook of Heuristics*,
https://doi.org/10.1007/978-3-032-00385-0_30

interval [0, 1). A decoder maps each vector of random keys to a solution of the optimization problem being solved and computes its cost. The benefit of this approach is that all genetic operators and transformations can be maintained within the unitary hypercube, regardless of the problem being addressed. This enhances the productivity and maintainability of the core framework. The algorithm starts with a population of p vectors of random keys. At each iteration, the vectors are partitioned into two sets: a smaller set of high-valued elite solutions and the remaining non-elite solutions. All elite elements are copied, without change, to the next population. A small number of random-key vectors (the mutants) are added to the population of the next iteration. The remaining elements of the population of the next iteration are generated by combining, with the parametrized uniform crossover of Spears and DeJong (On the virtues of parameterized uniform crossover. In: Proceedings of the Fourth International Conference on Genetic Algorithms, pp 230–236, 1991), pairs of solutions. This chapter reviews random-key genetic algorithms and describes an effective variant called biased random-key genetic algorithms.

Keywords

Genetic algorithm · Random keys · Optimization · Random-key genetic algorithm · Biased random-key genetic algorithm · Metaheuristic · Parameter tuning

Introduction

Genetic algorithms (GAs) are inspired by Darwin's theory of evolution. Holland [39] introduces this approach, which considers that a possible solution to a combinatorial optimization problem is an individual inside a population of potential solutions. In this population, evolutionary operators are applied at each iteration/generation to obtain new populations and solutions.

A class of GAs is described by Bean [9], whose solutions are represented by a vector of random keys, called a chromosome. A *random key* is a real number belonging to the interval [0, 1). This makes the evolutionary process of the *random-key genetic algorithms* (RKGAs) problem-independent, as the evolutionary operators can be specified for the chromosomes, which allows for a generalization of the framework. The quality of a solution is calculated by the *decoder* procedure, which maps a chromosome to a fitness value. In [9], the decoding procedure simply orders the elements of the vector of random keys, which produces a permutation of the indices of the sorted elements.

In the RKGA evolutionary process, the algorithm starts with an initial population $\mathcal{P}$ of vectors of n random keys. In each generation, the vectors are partitioned into a set of the best solutions $\mathcal{P}^e < |\mathcal{P}|/2$, called the elite set, and a set with the remainder of non-elite solutions of the population. All elite solutions are copied, unchanged, into the population of the next generation. This elitism characterizes the Darwinian

principle in the RKGA. Next, a set $\mathcal{P}^m$ of randomly created vectors is introduced into the next population. This set comprises *mutants*, which play the same role as the mutation operators of classical GAs, i.e., they help increase population diversity and avoid convergence to non-global local optimum. To complete the elements of the next population, $|\mathcal{P}| - |\mathcal{P}^e| - |\mathcal{P}^m|$ are generated by combining solutions from the entire population using a parametrized uniform crossover [107, 108]. In this crossover, consider a and b to be the vectors chosen for mating, and let c be the offspring produced. Random key $c[i]$, the i-th component of the offspring vector, receives the i-th key of one of its parents. It receives the key $a[i]$ with probability ρ_a and $b[i]$ with probability $\rho_b = 1 - \rho_a$.

Biased Random-Key Genetic Algorithms

A *biased random-key genetic algorithm* (BRKGA), meanwhile, also uses an elitist strategy in the mating procedure, not only regarding the reproduction of the elite set.

In BRKGA, one parent is always chosen from the elite set. When performing the coin toss to choose whose parent passes the gene to the offspring, the gene of the elite parent has a probability of being inherited $\rho > 0.5$. This means that the offspring has a higher chance of inheriting the keys of the elite parent and of retaining substructures of a good solution while still allowing the insertion of substructures of a not-so-good solution. This difference between the two algorithms tends to result in BRKGA outperforming Bean's algorithm [38]. As a result, most of the literature published after the introduction of BRKGA tends to use the biased variant. A flowchart detailing the BRKGA framework can be seen in Fig. 1. In it, the problem-independent structure of classical BRKGA is highlighted. The decoding procedure in black is problem-dependent, as it makes the correlation between a random-key vector and a feasible problem solution.

The double elitism of BRKGA leads to rapid convergence to high-quality solutions. Alongside its problem-agnostic approach, it has led to a significant increase in use, with over 250 papers published until 2025 [56, 57]. The BRKGA was formally introduced in Gonçalves and Resende [36], but some of its characteristics can be seen in the early 2000s [15, 27, 35]. Since then, several applications, hybridizations, and features have been described in the literature. Among the problems in which BRKGA was applied since 2020, the following are considered:

- *Scheduling*: Abreu et al. [1], Abreu et al. [2], Bolsi et al. [11], Fontes and Homayouni [31], Fontes et al. [32], Huang et al. [42], Ibarra-Rojas et al. [44], Kong et al. [48], Kong et al. [49], Kummer et al. [50], Liu et al. [52], Maecker et al. [61], Queiroga et al. [87], Rocholl and Mönch [94], Silva-Soto and Ibarra-Rojas [103], Soares and Carvalho [105], Soares and Carvalho [104], Xie et al. [116], Yang et al. [118], Yu et al. [119], Zhang et al. [120], and Zhao et al. [121]
- *Network configuration*: Mikulski and Tomczewski [68], Pinto et al. [85], Raposo et al. [89], and Usberti et al. [111]

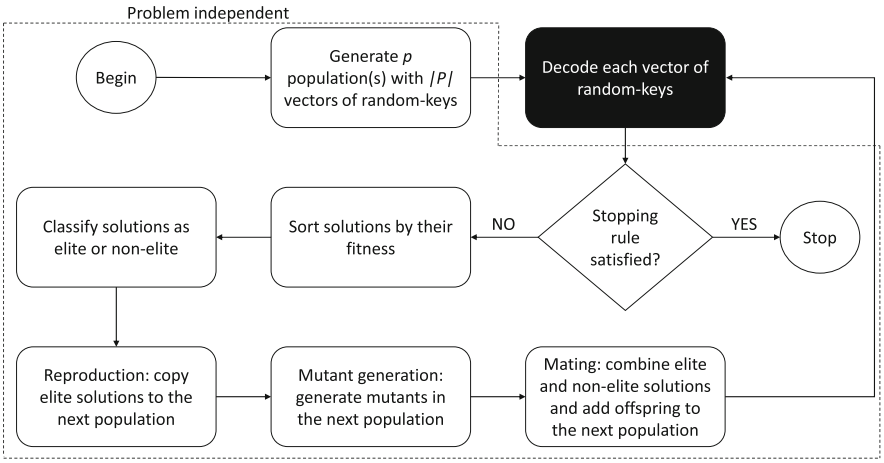


Fig. 1 Flowchart detailing BRKGA process (Adapted from Gonçalves and Resende [36])

- *Location*: Freitas et al. [33], Johnson et al. [46], Londe et al. [53], Mauri et al. [65], Mendoza-Andrade et al. [67], and Villicaña-Cervantes and Ibarra-Rojas [113]
- *Cutting and packing*: Amaro Júnior et al. [3], Gonçalves and Wäscher [37], Lu et al. [60], Mandal et al. [62], Oliveira et al. [75], Oliveira and Romero [71], Souza Queiroz and Andretta [106], and Zhou et al. [122]
- *Traveling salesman and vehicle routing*: Amaro Júnior et al. [4], Carrabs [16], Chagas et al. [17], Chagas et al. [18], Chaves et al. [21], Damm et al. [25], Huang et al. [40], Huang et al. [41], Ibarra-Rojas and Silva-Soto [43], Marques Jr et al. [63], Oliveira et al. [77], Santiyuda et al. [95], Schenekemberg et al. [96], Schenekemberg et al. [97], and Schuetz et al. [98]
- *Clustering*: Brito et al. [13], Fadel et al. [28], Martarelli and Nagano [64], and Xu et al. [117]
- *Graph problems*: Lima et al. [51], Londe et al. [54], Londe et al. [55], Melo et al. [66], Pinto et al. [86], Silva et al. [102], and Silva et al. [101]
- *Automatic tuning of parameters in heuristics*: Cicek and Ozturk [22], Falls et al. [29], Japa et al. [45], and Sun and Zhou [109]
- *Other applications*: Morgan et al. [69], Ochoa et al. [70], Pan et al. [79], Pastore et al. [81], Pinacho-Davidson and Blum [84], Ramos et al. [88], Reixach et al. [90], and Silva et al. [100]

A Model for the Implementation of a BRKGA

Algorithm 1 shows a pseudo-code of a BRKGA for the minimization of $f(x)$, where $x \in X$ and X is a discrete set of solutions and $f : X \to \mathbb{R}$. This implementation is a multi-start variant of a BRKGA where several populations are evolved in sequence,

Algorithm 1: Model for biased random-key genetic algorithm with restart

```
1  BRKGA(|P|, |P_e|, |P_m|, n, ρ_a)
2  Initialize value of the best solution found: f* ← ∞;
3  while stopping criterion not satisfied do
4      Generate a population P with |P| vectors of n random keys;
5      while restart criterion not satisfied do
6          Evaluate the cost of each new solution in P;
7          Partition P into two sets: P_e and P_ē;
8          Initialize population of next generation: P+ ← P_e;
9          Generate set P_m of mutants, each mutant with n random keys;
10         Add P_m to population of next generation: P+ ← P+ ∪ P_m;
11         foreach i ← 1 to |P| − |P_e| − |P_m| do
12             Select parent a at random from P_e;
13             Select parent b at random from P_ē;
14             foreach j ← 1 to n do
15                 Throw a biased coin with probability ρ > 0.5 of resulting heads;
16                 if heads then c[j] ← a[j];
17                 else c[j] ← b[j];
18             end
19             Add offspring c to population of next generation: P+ ← P+ ∪ {c};
20         end
21         Update population: P ← P+;
22         Find best solution χ+ in P: χ+ ← argmin{f(χ) | χ ∈ P};
23         if f(χ+) < f* then
24             χ* ← χ+;
25             f* ← f(χ+);
26         end
27     end
28 end
29 return χ*
```

and the best solution among all in the population is returned as the output of the algorithm.

In line 2, the value f^* of the best solution found is initialized to a large value, i.e., not smaller than $f(x^0)$, where $x^0 \in X$ is some feasible solution to the problem. Evolution of each population is done in lines 3–28. The algorithm halts when a stopping criterion in line 3 is satisfied. This criterion can be, for example, the number of evolved populations, the total time, or the quality of the best solution found.

In line 4, the population being evolved is initialized with $|\mathcal{P}|$ vectors of random keys. Evolution of population $\mathcal{P}$ takes place in lines 5–27. This evolution ends when a restart criterion is satisfied in line 5. This criterion can be, for example, a maximum number of generations without improvement in the value of the best solution in $\mathcal{P}$. At each generation, or iteration, the following operations are carried out: In line 6, all new solutions (offspring and mutants) are decoded and their costs evaluated. Note that each decoding and evaluation in this step can be computed simultaneously, i.e., in parallel. In line 7, population $\mathcal{P}$ is partitioned into two subpopulations $\mathcal{P}_e$

(elite) and $\mathcal{P}_{\bar{e}}$ (non-elite), where $\mathcal{P}_e$ is such that $|\mathcal{P}_e| < |\mathcal{P}|/2$ and contains $|\mathcal{P}_e|$ of the best solutions in $\mathcal{P}$ and $\mathcal{P}_{\bar{e}}$ consists of the remaining solutions in $\mathcal{P}$, that is, $\mathcal{P}_{\bar{e}} = \mathcal{P} \setminus \mathcal{P}_e$. $\mathcal{P}^+$ is the population of the next generation. It is initialized in line 8 with the elite solutions of the current generation. In line 9, the mutant subpopulation $\mathcal{P}_m$ is generated. Each mutant is a vector of n random keys. The number of generated mutants in general is such that $|\mathcal{P}_m| < |\mathcal{P}|/2$. This subpopulation is added to the population $\mathcal{P}^+$ of the next generation in line 10.

With $|\mathcal{P}_e| + |\mathcal{P}_m|$ vectors inserted in population $\mathcal{P}^+$, it is necessary to generate $|\mathcal{P}| - |\mathcal{P}_2| - |\mathcal{P}_m|$ new offspring to complete the $|\mathcal{P}|$ vectors that form population $\mathcal{P}^+$. This is done in lines 11–20. In lines 12 and 13 parents a and b are chosen, respectively, at random from subpopulations $\mathcal{P}_e$ and $\mathcal{P}_{\bar{e}}$. The generation of offspring c from parents a and b occurs in lines 14–18. A biased coin (with probability $\rho > 0.5$ of flipping to heads) is thrown n times. If the i-th toss is a head, the offspring inherits the i-th key of parent a. Otherwise, it inherits the i-th key of parent b. After the offspring is generated, c is added to population $\mathcal{P}^+$ in line 19.

The generation of $\mathcal{P}^+$ ends when it consists of $|\mathcal{P}|$ elements. In line 21, $\mathcal{P}^+$ is copied to $\mathcal{P}$ to start a new generation. The best solution in the current population in evolution is computed in line 22, and if its value is better than all solutions examined so far, the solution and its cost are saved in lines 24 and 25 as χ^* and f^*, respectively. χ^*, the best solution found across all populations, is returned by the algorithm in line 29.

Optional Procedures of the Framework

There are several ways to modify the BRKGA algorithm in order to increase solution quality, convergence, diversity, and other criteria. One of these is the use of the restart. In BRKGA, the restart or reset of the population is the elimination of all current individuals and the recreation of the entire population with random chromosomes. This is done, generally, after I_r number of iterations without improvement in the best overall solution $\chi*$, similar to the restart strategy for GRASP with path-relinking [91], has been shown to increase individual diversity and prevent premature convergence [80, 110].

A partial restart is also possible. The shaking method [6] is akin to a perturbation of the Iterated Local Search (ILS) algorithm [59] and acts by modifying a part of the genes of the elite solutions, while the non-elite chromosomes are completely restarted. That means some of the structures gained in the evolutionary process are preserved in the elite set, while the complete population has guaranteed diversity. The shake operator is usually applied after I_s iterations without improvement in the best overall solution $\chi*$ and, like the ILS perturbation, has strength parameters SK_l, indicating the lower bound, and SK_u, corresponding to the upper bound.

Another common feature is the island model [115]. Inspired by divergent evolution, the island model is the simultaneous evolution of p parallel populations.

The idea is that parallel evolution will lead to different characteristics in the solutions, thereby enabling a more extensive exploration of the solution space. Every g generations, i elite individuals are exchanged among populations, which increases diversity and helps spread novel and interesting structures among the solutions.

Many authors also attempt to introduce interesting structures into the initial population by injecting warm start solutions. This addition is shown to increase the performance and convergence of the method. In general, one or more good solutions are created with other heuristics [1, 4, 18, 54, 64, 100, 116] or mathematical models [17, 98] and then inserted into the initial population. In some cases, the entire initial population can be created using alternative methods, as is done in [120], where the solutions have specific characteristics that are not easily obtained in randomly generated individuals.

One more recent addition to the BRKGA framework is the use of multi-parent crossover [7]. In it, several parents are considered in the mating process, and the choice of inheritance is done using a ranking function. Consider the following example: there are three total parents, denoted as a, b, and c. Of those, both a and b belong to the elite set, while c belongs to the non-elite set. For multi-parent crossover, those parents would be ranked according to their fitness values, so that the first in order would have better fitness than the following ones, and so forth. In this case, considering a minimization problem, observe that $f(a) < f(b) < f(c)$. The ranking of solutions would, thus, be $r(a) = 1$, $r(b) = 2$, and $r(c) = 3$. With these values, the procedure uses a bias function $\Phi(r)$ [12] to calculate the probabilities of each parent being chosen in the mating. The bias function is nonincreasing, so parents with better fitness have a higher chance of being chosen and can belong to several classes, such as exponential, linear, logarithmic, and others.

One last optional procedure is Implicit Path-Relinking [7]. The classical path-linking is an intensification strategy that explores the neighborhood in the path between two distinct solutions [34, 92]. Typically, path-relinking is problem-dependent, due to the exploration of the intermediate solutions. However, the implicit variant leverages the random-key structure of BRKGA to perform the procedure within the random-key solution space, with differences depending on the type typ of the decoding process. This procedure works by first selecting two solutions from different populations using method sel, which can be the best or randomly chosen among the elite individuals. The selected individuals must have a minimum distance of md, which is calculated by either the Hamming (direct IPR) or Kendall Tau (permutation IPR) distances. A percentage $cp\%$ of the total pairs of elite chromosomes are tested as candidates from IPR, and if no pairs have the minimum distance, then the IPR procedure is not executed. This limitation of candidate pair testing is done to decrease the computational burden of the algorithm. Two other parameters are used in IPR. Parameter $ps\%$ is the size of the path between the two candidates that is evaluated at each IPR iteration, given as a percentage of the chromosome size. In each test, a block of genes of size bs is modified in the base solution to become similar to the one in the guide solution, thus obtaining intermediate solutions that are then explored.

Hybridizing a Random-Key Genetic Algorithm

To increase performance and/or adapt the algorithm to problems with specific characteristics, the authors often incorporate additional methods or frameworks to work in conjunction with BRKGA. This practice is very common for BRKGA, especially when the addition is a local search.

The use of local search (LS) in a genetic framework is viewed as an application of Lamarck's theory of evolution, in which individuals pass on the characteristics they acquire during their lifetime to their descendants [23]. It is particularly common for BRKGA, as it has been shown to significantly increase solution quality and convergence [56]. The use of local search can be within [54, 61] or outside the decoding process, with the latter case applied to all chromosomes in every iteration [14,100], only the best incumbent solution after evolution is complete [18,65], or to special chromosomes in some generations [2, 16, 45, 53]. Established methods can also be used as LS, such as Variable Neighborhood Search [21, 60, 96, 104, 105], Clustering Search [19], ILS [99], IGA [1], and INLS [117].

When not performed as a local search, hybridization is typically used to adapt the framework to specific problem characteristics. Some examples are:

- *Multi-objective optimization*: Bierbüße et al. [10], Bolsi et al. [11], Chagas et al. [18], Damm et al. [25], Fontes and Homayouni [31], Fontes et al. [32], Ibarra-Rojas et al. [44], Liu et al. [52], Marques Jr et al. [63], Mikulski and Tomczewski [68], Ramos et al. [88], Raposo et al. [89], Santiyuda et al. [95], and Silva-Soto and Ibarra-Rojas [103]
- *Matheuristics*: Lima et al. [51], Pinacho-Davidson and Blum [84], Pinto et al. [86], and Sun and Zhou [109]
- *Machine learning*: Amaro Júnior et al. [4], Damm et al. [25], Pan et al. [79], Schenekemberg et al. [96], Schenekemberg et al. [97], Xie et al. [116], Xu et al. [117], and Yang et al. [118]
- *Stochastic optimization and scenario generation*: Huang et al. [42], Londe et al. [53], Oliveira et al. [73], Oliveira et al. [74], and Oliveira et al. [75]

Customizing a RKGA

To customize an RKGA, one needs to define how a solution is represented, or encoded, and how the corresponding decoding procedure is done. Also, it is necessary to choose and customize any hybrid methods, if applicable. After this, the algorithm's parameters must be tuned.

Since each solution is represented as a vector of n random keys, it is necessary only to specify a value for n for the encoding. It may represent the number of vertices in a graph, or the total number of jobs and pauses in parallel machines, or the possible locations for facility installation, or the number of parameters to be analyzed, for example.

The decoder is a deterministic procedure that takes as input the chromosome of n random keys and produces as output the solution of the problem alongside the corresponding fitness value. Three types of decoding procedures can be defined as follows: indicator, permutation, and a mix of both. In a permutation decoder, the solution is obtained by sorting the random keys and constructing a feasible solution with the permutation of the indices. Examples of problems that often have permutation-based decoders are Location [53, 65], Cutting and Packing [3, 71], Vehicle Routing [43, 96], Traveling Salesman Problem [4, 18], and Graph Problems [51, 101].

An indicator decoder, in contrast, obtains meaning directly from the values of the random keys. For example, consider that one wishes to assign each point, represented by one random key, to one of the four classes. In this case, the algorithm may consider that the interval [0, 1) is divided into four equal intervals of size 0.25, with the first class being indicated by values in the interval [0, 0.25), the second class by the interval [0.25, 0.50), and so forth. For solution c, if the i-th gene has value $0.25 \leq c[i] < 0.50$, then the corresponding i-th element belongs to the second class. The use of indicator decoders is common in Network Configuration [7, 8, 82, 89], Clustering [64, 72], and Parameter Optimization [22, 29, 45].

Sometimes, a problem has characteristics that make a pure indicator- or permutation-based representation inadequate. When this happens, the authors tend to mix those two, generally by partitioning the chromosome into two or more parts. One example of this practice is done in [113] to locate mobile labs, in which the first genes are sorted to obtain the order of service points, while the last genes indicate an increase of the area of service radius. Another example is [33] for a two-level hub location problem, where the random-key interval is divided by the number of desired hubs. For the genes in each interval, the values are sorted, with the order of the directed graph being indicated by the ordered genes and the first gene corresponding to the hub. In [5], a complex problem related to wireless network design is addressed using a five-part chromosome. The first two sections consist of random keys that indicate the pertaining and parameterization of the solution. The next two sections are used to establish the evaluation order of the deployed equipment and its relation. Lastly, the final section serves as indicators to determine the depths of the network trees.

Tuning an RKGA

Table 1 presents all the parameters that were commented on in the traditional BRKGA and the optional procedures, together with their recommended values from the current literature. One can note that if all procedures are desired, then almost 20 parameters need to be specified to run a BRKGA.

Due to the number of parameters and how much they affect performance, the authors have used many strategies for parameter configuration, either before or during the execution of experiments. When done before, this is called *offline tuning* and is widely used for metaheuristics [26].

Table 1 BRKGA parameters and recommended ranges

Parameter	Recommended value
$\|\mathcal{P}\|$: size of population	$\min\{[100, 500], \lfloor 10 \times n \rfloor\}$
$\mathcal{P}_e\%$: percentage of elite partition of population	[0.1, 0.5]
$\mathcal{P}_m\%$: percentage of mutant partition of population	[0.1, 0.5]
ρ: probability of inheriting key from elite parent	[0.5, 1]
p: number of parallel populations	$\{1, \ldots, 3\}$
g: frequency for population interchange	[50, 500]
i: number of elite migrants between parallel populations	{1, 2}
I_r: iterations without improvement for restart	[200, 500]
I_s: iterations without improvement for shaking	[20, 100]
SK_l: lower bound for shake intensity	[0.1, 0.5]
SK_u: upper bound for shake intensity	[0.5, 0.9]
π_t: total number of parents in multi-parent crossover	[3, 10]
π_e: total number of elite parents in multi-parent crossover	$[1, 7], \pi_e < \pi_t$
$\Phi(r)$: type of bias function for multi-parent crossover	$\{1/\log(r+1), r^{-1}, r^{-2}, r^{-3}, e^{-r}, 1/\pi_t\}$
sel: individual selection method for path-relinking	[*BESTSOLUTION*, *RANDOMELITE*]
$cp\%$: percentage of chromosome pairs to be tested for path-relinking	[50, 500]
md: minimum distance between chromosome pairs for path-relinking	[0.0, 0.30]
bs: block size for path-relinking	1.00
$ps\%$: path size for path-relinking	[0.01, 1]
I_{ipr}: frequency of path-relinking	[50, 500]

For BRKGA, the authors have used several offline strategies. One of the simplest methods is a grid search, also known as the full factorial design of experiments. Grid search evaluates all possible combinations of given discrete parameters. However, the complexity of the method increases with each parameter and each candidate range, making it costly to perform if there are many parameters with their own interactions and codependence [30]. Some studies that used grid search are [16, 24, 83, 93].

Other authors utilize parameters suggested by similar groups of problem instances in the literature. However, the performance of a particular algorithm with a specific configuration on some instances is of limited utility, and the generalization of configurations should be done cautiously [47]. The use of literature suggestions is done in [22, 74, 123].

Lastly, one may use an automated method of parameterization. For BRKGA, the authors seem to prefer Iterated F-Race [58], used recently in papers such as [5–8, 53–55, 65].

The parameters may also be tuned during the execution of the algorithm. This process, known as *online parameter control*, falls within the class of machine learning techniques and has been applied to BRKGA in two distinct approaches. In both cases, the parameters are updated at each algorithm generation, aiming to balance intensification and exploitation during evolution. In [4, 20, 99, 117], the authors use the Adaptive BRKGA (A-BRKGA), which updates parameters for population size, elite proportion, mutant proportion, crossover probability, and maximum number of generations at each iteration. It also utilizes two self-adaptive parameters, denoted as α and β, where α indicates the number of members from the elite restricted list that evolve, and β represents the probability of perturbing individuals that are too similar. At each generation, the values of population size, mutant proportion, and α decrease, while elite proportion increases. β and crossover probability are incorporated as genes into the chromosomes and are thus always modified randomly.

The other online approach is called BRKGA-QL [21, 96]. It uses a Q-Learning algorithm [114] to control the same parameters of A-BRKGA in an environment of actions, states, and rewards. Simply put, each possible parameter modification is mapped to a possible reward, and the algorithm chooses one of the actions, with an η probability of selecting the action with the highest reward in terms of intensification. The value of η decays during evolution, so that in later generations, actions that increase diversification tend to be chosen. BRKGA-QL was extended to work with multi-objective problems [25] and had a new reward scheme and diversity methods introduced in [97].

In [112], the author also studies online approaches for BRKGA. It is demonstrated that a random parameter strategy can yield better results compared to more elaborate schemes. This strategy defines upper and lower bounds for parameters $\mathcal{P}$, $\mathcal{P}^e$, $\mathcal{P}^m$, and ρ. Then, in each generation, the parameter values are randomly sampled inside uniform probability distributions between the specified bounds. This approach outperformed the common F-Race-tuned BRKGA in problems with simple decoding strategies.

APIs for BRKGA

To simplify the implementation of BRKGAs, Toso and Resende [110] proposed an *Application Programming Interface (API)* in C++ for BRKGA. The API is efficient and user-friendly. The library is portable and automatically handles several aspects of the BRKGA, including population management and evolutionary dynamics. The API is implemented in C++ and uses an object-oriented architecture. In systems with available *OpenMP* [78], the API enables parallel decoding of random key vectors. The user only needs to implement the decoder and specify the stopping criteria, restart and population exchange mechanisms, as well as the parameters of

the algorithm. The API is open source and can be downloaded from http://github.com/rfrancotoso/brkgaAPI.

Andrade et al. [7] modernized Toso and Resende [110] leveraging the BRKGA with multi-parent crossover and implicit path-relinking, alongside a multi-population strategy, warm start solution injection, restart, and partial restart features. This framework is also easy to use, as it automatically handles all of those features during the evolutionary process, with the user needing to define only the parameters, decoder, and instance class. The C++ version is available at https://github.com/ceandrade/brkga_mp_ipr_cpp, the Python version at https://github.com/ceandrade/brkga_mp_ipr_python, and the Julia package at https://github.com/ceandrade/BrkgaMpIpr.jl.

One last API, called BRKGACuda 2.0, is presented by Oliveira et al. [76]. This novel application is optimized to work with parallelism and GPUs, significantly accelerating the algorithm. The API provides all of the classical BRKGA evolutionary operators, with the user only needing to specify the decoder. This C++ algorithm can be found at https://github.com/discovery-unicamp/brkga-cuda.

Conclusion

This chapter reviewed random-key genetic algorithms, covering both the unbiased and biased variants. After introducing Bean's algorithm [9], this chapter highlights the minor differences between the RKGA and the BRKGA that contribute to the improved performance of the biased variant. A model for implementing the BRKGA is described, followed by a discussion of optional procedures within the framework. The use of hybrid methods, as well as the customization of the algorithm regarding decoding processes and tuning methodologies, is also detailed. Finally, this chapter concludes by presenting three APIs for BRKGA that allow for easy implementation of the algorithm.

Cross-References

- Biased Random-Key Genetic Programming
- Evolutionary Algorithms
- Genetic Algorithms
- Multi-start Methods
- Restart Strategies
- Theory of Local Search

Acknowledgments This work was supported by the Brazilian National Council for Scientific and Technological Development (CNPq) under Grant [number 312212/2021-6], Brazilian Coordination for the Improvement of Higher Level Personnel (CAPES) under Grants [numbers 001 and 88887.815411/2023], and the Carlos Chagas Filho Research Support Foundation of the State of Rio de Janeiro (FAPERJ) under Grants [numbers 211.086/2019, 211.389/2019, and 211.588/2021].

Competing Interest Declaration The author(s) has no competing interests to declare that are relevant to the content of this manuscript.

References

1. Abreu LR, Tavares-Neto RF, Nagano MS (2021) A new efficient biased random key genetic algorithm for open shop scheduling with routing by capacitated single vehicle and makespan minimization. Eng Appl Artif Intell 104:104373. https://doi.org/10.1016/j.engappai.2021.104373
2. Abreu LR, Athayde Prata B, Gomes AC, Braga-Santos SA, Nagano MS (2022) A novel BRKGA for the customer order scheduling with missing operations to minimize total tardiness. Swarm Evol Comput 75:101149. https://doi.org/10.1016/j.swevo.2022.101149
3. Amaro Júnior B, Santos MC, Carvalho GN, Araújo LJP, Pinheiro PR (2021) Metaheuristics for the minimum time cut path problem with different cutting and sliding speeds. Algorithms 14(11):305. https://doi.org/10.3390/a14110305
4. Amaro Júnior B, Carvalho GN, Santos MC, Pinheio PR, Celedonio JWL (2023) Evolutionary algorithms for optimization sequence of cut in the laser cutting path problem. Appl Sci 13(18):10133. https://doi.org/10.3390/app131810133
5. Andrade CE, Resende MGC, Zhang W, Sinha RK, Reichmann KC, Doverspike RD, Miyazawa FK (2015) A biased random-key genetic algorithm for wireless backhaul network design. Appl Soft Comput J 33:150–169. https://doi.org/10.1016/j.asoc.2015.04.016
6. Andrade CE, Silva T, Pessoa LS (2019) Minimizing flowtime in a flowshop scheduling problem with a biased random-key genetic algorithm. Expert Syst Appl 128:67–80. https://doi.org/10.1016/j.eswa.2019.03.007
7. Andrade CE, Toso RF, Gonçalves JF, Resende MGC (2021) The multi-parent biased random-key genetic algorithm with implicit path-relinking and its real-world applications. Eur J Oper Res 289(1):17–30. https://doi.org/10.1016/j.ejor.2019.11.037
8. Andrade CE, Pessoa LS, Stawiarski S (2024) The physical cell identity assignment problem: a practical optimization approach. IEEE Trans Evol Comput 28(2):282–292. https://doi.org/10.1109/TEVC.2022.3185927
9. Bean JC (1994) Genetic algorithms and random keys for sequencing and optimization. ORSA J Comput 6(2):154–160. https://doi.org/10.1287/ijoc.6.2.154
10. Bierbüße J, Mönch L, Biele A (2025) Heuristic approaches for a multi-mode resource availability cost problem in aircraft manufacturing. Comput Oper Res 176:106888. https://doi.org/10.1016/j.cor.2024.106888
11. Bolsi B, Lima VL, Alves Queiroz TA, Iori M (2022) Heuristic algorithms for integrated workforce allocation and scheduling of perishable products. Int J Prod Res 61:1–16. https://doi.org/10.1080/00207543.2022.2144525
12. Bresina JL (1996) Heuristic-biased stochastic sampling. In: Proceedings of the thirteenth national conference on artificial intelligence-volume 1, pp 271–278
13. Brito J, Fadel A, Semaan G (2020) A genetic algorithm applied to optimal allocation in stratified sampling. Commun Stat Simul Comput 51:3714–3732. https://doi.org/10.1080/03610918.2020.1722832
14. Brito JA, Semaan G, Fadel A (2022) The effective BRKGA algorithm for the k-medoids clustering problem. RAIRO-Oper Res 56(4):3137–3153. https://doi.org/10.1051/ro/2022141
15. Buriol LS, Hirsch MJ, Pardalos PM, Querido T, Resende MGC, Ritt M (2010) A biased random-key genetic algorithm for road congestion minimization. Optim Lett 4(4):619–633. https://doi.org/10.1007/s11590-010-0226-6
16. Carrabs F (2021) A biased random-key genetic algorithm for the set orienteering problem. Eur J Oper Res 292(3):830–854. https://doi.org/10.1016/j.ejor.2020.11.043
17. Chagas JBC, Toffolo TAM, Souza MJF, Iori M (2020) The double traveling salesman problem with partial last-in-first-out loading constraints. Int Trans Oper Res 29(4):2346–2373. https://doi.org/10.1111/itor.12876

18. Chagas JBC, Blank J, Wagner M, Souza MJF, Deb K (2021) A non-dominated sorting based customized random-key genetic algorithm for the bi-objective traveling thief problem. J Heurist 27(3):267–301. https://doi.org/10.1007/s10732-020-09457-7
19. Chaves AA, Lorena LAN, Senne ELF, Resende MGC (2016) Hybrid method with CS and BRKGA applied to the minimization of tool switches problem. Comput Oper Res 67:174–183. https://doi.org/10.1016/j.cor.2015.10.009
20. Chaves AA, Gonçalves JF, Lorena LAN (2018) Adaptive biased random-key genetic algorithm with local search for the capacitated centered clustering problem. Comput Ind Eng 124:331–346. https://doi.org/10.1016/j.cie.2018.07.031
21. Chaves AA, Vianna BL, da Silva TT, Schenekemberg CM (2024) A parallel branch-and-cut and an adaptive metaheuristic to solve the family traveling salesman problem. Expert Syst Appl 238:121735. https://doi.org/10.1016/j.eswa.2023.121735
22. Cicek ZIE, Ozturk ZK (2021) Optimizing the artificial neural network parameters using a biased random key genetic algorithm for time series forecasting. Appl Soft Comput 102:107091. https://doi.org/10.1016/j.asoc.2021.107091
23. Cotta C, Mathieson L, Moscato P (2018) Memetic algorithms. In: Martí R, Pardalos PM, Resende MGC (eds) Handbook of heuristics. Springer International Publishing, Cham, pp 607–638. https://doi.org/10.1007/978-3-319-07124-4_29
24. Damm RB, Resende MGC, Ronconi DP (2016) A biased random key genetic algorithm for the field technician scheduling problem. Comput Oper Res 75:49–63. https://doi.org/10.1016/j.cor.2016.05.003
25. Damm R, Chaves A, Riveaux J, Ronconi D (2025) Scheduling technicians and tasks through an adaptive multi-objective biased random-key genetic algorithm. Ann Oper Res 346(2):945–980. https://doi.org/10.1007/s10479-024-06325-6
26. Eiben A, Hinterding R, Michalewicz Z (1999) Parameter control in evolutionary algorithms. IEEE Trans Evol Comput 3(2):124–141. https://doi.org/10.1109/4235.771166
27. Ericsson M, Resende MGC, Pardalos PM (2002) A genetic algorithm for the weight setting problem in OSPF routing. J Comb Optim 6(3):299–333. https://doi.org/10.1023/A:1014852026591
28. Fadel AC, Ochi LS, Brito JADM, Semaan GS (2021) Microaggregation heuristic applied to statistical disclosure control. Inf Sci 548:37–55. https://doi.org/10.1016/j.ins.2020.09.069
29. Falls M, Bernardello R, Castrillo M, Acosta M, Llort J, Galí M (2022) Use of genetic algorithms for ocean model parameter optimisation: a case study using pisces-v2_rc for north atlantic particulate organic carbon. Geosci Model Develop 15(14):5713–5737. https://doi.org/10.5194/gmd-15-5713-2022
30. Feurer M, Hutter F (2019) Hyperparameter optimization. Springer International Publishing, Cham
31. Fontes DBMM, Homayouni SM (2023) A bi-objective multi-population biased random key genetic algorithm for joint scheduling quay cranes and speed adjustable vehicles in container terminals. Flex Serv Manuf J 35(1):241–268. https://doi.org/10.1007/s10696-022-09467-6
32. Fontes DBMM, Homayouni SM, Fernandes JC (2023) Energy-efficient job shop scheduling problem with transport resources considering speed adjustable resources. Int J Prod Res 62:1–24. https://doi.org/10.1080/00207543.2023.2175172
33. Freitas CC, Aloise DJ, Da Costa Fontes FF, Santos AC, Da Silva Menezes M (2023) A biased random-key genetic algorithm for the two-level hub location routing problem with directed tours. OR Spectrum 45:1–22. https://doi.org/10.1007/s00291-023-00718-y
34. Glover F (1997) Tabu search and adaptive memory programming – advances, applications and challenges. In: Interfaces in computer science and operations research. Springer, Berlin, pp 1–75. https://doi.org/10.1007/978-1-4615-4102-8_1
35. Gonçalves JF, Almeida JR (2002) A hybrid genetic algorithm for assembly line balancing. J Heurist 8(6):629–642. https://doi.org/10.1023/A:1020377910258
36. Gonçalves JF, Resende MGC (2011) Biased random-key genetic algorithms for combinatorial optimization. J Heurist 17(5):487–525. https://doi.org/10.1007/s10732-010-9143-1

37. Gonçalves JF, Wäscher G (2020) A MIP model and a biased random-key genetic algorithm based approach for a two-dimensional cutting problem with defects. Eur J Oper Res 286(3):867–882. https://doi.org/10.1016/j.ejor.2020.04.028
38. Gonçalves JF, Resende MGC, Toso RF (2014) An experimental comparison of biased and unbiased random-key genetic algorithms. Pesquisa Oper 34(2):143–164. https://doi.org/10.1590/0101-7438.2014.034.02.0143
39. Holland JH (1975) Adaptation in natural and artificial systems: an introductory analysis with applications to biology, control, and artificial intelligence. University Michigan Press, Ann Arbor
40. Huang Y, Santos AC, Duhamel C (2020) Bi-objective methods for road network problems with disruptions and connecting requirements. J Oper Res Soc 71(12):1959–1971. https://doi.org/10.1080/01605682.2019.1639479
41. Huang Y, Santos AC, Duhamel C (2020) Model and methods to address urban road network problems with disruptions. Int Trans Oper Res 27(6):2715–2739. https://doi.org/10.1111/itor.12641
42. Huang Z, Huang L, Li D (2024) Co-evolutionary algorithm for two-stage hybrid flow shop scheduling problem with suspension shifts. Mathematics 12(16):2575. https://doi.org/10.3390/math12162575
43. Ibarra-Rojas OJ, Silva-Soto Y (2021) Vehicle routing problem considering equity of demand satisfaction. Optim Lett 15(6):2275–2297. https://doi.org/10.1007/s11590-021-01704-5
44. Ibarra-Rojas OJ, Cervantes-Sanmiguel KI, Quintanar-Sanchez DL, Jacobo-Romero YY, Rios-Solis Y (2025) Integration of timetabling to coordinate overlapping lines and vehicle scheduling with interlining. Transp A Transp Sci . https://doi.org/10.1080/23249935.2025.2474640
45. Japa L, Serqueira M, Mendonça I, Aritsugi M, Bezerra E, González PH (2023) A population-based hybrid approach for hyperparameter optimization of neural networks. IEEE Access 11:50752. https://doi.org/10.1109/ACCESS.2023.3277310
46. Johnson DS, Breslau L, Diakonikolas I, Duffield N, Gu Y, Hajiaghayi M, Karloff H, Resende MGC, Sen S (2020) Near-optimal disjoint-path facility location through set cover by pairs. Oper Res 68(3):896–926. https://doi.org/10.1287/opre.2019.1956
47. Karimi-Mamaghan M, Mohammadi M, Meyer P, Karimi-Mamaghan AM, Talbi EG (2022) Machine learning at the service of meta-heuristics for solving combinatorial optimization problems: a state-of-the-art. Eur J Oper Res 296(2):393–422. https://doi.org/10.1016/j.ejor.2021.04.032
48. Kong M, Liu X, Pei J, Cheng H, Pardalos PM (2020) A BRKGA-DE algorithm for parallel-batching scheduling with deterioration and learning effects on parallel machines under preventive maintenance consideration. Ann Math Artif Intell 88(1–3):237–267. https://doi.org/10.1007/s10472-018-9602-1
49. Kong M, Pei J, Xu J, Liu X, Yu X, Pardalos PM (2020) A robust optimization approach for integrated steel production and batch delivery scheduling with uncertain rolling times and deterioration effect. Int J Prod Res 58(17):5132–5154. https://doi.org/10.1080/00207543.2019.1693659
50. Kummer AF, Araújo OCB, Buriol LS, Resende MGC (2022) A biased random-key genetic algorithm for the home health care problem. Int Trans Oper Res 31:1–31. https://doi.org/10.1111/itor.13221
51. Lima A, Aquino ALL, Nogueira B, Pinheiro RGS (2022) A matheuristic approach for the minimum broadcast time problem using a biased random-key genetic algorithm. Int Trans Oper Res 31:246–273. https://doi.org/10.1111/itor.13146
52. Liu C, Han Y, Wang Y, Li J, Liu Y (2024) Multi-population coevolutionary algorithm for a green multi-objective flexible job shop scheduling problem with automated guided vehicles and variable processing speed constraints. Swarm Evol Comput 91:101774. https://doi.org/10.1016/j.swevo.2024.101774
53. Londe MA, Andrade CE, Pessoa LS (2021) An evolutionary approach for the p-next center problem. Expert Syst Appl 175:114728. https://doi.org/10.1016/j.eswa.2021.114728

54. Londe MA, Andrade CE, Pessoa LS (2022) Exact and heuristic approaches for the root sequence index allocation problem. Appl Soft Comput 130:109634. https://doi.org/10.1016/j.asoc.2022.109634
55. Londe MA, Andrade CE, Pessoa LS (2025) A multi-stage approach for root sequence index allocation. Eur J Oper Res https://doi.org/10.1016/j.ejor.2025.05.015. Article in Press
56. Londe MA, Pessoa LS, Andrade CE, Resende MG (2025) Biased random-key genetic algorithms: a review. Eur J Oper Res 321:1–22. https://doi.org/10.1016/j.ejor.2024.03.030
57. Londe MA, Pessoa LS, Andrade CE, Resende MG (2025) Early years of biased random-key genetic algorithms: a systematic review. J Global Optim https://doi.org/10.1007/s10898-024-01446-5. To Appear
58. López-Ibáñez M, Dubois-Lacoste J, Cáceres LP, Birattari M, Stützle T (2016) The irace package: iterated racing for automatic algorithm configuration. Oper Res Perspect 3:43–58. https://doi.org/10.1016/j.orp.2016.09.002
59. Lourenço HR, Martin OCM, Stützle T (2003) Iterated local search. In: Handbook of metaheuristics. Springer, Berlin, pp 320–353. https://doi.org/10.1007/0-306-48056-5_11
60. Lu Z, Hu K, Ng TS (2023) Improving additive manufacturing production planning: a sub-second pixel-based packing algorithm. Comput Ind Eng 181:109318. https://doi.org/10.1016/j.cie.2023.109318
61. Maecker S, Shen L, Mönch L (2023) Unrelated parallel machine scheduling with eligibility constraints and delivery times to minimize total weighted tardiness. Comput Oper Res 149:105999. https://doi.org/10.1016/j.cor.2022.105999
62. Mandal SK, de Queiroz TA, Miyazawa FK (2024) A biased random-key genetic algorithm for the 2d strip packing problem with order and stability constraints. Pesquisa Operacional 44. https://doi.org/10.1590/0101-7438.2023.043.00284365
63. Marques Jr EL, Coelho VN, Coelho IM, Satoru Ochi L, Maculan N, Mladenović N, Coelho BN (2023) A two-phase multi-objective metaheuristic for a green UAV grid routing problem. Optim Lett 17:1–24. https://doi.org/10.1007/s11590-023-02013-9
64. Martarelli NJ, Nagano MS (2020) Unsupervised feature selection based on bio-inspired approaches. Swarm Evol Comput 52:100618. https://doi.org/10.1016/j.swevo.2019.100618
65. Mauri GR, Biajoli FL, Rabello RL, Chaves AA, Ribeiro GM, Lorena LAN (2021) Hybrid metaheuristics to solve a multiproduct two-stage capacitated facility location problem. Int Trans Oper Res 28(6):3069–3093. https://doi.org/10.1111/itor.12930
66. Melo RA, Ribeiro CC, Riveaux JA (2023) A biased random-key genetic algorithm for the minimum quasi-clique partitioning problem. Ann Oper Res 1–33. https://doi.org/10.1007/s10479-023-05609-7
67. Mendoza-Andrade N, Ruiz-y-Ruiz E, Rodriguez-Romo S (2025) Solution algorithms for the capacitated location tree problem with interconnections. Algorithms 18(1):50. https://doi.org/10.3390/a18010050
68. Mikulski S, Tomczewski A (2021) Use of energy storage to reduce transmission losses in meshed power distribution networks. Energies 14(21):7304. https://doi.org/10.3390/en14217304
69. Morgan SJ, McGrath CN, Weck OL (2023) Optimization of multispacecraft maneuvers for mobile target tracking from low earth orbit. JSpacecraft Rockets 60(2):581–590. https://doi.org/10.2514/1.A35457
70. Ochoa G, Malan K, Blum C (2021) Search trajectory networks: a tool for analysing and visualising the behaviour of metaheuristics. Appl Soft Comput 109:107492. https://doi.org/10.1016/j.asoc.2021.107492
71. Oliveira EV, Romero R (2022) Improved biased random key genetic algorithm for the two-dimensional non-guillotine cutting problem. Pesquisa Oper 42:1–24. https://doi.org/10.1590/0101-7438.2022.042.00260406
72. Oliveira RMD, Chaves AA, Lorena LAN (2017) A comparison of two hybrid methods for constrained clustering problems. Appl Soft Comput J 54:256–266. https://doi.org/10.1016/j.asoc.2017.01.023

73. Oliveira BB, Carravilla MA, Oliveira JF, Costa AM (2019) A co-evolutionary matheuristic for the car rental capacity-pricing stochastic problem. Eur J Oper Res 276(2):637–655. https://doi.org/10.1016/j.ejor.2019.01.015
74. Oliveira BB, Carravilla MA, Oliveira JF, Resende MGC (2021) A C++ application programming interface for co-evolutionary biased random-key genetic algorithms for solution and scenario generation. Optim Methods Softw 37(3):1065–1086. https://doi.org/10.1080/10556788.2021.1884250
75. Oliveira BB, Carravilla MA, Oliveira JF (2022) A diversity-based genetic algorithm for scenario generation. Eur J Oper Res 299(3):1128–1141. https://doi.org/10.1016/j.ejor.2021.09.047
76. Oliveira BA, Xavier EC, Borin E (2024) BrkgaCuda 2.0: a framework for fast biased random-key genetic algorithms on GPUs. Soft Comput 28(21):12689–12704. https://doi.org/10.1007/s00500-024-10336-7
77. Oliveira JaLA, Aquino AL, Pinheiro RG, Nogueira B (2024) Optimizing public transport system using biased random-key genetic algorithm. Appl Soft Comput 158:111578. https://doi.org/10.1016/j.asoc.2024.111578
78. OpenMP (2013) http://openmp.org/wp/. Last Visited on May 11, 2013
79. Pan Y, Zhang L, Yan Z, Lwin M, Skibniewski M (2021) Discovering optimal strategies for mitigating COVID-19 spread using machine learning: experience from Asia. Sustain Cities Soc 75:103254. https://doi.org/10.1016/j.scs.2021.103254
80. Pandey HM, Chaudhary A, Mehrotra D (2014) A comparative review of approaches to prevent premature convergence in GA. Appl Soft Comput 24:1047–1077. https://doi.org/10.1016/j.asoc.2014.08.025
81. Pastore T, Menna C, Asprone D (2022) Bézier-based biased random-key genetic algorithm to address printability constraints in the topology optimization of concrete structures. Struct Multidisc Optim 65(2):64. https://doi.org/10.1007/s00158-021-03119-3
82. Pedrola O, Careglio D, Klinkowski M, Velasco L, Bergman K, Solé-Pareta J (2013) Metaheuristic hybridizations for the regenerator placement and dimensioning problem in sub-wavelength switching optical networks. Eur J Oper Res 224(3):614–624. https://doi.org/10.1016/j.ejor.2012.08.011
83. Pessoa LS, Santos AC, Resende MGC (2017) A biased random-key genetic algorithm for the tree of hubs location problem. Optim Lett 11(7):1371–1384. https://doi.org/10.1007/s11590-016-1082-9
84. Pinacho-Davidson P, Blum C (2020) Barrakuda: a hybrid evolutionary algorithm for minimum capacitated dominating set problem. Mathematics 8(11):1–26. https://doi.org/10.3390/math8111858
85. Pinto BQ, Ribeiro CC, Rosseti I, Noronha TF (2020) A biased random-key genetic algorithm for routing and wavelength assignment under a sliding scheduled traffic model. J Global Optim 77(4):949–973. https://doi.org/10.1007/s10898-020-00877-0
86. Pinto BQ, Ribeiro CC, Riveaux JA, Rosseti I (2021) A BRKGA-based matheuristic for the maximum quasi-clique problem with an exact local search strategy. RAIRO-Oper Res 55:S741–S763. https://doi.org/10.1051/ro/2020003
87. Queiroga E, Pinheiro RGS, Christ Q, Subramanian A, Pessoa AA (2021) Iterated local search for single machine total weighted tardiness batch scheduling. J Heurist 27(3):353–438. https://doi.org/10.1007/s10732-020-09461-x
88. Ramos S, Roque LA, Gomes A, Soares J, Rolle JC, Vale Z (2025) Multi-objective model for residential energy management in context of individual self-consumption. Math Comput Simul 231:120–127. https://doi.org/10.1016/j.matcom.2024.11.017
89. Raposo AAM, Rodrigues AB, da Guia da Silva M (2020) Robust meter placement for state estimation considering distribution network reconfiguration for annual energy loss reduction. Electric Power Syst Res 182:106233. https://doi.org/10.1016/j.epsr.2020.106233
90. Reixach J, Blum C, Djukanovic M, Raidl GR (2025) A biased random key genetic algorithm for solving the longest common square subsequence problem. IEEE Trans Evol Comput 29(2):390–403. https://doi.org/10.1109/TEVC.2024.3413150

91. Resende MGC, Ribeiro CC (2011) Restart strategies for GRASP with path-relinking heuristics. Optim Lett 5:467–478
92. Ribeiro CC, Resende MGC (2012) Path-relinking intensification methods for stochastic local search algorithms. J Heurist 18(2):193–214. https://doi.org/10.1007/s10732-011-9167-1
93. Rochman AN, Prasetyo H, Nugroho MT (2017) Biased random key genetic algorithm with insertion and gender selection for capacitated vehicle routing problem with time windows. AIP Confer Proc 1855(1):020025. https://doi.org/10.1063/1.4985470
94. Rocholl J, Mönch L (2021) Decomposition heuristics for parallel-machine multiple orders per job scheduling problems with a common due date. J Oper Res Soc 72(8):1737–1753. https://doi.org/10.1080/01605682.2019.1640589
95. Santiyuda G, Wardoyo R, Pulungan R (2024) Solving biobjective traveling thief problems with multiobjective reinforcement learning. Appl Soft Comput 161:111751. https://doi.org/10.1016/j.asoc.2024.111751
96. Schenekemberg CM, Chaves AA, Coelho LC, Guimarães TA, Avelino GG (2022) The dial-a-ride problem with private fleet and common carrier. Comput Oper Res 147:105933. https://doi.org/10.1016/j.cor.2022.105933
97. Schenekemberg CM, Chaves AA, Guimarães TA, Coelho LC (2024) Hybrid metaheuristic for the dial-a-ride problem with private fleet and common carrier integrated with public transportation. Ann Oper Res https://doi.org/10.1007/s10479-024-06136-9
98. Schuetz MJA, Brubaker JK, Montagu H, van Dijk Y, Klepsch J, Ross P, Luckow A, Resende MGC, Katzgraber HG (2022) Optimization of robot-trajectory planning with nature-inspired and hybrid quantum algorithms. Phys Rev Appl 18(5):054045. https://doi.org/10.1103/PhysRevApplied.18.054045
99. Silva TT, Chaves AA, Yanasse HH, Luna HPL (2019) The multicommodity traveling salesman problem with priority prizes: a mathematical model and metaheuristics. Comput Appl Math 38(4):188. https://doi.org/10.1007/s40314-019-0976-4
100. Silva G, Leite A, Ospina R, Leiva V, Figueroa-Zúñiga J, Castro C (2023) Biased random-key genetic algorithm with local search applied to the maximum diversity problem. Mathematics 11(14):3072. https://doi.org/10.3390/math11143072
101. Silva SE, Ribeiro CC, dos Santos Souza U (2023) A biased random-key genetic algorithm for the chordal completion problem. RAIRO-Oper Res 57(3):1559–1578. https://doi.org/10.1051/ro/2023081
102. Silva MC, Melo RA, Resende MG, Santos MC, Toso RF (2024) Obtaining the Grundy chromatic number: how bad can my greedy heuristic coloring be? Comput Oper Res 168:106703. https://doi.org/10.1016/j.cor.2024.106703
103. Silva-Soto YI, Ibarra-Rojas OJ (2021) Timetabling with flexible frequencies to synchronise groups of bus lines at common stops. Transp A Transp Sci 17(4):978–1001. https://doi.org/10.1080/23249935.2020.1822952
104. Soares LCR, Carvalho MAM (2020) Biased random-key genetic algorithm for scheduling identical parallel machines with tooling constraints. Eur J Oper Res 285(3):955–964. https://doi.org/10.1016/j.ejor.2020.02.047
105. Soares LC, Carvalho MA (2022) Application of a hybrid evolutionary algorithm to resource-constrained parallel machine scheduling with setup times. Comput Oper Res 139:105637. https://doi.org/10.1016/j.cor.2021.105637
106. Souza Queiroz LRD, Andretta M (2020) Two effective methods for the irregular knapsack problem. Appl Soft Comput J 95:106485. https://doi.org/10.1016/j.asoc.2020.106485
107. Spears WM, De Jong KA (1991) An analysis of multi-point crossover. In: Foundations of genetic algorithms, vol 1. Elsevier, Amsterdam, pp 301–315. https://doi.org/10.1016/B978-0-08-050684-5.50022-7
108. Spears WM, De Jong KA (1991) On the virtues of parameterized uniform crossover. In: Proceedings of the fourth international conference on genetic algorithms, pp 230–236
109. Sun B, Zhou Y (2022) Bayesian network structure learning with improved genetic algorithm. Int J Intell Syst 37(9):6023–6047. https://doi.org/10.1002/int.22833

110. Toso RF, Resende MGC (2015) A C++ application programming interface for biased random-key genetic algorithms. Optim Methods Softw 30(1):81–93. https://doi.org/10.1080/10556788.2014.890197
111. Usberti FL, González JFV, Cavellucci C (2024) A quadratic formulation for the optimal allocation of fault indicators. Electric Power Syst Res 231:110352. https://doi.org/10.1016/j.epsr.2024.110352
112. Vilela LZAR (2022) Strategies for parameter control in the biased random-key genetic algorithm. Master's thesis, Pontifical Catholic University of Rio de Janeiro. https://doi.org/10.17771/PUCRio.acad.61145
113. Villicaña-Cervantes D, Ibarra-Rojas OJ (2022) Accessible location of mobile labs for COVID-19 testing. Health Care Manag Sci 27:1–19. https://doi.org/10.1007/s10729-022-09614-3
114. Watkins CJ, Dayan P (1992) Q-learning. Mach Learn 8:279–292. https://doi.org/10.1007/BF00992698
115. Whitley D, Rana S, Heckendorn RB (1999) The island model genetic algorithm: on separability, population size and convergence. J Comput Inf Technol 7(1):33–47
116. Xie Y, Sheng Y, Qiu M, Gui F (2022) An adaptive decoding biased random key genetic algorithm for cloud workflow scheduling. Eng Appl Artif Intell 112:104879. https://doi.org/10.1016/j.engappai.2022.104879
117. Xu Y, Guo P, Zeng Y (2022) An iterative neighborhood local search algorithm for capacitated centered clustering problem. IEEE Access 10:34497–34510. https://doi.org/10.1109/ACCESS.2022.3162692
118. Yang F, Li C, Wang F, Yang Z, Gao K (2025) Scheduling two-stage healthcare appointment systems via a knowledge-based biased random-key genetic algorithm. Swarm Evol Comput 94:101864. https://doi.org/10.1016/j.swevo.2025.101864
119. Yu X, Qian J, Zhang Y, Kong M (2023) Supply chain scheduling method for the coordination of agile production and port delivery operation. Mathematics 11(15):3276. https://doi.org/10.3390/math11153276
120. Zhang X, Shan M, Zeng J (2022) Parallel batch processing machine scheduling under two-dimensional bin-packing constraints. IEEE Trans Reliab 72:1265. https://doi.org/10.1109/TR.2022.3201333
121. Zhao W, Liu Y, Wang Y, Yue X (2023) Distributed spot welding task allocation and sequential planning for multi-station multi-robot coordinate assembly processes. Int J Adv Manuf Technol 127(11–12):5233–5251. https://doi.org/10.1007/s00170-023-11750-1
122. Zhou G, Li D, Bian J, Zhang Y (2024) Airfreight forwarder's shipment planning: shipment consolidation and containerization. Comput Oper Res 161:106443. https://doi.org/10.1016/j.cor.2023.106443
123. Zudio A, da Silva Costa DH, Masquio BP, Coelho IM, Pinto PED (2018) BRKGA/VND hybrid algorithm for the classic three-dimensional bin packing problem. Electron Notes Discrete Math 66:175–182. https://doi.org/10.1016/j.endm.2018.03.023

Tabu Search

31

Manuel Laguna

Contents

Introduction . . . 942
Short-Term Memory . . . 943
Example 1 . . . 944
Example 2 . . . 947
Long-Term Memory . . . 948
Example 3 . . . 950
Strategic Oscillation . . . 950
Example 4 . . . 952
Example 5 . . . 953
Path Relinking . . . 955
Example 6 . . . 956
Conclusions . . . 957
Cross-References . . . 958
References . . . 958

Abstract

Tabu search (TS) is a solution methodology within the area of metaheuristics. While the methodology applies to optimization problems in general, most TS applications have been and continue to be in discrete optimization. A key and distinguishing feature of tabu search is the use of special strategies based on adaptive memory. The underlying philosophy is that an effective search for optimal solutions should involve a flexible process that responds to the objective function landscape in a manner that allows it to learn appropriate directions to

M. Laguna (✉)
Leeds School of Business, University of Colorado Boulder, Boulder, CO, USA
e-mail: laguna@colorado.edu

© Springer Nature Switzerland AG 2025

R. Martí et al. (eds.), *Handbook of Heuristics*,
https://doi.org/10.1007/978-3-032-00385-0_24

exploit specific areas of the solution space and useful departures to explore new terrain. The adaptive memory structures of tabu search enable the implementation of procedures that are capable of searching effectively and produce solutions of suitable quality within reasonable computational effort.

Keywords

Heuristic search · Metaheuristics · Optimization

Introduction

Tabu search (TS) has its origins in Fred Glover's seminal 1977 paper *Heuristics for Integer Programming Using Surrogate Constraints*. This article introduced three key concepts: (1) scatter search, (2) oscillating assignment, and (3) strongly determined and consistent variables. Scatter search was later developed into a full-blown metaheuristic that is now well-established in the literature. Once tabu search was fully developed, oscillating assignment became the TS mechanism known as strategic oscillation. The concept of strongly determined and consistent variables became part of the tabu search long-term memory strategies, as described below. Without using the term tabu, the knapsack heuristic in [1] illustrates the use of short-term memory. The tabu search name was coined in [2], where the method was introduced as a "metaheuristic superimposed on another heuristic." It was also the first time that "metaheuristic" was used in the literature.

A tabu search starts from an initial solution and explores the solution space by making transformations to move from one solution to the next. That is, TS belongs to the family of metaheuristic methodologies that are based on single-solution searches. This family includes methodologies such as simulated annealing and variable neighborhood search. In contrast, genetic algorithms and scatter search belong to population-based metaheuristics, which create, maintain, and evolve a population of solutions as a means to explore the solution space. TS is based on the premise that problem-solving, in order to qualify as intelligent, must incorporate adaptive memory and responsive exploration.

The choices to determine how to transform a solution are guided by information collected during the search through adaptive memory structures that minimize the reliance in the sampling techniques that are typically employed by memoryless designs. Adaptive memory contrasts with rigid memory designs characteristic of branch and bound strategies. Responsive exploration integrates the basic principles of intelligent search, i.e., exploiting good solution features while exploring new promising regions. Tabu search is concerned with finding new and more effective ways of taking advantage of the mechanisms associated with both adaptive memory and responsive exploration.

Short-Term Memory

Tabu search can be applied directly to verbal or symbolic statements of many kinds of decision problems, without the need to transform them into mathematical formulations. Nevertheless, it is useful to introduce mathematical notation to express a broad class of these problems, as a basis for describing certain features of tabu search. We characterize this class of problems as that of optimizing (minimizing or maximizing) a function $f(x)$ subject to $x \in X$, where the set X summarizes constraints on the vector of decision variables x, including those that compel all or some components of x to receive discrete values. The problem of interest may not have an easy mathematical representation, but the verbal stipulations could be coded as rules.

Tabu search begins in the same way as ordinary local or neighborhood search, proceeding iteratively from one point (solution) to another until a chosen termination criterion is satisfied. Each solution $x \in X$ has an associated neighborhood $N(x) \subset X$, and each solution $x' \in N(x)$ is reached from x by an operation called a *move*. Hence, x' is said to be a neighbor of x.

In a minimization problem, best improving refers to the strategy of identifying $x^* \in N(x)$ such that $f(x^*) < f(x')$ for all $x' \in N(x)$. The move value is the change in the objective function value from the current solution to the solution after the move, i.e., $v = f(x^*) - f(x)$. Therefore, $v < 0$ for improving moves and $v \geqslant 0$ for nonimproving moves. In steepest descent, the procedure only makes improving moves, and it stops when no improving move is available in the neighborhood of the current solution. That is, the best neighbor of a solution becomes the new current solution as long as the move reduces the objective function value. In contrast, the best-improving strategy in TS chooses the best neighbor of x^* of x even when $v \geqslant 0$. In spite of its attractiveness, in certain settings the best-improving strategy is sometimes impractical because it may be computationally too expensive, as when $N(x)$ is large or each element is costly to retrieve or evaluate.

The relevance of choosing good solutions from current neighborhoods is magnified when the guidance mechanisms of tabu search are introduced to go beyond the locally optimal termination point of a descent method. Thus, an important first-level consideration for tabu search is to determine an appropriate *candidate list strategy* for narrowing the examination of elements of $N(x)$ in order to achieve an effective trade-off between the quality of x_* and the effort expended to find it. A popular candidate list strategy is known as the first-improving strategy. This strategy is such that neighbors are scanned in a predetermined order and the search moves to the first $x' \in N(x)$ for which $f(x') < f(x)$. If no such a move exists, then the search moves to the best-nonimproving move in the neighborhood.

Short-term memory functions give the search the opportunity to continue beyond local optima, by allowing the execution of nonimproving moves coupled with the modification of the neighborhood structure of subsequent solutions. The modified neighborhood, denoted by $N^*(x)$, is the result of maintaining a selective history of the states encountered during the search.

The tabu classification serves to identify elements of $N(x)$ that are excluded from $N^*(x)$. Characterized in this way, TS may be viewed as a dynamic neighborhood method. This means that the neighborhood of x is not a static set, but rather a set that can change according to the history of the search. A TS process based on short-term strategies may allow a solution x to be visited more than once, but it is likely that the corresponding reduced neighborhood $N^*(x)$ will be different each time. From a practical standpoint, the method will characteristically identify an optimal or near-optimal solution long before a substantial portion of X is examined.

The approach of storing complete solutions (*explicit* memory) generally consumes a massive amount of space and time when applied to each solution generated. Therefore, instead of recording full solutions, TS memory structures are based on recording attributes (*attributive* memory). In addition, short-term memory is based on the most recent history of the search trajectory (*recency-based* memory). In recency-based attributive memory, selected attributes that occur in solutions recently visited are labeled *tabu-active* and moves that contain tabu-active elements, or particular combinations of these attributes, are those that are classified tabu. This prevents certain solutions from the recent past from belonging to $N^*(x)$ and hence from being revisited. Other solutions that share such tabu-active attributes are also similarly prevented from being visited.

The number of iterations (i.e., executed moves) that an attribute remains tabu-active is known as the tabu tenure. Tabu tenure can be static, i.e., fixed throughout the search, or dynamic, i.e., vary either systematically or probabilistically. Aspiration criteria are introduced in tabu search to determine when tabu activation rules can be overridden, thus removing a tabu classification otherwise applied to a move. The most common aspiration criterion consists of removing a tabu classification from a trial move when the move yields a solution better than the best obtained so far.

Example 1

The short-term memory structure of tabu search will be illustrated with the following instance of the classical 0-1 knapsack problem:

Maximize $67x_1 + 500x_2 + 98x_3 + 200x_4 + 120x_5 + 312x_6 + 100x_7 + 200x_8 + 180x_9 + 100x_{10}$

$$\text{Subject to } 5x_1 + 45x_2 + 9x_3 + 19x_4 + 12x_5 + 32x_6 + 11x_7 + 23x_8 + 21x_9 +$$

$$14x_{10} \leqslant 100$$

$$x \in \{0, 1\}$$

The initial solution is found using a greedy heuristic that consists of selecting items (i.e., setting variable values to 1), one by one, in decreasing order of their bang-for-buck ratio. The process ends when adding one more item exceeds the capacity of the knapsack. The bang for buck is the ratio of profit to weight,

where profits are the objective function coefficients and weights are the constraint coefficients. For simplicity, the variables in the example are indexed by their bang-for-buck ratio. That is, x_1 is the variable with the highest ratio and x_{10} is the variable with the lowest ratio. The construction of the initial solution is summarized in Table 1.

The move mechanism is defined as changing the value of a single variable from either zero to one or one to zero; this is also known as a "flip" move. The best-improving strategy is used, meaning that all moves are considered and the one that results in the neighbor with the best objective function value is chosen. The neighborhood of the current solution is then given in Table 2.

Since in this example the search is not allowed to visit infeasible solutions, there is no improving move in the neighborhood of the current solution. Under this restriction, the best move is the first in the table and consists of changing the value of x_1 from one to zero. The Neighbor column shows the indexes of the variables that are set to one as a result of the corresponding move. The short-term memory structure may be designed as follows:

Attribute: Index of the variable that was chosen to change values
Tabu classification: A move that includes a tabu-active variable is classified tabu
Tabu tenure: Three iterations

Table 1 Construction of initial solution for Example 1

Item	Profit	Weight	Ratio	Chosen?	Profit	Weight
1	67	5	13.40	1	67	5
2	500	45	11.11	1	500	45
3	98	9	10.89	1	98	9
4	200	19	10.53	1	200	19
5	120	12	10.00	1	120	12
6	312	32	9.75		0	0
7	100	11	9.09		0	0
8	200	23	8.70		0	0
9	180	21	8.57		0	0
10	100	14	7.14		0	0
				Total	985	90

Table 2 Neighborhood of the current solution in Example 1

No.	Move	Neighbor	Profit	Weight
1	$x_1 = 0$	(2, 3, 4, 5)	918	85
2	$x_2 = 0$	(1, 3, 4, 5)	485	45
3	$x_3 = 0$	(1, 2, 4, 5)	887	81
4	$x_4 = 0$	(1, 2, 3, 5)	785	71
5	$x_5 = 0$	(1, 2, 3, 4)	865	78
6	$x_6 = 1$	(1, 2, 3, 4, 5, 6)	1297	122
7	$x_7 = 1$	(1, 2, 3, 4, 5, 7)	1085	101
8	$x_8 = 1$	(1, 2, 3, 4, 5, 8)	1185	113
9	$x_9 = 1$	(1, 2, 3, 4, 5, 9)	1165	111
10	$x_{10} = 1$	(1, 2, 3, 4, 5, 10)	1085	104

Table 3 Ten iterations of the short-term memory procedure

Iteration	Tabu-active	Move	Solution	Profit	Weight
0			(1, 2, 3, 4, 5)	985	90
1		1	(2, 3, 4, 5)	918	85
2	1	7	(2, 3, 4, 5, 7)	1018	96
3	7 1	3	(2, 4, 5, 7)	920	87
4	3 7 1	5	(2, 4, 7)	800	75
5	5 3 7	8	(2, 4, 7, 8)	1000	98
6	8 5 3	7	(2, 4, 8)	900	87
7	7 8 5	3	(2, 3, 4, 8)	998	96
8	3 7 8	4	(2, 3, 8)	798	77
9	4 3 7	9	(2, 3, 8, 9)	978	98
10	9 4 3	8	(2, 3, 9)	778	75

Using this structure, attribute 1 is declared tabu-active and therefore a move that attempts to change the current value of x_1 is classified tabu for three iterations. Table 3 shows ten iterations of the procedure using the short-term memory structure defined above.

The tabu-active column shows the indexes of the variables that are tabu-active. Each index appears for three iterations. It starts in the first position, and it moves one position in each iteration until it is removed from the list. The Move column indicates the variable that has been chosen to "flip" its value. The Solution column shows the solution after the move has been executed with the resulting profit and weight shown in the last two columns, respectively. The best solution is found in the second iteration and has a profit of 1018.

This example is based on very simple "flip" moves. The neighborhood defined by the move mechanism is a design choice of critical importance for the performance of the method. Often, the neighborhood is defined by several move types that may vary in complexity. The attributes to be used for the tabu classification of moves depend on those choices, and they also influence the performance of the search. For instance, in this example, a slightly more complex move would consist of flipping the values of two variables. Instead of 10 neighbors, this move mechanism would generate 45 neighbors, corresponding to all combinations of variables taken two at a time. A potential short-term memory structure for this neighborhood is as follows:

Attributes: Indexes of the variables that were chosen to change values.

Tabu classification: A move that includes at least one tabu-active variable is classified tabu.

Tabu tenure: Three iterations for a variable flipping from 1 to 0 and two iterations for a variable flipping from 0 to 1.

The tabu tenure recognizes that in a typical knapsack problem there will be more variables outside the knapsack (i.e., with their values set to zero) than those inside the knapsack (i.e., with their values set to one). Therefore, if an item is taken from

the knapsack (i.e., its corresponding variable value is switched from one to zero), the tabu tenure keeps this item outside longer than when an item enters the knapsack. This illustrates that the flexibility of the TS methodology allows the analyst to be creative and take full advantage of context information.

Example 2

Given a matrix of weights, the linear ordering problem (LOP) consists of finding a permutation p of the columns (and rows) in order to maximize the sum of the weights in the upper triangle. Consider the LOP instance in Table 4.

The objective function value associated with the solution $p = (1, 2, 3, 4, 5, 6, 7)$ is $f(p) = 78$, corresponding to the sum of the 21 values in the upper triangle. The elements in the permutation correspond to columns (and rows) of the matrix. Suppose that a tabu search explores the solution space by moving an element $p(i)$ from its current position i to a position j, between elements $p(j-1)$ and $p(j)$. In permutation problems, these are commonly referred to as *insert* moves. For instance, inserting $p(6) = 6$ in position 2 results in $p' = (1, 6, 2, 3, 4, 5, 7)$ with $f(p') = 86$.

Since TS determines where to move via neighborhood search, it is important to be able to calculate move values with the fewest possible number of operations. One naïve way of calculating a move value consists of executing the proposed move and calculating, from scratch, the objective function of the resulting neighbor solution. Then, the value of the move is obtained as the difference between the objective function value of the current solution and the objective function value that results after executing the move. In many settings, this process is inefficient and unnecessary. Clever move value calculations can significantly improve the efficiency of the neighborhood exploration. In the case of the LOP, the move value involves only a portion of the entries in the column (and corresponding row) associated with the element that is changing positions. For instance, for the insertion of $p(6) = 6$ in position 2 to transforms p into p', only the entries in the bold rectangles in Table 5 are needed to calculate the move value v.

The move value in this case is $v = 1 - 4 + 6 - 0 + 2 - 6 + 13 - 4 = 8$. The objective function value is $f(p^{'}) = f(p) + v = 78 + 8 = 86$. A short-term memory structure for this problem may be defined as follows:

Table 4 Instance of the linear ordering problem

p	1	2	3	4	5	6	7
1	0	12	5	3	1	8	3
2	6	0	3	6	4	4	2
3	8	5	0	5	7	0	3
4	−2	7	2	0	−3	6	0
5	8	0	3	−1	0	4	1
6	9	1	6	2	13	0	4
7	2	9	4	−5	8	1	0

Table 5 Data for the move value calculation (bold-outlined rectangles)

p	1	2	3	4	5	6	7
1	0	12	5	3	1	8	3
2	6	0	3	6	4	4	2
3	8	5	0	5	7	0	3
4	−2	7	2	0	−3	6	0
5	8	0	3	−1	0	4	1
6	9	1	6	2	13	0	4
7	2	9	4	−5	8	1	0

Attribute: Index of the element that was inserted in a new position
Tabu classification: An insertion of a tabu-active element is classified tabu
Tabu tenure: Five iterations

Note that due to the nature of the moves and the tabu classification, elements during their tabu-active period may change positions. To illustrate this point, consider the insertion of $p(6) = 6$ in position 2 that results in $p' = (1, 6, 2, 3, 4, 5, 7)$. Element 6 becomes tabu-active, and therefore it is not allowed to participate in insertions for the next five iterations. However, other non-tabu-active elements may be inserted in either position 2 or position 1, causing element 6 to shift to position 3.

Long-Term Memory

In many applications, the short-term TS memory components are sufficient to produce very high-quality solutions. However, in general, TS becomes significantly stronger by including longer-term memory and its associated strategies. Long-term memory plays an important role in creating the right balance between intensification and diversification of a tabu search. Intensification strategies are based on modifying choice rules to encourage move combinations and solution features historically found good. They may also initiate a return to attractive regions to search them more thoroughly. Diversification on the other hand encourages the search process to examine unvisited regions and to generate solutions that differ in various significant ways from those seen before.

Long-term memory is often implemented using a frequency-based approach. Frequency-based memory provides a type of information that complements the information provided by recency-based memory, broadening the foundation for selecting preferred moves. Like recency, frequency often is weighted or decomposed into subclasses by considering solution quality.

Frequencies consist of ratios, whose numerators represent either *transition* counts or *residence* counts. A transition count is the number of iterations where an attribute changes (enters or leaves) the solutions visited. A residence count

is the number of iterations where an attribute belongs to solutions visited. The denominators generally represent one of three types of quantities: (1) the total number of occurrences of all events represented by the numerators (such as the total number of iterations), (2) the sum (or average) of the numerators, and (3) the maximum numerator value.

The use of longer-term memory does not require long solution runs before its benefits become visible. Often its improvements begin to be manifest in a relatively modest length of time and can allow solution efforts to be terminated somewhat earlier than otherwise possible, due to finding very high-quality solutions within an economical time span. The chance of finding still better solutions as time grows—in the case where an optimal solution has not already been found—is enhanced by using longer-term TS memory in addition to short-term memory.

Residence frequencies and transition frequencies sometimes convey related information but in general carry different implications. For example, residence measures, by contrast to transition measures, are not concerned with whether an attribute changes in moving from somewhat inferior solutions to better solutions. A high residence frequency may indicate that an attribute is highly attractive if the domain consists of high-quality solutions or may indicate the opposite, if the domain consists of low-quality solutions.

Frequency-based memory is used to define penalty and incentive values to modify the evaluation of moves and therefore determine which moves are selected. In a minimization problem, the modified move value function has the following mathematical form:

$$v' = v \times (1 + pq)$$

Here, v is the original move value, p is the penalty factor, q is the frequency ratio, and v' is the modified move value. When $p = 0$, the original move value is not modified. A value of $p > 0$ penalizes the move and v' makes it less attractive. A value of $p < 0$ provides an incentive, and v' makes the move more attractive.

Long-term memory is linked to the notion of *strongly determined* and *consistent* variables. A strongly determined variable is one that cannot change its value in a given high-quality solution without seriously degrading quality or feasibility, while a consistent variable is one that frequently takes on a specific value (or a highly restricted range of values) in good solutions. The development of useful measures of "strength" and "consistency" is critical to exploiting these notions, particularly by accounting for trade-offs determined by context. However, straightforward uses of frequency-based memory for keeping track of consistency, sometimes weighted by elements of quality and influence, have produced methods with very good performance outcomes.

For instance, a penalty/incentive function based on frequency ratios may be used in a restarting procedure to generate initial solutions based on penalty/incentive functions. The function may be designed to asses a penalty on consistent variables in order to induce diversification.

Table 6 Reordering of the variables with the modified bang-for-buck ratios

Variable	q	r	r'	Order by r'
1	0.28	13.40	9.65	6
2	1.00	11.11	0.00	1
3	0.38	10.89	6.75	3
4	0.39	10.53	6.42	4
5	0.47	10.00	5.30	10
6	0.00	9.75	9.75	7
7	0.39	9.09	5.55	5
8	0.50	8.70	4.35	9
9	0.41	8.57	5.06	8
10	0.16	7.14	6.00	2

Example 3

Suppose that a residency memory is added to the short-term memory TS for the knapsack problem described above. The memory accumulates the number of times each item is included in the knapsack. That is, the memory is a count of the number of times that a variable takes on the value of one. To create a frequency ratio, the numerator is defined as the number of iterations. The search is going to be restarted from a new initial solution. Restarting means that the short-term memory will be erased and the long-term memory will be used during the construction of the new solution. The construction is guided by the following modified bang-for-buck ratio:

$$r' = r \times (1 - q)$$

The original bang-for-buck ratio is r and q is the frequency ratio. In this penalty function, the implicit value of p is -1. The function penalizes items with high-frequency ratios (i.e., the consistent variables). In this example, long-term memory is being used to induce diversification by attempting to construct a solution that has not been visited so far. Table 6 shows the calculation of the modified bang-for-buck ratios for a given set of q values.

The last column in the table above shows the order in which the items will be considered to be included in the knapsack according to the modified ratio. This results in a solution with variables 6, 1, 3, 4, 10, and 7 set to 1. The total profit is 877 and the weight is 90. This solution, which has not been visited so far, becomes the restarting point for the search.

Strategic Oscillation

Strategic oscillation is closely linked to the origins of tabu search and provides a means to achieve an effective interplay between intensification and diversification

over the intermediate to long term. Strategic oscillation operates by orienting moves in relation to the feasibility boundary, which represents a point where a search would normally stop. Instead of stopping when this boundary is reached, however, the rules for selecting moves are modified, to permit the region defined by the feasibility boundary to be crossed. The approach then proceeds for a specified depth beyond the oscillation boundary and turns around. The oscillation boundary again is approached and crossed, this time from the opposite direction, and the method proceeds to a new turning point.

The process of repeatedly approaching and crossing the feasibility boundary from different directions creates an oscillatory behavior, which gives the method its name. Control over this behavior is established by generating modified evaluations and rules of movement, depending on the region navigated and the direction of search.

The implementation of strategic oscillation entails the selection of a proximity measure and the amplitude of the oscillation. The proximity measure assigns a numerical value to moving toward or away from the feasibility boundary. This measure serves as guidance to create the oscillation pattern. The amplitude determines the point at which the search is directed to turn around once it has crossed the feasibility boundary in either direction.

Proximity measures are defined in reference to the problem constraints. For instance, for a combinatorial optimization problem consisting of selecting k out of a given number elements, a proximity measure t may be the difference between the number of elements that have been selected and k. Feasible solutions are those for which $t = 0$. Solutions with $t > 0$ have too many elements and solutions with $t < 0$ have too few. Moves can be designed to keep the search at $t = 0$ or to create an oscillation pattern around $t = 0$.

The oscillation amplitude is controlled by a number of moves m. If a move is made that causes the search to cross the feasibility boundary, then $m - 1$ is the number of additional moves that the search is allowed to make in the same direction (i.e., moving away from the feasibility boundary) before turning around.

In combinatorial optimization problems that require the selection of k elements, the rule to delete elements from the solution will typically be different in character from the one used for adding elements. In other words, one rule is not simply the inverse of the other. Rule differences are features of strategic oscillation that provide an enhanced heuristic vitality. The application of different rules may be accompanied by crossing a boundary to different depths on different sides. An option is to approach and retreat from the boundary while remaining on a single side, without crossing (i.e., electing a crossing of "zero depth").

In both one-sided and two-sided oscillation approaches, it is frequently important to spend additional search time in regions close to the feasibility boundary and, especially, to spend time at the boundary itself. In problems for which feasibility is determined by a set of constraints, vector-valued functions can be used to control the oscillation. In this case, controlling the search by bounding this function can be viewed as manipulating a parameterization of the constraint set.

Example 4

The feasibility boundary in the knapsack example is defined by the capacity of the knapsack, which in the example introduced above is a total weight of 100. To create a simple oscillation around the feasible boundary, the search is allowed to cross to the infeasible region and m is set to 1. The proximity measure t is defined as the difference between the total weight of the items in the knapsack and the knapsack capacity. Therefore, $t > 0$ indicates an infeasible solution and $t \leqslant 0$ corresponds to feasible solutions. The procedure operates with the following rules:

- When approaching the feasibility boundary from the feasible region, select the best non-tabu feasible move. If no feasible move is available, then select the non-tabu move that improves profit the most
- When approaching the feasibility boundary from the infeasible region, select the non-tabu move that removes the variable with the smallest bang-for-buck ratio.

The short-term memory structure is the same as the one defined in the "Short-Term Memory" section. Therefore, the move selections have to be done by considering the tabu classifications. Table 7 shows ten iterations of TS with strategic oscillation. The initial solution is given by the bang-for-buck heuristic.

The oscillation can be observed in the Weight column. Every time that the weight exceeds 100, the search turns around and moves are made to decrease the weight. The same occurs when the weight falls below 100. It is noteworthy to discuss the move made in the last iteration. The move is selected even though the move is classified tabu (note that 3 appears in the tabu-active list). At iteration 10, the initial solution is the best feasible solution found so far, and it has an objective function value of 985. Therefore, an aspiration criterion that overrides the tabu status of a move can be invoked (as discussed in the "Short-Term Memory" section). The

Table 7 Iterations of TS with strategic oscillation

Iteration	Tabu-active	Move	Solution	Profit	Weight
0			(1, 2, 3, 4, 5)	985	90
1		6	(1, 2, 3, 4, 5, 6)	1297	122
2	6	5	(1, 2, 3, 4, 6)	1177	110
3	5 6	4	(1, 2, 3, 6)	977	91
4	4 5 6	8	(1, 2, 3, 6, 8)	1177	114
5	8 4 5	6	(1, 2, 3, 8)	865	82
6	6 8 4	9	(1, 2, 3, 8, 9)	1045	103
7	9 6 8	3	(1, 2, 8, 9)	947	94
8	3 9 6	4	(1, 2, 4, 8, 9)	1147	113
9	4 3 9	8	(1, 2, 4, 9)	947	90
10	8 4 3	3	(1, 2, 3, 4, 9)	1045	99

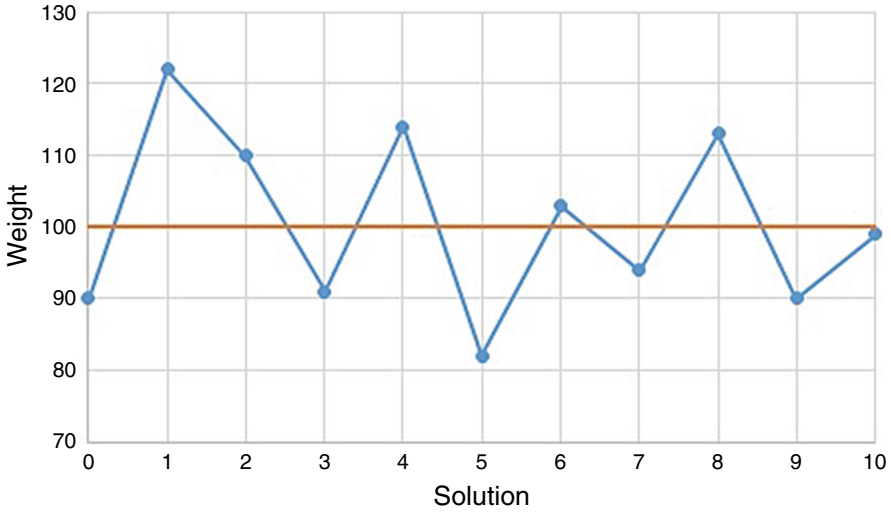


Fig. 1 Strategic oscillation around the knapsack capacity of 100

criterion states that the tabu classification can be overridden if the move leads to a solution that is better than the best solution found so far. Figure 1 shows a graphical representation of the oscillation pattern.

Example 5

The problem of maximizing diversity deals with selecting a subset of k elements from a given set of n elements in such a way that the diversity among the chosen elements is maximized. The best-known problem in this class deals with maximizing the sum of the diversity of the chosen elements. That is, if d_{ij} is the diversity measure between elements i and j, then the problem consists of maximizing $\sum_{i,j \in K} d_{ij}$, where K is the subset of chosen elements. The proximity measure t may be defined as the difference between the number of elements that have been selected and k, for an amplitude of m, t ranges between $-m$ and m, with $t = 0$ representing a feasible solution.

A swap move is defined as the exchange of an element currently in K with an element that is not currently in K. In this context, a swap neighborhood maintains feasibility as long as the starting solution is feasible (i.e., the solution includes k elements). In contrast, an add/delete neighborhood will not. A strategic oscillation may be defined as follows.

1. Construct a feasible solution K
2. Add elements $j \notin K$ that increases diversity the most. Elements are added one at a time until m is reached.

3. Delete elements $j \in K$ that decrease the diversity the least. Elements are deleted one at a time until m is reached.
4. Repeat 2 and 3 until a termination criterion is satisfied.

Short-term memory tabu structures may be embedded in this simple design in order to avoid cycling. For instance, elements that have been added may be declared tabu-active and are not allowed to be deleted during the next deletion cycle, and vice versa. Consider the diversity values in Table 8 for a problem with $k = 5$ and $n = 10$.

Let's assume that the starting solution is given by (1, 2, 3, 4, 5) with an objective function value of 45 and let $m = 2$. Table 9 shows 16 iterations of the strategic oscillation process, where a solution is defined by a set of 10 binary variables x_j:

Table 8 Instance of the maximum diversity problem

	1	2	3	4	5	6	7	8	9	10
1		4	5	0	3	2	0	2	4	2
2			7	9	2	6	4	8	4	6
3				3	4	0	8	7	6	4
4					8	7	5	5	4	1
5						4	8	6	2	1
6							6	5	4	5
7								0	6	8
8									6	1
9										3
10										

Table 9 Sixteen iterations of the strategic oscillation process

Iter	x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	x_9	x_{10}	Move	t	MaxSum
0	1	1	1	1	1	0	0	0	0	0	+8	0	45
1	1	1	1	1	1	0	0	1	0	0	+9	1	73
2	1	1	1	1	1	0	0	1	1	0	−1	2	99
3	0	1	1	1	1	0	0	1	1	0	−5	1	81
4	0	1	1	1	0	0	0	1	1	0	−9	0	59
5	0	1	1	1	0	0	0	1	0	0	−3	−1	39
6	0	1	0	1	0	0	0	1	0	0	+6	−2	22
7	0	1	0	1	0	1	0	1	0	0	+5	−1	40
8	**0**	**1**	**0**	**1**	**1**	**1**	**0**	**1**	**0**	**0**	**+7**	**0**	**60**
9	0	1	0	1	1	1	1	1	0	0	+3	1	83
10	0	1	1	1	1	1	1	1	0	0	−6	2	112
11	0	1	1	1	1	0	1	1	0	0	−8	1	84
12	0	1	1	1	1	0	1	0	0	0	−2	0	58
13	0	0	1	1	1	0	1	0	0	0	−4	−1	36
14	0	0	1	0	1	0	1	0	0	0	+9	−2	20
15	0	0	1	0	1	0	1	0	1	0	+10	−1	34
16	0	0	1	0	1	0	1	0	1	1		0	50

The first column (Iter) shows the iteration number, where 0 is the initial solution. The next 10 columns show the values of the binary variables, where a value of 1 indicates that the element has been chosen. The Move column indicates the index of the element that enters (plus sign) or leaves (minus sign) the solution. The proximity measure is given in the column labeled t. The objective function value is shown in the last column. The solution at iteration 8 is the best feasible solution, with a MaxSum value of 60 and $t = 0$. Note that an aspiration criterion is being used to override the tabu status. In particular, the tabu status of a move is overridden if it leads to a solution with an objective function value that is the best visited so far for the corresponding t value. For instance, element 3 is added at iteration 9 even though this element is tabu (since it was deleted in the last deletion cycle). However, the addition of 3 results in the solution at iteration 10 that has the best objective function value of all those solutions with $t = 2$ that have been visited so far. The same logic is applied to allow the deletion of element 6 at iteration 10.

Path Relinking

A useful integration of intensification and diversification strategies occurs in the approach called *path relinking*. This approach generates new solutions by exploring trajectories that connect reference solutions—by starting from one of these solutions, called an *initiating solution*, and generating a path in the neighborhood space that leads toward the other solutions, called *guiding solutions*. This is accomplished by selecting moves that introduce attributes contained in the guiding solutions.

Path relinking is a strategy that seeks to incorporate attributes of high-quality solutions, by creating inducements to favor these attributes in the moves selected. However, instead of using an inducement that merely encourages the inclusion of such attributes, the path relinking approach subordinates all other considerations to the goal of choosing moves that introduce the attributes of the guiding solution, in order to create a "good attribute composition" in the current solution.

The procedure, at each step, chooses the best move, according to the change in the objective function value, from the restricted set of moves that incorporate a maximum number of the attributes of the guiding solutions. As in other applications of TS, aspiration criteria can override this restriction to allow other moves of particularly high quality to be considered.

Membership in the reference set is determined by setting a threshold which is connected to the objective function value of the best solution found during the search. For example, reference solutions might be those whose objective function value is within 10% of the best objective function value known so far.

Path relinking can be used to generate intensification strategies by choosing reference solutions that lie in a common region or that share common features. Similarly, diversification strategies based on path relinking characteristically select reference solutions that come from different regions or that exhibit contrasting features. The initiating solution can be used to give a beginning partial construction, by specifying particular attributes as a basis for remaining constructive steps.

Constructive neighborhoods can be viewed as a feasibility-restoring mechanism since a null or partially constructed solution does not satisfy all conditions to qualify as feasible. Similarly, path relinking can make use of destructive neighborhoods, where an initial solution is "overloaded" with attributes donated by the guiding solutions, and such attributes are progressively stripped away or modified until reaching a set with an appropriate composition. Destructive neighborhoods represent an instance of a feasibility-restoring function, as where an excess of elements may violate explicit problem constraints.

Path relinking consists of the following steps:

(a) Identify the neighborhood structure and associated solution attributes for path relinking (possibly different from those of other TS strategies applied to the problem).
(b) Select a collection of two or more reference solutions, and identify which members will serve as the initiating solution and the guiding solution(s). (Reference solutions can be infeasible, such as "incomplete" or "overloaded" solution components treated by constructive or destructive neighborhoods.)
(c) Move from the initiating solution toward the guiding solution(s), generating one or more intermediate solutions.

Example 6

Consider the two reference solutions in Table 10.

These solutions were found with a TS that used simple moves consisting of flipping one variable at a time. Both of these solutions are feasible. Suppose that solution 2 is chosen as the initiating solution and that solution 1 is the guiding solution. The relinking process starts by identifying the values that are common to both solutions. These correspond to variables 2, 3, and 5. Swap moves are used during the path relinking process in order to keep the intermediate solutions close to the feasibility boundary. A swap in this context corresponds to adding one item to the knapsack while taking another item out of the knapsack. This means that a variable that is set to 1 in the initiating solution and to 0 in the guiding solution must be paired with a variable that is set to 0 in the initiating solution and to 1 in the guiding solution. The pairs in our example are (1, 7), (1, 9), (4, 7), and (4, 9). Table 11 shows the intermediate solutions that result from these swaps:

Table 10 Two reference solutions

Solution	x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	x_9	x_{10}	Profit	Weight
1	0	1	1	0	1	0	1	0	1	0	998	98
2	1	1	1	1	1	0	0	0	0	0	985	90

Table 11 Swaps in the path relinking process of Example 6

Swap	x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	x_9	x_{10}	Profit	Weight
(1, 7)	0	1	1	1	1	0	1	0	0	0	1018	96
(1, 9)	0	1	1	1	1	0	0	0	1	0	1098	106
(4, 7)	1	1	1	0	1	0	1	0	0	0	885	82
(4, 9)	1	1	1	0	1	0	0	0	1	0	965	92

The best feasible swap is (1, 7), and the search moves to the intermediate solution with profit of 1018 and weight of 96. After the move, there is only one swap left to reach the guiding solution, namely, (4, 9). In this case, the process produced only one intermediate solution; however, in general, path relinking visits more solutions during the transformation of the initiating solution into the guiding solution.

Conclusions

This chapter presented the essential elements of tabu search, namely, short-term memory, long-term memory, strategic oscillation, and path relinking. Attention was given to these core components of the tabu search framework for the following reasons:

- Such a focus may help to uncover a better form for the strategies associated with these core components.
- Weaknesses and strengths of these core components, when studied in isolation from other ideas, may stand out more clearly, thus yielding insights into the features that a complete approach may require to produce better methods.
- For methods which are susceptible to highly modular implementations, as typically occurs for tabu search, simpler designs can readily be made a part of more complex designs.

While TS implementations using a combinations of these elements are likely to produce decent outcomes, readers are encouraged to pursue more complete approaches by examining strategies outlined in the publications referenced below, in particular those in the Tabu Search book [4].

A great deal remains to be learned about tabu search. Evidently, very little is known about how human beings use memory in problem-solving. It is not inconceivable that discoveries about effective uses of memory within search methods will provide clues about strategies that humans skillfully employ. The potential links between the areas of heuristic search and psychology have barely been examined. The numerous successes of tabu search implementations provide encouragement that such relationships might be profitable to investigate more fully.

Cross-References

- ▶ Adaptive and Multi-level Metaheuristics
- ▶ Fundamentals of Scatter Search
- ▶ Theory of Local Search

Competing Interest Declaration The author(s) has no competing interests to declare that are relevant to the content of this manuscript.

References

1. Glover F (1977) Heuristics for integer programming using surrogate constraints. Decis Anal 8:156–166
2. Glover F (1986) Future paths for integer programming and links to artificial intelligence. Comput Oper Res 13:533–549
3. Glover F (1990) Tabu search: a tutorial. Interfaces 20(4):74–94
4. Glover F, Laguna M (1997) Tabu search. Springer, New York
5. Glover F, Laguna M (2002) Tabu search. In: Pardalos PM, Resende MGC (eds) Handbook of applied optimization. Oxford University Press, New York, pp 194–208
6. Glover F, Laguna M (2013) Tabu search. In: Du DZ, Pardalos PM (eds) Handbook of combinatorial optimization, 2nd edn. Springer, New York, pp 3261–3362
7. Glover F, Laguna M, Martí R (2007) Principles of tabu search, Chapter 23. In: Gonzalez T (ed) Approximation algorithms and metaheuristics. Chapman & Hall/CRC, New York
8. Schneider U (2010) A tabu search tutorial based on a real-world scheduling problem. Cent Eur J Oper Res 19(4):467–493

Variable Neighborhood Search

32

Pierre Hansen and Nenad Mladenović

Contents

Introduction 960
Basic Schemes 961
Some Extensions 969
VNS for Mixed-Integer Linear Programming 974
VNS for Continuous Global Optimization 977
Variable Neighborhood Programming 979
Discovery Science 983
Conclusions 984
References 986

Abstract

Variable neighborhood search (VNS) is a metaheuristic for solving combinatorial and global optimization problems. Its basic idea is systematic change of neighborhood both within a descent phase to find a local optimum and in a perturbation phase to get out of the corresponding valley. In this chapter we present the basic schemes of variable neighborhood search and some of its extensions. We next present four families of applications of VNS in which it has proved to be very successful: (i) finding feasible solutions to large mixed-integer linear programs, by hybridization of VNS and local branching, (ii) finding good feasible solutions

P. Hansen (✉)
GERAD and Ecole des Hautes Etudes Commerciales, Montréal, QC, Canada
e-mail: pierreh@crt.umontreal.ca

N. Mladenović
GERAD and Ecole des Hautes Etudes Commerciales, Montréal, QC, Canada

LAMIH, Université de Valenciennes, Valenciennes, France

Mathematical Institute, SANU, Belgrade, Serbia

© Springer Nature Switzerland AG 2025

R. Martí et al. (eds.), *Handbook of Heuristics*,
https://doi.org/10.1007/978-3-032-00385-0_19

to continuous nonlinear programs, (iii) finding programs in automatic fashion (artificial intelligence field) by building *variable neighborhood programming* methodology, and (iv) exploring graph theory in order to find conjectures, refutations, and proofs or ideas of proofs.

Keywords

Optimization · Metaheuristics · Artificial intelligence · Variable neighborhood search

Introduction

Optimization tools have greatly improved during the last two decades. This is due to several factors: (i) progress in mathematical programming theory and algorithmic design, (ii) rapid improvement in computer performances, and (iii) better communication of new ideas and integration of them in largely used complex softwares. Consequently, many problems long viewed as out of reach are currently solved, sometimes in very moderate computing times. This success, however, has led to address much larger instances and more difficult classes of problems. Many of these may again only be solved heuristically. Therefore thousands of papers describing, evaluating, and comparing new heuristics appear each year. Keeping abreast of such a large literature is a challenge. Metaheuristics or general frameworks for building heuristics are therefore needed in order to organize the study of heuristics. As evidenced by this handbook, there are many of them. Some desirable properties of metaheuristics [24, 27, 29] are listed in the concluding section of this chapter.

Variable neighborhood search (VNS) is a metaheuristic proposed by some of the present authors a dozen years ago [40]. Earlier work that motivated this approach is given in [8, 13, 18, 38]. It is based upon the idea of systematic change of neighborhood both in a descent phase to find a local optimum and in a perturbation phase to get out of the corresponding valley. Originally designed for approximate solution of combinatorial optimization problems, it was extended to address, nonlinear programs, and recently mixed-integer nonlinear programs. In addition VNS has been used as a tool for automated or computer-assisted graph theory. This led to the discovery of over 1500 conjectures in that field, the automated proof of more than half of them as well as the unassisted proof of about 400 of them by many mathematicians. Moreover, VNS methodology is recently applied in artificial intelligence as automatic programming technique.

Applications are rapidly increasing in number and pertain to many fields: location theory, cluster analysis, scheduling, vehicle routing, network design, lot sizing, artificial intelligence, engineering, pooling problems, biology, phylogeny, reliability, geometry, telecommunication design, etc. References are too numerous to be listed here, but many of them can be found in [25, 28, 29] and in the following special issues devoted to VNS: *IMA Journal of Management Mathematics* [37], *European*

Journal of Operational Research [29], *Journal of heuristics* [47], *Computers and Operations Research* [45] and *Journal of Global Optimization* [11].

This chapter is organized as follows. In the next section we present the basic schemes of VNS, i.e., variable neighborhood descent (VND), reduced VNS (RVNS), basic VNS (BVNS), and general VNS (GVNS). In addition, to the list of basic schemes, we add skewed VNS (SVNS) and variable neighborhood decomposition search (VNDS). Three important extensions are presented in section "Some Extensions": primal-dual VNS (PD-VNS), formulation space search (FSS), and recently developed variable formulation search (VFS). The remainder of the paper describes applications of VNS to several classes of large-scale and complex optimization problems for which it has proven to be particularly successful. Finding feasible solutions to large mixed-integer linear programs with VNS is discussed in section "VNS for Mixed-Integer Linear Programming." Section "VNS for Continuous Global Optimization" addresses ways to apply VNS in continuous global optimization. Recent application of VNS in *automatic programming* field is discussed in section "Variable Neighborhood Programming." Applying VNS to graph theory per se (and not just to particular optimization problems defined on graphs) is discussed in section "Discovery Science." Conclusions are drawn in section "Conclusions." There the desirable properties of any metaheuristic are listed as well.

Basic Schemes

A deterministic optimization problem may be formulated as

$$\min \{f(x) | x \in X, X \subseteq \mathcal{S}\}, \tag{1}$$

where $\mathcal{S}$, X, x, and f, respectively, denote the *solution space* and *feasible set*, a *feasible solution*, and a real-valued *objective function*. If $\mathcal{S}$ is a finite but large set, a *combinatorial optimization* problem is defined. If $\mathcal{S} = \mathbb{R}^n$, we refer to *continuous optimization*. A solution $x^* \in X$ is *optimal* if

$$f\left(x^*\right) \leq f(x), \quad \forall x \in X.$$

An *exact algorithm* for problem (1), if one exists, finds an optimal solution x^*, together with the proof of its optimality, or shows that there is no feasible solution, i.e., $X = \varnothing$, or the solution is unbounded. Moreover, in practice, the time needed to do so should be finite (and not too long). For continuous optimization, it is reasonable to allow for some degree of tolerance, i.e., to stop when sufficient convergence is detected.

Let us denote with $\mathcal{N}_k$, $(k = 1, \ldots, k_{\max})$, a finite set of preselected neighborhood structures, and with $\mathcal{N}_k(x)$ the set of solutions in the kth neighborhood of x. Most local search heuristics use only one neighborhood structure, i.e., $k_{\max} = 1$. Neighborhoods $\mathcal{N}_k$ may be induced from one or more metric (or quasi-metric)

functions introduced into a solution space S. An *optimal solution* x_{opt} (or global minimum) is a feasible solution where a minimum is reached. We call $x' \in X$ a *local minimum* of (1) with respect to $\mathcal{N}_k$ (w.r.t. $\mathcal{N}_k$ for short), if there is no solution $x \in \mathcal{N}_k(x') \subseteq X$ such that $f(x) < f(x')$. Metaheuristics (based on local search procedures) try to continue the search by other means after finding the first local minimum. VNS is based on three simple facts:

Fact 1

A local minimum w.r.t. one neighborhood structure is not necessarily so for another;

Fact 2

A global minimum is a local minimum w.r.t. all possible neighborhood structures;

Fact 3

Local minima w.r.t. one or several $\mathcal{N}_k$ are relatively close to each other, for many problems.

This last observation, which is empirical, implies that a local optimum often provides some information about the global one. This may for instance be several variables with the same value in both. However, it is usually not known which ones are such. An organized study of the neighborhoods of this local optimum is therefore in order, until a better solution is found.

In order to solve (1) by using several neighborhoods, facts 1–3 can be used in three different ways: (i) deterministic, (ii) stochastic, and (iii) both deterministic and stochastic. We first give in Algorithm 1 the steps of the neighborhood change function that will be used later.

Algorithm 1: Neighborhood change

Function `NeighborhoodChange` (x, x', k);
1 **if** $f(x') < f(x)$ **then**
2 $\quad x \leftarrow x'$; $k \leftarrow 1$ // Make a move
else
3 $\quad k \leftarrow k + 1$ // Next neighborhood

Function `NeighborhoodChange()` compares the new value $f(x')$ with the incumbent value $f(x)$ obtained from the kth neighborhood (line 1). If an improvement is obtained, k is returned to its initial value and the new incumbent updated (line 2). Otherwise, the next neighborhood is considered (line 3).

(i) The *variable neighborhood descent* (VND) method is obtained if a change of neighborhoods is performed in a deterministic way. Its steps are presented in Algorithm 2, where neighborhoods are denoted as N_ℓ, $\ell = 1, \ell_{\max}$.

Algorithm 2: Steps of the basic VND

Function VND $(x, \ell_{\max})$
1 $\ell \leftarrow 1$
2 **repeat**
3 $\quad x' \leftarrow \arg\min_{y \in N_\ell(x)} f(x)$ // Find the best neighbor in $N_\ell(x)$
4 $\quad$ NeighborhoodChange (x, x', ℓ) // Change neighborhood
until $\ell = \ell_{\max}$

Most local search heuristics use in their descents a single or sometimes two neighborhoods ($\ell_{\max} \leq 2$). Note that the final solution should be a local minimum w.r.t. all $\ell_{\max}$ neighborhoods, and thus chances to reach a global one are larger than by using a single structure. Besides this *sequential* order of neighborhood structures in VND above, one can develop a *nested* strategy. Assume, e.g., that $\ell_{\max} = 3$; then a possible nested strategy is: perform VND from Algorithm 2 for the first two neighborhoods from each point x' that belongs to the third ($x' \in N_3(x)$). Such an approach is successfully applied in [5, 7, 25].

(ii) The *reduced VNS* (RVNS) method is obtained if random points are selected from $\mathcal{N}_k(x)$ and no descent is made. Rather, the values of these new points are compared with that of the incumbent, and updating takes place in case of improvement. We assume that a stopping condition has been chosen, among various possibilities, e.g., the maximum CPU time allowed $t_{\max}$, or the maximum number of iterations between two improvements. To simplify the description of the algorithms, we always use $t_{\max}$ below. Therefore, RVNS uses two parameters: $t_{\max}$ and $k_{\max}$. Its steps are presented in Algorithm 3.

Algorithm 3: Steps of the reduced VNS

Function RVNS$(x, k_{\max}, t_{\max})$
1 **repeat**
2 $\quad k \leftarrow 1$
3 $\quad$ **repeat**
4 $\quad\quad x' \leftarrow$ Shake(x, k)
5 $\quad\quad$ NeighborhoodChange (x, x', k)
$\quad$ **until** $k = k_{\max}$
6 $\quad t \leftarrow$ CpuTime ()
until $t > t_{\max}$

With the function Shake represented in line 4, we generate a point x' at random from the kth neighborhood of x, i.e., $x' \in \mathcal{N}_k(x)$. Its steps are given in Algorithm 4,

where it is assumed that points from $\mathcal{N}_k(x)$ are $\{x^1, \ldots, x^{|\mathcal{N}_k(x)|}\}$. RVNS is useful for very large instances for which local search is costly. It can be used as well for finding initial solutions for large problems before decomposition. It has been observed that the best value for the parameter $k_{\max}$ is often 2 or 3. In addition, a maximum number of iterations between two improvements is usually used as the stopping condition. RVNS is akin to a Monte Carlo method, but is more systematic (see, e.g., [42] where results obtained by RVNS were 30% better than those of the Monte Carlo method in solving a continuous min-max problem). When applied to the p-median problem, RVNS gave equally good solutions as the *fast interchange* heuristic of [56] while being 20–40 times faster [30].

Algorithm 4: Steps of the shaking function

Function `Shake`(x, x', k)
1 $w \leftarrow [1+\texttt{Rand}(0, 1) \times |\mathcal{N}_k(x)|]$
2 $x' \leftarrow x^w$

(iii) The *basic VNS* (VNS) method [40] combines deterministic and stochastic changes of neighborhood. The deterministic part is represented by a local search heuristic. It consists in (a) choosing an initial solution x, (b) finding a direction of descent from x (within a neighborhood $N(x)$), and (c) moving to the minimum of $f(x)$ within $N(x)$ along that direction. If there is no direction of descent, the heuristic stops, and otherwise it is iterated. Usually the steepest descent direction, also referred to as *best improvement*, is used (`BestImprovement`). This set of rules is summarized in Algorithm 5, where we assume that an initial solution x is given. The output consists of a local minimum, also denoted by x, and its value.

Algorithm 5: Best improvement (steepest descent) heuristic

Function `BestImprovement`(x)
1 **repeat**
2 $\quad x' \leftarrow x$
3 $\quad x \leftarrow \arg\min_{y \in N(x)} f(y)$
until $(f(x) \geq f(x'))$

As *steepest descent* heuristic may be time-consuming, an alternative is to use the *first descent* heuristic. Vectors $x_i \in N(x)$ are then enumerated systematically and a move is made as soon as a direction for the descent is found. This is summarized in Algorithm 6.

Algorithm 6: First improvement (first descent) heuristic

```
Function FirstImprovement(x)
1  repeat
2  |  x' ← x; i ← 0
3  |  repeat
4  |  |  i ← i + 1
5  |  |  x ← arg min{f(x), f(x_i)}, x_i ∈ N(x)
   |  until (f(x) < f(x_i) or i = |N(x)|)
   until (f(x) ≥ f(x'))
```

The stochastic phase is represented by random selection of one point from the k-th neighborhood. The steps of the BVNS are given on Algorithm 7.

Algorithm 7: Steps of the basic VNS

```
Function BVNS(x, k_max, t_max)
1  t ← 0
2  while t < t_max do
3  |  k ← 1
4  |  repeat
5  |  |  x' ← Shake(x, k)                  // Shaking
6  |  |  x'' ← BestImprovement(x')          // Local search
7  |  |  NeighborhoodChange(x, x'', k)     //Change neighborhood
   |  until k = k_max
8  |  t ← CpuTime()
```

Often successive neighborhoods $\mathcal{N}_k$ are nested. Note that point x' is generated at random in Step 4 in order to avoid cycling, which might occur if a deterministic rule were applied. Basic VNS is also illustrated in Fig. 1.

Example. We illustrate the basic step on a minimum k-cardinality tree instance taken from [34] (see Fig. 2). The minimum k-cardinality tree problem on graph G (k-card for short) consists in finding a sub-tree of G with exactly k edges whose sum of weights is minimum.

The steps of BVNS for solving the 4-card problem are illustrated in Fig. 3. In Step 0 the objective function value, i.e., the sum of edge weights, is equal to 40; it is indicated in the right bottom corner of the figure. That first solution is a local minimum with respect to the edge-exchange neighborhood structure (one edge in, one out). After shaking, the objective function is 60, and after another local search, we are back to the same solution. Then, in Step 3, we take out 2 edges and add another 2 at random, and after a local search, an improved solution is obtained with

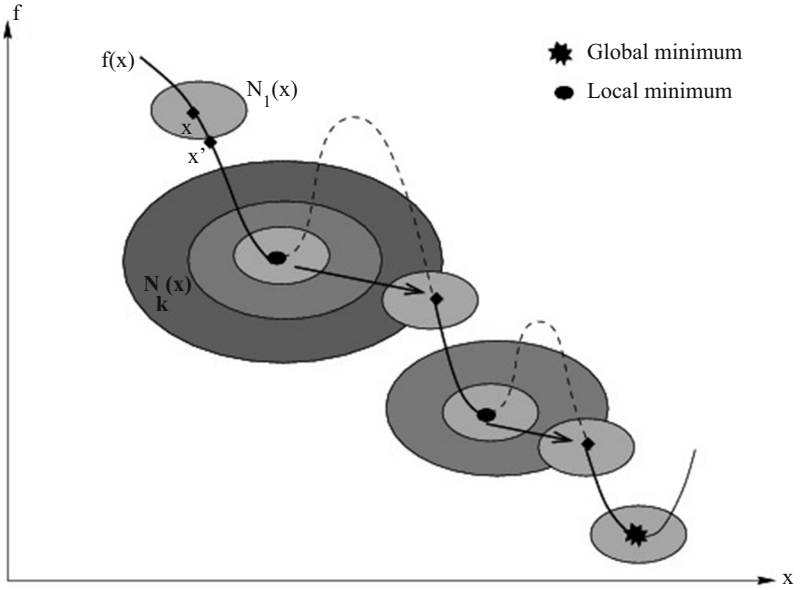


Fig. 1 Basic VNS

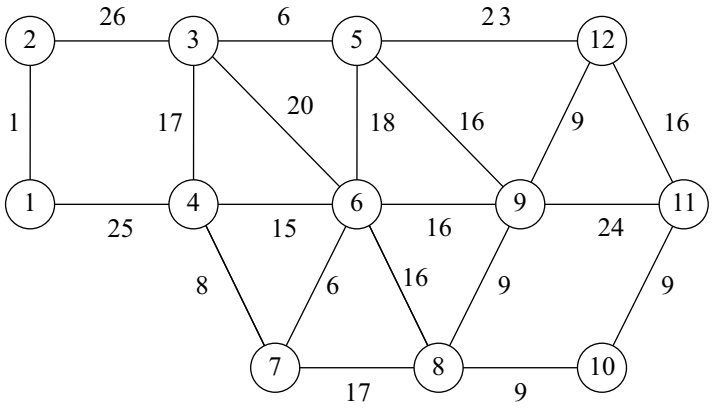


Fig. 2 4-cardinality tree problem

a value of 39. Continuing in that fashion, we obtain in Step 8 the optimal solution with an objective function value equal to 36.

(iv) *General VNS.* Note that the local search Step 5 may also be replaced by VND (Algorithm 2). Using this general VNS (VNS/VND) approach led to some of the most successful applications reported (see, e.g., [1, 7, 9, 10, 25, 31, 52, 53]). The steps of general VNS (GVNS) are given in Algorithm 8 below.

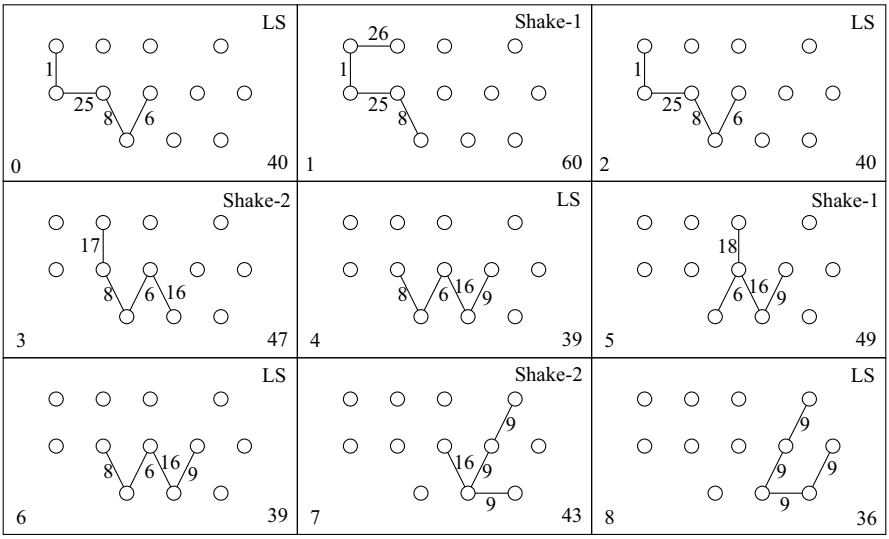


Fig. 3 Steps of the basicVNS for solving 4-card tree problem

Algorithm 8: Steps of the general VNS

Function GVNS $(x, \ell_{\max}, k_{\max}, t_{\max})$
repeat
  $k \leftarrow 1$
  repeat
    $x' \leftarrow$ Shake(x, k)
    $x'' \leftarrow$ VND$(x', \ell_{\max})$
    NeighborhoodChange(x, x'', k)
  until $k = k_{\max}$
  $t \leftarrow$ CpuTime()
until $t > t_{\max}$

(v) The *skewed VNS* (SVNS) method [23] addresses the problem of exploring valleys far from the incumbent solution. Indeed, once the best solution in a large region has been found, it is necessary to go quite far to obtain an improved one. Solutions drawn at random in faraway neighborhoods may differ substantially from the incumbent, and VNS may then degenerate, to some extent, into the multistart heuristic (in which descents are made iteratively from solutions generated at random, and that is known not to be very efficient). So some compensation for distance from the incumbent must be made, and a scheme called skewed VNS is proposed for that purpose. Its steps are presented in

Algorithms 9, 10, and 11. The KeepBest(x, x') function in Algorithm 10 simply keeps the better one of solutions x and x'.

Algorithm 9: Steps of neighborhood change for the skewed VNS

Function NeighborhoodChangeS(x, x'', k, α)
if $f(x'') - \alpha\rho(x, x'') < f(x)$ **then**
  $x \leftarrow x''$; $k \leftarrow 1$
else
  $k \leftarrow k + 1$

Algorithm 10: Steps of the skewed VNS

Function SVNS ($x, k_{\max}, t_{\max}, \alpha$)
repeat
  $k \leftarrow 1$; $x_{\text{best}} \leftarrow x$
  repeat
    $x' \leftarrow$ Shake(x, k)
    $x'' \leftarrow$ FirstImprovement(x')
    KeepBest (x_{best}, x)
    NeighborhoodChangeS(x, x'', k, α)
  until $k = k_{\max}$
  $x \leftarrow x_{\text{best}}$
  $t \leftarrow$ CpuTime()
until $t > t_{\max}$

Algorithm 11: Keep the better solution

Function KeepBest(x, x');
if $f(x') < f(x)$ **then**
  $x \leftarrow x'$

SVNS makes use of a function $\rho(x, x'')$ to measure distance between the incumbent solution x and the local optimum found x''. The distance function used to define $\mathcal{N}_k$, as in the above examples, could be used also for this purpose. The parameter α must be chosen in such a way to accept movement to valleys faraway from x when $f(x'')$ is larger than $f(x)$ but not too much larger (otherwise one will always leave x). A good value for α is to be found experimentally in each case. Moreover, in order to avoid frequent moves from x to a close solution, one may take a smaller value for α when $\rho(x, x'')$ is small. More sophisticated choices for a function of $\alpha\rho(x, x'')$ could be made through some learning process.

(vi) *Variable neighborhood decomposition search* (VNDS). The VNDS method [30] extends the basic VNS into a two-level VNS scheme based upon decomposition of the problem. Its steps are presented in Algorithm 12, where t_d is an additional parameter and represents the running time given for solving decomposed (smaller sized) problems by VNS.

Algorithm 12: Steps of VNDS

Function VNDS $(x, k_{\max}, t_{\max}, t_d)$
1 **repeat**
2 $\quad k \leftarrow 1$
3 $\quad$ **repeat**
4 $\quad\quad x' \leftarrow$ Shake (x, k); $y \leftarrow x' \setminus x$
5 $\quad\quad y' \leftarrow$ VNS(y, k, t_d); $x'' = (x' \setminus y) \cup y'$
6 $\quad\quad x''' \leftarrow$ FirstImprovement(x'')
7 $\quad\quad$ NeighborhoodChange(x, x''', k)
$\quad$ **until** $k = k_{\max}$
until $t > t_{\max}$

For ease of presentation, but without loss of generality, we assume that the solution x represents the set of some elements. In Step 4 we denote with y a set of k solution attributes present in x' but not in x ($y = x' \setminus x$). In Step 5 we find the local optimum y' in the space of y; then we denote with x'' the corresponding solution in the whole space S ($x'' = (x' \setminus y) \cup y'$). We notice that exploiting some *boundary effects* in a new solution can significantly improve the solution quality. That is why, in Step 6, we find the local optimum x''' in the whole space S using x'' as an initial solution. If this is time-consuming, then at least a few local search iterations should be performed.

VNDS can be viewed as embedding the classical successive approximation scheme (which has been used in combinatorial optimizationat least since the 1960s; see, e.g., [21]) in the VNS framework.

Some Extensions

(i) *Primal-dual VNS*. For most modern heuristics, the difference in value between the optimal solution and the obtained one is completely unknown. Guaranteed performance of the primal heuristic may be determined if a lower bound on the objective function value is known. To that end, the standard approach is to relax the integrality condition on the primal variables, based on a mathematical programming formulation of the problem. However, when the dimension of the problem is large, even the relaxed problem may be impossible to solve exactly by standard commercial solvers. Therefore, it seems a good idea to solve dual relaxed problems heuristically as well. In this way we get guaranteed bounds

on the primal heuristic's performance. The next problem arises if we want to get an exact solution within a branch-and-bound framework since having the approximate value of the relaxed dual does not allow us to branch in an easy way, e.g., exploiting complementary slackness conditions. Thus, the exact value of the dual is necessary.

In primal-dual VNS (PD-VNS) [22] one possible general way to get both the guaranteed bounds and the exact solution is proposed. Its steps are given in Algorithm 13. In the first stage a heuristic procedure based on VNS is used to obtain a near-optimal solution. In [22] it is shown that VNS with decomposition is a very powerful technique for large-scale simple plant location problems (SPLP) with up to 15,000 facilities and 15,000 users. In the second phase the objective is to find an exact solution of the relaxed dual problem. Solving the relaxed dual is accomplished in three stages: (i) find an initial dual solution (generally infeasible) using the primal heuristic solution and complementary slackness conditions, (ii) find a feasible solution by applying VNS to the unconstrained nonlinear form of the dual, and (iii) solve the dual exactly starting with initial feasible solution using a customized "sliding simplex" algorithm that applies "windows" on the dual variables substantially reducing the size of the problem. In all problems tested, including instances much larger than previously reported in the literature, the procedure was able to find the exact dual solution in reasonable computing time. In the third and final phase armed with tight upper and lower bounds, obtained respectively, from the heuristic primal solution in phase one and the exact dual solution in phase two, we apply a standard branch-and-bound algorithm to find an optimal solution of the original problem. The lower bounds are updated with the dual sliding simplex method and the upper bounds whenever new integer solutions are obtained at the nodes of the branching tree. In this way it was possible to solve exactly problem instances of sizes up to 7000 × 7000 for uniform fixed costs and 15,000 × 15,000 otherwise.

Algorithm 13: Steps of the basic PD-VNS

Function `PD-VNS` $(x, \ell_{\max}, k_{\max}, t_{\max})$
1 `BVNS` $(x, \ell_{\max}, k_{\max}, t_{\max})$ // Solve primal by VNS
2 `DualFeasible`(x, y) // Find (infeasible) dual such that $f_P = f_D$
3 `DualVNS`(y) // Use VNS do decrease infeasibility
4 `DualExact`(y) // Find exact (relaxed) dual
5 `BandB`(x, y) // Apply branch-and-bound method

(ii) *Variable neighborhood formulation space search*. Traditional ways to tackle an optimizationproblem consider a given formulation and search in some way through its feasible set X. The fact that a same problem may often be formulated in different ways allows to extend search paradigms to include jumps from one

formulation to another. Each formulation should lend itself to some traditional search method, its "local search" that works totally within this formulation, and yields a final solution when started from some initial solution. Any solution found in one formulation should easily be translatable to its equivalent solution in any other formulation. We may then move from one formulation to another using the solution resulting from the former's local search as initial solution for the latter's local search. Such a strategy will of course only be useful when local searches in different formulations behave differently.

This idea was recently investigated in [43] using an approach that systematically changes formulations for solving circle packing problems (CPP). It is shown there that a stationary point of a nonlinear programming formulation of CPP in Cartesian coordinates is not necessarily also a stationary point in a polar coordinate system. A method called *reformulation descent* (RD) that alternates between these two formulations until the final solution is stationary with respect to both is suggested. Results obtained were comparable with the best known values, but they were achieved some 150 times faster than by an alternative single formulation approach. In the same paper the idea suggested above of *formulation space search* (FSS) is also introduced, using more than two formulations. Some research in that direction has been reported in [32, 43, 44, 50]. One methodology that uses variable neighborhood idea in searching through the formulation space is given in Algorithms 14 and 15. Here ϕ (ϕ') denotes a formulation from given space $\mathcal{F}$, x (x') denotes a solution in the feasible set defined with that formulation, and $\ell \leq \ell_{\max}$ is the formulation neighborhood index. Note that Algorithm 15 uses a reduced VNS strategy in $\mathcal{F}$.

Algorithm 14: Formulation change function

Function `FormulationChange`$(x, x', \phi, \phi', \ell)$;
if $f(\phi', x') < f(\phi, x)$ **then**
  $\phi \leftarrow \phi'$; $x \leftarrow x'$; $\ell \leftarrow \ell_{\min}$
else
  $\ell \leftarrow \ell + \ell_{\text{step}}$

Algorithm 15: Reduced variable neighborhood FSS

Function `VNFSS`$(x, \phi, \ell_{\max})$
repeat
  $\ell \leftarrow 1$ // Initialize formulation in $\mathcal{F}$
  while $\ell \leq \ell_{\max}$ **do**
    `ShakeFormulation`(x,x',ϕ,ϕ',ℓ) // $(\phi',x') \in (N_\ell(\phi),\mathcal{N}(x))$ at random
    `FormulationChange`(x,x',ϕ,ϕ',ℓ) // Change formulation
until *some stopping condition is met*

(iii) *Variable formulation search*. Many optimization problems in the literature, like min-max type of problems, present a flat landscape. This means that, given a formulation of the problem, there are many neighboring solutions with the same value of the objective function. When this happens, it is difficult to determine which neighborhood solution is more promising to continue the search. To address this drawback, the use of alternative formulations of the problem within VNS is proposed in [41, 46, 49]. In [49] it is named variable formulation search (VFS). It combines the change of neighborhood within the VNS framework, with the use of alternative formulations. In particular, the alternative formulations will be used to compare different solutions with the same value of the objective function, when considering the original formulation.

Let us assume that, besides the original formulation and the corresponding objective function $f_0(x)$, there are p other formulations denoted as $f_1(x), \ldots, f_p(x)$, $x \in X$. Note that two formulations are equivalent if the optimal solution of one is the optimal solution of the other and vice versa. Without loss of clarity, we will denote different formulations as different objectives $f_i(x)$, $i = 1, \ldots, p$. The idea of VFS is to add the procedure `Accept`(x, x', p), given in Algorithm 16 in all three basic steps of BVNS: `Shaking`, `LocalSearch`, and `NeighbourhoodChange`. Clearly, if a better solution is not obtained by any formulation among the p preselected, the move is rejected. The next iteration in the loop of Algorithm 16 will take place only if the objective function values according to all previous formulations are equal.

Algorithm 16: Accept procedure with p secondary formulations

```
1: logical function Accept (x, x', p)
2: for i = 0 to p do
3:    condition1 = f_i(x') < f_i(x)
4:    condition2 = f_i(x') > f_i(x)
5:    if (condition1) then
6:       Accept← True; return
7:    else if (condition2) then
8:       Accept ← False return
9:    end if
10: end for
```

If `Accept` (x, x', p) is included into `LocalSearch` subroutine of BVNS, then it will not stop the first time a non-improved solution is found. In order to stop `LocalSearch` and thus claim that x' is local minimum, x' should not be improved by any among the p different formulations. Thus, for any particular problem, one needs to design different formulations of the problem considered and decide the order they will be used in the `Accept` subroutine. Answers to those two questions

are problem specific and sometimes not easy. The `Accept` (x, x', p) subroutine can obviously be added to `NeighbourhoodChange` and `Shaking` steps of BVNS from Algorithm 7 as well.

In [46], three evaluation functions, or acceptance criteria, within `Neighborhood Change` step are used in solving **bandwith minimization problem**. This min-max problem consists in finding permutations of rows and columns of a given square matrix such that the maximal distance of nonzero element from the main diagonal in the corresponding row is minimum. Solution x may be presented as a labeling of a graph and the move from x to x' as $x \leftarrow x'$. Three criteria used are (1) the simplest one is based on the objective function value $f_0(x)$ (bandwidth lenth), (2) the total number of critical vertices $f_1(x)$ ($f_1(x') < f_1(x)|$), and (3) $f_3(x, x') = \rho(x, x') - \alpha$, if ($f_0(x') = f_0(x)$ and $f_1(x') = f_1(x)$), but x and x' are relatively far one from another $\rho(x, x') > \alpha$, where α is an additional parameter. The idea for a move to even worse solution if it is very far is used within skewed VNS. However, move to the solution with the same value only if its Hamming distance from the incumbent is greater than α that is performed in [46].

In [41] different mathematical programming formulation of the original problem is used as a secondary objective within `Neighborhood Change` function of VNS. There, two combinatorial optimization problems on the graph are considered: **metric dimension problem** and **minimal doubly resolving set problem**.

More general VFS approach is done in [49], where the **cutwidth graph minimization problem** (CWP) is considered. CWP also belongs to min-max problem family. For a given graph, one needs to make sequence of nodes such that the maximum cutwidth is minimum. The definition of the cutwidth of the graph is clear from Fig. 4, where at (a) graph G with six vertices and nine edges is given.

At Fig. 4b, ordering x of the vertices of the graph in (a) with the corresponding cutwidth CW values of each vertex is presented. It is clear that the CW represents

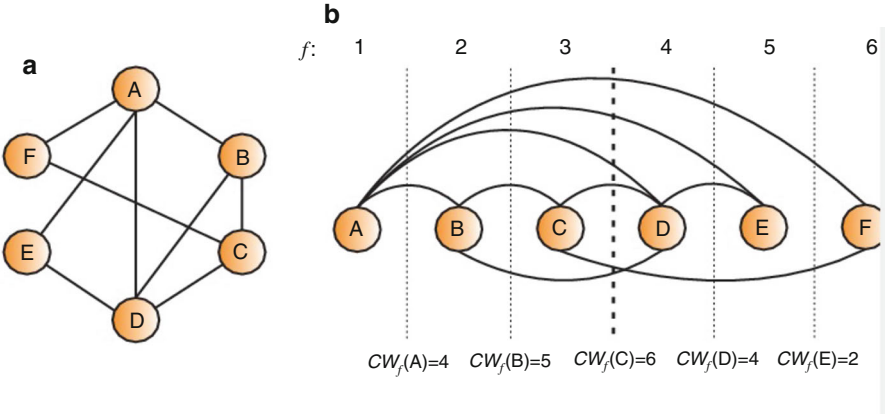


Fig. 4 Cutwidth minimization example as in [49]

Table 1 Impact of the use of alternative formulations in the search process, within 30 s

	BVNS	VFS_1	VFS_2	VFS_3
Avg.	137.31	93.56	91.56	90.75
Dev. (%)	192.44	60.40	49.23	48.22

Table 2 Comparison with the state-of-the-art algorithms over the grid and HB data sets

	81 'grid' test instances				86 HB instances			
	GPR [2]	SA [12]	SS [48]	VFS [49]	GPR [2]	SA [12]	SS [48]	VFS [49]
Avg.	38.44	16.14	13.00	12.23	364.83	346.21	315.22	314.39
Dev. (%)	201.81	25.42	7.76	3.25	95.13	53.30	3.40	1.77
#Opt.	2	37	44	59	2	8	47	61
CPU t (s)	235.16	216.14	210.07	90.34	557.49	435.40	430.57	128.12

the number of cut edges between two consecutive nodes in the solution x. The cutwidth value $f_0(x) = CW(x)$ of the ordering $x = (A, B, C, D, E, F)$ is equal to $f_0(x) = \max\{4, 5, 6, 4, 2\} = 6$ (see Fig. 4). One needs to find order x that minimizes the maximum cutwidth value of each vertex.

Two additional formulations are used in [49] and implemented within VND local search. In Table 1 the results obtained with four different VFS variants, when executing them for 30 s over each instance of the *test* data set, are presented. The column BVNS represents a heuristic based on the BVNS which make use only of the original formulation of the CMP. VFS_1 denotes BVNS heuristic that uses only one secondary criterion. VFS_2 is equivalent to the previous one with the difference that now only f_2 is considered (instead of f_1). Finally, the fourth column of the table, denoted as VFS_3, combines the original formulation of the CMP with the two alternative ones, in the way presented in Algorithm 16. All the algorithms were configured with $k_{\max} = 0.1n$ and they start from the same random solution. It appears that the significant improvements in the solution quality are obtained when at least one secondary formulation is used in case of ties (compare, e.g., 192.44% and 60.40% deviations from the best known solutions reported by BVNS and VFS_1, respectively). Additional improvement is obtained if all three formulations are used (in VFS_3).

Comparison of VFS_3 and state-of-the-art heuristics are given in Table 2. It appears that the best quality results are obtained by VFS in less computing time.

VNS for Mixed-Integer Linear Programming

The mixed-integer linear programming (MILP) problem consists of maximizing or minimizing a linear function, subject to equality or inequality constraints and integrality restrictions on some of the variables. The mixed integer programming problem (P) can be expressed as

$$(MILP) \quad \begin{bmatrix} \min & \sum_{j=1}^{n} c_j x_j & \\ \text{s.t.} & \sum_{j=1}^{n} a_{ij} x_j \geq b_i & \forall i \in M = \{1, 2, \ldots, m\} \\ & x_j \in \{0, 1\} & \forall j \in \mathcal{B} \neq \varnothing \\ & x_j \geq 0, \text{integer} & \forall j \in \mathcal{G} \\ & x_j \geq 0 & \forall j \in \mathcal{C} \end{bmatrix}$$

where the set of indices $N = \{1, 2, \ldots, n\}$ is partitioned into three subsets $\mathcal{B}$, $\mathcal{G}$, and $\mathcal{C}$, corresponding to binary, general integer, and continuous variables, respectively.

Numerous combinatorial optimization problems, including a wide range of practical problems in business, engineering, and science, can be modeled as MILP problems. Its several special cases, such as knapsack, set packing, cutting and packing, network design, protein alignment, traveling salesman, and other routing problems, are known to be NP-hard [19]. There are several commercial solvers such as CPLEX [33] for solving MILPs. Methods included in such software packages are usually of branch-and-bound (B&B) or of branch-and-cut (B&C) types. Basically, those methods enumerate all possible integer values in some order and perform some restrictions for the cases where such enumeration cannot improve the currently best solution.

The connection between local search type of heuristics and exact solvers may be established by introducing the so-called local branching constraint [17]. By adding just one constraint into the (MILP), the kth neighborhood of MILP is defined. This allows the use of all local search-based Metaheuristics, such as tabu search, simulating annealing, VNS, etc. More precisely, given two solutions x and y of the problem (MILP), we define the distance between x and y as

$$\delta(x, y) = \sum_{j \in \mathcal{B}} | x_j - y_j |.$$

Let X be the solution space of the problem (MILP) considered. The neighborhood structures $\{\mathcal{N}_k \mid k = 1, \ldots, k_{\max}\}$ can be defined, knowing the distance $\delta(x, y)$ between any two solutions $x, y \in X$. The set of all solutions in the kth neighborhood of $y \in X$ is denoted as $\mathcal{N}_k(y)$ where

$$\mathcal{N}_k(y) = \{x \in X \mid \delta(x, y) \leq k\}.$$

For the pure 0-1 MILP given above, ($\mathcal{G} = \varnothing$, $\delta(.,.)$ represents the Hamming distance, and $\mathcal{N}_k(y)$ may be expressed by the following so-called local branching constraint:

$$\delta(x, y) = \sum_{j \in S} (1 - x_j) + \sum_{j \in \mathcal{B} \setminus S} x_j \leq k, \tag{2}$$

where $S = \{j \in \mathcal{B} | \, y_j = 1\}$.

In [31] a general VNS procedure for solving 0-1 MILPs is developed (see Algorithm 17).

Algorithm 17: Steps of the VNS branching

```
Function VnsBra(total_time_limit, node_time_limit, k_step, x_opt);
 1  TL := total_time_limit; UB := ∞; first := true
 2  stat := MIPSOLVE(TL, UB, first, x_opt, f_opt)
 3  x_cur:=x_opt; f_cur:=f_opt
 4  while (elapsedtime < total_time_limit) do
 5      cont := true; rhs := 1; first := false
 6      while (cont or elapsedtime < total_time_limit) do
 7          TL = min(node_time_limit, total_time_limit-elapsedtime);
 8          add local br. constr. δ(x, x_cur) ≤ rhs; UB := f_cur
 9          stat := MIPSOLVE(TL, UB, first, x_next, f_next)
10          switch stat do
11              case "opt_sol_found":
12                  reverse last local br. const. into δ(x, x_cur) ≥ rhs + 1
13                  x_cur := x_next; f_cur := f_next; rhs := 1
14              case "feasible_sol_found":
15                  reverse last local br. constr. into δ(x, x_cur) ≥ 1
16                  x_cur := x_next; f_cur := f_next; rhs := 1
17              case "proven_infeasible":
18                  remove last local br. constr.; rhs := rhs+1
19              case "no_feasible_sol_found":
20                  cont := false
21      if f_cur < f_opt then
22          x_opt := x_cur; f_opt := f_cur; k_cur := k_step
        else
23          k_cur := k_cur+k_step
24      remove all added constraints; cont := true
25      while cont and (elapsedtime < total_time_limit) do
26          add constraints k_cur ≤ δ(x, x_opt) < k_cur + k_step
27          TL := total_time_limit-elapsedtime; UB := ∞; first := true
28          stat := MIPSOLVE(TL, UB, first, x_cur, f_cur)
29          remove last two added constraints; cont =false
30          if stat = "proven_infeasible" or "no_feasible" then
31              cont :=true; k_cur := k_cur+k_step
```

An exact MILP solver (CPLEX) is used as a black box for finding the best solution in the neighborhood, based on the given formulation (MILP) plus the added local branching constraints. Shaking is performed using the Hamming distance defined above. The detailed explanation of VNB method given in Algorithm 17 below and the meaning of all variables (*x_cur, x_opt, UB, first, f_opt, cont, rhs,* etc.) and constants (`total_time_limit, node_time_limit, k_step`) used may be found in [31].

VNS for Continuous Global Optimization

The continuous constrained nonlinear global optimization problem (GOP) in general form is given as follows:

$$\text{(GOP)} \quad \begin{bmatrix} \min & f(x) & \\ \text{s.t.} & g_i(x) \leq 0 & \forall i \in \{1, 2, \ldots, m\} \\ & h_i(x) = 0 & \forall i \in \{1, 2, \ldots, r\} \\ & a_j \leq x_j \leq b_j & \forall j \in \{1, 2, \ldots, n\} \end{bmatrix}$$

where $x \in R^n$, $f : R^n \rightarrow R$, $g_i : R^n \rightarrow R$, $i = 1, 2, \ldots, m$, and $h_i : R^n \rightarrow R$, $i = 1, 2, \ldots, r$ are possibly nonlinear continuous functions, and $a, b \in R^n$ are the variable bounds.

GOP naturally arises in many applications, e.g., in advanced engineering design, data analysis, financial planning, risk management, scientific modeling, etc. Most cases of practical interest are characterized by multiple local optima, and, therefore, in order to find the globally optimal solution, a global scope search effort is needed.

If the feasible set X is convex and objective function f is convex, then GOP is relatively easy to solve, i.e., the Karush-Kuhn-Tucker conditions may be applied. However, if X is not a convex set or f is not a convex function, we could have many local minima, and thus, the problem may not be solved by using classical techniques.

For solving GOP, VNS has been used in two different ways: (a) with neighborhoods induced by using an ℓ_p norm and (b) without using an ℓ_p norm.

(a) *VNS with ℓ_p norm neighborhoods* [3, 4, 15, 36, 39, 42]. A natural approach in applying VNS for solving GOP is to induce neighborhood structures $\mathcal{N}_k(x)$ from the ℓ_p metric:

$$\rho(x, y) = \left(\sum_{i=1}^{n} |x_i - y_i|^p \right)^{1/p} \quad (1 \leq p < \infty) \tag{3}$$

or

$$\rho(x, y) = \max_{1 \leq i \leq n} | x_i - y_i | \quad (p \rightarrow \infty). \tag{4}$$

The neighborhood $\mathcal{N}_k(x)$ denotes the set of solutions in the k-th neighborhood of x, and using the metric ρ, it is defined as

$$\mathcal{N}_k(x) = \{y \in X \mid \rho(x, y) \leq \rho_k\}, \tag{5}$$

or

$$\mathcal{N}_k(x) = \{y \in X \mid \rho_{k-1} \leq \rho(x, y) \leq \rho_k\}, \tag{6}$$

where ρ_k, known as the radius of $\mathcal{N}_k(x)$, is monotonically increasing with k.

For solving box constraint GOP, both [15, 36] use neighborhoods as defined in (6). The basic differences between the two are as follows: (1) in the procedure suggested in [36] the ℓ^∞ norm is used, while in [15] the choice of metric is either left to the analyst or changed automatically according to some predefined order; (2) as a local search procedure within VNS, the commercial solver SNOPT [20] is used in [36], while in [15] the analyst may choose one out of six different convex minimizers. A VNS-based heuristic for solving the generally constrained GOP is suggested in [39]. There, the problem is first transformed into a sequence of box-constrained problems within well-known exterior point method:

$$\min_{a \leq x \leq b} F_{\mu,q}(x) = f(x) + \frac{1}{\mu} \sum_{i=1}^{m} (\max\{0, q_i(x)\})^q + \sum_{i=1}^{r} |h_i(x)|^q, \tag{7}$$

where μ and $q \geq 1$ are a positive penalty parameter and penalty exponent, respectively. Algorithm 18 outlines the steps for solving the box constraint subproblem as proposed in [39]:

Algorithm 18: Steps of continuous VNS using ℓ_p norm

Function `Glob-VNS` $(x^*, k_{\max}, t_{\max})$
1 Select the set of neighborhood structures $\mathcal{N}_k$ $k = 1, \ldots, k_{\max}$
2 Select the array of random distributions types and an initial point $x* \in X$
3 $x \leftarrow x^*$, $f^* \leftarrow f(x)$, $t \leftarrow 0$
4 **while** $t < t_{\max}$ **do**
5 $\quad k \leftarrow 1$
6 $\quad$ **repeat**
7 $\quad\quad$ **for** *all distribution types* **do**
8 $\quad\quad\quad y \leftarrow$ `Shake`(x^*, k) // Get $y \in \mathcal{N}_k(x^*)$ at random
9 $\quad\quad\quad y' \leftarrow$ `BestImprovment`(y) // Apply LS to obtain a local minimum y'
10 $\quad\quad\quad$ **if** $f(y') < f^*$ **then**
11 $\quad\quad\quad\quad x^* \leftarrow y'$, $f^* \leftarrow f(y')$, `go to line 5`
12 $\quad\quad k \leftarrow k + 1$
$\quad$ **until** $k = k_{\max}$
13 $\quad t \leftarrow$ `CpuTime()`

The `Glob-VNS` procedure from Algorithm 18 contains the following parameters in addition to $k_{\max}$ and $t_{\max}$:

1. *Values of radii* ρ_k, $k = 1, \ldots, k_{\max}$. Those values may be defined by the user or calculated automatically in the minimizing process;

2. *Geometry* of neighborhood structures $\mathcal{N}_k$, defined by the choice of metric. Usual choices are the ℓ_1, ℓ_2, and $\ell\infty$ norms;
3. *Distribution* used for obtaining the random point y from $\mathcal{N}_k$ in the *shaking* step. Uniform distribution in $\mathcal{N}_k$ is the obvious choice, but other distributions may lead to much better performance on some problems.

Different choices of geometric neighborhood shapes and random point distributions lead to different VNS-based heuristics.

(b) *VNS without using ℓ_p norm*. Two different neighborhoods, $N_1(x)$ and $N_2(x)$, are used in VNS-based heuristic suggested in [55]. In $N_1(x)$, r (a parameter) random directions from the current point x are generated and one-dimensional searches along each performed. The best point (out of r) is selected as a new starting solution for the next iteration, if it is better than the current one. If not, as in VND, the search is continued within the next neighborhood $N_2(x)$. The new point in $N_2(x)$ is obtained as follows. The current solution is moved parallel to each x_j ($j = 1, \ldots, n$) by value Δ_j, taken at random from interval $(-\alpha, \alpha)$, i.e., $x_j^{(\text{new})} = x_j + \Delta_j$ or $x_j^{(\text{new})} = x_j - \Delta_j$. Points obtained by the plus or minus sign for each variable define neighborhood $N_2(x)$. If change of $x_j^{(\text{new})}$ by 1% to the right gives a better solution than in $x^{(\text{new})}$, the + sign is chosen; otherwise the–sign is chosen.

Neighborhoods N_1 and N_2 are used for designing two algorithms. The first, called VND, iterates these neighborhoods until there is no improvement in the solution value. In the second variant, a local search is performed in N_2 and $k_{\max}$ set to 2 for the shaking step. In other words, a point from neighborhood $k = 2$ is obtained by generating a random direction followed by line search along it (as prescribed for N_1) and then by changing each of the variables (as prescribed for N_2).

Variable Neighborhood Programming

Building an intelligent machine is an old dream that, thanks to computers, began to take shape. Automatic programming is an efficient technique that has contributed an important development in the field of artificial intelligence. Genetic programming (GP) [35], inspired by genetic algorithm (GA), is among the few evolutionary algorithms used to evolve population of programs. The main difference between GP and GA is the presentation of a solution. An individual in GA can be a string, while GPs individuals are programs. The usual way to present program within GP is by using a tree. For example, assume that the current solution of the problem is the following function:

$$f(x_1, \ldots, x_5) = \frac{x_1}{x_2 + x_3} + x_4 - x_5.$$

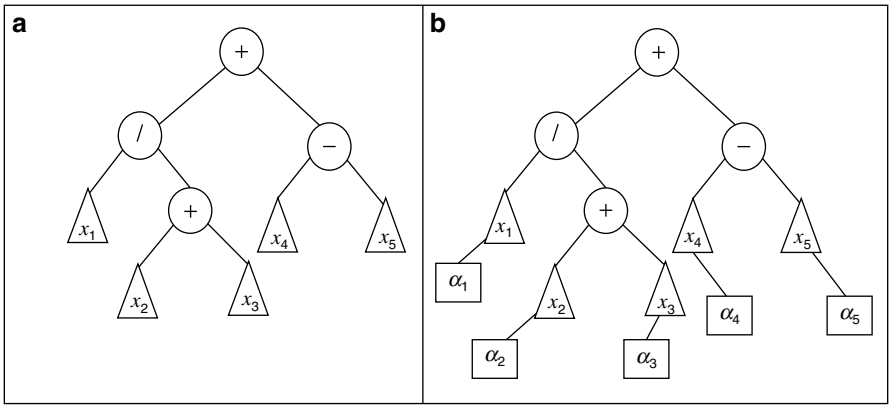


Fig. 5 Current solution representation in automatic programming problem: $\frac{x_1}{x_2+x_3} + x_4 - x_5$. (**a**) GP solution representation. (**b**) VNP solution representation

Then the code (tree) that calculates it using GP may be presented as in Fig. 5a.

Souhir et al. [16] recently adapted VNS rules in solving automatic programming problem. They first suggested an extended solution presentation by adding coefficients to variables. Each terminal node was attached to its own parameter value. These parameters gave a weight for each terminal node, with values from the interval [0, 1]. This form of presentation allowed VNP to examine parameter values and the remaining tree structures in the same iteration. Let $G = \{\alpha_1, \alpha_2, \ldots, \alpha_n\}$ denote a parameter set. In Fig. 5b an example of VNP's solution representation is illustrated.

Neighborhood structures. At such solution presentation as a tree T, nine different neighborhood structures are proposed in [16]. To save the space, we will just mention some of them:

- $N_1(T)$—**Changing a node value operator**. This neighborhood structure conserves the skeleton of the tree and changes only the values of a functional or a terminal node. Each node can obtain many values from its corresponding set. Let x_i be the current solution; its neighbor x_{i+1} differs from x_i by just a single node. A move within this neighborhood structure is shown in Fig. 6.
- $N_2(T)$—**Swap operator**. By this operator a first node from the current tree is taken at random, and a new sub-tree is generated as presented in Fig. 7a, b. Then the selected node is attached in the place of the sub-tree. In this move, the constraint related to the maximum tree size should be respected. More details can be seen in Fig. 7.
- $N_3(T)$—**Changing parameter values**. In the previous neighborhood structures, the tree form and its node values were considered. In $N_3(T)$ neighborhood, attention is paid on parameters. So, the position and value of nodes are kept in order to search the neighbors in the parametric space. Figure 8 illustrates details. The change from one value to another is at random.

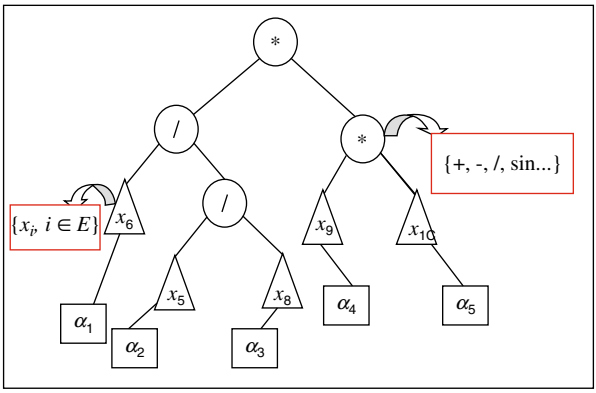


Fig. 6 Neighborhood structure N_1: changing a node value

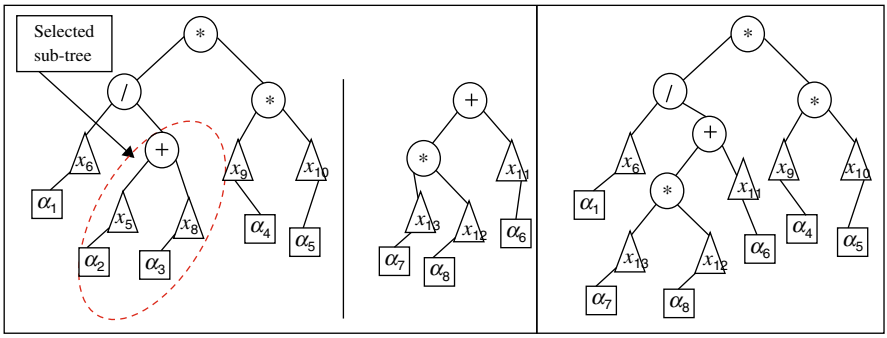


Fig. 7 Neighborhood structure N_2: swap operator

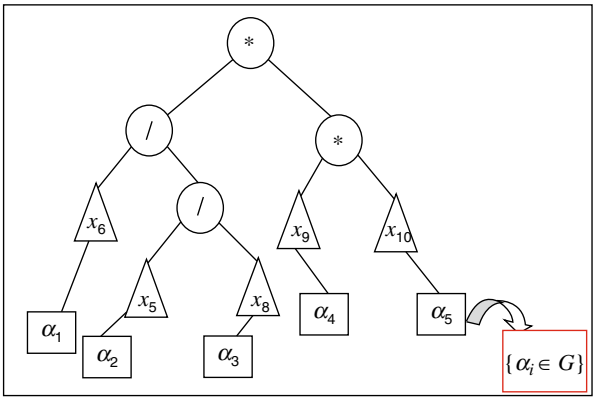


Fig. 8 Neighborhood structure N_3: change parameters

These neighborhood structures may be used in both local search step (N_ℓ, $\ell \in [1, \ell_{\max}]$) and in the shaking step ($\mathcal{N}_k$, $k \in [1, k_{\max}]$) of the VNS.

VNP shaking. The shaking step allows the diversification in the search space. Our proposed VNP algorithm does not use exactly the same neighborhood structures N_ℓ as for the local search. That is the reason why neighborhoods used in shaking are denoted differently: $\mathcal{N}_k(T), k = 1, k_{\max}$. $\mathcal{N}_k(T)$ may be constructed by repeating k times one or more moves from the set $\{N_\ell(T), |\ell = 1, \ldots, \ell_{\max}\}$, explained earlier. Nevertheless, in the shaking phase, we use a neighborhood structure that mainly affects the skeleton of a presented solution with its different forms for the perturbation. For more clear explanation, we take the swap operator $N_3(T)$ as an example. Let m denote the maximum size of a solution. We can get a solution from the kth neighborhood of T using the swap operator: k may represent the size of the new generated sub-tree. If n denotes the size of the original tree after deleting the old sub-tree, then $n + k_{\max} \leq m$. The objective of this phase is to provide a good starting point for the local search.

VNP objective function. The evaluation consists of defining a fitness or objective function assessing the proposed problem. This function is defined according to the problem considered. After running each solution (program) on training data set, fitness may be measured by counting how many training cases the current solution result is correct or near to the exact solution.

An example: time series forecasting (TSF) problem. Two widely used benchmark data sets of TSF problem are explored in [16] to examine VNP capabilities: Mackey-Glass series and Box-Jenkins set. The parameters for the VNP implementation are chosen after some preliminary testing are given in Table 3.

The root-mean-square error (RMSE) is used as fitness function, as it is usual in the literature:

$$f(T) = \sqrt{\frac{1}{n}\sum_{j=1}^{n}\left(y_t^j - y_{\text{out}}^j\right)^2}$$

where n represents the total number of samples and y_{out}^j and y_t^j are outputs of the sample number j obtained by the VNP model and the desired one, respectively. Here we will just illustrate comparison on Box-Jenkins instance.

Table 3 VNP parameters adjustment for the forecasting problem

Parameters	Values
The functional set	$F = \{+, *, pow\}$;
The terminal sets	$\{(x_i, c), i \in [1, \ldots, m], m =$ number of inputs, $c \in R\}$;
Neighborhood structures	$\{N_1, N_2, N_3\}$;
Minimum tree length	20 nodes;
Maximum tree length	200 nodes;
Maximum number of iterations	50,000

Table 4 Comparison of testing error on Box-Jenkins dataset

Method	Prediction error RMSE
ODE [54]	0.5132
HHMDDE [14]	0.3745
FBBFNT [6]	0.0047
VNP [16]	**0.0038**

The gas furnace data of Box and Jenkins were collected from a combustion process of a methane-air mixture [15]. This time series has found a widespread application as a benchmark example in many practical sciences for testing prediction algorithms. The data set contains 296 pairs of input-output values. The input $u(t)$ corresponds to the gas flow, and the output $y(t)$ presents the CO_2 concentration in outlet gas. The inputs are $u(t-4)$ and $y(t-1)$, and the output is $y(t)$. In this work, 200 samples are used in the training phase, and the remaining samples are used for the testing phase. The performance of the evolved model is evaluated by comparing it with the abovementioned approaches. The RMSE achieved by VNP output model is 0.0038, which appeared to be better than the RMSE obtained by other approaches. Table 4 shows that VNP approach proves effectiveness and generalization ability.

Discovery Science

In all the above applications, VNS is used as an optimization tool. It can also lead to results in "discovery science," i.e., help in the development of theories. This has been done for graph theory in a long series of papers with the common title "Variable neighborhood search for extremal graphs" and reporting on the development and applications of the system AutoGraphiX (AGX) [10]. This system addresses the following problems:

- Find a graph satisfying given constraints;
- Find optimal or near optimal graphs for an invariant subject to constraints;
- Refute a conjecture;
- Suggest a conjecture (or repair or sharpen one);
- Provide a proof (in simple cases) or suggest an idea of proof.

Then a basic idea is to consider all of these problems as parametric combinatorial optimization problems on the infinite set of all graphs (or in practice some smaller subset) with a generic heuristic. This is done by applying VNS to find extremal graphs, with a given number n of vertices (and possibly also a given number of edges). Then a VND with many neighborhoods is used. Those neighborhoods are defined by modifications of the graphs such as the removal or addition of an edge, rotation of an edge, and so forth. Recently all moves involving four vertices or less were considered jointly in learning optimization framework. Once a set of extremal or near-extremal graphs, parameterized by their order, is found, their properties are

explored with various data mining techniques, leading to conjectures, refutations, and simple proofs or ideas of proof.

All papers in this area are divided into the following groups in [28], where the extensive list of references for each class of problems can be found:

(i) *Principles of the approach* and its implementation;
(ii) *Applications to spectral graph theory*, e.g., finding bounds on the index or spectral radius of the adjacency matrix for various families of graphs and finding graphs maximizing or minimizing the index subject to some conditions;
(iii) *Studies of classical graph parameters*, e.g., independence, chromatic number, clique number, and average distance;
(iv) *Studies of little known or new parameters of graphs*, e.g., irregularity, proximity, and remoteness;
(v) *New families of graphs discovered by AGX*, e.g., bags, which are obtained from complete graphs by replacing an edge by a path, and bugs, which are obtained by cutting the paths of a bag;
(vi) *Applications to mathematical chemistry*, e.g., study of chemical graph energy and of the Randić index;
(vii) *Results of a systematic study of pairwise comparison of 20 graph invariants*, involving the four usual operators: plus, minus, ratio, and product, which led to almost 1500 new conjectures, more than half of which were automatically proved by AGX and over 300 by various mathematicians;
(viii) *Refutation or strengthening of conjectures from the literature*;
(ix) *Surveys and discussions about various discovery systems in graph theory*, assessment of the state-of-the-art and the forms of interesting conjectures together with proposals for the design of more powerful systems.

Conclusions

The general schemes of variable neighborhood search have been presented, discussed, and illustrated by examples. In order to evaluate the VNS research program, one needs a list of the desirable properties of Metaheuristics. The following eight of these are presented in Hansen and Mladenović [27]:

(i) *Simplicity:* the metaheuristic should be based on a simple and clear principle, which should be widely applicable;
(ii) *Precision:* the steps of the metaheuristic should be formulated in precise mathematical terms, independent of possible physical or biological analogies which may have been the initial source of inspiration;
(iii) *Coherence:* all steps of the heuristics for particular problems should follow naturally from the principle of the metaheuristic;

(iv) *Efficiency:* heuristics for particular problems should provide optimal or near-optimal solutions for all or at least most realistic instances. Preferably, they should find optimal solutions for most problems of benchmarks for which such solutions are known, when available;
(v) *Effectiveness:* heuristics for particular problems should take a moderate computing time to provide optimal or near-optimal solutions;
(vi) *Robustness:* the performance of heuristics should be consistent over a variety of instances, i.e., not merely fine-tuned to some training set and less good elsewhere;
(vii) *User-friendliness:* heuristics should be clearly expressed, easy to understand, and, most important, easy to use. This implies they should have as few parameters as possible, ideally none;
(viii) *Innovation:* preferably, the principle of the metaheuristic and/or the efficiency and effectiveness of the heuristics derived from it should lead to new types of application.
(ix) *Generality:* the metaheuristic should lead to good results for a wide variety of problems;
(x) *Interactivity:* the metaheuristic should allow the user to incorporate his knowledge to improve the resolution process;
(xi) *Multiplicity:* the metaheuristic should be able to present several near-optimal solutions from which the user can choose one.

As shown above, VNS possesses, to a great extent, all of the above properties. This has led to heuristics which are among the very best ones for many problems. Interest in VNS is clearly growing at speed. This is evidenced by the increasing number of papers published each year on this topic (15 years ago, only a few; 10 years ago, about a dozen; about 50 in 2007). According to Google Scholar, the first two papers on VNS were cited almost 4000 times!

Moreover, the 18th and 28th EURO Mini Conferences were entirely devoted to VNS. It led to special issues of *IMA Journal of Management Mathematics* in 2007 [37], *European Journal of Operational Research* [29], *Journal of heuristics* [47] in 2008, *Computers and Operations Research* in 2014 [45], and *Journal of Global Optimization* in 2016 [11]. The 3rd International VNS meeting took place in Tunisia, and three new special issues entirely devoted to VNS are in preparation (*Computers and Operations Research*, *Optimization Letters*, and *Yugoslav Journal of Operations Research*). In retrospect, it appears that the good shape of the VNS research program is due to the following decisions, strongly influenced by Karl Popper's philosophy of science [51]: (i) in devising heuristics favor insight over efficiency (which comes later) and (ii) learn from the heuristics mistakes.

Competing Interest Declaration The author(s) has no competing interests to declare that are relevant to the content of this manuscript.

References

1. Aloise DJ, Aloise D, Rocha CTM, Ribeiro CC, Ribeiro JC, Moura LSS (2006) Scheduling workover rigs for onshore oil production. Discret Appl Math 154(5):695–702
2. Andrade DV, Resende MGC (2007) GRASP with path-relinking for network migration scheduling. In: Proceedings of international network optimization conference (INOC), Spa
3. Audet C, Báchard V, Le Digabel S (2008) Nonsmooth optimization through mesh adaptive direct search and variable neighborhood search. J Glob Optim 41(2):299–318
4. Audet C, Brimberg J, Hansen P, Mladenović N (2004) Pooling problem: alternate formulation and solution methods. Manag Sci 50:761–776
5. Belacel N, Hansen P, Mladenović N (2002) Fuzzy J-means: a new heuristic for fuzzy clustering. Pattern Recogn 35(10):2193–2200
6. Bouaziz S, Dhahri H, Alimi AM, Abraham A (2013) A hybrid learning algorithm for evolving flexible beta basis function neural tree model. Neurocomputing 117:107–117. https://doi.org/10.1016/j.neucom.2013.01.024
7. Brimberg J, Hansen P, Mladenović N, Taillard É (2000) Improvements and comparison of heuristics for solving the multisource Weber problem. Oper Res 48(3):444–460
8. Brimberg J, Mladenović N (1996) A variable neighborhood algorithm for solving the continuous location-allocation problem. Stud Locat Anal 10:1–12
9. Canuto S, Resende M, Ribeiro C (2001) Local search with perturbations for the prize-collecting Steiner tree problem in graphs. Networks 31(3):201–206
10. Caporossi G, Hansen P (2000) Variable neighborhood search for extremal graphs 1. The AutoGraphiX system. Discret Math 212:29–44
11. Carrizosa E, Hansen P, Moreno-Perez JA (2015) Variable neighborhood search. J Glob Optim (Spec Issue) 63(3):427–629
12. Cohoon J, Sahni S (1987) Heuristics for backplane ordering. J VLSI Comput Syst 2:37–61
13. Davidon WC (1959) Variable metric algorithm for minimization. Argonne National Laboratory report ANL-5990
14. Dhahri H, Alimi AM, Abraham A (2012) Hierarchical multi-dimensional differential evolution for the design of beta basis function neural network. Neurocomputing 97:131–140
15. Dražić M, Kovacevic-Vujcić V, Cangalović M, Mladenović N (2006) GLOB – a new VNS-based software for global optimization. In: Liberti L, Maculan N (eds) Global optimization: from theory to implementation. Springer, New York, pp 135–144
16. Elleucha S, Jarbouia B, Mladenovic N (2015) Variable neighborhood programming a new automatic programming method in artificial intelligence, Gerad technical report G-2016-21, HEC Montreal, Canada
17. Fischetti M, Lodi A (2003) Local branching. Math Program 98(1–3):23–47
18. Fletcher R, Powell MJD (1963) Rapidly convergent descent method for minimization. Comput J 6:163–168
19. Garey MR, Johnson DS (1978) Computers and intractability: a guide to the theory of NP-completeness. Freeman, New York
20. Gill P, Murray W, Saunders MA (2002) SNOPT: an SQP algorithms for largescale constrained optimization. SIAM J Optim 12(4):979–1006
21. Griffith RE, Stewart RA (1961) A nonlinear programming technique for the optimization of continuous processing systems. Manag Sci 7:379–392
22. Hansen P, Brimberg J, Urošević D, Mladenović N (2007) Primal-dual variable neighborhood search for the simple plant location problem. INFORMS J Comput 19(4):552–564
23. Hansen P, Jaumard B, Mladenović N, Parreira A (2000) Variable neighborhood search for weighted maximum satisfiability problem. Les Cahiers du GERAD G–2000–62, HEC Montréal
24. Hansen P, Mladenović N (2001) Variable neighborhood search: principles and applications. Eur J Oper Res 130:449–467

25. Hansen P, Mladenović N (2001) J-means: a new local search heuristic for minimum sum-of-squares clustering. Pattern Recogn 34:405–413
26. Hansen P, Mladenović N (2001) Developments of variable neighborhood search. In: Ribeiro C, Hansen P (eds) Essays and surveys in metaheuristics. Kluwer, Dordrecht/London, pp 415–440
27. Hansen P, Mladenović N (2003) Variable neighborhood search. In: Glover F, Kochenberger G (eds) Handbook of metaheuristics. Kluwer, Boston, pp 145–184
28. Hansen P, Mladenović N, Brimberg J, Moreno-Perrez JA (2010) Variable neighborhood search. In: Gendreau M, Potvin J-Y (eds) Handbook of metaheuristics, 2nd edn. Kluwer, New York, pp 61–86
29. Hansen P, Mladenović N, Moreno Pérez JA (2008) Variable neighborhood search. Eur J Oper Res 191(3):593–595
30. Hansen P, Mladenović N, Pérez-Brito D (2001) Variable neighborhood decomposition search. J Heuristics 7(4):335–350
31. Hansen P, Mladenović N, Urošević D (2006) Variable neighborhood search and local branching. Comput Oper Res 33(10):3034–3045
32. Hertz A, Plumettaz M, Zufferey N (2008) Variable space search for graph coloring. Discret Appl Math 156(13):2551–2560
33. ILOG (2006) CPLEX 10.1. User's manual
34. Jornsten K, Lokketangen A (1997) Tabu search for weighted k-cardinality trees. Asia-Pac J Oper Res 14(2):9–26
35. Koza JR (1992) Genetic programming: on the programming of computers by means of natural selection. MIT Press, Cambridge
36. Liberti L, Dražić M (2005) Variable neighbourhood search for the global optimization of constrained NLPs. In: Proceedings of GO workshop, Almeria
37. Melián B, Mladenović N (2007) Editorial. IMA J Manag Math 18(2):99–100
38. Mladenovic N (1995) Variable neighborhood algorithm – a new metaheuristic for combinatorial optimization. In: Optimization days conference, Montreal, p 112
39. Mladenović N, Dražić M, Kovačevic-Vujčić V, Čangalović M (2008) General variable neighborhood search for the continuous optimization. Eur J Oper Res 191(3):753–770
40. Mladenović N, Hansen P (1997) Variable neighborhood search. Comput Oper Res 24:1097–1100
41. Mladenovic N, Kratica J, Kovacevic-Vujcic V, Cangalovic M (2012) Variable neighborhood search for metric dimension and minimal doubly resolving set problems. Eur J Oper Res 220(2):328–337
42. Mladenović N, Petrović J, Kovačević-Vujčić V, Čangalović M (2003) Solving spread spectrum radar polyphase code design problem by tabu search and variable neighborhood search. Eur J Oper Res 151:389–399
43. Mladenović N, Plastria F, Urošević D (2005) Reformulation descent applied to circle packing problems. Comput Oper Res 32:2419–2434
44. Mladenović N, Plastria F, Uroševic D (2007) Formulation space search for circle packing problems. Engineering stochastic local search algorithms. Designing, implementing and analyzing effective heuristics. Lect Notes Comput Sci 4638:212–216. https://link.springer.com/book/10.1007/978-3-540-74446-7
45. Mladenovic N, Salhi S, Hnafi S, Brimberg J (eds) (2014) Recent advances in variable neighborhood search. Comput Oper Res 52(B):147–148
46. Mladenovic N, Urosevic D, Pérez-Brito D, García-González CG (2010) Variable neighbourhood search for bandwidth reduction. Eur J Oper Res 200(1):14–27
47. Moreno-Vega JM, Melián B (2008) Introduction to the special issue on variable neighborhood search. J Heuristics 14(5):403–404
48. Pantrigo JJ, Martí R, Duarte A, Pardo EG (2012) Scatter search for the cutwidth minimization problem. Ann Oper Res 199:285–304
49. Pardo EG, Mladenovic N, Pantrigo JJ, Duarte A (2013) Variable formulation search for the cutwidth minimization problem. Appl Soft Comput 13(5):2242–2252

50. Plastria F, Mladenović N, Urošević D (2005) Variable neighborhood formulation space search for circle packing. In: 18th mini Euro conference VNS, Tenerife
51. Popper K (1959) The logic of scientific discovery. Hutchinson, London
52. Ribeiro CC, de Souza MC (2002) Variable neighborhood search for the degree-constrained minimum spanning tree problem. Discret Appl Math 118(1–2):43–54
53. Ribeiro CC, Uchoa E, Werneck R (2002) A hybrid GRASP with perturbations for the Steiner problem in graphs. INFORMS J Comput 14(3):228–246
54. Subudhi B, Jena D (2011) A differential evolution based neural network approach to nonlinear system identification. Appl Soft Comput 11:861–871. https://doi.org/10.1016/j.asoc.2010.01.006
55. Toksari AD, Güner E (2007) Solving the unconstrained optimization problem by a variable neighborhood search. J Math Anal Appl 328(2):1178–1187
56. Whitaker R (1983) A fast algorithm for the greedy interchange of large-scale clustering and median location problems. INFOR 21:95–108

Part IV
Analysis and Implementation

A History of Metaheuristics

33

Kenneth Sörensen, Marc Sevaux, and Fred Glover

Contents

Introduction . . . 992
Period 0: The Pre-theoretical Period . . . 995
Period 1: The Early Period . . . 996
Period 2: The Method-Centric Period . . . 998
Period 3: The Framework-Centric Period . . . 1002
The Metaphor-Centric Period . . . 1004
Period 4: The Scientific Period? . . . 1005
Conclusions . . . 1007
Cross-References . . . 1008
References . . . 1008

Abstract

This chapter describes the history of metaheuristics in five distinct periods, starting long before the first use of the term and extending far into the future.

The field of metaheuristics has undergone several paradigm shifts—changes that have transformed the way researchers approach the development of heuristic

K. Sörensen (✉)
University of Antwerp, Antwerp, Belgium
e-mail: kenneth.sorensen@uantwerpen.be

M. Sevaux
Lab-STICC, CNRS, Université Bretagne Sud, Lorient, France
e-mail: marc.sevaux@univ-ubs.fr

F. Glover
Entanglement, Inc., Coral Gables, FL, USA
e-mail: fred@entanglement.com

© Springer Nature Switzerland AG 2025

R. Martí et al. (eds.), *Handbook of Heuristics*,
https://doi.org/10.1007/978-3-032-00385-0_4

methods. Most notably, there has been a shift from the *method-centric* period, in which metaheuristics were thought of as algorithms, to the *framework-centric* period, in which researchers view metaheuristics as more general high-level frameworks, i.e., consistent collections of concepts and ideas that offer guidelines on how to solve an optimization problem heuristically.

Tremendous progress has been made in the development of heuristics over the years. Optimization problems that seemed intractable only a few decades ago can now be efficiently solved. Nevertheless, there is still much room for evolution in the research field—an evolution that will allow it to transition into the *scientific period*. In this period, we will see more structured knowledge generation that will benefit both researchers and practitioners.

Unfortunately, a significant fraction of the research community has deluded itself into thinking that scientific progress can be achieved by resorting to ever more outlandish metaphors as the basis for the so-called novel methods. Even though considerable damage to the research field will have been inflicted by the time these ideas are eradicated, there is no doubt that science will ultimately prevail.

Keywords

History · Metaheuristics

Introduction

Optimization is the process of selecting or approximating the best solution from a set of alternatives by determining a configuration of inputs that maximizes or minimizes an output (objective function) while adhering to constraints. Over the course of history, several efficient algorithms have been developed to solve—i.e., to find the optimal solution for—specific optimization problems. A prime example is Dijkstra's [12] algorithm for the shortest path problem, which serves as the foundation for all navigation software and is heavily used in Internet-routing protocols. Problem-specific algorithms, however, have the disadvantage of only solving a single well-defined problem. Mathematical programming models (linear programming, (mixed) integer programming, etc.) have been crucial in generalizing these approaches, enabling the formulation of complex real-world problems into decision variables, objective functions, and constraints. When these models follow a very specific structure (e.g., all constraints and the objective function are linear functions of real-valued decision variables), standardized methods like Dantzig's simplex method [11] can be used to find the optimal solution.

Despite the success of the mathematical programming paradigm, however, it encounters limitations when addressing large-scale or complex problems, such as those involving nonlinear constraints or discrete variables in combinatorial optimization. As the problem size increases, solving for the optimal solution

generally becomes computationally impractical. This empirical fact is supported by *complexity theory*, which provides insight into why some combinatorial problems are likely to never be solvable in a reasonable time. The real world, however, requires solutions regardless of empirical or theoretical objections. As a result, researchers have turned to *heuristic* methods, which relax the goal of finding the optimal solution and instead aim for a feasible one in a practical amount of time. Heuristic optimization allows researchers to solve large-scale problems efficiently, though the solutions are not guaranteed to be optimal, and many heuristic algorithms (like the [6] algorithm for the vehicle-routing problem) have been developed.

Just like problem-specific exact methods, however, problem-specific heuristics are limited to solving a single problem, and, with researchers and practitioners wishing to solve different, more complicated problems, a need for more general, problem-agnostic algorithms arose. This need has been met by the development of a class of methods called *metaheuristics*, which provide *frameworks* or high-level strategies to guide the development of heuristic optimization algorithms. As linear and mixed-integer programming generalized *exact* algorithms for solving specific problems, metaheuristics similarly generalized problem-specific *heuristic* algorithms.

Even though people have used heuristics throughout history, and the human brain is equipped with a formidable heuristic engine to solve an enormous array of challenging optimization problems, the *scientific study* of heuristics (and, by extension, metaheuristics) is a relatively young endeavor. It is not an exaggeration to claim that the field of (meta)heuristics, especially compared to other fields of study like physics, chemistry, and mathematics, has yet to reach a mature state. Nevertheless, enormous progress has been made since the first metaheuristics concepts were established.

No history is ever neutral, and a history of metaheuristics—or any other topic—can be written in many different ways. A straightforward history of metaheuristics would consist of an annotated and chronological list of metaheuristic methods. Useful as such a list may be, it suffers from a lack of insight into the development of the field as a whole.

Taking a bird's eye view of the field of metaheuristics, one has to conclude that there has been a large amount of progressive insight over the years. Moreover, this progressive insight has not reached its endpoint: The way researchers and practitioners look at metaheuristics is still continually shifting. Even the answer to the question what a metaheuristic is has changed quite a lot since the word was first coined in the second half of the 1980s. The exploration of this shifting insight is the topic of this chapter. We believe it allows us to truly understand the historical advancements and learn many interesting lessons that could be useful for the future development of research in metaheuristics. We did not limit our discussion in this chapter to metaheuristics that have been formally written down and published. When studying the history of metaheuristics with an open mind, one has to conclude that people have been using heuristics and metaheuristics long before the term even existed.

We have therefore adopted a different approach to write "our" history of metaheuristics. Our approach starts well before the term "metaheuristic" was coined and is based on the premise that people have looked at metaheuristics through different sets of glasses over the years. The way in which people—not only researchers—have interpreted the different metaheuristic concepts has shaped the way in which the field has been developing. To understand the design choices that people have been making when developing metaheuristic optimization algorithms, it is paramount that these choices are understood in relationship to the trends and viewpoints of the time during which the development took place.

Our history divides time into five distinct periods. The crispness of the boundaries between each pair of consecutive periods, however, is a gross simplification of reality. The real (if one can use that word) time periods during which the paradigm shifts took place are usually spaced out over several years, but it is difficult, if not impossible, to trace the exact moments in time at which the paradigm shifts began and ended. More importantly, not every researcher necessarily makes the transition at the same time:

- The *pre-theoretical* period (until c. 1940), during which heuristics and even metaheuristics are used but not formally studied
- The *early* period (c. 1940–c. 1980), during which the first formal studies on heuristics appear
- The *method-centric* period (c. 1980–c. 2000), during which the field of metaheuristics truly takes off and many different methods are proposed
- The *framework-centric* period (c. 2000–now), during which the insight grows that metaheuristics are more usefully described as frameworks, and not as methods
- The *scientific* period (the future), during which the design of metaheuristics becomes a science instead of an art

Until recently, a clear definition of the word metaheuristic has been lacking, and it could be argued that it is still disputed. In this chapter, we adopt the definition of [51].

> A metaheuristic is a high-level problem-independent algorithmic framework that provides a set of guidelines or strategies to develop heuristic optimization algorithms. The term is also used to refer to a problem-specific implementation of a heuristic optimization algorithm according to the guidelines expressed in such a framework.

The term "metaheuristic" has been used (and is used) for two entirely different things. One is a high-level framework, a set of concepts and strategies that blend together and offer a perspective on the development of optimization algorithms. In this sense, variable neighborhood search [39] is nothing more (or less) than the idea to use different local search operators to work on a single solution, together with a perturbation operator once all neighborhoods have reached a local optimum. There is a compelling motivation, as well a large amount of empirical evidence, as to why multi-neighborhood search is indeed a very good idea. This motivation essentially

comes down to the fact that a local optimum for one local search operator (or one neighborhood structure) is usually not a local optimum for another local search operator. The idea to switch to a different local search operator once a local optimum has been found is therefore both sensible and in practice extremely powerful.

The second meaning of the term "metaheuristic" denotes a specific implementation of an algorithm based on such a framework (or on a combination of concepts from different frameworks) designed to find a solution to a specific optimization problem. The variable neighborhood search (-based) algorithm for the location-routing problem by [34] is an example of a metaheuristic in this sense.

In this chapter, we will use the term "metaheuristic framework" to refer to the first sense and "metaheuristic algorithm" to refer to the second sense of the word "metaheuristic."

As mentioned, a history of any topic is not neutral. We therefore do not attempt to hide the fact that certain ways in which the field has progressed seem to us less useful and, at times, even harmful to the overall development of the field. We also discuss this phenomenon, which has plagued—and continues to plague—the field of metaheuristics.

Period 0: The Pre-theoretical Period

Optimization problems are all around us. When we decide upon the road to take to work, when we put the groceries in the fridge, when we decide which investments to make so as to maximize our expected profit, we are essentially solving an optimization problem (a shortest path problem, a packing problem, and a knapsack problem, respectively). For human beings (and many animal species), solving an optimization problem does not require any formal training—something which is immediately clear from the examples given here. The difference between exact solutions and approximate solutions, the difference between easy and hard optimization problems, or between fast (polynomial) and slow (exponential) algorithms are all moot to the average problem solver.

Indeed, the human mind seems to be formidably equipped from early childhood to solve an incredible range of problems, many of which could be easily modeled as optimization problems. Most likely, the ability to solve optimization problems adequately and quickly is one of the most important determinants of the probability of survival in all sentient species and has therefore been favored by evolution throughout time. Clearly, the human (and animal) mind solves optimization problems *heuristically* and not exactly—i.e., the solutions produced by the brain are by no means guaranteed to be optimal. Given what we now know about exact solution procedures, this makes perfect sense. When selecting which berries to pick from a bush so as to maximize the nutritional content of the selection (under the hard constraint that a saber-toothed tiger will catch us in less than a minute), it is much more important that this selection be calculated quickly rather than optimally. Given our knowledge on exact solution methods, we can now say that the calculation of the exact solution of our berry-picking problem would almost certainly be found only

after the saber-toothed tiger has sunk its fangs into us. Moreover, it would almost certainly require too much computing power from the brain, quickly depleting the body's scarce energy resources.

Given the diversity of problems the human mind must solve, including problems with which it has no prior experience, there is very little doubt that the human mind has the capacity (whether evolved or learned) to use metaheuristic strategies. Just like the metaheuristics for optimization that form the subject of this book, such strategies are not heuristics in themselves but are used to derive heuristics from. For example, when confronted with a new problem to which a solution is not immediately obvious (e.g., determining the berries to pick), the human mind will automatically attempt to find similar problems it has solved in the past (e.g., determining which roots to dig up from the ground) and attempt to derive the rules it has learned by solving this problem. This strategy is called *learning by analogy* [4]. Another example is called means end analysis [47] and can be summarized as follows: Given a current state and a goal state, choose an action that will lead to a new state that is closer to the goal state than the current state. This rule is iteratively applied until the goal state has been reached or no other state can be found closer to the goal state than the current state. Obviously, this strategy is a more general counterpart of all formal optimization heuristics that can be categorized as local search, in which a solution is iteratively improved using small, incremental operations we have come to call *moves*. The technique of *path relinking* [20], in which an incumbent solution is transformed, one move at a time, into a guiding solution, is another example of a formalized means-end-analysis strategy.

Whereas heuristics (and even metaheuristics) are completely natural to us humans, exact methods seem to be a very recent invention—coinciding with the introduction of the field of Operations Research around WWII. On the other hand, even though heuristics have been applied since the first life on earth evolved, the *scientific study* of heuristics also had to wait until the twentieth century. It could be hypothesized that heuristics come so naturally to us that we had to wait for the development of a formal theory of optimization—particularly linear programming—before anyone considered them a topic worthy of study. Much like a fish does not notice the water it is swimming in, heuristics may have been too ingrained in our thinking to attract attention as a distinct field of research.

Period 1: The Early Period

In 1945, immediately after WWII, the Hungarian mathematician George Pólya, then working at Stanford University, published a small volume called "How to Solve it" [41]. In his book, he argued that problems can be solved by a limited set of generally applicable strategies, most of which serve to make the problem simpler to solve. The book's focus was not on optimization problems, but on the more general class of "mathematical" problems, i.e., problems that can be modeled and solved by mathematical techniques. Nevertheless, most of the solution strategies proposed book are equally applicable to develop optimization algorithms.

The "analogy" principle, e.g., tells the problem solver to look for another problem that closely resembles the problem at hand, and to which a solution method is known. By studying the similarities and differences between both problems, ideas can be garnered to solve the original problem. The principle of "induction" is to solve a problem by deriving a generalization from some examples. The "auxiliary problem" idea asks whether a subproblem exists that can help to solve the overall problem.

Even though it is a bit of stretch to call these principles "meta-heuristics," it is clear that the start of the field of OR also marks the age during which people start thinking about more general principles that are useful in the design of heuristic algorithms (or solution methods for other types of problems). A case can be made for the fact that many of Pólya's principles are still heavily used today by heuristic designers. Looking for similar problems in the literature or elsewhere, and modifying the best-known methods for them to suit the problem at hand (analogy), is an extremely common strategy to arrive at a good heuristic fast. Solving some simple examples by hand, and using the lessons learned from your own (or someone else's) perceived strategy to derive an intelligent solution strategy from (induction), is also a useful technique. Finally, decomposing a problem into smaller subproblems and developing specialized techniques for each of them (auxiliary problem) has proven to be a powerful heuristic design strategy on a large number of occasions.

What is important is that none of Pólya's strategies actually solves any problem, nor can they be called "algorithms" in themselves. Instead, they are high-level, meta-strategies that are useful to influence the way a heuristic designer thinks about a problem. In that sense at the very least, they are very like the more advanced and specialized metaheuristic frameworks that we have today.

Several very high-level algorithmic ideas also came about around this period. The fact that good solutions can be reached by a constructive procedure, for example, is one of them. A constructive algorithm is one that starts from an empty solution and iteratively adds one element at a time until a complete solution has been formed. Simple rules for selecting this element from the set of all potential elements have led to different types of algorithms. The *greedy* selection rule selects the best (value for each) element at each iteration. Kruskal's or Prim's algorithm for the minimum spanning tree problem, Dijkstra's algorithm for the shortest path problem, etc., are all examples of greedy heuristics [9]. *Regret* algorithms present a similar class of optimization procedures that select, at each iteration, the element for which *not* selecting its best value results in the highest penalty cost. Vogel's approximation method [46] for the transportation problem is a well-known example. Again, calling the greedy idea or the regret idea "metaheuristics" is a bit of a stretch, but they *are* high-level strategies, and they are *not* algorithms themselves.

Also during this period, [48] see heuristics specifically as fit to solve what they call "ill-structured" problems. Contrary to well-structured problems, such problems cannot be formulated explicitly or solved by known and feasible computational techniques. Their predictions in 1958 have turned out to be slightly optimistic, but it cannot be denied that heuristics have turned out to be more flexible problem-solving strategies than exact methods.

Even though the heuristics developed in the early period were very simple, the realization that high-level strategies existed that could be used as the basis for the development of heuristics for *any* optimization problem led to insights that paved the way for more complex meta-strategies. Together with the widespread availability of computers, these developments took the field of heuristics into the next period in this history, the method-centric period.

Period 2: The Method-Centric Period

Even though the frameworks and ideas developed during what we have called the early period lacked the comprehensiveness of the later developed metaheuristic frameworks like tabu search [19], it is not too far-fetched to call them early metaheuristics. Like later metaheuristic frameworks, these methods offered—in the form of some generally applicable strategies—inspiration for the development of optimization algorithms. Of course, these principles still needed to be instantiated for each different optimization problem, but at least the process of coming up with an optimization strategy did not have to start from scratch.

Much of the work done in the early period can be characterized under the umbrella term of *artificial intelligence* because it involves mimicking human problem-solving behavior and learning lessons from this behavior on a more abstract level. Starting in the 1960s however, an entirely different line of research into problem-solving methods came to life. These methods used an analogy with life's main problem-solving method: evolution.

Evolution by natural selection has been called "the best idea ever" [5]. No single idea explains as much as Darwin's realization that species evolve over time to adapt to their environment. The way in which this happens, by natural selection of inheritable characteristics, is both so clever and so simple it begs the question why the world needed to wait until the second half of the nineteenth century before someone thought of it. Nevertheless, it took another century and the advent of the computer before researchers would become interested in simulating the process of natural evolution.

Although researchers in the late 1950s and early 1960s had developed what we would now label as evolutionary algorithms, their main aim was not to solve optimization problems, but to study the phenomenon of natural evolution. The insight that the principles of natural evolution could be used to solve optimization problems in general came in the early 1960s, when researchers like Box, Friedman, and several others had independently developed algorithms inspired by evolution for function optimization and machine learning. One of the first methods to receive some share of recognition was the so-called evolution strategy [as later reported in 42]. Evolution strategy was still quite far from what we would call an evolutionary algorithm: It did not use a population or crossover. One solution (called the parent) was mutated, and the best of the two solutions became the parent for the next round of mutation.

Evolutionary programming, introduced a few years later [16], represented solutions as finite-state machines but also lacked the concepts of both a population and crossover. The true start of the field of evolutionary algorithms came with the seminal work of John Holland [30], who was the first to recognize the importance of both concepts. With his schemata theorem, which essentially states that high-quality schemata ("parts") of solutions will increase in frequency in successive iterations of the algorithm, Holland was also among the pioneers of theory building in metaheuristics. The schemata theory was criticized later for its limited use and lack of general applicability, but it demonstrated that the field of metaheuristics needed not forever be devoid of theoretical underpinning.

It was perhaps the book by [28] (a student of John Holland) that truly sparked the evolutionary revolution. Evolutionary methods became extremely popular, journals and conferences specifically devoted to this topic sprouted and an exponentially increasing number of papers appeared in the literature. A large number of variants were proposed, each with its own specific characteristics. Extraordinary claims were made, not necessarily grounded in empirical evidence. The quest for a generic heuristic optimization method that could solve any problem efficiently, without requiring problem-specific information, seemed finally to be on the right track.

In the 1980s, the first papers start to appear that introduced general problem-solving frameworks not based on natural evolution. One of the first used another metaphor: annealing, the controlled heating and cooling process used in metallurgy and glass production to remove stresses from the material [35]. Simulated annealing used random solution changes and "accepted" these if they improved the solution or, if they did not, with a probability inversely proportional to the solution quality decrease and proportional to an external parameter called the "temperature."

For a while, it might have seemed that the development of metaheuristics was all about finding a suitable process to imitate. The 1980s, however, also saw the development of several methods that reached back to the early period and used ideas derived from human problem solving. One of the most powerful ideas was that solutions could be gradually improved by iteratively making small changes, called *moves*, to them. To this end, an algorithm would investigate all or some of the solutions that could be reached from the *current* solution by executing a single move. Together, these solutions form the *neighborhood* of the current solution.

Perhaps the most influential of the AI-based methods was tabu search [19]. The basic premise of this framework is that a local search algorithm could be guided toward a good solution by using some of the information gathered during the search in the past. To this end, the tabu search framework defined a number of memory structures that captured aspects of the search. The most emblematic is without a doubt the *tabu list*, a list that records attributes of solutions and prohibits, for a certain number of iterations, any solutions that exhibit an attribute on the tabu list.

The tabu search framework, however, extended far beyond the introduction of the tabu list as a memory structure. Probabilistic Tabu Search, introduced by [23], proposed a novel concept: selecting moves probabilistically from a list of the best-performing candidates. This idea later gained significant traction in its constructive

form (replacing probabilistic choice with random choice) under the name GRASP (see below). Curiously, it has been largely overlooked in the context of local search metaheuristics.

While the foundational aspects of tabu search are widely recognized, its full potential is often left untapped. Tabu search, a cornerstone of modern metaheuristics, was developed with a philosophy rooted in deriving and exploiting principles of intelligent problem solving [24]. One of its foundational concepts is the use of *flexible memory*, a feature designed to both capture historical information and leverage it for better decision-making. This duality—acquiring and profiting from information—sets tabu search apart as a highly adaptive and strategic optimization method. However, as the 1997 Tabu Search book [21] elaborates, its deeper mechanisms and broader potential often remain underexplored.

The memory structures in tabu search are defined along four principal dimensions: *recency, frequency, quality, and influence*. These dimensions, when combined with the logical structure and connectivity of the solution space, form a powerful framework for problem solving. Yet, despite their significance, many implementations narrowly focus on short-term "tabu memory," neglecting the broader strategic potential. This oversight diminishes the effectiveness of tabu search implementations, as the principles governing *frequency-based memory*, *aspiration criteria*, and *complementary memory structures* often go unused.

Glover and Laguna [21] examine several aspects of tabu search, like *move values and updates* and *complementary tabu memory structures*, which highlight methods to optimize solution pathways through effective memory utilization. Additionally, the discussions on *aspiration criteria* and *frequency-based memory* offer insights into balancing intensification and diversification, a critical aspect for escaping local optima and exploring the solution space more broadly. Similarly, [21] draws connections between tabu search and other metaheuristics like simulated annealing, genetic algorithms, and neural networks, offering complementary insights that could inspire more robust hybrid approaches.

Another underappreciated feature is *path relinking*, which facilitates both intensification and diversification by constructing solution paths between elite solutions, and which can be seen as a deterministic variant of the crossover concept in evolutionary algorithms. This concept, which emphasizes connectivity and systematic exploration, has immense potential but is rarely implemented to its full capacity. Sörensen and Schittekat [50], e.g., develop a path relinking method that creates a chain of intermediate solutions between two given solutions for the vehicle-routing problem in such a way that two neighboring solutions in the chain are separated by exactly one move and the length of the chain is minimal.

The same paper that introduced tabu search also coined the word "metaheuristic" [19]. However, not everybody agreed with this term and a push was made to use the (more modest) term "modern heuristics" instead. Clearly, not everybody agreed that the limited set of metaheuristics proposed by the 1980s had a higher-level aspect to them. Many still viewed them essentially as (admittedly, more complicated than their simple counterparts) algorithms, i.e., unambiguous step-by-

step sets of operations to be performed. Indeed, it is very common in the late 1980s for a "new metaheuristic" to be described based on a flowchart or another typical algorithmic representation. The widespread realization that metaheuristics could and should be viewed as general frameworks rather than as algorithms would come during the next period, the framework-centric period.

Threshold accepting [14], a simple variant of simulated annealing, demonstrated that a metaphor was certainly not necessary to develop a powerful general-purpose optimization framework. The great deluge method and record-to-record travel [13] differed from threshold accepting only by the way in which they accepted new solutions. Still, each of these was seen as different methods.

Interestingly, neural networks [32] were among the limited set of metaheuristics proposed by the late 1980s. These methods, inspired by the functioning of the brain (including neurons and synapses), were initially introduced in the context of pattern recognition, a domain where they continue to be used.

In recent years, the link between metaheuristics and neural networks—now commonly referred to as deep learning models—has become a topic of intense exploration. The idea that neural networks could potentially replace traditional metaheuristics has been gaining attention in the research community. However, we are far from realizing this vision. While neural networks show promise in learning and guiding heuristic decisions, they often lack the interpretability, robustness, and general-purpose applicability that make metaheuristics so versatile. Current efforts are focused on integrating the strengths of both approaches rather than viewing them as competing methodologies.

By 1995, research in metaheuristics had grown to a level that could sustain its own conference series, and thus the MIC (Metaheuristics International Conference) series was established. This conference still exists and was organized in Lorient (F) in July 2024. In the same year, the first issue of the Journal of Heuristics (http://link.springer.com/journal/10732/1/1/page/1), the only journal dedicated to publishing research in metaheuristics, was published. Like MIC, the Journal of Heuristics continues to be published.

Several other frameworks that had been proposed around the early 1990s also gained increasing interest during the mid-1990s. The innovation proposed in the GRASP (greedy randomized adaptive search procedure) framework was to modify a greedy heuristic by selecting at each iteration not necessarily the best element, but one of the best elements randomly [15]. GRASP adopted a probabilistic move selection like probabilistic tabu search later, when it was coupled with path relinking. Similarly, Ant Colony Optimization [7] proposed not only to mix deterministic and stochastic information but also proposed a way for solutions to exchange information.

Evolutionary algorithms invariably introduce a large amount of randomness in the search process. Some authors argued that it might be beneficial to reduce the reliance on randomness, but rather create algorithms that perform a more systematic search of the solution space. Scatter search and path relinking, both introduced in [20], are the most notable examples in which the principles of evolutionary algorithms were used with the randomness removed from them [22].

By the second half of the 1990s, however, it gradually became clear that metaheuristics based on metaphors would not necessarily lead to good approaches. The promised black-box optimizers that would always "just work" and that had attracted so much attention, seemed elusive. Even the theoretical studies lost some of their shine. The convergence results obtained for simulated annealing [29], because they only worked when an infinite running time was available, were not as compelling for practical situations as initially thought. Similarly, the automatic detection of good building blocks by genetic algorithms only really worked if such building blocks actually existed, and if they were not continually being destroyed by the crossover and mutation operators operating on the solutions. Even though the early metaheuristic frameworks offered some compelling ideas, they did not remove the need for an experienced heuristic designer. The advent of metaheuristics had not changed the simple fact that a metaheuristic that extensively exploited the characteristics of the optimization problem at hand would almost always be superior to one that took a black-box approach, regardless of the metaheuristic framework used.

In general, researchers during the method-centric period proposed *algorithms*, i.e., formalized structures that were meant to be followed like a cook book recipe. More often than not, the "new metaheuristic" was given a name, even when the difference between the new method and an existing method was small.

Period 3: The Framework-Centric Period

The insight that metaheuristics could be more usefully described as high-level algorithmic *frameworks*, rather than as algorithms, was a natural thing to happen. The main indicator that this mindset change was taking place—a change that has given rise to a period that we have dubbed the *framework-centric* period—is the increasing popularity of the so-called hybrid metaheuristics during the early 2000s. Indeed, this period could by rights have been called the "hybrid metaheuristic period." Whereas earlier researchers used to restrain themselves to a single metaheuristic framework, more and more researchers around the turn of the century combined ideas from different frameworks into a single heuristic algorithm. Some combinations became more popular than others, like the use of a constructive heuristic to generate an initial solution for a local search algorithm, or the use of GRASP to generate solutions that are then combined using path relinking.

One type of hybrid metaheuristic even received a distinct name: The use of local search (or any "local learning" approach) to improve solutions that are obtained by an evolutionary algorithm was called a *memetic* algorithm [40]. In 2004, the term "hybrid metaheuristic" had become common, and a new conference series with the same name was started.

The hybridization of metaheuristics, however, did not restrict itself to a combination of a metaheuristic with another metaheuristic. Opening up the individual algorithmic frameworks allowed researchers to combine a metaheuristic with any auxiliary method available. Constraint programming, linear programming,

and mixed-integer programming were all used in combinations with ideas from metaheuristics. The combination of metaheuristics and exact methods was coined "matheuristics" [38] (though these methods too had many antecedents in the metaheuristics literature). In 2006, the first edition of the Matheuristics conference took place.

Soon after its introduction, the term "hybrid" metaheuristic would become obsolete, as researchers made a general transition from seeing metaheuristics as algorithms of which some components could be borrowed by other metaheuristics, to general sets of concepts ("frameworks"). The "metaheuristic framework" concept entailed that metaheuristics were nothing more (or less) than a more or less coherent set of ideas, which could of course be freely combined with other ideas. Today, many researchers develop metaheuristics using their experience and knowledge about which methods will work well for certain problems and which most likely will not.

Some general patterns started to appear in the literature on which methods work well for which problems, and the community gravitated toward approaches that always delivered. For almost any variant of the vehicle-routing problem, e.g., a large majority of approaches use some form of local search as their main engine, generally in a multi-neighborhood framework like variable neighborhood search [39]. The use of several different local search operators or the use of several different constructive procedures in general is now a well-regarded strategy and often used as the first choice by heuristic designers. Clearly, variable neighborhood search presented a framework within which the use of multiple neighborhoods could be captured, but many other ways of combining several local search operators in a single heuristic are possible. The use of several constructive heuristics (usually combined with several destructive operators) in a single heuristic became known under the name large neighborhood search [1, 44, 57].

Crucial in this period, which is still ongoing, is that researchers do not have to propose a "new algorithm" anymore to get their papers published. By combining the most efficient operators of existing metaheuristic frameworks, and carefully tuning the resulting heuristic, algorithms can be created that solve any real-life optimization problem efficiently. Researchers can now focus on studying a single, mundane aspect of a metaheuristic framework in detail like, e.g., its stopping rules [43].

Rather than developing ever more complicated methods, in the first decades of the 2000s, a focus can be observed on frameworks that present a much simpler approach that—in many cases—is not much less effective. Iterated local search [36], which proposes to use a single local search operator in alternation with a single perturbation operator, has a long history and dates back to at least the early 1980s [3], yet is still as popular as ever. Its constructive counterpart, often called iterated greedy [17, 45], has recently gained a large amount of traction, despite the fact that it can be seen as a restricted form of strategic oscillation (SO), a technique introduced in [18] often employed in the context of tabu search. SO offers a multitude of concepts and ideas to allow the search to move within and between regions demarcated by various boundaries within the search space, such as those defining feasibility or local optimality or thresholds for various functions

(such as objective functions or sums of variables). Reference to such boundaries enriches the search process by introducing different types of moves and evaluations depending on which side of the boundary the search lies on, and on whether the search is moving toward or away from the boundary. Moves leading toward or away from a boundary are joined by moves launched at the boundary and at selected distances from the boundary which involve more complex searches (for example, by employing exchange moves in place of constructive or destructive moves). Successful applications of strategic oscillation are reported in [8,21,33,37]. An apparent paradox is that the simplification of a metaheuristic to a few simple rules, which may restrict both its scope and its power, seems to increase its popularity. A possible explanation is that it renders the framework more accessible to nonexpert users, who—contrary to the scientific community—value robustness, ease of development, and flexibility over performance.

Traditionally, the metaheuristic community has put a heavy focus on *performance*. Research is only considered good if (and only if) it produces a heuristic algorithm that "performs" well with respect to some benchmark, such as another heuristic or a lower bound. This has been called the "up-the-wall game" (though it might also be called the "one-upmanship game"). All other contributions (e.g., heuristics that are many times simpler than the best-performing heuristic in the literature, studies on heuristics that should perform well but for some reason do not, ...) are much more difficult to publish. However, several researchers have pointed out the adverse effects of this paradigm (one of the earliest was probably [31]), and some recent contributions that go beyond the up-the-wall game demonstrate the framework-centric period is gradually transforming into the *scientific* period. In this period the study of metaheuristics will shift its focus from performance to understanding. Unfortunately, however, not all of the metaheuristics community makes the transition to the framework-centric period, and we are forced to report on a period which essentially runs in parallel with this period.

The Metaphor-Centric Period

Starting in the 1980s, a subfield emerged of research (we hesitate to put quote marks around the terms for reasons explained below) that focuses on developing new metaheuristic methods based on metaphors drawn from natural or man-made processes. In our historical narrative, this period has not been assigned a distinct number because it does not fit chronologically between other periods but instead represents a sidestep occurring in parallel with the framework-centric period.

While metaphors have historically served as a source of inspiration for the development of early metaheuristics, they have always been understood to have limitations. A metaphor is, by definition, a simplified analogy that eventually breaks down when scrutinized too closely. It can inspire innovation but does not inherently provide a robust justification for metaheuristic design choices or serve as a foundation for entirely new frameworks.

In recent years, however, a troubling trend has emerged in a subset of the metaheuristics community: a fixation on creating "novel" metaheuristics solely by introducing new metaphors. This metaphor-based subfield has produced an ever-growing list of processes used as inspiration, ranging from natural phenomena like ants, bees, termites, and bats, to man-made and abstract concepts such as league championships, cloud formation, black holes, and even the Lorentz transformation. The breadth of these inspirations appears limitless, with little regard for whether the underlying process genuinely involves optimization.

One would expect that, at a minimum, a process used as the basis for a metaheuristic should inherently involve some form of optimization. For example, annealing processes minimize energy levels, natural evolution minimizes the mismatch between a species and its environment, and ants minimize the distance between their nest and food sources. Yet, many of the proposed frameworks are based on processes that, by no stretch of the imagination, involve optimization—fireworks, mine blasts, and cloud formation are just a few examples.

The causes and consequences of this "metaphor fallacy" have been analyzed extensively in other publications [2, 49, 56], where the conclusion is clear: This approach is not scientific. This is not the place to revisit every argument against metaphor-based metaheuristics, but it suffices to say that such methods represent a deviation from the scientific rigor expected in the field. Nevertheless, metaphor-driven "novelty" has taken up a (dark) page in the history of metaheuristics—a page that the field would do well to turn quickly.

Period 4: The Scientific Period?

For a long time, the field of metaheuristics has had difficulties to be taken seriously. In 1977, one of the authors of this chapter wrote "[exact] algorithms are conceived in analytic purity in the high citadels of academic research, heuristics are midwifed by expediency in the dark corners of the practitioner's lair [...] and are accorded lower status." [18] Traditionally, the theoretical underpinning of heuristics and metaheuristics has not been on par with that of other areas in OR, more specifically exact methods. The development of heuristic optimization algorithms, whether using a metaheuristic framework or not, is guided by experience, not theory. Early attempts to firmly ground the development of metaheuristics in theory have not delivered upon their promises. Understanding the behavior of metaheuristics on a fundamental level has proven to be a difficult task, notwithstanding several noteworthy efforts [e.g., 54, 55].

Nevertheless, it is hard to argue with success. The obvious usefulness of metaheuristics in practical optimization problems has drawn researchers to improve the frameworks and methods developed. To solve a large majority of real-life optimization problems, heuristics are—and will remain—the only option, whether developed using a metaheuristic framework or not. Nevertheless, there are many things that can be improved about the way the metaheuristic community operates. To list just a few:

- The establishing of adequate *testing protocols* to ensure that algorithms perform as well as they are claimed to do. Here it seems the community still has a lot to learn. Unlike many fields of computational analysis, it is common in the field of metaheuristics to both develop and test an algorithm against the same set of benchmark instances, rather than separating these instances into a training set and a test set, as is standard practice in machine learning and other computational disciplines. This lack of separation introduces a risk of overfitting, where the algorithm is unintentionally fine-tuned to perform well on the benchmark set, rather than demonstrating robust performance across unseen instances.
- The introduction of *meta-analysis* (i.e., a review of a clearly formulated question that uses systematic methods to identify, select, and evaluate relevant research, as well as to collect and analyze data from relevant studies) to the field of metaheuristics ([53]; Hvattum, 2015, Personal communication). Again, there is still a lot of room for improvement, and the initial attempts to introduce this type of analysis into the metaheuristic research have not yet gained sufficient traction.
- The requirement to disclose *source code*, so that researchers can check and build on each other's work in a more efficient way, without reinventing the wheel. Like the previous point, journals that require source code to go along with the published paper are still few and far between.
- A point where considerable progress has been made over the past few years is in the development of powerful *general-purpose heuristic solvers* to decrease development time, like CPLEX or Gurobi, but then heuristically. Hexaly (https://www.hexaly.com) (formerly LocalSolver), for example, is now a robust product that is used heavily in several production environments (e.g., Amazon Web Services uses it to schedule server capacity). TimeFold is a commercially supported open-source general-purpose solver that focuses on solving planning and scheduling problems (https://timefold.ai/). InsideOpt solver (https://insideopt.com/) boasts the ability to handle uncertain data and multiple objectives.

  These existing solvers provide decision-makers with an alternative to hand-developing a heuristic optimization algorithm for their problem, since they are able to solve a model using a set of built-in heuristic operators (usually based on constructive or local search metaheuristics).
- Supporting these general-purpose solvers, the development of a powerful and generally accepted *modeling language* geared more toward the development of heuristics and less toward the MIP paradigm. While there definitely is no generally accepted modeling language, we do see that the examples mentioned above offer a far more expressive set of constructs that can be used to make the modeling of real-life problems much easier and more intuitive. Hexaly, e.g., offers a *list*-type variable that essentially encodes a permutation directly. This allows the modeling of all types of routing and scheduling problems in a far more natural way than MIP solvers (which require expressing routes and schedules using binary variables, combined with sub-tour elimination constraints, e.g.).

Importantly, the change from a performance-driven community to a community in which scientific understanding is more important will take place during the

scientific period. Without doubt, this will lead to the development of even better heuristics, even more efficient, but it will also lead to heuristics that are usable outside of the developer's lab environment.

To finish this section, we want to mention two topics that have garnered a lot of attention recently:

- The recent advances in *machine learning*, combined with the ubiquity of large language models, have sparked renewed interest in the combination of machine learning and metaheuristic approaches. This integration is being explored in both directions [52]. Machine learning algorithms often involve (or even constitute) optimization problems, and heuristics play an important role in solving these, especially given the size of typical machine learning problems. Conversely, machine learning tools have significant potential for modeling optimization problems and designing optimization algorithms [10]. The field that studies the integration of machine learning and metaheuristics is too vast to summarize in a single paragraph, but there is little doubt that these domains will converge in the coming years—which will lead to considerable advances in both.
- Metaheuristics are increasingly being explored in the context of *quantum computing*. Although still in the experimental stage, this integration holds potential for advancing optimization methods in problems where classical computing faces scalability limitations. An in-depth treatment of the current research in this area is out of the scope of this chapter, so we limit ourselves to mentioning the use of the QUBO (Quadratic Unconstrained Binary Optimization) model [25–27]. A significant reason for the QUBO model's usefulness lies in its mathematical equivalence to the Ising spin glass problem, a well-studied problem in physics. This equivalence allows QUBO problems to be directly mapped onto quantum annealers (like those from D-Wave Systems) or gate-based quantum computers (from companies like IBM and Google).

Conclusions

Outlining the history of the field of metaheuristics in a few pages is not an easy undertaking, and completeness is a goal that simply cannot be achieved. In this chapter, we have attempted to clarify the evolution of this field by not focusing exclusively on important events or publications, but by identifying the important paradigm shifts that the field has encountered. What is certain is that the use of metaheuristics is older—much older—than the term itself. As mentioned, our brain itself houses some powerful metaheuristics that have helped humans survive since the dawn of mankind. The scientific study of metaheuristics, however, had to wait until the second half of the previous century.

Scientific communities invariably develop a conceptual framework within which a few axioms are held to be true. This can also be said of the metaheuristics community. It is those shared truths that we have attempted to uncover in this chapter. Even though the field of metaheuristics is still young, it has already

undergone several paradigm shifts that have changed the way researchers approach the development of heuristic optimization methods.

The transition from the method-centric to the framework-centric period has been beneficial for the entire community, and there is no doubt that the transition toward the scientific period can take the field further in the right direction. Metaheuristics are a fascinating area of study with highly significant practical ramifications, and the field will certainly keep evolving in the foreseeable future. There is no doubt that a more scientific, less dogmatic, and broader point of view can help us all achieve our goals: the development of efficient methods to solve the most challenging and important real-life optimization problems.

Cross-References

- Evolutionary Algorithms
- Fundamentals of Scatter Search
- Genetic Algorithms
- GRASP
- Hyper-heuristics
- Iterated Local Search
- Memetic Algorithms
- Tabu Search
- Variable Neighborhood Search

References

1. Ahuja RK, Ergun Ö, Orlin JB, Punnen AP (2002) A survey of very large-scale neighborhood search techniques. Discret Appl Math 123(1):75–102
2. Aranha C, Camacho Villalón CL, Campelo F, Dorigo M, Ruiz R, Sevaux M, Sörensen K, Stützle T (2021) Metaphor-based metaheuristics, a call for action: the elephant in the room. Swarm Intel 16(1):1–6. https://doi.org/10.1007/s11721-021-00202-9
3. Baxter J (1981) Local optima avoidance in depot location. J Oper Res Soc 32(9):815–819
4. Carbonell JG (1983) Learning by analogy: formulating and generalizing plans from past experience. In: Machine learning. Springer Science & Business Media, pp 137–161. https://doi.org/10.1007/978-3-662-12405-5_5
5. Chu T (2014) Human purpose and transhuman potential: a cosmic vision for our future evolution. Origin Press. ISBN 978-1-57983-0250
6. Clarke G, Wright JW (1964) Scheduling of vehicles from a central depot to a number of delivery points. Oper Res 12(4):568–581. https://doi.org/10.1287/opre.12.4.568
7. Colorni A, Dorigo M, Maniezzo V (1992) Distributed optimization by ant colonies. In: Varela FJ, Bourgine P (eds) Proceedings of the first European conference on artificial life. MIT Press, Cambridge, MA, pp 134–142
8. Corberán Á, Peiró J, Campos V, Glover F, Martí R (2016) Strategic oscillation for the capacitated hub location problem with modular links. J Heurist 22(2):221–244
9. Cormen TH, Leiserson CE, Rivest RL, Stein C (2009) Introduction to algorithms, 3rd edn. MIT Press, Cambridge, MA. ISBN 978-0-262-03384-8

10. da Costa Oliveira AL, Britto A, Gusmão R (2023) Machine learning enhancing metaheuristics: a systematic review. Soft Comput 27(21):15971–15998
11. Dantzig GB (1951) Maximization of a linear function of variables subject to linear inequalities, pp 339–347. Wiley, New York
12. Dijkstra EW (1959) A note on two problems in connexion with graphs. Numerische Mathematik 1(1):269–271. https://doi.org/10.1007/BF01386390
13. Dueck G (1993) New optimization heuristics. J Comput Phys 104(1):86–92. https://doi.org/10.1006/jcph.1993.1010
14. Dueck G, Scheuer T (1990) Threshold accepting: a general purpose optimization algorithm appearing superior to simulated annealing. J Comput Phys 90(1):161–175. https://doi.org/10.1016/0021-9991(90)90201-b
15. Feo TA, Resende MGC (1995) Greedy randomized adaptive search procedures. J Glob Optim 6(2):109–133. https://doi.org/10.1007/bf01096763
16. Fogel LJ, Owens AJ, Walsh MJ (1966) Artificial intelligence through simulated evolution. Wiley, New York
17. García-Martínez C, Rodriguez FJ, Lozano M (2014) Tabu-enhanced iterated greedy algorithm: a case study in the quadratic multiple knapsack problem. Eur J Oper Res 232(3):454–463
18. Glover F (1977) Heuristics for integer programming using surrogate constraints. Decis Sci 8(1):156–166. https://doi.org/10.1111/j.1540-5915.1977.tb01074.x
19. Glover F (1986) Future paths for integer programming and links to artificial intelligence. Comput Oper Res 13(5):533–549. https://doi.org/10.1016/0305-0548(86)90048-1
20. Glover F (1998) A template for scatter search and path relinking. In: Artificial evolution. Lecture notes in computer Science. Springer Science & Business Media, pp 1–51. https://doi.org/10.1007/bfb0026589
21. Glover F, Laguna M (1997) Tabu search. Kluwer Academic Publishers/Springer, Dordrecht
22. Glover F, Laguna M, Martí R (2000) Fundamentals of scatter search and path relinking. Control Cybern 29(3):653–684
23. Glover F (1989) Tabu search—part I. ORSA J Comput 1(3):190–206
24. Glover F, Laguna M (1993) Tabu search. In: Modern heuristic techniques for combinatorial problems, pp 70–150
25. Glover F, Kochenberger G, Du Y (2022) Applications and computational advances for solving the QUBO model. In: The quadratic unconstrained binary optimization problem: theory, algorithms, and applications. Springer, Berlin, pp 39–56
26. Glover F, Kochenberger G, Hennig R, Du Y (2022) Quantum bridge analytics I: a tutorial on formulating and using QUBO models. Ann Oper Res 314(1):141–183
27. Glover F, Kochenberger G, Ma M, Du Y (2022) Quantum bridge analytics II: QUBO-plus, network optimization and combinatorial chaining for asset exchange. Ann Oper Res 314(1):185–212
28. Goldberg D (1989) Genetic algorithms in search, optimization and machine learning. Addison-Wesley, Reading
29. Granville V, Krivanek M, Rasson J-P (1994) Simulated annealing: a proof of convergence. IEEE Trans Pattern Anal Mach Intel 16(6):652–656. https://doi.org/10.1109/34.295910
30. Holland J (1975) Adaptation in natural and artificial systems. University of Michigan Press, Ann Arbor
31. Hooker JN (1995) Testing heuristics: we have it all wrong. J Heurist 1:33–42
32. Hopfield J (1982) Neural networks and physical systems with emergent collective computational capabilities. Proc Natl Acad Sci 79(8):2254–2558
33. Hvattum LM, Løkketangen A, Glover F (2004) Adaptive memory search for boolean optimization problems. Discret Appl Math 142(1):99–109
34. Jarboui B, Derbel H, Hanafi S, Mladenović N (2013) Variable neighborhood search for location routing. Comput Oper Res 40(1):47–57. https://doi.org/10.1016/j.cor.2012.05.009
35. Kirkpatrick S, Gelatt CD, Vecchi MP (1983) Optimization by simulated annealing. Science 220(4598):671–680. https://doi.org/10.1126/science.220.4598.671

36. Lourenço HR, Martin OC, Stützle T (2003) Iterated local search. In: Handbook of metaheuristics. Springer US, Boston, MA, pp 320–353
37. Lozano M, Glover F, García-Martínez C, Rodríguez FJ, Martí R (2014) Tabu search with strategic oscillation for the quadratic minimum spanning tree. IIE Trans 46(4):414–428
38. Maniezzo V, Stützle T, Voß S (eds) (2010) Matheuristics. Springer. https://doi.org/10.1007/978-1-4419-1306-7
39. Mladenović N, Hansen P (1997) Variable neighborhood search. Comput Oper Res 24(11): 1097–1100. https://doi.org/10.1016/s0305-0548(97)00031-2
40. Moscato P (1989) On evolution, search, optimization, genetic algorithms and martial arts—towards memetic algorithms. Technical report 826, Caltech Concurrent Computation Program, Pasadena
41. Polya G (1945) How to solve it: a new aspect of mathematical method. Princeton University Press, New Jersey
42. Rechenberg I (1989) Evolution strategy: nature's way of optimization. In: Optimization: methods and applications, possibilities and limitations. Springer Science & Business Media, pp 106–126. https://doi.org/10.1007/978-3-642-83814-9_6
43. Ribeiro CC, Rosseti I, Souza RC (2011) Effective probabilistic stopping rules for randomized metaheuristics: GRASP implementations. In: Learning and intelligent optimization. Lecture notes in computer science, vol 6683. Springer Science & Business Media, pp 146–160. https://doi.org/10.1007/978-3-642-25566-3_11
44. Ropke S, Pisinger D (2006) An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows. Transp Sci 40(4):455–472
45. Ruiz R, Stützle T (2008) An iterated greedy heuristic for the sequence dependent setup times flowshop problem with makespan and weighted tardiness objectives. Eur J Oper Res 187(3): 1143–1159
46. Shore HH (1970) The transportation problem and the VOGEL approximation method. Decis Sci 1(3–4):441–457. https://doi.org/10.1111/j.1540-5915.1970.tb00792.x
47. Simon HA (1996) The sciences of the artificial, 3rd edn. MIT Press, Cambridge
48. Simon HA, Newell A (1958) Heuristic problem solving: the next advance in operations research. Oper Res 6(1):1–10. https://doi.org/10.1287/opre.6.1.1
49. Sörensen K (2015) Metaheuristics—the metaphor exposed. Int Trans Oper Res 22(1):3–18. https://doi.org/10.1111/itor.12001
50. Sörensen K, Schittekat P (2013) Statistical analysis of distance-based path relinking for the capacitated vehicle routing problem. Comput Oper Res 40(12):3197–3205
51. Sörensen K, Glover FW (2013) Metaheuristics. In: Gass S, Fu M (eds) Encyclopedia of operations research and management science. Springer, Boston, MA, pp 960–970
52. Talbi E-G (2021) Machine learning into metaheuristics: a survey and taxonomy. ACM Comput Surv (CSUR) 54(6):1–32
53. Turkeš R, Sörensen K, Hvattum LM (2021) Meta-analysis of metaheuristics: quantifying the effect of adaptiveness in adaptive large neighborhood search. Eur J Oper Res 292(2):423–442. https://doi.org/10.1016/j.ejor.2020.10.045
54. Watson J-P, Barbulescu L, Whitley LD, Howe AE (2002) Contrasting structured and random permutation flow-shop scheduling problems: search-space topology and algorithm performance. INFORMS J Comput 14(2):98–123. https://doi.org/10.1287/ijoc.14.2.98.120
55. Watson J-P, Beck JC, Howe AE, Whitley LD (2003) Problem difficulty for tabu search in job-shop scheduling. Artif Intel 143(2):189–217. https://doi.org/10.1016/s0004-3702(02)00363-6
56. Weyland D (2010) A rigorous analysis of the harmony search algorithm. Int J Appl Metaheuristic Comput 1(2):50–60. https://doi.org/10.4018/jamc.2010040104
57. Yagiura M, Iwasaki S, Ibaraki T, Glover F (2004) A very large-scale neighborhood search algorithm for the multi-resource generalized assignment problem. Discret Optim 1(1):87–98

Alternate Optima in Combinatorial Optimization

34

Marc Benito-Marimón, Anna Martínez-Gavara, and Rafael Martí

Contents

Introduction . . . 1012
Related Work . . . 1013
Solution Robustness . . . 1014
Problem Flexibility . . . 1014
Inference from an Optimal Solution . . . 1014
Linear Ordering Problem . . . 1015
Mathematical Models . . . 1017
Algorithmic Approach . . . 1020
Practical Cases with Multiple Optima . . . 1024
College Rankings and Fairness . . . 1024
Premier League . . . 1026
Conclusion . . . 1027
Cross-References . . . 1028
References . . . 1028

Abstract

This chapter explores the importance of studying the entire set of optimal solutions in combinatorial optimization problems, rather than focusing solely on a single optimum. Understanding alternate optima is crucial for robust decision-making and comprehensive problem analysis. Using the Linear Ordering Problem (LOP) as a primary example, we illustrate how multiple optimal rankings can lead to significantly different interpretations of results, potentially

M. Benito-Marimón (✉) · A. Martínez-Gavara
Universidad de Valencia, Valencia, Spain
e-mail: marc.benito@uv.es; gavara@uv.es

R. Martí
Operations Research and Statistics Department, School of Mathematics, University of Valencia, Valencia, Spain
e-mail: rafael.marti@uv.es

© Springer Nature Switzerland AG 2025

R. Martí et al. (eds.), *Handbook of Heuristics*,
https://doi.org/10.1007/978-3-032-00385-0_81

affecting real-world decisions. This chapter discusses various motivations for considering the optimal solution set, including improved insight into problem structure, enhanced stability analysis, and more accurate decision support. We examine how this approach can reveal hidden trade-offs, identify critical variables, and provide a more complete understanding of the solution space. By highlighting these motivations across various domains, we aim to encourage a shift in perspective within the field of combinatorial optimization, promoting more thorough and reliable analysis of complex problems. Although single optimum models can be extended to identify multiple optimal solutions, they have important limitations to target medium and large instances, as can be expected. We illustrate this typical performance on the LOP and how heuristics constitute a practical alternative.

Keywords

Multiple optimal solutions · Linear ordering problem · Variable neighborhood search metaheuristics

Introduction

In the field of combinatorial optimization, traditional focus has mainly relied on finding a single optimal solution. While this approach is valuable, it often overlooks a critical aspect: the existence and implications of multiple optimal solutions. As optimization techniques play an important role in decision-making across various domains, from logistics and finance to healthcare and environmental management, the importance of understanding and analyzing the entire set of optimal solutions becomes fundamental.

Studying alternate optima is not merely an academic exercise; it holds profound practical implications for decision-makers and analysts alike. The objective of this chapter is to give relevance to the study of the optimal solution set, emphasizing three key aspects: solution robustness, problem flexibility, and the challenges associated with drawing conclusions from one particular optimal solutions.

Firstly, the concept of solution robustness is intrinsically tied to the existence of alternate optima. In real-world applications, the parameters and constraints of optimization problems are often subject to uncertainties or fluctuations. By examining the set of all optimal solutions, we can assess the stability of our results and identify solutions that remain optimal or near optimal under various scenarios. This robustness analysis is crucial for implementing solutions that can withstand the test of time and changing conditions.

Secondly, the presence of multiple optimal solutions helps to address the needed flexibility of real-world optimization problems. For empirical problems, it is interesting to allow decision-makers to choose among equally optimal alternatives based on secondary criteria or qualitative factors not captured in the original objective function. If multiple optimal solutions are known, the implementation can be quickly switched to another solution without much interrupting the optimal

system performance. Understanding the full set of optimal solutions thus provides a richer context for decision-making, enabling more nuanced choices. If multiple optimal solutions are known, the implementation can be quickly switched to another solution without interrupting the optimal system performance. We may say that the study of alternate optimal solutions constitutes a bridge between single and multi-objective optimization and provides insights to disclose if a multi-objective model would suit better to the optimization problem that we are formulating due, for example, to the existence of different structures in the alternate optimal solutions.

Thirdly, and perhaps most critically, the existence of alternate optima poses significant challenges when attempting to draw conclusions from optimization results. In many fields, such as data analysis, policy-making, or scientific research, optimization techniques are used not just to find a solution, but to infer underlying structures or relationships. When multiple optimal solutions exist, conclusions drawn from a single optimum may be misleading or incomplete. This chapter will explore how considering the entire set of optimal solutions can lead to more reliable and comprehensive insights.

Throughout this chapter, we will use the Linear Ordering Problem (LOP) and some results from [11] as a primary example to illustrate these concepts. The LOP, with its applications in diverse areas such as economics, sociology, and bioinformatics, serves as an excellent case study for the implications of alternate optima. We will show how different optimal rankings in the LOP can lead to substantially different interpretations of the underlying data, highlighting the necessity of a more pragmatic approach to optimization analysis. By examining these aspects, this chapter aims to shift the paradigm in combinatorial optimization from a singular focus on finding "the" optimal solution to a more comprehensive understanding of the optimal solution space. This perspective not only enhances the theoretical foundations of optimization but also significantly improves its practical applications, leading to more robust solutions and insightful decision-making processes.

Related Work

The process of finding all the optimal solutions for a given optimization problem is not always practical for real-world situations. For instance, in well-known problems such as the Linear Ordering Problem, we may find instances with more than 500,000 optimal solution, making it unrealistic to compute the entire set of optimal solutions within reasonable computing time. So, depending on the problem, you may need to propose an alternative, such as computing only representative solutions or the ones that express better the characteristics of the optimal set of solutions. From a theoretical perspective, previous work focused on obtaining alternative optimal solutions for Mixed-Integer Linear Programs (MILPs). Implementations of these kinds of algorithms can be seen in the pool-based modes of popular solvers such as Gurobi and CPLEX. Additionally, due to the high computational cost of generating the complete set of optimal solutions—which can be substantial—there are studies [4] aimed at finding not just the optimal solution but also a set of diverse solutions that maximizes variation. However, because these studies have a general scope, they often lack efficiency and scalability for specific combinatorial problems.

In this section, we will review the previous work done regarding each one of the three key aspects mentioned above, solution robustness, problem flexibility, and the challenge of inferring conclusions from a combinatorial optimization problem. It is worth mentioning that in well-studied problems such as the Capacitated Vehicle Routing Problem (CVRP) [6] or job scheduling, the issues of robustness and flexibility are often addressed through more specific methodologies. These may include incorporating new constraints and variables into the models or utilizing multi-objective models that account for uncertainties or flexibility.

Solution Robustness

The concept of solution robustness in optimization problems was first formally introduced in [15]. While not specifically focused on alternate optima, it laid the groundwork for considering uncertainties and variations in optimization problems. Specifically, Mulvey et al.[15] proposed a framework for robust optimization that aimed to find solutions that perform well across a range of scenarios, rather than just in a single, deterministic case. When faced with an optimization problem, this framework modifies the mathematical model of the problem including probability distributions in order to maximize the stability of an optimal solution, instead of computing all the solutions and analyzing the stability.

Problem Flexibility

In practical optimization problems dealing with high-volatility scenarios, it is important to have multiple optimal solutions. This allows for quick adaptation to changing conditions without interrupting the performance of the system. It is unnecessary to calculate all the optimal solutions; instead, it is sufficient to identify a diverse set of solutions that try to cover all possible scenarios. This ensures that if one solution becomes unfeasible due to changes in the environment, there is always an alternative to rely on. In such cases, a general multimodal optimization algorithm may be sufficient. For instance, in wireless sensor networks (WSNs) [17], multiple optimal solutions for node placement may initially satisfy coverage and connectivity requirements. However, as the network operates, factors such as battery depletion, shifting traffic patterns, or node failures can necessitate adjustments. Having multiple preidentified optimal configurations allows the system to switch between solutions dynamically, ensuring continuous performance while balancing energy consumption or maintaining network integrity.

Inference from an Optimal Solution

As stated above, drawing a conclusion after a single optimal solution may be biased and premature. For example, in the field of evolutionary biology, the inference of phylogenetic trees is a critical task that aims to reconstruct the evolutionary relation-

ships among species or genes. When using the maximum-parsimony method, which seeks to minimize the number of evolutionary changes required to explain observed traits, multiple phylogenetic trees can equally well explain the observed genetic or morphological data. This phenomenon, known as phylogenetic uncertainty, underscores the importance of obtaining and analyzing multiple optimal solutions rather than settling for a single "best" tree. By considering alternate optimal trees, researchers can gain a more nuanced understanding of evolutionary relationships, assess the robustness of their conclusions, and avoid overconfidence in a single reconstruction that may be arbitrary among equally plausible alternatives. For instance, [16] demonstrated that different optimal trees might suggest conflicting evolutionary hypotheses, such as the timing of speciation events or the origin of specific traits. Moreover, as shown by [8], the consensus of multiple optimal trees often provides a more accurate representation of evolutionary relationships than any single tree. This approach also allows for the identification of stable clades across different reconstructions, providing a measure of confidence in specific evolutionary groupings. Furthermore, exploring the landscape of optimal and near-optimal trees can reveal important biological insights, such as rapid evolutionary radiations or instances of horizontal gene transfer, which might be masked when focusing on a single optimal solution. Therefore, in phylogenetic analysis, the pursuit and examination of multiple optimal solutions is not merely a statistical exercise, but a fundamental approach to gaining a comprehensive and reliable understanding of evolutionary history.

Linear Ordering Problem

In this section we will use the Linear Ordering Problem (LOP) as a vehicular example to show why and how to deal with multiple optimal solutions.

The Linear Ordering Problem (LOP) involves finding a permutation p of the columns and rows of an $n \times n$ weight matrix $C = (c_{ij})$ in order to maximize the sum of the weights in the triangular region above the main diagonal. This matrix is known in some applications as the dominance matrix, where the entry (i, j) indicates the degree to which element i dominates element j.

Linear ordering problems arise whenever there is a need to determine rankings for certain objects. For instance, given a set of n objects $O_1, O_2, \ldots, O_n$ rated by m individuals based on their individual preferences, the goal is to establish a ranking that globally reflects these individual rankings as closely as possible. This formulation is one of the oldest applications of the LOP, originally proposed in 1959 by [9]. In this problem, the entry (i, j) in the dominance matrix corresponds to the number of individuals who prefer object O_i over object O_j.

The most significant application of the Linear Ordering Problem (LOP) is likely the triangulation of input-output tables in economics. This was a central component of the study for which Leontiev[10] received the Nobel Prize in 1973. These tables illustrate the interdependencies among different branches of the economy, categorized into sectors, where the entries reflect the transactions between those sectors.

Triangulation is a method used to establish a hierarchy of all sectors in such a way that the flow of transactions that contradicts this hierarchy (specifically, transactions from sectors ranked lower to those ranked higher) is minimized. Establishing such rankings provides useful information of a country's industrial structure and allows for comparisons between different countries.

The LOP can be modeled using the following linear integer model, featuring binary variables x_{ij}, where $i, j = 1, \ldots, n$, indicating whether element i precedes element j.

$$\begin{aligned} \max f(x) &= \sum_{(i,j)\in A} c_{ij} x_{ij} \\ x_{ij} + x_{ji} &= 1, \text{ for } i, j = 1, \ldots, n, i < j, \\ x_{ij} + x_{jk} + x_{ki} &\leq 2, \text{ for } i, j, k = 1, \ldots, n, i < j, i < k, j \neq k, \\ x_{ij} &\in \{0, 1\}, \text{ for all } i, j = 1, \ldots, n. \end{aligned} \tag{1}$$

This 0/1-IP can be considered as the canonical IP formulation of the problem [13].

Considering only a single optimal ranking can result in premature conclusions, especially when there are multiple optimal rankings that differ significantly. In the case of the LOP, it is evident that having more than one optimal solution may be a critical issue that jeopardizes some of the interpretations of the solutions mentioned above.

The examination of various optimal solutions has significant implications; however, it has been largely ignored in the literature on Linear Ordering Problems (LOP) according to [13]. Anderson et al. [1] were the first to investigate the analysis of alternate optimal LOP solutions, revealing that certain elements maintain relative stability across different optimal solutions, while others may vary significantly in their rankings, thereby threatening the applicability of LOP. To exemplify this situation, the authors present the following 10 optimal solutions for the `N-pal13` instance (referred to as the *Paley graph*), sourced from the publicly available LOLIB—Linear Ordering LIBrary [12]:

1. ⟨8, 9, 3, 12, 7, 1, 10, 4, 13, 2, 11, 5, 6⟩
2. ⟨8, 9, 3, 12, 6, 1, 10, 4, 13, 7, 2, 11, 5⟩
3. ⟨8, 9, 3, 12, 1, 10, 4, 13, 7, 2, 11, 5, 6⟩
4. ⟨9, 12, 1, 10, 4, 13, 7, 8, 2, 11, 5, 3, 6⟩
5. ⟨9, 12, 7, 1, 10, 4, 13, 8, 2, 11, 5, 3, 6⟩
6. ⟨8, 11, 9, 3, 12, 6, 1, 10, 4, 13, 7, 2, 5⟩
7. ⟨8, 11, 9, 3, 12, 6, 7, 1, 10, 4, 13, 2, 5⟩
8. ⟨8, 9, 3, 12, 6, 7, 1, 10, 4, 13, 2, 11, 5⟩
9. ⟨9, 3, 12, 7, 1, 10, 4, 13, 8, 2, 11, 5, 6⟩
10. ⟨9, 3, 12, 1, 10, 4, 13, 7, 8, 2, 11, 5, 6⟩

These optimal solutions can be easily obtained with the *pool mode* of a commercial software for MILPs such as Gurobi or CPLEX. However, we have

empirically found that this pool mode has important limitations in practice, because in most of the cases a complete enumeration of all the optimal solutions is not possible due to its very large cardinality.

For example, in the LOLIB set of instances, we can find the Input-Output subset, in which the instances range from $n = 44$ to 56. In spite of the medium size of these instances, we have found that the size of the set of optimal solutions is very large (ranging in most of the cases from 1,000 to 500,000 solutions). This seems to indicate that in a way or another we have to simplify this information to be practical to the user.

It is clear in the optimal solutions of the example above that elements 8 and 9 consistently occupy the top positions. In contrast, other elements, such as 3 and 6, vary significantly in their rankings depending on the specific optimal solution considered. For instance, in Solution 1, element 3 is ranked third, while in Solution 4, it drops to twelfth place. Similarly, element 6 is ranked fifth in Solution 2 but often appears in the last place in many other optimal solutions. Given that most methods proposed for the Linear Ordering Problem (LOP) typically identify only one optimal solution—or a good one in the case of heuristics—it is important to moderate our conclusions when interpreting the individual rankings of elements. This is particularly true in applications where we seek to establish a ranking based on pairwise comparisons, such as in sports tournaments or lists of top universities. Ranking in sports is usually a controversial topic, as can be read in the sports section of well-reputed journals, such as the New York Times where only in this month we can read about it in the context of the Challenger Tour finished on the ATP circuit or the college football Playoff rankings. In these contexts, the standard LOP model alone may not be suitable, and additional factors should be considered to arrive at the most representative ranking.

Mathematical Models

When examining the representativeness of an optimal ordering from a Linear Order Problem (LOP) instance, it is important to analyze the distance between different optimal solutions. If we can identify two optimal solutions where their elements are positioned very differently from each other (indicating a large distance), then neither solution can be considered representative. Conversely, if all optimal solutions are quite similar (suggesting they are close to one another), any of these solutions can be seen as a good representative.

In this context, studies by [3] and [1] have proposed using the Kendall-Tau rank distance, developed by Maurice Kendall in 1938, to measure the number of discordant pairs between two rankings. The goal is to identify the two optimal solutions at the largest Kendall-Tau distance. The authors outlined a two-step method: First, they solve the canonical LOP to find the optimal value c^*; second, they solve a mathematical model aimed at maximizing the distance between all pairs of optimal LOP solutions, thus returning the two most distant optimal solutions. We found this approach very intuitive and informative. However, this method only gives information about the diameter of the optimal solution space, but not about its

geometry. In other words, these solutions may not be representative enough, as there may be alternative solutions that are quite different from the two already considered. Let us clarify this with the previously mentioned example.

We have examined 10 optimal solutions for the N-pal13 instance. The two solutions at the maximum distance according to the model in [3] are:

⟨9, 12, 1, 10, 4, 13, 7, 8, 2, 11, 5, 3, 6⟩
⟨8, 11, 9, 3, 12, 6, 7, 1, 10, 4, 13, 2, 5⟩

These two extreme solutions provide some insights into the ranking of elements across the optimal solutions. For example, element 7 is always ranked 7th, while element 8 is ranked 8th in the first solution and 1st in the second. We can represent the range of positions for each element as [min, max]. So for element 7, the range would be [7,7], and for element 8 the range would be [1,8]. However, this information from the two most distant solutions does not provide an accurate picture on the set of optimal solutions, and it may even be misleading. The following optimal solutions show that element 7 can actually be ranked 3rd, 7th, or 10th. So the ranges derived from the two extreme solutions do not accurately capture the full variation in element positions across all the optimal solutions. A more comprehensive analysis is needed to summarize how the elements are ranked across all the optimal solutions:

⟨8, 9, 3, 12, 6, 1, 10, 4, 13, 7, 2, 11, 5⟩
⟨8, 11, 9, 3, 12, 6, 7, 1, 10, 4, 13, 2, 5⟩
⟨9, 12, 7, 1, 10, 4, 13, 8, 2, 11, 5, 3, 6⟩

Using a similar approach, the following model from [11] generalizes the model to compute not just the two most distant solutions, but the m most distant solutions, to obtain a better representation of the entire set of optimal solutions.

$$
\begin{aligned}
\min \quad & \sum_{\substack{t,t'=1 \\ t<t'}}^{n} \sum_{\substack{i,j=1 \\ i\neq j}}^{n} z_{ij}^{(t,t')} \\
\text{s.t.} \quad & x_{ij}^t + x_{ji}^t = 1, && \forall i < j : i, j \in \{1,\dots,n\}, t \in \{1,\dots,m\} \\
& x_{ij}^t + x_{jk}^t + x_{ki}^t \leq 2, && \forall i < j, i < k, j \neq k : i, j, k \in \{1,\dots,n\}, t \in \{1,\dots,m\} \\
& \sum_{\substack{i,j=1 \\ i\neq j}}^{n} c_{ij} x_{ij}^t = c^* && t \in \{1,\dots,m\} \\
& x_{ij}^t + x_{ij}^{t'} - z_{ij}^{(t,t')} \leq 1, && \forall i \neq j : i, j \in \{1,\dots,n\} \forall t \neq t' : t < t', t, t' \in \{1,\dots,m\} \\
& x_{ij}^t, z_{ij}^{(t,t')} \in \{0,1\}, && \forall i \neq j : i, j \in \{1,\dots,n\}, \forall t \neq t' : t < t', t, t' \in \{1,\dots,m\}.
\end{aligned} \tag{2}
$$

We define an $n \times n$ dimensional binary variable x^t for each optimal solution, $t \in \{1,\dots,m\}$. As in the previous formulation, x_{ij}^t takes the value 1 if element

i precedes j in the t optimal solution. For each pair of optimal solutions x^t and $x^{t'}$, the variable $z_{ij}^{(t,t')}$ takes the value 1 if elements i and j are in the same relative positions in solutions x^t and $x^{t'}$. This is due to the constraint $x_{ij}^t + x_{ij}^{t'} - z_{ij}^{(t,t')} \leq 1$. To obtain the m-farthest optimal solutions, the objective function minimizes the sum of $z^{(t,t')}$-variables.

It is evident that the model described in equation (2) can have a very large number of variables. This is because, for every pair of optimal solutions we aim to identify, it introduces an $n \times n$ dimensional binary variable $z^{(t,t')}$. However, from a practical perspective, we will focus on relatively small values for the number of optimal distant solutions m.

Based on the results on the Maximum Diversity Problem (MDP)[13], instead of maximizing the overall sum of the distance among m optimal solutions, we can maximize the minimum distance among all the pairs of m solutions. According to previous work in the MDP, when the goal is to maximize the diversity among m selected points from a larger set, the model that focuses on maximizing the minimum distance between points, called MaxMin, generates solutions with a very different structure compared to the model that maximizes the sum of distances, MaxSum. Specifically, the MaxMin model produces equidistant points distributed throughout the space, and it does not avoid selecting points in central regions like the MaxSum model does. From [11] we propose a second model called m-farthest-min that aims to find the m most distant solutions by maximizing the minimum distance among the optimal solutions. We refer to the model in equation (2) as the m-farthest-sum model and that in equation (3) as the m-farthest-min model. It is worth noting that a potential downside of the sum of distances-based model (m-farthest-sum) is that it may generate the same optimal solution multiple times when maximizing the overall sum of distances. In contrast, the minimum distance-based model (m-farthest-min) does not have this issue, because repeating a solution would result in a minimum distance of 0, which is not desirable.

$$
\begin{aligned}
\min \quad & z \\
\text{s.t.} \quad & x_{ij}^t + x_{ji}^t = 1, && \forall i < j : i, j \in \{1, \ldots, n\}, t \in \{1, \ldots, m\} \\
& x_{ij}^t + x_{jk}^t + x_{ki}^t \leq 2, && \forall i < j, i < k, j \neq c : i, j, k \in \{1, \ldots, n\}, t \in \{1, \ldots, m\} \\
& \sum_{\substack{i,j=1 \\ i \neq j}}^{n} c_{ij} x_{ij}^t = c^* && t \in \{1, \ldots, m\} \\
& x_{ij}^t + x_{ij}^{t'} - z_{ij}^{(t,t')} \leq 1, && \forall i \neq j : i, j \in \{1, \ldots, n\} \forall t \neq t' : t < t', t, t' \in \{1, \ldots, m\}, \\
& z \geq \sum_{\substack{i,j=1 \\ i \neq j}}^{n} z_{ij}^{(t,t')} && \forall t \neq t' : t < t', t, t' \in \{1, \ldots, m\} \\
& x_{ij}^t, z_{ij}^{(t,t')} \in \{0, 1\}, && \forall i \neq j : i, j \in \{1, \ldots, n\}, \forall t \leq t' : t < t', t, t' \in \{1, \ldots, m\}.
\end{aligned} \tag{3}
$$

It is important to note that these models are designed to calculate the variable z_{ij}, which represents the count of pairs (i, j) that maintain the same relative order in different optimal solutions (for instance, t and t'). Consequently, to enhance the separation between solutions, they aim to reduce the total number of pairs of elements that are in the same relative order in these solutions. Specifically, the m-`farthest-sum` model focuses on minimizing the following expression within the objective function of (2):

$$\sum_{\substack{i,j=1 \\ i \neq j}}^{n} z_{ij}^{(t,t')}.$$

This expression counts the number of pairs (i, j) in the same relative position in solutions t and t'. The objective function adds this expression for all pairs of solutions, thus obtaining:

$$\sum_{t,t'=1 t<t'}^{n} \sum_{\substack{i,j=1 \\ i \neq j}}^{n} z_{ij}^{(t,t')}.$$

To minimize this expression is equivalent to maximize the Kendall-Tau distance between solutions.

The m-`farthest-min` model is shown in (3), where we can see that the right-hand side in the new constraint:

$$z \geq \sum_{\substack{i,j=1 \\ i \neq j}}^{n} z_{ij}^{(t,t')}$$

computes, as in (2), the number of pairs of elements in the same relative ordering in solutions t and t'. With the minimization of the z variable in the objective function, it takes in this constraint the value of the largest number of elements with the same ordering in a pair of solutions (t, t'). Note that this pair corresponds to the two solutions at minimum distance (since they are the two with more pairs in common).

Algorithmic Approach

The models described exhibit exponential growth, making them unsuitable for large instances (and even for medium-sized instances). Therefore, based on the same two-step methodology, we present a VNS heuristic from [11] to obtain approximate solutions in short computational times. This approach involves solving the Linear Ordering Problem (LOP) in the first step to achieve the best possible value. In the

second step, we identify the m-farthest solutions based on this objective function value. It is important to note that since the heuristic does not guarantee the optimality of the objective value in the first step, the second step may explore suboptimal regions of the solution space, and these suboptimal regions may not correlate with the optimal one.

We propose a two-step algorithm based on the Variable Neighborhood Search (VNS) methodology. In Algorithm 1, we show the pseudo-code of our complete approach. The algorithm takes five inputs: the number of m alternative solutions that we want to obtain, the largest neighborhood sizes k^1_{max} and k^2_{max}, the maximum number of iterations it_{max}, and the maximum CPU time τ_{max}. As in the mathematical models proposed above, the heuristic algorithm has basically two phases. In the first one, the algorithm finds the best solution (i.e., its approximation to the optimal LOP solution) with the BVNS method, numbered as step 2 in the pseudo-code. In the second phase (numbered as step 3), it finds the subset of the m-farthest solutions corresponding to the value identified in the first phase, which will ultimately determine the elements' ranking (computed in step 4). We denote as Δ^S the set of rankings of the elements in the m alternative solutions obtained.

As outlined in the mathematical models presented earlier, the heuristic algorithm consists of two main phases. In the first phase, the algorithm uses the BVNS method to identify the best solution, which serves as an approximation of the optimal LOP solution (this step is numbered as 2 in the pseudo-code). In the second phase (numbered as 3), the algorithm identifies the subset of m solutions that are farthest. This subset ultimately determines the ranking of the elements, as computed in step 4. Let us explain how this algorithm works.

Algorithm 1: `Two-step VNS`$(m, k^1_{max}, it_{max}, k^2_{max}, \tau_{max})$

1: $p \leftarrow random_solution()$
2: $p \leftarrow$`BVNS`(p, k^1_{max}, it_{max}) see Algorithm 2
3: $S \leftarrow$ `VNS`$(p, m, k^2_{max}, \tau_{max})$ see Algorithm 3
4: $\Delta^S \leftarrow elements_range(S)$
5: **return** Δ^S

Algorithm 2: `BVNS`(p, k^1_{max}, it_{max})

1: **while** $it < it_{max}$ **do**
2: $k \leftarrow 1$
3: **while** $k \leq k^1_{max}$ **do**
4: $p' \leftarrow Shake(p, k)$
5: $p'' \leftarrow LocalSearch(p')$
6: $k \leftarrow NeighborhoodChange(p, p'', k)$
7: **end while**
8: **end while**
9: **return** p

Algorithm 3: VNS $(p, m, k^2_{max}, \tau_{max})$

```
1: c ← f(p)
2: S ← {p}
3: while τ < τ_max do
4:     k ← 1
5:     q ← select_random(S)
6:     while k < k²_max do
7:         p' ← Shake(k, q)
8:         p'' ← LocalSearch(p')
9:         if f(p'') == c then
10:            S' ← Insert(S, p'', m)
11:            if K_d(S') > K_d(S) then
12:                S ← S'
13:                k ← 1
14:            else
15:                k ← k + 1
16:            end if
17:        end if
18:    end while
19: end while
20: return S
```

VNS was introduced in [14] and, since then, has gained significant relevance among metaheuristics. Several variants and extensions have been proposed, as detailed in [7]. Starting from an initial solution, the rationale behind VNS is to exploit the notion of changing neighborhoods, both in reaching the local optimum and in escaping from the valleys that contain them. This chapter considers the neighborhoods based on insertion moves proposed in [5] for the LOP. Given a solution (permutation) $p = (p_1, \ldots, p_n)$, where p_i represents the element in position i in the solution p, we define as $\texttt{insert}(p_j, i)$ dropping element p_j from solution p and adding it in position i. Then, the neighborhood $N_1(p)$ consists of the set of solutions obtained by the moves $\texttt{insert}(p_j, j-1)$ and $\texttt{insert}(p_j, j+1)$, which essentially represents the set of solutions obtained by switching one element either directly one position before or after their current position. The neighborhood $N_2(p)$ is defined as the solutions obtained by inserting any element p_j from p into a different position. In general, we define $N_k(p)$ as the set of solutions reached by applying N_2 recursively to solution $p, k-1$ times, which is equivalent to performing $k-1$ moves in p.

In the **first step** of the algorithm, we employ a Basic VNS (BVNS) method to solve the LOP, to obtain the objective function value of a high-quality solution that eventually can be the optimal solution. BVNS balances diversification and intensification by integrating deterministic and random changes within its neighborhood structures. Algorithm 2 shows the pseudo-code of the BVNS. The BVNS algorithm takes three inputs: a randomly generated initial solution, the largest neighborhood size k^1_{max}, and the maximum number of iterations it_{max}. At each iteration it, the algorithm first sets the parameter k to 1 (step 2). The main loop then iterates through

neighborhoods (steps 3–7). First, a solution p' is randomly generated from the kth neighborhood of solution p (step 4). A local search is applied to the obtained solution p' to find a local optimum p'' (step 5). Specifically, the local search iterates through each element (p'_j) in p', inserting it into the position that yields the best objective function value. If the improved solution p'' outperforms the current solution p, it becomes the new incumbent solution $p \leftarrow p''$, and the neighborhood exploration restarts at the smallest size $k \leftarrow 1$ (step 6). Otherwise, the algorithm explores a larger neighborhood ($k = k + 1$) until reaching the largest one.

As mentioned above, the **second step** of the algorithm aims to find the m-farthest alternative solutions according to the *Kendall-Tau distance*, which quantifies the dissimilarity between two permutations. Given two solutions (permutations) p and p' of length n, the Kendall-Tau distance, denoted as $K_d(p, p')$, is defined as the number of pairs (i, j) such that $1 \leq i < j \leq n$ and the orders of p and p' differ on these pairs. Formally,

$$K_d(p, p') = \left| \{(i, j) \mid 1 \leq i < j \leq n,\ (p_i < p_j \wedge p'_i > p'_j \text{ or } (p_i > p_j \wedge p'_i < p'_j \} \right|,$$

where p_i denotes the position of the element i in the permutation p. Intuitively, the Kendall-Tau distance counts the number of inversions needed to convert one permutation into another. An *inversion* is a pair of elements in the opposite order in the two permutations. Given a subset of m solutions $S = \{p^1, \ldots, p^m\}$, the Kendall-Tau distance of the set S, $K_d(S)$, is computed as the sum of the Kendall-Tau distances of each pair of solutions in S. In the VNS algorithm, a subset S' is better than a subset S if it has a larger Kendall-Tau distance, i.e., $K_d(S') > K_d(S)$.

Algorithm 3 outlines the pseudo-code for computing the m-farthest solutions. It takes the following inputs: the best solution found in the `BVNS`, p; the number m of desired farthest alternative solutions; the maximum size of the neighborhood k^2_{max}; and the maximum CPU time τ_{max}.

The algorithm first initializes the objective function value of the best solution found p (step 1) and the set S of alternative high-quality solutions (step 2). It then enters in a loop that continues until the time limit τ_{max} is reached. Inside the loop, a solution q is randomly selected from the set S (step 5). The main loop then iterates based on a basic variable neighborhood search scheme (steps 6–18). The shake procedure is applied to randomly select a solution p' in the neighborhood $N_k(q)$ (step 7). Then, a local search is performed on p' to find a local optimum within its N_2 neighborhood (step 8).

If the resulting solution of the local search p'' is a new high-quality solution (i.e., not already in S), the new set S' is constructed. If $|S| < m$, S' simply includes S and the new local optimum p'' (if it is a different solution). If $|S| = m$, S' is S by exchanging solution p'' with the solution that maximizes the Kendall-Tau distance gain (step 10). Finally, if the new set S' improves the overall Kendall distance compared to S, then S is updated with S', and the neighborhood size is reset to 1 (step 13). Otherwise, the neighborhood size is incremented by 1 (step 15).

Practical Cases with Multiple Optima

College Rankings and Fairness

When we use the Linear Ordering Problem to create a ranking, we take on a responsibility toward both the elements we are ranking and the users who will rely on our ranking. This is why it is important to incorporate the concept of fairness into the process. Biega et al. [2] present an example of fairness in the context of Airbnb room rankings. When an Airbnb user makes a search, an ordered list of rooms is displayed, and typically, the rooms ranked higher receive much more attention than those shown at the bottom of the list. Therefore, if we consistently show the same ranking of rooms for the same user query, we are being systematically unfair to those ranked last. The authors suggested considering a range of rankings and varying the displayed ranking over time to ensure that the attention given to rooms is fair in the aggregate. Biega et al. [2] define the unfairness of a set of m rankings of n elements as

$$unfairness(p^1, \ldots, p^m) = \sum_{i=0}^{n} \Big| \sum_{j=1}^{m} A_{ij} - \sum_{j=1}^{m} R_{ij} \Big|, \tag{4}$$

where $\sum_{j=1}^{m} A_{ij}$ is the total aggregated attention received by the element ith and $\sum_{j=1}^{m} R_{ij}$ is the relevance of the ith element.

We will use the US News and World Report college data used in [3] as a case study to examine how the models proposed above can help reduce unfairness when processing repeated queries for college rankings. Every year, U.S. News and World Report ranks colleges based on various quantitative factors, such as student-faculty ratio and acceptance rate. We can use these factors to create a dominance matrix and apply the Linear Ordering Problem (LOP) to rank the colleges. Although we are unsure of the exact algorithm used by U.S. News and World Report, LOP is a reliable option for this purpose. For simplicity, we will focus on a subset of 11 liberal arts colleges and construct a dominance matrix 11×11 where the entry (i, j) indicates in how many factors college i performs better than college j. Since one can choose which features to consider when creating the dominance matrix, the matrix we will use only reflects a possible query made by an individual.

Using the LOP mathematical model and the pool mode of the Gurobi solver, we obtain that our input dominance matrix has 72 optimal rankings. In our case, the true relevance of an element i will be the average position of element i across the optimal solution set, $\sum_{j=0}^{72} \frac{r_{ij}}{72}$, and the attention received by the element i will be the average position of the element i across the set of m solutions displayed. The situation is as follows: An individual requests a ranking of colleges based on attributes he considers important, and if the query is repeated by another individual, we must decide what information to present to them. It has become apparent that consistently showing the same ranking is unfair to colleges that may rank higher in other optimal rankings according to the same criteria. To address this issue, we propose four systems for responding to repeated queries:

- Cyclically return the two most distant solutions according to the m-farthest-sum model.
- Cyclically return the four most distant solutions according to the m-farthest-sum model.
- Cyclically return the four most distant solutions according to the m-farthest-min model.
- Cyclically return the four most distant solutions according to the m-farthest-sum model and the four most distant solutions according to the m-farthest-min model.

After all, unfairness measures how well a set of solutions represents the distributions of each element's positions across the optimal solution set. In our case, we can see in Fig. 1 that the best model by itself is the MaxMin model. However, the MaxSum produces solutions different to the MaxMin model, and that is why when we combine them, we get the most representative solutions.

In this case study, we have seen a method to reduce the unfairness in the aggregate toward the colleges when processing repeated queries, which we can extrapolate to every methodology that produces a ranking as an output for an individual request. It is worth noting that we are using a small instance, and the solvers are capable enough to compute the whole optimal set. However, larger instances may require a heuristic approach.

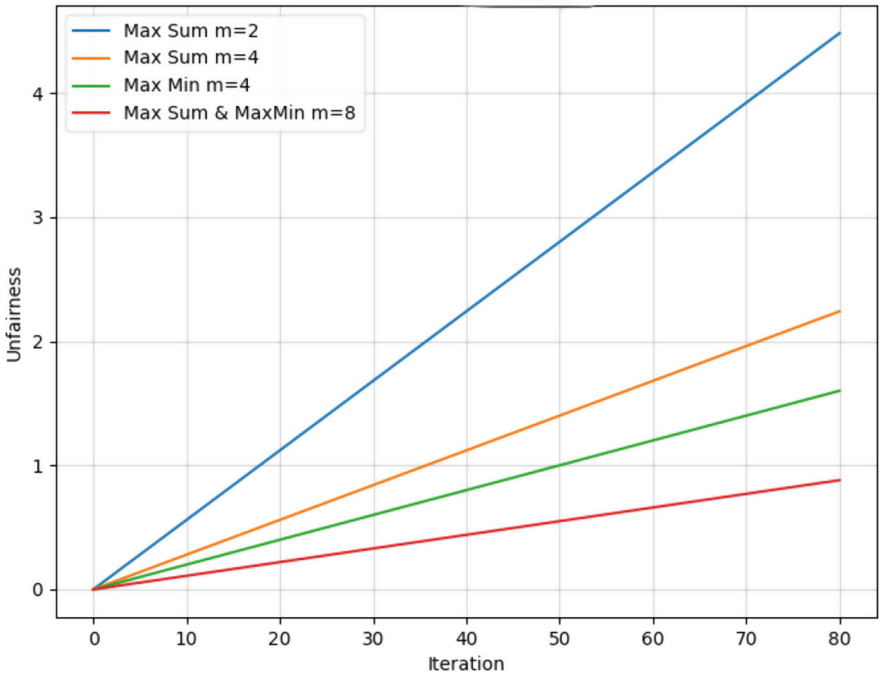


Fig. 1 Unfairness of different systems when used for repeated queries

In the next case study we will see how to sum up the information of the whole optimal solution set in a valuable way for the user.

Premier League

We are going to use the LOP to rank the teams of the Premier League on the 2017–2018 season. The standard method for ranking soccer teams based on league results is to award each team three points for a win and one point for a draw and then arrange them according to their total points. This approach has three main drawbacks. First, it fails to consider the margin of victory, which may indicate the team's strength. Second, it lacks nuance since it does not differentiate between victories over strong and weak teams; winning against a top team is valued the same as beating the last-place team. Third, a team that frequently draws may accumulate more points than a team with a combination of wins and losses, even if the latter has demonstrated the capability to defeat opponents. These factors justify the use of the LOP model to create a more accurate ranking of the teams.

First of all, we need a dominance matrix in order to classify the teams. Since 20 teams played that season, we create a 20×20 matrix where the entry (i, j) is set as the number of goals that team i scored against team j in the entire season. For example, if we associate Liverpool with the first row and column, and Manchester United with the second one, and they faced each other twice resulting in scores of 1–2 and 2–1, respectively, then the entry (1, 2) will be 3, and the entry (2, 1) will be 3 as well.

The traditional approach offers users a single optimal ranking. However, this can be problematic, as there may be multiple optimal solutions available, and presenting every optimal solution does not seem particularly useful. It is important to note that the difficulty of the multiple optima problem can vary based on how much the positions of each element differ within the optimal set. Therefore, it would be beneficial to provide users with the ranges of variation for each element. To accurately determine these ranges, we need to compute the entire optimal set, which can be challenging even for medium-sized instances. However, as noted in [11], we can approximate these ranges with high precision using the previous models or by employing the VNS algorithm to compute 3 or 4 distant solutions. It is worth mentioning that obtaining four or more solutions with the mathematical models can be challenging, even in cases with 30 or 40 elements, which makes it necessary to have an heuristic approach.

Figure 2 illustrates that the positions of most teams vary significantly, which supports the idea that analyzing a single optimal solution can be misleading. By comparing the ranges of optimal positions of each team with the official rankings, we can extract insightful information. For instance, Manchester City clearly emerged as the most dominant team. However, the official ranking method did not adequately represent the overall strengths of teams like Tottenham and Stoke City, which clearly demonstrate a better dominance than what the official ranking

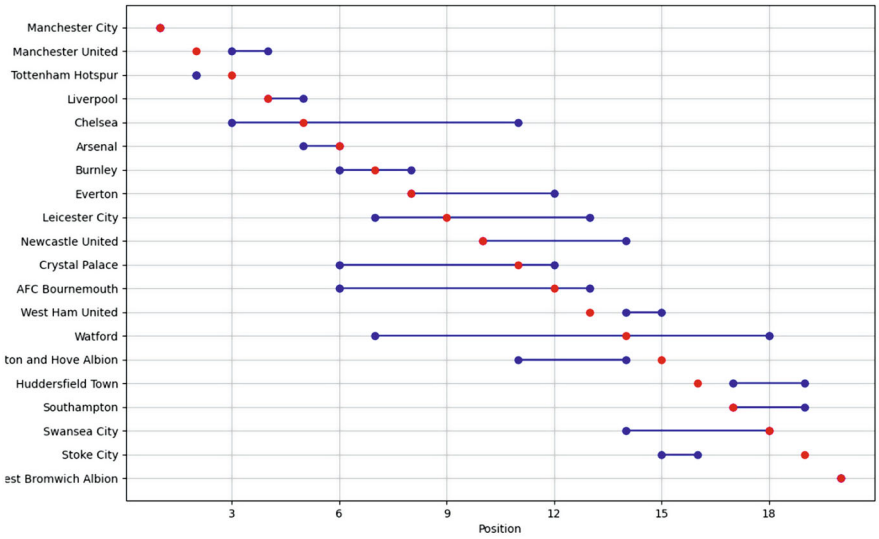


Fig. 2 Ranges of teams positions in LOP across the optimal solution set versus official classification (in red points)

shows. Additionally, some teams, such as Watford, show considerable variability in their positions, suggesting that they did not demonstrate decisively dominance throughout the season.

Conclusion

This chapter illustrates the importance of examining the full set of optimal solutions rather than concentrating on just one of them, as shown on the Linear Ordering Problem. This comprehensive approach provides valuable insights into the structure of solution space, enhances decision-making robustness, and helps reduce potential biases in interpreting results.

The practical implications of considering multiple optimal solutions go beyond theoretical optimization and find it realm into real-world challenges. The case studies demonstrate that understanding the variability in rankings or outcomes fosters fairness and equity while offering a more nuanced view of the data. Effectively summarizing the entire solution space not only enriches the interpretability of the results but also provides actionable insights to the policy-makers.

Given the usual large size of the optimal solution spaces, this work emphasizes the need for methodologies that not only identify these solutions but also represent them effectively. Techniques such as m-farthest models and heuristic-based approaches like Variable Neighborhood Search (VNS) serve as valuable tools for

navigating the complexities of the solution space while ensuring computational feasibility. It is worth noting that the two-phase concept used in the VNS algorithm should be easily extensible to other heuristics such as GRASP or Tabu Search. In particular, the identification of the optimal value first, and the generation of diverse alternate optima, is a simple and effective scheme that can be easily integrated in different metaheuristics. We hope that this initial approach triggers the interest of researchers in the practical topic of identifying multiple optimal solutions, and explore this area in the frontier of the so-called multi-objective optimization.

In conclusion, embracing the concept of alternate optima in combinatorial optimization represents more than just a change in analytical perspective; it is a crucial step toward achieving more robust, fair, and insightful solutions across various application domains. Future research should focus on refining these methods with an emphasis on scalability and practicality to maintain their relevance and applicability to increasingly complex decision-making problems.

Cross-References

▸ Diversity Optimization Models
▸ Variable Neighborhood Search

Acknowledgments This research has been partially supported by the Ministerio de Ciencia e Innovación of Spain (Grant Refs. PID2024-160226OB-C21 and PID2021-125709OB-C21) funded by MCIN/ AEI/ 10.13039/ 501100011033/FEDER, UE, including the program PRE2022-103687. It has also been supported by the Generalitat Valenciana (CIAICO 2021/224).

Competing Interest Declaration The author(s) has no competing interests to declare that are relevant to the content of this manuscript.

References

1. Anderson PE, Chartier TP, Langville AN, Pedings-Behling KE (2022) Fairness and the set of optimal rankings for the linear ordering problem. Optim Eng 23:1289–1317. https://doi.org/10.1007/s11081-021-09650-y
2. Biega AJ, Gummadi KP, Weikum G (2018) Equity of attention: amortizing individual fairness in rankings. In: The 41st international ACM SIGIR conference on research & development in information retrieval. Association for Computing Machinery, pp 405–414
3. Cameron TR, Charmot S, Pulaj J (2021) On the linear ordering problem and the rankability of data. Found Data Sci 3:133–149. https://arxiv.org/abs/2104.05816v1, https://doi.org/10.3934/fods.2021010
4. Danna E, Woodruff DL (2009) How to select a small set of diverse solutions to mixed integer programming problems. Oper Res Lett 37:255–260
5. Garcia CG, Pérez-Brito D, Campos V, Martí R (2006) Variable neighborhood search for the linear ordering problem. Comput Oper Res 33:3549–3565
6. Gounaris CE, Wiesemann W, Floudas CA (2013) The robust capacitated vehicle routing problem under demand uncertainty. Oper Res 61:677–693. https://doi.org/10.1287/opre.1120.1136

7. Hansen P, Mladenović N, Todosijević R, Hanafi S (2017) Variable neighborhood search: basics and variants. EURO J Comput Optim 5:423–454
8. Hillis DM, Heath TA, John KS (2005) Analysis and visualization of tree space. Syst Biol 54:471–482. https://doi.org/10.1080/10635150590946961
9. Kemeny JG (1959) Mathematics without numbers. Daedalus 88:577–591
10. Leontief W (1986) Input-output economics. Oxford University Press, New York
11. Benito-Marimon M, Rafael Marti AMG (2024) Multiple optimal experiments. Technical Report, Universidad de Valencia
12. Martí R, Duarte A, Reinelt G (2010) LOLIB—linear ordering problem library. https://www.uv.es/rmarti/paper/lop.html. [Online; accessed 29 Apr 2024]
13. Martí R, Reinelt G (2022) Exact and heuristic methods in combinatorial optimization. A study on the linear ordering and the maximum diversity problem, vol 175. Springer, Heilderberg
14. Mladenović N, Hansen P (1997) Variable neighborhood search. Comput Oper Res 24:1097–1100
15. Mulvey J, Vanderbei R, Zenios S (1995) Robust optimization of large-scale systems. Oper Res 43:264–281. https://doi.org/10.1287/opre.43.2.264
16. Rohlf F (2005) J. Felsenstein, Inferring phylogenies, Sinauer Association, 2004, pp xx + 664. J Classif 22:139–142. https://doi.org/10.1007/s00357-005-0009-4
17. Younis M, Akkaya K (2008) Strategies and techniques for node placement in wireless sensor networks: a survey. Ad Hoc Netw 6:621–655

Heuristics in Design of Deep Neural Networks

35

Ricardo Martins de Abreu Silva and Andersson Alves da Silva

Contents

Introduction and Motivation 1032
State of the Art 1034
Heuristic Principles for the Design of Artificial Neural Networks 1037
Metaheuristic Design of Feedforward Neural Networks: A Review of Two Decades of Research 1038
Optimization of Deep Neural Networks: A Survey and Unified Taxonomy 1039
Swarm and Evolutionary Computing Approaches for Deep Learning 1040
A Technical View on Neural Architecture Search 1040
On the Automated, Evolutionary Design of Neural Networks: Past, Present, and Future 1041
Neuroevolution in Deep Neural Networks: Current Trends and Future Challenges 1042
A Comprehensive Survey of Neural Architecture Search: Challenges and Solutions 1042
A Survey on Evolutionary Construction of Deep Neural Networks 1043
Evolutionary Neural Networks for Deep Learning: A Review 1044
A Comprehensive Survey on Optimizing Deep Learning Models by Metaheuristics 1045
Evolutionary Design of Neural Network Architectures: A Review of Three Decades of Research 1046
Survey on Evolutionary Deep Learning: Principles, Algorithms, Applications, and Open Issues 1047
A Survey: Evolutionary Deep Learning 1047
A Survey on Evolutionary Neural Architecture Search 1048
Biased Random Key Genetic Algorithm 1049
BRKGA in Deep Learning Design 1051
BRKGA-DFNN Proposal 1051
BRKGA-CNN Proposal 1057
BRKGA-GNN Proposal 1064
Experiments: Datasets, Models, and Configurations of Models 1071
BRKGA-DFNN 1071

R. M. de Abreu Silva (✉) · A. A. da Silva
Federal University of Pernambuco, Recife, PE, Brazil
e-mail: rmas@cin.ufpe.br; aas9@cin.ufpe.br

© Springer Nature Switzerland AG 2025

R. Martí et al. (eds.), *Handbook of Heuristics*,
https://doi.org/10.1007/978-3-032-00385-0_74

BRKGA-CNN 1072
BRKGA-GNN 1074
Results and Discussion 1075
BRKGA-DFNN 1075
BRKGA-CNN 1076
BRKGA-GNN 1077
Conclusion and Future Research Directions 1077
Final Remarks 1079
Cross-References 1080
References 1080

Abstract

The complexity of Deep Neural Networks (DNNs) has driven advancements in Neural Architecture Search (NAS), Hyperparameter Optimization (HPO), and Learning Rule Optimization (LRO). This study reviews heuristic methodologies, focusing on Evolutionary Algorithms (EAs) and Swarm Intelligence (SI). We analyze Genetic Algorithms (GAs), Particle Swarm Optimization (PSO), and Multi-Objective Optimization (MOO), emphasizing the Biased Random Key Genetic Algorithm (BRKGA). BRKGA encodes neural architectures and hyperparameters as continuous vectors, enhancing search efficiency in NAS and HPO. We evaluate BRKGA on Feedforward Neural Networks (FNNs), Convolutional Neural Networks (CNNs), and Graph Neural Networks (GNNs), demonstrating their effectiveness in tuning learning rates, dropout rates, and batch sizes. Additionally, we explore its role in LRO, optimizing adaptive weight updates and gradient modulation. Experiments on benchmark datasets show that BRKGA consistently yields promising architectures and hyperparameter configurations, balancing accuracy, efficiency, and adaptability. Our findings highlight BRKGA as a viable alternative for NAS, HPO, and LRO, particularly in complex search spaces where structured exploration is essential. Finally, challenges in heuristic-driven NAS, HPO, and AutoML are examined, along with future research directions in scalable optimization, adaptive learning mechanisms, and neuromorphic computing.

Keywords

Biased random key genetic algorithm · Deep feedforward neural networks · Convolutional neural networks · Graph neural networks · Neural architecture search · Hyperparameter optimization · Learning rule optimization

Introduction and Motivation

Deep Neural Networks (DNNs) have revolutionized artificial intelligence, driving advances in fields such as computer vision, natural language processing, and bioinformatics. However, their increasing architectural complexity and vast

hyperparameter search spaces pose significant challenges in network design, training strategy selection, and optimization. Traditional approaches rely heavily on gradient-based methods such as Stochastic Gradient Descent (SGD) and Adam, which, despite their effectiveness, suffer from issues such as vanishing gradients, local minima stagnation, and sensitivity to initialization [34, 43]. Consequently, there is a growing demand for automated methodologies capable of discovering optimal DNN configurations with minimal human intervention.

Neural Architecture Search (NAS) has emerged as a powerful paradigm for automating the design of deep networks, leveraging search strategies that balance exploration and exploitation. Among these, evolutionary algorithms (EAs) and swarm intelligence (SI) techniques have gained prominence, offering robust solutions for navigating complex, high-dimensional search landscapes [57, 58]. Within this family of metaheuristics, the Biased Random Key Genetic Algorithm (BRKGA) presents a particularly promising approach, introducing a structured bias in genetic inheritance that accelerates convergence while preserving population diversity [25].

While NAS focuses on optimizing network topology and connectivity, Hyperparameter Optimization (HPO) aims to refine key training parameters, such as learning rates, batch sizes, and weight decay coefficients, which significantly impact model convergence and generalization [7]. Similarly, Learning Rule Optimization (LRO) seeks to adapt the learning dynamics themselves, exploring alternative update mechanisms beyond conventional backpropagation [6]. HPO and LRO can be effectively tackled using heuristic-driven approaches, particularly when combined with NAS in an Automated Machine Learning (AutoML) framework.

This chapter provides a comprehensive review of heuristics in NAS, HPO, and LRO, with a particular emphasis on BRKGA as a novel and underexplored technique for optimizing deep learning architectures and hyperparameters. We investigate its application in three neural network paradigms: Deep Feedforward Neural Networks (DFNNs), Convolutional Neural Networks (CNNs), and Graph Neural Networks (GNNs). Unlike traditional NAS techniques, which often require expensive computational resources and large-scale datasets, our study focuses on moderate-sized benchmarks, aiming to illustrate the practical benefits and adaptability of BRKGA for neural network optimization rather than maximizing competitive performance. The experimental results indicate that BRKGA can efficiently discover well-performing architectures while maintaining a favorable balance between accuracy and computational cost.

Beyond the state-of-the-art review, this work contributes to the field by demonstrating how BRKGA can be tailored to different neural network paradigms, highlighting its ability to adapt to various architectural and hyperparameter constraints. Furthermore, the potential of BRKGA in advancing AutoML frameworks, enhancing model interpretability, and paving the way for neuromorphic computing applications is examined. The insights presented in this chapter aim to foster new directions in heuristic-driven NAS, HPO, and LRO, encouraging further research into scalability and integration with hybrid optimization frameworks.

The remainder of this chapter is structured as follows. First, we provide a comprehensive review of heuristic approaches in NAS, HPO, and LRO, analyzing

state-of-the-art methods and their advantages and inherent limitations. Next, we introduce the BRKGA framework, detailing its adaptation for optimizing DFNNs, CNNs, and GNNs. We then describe the experimental setup and results, demonstrating the effectiveness of BRKGA in NAS, HPO, and LRO through moderate-sized benchmark datasets, highlighting its ability to balance exploration and exploitation in architecture optimization. Subsequently, we conduct a critical analysis of the results, comparing BRKGA with existing NAS techniques and discussing its practical implications in deep learning model design. Finally, we present conclusions and outline future research directions, emphasizing the potential of BRKGA in AutoML, scalable NAS frameworks, and energy-efficient deep learning architectures, with particular attention to its applicability in resource-constrained environments and neuromorphic computing.

State of the Art

This section presents the main works in the modern literature that address the optimization of Deep Neural Network (DNN) architectures and the search for optimal DNN structures through the application of optimization models. It provides an overview of existing approaches, highlighting key methodologies, advancements, and challenges in the field. Tables 1 and 2 present the main works discussed in the state of the art and their respective references.

The field of heuristic-driven Neural Network (NN) design has witnessed significant advancements over the past decades, with numerous studies exploring evolutionary algorithms, metaheuristic optimization, and automated search techniques. This section provides a critical analysis of the 15 selected articles, identifying their contributions, limitations, interrelations, and potential impact on the future of Neural Architecture Search (NAS).

The surveyed papers can be broadly categorized into three interrelated research themes: (1) metaheuristic and evolutionary approaches to NN optimization [50, 55, 69, 71], (2) NAS methodologies, particularly those integrating evolutionary strategies [4, 33, 48, 59, 68, 82], and (3) hybrid and automated learning frameworks within AutoML [1, 13, 21, 45, 46].

Despite their distinct focus areas, these works exhibit substantial overlap. Studies on NAS consistently highlight the necessity of robust heuristic strategies to mitigate computational inefficiencies, demonstrating how evolutionary-based NAS [48, 59, 82] outperforms conventional search methods in terms of adaptability and exploration capability. Furthermore, works on hybridization strategies [1, 45] emphasize the synergistic effects of combining evolutionary search with gradient-based techniques, reinforcing the argument for automated, scalable NAS.

Several studies have contributed significant innovations to heuristic-based NN design. Early foundational work, such as [71], pioneered heuristic-driven optimization principles, laying the groundwork for more advanced NAS frameworks. Subsequent research introduced domain-specific improvements: for instance, [55]

Table 1 Summary of key papers in this research (Part 1)

#	Title	Summary
1	Heuristic Principles for the Design of Artificial Neural Networks [71]	This foundational work introduces a heuristic-driven framework for designing artificial neural networks (ANNs), emphasizing the role of expert knowledge in optimizing architecture selection, hyperparameter tuning, and training strategies. The study presents a structured approach to balancing model complexity and generalization, aiming to mitigate overfitting while ensuring computational efficiency
2	Metaheuristic Design of Feedforward Neural Networks: A Review of Two Decades of Research [55]	This survey critically evaluates two decades of research on metaheuristic approaches for optimizing feedforward neural networks (FNNs), contrasting them with traditional gradient-based methods. The review systematically categorizes algorithms based on their convergence properties, search efficiency, and adaptability to complex, high-dimensional optimization landscapes
3	Optimization of Deep Neural Networks: A Survey and Unified Taxonomy [68]	This paper presents a comprehensive taxonomy for deep neural network (DNN) optimization, unifying various approaches under a structured framework. The study explores architecture search, hyperparameter tuning, and weight optimization, with a focus on AutoML techniques and the integration of parallel computing strategies to enhance scalability
4	A Survey of Swarm and Evolutionary Computing Approaches for Deep Learning [13]	This survey investigates the synergy between swarm intelligence, evolutionary computing, and deep learning, detailing how bio-inspired algorithms enhance neural architecture search, hyperparameter selection, and training convergence. The review highlights the computational advantages of hybrid strategies and their applicability to large-scale learning problems
5	A Technical View on Neural Architecture Search [33]	This technical survey provides an in-depth classification of Neural Architecture Search (NAS) methodologies, analyzing reinforcement learning-based NAS, gradient-based approaches, and evolutionary strategies. The paper discusses search space constraints, efficiency-enhancing techniques, and emerging trends in automated neural network design
6	On the Automated, Evolutionary Design of Neural Networks: Past, Present, and Future [4]	This article reviews the historical evolution of neuroevolutionary techniques for designing artificial neural networks, emphasizing their role in optimizing topology, weight initialization, and learning dynamics. The study discusses the computational trade-offs of evolutionary approaches and explores hybrid models that combine neuroevolution with gradient-based learning
7	Neuroevolution in Deep Neural Networks: Current Trends and Future Challenges [21]	This paper examines state-of-the-art neuroevolutionary techniques for deep learning, analyzing their impact on reinforcement learning, recurrent neural networks, and adversarial robustness. The study identifies key challenges in high-dimensional search spaces and discusses the role of evolutionary algorithms in improving scalability and generalization
8	A Comprehensive Survey of Neural Architecture Search: Challenges and Solutions [59]	This survey presents a rigorous analysis of NAS methodologies, emphasizing challenges such as computational cost, search inefficiency, and performance evaluation. The study explores modular search spaces, differentiable architecture search (DARTS), and multi-objective optimization strategies for balancing accuracy, efficiency, and interpretability

Table 2 Summary of key papers in this research (Part 2)

#	Title	Summary
9	A Survey on Evolutionary Construction of Deep Neural Networks [82]	This study systematically reviews evolutionary strategies for deep neural network construction, framing NAS as a large-scale, multi-objective optimization problem. The paper classifies approaches based on direct and indirect encoding schemes, highlighting their applications in hardware-aware NAS, automated hyperparameter tuning, and model compression
10	Evolutionary Neural Networks for Deep Learning: A Review [50]	This review explores Evolutionary Neural Networks (ENNs), emphasizing their role in architecture search, adaptive learning rules, and metaheuristic-driven optimization. The study contrasts ENNs with conventional NAS techniques and discusses their applicability in edge computing and neuromorphic AI
11	A Comprehensive Survey on Optimizing Deep Learning Models by Metaheuristics [1]	This paper reviews metaheuristic-driven optimization techniques for deep learning, classifying strategies based on their exploration-exploitation trade-offs. The study discusses applications in NAS, hyperparameter tuning, and training acceleration, with a focus on hybrid models that integrate evolutionary search with reinforcement learning
12	Evolutionary Design of Neural Network Architectures: A Review of Three Decades of Research [69]	This historical review examines three decades of research in evolutionary neural network design, detailing key advancements in modular learning, hierarchical search, and coevolutionary strategies. The study highlights emerging trends in hardware-efficient deep learning and automated architecture refinement
13	Survey on Evolutionary Deep Learning: Principles, Algorithms, Applications, and Open Issues [46]	This survey explores the intersection of evolutionary algorithms and deep learning, categorizing optimization methodologies based on their adaptability to nonstationary environments. The study discusses coevolutionary techniques, self-adaptive NAS frameworks, and real-world applications in federated learning and autonomous systems
14	A Survey: Evolutionary Deep Learning [45]	This paper investigates the role of evolutionary deep learning (EDL) in the context of AutoML, discussing hybrid optimization techniques that integrate evolutionary search with gradient-based learning. The study highlights key challenges in search efficiency and model interpretability
15	A Survey on Evolutionary Neural Architecture Search [48]	This article reviews Evolutionary Neural Architecture Search (ENAS), providing a comparative analysis of evolutionary, reinforcement learning-based, and gradient-based NAS strategies. The study discusses key challenges in search scalability, architecture transferability, and hardware-aware optimization

systematically compared two decades of metaheuristic NN optimization, providing a robust classification scheme for search algorithms.

The application of evolutionary techniques to NAS has demonstrated notable performance improvements, particularly in handling large search spaces. The incorporation of neuroevolution principles [4, 21] has provided an efficient alternative to reinforcement learning-based NAS methods, which often suffer from high computational costs. Additionally, research on multi-objective NAS [82] has addressed

critical trade-offs between accuracy, efficiency, and model interpretability, further advancing the state of the art.

Despite these advancements, heuristic-driven NN design faces several persistent challenges. A key limitation of evolutionary NAS techniques [48, 59] is their high computational overhead, which remains an obstacle for large-scale deployment in real-world applications. Although surrogate modeling and weight-sharing techniques have been proposed to mitigate this issue [37, 57], they have yet to be fully integrated into mainstream NAS frameworks.

Another significant limitation is the lack of interpretability and theoretical guarantees in heuristic approaches. While studies such as [46] highlight the potential of evolutionary deep learning in improving model robustness, they also underscore the opacity of evolved architectures, making their adoption challenging in critical applications requiring explainability.

One notable research gap lies in the integration of neuromorphic computing with heuristic NAS. Although some studies acknowledge the importance of hardware-aware NAS [11, 34], few explicitly explore the potential of neuromorphic-inspired evolutionary models. Bridging this gap could lead to the development of energy-efficient neural networks tailored for edge AI applications.

Additionally, while hybridization strategies have been extensively discussed [1, 45], there is a need for more systematic benchmarking of hybrid NAS methods. Future research should focus on designing standardized evaluation protocols that compare hybrid, evolutionary, and reinforcement learning-based NAS under uniform conditions.

Looking ahead, heuristic-driven NN design is poised to play a pivotal role in the future of automated machine learning (AutoML) and scalable NAS. The increasing complexity of deep learning models necessitates more efficient search algorithms, and evolutionary strategies are well-positioned to address these challenges. By incorporating adaptive search heuristics, self-supervised learning paradigms, and hardware-aware optimizations, the next generation of heuristic NAS techniques is expected to achieve greater efficiency, adaptability, and interpretability.

Ultimately, the intersection of evolutionary computing, neuromorphic hardware, and AutoML frameworks will define the next frontier of NAS research. As the field progresses, addressing the existing computational bottlenecks and enhancing the theoretical rigor of heuristic optimization methods will be critical in realizing fully automated, high-performance deep learning architectures. The following section provides a more detailed explanation of each of the key works analyzed on the topic of this chapter.

Heuristic Principles for the Design of Artificial Neural Networks

The paper "Heuristic Principles for the Design of Artificial Neural Networks" by Walczak and Cerpa [71] provides a structured framework for heuristic-driven optimization of Artificial Neural Networks (ANNs), emphasizing the integration of domain expertise and algorithmic heuristics to enhance model efficiency, gener-

alization, and interpretability. The authors categorize heuristic methodologies into three critical aspects: input feature selection, learning method selection, and hidden layer configuration, demonstrating that reducing dimensionality through Pearson correlation filtering, chi-square tests, and stepwise regression significantly improves model performance by mitigating noise and redundancy [9, 28]. Additionally, the study highlights that alternative learning paradigms, such as Radial Basis Function (RBF) networks, counter propagation, and Adaptive Resonance Theory (ART), outperform gradient-based methods in noisy and small-sample datasets, reinforcing evidence from deep learning optimization research [32, 43]. The paper further introduces a decision factor (DF) heuristic, correlating the number of hidden neurons with independent decision factors in a given problem, a principle validated in foreign exchange rate forecasting and financial modeling, aligning with Occam's Razor in neural model selection [30, 42]. The study concludes by advocating for automated heuristic-driven Neural Architecture Search (NAS) that integrates evolutionary computation, metaheuristics, and reinforcement learning, fostering a paradigm shift toward interpretable and adaptive ANN design for critical applications in finance, healthcare, and risk assessment.

Metaheuristic Design of Feedforward Neural Networks: A Review of Two Decades of Research

The paper "Metaheuristic Design of Feedforward Neural Networks: A Review of Two Decades of Research" by Ojha et al. [55] presents a structured analysis of metaheuristic-driven optimization techniques for Feedforward Neural Networks (FNNs), addressing the limitations of traditional gradient-based learning in high-dimensional, non-convex search spaces. The study classifies metaheuristic optimization into three key domains: Neural Architecture Search (NAS), weight optimization, and adaptive learning rule selection, systematically comparing single-solution heuristics, such as Simulated Annealing (SA) and Tabu Search (TS), with population-based strategies, including Genetic Algorithms (GAs), Particle Swarm Optimization (PSO), Differential Evolution (DE), and Artificial Bee Colony (ABC). A significant contribution is the differentiation between direct encoding (explicitly defining network structures) and indirect encoding (evolving architectures generatively), where the latter has demonstrated superior scalability and adaptability [58, 65]. Furthermore, the paper explores the role of metaheuristics in hardware-aware NAS, optimizing models under latency, energy efficiency, and memory constraints [16], and highlights heuristic-driven pruning, quantization, and knowledge distillation as effective techniques for enhancing computational efficiency while maintaining accuracy [30]. The study also examines surrogate-assisted optimization and parallel evolutionary searches, mitigating the computational burden of large-scale neural architecture evolution. Despite these advancements, high computational costs, lack of interpretability in evolved architectures, and challenges in convergence stability remain open research questions, with potential solutions in self-adaptive

metaheuristics, neuroevolution-augmented reinforcement learning, and biologically inspired network growth models [34]. The study concludes that metaheuristic-based FNN optimization is positioned as a cornerstone of next-generation deep learning, particularly in applications requiring reinforcement learning, autonomous decision-making, and dynamic model adaptation.

Optimization of Deep Neural Networks: A Survey and Unified Taxonomy

The paper "Optimization of Deep Neural Networks: A Survey and Unified Taxonomy" by Talbi [68] provides a structured and comprehensive review of optimization methodologies for Deep Neural Networks (DNNs), addressing fundamental challenges in NAS, hyperparameter optimization (HPO), and joint NAS-HPO frameworks. Traditional gradient-based techniques, such as stochastic gradient descent (SGD), often struggle in high-dimensional, multimodal, and computationally expensive search spaces, necessitating the integration of metaheuristic and population-based algorithms to enhance search capabilities. The study introduces a unified taxonomy of optimization techniques, categorizing them based on search representation, objective functions, constraints, initialization strategies, and variation operators. It offers a systematic perspective on how different strategies interact [16]. A key focus is on metaheuristic approaches, including GA, DE, PSO, SA, and Reinforcement Learning (RL), which have demonstrated efficiency in evolving neural architectures, balancing exploration-exploitation trade-offs, and enabling multi-objective optimization [58]. The study also highlights the growing role of Bayesian optimization and Monte Carlo Tree Search (MCTS) in refining HPO within AutoML pipelines, significantly reducing computational overhead [20]. Additionally, model compression techniques, such as pruning, quantization, and knowledge distillation, are analyzed as crucial mechanisms for deploying DNNs in resource-constrained environments [30], while hardware-aware NAS leverages constraint-handling strategies for optimizing performance on neuromorphic and edge computing architectures [34]. The study also explores parallel optimization frameworks and surrogate-based modeling, which accelerate large-scale NAS by employing low-fidelity approximations and distributed computing techniques. Despite the advancements in metaheuristic-driven NAS, open challenges persist, including high computational costs, lack of interpretability in evolved architectures, and stability issues in convergence. The author suggests future research should focus on hybrid NAS mechanisms combining evolutionary and gradient-based methods, coevolutionary learning, and biologically inspired models for adaptive deep learning optimization [65]. This taxonomy-based review serves as a key reference for researchers seeking to develop scalable, high-performance, and energy-efficient deep learning models, reinforcing the critical role of heuristic-based optimization in AutoML, reinforcement learning, and dynamic AI frameworks.

Swarm and Evolutionary Computing Approaches for Deep Learning

The paper "A Survey of Swarm and Evolutionary Computing Approaches for Deep Learning" by Darwish et al. [13] provides a comprehensive review of Swarm Intelligence (SI) and Evolutionary Computing (EC) techniques for optimizing DNNs, addressing key challenges in NAS, hyperparameter selection, and model compression. Traditional gradient-based approaches often struggle with local minima, sensitivity to initialization, and high computational cost in large-scale models [53], necessitating the adoption of bio-inspired metaheuristic algorithms such as PSO, GA, DE, ACO, and Cuckoo Search (CS) to enhance search efficiency and model generalization [66]. The study categorizes EC-driven deep learning optimization into three major areas: NAS, hyperparameter tuning, and model compression, highlighting that evolutionary-based NAS leverages mutation, crossover, and selection operators to efficiently explore diverse architectures with multi-objective optimization frameworks balancing accuracy, computational efficiency, and energy consumption [58]. Additionally, swarm-based hyperparameter optimization dynamically tunes learning rates, dropout rates, and weight initializations, enhancing convergence stability and training efficiency [79]. In contrast, EC-based model compression techniques, including pruning, quantization, and knowledge distillation, have effectively reduced model complexity for resource-constrained environments [31]. The survey also explores hybrid NAS strategies integrating reinforcement learning (RL) and differentiable search techniques, demonstrating improved efficiency and convergence properties [16]. Despite significant advances, challenges remain, notably the computational cost of evolutionary searches, the lack of interpretability in EC-based models, and the need for hybrid heuristics that integrate gradient-based learning with evolutionary strategies. The study concludes by discussing the future of neuromorphic computing, federated evolutionary models, and AutoML-driven deep learning architectures, underscoring their potential to drive scalable, autonomous AI systems.

A Technical View on Neural Architecture Search

The paper "A Technical View on Neural Architecture Search" by Hu and Yu [33] provides a structured and in-depth analysis of NAS, a pivotal component of AutoML aimed at optimizing Deep Neural Networks (DNNs) by eliminating manual design overhead. The study categorizes NAS into three primary paradigms—reinforcement learning (RL)-based NAS, gradient-based NAS, and evolutionary algorithm (EA)-based NAS—each addressing distinct optimization challenges. RL-based NAS formulates architecture search as a Markov Decision Process (MDP), where an agent refines network topologies using policy gradient or Q-learning strategies; however, early implementations suffered from high computational

costs, necessitating optimizations like parameter sharing [57], reward shaping, and meta-learning techniques [16]. Gradient-based NAS, exemplified by Differentiable Architecture Search (DARTS) [47], reinterprets NAS as a continuous optimization problem, significantly improving efficiency. However, challenges such as architectural collapse and optimization instability persist [78]. EA-based NAS leverages mutation, crossover, and selection mechanisms to evolve architectures, ensuring broader search space exploration and enabling multi-objective optimization [58], albeit at a high computational cost, prompting recent advancements in surrogate-assisted optimization [37] and hardware-aware NAS [11]. The study further explores search space structuring, advocating for modular and cell-based NAS [84], and discusses low-fidelity evaluation techniques, including early stopping, knowledge distillation, and multi-objective trade-offs, to balance accuracy, efficiency, and resource constraints. The authors emphasize the potential of neuromorphic computing, self-adaptive NAS, and NAS for nonstandard deep learning models, highlighting its growing role in AutoML, scalable AI, and hardware-efficient deep learning.

On the Automated, Evolutionary Design of Neural Networks: Past, Present, and Future

The paper "On the Automated, Evolutionary Design of Neural Networks: Past, Present, and Future" by Baldominos et al. [4] provides a comprehensive review of Neuroevolution, an approach that integrates EAs for optimizing ANNs, extending from early shallow architectures to DNNs. The study classifies neuroevolutionary techniques into topology evolution, refining layer depth, neuron connectivity, and activation functions; weight evolution, enhancing parameter optimization when traditional gradient-based methods struggle; and hybrid Neuroevolution, where evolutionary techniques complement backpropagation, as exemplified in Neuroevolution of Augmenting Topologies (NEATs) [65] and Evolution Strategies (ESs). The paper highlights evolutionary-based NAS, leveraging GA, PSO, DE, and Bayesian Optimization to discover optimal architectures while balancing accuracy, computational efficiency, and model complexity [16, 58]. Additionally, the authors discuss hardware-aware Neuroevolution, where optimization is constrained by latency, energy efficiency, and memory footprint, essential for edge computing and real-time AI applications [30]. While Neuroevolution has demonstrated remarkable progress, key challenges remain, including high computational cost, mitigated through parallelized search strategies and surrogate-assisted optimization, scalability in evolving deep architectures, and interpretability, as evolved models often lack transparency. The study concludes by emphasizing future research directions in self-adaptive evolutionary mechanisms, reinforcement learning-assisted NAS, and neuromorphic computing-inspired models, reinforcing Neuroevolution as a pivotal tool in the future of AutoML and next-generation AI systems [34].

Neuroevolution in Deep Neural Networks: Current Trends and Future Challenges

The paper "Neuroevolution in Deep Neural Networks: Current Trends and Future Challenges" by Galván and Mooney [21] provides an in-depth review of Neuroevolution, where EAs optimize DNNs, particularly in NAS and training optimization. Unlike gradient-based approaches such as stochastic gradient descent (SGD), Neuroevolution explores high-dimensional, multimodal search spaces using GA, DE, Evolution Strategies (ESs), and Neuroevolution of Augmenting Topologies (NEATs), facilitating architecture discovery and hyperparameter tuning [58, 65]. The study emphasizes computational efficiency advancements through hardware-aware NAS, surrogate-assisted optimization, parallel evolutionary strategies, and multi-objective optimization frameworks that balance accuracy, computational cost, and energy consumption, crucial for low-power AI applications [16]. Additionally, the paper explores hybrid neuroevolutionary models, integrating evolutionary search with gradient-based optimization, improving convergence and generalization [34]. Applications include image classification, adversarial robustness, federated learning, and meta-learning, where evolutionary techniques enhance hyperparameter tuning, knowledge distillation, and weight quantization, driving scalability and adaptability [30]. Despite its potential, Neuroevolution faces computational cost and scalability challenges, motivating research into asynchronous parallel search, efficient mutation strategies, and self-adaptive evolutionary mechanisms. The study concludes that Neuroevolution will be instrumental in AutoML and self-learning AI, advancing the field toward autonomous, scalable, and efficient deep learning models [4].

A Comprehensive Survey of Neural Architecture Search: Challenges and Solutions

The paper "A Comprehensive Survey of Neural Architecture Search: Challenges and Solutions" by Ren et al. [59] provides an extensive review of NAS, a core component of AutoML that automates the design of DNNs by optimizing architectures under computational and performance constraints. The study classifies NAS methodologies into search space design, search strategy optimization, and evaluation techniques, analyzing their evolution and identifying persistent challenges such as high computational cost, inefficient search strategies, and the trade-off between model complexity and inference speed. The authors highlight advancements in modular and cell-based search spaces, which improve search efficiency while preserving expressiveness [16], and discuss the impact of continuous search strategies, particularly Differentiable Architecture Search (DARTS), which enables gradient-based optimization in NAS [47]. Additionally, recycling neural architecture, which leverages previously optimized networks to reduce redundant computations, is a promising approach [58].

The paper also explores computational efficiency improvements, particularly in efficient evaluation techniques such as early stopping, weight sharing (e.g., ENAS and one-shot NAS), and surrogate modeling, which significantly reduce training overhead while maintaining ranking fidelity [37,57]. Furthermore, the study emphasizes the growing relevance of multi-objective NAS (MOO-NAS), which optimizes architectures while balancing accuracy, computational cost, and energy efficiency, making it particularly relevant for edge AI and real-time applications [11]. The comparative analysis of reinforcement learning-based NAS, evolutionary NAS, gradient-based NAS, and surrogate-assisted NAS further clarifies their respective advantages and trade-offs, emphasizing the necessity of hardware-aware NAS, where architectures are codesigned with latency, memory footprint, and energy constraints in mind [34].

Given the rapid evolution of NAS, the authors advocate for meta-learning, self-adaptive evolutionary strategies, and hybrid NAS frameworks that integrate gradient-based, reinforcement learning, and evolutionary optimization to achieve scalable and hardware-efficient deep learning models. The study concludes that NAS will be pivotal in the next generation of self-optimizing AI systems, bridging the gap between theoretical advancements and real-world deployment.

A Survey on Evolutionary Construction of Deep Neural Networks

The paper "A Survey on Evolutionary Construction of Deep Neural Networks" by Zhou et al. [82] provides an in-depth analysis of EAs as a heuristic-driven approach to NAS, offering a robust alternative to gradient-based optimization, which often struggles with local optima entrapment, sensitivity to initialization, and poor generalization in complex search spaces. The study systematically categorizes evolutionary deep learning techniques into direct vs. indirect encoding, global vs. modular search, and single-objective vs. multi-objective optimization frameworks, emphasizing the significance of coevolutionary algorithms for simultaneous optimization of network topology and hyperparameters [16, 58]. Moreover, multi-objective evolutionary approaches (MOO-EAs), such as Pareto-based evolutionary search, address key accuracy, computational complexity, and energy efficiency trade-offs, making them particularly relevant for real-world deployment [34].

The paper further explores hardware-aware NAS, where evolutionary search is constrained by latency, memory footprint, and energy consumption, a crucial factor in edge AI applications [11]. Additionally, it examines pruning, quantization, and knowledge distillation techniques, where evolutionary algorithms facilitate model compression without substantial accuracy loss. To mitigate the computational overhead of evolutionary NAS, the study highlights advancements in parallel evolutionary search, surrogate-assisted modeling, and low-fidelity performance estimations, accelerating convergence while preserving exploration capabilities [37,57].

Despite its advantages, the paper acknowledges persistent challenges, such as high computational cost, scalability limitations, and lack of interpretability in

evolved architectures, advocating for emerging solutions like asynchronous parallel search, surrogate-assisted optimization, and self-adaptive evolutionary strategies. Future research directions emphasize interpretable neuroevolution, which enhances model transparency, and neuromorphic-inspired evolutionary computing, aligning neuroevolution with biologically plausible learning models. As Evolutionary Algorithms continue to integrate with AutoML, they are expected to drive the next generation of scalable, efficient, and adaptive deep learning models, fostering autonomous learning with minimal human intervention and optimized resource allocation.

Evolutionary Neural Networks for Deep Learning: A Review

The paper "Evolutionary Neural Networks for Deep Learning: A Review" by Ma and Xie [50] presents an in-depth analysis of Evolutionary Neural Networks (ENNs), which integrate EAs with DNNs to optimize architectures, hyperparameters, and training strategies, mitigating the inherent limitations of gradient-based optimization, such as vanishing gradients, local minima stagnation, and sensitivity to initialization [34, 43]. The study systematically categorizes ENNs into three optimization paradigms: weight evolution, replacing stochastic gradient descent (SGD) with GA, DE, and covariance matrix adaptation evolution strategy (CMA-ES) [65]; neural architecture evolution, leveraging Neuroevolution of Augmenting Topologies (NEAT), hierarchical genetic search, and reinforcement learning-assisted NAS [57, 58]; and learning rule evolution, applying metaheuristic-driven optimization strategies for adaptive training.

A pivotal contribution of this survey is its exploration of encoding strategies, distinguishing between direct encoding, which explicitly defines architectural parameters, and indirect encoding, which employs generative encodings and developmental models for improved search efficiency. The authors further discuss multi-objective evolutionary strategies, optimizing trade-offs between accuracy, computational efficiency, and hardware constraints, making ENNs particularly suited for resource-aware computing and edge AI applications [37, 82]. Additionally, advancements in hardware-aware NAS are reviewed, where architectures are evolved under latency, memory, and energy constraints, facilitating efficient deployment in IoT and low-power AI environments [11, 58].

The paper also examines practical applications of ENNs, including image classification, reinforcement learning, adversarial robustness, and model compression, highlighting pruning, quantization, and knowledge distillation as critical for scalability and computational efficiency [34, 42]. Despite these advancements, challenges such as scalability, interpretability, and computational overhead persist. The study explores emerging solutions, including parallelized evolutionary search, surrogate-assisted evaluation, and neuromorphic-inspired computing, which enhance the feasibility of ENNs in large-scale real-world applications [16, 50].

As Evolutionary Neural Networks continue to evolve, their integration with AutoML frameworks is poised to drive the next generation of self-optimizing

deep learning models, reducing human intervention while ensuring adaptability in dynamic environments.

A Comprehensive Survey on Optimizing Deep Learning Models by Metaheuristics

The paper "A Comprehensive Survey on Optimizing Deep Learning Models by Metaheuristics" by Akay et al. [1] provides a systematic and in-depth analysis of metaheuristic algorithms as optimization strategies for DNNs, addressing the limitations of gradient-based techniques such as SGD and Adam, which often suffer from vanishing gradients, local optima, and hyperparameter sensitivity [34, 43]. By leveraging EAs and swarm intelligence techniques, metaheuristic optimization enables a more exploratory search in high-dimensional, multimodal spaces, leading to improved generalization and computational efficiency [58, 65]. The study categorizes metaheuristic-driven DNN optimization into NAS, hyperparameter tuning, training acceleration, and feature selection, systematically evaluating single-solution-based approaches (e.g., Simulated Annealing (SA), Tabu Search (TS)) and population-based approaches (e.g., GA, PSO, Artificial ABC, and DE). A significant contribution of the survey is its discussion of multi-objective optimization (MOO) frameworks, particularly those employing Pareto-based selection strategies to balance accuracy, model complexity, and computational cost, making them highly relevant for resource-aware AI applications [37, 82].

The study further explores metaheuristic-based NAS, distinguishing between direct encoding, where architectural configurations are explicitly represented, and indirect encoding, which employs evolutionary generative models to optimize network topologies. The authors highlight the importance of hardware-aware NAS, where metaheuristic search is constrained by real-world deployment factors such as memory footprint, energy consumption, and latency, making it a crucial approach for edge computing and IoT applications [11, 58]. Additionally, the paper reviews metaheuristic-driven pruning, quantization, and knowledge distillation as effective strategies for model compression and inference acceleration, particularly in low-power AI systems.

A key challenge in metaheuristic-based DNN optimization is the high computational cost of evolutionary searches. The study discusses solutions such as parallelized metaheuristic frameworks, surrogate-assisted NAS, and hybrid evolutionary-gradient approaches, which integrate reinforcement learning, surrogate modeling, and distributed optimization to enhance efficiency [34, 57]. Moreover, coevolutionary search mechanisms are identified as promising strategies for dynamically adapting mutation strategies and search heuristics, ensuring more robust model convergence.

Despite their advantages, metaheuristic-driven optimization methods still face limitations, including scalability concerns, interpretability challenges, and computational overhead. The authors propose neuromorphic computing-inspired evolutionary approaches, bio-inspired optimization heuristics, and self-adaptive evolu-

tionary NAS as potential directions to enhance adaptability and search efficiency [42,50]. Ultimately, the study advocates for hybrid metaheuristic-based frameworks, combining gradient-based and heuristic search strategies to achieve state-of-the-art performance in Automated Machine Learning (AutoML) and scalable deep learning applications, enabling the next generation of self-optimizing AI models.

Evolutionary Design of Neural Network Architectures: A Review of Three Decades of Research

The paper "Evolutionary Design of Neural Network Architectures: A Review of Three Decades of Research" by Ünal and Başçiftçi [69] provides a comprehensive historical and technical analysis of EC in NAS, tracing advancements from early GA and ES to contemporary hybrid NAS frameworks and hardware-aware evolutionary methods. The study categorizes evolutionary neural network (ENN) design into three primary phases: early ANN optimization (1980s–1990s), hybrid EC-gradient NAS methodologies, and the modern deep learning era, emphasizing the increasing complexity of high-dimensional, multimodal search spaces, where traditional gradient-based optimization, such as SGD, struggles with vanishing gradients, local optima stagnation, and limited adaptability [43, 65]. The paper explores chromosome encoding strategies, distinguishing between direct encoding, where network topologies and weights are explicitly represented, and indirect encoding, leveraging graph-based, hierarchical, and modular representations to enhance scalability and search efficiency [57, 58].

A key contribution of this work is the integration of evolutionary NAS with gradient-based learning, particularly in the context of NEAT and hierarchical NAS frameworks, where multi-objective evolutionary approaches (MOO-EAs) optimize trade-offs between accuracy, computational efficiency, and energy consumption, using Pareto-based selection mechanisms [34, 82]. The study further highlights hardware-aware NAS, where evolutionary optimization is constrained by real-world deployment factors, such as latency, memory footprint, and energy efficiency, making it highly relevant for edge AI, federated learning, and neuromorphic computing [11, 37]. Additionally, surrogate-assisted NAS techniques, employing low-fidelity performance estimation, are examined to mitigate the high computational overhead of evolutionary search [57]. Despite significant advancements, the study identifies key challenges, including search cost, architecture transferability, and lack of interpretability. The authors discuss asynchronous parallel evolution, reinforcement learning-assisted NAS, and self-adaptive evolutionary mechanisms, which dynamically adjust mutation and crossover strategies to enhance search diversity and convergence speed [57, 58]. Furthermore, the potential of bio-inspired coevolutionary strategies, mimicking synaptic plasticity in biological neural systems, is explored as a foundation for lifelong learning and fully automated deep model adaptation. Looking forward, the authors argue that the fusion of evolutionary computation, neuromorphic computing, and AutoML frameworks is expected to

drive the next generation of intelligent, scalable, and hardware-efficient deep learning architectures, significantly reducing human intervention in NAS processes.

Survey on Evolutionary Deep Learning: Principles, Algorithms, Applications, and Open Issues

The paper "Survey on Evolutionary Deep Learning: Principles, Algorithms, Applications, and Open Issues" by Li et al. [46] presents a comprehensive review of Evolutionary Deep Learning (EDL), a paradigm that integrates EC techniques into DNNs to enhance adaptability, efficiency, and automation. The study systematically categorizes EDL into three core components: data preprocessing, model generation, and model deployment, providing a rigorous analysis of search paradigms (single-objective vs. multi-objective optimization), solution representations (linear, hierarchical, and graph-based encoding), and their impact on NAS [16, 57]. The authors highlight the advantages of heuristic-driven, population-based optimization methods, particularly GA, PSO, DE, and neuroevolutionary frameworks, which offer robust exploration capabilities in high-dimensional search spaces where gradient-based methods struggle with local optima and sensitivity to initialization [58, 65]. A pivotal contribution of this work is the detailed examination of multi-objective evolutionary NAS, where Pareto-based search mechanisms effectively balance trade-offs between model accuracy, computational efficiency, and energy constraints, making these approaches particularly relevant for resource-aware deep learning applications [34, 82]. Additionally, the study explores hybrid NAS frameworks, which integrate evolutionary algorithms with gradient-based optimization, accelerating convergence and improving generalization [37]. The authors further investigate evolutionary model compression techniques, including pruning, quantization, and knowledge distillation, essential for deploying DNNs in edge AI and low-power environments [11]. Despite the significant advancements in EDL, computational overhead remains a major limitation, prompting research into surrogate-assisted NAS, hardware-aware evolutionary optimization, and early stopping mechanisms to enhance search efficiency [57]. The paper concludes by emphasizing that EDL is poised to play a transformative role in the evolution of AutoML, driving the development of self-adaptive, scalable, and high-performance AI systems with minimal human intervention.

A Survey: Evolutionary Deep Learning

The paper "A Survey: Evolutionary Deep Learning" by Li and Liu [45] presents a comprehensive and structured review of Evolutionary Deep Learning (EDL), an emerging paradigm that integrates EC with Deep Learning (DL) to optimize neural architectures, hyperparameters, and training strategies. The authors emphasize that traditional deep learning relies on manual design and computationally expensive trial-and-error approaches. In contrast, EDL automates and enhances model con-

struction through heuristic-driven search strategies inspired by biological evolution, such as GA, PSO, DE, and ACO [16, 65]. The study classifies EDL into four primary optimization categories: Learning Rule Optimization (LRO), which adapts training strategies dynamically; Hyperparameter Optimization (HPO), automating the tuning of key parameters; NAS, optimizing the structural configuration of deep networks; and evolutionary approaches for model compression and transfer learning, which improve generalization while reducing computational overhead [34,58]. A key contribution of this survey is its historical review of EC in DL, tracing its evolution from early evolutionary artificial neural networks (EANNs) to modern multi-objective NAS and hardware-aware evolutionary learning [82]. The study critically examines direct encoding (explicit parameter representation) and indirect encoding (generative models and graph-based representations) in evolutionary NAS, alongside hybrid NAS approaches, where evolutionary search is combined with gradient-based optimization for improved efficiency and convergence [37, 57]. Furthermore, the authors explore Pareto-based NAS, which optimizes trade-offs between accuracy, inference latency, energy consumption, and model complexity, making EDL particularly relevant for real-time AI applications and edge computing [11]. Despite advancements, computational cost remains a challenge, prompting research into surrogate-assisted evolutionary search, distributed optimization, and early stopping mechanisms, which reduce evaluation overhead while preserving search quality [34, 57]. The study concludes by highlighting future directions, including self-adaptive neuroevolution, federated evolutionary learning, and neuromorphic computing-inspired EDL, reinforcing that EDL is poised to revolutionize AutoML and self-evolving AI systems.

A Survey on Evolutionary Neural Architecture Search

The paper "A Survey on Evolutionary Neural Architecture Search" by Liu et al. [48] provides a comprehensive and systematic review of Evolutionary Neural Architecture Search (ENAS), a subfield of NAS that leverages EC to automate and optimize DNN design. The authors highlight the computational challenges and human biases inherent in manually crafting DNN architectures, emphasizing that ENAS employs population-based heuristic search to efficiently explore the vast, high-dimensional, and multi-objective NAS space [16, 58]. The study categorizes ENAS into three core components: (1) search space design, where architectural representations range from layer-wise, cell-based, to graph-based encodings, (2) search strategies, encompassing GA, PSO, DE, and Estimation of Distribution Algorithms (EDAs), and (3) evaluation strategies, focusing on surrogate-assisted NAS, weight inheritance, and hardware-aware constraints to optimize architectures under computational limitations [37, 65]. A key contribution of the paper is its critical analysis of ENAS heuristics, particularly hierarchical evolution, multi-objective NAS (MOO-NAS), and energy-efficient architecture design, where Pareto-based methods balance trade-offs between accuracy, computational efficiency, and inference latency [82]. The study further explores parallel and distributed ENAS

frameworks, which exploit cloud computing and multi-GPU training to accelerate search and convergence [11, 57]. Moreover, pruning, quantization, and knowledge distillation via evolutionary strategies are discussed as crucial methodologies for optimizing models in edge AI and mobile computing [34]. Despite its advancements, ENAS still faces challenges in scalability, interpretability, and sample efficiency, prompting future research into self-adaptive ENAS, hybrid gradient-evolutionary NAS, and applications in neuromorphic computing and reinforcement learning [45]. The authors conclude by positioning ENAS as a transformative technology in AutoML, highlighting its role in driving the next generation of self-optimizing and high-performance deep learning models.

Biased Random Key Genetic Algorithm

The Biased Random Key Genetic Algorithm (BRKGA) is an advanced evolutionary metaheuristic derived from the Genetic Algorithm (GA) and represents an improved variation of the Random Key Genetic Algorithm (RKGA) [5]. Initially proposed by [24] and further developed by Ericsson et al. [17], the BRKGA has been widely explored for its effectiveness in solving complex optimization problems [49]. By introducing a bias in the inheritance mechanism, the BRKGA enhances the convergence speed and the quality of solutions compared to its predecessors.

In the BRKGA, each solution is encoded as a chromosome composed of random keys—real-valued numbers in the interval [0, 1]—providing a problem-independent representation. The algorithm commences with generating an initial population of size p, which is then decoded into problem-specific solutions using a dedicated decoder. These solutions are evaluated via a fitness function, and the algorithm proceeds only if a predefined stopping criterion (such as a maximum number of generations or a convergence threshold) has not been met.

The population is partitioned into three groups: an elite subset containing the p_e best-performing individuals, a non-elite subset comprising the remaining $p - p_e - p_m$ individuals, and a mutant subset of p_m individuals generated entirely randomly. This structure facilitates a clear separation between exploitation and exploration within the search space. Elite individuals are carried over to the next generation, ensuring that superior solutions persist, while mutants help maintain genetic diversity and prevent premature convergence.

A key feature of BRKGA is its biased crossover mechanism, which is employed to generate offspring. As shown in Fig. 1, which was adapted from the work of [26], the evolutionary process from generation k to $k + 1$ involves ranking the population by fitness and then producing new individuals through crossover between one elite and one non-elite parent. During the crossover each gene in the offspring's chromosome is determined by generating a random number; if this number is less than the inheritance probability parameter ρ_e (with $\rho_e > 0.5$), the gene is inherited from the elite parent, otherwise from the non-elite parent. This probabilistic scheme ensures that high-quality genetic material is preferentially transmitted while incorporating novel traits to foster diversity.

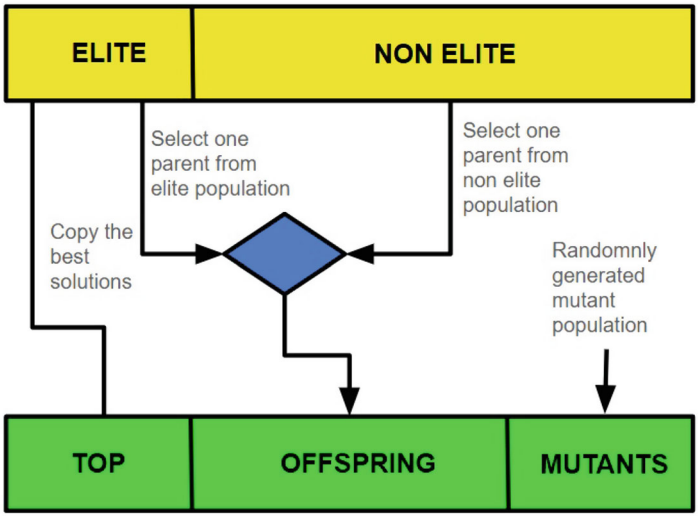


Fig. 1 Evolutionary process from generation k to $k + 1$ in the BRKGA (Source: based on the paper of [26])

The iterative process of selection, biased crossover, mutation, and decoding continues until the stopping criterion is satisfied. The primary advantage of BRKGA over traditional genetic algorithms lies in its systematic bias toward elite solutions, combined with a robust mechanism for introducing random variations. This balance between exploitation and exploration enables efficient navigation of complex search spaces, rendering BRKGA a versatile and powerful tool for combinatorial and continuous optimization problems. Furthermore, its parameterized structure allows for fine-tuning to accommodate specific application requirements, thereby enhancing its overall adaptability and performance.

The iterative process of selection, biased crossover, mutation, and decoding continues until the stopping criterion is satisfied. The primary advantage of BRKGA over traditional genetic algorithms lies in its systematic bias toward elite solutions, combined with a robust mechanism for introducing random variations. This balance between exploitation and exploration enables efficient navigation of complex search spaces, rendering BRKGA a versatile and powerful tool for combinatorial and continuous optimization problems. The algorithm has been successfully applied to a diverse range of optimization problems in the literature. Examples include combinatorial problems such as vehicle routing [52], network design [2], job scheduling [10], and facility location [64]. In addition, BRKGA has been leveraged in real-world applications, including telecommunication network optimization [60], bioinformatics [44, 51], and artificial neural network architecture optimization [18, 62]. These widespread applications underscore the versatility and effectiveness of BRKGA as a state-of-the-art optimization technique.

BRKGA is a sophisticated and widely adopted metaheuristic due to its efficient balance between exploration and exploitation, structured bias toward high-quality solutions, and flexibility in handling various problem domains. Its strong theoretical foundation and empirical success across numerous applications further establish its relevance in the field of optimization. Furthermore, its parameterized structure allows for fine-tuning to accommodate specific application requirements, enhancing its overall adaptability and performance.

BRKGA in Deep Learning Design

This section presents three approaches to applying the Biased Random Key Genetic Algorithm (BRKGA) to the neural architecture design and/or hyperparameter optimization of deep learning models. We will present the proposed methodology to optimize Deep Feedforward Neural Networks (DFNNs), an approach to optimize Convolutional Neural Network (CNN) architectures, and a proposal to optimize Graph Neural Network (GNN) models.

BRKGA-DFNN Proposal

The proposed approach utilizes the Biased Random Key Genetic Algorithm (BRKGA) to optimize Deep Feedforward Neural Network (DFNN) architectures. The optimization framework is governed by a set of key parameters, including population size (p), elite population proportion (p_e), mutant population proportion (p_m), elite inheritance probability (ρ_e), and the algorithm's stopping criterion. Additionally, domain-specific parameters include datasets for training (x_{train}), validation (x_{valid}), and testing (x_{test}). DFNN-related hyperparameters include the number of configurable parameters in hidden layers ($n_{\text{param_hidden}}$) and the maximum number of allowable hidden layers (MaxLayer).

The optimization process follows an evolutionary paradigm, iteratively refining candidate solutions to discover high-performing DFNN architectures. The BRKGA framework is adapted to optimize architectural structures while preserving a balance between exploration and exploitation. The BRKGA-DFNN approach incorporates a multilayer evolutionary strategy, where the algorithm concurrently optimizes architectures with a predefined number of hidden layers. This structured search strategy facilitates scalable and adaptive optimization.

The proposed framework encompasses four core procedures essential for effective search: (1) structured representation of DFNN architectures, (2) robust initialization of individuals and mutants, (3) a lightweight decoder for chromosome-to-architecture mapping, and (4) an efficient fitness evaluation process.

Representation of Feedforward Neural Networks

The representation strategy is crucial when designing optimization algorithms for DFNN architectures. We employ a direct encoding approach, where each indi-

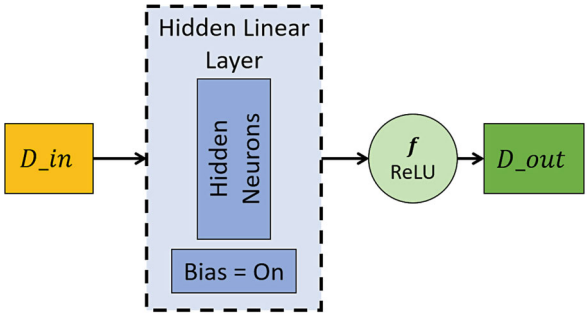


Fig. 2 Schematic representation of a Feedforward Neural Network, comprising an input layer, hidden layer, activation function, and an output layer

vidual's chromosome encodes key architectural properties necessary for defining a trainable DFNN model for classification tasks. Importantly, this study focuses on optimizing the structural composition of DFNNs, excluding weight parameter optimization.

To streamline DFNN optimization, we adopt a linear network structure, as depicted in Fig. 2. This framework includes critical parameters: input feature dimension D_{in} (number of neurons in the input layer), hidden feature dimension (hidden layer neuron count), the inclusion or exclusion of bias terms, and output feature dimension D_{out} (neurons in the output layer). The hidden layer integrates nonlinearity via a predefined activation function (f). Given that input and output dimensions are problem-dependent, they are not treated as hyperparameters. Consequently, BRKGA-DFNN optimizes three key aspects: (1) hidden layer neuron count, (2) bias presence in hidden layers, and (3) activation function type.

The BRKGA-DFNN algorithm (Fig. 3) takes as input the BRKGA parameters, the DFNN-related hyperparameters, and the dataset used for training and evaluation. Initially, the chromosome length for each individual is determined (line 1), corresponding to the number of architectural parameters to be optimized. The number of individuals generated through the crossover process is then calculated (line 2). In line 4, the initial population is randomly generated. Each individual undergoes a decoding step (line 5), where its genetic representation is mapped to a valid DFNN architecture. Following this, each candidate solution is evaluated based on model performance in the validation dataset (line 6). The evolutionary process continues until the predefined stopping criterion is met (line 7). At each iteration, individuals are ranked according to their fitness values, and the population is divided into elite and non-elite groups (lines 8–10). To maintain genetic diversity, a subset of mutants is introduced into the population (line 11). The offspring population is generated through crossover (line 12), where elite individuals contribute with higher probability, preserving beneficial traits while ensuring exploration. The newly generated individuals, along with the mutants and elites, form the next generation of the population (line 13). These individuals undergo decoding (line 14) and fitness reevaluation (line 15) before being reintegrated into the population for the subsequent iteration. To ensure robustness, elite individuals from the previous generation are directly retained (line 16), preventing performance degradation due

begin BRKGA-DFNN (p, p_e, p_m, ρ_e, $criteria$, n_param_hidden, $MaxLayer$, x)

1 N = Eq. (25);

2 $p_o = p - [p \times (p_e + p_m)]$;

3 best_ind = NULL;

4 population ← PopInitial(p, N);

5 population.solution←Decoder(population, n_param_hidden, $MaxLayer$);

6 population.fitness ← Fitness(population, x_{train}, x_{valid});

7 **while** stopping criterion not satisfied **do**

8 population ← PopSort(population);

9 elite_pop ← population[1 : $p_e \times p$];

10 non_elite_pop ← population[$p_e \times p$: p];

11 mutants ← PopInitial($p_m \times p$, N);

12 offspring ← Crossover(elite_pop, non_elite_pop, p_o);

13 new_pop ← mutants + offspring;

14 new_pop.solution←Decoder(new_pop, n_param_hidden, $MaxLayer$);

15 new_pop.fitness ← Fitness(new_pop, x_{train}, x_{valid});

16 population ← elite_pop + new_pop;

17 best_ind ← UpdateBestIndividual(population, best_ind);

18 **end-while**;

19 best_ind.fitness ← Fitness(best_ind, x_{train}, x_{test});

20 **return** best_ind;

end BRKGA-DFNN

Fig. 3 Pseudo-code of a BRKGA to optimize Deep Feedforward Neural Network architectures

to excessive exploration. Throughout the process, the algorithm tracks the best-performing individual found (line 17), continuously updating it as the search progresses. Upon reaching the termination condition, the best solution found undergoes final validation using the test dataset (line 19). This step provides an unbiased estimate of generalization performance, ensuring that the optimized DFNN architecture is evaluated under realistic conditions. The algorithm ultimately returns the best-performing architecture, ready for deployment in classification tasks.

Initialization of Individuals

The initial population and mutant individuals in the BRKGA framework are represented as chromosomes composed of randomly generated keys. The chromosome length is determined by the number of hyperparameters to be optimized. Each gene in the chromosome corresponds to a specific layer or hyperparameter and is represented as L_k and H_p^k, where L_k denotes the position of the gene related to the k-th hidden layer ($k \in 1, \ldots, MaxLayer$). The parameter $MaxLayer$ defines the maximum number of hidden layers allowed in the architecture. H_p^k refers to the gene position associated with the p-th hyperparameter ($p = 1, \ldots, n_param_hidden$, where n_param_hidden is the number of hyperparameters per hidden layer) in the k-th hidden layer. Consequently, the total chromosome length (N) includes all genes encoding the hidden layers and their corresponding hyperparameters, as defined in Eq. (1).

$$N = MaxLayer \times (n_param_hidden + 1) \tag{1}$$

In this representation, we define three hyperparameters for each hidden layer ($n_param_hidden = 3$): H_1^k, H_2^k, and H_3^k. The first hyperparameter (H_1^k) determines the number of neurons in the k-th hidden layer. The second (H_2^k) represents whether bias is applied to the k-th hidden layer. Lastly, H_3^k specifies the activation function assigned to the k-th hidden layer. Figure 4 illustrates the chromosome representation for a BRKGA-DFNN individual with $MaxLayer = 3$.

Decoder

The $Decoder()$ function is responsible for translating the encoded random key chromosome of an individual into a feasible DFNN configuration. This function plays a crucial role in interpreting the encoded genetic information and mapping it to a structured neural architecture. The first $MaxLayer$ genes in the chromosome

Layers			L_1			L_2			L_3		
L_1	L_2	L_3	H_1^1	H_2^1	H_3^1	H_1^2	H_2^2	H_3^2	H_1^3	H_2^3	H_3^3
0.5323	0.1524	0.7419	0.2685	0.8416	0.5361	0.9886	0.0174	0.2542	0.8412	0.0987	0.3683

Fig. 4 Representation of an individual's chromosome for the BRKGA-DFNN approach with $MaxLayer = 3$

determine the activation status of each hidden layer. As defined in Eq. (2), a hidden layer is considered active if its corresponding gene value exceeds 0.5; otherwise, it remains inactive.

$$layers = \begin{cases} 0, & if\ L_k \leq 0.5 \\ 1, & otherwise \end{cases}, \ \forall\, k \in [1, MaxLayer] \tag{2}$$

The number of neurons in each hidden layer (H_1^k) is determined by a transformation that ensures the neuron count remains within a valid range, bounded by the input dimensionality (D_{in}) and the number of output classes (D_{out}). This is defined in Eq. (3). Similarly, the bias status (H_2^k) of each hidden layer is determined using a binary thresholding operation, as outlined in Eq. (4).

Additionally, each hidden layer is assigned an activation function (H_3^k), selected from a predefined set of 17 well-established activation functions, including *Sigmoid*, *Hardshrink*, *Hardtanh*, *LeakyReLU*, *LogSigmoid*, *Tanhshrink*, *PReLU*, *ReLU*, *RReLU*, *ReLU6*, *SELU*, *CELU*, *ELU*, *Softplus*, *Softshrink*, *Softsign*, and *Tanh*. The selection process involves scaling the third gene value by 16 and rounding it to the nearest integer, as specified in Eq. (5).

$$n_neurons = round(H_1^k \times (D_{in} - D_{out}) + D_{out}) \tag{3}$$

$$Bias = \begin{cases} \text{off}, & if\ H_2^k \leq 0.5 \\ \text{on}, & otherwise \end{cases} \tag{4}$$

$$Activation_function = round(H_3^k \times 16) \tag{5}$$

Figure 5 illustrates the decoding process for the chromosome representation depicted in Fig. 4. The Decoder processes only the hyperparameters associated with active layers 1 and 3 (highlighted in blue and green, respectively), while inactive layer 2 (highlighted in pink) is disregarded. Consequently, the final DFNN configuration consists exclusively of active layers, as shown in Fig. 5.

Fitness Evaluation

The $Fitness()$ function evaluates the performance of each individual in the population by assessing the quality of the decoded DFNN model. Each individual undergoes training for 100 epochs using the cross-entropy loss function, the Adam optimizer for parameter updates, and a mini-batch size of 124. After training, individuals are assessed on the validation dataset to guide the evolutionary search. However, only the best-performing individual from the final generation ($best_ind$) is further evaluated on the test dataset to measure its generalization capability.

In our experimental setup, datasets were partitioned into a 70% training subset and a 30% testing subset, with 10% of the test data allocated for validation. For each instance in the dataset, the $Fitness()$ function computes the classification error by comparing the model's predicted output ($\hat{y}_t$) with the actual target label (y_t). The

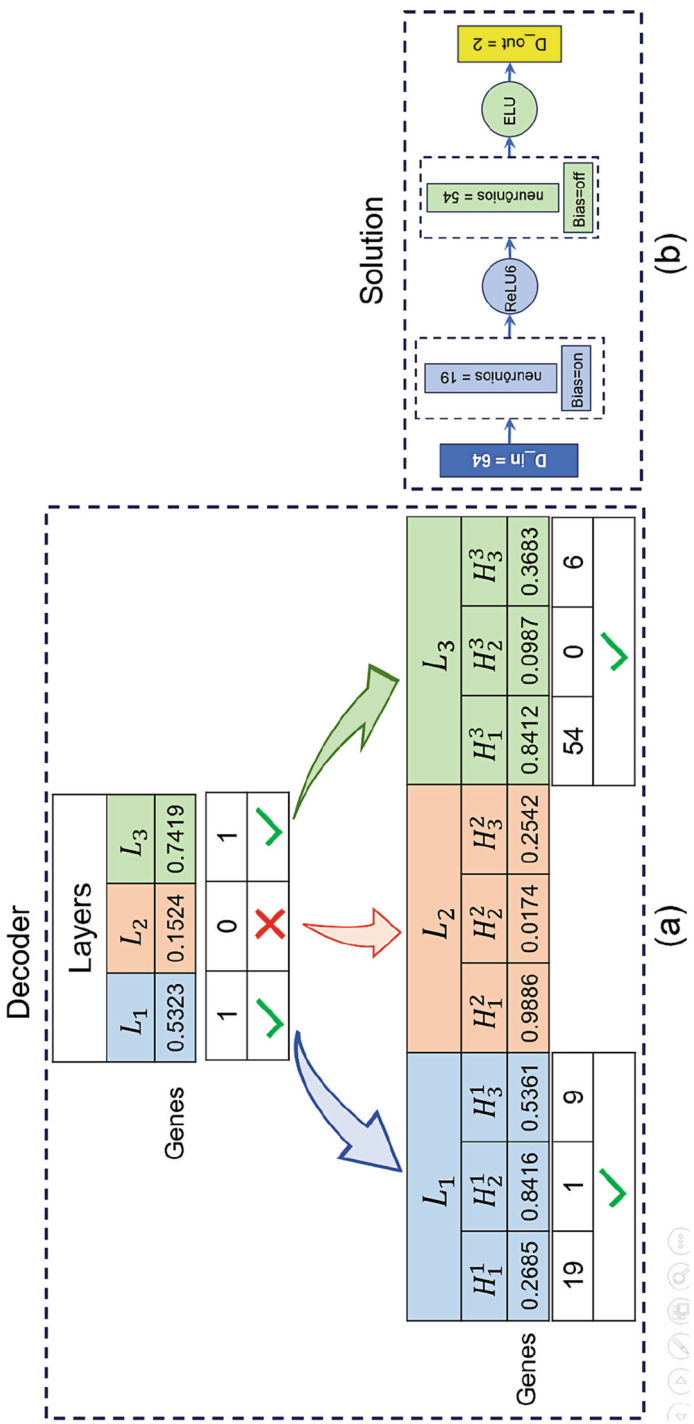


Fig. 5 Illustration of the transformation of an individual's chromosome into a DFNN solution using the BRKGA-DFNN Decoder

overall fitness of an individual is determined by the classification error across the dataset T, as defined in Eq. (6).

$$fitness = \frac{100}{len(T)} \sum_{t=1}^{len(T)} e(t), \quad e(t) = \begin{cases} 0, & if\ \hat{y}_t = y_t \\ 1, & if\ \hat{y}_t \neq y_t \end{cases}, \ \forall\, t \in T \tag{6}$$

BRKGA-CNN Proposal

This section presents the methodology for optimizing Convolutional Neural Network (CNN) architectures using the Biased Random Key Genetic Algorithm (BRKGA), referred to as BRKGA-CNN. The approach incorporates BRKGA parameters, problem-specific settings (tailored for image classification), and hyperparameters defining the CNN architecture. BRKGA-CNN iteratively identifies the best solution by selecting the best-performing individual within the population until a predefined termination criterion is met, adhering to the standard BRKGA framework.

The BRKGA-CNN framework comprises five core procedures that adapt BRKGA for efficient CNN hyperparameter optimization. These include a compact representation of CNN architectures, an encoding scheme for individuals and mutant generation, a robust decoding mechanism for solution interpretation, and an evaluation function for assessing individual performance.

Representation of Convolutional Neural Networks

The proposed methodology is explicitly designed for CNN architectures, focusing on discovering optimal configurations by optimizing key hyperparameters rather than solely adjusting layer weights. In this approach, a BRKGA individual consists of a chromosome with N genes, a fitness score (indicating the quality of the solution), and a corresponding CNN model. The chromosome encodes the essential hyperparameters of each CNN layer, forming the foundation for network construction.

A CNN model is composed of CNN blocks and Fully Connected (FC) blocks. A CNN block integrates a convolutional layer ($Conv$), a Batch Normalization layer (BN), a Pooling layer ($Pool$), and an activation function (f_{CNN}). The FC block includes a Dropout layer, a hidden Linear layer, and an activation function (f_{FC}).

The convolutional layer is characterized by hyperparameters such as the number of output channels ($out_channels$), kernel size, and stride. The Batch Normalization layer requires only the specification of the number of output channels. Given the performance of the Max Pooling method reported in prior works [23, 73], we exclusively use Max Pooling in this study, with kernel size and stride as its primary hyperparameters.

Fig. 6 Representation of the hyperparameters in a CNN block and an FC block

CNN Block					
Output Channels	*kernel*	*stride*	BN	MP	f_{CNN}
0.629	0.413	0.891	0.134	0.597	0.742

FC Block			
Dropout rate	Output	*Bias*	f_{FC}
0.716	0.301	0.552	0.942

For the FC block, the Dropout layer requires defining the dropout rate, while the hidden Linear layer involves hyperparameters such as the number of hidden neurons (output dimension size) and the presence of bias. Additionally, the choice of activation function is a crucial hyperparameter for both CNN and FC blocks due to its significant impact on network performance.

The arrangement of hyperparameters within each block shapes the overall architecture, leading to a vast search space of potential configurations. BRKGA-CNN seeks to identify the optimal combination of hyperparameters across these blocks to construct an efficient CNN for image classification tasks. Figure 6 illustrates the specific hyperparameters optimized for both CNN and FC blocks.

Each CNN block is encoded in the chromosome using six genes, representing (1) number of output channels, (2) kernel size, (3) stride, (4) inclusion of Batch Normalization, (5) presence of a Max Pooling layer, and (6) choice of activation function (f_{CNN}). Similarly, each FC block is encoded using four genes, representing (1) presence and dropout rate of the Dropout layer, (2) number of neurons in the hidden Linear layer ($Output$), (3) bias status, and (4) choice of activation function (f_{FC}).

A CNN model must include at least one CNN block and can incorporate up to MB_{CNN} CNN blocks. Similarly, it must contain at least one FC block, with an upper limit of MB_{FC} FC blocks. The chromosome includes dedicated genes determining the number of CNN and FC blocks used in the model. Each gene within this segment indicates whether a particular block is active, thereby influencing the final architecture of the evolved solution.

Representation of Individuals

The initial and mutant individuals in BRKGA-CNN are generated randomly, with key values ranging from 0 to 1. The number of individuals and the chromosome length are required as input parameters. The chromosome length is determined by the number of CNN and FC blocks and their associated hyperparameters. Each chromosome gene is defined as follows: BC_{kc}, BF_{kf}, HC_{dc}^{kc}, and HF_{df}^{kf}. Here, BC_{kc} and BF_{kf} correspond to the positions in the chromosome that represent the CNN and FC blocks, respectively. Specifically, kc represents the kc-th CNN block ($kc = 1, \ldots, MB_{\text{CNN}}$), and kf represents the kf-th FC block ($kf = 1, \ldots, MB_{\text{FC}}$).

The values of MB_{CNN} and MB_{FC} denote the predefined maximum number of CNN and FC blocks, respectively.

The gene HC_{dc}^{kc} represents the dc-th hyperparameter of the kc-th CNN block ($dc = 1, \ldots, d_{\text{CNN}}$, $kc = 1, \ldots, MB_{\text{CNN}}$), while HF_{df}^{kf} encodes the df-th hyperparameter of the kf-th FC block ($df = 1, \ldots, d_{\text{FC}}$, $kf = 1, \ldots, MB_{\text{FC}}$). Consequently, the total chromosome length (N) of an individual in BRKGA-CNN is given by

$$N = MB_{\text{CNN}} \times (d_{\text{CNN}} + 1) + MB_{\text{FC}} \times (d_{\text{FC}} + 1) \tag{7}$$

Building upon this representation, Fig. 7 details the BRKGA-CNN approach. The algorithm takes as input the standard BRKGA parameters, which include the population size (p), elite size (p_e), mutant size (p_m), elite inheritance probability (ρ_e), and termination criteria defined by the maximum number of generations and stagnation in fitness improvement. Additionally, it requires the CNN-specific parameters: the maximum number of CNN blocks (MB_{CNN}), the maximum number of FC blocks (MB_{FC}), the number of hyperparameters per CNN block ($d_{\text{CNN}} = 6$), and per FC block ($d_{\text{FC}} = 4$). The dataset is also provided, which is split into training (x_{train}), validation (x_{valid}), and test (x_{test}) subsets.

The algorithm begins by determining the chromosome length (line 1), which corresponds to the number of parameters to be optimized. It then sets the population size and defines the number of individuals generated during the crossover process (line 2). The best individual is initially set to an empty state (line 3). The initial population is randomly generated (line 4), and each individual undergoes a decoding process (line 5), where its chromosome representation is mapped to a CNN architecture. Each individual's fitness is then evaluated (line 6), typically based on classification accuracy on the validation dataset. The evolutionary process proceeds iteratively until the stopping criterion is met (line 7). At each generation, individuals are sorted based on fitness values (line 8) and divided into elite and non-elite groups (lines 9–10). A set of mutant individuals is introduced (line 11) to preserve genetic diversity, while the offspring population is generated through crossover (line 12), where genetic material is recombined from elite and non-elite parents according to a predefined inheritance probability. The new population is formed by combining mutants and offspring (line 13). Each individual is then decoded (line 14) to construct a valid CNN architecture and subsequently evaluated (line 15). The elite individuals from the previous generation are carried forward (line 16) to ensure continuity in high-performing solutions. The best individual is updated in each iteration to track the best architecture achieved (line 17). Upon meeting the stopping criterion, the best individual undergoes a final evaluation using the test dataset, ensuring the resulting CNN model generalizes well to unseen data.

To illustrate the encoding strategy, Fig. 8 presents an example chromosome structure with two CNN blocks ($MB_{\text{CNN}} = 2$) and two FC blocks ($MB_{\text{FC}} = 2$), resulting in a chromosome consisting of 24 genes.

begin BRKGA-CNN (p, p_e, p_m, ρ_e, $criteria$, MB_{CNN}, MB_{FC}, d_{CNN}, d_{FC}, x)

1 N = Eq. (7);

2 $p_o = p - [p \times (p_e - p_m)]$;

3 best_ind = NULL;

4 population ← PopInitial(p, N);

5 population.solution ← Decoder(population);

6 population.fitness ← Fitness(population, x_{train}, x_{valid});

7 **while** stopping criterion not satisfied **do**

8 population ← PopSort(population);

9 elite_pop ← population[1 : $p_e \times p$];

10 non_elite_pop ← population[$p_e \times p; p$];

11 mutants ← PopInitial($p_m \times p$, N);

12 offspring ← Crossover(elite_pop, non_elite_pop, p_o);

13 new_pop ← mutants + offspring;

14 new_pop.solution ← Decoder(new_pop);

15 new_pop.fitness ← Fitness(new_pop, x_{train}, x_{valid});

16 population ← elite_pop + new_pop;

17 best_ind ← UpdateBestIndividual(population, best_ind);

18 **end-while**;

19 best_ind.fitness ← Fitness(best_ind, x_{train}, x_{test});

20 **return** best_ind;

end BRKGA-CNN

Fig. 7 Pseudo-code of a BRKGA to optimize Convolutional Neural Network architectures

Blocks				CNN Block 1						CNN Block 2						FC Block 1				FC Block 2			
BC_1	BC_2	BF_1	BF_2	HC_1^1	HC_2^1	HC_3^1	HC_4^1	HC_5^1	HC_6^1	HC_1^2	HC_2^2	HC_3^2	HC_4^2	HC_5^2	HC_6^2	HF_1^1	HF_2^1	HF_3^1	HF_4^1	HF_1^2	HF_2^2	HF_3^2	HF_4^2
0.532	0.635	0.408	0.741	0.629	0.413	0.801	0.134	0.597	0.742	0.432	0.088	0.774	0.562	0.312	0.927	0.325	0.251	0.973	0.145	0.716	0.301	0.552	0.942

Fig. 8 Representation of an individual's chromosome for the proposed BRKGA-CNN. Example with two CNN blocks and two FC blocks

Decoder

The $Decoder()$ function within the BRKGA-CNN framework is designed to translate an individual's random key chromosome into a valid CNN architecture. The activation of CNN and fully connected (FC) blocks is determined by the first MB_{CNN} and MB_{FC} genes, respectively. Equation (8) defines the activation criteria, where $MaxBlock$ represents the total number of CNN and FC blocks. Once the active blocks are identified, the $Decoder()$ assigns values to their respective hyperparameters.

$$blocks[i] = \begin{cases} \text{deactivated}, if\ genes[i] \leq 0.5 \\ \text{activated}, otherwise \end{cases}, \forall\, i \in [1, MaxBlock] \tag{8}$$

The first gene (HC_1^{kc}) in each CNN block determines the number of output channels, scaled by a predefined maximum value ($max_channels$) and rounded to the nearest integer:

$$out_channels = round(HC_1^{kc} \times max_channels) \tag{9}$$

Similarly, the second gene (HC_2^{kc}) encodes the convolutional kernel size, scaled by $max_kernels$:

$$kernel_size = round(HC_2^{kc} \times max_kernels) \tag{10}$$

The third gene (HC_3^{kc}) defines the stride, ensuring its value remains within practical limits:

$$stride = \begin{cases} 1, if\ HC_3^{kc} \leq 0.5 \\ 2, otherwise \end{cases} \tag{11}$$

A Batch Normalization (BN) layer is included based on the fourth gene (HC_4^{kc}):

$$BN = \begin{cases} (out_channels), if\ HC_4^{kc} \leq 0.5 \\ Null, otherwise \end{cases} \tag{12}$$

Max Pooling (MP) layers are similarly determined based on the fifth gene (HC_5^{kc}):

$$\text{MP} = \begin{cases} (kernel_size,\ stride),\ if\ HC_5^{kc} \leq 0.5 \\ Null,\ otherwise \end{cases} \tag{13}$$

To select an activation function for both CNN and FC blocks, the sixth gene (HC_6^{kc}) of each CNN block, or the fourth gene (HF_4^{kf}) of each FNN block, is mapped to one of the 17 predefined activation functions:

$$f_{\text{CNN}} = round(HC_6^{kc} \times 16) \tag{14}$$

$$f_{\text{FC}} = round(HF_4^{kf} \times 16) \tag{15}$$

For FC blocks, the inclusion of a Dropout layer (HF_1^{kf}) depends on the first gene, with the dropout rate computed as

$$dropout = \begin{cases} round(HF_1^{kf},\ 2),\ if\ HF_1^{kf} \leq 0.5 \\ Null,\ otherwise \end{cases} \tag{16}$$

The second gene in the FC block determines the number of hidden neurons:

$$neurons = round(HF_2^{kf} \times (in_feat - D_out) + D_out) \tag{17}$$

Finally, the bias status (HF_3^{kf}) is set based on the third gene:

$$bias = \begin{cases} \text{off},\ if\ HF_3^{kf} \leq 0.5 \\ \text{on},\ otherwise \end{cases} \tag{18}$$

Figure 9 illustrates the decoding process applied to a chromosome depicted in Fig. 8. The first two genes determine the activation of CNN blocks (BC_1, BC_2), while the third and fourth genes determine the activation of FC blocks (BF_1, BF_2). If $BF_1 \leq 0.5$, FC block 1 is deactivated, whereas the fourth gene determines whether FC block 2 is included. The final CNN architecture is then assembled by decoding the active blocks, as illustrated at the bottom of Fig. 9. Finally, if all $MaxBlock$ genes are inactive, a default architecture with one CNN and one FC block is selected, using the highest-ranking genes in each category.

Fitness Evaluation

The $Fitness()$ function evaluates the quality of each individual by assessing its corresponding CNN model. The CNN architecture, decoded by the $Decoder()$ function, undergoes training within the $Fitness()$ function. Each model is trained for a predetermined number of epochs using the Cross-Entropy Loss function and the Adam optimizer [38], with a learning rate set to $1e$–4 and a mini-batch size of 100. The training process is conducted on the training dataset (x_{train}), while the

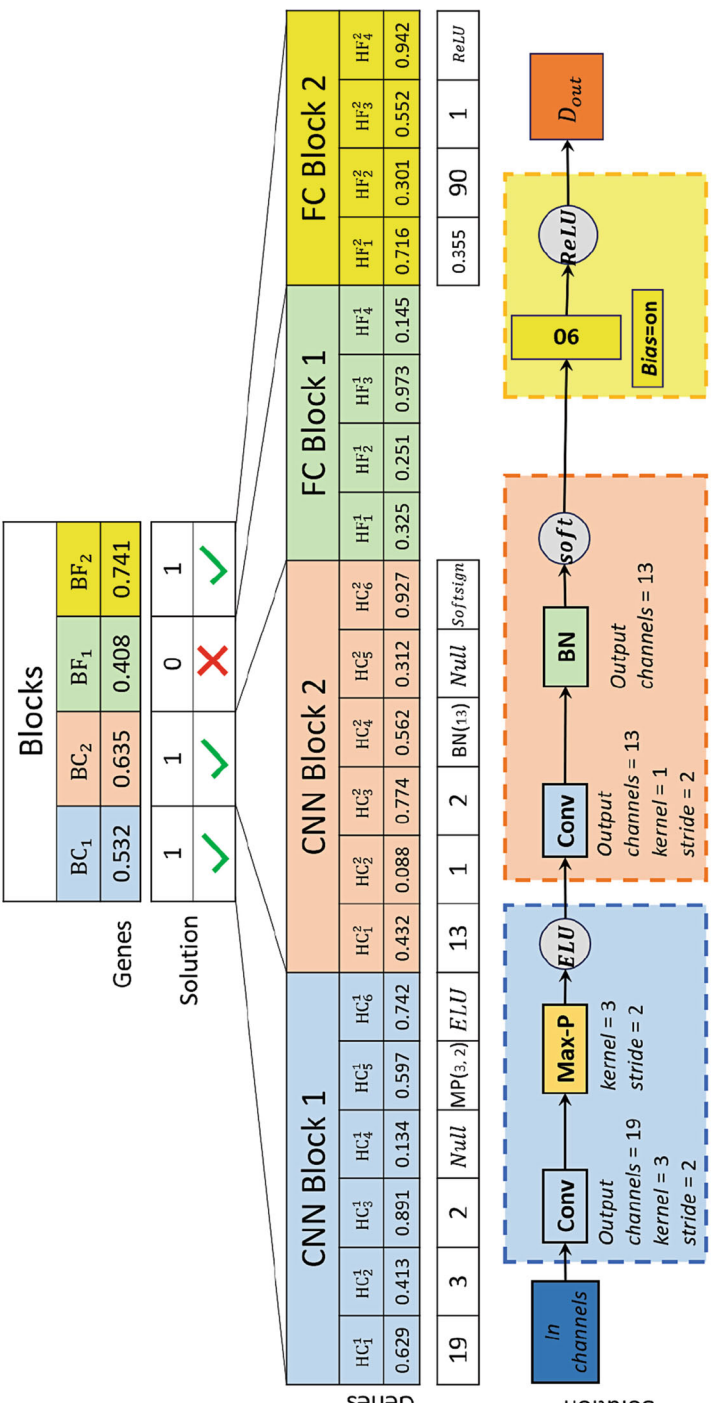


Fig. 9 Illustration of the chromosome decoding process in the BRKGA-CNN framework

validation dataset (x_{valid}) is used to monitor performance. Only the best-performing individual ($best_ind$) is subsequently evaluated on the test dataset (x_{test}), ensuring an unbiased assessment of its generalization capability.

The fitness score of a solution is determined using the classification error metric. For each instance t in the dataset T, Eq. (19) quantifies the discrepancy between the model's predicted output ($\hat{y}_t$) and the true class label (y_t). An error value of 0 is assigned if the prediction ($\hat{y}_t$) is correct; otherwise, the error is 1. Thus, a lower fitness score reflects a more accurate model. Equation (19) formalizes the fitness calculation, which serves as the optimization criterion for guiding the search toward higher-performing CNN architectures.

$$\text{Fitness} = \frac{1}{\text{len}(T)} \sum_{t=1}^{\text{len}(T)} \text{Error}(\hat{y}_t, y_t) \tag{19}$$

BRKGA-GNN Proposal

Representation of Graph Neural Networks

The BRKGA-GNN approach is designed to optimize the architecture of Graph Neural Networks (GNNs), distinguishing itself from methods such as Genetic-GNN, which simultaneously optimize hyperparameters and weights. In BRKGA-GNN, each individual (chromosome) encodes N genes, a fitness value, and a corresponding GNN solution. The chromosome represents key GNN hyperparameters essential for constructing a general GNN model, which consists of an input layer, multiple GNN layers, and an output layer responsible for generating embeddings. Figure 10 illustrates these components. The hyperparameters related to attention are:

- Attention Function: Unlike GraphNAS and Auto-GNAS, which use fixed attention mechanisms, BRKGA-GNN evaluates nine well-known functions (GCN

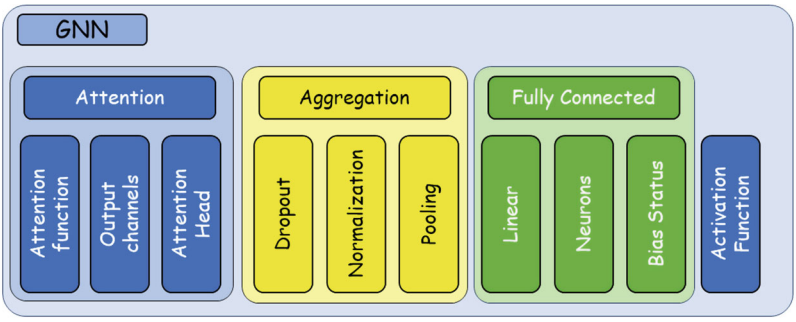


Fig. 10 Representation of the hyperparameters of GNN layers

[39], GraphSAGE [29], GAT [70], GraphConv [54], GIN [76], ChebConv [14], SGC [74], TAG [15], and ARMA [8]) to define node feature aggregation.
- Output channels determine the dimensionality of intermediate representations, with an upper bound defined by $max_out_channels$.
- Attention Heads enhance expressiveness via multi-head mechanisms, with a limit set by $max_attention_head$.

The hyperparameters related to aggregation are:

- Dropout: It regularizes training by randomly deactivating units, reducing overfitting.
- Normalization: Techniques like BatchNorm [35], LayerNorm [3], GraphNorm [12], and PairNorm [80] stabilize training by standardizing input features.
- Pooling Layer: It aggregates node features for graph-level representations using global add, mean, and max pooling methods.
- Linear Layer: It adds flexibility with fully connected layers, transforming features across higher dimensions ($max_out_features$).

Activation Functions: BRKGA-GNN explores 20 activation functions, including ReLU and GELU, to enhance expressiveness, broadening the search space for optimal architectures.

Furthermore, we explore the training hyperparameters: Weight Decay and Learning Rate. Weight Decay is responsible for regularizing by penalizing large weights and mitigating overfitting. The Learning Rate balances optimization speed and stability, which is crucial for effective model convergence.

The hyperparameters considered in BRKGA-GNN fall into four categories: attention mechanisms, aggregation strategies, activation functions, and training hyperparameters.

- **Attention Mechanisms**:
  - Attention Function: Unlike GraphNAS and Auto-GNAS, which use fixed attention mechanisms, BRKGA-GNN explores nine widely used attention functions: GCN [39], GraphSAGE [29], GAT [70], GraphConv [54], GIN [76], ChebConv [14], SGC [74], TAG [15], and ARMA [8].
  - Output Channels define the dimensionality of intermediate representations, constrained by $max_out_channels$.
  - Attention Heads expand expressiveness using multi-head attention mechanisms, with a maximum limit set by $max_attention_head$.
- **Aggregation Strategies**:
  - Dropout: It regularizes training by randomly deactivating neurons, mitigating overfitting effects.
  - Normalization: This incorporates BatchNorm [35], LayerNorm [3], GraphNorm [12], and PairNorm [80] to stabilize training through feature standardization.
  - Pooling Layer: It aggregates node features to construct graph-level representations using global add pooling, mean pooling, and max pooling.

  - Linear Layer: It adds a fully connected layer to facilitate transformations across higher-dimensional feature spaces, constrained by $max_out_features$.
- **Activation Functions**: BRKGA-GNN optimizes among 20 activation functions, including ReLU, GELU, and other common nonlinearities, broadening the search space for expressive and efficient architectures.
- **Training Hyperparameters**: Additionally, BRKGA-GNN explores two training hyperparameters that impact optimization stability and generalization performance:
  - Weight Decay: It regularizes training by penalizing large weight magnitudes, effectively mitigating overfitting.
  - Learning Rate: It controls the step size in optimization updates, balancing convergence speed and stability for improved model training.

Representation of Individuals

In the BRKGA-GNN framework, individuals are encoded as chromosomes that capture the hyperparameters for GNN layers and training configurations. Each chromosome consists of random keys, where each key maps to a specific parameter within the GNN's architecture and training setup. The chromosome length is determined by the number of layers (n_{layers}), the number of hyperparameters per layer (n_{param_gnn}), and the number of training hyperparameters (n_{param_train}), as defined in Eqs. (20) and (21).

$$H = n_{param_gnn} + n_{param_train} \tag{20}$$

$$N = n_{layers} \times (n_{param_gnn} + 1) + n_{param_train} \tag{21}$$

Each gene within the chromosome consists of two components: GL_k and P_h^k. Here, GL_k denotes the position of the gene associated with the k-th GNN layer, where $k \in \mathbb{R}^{|n_{layers}|}$ and $k = 1, 2, \ldots, n_{layers}$, indicating the predefined maximum number of layers. Likewise, P_h^k refers to the position of the gene corresponding to the h-th hyperparameter, where $h \in \mathbb{R}^{|H|}$ and $h = 1, 2, \ldots, H$, where H represents the total number of hyperparameters, as given in Eq. (20).

Thus, the chromosome length N encapsulates genes for the maximum allowable layers, training hyperparameters, and specific hyperparameters for each GNN layer, as defined by Eq. (21).

We define ten hyperparameters per GNN layer ($n_{param_gnn} = 10$), denoted as P_1^k to P_{10}^k for the k-th layer. Additionally, two training hyperparameters ($n_{param_train} = 2$), represented by P_{11} and P_{12}, influence overall training dynamics. Figure 11 illustrates an individual's chromosome for $n_{layers} = 2$, highlighting the first and second layers in green and red.

A detailed breakdown of each hyperparameter and its corresponding search space is provided below. Tables 3 and 4 summarize the search spaces for GNN layer and training hyperparameters, respectively.

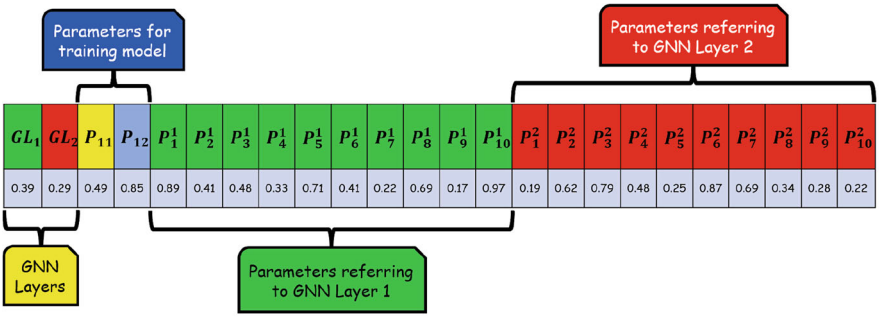


Fig. 11 Illustration of an individual's chromosome from the proposed BRKGA-GNN, with two GNN layers

Table 3 Search space for GNN layer hyperparameters

Component	Parameter	Search space
P_1^k	Attention function	(GCN, GraphSAGE, GAT, GraphConv, GIN, ChebConv, SGC, TAG, ARMA)
P_2^k	Output channels	[1, max_out_channels]
P_3^k	Attention head	[1, max_attention_head]
P_4^k	Dropout	(0.05, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6)
P_5^k	Normalization	(BatchNorm, LayerNorm, GraphNorm, PairNorm)
P_6^k	Pooling	(Global add pool, global mean pool, global max pool)
P_7^k	Linear	(True, False)
P_8^k	Number of hidden neurons	[1, max_out_features]
P_9^k	Bias status	(On, Off)
P_{10}^k	Activation function	(Sigmoid, Hardshrink, Hardtanh, Hardsigmoid, LeakyReLU, LogSigmoid, Tanhshrink, PReLU, ReLU, RReLU, ReLU6, SELU, CELU, ELU, Softplus, Softshrink, Softsign, Tanh, GELU e SiLU)

Table 4 Search space for model training hyperparameters

Component	Parameter	Search space
P_{11}	Weight decay rate	(5e–4, 8e–4, 1e–3, 4e–3)
P_{12}	Learning rate	(5e–4, 1e–3, 5e–3, 1e–2)

- **Attention function** (P_1^k) defines the attention mechanism employed in each GNN layer. The model evaluates nine prominent attention functions: GCN, GraphSAGE, GAT, GraphConv, GIN, ChebConv, SGC, TAG, and ARMA, as available in the PyTorch Geometric library.
- **Output channels** (P_2^k) specifies the number of output channels per attention function, constrained by $max_out_channels$.
- **Attention head** (P_3^k) determines the number of attention heads per layer, limited by $max_attention_head$.

- **Dropout** (P_4^k) defines the dropout rate, ranging from 0.05 to 0.6.
- **Normalization** (P_5^k) specifies the normalization strategy, including BatchNorm, LayerNorm, GraphNorm, and PairNorm, to enhance training stability.
- **Pooling** (P_6^k) determines the pooling technique, choosing among global add pooling, mean pooling, and max pooling.
- **Linear** (P_7^k) indicates whether a fully connected layer is included (True/False).
- **Number of hidden neurons** (P_8^k) specifies the number of neurons in the linear layer, constrained by $max_out_features$.
- **Bias status** (P_9^k) defines whether bias is applied (On/Off).
- **Activation function** (P_{10}^k) determines the activation function per layer, with options including Sigmoid, LeakyReLU, ReLU, and GELU.
- **Weight decay rate** (P_{11}) specifies the weight decay rate for regularization, with values between 5e–4 and 4e–3.
- **Learning rate** (P_{12}) defines the learning rate for training, with values ranging from 5e–4 to 1e–2.

After defining the hyperparameter search spaces, individuals are initialized with fixed chromosome dimensions. BRKGA-GNN follows the standard BRKGA methodology, where the decoder (detailed in the following section) maps each individual's chromosome into a fully configured GNN architecture.

Decoder

The $Decoder()$ function translates a chromosome with random keys into a solution for the problem. The solution returned is a trainable GNN model comprising several GNN layers. Furthermore, the $Decoder()$ function defines two critical training parameters for the model: the learning rate and the weight decay rate.

In the BRKGA-GNN framework, individuals are encoded as chromosomes that capture the hyperparameters for GNN layers and training configurations. Each chromosome consists of random keys, where each key maps to a specific parameter within the GNN's architecture and training setup. The chromosome length is determined by the number of layers (n_{layers}), the number of hyperparameters per layer (n_{param_gnn}), and the number of training hyperparameters (n_{param_train}), as defined in Eqs. (20) and (21).

The first n_{layers} genes of the chromosome correspond to the GNN layers within the solution. To ascertain whether the k-th GNN layer is active, the $active_gnn$ list is constructed for all $k \in [1, n_{layers}]$ as described in Eq. (22).

$$active_gnn[k] = \begin{cases} True, \ if \ GL_k \leq 0.5 \\ False, \ otherwise \end{cases} \tag{22}$$

If $active_gnn[k]$ is True, the corresponding GNN layer is included in the model; otherwise, it is excluded. Each active layer contributes as a component in the finalized GNN structure. Following the layer configuration, the subsequent two genes encode the training hyperparameters. These genes are captured in the array

$genes_train$: $genes_train = genes[n_{layers} : n_{layers} + n_{param_train}]$, where $n_{param_train} = 2$. The weight decay (P_{11}) and learning rate (P_{12}) are defined by Eq. (23) and Eq. (24), respectively:

$$P_{11} = round(genes_train[1] \times len(weight_decays)) \tag{23}$$

$$P_{12} = round(genes_train[2] \times len(learning_rates)) \tag{24}$$

Subsequent genes encode the hyperparameters for each GNN layer. The following process outlines the decoding for a single layer, which is similarly applied to other layers:

- **Attention Function:** The first gene selects an attention function. The gene value, when scaled by the number of available functions, determines the function from the list $attention_list$ (Eq. (25)):

$$P_1^k = attention_list[round(genes[1] \times len(attention_list))] \tag{25}$$

- **Output Channels:** The second gene defines the number of output channels (P_2^k), scaled by $max_out_channels$ and rounded up (Eq. (26)):

$$P_2^k = round(genes[2] \times max_out_channels) + 1 \tag{26}$$

- **Attention Heads:** The third gene determines the number of attention heads (P_3^k), scaled and rounded similarly (Eq. (27)):

$$P_3^k = round(genes[3] \times max_attention_head) + 1 \tag{27}$$

- **Dropout:** The fourth gene decides the inclusion of a dropout layer (P_4^k) based on Eq. (11), with its rate calculated by Eq. (29):

$$P_4^k = \begin{cases} Dropout(p), & if\ genes[4] \leq 0.5 \\ False, & otherwise \end{cases} \tag{28}$$

$$p = drop_rates[round(2 \times genes[4] \times len(drop_rates))] \tag{29}$$

- **Normalization:** The fifth gene determines whether a normalization layer is added, with its type defined in Eq. (31):

$$P_5^k = \begin{cases} (BN), & if\ genes[5] \leq 0.5 \\ False, & otherwise \end{cases} \tag{30}$$

$$BN = norm_list[round(2 \times genes[5] \times len(norm_list))] \tag{31}$$

- **Pooling Layer:** The sixth gene dictates the inclusion of a pooling layer (P_6^k) and its type (Eq. (33)):

$$P_6^k = \begin{cases} (pooling), \ if \ genes[6] \leq 0.5 \\ False, \ otherwise \end{cases} \tag{32}$$

$$pooling = pool_list[round(2 \times genes[6] \times len(pool_list))] \tag{33}$$

- **Linear Layer:** The seventh gene determines the inclusion of a linear layer (P_7^k) (Eq. (14)). If included, the number of hidden units (P_8^k) is calculated (Eq. (17)), followed by the bias status (P_9^k) (Eq. (18)):

$$P_7^k = \begin{cases} \text{Linear(neurons, bias)}, \ if \ genes[7] \leq 0.5 \\ False, \ otherwise \end{cases} \tag{34}$$

$$P_8^k = round(genes[8] \times max_out_features) + 1 \tag{35}$$

$$P_9^k = \begin{cases} Bias(\text{off}), \ if \ genes[9] \leq 0.5 \\ Bias(\text{on}), \ otherwise \end{cases} \tag{36}$$

- **Activation Function:** The activation function (P_{10}^k) is chosen based on the tenth gene, scaled and rounded to select from act_func_list (Eq. (15)):

$$P_{10}^k = act_func_list[round(genes[10] \times n_activations)] \tag{37}$$

Once all layers are processed, the chromosome represents a fully defined trainable GNN model, ready for training and evaluation. For example, Fig. 12 illustrates the decoding of a chromosome with $n_{layers} = 2$, resulting in a two-layer GNN model, with parameters such as $max_out_channels = 250$ and $max_out_features = 100$.

Fitness Evaluation

Once the Decoder function outputs a GNN solution, the fitness evaluation function measures the individual's fitness during model training. The training uses the Negative Log Likelihood Loss (*NLLLoss*) minimized via the Adam optimizer, with hyperparameters P_{11} (weight decay) and P_{12} (learning rate) set during decoding. The model is trained on mini-batches from the training set and managed by a data controller.

Fitness is assessed on the validation set, where the model's accuracy determines its fitness score. The fitness function calculates accuracy as per Eq. (19), with T as the total samples in the validation set, $\hat{y}_t$ as the predicted label, and y_t as the ground truth. A lower fitness score reflects fewer prediction errors, indicating higher accuracy. After evolution, the test set evaluates the top solution from BRKGA-GNN,

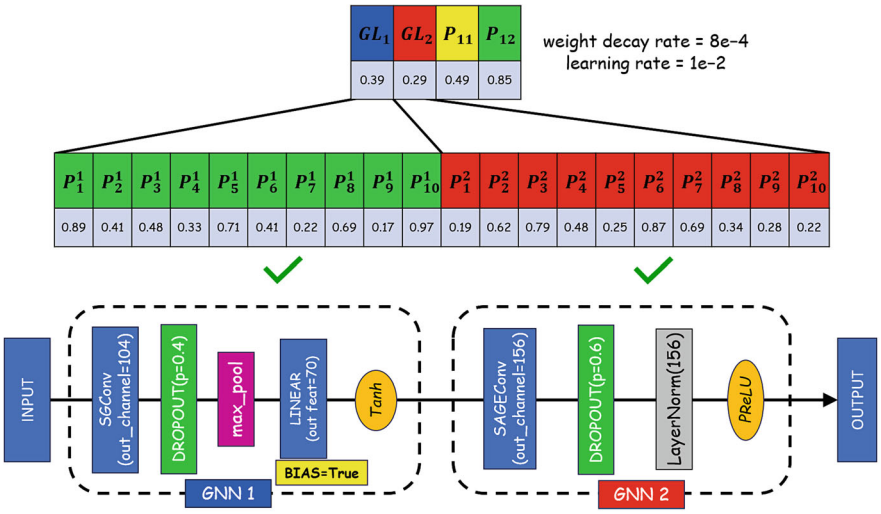


Fig. 12 Illustration of the decoding process of an individual's chromosome in BRKGA-GNN, resulting in a GNN solution with two layers

ensuring an unbiased generalization assessment. This fitness evaluation integrates into the evolutionary process, refining GNN architectures based on training and validation performance for robust development.

Experiments: Datasets, Models, and Configurations of Models

BRKGA-DFNN

To evaluate the performance of our proposed BRKGA-DFNN, experiments were conducted using two established benchmark datasets and a set of state-of-the-art heuristic models, with carefully selected configurations to ensure fair and robust comparisons.

Datasets

The experimental evaluation utilized the MNIST [41] and Fashion-MNIST [75] datasets, both widely recognized in the machine learning community. The MNIST dataset is a widely used benchmark, comprising 70,000 grayscale images of handwritten digits ranging from 0 to 9. Each image has a resolution of 28 $\times$ 28 pixels, with 60,000 samples designated for training and 10,000 for testing. Its extensive use in classification tasks underscores its value for performance evaluation across various algorithms. Fashion-MNIST presents a more complex challenge, replacing digits with images of clothing items across ten categories: T-shirt/top, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, and Ankle Boot. Like MNIST, it contains 70,000 grayscale images of 28 $\times$ 28 pixels, with a 60,000/10,000

split for training and testing, respectively. This dataset has become a standard for benchmarking image classification models in fashion-related contexts.

Comparative Models

To benchmark BRKGA-DFNN, we compared its performance against four heuristic-based models designed for DFNN optimization: TS-DFNN [27], GRASP-DFNN [63], IPSONet [77], and HyperBRKGA [36]. Each model was implemented according to its respective author's specifications, ensuring consistent experimental procedures across all models.

Experimental Configurations

The experiments were executed in the Google Colaboratory environment, leveraging an Nvidia T4 Tensor Core GPU. The local machine used to access the platform featured an Intel Core i5 processor, 16 GB of RAM, and a 512 GB SSD. The BRKGA-DFNN algorithm was implemented in Python 3.10 with PyTorch 1.9.0 [56], adhering to Object-Oriented Programming principles. The code will be made publicly available to facilitate reproducibility.

Preliminary trials determined the optimal configuration for BRKGA parameters. The final settings included a population size of 20, an elite proportion of 0.20, a mutant proportion of 0.15, and a crossover probability of 0.65 for elite parents. Two stopping criteria were applied: a maximum of 20 generations or ten consecutive generations without improvement. Baseline models adhered to the same stopping criteria for consistency, with each experiment repeated 30 times. The best solution for each model was determined by the lowest classification error achieved on the test datasets.

BRKGA-CNN

This subsection outlines the datasets, comparative models, and configurations used in the experiments to evaluate the proposed BRKGA-CNN approach.

Datasets

The experiments were conducted using five datasets summarized in Table 5. A preprocessing step was applied to each dataset to normalize input features and encode target labels for classification tasks. Among the selected datasets were MNIST [41] and four variations of the original MNIST dataset, as proposed by Larochelle et al. [40], which were utilized to create more challenging learning

Table 5 Datasets used in BRKGA-CNN experiments

Datasets	Train set	Test set	Features	Classes
MNIST	60,000	10,000	28×28	10
MNIST-BI	12,000	50,000	28×28	10
MNIST-RB	12,000	50,000	28×28	10
MNIST-RD	12,000	50,000	28×28	10
MNIST-RD+BI	12,000	50,000	28×28	10

environments. These variations aim to test the generalization capabilities of the best-performing models. Each of these datasets includes 10,000 training examples, 2,000 validation examples, and 50,000 testing examples, differing from the original MNIST in the reduced number of training examples, which increases the difficulty level. The variations are described as follows:

- **MNIST-RD (Rotated Digits)** contains digits (0–9) rotated by an angle uniformly sampled between 0 and 2π radians. This variation incorporates the rotation angle in addition to the inherent writing style variations from the original MNIST.
- **MNIST-RB (Random Background)** adds a random background to each digit image, with pixel values uniformly generated between 0 and 255.
- **MNIST-BI (Background Image)** superimposes digit images on 20 randomly selected low-quality black and white images sourced from the Internet.
- **MNIST-RD+BI (Rotated Digits + Background Image)** combines the MNIST-RD and MNIST-BI variations.

Comparative Models

For comparison, models that optimize CNN architectures using metaheuristic algorithms were selected. These models align with the objective of the BRKGA-CNN approach, focusing on image classification tasks. The selected models have demonstrated superior performance on benchmark datasets, making them suitable for assessing the effectiveness of BRKGA-CNN. The models chosen include the IPPSO method by [72], the evoCNN model by [67], and the psoCNN design by [19].

Experimental Configurations

All experiments were executed in the Google Colaboratory environment, utilizing an Nvidia T4 Tensor Core GPU. The system configuration included an Intel Core i5 processor, 16 GB RAM, and a 512 GB SSD. The BRKGA-CNN implementation was developed in Python 3.10, leveraging PyTorch 1.9.0 [56] and adhering to Object-Oriented Programming principles.

Parameter optimization for BRKGA-CNN involved systematic trials using one dataset and performing 20 independent runs of the BRKGA algorithm. The parameters varied included the population size p between [10, 25], elite proportion p_e within [0.05, 0.25], mutant proportion p_m within [0.05, 0.25], and elite inheritance probability ρ_e within [0.51, 0.75]. The selected configuration for BRKGA-CNN was population size $p = 20$, elite proportion $p_e = 0.20$, mutant proportion $p_m = 0.15$, and elite inheritance probability $\rho_e = 0.70$. The algorithm was terminated after 20 generations or 10 consecutive generations without improvement in the best individual.

CNN architecture-specific parameters were set as follows: $MB_{\text{CNN}} = 20$, $MB_{\text{FC}} = 5$, $max_channels = 150$, $max_kernels = 7$, and the number of training epochs fixed at 10. The BRKGA-CNN approach was tested over 30 independent runs with fixed seeds to ensure reproducibility. The optimal solution for each dataset was recorded, and the implementation is publicly available in a repository for community use.

BRKGA-GNN

This subsection details the datasets, baseline models, and configurations employed in the experiments to assess the proposed BRKGA-GNN approach.

Datasets

For our experiments, we utilized a citation network dataset where nodes represent documents, and edges denote citation relationships among them. The Cora dataset [39] contains 2,708 research papers categorized into seven distinct classes, such as Reinforcement Learning and Genetic Algorithms. A total of 5,429 citation links interconnect these nodes. Each paper is represented by a feature vector of 1,433 dimensions, indicating the presence or absence of specific keywords.

Comparative Models

To evaluate the effectiveness of BRKGA-GNN, we compared its performance against both handcrafted and NAS-based GNN models. The handcrafted architectures included Chebyshev [14], GCN (Graph Convolutional Network) [39], GAT (Graph Attention Network) [70], and LGCN (Learnable Graph Convolutional Network) [22]. For the NAS-based approaches, we considered GraphNAS [61], Auto-GNN [83], SNAG (Simplified NAS for GNNs) [81], and Genetic-GNN [61].

Experimental Configurations

We conducted a parameter selection process for BRKGA-GNN to balance computational efficiency with solution quality. The goal was to identify configurations that discover high-performing GNN architectures within a reasonable timeframe.

The parameter selection targeted two categories: BRKGA parameters and GNN architecture parameters. For BRKGA, we evaluated parameters such as population size (p), number of populations (k), elite population proportion (p_e), mutant population proportion (p_m), and elite inheritance probability (ρ_e). For the GNN architecture, parameters included the number of layers (n_{layers}), the maximum number of output channels for the attention mechanism ($max_out_channels$), maximum attention heads ($max_attention_head$), maximum hidden neurons in the linear layer ($max_out_features$), and the number of training epochs per individual ($epochs$) during fitness evaluation. The experiments on the Cora dataset aimed to find the optimal parameter configuration that yielded the highest test accuracy. Each execution adhered to stopping criteria ($criteria$): a maximum of 20 generations or 10 generations without improvement. These criteria ensured meaningful exploration of the parameter space while preventing excessive computational costs.

At the end of the parameter definition runs, we set the following values for the BRKGA parameters: $p = 50$, $k = 2$, $p_e = 0.1$, $p_m = 0.2$, and $\rho_e = 0.65$. For the GNN layer parameters, we set the following values: $n_{layers} = 2$, $max_out_channels = 100$, $max_attention_head = 16$, $max_out_features = 500$, and $epochs = 100$.

Results and Discussion

BRKGA-DFNN

The experimental evaluation involved 30 independent runs for each model, with the lowest classification error rates on the test datasets summarized in Table 6. This table highlights two key metrics for each model: the classification error of the best-performing solution ("best") and the average classification error across all runs ("mean"). The following discussion analyzes the performance of the models on each dataset, focusing on these two metrics for comparison.

For the MNIST dataset, the proposed BRKGA-DFNN model exhibited exceptional performance, achieving the lowest error rates for both the best solution (1.85%) and the mean value (2.83%). Among the competing models, IPSONet delivered competitive results for the best solution (2.00%), while HyperBRKGA secured the second-best mean classification error (2.97%). Conversely, the TS-DFNN model displayed the highest error rates, indicating lower performance than the other approaches.

For the Fashion-MNIST dataset, BRKGA-DFNN also outperformed the benchmark models, recording the lowest classification error for the best solution (11.46%) and the mean metric (12.29%). In contrast, TS-DFNN, IPSONet, and HyperBRKGA exhibited higher error rates for the best solutions, with respective values of 14.50%, 13.48%, and 12.90%. This trend further underscores the robustness of BRKGA-DFNN in tackling more challenging classification tasks.

The analysis of "best" and "mean" metrics revealed a consistent pattern of superior performance by BRKGA-DFNN across both datasets. The "best" metric showcases the model's ability to achieve optimal results under ideal conditions, while the "mean" metric reflects its overall reliability across multiple runs. The BRKGA-DFNN approach demonstrated excellence in both aspects, ensuring its place as the leading model in this comparative study.

Table 6 Results of optimization models for DFNN hyperparameters: best value and average classification error for the datasets

Model	Datasets	
	MNIST	Fashion-MNIST
TS-DFNN	3.16 (best)	14.50 (best)
	4.38 (mean)	15.22 (mean)
GRASP-DFNN	2.32 (best)	12.64 (best)
	4.10 (mean)	15.28 (mean)
IPSONet	2.00 (best)	13.48 (best)
	3.66 (mean)	15.36 (mean)
HyperBRKGA	2.12 (best)	12.90 (best)
	2.97 (mean)	13.38 (mean)
BRKGA-DFNN	**1.85 (best)**	**11.46 (best)**
	2.83 (mean)	**12.29 (mean)**

These findings emphasize the effectiveness and robustness of the BRKGA-DFNN model. Its consistently delivering top-ranking results across diverse datasets highlight its potential as a reliable and efficient solution for optimizing Deep Feedforward Neural Networks.

BRKGA-CNN

After performing 30 independent runs of the BRKGA-CNN algorithm on each dataset, we recorded the best classification error rates on the respective test sets. Table 7 provides a detailed comparative analysis between BRKGA-CNN and other benchmark models. The evaluation primarily focused on performance metrics across the datasets, particularly the classification error of the optimal solution (i.e., the solution with the lowest classification error) and the average classification error observed across the 30 runs.

The psoCNN model [19] achieved the best result for the MNIST-RD dataset, while BRKGA-CNN outperformed the comparative models on MNIST, MNIST-BI, MNIST-RB, and MNIST-RD+BI. These results demonstrate that BRKGA-CNN outperformed the comparative models on these specific datasets.

For the MNIST-RD dataset, the optimal solution obtained by BRKGA-CNN was 3.60%. Although BRKGA-CNN did not surpass the best result of 3.58% achieved by psoCNN for MNIST-RD, its performance remained highly competitive. Additionally, BRKGA-CNN demonstrated superior average performance over 30 runs, with an average classification error of 4.95%, compared to 6.42% achieved by psoCNN, also based on 30 iterations. It is important to note that while psoCNN used a higher maximum output channel count of 256, BRKGA-CNN was restricted to $max_channels = 150$, which may have influenced its performance in MNIST-RD.

Table 7 BRKGA-CNN test set classification error values for the MNIST, MNIST-RD, MNIST-RB, MNIST-BI, and MNIST-RD+BI datasets compared to the comparative models

	Datasets				
Model	MNIST	MNIST-BI	MNIST-RB	MNIST-RD	MNIST-RD+BI
evoCNN (best)	1.18%	4.53%	2.80%	5.22%	35.03%
evoCNN (mean)	1.28%	4.62%	3.59%	5.46%	37.38%
IPPSO (best)	1.13%	–	–	–	34.50%
IPPSO (mean)	1.21%	–	–	–	33.00%
psoCNN (best)	0.32%	1.90%	1.79%	**3.58%**	14.28%
psoCNN (mean)	0.44%	2.40%	2.53%	6.42%	20.98%
BRKGA-CNN (best)	**0.31%**	**1.85%**	**1.50%**	3.60%	**12.45%**
BRKGA-CNN (mean)	0.51%	2.21%	2.89%	4.95%	16.43%

Table 8 Test accuracy performance comparison for architectures on the Cora dataset: state-of-the-art handcrafted architectures and best results found by NAS baselines

Categories	Methods	Cora
Handcrafted	Chebyshev	81.2%
	CGCN	81.5%
	GAT	$83.0 \pm 0.7\%$
	LGCN	$83.3 \pm 0.5\%$
GNN NAS	GraphNAS	$84.2 \pm 1.0\%$
	Auto-GNN	$83.6 \pm 0.3\%$
	SNAG	$83.5 \pm 0.5\%$
	Genetic-GNN	$83.8 \pm 0.5\%$
	BRKGA-GNN	$\mathbf{84.6 \pm 0.2}\%$

BRKGA-GNN

The experimental findings, summarized in Table 8, illustrate the performance comparison of several methods for transductive node classification on the Cora dataset. These include state-of-the-art handcrafted GNN architectures, NAS-based models, and the proposed BRKGA-GNN approach.

The results indicate that BRKGA-GNN consistently surpasses both handcrafted and NAS-based models on the Cora dataset. On the Cora dataset, BRKGA-GNN achieves a test accuracy of 84.6%, outperforming the top NAS-based method, GraphNAS, which reaches $84.2 \pm 1.0\%$, and the best-performing handcrafted model, LGCN, which records $83.3 \pm 0.5\%$. BRKGA-GNN also exhibits remarkable robustness, as evidenced by its low variance (0.2%), highlighting the stability of the proposed approach.

Thus, the BRKGA-GNN consistently achieves the highest accuracy in the benchmark dataset, showcasing its effectiveness in exploring the architectural space and identifying superior configurations. This success is mainly due to the balanced exploration and exploitation mechanisms of the BRKGA algorithm.

Conclusion and Future Research Directions

This chapter explored the role of heuristics in the design and optimization of Deep Neural Networks (DNNs), with a particular focus on Neural Architecture Search (NAS), Hyperparameter Optimization (HPO), and Learning Rule Optimization (LRO). While NAS has become an essential component in deep learning automation, most existing approaches rely on gradient-based optimization or reinforcement learning, which demand extensive computational resources and are highly sensitive to hyperparameter tuning. In contrast, our study introduced BRKGA as a computationally feasible and versatile heuristic that efficiently navigates the NAS search space while maintaining a favorable balance between exploration and

exploitation. Moreover, we emphasized the potential of heuristic-driven approaches to hyperparameter tuning and adaptive learning rule discovery, broadening the applicability of evolutionary techniques beyond architecture search.

We demonstrated the effectiveness of BRKGA in optimizing three distinct neural network paradigms—Deep Feedforward Neural Networks (DFNNs), Convolutional Neural Networks (CNNs), and Graph Neural Networks (GNNs)—across a set of moderate-sized benchmark datasets. The experimental results showed that BRKGA successfully identified competitive architectures with lower computational overhead, highlighting its potential as an alternative to mainstream NAS techniques. Furthermore, we explored how heuristic-driven optimization can enhance HPO by adaptively selecting key training parameters such as learning rates, dropout rates, and activation functions. Additionally, LRO, which focuses on optimizing training dynamics, remains an underexplored but promising avenue for improving network generalization and convergence rates. The results emphasize that evolutionary heuristics remain underutilized across NAS, HPO, and LRO, despite their robust search capabilities in high-dimensional and multi-objective optimization problems. emphasize that evolutionary heuristics remain underutilized across NAS, HPO, and LRO despite their robust search capabilities in high-dimensional and multi-objective optimization problems.

The advancements in NAS, HPO, and LRO present several open research challenges and opportunities, particularly in terms of scalability, integration with Automated Machine Learning (AutoML), and their implications for neuromorphic computing. While this study focused on moderate-sized benchmark datasets, future research should extend heuristic-driven methodologies to broader applications, addressing key issues such as computational complexity, interpretability, and hardware efficiency.

One of the primary challenges remains scalability, particularly when optimizing deep architectures in high-dimensional search spaces. Large-scale datasets demand more efficient NAS frameworks that leverage parallel and distributed computing. Evolutionary algorithms, including Genetic Algorithms (GAs), Estimation of Distribution Algorithms (EDAs), and Biased Random Key Genetic Algorithm (BRKGA), have demonstrated their potential in optimizing neural networks, but further research is required to enhance their performance in large-scale settings. Exploring hybrid approaches that integrate reinforcement learning and gradient-based optimization with evolutionary heuristics could provide new insights into improving search efficiency.

Beyond NAS, hyperparameter selection remains a crucial component of deep learning optimization. Future research should explore evolutionary hyperparameter tuning frameworks, particularly in dynamic learning environments where models must adapt to nonstationary data distributions. Metaheuristic approaches could improve the search for optimal hyperparameters across multiple training scenarios, reducing the reliance on exhaustive grid search and Bayesian optimization techniques.

Another promising direction involves the integration of NAS and HPO into AutoML frameworks. Automating model selection, hyperparameter tuning, and

learning rule discovery are essential to minimizing human intervention in deep learning optimization. Current AutoML tools, such as Auto-Keras and Google's AutoML, have already integrated NAS into their pipelines, but further enhancements could arise from hybrid heuristic approaches. However, the combination of heuristic search techniques with differentiable architecture search and reinforcement learning-based strategies could enhance the efficiency of these systems, particularly in resource-constrained environments.

The emergence of neuromorphic computing introduces further optimization challenges for NAS, HPO, and LRO. The development of spiking neural networks (SNNs) and energy-efficient deep learning models requires specialized architecture search techniques that consider hardware constraints such as latency, power consumption, and memory footprint. Evolutionary strategies have shown potential in optimizing architectures for neuromorphic platforms like Intel Loihi and IBM TrueNorth. However, further research is needed to adapt NAS and HPO for biologically inspired computing paradigms.

Another critical issue in deep learning automation is the interpretability of automatically generated architectures and training procedures. Understanding their decision-making processes becomes increasingly difficult as deep learning models grow in complexity. Explainable AI (XAI) techniques should be integrated into NAS, HPO, and LRO frameworks to ensure transparency, interpretability, and trustworthiness. Future studies should focus on post hoc analysis, feature importance estimation, and neural architecture visualization techniques to enhance human-AI collaboration.

The optimization of NAS, HPO, and LRO for real-time and edge AI applications also remains an open challenge. The increasing demand for low-latency, energy-efficient deep learning models highlights the need for hardware-aware optimization strategies. Neural networks optimized for edge devices, federated learning, and autonomous systems require search algorithms that balance accuracy, inference speed, and computational efficiency. Evolutionary heuristics could be crucial in designing models that meet these requirements.

Beyond traditional feedforward and convolutional networks, research should explore new architectures, such as GNNs and transformer-based models. Evolutionary strategies, including multi-objective optimization techniques, could be instrumental in adapting NAS, HPO, and LRO to complex graph structures and domain-specific tasks in bioinformatics, social network analysis, and scientific computing.

Final Remarks

The integration of evolutionary heuristics into NAS, HPO, and LRO represents a compelling avenue for the future of deep learning automation. As AutoML frameworks continue to evolve, incorporating efficient and adaptive search algorithms such as BRKGA could significantly enhance the scalability and robustness of neural architecture optimization. Additionally, the synergy between NAS, HPO, LRO, and neuromorphic computing opens new possibilities for biologically inspired AI

models that mimic brain-like computation while maximizing energy efficiency. Moving forward, the intersection of heuristics, AutoML, and hardware-aware optimization will likely shape the next generation of self-optimizing deep learning systems.

Cross-References

- Biased Random-Key Genetic Programming
- Genetic Algorithms
- GRASP
- Multi-start Methods
- Variable Neighborhood Descent
- Variable Neighborhood Search

Acknowledgments Ricardo Martins de Abreu Silva acknowledges the financial support from CAPES (Coordination for the Improvement of Higher Education Personnel), under Grant No. 88887.935176/2024-00, and from CNPq (National Council for Scientific and Technological Development), under Grant No. 443923/2023-0.

Competing Interest Declaration The author(s) has no competing interests to declare that are relevant to the content of this manuscript.

References

1. Akay B, Karaboga D, Akay R (2022) A comprehensive survey on optimizing deep learning models by metaheuristics. Artif Intell Rev 55:829–894. https://doi.org/10.1007/s10462-021-09992-0
2. Andrade CE, Resende MG, Zhang W, Sinha RK, Reichmann KC, Doverspike RD, Miyazawa FK (2015) A biased random-key genetic algorithm for wireless backhaul network design. Appl Soft Comput 33:150–169. https://doi.org/10.1016/j.asoc.2015.04.016. https://www.sciencedirect.com/science/article/pii/S1568494615002410
3. Ba JL, Kiros JR, Hinton GE (2016) Layer normalization. https://arxiv.org/abs/1607.06450. 1607.06450
4. Baldominos A, Saez Y, Isasi P (2020) On the automated, evolutionary design of neural networks: past, present, and future. Neural Comput Appl 32:519–545. https://doi.org/10.1007/s00521-019-04160-6
5. Bean JC (1994) Genetic algorithms and random keys for sequencing and optimization. ORSA J Comput 6(2):154–160
6. Bengio Y, Simard P, Frasconi P (1994) Learning long-term dependencies with gradient descent is difficult. IEEE Trans Neural Netw 5(2):157–166
7. Bergstra J, Bengio Y (2012) Random search for hyper-parameter optimization. J Mach Learn Res 13(2):281–305
8. Bianchi FM, Grattarola D, Livi L, Alippi C (2022) Graph neural networks with convolutional ARMA filters. IEEE Trans Pattern Analy Mach Intell 44(7):3496–3507. https://doi.org/10.1109/TPAMI.2021.3054830
9. Bishop CM (2006) Pattern recognition and machine learning. information science and statistics, 1st edn. Springer, New York

10. Blum C, Thiruvady D, Ernst AT, Horn M, Raidl GR (2019) A biased random key genetic algorithm with rollout evaluations for the resource constraint job scheduling problem. In: AI 2019: advances in artificial intelligence: 32nd Australasian joint conference, Adelaide, SA, December 2–5, 2019, Proceedings 32, Springer, pp 549–560. https://doi.org/10.1007/978-3-030-35288-2_44
11. Cai H, Zhu L, Han S (2020) Once-for-all: train one network and specialize it for efficient deployment. In: International conference on learning representations (ICLR), OpenReview. https://openreview.net/forum?id=HylxE1HKwS
12. Cai T, Luo S, Xu K, He D, Liu T, Wang L (2020) Graphnorm: a principled approach to accelerating graph neural network training. CoRR abs/2009.03294. https://arxiv.org/abs/2009.03294. 2009.03294
13. Darwish A, Hassanien AE, Das S (2020) A survey of swarm and evolutionary computing approaches for deep learning. Artif Intell Rev 53:1767–1812. https://doi.org/10.1007/s10462-019-09719-2
14. Defferrard M, Bresson X, Vandergheynst P (2016) Convolutional neural networks on graphs with fast localized spectral filtering. In: Lee D, Sugiyama M, Luxburg U, Guyon I, Garnett R (eds) Advances in neural information processing systems, vol 29. Curran Associates, Red Hook. https://proceedings.neurips.cc/paper_files/paper/2016/file/04df4d434d481c5bb723be1b6df1ee65-Paper.pdf
15. Du J, Zhang S, Wu G, Moura JMF, Kar S (2017) Topology adaptive graph convolutional networks. CoRR abs/1710.10370. http://arxiv.org/abs/1710.10370. 1710.10370
16. Elsken T, Metzen JH, Hutter F (2019) Neural architecture search: a survey. J Mach Learn Res 20(55):1–21
17. Ericsson M, Resende MGC, Pardalos PM (2002) A genetic algorithm for the weight setting problem in OSPF routing. J Comb Optim 6:299–333
18. Erzurum Cicek ZI, Kamisli Ozturk Z (2021) Optimizing the artificial neural network parameters using a biased random key genetic algorithm for time series forecasting. Appl Soft Comput 102:107091. https://doi.org/10.1016/j.asoc.2021.107091. https://www.sciencedirect.com/science/article/pii/S1568494621000144
19. Fernandes Junior FE, Yen GG (2019) Particle swarm optimization of deep neural networks architectures for image classification. Swarm Evolut Comput 49:62–74. https://doi.org/10.1016/j.swevo.2019.05.010. https://www.sciencedirect.com/science/article/pii/S2210650218309246
20. Feurer M, Hutter F (2019) Hyperparameter optimization. In: Automated Machine Learning: Methods, Systems, Challenges. Springer, Cham, pp 3–33. https://doi.org/10.1007/978-3-030-05318-5_1
21. Galván E, Mooney P (2020) Neuroevolution in deep neural networks: Current trends and future challenges. arXiv preprint https://arxiv.org/abs/2006.05415. 2006.05415v1
22. Gao H, Wang Z, Ji S (2018) Large-scale learnable graph convolutional networks. In: Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining, association for computing machinery, New York, NY, KDD '18, pp 1416–1424. https://doi.org/10.1145/3219819.3219947
23. Giusti A, Cireşan DC, Masci J, Gambardella LM, Schmidhuber J (2013) Fast image scanning with deep max-pooling convolutional neural networks. In: 2013 IEEE international conference on image processing, IEEE, pp 4034–4038
24. Gonçalves JF, De Almeida JR (2002) A hybrid genetic algorithm for assembly line balancing. J Heurist 8:629–642
25. Gonçalves JF, Resende MG (2011) Biased random-key genetic algorithms for combinatorial optimization. J Heurist 17(5):487–525
26. Gonçalves JF, Resende MG (2011) A parallel multi-population genetic algorithm for a constrained two-dimensional orthogonal packing problem. J Comb Optim 22(2):180–201. https://doi.org/10.1007/s10878-009-9282-1
27. Gupta TK, Raza K (2020) Optimizing deep feedforward neural network architecture: a tabu search based approach. Neural Process Lett 51:2855–2870

28. Guyon I, Elisseeff A (2003) An introduction to variable and feature selection. J Mach Learn Res 3:1157–1182. http://www.jmlr.org/papers/v3/guyon03a.html
29. Hamilton W, Ying Z, Leskovec J (2017) Inductive representation learning on large graphs. In: Guyon I, Luxburg UV, Bengio S, Wallach H, Fergus R, Vishwanathan S, Garnett R (eds) Advances in neural information processing systems, vol 30. Curran Associates, Newry. https://proceedings.neurips.cc/paper_files/paper/2017/file/5dd9db5e033da9c6fb5ba83c7a7ebea9-Paper.pdf
30. Han S, Pool J, Tran J, Dally WJ (2015) Learning both weights and connections for efficient neural networks. In: Cortes C, Lawrence ND, Lee DD, Sugiyama M, Garnett R (eds) Advances in neural information processing systems (NeurIPS), vol 28. Curran Associates, Newry. https://proceedings.neurips.cc/paper/2015/file/ae0eb3eed39d2bcef4622b2499a05fe6-Paper.pdf
31. He Y, Lin J, Liu Z, Wang H, Li L, Han S (2018) AMC: AutoML for model compression and acceleration on mobile devices. In: European conference on computer vision (ECCV). Springer, Berlin, pp 784–800. https://doi.org/10.1007/978-3-030-01234-2_47
32. Hinton G, Deng L, Yu D, Dahl GE, Rahman Mohamed A, Jaitly N, Senior A, Vanhoucke V, Nguyen P, Sainath T, Kingsbury B (2012) Deep neural networks for acoustic modeling in speech recognition: the shared views of four research groups. IEEE Signal Process Mag 29(6):82–97. https://doi.org/10.1109/MSP.2012.2205597
33. Hu YQ, Yu Y (2020) A technical view on neural architecture search. Int J Mach Learn Cybern 11:795–811. https://doi.org/10.1007/s13042-020-01062-1
34. Hutter F, Kotthoff L, Vanschoren J (eds) (2019) Automated machine learning: methods, systems, challenges. Springer, Berlin. https://doi.org/10.1007/978-3-030-05318-5
35. Ioffe S, Szegedy C (2015) Batch normalization: Accelerating deep network training by reducing internal covariate shift. CoRR abs/1502.03167. http://arxiv.org/abs/1502.03167. 1502.03167
36. Japa L, Serqueira M, MendonçA I, Aritsugi M, Bezerra E, González PH (2023) A population-based hybrid approach for hyperparameter optimization of neural networks. IEEE Access 11:50752–50768. https://doi.org/10.1109/ACCESS.2023.3277310
37. Kandasamy K, Neiswanger W, Schneider J, Póczos B, Xing EP (2018) Neural architecture search with Bayesian optimization and optimal transport. In: Advances in neural information processing systems (NeurIPS), vol 31. Curran Associates, Newry. https://papers.nips.cc/paper_files/paper/2018/file/370ea3896055a7f03432cf2096562a67-Paper.pdf
38. Kingma DP, Ba J (2017) Adam: A method for stochastic optimization. arXiv preprint arXiv:14126980. 1412.6980
39. Kipf TN, Welling M (2016) Semi-supervised classification with graph convolutional networks. CoRR abs/1609.02907. http://arxiv.org/abs/1609.02907. 1609.02907
40. Larochelle H, Erhan D, Courville A, Bergstra J, Bengio Y (2007) An empirical evaluation of deep architectures on problems with many factors of variation. In: Proceedings of the 24th international conference on machine learning, pp 473–480
41. LeCun Y, Boser B, Denker JS, Henderson D, Howard RE, Hubbard W, Jackel LD (1989) Backpropagation applied to handwritten zip code recognition. Neural Comput 1(4):541–551
42. LeCun Y, Denker JS, Solla SA (1989) Optimal brain damage. In: Touretzky DS (ed) Advances in neural information processing systems (NeurIPS). MIT Press, Cambridge, pp 598–605. https://proceedings.neurips.cc/paper/1989/file/6c9882bbac1c7093bd25041881277658-Paper.pdf
43. LeCun Y, Bengio Y, Hinton G (2015) Deep learning. Nature 521:436–444. https://doi.org/10.1038/nature14539
44. Leonhart PF, Spieler E, Ligabue-Braun R, Dorn M (2019) A biased random key genetic algorithm for the protein–ligand docking problem. Soft Comput 23:4155–4176. https://doi.org/10.1007/s00500-018-3065-5
45. Li Y, Liu J (2023) A survey: evolutionary deep learning. Soft Comput 27:9401–9423. https://doi.org/10.1007/s00500-023-08316-4

46. Li N, Ma L, Yu G, Xue B, Zhang M, Jin Y (2023) Survey on evolutionary deep learning: Principles, algorithms, applications, and open issues. ACM Comput Surveys 56(2):41. https://doi.org/10.1145/3603704
47. Liu H, Simonyan K, Yang Y (2019) Darts: differentiable architecture search. In: International conference on learning representations (ICLR), openReview. https://openreview.net/forum?id=S1eYHoC5FX
48. Liu Y, Sun Y, Xue B, Zhang M, Yen GG, Tan KC (2023) A survey on evolutionary neural architecture search. IEEE Trans Neural Netw Learn Syst 34(2):550–569. https://doi.org/10.1109/TNNLS.2021.3100554
49. Londe MA, Pessoa LS, Andrade CE, Resende MG (2025) Biased random-key genetic algorithms: a review. Eur J Oper Res 321(1):1–22. https://doi.org/10.1016/j.ejor.2024.03.030. https://www.sciencedirect.com/science/article/pii/S0377221724002303
50. Ma Y, Xie Y (2022) Evolutionary neural networks for deep learning: a review. Int J Mach Learn Cybern 13:3001–3018. https://doi.org/10.1007/s13042-022-01578-8
51. Marchi F, Parpinelli RS (2021) A multi-objective approach to the protein structure prediction problem using the biased random-key genetic algorithm. In: 2021 IEEE congress on evolutionary computation (CEC), pp 1070–1077. https://doi.org/10.1109/CEC45853.2021.9504745
52. Martinez C, Loiseau I, Resende M, Rodriguez S (2011) BRKGA algorithm for the capacitated arc routing problem. Electron Not Theoret Comput Sci 281:69–83. Proceedings of the 2011 latin american conference in informatics (CLEI). https://doi.org/10.1016/j.entcs.2011.11.026. https://www.sciencedirect.com/science/article/pii/S1571066111001757.
53. Mirjalili S, Lewis A, Mahdavi AS (2019) Nature-inspired optimization algorithms. Springer, Berlin
54. Morris C, Ritzert M, Fey M, Hamilton WL, Lenssen JE, Rattan G, Grohe M (2019) Weisfeiler and leman go neural: higher-order graph neural networks. Proc AAAI Confer Artif Intell 33(01):4602–4609. https://doi.org/10.1609/aaai.v33i01.33014602. https://ojs.aaai.org/index.php/AAAI/article/view/4384
55. Ojha VK, Abraham A, Snášel V (2017) Metaheuristic design of feedforward neural networks: a review of two decades of research. Engineering Appl Artif Intell 60:97–116. https://doi.org/10.1016/j.engappai.2017.01.013
56. Paszke A, Gross S, Massa F, Lerer A, Bradbury J, Chanan G, Killeen T, Lin Z, Gimelshein N, Antiga L, Desmaison A, Kopf A, Yang E, DeVito Z, Raison M, Tejani A, Chilamkurthy S, Steiner B, Fang L, Bai J, Chintala S (2019) PyTorch: an imperative style, high-performance deep learning library. In: Wallach H, Larochelle H, Beygelzimer A, d'Alché-Buc F, Fox E, Garnett R (eds) Advances in neural information processing systems, vol 32. Curran Associates, Newry. https://proceedings.neurips.cc/paper_files/paper/2019/file/bdbca288fee7f92f2bfa9f7012727740-Paper.pdf
57. Pham H, Guan MY, Zoph B, Le QV, Dean J (2018) Efficient neural architecture search via parameter sharing. In: Proceedings of the 35th international conference on machine learning (ICML), PMLR, Stockholm, proceedings of machine learning research, vol 80, pp 4092–4101. http://proceedings.mlr.press/v80/pham18a.html
58. Real E, Aggarwal A, Huang Y, Le QV (2019) Regularized evolution for image classifier architecture search. In: Proceedings of the AAAI conference on artificial intelligence, vol 33. AAAI Press, Washington, pp 4780–4789. https://doi.org/10.1609/aaai.v33i01.33014780
59. Ren P, Xiao Y, Chang X, Huang PY, Li Z, Chen X, Wang X (2021) A comprehensive survey of neural architecture search: Challenges and solutions. ACM Comput Surv 54(4):76. https://doi.org/10.1145/3447582
60. Resende MG (2012) Biased random-key genetic algorithms with applications in telecommunications. Top 20(1):130–153. https://doi.org/10.1007/s11750-011-0176-x
61. Shi M, Tang Y, Zhu X, Huang Y, Wilson D, Zhuang Y, Liu J (2022) Genetic-GNN: evolutionary architecture search for graph neural networks. Knowl-Based Syst 247:108752. https://doi.org/10.1016/j.knosys.2022.108752. https://www.sciencedirect.com/science/article/pii/S0950705122003525

62. Silva AA, Xavier AS, Silva RMA (2021) A hyperparameter search approach using BRKGA for deep feedforward neural network. In: 2021 international conference on electrical, computer, communications and mechatronics engineering (ICECCME), pp 1–6. https://doi.org/10.1109/ICECCME52200.2021.9590907
63. Silva AA, Xavier AS, Macêdo D, Zanchettin C, Oliveira AL (2022) An adapted grasp approach for hyperparameter search on deep networks applied to tabular data. In: 2022 international joint conference on neural networks (IJCNN), IEEE, pp 1–8
64. Souto G, Morais I, Faulhaber L, Ribeiro GM, Henrique González P (2021) A hybrid BRKGA approach for the two stage capacitated facility location problem. In: 2021 IEEE congress on evolutionary computation (CEC), pp 2007–2014. https://doi.org/10.1109/CEC45853.2021.9504856
65. Stanley KO, Miikkulainen R (2002) Evolving neural networks through augmenting topologies. Evol Comput 10(2):99–127. https://doi.org/10.1162/106365602320169811
66. Stanley KO, Clune J, Lehman J (2019) Designing neural networks through neuroevolution. Nat Mach Intell 1(1):24–35. https://doi.org/10.1038/s42256-018-0006-z
67. Sun Y, Xue B, Zhang M, Yen GG (2020) Evolving deep convolutional neural networks for image classification. IEEE Trans Evol Comput 24(2):394–407. https://doi.org/10.1109/TEVC.2019.2916183
68. Talbi EG (2020) Optimization of deep neural networks: a survey and unified taxonomy. HAL Open Science https://inria.hal.science/hal-02570804v2. hal-02570804
69. Ünal HT, Başçiftçi F (2022) Evolutionary design of neural network architectures: a review of three decades of research. Artif Intell Rev 55:1723–1802. https://doi.org/10.1007/s10462-021-10049-5
70. Veličković P, Cucurull G, Casanova A, Romero A, Liò P, Bengio Y (2018) Graph attention networks. https://arxiv.org/abs/1710.10903. 1710.10903
71. Walczak S, Cerpa N (1999) Heuristic principles for the design of artificial neural networks. Inf Softw Technol 41(2):107–117. https://doi.org/10.1016/S0950-5849(98)00116-5
72. Wang B, Sun Y, Xue B, Zhang M (2018) Evolving deep convolutional neural networks by variable-length particle swarm optimization for image classification. In: 2018 IEEE congress on evolutionary computation (CEC), pp 1–8. https://doi.org/10.1109/CEC.2018.8477735
73. Wu H, Gu X (2015) Max-pooling dropout for regularization of convolutional neural networks. In: Neural information processing: 22nd international conference, ICONIP 2015, Istanbul, November 9–12, 2015, proceedings, part I 22. Springer, Berlin pp 46–54
74. Wu F, Souza A, Zhang T, Fifty C, Yu T, Weinberger K (2019) Simplifying graph convolutional networks. In: Chaudhuri K, Salakhutdinov R (eds) Proceedings of the 36th international conference on machine learning, PMLR, proceedings of machine learning research, vol 97, pp 6861–6871. https://proceedings.mlr.press/v97/wu19e.html
75. Xiao H, Rasul K, Vollgraf R (2017) Fashion-MNIST: a novel image dataset for benchmarking machine learning algorithms. arXiv preprint arXiv:170807747
76. Xu K, Hu W, Leskovec J, Jegelka S (2018) How powerful are graph neural networks? CoRR abs/1810.00826. http://arxiv.org/abs/1810.00826. 1810.00826
77. Yu J, Xi L, Wang S (2007) An improved particle swarm optimization for evolving feedforward artificial neural networks. Neural Process Lett 26:217–231. https://doi.org/10.1007/s11063-007-9053-x
78. Zela A, Elsken T, Saikia T, Marrakchi Y, Brox T, Hutter F (2020) Understanding and robustifying differentiable architecture search. In: International conference on learning representations (ICLR), openReview. https://openreview.net/forum?id=H1gDNyrKDS
79. Zhang X, Zhong Y, Zhang L (2019) A survey on hyperparameter optimization for deep learning-based image analysis. IEEE Access 7:150937–150960. https://doi.org/10.1109/ACCESS.2019.2944819
80. Zhao L, Akoglu L (2019) PairNorm: tackling oversmoothing in GNNs. CoRR abs/1909.12223. http://arxiv.org/abs/1909.12223. 1909.12223
81. Zhao H, Wei L, Yao Q (2020) Simplifying architecture search for graph neural network. CoRR abs/2008.11652. https://arxiv.org/abs/2008.11652. 2008.11652

82. Zhou X, Qin AK, Gong M, Tan KC (2021) A survey on evolutionary construction of deep neural networks. IEEE Trans Evol Comput 25(5):894–914. https://doi.org/10.1109/TEVC.2021.3079985
83. Zhou K, Huang X, Song Q, Chen R, Hu X (2022) Auto-GNN: neural architecture search of graph neural networks. Front Big Data 5:1029307
84. Zoph B, Vasudevan V, Shlens J, Le QV (2018) Learning transferable architectures for scalable image recognition. In: Proceedings of the IEEE conference on computer vision and pattern recognition (CVPR), IEEE, Salt Lake City, Utah, pp 8697–8710. https://doi.org/10.1109/CVPR.2018.00907

Parallel Meta-heuristic Search

36

Teodor Gabriel Crainic

Contents

Introduction 1088
Meta-heuristics and Parallelism: A Taxonomy 1089
Decomposition Strategies: Sources of Parallelism 1091
Search Strategies 1093
Performance Measures 1095
Algorithm-Based Parallelization Strategies 1096
Explicit Search-Space Decomposition 1099
Implicit SSD: Independent Multi-search 1102
Cooperative Search 1103
Synchronous Cooperation 1107
Asynchronous Cooperation 1109
Knowledge Creation in Cooperative Search 1114
Conclusions 1122
References 1124

Abstract

This chapter presents an unified and up-to-date overview of the parallel meta-heuristics field. It synthesizes, classifies, and describes the main concepts and general parallelization strategies for meta-heuristics, including their instantiation for neighbourhood- and population-based methods. It also recalls the main contributions to the field, and identifies open questions and research challenges. The presentation is structured according to a three-dimensional classification

T. G. Crainic (✉)
School of Management, Université du Québec à Montréal, Montréal, QC, Canada

CIRRELT – Interuniversity Research Centre on Enterprise Networks, Logistics and Transportation, Montréal, QC, Canada
e-mail: TeodorGabriel.Crainic@cirrelt.net

© Springer Nature Switzerland AG 2025

R. Martí et al. (eds.), *Handbook of Heuristics*,
https://doi.org/10.1007/978-3-032-00385-0_40

of parallel meta-heuristics design strategies: the number of levels indicating whether decomposition is applied once only or recursively; the decomposition strategy reflecting the sources of parallelism in meta-heuristics, algorithm, search space, or mathematical structure; and the search strategy, given a particular level and decomposition approach, defined by the number of processes controlling the search, the communication and learning mechanism, and the diversity of the individual methods and initial solutions involved. Six major classes of parallel meta-heuristics strategies are thus discussed: low-level decomposition of computing-intensive tasks with no modification to the original algorithm, explicit decomposition of the search space, independent multi-search, as well as synchronous, asynchronous, and knowledge-creating cooperative multi-search.

Keywords

Parallel computation · Meta-heuristic search · Parallelization strategies · Algorithmic design taxonomy

Introduction

This chapter is concerned with the very broad domain of approximate solution methods, generally identified as *heuristics*, developed to address optimization problems of principally a combinatorial nature. More precisely, the chapter targets *meta-heuristics*, that is, heuristics involving at least two solution methods, one of which guides the global search for good solutions while avoiding the traps of local optima. Obviously, at least one of the solution methods involved is heuristic. The meta-heuristic term designates a very large array of neighborhood (the term "trajectory" is also found in the literature) and population-based algorithmic designs, as witnessed by this handbook and several review/synthesis books, chapters, and papers, e.g., [1,3,16,17,53,61,62,69–71,75,80,81,88,92,96,117,120]. We include *matheuristics* [91] and *hyper-heuristics* [18] into the large meta-heuristic solution-method class, but do not address heuristics based entirely on exact solution methods, such as stopping early a Branch-and-Bound search or rounding the continuous values of the optimal solution to a linear relaxation of an integer program.

Two major goals characterize the development of meta-heuristics that take advantage of parallel computing. Common to all parallel computing development efforts, the objective is to solve larger problem instances, faster. That is, address larger problem instances than what is achievable by sequential methods, and do this in reasonable computing times. A second objective characterizes approximate solution methods, and it concerns what some authors call the robustness of the method, that is, its capability to offer a consistently high level of performance over a wide variety of problem settings and instance characteristics. The *algorithmic design* of the parallel meta-heuristic search, also called *parallelization strategy*, correctly coded and implemented on an appropriate computing infrastructure, is core to achieving these objectives.

Indeed, the literature shows that, in appropriate settings, parallel meta-heuristics are more robust than sequential versions, while also generally requiring less extensive and expensive parameter-calibration efforts. The literature also emphasizes that a somewhat limited number of parallelization strategies may be identified, independently of their particular problem settings, initial sequential meta-heuristics, and implementation details. It also shows that most of these parallelization strategies yield new algorithms with behaviors different from those of their sequential counterparts.

The objective of this chapter therefore is to present a unified and up-to-date overview of the parallel meta-heuristics field. It synthesizes, classifies, and describes the main concepts and general strategies for the design of parallel meta-heuristics, including their instantiation for neighborhood- and population-based methods, recalls the main contributions to the field, and identifies a number of open questions and research challenges. The chapter focuses on the *design* of parallel meta-heuristic search algorithms, rather than on their implementation using particular coding languages and computing architectures. It does, however, identify new trends, challenges, and opportunities that some of the new computing-platform developments bring to the field.

The chapter updates and enhances material presented previously, in particular Crainic [29, 30]. The interested reader may further consult a number of surveys, taxonomies, and syntheses, e.g., [2,4,7,27–29,33–36,38,46,47,56,94,103,113,120].

The chapter is organized as follows. Section "Meta-heuristics and Parallelism: A Taxonomy" introduces an enhanced classification of design strategies for parallel meta-heuristics, together with a discussion of the potential for parallel computing in meta-heuristics and a brief description of performance indicators for parallel meta-heuristics. Section "Algorithm-Based Parallelization Strategies" addresses so-called low-level approaches focusing on accelerating computing-intensive tasks without modifying the basic algorithmic design. Methods based on the explicit decomposition of the search space are treated in section "Explicit Search-Space Decomposition," while strategies based on the simultaneous exploration of the search space by several independent meta-heuristics constitute the topic of section "Implicit SSD: Independent Multi-search." General cooperation principles are discussed in section "Cooperative Search" and are detailed in sections "Synchronous Cooperation" and "Asynchronous Cooperation." Section "Knowledge Creation in Cooperative Search" is dedicated to strategies aimed at enhancing the performance of cooperation, by taking advantage of the exchanged data to create new information relevant to both the cooperating solvers and the global search. We conclude in section "Conclusions."

Meta-heuristics and Parallelism: A Taxonomy

Parallel computing for meta-heuristics means decomposing the total task of finding the "best" solution to a given problem, that is, the global computing work associated to the search for that solution, into smaller/simpler computing *tasks*. These tasks are

then assigned to a number of available processors, to be performed concurrently (as much as possible, hopefully completely given sufficient computing resources). In the following, we refer to the *process* performing a given task (one process per task) and to the *solver*, that is, the solution method, associated to that process.

This section introduces the taxonomy used to define, describe, and analyze the parallelization strategies for meta-heuristics and the resulting main classes of parallel meta-heuristics proposed in the literature. The taxonomy unifies and enhances previous versions, in particular Crainic, Toulouse, and Gendreau [40], Crainic and Hail [34], and Crainic [29, 30]. This classification is sufficiently general to encompass the principal parallel meta-heuristic classes, while avoiding a level of detail incompatible with the scope and dimension limits of the chapter. We conclude the section with a brief discussion of performance indicators for parallel meta-heuristics.

The taxonomy is structured along three main dimensions, compactly presented in Fig. 1. The *Number of Levels* indicates whether decomposition is applied once only or recursively. Most parallel meta-heuristics found in the literature implement a *single-level decomposition*, noted *1L*, that is, the decomposition and search strategy are defined for the complete problem, yielding a number of tasks to be addressed concurrently (at each iteration of the parallel meta-heuristic). *Multiple levels* (*nL*) of decomposition are defined when one or several of those tasks may be further decomposed, the number of levels corresponding to the depth of the resulting task tree. Thus, for example, a *2L* strategy means that some initially obtained (first level) tasks are further decomposed, while a *3L* strategy implies a further decomposition of some of the second-level tasks.

The *Decomposition* dimension, detailed in section "Decomposition Strategies: Sources of Parallelism," reflects the sources of parallelism in meta-heuristics, in other words, what is decomposed in order to obtain the tasks to be addressed simul-

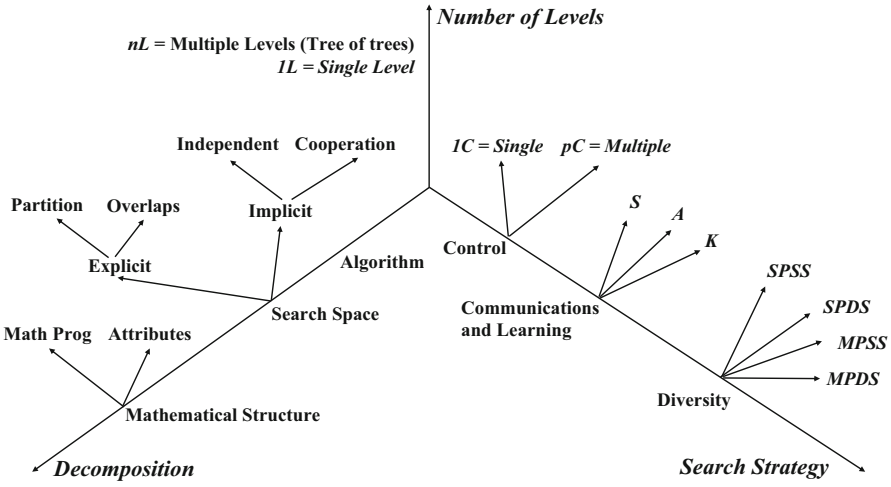


Fig. 1 Parallel meta-heuristic taxonomy

taneously. The decomposition may concern either the *Algorithm* being parallelized, or the *Search Space* one associates to the formulation, or the *Mathematical Structure* of the problem and formulation.

The *Search Strategy* dimension focuses on the main parallelization strategies for meta-heuristics, given a particular level and decomposition approach. (Notice that previous versions of the taxonomy include this dimension only, the other two dimensions being addressed separately.) Section "Search Strategies" details the three components defining a parallel meta-heuristic according to (1) *Control* of the global search with respect to the number of processes involved (one or multiple); (2) *Communication and Learning*, that is, how inter-process communications are handled, **S**ynchronously or **A**synchronously, and whether those communications are used to gather information and build search-guidance **K**nowledge; and (3) *Diversity*, i.e., how the algorithms targeting the individual tasks are defined and instantiated: **S**ame or **M**ultiple initial **P**oints and **S**ame or **D**ifferent **S**earch algorithms.

The parallel meta-heuristic design and literature synthesis of the next sections is based on this taxonomy. The characterization of the methods and contributions is captured through labels of the general form *L/Dec/C/C&L/Div*, where

L: Number of levels
Dec = Decomposition approach (see section "Decomposition Strategies: Sources of Parallelism")
- *A*: Algorithm
- *SE*: Search Space explicit decomposition
- *SI*: Search Space implicit decomposition – Independent multi-search
- *SC*: Search Space implicit decomposition – Cooperative multi-search
- *MM*: Mathematical structure decomposition – Mathematical programming
- *MA*: Mathematical structure decomposition – Attributes

Search Strategy (see section "Search Strategies"):

- *C* = *1C*, *pC*: Type of control of the search strategy
- *C&L* = *S*, *A*, *K*: Communication and learning mechanism
- *D* = *SPSS*, *SPDS*, *MPSS*, *MPDS*: Solver and starting-point diversity

Decomposition Strategies: Sources of Parallelism

Parallel/distributed/concurrent computing means that several processes work simultaneously on several processors to address a given problem instance, aiming to identify the best possible solution for that instance. Parallelism thus follows from a decomposition of the total computational load and the distribution of the resulting tasks to available processors, which run in parallel, possibly exchanging information. According to how "small" or "large" the tasks are in terms of algorithmic work or search space, the parallelization is called *fine-* or *coarse-grained*, respectively. The decomposition may concern the algorithm, the search space, or the problem and formulation mathematical structure.

Also called "functional parallelism," *algorithm*-based decomposition focuses on computing-intensive parts of the algorithm. The goal is to separate the corresponding work into a number of tasks (processes or threads), working on the same data or on dedicated parts of the data, and to allocate them to different processors. The concurrent execution of the innermost loop iterations, e.g., evaluating neighbors, computing the fitness of individuals, or having ants forage concurrently, provides the main source of functional parallelism for meta-heuristics. In fact, this is often also the only source of readily available parallelism in meta-heuristics. Indeed, the execution of most other steps in a meta-heuristic algorithm depends on the status of the search, e.g., what has been performed so far and the values of the decision variables, which requires the computation of the previous steps to be completed. Even when this may be performed in parallel, the corresponding computational tasks must be synchronized to wait for the slowest one to complete, which generally yields significant delays, making such parallel computation non-relevant. Section "Algorithm-Based Parallelization Strategies" reviews the main developments and trends based on this decomposition approach.

Search space separation (*SSD*) constitutes a second major class of decomposition strategies, as parallelism for meta-heuristics may also be found in the domain of the problem formulation or the corresponding search space (for brevity reasons and without loss of generality, the term *search space* is used in this chapter).

The fundamental idea of the *explicit* SSD (the term "domain decomposition" is also found in the literature) is to create as many tasks as the number of resulting subspaces. Each task then addresses the original problem/formulation on its particular subspace. Notice that, theoretically, the parallelism present in the search space is as large as the space itself, provided a processor is assigned to each solution. Obviously, this is not practical. Hence, a coarser decomposition is used in most cases of interest, and an exact or heuristic search method is assigned to each task.

Section "Explicit Search-Space Decomposition" reviews the main developments and trends based on this decomposition approach. The explicit SSD may proceed through a *partition* of the space, i.e., there is no intersection between any two subspaces created, or through a separation that allows *overlaps* between two or more subspaces. Both cases raise a number of issues with respect to the overall meta-heuristic search strategy, e.g., how to separate and create the tasks; how to build a complete solution out of the ones obtained on each subspace; how to allocate resources for an efficient exploration avoiding, for example, regions with poor-quality solutions. Trade-offs must be examined and decided. Thus, for example, partitioning may simplify the treatment of some of these issues, e.g., reconstruction of complete solutions, due to its assurance that no data dependencies exist between the optimization processes of the various tasks, which may therefore be fully performed in parallel, yielding unique solutions for each subspace. Partitioning may hinter exploration; however, as solution transformations (e.g., moves and combinations of individuals) performed close to subspace boundaries overlook possibilities involving solutions on the "other" side of the border. Allowing overlapping subspaces addresses this flaw, but requires a more careful construction

of a complete solution out of partial solutions involving a certain number of the same variables with possibly different values.

Implicit SSD is performed through concurrent explorations of the search space by several solvers, which could implement the same solution method but start their exploration from different points of the search space. The final complete solution to the problem at hand is then selected when all solvers have completed their explorations. Implicit SSD belongs to the larger class of *multi-search* meta-heuristics (the terms "multiple walks" is also to be found in the literature), which also includes the algorithms produced through the attribute-based decomposition of the mathematical structure of the problem and model introduced further.

Two classes of implicit SSD strategies are defined. *Independent* search involves several solvers that do not exchange any information, except at the very end when the best solution is extracted. As discussed in section "Implicit SSD: Independent Multi-search," independent search strategies are very easy to implement and may display interesting performances. They are generally outperformed, however, by well-designed and implemented parallelization strategies based on *Cooperation*. Cooperative search is based on inter-process communication and information-exchange mechanisms. A rather broad spectrum of cooperation strategies may be defined, from the simple exchange of the current best solutions, to learning mechanisms based on exchanged data that yield new information to guide the global and individual searches. Cooperative strategies are at the core of the most successful developments in parallel meta-heuristics and are the object of the last three sections (preceding the "Conclusions").

Parallelism may also be obtained by decomposing along the mathematical structure of the problem and formulation. The tasks generated through such *mathematical structure*-based decomposition strategies address partial problems (called subproblems by a number of authors) obtained either through mathematical programming or attribute-based heuristic approaches. Mathematical-programming decomposition implicitly defines how a complete solution is built out of partial ones. In the latter case, some tasks work on the partial problems corresponding to the particular sets of attributes defined in the decomposition, while others combine the resulting partial solutions into complete solutions to the original problem. As described in section "Knowledge Creation in Cooperative Search," attribute-based mathematical structure decomposition belongs to the class of multi-search strategies and is largely based on cooperation.

Search Strategies

We build upon and enhance the classification of [34], generalizing that of [40], to describe the parallel search strategies for meta-heuristics and their characterizations in terms of how the global problem-solving parallel search is controlled, how information is exchanged among processes creating, eventually, new information, and the diversity of searches involved.

Control makes up the first characteristic (called "Search Control Cardinality" in previous versions). It specifies whether the global search is controlled by a *single* process or by *multiple* processes, which may collaborate or not. The two categories are identified as *1-control (*1C*)* and *p-control (*pC*)*, respectively. A process involved in the control of the global parallel search is named *controller* or *master*. The term *collegial* is used to designate a multiple-process control with collaborating masters, that is, when the controlling masters exchange information while the search proceeds and collectively steer the search path and determine its termination moment. (Recall that this chapter addresses the design of parallel meta-heuristics, not their implementation on particular computing infrastructures, which always identifies a particular process in charge of distributing the initial work and collecting the appropriate information to determine that the search ends.)

The second search characteristic, *Communications and Learning*, addresses the issue of information exchanges among processes and the utilization of those exchanges to build new information, which may be used to control and guide the search (this characteristics was previously named "Search Control and Communications"). Two types of communications are generally identified in parallel computing: *synchronous* and *asynchronous*. In the former case, all concerned processes stop and engage in some form of communication and information exchange at moments exogenously determined (number of iterations, time intervals, specified algorithmic stages, etc.), either hard-coded or imposed by a control (master) process. In the latter case, each process is in charge of its own search and of establishing communications with other processes according to its own internal rules and status.

Learning in this context refers to the integration of mechanisms to (1) store all or part of the information exchanged, (2) derive/extract additional information from these exchanges and stored data, and (3) use this enhanced knowledge to build a global view of the status of the parallel search and guiding information for the individual searches involved and, hence, for the global search.

Three categories are defined to reflect the communication type and the learning capabilities offered by a search strategy:

- *Synchronous* (*S*) communications with no learning ("Rigid Synchronization," RS, previously)
- *Asynchronous* (*A*) communications with no learning ("Collegial," C, previously)
- *Knowledge* (*K*; replaces the previous "Knowledge Synchronous," KS, and "Knowledge Collegial," KC, categories)

More than one solution method or variant may be involved in a parallel meta-heuristic, and the methods may be meta-heuristics or exact. The third component of the Search Strategy dimension of the taxonomy describes the degree of *Diversity* of the global parallel search with respect to the nature of the methods involved and the corresponding starting solutions, the same or different in both cases (this dimension was identified as "Differentiation" in previous versions). Note that one characterizes two solvers as "different" even when based on the same methodology (e.g., Tabu Search, TS, or Genetic Algorithm, GA), provided they use different search strategies

in terms of components (e.g., neighborhoods or selection mechanism) or parameter values. The four classes are:

- *SPSS*: Same initial Point/Population, Same search Strategy
- *SPDS*: Same initial Point/Population, Different search Strategies
- *MPSS*: Multiple initial Points/Populations, Same search Strategies
- *MPDS*: Multiple initial Points/Populations, Different search Strategies

where "point" relates to neighborhood-based single-solution methods, while "population" is used for population-based ones.

Performance Measures

The traditional goal when designing parallel solution methods is to reduce the time required to "solve," exactly or heuristically, given problem instances or to address larger instances without increasing the computational effort. For exact solution methods running until the optimal solution is obtained, this translates into the well-known *speedup* performance measure, computed as the ratio between the wall-clock time required to solve the problem instance in parallel with p processors and the corresponding solution time of the best-known sequential algorithm. A somewhat less restrictive measure replaces the latter with the time of the parallel algorithm run on a single processor. See Barr and Hickman [10] and Schryen [114] for detailed discussions of this issue, including additional performance measures.

Speedup measures are difficult to define, however, when the optimal solution is not guaranteed, including when the exact method is stopped before optimality is reached, which is obviously the case for meta-heuristics, in all senses of the term. Moreover, most strategies designing parallel meta-heuristics yield solutions that are different in value, composition, or both, from those of the sequential versions (when they exist). Hence, other than the acceleration of the search and the computational efficiency given the computing power committed to it, an equally important objective for parallel meta-heuristics is to what extend they outperform their sequential counterparts in terms of solution quality or, at least, support the claim of quality through a broader exploration of the search space. In other words, the parallel method should not require a higher overall computation effort than the sequential method or should justify the effort by higher-quality solutions.

Search *robustness* is another characteristic increasingly expected of parallel heuristics. Here, robustness is defined with respect to a problem setting, in the sense of providing "equally" good solutions to a large and varied set of problem instances, without excessive calibration, neither during the initial development, nor when addressing new instances. As discussed by Crainic and Toulouse [35, 36], multi-search methods, particularly those based on cooperation, are noticeable in this respect. They display a behavior quite different from those of the sequential methods involved and offer enhanced performances compared to sequential methods and other parallelization strategies in terms of solution quality and method robustness.

They are thus generally acknowledged as proper meta-heuristics in their own right [2].

Algorithm-Based Parallelization Strategies

Functional-parallelism-based strategies, exploiting the potential for task decomposition within the inner-loop computations of meta-heuristics, aim to accelerate the search without modifying the algorithmic logic, the search space, and the behavior of the sequential meta-heuristic. Hence, the label *low level* often associated with such strategies, and their still prevalent utilization as the lowest level of hierarchical parallelization strategies, or when addressing problem settings requiring a significant part of the computing effort to be spent in inner-loop algorithmic components or in function evaluations. The utilization of "recent" computing elements increasingly ubiquitous within most computers, such as the *graphical processing units* (*GPU*), is reviving the interest in this class of strategies, as impressive reductions in computing times may be obtained.

Most low-level parallel strategies belong to the 1/A/1C/S/SPSS class and are usually implemented according to the classical *master-worker* parallel programming model (the term "slave" is also abundantly found in the literature, particularly when the task concerns computing only, without any search). Typically, the exploration is initialized from a single solution or population, and the "master" process executes the single-control sequential meta-heuristic, decomposing computation-intensive tasks into subtasks dispatched to "worker" processes. Worker subtasks are obtained from the same unique point, solution or population, and are essentially of the same type. Hence, the SPSS characterization of such strategies. Workers receive their tasks from the master, execute them in parallel, and return the results to the master, which, once all the results are in, resumes the normal logic of the sequential meta-heuristic. The master has thus a complete view and control of the search status and execution; it decides the work allocation for all other processes and initiates communications. No communications take place among worker processes. Figure 2 illustrates this strategy (for a very limited number of processes, as in all figures in the chapter).

The neighborhood-evaluation procedure of the local search heuristics, used alone or as component of neighborhood- or population-based meta-heuristics (the latter implementing advanced "schooling" for offspring) is generally parallelized according to 1/A/1C/S/SPSS designs. The master groups the neighbors into tasks and sends them to workers. Each worker then executes the neighbour evaluation and, possibly, limited exploration (a few moves only out of each received neighbour or best-valued among them) procedure on its respective part of the neighborhood, and sends back the best, or first improving, neighbour found. The master waits for all workers to terminate their computations, selects the best move, and proceeds with the search.

The appropriate decomposition granularity, that is, the size of the tasks, depends upon the particular problem and computer architecture, but is generally computa-

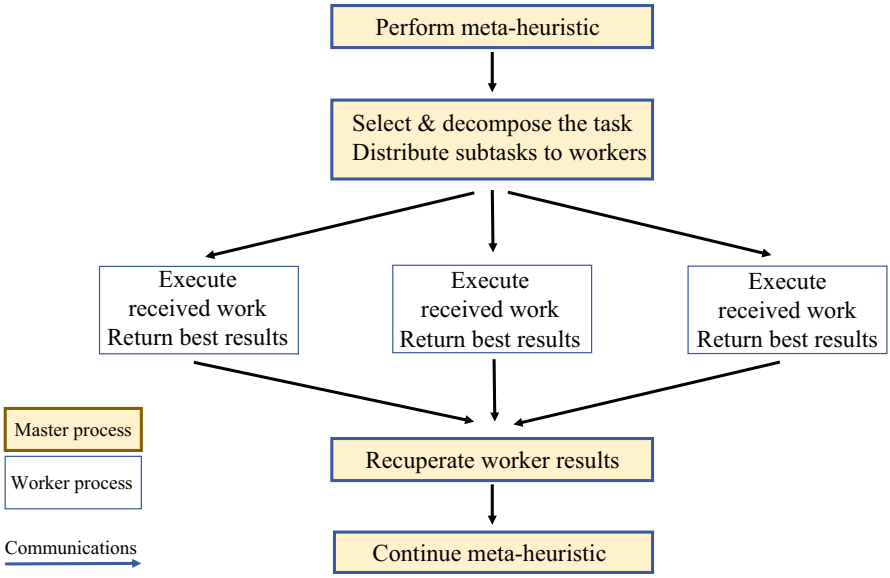


Fig. 2 Low-level 1/A/1C/S/SPSS parallelization strategy

tionally sensitive to inter-processor communication times and work-load balancing. The approach most often encountered in the literature partitions the set of neighbors, uniformly (sequentially) distributing them into subsets of same cardinality, determined by the number of processes available. Such an uniform-partition approach is appropriate when the work associated to each neighbor evaluation is similar and the processors are homogeneous. This is not always the case, however, even when processors are of the same type. The issue is illustrated in [49] for the permutation-based local search neighborhood (which performs very well within a sequential meta-heuristic) applied to the problem of scheduling dependent tasks on homogeneous processors. The dependency yields neighborhoods of unequal sizes, requiring different evaluation efforts. Computational results show that the uniform partition methods are not appropriate, the performance worsening with the amplitude of the variance in evaluation effort. A fixed coarse-grained nonuniform decomposition, based on the estimated computational effort of each subset, offers superior results, but it is not robust as it requires calibration each time the problem size or the number of processors changes. The best performing strategy, called *dynamic fine-grained* by the authors, defines each neighbor evaluation as a single task, the master dynamically dispatching these on a first-available, first-served basis to workers as they complete their tasks. The master still waits until all evaluations are completed, but the dynamic fine-grained strategy provides maximum flexibility and good load balancing.

Similar observations may be made regarding population-based meta-heuristics. In theory, all GA operators may be addressed through a 1/A/1C/S/SPSS design, and

the degree of possible parallelism is equal to the population size. In practice, the computations associated to most operators are not sufficiently heavy to warrant parallelization, while overhead costs may significantly reduce the degree of parallelism and increase the granularity of the tasks. Consequently, the fitness evaluation is often the target of algorithm-based parallelism for genetic-evolutionary methods, usually implemented using the master–worker model. The master partitions the individuals among workers, which compute and return the fitness of each individual, as well as aggregates figures to facilitate the computation by the master of the average population fitness once it receives the worker reports.

The algorithm-based parallelism for Ant-Colony Optimization (ACO) and, generally, swarm-based methods is found at the level of the individual ants. Ants share information indirectly through the pheromone matrix, which is updated once all solutions are constructed. There are no modifications of the pheromone matrix during a construction cycle and, thus, each individual ant performs its solution-construction procedure without data dependencies on the progress of the other ants. Many parallel ACO methods proposed in the literature implement some form of 1C/S/SPSS strategy according to the master-worker model (e.g., [52] and references herein). The master builds tasks consisting of small colonies of one or a few ants and distributes them to the available processors. Workers perform the construction heuristic and return their solution(s) to the master, which updates the pheromone matrix, returns it to the workers, and so on. To further speed up computation, the pheromone update can be partially computed at the worker level, each worker computing the update associated to its solutions. This fine-grained version with central matrix update outperformed the sequential version of the algorithm in most cases. It is acknowledged, however, that it does not scale when implemented on "traditional" processors (i.e., exploiting the central processing units – CPUs), and that, similarly to other meta-heuristics, it is outperformed by more advanced multi-search methods.

Scatter Search (SS) and Path Relinking (PR) implement different population-based evolution strategies, where a restricted number of elite solutions are combined, the result being enhanced through a local search or a full-fledged meta-heuristic, usually neighborhood-based. Consequently, the 1/A/1C/S/SPSS strategies discussed earlier apply straightforwardly, as in [57–59] for the p-median and the feature-selection problems. A different algorithm-based SS decomposition strategy may be obtained by running concurrently the combination and improvement operators on several subsets of solutions in the reference set. Here, the master generates tasks by extracting a number of solution subsets, which are sent to workers. Each worker then combines and improves its solutions, returning the results to the master for the global update of the reference set. Each subset sent to a worker may contain the exact or higher number of solutions required by the combination operator. In the former case, the worker performs an "iteration" of the SS algorithm [57–59]. In the latter case, several combination-improvement sequences could be executed and solutions could be returned to the master as they are found or all together at the end of all sequences. Load-balancing capabilities should be added to the master to avoid differences in work quantity and computing times between workers.

To conclude, algorithm decomposition with 1-control parallel strategies is particularly attractive when neighborhoods or populations are large, or when the neighbor or individual evaluation is costly. Computing-time gains may then be obtained, as illustrated by many early contributions to the field (discussed in the surveys indicated in the Introduction), as well as in the parallelization of the Variable Neighborhood Search (VNS) meta-heuristic proposed by [24]. The proposed parallel algorithm runs a single VNS, which executes in parallel the computations related to three steps: (1) identification of the initial solution: each process runs the local search out of a given starting solution; (2) the randomization ("shaking" in VNS vocabulary) of the solution provided by the local-search run: copies of the solution are distributed and each process performs a random modification; (3) the local search: the neighborhood evaluation is distributed. Each time, the master selects the best alternative.

Impressive gains may be obtained by taking advantage of the current computing platforms integrating multi-core central processing units (CPUs – the "traditional" processor) and more recent types of computing units, graphical processing units (GPUs), in particular. Enhanced with data streaming capabilities, the latter provide hardware data parallelism and the means for each processor to perform the same task on different (but rather small) parts of the distributed data [14, 15]. This hardware technology offers the possibility of extensive very low-level parallelization reminiscent of the work performed by the massively parallel computers of the late 1980s. Neighborhood evaluations, population fitness computation, and, even, the evolution of individuals in swarms may benefit from such a hardware-oriented parallelization, significant speedups having been observed, e.g., [14, 15, 23, 50, 56, 74, 95, 108, 121, 130]. A number of remarks are in order, however. First, the utilization of this technology is not straightforward, and work must be dedicated to its conceptual, technical, and experimental aspects (see, e.g., [93]). Second, there is also the need to examine the sequential and parallel meta-heuristic designs to identify and valuate where this technology would bring the most benefits, besides those already identified. The work of [109] is a step on this research path. Finally, as discussed in the following sections, more advanced multi-search strategies outperform low-level strategies in most cases, in particular with respect to solution quality. Consequently, hierarchical settings combining multi-search strategies and 1/A/1C/S/SPSS evaluation procedures, all on CPU-based architectures, are generally used currently. More research is needed in this area to account for the massively parallel possibilities of new types of computing units.

Explicit Search-Space Decomposition

The basic idea of this class of strategies is intuitively simple and appealing: separate the search space into smaller subspaces, address the resulting subproblems by applying the sequential meta-heuristic on each, collect the respective partial solutions, and reconstruct an entire solution out of the partial ones. This apparently simple idea may take several forms, however, according to the type of separation

performed, the permitted links among the resulting subproblems, the possible iterative modification of the separation, and the type of control of the parallel meta-heuristic.

As indicated in the taxonomy, the decomposition may yield *partition* or a *cover* of the complete search space. In the former case, the resulting subspaces are disjoint, while some overlaps are allowed among groups of subspaces in the latter. Thus, for example, the customer nodes, and the corresponding arc-design variables, of a Vehicle Routing Problem (VRP) may be partitioned into customer subsets (including the depot in each subset), while a cover would allow customers in a subset located "close by" another subset to belong to both. Note that covers may be defined implicitly by allowing the search within a given subspace to reach out to some part of one or several other subspaces through, e.g., neighborhood moves or individual crossovers. The union of the subspaces makes up the complete space in all cases.

Strict partitioning restricts the solvers to their subspaces, resulting in part of the search space being unreachable and the loss of exploration quality for the parallel meta-heuristic. Covers, through explicit or implicit overlapping, partially address this issue. Indeed, to guarantee that all potential solutions are reachable, one must make overlapping cover the entire search space, which would negate the benefits of decomposition. To avoid these drawbacks, one can change the separation and start again. This idea translates into a strategy encountered quite frequently, where the separation is modified periodically, and the search is restarted using the new decomposition. A complete-solution reconstruction feature is almost always part of the procedure. Note that this approach also provides the opportunity to define non-exhaustive separations, i.e., where the union of the subspaces is smaller than the complete search space.

This strategy is naturally implemented using master–worker L/SE/1C/S schemes, with MPSS or MPDS search differentiation (Fig. 3). The master determines the separation and sends partial subsets (or information to define them out of the initial space – this reduces the communication overhead) to workers, synchronizes them and collects their solutions, reconstructs complete solutions, modifies the separation, and determines when stopping conditions are met. Workers concurrently and independently perform the search on their assigned subsets. Most implementations in the literature address problem settings for which a large number of iterations can be performed in a relatively short time and restarting the method with a new decomposition does not require an unreasonable computational effort (see, e.g., [65] for real-time ambulance fleet management), a full-fledged meta-heuristic being generally used on each subspace.

Explicit space separation may also be performed in a multi-control decision-making framework with MPSS or MPDS search-differentiation (Fig. 4). According to an L/S/pC/S strategy, the separation is collegially decided and modified following information-exchange phases (round-robin bilateral or many-to-many exchanges) activated at given synchronization points. Solvers may exchange their best solutions only (e.g., routes in a VRP), or add the so-called context information that may be used when modifying the separation (e.g., unserviced customers and empty vehicles in a VRP [118]), respectively.

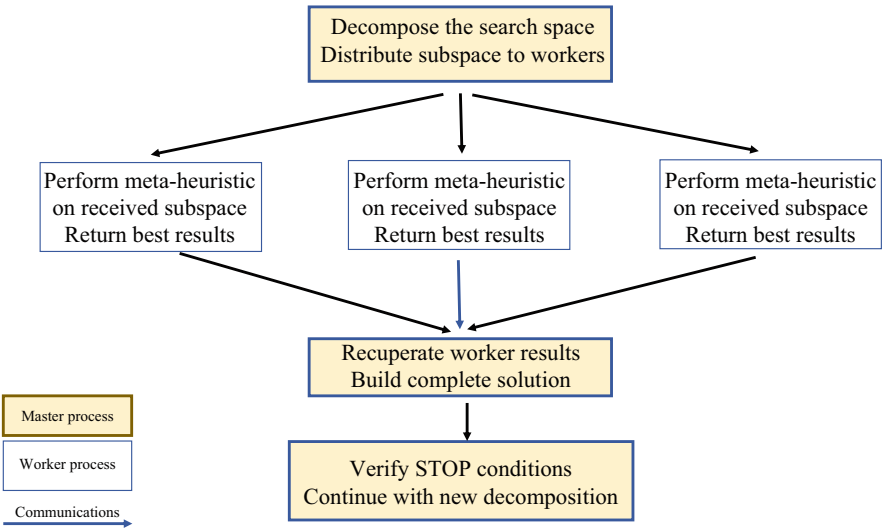


Fig. 3 Explicit search-space single-control decomposition L/SE/1C/S/ strategy

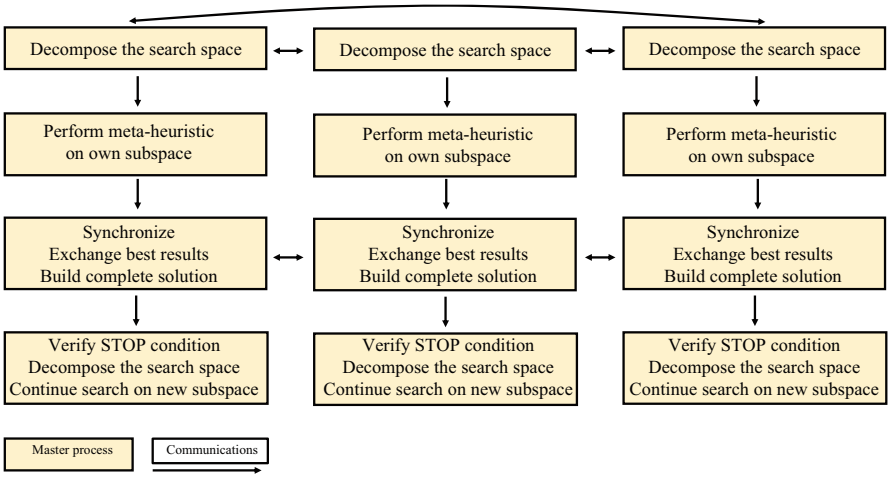


Fig. 4 Explicit search-space multiple-control decomposition L/SE/pC/S strategy

We conclude this section with two observations. First, we notice that explicit search-space decomposition corresponds to imposing restricted value ranges to all the decision variables of the formulation. In practice, this is rather performed by identifying a subset of variables, and corresponding constraints, eventually, and discarding or fixing the other variables and constraints, the goal being to obtain smaller, easier to address subproblems. However, it is not always possible, or even desirable, to discard. To illustrate, consider an explicit search-space decomposition

according to subsets of demands for the VRP and the Multicommodity Capacitated Network Design problem (MCND). VRP demand is defined at customer nodes, and the decomposition separates those into subsets. One may then easily eliminate the VRP customers that do not belong to a given subspace, as well as the network links connecting them (the depot node and the appropriate depot-customer links must be included), and solve the resulting restricted VRP. MCND demand is defined by origin-destination pairs of nodes. Partitioning the set of OD demands is straightforward. But, eliminating the OD pairs not part of a given subset, as well as the links making up the paths connecting them, would result in unconnected networks and infeasible subproblems. Separation by variable fixing (and projection of the corresponding constraints) is thus preferable. It yields the desired search-space decomposition into smaller subproblems. It is also more general, offering increased flexibility, as variables may be fixed at values other than zero. Moreover, considering the complete vector of decision variables provides the opportunity of a more thorough evaluation of solutions and possible changes.

The second observation is that explicit search-space decomposition strategies induce different search behavior and solution quality compared to those of the sequential meta-heuristic. Their performance appears dependent on how well and efficiently one avoids overlooking important regions of the search space, while purposely advancing toward good complete solutions. The effort to achieve these goals if often too high compared to the final benefits, particularly with respect to the performance of an efficient cooperation-based meta-heuristic. Hence, using explicit search-space decomposition as stand-alone strategy does not present significant interest.

On the other hand, one observes a continuous increase in the complexity of the problem settings and the dimensions of the instances one needs to address. In this context, combining search-space decomposition (through variable fixing, principally) and cooperative search appears very promising. The Integrative Cooperative Search [82] is a step in this direction (see section "Knowledge Creation in Cooperative Search").

Implicit SSD: Independent Multi-search

We dedicate a section to the *independent multi-search* search-space decomposition strategy as it was among the first to be proposed in the literature and is also the most simple and straightforward p-control parallelization strategy, generally offering an interesting performance.

Independent multi-search seeks to accelerate the exploration of the search space toward a good solution, compared to the classical multi-start sequential search. It proceeds by initiating simultaneous solvers, with or without different search strategies, from different initial points, and selecting at the end the best among the best solutions obtained by all solvers (Fig. 5).

Independent multi-search methods belong to the L/SI/pC/S class of the taxonomy, with a MPSS or MPDS diversity strategy. They do not change the behavior

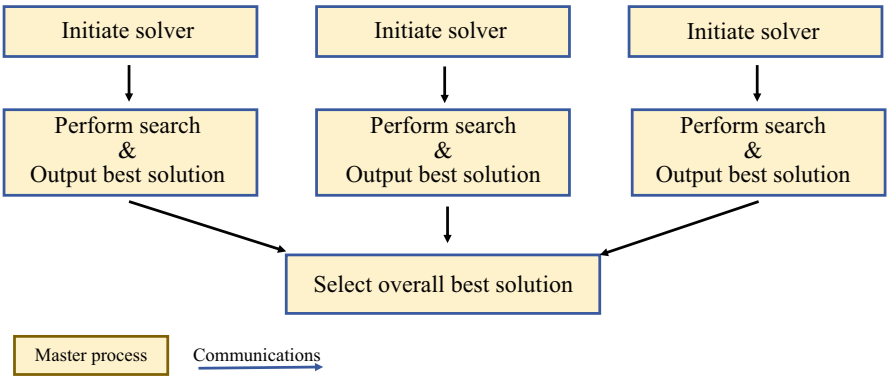


Fig. 5 Implicit search-space multiple-control decomposition L/SI/pC/S strategy

of the corresponding multi-start heuristic. No attempt is made to take advantage of the multiple solvers running in parallel other than to identify the best overall solution at the final synchronization step. The efficiency of independent multi-search follows from the sheer quantity of computing power it allows one to apply to a given problem. The references identified in the Introduction describe numerous applications of the independent multi-search strategy to a variety of combinatorial optimization problems.

Independent multi-search offers an easy access to parallel meta-heuristic computation, offering a tool when looking for a "good" solution without investment in methodological development or coding. Such methods are generally outperformed by cooperative strategies, however, through mechanisms enabling the independent solvers to share, during the search, the information their exploration generates. As explained in the following sections, this sharing and the eventual creation of new information out of the shared one yield in most cases a collective output of superior solutions compared to independent and sequential search.

Cooperative Search

Cooperative (multiple or multi) search has emerged as one of the most successful classes of meta-heuristic methodologies to address hard optimization problems. While independent multi-search seeks to accelerate the multi-start sequential heuristic, search strategies based on cooperation go further and integrate *cooperation mechanisms* to share, *while the search is in progress*, the information obtained from this diversified exploration of the same problem instance. The global search behavior of a cooperative meta-heuristic emerges from those interactions and sharing, including, eventually, the creation of new information out of the exchanged data. (The similarity between this behavior and that of systems where decisions emerge from interactions among autonomous and equal "colleagues" has inspired

the name“collegial control” associated to cooperative-search strategies in earlier versions of the taxonomy.) This behavior is different compared to those of the solvers acting individually or in sequence (as in the multi-start case). It actually makes cooperative search a “new” meta-heuristic class in its own right, which provides in many cases better performances and solutions compared to sequential and parallel independent search [37].

Cooperative-search strategies are defined by the multiple solvers engaged in cooperation (*pC* control) and the *information-sharing cooperation mechanism* defining their interactions. The solvers define trajectories in the search space from possibly different initial points or populations, by using possibly different meta-heuristic or exact-search strategies. The goals of the cooperation mechanism are to (1) improve the performance of the solvers involved, and (2) create as much as possible a “complete” image of the status of the global search to enable guiding it, through participating solvers, toward a better performance, in terms of solution quality and computational efficiency, than the simple concatenation of results obtained by non-cooperating solvers.

A list of questions related to addressing this challenge that was initially proposed by Toulouse, Crainic, and Gendreau [123], and refined in later contributions. It is still relevant today:

- What information to exchange?
- Between what processes to exchange?
- When to exchange?
- How to exchange?
- How to use/act on imported/received data?
- What knowledge to create and use out of exchanged data?

More than implementation details, answers to these questions constitute the core design parameters of a cooperative meta-heuristic. Thus, for example, a (not very good) cooperation mechanism has *all* solvers stop and *synchronize periodically*, exchange their *best local solutions*, and *restart* their individual searches from the newly-identified *best overall solution*. Cooperation mechanisms proposed in the literature to answer these questions are described in the next sections, following a number of general observations.

Exchanged information must be *meaningful* and exchanges must be *timely*. “Good” solutions make up the most often exchanged type of information, usually taking the form of the overall best solution or the current-best solution of a solver being sent to the others. Broadcasting all the new best solutions a solver identifies is counter productive in most cases, however. This is particularly true when the solver performs a series of improving moves or generations, as solutions are generally “similar” (particularly for neighborhood-based procedures), and the receiving solvers either have no chance to act on the in-coming information (unless special receiving mechanisms are embedded in all solvers) before receiving a new solution, or may embark on explorations similar to that of the sending solver. It was also observed that always broadcasting the overall best solution to all cooperating

solvers is generally bad as it rapidly decreases the diversity of the search, increasing the amount of worthless computational work (many solvers will search in the same region) and bringing an early "convergence" to a not-so-good solution. Sending out the local optima after a series of improving moves, groups of solutions, or a random selection of elite solutions, the latter biased toward good or good-and-different solutions, are among the strategies aimed at addressing these issues.

Context information may also be shared profitably when embedded in the mechanisms used to guide the search. Context information refers to data collected by a solver during its own exploration, such as the statistical information relative to the presence of particular solution elements in improving solutions (e.g., the medium and long-term Tabu Search memories), the impact of particular moves on the search trajectory (e.g., the scores of moves of a Large Adaptive Neighborhood Search), population diversity measures, individual resilience across generations, etc. A limited number of studies indicate the interest of context-information exchanges, but more research is needed on this topic.

Cooperating solvers may exchange information directly or indirectly. *Direct* exchanges of information occur either when the concerned solvers agree on a meeting point in time to share information, or when a solver broadcasts its information to one or several other solvers without prior mutual agreement. The latter case is to be avoided as it requires solvers to include capabilities to store received information without disturbing their own search trajectories until they are ready to consider it. Failure to implement such mechanisms generally results in bad performances, as observed for strategies combining uncontrolled broadcasting of information and immediate acceptance of received data.

Indirect exchanges of information are performed through independent data structures that become shared resources of data. Solvers may access them asynchronously and according to their own internal logic to post and retrieve information. Such data structures are called *blackboard* in the computer-science and artificial-intelligence vocabulary, while *memory*, *pool*, and *data warehouse* (*reference* and *elite set* are also sometimes used) are equivalent terms found in the parallel meta-heuristic literature. The term *memory* is used in this chapter.

Centralized-memory mechanisms have been used in most parallel meta-heuristic contributions. They receive, eventually process, and post information received from cooperating solvers, which, in turn, may retrieve this information independently. Distributed memory mechanisms may be contemplated, where a number of memories are interconnected, each servicing a number of solvers. Such hierarchical structures, with several layers of solvers and memories, appear interesting when a large number of processors is involved, when computations are to take place on grids or loosely coupled distributed systems, and for integrative cooperation strategies. Issues related to data availability, redundancy, and integrity must then be addressed, as well as questions relative to the balancing of workloads and the volume of information exchanged. More research is needed on this topic.

Communications proceed according to an interaction topology represented by a *communication graph* specifying the processes that may engage in direct exchanges and, thus, directing the flow of information within the cooperative search. Each

node of the graph represents a solver or a memory. Edges define pairs of solvers or a solver–memory pair, which may engage in direct communications. Figure 6 illustrates two communications graphs involving solvers only, a complete graph on the left, and a grid on the right, while communications pass through a central memory in the setting of Fig. 7. The projection of this graph on the physical interconnection topology of the computer or computers executing the parallel program is generally part of the implementation design.

When and how information is shared specifies the frequency of cooperation activities, who initiates them and when, and whether the concerned solvers must synchronize, each stopping its activities and waiting for all others to be ready, or not. These two cases correspond to the *synchronous* and *asynchronous* characteristics of the Communications and Learning component of the taxonomy and are discussed in the following sections. A general observation for both cases, however, is that exchanges should not be too frequent to avoid excessive communication overheads as well as premature "convergence" to local optima [124, 125, 127, 128].

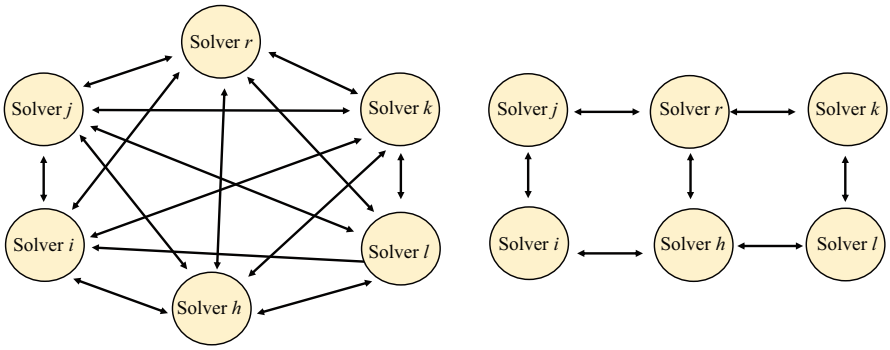


Fig. 6 Interconnection communication graphs

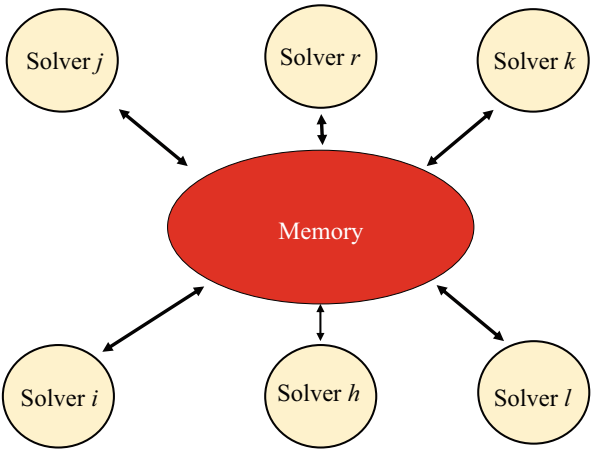


Fig. 7 Indirect memory-based communication graph

Two observations to conclude this general discussion about cooperation. First, it is worth noticing that cooperation is somewhat biased toward intensifying the search in regions of the space that have already been explored and where interesting solutions have been identified. This is particularly true for "simple" cooperation mechanisms based on synchronization or on exchanging current-best solutions only. It is thus important to equip the cooperation strategies with diversification capabilities, such as, solver "welcoming" mechanisms to adapt incoming solutions to the local search environment and status (e.g., a crossover operator applied to the incoming and local solutions [90]), probabilistic or diversity-driven selection of exchanged solutions [131], or creation of new solutions and guidance information [82].

Second, the main principles of cooperative parallelization are the same for neighborhood- and population-based meta-heuristics, even though denominations and implementation approaches may differ. We thus structure the presentation that follows based on these principles and general strategies, rather than by meta-heuristic class.

Synchronous Cooperation

Synchronous cooperation, without knowledge creation, follows a L/SC/pC/S strategy, with any of the SPDS, MPSS or MPDS search diversity approaches. We dedicate this section to recalling the main concepts of synchronous cooperation, some of which found their way into more advanced strategies, encouraging interested readers to consult the contributions indicated in the Introduction for details and references.

All the solvers involved in the cooperation stop their activities at particular moments and engage in an information-exchange phase, which must be completed before any solver can restart its exploration from that synchronization point. Synchronization moments may be determined by conditions imposed exogenously to all solvers (e.g., number of iterations from the last synchronization point) or detected by an a priori designated solver. Synchronization may use a complete communication graph or a more restricted, less densely connected topology (e.g., ring, torus, and grid graph; Fig. 6). *Global* exchanges of information among all cooperating solvers take place in the former case, while information follows a *diffusion* process through *local* exchanges among neighbouring processes in the latter. In all cases, one aims to re-create a state of complete knowledge at particular points in the global search and, thus, to hopefully guide it into a coordinated evolution toward the problem solution.

In a restricted view of the concept, most early 1/SC/pC/S cooperative search meta-heuristics based on global exchanges use a designated *master* process, which may or not include one of the participating solvers. The master manages the synchronization mechanism in a master–worker implementation. It initiates the global search starting the solvers, stops all solvers at synchronization points, gathers the information, updates the global data, verifies the termination criteria of the search and, either effectively terminates it or distributes the shared information

(a good solution, generally, the overall best solution in many cases) and sends a signal to the solvers to continue the search. For coarse-grained island (each island corresponding to a sub-population) implementations of cooperating GA methods, synchronization means the communication master initiates the *migration* operator to exchange among the islands the best or a small group of some of the best individuals in each. A similar approach is seen for ACO systems, dividing the colony into sub-colonies individually assigned to solvers, the master updating the pheromone matrix, and starting a new search phase, based on the received solver results [54]. A more sophisticated approach is proposed in [98] (and alluded to in [6]), where the master dynamically adjusts the search parameters of cooperating TS solvers according to the results each has obtained so far. The method performed well on the 0-1 Multidimensional Knapsack Problem, which is encouraging, as the idea of dynamic adjustment of the search parameters may be generalized to other problem settings and more sophisticated cooperation mechanisms.

A truer global pC/S cooperative scheme empowers solvers to initiate synchronization. Once it reaches a predetermined status, a solver sends the stopping signal, broadcasts its data (current best solution or group of solutions, in most cases), followed by similar broadcasts performed by the other solvers. Once all information is shared, each solver performs its own import procedures on the received data and proceeds with its exploration of the search space until the next synchronization event. Most synchronous coarse-grained island parallelizations of GA-based evolutionary methods fall under this category, where migration operators are applied at regular intervals. Similarly, global synchronization for ACO applications where each colony evolves its own pheromone matrix, generally means that, after a fixed number of iterations, colonies exchange elite solutions that are used to update the pheromone matrix of the receiving colony. Expanding this idea to hierarchical designs, a 2/SC/pC/S strategy for genetic methods may have the fitness computation performed at the second level through a master–worker low-level parallelization [73]. Note that the associated overhead due to the latter parallelization may became significant for larger numbers of processors.

Synchronization based on global exchanges of information generally incurs an excessive communication overhead and computing-time inefficiency as exchanges are initiated only when the slowest solver is ready to start. The overhead may further increase in hierarchical parallel contexts. A second main drawback of synchronous cooperation is that solvers relying heavily on the same information end up exploring the same search-space regions, which results in loss of diversity and favors premature "convergence" to local optima. Two approaches have been proposed to overcome this drawback.

The first approach is to avoid sharing local-best solutions only. The 1/SC/pC/S/MPDS iterated Tabu Search proposed for the VRP by [25] illustrates this approach, where solvers synchronize after a number of consecutive iterations without improvement within each individual search. Solvers then exchange a number of the good solutions obtained and, then, each individual solver builds a new starting solution by selecting routes probabilistically among those received and its own. Computational results show this method to be flexible and efficient for

several classes of routing settings with several depots, periodicity of demands, and time windows.

The second approach is based on *diffusion*, supported by sparse communication graphs (e.g., the grid in Fig. 6). Then, direct communications at synchronization points are possible only with neighboring solvers, i.e., with adjacent nodes in the graph. The quantity of information each solver processes and relies upon is thus significantly reduced. Information is still shared between non-adjacent solvers, but at the reduced diffusion speed of chains of local exchanges and data modifications by the intervening solvers. This idea was less explored compared to the global-exchange strategy, even though synchronous cooperative mechanisms based on diffusion have a less negative impact on the diversity of the exploration and have yielded good results (e.g., [97, 122]).

To conclude, synchronous-cooperation strategies generally outperform the sequential versions and independent multi-search parallelization strategies. Yet, they also display a number of inherent drawbacks. Asynchronous information sharing addresses these challenges and, indeed, cooperation strategies based on asynchronous exchanges generally outperformed synchronous methods. We discuss these strategies in the next two sections.

Asynchronous Cooperation

Historically, multi-search independent and synchronous cooperative methods were the first to be developed. The focus has shifted to asynchronous cooperation, which may be considered as defining the "state-of-the-art" in parallel multi-search meta-heuristics.

A L/SC/pC cooperation strategy, with SPDS, MPSS, or MPDS search diversity, is asynchronous when solvers initiate exchange activities according to their own internal logic, without coordination with other solvers. Thus, e.g., a solver may make available its current best solution by posting it on a memory, or may ask for an external solution when it fails to improve the quality of its best solution for a certain number of iterations. Asynchronous cooperative strategies follow either pC/A or pC/K communication-and-learning principles, the main difference being that the latter create new knowledge based on the information exchanged between solvers; pC/K strategies are addressed in the next section.

Asynchronous communications provide the means to build cooperation and information sharing among solvers without incurring the overheads associated with synchronization. They also bring adaptability to parallel cooperative meta-heuristics, which may more easily react and dynamically adapt to the exploration of the search space than independent or synchronous searches. These benefits come with potential issues one must care for, however. Two major ones are:

- The information related to the global search available to a solver performing its search may be less "complete" than in a synchronous environment; This implies

that some solvers might explore more or less the same regions. The learning mechanisms discussed in the next section help addressing this issue.
- High data-exchange frequency may be damageable; It may disrupt the work of receiving solvers and induce an erratic search behavior, similar to a random walk, particularly when the local search is altered significantly by the incoming data (e.g., the search is restarted from the received solution); Providing solver processes with mechanisms to "quarantine" incoming messages and data alleviates in part this issue, but at a generally high efficiency cost.

Hence, the interest in applying information-sharing principles based on quality, meaningfulness, and parsimony [39,40,123]. With respect to the information shared, these principles mean:

- Exchange very good solutions or individual(s) only, rather than "all" locally improving ones; Many successful contributions in the literature involve sharing local optima only;
- *Diversify* the shared information. A diversifying strategy, for example, (1) collects the very good solutions sent by the solvers into a global *elite set*, and (2) returns to a requesting solver a solution selected randomly, but biased by quality, among the elite set. Such a strategy outperformed sending always the best available solution in the elite set [39].

When to initiate and perform cooperation activities, as well as how to use the incoming information, is part of the parallel design with potentially important impact on performance. Most strategies proposed in the literature implement the same idea, to send and request information jointly. There is no need to do this, however, even though it can decrease the amount of communications. It may thus be interesting for neighbourhood-based methods to make available right away their newly found local optima or improved overall solutions and not wait for the algorithmic step when requesting and examining external information is appropriate. Similarly, population-based methods may migrate a number of individuals when a significant improvement is observed in the quality and diversity of their elite group of individuals.

Regarding the request for external information, it may be based on a a priory fixed number of iterations, but this approach should be restricted to meta-heuristics without search-diversification steps, e.g., Tabu Search based on continuous diversification. In most other cases, the principle of parsimonious communications implies selecting moments when the status of the search changes significantly, e.g., when the best solution or the elite subpopulation did not improve for a number of iterations. At such moments, solvers generally engage into some form of search *diversification* phase involving the choice of a different solution to initiate a new phase, e.g., the TS diversification based on long-term memories, the change of neighborhood in VNS, the random modification of the current local-best solution in GRASP, and the complete or partial regeneration of population in population-based meta-heuristics. External information, which generally includes at least one good solution, may

prove particularity interesting at that moment. How it is to be used depends on the particular logic of the receiving solver. It may be used, for example, to initiate a diversification phase, or to modify the search trajectory through a combination with a "local" solution, or to modify the solver behavior in the long run through an insertion into an elite set. As already mentioned, one tries to avoid frequent imports followed by a replacement of the current solution or population, which generally results in a random search.

Direct and indirect exchange pC/A strategies may be used with any meta-heuristic. Historically, most GA-based asynchronous cooperative meta-heuristics relied on direct exchanges over complete [20] or sparse (e.g., ring) [89] communication graphs. These methods generally implement a coarse-grained island model, migration being triggered by conditions within individual island populations. The selected migrant individuals are then directed toward either adjacent islands when the graph is sparse or, when the graph is complete or dense, to all other populations or a dynamically selected subset. The work of [129] illustrates the latter case, where migration is initiated by an island that identifies a new best solution, which it sends to all other islands. To enforce diversity, the migrant is accepted only when different from the local population and better than the worst individual in that population. We also mention the work of [76], who introduced genetic solvers with different strategies, which was a novelty in the GA-island field (previously, all island populations were evolved by the same algorithm), and observed significant improvements compared to more traditional island-based pC/A models. The parallelization of ACO methods may use the same approach, where partitions of the initial colony play the role of islands. The contribution of [87] is interesting in this context for a novel way of selecting the receiving subcolony (island). Here, a solver initiates an exchange when the evolution of its colony becomes stagnant (no longer improving) by selecting an exchange partner probabilistically based on the relative distance (the most different best solution) and fitness (of the best solution); it then requests the current best solution from the selected partner and, upon reception, updates its pheromone matrix and continues the search. An interesting extension of the island-cooperation idea to a 2-level strategy defines meta-heuristic-specific "archipelagos" of islands, with particular Communication-and-Learning strategies at each level [5]. The authors propose GA and ACO archipelagos with asynchronous cooperation between them, while synchronous exchanges are enforced among the islands of each.

Relatively little attention has been dedicated to direct exchanges for asynchronous cooperation among trajectory-based meta-heuristics. We mention the work of [116] reporting encouraging results for VNS-based 1/SC/pC/A strategies over uni- and bidirectional ring topologies. More research on this topic is needed.

Historically, the sharing of information in most asynchronous cooperative search strategies outside the genetic-evolutionary community is therefore based on some form of indirect communications through a centralized device, the *central memory* illustrated in Fig. 7 [27, 39, 40]. A solver involved in such a cooperation deposits good solutions, local optima generally, into the central memory, from where, when needed, it also retrieves information sent by other cooperating solvers. Classical

retrieval mechanisms are based on random selection, which may be uniform or biased to favor solutions with high rankings based on solution value and diversity. The central memory accepts incoming solutions for as long as it is not full, acceptance becoming conditional to the relative interest of the incoming solution compared to the "worst" solution in the memory, otherwise. Diversity criteria are increasingly considered, a slightly worse solution being preferred if it increases the diversity of solutions in the central memory. Population culling may also be performed (deleting, e.g., the worst half of the solutions in memory).

Central-memory-based cooperative search strategies are described in the literature for most meta-heuristic classes. To the best of our knowledge, the first central-memory approach was proposed in [39] for a 1/SC/pC/A parallel Tabu Search addressing a multi-commodity location problem with balancing requirements. In the proposed strategy, individual TSs send to the memory their local-best solutions when improved and import a solution selected probabilistically biased by rank before engaging in a diversification phase. This method outperformed, in terms of solution quality, the sequential version as well as several synchronous and broadcast-based asynchronous cooperative strategies. The same approach was applied to the fixed cost, capacitated, multicommodity network design problem with similar results [32].

Asynchronous cooperation strategies with some form of central memory were proposed for a variety of problem settings, e.g., cutting [12], container loading [13], labor-constrained scheduling [22], and VRP with time windows (VRPTW) [84]. Another set of studies focused on asynchronous cooperation with indirect communications applied to particular classes of meta-heuristics, including Simulated Annealing (SA), e.g., [9, 86, 111]; VNS, e.g., [41, 105], the latter proposing a self-adapting mechanism for the main search parameters based on recent performance, and solution selection out of the ten best present in memory; GRASP with cooperation based on applying Path Relinking to solutions from memory [107]; and TS with memory hosting a reference set and long-term global memories built on short-term local memories sent by solvers [77].

Notice that cooperating solvers need not belong to the same meta-heuristic class. The next section presents several examples where different meta-heuristics collaborate within pC/K strategies. In the classical pC/A case, we find contributions following the same broad strategy described earlier when calling sequentially on meta-heuristics belonging to different types. The two-phase approach of [60] for the VRP with Time Window (VRPTW) is a typical example of such a method, where each solver first applies an evolution strategy to reduce the number of vehicles, followed by a TS to minimize the total distance travelled. A somewhat different two-phase pC/A parallel strategy is proposed in [11] for the Steiner problem, where each phase, using Reactive TS and PR respectively, implements the pC/A asynchronous central memory strategy, all processes switching from the first to the second phase simultaneously.

Multi-level cooperative search proposes a different pC asynchronous cooperative strategy based on controlled diffusion of information [126]. Solvers are arrayed in a linear, conceptually vertical, communication graph and a local memory is associated

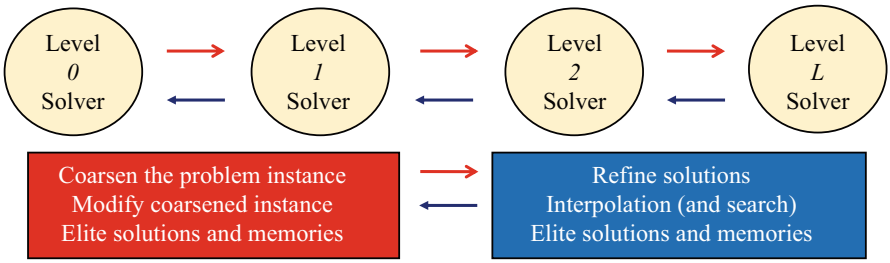


Fig. 8 Multi-level cooperation

with each. Each solver works on the original problem but at a different level of aggregation or "coarsening" of the problem data (e.g., selected network arcs being collapsed into nodes that inherit some of the arc attributes). The solver addressing the complete original problem works at the lowest aggregation level, identified in Fig. 8 as Level 0. Level L identifies the highest coarsening defined for the problem. Solvers communicate exclusively with the two solvers directly above and below, that is, at higher and lower aggregation levels, respectively (obviously, workers at levels 0 and L exchange with the next higher and lower level, respectively).

Coarsening reduces the dimensions of the problem instance at hand, which becomes easier to address. One may thus identify very good solutions faster. Moreover, one expects that a very good solution to an aggregated version of the problem will yield at least a good solution when projected on a more disaggregated problem setting. This projection process is called *refinement*. The idea of a coarsen-refine method therefore is to successively coarsen the instance until the desired resolution level and then to successively refine that solution until feasible to the original problem setting. The multi-level cooperation builds on this idea through asynchronous up and down communications among adjacent solvers, each taking advantage of the structure and quality of the solution received from the next lower or higher solver. Communications are regulated by the local memories that receive the information coming from the immediate neighbors and make it accessible to the local solver at moments dynamically determined according to its internal logic.

In the original implementation, solvers were exchanging improved solutions, incoming solutions not being transmitted further until modified locally for a number of iterations to enforce the controlled diffusion of information. Excellent results were obtained for various problem settings including graph and hypergraph partitioning [101, 102], network design [43], feature selection in biomedical data [99], and covering design [48]. It is noteworthy that one can implement multi-level cooperative search through a central memory by adequately defining the communication protocols. Although not yet fully defined and tested, this idea is interesting as it opens opportunities for richer exchange mechanisms combining controlled diffusion and general availability of global information.

Central-memory asynchronous cooperation strategies are generally offering very good results, yielding high-quality solutions. They are also computationally efficient

as no overhead is incurred for synchronization. No broadcasting is taking place, and there is no need for complex mechanisms to select the solvers that will receive or send information and to control the cooperation. It has also proved efficient in handling the issue of premature "convergence" in cooperative search, by diversifying the information received by the solvers through probabilistic selection from the memory and by a somewhat large and diverse population of solutions in the central memory; solvers may thus import different solutions even when their cooperation activities are taking place within a short time span. The central memory is thus an efficient algorithmic device that allows for a strict asynchronous mode of exchange, with no predetermined connection pattern, where no solver is interrupted by another for communication purposes, but where any solver may access at all times the data previously sent out by the other solvers.

The performance of central-memory cooperation and the availability of the exchanged information kept in the memory have brought up the question of whether one could design more advanced cooperation mechanisms taking advantage of the information exchanged among cooperating solvers. The pC/K strategies described in the next section are the result of this area of research.

Knowledge Creation in Cooperative Search

Cooperation, particularly in the central-memory asynchronous form, offers many possibilities for algorithm development. Particularly noteworthy are the flexibility in terms of the different meta-heuristic and exact methods that can be combined, and the population of elite solutions being hosted in the central memory and continuously enhanced by the cooperating solvers. One can thus select cooperating methods that complement each other, some of which heuristically construct new solutions, execute neighborhood-based improving meta-heuristics, evolve populations of solutions, or perform post-optimization procedures on solutions in memory.

The study of [31] illustrates the interest of these ideas. The authors combine a GA solver and several solvers executing the pC/A Tabu Search for multicommodity location-allocation with balancing requirements of [39]. The TSs aggressively explore the search space, building the elite solution set in the central memory, while the GA contributes toward increasing the diversity, and hopefully the quality, of the solutions in the central memory, to be imported by the cooperating TSs when needed. The GA launches with a population composed of the initial set of elite solutions generated by the TS solvers. Asynchronous migration subsequently transfers the best solution of the genetic pool to the central memory and solutions in the central memory toward the genetic population. This strategy did perform well, especially on larger instances. It also yielded the interesting observations that (1) including different types of solvers in the cooperation is beneficial, and (2) while the best overall solution might have never been found by the genetic solver, its inclusion allowed the TS solvers to find better solutions, more diversity among solutions in memory translating into a more effective diversification of the global search.

Following the studies on memory and learning preformed for Tabu Search and synthesized in [70], several contributions in the parallel meta-heuristic literature, including the pioneering [31, 110], focused on the utilization of the information exchanged among cooperating solvers to construct and continuously enhance an understanding of the history and status of the global search. The analysis of the elite population solvers build in memory, together with associated context data, becomes a *learning mechanism* providing information relative to, e.g., regions of the search space already explored and the relative quality of solutions identified in those regions, the performance of the cooperating solvers given the information received from the central memory, the behavior of critical decision variables (e.g., arc or node selection in network design) relative to good solutions, etc. This information may then be used to create new *knowledge* to guide the search. Such knowledge may include new and diverse solutions or components, "ideal" target solutions, attribute-based patterns to bias probabilistic selections of solver moves, etc.

Cooperative strategies including mechanisms to create new information belong to the *p-control, knowledge communications-and-learning* pC/K class of the taxonomy. Most contributions to this field have cooperating solvers work on the complete problem, a few being dedicated to the learning mechanism. The bulk of the section is dedicated to these strategies. We conclude the section with a discussion on developments targeting the *attribute-based decomposition of the mathematical structure* of the problem at hand, where solvers work on particular parts of the problem or on integrating the resulting partial solutions into complete ones.

Historically, two main classes of pC/K cooperative mechanisms are found in the literature, both based on the idea of exploiting a set of elite solutions exchanged by cooperating solvers working on the complete problem, but differing in the information kept in memory. *Adaptive-memory* methods [110, 119] store and score partial elements of good solutions and combine them to create new complete solutions that are then improved by the cooperating solvers. *Central-memory* methods exchange complete elite solutions among neighborhood and population-based meta-heuristics and use them to create new solutions and knowledge to guide the cooperating solvers [27, 36, 39]. Notice that the latter method generalizes the former and, thus, the two are becoming increasingly unified.

The adaptive-memory terminology for parallel meta-heuristics was coined in a paper [110] proposing TS-based heuristics for routing problems (see [8] for a thorough application of the concept to the VRPTW). More comprehensive presentations of adaptive-memory concepts, applications, and challenges may be found in [68, 119]. The main idea is to keep in memory the individual components (vehicle routes in VRP) making up the elite solutions found by the cooperating solvers, together with memories counting for each component the frequency of inclusion in the best solutions encountered so far, as well as its score and rank among the population in memory, computed in particular from the objective value of its respective solutions. Solvers construct solutions out of probabilistically selected (biased by rank) solution components in memory, enhance them (TS in the initial contribution), and deposit their best solutions in the adaptive memory.

The probabilistic selection yields, in almost all cases, a new solution made up of components from different elite solutions, thus inducing a diversification effect.

A number of early developments provided insights into algorithmic design. Applying the concept to the VRP, [115] proposes a set-covering heuristic to select from the memory the components to generate the new initial solution of a cooperating solver. At about the same time, [64] explores 2-level hierarchical strategies in the context of real-time vehicle routing and dispatching. A cooperating adaptive-memory SC/pC/K/MPSS strategy is applied at the first level, where each TS solver implements a SE/1C/S route decomposition with the help of several workers on the second level.

Generalizing the pC/A and adaptive-memory strategies, pC/K central-memory mechanisms keep full solutions, as well as attributes and context information sent by the solvers involved in cooperation (in later versions, partial solutions may also be kept). The central memory also keeps newly created information out of the exchanged data. Classical statistics-based memories recording the performance of individual solutions, solution components, and solvers provide part of this new knowledge. More advanced strategies include information extraction and creation [85], machine learning mechanisms [19], and new solution-creation procedures (population-based, generally). Search guidance mechanisms based on this knowledge may thus be gradually built.

Solvers indirectly exchange complete elite solutions and context information though the central memory. Solvers may perform constructive, improving and post-optimization heuristics [82, 84, 85], neighborhood-based methods like TS and GRASP [21, 51, 78, 79, 82], population-based methods like GA [51, 82, 84, 85], SS [104], and PR [42], as well as exact solution methods [72] on possibly restricted versions of the problem. The particular solvers to include in cooperation depend on the application. They should be efficient for the problem at hand, of course. Additionally, they should also aim to cover different regions of the search space in such a way that they contribute not only to the quality but also to the diversity of the elite population being built in the central memory.

Central-memory mechanisms thus perform two main tasks: data-warehousing and communications with solvers, on the one hand, information-creation and search-guiding, on the other hand. To distinguish between the two, we single out the latter as the *Search Coordinator* (*SC*). The simplest SC mechanism was used in the pC/A strategies of the previous section, where solutions in memory were ordered and rank-biased randomly extracted to answer solver requests. More sophisticated mechanisms are briefly described in the following.

The cooperative meta-heuristic proposed by [84] for the VRPTW uses a simple pC/K mechanism, involving four solvers, two GAs, with order and edge recombination crossovers, respectively, and two acknowledged TSs, Unified Tabu Search [26] and TABUROUTE [63]. The cooperating solvers share their improved best solutions, and request solutions as needed, i.e., the GAs for crossover operations, the Unified Tabu at regular intervals, and TABUROUTE at diversification time. The central-memory SC performs post-optimization procedures, to reduce the number of vehicles and the total traveled distance, on the received solutions before making

them available for sharing. This algorithm, without any calibration or tailoring, proved to be competitive with the best meta-heuristics of its day in linear speedups.

A general learning and guidance mechanism based on atomic network-optimization elements, arcs in particular, is proposed in [85]. The arc-based SC is independent of particular problem characteristics (e.g., routes in the original VRPTW application), and may thus be broadly applied to network-based problem settings. Starting from the classical memory concepts pioneered for TS [66, 67, 70], the authors combined two ideas: first, that an arc appearing often in good solutions and less frequently in bad solutions may be worthy of consideration for inclusion in a tentative solution, and vice versa; second, that this worthiness increases when the behavior appears stable in time. The authors thus consider the evolution of the frequency of inclusion of arcs in solutions of different quality, that is, in the elite (e.g., the 10% best), average (between the 10% and 90% best), and worst (the last 10%) groups of solutions in the central memory. *Patterns* of arcs are then defined representing subsets of arcs (not necessarily adjacent) with similar frequencies of inclusion in particular population groups. Guidance is obtained by transmitting arc patterns to individual solvers, indicating whether the arcs in the pattern should be "fixed" or "prohibited" to intensify or diversify the search, respectively. Solvers account for these instructions by using the patterns to bias the selection of arcs for move or reproduction operations. A four-phase fixed schedule (two phases of diversification at the beginning to broaden the search, followed by two intensification phases to focus the search around promising regions) was used with excellent results in terms of solution quality and computing efficiency compared to the best-performing methods of the day (see [83] for a dynamic version of this mechanism).

A different SC mechanism for a pC/K meta-heuristic with TS solvers was proposed by [79] for the VRP. Data sharing is relatively simple; solvers periodically (after a number of iterations or when the solution did not improve for a number of iterations) send best solutions to the central memory, and receive a solution back from it, the search being resumed from the received solution. The SC mechanism's objective is to identify and extract information from the solutions in memory to guide solvers toward intensification and diversification phases. This is obtained by dynamically (on reception) clustering solutions according to the number of edges in common. Thus, solutions in a given cluster share a certain number of edges, this set of edges and solutions being assumed to represent a region of the search space. Search history indicators are associated with clusters giving the number of solutions in the cluster and the quality of the solutions. This information is used to infer how thoroughly the corresponding region has been explored and how promising it might still be. Clusters are sorted according to the average solution value of their feasible solutions, and the cluster with the lowest value, that is, with the largest number of very good solutions, is selected for intensification, while the solution with the lowest number of good solutions is selected for diversification (recall that the authors were addressing a minimization optimization problem). A solution is then selected in the corresponding cluster, and it is sent to the requesting solver. Excellent results were obtained in terms of solution quality and computation effort (an almost linear

speedup was observed with up to 240 processors) compared to the state-of-the-art methods of the day.

The study of [19] focuses on the issue of flexibility and adaptability of cooperative parallel meta-heuristics. The proposed cooperation involves a number of solvers of different types exchanging through a central memory. An additional process is connected to the central memory and is in charge of the SC mechanism, collecting information from the other solvers and sending guidance instructions to modify their behavior. The SC is based on a set of fuzzy rules initially instantiated through a supervised knowledge extraction process. The rules are meant to decide when to reinitialize a solver either with the current best solution or a new set of parameter values. The former aims at changing the behavior of the currently "worst" performing solver, bringing it within a portion of the search space closer to the overall best solution. The latter also focuses on the "worst" performing solver, changing the parameter values which did not change for some time. Encouraging results were obtained on $\{0, 1\}$-knapsack problem, which indicates that more research is needed on this topic. The flexibility and adaptability issue is also at the core of the work of [100]. Set within the hyperheuristic context [17, 18], the cooperation involves three solvers based on, respectively, a GA, a SA, and an AOC. The solvers stop after a certain number of solver-specific measures and store their best solutions in the central memory. The SC mechanisms rank the solvers based on performance, e.g., computing time and solution value. The idling solver with the highest ranking then receives a new intital solution from memory and restarts its search.

The pC/K/MPDS method proposed in [72] for the VRP illustrates how specialized solvers may address different issues in a cooperative meta-heuristic, including the generation of new knowledge. Two types of solvers were defined. The so-called heuristic solvers improve solutions received from the SC associated with the central memory. On completing the task, the solvers return a given number of the best solutions found and the corresponding routes (a post-optimization procedure is first run on each route). Simultaneously, exact solvers aim at identifying new solutions by solving series of set covering problems, starting from a limited set of routes. Each time a set covering problem is solved, the solution is returned to the central memory and a predefined number of the current best solutions is retrieved for the next run. Set-covering solvers have also access to the ordered list of best routes in memory, and they select within to complete their problems. The number of routes selected to set up a set covering problem is dynamically modified during the search to control the corresponding computational effort. The method performed very well, both in terms of solution quality and computational effort (an almost-linear speedup was observed).

The inclusion of specialized solvers in cooperation is also part of the algorithmic design proposed in [112]. The cooperative algorithm involves meta-heuristic solvers to construct and improve solutions, as well as a somewhat exact solver generating new variables (column generation of partial solutions, heuristically extended to complete ones through) to further diversify the search. The SC mechanism guides

the cooperative meta-heuristic by sending solutions to the appropriate solvers to intensify or diversify the search.

The parallel GRASP algorithm proposed in [21] makes use of different information-sharing strategies at different stages of the parallel meta-heuristic. Each solver runs the same GRASP meta-heuristic, sharing not only its best solutions but also results of intermediate computations, which reduces the computing time spent performing redundant operations over diverse solvers. Thus, the data updates and corresponding evaluations performed by a solver while building its initial solution are immediately stored in the common central memory for access by all other solvers. Solvers also make immediately available their improved best solutions found during their local search phases. Finally, the parallel meta-heuristic proceeds in two phases. The first runs the solvers without external limits for a given duration. Solvers synchronize at the end of the first phase, their best solutions and context information are evaluated and aggregated, yielding a restricted set of data on which solvers will work during the second phase. The parallel meta-heuristic proved very efficient on three different problem settings.

We complete this section by reviewing developments targeting multi-attribute, "rich," problem settings displaying a large number of attributes characterizing their feasibility and optimality structures. Traditionally in the literature, such problems are simplified, or sequentially solved through a series of particular cases, where part of the overall problem is fixed or ignored, or both. The main idea of this generation of pC/K meta-heuristics is to decompose the mathematical structure of the formulation along sets of decision variables, called *decision-set attribute decomposition* in [82]. The goal of this decomposition is to obtain simpler but meaningful problem settings, in the sense that efficient solvers can be "easily" obtained for the partial problems either by opportunistically using existing high-performing methods or by developing new ones. Thus, an opportunistic rule may decompose the multi-depot periodic VRP (MDPVRP) along the depot and period decision sets to create two partial problems, a periodic VRP (PVRP) and a multi-depot VRP (MDVRP), high-quality solvers being available in the literature for both problems. The central-memory asynchronous cooperative search framework then brings together these partial problems and their associated solvers, together with integration mechanisms, reconstructing complete solutions, and search-guidance mechanisms.

According to our best knowledge, [42, 51] were the first to propose such a methodology in the context of designing wireless networks, where seven attributes were considered simultaneously. The proposed 1/MA/pC/K/MPDS meta-heuristic has TS solvers working on limited subsets of attributes, the others being fixed, and a GA combining the partial solutions generated by the TS procedures into complete solutions to the initial problem.

The general method, called *Integrative Cooperative Search* (*ICS*), was introduced in [82] (see [44, 45] for initial developments) and illustrated through an application to the MDPVRP. As illustrated in Fig. 9, the main components of ICS, which need to be instantiated for each application, are the

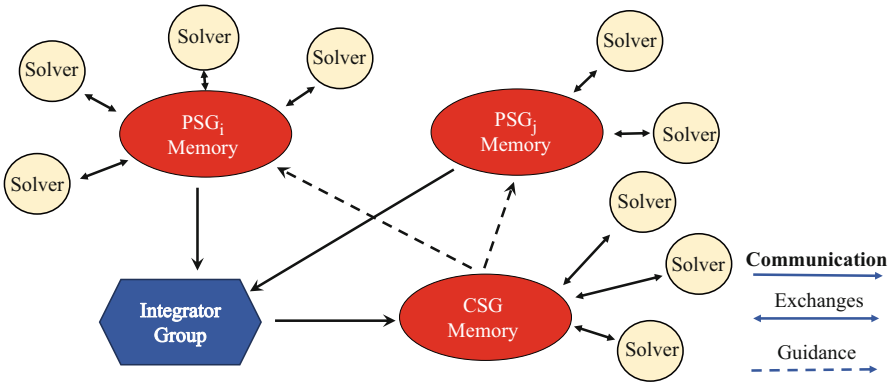


Fig. 9 Integrative cooperative search

- Decomposition rule
- *Partial Solver Groups* (*PSG*) addressing the partial problems resulting from the decomposition
- *Integrator Group* (*IG*) selecting partial solutions from PSGs, combining them, and sending the resulting complete solutions to the
- *Complete Solver Group* (*CSG*), which provides the central memory functionalities of ICS and, possibly, enhances the complete solutions.

Notice that, in order to facilitate the cooperation, a unique solution representation, obtained by fixing rather than eliminating variables when defining partial problems, is used throughout ICS. Also notice that memories associated to the PSGs and the CSG need not be distributed on different computing units (the display in Fig. 9 is for illustration only). As for all parallelization strategies discussed in this chapter, the implementation is specific to the available computing infrastructure.

A Partial Solver Group may contain one or several solvers targeting the particular partial problem assigned to the group. When several solvers are involved, the PSG is organized according to a pC/K strategy, with its own central memory keeping partial elite solutions and associated context information. The PSG's SC manages the central memory and the exchanges with the other components of the ICS, the CSG in particular. Two PSGs were defined in [82], one for the PVRP and the other for the MDVRP. The same two algorithms were used as partial solvers within each PDG, the hybrid genetic algorithm HGSADC [131] and GUTS, a generalized version of the Unified Tabu Search [26].

Integrator procedures build complete solutions by mixing partial solutions with promising features extracted from the central memories of the PSGs. The resulting complete solutions, together with critical features extracted from the partial solutions, are transmitted to the CSG. Integrators aim for solution quality and diversity, as well as computational efficiency. Because partial solutions are defined by variable fixing rather than deletion, the simplest Integrator consists of selecting high-quality

partial solutions (with respect to solution value or the inclusion of particular decision combinations) and passing them directly to the Complete Solver Group. Population-based meta-heuristics make natural integrators, as well as solvers of optimization formulations combining solutions or solution elements (e.g., set covering for VRP) to yield complete solutions to the problem at hand. The work of [55] belongs to the latter category, proposing particular optimization models for rich VRP settings, which preserve desired critical variables and attributes, present in partial solutions, when selecting and combining routes. Several Integrators can be involved in an ICS meta-heuristic, increasing the diversity of the population of complete solutions. Four Integrators were thus included in the MDPVRP application, the simple one passing good solutions to the CSG, the HGSADC individual enhancement operator (crossover and education), and two of the methods proposed by [55], the first transmitting the attributes for which there was "consensus" in the input solutions, while the second "promoted" them only through penalties added to the objective function. The last three integrators start from pairs of partial solutions randomly selected among the best 25% of the central memory populations of the two PSGs.

The Complete Solver Group includes the ICS central memory together with the SC learning, guidance, and monitoring mechanism. Complete solutions, together with the context information, are received from Integrators. When the CSG includes solvers, these solutions are further enhanced and new ones may be created. Following the concepts described earlier on in this section, knowledge is extracted from the solutions and context data received and created (e.g., the frequency of appearance of each (customer, depot, pattern) triplet in the complete solution set for the MDPVRP, together with the cost of the best solution containing it). This new information is used to build guiding instructions (solutions, patterns of variables to favour or avoid, etc.) sent to the appropriate PSGs according to a monitoring process.

Monitoring is performed by following the evolution of the PSGs (e.g., the number of improving solutions generated during a certain time period) to detect undesired situations, such as loss of diversity in the partial or complete populations, stagnation in improving the quality of the current best solution, awareness that some zones of the search space – defined by particular values for particular decision sets – have been scarcely explored, etc. Whenever one of these situations is detected, the CSG sends guidance instructions to the particular PSG. The particular type of guidance is application-specific, but one may inject new solutions or elements, modify the values of the fixed attributes for the PSG to orient its search toward a different area, change the attribute subset under investigation (i.e., change the decomposition of the decision-set attributes), or modify/replace the solution method in a Partial Solver or Integrator. The last two should not occur too frequently. Guidance in [82] takes the form of three solutions, either randomly selected from the complete solution set, or built by the CSG out of promising solution elements with respect to the search history.

Very good results are reported even when compared to the state-of-the-art meta-heuristic for the MDPVRP [82]. The experimental results also indicated that (1) one should use solvers with similar time performances in order to have them contributing

reasonably equally to the cooperation; (2) using the PSG central memory as population for all cooperating GA solvers appears beneficial for short computation runs, while creating solver-specific populations offers better performances for longer runs; (3) embedding good solvers in the CSG enhances slightly the already excellent performance of the ICS parallel meta-heuristic.

Conclusions

This chapter presented a synthesis and state-of-the-art survey of the main parallel meta-heuristic ideas, discussing main parallelization concepts and general algorithm-design principles and strategies. The presentation is structured by an enhanced three-dimensional classification of design strategies for parallel meta-heuristics: the number of levels indicating whether decomposition is applied once only or recursively; the decomposition strategy reflecting the sources of parallelism in meta-heuristics, algorithm, search space, or mathematical structure; and the search strategy, given a particular level and decomposition approach, defined by the number of processes controlling the search, the communication and learning mechanism, and the diversity of the individual methods involved and their initial solutions.

Six major classes of parallel meta-heuristics strategies were discussed: low-level decomposition of computing-intensive tasks with no modification to the original algorithm, explicit decomposition of the search space, independent multi-search, as well as synchronous, asynchronous, and knowledge-creating cooperative multi-search. It is noteworthy that this series also reflects the historical sequence of the development of parallel meta-heuristics, which are now acknowledged to make up their own class of meta-heuristics.

The literature survey reveals a rich tapestry of contributions based on one of these strategies in wide array of applications. Two observations strongly indicate, however, that more research is still needed.

First, let us not forget that most optimization problems of interest are complex and computationally difficult. Addressing these problems is increasingly challenging. On the one hand, the dimensions of the instances one faces keep increasing. On the other hand, the problem settings are becoming more complex, illustrated by the network optimization problems involving several layers of interwoven design decisions, the integrated hierarchical hub location and scheduled service network design problems, and the general trend toward explicitly addressing the uncertainty inherently present in most planning and management problems. Harnessing and developing the power of parallel meta-heuristics are crucial to efficiently address realistically dimensioned instances of such problem settings. To achieve these goals requires to study the behaviour of the parallelization strategies within each particular context and to identify the most appropriate one or combination thereof.

These developments are linked to the research needs coming from the second observation. Namely, that knowledge and expertise have not progressed equally across parallelization approaches, meta-heuristic designs, and problem settings. It

must be emphasized that each of the strategy classes identified in the chapter fulfills a particular type of task, and all are needed in some circumstances. Thus, the idea that everything seems to be known regarding low-level parallelization strategies is not true. Most studies on accelerating computing-intensive tasks targeted the evaluation of a population or neighborhood in classic meta-heuristic frameworks but, as a number of recent studies show, the best strategy to accelerate a local-search procedure may prove less effective when the local search is embedded into a full meta-heuristic or hierarchical solution method. On the other hand, the evolution of computing infrastructure, in particular, the integration of graphical processing units within computing platforms, opens up interesting but challenging perspectives. In both cases, more research is needed to understand their behavior and identify the most appropriate combination of strategies, particularly low-level and cooperative search, for various meta-heuristics, problem settings, and computing platforms.

Search-space decomposition also seems to have been thoroughly studied and has been overlooked in the last years, maybe due to the rapid and phenomenal increase in the memory available and the speed of access. Yet, the increased challenges of the evolving combinatorial optimization problem settings, alluded to above, require more research related, in particular, to dynamic search-space decomposition and the combination of cooperative search and search-space decomposition (the Integrative Cooperative Search appears as a first step in this direction).

Asynchronous cooperation, particularly when relaying on memories as communication mechanisms, provides a powerful, flexible, and adaptable framework for parallel meta-heuristics that consistently achieved good results in terms of computing efficiency and solution quality for many meta-heuristic and problem classes. A number of challenging research issues are worth investigating.

A first issue concerns the exchange and utilization of context data gathered locally by cooperating solvers, to infer the status of the global search and generate appropriate guiding instructions. Thus, contrasting and aggregating the information reflecting the various search trajectories of the cooperating solvers may be used to identify regions of the search space that were neglected or over explored. The information could also be used to evaluate the relative performance of the solvers conducting, eventually, to adjust the search parameters of particular solvers or even change the search strategy. The so-called "strategic" decision variables or parameters could thus be more easily identified, which could prove very profitable in terms of search guidance.

A related issue concerns the learning processes and the creation of new information out of the shared data. Important questions concern the identification of information that may be derived from the exchanged solutions and its usefulness in inferring the status of the global search, and determining the appropriate guiding information to be sent to solvers. Research in this direction is still at the very beginning but has already proved its worth, in particular in the context of the integrative cooperative methods.

A third broad issue concerns the cooperation of different types of meta-heuristics, as well as the cooperation of meta-heuristics with exact solution methods. The so-called hybrid, hyper, and matheuristic methods represent significant types of method

combinations in the sequential optimization field, but very few studies explicitly have targeted associated parallelization strategies. How different methods behave when involved in cooperative search and how the latter behaves given various combinations of methods is an important issue that should yield valuable insights into the design of parallel meta-heuristic algorithms, cooperative ones in particular. A particularly challenging but fascinating direction for cooperative search and ICS is represented by the multi-scenario representation of stochastic optimization formulations, for which almost nothing beyond low-level scenario-decomposition has been proposed yet. The work of [106] on parallel Benders decomposition illustrates the interest of asynchronous cooperation and the challenges of applying them for stochastic network design. Transversal studies comparing the behavior and performance of particular parallel meta-heuristic strategies over different problem classes, and of different parallel strategies and implementations for the same problem class, would be very valuable in this context, as in the broader field of parallel meta-heuristics.

Acknowledgments While working on the project, the author was Adjunct Professor, Department of Computer Science and Operations Research, Université de Montréal. The author gratefully acknowledges the financial support provided by the Natural Sciences and Engineering Council of Canada (NSERC), through its Discovery grant program, and by the Fonds de recherche du Québec through their infrastructure grants. The author is also grateful to Ms. Diane Larin for her invaluable linguistic and editorial assistance.

References

1. Aarts EHL, Korst JHM (1989) Simulated annealing and Boltzmann machines. Wiley, New York
2. Alba E (ed) (2005) Parallel metaheuristics: a new class of algorithms. Wiley, Hoboken
3. Alba E, Dorronsoro B (2008) Cellular genetic algorithms. Springer, New York
4. Almeida ALB, Lima J de C, Carvalho MAM (2022) Systematic literature review on parallel trajectory-based metaheuristics. ACM Comput Sur 55(8):1–34
5. Ammi M, Chikhi S (2015) A generalized island model based on parallel and cooperating metaheuristics for effective large capacitated vehicle routing problem solving. J Comput Inf Technol 23(2):141–155
6. Arkhipov DI, Wu D, Vu T, Regan AR (2020) A parallel genetic algorithm framework for transportation planning and logistics management. IEEE Access 8(106506)
7. Azencott R (1992) Simulated annealing parallelization techniques. Wiley, New York
8. Badeau P, Gendreau M, Guertin F, Potvin JY, Taillard E (1997) A parallel tabu search heuristic for the vehicle routing problem with time windows. Transp Res Part C Emerging Technol 5(2):109–122
9. Banos R, Ortega J, Gil C, Fernandez A, de Toro F (2013) A simulated annealing-based parallel multi-objective approach to vehicle routing problems with time windows. Exp Syst Appl 40(5):1696–1707
10. Barr RS, Hickman BL (1993) Reporting computational experiments with parallel algorithms: issues, measures, and experts opinions. ORSA J Comput 5(1):2–18
11. Bastos MP, Ribeiro CC (1999) Reactive tabu search with path-relinking for the steiner problem in graphs. In: Voß S, Martello S, Roucairol C, Osman IH (eds) Meta-heuristics 98: theory & applications. Kluwer Academic Publishers, Norwell, pp 31–36

12. Blazewicz J, Moret-Salvador A, Walkowiak R (2004) Parallel tabu search approaches for two-dimentional cutting. Parallel Process Lett 14(1):23–32
13. Bortfeldt A, Gehring H, Mack D (2003) A parallel tabu search algorithm for solving the container loading problem. Parallel Comput 29(5):641–662
14. Brodtkorb AR, Hagen TR, Schulz C, Hasle G (2013) GPU computing in discrete optimization. Part I: introduction to the GPU. EURO J Transp Logist 2(1–2):129–157
15. Brodtkorb AR, Hagen TR, Schulz C, Hasle G (2013) GPU computing in discrete optimization. Part II: survey focussed on routing problems. EURO J Transp Logist 2(1–2):159–186
16. Burke E, Kendall G (eds) (2005) Search methodologies – introductory tutorials in optimization and decision support techniques. Springer, New York
17. Burke E, Kendall G (eds) (2014) Search methodologies – introductory tutorials in optimization and decision support techniques, 2nd edn. Springer, New York
18. Burke E, Kendall G, Newall J, Hart E, Ross P, Schulenburg S (2003) Hyper-heuristics: an emerging direction in modern search technology. In: Glover F, Kochenberger GA (eds) Handbook of metaheuristics. Kluwer Academic Publishers, Norwell, pp 457–474
19. Cadenas JM, Garido MC, Muñoz E (2009) Using machine learning in a cooperative hybrid parallel strategy of metaheuristics. Inf Sci 179(19):3255–3267
20. Cantú-Paz E (2005) Theory of parallel genetic algorithms. In: Alba E (ed) Parralel Metaheuristics: a new class of algorithms. Wiley, Hoboken, pp 425–445
21. Carrabs F, Cerulli R, Mansini R, Moreschini L, Serra D (2024) Solving the set covering problem with conflicts on sets: a new parallel GRASP. Comput Oper Res 166(106620)
22. Cavalcante CBC, Cavalcante VF, Ribeiro CC, Souza MC (2002) Parallel cooperative approaches for the labor constrained scheduling problem. In: Ribeiro C, Hansen P (eds) Essays and surveys in metaheuristics. Kluwer Academic Publishers, Norwell, pp 201–225
23. Cecilia JM, Garciá JM, Nisbet A, Amos M, Ujaldón M (2013) Enhancing data parallelism for ant colony optimization on GPUs. J Parallel Distrib Comput 73(1):42–51
24. Chagas GO, Lorena LAN, dos Santos RDC, Renaud J, Coelho LC (2024) A parallel variable neighborhood search for α-neighbor facility location problem. Comput Oper Res 165(1406589)
25. Cordeau JF, Maischberger M (2012) A parallel iterated tabu search heuristic for vehicle routing problems. Comput Oper Res 39(9):2033–2050
26. Cordeau JF, Laporte G, Mercier A (2001) A unified tabu search heuristic for vehicle routing problems with time windows. J Oper Res Soc 52(8):928–936
27. Crainic TG (2005) Parallel computation, co-operation, tabu search. In: Rego C, Alidaee B (eds) Metaheuristic optimization via memory and evolution: tabu search and scatter search. Kluwer Academic Publishers, Norwell, pp 283–302
28. Crainic TG (2008) Parallel solution methods for vehicle routing problems. In: Golden BL, Raghavan S, Wasil EA (eds) The vehicle routing problem: latest advances and new challenges. Springer, New York, pp 171–198
29. Crainic TG (2018) Parallel meta-heuristic search. In: Martí R, Pardalos PM, Resende MGC (eds) Handbook of heuristics. Springer, New York, pp 809–847
30. Crainic TG (2019) Parallel metaheuristics and cooperative search. In: Gendreau M, Potvin JY (eds) Handbook of metaheuristics, 3rd edn. Springer, New York, pp 419–451
31. Crainic TG, Gendreau M (1999) Towards an evolutionary method – cooperating multithread parallel tabu search hybrid. In: Voß S, Martello S, Roucairol C, Osman, IH (eds) Meta-heuristics 98: theory & applications. Kluwer Academic Publishers, Norwell, pp 331–344
32. Crainic TG, Gendreau M (2002) Cooperative parallel tabu search for capacitated network design. J Heuristics 8(6):601–627
33. Crainic TG, Gendreau M (2021) Heuristics and metaheuristics for fixed-charge network design, chap 4. In: Crainic TG, Gendreau M, Gendron B (eds) Network design with applications in transportation and logistics. Springer, Boston, pp 91–138
34. Crainic TG, Hail N (2005) Parallel meta-heuristics applications. In: Alba E (ed) Parallel metaheuristics: a new class of algorithms. Wiley, Hoboken, pp 447–494

35. Crainic TG, Toulouse M (1998) Parallel metaheuristics. In: TG Crainic, G Laporte (eds) Fleet management and logistics. Kluwer Academic Publishers, Norwell, pp 205–251
36. Crainic TG, Toulouse M (2003) Parallel strategies for meta-heuristics. In: Glover F, Kochenberger GA (eds) Handbook in metaheuristics. Kluwer Academic Publishers, Norwell, pp 475–513
37. Crainic TG, Toulouse M (2008) Explicit and emergent cooperation schemes for search algorithms. In: Maniezzo, V, Battiti R, Watson J-P (eds) Learning and intelligent optimization. Lecture notes in computer science, vol 5315. Springer, Berlin, pp 95–109
38. Crainic TG, Toulouse M (2010) Parallel Meta-Heuristics. In: Gendreau M, Potvin JY (eds) Handbook of metaheuristics, 2nd edn. Springer, New York, pp 497–541
39. Crainic TG, Toulouse M, Gendreau M (1996) Parallel asynchronous tabu search for multicommodity location-allocation with balancing requirements. Ann Oper Res 63:277–299
40. Crainic TG, Toulouse M, Gendreau M (1997) Towards a taxonomy of parallel tabu search algorithms. INFORMS J Comput 9(1):61–72
41. Crainic TG, Gendreau M, Hansen P, Mladenović N (2004) Cooperative parallel variable neighborhood search for the p-median. J Heuristics 10(3):293–314
42. Crainic TG, Di Chiara B, Nonato M, Tarricone L (2006) Tackling electrosmog in completely configured 3G networks by parallel cooperative meta-heuristics. IEEE Wirel Commun 13(6):34–41
43. Crainic TG, Li Y, Toulouse M (2006) A first multilevel cooperative algorithm for the capacitated multicommodity network design. Comput Oper Res 33(9):2602–2622
44. Crainic TG, Crisan GC, Gendreau M, Lahrichi N, Rei W (2009) A concurent evolutionary approach for cooperative rich combinatorial optimization. In: Genetic and evolutionary computation conference – GECCO, 8–12 July 2009, Montréal. ACM, CD-ROM
45. Crainic TG, Crisan GC, Gendreau M, Lahrichi N, Rei W (2009) Multi-thread integrative cooperative optimization for rich combinatorial problems. In: The 12th international workshop on nature inspired distributed computing – NIDISC'09, 25–29 May 2009, Rome, CD-ROM
46. Crainic TG, Davidović T, Ramljak D (2014) Designing parallel meta-heuristic methods. In: Despotovic-Zrakic, M, Milutinovic, V, Belic, A (eds) High performance and cloud computing in scientific research and education. IGI Global, Hershey, pp 260–280
47. Cung VD, Martins SL, Ribeiro CC, Roucairol C (2002) Strategies for the parallel implementations of metaheuristics. In: Ribeiro C, Hansen P (eds) Essays and surveys in metaheuristics. Kluwer Academic Publishers, Norwell, pp 263–308
48. Dai C, Li B, Toulouse M (2009) A multilevel cooperative tabu search algorithm for the covering design problem. J Comb Math Comb Comput 68:35–65
49. Davidović T, Crainic TG (2015) Parallel local search to schedule communicating tasks on identical processors. Parall Comput 48:1–14
50. Delévacq A, Delisle P, Gravel M, Krajecki M (2013) Parallel ant colony optimization on graphics processing units. J Parallel Distrib Comput 73(1):52–61
51. Di Chiara B (2006) Optimum planning of 3g cellular systems: radio propagation models and cooperative parallel meta-heuristics. Ph.D. thesis, Dipartimento di ingegneria dell'innovatione, Universitá degli Studi di Lecce, Lecce, Italy
52. Doerner KF, Hartl RF, Benkner S, Lucka M (2006) Cooperative savings based ant colony optimization – multiple search and decomposition approaches. Parallel Process Lett 16(3):351–369
53. Dokeroglu T, E S, Kucukyilmaz T, Cosar A (2019) A survey on new generation metaheuristic algorithms. Comput Ind Eng 137(106040)
54. Drias H, Ibri A (2003) Parallel ACS for Weighted MAX-SAT. In: Mira J, Álvarez, J (eds) Artificial neural nets problem solving methods – Proceedings of the 7th international workconference on artificial and natural neural networks. Lecture notes in computer science, vol 2686. Springer, Heidelberg, pp 414–421
55. El Hachemi N, Crainic TG, Lahrichi N, Rei W, Vidal T (2015) Solution integration in combinatorial optimization with applications to cooperative search and rich vehicle routing. J Heuristics 21(5):663–685

56. Essaid M, Idoumghar L, Lepagnot J, Brévilliers M (2018) GPU parallelization strategies for metaheuristics: a survey. Int J Parallel Emergent Distrib Syst 34(5):497–522
57. García-López F, Melián-Batista B, Moreno-Pérez JA, Moreno-Vega JM (2003) Parallelization of the scatter search for the p-median problem. Parallel Comput 29:575–589
58. García-López F, García Torres M, Melián-Batista B, Moreno-Pérez JA, Moreno-Vega JM (2005) Parallel scatter search. In: Alba E (ed) Parallel metaheuristics: a new class of metaheuristics. Wiley, Hoboken, pp 223–246
59. García-López F, García Torres M, Melián-Batista B, Moreno-Pérez JA, Moreno-Vega JM (2006) Solving feature subset selection problem by a parallel scatter search. Eur J Oper Res 169(2):477–489
60. Gehring H, Homberger J (2002) Parallelization of a two-phase metaheuristic for routing problems with time windows. J Heuristics 8(3):251–276
61. Gendreau M, Toulouse JY Potvin (2010) Handbook of metaheuristics, 2nd edn. Springer, New York
62. Gendreau M, Toulouse JY Potvin (2019) Handbook of metaheuristics, 3rd edn. Springer, New York
63. Gendreau M, Hertz A, Laporte G (1994) A tabu search heuristic for the vehicle routing problem. Manag Sci 40(10):1276–1290
64. Gendreau M, Guertin F, Potvin JY, Taillard ÉD (1999) Tabu search for real-time vehicle routing and dispatching. Transp Sci 33(4):381–390
65. Gendreau M, Laporte G, Semet F (2001) A dynamic model and parallel tabu search heuristic for real-time ambulance relocation. Parallel Comput 27(12):1641–1653
66. Glover F (1989) Tabu search – part I. ORSA J Comput 1(3):190–206
67. Glover F (1990) Tabu search – part II. ORSA J Comput 2(1):4–32
68. Glover F (1996) Tabu search and adaptive memory programming – advances, applications and challenges. In: Barr R, Helgason R, Kennington J (eds) Interfaces in computer science and operations research. Kluwer Academic Publishers, Norwell, pp 1–75
69. Glover F, Kochenberger GA (eds) (2003) Handbook of metaheuristics. Kluwer Academic Publishers, New York
70. Glover F, Laguna M (1997) Tabu search. Kluwer Academic Publishers, Norwell
71. Goldberg DE (1989) Genetic algorithms in search, optimization and machine learning. Addison-Wesley, Reading
72. Groër C, Golden B, Wasil E (2011) A parallel algorithm for the vehicle routing problem. INFORMS J Comput 23(2):315–330
73. Hidalgo JI, Prieto M, Lanchares J, Baraglia R, Tirado F, Garnica O (2003) Hybrid parallelization of a compact genetic algorithm. In: Proceedings of the 11th uromicro conference on parallel, distributed and network-based processing, pp 449–455
74. Hijazi NM, Faris H, Aljarah I (2021) A parallel metaheuristic approach for ensemble feature selection based on multi-core architectures. Exp Syst Appl 182(115290)
75. Holland JH (1975) Adaptation in natural and artificial systems. University of Michigan Press, Ann Arbor
76. Izzo D, Rucinski M, Ampatzis C (2009) Parallel global optimisation meta-heuristics using an asynchronous island-model. In: CEC'09 – IEEE congress on evolutionary computation. IEEE, pp 2301–2308
77. James T, Rego C, Glover F (2009) A cooperative parallel tabu search algorithm for the quadratic assignment problem. Eur J Oper Res 195(3):810–826
78. Jin J, Crainic TG, Løkketangen A (2012) A parallel multi-neighborhood cooperative tabu search for capacitated vehicle routing problems. Eur J Oper Res 222(3):441–451
79. Jin J, Crainic TG, Løkketangen A (2014) A cooperative parallel metaheuristic for the capacitated vehicle routing problem. Comput Oper Res 44:33–41
80. Laguna M, Martí R (2003) Scatter search: methodology and implementations in C. Kluwer Academic Publishers, Norwell
81. Laguna M, Martinez-Gavara A, Perez-Peló S, Resende MGC (2023) 20 years of greedy randomized adaptive search procedures with path relinking. https://doi.org/10.48550/arXiv.2312.12663

82. Lahrichi N, Crainic TG, Gendreau M, Rei W, Crisan CC, Vidal T (2015) An integrative cooperative search framework for multi-decision-attribute combinatorial optimization. Eur J Oper Res 246(2):400–412
83. Le Bouthillier A (2007) Recherches coopératives pour la résolution de problèmes d'optimisation combinatoire. Ph.D. thesis, Département d'informatique et de recherche opérationnelle, Université de Montréal, Montréal
84. Le Bouthillier A, Crainic TG (2005) A cooperative parallel meta-heuristic for the vehicle routing problem with time windows. Comput Oper Res 32(7):1685–1708
85. Le Bouthillier A, Crainic TG, Kropf P (2005) A guided cooperative search for the vehicle routing problem with time windows. IEEE Intell Syst 20(4):36–42
86. Lee SY, Lee KG (1996) Synchronous and asynchronous parallel simulated annealing with multiple Markov chains. IEEE Trans Parallel Distrib Syst 7(10):993–1007
87. Ling C, Hai-Ying S, Shu W (2012) A parallel ant colony algorithm on massively parallel processors and its convergence analysis for the travelling salesman problem. Inf Sci 199: 31–42
88. Lopes Silva MA, de Souza SR, Souza MJF, de França Filho MF (2018) Hybrid metaheuristics and multi-agent systems for solving optimization problems: a review of frameworks and a comparative analysis. Applied Soft Comput 71:433–459
89. Luque G, Alba E (eds) (2011) Parallel genetic algorithyms: theory and real world applications. Springer, New York
90. Luque G, Luna F, Alba E, Nesmachnow S (2011) Exploring the accuracy of a parallel cooperative model for trajectory-based metaheuristics. In: Moreno-Díaz R, Pichler F, Arencibia AQ (eds) Computer aided systems theory – EUROCAST 2011 Part I. Lecture notes in computer science, vol 6927. Springer, pp 319–326
91. Maniezzo V, Boschetti MA, Stützle T (2021) Matheuristics – algorithms and implementations. Springer, New York
92. Martí R, Pardalos PM, Resende MGC (eds) (2018) Handbook of heuristics. Springer, New York
93. Mehdi M, Loukil L, Bendjoudi A, Melab N (2013) Parallel GPU-accelerated Metaheuristics. In: Couturier R (ed) Designing scientific applications on GPUs, Chapman and Hall/CRC, pp 202–216
94. Melab N, Talbi EG, Cahon S, Alba E, Luque G (2006) Parallel Metaheuristics: models and frameworks. In: EL-Ghazali Talbi (ed) Parallel combinatorial optimization. Wiley, New York, pp 149–162
95. Melab N, Luong TV, Boufaras K, Talbi EG (2011) Towards ParadisEO-MO-GPU: a framework for GPU-based local search metaheuristics. In: Cabestany J, Rojas I, Joya G (eds) Advances in computational intelligence. Lecture notes in computer science, vol 6691. Springer, New York, pp 401–408
96. Michalewicz Z (1992) Genetic algorithms + Data structures = Evolution programs. Springer, Berlin
97. Middendorf M, Reischle F, Schmeck H (2002) Multi colony ant algorithms. J Heuristics 8(3):305–320
98. Niar S, Fréville A (1997) A Parallel Tabu Search Algorithm For The 0–1 Multidimensional Knapsack Problem. In: 11th international parallel processing symposium (IPPS '97), Geneva. IEEE, pp 512–516
99. Oduntan IO, Toulouse M, Baumgartner R, Bowman C, Somorjai R, Crainic TG (2008) A Multilevel tabu search algorithm for the feature selection problem in biomedical data sets. Comput Math Appl 55(5):1019–1033
100. Oteiza PP, Rodríguez A, Brignole NB (2018) Parallel Cooperative optmization through hyperheuristics. In: Eden MR, Ierapetritou M, Towler GP (eds) Proceedings of the 13th international symposium on process systems engineering – PSE 2018. Elsevier, pp 201–225
101. Ouyang M, Toulouse M, Thulasiraman K, Glover F, Deogun JS (2000) Multi-level cooperative search: application to the netlist/hypergraph partitioning problem. In: Proceedings of international symposium on physical design. ACM Press, pp 192–198

102. Ouyang M, Toulouse M, Thulasiraman K, Glover F, Deogun JS (2002) Multilevel cooperative search for the circuit/hypergraph partitioning problem. IEEE Trans Comput Aided Des 21(6):685–693
103. Pedemonte M, Nesmachnow S, Cancela H (2011) A survey of parallel ant colony optimization. Appl Soft Comput 11(8):5181–5197
104. Penas DR, Henriques D, González P, Doallo R, Saez-Rodriguez J, Banga JR (2017) A Parallel metaheuristic for large mixed-integer dynamic optimization problems, with applications in computational biology. PLoS One 12(8):207–218
105. Polacek M, Benkner S, Doerner KF, Hartl RF (2008) A cooperative and adaptive variable neighborhood search for the multi depot vehicle routing problem with time windows. Bus Res 1(2):207–218
106. Rahmaniani R, Crainic TG, Gendreau M, Rei W (2024) An asynchronous parallel benders decomposition method for stochastic network design problems. Comput Oper Res 162 (106459)
107. Ribeiro CC, Rosseti I (2007) Efficient parallel cooperative implementations of GRASP heuristics. Parallel Comput 33(1):21–35
108. Rico-Garcia H, Sanchez-Romero JL, Gomis HM, Rao RzV (2020) Parallel implementation of metaheuristics for optimizing tool path computation on CNC machining. Comput Ind 123(103322)
109. Rios E, Ochi LS, Bœres C, Cœlho VN, Cœlho IM, Faria R (2017) Exploring parallel multi-GPU local search strategies in a metaheuristic framework. J Parallel Distrib Comput. https://doi.org/10.1016/j.jpdc.2017.06.011
110. Rochat Y, Taillard ÉD (1995) Probabilistic diversification and intensification in local search for vehicle routing. J Heuristics 1(1):147–167
111. Sanvicente-Sánchez H, Frausto-Solís J (2002) MPSA: a methodology to parallelize simulated annealing and its application to the traveling salesman problem. In: Coello Coello C, de Albornoz A, Sucar L, Battistutti O (eds) MICAI 2002: Advances in artificial intelligence. Lecture notes in computer science, vol 2313. Springer, Heidelberg, pp 89–97
112. Saviniec L, Santos MO, dos Santos LMR (2004) Pattern-based models and a cooperative parallel netaheuristic for high school timetabling problems. Eur J Oper Res 280(3):1064–10813
113. Schryen G (2020) Parallel Computational optimization in operations research: a new integrative framework, literature review and research directions. Eur J Oper Res 287(1):1–18
114. Schryen G (2024) Speedup and efficiency of computational parallelization: a unifying approach and asymptotic analysis. J Parallel Distrib Comput 187(104835)
115. Schulze J, Fahle T (1999) A parallel algorithm for the vehicle routing problem with time window constraints. Ann Oper Res 86:585–607
116. Sevkli M, Aydin ME (2007) Parallel variable neighbourhood search algorithms for job shop scheduling problems. IMA J Manag Math 18(2):117–133
117. Shami TM, El-Saleh AA, Alswaitti M, Al-Tashi Q, Summakieh MA, Mirjalili S (2022) Particle swarm optimization: a comprehensive survey. IEEE Access 10:10031–10061
118. Taillard ÉD (1993) Parallel iterative search methods for vehicle routing problems. Networks 23(8):661–673
119. Taillard ÉD, Gambardella LM, Gendreau M, Potvin JY (1997) Adaptive memory programming: a unified view of metaheuristics. Eur J Oper Res 135(1):1–10
120. Talbi EG (ed) (2009) Metaheuristics: from design to implementation. Wiley, Hoboken
121. Tan Y, Ding K (2016) A survey on GPU-based implementation of swarm intelligence algorithms. IEEE Trans Cybern 46(9):2168–2267
122. Tongcheng G, Chundi M (2002) Radio network using coarse-grained parallel genetic algorithms with different neighbor topology. In: Proceedings of the 4th world congress on intelligent control and automation, vol 3, pp 1840–1843
123. Toulouse M, Crainic TG, Gendreau M (1996) Communication issues in designing cooperative multi thread parallel searches. In: Osman IH, Kelly JP (eds) Meta-heuristics: theory & applications. Kluwer Academic Publishers, Norwell, pp 501–522

124. Toulouse M, Crainic TG, Sansó B, Thulasiraman K (1998) Self-organization in cooperative search algorithms. In: Proceedings of the 1998 IEEE international conference on systems, man, and cybernetics. Omnipress, Madisson, pp 2379–2385
125. Toulouse M, Crainic TG, Sansó B (1999) An experimental study of systemic behavior of cooperative search algorithms. In: Voß S, Martello S, Roucairol C, Osman IH (eds) Metaheuristics 98: theory & applications. Kluwer Academic Publishers, Norwell, pp 373–392
126. Toulouse M, Thulasiraman K, Glover F (1999) Multi-Level Cooperative Search: A New Paradigm for Combinatorial Optimization and an Application to Graph Partitioning. In: Amestoy P, Berger P, Daydé M, Duff I, Frayssé V, Giraud L, Ruiz D (eds) 5th international euro-par parallel processing conference. Lecture notes in computer science, vol 1685. Springer, Heidelberg, pp 533–542
127. Toulouse M, Crainic TG, Thulasiraman K (2000) Global optimization properties of parallel cooperative search algorithms: a simulation study. Parallel Comput 26(1):91–112
128. Toulouse M, Crainic TG, Sansó B (2004) Systemic behavior of cooperative search algorithms. Parallel Comput 30(1):57–79
129. Vallada E, Ruiz R (2009) A cooperative metaheuristics for the permutation flowshop scheduling problem. Eur J Oper Res 193(2):365–376
130. Van Luong T, Melab N, Talbi EG (2013) GPU computing for parallel local search metaheuristic algorithms. IEEE Trans Comput 62(1):173–185
131. Vidal T, Crainic TG, Gendreau M, Lahrichi N, Rei W (2012) A hybrid genetic algorithm for multi-depot and periodic vehicle routing problems. Oper Res 60(3):611–624

Time Complexity Analysis of Stochastic Search Algorithms

37

Pietro S. Oliveto

Contents

Introduction . . . 1132
Computational Complexity of Stochastic Search Algorithms . . . 1134
Evolutionary Algorithms . . . 1135
  Conclusions . . . 1138
Test Functions . . . 1139
Tail Inequalities . . . 1142
Artificial Fitness Levels (AFL) . . . 1144
  AFL: Upper Bounds . . . 1145
  AFL: Lower Bounds . . . 1150
  Level-Based Analysis of Non-elitist Populations . . . 1152
  Conclusions . . . 1156
Drift Analysis . . . 1157
  Additive Drift Theorem . . . 1157
  Multiplicative Drift Theorem . . . 1159
  Variable Drift Theorem . . . 1162
  Negative-Drift Theorem . . . 1164
  Conclusions . . . 1166
Final Overview . . . 1166
Cross-References . . . 1167
References . . . 1168

P. S. Oliveto (✉)
Department of Computer Science and Engineering, Southern University of Science and Technology, Shenzhen, China
e-mail: olivetop@sustech.edu.cn

© Springer Nature Switzerland AG 2025

R. Martí et al. (eds.), *Handbook of Heuristics*,
https://doi.org/10.1007/978-3-032-00385-0_35

Abstract

Theoretical analyses of stochastic search algorithms, albeit few, have always existed since these algorithms became popular. Starting in the 1990s, a systematic approach to analyze the performance of stochastic search heuristics has been put in place. This quickly increasing basis of results allows, nowadays, the analysis of sophisticated algorithms such as population-based evolutionary algorithms, ant colony optimization and artificial immune systems. Results are available concerning problems from various domains including classical combinatorial and continuous optimization, single- and multi-objective optimization, and noisy and dynamic optimization as well as more advanced applications in emerging technologies such as automated algorithm configuration, automated algorithm design, and genetic programming. This chapter introduces the mathematical techniques that are most commonly used in the runtime analysis of stochastic search heuristics in discrete search spaces. Careful attention is given to the very popular artificial fitness levels and drift analysis techniques for which several variants are presented. To aid the reader's comprehension of the presented mathematical methods, these are applied to the analysis of simple evolutionary algorithms for artificial example functions. The chapter is concluded by providing references to more complex applications and further extensions of the techniques for the obtainment of advanced results.

Keywords

Stochastic search algorithms · Computational complexity · Runtime analysis · Evolutionary algorithms · (1+1) EA · Drift analysis · Artificial fitness levels · Functions of unitation · Tail inequalities

Introduction

Stochastic search algorithms, also called randomized search heuristics, are general-purpose optimization algorithms that are often used when it is not possible to design a specific algorithm for the problem at hand. Common reasons are the lack of available resources (e.g., enough money and/or time) or because of an insufficient knowledge of the complex optimization problem which has not been studied extensively before. Other times, the only way of acquiring knowledge about the problem is by evaluating the quality of candidate solutions.

Well-known stochastic search algorithms are random local search (▸ Chap. 17, "Theory of Local Search") and simulated annealing. Other more complicated approaches are inspired by processes observed in nature. Popular examples are evolutionary algorithms (EAs) (▸ Chap. 6, "Evolutionary Algorithms") inspired by the concept of natural evolution, ant colony optimization (ACO) (▸ Chap. 19, "Ant Colony Optimization: A Component-Wise Overview") inspired by ant foraging

behavior, and artificial immune systems (AIS) inspired by the immune system of vertebrates.

The main advantage of stochastic search heuristics is that, being general-purpose algorithms, they can be applied to a wide range of applications without requiring hardly any knowledge of the problem at hand. Also, the simplicity for which they can be applied implies that practitioners can use them to find high-quality solutions to a wide variety of problems without needing skills and knowledge of algorithm design. Indeed, numerous applications report high-performance results which make them widely used in practice. However, through experimental work and applications, it is difficult to understand the reasons for these successes. In particular, given a stochastic search algorithm, it is unclear on which kind of problems it will achieve good *performance* and on which it will perform poorly. Even more crucial is the lack of understanding of how the parameter settings influence the performance of the algorithms. The goal of a rigorous theoretical foundation of stochastic search algorithms is to answer questions of this nature by *explaining* the success or the failure of these methods in practical applications. The benefits of a theoretical understanding are threefold: (a) guiding the choice of the best algorithm for the problem at hand, (b) determining the optimal parameter settings (see also Chap. "Automatic Tuning of Parameters"), and (c) aiding the algorithm design, ultimately leading to the achievement of better algorithms.

Theoretical studies of stochastic optimization methods have always existed, albeit few, since these algorithms became popular. In particular, the increasing popularity gained by evolutionary (▸ Chap. 6, "Evolutionary Algorithms") and genetic algorithms (▸ Chap. 22, "Genetic Algorithms") in the 70s led to various attempts at building a theory for these algorithms. However, such initial studies attempted to provide insights on the *behavior* of evolutionary algorithms rather than estimating their performance. The most popular of these theoretical frameworks was probably the *schema theory* introduced by Holland [39] and made popular by Goldberg [33]. In the early 90s, a very different approach appeared to the analysis of evolutionary algorithms and consequently randomized search heuristics in general, driven by the insight that these heuristics are indeed randomized algorithms, albeit general-purpose ones, and as such they should be analyzed in a similar spirit to that of classical randomized algorithms [58, 59]. For the last 35 years, this field has kept growing considerably, and nowadays several advanced and powerful tools have been devised that allow the analysis of the performance of involved stochastic search algorithms for problems from various domains. These include problems from classical combinatorial and continuous optimization, dynamic optimization, and noisy optimization. The generality of the developed techniques has allowed their application to the analyses of several families of stochastic search algorithms including evolutionary algorithms, local search, metropolis, simulated annealing, ant colony optimization, artificial immune systems, particle swarm optimization (▸ Chap. 28, "Particle Swarm Optimization"), and estimation of distribution algorithms (▸ Chap. 20, "Estimation of Distribution Algorithms"). Recently, they have also been applied to the analysis of emerging technologies such as algorithm

configurators, automated algorithm design (hyper-heuristics) (▸ Chap. 24, "Hyper--heuristics"), and the evolution of computer programs (genetic programming).

The aim of this chapter is to introduce the reader to the most common and powerful tools used in the performance analysis of randomized search heuristics. Since the main focus is the understanding of the methods, these will be applied to the analysis of very simple evolutionary algorithms for artificial example functions. The hope is that the structure of the functions and the behavior of the algorithms are easy to grasp so the attention of the reader may be mostly focused on the mathematical techniques that will be presented. At the end of the chapter, references to complex applications of the techniques for the obtainment of advanced results will be pointed out for further reading.

Computational Complexity of Stochastic Search Algorithms

From the perspective of computer science, stochastic search heuristics are randomized algorithms although more general than problem-specific ones. Hence, it is natural to analyze their performance in the classical way as done in computer science. From this perspective, an algorithm should be *correct*, i.e., for every instance of the problem (the input), the algorithm halts with the correct solution (i.e., the correct output), and it should be efficient in terms of its *computational complexity* i.e., the algorithm uses the computational resources wisely. The resources usually considered are the number of basic computations to find the solution (i.e., time) and the amount of memory required (i.e., space).

Differently from problem-specific algorithms, the goal behind general-purpose algorithms such as stochastic search heuristics is to deliver good performance independently of the problem at hand. In other words, a general-purpose algorithm is "correct" if it *visits the optimal solution* of *any problem* in finite time. If the optimum is never lost afterward, then a stochastic search algorithm is said to *converge* to the optimal solution. In a formal sense, the latter condition for convergence is required because most search heuristics are not capable of recognizing when an optimal solution has been found (i.e., they do not halt). It has been known for over two decades that a few relatively simple conditions suffice to ensure convergence [69]. Hence, what is more relevant and can make a huge difference on the usefulness of a stochastic search heuristic for a given problem is its *time complexity*. In each iteration, the evaluation of the quality of a solution is generally far more expensive than its other algorithmic steps. As a result, it is very common to measure *time* as the number of evaluations of the fitness function (also called objective function) rather than counting the number of basic computations. Since randomized algorithms make random choices during their execution, the runtime of a stochastic search heuristic A to optimize a function f is a random variable $T_{A,f}$. The main measure of interest is:

1. The *expected runtime* $\mathbb{E}\left[T_{A,f}\right]$: The expected number of fitness function evaluations until the optimum of f is found.

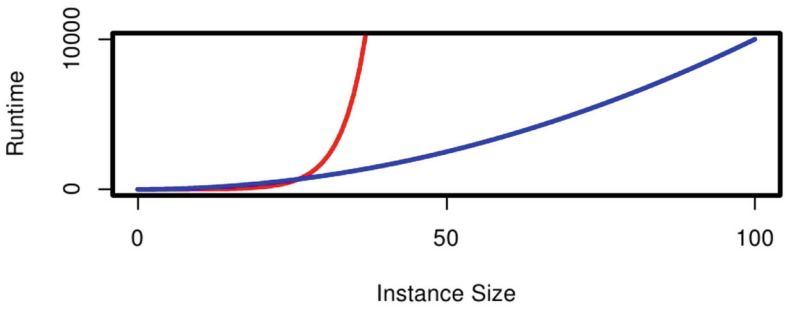


Fig. 1 An efficient search heuristic (blue) versus an inefficient search heuristic (red) for a given instance

For skewed runtime distributions, the expected runtime may be a deceiving measure of algorithm performance. The following measure therefore provides additional information.

2. The *success probability in t steps* $\Pr\left(T_{A,f} \leq t\right)$: The probability that the optimum is found within t steps.

Just like in the classical theory of efficient algorithms, the time is analyzed in relation to growing input length and usually described using asymptotic notation [4]. A search heuristic A is said to be *efficient* for a function (class) f if the runtime grows as polynomial function of the instance size. On the other hand, if the runtime grows as an exponential function, then the heuristic is said to be *inefficient*. See Fig. 1 for an illustrative distinction.

Evolutionary Algorithms

A general framework of a *steady-state* genetic algorithm is the (μ+λ) GA defined in Algorithm 1. The algorithm evolves a population of μ candidate solutions, generally called the *parent population*. At each generation, an *offspring population* of λ individuals is created. Each of the λ individuals is generated by selecting two individuals from the parent population uniformly at random and by applying a *uniform crossover* operator to create an offspring to which a *standard bit mutation* operator is then applied. The generation is concluded by selecting the μ fittest individuals out of the $\mu + \lambda$ parents and offspring. Algorithm 1 presents a formal definition.

In order to apply the algorithm for the optimization of a fitness function $f : \{0, 1\}^n \to \mathbb{R}$, some parameters need to be set: the population size μ, the offspring population size λ, and the mutation rate p_m. Traditionally, $p_m = 1/n$ is considered a good setting for the mutation rate. Also, in practical applications, a stopping criterion has to be defined since the algorithm does not halt. A fixed number of generations or a fixed number of fitness function evaluations are usually decided

Algorithm 1: Steady-state (μ+λ) GA

1: **Initialisation**:
Initialise $P_0 = \{x^{(1)}, \ldots, x^{(\mu)}\}$ with μ individuals chosen uniformly a random from $\{0, 1\}^n$;
$t \leftarrow 0$;
2: **for** $i = 1, \ldots, \lambda$ **do**
3: **Selection for Reproduction**: Choose $x, y \in P_t$ uniformly at random with replacement;
4: **Crossover**: Create $z^{(i)}$ by picking each bit from either parent with probability $1/2$;
5: **Mutation**: Flip each bit in $z^{(i)}$ with probability p_m;
6: **end for**
7: **Selection for Replacement**
Create the new population P_{t+1} by choosing the best μ individuals out of $\{x^{(1)}, \ldots, x^{(\mu)}, z^{(1)}, \ldots, z^{(\lambda)}\}$;
8: $t \leftarrow t + 1$; Continue at 2;

in advance. Since the objective of the analysis is to calculate the time required to reach the optimal (approximate) solution for the first time, no stopping condition is required, and one can assume that the algorithms are allowed to run forever. The + symbol in the algorithm's name indicates that *elitist truncation selection* is applied, which is typical of steady-state genetic algorithms. This means that the whole population consisting of both parents and offspring are sorted according to fitness and the best μ are retained for the next generation. Some criterion needs to be decided in case the best μ individuals are not uniquely defined. Ties between solutions of equal fitness may be broken uniformly at random. Often offspring are preferred over parents of equal fitness. The crossover operator employed is referred to as *uniform crossover*. If no crossover is employed, then only one individual is selected for reproduction in Line 3, and the offspring is created by only applying the mutation operator. In this case, the algorithm is typically referred to as $(\mu + \lambda)$ EA. In the latter case, if $\mu = \lambda = 1$ are set, then the standard (1+1) EA is obtained, a very simple and well-studied evolutionary algorithm. On the other hand, if some stochastic selection mechanism was used instead of the elitist mechanism, then Algorithm 1 would become a *generational* genetic algorithm (GA) if the crossover operator was retained or a *generational* EA otherwise. Given the importance of the (1+1) EA in this chapter, a formal definition is given in Algorithm 2.

Algorithm 2: (1+1) EA

1: **Initialisation**:
Initialise $x^{(0)}$ uniformly a random from $\{0, 1\}^n$;
$t \leftarrow 0$;
2: **Variation**:
Create y by flipping each bit in $x^{(t)}$ with probability $p_m = 1/n$;
3: **Selection for Replacement**
4: **if** $f(y) \geq f(x^{(t)})$ **then**
5: $x^{(t+1)} \leftarrow y$
6: **end if**
7: $t \leftarrow t + 1$; Continue at 2;

The algorithm is initialized with a random bitstring. At each generation, a new candidate solution is obtained by flipping each bit with probability $p_m = 1/n$. The number of bits that flip can be represented by a binomial random variable $X \sim \text{Bin}(n, p)$ where n is the number of bits (i.e., the number of trials) and $p = 1/n$ is the probability of a success (i.e., a bit actually flips), while $1 - p = 1 - 1/n$ is the probability of a failure (i.e., the bit does not flip). Then, the expected number of bits that flip in one generation is given by the expectation of the binomial random variable, $\mathbb{E}[X] = n \cdot p = n \cdot 1/n = 1$.

The algorithm behaves in a very different way compared to the random local search (RLS) algorithm that flips *exactly* one bit per iteration. Although the (1+1) EA flips exactly one bit in expectation per iteration, many more bits may flip or even none at all. In particular, the (1+1) EA is a *global optimizer* because there is a positive probability that any point in the search space is reached in each generation. As a consequence, the algorithm will find the global optimum in finite time. On the other hand, RLS is a local optimizer since it gets stuck once it reaches a local optimum because it only flips one bit per iteration.

The probability that a binomial random variable $X \sim \text{Bin}(n, p)$ takes value j (i.e., j bits flip) is

$$\Pr(X = j) = \binom{n}{j} p^j (1-p)^{n-j}.$$

Hence, the probability that the (1+1) EA flips exactly one bit is

$$\Pr(X = 1) = \binom{n}{1} \cdot \left(\frac{1}{n}\right) \cdot \left(1 - \frac{1}{n}\right)^{n-1} = \left(1 - \frac{1}{n}\right)^{n-1} \geq 1/e \approx 0.37.$$

So the outcome of one generation of the (1+1) EA is similar to that of RLS only approximately 1/3 of the generations. The probability that two bits flip is exactly half the probability that one flips:

$$\begin{aligned}
\Pr(X = 2) &= \binom{n}{2} \left(\frac{1}{n}\right)^2 \left(1 - \frac{1}{n}\right)^{n-2} \\
&= \frac{n(n-1)}{2} \left(\frac{1}{n}\right)^2 \left(1 - \frac{1}{n}\right)^{n-2} \\
&= \frac{1}{2} \left(1 - \frac{1}{n}\right)^{n-1} \approx 1/(2e).
\end{aligned}$$

On the other hand, the probability no bits flip at all is

$$\Pr(X = 0) = \binom{n}{0} (1/n)^0 \cdot (1 - 1/n)^n \approx 1/e.$$

The latter result implies that in more than 1/3 of the iterations no bits flip. This should be taken into account when evaluating the fitness of the offspring, especially for expensive fitness functions.

In general, the probability that i bits flip decreases exponentially with i

$$\Pr(X = i) = \binom{n}{i} \cdot \frac{1}{n^i} \cdot \left(1 - \frac{1}{n}\right)^{n-i} = \frac{1}{i!} \cdot \left(1 - \frac{1}{n}\right)^{n-i} \approx \frac{1}{i! \cdot e}.$$

In the worst case, all the bits may need to flip to reach the optimum in one step. This event has probability $1/n^n$. Since this is always a lower bound on the probability of reaching the optimum in each generation, by a simple waiting time argument, an upper bound of $O(n^n)$ may be derived for the expected runtime of the (1+1) EA on any pseudo-Boolean function $f : \{0, 1\}^n \to \mathbb{R}$. It is simple to design an example trap function for which the algorithm actually requires $\Theta(n^n)$ expected steps to reach the optimum [31]. This simple result further motivates why it is fundamental to gain a foundational understanding of how the runtime of stochastic search heuristics depends on the parameters of the problem and on the parameters of the algorithms.

Conclusions

Elitist steady-state genetic algorithms [73, 74, 76] were introduced later compared to traditional generational ones [33]. It was originally thought that steady-state EAs/GAs induce a much higher selective pressure compared to the generational model due to the truncation selection for replacement operator (i.e., elitism) and thus are prone to faster premature convergence [34]. For this reason, it has been common to select parents for reproduction uniformly at random rather than using selection operators that prefer better individuals (e.g., tournament or linear ranking selection) which are instead necessary in the generational model [8]. Only recently Corus et al. have shown that in fact the steady-state model allows to control the selective pressure much more carefully than is possible with the generational model and as a result allows for a better balance between exploration and exploitation [11, 14]. Nevertheless, a class of functions has been recently presented where all $(\mu + \lambda)$ steady-state EAs require exponential time as they get trapped with high probability in local optima, while generational EAs are efficient by being able to make good use of their non-elitism to eventually escape [18].

This section focused on the widely used standard bit mutation (SBM) operator and its traditional mutation rate $p_m = 1/n$. This parameter setting is widely considered appropriate as a rule of thumb for mutation-based EAs. For instance, it is the best setting for the (1+1)-EA optimizing linear functions which is also the optimal standard bit mutation-based algorithm for the problem [79]. However, runtime analyses have increasingly suggested the use of higher mutation rates in crossover-based steady-state GAs which may be beneficial both for hillclimbing [5, 6], making them even faster than the (1+1)-EA for ONEMAX, and for escaping

from local optima [17, 30]. In recent years, theory-driven heavy-tailed "power law" mutation operators that have a higher probability of mutating many bits have gained increasing popularity also in mutation-only algorithms [27, 44]. The use of operators with higher mutation rates has been shown to enhance the capabilities of steady-state evolutionary algorithms at escaping from local optima and is naturally present in artificial immune systems [9, 10, 12, 13].

Test Functions

Test functions are artificially designed to analyze the performance of stochastic search algorithms when they face optimization problems with particular structures. These functions are used to highlight characteristics of function classes which may make the optimization process easy or hard for a given algorithm. For this reason, they are often referred to as *toy problems*. The analysis on test functions of simple and well-understood structure has allowed the development of several general techniques for the analysis. Afterward, these techniques have allowed to analyze the same algorithms for more complicated problems with practical applications such as classical combinatorial optimization problems. Furthermore, in recent years, several standard techniques originally developed for simple algorithms have been extended to allow the analyses of more realistic algorithms. In this section, the test functions that will be used as example functions throughout the chapter are introduced.

The most popular test function is ONEMAX (x) := $\sum_{i=1}^{n} x_i$, which simply counts the number of one-bits in the bitstring. The global optimum is a bitstring of only one-bits. ONEMAX is the easiest function with unique global optimum for the (1+1) EA.

A particularly difficult test function for stochastic search algorithms is the *needle-in-a-haystack* function. NEEDLE(x) := $\prod_{i=1}^{n} x_i$ consists of a huge plateau of fitness value zero apart from only one optimal point of fitness value one represented by the bitstring of only one-bits. This function is hard for search heuristics because all the search points apart from the optimum have the same fitness. As a consequence, the algorithms cannot gather any information about where the needle is by sampling search points.

Both ONEMAX and NEEDLE (as defined above) have the property that the function values only depend on the number of ones in the bitstring. The class of functions with this property is called *functions of unitation*.

$$\text{UNITATION}(x) := f\left(\sum_{i=1}^{n} x_i\right).$$

Throughout this chapter, functions of unitation will be used as a general example class to demonstrate the use of the techniques that will be introduced. For simplicity of the analysis, the optimum is assumed to be the bitstring of only one-bits.

For the analysis, the function of unitation will be divided into three different kinds of subblocks: linear blocks, gap blocks, and plateau blocks. Each block will

be defined by its length parameter m (i.e., the number of bits in the block) and by its position parameter k (i.e., each block starts at bitstrings with $m + k$ zeroes and ends at bitstrings with k zeroes). Given a unitation function, it is divided into subblocks proceeding from left to right from the all-zeroes bitstring toward the all-ones bitstring. If the fitness increases with the number of ones, then a *linear block* is created. The linear block ends when the function value stops increasing with the number of ones.

$$\textsc{Linear}(|x|) = \begin{cases} a|x| + b & \text{if } k < n - |x| \leq k + m \\ 0 & \text{otherwise.} \end{cases}$$

See Fig. 2 for an illustration.

If the fitness function decreases with the number of ones, then a *gap block* is created. The gap block ends when the fitness value reaches for the first time a higher value than the value at the beginning of the block.

$$\textsc{Gap}(|x|) = \begin{cases} a & \text{if } n - |x| = k + m \\ 0 & \text{otherwise.} \end{cases}$$

See Fig. 3 for an illustration.

If the fitness remains the same as the number of ones in the bitstrings increases, then a *plateau block* is created. The block ends at the first point where the fitness value changes.

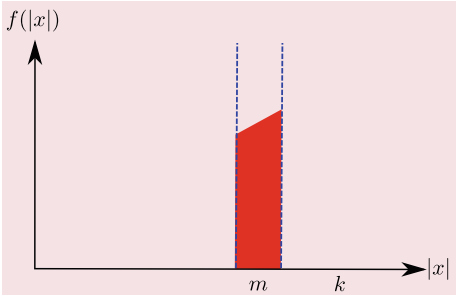


Fig. 2 A linear unitation block of length m starting at position $m + k$ and ending at position k. A linear unitation block of length n is the ONEMAX function

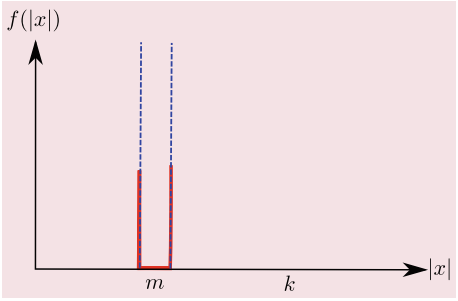


Fig. 3 A gap unitation block of length m starting at position $m + k$ and ending at position k. A gap unitation block of length $n - 1$ is the NEEDLE function

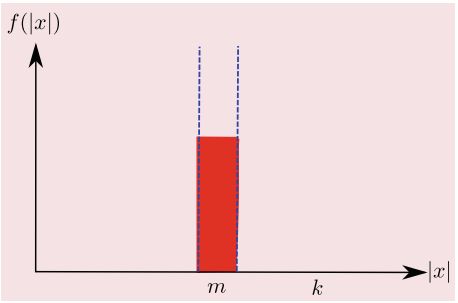


Fig. 4 A plateau unitation block of length m starting at position $m + k$ and ending at position k

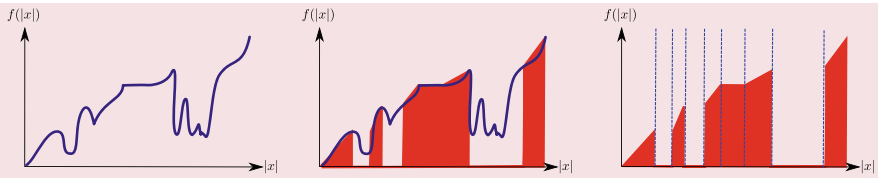


Fig. 5 An illustration of how the search space of a unitation function may be subdivided into blocks of the three kinds

$$\textsc{Plateau}(|x|) = \begin{cases} a & \text{if } k < n - |x| \leq k + m \\ 0 & \text{otherwise.} \end{cases}$$

See Fig. 4 for an illustration.

By proceeding from left to right, the whole search space is subdivided into blocks. See Fig. 5 for an illustration. Let the unitation function be subdivided into r sub-functions $f_1, f_2, \ldots f_r$, and let T_i be the runtime for an elitist search heuristic to optimize each sub-function f_i. Then by linearity of expectation, an upper bound on the expected runtime of an elitist stochastic search heuristic for the unitation function is

$$\mathbb{E}[T] \leq \mathbb{E}\left[\sum_{i=1}^{r} T_i\right] = \sum_{i=1}^{r} \mathbb{E}[T_i].$$

Hence, an upper bound on the total runtime for the unitation function may be achieved by calculating upper bounds on the runtime for each block separately. Once these are obtained, summing all the bounds yields an upper bound on the total runtime. Attention needs to be taken when calculating upper bounds on the runtime to overcome a plateau block when this is followed by a gap block because points straight after the end of the plateau will have lower fitness values, hence will not be accepted. In these special cases, the upper bound for the PLATEAU block needs to be multiplied by the upper bound for the GAP block to achieve a correct upper bound on the runtime to overcome both blocks. In the remainder of the chapter, upper and lower bounds for each type of block will be derived as example applications of the

presented runtime analysis techniques. The reader will then be able to calculate the runtime of the (1+1) EA and other evolutionary algorithms for any such unitation function.

By simply using waiting time arguments, it is possible to derive upper and lower bounds on the runtime of the (1+1) EA for the GAP block. Assuming that the algorithm is at the beginning of the gap block, then to reach the end it is *sufficient* to flip m zero-bits into one-bits and leave the other bits unchanged. On the other hand, it is a *necessary* condition to flip at least m zero-bits because all search points achieved by flipping less than m zero-bits have a fitness value of zero and would not be accepted by selection. Given that there are $m + k$ zero-bits available at the beginning of the block, the following upper and lower bounds on the probability of reaching the end of the block follow

$$\left(\frac{m+k}{nm}\right)^m \frac{1}{e} \leq \binom{m+k}{m}\left(\frac{1}{n}\right)^m \frac{1}{e} \leq p \leq \binom{m+k}{m}\left(\frac{1}{n}\right)^m \leq \left(\frac{(m+k)e}{nm}\right)^m.$$

Here the outer inequalities are achieved by using $\left(\frac{n}{k}\right)^k \leq \binom{n}{k} \leq \left(\frac{en}{k}\right)^k$ for $k \geq 1$. Then by simple waiting time arguments, the expected time for the (1+1) EA to optimize a GAP block of length m and position k is upper and lower bounded by

$$\left(\frac{nm}{(m+k)e}\right)^m \leq \binom{m+k}{m}^{-1} n^m \leq \mathbb{E}[T] \leq en^m \binom{m+k}{m}^{-1} \leq e\left(\frac{nm}{m+k}\right)^m.$$

Tail Inequalities

The runtime of a stochastic search algorithm A for a function (class) f is a random variable $T_{A,f}$, and the main goal of a runtime analysis is to calculate its expectation $\mathbb{E}\left[T_{A,f}\right]$. Sometimes the expected runtime may be particularly large, but there may also be a high probability that the actual optimization time is significantly lower. In these cases, a result about the *success probability* within t steps helps considerably the understanding of the algorithm's performance. In other occasions, it may be interesting to simply gain knowledge about the probability that the actual optimization time deviates from the expected runtime. In such circumstances, *tail inequalities* turn out to be very useful tools by allowing to obtain bounds on the runtime that hold with high probability. An example of the expectation of a random variable and its probability distribution is given in Fig. 6.

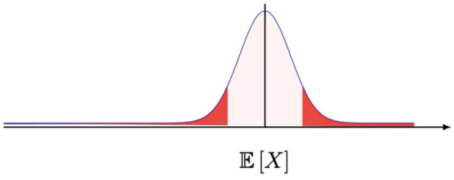


Fig. 6 The expectation of a random variable and its probability distribution. The tails are highlighted in red

Given the expectation of a random variable, which often may be estimated easily, tail inequalities give bounds on the probability that the actual random variable deviates from its expectation [58, 59]. The most simple tail inequality is Markov's inequality. Many strong tail inequalities are derived from Markov's inequality.

Theorem 1 (Markov's Inequality). *Let X be a random variable assuming only nonnegative values. Then for all $t \in \mathbb{R}^+$,*

$$\Pr(X \geq t) \leq \frac{\mathbb{E}[X]}{t}.$$

The power of the inequality is that no knowledge about the random variable is required apart from it being nonnegative.

Let X be a random variable indicating the number of bits flipped in one iteration of the (1+1) EA. As seen in the previous section, one bit is flipped per iteration in expectation, i.e., $\mathbb{E}[X] = 1$. One may wonder what is the probability that more than one bit is flipped in one time step. A straightforward application of Markov's inequality reveals that in at least half of the iterations either one bit is flipped or none:

$$\Pr(X \geq 2) \leq \frac{\mathbb{E}[X]}{2} = \frac{1}{2}.$$

Similarly, one may want to gain some information on how many *ones* are contained in the bitstring at initialization, given that in expectation there are $\mathbb{E}[X] = n/2$ (here X is a binomial random variable with parameters n and $p = 1/2$). An application of Markov's inequality yields that the probability of having more than $(2/3)n$ ones at initialization is bounded by

$$\Pr(X \geq (2/3)n) \leq \frac{\mathbb{E}[X]}{(2/3)n} = \frac{n/2}{(2/3)n} = 3/4. \tag{1}$$

Since X is binomially distributed, it is reasonable to expect that, for large enough n, the actual number of obtained ones at initialization would be more concentrated around the expected value. In particular, while the bound is obviously correct, the probability that the initial bitstring has more than $(2/3)n$ ones is much smaller than $3/4$. However, to achieve such a result, more information about the random variable should be required by the tail inequality (i.e., that it is binomially distributed). An important class of tail inequalities used in the analysis of stochastic search heuristics are Chernoff bounds.

Theorem 2 (Chernoff Bounds). *Let $X_1, X_2, \ldots X_n$ be independent random variables taking values in $\{0, 1\}$. Define $X = \sum_{i=1}^{n} X_i$, which has expectation $E(X) = \sum_{i=1}^{n} \Pr(X_i = 1)$:*

(a) $\Pr(X \leq (1-\delta)\mathbb{E}[X]) \leq e^{\frac{-\mathbb{E}[X]\delta^2}{2}}$ *for* $0 \leq \delta \leq 1$.

(b) $\Pr(X > (1+\delta)\mathbb{E}[X]) \leq \left(\frac{e^{\delta}}{(1+\delta)^{1+\delta}}\right)^{\mathbb{E}[X]}$ *for* $\delta > 0$.

An application of Chernoff bounds reveals that the probability that the initial bitstring has more than $(2/3)n$ one-bits is exponentially small in the length of the bitstring. Let $X = \sum_{i=1}^{n} X_i$ be the random variable summing up the random values X_i of each of the n bits. Since each bit is initialized with probability $1/2$, it holds that $\Pr(X_i = 1) = 1/2$ and $\mathbb{E}[X] = n/2$. By fixing $\delta = 1/3$, it follows that $(1+\delta)\mathbb{E}[X] = (2/3)n$, and finally by applying inequality (b),

$$\Pr(X > (2/3)n) \leq \left(\frac{e^{1/3}}{(4/3)^{4/3}}\right)^{n/2} < \left(\frac{29}{30}\right)^{n/2}.$$

In fact, an exponentially small probability of deviating from $n/2$ by a constant factor of the search space c/n for any constant $c > 0$ may easily be obtained by Chernoff bounds, and even for a deviation of $\Omega(\sqrt{n})$.

For a comprehensive overview of tail inequalities and other probabilistic tools that have proven to be useful in the analysis of randomized search heuristics, we refer to the book chapter by B. Doerr [19].

Artificial Fitness Levels (AFL)

The artificial fitness levels technique is a very simple method to achieve upper bounds on the runtime of elitist stochastic optimization algorithms. Albeit its simplicity, it often achieves very good bounds on the runtime.

The idea behind the method is to divide the search space of size 2^n into m disjoint fitness-based partitions $A_1, \ldots A_m$ of increasing fitness such that $f(A_i) < f(A_j)$ $\forall i < j$. The union of these partitions should cover the whole search space, and the level of highest fitness A_m should contain the global optimum (or all global optima if there is more than one) (see Fig. 7).

Definition 1. A tuple $(A_1, A_2, \ldots, A_m)$ is an f-**based partition** of $f : \mathcal{X} \to \mathbb{R}$ if:

1. $A_1 \cup A_2 \cup \cdots \cup A_m = \mathcal{X}$
2. $A_i \cap A_j = \emptyset$ for $i \neq j$
3. $f(A_1) < f(A_2) < \cdots < f(A_m)$
4. $f(A_m) = \max_x f(x)$

For functions of unitation, a natural way of defining a fitness-based partition is to divide the search space into $n+1$ levels, each defined by the number of

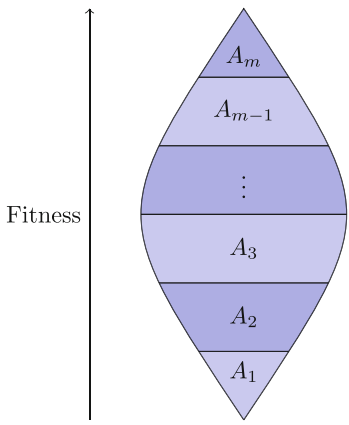


Fig. 7 A partition of the search space satisfying the conditions of an f-based partition

ones in the bitstring. For the ONEMAX function, where fitness increases with the number of ones in the bitstring, the fitness levels would be naturally defined as $A_i := \{x \in \{0,1\}^n \mid \text{ONEMAX}(x) = i\}$.

AFL: Upper Bounds

Given a fitness-based partition of the search space, it is obvious that an elitist algorithm using only one individual will only accept points of the search space that belong to levels of higher or equal fitness to the current level. Once a new fitness level has been reached, the algorithm will never return to previous levels. This implies that each fitness level has to be left at most once by the algorithm. Since in the worst case all fitness levels are visited, the sum of the expected times to leave all levels is an upper bound on the expected time to reach the global optimum. The artificial fitness levels method simplifies this idea by only requiring a lower bound s_i on the probability of leaving each level A_i rather than asking for the exact probabilities to leave each level.

Theorem 3 (Artificial Fitness Levels). *Let $f : \mathcal{X} \to \mathbb{R}$ be a fitness function, $A_1 \dots A_m$ a fitness-based partition of f, and $s_1 \dots s_{m-1}$ be lower bounds on the corresponding probabilities of leaving the respective fitness levels for a level of better fitness. Then the expected runtime of an elitist algorithm using a single individual is $\mathbb{E}\left[T_{A,f}\right] \leq \sum_{i=1}^{m-1} 1/s_i$.*

The artificial fitness level method will now be applied to derive an upper bound on the expected runtime of (1+1) EA for the ONEMAX function. Afterward, the bound will be generalized to general linear blocks of unitation.

Theorem 4. *The expected runtime of the (1+1) EA on* ONEMAX *is $O(n \log n)$.*

Proof. The artificial fitness levels method will be applied to the $m = n + 1$ partitions defined by the number of ones in the bitstring, i.e., $A_i := \{x \in \{0,1\}^n \mid \textsc{Onemax}(x) = i\}$. This means that all bitstrings with i ones and $n - i$ zeroes belong to fitness level A_i. For each level A_i, the method requires a lower bound on the probability of reaching any level A_j where $j > i$. To reach a level of higher fitness, it is necessary to increase the number of ones in the bitstring. However, it is sufficient to flip a *zero* into a *one* and leave the remaining bits unchanged. Since the probability of flipping a bit is $1/n$ and there are $n - i$ zeroes that may be flipped, a lower bound on the probability to reach a level of higher fitness from level A_i is

$$s_i \geq (n-i) \cdot \frac{1}{n} \cdot \left(1 - \frac{1}{n}\right)^{n-1} \geq \frac{n-i}{en},$$

where $(1 - 1/n)^{n-1}$ is the probability of leaving $n - 1$ bits unchanged and the inequality follows because $(1 - 1/n)^{n-1} \geq 1/e$ for all $n \in \mathbb{N}$.

Then by the artificial fitness levels method (Theorem 3),

$$\begin{aligned}\mathbb{E}\left[T_{(1+1)\ EA,\textsc{Onemax}}\right] &\leq \sum_{i=1}^{m-1} 1/s_i \leq \sum_{i=0}^{n-1} \frac{en}{n-i} = en \sum_{i=1}^{n} \frac{1}{i} \\ &= en \ln n + O(n) = O(n \log n).\end{aligned}$$

□

Theorem 5. *The expected runtime of the (1+1)-EA for a* linear block *of length m ending at position k is* $O(n \log((m+k)/k))$.

Proof. Apply the artificial fitness levels method where each partition A_i consists of the bitstrings in the block with i zeroes. Then the probability of leaving a fitness level is bounded by $s_i \geq i/n \cdot (1 - 1/n)^{n-1} \geq i/en$. Given that at most m fitness levels need to be left and that the block starts at position $m + k$ and ends at position k, by Theorem 3, the expected runtime is

$$\mathbb{E}[T] \leq \sum_{i=k+1}^{k+m} \frac{en}{i} \leq en \sum_{i=k+1}^{k+m} \frac{1}{i} \leq en \left(\sum_{i=1}^{k+m} \frac{1}{i} - \sum_{i=1}^{k} \frac{1}{i}\right) \leq en \ln\left(\frac{m+k}{k}\right).$$

□

The AFL method may also be applied for the analysis of the $(\mu+1)$ EA which uses a parent population of size μ. Apart from having a population, the algorithm differs from the (1+1)-EA in another aspect: The selection for replacement step selects uniformly at random among the individuals with lowest fitness rather than favoring the offspring. Since different individuals may be on different fitness levels

during the optimization process, a potential function tracking the fitness level of the best individual can be used to measure the progress of the algorithm toward the optimum. Then, the (1+1)-EA's bounds on the probability to leave a given level A_i may be used, and by considering that the best individual is selected with probability $1/\mu$, a trivial upper bound of $E[T_{A,f}] \leq \sum_{i=1}^{m-1} \mu/s_i$ on the runtime of the (μ+1) EA is provided by the AFL method. This bound is a factor of μ larger than that of the (1+1)-EA.

A more careful insight is that the probability of leaving a level would be considerably higher if more individuals were on the current level rather than just one, and such an increase can happen quickly since it just requires that an individual on the level is selected and creates a copy of itself by not flipping any bits, which in turn occurs with constant probability. The probability of leaving the level is indeed the same as that of the (1+1)-EA if all the individuals are on the current level. Thus, by waiting for a sufficient amount of individuals of the population to *take over* a given fitness level A_i before calculating the probability to reach a fitness level of higher fitness, a tight upper bound on the expected runtime of the (μ+1) EA can be derived by pessimistically assuming that the current level is not left before the take-over has happened. The method may be formalized as follows.

Theorem 6 (Artificial Fitness Levels for Elitist Populations). *Let $f : \mathcal{X} \to \mathbb{R}$ be a fitness function, $A_1 \ldots A_m$ a fitness-based partition of f, and let the algorithm be in the fitness level A_i of the best individual of the population. Furthermore, let $s_1 \ldots s_{m-1}$ be lower bounds on the corresponding probabilities of leaving the respective fitness levels for a level of better fitness given that there are at least $\chi(i)$ individuals on level $A(i)$ and $E[T_{\chi(i)}]$ an upper bound on the expected time to create such a number of individuals on the respective level. Then the expected runtime of an elitist population-based algorithm is $\mathbb{E}\left[T_{A,f}\right] \leq \sum_{i=1}^{m-1} \left(1/s_i + E[T_{\chi(i)}]\right)$.*

We now use the method to derive an upper bound on the expected runtime of the (μ+1) EA for ONEMAX.

Theorem 7. *The expected runtime of the (μ+1) EA on* ONEMAX *is $O(\mu n + n \log n)$.*

Proof. We estimate the time required for the algorithm to reach a population on the level of size at least $\min\{n/(n-i), \mu\}$ when it is on level A_i. If there are j individuals on the level, it suffices to select one of them and create a copy by not flipping any bits to create another individual on the same level. Since this happens with probability at least

$$\frac{j}{\mu}\left(1 - \frac{1}{n}\right)^n \geq \frac{j}{e\mu},$$

the expected time for the population to reach the desired size is at most

$$E[T_{\chi(i)}] \leq \sum_{j=1}^{\lceil n/(n-i)\rceil - 1} \frac{2e\mu}{j} \leq 2e\mu \ln\left(\frac{en}{n-i}\right)$$

if the fitness level is not left before.

Now the expected times to reach the desired number of individuals on each of the n levels can be bounded from above by

$$2e\mu \sum_{i=0}^{n-1} \ln\left(\frac{en}{n-i}\right) = 2e\mu \ln\left(\frac{e^n n^n}{n!}\right) \leq 2e\mu \ln(e^{2n}) = 4e\mu n,$$

where we used Stirling's approximation $n! \geq (n/e)^n$ for the inequality. All that remains to be calculated is the expected time to leave each level given that we have at least $\min\{n/(n-i), \mu\}$ individuals on the level. Let j be the number of individual on fitness level A_i. Then the probability of leaving level A_i is at least,

$$\begin{aligned} s_i &\geq \frac{j}{\mu}\frac{n-i}{n}\left(1-\frac{1}{n}\right)^{n-1} \geq \frac{j(n-i)}{e\mu n} \geq \frac{\min\{n/(n-i), \mu\}(n-i)}{e\mu n} \\ &= \frac{\min\{n, \mu(n-i)\}}{e\mu n}. \end{aligned}$$

Thus the expected time to leave all the fitness levels is at most

$$\sum_{i=0}^{n-1}\left(\frac{e\mu n}{n} + \frac{e\mu n}{\mu(n-i)}\right) \leq en\ln n + e\mu n.$$

Finally, taking into account the μ evaluations at initialization and summing also the $4e\mu n$ evaluations required to fill up each level, we get a total expected runtime of at most $\mu + en\ln n + 5e\mu n = O(n\log n + \mu n)$. □

The AFL method may be further extended to analyze the expected runtime of crossover-based algorithms such as the (μ+1)-GA. For the crossover operator to be useful, it is necessary that the population has diversity. A crossover between two identical individuals produces an offspring that is identical to the parent. On the other hand, if diversity is present, the probability that an offspring assumes the bit values of a given parent where the two parents differ is higher compared to relying on mutation, because a bit is inherited from either parent with probability $1/2$, while a bit is only mutated with probability $1/n$.

One may use the basic AFL method (Theorem 3) to derive trivial upper bounds on the runtime of the (μ+1)-GA by considering that the best individual is selected twice for reproduction with probability $1/\mu^2$, or that a copy of an individual on the current level may be created with probability at least $1/(e\mu^2)$ for an application

of the AFL method for populations (Theorem 6). If s_i is a lower bound on the probability of mutation to leave the level for a better one, then the expected time of the (μ+1)-GA is at most $\mathbb{E}\left[T_{(\mu+1)-GA,f}\right] \leq \sum_{i=1}^{m-1} \mu^2/s_i$. This, for instance, leads to an upper bound of $O(\mu^2 n \log n)$ for ONEMAX using the basic method which however is only asymptotically tight for populations of constant size. Similarly one may hope to get a slightly better bound of $O(\mu^2 n + n \log n)$ on the expected runtime using the artificial fitness levels for elitist populations method. However, the result does not provide any insights into what population sizes are more appropriate and relies on crossover being applied only when diversity is not present in the population, thus not being useful at all. The following extension of the AFL method for the analysis of genetic algorithms allows to better appreciate the effectiveness of populations and crossover at hillclimbing.

We now consider the algorithm to be in fitness level A_i if the whole population is on the level. A Markov chain (MC_i), $1 \leq i \leq m$, as illustrated in Fig. 8, may be used to track whether the population on the current fitness level has diversity or not. Thus m Markov chains are considered, one per fitness level. State $S_{1,i}$ refers to no diversity, i.e., all the individuals have the same genotype, while state $S_{2,i}$ refers to the population having at least two individuals on the level with different genotypes. Thus, from the second state, crossover may exploit the present diversity. For the method to be applicable, from a diversity-less population in state $S_{1,i}$, it has to be possible to create the diversity via mutation with probability p_d. Then, however, from state $S_{2,i}$, it may also be possible to lose the diversity again with some *relapse* probability p_r, for example, if one diverse individual is taken over by the others by creating a copy. The absorbing state $S_{3,i}$ is reached when a solution at a better fitness level is found. This may happen via mutation with probability p_m from state $S_{1,i}$ (because crossover is ineffective when no diversity is present) and via crossover followed by mutation from state $S_{2,i}$. Formally, when the fitness level is left by one individual, the algorithm is not in level A_i anymore. We then wait for this individual to take over the whole population, and we pessimistically assume that the algorithm does not make any further progress in the mean time. Thus, by summing up the absorbing times of the $m-1$ Markov chains, one for each fitness level, and taking into account the $m-1$ take over times before the optimum is hit, we get a valid upper bound on the expected runtime of the (μ+1)-GA. In the following, we formalize the method.

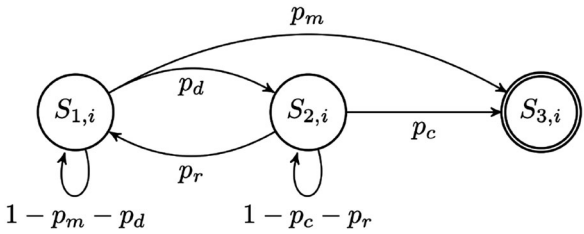


Fig. 8 An illustration of the Markov chain for level A_i designed to distinguish between the algorithm having a diverse population (State $S_{2,i}$) and having one that is not diverse (State $S_{1,i}$)

Theorem 8 (Artificial Fitness Levels for Elitist Populations and Crossover). *Let $f : \mathcal{X} \to \mathbb{R}$ be a fitness function, and $A_1 \dots A_m$ a fitness-based partition of f. Let M be a Markov chain with the topology of Fig. 8. Let $p'_m(i) > 0$, $p'_d(i) > 0$, $p'_c(i) > 0$ be lower bounds, respectively, on the probabilities $p_m(i)$, $p_d(i)$ and $p_c(i)$, and $p'_r(i) > 0$ an upper bound on $p_r(i)$. Then if also $p'_c(i) > p'_m(i)$,*

$$\mathbb{E}\left[T_{A,f}\right] \leq \sum_{i=1}^{m-1} \left(\frac{p'_c(i) + p'_r(i)}{p'_c(i)p'_d(i) + p'_c(i)p'_m(i) + p'_m(i)p'_r(i)} + \frac{1}{p'_c(i)} + E[T_{\chi_\mu(i)}] \right),$$

where $E[T_{\chi_\mu(i)}]$ is the expected time for an individual who has left level A_i to take over the rest of the population.

By calculating the transition probabilities, the method can be applied to prove the following theorem which shows that the (μ+1)-GA is faster than the (1+1)-EA for ONEMAX. In fact, since the (1+1)-EA with mutation rate $1/n$ is the fastest algorithm using standard bit mutation for ONEMAX, the theorem shows that the (μ+1)-GA is faster than any possible algorithm using only standard bit mutation as variation operator.

Theorem 9. *The expected runtime of the (μ+1)-GA for* ONEMAX *is at most* $(3/4)en \ln n(1 + o(1))$ *if* $3 \leq \mu = o(\log n / \log \log n)$.

AFL: Lower Bounds

The artificial fitness levels method has been extended to obtain lower bounds on the runtime of stochastic search algorithms. Since lower bounds are aimed for, the method also needs to take into account the probability that some levels may be skipped by the algorithm, apart from the probabilities of leaving each fitness level.

Theorem 10 (Artificial Fitness Levels for Lower Bounds). *Consider a fitness function $f : \mathcal{X} \to \mathbb{R}$ and $A_1 \dots A_m$ a fitness-based partition of f. Let u_i be the probability of starting in level A_i,s_i be an upper bound on the probability of leaving A_i, and $p_{i,j}$ be an upper bound on the probability of jumping from level A_i to level A_j. If there exists some $0 < \chi \leq 1$ such that for all $j > i$*

$$p_{i,j} \geq \chi \cdot \sum_{k=j}^{m-1} p_{i,k},$$

then the expected runtime of an elitist algorithm using a single individual is

$$\mathbb{E}\left[T_{A,f}\right] \geq \chi \cdot \sum_{i=1}^{m-1} u_i \sum_{j=i}^{m-1} \frac{1}{s_j}.$$

The method will first be illustrated for the (1+1) EA on the ONEMAX function. Afterward, the result will be generalized to general linear blocks of unitation.

Theorem 11. *The expected runtime of the (1+1) EA on* ONEMAX *is* $\Omega(n \log n)$.

Proof. Apply the artificial fitness levels method on the $n+1$ partitions defined by the number of ones in the bitstring, i.e., $A_i := \{x \in \{0,1\}^n \mid \text{ONEMAX}(x) = i\}$. This means that all bitstrings with i ones and $n-i$ zeroes belong to fitness level A_i. To apply the artificial fitness levels method, bounds on s_i and χ need to be derived. An upper bound on the probability of leaving fitness level A_i is simply $s_i \leq (n-i)/n$ because it is a necessary condition that at least one zero flips to reach a better fitness level. The bound follows because each bit flips with probability $1/n$, and there are $n-i$ zeroes available to be flipped. In order to obtain an upper bound on χ, the method requires a lower bound on $p_{i,j}$ and an upper bound on $\sum_{k=j}^{m-1} p_{i,k}$. For the lower bound on $p_{i,j}$, notice that in order to reach level A_j, it is sufficient to flip $j-i$ zeroes out of the $n-i$ zeroes available and leave all the other bits unchanged. Hence, the following bound is obtained:

$$p_{ij} \geq \binom{n-i}{j-i} \left(\frac{1}{n}\right)^{j-i} \left(1-\frac{1}{n}\right)^{n-(j-i)}.$$

For an upper bound on the sum, notice that to reach any level A_k $k \geq j$ from level A_i, it is necessary to flip at least $j-i$ zeroes out of the $n-i$ available zeroes. So,

$$\sum_{k=j}^{n-1} p_{i,k} \leq \binom{n-i}{j-i} \left(\frac{1}{n}\right)^{j-i},$$

and for $\chi := 1/e$, the condition of Theorem 10 is satisfied as follows:

$$p_{i,j} \geq \left(1-\frac{1}{n}\right)^{n-(j-i)} \cdot \sum_{k=j}^{n-1} p_{i,k} \geq \chi \cdot \sum_{k=j}^{n-1} p_{i,k}.$$

By Eq. (1), the probability that the initial search point has less than $(2/3)n$ 1-bits is at least

$$\sum_{i=1}^{(2/3)n} u_i \geq 1 - \frac{3}{4}.$$

The statement of Theorem 10 now yields

$$\begin{aligned}
\mathbb{E}\left[T_{A,f}\right] &\geq \left(\frac{1}{e}\right) \cdot \sum_{i=1}^{n-1} u_i \sum_{j=i}^{n-1} \frac{1}{s_j} \\
&> \left(\frac{1}{e}\right) \cdot \left(\sum_{i=1}^{(2/3)n} u_i\right) \sum_{j=(2/3)n}^{n-1} \frac{1}{s_j} \\
&\geq \left(\frac{1}{e}\right) \cdot \left(1 - \frac{3}{4}\right) \sum_{j=(2/3)n}^{n-1} \frac{n}{n-j} \\
&\geq \left(\frac{n}{4e}\right) \cdot \sum_{j=1}^{n/3} \frac{1}{j}.
\end{aligned}$$

It now follows that $\mathbb{E}\left[T_{A,f}\right] = \Omega(n \log n)$. □

Similarly, the following result may also be proved for linear blocks of unitation functions by defining the fitness partitions as $A_i := \{x : n - |x| = k + m - i\}$ for $0 \leq i \leq m$.

Theorem 12. *The expected runtime of the (1+1)-EA for a* linear block *of length m ending at position k is* $\Omega(n \log((m+k)/k))$.

Level-Based Analysis of Non-elitist Populations

A weakness of the classical artificial fitness level technique is that it is limited to search heuristics that use elitism such as steady-state EAs. The so-called *level-based analysis* is a generalization of fitness level theorems for non-elitist evolutionary algorithms which is also applicable to search heuristics using higher arity operators such as crossover.

The theorem applies to any algorithm that can be expressed in the form of Algorithm 3, such as genetic algorithms and estimation of distribution algorithms. The main component of the algorithm is a random operator D which given the current population $P_t \in \mathcal{X}^\lambda$ returns a probability distribution $D(P_t)$ over the search space $\mathcal{X}$. The next population P_{t+1} is obtained by sampling individuals independently from this distribution.

In contrast to classical fitness level theorems, the level-based theorem (Theorem 13) only assumes a partition $(A_1, \ldots, A_{m+1})$ of the search space $\mathcal{X}$, and not an f-based partition (see Definition 1). Each of the sets A_j, $j \in [m+1]$ is called a *level*, and the symbol $A_j^+ := \bigcup_{i=j+1}^{m+1} A_i$ denotes the set of search points above level A_j. Given a constant $\gamma_0 \in (0, 1)$, a population $P \in \mathcal{X}^\lambda$ is considered to be at

Algorithm 3: Population-based algorithm with independent sampling

1: **Initialisation**:
$t \leftarrow 0$; Initialise P_t uniformly at random from $\mathcal{X}^\lambda$.
2: **Variation and Selection**:
3: **for** $i = 1 \ldots \lambda$ **do**
Sample $P_{t+1}(i) \sim D(P_t)$
4: **end for**
5: $t \leftarrow t + 1$; Continue at 2

level A_j with respect to γ_0 if $|P \cap A_{j-1}^+| \geq \gamma_0 \lambda$ and $|P \cap A_j^+| < \gamma_0 \lambda$ meaning that at least a γ_0 fraction of the population is in level A_j or higher.

Theorem 13 (Level-Based Theorem). *Given any partition of a finite set $\mathcal{X}$ into m nonoverlapping subsets $(A_1, \ldots, A_{m+1})$, define $T := \min\{t\lambda \mid |P_t \cap A_{m+1}| > 0\}$ to be the first point in time that elements of A_{m+1} appear in P_t of Algorithm 3. If there exist parameters $z_1, \ldots, z_m, z_* \in (0, 1]$, $\delta > 0$, and a constant $\gamma_0 \in (0, 1)$ such that for all $j \in [m]$, $P \in \mathcal{X}^\lambda$, $y \sim D(P)$, and $\gamma \in (0, \gamma_0]$, it holds:*

(C1) $\Pr\left(y \in A_j^+ \mid |P \cap A_{j-1}^+| \geq \gamma_0 \lambda\right) \geq z_j \geq z_*$

(C2) $\Pr\left(y \in A_j^+ \mid |P \cap A_{j-1}^+| \geq \gamma_0 \lambda \text{ and } |P \cap A_j^+| \geq \gamma \lambda\right) \geq (1 + \delta)\gamma$, *and*

(C3) $\lambda \geq \frac{2}{a} \ln\left(\frac{16m}{ac\varepsilon z_*}\right)$ *with* $a = \frac{\delta^2 \gamma_0}{2(1+\delta)}$, $\varepsilon = \min\{\delta/2, 1/2\}$ *and* $c = \varepsilon^4/24$

then

$$\mathbb{E}[T] \leq \frac{2}{c\varepsilon}\left(m\lambda(1 + \ln(1 + c\lambda)) + \sum_{j=1}^{m} \frac{1}{z_j}\right).$$

The theorem provides an upper bound on the expected optimization time of Algorithm 3 if it is possible to find a partition $(A_1, \ldots, A_{m+1})$ of the search space $\mathcal{X}$ and accompanying parameters $\gamma_0, \delta, z_1, \ldots, z_m, z_*$ such that conditions (C1), (C2), and (C3) are satisfied. Condition (C1) requires a nonzero probability z_j of creating an individual in level A_{j+1} or higher if there are already at least $\gamma_0 \lambda$ individuals in level A_j or higher. In typical applications, this imposes some conditions on the variation operator. The condition is analogous to the probability s_j in the artificial fitness level technique. Condition (C2) requires that if in addition there are $\gamma\lambda$ individuals at level A_{j+1} or better, then the probability of producing an individual in level A_{j+1} or better should be larger than γ by a multiplicative factor $1 + \delta$. In typical applications, this imposes some conditions on the strength of the selective pressure in the algorithm. Finally, condition (C3) imposes minimal requirements on the population size in terms of the parameters above.

As an example application of the level-based theorem, the (μ, λ) EA is analyzed, which is the non-elitist variant of the $(\mu + \lambda)$ EA shown in Algorithm 1 (without crossover). The two algorithms differ in the selection step (line 7) where the new

population P_{t+1} in (μ, λ) EA is chosen as the best μ individuals out of $\{y_1, \ldots, y_\lambda\}$ and breaking ties uniformly at random. While the $(\mu + \lambda)$ EA always retains the best μ individuals in the population (hence the name elitist), the (μ, λ) EA always discards the old individuals $x^{(1)}, \ldots, x^{(\mu)}$.

At first sight, it may appear as if the (μ, λ) EA cannot be expressed in the form of Algorithm 3. The μ individuals $x^{(1)}, \ldots, x^{(\mu)}$ that are kept in each generation are not independent due to the inherent sorting of the offspring. However, taking a different perspective, the population of the algorithm at time t could also be interpreted as the λ offspring $y^{(1)}, \ldots, y^{(\lambda)}$. In this alternative interpretation, the new population is now created by sampling uniformly at random among the μ best individuals in the population and applying the mutation operator. The operator D in Algorithm 3 can now be defined as in Algorithm 4.

Algorithm 4: Operator D corresponding to (μ, λ) EA

1: **Selection**:
Sort the population $P_t = (y^{(1)}, \ldots, y^{(\lambda)})$ such that $f(y^{(1)}) \geq f(y^{(2)}) \geq \ldots \geq f(y^{(\lambda)})$.
Select x uniformly at random among $\{y^{(1)}, \ldots, y^{(\mu)}\}$.
2: **Variation (mutation)**:
Create x' by flipping each bit in x with probability χ/n.
3: **return** x'

The following lemma will be useful when estimating the probability that the mutation operator does not flip any bit positions.

Lemma 1. *For any $\delta \in (0, 1)$ and $\chi > 0$, if $n \geq (\chi + \delta)(\chi/\delta)$, then*

$$\left(1 - \frac{\chi}{n}\right)^n \geq (1 - \delta)e^{-\chi}.$$

Proof. Note first that $\ln(1 - \delta) < -\delta$, hence

$$\left(\frac{n}{\chi} - 1\right)(\chi - \ln(1 - \delta)) \geq n + \frac{n\delta}{\chi} - (\chi + \delta) \geq n.$$

By making use of the fact that $(1 - 1/x)^{x-1} \geq 1/e$ and simplifying the exponent n as above

$$\left(1 - \frac{\chi}{n}\right)^n \geq \left[\left(1 - \frac{\chi}{n}\right)^{(n/\chi)-1}\right]^{\chi - \ln(1-\delta)} \geq (1 - \delta)e^{-\chi}.$$

□

The expected optimization time of the (μ, λ) EA on ONEMAX can now be expressed in terms of the mutation rate χ/n and the problem size n assuming some constraints on the population sizes μ and λ. The theorem is valid for a wide range

of mutation rates χ/n. In the classical setting of $\chi = 1$, the expected optimization time reduces to $O(n\lambda \ln \lambda)$.

Theorem 14. *The expected optimization time of the (μ, λ) EA with bitwise mutation rate χ/n where $\chi \in (0, n/2)$, and population sizes μ and λ satisfying for any constant $\delta \in (0, 1)$*

$$\frac{\lambda}{\mu} \geq \left(\frac{1+\delta}{1-\delta}\right) e^{\chi}, \quad \text{and} \quad \lambda \geq \frac{4}{\delta^2 e^{\chi}} \ln\left(\frac{24576n(n+1)}{\delta^7 \chi}\right)$$

on ONEMAX *is for any $n \geq (\chi + \delta)/(\chi/\delta)$ no more than*

$$\frac{1536n}{\delta^5}\left(\lambda \ln(\lambda) + \frac{e^{\chi} \ln(n+2)}{\chi(1-\delta)}\right) + O(n\lambda).$$

Proof. Apply the level-based theorem with the same $m := n + 1$ partitions as in the proof of Theorem 11. Since the parameter δ is assumed to be some constant $\delta \in (0, 1)$, it also holds that the parameters a, ε, and c are positive constants. The parameters $\gamma_0, z_1, \ldots, z_m$, and z_* will be chosen later.

To verify that conditions (C1) and (C2) hold for any $j \in [m]$, it is necessary to estimate the probability that operator D produces a search point x' with $j + 1$ one-bits when applied to a population P containing at least $\gamma_0\lambda$ individuals, each having at least j one-bits (formally $|P \cap A^+_{j-1}| \geq \gamma_0\lambda$). Such an event is called a *successful sample*.

Condition (C1) asks for bounds z_j for each $j \in [m]$ on the probability that the search point x' returned by Algorithm 4 contains $j + 1$ one-bits. First choose the parameter setting $\gamma_0 := \mu/\lambda$. This parameter setting is convenient, because the selection step in Algorithm 4 always picks an individual x among the best $\mu = \gamma_0\lambda$ individuals in the population. By the assumption that $|P \cap A^+_{j-1}| \geq \gamma_0\lambda$, the algorithm will always select an individual x containing at least $j + i$ one-bits for some nonnegative integer $i \geq 0$.

Assume without loss of generality that the first j-bit positions in the selected individual x are one-bits, and let k, $j < k \leq n$, be any of the other bit positions. If there is a zero-bit in position k or if $i \geq 2$, then a successful sample occurs if the mutation operator flips only bit position k. If there is a one-bit in position k, and if $i = 1$, then the step is still successful if the mutation operator flips none of the bit positions. Since the probability of not flipping a position is higher than the probability of flipping a position, i.e., $1-\chi/n \geq \chi/n$, the probability of a successful sample is therefore in both cases at least

$$(n - j)(\chi/n)(1 - \chi/n)^{n-1}. \tag{2}$$

By Lemma 1, the probability above is at least $z_j := (n - j)(\chi/n)e^{-\chi}(1 - \delta)$. The parameter z_* is chosen to be the minimal among these probabilities, i.e., $z_* := (\chi/n)e^{-\chi}(1 - \delta)$.

Condition (C2) assumes in addition that $\gamma\lambda < \mu$ individuals have fitness $j+1$ or higher. In this case, it suffices that the selection mechanism picks one of the best $\gamma\lambda$ individuals among the μ individuals and that none of the bits are mutated in the selected individual. The probability of this event is at least

$$\frac{\gamma\lambda}{\mu}(1-\chi/n)^n \geq \frac{\gamma\lambda}{\mu}e^{-\chi}(1-\delta).$$

Hence, to satisfy condition (C2), it suffices to require that

$$\frac{\gamma\lambda}{\mu}\exp(-\chi)(1-\delta) \geq \gamma(1+\delta),$$

which is true whenever

$$\frac{\lambda}{\mu} \geq \left(\frac{1+\delta}{1-\delta}\right)e^{\chi}.$$

To check condition (C3), notice that $\varepsilon = \delta/2$, and $c = \delta^4/384$, hence

$$a = \frac{\delta^2(\lambda/\mu)}{2(1+\delta)} \geq \frac{\delta^2 e^{\chi}}{2(1-\delta)}, \text{ and}$$

$$ac\varepsilon z_* \geq \frac{\delta^2\chi}{2n}c\varepsilon = \frac{\delta^7\chi}{1536n}.$$

Condition (C3) is now satisfied, because the population size λ is required to fulfil

$$\frac{2}{a}\ln\left(\frac{16m}{ac\varepsilon z_*}\right) \leq \frac{4(1-\delta)}{\delta^2 e^{\chi}}\ln\left(\frac{24576mn}{\delta^7\chi}\right) \leq \lambda.$$

All conditions are satisfied, and the theorem follows. □

Conclusions

The artificial fitness levels method was first described by Wegener [75]. The original method was designed for the achievement of upper bounds on the runtime of stochastic search heuristics using only one individual such as the (1+1) EA. Since then, several extensions of the method have been devised for the analysis of more sophisticated algorithms. Sudholt introduced the method presented in section "AFL: Lower Bounds" for the obtainment of lower bounds on the runtime [71]. In an early study, Witt was the first to use a potential function that generalizes the fitness level argument of [75] to analyze the (μ+1) EA [77]. The resulting method is introduced in section "AFL: Upper Bounds" (Theorem 6). His analysis achieved tight upper bounds on the runtime of the (μ+1) EA on LEADINGONES and ONEMAX by waiting

for a sufficient amount of individuals of the population to *take over* a given fitness level A_i before calculating the probability to reach a level of higher fitness. Chen et al. extended the analysis to offspring populations by analyzing the $(N+N)$ EA, also taking into account the take-over process [3]. Corus and Oliveto introduced the method (Theorem 8) for the analysis of the $(\mu+1)$ GA using populations and crossover by coupling each fitness level with a Markov chain to analyze the diversity of the population on each level [5]. They subsequently extended the Markov chain to contain $\mu/2$ states plus an absorbing state for each fitness level to consider the exact amount of diversity present in the population on each level before improving [6]. This extension allows to show that the $(\mu+1)$ GA is faster than any unbiased mutation-only randomized search heuristic, as well as the utility of populations since the provided upper bounds on the expected runtime decrease with the population size and are indeed lower than the lower bound for the (2+1) GA provided by [66]. Lehre introduced a general fitness level method for arbitrary population-based EAs with non-elitist selection mechanisms and unary variation operators [46]. This technique was later generalized further into the level-based method presented in section "Level-Based Analysis of Non-elitist Populations" [8]. The method allows the analysis of sophisticated non-elitist heuristics such as generational genetic algorithms equipped with mutation, crossover, and stochastic selection mechanisms, for classical, noisy, and uncertain optimization [15] as well as estimation of distribution algorithms [16]. Recently, it has been successfully applied to the analysis of competitive coevolutionary algorithms [2, 47].

Drift Analysis

Drift analysis is a very flexible and powerful tool that is widely used in the analysis of stochastic search algorithms. The high-level idea is to predict the long-term behavior of a stochastic process by measuring the expected progress toward a target in a single step. Naturally, a measure of progress needs to be introduced, which is generally called a *distance function*. Given a random variable X_k representing the current state of the process at step k, over a finite set of states S, a distance function $d : S \to \mathbb{R}_0^+$ is defined such that $d(X_k) = 0$ if and only if X_k is a target point (e.g., the global optimum). Drift analysis aims at deriving the expected time to reach the target by analyzing the decrease in distance in each step, i.e., $d(X_{k+1}) - d(X_k)$. The expected value of this decrease in distance $\Delta_k = \mathbb{E}[d(X_{k+1}) - d(X_k) \mid X_k]$ is called the *drift*. See Fig. 9 for an illustration. If the initial distance from the target is $d(X_0)$ and a bound on the drift Δ (i.e., the expected improvement in each step) is known, then bounds on the expected runtime to reach the target may be derived.

Additive Drift Theorem

The additive drift theorem allows to derive both upper and lower bounds on the runtime of stochastic search algorithms. Consider a distance function $Y_k = d(X_k)$

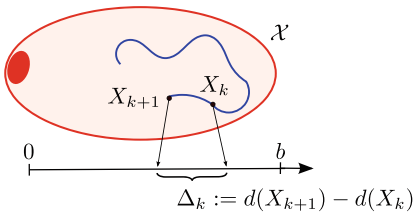


Fig. 9 An illustration of the drift Δ at time step k of a process represented by the random variable X and a distance function d

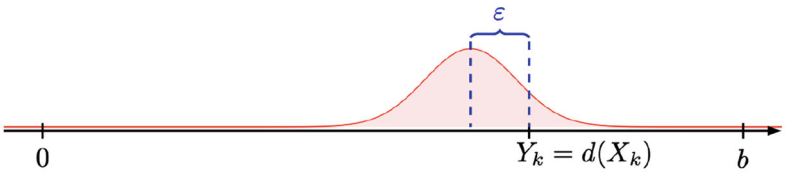


Fig. 10 An illustration of the condition of the additive drift theorem. If the expected distance to the optimum decreases of *at least* ε at each step (i.e., condition C1+), then an upper bound on the runtime is achieved. If the distance decreases of *at most* ε at each step (i.e., condition C1−), then a lower bound on the runtime is obtained

indicating the current distance, at time k, of the stochastic process from the optimum. The theorem simply states that if at each time step k, the drift is *at least* some value $-\varepsilon$ (i.e., the process has moved closer to the target), then the expected number of steps to reach the target is *at most* Y_0/ε. Conversely, if the drift in each step is *at most* some value $-\varepsilon$, then the expected number of steps to reach the target is *at least* Y_0/ε (Fig. 10).

Theorem 15 (Additive Drift Theorem). *Given a stochastic process* $X_1, X_2, \ldots$ *over an interval* $[0, b] \subset \mathbb{R}$ *and a distance function* $d : S \to \mathbb{R}_0^+$ *such that* $d(X_k) = 0$ *if and only if* X *contains the target. Let* $Y_k = d(X_k)$ *for all* k*, define* $T := \min\{k \geq 0 \mid Y_k = 0\}$*, and assume* $\mathbb{E}[T] < \infty$.

Verify the following conditions:

(C1+) $\forall k \quad \mathbb{E}[Y_{k+1} - Y_k \mid Y_k > 0] \leq -\varepsilon$
(C1−) $\forall k \quad \mathbb{E}[Y_{k+1} - Y_k \mid Y_k > 0] \geq -\varepsilon$

Then:

1. *If (C1+) holds for an* $\varepsilon > 0$*, then* $\mathbb{E}[T \mid Y_0] \leq b/\varepsilon$.
2. *If (C1−) holds for an* $\varepsilon > 0$*, then* $\mathbb{E}[T \mid Y_0] \geq Y_0/\varepsilon$.

An example application of the additive drift theorem follows concerning the (1+1) EA for plateau blocks of functions of unitation of length m positioned such that $k > n/2 + \varepsilon n$.

Theorem 16. *The expected runtime of the (1+1)-EA for a* plateau block *of length* m *ending at position* $k > n/2 + \varepsilon n$ *is* $\Theta(m)$.

Proof. The additive drift theorem will be applied to derive both upper and lower bounds on the expected runtime. The starting point is a bitstring X_0 with $m + k$ zeroes, and the target point is a bitstring X_t with k zeroes. Choose to use the natural distance function $Y_t = d(X_t) := n - |X_t|$ that counts the number of zeroes in the bitstring. Subtract k from the distance such that target points with k zeroes have distance 0 and the initial point has distance m. As long as points on the plateau are generated, they will be accepted because all plateau points have equal fitness. Given that each bit flips with probability $1/n$, and at each step the current search point has Y_t zeroes and $n - Y_t$ ones, the drift is

$$\Delta_t := \mathbb{E}\left[Y_t - Y_{t+1} \mid Y_t > 0\right] = \frac{Y_t}{n} - \frac{n - Y_t}{n} = \frac{2 \cdot Y_t}{n} - 1.$$

A lower bound on the drift is obtained by considering that as long as the end of the plateau has not been reached, there are always at least k zeroes that may be flipped (i.e., $Y_t \geq k$). Accordingly for an upper bound, at most $m + k$ zeroes may be available to be flipped (i.e., $Y_t \leq m + k$). Hence,

$$\frac{2k}{n} - 1 \leq \Delta_t \leq \frac{2(m+k)}{n} - 1.$$

Then by additive drift analysis (Theorem 15),

$$\mathbb{E}\left[T \mid Y_0\right] \leq \frac{m}{(2k)/n - 1} = \frac{mn}{2k - n} = O(m)$$

and

$$\mathbb{E}\left[T \mid Y_0\right] \geq \frac{m}{2(m+k)/n - 1} = \frac{mn}{2(m+k) - n} = \Omega(m),$$

where the last equalities hold as long as $k > n/2 + \varepsilon n$. □

Note again that if the plateau block is followed by a gap block, then an upper bound on the expected time to optimize both blocks is achieved by multiplying the upper bounds obtained for each block. This is necessary because points in the gap will not be accepted by the (1+1) EA.

Multiplicative Drift Theorem

In the additive drift theorem, the worst-case decrease in distance is considered. If the expected decrease in distance changes considerably in different areas of the search space, then the estimate on the drift may be too pessimistic for the obtainment of tight bounds on the expected runtime.

Drift analysis of the (1+1) EA for the classical ONEMAX function will serve as an example of this problem. Since the global optimum is the all-ones bitstring

and the fitness increases with the number of ones, a natural distance function is $Y_t = d(X_t) = n - \textsc{Onemax}(X_t)$, which simply counts the number of zeroes in the current search point. Then the distance will be zero once the optimum is found. Points with less one-bits than the current search point will not be accepted by the algorithm because of their lower fitness. So the drift is always positive, i.e., $\Delta_t \geq 0$, and the amount of progress is the expected number of ones gained in each step. In order to find an upper bound on the runtime, a lower bound on the drift is needed (i.e., the worst-case improvement). Such worst case occurs when the current search point is optimal except for one 0-bit. In this case, the maximum decrease in distance that may be achieved in a step is $Y_t - Y_{t+1} = 1$, and to achieve such progress, it is necessary that the algorithm flips the zero into a one and leaves the other bits unchanged. Hence, the drift is

$$\Delta_t \geq 1 \cdot \frac{1}{n}\left(1 - \frac{1}{n}\right)^{n-1} \geq \frac{1}{en} := \varepsilon.$$

Since the expected initial distance is $\mathbb{E}[Y_0] = n/2$ due to random initialization, the drift theorem yields

$$\mathbb{E}[T \mid Y_0] \leq \frac{\mathbb{E}[Y_0]}{\varepsilon} = \frac{n/2}{1/(en)} = e/2 \cdot n^2 = O(n^2).$$

In section "Artificial Fitness Levels (AFL)," it was proven that the runtime of the (1+1) EA for ONEMAX is $\Theta(n \log n)$; hence, a bound of $O(n^2)$ is not tight. The reason is that on functions such as ONEMAX the amount of progress made by the algorithm depends crucially on the distance from the optimum. For ONEMAX in particular, larger progress per step is achieved when the current search point has many zeroes that may be flipped. As the algorithm approaches the optimal solution, the amount of expected progress in each step becomes smaller because search points have increasingly more one-bits than zero-bits in the bitstring. In such cases, a distance function that takes into account these properties of the objective function needs to be used. For ONEMAX, a correct bound is achieved by using a distance function that is logarithmic in the number of zeroes i, i.e., $Y_t = d(X_t) := \ln(i+1)$, where a 1 is added to i in the argument of the logarithm such that the global optimum has distance zero (i.e., $\ln(1) = 0$). With such distance measure, the decrease in distance when flipping a zero and leaving the rest of the bitstring unchanged is

$$\ln(i+1) - \ln(i) = \ln\left(1 + \frac{1}{i}\right) \geq \frac{1}{2i},$$

where the last inequality holds for all $i \geq 1$. Since it is sufficient to flip a zero and leave everything else unchanged to obtain an improvement, the drift is

$$\Delta_t \geq \frac{i}{en} \cdot \frac{1}{2i} = \frac{1}{2en} := \varepsilon.$$

Given that the maximum possible distance is $Y_0 \geq \ln(n+1)$, the drift theorem yields

$$\mathbb{E}[T] \leq \frac{Y_0}{1/(2en)} = 2en \cdot \ln(n+1) = O(n \log n).$$

The multiplicative drift theorem was introduced as a handy tool to deal with situations as the one described above where the amount of progress depends on the distance from the target.

Theorem 17 (Multiplicative Drift Theorem [24]). *Let $\{X_t\}_{t \in \mathbb{N}_0}$ be random variables describing a Markov process over a finite state space $S \subseteq \mathbb{R}$. Let T be the random variable that denotes the earliest point in time $t \in \mathbb{N}_0$ such that $X_t = 0$. If there exist $\delta, c_{\min}, c_{\max} > 0$ such that for all $t < T$:*

1. $\mathbb{E}[X_t - X_{t+1} \mid X_t] \geq \delta X_t$ *and*
2. $c_{\min} \leq X_t \leq c_{\max}$

then

$$\mathbb{E}[T] \leq \frac{2}{\delta} \cdot \ln\left(1 + \frac{c_{\max}}{c_{\min}}\right).$$

The following derivation of an upper bound on the runtime of the (1+1) EA for linear blocks illustrates the multiplicative drift theorem.

Theorem 18. *The expected time for the (1+1)-EA to optimize a linear unitation block of length m ending at position k is $O(n \log((m+k)/k))$.*

Proof. Let X_t be the number of zero-bits in the bitstring at time step t, representing the distance from the end of the linear block. By remembering that increases in distance are not accepted due to elitism, the expected decrease in distance at time step can be bounded by

$$\mathbb{E}[X_{t+1}|X_t] \leq X_t - 1 \cdot \frac{X_t}{en} = X_t\left(1 - \frac{1}{en}\right)$$

simply by considering that if a zero-bit is flipped and nothing else, then the distance decreases by 1. Then the drift is

$$\mathbb{E}[X_t - X_{t+1}|X_t] \geq X_t - X_t\left(1 - \frac{1}{en}\right) = \frac{1}{en}X_t := \delta X_t.$$

By fixing $k = c_{\min} \leq X_t \leq c_{\max} = m + k$, the multiplicative drift theorem yields

$$\mathbb{E}[T] \leq \frac{2}{\delta} \cdot \ln\left(1 + \frac{c_{\max}}{c_{\min}}\right) = 2en \ln(1 + (m+k)/k) = O(n \log((m+k)/k)).$$

□

By fixing $1 = c_{\min} \leq X_t \leq c_{\max} = n$, an $O(n \log n)$ bound on the expected runtime of the (1+1) EA for ONEMAX is achieved.

Variable Drift Theorem

The multiplicative drift theorem is applicable when the drift of a stochastic process is linear with respect to the current position. However, in some stochastic processes, the drift is nonlinear in the current position, i.e.,

$$\mathbb{E}[X_t - X_{t+1} \mid X_t \geq x_{\min}] \geq h(X_t) \tag{3}$$

for some function h. The following variable drift theorem provides bounds on the expectation and the tails of the hitting time distribution of such processes, given some assumptions about the function h.

Theorem 19 (Corollary 1 in [48]). *Let $(X_t)_{t\geq 0}$ be a stochastic process over some state space $S \subseteq \{0\} \cup [x_{\min}, x_{\max}]$, where $x_{\min} \geq 0$. Let $h\colon [x_{\min}, x_{\max}] \to \mathbb{R}^+$ be a differentiable function. Then the following statements hold for the first hitting time $T := \min\{t \mid X_t = 0\}$:*

(i) If $\mathbb{E}[X_t - X_{t+1} \mid X_t \geq x_{\min}] \geq h(X_t)$ and $h'(x) \geq 0$, then

$$\mathbb{E}[T \mid X_0] \leq \frac{x_{\min}}{h(x_{\min})} + \int_{x_{\min}}^{X_0} \frac{1}{h(y)}\,\mathrm{d}y.$$

(ii) If $\mathbb{E}[X_t - X_{t+1} \mid X_t \geq x_{\min}] \leq h(X_t)$ and $h'(x) \leq 0$, then

$$\mathbb{E}[T \mid X_0] \geq \frac{x_{\min}}{h(x_{\min})} + \int_{x_{\min}}^{X_0} \frac{1}{h(y)}\,\mathrm{d}y.$$

(iii) If $\mathbb{E}[X_t - X_{t+1} \mid X_t \geq x_{\min}] \geq h(X_t)$ and $h'(x) \geq \lambda$ for some $\lambda > 0$, then

$$\Pr(T \geq t \mid X_0) < \exp\left(-\lambda\left(t - \frac{x_{\min}}{h(x_{\min})} - \int_{x_{\min}}^{X_0} \frac{1}{h(y)}\,\mathrm{d}y\right)\right).$$

(iv) If $\mathbb{E}[X_t - X_{t+1} \mid X_t \geq x_{\min}] \leq h(X_t)$ and $h'(x) \leq -\lambda$ for some $\lambda > 0$, then

$$\Pr(T < t \mid X_0 > 0) < \frac{e^{\lambda t} - e^{\lambda}}{e^{\lambda} - 1} \exp\left(-\frac{\lambda x_{\min}}{h(x_{\min})} - \int_{x_{\min}}^{X_0} \frac{\lambda}{h(y)}\,\mathrm{d}y\right).$$

To illustrate the variable drift theorem, an upper bound on the optimization time of the (1+1) EA on the class of linear functions with bounded coefficients will be derived. More formally, this class of functions contains any function of the form

$$f(x) := \sum_{i=1}^{n} w_i x_i,$$

with bounded, positive coefficients $w_1, \ldots, w_n \in (w_{\min}, w_{\max})$, where $0 < w_{\min} < w_{\max}$.

The drift function h in this example turns out to be linear; hence, the multiplicative drift theorem could have been applied instead.

Theorem 20. *The expected optimization time of the (1+1) EA on linear functions is less than* $t(n) := en(\ln(n) + \ln(w_{\max}/w_{\min}) + 1)$*, and the probability that the optimization time exceeds* $t(n) + ren$ *for any* $r \geq 0$ *is no more than* e^{-r}.

Proof. Define the distance X_t at time k to be the function value that "remains" at time k, i.e.,

$$X_t := \left(\sum_{i=1}^{n} w_i\right) - \left(\sum_{i=1}^{n} w_i x_i^{(t)}\right) = \sum_{i=1}^{n} w_i \left(1 - x_i^{(t)}\right),$$

where $x_i^{(t)}$ is the i-th bit in the current search point at time t. For any $i \in [n]$, assume that the mutation operator flipped only bit position i, and no other bit positions, an event denoted by the symbol $\mathcal{E}_i$. If $x_i^{(t)} = 0$, then bit position i flipped from 0 to 1 and the distance reduced by w_i. Otherwise, if $x_i^{(t)} = 1$, then bit position i flipped from 1 to 0, the new search point was not accepted, and the distance reduced by 0. Hence, the distance always reduces by $w_i(1 - x_i^{(t)})$ when event $\mathcal{E}_i$ occurs. Using the law of total probability, and noting that the distance can never increase, one obtains

$$\begin{aligned}\mathbb{E}[X_t - X_{t+1} \mid X_t = r] &\geq \sum_{i=1}^{n} \Pr(\mathcal{E}_i \mid X_t = r)\, \mathbb{E}[X_t - X_{t+1} \mid \mathcal{E}_i \wedge X_t = r] \\ &\geq \left(\frac{1}{n}\right)\left(1 - \frac{1}{n}\right)^{n-1} \sum_{i=1}^{n} w_i \left(1 - x_i^{(t)}\right) \geq \frac{r}{en}.\end{aligned}$$

Therefore, $\mathbb{E}[X_t - X_{t+1} \mid X_t] \geq h(X_t)$ for the function $h(x) = x/en$, which has derivative $h'(x) = 1/en$. By Theorem 19 part (i), it follows that

$$\begin{aligned}\mathbb{E}[T \mid X_0] &\leq \frac{x_{\min}}{h(x_{\min})} + \int_{x_{\min}}^{X_0} \frac{1}{h(y)}\,\mathrm{d}y \\ &= \frac{w_{\min}}{h(w_{\min})} + \int_{w_{\min}}^{nw_{\max}} \frac{en}{y}\,\mathrm{d}y \\ &= en + en(\ln(nw_{\max}) - \ln(w_{\min})) \\ &= en(\ln(n) + \ln(w_{\max}/w_{\min}) + 1) =: t(n).\end{aligned}$$

Furthermore, Theorem 19 part (iii) with $\lambda := 1/en$ gives for any $r \geq 0$

$$\Pr(T \geq t(n) + enr) \leq e^{-r}.$$

□

Negative-Drift Theorem

The drift theorems presented in previous subsections are designed to prove polynomial bounds on the runtime of stochastic search algorithms. For this, a positive drift toward the optimum is required. On the other hand, a negative drift indicates that the stochastic process moves away from the optimum in expectation at each step. In such a case, it is unlikely that the algorithm is efficient for the function it is attempting to optimize. Rather, an exponential lower bound on the runtime could probably be proved. The negative-drift theorem is the standard technique used for the purpose.

Apart from a negative drift, the theorem also requires a second condition showing that large jumps are unlikely. The intuitive reason for this second condition is that if large jumps were possible, then the algorithm could maybe be able to jump to the optimum even if in expectation it is drifting away. For technical reasons also, large jumps heading away from the optimum need to be excluded (see [63]) (Fig. 11).

Theorem 21 (Negative-Drift Theorem). *Let X_t, $t \geq 0$, be the random variables describing a Markov process over the state space S and denote the increase in distance in one step $D_t(i) := (X_{t+1} - X_t | X_t = i)$ for $i \in S$ and $t \geq 0$. Suppose there exists an interval $[a, b]$ of the state space and three constants $\delta, \varepsilon, r > 0$ such that for all $t \geq 0$ the following conditions hold:*

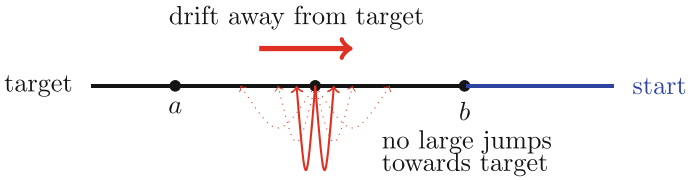


Fig. 11 An illustration of the two conditions of the negative-drift theorem

1. $\Delta_t(i) = \mathbb{E}[D_t(i)] \geq \varepsilon$ *for* $a < i < b$
2. $\Pr(|D_t(i)|) = j) \leq \frac{1}{(1+\delta)^{j-r}}$ *for* $i > a$ *and* $j \geq 1$

Then there exists a constant c^* *such that for* $T := \min\{t \geq 0 : X_t \leq a | X_0 \geq b\}$ *it holds* $Pr(T \leq 2^{c^*(b-a)}) = 2^{-\Omega(b-a)}$.

The following theorem is an example application showing exponential runtime of the (1+1) EA for the NEEDLE function.

Theorem 22. *Let* $\eta > 0$ *be constant. Then there is a constant* $c > 0$ *such that with probability* $1 - 2^{-\Omega(n)}$ *the (1+1) EA on* NEEDLE *creates only search points with at most* $n/2 + \eta n$ *ones in* 2^{cn} *steps.*

Proof. Apply the negative-drift theorem and set the interval $[a, b]$ as $a := n/2 - 2\gamma n$ zeroes and $b := n/2 - \gamma n$ zeroes, with γ a positive constant. By Chernoff bounds, the probability that the initial random search point has less than $n/2 - \gamma n$ zeroes is $e^{-\Omega(n)}$, implying that the algorithm starts outside the interval $[a, b]$ as desired. The remainder of the proof shows that the two conditions of the drift theorem hold.

Since the interval is a plateau, all created points have the same fitness and are accepted. Given that the current search point has i zeroes and that each bit is flipped with probability $1/n$, the drift is

$$\Delta_t(i) = \frac{n-i}{n} - \frac{i}{n} = \frac{n-2i}{n} \geq 2\gamma := \varepsilon,$$

where the last inequality is achieved because in the interval there are always at least b zeroes, i.e., $i \geq n/2 - \gamma n$. Concerning the second condition for standard bit mutation, the probability of flipping j bits decreases exponentially with the number of bits j

$$\Pr(|\Delta_t(i)| \geq j) \leq \binom{n}{j}\left(\frac{1}{n}\right)^j \leq \left(\frac{n^j}{j!}\right)^j \leq \frac{1}{j!} \leq \left(\frac{1}{2}\right)^{j-1}$$

and the condition holds for $\delta = r = 1$. By the negative-drift theorem, the optimum is found within $2^{c*(b-a)} = 2^{cn}$ steps with probability at most $2^{-\Omega(b-a)} = 2^{-\Omega(n)}$. □

The proof can be generalized to plateau blocks of functions of unitation positioned such that $k + m < (1/2 - \varepsilon)n$.

Theorem 23. *The time for the (1+1) EA to optimize a* plateau block *of length m at position such that* $k + m < (1/2 - \varepsilon)n$ *is at least* $2^{\Omega(m)}$ *with probability at least* $1 - 2^{-\Omega(m)}$.

Proof. The proof follows the same arguments as the proof of Theorem 22 by setting $b := m + k$ zeroes and $a := k$ zeroes. □

Conclusions

Drift analysis dates back to 1892 when it was applied for the analysis of stability equilibria in ordinary differential equations [56]. The first use of drift techniques for the runtime analysis of stochastic search heuristics was performed by Sasaki and Hajek to analyze simulated annealing on instances of the maximum matching problem [70]. Drift analysis was applied in a considerable number of applications in evolutionary computation after He and Yao introduced the additive drift technique to the field [38]. Several extensions for the analysis of more sophisticated algorithms and processes have been since devised. The multiplicative drift method introduced in section "Multiplicative Drift Theorem" is due to Doerr et al. [25]. An improved version introduced by Doerr and Goldberg allows to derive bounds also on the probability to deviate from the expected runtime [22]. Witt subsequently introduced a multiplicative drift theorem to achieve lower bounds on the runtime [78]. While the multiplicative drift theorem may only be used when the drift is linear, a variable drift theorem was introduced to deal with cases where the expected progress is nonlinear [42] with respect to the current position. A version for lower bounds has also been derived [26]. A general variable drift theorem with tail bounds was introduced in [48]. This theorem subsumes most existing drift theorems, and a special case of this theorem was given in section "Variable Drift Theorem" (see Theorem 19).

Oliveto and Witt proposed the negative-drift theorem presented in section "Negative-Drift Theorem" to derive exponential lower bounds on the runtime of randomized search heuristics [62]. This theorem was the main tool used in the first analysis of the standard *simple genetic algorithm (SGA)* [64,65]. Finally, Lehre extended the negative-drift theorem to allow a systematic analysis of population-based heuristics using non-elitist selection mechanisms [45]. For an extensive treatment of drift analysis, we refer the reader to the book chapter by Lengler [49].

Final Overview

This chapter has presented the most commonly used techniques for the time complexity analysis of stochastic search heuristics. Example applications have been shown concerning simple evolutionary algorithms on classical test problems. The same techniques have allowed the obtainment of time complexity results on more sophisticated function classes, such as standard combinatorial optimization problems. The reader is referred to [61] for an overview of such advanced results concerning evolutionary algorithms and ant colony optimization and to [60] for an overview of the results in terms of parameterized complexity. Several other techniques have been used to analyze stochastic search heuristics, such as typical runs, family trees, martingales, probability generating functions, and branching

processes. For an introduction to these tools for the analysis of evolutionary algorithms, the reader is referred to the monograph by Jansen [40] which covers a more extensive set of techniques than is possible in this book chapter. Considerable progress has been made in the analysis of populations and diversity which are at the heart of evolutionary computation. We refer to Sudholt's book chapter for an overview of the theory of the mechanisms used to artificially enforce a diverse population [72]. Corus et al. have recently shown how it is possible to set the selective pressure of population-based steady-state algorithms to obtain an appropriate balance between exploration and exploitation far beyond the limits that were previously thought possible, without the need of any artificial diversity enforcing mechanism [11].

Several other aspects of stochastic search optimization have been investigated theoretically, such as simulated annealing, evolution strategies for continuous optimization, particle swarm optimization (▸ Chap. 28, "Particle Swarm Optimization"), memetic algorithms (▸ Chap. 27, "Memetic Algorithms"), and multi-objective optimization techniques (▸ Chap. 10, "Multiobjective Optimization"). These topics are overviewed in [1]. Significant results have been achieved in areas covered in the recent book collection edited by Doerr and Neumann [23]. Among these, certainly worth mentioning are the considerable advances in black-box optimization [20], which may be regarded as a complementary line of research to runtime analysis. Rather than determining the time required by a given heuristic for a problem class, black-box optimization focuses on determining the best possible performance achievable by any search heuristic for that class of problems. A variation to classical runtime analysis is to determine the expected solution quality achieved by a stochastic search heuristic if it is only allowed a predefined budget [41]. Fixed budget computation analyses are driven by the consideration that in practical applications only a predefined amount of resources are available and the hope is to use such resources at best to achieve solutions of the highest quality. Other areas covered within the book collection that have witnessed considerable progress are the theory of estimation of distribution algorithms [43], artificial immune systems [80], and genetic programming [51], for which the very first results are available regarding the time and space complexity for evolving functions with proper input/output behavior [29, 50], as well as an emerging theory for geometric semantic genetic programming [7]. An area that has recently witnessed strong momentum is the theory of parameter control and automated algorithm design (i.e., hyper-heuristics) [21, 28, 44, 52–54, 57, 67, 68], while results have also started emerging regarding the analysis of automated algorithm configurators [35–37] and evolutionary neural architecture search [32, 55].

Cross-References

▸ Ant Colony Optimization: A Component-Wise Overview
▸ Estimation of Distribution Algorithms
▸ Evolutionary Algorithms

- Genetic Algorithms
- Hyper-heuristics
- Memetic Algorithms
- Multiobjective Optimization
- Particle Swarm Optimization
- Theory of Local Search

Acknowledgments P. S. Oliveto is grateful to P. K. Lehre for his contribution to the first edition of this book chapter on which this extended and updated edition is based. P.S. Oliveto is also grateful to D. Corus for reviewing the book chapter.

Competing Interest Declaration The author(s) has no competing interests to declare that are relevant to the content of this manuscript.

References

1. Auger A, Doerr B (eds) (2011) Theory of randomized search heuristics. World Scientific Review, Singapore
2. Benford A, Lehre PK (2024) Runtime analysis of coevolutionary algorithms on a class of symmetric zero-sum games. In: Proceedings of the genetic and evolutionary computation conference (GECCO 2024). ACM, New York, pp 1542–1550
3. Chen T, He J, Sun G, Chen G, Yao X (2009) A new approach for analyzing average time complexity of population-based evolutionary algorithms on unimodal problems. IEEE Trans Syst Man Cybern Part B Cybern 39(5):1092–106
4. Cormen TH, Leiserson CE, Rivest RL, Stein C (2001) Introduction to algorithms. MIT Press, Cambridge
5. Corus D, Oliveto PS (2018) Standard steady state genetic algorithms can hillclimb faster than mutation-only evolutionary algorithms. IEEE Trans Evolut Comput 22(5):720–732
6. Corus D, Oliveto PS (2020) On the benefits of populations for the exploitation speed of standard steady-state genetic algorithms. Algorithmica 82:3676–3706
7. Corus D, Oliveto PS (2024) On the generalisation performance of geometric semantic genetic programming for boolean functions: Learning block mutations. ACM Trans Evolut Learn Optim 4(4):1–13
8. Corus D, Dang D-C, Eremeev AV, Lehre PK (2018) Level-based analysis of genetic algorithms and other search processes. IEEE Trans Evolut Comput 22(5):707–719
9. Corus D, Oliveto PS, Yazdani D (2019) Artificial immune systems can find arbitrarily good approximations for the NP-hard number partitioning problem. Artif Intell 247:180–196
10. Corus D, Oliveto PS, Yazdani D (2019) On inversely proportional hypermutations with mutation potential. In: Proceedings of the genetic and evolutionary computation conference (GECCO 2019), pp 215–223
11. Corus D, Lissovoi A, Oliveto PS, Witt C (2021) On steady-state evolutionary algorithms and selective pressure: why inverse rank-based allocation of reproductive trials is best. ACM Trans Evolut Learn Optim 1(1):1–38
12. Corus D, Oliveto PS, Yazdani D (2021) Fast immune system-inspired hypermutation operators for combinatorial optimization. IEEE Trans Evolut Comput 25(5):956–970
13. Corus D, Oliveto PS, Yazdani D (2025) Fast contiguous somatic hypermutations for single-objective optimisation and multi-objective optimisation via decomposition. In: The 39th annual AAAI conference on artificial intelligence, AAAI-25. AAAI Press
14. Corus D, Oliveto PS, Zheng F (2025) Hybrid selection allows steady-state evolutionary algorithms to control the selective pressure in multimodal optimisation. In: Proceedings of the GECCO 2025 (to appear)

15. Dang, D-C, Lehre PK (2015) Efficient optimisation of noisy fitness functions with population-based evolutionary algorithms. In: Proceedings of the 2015 ACM conference on foundations of genetic algorithms XIII - FOGA '15, pp 62–68. ACM Press, New York
16. Dang D-C, Lehre PK (2015) Simplified runtime analysis of estimation of distribution algorithms. In: Proceedings of the 2015 on genetic and evolutionary computation conference, GECCO '15, pp 513–518. ACM, New York
17. Dang D-C, Friedrich T, Kötzing T, Krejca MS, Lehre PK, Oliveto PS, Sudholt D, Sutton AM (2018) Escaping local optima using crossover with emergent diversity. IEEE Trans Evolut Comput 22(3):484–497
18. Dang D-C, Eremeev A, Lehre PK (2021) Non-elitist evolutionary algorithms excel in fitness landscapes with sparse deceptive regions and dense valleys. In: GECCO '21: proceedings of the genetic and evolutionary computation conference, pp 1133–1141. ACM, New York
19. Doerr B (2020) Probabilistic tools for the analysis of randomized optimization heuristics. In: Doerr B, Neumann F (eds) Theory of evolutionary computation: recent developments in discrete optimization, pp 1–88. Springer International Publishing, Cham
20. Doerr C (2020) Complexity theory for discrete black box optimization heuristics. In: Doerr B, Neumann F (eds) Theory of evolutionary computation: recent developments in discrete optimization, pp 133–212. Springer International Publishing, Cham
21. Doerr B, Doerr C (2020) Theory of parameter control for discrete black-box optimization: Provable performance gains through dynamic parameter choices. In: Doerr B, Neumann F (eds) Theory of evolutionary computation: recent developments in discrete optimization, pp 271–321. Springer
22. Doerr B, Goldberg LA (2013) Adaptive drift analysis. Algorithmica 65(1):1–27
23. Doerr B, Neumann F (2020) Theory of evolutionary computation: recent developments in discrete optimization. Springer, Berlin
24. Doerr B, Johannsen D, Winzen C (2010) Drift analysis and linear functions revisited. In: IEEE congress on evolutionary computation (CEC '10), pp 1967–1974
25. Doerr B, Johannsen D, Winzen C (2010) Multiplicative drift analysis. In: GECCO '10: proceedings of the 12th annual conference on genetic and evolutionary computation, pp 1449–1456. ACM, New York
26. Doerr B, Fouz M, Witt C (2011) Sharp bounds by probability-generating functions and variable drift. In: GECCO '11: proceedings of the 13th annual conference on genetic and evolutionary computation, pp 2083–2090. ACM, New York
27. Doerr B, Le HP, Makhmara R, Nguyen TD (2017) Fast genetic algorithms. In: GECCO '17: proceedings of the genetic and evolutionary computation conference, pp 777–784. ACM, New York
28. Doerr B, Lissovoi A, Oliveto PS, Warwicker JA (2018) On the runtime analysis of selection hyper-heuristics with adaptive learning periods. In: Proceedings of the genetic and evolutionary computation conference, GECCO '18, pp 1015–1022. ACM
29. Doerr B, Lissovoi A, Oliveto PS (2019) Evolving Boolean functions with conjunctions and disjunctions via genetic programming. In: Proceedings of the genetic and evolutionary computation conference, GECCO 2019, pp 1003–1011. ACM, New York
30. Doerr B, Echarghaoui A, Jamal M, Krejca MS (2024) Runtime analysis of the (μ+1) ga: Provable speed-ups from strong drift towards diverse populations. In: The 38th annual AAAI conference on artificial intelligence (AAAI-24), pp 20683–20691. AAAI
31. Droste S, Jansen T, Wegener I (2002) On the analysis of the (1+1) evolutionary algorithm. Theor Comput Sci 276:51–81
32. Fischer P, Warwicker JA, Witt C (2024) A runtime analysis of bias-invariant neuroevolution and dynamic fitness evaluation. In: Proceedings of the genetic and evolutionary computation conference, pp 1560–1568. ACM, New York
33. Goldberg DE (1989) Genetic algorithms in search, optimization and machine learning. Addison-Wesley, Reading
34. Goldberg DE, Deb K (1991) A comparative analysis of selection schemes used in genetic algorithms. In: Proceedings of foundations of genetic algorithms I (FOGA 1991), pp 69–93

35. Hall GT, Oliveto PS, Sudholt D (2020) Analysis of the performance of algorithm configurators for search heuristics with global mutation operators. In: Proceedings of the 2020 on genetic and evolutionary computation conference, GECCO '20, pp 823–831. ACM, New York
36. Hall GT, Oliveto PS, Sudholt D (2020) Fast perturbative algorithm configurators. In: Proceedings of the 16th international conference on parallel problem solving from nature (PPSN XVI). Lecture notes in computer science, vol 12269, pp 19–32. Springer
37. Hall GT, Oliveto PS, Sudholt D (2022) On the impact of the performance metric on efficient algorithm configuration. Artif Intell 303:103629
38. He J, Yao X (2001) Drift analysis and average time complexity of evolutionary algorithms. Artif Intell 127(1):57–85
39. Holland J (1975) Adaptation in natural and artificial systems. University of Michigan Press, Ann Arbor
40. Jansen T (2013) Analyzing evolutionary algorithms. The computer science perspective. Springer, Berlin
41. Jansen T (2020) Analysing stochastic search heuristics operating on a fixed budget. In: Doerr B, Neumann F (eds) Theory of evolutionary computation: recent developments in discrete optimization, pp 249–270. Springer International Publishing, Cham
42. Johannsen D (2012) Random combinatorial structures and randomized search heuristics. PhD Thesis, University of Saarland
43. Krejca M, Witt C (2020) Theory of estimation of distribution algorithms. In: Doerr B, Neumann F (eds) Theory of evolutionary computation: recent developments in discrete optimization, pp 405–442. Springer International Publishing, Cham
44. Krejca M, Witt C (2024) A flexible evolutionary algorithm with dynamic mutation rate archive. In: Proceedings of the GECCO 2024, pp 1578–1586
45. Lehre PK (2010) Negative drift in populations. In: PPSN (1), pp 244–253
46. Lehre PK (2011) Fitness-levels for non-elitist populations. In: Proceedings of the 13th annual conference on genetic and evolutionary computation - GECCO '11, p 2075. ACM Press, New York
47. Lehre PK, Lin S (2025) Towards runtime analysis of population-based co-evolutionary algorithms on sparse binary zero-sum games. In: The 39th annual AAAI conference on artificial intelligence. AAAI
48. Lehre PK, Witt C (2013) Concentrated hitting times of randomized search heuristics with variable drift. In: Proceedings of the 25th international symposium on algorithms and computation - ISAAC 2014, pp 686–697
49. Lengler J (2020) Drift analysis. In: Doerr B, Neumann F (eds) Theory of evolutionary computation: recent developments in discrete optimization, pp 89–132. Springer International Publishing, Cham
50. Lissovoi A, Oliveto PS (2019) On the time and space complexity of genetic programming for evolving boolean conjunctions. J Artif Intell Res 66:655–689
51. Lissovoi A, Oliveto PS (2020) Computational complexity analysis of genetic programming. In: Doerr B, Neumann F (eds) Theory of evolutionary computation: recent developments in discrete optimization, pp 475–518. Springer International Publishing, Cham
52. Lissovoi A, Oliveto P, Warwicker JA (2020) How the duration of the learning period affects the performance of random gradient selection hyper-heuristics. In: Proceedings of the AAAI conference on artificial intelligence, AAAI' 20, pp 2376–2383. MIT Press
53. Lissovoi A, Oliveto PS, Warwicker JA (2020) Simple hyper-heuristics control the neighbourhood size of randomised local search optimally for LeadingOnes. Evolut Comput 28(3):437–461
54. Lissovoi A, Oliveto PS, Warwicker JA (2023) When move acceptance selection hyper-heuristics outperform metropolis and elitist evolutionary algorithms and when not. Artif Intell 314:103804
55. Lu Z, Bian C, Qian C, Sun Y (2024) Runtime analysis of population-based evolutionary neural architecture search for a binary classification problem. In: Proceedings of the genetic and evolutionary computation conference, pp 358–366. ACM, New York

56. Lyapunov AM (1892) The general problem of the stability if motion (in Russian). Kharkov Mathematical Society, Kharkiv
57. Ma Y, Oliveto PS, Warwicker JA (2025) Random gradient hyper-heuristics can learn to escape local optima in multimodal optimisation. In: Proceedings of the GECCO 2025 (to appear)
58. Mitzenmacher M, Upfal E (2005) Probability and computing: randomized algorithms and probabilistic analysis. Cambridge University Press, Cambridge
59. Motwani R, Raghavan P (1995) Randomized algorithms. Cambridge University Press, Cambridge
60. Neumann F, Sutton A (2020) Parameterized complexity analysis of randomized search heuristics. In: Doerr B, Neumann F (eds) Theory of evolutionary computation: recent developments in discrete optimization, pp 213–243. Springer International Publishing, Cham
61. Neumann F, Witt C (2010) Bioinspired computation in combinatorial optimization – algorithms and their computational complexity. Springer, Berlin
62. Oliveto PS, Witt C (2011) Simplified drift analysis for proving lower bounds in evolutionary computation. Algorithmica 59(3):369–386
63. Oliveto PS, Witt C (2012) Erratum: Simplified drift analysis for proving lower bounds in evolutionary computation. Technical Report arXiv:1211.7184, arXiv preprint
64. Oliveto PS, Witt C (2014) On the runtime analysis of the simple genetic algorithm. Theor Comput Sci 545:2–19
65. Oliveto PS, Witt C (2015) Improved time complexity analysis of the simple genetic algorithm. Theor Comput Sci 605:21–41
66. Oliveto PS, Sudholt D, Witt C (2022) Tight bounds on the expected runtime of a standard steady state genetic algorithm. Algorithmica 84:1603–1658
67. Rajabi A, Witt C (2022) Self-adjusting evolutionary algorithms for multimodal optimization. Algorithmica 84:1694–1723
68. Rajabi A, Witt C (2023) Stagnation detection with randomized local search. Evolut Comput 31(1):1–29
69. Rudolph G (1998) Finite Markov chain results in evolutionary computation: a tour d'horizon. Fundam Inf 35(1–4):67–89
70. Sasaki GH, Hajek B (1988) The time complexity of maximum matching by simulated annealing. J Assoc Comput Mach 35(2):387–403
71. Sudholt D (2010) General lower bounds for the running time of evolutionary algorithms. In: Proceedings of the 11th international conference on parallel problem solving from nature (PPSN XI). Lecture notes in computer science, vol 6238, pp 124–133. Springer
72. Sudholt D (2020) The benefits of population diversity in evolutionary algorithms: A survey of rigorous runtime analyses. In: Doerr B, Neumann F (eds) Theory of evolutionary computation: recent developments in discrete optimization, pp 359–404. Springer International Publishing, Cham
73. Syswerda G (1989) Uniform crossover in genetic algorithms. In: Proceedings of the third international conference on genetic algorithms, pp 2–9
74. Syswerda G (1991) A study of reproduction in generational and steady-state genetic algorithms. In: Proceedings of foundations of genetic algorithms I (FOGA 1991), pp 94–101
75. Wegener I (2002) Methods for the analysis of evolutionary algorithms. In: Sarker R, Yao X (eds) Evolutionary optimization, pp 125–141. Kluwer Academic, New York
76. Whitley D (1989) The genitor algorithm and selection pressure: why rank-based allocation of reproductive trials is best. In: Proceedings of the third international conference on genetic algorithms, pp 116–121
77. Witt C (2006) Runtime analysis of the (μ+1) EA on simple pseudo-boolean functions. Evolut Comput 14(1):65–86
78. Witt C (2012) Optimizing linear functions with randomized search heuristics - the robustness of mutation. In: Dürr C, Wilke T (eds) 29th international symposium on theoretical aspects of computer science (STACS 2012). Leibniz international proceedings in informatics (LIPIcs), vol 14, pp 420–431, Dagstuhl. Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik.

79. Witt C (2013) Tight bounds on the optimization time of a randomized search heuristic on linear functions. Comb Prob Comput 22(2):294–318
80. Zarges C (2020) Theoretical foundations of immune-inspired randomized search heuristics for optimization. In: Doerr B, Neumann F (eds) Theory of evolutionary computation: recent developments in discrete optimization, pp 443–474. Springer International Publishing, Cham

Part V
Applications

City Logistics

38

Jaume Barceló, Hanna Grzybowska, and Jesús Arturo Orozco

Contents

Introductory Remarks on City Logistics: The Nature of the Problem . . . 1176
Logistics and City Logistics . . . 1176
The Relevance of City Logistics . . . 1177
A Systems Approach to City Logistics . . . 1178
Organizational Aspects and Decision-Making in City Logistics . . . 1180
Fleet Management and ICT Applications . . . 1183
Strategic Decisions in City Logistics . . . 1187
Two-Echelon Single-Source Location Problems . . . 1187
Two-Echelon Location Routing . . . 1193
Operational Decisions: Routing Problems . . . 1194
Pickup and Delivery Vehicle Routing Problem with Time Windows . . . 1194
Real-Time Management . . . 1203
Computation of Feasibility . . . 1205
Heuristic Approach . . . 1208
Extensions . . . 1213
Concluding Remarks . . . 1216
Cross-References . . . 1216
References . . . 1216

J. Barceló (✉)
Department of Statistics and Operations Research, Universitat Politècnica de Catalunya, Barcelona, Spain
e-mail: jaume.barcelo@upc.edu

H. Grzybowska
School of Civil and Environmental Engineering, Research Centre for Integrated Transport Innovation, University of New South Wales, Sydney, NSW, Australia
e-mail: h.grzybowska@unsw.edu.au

J. A. Orozco
Department of Operations Management, IPADE Business School, Mexico City, Mexico
e-mail: jaorozco@ipade.mx

© Springer Nature Switzerland AG 2025

R. Martí et al. (eds.), *Handbook of Heuristics*,
https://doi.org/10.1007/978-3-032-00385-0_55

Abstract

This chapter provides an introductory overview of city logistics systems, highlighting the specific characteristics that make them different from general logistics problems. It analyzes the types of decisions involved in managing city logistics applications, from strategic, tactical, and operational, and identifies the key models to address them. This analysis identifies types of problems, location, location routing, and variants of routing problems with time windows, all those with ad hoc formulations, derived from the constraints imposed by policy and operational regulations, technological conditions, or other specificities of urban scenarios, which result in variants of the classical models that, for its size and complexity, become a fertile field for metaheuristic approaches to define algorithms to solve the problems. Some of the more relevant cases are studied in this chapter, and guidelines for further and deeper insights on other cases are provided to the reader through a rich set of bibliographical references.

Keywords

City logistics · Decision support systems · Location models · Metaheuristics · Vehicle routing with time windows

Introductory Remarks on City Logistics: The Nature of the Problem

City logistics systems exhibit intrinsic characteristics that differentiate them from the general logistics systems, as part of the supply chain, in terms of the specificities of the scenarios where they occur (urban and large metropolitan areas), as well as for the economic relevance that they have as part of the freight distribution system. This section is aimed at explaining and highlighting these two crucial aspects.

Logistics and City Logistics

Logistics, as defined by the Council of Logistics Management CLM [13], is "that part of the supply chain process that plans, implements, and controls the efficient, effective flow and storage of goods, services, and related information from the point of origin to the point of consumption in order to meet customers' requirements." However, when logistics activities take place in urban areas, they show unique characteristics making them different from the general logistics activities. Thus, in order to differentiate the two phenomena and to highlight the special characteristics, the transport in urban areas, and specifically the freight flows associated with the supply of city centers with goods, is usually referred to as "city logistics," "urban freight distribution," or "last mile logistics."

Taniguchi et al. [51] define city logistics as "the process of totally optimizing the logistics and transport activities by private companies in urban areas while considering the traffic environment, traffic congestion and energy consumption within the framework of a market economy." This definition can be updated by adding to the concept of the energy consumption the contemporary concerns on the environmental impacts such as emissions, urban noise, vibration, etc., generated by logistic vehicles in urban areas. These impacts affect not only the economy but also the quality of life of the city residents.

An analysis of city logistics activities, and how they are performed, results with an immediate identification of their specific intrinsic characteristics. It allows for establishing substantial differences between logistics and city logistics justifying the entity of city logistics as an individual field with proper identity. Some of the main city logistics characteristics include:

- Spatial restrictions:
  - Urban microstructure determined by the urban network (e.g., one-way streets, dead-end streets, etc.) causes the paths between customers, or between the depot and the customers, to differ depending on the order of the visits.
  - Limited vehicle access. Most cities mandate limited access for some areas of the city for specific types of vehicles during specific time windows.
  - Small-quantity deliveries.
  - High density of delivery points.
- Traffic infrastructure: traffic schemes banning specific turnings, traffic lights and their associated traffic control plans, etc.
- Environmental concerns and sensitivities:
  - Growing role of small specialized urban vehicles (i.e., environmentally friendly vehicles commonly referred to as green vehicles) as a consequence of sustainable urban policies implying energy savings, reduction of emissions, noise, etc.
  - Low automation and critical human role when manual deliveries are necessary.
  - High operational and environmental costs, as a consequence of the human involvement among other reasons.

The Relevance of City Logistics

Urban logistics operations consist of the set of activities related with the distribution of goods and provision of services within an urban area. There is a wide range of examples of urban logistics that includes parcel delivery, material collection, goods storage, waste collection, home delivery services, or electrical appliance reparation services, among others. Current urban activities are far away from what they were a couple of decades ago. New challenges have emerged, new technologies have been developed, and urban population dynamics have changed.

The road system is the main transportation modality in most countries. In Europe, 68% of goods transportation is made by roads (BESTUFS report [6]). Countries with less developed rail networks tend to have higher road utilization. Moreover, a high proportion of all goods are delivered within the cities. Cities as London and Dublin estimate this proportion in slightly more than 40% (BESTUFS report [6]), while almost 70% of the deliveries are concentrated in Tokyo (Taniguchi et al., [51]). In summary, road transportation is the **main** freight transportation mode, and **high proportion** of the **total** goods transportation happens in **cities**.

The significant impact of commercial vehicles in daily traffic is evident. According to the BESTUFS report [6], about a fifth of a city's traffic flow is made up of commercial trucks. In London, freight transport accounts for 20–26% of the total traffic, while in Italy this proportion is estimated to be 18%. In the same study, it is reported that urban freight in French cities represents between 13% and 20% of the total traffic. Overall, the BESTUFS [6] estimates that between 15% and 20% of the average flow in cities during rush hours correspond to logistics fleets (i.e., light goods vehicles, heavy goods vehicles, etc.). Consequently, logistics fleets are a net contributor to traffic congestion in urban and metropolitan areas and have a relevant impact on energy consumption and on the quality of life in cities.

Although urban freight distribution represents a small fraction of the total transportation length, the Council of Supply Chain Management Professionals, Goodman [29] estimates that the last mile cost accounts for about 28% of the total transportation cost. In a study made by the European Logistics Association and AT Kearney [21], it is shown that transportation activities account for approximately 43% of the total logistics costs. Assuming that the cost of delivering within urban areas (i.e., the last mile cost) is 28%, we can then estimate that the cost of urban freight activities represents about 12% of the total logistics costs.

A Systems Approach to City Logistics

The complexity of city logistics systems should be addresses from a systemic perspective accounting for all of its components and, what is more important, their interactions determining the dynamics of the system and the way it behaves. Namely, it should be addressed in terms of the conditions, operational but also technological, which are imposed by the relationships among stakeholders, which are synthetically described in this section.

Any systemic approach to city logistics must look at the system as a whole, identify its components, and account for their mutual interactions. For example, consider the impacts that city logistics activities have on traffic congestion, and at the same time, consider how in return the city logistics activities are impacted by traffic congestion and operational constraints of the urban areas.

Thus, the *city logistics models must account for the two-way interaction*. They should include the effects of the city logistics deliveries and commercial activities on urban traffic congestion, and conversely they should include operational constraints in routing and logistics optimization models considering time-varying

traffic congestion allowing defining how city logistics activities are impacted by traffic congestion.

Notwithstanding, a systemic approach should neither forget that urban freight distribution is mainly a private sector activity, designed to maximize the profits, which takes place in a public scenario under rules and constraints determined by public authorities. In the words of Boudoin et al. [10], "today, the city as a geographic and economical space is the centre of attention of decision makers of the private and public sectors, considering that the performance of the logistics system depends also greatly on the decisions taken by these two categories of actors. The urban logistics spaces are in the core of the goods distribution device, as they are interfaces between interurban and urban, private and public, producer and consumer." A systemic approach must also take into account that city logistics is a scenario with various stakeholders with possibly different and conflicting interests.

Public stakeholders will usually be interested in achieving social, economic, environmental, or energetic objectives, while private stakeholders, namely, private shippers and freight carriers, aim to reduce their freight costs and to optimize their traffic flows in accordance with their specific needs, which are not conforming to the objectives of an overall optimization. Therefore, a systemic approach to city logistics must:

1. Be supported by a holistic view in modeling approaches integrating urban and logistic planning which will allow innovative approaches to urban logistics solution.
2. Suitable to be applied in practice in terms of a framework enabling the understanding between stakeholders, fostering new ways of stakeholder collaboration, and providing policy frameworks allowing sustainable business models.
3. Become operational in terms of decision support tools to efficiently implement the desirable policies.

In other words, the conceptual approach to such systemic view must be able to appropriately account for the roles of the main stakeholders and the relationships between them as highlighted in the conceptual diagram in Fig. 1.

Freight operators supply their services under supply conditions and contracts with customers imposing constraints on:

- Fleet routing and scheduling.
- Service times and delivery conditions that could be determined by the characteristics of the delivery (i.e., loading and unloading) point.
- Service time windows imposed either by the customer requirements or by the regulatory conditions determined by local authorities.

The supply regulations established by the local government strongly determine the way in which logistics fleets operate. The restrictions to access inner city, namely, in cities with historical centers, reinforced in most cases by access control systems, usually impose time constraints that determine the routings and scheduling,

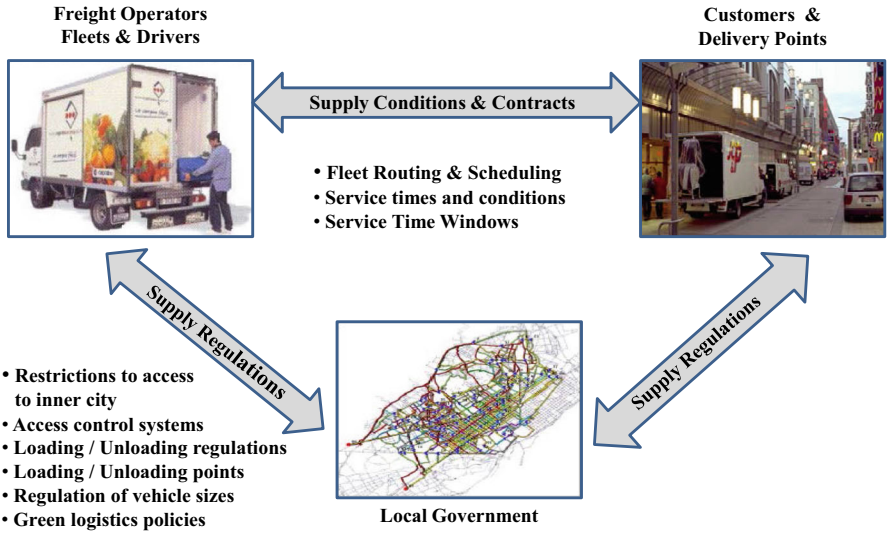


Fig. 1 Stakeholders' interaction in city logistics operations

which can also be affected by the existence, or not, of loading/unloading points and regulations on their use. Furthermore, the regulations on vehicle sizes authorized to operate in urban areas, or green logistics policies imposing thresholds on the acceptable levels of emissions, could determine the fleet compositions.

Organizational Aspects and Decision-Making in City Logistics

The proposed systemic view must be translated in terms of a model, suitably representing the system, which, further than providing a deep understanding on how the system works, should provide support to any type of operational decisions on the system. Two key aspects in this process are, respectively, identify and understand the main logistics operations and identify and distinguish the decision levels in the management of the city logistics system. This view is neatly aligned with the key approach of Operations Research: acquire enough knowledge about a system to understand how the system works, and formulate the system's knowledge in terms of modeling hypothesis that can be subsequently translated in terms of a formal model (usually a mathematical model) which can be used to make decisions about the system. Therefore, understanding the nature of the decisions in city logistics, which of them depend on the nature of the system, and which are conditioned by the relationships among stakeholders is part of such knowledge acquisition to build models of city logistics. Decision levels for fleet management applications (Goel [28]) are primarily:

- **Strategic:**
  - Decisions that usually concern a large part of the organization, have major financial impact, and may have long-term effects: typically concern the design of the transportation system.
  - The **size and mix** of vehicle fleet and equipment.
  - The **type and mix** of transportation services offered.
  - The territory coverage including **terminal location**.
  - Strategic alliances and cooperation, including the integration of information systems.
- **Tactical:**
  - Concerns short- or medium-term activities: typically involve decisions about how to effectively and efficiently use the existing infrastructure and how to organize operations according to strategic objectives.
  - Equipment acquisition and replacement.
  - Capacity adjustments in response to demand forecasts.
  - Static pricing and pricing policies for contract and spot pricing.
  - Acquisition of regularly requested services, including pricing and provisional routing.
  - Long-term driver to vehicle assignments.
  - Cost and performance analysis.
- **Operational:**
  - Commercial vehicle operations underlay a variety of external influences which cannot be foreseen: real-time management decisions have to be made to appropriately react on discrepancies between planned and actual state of the transportation system.
  - Load acceptance.
  - Real-time dispatching considering the actual state of transportation system.
  - Instructing drivers about their tasks.
  - Monitoring the transportation processes, including tracking and tracing and arrival time estimations.
  - Incident management.
  - Observing the state of order processing.
- **Real Time:**
  - Concerns all activities needed to monitor, control, and plan transportation processes.
  - The required information that dispatchers must collect to manage the fleet include vehicle positions and traffic conditions.
  - The decisions that dispatchers must make to manage the fleet include diversion of vehicles from current route to new destinations and insertion of customers into predefined routes.
  - The most challenging task is the generation of dynamic schedules.
  - Determination of plans indicating which vehicle should visit, pick up, deliver, or service a customer and at what time.

As a consequence of the fact that the stakeholders can play different roles in the city logistics scenarios, they can adopt various organizational forms. These organizational forms can be individual with no self-coordination, individual but with different degrees of coordination in which the private stakeholders are the main actors, or super-coordinated where public organizations play the main role currently referred to as goods distribution centers. Each scenario implies different levels of decision from the long-term strategic decisions on the locations of warehouses or distribution centers to the operational decisions concerning fleets and routing and scheduling of fleet vehicles. Moreover, the real-time decisions on rerouting and rescheduling vehicles as far as traffic and operational conditions change become each time more relevant due to technological evolution. Efficiency in these scenarios may be achieved through better fleet management practices, rationalization of distribution activities, traffic control, freight consolidation/coordination, and deployment of intermediate facilities [36].

This last measure plays a fundamental role in most of the city logistics projects based on distribution system, where transportation to and from an urban area is performed through platforms. These platforms are called city distribution centers (CDC), goods distribution centers, or city terminals (CT) [42]. They are located far from city limits. Freights, directed to a city and arriving by different transportation modes and vehicles, are consolidated at platforms on trucks in charge of final distribution. According to Boccia et al. [8], "these platforms have had a great impact on effectiveness of freight distribution in urban areas, but their use is showing some deficiencies because of two main reasons: their position (often far from final customers) and the constrained structure of urban areas."

To overcome the limits of such single-echelon system, distribution systems have been proposed, where another intermediate level of facilities is added between platforms and final customers. These facilities, referred to as satellites or transit points (Fig. 2), perform no storage activities and are devoted to transfer and consolidate freights coming from platforms on trucks into smaller vehicles, more suitable for distribution in city centers (Crainic et al. [16, 17]). This two-echelon system could determine an increase of costs due to additional operations at satellites. However, these costs should be compensated by freight consolidation, decrement of empty trips, economy of scale, reduction of traffic congestion, and environmental safety.

The model simultaneously determines how many depots (CT or CDC) should be open, their locations at the first echelon, how many vehicles (urban vehicles) are needed to supply the satellites, how many satellites should be open at the second echelon, which vehicles (city freighters) should operate from which open satellites, and which customers each vehicle should service.

Most of these decisions must be supported appropriately and that implies the use of suitable models (Giani et al. [24]). Location models (Daskin and Owen [19]) become the key components of strategic decision-making concerning decisions on warehouse or CT locations. Vehicle routing models are at the core of operational and real-time fleet management decisions and processes. Therefore, they deserve a special attention and become the main operational tools to achieve the objectives of

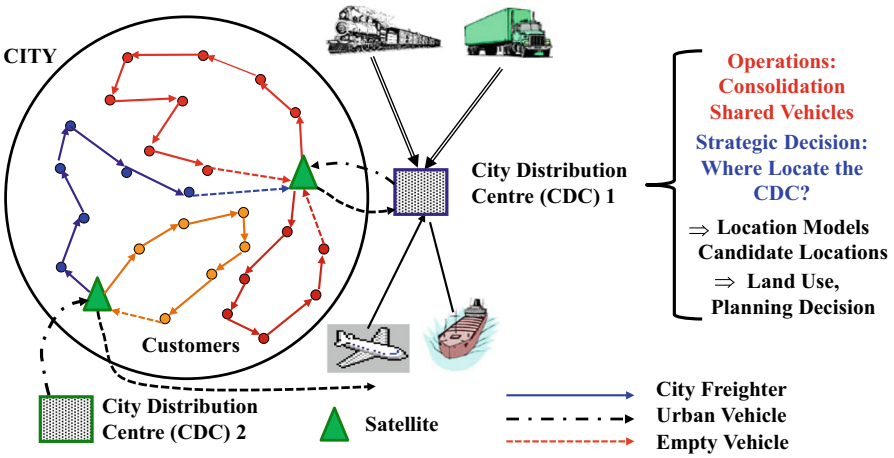


Fig. 2 Two-echelon freight distribution systems and implied decisions

private stakeholders, shippers, and freight carriers. Routing problems in urban areas have specific characteristics that differentiate them from generic routing problems. Bodin et al. [9] have coined the term "street routing" to highlight these differences.

Fleet Management and ICT Applications

We have already highlighted the key role of decision-making in city logistics, and the role of models to support strategic decisions has been highlighted in the previous section. However, the tactical and operational decisions should not be forgotten, as they usually determine the quality of the service, which is highly appreciated by customers. Operational decisions in urban areas are changing very fast due to the impacts of the new telecommunication technologies. Further than enabling faster and more flexible operational modes, the new telecommunication technologies induce deep sociological changes in customers' uses, which in turn foster a quick evolution of management policies. The implications of the latter will be discussed in this section.

The growing importance of the real-time fleet management applications is mainly due to the recent significant evolution of pervasive automatic vehicle location (AVL) technologies (Goel [28]*). Fleet management in urban areas has to explicitly account for the dynamics of traffic conditions leading to congestions and variability in travel times severely affecting the distribution of goods and the provision of services.* An efficient management should be based on decisions accounting for all factors conditioning the problem: customers' demands and service conditions (i.e., time windows, service times, and others), fleet operational conditions (e.g., positions, states and availabilities of vehicles, etc.), and traffic conditions.

Instead of making decisions based on trial and error and operator's experience, a sounder procedure would be to base the decisions on the information provided by a decision support system (DSS). Regan et al. [47, 48] provide a conceptual framework for the evaluation of real-time fleet management systems which considers dynamic rerouting and scheduling decisions implied by operations with real-time information as new orders or updated traffic conditions. The recent advances in information and communications technologies (ICT) have prompted the research on dynamic routing and scheduling problems. At present, easy and fast acquisition and processing of the real-time information is feasible and affordable. The information, which becomes the input to dynamic models, captures the dynamic nature of the addressed problems allowing for more efficient dynamic fleet management decisions.

Barceló et al. [4, 5] propose and computationally explore a methodological proposal for a decision support system to assist in the decision-making concerning the real-time management of a city logistics fleet in dynamic environments when real-time information is available. Figure 3 depicts the architecture of such a DSS based on a dynamic router and scheduler. The feasibility of the proposed solutions depends on:

- The availability of the real-time information, which we assume will be provided by ICT applications, combined with the knowledge of the scheduled plan with the current fleet and customer status.

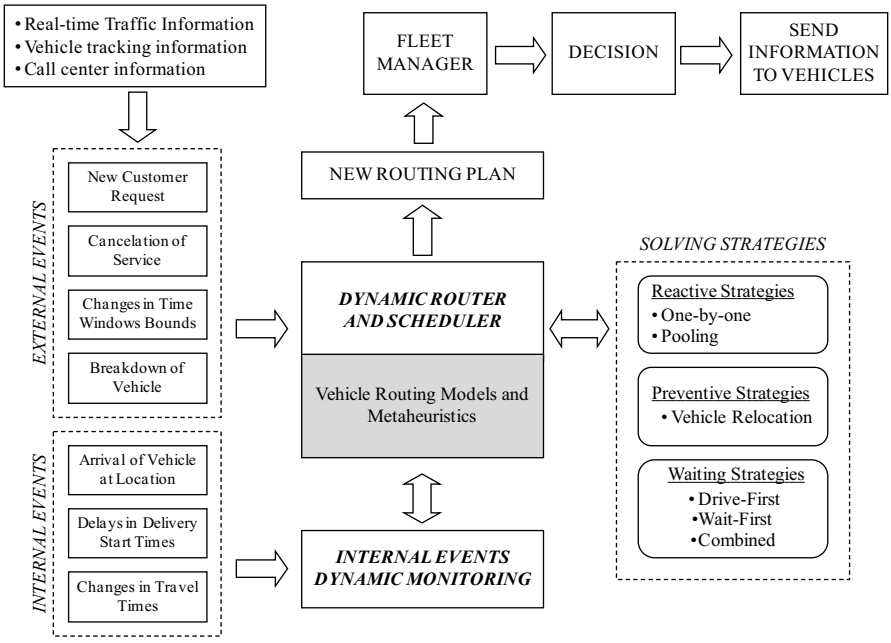


Fig. 3 Logical architecture of a decision support system for real-time fleet management based on a dynamic router and scheduler

- The quality of the vehicle routing models and algorithms to efficiently tackle the available information to provided solutions.

Unlike the classic approach, where routes are planned with the known demand and they are unlikely to be changed throughout the planning period, the real-time fleet management approach assumes that real-time information is constantly revealed to the fleet manager who has to decide whether the current routing plan should be modified or not. The process assumes partial knowledge of the demand. At the beginning of the considered time period (i.e., one work day), an initial schedule for the available fleet to serve the known demand is proposed. This initial operational plan can be modified later on, when the operations are ongoing and new real-time information is available. The new information can concern new demands, unsatisfied demands, changes in the routes due to traffic conditions, changes in the fleet availability (i.e., vehicle breakdowns), etc. It constitutes an input to a dynamic router and scheduler (DRS) which provides a proposal of a new dynamic operational plan prepared on the basis of real-time information.

Figure 4 depicts an example of how the conceptual process described in Fig. 3 can be handled by a dynamic routing and scheduling system. The routes, initially assigned to a set of five vehicles, are identified with various colors. The arrows indicate the order in which customers are to be served according to the initial schedule. We assume that vehicles can be tracked in real time. At time t, after the fleet has started to perform its initial operational plan, a new customer calls requiring a service which has not been scheduled. If real-time information, such as positions

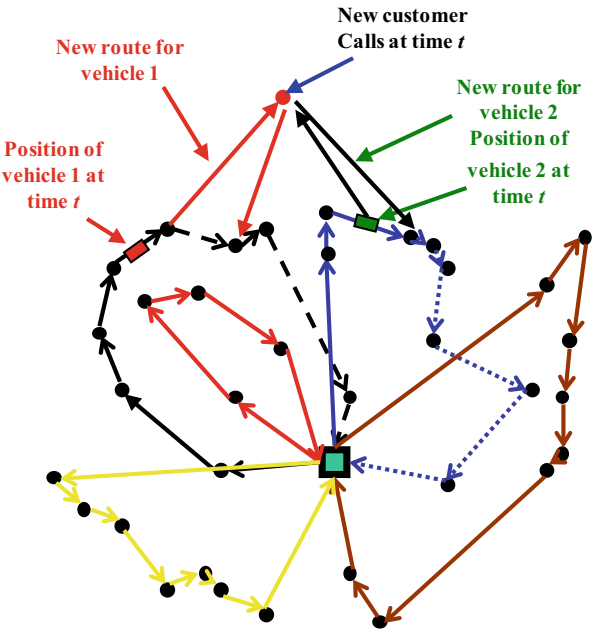


Fig. 4 Dynamic vehicle rerouting in a real-time fleet management system

and states of the vehicles and current and forecasted traffic conditions, is available to the fleet manager, he or she can use it to make a better decision on which vehicle to assign to the new customer and whether the new assignment results in a direct diversion from a route (vehicle 2) or a later scheduling (vehicle 1).

Vehicle routing problem (VRP) techniques constitute a fundament for the transportation, distribution, and city logistics systems modeling. The static version of the VRP has been widely studied in the literature. Among others, an extensive survey on VRP was provided by Fisher [22] and by Toth and Vigo [54]. The static approaches reckon with all the required information to be known a priori and constant throughout the time. However, in most of the real-life cases, a large part of the data is revealed to the decision-maker when the operations are already in progress. Thus, the dynamic VRP assumes partial knowledge of the demand and that new real-time information is revealed to the fleet manager throughout the operational period. A comprehensive review of the dynamic VRP can be found in Ghiani et al. [24]. Psaraftis [45, 46] and Powell et al. [44] contrast the two variants of the problem and clearly distinguish the dynamic VRP from its static version.

A wide variety of vehicle routing problems with time windows (VRPTW) becomes the engine of real-time decision-making processes to address situations where real-time information is revealed to the fleet manager, and he or she has to make decisions in order to modify an initial routing plan with respect to the new needs (Barceló and Orozco [1]). Real-time routing problems are mainly driven by events which are the cause of such modifications. Events take place in time, and their nature may differ according to the type of service provided by a motor carrier. The most common type of event is the arrival of a new order. When a new order is received from a customer, the fleet manager must decide which vehicle to assign to the new customer and what is the new schedule that this vehicle must follow.

A standard practice of introducing the dynamism into the definition of a routing problem is to determine specific features as *time-dependency*. The VRP with time-dependent travel times acknowledges the influence of traffic conditions on the routing planning. An introduction to the problem and a made heuristic development is provided in Malandraki and Daskin [37]. Further algorithm developments for the static VRP with time-dependent travel times can be found in Ichoua et al. [32], Fleishman et al. [23], and Tang [50]. The dynamic VRP with time-dependent information has also been addressed by Chen et al. [12] and Potvin et al. [43].

Thomas and White [53] addressed the problem by assuming that stochastic information was represented by the time of arrival of a new request. The objective of their approach was to find the best policy for selecting the next node to visit that minimizes the total travel time. They called this problem the *anticipatory routing problem* as vehicles anticipate the arrival of a new customer by changing their path if the request is received while they are in transit. The authors assumed that a new service may be delivered only if the reward or benefit is sufficiently high. The problem was modeled as a finite-horizon Markov decision process, and the authors used standard stochastic dynamic programming methods to solve each one of the proposed instances. Thomas [52] extended the results by incorporating waiting strategies with the objective of maximizing the expected number of new customers served. They also modeled the problem using a finite-horizon Markov

decision process. The authors proposed heuristic algorithms for real-time decision-making.

In the approach proposed by Barcelo et al. [2, 3], time-dependent information is generated by means of traffic simulation of a real-world urban network, in order to emulate the role of a real-time traffic information system providing reliable real-time information on traffic conditions and short-term forecasts.

In order to test the management strategies and algorithms assuming the availability of the real-time information, the dynamic traffic simulation model emulates the current travel times estimated as a function of the prevailing traffic conditions and the short-term forecast of the expected evolution of travel times that an advanced traffic information system would provide. This is a basis for making more realistic decisions on the feasibility of providing the requested services within the specified time windows. Furthermore, simulation can also emulate real-time vehicle tracking giving access to positions and availabilities of the fleet vehicles, which is the information required by a DRS.

This example was inspired by the modeling framework proposed by Taniguchi et al. [51], including models needed by the authorities to support long-term planning decisions accounting for the already mentioned interactions between city logistics activities and urban congestion.

Strategic Decisions in City Logistics

The current evolution of city logistics systems has raised the interest in the variants of location models to support the concerned strategic decisions. In many cases they are induced by regulations imposing conditions on the location of the CTs or CDCs and restrictions on the vehicle types allowed to operate in the inner city, leading to organizational restructuring of the services from intermediate satellites. The potential scenarios illustrated in Fig. 2 can be generically addressed by two types of location models, described in this section, depending on whether the routings of the service vehicles are explicitly included in the model or not.

Two-Echelon Single-Source Location Problems

The increasing importance of the e-commerce results with interesting organizational decisions. For example, more and more people purchase products online, but are not usually at home at daytime and cannot accept the deliveries, return the unaccepted deliveries, or deliver the parcels to be sent to other customers. As a consequence, there are required alternative solutions for home deliveries.

An example of such alternative solution was described in BESTUFS II [7]: the Packstations used by Deutsche Post and DHL in Germany. Similar solutions were also deployed in other countries. Usually, they are locker facilities installed in specific locations, which provide automated booths, or locker boxes for self-service collection and delivery of parcels.

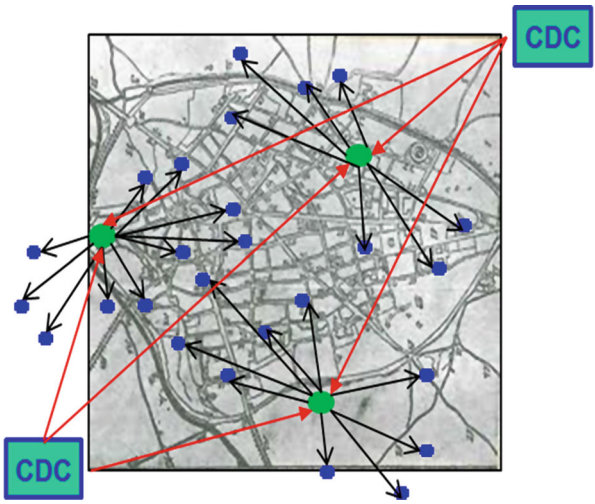


Fig. 5 Two-echelon single-source location problem

In this case, the logistics system can be considered as a particular case of a *two-echelon single-source location problem*, in which special fleet of vehicles service the satellite-automated booths (marked in green in Fig. 5); the special fleet can consist of urban vehicles, such as vans or trucks, fulfilling specific urban regulations regarding sizes and technologies, for example, special vehicles with less than 3.5 tonnes weight, with electrical engines, or propelled by biofuel or low emission fuels. Customers (marked in blue in Fig. 5) travel to the automated locker boxes for service.

In the pilots reported in BESTUFS II [7] the locker boxes have been installed in public spaces (e.g., main station, market place, petrol stations, etc.), and also at parking places of big companies. However, the search for systematic, general solutions can be formulated in terms of two-echelon single-source models. This is the problem arising in a two-stage distribution process, with deliveries being made from first-echelon facilities (e.g., city terminals) to second-echelon facilities (e.g., satellites) and from there to customers. The two-echelon, single-source, and capacitated facility location problem can be considered as an extension of the single-source capacitated facility location problem, dealing with the problem of simultaneously locating facilities in the first and second echelons where:

- Each facility in the second echelon has limited capacity and can be supplied by only one facility in the first echelon,
- Each customer is served by only one facility in the second echelon.

The model simultaneously determines how many depots (CTs or CDCs) should be open, what are their locations at the first echelon, how many vehicles (urban vehicles) are needed to supply the satellites, how many satellites should be open at the second echelon, which vehicles (city freighters) should operate from which open satellites, and which customers each vehicle should service.

Notation:

$I = \{1, 2, \ldots, m\}$: Set of potential facilities (satellites)
$J = \{1, 2, \ldots, n\}$: Set of customers
$K = \{1, 2, \ldots, nn\}$: Set of potential depots (CDCs)
a_j: demand of customer j, $\forall j \in J$
b_i: capacity of facility (satellite) i, $\forall i \in I$
f_{ik}: cost of assigning facility i to depot k, $\forall i \in I, \forall k \in K$
c_{ijk}: cost of facility i from depot k servicing customer j, $\forall i \in I, \forall j \in J, \forall k \in K$
g_k: cost of setting a depot at location k, $\forall k \in K$

Decision variables:

$$Y_{ik} = \begin{cases} 1 & if\ facility\ i\ is\ open\ and\ served\ from\ depot\ k, \forall i \in I, \forall k \in K \\ 0 & otherwise \end{cases}$$

$$X_{ijk} = \begin{cases} 1 & if\ facility\ i\ served\ by\ depot\ k\ services\ customer\ j, \forall i \in I, \\ & \forall j \in J, \forall k \in K \\ 0 & otherwise \end{cases}$$

$$Z_k = \begin{cases} 1 & if\ a\ depot\ is\ set\ a\ location\ k, \forall k \in K \\ 0 & otherwise \end{cases}$$

The model:

$$p : \text{Min} \sum_{i \in I} \sum_{k \in K} \sum_{j \in J} c_{ijk} x_{ijk} + \sum_{i \in I} \sum_{k \in K} f_{ik}\ \ y_{ik} + \sum_{k \in k} g_k z_k \tag{1}$$

subject to

$$\sum_{j \in J} a_j x_{ijk} \leq b_i \quad \forall i \in I, \forall k \in K \tag{2}$$

$$\sum_{i \in I} \sum_{k \in K} x_{ijk} = 1 \quad \forall j \in J \tag{3}$$

$$\sum_{k \in k} y_{ik} \leq 1 \ \ \forall i \in I \tag{4}$$

$$x_{ijk} \leq y_{ik} \ \ \forall i \in I, \forall j \in J, k \in K \tag{5}$$

$$y_{ik} \leq Z_k \quad \forall i \in I, k \in K \tag{6}$$

$$x_{ijk}, y_{ik}, z_k \in \{0, 1\} \quad \forall i \in I, \forall j \in J, k \in K \tag{7}$$

The objective function (1) includes the total cost of assigning customers to facilities, the cost of establishing facilities, and the cost of opening depots. The

side constraint (2) ensures that the customer demand serviced by a facility does not exceed its capacity. The Eq. (3) ensures that each customer is assigned to exactly one facility. Each facility can be serviced by only one depot, as defined by constraint (4). The inequality (5) states that the assignments are made only to open facilities (i.e., customers are allocated only to open facilities). Lastly, the formulation (6) ensures that facilities are allocated only to open depots.

Tragantalerngsak et al. [55] propose a variety of Lagrangian heuristics for this problem. Taking into account the computational results achieved, one of the most performing Lagrangian decompositions analyzed is based on the possibility of strengthening the resulting subproblems as a consequence of the observation that at least one depot must be open. Therefore, without any loss of generality, there can be used the constraint:

$$\sum_{k \in K} z_k \geq 1 \tag{8}$$

Also, there may be included a constraint which forces to open sufficient facilities to supply all customer demands:

$$\sum_{i \in I} \sum_{k \in K} y_{ik} b_i \geq \sum_{j \in J} a_j \tag{9}$$

This will improve the lower bound provided by the relaxation.

By relaxing the constraints (3) and (6), with Lagrangian multipliers λ and ω, respectively, the problem can be separated into the two following Lagrangian subproblems LRxy and LRz:

$$LR_z : \min \sum_{k \in K} \left(g_k - \sum_{i \in I} \omega_{ik} \right) z_k \tag{10}$$

subject to

$$\sum_{k \in K} z_k \geq 1 \tag{11}$$

$$z_k \in \{0, 1\} \quad \forall k \in K \tag{12}$$

and

$$LR_{xy} : \min \sum_{i \in I} \sum_{k \in K} \sum_{j \in J} \left(c_{ijk} - \lambda_j \right) x_{ijk} + \sum_{i \in I} \sum_{k \in K} \left(f_{ik} + \omega_{ik} \right) \tag{13}$$

subject to

$$\sum_{j \in J} a_j x_{ijk} \leq b_i \quad \forall i \in I, \forall k \in K \tag{14}$$

$$\sum_{k \in K} y_{ik} \leq 1 \forall i \in I \tag{15}$$

$$\sum_{i \in I} \sum_{k \in K} b_i y_{ik} \geq \sum_{j \in J} a_j \tag{16}$$

$$x_{ijk} \leq y_{ik} \quad \forall i \in I, \forall j \in J, k \in K \tag{17}$$

$$x_{ijk}, y_{ik} \in \{0, 1\} \quad \forall i \in I, \forall j \in J, k \in K \tag{18}$$

Let $g1_k = g_k - \sum_{i \in I} \omega_{ik}$; then problem LR_{xy} can be solved by inspection:

$$z_k = \begin{cases} 1 \text{ if } g1_k \leq 0 \\ 0 \text{ Otherwise} \end{cases} \tag{19}$$

In the case where all $g1_k > 0$, set $z_{k'} = 1$, where $g1_{k'} = \min_k \{g1_k\}$ and $z_k = 0$, $k \neq k'$.

The problem can be reformulated as

$$\min \sum_{i \in I} \sum_{k \in K} v_{ik} y_{ik} \tag{20}$$

subject to

$$\sum_{k \in K} y_{ik} \leq 1 \quad \forall i \in I \tag{21}$$

$$\sum_{i \in I} \sum_{k \in K} b_i y_{ik} \geq \sum_{j \in J} a_j \tag{22}$$

$$y_{ik} \in \{0, 1\} \quad \forall i \in I, k \in K \tag{23}$$

where $v_i k$ is the optimal solution of

$$\min \sum_{j \in J} \left(c_{ijk} - \lambda_j\right) x_{ijk} + (f_{ik} + \omega_{ik}) \tag{24}$$

subject to

$$\sum_{j \in J} a_j x_{ijk} \leq b_i \quad \forall i \in I, \forall k \in K \tag{25}$$

$$x_{ijk} \in \{0, 1\} \quad \forall i \in I, \forall j \in J, k \in K \tag{26}$$

The problem of finding v_{ik} is now a 0-1 *knapsack problem* involving all facilities, but the fact that each facility can be served only from one depot enables to rewrite the problem as follows:

$$\text{Let } v_i = \min_k \{v_i k\} \tag{27}$$

Solve

$$\min \sum_{i \in I} v_i u_i \tag{28}$$

subject to

$$\sum_{i \in I} b_i u_i \geq \sum_{j \in J} a_j \tag{29}$$

$$u_i \in \{0, 1\} \quad \forall i \in I \tag{30}$$

This is a 0-1 knapsack problem with n variables. The lower bound for problem P is then given by $LBD = v(LR_{xy}) + v(LR_z)$. The upper bound UBD can be found from the solution of the problem LR_z as follows:

Let z_k^* be the solution to LR_z and $\tilde{K} = \left\{k | z_k^* = 1\right\}$. Solve LR_{xy} again but over the depots in $\tilde{K}$. The solution from this restricted set is then used to find a feasible solution. Let $\mathbf{u}_*$ be the solution to this problem, and denote $\tilde{I} = \left\{i | u_i^* = 1\right\}$.

Then

$$y_{ik}^* = \begin{cases} 1 \text{ if } i \in \tilde{I} \text{ and } k \in \tilde{K} \\ 0 \text{ otherwise} \end{cases} \tag{31}$$

The corresponding solution $\boldsymbol{x}_*$ is obtained as the solution giving the v_{ik} coefficients if $i \in \tilde{I}$, $k \in \tilde{K}$, $y_{ik}^* = 1$. Otherwise, $x_{ijk}^* = 0$.

Using these solutions no capacity constraints are violated, but there may be some customers j that were assigned to multiple open facilities or not assigned to any facilities. A *generalized assignment problem* is constructed from these customers, open facilities, and the remaining capacity of open facilities. The solution to these problems reassigns these customers and provides the expected upper bound.

Once the lower bound *LBD* and upper bound *UBD* of the optimal objective function have been calculated, the Lagrangian multipliers λ and ω are updated until a convergence criterion is met.

Two-Echelon Location Routing

There are more variants of the location problems arising from a combination of organizational decisions and urban regulations on the characteristics and conditions of the logistics fleets, as discussed in section "Organizational Aspects and Decision-Making in City Logistics" and illustrated in Fig. 2. A more appealing version of the addressed location problem is the *two-echelon location routing*. In this case, in the first echelon, city freighters service satellites from city distribution centers. In the second echelon, special small urban vehicles serve customers from satellites in downtown regulated zones.

A special case of the two-echelon location routing, also prompted by the growth of e-commerce in urban areas, is the one depicted in Fig. 6. Here, the services to the satellites from the CDCs are provided by special fleets during the night. During the day, special city freighters (e.g., electrical vehicles, tricycles, etc.) serve the customers from the satellites.

Nagy and Salgy [38] elaborated a rather complete state-of-the-art report on this problem. They proposed a classification scheme and analyzed a number of variants as well as the exact and heuristic algorithmic approaches. Drexl and Scheinder [20] updated this state-of-the-art survey, providing an excellent panoramic overview of the current approaches. Metaheuristics dominate the panorama since they look more appropriate to deal with real instances of the problem. Perhaps one of the most appealing is that of Nguyen et al. [39] based on a GRASP approach combined with learning processes and path relinking. Another interesting heuristic, an approach to a relevant variant of the problem based on a Tabu Search accounting for time

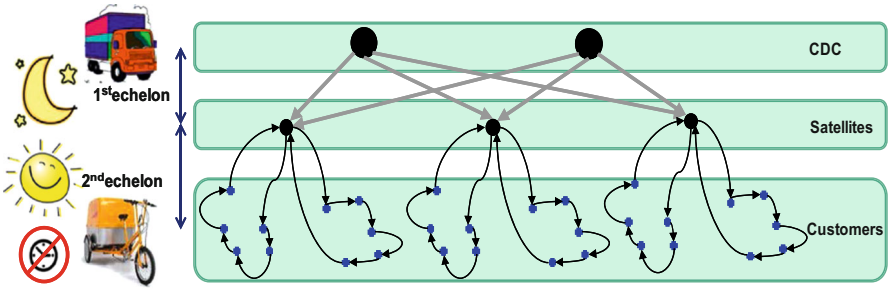


Fig. 6 An example of two-echelon location routing accounting for special urban regulations

dependencies, can be found in Nguyen et al. [40]. The interested reader is directed to these references given that space limitations do not allow including them in this chapter.

Operational Decisions: Routing Problems

According to Taniguchi et al. [51], vehicle routing and scheduling models provide the core techniques for modeling city logistics operations. Once the facilities, or the city logistics centers, have been located, the next step is to decide on the efficient use of the fleet of vehicles that must service the customers or make a number of stops to pick up and/or deliver passengers or products. Vehicle routing problems constitute a whole world given the many operational variants using them. The book of Toth and Vigo [54] provides a comprehensive and exhaustive overview of routing problems. To illustrate their role in city logistics, we have selected the relevant case of pickup and delivery models with time dependencies, which play a key role in courier services, e-commerce, and other related applications.

Pickup and Delivery Vehicle Routing Problem with Time Windows

Pickup and delivery vehicle routing problem with time windows (PDVRPTW) is a suitable approach for modeling routes and service schedules for optimizing the performance of freight companies in the city logistics context (e.g., the couriers). It is also a good example to demonstrate the operational decisions in the routing problems.

In this problem, each individual request includes pickup and a corresponding delivery of specific demand. The relationships between customers are defined by pairing (also known as coupling) and precedence constraints. The first constraint links two particular customers in a pickup-delivery pair, while the second one specifies that each pickup must be performed before the corresponding delivery.

Therefore, the main objective of the PDVRPTW is to determine, for the smallest number of vehicles from a fleet, a set of routes with a corresponding schedule, to serve a collection of customers with determined pickup and delivery requests, in such a way that the total cost of all the trips is minimal and all side constraints are satisfied. In other words, it consists of determining a set of vehicle routes with assigned schedules such that:

- Each route starts and ends at a depot (a vehicle leaves and returns empty to the depot),
- Each customer is visited exactly once by exactly one vehicle,
- The capacity of each vehicle is never exceeded,

- A pair of associated pickup-delivery customers is served by the same vehicle (pairing constraint),
- Cargo sender (pickup) is always visited before its recipient (delivery) (precedence constraint),
- Service takes place within customers' time window intervals (time windows constraint),
- The entire routing cost is minimized.

In order to describe mathematically the demonstrated PDVRPTW, we define for each vehicle k a complete graph $G_k \subseteq G$, where $G_k = (N_k, A_k)$. The set N_k contains the nodes representing the depot and the customers, which will be visited by the vehicle k. The set $A_k = \{(i, j): i, j \in N_k, i \neq j\}$ comprises all the feasible arcs between them. Thus, the problem formulation takes the form

$$\min \sum_{k \in K} \sum_{(i,j) \in A_k} c_{ijk} x_{ijk} \tag{32}$$

subject to

$$\sum_{k \in K} \sum_{j \in N_k \cup \{n+1\}} x_{ijk} = 1 \qquad \forall i \in N^+, \tag{33}$$

$$\sum_{i \in N_k{}^+} \sum_{j \in N_k} x_{ijk} - \sum_{j \in N_k} \sum_{i \in N_k{}^-} x_{jik} = 0 \qquad \forall k \in K, \tag{34}$$

$$\sum_{i} \sum_{j \neq i} x_{jik} \leq \mid S \mid -1 \qquad \forall S \subseteq N : \mid S \mid \geq 2, \forall k \in K \tag{35}$$

$$\sum_{j \in N_k^+ \cup \{n+1\}} x_{0jk} = 1 \qquad \forall k \in K, \tag{36}$$

$$\sum_{i \in N_k \cup \{0\}} x_{ijk} - \sum_{i \in N_k \cup \{n+1\}} x_{jik} = 0 \qquad \forall k \in K, \quad j \in N_k, \tag{37}$$

$$\sum_{i \in N_k^- \cup \{0\}} x_{i,n+1,k} = 1 \qquad \forall k \in K, \tag{38}$$

$$x_{ijk} \left(z_{ik} + s_i + c_{ijk} - z_{jk} \right) \leq 0 \qquad \forall k \in K, \quad (i, j) \in A_k, \tag{39}$$

$$e_i \leq z_{ik} \leq l_i \qquad \forall k \in K, \quad i \in N_k \cup \{0\}, \tag{40}$$

$$z_{i,k} + c_{i,p(i),k} - z_{p(i),k} \leq 0 \qquad \forall k \in K, i \in N_k^+ \tag{41}$$

$$x_{ijk}\left(q_{ik} + d_j - q_{jk}\right) = 0 \qquad \forall k \in K, (i, j) \in A_k, \tag{42}$$

$$d_i \leq q_{i,k} \leq Q \qquad \forall k \in K, i \in N_k^+ \tag{43}$$

$$0 \leq q_{p(i),k} \leq Q - d_i \qquad \forall k \in K, i \in N_k^+ \tag{44}$$

$$q0k = 0 \qquad \forall k \in K, \tag{45}$$

$$x_{ijk} \in \{0, 1\} \qquad \forall k \in K, (i, j) \in A_k, \tag{46}$$

where

N:	set of customers, where $N = N^+ \cup N^-$, $\lvert N^+\rvert = \lvert N^-\rvert$,
N^+:	set of all customers that notify pickup request,
$N-$:	set of all customers that notify delivery request,
c_{ijk}:	nonnegative cost of a direct travel between nodes i and j performed by vehicle k, assuming that $c_{ijk} = c_{jik} \forall i, j \in V$,
i:	customer, where $i \in N_k^+$,
$p(i)$:	pair partner of customer i, where $p(i) \in N_k^-$,
$_{di}$:	customer's demand that will be picked up/delivered at $i/p(i)$, respectively, where $d_i = d_{p(i)}$,
$_{qik}$:	vehicle's k capacity occupancy after visiting customer i,
$_{ai}$:	arrival time at customer i,
$_{wi}$:	waiting time at the customer i, where $w_i = max\{0, e_i - a_i\}$,
$_{zik}$:	start of service at customer i by vehicle k, where $z_i = a_i + w_i$.

The nonlinear formulation of the objective function (32) minimizes the total travel cost of the solution that assures its feasibility with respect to the specified constraints. Equation (33) assigns each customer to exactly one route, while formulation (34) is a pairing constraint, which ensures that the visit of each pickup-delivery pair of customers (i^+, $p(i^+)$) is performed by the same vehicle k. The inequality (35) eliminates the possibility of construction of potential sub-tours. The three following constraints secure the commodity flow. Equality (36) defines the depot as every route's source and states that the first visited customer is the one with a pickup request. Likewise, formulation (38) determines the depot as every route's sink, and the last visited customer is the one that demands a delivery service. The degree constraint (37) specifies that the vehicle may visit each customer only once. The schedule concordance is maintained by Eqs. (39) and (40) according to which, in case that a vehicle arrives to a customer early, it is permitted to wait and start the service within the time window interval only. The precedence constraint (41) assures that for each pair of customers the pickup i is always visited before its delivery partner $p(i)$. The next three restrictions express the dependencies between

the customers' demands and the vehicles' restrained current and total capacities. Equation (42) indicates that after visiting the customer j, the current occupancy of the carriage loading space of the vehicle k is equal to the sum of the load carried after visiting the preceding customer i and the demand collected at customer j. According to inequality (43), the dimension of current occupancy of the total capacity of the vehicle k after visiting a pickup customer i shall be neither smaller than its demand d_i nor bigger than the entire vehicle's capacity. Similarly, following formulation (44), the current capacity of the vehicle k after visiting a delivery customer $p(i)$ shall never be smaller than zero and bigger than the difference between the total vehicle capacity and the size of its delivery request $d_{p(i)}$. The capacity constraint that considers the depot (45) states that the vehicle does not provide it with any service. The last formulation (46) expresses the binary and nonnegative nature of the problem-involved variable.

The exact algorithms are able to solve to optimality only the VRP problems with small number of customers (Cordeau et al. [15]). The heuristics do not guarantee optimality, but since they are capable of providing, for large-sized problems, a feasible solution in a relatively short amount of time, they strongly dominate among all the methods. What is more, the heuristics are proven to be quite flexible in adaptation to different VRP problem variations, which is of special importance when considering the real-world applications.

Tabu Search (TS) has been applied by many researchers to solve VRP. It is known to be a very effective method providing good, near-optimal solutions to the difficult combinatorial problems. The TS term was firstly introduced by Glover [27], and the concepts on which TS is based on had been previously analyzed by the same author Glover [26]. The main intention for TS creation was the necessity to overcome the barriers, stopping the local search heuristics from reaching better solutions than the local optima and explore intelligently a wider space of the possible outcomes. In this context, TS might be seen as extension of the local search methods or as a combination of them and the specific memory structures. The adaptive memory is the main component of this approach. It permits to flexibly and efficiently search the neighborhood of the solution.

The input to TS constitutes an initial solution created beforehand by a different algorithm (Fig. 7). An initial solution construction heuristic determines tours according to certain, previously established rules, but does not have to improve them. Its characteristic feature is that a route is built successively and the parts already constructed remain unchanged throughout the process of the execution of the algorithm.

Sweep algorithm is a good example of a VRP initial solution construction heuristic. It was introduced by Gillett and Miller [25]. However, its beginnings might be noticed in earlier published work of other authors, e.g., Wren [57] and Wren and Holliday [58]. The name of the algorithm describes its basic idea very well. A route is created in the process of gradually adding customers to a route. The selection of customers to add resembles the process of sweeping by a virtual ray that takes its beginning in the depot. When the route length, capacity, or the other previously set constraints are met, the route is closed, and the construction of a new

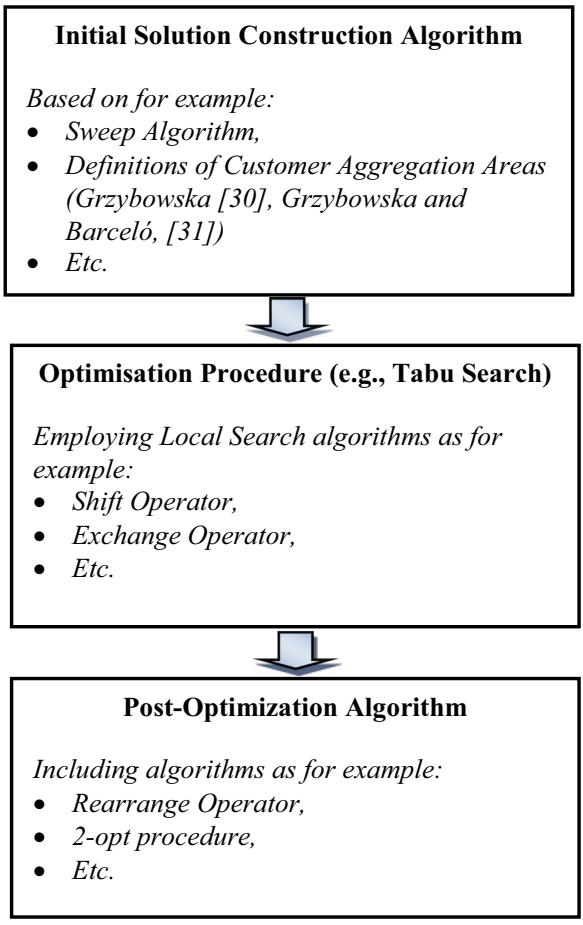


Fig. 7 Composite approach to solve a VRP

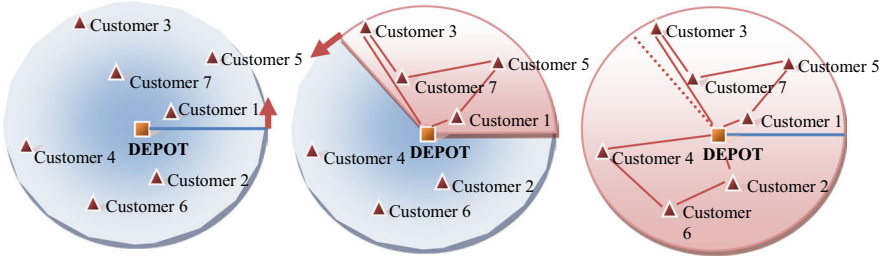


Fig. 8 Sweep algorithm

route is started. The whole procedure repeats until all the customers are "swept" in the routes. A graphic representation of the sweep algorithm is provided in Fig. 8.

The solution provided by the classic sweep algorithm for PDVRPTW most likely will violate precedence and pairing constraints. The initial solution does not have to be feasible since it will be improved later on by TS in the optimization step.

However, a feasible initial solution for PDVRPTW can be provided by a modified sweep algorithm accounting for side constraints. As a result, when a customer is met by the sweeping ray, it is added to the currently built route together with its corresponding partner, respecting the precedence constraint. The details on sweep algorithm adapted to provide initial solution for PDVRPTW are presented in Algorithm 1.

Algorithm 1: Sweep Algorithm for PDVRPTW

1. Let L be the list of all the customers $L = N \backslash \{0\}$
2. Sort all the customers by increasing angle $\angle A0S$, where S is the current customer, 0 is the depot, and A according to the chosen variant is either randomly chosen or fixed reference point.
3. Divide L into k sub-lists such that each sub-list l satisfies:

$$\angle A0S \in \left(\frac{2l-2-k}{k}\pi, \frac{2l-k}{k}\pi \right], \forall l \in K = \{1, \ldots, k\}$$

4. Sort all the customers in each sub-list l in decreasing order according to the travel cost between the depot and the customer.
5. **If** the sub-list l is not empty, **then** select the first customer, search for its partner, **and** insert both customers in a route in the least cost incrementing position. Respect the precedence constraint.
6. Delete the inserted customers from the sub-lists and go to step 5.
7. Repeat steps 5 and 6 for all the sub-lists.

Unified Tabu Search heuristic proposed by Cordeau et al. [14] can be used to optimize the initial solution. Originally, it was designed to solve a VRPTW. However, it can be adjusted to solve PDVRPTW. The main change regards the fact that each modification of a route concerns a pickup-delivery pair of customers instead of an individual customer. This affects the architecture and functioning of TS structures (e.g., adaptive memory). In addition, the local search algorithms used in TS need to consider the constraints of pairing and precedence.

TS starts with establishing the initial solution as best ($s_* = s_{\text{ini}}$). In each iteration, the employed local search algorithm defines a *neighborhood* $N(s_*)$ of the current best solution s_* by performing a collection of a priori designed *moves* modifying the original solution. The new solution s that upgrades the original solution, and characterizes with the best result of the objective function, is set as the best ($s_* = s$). This repetitive routine (i.e., *intensification phase*) lasts until no further improvement can be found, which is interpreted as reaching the optimum. It uses the information stored in the short-term memory (i.e., *"recency" memory*) recording a number of consecutive iterations in which certain components of the best solution have been present. The short-time memory eliminates the possibility of cycling by prohibiting the moves leading back to the already known results, during certain number of iterations. The moves are labeled as *tabu* and placed at the last position in the *tabu list*—a short-term memory structure of length defined by a parameter called *tabu*

tenure. The value of the tabu tenure might be fixed or regularly updated according to the preestablished rules, e.g., recurrently reinitialized at random from the interval limited by specific minimal and maximal value (Taillard [49]). The higher the value of the tabu tenure, the larger the search space to explore is.

The consequent choice of the next best solution is determined by the current neighborhood and defines the general direction of the search. The lack of broader perspective in most of the cases leads to finding a solution, which represents a local optimum instead of a global. It is also due to the decision on when to finish the whole procedure improvement (i.e., *stopping criterion*), which might be determined by a designated time limit for the complete performance or by a previously established number of repetitions, which do not bring any further improvement. In order to overcome this handicap, TS stops the local search algorithm and redirects the search in order to explore intelligently a wider space of the possible solutions (i.e., *diversification phase*). This includes permitting operations, which result in deterioration of the currently best solution. This phase requires access to the information accumulated during the whole search process and stored in the long-term memory structure (i.e., *frequency memory*). The success of the complete method depends on the balance established between these two phases, which complement each other instead of competing.

The functioning of the algorithm based on tabu bans is very efficient. However, oftentimes it may result in losing improvement opportunities by not accepting highly attractive moves, if they are prohibited. In such cases, the *aspiration criteria* should be activated. It is an algorithmic mechanism, which consists in canceling the tabu restrictions and permitting the move, if it results in construction of the new solution with the best yet value of the objective function. The implementation of TS is presented by Algorithm 2.

Algorithm 2: Tabu Search

1. Let η be the maximal number of permitted TS iterations.
2. Let α and β be the parameters of preestablished value equal to one.
3. Let Δ be the value of the objective function.
4. Let s be the initial solution.
5. Let s^* be the best solution and $s* = s$
6. Let $c(s^*)$ be the cost of the best solution.
7. **If** the solution s is feasible, **then** set $c(s^*) = c(s)$; **else** set $c(s^*) = \infty$
8. Let $B(s)$ be adaptive memory structure, whose arguments are initially set equal to zero.
9. **For** all the iterations $\kappa < \eta$, execute local search operator and create neighborhood $N(s)$
10. Select a solution from the neighborhood that minimizes the Δ function.
11. **If** solution s is feasible **and** $c(s) < c(s*)$, **then** set $s^* = s$ **and** $c(s^*) = c(s)$
12. Update $B(s)$ according to the performed move.
13. Update α and β

To defineSearch the neighborhood of a current solution, TS uses a local search algorithm. Local search algorithms are commonly used as intermediate routines performed during the main search process of more complex heuristics. However, they constitute individual algorithms. Some of them are referred to as *operators* with specified features (e.g., *shift operator*, *exchange operator*, *rearrange operator*, etc.), while the others possess their own denomination (e.g., the local optimization methods involving neighborhoods, which apply to the original solution a number of modifications equal to k, are known as the *k-opt* heuristics).

Shift operator is aPickup-Delivery Customer Pair Shift Operator good example of a local search algorithm in TS. Its objective is to remove a customer from its original route and feasibly insert it in another route of the current solution, in such a way that its total cost is minimized. When solving PDVRPTW the shift move includes a pickup-delivery customer pair instead of an individual customer. During the entire search process, all the pickup-delivery pairs are successively moved, and all the possible reinsertion locations in the existing routes are checked. Only the moves which are in accordance with all the side constraints shall be accepted. Algorithm 3 presents the functioning of shift operator in TS for solving PDVRPTW.

Algorithm 3: Pickup-Delivery Customer Pair Shift Operator

Let s be the current solution containing a set of routes R
Calculate $c(s)$, $q(s)$, and $w(s)$
Let $s^* = s$ be the best solution with cost $c(s^*) = c(s)$
Let $f(s^*) = \infty$ be the value of the objective function of the best solution s^*
Let $\Delta^* = \infty$ be the best value of the objective function
Let $B(s)$ be the empty adaptive memory matrix
For each route $r_1 \in R$ **and for** each route $r_2 \in R$, **such that** $r_1 \neq r_2$
For each pickup-delivery customer pair $m \in r_1$
Remove pair m from route r_1
Set bool TABU = FALSE
If the value of the tabu status in $B(s)$ for r_2 **and** pair m is $\neq 0$, **then** TABU = TRUE
Find the best insertion of pair m in r_2 and obtain new solution s'
Calculate $c(s')$, $q(s')$, and $w(s')$
Let $f(s')$ be the value of the objective function of the new solution s'
Let $p(s')$ be the value of the penalty function of the new solution s'
Let $\Delta = 0$ be the value of the objective function
If $f(s') < f(s^*)$, **then** $\Delta = f(s')$; **else** $\Delta = f(s') + p(s')$
Check the feasibility of solution s'
Set bool AspirationCriteria = FALSE
If $c(s') < c(s^*)$ **and** solution s' is feasible, **then** AspirationCriteria = TRUE
If $\Delta < \Delta^*$ **and** (Tabu = TRUE **or** AspirationCriteria = TRUE), **then** $\Delta^* = \Delta$ **and** the best move was found

The algorithm uses the values of violations of the constraints for the calculation of the objective function, as well as for determining the rate of the penalties, which

need to be imposed for not complying with initial restrictions. These values are calculated according to the following formulas:

$$\Delta = f(s) + p(s) \tag{47}$$

$$f(s) = c(s) + \alpha \cdot q(s) + \beta \cdot w(s) \tag{48}$$

$$p(s) = \lambda \cdot c(s) \cdot \sqrt{n \cdot k} \cdot \varphi(s) \tag{49}$$

$$\varphi(s) = \sum_{(m,r) \in B(s)} \rho_{m,r} \tag{50}$$

where

Δ: objective function,
$f(s)$: cost evaluating function for the solution s,
$p(s)$: penalties evaluating function for the solution s,
$c(s)$: cost of solution s,
$q(s)$: vehicle capacity constraint violation function for solution s,
w(s): time windows constraint violation function for solution s,
α: parameter related to the violation of the vehicle capacity constraint,
β: parameter related to the violation of the time windows constraint,
λ: scaling factor,
n: total number of customers,
k: total number of vehicles
φ(s): parameter controlling the addition frequency,
$B(s)$: adaptive memory matrix,
$\rho_{m,r}$: number of times the pickup-delivery customer pair m has been introduced into route r.

It is a usual practice to complement the TS heuristic with a *post-optimization* step. Its objective is to further improve the solution provided by TS. It is important to note that the final result has to be obligatorily feasible. Local search heuristics are often used for post-optimization purposes and *2-opt heuristic* is a good example. The 2-opt procedure is the most common representative of the family of *k-opt heuristics*. It has been introduced by Croes [18] for solving a Travelling Salesman Problem. The main idea of the 2-opt method is valid for the other k-opt heuristics. In the main, it consists of removing two nonconsecutive arcs connecting the route in a whole and substituting them by another two arcs reconnecting the circuit in such a way that a new solution which fulfills the predefined objectives is obtained. This move is commonly called a *swap* since it consists of swapping two customers in the original sequence. The swap can be performed in accordance with one of the following strategies: (i) search until the first possible improvement is found, and perform the swap, and (ii) search through the entire tour and all possible improvements, and perform only the swap resulting in the best improvement. A graphic representation of 2-opt procedure is provided in Fig. 9.

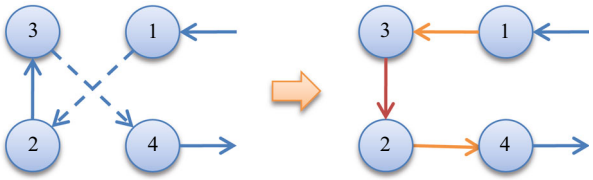


Fig. 9 2-opt algorithm

As in the case of the previously presented algorithms, the classic formulation of the 2-opt algorithm needs to be adapted to solve the PDVRPTW. The final solution needs to be strictly feasible; thus all the side constraints, including pairing and precedence, need to be respected. The PDVRPTW adapted 2-opt algorithm is presented2-opt Algorithm by Algorithm 4.

Algorithm 4: 2-opt Algorithm Adapted to PDVRPTW

1. Let R be the set of routes defining the current solution s
2. **For** each route $r \in R$
3. Let $r*$ be the best route and $r^* = r$
4. Calculate $c(s^*)$, $q(s*)$, and $w(s*)$
5. **While** best move is not found
6. Let $pos(i)$ be the position of customer i in the sequence of customers of route r
7. Let $pos(j)$ be the position of customer j in the sequence of customers of route r
8. **For** each customer i and j in route r **such that** $pos(j) = pos(i) + 2$
9. Create new empty route r'
10. **For** each customer h in route r
11. **If** $pos\ (h) \leq pos$(i) **or if** $pos(h) \geq pos(j + 1)$, **then** append customer h to r'; **else** append customer h at position $pos(i) + pos(i) - pos(h) + 1$ to r'
12. Calculate $c(s')$, $q(s')$, and $w(s')$ and check the feasibility of route r'
13. **If** $c(r') < c(r*)$ **and** route r' is feasible, **then** $c(s^*) = c(s')$ **and** the best move was found

Real-Time Management

Under this category we cluster what are considered the most relevant city logistics services made available by the pervasive penetration of the ICT technologies. These technologies made it possible for the demand to occur anywhere and at any time. This entails a request for the system to provide the capability of suitably responding to the demand in real time, and in a way that at the time satisfies the quality requirements of the customers, and also provides the most suitable benefit to the company. This section describes the main characteristics

highlighting the dynamic aspects of the problem, which must be addressed by efficient computational time-dependent versions of the ad hoc algorithms. This is an area of applications characterized by the synergies between technologies, decision-making, and sophisticated routing algorithms. The domain of application, however, has recently been expanded to account also for emerging versions of public transport services, special cases of demand-responsive transport services, which can be formally formulated in similar terms (just replacing freight or parcels by persons). This application will be considered in section "Extensions".

Let us suppose that an initial operational plan has been defined with k routes and that there is an *advanced traffic information system* providing real-time information that allows us to calculate current travel times for every pair of clients. Since most of real-world fleets are usually equipped with *automatic vehicle location* (AVL) technologies, it also assumed that the fleet manager is capable of knowing the exact location of the vehicles at every time and has a two-way communication with the fleet.

In our proposed *rerouting algorithm* (RRA), we compute the feasibility of the remaining services while we keep track of the current state and location of the vehicles. In order to simplify the tracking process, we consider two approaches: either to compute feasibility once a vehicle finishes a service and informs about its current situation to the fleet manager or in a periodical manner after updating the fleet's situation. In the first approach, the trigger for the computations is only the departure of the vehicle, but there might be risks when vehicles take longer times to complete the service. The second approach is computationally more intensive, but it may prevent future failures in the service when unexpected service times are present.

In the case of the vehicle routing with time windows, there are two critical factors that define the feasibility of a route: capacity and time windows. Assuming that no variations in demand are expected (e.g., more units to pick up), capacity can be neglected as the corresponding route (static or dynamic) is built taking into account this constraint. In such case, time windows constraints become more critical as there are many uncontrollable parameters (e.g., unexpected traffic conditions, delayed service, etc.) that may affect the feasibility of a service and, therefore, the performance of the route.

Figure 10 describes the full dynamic monitoring process. On one hand, we have the advanced traffic information system which provides current and forecasted travel times of the road network to the fleet manager. On the other hand, we have the fleet management center that is able to communicate with the vehicles and receive their position along with other data such as current available capacity and status.

These two sources are inputs to a dynamic tracking system which through a desired strategy (periodic, after service or both) computes the current feasibility of the routing plan. If an unfeasible service is detected, the RRA is triggered to find a feasible current plan. If a customer cannot be allocated to a vehicle that is on the road, the RRA tries to assign an idle vehicle housed at the depot. If no vehicles are available, a penalty is applied.

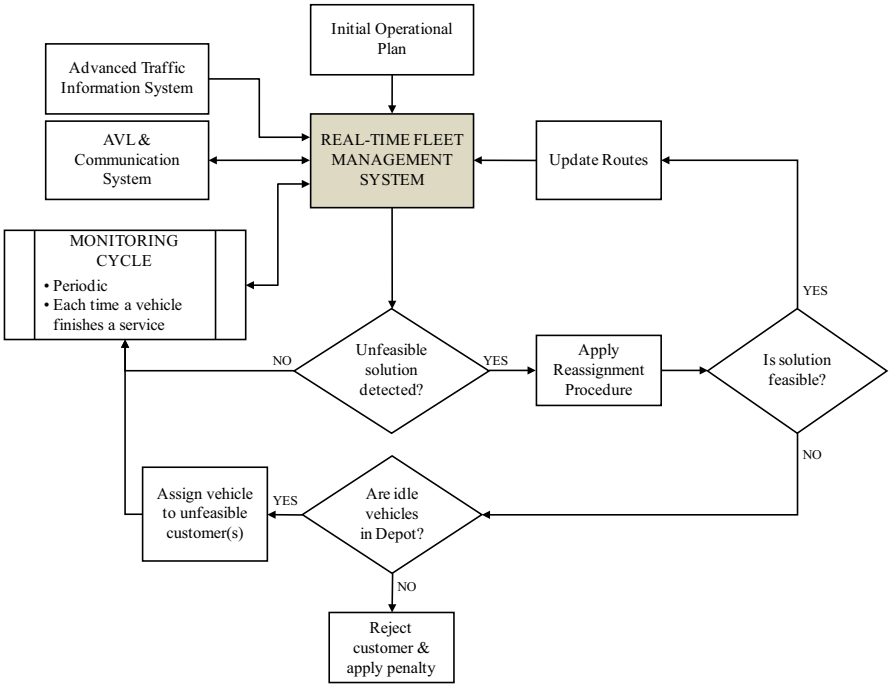


Fig. 10 Rerouting algorithm through dynamic tracking of the fleet

Computation of Feasibility

Computing the current feasibility of a route depends on the state of the vehicle at the time of evaluation. If a vehicle is traveling to the next scheduled customer or departing from a customer's location after service, the feasibility of the route is obtained by computing the expected arrival times to the next customers on the route. If at the time of evaluation, a vehicle has already started a service or waiting to start it, we must first estimate the departure time from the current location and then the arrival times in the rest of the route.

Under our approach, we assume that drivers send messages to the fleet management center when they arrive to and start or end a service with a customer. Therefore, at the time of evaluation, if a vehicle providing service has not finished under the estimated time lapse, the algorithm assumes that the remaining service time is the expected time used in the computation of the route. That is, if a service is assumed to take 10 min and, at the time of evaluation, the vehicle has 14 min without sending the end-of-service signal, the algorithm assumes, in a preventive way, that the vehicle needs another 10 min to complete the service.

Formally, the feasibility conditions can be computed as follows. Let $\hat{a}_i$ be the estimated arrival time of a vehicle at customer i, E_i the lower bound of the time

window of customer i, e_i the estimated service start time at customer i, S_i the estimated service time at customer i, and $T_{ij}(t)$ the travel time between customer i and j when vehicle departs at time t. Let V_k be a dummy node in the network representing vehicle k which route is given by $\{1, 2, \ldots, i, i+1, \ldots, n-1, n\}$.

If at time t_e, the vehicle has just finished a service at customer i or it is traveling to customer $i+1$, the estimated arrival and service start times at the next scheduled customers can be computed as follows:

$$\hat{a}_i = \begin{cases} t_e + T_{V_k,h}(t_e) & \text{if } h = i+1 \\ e_{h-1} + S_{h-1} + T_{h-1,h}(e_{h-1} + S_{h-1}) & \text{if } h = i+2, i+3, \ldots, n \end{cases} \tag{51}$$

$$e_h = \max\left\{E_h, \hat{a}_h\right\}, \quad \text{for } h = i+1, i+2, \ldots, n \tag{52}$$

On the other hand, if at time t_e, the vehicle is waiting to start service or providing service to customer i, we first compute d_i, the expected departure time from customer i. Let a_i be the real arrival time of a vehicle at customer i's location. Therefore, d_i is computed as follows:

$$d_i = \begin{cases} t_e + S_i & \text{if } \max\{\mathrm{E_i}, a_i\} + S_i < t_e \\ \max\{E_i, a_i\} + S_i & \text{otherwise} \end{cases} \tag{53}$$

In this case, the expected arrival times in subsequent customers $h = i+1, i+2, \ldots, n$ are given by:

$$\hat{a}_h = \begin{cases} d_i + T_{i,i+1}(d_i) & \text{if } h = i+1 \\ e_{h-1} + S_{h-1} + T_{h-1,h}(e_{h-1} + S_{h-1}) & \text{if } h = i+2, i+3, \ldots, n \end{cases} \tag{54}$$

where e_h is computed as in (2). The service at customer h becomes then *unfeasible* if $\hat{a}_h > L_h$, where L_h is the upper bound of the required time window in customer h. Figure 11 depicts a situation when one of the services becomes unfeasible.

The proposed RRA consists of computing feasibility conditions every time a vehicle has finished a service and is ready to depart from its current location. If one or more customers are detected to be in an unfeasible sequence, they are withdrawn and reinserted in the route using a greedy *dynamic insertion* heuristic (DINS). A re-optimization procedure is then applied using a *Tabu Search*-based metaheuristic (DTS). The greedy insertion heuristic is described in the following subsection.

If DINS algorithm does not find a feasible assignment to the vehicles on route, it allocates the withdrawn order to an idle vehicle in the depot. If no vehicles are available, the algorithm rejects the customer. The pseudo-code of the RRA is the following:

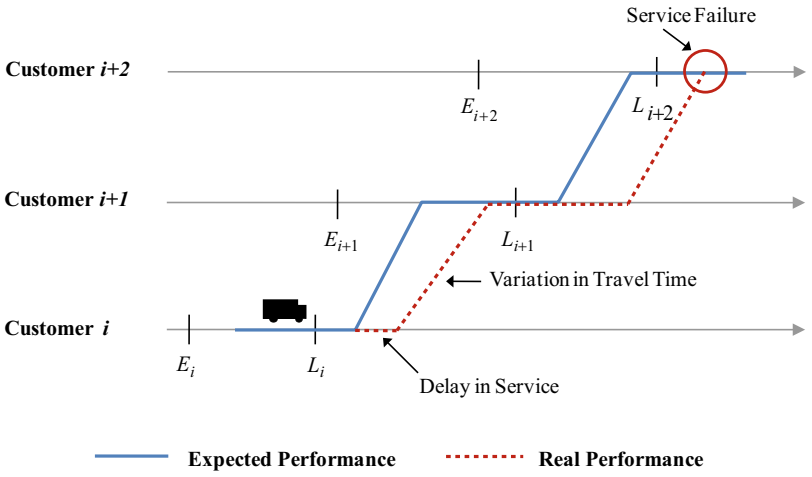


Fig. 11 Example of service failure due to delays [41]

Algorithm 5: Rerouting Algorithm

After a vehicle k finishes a service:

1. Compute feasibility conditions for all unvisited customers in the route of vehicle k
2. Let U_k be the set of unvisited customers which are now unfeasible in current scheduling
3. **If** $U_k = \varnothing$, **then** STOP; **else** continue to step 4
4. **For** each customer $u \in U_k$, withdraw u from route applying operator DELETE(u)
5. **For** each customer $u \in U_k$, reinsert u in current routes applying DINS algorithm
6. **If** reinsertion is not possible for a customer, **then** create new route with available vehicle (if any)
7. Apply DTS algorithm to re-optimize routes.

The RRA assumes that all customers that are found to be unfeasible have the same importance, ranking, or priority. Therefore, if there is more than one unfeasible customer, step 5 selects randomly, with equal probability, the next customer to be inserted in the routes.

Heuristic Approach

The RRA is a two-phase methodology that first identifies customers that are likely to be not serviced on time and then reassigns them in order to optimize the service level of the fleet. After the identification of customers, the algorithm withdraws those customers from the routes and uses them to build a set of customers U_k to be reassigned. The reassignment algorithm works by first inserting customers in U_k with a greedy insertion heuristic, and then, routes are re-optimized using a Tabu Search metaheuristic.

Dynamic Insertion Heuristic (DINS)

The heuristic used in the DRS is derived from the basic insertion heuristic proposed in Campbell and Savelsbergh [11], where every unrouted customer is evaluated at every insertion point. The evaluation of this movement consists in checking the feasibility and profitability of every insertion, which is the negative of the amount of additional travel time added to the route. The customer with a feasible insertion having the maximum profitability is then selected and inserted in the route. We have adapted this heuristic by introducing some new elements to reflect the dynamics of the operations of a fleet in an urban environment.

The objective of DINS heuristic is to insert a new client into the current routing plan once the vehicles have started the services. The general idea of the algorithm is simple. When a new client arrives to the system, the algorithm checks the current state of the vehicles. Then, routes with insufficient time to visit the new client within their schedule are rejected. Finally, the algorithm searches for the least cost feasible insertion in the candidate routes. If no feasible insertion is possible and there are idle vehicles at the depot, a new route is created including the new customer.

We assume that there are n routed customers in R different routes at the beginning of the time planning horizon. We also suppose that there is a single depot with a homogeneous fleet of vehicles with capacity Q. We define the following notation:

$T_{i,j}(t)$: travel time between customers i and j when vehicle departs at time t,
V_r: vehicle assigned to route r, $r = 1, 2, .., R$,
q_r: total demand assigned to route r,
d_i: demand of customer i,
E_i: time window lower bound of customer i,
L_i: time window upper bound of customer i,
e_i: earliest service start time at customer i,
l_i: latest service start time at customer i,
S_i: service time at customer i.

The time window of a customer i is denoted by (E_i, L_i). The values e_i and l_i refer to the earliest and latest time a service can take place at customer i, and they must

satisfy the following condition: $E_i \leq e_i \leq l_i \leq L_i$. If we wish to insert customer w at time instant $t > 0$, the vehicles of the fleet may have one of the following three states:

1. The vehicle is in service at some customer i (SER).
2. The vehicle is moving to the next planned customer on the route or waiting at the customer location to start service within the time window (MOV).
3. The vehicle is idle at the depot, without a previously assigned route (IDL).

The state of a vehicle will let us know when a vehicle should be diverted from its current route, be assigned to a new one if it is idle, or keep the planned trip. Whenever a new customer arrives, the status of each vehicle must be known to compute travel times from their current positions to the new customer.

The first iteration of the algorithm consists of rejecting those routes that are definitely infeasible. To do this, it is necessary to compute the total available time of the routes, i.e., the sum of the available time (if any) of a vehicle on the assigned route. We define the available time of a vehicle as those periods of time where (i) a vehicle is waiting to provide service to a customer or (ii) a vehicle has finished the scheduled route and is returning to the depot. This is, in fact, the maximum slack time a vehicle has available to travel and service customer w.

The calculation of the available route time can be done by estimating the arrival times at each of the scheduled customer locations that has not been served at the time of insertion of the customer w. The arrival time a_i at customer i is computed from the current position of the vehicle, and we assume that the vehicle will start service as soon as it arrives, if the customer's time window is already open; otherwise, the vehicle waits. The calculation of arrival times can be made as follows.

If, at instant t, the vehicle is moving toward the next customer i, then the estimated arrival time is given by $a_i = t + T_{Vr,i}(t)$. The arrival times at subsequent customers on the route are computed as follows:

$$a_i = \max\{E_{i-1}, a_{i-1}\} + S_{i-1} + T_{i-1,i}\,(\max\{E_{i-1}, a_{i-1}\} + S_{i-1}) \tag{55}$$

Hence, the available time of a vehicle in route r at some node i (including the depot) is the difference between the associated lower limit of the time window and the arrival time of the vehicle to that location. That is, $W_i^r = \max\{E_i - a_i, 0\}$. Therefore, the total available time of a route r is given by $AT_{v_r} = \sum_i W_i^r$.

From each route, we choose the unvisited node that is the closest to the new client. If the travel time from that node to the new customer is greater than the total available time AT_{Vr}, then we reject the route. If every route is rejected, a new route must be created for this new customer. The routes obtained, as a result of this previous iteration, are *possibly feasible* in the sense that vehicles serving those routes have enough time to travel to the new customer. This is a necessary

condition, but not sufficient, because we have to check for time windows feasibility. The second major iteration of the algorithm consists, therefore, in checking the feasibility and profitability of an insertion in every arc of the possible feasible routes. The feasible insertion with the highest profitability is then selected, and the corresponding route must be updated.

Feasibility In order to evaluate the feasibility of the insertion of customer w between nodes i and $i+1$, we must compute e_w and l_w, the expected earliest and latest service start times, respectively. The earliest service start time at the inserted customer w is given by:

$$e_w = \begin{cases} \max\left\{E_w, t + T_{V_r,w}(t)\right\} & \text{if } V_r = i \\ \max\left\{E_w, e_i + S_i + T_{i,w}(e_i + S_i)\right\} & \text{otherwise} \end{cases} \tag{56}$$

The latest service start time is calculated through a backward procedure starting with the depot (the last node to be visited by the vehicle) as follows:

$$l_w = \min\left\{L_w, \tilde{t}_w - S_w\right\} \tag{57}$$

where $\tilde{t}_i = \arg\max_t \left\{t + T_{i,i+1}(t) - l_{i+1}\right\}, \ \forall t \leq l_{i+1} - T_{i,i+1}(t)$ is the latest possible departure time that allows the vehicle to arrive to the next scheduled customer $i+1$ at time l_{i+1}. If $e_w \leq l_w$ and $D_w < Q - q_r$, the insertion is feasible.

Profitability The profit of an insertion is defined as the negative of the additional travel time incurred from inserting a customer in a route. If customer w is to be inserted between nodes i and $i+1$, the profitability of this insertion is given by:

$$\begin{aligned} \text{Profit} = \big\lfloor T_{i,w}(\max\{E_i, a_i\} + S_i) + T_{w,i+1}(\max\{E_w, a_w\} + S_w) \\ - T_{i,i+1}(\max\{E_i, a_i\} + S_i) \big\rfloor \end{aligned} \tag{58}$$

If no insertion is possible and there are available vehicles at the depot, then a new route that includes customer w is created. If the whole fleet of vehicles is already occupied, then the call is rejected, and a penalty may be applied. Given a customer w to be inserted at time t into a set R of routes, the pseudo-code of the DINS heuristic is as follows:

The solution obtained by DINS heuristic can be further optimized by applying DTS.

Algorithm 6: DINS Heuristic

1. **For** each route r in R **do**:
Compute available time AT_{Vr}
Find travel time from w to closest neighbor
If available time is not enough, **then** reject route r
Else, add route r to R', the set of possible feasible routes
2. **For** each route r in R' **do**:
Check the status of the vehicle and set its position as the starting node of the route.
For each arc $(i, i+1)$ of the remaining route sequence **do**:
If insertion of w is feasible and profit is improved **then**:
Store current profit and insertion places
3. **If** insertion is possible **then:**
Insert w in least cost arc and update selected route.
4. **Else,** create a new route with an available idle vehicle (if any).

Dynamic Tabu Search (DTS)

DTS constitutes an adaption of the *Unified Tabu Search* (UTS) heuristic proposed by Cordeau et al. [14]. The modifications of the original method were made in order to include the dynamic aspects of the addressed problem. One of the most important modifications regards the engagement of the operators, which perform the local search only on the unvisited customers of the scheduled routes.

Similarly as UTS, DTS is an iterative process searching for the best solution s^*. In each iteration, the algorithm searches for the best non-tabu solution s' in the neighborhood space $N(s)$ of current solution s, which minimizes the value of the cost function $f_D(s)$ or satisfies the aspiration criteria. The solutions found in the search process are evaluated by the cost function:

$$\mathrm{f_D}(s) = c_D(s) + \alpha q(s) + \beta d_D(s) + \gamma w_D(s) \tag{59}$$

where

$c_D(s)$: estimated total travel time of the routes for the pending scheduled customers,
$q(s)$: total violation of the vehicles' capacity,
$d_D(s)$: total violation of the routes duration,
w_D(s): total violation of the time windows constraint of the customers to be visited.

The adaptive memory matrix $B(s)$ denotes the attribute set of a solution s and is defined as follows: $B(s) = \{(i, k)$, where customer i (or a pair of customers in the case of PDVRPTW) is visited by vehicle $k\}$.

The penalty function $p(s)$ was added to the objective function in order to diversify the search of the solutions toward new or not thoroughly explored areas. For each solution s' such that $f_D(s') \geq f_D(s)$, the penalty is calculated as follows:

$$\mathrm{p}\left(s'\right) = \lambda c_D\left(s'\right)\sqrt{n_u m} \sum_{(i,k)\in B(s')} \rho \tag{60}$$

where

n_u: number of customers that have not been visited (or pairs of customers in the case of PDVRPTW),
m: current number of routes,
λ: positive value parameter to control the intensity of diversification,
ρ_{ik}: addition frequency parameter equal to the number of times the attribute (i, k) has been added to a solution.

The value of the addition frequency parameter, which controls the intensity of diversification, is updated regularly and at the end of each iteration of DTS.

The aspiration criteria defined for the DTS accepts a solution if its cost improves the best solution found so far. Also, in order to achieve quick online solutions, we added a stopping criterion which takes into account the rate of improvement in the search process. When the κ number of iterations has been reached without observing an improvement larger than ε, then the search process is stopped.

Similarly as in UTS (Cordeau et al. [14]), in DTS we employ a relaxation mechanism facilitating the exploration of the solution space. The mechanism is particularly useful in the cases when tight constraints are defined (e.g., hard time windows, pairing, and precedence constraints). The relaxation is achieved by dynamically adjusting the values of the parameters: α, β, and γ. When the solution s' is feasible, the value of the best feasible solution is updated, and the current values of the three parameters are reduced by dividing them by the factor $(1 + \delta)$. In the contrary case, the current values of the parameters are increased by multiplying them by the same factor. δ is a parameter of fixed value.

DTS was designed in a way, so that it becomes a flexible framework in which various local search operators can be embedded. As a result, depending on the addressed problem, a different operator can be enabled. Algorithm 7 provides a general description of the DTS. The reader interested in details is directed to Barceló et al. [4, 5].

Algorithm 7: Dynamic Tabu Search

1. **If** solution s is feasible **then**:
2. Set $s^* = s, \alpha = 1, \beta = 1, \gamma = 1$
3. Set $c(s^*) = c(s)$ **else** set $c(s^*) = \infty$
4. **For** $i = 1, \ldots, \eta$ **do**:
5. Select solution $s' \in N(s)$ that minimizes the objective function $\Delta = f(s') + p(s')$ such that s' is not tabu **or** satisfies aspiration criteria
6. **If** s' is feasible **and** $c(s') < c(s^*)$, **then** set $s^* = s$' **and** $c(s^*) = c(s')$
7. **If** $q(s') = 0$, **then** $\alpha = \alpha/(1+\delta)$, **else** $\alpha = \alpha(1+\delta)$
8. **If** $\mathrm{d}(s') = 0$, **then** $\beta = \beta/(1+\delta)$, **else** $\beta = \beta(1+\delta)$
9. **If** $\mathrm{w}(s') = 0$, **then** $\gamma = \gamma/(1+\delta)$, **else** $\gamma = \gamma(1+\delta)$
10. Set $s = s$'
11. Update α, β and γ
12. **If** number of iterations without improvement $= \kappa$, **and** last improvement $< \varepsilon$, **then** STOP

Extensions

The variants of pickup and delivery problems with time windows and time-dependent link travel times discussed in section "Operational Decisions: Routing Problems", as core components of the real-time fleet management applications analyzed in section "Real-Time Management" (as we have already mentioned), provide another area of application in urban scenarios which can be considered advanced implementations of demand-responsive transport systems. Examples of such could be the special shared taxi services provided by Uber [56] or Kutsuplus, uber-like minibus demand-responsive transit system experimented in Helsinki KUTSUPLUS [33].

A summary description of the Real-Time Multiple Passenger Ridesharing system and how it works could be the following. A customer asks for a service at a given time; the service could consist of either picking up a passenger (or various passengers) or a parcel, at a given location pickup location, at a given time (or within a time window) and delivering the passenger(s) or the parcel, at a delivery location at a given time (or within a time window). The service vehicles already in operation follow routes individually calculated depending on the customers' initial and final positions, the current traffic conditions, and the defined passengers' time windows.

Figure 12 illustrates conceptually how the demand-responsive mobility services work. Let's assume a fleet consisting of two vehicles V1 and V2, both with initially

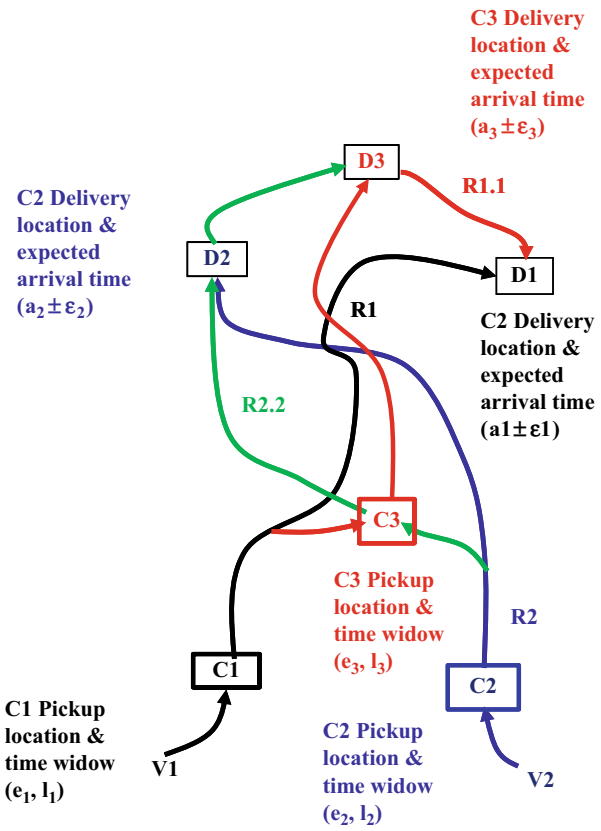


Fig. 12 Example of demand-responsive mobility services

assigned plans which have been previously optimized. At a given time a customer C1 calls the system for a service, the customer's location is automatically recorded by the system. The customer tells the system which is his/her time expectancy of pickup, in other words the time window (e_1, l_1) of his/her waiting time expectancy (e.g., e_1 could be the time at which the call is made, and l_1 the latest time he/she expects to be picked up). The customer also informs the system of his/her destination that is delivery point, D1, and likely the time at which he/she would approximately like to arrive at the destination, let's say $(a_1 \pm \varepsilon_1)$, where ε_1 represents an acceptable slack time.

The system, which is aware of locations and status of each vehicle in the fleet (e.g., fleet vehicles are GPS tracked, and the ICT functions inform the system on the current level of occupancy of the vehicle, i.e., the number of passengers, their destinations, and time constraints), as well as the current network conditions (e.g., a real-time traffic information system keeps the decision support system updated about travel times, congestions, incidents, and so on), determines which of the vehicles is the most appropriate to provide the service to customer C1 both in

terms of quality of the service and profitability. Let's assume that on the basis of the available information, the dispatching system assigns the service to vehicle V1 and the route R1 to pick up the customer and take him or her to his/her destination.

In a similar way, let's assume that customer C2, who is located at C2, has to be picked up within the time window (e_2, l_2) and wants to travel to destination D2. He or she expects to arrive around $(a_2 \pm \varepsilon_2)$ and is assigned to vehicle V2 that will follow route R2 to pick him/her up and travel to the destination D2.

The critical situation showing how the performance of the entire system depends on the quality of the decision support system that manages the fleet, and dispatches the services, arises when some time later, let's say at time t, a new customer C3, at location C3, asks for a pickup service within the time window (e_3, l_3) to travel to destination D3 where he/she expects to arrive around $(a_3 \pm \varepsilon_3)$.

In order to provide the new customer the requested service, the system faces many alternatives. First, it can open a new route assigning one of the empty vehicles of the fleet to the new client, but this action may imply a high cost. A better alternative would be to assign one of the en route vehicles that are closer to the client, if the time constraints of the customers that are already being served allow this. In this example, both proposed solutions accept a diversion policy, so, either of the vehicles could divert from their original routes to serve the new client and return to their originally assigned schedules.

One of the alternatives is to assign the new customer service to the vehicle V1, which exchanges its route R1 for the new route R1.1 (described in the figure with red arrows). This vehicle picks up customer C3 within the accepted time window, and then it drives to the D3 destination to deliver the customer. After this, vehicle V1 follows its new route and delivers customer C1 at the corresponding destination D1 taking into account the time constraints of all the served customers. The second possibility is that the new service of customer C3 is assigned to vehicle V2, which diverts its route to pick up the customer by following route R2.2 (described in the figure with green arrows). In this case, the vehicle delivers customer C2 at its destination D2 first, and then, it goes to the destination D3 of customer C3. Also in this case, we need to take into account the time constraints with respect to all involved customers. The decision will be made accounting for criteria that, while ensuring the quality of the service provided to the customers, achieve the maximum system profitability.

The management of the system is based on a decision support system similar to the one described in section "Real-Time Management". It allows making decisions on which vehicle to assign to each new coming customer. The dynamic rerouting is a particular case of the pickup and delivery problem with time windows, with time-dependent travel times, similar to the one described in section "Operational Decisions: Routing Problems".

Linares et al. [34, 35] provide details on a simulation study on the performance of these types of systems as a function of the expected demand for service relative to the total demand of conventional car trips in the selected urban scenario, the size of fleet providing the service (i.e., 500, 750, 1000 . . . vehicles), and the capacity of the fleet vehicles, six or eight passengers.

Concluding Remarks

The main objective of this chapter is to introduce the main characteristics of city logistics problems that make them a special category—clearly differentiated from the general logistics problems. The analysis of these characteristics and their peculiarities regarding the type of decisions to make (i.e., strategic, tactic, and operational) allows to identify various classes of problems (i.e., location, location routing, routing with time windows, etc.) each one becoming a fertile domain for heuristics, given the complexity and size of the problem to solve in real life. A selected set of examples of such heuristics has been described in this chapter, with special attention paid to those particularly relevant to urban scenarios due to the imposed constraints such as the regulatory policies imposed by the authorities, time constraints imposed by the conditions of servicing customer in these scenarios, and the time dependencies implied by the variability of travel times in urban congestion. For the interested reader the chapter is complemented with a wide set of references.

Cross-References

▸ GRASP
▸ Supply Chain Management
▸ Tabu Search

Competing Interest Declaration The author(s) has no competing interests to declare that are relevant to the content of this manuscript.

References

1. Barceló J, Orozco A (2010) A decision support system to assist the design and evaluation of city logistics applications with real-time information. In: TRB10-3526, presented at the 89th TRB annual meeting, Washington, DC
2. Barceló J, Grzybowska H, Pardo S (2005) Combining vehicle routing models and microscopic traffic simulation to model and evaluating city logistics applications. In: The proceedings of the 16th mini-EURO conference and 10th meeting of EWGT, Italy
3. Barceló J, Grzybowska H, Pardo S (2007) Vehicle routing and scheduling models, simulation and city logistics. In: Dynamic fleet management. Springer, New York, pp 163–195
4. Barcelo J, Grzybowska H, Orozco JA (2008) A simulation based decision support system for city logistics applications. In: 15th world congress on intelligent transport systems and ITS America's 2008 annual meeting, New York
5. Barceló J, Orozco JA, Grzybowska H (2013) Making real-time fleet management decisions under time dependent conditions in urban freight distribution. In: Ben-Akiva M, Meersman H, van de Voorde E (eds) Freight transport modelling. Emerald Group Publishing Limited, Bingley
6. BESTUFS (2006). http://www.bestufs.net/
7. BESTUFS II, D5.2 quantifications of urban freight transport effects II. http://www.bestufs.net
8. Boccia M, Crainic TG, Sforza A, Sterle C (2011) Location-routing models for designing a two-echelon freight distribution system, CIRRELT-2011-06, Montréal

9. Bodin L, Maniezzo V, Mingozzi A (1999) Street routing and scheduling problems, Chap. 12. In: Hall RW (ed) Handbook of transportation science. Kluwer, Boston
10. Boudoin D, Morel C, Gardat M (2014) Supply chains and urban logistics platforms. In: González-Feliu J, Semet F, Routhier J-L (eds) Sustainable urban logistics: concepts, methods and information systems. Springer, Berlin/New York
11. Campbell AM, Savelsbergh M (2004) Efficient insertion heuristics for vehicle routing and scheduling problems. Transp Sci 38(3):369–378
12. Chen HK, Hsueh CF, Chang MS (2006) The real-time time-dependent vehicle routing problem. Transp Res E 42:383–408
13. CLM (2009) Definition for logistic, council of logistics management. http://cscmp.org/aboutcscmp/definitions.asp
14. Cordeau JF, Laporte G, Mercier A (2001) A unified tabu search heuristic for vehicle routing problems with time windows. J Oper Res Soc 52:928–936
15. Cordeau JF, Gendreau M, Laporte G, Potvin J-Y, Semet F (2002) A guide to vehicle routing heuristic. J Oper Res Soc 53:512–522
16. Crainic TG, Ricciardi N, Storchi G (2004) Advanced freight transportation systems for congested urban areas. Transp Res C Emerg Tech 12(2):119–137
17. Crainic TG, Ricciardi N, Storchi G (2009) Models for evaluating and planning city logistic transportation systems. Transp Sci 43:432–454
18. Croes GA (1958) A method for solving travelling salesman problems. Oper Res 6: 791–812
19. Daskin MS, Owen SH (1999) Location models in transportation, chap. 10. In: Hall RW (ed) Handbook of transportation science. Kluwer, Boston
20. Drexl M, Schneider M (2013) A survey of location-routing problems. Technical report LM-2013-03, Gutenberg School of Management and Economics, Johannes Gutenberg University, Mainz
21. European Logistics Association (2004) A differentiation for performance: excellence in logistics. Dt. Verkehrs-Verl, Hamburg
22. Fisher M (1995) Vehicle routing, chapter 1. In: Ball MO et al (eds) Handbooks in OR and MS, vol 8. North Holland, Amsterdam
23. Fleischmann B, Gietz M, Gnutzmann S (2004) Time-varying travel times in vehicle routing. Transp Sci 38(2):160–173
24. Ghiani G, Laporte G, Musmanno R (2004) Introduction to logistics systems planning and control. Wiley, Chichester/Hoboken
25. Gillett B, Miller L (1974) A heuristic algorithm for the vehicle dispatch problem. Oper Res 22:340–349
26. Glover F (1977) Heuristics for integer programming using surrogate constraints. Decis Sci 8:156–166
27. Glover F (1986) Future paths for integer programming and links to artificial intelligence. Comput Oper Res 13:533–549
28. Goel A (2008) Fleet-telematics: real-time management and planning of commercial vehicle operations. Springer, New York
29. Goodman RW (2005) Whatever you call it, just don't think of last-mile logistics, last. Glob Logist Supply Chain Strateg 9(12):46–51
30. Grzybowska H (2012) Combination of vehicle routing models and dynamic traffic simulation for city logistics applications. Doctoral dissertation. Ph.D. dissertation. Universitat Politècnica de Catalunya, Spain
31. Grzybowska H, Barceló J (2012) Decision support system for real-time urban freight management. Procedia-Soci Behav Sci 39:712–725
32. Ichoua S, Gendreau M, Potvin JY (2003) Vehicle dispatching with time-dependent travel times. Eur J Oper Res 144:379–396
33. KUTSUPLUS (2013). http://www.wired.com/2013/10/on-demand-public-transit/
34. Linares MP, Barceló J, Carmona C, Montero L (2016) Analysis and operational challenges of dynamic ride sharing demand responsive transportation models, 2016. International sympo-

sium of transport simulation (ISTS'16 conference), transportation research procedia. Available online at www.sciencedirect.com
35. Linares MP, Montero L, Barceló J, Carmona C (2016) A simulation framework for real-time assessment of dynamic ride sharing demand responsive transportation models. In: TMK R, Frazier PI, Szechtman R, Zhou E, Huschka T, Chick SE (eds) . Proceedings of the 2016 winter simulation conference, Washington, DC
36. Lourenço HR (2005) Logistics management: an opportunity for metaheuristics. In: Rego C, Alidaee B (eds) Metaheuristics optimization via memory and evolution. Kluwer Academic, Boston, pp 329–356. ISBN:978-1-4020-8134-7
37. Malandraki C, Daskin MS (1992) Time dependent vehicle routing problems: formulations, properties and heuristics algorithms. Transp Sci 26(3):185–200
38. Nagy G, Salhi S (2007) Location-routing: issues, models and methods. Eur J Oper Res 177:649–672
39. Nguyen V-P, Prins C, Prodhon C (2012) Solving the two-echelon location routing problem by a GRASP reinforced by a learning process and path relinking. Eur J Oper Res 216:113–126
40. Nguyen PK, Crainic TG, Toulouse M (2013) A tabu search for time-dependent multi-zone multi-trip vehicle routing problem with time windows. Eur J Oper Res 231:43–56
41. Orozco J (2011) A microscopic traffic simulation based decision support system for real–time fleet management. Ph.D. disserattion. Universitat Politècnica de Catalunya, Spain
42. PORTAL (2003) Inner urban freight transport and city logistics. http://www.eu-portal.net
43. Potvin J-Y, Xu Y, Benyahia I (2006) Vehicle routing and scheduling with dynamic travel times. Comput Oper Res 33:1129–1137
44. Powell WB, Jaillet P, Odoni A (1995) Chap. 3: Stochastic and dynamic networks and routing. In: Handbooks in operations research and management science, vol 8. Elsevier
45. Psaraftis HN (1988) Dynamic vehicle routing problems. In: Golden B, Assad A (eds) Vehicle routing: methods and studies. North-Holland, Amsterdam
46. Psaraftis HN (1995) Dynamic vehicle routing: status and prospects. Ann Oper Res 61:143–164
47. Regan AC, Mahmassani HS, Jaillet P (1997) Dynamic decision making for commercial fleet operations using real-time information. Transp Res Rec 1537:91–97
48. Regan AC, Mahmassani HS, Jaillet P (1998) Evaluation of dynamic fleet management systems: simulation framework. Transp Res Rec 1645:176–184
49. Taillard É (1991) Robust taboo search for the quadratic assignment problem. Paral Coput 17:443–455
50. Tang H (2008) Efficient implementation of improvement procedures for vehicle routing with time-dependent travel times. Transp Res Rec 2089:66–75
51. Taniguchi E, Thompson RG, Yamada T, Van Duin R (2001) City logistics: network modeling and intelligent transport systems. Pergamon, Amsterdam
52. Thomas B (2007) Waiting strategies for anticipating service requests from known customer locations. Transp Sci 41(3):319–331
53. Thomas B, White C III (2004) Anticipatory route selection. Transp Sci 38(4):473–487
54. Toth P, Vigo D (2014) Vehicle routing: problems, methods, and applications. MOS-SIAM series on optimization, 2nd edn ISBN:978-1-611973-58-7
55. Tragantalerngsak S, Holt J, Rönnqvist M (1997) Lagrangian heuristics for the two-echelon, single-source, capacitated facility location problema. Eur J Oper Res 102:611–625
56. UBER (2015). http://www.uber.com
57. Wren A (1971) Computers in transport planning and operation. Ian Allan, London
58. Wren A, Holliday A (1972) Computer scheduling of vehicles from one or more depots to a number of delivery points. Oper Res Q 23:333–344

Cutting and Packing

39

Ramón Alvarez-Valdes, Maria Antónia Carravilla, and José Fernando Oliveira

Contents

Introduction 1220
Cutting and Packing Problems 1221
Dimensionality 1221
Geometry 1222
Problem Types 1225
Additional Characteristics and Constraints 1227
Rectangular Problems (2D and 3D) 1230
Constructive Heuristics 1230
Local Search Heuristics 1234
Metaheuristics 1236
Matheuristics 1245
Irregular Problems 1247
Constructive Heuristics 1247
Local Search Heuristics 1254
Matheuristics 1257
Metaheuristics 1259
Conclusion 1263
Cross-References 1264
References 1264

R. Alvarez-Valdes (✉)
Universitat de València, Burjassot, Spain
e-mail: ramon.alvarez@uv.es

M. A. Carravilla · J. F. Oliveira
INESC TEC and Faculty of Engineering, University of Porto, Porto, Portugal
e-mail: mac@fe.up.pt; jfo@fe.up.pt

© Springer Nature Switzerland AG 2025

R. Martí et al. (eds.), *Handbook of Heuristics*,
https://doi.org/10.1007/978-3-032-00385-0_43

Abstract

Cutting and Packing (C&P) problems arise in many industrial and logistic applications, whenever a set of small items, with different shapes, has to be assigned to large objects with specific shapes so as to optimize some objective function. Besides some characteristics common to combinatorial optimization problems, the distinctive feature of this field is the existence of a geometric subproblem, to ensure that the items do not overlap and are completely contained in the large objects. The geometric tools required to deal with this subproblem depend on the shapes (rectangles, circles, irregular) and on the specific conditions of the problem being solved. In this chapter, after an introduction that describes and classifies Cutting and Packing problems, we review the basic strategies that have appeared in the literature for designing constructive algorithms, local search procedures, and metaheuristics for problems with regular and irregular shapes.

Keywords

Cutting stock · Bin packing · Strip packing · Container loading nesting

Introduction

Cutting and Packing (C&P) problems are hard combinatorial optimization problems (almost all are NP-hard) that arise in the context of several real-world applications, both in industry and in services, whenever one or more large objects or container's space has to be divided into smaller items, so that waste is minimized. These problems include cutting paper rolls into narrower rolls in the paper industry, cutting large boards of wood into smaller rectangular panels in the furniture industry, or cutting the irregularly shaped components of garment from fabric rolls, but they include also packing boxes into containers or loading items on pallets in logistics applications. All the problems have in common the existence of a geometric subproblem, originated by the natural item non-overlapping constraints.

In the literature, these problems have received an increasing amount of attention. In the 1960s, the seminal papers were published, and the number of publications has exponentially grown since then. As combinatorial optimization problems, C&P problems may be solved using all the optimization approaches and techniques available nowadays: linear programming techniques, branch and bound search algorithms, constraint logic programming, dedicated heuristics, metaheuristics, mathheuristics etc. Unfortunately, due to their combinatorial nature, exact techniques are not capable of efficiently tackling these problems for large instances; therefore, heuristic approaches must be used.

The remaining of this chapter is organized as follows. In the next section, C&P problems will be formally defined and categorized, according to their objective, and to their quantitative and geometric parameters. Then, in the following two sections, heuristic approaches for rectangular problems and irregular problems, both two-

and three-dimensional, are presented, ranging from simple constructive heuristics to mathematical programming model-based heuristics and including metaheuristics.

Cutting and Packing Problems

A carpenter wants to build a wardrobe for which he/she needs to cut a set of rectangular items in the quantities and with the dimensions presented in Table 1. To cut these items, the carpenter has at his/her workshop two large wood panels of 3 by 2 m and three other large wood panels of 2.5 by 2.5 m. How should the carpenter cut the small items from the available large panels to minimize the waste (the trim of the cutting process)?

This toy example allows us to introduce the main concepts of Cutting and Packing problems. In general, in Cutting and Packing problems, "small" items have to be assigned to "large" objects under both geometric constraints (assuring that the items do not overlap and are contained inside the objects) and quantitative constraints (e.g., at least a given quantity of each item type has to be cut). The objective of the problem may be either related to the minimization of the value of the large objects that are used or to the maximization of the value of the small items that are cut. The solution to the problem is one or more cutting patterns describing the geometric disposition of the small items in/on the large objects (Fig. 1). It should be emphasized that, in general, to describe the solution of a Cutting and Packing problem is not enough to say which small items are cut from which large objects (the assignment part of the problem). Part of the solution is describing how the small items are arranged on the large objects (the geometric problem).

These problems can be categorized according to a typology proposed by Wäscher et al. [105]. This typology takes into account the problem objective, the number of objects and items, and the geometry of the items. The different types of Cutting and Packing problems will be described by closely following the Wäscher et al. typology.

Dimensionality

Cutting and Packing problems are usually classified according to their dimensionality as one-, two-, and three-dimensional problems. This classification is directly

Table 1 Example of a two-dimensional rectangular cutting problem

Item type	Quantity	Length (m)	Width (m)
1	2	2.00	0.50
2	2	1.80	1.00
3	3	1.90	0.65
4	2	2.10	0.55
5	6	0.65	0.23

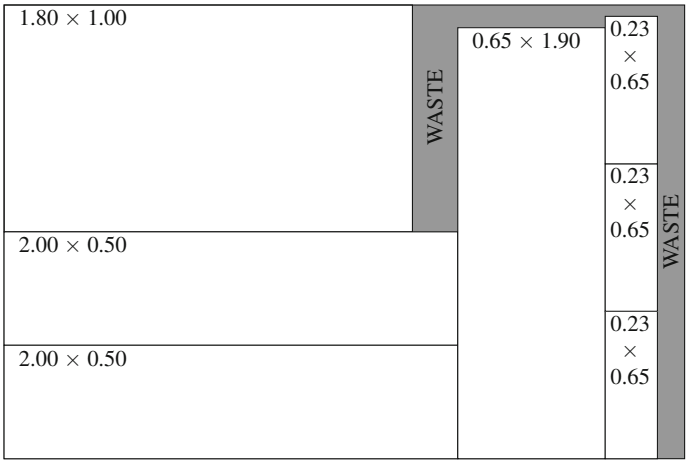


Fig. 1 A two-dimensional cutting pattern example

related to the number of physical dimensions relevant for the problem. If it is true that our world and all the objects in it are three-dimensional, when planning how to cut narrower paper rolls from a wider roll, only one dimension is relevant as the other two are kept equal in small items and large objects, i.e., the problem is one-dimensional (1D). Applying the same reasoning to the initial carpenter problem, it is the length and the width of the large objects that must be cut in order to match the length and the width of the small items, not changing the thickness of the wood boards; this is therefore a two-dimensional problem (2D). When packing parallelepiped boxes inside a truck or a container, the three dimensions are relevant for the problem, and it is therefore a three-dimensional problem (3D). Although four- and even higher-dimensional problems are theoretically possible, very few relevant practical applications have been described until now (Christensen et al. [35]), and therefore, they will be disregarded in this text. In Fig. 2, examples of 1D, 2D, and 3D problems arising in the paper, garment, furniture, and logistics industries are shown.

Geometry

The second most distinctive characteristic of the Cutting and Packing problems is the geometry associated with the dimensions that are relevant for the problem. In 1D problems, the geometric problem is trivial as, once the assignment problem is solved and it is known which small items are cut from each large object, the cutting order is not relevant as any sequence is feasible. Two- and three-dimensional problems are classified as rectangular, circular, or irregular problems. This division is related with the type of geometric tools needed to handle the geometric constraints of the problem. Geometric feasibility is guaranteed if the small items do not overlap each

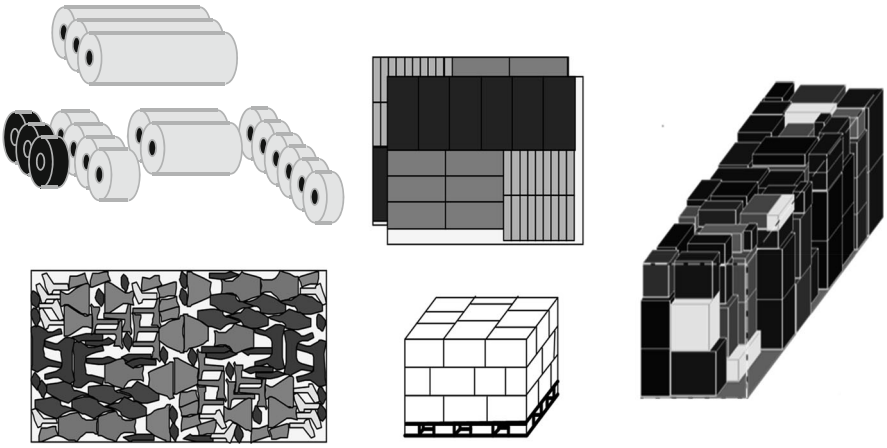


Fig. 2 Examples of one-, two-, and three-dimensional Cutting and Packing problems

other and are completely contained inside the large object. For the sake of simplicity, in the following only two dimensions will be considered, but the same reasoning and concepts hold for three-dimensional problems.

When dealing with rectangular shapes, analyzing if they do not overlap each other is just a question of coordinate comparison. Let rectangle i, with dimensions (l_i, w_i), have its left lower corner placed on coordinates (x_i, y_i) of the Cartesian plane, and let rectangle j, with dimensions (l_j, w_j), have its left lower corner placed on coordinates (x_j, y_j) of the Cartesian plane. Rectangle j does not overlap rectangle i if it is either above, below, or to the left or right of rectangle j, i.e.,

$$x_j + l_j \leq x_i \quad \vee \quad x_j \geq x_i + l_i \quad \vee \quad y_j \geq y_i + w_i \quad \vee \quad y_j + w_j \leq y_i$$

The containment conditions of rectangle i inside the larger rectangle (L, W) (assuming that its left lower corner is placed on coordinates $(0, 0)$) are:

$$x_i \geq 0 \quad \wedge \quad x_i + l_i \leq L \quad \wedge \quad y_i \geq 0 \quad \wedge \quad y_i + w_i \leq W$$

In circular Cutting and Packing problems, small items have circular shapes and the large object can be either another (larger) circle or a rectangle. The condition for the items not to overlap is now based on the computation of the distance between the centers of the circles. Let circle i, with radius r_i, have its center placed on coordinates (x_i, y_i) of the Cartesian plane, and let circle j, with radius r_j, have its center placed on coordinates (x_j, y_j) of the Cartesian plane. Circle j does not overlap circle i if the distance between the two centers is greater than or equal to the sum of their radii, i.e.,

$$\sqrt{(x_i - x_j)^2 + (y_i - y_j)^2} \geq r_i + r_j$$

It should be noticed that it is not necessary to compute the computationally expensive square root, and this condition can be replaced by the equivalent one having both sides squared.

The containment condition of circle i, with its center placed on coordinates (x_i, y_i), inside the larger circle of radius R and center on coordinates $(0, 0)$ is:

$$\sqrt{x_i^2 + y_i^2} \leq R - r_i$$

If the container is a rectangle (L, W), with its bottom left corner on coordinates $(0, 0)$, the containment conditions are:

$$x_i - r_i \geq 0 \quad \wedge \quad x_i + r_i \leq L \quad \wedge \quad y_i - r_i \geq 0 \quad \wedge \quad y_i + r_i \leq W$$

All shapes other than rectangles and circles are considered irregular and require more complex geometric algorithms to check and enforce geometric feasibility. A complete and thorough tutorial on the geometry of irregular Cutting and Packing problems (aka as nesting problems) can be found in Bennell et al. [20]. As the no-fit polygon is the most widely used geometric tool to analyze if two irregular shapes overlap, it will be described for the simpler case of convex polygons.

Let P_i be a polygon with n_i vertices with coordinates $(a_i^k, b_i^k), k = 1, \ldots, n_i$ relative to an arbitrary reference point. Let P_j be another polygon with n_j vertices with coordinates $(a_j^m, b_j^m), m = 1, \ldots, n_j$ also relative to an arbitrary reference point. Consider that the reference point of polygon P_i is placed on coordinates (x_i, y_i) and that the reference point of polygon P_j is placed on coordinates (x_j, y_j). The direct comparison of the relative positions of two polygons is a complex and computationally heavy task, mainly in the general case when the polygons are non-convex. An alternative is to resort to the concept of no-fit polygon, a new polygon that captures the geometric information of both polygons. In fact, the edges of the no-fit polygon are the edges of the two polygons taken in a special order. With the no-fit polygon, checking the relative position of two polygons is reduced to checking the relative position of one point against the no-fit polygon.

To build the no-fit polygon, consider polygon P_i fixed and polygon P_j sliding around P_i, always in contact but never intersecting P_i. The trace of the reference point of P_j during this movement is the no-fit polygon of P_j in relation to P_i: $NFP_{P_iP_j}$ (Fig. 3). The reference point of $NFP_{P_iP_j}$ is the same as the reference point of P_i. In order to guarantee that all the small items do not overlap, it is necessary to build the no-fit polygons for all pairs of items. They can, however, be built in a preprocessing phase, as they do not depend on the absolute coordinates of the reference point of P_i.

Let $NFP_{P_iP_j}$ be the no-fit polygon of polygon P_j in relation to polygon P_i, with N_{ij} vertices with coordinates $(a_{ij}^k, b_{ij}^k), k = 1, \ldots, N_{ij}$, and with its reference point placed on coordinates (x_i, y_i). Polygon P_j will not intersect polygon P_i if its reference point is on coordinates (x_j, y_j) such that:

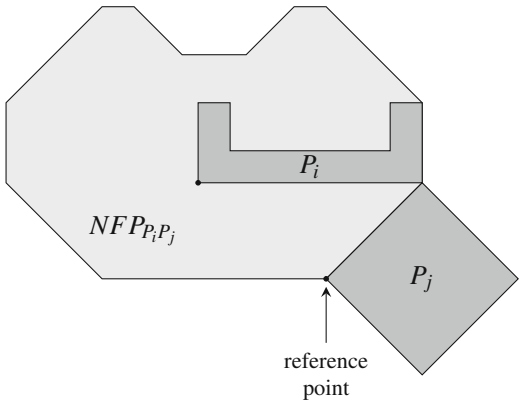


Fig. 3 The no-fit polygon of polygon P_j in relation to polygon P_i

$$\exists k \in \{1, \ldots, N_{ij} - 1\} : y_j \geq \frac{b_{ij}^{k+1} - b_{ij}^{k}}{a_{ij}^{k+1} - a_{ij}^{k}}(x_j - a_{ij}^{k}) + b_{ij}^{k}$$

i.e., the comparison of the coordinates of one point against one polygon.

Note that the direction of the inequality in the aforementioned expressions (" $\geq$" or " $\leq$") depends on the orientation chosen for the polygons, i.e., if vertex $k + 1$ is on the left-hand side or on the right-hand side of the line defined by the vertices $k - 1$ and k.

Problem Types

From the non-geometric point of view, Cutting and Packing problems can be formally defined as follows:

> A set of small items of m distinct types, with distinct geometries and values v_i, $i = 1, \ldots, m$, have to be assigned in a minimum amount of d_i^L and a maximum amount of d_i^U units, to a set of n distinct types of large objects, with distinct geometries and values V_j, $j = 1, \ldots, n$ available in a limited amount of D_j units.

The objective of the problem may be either *output maximization* or *input minimization*. If there are more small items to cut/pack[1] than space available in the large objects, the goal is to select the subset of small items to pack so that the value extracted from the large objects (the value of the small items effectively packed) is maximized. Therefore, these problems are of *output maximization*, where the *output*, the value v_i of the small items, may be proportional or not to their area/volume. In this situation, as there are more small items to pack than large objects available, the value V_j of the large objects is irrelevant as all the large objects will be used. If there are more large objects available than the ones needed to pack all the small items, the

[1] In what follows, we will use the term *pack* to refer indistinctly to Cutting and Packing processes.

goal is to select the subset of large objects to use so that the value of the used large objects is minimized. Therefore, these problems are of *input minimization*, where the *input*, the value V_j of the large objects, may be proportional or not to their area/volume. In this situation, as there are more large objects available than the ones needed to pack all the small items, the value v_i of the small items is irrelevant as all small items will be packed.

Depending on the values of m, d_i^L, d_i^U, n, and D_j, and on the objective of the problem, we will get the six main Cutting and Packing problem types, according to Wäscher et al.'s typology:

- Identical Item Packing Problem (IIPP)
- Placement Problem (PP)
- Knapsack Problem (KP)
- Cutting Stock Problem (CSP)
- Bin Packing Problem (BPP)
- Open Dimension Problem (ODP)

The first three are *output maximization* problems and the last three are *input minimization* problems.

In the IIPP, the small items are of only one type ($m = 1$), i.e., all the items are geometrically equal, but there is not a maximum demand for the item ($d_i^U = \infty$), i.e., that item should be packed as many times as possible. In the PP, m types of items have to be packed, but there is an upper limit on the number of items of each type to be packed (d_i^U). In the KP, all the small items are geometrically different, and therefore, the number of items is equal to the number of types of items (m) and the upper limit on the number of items to pack of each type is 1 ($d_i^U = 1$).

In what concerns the *input minimization* problems, while in the CSP, there are few types of items (m), with a demand d_i^L greater than or equal to 1, in the BPP, all the small items are different and have a demand of 1 ($d_i^L = 1$). In both cases, the large objects may be all different, i.e., there is only one unit of each large object ($D_j = 1$), the large objects may be of several types n with an availability D_j for each type, or they may be of just one type ($n = 1$) with an infinite availability ($D_j = \infty$). In the ODP, there is an "infinite" availability of the large object due to the open (not limited) dimension of the large object. An example is the rectangular strip packing problem, in which small rectangles have to be packed on a rectangle of "infinite" length, so that the length used is minimized. This happens in the textile industry where items are cut from fabric rolls. In practical applications of ODP, the m types of small items have a demand d_i. The characterization of the six types of problems, based on the values of their parameters, is resumed in Table 2. The symbol "–" stands for "*not relevant for the problem type characterization*."

In *output maximization* problems, in which there is an upper bound on the number of small items of each type to pack (d_i^U), there may also exist a lower bound d_i^L, meaning that it is mandatory to pack at least d_i^L small items of type i.

Table 2 Characterization of the six types of problems based on the values of their parameters

Problem	Objective	m	d_i^L	d_i^U	n	D_j
IIPP	output maximization	1	–	∞	–	–
PP	output maximization	≥ 1	–	≥ 1	–	–
KP	output maximization	≥ 1	–	1	–	–
CSP	input minimization	≥ 1	≥ 1	–	1 ≥ 1	∞ 1
BPP	input minimization	≥ 1	1	–	1 ≥ 1	∞ 1
ODP	input minimization	≥ 1	≥ 1	–	1	1

The six types of problems are represented in Figs. 4 and 5 for the two-dimensional rectangular Cutting and Packing problem. The small items and large objects effectively cut/used in a hypothetical solution are represented in dark gray to highlight the difference between *output maximization* problems where all large objects are used and *input minimization* problems where all small items are pack.

Additional Characteristics and Constraints

Many additional constraints to the cutting/packing patterns arise from the practical applications of these problems. These additional constraints are specific for each dimensionality or kind of geometry; however, two characteristics emerge as common to almost all two- and three-dimensional Cutting and Packing problems: small item orientation and guillotine patterns.

Given the physical characteristics of the raw materials from which the large objects are made, sometimes it is not possible to rotate small items, i.e., they have to be packed with the dimensions and orientation given: length strictly along the length and width strictly along the width. It is the case of wood boards or fabric rolls, for instance. In other cases, there is no problem in rotating the small items when placing them on the large objects, as it happens in the steel industry. Even when small item rotations are allowed, it is usual to differentiate the cases where only 90 and 180 degree rotations are allowed, which lead to orthogonal patterns, from other situations when other rotation angles are also admissible, in what is usually designated as free rotation. In three-dimensional packing problems, as container loading problems, there may be "this-side-up" constraints that restrict the number of feasible orientations in which a box may be loaded in the truck or container.

When dealing with rectangles, guillotine constraints may be imposed to the cutting patterns. A cutting pattern is said to be *guillotinable* if all the small items can be extracted with top-down (right-left) cuts made on the large object. Each cut produces two rectangles that may be the demanded small items or may be bigger rectangles to cut according to the same rule. Guillotine cutting patterns can be classified according to the number of times that the cutting tool has to be turned 90 degrees. If a sequence of horizontal (vertical) cuts, generating strips, followed by a

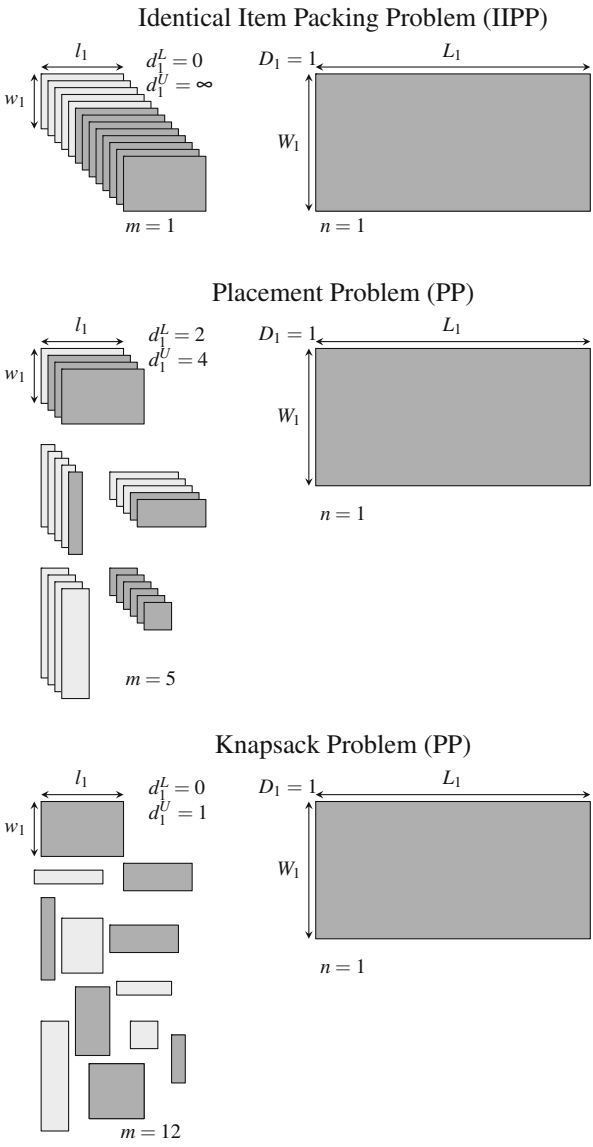


Fig. 4 Illustration of the three types of *output maximization* Cutting and Packing problems for the two-dimensional rectangular form of the problem

sequence of vertical (horizontal) cuts on each strip completely executes the cutting pattern, then this pattern is called a two-stage pattern. If an additional horizontal (vertical) cut is needed to cut the small items, then the pattern is classified as a three-stage pattern. The pattern is said to be n-stage if the cutting tool must be turned n times to complete the pattern. Figure 6 presents two-, three-, and n-stage guillotine cutting patterns and a non-guillotine cutting pattern. These concepts can be extended to three dimensions, when dealing with the cut of parallelepipeds.

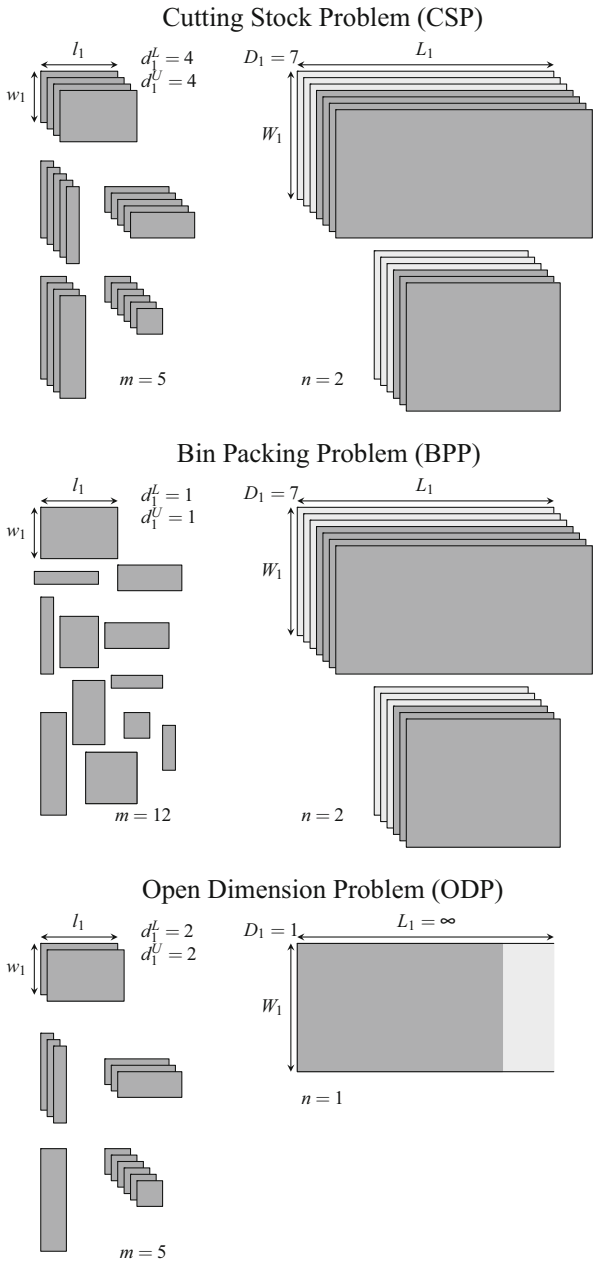


Fig. 5 Illustration of the three types of *input minimization* Cutting and Packing problems for the two-dimensional rectangular form of the problem

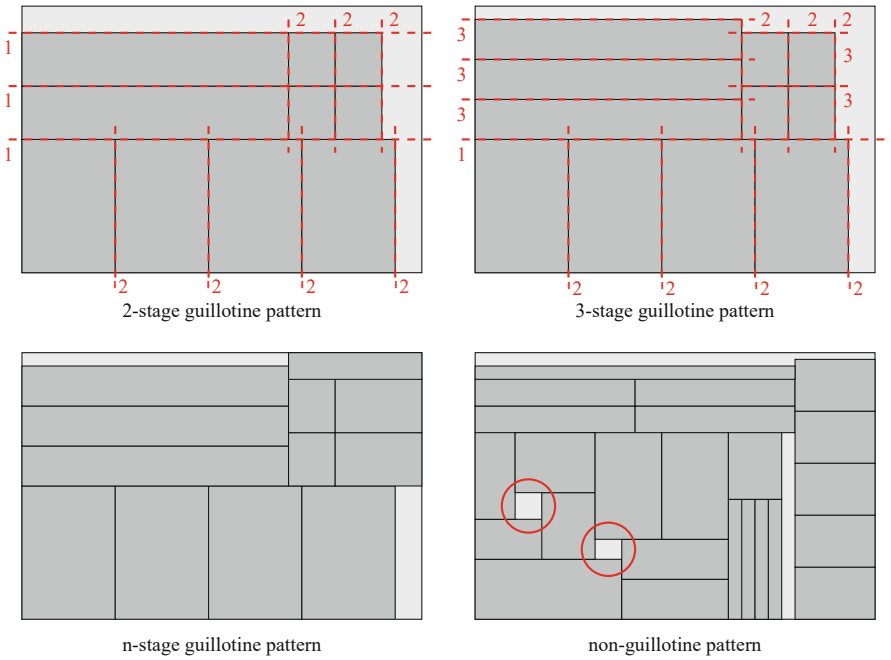


Fig. 6 Two-, three-, and n-stage guillotine cutting patterns and a non-guillotine cutting pattern

Rectangular Problems (2D and 3D)

This section describes the most widely used techniques for developing heuristic algorithms for Cutting and Packing problems involving rectangular items in two and three dimensions. First, constructive procedures will be described, then local search methods, and finally, metaheuristic algorithms successfully used for the basic versions of the problems. The section will conclude with some references to model-based heuristics.

Constructive Heuristics

Most of the heuristic algorithms include some constructive procedure in which a feasible solution is iteratively built by adding at each iteration a subset of items to the existing partial solution. All of these procedures involve two basic steps: selection of the items to be packed and selection of the space in which to pack them. Depending of the order in which these two steps are called, two types of algorithms can be distinguished:

1. *Item-driven algorithms*
   In this type of algorithms, the item to be packed is chosen first and then the best location for packing it is selected. The best-known example of this type is the *Bottom-Left* (*BL*) algorithm, proposed by Baker et al. [16], and then used by many other authors. In the *BL* algorithm, an ordered list of items is given and at each step the first item in the list is taken and placed to the lowest and left-most position in the empty space. In Jakobs's implementation [64], the position is obtained by putting the item at the upper right corner of the space and then shifting it alternately as far as possible to the bottom and then as far as possible to the left, as in Fig. 7a. Liu and Teng [75] developed an improved implementation giving priority to the downward movement, so the item is only shifted to the left when it cannot be shifted downward, as in Fig. 7b. They showed that, unlike Jakobs's method, there is always an ordering of the items that can produce the optimal solution.

   A refinement of the *BL* algorithm is the *Bottom-Left-Fill* (*BLF*) algorithm proposed by Chazelle [31], in which the empty spaces between items already packed are also considered to accommodate the new item. The algorithm has to maintain a list of points in a bottom-left order to indicate where the new item can be placed. These points are checked in order until a feasible position is found. Figure 8 compares *BLF* with *BL* on an example. A new item has to be added to the partial solution and is initially placed at the upper right corner (Fig. 8a). In Fig. 8b, the *BLF* algorithm is used. The figure shows the candidate points and the position selected for the new item. Using the *BL* algorithm the solution in Fig. 8c is obtained. Hopper and Turton [60] showed that the *BLF* heuristic outperforms the *BL* routine by up to 25% with the performance gain being higher for larger problems. However, its complexity is $O(n^3)$, while the complexity of *BL* is just $O(n^2)$.

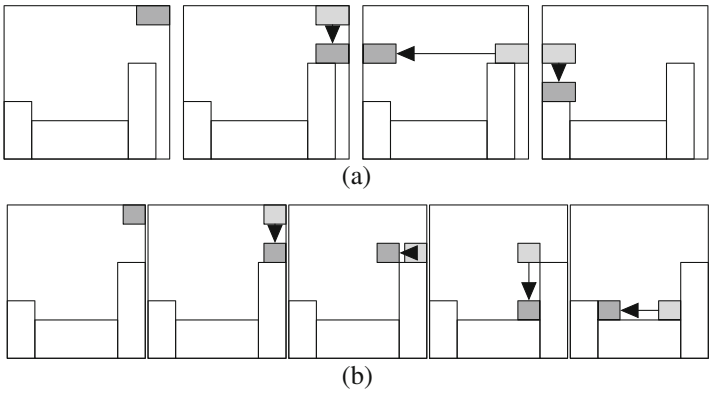


Fig. 7 Bottom-Left Algorithm. (**a**) Jakob's implementation. (**b**) Liu and Teng's implementation

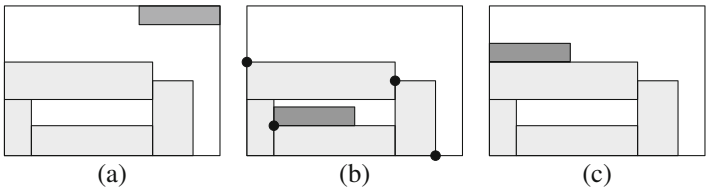


Fig. 8 Comparing Bottom-Left-Fill and Bottom-Left Algorithms. (**a**) Placing a new item. (**b**) Placed with BLF. (**c**) Placed with BL

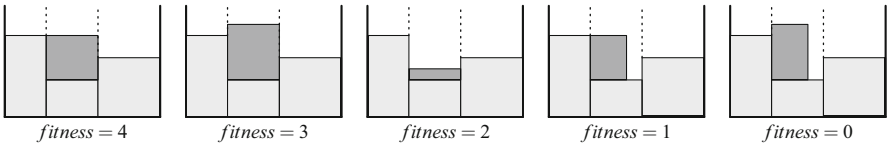


Fig. 9 Scoring rule of Leung et al.

2. *Space-driven algorithms*

In this type of algorithms, a space is selected first from the list of available spaces and then the item or the subset of items that fits best into the space is chosen to be packed.

An example of this type of procedure is the *Best-Fill* (*BF*) algorithm proposed by Burke et al. [26] for the strip packing problem in which, at each step, the lowest space is chosen and then the item that fits best is chosen to occupy it.

A further refinement of the *BF* algorithm is the *Scoring* rule by Leung et al. [69], in which the quality of the fitting is evaluated using a fitness measure that takes into account other factors besides the way in which the base of the item fills the base of the space. Figure 9 shows the empty space selected and the fitness values corresponding to candidate items.

Apart from these simple cases, the algorithms in which the space is chosen first are usually more complex, because they have to decide how to manage the empty spaces and because they usually choose subsets of items to fill the spaces instead of just one item.

Concerning the management of the spaces, there are basically two alternatives: partition the empty space into smaller disjoint spaces or consider non-disjoint maximal spaces [66].

Partitions are very useful when guillotine cuts are considered or in three-dimensional problems when the items have to be fully supported from below by other items or the base of the large object. They are also easier to manage, because every time an item is placed in a space, it produces some new spaces to substitute the previous one, without modifying any other space in the list.

Conversely, maximal spaces are more flexible and can produce solutions that partitions are not able to attain, but they are more difficult to manage. As maximal spaces are not disjoint, putting an item into one space may modify other spaces and the whole list of spaces has to be checked. An example of

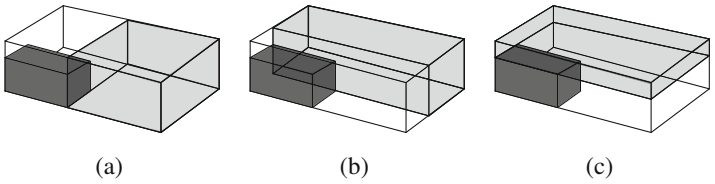


Fig. 10 Space management in 3D: maximal spaces

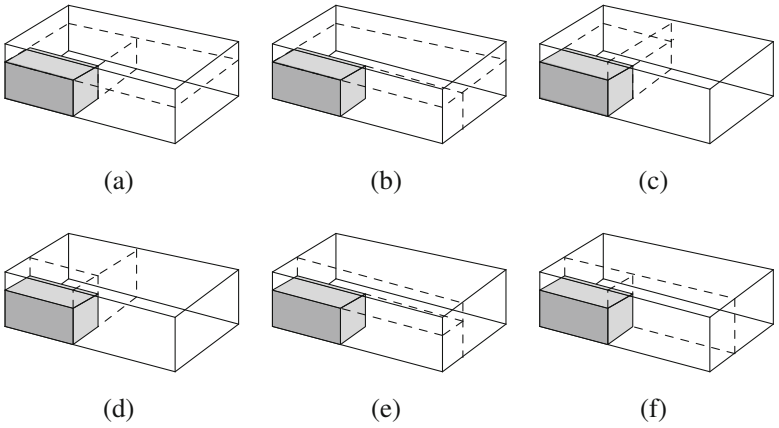


Fig. 11 Space management in 3D: partitions

maximal spaces appears in Fig. 10. Initially the parallelepiped is empty and when an item is packed at its bottom left corner, three maximal spaces are generated. Figure 11 shows an example of partition. In this case, when putting the item at the bottom-left corner of the parallelepiped, a decision has to be taken about the new spaces generated. Six possibilities appear, depending on the horizontal or vertical cuts. Most approaches follow the strategy of generating the largest possible front space, then maximizing the side space, and finally, the top space holds the remaining volume. This strategy aims to produce walls of objects that are transverse to the length of the paralellepiped, while providing full support for the objects. A notable exception is presented in Xiang et al. [109], where priority is given to the size of the top space, resulting in packing patterns based on layers instead of walls.

The other aspect that the algorithms have to decide is if, at each step, just one item is selected to be packed or if a subset of items with a certain structure is selected. In this second case, the items come in *blocks*, that is, arranged in a structure of rows and columns. Blocks have been used by many authors and can be considered the standard way of packing items once the space has been chosen. In most cases, the blocks are *homogeneous*, that is, composed of just one type of item, as in Fig. 12a, but sometimes, in more recent studies (Fanslau and Bortfeldt [47], Zhu et al. [112], Wei and Lim [106]), they are *heterogeneous*, including

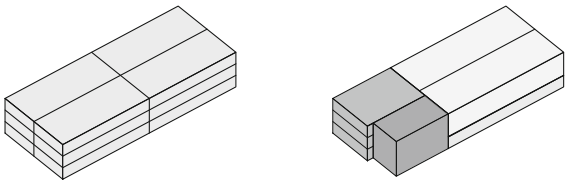


Fig. 12 Homogeneous and heterogeneous blocks

(a) Homogenous block (b) Heterogenous block

items of different types as long as their dimensions do not differ more than a given threshold, as in Fig. 12b.

Finally, there are algorithms that select simultaneously both elements, the item or block to be packed and the space in which to pack it. An example is the procedure proposed by Wu et al. [108] for packing rectangles into a rectangle, in which, following the principle that "Golden are the corners; silvery are the sides; and strawy are the voids," all the unpacked items are evaluated to be placed at each corner of the remaining empty space. The placement algorithm proposed by Wei et al. [107] considers all pairs (p, r) (position-rectangle), selects one of them according to some fitness criteria, and places the rectangle in this position.

Local Search Heuristics

There are several ways to define and explore the neighborhood of a solution. Two main groups of local search procedures can be distinguished: those in which the solution is given by an ordered list of items, that will be decoded into a physical solution by means of an algorithm such as BL or BLF, and those in which the layout of the solution is used to define the neighborhood.

1. *Procedures based on ordered lists of items*
   In Hopper and Turton [60], the problem is represented by a permutation of items, giving the order in which the items are packed. They define two manipulation operations. The first one swaps two randomly selected items in the permutation. The second operator flips the orientation of one randomly selected item. In the translation to the next solution, only one of the operators is applied with a 50% chance. The initial solution is randomly generated.

   Leung et al. [70] apply the scoring algorithm [69] to an ordered list of items. If two items have the same score, the first item on the list is packed first. Therefore, different item sequences can generate different solutions. They exhaustively study the sequences produced by exchanging the positions of every pair of items and keep the best solution found.

   Ceschia and Schaerf [30] deal with a multi-container problem and work with a set of sequences, one for each container. Each element of a sequence is a homogeneous block. A move consists of moving some items of a block to another position of the same or another sequence, keeping or changing their orientation.

An example appears in Fig. 13 [30], involving two containers. Part of the block in position 4 in the first sequence is moved to position 4 in the second sequence.

2. *Procedures based on the layout of the solutions*
   The movements determining the neighborhood of a solution can be defined directly on the layout of the solution, taking into account the position of the items on the large objects. Some examples of these moves can be seen in the following figures. Figure 14 [9] shows a simple move in which an item is selected to change its position on the strip. Once it is selected, it is removed and packed again on an empty space in which the item can fit or not. It is does not fit, as in the example of the figure, the overlapping items are removed and packed again by using a constructive procedure.

   In Fig. 15 [86], a block of items that were left initially unpacked are inserted into a bin. The overlapping items are removed and the new empty spaces are filled again.

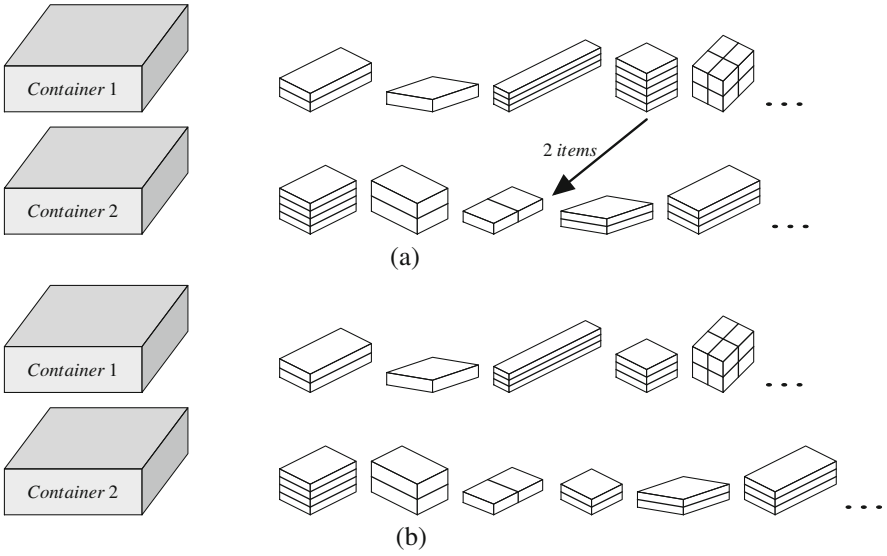


Fig. 13 Ceschia and Schaerf's movement. (**a**) Initial solution. (**b**) New solution

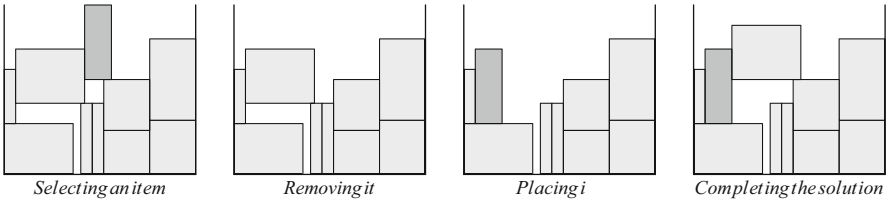


Fig. 14 Changing the position of an item

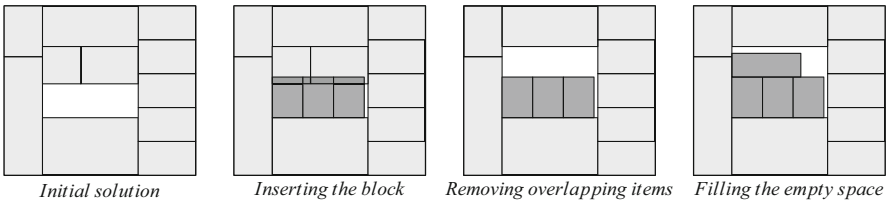


Fig. 15 Inserting a block

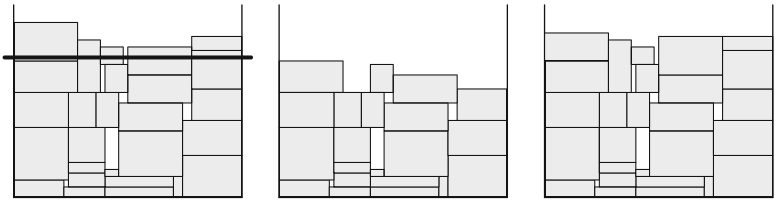


Fig. 16 Removing and filling in a two-dimensional problem

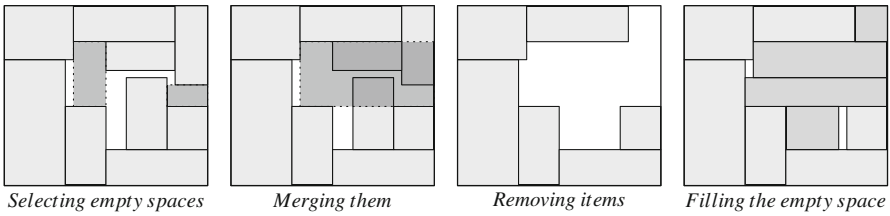


Fig. 17 Merging empty spaces in two and three dimensional problems

A frequently used type of movement consists of removing part of the solution and rebuilding the partial solution in a way that produces a different solution. A two-dimensional example appears in Fig. 16 [9], in which the last part of the solution of a strip packing problem is removed and the strip filled again, using a different procedure. Finally, Fig. 17 [86] shows a more complex move, which can be applied to two- and three-dimensional problems. Two empty spaces are selected and then the smallest rectangle (or parallelepiped in 3D) containing them is determined and the items contained totally or partially in it are removed. The resulting space is filled again, producing a new solution.

Metaheuristics

Multi-start Algorithms

Lesh et al. [67] develop the $Bottom - Left - Descending$ (BLD) algorithm, in, which the known BL algorithm is used four times, ordering the items by decreasing height, width, area, and perimeter. In order to improve BLD, they

develop a stochastic variation, algorithm BLD^*. BLD^* starts with a fixed order (say decreasing height) and generates random permutations from this order. Items are selected in order, one at a time. For each selection, BLD^* goes down the list of remaining items, accepting each item with probability p, until an item is accepted. If the last item is reached and not selected, it restarts at the beginning of the list. After an item is accepted, the process starts again from the beginning of the list. The probability of obtaining some ordering y, starting from some fixed ordering x, is proportional to $(1 - p)^{Ken(x,y)}$, where $Ken(x, y)$ is the Kendall–Tau distance between the two permutations.

Belov et al. [17] propose two iterative heuristics for the strip packing problem. The first one uses a special single-pass heuristic *SubKP* in each iteration. *SubKP* fills the selected empty space by solving a 1D knapsack problem, considering only item widths. At each iteration, the profits of the items change according to a sequential value correction method [82]. The second iterative heuristic, *BS(BLR)* uses at each iteration a single-pass heuristic *Bottom-Left-Right (BLR*), a modification of *BL*, in which the item can be assigned to the left or to the right in the most bottom-left free space large enough for that item. To construct item orderings for *BLR*, a simplified *BubblePermute* [68] procedure is used.

Araujo and Armentano [11] also propose a multi-start random constructive algorithm. At each step of each constructive process, a list of homogeneous blocks are evaluated and ranked. A biased random selection determines the block to pack but, before packing it, another biased random procedure reduces the rows and/or columns of the block. These two randomizing strategies guarantee the diversity of the solutions obtained.

Genetic Algorithms

Most of the genetic algorithms proposed for solving Cutting and Packing problems use a representation in which each solution is given by an ordered list of items and a decoding algorithm translates this list into a physical solution.

Jakobs [64] already used this representation in his genetic algorithm, with an specific crossover operator and a mutation operator that exchanges the position of some elements of the permutation. The solutions are decoded by using the *BL* algorithm.

Hopper and Turton [60] use this representation, with partially matched crossover (PMX) and order-based mutation. They also consider elitism and seed the initial population with a solution obtained by a heuristic placement procedure. As decoding algorithm they use *BL* and *BLF*.

Leung et al. [72] also use as chromosomes ordered list of items, a mutation operator consisting of swapping the positions of two items randomly chosen, and two-point and cycle crossover operators. They also include a simulated-annealing operator to induce competition between parents and children. If the children are better, they are accepted, but if the children are not as good as their parents, they can still be accepted with some probability. This helps to prevent the population from becoming homogeneous too early. The solutions are decoded using a procedure similar to the *BLF*.

Iori et al. [63] also use the ordered list representation. The main difference with respect to standard procedures is that they distinguish between improving and non-improving phases, depending on the number of individuals generated without improving the best solution obtained. The phase influences the fitness evaluation, the elitism strategy, and the use of mutation. Another special feature is the use of local search to improve the individual with the largest fitness. Although they use a more sophisticated decoder, Feng et al. [49] follow a similar idea.

A different approach to genetic algorithms based on list of items is the Biased Random-Key Genetic Algorithm proposed by Gonçalves and Resende [56] for the Bin Packing Problem. If the instance being solved has n items, the chromosome has $2n$ genes, the first n for the sequence of items, and the last n for their orientations. Each individual of the population is a vector of $2n$ random keys. Initially, the values are generated at random in the real interval [0, 1]. At each iteration, some mutants are added, generated in the same way that the elements of the initial population. Also, a small group of elite individuals are taken from the previous generation. The remaining members are obtained by parameterized uniform crossover, combining elite and non-elite individuals. For each component of the vector, the elite individual component is taken with a given probability. Once the individuals are formed, they are decoded, first ordering the items according to the values of their random keys, to produce an item sequence, and then placing the items into the bins using some placement algorithms derived from *BLF*. Figure 18 shows how the part of the chromosome corresponding to the items is sorted to produce an item sequence. A similar approach is used in Zudio et al. [113], but with the addition of using a Variable Neighborhood Descendent algorithm to improve the solutions obtained by crossover.

A completely different structure for a genetic algorithm is proposed by Bortfeldt and Gehring [24] for the container loading problem. Unlike all previous algorithms, they use the layer structure of the solution. Each feasible solution is composed of a set of layers, whose height and width are the same as the container and whose depth

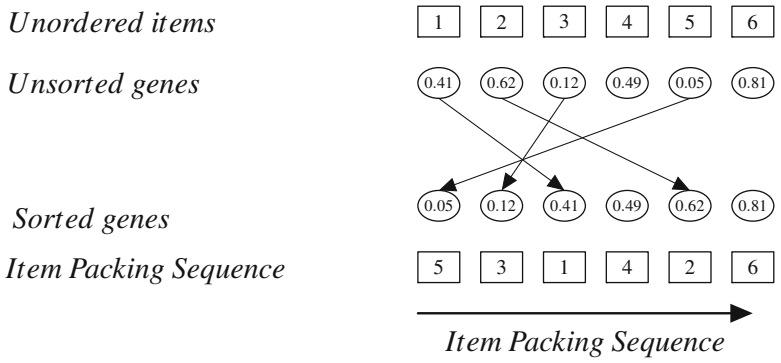


Fig. 18 Decoding of the item sequence in the algorithm of Gonçalves and Resende

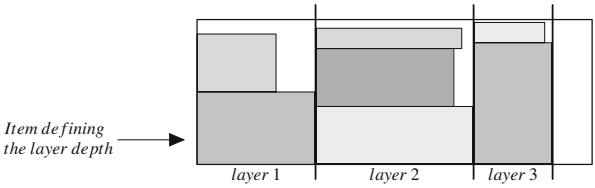


Fig. 19 Layer structure of the Bortfeldt and Gehring's solutions

is defined by one of their items (see Fig. 19). The constructive algorithm builds a solution with this layer structure and the genetic operators build new solutions by combining layers of previous solutions and including new layers for completing solutions when necessary.

Simulated Annealing Algorithms

Hopper and Turton [60] propose a simulated annealing algorithm in which the neighborhood is defined by the set of solutions that can be reached applying two manipulation operations. The first one swaps two randomly selected items in the permutation. The second operator flips the orientation of one randomly selected item.

In Lai and Chan [66], the neighborhood is obtained by swaps of randomly selected pairs of items and the solutions are decoded using an algorithm similar to *BLF*. In Leung et al. [69, 70], simulated annealing is used jointly with a constructive and a local search algorithm to escape from local minima. Neighbors are produced again by swapping pairs of items and the constructive algorithms work as decoders. Ceschia and Schaerf [30] also enhance their local search procedure by adding a simulated annealing phase to escape from local optima. More recently, Rakotonirainy and van Vuuren [91] proposed two simulated annealing procedures. In the first, the list of items is decoded by a constructive heuristic and the moves are made on the list. In the second, moves are made directly on the layout of the solution, removing part of the solution and completing it again.

Burke et al. [27] propose a hybrid procedure in which a solution is generated by a two-step process that first applies the best-fill heuristic to place an initial number of items onto the large object. Then, the remaining unassigned items are placed using the bottom-left-fill heuristic with different input orderings as guided by the simulated annealing search procedure. The locations of the items assigned in the initial best-fit stage of the process remain fixed throughout, whereas the items placed during the second stage will move locations according to the orderings produced by simulated annealing.

Tsao et al. [102] also propose a hybrid procedure in which a genetic crossover operator is used in addition to the traditional swap and insert operators. In the initialization phase, a pool of solutions is generated. During the search procedure, neighbors are generated by randomly selecting two solutions from the pool and applying a crossover operator to them. The usual Simulated Annealing acceptance/rejection rule is then applied.

Tabu Search Algorithms

The basic elements of a tabu search algorithm, apart from a feasible starting solution, are a move defining the neighborhood of the current solution, an evaluation of neighbors, a tabu list, an aspiration criterion, and some intensification and diversification strategies. A good example is *TSpack*, developed by Lodi et al. [77] for Bin Packing Problems. Starting from a solution in which each item is packed into a different bin, a move consists of selecting a target bin to be emptied, selecting an item in it, j, and solving the packing problem composed of a subset of k bins plus item j. Improving moves that reduce the number of bins are accepted, as well as moves maintaining the number of bins but packing j out of the target bin. The number k of bins packed again is a parameter that controls the size of the neighborhood and the search effort. Small values of k correspond to intensification while large values of k correspond to diversification. Nevertheless, when the search stalls without finding an acceptable move and k reaches its maximum value, other more aggressive diversification strategies are used to change the way in which the target bin is selected and to destroy part of the solution and build it again in a way in which the new solution is substantially different. Algorithm *TSpack* has been adapted by Bennell et al. [18] for solving a bin packing problem with due dates.

Alvarez-Valdes et al. [6] developed a tabu search algorithm for a two-dimensional cutting problem with guillotine cuts. The initial solution is built by a greedy constructive algorithm and the neighbors of a solution are defined taking at random one rectangle, considering its adjacent rectangles, one at a time, and emptying the minimum region containing both and respecting the guillotine cut structure. The region is filled again several times by using a GRASP algorithm, producing a set of neighboring solutions. The best non-tabu solution among them is selected. The tabu list keeps the rectangle being emptied and its size varies dynamically after a given number of non-improving moves. Intensification and diversification strategies are defined by modifying the value of the items for the greedy constructive procedure. If items appearing more often in the good solutions are given larger values, the search intensifies around best solutions. If these items receive lower values, they are less attractive to be in the solutions and the search diversifies.

Tabu search algorithms for Identical Item Packing Problems and Cutting Stock Problems are also based on the use of blocks [7, 8, 90]. The moves are defined as block insertion, block reduction, or block expansion. IIPP is an example of flat landscape, with many solutions having the same number of items. In order to distinguish among them and lead the search toward promising regions of the solution space, the original objective function is modified by adding other elements, favoring solutions with good characteristics, such as having the empty spaces as near as possible. Long-term memory is used for intensification and diversification purposes, favoring or penalizing blocks that have appeared often throughout the search.

Tabu search algorithms can also be developed using ordered lists of items, as in Wei et al. [107]. The move is defined as the swap of two items, although only a subset of non-tabu moves are explored at each iteration, because they are

evaluated building the corresponding solutions (or decoding them in the terminology of genetic algorithms).

GRASP Algorithms

GRASP algorithms for Cutting and Packing are based on the constructive procedures and the local search moves described above [2,6,9,38]. If at the improvement phase several moves are defined, the GRASP algorithm can be hybridized with a variable neighborhood descent (VND) procedure that controls the moves at each GRASP iteration. An example can be found in the algorithm by Parreño et al. [86] for the Bin Packing Problem, in which several moves based on removing the last part of the solution are combined with a more aggressive move that empties pairs of bins, as it appears in Fig. 20. Once a solution of N bins has been obtained, the target is set to obtain a solution with $N - 1$ bins. When the constructive procedure fills $N-1$ bins, it stops, leaving some items unpacked. Then, the improving moves try to pack all these items into the $N - 1$ bins. Many of the proposed GRASP algorithms use the reactive GRASP strategy, in which the value for the parameter controlling the randomization procedure adjusts itself using the information of the search.

Alvarez-Valdes et al. [3] solve a two-stage cutting stock problem combining two GRASP algorithms, one based on items and the other based on strips. The solutions obtained by the two GRASP algorithms have different structures and their combination by means of a path relinking procedure leads to very efficient solutions.

Path Relinking Algorithms

Path relinking algorithms have been applied to Cutting and Packing problems in connection with tabu search algorithms, as they were initially designed [6], but also

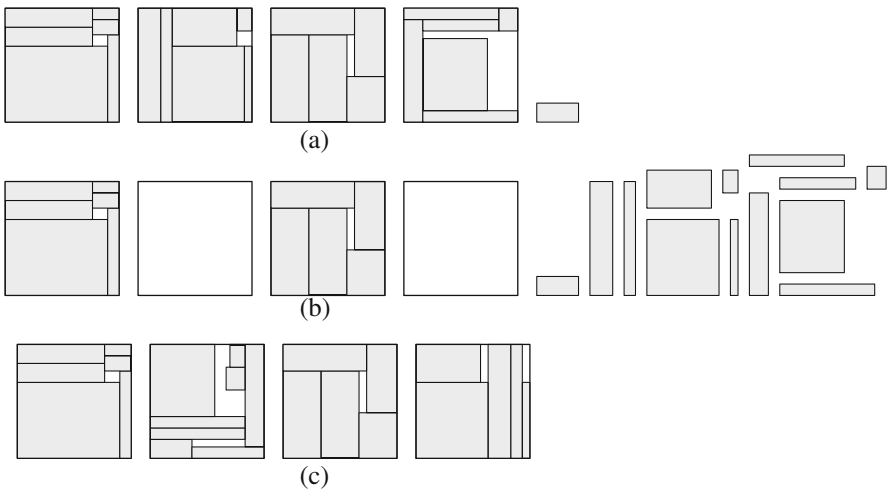


Fig. 20 Improvement move for a Bin Packing Problem. (**a**) Current solution. (**b**) Emptying two bins. (**c**) Pack first the unpacked item and then the removed items

connected to GRASP algorithms, in which a set of elite solutions is kept throughout the search. The work by Alvarez-Valdes et al. [10] on the Multiple Bin-Size Bin Packing Problem combines several variants of path relinking. Figure 21, taken from it, shows the process in which some part of the layout of a guiding solution B is imposed on the initial solution A, producing a new solution that can be considered to lie in the path between them.

Tree Search Algorithms

In tree search algorithms, the solution space is searched using a tree in which each node represents a partial solution, starting from the empty solution at the root node. From each node, branching creates a new level with as many new nodes as possible ways of adding a new element, item, or block, to the partial solution. As this tree would grow exponentially, there are several ways of controlling its growth, limiting the successors of each node to only those most promising, according to a certain criterion, usually the value of the solution obtained by completing the partial solution with a greedy heuristic, as proposed in the Pilot Method by Duin and Voß [42].

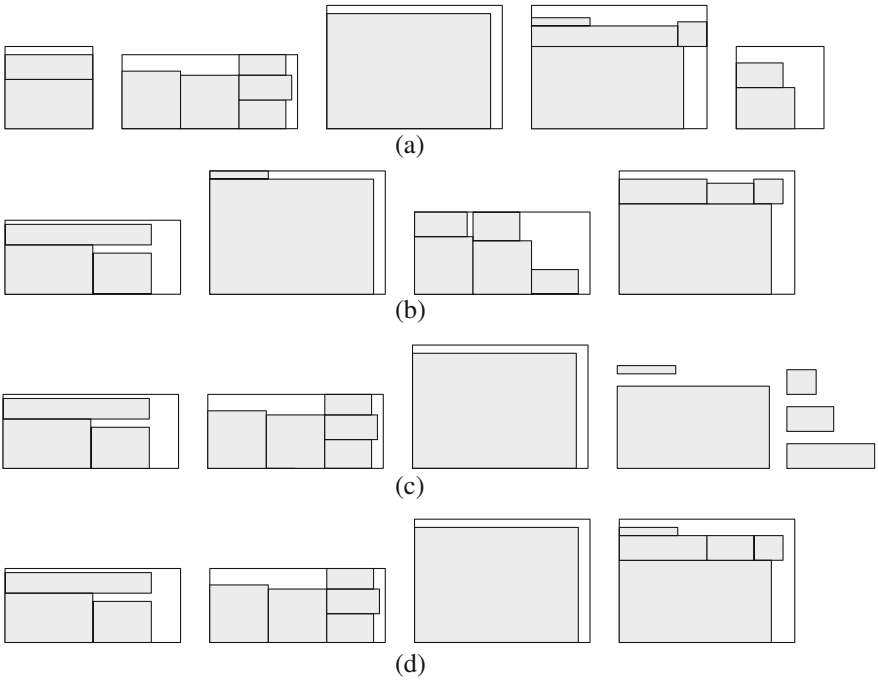


Fig. 21 Path Relinking for a Multiple Bin-Size Bin Packing Problem. (**a**) Initial solution A. (**b**) Guiding solution B. (**c**) Impose on A the first fin of B and remove bins with repeated items. (**d**) New complete solution

When solving a container loading problem, Eley [45] uses a parameter *breadth* to control the number of successors and in his computational study fixes this breadth to 7. Ren et al. [92] follow the same strategy but, at each level, only blocks getting the maximum value in at least one of the five proposed criteria are used to create a new branch. The scheme of Christensen and Rousoe [36] is also similar to Eley's, but they use a dynamic breadth, which adjusts itself dynamically to exhaust the available time while guaranteeing the completion of the tree search.

Pisinger [88] also proposes a tree search for the container loading problem in which the container is first decomposed into a set of layers, whose height and width are the same as the container, and then each layer is decomposed into a set of strips. A partial solution at a node is composed of a set of layers, partially filling the depth of the container. At each branching node, a limited set of layers are considered to be added to the partial solution, according to a ranking rule, and the solution is increased by filling the selected layer with strips. Again, a limited set of strips is considered, and the strips are filled by solving 1D knapsack problems.

Fanslau and Bortfeldt [47] have developed a more complex tree search method. Again, a node of the tree is a partial solution, in this case composed of a set of heterogeneous blocks. A basic search phase transforms a partial solution with i blocks into another with $i + d$ blocks, where d is the depth of the search. The value of an augmented solution is obtained by completing it using a heuristic procedure. Basic searches can be linked to compose more complex search patterns, as shown in Fig. 22, in which, starting from a partial solution with i blocks, a search with depth $d = 3$ is divided into 1 or 2 basic searches. The basic search of depth 3 on the left-hand side produces 8 augmented incomplete solutions (white circles) that are completed to be evaluated (dark circles). The right-hand side shows two linked basic searches. The first one, of depth 2, produces 4 solutions, which are completed. The best complete solution (double dark circle) indicates the incomplete solution from which to start the second basic search of depth 1. The best solution found indicates which block $i + 1$ is added to the partial solution.

Tree search algorithms were used by several teams competing in the ROADEF Challenge 2020, in which a large cutting problem arising in the glass industry had to be solved. Libralesso and Fontan [73] developed a tree search, including simple

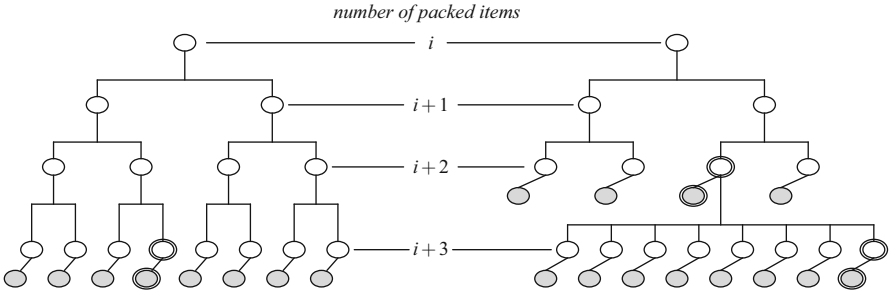


Fig. 22 Tree search structure of Fanslau and Bortfeldt

bounds, dominance, symmetry-breaking rules, and a new evaluation function, and used it in an iterative way in which at each iteration the number of active nodes is increased until optimality is proven or a time limit is reached. Parreño et al. [87] proposed a Beam Search algorithm, combining local and global evaluations of the nodes and a lookahead procedure, and applied a local search to best solution found.

More recently, Arruda et al. [14] have combined and extended the ideas of the BF algorithm and Pilot Method to develop a tree search algorithm in which at each branching level only items fitting into the lowest space of the skyline are considered and evaluated by completing the partial solution with several heuristics.

The OR/AND graph approach developed by Arenales and Morabito [13] for the two-dimensional cutting problem can be also considered as a tree search procedure. At each node of the tree, there is a rectangle obtained by repeatedly cutting the original stock sheet and the cuts that can be applied to this rectangle define the successors of the node.

Also noteworthy are the tree-search approaches of Araya and Riff [12] and Liu et al. [76]. The former avoids exhaustive search of the tree by using a Beam Search algorithm. Several blocks of items with high space utilization are pre-generated and then placed in the empty spaces. Each node of the tree corresponds to a partial solution, defined by the list of maximal empty spaces, the list of items to pack, and a list of blocks. Branching corresponds to packing a block into one of the empty spaces. Liu et al. [76] resort to full tree exploration and, in the initialization phase, build stacks of items instead of general blocks. These stacks are used to build walls along orthogonal slides of the paralellepiped. Branching is done by selecting stacks to build a new wall.

Other Metaheuristic Schemes

Burke et al. [25] propose a Squeaky Wheel Optimization algorithm for the strip packing problem in which at each iteration the ordered list of items is decoded into a feasible solution by means of the Best-Fit algorithm. The items protruding a given height are considered badly placed and their value increases. Therefore, they get better positions in the ordered list and are placed earlier in the next iteration.

Guided Local Search has been used by Faroe et al. [48] for the 3D Bin Packing Problem and by Hifi et al. [59] for the multidimensional knapsack problem. The guided local search moves out of a local maximum/minimum by penalizing particular solution features that it considers should not occur in a near-optimal solution. It defines a modified objective function, augmented with a set of penalty parameters on these features. The usual local search method is then used to improve the augmented objective function. The cycle of local search and penalty parameter update can be repeated as often as required.

Liu et al. [74] use the Particle Swarm Optimization framework to develop an algorithm for the multiobjective bin packing problems. Each particle encodes a packing solution, which includes the number of bins used and the order in which the items are packed into the bin by the *BLF* heuristic. Chen et al. [33] also use particle swarm optimization for solving a two-dimensional cutting problem, using as a decoder a heuristic algorithm, based on the *BL* procedure, but adapted to

the specific problem being solved. A similar idea is used by Koonsintananan and Kimpan [65] for the three-dimensional packing problem.

Zhang et al. [110] take the Variable Neighbourhood Search framework to develop an algorithm for the strip packing problem. Instead of swapping items, as in other proposals, the neighborhoods consist of swapping bloks, sequences of items in the item list. By varying the size of the blocks, a series of increasingly complex neighborhoods are defined.

Capua et al. [29] develop an Iterated Local Search for the bin packing problem with conflicts, focusing on the efficient implementation of exponential-size neighborhoods, Borgulya [23] combines a Estimation of Distribution Algorithm with a genetic algorithm to solve a variable bin-size bin packing algorithm.

Ant Colony Optimization has also been used to solve the tree-dimensional packing problem. In this approach, the solution is coded as the number and type of items to pack, along with the loading sequence (Zhang et al. [111]).

Huang et al. [61] propose a differential evolution algorithm for this problem. Again, the chromosome encodes the item sequence and a bottom-left heuristic decodes the chromosome. A ternary tree is used to store and manage the blanks.

Matheuristics

For one- and multi-dimensional cutting stock problems, Gilmore and Gomory [51–53] proposed in 1961–1965 to solve the following integer program:

$$Min \sum_{q \in Q} c_q x_q$$

$$s.t. \quad \sum_{q \in Q} a_{iq} x_q \geq d_i, \quad i = 1, .., m \tag{1}$$

$$x_q \geq 0 \ \ and \ \ integer, \forall q \in Q \tag{2}$$

where Q is the set of all feasible cutting patterns for all stock sheets S_p, x_q the number of times pattern q is used in the solution, a_{iq} the number of times item i appears in pattern q, d_i the demand of item i and c_q the cost of pattern q.

As the set of patterns Q cannot be completely described except for very small problems, they developed a column generation scheme that can be summarized as follows:

1. Generate an initial set $\widetilde{Q}$ of m cutting patterns, where each pattern contains one type of item.
2. Solve the linear relaxation of the earlier formulated problem considering only the variables corresponding to patterns in $\widetilde{Q}$.
3. For each type of sheet S_p, solve the subproblem:

$$z_p = Max \sum_i \pi_i a_i$$

$$\text{s.t.} \quad \{a_1, \ldots, a_m\} \text{ is a feasible cutting pattern for } S_p$$

where π is the vector of dual prices of the LP solution. If, for some p, $z_p > c_p$, then the column corresponding to that solution is added to $\widetilde{Q}$ and Step 2 is returned to in order to solve the enlarged LP problem. Otherwise, the current solution is rounded to get an integer solution and the process ends.

The question now is how to solve efficiently the subproblem of Step 3. Note that, even if the subproblem is solved by exact methods, the whole procedure is heuristic, due to the rounding step. The quality of the integer solution will depend not only on the algorithm used to solve the subproblem but also on the rounding procedure and on the structure of the demands.

Since the seminal work of Gilmore and Gomory, many authors have used this framework to solve Cutting and Packing problems or at least to have a reference or a starting point for new developments to overcome its limitations and adjust it to other problems. Examples go from Dyckhoff [43], Stadtler [97], and Valerio de Carvalho [104] for 1D problems, to Riehme et al. [93] for 2D cutting problems with extremely varying demands, or to Alvarez-Valdes et al. [5], who develop and compare several procedures to solve the subproblems as well as several rounding procedures.

Matheuristics have been increasingly used in recent years to combine the power of exact methods with the flexibility of heuristics. In this field, there is not a fixed common framework, each author for each problem identifies the part to be solved by a mathematical programming solver and the part to be run by heuristic methods.

Many of the approaches to the cutting problem described earlier would now be called matheuristics. However, matheuristics have also been developed for other types of cutting and packing problems.

Polyakovskiy and M'Hallah [89] develop a two-phase matheuristic for the bin packing problem with due dates. The first phase builds an initial solution using a heuristic that applies an iterative constraint programming-based neighborhood search. The second phase improves the solution by solving approximately a series of assignment low-level mixed integer linear problems.

Birgin et al. [22] address the multi-period two-dimensional non-guillotine cutting stock problem with usable leftovers and propose a forward-looking matheuristic approach. The method solves a sequence of single-period subproblems but the forward-looking approach takes into account the future impact of the decisions of the period, thus allowing the method to suggest the purchase of some extra raw material whose leftovers are expected to be used in future periods, resulting in a lower overall cost.

Grancolas and Pain-Barre [57] propose a metaheuristic approach for the two-dimensional strip packing problem, consisting of two procedures: a local search

algorithm that provides satisfactory placements of the items on the horizontal axis, and an exact procedure that searches the positions of the items on the vertical axis, starting from the most promising ones, and that can be stopped at any time, when the current solution is provably optimal or a given time limit is reached.

Silva et al. [37] addresses the three-dimensional packing problem by decomposing the problem into three phases. In the first phase, walls of items are generated using a constructive heuristic, and in the second phase, a knapsack mathematical programming model is used to select the most promising walls. Finally, in the third phase, a one-dimensional packing problem is solved to place the selected walls in the large parallelepiped.

A decomposition strategy for the same problem is also followed by Toffolo et al. [100]. In this case, in a first phase, stacks of items are built using an exact dynamic programming algorithm, while in a second phase, the two-dimensional problem of packing the stacks of items on the floor of the parallelepiped is solved using a best-fit heuristic.

Irregular Problems

Irregular packing problems are Cutting and Packing problems with convex and non-convex shapes (aka nesting problems). Although different shapes can be considered, from now on, it will be assumed that the small items are described by polygons. In fact, most of the heuristics for these problems assume a polygonal shape for the small items. Moreover, although any type of Cutting and Packing problem, according to Wäscher et al. typology, can deal with irregular shapes, in practice, one can almost only find Placement Problems and Open Dimension Problems. In the former, a large board and a set of irregular items with a maximum demand are given, and the goal is to lay out a subset of items on the board so that the waste is minimized, while in the latter, the large object has fixed width and variable length, big enough to accommodate all the items (in some papers it is described as having infinite length). and the goal is to lay out all the small items so that the length of the board that is effectively used to pack the items is minimized. This is the variant of the problem, usually called irregular strip packing problem. Some papers approach the irregular Bin-Packing Problem [28, 79] and the irregular Cutting-Stock Problem [39, 96], but henceforward, the focus will be on the irregular packing problem with a single rectangular board, either with fixed or variable length, as this corresponds to the majority of the literature in the field.

Constructive Heuristics

Constructive heuristics build a solution for the problem by placing the pieces one by one. Two design decisions have to be made in a constructive heuristic: (1) given a set of pieces to place, in which sequence should the pieces be placed; (2) given a set of

pieces already placed on the board, where to place the next piece. The constructive heuristics vary on the strategies used for these two phases.

The bottom-left placement rule is the basis for most of the constructive heuristics available in the literature. The basic idea of the bottom-left rule is that each piece, in turn, is placed as low in the layout as possible and, for equal height, as much to the left as possible. The name of the rule and the description just made, suppose a vertical strip (width in the horizontal and length in the vertical). However, in irregular packing problems, the board/strip is most commonly represented with the width along the vertical direction and the length (which is aimed to be minimized in irregular strip packing) along the horizontal. Therefore, although the authors in the literature still refer to bottom-left rules, the pieces are actually placed as much to the left as possible (the minimum possible x-coordinate value), and for the same x-position, the lowest y-coordinate value is selected (Fig. 23). The basic implementation of the bottom-left placement rule places the piece on a feasible position of the layout (usually the up-right corner of the board) and starts moving the piece to the left, considering a discrete set of points where pieces can be placed (a grid is defined over the board), and as soon as an overlap with an already placed piece is detected (resorting to one of the geometric representation and manipulation strategies presented in section "Geometry") the movement is backtracked and a vertical move is tried. If successful, the horizontal movement is resumed. This process stops when a piece cannot move horizontally or vertically without overlapping other pieces or crossing the borders of the board (Algorithm 1). In this bottom-left implementation, unused spaces may become holes in the layout, as pieces may block access to those regions to the following pieces.

Art [15] proposed the first heuristic based on a bottom-left rule. On top of the bottom-left rule used to place each piece on the board, a compound rule for sequencing the pieces (selecting the next piece to place) was considered:

Fig. 23 Bottom-left placement rule

Algorithm 1: Bottom-left basic implementation

```
Place the piece on an up-right feasible placement position
while Piece does not overlap other pieces and is completely inside the board do
    Move piece one grid unit left
    if Piece overlaps another piece or is not completely inside the board then
        Move piece one grid unit right
        Move piece one grid unit down
    end if
end while
Move piece one grid unit up
Bottom-left position found
```

- consider the set of pieces that have not yet been placed;
- among these, select the set of pieces that can be placed on an x-coordinate that is within a given tolerance of the minimum x-coordinate where a piece can be placed;
- among these, select the set of pieces with an area within a given tolerance of the maximum area;
- among these, select the set of pieces with a minimum probable waste;
- among these, select the piece that can be placed in the lowest y-coordinate.

This compound rule is rather interesting as it highlights some of the main drawbacks of constructive heuristics.

To avoid the greedy effect of constructive heuristics, the authors try not to have too rigid and fixed rules to select the next piece to place. Whatever the rule we choose, independently of how clever and fit to the problem it is, a greedy choice at the beginning originates, leaving to the end the worst pieces to place, "destroying" solutions that looked promising at the beginning. Art tries to overcome this effect by balancing "doing well now" with "not doing bad in the end." Therefore, he does not commit to the piece that originates the most bottom-left placement but instead considers a set of pieces that are near (doing well now) and then, among these, selects the biggest ones, which are harder to place in the end (not doing badly in the end).

Another drawback of constructive heuristics is, by definition, that they do not backtrack on the decisions made, i.e., once a piece is placed, it is not removed to try another piece. In each step, more than one piece may be tried, and the one considered best is chosen, but in later phases, this step cannot be revisited, and a different decision is made. Backtracking is a characteristic of search heuristics that will be discussed in section "Local Search Heuristics." Art tries to overcome this drawback by attempting to anticipate the final effect of placing a piece through the concept of probable waste, i.e., the space that will become unusable after placing this piece. Figure 24 illustrates the concept. Similar implementations of this bottom-left placement rule were later on used by Oliveira and Ferreira [84] and Jakobs [64] as a block of more complex and sophisticated heuristics.

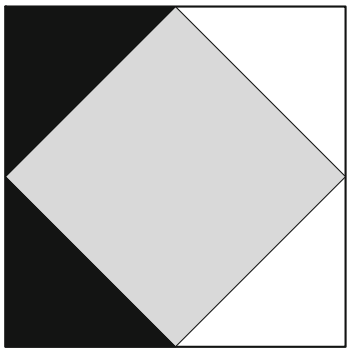


Fig. 24 Probable waste introduced by a piece (dark region on the left), according to Art [15]

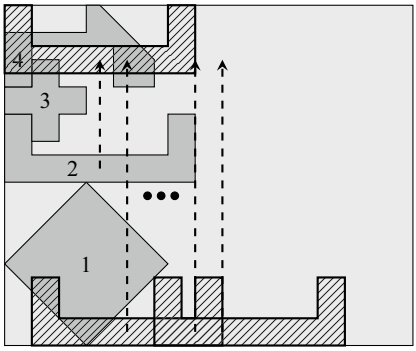


Fig. 25 Bottom-left placement rule, with the piece starting from unfeasible placement positions

Art [15] introduced interesting concepts, but the computational resources available at that time prevented a proper exploration of these ideas (the geometric representation of the pieces had to be very rough), and the bottom-left placement rule was only recovered in the middle of the eighties. Segenreich and Braga [95] propose another bottom-left placement rule. A grid is also defined over the board, and the pieces start from the origin of the coordinates. If the position is unfeasible, the piece is moved one grid unit along the y-axis, and feasibility is rechecked. Once the board's width is reached, the piece is moved one grid unit along the x-axis, the y-coordinate is reset, and the process is repeated until a feasible placement position is found. Given how the piece moves in this search for a feasible position, this is the most bottom-left placement for the piece. To improve the heuristic performance, the initial position for a piece is not always the origin of the coordinates but the position where the last piece of that type was placed, as for sure there is not a most bottom-left position prior to that one (Fig. 25). The most important characteristic of this implementation of the bottom-left placement rule is its hole-filling capacity, i.e., spaces left empty in the middle of the layout can be used to place smaller pieces that are next in the sequence. Dowsland et al. [40] and Takahara et al. [99] also used this bottom-left implementation.

These bottom-left implementations suppose discrete movements of the pieces within the grid. Resorting to the concept of no-fit polygon, Gomes and Oliveira [54]

and Dowsland et al. [41] proposed implementations of the bottom-left placement rules with continuous (real-domain) placement positions. When placing a piece j on the board, Gomes and Oliveira describe the possible placement points by:

- the vertices of the no-fit polygons NFP_{ij} of piece j in relation to pieces $i \in \mathscr{I}$, where $\mathscr{I}$ is the set of pieces that have been already placed;
- the vertices of the rectangle R_j that represents the extreme points where piece j can be placed without crossing the board border;
- the intersections of the edges of all NFP_{ij}, where $i \in \mathscr{I}$;
- the intersections of the R_j edges with the edges of all NFP_{ij}, where $i \in \mathscr{I}$.

Among these, the feasible placement points are on the boundary or outside all the NFP_{ij} and on the boundary or inside R_j, i.e., points, leading to a placement of piece j that does not overlap other already placed pieces and is inside the board (Fig. 26).

Chehrazad et al. [32] introduced a novel placement algorithm to address nesting problems, using a semi-discrete approach to represent both the pieces and the strip. In this method, the pieces and the strip are represented by equidistant vertical line segments. The discretization algorithm uses a sweep-line method and applies minimal extensions to the line segments of a piece to ensure that non-overlapping placement of the segments, representing two pieces, cannot cause overlap of the original pieces. The authors implemented a bottom-left-fill greedy placement procedure, using an optimised ordering of the segments overlap tests. The efficiency of this algorithm stems from its ability to avoid computing intersections between pieces, making it both fast and scalable. Therefore, it can be used as a fundamental component to effectively implement metaheuristics.

A completely different constructive heuristic was proposed in Oliveira et al. [85] and later on improved by Bennell and Song [21] – TOPOS.

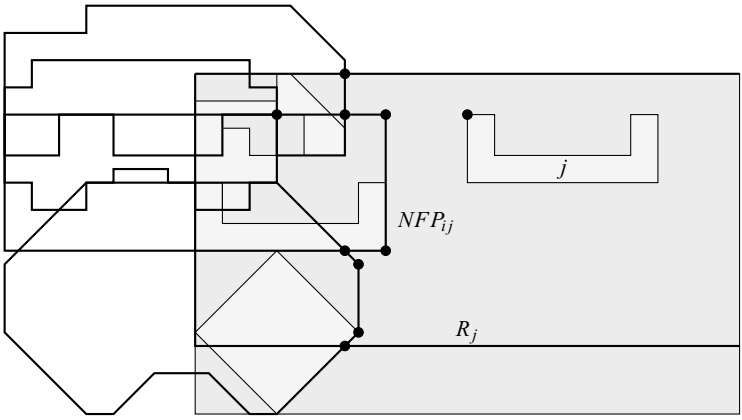


Fig. 26 Bottom-left placement rule, implemented over vertices of the NFP

In TOPOS, as in any other constructive algorithm, pieces are placed one by one on the board, building partial solutions. For each piece, the no-fit polygon of that piece in relation to the external contour of the set of pieces already placed (a partial solution), which form a kind of super-piece, is used to determine feasible placement points on the board. Once the placement point of a piece is determined, the piece is "merged" with the previous partial solution, and the external contour is updated. In the original implementation of Oliveira et al., new pieces could only be placed on the exterior of the partial solution, while in Bennell and Song, pieces may be placed inside the partial solution, i.e., it allows hole-filling. The most distinctive characteristic of TOPOS is that the partial solutions do not have a fixed position on the board. Their placement on the board may change according to the need to have more space below, above, on the left, or on the right of the partial solution to better accommodate new pieces. TOPOS proposed two strategies to answer the two design decisions of a constructive heuristic: where to place a given piece and which should be the next piece to place.

To decide where to place a given piece, three different placement rules are used:

1. minimum area rectangular enclosure – the piece is placed on the point where the area of the rectangle that encloses the piece and the partial solution is minimum;
2. minimum length rectangular enclosure – the piece is placed on the point where the length of the rectangle that encloses the piece and the partial solution is minimum;
3. maximum overlap between the two rectangular enclosures – the piece is placed on the point where the area of the overlap between the piece rectangular enclosure and the partial solution rectangular enclosure is minimum.

The candidates to placement points (the points where the three criteria are evaluated) are the vertices of the no-fit polygon and special points on the edges of the no-fit polygon. On these special points, the criterion that is being used to select the placement point is optimized, when the constraint of belonging to that edge is considered. For example, in Fig. 27, point X corresponds to the point of the edge $\overline{AB}$ where the area of the rectangular enclosure of the two polygons is minimum [85].

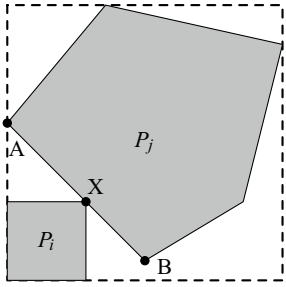


Fig. 27 X is a special point of edge $\overline{AB}$ where the area of the rectangle enclosure of the two polygons is minimum

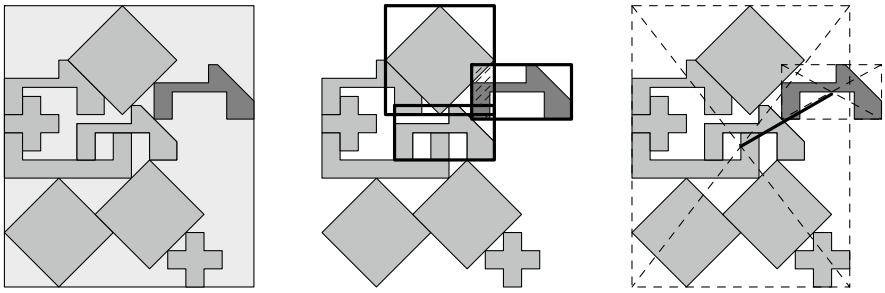


Fig. 28 TOPOS next-piece-to-place selection criteria

Once selected where each potential next-piece-to-place should be effectively placed, the several alternatives are compared using three different criteria, illustrated in Fig. 28:

Waste – The piece adding less waste to the partial solution is chosen.
Overlap – The piece whose rectangular enclosure overlaps more with the rectangular enclosures of the pieces belonging to the current partial solution is chosen.
Distance – The piece that is placed nearest to the geometric center of the partial solution is chosen.

These criteria may be combined or used independently. *Waste* and *Distance* can be used either in absolute value or in relative value, divided respectively by the area of the piece or by the half-perimeter of the rectangular enclosure of the piece. Bennell and Song [21] proposed two additional ways of using these criteria besides the original aggregation by sum: *Vector*, where the number of criteria in which a piece is better than the others is used to decide which is the best piece to place; *Priority*, where the criteria are used in a hierarchical way, with the lower levels being used to solve ties in the upper levels. Finally, in both implementations of TOPOS, all allowed orientations for each piece are tested.

Jostle [40], by Dowsland et al., is also a constructive heuristic where the way the next piece to place is selected is particularly interesting. Starting from a random or any priority-based sequence of the pieces, pieces are placed on the board following a leftmost strategy. The pieces are then sequenced by decreasing order of the x-coordinates of their right-most vertices and placed again on the board, but now according to a right-most strategy. Pieces are reordered by increasing order of the x-coordinates of their left-most vertices, and the process is repeated for a predefined number of iterations or until the solution does not change. This heuristic mimics the process of shaking objects on a tray so that they fit better together.

Local Search Heuristics

Local search heuristics may be divided into two groups: (1) heuristics that search over the actual problem solution space, i.e., complete layouts where all the pieces are laid down on the board; (2) heuristics that search over a solution representation, or codification, which in irregular packing problems is usually the sequence of pieces. The second group requires an additional constructive placement heuristic, or decoder, to transform the sequence into an actual layout (Fig. 29). Neighborhood structures based on swaps (dotted line), inserts (dashed line), or rotations (solid line) are defined over the sequences.

Heuristics Based on Searching Over Sequences

Heuristics based on searching over sequences vary depending on how a sequence is changed into another one (the neighborhood structure) and how the search process is controlled. Gomes and Oliveira [54] proposed a probabilistic 2-exchange heuristic in which neighbor solutions are generated by swapping a pair of pieces in the sequence. Given all pairs of pieces that can be exchanged, three different criteria were explored to decide which should be the next sequence/center of the neighborhood:

- first-better – accepts the first swap that leads to a sequence, which, decoded into a layout, is better than the current solution;
- best-better – accepts the best swap, i.e. the swap, which, decoded into a layout, leads to the best solution among all the swaps;
- random-better – accepts a swap chosen randomly among all the swaps, which, decoded into a layout, lead to better solutions than the current solution.

The search stops when no better solution can be generated from the current sequence.

More sophisticated search control strategies can be developed based on this "search over sequences" paradigm, as will be seen in section "Metaheuristics."

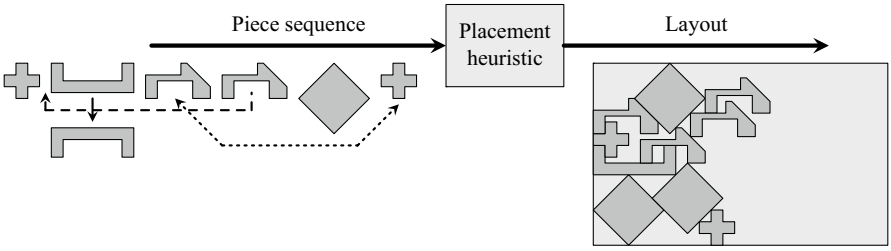


Fig. 29 Search over sequences, with swap (dotted line), insert (dashed line), and rotation (solid line) moves

An alternative search procedure building on piece sequences is based on tree-search methods. Albano and Sapuppo [1] proposed the very first tree-search algorithm for irregular packing problems. A new piece was placed on the board at each level of the tree. The tree's root represented the empty board, and the leaves a full layout. The authors applied strategies to improve the search efficiency by reducing the number of nodes explored. Some examples of these strategies are limiting the number of child nodes generated from a node and limiting the number of nodes kept in the list of nodes waiting to be explored. The decision to keep a node is based on indicators such as the waste of the partial solution or the waste added by placing a piece. These ideas became later rather popular under the name of *beam search* (see section "Metaheuristics").

Heuristics Operating Over Complete Layouts

The other group of local search heuristics operates on actual layouts, directly moving pieces by using the usual neighbor generation operators: swapping two pieces, moving a piece to another place, and rotating a piece (Fig. 30). In this type of heuristics, feasibility is usually not enforced, i.e., solutions where pieces overlap are allowed during the search. This is because only accepting moves to placements without overlap would restrict the search space and eventually lead to premature stopping in poor local optima.

This means that the overlap has to be penalized in the objective function. Once overlap between two pieces is detected, different measures of the overlap can be used in the objective function, being the most common the area, the enclosing rectangle intersection area, the penetration depth, and the orthogonal penetration depth (Fig. 31).

The use of the overlap measure in the objective function will depend on the type of problem tackled:

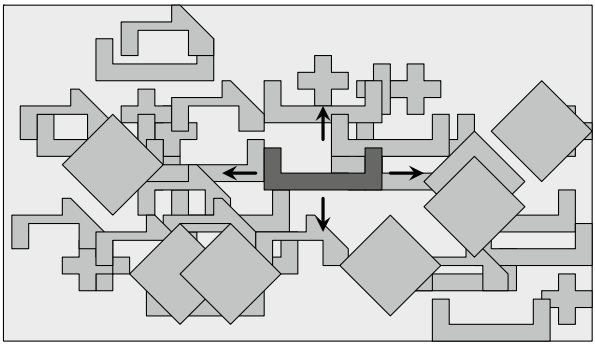


Fig. 30 Search over layouts: illustration of a search move where a piece is selected and moved to a different position

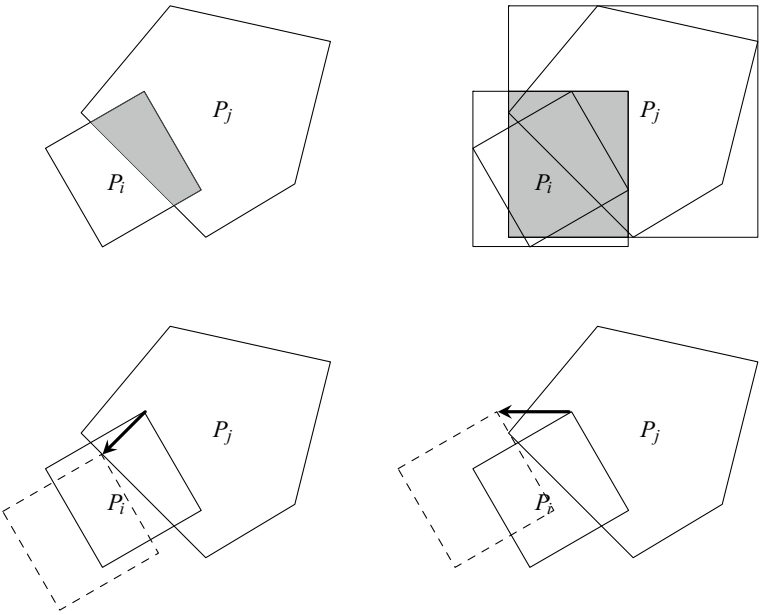


Fig. 31 Different measures of overlap can be used in the objective function: area (up-left), enclosing rectangle intersection area (up-right), penetration depth (down-left), and orthogonal penetration depth (down-right)

- For problems where the width and the length of the board are fixed, the set of pieces to be placed is given, and the objective is to find a placement such that the overlap is minimized. If a solution with no overlap is found, the heuristic is run with an additional piece so that the number of pieces placed on the board is maximum.
- For problems where the board has a fixed width and a variable length, all pieces have to be placed. The objective is to find a placement with no overlap so that the used board's length is minimized. To achieve that goal, the objective function has two components: the length of the layout and the amount of overlap. The weight of the overlap component determines the behaviour of the search algorithm: a high weight leads to a fast convergence to a layout with no overlap but potentially with a big length; a low weight may lead to a solution with a small length but some overlap, an infeasible solution for the irregular packing problem.

Because of their intrinsic difficulty in eliminating overlap, these heuristics have been mainly used embedded in more sophisticated solution approaches, either hybridized with mathematical programming models (section "Matheuristics") or in metaheuristic frameworks (section "Metaheuristics").

Matheuristics

Most of the mathematical programming models for the irregular packing problem rely on the concept of no-fit polygon to write the no-overlap constraints. Consider the polygons P_i and P_j and their $NFP_{P_iP_j}$, represented in Fig. 32. Guaranteeing that P_j does not overlap P_i is simply a matter of imposing conditions on the variables representing the coordinates of the placement point of P_j, i.e., x_j and y_j. These conditions must lead to the consequence that (x_j, y_j) is on the exterior or on the boundary of $NFP_{P_iP_j}$. The mathematical programming models that use the placement points of the pieces as decision variables (e.g. Gomes and Oliveira [55], Fischetti and Luzzi [50] and Alvarez-Valdes et al. [4]) describe the exterior of the no-fit polygon in different ways. The most common approach uses a covering model, i.e., decomposing the exterior into a set of regions that may overlap but completely cover the exterior of the no-fit polygon. In the example depicted in Fig. 32, the equations of the supporting lines of the edges of the no-fit polygon, denoted by $e_{ijk}(x_i, y_i, x_j, y_j)$, $k = 1, \ldots, 12$, would provide the following systems of linear inequalities (in this case, each system has either one or three linear constraints):

$$\begin{bmatrix} e_{ij1}(x_i, y_i, x_j, y_j) \geq 0 \\ e_{ij2}(x_i, y_i, x_j, y_j) \geq 0 \\ e_{ij3}(x_i, y_i, x_j, y_j) \geq 0 \\ e_{ij4}(x_i, y_i, x_j, y_j) \geq 0 \\ e_{ij5}(x_i, y_i, x_j, y_j) \geq 0 \\ e_{ij6}(x_i, y_i, x_j, y_j) \geq 0 \\ e_{ij7}(x_i, y_i, x_j, y_j) \geq 0 \\ e_{ij8}(x_i, y_i, x_j, y_j) \geq 0 \\ e_{ij9}(x_i, y_i, x_j, y_j) \geq 0 \wedge e_{ij10}(x_i, y_i, x_j, y_j) \geq 0 \wedge e_{ij11}(x_i, y_i, x_j, y_j) \geq 0 \\ e_{ij12}(x_i, y_i, x_j, y_j) \geq 0 \end{bmatrix}$$

As only one of the systems has to hold, this leads to using auxiliary binary variables to model the disjunction of constraints. Notice that this set of systems of linear inequalities has to be defined for each pair of polygons, and at least one inequality or system of inequalities has to hold for each pair of polygons. An alternative to this covering model is the partitioning model (e.g., [4]), where the systems of inequalities are defined so that their geometric representations on the plan do not overlap. In this case, exactly one inequality or system of inequalities has to hold for each pair of polygons. However, in any case, a feasible placement for all pieces (a layout) corresponds to fixing specific binary variables. The inequality or system of inequalities that hold for each pair of polygons on this layout will be henceforth designated as the active constraints of the layout and altogether build a linear programming model, as the non-fixed variables (the placement coordinates of all pieces) are real-domain variables.

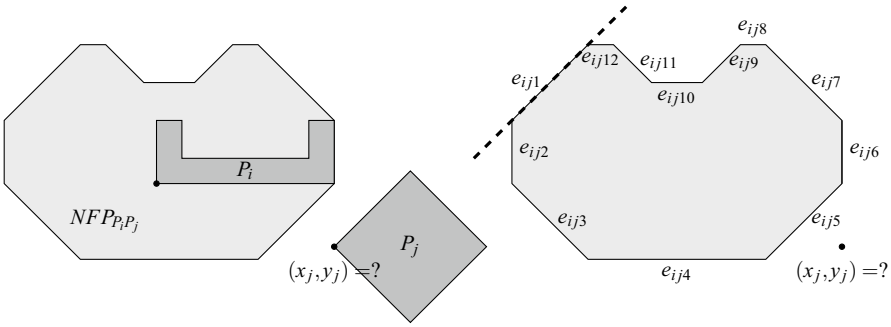


Fig. 32 Geometric information provided by the no-fit polygon to write the no-overlap constraints in a mathematical programming model

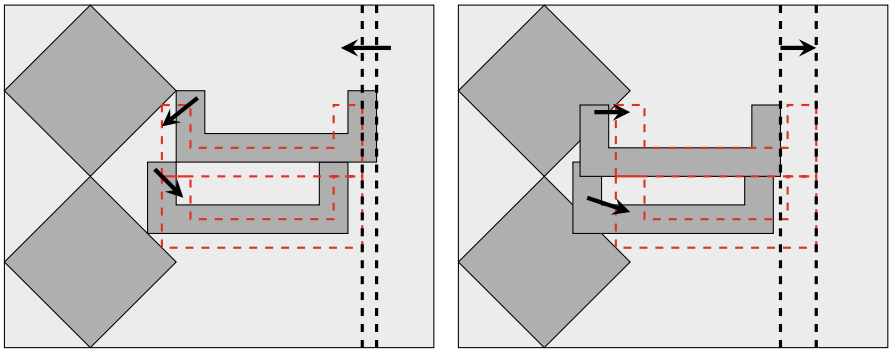


Fig. 33 Applying the compaction (on the left) and separation (on the right) models, with the solid gray pieces representing the initial solution and the dashed outlines representing the optimal solution

The linear programming model describing a feasible layout only defines the relative positions of the pieces, not their absolute positions. Therefore, depending on the objective function, different layouts may be obtained. If the objective function is the minimization of the layout's length (i.e., the right-most vertex of all pieces' vertices), then the *compaction problem* is defined. In this problem, pieces are coordinately and continuously moved so that their relative positions are kept, but the length of the layout is minimized, as represented in Fig. 33 [19,55,80,98]. It is also possible to use a similar model to eliminate overlap in an unfeasible layout (Fig. 33). To achieve this, an active constraint has to be chosen for each pair of pieces. In cases where pairs overlap, an artificial variable is added so that the constraint holds with a non-zero value of the artificial variable, and the objective function will be the minimization of the sum of the artificial variables. If the optimal solution of the linear programming model has all artificial variables equal to zero, then the layout is feasible (without overlap).

Based on these compaction and separation models, hybrid solution methods can be developed [19, 55]. The basis of these methods are local search heuristics that search over the layouts and generate new solutions by swapping two pieces or moving a single piece to a different position on the layout (insertion move). By doing this, most probably the layout will have overlaps. A separation problem will be solved to try to eliminate the overlap and if it obtains a feasible solution, the solution will be improved through a compaction problem. The way the search over the solution space is controlled varies a lot. However, metaheuristic approaches have been used in most cases (see section "Metaheuristics").

Another idea explored by Martinez-Sykora [78] is to design a constructive heuristic based on the mixed-integer mathematical programming model with both the continuous and binary variables. In this approach, a piece is inserted in the layout by adding to the model: its placement variables, the constraints concerning all the no-fit polygons of this piece in relation to all pieces already placed in the layout, and the binary variables related to the edges of these no-fit polygons. All binary variables regarding the pieces already placed are kept at their previous values (i.e., the relative positions of the pieces already placed cannot change), but their continuous placement variables are freed to allow adjustments to make room for the new piece.

Another approach was proposed by Cherri et al. [34]. This heuristic is based on a different model, the *Dotted-Board-Model* [101], a discrete model, where the pieces are constrained to be placed on a discrete set of points of the layout (a grid). The decision variables are the points on the grid and state whether a piece of a given type is placed on that point. To impose the no-overlap constraints, no-fit polygons are used to exclude placement points that would lead to overlapping situations. Although this model solves larger instances than previous models, the *Dotted-Board-Model* is not yet viable for large instances, and a relax-and-fix heuristic with local branching constraints is proposed in [34]. Following the relax-and-fix paradigm, there is a set of variables that are already fixed in each iteration of the method, another set that the model can freely specify, and a third set of relaxed variables. In the following iteration, the variables of the second set become the variables of the first set, a group of variables of the third set is promoted to the second set, and new variables are added to the third set. The method stops when the third set is empty. To solve the irregular packing problem, the board is divided into vertical slices and the decision variables belonging to each slice are progressively relaxed, free, and fixed (Fig. 34). To add some flexibility to the previously placed pieces, some pieces in the fixed set are also freed (local branching constraints).

Metaheuristics

Most of the recent approaches to solving the irregular packing problem solutions are metaheuristics. However, the building blocks of these sometimes rather sophisticated search methods are the constructive, local search and mathematical-

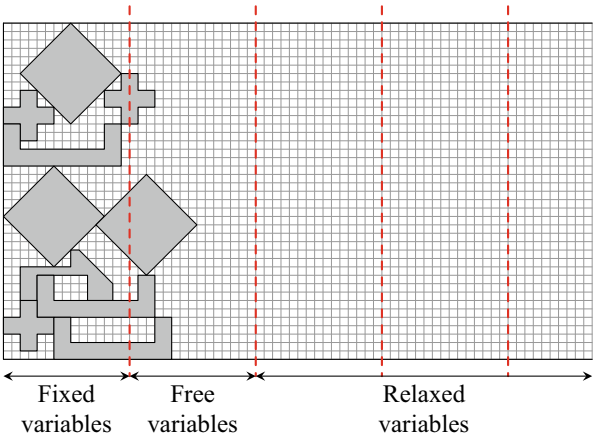


Fig. 34 Relax-and-fix iteration applied to the *Dotted-Board Model*

model-based heuristics presented in the previous sections. After all, metaheuristics are "just" a more sophisticated way of conducting the search over the solution space so that it does not stop at local minima.

Two of the very first metaheuristic approaches for the irregular packing problem were already referred to in the previous section [19, 55]. In both cases, pieces are moved around the layout by swap and insert moves, and overlaps are removed by a separation algorithm and compacted again by a compaction algorithm. In Bennell and Dowsland [19], the search is guided by a tabu search procedure while in Gomes and Oliveira [55] simulated annealing is used.

Egeblad et al. [44] propose a local search scheme in which the neighborhood is any horizontal or vertical translation of a given polygon from its current position. The goal is to minimize the overlap for a given layout length, where the layout length is reduced in each iteration. To escape local minima, the meta-heuristic guided local search is applied as described in Algorithm 2.

A three-level algorithm is proposed by Imamichi et al. [62] for the irregular strip packing problem. As the length of the board is variable, the upper level controls it by increasing the length when it is not possible to place all the pieces without overlapping and decreasing the length when the layout is feasible. The algorithm stops when decreasing the length produces a layout with overlap after a feasible solution (without overlap). The second level minimizes the overlap, given a board of fixed length. An iterated local search metaheuristic is used, where the perturbation phase consists of swapping two randomly chosen polygons, and the local search phase is a separation algorithm that aims to eliminate the existing overlap. The separation algorithm (third level of the approach) is based on solving a mathematical programming model with a nonlinear objective function. The nonlinearity of the objective function results from the use of the penetration depth as a measure of overlap.

Algorithm 2: Guided local search for irregular packing

Given a set of polygons $\mathscr{P}$.
Generate an initial layout p.
for each pair of polygons $s_i, s_j \in \mathscr{P}$ **do**
  Set penalty counter $\Phi_{ij} = 0$.
end for
while $g(p) > 0$ (p contains overlap) **do**
  Local search:
  while p is not a local minimum **do**
    Select polygon s_i.
    Create p' from p by sliding s_i horizontally and vertically
    to the position in which $h(p')$ is minimized, with
    $h(p) = g(p) + \lambda \sum_i \sum_j \Phi_{ij} I_{ij}(p)$, in which $I_{ij}(p) = 0$ if $\text{overlap}_{ij}(p) = 0$, 1 otherwise.
    Set $p = p'$.
  end while
  penalize:
  for each pair of polygons $s_i, s_j \in \mathscr{P}$ **do**
    Compute the utility function $\mu_{ij}(p) = I_{ij}(p) \frac{\text{overlap}_{ij}(p)}{1 + \Phi ij}$.
  end for
  for each pair of polygons $s_i, s_j \in \mathscr{P}$ such that μ_{ij} is maximal **do**
    Set $\Phi ij = \Phi ij + 1$.
  end for
end while
Return p

Umetani et al. [103] propose an approach combining the two previous ones. For the overlap minimization problem, the orthogonal penetration depth is used, while a guided local search is used to generate new solutions during the search for the best layout, given a fixed length for the board. The length starts at a value large enough to admit a feasible solution and is iteratively reduced until the best solution found with the guided local search method has overlap. The algorithm stops and returns the solution obtained with the previous board's length.

The same three-level structure can be found in Leung et al. [71]. The overall approach (Algorithm 3) is rather complex and is based on two neighborhood structures, one based on swapping two polygons and the other based on moving a polygon to a different position in the layout. A nonlinear programming model is used to minimize the overlap during the search process (separation algorithm) and the overall search has two phases: local search and tabu search. At the end, a heuristic is run to improve the final solution. The parameters r_{dec} and r_{inc} control how aggressively the length of the board is decreased and increased during the search procedure.

The same structure is used in Elkeran [46]. The initial solution is generated by the constructive algorithm by Gomes and Oliveira [54]. To improve the algorithm's efficiency, a pairwise clustering of the most non-convex pieces is done, although these clusters do not appear in the final solutions. The board's length is also alternately decreased and increased. The most innovative part of the algorithm is the

Algorithm 3: Extended local search for irregular packing

Best length: L_{best}.
Current length: L_{curr}.
Generate an initial solution $\longrightarrow$ L_{curr}
$L_{best} = L_{curr}$.
while in a time limit **do**
  Decrease the length of the board: $L_{curr} = (1 - r_{dec})L_{best}$.
  Run local search algorithm based on swapping two pieces and minimizing the consequent overlap by a nonlinear separation algorithm $\rightarrow$ L_{curr}.
  if local optimum is feasible (has no overlap) **then**
    $L_{best} = L_{curr}$
  else
    $L_{curr} = (1 + r_{inc})L_{best}$.
    Run tabu search algorithm based on moving a piece to a different position on the layout $\rightarrow$ L_{curr}.
  end if
end while
Run heuristic procedure over the best solution found:
small movements of the pieces + overlap minimization + separation algorithm
Return best solution found

use of the populational metaheuristic Cuckoo Search for the overlap minimization level. The Cuckoo Search algorithm uses a balanced combination of a local random walk and a global explorative random walk, controlled by a switching parameter, to evolve from one population to another. Cuckoo search is responsible for moving a polygon on the board to a less overlapping position, while guided local search is used to escape local minima.

All the metaheuristics presented so far search over the layouts. Other metaheuristic approaches are based on searching over sequences. It is the case of Bennell and Song [21] that propose the use of the Beam Search metaheuristic, which is a tree-search procedure that attempts to avoid the drawbacks of fixed tree-search strategies (e.g., depth-first, breadth-first) without the computational burden of an exhaustive search. In this approach, solutions are represented by a sequence of pieces to be packed. The modified version of TOPOS (see section "Constructive Heuristics") is used to decode these sequences. The tree represents the construction of a partial solution where each node adds a new element to the sequence. At each level, a local evaluation function (in this case, the TOPOS evaluation criteria) evaluates all child nodes. Child nodes compete only with other children branching from the same parent node. This evaluation provides a rough approximation of the solution quality by measuring the incremental cost of adding an element to the partial solution. Using this evaluation, a subset of size α of the best nodes is selected (α is the filter width). The selected filtered nodes are then subject to a global evaluation and the best β nodes are retained for branching (β is the beam width).

Another metaheuristic based on searching over sequences can be found in Sato et al. [94]. It is a bi-level approach, with the top level controlling the board's length (increasing and decreasing it) and the bottom level finding a layout without overlap. The overlap minimization algorithm is simulated annealing. The initial solution is random, and the neighborhood is based on modifications of the placement sequence (two items are swapped), piece orientation (several rotations are allowed), piece position (for non-simple NFPs, in which contour or degenerated vertex or edge the placement occurs). The decoding heuristic is the bottom-left NFP vertex-oriented heuristic of Gomes and Oliveira [54].

Mundim et al. [83] also used a sequence-based search approach managed by a biased random key genetic algorithm. The most notable feature of this work is the consideration of an open-dimensional problem, where both dimensions are variable.

Conclusion

Due to the combinatorial nature of Cutting and Packing problems, exact techniques are not capable of efficiently tackling them for large instances, and therefore, heuristic approaches must be used. In this chapter, the most important heuristic techniques that have been applied to Cutting and Packing problems have been presented, going from simple constructive heuristics to rather complex hybrid metaheuristics approaches, and including local search heuristics and mathematical-model-based approaches. To frame these methods, a comprehensive section on Cutting and Packing problems was written, including the discussion of these problems under Wäscher et al. typology, and the rather important geometric representation layer that these problems require.

Given the broad range of Cutting and Packing problems, it was chosen to focus on two- (both rectangular and irregular) and three-dimensional (just rectangular) problems, as these better illustrate the full use of heuristic methods when approaching this class of problems. One-dimensional problems have no geometric considerations, on the one hand, and, on the other hand, are better solved by exact methods than the other problems. Among the two-dimensional problems, and due to space constraints, circle packing was not referred to in this chapter. However, not only the main approaches closely follow what has been done for the other two-dimensional problems, but also the interested reader can easily refer to the review by Hifi and M'Hallah [58] and following work. Integrating Cutting and Packing problems with other optimization problems that in real life have a direct relation to them, in the sense that the solutions of one problem impact the resolution of the other, is a recent trend in the field. Once more, given the difficulty and complexity of the problems, heuristic techniques have to be used to tackle the integrated problems, such as Vehicle Routing and Container Loading [81].

Cross-References

- Genetic Algorithms
- GRASP
- Guided Local Search
- Iterated Local Search
- Random-Key Genetic Algorithms: Principles and Applications
- Tabu Search
- Variable Neighborhood Descent
- Variable Neighborhood Search

References

1. Albano A, Sapuppo G (1980) Optimal allocation of two-dimensional irregular shapes using heuristic search methods. IEEE Trans Syst Man Cybern 10(5):242–248
2. Alonso M, Alvarez-Valdes R, Parreño F, Tamarit J (2014) A reactive GRASP algorithm for the container loading problem with load-bearing constraints. Eur J Ind Eng 8, 669–694
3. Alvarez-Valdes R, Marti R, Parajon A, Tamarit J (2007) GRASP and path relinking for the two-dimensional two-stage cutting-stock problem. INFORMS J Comput 19:261–272
4. Alvarez-Valdes R, Martinez A, Tamarit J (2013) A branch & bound algorithm for cutting and packing irregularly shaped pieces. Int J Prod Econ 145(2):463–477
5. Alvarez-Valdes R, Parajon A, Tamarit J (2002) A computational study of LP-based heuristic algorithms for the two-dimensional guillotine cutting stock problems. OR Spectrum 24: 179–192
6. Alvarez-Valdes R, Parajon A, Tamarit J (2002) A tabu search algorithm for large-scale guillotine (un)constrained two-dimensional cutting problems. Comput Oper Res 29:925–947
7. Alvarez-Valdes R, Parreño F, Tamarit J (2005) A tabu search algorithm for the pallet loading problem. OR Spectrum 27:43–61
8. Alvarez-Valdes R, Parreño F, Tamarit J (2007) A tabu search algorithm for a two-dimensional non-guillotine cutting problem. Eur J Oper Res 183:1167–1182
9. Alvarez-Valdes R, Parreño F, Tamarit J (2008) Reactive GRASP for the strip packing problem. Comput Oper Res 35:1065–1083
10. Alvarez-Valdes R, Parreño F, Tamarit J (2013) A GRASP/path relinking algorithm for two- and three-dimensional multiple bin-size bin packing problems. Comput Oper Res 40:3081–3090
11. Araujo O, Armentano V (2007) A multi-start random constructive heuristic for the container loading problem. Pesquisa Operacional 27(2):311–331
12. Araya I, Riff M-C (2014) A beam search approach to the container loading problem. Comput Oper Res 43(1):100–107. Cited by: 53
13. Arenales M, Morabito R (1995) An AND/OR-graph approach to the solution of two-dimensional non-guillotine cutting problems. Eur J Oper Res 84:599–617
14. Arruda V, Sirisola L, Soma N (2024) Rectangle packing with a recursive pilot method. Comput Oper Res 161:106447
15. Art RC (1966) An approach to the two dimensional, irregular cutting stock problem. IBM Cambridge Scientific Center Report pp 1–35
16. Baker B, Coffman E, Rivest R (1980) Orthogonal packings in two dimensions. SIAM J Comput 9(4):846–855
17. Below G, Scheithauer G, Mukhacheva E (2008) One-dimensional heuristics adapted for two-dimensional rectangular strip packing. J Oper Res Soc 59:823–832

18. Bennell J, Lee L, Potts C (2013) A genetic algorithm for two-dimensional bin packing with due dates. Int J Prod Econ 145:547–560
19. Bennell JA, Dowsland KA (2001) Hybridising tabu search with optimisation techniques for irregular stock cutting. Manag Sci 47(8):1160–1172
20. Bennell JA, Oliveira JF (2008) The geometry of nesting problems: a tutorial. Eur J Oper Res 184(2):397–415
21. Bennell JA, Song X (2010) A beam search implementation for the irregular shape packing problem. J Heurist 16(2):167–188
22. Birgin E, Romao O, Ronconi D (2023) A forward-looking matheuristic approach for the multi-period two-dimensional non-guillotine cutting stock problem with usable leftovers. Expert Syst Appl 223:11986
23. Borgulya I (2024) A hybrid estimation of distribution algorithm for the offline 2d variable-sized bin packing problem. Central Eur J Oper Res 32:45–65
24. Bortfeldt A, Gehring H (2001) A hybrid algorithm for the container loading problem. Eur J Oper Res 131:143–161
25. Burke E, Hyde M, Kendall G (2011) A squeaky wheel optimisation methodology for two-dimensional strip packing. Comput Oper Res 38:1035–1044
26. Burke E, Kendall G, Whitwell G (2004) A new placement heuristic for the orthogonal stock-cutting problem. Oper Res 52(4):655–671
27. Burke E, Kendall G, Whitwell G (2009) A simulated annealing enhancement of the best-fit heuristic for the orthogonal stock-cutting problem. INFORMS J Comput 21(3):505–516
28. Cai S, Deng J, Lee LH, Chew EP, Li H (2023) Heuristics for the two-dimensional irregular bin packing problem with limited rotations. Comput Oper Res 160:106398
29. Capua R, Frotal Y, Ochil L, Vidal T (2018) A study on exponential-size neighborhoods for the bin packing problem with conflicts. J Heurist 24:667–695
30. Ceschia S, Schaerf A (2013) Local search for a multi-drop multi-container loading problem. J Heurist 19:275–294
31. Chazelle B (1983) The bottom-left bin-packing heuristic: an efficient implementation. IEEE Trans Comput 32(8):697–707
32. Chehrazad S, Roose D, Wauters T (2022) A fast and scalable bottom-left-fill algorithm to solve nesting problems using a semi-discrete representation. Eur J Oper Res 300(3):809–826
33. Chen W, Zhai P, Zhu H, Zhang Y (2014) Hybrid algorithm for the two-dimensional rectangular layer-packing problem. J Oper Res Soc 65:1068–1077
34. Cherri L, Carravilla MA, Toledo F (2016) A model-based heuristic for the irregular strip packing problem. Pesquisa Operacional 36:447–468
35. Christensen H, Khan A, Pokutta S (2017) Approximation and online algorithms for multidimensional bin packing: a survey. Comput Sci Rev 24:63–79
36. Christensen S, Rousoe D (2009) Container loading with multi-drop constraints. Int Trans Oper Res 16:727–743
37. da Silva E, Leão A, Toledo F, Wauters T (2020) A matheuristic framework for the three-dimensional single large object placement problem with practical constraints. Comput Oper Res 124. Cited by: 9; All Open Access, Green Open Access
38. da Silveira J, Miyazawa F, Xavier E (2013) Heuristics for the strip packing problem with unloading constraints . Comput Oper Res 40:991–1003
39. de Souza Queiroz LR, Andretta M (2020) Two effective methods for the irregular knapsack problem. Appl Soft Comput 95:106485
40. Dowsland KA, Dowsland WB, Bennell JA (1998) Jostling for position: local improvement for irregular cutting patterns. J Oper Res Soc 49:647–658
41. Dowsland KA, Vaid S, Dowsland WB (2002) An algorithm for polygon placement using a bottom-left strategy. Eur J Oper Res 141(2):371–381
42. Duin C, VoßS (1999) The pilot method: a strategy for heuristic repetition with application to the steiner problem in graphs. Networks 34:181–191
43. Dyckhoff H (1981) A new linear programming approach to the cutting stock problem. Oper Res 29:1092–1104

44. Egeblad J, Nielsen BK, Odgaard A (2007) Fast neighborhood search for two- and three-dimensional nesting problems. Eur J Oper Res 183(3):1249–1266
45. Eley M (2002) Solving container loading problems by block arrangement. Eur J Oper Res 141:393–409
46. Elkeran A (2013) A new approach for sheet nesting problem using guided cuckoo search and pairwise clustering. Eur J Oper Res 231(3):757–769
47. Fanslau T, Bortfeldt A (2010) A tree search algorithm for solving the container loading problem. INFORMS J Comput 22(2):222–235
48. Faroe O, Pisinger D, Zachariasen M (2003) Guided local search for the three-dimensional bin-packing problem. INFORMS J Comput 15(3):267–283
49. Feng X, Moon I, Shin J (2015) Hybrid genetic algorithms for the three-dimensional multiple container packing problem. Flexible Serv Manuf J 27(2–3):451–477. Cited by: 27
50. Fischetti M, Luzzi I (2009) Mixed-integer programming models for nesting problems. J Heurist 15(3):201–226
51. Gilmore P, Gomory R (1961) A linear programming approach to the cutting-stock problem. Oper Res 9:849–859
52. Gilmore P, Gomory R (1963) A linear programming approach to the cutting-stock problem – part II. Oper Res 11:863–888
53. Gilmore P, Gomory R (1965) Multistage cutting stock problems of two and more dimensions. Oper Res 13:94–120
54. Gomes AM, Oliveira JF (2002) A 2-exchange heuristic for nesting problems. Eur J Oper Res 141(2):359–370
55. Gomes AM, Oliveira JF (2006) Solving Irregular Strip Packing problems by hybridising simulated annealing and linear programming. Eur J Oper Res 171(3):811–829
56. Gonçalves J, Resende M (2013) A biased random key genetic algorithm for 2D and 3D bin packing problems. Int J Prod Econ 145:500–510
57. Grandcolas S, Pain-Barre C (2022) A hybridmetaheuristic for the two-dimensional strip packing problem. Ann Oper Res 309:79–102
58. Hifi M, M'Hallah R (2009) A literature review on circle and sphere packing problems: models and methodologies. Adv Oper Res 2009, Article ID 150624:1–22
59. Hifi M, Michrafy M, Sbihi A (2004) Heuristic algorithms for the multiple-choice multidimensional knapsack problem. J Oper Res Soc 55:1323–1332
60. Hopper E, Turton B (2001) An empirical investigation of meta-heuristic and heuristic algorithms for a 2D packing problem. Eur J Oper Res 128:34–57
61. Huang Y, Lai L, Li W, Wang H (2022) A differential evolution algorithm with ternary search tree for solving the three-dimensional packing problem. Inf Sci 606:440–452. Cited by: 15
62. Imamichi T, Yagiura M, Nagamochi H (2009) An iterated local search algorithm based on nonlinear programming for the irregular strip packing problem. Discret Optim 6(4):345–361
63. Iori M, Martello S, Monaci M (2003) Metaheuristic algorithms for the Strip Packing Problem. In: Pardalos P, Korotkikh V (eds) Optimization in industry: new frontiers. Kluwer Academic Publishers, Dordrecht, The Netherlands, pp 159–179
64. Jakobs S (1996) On genetic algorithms for the packing of polygons. Eur J Oper Res 88: 165–181
65. Koonsintananan S, Kimpan W (2019) Applied particle swarm optimization in solving container loading problem for logistics. In: 11th international conference on knowledge and smart technology, Phuket, Thailand, pp 88–93
66. Lai K, Chan J (1997) Developing a simulated annealing algorithm for the cutting stock problem. Comput Ind Eng 32:115–127
67. Lesh N, Marks J, McMahon A, Mitzenmacher M (2003) New heuristic and interactive approaches to 2D rectangular strip packing. Technical Report TR2003-18, Mitsubishi Electric Research Laboratories, Cambridge
68. Lesh N, Mitzenmacher M (2006) Bubblesearch: a simple heuristic for improving priority-based greedy algorithms. Inf Process Lett 97:161–169

69. Leung S, Zhang D, Sim K (2011) A hybrid simulated annealing metaheuristic algorithm for the two-dimensional knapsack packing problem. Eur J Oper Res 215:57–69
70. Leung S, Zhang D, Zhou C, Wu T (2012) A hybrid simulated annealing metaheuristic algorithm for the two-dimensional knapsack packing problem. Comput Oper Res 39:64–73
71. Leung SC, Lin Y, Zhang D (2012) Extended local search algorithm based on nonlinear programming for two-dimensional irregular strip packing problem. Comput Oper Res 39(3):678–686
72. Leung T, Chan C, Troutt M (2003) Application of a mixed simulated annealing-genetic algorithm heuristic for the two-dimensional orthogonal packing problem. Eur J Oper Res 145:530–542
73. Libralesso L, Fontan F (2021) An anytime tree search algorithm for the 2018 roadef/euro challenge glass cutting problem. Eur J Oper Res 291:883–893
74. Liu D, Tan K, Huang S, Goh C, Ho W (2008) On solving multiobjective bin packing problems using evolutionary particle swarm optimization. Eur J Oper Res 190:357–382
75. Liu D, Teng H (1999) An improved BL-algorithm for genetic algorithm of the orthogonal packing of rectangles. Eur J Oper Res 112:413–420
76. Liu S, Tan W, Xu Z, Liu X (2014) A tree search algorithm for the container loading problem. Comput Ind Eng 75(1):20–30. Cited by: 28
77. Lodi A, Martello S, Vigo D (2004) TSpack: a unified tabu search code for multi-dimensional bin packing problems. Ann Oper Res 131:203–213
78. Martinez Sykora A (2013) Nesting problems: exact and heuristic algorithms. Phd thesis, University of Valencia, Spain
79. Martinez-Sykora A, Alvarez-Valdes R, Bennell J, Tamarit JM (2015) Constructive procedures to solve 2-dimensional bin packing problems with irregular pieces and guillotine cuts. Omega 52:15–32
80. Milenkovic VJ, Daniels K (1999) Translational polygon containment and minimal enclosure using mathematical programming. Int Trans Oper Res 6:525–554
81. Moura A, Oliveira JF (2008) An integrated approach to the vehicle routing and container loading problems. OR Spectrum 31(4):775–800
82. Mukhacheva E, Belov G, Kartak V, Mukhacheva A (2000) Linear one-dimensional cutting-packing problems: numerical experiments with sequential value correction method (SVC) and a modified branch-and-bound method (MBB). Pesquisa Operacional 20:153–168
83. Mundim LR, Andretta M, de Queiroz TA (2017) A biased random key genetic algorithm for open dimension nesting problems using no-fit raster. Expert Syst Appl 81:358–371
84. Oliveira JF, Ferreira JS (1993) Algorithms for nesting problems. Lecture notes in economics and mathematical systems – applied simulated annealing, Heidelberg, Germany, pp 256–279
85. Oliveira JF, Gomes AM, Ferreira JS (2000) TOPOS – a new constructive algorithm for nesting problems. OR Spektrum 22(2):263–284
86. Parreño F, Alvarez-Valdes R, Oliveira J, Tamarit J (2010) Neighbourhood structures for the container loading problem: a VNS implementation. J Heurist 16(1):1–22
87. Parreño F, Alonso M, Alvarez-Valdes R (2020) Solving a large cutting problem in the glass manufacturing industry. Eur J Oper Res 287:378–388
88. Pisinger D (2002) Heuristics for the container loading problem. Eur J Oper Res 141:382–392
89. Polyakovskiy S, M'Hallah R (2018) A hybrid feasibility constraints-guided search to the two-dimensional bin packing problem with due dates. Eur J Oper Res 266:819–839
90. Pureza V, Morabito R (2006) Some experiments with a simple tabu search algorithm for the manufacturers pallet loading problem. Comput Oper Res 33:804–819
91. Rakotonirainy R, van Vuuren J (2020) Improved metaheuristics for the two-dimensional strip packing problem. Appl Soft Comput J 92:106268
92. Ren J, Tian Y, Sawaragi T (2011) A tree search method for the container loading problem with shipment priority. Eur J Oper Res 214:526–535
93. Riehme J, Scheithauer G, Terno J (1996) The solution of two-stage guillotine cutting stock problems having extremely varying order demands. Eur J Oper Res 91:543–552

94. Sato AK, Martins TC, Tsuzuki MSG (2012) An algorithm for the strip packing problem using collision free region and exact fitting placement. Comput-Aided Des 44(8):766–777
95. Segenreich SA, Braga LMPF (1986) Optimal nesting of general plane figures: a Monte Carlo heuristical approach. Comput Graph 10(3):229–237
96. Song X, Bennell JA (2013) Column generation and sequential heuristic procedure for solving an irregular shape cutting stock problem. J Oper Res Soc 65(7):1037–1052
97. Stadtler H (1990) A one-dimensional cutting stock problem in the aluminium industry and its solution. Eur J Oper Res 44:209–223
98. Stoyan YG, Novozhilova MV, Kartashov AV (1996) Mathematical model and method of searching for a local extremum for the non-convex oriented polygons allocation problem. Eur J Oper Res 92(1):193–210
99. Takahara S, Kusumoto Y, Miyamoto S (2003) Solution for textile nesting problems using adaptive meta-heuristics and grouping. Soft Comput 7:154–159
100. Toffolo TA, Esprit E, Wauters T, Vanden Berghe G (2017) A two-dimensional heuristic decomposition approach to a three-dimensional multiple container loading problem. Eur J Oper Res 257(2):526–538. Cited by: 45
101. Toledo FMB, Carravilla MA, Ribeiro C, Oliveira JF, Gomes AM (2013) The dotted-board model: a new MIP model for nesting irregular shapes. Int J Prod Econ 145:478–487
102. Tsao Y-C, Tai J-Y, Vu T-L, Chen T-H (2024) Multiple bin-size bin packing problem considering incompatible product categories. Expert Syst Appl 247. Cited by: 0
103. Umetani S, Yagiura M, Imahori S, Imamichi T, Nonobe K, Ibaraki T (2009) Solving the irregular strip packing problem via guided local search for overlap minimization. Int Trans Oper Res 16(6):661–683
104. Valerio de Carvalho J (2002) LP models for bin packing and cutting stock problems. Eur J Oper Res 141:253–273
105. Wäscher G, Haussner H, Schumann H (2007) An improved typology of cutting and packing problems. Eur J Oper Res 183:1109–1130
106. Wei L, Lim A (2015) A bidirectional building approach for the 2D constrained guillotine knapsack packing problem. Eur J Oper Res 242:63–71
107. Wei L, Oon W, Zhu W, Lim A (2011) A skyline heuristic for the 2D rectangular packing and strip packing problems. Eur J Oper Res 215:337–346
108. Wu Y, Huang W, Lau S, Wong C, Young G (2002) An effective quasi-human based heuristic for solving the rectangle packing problem. Eur J Oper Res 141:341–358
109. Xiang X, Yu C, Xu H, Zhu SX (2018) Optimization of heterogeneous container loading problem with adaptive genetic algorithm. Complexity 2018. Cited by: 14; All Open Access, Gold Open Access
110. Zhang D, Che Y, Ye F, Si Y, Leung S (2016) A hybrid algorithm based on variable neighbourhood for the strip packing problem. J Comb Optim 32:513–530
111. Zhang D, Gu C, Fang H, Ji C, Zhang X (2022) Multi-strategy hybrid heuristic algorithm for single container weakly heterogeneous loading problem. Comput Ind Eng 170. Cited by: 5; All Open Access, Green Open Access
112. Zhu W, Oon W, Lim A, Weng Y (2012) The six elements to block-building approaches for the single container loading problem. Appl Intell 37:431–445
113. Zudio A, da Silva Costa DH, Masquio BP, Coelho IM, Pinto PED (2018) Brkga/vnd hybrid algorithm for the classic three-dimensional bin packing problem. Electron Notes Discret Math 66:17–182. Cited by: 17

Diversity Optimization Models

40

Fernando Sandoya, Anna Martínez-Gavara, Ricardo Aceves, Abraham Duarte, Nilse Romero, Osvaldo Palma, and Rafael Martí

Contents

Introduction 1270
Mathematical Models and Formulation 1271
Diversity Models 1272
Equity Models 1272
Related Models 1273
Correlation Between Models 1274
Diversity, Dispersion, and Equity 1276
Metaheuristics 1280
The Max-Min Model 1280
The Max-Sum Model 1282
The Max-Mean Model 1285
The Max-MinSum Model 1287

F. Sandoya
Escuela Superior Politécnica del Litoral, Guayaquil, Ecuador

Universidad de Guayaquil, Guayaquil, Ecuador
e-mail: fsandoya@espol.edu.ec

A. Martínez-Gavara
Universidad de Valencia, Valencia, Spain
e-mail: gavara@uv.es

R. Aceves · N. Romero · O. Palma
Universidad Nacional Autónoma de México, México City, México
e-mail: aceves@unam.mx; nilse9308@comunidad.unam.mx; matpcoe@outlook.com

A. Duarte
Department of Computer Science and Statistics, Universidad Rey Juan Carlos, Mostoles, Spain
e-mail: abraham.duarte@urjc.es

R. Martí (✉)
Operations Research and Statistics Department, School of Mathematics, University of Valencia, Valencia, Spain
e-mail: rafael.marti@uv.es

© Springer Nature Switzerland AG 2025

R. Martí et al. (eds.), *Handbook of Heuristics*,
https://doi.org/10.1007/978-3-032-00385-0_61

The Min-Diff Model 1288
Conclusions 1290
Cross-References 1291
References 1291

Abstract

The challenge of maximizing the diversity of a collection of points arises in a variety of social studies, which translates into the growing interest in the treatment of diversity in mathematical models. Although the terms diversity and dispersion can be found in many optimization problems to be indistinguishable, we undertake to explore the different models behind them.

This chapter describes the mathematical models for two diversity problems and three equity problems. In addition, it also includes two related models that have recently received special attention. This chapter also reviews heuristics and metaheuristics for finding near-optimal solutions for these problems, such as GRASP, Tabu Search, Genetic algorithms, Particle Swarm, and Ant Colony Optimization. We include pseudocodes of some of these methods.

Keywords

Maximum diversity problem · Optimizing diversity · Binary optimization

Introduction

Maximizing diversity involves the selection of a subset of elements from a given set in a manner that maximizes the distance or similarity between them [19]. Numerous models have been proposed for this purpose, all of which rely on a diversity measure, typically expressed as a distance function within the space where the objects reside. The definition of this distance between elements is customized for specific applications and may vary in how the overall diversity of the selected elements is calculated. These models represent a significant class of $\mathcal{NP}$-hard problems in combinatorial optimization, with practical applications across a wide range of domains, as outlined in [21]. They include plant breeding, social issues, ecological preservation, pollution control, product design, capital investment, workforce management, curriculum design, and genetic engineering.

As Scott Page asserts in his book [41]: *"Diverse perspectives and tools enable collections of people to find more and better solutions and contribute to overall productivity."* Consequently, the identification of diverse groups of people becomes a focal point in large companies and institutions. The concept of diversity, also known as dispersion, is widely used; however, its definition is often ambiguous, leading to various interpretations. As highlighted by [44], diversity in the context of combinatorial optimization can refer to dispersion among selected points or to representativeness, where the selected points act as class representatives of subsets within the given set.

Arguably, the model most extensively studied concerning diversity is the Maximum Diversity Problem (MDP), also known as the Max-Sum Diversity Model [18], where the objective is to maximize the sum of distances between selected elements. The Max-Min Diversity Problem (MMDP), alternatively, aims to maximize the minimum distance among the selected elements and has also received considerable attention in recent research [50]. Despite their related nature, it is important to note that a method developed for the MDP may not necessarily perform effectively for the MMDP and vice versa. We illustrate this distinction with an example in section "Correlation Between Models."

Equity models, closely related to diversity, integrate the notion of fairness among candidates. These models have been largely ignored and have only recently been considered in the scientific literature. Particular emphasis is needed for the research in [47], where four distinct equity models are introduced. These models emerge within the realms of location of urban public facilities, selection of diverse/similar groups, and identification of sub-graphs, providing avenues to address fair diversification or assimilation among network members. In this chapter, we address both diversity and equity models.

Recently, a review on the state of the art in discrete diversity and dispersion [33] describes the evolution of models and methods from an Operations Research perspective. The authors traced the development of this field, beginning in the late 1980s, and identified three distinct time periods according to the proposal, development, and application of models and solving methods. The field of diversity is today a research area in itself with significant scientific contributions, as shown in a monographic book on it [34]. The book is organized based on the techniques applied, ranging from simple constructive mechanisms to complex evolutionary algorithms, including examples of how to customize these methods to solve specific combinatorial optimization problems. Finally, the MDPLIB 2.0 library has to be mentioned as a reference in the field, which contains data sets widely used in this class of problems. The experiments in this chapter are based on this library.

This chapter is organized as follows. In section "Mathematical Models and Formulation," we define the different diversity models and the correlation between their solutions. In section "Diversity, Dispersion, and Equity," we review the different diversity measures and their mathematical expression and graphically explore their meaning. Finally, in section "Metaheuristics," we describe the most recent papers on heuristic and metaheuristic algorithms to solve the models introduced previously and their pseudocodes.

Mathematical Models and Formulation

Given a graph $G = (V, E)$, where V is the set of n nodes and E is the set of edges, let d_{ij} be the inter-element distance between any two elements i and j, let $M \subseteq V$ be the set of m selected elements, and define $U = V \setminus M$ as the set of unselected elements.

Diversity Models

The *Max-Min Diversity Problem* (MMDP) can be formulated as follows:

$$\begin{aligned} \text{(MMDP) Maximize } z_{MM} = & \min_{\substack{1\le i<j\le n \\ x_i=x_j=1}} d_{ij}x_ix_j \\ \text{st} \quad & \sum_{i=1}^{n} x_i = m \\ & x_i \in \{0,1\} \qquad i = 1,\ldots,n. \end{aligned} \tag{1}$$

The Maximum Diversity problem, MDP, also called Max-Sum can be formulated in a similar way by simply replacing the objective function, $z_{MM}(x)$, in the formulation above with the expression $z_{MS}(x)$ as

$$z_{MS}(x) = \sum_{1\le i<j\le n} d_{ij}x_ix_j. \tag{2}$$

The Max-Sum and Max-Min literature includes extensive surveys [1, 13, 28], exact methods [1, 18, 45], and heuristics [18, 25, 27, 48, 50].

Equity Models

In [47], the authors summarize some of the different dispersion models, defining them as follows. The *Max Mean Dispersion Problem* (Max-Mean) can be formulated as the following 0–1 integer linear programming problem:

$$\begin{aligned} \text{Maximize} \quad & \frac{\sum_{i=1}^{n-1}\sum_{j=i+1}^{n} d_{ij}x_ix_j}{\sum_{i=1}^{n} x_i} \\ \text{st} \quad & \sum_{i=1}^{n} x_i \ge 2 \\ & x_i \in \{0,1\} \qquad i = 1,\ldots,n. \end{aligned} \tag{3}$$

In (3) the cardinality restriction is not imposed. This problem is a version of the Max-Sum problem where the number of elements to be selected is unknown. The generalized version of this problem is called *Generalized Max-Mean Dispersion Problem* and is formulated as follows:

$$\begin{array}{lll} \text{Maximize} & \dfrac{\sum_{i=1}^{n-1}\sum_{j=i+1}^{n} d_{ij}x_i x_j}{\sum_{i=1}^{n} w_i x_i} & \\ \text{st} & \sum_{i=1}^{n} x_i \geq 2 & \\ & x_i \in \{0,1\} & i = 1,\ldots,n, \end{array} \tag{4}$$

where w_i is the weight assigned to element $i \in V$.

Then, the *Maximum Minsum Dispersion Problem* (Max-Minsum DP) consists of selecting a set $M \subseteq V$ of m elements such that the smallest total dispersion associated with each selected element i is maximized. The problem is formulated in [47] as follows:

$$\begin{array}{lll} \text{Maximize} & \left\{ \min_{i:x_i=1} \sum_{j:j\neq i} d_{ij}x_j \right\} & \\ \text{st} & \sum_{i=1}^{n} x_i = m & \\ & x_i \in \{0,1\} & i = 1,\ldots,n. \end{array} \tag{5}$$

Finally, the *Minimum Differential Dispersion Problem* (Min-Diff) consists of finding the best subset $M \subseteq V$ with respect to the measure "Diff" defined in Table 4. This problem is a 0-1 integer linear programming problem and can be described as

$$\begin{array}{lll} \text{Minimize} & \left\{ \max_{i:x_i=1} \sum_{j:j\neq i} d_{ij}x_j - \min_{i:x_i=1} \sum_{j:j\neq i} d_{ij}x_j \right\} & \\ \text{st} & \sum_{i=1}^{n} x_i = m & \\ & x_i \in \{0,1\} & i = 1,\ldots,n. \end{array} \tag{6}$$

Related Models

Given a graph $G = (V, E)$ where V is a set of n nodes and E is a set of edges, let $w_i \geq 0$ be the weight of node $i \in V$ and let c_{ij} be the benefit of edge $(i, j) \in E$. The *Capacitated Clustering Problem* (CCP) involves partitioning V into p clusters in such a way that the sum of the weights of the elements in each cluster is within some integer capacity limits, L and U, and the sum of the benefits between the pairs of elements in the same cluster is maximized. The CCP can be formulated as a quadratic integer program with binary variables x_{ik} that take the value of 1 if

element i is in cluster k and 0 otherwise.

$$\begin{array}{lll} \text{Maximize} & \sum_{k=1}^{p}\sum_{i=1}^{n-1}\sum_{j>i}^{n} c_{ij}x_{ik}x_{jk} & \\ \text{st} & \sum_{k=1}^{p} x_{ik} = 1 & i = 1,\ldots,n \\ & L \leq \sum_{i=1}^{n} w_i\, x_{ik} \leq U & k = 1,\ldots,p \\ & x_{ik} \in \{0,1\} & i = 1,\ldots,n; k = 1,\ldots,p. \end{array} \tag{7}$$

The objective function adds the total benefit of all pairs of elements that belong to the same cluster. The first set of constraints forces the assignment of each element to a cluster. The second set of constraints forces the sum of the weights of the pairs of elements in the same cluster to be between L and U. The maximally diverse grouping problem (MDGP) consists of grouping a set of elements into p mutually disjoint groups in such a way that the diversity among the elements in each group is maximized. The diversity among the elements in a group is calculated as the sum of the individual distance between each pair of elements. The objective of the problem is to maximize the overall diversity, i.e., the sum of the diversity of all groups, when the size of each group is within a specified range. Clearly, MDGP is a special case of CCP for which $w_i = 1$ for all nodes i, and the distance between each pair of nodes (i, j) is the benefit c_{ij}. Therefore, from the CCP formulation above, the MDGP can be formulated as

$$\begin{array}{lll} \text{Maximize} & \sum_{k=1}^{p}\sum_{i=1}^{n-1}\sum_{j>i}^{n} c_{ij}x_{ik}x_{jk} & \\ \text{st} & \sum_{k=1}^{p} x_{ik} = 1 & i = 1,\ldots,n \\ & L \leq \sum_{i=1}^{n} x_{ik} \leq U & k = 1,\ldots,p \\ & x_{ik} \in \{0,1\} & i = 1,\ldots,n; k = 1,\ldots,p. \end{array} \tag{8}$$

The MDGP is called the k-partition problem in [14] and the equitable partition problem in [40].

Correlation Between Models

In [50], the authors illustrated by the example in Table 1 that the correlation between the values of the solutions in the Max-Sum and Max-Min problems can be relatively low.

Let us consider that we have seven elements, of which we must select five. Furthermore, the distances between each pair of elements are given by the matrix of Table 1. For such a small example, we can enumerate all possible solutions (selections of elements) and compute, for each one, the value of the Max-Sum DP

Table 1 Distance matrix of an instance with seven elements

	1	2	3	4	5	6	7
1	–	4.6	6.2	2.1	3.5	3.6	4.4
2	4.6	–	6.6	7.1	8.2	2.4	5.3
3	6.2	6.6	–	2.1	3.5	3.6	4.4
4	2.1	7.1	2.1	–	5.5	1.1	2.3
5	3.5	8.2	3.5	5.5	–	6.4	3.4
6	3.6	2.4	3.6	1.1	6.4	–	5.4
7	4.4	5.3	4.4	2.3	3.4	5.4	–

Table 2 Correlation coefficients among the mean results in the 30 instances, with $d_{ij} \in U[0, 20]$, $n = 20, m = 5$

Mean value				
	Max-Sum	Max-Min	Max-MinSum	Min-Diff
Max-Sum	1	0.60[a]	0.96[a]	−0.17
Max-Min	0.60[a]	1	0.73[a]	−0.63[a]
Max-MinSum	0.96[a]	0.73	1	−0.44
Min-Diff	−0.17	−0.63[a]	−0.44	1

[a]Correlation is significative at 0.01 level

Table 3 Correlation coefficients among the best results in the 30 instances, with $d_{ij} \in U[0, 20]$, $n = 20, m = 5$

	Best value			
	Max-Sum	Max-Min	Max-MinSum	Min-Diff
Max-Sum	1	0.78[a]	0.90[a]	0.07
Max-Min	0.78[a]	1	0.77[a]	0.03
Max-MinSum	0.90[a]	0.77[a]	1	−0.07
Min-Diff	0.07	0.03	−0.07	1

[a]Correlation is significative at 0.01 level

and the value of the Max-Min DP. The correlation between both objective functions is 0.61, which can be considered relatively low. Therefore, we should not expect a method for one of these problems to obtain good solutions for the other.

We now extend that analysis by including the Max-MinSum, Min-Diff, Max-Sum, and Max-Min models. We do not include the Max-Mean models because they do not set the number of elements to be selected as the others. We generate 30 instances with 20 elements from which five have to be selected. Distances are generated randomly according to a uniform distribution in the range $[0, 20]$. For each instance, we enumerate all its solutions (selections of five elements) and compute the four objective values (Max-Sum, Max-Min, Max-MinSum, and Min-Diff). Then, we compute two values per instance and objective, the mean and best values across all the solutions. Tables 2 and 3 show the correlations obtained from these two values, respectively.

There is no significant relation between the Min-Diff model solutions and the rest of the models. Despite obtaining a correlation coefficient of -0.63 in the

mean values between Min-Diff and Max-Min, this correlation is very low in the best case. So, this problem can be considered to be different enough from the rest of the models. The largest correlation coefficient is obtained between Max-Sum and Max-MinSum. The correlation coefficient of 0.78 between the Max-Sum and Max-Min best value solutions is also notable. Although these correlation values seem large enough, we have empirically found that a method developed for one model problem does not necessarily perform well on another or vice versa.

Diversity, Dispersion, and Equity

The terms "diversity," "dispersion," and "equity" can be found in many optimization problems in the context of selection. Although the first two are indistinguishable, the term equity seems to have a subtle difference with them. However, in most papers, these three terms are used with a certain ambiguity.

The problem is that the term diversity is not properly defined and therefore results in different interpretations. In any case, we can say that it is associated with a global characteristic of the selected subset and should be evaluated according to the distances d_{ij} considered.

The growing interest in dealing with diversity resulted in an effort to study the management of equity in the last years, i.e., organizations are becoming increasingly interested in ensuring the equitable treatment of individuals or institutions. Generally speaking, equity is synonymous with fairness, objectivity, or impartiality. Many authors, such as French in [16], argue that equity is relative to justice, such as the distribution of resources or public service infrastructure.

In graphical terms, when we select a diverse subset from a given set, we expect it to be scattered. In other words, we do not expect it to be concentrated in a small region. In this section, we graphically show the solutions obtained when solving different models (Max-Sum, Max-Min, Max-Minsum, and Min-Diff), in an attempt to disclose their characteristics. Given a set $V = \{1, 2, \cdots, n\}$, an inter-element relationship d_{ij} defined between each pair of elements, and a subset $M \subseteq V$, we denote the diversity of M as $div(M)$. In the example shown in Fig. 1, inter-element distances are calculated using the Euclidean metric, and the results were obtained with mathematical programming models formulated in section "Mathematical Models and Formulation" and obtained the exact solution with CPLEX.

If we are looking for solutions scattered in the set of elements, we might say that objects selected in Fig. 1b, obtained with the Max-Min model, are better distributed than those selected in the other models in Fig. 1 (see section "Mathematical Models and Formulation" for more details in these models). In this figure the elements with red square icons are the diverse set selected, and the other ones are the unselected elements. In the Max-Sum and Max-MinSum models (Fig. 1a and c respectively), the selected elements are located in the outer part of the graphic, which seems to indicate that they are dispersed.

Notice that in this example, we are using the Euclidean metric, and therefore our interpretation is biased by our geometric conception. In particular, this interpretation

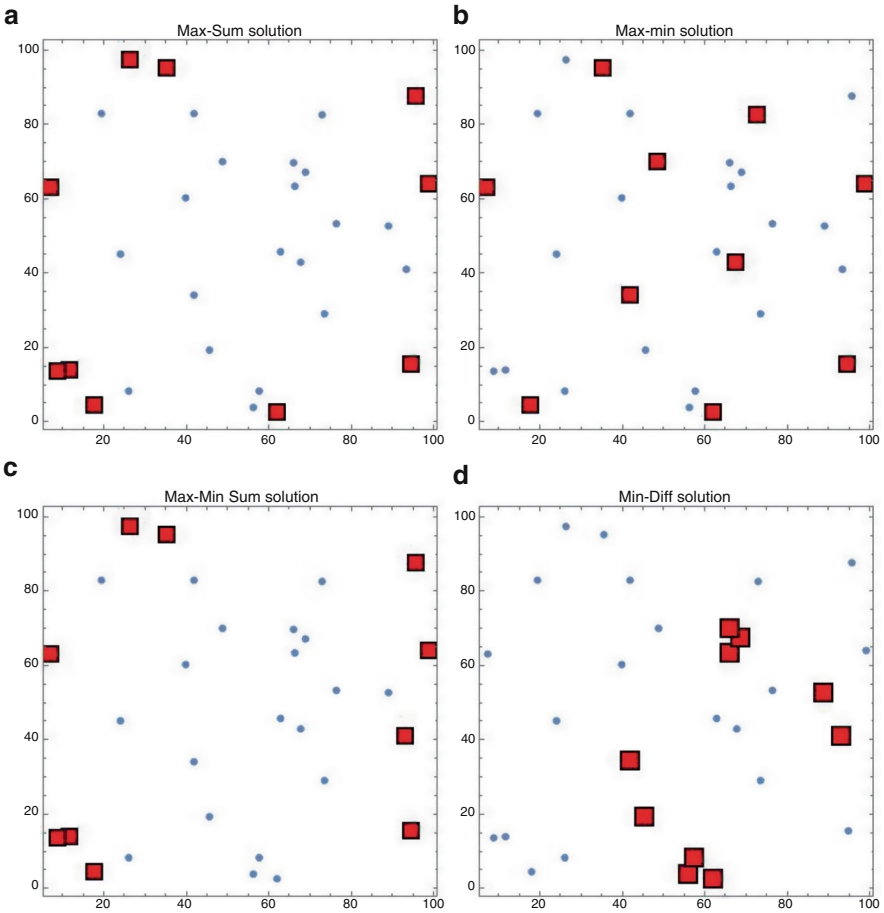


Fig. 1 Comparison of different models of a set of 30 elements with 10 selected elements

is no longer valid for a general measure of affinity or similarity. Then a more general interpretation of diversity implies abandoning this intuitive and geometric conception and establishing diversity from another point of view. Thus, the problem of measuring diversity in a given set is not unique and gives rise to different problems depending on the metric used. We have identified five different measures to compute diversity, equity, or dispersion from a given distance value calculated in M. Table 4 summarizes them.

Consider two examples in the rectangle $[0, 10]\times[0, 10]$ to illustrate the difference among the different dispersion measures based on the Euclidean distance. The position of the elements is obtained at random in both examples. In the first example, Example 1, we consider the elements $n = 12$, whose position is represented in Fig. 2.

Table 4 Description of the different diversity measures

Measure	Mathematical function	Description
Sum	$\sum_{i<j,i,j\in M} d_{ij}$	This measure may address diversification among selected elements to distance.
Min	$\min_{i<j,i,j\in M} d_{ij}$	Focus on the minimum distance between the selected elements.
Mean	$\frac{\sum_{i<j,i,j\in M} d_{ij}}{\lvert M\rvert}$	Related to the Sum Measure, is an average equity measure.
Minsum	$\min_{i\in M} \sum_{j\in n, j\neq i} d_{ij}$	This measure considers the minimum aggregate dispersion among elements.
Diff	$\max_{i\in M} \sum_{j\in M, j\neq i} d_{ij} - \min_{i\in M} \sum_{j\in M, j\neq i} d_{ij}$	This measure can be understood as the difference between the largest and smallest values of the dispersion sum.

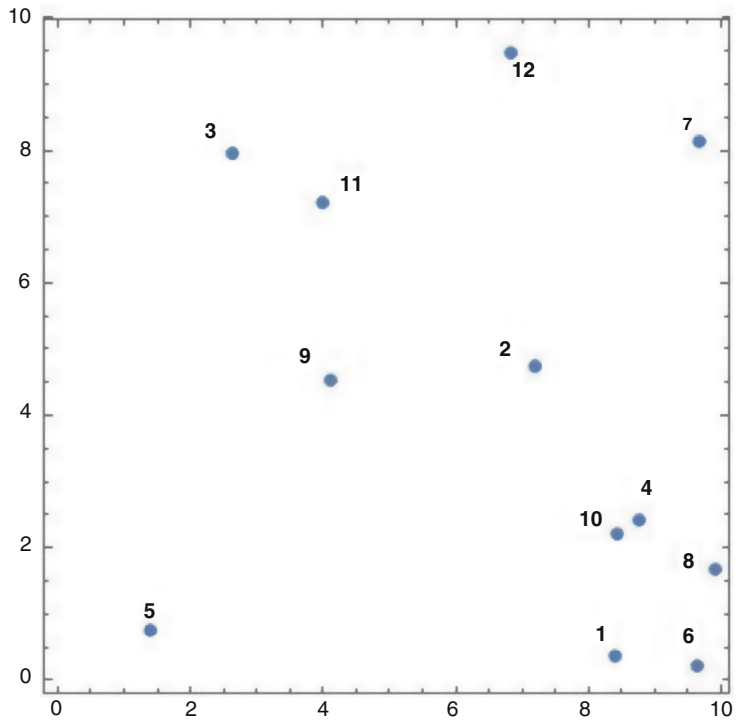


Fig. 2 Position of the elements in Example 1

The aim of this example is to observe the value of the different dispersion measures. Let us consider two different subsets of selected elements. The solution A contains the elements $\{2, 3, 7, 9\}$, and the solution B contains the elements $\{1, 3, 8, 10\}$. In Fig. 3, we show in large squares in red the selected elements of these two solutions.

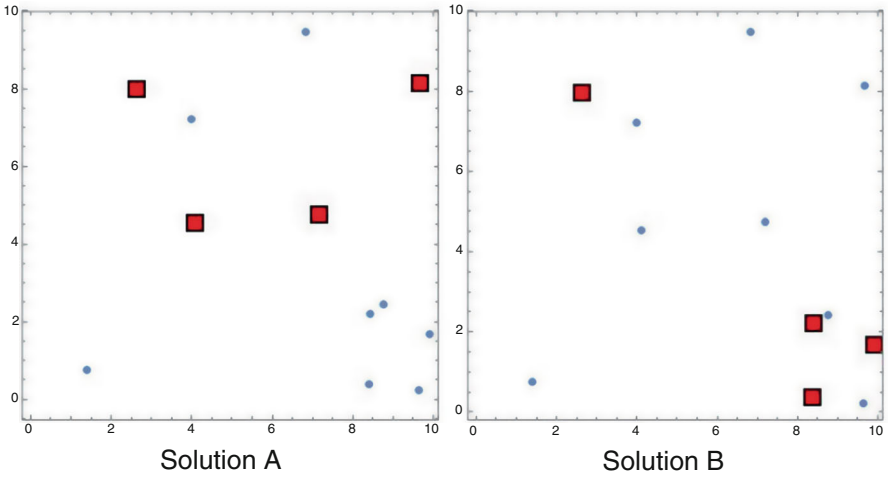


Fig. 3 The two subsets of selected elements in Example 1

Table 5 Diversity measures for the two subsets of selected elements in Example 1

Model	Max-Sum	Max-Min	Max-Mean	Max-MinSum	Min-Diff
Solution A	30.26	3.09	7.56	12.87	4.99
Solution B	32.10	1.70	8.10	12.30	15.30

The numerical value of the different diversity measures is shown in Table 5. We can see that the Max-Min, Max-MinSum, and Min-Diff models give better results in solution A than in B. However, the solution B is better than A in the other models. According to this, one could think that the five models considered can be classified into two categories. However, the next example illustrates that it is not always in this way.

Let us now consider a second example, Example 2, with 500 elements, where we have to select 80 elements. As above, we consider two solutions with different subsets of elements (see Fig. 4) and show the numerical measures of each model in Table 6.

In this example, A exhibits better diversity measures than B in all cases except the Max-Min model. These results seem to confirm that the diversity depends on the model we are applying. Therefore, as is customary when solving an optimization problem, we first have to model it, which in our case means to select the diversity model that better reflects our goal.

Finally, in terms of developing a solution method, the complexity of the objective function evaluation is $\mathcal{O}(m^2)$ for most diversity models, where m is the number of selected elements. This includes the Max-Sum, Max-Min, and Max-Mean models. However, in the Max-MinSum and Min-Diff models, the complexity is $\mathcal{O}(m^3)$ in a direct evaluation of the objective function.

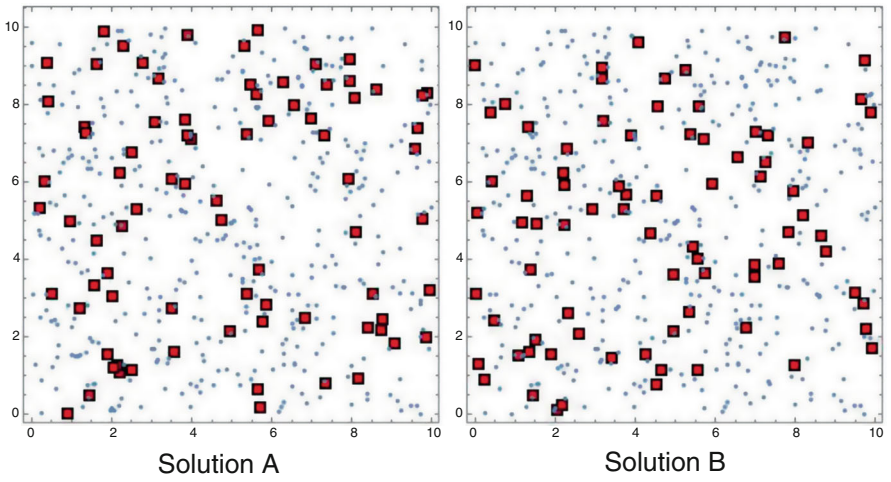


Fig. 4 The two subsets of selected elements in Example 2

Table 6 Diversity measures for the two subsets of selected elements in Example 2

Model	Max-Sum	Max-Min	Max-Mean	Max-MinSum	Min-Diff
Solution *A*	17021.0	0.123	212.76	315.73	264.43
Solution *B*	15995.9	0.169	199.95	293.51	287.97

Metaheuristics

The literature presents a wide range of heuristic and metaheuristic algorithms to address diversity problems. However, research in this field is fragmented due to several factors, such as the lack of standardized nomenclature and the absence of a unifying definition of diversity. Despite these challenges, some efforts have been made to review existing methodologies. This section focuses on the main heuristic and metaheuristic algorithms proposed for specific models such as Max-Sum, Max-Min, Max-Mean, Max-MinSum, and Min-Diff. A notable example is reference [2], which presents a detailed comparison of 10 heuristics and 20 metaheuristics designed specifically for the Max-Sum model.

The Max-Min Model

In [50], two constructive methods based on the Greedy Randomized Adaptive Search Procedure (GRASP) and various approaches to hybridize them with the path relinking methodology are proposed. The first GRASP construction method involves evaluating each candidate element using a greedy function to identify

the best elements, which are then added to the Restricted Candidate List (RCL). Subsequently, the next element to be incorporated into the partial solution is randomly selected from RCL. In contrast, the second GRASP construction method is based on an alternative scheme introduced in [49], where RCL is entirely constructed at random from elements in the Candidate List (CL). Consequently, in this later case, RCL can be viewed as a random sample of CL of a predetermined size. Afterward, the greedy function is applied to assess all elements in RCL to choose the best one. In this alternative design, the sequence of randomness followed by greediness is inverted. The local search mechanism implements the mildest ascent technique along with the path relinking methodology. This second strategy proves to be more efficient than the classic one within this model. Similar behavior of the GRASP scheme is observed in other models of maximum diversity, as observed in [32].

In [46], the authors introduced a straightforward yet efficient tabu search algorithm centered on drop-and-add moves. The local search phase involves swapping a selected element with an unselected one, provided that it results in an improvement over the current solution. The algorithm terminates when no further improvement is observed after a specified number of iterations. In addition, a short-term tabu search memory is incorporated by marking the most recent moves as tabu. Della Groce et al. proposed an Iterated Local Search heuristic algorithm in [9]. This algorithm tackles the Max-Min Dispersion Problem by reformulating it into a dichotomic search. At each iteration of the search, a clique decision problem is resolved.

In [22], the objective is to identify a subset of p locations to achieve two main objectives: to minimize the maximum dissimilarity between the selected locations and the others and to maximize the minimum distance between the selected locations themselves. Taking advantage of the relationship between the max-min diversity problem and the node packing problem, they propose an integer programming model and an incremental algorithm to find the non-dominated frontier in the biobjective optimization problem.

An interesting variant is recently proposed [54] under the name Fair Maximum-Minimum Diversity (FDM). Each point belongs to one of s different groups, and we are given a set of integers $k_1, \ldots, k_s$, the FDM problem is to select k_i points belonging to group $i \leq s$, such that the minimum pairwise distance between selected points is maximized. The authors proposed a $1 - \epsilon/4$ approximation for FDM when dealing with two groups in the data set, and a second algorithm, SFDM2, provides a $1 - \epsilon/3m + 2$ approximation for FDM with an arbitrary number m of groups.

We can find many metaheuristic algorithms applied to the Max-Min diversity problem. To illustrate one of them, we briefly describe a Particle Swarm Optimization (PSO).

The pseudocode in Algorithm 1 presents a Particle Swarm Optimization (PSO) specifically designed to solve the Max-Min Diversity Problem (MMDP). The PSO algorithm operates by initializing a swarm of particles, with each particle representing a potential solution (subset M). These particles have positions and velocities, where the position encodes a candidate subset, and the velocity dictates how the particle moves within the solution space. The diversity measure for each

Algorithm 1: PSO for Max-Min Diversity Problem

Require: Set $V = \{1, 2, \ldots, n\}$, subset size m, distance function d
Ensure: Subset $M \subseteq V$ that maximizes the minimum distance between elements
Initialize particles with random positions and velocities
Initialize $pBest$ and $gBest$
while the stopping criterion is not met **do**
  for each particle i in the population **do**
    Update the particle's velocity based on $pBest$ and $gBest$ and the particle's position based on the new velocity
    Evaluate the minimum distance between the elements of the new position
    if the diversity of the new position (min distance) is greater than $pBest$'s diversity **then**
      Update $pBest$ with the new position
    end if
  end for
  Update $gBest$ with the best $pBest$ found among all particles
end while
return M, the subset from $gBest$ that maximizes the minimum distance between its elements

particle is calculated as the minimum distance between any two elements within its subset. Throughout the optimization process, the particles update their velocities and positions based on two factors: the best solution found in the trajectory of this particle ($pBest$) and the best global solution ($gBest$), which is the best subset identified by any particle in the swarm. By iteratively refining these subsets, the algorithm directs the swarm toward regions of the solution space that exhibit higher diversity according to the MMDP criterion. This iterative process continues until a predefined stopping condition, such as a maximum number of iterations, is met. Ultimately, the algorithm returns the subset with the best value (the largest minimum pairwise distance encountered during the search), effectively solving the Max-Min Diversity Problem.

The Max-Sum Model

In this section, we turn our attention to the Max-Sum model and review the most effective algorithms to solve it. We start with a memetic algorithm [26], which is initiated by creating the initial population through a greedy randomized procedure. To ensure diversity within this initial pool of solutions, each solution of size n contains m bits set to 1, adhering to the classical binary pattern. Two parents are randomly selected with the condition that neither one is used more than once per generation. In subsequent iterations, new parents are chosen by selecting the best from previous parents and the top-performing offspring without repetition. Subsequently, $p/2$ children are produced through crossover and mutation procedures, where p represents the number of parents. The uniform crossover generates a single child, with chromosomes selected from one parent with a

probability less than or equal to 0.5 and from the other parent with a probability greater than 0.5. Mutation involves flipping bits from 1 to 0 or vice versa, which can occur at each bit position in the string with a certain probability. After each crossover and mutation, a repair process is employed to ensure feasibility, i.e., the chromosome contains m bits set to 1.

A local search process, based on $k - opt$ movements, is applied to the feasible solutions within the memetic algorithm. This process consists of two loops. The inner loop generates solutions by flipping elements of a solution x, until the best solution is not found over t repetitions. In the outer loop, the best solution found is evaluated. Overall, the algorithm proves effectiveness, but with longer execution times compared to other algorithms.

In [15], the authors introduce a memetic self-adaptive evolution strategy. Evolution strategies belong to a class of evolutionary algorithms primarily based on mutation. The authors conduct comparisons with state-of-the-art algorithms: Variable Neighborhood Search (VNS) [4], Learnable Tabu Search (LTS) [53], and Iterated Tabu Search (ITS) [43]. The evolution strategy yields satisfactory results when the aim is not to achieve optimal outcomes; otherwise, simpler heuristics may be preferable.

An improved technique is presented in [6] for analyzing the local search algorithm applied to the problem of maximum sum diversification (max-sum). This enhancement of the local search focuses on negative distances, including those that do not satisfy metric properties. The authors prove that applying local search to these distances provides approximation schemes with polynomial-time complexity. In line with that, [56] address problems where constraints are not symmetric, which means that the relationships between variables are not necessarily bidirectional. This expands the algorithm's applicability to a broader range of real-world optimization problems in the context of the Max-Sum model.

One of the seminal papers on metaheuristic optimization for the Max-Sum problem [11] describes the adaptation of GRASP and Tabu Search to this context, providing Tabu Search with better outcomes. Algorithm 2 shows a pseudocode of a standard tabu search for the Max-Sum model.

Algorithm 2 starts by initializing a current solution, the best solution found so far, and a tabu list that prevents revisiting recently explored solutions. In each iteration, candidate solutions are generated and evaluated, looking for solutions that achieve high values in the sum of their distances between elements, with the best candidate that is not in the tabu list or that meets an aspiration criterion based on the best neighbor. If this neighbor solution improves upon the current solution, the algorithm updates it and the tabu list accordingly. The process continues for a fixed number of iterations, ultimately returning the best solution found. This approach ensures an effective exploration of the solution space while avoiding local optima by incorporating strategic diversification through the tabu mechanism.

Some publications on the Max-sum model approach new applications. This is the case of [52], which considers continuously arriving data sets, such as social media streams or online transactions, while ensuring fairness in the representation of different groups of elements. The paper proposes streaming algorithms, which

Algorithm 2: Tabu Search for Max-Sum Model

Require: Set V, size n, subset size m, distances, max_iter, $tabu_list_size$, $candidate_list_size$, $aspiration_criterion$, Initialize $current_solution$, $best_solution$, $tabu_list$

```
for i = 1 to max_iter do
    Generate candidates
    for each candidate in candidates do
        if candidate ∉ tabu_list or aspiration_criterion(candidate) then
            Generate and evaluate neighbor
            if neighbor better than best_neighbor then
                best_neighbor ← neighbor
            end if
        end if
    end for
    if best_neighbor ≠ NULL then
        Update current_solution, best_solution and tabu_list
    end if
end for
```

are capable of online data processing. These algorithms are designed to efficiently select items in real time, simultaneously optimizing both diversity and equity in the selection process.

In [42], an innovative algorithm is presented that combines the biased random-key genetic algorithm (BRKGA) with local search to solve this model. The uniqueness of this is the fact that the BRKGA algorithm considers a population of size p, represented by a chromosome composed of genes. This chromosome consists of a vector with random values (keys). In [42], the authors use an open-source application programming interface (API) proposed in [51] for the implementation of the BRKGA. This API enables the parallelization of certain operations, such as solution decoding and independent evolution of multiple populations, with inter-population solution exchange, which helps convert the encoded solution (random keys) into a solution of the MDP model.

Several metaheuristics for this problem in [2] are compared to identify additional characteristics for the best improvement. All these metaheuristics are based on a common tabu search module [20]: an Exploring Tabu Search [8], a Variable Neighborhood Search [23], a Scatter-Search [29], and a Random Restart algorithm. These algorithms exhibit competitiveness with the current state of the art. Particularly noteworthy are algorithms based on a random restart or a nearly random restart, where elements of the current best-known solution are prohibited. The authors suggest that due to the simple structure of the MDP, simpler algorithms might yield better results. They argue that the best algorithm available in the literature is the one proposed in [4]. In that paper, the authors present a study to solve the "heaviest $k-$sub-graph" (HSP) problem, with the Max-Sum problem being a special case of HSP. Computational experiments are conducted using instances of the Max-Sum problem to evaluate the efficiency of the proposed algorithms. The authors propose construction methods based on GRASP, Greedy, Scatter-Search, Tabu Search, and

a multi-boot method. The variable neighborhood search method used in the article is equivalent to the one proposed in [24]. This method proves to be more efficient due to the adopted mechanism of variable neighborhoods compared to state-of-the-art heuristics employing specialized hybrid versions of GRASP and Tabu Search. The conclusion drawn is that VNS-based methods can be advantageous for both the HSP and Max-Sum problems, yielding higher quality results albeit at a higher computational cost.

A broader problem related to the Max-Sum problem is the Capacitated Clustering Problem (CCP). This problem involves creating a predefined number of clusters or groups from a set of elements, ensuring that the total weight of elements in each cluster stays within specified capacity limits while maximizing the total benefits of elements within the same cluster. A special case of CCP is the Maximally Diverse Grouping Problem (MDGP), where all elements have a weight of one unit. MDGP is also known as the k-partition problem in [14] or the equitable partition problem in [40].

In [17], the authors introduce a heuristic approach to MDGP based on Tabu Search with Strategic Oscillation (TS-SO). Initially, a greedy construction method is employed to generate an initial solution. Subsequently, the search space in TS-SO encompasses all possible node insertions and swaps. The third phase, Strategic Oscillation (SO), explores solutions where group cardinality constraints might be violated. Specifically, the method allows the number of elements in a group to exceed the specified limits by a certain amount denoted so. After each successful application of the improvement method, the value of so is reset to one to establish an oscillation pattern.

In [35], the authors propose a tabu search and various GRASP (Greedy Randomized Adaptive Search Procedure) variants to find high-quality solutions to this NP-hard problem. These variants leverage different neighborhoods, including a novel one that features a one-for-two swapping strategy. The authors also hybridize both methodologies in this chapter to achieve better outcomes. These algorithms are benchmarked against state-of-the-art techniques. In [10], a GRASP coupled with VNS was proposed for a facility planning application in the context of US mail processing in Postal Service distribution centers.

The Max-Mean Model

In recent years, there has been a growing interest in developing innovative heuristic methods to solve the Max-Mean model.

The authors in [32] propose a GRASP (Greedy Randomized Adaptive Search Procedure) algorithm that combines greedy search with randomization. It exploits the quasi-concave nature of the models for an effective exploration and utilizes a variable neighborhood search with multiple neighborhoods to intensify the search. Additionally, it employs path relinking with a diversity control mechanism to explore a wider search space. The authors compare their algorithm with adaptations of existing Max-Sum solvers.

On the other hand, [5] introduce a method based on greedy construction and tabu search (TS). It first constructs an initial solution with two greedy algorithms, followed by a two-stage TS process. The first stage intensifies the search around the current solution (short-term TS), while the second stage (long-term TS) diversifies the search by potentially accepting worse solutions to escape local optima. Both stages begin from the current solution and terminate when the best solution remains unchanged for a set number of iterations.

Among recent works, in [7], the authors propose a novel approach to parallel ant colony systems (ACS). It introduces a new visibility calculation method and a pheromone trail inspired by the tabu list memory structure of tabu search algorithms. The visibility is based on the frequency of solutions in the global set, increasing the probability of selecting more frequent solutions. Conversely, the pheromone trail reduces the probability of revisiting elements that have already been chosen. In addition, the algorithm considers elements that were not selected for a long time as dispensable and eliminates them with a higher probability.

Here, in Algorithm 3, we present a simplified pseudocode to illustrate the implementation of an Ant Colony Optimization (ACO) algorithm designed to solve the Max-Mean model.

This streamlined pseudocode describes an Ant Colony Optimization (ACO) algorithm designed to solve the Max-Mean Diversity Problem, which focuses on maximizing the average pairwise distance (mean diversity) between elements in a selected subset. The algorithm begins by initializing the levels of pheromones at all edges connecting elements, which influences the probability of selecting elements during subset construction. Each ant in the algorithm iteratively builds a subset by probabilistically choosing elements based on these pheromones and visibility, a heuristic measure related to maximizing mean diversity. After constructing the subsets, the mean diversity of each subset is evaluated. The pheromone levels are

Algorithm 3: ACO for Max-Mean Model

Require: Set $V = \{1, 2, \ldots, n\}$, cardinality m, distance function d
Ensure: Subset $M \subseteq V$ with maximum mean diversity
Initialize pheromones τ on all edges with a small value, and define parameters α, β
for $iter = 1$ to max_{iter} **do**
  for each ant k **do**
    Select the initial element for M_k randomly
    while $|M_k| < m$ **do**
      Calculate probabilities for each $i \notin M_k$ based on τ and η
      Select the next element based on these probabilities
    end while
    Evaluate mean diversity of M_k using the average pairwise distance
  end for
  Apply pheromone evaporation and update pheromones on edges of the best M found (Pheromone update favors edges with higher mean diversity contribution)
end for
return Best M found

then updated, reinforcing edges that contribute to the best subsets to guide the search in future iterations. This process continues until a predefined number of iterations is reached, which ultimately returns the subset with the highest mean diversity found. The ACO algorithm effectively combines exploration and exploitation to iteratively enhance the search for a highly diverse subset according to the Max-Mean model.

In [39], the authors present a hybrid metaheuristic algorithm that combines the Estimation of Distribution Algorithm and Tabu Search (EDA-TS). The algorithm starts by creating an initial elite set of diverse, high-quality solutions and records the best solution. Then it generates a conditional preference probability matrix ($CPPM$) to guide the construction of a solution, since this matrix contains information about the probability of assigning specific values to variables. Then, the Tabu Search method is applied to improve the constructed solution by exploring the solution space locally. An update criterion determines whether the improved solution should be included in the elite set. Finally, when the current solution shows no improvement, a reconstruction method generates a new solution to avoid becoming trapped in local optima.

Furthermore, in [38], the authors present a hybrid reinforcement learning algorithm with tabu search, where the innovation lies in the design of a strategy using a Q-learning mechanism to identify possible regions where the tabu search algorithm may get trapped in a local optimum. This mechanism is used to construct a feasible set of initial solutions.

The Max-MinSum Model

The Max-MinSum DP problem is a challenging combinatorial optimization task, ideal for evaluating search strategies. In their study, [47] present computational experiments applying a GRASP approach to solve the Max-MinSum dispersion problem. They define M_k as a partial solution with k elements selected ($1 \leq k < m$). GRASP initializes M_1 by randomly selecting an element and then builds a candidate list (CL) of unassigned elements in each iteration. For each element i in CL, the method calculates its marginal contribution to the objective function associated with $M_k C1$. Elements in CL are sorted according to their contributions, forming a restricted candidate list (RCL) from which an element is randomly chosen to add to $M_k C1$. An improvement phase follows construction, where elements are exchanged if it improves the objective function; otherwise, the pair is discarded.

In [36], the authors explore six GRASP variants, focusing on different solution generation and improvement methods. The improvement phases involve an exhaustive exploration of all possible exchanges (IM1) or prioritizing contributions of selected and unselected elements (IM2). The best construction phase, CM3, incorporates strategic oscillation (SO) to navigate the feasibility boundary defined by constraints. Experiments show that double-sided strategic oscillation with CM3 and IM2 provides the best overall results, outperforming existing specialized procedures and general-purpose optimizers.

In [30], the authors efficiently solve the computational complexity problem of this model. For this reason, they propose a tabu search algorithm with dynamic neighborhood size (TSDNS), which integrates a solution based on a tabu search strategy with new hash functions and a mechanism to adjust the neighborhood size. For the initialization of tabu lists, three hash vectors are created and an initial solution is generated by randomly selecting m different elements from the set V of elements. The generation of a neighborhood of dynamic size is done using a swap operator, which swaps the values of two variables to generate a new neighboring solution. These swaps are stored in the neighborhood N_{swap}^{full}. However, evaluating the entire neighborhood would consume too much time, so they propose to build a high-quality subset of N_{swap}^{full} as the neighborhood for the TSDNS algorithm. The three hash functions are used to map the neighborhood of the solution, and the three binary hash vectors are used to record the status of the tabu solutions. All of them are considered to avoid possible collisions when using a single hash vector and a single hash function.

In [55], the authors propose a two-phase tabu search intensification algorithm (TPITS), which combines an intensification-based tabu search to avoid revisiting previously found solutions and an attribute-based tabu search to refine the search in the second phase. For initialization, two tabu lists are used: one for solutions and another for attributes. The elements are then randomly selected from the set of available elements to generate the initial solution, with the best solution chosen during this search. In general, the TPITS algorithm consists of three main components: a dynamic neighborhood size to build a neighborhood, a tabu search strategy focused on intensification, and a tabu search strategy based on attributes.

Genetic algorithms can also be used for the Max-MinSum model. In Algorithm 4, we present a simplified pseudocode to demonstrate the implementation of a Genetic Algorithm (GA). This pseudocode describes a Genetic Algorithm implementation tailored to solve the Max-MinSum Diversity Problem, where the goal is to find a subset M that maximizes the minimum sum of inter-element distances. The algorithm begins by generating an initial population of candidate solutions, each representing a subset of elements. These solutions are evaluated on the basis of the Max-MinSum diversity criterion. Through iterative steps, the algorithm selects parent solutions, applies the crossover operator to generate offspring, and introduces the mutations to maintain genetic diversity. The offspring are then evaluated, and the population is updated by selecting the best solutions. This process continues until a stopping criterion, such as a maximum number of iterations, is reached. At this point, the best solution found is returned as the optimal subset that addresses the diversity problem.

The Min-Diff Model

Recent research on the Min-Diff model is summarized in four key papers:[3,12,37]. In [3], a two-phase heuristic algorithm is introduced. The constructive phase

Algorithm 4: Genetic algorithm for Max-MinSum DP

Require: Set $V = \{1, 2, \ldots, n\}$, subset size m, distance function d, population size p, maximum iterations max_iter

Ensure: Subset $M \subseteq V$ that optimizes the diversity criterion by maximizing the minimum sum of inter-element distances

1: Generate an Initial Population with p solutions, where each solution is a subset of m elements from V
2: Evaluate the diversity of each solution in the Initial Population using the Max-MinSum criterion
3: **for** $iter = 1$ to max_iter **do**
4: Select Parents from the Population based on their diversity values
5: Apply Crossover to generate Offspring from selected Parents
6: Apply Mutation to Offspring to introduce genetic diversity
7: Evaluate the diversity of the Offspring using the Max-MinSum criterion
8: Update Population by selecting the best solutions from the combined pool of current Population and Offspring
9: **end for**
10: Extract the BestSolution M from the final Population
11: **return** M

leverages the connection between this problem and the Maximum Clique Problem (MCP), while the improvement phase employs a tabu search procedure. These phases are iteratively repeated, with a simple diversification mechanism added to the constructive phase. Furthermore, this work presents a method to decrease computational complexity at each iteration, transitioning from a quadratic evaluation to a linear update mechanism.

In [12], the authors propose various hybrid heuristics, with the most effective being a GRASP combined with greedy construction samples and a variable neighborhood search for local improvement. Eight constructive procedures, four local search procedures (including one based on VNS), and four path relinking strategies are considered. Using exterior path relinking, a variant introduced by Glover in [22], proved competitive in extensive computational experiments in 190 cases of the existing literature. This algorithm not only outperformed the GRASP heuristic of [43] and nearly reached optimality for most instances solvable by CPLEX, but it also improved CPLEX's upper bound for challenging instances.

The pseudocode shown in Algorithm 5 illustrates a simplified implementation of the Greedy Randomized Adaptive Search Procedure (GRASP) algorithm for solving the Min-Diff Dispersion Problem.

The algorithm begins by initializing the best solution and best diversity values. For each iteration, a candidate solution is constructed using a greedy randomized approach, followed by a local search to refine the solution. The diversity of each candidate is then calculated as the difference between the maximum and minimum dispersion sums. If the calculated diversity of the current solution is lower than the best diversity recorded previously, the current solution is stored as the best new solution. This iterative process continues until the maximum number of iterations is reached, at which point the algorithm returns the best subset found that minimizes

Algorithm 5: GRASP to solve the Min-Diff DP

Require: Set $V = \{1, 2, \ldots, n\}$, subset size m, distances $distances$, maximum iterations max_iter, α

Ensure: Subset $M \subseteq V$ that optimizes diversity criterion by maximizing the difference between the largest and smallest values of the dispersion sum

```
best_solution ← ∅
best_diversity ← −∞
for i = 1 to max_iter do
    solution ← ConstructSolution(V, n, m, distances)
    local_solution ← LocalSearch(solution, distances)
    diversity ← CalculateDiversity(local_solution, distances)
    if diversity > best_diversity then
        best_solution ← local_solution
        best_diversity ← diversity
    end if
end for
return best_solution
```

the Diff diversity criterion. This approach effectively balances exploration and exploitation, focusing on maximizing the dispersion difference to ensure a robust search for the most diverse subset.

In [37], the authors propose a basic variable neighborhood search (VNS) heuristic, focusing primarily on interchanged neighborhood structures for both the intensification and diversification phases. They demonstrate that VNS outperforms a complex method that combines GRASP, variable neighborhood search, and exterior path relinking metaheuristics. The authors emphasize that their results underscore the essence of heuristics: creating efficient and effective algorithms that remain as simple as possible, a succinct expression of "less is more."

Finally, in [31], the authors propose an intensification-driven tabu search algorithm to solve the Min-Diff dispersion problem by integrating a restricted neighborhood, a solution-based tabu strategy, and an intensified search mechanism to create a search that effectively takes advantage of both intensification and diversification elements.

Conclusions

In this chapter, we reviewed two diversity and three equity models and also considered two related models. We identified previous mathematical models and heuristic methods for these variants. Most of them are based on constructive and local search-based methods. This is the case of the GRASP and Tabu Search methodologies, which have been customized for some of these models with excellent results. Surprisingly, evolutionary methods, such as popular genetic algorithms, have been considered in just a few cases.

We noted that although the problems considered here are related, all previous efforts have been devoted to tackle very specific models, and no generic solver has been proposed to deal with the entire family. In addition, we studied the solution structure obtained with the different models, and we identified some differences and similarities between them.

The literature on heuristic and metaheuristic methods for solving diversity problems is extensive; however, the current state of the art for exact methods remains less satisfactory. Although efficient exact methods have been developed specifically for the Max-Min Diversity Problem, there are relatively few studies on the exact resolution of other diversity models. In addition, the development of generic solvers that deal with several models would be of interest. So, there is a definite need for further research on algorithms, mainly for exact algorithms but also for heuristics.

Cross-References

▸ Tabu Search
▸ Theory of Local Search
▸ Variable Neighborhood Descent

Acknowledgments This research has been supported by the Ministerio de Ciencia e Innovación of Spain (Grant Ref. PID2021-125709OB-C21) funded by MCIN /AEI /10.13039/501100011033 / FEDER, UE. It has also been supported by the Generalitat Valenciana (CIAICO/2021/224).

Competing Interest Declaration The author(s) has no competing interests to declare that are relevant to the content of this manuscript.

References

1. Ağca S, Eksioglu B, Ghosh JB (2000) Lagrangian solution of maximum dispersion problems. Naval Res Logist 47:97–114
2. Aringhieri R, Cordone R (2011) Comparing local search metaheuristics for the maximum diversity problem. J Oper Res Soc 62:266–280
3. Aringhieri R, Cordone R, Grosso A (2015) Construction and improvement algorithms for dispersion problems. Eur J Oper Res 242:21–33
4. Brimberg J, Mladenovic N, Urosevic D, Ngai E (2009) Variable neighborhood search for the heaviest k-subgraph. Comput Oper Res 36:2885–2891
5. Carrasco R, Pham A, Gallego M, Gortázar F, Martí R, Duarte A (2014) Tabu search for the max-mean dispersion problem, pp 1–17
6. Cevallos A, Eisenbrand F, Zenklusen R (2019) An improved analysis of local search for max-sum diversification. Math Oper Res 44(4):1494–1509
7. Cura T, Özdemir M (2019) A new use of the ant system algorithm for the max-mean dispersion problem. Comput Ind Eng 135:628–642
8. Dell'Amico M, Trubian M (1998) Solution of large weighted equicut problems. Eur J Opl Res 106:500–521
9. Della Groce F, Grosso A, Locatelli M (2009) A heuristic approach for the max-min diversity problem based on max-clique. Comput Oper Res 36(8):2429–2433

10. Deng Y, Bard JF (2011) A reactive GRASP with path relinking for capacitated clustering. J Heurist 17:119–152
11. Duarte A, Martí R (2007) Tabu search and GRASP for the maximum diversity problem. Eur J Oper Res 178:71–84
12. Duarte A, Sánchez-Oro J, Resende M, Glover F, Martí R (2015) Greedy randomized adaptative search procedure with exterior path relinking for differential dispersion minimization. Inf Sci 296:46–60
13. Erkut E, Neuman S (1989) Analytical models for locating undesirable facilities. Eur J Oper Res 40:275–291
14. Feo T, Goldschmidt O, Khellaf M (1992) Onehalf approximation algorithms for the k-partition problem. Oper Res 40:S170–S173
15. Freitas ARR, Guimaraes FG, Pedrosa Silca RC, Souza MJF (2014) Memetic self-adaptive evolution strategies applied to the maximum diversity problem. Optim Lett 8(2):705–714
16. French E (2005) The importance of strategic change in achieving equity in diversity. Wiley InterScience, Strategic Change 14:35–44
17. Gallego M, Laguna M, Martí R, Duarte A (2013) Tabu search with strategic oscillation for the maximally diverse grouping problem. J Oper Res Soc 64(5):724–734
18. Ghosh JB (1996) Computational aspects of the maximum diversity problem. Oper Res Lett 19:175–181
19. Glover F, Kuo CC, Dhir KS (1995) A discrete optimization model for preserving biological diversity. Appl Math Model 19:696–701
20. Glover F, Laguna M (1997) Tabu search. Kluwer Academic Publishers, Boston
21. Glover F, Kuo CC, Dhir KS (1998) Heuristic algorithms for the maximum diversity problem. J Inf Optim Sci 19(1):109–132
22. Golbarg KT, Yahya F (2018) Effective methods for solving the Bi-criteria p -Center and p -Dispersion problem. Comput Oper Res 101:43–54
23. Hansen P, Mladenovic N (2001) Variable neighborhood search: principles and applications. Eur J Opns Res 130:449–467
24. Hansen P, Mladenovic N (2003) Variable neighborhood search. In: Glover F, Kochenberger G (eds) Handbook of metaheuristics. International series in operations research & management science, Springer, Berlin
25. Hassin R, Rubinstein S, Tamir A (1997) Approximation algorithms for maximum dispersion. Oper Res Lett 21:133–137
26. Katayama K, Narihisa H (2005) An evolutionary approach for the maximum diversity problem. In: Recent advances in memetic algorithms, vol 166. Springer, Heidelberg, pp 31–47
27. Kincard RK (1992) Good solutions to discrete noxious location problems via metaheuristics. Ann Oper Res 40:265–281
28. Kuo CC, Glover F, Dhir KS (1993) Analyzing and modeling the maximum diversity problem by zero-one programming. Decis Sci 24:1171–1185
29. Laguna M, Marí R (2003) Scatter search: methodology and implementations in C. Kluwer Academic Publishers, Dordrecht
30. Lai X, Fu Z (2019) A tabu search approach with dynamical neighborhood size for solving the maximum min-sum dispersion problem. IEEE Access 7:181357–181368
31. Lai X, Hao J, Glover F, Yue D (2019) Intensification-driven tabu search for the minimum differential dispersion problem. Knowl-Based Syst 167:68–86
32. Martí R, Sandoya F (2013) GRASP and path relinking for the equitable dispersion problem. Comput Oper Res 40:3091–3099
33. Martí R, Martínez-Gavara A, Pérez-Peló S, Sánchez-Oro J (2022) A review on discrete diversity and dispersion maximization from an OR perspective. Eur J Oper Res 299(3):795–813
34. Martí R, Martínez-Gavara A (eds) (2023) Discrete diversity and dispersion maximization: a tutorial on metaheuristic optimization
35. Martínez-Gavara A, Campos V, Gallego M, Laguna M, Martí R (2015) Tabu search and GRASP for the capacited clustering problem. Comput Optim Appl 62:589–607

36. Martínez-Gavara A, Campos V, Laguna M, Martí R (2017) Heuristic solution approaches for the maximum MinSum dispersion problem. J Glob Optim 67:671–686
37. Mladenović N, Todosijević R, Urosević D (2016) Less is more: basic variable neighborhood search for minimum differential dispersion problem. Inf Sci 326:160–171
38. Nijimbere D, Zhao S, Gu X, Esangbedo O (2021) Tabu search guided by reinforcement learning for the max-mean dispersion problem J Ind Manag Optim 17(6):3223–3246
39. Nijimbere D, Zhao S, Liu H, Peng B, Zhang A (2019) A hybrid metaheuristic of integrating estimation of distribution algorithm with tabu search for the max-mean dispersion problem. Mathematical problems in engineering. Hindawi, vol 2019, pp 1–16
40. O'Brien FA, Mingers J (1995) The equitable partitioning problem: a heuristic algorithm applied to the allocation of university student accommodation. Warwick Business School, Research Paper no. 187
41. Page SE (2007) The difference: how the power of diversity creates better groups, firms, schools, and societies. Princeton University Press, New Jersey
42. Silva G, Leite A, Ospina R, Leiva V, Figueroa-Zúñiga J, Castro C (2023) Biased random-key genetic algorithm with local search applied to the maximum diversity problem. Mathematics 11(14):3072
43. Palubeckis G (2007) Iterated tabu search for the maximum diversity problem. Appl Math Comput 189(1):371–383
44. Parreño F, Álvarez-Valdés R, Martí R (2021) Measuring diversity. A review and an empirical analysis. Eur J Oper Res 289(2):515–532
45. Pisinger D (2006) Upper bounds and exact algorithms for p-dispersion problems. Comput Oper Res 33:1380–1398
46. Porumbel D, Hao J, Glover F (2011) A simple and effective algorithm for the MaxMin diversity problem. Ann Oper Res 186(1):275–293
47. Prokopyev OA, Kong N, Martinez-Torres DL (2009) The equitable dispersion problem. Eur J Oper Res 197:59–67
48. Ravi SS, Rosenkrantz DJ, Tayi GK (1994) Heuristic and special case algorithms for dispersion problems. Oper Res 42:299–310
49. Resende MGC, Werneck R (2004) A hybrid heuristic for the p-median problem. J Heurist 10(1):59–88
50. Resende MGC, Martí R, Gallego M, Duarte A (2010) GRASP and path relinking for the max-min diversity problem. Comput Oper Res 37(3):498–508
51. Toso, RF, Resende MGC (2015) A C++ application programming interface for biased random-key genetic algorithms. Optim Methods Softw 30(1):81–93
52. Wang Y, Fabbri F, Mathioudakis M (2022) Streaming algorithms for diversity maximization with fairness constraints. In: 2022 IEEE 38th international conference on data engineering (ICDE). IEEE, pp 41–53
53. Wang J, Zhou Y, Cai Y, Yin J (2012) Learnable tabu search guided by estimation of distribution for maximum diversity problems. Soft Comput 16(4):711–728
54. Wang Y, Fabbri F, Mathioudakis M, Li J (2023). Fair max–min diversity maximization in streaming and sliding-window models. Entropy 25(7):1066
55. Wang Y, Lü Z, Su Z (2021) A two-phase intensification tabu search algorithm for the maximum min-sum dispersion problem Comput Oper Res 135:105427
56. Zivan R, Parash T, Cohen-Lavi L, Naveh Y (2020) Applying max-sum to asymmetric distributed constraint optimization problems. Auton Agents Multi-Agent Syst 34:1–29

Evolutionary Algorithms for the Inverse Protein Folding Problem

41

Sune S. Nielsen, Grégoire Danoy, Wiktor Jurkowski, Roland Krause, Reinhard Schneider, El-Ghazali Talbi, and Pascal Bouvry

Contents

Introduction . . . 1296
Amino Acids and Protein Structure . . . 1297
Inverted Protein Folding . . . 1299
Diversity Preservation as a Tool . . . 1299
Related Work . . . 1300
Protein Design . . . 1300
Multimodal Optimization and Niching . . . 1302
Problem Description . . . 1303
Sequence Identity . . . 1304
Problem Model . . . 1304
Secondary Structure Definition . . . 1305
Secondary Structure Estimation . . . 1305
Diversity Measure . . . 1306
Algorithm Design . . . 1307
Removal of Doubles . . . 1307
Quantile Constraint . . . 1308

S. S. Nielsen (✉) · G. Danoy · P. Bouvry
Computer Science and Communications (CSC) Research Unit, FSTC, University of Luxembourg, Luxembourg City, Luxembourg
e-mail: sune.nielsen@uni.lu; gregoire.danoy@uni.lu; pascal.bouvry@uni.lu

W. Jurkowski
The Genome Analysis Centre (TGAC), Norwich Research Park, Norwich, UK
e-mail: wiktor.jurkowski@tgac.ac.uk

R. Krause · R. Schneider
Luxembourg Centre for Systems Biomedicine (LCSB), University of Luxembourg, Luxembourg City, Luxembourg
e-mail: roland.krause@uni.lu; reinhard.schneider@uni.lu

E.-G. Talbi
Université des sciences et technologies de Lille, INRIA Lille Nord Europe, Villeneuve d'Ascq, France
e-mail: el-ghazali.talbi@inria.fr; el-ghazali.talbi@lifl.fr

© Springer Nature Switzerland AG 2025

R. Martí et al. (eds.), *Handbook of Heuristics*,
https://doi.org/10.1007/978-3-032-00385-0_59

Algorithm Experiments 1308
Protein Samples 1308
Experimental Setup 1308
Algorithm Results 1309
Structure Validation 1312
Primary and Secondary Structure Validation Results 1312
Tertiary Structure Validation Results 1314
Conclusion 1316
Cross-References 1317
References 1318

Abstract

Protein structure prediction is an essential step in understanding the molecular mechanisms of living cells with widespread application in biotechnology and health. The inverse folding problem (IFP) of finding sequences that fold into a defined structure is in itself an important research problem at the heart of rational protein design. In this chapter, a multi-objective genetic algorithm (MOGA) using the diversity-as-objective (DAO) variant of multi-objectivization is presented, which optimizes the secondary structure similarity and the sequence diversity at the same time and hence searches deeper in the sequence solution space. To validate the final optimization results, a subset of the best sequences was selected for tertiary structure prediction. Comparing secondary structure annotation and tertiary structure of the predicted model to the original protein structure demonstrates that relying on fast approximation during the optimization process permits to obtain meaningful sequences.

Keywords

Genetic algorithm · Diversity preservation · Inverse folding problem

Introduction

The relation between the amino acid sequence of a protein and its three-dimensional structure is a principal research effort of structural biology. Obtaining the folded structure of a protein allows functional studies in silico and has given rise to the field of protein engineering.

Proteinsare responsible for the majority of molecular functions in a cell. A simplified illustration of a real protein is provided in Fig. 1. Understanding protein folding has immense implications from health to biotechnology applications. Protein engineering in general aims at designing molecules with desired properties, and a method that allows to successfully design such molecules would find applications in a number of areas. For example, it could allow to design improved enzymes for biotechnology applications such as wastewater treatment or biomass

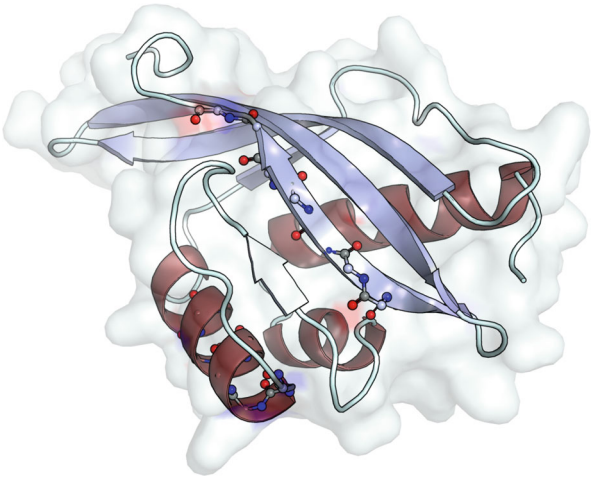


Fig. 1 Protein example *1OH0* with its surface shown semitransparent. *Helix* and *sheet* secondary structure segments are shown in dark red and light blue, respectively. Selected atoms are displayed for further clarification

production [7] or new antibodies specific toward already known targets, e.g., a given pathogen like HIV, by binding to its envelope spikes to neutralize the virus [19]. Since the advent of genome sequencing, all protein-coding genes of an organism can be obtained with ease, but structure prediction capabilities were only slightly improved over the last two decades and remain poor. If no homologous structure to a given sequence exists (the ab initio problem), finding the correct structure remains an essentially intractable, which hampers even the comparably easy task of classifying protein sequences into families.

Amino Acids and Protein Structure

A protein sequence is the code that describes the linear combination of any of the 20 common amino acids, also referred to as residues. The amino acid residues are basic organic building blocks consisting mainly of carbon (C), hydrogen (H), oxygen (O), and nitrogen (N) atoms. Common for all amino acids are their *amine* and *carboxylic acid* functional groups which bind through peptide bonds to form the protein backbone of $N - C_\alpha - C$ atoms as shown in Fig. 2. When ordered from left to right, as in the figure, the *amine* group, here represented by its nitrogen (N) atom for simplicity, is situated to the left of the amino acid, respectively, at the beginning of the chain. The side chains, noted as R_i, vary with each of the possible amino acids and can vary both in size and other properties, such as charge, acidity, and hydropathy. A typical protein sequence is 50–300 residues long. Due to the rotational freedom of the atom bonds and the molecular forces acting between the residues, it folds into one canonical three-dimensional structure. These intermolecular forces are the sum of a number of complex interaction forces largely depending on the mentioned properties of the residues, but also on the

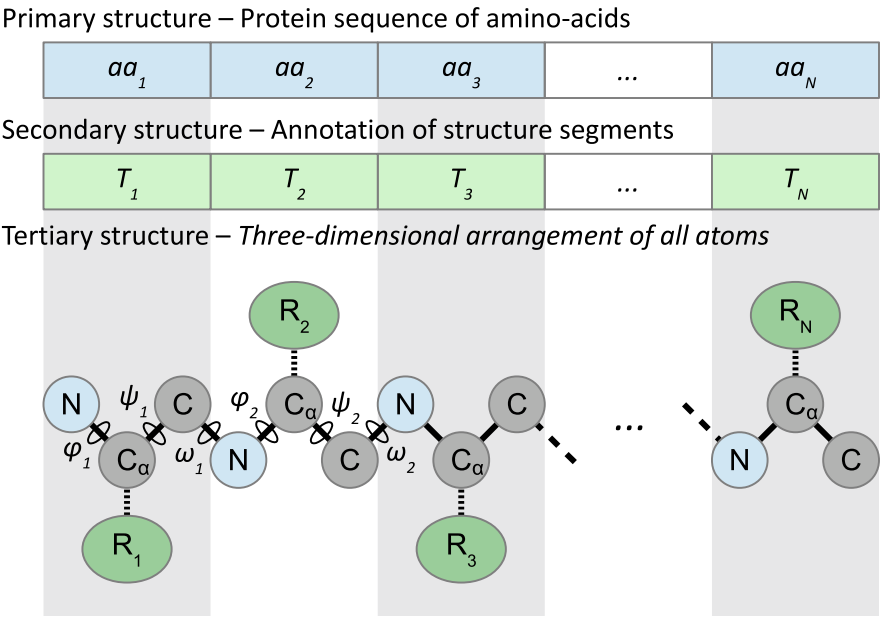


Fig. 2 Three levels of protein structure

distance and orientation of interacting atoms and structures. In general the protein structure will try to adapt a lower-energy configuration like a bolder that will roll down a mountain into the valley due to the gravitational force. In the case of proteins, such a more relaxed state corresponds to parts of the protein being either stacked or curled together referred to as *sheets* or *helices* as seen in Fig. 1. The remaining unstructured segments are commonly referred to as *loops* and serve as flexible connections between the other segments. The structure of a protein can be defined in different levels (see Fig. 2). The primary structure is the protein sequence of N amino acids $\{aa_i\}$ where $1 \leq i \leq N$ is the residue position. The secondary structure defines the organization of *helices*, *sheets*, and *loops* of the tertiary structure and can be expressed by a type $\{T_i\} \in \{H, E, L\}$ for each position i in the protein. If, for example, a protein consists of a *helix* and two *sheets*, its secondary structure would look like this: $\{L, L, H, H, H, H, L, E, E, L, E, E, L\}$. The tertiary structure completely describes the arrangement of all atoms of a protein in the three-dimensional space. The ensemble of three-dimensional positions of C_α atoms is commonly referred to as the alpha-trace which provides a rough residue type- and rotation-independent view of the protein configuration. Similar protein sequences generally obtain the same configuration or fold, but sequences not recognizable by similarity can nevertheless fold into 3D structures that are easily brought into congruence. Recommended reading for more in-depth information about proteins and their function in cells is the book by Alberts et al. [2].

Inverted Protein Folding

Conventional protein folding prediction research is concerned with finding or predicting the folded structure of a given amino acid sequence. As the problem is not solved, even to the present day, scientists have early on sought to simplify the task by solving the inverse problem. With the hierarchical definition of Fig. 2 in mind, the inverse folding problem (IFP) can be defined as follows: given a primary structure (protein sequences) and its corresponding tertiary structure, find alternative sequences that will result in the same tertiary structure. The inverted problem is thought of as a simplification because the structure is given, and sequence to structure compatibility becomes the main difficulty. When the structure is unknown (the ab initio case), the number of possible configuration solutions is enormous. A central part of any protein design process is to obtain, or come close to, a target tertiary structure with a certain degree of freedom in the choice of protein sequence. Hence solving the IFP would be a key to successfully engineer proteins. Furthermore, the IFP is of general scientific interest to study the size, shape, and characteristics of the sequence space that matches a given target structure.

Diversity Preservation as a Tool

In this chapter, the fact that matching secondary structures is a necessary, but not a sufficient condition for proteins to have the same tertiary structures, is exploited to reduce the IFP to its simplest formulation: given a protein's secondary structure and its corresponding protein sequence as input, find a set of highly dissimilar protein sequences that could result in the most similar secondary structure. The multi-objective genetic algorithm (MOGA) variant presented here is hence designed for maintaining high diversity, which in turn allows it to explore the decision space of sequences more efficiently and find better solutions than a conventional algorithm. This essentially makes the diversity preservation characteristic central in two aspects: (1) it increases the algorithm's performance in that it continuously pushes the exploration to new areas of the search space while (2) addressing the part of the problem statement of finding a set of protein sequences (i.e., problem solutions) that show large diversity among each other. The latter aspect is thoroughly covered in ▸ Chap. 40, "Diversity Optimization Models," though it should be noted that the representation of solutions and distance among them is different from this work.

An extended validation test is run predicting the final folded structure of as many as 300 generated sequences which are then analyzed in terms of secondary and tertiary structure. This test aims at answering the question of how well the target tertiary structure can be matched solely by taking the secondary structure into account.

The remainder of this chapter is organized as follows: in section "Related Work" the current work is situated in related literature; then a detailed description of the problem and the biological background is introduced in section "Problem

Description" and modeling thereof described in section "Problem Model". In section "Algorithm Design" the methodology achieving adjustable level of diversity in the genetic algorithm is presented. Section "Algorithm Experiments" describes the experiments conducted and the results obtained in terms of algorithm performance, with a validation study of secondary and tertiary structure in sections "Primary and Secondary Structure Validation Results" and "Tertiary Structure Validation Results". Finally the results and perspectives are summarized in section "Conclusion".

Related Work

This section reviews some of the most relevant works related to the two main areas covered in this chapter: protein design and diversity preservation in metaheuristics.

Protein Design

Most applied work of the IFP is concerned with protein design. Since the first design of a peptide by Gutte et al. [14] using secondary structure rules, numerous works have described different approaches to the IFP problem. The earliest reference to the inverted approach is found in an article by Pabo [24] referring to Drexler [11] stating that protein design engineers could in theory choose from a vast subset of possible sequences containing strategically placed groups that would have a predictable fold. Another early attempt at tackling the IFP is done by Ponder and Richards [25] who used a systematic exhaustive approach of enumerating a selected subset of residue positions. Central to the approach is the focus on packing criteria of core residues, taking a latest available side-chain rotamer library into account. Core residues are internal or buried residues not in contact with solvent. They contribute to the general structure of the protein and rather seldom to its primary function. A rotamer library is a library of known side-chain arrangements in 3D for each residue which is important to consider when evaluating the space filling of the core structure.

A few years later, Bowie et al. [5] introduced a 3D to 1D score for each secondary structure type and six environmental classes determined by (1) area buried in the protein structure and (2) fraction of polar side-chain area. By analyzing 16 known structures, the overall relative probability of observing a residue in a defined environment class is computed. From this and the target tertiary structure, a 3D profile can be generated taking the environment at each residue position into account. The 3D to 1D score is calculated by matching a sequence to the 3D profile of a structure. The result is expressed relatively using the Z-score, indicating the number of standard deviations above the mean of other sequences of same length. Using this method they were able to clearly separate homologs (evolutionary-related proteins) in terms of Z-score from a large set of sequences. Kuhlman and Baker [21] used a Monte Carlo approach of residue and rotamer substitution at 11 nonadjacent core positions, evaluating a free energy function. The lowest energy sequence of five

algorithm runs was chosen, and as a final result half of the generated residues were identical to the reference protein, referred to as "wild type."

The first to use a genetic algorithm (GA) was Jones in [17]. To assess 3D-1D compatibility and define an objective function, a set of statistically determined potentials known from fold recognition are used: pairwise potential and solvation potential. To prevent the generation of unlikely sequences, a residue composition term with an arbitrary weight is added corresponding to the target folding class ($\alpha\alpha$, $\alpha\beta$, $\beta\beta$). Jones concluded that there is no way to be sure the resulting sequences have not been overdesigned as the optimal sequence scores significantly better than the reference. He speculates that the energy optimal shape might be very steep and too hard for the real-world protein to fold into. Therefore, the algorithm should possibly be stopped earlier.

Mayo et al. [29] successfully used backbone flexibility in the design process by generating a set of perturbed backbones and applying enumeration of ten varying residue positions applying dead-end elimination to cut the search space. Similarly, Harbury et al. [15] incorporated such backbone freedom in their design approach. Both latter approaches were evaluated by synthesizing the proteins in the lab. Isogai et al. [16] used a recursive approach searching the 3D profile of the target structure keeping two residues fixed and applying a penalty to residues that protrude into the space with a repulsive function. Collisions among side chains were removed manually by replacing residues with smaller ones. The design was successfully synthesized, but the binding site did not stably bind oxygen.

Wernisch et al. [33] sought to combine the latest approaches into an automatic software solution named DESIGNER. The CHARMM package [6] is used for force-field calculation among side chains and backbone taking all hydrogen atoms bonded and nonbonded into account as well as adding van der Waals forces and electrostatic interaction. Both an exact enumeration approach (branch and bound) and a simple heuristic selecting the optimal rotamer for one random position at the time until a local optimum has been reached were tested. Different experiments aimed at analyzing different setting effects on the results were conducted. One test compares the effect of neglecting the reference energy and solvation energy terms, respectively, when redesigning 11 buried positions in the core. The choice of energy terms largely impacts the amount of polar amino acids, and neglecting the solvation term produced better packing with less cavities. Another test aimed at optimizing the protein surface with its larger proportion of polar amino acids. Again 11 positions are variable and varying settings are tested. First backbone and rotamers are kept fixed, and then alternative rotamers were allowed. Wernisch et al. consider that the energy calculations are approximations. Therefore, the software allows for outputting multiple solutions within a user-defined energy window. When packing constraints apply, DESIGNER generated sequences close to the reference.

Voigt et al. [32] combined the field of directed evolution with that of computational design and seek to benefit from both. Directed evolution is concerned with improving specific protein properties or functions mainly by applying a series of mutations to the target as mutagenesis in nature. In their computational

method, energy was used to predict structural stability, and residues with low entropy are detected as more tolerant to mutations. They also argued that coupled residues should be substituted together as several replacements need to take place to demonstrate improvement. High variability was observed on the exposed residues, and in general the variability should guide mutagenesis to allow the generation of a family of divergent sequences with structural integrity intact.

Klepeis et al. [20] presented a two-stage approach where an integer program is first used to generate a list of low-energy sequences which are then evaluated in terms of their fold. Using a force field based on pairwise C_α, distance-dependent interaction potential gives a more relaxed backbone flexibility constraint with less empirically tuned parameters. Validation was done by improving the activity of Compstatin, a 13-residue-long peptide fundamental in inhibiting complement activation. Certain residue positions and types were restricted based on knowledge about the functional nature and with the goal of increasing activity. Experimental results on 14 designed sequences showed significant activity improvements in most cases, one analogue was six to seven times more active than the wild-type underlining. This two-stage approach with small variations is used to design a template for human β-defensiv-2 in [12] and with more advanced second stage in [3, 4].

Smadbeck et al. [28] have recently streamlined the two-stage process and present a server implementation with a usage example. The web interface allows for specifying all inputs: template (rigid/flexible), energy function (C_α, centroid, or any), and biological constraints (on charge and content). Stage two workflow consists of two independent fold specificity and approximate binding affinity modules. These include programs such as CYANA, TINKER, and AMBER for the, first, Rosetta (ab initio, dock, and design) and OREO for the latter.

Finally, Mitra et al. [23] used templates of structure families in combination with a force field to guide the search rather than physics-only-based force fields. Due to shortcomings of the latter, evolutionary-based designs have been demonstrated to be more stable. Experiments were conducted with one of the leading protein structure prediction frameworks, I-TASSER [37]. Previous works have shown that I-TASSER is able to distinguish successful designs from unsuccessful ones and is therefore used as validation of the results also in this work.

The research of the last three decades on the IFP problem has produced many methods, but their complexity and exhaustive nature effectively limits the size of the sequence or decision space that can be sampled. In addition, the final output of these methods consists of a single or few sequences close to the input sequence, where a larger and more diverse set of sequences would be desirable for practitioners.

Multimodal Optimization and Niching

In metaheuristics the subject of exploration vs. exploitation characteristics has been thoroughly studied. For population-based optimization algorithms, it is well

known that a higher level of population diversity results in more exploration at the expense of exploitation. An elevated population diversity is especially desirable for *multimodal*, *deceptive*, and/or *dynamic* problems. In general, if diversity tends toward zero, it indicates that the algorithm has converged toward a single solution, which might be an undesired behavior if it occurs too early. A number of works have focused on maintaining and controlling diversity, such as crowding methods by DeJong [8], fitness sharing by Goldberg and Richardson [13], cellular algorithms by Alba and Dorronsoro [1], and diversity-preserving selection strategies based on hamming distance by Shimodaira [27] and on altruism by Laredo et al. [22]. Another approach consists in designing new objectives through multi-objectivization, with which the problem is transformed into a bi- or multi-objective one. Extending problems with an objective designed specifically for diversity preservation has been proposed by Toffolo and Benini [30], by Deb and Saha [9], and most recently by Wessing et al. [34]. In these works, objectives have been designed based on the hamming distance to the closest neighbor, the distance to the nearest better, and number of individuals in the neighborhood.

In this chapter, the diversity-preserving objective is based on the average distance of each individual to all others which directly targets the global diversity measure stated by the problem, contrary to the pairwise local view of existing works. Given the discrete nature, complexity, and multimodality of the problem, an effective diversity limiting mechanism is required. The proposed approach achieves this with the added value of making the population diversity highly variable depending on a single algorithm setting.

Problem Description

The focus in this work is on finding multiple and diversified solutions to the inverse folding problem (IFP).

A simplified model is developed to match solely the reference secondary structure—a requirement for the tertiary structure; see Fig. 3 for a schematic representation. This is motivated by the fact that computing the tertiary structure of a given input sequence is computationally very expensive which would prevent the usage of a metaheuristic on the entire sequence. The found solutions should be a collection of very dissimilar sequences, as well as dissimilar to the input sequence or its homologs, the naturally occurring, evolutionary-related sequences of the input sequence.

A single solution is represented as a sequence $A = \{aa_i\}$ composed of N residue positions, where $1 \leq i \leq N$ and $aa_i \in \{1, 2, \ldots 20\}$ correspond to the set of 20 possible amino acids. As the solution space consists of a total of 20^N different combinations, considering that N is around 50–200 for typical design targets, alternatives to exhaustive exploration are mandatory.

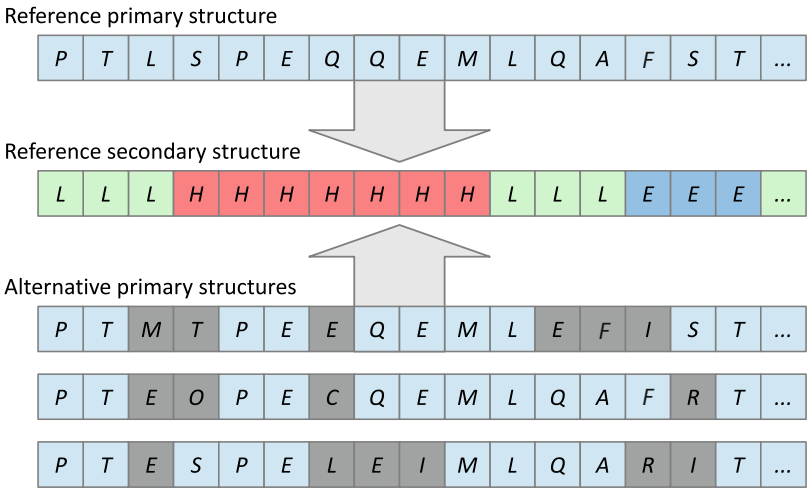


Fig. 3 Primary and secondary structure in the inverted folding problem

Sequence Identity

Sequence identity is a common measure designed to assess the similarity of proteins occurring in nature in terms of their primary structure only. When computing sequence identity, gaps are taken into account during the alignment of the sequences to be able to detect evolutionary relations among the compared proteins even if their sequences are of different lengths. In this work, all sequences being compared have the length of the target sequence and are generated by a random process. The chances of the same subsequence to occur in two different sequences with an offset diminish quickly as the subsequence length increases, which justifies ignoring gaps in the model. For the comparison of final results, the generally accepted approach with taking gaps into account is used.

Problem Model

This section presents the corresponding optimization problem. Two objective functions are first defined for integer encoded solutions $A = \{aa_i\}$. The first function estimates the similarity of secondary structure, in which definition and estimation are provided in sections "Secondary Structure Definition" and "Secondary Structure Estimation," respectively. The second function presented in section "Diversity Measure" is designed to address the diversity requirements of the problem and of the algorithm.

Secondary Structure Definition

Secondary structure refers to the annotation of structure segmentation as seen in Figs. 2 and 3. These segments are the result of the protein naturally folding so that different parts of its 3D structure connect through bonds between amino acids on separated residue positions in the sequence. Tertiary structure annotations are done using the "Define Secondary Structure of Proteins" (DSSP) tool [18]. As only the three structure types, *helices (H)*, *sheets (E)*, and *loops (L)*, are considered throughout this work, some simplification is required. In the documentation of DSSP, the following possible annotation types are found:

- $G =$ 3-turn helix (310 helix). Min length three residues.
- $H =$ 4-turn helix (α helix). Min length four residues.
- $I =$ 5-turn helix (π helix). Min length five residues.
- $T =$ hydrogen-bonded turn (three, four, or five turn)
- $E =$ extended strand in parallel and/or antiparallel β-sheet conformation. Min length two residues.
- $B =$ residue in isolated-bridge (single pair β-sheet hydrogen bond formation)
- $S =$ bend (the only non-hydrogen bond-based assignment).
- $C =$ coil (residues which are not in any of the above conformations).

With *helices* characterized by a corkscrew shape, *sheets* as parallel-connected segments, and *loops* as everything else, the above structure types are simplified as follows:

$$G, H, I \Rightarrow H; E, B \Rightarrow E; T, C, S \Rightarrow L$$

Secondary Structure Estimation

The goal of this objective is to distinguish sequences by assigning better score to sequences that may match the reference secondary structure better. Using the tool PROFphd, updated to ReProf [26], the likely secondary structure type $T_{\text{pred}}(i)$ can be predicted per amino acid aa_i in A with a reliability $R_{\text{pred}}(i) \in \{0 \ldots 9\}$ by means of posterior neural network training. With $T_{\text{ref}}(i)$ the actual type found at position i of the reference secondary structure, the estimated similarity score $F_{\text{sec}}(A)$ is calculated as a sum of reliability weighted (mis)matches:

$$F_{\text{sec}}(A) = \frac{\Sigma_{\max} - \sum_{i=1}^{N} s_i \cdot \left(C^R_{\text{pred}} + R_{\text{pred}}(i) \right)}{\Sigma_{\max}}, F_{\text{sec}}(A) \in \{0 \ldots 2\}. \quad (1)$$

where

$$s_i = \begin{cases} 1 & \text{if } T_{\text{pred}}(i) = T_{ref}(i) \\ -1 & \text{if } T_{\text{pred}}(i) \neq T_{ref}(i) \end{cases}$$

and

$$\Sigma_{\max} = \left(C^R_{\text{pred}} + \max R_{\text{pred}}\right) \cdot N$$

Equation 1 is normalized by the maximum possible sum, $\Sigma_{\max}$, which may occur if all positions are perfectly matched with the highest possible probability. In this case the score becomes 0 and it can never become negative. C^R_{pred} is a constant which purpose is to increase the contribution to the score of a matching type prediction that has a low reliability R_{pred}. In the current work, it was chosen such that $C^R_{\text{pred}} + \max R_{\text{pred}} = 20$. The reference types $T_{\text{ref}}(i)$ are extracted from the reference structure S_{ref} per residue position i as described in section "Secondary Structure Definition".

Diversity Measure

As a requirement stated in the problem description, the algorithm should not only find a single very good solution, but rather a number of good solutions as different as possible from each other and from the reference sequence. This diversity requirement is closely related to the models described in ▸ Chap. 40, "Diversity Optimization Models." However, as the problem solutions in this work represent protein sequences, not binary selection of elements, a slightly different approach to the distance measure is taken. An effective and simple measure of distance between two sequences is the Hamming distance, defined as the number of single-point permutations necessary to convert one into the other. Not taking gaps or varying sequence lengths into account, for two sequences $A = \{aa_i\}$ and $A' = \left\{aa'_i\right\}$ where $1 \leq i \leq N$, the Hamming distance between them is defined as:

$$d_{\text{Hamm}}\left(A, A'\right) = \sum_{i=1}^{N} d_i, \quad d_i = \begin{cases} 0 & \text{if } aa_i = aa'_i \\ 1 & \text{otherwise} \end{cases}. \tag{2}$$

To obtain a *nonnegative* objective value for minimization, the average Hamming distance to all other $M - 1$ individuals in the current population minus the sequence length N is computed:

$$F_{\text{div}}(A) = N - \frac{1}{M-1} \sum_{i=1}^{M-1} d_{\text{Hamm}}\left(A, A_i\right), F_{\text{div}}(A) \in \{0 \ldots N\}. \tag{3}$$

This function favors individuals farthest away from the rest of the population. In addition, if a sequence similar to the input sequence exists in the population, the function will have a mutually repulsive effect and penalize it. In summary the function addresses two problem requirements: (1) promoting diversity and (2) promoting sequences which are not equal to the reference sequence.

Algorithm Design

In this chapter the DAO-QC NSGA-II algorithm proposed to tackle the IFP is presented. The modification of the NSGA-II [10], a well-known multi-objective genetic algorithm, includes the diversity objective (DAO) $F_{\text{div}}(A)$ that enhances the explorative characteristic of the algorithm.

This favorable feature for solving *multimodal* problems is complemented by two modifications of the original algorithm highlighted in Algorithm 1: removal of doubles described in section "Removal of Doubles" and quantile constraint to promote good individuals in section "Quantile Constraint".

Algorithm 1: DAO-QC NSGA-II

1: $Initialize(P_0)$ {randomly generated individuals}
2: $t \leftarrow 0$
3: **while** $t < t_{\max}$ **do**
4: $\quad Q_t \leftarrow makeNewPop(P_t)$ {selection, mutation, recombination}
5: $\quad R_t \leftarrow P_t \cup Q_t$
6: $\quad mutateDoubles(R_t)$ {eliminate doubles by mutation}
7: $\quad F \leftarrow fastNonDominatedSort(R_t)$
8: $\quad P_t \leftarrow truncate(F)$ {based on domination and crowding}
9: $\quad setQuantileConstraint(P_t)$ {to penalize worst quantile}
10: **end while**

Removal of Doubles

In the context of diversity preservation, having two or more identical individuals in the population is undesired. Especially as in [30] when diversity for a sequence A is defined as the minimal distance to any other sequence A', a sequence $A = A'$ must be avoided. With the diversity calculation proposed in section "Diversity Measure", this issue has less impact, but nevertheless doubles are removed in this work. The procedure is executed in line 6 of Algorithm 1 after the application of genetic operations and before non-dominated sorting and crowding-based truncation of the unified population R_t takes place in NSGA-II.

When two identical sequences are detected, one of them is mutated with a probability of $\frac{5}{N}$ to distance the individual with a Hamming distance of 5 on average.

Quantile Constraint

A consequence of the nature of the objectives $F_{\text{sec}}(A)$ and $F_{\text{div}}(A)$ is that the latter is much easier to optimize; hence, the population quickly consists of very diversified individuals with poor fitness according to $F_{\text{sec}}(A)$. To counter this effect, the quantile constraint (QC) is introduced at the end of every generation, in line 9 of Algorithm 1. Given a quantile size C_q, the population P_t at time t is divided according to $F_{\text{sec}}(A)$ into a C_q%-sized partition and a $100 - C_q$%-sized partition. All individuals in the former, less fit, partition are assigned a constraint penalty that prevents the constrained individuals from mating and surviving the next generations. Hence, the population is cleaned from individuals far spread in the solution space, but with poor $F_{\text{sec}}(A)$ score. The selection pressure can then be selectively adjusted by changing the size of the quantile C_q, which has been tested using $C_q \in \{0\%, 5\%, 10\%, 25\%\}$.

Algorithm Experiments

This section presents and compares the experimental results obtained with the proposed DAO-QC MOGA to a standard generational GA on two protein samples. The experimental setup is first introduced, starting with the two protein samples used and followed by the algorithms' parameters. These initial experiments focus on analyzing the effect of different quantile constraint settings on the proposed algorithms' performance. To this end, the diversity, convergence, and final fitness are compared to a standard generational GA for both of the test samples.

Protein Samples

The two chosen protein samples, namely, *1OAI* and *1URR*, are illustrated in Fig. 4a, b, respectively. *1OAI* is characterized by a length of 59 residues and a secondary structure that consists of 4 helices. *1URR* is 97 residues long, and its secondary structure is composed of 2 helices and 6 beta-sheets.

Experimental Setup

Table 1 summarizes the settings of both the standard generational GA and the proposed DAO-QC MOGA, i.e., the GA extended by multi-objectivization with diversity as objective (DAO) and quantile constraint (QC). Both algorithms use a population of 100 individuals, a binary tournament selection, 1-point crossover with probability $p_c = 1.0$, and uniform mutation with probability $p_m = \frac{1}{N}$. The termination condition was set to 30,000 fitness function evaluations, and each experiment was repeated 30 times. Four different values of quantile constraint C_q are considered for DAO-QC NSGA-II: 0%, 5%, 10%, and 25% of the population.

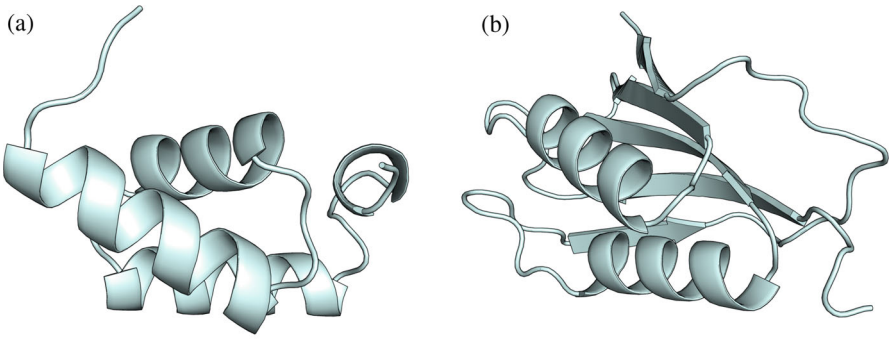


Fig. 4 Three-dimensional structure of the samples. (**a**) *1OAI*. (**b**) *1URR*

Table 1 Algorithm settings

Setting	Value
Population size	100
Algorithm	NSGA-II and std. GA
Termination condition	30,000 function evaluations
Selection	Binary tournament (BT)
Crossover operator	1-point, $p_c = 1.0$
Mutation operator	Uniform, $p_m = \frac{1}{N}$
Quantile constraint	$C_q \in \{0\%, 5\%, 10\%, 25\%\}$

Algorithm Results

In the following the results of the standard GA and the DAO-QC NSGA-II with four different C_q settings are presented and compared in terms of average population fitness, population diversity, and convergence of these indicators based on 30 individual runs.

Tables 2, 3, 4, and 5 show all pairwise comparisons of the algorithm mean value difference. The Wilcoxon test indicator [35] with a 5% significance level provides statistical confidence in comparing the sets with symbols "▲," "▽," and "-" indicating superior, inferior, and no difference. In terms of fitness, the algorithms are ordered in the following way: DAO-QC25 > DAO-QC10 ∼ GA > DAO-QC5 > DAO-QC0 for sample *1OAI* and DAO-QC25 > DAO-QC10 > GA > DAO-QC5 > DAO-QC0 for sample *1URR* with statistical confidence. In terms of diversity, the order becomes DAO-QC0 > DAO-QC5 > DAO-QC10 > DAO-QC25 > GA and DAO-QC0 > DAO-QC5 > DAO-QC10 > DAO-QC25 ∼ GA for samples *1OAI* and *1URR*, respectively. As expected, the higher diversity of the DAO-QC0 approach comes at the expense of a lower average fitness due to the exploration/exploitation trade-off. However, an increase of C_q to 10% or 25% leads to increased exploitation, allowing the DAO-QC NSGA-II algorithm to be constantly ahead of the GA in terms of average fitness until depletion of the evaluation budget as seen in Figs. 5 and 6. Further, the appropriate setting ($C_q = 25\%$ for *1OAI*, $C_q = 10\%$ for *1URR*) allows

Table 2 *1OAI* average fitness cross-comparison

	GA	DAO-QC0	DAO-QC5	DAO-QC10	DAO-QC25
GA	/	−0.101▽	−0.0272▽	0.000138 -	0.00896▲
DAO-QC0		/	0.0743▲	0.102▲	0.110▲
DAO-QC5			/	0.0273▲	0.0361▲
DAO-QC10				/	0.00882▲
DAO-QC25					/

Table 3 *1OAI* average diversity cross-comparison

	GA	DAO-QC0	DAO-QC5	DAO-QC10	DAO-QC25
GA	/	−46.996▽	−45.068▽	−36.946▽	−14.065▽
DAO-QC0		/	1.928▲	10.050▲	32.931▲
DAO-QC5			/	8.122▲	31.003▲
DAO-QC10				/	22.880▲
DAO-QC25					/

Table 4 *1URR* average fitness cross-comparison

	GA	DAO-QC0	DAO-QC5	DAO-QC10	DAO-QC25
GA	/	−0.117▽	−0.026▽	0.0229▲	0.0321▲
DAO-QC0		/	0.0909▲	0.140▲	0.149▲
DAO-QC5			/	0.0489▲	0.058▲
DAO-QC10				/	0.00911▲
DAO-QC25					/

Table 5 *1URR* average diversity cross-comparison

	GA	DAO-QC0	DAO-QC5	DAO-QC10	DAO-QC25
GA	/	−46.611▽	−42.754▽	−25.651▽	−1.389 -
DAO-QC0		/	3.857▲	20.959▲	45.221▲
DAO-QC5			/	17.102▲	41.365▲
DAO-QC10				/	24.262▲
DAO-QC25					/

the DAO-QC NSGA-II to outperform the GA in terms of fitness and diversity *at the same time*. Remaining observations to mention are steeper final fitness slopes for the sample *1URR* with settings $C_q \in \{10\%, 25\%\}$, than the standard GA and specifically for the sample *1OAI*; the diversity is observed to clearly start increasing once the fitness has converged. The steeper final slopes and the increased performance in fitness can be partially explained by the constantly high, and at times increasing, diversity combined with the highly *multimodal* nature of the problem. An elevated diversity clearly increases the chances of the algorithm discovering good new solutions in the rugged fitness landscape of this type of problem.

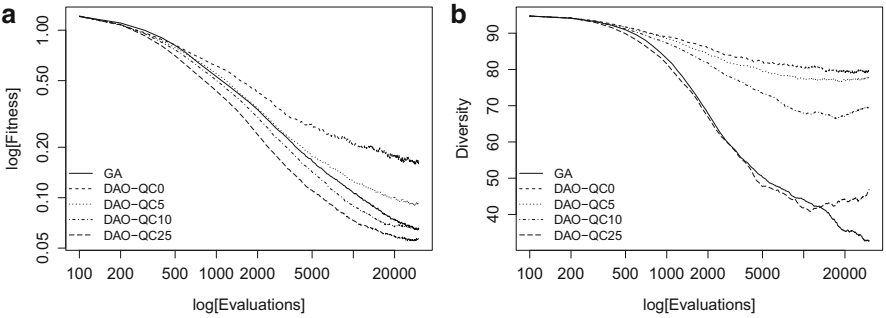


Fig. 5 Convergence of *1OAI*. (**a**) Average fitness convergence. (**b**) Average diversity convergence

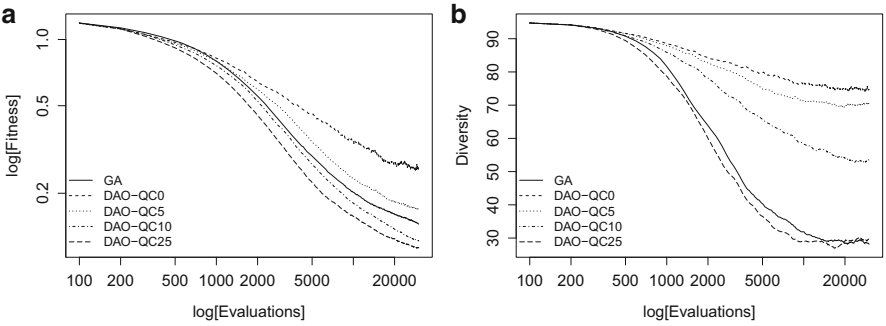


Fig. 6 Convergence of *1URR*. (**a**) Average fitness convergence. (**b**) Average diversity convergence

Table 6 Summary of final values

	1OAI		1URR	
	Average fitness	Average diversity	Average fitness	Average diversity
GA	$0.136_{\pm 1.52e-01}$	$43.578_{\pm 1.32e+01}$	$0.242_{\pm 1.77e-01}$	$35.565_{\pm 1.42e+01}$
DAO-QC0	$0.235_{\pm 1.41e-01}$	$80.992_{\pm 3.23e+00}$	$0.363_{\pm 1.62e-01}$	$77.163_{\pm 4.29e+00}$
DAO-QC5	$0.156_{\pm 1.45e-01}$	$78.598_{\pm 3.69e+00}$	$0.271_{\pm 1.80e-01}$	$72.763_{\pm 5.74e+00}$
DAO-QC10	$0.124_{\pm 1.45e-01}$	$70.564_{\pm 6.56e+00}$	$0.218_{\pm 1.75e-01}$	$59.400_{\pm 1.00e+01}$
DAO-QC25	$0.105_{\pm 1.37e-01}$	$47.484_{\pm 1.11e+01}$	$0.193_{\pm 1.65e-01}$	$34.182_{\pm 1.39e+01}$

Table 6 shows the final average fitness and diversity values of all algorithms on both samples with their respective standard deviation. In each column the darker background emphasizes the best result, while the lighter background emphasizes the worst result. With $C_q = 25\%$ the proposed algorithm clearly outperforms the GA with statistical confidence for both samples with average values 0.105 vs. 0.136 and 0.193 vs. 0.242, respectively. From the figure and the table, it is also evident that the

value of the quantile or QC setting has a direct impact on the population diversity, providing an effective tool for achieving the level of exploitation vs. exploration preferred.

Structure Validation

In this second experimental step, the protein sequences generated by the best performing algorithm are validated. To this end the I-TASSER [37] prediction tool is used to generate their secondary and tertiary structures that will be compared to the structure of the targeted protein. For each sample, the 5 best generated sequences of the final population in each of the 30 individual runs are selected. This means a total of 300 I-TASSER runs for the 2 protein samples, each run taking around 2 days, which amounts to almost 2 years of CPU time. It is to be noted that the I-TASSER prediction itself is subject to erroneous results; hence, a 100% certainty can never be achieved unless the proteins are synthesized in a wet lab. In the following, the sequences and their I-TASSER predictions are analyzed in terms of primary and secondary structure in section "Primary and Secondary Structure Validation Results" and then tertiary structure in section "Tertiary Structure Validation Results".

Primary and Secondary Structure Validation Results

The goal in this section is to analyze how well the secondary structure of the reference protein is reproduced in the predicted model.

Table 7 shows a summary of the two proteins tested. Clearly, the generated sequences share very little resemblance with the original input sequence seen from a sequence identity of about 20% and 15%, respectively, with a very low deviation. Achieving low sequence identity by itself is not a challenging task unless a good structure match is obtained at the same time. The table shows this as the average percentage, μ, of positions in the secondary annotation of the I-TASSER predicted model that correctly matches those of the input annotation. Average percentage μ and standard deviation of the average percentage σ are given for each of the three structure types *H*, *E*, and *L*. As it can be seen, the helices are correctly predicted on more than 90% of the positions in both proteins. For the slightly bigger *1URR* sample which contrary to *1OAI* contains many extended sheets, the sheet match percentage is lower—slightly below 50%.

Table 7 Summary of secondary structure prediction match

Protein	μ_{Identity}	σ_{Identity}	μ_{Helix}	σ_{Helix}	μ_{Sheet}	σ_{Sheet}	μ_{Loop}	σ_{Loop}
1OAI	20.67	4.100	93.348	6.343	0	0	82.814	7.368
1URR	15.23	3.225	93.787	6.563	42.108	8.898	85.523	8.239

Figures 7 and 8 illustrate the same data as histograms. Figures 7a and 8a clearly demonstrate that *helix* structures are very well matched in all 300 structure predictions. Almost all of the tested generated individuals have a match percentage of over 80%, and the majority is above 90% for both samples.

For *loop* segments presented in Figs. 7b and 8c, the majority is still above 80% but with a high spread. The statistics for *sheet* segments show that there is a limit to

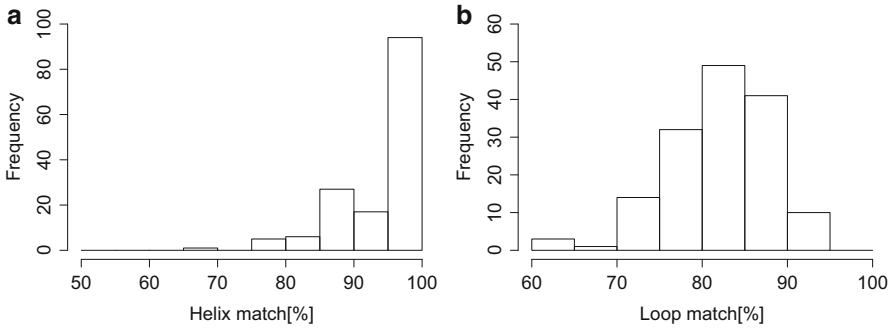


Fig. 7 Match histograms of *1OAI*. (**a**) *Helix* match histogram. (**b**) *Loop* match histogram.

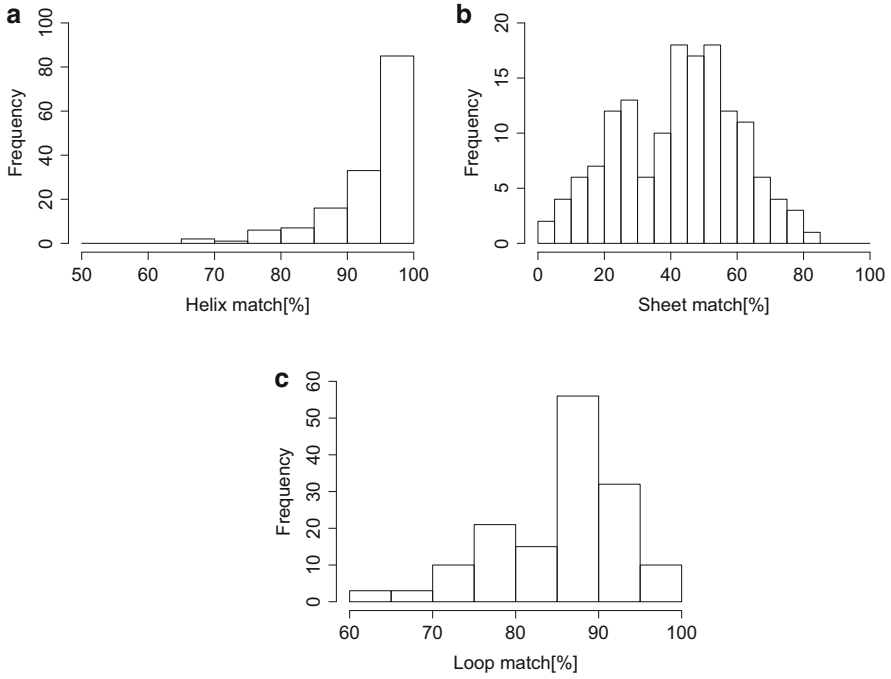


Fig. 8 Match histograms of *1URR*. (**a**) *Helix* match histogram. (**b**) *Sheet* match histogram. (**c**) *Loop* match histogram

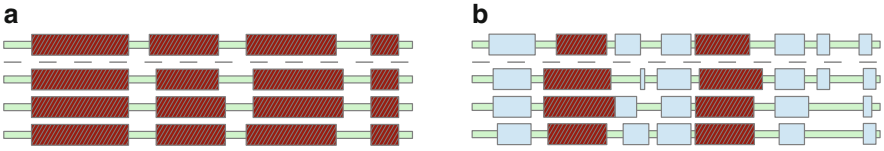


Fig. 9 Secondary structure of reference (on top) compared to three selected generated models. Darker sections are *helices*, lighter are *sheets*, and the rest represents *loop* structure. (**a**) *1OAI*. (**b**) *1URR*

the performance of an approach optimizing only an approximate secondary structure prediction. Considering that the *1URR* sample consists of *six sheet* segments across the whole of its length, then 42% can be considered as a rather good result. The lower success rate of predicting sheets is due to the fact that a sheet can only be observed in the secondary structure if the I-TASSER predicted structure actually did fold close enough to the reference tertiary structure, to allow the extended sheet to form. A *helix* is a much more local structure mostly independent of the global fold, hence easier to achieve in this analysis.

Figure 9 and Table 9 show the alignment of three of the best aligned individually generated sequences. This is to show specific examples of the results which have been averaged in Table 7, and the tendency remains the same: *helices* are very well defined with above 95% positions matched, *loops* slightly less with ±90%, and ±80% for samples *1OAI* and *1URR*, respectively. The *1OAI* sample is clearly an easier target due to its *helix*-only structure compared to the majority of *sheet* structures in the *1URR* sample. The other columns of the table will be discussed in the next section.

Tertiary Structure Validation Results

In the following the tertiary structure of the predicted proteins is validated by three-dimensional comparison.

The TM-Score detailed in [39] is a measure that is used to assess the similarity between two structures, with larger values indicating greater resemblance and 1.0 a maximum value for identical structures. According to Xu and Zhang [36], two proteins can be considered to be in the same fold if comparing them gives a TM-Score above 0.5. Though the average TM-Score is above 0.4 and close to 0.5 for the first sample, this is actually the case for $1{-}in{-}5$ for *1OAI* and $1{-}in{-}15$ for *1URR* as seen in Table 8. The table further shows the number N of predictions that had a TM-Score above 0.2, 0.4, 0.6, 0.7, and 0.8. The general results presented in section "Primary and Secondary Structure Validation Results" are confirmed here, and it is clear that the *sheet* structures of *1URR* are hard to match and that the approach is much more successful in predicting *helix* structures (see Table 9).

Table 8 Summary of tertiary structure prediction match

Protein	$\mu_{TM\text{-Score}}$	$\sigma_{TM\text{-Score}}$	$N_{TM>0.2}$	$N_{TM>0.4}$	$N_{TM>0.5}$	$N_{TM>0.6}$	$N_{TM>0.7}$	$N_{TM>0.8}$
1OAI	0.493	0.135	150	102	51	32	18	4
1URR	0.416	0.061	150	91	10	0	0	0

Table 9 Three selected generated models and their alignment scores with *1OAI* and *1URR* as reference

Nr.	Identity	N < 5*A*	$RMSD_{N<5A}$	RMSD	GDT_{TS}	TM-Score	Helix	Sheet	Loop
1	13.6	58	1.21	1.760	92.797	0.8667	95.12	0	94.44
2	25.4	58	1.35	1.838	88.983	0.8350	95.12	0	88.89
3	18.6	56	1.84	2.722	88.136	0.8015	97.56	0	94.44

Nr.	Identity	N < 5*A*	$RMSD_{N<5A}$	RMSD	GDT_{TS}	TM-Score	Helix	Sheet	Loop
1	19.6	73	2.85	7.484	50.258	0.5374	96	75.68	71.43
2	20.6	67	3.20	4.933	50.773	0.5027	100	81.08	80
3	17.5	74	2.94	9.059	48.711	0.5138	100	72.97	80

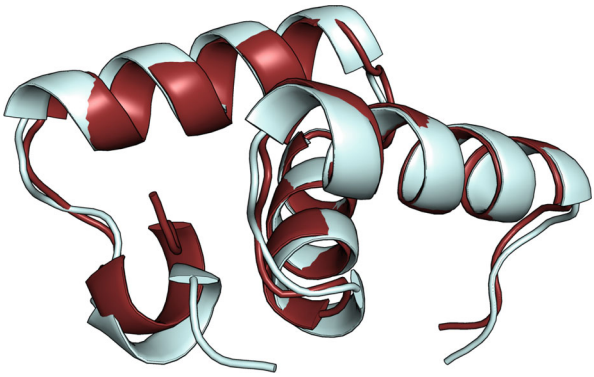


Fig. 10 Super-positioning of a predicted model (dark) with 1OAI reference (light)

The last step in the tertiary validation consists in superposing the fully I-TASSER predicted tertiary structure model of one generated sequence with the target reference. This is illustrated in Figs. 10 and 11 where the first of the three individually generated sequences in Table 9 and Fig. 9 is used.

The models for *1OAI* are all very close to the reference seen from the high *helix* and *loop* match percentage, and in addition the first model for *1OAI* has a very low sequence identity and at the same time very high *TM* and *GDT* scores (see Table 9). The first model for *1URR* also has very high *helix* match percentage and good *loop* and *sheet* percentages. However, the *TM* and *GDT* scores are less satisfactory. This result is visible in Fig. 11 where the *helices* and *sheets* cannot be fully aligned with the reference and the fact that one *sheet* has been bound to the structure in the wrong location (at the top of the figure rather than at the bottom).

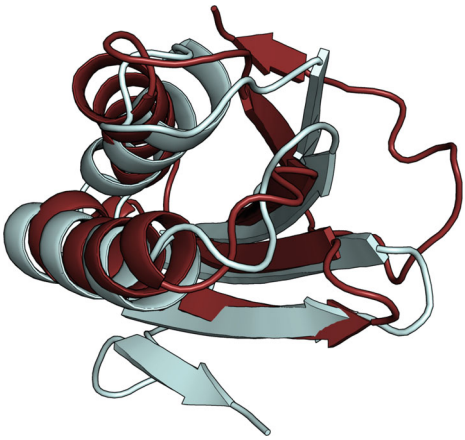


Fig. 11 Super-positioning of a predicted model (dark) with 1URR reference (light)

In Table 9 the second column shows sequence identity with gaps, the third shows the length of the longest continuous segment $N < 5A$ that can be fitted below a $5A$ threshold after super-positioning the two structures. The root-mean-square deviation (RMSD) measure is based on the pairwise distance between every residue position in the two tertiary structures, and the fourth column regards only those positions counted in column three, the fifth column regards the total of position. The global distance test (GDT) total score (TS) is a measure indicating the total average of the average percentage of residue positions that can be fitted below each of the thresholds $\{0.5A, 1.0A, 1.5A, \ldots 10.0A\}$. The final four columns are TM-Score and the percentage match of *helix*, *sheet*, and *loop* positions already discussed. Columns three to six were computed with the tertiary structure alignment tool LGA detailed in [38] with default global distance test (GDT) and longest continuous segment (LCS) analysis settings.

Conclusion

In this chapter an evolutionary-based approach to find a large amount of protein sequences that may result in a given reference secondary and tertiary structure was presented. This problem, referred to as the inverse folding problem (IFP), has received a lot of attention in theoretical chemistry and biophysics over the last 30 years, mostly for its potential application in protein design. It is also of interest to study the extent of the sequence space that may produce similar tertiary structures and how far from the original reference sequence such solutions can be found.

By defining the task as finding highly diverse sequences with most similar secondary structures, an optimization problem was modeled to find many well-scoring sequences in a few hours, which is fast compared to state-of-the-art methods. To achieve high diversity, the requirement has been adapted as an additional objective and extending the problem through *multi-objectivization* to become multi-objective

with diversity as objective (DAO). Combining the quantile constraint (QC) with the DAO approach allows to shift focus arbitrarily between diversity and fitness, and final results found significantly better than the standard GA with statistical significance. At the same time, the final diversity remains significantly higher for all QC-settings except the DAO-QC25 which produces diversity comparable to the standard GA for the *1URR* sample. For the *1OAI* sample with increased QC setting, a clear increase in diversity is observed toward the end of the run, once very good fitness values have been found. In addition to the higher performance on diversity, the algorithm fitness convergence was observed as being generally faster and partially steeper toward the end of runs, than for the standard GA.

For further validation, the five best generated sequences of each independent run of the $DAO - QC25$ algorithm variant were selected systematically and their folded structure predicted by I-TASSER, an established structure prediction software. The 300 predicted tertiary structures were annotated by DSSP for secondary structure analysis of *helix*, *sheet*, and *loop* formations. As could be expected, the method works better for the sample with more defined *helical* secondary structure, and less well in *sheet* and *loop* regions, especially as the latter region is not expressed by the objective function. Indeed *sheet* formations require the tertiary structure to fold properly to be captured in secondary structure. Nevertheless the *1URR* sample *sheet* match percentage is slightly below 50% averaged over all generated predictions. In addition the majority of match percentages are above 80% for *loops* and above 90% for *helices* in both samples. Tertiary structure validation was done by comparing the predicted structures to their respective reference by tertiary structure super-position. For both samples meaningful predictions were generated with a $TM-Score$ above 0.5 observed $1-in-5$ for *1OAI* and $1-in-15$ for *1URR*. These results indicate that this approach is able to generate a massive amount of sequences, with a reasonable amount being likely to actually fold as expected. At the same time, the limits in terms of achieving larger formations of *sheets* are demonstrated.

Future and ongoing works will address the identification of those sequences that actually fold into the reference structure by designing new objectives and constraints and also addressing *loop* and *beta-sheet* regions. Independent of this, sequences found could already be used as starting points for other exact protein design methods and possibly generate successful designs with a very low sequence identity compared to the reference. Additional possible applications could be generating meaningful decoy sets for other studies or finding bridges in sequence space between known proteins of the same structural classes.

Cross-References

- ▸ Diversity Optimization Models
- ▸ Evolutionary Algorithms
- ▸ Genetic Algorithms
- ▸ Multiobjective Optimization

Acknowledgments Work was funded by the National Research Fund of Luxembourg (FNR) as part of the EVOPERF project at the University of Luxembourg with the AFR contract no. 1356145. Experiments were carried out using the HPC facility of the University of Luxembourg [31].

Competing Interest Declaration The author(s) has no competing interests to declare that are relevant to the content of this manuscript.

References

1. Alba E, Dorronsoro B (2005) The exploration/exploitation tradeoff in dynamic cellular genetic algorithms. IEEE Trans Evol Comput 9(2):126–142
2. Alberts B, Johnson A, Lewis J, Raff M, Roberts K, Walter P (2002) Molecular biology of the cell. Garland Science, New York
3. Bellows ML, Fung HK, Taylor MS, Floudas CA, Lopez de Victoria A, Morikis D (2010) New compstatin variants through two de novo protein design frameworks. Biophys J 98(10):2337–2346
4. Bellows ML, Taylor MS, Cole PA, Shen L, Siliciano RF, Fung HK, Floudas CA (2010) Discovery of entry inhibitors for HIV-1 via a new de novo protein design framework. Biophys J 99(10):3445–3453
5. Bowie JU, Lüthy R, Eisenberg D (1991) A method to identify protein sequences that fold into a known three-dimensional structure. Science (New York, NY) 253(5016):164–170
6. Brooks BR, Bruccoleri RE, Olafson BD, States DJ, Swaminathan S, Karplus M (1983) Charmm – a program for macromolecular energy, minimization, and dynamics calculations. J Comput Chem 4(2):187–217
7. Chen W, Brühlmann F, Richins RD, Mulchandani A (1999) Engineering of improved microbes and enzymes for bioremediation. Curr Opin Biotechnol 10(2):137–141
8. De Jong AK (1975) Analysis of the behavior of a class of genetic adaptive systems. PhD thesis, University of Michigan, Ann Arbor. Dissertation Abstracts International 36(10):5140B, University Microfilms Number 76–9381
9. Deb K, Saha A (2010) Finding multiple solutions for multimodal optimization problems using a multi-objective evolutionary approach. In: Proceedings of the 12th annual conference on genetic and evolutionary computation. ACM, pp 447–454
10. Deb K, Pratap A, Agarwal S, Meyarivan T (2002) A fast and elitist multiobjective genetic algorithm: NSGA-II. IEEE Trans Evol Comput 6(2):182–197
11. Drexler KE (1981) Molecular engineering: an approach to the development of general capabilities for molecular manipulation. Proc Natl Acad Sci 78(9):5275–5278
12. Fung HK, Floudas CA, Taylor MS, Zhang L, Morikis D (2008) Toward full-sequence de novo protein design with flexible templates for human beta-defensin-2. Biophys J 94(2):584–599
13. Goldberg DE, Richardson J (1987) Genetic algorithms with sharing for multimodal function optimization. In: Grefenstette JJ (ed) Genetic algorithms and their applications: proceedings of the second international conference on genetic algorithms. Lawrence Erlbaum, Hillsdale, pp 41–49
14. Gutte B, Däumigen M, Wittschieber E (1979) Design, synthesis and characterisation of a 34-residue polypeptide that interacts with nucleic acids. Nature 281(5733):650–655
15. Harbury PB, Plecs JJ, Tidor B, Alber T, Kim PS (1998) High-resolution protein design with backbone freedom. Science 282(5393):1462–1467
16. Isogai Y, Ota M, Fujisawa T, Izuno H, Mukai M, Nakamura H, Iizuka T, Nishikawa K (1999) Design and synthesis of a globin fold. Biochemistry 38(23):7431–7443
17. Jones DT (1994) De novo protein design using pairwise potentials and a genetic algorithm. Protein Sci 3:567–574
18. Kabsch W, Sander C (1983) Dictionary of protein secondary structure: pattern recognition of hydrogen-bonded and geometrical features. Biopolymers 22(12):2577–2637

19. Klein F, Mouquet H, Dosenovic P, Scheid JF, Scharf L, Nussenzweig CM (2013) Antibodies in HIV-1 vaccine development and therapy. Science (New York) 341(6151):1199–1204
20. Klepeis JL, Floudas CA, Morikis D, Tsokos CG, Lambris JD (2004) Design of peptide analogues with improved activity using a novel de novo protein design approach. Ind Eng Chem Res 43(14):3817–3826
21. Kuhlman B, Baker D (2000) Native protein sequences are close to optimal for their structures. Proc Natl Acad Sci 97(19):10383–10388
22. Laredo JLJ, Nielsen SS, Danoy G, Bouvry P, Fernandes CM (2014) Cooperative selection: improving tournament selection via altruism. Accepted for publication in EvoCOP14 – 14th European conference on evolutionary computation in combinatorial optimisation
23. Mitra P, Shultis D, Brender JR, Czajka J, Marsh D, Gray F, Cierpicki T, Zhang Y (2013) An evolution-based approach to de novo protein design and case study on mycobacterium tuberculosis. PLoS Comput Biol 9(10):e1003298
24. Pabo C (1983) Molecular technology. Designing proteins and peptides. Nature 301(5897):200
25. Ponder JW, Richards FM (1987) Tertiary templates for proteins: use of packing criteria in the enumeration of allowed sequences for different structural classes. J Mol Biol 193(4):775–791
26. Rost B, Sander C (1994) Combining evolutionary information and neural networks to predict protein secondary structure. Proteins 19(1):55–72
27. Shimodaira H (1997) DCGA: a diversity control oriented genetic algorithm. In: ICTAI, pp 367–374
28. Smadbeck J, Peterson MB, Khoury GA, Taylor MS, Floudas CA (2013) Protein wisdom: a workbench for in silico de novo design of biomolecules. J Vis Exp 77:50476
29. Su A, Mayo SL (1997) Coupling backbone flexibility and amino acid sequence selection in protein design. Protein Sci 6(8):1701–1707
30. Toffolo A, Benini E (2003) Genetic diversity as an objective in multi-objective evolutionary algorithms. Evol Comput 11(2):151–167
31. Varrette S, Bouvry P, Cartiaux H, Georgatos F (2014) Management of an academic HPC cluster: the UL experience. In: Proceedings of the 2014 international conference on high performance computing & simulation (HPCS 2014). Bologna
32. Voigt CA, Mayo SL, Arnold FH, Wang Z-G (2001) Computational method to reduce the search space for directed protein evolution. Proc Natl Acad Sci USA 98(7):3778–3783
33. Wernisch L, Hery S, Wodak S (2000) Automatic protein design with all atom force-fields by exact and heuristic optimization. J Mol Biol 301(3):713–736
34. Wessing S, Preuss M, Rudolph G (2013) Niching by multiobjectivization with neighbor information: trade-offs and benefits. In: 2013 IEEE congress on evolutionary computation (CEC), pp 103–110
35. Wilcoxon F (1945) Individual comparisons by ranking methods. Biom Bull 1(6):80–83
36. Xu J, Zhang Y (2010) How significant is a protein structure similarity with tm-score = 0.5? Bioinformatics 26(7):889–895
37. Yang J, Yan R, Roy A, Xu D, Poisson J, Zhang Y (2015) The i-TASSER suite: protein structure and function prediction. Nat Methods 12(1):7–8
38. Zemla A (2003) LGA: a method for finding 3D similarities in protein structures. Nucleic Acids Res 31(13):3370–3374
39. Zhang Y, Skolnick J (2004) Scoring function for automated assessment of protein structure template quality. Proteins Struct Funct Bioinf 57(4):702–710

Graph Layout Problems

42

Sergio Cavero, Eduardo G. Pardo, Abraham Duarte, and Rafael Martí

Contents

Introduction 1322
The Cutwidth Minimization Problem 1325
The Minimum Linear Arrangement Problem 1329
The Vertex Separation Problem 1333
The SumCut/Profile Problem 1336
The Bandwidth Minimization Problem 1338
Conclusions and Future Lines of Research 1342
Cross-References 1343
References 1343

Abstract

The term layout problem was introduced in the context of Very Large-Scale Integration (VLSI) circuit design. Their main objective is to project an original graph onto a predefined host graph with a well-known topology, such as a path, cycle, or

S. Cavero
Departamento de Informática y Estadística, Universidad Rey Juan Carlos, Móstoles, Spain
e-mail: sergio.cavero@urjc.es

E. G. Pardo (✉)
Departamento de Informática y Estadística, Universidad Rey Juan Carlos, Madrid, Spain
e-mail: eduardo.pardo@urjc.es

A. Duarte
Department of Computer Science and Statistics, Universidad Rey Juan Carlos, Mostoles, Spain
e-mail: abraham.duarte@urjc.es

R. Martí
Operations Research and Statistics Department, School of Mathematics, University of Valencia, Valencia, Spain
e-mail: rafael.marti@uv.es

© Springer Nature Switzerland AG 2025

R. Martí et al. (eds.), *Handbook of Heuristics*,
https://doi.org/10.1007/978-3-032-00385-0_45

grid graph, among others. In this chapter, we review the five most relevant graph layout problems: the Cutwidth, the Minimum Linear Arrangement, the Vertex Separation, the SumCut, and the Bandwidth. Each problem is presented with its formal definition, and it is illustrated with a detailed example. Additionally, we describe their state-of-the-art heuristic methods and the instances used in their evaluation. Since graph layouts represent a challenge for optimization methods in general and for heuristics in particular, this review pays special attention to strategies and methodologies that provide high-quality solutions.

Keywords

Embedding · Graph layout · Heuristics · Cutwidth · Bandwidth · Optimization

Introduction

In recent years, there has been a growing interest in studying graph layout problems. From a historical perspective, the terms *layout* and *layout problem* come from their early application to the optimal layout of circuits in Very Large-Scale Integration (VLSI) design. A VLSI circuit can be modeled by means of a graph, where the edges represent the wires and the vertices represent the modules. Specifically, given a set of modules, we look for placing these modules on a board in both a nonoverlapping manner and wiring together the terminals on the different modules. In general, there are two stages: placement and routing. The former consists of placing the modules on a board, while the latter consists of wiring together the terminals on different modules that should be connected. Figure 1 presents an example of a circuit design with six modules (identified with a different letter, A–F) and their corresponding connections through different tracks. This circuit can be modeled as a graph, as is shown below. Notice that the order of the modules can determine, among others, the number of tracks needed to wire the circuit and, therefore, the space needed to build it.

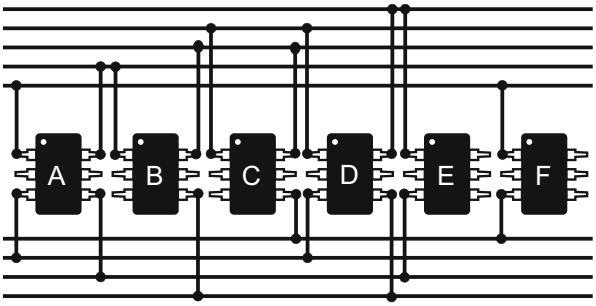


Fig. 1 Circuit design with six modules and its corresponding connections

From a theoretical point of view, the main objective of graph layout problems is to project an original graph $G = (V_G, E_G)$ onto a predefined host graph $H = (V_H, E_H)$. This projection, commonly known as embedding, consists of defining a function, mapping the vertices of G into the vertices of H, and associating a path in H for each edge of G. This function assesses that $V_G \subseteq V_H$ and $E_G \subseteq E_H$. According to [53], the quality of an embedding depends on the *dilation*, the *congestion*, and the *load*. The *dilation* of an embedding is the length of the largest associated path in H. The *congestion* is the largest number of paths that share an edge of H. Finally, its *load* is the maximum number of vertices of G that are mapped to a vertex of H.

Let G be a graph (finite, undirected, and without loops), where its set of vertices is denoted by V (with $|V| = n$) and its set of edges by E (with $|E| = m$). The notation (u, v) stands for an undirected edge between the vertices u and v. In Fig. 2, an example is shown of a general graph G obtained as the result of modeling the circuit in Fig. 1 as a graph. The graph is made up of six vertices corresponding to each of the modules in the circuit (say the vertex set $V = \{\texttt{A}, \texttt{B}, \texttt{C}, \texttt{D}, \texttt{E}, \texttt{F}\}$) and nine edges corresponding to the circuit connections (say the edge set $E = \{(\texttt{A}, \texttt{B}), (\texttt{A}, \texttt{D}), (\texttt{A}, \texttt{E}), (\texttt{A}, \texttt{F}), (\texttt{B}, \texttt{C}), (\texttt{B}, \texttt{D}), (\texttt{C}, \texttt{D}), (\texttt{C}, \texttt{F}), (\texttt{D}, \texttt{E})\}$). Notice that vertex A in Fig. 2 corresponds with module A in Fig. 1, vertex B in Fig. 2 corresponds with module B in Fig. 1, and so on.

The case in which a graph with n vertices is projected over a path graph with *load* equal to 1 is perhaps the simplest nontrivial embedding problem. In particular, it consists of defining a bijective function, ϕ, where each vertex and edge of G have a unique correspondence in H. The function $\phi : V \rightarrow \{1, \ldots, n\}$ assigns distinct integer numbers (from 1 to n) to each of the n vertices of G. Then, the resulting graph H is usually represented by aligning its vertices on a horizontal line, where each vertex u is located in position $\phi(u)$. This graphical representation is usually known as linear layout, although it has received other names in the literature, for example, linear ordering [4], linear arrangement [188], numbering [36], embedding in the line [153], or labeling [98]. In Fig. 3, an example of linear layout is shown where vertex A occupies the first position on the line, vertex B the second one, and so on. The horizontal dotted line represents the host graph structure. For the sake of simplicity, the linear layout is usually represented as an array $\phi = \{\texttt{A}, \texttt{B}, \texttt{C}, \texttt{D}, \texttt{E}, \texttt{F}\}$, where the position of each vertex $u \in V$ in this array corresponds to $\phi(u)$.

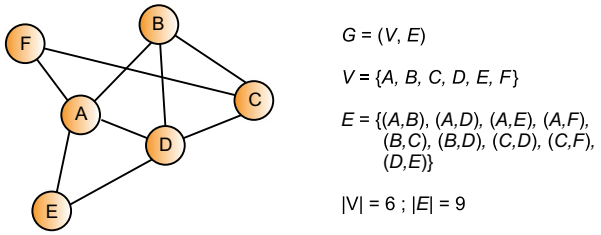


Fig. 2 The graph G with six vertices and nine edges represents the circuit in Fig. 1

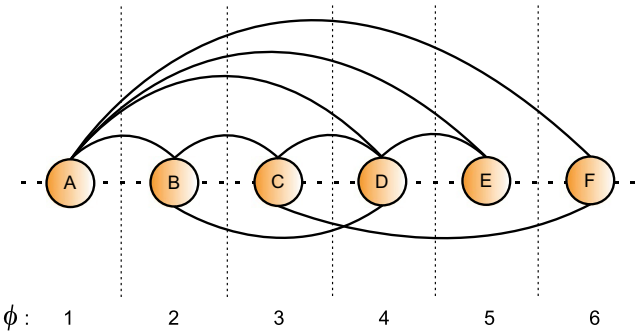
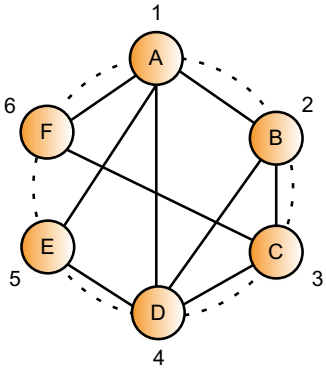


Fig. 3 A possible linear layout of graph G

Fig. 4 A possible circular layout of graph G



Linear layout is, probably, the most studied family of problems in the context of graph layout problems, as shown below. However, there exist other interesting embeddings in host graphs that are more general than a path, for instance, in closed paths, usually known as cycles [30, 129, 161, 191]. A circular layout is a function $\rho : V \rightarrow \{1, \ldots, n\}$ that also assigns distinct integer numbers (from 1 to n) to each of the n vertices of the graph, but arranging the vertices of the graph in a cycle where the last vertex (the one with label n) is next to the first vertex (the one with label 1). This embedding also has a *load* equal to 1. In Fig. 4, a possible circular layout of the vertices of the graph in Fig. 2 is shown, where the dotted circle represents the corresponding circular host graph structure.

The embedding of graphs in grids has been studied in [11, 125, 140]. When dealing with grids, it is convenient to label each vertex with a pair of coordinates. A grid layout is a function $\delta : V \rightarrow \{1, \ldots, n_1\} \times \{1, \ldots, n_2\}$ where n_1 denotes the maximum horizontal number, while n_2 indicates the maximum vertical number. Notice that in optimization problems where the *load* is equal to 1, it means that $n = n_1 * n_2$. In Fig. 5, an example of embedding the graph depicted in Fig. 2 on a 3×2 grid is shown. Again, the dotted lines represent the corresponding host graph structure.

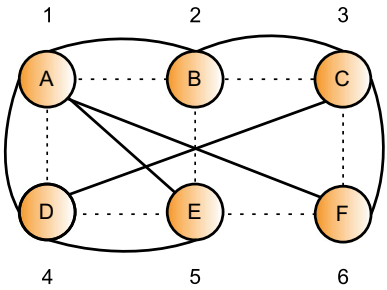


Fig. 5 A possible grid layout of graph G

There are also other embeddings of practical importance, where the host graph is neither a line nor a cycle nor a grid. Only to mention a few, in [5, 55, 92, 95], the embedding of graphs on trees is studied, in [97, 198] the embedding of graphs on torus, and in [112, 127] the embedding of graphs in hypercubes.

In this chapter, the attention is focused on minimization layout problems, by reviewing some of the most relevant optimization problems that fall into this category. We start by introducing the version of the problem onto a linear layout and then review the embedding on other types of graph. In particular, this review is focused on the problems that have been intensively studied by the heuristic community. See the following sections for each problem: "The Cutwidth Minimization Problem," "The Minimum Linear Arrangement Problem," "The Vertex Separation Problem," "The SumCut/Profile Problem," and "The Bandwidth Minimization Problem." Finally, in section "Conclusions and Future Lines of Research," we present the conclusions of this study and outline potential lines for future research.

The Cutwidth Minimization Problem

The Cutwidth Minimization Problem (CMP) is an NP-hard min-max linear layout problem [76]. It consists of finding an ordering of the vertices of a graph on a line, in such a way that the maximum number of edges between each pair of consecutive vertices is minimized. In terms of graph embedding theory, this problem consists of finding the linear layout (*load* equals to 1) that minimizes the *congestion* (see the definition above).

The classical combinatorial formulation can be described as follows. Given a graph $G = (V, E)$ with $|V| = n$, a linear layout $\phi : V \rightarrow \{1, 2, \ldots, n\}$ of G assigns the integers $\{1, 2, \ldots, n\}$ to the vertices in V, in such a way that each vertex receives a different label. Therefore, the host graph H is a path with the same number of vertices and edges. The cutwidth of a vertex v with respect to ϕ, denoted as $Cut(v, \phi, G)$, is the number of edges $(u, w) \in E$ satisfying $\phi(u) \leq \phi(v) < \phi(w)$. Therefore,

$$Cut(v, \phi, G) = |\{(u, w) \in E : \phi(u) \leq \phi(v) < \phi(w)\}|$$

The cutwidth of G with respect to ϕ is defined as the maximum value of all $Cut(v, \phi, G)$ for $v \in V$. More formally:

$$Cut(\phi, G) = \max_{v \in V} Cut(v, \phi, G)$$

The optimum cutwidth of G is then defined as the minimum $Cut(\phi, G)$ over all possible layouts Φ of G.

Figure 6a shows a possible linear layout ϕ of the graph G depicted in Fig. 2. This layout sets the vertices in a line, as it is commonly represented in the CMP. Specifically, since $\phi(\mathtt{A}) = 1$, vertex A comes first, followed by vertex B ($\phi(\mathtt{B}) = 2$), and so on. In Fig. 6b it is depicted the computation of the *Cut*-value for each vertex. For example, the *Cut*-value of vertex A is $Cut(\mathtt{A}, \phi, G) = 4$, because the edges (A, B), (A, D), (A, E), and (A, F) have an endpoint in A, labeled with 1, and the other endpoint is a vertex labeled with a value larger than 1. Similarly, the *Cut*-value of vertex C is $Cut(\mathtt{C}, \phi, G) = 6$, computed by counting the appropriate

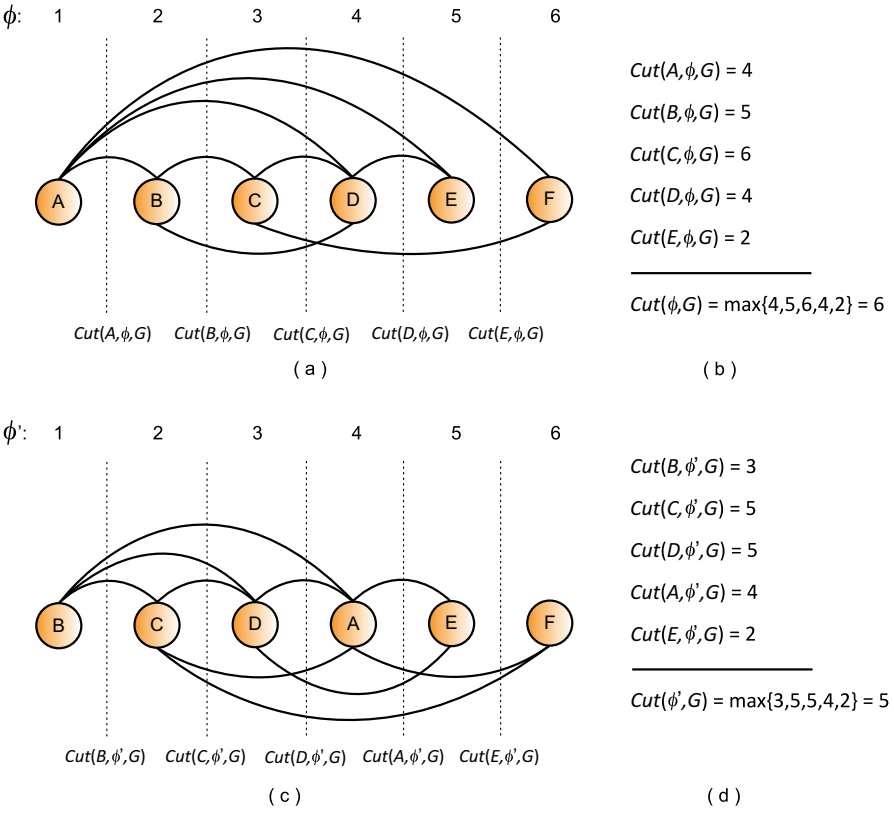


Fig. 6 (**a**) Example of linear layout ϕ. (**b**) Computation of *Cut*-values associated to ϕ. (**c**) Example of linear layout ϕ'. (**d**) Computation of *Cut*-values associated to ϕ'

number of edges (i.e., (A, D), (A, E), (A, F), (B, D), (C, D), and (C, F)). Then, since the cutwidth of G, $Cut(\phi, G)$, is the maximum of the $Cut(v, \phi, G)$ for all vertices $v \in V$, in this example, it is obtained that $Cut(\phi, G) = Cut(\mathtt{C}, \phi, G) = 6$.

In Fig. 6c, a different linear layout ϕ' of the graph G is shown. The associated *Cut*-values are depicted in Fig. 6d. Considering that the CMP is a minimization problem, this second linear layout ϕ' is better than ϕ since $Cut(\phi', G) = Cut(\mathtt{D}, \phi', G) = 5$, which is smaller than $Cut(\phi, G) = Cut(\mathtt{C}, \phi, G) = 6$.

The CMP has been referred to in the literature with different names, such as Minimum Cut Linear Arrangement [50, 192] or Network Migration Scheduling [7, 8]. There can also be found a generalization of the CMP for hyper-graphs named Board Permutation Problem [40, 41]. Several practical applications of this problem have been reported in different areas of engineering and computer science, such as circuit design [4, 41, 132], network reliability [101], information retrieval [22], automatic graph drawing [33, 146, 187], protein engineering [12], or network migration [165]. This last application is, as far as the authors know, the newest one. It refers to the problem where internodal traffic from an obsolete telecommunication network needs to be migrated to a new network. This problem appears nowadays in phone traffic, where it is performed a migration from 4ESS switch-based networks to IP router-based networks [165]. Nodes are migrated, one at each time period, from the old to the new network. All traffic originating or terminating at a given node in the old network is moved to a specific node in the new network, with the objective of minimizing the connections, at the same time, between the old and the new networks.

In the scientific literature, the CMP has been addressed from theoretical, exact, and heuristic approaches. From a theoretical perspective, researchers studying the CMP proposed lower and upper bounds, and also its connection with other related problems [9, 15, 38, 73, 111, 113, 118]. Most of the exact approaches present polynomial-time algorithms for particular graphs, such as hypercubes [87], trees [37, 194, 199], or grids [174], among others [89, 90]. There are also several exact methods for general graphs: integer linear programming formulations [45, 128, 130] and Branch and Bound algorithms [137, 147]. These approaches can only solve relatively small instances. Therefore, different heuristic procedures have been proposed to address this problem.

The work by Cohoon and Sahni [41] was the first approach to propose heuristic algorithms for the generalized version of the problem. These authors introduced several constructive methods and local search procedures, embedded into a Simulated Annealing metaheuristic [108]. Twenty years later, Andrade and Resende [7, 8] customized a Greedy Randomized Adaptive Search Procedure (GRASP) [69, 70] hybridized with a Path Relinking method [80]. Later, Pantrigo et al. in [149] introduced a Scatter Search algorithm [79, 117], which outperformed previous methods. More recently, Pardo et al. [151, 152] proposed several heuristics based on the Variable Neighborhood Search (VNS) methodology [142]. In particular, in [151], the authors applied several reduced VNS and basic VNS schemas to tackle the problem. Later, in [152], they applied a new variant of VNS, called Variable Formulation Search (VFS), to the CMP. This new algorithm is especially useful for

dealing with min-max (or max-min) optimization problems, where it is usual that many solutions have the same value of the objective function, which makes them especially hard for a heuristic search. VFS makes use of alternative formulations of the problem to determine which solution is more promising when they have the same value of the objective function in the original formulation.

Another interesting variant of VNS was presented by Duarte et al. in [59] by introducing different parallel designs of VNS to solve the CMP, which differs in the VNS stages to be parallelized as well as in the communication mechanisms among processes. Duarte et al. [58] investigate the potential of VNS to address multi-objective combinatorial optimization problems, with the CMP serving as a case study. Additionally, Fraire-Huacuja et al. [72] studied the implementation of a fuzzy logic controller, which is used to modify the parameters of a genetic algorithm during its execution to facilitate the exploration process. This method, which is referred to as a Fuzzy Genetic Algorithm (FGA), was evaluated and compared to a basic genetic algorithm that did not employ fuzzy logic. The authors tested their hypothesis using the CMP, and their findings indicated that the FGA exhibited marginal superiority over the corresponding genetic algorithm. However, when the VFS proposed in [152] was considered, the FGA demonstrated suboptimal performance and also had a longer run time.

In 2021, Santos et al. proposed a heuristic based on Adaptive Large Neighborhood Search that outperforms previous methods [183]. The algorithm consists of a greedy constructive procedure coupled with a swap local search including insert and remove operators to effectively explore the search space. Statistical tests proved that the proposed method exhibits a notable advantage in solution quality, when compared to state-of-the-art algorithms. Finally, in a recent publication, Cavero et al. [34] show that the correlation between the CMP and other similar problems (which are also introduced in this chapter) permits their simultaneous optimization of the others. The paper presents an analysis of various strategies based on VNS, which provides insights about the effectiveness of this methodology.

The literature on CMP is extensive and can be further enriched by including research studies with other host graphs. In particular, in recent years, embedding in a cycle graph has gained prominence among the community. For this specific case, CMP is known as the "Cyclic Cutwidth Minimization Problem" (CCMP). The CCMP has been studied both theoretically [1, 10, 26, 39, 66, 94, 96, 166, 184–186] and empirically [28, 29, 88, 93]. The cutwidth has also been studied for embedding specific graphs into trees, hypercubes, and grids. In this context, only theoretical studies have been published (such as [11, 14, 113]). These studies primarily focus on the structural properties and potential applications of such embeddings, providing a foundation for future practical implementations.

Regarding the instances used to evaluate this problem, most of the aforementioned authors have considered the instances reported in [149]. In particular, authors proposed the use of three sets of instances: *small*, *grid*, and *Harwell-Boeing*. The *small* data set consists of 84 graphs introduced in the context of the Bandwidth Reduction Problem [134]. The number of vertices of the graphs in this set ranges from 16 to 24. The exact value of the CMP for the instances in this set is known,

as they were solved to optimality in [137]. The *grid* data set consists of 81 matrices constructed as the Cartesian product of two paths [161], and they were originally introduced in the context of CMP by [174]. The size of the considered instances ranges from 9 vertices to 729. In this case, the optimal value is also known by construction [161]. Finally, the *Harwell-Boeing* data set is a subset of 87 instances extracted from the Harwell-Boeing sparse matrix collection which is accessible at the public domain matrix market library [42]. This collection was originally developed by Iain Duff, Roger Grimes, and John Lewis [61], and it consists of a set of standard test matrices arising from problems in linear systems, least squares, and eigenvalue calculations from a wide variety of scientific and engineering disciplines [62].

In the context of CMP, the instances have a number of vertices that range from 30 to 700. This data set is, in fact, the hardest for the algorithms, among the previously defined in the state of the art. A new set of instances, named as *Rome Graphs*, has recently been incorporated into the benchmark for this problem. This novel set comprises 11,534 instances, where the number of vertices ranges from 10 to 100. It was first introduced in [45] as a means of testing the efficacy of integer linear programming formulations within a linear ordering context. Compared to other sets, this one is relatively recent and has not been widely utilized.

The Minimum Linear Arrangement Problem

The Minimum Linear Arrangement Problem, usually known as MinLA, is also based on the *Cut*-value defined in section "The Cutwidth Minimization Problem." In this case, a linear layout with *load* equal to 1 is sought that minimizes the sum of the *congestion* on each position of the layout. It is equivalent to the minimization of the sum of *Cut*-values. More formally, given graph G and a linear layout ϕ, the linear arrangement value $LA(\phi, G)$ is defined as follows:

$$LA(\phi, G) = \sum_{v \in V} cut(v, \phi, G)$$

The optimum MinLA of G is then defined as the minimum $LA(\phi, G)$ in all possible layouts Φ of G.

Considering the linear layouts depicted in Fig. 6a, c it is shown in Fig. 7a, b the computation of the LA-values, respectively. In particular, the objective function value associated with the first layout ϕ is $LA(\phi, G) = 4 + 5 + 6 + 4 + 2 = 21$. Similarly, the objective function value associated with ϕ' is $LA(\phi', G) = 3 + 5 + 5 + 4 + 2 = 19$. Therefore, ϕ' is a better solution than ϕ since the MinLA is a minimization problem.

The Minimum Linear Arrangement Problem is an NP-hard problem [75], and it is related to two other well-known layout problems: the Bandwidth and Profile minimization problems. However, as pointed out in [139], an optimal solution for one of these problems is not necessarily optimal for the other related problems.

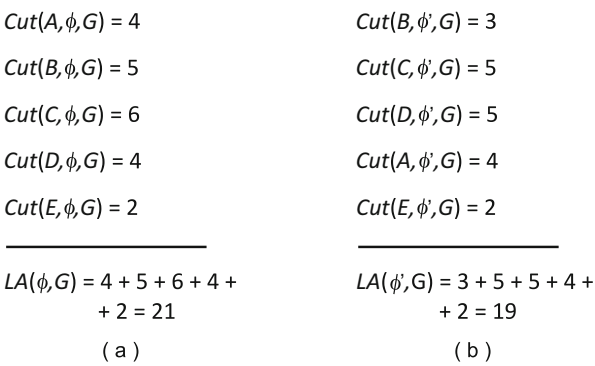


Fig. 7 (**a**) Computation of the objective function of the MinLA for the linear layout ϕ shown in Fig. 6a (**b**) Computation of the objective function of the MinLA for the linear layout ϕ' shown in Fig. 6c

The Minimum Linear Arrangement Problem is also known as the Optimal Linear Ordering, Edge Sum, Minimum-1-Sum, Bandwidth Sum, or Wire-Length problem. It was originally introduced in [86], with the goal of designing error-correcting codes with minimal average absolute errors on certain classes of graphs. Twenty years later, this problem was considered in [141] as an oversimplified model of some nervous activity in the cortex. The MinLA also has applications in single machine job scheduling [3, 162] and in graph drawing [187] among others [3, 35, 74, 102].

While MinLA is generally challenging (NP-hard in terms of its complexity), polynomial-time exact algorithms have been developed for specific graph types, including trees, rooted trees, hypercubes, meshes, and outerplanar graphs, among others. For a comprehensive overview, we refer the reader to [156] and [201]. Additionally, upper and lower bounds have been proposed for general and specific graphs [24, 25]. From an exact perspective, it is possible to find different mathematical formulations for small instances [6, 46].

Many different heuristic algorithms have been proposed for solving the MinLA problem. Juvan and Mohar [98] introduced the spectral sequencing method (SSQ). This method computes the eigenvectors of the Laplacian matrix of G. Then, it orders the vertices according to the second smallest eigenvector. As stated by Petit in [156], the rationale behind the SSQ heuristic is that vertices connected to an edge will tend to be assigned to numbers that are close to each other, thus providing a good solution to the MinLA problem. McAllister proposed [139] a heuristic method for the MinLA, which basically consists of a constructive procedure that labels the vertices in sequential order. Vertices are selected according to their degree with respect to previously labeled ones. This method compares favorably with previous methods for this and related problems. Later, Petit in [156] reviewed the existing lower bounds and heuristic methods, proposed new ones, and introduced a set of instances that were considered since then the standard for this problem. Among the methods reviewed in [156], it is possible to find the Juvan-Mohar method [98], the Gomory-Hu tree method [4], and the edge method, which the author improved by adding a degree method. Petit concluded that the Juvan-Mohar method and the degree method provide the best lower bounds; however, their values are far from

those of the best-known solutions, and therefore they are of very limited interest from a practical point of view.

In [156], Petit additionally introduced both constructive and local search procedures. The Successive Augmentation (SAG) is a greedy heuristic that constructs a step-by-step solution, by extending a partial layout until all vertices have been enumerated. At each step, the best free label is assigned to the current vertex. Vertices are examined in the order given by a breadth-first search. Once a solution has been constructed, three different heuristics, based on local search, are considered: hill climbing, full search, and Simulated Annealing (SA). In the hill-climbing method, moves are selected at random; in the full search, the entire neighborhood of a solution is examined at each iteration, in search of the best available move. Finally, the SA algorithm implements the temperature parameter as described in [108] for move selection. Petit, in [156], considered two neighborhoods, called *flip2* and *flip3*. The former exchanges the label of two vertices, while the latter "rotates" the label of three vertices. The experimentation in [156] shows that the neighborhood based on the exchange of two labels (*flip2*) produces better results than the rotation (*flip3*). Overall, the experimentation concludes that the SA method outperforms the others, although it employs much longer running times (not reported in the paper). Therefore, the author recommends employing the hill climbing as well as the spectral sequencing methods.

A more elaborate Simulated Annealing algorithm was proposed in [155]. The author introduced a new neighborhood, *flipN*, based on the normal distribution of the distances between the vertices labels. The Simulated Annealing algorithm based on the *flipN* neighborhood (SAN) improves upon the previous SA method described above. Moreover, to speed up the method, the initial solution is obtained with the SSQ algorithm. The combined method, SSQ + SAN, is able to outperform previous methods.

Later, Safro et al. [176] introduced a multilevel algorithm for the MinLA problem, employing weighted edge contractions to create a hierarchy of graphs. This approach significantly reduced the size of the problem while preserving its structure. In [167], Rodriguez-Tello et al. proposed a Memetic Algorithm [145] that exhibits competitive results on benchmark instances. Sharma and Srivastava [177] developed a hybrid evolutionary algorithm that integrates a genetic algorithm with simulated annealing, achieving improved solutions for various graph types, particularly for sparse graphs with a large number of vertices.

In [168], the authors proposed a new algorithm based on the Simulated Annealing methodology. The Two-Stage Simulated Annealing (TSSA) performs two steps. In the first one, a solution is constructed with the procedure described in [139]; then, in the second step, it performs a Simulated Annealing procedure based on exchanges of labels. This method introduces two new elements to solve the MinLA: a combined neighborhood and a new evaluation function. Given a vertex v, the first neighborhood, selected with a probability of 0.9, examines the vertices u such that their label $\phi(u)$ is close to the median of the labels of the vertices adjacent to v (at a maximum distance of 2). The second neighborhood, selected with a probability 0.1, exchanges the labels of two vertices selected at random with diversification

(exploration) purposes. The method evaluates the solutions with a function that is more discriminating than the original one. The authors compared their TSSA method with the best-known algorithms, and they concluded that their method outperforms the previous algorithms. This method has been the state of the art for many years.

More recent approaches include the one by Martí et al. [136] with an algorithm based on a Scatter Search with Path Relinking and the one proposed by Mladenovic et al. [143], with a Skewed General Variable Neighborhood Search (SGVNS), which is, as far as the authors know, the latest proposal for the problem. The method presented in [143] starts from a greedy-constructed solution as an initial point. The authors propose the use of a local search, based on four different neighborhood structures. These neighborhoods are explored with a Variable Neighborhood Descent (VND) procedure. Then, the VND, together with a shake procedure, based on random moves within the four neighborhoods defined, conforms a General VNS (GVNS) method. This method is finally extended by the use of several ideas from Skewed VNS where the sharp move acceptance criterion is relaxed. However, the procedures proposed in [136] and [143] were unable to outperform the results obtained by the TSSA presented in [168].

Researchers in this field have recently considered MinLA as a test case for new proposals in heuristic optimization, shifting their focus from solely attempting to surpass the state-of-the-art algorithms to exploring novel methodological approaches and applications. For instance, Glover and Rego [82] investigated new relationships for multi-neighborhood search in the context of MinLA, emphasizing theoretical insights rather than purely computational performance. Similarly, as previously introduced, Cavero et al. [34] considered MinLA, along with other graph layout problems, to showcase their variable formulation search approach.

Regarding the connections of MinLA with other graph layout problems, it is interesting to discuss the Cyclic Bandwidth Sum Problem (CBSP) which exhibits a strong correlation with it. Specifically, the CBSP can be conceptualized as the study of MinLA on a cycle host graph. This relationship is formally expressed by the inequality: $\text{MinLA}(G)/2 \leq \text{CBSP}(G) \leq \text{MinLA}(G)$. Although CBSP has not received as much attention as MinLA in the literature, the past decade has witnessed significant advancements in heuristic approaches to address it [31, 171, 172].

As mentioned above, there is a set of instances considered standard as the comparison framework in the literature for MinLA. In particular, this set was introduced by Petit et al. [156], and it consists of 21 graphs ($62 \leq n \leq 10{,}240$) being n the number of vertices of the graph. As the authors indicate in [156], choosing a particular class of graphs to serve as a benchmark for the MinLA problem is difficult, because there are no large real instances for this problem. The purpose of the authors, when they formed the data set, was to measure heuristics in sparse graphs, so, therefore, they selected different kinds of graphs belonging to the following families: general random graphs, geometric random graphs, graphs with known optima (trees, hypercubes, meshes), graphs from finite element discretizations, graphs from VLSI design, and graphs from graph drawing competitions.

The Vertex Separation Problem

The Vertex Separation Problem (VSP) was originally motivated by the idea of finding good separators for graphs [126].

In particular, the VSP consists of finding a linear layout (*load* equals to 1) where the separator is minimized. In mathematical terms, being p a position in the layout, the set $L(p, \phi, G) = \{u \in V : \phi(u) \leq p\}$ is defined, which represents the set of vertices in the linear layout placed in a position lower than or equal to p. Symmetrically, the set $R(p, \phi, G) = \{u \in V : \phi(u) > p\}$ contains those vertices placed in a position greater than p. In general, $L(p, \phi, G)$ can simply be referred to as the set of *left vertices* with respect to position p in ϕ, and $R(p, \phi, G)$ is the set of the *right vertices* with respect to p in ϕ. The separation value at position p of the layout ϕ, $Sep(p, \phi, G)$, is defined as the number of vertices in $L(p, \phi, G)$ with one or more adjacent vertices in $R(p, \phi, G)$. More formally:

$$Sep(p, \phi, G) = |\{u \in L(p, \phi, G) : \exists\, v \in R(p, \phi, G) \wedge (u, v) \in E\}|$$

The vertex separation (VS) value of a layout ϕ is the maximum of the *Sep*-values among all positions in ϕ:

$$VS(\phi, G) = \max_{1 \leq p < n} Sep(p, \phi, G)$$

The Vertex Separation Problem (VSP) then consists of finding a layout, $\phi^\star$, that minimizes the *VS*-value of the graph G. In mathematical terms:

$$\phi^\star = \arg\min_{\phi \in \Phi} VS(\phi, G)$$

where Φ represents the set of all possible linear layouts of G.

Considering the graph example introduced in Fig. 2 and the layout ϕ' shown in Fig. 6c, in Fig. 8a–e, the *Sep*-value of each position p in the layout ϕ' is depicted, denoted as $Sep(p, \phi', G)$. For instance, $Sep(1, \phi', G) = 1$ (see Fig. 8a) because $L(1, \phi', G) = \{\texttt{B}\}$ and $R(1, \phi', G) = \{\texttt{C}, \texttt{D}, \texttt{A}, \texttt{E}, \texttt{F}\}$, and there is only one vertex in L having an adjacent vertex in R. Similarly, $Sep(4, \phi', G) = 3$ (see Fig. 8d) because $L(4, \phi', G) = \{\texttt{B}, \texttt{C}, \texttt{D}, \texttt{A}\}$ and $R(4, \phi', G) = \{\texttt{E}, \texttt{F}\}$, and the vertices C, D, and A in L have an adjacent vertex in R. Notice that the vertex B does not affect the *Sep*-value, since it does not have adjacent vertices in R. The objective function is then computed as the maximum of these *Sep*-values. In particular, this value is $VS(G, \phi') = 3$, associated with positions 3 and 4. For the sake of completeness, all the *Sep*-values as well as the objective function value are shown in Fig. 8f.

As in the previous optimization problems defined above, a second example of the layout ϕ'' (see Fig. 9a, b) is included. The value of the objective function of this solution is 4.

The VSP is strongly related to other well-known graph problems, such as the Path Width [107], the Node Search Number [110], or the Interval Thickness [109],

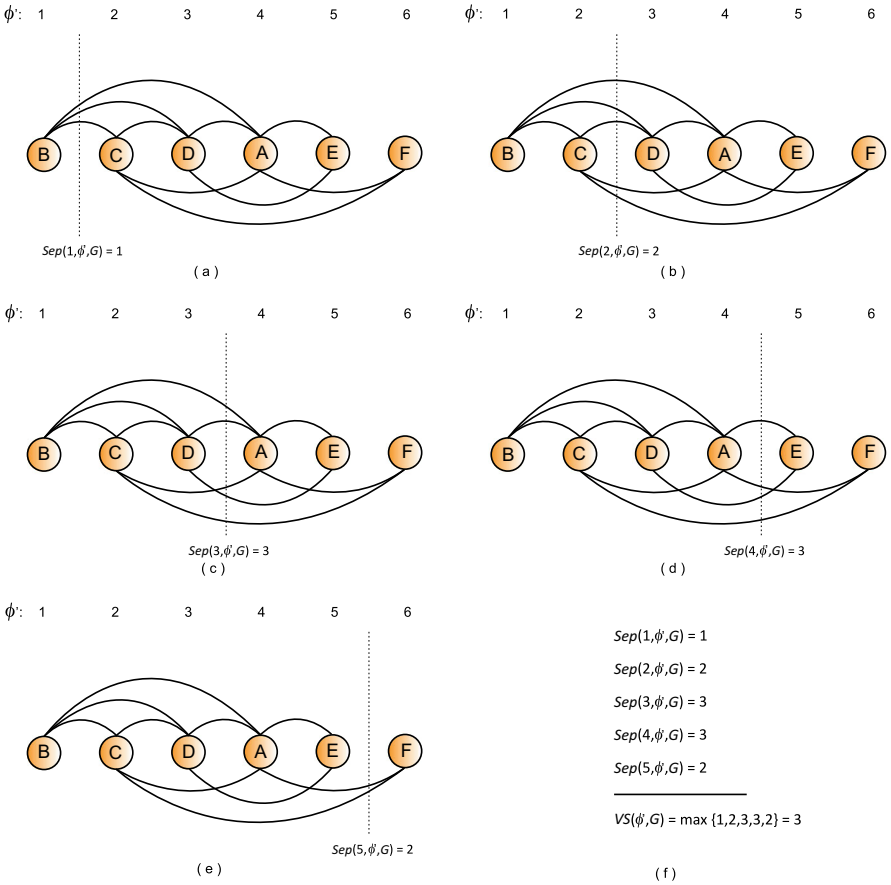


Fig. 8 (**a**)–(**e**) Computation of $Sep(p, \phi', G)$, with $p = \{1, \ldots, 5\}$, for the layout $\phi' = \{\mathtt{B}, \mathtt{C}, \mathtt{D}, \mathtt{A}, \mathtt{E}, \mathtt{F}\}$ introduced in Fig. 6c. (**f**) Computation of $VS(\phi', G)$

among others. The description of the equivalences among these problems can be found in [68, 107, 110]. All these optimization problems are NP-hard and have practical applications in VLSI design [119], computer language compiler design [17], or graph drawing [63].

It is possible to find efficient exact approaches to solve the VSP on special classes of graphs. A linear algorithm to compute the optimal vertex separation of a tree is proposed in [64] as well as an $O(n \log n)$ algorithm to find the corresponding optimal layout. The algorithm was improved in [189] with a linear-time procedure to find the optimal layout. In [154], an alternative method to compute the vertex separation of trees was also proposed, and in [65] an $O(n \log n)$ algorithm computes the vertex separation of unicyclic graphs (i.e., trees with an extra edge). A polynomial-time algorithm to compute the Path Width (which is identical to VSP) is proposed in [19]. However, the algorithm cannot be considered from a

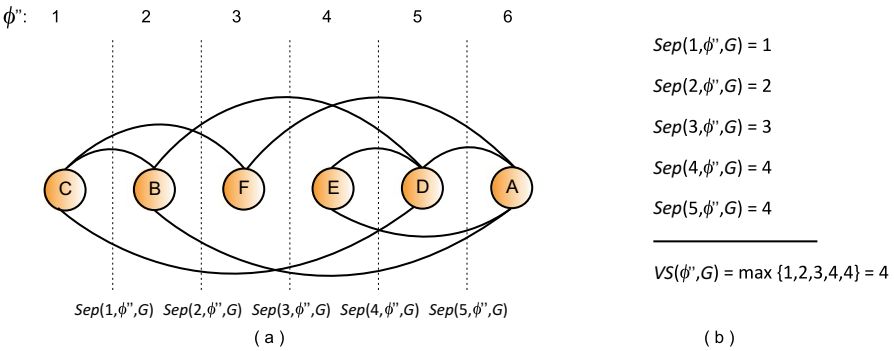


Fig. 9 (**a**) Computation of $Sep(p, \phi'', G)$ with $p = \{1, \ldots, 5\}$, for the layout $\phi'' = \{\mathtt{C}, \mathtt{B}, \mathtt{F}, \mathtt{E}, \mathtt{D}, \mathtt{A}\}$. (**b**) Computation of $VS(\phi'', G)$

practical point of view, since the bound on its time complexity is $\Omega(n)$, see [65]. In [21] a polynomial-time algorithm is proposed to optimally solve the VSP for n-dimensional grids. Co-graphs and permutational graphs can also be optimally solved as it was stated in [18–20]. More recently, Mallach [133] proposed MIP formulations based on linear ordering for this problem, providing new insights into exact solution approaches and improving the results obtained in [13, 190].

Approximation algorithms have also been proposed for the VSP together with complex metaheuristics based on Variable Neighborhood Search. Specifically, [16] proposes an $O(\log^2 n)$-approximation algorithm for general graphs and a $O(\log n)$-approximation algorithm for planar graphs. Similar results for binomial random graphs are presented in [53]. Castillo-García et al. [27] conducted a comparative study on constructive heuristics for the VSP, providing insights into their performance. Later, Duarte et al. [57] proposed a Basic VNS for the VSP, introducing two constructive procedures and a local search based on interchange moves, which showed promising results. Building upon this work, Huacuja et al. [71] explored the optimization of VSP using genetic algorithms, thus expanding the range of metaheuristic approaches applied to the problem. More recently, Sánchez-Oro et al. [182] proposed three VNS algorithms: Reduced VNS (RVNS), VND, and General VNS (GVNS). RVNS mainly focuses on the diversification, VND mainly focuses on the intensification, and GVNS balances intensification and diversification. Computational experiments reveal that the best procedure (GVNS) improves the state of the art in both quality and computing time. These results were validated using nonparametric statistical tests, establishing GVNS as the current state-of-the-art heuristic for the VSP.

In terms of the instances used to evaluate these heuristic algorithms, the first set of instances was defined in [57]. The authors selected 173 instances divided in three different subsets as a test bed. In particular, two of the subsets were formed by instances with a defined structure (trees and grids). For those instances, the optimum value is known by definition for the VSP. Additionally, they also selected

an additional subset of random graphs, whose instances have previously been used in the literature to evaluate other linear layout optimization problems. The grid subset consists of 50 matrices constructed as the Cartesian product of two paths. The number of vertices of this subset ranges from 25 to 2916. The tree subset is made up of 50 instances where the number of vertices ranges from 22 to 202. Finally, the random graph subset was formed by 73 instances derived from the Harwell-Boeing sparse matrix collection [42, 61, 62] introduced at the end of section "The Cutwidth Minimization Problem." In this case, the size of the instances, in terms of the number of vertices, ranges from 24 to 960. The aforementioned set of instances has been used later in [178] and [182].

The SumCut/Profile Problem

Considering the separation value defined in the previous section, the SumCut of a graph G with respect to the linear layout ϕ, denoted as $SC(\phi, G)$, is calculated as the sum of Sep-values of each position. In mathematical terms:

$$SC(\phi, G) = \sum_{i=1}^{n} Sep(i, \phi, G)$$

The SumCut Minimization Problem then consists of minimizing the value of $SC(\phi, G)$ in all possible linear layouts Φ of G:

$$SC(G) = \min_{\phi \in \Phi} SC(\phi, G)$$

Considering the linear layouts ϕ' and ϕ'' depicted in Figs. 8a and 9a, respectively, in Fig. 10a, the computation of the SC-values associated to each layout is shown. In particular, the objective function value associated with the first layout ϕ' is $SC(\phi', G) = 1 + 2 + 3 + 3 + 2 = 11$. Similarly, the objective function value associated with ϕ'' is $SC(\phi'', G) = 1+2+3+4+5 = 14$. Therefore, ϕ' is a better solution than ϕ'' since the SumCut is a minimization problem.

The SumCut was proved to be NP-complete for co-bipartite graphs in [200] and for general graphs in [51, 84]. This optimization problem is equivalent to the Profile Minimization Problem, as stated in [162]. Specifically, the reverse solution of the SumCut corresponds to a solution of the Profile. Both optimization problems have been extensively studied. See, for instance, [52–54]. Practical applications of these problems appear in genetics. The goal of the Human Genome Project is to sequence the DNA of humans, as well as other species, with the goal of elucidating the genetic information contained therein. In order to construct a physical map of a large DNA molecule, it is necessary to extract clones from it. Then, a fingerprint of each clone is obtained. Finally, DNA molecule is reassembled, determining how the clones overlap among them. Each clone is a sequence of nucleotides drawn from the set {adenine, cytosine, guanine, thymine}, so the reassembly process consists of

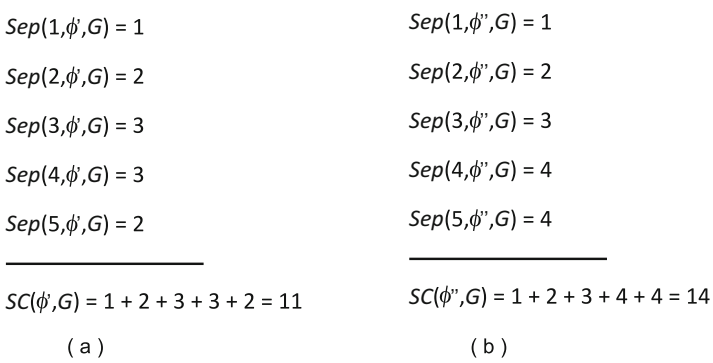


Fig. 10 (**a**) Computation of the objective function of SC for the linear layout shown in Fig. 8a. (**b**) Computation of the objective function of SC for the linear layout shown in Fig. 9a

permuting a linear layout of a graph. In [103] the author described an application in archeology, where it is necessary to serialize different artifacts (fossil, hardware, jewels, etc.). The serialization is known in archeology as "seriation" and consists of placing in chronological order different artifacts in the same culture using a relative dating method. Specifically, the practical application is based on the rearrangement of a matrix, which can be translated on the reordering of a linear layout of a graph.

Reducing the profile of a matrix is a relevant problem in mathematics since it leads to a reduction of the amount of space needed for some storage scheme. On the whole, it achieves an improvement of the performance of several operations such as Cholesky factorization of non-singular systems of equations [175]. Recently the profile reduction has been used in new areas like information retrieval to browse hypertext [22]. The Profile Minimization Problem was originally proposed as a way to reduce the storage space needed to save a sparse matrix [193], but it was shown to be equivalent to the SumCut Minimization Problem [162]. An important application of the Profile Minimization Problem arises in clone fingerprinting [102].

As far as the heuristics are concerned, Cuthill and McKee proposed in [43] the reverse Cuthill-McKee (RCM) algorithm, in order to get the minimum profile of a graph. In order to obtain a solution for the SumCut Minimization Problem, it is only needed to reverse the solution generated. Gibbs et al. [78] solved the SumCut using a new algorithm based on the RCM. The paper describes three problems of the RCM and presents a new algorithm that solves the described problems. Lewis described a method to reorder sparse matrices in order to reduce their profile [120], using the Gibbs-King algorithm to improve the results from the RCM algorithm. The previous algorithms were used not only in Profile and SumCut problems but also in the Bandwidth Minimization Problem, making a small adjustment in the last step: the numbering of the nodes. Lewis [121] presented a Simulated Annealing to reduce the profile of a matrix. The algorithm starts with a previously calculated solution, and it improves the solution by using the Simulated Annealing technique. The original solution is calculated using either the RCM algorithm or the Gibbs-

King algorithm, and the instances used are a subset of the Harwell-Boeing sparse matrix collection [42, 61, 62].

In [180] a new heuristic method based on the GRASP methodology is introduced. As it is shown in the computational experience, the proposed method is competitive with respect to previous approaches. This algorithm is further improved in [179] by coupling it with a Path Relinking post-optimization stage. Then, Sánchez-Oro et al. [181] proposed a new method based on the Scatter Search methodology. Among several mechanisms, this procedure includes Path Relinking as the basis for combining solutions to generate new ones. The most effective heuristic procedure identified in the existing literature is presented by Palubeckis [148] and implements a hybrid VNS with simulated annealing. In a recent contribution, Kardam and Srivastava presented a tabu-embedded simulated annealing algorithm to identify optimal solutions to specific graph classes, including trees of degree four and complete bipartite graphs [100]. This approach represents an advance over a previous research study published by the same authors [99].

In addition to heuristic approaches, some researchers have worked on identifying and proving the optimal profiles of specific classes of graphs. See, for example, [77, 116, 196, 197].

The current reference instances to evaluate the performance of the heuristic algorithms for this problem were proposed in [181] which is, as far as we know, the most complete data set of instances for the problem. In particular, they propose the use of the three different subsets of instances: Harwell-Boeing, bipartite, and tree. The Harwell-Boeing subset was formed by 73 sparse matrices whose number of vertices ranges from 24 to 960, extracted from the well-known collection Harwell-Boeing sparse matrix collection [42, 61, 62]. Notice that instances belonging to this family had been used before in [43] as a test bed for the problem. The bipartite subset is formed by 98 bipartite graphs with a number of vertices ranging from 4 to 142. A complete bipartite graph is such that the set of vertices V can be divided into two subsets V_1 and V_2 in such a way that there exists an edge between every pair of vertices belonging to different subsets. Finally, the tree subset consists of 91 instances based on trees with diameter 4 and a number of vertices ranging from 10 to 100. Optimal objective function values are known by construction for the instances in bipartite and tree subsets.

The Bandwidth Minimization Problem

The Bandwidth Minimization Problem (BMP), also known as Bandwidth Reduction Problem or Matrix Bandwidth Minimization, is an NP-hard min-max linear layout problem [150]. It consists of finding an ordering of the vertices of a graph on a line, in such a way that the largest distance between two adjacent vertices in the corresponding line is minimized. In terms of graph embedding theory, this problem consists of finding the linear layout (*load* equals to 1) that minimizes the *dilation*. In terms of circuit design, the BMP asks for a layout that minimizes the maximum edge length between each pair of connected modules.

Given a graph G and a linear layout ϕ, the bandwidth of a vertex v, $B(v, \phi, G)$, is the maximum of the differences between $\phi(v)$ and the labels of its adjacent vertices. That is:

$$B(v, \phi, G) = \max_{(u,v)\in E} \{|\phi(v) - \phi(u)|\}$$

The bandwidth of a graph G with respect to a layout ϕ is then

$$B(\phi, G) = \max_{v\in V} B(v, \phi, G)$$

The optimal bandwidth of graph G is thus the minimum $B(\phi, G)$ value over all possible layouts Φ of G.

Considering the linear layouts ϕ and ϕ' depicted in Fig. 6a, c, respectively, in Fig. 11a, b, the computation of their bandwidth value is shown. In particular, the objective function value associated with the first layout ϕ is $B(\phi, G) = \max\{5, 2, 3, 3, 4, 5\} = 5$. Similarly, the objective function value associated with ϕ' is $B(\phi', G) = \max\{3, 3, 4, 2, 2, 4\} = 4$. Therefore, ϕ' is a better solution than ϕ since the BMP is a minimization problem.

This problem has also been defined in terms of matrices. In particular, given the incidence matrix of graph G, the problem consists of finding a permutation of the rows and columns of this matrix that keeps all the nonzero elements in a band that is as close as possible to the main diagonal. For that reason, it is possible to find references to this problem as the Matrix Bandwidth Minimization problem.

The main application of the BMP is to solve non-singular systems of linear algebraic equations. The preprocessing of the coefficient matrix to reduce its bandwidth results in substantial savings in the computational effort associated with

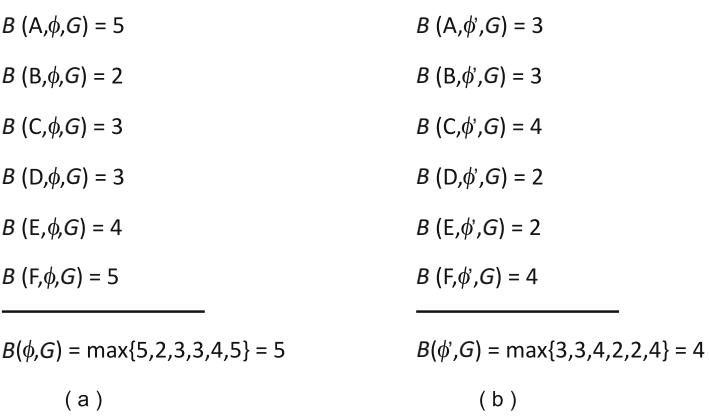


Fig. 11 (**a**) Computation of the BMP objective function for the linear layout ϕ shown in Fig. 6a. (**b**) Computation of the BMP objective function for the linear layout ϕ' shown in Fig. 6c

solving the system of equations. The context of these applications includes aircraft structures, liquid nitrogen gas tanks, propel blades, and submarines. In order to tackle the problem, Del Corso and Manzini in [44] proposed two exact algorithms to solve randomly generated graphs up to 100 nodes.

Similarly, Martí et al. proposed an exact method in [134] based on a branch and bound algorithm, together with several lower bounds, to compute the optimal solution for small- and medium-sized instances. However, the complexity of the problem makes that most of the efforts performed by researchers have been heuristic approaches. In this sense, for many years, researchers were only interested in designing relatively simple heuristic procedures and sacrificed the quality of the solution for speed. This is the case of the reverse method by Cuthill and McKee [43] and the GPS procedure by Gibbs et al. [78]. These two methods yield similar results in terms of solution quality; however, GPS is considerably faster, with an average speed that is about eight times faster than the reverse Cuthill-McKee procedure.

Metaheuristic algorithms have been widely and successfully applied to this problem for several decades. A Simulated Annealing procedure was introduced by Dueck and Jeffs in [60], which is based on an insertion mechanism and does not take advantage of the graph structure as the GPS method does. Martí et al. in [135] propose a Tabu Search method [81] for this problem, based on swap moves that exchange the labels of a pair of vertices and with a memory structure that proves to be remarkably effective. In 2004, Piñana et al. in [157] proposed an algorithm based on GRASP for the problem. In this proposal, the constructive step is based on GPS and the local search is based on exchanges. The GRASP method is coupled with a Path Relinking phase for improved outcomes. This algorithm clearly outperforms all the previous heuristic approaches. In 2006, Lim proposed several algorithms for the problem. In [122] they proposed an Ant Colony Optimization (ACO) [56] method with hill climbing. The same year, in [124], they introduced the node-shift heuristic which computes the desired label of each vertex according to the label of its adjacent vertices and then orders all the vertices in the graph with respect to these desired labels; finally, all the vertices are relabeled following this ordering. This innovative method is repeated until no vertex changes its label and it is coupled with a local search hill climbing. These authors also proposed a Genetic Algorithm [83,91] that generates the solutions in the initial population with a level structure procedure (as the GPS does) and implements a classic midpoint crossover as a combination operator.

In 2007, Lim et al. [123] proposed a Particle Swarm Optimization (PSO) [104] and a hill-climbing method. In 2008, Rodríguez-Tello et al. [169] presented a Simulated Annealing method based on a new neighborhood definition. Instead of swapping the labels of two vertices, these authors introduced a more elaborated move definition that leads to an efficient search, as shown in their computational results. Other notable approaches from this period include several strategies based on Variable Neighborhood Search proposed by Mladenovic et al. in [144], an adaptation of Tabu Search and Scatter Search by Campos et al. in [23] and a Genetic Algorithm proposed by Pop and Matei [158].

In 2011, Koohestani and Poli [114] introduced a hyper-heuristic approach for bandwidth reduction based on genetic programming, marking a shift toward more sophisticated metaheuristic techniques. Since then, there have been significant advances in metaheuristic approaches for this problem. Pop et al. [159] further improved their genetic algorithm proposed in [158], demonstrating its efficiency compared to other existing methods. Torres-Jimenez et al. [195] introduced a dual representation simulated annealing algorithm (DRSA) that showed superior performance in benchmark instances and which is considered, since then, the state of the art of the problem. Gonzaga de Oliveira et al. [85] conducted a comprehensive evaluation of low-cost heuristics for matrix bandwidth and profile reductions, identifying the most promising methods for various applications.

In recent years, there has been a trend toward hybridization. Mafteiu-Scai et al. [131] applied Brain Storm Optimization to the problem, while De Oliveira and Silva [47] proposed an ant colony hyper-heuristic approach for evolving reordering algorithms. Later, De Oliveira and Silva [48] combined low-cost heuristics with a Hill-Climbing strategy, demonstrating improved performance on matrices with symmetric sparsity patterns. In 2023, Koohestani [115] introduced a new metric to guide informed search methods, addressing limitations in previous approaches. Most recently, Abreu and Gonzaga de Oliveira [2] proposed a hybrid method combining Iterated Local Search and Tabu Search, while Maues et al. [138] introduced a multi-centrality heuristic for the Bandwidth Reduction Problem, showcasing the ongoing innovation in this field.

While the classical BMP focuses on linear layouts, there is growing interest in studying this problem where the embedding is performed into alternative host graph structures. Of particular significance are embeddings into cycles, grids, and trees. The Cyclic Bandwidth Problem (CBP), which considers embeddings into cycle graphs, has garnered attention with works such as Rodriguez-Tello et al. [170] proposing a tabu search approach, Ren et al. [163, 164] developing iterated local search algorithms, and more recently, Fertin et al. [67] combining preexisting knowledge with constraint programming techniques. Grid embeddings, explored in the Two-Dimensional Bandwidth Minimization Problem (TDBMP), have been addressed by Rodríguez-García et al. [173] with exact and heuristic approaches, Cavero et al. [32] proposing an efficient iterated greedy algorithm, and Khandelwal et al. [105, 106] exploring reduced variable neighborhood search and simulated annealing techniques. On the other hand, the study of graphs into trees has been mainly studied from a theoretical perspective, with seminal works by Bienstock [14] characterizing graphs that have small optimal overlap, Ostrovskii [49] investigating minimal congestion trees, and Rajasingh et al. [160] focusing on embedding cycles and wheels into arbitrary trees.

The data set of instances used as a reference to test algorithms for the BMP is again derived from the Harwell-Boeing sparse matrix collection [42, 61, 62]. Its application can be traced back to the first papers on heuristic approaches. In particular, the current data set is composed of 113 instances, and it is divided into two subsets. The first subset consists of 33 instances with a number of vertices ranging from 30 to 199, and the second subset includes 80 instances with a number

of vertices ranging from 200 to 1000. This data set was first introduced in [135], where the authors selected 126 instances, and then it was reduced to its current state in [157] by considering just 113 instances. The data set has been used in almost all the papers in the latter literature of the BMP.

Conclusions and Future Lines of Research

In this chapter, a family of graph layout problems is reviewed. Graph layout problems are optimization problems where the main objective is to project an original graph into a predefined host graph. In particular, five linear layout problems are reviewed, where the host graph is a path graph, and the aim is to minimize an objective function. This is probably the most studied family of problems in the heuristic literature within the context of graph layout problems. Specifically, this chapter reviews the Cutwidth Minimization Problem, the Minimum Linear Arrangement Problem, the Vertex Separation Problem, the SumCut Problem, and the Bandwidth Minimization Problem. Each problem is first defined and then illustrated with a detailed example. Additionally, the most relevant methods in the literature and the sets of instances used to evaluate them are also reviewed. The wide variety of heuristic algorithms sets a deep understanding of strategies to reach high-quality solutions to the minimization of linear layout problems. However, recent trends are pushing the boundaries of what is possible in graph layout optimization:

- Integration of exact algorithms with heuristics to find the optimal solution for larger graphs.
- Parallel algorithms enable the processing of significantly larger graphs by distributing computations.
- Machine learning techniques are being explored to learn effective layout strategies from the input data.
- Multi-objective optimization approaches are being developed to balance competing layout criteria.
- Techniques for dynamic graphs are emerging to efficiently update layouts as graphs evolve over time.
- Domain-specific heuristics are being tailored for applications in areas like social networks and bioinformatics.
- Interactive visualization systems are incorporating user feedback to guide the layout process.

These advancements are not only improving the scalability and efficiency of graph layout algorithms, but also expanding their applicability to more complex, real-world scenarios. In this sense, future research should also focus on extending current knowledge to embeddings graphs in nonlinear structures like grids, trees, cycles, or even general graphs.

Cross-References

- Network Optimization
- Scheduling Heuristics

Acknowledgments This research has been partially supported by grants: PID2024-160226OB-C21, PID2024-160226OB-C22, PID2021-125709OB-C21 and PID2021-125709OA-C22 and PID2021-126605NB-I00, funded by MCIN/AEI/10.13039/501100011033 and "ERDF A way of making Europe"; project CIAICO/2021/224, funded by Generalitat Valenciana; project M2988, funded by "Proyectos Impulso de la Universidad Rey Juan Carlos 2022"; the "Cátedra en Innovación y Digitalización Empresarial funded by Universidad Rey Juan Carlos and Second Episode" (Ref. MCA06); and "Red Española de optimización heurística 4.0 digitalización" (Ref. RED2022-134480-T).

Competing Interest Declaration The author(s) has no competing interests to declare that are relevant to the content of this manuscript.

References

1. Abbott HL (1966) Hamiltonian circuits and paths on the n-cube. Can Math Bull 9(5):557–562
2. Abreu AAAM, Gonzaga de Oliveira SL (2024) Iterated local search with Tabu search for the bandwidth reduction problem in graphs. In: Computational science and its applications—ICCSA 2024. Springer Nature, Switzerland, pp 125–136
3. Adolphson DL (1977) Single machine job sequencing with precedence constraints. SIAM J Comput 6:50–54
4. Adolphson DL, Hu TC (1973) Optimal linear ordering. SIAM J Appl Math 25(3):403–423
5. Álvarez C, Cases R, Díaz J, Petit J, Serna M (2000) Approximation and randomized algorithms in communication networks. In: Routing trees problems on random graphs. ICALP workshops 2000. Carleton Scientific, p 99111
6. Amaral ARS (2009) A mixed 0-1 linear programming formulation for the exact solution of the minimum linear arrangement problem. Optim Lett 3:513–520
7. Andrade DV, Resende MGC (2007) GRASP with evolutionary path-relinking. In: Seventh metaheuristics international conference (MIC), Montreal
8. Andrade DV, Resende MGC (2007) GRASP with path-relinking for network migration scheduling. In: Proceedings of international network optimization conference (INOC), Spa
9. Arora S, Frieze A, Kaplan H (2002) A new rounding procedure for the assignment problem with applications to dense graph arrangement problems. Math Program 92:1–36
10. Aschenbrenner R (2001) A proof for the cyclic cutwidth of Q5. California State University, Technical Report
11. Bezrukov SL, Chavez JD, Harper LH, öttger MR, Schroeder UP (2000) The congestion of n-cube layout on a rectangular grid. Discret Math 213:13–19
12. Blin G, Fertin G, Hermelin D, Vialette S (2008) Fixed-parameter algorithms for protein similarity search under mRNA structure constraints. J Discret Algorithms 6:618–626
13. Biedl TC, Bläsius T, Niedermann B, Nöllenburg M, Prutkin R, Rutter I Using ILP/SAT to determine pathwidth, visibility representations, and other grid-based graph drawings. CoRR abs/308.6778v2
14. Bienstock D (1990) On embedding graphs in trees. J Comb Theory Ser B 49(1):103–136
15. Bodlaender HL (1986) Classes of graphs with bounded tree-width. Technical Report RUU-CS-86-22, Utrecht University

16. Bodlaender HL, Gilbert JR, Hafsteinsson H, Kloks T (1995) Approximating treewidth, pathwidth, frontsize, and shortest elimination tree. J Algorithms 18(2):238–255
17. Bodlaender HL, Gustedt J, Telle JA (1998) Linear-time register allocation for a fixed number of registers. In: Proceedings of the ninth annual ACM-SIAM symposium on discrete algorithms (SODA'98). Society for Industrial and Applied Mathematics, Philadelphia, pp 574–583
18. Bodlaender HL, Kloks T, Kratsch D (1993) Treewidth and pathwidth of permutation graphs. In: Lingas A, Karlsson R, Carlsson S (eds) Automata, languages and programming. Lecture notes in computer science, vol 700. Springer, Berlin/Heidelberg, pp 114–125
19. Bodlaender HL, Kloks T, Kratsch D (1995) Treewidth and pathwidth of permutation graphs. SIAM J Discret Math 8(4):606–616
20. Bodlaender HL, Möhring RH (1990) The pathwidth and treewidth of cographs. SIAM J Discret Math 6(6):181–188
21. Bollobás B, Leader I (1991) Edge-isoperimetric inequalities in the grid. Combinatorica 11(4):299–314
22. Botafogo RA (1993) Cluster analysis for hypertext systems. In: 16th annual international ACM-SIGIR conference on research and development in information retrieval, Pittsburgh, pp 116–125
23. Campos V, Piñana E, Martí R (2011) Adaptive memory programming for matrix bandwidth minimization. Ann Oper Res 183(1):7–23
24. Caprara A, Letchford AN, Salazar-González J-J (2010) Lower bounds for the minimum linear arrangement of a graph. Electron Notes Discret Math 36:843–849
25. Caprara A, Letchford AN, Salazar-González J-J (2011) Decorous lower bounds for minimum linear arrangement. INFORMS J Comput 23(1):26–40
26. Castillo C (2003) A proof for the cyclic cutwidth of Q6. REU Project, Cal State University, San Bernardino
27. Castillo-García N, Huacuja HJF, Rangel RAP, Flores JAM, Barbosa JJG, Valadez JMC (2015) Comparative study on constructive heuristics for the Vertex Separation Problem. In: Design of intelligent systems based on fuzzy logic, neural networks and nature-inspired optimization. Springer International Publishing, pp 465–474
28. Cavero S, Pardo EG, Duarte A (2021) Influence of the alternative objective functions in the optimization of the cyclic cutwidth minimization problem. In: Conference of the Spanish association for artificial intelligence. Springer, pp 139–149
29. Cavero S, Pardo EG, Laguna M, Duarte A (2021) Multistart search for the cyclic cutwidth minimization problem. Comput Oper Res 126:105116
30. Cavero S, Pardo EG, Duarte A (2022) A general variable neighborhood search for the cyclic antibandwidth problem. Comput Optim Appl 81(2):657–687
31. Cavero S, Pardo EG, Duarte A, Rodriguez-Tello E (2022) A variable neighborhood search approach for cyclic bandwidth sum problem. Knowl-Based Syst 246:108680
32. Cavero S, Pardo EG, Duarte A (2023) Efficient iterated greedy for the two-dimensional bandwidth minimization problem. Eur J Oper Res 306(3):1126–1139
33. Cavero S, Pardo EG, Glover F, Martí R (2024) Strategic oscillation tabu search for improved hierarchical graph drawing. Expert Syst Appl 243:122668
34. Cavero S, Colmenar JM, Pardo EG (2024) A variable formulation search approach for three graph layout problems. In: Metaheuristics international conference. Springer, pp 390–396
35. Chen PS (1976) The entity-relationship model–toward a unified view of data. ACM Trans Database Syst 1:9–36
36. Chinn PZ, Chvátalová J, Dewdney AK, Gibbs NE (1982) The bandwidth problem for graphs and matrices—a survey. J Graph Theory 6(3):223–254
37. Chung MJ, Makedon F, Sudborough IH, Turner J (1982) Polynomial time algorithms for the min cut problem on degree restricted trees. In: Proceedings of the 23rd annual symposium on foundations of computer science (SFCS'82). IEEE Computer Society, Washington, DC, pp 262–271
38. Chung FRK, Seymour PD (1989) Graphs with small bandwidth and cutwidth. Discret Math 75(1–3):113–119

39. Clarke DW (2002) The cyclic cutwidth of mesh cubes. PhD Thesis, California State University. https://scholarworks.lib.csusb.edu/etd-project/2329
40. Cohoon J, Sahni S (1983) Exact algorithms for special cases of the board permutation problem. In: Proceedings of the Allerton conference on communication, control and computing, Monticello, pp 246–255
41. Cohoon J, Sahni S (1987) Heuristics for backplane ordering. J VLSI Comput Syst 2:37–61
42. Harwell-Boeing Sparse Matrix Collection. Public domain matrix market (2016). http://math.nist.gov/matrixmarket/data/harwell-boeing/
43. Cuthill E, McKee J (1969) Reducing the bandwidth of sparse symmetric matrices. In: Proceedings of the 1969 24th national conference. ACM, New York, pp 157–172
44. Corso GMD, Manzini G (1999) Finding exact solutions to the bandwidth minimization problem. Computing 62(3):189–203
45. Coudert D (2016) A note on integer linear programming formulations for linear ordering problems on graphs. PhD thesis, Inria; I3S; Universite Nice Sophia Antipolis; CNRS
46. de Andrade RC, e Bonates TO, Campêlo M, da Silva Ferreira M (2022) A compact quadratic model and linearizations for the minimum linear arrangement problem. Discret Appl Math 323:134–148
47. de Oliveira SLG, Silva LM (2020) Evolving reordering algorithms using an ant colony hyperheuristic approach for accelerating the convergence of the ICCG method. Eng Comput 36(4):1857–1873. Springer
48. de Oliveira SLG, Silva LM (2021) Low-cost heuristics for matrix bandwidth reduction combined with a Hill-Climbing strategy. RAIRO-Oper Res 55(4):2247–2264. EDP Sciences
49. Ostrovskii MI (2004) Minimal congestion trees. Discret Math 285(1–3):219–226
50. Díaz J, Gibbons A, Pantziou GE, Serna MJ, Spirakis PG, Torán J (1997) Parallel algorithms for the minimum cut and the minimum length tree layout problems. Theor Comput Sci 181:267–287
51. Díaz J, Gibbons A, Paterson M, Torán J (1991) The minsumcut problem. In: Dehne F, Sack J-R, Santoro N (eds) WADS. Lecture notes in computer science, vol 519. Springer, Berlin/Heidelberg, pp 65–89
52. Díaz J, Penrose MD, Petit J, Serna MJ (2000) Convergence theorems for some layout measures on random lattice and random geometric graphs. Comb Prob Comput 9:489–511
53. Díaz J, Petit J, Serna MJ (2002) A survey of graph layout problems. ACM Comput Surv (CSUR) 34(3):313–356
54. Díaz J, Petit J, Serna MJ, Trevisan L (1998) Approximating layout problems on random sparse graphs. Technical report LSI-98-44-R, Departament de Llenguatges i Sistemes Informàtics, Universitat Politècnica de Catalunya (Presented in the fifth Czech-Slovak international symposium on combinatorics, graph theory, algorithms and applications)
55. Ding G, Oporowski B (1995) Some results on tree decomposition of graphs. J Graph Theory 20(4):481–499
56. Dorigo M (1992) Optimization, learning and natural algorithms. PhD thesis, Politecnico di Milano, Italia
57. Duarte A, Escudero LF, Martí R, Mladenović N, Pantrigo JJ, Sánchez-Oro J (2012) Variable neighborhood search for the vertex separation problem. Comput Oper Res 39(12):3247–3255
58. Duarte A, Pantrigo JJ, Pardo EG, Mladenović N (2015) Multi-objective variable neighborhood search: an application to combinatorial optimization problems. J Glob Optim 63(3):515–536
59. Duarte A, Pantrigo JJ, Pardo EG, Sánchez-Oro J (2016) Parallel variable neighbourhood search strategies for the cutwidth minimization problem. IMA J Manag Math 27(1):55–73
60. Dueck G, Jeffs J (1995) A heuristic bandwidth reduction algorithm. J Comb Math Comput 18:97–108
61. Duff IS, Grimes RG, Lewis JG (1989) Sparse matrix test problems. ACM Trans Math Softw 15(1):1–14
62. Duff IS, Grimes RG, Lewis JG (1992) User's guide for the Harwell-Boeing sparse matrix collection (release I), Rutherford Appleton Laboratory, Chilton

63. Dujmovic V, Fellows MR, Kitching M, Liotta G, McCartin C, Nishimura N, Ragde P, Rosamond F, Whitesides S, Wood DR (2008) On the parameterized complexity of layered graph drawing. Algorithmica 52(2):267–292
64. Ellis JA, Sudborough IH, Turner JS (1994) The vertex separation and search number of a graph. Inf Comput 113(1):50–79
65. Ellis JA, Markov M (2004) Computing the vertex separation of unicyclic graphs. Inf Comput 192(2):123–161
66. Erbele J, Chavez JD, Trapp R (2003) The cyclic cutwidth of Qn. Manuscript, California State University, San Bernardino
67. Fertin G, Monfroy E, Vasconcellos-Gaete C (2024) Best of both worlds: solving the cyclic bandwidth problem by combining pre-existing knowledge and constraint programming techniques. In: International conference on computational science, pp 197–211
68. Fellows MR, Langston MA (1994) On search, decision, and the efficiency of polynomial-time algorithms. J Comput Syst Sci 49(3):769–779
69. Feo TA, Resende MGC (1989) A probabilistic heuristic for a computationally difficult set covering problem. Oper Res Lett 8(2):67–71
70. Feo TA, Resende MGC (1995) Greedy randomized adaptive search procedures. J Glob Optim 6(2):109–133
71. Huacuja HJF, Castillo-García N (2016) Optimization of the vertex separation problem with genetic algorithms. In: Ochoa-Zezzatti A, Sánchez J, Cedillo-Campos MG, de Lourdes M (eds) Handbook of research on military, aeronautical, and maritime logistics and operations. IGI Global, Hershey, Pennsylvania, USA, pp 13–31. https://doi.org/10.4018/978-1-4666-9779-9.ch002 (Chapter 2)
72. Fraire-Huacuja HJ, López-Locés MC, García NC, Pecero JE, Rangel RP (2017) Solving the cut width optimization problem with a genetic algorithm approach. In: Melin P, Castillo O, Kacprzyk J (eds) Nature-inspired design of hybrid intelligent systems. Studies in Computational Intelligence, vol 667. Springer, Cham, pp 729–738. https://doi.org/10.1007/978-3-319-47054-2_48
73. Gaar E, Puges D, Wiegele A (2024) Strong SDP based bounds on the cutwidth of a graph. Comput Oper Res 161:106449
74. Gane C, Sarson T (1977) Structured systems analysis: tools and techniques, 1st edn. McDonnell Douglas Systems Integration Company. Prentice-Hall, Upper Saddle River, New Jersey
75. Garey MR, Johnson DS (1979) Computers and intractability: a Guide to the theory of np-completeness. W.H. Freeman, San Francisco
76. Gavril F (1977) Some NP-complete problems on graphs. In: Proceedings of the eleventh conference on information sciences and systems, Baltimore, pp 91–95
77. Guan Y, Williams KL (2003) Profile minimization on triangulated triangles. Discret Math 260(1–3):69–76. Elsevier
78. Gibbs NE, Poole WG Jr, Stockmeyer PK (1976) An algorithm for reducing the bandwidth and profile of a sparse matrix. SIAM J Numer Anal 13(2):236–250
79. Glover F (1977) Heuristics for integer programming using surrogate constraints. Decis Sci 8(1):156–166
80. Glover F (1997) Tabu search and adaptive memory programming—advances, applications and challenges. Springer, Boston, pp 1–75
81. Glover F, Laguna M (1997) Tabu search. Kluwer Academic Publishers, Norwell
82. Glover F, Rego C (2017) New relationships for multi-neighborhood search for the minimum linear arrangement problem. J Discret Algorithms 46:16–24
83. Golberg DE (1989) Genetic algorithms in search, optimization, and machine learning. Addison Wesley 1989:102
84. Golovach P (2009) The total vertex separation number of a graph. Discret Math Appl 6(7):631–636
85. Gonzaga de Oliveira SL, Bernardes Junior AB, Chagas GO (2018) An evaluation of low-cost heuristics for matrix bandwidth and profile reductions. Comput Appl Math 37(2):1412–1471. Springer

86. Harper LH (1964) Optimal assignments of numbers to vertices. J Soc Ind Appl Math 12(1):131–135
87. Harper LH (1966) Optimal numberings and isoperimetric problems on graphs. J Comb Theory 1:385–393
88. He M, Wu Q, Lu Y (2022) Breakout local search for the cyclic cutwidth minimization problem. J Heuristics 28(5):583–618
89. Heggernes P, Lokshtanov D, Mihai R, Papadopoulos C (2011) Cutwidth of split graphs and threshold graphs. SIAM J Discret Math 25(3):1418–1437
90. Heggernes P, Van't Hof P, Lokshtanov D, Nederlof J (2012) Computing the cutwidth of bipartite permutation graphs in linear time. SIAM J Discret Math 26(3):1008–1021
91. Holland JH (1975) Adaptation in natural and artificial systems: an introductory analysis with applications to biology, control, and artificial intelligence. University of Michigan Press, Ann Arbor
92. Hu TC (1974) Optimum communication spanning trees. SIAM J Comput 3(3): 188–195
93. Jain P, Srivastava K, Saran G (2016) Minimizing cyclic cutwidth of graphs using a memetic algorithm. J Heuristics 22(6):815–848
94. James B (1996) The cyclical cutwidth of the three-dimensional and four dimensional cubes. Cal State University, San Bernardino McNair Scholar's Program Summer Research Journal
95. Johnson DS, Lenstra JK, Kan AHGR (1978) The complexity of the network design problem. Networks 8(4):279–285
96. Johnson M (2003) The linear and cyclic cutwidth of the complete bipartite graph. REU Project, Cal State University, San Bernardino
97. Juvan M, Marincek J, Mohar B (1995) Embedding a graph into the torus in linear time. Springer, Berlin/Heidelberg, pp 360–363
98. Juvan M, Mohar B (1992) Optimal linear labelings and eigenvalues of graphs. Discret Appl Math 36:153–168
99. Kardam YS, Srivastava K, Sharma R (2017) Minimizing profile of graphs using a hybrid simulating annealing algorithm. Electron Notes Discret Math 63:381–388. Elsevier
100. Kardam YS, Srivastava K (2020) Tabu-embedded simulated annealing algorithm for profile minimization problem. In: Computational methods and data engineering: proceedings of ICMDE 2020, vol 1. Springer, pp 165–179
101. Karger DR (1999) A randomized fully polynomial time approximation scheme for the all-terminal network reliability problem. SIAM J Comput 29(2):492–514
102. Karp RM (1993) Mapping the genome: some combinatorial problems arising in molecular biology. In: Proceedings of the twenty-fifth annual ACM symposium on theory of computing. ACM, New York, pp 278–285
103. Kendall D (1969) Incidence matrices, interval graphs and seriation in archeology. Pac J math 28(3):565–570
104. Kennedy J, Eberhart R (1995) Particle swarm optimization. In: Proceedings of the IEEE international conference on neural networks, Yokohama, vol 4, pp 1942–1948
105. Khandelwal A, Srivastava K, Saran G (2023) Grid bandwidth minimization problem: simulated annealing approach. Mapana J Sci 22(Special Issue)
106. Khandelwal A, Srivastava K, Saran G (2023) A reduced variable neighbourhood search algorithm for grid bandwidth minimization problem. In: 2023 7th international conference on intelligent computing and control systems (ICICCS), pp 795–800
107. Kinnersley NG (1992) The vertex separation number of a graph equals its path-width. Inf Process Lett 42(6):345–350
108. Kirkpatrick S, Gelatt CD, Vecchi MP (1983) Optimization by simulated annealing. Science 220(4598):671–680
109. Kirousis LM, Papadimitriou CH (1985) Interval graphs and searching. Discret Math 55(2):181–184
110. Kirousis LM, Papadimitriou CH (1986) Searching and pebbling. Theor Comput Sci 47(0):205–218

111. Kloeckner B (2009) Cutwidth and degeneracy of graphs. arXiv preprint arXiv:0907.5138
112. Klugerman M, Russell A, Sundaram R (1998) On embedding complete graphs into hypercubes. Discret Math 186(13):289–293
113. Korach E, Solel N (1993) Tree-width, path-width, and cutwidth. Discret Appl Math 43(1):97–101
114. Koohestani B, Poli R (2011) A hyper-heuristic approach to evolving algorithms for bandwidth reduction based on genetic programming. In: International conference on innovative techniques and applications of artificial intelligence. Springer, pp 93–106
115. Koohestani B (2023) On the solution of the graph bandwidth problem by means of search methods. Appl Intell 53(7):7988–8004. Springer
116. Lai YL, Williams K (1999) A survey of solved problems and applications on bandwidth, edge sum, and profile of graphs. J Graph Theory 31(2):75–94. Wiley Online Library
117. Laguna M, Martí R (2012) Scatter search: methodology and implementations in C, vol 24. Springer Science & Business Media, New York
118. Leighton T, Rao S (1999) Multicommodity max-flow min-cut theorems and their use in designing approximation algorithms. J ACM 46(6):787–832
119. Leiserson CE (1980) Area-efficient graph layouts (for VLSI). In: IEEE symposium on foundations of computer science, Syracuse, pp 270–281
120. Lewis JG (1982) The Gibbs-Poole-Stockmeyer and Gibbs-King algorithms for reordering sparse matrices. ACM Trans Math Softw (TOMS) 8(2):190–194
121. Lewis RR (1994) Simulated annealing for profile and fill reduction of sparse matrices. Int J Numer Methods Eng 37(6):905–925
122. Lim A, Lin J, Rodrigues B, Xiao F (2006) Ant colony optimization with hill climbing for the bandwidth minimization problem. Appl Soft Comput 6(2):180–188
123. Lim A, Lin J, Xiao F (2007) Particle swarm optimization and hill climbing for the bandwidth minimization problem. Appl Intell 26(3):175–182
124. Lim A, Rodrigues B, Xiao F (2006) Heuristics for matrix bandwidth reduction. Eur J Oper Res 174(1):69–91
125. Lin YX (1994) Two-dimensional bandwidth problem. In: Combinatorics, graph theory, algorithms and applications 1993, Beijing, pp 223–232
126. Lipton RJ, Tarjan RE (1979) A separator theorem for planar graphs. SIAM J Appl Math 36(2):177–189
127. Livingston M, Stout QF (1988) Embeddings in hypercubes. Math Comput Model 11(0):222–227
128. López-Locés MC, Castillo-García N, Huacuja HJF, Bouvry P, Pecero JE, Rangel RAP, Barbosa JJG, Valdez F (2014) A new integer linear programming model for the cutwidth minimization problem of a connected undirected graph. In: Recent advances on hybrid approaches for designing intelligent systems. Springer, Cham, pp 509–517
129. Lozano M, Duarte A, Gortázar F, Martí R (2013) A hybrid metaheuristic for the cyclic antibandwidth problem. Knowl-Based Syst 54:103–113
130. Luttamaguzi J, Pelsmajer M, Shen Z, Yang B (2005) Integer programming solutions for several optimization problems in graph theory. Technical report, Center for Discrete Mathematics and Theoretical Computer Science, DIMACS
131. Mafteiu-Scai LO, Mafteiu-Scai E, Voina T (2017) Bandwidths optimization on sparse matrices using brain storm optimization. In: 2017 19th international symposium on symbolic and numeric algorithms for scientific computing (SYNASC). IEEE, pp 219–224
132. Makedon F, Sudborough IH (1989) On minimizing width in linear layouts. Discret Appl Math 23(3):243–265
133. Mallach S (2018) Linear ordering based MIP formulations for the vertex separation or pathwidth problem. J Discret Algorithms 52:156–167
134. Martí R, Campos V, Piñana E (2008) A branch and bound algorithm for the matrix bandwidth minimization. Eur J Oper Res 186(2):513–528
135. Martí R, Laguna M, Glover F, Campos V (2001) Reducing the bandwidth of a sparse matrix with tabu search. Eur J Oper Res 135(2):450–459

136. Martí R, Pantrigo JJ, Duarte A, Campos V, Glover F (2011) Scatter search and path relinking: a tutorial on the linear arrangement problem. Int J Swarm Intell Res (IJSIR) 2(2):1–21
137. Martí R, Pantrigo JJ, Duarte A, Pardo EG (2013) Branch and bound for the cutwidth minimization problem. Comput Oper Res 40(1):137–149
138. Maues J, Mendonça I, Amorim G, Gonzaga de Oliveira SL, Pereira AI, Brandão D, González PH (2024) A multi-centrality heuristic for the bandwidth reduction problem. In: Computational science and its applications—ICCSA 2024. Springer Nature, Switzerland, pp 3–15
139. Mcallister AJ (1999) A new heuristic algorithm for the linear arrangement problem. Technical report, University of New Brunswick
140. Miller Z, Sudborough IH (1991) A polynomial algorithm for recognizing bounded cutwidth in hypergraphs. Theory Comput Syst 24:11–40
141. Mitchison G, Durbin R (1986) Optimal numberings of an n × n array. Algebr Discret Methods 7(4):571–582
142. Mladenović N, Hansen P (1997) Variable neighborhood search. Comput Oper Res 24(11):1097–1100
143. Mladenović N, Urošević D, Pérez-Brito D (2014) Variable neighborhood search for minimum linear arrangement problem. Yugosl J Oper Res 24(2):2334–6043. ISSN:0354-0243
144. Mladenović N, Urošević D, Pérez-Brito D, García-González CG (2010) Variable neighbourhood search for bandwidth reduction. Eur J Oper Res 200(1):14–27
145. Moscato P, Cotta C (2010) A modern introduction to memetic algorithms. In: Gendreau M, Potvin J-Y (eds.) Handbook of metaheuristics, vol 146. Springer, New York, pp 141–183
146. Mutzel P (1995) A polyhedral approach to planar augmentation and related problems. In: Proceedings of the third annual European symposium on algorithms, ESA'95. Springer, London, pp 494–507
147. Palubeckis G, Rubliauskas D (2012) A branch-and-bound algorithm for the minimum cut linear arrangement problem. J Comb Optim 24(4):540–563
148. Palubeckis G (2017) A variable neighborhood search and simulated annealing hybrid for the profile minimization problem. Comput Oper Res 87:83–97. Elsevier
149. Pantrigo JJ, Martí R, Duarte A, Pardo EG (2012) Scatter search for the cutwidth minimization problem. Ann Oper Res 199:285–304
150. Papadimitriou CH (1976) The NP-completeness of the bandwidth minimization problem. Computing 16:263–270
151. Pardo EG, Mladenović N, Pantrigo JJ, Duarte A (2012) A variable neighbourhood search approach to the cutwidth minimization problem. Electron Notes Discret Math 39(0):67–74
152. Pardo EG, Mladenović N, Pantrigo JJ, Duarte A (2013) Variable formulation search for the cutwidth minimization problem. Appl Soft Comput 13(5):2242–2252
153. Pardo EG, Soto M, Thraves C (2015) Embedding signed graphs in the line. J Comb Optim 29(2):451–471
154. Peng SL, Ho CW, Hsu TS, Ko MT, Tang C (1998) A linear-time algorithm for constructing an optimal node-search strategy of a tree. In: Hsu W-L, Kao M-Y (eds) Computing and combinatorics. Lecture notes in computer science, vol 1449. Springer, Berlin/Heidelberg, pp 279–289
155. Petit J (2003) Combining spectral sequencing and parallel simulated annealing for the MinLA problem. Parallel Process Lett 13(1):77–91
156. Petit J (2003) Experiments on the minimum linear arrangement problem. ACM J Exp Algorithm 8:2–3
157. Piñana E, Plana I, Campos V, Martí R (2004) GRASP and path relinking for the matrix bandwidth minimization. Eur J Oper Res 153(1):200–210
158. Pop PC, Matei O (2011) An improved heuristic for the bandwidth minimization based on genetic programming. In: Hybrid artificial intelligent systems. Springer, Berlin, pp 67–74
159. Pop P, Matei O, Comes CA (2014) Reducing the bandwidth of a sparse matrix with a genetic algorithm. Optimization 63(12):1851–1876. Taylor & Francis

160. Rajasingh I, William A, Quadras J, Manuel P (2004) Embedding of cycles and wheels into arbitrary trees. Networks 44(3):173–178
161. Raspaud A, Schröder H, Sỳkora O, Torok L, Vrto I (2009) Antibandwidth and cyclic antibandwidth of meshes and hypercubes. Discret Math 309(11):3541–3552
162. Ravi R, Agrawal A, Klein P (1991) Ordering problems approximated: single-processor scheduling and interval graph completion. In: Proceedings of the ICALP. Springer, Berlin, pp 751–762
163. Ren J, Hao JK, Rodriguez-Tello E (2019) An iterated three-phase search approach for solving the cyclic bandwidth problem. IEEE Access 7:98436–98452
164. Ren J, Hao JK, Rodriguez-Tello E, Li L, He K (2020) A new iterated local search algorithm for the cyclic bandwidth problem. Knowl-Based Syst 203:106136
165. Resende MGC, Andrade DV (2009) Method and system for network migration scheduling. United States Patent Application Publication. US2009/
166. Rios FR (1996) Complete graphs as a first step toward finding the cyclic cutwidth of the n-cube. Cal State University, San Bernardino McNair Scholar's Program Summer Research Journal
167. Rodriguez-Tello E, Hao J-K, Torres-Jimenez J (2005) Memetic algorithms for the MinLA problem. In: International conference on artificial evolution (Evolution Artificielle), pp 73–84
168. Rodriguez-Tello E, Hao J-K, Torres-Jimenez J (2008) An effective two-stage simulated annealing algorithm for the minimum linear arrangement problem. Comput Oper Res 35(10):3331–3346
169. Rodriguez-Tello E, Hao J-K, Torres-Jimenez J (2008) An improved simulated annealing algorithm for bandwidth minimization. Eur J Oper Res 185(3):1319–1335
170. Rodriguez-Tello E, Romero-Monsivais H, Ramirez-Torres G, Lardeux F (2015) Tabu search for the cyclic bandwidth problem. Comput Oper Res 57:17–32
171. Rodriguez-Tello E, Lardeux F, Duarte A, Narvaez-Teran V (2019) Alternative evaluation functions for the cyclic bandwidth sum problem. Eur J Oper Res 273(3):904–919
172. Rodriguez-Tello E, Narvaez-Teran V, Lardeux F (2019) Dynamic multi-armed bandit algorithm for the cyclic bandwidth sum problem. IEEE Access 7:40258–40270
173. Rodríguez-García MÁ, Sanchez-Oro J, Rodriguez-Tello E, Monfroy E, Duarte A (2021) Two-dimensional bandwidth minimization problem: exact and heuristic approaches. Knowl-Based Syst 214:106651
174. Rolim J, Sýkora O, Vrt'o I (1995) Optimal cutwidths and bisection widths of 2- and 3-dimensional meshes. In: Graph-theoretic concepts in computer science. Lecture notes in computer science, vol 1017. Springer, Berlin/Heidelberg, pp 252–264
175. Saad Y (2003) Iterative methods for sparse linear systems. SIAM, Philadelphia
176. Safro I, Ron D, Brandt A (2006) Graph minimum linear arrangement by multilevel weighted edge contractions. J Algorithms 60(1):24–41
177. Sharma R, Srivastava K (2009) A new hybrid evolutionary algorithm for the MinLA problem. Int J Oper Res 5(2):229–249
178. Sánchez-Oro J, Duarte A (2012) An experimental comparison of variable neighborhood search variants for the minimization of the vertex-cut in layout problems. Electron Notes Discret Math 39:59–66
179. Sánchez-Oro J, Duarte A (2012) Grasp with path relinking for the sumcut problem. Int J Comb Optim Probl Inf 3:3–11
180. Sánchez-Oro J, Duarte A Grasp for the sumcut problem. In: XVII international congress on computer science research, Morelia (México), 26–28 Oct 2011
181. Sánchez-Oro J, Laguna M, Duarte A, Martí R (2015) Scatter search for the profile minimization problem. Networks 65(1):10–21
182. Sánchez-Oro J, Pantrigo JJ, Duarte A (2014) Combining intensification and diversification strategies in VNS. An application to the vertex separation problem. Comput Oper Res 52(Part B):209–219

183. Santos VG, de Carvalho MAM (2021) Tailored heuristics in adaptive large neighborhood search applied to the cutwidth minimization problem. Eur J Oper Res 289(3):1056–1066
184. Schröder H, Sỳykoa O, Vrt'o I (1999) Cyclic cutwidth of the mesh. In: International conference on current trends in theory and practice of computer science. Springer, pp 449–458
185. Schröder H, Sỳkora O, Vrt'o I (2004) Cyclic cutwidths of the two-dimensional ordinary and cylindrical meshes. Discret Appl Math 143(1):123–129
186. Sciortino V, Chavez JD, Trapp R (2002) The cyclic cutwidth of a P2× P2× Pn mesh. REU Project, Cal State University, San Bernardino
187. Shahrokhi F, Sýkora O, Székely LA, Vrt'o I (2001) On bipartite drawings and the linear arrangement problem. SIAM J Comput 30(6):1773–1789
188. Shiloach Y (1979) A minimum linear arrangement algorithm for undirected trees. SIAM J Comput 8(1):15–32
189. Skodinis K (2000) Computing optimal linear layouts of trees in linear time. In: Proceedings of the 8th annual European symposium on algorithms, ESA'00. Springer, London, pp 403–414
190. Solano F, Pióro M (2010) Lightpath reconfiguration in WDM networks. J Opt Commun Netw 2(12):1010–1021
191. Sýkora O, Torok L, Vrt'o I (2005) The cyclic antibandwidth problem. Electron Notes Discret Math 7th Int Colloq Graph Theory 22:223–227
192. Takagi K, Takagi N (1999) Minimum cut linear arrangement of p-q dags for VLSI layout of adder trees. IEICE Trans Fundam Electron Commun Comput Sci E82-A(5):767–774
193. Tewarson RP (1973) Sparse matrices, vol 69. Academic, New York
194. Thilikos DM, Serna MJ, Bodlaender HL (2005) Cutwidth II: algorithms for partial w-trees of bounded degree. J Algorithms 56(1):25–49
195. Torres-Jimenez J, Izquierdo-Marquez I, Garcia-Robledo A, Gonzalez-Gomez A, Bernal J, Kacker RN (2015) A dual representation simulated annealing algorithm for the bandwidth minimization problem on graphs. Inf Sci 303:33–49. Elsevier
196. Tsao YP, Chang GJ (2006) Profile minimization on products of graphs. Discret Math 306(8–9):792–800. Elsevier
197. Tsao YP, Chang GJ (2007) Profile minimization on compositions of graphs. J Comb Optim 14(2):177–190. Springer
198. Woodcock JR (2006) A faster algorithm for torus embedding. PhD thesis, University of Victoria
199. Yannakakis M (1985) A polynomial algorithm for the min-cut linear arrangement of trees. J ACM (JACM) 32:950–988
200. Yuan J, Lin Y, Liu Y, Wang S (1998) NP-completeness of the profile problem and the fill-in problem on cobipartite graphs. J Math Study 31(3):239–243
201. Zündorf M (2022) Minimum linear arrangement revisited. MA thesis, Karlsruhe Institute of Technology. Available at: https://scale.iti.kit.edu/publications/Zu22.pdf

Heuristics in Vehicle Routing

43

Claudia Archetti, Kris Braekers, and Diego Cattaruzza

Contents

Introduction 1354
Classical Heuristics 1356
Classical Constructive Heuristics 1356
Classical Improvement Heuristics 1357
Metaheuristics 1358
Hybridization and State-of-the-Art Frameworks 1359
Unified Methods 1362
Metaheuristics: Discussion 1363
Matheuristics 1363
Decomposition Approaches 1364
Improvement Heuristics 1367
Branch-and-Price/Column-Generation-Based Approaches 1368
Matheuristics: Discussion 1370
New Research Trends 1371
Knowledge Discovery and Learning 1371
Heuristic Methods for Large-Scale Instances 1373

C. Archetti (✉)
Department of Economics and Management, University of Brescia, Brescia, Italy
e-mail: claudia.archetti@unibs.it

K. Braekers
Research Group Logistics, Hasselt University, Diepenbeek, Belgium
e-mail: kris.braekers@uhasselt.be

D. Cattaruzza
Univ. Lille, CNRS, Centrale Lille, Inria, UMR 9189 – CRIStAL Centre de Recherche an Informatique Signal et Automatique de Lille, Lille, France

Department of Mathematics, Computer Science and Physics, University of Udine, Udine, Italy
e-mail: diego.cattaruzza@uniud.it

© Springer Nature Switzerland AG 2025

R. Martí et al. (eds.), *Handbook of Heuristics*,
https://doi.org/10.1007/978-3-032-00385-0_65

Instances and Libraries 1375
Instances 1375
Publicly Available Implementations 1376
Conclusion 1376
References 1377

Abstract

Vehicle routing problems are among the most widely studied combinatorial optimization problems. Different classes of routing problems exist which all share a common set of decisions: assigning customers to vehicles and defining the sequence of visit for each vehicle, i.e., determining vehicle routes in order to minimize a given cost function. Problems might differ in the objective function and in the presence of additional constraints. However, routing decisions (plus eventually additional problem-specific decisions) make routing problems extremely challenging. For this reason, the research community has concentrated big efforts on the development of efficient heuristic solution approaches. In this chapter, we first briefly introduce the earliest and most classical heuristic algorithms. Then we concentrate on the most recent developments on heuristics for routing problems, namely, metaheuristics, matheuristics, learning approaches, and heuristics for large-scale problems. We provide reference contributions and discuss the key features of each class of methods, concentrating on the most recent research trends.

Keywords

Routing · Heuristics · Metaheuristics · Matheuristics · Knowledge discovery · Learning

Introduction

The Capacitated Vehicle Routing Problem (CVRP) was first introduced by [39] under the name "The Truck Dispatching Problem." The CVRP can be defined on a directed graph $\mathcal{G} = (\mathcal{V}, \mathcal{A})$ where $\mathcal{V} = \{0, 1, \ldots, N\} = \{0\} \cup \mathcal{N}$ is the set of vertices and $\mathcal{A}$ is the set of arcs. Node 0 represents the depot and $\mathcal{N} = \{1, \ldots, N\}$ is the set of customers to be served. Each customer $i \in \mathcal{N}$ requires a positive quantity Q_i of a product. At the depot, a fleet of M vehicles, each of capacity Q, is available to perform deliveries. Each arc $(i, j) \in \mathcal{A}$ is associated with a nonnegative weight C_{ij} which represents the traveling cost to reach node $j \in \mathcal{V}$ from node $i \in \mathcal{V}$.

The CVRP calls for determining at most M routes such that:

- Each route starts and ends at the depot 0.
- Each customer is visited exactly once.

- The sum of the demands of the customers served in each route does not exceed the vehicle capacity Q.
- The total traveling cost is minimized.

Since its introduction, a huge number of papers have studied the CVRP. Scholars have also concentrated their efforts on variants and extensions of the problem. These variants are meant to better capture the reality of operations by considering real-life aspects of routing problems. One of the most studied variants is for example the Vehicle Routing Problem with Time Windows (VRPTW, [28, 29, 42, 119]) in which each customer $i \in \mathcal{N}$ is associated with a time interval $[E_i, L_i]$ during which service should take place. The VRPTW has attracted a lot of attention and recently more complicated variants emerged, like the VRP with multiple time windows (see, e.g., [65] for an interesting methodological contribution).

An extensive review of the complete vehicle routing domain is out of the scope of the present chapter. The interested reader is referred to [27, 37, 56, 95, 123] and the references therein. Even providing an exhaustive list of variants has become a complicated task. Some efforts have been made to provide a taxonomy or to classify routing problems based on the problem attributes considered (e.g., [27, 45, 76]). A more recent discussion of problem attributes can be found in, e.g., [133].

Scope The scope of the present chapter is to specifically review heuristic methods that have been developed to solve Vehicle Routing Problems (VRPs). Works with a similar aim have already been conducted, as for example in the book chapters of [81, 82] or the papers by [28, 29, 88, 129].

As a result, we only briefly discuss classical heuristics for VRPs as this topic has already been widely covered in previous published material. On the other hand, we concentrate our discussion on state-of-the-art metaheuristics, on matheuristics and on recent advances on the topic such as knowledge discovery, machine learning, and the resolution of large-scale instances.

Still, due to the huge amount of publications on the subject, it is not possible to provide an exhaustive list and discussion within the context of a single chapter. We rather provide an overview on some selected topics and guide the reader toward references for an in-depth analysis.

Organization The reminder of the chapter is organized as follows. Section "Classical Heuristics" provides an overview on classical heuristics for VRPs. The topic of this section has already been covered by other works. Thus, it is simply intended to provide a glance at the subject and guide the interested reader toward appropriate material. Section "Metaheuristics" is dedicated to metaheuristics for VRPs. As for section "Classical Heuristics," the topic of this section has been already partially covered by other works. As a consequence, we only concentrate on recent advances on the subject. Section "Matheuristics" concentrates on matheuristics for VRPs where the focus is on discussing the most popular schemes and commenting on the potential of matheuristics. Section "New Research Trends" presents latest trends on

the subject such as the inclusion of knowledge discovery or machine learning bricks in the resolution of VRPs. Section "Instances and Libraries" lists available open-source codes dedicated to the resolution of VRPs and instances classically used by the community to compare solution approaches. Finally, section "Conclusion" concludes the chapter.

Classical Heuristics

The term *classical heuristics* indicates procedures that do not contain components that allow the objective function to deteriorate from one iteration to the next [80]. As a consequence, classical heuristics get trapped at the first local optimum they encounter. Including mechanisms to escape from local optima leads to the so-called *metaheuristics*.

Classical heuristics have been the focus of the research development from the introduction of the VRP to the 1990s. The main characteristics of classical heuristics are simplicity and rapidity of execution. Despite that, successful classical heuristics are able to provide good-quality solutions. This is the reason why they are still embedded in most of the state-of-the-art solvers.

Classical heuristics are usually classified in two categories: constructive (section "Classical Constructive Heuristics") and improvement heuristics (section "Classical Improvement Heuristics"). Procedures in the former family build one step after the other a solution from scratch. Procedures in the latter family get a solution as input and attempt to improve its quality.

The material presented in this section has already been largely discussed in previous papers or book chapters such as for example [28, 82, 88, 129] to which the interested reader is referred. To avoid repetitions and due to space limitations, here we present the conventional classification of heuristics, report what we believe are the key concepts, and provide references to the interested reader.

Classical Constructive Heuristics

Constructive heuristics, as the name indicates, have the objective of constructing a solution from scratch. Classical constructive methods for routing problems are usually categorized into two subcategories. The first contains construction methods properly said, whose objective is to build a solution from scratch attempting to minimize the routing cost of the overall solution. Simple insertion heuristics that insert customers one after another in routes fall in this category [31]. Another well-known example of such methods is the Clarke and Wright heuristic [35].

The second category contains two-phase methods which take advantage of the two decisions to be taken in routing problems, that is, which customers are served together by the same vehicle and how to route vehicles, to decompose the construction in two phases. Based on which decision is taken first, such methods fall in the *Route First-Cluster Second* category or *Cluster First-Route Second* category.

An example of former is the Split algorithm [17, 103], whereas an example of the latter is the Sweep algorithm [54].

Classical Improvement Heuristics

Classical improvement heuristics start from a given initial solution and attempt to improve its quality, usually by applying a sequence of modifications to the initial solution received as an input.

Modifications are usually classified into *intra-route* and *inter-route* modifications. The procedures in the former category do not modify the assignment of customers to vehicles but rather focus on improving the routing cost needed to visit all customers assigned to a given vehicle. Instead, *inter-route* heuristics allow moving customers from one route to another.

As *intra-route* heuristics, one can use heuristics designed for the Traveling Salesman Problem (TSP) such as the k-opt heuristic. Basically, k-opt removes k edges of the current solution and replaces them with k other edges. The reader is referred to [62, 87]. Recent works have proposed procedures to reduce the exploration burden of 2-,3-, and 4-opt neighborhoods [77–79]. Classical *inter-route* heuristics are relocation of sequences of customers from one route to another route or the exchange of sequences of customers between routes.

For a more detailed discussion of both *intra-route* and *inter-route* operators for routing problems, we refer to [88, 129]. Clearly, specific problem variants may require specific types of heuristics; see, for example, [99] for a discussion of neighborhoods for paired pickup-and-delivery problems and their efficient exploration.

The performance of the improvement heuristics relies on the efficient exploration of the neighborhood of a solution and evaluation of the impact of modifications. Constant time evaluation and feasibility checks can be easily obtained when the problem at hand is the CVRP. However, when side constraints such as time windows are taken into account, achieving such constant time evaluations requires special care.

A first work in this direction in the context of the VRPTW is the paper of [114], which proposes a methodology to check the feasibility of a move in constant time, which allows to fully explore the neighborhood defined by k-opt moves in $O(N^k)$, given the requirement that the lexicographic order induced by the arcs in the current solution is followed to perform subsequent checks.

However, most heuristics for the VRPTW allow violations of the TW constraints by penalizing them in the objective function. Usually, a penalization function that is linear on the delay is used (see, for example, [15]). This allows an evaluation of classical neighborhoods in $O(N)$. Nagata et al. [97] propose the concept of *time-warp*: Whenever the vehicle arrives late at a customer, it is assumed that it can travel back in time to meet the closing of the TW and to start the service of the corresponding customer without delay, but at the expense of paying a penalty proportional to the original delay. This result is extended in [129], which shows that classical operators for routing problems can be seen as a concatenation

of subsequences of customers and TW penalty violation can be evaluated in constant time.

It goes without saying that specified information needs to be stored to obtain such computational complexity. The interested reader is referred to [129, 132].

Metaheuristics

Historically, metaheuristic methods have been classified into two categories: trajectory-based and population-based methods. The first class of methods manage a single solution at a time, which is attempted to be improved as much as possible. The second class of methods simultaneously manage a set of solutions.

Trajectory-based metaheuristics Until the 1990s the heuristic procedures developed by researchers to solve routing problems were mainly trajectory-based due to the fact they are rather simple, easy to understand and to implement. A well-known example includes the TABUROUTE algorithm of [53]. Trajectory-based procedures start from an initial solution s_0 and, at each iteration, move from a solution s_i to another one s_{i+1} in its neighborhood. If only improving solutions are accepted, the procedure converges to a so-called local minimum. To allow the process to continue the exploration of the solution space, also non-improving solutions may be accepted. However, in such case, care should be taken to avoid cycling.

Different mechanisms have been proposed in this regard. This has led to well-known procedures such as Iterated Local Search (ILS), Tabu Search (TS), Simulated Annealing (SA), Deterministic Annealing (DA), Variable Neighborhood Search (VNS), and Guided Local Search (GLS), which have all been widely applied to routing problems. The interested reader is referred to [75, 81, 88]. These procedures typically make use of the classical improvement heuristics from section "Classical Improvement Heuristics" in the local search part.

Population-based metaheuristics Population-based metaheuristics maintain a set of solutions (a *population*) and are often inspired by natural phenomena to gradually find better solutions. Evolutionary Algorithms (EA), such as Genetic Algorithms (GA), are one example. They simulate the paradigm of evolution by creating new solutions (i.e., *individuals*) from combining interesting parts of current solutions in the population. Other population-based approaches that rely on recombining solutions include Path Relinking (PR) and Scatter Search (SS). Finally, approaches such as Ant Colony Optimization (ACO) and Particle Swarm Optimization (PSO) are based on ideas from swarm intelligence. See for example [75, 129] for a more detailed discussion.

Although these population-based methods gained some initial attention, they were often not very popular neither competitive. Good-quality results started to be achieved when local search procedures were systematically embedded in the design of the algorithm, giving rise to procedures that are sometimes called *memetic* algorithms ([98], see also section "Hybridization and State-of-the-Art

Frameworks"). In addition, it is noteworthy that many current well-performing population-based algorithms rely on a giant tour representation of a solution (i.e., a solution is encoded as a single sequence of customer visits). This concept has gained much popularity since [103] made use of the Route-First Cluster-Second procedure proposed in [17] to evaluate such individuals in the population. Prins [103] applies a procedure, called *Split*, to cut/split the sequence of visits in the routes to be performed by vehicles. The strength of the *Split* procedure is to be able to obtain the optimal solution associated with a sequence of visits in polynomial time. More recently, [126] showed that *Split* can be implemented to obtain a linear complexity in the cardinally of $\mathcal{N}$.

Finally, in the past two decades, we have witnessed a myriad of new metaheuristic approaches based on all kinds of human, natural, or other phenomena, which are often population-based. However, these so-called metaphor-based heuristics are receiving increasing levels of criticism from the metaheuristics research community, among others due to their common lack of novelty and use of nonstandard terminology (see, e.g., [120, 134]). For a recent discussion on this matter, we refer to [6].

In the remainder of this section, we turn to current state-of-the-art metaheuristic approaches for routing problems (section "Hybridization and State-of-the-Art Frameworks"), discuss some unification attempts (section "Unified Methods"), and provide a concluding discussion (section "Metaheuristics: Discussion").

Hybridization and State-of-the-Art Frameworks

It is well known by now that most well-performing metaheuristic algorithms rely on some form of local search and typically use ideas of different classical frameworks [123]. This leads to the so-called hybrid metaheuristics. Commonly applied ideas in the context of trajectory-based algorithms include the use of multiple neighborhoods (cfr. VNS), restart procedures (cfr. Greedy Randomized Adaptive Search Procedure, GRASP), accepting worsening solutions (cfr. SA), perturbation mechanisms (cfr. ILS), and short- and long-term memory aspects (cfr. TS, GLS). For population-based methods, hybridization often results in the integration of local search procedures to improve new individual solutions.

There is no single metaheuristic algorithm that can deal with every routing problem (see also section "Unified Methods,") and the best-performing algorithms on different problem variants may be composed of considerably different building blocks. Yet, the remainder of this section focuses on a few algorithm frameworks that have become highly popular in recent years and/or have recently obtained state-of-the-art results on the classical CVRP and several closely related variants. We provide a discussion of the advantages of each framework. Although this is by no means an exhaustive list, we believe that these algorithm frameworks are representative of the current state of the art on metaheuristic algorithms. Note that some other recent and highly competitive algorithms (e.g., [3, 13]) are discussed in section "New Research Trends."

Hybrid Genetic Search

A state-of-the-art algorithm for the CVRP and many of its variants is the Hybrid Genetic Search with Advanced Diversity Control algorithm (or Hybrid Genetic Search (HGS) in short) introduced in [128, 130]. It was later extended to allow addressing an even wider set of VRP variants [131], while recently an open-source re-implementation specifically for the CVRP was presented [127]. The algorithm provides state-of-the-art results on numerous problem variants and, exemplary for its effectiveness and popularity, it was the backbone for most of the competitive approaches in the recent 12th DIMACS Implementation Challenge on vehicle routing (http://dimacs.rutgers.edu/programs/challenge/vrp/).

The HGS algorithm relies on three key elements: exploration via the use of a population-based evolutionary search framework, intensification via local search procedures to improve new individuals in a population, and advanced population management with specific attention for population diversity.

In terms of the evolutionary search, solutions are encoded using a giant tour representation, and the algorithm maintains a subpopulation of feasible solutions and one of infeasible solutions. For the latter solutions, infeasibilities are penalized in the objective function. New individuals (*offspring*, *childs*) are found by applying a crossover operator on two solutions (*parents*) from the current population, selected via a binary tournament. Next, they are improved (*educated*) using local search operators, and, if required, restoring feasibility is attempted (*repair*). The population management scheme, which decides which solutions to add and maintain in the population, evaluates solutions on a measure that combines their objective value and a diversity measure, indicating a solution's diversity from other solutions in the population. It ensures population diversity while always maintaining the best (*elite*) solutions, thereby guaranteeing good performance and avoiding premature convergence.

One of the advantages of HGS is that it is very flexible in the sense that one can apply it to every problem for which a solution can be represented as a giant tour and decoding (or splitting) the tour into a solution for that giant tour is relatively computationally efficient. Finally, note that researchers are continuously exploring improvements of HGS and its components (e.g., [67, 91]), applying the HGS framework to other routing problems (e.g., [107]) and its hybridization with other heuristic frameworks (e.g., [117]).

Iterated Local Search

The Iterated Local Search (ILS) framework [89] has been frequently applied to address routing problems. It repeatedly optimizes a solution until a local optimum is found, after which the solution is perturbed to escape from this local optimum. A nice application includes the work of [100] which is a multi-start local search where multiple neighborhoods are applied in random order. The algorithm has been enriched with a Set Partitioning phase (resulting in a matheuristic algorithm, see section "Matheuristics") and applied to a variety of routing problems by [121].

More recently, state-of-the-art results on the CVRP are obtained by [93, 96]. Máximo and Nascimento [96] propose a hybrid ILS and Path Relinking (PR) algorithm. They first propose an adaptive version of ILS (AILS) in which both the degree of perturbation and the acceptance criterion are managed by adaptive strategies. The perturbation degree is guided by the distance between local optima and reference solutions. For the adaptive acceptance criterion, the underlying rationale is the observation that promising regions of the solution space contain several good-quality solutions, calling for intensification of the search (i.e., reduce the acceptance threshold), while non-promising regions require diversification. Next, they hybridize the AILS algorithm with a PR procedure (AILS-PR). Applied to the CVRP benchmark instances of [125], the AILS-PR algorithm leads to the current state-of-the-art results. To efficiently solve very large-scale instances, [93] propose to remove the PR procedure but instead integrate its underlying idea in the AILS algorithm, leading to a so-called two-phase AILS algorithm, or AILS-II, which is competitive for small- and medium-scale instances and outperforms previous approaches on large-scale instances.

A clear advantage of ILS is its conceptual simplicity. Yet, it may require problem-tailored local search operators, making it less straightforward to develop flexible algorithms that can tackle multiple problem variants. Additionally, in order to achieve state-of-the-art performance, it seems that integration with other methodologies (e.g., set partitioning, path relinking) may be worthwhile.

Large Neighborhood Search

Large Neighborhood Search (LNS) is a metaheuristic framework which builds upon the ruin-and-recreate principle introduced by [115, 116]. The idea is to iteratively partially destroy a solution (i.e., removing some customers) and then repair the solution again (i.e., reinserting the customers in the solution). As such, a large neighborhood of the current solution is implicitly defined, making it possible to move to solutions that cannot be attained by simple local changes as in classical improvement heuristics. The successful general-purpose LNS algorithm of [101] can address a variety of VRPs and employs multiple operators to destroy and repair solutions, randomly selecting which ones to apply in each iteration. An adaptive version (ALNS) is introduced by [109] in which the selection of operators is based on their ability to improve solutions in past iterations. For more details, the reader is also referred to [102].

(A)LNS can be considered a hybrid algorithm as typically multiple destroy and repair operators are applied, and often deteriorating solutions may be accepted during the search (e.g., using a SA-like acceptance criterion). Basic removal operators include *random* (randomly selecting customers to remove), *worst* (removing customers which result in the largest objective value improvements when removed), and *related* removal (removing requests which are similar in some way, for example, with respect to spatial or temporal aspects, and hence easy to interchange in solutions). Repair operators are often based on greedy heuristic approaches, like *best*, *greedy*, or *k-regret* insertion, although exact methods could be applied as well.

Over the past two decades, (A)LNS has become one of the most commonly applied heuristic frameworks for routing problems. Although it may not (or no longer) provide the very best results on classical problems like the CVRP and VRPTW, it seems mainly popular for more complex and tightly constrained problem variants for which it may be more difficult to find powerful local search procedures, and for problems like paired pickup-and-delivery problems (e.g., the Pickup-and-Delivery Problem with Time Windows, PDPTW) for which giant tour representations (like the one used in HGS) are more difficult to handle, for example, in terms of move evaluation (see, e.g., [32, 38]). Another advantage of (A)LNS is its ease and speed of implementation, as implementing one or few operators is sufficient to obtain a working algorithm. However, achieving excellent results often requires implementing and testing multiple problem-specific removal and insertion operators (e.g., [40, 44]). Along those lines, some recent contributions have shown the value of alternating between large and small neighborhoods [44, 83].

In contrast, [32] take a different approach in their very effective LNS-based algorithm, denoted Slack Induction by String Removals (SISRs). This method only relies on a single removal and a single repair operator, *adjacent string removal*, and *greedy insertion with blinks*, respectively. The removal operator focuses on introducing both spatial and capacity slack in solutions by removing adjacent strings of customers (i.e., segments of different routes that are spatially close to each other). Repairing a solution is done through a slight variation of *greedy* insertion by randomly ignoring some insertion positions. Finally, the algorithm also contains a dedicated procedure which minimizes the number of vehicles used, and it has been shown to lead to very competitive results on the CVRP and several variants.

Unified Methods

Routing problems are ubiquitous and appear in diverse applications and contexts, each with their own specific objectives and constraints, and often part of a larger decision-making process. As such, the list of VRP variants that have been studied is endless. While early research mainly focused on a limited number of well-defined problem variants with simple objectives and few side constraints, the last decades have seen the appearance of more complex problem variants with many side constraints and more intricate objective functions (sometimes denoted *rich* VRPs or *multi-attribute* VRPs).

Even though there are some generally accepted generic metaheuristic frameworks (e.g., ILS, LNS, GA), these often require tailor-made components for searching and evaluating new solutions. As a result, in many cases, heuristic algorithms are developed for very specific problem variants, whether it being classical problems or multi-attribute ones. Although this has its merits, it often limits the generalizability of these algorithms and may hinder the uptake of new ideas.

In contrast, unified algorithms aim at addressing a variety of problems with a single algorithm implementation. Over the past decades, only limited research attention has been devoted to such unified methods. Some examples include [36, 68, 110, 121, 131]. Results indicate that having more general methods does

not necessarily come at the expense of solution quality [131]. Therefore, it seems worthwhile to further focus on such methods in the future. This may also link to the domain of hyper-heuristics in which the goal is to find automated ways of selecting or generating (meta)heuristic algorithms depending on the problem at hand (see, e.g., [30, 43] for recent overviews on this domain).

Metaheuristics: Discussion

As is clear from the previous sections, the metaheuristics field has evolved considerably over the past decades. Before the 2000s, competitive approaches for VRPs were mostly trajectory-based (e.g., [53]), relying on one of the several classical frameworks and often on simple local search procedures. Later, different new trends appeared. These include the use of multiple, more complex and larger neighborhoods and applying conceptual ideas and building blocks from multiple original frameworks, leading to hybrid algorithms. Furthermore, recent years have shown the effectiveness and efficiency of population-based algorithms based on giant tour representations, especially when being combined with local search-based intensification steps and population diversity management. At the same time, researchers are considering more and more problem extensions, inspired by real-life characteristics and constraints, leading to an ever increasing list of VRP variants being considered. Together, this has led to a huge variety of (often complex) metaheuristic algorithm configurations being applied to many different problems, making it often hard to compare contributions and generalize findings. As such, instead of developing further overly complex hybrid algorithms, a key future research opportunity may include developing more simple and more generic metaheuristic algorithms.

Matheuristics

According to the definition proposed in [25] and adopted also in [10], *matheuristics* are "heuristic algorithms made by the interoperation of heuristics/metaheuristics and mathematical programming techniques." More specifically, as done in [10], we will consider in the following only approaches embedding the solution of a mathematical programming formulation. Other approaches have been classified as matheuristics as well, like heuristics which embed an exact algorithm based on dynamic programming (see [104] and more recently [23, 48]), but they will not be the topic of this section.

Archetti and Speranza [10] classify matheuristic algorithms for routing problems in three main classes:

1. *Decomposition approaches*: As the term says, in this case the problem is decomposed into multiple subproblems and at least one of them is solved through a mathematical programming formulation.

2. *Improvement heuristics*: In this case the mathematical programming formulation is used to improve the solution found through a heuristic approach.
3. *Branch-and-price/column-generation-based approaches*: These approaches include both the well-known class of restricted master heuristics and the approaches based on heuristic column generation.

We are going to adopt the same classification in the following, as we believe it is still consistent even considering the more recent literature that was not included in the former survey. However, the goal here is to explain the general idea behind each of the matheuristic schemes mentioned above and present the most relevant applications in the routing literature, rather than presenting an exhaustive survey of contributions since the paper of [10].

Also, we will propose an analysis of the advantages and disadvantages of these schemes, discussing the motivations of why they became popular for some problem classes, when possible.

Decomposition Approaches

As their name implies, decomposition approaches are based on the idea of decomposing a problem in subproblems (typically easier to solve than the original problem). Each subproblem is solved separately, and the corresponding solutions are combined to obtain a solution to the original problem.

A first approach belonging to this class dates back to 1981 with the work by [49], in which a matheuristic is presented for the classical CVRP. Specifically, the algorithm first assigns customers to vehicles by solving to optimality an assignment problem. Then, the route of each vehicle is determined by solving a TSP over the customers assigned to the vehicle plus the depot. The assignment problem solved in the first phase uses, as objective function, an approximation of the routing cost related to the constructed assignment. The authors propose different schemes to approximate the routing cost, the first and simpler one being based on the idea of:

1. Identifying seed customers, one per vehicle
2. Assigning remaining customers to seed customers, on the basis of a given insertion cost

An illustration of the two steps of the first phase is presented in Fig. 1, taken from [49], where customers 4, 7, 10, 13, 18, 20, and 24 are seed customers (step 1) and the dashed lines represent the calculation of the insertion cost for assigning customer 23 to the cluster of seed customer 24.

In the second phase, the TSP associated with each vehicle is solved through a mathematical formulation in [49].

This scheme belongs to the class Cluster First–Route Second, already introduced earlier. Many matheuristics belonging to this class used this idea, developing different approaches for the solution of each of the two phases. Specifically, it is

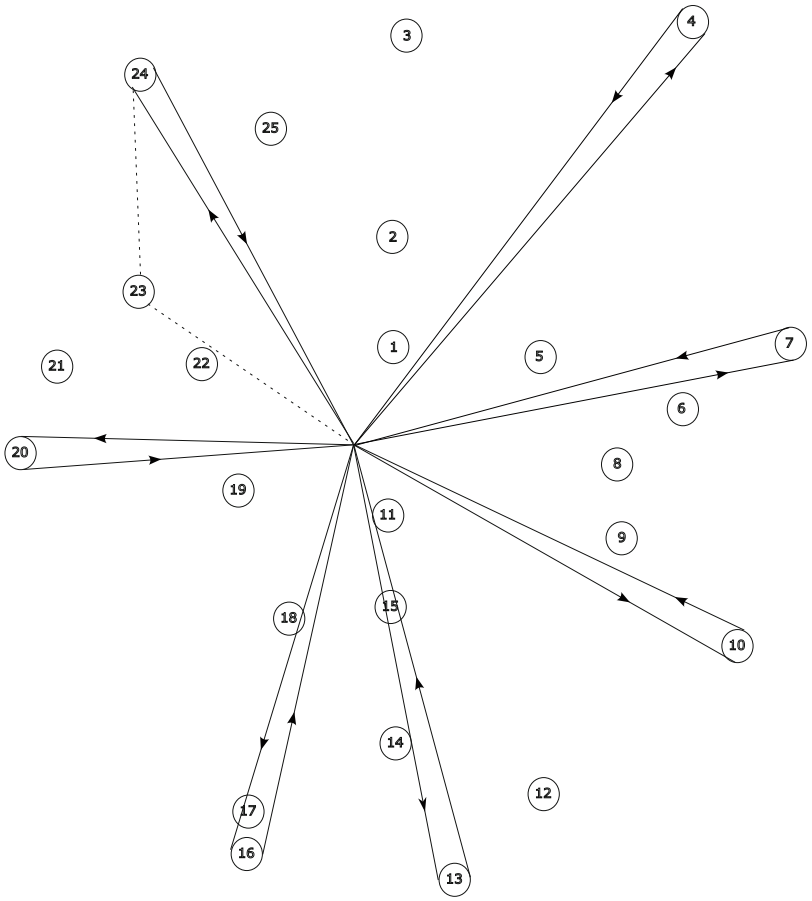


Fig. 1 Illustration of the two steps of phase 1 of Fisher-Jaikumar's algorithm

quite common to solve the second phase by using a heuristic for the TSP. For example, in [1] the authors use the Lin-Kernighan heuristic [87]. Also, as noted in [10], the scheme originally proposed in [49] has evolved in different ways, one of the most effective being the iterative use of the two steps described above. The main idea behind such an iterative approach is to modify the objective function used as an approximation of the routing cost in the first step at each successive iteration, with the aim of obtaining different (and hopefully better) solutions at each iteration. One of the first contributions proposing such a scheme is [47] where the authors solve an Inventory Routing Problem (IRP). Also, [1] proposed a similar idea for the solution of the Production Routing Problem (PRP). We now briefly describe the main idea behind the iterative approach proposed in the latter paper, as it presents a simple and easy rule for the update of the approximation cost. Specifically, the approximated routing costs are set and updated as follows:

1. In the first iteration, and differently from [49], each customer is associated with an approximated routing cost corresponding to the back-and-forth trip from the depot to the customer.
2. In the following iterations, the approximated routing cost associated with each pair customer route is calculated as the cheapest cost for inserting the customer in the route.

Thus, the idea is to keep updating the routing costs in the attempt to obtain a new (and hopefully better) solution for the PRP, once the solution is completed by determining the routes associated with the new assignment.

A second popular type of methods belonging to the class of decomposition approaches is the one known as *rolling horizon* methodologies. These methods focus on routing problems involving the time dimension, specifically, routing problems over multiple periods, and can be considered as a specific case of the general rollout policy for sequential decision-making [22]. The main idea is to decompose the problem into subproblems associated with a shorter time horizon than the original problem and to solve them sequentially. The solution of a subproblem determines the value of the corresponding set of variables which remain then fixed when solving the subsequent subproblems. This approach was first proposed in [105] for the solution of an IRP arising in maritime logistics. It is in general mostly applied to IRPs and PRPs, problems for which the time horizon spans over multiple periods. The general scheme of the rolling horizon approach is presented in Fig. 2, taken from [105]. At each iteration k, subproblems already solved determine what is referred to as *frozen periods* in the figure. The *central period* corresponds to the subproblem solved at iteration k. Subproblems corresponding to periods following the central period might be ignored or included with some relaxed constraints, as done in [105].

Finally, we mention the recent increase in contributions in which Kernel Search (KS) is applied to different variants of routing problems. KS is a matheuristic scheme, belonging to the class of decomposition approaches, consisting of the following main steps. Given a mathematical formulation of the problem to be solved:

1. Rank variables according to a given criterion.
2. Cluster the variables in bundles following the order defined in step 1.

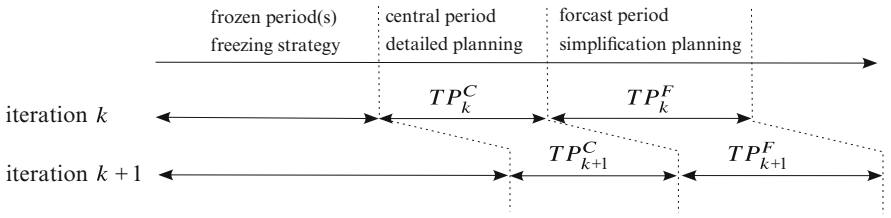


Fig. 2 Illustration of the rolling horizon approach

3. Iteratively solve subproblems corresponding to the original problem formulation restricted to the set of variables in each bundle. Bundles are considered sequentially as generated in step 2.
4. Return the best solution found in step 3.

KS is a general matheuristic scheme that has been successfully applied to many combinatorial optimization problems [23, 24, 60, 92]. However, its application to routing problems is relatively recent [8, 55, 71]. One of the reasons is related to the fact that step 1 poses specific challenges in routing problems. In fact, in most of the earlier contributions on KS applied to combinatorial optimization problems different than routing, the criterion used to sort variables in step 1 is based on the solution of the relaxation of the original problem formulation. However, in routing problems, this criterion does not work properly as:

- Compact formulations tend to provide weak bounds. Thus, the information gathered from the linear relaxation is almost useless.
- Non-compact formulations (for example, those with exponentially many subtour elimination constraints) are difficult to handle and, often, they are time-consuming so not suitable for a heuristic approach.

This drawback has been overcome in [8, 55, 71] by proposing a ranking of variables not based on solving a problem formulation but instead, for example, as in [8], on the frequency with which variables appear in good solutions found by a heuristic algorithm run a priori.

Improvement Heuristics

Improvement heuristics use a mathematical programming formulation to improve the solution found by a heuristic. As described in [10], the mathematical formulation can be used as a move operator, as an intensification operator, and, also, in a single shot or iteratively. Some interesting schemes that emerged in the literature are the following:

- *Exploiting the information gathered from the heuristic run to solve a restricted formulation*: The main idea in this case is to store information related to solutions visited by a heuristic algorithm during execution, with the assumption that these solutions are good, at least in their corresponding neighborhood. This information is then exploited by solving a restricted version of the mathematical formulation associated with the original problem, where, for example, variables associated with features of solutions that have never (or almost never) been visited by the heuristic are set to zero, as not promising. An example of this approach is provided in [9] for the solution of the Directed Profitable Rural Postman Problem (DPRPP), which is a generalization of RPP in which a profit is associated with a subset of the arcs and the objective is to find a tour that

maximizes the difference between the total collected profit and the traveling cost. The authors propose a matheuristic where first a TS is run. During execution of the TS, information about profitable arcs traversed in the visited solutions is collected. At the end of the TS run, this information is used to classify profitable arcs in *good, bad, dubious*. Finally, the DPRPP formulation is solved by imposing constraints (namely, variable fixing) that forbid to choose bad arcs, impose to choose good arcs, and leave a free decision for dubious arcs. This way the size of the solution space is reduced, and thus the formulation can be solved efficiently, hopefully providing an improving solution with respect to the one found by the TS.

- *Using a mathematical programming formulation as an operator*: There have been different applications of this kind of scheme. One that deserves to be mentioned is related to the application of such an operator to variants of routing problems which include quantity decisions, i.e., when it has to be decided what is the quantity that should be delivered to customers when visited. This situation happens, for example, in the Split Delivery Vehicle Routing Problem (SDVRP), the IRP, and the PRP. In this case, in order to decide the quantity to be delivered at each customer visit, one needs to define an operator based on continuous variables, contrary to most of the standard routing operators which are combinatorial in nature. For example, the large majority of the heuristic algorithms proposed in the literature for the solution of the IRP use a mathematical programming formulation to define delivery quantities (see [7] for an earlier example).
- *Using a mathematical programming formulation as an intensification operator*: The idea in this case is to use the mathematical programming formulation to intensify the search around an already good solution. An early example of this scheme is presented in [51] for the CVRP. The algorithm takes as input an existing solution (the best-known solution from the literature). Then, chains of customers are removed from the solution, and a Mixed Integer Linear Program (MILP) is solved to find the best reinsertion points. The procedure is iterated until improvement. Another popular scheme in this class is the one where heuristics are first run to construct multiple solutions and obtain a large number of routes (hopefully of good quality). All routes are then collected and passed to a route-based formulation which selects the best ones whose union gives a feasible solution. The scheme has been originally presented in 1976 by [50] for the solution of the CVRP and has been widely applied to different routing variants afterward. These approaches can also be classified as *column-generation-based approaches* as presented in the next section.

Branch-and-Price/Column-Generation-Based Approaches

This class of matheuristic approaches has gained popularity in the last two to three decades thanks to the success of column generation/branch-and-price exact approaches for different complex combinatorial optimization problems. For example,

branch and price is currently the leading methodology for solving different variants of routing problems to optimality. The research in the field has evolved in different directions, one of which is focused on how to exploit the potential of the column generation/branch-and-price scheme to get high-quality feasible solutions.

The approach which has by far gained the largest popularity is the so-called *restricted master heuristic*, on which we concentrate in the following. The algorithm is based on solving a master problem which typically corresponds to a set packing/set partitioning formulation. For example, the set packing formulation for the classical CVRP uses variables associated with feasible routes, i.e., routes where the vehicle capacity constraint is satisfied. As there are exponentially many feasible routes, a column generation approach is used to solve the relaxation of the master problem where variables are dynamically generated. In an exact approach, columns are generated until the relaxation of the master problem is solved to optimality. Then, the branching phases start and column generation is retrieved in every node of the branch-and-bound tree. Restricted master heuristics typically work in two ways:

1. The column generation phase is performed at the route node only. This means that once the relaxation of the original master problem is solved, all generated columns are passed to the MIP solver which solves the master problem over this restricted set of columns. This is a very popular approach, which is also often embedded in exact branch-and-price approaches with the aim of finding good primal bounds, allowing to prune nodes in the branch-and-bound tree [41, 90].
2. The column generation phase is performed heuristically, meaning that the relaxation of the master problem is not solved to optimality. This approach has the disadvantage that it does not provide a valid dual bound which can be used to measure the quality of feasible solutions. On the other side, a big advantage is that the column generation phase is much faster. Indeed, it is known [41, 90] that the most time demanding procedure in column generation/branch-and-price algorithms is the exact solution of the pricing problem.
   Concerning how to solve the pricing problem heuristically, the most common approach is to simplify the exact pricing algorithm which, in the case of routing problems, corresponds typically to a labeling algorithm solving the Elementary Shortest-Path Problem with Resource Constraints (ESPPRC). For example, one might solve the ESPPRC on a reduced graph where only a subset of arcs are considered. However, there exist also contributions in which the heuristic pricing algorithm is based on local search and/or metaheuristic algorithms (see [138] for a recent contribution).

As mentioned earlier, this class of matheuristic schemes has grown rapidly thanks to the success of branch-and-price algorithms for the solution of different classes of routing problems, even if the earliest contribution in this class is even prior to that, i.e., in 1976 by [50]. This contribution belongs also to the second class of approaches listed above.

Matheuristics: Discussion

Matheuristics have been extensively applied to different classes of routing problems [10]. Many contributions have tackled the classical CVRP. This happened for obvious reasons: The CVRP is among the most studied problems in routing, so it attracted many more contributions than other routing problems. However, the current state-of-the-art heuristic algorithms for the CVRP are based on metaheuristic schemes and local search operators, as mentioned in section "Metaheuristics." Nevertheless, there exist classes of routing problems for which the biggest effort in the development of heuristic approaches has focused on matheuristics. As an example we mention the Inventory Routing Problem (IRP). We already referred to the IRP earlier. However, we now present its definition in order to discuss why matheuristics have been so popular for its solution. The IRP is defined as follows. There is a single supplier who is responsible of defining the replenishment plan to fulfill the demand of customers over a given planning horizon. Each customer faces a demand in each period of the planning horizon. The replenishment plan has to be such that stockout never occurs at all customers nor at the supplier, and the total cost, given by the sum of routing and inventory costs, is minimized. The current state-of-the-art heuristic algorithm for the IRP is the branch-and-cut embedded matheuristic proposed in [118], which won the IRP track of the 12th DIMACS Implementation Challenge. This work presents an exhaustive review of the exact and heuristic approaches of the IRP, and what emerges is that the vast majority of the heuristics are matheuristics (11 of the 13 contributions cited in the paper). A similar situation happens for the Production Routing Problem (PRP), which is an extension of the IRP in which production decisions are included as well. So, a natural question emerges: Why have these problems been tackled mostly with matheuristics, while metaheuristics are not so popular for these classes of problems? There might be different answers to this question. However, we now discuss what is our opinion concerning the success of matheuristics for IRPs (same arguments apply for PRPs).

In the IRP, in addition to the classical combinatorial decisions related to when each customer should be visited, what are the customers visited by each vehicle, and what is the visiting sequence, there are also decisions related to the quantity that should be delivered to each customer at each visit. The latter decision has two peculiarities:

1. It is a continuous (and not combinatorial) decision.
2. Determining the best quantity to be delivered to a customer in a given visit is not straightforward as this depends on the quantity received by the customer in other visits and, also, on the quantity delivered to other customers in the same route.

Classical operators used in heuristics for routing problems have difficulties in handling the two features above: First, they deal with discrete choice sets; second, they evaluate the local change made by the move. So, in case of a continuous decision, evaluating all options is clearly not viable. Also, a local evaluation of the quantity decision does not work properly because of feature 2 mentioned above.

Instead, quantity decisions can be handled by solving a mathematical programming formulation which deals with continuous variables and includes constraints related to vehicle capacity and inventory level.

Thus, in our opinion, matheuristics have gained success for these classes of problems as they can easily handle interrelated continuous decisions, where classical local search combinatorial operators have difficulties. Also, a mathematical programming formulation might help to evaluate a move when the associated cost is not easily represented through a formula. Still, the option of tackling a problem through a matheuristic comes with a cost. Indeed, embedding the solution of a mathematical programming formulation in a heuristic scheme is time-consuming, even when the solution is fast or when it is called few times. This is why matheuristics may not be suitable for problems in which a decision has to be taken fast, like in real-time and dynamic problems.

New Research Trends

The huge efforts the community has made in past years to solve both exactly and heuristically the CVRP and most if its variants and extensions allow to safely claim that classical variants of the VRP are nowadays efficiently solvable, at least for instances with up to a few hundreds of customers. This has pushed scholars to look for other challenges and explore different research directions. In this section we discuss advances in knowledge discovery and learning approaches to tackle the CVRP and its variants (section "Knowledge Discovery and Learning") as well as the resolution of large-scale instances (section "Heuristic Methods for Large-Scale Instances").

Knowledge Discovery and Learning

In recent years, efforts have been made by the Machine Learning (ML) community to apply learning approaches to solve VRPs as well as efforts by the Operations Research community to integrate ML and Knowledge Discovery mechanisms in heuristics to solve VRPs.

The two communities share different backgrounds and practices which may lead to nonuniform performance evaluation of the proposed procedures. To fill this gap [2] have recently proposed a set of guidelines to conduct computational testing of ML approaches to vehicle routing problems.

Although a complete review of literature on the interactions between both domains is out of the scope of this chapter, we would like to point to some recent contributions that may inspire other researchers.

Knowledge Discovery

To boost the performances of heuristic algorithms, knowledge of some kind of attributes of instances and/or solutions may be collected and used. This is for

example the case in Adaptive Large Neighborhood Search (ALNS) [109] in which the success of competing subheuristics is recorded and used to determine the frequency at which they are used. Specifically for ALNS, it is worth noting that recently [124] questioned the benefits of the adaptive layer to the optimization process. In their discussion the authors conclude that the adaptive layer has most sense to be included in the algorithm if the instances tackled are heterogeneous. In this case, different destroy and repair heuristics target different categories of instances.

In recent years, it has become evident that the efficiency of a heuristic strategy is deeply affected on how well it is able to exploit the characteristics of the problem being solved. Differently said, to increase their efficiency, heuristics require some knowledge on the characteristics that distinguish a good solution from a not-so-good solution.

Whereas it is well known that good solutions share a common structure, it is not clear how to differentiate good from not-so-good solutions. Different attempts have been recently made by researchers to fill this gap, that is to investigate whether close-to-optimal solutions can be distinguished from not-so-close-to-optimal ones by looking at the structure of the solutions, disregarding their objective function values. The goal is then to guide the search based on this information, as for example by removing solution attributes that do not frequently appear in close-to-optimal solutions and replacing them with attributes that do. One successful and well-known example of this approach can be found in [122], where the authors observed that long arcs have a low chance to be part of high-quality solutions. A different approach, using similar ideas, can be found in the matheuristic approach of [9], already discussed in section "Improvement Heuristics."

Recent advancement in computation power and storage opportunities allow to examine a large amount of data, thus making data mining a viable tool. In this regard, [14] attempt to determine metrics which allow to distinguish near-optimal solutions from nonoptimal solutions. They generated 192,000 VRP solutions for which several metrics are computed. The solutions, their metrics, and a flag to characterize them as near- or nonoptimal lead to a data set that is used to train and validate a predictive model. Based on the class of the instances, a solution can be predicted to be a near- or a nonoptimal solution with an accuracy that can reach 90%. The authors show how the knowledge that is obtained can be used in a Guided Local Search algorithm to penalize edges that are likely to belong to nonoptimal solutions [13].

In the same spirit, [12] introduce pattern injection local search (PILS), an optimization strategy that uses pattern mining to discover frequent patterns in high-quality solutions. During local search, attempts of injecting such patterns in the solution are made. The authors propose a procedure to optimally reconnect the route fragments that occur during the pattern injection phase.

Machine Learning

In recent years, the Operations Research community gained interest in the exploitation of ML tools to solve combinatorial optimization problems in general [21, 70] and the VRP in particular [64, 91, 106, 135].

Routing problems have also been the subject of such studies from the ML community, and these studies can be divided in two categories: a category of works that aim at learning construction heuristics and a category that aims at learning improvement heuristics. The works of [20, 73] are of the first category. They make use of Reinforcement Learning to build heuristic solutions for different routing problems. On the improvement heuristics side, e.g., [137] propose a Deep Reinforcement Learning framework to automatically learn high-quality solution picking policies that work with pairwise operators such as 2-opt, exchange, and relocation of customers. It is worth noting that most of these works seem to focus on presenting frameworks that can automatically learn how to build or how to improve solutions without human guidance. Computational experiments are typically run on both the TSP and (variants of) the VRP. The results that are obtained are comparable to those obtained by highly optimized and specialized traditional solvers.

Learning has especially been proven to be a powerful tool to tackle dynamic and stochastic optimization problems. Recent examples from the vehicle routing literature are the works of [16], which considers the dynamic VRPTW, and [58], which considers the IRP with stochastic demands. In these contributions, the ML pipeline incorporates a combinatorial optimization layer. Roughly speaking, the ML layer incorporates uncertainty by parameterizing in an adequate way an instance of the underlying deterministic optimization problem. Such approaches may then be very useful to tackle systems that are highly dynamic or stochastic as for example the e-commerce sector, as mentioned in [16].

On the other side, it has also been shown that results obtained by introducing information that comes from sophisticated ML tools can sometimes be obtained from simpler considerations. For example, [111] integrate measures of customer relatedness in the local search and crossover procedures of HGS to prune the neighborhood exploration and to reconnect chromosomes. They show that such relatedness information is beneficial to the performance of the algorithm, regardless the source of the information (simple distance measures or heatmaps resulting from a graph neural network). However, both approaches lead to equally good solutions, i.e., simple considerations based on the distance between customers are sufficient to achieve very good results.

Heuristic Methods for Large-Scale Instances

The computational effort required to solve vehicle routing problems increases with problem size, no matter whether exact or heuristic approaches are considered. In most cases, the number of customers or nodes in a problem is used as a proxy for the problem size.

The size of the problem instances to address obviously depends on the specific problem considered and its (potential) real-life application. However, computational limitations may also force scholars to reduce instance sizes. Early research on vehicle routing problems therefore primarily focused on problems of up to at most a few hundreds of customers. For example, commonly used sets of benchmark

instances for classical problems like the CVRP and VRPTW contain instances of only up to 200 or 100 customers, respectively [33, 34, 119].

Later, at the start of the 2000s, the interest in solving larger instances grew, driven by increased computational resources and the increasing popularity of powerful metaheuristic solution frameworks compared to simple heuristic procedures. At that time, it became common to address instances of a few hundreds of customers and several new benchmark sets were proposed (e.g., CVRP instances of up to 483 and 1200 customers by [57] and [84], respectively, and VRPTW and PDPTW instances of up to 1000 customers by [52] and [85], respectively). Later, another extensive set of instances of 1000 customers for the CVRP was proposed by [125].

In recent years, there is a growing interest in even larger, the so-called very large-scale instances of tens of thousands of customers. The motivation for such large problems stems from real-life applications like waste collection, mail delivery, and urban e-commerce deliveries [11, 74].

To avoid excessive computation times for such problems, various acceleration techniques have been proposed. Generally, the difficulty lies in finding a good balance between efficiency and efficacy of local search procedures. Common techniques include filtering the neighbors considered (e.g., using granular neighborhoods [18, 122]), early pruning of certain moves (e.g., Sequential Search [69]), and using smart move evaluation procedures (e.g., Static Move Descriptors [19, 139]). In addition, there is a focus on reducing memory usage.

Another direction to tackle large-scale problems is to apply decomposition techniques [112]. Here, smaller subproblems are iteratively solved and their solutions are used to build up a global solution. Among the difficulties of such methods we can mention the design of the strategy to define the subproblems as well as to determine their dimension in order to end up with subproblems that allow to build parts of a solution while keeping the computational time reasonable.

In terms of (very) large-scale problems, we highlight a few recent contributions. [74] address problems of up to 20,000 customers. They present a Guided Variable Neighborhood Search heuristic algorithm which combines VNS with ideas from GLS. They report on several speed-up techniques and a specific data structure to reduce computation times and memory usage. Arnold et al. [11] introduce new instances of up to 30,000 customers, based on real-life parcel distribution data from Belgium and extend the algorithm in [13] to solve them. They demonstrate that in good-quality solutions nodes are nearly always connected to one of their closest neighbors. This information is used to prune the search of their local search operators, reducing computation without impacting solution quality. Additionally, to reduce memory usage, they propose to only store distance information on the shortest edges and compute distances of other edges on the fly. Accorsi and Vigo [3] propose an ILS algorithm that at each iteration maintains the optimization to the area that has been affected by the shaking step. This is accomplished through a heuristic pruning technique that uses a dedicated data structure to keep track of a limited set of vertices involved in recent solution changes. In [4], the same authors propose an extension of their algorithm which implements advanced techniques to reduce memory usage and the complexity of the neighborhood explorations. They tested the

algorithm on newly proposed instances with up to 1 million customers. Recently, as discussed in section "Iterated Local Search," [93] present a highly effective heuristic algorithm, providing state-of-the-art results on the instances of [11] with 30,000 customers.

Finally, note that tackling very large-scale instances is not a pure academic challenge but may have specific industry applications. For example, [5, 26] present a real-life, complex IRP problem with dimensions that go well beyond classical IRP benchmark instances. The problem presented in [5] considers instances with up to 324 customers, 21 planning days, and 13 drivers, while [26] tackle instances with about 600 customers, 30 commodities, 16 depots, and 21 planning days.

Instances and Libraries

In this section we give a brief overview on libraries and benchmark instances available in the literature for different classes of routing problems.

Instances

The availability of good benchmark instances plays a crucial role in research on vehicle routing heuristics as they provide a common ground for comparing different algorithms and approaches. However, it is not so obvious what determines the *goodness* of set of instances. Elements to consider are for example the level of *realism* or their representativity of real-life problems (e.g., in terms of instance size, clustering of locations, network topology, side constraints like loads, time windows, etc.), and the level of diversity.

Unfortunately, research on heuristics for classical vehicle problems has mainly been using the same sets of early artificial benchmark instances which lack realism and diversity. These include for example instances of [33, 34, 57, 84] for the CVRP, [52, 119] for the VRPTW, and [85, 108] for the PDPTW.

More recently, several researchers have proposed (ways to generate) more realistic sets of benchmark instances for these problems (see, e.g., [113, 125]). In addition, two new sets of instances based on real-life data for the CVRP were introduced in the context of the 12th DIMACS Implementation Challenge (http://dimacs.rutgers.edu/programs/challenge/vrp/), and a set of more than 9000 historical routes with diverse features was made available in the context of the 2021 Amazon Last Mile Routing Research Challenge [94]. Additionally, [4] proposed instances with up to 1 million customers. For a more detailed overview and discussion on instances for the CVRP, we refer to [125] and to the CVRPLIB repository on which instances and results are publicly available (http://vrp.galgos.inf.puc-rio.br).

For other, more specific and less commonly studied problems, researchers typically either extend instances of the classical benchmarks, or randomly generate their own instances, based on real-life data or not. For a discussion, see [61]. In recent years, there seems to be a growing request of the community to make

such problem instances publicly available for fellow researchers. Unfortunately, this process is not very streamlined. Instances are made available in different ways (e.g., personal websites, local repositories, publishers' repositories, etc.), in diverse formats, and often with limited explanation. There have been some efforts by the community to offer a more structured approach, like VRP-REP (http://www.vrp-rep.org/), an online repository to which researchers can contribute their instances and solutions using a standardized format. However, the uptake is unfortunately limited.

Publicly Available Implementations

Developing effective and efficient (heuristic) algorithms is often the primary concern of researchers when addressing vehicle routing problems. However, as the quality of heuristic algorithms is typically assessed experimentally, rather than theoretically, implementing such algorithms is another important aspect. Efficient implementations are often crucial to compete in the highly competitive research area. A discussion of best practices in this context is beyond the scope of this chapter. However, we do want to highlight a number of contributions that can be a starting point when tackling vehicle routing problems.

First, there are publicly available tools that allow to solve routing problems without any knowledge of programming or even heuristics. An example is the VRP Spreadsheet Solver developed by Güneş Erdoğan [46]. It applies the Large Neighborhood Search framework to solve many VRP variants with up to 200 customers, uses GIS to obtain real-life distances and travel times, and allows visualization of solutions. Second, in recent years, several researchers have made their code publicly available. An early example is the library coded in C/C++ presented in [59]. More recent examples include open-source implementations of state-of-the-art algorithms like the HGS algorithm of [128, 130] for the CVRP ([127], coded in C++), and for CVRP and VRPTW ([136], coded in Python), of the hybrid ILS-SP algorithm of [121] for the CVRP (https://github.com/cvneves/ILS-SP-CVRP), and of the LKH-3 algorithm of [63] (http://webhotel4.ruc.dk/~keld/research/LKH-3/) which can be applied to a large variety of problems, including the CVRP, the VRPTW, and pickup-and-delivery problems. Also, for more specific routing problems, packages/libraries are available, like `RoutingBlocks`, a Python package for developing algorithms for routing problems with intermediate stops [72].

Conclusion

In this chapter, we have provided an overview of heuristic methods for vehicle routing problems. Due to the large amount of research available on the topic, we briefly introduced the earliest and most classical heuristic algorithms for routing problems. We then concentrated on the most recent developments which have

been proposed to tackle routing problems heuristically, namely, metaheuristics, matheuristics, learning approaches, and heuristics for large-scale problems. We have provided reference contributions and discussed the key features of each class, concentrating on the most recent research trends.

Advances in both exact and heuristic methods allow to claim that classical variants of the VRP are nowadays efficiently solvable, at least for instances with up to few hundreds of customers. Challenges remain open on how to efficiently tackle the very large instances, where studies are present, but mostly limited to the CVRP.

Scholars have studied the benefits of including knowledge discovery and ML tools into the resolution of routing problems. While it is clear that including knowledge of some kind in the procedures positively impacts the quality of the outcome, further research is still needed to shed light on for which problems ML is of most interest. Indeed, studies show a positive impact of the use of ML to problems with dynamic or stochastic dimensions, while a high impact of ML to tackle normal size instances for deterministic problems has not yet been demonstrated. In the latter case, classical optimization procedures remain competitive. On the other hand, an open direction for further research is to investigate the benefits of learning to tackle large-scale instances. Works on this direction have been already conducted (see, for example, [66, 86]) but are still limited.

Finally, a discussion which is there since decades is related to the benefit of developing unified approaches which can effectively handle different problem variants. Some effort has been done, as discussed above, but still scientific contributions tend to propose approaches which are strongly tailored to the specific problem at hand. Evidence from practice instead shows the growing development of unified frameworks, proposed by consultancy companies specialized in optimization and routing. Thus, we believe some stronger effort of the research community should be made in this direction.

Competing Interest Declaration The author(s) has no competing interests to declare that are relevant to the content of this manuscript.

References

1. Absi N, Archetti C, Dauzère-Pérès S, Feillet D (2015) A two-phase iterative heuristic approach for the production routing problem. Transp Sci 49(4):784–795
2. Accorsi L, Lodi A, Vigo D (2022) Guidelines for the computational testing of machine learning approaches to vehicle routing problems. Oper Res Lett 50:229–234
3. Accorsi L, Vigo D (2021) A fast and scalable heuristic for the solution of large-scale capacitated vehicle routing problems. Transp Sci 55(4):832–856
4. Accorsi L, Vigo D (2024) Routing one million customers in a handful of minutes. Comput Oper Res 164:106562
5. André J, Bourreau E, Wolfler Calvo R (2020) Introduction to the special section: Roadef/euro challenge 2016—inventory routing problem. Transp Sci 54(2):299–301
6. Aranha C, Camacho Villalón CL, Campelo F, Dorigo M, Ruiz R, Sevaux M, Sörensen K, Stützle T (2022) Metaphor-based metaheuristics, a call for action: the elephant in the room. Swarm Intell 16:1–6

7. Archetti C, Bertazzi L, Hertz A, Speranza MG (2012) A hybrid heuristic for an inventory routing problem. INFORMS J Comput 24(1):101–116
8. Archetti C, Guastaroba G, Huerta-Muñoz DL, Speranza MG (2021) A kernel search heuristic for the multivehicle inventory routing problem. Int Trans Oper Res 28(6):2984–3013
9. Archetti C, Guastaroba G, Speranza M (2014) An ILP-refined tabu search for the directed profitable rural postman problem. Discret Appl Math 163:3–16
10. Archetti C, Speranza MG (2014) A survey on matheuristics for routing problems. EURO J Comput Optim 2(4):223–246
11. Arnold F, Gendreau M, Sörensen K (2019) Efficiently solving very large-scale routing problems. Comput Oper Res 107:32–42
12. Arnold F, Santana I, Sörensen K, Vidal T (2021) PILS: exploring high-order neighborhoods by pattern mining and injection. Pattern Recogn 116:1–10
13. Arnold F, Sörensen K (2019) Knowledge-guided local search for the vehicle routing problem. Comput Oper Res 105:32–46
14. Arnold F, Sörensen K (2019) What makes a VRP solution good? The generation of problem-specific knowledge for heuristics. Comput Oper Res 106:280–288
15. Badeau P, Guertin F, Gendreau M, Potvin J-Y (1997) A parallel tabu search heuristic for the vehicle routing problem with time windows. Transp Res Part C 5(2):109–122
16. Baty L, Jungel K, Klein PS, Parmentier A, Schiffer M (2024) Combinatorial optimization-enriched machine learning to solve the dynamic vehicle routing problem with time windows. Transp Sci 58(4):685–918
17. Beasly JE (1983) Route first-cluster second methods for vehicle routing. Omega 11(4):403–408
18. Becker C, Gauthier JB, Gschwind T, Schneider M (2023) In-depth analysis of granular local search for capacitated vehicle routing. Discret Appl Math 329:61–86
19. Beek O, Raa B, Dullaert W, Vigo D (2018) An efficient implementation of a static move descriptor-based local search heuristic. Comput Oper Res 94:1–10
20. Bello I, Pham H, Le QV, Norouzi M, Bengio S (2017) Neural combinatorial optimization with reinforcement learning. In: 5th international conference on learning representations, ICLR 2017
21. Bengio Y, Lodi A, Prouvost A (2021) Machine learning for combinatorial optimization: a methodological tour d'horizon. Eur J Oper Res 290(2):405–421
22. Bertsekas DP, Tsitsiklis JN, Wu C (1997) Rollout algorithms for combinatorial optimization. J Heurist 3:245–262
23. Boschetti MA, Letchford AN, Maniezzo V (2023) Matheuristics: survey and synthesis. Int Trans Oper Res 30(6):2840–2866
24. Boschetti MA, Maniezzo V (2022) Matheuristics: using mathematics for heuristic design. 4OR 20(2):173–208
25. Boschetti MA, Maniezzo V, Roffilli M, Bolufé Röhler A (2009) Matheuristics: optimization, simulation and control. In: International workshop on hybrid metaheuristics. Springer, pp 171–177
26. Bouvier L, Dalle G, Parmentier A, Vidal T (2023) Solving a continent-scale inventory routing problem at Renault. Transp Sci 58(1):131–151
27. Braekers K, Ramaekers K, Van Nieuwenhuyse I (2016) The vehicle routing problem: state of the art classification and review. Comput Ind Eng 99:300–313
28. Bräysy O, Gendreau M (2005) Vehicle routing problem with time windows, part I: route construction and local search algorithms. Transp Sci 39(1):104–118
29. Bräysy O, Gendreau M (2005) Vehicle routing problem with time windows, part II: metaheuristics. Transp Sci 39(1):119–139
30. Burke EK, Hyde MR, Kendall G, Ochoa G, Özcan E, Woodward JR (2019) A classification of hyper-heuristic approaches: revisited. Springer International Publishing, Cham, pp 453–477
31. Campbell AM, Savelsbergh M (2004) Efficient insertion heuristics for vehicle routing and scheduling problems. Transp Sci 38(3):369–368

32. Christiaens J, Vanden Berghe G (2020) Slack induction by string removals for vehicle routing problems. Transp Sci 54(2):417–433
33. Christofides N, Eilon S (1969) An algorithm for the vehicle-dispatching problem. J Oper Res Soc 20(3):309–318
34. Christofides N, Mingozzi A, Toth P (1979) The vehicle routing problem. In: Christofides N, Mingozzi A, Toth P, Sandi C (eds) Combinatorial optimization. Wiley, Chichester, pp 315–338
35. Clarke G, Wright J (1964) Scheduling of vehicles from a central depot to a number of delivery points. Oper Res 12(4):568–581
36. Cordeau J-F, Laporte G, Mercier A (2001) A unified tabu search heuristic for vehicle routing problems with time windows. J Oper Res Soc 52(8):928–936
37. Cordeau J-F, Laporte G, Savelsbergh MW, Vigo D (2007) Chapter 6: Vehicle routing. In: Barnhart C, Laporte G (eds) Transportation. Handbooks in operations research and management science, vol 14. Elsevier, Amsterdam, pp 367–428
38. Curtois T, Landa-Silva D, Qu Y, Laesanklang W (2018) Large neighbourhood search with adaptive guided ejection search for the pickup and delivery problem with time windows. EURO J Transp Logist 7:151–192
39. Dantzig G, Ramser J (1959) The truck dispatching problem. Manag Sci 6(1):80–91
40. Delaet A, Ramaekers K, Hirsch P, Molenbruch Y, Braekers K (2024) A matheuristic for integrated medium-term home healthcare planning. Eur J Oper Res 319(2):543–556
41. Desaulniers G, Desrosiers J, Solomon MM (2006) Column generation, vol 5. Springer Science & Business Media, New York
42. Desaulniers G, Madsen OB, Røpke S (2014) Chapter 5: The vehicle routing problem with time windows. In: Toth P, Vigo D (eds) Vehicle routing: problems, methods, and applications, 2nd edn. SIAM, Philadelphia, pp 119–159
43. Dokeroglu T, Kucukyilmaz T, Talbi E-G (2024) Hyper-heuristics: a survey and taxonomy. Comput Ind Eng 187:109815
44. Dumez D, Lehuédé F, Péton O (2021) A large neighborhood search approach to the vehicle routing problem with delivery options. Transp Res Part B: Methodol 114(2021):103–132
45. Eksioglu B, Vural AV, Reisman A (2009) The vehicle routing problem: a taxonomic review. Comput Ind Eng 57(4):1472–1483
46. Erdoğan G (2017) An open source spreadsheet solver for vehicle routing problems. Comput Oper Res 84:62–72
47. Federgruen A, Zipkin P (1984) A combined vehicle routing and inventory allocation problem. Oper Res 32(5):1019–1037
48. Fischetti M, Fischetti M (2018) Matheuristics. In: Martí R, Pardalos PM, Resende MGC (eds) Handbook of heuristics. Springer International Publishing, Cham, pp 121–153
49. Fisher ML, Jaikumar R (1981) A generalized assignment heuristic for vehicle routing. Networks 11(2):109–124
50. Foster BA, Ryan DM (1976) An integer programming approach to the vehicle scheduling problem. J Oper Res Soc 27:367–384
51. Franceschi RD, Fischetti M, Toth P (2006) A new ILP-based refinement heuristic for vehicle routing problems. Math Program 105:471–499
52. Gehring H, Homberger J (1999) A parallel hybrid evolutionary metaheuristic for the vehicle routing problem with time windows. In: Miettinen K, Makela M, Toivanen J (eds) Proceeding of EUROGEN99—short course on evolutionary algorithms in engineering and computer science. University of Jyväskylä, pp 57–64
53. Gendreau M, Hertz A, Laporte G (1994) A tabu search heuristic for the vehicle routing problem. Manag Sci 40(10):1276–1290
54. Gillet BE, Miller LR (1974) A heuristic algorithm for the vehicle dispatch problem. Oper Res 22(2):340–349
55. Gobbi A, Manerba D, Mansini R, Zanotti R (2023) Hybridizing adaptive large neighborhood search with kernel search: a new solution approach for the nurse routing problem with incompatible services and minimum demand. Int Trans Oper Res 30(1):8–38

56. Golden BL, Raghavan S, Wasil EA (2008) The vehicle routing problem: latest advances and new challenges. Springer, New York
57. Golden BL, Wasil EA, Kelly JP, Chao I-M (1998) The impact of metaheuristics on solving the vehicle routing problem: Algorithms, problem sets, and computational results. In: Crainic TG, Laporte G (eds) Fleet management and logistics. Springer, Boston, pp 33–56
58. Greif T, Bouvier L, Flath CM, Parmentier A, Rohmer SU, Vidal T (2024) Combinatorial optimization and machine learning for dynamic inventory routing. arXiv preprint arXiv:2402.04463
59. Gröer C, Golden B, Wasil E (2010) A library of local search heuristics for the vehicle routing problem. Math Program Comput 13(11):79–101
60. Guastaroba G, Speranza MG (2012) Kernel search: an application to the index tracking problem. Eur J Oper Res 217(1):54–68
61. Gunawan A, Kendall G, McCollum B, Seow H-V, Lee LS (2021) Vehicle routing: review of benchmark datasets. J Oper Res Soc 72(8):1794–1807
62. Helsgaun K (2009) General *k*-opt submoves for the Lin–Kernighan TSP heuristic. Math Program Comput 1:119–163
63. Helsgaun K (2017) An extension of the Lin-Kernighan-Helsgaun TSP solver for constrained traveling salesman and vehicle routing problems. Technical report, Roskilde University
64. Hildebrandt FD, Thomas BW, Ulmer MW (2023) Opportunities for reinforcement learning in stochastic dynamic vehicle routing. Comput Oper Res 150:106071
65. Hoogeboom M, Dullaert W, Lai D, Vigo D (2020) Efficient neighborhood evaluations for the vehicle routing problem with multiple time windows. Transp Sci 54(2):400–416
66. Hou Q, Yang J, Su Y, Wang X, Deng Y (2023) Generalize learned heuristics to solve large-scale vehicle routing problems in real-time. In: The eleventh international conference on learning representations
67. Hvattum LM (2022) Adjusting the order crossover operator for capacitated vehicle routing problems. Comput Oper Res 148:105986
68. Irnich S (2008) A unified modeling and solution framework for vehicle routing and local search-based metaheuristics. INFORMS J Comput 20(2):270–287
69. Irnich S, Funke B, Grünert T (2006) Sequential search and its application to vehicle-routing problems. Comput Oper Res 33(8):2405–2429
70. Karimi-Mamaghan M, Mohammadi M, Meyer P, Karimi-Mamaghan AM, Talbi E-G (2022) Machine learning at the service of meta-heuristics for solving combinatorial optimization problems: a state-of-the-art. Eur J Oper Res 296(2):393–422
71. Kinene A, Birolini S, Cattaneo M, Granberg TA (2023) Electric aircraft charging network design for regional routes: a novel mathematical formulation and kernel search heuristic. Eur J Oper Res 309(3):1300–1315
72. Klein PS, Schiffer M (2024) RoutingBlocks: an open-source Python package for vehicle routing problems with intermediate stops. INFORMS J Comput 36(4):966–973
73. Kool W, van Hoof H, Welling M (2019) Attention, learn to solve routing problems! In: 7th international conference on learning representations, ICLR 2019
74. Kytöjoki J, Nuortio T, Bräysy O, Gendreau M (2007) An efficient variable neighborhood search heuristic for very large scale vehicle routing problems. Comput Oper Res 34(9):2743–2757
75. Labadie N, Prins C, Prodhon C (2016) Metaheuristics for vehicle routing problems. Wiley, London & Hoboken
76. Lahyani R, Khemakhem M, and Semet F (2015) Rich vehicle routing problems: from a taxonomy to a definition. Eur J Oper Res 241(1):1–14
77. Lancia G, Dalpasso M (2020) Finding the best 3-opt move in subcubic time. Algorithms 13(11):306
78. Lancia G, Dalpasso M (2024) Algorithmic strategies for a fast exploration of the TSP 4-OPT neighborhood. J Heurist 30:109–144
79. Lancia G, Vidoni P (2024) Average case subquadratic exact and heuristic procedures for the traveling salesman 2-OPT neighborhood. INFORMS J Comput Ahead of Print https://doi.org/10.1287/ijoc.2023.0169

80. Laporte G (2009) Fifty years of vehicle routing. Transp Sci 43(4):408–416
81. Laporte G, Røpke S, Vidal T (2014) Chapter 4: Heuristics for the vehicle routing problem. In: Toth P, Vigo D (eds) Vehicle routing: problems, methods, and applications, 2nd edn. SIAM, Philadelphia, pp 87–116
82. Laporte G, Semet F (2002) Chapter 5: Classical heuristics for the capacitated VRP. In: Toth P, Vigo D (eds) The vehicle routing problem, 1st edn. SIAM, Philadelphia, pp 109–128
83. Le Colleter T, Dumez D, Lehuédé F, Péton O (2023) Small and large neighborhood search for the park-and-loop routing problem with parking selection. Eur J Oper Res 308(3):1233–1248
84. Li F, Golden B, Wasil E (2005) Very large-scale vehicle routing: new test problems, algorithms, and results. Comput Oper Res 32(5):1165–1179
85. Li H, Lim A (2001) A metaheuristic for the pickup and delivery problem with time windows. In: Proceedings 13th IEEE international conference on tools with artificial intelligence, ICTAI 2001, pp 160–167
86. Li S, Yan Z, Wu C (2021) Learning to delegate for large-scale vehicle routing. In: 35th conference on neural information processing systems, NeurIPS 2021
87. Lin S, Kernighan BW (1973) An effective heuristic algorithm for the traveling-salesman problem. Oper Res 21(2):498–516
88. Liu F, Lu C, Gui L, Zhang Q, Tong X, Yuan M (2023) Heuristics for vehicle routing problem: a survey and recent advances. arXiv preprint arXiv:2303.04147
89. Lourenço HR, Martin OC, Stützle T (2003) Iterated local search. Springer, Boston, pp 320–353
90. Lübbecke ME, Desrosiers J (2005) Selected topics in column generation. Oper Res 53(6):1007–1023
91. Maia MRDH, Plastino A, Souza UDS (2024) An improved hybrid genetic search with data mining for the CVRP. Networks 83(4):692–711
92. Maniezzo V, Boschetti MA, Stützle T (2021) Kernel search. In: Matheuristics: algorithms and implementations. Springer, Boston, pp 189–197
93. Máximo VR, Cordeau J-F, Nascimento MCV (2024) AILS-II: an adaptive iterated local search heuristic for the large-scale capacitated vehicle routing problem. INFORMS J Comput 36(4):974–986
94. Merchán D, Arora J, Pachon J, Konduri K, Winkenbach M, Parks S, Noszek J (2024) 2021 Amazon last mile routing research challenge: data set. Transp Sci 58(1):8–11
95. Mor A, Speranza M (2022) Vehicle routing problems over time: a survey. Ann Oper Res 314:255–275
96. Máximo VR, Nascimento MC (2021) A hybrid adaptive iterated local search with diversification control to the capacitated vehicle routing problem. Eur J Oper Res 294(3):1108–1119
97. Nagata Y, Bräysy O, Dullaert W (2010) A penalty-based edge assembly memetic algorithm for the vehicle routing problem with time windows. Comput Oper Res 37:724–727
98. Neri F, Cotta C (2012) Memetic algorithms and memetic computing optimization: a literature review. Swarm Evol Comput 2:1–14
99. Pacheco T, Martinelli R, Subramanian A, Toffolo TAM, Vidal T (2023) Exponential-size neighborhoods for the pickup-and-delivery traveling salesman problem. Transp Sci 57(2):463–481
100. Penna PHV, Subramanian A, Ochi LS (2013) An iterated local search heuristic for the heterogeneous fleet vehicle routing problem. J Heurist 19:201–232
101. Pisinger D, Ropke S (2007) A general heuristic for vehicle routing problems. Comput Oper Res 34(8):2403–2435
102. Pisinger D, Ropke S (2019) Large neighborhood search. In: Gendreau M, Potvin J-Y (eds) Handbook of metaheuristics. Springer International Publishing, Cham, pp 99–127
103. Prins C (2004) A simple and effective evolutionary algorithm for the vehicle routing problem. Comput Oper Res 31:1985–2002
104. Puchinger J, Raidl GR (2005) Combining metaheuristics and exact algorithms in combinatorial optimization: a survey and classification. In: International work-conference on the interplay between natural and artificial computation. Springer, pp 41–53

105. Rakke JG, Stålhane M, Moe CR, Christiansen M, Andersson H, Fagerholt K, Norstad I (2011) A rolling horizon heuristic for creating a liquefied natural gas annual delivery program. Transp Res Part C: Emerg Technol 19(5):896–911
106. Raza SM, Sajid M, Singh J (2022) Vehicle routing problem using reinforcement learning: recent advancements. In: Gupta D, Sambyo K, Prasad M, Agarwal S (eds) Advanced machine intelligence and signal processing. Springer, Singapore, pp 269–280
107. Rocha Y, Subramanian A (2023) Hybrid genetic search for the traveling salesman problem with hybrid electric vehicle and time windows. Comput Oper Res 155:106223
108. Ropke S, Cordeau J-F (2009) Branch and cut and price for the pickup and delivery problem with time windows. Transp Sci 43(3):267–286
109. Ropke S, Pisinger D (2006) An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows. Transp Sci 40(4):455–472
110. Ropke S, Pisinger D (2006) A unified heuristic for a large class of vehicle routing problems with backhauls. Eur J Oper Res 171(3):750–775
111. Santana Í, Lodi A, Vidal T (2023) Neural networks for local search and crossover in vehicle routing: a possible overkill? In: Cire AA (ed) Integration of constraint programming, artificial intelligence, and operations research. Springer Nature, Switzerland, pp 184–199
112. Santini A, Schneider M, Vidal T, Vigo D (2023) Decomposition strategies for vehicle routing heuristics. INFORMS J Comput 35(3):543–559
113. Sartori CS, Buriol LS (2020) A study on the pickup and delivery problem with time windows: matheuristics and new instances. Comput Oper Res 124:105065
114. Savelsbergh M (1985) Local search in routing problems with time windows. Ann Oper Res 4:285–305
115. Schrimpf G, Schneider J, Stamm-Wilbrandt H, Dueck G (2000) Record breaking optimization results using the ruin and recreate principle. J Comput Phys 159(2):139–171
116. Shaw P (1998) Using constraint programming and local search methods to solve vehicle routing problems. In: Maher M, Puget J-F (eds) Principles and practice of constraint programming—CP98. Springer, Berlin/Heidelberg, pp 417–431
117. Simensen M, Hasle G, Stalhane M (2022) Combining hybrid genetic search with ruin-and-recreate for solving the capacitated vehicle routing problem. J Heurist 28:653–697
118. Skålnes J, Vadseth ST, Andersson H, Stålhane M (2023) A branch-and-cut embedded matheuristic for the inventory routing problem. Comput Oper Res 159:106353
119. Solomon M (1987) Algorithms for the vehicle routing and scheduling problems with time window constraints. Oper Res 35(2):254–265
120. Sörensen K (2015) Metaheuristics—the metaphor exposed. Int Trans Oper Res 22(1):3–18
121. Subramanian A, Uchoa E, Ochi LS (2013) A hybrid algorithm for a class of vehicle routing problems. Comput Oper Res 40(10):2519–2531
122. Toth P, Vigo D (2003) The granular tabu search and its application to the vehicle-routing problem. INFORMS J Comput 15(4):333–346
123. Toth P, Vigo D (2014) Vehicle routing: problems, methods, and applications, 2nd edn. SIAM, Philadelphia
124. Turkeš R, Sörensen K, Hvattum LM (2021) Meta-analysis of metaheuristics: quantifying the effect of adaptiveness in adaptive large neighborhood search. Eur J Oper Res 292(2):423–442
125. Uchoa E, Pecin D, Pessoa A, Poggi M, Vidal T, Subramanian A (2017) New benchmark instances for the capacitated vehicle routing problem. Eur J Oper Res 257(3):845–858
126. Vidal T (2016) Technical note: split algorithm in $O(n)$ for the capacitated vehicle routing problem. Comput Oper Res 69:40–47
127. Vidal T (2022) Hybrid genetic search for the CVRP: open-source implementation and SWAP* neighborhood. Comput Oper Res 140:105643
128. Vidal T, Crainic TG, Gendreau M, Lahrichi N, Rei W (2012) A hybrid genetic algorithm for multidepot and periodic vehicle routing problems. Oper Res 60(3):611–624
129. Vidal T, Crainic TG, Gendreau M, Prins C (2013) Heuristics for multi-attribute vehicle routing problems: a survey and synthesis. Eur J Oper Res 231(1):1–21

130. Vidal T, Crainic TG, Gendreau M, Prins C (2013) A hybrid genetic algorithm with adaptive diversity management for a large class of vehicle routing problems with time-windows. Comput Oper Res 40(1):475–489
131. Vidal T, Crainic TG, Gendreau M, Prins C (2014) A unified solution framework for multi-attribute vehicle routing problems. Eur J Oper Res 234(3):658–673
132. Vidal T, Crainic TG, Gendreau M, Prins C (2015) Timing problems and algorithms: time decisions for sequences of activities. Networks 65(2):102–128
133. Vidal T, Laporte G, Matl P (2020) A concise guide to existing and emerging vehicle routing problem variants. Eur J Oper Res 286(2):401–416
134. Weyland D (2015) A critical analysis of the harmony search algorithm—how not to solve sudoku. Oper Res Perspect 2:97–105
135. Winkenbach M, Spinler S, Pachon J, Konduri K (2024) Introduction to the special issue on machine learning methods and applications in large-scale route planning problems. Transp Sci 58(1):1–7
136. Wouda NA, Lan L, Kool W (2024) PyVRP: a high-performance VRP solver package. INFORMS J Comput 36(4):939–1146
137. Wu Y, Song W, Cao Z, Zhang J, Lim A (2022) Learning improvement heuristics for solving routing problems. IEEE Trans Neural Netw Learn Syst 33(9):5057–5069
138. Yuan Y, Cattaruzza D, Ogier M, Semet F, Vigo D (2021) A column generation based heuristic for the generalized vehicle routing problem with time windows. Transp Res Part E: Logist Transp Rev 152:102391
139. Zachariadis EE, Kiranoudis CT (2010) A strategy for reducing the computational complexity of local search-based methods for the vehicle routing problem. Comput Oper Res 37(12):2089–2105

Last-Mile Delivery with Crowdshipping

44

Tiago Monteiro, João Pedro Pedroso, and Ana Viana

Contents

Introduction 1386
Last-Mile Delivery with Crowdshipping 1387
Mathematical Formulation 1387
Related Problems and Variants 1389
Metaheuristics Approaches to Related Problems 1391
Iterated Local Search 1392
Variable Neighborhood Search 1396
Large Neighborhood Search 1398
Tabu Search 1400
Genetic and Evolutionary Algorithms 1400
Hybrid Approaches 1402
Other Metaheuristics 1405
Conclusions 1405
Cross-References 1406
References 1406

T. Monteiro (✉)
INESC Technology and Science, Porto, Portugal

Faculty of Engineering, University of Porto, Porto, Portugal
e-mail: tiago.p.monteiro@inesctec.pt

J. P. Pedroso
CMUP and Department of Computer Science, Faculty of Sciences, University of Porto, Porto, Portugal
e-mail: jpp@fc.up.pt

A. Viana
INESC Technology and Science, Porto, Portugal

ISEP, Polytechnic of Porto, Porto, Portugal
e-mail: agv@isep.ipp.pt

© Springer Nature Switzerland AG 2025

R. Martí et al. (eds.), *Handbook of Heuristics*,
https://doi.org/10.1007/978-3-032-00385-0_78

Abstract

Last-mile delivery with crowdshipping is a logistics solution that, besides the store's professional fleet, considers the possibility of having ordinary people with no professional relation with the store (a.k.a. occasional couriers) delivering some packages. The associated problem combines two prominent problems in the literature: routing and assignment. Packages are assigned to occasional couriers, and routes are generated for the store's professional fleet and occasional couriers. In this chapter, we describe the first mathematical model proposed to solve this problem and provide a comprehensive review of metaheuristics used in the literature to address challenges related to last-mile delivery with crowdshipping.

Keywords

Last-mile delivery · Vehicle routing problem · Crowdshipping · Occasional couriers · Metaheuristics

Introduction

E-commerce has transformed global retail, providing customers with unparalleled convenience, variety, and access to nearly every product. The technological advancements associated with changing customer preferences have established e-commerce as the main force shaping the future of retail. Customers now have the power to browse and purchase products from nearly anywhere and at any time using their electronic devices. In order to fulfill customer expectations, retailers' capacity for innovation and adaptation are key factors in their success, since they are forced to rethink their approaches and look into new solutions to remain competitive. Our work focuses on the supply chain's last-mile delivery step, which involves moving goods from distribution centers to customers' houses.

One innovative solution in last-mile delivery is based on crowdsourcing. In this concept, organizations decentralize delivery and outsource it, totally or partially, to a wide network of contributors. This allows organizations to access a wide range of skills and labor at reduced costs compared to traditional methods. Crowdsourcing has broad applications, and, when applied to last-mile delivery, it is named *crowdshipping*. In this case, the "crowd" (a.k.a. occasional couriers—OCs) are ordinary people with no professional relation with the retailers, who agree to deliver goods to the final customer for financial compensation.

Platforms such as Amazon Flex [1] and Roadie [36] are examples of crowdshipping platforms. They leverage nonprofessional couriers to carry out last-mile deliveries, allowing companies to expand their delivery capacity. Amazon Flex enables individuals to use their vehicles to deliver parcels on a flexible schedule, while Roadie connects businesses and individuals with couriers who are already en route to their delivery destinations. Both services exemplify the growth of gig economy-based logistics, which can offer cost-saving benefits for companies and flexible earning opportunities for couriers [38].

This study conducts a comprehensive analysis of academic literature concerning crowdsourcing in the last-mile delivery context. It specifically emphasizes works that incorporate metaheuristics into their proposed solutions. The remainder of this chapter introduces an optimization problem associated with last-mile delivery using crowdshipping. It provides an overview of the main scientific challenges in this area and analyzes the metaheuristics developed to address the different problem variants.

Last-Mile Delivery with Crowdshipping

In this section, we outline the problem of last-mile delivery with crowdshipping and present the mathematical model for its basic configuration. We then provide a literature overview, highlighting the key lines of research and advancements in crowdsourced last-mile delivery solutions.

The problem, first addressed in [2], and named vehicle routing problem with occasional drivers (VRPOD), can be described as follows: Given a set of online orders to be fulfilled within a specified day or time period and a workforce composed of a professional fleet (PF) and occasional couriers, we aim to find the orders assignment and delivery routes that minimize total delivery costs. The solution's total cost comprises the cost of professional fleet routes and the compensations paid to occasional couriers. We assume that each delivery route must start from a store/depot, and both professional vehicles and occasional couriers have a capacity limit restricting the number of deliveries they can handle.

Figure 1 illustrates our case. Here, the store/depot has four online orders to fulfill. Three customers are served by a professional fleet vehicle, which starts and ends its route at the store. An occasional courier handles the remaining order. This OC picks up the package at the store and incorporates the delivery into their preexisting route home, making only a slight detour to complete the task.

Mathematical Formulation

The vehicle routing problem with occasional drivers is modeled as a combination of an assignment problem (deliveries are assigned to occasional couriers) and a

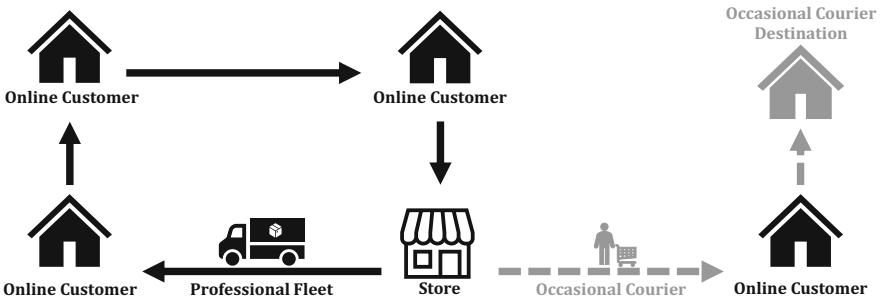


Fig. 1 Example of crowdshipping scheme

capacitated vehicle routing problem. The authors in [2] proposed a static integer programming formulation for this problem and formulated it as follows. Let $G = (N, A)$ represent a complete directed graph, where N is the set of nodes and A is the set of arcs. The node set N comprises three subsets: the store or depot (node 0), customer locations C, and destinations for occasional couriers K. For simplicity, occasional couriers are identified with their destinations. Each arc $(i, j) \in A$ has a distance d_{ij} and an associated cost c_{ij}. Each customer $i \in C$ is characterized by a demand q_i. Professional fleet vehicles have Q capacity, and an unlimited number is assumed to be available. Furthermore, customer's demand and occasional couriers' availability are known in advance.

An occasional courier $k \in K$ is willing to deliver to customer i when the extra distance incurred by the occasional courier to deliver to customer i respects a limit defined by their flexibility. Each occasional courier k can make at most one delivery and is compensated by p_{ik} when delivering to customer i. Each customer's demand must be satisfied either by the professional fleet, whose route starts and ends at the depot, or by an occasional courier, whose route starts from the depot to the customer's location and continues to the occasional courier's destination. Parameter β_{ik} indicates whether occasional courier k can serve customer i.

The following decision variables are used:

- x_{ij}, a binary variable indicating whether a regular vehicle traverses arc (i, j), or not
- y_{ij}, indicating the load a regular vehicle carries on arc (i, j)
- z_i, a binary variable indicating whether a customer is visited by a regular vehicle, or not
- w_{ik}, a binary variable indicating whether customer i is visited by occasional courier k, or not

The VRPOD can be formulated as follows:

$$\text{minimize} \quad \sum_{(i,j)\in A} c_{ij}x_{ij} + \sum_{i\in C}\sum_{k\in K} p_{ik}w_{ik}$$

$$\text{Subject to:} \quad \sum_{j|(i,j)\in A} x_{ij} = \sum_{j|(j,i)\in A} x_{ji} = z_i \qquad \forall i \in C \tag{1}$$

$$\sum_{j|(0,j)\in A} x_{0j} - \sum_{j|(j,0)\in A} x_{j0} = 0 \tag{2}$$

$$\sum_{j|(j,i)\in A} y_{ji} - \sum_{j|(i,j)\in A} y_{ij} = \begin{cases} q_i z_i & \forall i \in C \\ -\sum_{j\in C} q_j z_j & i = 0 \end{cases} \tag{3}$$

$$y_{ij} \le Q x_{ij} \qquad \forall (i, j) \in A \tag{4}$$

$$y_{i0} = 0 \qquad \forall i \in C \tag{5}$$

$$w_{ik} \leq \beta_{ik} \qquad \forall i \in C \quad \forall k \in K \tag{6}$$

$$\sum_{i \in C} w_{ik} \leq 1 \qquad \forall k \in K \tag{7}$$

$$\sum_{k \in K} w_{ik} + z_i = 1 \qquad \forall i \in C \tag{8}$$

$$x_{ij} \in \{0, 1\} \qquad \forall (i, j) \in A$$

$$y_{ij} \in [0, Q] \cap \mathbb{Z} \qquad \forall (i, j) \in A$$

$$z_i \in \{0, 1\} \qquad \forall i \in C$$

$$w_{ik} \in \{0, 1\} \qquad \forall i \in C \quad \forall k \in K.$$

The objective function aims at minimizing the total cost given by the sum of professional fleet delivery costs and the compensation given to occasional couriers. Constraints (1) and (2) ensure route flow conservation, and Constraint (3) guarantees demand satisfaction and subtours elimination. Constraint (4) guarantees that vehicle capacity is respected, and (5) ensures that they return empty to the depot. Constraint (6) ensures that a customer is only assigned to an occasional courier willing to serve that customer, and Constraint (7) guarantees that each occasional courier serves at most one customer. Constraint (8) ensures that each customer is served exactly once.

Related Problems and Variants

In last-mile delivery with crowdshipping, there are different research lines regarding the planning and operational levels involved in this delivery strategy. Extensive research followed the work of [2], with new avenues being explored. Major attention has been given to: (1) dynamic and stochastic information integration, (2) compensating strategies, and (3) usage of transshipment points.

Dynamic and Stochastic Information Integration

A main concern of this problem is guaranteeing enough delivery capacity, i.e., enough occasional couriers at the right location and time to perform the deliveries on time. The number of occasional couriers and orders to fulfill may vary in time and space and may not coincide. Moreover, companies must account for the uncertainty of occasional couriers that may not appear or decline an offered delivery, and for the uncertain demand for a specific period, since delivery requests can arrive dynamically. Different works address these challenges. Considering a setting where a delivery service platform automatically matches delivery tasks and occasional couriers, the authors in [4] propose a rolling horizon framework and design an exact solution method to solve the matching problem as new information becomes available. In [3], authors also consider a re-optimization solution, where

online orders arrive dynamically without information anticipation. They propose a solution combining insertion and re-optimization algorithms that penalize deliveries made on the same day outside specified time windows, and deliveries postponed to the next day. Insertion evaluates each delivery request individually, while re-optimization periodically passes a solution generated by insertion to a variable neighborhood search (VNS) algorithm. Computational results for different ratios between static (known in advance) and dynamic customers that arrive online during the distribution process show that dynamism highly impacts performance and that the re-optimization algorithm provides better solutions than the simple insertion rule. The authors of [8] study the impact of incorporating probabilistic information about future online orders and in-store customer arrivals. For that effect, and considering a same-day delivery setting where online orders and OCs may arrive throughout the day, they propose a sample-scenario planning solution and show that incorporating and exploiting probabilistic information about the future result in more cost-effective solutions compared to a full myopic strategy, in which only the systems' current state is considered when making decisions.

In [16], the authors integrate the OCs' willingness to accept or reject deliveries assigned to them and study the influence of their decision on the expected delivery cost. They determine the subset of customers proposed as occasional couriers by using a local search heuristic and calculating the solution's expected value, assuming that, in the second stage, a professional fleet carries out the remaining deliveries. Another two-stage stochastic model is proposed in [7]. In the first stage, partial routes are defined without knowledge of OCs that can arise later in the day. Those decisions can be revised in the second stage when OCs become available. The work is extended in [40], which presents a multistage formulation of the problem. The authors of [41] present a two-stage stochastic model that incorporates the uncertainty of the arrival of orders and the presence of customers to receive deliveries. In cases where customers are not present, the deliveries have to be returned to the depot. To solve the problem exactly, the authors propose a branch-and-price algorithm and, for larger problems, a column generation heuristic. Later, in [43], they present an extension of the model where the destinations of the online orders are not known in advance (open vehicle routing problem).

Compensation Strategies

The compensation offered to occasional couriers plays a significant role in the success of crowdsourcing solutions for last-mile delivery [2]. At the same time, companies must balance the amount offered to OCs to provide fair compensation that promotes OC participation and optimizes costs and efficiency. Some studies already focus on compensation schemes. In [42], different compensation schemes for the OCs are explored to determine the compensation that minimizes the total delivery cost. They propose a large neighborhood search (LNS) heuristic, considering the stochastic nature of occasional courier acceptance. A data-driven approach to designing a compensation scheme for crowdsourced deliveries is proposed by [5]. The authors determine an optimal compensation by modeling the OC's willingness to conduct deliveries using logistic regression based on factors

such as delivery distance and parcel weight. In contrast with previous works in which the compensation decision focuses only on the OC side, [22] also integrates the sender's willingness to pay. They advise companies by testing the model under four compensation schemes, five demand and supply scenarios, and three objectives: senders' cost minimization, platform profit maximization, and occasional couriers' compensation maximization.

Transshipment Points Integration

Intermediary facilities such as transshipment points have become increasingly popular as alternatives to traditional last-mile delivery methods. These facilities, e.g., parcel lockers, allow customers to pick up their orders conveniently. Furthermore, retailers that rely on systems that use these structures can propose to customers who pick up their orders at a locker to deliver to other customers who prefer to receive their orders at their homes. The integration of lockers from where OCs pick up parcels was proposed in [46]. The authors propose a min-cost network flow model where OCs can perform multiple deliveries and are rewarded according to the additional distance they travel. The authors in [35] consider automatic lockers for occasional couriers' drop-off, pickup, or use as intermediate transfer points where parcels can be dropped off by an OC and picked up by another, i.e., transferred from lockers. They present a stochastic dynamic algorithm for a simplified version of the problem where capacity constraints of lockers and OCs are not considered. The authors of [28] proposed a mixed-integer program and a variable neighborhood search heuristic to solve the capacitated version of the problem. In [9], the authors analyze the usage of existing urban facilities such as shops or lockers as occasional depots that are activated only when necessary. They present a mixed-integer linear programming model designed to optimize the deployment of these depots with company vehicles and two groups of OCs: classical OCs, who transport goods between the central depot and occasional depots, and neighborhood OCs, who deliver parcels from the occasional or central depot to customers. A two-echelon last-mile delivery framework incorporating parcel lockers and crowdsourcing is presented in [39]. The occasional couriers can perform deliveries by deviating slightly from their initial routes.

Metaheuristics Approaches to Related Problems

This chapter examines research that proposes and applies metaheuristics to address last-mile delivery problems with crowdshipping. While mathematical programming decomposition algorithms have advanced in solving vehicle routing problems, optimal solutions are still limited to relatively small instances. However, solutions for significantly larger instances are needed in real-world scenarios [21]. Several metaheuristics, including variable neighborhood search, genetic algorithms (GA), and tabu search (TS) (see Fig. 2), have been introduced in the literature to address large-scale instances of crowdshipping effectively. Below we review the literature on last-mile delivery with crowdshipping. Differently from previous sections, where

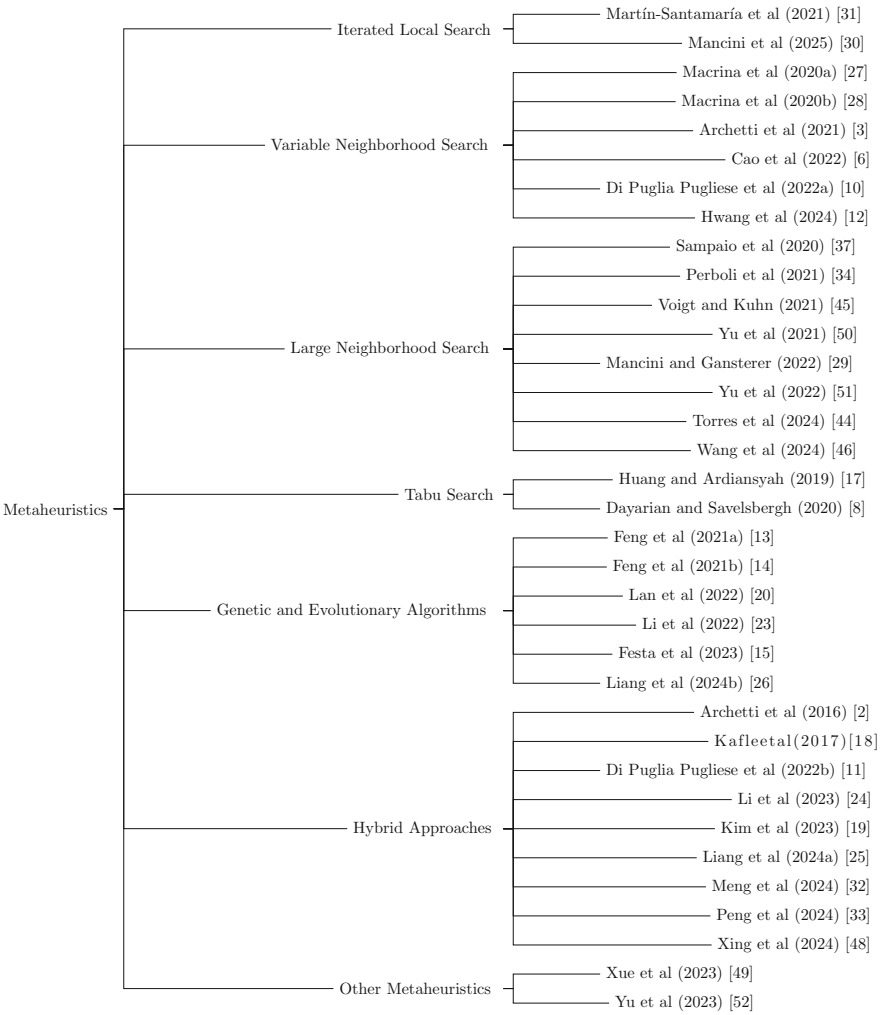


Fig. 2 Classification of metaheuristics considered for last-mile delivery with crowdshipping

contributions were aggregated by problem variant, in this section, we divide them according to the metaheuristic used to solve the problem. Table 1 provides a detailed summary of the constructive, improvement, destruction, and insertion operators employed in the reviewed studies.

Iterated Local Search

An interesting part of literature resorted to iterated local search (ILS) to solve the problem. In [31], the authors employed ILS to address the VRPOD, using greedy

Table 1 Summary of constructive, improvement, destruction, and insertion operators

	Construct				Improve										Destroy			Insert		
					Intra-route					Inter-route										
Paper	Greedy	Nearest neighbor	Exact methods	Other	2-opt	Move	Swap	Relocate	Other	2-opt	Move	Swap	Relocate	Other	Worst	Random	Other	Greedy	Random	Other
Archetti et al. [2]	✓		✓								✓	✓		✓		✓		✓		
Kafle et al. [18]		✓			✓							✓	✓							
Huang and Ardiansyah [17]		✓		✓	✓							✓					✓	✓		✓
Dayarian and Savelsbergh [8]				✓													✓			✓
Macrina et al. [27]	✓				✓	✓	✓		✓	✓	✓	✓		✓			✓			✓
Macrina et al. [28]	✓				✓	✓	✓		✓	✓	✓	✓		✓			✓			✓
Sampaio et al. [37]	✓														✓		✓	✓		✓
Archetti et al. [3]	✓				✓	✓	✓		✓	✓	✓	✓		✓			✓			✓
Feng et al. [13]				✓																
Feng et al. [14]	✓			✓																
Martín-Santamaría et al. [31]				✓	✓						✓	✓		✓		✓	✓	✓		✓

(continued)

Table 1 (continued)

Paper	Construct				Improve										Destroy			Insert		
					Intra-route					Inter-route										
	Greedy	Nearest neighbor	Exact methods	Other	2-opt	Move	Swap	Relocate	Other	2-opt	Move	Swap	Relocate	Other	Worst	Random	Other	Greedy	Random	Other
Perboli et al. [34]	✓														✓	✓	✓	✓		✓
Voigt and Kuhn [45]				✓											✓	✓	✓	✓	✓	✓
Yu et al. [50]	✓				✓		✓		✓	✓		✓		✓	✓	✓	✓	✓		✓
Cao et al. [6]				✓	✓		✓	✓			✓	✓	✓	✓						
Di Puglia Pugliese et al. [10]	✓				✓	✓	✓		✓	✓	✓	✓		✓			✓			✓
Di Puglia Pugliese et al. [11]				✓	✓		✓			✓		✓		✓						
Lan et al. [20]				✓	✓			✓					✓	✓						
Li et al. [23]				✓																
Mancini and Gansterer [29]			✓												✓	✓				✓
Yu et al. [51]				✓			✓				✓	✓			✓	✓		✓		
Festa et al. [15]				✓																

Kim et al. [19]	✓				✓		✓		✓			✓		✓	✓	✓		✓		✓
Li et al. [24]				✓																
Xue et al. [49]				✓																
Yu et al. [52]		✓	✓						✓			✓								
Hwang et al. [12]				✓	✓				✓											
Liang et al. [25]				✓											✓	✓		✓		
Liang et al. [26]	✓			✓						✓										
Meng et al. [32]				✓					✓											
Peng et al. [33]				✓																
Torres et al. [44]	✓															✓	✓	✓		✓
Wang et al. [47]			✓													✓	✓	✓		✓
Xing et al. [48]				✓																
Mancini et al. [30]				✓															✓	✓

randomized adaptive search procedure (GRASP) as the constructive method to generate a set of varied initial solutions. For local search, the authors employed a variety of neighborhood operators (including 2-opt, 1-move, swap-move, and in-move), while perturbation was promoted through random moves, route cost, and reassignment. Similarly, [30] applied ILS to a last-mile delivery problem, focusing on assigning bundles of customer requests to occasional couriers within an auction-based framework that considers absenteeism of OCs, which can cancel with little notice, potentially disrupting service. The authors introduced an auction-based system where couriers can bid on delivery bundles and act as reserved couriers to ensure availability (they receive a fee to ensure their presence but are not guaranteed to be assigned to a specific bundle). The problem was modeled as a two-stage stochastic recourse model, solved using a self-learning matheuristic (SLM) alongside ILS, where SLM served as the local search operator by iteratively refining first-stage decisions to minimize second-stage costs. Perturbations were introduced to diversify solutions, with computational results showing this hybrid method's superiority over deterministic approaches and VNS.

Variable Neighborhood Search

To the extent knowledge of the authors, VNS was used for last-mile delivery problem with crowdsourcing in [27]. The study addresses the vehicle routing problem with occasional drivers and time windows with multiple deliveries (VRPODTWmd), an extension of [2], which allows occasional couriers to serve multiple customers. To create the initial solution, the authors propose an insertion heuristic that prioritizes assigning customers to occasional couriers. The neighborhood search comprises operators such as 2-opt, relocation, and inter- and intra-route swaps. The shaking phase employs a semi-random operator selection strategy guided by adaptive scoring, while the variable neighborhood descent (VND) phase systematically refines the solutions. Computational experiments reveal that the proposed VNS significantly outperforms a commercial solver (CPLEX) on larger problem instances. Later, an enhanced version of this variable neighborhood search is proposed in [10] to solve the same problem variant. This approach integrates a variable neighborhood search metaheuristic with machine learning techniques, including Q-learning, to strategically explore the most promising regions of the solution space. Five variations of VNS are derived from how the learning strategies are applied in the shaking and local search phases. Computational results demonstrate that the proposed methods achieve high performance in both effectiveness and efficiency.

In [28], the authors extended the work in [2] by introducing transshipment points and time windows. They proposed a mixed-integer program and employed a variable neighborhood search to solve instances of bigger size. The proposed procedure starts by generating an initial solution using a matheuristic that initially solves an assignment model, and if the obtained solution has unserved customers, it utilizes an insertion heuristic to place them. They consider 11 local search operators based

on four local search moves (move, swap, swap intra-route, and new route). For the shaking phase, they introduce a semi-random choice based on a score for each local search move (score is incremented if there is an improvement in the solution cost; otherwise, the scores are reduced), in which they randomly select two moves among the five with the highest score. The local search phase uses a VND. The results reveal that compared to CPLEX, the proposed metaheuristic balances solution quality and computational efficiency with a reasonable optimality gap in shorter run times.

The study in [3] addresses the online vehicle routing problem with occasional drivers, a scenario in which future customer requests are unknown, and penalties are imposed for same-day deliveries outside designated time windows and for postponing deliveries to the next day. The authors present a mathematical model and design and implement a solution approach that integrates insertion and re-optimization algorithms. In this approach, a greedy insertion algorithm evaluates each delivery request independently, assigning it to the optimal position within the solution. Periodically, the re-optimization algorithm passes the insertion-generated solution to a variable neighborhood search algorithm, where routes for both PF and OCs are improved using local search procedures, including relocate and swap operations within and between routes. Computational experiments conducted for different ratios between static (pre-known) and dynamic (real-time) customer requests reveal that dynamic arrivals significantly impact performance and demonstrate that the re-optimization algorithm yields superior solutions compared to the insertion-only approach.

In [6], the authors address the green vehicle routing problem with passing-by couriers, which involves optimizing routes for both professional and occasional (passing-by) couriers to reduce costs and environmental impact while fulfilling customer delivery demands. The objective is to balance carbon emissions and operational costs and enhance driver satisfaction. To solve this problem, the authors introduce a memory-based variable neighborhood search (MVNS) metaheuristic. This approach starts by generating an initial solution for professional couriers using the put forward insertion heuristic, which computes the cost of inserting a new customer into the current route, and any remaining unrouted customers are then assigned to feasible routes of passing-by couriers using an insertion algorithm. Local search and a shaking step employ neighborhood structures with operators such as relocate, swap, 2-opt, and cross-exchange, enabling adjustments within individual routes (inner relocate and inner swap) and between routes. Throughout the search, memory-based restart mechanisms utilize historical elite solutions to guide the exploration of promising areas in the solution space. Computational results indicate that MVNS outperforms the standard VNS approach.

In [12], the authors address the supermarket chain grocery delivery crowdshipping problem by proposing a mixed-integer linear programming model and designing an adaptive variable neighborhood search (AVNS) heuristic to solve large-scale practical instances efficiently. The initial solution is constructed using a schedule concatenation method, which iteratively assigns grocery orders to couriers based on the earliest feasible delivery times, followed by a forward time slack adjustment to minimize delivery service times. The neighborhood search incorporates

intra-route operations such as 2-opt and Or-opt. The AVNS extends the traditional variable neighborhood search framework by introducing adaptive mechanisms, hybrid adaptive large neighborhood search (HALNS) elements, guided neighborhood selection, and a worse-solution acceptance strategy to handle complex constraints better and escape local optima. Computational experiments demonstrate that the AVNS achieves superior solution quality and efficiency compared to conventional methods, effectively solving instances with up to 200 orders.

Large Neighborhood Search

So far, large neighborhood search has been the most popular metaheuristic used to solve the problems associated with last-mile delivery with crowdshipping. An adaptive large neighborhood search algorithm is proposed in [37] to solve a multi-depot pickup and delivery problem with time windows and transfers, considering occasional couriers that express their availability to perform delivery tasks for a given period. A greedy insertion algorithm generates the initial solution, and the neighborhood operators focus on removing requests from the current solution (e.g., worst request removal and random request removal) and reinserting them to create a modified solution using three different insertion operators. At each iteration, one of the neighborhoods is selected, prioritizing neighborhoods that have succeeded in earlier iterations. Improved solutions are always accepted, but a diversification mechanism is incorporated, sometimes accepting worse solutions. In [34], the authors propose a large neighborhood search approach to solve the dynamic and stochastic vehicle routing problem by integrating occasional couriers within a simulation-optimization framework. The initial solution is generated using a greedy insertion heuristic, which places each request at the most cost-effective position. Different destroy operators (random, worst, related, and cluster removal) and repair operators (greedy and second cheapest) are applied to improve the solution. A case study conducted in the city of Turin (Italy) demonstrates that combining crowdsourced couriers with green carriers, such as cargo bikes, offers significant economic and environmental cost savings. However, it leads to a decrease in delivery efficiency.

An adaptive large neighborhood search algorithm with a simulated annealing (SA) acceptance criterion to diversify the search is proposed in [45]. The initial solution is obtained by randomly applying repair operators until every request is served. Five destroy operators are considered to remove requests (such as random, worst, shaw, and least efficient removal) from the current solution, and six operators are considered to build a new solution (such as random, best, and regret insertion). Compared with the exact solution methods, the proposed procedure delivers equivalent results for small instances and scales well with the problem size.

The work in [50] addresses a variant of the vehicle routing problem, specifically, the two-echelon vehicle routing problem with time windows, covering options, and occasional couriers. This problem involves two fleets, city freighters and occasional couriers, which serve customers with two delivery options: home delivery and

alternative delivery locations (covering options) for customer pickups. The objective is to minimize total costs, including routing, connection, and compensation costs for OCs. The authors propose an adaptive large neighborhood search metaheuristic to tackle this problem. The initial solution is constructed in two phases based on the cheapest insertion, first addressing second-echelon routes for city freighters and OCs and then constructing first-echelon truck routes. During ALNS iterations, destroy and repair operators are applied to iteratively modify the solution by removing and reinserting customers and reconfiguring routes. Thirteen destroy operators, targeting customer nodes or entire routes (e.g., random and worst), and seven repair operators, which reinsert customers into feasible routes while considering time windows and synchronization constraints (e.g., greedy heuristic), are employed. Additionally, the authors explore local search integration within ALNS, utilizing swap, reinsertion, and 2-opt neighborhood operators, and demonstrate that this integration leads to improved results.

The problem considered in [29] is the VRPOD and bundles of orders that optimize the assignment of customer bundles to the OCs and PF vehicles while minimizing total delivery costs. The proposed solution utilizes large neighborhood search metaheuristic, starting with an initial feasible solution obtained from a relaxed problem formulation. The LNS employs destroy operators that randomly remove customers or arcs from existing routes (e.g., random and worst removal), rebuilding it based on rules. The LNS obtained near-optimal solutions in a short computational time for the larger instances.

In [51], authors introduce a new variant of the two-echelon vehicle routing problem called the two-echelon vehicle routing problem with time windows, intermediate facilities, and occasional drivers. The problem integrates parcel lockers and crowdshipping into a two-echelon distribution network to minimize total operational costs. The authors formulate the problem as a mixed-integer linear programming model and develop a hybrid adaptive large neighborhood search algorithm to solve it. The HALNS incorporates set partitioning to collect routes generated during the destroy, repair, and VND (local search) phases. Based on extensive computational experiments, it was concluded that HALNS demonstrates strong performance. Furthermore, it was highlighted that the key components of HALNS, namely VND and the set partitioning problem, are important for ensuring its robust and reliable high-quality performance.

The study in [44] addresses optimization of route pricing to enhance the participation of occasional couriers in delivery tasks. It formulates a multi-objective problem to minimize routing costs and maximize OCs participation. The authors propose a comprehensive framework that integrates pricing, route optimization, and the uncertainty of occasional courier route acceptance, employing an adaptive large neighborhood search metaheuristic. The methodology incorporates a variety of construction algorithms and neighborhood operations, including greedy insertion, near removal, random removal, random route removal, first customer removal, and last customer removal. Additionally, the ALNS is supplied with a route assignment and pricing procedure, which determines the optimal price and acceptance probability for a subset of routes. The study also includes an econometric model to analyze the

behavior of OCs in response to route attributes. Computational experiments reveal that the proposed ALNS outperforms a column generation heuristic.

The integration of crowdshipping with high-speed railways for express delivery services is studied in [47]. A mathematical model is presented to tackle the freight allocation problem in the rail segment and the OC routing problem in the road segment. The authors employ a depth-first search-based algorithm for small-scale instances and a customized adaptive large neighborhood search algorithm for larger instances, incorporating different neighborhood operations to improve solution quality. The initial solution is generated by solving a freight allocation plan using the CPLEX solver based on the transportation plans for origin-destination pairs. At the same time, the customized ALNS algorithm employs neighborhood operators such as insertion, swap, and removal to improve the solution iteratively.

Tabu Search

Tabu search is used in [17] to study a two-echelon problem where deliveries are handled by either PF vehicles or OCs through transfer points, highlighting that not all deliveries can be outsourced due to the lack of transfer points in certain neighborhoods. The construction of initial solutions balances PF vehicles and OCs, employing a sweep algorithm combined with a nearest-neighbor heuristic for PF routes and a savings algorithm for OC routes. The improvement phase is based on tabu search, incorporating tailored intra- and inter-route neighborhood operators, such as insertion, swaps, and destroy-repair moves. Computational results indicate that the proposed solution is effective, delivering near-optimal solutions for small-scale instances and high-quality solutions for large-scale scenarios.

The authors of [8] utilize tabu search within their sample-scenario planning framework to address the dynamic and stochastic same-day delivery environment, where online orders and OCs willing to make deliveries arrive throughout the day. Their approach incorporates removal and insertion moves during TS neighborhood exploration.

Genetic and Evolutionary Algorithms

Genetic algorithm is also used to solve last-mile delivery problems with crowdshipping. The study in [14] explores the integrated production and transportation scheduling problem with crowdsourced delivery (IPTS-CD). This problem can be decomposed into two subproblems: first, solving an IPTS to obtain an integrated schedule that includes both the production schedule and truck delivery routes, and after a crowdsourced bid selection problem, in which truck delivery routes are adjusted by adding some valuable crowdsourced deliveries. The authors formulate it as a mixed-integer linear program and propose a GA to solve it. The initial population routes are obtained randomly and improved with a greedy algorithm defining the production scheduling based on when truck is dispatched. Comparative

evaluations against CPLEX demonstrated the GA's superior solution quality and computational efficiency performance. In [23], the authors propose a mathematical model and GA to solve the meal crowdsourcing delivery problem. The initial population uses an order-pair insertion method that ignores OCs locations during allocation.

The work in [26] addresses the urban parcel crowdsourced delivery problem that optimizes scheduling OCs who pass through delivery points, aiming to maximize delivery efficiency and reduce traffic and environmental costs. The authors use a genetic algorithm and an insertion heuristic, creating an initial solution by matching parcels with couriers based on proximity and capacity constraints. They apply a rolling optimization model for dynamic order insertion that reassigns new orders with minimal disruption to predefined routes based on disturbance management. Destroy and repair operators adjust parcel assignments to available couriers as routes evolve. Results show that the model increases the matching ratio between parcels and passing couriers and improves profit compared to static models.

A biased random-key genetic algorithm (BRKGA) with implicit path relinking is proposed in [15]. The authors address the variant of VRPOD with time windows. The objective is to minimize total delivery costs while satisfying time windows and capacity constraints. The BRKGA encodes solutions as random-key vectors and decodes them into the delivery router using a custom decoder. The initial population is generated randomly, and the algorithm evolves through elite preservation, mutation, and crossover operators. An implicit path-relinking mechanism (intensification strategy that exploits the neighborhood formed in the path between two feasible solutions) further refines solutions, and a restart strategy (a new seed for the random-key generator) is employed to escape local optima. The results showed that the proposed BRKGA is more effective and efficient than the VNS proposed in [27] in the instances considered.

The following studies apply evolutionary algorithms to tackle the last-mile delivery problems with crowdshipping. An evolutionary multitask optimization is employed in [13] to address the vehicle routing problem with heterogeneous capacity, time windows, and occasional drivers (VRPHTO). The authors propose an evolutionary multitasking algorithm (EMA) to solve multiple VRPHTO instances concurrently. The EMA employs a permutation-based unified representation to encode multiple VRPHTO problems within a single population. Neighbor solutions are generated via order crossover and mutation operators. Experimental results reveal that the EMA surpasses traditional single-task solvers by reducing costs and enhancing efficiency, particularly when tackling related problem instances simultaneously.

Building on evolutionary optimization techniques, [20] investigates the multi-objective two-echelon city dispatching problem with multi-sourcing and crowd-shipping. The study introduces a multi-objective mathematical model that balances company costs, customer satisfaction (measured as the deviation between actual and ideal delivery times), and crowd-shipper income satisfaction. To solve this multi-objective problem, the authors develop the multidirectional evolutionary algorithm (MDEA) that guides multiple neighborhood operators to explore the

Pareto front. The algorithm framework includes initialization, multidirectional evolution, and environment selection. In the initialization step, the population is initialized following a random strategy. The authors propose three neighborhood operators for the multidirectional evolution: basic, objective wise, and VNS. Basic neighborhood operators include commonly used operators such as relocate and 2-opt. The objective-wise operators are designed to improve each of the three objective functions considered. Finally, the environmental selection adopts a reference vector-based selection strategy to select the next generation of individuals. Results demonstrate that the MDEA outperforms several popular multi-objective approaches.

Hybrid Approaches

Hybrid approaches, combining multiple metaheuristics to leverage their complementary strengths, have also been proposed.

In the seminal work on VRPOD [2], the authors proposed a multi-start heuristic and compared it to the solutions obtained using CPLEX. Their heuristic combines variable neighborhood search, tabu search, and integer programming. The heuristic begins with five initial solutions, each differing in the method used for assigning customers to OCs. Each solution is obtained as follows: (1) all customers are served by a professional fleet (no occasional couriers are employed), (2) solve an integer program model to determine the customers served by occasional couriers seeking to serve the far-away customers first, (3) solve the same integer program model, but now maximize the demand delivered by the occasional couriers, (4) solve an integer program model that maximizes the number of used occasional couriers, followed by the minimization of the cost (hierarchically), (5) solve a relaxation of the proposed model, disregarding the integrality constraint on the variables x. Subsequently, the PF routes are constructed using a sequential greedy insertion algorithm, further refined through a 2-exchange improvement. After, a VNS-based approach is applied to each solution, with tabu search being employed in the shaking phase where all non-tabu moves, from 1-move, swap-move, in-move, and out-move, are evaluated and the best one is chosen. Tabu search is followed by a random operator (denoted as *Jump*) that randomly selects a customer and inserts it in the best route using the cheapest insertion criterion. Finally, the professional fleet routes are subject to a 2-exchange neighborhood (local search phase).

In [18], the authors consider a model where pedestrians and/or cyclists act as OCs, and a truck performs the remaining deliveries. To solve the problem, the authors decompose it into two subproblems: first, the selection of the bids submitted by OCs for undertaking a subset of jobs and, after, the routing and scheduling of trucks (modeled as a simultaneous pickup and delivery problem). The first problem is solved using a branch-and-bound algorithm, and the second is solved using simulated annealing. A tabu search iteratively solves both subproblems to improve the overall solution by exploring the selection of relay points.

A hybrid heuristic algorithm that combines genetic algorithm and tabu search is proposed in [24]. The work addresses a multistage two-echelon crowdsourcing logistics assignment model, in which parcels to be delivered in each stage are known a priori, but the information on OCs in future stages is not completely known. The authors formulated a mixed-integer linear program and proposed a genetic algorithm that integrates a tabu search to optimize the sequence of deliveries for each courier. The initial population is built using four methods: one based on pure randomization and three based on roulette wheel selection, which assign weights to couriers based on distance to subdepots, distance to destination, and length of the original route. The GA-TS algorithm outperforms CPLEX regarding computational efficiency and cost effectiveness.

The study presented in [33] addresses the outsourcing service price problem in collaborative crowdshipping, focusing on the interaction between parcel delivery service providers and on-demand mobility service providers, such as taxis and ride-hailing vehicles (acting as OCs). The problem is modeled as a bi-level optimization problem with the upper level addressing the parcel delivery service providers' objective of minimizing total costs, formulated as a multi-depot pickup and delivery problem with pricing. The lower level focuses on maximizing OCs' profit by optimizing the allocation between parcel deliveries and passenger rides, which is formulated as a crowdshipping problem with ridesharing. To solve this problem, the authors propose a customized iterative hybrid algorithm that combines two granular tabu search (GTS) algorithms within a genetic algorithm. The process begins with a randomly generated initial population of potential outsourcing service prices, represented in binary form. Afterward, the GTSs solve the lower-and upper-level problems sequentially by applying profit-oriented and cost-oriented granularity strategies. Numerical experiments are conducted on randomly generated instances and a real-life case study, demonstrating that the proposed framework effectively balances the interests of both participants.

Another work that uses genetic algorithms is [25]. The authors present a crowdsourcing framework for green urban parcel delivery, leveraging private cars (OCs) already passing delivery points. The problem focuses on optimizing urban delivery operations to minimize both environmental costs from carbon emissions and enterprise costs by utilizing existing resources and incentivizing private couriers with bonuses. A mixed-integer programming model is formulated to define the problem, and a hybrid metaheuristic algorithm, integrating a GA with ALNS, is proposed for solution optimization. The initial solution is constructed by matching parcels to OCs based on spatial and temporal constraints. Destroy operators (random and worst removal) remove parcels from delivery routes, while repair operators reassign parcels to feasible routes using greedy heuristics (minimizing distance) or random allocation, ensuring compliance with capacity and time window constraints.

Express crowdsourcing task allocation considering distribution mode under customer classification is studied in [48]. Customers can have their parcels delivered in two modes: home delivery carried by OCs and self-pickup. Customers are also

classified into three segments—efficient, potential, and marginal—based on their historical consumption pattern, analyzed using an entropy-weighted RFM (Recency, Frequency, Monetary) model so that priority is given to the high-quality customers (based on penalty cost function). To address the problem, the authors present a customer classification genetic algorithm (CCGA). The algorithm randomly generates the initial population and integrates simulated annealing to accept or reject the new individuals. Experimental results highlight the superiority of the CCGA, which combines GA and SA, over traditional GA, demonstrating its enhanced effectiveness in addressing the problem.

The work in [19] uses a hybridization of ALNS and SA to solve a two-echelon vehicle routing problem with heterogeneous OCs: the dedicated OCs, and in-store customers who deliver orders on their way home from a store (en route OCs). A mixed-integer linear programming model and an ALNS that integrates SA are proposed. The ALNS algorithm uses a greedy insertion heuristic to generate initial solutions, followed by neighborhood search heuristics such as intra-swap, inter-swap, intra-shift, inter-shift, and 2-Opt. Additionally, new local search algorithms—transfer node and candidate swap, transfer node replacement, and transfer node replacement with swap—are introduced to enhance the search space by adjusting transfer nodes for couriers and OCs. The performance of the proposed algorithm is compared with an MILP solver, GA, neighborhood search, and Monte Carlo simulation.

The work in [32] addresses the crowdsourced delivery scheduling problem in dispatch mode, combining order allocation and travel route generation. To solve this problem, the authors propose a variable neighborhood search based on the Metropolis criterion (VNSMC). This metaheuristic incorporates the concept of SA and introduces a new solution acceptance strategy based on the Metropolis criterion to enhance the exploration of the solution space. The VNSMC starts by randomly generating an initial solution and applying a shaking procedure based on order removal and reassignment. Afterward, a local search based on a combination of two local search operators (migration and exchange) is proposed. Routing decisions are carried out by a branch-and-bound method. To evaluate the performance of the proposed VNSMC, the authors also developed an SA, VNS without Metropolis criterion, and GA to solve the problem and concluded that the proposed VNSMC obtains better solutions.

Finally, in [11], the VRPODTWmd is addressed using several variants of the GRASP as the primary metaheuristic. The initial solution is constructed in a greedy randomized phase, employing two strategies: the classical restricted candidate list and a biased randomized version. Local search is performed using neighborhood operators such as 2-opt and swap. Furthermore, two hybrid approaches are proposed, integrating VNS and VND as local search strategies within GRASP. Comparative results demonstrate that the hybrid GRASP with VND and VNS achieves the best balance between computational efficiency and solution quality, outperforming the methods proposed in [2] and the VNS of [28].

Other Metaheuristics

Other metaheuristics have been proposed to address last-mile delivery with crowdsourcing problems.

In [52], the authors solve the vehicle routing problem with simultaneous pickup and delivery and occasional drivers using simulated annealing. Their approach begins with a two-phase initial solution construction: First, a relaxed mixed-integer linear programming model assigns customers to occasional couriers, and second, a nearest-neighbor insertion heuristic generates feasible routes for permanent fleet vehicles. The neighborhood search phase incorporates the operator's swap, insertion, and reverse, specifically designed to manage interactions between PF vehicles and OCs. Computational results demonstrate that the proposed SA method outperforms CPLEX in terms of solution quality and computational efficiency.

Adaptive ant colony optimization (ACO) is utilized in [49] to solve the crowdsourcing multi-depot vehicle routing problem with time windows. The problem, which only considers either traditional or crowdsourcing (separated), is divided into two stages: first, a division stage, where it employs a k-means clustering algorithm based on the elbow method to transform the multi-depot problem into several single-depot problems, and after, in the second stage, the optimization stage, where an adaptive ant colony (ADACO) algorithm is proposed to determine the distribution routes for each distribution center. The basic version of ACO is improved with state transfer and pheromone updates to avoid being enclosed in local optima. Comparative analyses show the higher capacity of ADACO to handle larger instances than a commercial solver (Gurobi).

Conclusions

This chapter presents an analytical review of the papers proposing and applying metaheuristics to solve last-mile delivery with crowdshipping problems. The integration of occasional couriers into delivery operations has introduced unique challenges, including the dynamic nature of customer requests and the need to balance cost efficiency with environmental objectives. Metaheuristic methods have proven highly effective in addressing several problem variants since they can handle large, complex search spaces and provide high-quality solutions within reasonable computational times. Methods such as genetic algorithms, tabu search, variable neighborhood search, and large neighborhood search were tailored to exploit specific problem structures, often by designing custom operators or hybrid frameworks that combine multiple metaheuristics. Furthermore, computational experiments demonstrate that metaheuristics can effectively balance trade-offs between cost, efficiency, and sustainability, making them well-suited for crowdshipping systems. As the field progresses, advancements in data-driven optimization and hybrid

computational frameworks are expected to enhance the efficiency of last-mile delivery with crowdshipping.

Cross-References

- A History of Metaheuristics
- Ant Colony Optimization: A Component-Wise Overview
- Evolutionary Algorithms
- Genetic Algorithms
- GRASP
- Heuristics in Vehicle Routing
- Iterated Local Search
- Tabu Search
- Theory of Local Search
- Variable Neighborhood Descent
- Variable Neighborhood Search

Acknowledgments Tiago Monteiro acknowledges the Portuguese Science and Technology Foundation (FCT) for the PhD grant (2020.06686.BD).

Competing Interest Declaration The authors has no competing interests to declare that are relevant to the content of this manuscript.

References

1. Amazon (2023) Amazon flex. https://flex.amazon.com. Accessed 10 Jul 2023
2. Archetti C, Savelsbergh M, Speranza MG (2016) The vehicle routing problem with occasional drivers. Eur J Oper Res 254(2):472–480. https://doi.org/10.1016/j.ejor.2016.03.049
3. Archetti C, Guerriero F, Macrina G (2021) The online vehicle routing problem with occasional drivers. Comput Oper Res 127:105–144. https://doi.org/10.1016/j.cor.2020.105144
4. Arslan AM, Agatz N, Kroon L, Zuidwijk R (2019) Crowdsourced delivery - a dynamic pickup and delivery problem with ad hoc drivers. Transport Sci 53(1):222–235. https://doi.org/10.1287/trsc.2017.0803
5. Barbosa M, Pedroso JP, Viana A (2023) A data-driven compensation scheme for last-mile delivery with crowdsourcing. Comput Oper Res 150:106059. https://doi.org/10.1016/j.cor.2022.106059
6. Cao L, Ye Cm, Cheng R, Wang Zk (2022) Memory-based variable neighborhood search for green vehicle routing problem with passing-by drivers: a comprehensive perspective. Compl Intell Syst 8:2507–2525. https://doi.org/10.1007/s40747-022-00661-5
7. Dahle L, Andersson H, Christiansen M (2017) The vehicle routing problem with dynamic occasional drivers. In: Bektaş T, Coniglio S, Martinez-Sykora A, Voß S (eds) Computational logistics. Springer International Publishing, Cham, pp 49–63. https://doi.org/10.1007/978-3-319-68496-3_4
8. Dayarian I, Savelsbergh M (2020) Crowdshipping and same-day delivery: employing in-store customers to deliver online orders. Prod Oper Manag 29(9):2153–2174. https://doi.org/10.1111/poms.13219

9. Di Puglia Pugliese L, Guerriero F, Macrina G, Scalzo E (2021) Crowd-shipping and occasional depots in the last mile delivery. Springer International Publishing, Cham, pp 213–225. https://doi.org/10.1007/978-3-030-86841-3_18
10. Di Puglia Pugliese L, Ferone D, Festa P, Guerriero F, Macrina G (2022) Combining variable neighborhood search and machine learning to solve the vehicle routing problem with crowd-shipping. Optim Lett 17:1–23. https://doi.org/10.1007/s11590-021-01833-x
11. Di Puglia Pugliese L, Ferone D, Festa P, Guerriero F, Macrina G (2022) Solution approaches for the vehicle routing problem with occasional drivers and time windows. Optim Methods Softw 37:1–31. https://doi.org/10.1080/10556788.2021.2022142
12. Hwang FJ, Bohan H, Kovalyov MY (2024) Supermarket-chain grocery delivery optimization through crowdshipping. Int J Prod Res 63(5):1–28. https://doi.org/10.1080/00207543.2024.2389550
13. Feng L, Zhou L, Gupta A, Zhong J, Zhu Z, Tan KC, Qin K (2021) Solving generalized vehicle routing problem with occasional drivers via evolutionary multitasking. IEEE Trans Cyber 51(6):3171–3184. https://doi.org/10.1109/TCYB.2019.2955599
14. Feng X, Chu F, Chu C, Huang Y (2021) Crowdsource-enabled integrated production and transportation scheduling for smart city logistics. Int J Prod Res 59(7):2157–2176. https://doi.org/10.1080/00207543.2020.1808258
15. Festa P, Guerriero F, Resende MGC, Scalzo E (2023) A BRKGA with implicit path-relinking for the vehicle routing problem with occasional drivers and time windows. In: Di Gaspero L, Festa P, Nakib A, Pavone M (eds) Metaheuristics. Springer International Publishing, Cham, pp 17–29
16. Gdowska K, Viana A, Pedroso JP (2018) Stochastic last-mile delivery with crowdshipping. Transport Res Procedia 30:90–100. https://doi.org/10.1016/j.trpro.2018.09.011
17. Huang K, Ardiansyah MN (2019) A decision model for last-mile delivery planning with crowdsourcing integration. Comput Ind Eng 135:898–912. https://doi.org/10.1016/j.cie.2019.06.059
18. Kafle N, Zou B, Lin J (2017) Design and modeling of a crowdsource-enabled system for urban parcel relay and delivery. Transport Res Part B Methodol 99:62–82. https://doi.org/10.1016/j.trb.2016.12.022
19. Kim B, Jeong HY, Lee S (2023) Two-echelon collaborative routing problem with heterogeneous crowd-shippers. Comput Oper Res 160:106389. https://doi.org/10.1016/j.cor.2023.106389
20. Lan YL, Liu F, Ng WWY, Gui M, Lai C (2022) Multi-objective two-echelon city dispatching problem with mobile satellites and crowd-shipping. IEEE Trans Intell Transport Syst PP:1–14. https://doi.org/10.1109/TITS.2022.3140351
21. Laporte G, Ropke S, Vidal T (2014) Chapter 4: Heuristics for the vehicle routing problem. In: Vehicle routing: problems, methods, and applications, 2nd edn. SIAM, Philadelphia, pp 87–116. https://doi.org/10.1137/1.9781611973594.ch4
22. Le TV, Ukkusuri SV, Xue J, Van Woensel T (2021) Designing pricing and compensation schemes by integrating matching and routing models for crowd-shipping systems. Transport Res Part E Logist Transport Rev 149:102209. https://doi.org/10.1016/j.tre.2020.102209
23. Li J, Noto M, Yang Z (2022) Research on delivery network optimization based on crowdsourcing theory. J Adv Comput Intell Intell Inf 26:147–159. https://doi.org/10.20965/jaciii.2022.p0147
24. Li MY, Zhang WS, Wu Y, Wang L (2023) Multi-stage two-echelon crowdsourcing logistics assignment model with future committing drivers. J Cleaner Prod 428:139397. https://doi.org/10.1016/j.jclepro.2023.139397
25. Liang X, Yang H, Wang Z (2024) A crowdsourcing framework for green urban parcel delivery: utilizing private cars that will be in passing. J Cleaner Prod 466:142762. https://doi.org/10.1016/j.jclepro.2024.142762
26. Liang X, Yang H, Wang Z (2024) Rolling optimal scheduling for urban parcel crowdsourced delivery with new order insertion. Comput Oper Res 171:106779. https://doi.org/10.1016/j.cor.2024.106779

27. Macrina G, Di Puglia Pugliese L, Guerriero F (2020) A variable neighborhood search for the vehicle routing problem with occasional drivers and time windows. In: Proceedings of the 9th international conference on operations research and enterprise systems - ICORES, INSTICC, SciTePress, pp 270–277. https://doi.org/10.5220/0009193302700277
28. Macrina G, Pugliese LDP, Guerriero F, Laporte G (2020) Crowd-shipping with time windows and transshipment nodes. Comput Oper Res 113:104806. https://doi.org/10.1016/j.cor.2019.104806
29. Mancini S, Gansterer M (2022) Bundle generation for last-mile delivery with occasional drivers. Omega 108:102582. https://doi.org/10.1016/j.omega.2021.102582
30. Mancini S, Gansterer M, Triki C (2025) An effective mitigation strategy to hedge against absenteeism of occasional drivers. Comput Oper Res 173:106858. https://doi.org/10.1016/j.cor.2024.106858
31. Martín-Santamaría R, López-Sánchez AD, Delgado-Jalón ML, Colmenar JM (2021) An efficient algorithm for crowd logistics optimization. Mathematics 9(5):509. https://doi.org/10.3390/math9050509
32. Meng F, Zhou X, Piao X, Chu D (2024) Variable neighbourhood search based on metropolis criterion for crowdsourced delivery scheduling problem in dispatch model. Comput Oper Res 167:106662. https://doi.org/10.1016/j.cor.2024.106662
33. Peng S, Park WY, Eltoukhy A, Xu M (2024) Outsourcing service price for crowd-shipping based on on-demand mobility services. Transport Res Part E Logist Transport Rev 183:103451. https://doi.org/10.1016/j.tre.2024.103451
34. Perboli G, Rosano M, Wei Q (2021) A simulation-optimization approach for the management of the on-demand parcel delivery in sharing economy. IEEE Trans Intell Transport Syst 23:1–13. https://doi.org/10.1109/TITS.2021.3094851
35. Raviv T, Tenzer E (2018) Crowd-shipping of small parcels in a physical internet. Technical Report, Tel Aviv University. https://doi.org/10.13140/RG.2.2.32169.08809
36. Roadie (2023) The on-the-way delivery service. https://www.roadie.com. Accessed 10 Jul 2023
37. Sampaio A, Savelsbergh M, Veelenturf LP, Van Woensel T (2020) Delivery systems with crowd-sourced drivers: a pickup and delivery problem with transfers. Networks 76(2):232–255. https://doi.org/10.1002/net.21963
38. Sampaio Oliveira A, Savelsbergh MW, Veelenturf L, van Woensel T (2019) Crowd-based city logistics. In: Sustainable transportation and smart logistics. Elsevier, Amsterdam. https://doi.org/10.1016/B978-0-12-814242-4.00015-6
39. Santos A, Viana A, Pedroso JP (2022) 2-echelon lastmile delivery with lockers and occasional couriers. Transport Res Part E Logist Transport Rev 162:102714. https://doi.org/10.1016/j.tre.2022.102714
40. Skålnes J, Dahle L, Andersson H, Christiansen M, Hvattum LM (2020) The multistage stochastic vehicle routing problem with dynamic occasional drivers. In: Lalla-Ruiz E, Mes M, Voß S (eds) Computational logistics. Springer International Publishing, Cham, pp 261–276. https://doi.org/10.1007/978-3-030-59747-4_17
41. Torres F, Gendreau M, Rei W (2021) Vehicle routing with stochastic supply of crowd vehicles and time windows. Transport Sci. https://doi.org/10.1287/trsc.2021.1101
42. Torres F, Gendreau M, Rei W (2022) Crowd-Shipping: determining the compensation of crowd-drivers with stochastic route acceptance. Technical report, CIRRELT. https://www.cirrelt.ca/documentstravail/cirrelt-2022-03.pdf
43. Torres F, Gendreau M, Rei W (2022) Crowdshipping: an open VRP variant with stochastic destinations. Transport Res Part C Emerging Technol 140:103677. https://doi.org/10.1016/j.trc.2022.103677
44. Torres F, Gendreau M, Rei W (2024) Pricing routes: Compensation of crowd-drivers. https://transp-or.epfl.ch/documents/technicalReports/Torres2024_0.pdf. Online; Accessed 20 Aug 2024
45. Voigt S, Kuhn H (2021) Crowdsourced logistics: The pickup and delivery problem with transshipments and occasional drivers. Networks 79:1–24. https://doi.org/10.1002/net.22045

46. Wang Y, Zhang D, Liu Q, Shen F, Lee LH (2016) Towards enhancing the last-mile delivery: An effective crowd-tasking model with scalable solutions. Transport Res Part E Logist Transport Rev 93:279–293. https://doi.org/10.1016/j.tre.2016.06.002
47. Wang X, Zhen L, Wang S (2024) Optimizing an express delivery mode based on high-speed railway and crowd-couriers. Transp Pol 159:157–177. https://doi.org/10.1016/j.tranpol.2024.10.009
48. Xing X, Sun C, Chen X (2024) Research on express crowdsourcing task allocation considering distribution mode under customer classification. Sustainability 16(18):7936. https://doi.org/10.3390/su16187936
49. Xue S (2023) An adaptive ant colony algorithm for crowdsourcing multi-depot vehicle routing problem with time windows. Sustain Oper Comput 4:62–75. https://doi.org/10.1016/j.susoc.2023.02.002
50. Yu VF, Jodiawan P, Hou ML, Gunawan A (2021) Design of a two-echelon freight distribution system in last-mile logistics considering covering locations and occasional drivers. Transport Res Part E Logist Transport Rev 154:102461. https://doi.org/10.1016/j.tre.2021.102461
51. Yu VF, Nguyen PKM, Putra K, Gunawan A, Budi Dharma IGB (2022) The two-echelon vehicle routing problem with transshipment nodes and occasional drivers: Formulation and adaptive large neighborhood search heuristic. J Adv Transport 2022:1–23. https://doi.org/10.1155/2022/5603956
52. Yu VF, Aloina G, Jodiawan P, Gunawan A, Huang TC (2023) The vehicle routing problem with simultaneous pickup and delivery and occasional drivers. Expert Syst Appl 214:119118. https://doi.org/10.1016/j.eswa.2022.119118

Maritime Container Terminal Problems

45

Christopher Expósito-Izquierdo, Eduardo Lalla-Ruiz,
Jesica de Armas, Belén Melián-Batista, and J. Marcos Moreno-Vega

Contents

Introduction 1412
Optimization Problems 1414
Berth Allocation 1415
Quay Crane Scheduling Problem 1422
Internal Vehicle Scheduling 1426
Container Storage 1433
Conclusion 1440
References 1441

Abstract

Maritime container terminals are essential infrastructures in global supply chains. Their high management complexity and heterogeneous processes make them an interesting field to apply heuristics. This chapter provides a comprehensive overview of the main optimization problems found at maritime container terminals, including berth allocation, quay crane scheduling, yard planning, and

C. Expósito-Izquierdo (✉) · B. Melián-Batista · J. Marcos Moreno-Vega
Department of Computer Engineering and Systems, Universidad de La Laguna,
Santa Cruz de Tenerife, Spain
e-mail: cexposit@ull.edu.es; mbmelian@ull.edu.es; mbmelian@ull.es; jmmoreno@ull.edu.es; jmmoreno@ull.es

E. Lalla-Ruiz
Department of High-Tech Business and Entrepreneurship, University of Twente, Enschede,
The Netherlands
e-mail: e.a.lalla@utwente.nl

J. de Armas
Department of Economics and Business, University Pompeu Fabra, Barcelona, Spain
e-mail: jesica.dearmas@upf.edu; jdearmas@ull.es

© Springer Nature Switzerland AG 2025

R. Martí et al. (eds.), *Handbook of Heuristics*,
https://doi.org/10.1007/978-3-032-00385-0_57

internal vehicle scheduling. To solve these problems, a range of heuristic methods are presented and analyzed. The computational results show the effectiveness of these heuristics in practical scenarios, as they deliver high-quality solutions in short computational times.

Keywords

Container · Heuristics · Logistics · Maritime container terminal

Introduction

Containerization is, without a doubt, the engine of economic globalization, which enables the move of goods worldwide within global intermodal supply chains using standardized containers. The containers are usually built of weathering steel and have locking systems and well-known international dimensions. The capacity unit of measure in transport is the 20-foot equivalent unit, which refers to a container with a length of 20 feet. Due to their physical standardized structure, containers are easily transported by different means and can be stacked on top of each other, reducing the surface dedicated to their storage.

The United Nations Conference on Trade And Development (UNCTAD) examines trends in seaborne trade and analyzes the comparative performance of different geographic regions and countries worldwide. In this regard, the Review of Maritime Transport is an annual publication edited by the UNCTAD secretariat since 1968 to present a global maritime market analysis. One of the main assertions included in the last edition of the review in 2023 [64] remarks that the merchandise trade and seaborne shipments have increased in tandem over the last years. Particularly, the world merchandise trade has grown about twice as fast as the world's gross domestic product. Simultaneously, the seaborne trade increased by green2.4% in 2023, allowing the move of about green12.4 billion tons of goods in ports worldwide. However, containerized trade is the fastest-growing market segment, giving rise to that green1.8 billion tons, which are being transported nowadays by containers worldwide.

To cope with the huge aforementioned seaborne trade, over the last decades, public institutions and freight shipping companies have funded the building of maritime container terminals in strategic regions as engines of economic development and investment elements. A maritime container terminal is an extensive infrastructure located in a seaport to join maritime and land transport means. The main transport means usually found at a maritime container terminal are vessels, trucks, inland waterway systems, and trains. The main goal of a maritime container terminal is, hence, to perform the transshipment of freights between different means of transport efficiently. With this goal in mind, a maritime container terminal is responsible for unloading and loading the incoming vessels. The freights are temporarily stored by the terminal until their final means of transport pick them up and retrieve them from the terminal.

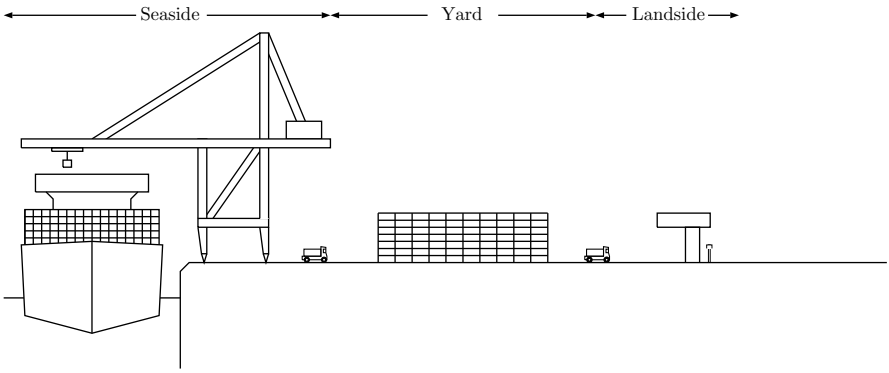


Fig. 1 Overview of a maritime container terminal

As depicted from left to right in Fig. 1, a maritime container terminal is typically split into the following main areas:

- *Seaside*. It is the part of the maritime container terminal that serves the incoming vessels. It contains the most expensive machinery found at the maritime container terminal, the quay cranes, which perform the transshipment operations of containers between the terminal and the vessels. Roughly speaking, a quay crane grabs the containers from the vessels when they are berthed through its spreader during unloading operations. Then, the containers are lifted and moved ashore. Finally, the containers are placed on trucks to be moved directly toward the yard or outside the terminal. This sequence of movements is reversed during loading operations.
- *Yard*. It is a temporal storage that usually takes up more than 50% of the terminal surface and in which those containers arrived at the terminal are stacked until their subsequent retrieval. The yard is split into several blocks, that is, sets of container bays arranged in parallel. A bay is a delimited two-dimensional stock disposed in the vertical direction, where containers can be placed one on top of another. The blocks are served by specialized machinery, such as reach stackers, rubber-tired gantry cranes, or rail-mounted gantry cranes.
- *Landside*. It is an interface to communicate the maritime container terminal with external trucks and trains. Several gates are available to supervise the movement of containers.

Some particular facilities that can be found in maritime container terminals are the container freight station, interchange area, and railhead. These are, respectively, dedicated to managing consignments, allowing road vehicles to deliver and retrieve cargoes from the terminal, and inspecting the containers arriving or leaving by train.

Nowadays, the large volume of freights and the unceasing competition among maritime container terminals give rise to increasing vessel operators' demand for reliable services. A suitable performance of a maritime container terminal is

obtained by harmonizing the technical equipment and staff. However, maritime container terminals are complex to manage due to their large number of heterogeneous processes. In this environment, heuristics stand as a promising optimization technique to be considered. The reason is that, as discussed in this chapter, most of the optimization problems addressed in a maritime container terminal belong to the $\mathcal{NP}$-hard class of problems. In this regard, even in small-size scenarios of some of these optimization problems, efficient exact approaches do not exist. Nevertheless, heuristics report high-quality solutions through affordable computational times.

In the past decade, ML approaches have gained relevant attention in addressing port optimization problems, especially those ML approaches that integrate optimization. Under this paradigm, Machine Learning techniques are used to identify data patterns, predict a variable's value, and direct the search process toward high-quality solutions. The literature reviews conducted in [29, 59] emphasize the predominant use of ML methods in predicting relevant variables in port operations. By analyzing large datasets from the maritime port sector, ML facilitates accurate estimation of variables essential for optimizing maritime container terminal management operations. Moreover, the joint use of ML with heuristics to improve port operations management has shown relevant results, e.g., [19, 20, 81].

The remainder of this chapter introduces the main optimization problems found at a maritime container terminal, how they are interconnected, and analyzes heuristics to address them. Some conclusions and further lines of study are presented at the end of the chapter.

Optimization Problems

As mentioned before, maritime container terminals are huge infrastructures aimed at managing increasing transshipment flows of containers within transport multimodal networks. Their many logistical problems and their interrelations constitute a challenge for practitioners and researchers. Henesey et al. [37] divide the problems taking place at container terminals into four large subsystems or categories, namely *ship to shore*, *transfer*, *storage*, and *delivery*. In this regard, due to the similarity of the operations between transfer and delivery, they are here considered as a single category. Moreover, in the following, some of the most highlighted problems within these categories are briefly described:

- *Ship to shore*. Movements of loaded/unloaded containers from/to the sea to the container terminal:
  - *Ship routing* (Korsvik and Fagerholt [49]). These are particular routing problems where the vessels have to be routed along a set of ports so that their cargo is picked up and delivered from/to them.
  - *Stowage planning* (Monaco et al. [61]). In these problems, the goal is to determine the positions to be occupied by each container within a container vessel along its shipping route.

  - *Berth allocation* (Bierwirth and Meisel [5]). The problems under this category seek to determine the berthing position and berthing time of those container vessels arriving at the terminal.
  - *Quay crane assignment* (Meisel and Bierwirth [60]). Once a vessel is berthed, these problems seek to determine the fixed number of quay cranes to be assigned to the vessel according to its workload.
  - *Quay crane scheduling* (Meisel and Bierwirth [4]). Given a vessel and a subset of the available quay cranes, these problems aim to determine the quay crane schedules for performing the vessel loading/unloading tasks.
- *Transfer and delivery*. Bidirectional movement of containers from the quay to the storage area, from the storage area to the hinterland, or vice versa:
  - *Vehicle dispatching* (Angeloudis et al. [1]). Internal vehicles usually move containers within transshipment flows from the quay to the yard and vice versa. In these problems, the management of those vehicles is addressed.
  - *Gate operations planning* (Chen et al. [12]). The management of vehicles with containers from the yard to the gate and vice versa is dealt by these problems.
- *Storage*. Movement and warehousing of containers until their retrieval for continuing their routes by vessels, trucks, or trains:
  - *Yard crane scheduling* (Gharehgozli et al. [33]). These problems are related to the schedule of the cranes within the yard at a container terminal in such a way that the loading, relocation, and unloading operations are performed.
  - *Container storage* (Caserta et al. [10]). These problems are aimed at determining the movements to be performed by the yard cranes to store/retrieve the containers on/from the yard.

The remainder of this chapter focuses on four particular optimization problems: berth allocation problem, quay crane scheduling problem, vehicle dispatching, and container storage.

Berth Allocation

The berth allocation problem (BAP) is an $\mathcal{NP}$-hard optimization problem that seeks to determine the berthing position and berthing time of each incoming vessel that arrives to the port within a given planning horizon. The primary constraint involved in the BAP concerns that the berthing positions of the incoming vessels cannot be overlapped in the same time step.

According to the arrival of the incoming vessels, two variants of the BAP can be identified: static BAP and dynamic BAP. Firstly, in the static BAP, all the incoming vessels are already waiting to be berthed before the planning horizon starts. Secondly, the dynamic BAP considers that the vessels arrive at the port at any moment during the planning horizon. In both cases, each vessel could have a certain time window in which the vessel must be served by the maritime container terminal.

Moreover, the berth layout of the maritime container terminal gives rise to the following variants of the BAP:

- *Continuous BAP*. In this layout, the quay is treated as a continuous length and not divided into predefined berths; thus, the vessels can berth at any position.
- *Discrete BAP*. The quay is physically divided into segments called *berths* so that each berth can serve a certain number of vessels at each time step.
- *Hybrid BAP*. This type involves discretizing the continuous quay to have intermediate options. The continuous quay is divided into a set of berths, and a vessel can occupy more than one berth at a time or share its assigned berth with other container vessels. On the other hand, other hybrid considerations might be certain parts of the quay with fixed berthing slots while others are treated as continuous.

These categories can include BAP variants that consider time-dependent quay and port access limitations (e.g., [18,52,77]), multiple quays (e.g., [30]), incorporate discontinuities within them (e.g., [58]), irregular layouts (e.g., [17]), or adjustable or mobile quays (e.g., [23]), accommodating a broader range of operational scenarios and infrastructure constraints.

According to the planning level to consider, the BAP can be classified as follows:

- *Operational*. It covers decisions ranging from one up to several days. Thus, the BAP within this level is aimed at optimizing the delays and waiting times of container vessels. This variant of BAP has received more attention than the others.
- *Tactical*. At this level, the decisions cover operations ranging from 1 week up to several months. Some of the objectives here are to optimize the transshipment flows among vessels, cycling visiting of the vessels, fulfilment of contracts among shipping companies and terminal managers, and route design.
- *Strategic*. The decisions covered at this level range from one to several years. They seek to establish specific and dedicated berths, strategic cooperation agreements between terminal and shipping companies, etc.

Over the past few years, the integration of container terminals with other types of terminals, such as bulk or RoRo (Roll-on/Roll-off), has given rise to what is known as multipurpose terminals. These terminals are capable of handling various cargo types, resulting in specific BAPs such as the Bulk Berth Allocation Problem [68] or the BAP in the context of automotive transshipment [21].

In the related literature, most works address the BAP at the operational level. To a lesser extent, some works consider the tactical level, and a few think the strategic level. Due to this, we analyze how heuristics can be used to tackle the BAP at the operational level. In this regard, one of the most used variants of BAP is the dynamic BAP (DBAP) proposed by Imai et al. [42], who address the problem over a discrete layout. Then, Cordeau et al. [16] formulate it as a multi-depot vehicle

routing problem (MD-VRP). Additionally, they also include berth and vessel time windows. The authors propose a benchmark suite based on real data from the container terminal of Gioia Tauro (Italy).

The input data for the DBAP consist of a set of n incoming vessels to serve, denoted as V, a set of m berths, denoted as B, and a planning horizon, H. Each container vessel $v \in V$ has a certain time window, $[a_v, d_v]$, which the maritime container terminal must serve, whereas its service priority is p_v. The handling time of vessel $v \in V$ in berth $b \in B$ is denoted as s_{vb}. This way, the vessel's turnaround time is the sum of its waiting and handling time to be berthed. Furthermore, each berth $b \in B$ has a known time window in which vessels can be served, $[s_b, e_b]$. This problem aims to minimize the sum of (weighted) turnaround times of the vessels.

Figure 2 shows a solution example of the DBAP. This figure shows a schedule and an assignment plan for $n = 6$ vessels in $m = 3$ berths. The rectangles represent the vessels, and service priority is provided inside each vessel. The time windows of the vessels are represented by the lines at the bottom of the figure. In this case, for example, vessel 1 arrives at time step 4, and it should be served before time step 14. Moreover, the time window of each berth is limited by the non-hatched areas. Table 1 reports the different handling times of each vessel depending on the assigned berth. For example, the handling time of vessel 1 at berth 1 would be 7, whereas its handling time would be 8 at berth 2.

As seen in the previous example, vessels 5 and 6 would have to wait for berthing in their respective assigned berths. In this regard, since their service priorities are

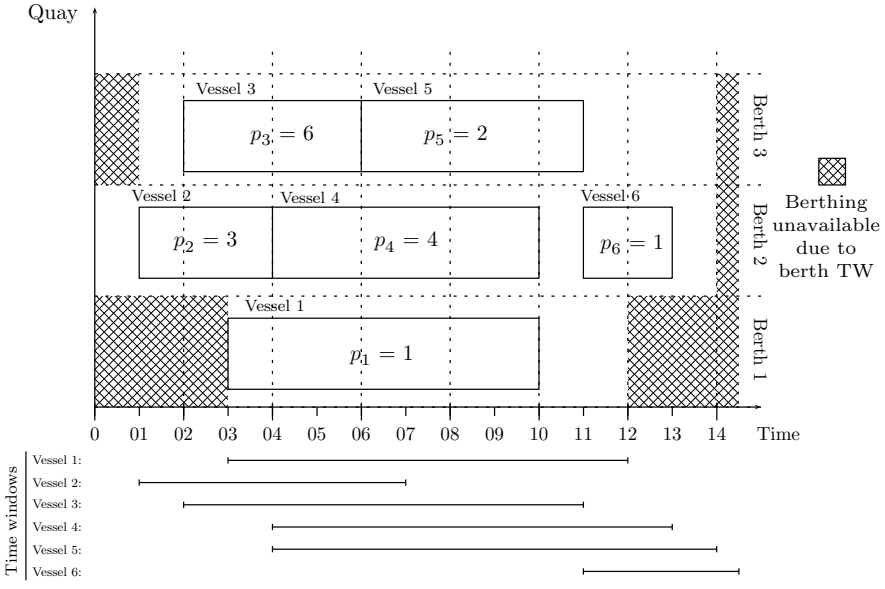


Fig. 2 Solution example of the Dynamic Berth Allocation Problem composed of 6 vessels and 3 berths

Table 1 Vessel handling times depending on the berth

Vessel	Berth 1	Berth 2	Berth 3
1	7	8	5
2	2	3	4
3	5	5	4
4	4	6	5
5	5	8	5
6	4	2	3

$p_5 = 2$ and $p_6 = 1$, their waiting times will have less impact on the objective function value than delaying other vessels, for example, vessels 3 and 4, for which their service priorities are $p_3 = 6$ and $p_4 = 4$, respectively. That is, if their berthing time is delayed, the waiting time of each vessel is multiplied by 6 and 4, respectively. In this example, the weighted turnaround time of the six vessels is calculated as follows. Starting from vessel 1 and increasing a unit until vessel 6, the objective function value is $1 \cdot 7 + 3 \cdot 3 + 6 \cdot 4 + 6 \cdot 4 + 5 \cdot 2 + 2 \cdot 1 = 76$.

At container terminals, some queue rules are used when scheduling incoming vessels to berth positions. The vessels are sorted according to a specific rule. Some examples are described in the following:

- *Random (R)*. The incoming container vessels are randomly selected to be served by the terminal.
- *First come, first served (FCFS)*. This rule is based on serving vessels according to their arrival times. That is, the earliest vessel is the first to be served.
- *Shortest processing time (SPT)*. The vessels are sorted according to their required handling times. In this case, that vessel requiring the shortest handling time berths first.
- *Earliest due date (EDD)*. It serves the vessels based on the end of their time windows. This way, that vessel whose time window ends earliest is served first.

The previous rules allow the permutation of the incoming container vessels to serve. Four heuristics can be obtained using these rules so the vessels are iteratively assigned to the best possible berth. This berth is selected by minimizing the impact on the objective function value. The names of the heuristics are derived from the names of the rules. That is, *random greedy method* (R-G, Cordeau et al. [16]), *first come, first served greedy* (FCFS-G, Cordeau et al. [16]), *shortest processing time greedy* (SPT-G), and *earliest due date greedy* (EDD-G).

Algorithm 1 presents the general framework of the heuristics proposed in the literature (FCFS-G and R-G). It should be noted that the heuristics proposed in this chapter, namely EDD-G and SPT-G, are also included within this general heuristic framework.

As indicated in Algorithm 1, the initial solution, S, is empty (line 1). Its associated objective function value, $f_{obj}(S)$, is set to 0 (line 2). Once this initialization is done, depending on the given rule, a permutation of vessels R is obtained (line 3). So, for example, in FCFS-G, the first element of the permutation, $R(1)$, is the first

Algorithm 1: General heuristic framework for the Berth Allocation Problem

1: $S \leftarrow \emptyset$
2: $f_{obj}(S) = 0$
3: $R \leftarrow$ Create a list of vessels according to the selected rule
4: **for** $i \leftarrow 1$ to n **do**
5: $\quad v \leftarrow$ Select vessel $R(i)$
6: $\quad b \leftarrow$ Select berth that allows the minimum impact on $f_{obj}(S)$
7: $\quad$ Assign v to b in S
8: **end for**
9: Return S

vessel that arrives at port. With the permutation already defined, the vessels are assigned one at a time (line 5) to the best possible berth following the order given by the sorted list (lines 4–8). The best possible berth is determined according to the impact on the objective function value of the solution (line 6). Each vessel is assigned to the berth that least increases the objective function value of the solution built until the moment.

Furthermore, a tabu search (see ▸ Chap. 31, "Tabu Search") for solving this problem is proposed by Cordeau et al. [16]. Their proposed approach, $\mathtt{T^2S}$, [16] uses R-G and FCFS-G for restarting the search. In this regard, to improve the performance of their solution approach, Lalla-Ruiz et al. propose [51], on the one hand, the use of one additional neighborhood structure $\mathtt{T^2S^*}$ and, on the other hand, the use of a path-relinking procedure (see the ▸ Chap. 23, "GRASP"), $\mathtt{T^2S^*} + \mathtt{PR}$, for restating the search. In the path-reliking procedure, an elite set of solutions, ES, consisting of a subset of the local optima encountered during the process, is used to create new starting points for the tabu search. Then, a path-relinking from one solution randomly picked from ES and one generated by R-G is performed.

Algorithm 2 shows the pseudo-code of $\mathtt{T^2S^*} + \mathtt{PR}$. First of all, the elite set, ES, is initialized with the sets of local optima $\mathtt{T^2S}^*_{min}(s_{R-G})$ (line 3) and $\mathtt{T^2S}^*_{min}(s_{FCFS-G})$ (line 4). Then, a random solution, $s_{initial}$, is generated by running R-G (line 6), and a solution $s_{elite} \in ES$ is randomly selected (line 7). Once that, a path relinking connecting $s_{initial}$ with s_{elite} is performed (line 8). Finally, the midpoint solution of this path, s, is selected as the starting point to run the tabu search $\mathtt{T^2S^*}$ (line 9). The local optima reached along this search also update the elite set of solutions (line 9). These steps (lines 5–9) are repeated until a stopping condition is met.

Comparative Analysis

This subsection is devoted to comparing the heuristics to solve the DBAP. The problem instances used to evaluate the proposed heuristics represent the problems proposed by Cordeau et al. [16]. These instances were generated by considering a statistical analysis of the traffic and berth allocation data at the maritime container terminal of Gioia Tauro (Italy) [15]. Moreover, a representative set of instances from those proposed by Lalla-Ruiz et al. [51] are used. All the heuristics have

Algorithm 2: Tabu search with path relinking for the dynamic berth allocation problem

```
1: s_{R-G} = R-G()
2: s_{FCFS-G} = FCFS-G()
3: ES = T^2S^*_{min}(s_{R-G})
4: ES = ES ∪ T^2S^*_{min}(s_{FCFS-G})
5: while !stopping condition do
6:     s_{initial} = R-G()
7:     s_{elite} = Select(ES)
8:     s = PR(s_{initial}, s_{elite})
9:     ES = ES ∪ T^2S^*_{min}(s)
10: end while
```

been implemented in ANSI C and executed on a computer equipped with an Intel 3.16 GHz and 4 GB of RAM.

Table 2 illustrates the results obtained for a representative set of instances aforementioned. The first three columns correspond to the problem instance description, namely the number of vessels, n, and, number of berths, m, and the instance identifier, *Id*. In columns *R-G*, *FCFS-G*, *SPT-G*, and *EDD-G*, the objective function values provided by the different heuristics for the DBAP are reported. That is, the objective function value, $Obj.$, and the computational time measured in seconds, *t(s.)*. In the last row, the average values are provided.

Furthermore, as indicated by Lalla-Ruiz et al. [51], the GSPP model implemented in CPLEX runs out of memory for some problem instance sets where other characteristics are considered. Therefore, in Table 3, the results provided by $\mathrm{T}^2\mathrm{S}$, $\mathrm{T}^2\mathrm{S}^*$, and $\mathrm{T}^2\mathrm{S}^* + \mathrm{PR}$ are reported for a set of instances where CPLEX is not able to provide a feasible solution for any instance within it. The table is divided, in column $Instance$, containing the number of vessels, n, number of berths, m, and the instance identifier, *Id*. In columns $\mathrm{T}^2\mathrm{S}$, $\mathrm{T}^2\mathrm{S}^*$, and $\mathrm{T}^2\mathrm{S}^* + \mathrm{PR}$ the objective function value, $Obj.$, and the computational time measured in seconds, *t(s.)*, are reported. Moreover, the relative error, $Gap_1(\%)$, with respect to $\mathrm{T}^2\mathrm{S}$ is also reported. In the case of $\mathrm{T}^2\mathrm{S}^* + \mathrm{PR}$, the relative error, $Gap_2(\%)$, with respect to $\mathrm{T}^2\mathrm{S}^*$ is shown.

As seen in Table 2, the best heuristic for the DBAP, irrespective of the size of the instance, is the FCFS-G. On the other hand, R-G and SPT-G exhibit, on average, a similar performance in terms of objective function value, while the policy of serving the vessels according to the due date reports the worst values for these instances. It should be noted that the computational time required is less than 0.001 s, as expected for these heuristics. This advises their use as initialization methods. In this regard, FCFS-G and R-G are the heuristics most used in the related literature, which, in light of the results, are the best options in terms of the quality of the solutions. It is important to highlight that R-G is not a deterministic heuristic. Therefore, it provides different solutions that may be beneficial in multi-start methods.

The results shown in Table 3 indicate that among the three algorithms, the use of a restarting strategy based on path-relinking between a solution from an elite set and one generated by the R-G heuristic improves the quality of the solutions in terms

Table 2 Comparison of the heuristics for the Discrete Berth Allocation Problem. Best values in bold

Instance			R-G		FCFS-G		SPT-G		EDD-G	
n	*m*	*Id*	Obj.	t (s.)	Obj.	t (s.)	Obj.	t (s.)	Obj.	t (s.)
30	3	1	3237	<0.001	**2039**	<0.001	2667	<0.001	4702	<0.001
		2	3523	<0.001	**2829**	<0.001	4246	<0.001	3903	<0.001
40	7	1	2397	<0.001	**1612**	<0.001	3102	<0.001	4185	<0.001
		2	2987	<0.001	**1633**	<0.001	2755	<0.001	3961	<0.001
55	7	1	5874	<0.001	**3591**	<0.001	5695	<0.001	7576	<0.001
		2	5761	<0.001	**3565**	<0.001	5084	<0.001	7246	<0.001
60	13	1	2802	<0.001	**1437**	<0.001	2776	<0.001	2987	<0.001
		2	2861	<0.001	**1278**	<0.001	2761	<0.001	3361	<0.001
Average:			3680.25	<0.001	2248	<0.001	3635.75	<0.001	4740.13	<0.001

Table 3 Comparison among $\mathtt{T^2S}$, $\mathtt{T^2S^*}$, and $\mathtt{T^2S^*}$ + PR for the instances proposed by Lalla-Ruiz et al. [51]. Best values in bold

Instance			$\mathtt{T^2S}$		$\mathtt{T^2S^*}$			$\mathtt{T^2S^*}$ + PR			
n	*m*	*Id*	Obj.	t (s.)	Obj.	$\text{Gap}_1(\%)$	t (s.)	Obj.	$\text{Gap}_1(\%)$	$\text{Gap}_2(\%)$	t (s.)
40	7	1	1489	8.16	1467	−1.48	1.20	**1460**	−1.95	−0.48	1.11
		2	1423	8.26	1381	−2.95	1.01	**1375**	−3.37	−0.43	1.32
		3	2149	8.13	**2119**	−1.40	0.84	**2119**	−1.40	0.00	1.17
		4	1618	7.99	1600	−1.11	1.18	**1597**	−1.30	−0.19	1.78
		5	1885	8.37	1849	−1.91	1.11	**1847**	−2.02	−0.11	1.45
		6	2104	8.11	**2080**	−1.14	0.86	**2080**	−1.14	0.00	1.37
		7	1863	7.91	1845	−0.97	1.25	**1841**	−1.18	−0.22	1.56
		8	2040	8.16	**2026**	−0.69	1.18	**2026**	−0.69	0.00	1.70
		9	1901	8.01	1888	−0.68	1.06	**1880**	−1.10	−0.42	1.48
		10	1922	7.89	1905	−0.88	0.71	**1892**	−1.56	−0.68	1.59

of the objective function value. In this regard, $\mathtt{T^2S^*}$, and $\mathtt{T^2S^*}$ + PR can provide a feasible solution in less than 2 s.

Recent Heuristics Developments for the DBAP

The limitations of solving the DBAP with exact algorithms have been addressed in recent years. Kramer et al. [50] proposed improved optimization models and a variable-fixing exact approach to accelerate the solving of the DBAP and enable the solving of larger DBAP scenarios. While computational times remained practical for small to medium-sized instances, solving the largest proposed scenarios, ranging from 200 to 250 vessels, to optimality within a 2-hour limit was unreachable. This finding is particularly relevant, given the need for fast approaches in port contexts

requiring frequent execution, such as re-optimization frameworks or approaches coping with dynamic events.

Regarding heuristics, despite the development of numerous metaheuristics over the past 15 years aimed at solving the DBAP, relatively few new heuristic strategies have been introduced. Wawrzyniak et al. [71] listed other possible heuristics, including FCFS, SPT, and R. The additional greedy heuristics are also based on a sorting rule and range from the Longest Processing Time (LPT) to the Greatest Importance (GI) heuristic. Moreover, combinations such as Greatest Importance-Shortest Processing time (GISPT) or vice versa (SPTGI) were considered. Lastly, Ankita and Mathirajan [2] introduced a heuristic method based on an early-to-finish strategy. This particular heuristic combines the principles of FCFS and SPT, ordering vessels by their expected completion time per each combination vessel berth and prioritizing, in case of a tie, the combination regarding the maximum difference between the two least handling times.

It is well known that the arrival times of vessels at port might differ from the scheduled ones. This fact significantly impacts and disrupts berth plans, making them less efficient and potentially unfeasible (e.g., overlap between vessel schedules). In this sense, machine learning approaches are useful for forecasting the arrival times at the port. In Lorenz et al. [48], a robust optimization procedure based on Dynamic Time Buffers that includes estimated arrival times is proposed. This combined approach significantly reduces the number of overlaps. By using ML approaches, stochastic heuristic approaches like those proposed in [20] can reinforce berth planning. The authors proposed simulation-optimization heuristic approaches to proactively and proactively-and-reactively generate robust berth plans to cope with varying arrival and handling times. Their approach also considered the incorporation of time buffers to absorb the impact of disruptions caused by changes in the vessels' arrival times.

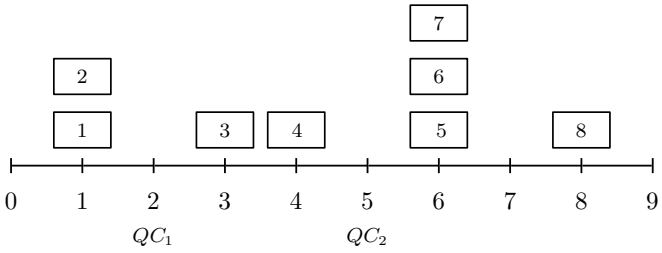
Quay Crane Scheduling Problem

The quay crane scheduling problem (QCSP) can be defined as the problem of determining the finishing times of the tasks performed by each quay crane allocated to a container vessel to minimize its turnaround time (Meisel and Bierwirth [4]). Input data for the QCSP consist of n tasks, denoted as Ω, and a set of m quay cranes, denoted as Q. Each $t \in \Omega$ comprises a set of containers with similar characteristics (e.g., weight, destination port, dimensions) placed together in the same cross section, l_t, into the container vessel. The processing time of t is the time a quay crane requires to load or unload its containers and is denoted as p_t. Furthermore, each $q \in Q$ is available after its earliest ready time, r^q; is initially located on a cross section, l_0^q; and can travel between two adjacent positions with a travel time, $\hat{t}$.

The QCSP has a set of particular constraints. In the first place, within each cross section of the container vessel, tasks are sorted according to known precedence relationships. Thus, tasks have to be performed orderly: unloading operations before

loading operations, loading operations into a hold before loading operations on the deck, and so forth. Moreover, the quay cranes are rail mounted, and then, they cannot cross each other. Additionally, they have to keep a safety distance, δ, between them in order to prevent possible collisions (Kim and Park [46]).

Figure 3 illustrates an example of the QCSP and a feasible schedule for it where quay cranes are available from the start of the service time (e.g., $r^q = 0, \forall q \in Q$), $\delta = 1$, and $\hat{t} = 1$. Within each cross section of the vessel, tasks are sorted. For instance, task 5 has to be performed before task 6 and task 6 before task 7. In this example, tasks 1, 2, 3, 5, 6, and 7 are performed by quay crane 1, and tasks 4 and 8 by quay crane 2. The turnaround time of this container vessel is 52 time units.



Task, t	1	2	3	4	5	6	7	8
Position, l_t	1	1	3	4	6	6	6	8
Processing Time, p_t	10	8	10	15	7	6	5	10

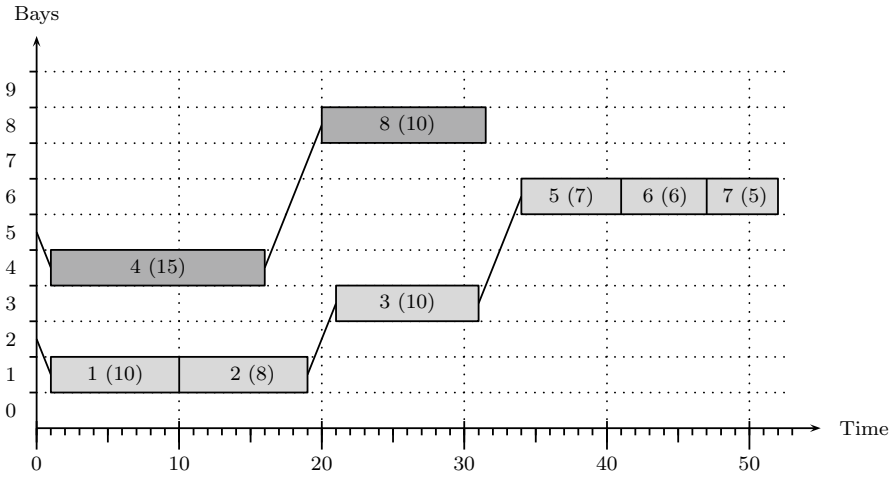


Fig. 3 Example of Quay Crane Scheduling Problem composed of $n = 8$ tasks and $m = 2$ quay cranes

Algorithm 3: Hybrid estimation of distribution algorithm with local search for the quay crane scheduling problem

1: $g = 0$
2: Initialize the probabilistic learning model, $\mathcal{P}(g)$
3: $\mathcal{S}(g) \leftarrow$ Generate the initial population from $\mathcal{P}(g)$
4: **repeat**
5: $\quad \mathcal{S}_e(g) \leftarrow$ Select the subset of elite solutions from $S(g)$
6: $\quad \mathcal{S}(g+1) \leftarrow$ Create population composed of solutions in $\mathcal{S}_e(g)$ and solutions from $\mathcal{P}(g)$
7: $\quad$ **if** λ generations without improvement **then**
8: $\quad\quad$ Apply Local Search to best solutions from $\mathcal{S}(g)$
9: $\quad$ **end if**
10: $\quad \mathcal{P}(g+1) \leftarrow$ Create next probabilistic learning model
11: $\quad g = g + 1$
12: **until** Stopping criterion is met

An interesting heuristic approach to solve the QCSP is to explore the search space of unidirectional schedules (Meisel and Bierwirth [4]). A schedule is referred to as unidirectional if all quay cranes move with a similar sense of movement along their service times. In the following, a hybrid estimation of distribution algorithm with local search (EDA/LS, Expósito-Izquierdo et al. [24]) is described to solve the QCSP from a unidirectional point of view. This approach uses prior knowledge about the problem to achieve high-quality solutions. Additionally, a novel restart strategy (see ▸ Chap. 12, "Restart Strategies") is introduced to prevent the premature convergence of the search and guide it to insufficiently explored regions. Finally, the proper finishing time of the search is determined by means of an adaptive stopping criterion.

Algorithm 3 depicts the pseudocode of the EDA/LS. The proposed scheme keeps, at each generation g, a population $\mathcal{S}(g)$ with N solutions (line 3). Additionally, a probabilistic learning model $\mathcal{P}(g)$ based on a probability matrix with m rows and n columns is considered (line 2). Each value $p_{qt}(g)$ defines the probability of assigning task $t \in \Omega$ to quay crane $q \in Q$ in the model sampling step, as follows:

$$\mathcal{P}(g) = \begin{pmatrix} p_{11}(g) & p_{12}(g) & \cdots & p_{1n}(g) \\ p_{21}(g) & p_{22}(g) & \cdots & p_{2n}(g) \\ \vdots & \vdots & \ddots & \vdots \\ p_{m1}(g) & p_{m2}(g) & \cdots & p_{mn}(g) \end{pmatrix}$$

In most EDAs, the initial population is randomly created according to the uniform distribution over all feasible solutions. The QCSP structure allows the inclusion of a priori knowledge in the scheme of EDA/LS straightforwardly. Intuitively, the probability that a quay crane performs a certain task in a promising solution is highly influenced by the distance between them. The initialization step here is based on a Gaussian function in a discrete way to compute initial assignment probabilities. Its use is based on the fact that it lets to assign a higher weight to those quay

cranes initially located close to the corresponding task, and it is decremented as the distance between them increases. At each new generation, a new population is obtained. The new population is composed of the subset of solutions with the lowest objective function value, $\mathcal{S}_e(g)$, (defined by a percentage α, whose value is set by the user) from the previous population (elitism criterion) and new unidirectional ones sampled from the probabilistic learning model (line 6). In the sampling step, the tasks are assigned to a specific quay crane considering the cross sections in which they are located (from the leftmost up to the rightmost) and following the precedence relationships. At each step, one of the available quay cranes is selected according to the roulette wheel selection based on a pseudo-random generator. The subset of elite solutions is also used to update the probabilistic learning model so that the probability of a task being assigned to a quay crane is defined by the number of times that assignment has been previously performed. This is formally expressed as follows:

$$p_{qt}(g) = \frac{c_{qt}(g)}{\sum_{k \in Q} c_{kt}(g)}, \forall q \in Q, t \in \Omega, \tag{1}$$

where $c(g)$ is a matrix composed of m rows and n columns such that $c_{qt}(g)$ is the number of times task $t \in \Omega$ has been performed by quay crane $q \in Q$ in a high-quality solution found during the search.

In order to overcome the lack of intensification of EDAs, this scheme incorporates a local search. This local search is based on a neighborhood structure composed of movements of reassignment and interchange of tasks between quay cranes. The next current schedule in the local search is selected according to the first improvement strategy. Additionally, the local search is applied to a subset of solutions with the lowest objective function value (line 8) defined by the percentage β (whose value is set by the user) after $\lambda > 0$ generations without improvement (lines 7–9). After applying the local search, a subset of problem variables, $\mathcal{V}$, is selected. $\mathcal{V}$ is defined by the set of tasks performed by different quay cranes in the improved solutions. The effectiveness of the proposed local search is used as an indicator of the search process convergence so that the algorithm execution is stopped when it is not possible to improve any solution from the selected subset of solutions from the population.

Computational Analysis

This section compares the EDA/LS described earlier and the most competitive algorithm from the related literature: unidirectional scheduling and UDS (Bierwirth and Meisel [4]). This comparison is carried out on a benchmark suite proposed by Kim and Park [46] and subsequently extended by Bierwirth and Meisel [4] (90 instances). For each instance, the EDA/LS has been executed on a computer equipped with an Intel 3.16 GHz and 4 GB of RAM, whereas the computational results presented for UDS are those reported in the corresponding original paper. Computational times for those instances that cannot be solved within the maximum computational time of 1 h by the UDS are not reported.

Table 4 Comparison between EDA/LS and UDS. Largest instances

	UDS		EDA/LS		
Instance	f_{UDS}	$t_{UDS}(m.)$	f_{EDA}	$t_{EDA}(m.)$	Gap (%)
k83	948	6,37	948	0,63	0,00
k84	897	3,29	897	0,68	0,00
k85	972	5,82	972	0,68	0,00
k86	816	–	819	0,63	0,37
k87	**867**	–	867	0,80	0,00
k88	768	43,73	768	0,81	0,00
k89	843	10,96	843	0,66	0,00
k90	**1053**	24,95	1056	0.69	0,29
k91	**837**	10,74	840	0,70	0,36
k92	**897**	34,61	903	0,62	0,67
Avg.:	889,80	>60,00	891,30	0,69	0,17
k93	**816**	–	822	0,86	0,74
k94	**786**	–	795	0,92	1,15
k95	834	–	834	0,99	0,00
k96	819	–	**804**	0,89	−1,83
k97	720	–	**717**	**0,93**	−0,42
k98	**735**	23,97	741	0,89	0,82
k99	852	–	855	1,03	0,35
k100	900	–	**891**	**1,07**	−1,00
k101	**813**	–	816	0,87	0,37
k102	903	–	**900**	**0,86**	**−0,33**
Avg.:	817,80	>60,00	817,50	0,93	−0,04

Table 4 shows an extract of the computational results in the largest instances ($k83$–$k102$). The objective function values (f_{UDS} and f_{EDA}) and computational times ($t_{UDS}(m.)$ and $t_{EDA}(m.)$) are shown for each approach. The results demonstrate the suitability of the EDA/LS when solving the QCSP. It is effective and efficient when solving the largest problem instances since it presents short gaps, and the best solutions for four problem instances have been improved. In addition, the computational times are lesser than 1 min in all cases.

Internal Vehicle Scheduling

The internal vehicle scheduling problem (IVSP) is an $\mathcal{NP}$-hard optimization problem that consists of assigning jobs to the internal vehicles of the container terminal, in such a way that the time the incoming vessels spend at the port (i.e., their turnaround times) is minimized. In the related literature, this problem has been treated either as vehicle routing problem (VRP) or as scheduling problem (the reader is referred to ▸ Chap. 51, "Particle Swarm Optimization for the Vehicle Routing

Problem: A Survey and a Comparative Analysis" for VRP and ▸ Chap. 56, "Supply Chain Management" for further details). In the IVSP, a set of n vehicles, denoted as V, and a set of m jobs, denoted as $W = \{W_1, W_2, \ldots, W_m\}$, are known. In addition, a set of locations, denoted as L, in which the jobs can start and end is given. A job $j \in W$ corresponds to the transportation of a container from a pick-up location to a delivery location that must be performed from a certain time, t_j. There are two types of jobs:

- *Loading job*. It is the transportation of a container from the yard toward the vessel into which it is going to be loaded. Thus, the pick-up location is a stack on the yard, and the delivery location is at the corresponding quay crane.
- *Discharging job*. It is the transportation of a certain container from its source vessel toward the yard. Therefore, the pick-up location is at the corresponding quay crane, and the delivery location is a known stack on the yard.

Thereby, each job has to be assigned to one of the vehicles. This way, each vehicle can perform a sequence of jobs so that after completing a job, a vehicle can start another one. It should be noted that each job consists of (i) an empty drive of the corresponding vehicle from its last location $i \in L$ to the pick-up location, $k \in L$, whose time is t_{ik}, (ii) a hand-over time at the pick-up location, h_p, (iii) a drive to the delivery location, $l \in L$, and (iv) a hand-over time at the delivery location, h_d. Figure 4 illustrates these times involved in a job, where $departure$ corresponds to location i, $Source\ quay\ crane/stack$ corresponds to location k of the quay crane or a stack depending on the type of job, and finally, $destination\ quay\ crane/stack$ corresponds to location l of the quay crane or stack depending on the kind of job.

Once a vessel arrives at the terminal, its containers are first discharged from the vessel onto the internal vehicles by the available quay cranes. Next, the internal vehicles transport the containers to their pre-specified stacks in the yard. When a vehicle arrives at its destination, a container is unloaded from it by a yard crane. Typically, after most or all containers have been discharged from the vessel, the outgoing containers from the yard are transported to the quay by those internal vehicles and are loaded into the vessel by the quay cranes.

A fast transshipment of containers associated with the vessels is important to both the shipping companies since they can reduce their operational costs, and the

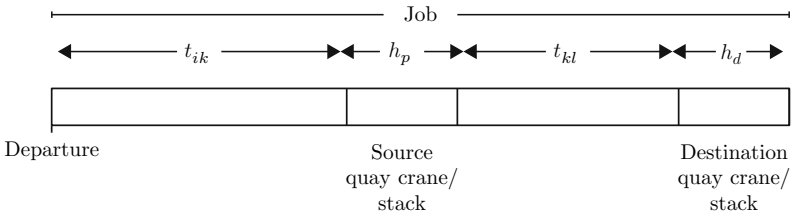


Fig. 4 Breakdown of the times involved in a job

terminal, which can serve many vessels per day. However, in order to achieve this reduction of the turnaround time, different objective functions can be defined, such as minimizing the quay crane waiting times for vehicles, minimizing the vehicles waiting times at quay cranes, minimizing the empty travel times, and minimizing of the cost associated with the jobs.

There are different dispatching approaches to establish the relationship between vehicles and quay cranes. Traditionally, in the static dispatching approach, one vehicle serves only one quay crane. When it finishes an unloading job in the corresponding block in the yard, it must travel back to the bounding quay crane for another unloading job. Empty travel always happens with this dispatching approach. Furthermore, a dynamic dispatching approach means the vehicles can serve any quay crane. When a vehicle finishes its unloading job in the blocks, it can choose either a loading job near it or an unloading job at the quayside. A dynamic dispatching model will help reduce empty travel.

Moreover, a variety of transporters are used in container terminals, such as yard trucks (YTs), multi-trailers, straddle carriers, automated guided vehicles (AGVs), and automated lift vehicles (ALVs). Their heterogeneous technical characteristics allow them to cover many practical scenarios. In this regard, selecting and dispatching technical machinery around a given maritime container terminal are a strategic decision that must be considered according to the environment's features, the cargo volume, etc.

Therefore, when addressing the IVSP, the terminal decision-maker must consider the following: firstly, how many and what type of vehicles are deployed at the terminal; secondly, what kind of relationship is established between vehicles and quay cranes; and finally, what will be the objective to achieve.

Since IVSP is an operational problem (i.e., it has to be solved daily) and is usually integrated with other closely related problems in container terminals (e.g., quay crane scheduling, container storage), it is important to solve it quickly and provide results of the highest possible quality. For this reason, heuristics are appropriate methods to tackle this problem. However, most of the heuristics proposed in the literature are based on particular formulations of the problem with different objective functions, which hinders the comparison among them (e.g. Angeloudis and Bel [1] maximize benefits; Bish et al. [6] minimize travel distances and the transportation time; Briskorn, et al. [8] minimize the weighted sum of earliness, tardiness, and empty travel time; Shang [66] minimizes a weighted sum of cost, etc.). In the following, we describe some basic heuristic strategies to solve the problem:

- *First come, first served (FCFS)*. Each idle vehicle selects the job with the earliest emergence time. In this case, the assignment concept is reversed. This is the conventional scheduling rule followed in container terminals to dispatch vehicles, and its results are taken as reference when a new method is proposed (Shang [66]):
- *First*. A due job will be assigned to the first available vehicle (Angeloudis and Bell [1]).

- *Fastest*. A due job will be assigned to the vehicle that needs less time to complete the job when it finishes its previous one.
- *Earliest*. A due job will be assigned to that vehicle, which will take less time to be available and complete the job.
- *Closest*. A newly idle vehicle will be assigned to the job with the shortest setup operation. In this case, the assignment concept is reversed (Angeloudis and Bell [1]).
- *Shortest-Empty-Travel-Time-First* ($SETTF$). This is based on the FCFS strategy. When a vehicle completes its current job, an unassigned container with the shortest empty travel time is allocated to this vehicle [70].
- *Greedy*. A vehicle will be assigned to the least costly job. The cost of performing job $j \in W$ once the job $i \in W$ has been already performed is denoted as C_{ij} and formally expressed as follows (Angeloudis and Bell [1]):

$$C_{ij} = \alpha L_{ij} + \beta T_{ij} + \gamma U_{ij} + \delta D_i \tag{2}$$

where α, β, γ, and δ are weights, L_{ij} is the distance between the location of the vehicle when finishing job i and the location of the vehicle when starting job j, T_{ij} is the travel time (expected) from the location of the vehicle when finishing of job i to the location of the vehicle when starting of job j, U_{ij} is the uncertainty index associated with transition from job i to job j, and, finally, D_i is equal to 0 if i is not a current vehicle task (assigned job, maintenance, or idle state) or equal to the expected remaining time until job completion i, otherwise.

Figure 5 illustrates an example of a solution, for instance, with four vehicles and ten jobs using $First$ heuristic. The times the jobs can be performed (i.e., t_j, where $j \in W$) are marked by a dashed line. Firstly, four jobs appear at time zero, and they are assigned to the available vehicles. Then, a new job W_5 arises at time instant 5,

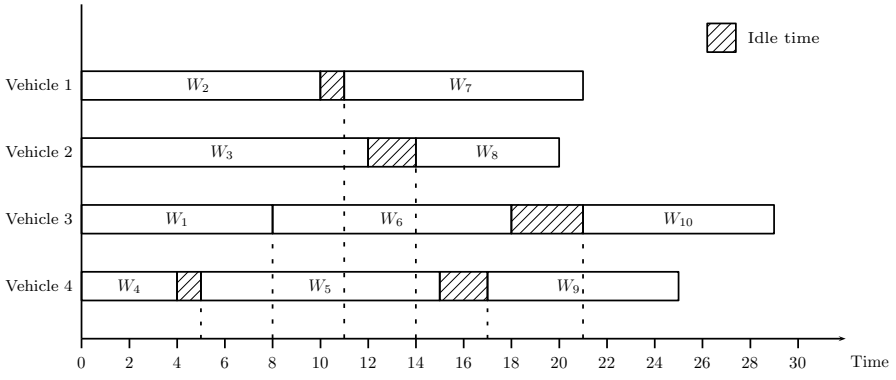


Fig. 5 Example of Vehicle Scheduling Problem using $First$ heuristic

Algorithm 4: General heuristic framework for the Vehicle Scheduling Problem

```
1: S ← ∅
2: f_obj(S) = 0
3: Ω ← List of jobs ordered by appearing time
4: V ← Set of vehicles
5: for i = 1 → m do
6:     v ← Select vehicle from V servicing Ω(i) with the minimum impact on f_obj(S)
7:     Assign v to Ω(i) in S
8:     Update the arrival and departure times of v
9: end for
10: Return S
```

and it is assigned to $Vehicle$ 4, which is the first available vehicle. Next job W_6 appears at time instant eight, and it is assigned to $Vehicle$ 3, which is available just at this time. The process continues following this strategy so that it ends at time 29. A vehicle remains idle whenever a job is not assigned to a vehicle.

Finally, Algorithm 4 shows a heuristic framework for this problem. A minimization function is supposed to be used. Firstly, an empty solution is established, and the objective function is set to zero (lines 1–2). Then, for each job that appears during the planning horizon, a vehicle $v \in V$ is chosen to make it, trying to minimize the impact on the objective function value (line 6). These partial solutions are included in the final one (line 7). Finally, when all the jobs included in Ω have been already assigned to one of the vehicles, the process finishes, and the solution is returned (line 9).

Computational Analysis

This subsection compares some heuristics for the VSP. As mentioned before, the objective function of this problem may vary from one work to another due to the different ways of reducing the turnaround. For this reason, some performance indicators have been used here to compare three of the heuristics explained earlier: $first$, $closest$, and $greedy$. In the following, several indicators are presented to evaluate the performance of the heuristics by means of simulation (Angeloudis and Bell [1]). That is:

1. Vehicle productivity (moves/h): It examines the number of container moves that can be accomplished within an hour by a single vehicle in the fleet so that, unlike the remaining indicators, the higher, the better.
2. Average job delay (s.): It measures the average period between the emergence of a new job and its execution time.
3. Fleet utilization (%): It reflects the percentage of the fleet likely to be involved in a handling operation.
4. Average distance not in-service (m./job): It measures the average traveled distance between the location of the vehicle when finishing a job and the location of the vehicle when starting another.
5. CPU time per instance (ms.): It is the CPU time for solving an instance.

Table 5 Comparison of the heuristics for the Vehicle Scheduling (Angeloudis and Bell [1]). Best heuristic values in bold

	BKS	First		Closest		Greedy	
	Value	Value	Gap(%)	Value	Gap(%)	Value	Gap(%)
1	8.87	5.43	38.78	**5.66**	36.19	5.58	37.09
2	304.00	329.00	8.22	329.00	8.22	**325.00**	6.91
3	93.00	**99.00**	6.45	**99.00**	6.45	**99.00**	6.45
4	355.92	743.11	108.79	**713.71**	100.53	744.78	109.25
5	669.00	**0.00**	−100.00	1.00	−99.85	2.00	−99.70

Table 5 summarizes the computational results for $first$, $closest$, and $greedy$ heuristics. The first column shows the number associated with each performance indicator. The second column shows the best-known solutions (BKS). The remaining columns show the results obtained by each heuristic and the gap regarding BKS values. As can be seen, the heuristic that provides the best results for more indicators is $closest$, which presents the lowest gap for indicators 1, 3, and 4, as well as a very short CPU time. This highlights that serving a job according to the closeness in terms of setup operations benefits, on the overall, the quality of the solution in the majority of the indicators.

Trends in IVSP

Research in the realm of IVSP has evolved from simple rules and heuristics to more sophisticated algorithms, particularly metaheuristics. While heuristics are computationally efficient, their inherently greedy and myopic nature often leads to suboptimal operational outcomes. Metaheuristics, on the other hand, provide superior performance by incorporating advanced search mechanisms. Unlike simpler heuristics, metaheuristics consider a broader spectrum of possibilities and interactions, allowing them to achieve optimal or near-optimal solutions. Table 6 summarizes the main features considered in the most recent IVSP studies, highlighting this shift toward more complex and effective solution strategies. Thus, the use of metaheuristics represents a paradigm shift in addressing IVSP. Among the various metaheuristics, Evolutionary Computation (EC) and, specifically, Genetic Algorithms (GA) have emerged as the forefront solutions for addressing the challenges of IVSP. These algorithms harness principles inspired by natural evolution, employing genetic operators such as crossover and mutation to refine solutions and explore the solution space more comprehensively and iteratively. The adaptability and robustness of GA and EC contribute to their effectiveness in handling the complexities inherent in the problem.

Additionally, holistic approaches to addressing the challenges within container terminals have arisen. One noteworthy trend is the integration of scheduling and routing for internal vehicles, recognizing the interdependence of these two facets in optimizing terminal operations. The joint consideration of scheduling and routing ensures a more comprehensive and synergistic approach, enabling a finer

Table 6 Features and trends in IVSP

Ref.	Problem	Vehicle	Method	Multi-objective	Objectives	Integrated (QCs and YCs)	Conflict-free path planning	Variable speed
[79]	Scheduling+Routing	AGV	GA	No	Minimize makespan	Yes	Yes	No
[69]	Scheduling+Routing	AGV	GA-ACO	Yes	Minimize energy consumption/minimize travel distance	No	No	Yes
[39]	Scheduling	Yard Trucks	GA	Yes	Minimize makespan/minimize non-loaded traveling distance	No	No	No
[83]	Scheduling+Routing	AGV	GA-PSO	No	Minimize delay time of AGVs	Yes	Yes	Yes
[67]	Scheduling+Routing	AGV	GAs	No	Minimize maximum makespan of all containers	Yes	Yes	No
[57]	Scheduling	AGV	GA	No	Minimize makespan	Yes	No	No
[40]	Scheduling+Routing	AGV	A*	No	Minimize travel distance and time	No	Yes	No
[13]	Scheduling	Yard Trucks	GA	No	Minimize makespan	No	No	No
[76]	Scheduling+Routing	AGV	GA + Reinforcement Learning	No	Minimize makespan	Yes	Yes	Yes
[80]	Scheduling+Routing	AGV	Dijkstra + Qlearning	No	Minimize cost	No	Yes	No
[70]	Scheduling+Routing	AGV	Greedy heuristic	No	Minimize makespan	Yes (partially)	Yes	No
[74]	Scheduling	AGV	Greedy heuristic	No	Minimize energy consumption	Yes	No	Yes
[55]	Scheduling+Routing	AGV	GA	No	Minimize travel distance	No	Yes	Yes
[22]	Scheduling	AGV	NSGA-II	Yes	Minimize makespan/minimize unladen time of AGVs	Yes (partially)	No	No

balance between operational efficiency and resource utilization. Conflict-free path planning has also emerged as a critical aspect of IVSP, addressing the potential bottlenecks and inefficiencies arising from collisions or congested pathways. These advancements enhance operational safety, mitigate delays, and optimize overall terminal efficiency by incorporating sophisticated path-planning algorithms that ensure vehicle conflict-free trajectories.

Vehicle variable speed considerations represent yet another noteworthy trend. Recognizing that different operational scenarios may warrant varied vehicle speeds, integrating variable speed profiles into scheduling algorithms enables adaptability to dynamic terminal conditions. This flexibility enhances the responsiveness of vehicle operations. On the other hand, from an objectives perspective, another trend involves the consideration of multiple objectives in IVSP algorithms. Some recent research has recognized the multifaceted nature of terminal operations, incorporating diverse objectives such as energy efficiency and resource utilization. This multifaceted optimization contributes to a more nuanced and adaptive vehicle scheduling framework, aligning with the diverse goals of modern container terminals.

Moreover, recent research is increasingly acknowledging the interconnected nature of terminal equipment. The integration of scheduling for vehicles and other critical equipment, such as QCs and YCs, is gaining prominence. This holistic perspective recognizes that the efficiency of vehicle operations is intrinsically linked to the scheduling and coordination of various terminal resources. As such, an integrated approach ensures a synchronized and optimized workflow across different facets of terminal operations.

Container Storage

Container storage is addressed at three different levels: strategic, tactical, and operational, within maritime container terminals. At the strategic level, decisions include defining the yard layout and choosing the type of handling machinery, such as rubber-tired gantry cranes, rail-mounted gantry cranes, and overhead bridge cranes. The tactical level involves determining the storage capacity of the yard and deploying the handling machinery. Operational decisions focus on the short-term movements of containers within the yard, ranging from a few hours to several days.

Figure 6 illustrates a traditional layout of yard in a maritime container terminal in which the top and horizontal views are shown. It comprises a set of similar three-dimensional storage areas, termed yard blocks. In this case, five-yard blocks are depicted. The yard blocks can be placed in a perpendicular (as Fig. 6) or parallel direction to the quay. Each yard block is composed of a set of bays (e.g., ten bays in Fig. 6). A bay is a two-dimensional storage composed of S homogeneous stacks made up of T tiers each one (e.g., bays in Fig. 6 are composed of $S = 8$ stacks and $T = 5$ tiers). The storage capacity of a terminal is expressed in terms of the number of containers that can be stored on it and derived from the dimensions of its yard

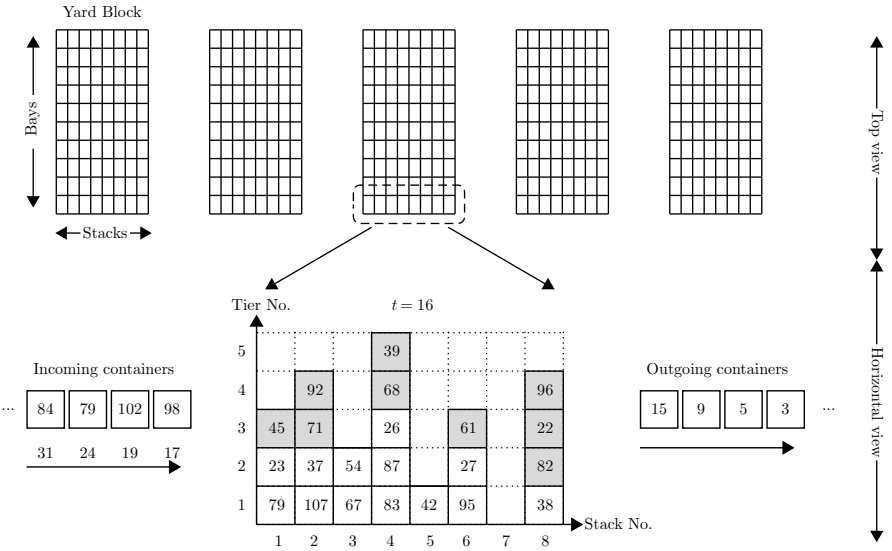


Fig. 6 Layout of a yard in a maritime container terminal

blocks. In this regard, the capacity of a given bay depends upon its stacks and tiers, whereas the capacity of a yard block is the sum of the capacities of its individual bays.

Yard Crane Scheduling Problem

Yard cranes usually manage the yard blocks in a maritime container terminal. Traditionally, one crane is deployed in each yard block. These handling machinery move along the yard blocks through a pair of rail tracks placed on both sides. To access a certain container, the crane is positioned over the bay in which the container is located. The trolley is moved toward its stack, and the spreader is lowered to achieve the container. Finally, the container is hooked up and picked up to be removed from its current location.

In container terminals, multiple challenges emerge on the yard side, necessitating strategic solutions. These include designing schedules for yard cranes and trucks, orchestrating efficient vehicle paths throughout the terminal, and making critical decisions on container storage locations and stacking methodologies within storage areas. Both incoming and outgoing containers spend some time in container terminals awaiting shipment via vessels or delivery by trucks to their final destinations. The yard cranes manage the exchange of containers within storage areas or with the yard trucks. Additionally, yard cranes carry out maintenance tasks like premarshaling and relocating containers.

The yard crane scheduling problem (YCSP) determines the order of container movements carried out by each yard crane. This plays a highlighted role in the

competitiveness of container terminals. Efficient yard crane scheduling can trim down turnaround times for external trucks, decrease energy usage, and shrink overall service duration [78]. Boysen et al. [7] introduces a framework encompassing all crane types, with terminal layout, container movement characteristics, and objectives as key components. Carlo et al. [9] provides a survey specifically focused on yard operations. These surveys encompass a range of crane numbers and configurations.

Initial research is related to non-automated Asian terminals featuring a truck lane along the block where trucks could be positioned adjacent to the bay for container storage or retrieval. Kim and Kim [45] was a pioneer study of the single-yard crane problem, focusing solely on retrieval moves to minimize total container handling time. The authors proposed an integer linear model and later explored genetic and beam search algorithms [47]. Narasimhan and Palekar [62] investigates a similar problem, suggesting exact and heuristic algorithms. Ng and Mak [63] expands the problem scope to include retrieval, storage moves, and ready times, offering first a branch and bound algorithm and later a heuristic approach to minimize job waiting times.

In recent years, attention has shifted to the challenges of automated European terminals, where containers in a yard block can only be accessed through I/O points at the seaside and landside ends of the block. Gharehgozli et al. [35] tackles retrieval moves from specified block positions and storage moves from I/O points to designated positions within the block, aiming to minimize total crane travel time. Gharehgozli et al. [36] extends this by considering open location selections for storage moves. The authors develop an exact three-phase algorithm for small instances and a heuristic for larger ones. Galle et al. [31] explores combined yard scheduling and container relocation for single crane scenarios, proposing an integer model and a local search heuristic adaptable to various yard configurations.

Another relevant aspect in yard crane scheduling problems is the objective definition. As noted in [44], yard crane operations are intertwined with seaside and landside processes, thus minimizing crane travel time may not always align with vessel and truck turnaround time minimization. Prior research predominantly focuses on minimizing total crane travel time, but some studies integrate release and due time considerations. Li et al. [53, 54] and Wu et al. [73] study non-automated terminals with multiple cranes per block, addressing crane interference and container distance requirements. Huang and Li [41] proposes a scheduling system to minimize the total weighted maximum tardiness of loading jobs. Zheng et al. [82] and Liu et al. [56] extend this with uncertain release times using stochastic programming formulations. Gao and Ge [32] jointly optimizes truck and yard crane scheduling to minimize total crane distance and truck waiting times. Chen et al. [14] tackles the problem of yard cranes and AGVs combined. Xing et al. [75] considers quay cranes, AGVs, and yard cranes together, aiming to minimize crane makespan and AGV consumption. Lastly, automated terminals with multiple cranes per block have also been studied, focusing on AGV delays and external truck waiting times [34].

Container Relocation Problems

During a certain planning horizon, H, containers arrive (incoming containers) and leave the yard (outgoing containers). The set of containers is denoted as C. In this regard, each container $c \in C$ has a known arrival time, denoted as $a(c)$, and a retrieval time, denoted as $r(c)$, where $r(c) > a(c)$. The objective of the yard cranes is to store and retrieve the containers according to their arrival and retrieval times. With this goal in mind, the feasible movements to be performed by a crane are defined on the basis of the intrinsic Last In First Out (LIFO) structure of the stacks. That is:

- *Storage movement*. The next incoming container is placed at the top of a stack with at least one empty slot.
- *Retrieval movement*. The next container to retrieve is taken out of its bay whenever it is placed at the top of a stack.
- *Relocation movement*. A container is moved from the top of a stack to the top of another one with at least one empty slot.

The incoming and outgoing containers in a bay give rise to the definition of the following closely related $\mathcal{NP}$-hard optimization problems:

- *Stacking Problem*. It aims to determine the shortest sequence of movements to be performed by the crane to store and retrieve the containers in/from the bay [25].
- *Container Relocation Problem*. It seeks to determine the shortest sequence of relocation movements to retrieve a subset of containers. It is assumed that all the containers are already stored in the bay, and no new incoming containers arrive [27].
- *Pre-Marshalling Problem*. Its objective is to find the shortest sequence of movements to arrange the containers within a given bay, such that any container is placed above another container with earlier retrieval time in the same stack. In this case, neither incoming nor outgoing containers are considered [26].

The main operational decision derived from the definitions of the previous optimization problems is how to evaluate the attractiveness of a certain stack when storing or relocating a container within the bay. Consider the example depicted in Fig. 6. The next incoming container arrives at period 17 and must be retrieved from the bay at period 98. However, its target stack must be selected adequately to minimize the number of future relocation movements. Similarly, target stacks for containers placed above the next to retrieve must be selected. For instance, see the container with retrieval time 96 placed on top of the container with retrieval time 22 in Fig. 6.

Algorithm 5 depicts a general heuristic framework to store and retrieve containers. It is composed of two main steps: retrieving and storing containers. With this goal in mind, at each time period, $t \in [1, 2, \ldots, H]$, the sets of containers to retrieve and store from/in the bay are identified. On the one hand, each container c_{out} to

Algorithm 5: General heuristic framework to store and retrieve containers

```
1: for t = 1 → H do
2:     R_C(t) ← Set of containers to retrieve at time t
3:     S_C(t) ← Set of containers to store at time t
4:     for c_out ∈ R_C(t) do
5:         O(c_out) ← Set of containers currently placed above c_out
6:         Relocate containers from O(c_out) in other stacks
7:         Retrieve c_out from the top of its stack
8:     end for
9:     for c_in ∈ S_C(t) do
10:        s ← Select target stack for c_in
11:        t ← Select target tier for c_in
12:        Relocate c_in in slot (s, t)
13:    end for
14: end for
```

retrieve at period t is checked (lines 4–8). This way, the set of containers currently placed above it, denoted as $O(c_{out})$ (line 5), must be relocated in alternative stacks due to the fact that they are avoiding the retrieval of c_{out} (line 6). How the target positions of these containers are determined constitutes the major difference among the proposals developed by different authors so far. Some examples are described in section "Container Target Positions." Lastly, c_{out} can be directly retrieved from the bay because it would be placed at the top of its stack (line 7). On the other hand, once all the containers to retrieve from the bay at time t have been already retrieved, incoming containers are stored. In this regard, a suitable target stack and tier are first determined (lines 10–11) for each container to store and then moved to them (line 12). It should be noted that in Algorithm 5, retrieval operations (lines 4–8) are performed before storage operations (lines 9–13) in the same time period; however, according to the preferences of the decision-maker, the priority of storage and retrieval operations can be easily exchanged.

Container Target Positions

The main decision to take in the previous heuristic framework is where the containers must be stored or relocated. This is the case for the incoming containers and those currently placed above the next to be retrieved from the bay. In all cases, the problem is, given a certain container, to determine its most suitable target slot (i.e., stack, and tier).

Determining the target slot of a container can be split into two parts: identifying the target stack and establishing a tier in it. Most authors have proposed attractiveness measures to select a target stack, which indicates the suitability of placing a container in a stack. This way, the attractiveness of stack s to place container c is given by a scoring function with the shape $f(c, s)$. In all cases, we define the set of feasible stacks of c, denoted as $S(c)$. It comprises those stacks in which c is not currently placed and with at least one empty slot. Thus, it is assumed that $s \in S(c)$. Some examples of scoring functions are shown in the following:

1. *Random*. The target stack is randomly selected among those feasible stacks for c. That is:

$$f(c, s) = \frac{1}{|S(c)|} \tag{3}$$

2. *Conflict Minimization* (Rei and Pedroso [65]). It seeks to place c in a stack in which it is not going to require future relocations. The set of stacks that allows this fact is denoted as $S(c)'$ and, therefore, the scoring function is defined as follows:

$$f(c, s) = \begin{cases} \frac{1}{|S(c)'|} & \text{if } S(c)' \neq \emptyset \\ \frac{1}{|S(c)|} & \text{otherwise} \end{cases} \tag{4}$$

3. *Flexibility Optimization* (Rei and Pedroso [65]). It aims to place containers with close retrieval times together, whereas in those scenarios in which future relocation movements are unavoidable, these are delayed as much as possible. This is expressed as follows:

$$f(c, s) = \begin{cases} K & \text{if } s \text{ is empty} \\ M_s & \text{if } M_s \geq r(c) \\ 2 \cdot K - M_s & \text{if } M_s < r(c) \end{cases}, \tag{5}$$

   where K is a constant representing a value larger than the latest retrieval time of a container in the bay, and M_s represents the earliest retrieval time of a container in the stack s. These are formally defined as follows: $K = 1 + \max\{r(c')|c' \in C\}$ and $M_s = \min\{r(c')|c' \in C \wedge c' \text{ in } s\}$.
4. *Parameterized Flexibility Optimization* (Rei and Pedroso [65]). It is an extension of the FO in which the best stack is selected with probability $1 - \rho$ (where ρ is a parameter whose value is set by the user), whereas one of the remaining stacks is selected with probability ρ.

In the previous functions, the best stack is selected as the target of c. Moreover, other authors have proposed explicit expressions to select the target stack of container c, denoted as s^*. This is the case, for instance, of Caserta et al. [11], who propose to select the target stack as follows:

$$s^* = \begin{cases} \arg\min_{s \in S(c)}\{\min(s) | \min(s) > r(c)\} & \text{if } \exists s : \min(s) > r(c) \\ \arg\max_{s \in S(c)}\{\min(s)\} & \text{otherwise} \end{cases}, \tag{6}$$

where

$$\min(s) = \begin{cases} 1 + \max\{r(c')|c' \in C\} & \text{if } s \text{ is empty} \\ \min\{r(c')|c' \in C \wedge c' \text{ in } s\} & \text{otherwise} \end{cases}.$$

In this case, Eq. (6) seeks first to place container c in one of the stacks in which it is not going to require additional relocations. Particularly, that stack storing a container with the earliest retrieval time is chosen. However, when this is not possible, c is placed in that stack in which the next retrieval movement must be performed as later as possible.

Computational Analysis

The previous heuristics' performances are assessed in the following. With this goal in mind, a benchmark suite has been considered. It is composed of a wide range of problem instances with a number of containers ranging from 50 to 300. In each case, the containers must be stored in and retrieved from a bay composed of $S = 10$ stacks and without a maximum stacking limit.

Table 7 reports the computational results obtained by the heuristics under analysis over the problem instances from the aforementioned benchmark suite. In this regard, all the heuristics have been implemented in Java and executed on a computer equipped with an Intel 3.16 GHz and 4 GB of RAM. Ten problem instances with a certain number of containers have been used in each case. The column *Containers* shows the number of containers. Furthermore, columns *Random*, *CM*, *FO*, *PFO* ($\rho = 0.001$), and *CSV* present the average number of relocation movements of the solutions reported by the heuristics. The computational times required by the heuristics are not included in the comparison due to the fact that all of them are, even in large-scale scenarios, too negligible to draw any relevant conclusions.

As can be checked in the computational results, FO reports solutions with the lowest number of relocation movements of those heuristics in the comparison. The reason is that this heuristic allows the minimization of the differences in retrieval times of consecutive containers in the same stack of the bay. It should be noted that placing a container at the top of a stack by minimizing the differences in retrieval times reduces the probability of requiring additional relocation movements in the future. Analogously, CSV exploits this idea, and consequently, it also reports high-quality solutions.

Trends in Container Relocation Problems

In recent times, incorporating predefined knowledge has led to the development of heuristic methods designed to enhance computational speed, as exemplified

Table 7 Performance of the heuristics on a wide range of practical-size scenarios

Instance	Random	CM	FO	PFO ($\rho = 0.1$)	CSV
50	73.6	32.9	3.3	5.3	3.5
100	308.4	263.3	29.4	40.5	31.1
150	714.3	680.9	114.0	134.6	114.2
200	1270.2	1329.3	246.3	289.9	247.0
250	1949.4	2222.3	438.6	496.3	443.5
300	2879.4	3501.8	624.1	720.3	635.1

[3]. Despite these advancements, both exact and heuristic algorithms continue to require considerable time to solve numerous variants of container relocation problems. Machine learning algorithms are increasingly being utilized to address container relocation problems. However, research on applying this approach to problems regarding the placement of containers is notably sparse in scientific literature. A novel Deep Learning Heuristic Tree Search (DLTS) approach is proposed to tackle the container pre-marshaling problem in [38]. This study utilizes deep neural networks to extract strategies from existing near-optimal solutions to relocation scenarios. Other work in the same direction is [81]. Furthermore, as suggested in [28], reinforcement learning offers viable alternatives for addressing container relocation challenges. Wei et al. [72] and Jiang et al. [43] are illustrative examples.

Conclusion

Maritime container terminals are large infrastructures in which heterogeneous logistic processes are combined. This fact and the increasing volume of containers moved worldwide make their management extremely complex.

Up to now, exact approaches have not been demonstrated to be efficient due to the $\mathcal{NP}$-hard nature of most of the optimization problems found at maritime container terminals. In this regard, heuristics have stood as the most competitive alternatives because they report (near) optimal solutions within short computational times. This chapter presents and discusses several heuristics to solve several relevant optimization problems. The analysis of the performance of the heuristics indicates that some of them are appropriate when addressing realistic-size scenarios.

The synergies between machine learning and metaheuristics constitute a promising future research line. Machine learning techniques are useful for estimating the value of the variables used to model the problems of a maritime container terminal. Furthermore, they can be used and integrated as a component of metaheuristic frameworks to solve port logistics problems efficiently.

Other progress in heuristics also deserves attention. Namely, hybrid heuristics, which combine different heuristics approaches, have shown the potential to improve solution quality within reasonable computational times. Also, investigation and application of adaptive heuristics that adjust their search strategy based on the search's history or the dynamism present in the maritime terminal environment arise as a relevant future direction.

Acknowledgments This work has been partially funded by the Grants for Research Projects 2023, CajaCanarias Foundation and "La Caixa" Foundation, (project 2023DIG35), and by the Spanish Ministry of Economy and Competitiveness with FEDER funds (project code PID2019-104410RB-I00/AEI/10.13039/501100011033).

References

1. Angeloudis P, Bell MGH (2010) An uncertainty-aware AGV assignment algorithm for automated container terminals. Transp Res Part E: Logist Transp Rev 46(3):354–366
2. Ankita PU, Mathirajan M (2021) An efficient heuristic method for dynamic berth allocation problem. In: Proceedings of the international conference on industrial engineering and operations management. IEOM Society, pp 393–400
3. Bacci T, Mattia S, Ventura P (2023) The realization-independent reallocation heuristic for the stochastic container relocation problem. Soft Comput 27(7):4223–4233
4. Bierwirth C, Meisel F (2009) A fast heuristic for quay crane scheduling with interference constraints. J Scheduling 12(4):345–360
5. Bierwirth C, Meisel F (2010) A survey of berth allocation and quay crane scheduling problems in container terminals. Eur J Oper Res 202(3):615–627
6. Bish EK, Chen FY, Leong YT, Nelson BL, Ng JWC, Simchi-Levi D (2005) Dispatching vehicles in a mega container terminal. OR Spectrum 27(4):491–506
7. Boysen N, Briskorn D, Meisel F (2017) A generalized classification scheme for crane scheduling with interference. Eur J Oper Res 258(1):343–357
8. Briskorn D, Drexl A, Hartmann S (2006) Inventory-based dispatching of automated guided vehicles on container terminals. OR Spectrum 28(4):611–630
9. Carlo HJ, Vis IFA, Roodbergen KJ (2014) Storage yard operations in container terminals: literature overview, trends, and research directions. Eur J Oper Res 235(2):412–430
10. Caserta M, Schwarze S, VoßS (2011) Container rehandling at maritime container terminals. In: Böse JW (ed) Handbook of terminal planning Operations research/computer science interfaces series, vol 49. Springer, New York, pp 247–269
11. Caserta M, Schwarze S, VoßS (2012) A mathematical formulation and complexity considerations for the blocks relocation problem. Eur J Oper Res 219(1):96–104
12. Chen G, Govindan K, Yang Z (2013) Managing truck arrivals with time windows to alleviate gate congestion at container terminals. Int J Prod Econ 141(1):179–188
13. Chen X, Bai R, Qu R, Dong H, Chen J (2020) A data-driven genetic programming heuristic for real-world dynamic seaport container terminal truck dispatching. IEEE, pp 1–8
14. Chen X, He S, Zhang Y, Tong LC, Shang P, Zhou X (2020) Yard crane and AGV scheduling in automated container terminal: a multi-robot task allocation framework. Transp Res Part C Emerg Technol 114:241–271
15. Cordeau JF, Gaudioso M, Laporte G, Moccia L (2007) The service allocation problem at the gioia tauro maritime terminal. Eur J Oper Res 176(2):1167–1184
16. Cordeau JF, Laporte G, Legato P, Moccia L (2005) Models and tabu search heuristics for the berth-allocation problem. Transp Sci 39(4):526–538
17. Correcher JF, Van den Bossche T, Alvarez-Valdes R, Berghe GV (2019) The berth allocation problem in terminals with irregular layouts. Eur J Oper Res 272(3):1096–1108
18. Corry P, Bierwirth C (2019) The berth allocation problem with channel restrictions. Transp Sci 53(3):708–727
19. da Silva MRF, Agostino IRS, Frazzon EM (2023) Integration of machine learning and simulation for dynamic rescheduling in truck appointment systems. Simul Model Pract Theory 125:102747
20. de León AD, Lalla-Ruiz E, Melián-Batista B, Moreno-Vega JM (2017) A machine learning-based system for berth scheduling at bulk terminals. Expert Syst Appl 87:170–182
21. Dkhil H, Diarrassouba I, Benmansour S, Yassine A (2021) Modelling and solving a berth allocation problem in an automotive transshipment terminal. J Oper Res Soc 72(3):580–593
22. Duan Y, Ren H, Xu F, Yang X, Meng Y (2023) Bi-objective integrated scheduling of quay cranes and automated guided vehicles. J Marine Sci Eng 11:8
23. Emde S, Boysen N, Briskorn D (2014) The berth allocation problem with mobile quay walls: problem definition, solution procedures, and extensions. J Schedul 17:289–303

24. Expósito-Izquierdo C, González-Velarde JL, Melián-Batista B, Moreno-Vega JM (2013) Hybrid estimation of distribution algorithm for the quay crane scheduling problem. Appl Soft Comput 13(10):4063–4076
25. Expósito-Izquierdo C, Lalla-Ruiz E, de Armas J, Melián-Batista B, Moreno-Vega JM (2015) A heuristic algorithm based on an improvement strategy to exploit idle time periods for the stacking problem. Comput Ind Eng 87:410–424
26. Expósito-Izquierdo C, Melián-Batista B, Moreno-Vega JM (2012) Pre-marshalling problem: Heuristic solution method and instances generator. Expert Syst Appl 39(9):8337–8349
27. Expósito-Izquierdo C, Melián-Batista B, Moreno-Vega JM (2015) An exact approach for the blocks relocation problem. Expert Syst Appl 42(17–18):6408–6422
28. Fechter J, Beham A, Wagner S, Affenzeller M (2019) Approximate q-learning for stacking problems with continuous production and retrieval. Appl Artif Intell 33(1):68–86
29. Filom S, Amiri AM, Razavi S (2022) Applications of machine learning methods in port operations – a systematic literature review. Transp Res Part E: Logist Transp Rev 161:102722
30. Frojan P, Correcher JF, Alvarez-Valdes R, Koulouris G, Tamarit JM (2015) The continuous berth allocation problem in a container terminal with multiple quays. Expert Syst Appl 42(21):7356–7366
31. Galle V, Barnhart C, Jaillet P (2018) Yard crane scheduling for container storage, retrieval, and relocation. Eur J Oper Res 271(1):288–316
32. Gao Y, Ge Y-E (2023) Integrated scheduling of yard cranes, external trucks, and internal trucks in maritime container terminal operation. Maritime Policy Manag 50(5):629–650
33. Gharehgozli AH, Yu Y, de Koster R, Udding JT (2014) An exact method for scheduling a yard crane. Eur J Oper Res 235(2):431–447. Maritime Logistics
34. Gharehgozli AH, Laporte G, Yu Y, De Koster R (2015) Scheduling twin yard cranes in a container block. Transp Sci 49(3):686–705
35. Gharehgozli AH, Yu Y, de Koster R, Udding JT (2014) An exact method for scheduling a yard crane. Eur J Oper Res 235(2):431–447
36. Gharehgozli AH, Yu Y, Zhang X, de Koster R (2017) Polynomial time algorithms to minimize total travel time in a two-depot automated storage/retrieval system. Transp Sci 51(1):19–33
37. Henesey L (2006) Overview of transshipment operations and simulation. In: MedTrade conference, Malta, pp 6–7
38. Hottung A, Tanaka S, Tierney K (2020) Deep learning assisted heuristic tree search for the container pre-marshalling problem. Comput Oper Res 113:104781
39. Hu X, Guo J, Zhang Y (2019) Optimal strategies for the yard truck scheduling in container terminal with the consideration of container clusters. Comput Ind Eng 137:11
40. Hu Y, Dong L, Xu L (2020) Multi-AGV dispatching and routing problem based on a three-stage decomposition method. Math Biosci Eng 17:5150–5172
41. Huang SY, Li Y (2017) Yard crane scheduling to minimize total weighted vessel loading time in container terminals. Flexible Serv Manuf J 29:689–720
42. Imai A, Nishimura E, Papadimitriou S (2001) The dynamic berth allocation problem for a container port. Transp Res Part B Methodol 35(4):401–407
43. Jiang T, Zeng B, Wang Y, Yan W (2021) A new heuristic reinforcement learning for container relocation problem. In: Journal of physics: conference series, vol 1873. IOP Publishing, p 012050
44. Kemme N (2020) State-of-the-art yard crane scheduling and stacking. In: Handbook of terminal planning, pp 383–413
45. Kim KH, Kim KY (1999) An optimal routing algorithm for a transfer crane in port container terminals. Transp Sci 33(1):17–33
46. Kim KH, Park YM (2004) A crane scheduling method for port container terminals. Eur J Oper Res 156(3):752–768
47. Kim KY, Kim KH (2003) Heuristic algorithms for routing yard-side equipment for minimizing loading times in container terminals. Naval Res Logist (NRL) 50(5):498–514
48. Kolley L, Rückert N, Kastner M, Jahn C, Fischer K (2023) Robust berth scheduling using machine learning for vessel arrival time prediction. Flexible Serv Manuf J 35(1):29–69

49. Korsvik JE, Fagerholt K (2010) A tabu search heuristic for ship routing and scheduling with flexible cargo quantities. J Heurist 16(2):117–137
50. Kramer A, Lalla-Ruiz E, Iori M, Voß S (2019) Novel formulations and modeling enhancements for the dynamic berth allocation problem. Eur J Oper Res 278(1):170–185
51. Lalla-Ruiz E, Melián-Batista B, Moreno-Vega JM (2012) Artificial intelligence hybrid heuristic based on tabu search for the dynamic berth allocation problem. Eng Appl Artif Intell 25(6):1132–1141
52. Lalla-Ruiz E, Expósito-Izquierdo C, Melián-Batista B, Moreno-Vega JM (2016) A set-partitioning-based model for the berth allocation problem under time-dependent limitations. Eur J Oper Res 250(3):1001–1012
53. Li W, Wu Y, Goh M, Li W, Wu Y, Goh M (2015) A continuous-time model for multiple yard crane scheduling with last-minute job arrivals. Springer International Publishing, pp. 53–83
54. Li W, Wu Y, Petering MEH, Goh M, De Souza R (2009) Discrete time model and algorithms for container yard crane scheduling. Eur J Oper Res 198(1):165–172
55. Liang C, Zhang Y, Dong L (2023) A three stage optimal scheduling algorithm for AGV route planning considering collision avoidance under speed control strategy. Mathematics 11:1
56. Liu W, Zhu X, Wang L, Yan B, Zhang X (2021) Optimization approach for yard crane scheduling problem with uncertain parameters in container terminals. J Adv Transp 2021: 1–15
57. Luo J, Wu Y (2020) Scheduling of container-handling equipment during the loading process at an automated container terminal. Comput Ind Eng 149:11
58. Ma H-L, Chung SH, Chan HK, Cui L (2019) An integrated model for berth and yard planning in container terminals with multi-continuous berth layout. Ann Oper Res 273:409–431
59. Mansoursamaei M, Moradi M, González-Ramírez RG, Lalla-Ruiz E (2023) Machine learning for promoting environmental sustainability in ports. J Adv Transp 2023(1):1–17, 2144733
60. Meisel F, Bierwirth C (2013) A framework for integrated berth allocation and crane operations planning in seaport container terminals. Transp Sci 47(2):131–147
61. Monaco MF, Sammarra M, Sorrentino G (2014) The terminal-oriented ship stowage planning problem. Eur J Oper Res 239(1):256–265
62. Narasimhan A, Palekar US (2002) Analysis and algorithms for the transtainer routing problem in container port operations. Transp Sci 36(1):63–78
63. Ng WC, Mak KL (2005) Yard crane scheduling in port container terminals. Appl Math Model 29(3):263–276
64. United Nations Conference on Trade and Development (2023) Review of maritime transport
65. Rei RJ, Pedroso JP (2012) Heuristic search for the stacking problem. Int Trans Oper Res 19(3):379–395
66. Shang J (2010) A heuristic algorithm for the integrated yard truck scheduling in container terminal with twin 40-foot quay crane. In: 2010 international conference on computer, mechatronics, control and electronic engineering, vol 2, pp 386–389
67. Shouwen J, Di L, Zhengrong C, Dong G (2021) Integrated scheduling in automated container terminals considering AGV conflict-free routing. Transp Lett 13:501–513
68. Umang N, Bierlaire M, Vacca I (2013) Exact and heuristic methods to solve the berth allocation problem in bulk ports. Transp Res Part E Logist Transp Rev 54:14–31
69. Wang C, Jin C, Li Z (2019) Bilevel programming model of low energy consumption AGV scheduling problem at automated container terminal. IEEE, pp 195–199
70. Wang Z, Zeng Q (2022) A branch-and-bound approach for AGV dispatching and routing problems in automated container terminals. Comput Ind Eng 166:4
71. Wawrzyniak J, Drozdowski M, Sanlaville É (2020) Selecting algorithms for large berth allocation problems. Eur J Oper Res 283(3):844–862
72. Wei L, Wei F, Schmitz S, Kunal K (2021) Optimization of container relocation problem via reinforcement learning. Logist J Proc 2021(17):1–8
73. Wu Y, Li W, Petering MEH, Goh M, de Souza R (2015) Scheduling multiple yard cranes with crane interference and safety distance requirement. Transp Sci 49(4):990–1005

74. Xing Z, Liu H, Wang T, Chew EP, Lee LH, Tan KC (2023) Integrated automated guided vehicle dispatching and equipment scheduling with speed optimization. Transp Res Part E Logist Transp Rev 169:102993
75. Xing Z, Liu H, Wang T, Chew EP, Lee LH, Tan KC (2023) Integrated automated guided vehicle dispatching and equipment scheduling with speed optimization. Transp Res Part E Logist Transp Rev 169:102993
76. Xu B, Jie D, Li J, Yang Y, Wen F, Song H (2021) Integrated scheduling optimization of u-shaped automated container terminal under loading and unloading mode. Comput Ind Eng 162:12
77. Xu D, Li C-L, Leung JY-T (2012) Berth allocation with time-dependent physical limitations on vessels. Eur J Oper Res 216(1):47–56
78. Yan W, Huang Y, Chang D, He J (2011) An investigation into knowledge-based yard crane scheduling for container terminals. Adv Eng Inform 25(3):462–471
79. Yang Y, Zhong M, Dessouky Y, Postolache O (2018) An integrated scheduling method for AGV routing in automated container terminals. Comput Ind Eng 126:482–493
80. Yue L, Fan H (2022) Dynamic scheduling and path planning of automated guided vehicles in automatic container terminal. IEEE/CAA J Autom Sin 9:2005–2019
81. Zhang C, Guan H, Yuan Y, Chen W, Wu T (2020) Machine learning-driven algorithms for the container relocation problem. Transp Res Part B Methodol 139:102–131
82. Zheng F, Man X, Chu F, Liu M, Chu C (2019) A two-stage stochastic programming for single yard crane scheduling with uncertain release times of retrieval tasks. Int J Prod Res 57(13):4132–4147
83. Zhong M, Yang Y, Dessouky Y, Postolache O (2020) Multi-AGV scheduling for conflict-free path planning in automated container terminals. Comput Ind Eng 142:4

Metaheuristics for Medical Image Registration

46

Andrea Valsecchi, Enrique Bermejo, Sergio Damas, and Oscar Cordón

Contents

Introduction 1446
Image Registration 1447
Problem Statement 1447
Classic IR Algorithms 1451
Metaheuristics-Based Image Registration 1452
Suitability of MHs in Image Registration 1452
Early Evolutionary Image Registration Methods 1453
Outstanding IR Methods Based on MHs 1454
Feature-Based Techniques 1455
Intensity-Based Techniques 1456
Experimental Study 1458
Feature-Based Comparison: Registration of Brain Magnetic Resonance Images 1458
Second Experiment: Atlas-Based Segmentation of Brain Deep Nuclei 1460
Conclusion 1464
Cross-References 1464
References 1465

Abstract

In the last few decades, image registration (IR) has been a very active research area in computer vision. Applications of IR cover a broad range of real-world

A. Valsecchi (✉) · E. Bermejo · O. Cordón
Department of Computer Science and Artificial Intelligence, Unviersity of Granada, Granada, Spain
e-mail: andreavalsecchi@ugr.es; andrea.valsecchi@softcomputing.es; enric2186@ugr.es; ocordon@decsai.ugr.es

S. Damas
Department Software Engineering, University of Granada, Granada, Spain
e-mail: sdamas@ugr.es; sergio.damas@softcomputing.es

© Springer Nature Switzerland AG 2025

R. Martí et al. (eds.), *Handbook of Heuristics*,
https://doi.org/10.1007/978-3-032-00385-0_56

problems, including remote sensing, medical imaging, artificial vision, and computer-aided design. In particular, medical IR is a mature research field with theoretical support and two decades of practical experience. Formulated as either a continuous or combinatorial optimization problem, medical IR has been traditionally tackled by iterative numerical optimization methods, which are likely to get stuck in local optima and deliver suboptimal solutions. Recently, a large number of medical IR methods based on different metaheuristics, mostly belonging to evolutionary computation, have been proposed. In this chapter, we review the most recognized of these algorithms and develop an experimental comparison over real-world IR scenarios.

Keywords

Medical imaging · Image registration · Image segmentation

Introduction

In its most general formulation, image registration [60] is the task of aligning two or more images in order to establish a spatial correspondence of their common content. These images usually have the same or a similar subject but have been acquired under different conditions, such as time and viewpoint, or by multiple sensors. In medical image analysis, IR is a key technology that allows to "fuse" visual information from different sources [56]. Applications include combining images of the same subject from different modalities, detecting changes before/after treatment, aligning temporal sequences of images to compensate for motion between scans, image guidance during interventions, and aligning images from multiple subjects in cohort studies. The remarkable developments in medical imaging technology over the last few decades determine a constant demand for better image processing and analysis techniques. Dealing with novel, more diverse, and increasingly accurate sources of imaging data is the main challenge in IR, and it explains why it is still a very active research field.

The alignment between two images is specified as a spatial transformation, mapping the content of one image to the corresponding area of the other through a composition of translation, rotation, shear, and other kind of operations. A popular strategy among IR methods is to perform the alignment based on only salient and distinctive parts of the image, such as lines, corners, and contours, called *features*, ignoring the rest of the image content. This approach, called *feature based* [56], has the advantage of greatly reducing the complexity of the problem but relies on the ability to precisely detect the features, either manually or automatically. Any error during the feature extraction stage will propagate into the registration and can hardly be recovered at a later stage. Moreover, this approach is limited to the cases in which features provide enough information to characterize the image content. To avoid these drawbacks, it is possible to use the image intensities directly without any feature extraction, an approach called *intensity based* (or voxel-based)

[60]. Though more expensive in computational terms, intensity-based methods can achieve a superior level of accuracy and robustness.

Regardless of the division, the core of every IR technique is an *optimization process* that explores the space of geometrical transformations. Two strategies are available. In *parameters*-based approaches, the search is directly performed in the space of the registration transformation parameters, turning the registration into a continuous optimization problem. In *matching*-based approaches, instead, features or regions of the image are matched through a search in the space of possible correspondences. In this case, IR is formulated as a combinatorial optimization problem. Once a suitable matching has been found, the transformation parameters are derived accordingly through numerical methods. In both cases the search is guided by a *similarity metric*, a function that measures the degree of resemblance between the input images after the alignment. This can be done either by comparing the whole images or just their corresponding features. Traditional parameters-based methods use classic numerical optimization algorithms [32], while matching-based methods use matching algorithms like iterative closest point (ICP) [5].

Many characteristics of the IR problem, such as noise, discretization, and large differences in the order of magnitude of the transformation parameters, still pose a challenge to traditional optimization methods. A number of alternative approaches based on *metaheuristics* (MHs) [19] are often used to deal with complex real-world problems in computer vision and image processing. In the literature, despite their common structure, feature- and intensity-based methods are usually regarded as different categories of algorithms, therefore most publications deal with either one group or the other. Besides, feature-based approaches based on MHs are much more common than intensity-based ones, as the larger computational cost makes them less suitable for MHs. Nevertheless, both groups have been recently reviewed [13, 52], although separately.

This chapter is structured as follows. Section "Image Registration" provides a general introduction to medical IR and the traditional optimization algorithms used to solve it. Section "Metaheuristics-Based Image Registration" deals with the application of MHs to IR, while section "Outstanding IR Methods Based on MHs" reviews the most important medical IR methods of this kind. Section "Experimental Study" develops two experimental studies, comparing traditional and MH-based methods of the two different families, feature- and intensity-based, over real-world medical applications. Finally, conclusions are provided in section "Conclusion".

Image Registration

Problem Statement

A typical IR problem involves two images, conventionally called *model* (I_M) and *scene* (I_S), with different roles in the registration process. The model is the reference (or target) image, while the scene is the image that is transformed to reach the geometry of the other. Figure 1 illustrates such a process. The registration aims to

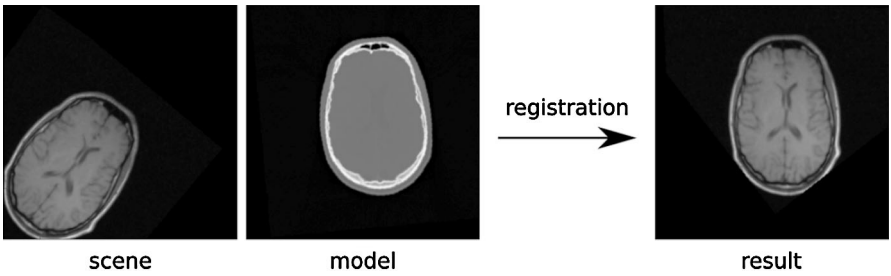


Fig. 1 An example of image registration. A MRI brain scan (*left image*) is aligned to the pose of the CT scan (*middle image*), resulting in the right image

find a geometric transformation T that aligns the scene to the model. In other words, T is the transformation that makes the model I_M and the transformed scene $T(I_S)$ as similar as possible according to the similarity metric of choice. In other terms, IR is a maximization problem over transformations, formally stated as

$$\underset{T \in \text{Transformations}}{arg\ \max} \quad \text{Similarity}\,(I_M, T\,(I_S))$$

A number of components characterize an IR method, but the main ones are just three: the kind of transformation used to relate the images, the similarity metric that measures the quality of the alignment, and the optimization procedure that performs the search for a suitable transformation. The optimization is usually an iterative process. The optimizer computes a candidate transformation, which is then applied to the scene image. Next, the similarity metric compares the model with the transformed scene image and returns a quality value that is sent back to the optimizer. Figure 2 shows a flow chart of the process. The loop ends when a suitable transformation has been found or the algorithm has performed a certain number of iterations.

Transformation Model

The transformation model determines which kind of geometrical transformation can be applied to the scene image to reach the model. This also controls which geometrical properties (e.g., size, shape, position, orientation, etc.) are preserved through the process. Figure 3 shows the effect of three common transformation models.

A rigid transformation involves arbitrary translation and rotation operations, thus it has six degrees of freedom for 3D images. A similarity transform also admits a scaling operation, specified by a seventh parameter. In more complex scenarios, it may also be necessary to correct for shears, for example, those caused by the gantry tilt of CT scanners, or for scaling on a per axis basis. Thus, an affine transformation (nine parameters) is required.

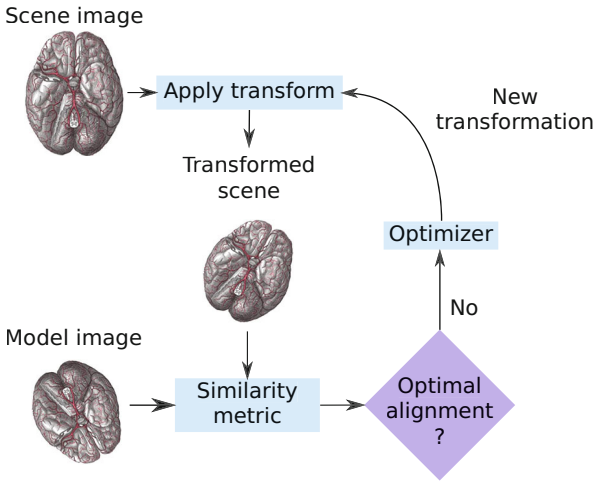


Fig. 2 The interactions among the components of a registration technique

Rigid, similarity, and affine transformations are frequently used for the registration of anatomical structures like the brain or bones; they are not applicable in the case where a significant local deformation is expected, e.g., in soft tissues like the breast. In these cases, deformable or non rigid transforms are required. In a typical approach, an object is deformed by manipulating an underlying mesh of control points. The resulting deformation controls the shape of the 3D object and produces a smooth and continuous transformation between control points.

The choice of the transformation model depends entirely on the needs of the application at hand. A too flexible transformation model is not just more complex and computationally expensive to apply but can also lead to results that are undesired or implausible with respect to the phenomenon under study. Bones being bent and tissues growing at an unrealistic rate are examples of such unacceptable outcomes in the medical context.

Similarity Metric

The performance of any IR method depends on an accurate estimation of the degree of alignment between the images, therefore the similarity metric is considered a crucial component [48]. Technically, a similarity metric is a real-valued function $F(I_A, I_B)$ that measures the resemblance of two images I_A, I_b. The quality of a transformation T is assessed by computing the similarity metric over the model I_M and the transformed scene $T(I_S)$, i.e. $F(I_M, T(I_S))$. The actual evaluation mechanism depends on the nature of the registration approach. In feature-based methods, the similarity metric usually measures the distance between corresponding features [1]. For instance, if the features are points, the alignment can be evaluated using the mean square error (MSE) between the position of a point in the model and that of the corresponding (or closest) point in the transformed scene, i.e.,

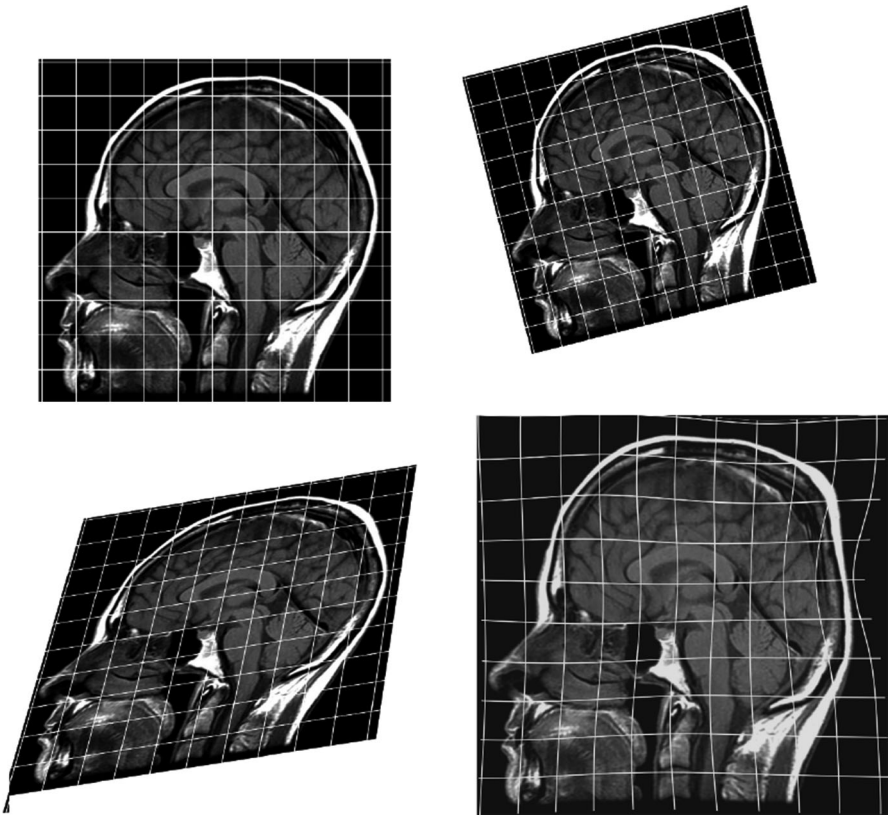


Fig. 3 Images obtained from the same scene (*top left*) by applying different transformations: similarity (*top right*), affine (*bottom left*) and B-spline (*bottom right*)

$$\text{MSE} \;=\; \frac{1}{r}\sum_{i=1}^{r} \|x_i - c\,(T\,(x_i))\|^2$$

where r is the number of points and c is the correspondence criterion.

In intensity-based approaches, similarity metrics are usually based on the resemblance of the intensity values in the two images. The subject of the images along with their modality determine what kind of relationship is established between their intensity distributions. For instance, if one assumes this relationship to be linear, the similarity between the images can be assessed by computing the linear correlation coefficient. When two images have been acquired using different sensors, a scenario called *multi modal* registration, the relationship between their intensity values can be strongly nonlinear. Metrics based on information theory, such as mutual information (MI) [39], are better suited for this scenario. MI is defined as

$$\mathrm{MI}\left(I_A, I_B\right) = \sum_{a \in I_A} \sum_{b \in I_B} p_{AB}\left(a, b\right) \log \frac{p_{AB}\left(a, b\right)}{p_A(a)\ \ p_B(b)}$$

where p_{AB} and p_A, p_B are, respectively, the joint and marginal probability distributions of the intensity values of the images.

Optimization Procedure

The third main component of an IR method is the *optimizer*. It is responsible for finding the best transformation, in terms of similarity metric, among the candidate transformations in the transformation model. Each optimizer has a different search strategy, which also depends on the nature of the algorithm. One approach is to perform the search directly in the space of the transformation parameters. This turns the registration into a continuous optimization problem; therefore classic numerical optimization algorithms can be used. Gradient descent, conjugated gradient descent, Newton's and quasi-Newton methods, Powell's method, and discrete optimization [25, 32] are among the most common choices along with approaches based on evolutionary computation and other metaheuristics [8, 43, 57]. IR algorithms that follow this approach are called *parameters based*. An alternative approach consists in searching for a matching between features, in feature-based methods, and areas of the image, in intensity-based ones. From the match, one can derive the parameters of the corresponding transformation using least-squares estimation or other more robust model fitting techniques [58]. This class of algorithms is called *matching based*. The ICP algorithm is the most representative example of this second approach [29, 44, 59].

Classic IR Algorithms

Iterative Closest Point

ICP [5] is a well-known feature-based algorithm in computer-aided design, originally proposed to recover the 3D transformation of pairs of range images. We are given a point set P of N_p points $\mathbf{p_i}$ from the data shape (the scene) and the model image X of N_x. In the original contribution, the approach dealt with 3D rigid transformations stored in a solution vector $\mathbf{q} = [q_1, q_2, q_3, q_4, t_1, t_2, t_3]$. The first four parameters correspond to a quaternion, determining the 3D rotation component, while the remaining three parameters store the translation vector. The procedure is initialized by setting $P_0 = P$, the initial registration transformation to $\mathbf{q_0} = [1, 0, 0, 0, 0, 0, 0]$ and the iteration counter k to zero. The next four steps are applied until reaching convergence within a tolerance threshold $\tau > 0$:

1. Compute the matching between the scene and the model points using the closest assignment rule. $Y_k = C(P_k, X)$.
2. Estimate the registration by least squares. $f_k = \rho(P_o, Y_k)$.
3. Apply the registration transformation to the scene image: $P_{k+1} = f_k(P_0)$.

4. Terminate the procedure if the change in MSE falls below τ. Otherwise increase k and go to step 1.

Note that ICP is not directly guided by a similarity metric but rather by the computed matching, as other matching-based IR methods. In this strategy, the function MSE only plays the role of the stopping criterion. Moreover, the transformation estimator is a numerical method that depends on the good outcomes of the matching step. Thus, the better the choice of the correspondences that is performed, the more precise the estimation of the transformation f. Consequently, the value of the similarity metric will be more accurate, leading to a proper convergence.

Gradient Descent

In intensity-based IR, gradient-based continuous optimization algorithms are the reference methods [25]. These algorithms consist of an iterative optimization process $\mu_{k+1} = \mu_k + a_k d_k$ where d_k is the search direction at iteration k, and a_k is a gain factor that controls the step size along the search direction. The search directions and the gain factors are chosen such that the sequence μ_k converges to a local minimum of the similarity metric.

The difference between gradient-based optimizers lies in the way the search direction and the gain factor are computed. The gradient descent method (GD) takes steps in the direction of the negative gradient of the cost function, i.e., $\mu_{k+1} = \mu_k - a_k g(\mu_k)$ where g is the derivative of the cost function, and the gain factor is the decaying function $a_k = a/(k + A)^\alpha$ with $a > 0$, $A \geq 1$ and $0 \leq \alpha \leq 1$. The quasi-Newton method (QN) also moves along the negative gradient direction. The gain factor is an approximation of the inverse Hessian matrix $[H(\mu_k)]^{-1}$, computed using the Broyden–Fletcher–Goldfarb–Shanno method. The adaptive stochastic gradient descent (ASGD) [26] is based on the Robbinsâ "Monro stochastic optimization procedure," which is designed to deal with noisy observation of the objective function. ASGD considers the solutions $\mu_{k+1} = \mu_k - \gamma_k(t_k) g_k$ where $t_{k+1} = \max(0, t_k + \text{sigm}(-g_k \cdot g_{k-1}))$ and $\gamma_k = \frac{a}{t_k + A}$. The "time" t_k is adapted depending on the inner product between the current and the previous gradients. If the gradients have the same direction, the time is reduced, leading to a larger step size.

Metaheuristics-Based Image Registration

Suitability of MHs in Image Registration

There are different strengths and limitations that have been stated either to justify or to avoid the use of these methods when tackling IR. Some of the advantages are:

- MHs do not depend on the starting solution, thus being more robust approaches, and provide specific strategies to escape from local optima.

- MHs have been used in a wide variety of optimization tasks within IR including numerical optimization and combinatorial optimization problems, i.e., facing both the transformation parameters and the matching-based IR approaches.
- MHs can easily handle arbitrary kinds of constraints and objective functions.
- MHs offer a framework in which including prior knowledge about the problem is easy. Thus, the search process is more appropriate, yielding a more efficient exploration of the search space. For instance, some feature-based IR approaches [7, 9, 12] improved the design of the objective function to exploit information related to the geometry of the images.
- MHs can also be easily combined with more traditional optimization techniques such as gradient-based methods [23, 45]. An outstanding approach to exploit the benefits of both strategies is their hybridization in the memetic computation paradigm [36, 37].

The most important shortcomings related to the use of MHs are:

- They require a tuning of the control parameters, which is often a manual, error-prone, expert-based procedure. This issue has been addressed using approaches based on automatic parameter tuning [51] or MHs with an adaptive behavior [36].
- Most MHs are time consuming; therefore they are usually avoided in real-time applications. Parallel and GPU implementations are increasingly more common [18, 42].
- Some MHs lack a formal proof of convergence to the global optimum, and there is hardly any theoretical result on the performance of MH. However, there is a very large amount of empirical results to support their effectiveness.

Early Evolutionary Image Registration Methods

The application of MHs to medical IR enjoyed a growing interest in the scientific community over the last 15 years, as shown in Fig. 4. The first attempts at IR can be found in the early eighties. The size of data as well as the number of parameters that are looked for prevent from an exhaustive search of the solutions. An approach based on a genetic algorithm (GA) [21] was proposed in 1984 for the 2D case and applied to angiographic images [17]. Later, in 1989, Mandava et al. [33] used a 64-bit structure to represent a possible solution when trying to find the eight parameters of a bilinear transformation through a binary GA. Brunnström and Stoddart [7] proposed a new method based on the manual prealignment of range images followed by an automatic IR process using a novel GA that searches for solutions following the matching-based approach. Tsang [49] used 48-bit chromosomes to encode three test points as a base for the estimation of the 2D affine registration function by means of a binary-coded GA. In the case of Yamany et al. [57] and Chalermwat et al. [8] proposals, the same binary coding is found when dealing with 3D and 2D rigid transformations, respectively. Yamany et al. enforced a range of $\pm 31°$

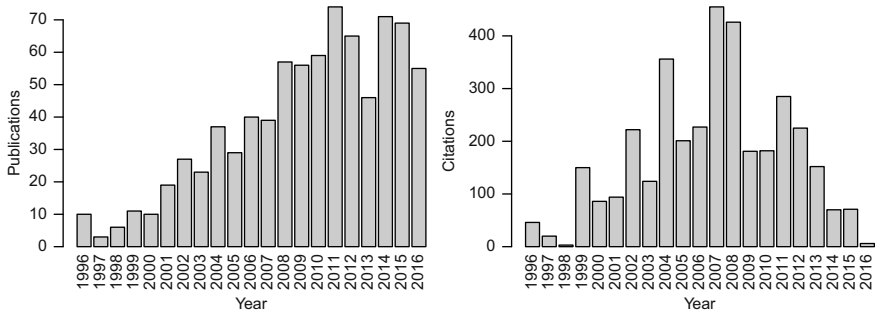


Fig. 4 The number of publications (*left*) and the number of citations (*right*) of literature contributions proposing MHs to solve medical image registration (The data shown in the graph was obtained from Thomson Reuter's Web of Science on November 29, 2016, using the query (Title OR Abstract OR Keywords) = ("image registration" OR "image alignment" OR "image matching") AND ("evolutionary algorithm" OR "evolutionary computation" OR "genetic programming" OR "genetic algorithm" OR "evolutionary programming" OR "evolution strategy" OR "differential evolution" OR "swarm intelligence" OR "bacterial foraging"))

over the angles of rotation and ± 127 units in displacement by defining a 42-bit chromosome with eight bits for each translation parameter and six bits for each rotation angle. Meanwhile, Chalermwat et al. used 12 bits for the coding of the 2D rotation parameter to get a search scope of $\pm 20.48°$, therefore allowing the use of a precision factor for the discretization of the continuous rotation angle interval. Other ten bits stored each of the two translation parameters (± 512 pixels).

All the latter approaches showed several pitfalls from an evolutionary computation perspective. On the one hand, they make use of the basic binary coding to solve inherently real-coded problems, when it is well known that binary coding suffers from discretization flaws (as problem solutions of search space never visited) and requires transformations to real values for each solution evaluation. Moreover, the kind of GA considered is usually based on the old-fashioned original proposal by Holland [22]. In this way, a selection strategy based on fitness-proportionate selection probability assignment and the stochastic sampling with replacement, as well as the classical one-point crossover and simple bit flipping mutation, are used. On the one hand, it is well known that such selection strategy causes a strong selective pressure, thus having a high risk of premature convergence of the algorithm. On the other hand, it has also been demonstrated that it is difficult for the single-point crossover to create useful descendants as it is excessively disruptive with respect to the building blocks.

Outstanding IR Methods Based on MHs

This section reviews the state of the art in MH-based proposals for medical IR, grouping the algorithms according to their nature.

Feature-Based Techniques

De Falco et al.'s Differential Evolution

In [14], the authors proposed an IR method based on the differential evolution (DE) [47]. DE is a parallel direct search method that has proved to be a promising candidate to solve real-valued optimization problems. DE combines simple arithmetic operators with the classical crossover, mutation, and selection genetic operators within an easy to implement scheme. It shows the advantage of considering few control parameters, namely, mutation factor and recombination rate. The fundamental idea of DE is a new scheme for generating trial solutions by adding the weighted differenced vector between two population members to a third one. The proposed method is applied to two 2D IR problems: mosaicking and changes in time of satellite images. Registration is carried out from the transformation parameters-based approach searching for the most suitable affine transformation (given by 11 real-coded parameters) in terms of maximization of the MI similarity metric.

Lomonosov et al.'s Genetic Algorithm

In [30], the authors tackle pairwise IR of range images, facing three real-world noisy measured datasets provided by their REPLICA laser range scanner and other two from the SAMPL database. They considered the transformation parameters-based approach using rigid transformations. The main novelties of this contribution are the inclusion of a degree of overlapping parameter in the solution vector and the utilization of the trimmed squares metric as objective function to be minimized. They constitute a different schematic approach for the IR problem that offers correct coarse IR results at overlaps under 50%. A random sampling procedure is performed in order to speed up the performance of the method. According to the evolutionary design of their method, a generational GA performing search in the seven-dimensional space formed by three translation parameters, three rotation parameters, and the newly added degree of overlapping parameter is considered. They used an integer coding representation of solutions which should be properly normalized onto the corresponding real-value range. Simple one-point crossover was employed, and two mutation operators were introduced. Shift mutation alters one parameter randomly by a value not exceeding 10% of the parameter range. Meanwhile, replacement mutation substitutes a parameter with a random value. Tournament and elitism were also employed.

Wachowiak et al.'s Particle Swarm Optimization

The authors contributed with a broad study of the performance of particle swarm optimization (PSO) algorithms [24] for solving the IR problem in biomedical applications [55]. In particular, they consider registering single slices (2D images) of 3D volumes to whole 3D volumes of medical images. In contrast to usual EAs which exploit the competitive characteristics of biological evolution (e.g., survival of the fittest), PSO exploits cooperative and social aspects, such as fish schooling, birds flocking, and insects swarming. However, both EAs and PSO approaches are considered population-based schemes. In particular, PSO algorithms start with a

random population (swarm) of individuals (particles) which iteratively change their search space location by performing movements based on a velocity vector. The authors addressed the IR problem from the transformation parameters-based approach, considering a rigid transformation and the MI similarity metric as the objective function to be maximized. The variant called *PSO7* is the one achieving the best performance. It refers to a basic PSO with the following formulation for the velocity vector update $v_i(t) = \chi[v_i(t-1) + \varphi_1\mu_1(p_i - x_i(t-1)) + \varphi_2\mu_2(g - x_i(t-1))]$ with $\varphi_1 = 2.1$, $\varphi_2 = 1.3$, and the constriction coefficient $\chi = 0.7298$. However, the performance of the method is dependent on the initial orientation of the images to be registered that should be provided by the user.

Cordón et al.'s Scatter Search

The main idea behind scatter search (SS) [28] is a systematic combination between solutions (as opposed to a randomized one, usually employed in GAs) taken from a considerably reduced pool of solutions, named reference set. The fact that the mechanisms within SS are not restricted to a single uniform design allowed the authors to explore and design different strategies that demonstrated to be effective tackling point matching IR problems [12]. Furthermore, new designs for three of the five SS components—the generator of diverse solutions, the improvement, and the combination methods—were proposed to develop a method outperforming the state-of-the-art point matching approaches. In particular, the authors adopted the same feature-based approach and the same representation of solutions based on permutations previously proposed in [9]. Similarity transformations present in 3D MRIs of human brains and 3D CTs of human wrists were also considered in this work. In particular, they succeeded at dealing with significant transformations between the two registered images, one of the ICP's pitfalls.

Santamaría et al.'s Scatter Search

In [45], the authors specifically performed a broad study about the performance capabilities of different memetic IR algorithms. The authors combined three existing MH-based methods [10, 11, 14] with several local search algorithms: XLS [6], Solis and Wets [46], and Powell [41]. Two different criteria (deterministic and probabilistic) were taken for the application of the local search, and different search intensity levels were tested. Thus, 57 different memetic IR methods were designed overall. The obtained experimental results in the 3D reconstruction of different human skull models, supported by a complementary nonparametric statistical test, revealed that the SS variant that made use of the deterministic local search application criterion and the XLS local search algorithm offered the best performance among all the developed memetic-based IR methods.

Intensity-Based Techniques

Valsecchi et al.'s Genetic Algorithm

r-GA$^+$ is an evolutionary IR method for medical imaging [50]. The optimization component of r-GA$^+$ is based on a GA with a real-coded design. A solution is

encoded as a real vector, storing the transformation parameters, and the variation operators are common choices for real-coded genetic algorithms: blend crossover (BLX-α) [16] and random mutation [2]. The fitness value of a solution t is simply the similarity metric between the two images when aligned according to t. No changes are required to handle different transformation models or similarity metrics. A distinctive feature of r-GA$^+$ is the use of multiple resolutions combined with a *restart* and a search space adaptation mechanism. The key idea is that if a low-quality solution is carried over to the second resolution, the process is unlikely to recover and produce a good final solution. Therefore, restart is used at the end of the first resolution until a suitable solution if found. This process is computationally cheap, because in the first resolution the algorithm is using a small version of the imaging data, and most of the total effort is spent on the further resolutions. In addition, as the second resolution is meant to be a refinement phase, the search is focused around the best solution by restricting the range of the transformation parameters.

Valsecchi et al.'s Scatter Search

The SS$^+$ algorithm [53] is based on a two-tier SS design, meaning there are two reference sets containing, respectively, the most high-quality and diverse solutions. The methods of the SS template have been specifically chosen for IR. The diversification generation method is based on frequency memory [20] to ensure the search space is explored in a uniform manner. The solution combination method is the BLX-α crossover, while the improvement method is a local search based on a the "parent-centric" version of BLX-α called PMX-α [31]. Last, the reference set update method maintain the highest-quality solutions in the quality reference set and the most diverse solutions in the diversity one, where the diversity value of a solution x measures the distance between the solution and the quality reference set S. SS$^+$ also includes a duplication control method, in order to prevent the reference set to contain only almost identical copies of the same solution. Moreover, SS$^+$ implements the same multi-resolution strategy as r-GA$^+$, including the restart and the dynamic boundary mechanism.

Bermejo et al.'s Bacterial Foraging Optimization Algorithm

r-BFOA is an evolutionary IR method for complex 3D scenarios [3]. The global optimization process of the original BFOA proposal [38] has been adapted to the IR problem, using a hybrid approach. Likewise r-GA$^+$, a solution is encoded as a real vector, which allows the use of a recombination mechanism between two solutions by means of the crossover operator BLX-α after each chemotactic step. In addition, after each reproduction step, XLS is applied to the best bacterium in the population. A comparative study of some design variants modifying the original BFOA scheme when applied to the IR problem is presented in [4]. Even though the BFOA algorithm provides an elimination-dispersal step which reintroduces random bacteria in the population, a restart mechanism has been applied to prevent the stagnation of the bacteria. This mechanism simulates the death of all the population, reintroducing new bacteria with the aim of providing a good solution to the second resolution.

Experimental Study

The aim of the experimentation is to carry out an objective, quantitative comparison of the methods described in section "Outstanding IR Methods Based on MHs". Two separated experiments have been developed considering, respectively, feature- and intensity-based methods. We also included the traditional optimization algorithms presented in section "Metaheuristics-Based Image Registration" as baseline. See [13, 53] for a more detailed description of these comparisons.

Each experiment uses a different dataset, from which we have created a number of registration scenarios. As most of the algorithms involved are of non-deterministic nature, we carried out a number of independent runs on each scenario. The analysis measures the performance of the algorithms on each scenario by computing their mean and standard deviation values and ranking the algorithms accordingly.

Feature-Based Comparison: Registration of Brain Magnetic Resonance Images

We use a dataset from the BrainWeb public repository of the McConnell Brain Imaging Centre [27]. The BrainWeb repository is a Simulated Brain Database (SBD) providing synthetic MRI data computationally generated. Full 3D data volumes have been simulated for both models using three sequences (T1-, T2-, or proton density- (PD) weighted) and a variety of slice thickness, noise levels, and levels of intensity nonuniformity (RF). Table 1 describes the particular settings considered to generate every BrainWeb image of our study (named BW1, BW2, and BW3).

To obtain a suitable set of features, we extracted the isosurface and selected the crest-line points with relevant curvature information from the original images, using a 3D crest-line edge detector [35] (Fig. 5). The resulting datasets have between 300 and 600 points per image (see Table 1).

We considered the parameter values originally proposed by the authors in every contribution. Nevertheless, we have adapted the majority of the methods by using the same objective function in order to carry out a fair comparison. We designed several IR problem instances, taking into account similarity transformations (rotation, translation, and uniform scaling) for medical applications, thus coping with the specific characteristics of this application domain. For each problem instance

Table 1 Detailed description of the BrainWeb image dataset generated by the on-line SBD system

Image	Model	Modality	Noise	Protocol	RF (%)	Crest-line points
BW1	Normal	T1	0	ICBM	20	583
BW2	Mild MS	T1	1	AI	20	348
BW3	Mild MS	T1	5	AI	20	284

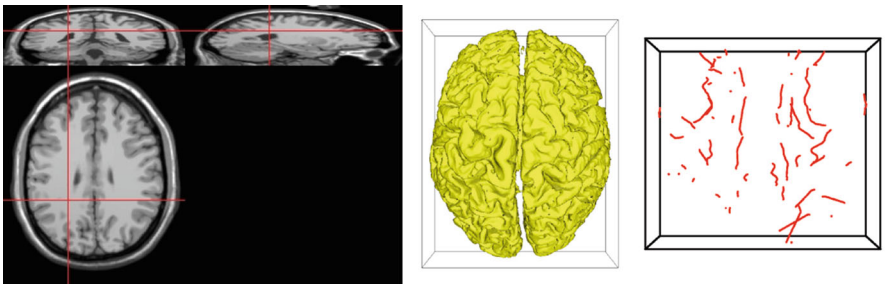


Fig. 5 From *left* to *right*: the original medical image, the corresponding extracted isosurface, and the crest line extracted from the isosurface

tackled by the IR methods, 30 different runs are performed. Each run considers a different similarity transformation. In order to perform a fair comparison among the methods included in this study, we considered CPU time as the stop criterion. After a preliminary study, we found that 20s was a suitable stop criterion to allow all the algorithms to converge properly.

The way a particular run is performed is as follows: a random (similarity) transformation is applied to the ground-truth image and then the IR method estimates the unknown inverse transformation. In particular, similarity transformations are randomly generated following a uniform probability distribution as follows: each of the three rotation axis parameters will be in the range $[-1, 1]$; the rotation angle will range in $[0°, 360°]$; the three translation parameters in $[-40$ mm, 40 mm$]$; the uniform scaling ranges in $[0.5, 2.0]$.

Results

The results of the first experiment are reported in Table 2. We computed the mean and standard deviation of the MSE and ranked the algorithm accordingly for each scenario. Table 3 shows the average rankings.

The average MSE values are considerably different in one algorithm to another even when the same scenario is considered. This, together with the generally high standard deviation values, points to the fact that the algorithms can occasionally perform poorly. Nevertheless, considering the results of Liu-ICP [29] as a baseline, all algorithms performed better than the baseline on average. Indeed, Liu-ICP delivered a consistent but poor performance, ranking last on all scenarios, with MSE values just below 3000. Cordón-SS, Lomonosov-GA, and Wachowiak-PSO have slightly similar results, with MSE values in mostly the range 500–1000 and similar rankings (3.67, 4 and 4.33, respectively). Santamaría-SS and DeFalco-DE delivered much better results compared to the rest of the algorithms. Impressively, Santamaría-SS always returned MSE values below 0.01. The same applies to DeFalco-DE in BW1-3 and BW2-3, but the algorithm scored 132 on BW1-2, a much higher value but still enough to rank second on that scenario. Therefore, while the two algorithms have quite similar rankings (1.33 and 1.67), in addition to ranking

Table 2 Detailed results of the first experiment. For each scenario, the table reports the average MSE, standard deviation and ranking of the algorithms in the comparison

		MSE		
	Algorithm	Mean	Sd	Rank
BW1-2	Liu-ICP	2788	2364	6
	Lomonosov-GA	838	2422	5
	DeFalco-DE	132	708	2
	Wachowiak-PSO	681	1817	4
	Cordón-SS	577	1715	3
	Santamaría-SS	0.003	0.001	1
BW1-3	Liu-ICP	3009	2279	6
	Lomonosov-GA	830	2426	4
	DeFalco-DE	0.013	0.002	2
	Wachowiak-PSO	1060	2496	5
	Cordón-SS	743	1596	3
	Santamaría-SS	0.011	0.003	1
BW2-3	Liu-ICP	2929	2094	6
	Lomonosov-GA	179	857	3
	DeFalco-DE	0.026	0.001	1
	Wachowiak-PSO	645	1908	4
	Cordón-SS	1133	1785	5
	Santamaría-SS	0.028	0.004	2

Table 3 First experiment: average rankings of the algorithms

Algorithm	Mean rank
Santamaría-SS	1.33
DeFalco-DE	1.67
Cordón-SS	3.67
Lomonosov-GA	4
Wachowiak-PSO	4.33
Liu-ICP	6

first overall, there is a clear advantage for Santamaría-SS in terms of consistency of its performance.

Second Experiment: Atlas-Based Segmentation of Brain Deep Nuclei

In the second experiment, we compare intensity-based algorithms over the registration of real brain MRI images without applying any transformation. The registration is used to perform atlas-based segmentation of deep brain structures [54]. The quality of the segmentation obtained in this phase is used to assess the effectiveness of the registration methods.

Atlas-based segmentation is a procedure that aims to automatically delineate a region of an image using an atlas (or an "average" image) of a similar subject in which the desired region has been already segmented. The first step is to register

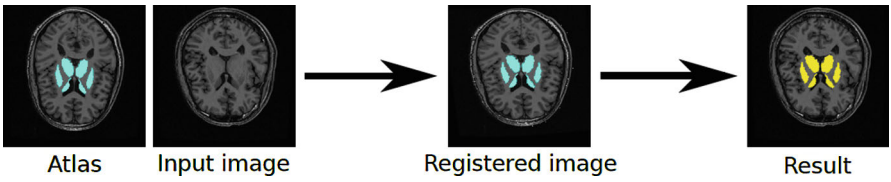


Fig. 6 The atlas-based segmentation of deep brain structures in brain MRI. First, the atlas is registered to the input image and the resulting transformation is applied to the labeled region of the atlas (*in blue*). The resulting region is overlapped on the input image (*in yellow*) to determine the output of the process

the atlas (the scene) to the input image (the model). The transformation resulting from this phase is then used to overlap the segmented region of the atlas to the scene. The region of the scene that overlaps the segmented region of the atlas is the result of the segmentation process. Figure 6 illustrates the process. Often, atlas-based segmentation is used as preliminary step in a more complex segmentation approach [34].

Thirteen T_1-weighted brain MRI were retrieved from the NMR database [40]. The deep nuclei structures in each image have been manually delineated by an expert in order to create the ground-truth data used to evaluate the registration. Figure 6 shows one of the images along with the corresponding deep nuclei. Eighteen registration scenarios were created by selecting pairs of different images at random.

The algorithms under comparison are SS^+, r-GA^+, r-BFOA, QN, and ASGD. The transformation model is affine transform, which involves rotation, translation, scaling, and shearing, and it can be represented using 12 real parameters. The stopping criterion is a time limit of 10 min. Note that no synthetic transformation is applied on the images; the registration aims to compensate for the variability in the pose of the patient during the acquisition of the images.

Results

The quality of atlas-based segmentation depends closely on the accuracy of the registration step, although the anatomical variability of the target region can limit its effectiveness. For each scenario we performed 32 independent runs of each algorithm. The segmented region obtained from the registration V_R is then compared with the ground-truth V_{GT}. The overlapping of the two regions is commonly measured using the Dice's coefficient [15], given by $\text{Dice}\ (V_R, V_{GT}) = \frac{2|V_R \cap V_{GT}|}{|V_R|+|V_{GT}|}$ where $|\cdot|$ is the number of voxels. A value of 1 means perfect overlapping, while 0 means the two regions do not overlap at all.

The results of the second experiment are reported in Table 4. We computed the mean and standard deviation of the overlap and ranked the algorithm accordingly for each scenario. Table 5 show the mean rankings for the algorithms in the comparison.

Table 4 Detailed results of the second experiment. For each scenario, the table reports the average overlap, standard deviation and ranking of the algorithms in the comparison

		MSE		
	Algorithm	Mean	Sd	Rank
1	ASGD	0.742	0.001	4
	r-BFOA	0.714	0.035	5
	r-GA$^+$	0.755	0.012	1
	QN	0.746		3
	SS$^+$	0.751	0.008	2
2	ASGD	0.679	0.003	1
	r-BFOA	0.661	0.038	5
	r-GA$^+$	0.670	0.012	4
	QN	0.677		2
	SS$^+$	0.673	0.026	3
3	ASGD	0.616	0.005	2
	r-BFOA	0.591	0.047	4
	r-GA$^+$	0.618	0.007	1
	QN	0.581		5
	SS$^+$	0.596	0.026	3
4	ASGD	0.677	0.003	3
	r-BFOA	0.622	0.064	5
	r-GA$^+$	0.679	0.008	2
	QN	0.687		1
	SS$^+$	0.677	0.008	4
5	ASGD	0.682	0.000	1
	r-BFOA	0.664	0.033	5
	r-GA$^+$	0.669	0.005	4
	QN	0.678		2
	SS$^+$	0.670	0.005	3
6	ASGD	0.691	0.001	3
	r-BFOA	0.660	0.063	5
	r-GA$^+$	0.706	0.016	2
	QN	0.685		4
	SS$^+$	0.722	0.018	1
7	ASGD	0.621	0.007	3
	r-BFOA	0.604	0.082	4
	r-GA$^+$	0.622	0.025	2
	QN	0.508		5
	SS$^+$	0.652	0.027	1
8	ASGD	0.756	0.010	3
	r-BFOA	0.715	0.048	4
	r-GA$^+$	0.760	0.009	2
	QN	0.636		5
	SS$^+$	0.773	0.012	1
9	ASGD	0.634	0.006	4
	r-BFOA	0.648	0.032	3
	r-GA$^+$	0.650	0.013	2
	QN	0.632		5
	SS$^+$	0.670	0.022	1

Table 4 (continued)

		MSE		
	Algorithm	Mean	Sd	Rank
10	ASGD	0.738	0.003	3
	r-BFOA	0.689	0.041	5
	r-GA$^+$	0.739	0.005	2
	QN	0.717		4
	SS$^+$	0.740	0.011	1
11	ASGD	0.717	0.011	3
	r-BFOA	0.709	0.025	4
	r-GA$^+$	0.728	0.010	2
	QN	0.544		5
	SS$^+$	0.750	0.013	1
12	ASGD	0.684	0.005	4
	r-BFOA	0.683	0.066	5
	r-GA$^+$	0.688	0.008	3
	QN	0.702		2
	SS$^+$	0.716	0.022	1
13	ASGD	0.686	0.009	4
	r-BFOA	0.673	0.033	5
	r-GA$^+$	0.729	0.004	1
	QN	0.696		3
	SS$^+$	0.718	0.014	2
14	ASGD	0.680	0.005	3
	r-BFOA	0.665	0.031	4
	r-GA$^+$	0.680	0.011	2
	QN	0.629		5
	SS$^+$	0.693	0.019	1
15	ASGD	0.741	0.001	4
	r-BFOA	0.690	0.041	5
	r-GA$^+$	0.750	0.007	3
	QN	0.764		2
	SS$^+$	0.769	0.023	1
16	ASGD	0.751	0.012	3
	r-BFOA	0.743	0.061	4
	r-GA$^+$	0.755	0.029	2
	QN	0.668		5
	SS$^+$	0.779	0.011	1
17	ASGD	0.754	0.004	2
	r-BFOA	0.705	0.034	5
	r-GA$^+$	0.749	0.014	3
	QN	0.736		4
	SS$^+$	0.756	0.020	1
18	ASGD	0.624	0.004	3
	r-BFOA	0.624	0.082	4
	r-GA$^+$	0.633	0.041	2
	QN	0.613		5
	SS$^+$	0.689	0.030	1

Table 5 Second experiment: the average rankings of the algorithms

Algorithm	Mean rank
SS^+	1.61
r-GA^+	2.22
ASGD	2.94
QN	3.72
r-BFOA	4.50

The overlap values can differ considerably across the scenarios, reflecting the fact that the effectiveness of this kind of segmentation can vary depending on the concrete anatomy of the patients. On average, r-BFOA scored the worst results in the comparison, ranking last in 10 out of 18 scenarios, with a mean rank of 4.5. High standard deviation values point to a quite uneven performance across different runs. QN ranked fourth on average, but overall the results are very inconsistent, with QN scoring in the top three in 7 out of 18 scenarios, but delivered much worse results in the remaining ones.

In general, r-GA^+ and ASGD have similar results, whereas SS^+ ranked constantly better than the others. ASGD delivered an acceptable performance, due to the low "magnitude" of the transformations involved, which is quite suitable for a local search method. However, note that the low variability of its overlap values means almost all solutions have a lower quality than the average solution found by the other two algorithms. r-GA^+ had a similar performance, while that of SS^+ is the best one of the comparison, with a mean rank of 1.61 against 2.22 and 2.94.

Conclusion

The adoption of MHs in order to solve optimization problems in the area of computer vision is steadily increasing. Medical IR is an excellent example of this trend, with feature-based being the most prevaling and mature approach. This chapter introduced the IR problem in depth and reviewed the design of some outstanding methods in the field, showing how IR have been successfully tackled using a wide spectrum of MH techniques. Despite the large interest in the community toward improving the traditional, well-known optimization methods for IR such as ICP, the experimentation in this chapter shows how algorithms based on MHs can provide superior results in terms of accuracy and robustness on challenging, real-world medical IR scenarios.

Cross-References

▸ Evolutionary Algorithms
▸ Fundamentals of Scatter Search
▸ Genetic Algorithms

Acknowledgments This work is supported by the Spanish "Ministerio de Economía y Competitividad" under the NEWSOCO project (ref. TIN2015-53067661) and the Andalusian Department of Innovación, Ciencia y Empresa under project TIC2011-7745, both including European Regional Development Funds (ERDF).

Competing Interest Declaration The author(s) has no competing interests to declare that are relevant to the content of this manuscript.

References

1. Audette MA, Ferrie FP, Peters TM (2000) An algorithmic overview of surface registration techniques for medical imaging. Med Image Anal 4(3):201–217
2. Bäck T, Fogel DB, Michalewicz Z (1997) Handbook of evolutionary computation. IOP Publishing Ltd/Oxford University Press, Bristol
3. Bermejo E, Cordón O, Damas S, Santamaría J (2013) Quality time-of-flight range imaging for feature-based registration using bacterial foraging. Appl Soft Comput 13(6):3178–3189
4. Bermejo E, Cordón O, Damas S, Santamaría J (2015) A comparative study on the application of advanced bacterial foraging models to image registration. Inf Sci 295:160–181
5. Besl PJ, McKay ND (1992) A method for registration of 3D shapes. IEEE Trans Pattern Anal Mach Intell 14:239–256
6. Beyer HG, Deb K (2001) On self-adaptive features in real-parameter evolutionary algorithms. IEEE Trans Evol Comput 5(3):250–270
7. Brunnström K, Stoddart A (1996) Genetic algorithms for free-form surface matching. International conference of pattern recognition, Vienna, pp 689–693
8. Chalermwat P, El-Ghazawi T, LeMoigne J (2001) 2-phase GA-based image registration on parallel clusters. Futur Gener Comput Syst 17:467–476
9. Cordón O, Damas S (2006) Image registration with iterated local search. J Heuristics 12:73–94
10. Cordón O, Damas S, Santamaría J (2006) A fast and accurate approach for 3D image registration using the scatter search evolutionary algorithm. Pattern Recogn Lett 27(11):1191–1200
11. Cordón O, Damas S, Santamaría J (2006) Feature-based image registration by means of the CHC evolutionary algorithm. Image Vis Comput 22:525–533
12. Cordón O, Damas S, Santamaría J, Martí R (2008) Scatter search for the 3D point matching problem in image registration. INFORMS J Comput 20:55–68
13. Damas S, Cordón O, Santamaría J (2011) Medical image registration using evolutionary computation: an experimental survey. IEEE Comput Intell Mag 6(4):26–42
14. De Falco I, Della Cioppa A, Maisto D, Tarantino E (2008) Differential evolution as a viable tool for satellite image registration. Appl Soft Comput 8(4):1453–1462
15. Dice LR (1945) Measures of the amount of ecologic association between species. Ecology 26(3):297–302
16. Eshelman LJ (1993) Real-coded genetic algorithms and interval schemata. In: Whitley LD (ed) Foundations of genetic algorithms 2. Morgan Kaufmann, San Mateo, pp 187–202
17. Fitzpatrick J, Grefenstette J, Gucht D (1984) Image registration by genetic search. In: IEEE Southeast conference. EEUU, Louisville, pp 460–464
18. Fok K, Wong T, Wong M (2007) Evolutionary computing on consumer graphics hardware. IEEE Intell Syst 22(2):69–78
19. Glover F, Kochenberger GA (eds) (2003) Handbook of metaheuristics. Kluwer Academic Publishers, Boston
20. Glover F, Laguna M, Martí R (2003) Scatter search. In: Ghosh A, Tsutsui S (eds) Advances in evolutionary computation: theory and applications. Springer, New York, pp 519–537
21. Goldberg DE (1989) Genetic algoritms in search and optimization. Addison-Wesley, EEUU, New York

22. Holland JH (1975) Adaptation in natural and artificial systems. The University of Michigan Press, Ann Arbor
23. Jenkinson M, Smith S (2001) A global optimisation method for robust affine registration of brain images. Med Image Anal 5(2):143–156
24. Kennedy J, Eberhart R (2001) Swarm intelligence. Morgan Kaufmann, San Francisco
25. Klein S, Staring M, Pluim JPW (2007) Evaluation of optimization methods for nonrigid medical image registration using mutual information and b-splines. IEEE Trans Image Process 16(12):2879–2890
26. Klein S, Pluim J, Staring M, Viergever M (2009) Adaptive stochastic gradient descent optimisation for image registration. Int J Comput Vis 81:227–239
27. Kwan RKS, Evans AC, Pike GB (1999) MRI simulation-based evaluation of image-processing and classification methods. IEEE Trans Med Imaging 18(11):1085–1097
28. Laguna M, Martí R (2003) Scatter search: methodology and implementations in C. Kluwer Academic Publishers, Boston
29. Liu Y (2004) Improving ICP with easy implementation for free form surface matching. Pattern Recogn 37(2):211–226
30. Lomonosov E, Chetverikov D, Ekart A (2006) Pre-registration of arbitrarily oriented 3D surfaces using a genetic algorithm. Pattern Recogn Lett 27(11):1201–1208
31. Lozano M, Herrera F, Krasnogor N, Molina D (2004) Real-coded memetic algorithms with crossover hill-climbing. Evol Comput 12(3):273–302
32. Maes F, Vandermeulen D, Suetens P (1999) Comparative evaluation of multiresolution optimization strategies for image registration by maximization of mutual information. Med Image Anal 3(4):373–386
33. Mandava VR, Fitzpatrick JM, Pickens DR (1989) Adaptive search space scaling in digital image registration. IEEE Trans Med Imaging 8(3):251–262
34. Mesejo P, Valsecchi A, Marrakchi-Kacem L, Cagnoni S, Damas S (2015) Biomedical image segmentation using geometric deformable models and metaheuristics. Comput Med Imaging Graph 43:167–178
35. Monga O, Deriche R, Malandain G, Cocquerez JP (1991) Recursive filtering and edge tracking: two primary tools for 3D edge detection. Image Vis Comput 9(4):203–214
36. Ong YS, Lim M, Zhu N, Wong K (2006) Classification of adaptive memetic algorithms: a comparative study. IEEE Trans Syst Man Cybern 36(1):141–152
37. Ong YS, Lim MH, Chen X (2010) Memetic computation – past, present & future. IEEE Comput Intell Mag 5(2):24–31
38. Passino K (2002) Biomimicry of bacterial foraging for distributed optimization and control. IEEE Control Syst 22(3):52–67
39. Pluim JPW, Maintz JBA, Viergever MA (2003) Mutual-information-based registration of medical images: a survey. IEEE Trans Med Imaging 22(8):986–1004
40. Poupon C, Poupon F, Allirol L, Mangin JF (2006) A database dedicated to anatomo-functional study of human brain connectivity. In: 12th annual meeting of the organization for human brain mapping, vol 646, Florence
41. Powell MJD (1964) An efficient method for finding the minimum of a function of several variables without calculating derivatives. Comput J 7(2):155–162
42. Robertson C, Fisher RB (2002) Parallel evolutionary registration of range data. Comput Vis Image Underst 87:39–50
43. Rouet JM, Jacq JJ, Roux C (2000) Genetic algorithms for a robust 3-D MR-CT registration. IEEE Trans Inf Technol Biomed 4(2):126–136
44. Rusinkiewicz S, Levoy M (2001) Efficient variants of the ICP algorithm. In: Third international conference on 3D digital imaging and modeling (3DIM 2001), Quebec, pp 145–152
45. Santamaría J, Cordón O, Damas S, García-Torres J, Quirin A (2009) Performance evaluation of memetic approaches in 3D reconstruction of forensic objects. Soft Comput 13(8–9):883–904
46. Solis FJ, Wets RJB (1981) Minimization by random search techniques. Math Oper Res 6(1):19–30

47. Storn R (1997) Differential evolution – a simple and efficient heuristic for global optimization over continuous spaces. J Glob Optim 11(4):341–359
48. Svedlow M, Mc-Gillem CD, Anuta PE (1976) Experimental examination of similarity measures and preprocessing methods used for image registration. In: Swain P, Morrison D, Parks D (eds) Symposium on machine processing of remotely sensed data, Indiana, vol 4(A), pp 9–17
49. Tsang PWM (1997) A genetic algorithm for aligning object shapes. Image Vis Comput 15:819–831
50. Valsecchi A, Damas S, Santamaría J, Marrakchi-Kacem L (2013) Genetic algorithms for voxel-based medical image registration. In: IEEE fourth international workshop on computational intelligence in medical imaging (CIMI 2013), pp 22–29
51. Valsecchi A, Dubois-Lacoste J, Stützle T, Damas S, Santamaría J, Marrakchi-Kacem L (2013) IEEE congress on evolutionary medical image registration using automatic parameter tuning. In: Evolutionary computation (CEC 2013), pp 1326–1333
52. Valsecchi A, Damas S, Santamaría J (2014) Evolutionary intensity-based medical image registration: a review. Curr Med Imaging Rev 10:283–297
53. Valsecchi A, Damas S, Santamaría J, Marrakchi-Kacem L (2014) Intensity-based image registration using scatter search. Artif Intell Med 60(3):151–163
54. Vemuri BC, Ye J, Chen Y, Leonard CM (2003) Image registration via level-set motion: applications to atlas-based segmentation. Med Image Anal 7(1):1–20
55. Wachowiak MP, Smolikova R, Zheng Y, Zurada JM, El-Maghraby AS (2004) An approach to multimodal biomedical image registration utilizing particle swarm optimization. IEEE Trans Evol Comput 8(3):289–301
56. Wang XY, Eberl S, Fulham M, Som S, Feng DD (2008) Data registration and fusion. In: Feng DD (ed) Biomedical information technology. Academic, Burlingto, pp 187–210
57. Yamany SM, Ahmed MN, Farag AA (1999) A new genetic-based technique for matching 3D curves and surfaces. Pattern Recogn 32:1817–1820
58. Zambanini S, Sablatnig R, Maier H, Langs G (2010) Automatic image-based assessment of lesion development during hemangioma follow-up examinations. Artif Intell Med 50(2):83–94
59. Zhang Z (1994) Iterative point matching for registration of free-form curves and surfaces. Int J Comput Vis 13(2):119–152
60. Zitová B, Flusser J (2003) Image registration methods: a survey. Image Vis Comput 21:977–1000

Metaheuristics for Natural Gas Pipeline Network Optimization

47

Roger Z. Ríos-Mercado

Contents

Introduction . . . 1470
Problem Description and Modeling Framework . . . 1472
Background . . . 1472
Description of Basic Model . . . 1473
Network Topology . . . 1474
Importance of Network Topology on Solution Algorithms . . . 1475
Solution Techniques: Classical Approaches . . . 1477
Tabu Search: An Approach for NLP Models . . . 1478
Nonsequential Dynamic Programming . . . 1478
The Tabu Search Approach . . . 1479
Empirical Performance . . . 1481
Ant Colony Optimization: An Approach for MINLP Models . . . 1481
Empirical Performance . . . 1485
Metaheuristic Approaches to Related Problems . . . 1485
Particle Swarm Optimization for Non-isothermal Systems . . . 1486
Simulated Annealing for Time-Dependent Systems . . . 1486
Simulated Annealing for Integrated Preventive Maintenance in Natural Gas Transmission Networks . . . 1487
NSGA-II for Multi-objective Optimization . . . 1487
Conclusion . . . 1488
Cross-References . . . 1489
References . . . 1490

R. Z. Ríos-Mercado (✉)
Graduate Program in Electrical Engineering, Universidad Autónoma de Nuevo León (UANL), San Nicolás de los Garza, NL, Mexico
e-mail: roger@yalma.fime.uanl.mx

© Springer Nature Switzerland AG 2025

R. Martí et al. (eds.), *Handbook of Heuristics*,
https://doi.org/10.1007/978-3-032-00385-0_51

Abstract

In this chapter, an overview on metaheuristic algorithms that have been very successful on tackling a particular class of natural gas pipeline network optimization problems is presented. In particular, the problem of minimizing fuel consumption incurred by the compressor stations driving natural gas in pipeline networks is addressed. This problem has been studied from different angles over the past few years by virtue of its tremendous economical impact. First, a general mathematical framework for this class of problems is presented. Then, the most relevant model properties and fundamental network topologies are thoroughly discussed. It is established how these different network topologies play a very important role on choosing an appropriate solution technique. This is followed by a presentation of current state-of-the-art metaheuristics for handling different versions of this problem. A discussion on metaheuristics developed to address related problems is included. Finally, some of the most relevant and important challenges of this very exciting area of research in natural gas transportation networks is highlighted.

Keywords

Natural gas transmission systems · Pipeline optimization · Nonlinear programming · Mixed-integer nonlinear programming · Tabu search · Ant colony optimization

Introduction

There are many interesting decision-making problems in the natural gas industry that have been studied over the years. These include fields such as pipeline design, gas storage, gathering, transportation, and marketing, to name a few. An important class of these problems, referred to as the minimum fuel consumption problem (MFCP), deals with how to operate a natural gas transportation network for delivering the gas from storage facilities to local distribution companies so as to minimize the fuel consumption employed by the compressor stations moving the gas along the network. Efficient design and operation of these complex networks can substantially reduce airborne emissions, increase safety, and decrease the very high daily operating costs due to the large amounts of fuel per day needed to operate the compressor stations driving the gas.

This type of networks is very complex and highly nonlinear, since the relationship between the flow variables in every arc and the pressure values at the interconnection points is represented by nonlinear equations, and, in some cases, by partial differential equations. Thus, in general, the class of MFCPs is very challenging due to the presence of nonlinearities and nonconvexities in the models representing such problems. These problems have been studied since the late 1960s

from many different angles, most of them based on classical hierarchical control and mathematical programming approaches.

It was until very recently that metaheuristics techniques were introduced for addressing some of these problems. One of the great advantages of metaheuristic algorithms over existing approaches is that the former does not depend on gradient-based information, so they can handle the nonlinear and nonconvex nature of the problems with relative ease. Furthermore, they can be combined with existing mathematical programming approaches in intelligent ways to derive hybrid metaheuristic methods.

The purpose of this chapter is to introduce the reader with an important class of challenging optimization problems in natural gas transportation networks and to give a detailed discussion on how metaheuristics have been successfully applied on addressing these problems. There are of course other important decision-making problems in the natural gas industry for which optimization techniques have made important contributions. A survey by Zheng et al [65] surveys optimization models in the natural gas industry, focusing on natural gas production, transportation, and marketing. Ríos-Mercado and Borraz-Sánchez [50] present an extensive review on classical techniques for fuel consumption minimization on transmission systems, including gathering, transmission, and local distribution. Schmidt et al. [53] present stationary nonlinear programming models of gas networks that are primarily designed to include detailed nonlinear physics in the final optimization steps for midterm planning problems. Farrokhifar et al. [19] present a comprehensive survey of literature in the contexts of coordinated planning of both natural gas and electricity systems.

It is important to note that we are focusing in the decision-making process of operating a pipeline network assuming the network is already designed. The problem consisting of how to design such network, known as a pipeline network design problem, is also an optimization problem where the decision variables are the diameter choices of the pipes, the flows, the potentials, and the states of various network components [25, 31]. This design problem is usually cast as a nonconvex mixed-integer nonlinear programming problem, and it is out of scope of the present work. The distribution problem [63] which consists of delivering the gas from the demand stations to the end customers is not considered either.

The chapter is organized as follows. The basic mathematical framework for the steady-state case, including important model properties, is presented in the first section. Then, in the following section, the existing classical approaches for handling different versions of this problem including steady-state and transient models are briefly highlighted. This is followed by two sections where both a Tabu Search algorithm for handling a nonlinear programming model and an Ant Colony Optimization algorithm for handling a mixed-integer nonlinear programming model are described. Other metaheuristic approaches for related problems in the natural gas industry are reviewed next. Final remarks and discussion on future research trends about metaheuristic techniques in natural gas optimization problems are given in the last section.

Problem Description and Modeling Framework

Background

Basically, the main purpose of a natural gas transmission network is to transport gas from storage facilities to local distribution companies. The gas is moved by pressure, and pressure is lost due to the friction of the gas flow with the inner wall of the pipelines. Thus, to keep the gas moving, compressor stations, whose primary role is to increase gas pressure, are needed. In turn, every compressor station is composed of several compressor units. These units may be identical or nonidentical and hooked up in different ways. The most typical configuration, which is assumed throughout this chapter, is that of identical compressor units hooked-up in parallel. It is well known that most of the operating costs in a pipeline network are due the amount of fuel consumed at the compressor stations.

When operating a natural gas transmission system aiming at minimizing fuel consumption, there are two main groups of decision variables that must be taken: (i) the mass flow rate through every pipe and compressor stations, and (ii) gas pressure values in each interconnection point. Additionally, decisions such has how many individual compressor units to operate within stations may be taken. Hence, the objective for a transmission network is to minimize the total fuel consumption of the compressor stations while satisfying specified delivery flow rates and minimum pressure requirements at the delivery terminals. The MFCP is typically modeled as a nonlinear or mixed-integer nonlinear network optimization problem. It is of course assumed that the network is given; that is, this is not a design problem.

Depending on how the gas flow changes with respect to time, we distinguish between systems in steady state and transient state. A system is said to be in steady state when the values characterizing the flow of gas in the system are independent of time. In this case, the system constraints, particularly the ones describing the gas flow through the pipes, can be described using algebraic nonlinear equations. In contrast, transient analysis requires the use of partial differential equations (PDEs) to describe such relationships. This makes the problem considerably harder to solve from the optimization perspective. In fact, optimization of transient models is one of the most challenging ongoing research areas. In the case of transient optimization, variables of the system, such as pressures and flows, are functions of time.

Gas transmission network problems differ from traditional network flow problems in some fundamental aspects. First, in addition to the flow variables for each arc, which in this case represent mass flow rates, a pressure variable is defined at every node. Second, besides the mass balance constraints, there exist two other types of constraints: (i) a nonlinear equality constraint on each pipe, which represents the relationship between the pressure drop and the flow; and (ii) a nonlinear nonconvex set which represents the feasible operating limits for pressure and flow within each compressor station. The objective function is given by a nonlinear function of flow rates and pressures. The problem is very difficult due to the presence of a nonconvex objective function and a nonconvex feasible region.

Description of Basic Model

Let $G = (V, A)$ be a directed graph representing a natural gas transmission network, where V is the set of nodes representing interconnection points, and A is the set of arcs representing either pipelines or compressor stations. Let V_s and V_d be the set of supply and demand nodes, respectively. Let $A = A_p \cup A_c$ be partitioned into a set of pipeline arcs A_p and a set of compressor station arcs A_c. That is, $(u, v) \in A_c$ if and only if u and v are the input and output nodes of compressor station (u, v), respectively.

Two types of decision variables are defined: Let x_{uv} denote the mass flow rate at arc $(u, v) \in A$, and let p_u denote the gas pressure at node $u \in V$. The following parameters are assumed known: B_u is the net mass flow rate in node u, and P_u^L and P_u^U are the pressure limits (lower and upper) at node u. By convention, $B_u > 0$ ($B_u < 0$) if $u \in V_s$ ($u \in V_d$) and $B_u = 0$ otherwise.

The basic mathematical model of the minimum fuel cost problem (MFCP) is given by:

$$\text{Minimize} \quad g(x, p) = \sum_{(u,v)\in A_c} g_{uv}(x_{uv}, p_u, p_v) \tag{1}$$

$$\text{subject to} \quad \sum_{v:(u,v)\in A} x_{uv} - \sum_{v:(v,u)\in A} x_{vu} = B_u \qquad u \in V \tag{2}$$

$$(x_{uv}, p_u, p_v) \in D_{uv} \qquad (u, v) \in A_c \tag{3}$$

$$x_{uv}^2 = R_{uv}(p_u^2 - p_v^2) \qquad (u, v) \in A_p \tag{4}$$

$$p_u \in [P_u^L, P_u^U] \qquad u \in V \tag{5}$$

$$x_{uv} \geq 0 \qquad (u, v) \in A \tag{6}$$

The objective function (1) measures the total amount of fuel consumed in the system, where $g_{uv}(x_{uv}, p_u, p_v)$ denotes the fuel consumption cost at compressor station $(u, v) \in A_c$. For a single compressor unit, the following function is typically used:

$$g^{(1)}(x_{uv}, p_u, p_v) = \frac{\alpha x_{uv}}{\eta} \left\{ \left(\frac{p_v}{p_u} \right)^m - 1 \right\},$$

where α and m are assumed constant and known parameters that depend on the gas physical properties, and η is the adiabatic efficiency coefficient. This adiabatic

coefficient is a function of (x_{uv}, p_u, p_v), that is, in general, a complex expression, implicitly defined. A function evaluation of η requires solving a linear system of algebraic equations. In practice, though, polynomial approximation functions that fit the function relatively well and are simpler to evaluate are employed. In other cases, when the fluctuations of η are small enough, η can be assumed to be a constant.

For a compressor station (u, v) with n_{uv} identical compressor units hooked-up in parallel which is very commonly found in industry, the fuel consumption is given by:

$$g_{uv}(x_{uv}, p_u, p_v) = n_{uv} g^{(1)}(x_{uv}/n_{uv}, p_u, p_v). \tag{7}$$

When all n_{uv} units are fixed and operating, we have a nonlinear programming (NLP) model. Treating n_{uv} as decision variables leads to mixed-integer nonlinear programming (MINLP) models.

Constraints (2) establish the mass balance at each node. Constraints (3) denote the compressor operating limits, where D_{uv} denote the feasible operating domain for compressor $(u, v) \in A_c$. Equations (4) express the relationship between the mass flow rate through a pipe and its pressure values at the end points under isothermal and steady-state assumptions, where R_{uv} (also known as the pipeline resistance parameter) is a parameter that depends on both the physical characteristics of the pipeline and gas physical properties. When the steady-state assumption does not hold, this relationship is a time-dependent partial differential equation which leads to transient models. Constraints (5) set the lower and upper limits of the pressure value at every node, and (6) set the nonnegativity condition of the mass flow rate variables. Further details of this model can be found in Wu et al. [61] and Ríos-Mercado [49].

Network Topology

There are three different kinds of network topologies: (a) linear or gun-barrel, (b) tree or branched, and (c) cyclic. Technically, the procedure for making this classification is as follows. In a given network, the compressor arcs are temporarily removed. Then each of the remaining connected components is merged into a big super-node. Finally, the compressor arcs are put back into their place. This new network is called the *associated reduced network*. Figure 1 illustrates the associated reduced network for a 12-node, 11-arc example. As can be seen, the reduced network has four supernodes (labeled S1, S2, S3, S4) and three arcs (the compressor station arcs from the original network).
Types of network topologies:

Linear topology: Reduced network is a single path.
Tree topology: Reduced network is a tree.
Cyclic topology: Reduced network has cycles (either directed or undirected).

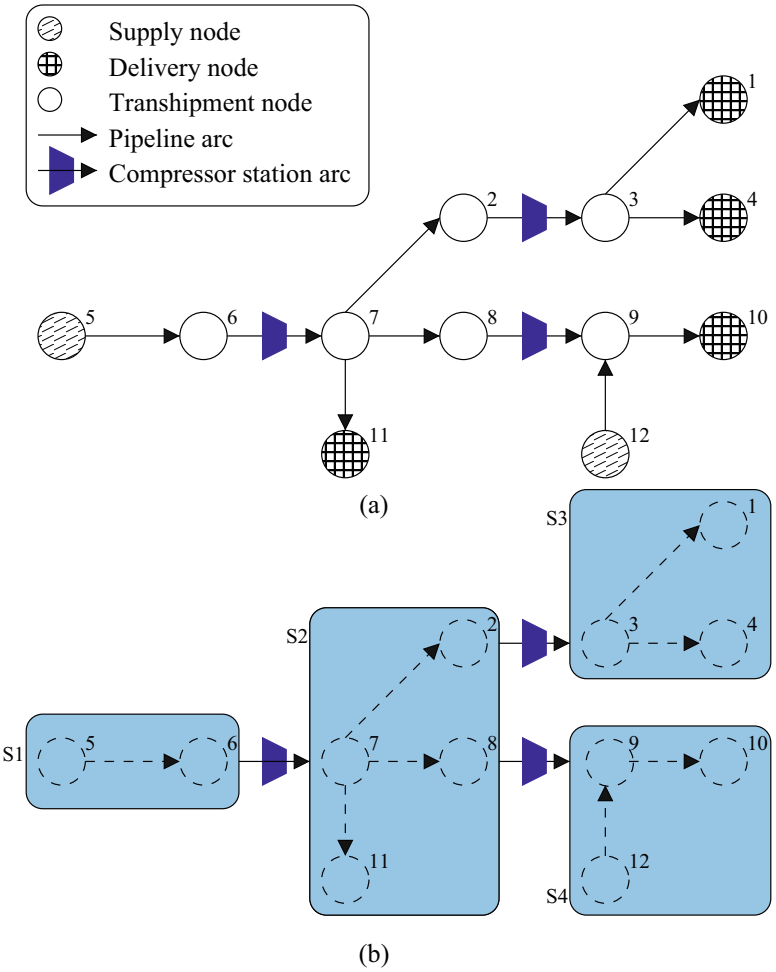


Fig. 1 Illustration of a reduced network. (**a**) Original network. (**b**) Associated reduced network

These different types of network topologies are shown in Fig. 2, were the original network is represented by solid line nodes and arcs, and the reduced network by dotted super nodes. Note that even though networks in Fig. 2a and b are not acyclic from a strict network definition, they are considered as noncyclic pipeline network structures.

Importance of Network Topology on Solution Algorithms

Let us consider the MFCP model given by (1)–(6), that is, the number of compressor units operating in each compressor station is known and fixed in advance. This is a

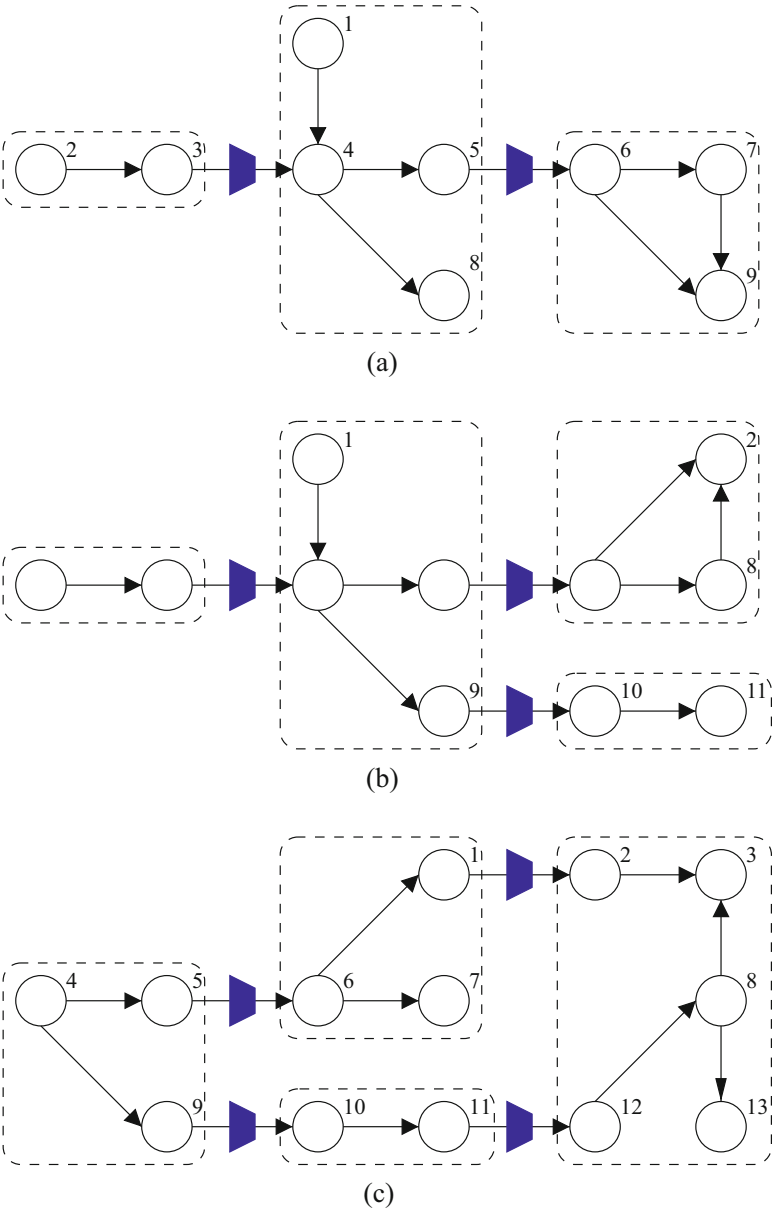


Fig. 2 Different kinds of pipeline network topologies. (**a**) Linear topology. (**b**) Tree topology. (**c**) Cyclic topology

nonconvex NLP. The network topology plays an important role, particular in steady-state topologies, when deriving algorithms for finding the optimal set of variables. Current state of the art on solution techniques for this steady-state MFCP reveals these important facts:

- There are theoretical results indicating that in noncyclic systems, the values of the flow variables can be uniquely determined and fixed beforehand [51]. Therefore, the problem reduces to finding out the optimal set of pressure variables at each node in the network. Of course, the problem is still hard to solve, but it reduces its dimension in terms of the decision variables.
- As a direct consequence of this, there exists successful implementations mostly based on dynamic programming (DP) that efficiently solve the problem in noncyclic instances by appropriately discretizing the pressure variables.
- When in a cyclic system, we impose the limitation of fixing the flow variables in each arc, a nonsequential dynamic programming (NDP), developed by Carter [8], can been successfully applied for finding the optimal set of pressure variables. Although this algorithm has the limitation of narrowing the set of solutions to those subject to a fixed set of flows, it can be used within other flow-modification-based approaches.

Solution Techniques: Classical Approaches

There is certainly a number of different optimization techniques that have been tried in the past to address problems in fuel cost minimization of natural gas transportation networks.

Steady-State NLP models: Most of the work for nonconvex NLP models has been based on steady-state models. One can find work on dynamic programming based techniques [5, 8, 15, 26, 29, 34, 52, 59, 60], including attempts to handle nonidentical compressor units [15], methods based on gradient search [20, 23, 45, 47], global optimization methods [24, 26], linearization techniques [12, 22], interior-point methods [21], exploiting model properties and lower bounding schemes [7, 51, 61], multi-objective optimization models [14], and multi-criteria approaches [40].

Steady-State MINLP models: There has also been studies on developing optimization methods for addressing MINLP models. In most of these models, integer variables for deciding which individual compressor units must be operating within a compressor station are introduced. Solution methodologies include mainly successive branch and bound [48, 56], outer approximation with augmented penalty [11, 66], sequential linear programming [23], and linearization techniques [38].

Transient models: Transient models are more challenging as the governing PDEs associated with the dynamics of the gas system must be taken into consideration. Efforts on addressing this class of very difficult problems include hierarchical control techniques [3, 30, 41–43, 46] in the early years, and, more recently, mathematical programming approaches [1, 16, 17, 28, 32, 36, 37, 44, 57, 64], Multi-objective optimization methods [10], and deep learning methods [2].

For a complete literature review and detailed discussion of some of these techniques, the reader is referred to the surveys of Zheng et al [65], Ríos-Mercado and Borraz-Sánchez [50], and Arya et al. [4]. In the following sections, we review

the most successful metaheuristic techniques applied to variations of the MFCP. Bear in mind that we are not including in the review below some papers on metaheuristics for natural gas pipeline optimization when the authors do not provide a full description of the algorithm (e.g., [58]).

Tabu Search: An Approach for NLP Models

For the past few years, Tabu Search (see ▸ Chap. 31, "Tabu Search") has established its position as an effective metaheuristic guiding the design and implementation of algorithms for the solution of hard combinatorial optimization problems in a number of different areas. A key reason for this success is the fact that the algorithm is sufficiently flexible to allow designers to exploit prior domain knowledge in the selection of parameters and subalgorithms. Another important feature is the integration of memory-based components.

When addressing the MFCP, even though we are dealing with a continuous optimization problem, Tabu Search (TS), with an appropriate discrete solution space, is a very attractive choice due to the the non-convexity of the objective function and the versatility of TS to overcome local optimality.

We now describe the TS-based approach of Borraz-Sánchez and Ríos-Mercado [6, 7], which is regarded as the most successful implementation of a metaheuristic for the MFCP. This TS takes advantage of the particular problem structure and properties and in fact can be regarded as a hybrid metaheuristic or matheuristic (see ▸ Chap. 8, "Matheuristics by Examples")

Let us consider the MFCP model given by (1)–(6), that is, the number of compressor units operating in each compressor station is known and fixed in advance. As established earlier, this is a nonconvex NLP.

Recall, from last section, that, in a cyclic system, if we fix the flow variables in each arc, NDP [8] can been successfully applied for finding the optimal set of pressure variables. Although this algorithm has the limitation of narrowing the set of solutions to those subject to a fixed set of flows, it can be used within flow-modification-based approaches such as the TS presented here.

It is clear that the TS approach is aiming at finding high-quality solutions for cyclic systems. It exploits the fact that for a given set of flows, an optimal set of pressure values can be efficiently found by NDP.

Nonsequential Dynamic Programming

We include in this section a brief description of the essence of the NDP algorithm. Further details can be found in [6]. Starting with a feasible set of flow variables, the NDP algorithm searches for the optimal set of node pressure values associated with that prespecified flow. Rather than attempting to formulate DP as a recursive algorithm, at a given iteration, the NDP procedure grabs two connected compressors and replace them by a "virtual" composite element that represents the optimal

operation of both compressors. These two elements can be chosen from anywhere in the system, so the idea of "sequential recursion" in classical DP does not quite apply here. After performing this step at a stage t, the system with t compressor stations has been replaced by an equivalent system with $t - 1$ stations. The procedure continues until only one virtual element, which fully characterizes the optimal behavior of the entire pipeline system, is left. Afterward, the optimal set of pressure variables can be obtained by a straightforward backtracking process. The computational complexity of this NDP technique is $O(|A_c|N_p^2)$, where N_p is the maximum number of elements in a pressure range discretization.

The Tabu Search Approach

The main steps of the algorithm are shown in Fig. 3. Here, a solution $Y = (X, P)$ is partitioned into its set of flow variables X and set of pressure variables P. First note that the search space employed by TS is defined by the flow variables X only because once the flow rates are fixed, the corresponding pressure variables are optimally found by NDP. Furthermore, we do not need to handle the entire set of flow variables, but only one per cycle. This is so because once you fix a flow rate in a cycle, the rest of the flows can be uniquely determined. Thus, a given state is represented by a vector $\hat{X} = (X_{\alpha_1}, \ldots, X_{\alpha_m})$, where α_w is an arc that belongs to a selected cycle w. Note that this set of arcs is arbitrarily chosen and that converting a flow from X to and from $\hat{X}$ is straightforward, so in the description, X and $\hat{X}$ are used interchangeably. This situation is illustrated in Fig. 4. The network represents the associated reduced network. It is clear that given a specified amount of net flow entering at node 1, only one arc in each cycle is needed to uniquely determine the flows in each arc of the network. In this case, the bold arcs (5,6) and (10,11), one per cycle, suffice.

We now describe each component.

- *Initial set of flows*: First, in Step 3, an initial set of feasible flows is found. Here, different methods such as classical assignment techniques can be applied in a straightforward manner.
- *Neighborhood definition*: In Step 5, a neighborhood $V(\bar{X})$ of a given solution $\bar{X} = \{\bar{x}_1, \bar{x}_2, \ldots, \bar{x}_m\}$ is defined as the set of solutions reachable from $\bar{X}$ via a modification of the current flow in each arc by Δ_x units in each of its components. This is given by

$$V(\bar{x}) = \{X' \in R^m \mid x'_w = \bar{x}_w \pm k\Delta_x, k = 1, 2, \ldots, N/2,\ w = 1, \ldots, m\}$$

where N is the predefined neighborhood size, Δ_x accounts for the mesh size, and the index w refers to the w-th cyclic component. Note that, for a given solution, the entire solution does not need to be stored but only the flow in the selected arc component to be modified. Note also that once this value is set, the rest of the

```
begin TS()
Input: An instance of the MFCP.
Output: A feasible solution (X, P).
1   (X, P)^best ← ∅;
2   TabuList ← ∅;
3   X̄ ← FIND_INITIAL_FLOW( );
4   while ( stopping criteria not met ) do
5       for ( X ∈ V(X̄) such that X ∉ TabuList ) do
6           P ← NDP( X );
7       end-for
8       Choose best (non-tabu) solution (X, P);
9       if (|TabuList| == TabuTenure) then
10          Remove oldest element from TabuList;
11      end-if
12      TabuList ← TabuList ∪ X;
13      (X, P)^best ← Best( (X, P), (X, P)^best );
14  end-while
15  return (X, P)^best;
end TS
```

Fig. 3 Pseudocode of Procedure TS

flow variables in the cycle are easily determined, so in this sense, it is precisely this mass flow rate which becomes the attribute of the solution.

- *Optimal pressure values*: In Steps 6–8, the corresponding set of pressure values for the given flow is found by invoking the NDP algorithm only for those flow values that are non-tabu.
- *Tabu list*: Then in Step 9, the best $X' \in V(\bar{X})$ which is non-tabu is chosen, and the corresponding subsets are updated accordingly. A tabu list (TabuList) stores recently used attributes, in our case, values of the X variables. The size of the

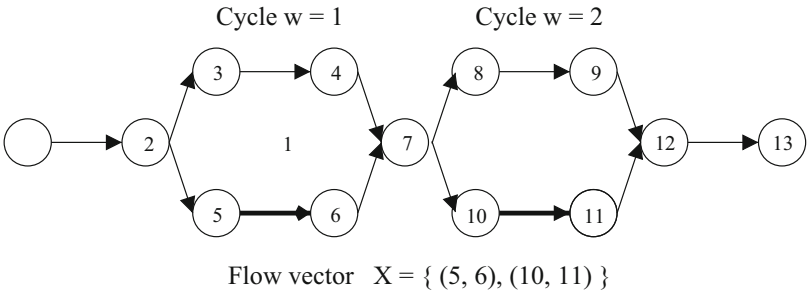


Fig. 4 Flow components of a feasible solution on a cyclic topology

TabuList (*TabuTenure*) controls the number of iterations a particular attribute is kept in the list.

- *Stopping criterion*: The search usually terminates after a given number of iterations, or when no significant change has been found in certain number of iterations.

As we know from theoretical properties of pipeline networks [51], the flow modification step is unnecessary for noncyclic topologies, because there exists a unique set of optimal flow values which can be determined in advance at preprocessing.

Empirical Performance

To illustrate the effectiveness of the TS, Table 1 shows the performance of the TS approach when it is compared with NDP. The table indicates the objective function value obtained by both methods and the relative improvement (RI) of the TS over the NDP solution when applied to 11 cyclic real-world size instances of up to 19 super-nodes and 7 compressor stations. As we can see, the TS method significantly outperformed NDP in terms of solution quality. The running times of the TS were about 220 and 400 CPU seconds. The running times of NDP were less than 20 CPU seconds. Although the NDP runs faster, the TS obtains solutions in less than 7/8 minutes, which is still very reasonable. Thus, it clearly pays off to spend additional computational effort, because the improvement in solution quality leads to substantial economical savings in real-world instances.

Ant Colony Optimization: An Approach for MINLP Models

Let us consider now the problem where, in addition to the flow variables in each arc and the pressure variables in each node, the decision process involves determining the number of operating units in each compressor as well. This leads to a MINLP

Table 1 Comparison between NDP and TS

Instance	NDP	TS	RI (%)
net-c-6c2-C1	2,317,794.61	2,288,252.53	1.27
net-c-6c2-C4	1,394,001.99	1,393,001.99	0.07
net-c-6c2-C7	1,198,415.69	1,140,097.39	4.86
net-c-10c3-C2	6,000,240.25	4,969,352.82	17.18
net-c-10c3-C4	2,533,470.72	2,237,507.93	11.68
net-c-15c5-C2	6,006,930.42	4,991,453.59	16.90
net-c-15c5-C4	3,669,976.44	3,371,985.41	8.11
net-c-15c5-C5	8,060,452.17	7,962,687.43	1.21
net-c-17c6-C1	9,774,345.45	8,659,890.72	11.40
net-c-19c7-C4	12,019,962.22	8,693,003.78	27.67
net-c-19c7-C8	8,693,003.78	7,030,280.45	19.12

model. In this section, the ant colony optimization (ACO) algorithm by Chebouba et al. [9] for this version of the MFCP is described.

Ant colony optimization (see ▸ Chap. 19, "Ant Colony Optimization: A Component-Wise Overview") is a relatively new evolutionary optimization method that has been successfully applied to a number of combinatorial optimization problems. ACO is based on the communication of a colony of simple agents (called ants), mediated by (artificial) pheromone trails. The main source of ACO is a pheromone trail laying and following behavior of real ants which use pheromones a communication medium. The pheromone trails in ACO serve as distributed, numerical information which the ants use to probabilistically construct solutions to the problem being solved and which the ants adapt during the algorithm's execution to reflect its search experience.

Regarding natural gas pipeline network optimization, Chebouba et al. [9] present an ACO metaheuristic for the MFCP with a variable number of compressor units within a compressor station. They focus on the linear topology case. As it was mentioned earlier, solving the MFCP on linear topologies has been successfully addressed by dynamic programming approaches when the number of compressor units is fixed and known, however, when the number of individual compressor units is variable and part of the decision process leads to a MINLP that has a higher degree of difficulty.

Consider the MINLP given by objective function (7) subject to constraints (2)–(6). When the number of individual compressor units within a compressor station are identical and hooked up in parallel, the linear system, as depicted in Fig. 2a, can be represented by a multigraph with the compressor stations aligned sequentially where the i-th compressor station (compressor arc $(i, i+1)$ in the figure) is modeled by a set of n_i arcs between suction node i and discharge pressure $i+1$ (see Fig. 5). Here, n_i is the number of individual compressor units, and each of the multi-arcs $(i, i+1)$ represents a decision on how many units are used in that particular station. Each multi-arc in the i-th station is denoted by $(i, i+1, r)$ (or simply (i, r)), where r identifies the number of individual compressor stations to be used in a particular solution. Let L be the set of edges in this multigraph given by

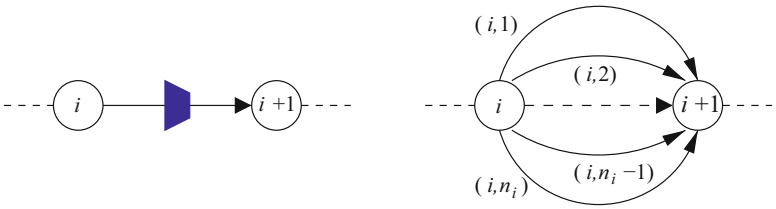


Fig. 5 Modeling compressor unit choices as a multigraph

$L = \{(i, r) : i \in \{1, \ldots, n\}, r \in \{1, \ldots, n_i\}\}$. In this case, the cost of arc (i, r) given by c_{ir} depends on the values of the pressure variables p_i and p_{i+1}. This will be determined during the construction of the solution. Following Eq. (7), the cost is then given by

$$c_{ir} = rg^{(1)}(x_{i,i+1}/r, p_i, p_{i+1}).$$

where it can be seen in a straightforward manner that, in the case of linear systems with known supply/demand values, the flow variables $x_{i,i+1}$ through the entire network can be determined and fixed beforehand. Furthermore, this cost is heuristically estimated once at the start of the procedure.

At the start of the algorithm, m ants are placed at the starting node. Ants build a solution while moving from node to an adjacent node by choosing one of the multi-arcs and by randomly generating values of the pressure variables for correct computation of the arc cost. During iteration t, each ant k carries out a partial path $T^k(t)$, and in this step, the choice of arc (i, r) depends on both the cost c_{ir} and the concentration of pheromone $\tau_{ir}(t)$ on arc (i, r) at iteration t. The pheromone trail takes into account the ant's current history performance. This pheromone amount is intended to represent the learned desirability of choosing the r-th edge at node i. The pheromone trail information is changed during problem solution to reflect the experience acquired by ants during problem-solving.

First, the algorithm introduces a transition rule depending on parameter $q_0 \in [0, 1]$, which determines the relative importance of intensification/diversification trade-off, every time an ant at node i chooses arc (i, r) according to the following transition rule:

$$r = \begin{cases} \arg\max_u (\tau_{iu}(t))^\alpha / (c_{iu})^\beta & \text{if } q \leq q_0, \\ s & \text{otherwise.} \end{cases}$$

where q is random variable uniformly distributed in [0, 1] and s is a random variable chosen according to the following probability function:

$$p_{is}^k(t) = \frac{(\tau_{is}(t))^\alpha / (c_{is})^\beta}{\sum_u (\tau_{iu}(t))^\alpha / (c_{iu})^\beta}$$

```
begin ACO()
Input: An instance of the MFCP.
Output: A feasible solution X.
1   t ← 0;
2   while ( stopping criteria not met ) do
3       t ← t + 1;
4       X^best ← ∅;
5       for (k = 1,....,m) do
6           Build solution X;
7           Apply local updating rule along path of X;
8           X^best ← Best(X, X^best);
9       end-for
10      Apply global updating rule along path of X^best;
11  end-while
12  return X^best;
end ACO
```

Fig. 6 Pseudocode of procedure ACO

As can be seen, low values of q_0 lead to diversification, and high values of q_0 stimulate intensification. Parameters α and β control the relative importance of the pheromone trail and greedy construction value. The main steps of the algorithm are shown in Fig. 6.

The pheromone trail is changed both locally (Step 7) and globally (Step 10) as follows:

- *Local updating*: Every time arc (i, r) is chosen by an ant, the amount of pheromone changes by applying this local trail update:

$$\tau_{ir}(t) \leftarrow (1 - \rho)\tau_{ir}(t) + \rho\tau_0$$

  where τ_0 is the initial pheromone value and ρ the evaporation rate.

Table 2 Comparison between DP and ACO

	DP		ACO		
Instance	OFV	Time	OFV	Time	RD (%)
Cs11–Nb6	49,114	10,332	49,217	721	0.21
Cs17–Nb9	81,050	28,262	81,052	1251	0.13
Cs23–Nb12	112,985	55,168	113,390	1869	0.36

- *Global updating*: Upon completion of a solution by every ant in the colony, the global trail updating is done as follows. The best ant (solution) from this finished iteration is chosen according to the best objective function value g^*. Then, in each arc $(i, i+1, r)$ used by this best ant, the trail is updated as:

$$\tau_{ir}(t+1) \leftarrow (1-\rho)\tau_{ir}(t) + \frac{\rho}{g^*}$$

Empirical Performance

To illustrate the usefulness of the ACO, the algorithm was tested on the Hassi R'mell-Arzew real-world pipeline network in Argelia consisting of five pipes, six nodes, five compressor stations, and three units in each compressor. The authors also built three additional cases with up to 23 compressor stations and 12 compressor units in each compressor. We must point out that these are noncyclic instances. The empirical evaluation includes a comparison with DP. Although these are noncyclic systems, the number of individual compressor units within each compressor station is considered a decision variable as well; thus, the DP may not guarantee an exact solution.

Table 2 displays the results in terms of objective function value (OFV) and running time when both DP and ACO are applied to three instances. The last column (RD) indicates the relative difference between the DP solution and the ACO solution. As we can see, the DP gives a slightly better solution, but it takes considerable more running time than the ACO. In the other hand, the difference in solution quality is very slim (less than 0.36%).

We conclude that ACO method performs reasonably well on these type of noncyclic networks. A great advantage is its relatively ease of implementation. The issue on how this algorithm can be modified so as to handle noncyclic systems remains an interesting topic for further investigation along this area.

Metaheuristic Approaches to Related Problems

In this section, we review some other related optimization problems in natural gas pipeline networks that have been addressed by metaheuristic methods.

Particle Swarm Optimization for Non-isothermal Systems

Wu et al [62] address a variation of the problem where, rather than minimizing fuel consumption, the focus is on maximizing a weighted combination of the maximum operation benefit and the maximum transmission amount. The operation benefit is defined as the sales income minus the costs. These costs include gas purchasing cost, pipeline's operation cost, management cost, and compressors running cost. The transmission amount is defined as the total gas volume that flows into the pipeline. In addition, a non-isothermal model is considered; that is, the authors consider the dynamics of the pipes being a function of temperature. Most of the literature focus on the isothermal case. They develop a particle swarm optimization (PSO) metaheuristic enhanced by an adaptive inertia weight strategy to adjust the weight value dynamically. In a PSO implementation (see ▶ Chap. 28, "Particle Swarm Optimization"), the inertia weight parameter is used to balance the global and local search ability. If the weight has a large value, the particle will search in a broader solution space. If the weight has a small value, the evolution process will focus on the space near to the local best particle. Thus, the global and local optimization performances of the algorithm can be controlled by dynamically adjusting the inertia weight value. This method adjusts the inertia weight adaptively based on the distance from the particles to the global best particle [55].

They tested their metaheuristic (named IAPSO) in the Sebeie-Ningxiae-Lanzhou gas transmission pipeline in China. Nine stations along the pipeline distribute gases to 16 consumers. There are four compressor stations with eight compressors to boost the gas pressure. The results show that IAPSO has fast convergence, obtaining reasonably good balances between the gas pipeline's operations benefit and its transportation amount.

Simulated Annealing for Time-Dependent Systems

As mentioned earlier, the previous two chapters addressed steady-state systems. However, when the steady-state assumption does not hold, the constraints that describe the physical behavior through a pipeline cannot be represented in the simplifying form as in (4). On the contrary, this behavior is governed by partial differential equations with respect to both flow and time. Therefore, to handle this situation, a discretization over the time variable must be done resulting in a highly complex optimization problem.

The resulting model is a mixed-integer nonlinear problem where now both flow variables and pressure variables are also a function of time; that is, we now have x_{ij}^t and p_i^t variables for every arc $(i, j) \in A$ and time step $t \in T$, where T is the set of time steps.

Although some efforts have been made to address transient systems, one of the most successful techniques for handling this problem is the simulated annealing (SA) algorithm of Mahlke et al. [35]. In that work, the authors use the following main ideas. First, they relax the equations describing the gas dynamic in pipes

by adding these constraints combined with appropriate penalty factors to the objective function. The penalty factor is dynamically updated resembling a strategic oscillation strategy. This gives the search plenty of flexibility. Then, they develop a suitable neighborhood structure for the relaxed problem where time steps as well as pressure and flow of the gas are decoupled. Their key idea of the neighborhood generation is a small perturbation of flow and pressure variables in the segments and nodes, respectively. An appropriate cooling schedule, an important feature of each SA implementation, is developed. They tested their metaheuristic on data instances provided by the German gas company E.ON Ruhrgas AG. The proposed SA algorithm yields feasible solutions in very fast running times.

Simulated Annealing for Integrated Preventive Maintenance in Natural Gas Transmission Networks

An interesting work proposing a framework for optimizing a natural gas pipeline including preventive maintenance scheduling operations is due to [39]. In his dissertation, the author proposes a simulated annealing metaheuristic to minimize compressor fuel cost in a natural gas transmission network through the integration of an annual maintenance plan for compressors in the network. The underlying multi-period mathematical model is an extension of the typical single-period MFCP models. Its solution is an operational/maintenance plan based on the best set of values uncovered for the operational and maintenance decision variables which results in the approximate overall minimization of the fuel consumed by the compressors in the network.

The proposed solution approach is applied to three different case studies from the literature which were developed for the evaluation of single-period MFCP models. The first case study involves a linear transmission network, the second a tree transmission network, and the third a cyclic transmission network. The time horizon of each case study is 52 weeks. Before execution of the SA algorithm, a set of experiments were carried out in order to determine appropriate combinations of model parameter values for each of the three case studies. The numerical results indicate that the algorithm is capable of finding high-quality solutions to all three instances.

NSGA-II for Multi-objective Optimization

The non-dominated sorting genetic algorithm II (NSGA-II) [13] is a multi-objective evolutionary heuristic that is used to tackle optimization problems with multiple objectives. This algorithm has been successfully applied to a variety of real-world multi-objective optimization problems, in particular with nonlinear objective functions. See ▸ Chap. 10, “Multiobjective Optimization.”)

The solution delivered by the NSGA-II is a set of non-dominated solutions which are a trade-off between the two objectives. Generally, every non-dominated solution is an acceptable solution. However, a decision-maker has to choose a single solution

from the optimal set by incorporating practical information and experience which significantly improves the operation of the system.

Kashani and Molaei [27] present a study that aims to find optimum values of three conflicting objective functions, namely, maximum gas delivery flow, maximum line pack, and minimum operating cost (sum of fuel consumption and carbon dioxide emission costs), simultaneously, for a natural gas pipeline network. They apply NSGA-II tom solve a small-scale case study with 5 compressor stations, 8 pipelines, and 14 nodes.

Demissie et al. [14] present a nonlinear multi-objective optimization model for optimizing the operation of natural gas pipeline networks in steady state. They consider linear, tree, and cyclic topologies. Their bi-objective optimization model aims at both minimizing fuel consumption and maximizing gas delivery flow rate. To address the problem, they apply a NSGA-II heuristic. They tested their algorithm in the data set instances by Wu et al. [61].

In a related work, Su et al [54] use NSGA-II to solve a bi-objective optimization problem that assesses the trade-off between reliability and power demand in natural gas pipeline networks under steady state. The model considers the uncertainties of the supply conditions and customer consumptions The effectiveness of the algorithm is tested on two typical pipeline networks, a tree-topology network and a loop-topology network. The results show that the developed optimization model is able to find solutions which effectively compromise the need of minimizing gas supply shortage risk and minimizing energy cost.

Conclusion

In this chapter, we have presented a description of successful metaheuristic implementations for handling very difficult optimization problems in fuel cost minimization of natural gas transportation networks. Compared to existing approaches, metaheuristics have the great advantage of not depending on gradient-based information such that they can handle nonlinearities and nonconvexities with relatively ease.

Nonetheless, metaheuristics have been widely applied mostly to discrete linear optimization problems and not to fully extent to handle the nasty problems within the natural gas industry. Therefore, there is a tremendous area of opportunity from the metaheuristic perspective in this very important field. One must have in mind that these are real-world problems where even a marginal improvement in the objective function value represent a significant amount of money to be saved given the total flow operation of these networks throughout the year. Therefore, further research in this area is justified and needed from the practical and scientific perspective.

Important research issues such as how to derive new metaheuristics or how the developed metaheuristics can be applied, extended, modified, so as to handle MFCPs under different assumptions (e.g., non-isothermal models, nonidentical compressor units, non-transient models, uncertainty) remain to be investigated. In

these lines, we have seen some preliminary efforts citing, for instance, the work of Mahlke et al. [35] who present a simulated annealing algorithm for addressing a MFCP under transient conditions. However, further work is needed. We know that advanced concepts in metaheuristic optimization research such as reactivity, adaptive memory, intensification/diversification strategies, or strategic oscillation are worthwhile investigating. Furthermore, as we have seen in this chapter, these models have a rich mathematical structure that allow for hybridization where part of the problem can be solved with mathematical programing techniques while being guided within a metaheuristic framework.

Over the past few years, we have also seen an increased interest in using machine learning techniques for handling certain variations of natural gas pipeline networks. One interesting feature of machine learning techniques such as deep reinforcement learning (DRL) is that they employ a model-free mechanism to optimize the system. For instance, Fan et al. [18] propose a method based on Bayesian networks and DRL to optimize the reliability of gas supply in natural gas pipeline networks. More recently, Liu et al. [33] propose an optimization framework for natural gas transportation pipeline networks based on DRL. The mathematical simulation model is derived from mass balance, hydrodynamics principles of gas flow, and compressor characteristics. The optimization control problem in steady state is formulated into a one-step Markov decision process and solved by DRL. The decision variables are selected as the discharge ratio of each compressor. This technique was empirically tested on very small scale noncyclic systems (gun-barrel and tree topologies). Although the technique was compared with DP and a GA, a step forward on this direction would be to investigate its applicability on the more challenging cyclic systems on larger real-world instances.

We hope we can stimulate the interest of the scientific community, particularly from the metaheuristic optimization field, to contribute to advance the state of the art in this very challenging research area.

Cross-References

- ▸ Ant Colony Optimization: A Component-Wise Overview
- ▸ Matheuristics by Examples
- ▸ Multiobjective Optimization
- ▸ Particle Swarm Optimization
- ▸ Tabu Search

Acknowledgments The author would like to thank the editors and an anonymous reviewer for their remarks and suggestions that helped improve the presentation of the work.

Competing Interest Declaration The author(s) has no competing interests to declare that are relevant to the content of this manuscript.

References

1. Aalto H (2008) Optimal Control of natural gas pipeline networks: a real-time, model-based, receding horizon optimisation approach. VDM Verlag, Saarbrücken
2. Anderson L, Turner M, Koch T (2022) Generative deep learning for decision making in gas networks. Math Methods Oper Res 95(3):503–532
3. Anglard P, David P (1988) Hierarchical steady state optimization of very large gas pipelines. In: Proceedings of the 20th PSIG annual meeting, Toronto
4. Arya AK, Jain R, Yadav S, Bisht S, Gautam S (2022) Recent trends in gas pipeline optimization. Mater Today Proc 57(4):1455–1461
5. Borraz-Sánchez C, Haugland D (2011) Minimizing fuel cost in gas transmission networks by dynamic programming and adaptive discretization. Comput Ind Eng 61(2):364–372
6. Borraz-Sánchez C, Ríos-Mercado RZ (2005) A hybrid meta-heuristic approach for natural gas pipeline network optimization. In: Blesa MJ, Blum C, Roli A, Sampels M (eds) Hybrid metaheuristics. Springer, Berlin, pp 54–65
7. Borraz-Sánchez C, Ríos-Mercado RZ (2009) Improving the operation of pipeline systems on cyclic structures by tabu search. Comput Chem Eng 33(1):58–64
8. Carter RG (1998) Pipeline optimization: dynamic programming after 30 years. In: Proceedings of the 30th PSIG annual meeting, Denver
9. Chebouba A, Yalaoui F, Smati A, Amodeo L, Younsi K, Tairi A (2009) Optimization of natural gas pipeline transportation using ant colony optimization. Comput Oper Res 36(6):1916–1923
10. Chen Q, Wu C, Zuo L, Mehrtash M, Wang Y, Bu Y, Sadiq R, Cao Y (2021) Multi-objective transient peak shaving optimization of a gas pipeline system under demand uncertainty. Comput Chem Eng 147:107260
11. Cobos-Zaleta D, Ríos-Mercado RZ (2002) A MINLP model for minimizing fuel consumption on natural gas pipeline networks. In: Proceedings of the XI Latin-Ibero-American conference on operations research, Concepción, Chile
12. De Wolf D, Smeers Y (2000) The gas transmission problem solved by an extension of the simplex algorithm. Manag Sci 46(11):1454–1465
13. Deb K, Pratap A, Agarwal S, Meyarivan T (2002) A fast and elitist multiobjective genetic algorithm: NSGA-II. IEEE Trans Evolut Comput 6(2):182–197
14. Demissie A, Zhu W, Belachew CT (2017) A multi-objective optimization model for gas pipeline operations. Comput Chem Eng 100:94–103
15. Deng T, Liang Y, Zhang S, Ren J, Zheng S (2019) A dynamic programming approach to power consumption minimization in gunbarrel natural gas networks with nonidentical compressor units. INFORMS J Comput 31(3):593–611
16. Domschke P, Geißler B, Kolb O, Lang J, Martin A, Morsi A (2011) Combination of nonlinear and linear optimization of transient gas networks. INFORMS J Comput 23(4):605–617
17. Ehrhardt K, Steinbach MC (2005) Nonlinear optimization in gas networks. In: Bock HG, Kostina E, Phu HX, Rannacher R (eds) Modeling, simulation and optimization of complex processes. Springer, Berlin, pp 139–148
18. Fan L, Su H, Wang W, Zio E, Zhang L, Yang Z, Peng S, Yu W, Zuo L, Zhang J (2022) A systematic method for the optimization of gas supply reliability in natural gas pipeline network based on bayesian networks and deep reinforcement learning. Reliab Eng Syst Safety 225:108613
19. Farrokhifar M, Nie Y, Pozo D (2020) Energy systems planning: a survey on models for integrated power and natural gas networks coordination. Appl Energy 262:114567
20. Flores-Villarreal HJ, Ríos-Mercado RZ (2003) Computational experience with a GRG method for minimizing fuel consumption on cyclic natural gas networks. In: Mastorakis NE, Stathopulos IA, Manikopoulos C, Antoniou GE, Mladenov VM, Gonos IF (eds) Computational methods in circuits and systems applications. WSEAS Press, Athens, pp 90–94
21. García-Marín S, González-Vanegas W, Murillo-Sánchez CE (2024) MPNG: A MATPOWER-based tool for optimal power and natural gas flow analyses. IEEE Trans Power Syst 39(4):5455–5464

22. Ghilardi LMP, De Pascali ML, Casella F, Barbati D, Palazzo R, Martelli E (2022) A MILP approach for the operational optimization of gas network. IFAC-PapersOnLine 55(9): 321–326
23. González Rueda AM, González Díaz J, Fernández de Córdoba MP (2019) A twist on SLP algorithms for NLP and MINLP problems: an application to gas transmission networks. Optim Eng 20:349–395
24. Habeck O, Pfetsch ME, Ulbrich S (2019) Global optimization of mixed-integer ODE constrained network problems using the example of stationary gas transport. SIAM J Optim 29(4):2949–2985
25. Hong B, Li X, Di G, Song S, Yu W, Chen S, Li Y, Gong J (2020) An integrated MILP model for optimal planning of multi-period onshore gas field gathering pipeline system. Comput Ind Eng 146:106479
26. Jin L, Wojtanowocz AK (2010) Optimization of large gas pipeline network – A case study in China. J Canadian Petrol Technol 49(4):36–43
27. Kashani AHA, Molaei R (2014) Techno-economical and environmental optimization of natural gas network operation. Chem Eng Res Design 92(11):2106–2122
28. Ke SL, Ti HC (2000) Transient analysis of isothermal gas flow in pipeline network. Chem Eng J 76(2):169–177
29. Lall HS, Percell PB (1990) A dynamic programming based gas pipeline optimizer. In: Bensoussan A, Lions JL (eds) Analysis and optimization of systems. Lecture notes in control and information sciences, vol 144. Springer, Berlin, pp 123–132
30. Larson RE, Wismer DA (1971) Hierarchical control of transient flow in natural gas pipeline networks. In: Proceedings of the IFAC symposium on distributed parameter systems. Banff, Alberta
31. Li Y, Dey SS, Sahinidis NV (2024) A reformulation-enumeration MINLP algorithm for gas network design. J Global Optim 90:931–963
32. Liu K, Biegler LT, Zhang B, Chen Q (2020) Dynamic optimization of natural gas pipeline networks with demand and composition uncertainty. Chem Eng Sci 215:115449
33. Liu ZE, Long W, Chen Z, Littlefield J, Jing L, Ren B, El-Houjeiri HM, Qahtani AS, Jabbar MY, Masnadi MS (2024) A novel optimization framework for natural gas transportation pipeline networks based on deep reinforcement learning. Energy AI 18:100434
34. Luongo CA, Gilmour BJ, Schroeder DW (1989) Optimization in natural gas transmission networks: A tool to improve operational efficiency. Technical Report, Stoner Associates, Houston
35. Mahlke D, Martin A, Moritz S (2007) A simulated annealing algorithm for transient optimization in gas networks. Math Methods Oper Res 66(1):99–115
36. Mak TWK, Van Hentenryck P, Zlotnik A, Bent R (2020) Dynamic compressor optimization in natural gas pipeline systems. INFORMS J Comput 31(1):40–65
37. Mantri VB, Preston LB, Pringle CS (1985) Transient optimization of a natural gas pipeline system. In: Proceedings of the 17th PSIG annual meeting, Albuquerque
38. Martin A, Möller M, Moritz S (2006) Mixed integer models for the stationary case of gas network optimization. Math Programm 105(2–3):563–582
39. Mendes A (2022) A modelling framework for integrated compressor maintenance scheduling and fuel cost minimisation in natural gas transmission networks. PhD thesis, Stellenbosch University, Stellenbosch
40. Mohammad NE, Yassmen YR, Aly S, Hussein MH (2024) A multi-objective optimization method for simulating the operation of natural gas transport system. Korean J Chem Eng 41:1609–1624
41. Osiadacz AJ (1994) Dynamic optimization of high pressure gas networks using hierarchical systems theory. In: Proceedings of the 26th PSIG annual meeting, San Diego
42. Osiadacz AJ (1998) Hierarchical control of transient flow in natural gas pipeline systems. Int Trans Oper Res 5(4):285–302
43. Osiadacz AJ, Bell DJ (1986) A simplified algorithm for optimization of large-scale gas networks. Optim Control Appl Methods 7(1):95–104

44. Osiadacz AJ, Chaczykowski M (2001) Comparison of isothermal and non-isothermal pipeline gas flow models. Chem Eng J 81(1):41–51
45. Osiadacz AJ, Kwestarz M (2021) Nonlinear steady-state optimization of large-scale gas transmission networks. Energies 14(10):2832
46. Osiadacz AJ, Swierczewski S (1994) Optimal control of gas transportation systems. In: Proceedings of the 3rd IEEE conference on control applications, Glasgow, pp 795–796. ISBN: 0-7803-1872-2
47. Percell PB, Ryan MJ (1987) Steady-state optimization of gas pipeline network operation. In: Proceedings of the 19th PSIG annual meeting, Tulsa
48. Pratt KF, Wilson JG (1984) Optimisation of the operation of gas transmission systems. Trans Instit Measure Control 6(5):261–269
49. Ríos-Mercado RZ (2002) Natural gas pipeline optimization. In: Pardalos PM, Resende MGC (eds) Handbook of applied optimization, Chap 18.8.3. Oxford University Press, New York, pp 813–825
50. Ríos-Mercado RZ, Borraz-Sánchez C (2015) Optimization problems in natural gas transportation systems: a state-of-the-art review. Appl Energy 147:536–555
51. Ríos-Mercado RZ, Wu S, Scott LR, Boyd EA (2002) A reduction technique for natural gas transmission network optimization problems. Ann Oper Res 117(1–4):217–234
52. Ríos-Mercado RZ, Kim S, Boyd EA (2006) Efficient operation of natural gas transmission systems: a network-based heuristic for cyclic structures. Comput Oper Res 33(8):2323–2351
53. Schmidt M, Steinbach MC, Willert BM (2015) High detail stationary optimization models for gas networks. Optim Eng 16(1):131–164
54. Su H, Zio E, Zhang J, Li X, Chi L, Fan L, Zhang Z (2019) A method for the multi-objective optimization of the operation of natural gas pipeline networks considering supply reliability and operation efficiency. Comput Chem Eng 131:106584
55. Suresh K, Ghosh S, Kundu D, Sen A, Das S, Abraham A (2008) Inertia-adaptive particle swarm optimizer for improved global search. In: Proceedings of the eighth international conference on intelligent systems design and applications. IEEE Computer Society, Los Alamitos, pp 253–258
56. Tabkhi F, Pibouleau L, Hernandez-Rodriguez G, Azzaro-Pantel C, Domenech S (2010) Improving the performance of natural gas pipeline networks fuel consumption minimization problems. AIChE J 56(4):946–964
57. Tao WQ, Ti HC (1998) Transient analysis of gas pipeline network. Chem Eng J 69(1):47–52
58. Wei X, Qiu R, Zhang B, Liu C, Wang G, Wang B, Liang Y (2024) Operation optimization of large-scale natural gas pipeline networks based on intelligent algorithm. Energy 310:133,258
59. Wong PJ, Larson RE (1968) Optimization of natural-gas pipeline systems via dynamic programming. IEEE Trans Autom Control AC–13(5):475–481
60. Wong PJ, Larson RE (1968) Optimization of tree-structured natural-gas transmission networks. J Math Analy Appl 24(3):613–626
61. Wu S, Ríos-Mercado RZ, Boyd EA, Scott LR (2000) Model relaxations for the fuel cost minimization of steady-state gas pipeline networks. Math Comput Modell 31(2–3):197–220
62. Wu X, Li C, Jia W, He Y (2014) Optimal operation of trunk natural gas pipelines via an inertia-adaptive particle swarm optimization algorithm. J Natl Gas Sci Eng 21:10–18
63. Zhang L, Zhang S, Xu C (2021) Network optimisation for transporting liquefied natural gas from stations to end customers. Int J Prod Res 59(6):1791–1813
64. Zhang S, Liu S, Deng T, Shen ZJM (2020) Transient-state natural gas transmission in gunbarrel pipeline networks. INFORMS J Comput 32(3):697–713
65. Zheng QP, Rebennack S, Iliadis NA, Pardalos PM (2010) Optimization models in the natural gas industry. In: Rebennack S, Pardalos PM, Pereira MVF, Iliadis NA (eds) Handbook of power systems I, energy systems. Springer, Berlin, pp 121–148
66. Zhou J, Peng J, Liang G, Peng C (2022) Operation optimization of multiroute cyclic natural gas transmission network under different objectives. J Pipeline Syst Eng Pract 13(1):1–11

Metaheuristics for Problems in Sports Scheduling

48

Celso C. Ribeiro, Sebastián Urrutia, and Dominique de Werra

Contents

Introduction 1494
Exact and Approximate Optimization Methods 1495
Elements of Graph Theory 1497
Basic Graph Model for Single Round-Robin Tournaments: The Circle Method 1498
Local Search and Neighborhoods 1500
Fundamentals of Metaheuristics for Sports Scheduling 1501
  Simulated Annealing 1502
  Tabu Search 1502
  Greedy Randomized Adaptive Search Procedures 1503
  Iterated Local Search 1504
  Variable Neighborhood Search 1504
  Genetic Algorithms 1505
Neighborhoods for Round-Robin Tournaments 1506
  Neighborhood "Home-Away Swap" (HAS) 1507
  Neighborhood "Team Swap" (TS) 1507
  Neighborhood "Round Swap" (RS) 1507
  Neighborhood "Partial Round Swap" (PRS) 1508
  Neighborhood "Partial Team Swap" (PTS) 1508
  Neighborhood "Generalized Partial Team Swap" (GPTS) 1510
  Considerations About the Neighborhoods 1510
  Connectivity of the Tournament Space 1511

C. C. Ribeiro (✉)
Institute of Computing, Universidade Federal Fluminense, Niterói, Brazil
e-mail: celso@ic.uff.br

S. Urrutia
Faculty of Logistics, Molde University College, Molde, Norway
e-mail: sebastian.urrutia@himolde.no

D. de Werra
Département de Mathématiques, Ecole Polytechnique Fédérale de Lausanne, Lausanne, Switzerland
e-mail: dominique.dewerra@epfl.ch

© Springer Nature Switzerland AG 2025

R. Martí et al. (eds.), *Handbook of Heuristics*,
https://doi.org/10.1007/978-3-032-00385-0_62

Concluding Remarks 1514
References 1517

Abstract

In this chapter, we consider the problem of scheduling time-constrained round-robin sports tournament problems, metaheuristics for its approximate solution, and their practical applications. This chapter uses graph theory as the fundamental tool. It presents a short introduction to its essential elements, definitions, and results relevant to the subject of this work. We also introduce the basic graph model used to represent single round-robin tournaments and the circle method for constructing timetables. We describe the different neighborhood structures that have been used in the literature of metaheuristics and local search procedures for scheduling time-constrained single round-robin sports tournaments: Home-Away Swap, Team Swap, Partial Team Swap, Round Swap, Partial Round Swap, and Generalized Partial Team Swap. They may be directly extended to double round-robin problems. We conclude by presenting a collection of applications of metaheuristics to sports scheduling problems.

Keywords

Round-robin tournaments · Time-constrained tournament · Circle method · Local search · Metaheuristics · Neighborhoods for round-robin problems · Applications of sports scheduling

Introduction

Metaheuristics are general high-level procedures that coordinate simple methods and rules to find high-quality (often optimal) approximate solutions to hard optimization problems. Among them, we find simulated annealing, tabu search, greedy randomized adaptive search procedures (GRASP), iterated local search (ILS), variable neighborhood search (VNS), genetic algorithms, and others. They have been applied to various academic and real-world problems, particularly sports scheduling.

A *round-robin tournament* (or *all-play-all tournament*) between an even number $2n$ of competitors is one in which each competitor plays precisely the same number of games with every other competitor in the tournament. The games are played in rounds in which each team plays exactly once. This definition implies that the tournament is time-constrained and excludes time-relaxed sports tournaments in which the number of available rounds is greater than the number of games each team must play. Although many sports tournaments are time-constrained, some are time-relaxed. An interesting application in the context of time-relaxed tournaments appeared in [12], where a heuristic is proposed to schedule the games of the National Hockey League in Canada and the United States.

Most professional sports leagues play either a single round-robin tournament (SRR), where each pair of teams meets once, or a double round-robin tournament (DRR), where each pair of teams meets twice. In general, each competitor plays against each other exactly m times. The number of rounds in a compact tournament (i.e., a tournament in which each competitor plays precisely once in each round) is equal to $m(2n - 1)$.

Double round-robin tournaments can be organized in different manners, such as unconstrained tournaments in which the games can be played in any arbitrary order; tournaments with constraints avoiding that the two games between the same pair of competitors be played in two consecutive rounds; tournaments organized in two separate phases, each of which is an SRR tournament itself (possibly with different timetables); and mirrored tournaments (in which both phases are SRRs with the same timetables, but the venues of the two games between each pair of competitors are interchanged from one phase to the other).

The remainder of this chapter is organized as follows. The next section overviews approximate optimization methods, from construction algorithms to local search and from local search to metaheuristics. Section "Elements of Graph Theory" presents the fundamental elements of graph theory used throughout this chapter. Section "Basic Graph Model for Single Round-Robin Tournaments: The Circle Method" presents the basic graph model for representing single round-robin tournaments and describes the basic circle method for constructing canonical schedules. Section "Local Search and Neighborhoods" examines neighborhoods, search spaces, and local search methods. Section "Fundamentals of Metaheuristics for Sports Scheduling" reviews some fundamental principles of metaheuristics and describes some of the most widely and successfully used metaheuristics in sports scheduling: simulated annealing, tabu search, greedy randomized adaptive search procedures, and iterated local search. Metaheuristics to sports scheduling problems use local search methods based on specially tailored neighborhoods. Section "Neighborhoods for Round-Robin Tournaments" shows different neighborhoods defined for round-robin tournament scheduling problems. The chapter concludes with the presentation of a non-exhaustive collection of applications of metaheuristics to problems in sports scheduling.

Exact and Approximate Optimization Methods

Combinatorial optimization, which includes sports scheduling problems as one of its application areas, consists in finding optimal solutions to problems defined over a discrete set of feasible solutions.

Any combinatorial optimization problem may be formulated as the constrained minimization (resp. maximization) of some function $f(S) : S \in F$. Here, E is a discrete ground set formed by the elements that form the problem solutions, $F \subseteq 2^E$ is the set of feasible solutions, and $f : 2^E \rightarrow \mathbb{R}$ is the objective function. Any solution S for this problem is formed by a subset of the elements of the ground set. We seek an *optimal solution* (or, simply, an *optimum*) $S^* \in F$ such that $f(S^*) \leq$

$f(S)$ (resp. $f(S^*) \geq f(S)$), for all $S \in F$. The ground set E, the objective function f, and the feasible set F are specific to each problem instance. Some scheduling problems are feasibility problems, that is, there is no objective function, and the problem is to find a solution that satisfies the restrictions imposed by F. In this case, one may relax a subset of the constraints that define F and consider the minimization of the number of violations of relaxed constraints as the objective function of the problem.

A method for solving an optimization problem is *exact* if it is guaranteed to produce, in finite time, a global optimum for this problem and a proof of its optimality, in the event that one exists or otherwise to show that no feasible solution exists. Globally optimal (or, simply, optimal) solutions are often referred to as exact solutions.

Efficient (i.e., running in polynomial time with respect to the instance size) exact algorithms are unknown (and are unlikely to exist) for a broad class of optimization problems classified as NP-hard. These problems are often referred to as *intractable*. Although the size of the problems that can be solved exactly to optimality has been continuously increasing due to algorithmic and technological developments, there are problems (or problem instances) that are not amenable to being solved by exact methods. Other approaches are needed based on different paradigms to tackle such challenging and large-size optimization problems.

Opposed to exact methods, *approximate methods* or *heuristics* provide feasible solutions that are not necessarily optimal. Approximate methods usually run faster than exact methods. They are capable of handling larger problem instances than exact methods.

Constructive heuristics build a feasible solution from scratch, like the circle method originally proposed by Kirkman [58] to construct a feasible round-robin schedule (see also Ribeiro et al. [104]) that constructs a canonical schedule for SRR tournaments or methods derived from the proof of Vizing's theorem [120] that are able to construct any schedule for SRR tournaments.

Local search methods start from a feasible solution and improve it by successive minor modifications until a solution that cannot be further improved is encountered. Although they often provide high-quality solutions whose values are close to those of the optimal solutions, they can become prematurely trapped in low-quality, locally optimal solutions in some situations.

Metaheuristics are based on distinct paradigms and offer different mechanisms to escape from locally optimal solutions (as opposed to greedy algorithms or local search methods). They are among the most effective solution strategies for solving challenging combinatorial optimization problems in practice. They often produce much better solutions than those obtained by the simple heuristics and rules they involve. The section "Fundamentals of Metaheuristics for Sports Scheduling" recalls the principles and templates of some of the metaheuristics used in sports scheduling, which have been instrumental and contributed to most developments and applications in the field.

Elements of Graph Theory

This section introduces the main graph theory definitions and results essential for understanding the models and neighborhoods presented in this chapter.

A graph $G = (V, E)$ consists of a set V of nodes and a set E of edges. Each edge $e = [u, v]$ is an unordered pair of distinct nodes. The nodes $u, v \in V$ are the endpoints of edge e. An edge $e = [u, v]$ may be given an orientation, say from u to v: in this case, it becomes an arc $a = (u, v)$.

A *simple graph* has neither loops (edges whose endpoints coincide) nor parallel edges (multiple edges between the same pair of endpoints).

The *degree* $d_G(v)$ of a node $v \in V$ in G is the number of edges having v as an endpoint (i.e., the number of edges adjacent to v). A graph $G = (V, E)$ is *complete* if there is an edge in E between every pair of nodes $u, v \in V$, with $u \neq v$. We denote by K_n the complete graph with $n = |V|$ nodes. The graph G is *d-regular* if $d_G(v) = d$ for every node $v \in V$. A *cycle* is a sequence of edges $[u_1, u_2], [u_2, u_3], \ldots, [u_{p-1}, u_p], [u_p, u_1]$, where $u_1, u_2, \ldots, u_{p-1}, u_p$ are nodes of V.

A graph $G = (V, E)$ is *bipartite* if its node set V may be partitioned into two subsets V_1 and V_2 such that no edge in E has both endpoints in V_1 or in V_2. If every possible edge between V_1 and V_2 exists in a bipartite graph G, then G is a *complete bipartite* graph. The complete bipartite graph with n nodes in one subset and m nodes in the other is denoted by $K_{n,m}$. A graph is bipartite if and only if it has no *odd cycle* (i.e., a cycle with an odd number of edges).

A subset $F \subseteq E$ *spans* the graph $G = (V, E)$ if every node $v \in V$ is the endpoint of at least one edge in F. In this case, F is called a *factor* of G. F is an ℓ-factor of G if every node $v \in V$ is an endpoint of exactly ℓ distinct edges in F.

A subset $F \subseteq E$ such that no two edges in F have a common node is called a *matching*. In a *perfect matching* F (also called a *1-regular matching*), each node $v \in V$ is adjacent to exactly one edge in F. We notice that a perfect matching is equivalent to a 1-factor.

A partition of the edge set E into matchings is an *edge coloring* of graph G. If all matchings in an edge coloring are perfect (i.e., 1-factors), then the edge coloring is a *1-factorization*.

The minimum number of matchings in an edge coloring of graph G is its *chromatic index* $\chi'(G)$. Clearly, $\chi'(G) \geq \Delta(G)$, where $\Delta(G)$ is the largest node degree in G.

Two 1-factorizations $\mathscr{F} = \{F_1, \ldots, F_k\}$ and $\mathscr{H} = \{H_1, \ldots, H_k\}$ of G are called *isomorphic* if there exists a bijective function φ from the node set V of G onto itself such that $\{F_1^{\varphi}, \ldots, F_k^{\varphi}\} = \{H_1, \ldots, H_k\}$. Here, F_i^{φ} denotes the set of all edges $[\varphi(x), \varphi(y)]$ where $[x, y]$ is an edge in F_i. The concept of isomorphic 1-factorizations can also be extended to *isomorphic edge colorings*.

A 1-factorization is said to be *perfect* when the edges in $F_i \cup F_j$ form a *Hamiltonian cycle* for each pair of distinct 1-factors F_i and F_j.

Remark 1. In a complete graph K_{2n}, any 1-factorization is equivalent to a minimum cardinality edge coloring. ■

This remark highlights that 1-factorizations are a particular case of edge colorings.

Remark 2. Formally, there is no order between the 1-factors of the 1-factorization $\mathcal{F} = \{F_1, \ldots, F_k\}$ (or between the colors of an edge coloring). In the context of this chapter, we refer to the *ordered 1-factorization* $\mathcal{F} = (F_1, \ldots, F_k)$ as an ordered sequence whenever the order in which the 1-factors (or colors) are taken is relevant. ■

This second remark and the notation defined therein are relevant because, in sports scheduling, the order of the factors of a 1-factorization matters. By analogy, we could also refer to an ordered edge coloring.

Basic Graph Model for Single Round-Robin Tournaments: The Circle Method

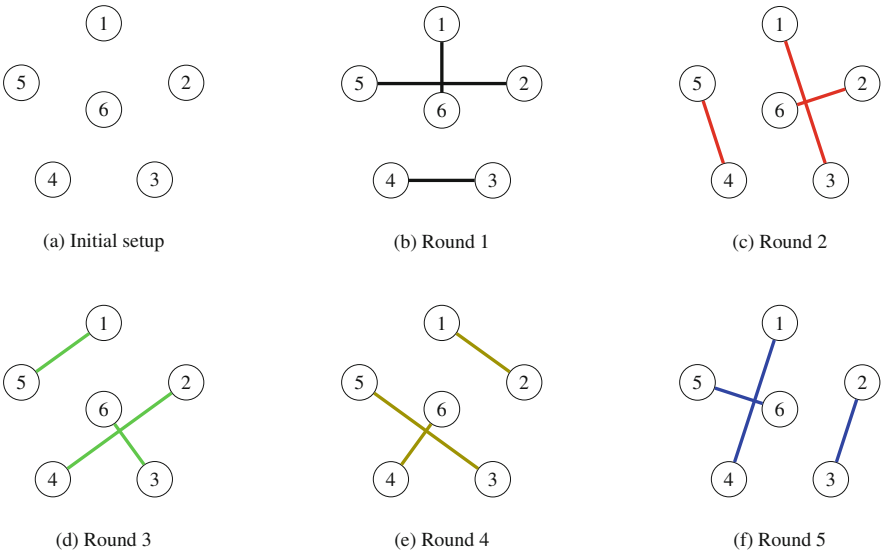
We consider a round-robin tournament between $2n$ teams. It is quite natural to associate the $2n$ teams, numbered $1, \ldots, 2n$, with the nodes of a graph. Each game to be played by teams i and j will be represented by an edge $[i, j]$.

There is a one-to-one correspondence between ordered 1-factorizations and single round-robin timetables. If we have to construct a timetable for the games of a single round-robin tournament, we seek an edge coloring of the edges of a complete graph K_{2n} with $2n$ nodes. Since K_{2n} is $(2n-1)$-regular, any timetable will require at least $2n - 1$ rounds, assuming each team plays at most one game in each round. Indeed, one can always construct a timetable using exactly $2n-1$ rounds (i.e., $\chi'(K_{2n}) = 2n-1$). Let $\mathcal{F} = (F_1, \ldots, F_{2n-1})$ be such an edge coloring where F_i is the 1-factor whose edges correspond to the games played in round $i = 1, \ldots, 2n-1$. $\mathcal{F}$ is indeed a 1-factorization of K_{2n} since each team has to play exactly one of its $2n - 1$ games during each one of the $2n - 1$ rounds of the timetable.

The *circle method* (or *polygon method*) provides a simple construction for the so-called *canonical 1-factorization* $\mathcal{F} = (F_1, \ldots, F_{2n-1})$ of K_{2n}.

We give in Fig. 1 a graphical interpretation of the circle method. We first position the teams 1 to $2n - 1$ in the vertices of a regular polygon. For round 1, we first match the highest team of the polygon (team 1) with that in the center (team $2n$). Next, we pair each team on the left side of the polygon with that on the right side at the same height. This perfect matching defines the games of round 1. We repeat this procedure for rounds $2, 3, \ldots, 2n - 1$, by rotating the teams in the polygon's vertices counterclockwise at each step (or round).

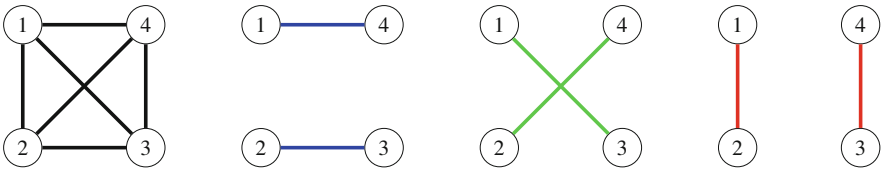
The chronological sequence of the rounds in which the games are scheduled in the 1-factorization $\mathcal{F} = (F_1, \ldots, F_{2n-1})$ induces an order on $F_1, \ldots, F_{2n-1}$.



(a) Initial setup (b) Round 1 (c) Round 2

(d) Round 3 (e) Round 4 (f) Round 5

Fig. 1 Circle method for a tournament with $2n = 6$ teams. (**a**) Initial setup. (**b**) Round 1. (**c**) Round 2. (**d**) Round 3. (**e**) Round 4. (**f**) Round 5



(a) K_4 (b) F_1 (c) F_2 (d) F_3

Fig. 2 Canonical 1-factorization of K_4. (**a**) K_4. (**b**) F_1. (**c**) F_2. (**d**) F_3

However, we shall refer to 1-factorizations without mentioning the order whenever no confusion may arise.

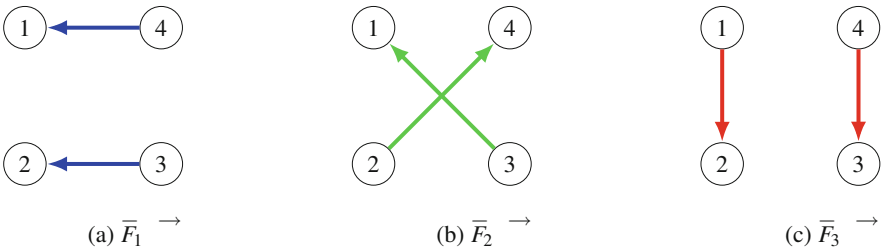
The circle method is frequently used to provide initial solutions to local search-based heuristics. Figure 2 shows a canonical 1-factorization $\mathscr{F} = (F_1, F_2, F_3)$ of K_4. In the first round, team 4 plays a game with team 1, while team 2 plays against team 3; then $F_1 = \{[4, 1], [2, 3]\}$. Similarly, $F_2 = \{[4, 2], [3, 1]\}$, and $F_3 = \{[4, 3], [1, 2]\}$.

Table 1 represents the timetable corresponding to the canonical 1-factorization of Fig. 2.

It is often the case that each team has its venue (or stadium) and each game between teams i and j is played either in the venue of team i or the venue of team j. In the first case, we say that team i plays a *home game* and that team j plays an *away game*. It is convenient to represent this game by an arc (j, i) oriented from j to i.

Table 1 Timetable corresponding to the canonical 1-factorization of Fig. 2

Rounds		
1	2	3
[4, 1]	[4, 2]	[4, 3]
[2, 3]	[3, 1]	[1, 2]



(a) $\overrightarrow{F}_1$ (b) $\overrightarrow{F}_2$ (c) $\overrightarrow{F}_3$

Fig. 3 Oriented edge coloring for the edge coloring of Fig. 2. (**a**) $\overrightarrow{F}_1$. (**b**) $\overrightarrow{F}_2$. (**c**) $\overrightarrow{F}_3$

In the second case, if the game between teams i and j is played in the home city of team j, we have an arc (i, j) oriented from i to j.

If every game in a single round-robin tournament is assigned to a venue, then each edge of K_{2n} receives an orientation and becomes an arc. Instead of an edge coloring, we now have an *oriented edge coloring*, denoted by $\overrightarrow{\mathscr{F}} = (\overrightarrow{F}_1, \ldots, \overrightarrow{F}_{2n-1})$. For the timetable associated with K_4 in Table 1, the edges may be oriented, e.g., as

$$\overrightarrow{F}_1 = \{(4, 1), (3, 2)\},\ \overrightarrow{F}_2 = \{(2, 4), (3, 1)\},\ \text{and}\ \overrightarrow{F}_3 = \{(4, 3), (1, 2)\},$$

as represented in Fig. 3.

Local Search and Neighborhoods

A feasible solution satisfies all constraints of the problem at hand. We recall that F is the set of feasible solutions for the combinatorial optimization problem. We denote by $\hat{F} = 2^E$ the set formed by all subsets of the ground set, which includes all feasible and infeasible solutions.

A local search heuristic starts from any feasible solution and improves it by successive minor modifications that visit other (feasible or infeasible) solutions until a solution that cannot be further improved is encountered. Local improvements are evaluated concerning neighbor solutions that can be obtained by slight modifications applied to the current solution.

A *neighborhood* of a solution $S \in F$ can be defined as any subset of F. More formally, a neighborhood is a mapping that associates each feasible solution $S \in F$ with a subset $N(S) = \{S_1, \ldots, S_p\} \subseteq F$ of feasible solutions.

Each solution $S' \in N(S)$ can be reached from S by an operator called a *move*. Typically, two neighboring solutions S and $S' \in N(S)$ differ only by a few elements

of the ground set, and a move from S to S' consists simply of the modification in one or more elements in S. Usually, $S \in N(S')$ if only if $S' \in N(S)$.

The *search space* (or *solution space* or *search graph*) of a given problem for some neighborhood N may be seen as a graph $\mathscr{G} = (F, M)$ whose node set corresponds to its set F of feasible solutions. Its edge-set M is such that there is an edge $(S, S') \in M$ between two solutions $S, S' \in F$ if and only if $S' \in N(S)$ and $S \in N(S')$. An extended search graph may be similarly defined, encompassing not only the set of feasible solutions F but, instead, the whole set $\hat{F} = 2^E$ formed by all subsets of elements of the ground set (feasible or infeasible).

Different neighborhoods can be defined and used to implement a local search method. Depending on the neighborhood used, the search space may be disconnected. This situation can lead to implementation difficulties and can even make it impossible for the search procedure to find reasonable solutions located in the part of the search space graph that cannot be reached from the initial solution. From another point of view, the larger the neighborhood, the denser the search space and the shorter the paths connecting any two solutions. On the other hand, larger neighborhoods require the evaluation of more neighboring solutions, leading to higher computation times.

It is noticeable that the search space does not need to be formed exclusively by feasible solutions in F. It can be enlarged to contain also infeasible solutions. In this situation, a path in the search space can visit feasible and infeasible solutions, but, in any case, it must terminate in a feasible solution.

Finding an appropriate neighborhood and the best way to explore it is a crucial step toward implementing effective and efficient local search methods and metaheuristics. Neighborhoods for round-robin scheduling problems will be described and compared in detail in the section "Neighborhoods for Round-Robin Tournaments."

Yagiura and Ibaraki [129] traced the history of local search since the work of [17]. Hoos and Stützle [50] developed a thorough study of the foundations and applications of stochastic local search, methods based on local search that use randomization to generate or select candidate solutions for combinatorial optimization problems. In their seminal work, Lin and Kernighan [70] developed a local search heuristic based on 2-opt and 3-opt exchanges for approximately solving the symmetric traveling salesman problem, which is one of the best approaches for the problem. Michiels et al. [77] discussed theoretical aspects of local search. Chapter 4 of the book by Resende and Ribeiro [95] describes enhancements in the implementation of local search methods, such as candidate lists in best-improving strategies and circular search in first-improving strategies; see also [36].

Fundamentals of Metaheuristics for Sports Scheduling

This section reviews the basic principles and ideas involved in the design of some of the main metaheuristics. We describe four widely and successfully used metaheuristics: simulated annealing, tabu search, greedy randomized adaptive

search procedures (GRASP), and iterated local search (ILS). The reader may check the broad and ever-evolving literature on the subject, in particular, the handbooks edited by Reeves [85], Glover and Kochenberger [35], Burke and Kendall [9, 10], and Gendreau and Potvin [30].

Simulated Annealing

The principle of simulated annealing lies in the analogy between the physical annealing process and the combinatorial optimization problem to be solved. In physics, annealing is a thermal process for obtaining the low-energy states of a solid in a heat bath. Solutions to the combinatorial problem are equivalent to states of the physical process, while the cost of each solution corresponds to the state's energy. Neighbor solutions correspond to states generated by a perturbation method. At each step of the simulated annealing, a new solution in the neighborhood of the current solution is randomly generated. The new solution replaces the current solution if it improves the latter or with a decreasing probability if it does not. The best-encountered solution is returned. It is a memoryless procedure that does not require a local search procedure.

The main advantage of simulated annealing is its simplicity of implementation because there is no local search. However, its convergence speed and solution quality depend on several parameter values and implementation choices that are not easy to tune. Furthermore, the algorithm is memoryless and does not use cost information about the entire neighborhood.

The work on optimization by simulated annealing was pioneered by Kirkpatrick et al. [59], with accounts of later developments and applications in textbooks [1,63]. Convergence and implementation strategies are also discussed in [2,45].

Tabu Search

Tabu search is a dynamic neighborhood method that uses memory to drive the search by escaping from local optima and avoiding cycling. Contrarily to memoryless heuristics such as simulated annealing, and to methods that use rigid memory structures such as branch-and-bound, tabu search makes use of flexible and adaptive memory designs.

For any solution S, in the case of tabu search, the neighborhood $N(S)$ is not a static set and can be modified according to the history of the search. At each local search iteration, tabu search looks for the neighbor solution that best improves the objective function (or, if none exists, for the one that least deteriorates the current solution). However, some moves to neighbors in $N(S)$ are forbidden and discarded. The set of forbidden (or *tabu*) moves is stored in a short-term memory, formed by the most recently performed moves, to prevent the search from returning to previously visited solutions. Contrary to simple local search procedures, tabu search chooses the move that least deteriorates the objective function value whenever there are no improving moves.

The basic tabu search scheme can be extended by incorporating more sophisticated strategies: aspiration criteria to override the tabu status of unvisited solutions, medium-term and long-term memories to implement intensification and diversification procedures, candidate list strategies to speed up the search, and hashing tables to accelerate and filter the search. These comprehensive strategies lead to much more powerful and effective implementations in terms of the quality of the solutions they obtain.

Tabu search is undoubtedly among the most effective approaches for solving hard combinatorial optimization problems. However, implementations of tabu search often involve setting many parameters, which have to be appropriately tuned for achieving good performance in practice.

The seminal papers of Glover [31, 32] established the fundamentals, extensions, and uses of tabu search. They provided solid foundations and originated most of the developments from where the field of metaheuristics flourished, as reported in the retrospective by Glover [34]. The reader is referred to the textbook of Glover and Laguna [36] for a thorough study of tabu search and related ideas. Glover et al. [37] gave an updated presentation of the fundamental ideas of tabu search with some hints for an efficient implementation. A variety of applications of tabu search with basic modeling suggestions were described by Hertz et al. [47].

Greedy Randomized Adaptive Search Procedures

A greedy randomized adaptive search procedure (GRASP) is a metaheuristic resulting from hybridizing a greedy randomized algorithm with a local search method embedded in a multi-start framework.

An especially appealing characteristic of GRASP is its ease of implementation. Few parameters must be set and tuned; therefore, development can focus on implementing efficient data structures to ensure fast computations in all iterations. Basic implementations of GRASP rely almost exclusively on two parameters. The first is the stopping criterion, often defined by the maximum number of iterations (or by the maximum number of iterations without improvement in the best solution found). The second is a parameter used to limit the size of the restricted candidate list within the greedy randomized algorithm used in the construction phase. GRASP is also amenable to efficient and straightforward parallel implementations (see, e.g., Ribeiro and Rosseti [98, 99]). Despite its simplicity and ease of implementation, GRASP is a very effective metaheuristic and produces the best known solutions for many problems.

GRASP was proposed by Feo and Resende [25] (see also [26]). Reviews and tutorials appeared in Ribeiro [97] and Resende and Ribeiro [90, 91, 92, 93]. Festa and Resende [27, 28] presented reviews of applications of GRASP. The book by Resende and Ribeiro [95] consolidated the fundamentals, advances, and extensions of optimization by GRASP.

GRASP, as initially proposed, is a memoryless procedure in which each iteration does not use information gathered in previous iterations. In other words, all its iterations are independent. Path relinking enhances the basic GRASP significantly,

adding a long-term memory mechanism to GRASP heuristics. GRASP with path relinking implements long-term memory using an elite set of diverse and high-quality solutions previously found during the search. In its most basic implementation, path relinking is applied at each iteration between the solution found at the end of the local search phase and any randomly selected solution from the elite set. The solution resulting from path relinking is a candidate for inclusion in the elite set. The hybridization of GRASP with path relinking can significantly improve solution time and quality.

Path relinking was initially proposed by Glover [33] as an intensification strategy exploring trajectories connecting elite solutions obtained by tabu search or scatter search. The use of path relinking embedded in a GRASP procedure as an intensification strategy applied to each locally optimal solution was first proposed in [64]. Several strategies (backward, forward, backward-and-forward, mixed, truncated, randomized, evolutionary) have been considered and combined with extensions and improvements of path relinking in successful implementations hybridized with tabu search, GRASP, and genetic algorithms (see, e.g., [11, 29, 75, 88–91, 96, 98, 101–103, 108]). Resende and Ribeiro [91] reviewed these strategies, showing that they involve trade-offs between computation time and solution quality. Chapters 8 and 9 of Resende and Ribeiro [95] reviewed path relinking strategies and hybridizations with GRASP in detail.

Iterated Local Search

Iterated local search (ILS) explores the solution space using a walk that moves from a locally optimal solution to a nearby one without the restriction of using exclusively moves in the neighborhood. Local search is applied to the initial solution. The main loop of the algorithm is performed until some stopping criterion is reached. Each iteration starts by applying a random perturbation to the current locally optimal solution, which results in an intermediate solution. Local search is again applied to the latter, resulting in a new local optimum. If this local optimum meets some acceptance criterion, it becomes the next solution of the solution trajectory. Otherwise, the algorithm remains in the same local optimum, and a new iteration resumes. A maximum number of perturbations is typically used as the stopping criterion.

Iterated local search was proposed in Martin et al. [74] and Martin and Otto [73] (see [72] for a review). Most of the work using ILS has been done with Markovian random walks, although some studies show that incorporating memory enhances its performance. ILS may also be hybridized with other metaheuristics such as GRASP.

Variable Neighborhood Search

Variable neighborhood search (VNS) is based on a systematic change in the neighborhood. It makes use of a finite set of $k_{\max}$ pre-selected neighborhood

structures identified as $N_1, \ldots, N_{k_{\max}}$ that may be induced by one or more metric functions introduced in the solution space. Let $N_k(S)$ denote the set of solutions in the k-th neighborhood of S.

At each VNS iteration, a solution is randomly sampled in the k-th neighborhood of the current solution S, and a local search is applied to the resulting solution. If the local minimum obtained does not improve the current solution, then the algorithm moves to the $(k+1)$-th neighborhood.

VNS is also a memoryless heuristic. It is easy to implement and relies on a few parameters: the stopping criterion and the number $k_{\max}$ of neighborhoods (and their definitions). It was originally proposed by Mladenovic and Hansen [78] (see [43,44] for reviews).

Genetic Algorithms

A genetic algorithm is a population-based method that uses selection, recombination, and mutation operators to generate new solutions in the search space, imitating the process of natural selection and evolution of species. Solutions are evaluated in terms of their fitness, which most often corresponds to the value of the objective function itself.

Contrary to the other metaheuristics reviewed in the previous sections, genetic algorithms explore a population of solutions that evolve along some generations, rather than a single trajectory emanating from a unique initial solution. The method starts from a population formed by a fixed number of feasible solutions that can be randomly generated or constructed by a more elaborate approach such as a greedy randomized algorithm. At each iteration (which is called a generation in this context), pairs of randomly selected solutions (parent solutions) are combined, and new solutions (offspring) are generated (crossover operator). Some randomly selected solutions go through small random modifications in their structure (mutation operator) to ensure diversification. The best-fitted solutions of the population are selected and passed to the new generation (selection operator), and a new iteration resumes.

Hybrid genetic algorithms, also called memetic algorithms, use specific knowledge available to solve the problem. Such strategies incorporate greedy randomized algorithms to create the initial population and the application of local search to some elements of the population of each generation. Genetic algorithms hybridized with optimization strategies usually perform much better than basic genetic algorithms.

Genetic algorithms were first presented in the book of Holland [48] (see also, e.g., Goldberg [39], Holland [49], Reeves and Rowe [87], and Michalewicz [76], among others). They are very appealing and easy to implement. However, many implementation choices exist for each of the above operators. Exemplary implementations require appropriate solution encodings as finite-length data strings and well-tuned strategies for the initial population generation, mate selection, crossover, mutation, and selection (see, e.g., [86]).

A significant difficulty with implementing genetic algorithms resides in the crossover operator. It occurs because, in most cases, the solutions obtained by crossover turn out to be infeasible and, therefore, unusable by the algorithm. Although some strategies have been proposed in the literature to handle infeasibilities (such as the use of penalties or repair operators), they are problem-specific and of complex generalized implementation in practice, with a limited number of successful cases.

A biased random-key genetic algorithm (BRKGA) encodes solutions as a vector of real numbers called random keys. A deterministic algorithm (called a decoder) takes a vector of random keys as the input and associates with it a feasible solution of the combinatorial optimization problem at hand, for which an objective value (or fitness function) can be computed. The crossover is said to be biased because one parent is always an elite (high-quality) solution, while the other is not. The elite parent still has a higher probability of passing its elements to the new generation. Biased random-key genetic algorithms can be implemented straightforwardly. The crossover is performed in the space of random keys, and the corresponding solutions are built by the decoder a posteriori. Therefore, the resulting solutions are always feasible. BRKGAs are particularly effective for scheduling problems or other problems whose solutions may be represented by permutation vectors (see Goncalves and Resende [40], Londe et al. [71], Noronha and Ribeiro [80], and Resende and Ribeiro [94] for reviews and [6–8, 81, 83] for some applications).

Neighborhoods for Round-Robin Tournaments

We noticed in section "Local Search and Neighborhoods" that different neighborhoods can be defined and imposed on the set of feasible solutions of a combinatorial optimization problem.

Local search is a common component to many metaheuristics. Local search methods embedded in metaheuristics may generate neighbors randomly, as in the case of simulated annealing. They may also explore solution neighborhoods using first-improving or best-improving strategies, as in the cases of tabu search or GRASP.

Solutions for single round-robin tournament scheduling problems will be represented by oriented ordered 1-factorizations of the complete graph K_{2n} for a league with $2n$ teams, as described in the section "Basic Graph Model for Single Round-Robin Tournaments: The Circle Method." This representation may also be used for problems associated with mirrored double round-robin scheduling tournaments. With this representation, each 1-factor is associated with a different color, which is used to color all edges of this 1-factor.

Different neighborhood structures have been used in the literature of local search procedures for scheduling time-constrained single round-robin sports tournaments, although sometimes they have been referred to by different names. We name them as *Home-Away Swap, Team Swap, Partial Team Swap, Round Swap, Partial Round Swap,* and *Generalized Partial Team Swap*. Edge orientations are disregarded for all

neighborhoods except Home-Away Swap. These neighborhoods may be extended to the case of double round-robin problems. They are described in the following sections, using the notation and definitions introduced in the section "Elements of Graph Theory."

Neighborhood "Home-Away Swap" (HAS)

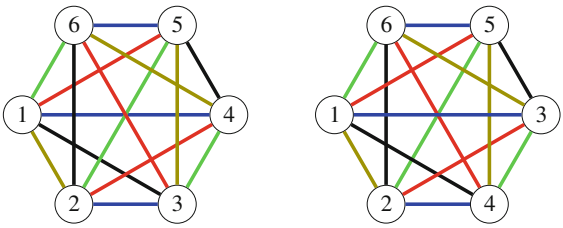
Moves in this neighborhood operate only on home-away patterns, leaving the timetable unchanged. The operator that defines the neighborhood consists of taking an arc of the oriented 1-factorization and reversing its orientation. In practice, it corresponds to exchanging the venue of the game between the two teams associated with the endpoints of this arc (see the representation illustrated in Fig. 3.).

Neighborhood "Team Swap" (TS)

Each move in this neighborhood is obtained by swapping two distinct nodes of the graph. After a TS move is applied to two nodes $v \neq w$, for every node $x \neq v$ and $x \neq w$, the arc between x and v gets the color previously used on the arc between x and w, and vice-versa. In practice, it corresponds to exchanging the opponents of the teams associated with nodes v and w in all tournament rounds. Figure 4 illustrates the application of a move in neighborhood TS to teams 3 and 4 for a schedule with six teams.

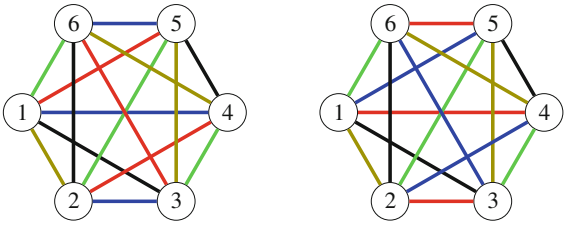
Neighborhood "Round Swap" (RS)

Moves in this neighborhood pick two distinct colors, j and k, and exchange them in all edges colored with any of these two colors. In other words, we swap the two 1-factors F_j and F_k associated with the colors j and k in the ordered 1-factorization.



(a) Current solution (b) Solution after move TS

Fig. 4 Graph on the left illustrates a 1-factorization of K_6. On the right, we see the 1-factorization obtained by a team swap move applied to teams (nodes) 3 and 4. (**a**) Current solution. (**b**) Solution after move TS



(a) Current solution (b) Solution after move RS

Fig. 5 Graph on the left illustrates a 1-factorization of K_6. On the right, we see the 1-factorization obtained by a round swap move applied to the colors red and blue. (**a**) Current solution. (**b**) Solution after move RS

In practice, it corresponds to swapping the two rounds j and k in the tournament sequence. Figure 5 illustrates the application of a move in neighborhood RS to the colors red and blue for a schedule with six teams.

Neighborhood "Partial Round Swap" (PRS)

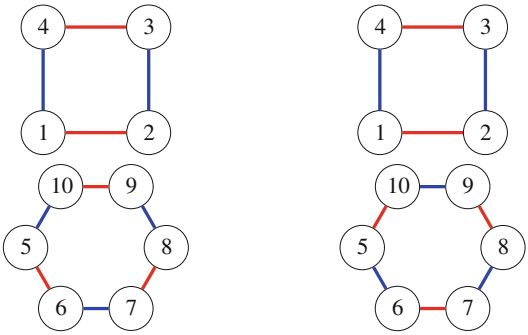
For moves in this neighborhood, we select two distinct colors and consider any cycle C in the subgraph formed by the edges colored with these colors. A neighbor is obtained by exchanging the colors in this cycle. If the edges of this cycle are incident to all vertices of the graph, i.e., if they form a Hamiltonian cycle, then this move is equivalent to a move in the RS neighborhood. Consequently, PRS and PS are equivalent in the case of perfect 1-factorizations. In practice, it corresponds to picking two rounds with a subset formed by an even number of teams that play exclusively among themselves in these two rounds. Figure 6 illustrates the application of a move in the PRS neighborhood.

A move in the neighborhood RS is equivalent to a sequence of moves in the neighborhood PRS.

Neighborhood "Partial Team Swap" (PTS)

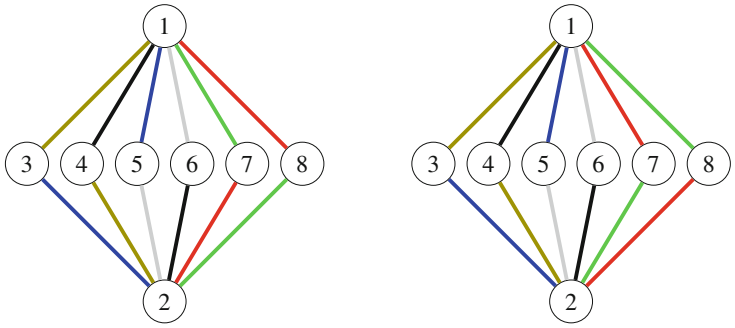
A move in this neighborhood takes two nodes v_1 and v_2 and a collection of chains $C = \{C_1, \ldots, C_p\}$ of length two between them. The set $\mathscr{S}$ of colors assigned to the edges of the chains in C adjacent to v_1 should be the same as the set of colors assigned to the edges of the chains in C adjacent to v_2. We choose $\mathscr{S}$ to be minimal, i.e., no proper subset of C satisfies this same condition. The edge-colored subgraph composed by v_1 and v_2 and the chains in C are called a *colorful chordless lantern*. Then, the two colors of each chain are exchanged.

If the number of colors occurring in these chains is always $2n - 2$ for any picked pair of nodes v_1 and v_2, then each PTS move is equivalent to a TS move. Such



(a) Current solution (b) Solution after move PRS

Fig. 6 Subgraph on the left is induced by edges with colors red and blue in a 1-factorization of K_{10}. On the right, we see the same subgraph after a partial round swap move has been applied by swapping the assignments of colors red and blue in the lower cycle. (**a**) Current solution. (**b**) Solution after move PRS



(a) Current solution (b) Solution after move PTS

Fig. 7 Subgraph on the left is obtained from a 1-factorization of K_8. On the right, we see the same subgraph after a partial team swap move with chains $C_1 =< v_1 = 1, 7, v_2 = 2 >$ and $C_2 =< v_1 = 1, 8, v_2 = 2 >$. (**a**) Current solution. (**b**) Solution after move PTS

1-factorizations have been named *L-blocking* in the literature because they block the possibility of obtaining a non-isomorphic 1-factorization by exchanging the colors of a colorful chordless lantern.

This move corresponds to picking the two teams corresponding to nodes v_1 and v_2 and a minimal subset of rounds in which both of them face the same opponents and then exchanging the opponents of v_1 and v_2 in each of these rounds. Figure 7 illustrates the application of a move in the PTS neighborhood with nodes $v_1 = 1$ and $v_2 = 2$.

A move in the neighborhood TS is equivalent to a sequence of moves in the neighborhood PTS.

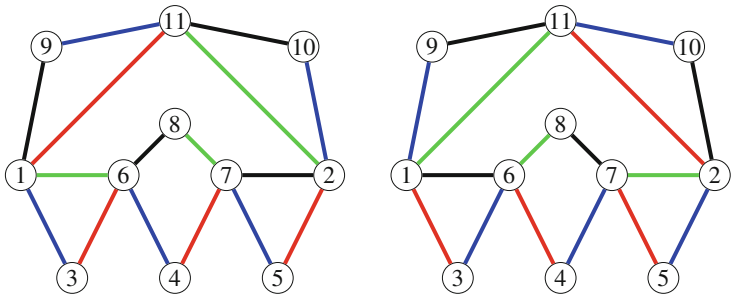
Neighborhood "Generalized Partial Team Swap" (GPTS)

This neighborhood involves two vertices v_1 and v_2 and a set of edge-disjoint alternating chains $\{C_1, \ldots, C_p\}$ of even length. We observe that GPTS generalizes both PRS and PTS. It generalizes PTS by allowing the chains to be longer than two. To see that it generalizes PRS, take v_1 and v_2 as two arbitrary nodes at even distance in the cycle C, and then the latter can be seen as two alternating chains of even length between v_1 and v_2. Figure 8 illustrates the application of a move in the GPTS neighborhood with nodes $v_1 = 1$ and $v_2 = 2$.

Considerations About the Neighborhoods

Several authors have used neighborhoods home-away swap (HAS), round swap (RS), team swap (TS), partial team swap (PTS), and partial round swap (PRS), although sometimes with different definitions, equivalent or not. Anagnostopoulos et al. [3] used moves in neighborhoods HAS, RS, TS, PRS, and PTS in their simulated annealing algorithm for the traveling tournament problem. Ribeiro and Urrutia [100] used moves in neighborhoods HAS, RS, and TS and in a restricted PRS in the local search of their hybrid heuristic for the mirrored traveling tournament problem. They also used ejection chain moves named game rotations as perturbations that are proved to be equivalent to PTS moves in Ribeiro et al. [104].

Guedes and Ribeiro [41] used moves in neighborhoods TS, RS, PRS, and PTS, as well as game rotation, row destruction, and column destruction perturbations in their hybrid ILS with multi-start heuristic for the minimum weighted carry-over problem. Costa et al. [14] used neighborhoods PRS and PTS in the local search of their ILS heuristic for the traveling tournament problem with fixed venues. They also showed that a move in the neighborhood TS (resp. RS) is equivalent to a sequence



(a) Current solution (b) Solution after move GPTS

Fig. 8 Subgraph on the left is obtained from a 1-factorization of K_{12}. On the right, we see the same subgraph after a generalized partial team swap move involving four alternating chains between nodes $v_1 = 1$ and $v_2 = 2$ (not all nodes of K_{12} appear in the figure). (**a**) Current solution. (**b**) Solution after move GPTS

of moves in the neighborhood PTS (resp. PRS). Di Gaspero and Schaerf [19] in their composite-neighborhood tabu search approach for the traveling tournament problem used neighborhoods HAS, TS, RS, PTS, and PRS. The last two are seen as repair chains, similar to ejection chains. Rosati et al. [105] also used neighborhoods TS, RS, PTS, and PRS in their multi-neighborhood simulated annealing for the International Timetabling Competition 2021 [115], together with an additional "partial swap teams phased" neighborhood. This neighborhood consists of the application of moves in the PTS neighborhood limited to one of the two SRR schedules of a double round-robin tournament organized in two separate phases (i.e., a phased DRR tournament).

Januario and Urrutia [51] used a neighborhood that can be seen as a particular case of GTPS, in which only one chain is allowed to have an arbitrary even length, while the others have length two. Januario et al. [52] unified the nomenclature by describing all the neighborhoods in terms of edge colorings of a graph.

Connectivity of the Tournament Space

Let $\mathscr{G} = (F, M)$ be the search space graph associated with some round-robin tournament scheduling problem. The node set F is formed by all ordered oriented 1-factorizations of K_{2n}, where $2n$ is the number of teams. Given a neighborhood structure $\mathscr{N}$, two nodes $u, v \in F$ are adjacent, i.e., $[u, v] \in M$, if and only if $u \in \mathscr{N}(v)$ and $v \in \mathscr{N}(u)$. The question arises to determine whether $\mathscr{G}$ is connected or not. If not, starting from some initial solution and applying a local search method, it might be impossible to reach some solutions because they are in different connected components of $\mathscr{G}$.

The size of the search space graph $\mathscr{G} = (F, M)$ of single round-robin tournament scheduling problems is huge even when compared with those of other classical combinatorial optimization problems, such as the traveling salesman problem. There is no known formula for the number of different 1-factorizations of K_{2n}. The number of non-isomorphic 1-factorizations of K_{2n} increases rapidly with $2n$. There are 526,915,620 non-isomorphic 1-factorizations of K_{12} [20] and 1,132,835,421,602,062,347 of K_{14} [56]. The number of single round-robin schedules is even much higher since for each of these non-isomorphic 1-factorizations, there are different round permutations, different home-away assignments for each game, and different teams' assignments to nodes.

In a perfect 1-factorization, the subgraph containing all vertices and edges belonging to the union of any two 1-factors is a Hamiltonian cycle. A perfect 1-factorization may not be desirable as the initial solution for local search algorithms for tournament scheduling problems because every move in the PRS neighborhood has the same effect as a move in the RS neighborhood, generating colorings that are isomorphic to the original one.

Kobayashi [61] showed that the values of $2n$ for which the canonical factorization is perfect are precisely the values of $p + 1$ for which $p > 2$ is prime. Januario et al. [53] showed that the values of $2n$ for which the canonical 1-factorization is

perfect and the PTS and TS neighborhoods coincide are the same for which a riffle permutation permutes all except the first and last of $2n$ distinct integers. Urrutia et al. [112] coined the term colorful chordless lantern and studied different properties of L-blocking 1-factorizations. They also investigated K-blocking 1-factorizations that do not contain an optimally colored even clique whose cardinality is smaller than the number of nodes.

The circle method is a simple strategy to obtain initial solutions for local search heuristics for single round-robin tournament scheduling problems. However, it generates only one type of 1-factorizations for K_{2n}, the canonical 1-factorization. It generates perfect 1-factorizations for every complete graph K_{2n} for which $2n = p + 1$, where p is a prime number. Therefore, whenever an initial solution is generated by the circle method, a move in the PRS neighborhood is equivalent to a move in the RS neighborhood. Consequently, these neighborhoods always lead to 1-factorizations that are isomorphic to the canonical one.

We may argue if the PTS neighborhood may be used to make the search escape from canonical 1-factorizations. In some cases, this neighborhood suffers from a similar drawback. PTS moves are equivalent to TS moves when there are $2n - 2$ chains of length two in the set $C = \{C_1, \ldots, C_p\}$ computed as explained in section "Neighborhoods for Round-Robin Tournaments." We could still ask if the neighborhoods PTS and TS are equivalent for some values of $2n$ for which the canonical 1-factorization is perfect. In order to provide an answer to this conjecture, we applied an exhaustive search procedure to check if that was the case for each value of $2n = p + 1 \leq 100$, where p is prime. We found that the PTS neighborhood is equivalent to the TS neighborhood for canonical 1-factorizations of K_{2n} for $2n =$ 4, 6, 12, 14, 20, 30, 38, 54, 60, 62, 68, and 84.

These results show that, for all the above values of $2n$, any local search heuristic using an initial solution constructed by the circle method will be trapped in a small portion of the search space formed by all 1-factorizations isomorphic to the canonical one.

In such cases, applying other constructive algorithms instead of the circle method, or using the GPTS neighborhood, might improve the effectiveness of the local search by exploring a broader portion of the search space.

An experiment using the circle method and the Vizing's algorithm [14] to build initial solutions for the traveling tournament problem with predefined venues was performed on instances with 18 and 20 teams. No significant improvements were observed for the instances with 18 teams when Vizing's algorithm was applied because, in this case, neighborhood PTS allows to escape from the canonical 1-factorization. However, for the instances with 20 teams, an improvement of almost 19% on average in the cost function was obtained when Vizing's algorithm replaced the circle method for constructing the initial solution. This experiment revealed that the coloring method based on Vizing's theorem could yield better results than canonical 1-factorizations.

It is not known whether neighborhood GPTS is connected or not. However, the example depicted in Tables 2 and 3 and Figs. 9 and 10 shows that the neighborhood GPTS can connect the canonical 1-factorization of K_{12} with a non-isomorphic

Table 2 Perfect 1-factorization of K_{12} for which neighborhoods PTS and TS are equivalent

$F_1 = \{(1, 12), (2, 11), (3, 10), (4, 9), (5, 8), (6, 7)\}$
$F_2 = \{(1, 10), (2, 9), (3, 8), (4, 7), (5, 6), (11, 12)\}$
$F_3 = \{(1, 8), (2, 7), (3, 6), (4, 5), (9, 11), (10, 12)\}$
$F_4 = \{(1, 6), (2, 5), (3, 4), (7, 11), (8, 10), (9, 12)\}$
$F_5 = \{(1, 4), (2, 3), (5, 11), (6, 10), (7, 9), (8, 12)\}$
$F_6 = \{(1, 2), (3, 11), (4, 10), (5, 9), (6, 8), (7, 12)\}$
$F_7 = \{(1, 11), (2, 10), (3, 9), (4, 8), (5, 7), (6, 12)\}$
$F_8 = \{(1, 9), (2, 8), (3, 7), (4, 6), (5, 12), (10, 11)\}$
$F_9 = \{(1, 7), (2, 6), (3, 5), (4, 12), (8, 11), (9, 10)\}$
$F_{10} = \{(1, 5), (2, 4), (3, 12), (6, 11), (7, 10), (8, 9)\}$
$F_{11} = \{(1, 3), (2, 12), (4, 11), (5, 10), (6, 9), (7, 8)\}$

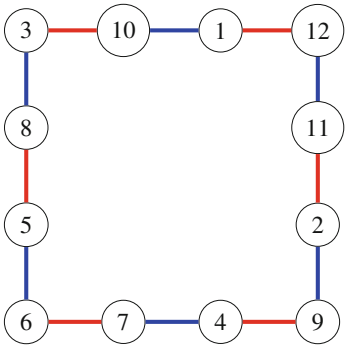


Fig. 9 Hamiltonian cycle built using the first (edges in red) and second (edges in blue) factors of the perfect 1-factorization of K_{12} in Table 2

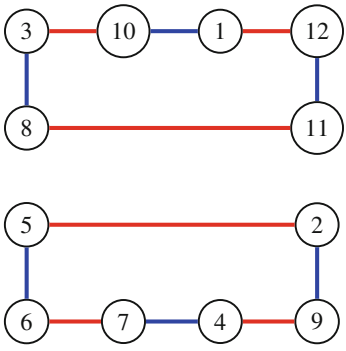


Fig. 10 Two cycles formed by the first (edges in red) and second (edges in blue) factors of the 1-factorization of K_{12} in Table 3

1-factorization that is not perfect. Table 2 illustrates a perfect 1-factorization of K_{12} for which neighborhoods PTS and TS are equivalent and gives its 11 1-factors.

Figure 9 shows the Hamiltonian cycle built using factors F_1 (edges in red) and F_2 (edges in blue) of the perfect 1-factorization of K_{12} in Table 2. Figure 10 shows the two resulting factors F'_1 and F'_2 that form two distinct cycles after a move in neighborhood GPTS. The resulting 1-factorization appears in Table 3. The edges in bold in some factor F'_a are those that previously belonged to a factor F_b, with $a \neq b$. Thus, neighborhood GPTS can connect the canonical 1-factorization of

Table 3 1-factorization of K_{12} obtained after the move in neighborhood GPTS. This 1-factorization is no longer perfect (edges in bold in some factor F'_a are those that previously belonged to a factor F_b, with $a \neq b$)

$F'_1 = \{(1, 12), \mathbf{(2, 5)}, (3, 10), (4, 9), (6, 7), \mathbf{(8,11)}\}$
$F'_2 = \{(1, 10), (2, 9), (3, 8), (4, 7), (5, 6), (11, 12)\}$
$F'_3 = \{\mathbf{(1, 6)}, (2, 7), \mathbf{(3, 4)}, \mathbf{(5, 8)}, (9, 11), (10, 12)\}$
$F'_4 = \{\mathbf{(1, 2)}, \mathbf{(3, 6)}, \mathbf{(4, 5)}, (7, 11), (8, 10), (9, 12)\}$
$F'_5 = \{(1, 4), (2, 3), (5, 11), (6, 10), (7, 9), (8, 12)\}$
$F'_6 = \{\mathbf{(1, 8)}, \mathbf{(2, 6)}, (3, 11), (4, 10), (5, 9), (7, 12)\}$
$F'_7 = \{(1, 11), (2, 10), (3, 9), (4, 8), (5, 7), (6, 12)\}$
$F'_8 = \{(1, 9), (2, 8), (3, 7), (4, 6), (5, 12), (10, 11)\}$
$F'_9 = \{(1, 7), \mathbf{(2,11)}, (3, 5), (4, 12), \mathbf{(6, 8)}, (9, 10)\}$
$F'_{10} = \{(1, 5), (2, 4), (3, 12), (6, 11), (7, 10), (8, 9)\}$
$F'_{11} = \{(1, 3), (2, 12), (4, 11), (5, 10), (6, 9), (7, 8)\}$

K_{12} with a non-isomorphic 1-factorization that is not perfect. We recall that the canonical 1-factorization of K_{12} is one of the cases for which both TS and PTS are equivalent, as well as RS and PRS. Although this is not a proof of connectivity, it shows that neighborhood GPTS creates some connections in the search space graph $\mathscr{G} = (F, M)$ associated with the union of neighborhoods HAS, TS, RS, PTS, and PRS.

Concluding Remarks

Sports scheduling was the theme of the Fifth International Timetabling Competition (ITC2021). Its goal was to promote and provide insights into the development of more generally applicable sports timetabling solvers, considering that most works in the literature deal with particular sports scheduling problems applied to a specific context (in terms of the league, sport, and region).

The proposed problem asked for a schedule for a compact double round-robin tournament with 16, 18, or 20 teams. In some instances, the schedule must be phased, meaning that it should be composed of two consecutive single round-robin tournaments with an opposite home-away assignment.

The constraints were grouped into soft and hard and divided into five categories. Capacity constraints force a team to play home or away in a given round, limit the number of games a team plays against a particular group of teams in a given period, set a limit on the number of consecutive home or away games, or enforce complementary patterns of home and away games. Game constraints enforce or forbid specific assignments of games to rounds. Fairness constraints impose that the maximum difference between the number of home games played by any pair of teams is at most two at any particular point in the tournament. Break constraints regulate the maximum number of home, away, and total breaks, as well as the set of rounds in which such breaks are allowed to happen. Finally, separation constraints impose lower and upper bounds on the number of rounds separating two games between the same teams.

Table 4 Applications of metaheuristics to sports scheduling problems

Metaheuristic	Problem	Reference
Simulated annealing	International Timetabling Competition 2021	Rosati et al. [105]
	Cricket scheduling (England)	Wright [128]
	Rugby scheduling (New Zealand)	Johnston and Wright [55]
	Football scheduling (England)	Kendall et al. [57]
	MLB umpire scheduling	Trick et al. [111]
	Baseball scheduling (Korea)	Jeong et al. [54]
	Traveling tournament problem	Anagnostopoulos et al. [3]
	Mirrored traveling tournament problem	Van Hentenryck and Vergados [119]
	Traveling tournament problem	Lim et al. [69]
	Scheduling competitions at multiple venues	Lim et al. [68]
	Basketball scheduling (New Zealand)	Wright [127]
	Cricket scheduling (New Zealand)	Wright [126]
	Cricket scheduling (Australia)	Willis and Terrill [123]
	Cricket scheduling (England)	Wright [124]
	Grouping and scheduling multiple sports	Li and Goossens [66]
	ITC2021	van Doornmalen et al. [21]
	ITC2021	Rosati et al. [105]
Tabu search	Indoor football scheduling (Belgium)	Van Bulck et al. [117]
	Traveling tournament problem	Goerigk and Westphal [38]
	Volleyball scheduling (Argentina)	Bonomo et al. [5]
	Traveling tournament problem	Di Gaspero and Schaerf [19]
	League scheduling	Hamiez and Hao [42]
	Tennis round-robin scheduling	Della Croce et al. [18]
	Cricket scheduling (England)	Wright [125]
	Multi-league sports scheduling	Li et al. [67]
Iterated local search	Traveling umpire problem	Wauters et al. [121]
	Traveling tournament problem with fixed venues	Costa et al. [14]
	Weighted carry-over effects	Guedes and Ribeiro [41]
	Traveling tournament problem with fixed venues	Costa et al. [13]

(continued)

Table 5 (continued)

Metaheuristic	Problem	Reference
	Biobjective referee assignment in amateur leagues	Duarte and Ribeiro [22]
	Referee assignment in amateur football leagues	Duarte et al. [23, 24]
	Mirrored traveling tournament problem	Ribeiro and Urrutia [100]
	ITC2021	Premananda et al. [84]
Variable neighborhood search	ITC2021	Van Bulck and Goossens [116]
Local search	Rugby scheduling (Wales)	Lewis and Thompson [65]
	Hockey scheduling (Finland)	Kyngäs and Nurmi [62]
	Referee assignment in professional football leagues	Yavuz et al. [130]
	ITC2021	Phillips et al. [82]
Genetic and memetic algorithms	Football scheduling (England)	While and Kendall [122]
	Traveling umpire problem	Trick and Yildiz [110]
	NHL scheduling (Canada and USA)	Costa [12]
	NHL scheduling (Canada and USA)	Craig et al. [16]
	Scheduling social golfers	Cotta et al. [15]
	Table tennis scheduling (Germany)	Schönberger et al. [107],
		Schönberger and Mattfeld [106]
Ant colony optimization	Traveling tournament problem	Uthus et al. [114]
	Maximum value tournament scheduling	Uthus et al. [113]

To encompass as many real problems as possible, the soft and hard constraints varied from one instance to another. The objective function consisted in the minimization of penalties associated with the violation of the soft constraints, while the hard constraints must necessarily be met. Some algorithms proposed for the problem are described in Van Bulck et al. [118].

Table 4 concludes this chapter by presenting a non-exhaustive collection of applications of metaheuristics to sports scheduling problems. We include both time-constrained and time-relaxed sports scheduling applications. For each metaheuristic, we indicate the problems that motivated the applications and the corresponding references where they appeared. The base metaheuristic was sometimes hybridized with integer programming, constraint programming, or another metaheuristic. Although constraint programming is an efficient approach for scheduling problems in sports with intricate feasibility patterns [4, 46, 79, 109], entries to its specific applications are omitted from this table (see, e.g., Knust [60] for an extended list of references about sports scheduling methods and applications).

Competing Interest Declaration The author(s) has no competing interests to declare that are relevant to the content of this manuscript.

References

1. Aarts E, Korst J (1989) Simulated annealing and Boltzmann machines: a stochastic approach to combinatorial optimization and neural computing. Wiley, Hoboken, New Jersey, U.S
2. Aarts E, Korst J (2002) Selected topics in simulated annealing. In: Ribeiro CC, Hansen P (eds) Essays and surveys in metaheuristics. Kluwer, Dordrecht, Netherlands, pp 1–37
3. Anagnostopoulos A, Michel L, Van Hentenryck P, Vergados Y (2006) A simulated annealing approach to the traveling tournament problem. J Schedul 9:177–193
4. Benoist T, Laburthe F, Rottembourg B (2001) Lagrange relaxation and constraint programming collaborative schemes for travelling tournament problems. In: Proceedings of the third international workshop on integration of AI and OR techniques in constraint programming for combinatorial optimization problems, pp 15–26
5. Bonomo F, Cardemil A, Durán G, Marenco J, Sabán D (2012) An application of the traveling tournament problem: the Argentine volleyball league. Interfaces 42:245–259
6. Brandão JS, Noronha TF, Resende MGC, Ribeiro CC (2015) A biased random-key genetic algorithm for single-round divisible load scheduling. Int Trans Oper Res 22:823–839
7. Brandão JS, Noronha TF, Resende MGC, Ribeiro CC (2017) A biased random-key genetic algorithm for scheduling heterogeneous multi-round systems. Int Trans Oper Res 27: 1061–1077
8. Buriol LS, Resende MGC, Ribeiro CC, Thorup M (2005) A hybrid genetic algorithm for the weight setting problem in OSPF/IS-IS routing. Networks 46:36–56
9. Burke E, Kendall G (eds) (2005) Search methodologies: introductory tutorials in optimization and decision support techniques. Springer, New York City, New York, US
10. Burke E, Kendall G (eds) (2014) Search methodologies: introductorial tutorials in optimization and decision support techniques, 2nd edn. Springer, New York City, New York, US
11. Canuto S, Resende MGC, Ribeiro CC (2001) Local search with perturbations for the prize-collecting Steiner tree problem in graphs. Networks 38:50–58
12. Costa D (1995) An evolutionary tabu search algorithm and the NHL scheduling problem. INFOR 33:161–178
13. Costa FN, Urrutia S, Ribeiro CC (2008) An ILS heuristic for the traveling tournament problem with fixed venues. In: Burke EK, Gendreau M (eds) Proceedings of the 7th international conference on the practice and theory of automated timetabling
14. Costa FN, Urrutia S, Ribeiro CC (2012) An ILS heuristic for the traveling tournament problem with predefined venues. Ann Oper Res 194:137–150
15. Cotta C, Dotú I, Fernández AJ, Van Hentenryck P (2006) Scheduling social golfers with memetic evolutionary programming. In: Almeida F, Aguilera MJB, Blum C, Moreno Vega JM, Pérez Pérez M, Roli A, Sampels M (eds) Hybrid metaheuristics. Lecture notes in computer science, vol 4030. Springer, New York City, New York, US, pp 150–161
16. Craig S, While L, Barone L (2009) Scheduling for the National Hockey League using a multi-objective evolutionary algorithm. In: Nicholson A, Li X (eds) AI 2009: advances in artificial intelligence. Lecture notes in computer science, vol 5866. Springer, New York City, New York, US, pp 381–390
17. Croes G (1958) A method for solving traveling-salesman problems. Oper Res 6:791–812
18. Della Croce F, Tadei R, Asioli P (1999) Scheduling a round robin tennis tournament under courts and players availability constraints. Ann Oper Res 92:349–361
19. Di Gaspero L, Schaerf A (2007) A composite-neighborhood tabu search approach to the traveling tournament problem. J Heuristics 13:189–207
20. Dinitz JH, Garnick DK, McKay BD (1994) There are 526,915,620 nonisomorphic one-factorizations of K_{12}. J Comb Des 2:273–285

21. van Doornmalen J, Hojny C, Lambers R, Spieksma F (2021) A hybrid model to find schedules for double round robin tournaments with side constraints. In: Proceedings of the 13th international conference on the practice and theory of automated timetabling-PATAT 2021, p 412
22. Duarte AR, Ribeiro CC (2008) Referee assignment in sports leagues: approximate and exact multi-objective approaches. In: 19th international conference on multiple criteria decision making, Auckland, pp 58–60
23. Duarte AR, Ribeiro CC, Urrutia S (2007) A hybrid ILS heuristic to the referee assignment problem with an embedded MIP strategy. In: Bartz-Beielstein T, Aguilera MJB, Blum C, Naujoks B, Roli A, Rudolph G, Sampels M (eds) Hybrid metaheuristics. Lecture notes in computer science, vol 4771. Springer, New York City, New York, US, pp 82–95
24. Duarte AR, Ribeiro CC, Urrutia S, Haeusler EH (2007) Referee assignment in sports leagues. In: Burke EK, Rudová H (eds) Practice and theory of automated timetabling VI. Lecture notes in computer science, vol 3867. Springer, New York City, New York, US, pp 158–173
25. Feo TA, Resende MGC (1989) A probabilistic heuristic for a computationally difficult set covering problem. Oper Res Lett 8:67–71
26. Feo TA, Resende MGC (1995) Greedy randomized adaptive search procedures. J Glob Optim 6:109–133
27. Festa P, Resende MGC (2002) GRASP: an annotated bibliography. In: Ribeiro CC, Hansen P (eds) Essays and surveys in metaheuristics. Kluwer, Dordrecht, Netherlands, pp 325–367
28. Festa P, Resende MGC (2009) An annotated bibliography of GRASP, Part II: applications. Int Trans Oper Res 16:131–172
29. Festa P, Pardalos P, Resende MGC, Ribeiro CC (2002) Randomized heuristics for the max-cut problem. Optim Methods Softw 7:1033–1058
30. Gendreau M, Potvin JY (eds) (2019) Handbook of metaheuristics, 3rd edn. Springer, New York City, New York, US
31. Glover F (1989) Tabu search – Part I. ORSA J Comput 1:190–206
32. Glover F (1990) Tabu search – Part II. ORSA J Comput 2:4–32
33. Glover F (1996) Tabu search and adaptive memory programming – advances, applications and challenges. In: Barr R, Helgason R, Kennington J (eds) Interfaces in computer science and operations research. Kluwer, Dordrecht, Netherlands, pp 1–75
34. Glover F (2022) Unforeseen consequences of "Tabu" choices – a retrospective. INFORMS J Comput 34:1306–1308
35. Glover F, Kochenberger G (eds) (2003) Handbook of metaheuristics. Kluwer, Dordrecht, Netherlands
36. Glover F, Laguna M (1997) Tabu search. Kluwer, Dordrecht, Netherlands
37. Glover F, Taillard E, de Werra D (1993) A user's guide to tabu search. Ann Oper Res 41:3–28
38. Goerigk M, Westphal S (2016) A combined local search and integer programming approach to the traveling tournament problem. Ann Oper Res 239:343–354
39. Goldberg DE (1989) Genetic algorithms in search, optimization and machine learning. Addison-Wesley, Boston, MA, US
40. Goncalves JF, Resende MGC (2011) Biased random-key genetic algorithms for combinatorial optimization. J Heuristics 17:487–525
41. Guedes ACB, Ribeiro CC (2011) A heuristic for minimizing weighted carry-over effects in round robin tournaments. J Schedul 14:655–667
42. Hamiez JP, Hao JK (2001) Solving the sports league scheduling problem with tabu search. In: Nareyek A (ed) Local search for planning and scheduling. Lecture notes in artificial intelligence, vol 2148. Springer, New York City, New York, US, pp 24–36
43. Hansen P, Mladenović N (2002) Developments of variable neighborhood search. In: Ribeiro CC, Hansen P (eds) Essays and surveys in metaheuristics. Kluwer, Dordrecht, Netherlands, pp 415–439
44. Hansen P, Mladenović N (2003) Variable neighborhood search. In: Glover F, Kochenberger G (eds) Handbook of metaheuristics. Kluwer, Dordrecht, Netherlands, pp 145–184

45. Henderson D, Jacobson S, Johnson A (2003) The theory and practice of simulated annealing. In: Glover F, Kochenberger G (eds) Handbook of metaheuristics. Kluwer, Dordrecht, Netherlands, pp 287–319
46. Henz M (2001) Scheduling a major college basketball conference – revisited. Oper Res 49:163–168
47. Hertz A, Taillard E, de Werra D (1997) Tabu search. In: Aarts E, Lenstra JK (eds) Local search in combinatorial optimization. Wiley, Hoboken, New Jersey, U.S, pp 121–143
48. Holland J (1975) Adaptation in natural and artificial systems. University of Michigan Press, Ann Arbor, Michigan, US
49. Holland J (1992) Genetic algorithms. Sci Am 267:44–50
50. Hoos H, Stützle T (2005) Stochastic local search: foundations and applications. Elsevier, Amsterdam, Netherlands
51. Januario T, Urrutia S (2016) A new neighborhood structure for round robin scheduling problems. Comput Oper Res 70:127–139
52. Januario T, Urrutia S, Ribeiro CC, de Werra D (2016) Edge coloring: a natural model for sports scheduling. Eur J Oper Res 254:1–8
53. Januario T, Urrutia S, de Werra D (2016) Sports scheduling search space connectivity: a riffle shuffle driven approach. Discret Appl Math 211:113–120
54. Jeong HM, Kim SW, Choi Y, Kim AJ, Eun J, Kim BJ (2012) Traveling baseball players' problem in Korea. J Korean Phys Soc 61:484–492
55. Johnston M, Wright M (2014) Prior analysis and scheduling of the 2011 Rugby Union ITM Cup in New Zealand. J Oper Res Soc 65:1292–1300
56. Kaski P, Ostergard PRJ (2009) There are 1,132,835,421,602,062,347 nonisomorphic one-factorizations of K_{14}. J Comb Des 17:147–159
57. Kendall G, McCollum B, Cruz FRB, McMullan P, While L (2013) Scheduling English football fixtures: consideration of two conflicting objectives. In: Talbi EG (ed) Hybrid metaheuristics. Studies in computational intelligence, vol 434. Springer, New York City, New York, US, pp 369–385
58. Kirkman TP (1847) On a problem in combinations. Camb Dublin Math J 2:191–204
59. Kirkpatrick S, Gelatt Jr C, Vecchi M (1983) Optimization by simulated annealing. Science 220(4598):671–680
60. Knust S (2022) Classification of literature on sports scheduling. Online reference at http://www2.inf.uos.de/knust/sportssched/sportlit_class/, last access on 12 Sep 2022
61. Kobayashi M (1989) On perfect one-factorization of the complete graph K_{2p}. Graphs Comb 5:351–353
62. Kyngäs J, Nurmi K (2009) Scheduling the Finnish major ice hockey league. In: Proceedings of the 2009 IEEE symposium on computational intelligence in scheduling. IEEE, pp 84–89
63. van Laarhoven P, Aarts E (1987) Simulated annealing: theory and applications. Kluwer, Dordrecht, Netherlands
64. Laguna M, Martí R (1999) GRASP and path relinking for 2-layer straight line crossing minimization. INFORMS J Comput 11:44–52
65. Lewis R, Thompson J (2011) On the application of graph colouring techniques in round-robin sports scheduling. Comput Oper Res 38:190–204
66. Li M, Goossens D (2024) Grouping and scheduling multiple sports leagues: an integrated approach. J Oper Res Soc. https://doi.org/10.1080/01605682.2024.2391516
67. Li M, Davari M, Goossens D (2023) Multi-league sports scheduling with different leagues sizes. Eur J Oper Res 307:313–327
68. Lim A, Rodrigues B, Zhang X (2006) Scheduling sports competitions at multiple venues – revisited. Eur J Oper Res 175:171–186
69. Lim A, Rodrigues B, Zhang X (2006) A simulated annealing and hill-climbing algorithm for the traveling tournament problem. Eur J Oper Res 174:1459–1478
70. Lin S, Kernighan B (1973) An effective heuristic algorithm for the traveling-salesman problem. Oper Res 21:498–516

71. Londe MA, Pessoa LS, Andrade CE, Resende MGC (2035) Biased random-key genetic algorithms: a review. Eur J Oper Res 321:1–22
72. Lourenço HR, Martin OC, Stutzle T (2003) Iterated local search. In: Glover F, Kochenberger GA (eds) Handbook of metaheuristics. Kluwer, Dordrecht, Netherlands, pp 321–353
73. Martin OC, Otto SW (1996) Combining simulated annealing with local search heuristics. Ann Oper Res 63:57–75
74. Martin OC, Otto SW, Felten EW (1991) Large-step Markov chains for the traveling salesman problem. Complex Syst 5:299–326
75. Martins SL, Ribeiro CC, Rosseti I (2004) Applications and parallel implementations of metaheuristics in network design and routing. In: Manandhar S, Austin J, Desai U, Oyanagi Y, Talukder AK (eds) Applied computing. Lecture notes in computer science, vol 3285. Springer, New York City, New York, US, pp 205–213
76. Michalewicz Z (1996) Genetic algorithms + Data structures = Evolution programs. Springer, New York City, New York, US
77. Michiels W, Aarts E, Korst J (2007) Theoretical aspects of local search. Springer, New York City, New York, US
78. Mladenovic N, Hansen P (1997) Variable neighborhood search. Comput Oper Res 34:1097–1100
79. Nemhauser GL, Trick MA (1998) Scheduling a major college basketball conference. Oper Res 46:1–8
80. Noronha TF, Ribeiro CC (2024) Biased random-key genetic algorithms: a tutorial with applications. In: ISMSI 2024: proceedings of the 2024 8th international conference on intelligent systems, metaheuristics & swarm intelligence (ISMSI). ACM, New York, pp 110–115
81. Noronha TF, Resende MGC, Ribeiro CC (2011) A biased random-key genetic algorithm for routing and wavelength assignment. J Glob Optim 50:503–518
82. Phillips AE, O'Sullivan M, Walker C (2021) An adaptive large neighbourhood search matheuristic for the itc2021 sports timetabling competition. In: Proceedings of the 13th international conference on the practice and theory of automated timetabling-PATAT, vol 2
83. Pinto BQ, Ribeiro CC, Rosseti I, Noronha TF (2020) A biased random-key genetic algorithm for routing and wavelength assignment under a sliding scheduled traffic model. J Glob Optim 77:949–973
84. Premananda IGA, Tjahyanto A, Mukhlason A (2024) Efficient iterated local search based metaheuristic approach for solving sports timetabling problems of International Timetabling Competition 2021. Ann Oper Res. https://doi.org/10.1007/s10479-024-06285-x
85. Reeves C (ed) (1993) Modern heuristic techniques for combinatorial problems. Blackwell, Oxford, UK
86. Reeves C (2003) Genetic algorithms. In: Glover F, Kochenberger G (eds) Handbook of metaheuristics. Kluwer, Dordrecht, Netherlands, pp 65–82
87. Reeves C, Rowe J (2002) Genetic algorithms: principles and perspectives. Springer, New York City, New York, US
88. Reeves C, Yamada T (1998) Genetic algorithms, path relinking and the flowshop sequencing problem. Evol Comput 6:45–60
89. Resende MGC, Ribeiro CC (2003) A GRASP with path-relinking for private virtual circuit routing. Networks 41:104–114
90. Resende MGC, Ribeiro CC (2003) Greedy randomized adaptive search procedures. In: Glover F, Kochenberger G (eds) Handbook of metaheuristics. Kluwer, Dordrecht, Netherlands, pp 219–249
91. Resende MGC, Ribeiro CC (2005) GRASP with path-relinking: Recent advances and applications. In: Ibaraki T, Nonobe K, Yagiura M (eds) Metaheuristics: progress as real problem solvers. Springer, New York City, New York, US, pp 29–63
92. Resende MGC, Ribeiro CC (2010) Greedy randomized adaptive search procedures: advances and applications. In: Gendreau M, Potvin JY (eds) Handbook of metaheuristics, 2nd edn. Springer, New York City, New York, US, pp 293–319

93. Resende MGC, Ribeiro CC (2014) GRASP: greedy randomized adaptive search procedures. In: Burke E, Kendall G (eds) Search methodologies: introductorial tutorials in optimization and decision support systems, chap 11, 2nd edn. Springer, New York City, New York, US, pp 287–312
94. Resende MGC, Ribeiro CC (2016) Biased-random key genetic algorithms: an advanced tutorial. In: Companion volume – proceedings of the 2016 genetic and evolutionary computation conference – GECCO'16. ACM, pp 483–514
95. Resende MGC, Ribeiro CC (2016) Optimization by GRASP: greedy randomized adaptive search procedures. Springer, New York City, New York, US
96. Resende MGC, Werneck RF (2004) A hybrid heuristic for the p-median problem. J Heuristics 10:59–88
97. Ribeiro CC (2002) GRASP: Une métaheuristique gloutone et probabiliste. In: Teghem J, Pirlot M (eds) Optimisation approchée en recherche opérationnelle, Hermès, Paris, France, pp 153–176
98. Ribeiro CC, Rosseti I (2002) A parallel GRASP heuristic for the 2-path network design problem. In: Monien B, Feldmann R (eds) Euro-Par 2002 parallel processing. Lecture notes in computer science, vol 2400. Springer, New York City, New York, US, pp 922–926
99. Ribeiro CC, Rosseti I (2007) Efficient parallel cooperative implementations of GRASP heuristics. Parallel Comput 33:21–35
100. Ribeiro CC, Urrutia S (2007) Heuristics for the mirrored traveling tournament problem. Eur J Oper Res 179:775–787
101. Ribeiro CC, Vianna DS (2009) A hybrid genetic algorithm for the phylogeny problem using path-relinking as a progressive crossover strategy. Int Trans Oper Res 16:641–657
102. Ribeiro CC, Uchoa E, Werneck R (2002) A hybrid GRASP with perturbations for the Steiner problem in graphs. INFORMS J Comput 14:228–246
103. Ribeiro CC, Martins SL, Rosseti I (2007) Metaheuristics for optimization problems in computer communications. Comput Commun 30:656–669
104. Ribeiro CC, Urrutia S, de Werra D (2023) Combinatorial models for scheduling sports tournaments. Springer, New York City, New York, US
105. Rosati RM, Petris M, Di Gaspero L, Schaerf A (2022) Multi-neighborhood simulated annealing for the sports timetabling competition ITC2021. J Schedul 25:301–319
106. Schönberger J, Mattfeld DC (2000) Automated timetable generation for rounds of a table-tennis league. In: Proceedings of the 2000 congress on evolutionary computation, pp 277–284
107. Schönberger J, Mattfeld DC, Kopfer H (2004) Memetic algorithm timetabling for non-commercial sport leagues. Eur J Oper Res 153:102–116
108. Souza MC, Duhamel C, Ribeiro CC (2003) A GRASP with path-relinking heuristic for the capacitated minimum spanning tree problem. In: Resende MGC, Souza J (eds) Metaheuristics: computer decision making. Kluwer, Dordrecht, Netherlands, pp 627–658
109. Trick MA (2003) Integer and constraint programming approaches for round robin tournament scheduling. In: Burke E, de Causmaecker P (eds) Practice and theory of automated timetabling IV. Lecture notes in computer science, vol 2740. Springer, New York City, New York, US, pp 63–77
110. Trick MA, Yildiz H (2012) Locally optimized crossover for the traveling umpire problem. Eur J Oper Res 216:286–292
111. Trick MA, Yildiz H, Yunes T (2012) Scheduling Major League Baseball umpires and the traveling umpire problem. Interfaces 42:232–244
112. Urrutia S, de Werra D, Januario T (2021) Recoloring subgraphs of K_{2n} for sports scheduling. Theor Comput Sci 877:36–45
113. Uthus DC, Riddle PJ, Guesgen HW (2008) Ant colony optimization and the single round robin maximum value problem. In: Dorigo M, Birattari M, Blum C, Clerc M, Stützle T, Winfield AFT (eds) Ant colony optimization and swarm intelligence. Lecture notes in computer science, vol 5217. Springer, New York City, New York, US, pp 243–250
114. Uthus DC, Riddle PJ, Guesgen HW (2009) An ant colony optimization approach to the traveling tournament problem. In: GECCO'09: proceedings of the 11th annual conference on genetic and evolutionary computation. ACM, pp 81–88

115. Van Bulck D, Goossen D (2023) The international timetabling competition on sports timetabling (ITC2021). Eur J Oper Res 308:1249–1267
116. Van Bulck D, Goossens D (2023) First-break-heuristically-schedule: constructing highly-constrained sports timetables. Oper Res Lett 51:326–331
117. Van Bulck D, Goossens DR, Spieksma F (2019) Scheduling a non-professional indoor football league: a tabu search based approach. Ann Oper Res 275:715–730
118. Van Bulck D, Goossens D, Clarner JP, Dimitsas A, Fonseca GH, Lamas-Fernandez C, Lester MM, Pedersen J, Phillips AE, Rosati RM (2024) Which algorithm to select in sports timetabling? Eur J Oper Res 318:575–591
119. Van Hentenryck P, Vergados Y (2006) Traveling tournament scheduling: a systematic evaluation of simulated annealling. In: Integration of AI and OR techniques in constraint programming for combinatorial optimization problems. Lecture notes in computer science, vol 3990. Springer, New York City, New York, US, pp 228–243
120. Vizing VG (1964) On an estimate of the chromatic class of a p-graph. Diskrete Analiz 3:25–30, in Russian
121. Wauters T, Malderen SV, Berghe GV (2014) Decomposition and local search based methods for the traveling umpire problem. Eur J Oper Res 238:886–898
122. While L, Kendall G (2014) Scheduling the English football league with a multi-objective evolutionary algorithm. In: Bartz-Beielstein T, Branke J, Filipič B, Smith J (eds) Parallel problem solving from nature – PPSN XIII. Lecture notes in computer science, vol 8672. Springer, New York City, New York, US, pp 842–851
123. Willis RJ, Terrill BJ (1994) Scheduling the Australian state cricket season using simulated annealing. J Oper Res Soc 45:276–280
124. Wright M (1992) A fair allocation of county cricket opponents. J Oper Res Soc 43:195–201
125. Wright M (1994) Timetabling county cricket fixtures using a form of tabu search. J Oper Res Soc 45:758–770
126. Wright M (2005) Scheduling fixtures for New Zealand cricket. IMA J Manag Math 16: 99–112
127. Wright M (2006) Scheduling fixtures for basketball New Zealand. Comput Oper Res 33:1875–1893
128. Wright M (2018) Scheduling an amateur cricket league over a nine-year period. J Oper Res Soc 69:1854–1862
129. Yagiura M, Ibaraki T (2002) Local search. In: Pardalos P, Resende M (eds) Handbook of applied optimization. Oxford University Press, pp 104–123
130. Yavuz M, Inan UH, Figlali A (2008) Fair referee assignments for professional football leagues. Comput Oper Res 35:2937–2951

Network Optimization

49

Luciana S. Buriol

Contents

Introduction 1524
Preliminaries 1525
Basic Network Problems 1525
Point-to-Point Shortest Path Problem (PPSPP) 1525
Maximum Flow Problem (MFP) 1526
Minimum Cost Flow Problem (MCFP) 1527
Multicommodity Flow Problems 1527
Weight Setting Problem (WSP) 1529
The WSP Applied to Telecommunication Networks 1529
Heuristic Approaches for the WSP 1531
Virtual Network Embedded Problem (VNEP) 1534
Single VNEP Variant 1535
Heuristic Approaches for the VNEP 1536
Conclusions 1538
Cross-References 1539
References 1539

Abstract

Several real-world problems can be modeled as graph problems. Graph algorithms and theories that have evolved for decades can be applied to solve the problem on hand. Interestingly, many of these graph problems can be solved polynomially, while small changes in a problem definition turn the problem difficult. In this chapter, we explore this path from polynomial network problems to NP-hard ones. Along the chapter, we visit several problems, dedicating more extended discussions to two real-world problems: the weight setting problem,

L. S. Buriol (✉)
Instituto de Informática, Universidade Federal do Rio Grande do Sul, Porto Alegre, Brazil
e-mail: buriol@inf.ufrgs.br

© Springer Nature Switzerland AG 2025

R. Martí et al. (eds.), *Handbook of Heuristics*,
https://doi.org/10.1007/978-3-032-00385-0_46

originated from telecommunication networks, and the virtual network embedded problem, a recent stated optimization problem from the computer network area. For these two problems, we discuss their heuristic solution, since only small instances can be solved exactly within a reasonable amount of time.

Keywords

Network optimization · Traffic engineering · Network virtualization

Introduction

Many problems can be modeled as graphs. Once a problem can be modeled as a graph, all graph algorithms and theories that have evolved for decades can be applied to analyze and solve the problem at hand. It is the case, for example, of social networks or webgraphs. In webgraphs, nodes represent webpages and edges are hyperlinks, while in social networks nodes usually represent people, and arcs represent interaction among them. In these two types of networks basically, the graph topology is analyzed, which usually configures complex and massive networks. Google and Facebook base some of their algorithms on these graphs. On the other hand, in biological networks, nodes can be represented by genes, and the correlation between each pair of nodes is represented through edge weights in a complete graph. By analyzing these graphs, scientists discover genes related to diseases and can then propose new drugs and observe how they affect the edge weights of the graph. Health schools and computer science groups have made several joint efforts in this direction.

Another class consists of graphs originating from services, such as telecommunication and road networks. These service graphs naturally are represented as a set of elements that includes a topology, but also additional information about nodes and arcs such as capacity, distance, demand, and delay are considered in computations over the network. In this chapter, we focus on problems over service networks. We will also discuss strategies applied to network optimization problems. Initially, we introduce related polynomial problems, and then we discuss constraints that turn the problems difficult to solve. Among the hard problems, we put some emphasis on two NP-hard problems: the weight setting problem (WSP) and virtual network embedded problem (VNEP). WSP has its main applications in traffic engineering problems from transit and telecommunications, while VNEP is a computer network problem.

We present most of the problems by their definition and also by their mathematical formulation. We aim to make the reader familiar with the mathematical problem modeling since optimization can often exploit this feature when solving a problem. Moreover, by analyzing their mathematical formulations, we can also highlight how characteristics of the problem change its modeling and affect its algorithms and solution space.

Preliminaries

Graphs naturally model network problems. Let $G = (V, A)$ denote a directed graph (DAG), where V is the set of nodes and A is the set of arcs. In a directed graph, the set of outgoing arcs from u is represented by $N^+(u)$, and similarly $N^-(u)$ represents the set of incoming arcs to node u. We denote the size of these sets by $|V| = n$ and $|A| = m$. An arc (u, v) has u as the tail node and v as the head node. Throughout this chapter we consider problems in simple graphs, that is, graphs without self-loops or parallel edges. However, directed graphs can admit symmetric arcs (u, v) and (v, u). A path in a graph is a sequence of arcs that connect pairs of neighbor nodes. The path is simple if there are no loops. Along the text, link and arc are used as synonymous.

Basic Network Problems

Initially, we introduce three representative polynomial time network problems that are found independently or as part of the core of more complex optimization problems. From several problem candidates, the point-to-point shortest path problem, maximum flow problem, and minimum cost flow problem were chosen to be discussed. These are fundamental network problems that have several variants that embrace a large number of practical applications.

Point-to-Point Shortest Path Problem (PPSPP)

Given a directed graph $G = (V, A)$ with weights w associated to arcs $w : A \to \mathcal{R}^+$, and source s and target t nodes, the PPSP problem consists in finding the path of minimum cost between nodes s and t. If a variable $x_a \in \{0, 1\}$ is associated to each $a \in A$, representing whether arc a is part of the solution, the point-to-point shortest path problem can be mathematically formulated as:

$$\textbf{min} \quad \sum_{a \in A} w_a x_a \tag{1}$$

$$\textbf{s.a} \quad \sum_{a \in N^+(v)} x_a - \sum_{a \in N^-(v)} x_a = \begin{cases} 1, \text{for } v = s, \\ 0, \text{for } v \in V \setminus \{s, t\}, \\ -1, \text{for } v = t. \end{cases} \quad x_a \in \{0, 1\} \text{ for } a \in A. \tag{2}$$

The objective is to minimize the cost path (Eq. 1). The path constraints (2) guarantee to find a valid path between nodes s and t. If $w : A \to \mathcal{R}^{+*}$, i.e., w has only positive values, the path found is always simple, otherwise, the shortest

path can have cycles of zero cost. The weight function w associated to arcs can represent time, cost, distance, or other measures, depending on the application. A typical application of this problem is to find a route in a road network of a navigation system, as a GPS, where w represents distance or time or the result of a cost function involving these two measures.

This problem can be solved exactly in time $O(m \log n)$ using binary heaps by the Dijkstra's algorithm. If additional information is available (or possible to compute), such as estimated distances (it cannot be an overestimation) between each pair of nodes, an admissible heuristic function can be used as part of the evaluation function of the A^* algorithm, and its time performance can be sped up. Also, in some situations, heuristics can be used such that a sub-optimal solution is returned. It is the case of *anytime algorithms* [30], which a feasible solution is requested in a short time, and as more time is available, the solution can be improved to a given ϵ distance error or up to optimality. Another case where the algorithm performance can be sped up is when a large number of queries of a path between a pair of nodes are requested, considering a static input [22, 28, 29]. In this case, information can be pre-computed and stored to be used in multiple queries. Finally, if one or more arc weight values of the input graph change, a previously calculated shortest path can be updated, instead of recomputed [35] from scratch. This approach is efficient when the application requires successive changes and updates of the graph.

Next, we discuss two flow problems. We focus on the integer version of the problems, i.e., the flow traversing each arc is always an integer value. For an extensive study in network flow problems, we refer to [1] which addresses theoretical and practical aspects of several problems.

Maximum Flow Problem (MFP)

The maximum flow problem (MFP) is a fundamental problem in network optimization. Given a directed graph $G = (V, A)$ with arc capacities $l : A \to \mathcal{R}$ associated with arcs, and source s and target t nodes, the problem consists in sending the maximum amount of flow between s and t without surpassing the arc capacities. If variables $x_a \in \mathcal{N}$ represent the amount of load on each $a \in A$, where $d_s = F$, $d_t = -F$, and $d_u = 0$ for $u \in V \setminus \{s, t\}$, and F stands for the maximal amount of flow traversing the network, this problem can be mathematically formulated as:

$$\textbf{max} \quad F = \sum_{a \in N^+(s)} x_a \tag{3}$$

$$\textbf{s.a} \quad \sum_{a \in N^+(v)} x_a - \sum_{a \in N^-(v)} x_a = d_v, \text{for } v \in V, \tag{4}$$

$$x_a \leq l_a, \text{for } a \in A, \tag{5}$$

$$x_a \in \mathcal{N}, \text{for } a \in A. \tag{6}$$

The objective function (3) maximizes the flow outgoing node s and traversing the network directed to node t, while constraints (5) assure capacities are not surpassed. Constraints (4) guarantee flow conservation: the difference between the total flow emanating from v and entering node v is equal to its supply/demand.

The MFP problem can be solved polynomially. In 1956 the pseudo-polynomial algorithm of Ford-Fulkerson was proposed. It has a time complexity of $O(m^2 \log F)$, where F is the maximum flow traversing the network. In 1972 a specialization of the previous algorithm was proposed which led to the Edmonds-Karp algorithm, whose time complexity is $O(nm^2)$, the first polynomial algorithm for solving the problem. A few other polynomial algorithms were proposed over the years, with a remark to the $O(n^3)$ algorithm of Goldberg and Tarjan [21] (fastest in practice) and the recently proposed $O(mn)$ algorithm of Orlin [33].

Minimum Cost Flow Problem (MCFP)

If besides capacities we have also costs $w : A \to \mathcal{R}$ associated with arcs, each node u can supply or demand a flow $d_u \in \mathcal{N}$, we can then define the minimum cost flow problem. The problem consists in transporting the flow through the network, respecting capacity constraints, and minimizing the cost function. With $x_a \in \mathcal{N}$ being the flow amount on arc a, the objective function of the problem is

$$\textbf{min} \quad \sum_{a \in A} w_a x_a$$

while the constrains are the same (4), (5), and (6) from the MFP, with d_u positive for supply nodes and set to negative for demand nodes. Moreover, the sum of all supplies should be equal to the sum of all demands, otherwise, the problem has no solution. This problem and several variants can be solved by the algorithms for the MFC, with a simple modification of the input graph [27, Sect. 7.7].

The arc capacities naturally define upper bounds on the flow on arcs. A common variant of flow problems is to consider also a lower bound on arc capacities, forcing that a given minimum amount of flow circulates on certain arcs. If lower bounds are considered on the flow problems above, then the problem is still polynomial [27, Sect. 7.7].

Consider the set S of source nodes and a set T of target nodes. In case $S \cap T = \varnothing$, both MFP and MCFP remain polynomial due to the Hoffman-Kruskal theorem on totally unimodular matrices [24]. However, if $S \cap T \neq \varnothing$, these problems are not polynomial anymore, and we discuss some variants in the next subsection.

Multicommodity Flow Problems

The MCFP and MFP route a commodity per time. In multicommodity flow problems, not one, but several commodities are routed simultaneously on the network.

Let $K = \{(o(1), d(1)), (o(2), d(2)), \ldots, (o(|K|), t(|K|)\} \subseteq V \times V$ denote the set of commodities or origin-destination (OD) pairs, where $o(k)$ and $t(k)$ represent, respectively, the origination and destination nodes for $k = 1, \ldots, |K|$. Each commodity k has an associated demand of traffic flow $d_k = d_{o(k),t(k)}$, i.e., for each OD pair $(o(k), t(k))$, there is an associated flow d_k that emanates from node $o(k)$ and terminates in node $t(k)$. The total set of commodities defines a demand matrix whose each entry $[o(k)][t(k)]$ specifies the demand $d_k = d_{o(k),t(k)}$.

The MFP and MCFP problems can be easily extended to a multicommodity flow version. Given an arbitrary OD matrix, the multicommodity MCFP can be formulated as:

$$\textbf{min} \quad \sum_{a \in A} w_a \sum_{k \in K} x_a^k \tag{7}$$

$$\textbf{s.a} \quad \sum_{k \in K} \sum_{a \in N^+(v)} x_a^k - \sum_{k \in K} \sum_{a \in N^-(v)} x_a^k = d_v^k, \text{ for } v \in V \tag{8}$$

$$\sum_{k \in K} x_a^k \leq l_a, \text{ for } a \in A, x_a \in \mathcal{N}, \text{ for } a \in A. \tag{9}$$

The time complexity of different variants of the problem is classified as polynomial or not, depending on the problem definition. For both, direct and undirect graphs, the decision version of the multicommodity integer flow problem is NP-complete even if the number of commodities is two [16].

Heuristics are largely applied as the solution approach for NP-hard multicommodity network optimization problems. The reasons are diverse. There are cases solved by mathematical programming approaches when the instances are small. However, for most cases, an exact approach does not solve medium instances of the problem even if a long time is available for processing. It is common to happen that the model cannot even be uploaded by the solver due to its size. There are also situations in which the problem cannot be mathematically modeled by linear or nonlinear constraints, as, for example, some electrical network characteristics such as electrical impedance. More often the problem can be modeled, but it results in a nontrivial nonlinear problem model that cannot be efficiently solved in a reasonable amount of time. Finally, some problems take as input data sets composed of multiple data that are in many cases estimates or average values. An exact solution, in this case, many times does not represent the best solution for the problem. For all these applications, heuristics are largely applied, and in most cases, they are the only possible practical resolution approach for the problem.

Even when a problem is solved heuristically, modeling the problem and solving it exactly is always important to find out which size an exact approach can be used in practice and if there are subsets of instances that are easier to solve exactly.

The next two subsections detail two NP-hard problems that are found in service networks and which are solved through metaheuristics. Their solution approaches are also presented and discussed.

Weight Setting Problem (WSP)

The weight setting problem (WSP) is defined in a directed weighted capacitated network G = (V,A), with $w : A \rightarrow \mathcal{N}$ and $l : A \rightarrow \mathcal{R}^+$. The problem consists in attributing a weight on each arc of the network such that when each flow from a demand matrix is sent along the shortest paths, a cost function $\phi(l_a)$, which depends on the arc flow l_a, is minimized. Usually, ϕ is a congestion cost function, and it can be defined in different ways according to specific goals.

The WSP has its primary application in telecommunication networks. The Internet is divided into autonomous systems (AS). Each AS controls its interior routing by an interior gateway protocol. Common interior gateway protocols, for instance, open shortest path first (OSPF), allow the operator to define the routes by setting integer weights on the network links. For example, in the main application described in this chapter, weights are set in the range [1 : 20], and ϕ minimizes the network congestion.

Another application of the WSP is in road networks. A road network can be represented as a directed graph $G = (V, A)$ where V represents the set of nodes (street or road intersections or points of interest) and A the set of arcs (street or road segments). Each arc $a \in A$ has an associated capacity c_a, and a time t_a, called the *free flow time*, that represents the time spent when traversing the unloaded arc a. To calculate the congestion on each link, a potential function Φ_a is computed as a function of the load or flow ℓ_a on arc a, along with other arc-tuning parameters. The flow in an arc is computed taking into account that a driver takes the shortest path. The cost of a path is calculated as the sum of tolls allocated in the path or as a function of the time and tolls. The tollbooth problem [37] consists in attributing toll values on r arcs of a road network, such that when demand is sent along the shortest path between origin and destination, the network congestion is minimized. The weight w_a that the WSP problem defines on an arc a represents in the tollbooth problem the toll charged for each unit of flow traversing arc a. Note that in this case, at most r weights are allocated.

We detail in the rest of this section the WSP applied to telecommunication networks, revising the current approaches being used to solve the problem. We initiate this discussion describing the problem formally.

The WSP Applied to Telecommunication Networks

WSP receives as input a capacitated directed graph $G = (V, A)$ with a capacity of $l_a \in \mathcal{R}^+$ for each arc $a \in A$. Besides that, an OD demand matrix, with $d_k \in \mathrm{R}\ \forall k \in K$, where K is the set of commodities or OD pairs, is also given as input. The *OSPF weight setting problem* consists in determining weights to be assigned to the links so as to optimize a cost function, typically associated with a network congestion measure. Thus, the problem defines an integer weight $w_a \in [1 : 2^{16}]$ (most of the studies in the literature use $w_a \in [1 : 20]$) $\forall a \in A$. The flow is routed along the

shortest paths between $o(k)$ and $t(k)$ for each OD pair. The shortest paths are calculated according to the link weights w_a defined by the problem. If there is more than one shortest path to an OD pair, the flow is evenly split at each node among all outgoing arcs that belongs to the shortest path graph to the flow destination. The cost function takes into account the amount of load on each arc of the network. In [19] a piece-wise linear function $\Phi = \sum_{a \in A} \phi_{l_a}$ is used to calculate the cost function Φ. However, depending on the application, one can consider simpler cost functions such as the ratio between the load and the capacity on the arc $\frac{l_a}{c_a}$. A ratio on an arc larger than one means the arc is overloaded.

A mathematical model for the weight setting problem is given in eqs. (10), (11), (12), (13), (14), (15), (16), (17), (18), (19), and (20). The decision variables for this model determine the weight value of an arc a. Denote by $w_a \in \{W_{\min}, W_{\min}+1, \ldots, W_{\max}\}$ as the weight value attributed to arc $a \in A$, where $W_{\min} \in \mathcal{N}$ and $W_{\max} \in \mathcal{N}$ are the minimum and maximum values of weights, respectively. For convenience we specify $W_{\min} = 1$. Let Q be the set of all nodes that are a destination of at least one OD pair. The auxiliary binary variable y_a^q is set to one ($y_a^q = 1$) if arc $a \in A$ is part of a shortest path to destination node $q \in \mathcal{Q}$. Finally, auxiliary variable δ_v^q is the shortest path distance from node $v \in V$ to destination node $q \in \mathcal{Q}$, and the constants M_1, M_2 and M_3 are sufficiently larger numbers. Below the problem tackled in [8] is formulated inspired in constraints presented in [7].

$$\min \Phi = \sum_{a \in A} \phi\left(l_a\right) \tag{10}$$

Subject to :

$$\ell_a = \sum_{q \in \mathcal{Q}} x_a^q, \quad \forall a \in A, \tag{11}$$

$$\sum_{a \in N_v^+} x_a^q - \sum_{a \in N_v^-} x_a^q = d_{v,q}, \quad \forall v \in V \setminus \{q\}, \quad \forall q \in \mathcal{Q}, \tag{12}$$

$$w_a + \delta_{a_h}^q - \delta_{a_t}^q \geq 0, \quad \forall a \in A, \quad \forall q \in \mathcal{Q}, \tag{13}$$

$$\delta_q^q = 0, \quad \forall q \in \mathcal{Q}, \tag{14}$$

$$w_a + \delta_{a_h}^q - \delta_{a_t}^q \geq \left(1 - y_a^q\right)/M_1, \quad \forall a \in A, \quad \forall q \in \mathcal{Q}, \tag{15}$$

$$w_a + \delta_{a_h}^q - \delta_{a_t}^q \leq \left(1 - y_a^q\right) M_2, \quad \forall a \in A, \quad \forall q \in \mathcal{Q}, \tag{16}$$

$$y_a^q \geq x_a^q, \quad \forall a \in A, \quad \forall q \in \mathcal{Q}, \tag{17}$$

$$M_3 y_a^q + M_3 y_b^q \leq 2M_3 - x_a^q + x_b^q, \quad \forall a, b \in \mathrm{A}_{N_v^+}^2, \forall v \in V, \quad \forall q \in \mathcal{Q}, \tag{18}$$

$$W_{\min} \leq w_a \leq W_{\max}, \quad \forall a \in A, \tag{19}$$

$$x_a^q \in \mathcal{N}, y_a^q \in \{0, 1\}, w_a \geq 0, \delta_v^q \geq 0; \forall a \in A, \forall v \in V, \forall q \in \mathcal{Q}, \tag{20}$$

Objective function (10) minimizes average user travel time. Constraints (11) define the total flow on each arc $a \in A$, while constraints (12) impose flow conservation. The input data d_{uv} is the flow originated in u with destination in v. The following constraints impose the flow of each commodity to follow the shortest path between the corresponding OD pair. An arc a belongs to the shortest path to destination q if the distance $\delta_{a_h}^q - \delta_{a_t}^q$ is equal to the arc cost, which in this case is w_a. Thus, the constraints (13) define the shortest path distance for each node $v \in V$ and each destination $q \in \mathcal{Q}$. For consistency, constraints (14) require, for all $q \in \mathcal{Q}$, that the shortest distance from q to itself be zero. Constraints (15) and (16) together with (13) and (14) determine whether arc $a \in A$ belongs to the shortest path and thus determine the values of y_a^q, for $q \in \mathcal{Q}$. Constraints (15) require that an arc that does not belong to the shortest path has reduced cost $w_a + \delta_{a_h}^q - \delta_{a_t}^q > 0$. It also prevents flow from traversing zero-cost cycles. Constraints (16) assure that if the reduced cost of arc $a \in A$ and destination $q \in \mathcal{Q}$ is equal to zero, then arc a belongs to the shortest path to destination q, i.e., $y_a^q = 1$. Constraints (17) assure that flow is sent only on arcs belonging to a shortest path. Constraints (18) guarantee that flow is split evenly among all shortest paths (even-split constraints). In these constraints, $\mathsf{A}^2_{N_v^+}$ is the set of all ordered groups of two distinct elements of N_v^+. In [37] this constraint is also used and presented in detail. Constraints (19) limit the minimum and the maximum weight value. The last constraints define the domains of the variables.

Heuristic Approaches for the WSP

Fortz and Thorup showed that the weight setting problem for OSPF is NP-hard [19]. Several authors have proposed further heuristic solutions for solving variants of the problem applied in different contexts. Table 1 summarizes a literature review on solution approaches for solving the WSP. The first column of the table refers to the problem application. The problem is found in telecommunication and road networks. We mean by network operation the optimization on the use of available resources, while network design is related to the allocation of resources to attend minimal conditions for the network operation. Column $|V|/|A|/|OD|$ presents the measures (number of nodes, arcs, and OD pairs) of the largest network considered in the computational results for each reference indicated in the third column of the table. Finally, the last column presents the solution approach of each work indicated in the previous column. The methods used are tabu search (TS), genetic algorithms (GAs), simulated annealing (SA), memetic algorithm (MA), multi-objective optimization (MultiObj), heuristic based on mathematical programming (MIP), and biased random-key genetic algorithms (BRKGAs).

Table 1 Solution approaches for solving the WSP applied in different contexts

Application	$\|V\|$ / $\|A\|$ / $\|OD\|$	Reference	Solution approach
Telecommunications operation	100/403/–	[19, 20]	TS
Telecommunications operation	100/503/9,900	[15]	GA
Telecommunications operation	56/224/3,080	[34]	SA
Telecommunications operation	100/503/9,900	[8]	MA
Telecommunications design	30/148/–	[7]	MultiObj
Telecommunications design	71/350/4,960	[9]	MA
Telecommunication operation	50/88/662	[5]	MIP
Road network operation	974/2,153/9,505	[37]	BRKGA

In some of the works indicated in the third column of the table, the authors also presented results from optimal approaches, for comparison purposes. For example, in [37], which applies WSP to road networks, the largest network that CPLEX was able to solve optimally in less than 30 min has dimensions 9/26/68. If the instance sizes double, in about 50% of the runs, no feasible solution is found in 30 min. However, the BRKGA proposed in the paper presents results for real-world instances two orders of magnitude larger, within the same 30 min.

Construction Heuristics for the WSP

For all telecommunication operation problems from Table 1, a solution can be simply represented as a vector w of m weights, one for each arc. Once the weight vector is generated, the solution cost can be calculated according to Fig. 1.

In order to update the new arc loads, the shortest paths are computed to all destination nodes $t \in T$ and arrive at a graph $g^t = (V, A^t)$, $\forall\ t \in T$ (Line 2). This is achieved using Dijkstra's well-known shortest path algorithm. Since shortest paths are computed to all destination nodes, the directions of all arcs are reversed in G, and the distances $\pi_u^t,\ \ \forall\ u \in V$ to a destination in T are computed. Given the shortest paths to each destination, auxiliar vectors g^t (Line 3) and δ^t (Line 4) are computed considering each destination t. Each position $g_a^t \in \{0, 1\}$ indicates if arc a belongs to at least one shortest path with destination in t. On the nodes side, each position δ_v^t stores the number of outgoing arcs of node v that belongs to at least one shortest path with destination in t. The flows L^t (Line 5) from all OD demand pairs with destination t are then routed. Finally, the total flows $l_a\ \forall a \in A$ (Line 7). The cost of a solution is computed according to an evaluation function.

Any solution composed of integer weights generated within the range $[1{:}2^{16}]$ is a feasible solution in all telecommunication operation studies from Table 1. In fact, the interval [1:20] is adopted by most of the methods, since values are still within the range accepted by OSPF protocol and it was observed that with a larger range the results do not improve [15, 19]. If one or more arcs have their capacities surpassed, the solution evaluation will reflect this in a high objective function cost. In practice, in telecommunication networks a link overload means retransmission.

```
procedure EvaluateSolution(w)
1   forall t ∈ T do
2       π^t ← ReverseDijkstra(w);
3       g^t ← ComputeSPG(w, π^t);
4       δ^t ← ComputeDelta(g^t);
5       L^t ← ComputePartialLoads(μ, δ, π, g^t);
6   end forall
7   l ← ComputeTotalLoads(L);
8   Φ ← Σ_{a∈A} φ_a;
9   return(Φ);
end procedure
```

Fig. 1 Pseudo-code for a WSP solution evaluation procedure

In road networks, a link overload means high traffic. Thus, the problem allows the overload imposing a high cost in the objective function.

Even if any random solution is a feasible solution, it is hard to design a high-quality initial solution for this problem. Thus, most of the approaches use initial random solutions. However, two solution approaches were adopted in [9] that usually provides good initial solutions. One defines weights for each arc a proportional to the inverse of the arc capacity $1/c_a$. A second approach solves a relaxation of the problem considering the model with constraints (13), (14), (15), (16), (17), and (18), i.e., constraints that force the flow to take the shortest paths are not considered. The relaxation returns a set of fractional weights, which are rounded to the closest integer.

Local Search Heuristics for the WSP

The neighborhood used by most of the methods changes the weight of a single arc per time. In [19] an arc is chosen at random, and its weight is changed by a value selected at random in the interval [1:20]. Every time a weight value of an arc changes, the solution is reevaluated. To avoid recomputing solutions that were already evaluated, a hash function is used to detect when a solution was previously evaluated.

In [8] the local search also changes one arc weight per time but in a different fashion. Only the arcs with higher traffic have their weight changed. The initial step of the local search is to evaluate the current solution, given its weight vector. Then, the arcs with the higher traffic calculated according to ϕ_a have their weights changed, one arc at a time. However, the weight can only increase, one unit per time, while the solution is improving, or up to a specific weight defined by the algorithm. Instead of recomputing the whole shortest path graph from scratch, the graph and flow are only updated, to reflect the change. Moreover, unit changes are faster to compute than random changes [10]. Updating the graph is about 15 times faster than recomputing from scratch, as reported by the authors.

Virtual Network Embedded Problem (VNEP)

In the last few years, the amount of data routed through telecommunication networks has increased considerably. To attend the increasing demand, while maintaining a high quality of service for low prices, new enabling technologies are being deployed. One of them employs virtualizing environments which allow for multiple networks to simultaneously and transparently share the same physical structure. Likewise, service providers can offer customized services or co-location for expanded network presence [18]. Virtualization is a growing trend in the implementation of cloud computing architectures.

The virtual network embedding problem (VNEP) is central to achieve network virtualization. It consists of mapping virtual networks with heterogeneous architectures on physical infrastructures. Virtual nodes are mapped into physical substrate nodes, and virtual links are mapped into paths in the physical substrate network. Additionally, physical resources are finite and must be used judiciously: nodes have a processing capacity, and links have a bandwidth limit. The main objective of the VNEP is to minimize the use of the underlying resources such as bandwidth while respecting the mapping constraints. VNEP is a multicommodity flow problem since each virtual link corresponds to an OD pair since it consumes capacity of the substrate networks.

Several variants of VNEP have been proposed in the literature. For example, one virtual network can be mapped at a time [12], or several virtual networks are mapped simultaneously [23, 25]. Physical nodes and links can be restricted to host a limited number of virtual nodes and links, or specific virtual links can have a location restriction. Some applications have security restrictions which limit the subset of physical nodes and links that can be used for mapping [3]. Additional constraints can also be present such as delay [26], efficient use of energy [6], and redundancy [36]. Some of the VNEP variants allow path splitting. Others do not. When path splitting is enabled, virtual links are mapped to multiple paths in the substrate network.

The VNEP was shown to be NP-hard by a reduction from the bin packing problem [32]. Due to the difficulty of solving the VNEP, few exact algorithms have

been proposed in the literature. For most practical purposes, heuristic algorithms can provide good sub-optimal solutions for the problem.

Single VNEP Variant

This section describes the VNEP variant that maps a single virtual network into the substrate. Thus, a VNEP instance has as input a virtual network and a physical substrate network. The physical substrate is represented by an undirected graph $G^S = \left(V^S, A^S\right)$ with a CPU capacity of C_s for each physical node $s \in \mathrm{V^S}$ and a bandwidth capacity B_a for each arc $a \in A^S$. The virtual network is represented by an undirected graph $G^V = \left(V^V, A^V\right)$ along with a demand C_v for each virtual node $v \in V^V$ and a bandwidth demand of B_k for each virtual link $k \in A^V$.

The objective of the proposed problem is to find a feasible mapping of the virtual nodes and links onto the physical network with minimal cost. A feasible mapping is a pair of functions (f_v, f_e): a mapping of nodes $f_v : V^V \to \mathrm{V^S}$ and a mapping of links $f_{e=(w,u)} : A^V \to P$, where P is the set of paths in the substrate graph with endpoints $f_v(w)$ and $f_v(u)$. Each virtual node has to be mapped into a single substrate node with enough CPU capacity to host it. A substrate node can host at most one virtual node. Each virtual link $(w, u) \in A^V$ has to be mapped to a path in the physical graph between the nodes $f_v(w)$ and $f_v(u)$. An arc can host several virtual links, but the sum of their demands should not surpass the capacity of the arc. The cost of a mapping is the amount of bandwidth used in the physical network by the mapping.

Figure 2 presents an instance of the problem composed of a physical network with four nodes (left) and a virtual network with three nodes (center). Edges and nodes are labeled with their capacities or demands. The optimal mapping is shown on the right side of the figure. The optimal solution is to map node a to C, b to A, and c to B; the virtual link (a, b) is mapped to $C - B - A$, and the virtual link (b, c) is mapped to $A - B$. The cost of this solution is 50.

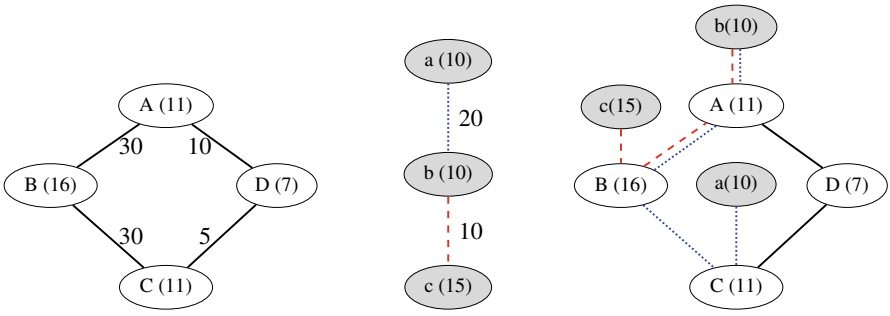


Fig. 2 An input instance for the VNEP (left and center) and the corresponding optimal solution (right)

An ILP model for VNEP with path splitting was presented in [13], while [2] considers a single path variant of the problem. Based on these models, [32] presented the ILP model below. Let the decision variables $x_{vs} = 1$ iff the substrate node s hosts the virtual node v. And let $y_{vwst} = 1$ iff the physical link (s, t) hosts the virtual link (v, w).

$$\min \sum_{(s,t)\in A^S} \sum_{(v,w)\in A^V} B_{vw}\, y_{vwst}$$

$$s.t. \sum_{v\in V^V} C_v x_{vs} \leq C_s \qquad \forall s \in V^S \tag{21}$$

$$\sum_{s\in V^S} x_{vs} = 1 \qquad \forall v \in V^V \tag{22}$$

$$\sum_{v\in V^V} x_{vs} \leq 1 \qquad \forall s \in V^S \tag{23}$$

$$\sum_{t\in V^S} (y_{vwst} - y_{vwts}) = x_{vs} - x_{ws} \qquad \forall (v, w) \in A^V, s \in V^S \tag{24}$$

$$\sum_{(v,w)\in A^V} B_{vw} y_{vwst} \leq B_{st} \qquad \forall (s, t) \in A^S \tag{25}$$

$$x_{vs} \in \{0, 1\} \qquad \forall v \in V^V, s \in V^S \tag{26}$$

$$y_{klmn} \in \{0, 1\} \qquad \forall (k, l) \in A^V, (m, n) \in A^S \tag{27}$$

The objective function minimizes the amount of bandwidth used. Constraints (21) ensure that the substrate capacities are not surpassed. Constraints (22) enforce that every virtual node is mapped to a substrate node, while (23) enforce that every substrate node hosts at most one virtual node. Constraints (24) ensure that every virtual link is mapped to a path into the substrate graph. Finally, constraints (25) ensure that the bandwidth capacities of physical edges are not surpassed.

Heuristic Approaches for the VNEP

Since the problem is NP-hard, and only small instances can be solved optimally [32], heuristics are a natural choice as solution approach to tackling this problem. In

this section, we discuss the main heuristic solution approaches adopted by different studies applied to variants of the problem.

Some authors use rounding schemes in an attempt to provide a solution for the problem. In [14] and [2] the authors present ILP models for different variants of VNEP, which they deemed impractical to solve optimally on large instances. To scale the instance sizes, they present rounding schemes applied to the relaxed solutions in order to obtain integer solutions. In both cases, a feasible solution is not guaranteed, but in practice the rounded solution is usually feasible.

In [14] a relaxation of the problem is solved by a solver, and a rounding algorithm is then applied to define to which substrate node each virtual node is mapped. Once nodes are mapped, a multicommodity flow algorithm is applied to determine the substrate path taken for each virtual link. The node mapping works as follows. One virtual node is mapped at a time. For each virtual node, a cluster comprised by a set of substrate nodes is defined according to a distance given as input. Among these nodes, one is selected for hosting the virtual node. The node is chosen according to the mapping variable and other input values.

In [2] the rounding scheme is different. Among the node mapping variables, the highest real value is rounded to 1, defining a mapping to a virtual node. Thus, this variable is fixed, and the relaxed problem is solved again. This procedure is repeated until all virtual nodes are mapped. In fact, four variations of this procedure were proposed in the paper. A similar procedure is applied to link mapping variables.

Considering metaheuristics, ant colony optimization is among the most used. Table 2 presents a summary of results found in the literature. The first column indicates whether the problem is the one defined in the previous section (basic) or if it has additional constraints. In this case, the type of additional constraints that impacts the solution of the problem the most is indicated. Column $|V^V|/|V^S|$ presents the number of virtual ($|V^V|$) and substrate nodes ($|V^S|$) of the largest network considered in the computational results for each reference indicated in the third column of the table. Finally, the last column presents the solution approach of each work indicated in the previous column. The solution approaches are simulated annealing (SA), rounding on the relaxed problem (LP-R), ant colony optimization (ACO), and branch and price (B and P).

Table 2 Solution approaches for solving the VNEP applied in different contexts

Additional constraints	$\|V^V\|$ / $\|V^S\|$	Reference	Solution approach
Node mapping	10/50	[13]	LP-R
QoS requirements	10/100	[17]	ACO
Repository of images	10/50	[2]	LP-R
Multiple mappings	–/35	[11]	ACO
Security	10/500	[4]	SA
Basic	8/80	[32]	B&P

Construction Heuristics for the VNEP

In [31, 32] the authors show that providing in polynomial time a solution that is guaranteed to be feasible is possible only if $P = NP$. Rounding schemes can also be used as construction heuristics. Thus, any proposed method generates solutions not guaranteed to be feasible. In [32] a construction heuristic was used at each node of the branch, bound tree aiming at improving the lower bound for that branch of the tree and then bounding branches earlier. The proposed heuristic works as follows. Each virtual node v is mapped to the free physical node s for which the value of $x_{v,s}$ is the largest. After all nodes are mapped, a breadth-first search is used to map virtual links to paths in the physical substrate graph. The mapping of edges can fail if no path with sufficient bandwidth is found.

In [4] a constructive heuristic is proposed and used by a simulated annealing metaheuristic algorithm. Virtual nodes are placed semi-randomly on substrate nodes, and then physical paths are allocated between these routers for each virtual link. The paths are calculated through Dijkstra's algorithm. The algorithm considers the weight of each physical link (i,j) to be the number of virtual links previously mapped to it and added by one. This weight calculation aims at favoring the selection of physical paths with higher amounts of free resources.

Local Search Heuristics for the VNEP

The most common neighborhoods used in this problem are the `swap` and `change`. In the `swap`, two virtual nodes swap the substrate nodes that host them. In the `change` neighborhood, a virtual node is moved to a substrate node which was not hosting any virtual node [4].

Conclusions

In this chapter, we discussed several network problems. A few computationally easy problems are presented since often they are the core of more complex problems. Moreover, often a small modification of a computationally easy problem can lead to an NP-hard problem. A fundamental multicommodity NP-hard problem was presented, and then two other real-world and more complex problems were discussed in detail: the weight setting problem (WSP) and the virtual network embedded problem (VNEP).

The WSP is used mainly for determining OSPF routing in telecommunication networks. However, it also finds applications in road networks, as the tollbooth problem described in this chapter. Some components of the main algorithms applied to solve the WSP in telecommunication networks were presented, and the problem was discussed. A similar approach was taken with regard to the VNEP. The problem was presented, motivated by components of proposed algorithms. Both are NP-hard network problems whose main solution methods rely on heuristics.

With the increase of the input data of almost all problems, the use of heuristics as a solution approach tends to increase. When designing a heuristic algorithm,

it is essential to justify the decisions taken by the algorithms proposed and the parameters used, and besides reporting the results, statistical analysis is always important to clarify the efficiency of the methods.

Cross-References

- ▸ Biased Random-Key Genetic Programming
- ▸ Evolutionary Algorithms
- ▸ Random-Key Genetic Algorithms: Principles and Applications

Competing Interest Declaration The author(s) has no competing interests to declare that are relevant to the content of this manuscript.

References

1. Ahuja RK, Orlin JB, Magnanti TL (1993) Network flows: theory, algorithms, and applications. Prentice-Hall, Upper Saddle River
2. Alkmim G, Batista D, da Fonseca N (2013) Mapping virtual networks onto substrate networks. J Internet Serv Appl 4:1–15
3. Bays L, Oliveira R, Buriol L, Barcellos M, Gaspary L (2012) Security-aware optimal resource allocation for virtual network embedding. In: Network and service management (CNSM), pp 378–384
4. Bays L, Oliveira R, Buriol L, Barcellos M, Gaspary L (2014) A heuristic-based algorithm for privacy-oriented virtual network embedding. In: 14th IEEE/IFIP network operations and management symposium (NOMS 2014), pp 1–8
5. Bley A (2011) An integer programming algorithm for routing optimization in IP networks. Algorithmica 60(1):21–45
6. Botero J, Hesselbach X, Duelli M, Schlosser D, Fischer A, de Meer H (2012) Energy efficient virtual network embedding. IEEE Commun Lett 16(5):756–759
7. Broström P, Holmberg K (2006) Multiobjective design of survivable IP networks. Ann Oper Res 147:235–253
8. Buriol L, Resende M, Ribeiro C, Thorup M (2005) A hybrid genetic algorithm for the weight setting problem in OSPF/IS-IS routing. Networks 46(1):36–56
9. Buriol L, Resende M, Thorup M (2007) Survivable IP network design with OSPF routing. Networks 49(1):51–64
10. Buriol L, Resende M, Thorup M (2008) Speeding up dynamic shortest-path algorithms. INFORMS J Comput 20:191–204
11. Cao W, Wang H, Liu L (2014) An ant colony optimization algorithm for virtual network embedding. In: Algorithms and architectures for parallel processing. Lecture notes in computer science, vol 8630, pp 299–309
12. Chowdhury N, Boutaba R (2010) A survey of network virtualization. Comput Netw 54(5):862–876
13. Chowdhury NMMK, Rahman MR, Boutaba R (2009) Virtual network embedding with coordinated node and link mapping. In: INFOCOM. IEEE, pp 783–791
14. Chowdhury M, Rahman M, Boutaba R (2012) ViNEYard: virtual network embedding algorithms with coordinated node and link mapping. IEEE/ACM Trans Networking 20(1):206–219
15. Ericsson M, Resende M, Pardalos P (2002) A genetic algorithm for the weight setting problem in OSPF routing. J Comb Optim 6:299–333

16. Even S, Itai A, Shamir A (1976) On the complexity of timetable and multicommodity flow problems. SIAM J Comput 5:691–703
17. Fajjari I, Aitsaadi N, Pujolle G, Zimmermann H (2011) VNE-AC: virtual network embedding algorithm based on ant colony metaheuristic. In: IEEE international conference on communications (ICC), pp 1–6
18. Feamster N, Gao L, Rexford J (2007) How to lease the internet in your spare time. SIGCOMM Comput Commun Rev 37(1):61–64
19. Fortz B, Thorup M (2000) Internet traffic engineering by optimizing OSPF weights. In: INFOCOM, pp 519–528
20. Fortz B, Thorup M (2004) Increasing internet capacity using local search. Comput Optim Appl 29(1):13–48
21. Goldberg A, Tarjan R (1988) A new approach to the maximum-flow problem. J ACM 35:921–940
22. Goldberg A, Kaplan H, Werneck R (2007) Better landmarks within reach. In: Workshop on experimental algorithms, pp 38–51
23. Guerzoni R, Trivisonno R, Vaishnavi I, Despotovic Z, Hecker A, Beker S, Soldani D (2014) A novel approach to virtual networks embedding for SDN management and orchestration. In: Network operations and management symposium (NOMS 2014). IEEE, pp 1–7
24. Hoffman A, Kruskal J (1956) Integral boundary points of convex polyhedra. Ann Math Stud 38:223–246
25. Houidi I, Louati W, Ameur W, Zeghlache D (2011) Virtual network provisioning across multiple substrate networks. Comput Netw 55(4):1011–1023
26. Inführ J, Raidl G (2011) Introducing the virtual network mapping problem with delay, routing and location constraints. In: Pahl J, Reiners T, VoßS (eds) Network optimization. Lecture notes in computer science, vol 6701. Springer, Berlin/Heidelberg, pp 105–117
27. Kleinberg J, Tardos E (2005) Algorithm design. Addison Wesley, Boston
28. Köhler E, Mühring R, Schilling H (2006) Fast point-to-point shortest path computations with arc-flags. In: 9th DIMACS implementation challenge
29. Lauther U (2006) An experimental evaluation of point-to-point shortest path calculation on road networks with precalculated edge-flags. In: 9th DIMACS implementation challenge
30. Likhachev M, Ferguson D, Gordon G, Stentz A, Thrun S (2008) Anytime search in dynamic graphs. Artif Intell 172:1613–1643
31. Moura L (2015) Branch & price for the virtual network embedding problem. Master's thesis, Federal University of Rio Grande do Sul, Porto Alegre
32. Moura L, Gaspary L, Buriol LS (2017) A branch-and-price algorithm for the single-path virtual network embedding problem. Netw Online 1–15. https://doi.org/10.1002/net.21798
33. Orlin J (2013) Max flows in o(nm) time, or better. In: Proceedings of the forty-fifth annual ACM symposium on theory of computing, pp 765–774
34. Pioro M, Szentsi A, Harmatos J, Juttner A, Gajownicczek P, Kozdrowski S (2002) On open shortest path first related network optimization problems. Perform Eval 48(4):201–223
35. Ramalingam G, Reps T (1996) An incremental algorithm for a generalization of the shortest-path problem. J Algorithms 21:267–305
36. Shamsi J, Brockmeyer M (2008) Efficient and dependable overlay networks. In: IEEE international symposium on parallel and distributed processing (IPDPS), pp 1–8
37. Stefanello F, Buriol L, Hirsch M, Pardalos P, Querido T, Resende M, Ritt M (2017) On the minimization of traffic congestion in road networks with tolls. Ann Oper Res 249:119–139

Optimization Problems, Models, and Heuristics in Wireless Sensor Networks

50

Vinicius Morais, Fernanda S. H. Souza, and Geraldo R. Mateus

Contents

Introduction 1542
Problem Definition 1545
Optimization Models 1547
Coverage and Density Control 1547
Routing or Connectivity 1548
Clustering and Mobile Sink 1549
Heuristic Methods 1551
Sensor Location, Placement, and Coverage 1552
Density Control, Clustering, Routing, and Sink Mobility 1553
Integrated Problems 1555
Conclusion 1556
Cross-References 1557
References 1557

Abstract

This chapter provides an overview and a comprehensive discussion of problems, models, algorithms, and applications in a vast and growing literature of wireless sensor networks. Being a particular kind of ad hoc network, many power management and communication protocols may be designed specifically for those networks. The critical issues considered in these protocols are the objectives, the

V. Morais (✉) · G. R. Mateus
Departamento de Ciência da Computação, Universidade Federal de Minas Gerais, Belo Horizonte, Brazil
e-mail: vwcmorais@dcc.ufmg.br; mateus@dcc.ufmg.br

F. S. H. Souza
Departamento de Ciência da Computação, Universidade Federal de São João del-Rei, São João del-Reil, Brazil
e-mail: fsumika@ufsj.edu.br

© Springer Nature Switzerland AG 2025

R. Martí et al. (eds.), *Handbook of Heuristics*,
https://doi.org/10.1007/978-3-032-00385-0_53

quality of communication, the energy consumption, and the network lifetime. Moreover, due to the large-scale aspect inherent in some applications, traditional exact solution approaches are not practical, so heuristics may be adopted instead. The chapter starts by introducing the main concepts in the design of WSN and a wide range of problems and applications. Basic formulations and algorithms are also discussed, together with their benefits and drawbacks.

Keywords

Optimization problems · Heuristics wireless sensor networks

Introduction

Wireless sensor networks (WSN) are a special kind of ad-hoc networks used to monitor events and phenomena in a given place [1, 2]. They have become popular due to their wide applicability in many different areas such as environmental (wildlife and environment), industrial (mines, production, manufacture), and health, in order to monitor temperature, humidity, pressure, and movement [3–5]. The study of such networks has been a fruitful area of research over the years and continues to enable the permanent evolution of technologies.

WSN consist of autonomous, small, and compact devices capable of performing activities such as sensing, processing, and communication. These devices are sensor nodes, and among them, may have one or more special nodes called sink, which is responsible for collecting the sensed data and managing the network. The sensors are made up of sensing boards, processor, communication radio, and battery. Because of their purposes, size, and low cost, they have serious restrictions of energy, low performance, and small radius of communication. For networks installed in areas with difficult access, it becomes unviable exchange or recharge the battery of a sensor. These characteristics imply a limited lifetime of the sensors and of the network itself.

The sensors are used to monitor a working area within a certain radius of coverage, besides being able to exchange information with other sensors and sinks. Two major communication patterns are often adopted: data collection and data dissemination. Data collection consists of sensors sending the collected data to a sink, while data dissemination concerns information from a sink being sent to the sensors. Those communication processes can be performed through one (single) or more (multi) hops or links. In this chapter whenever referring to the transmission of information, both senses are being considered. A sink aggregates the information received from sensors and usually has unlimited or renewable energy. It can be static or mobile, visiting the monitored region and collecting information from the sensors.

The sensor nodes are generally static but may also have mobility [6–8]. *Mobile WSN* consist of a number of sensors that can move on their own and interact with the physical environment. A sensor in a mobile network can also sense, process

information, and communicate like a static node. Mobility may be applied to the whole network or only to a subset of sensors. The degree of mobility can vary, including constant movement of the sensors or alternating periods of mobility/immobility. Those factors impact directly the network dynamics. Alongside with mobile networks, there are four more types of networks, which are distinguished mainly with respect to the place where the sensors are deployed. The sensors can be deployed on land, underground, and underwater [2]. A *terrestrial network*, for instance, consists of several sensors deployed on land, either in an ad-hoc or in a preplanned manner. In an *underground network*, the sensors are placed in caves, mines, or underground, while on an *underwater network*, the devices are deployed into rivers or oceans. Among these networks one can define a *multimedia network*, where the sensors are used to handle multimedia data. While a dense deployment of sensors is often employed in a terrestrial WSN, in an underwater WSN, a sparse deployment is used. The maintenance cost and difficulty of implementation of each network are directly dependent on the application constraints.

The classification of WSN depends primarily on their application. They are commonly classified into monitoring and tracking categories. Monitoring applications include patient monitoring in medical centers, security detection, habitat monitoring, power, inventory location, factory and automation processes, seismic and structural monitoring in the industrial or public context, and home-office environmental monitoring to provide basic services to smart environments, among others. Tracking applications include military missions, wildlife tracking, and traffic control. Consequently, the technologies used for each WSN are directly dependent on the network type. Refer to Yick et al. [2] for a survey of sensor technologies in practical situations.

WSN can also be classified into homogeneous or heterogeneous. WSN are said to be homogeneous, when composed by devices with the same hardware capabilities (processor, memory, battery, and communication device features). Conversely, WSN are heterogeneous when composed by devices with different capabilities. Another possibility to classify WSN is based on their organization. The networks can be hierarchical or flat. A sensor network is hierarchical when nodes are grouped for the purpose of communication, otherwise it is flat.

The lifetime of a WSN can be defined as the network operation time until the first sensor fails due to lack of energy. This chapter does not treat cases of failure for other reasons, such as mechanical failure or if a sensor is destroyed. Another definition of lifetime is given by the monitoring time until the coverage of the entire working area and network connectivity are not guaranteed.

Many factors influence the design of WSN. Fault tolerance, scalability, energy consumption, production cost, environment, network topology, hardware constraints, and transmission rate are some points to be considered. Due to the low battery capacity of the sensors, the energy consumption is one of the most important constraints on WSN [9]. In this context several problems are addressed to ensure coverage, monitoring, and connectivity, aiming to maximize the network lifetime within a minimum quality standard or quality of service (QoS) (QoS usually refers to quality as perceived by the user and/or application). In spite of

energy consumption being the most critical factor in WSN design, other parameters such as number of sensors, coverage, and network connectivity must also be considered.

The problems that appear in the context of WSN are studied independently or in an integrated manner. In general, to define them it is supposed that the working area to be covered is divided into small squares which represent points or demand nodes that require coverage by at least one sensor. This is a well-known approach applied in wireless network design. Each point concentrates the demand of its square. A point is considered covered by a sensor when the referred sensor is able to monitor it and if the point is within the radius of coverage of that sensor. The smaller are the square dimensions, the closer they are to describing continuous area. In the literature there are several works that do not make use of this discretization process, opting to explore the continuous space. Another assumption usually found in the literature is the use of unit disk graph to model the communication among sensors. A homogeneous transmission range for the sensors is considered and taken as the disk radius. Many works employ this model, although transmission ranges can be heterogeneous and also not defined as perfect disks. However, due to its theoretical simplicity, it is commonly applied. An evolution of this model, named quasi unit disk graph is proposed in [10].

Many algorithms can be devised to deal with WSN problems. Regarding optimality guarantees, they can be divided into exact approaches and heuristics. The former comprise algorithms developed to solve mathematical models based on integer linear programming (ILP), while the latter include proposals which relieve optimality conditions in order to provide scalability. Also, algorithms can perform in a centralized or distributed fashion. Centralized algorithms concern decision-making is done by a single entity in the network owning global information of the entire network. This entity is usually a sink node which is able to disseminate the solution for all sensors in the network. On the other hand, distributed algorithms indicate that many entities in the network take a joint decision based only on local information. Global information can improve the optimization process but is not always available and may not be applied in practice due to the overhead not allowed in real-time applications. Local information seems more reasonable in practice, but suffers from lower-quality solutions compared to the ones with global information. An alternative to take advantage of both methods is to use clustering, which can optimize locally inside each cluster. Thus, both exact methods and heuristics can be accomplished in such hybrid solutions. Distributed algorithms concern a wide topic of research [11–14] and are out of scope of this chapter.

The remainder of the chapter is organized as the following. A brief definition of classical problems appearing in WSN is presented in section "Problem Definition". Mixed integer linear programming formulations for these problems are outlined in section "Optimization Models". The aim is to formally describe some problems and give some intuition about exact solution approaches. Heuristic methods applied on the design of WSN are addressed in section "Heuristic Methods". Finally, the chapter is closed in section "Conclusion".

Problem Definition

To monitor inhospitable and inaccessible environments, sensors are deployed in an ad-hoc manner, launched from an aircraft or an unmanned vehicle. Thus, it is not possible to know beforehand their exact locations. However, knowing the location of these sensors is generally important to route data and to ensure connectivity. The problem of defining the positions of the sensors is known as *localization problem*. Some localization methods include Global Positioning System (GPS), beacon (anchor) nodes, and proximity-based localization that make use of the coordinates of neighbor nodes to determine their own localization. A review of location problems in WSN is given in [15].

It is straightforward to note that a fundamental requirement in the design of WSN is to ensure coverage of the working area. This problem is named as the *coverage problem*. The coverage level can be total or partial. In general, full coverage is often preferred. Therefore, the coverage problem chooses a set of active sensors such that each point is within a range of at least one sensor. The loss of coverage in at least one point can be characterized as end of the network lifetime.

To monitor accessible environments, the sensors may be optimally deployed in a preplanned manner. The *placement (deployment) problem* seeks to minimize the number of sensors deployed in order to guarantee connectivity and coverage of the entire monitoring area. The placement problem is a variant of the well-known art gallery problem [16], the problem of placing the fewest number of guards in a given area such that the surveillance is guaranteed. For the following problems, a usual assumption is to consider that the location of the sensors is known.

Another fundamental requirement in the design of WSN is the connectivity. The data collected by the sensors must be delivered to the sinks. Thus, routes must be established connecting each active sensor to a sink. A feasible route consists of links between pairs of sensors/sinks that respect the communication radius of each node. The fewer is the number of links, the smaller is the energy consumption. The *routing problem* aims to find feasible routes from each active sensor to the sinks, or vice versa, while optimizing a given resource. A solution for the routing problem consists of rooted trees, centered at the sinks, spanning a subset of active sensors. Automatically, the sensors nearest to the sinks may spend too much energy given the number of routes traversing those nodes. Such a drawback is known as the sink neighborhood problem [17]. There are two kinds of routing methods: reactive routing methods and proactive routing methods. In reactive methods the routes are calculated just in time. In proactive methods the routing tables are created and stored regardless of the moment the routes are used. However, in networks with large number of sensors, storing the routing tables can lead to excessive usage of memory. To overcome this problem, the sensors can be clustered and communication hierarchies be defined.

The sensors consume energy to perform communication, process, and aggregate data, besides maintenance, activation, or deactivation processes. There are several ways of dealing with the energy consumption. One possibility to extend the network

lifetime is to use a large number of sensors to ensure the desired coverage. However, to avoid using too much energy at a redundant coverage, the sensors do not perform at the same time. In this case, each sensor can be active, working on the network, or stay on standby, sleep mode, since energy consumption is extremely low in this mode. When active sensors fail due to depletion of their batteries, new sensors must be activated. The optimization problem arising in this context is the *density control problem* (DCP). The purpose of DCP is to schedule a sequence of activation and deactivation of sensor nodes in order to minimize the energy consumption and maximize the network lifetime. The network lifetime comes to an end when the network requirements are no longer ensured.

All the problems above have been considered for flat WSN. Whereas the communication among sensors and sinks is one of the most expensive operations in terms of energy consumption, an alternative to flat networks is to set hierarchies. In hierarchical networks, the sensors are clustered, and a single sensor is elected as the *cluster-head* in each cluster. It is up to the cluster-head aggregating and transmitting the information directly to a sink or through other cluster-heads. In such a *clustering problem*, from time to time, the cluster-head role can be swapped among the sensors within the same cluster to avoid premature failure due to lack of energy. Some classical combinatorial optimization problems such as p-median and p-center may apply in a clusterization scheme.

In the routing problem for hierarchical WSN, two kinds of routes are considered: *intra-cluster* route and *inter cluster* route. In intra-cluster routes the communication occurs among sensors and cluster-head within the same cluster through single or multi-hop strategies. In intercluster routes the connections are among cluster-heads and sinks. In both levels a tree topology must be defined to transmit the information from the sensors to their cluster-heads and from the cluster-heads to the sinks.

In some applications the mobility of the sinks is explored to gather sensed information through the network. This approach prevents the sensors to spend their limited energy in relaying of messages through a long path until the sinks. However, the message delivery latency can increase significantly, thanks to the long route traversed by a sink to visit the whole working area. In order to go around of such problem, clustering schemes must be used so that each sink only visits a reduced set of sensors (for instance, the cluster-heads). In this context, the communication between sensors and sink is only possible through a cluster-head using multi-hop or single-hop approaches to define the *intra-cluster* routes. Classical optimization problems such as shortest path and traveling salesman problem (TSP) are used to model the trails traversed by the mobile sinks.

Figure 1 illustrates optimization problems in WSN. Different tasks and objectives may be desired requiring specific approaches to deal with the problems described. In Fig. 2, for instance, two different approaches are presented exploring the mobility of the sink. In the first approach, a mobile sink visits each sensor in the working area to collect sensed data. This problem can be modeled with the TSP. Another alternative is to define a reduced TSP route, defining a subset of sensors to be visited by the mobile sink such that all the other sensors are close enough to establish the communication. This approach can be seen as a TSP by neighborhood.

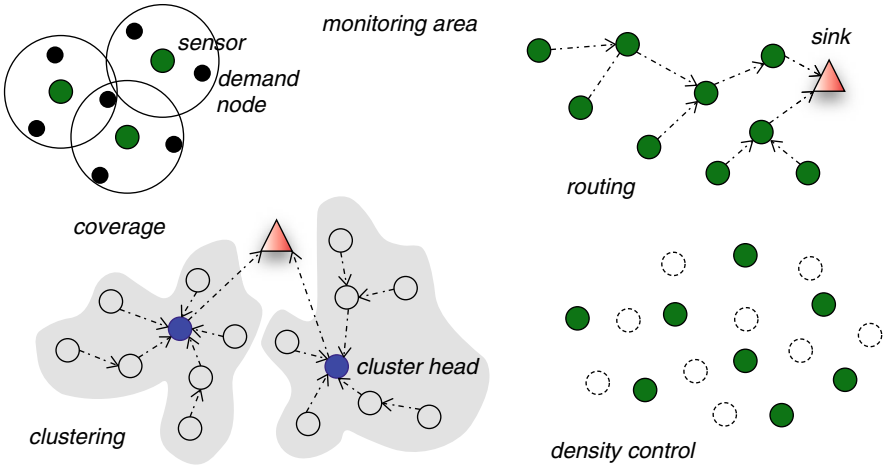


Fig. 1 Illustration of classical optimization problems in WSN

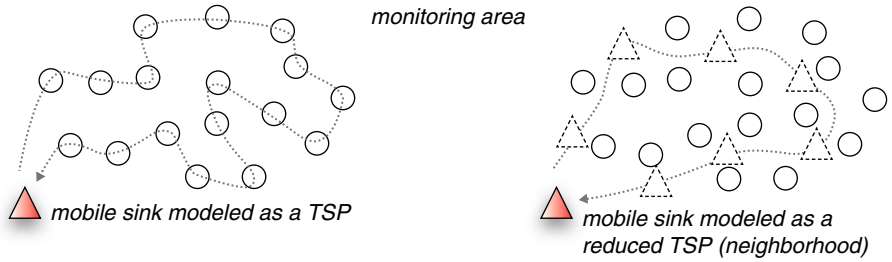


Fig. 2 Illustration of mobile sink approaches modeled with the TSP

Optimization Models

As presented earlier, many optimization problems arise in the design of WSN. Many of these problems are variants of classical combinatorial optimization problems that can be formulated as linear mixed-integer programming (MIP) models [18]. In the following, mathematical models for some of these problems are presented.

Coverage and Density Control

Mathematical formulations for coverage and density control problems can be found in [19–21]. The models consider the following notation: $S = \{1, \ldots, n\}$ is the set of sensors; $P = \{1, \ldots, p\}$ is the set of demand points to be covered by the sensors; matrix $A(n \times p)$ indicates whether a sensor $i \in S$ is able to cover ($a_{ij} = 1$) or not ($a_{ij} = 0$) the point $j \in P$ regarding the sensing range R_s^i. To each sensor $i \in S$, an activation energy E_a^i is also assigned.

A formulation to the coverage and density control problems uses binary variables $\{x_i \in \mathbb{B} : i \in S\}$ and $\left\{y_{ij} \in \mathbb{B} : i \in S,\ j \in P\right\}$ to define a subset of sensors selected to be active and the coverage relationship between S and P. If $x_i = 1$ holds, sensor $i \in S$ is active. If $y_{i,j} = 1$, demand point $j \in P$ is covered by sensor $i \in S$. Then, the coverage and density control formulation is given by:

$$\min \left\{ \sum_{i \in S} E_a^i x_i : (x, y) \in (2) - (5) \right\} \tag{1}$$

$$\sum_{i \in S} a_{ij} \ \ y_{ij} \geq 1 \qquad \forall j \in P; \tag{2}$$

$$a_{ij} \ \ y_{ij} \leq x_i \qquad \forall i \in S, \qquad \forall j \in P; \tag{3}$$

$$y_{ij} \in \{0, 1\} \qquad \forall i \in S, \qquad \forall j \in P; \tag{4}$$

$$x_i \in \{0, 1\} \qquad \forall i \in S. \tag{5}$$

Constraints (2), (3), (4), and (5) assure that active sensors cover the set of demand points. Constraints (2) guarantee that every demand point $j \in P$ is covered by at least one sensor $i \in S$. Constraints (3) ensure that only active sensors can provide the coverage of demand points. Constraints (4)–(5) impose variables to be binary. Finally, objective function (1) minimizes the energy consumed by the activation of sensor nodes.

Routing or Connectivity

Mathematical models exploring connectivity in the context of WSN are described in [19, 21, 22]. The models consider a set of static sinks $M = \{1, \ldots, m\}$ and a communication range (R_c^i) associated with each sensor $i \in S$. The matrix $B(n \times (n + m))$ indicates whether a sensor or sink $i \in S$ and $j \in S \cup M$ relies $(b_{ij} = 1)$ or not $(b_{ij} = 0)$ within the communication range R_c^i of each other. To each sensor $i \in S$ is also assigned the routing energy (E_r^i) required to communicate with other sensors or sinks.

Let $\left\{z_{jk}^i \in \mathbb{B} : i \in S,\ j \in S, k \in S \cup M\right\}$ denote decision variables used to indicate whether the sensors $j \in S$ and $k \in S \cup M$ belong $(z_{jk}^i = 1)$ or not $(z_{jk}^i = 0)$ to the communication path from sensor $i \in S$ to a sink node k. Alongside with the decision variables x_i defined in the first model, the routing problem, modeled as routing trees, can be formulated as:

$$\min \left\{ \sum_{i \in S} E_a^i \ \ x_i + \sum_{i \in S} \sum_{j \in S} \sum_{k \in S \cup M} E_r^i \ \ z_{jk}^i : (x, z) \in (7) - (12) \right\} \tag{6}$$

$$\sum_{k \in S \cup M} b_{jk} \; z^i_{jk} \leq x_j \quad \forall i \in S \backslash \{k\}, \quad \forall j \in S \backslash \{k\}; \tag{7}$$

$$\sum_{j \in S} b_{jk} \; z^i_{jk} \leq x_k \quad \forall i \in S \backslash \{k\}, \quad \forall k \in S \backslash \{j\} \cup M; \tag{8}$$

$$\sum_{j \in S} b_{jk} \; z^i_{jk} - \sum_{l \in S \cup M} b_{kl} \; z^i_{kl} = 0 \quad \forall i \in S, \qquad \forall k \in S \backslash \{i\}; \tag{9}$$

$$- \sum_{l \in S \cup M} b_{kl} \; z^i_{kl} = -x_i \qquad \forall i \in S, i = k; \tag{10}$$

$$z^i_{jk} \in \{0, 1\} \qquad \forall i \in S, \, j \in S, k \in S \cup M \tag{11}$$

$$x_i \in \{0, 1\} \qquad \forall i \in S. \tag{12}$$

Here, constraints (7), (8), (9), (10), (11), and (12) ensure connectivity among sensors and sinks. Constraints (7)–(8) assure that communication is only allowed among active sensors. Constraints (9) guarantee the flow balance for transshipment nodes. Constraints (10) indicate that the flow originates at active sensors. Constraints (11)–(12) impose the domain of the variables. Finally, the objective function (1) minimizes the routing and activation energies consumed.

Clustering and Mobile Sink

Valle et al. [23] proposed an integrated model for clustering and routing problems. The model considers a set of mobile sinks $M = \{1, \ldots, m\}$ and a set of arcs $A = \{(i, j), (j, i) : \forall i, j \in S, \, i \neq j\}$. Each arc $(i, j) \in A$ denotes a possible point-to-point movement for a sink. Additionally, d_{ij} is the Euclidean distance between i and $j \in S$. It is assumed that all sinks start from an initial sensor $i = 1$ and return to an artificial node 0, such that $\bar{A} = A \cup \{(i, 0) : \forall i \in S \backslash \{1\}\}$ and $\{(i, 0) : d_{i0} = d_{1i}, \forall i \in S \backslash \{1\}\}$. It is important to point out that each cluster-head is visited only once and by exactly one sink; a sensor $i \in S$ must be a cluster-head or be covered by a cluster-head; at least one cluster-head should belong to the set of neighbors of sensor i: $N(i) = \left\{ j \in S : d_{ij} \leq R^i_c \right\}$.

In order to model the integrated clustering and routing problem, assume the following decision variables: $\left\{ r^m_i \in \mathbb{B} : m \in M, i \in S \right\}$, taking value $r^m_i = 1$ if sensor $i \in S$ is a cluster-head in the route of the sink $m \in M$, otherwise $r^m_i = 0$; $\left\{ w^m_{ij} \in \mathbb{B} : m \in M, (i, j) \in \bar{A} \right\}$ that takes value $w^m_{ij} = 1$ if arc $(i, j) \in \bar{A}$ belongs to route of the sink $m \in M$, otherwise $w^m_{ij} = 0$; $\left\{ v^m_{ij} \in \mathbb{R}_+ : m \in M, (i, j) \in \bar{A} \right\}$, indicating the flow, originated at m, traversing arc $(i, j) \in \bar{A}$; and, finally, $\{h \in \mathbb{R}_+\}$

that denotes the longest length among all routes. The integrated clustering and routing problems can be formulated by the following MIP model:

$$\min \{h : (r, w, v, h) \in (14) - (27)\} \tag{13}$$

$$\sum_{i \in S \setminus \{1\}} v_{1,i}^m = \sum_{i \in \{0\} \cup S \setminus \{1\}} r_i^m \quad \forall m \in M; \tag{14}$$

$$\sum_{j \in \{0\} \cup S \setminus \{1\}} v_{ij}^m - \sum_{j \in S} v_{ji}^m = -r_i^m \quad \forall i \in S \setminus \{1\}, \quad \forall m \in M; \tag{15}$$

$$\sum_{i \in S} v_{i,0}^m = 1 \quad \forall m \in M; \tag{16}$$

$$v_{ij}^m \leq n w_{ij}^m \quad \forall (i, j) \in \bar{A}, \quad \forall m \in M; \tag{17}$$

$$w_{ij}^m \leq r_i^m \quad \forall (i, j) \in \bar{A}, \quad \forall m \in M; \tag{18}$$

$$w_{ij}^m \leq r_j^m \quad \forall (i, j) \in \bar{A}, \quad \forall m \in M; \tag{19}$$

$$\sum_{m \in M} r_i^m \leq 1 \quad \forall i \in S \setminus \{1\}; \tag{20}$$

$$\sum_{m \in M} \sum_{j \in N(i)} r_j^m + \sum_{m \in M} r_i^m \geq 1 \quad \forall i \in S \setminus \{1\}; \tag{21}$$

$$\sum_{j \in S \cup \{0\}} w_{ij}^m \leq 1 \quad \forall i \in S, \qquad \forall m \in M; \tag{22}$$

$$h \geq \sum_{(i,j)} \in \bar{A}\, d_{ij} w_{ij}^m \quad \forall m \in M; \tag{23}$$

$$r_1^m = r_0^m = 1 \quad \forall m \in M; \tag{24}$$

$$r_i^m \in \{0, 1\} \quad \forall i \in S \cup \{0\}, \quad \forall m \in M; \tag{25}$$

$$v_{ij}^m \geq 0 \quad \forall (i, j) \in \bar{A}, \quad \forall m \in M; \tag{26}$$

$$w_{ij}^m \in \{0, 1\} \quad \forall (i, j) \in \bar{A}, \quad \forall m \in M. \tag{27}$$

Constraints (14), (15), (16), (17), (18), (19), (20), (21), (22), (23), (24), (25), (26), and (27) ensure the integration of clustering and routing problems using mobile sinks. Constraints (14), (15), and (16) guarantee the flow conservation for nodes {1}, $S\backslash\{1\}$ and {0}, respectively. Note that constraints (14) assure that the flow of commodity m starting at node 1 is equal to the number of cluster-heads to be visited in the route plus the artificial node {0}. On the other hand, constraints (15) impose that each node i visited by $m \in M$ must consume a unit of flow. Inequalities (17), (18), and (19) are coupling constraints, ensuring that commodities only traverse arcs selected for the route and arcs of the route have endpoints consuming a unit of flow. Constraints (20) guarantee that a cluster-head is visited once by a single route. Constraints (21) assure that each sensor is a cluster-head or has a cluster-head as neighbor. Taken together, constraints (14), (15), (16) and (22) guarantee the topology of selected arcs induce M elementary routes connecting vertices 1 and 0. Constraints (23) define the longest length among all routes to be minimized by the objective function (13). Constraints (24) impose that the starting node 1 and the final artificial node 0 are part of all routes. Finally, constraints (25), (26), and (27) define the domain of variables.

Models (1), (6), and (13) can be implemented in optimization packages and solved with a branch-and-bound (BB) algorithm. However, the main drawback of this approach is that only instances of limited size are solvable in feasible computational time. Since WSN are designed as large-scale networks, it is not expected that a traditional BB is able to solve the problem to optimality. Therefore, one must resort to heuristic-based methods to tackle the problems.

Heuristic Methods

This section draws attention to heuristic-based methods used in the design of WSN. In general, heuristics are designed to face challenging and complex optimization problems in which exact optimization methods fail, especially when dealing with time-constrained online applications that are combinatorial in nature. Due to the practical and technological relevance of WSN, heuristics are preferentially the methods to be used. However, each heuristic is designed to tackle a specific problem. So, a given approach may not be generalized to solve different problems. Motivated by the need of general solution strategies, optimization researches have been focusing on the study of metaheuristics. A metaheuristic combines intensification and diversification procedures to perform guided searches through one or more neighborhood structures. Some metaheuristics used in the design of WSN are greedy randomized adaptive search procedure (GRASP) [24], path relinking (PR) [25], simulated annealing (SA) [26], iterated local search (ILS) [27], tabu search (TS) [28], variable neighborhood search (VNS) [29, 30], genetic algorithms (GA) [31], memetic algorithms (MA) [32], ant colony optimization (ACO) [33], and particle swarm optimization (PSO) [34]. For supplementary reading, Glover and Kochenberger [35] are referred to provide a good overview of the most popular metaheuristics. In addition to [36–40], here, the reader can find some hints of

how to use heuristics to solve practical WSN optimization problems when studied independently or in an integrated manner.

Sensor Location, Placement, and Coverage

As indicated above, the design of WSN has to concern with several classical problems. These include localization, placement, coverage, density control, clustering, routing, and sink mobility. In general, the purpose of solving these problems is to maximize the network lifetime, keeping connectivity, ensuring coverage, and reducing cost. Many existing applications require information about the geographic position of each node in the network. So, the localization problem must be solved first. The heuristics proposed in the literature for localization are generally based on GPS system, beacon (anchor) nodes, and proximity-based localization. These methods make use of the position of neighbor sensors, obtained during the installation of the network, to determine the position of a sensor. Molina and Alba [41], for instance, proposed heuristics based on SA, GA, and PSO to solve the localization problem. Their heuristics determine the sensor nodes locations using the trilateration technique. The global objective of their heuristic is to minimize the measured distance error comparing the node-to-node distances with the distance measured when using GPS. Shahrokhzadeh et al. [42] proposed a centralized (solved by a sink) SA-based heuristic for solving sensor nodes localization problem. Their heuristic is based on Euclidean distance and mathematical techniques, such as trilateration and triangulation.

The placement of the sensors is a key factor in the design of WSN. The deployment may be an activity performed during the installation of a network or may also be a continuous process to replace failed sensors or to improve the coverage over a certain area. As stated above, the placement problem seeks to minimize the number of sensors deployed in order to guarantee connectivity and coverage of the entire monitoring area. A number of deterministic greedy heuristics can be described to determine the minimum number of sensor needed. In general, those methods try to anticipate the sensor deployment in all candidates' points and keep the best position selected. Brazil et al. [43] modeled the placement problem as a version of the Steiner problem and solved it by means of a greedy-based heuristic. Sasikumar et al. [44] proposed two-phase heuristics for placing sensors in a heterogeneous network, called nearest to base station and max residual capacity. In this work, they separate the nodes on: sensors, relay nodes (RN), and base station (sink node). The RN node performs functions equivalent to a cluster-head, but may have unlimited energy. In the first phase, their heuristics place the RN nodes, wherein the placement problem is modeled as the minimum set covering problem. Then the sensors are placed on the second phase with a greedy criterion based either on the distance to the RN nodes or on residual energy remaining on the node. To solve the sink placement problem, Laszka et al. [45] provided a GA-based heuristic. SA-based heuristics for placement and coverage problems were also developed in [46].

Deschinkel [47] concentrates on a centralized scheme to solve the coverage problem, where a large number of sensors are randomly deployed in the monitoring area. The problem was modeled as the non-disjoint cover sets problem and solved by means of a column generation (CG) heuristic. In this procedure the subproblem is solved with a heuristic for the classical set covering problem that uses the dual multipliers from the master problem. They compare the results obtained by the heuristic with that provided by a MIP solver. Cardei et al. [48] also deal with coverage problem. The interesting aspect of their work is given when the authors consider the property that each sensor has adjustable sensing ranges. To solve the problem, a greedy-based heuristic was used.

Density Control, Clustering, Routing, and Sink Mobility

The solution algorithms for the placement problem may define a dense network. This characteristic increases the robustness against failures, however, leads to a large redundancy that can result in congestion and waste of energy. To overcome this problem, the density control problem is used to select a subset of sensors to be active while keeping the others inactive (sleep mode), which allows the reduction of the initial topology. Delicato et al. [49] formulated the DCP problem as a knapsack problem. In this approach, the expected time horizon is divided in small time periods. For each period, a subset of sensors must be activated. By formulating the DCP as a knapsack problem, each period is equivalent to a knapsack, so the problem is to find a set of nodes (items) to keep active (to be inserted in a knapsack) per period. In their model, coverage and connectivity are also considered. To solve the problem, a greedy-based heuristic was proposed. In this heuristic the sensors are sorted by their residual energy and then, following the given preference, the sensors are set to be active (inserted in a knapsack).

An interesting and effective multi-start (MS)-based heuristic to solve the density control problem is given by Karasabun et al. [50]. Their heuristic is composed by two phases, a constructive routine to define a set of initial active sensors and an iterative improving local search procedure. To ensure the network connectivity, the Steiner problem is solved over the active sensor nodes.

Clustering is another optimization problem considered when defining the network topology. The clustering problem directly contributes for the scalability of networks. Many existing protocols for defining clusters and selecting cluster-heads associated with them are based on random algorithms or heuristics to classical combinatorial optimization problems, such as p-median, dominating set, and set covering problems. In such approaches, every sensor node is either in a set of cluster-heads or is assigned to a cluster-head. In general, among the cluster-heads, a multi-hop communication scheme is used, while inside a cluster multi- and/or single-hop communication could be used. Santos et al. [51] modeled the clustering problem as the independent dominating set problem. Two greedy heuristics are described in that reference.

Matos et al. [52] call attention to the importance of rotating the sensor that acts as cluster-head within a single cluster. Cluster-heads consume much more energy than a sensor that does not perform this feature, since they need to aggregate, send, and receive sensing data. Thus, cluster-head rotation helps to avoid a premature death of a sensor, disconnecting the network. The rotation can be done from time to time or after some amount of data has been transferred by the network. In addition, Matos et al. [52] also proposed a centralized GRASP-based heuristic with path relinking as an intensification phase to solve the clustering problem. The problem was modeled as the p-median problem, where the cluster-heads are chosen among the alive sensors. The intra-clusters communication follows the single-hop strategy. Their heuristic is composed of three phases. In the constructive phase, a feasible solution is generated by the randomized greedy algorithm described in [53]. Then, a swap local search is applied. Finally, a path relinking routine is applied as intensification phase. To attest the effectiveness of their approach, they compared their heuristic with methods described in [54, 55]. Albath et al. [56] consider that a sensor to be selected as cluster-head must have enough residual energy. In their work, the problem of properly choosing cluster-heads was modeled as the minimum dominating set problem.

Looking for metaheuristics methods, Ferentinos and Tsiligiridis [57] described a GA-based heuristic used in an agriculture application. Their algorithm is designed to solve clustering and density control problem while respecting connectivity constraints. Another GA-based heuristic for clustering problem is addressed by Kuila et al. [58]. Ting and Liao [59] described the clustering problem as the k-cover problem and solved it with a MA-based heuristic.

The transmission of sensing data and the dissemination of information in WSN are referred as the routing problem. This problem is defined mainly when the nodes position and the cluster-heads selected are already known. The main objective when solving the routing problem is to guarantee the network connectivity. Routing in WSN design is tackled by many authors [19, 21, 22, 60] through a multi-hop strategy and fixed sink nodes as well as through mobile sinks [23, 61, 62].

Minimum spanning tree, shortest path, and TSP are preferentially used to model the routing problem in WSN design. In general the edge weight (length or cost) represents the energy needed to transmit data. Thus, routing algorithm based on Kruskal, Prim, Dijkstra, Bellman-Ford, or TSP methods can be applied. As stated in section "Problem Definition", the sink neighborhood problem [17] arises when dealing with fixed sink nodes. To overcome that problem, some alternatives take place. Some works consider multiple sinks or, in some contexts, explore the network by mobile sinks. More sinks result in shorter routes from sensors to their closest sink, while mobility may lead to an efficient scheme for the energy control.

Centralized and distributed heuristics to control and coordinate the current movement of multiple sinks seeking for the lifetime maximization in WSN are described by Basagni et al. [63]. In this reference, a sink is considered active when it is stopped, waiting for the sensor data, and inactive when it is moving through the network. In their approach at least one sink must move at a time, so it is necessary to define a schedule for the movements of sinks. The routing scheme was modeled with TSP problem and solved by a Christofides' heuristic [64].

To control the sink movements, Basagni et al. [17] introduced a distributed heuristic, named greedy maximum residual energy. In their heuristic, a sink defines its own route by moving from its current location to a new position by giving preference to areas with the highest residual energy. More precisely, the greedy criterion of the proposed heuristic is the amount of energy left in the sensors around a mobile sink that moves in direction of the sensors with higher energy to collect their data.

Integrated Problems

Most the works found in the WSN literature describe hierarchical approaches to the design of networks. Those works are drawn for solving WSN problems in multiple steps. Although less, there are works in the literature that address a number of classical WSN problems in an integrated manner [21, 65–72]. A natural clustering and routing problem is the minimum connected dominating set problem [73–75], the problem of finding a connected dominating set of cluster-heads with smallest cardinality. Aioffi et al. [66], for instance, have provided a constructive heuristic to an integrated problem, named integrated clustering and routing problem (ICRP). Their proposed heuristic is based on the cheapest insertion algorithm for the Euclidean TSP. In such method, a set of routes is iteratively defined, one for each mobile sink, at the same time as the sensors are grouped into clusters. Each sink node visits some cluster-heads, in such a way that every sensor node in the network is either a cluster-head visited by a route or lies within a cluster associated with a cluster-head in one of the possible routes. The greedy criterion of the proposed heuristic is based on the distance among the last cluster-head assigned to a route and the other cluster-heads not routed yet. Hybrid heuristics based on GRASP and ILS metaheuristics were also applied to solve the problem. Another integrated approach is due to Xing et al. [65] that also studied clustering and routing problems. To solve the problem, a heuristic was used in the following schemes. First, it solves the Steiner problem to select the cluster-heads and then uses a TSP local search heuristic to find a route for each mobile sink.

A TS-based heuristic for an integrated coverage, sink location, and routing problems can be found in Guney et al. [68]. The heuristic starts by finding near-optimal sensor locations satisfying the coverage requirements, and then it solves the sink location and the data routing problem modeled as the p-median problem. Türkogullari et al. [76] proposed a hybrid heuristic to solve large instance of an integrated problem involving sensor placement, density control, sink location, and routing problem with low conservation, energy consumption, and budget constraints. In such algorithm, the optimal sensor-to-sink routes are solved as a linear program (LP), while the sink location problem is modeled as a set covering problem and solved by a disjoint sets heuristic. Another heuristic based on exact approach, used to solve large and real-life sink location and routing problems, could be found in [77].

Table 1 Description of the cited references on heuristic-based methods for WSN problems. Consider the following acronyms list: coverage problem (CP), density control (DCP), localization problem (LP), routing problem (RP), clustering problem (CL), and placement problem (PP). Multi-hop communication (MhC), fixed sink (FS), mobile sink (MoS), and multiple sink (MuS)

References	Problems										Methods
Aioffi et al. [82]	CP	DCP	RP	CL	–	–	MhC	–	MoS	–	Greedy
Aioffi et al. [66]	–	–	RP	CL	–	–	–	–	MoS	MuS	GRASP, ILS
Alba and Molina [46]	CP	–	–	–	PP	–	–	–	–		SA
Albath et al. [56]	–	–	–	CL	–	–	–	FS	–	–	Greedy
Basagni et al. [17]	–	–	–	–	–	–	–	–	MoS	–	Greedy
Basagni et al. [63]	–	–	RP	–	PP	–	–	FS	MoS	MuS	Greedy
Brazil et al. [43]	–	–	–	–	PP	–	–	–	–	–	Greedy
Cardei et al. [48]	CP	DCP	–	–	–	–	–	–	–	–	Greedy
Delicato et al. [49]	–	DCP	–	–	–	–	–	–	–	–	Greedy
Deschinkel [47]	CP	–	–	–	–	–	–	–	–	–	CG heuristic
Ferentinos and Tsiligiridis [57]	–	DCP	RP	CL	–	–	–	FS	–	–	GA
Guney et al. [68]	CP	–	RP	–	–	LP	–	–	–	–	TS
Karasabun et al. [50]	–	DCP	–	–	–	–	–	–	–	–	MS
Kuila et al. [58]	–	–	–	CL	–	–	–	FS	–	–	GA
Laszka et al. [45]	–	–	–	–	PP	LP	–	–	–	–	GA
Lin and Uster [83]	–	–	–	–	PP	–	–	–	–	–	MS
Matos et al. [52]	CP	–	RP	–	–	–	MhC	–	–	–	GRASP, PR
Molina and Alba [41]	–	–	–	–	–	LP	–	–	–	–	SA, GA, PSO
Santos et al. [51]	–	–	–	CL	–	–	–	FS	–	–	Greedy
Sasikumar et al. [44]	–	–	–	–	PP	–	–	–	–	–	Greedy
Shahrokhzadeh et al. [42]	–	–	RP	–	PP	–	–	–	–	–	SA
Ting and Liao [59]	–	–	–	CL	–	–	–	–	–	–	MA
Turkogullari et al. [76]	–	–	RP	–	PP	–	–	–	–	–	MIP-heu.
Uster and Lin [67]	–	DCP	–	CL	–	–	–	–	–	MuS	Constructive heu.
Xing et al. [65]	–	–	RP	CL	–	–	–	–	–	–	Greedy

Table 1 summarizes the previous cited works, highlighting the problems solved and the heuristic methods used. For an overview of the mathematical aspects of network optimization problems, we also refer to [78–81].

Conclusion

In this chapter, an overview of classical optimization problems and solution methods in the design of wireless sensor networks are presented. WSN have been widely studied due to their importance in many practical applications. Unlike a general

network, WSN are designed for specific applications and take into account the application goals, the associated costs, the hardware capabilities, and other system constraints. Therefore, optimization techniques are shown to be an essential part in the design of new protocols. The integration of two or more problems, such as coverage, routing, density control, clustering, and so on, requires even more sophisticated approaches to achieve high-quality solutions. It is straightforward to conclude that optimization tools play a key role in the design of WSN. The referred methods in this chapter have the capacity to significantly improve metrics such as network lifetime, coverage rate, and communication delay. Looking to the future, WSN represent an attractive research area with several new possibilities where the major challenge is to approximate the theoretical tools for practical applications.

Cross-References

- Genetic Algorithms
- GRASP
- Iterated Local Search
- Memetic Algorithms
- Network Optimization
- Tabu Search
- Variable Neighborhood Search

Acknowledgments This work was partially supported by the Brazilian National Council for Scientific and Technological Development (CNPq), the Foundation for Support of Research of the State of Minas Gerais, Brazil (FAPEMIG), and Coordination for the Improvement of Higher Education Personnel, Brazil (CAPES). Vinicius Morais is funded by CAPES BEX 7461/14-3.

Competing Interest Declaration The author(s) has no competing interests to declare that are relevant to the content of this manuscript.

References

1. Akyildiz I, Su W, Sankarasubramaniam Y, Cayirci E (2002) Wireless sensor networks: a survey. Comput Netw 38(4):393–422
2. Yick J, Mukherjee B, Ghosal D (2008) Wireless sensor network survey Comput Netw 52(12):2292–2330
3. Xu N (2002) A survey of sensor network applications. In: IEEE communications magazine electronics, robotics and automotive mechanics conference, Washington, DC, pp 102–114
4. Arampatzis T, Lygeros J, Member S, Manesis S (2005) A survey of applications of wireless sensors and wireless sensor networks. In: Proceedings of the 13th Mediterranean conference on control and automation, Limassol, pp 719–724
5. Kuorilehto M, Hännikäinen M, Hämäläinen TD (2005) A survey of application distribution in wireless sensor networks. EURASIP J Wirel Commun Netw 2005:774–788
6. Yoneki E, Bacon J (2005) A survey of wireless sensor network technologies: research trends and middleware's role. Technical report UCAM-CL-TR-646, Computer Laboratory, University of Cambridge, Cambridge, UK

7. Munir S, Ren B, Jiao W, Wang B, Xie D, Ma J (2007) Mobile wireless sensor network: architecture and enabling technologies for ubiquitous computing. In: 21st international conference on advanced information networking and applications workshops, 2007, AINAW'07, Niagara Falls, vol 2, pp 113–120
8. Khan MI, Gansterer WN, Haring G (2013) Static vs. mobile sink: the influence of basic parameters on energy efficiency in wireless sensor networks. Comput Commun 36:965–978
9. Anastasi G, Conti M, Di Francesco M, Passarella A (2009) Energy conservation in wireless sensor networks: a survey. Ad Hoc Netw 7:537–568
10. Kuhn F, Wattenhofer R, Zollinger A (2008) Ad hoc networks beyond unit disk graphs. Wirel Netw 14:715–729
11. Al-Karaki JN, Kamal AE (2004) Routing techniques in wireless sensor networks: a survey. Wirel Commun 11:6–28
12. Langendoen K, Reijers N (2003) Distributed localization in wireless sensor networks: a quantitative comparison. Comput Netw 43:499–518
13. Madan R, Lall S (2006) Distributed algorithms for maximum lifetime routing in wireless sensor networks. IEEE Trans Wirel Commun 5:2185–2193
14. Kubisch M, Karl H, Wolisz A, Zhong LC, Rabaey J (2003) Distributed algorithms for transmission power control in wireless sensor networks. In: In: 2003 IEEE wireless communications and networking, WCNC 2003, vol 1, New Orleans, pp 558–563
15. Boukerche A, Oliveira HA, Nakamura EF, Loureiro AAF (2007) Localization systems for wireless sensor networks. IEEE Wirel Commun 14:6–12
16. Efrat A, Har-Peled S, Mitchell JSB (2005) Approximation algorithms for two optimal location problems in sensor networks. In: 2nd international conference on broadband networks, BroadNets 2005, vol 1, Boston, pp 714–723
17. Basagni S, Carosi A, Melachrinoudis E, Petrioli C, Wang ZM (2008) Controlled sink mobility for prolonging wireless sensor networks lifetime. Wirel Netw 14(6):831–858
18. Wolsey LA (1998) Integer programming. Wiley-Interscience, New York
19. Nakamura F, Quintão F, Menezes G, Mateus G (2005) An optimal node scheduling for flat wireless sensor networks. Lect Notes Comput Sci 3420:475–482
20. Jarray F (2013) A lagrangean-based heuristics for the target covering problem in wireless sensor network. Appl Math Model 37:6780–6785
21. Castaño F, Bourreau E, Velasco N, Rossi A, Sevaux M (2015) Exact approaches for lifetime maximization in connectivity constrained wireless multi-role sensor networks. Eur J Oper Res 241(1):28–38
22. Lee J-H, Moon I (2014) Modeling and optimization of energy efficient routing in wireless sensor networks. Appl Math Model 38(7–8):2280–2289
23. Valle CA, Martinez LC, da Cunha AS, Mateus GR (2011) Heuristic and exact algorithms for a min–max selective vehicle routing problem. Comput Oper Res 38(7):1054–1065
24. Feo TA, Resende MG (1995) Greedy randomized adaptive search procedures. J Glob Optim 6:109–133
25. Resende MG, Ribeiro CC (2005) Grasp with path-relinking: recent advances and applications. In: Ibaraki T, Nonobe K, Yagiura M (eds) Metaheuristics: progress as real problem solvers, Operations research/computer science interfaces series, vol 32. Springer, New York, pp 29–63
26. Kirkpatrick S, Gelatt CD, Vecchi MP (1983) Optimization by simulated annealing. Science 220:671–680
27. Lourenço HR, Martin OC, Stutzle T (2010) Chapter 12: Iterated local search: framework and applications. In: Glover F, Kochenberger G, Hillier F (eds) Handbook of metaheuristics. International series in operations research and management science, vol 57, 2nd edn. Springer, New York, pp 363–398
28. Glover F (1986) Future paths for integer programming and links to artificial intelligence. Comput Oper Res 13(5):533–549. Applications of integer programming
29. Hansen P, Mladenović N (2001) Variable neighborhood search: principles and applications. Eur J Oper Res 130(3):449–467

30. Hansen P, Mladenović N (2001) Variable neighborhood search. In: Pardalos P, Resende M (eds) Handbook of applied optimization. Oxford University Press, New York
31. Holland JH (1975) Adaptation in natural and artificial systems, 2nd edn. University of Michigan Press, Ann Arbor. 1992
32. Moscato P (1999) Memetic algorithms: a short introduction. In: Corne D, Dorigo M, Glover F, Dasgupta D, Moscato P, Poli R, Price KV (eds) New ideas in optimization. McGraw-Hill Ltd., Maidenhead, pp 219–234
33. Dorigo M, Blum C (2005) Ant colony optimization theory: a survey. Theor Comput Sci 344 (2–3):243–278
34. Kennedy J, Eberhart R (1995) Particle swarm optimization. Proceedings IEEE international conference on neural networks, Perth 4:1942–1948
35. Glover F, Kochenberger GA (2003) Handbook of metaheuristics. Kluwer Academic Publishers, Boston
36. Nieberg T (2006) Independent and dominating sets in wireless communication graphs. Ph.D. thesis,. University of Twente
37. Ren H, Meng M-H, Chen X (2006) Investigating network optimization approaches in wireless sensor networks. In: International conference on intelligent robots and systems. Deajeon, pp 2015–2021
38. Li J (2008) Optimization problems in wireless sensor and passive optical networks. Ph.D. thesis,. The University of Melbourne
39. Suomela J (2009) Optimisation problems in wireless sensor networks: local algorithms and local graphs. Ph.D. thesis,. University of Helsinki
40. Gogu A, Nace D, Dilo A, Meratnia N (2012) Review of optimization problems in wireless sensor networks. In: Hamilton Ortiz J (ed) Telecommunications networks – current status and future trends. InTech, Rijeka, pp 153–180
41. Molina G, Alba E (2011) Location discovery in wireless sensor networks using metaheuristics. Appl Soft Comput 11(1):1223–1240
42. Shahrokhzadeh M, Haghighat AT, Mahmoudi F, Shahrokhzadeh B (2011) A heuristic method for wireless sensor network localization. In: Procedia computer science, vol 5. The 2nd international conference on ambient systems, networks and technologies (ANT-2011)/the 8th international conference on mobile web information systems (MobiWIS 2011). Elsevier, Niagara Falls, pp 812–819
43. Brazil M, Ras CJ, Thomas DA (2009) Deterministic deployment of wireless sensor networks. In: Proceedings of the world congress on engineering 2009, vol 1, London, p 863
44. Sasikumar P, Vasudevan SK, Ramesh M (2010) Heuristic approaches with energy management for node placement in wireless sensor networks. Int J Comput Appl 1(37):34–736
45. Laszka A, Buttyán L, Szeszlér D (2013) Designing robust network topologies for wireless sensor networks in adversarial environments. Pervasive Mob Comput 9(4):546–563
46. Alba E, Molina G (2008) Optimal wireless sensor network layout with metaheuristics: solving a large scale instance. In: Lirkov I, Margenov S, Waśniewski J (eds) Large-scale scientific computing, Lecture notes in computer science, vol 4818. Springer, Berlin/Heidelberg, pp 527–535
47. Deschinkel K (2011) A column generation based heuristic for maximum lifetime coverage in wireless sensor networks. In: SENSORCOMM: the fifth international conference on sensor technologies and applications. Nice, pp 1–6
48. Cardei M, Wu J, Lu M, Pervaiz M (2005) Maximum network lifetime in wireless sensor networks with adjustable sensing ranges. IEEE international conference on wireless and mobile computing, networking and communications, Montreal 3:438–445
49. Delicato F, Protti F, Pirmez L, de Rezende JF (2006) An efficient heuristic for selecting active nodes in wireless sensor networks. Comput Netw 50(18):3701–3720
50. Karasabun E, Korpeoglu I, Aykanat C (2013) Active node determination for correlated data gathering in wireless sensor networks. Comput Netw 57(5):1124–1138
51. Santos AC, Bendali F, Mailfert J, Duhamel C, Hou KM (2009) Heuristics for designing energy-efficient wireless sensor network topologies. J Netw 4(6):436–444

52. Matos VO, Arroyo JEC, dos Santos AG, Goncalves LB (2012) An energy-efficient clustering algorithm for wireless sensor networks. Int J Comput Sci Netw Secu 12(10):6–15
53. Resende MGC, Werneck RF (2004) A hybrid heuristic for the p-median problem. J Heuristics 10:59–88
54. Heinzelman WB, Chandrakasan AP, Balakrishnan H (2000) Energy-efficient communication protocol for wireless microsensor networks. In: Proceedings of the 33rd annual Hawaii international conference on system sciences, vol 2, Hawaii, p 10
55. Heinzelman WB, Chandrakasan AP, Balakrishnan H (2002) An application-specific protocol architecture for wireless microsensor networks. IEEE Trans Wirel Commun 1:660–670
56. Albath J, Thakur M, Madria S (2010) Energy constrained dominating set for clustering in wireless sensor networks. In: 24th IEEE international conference on advanced information networking and applications (AINA), Perth, pp 812–819
57. Ferentinos KP, Tsiligiridis TA (2007) Adaptive design optimization of wireless sensor networks using genetic algorithms. Comput Netw 51(4):1031–1051
58. Kuila P, Gupta SK, Jana PK (2013) A novel evolutionary approach for load balanced clustering problem for wireless sensor networks. Swarm Evol Comput 12:48–56
59. Ting C-K, Liao C-C (2010) A memetic algorithm for extending wireless sensor network lifetime. Inf Sci 180(24):4818–4833
60. Ding L, Gao X, Wu W, Lee W, Zhu X, Du D-Z (2011) An exact algorithm for minimum cds with shortest path constraint in wireless networks. Optim Lett 5(2):297–306
61. Gandham S, Dawande M, Prakash R, Venkatesan S (2003) Energy efficient schemes for wireless sensor networks with multiple mobile base stations. In: Proceedings of IEEE globecom 2003, vol 1, San Francisco, pp 377–381
62. Behdani B, Yun YS, Smith JC, Xia Y (2012) Decomposition algorithms for maximizing the lifetime of wireless sensor networks with mobile sinks. Comput Oper Res 39(5):1054–1061
63. Basagni S, Carosi A, Petrioli C, Phillips CA (2009) Heuristics for lifetime maximization in wireless sensor networks with multiple mobile sinks. In: IEEE international conference on communications, ICC'09, Dresden, pp 1–6
64. Christofides N (1976) Worst-case analysis of a new heuristic for the traveling salesman problem. Technical report 388, Carnegie-Mellon University, Graduate School of Industrial Administration
65. Xing G, Wang T, Jia W, Li M (2008) Rendezvous design algorithms for wireless sensor networks with a mobile base station. In: Proceedings of the 9th ACM international symposium on mobile ad hoc networking and computing, MobiHoc'08. ACM, New York, pp 231–240
66. Aioffi WM, Valle CA, Mateus GR, da Cunha AS (2011) Balancing message delivery latency and network lifetime through an integrated model for clustering and routing in wireless sensor networks. Comput Netw 55(13):2803–2820
67. Üster H, Lin H (2011) Integrated topology control and routing in wireless sensor networks for prolonged network lifetime. Ad Hoc Netw 9(5):835–851
68. Güney E, Aras N, Altinel IK, Ersoy C (2012) Efficient solution techniques for the integrated coverage, sink location and routing problem in wireless sensor networks. Comput Oper Res 39:1530–1539
69. Türkogulları YB, Aras N, Altınel IK, Ersoy C (2010) A column generation based heuristic for sensor placement, activity scheduling and data routing in wireless sensor networks. Eur J Oper Res 207(2):1014–1026
70. Raiconi A, Gentili M (2011) Exact and metaheuristic approaches to extend lifetime and maintain connectivity in wireless sensors networks. In: Proceedings of the network optimization: 5th international conference, INOC 2011, Hamburg, 13–16 June 2011. Springer, Berlin/Heidelberg, pp 607–619
71. Gentili M, Raiconi A (2013) α-Coverage to extend network lifetime on wireless sensor networks. Optim Lett 7(1):157–172
72. Carrabs F, Cerulli R, D'Ambrosio C, Raiconi A (2015) Exact and heuristic approaches for the maximum lifetime problem in sensor networks with coverage and connectivity constraints. Technical report. University of Salerno

73. Blum J, Ding M, Thaeler A, Cheng X (2005) Connected dominating set in sensor networks and MANETs. Springer, Boston, pp 329–369
74. Gendron B, Lucena A, da Cunha AS, Simonetti L (2014) Benders decomposition, branch-and-cut, and hybrid algorithms for the minimum connected dominating set problem. INFORMS J Comput 26(4):645–657
75. Buchanan A, Sung JS, Butenko S, Pasiliao (2015) An integer programming approach for fault-tolerant connected dominating sets. INFORMS J Comput 27(1):178–188
76. Türkogulları YB, Aras N, Altınel IK, Ersoy C (2010) An efficient heuristic for placement, scheduling and routing in wireless sensor networks. Ad Hoc Netw 8(6):654–667
77. Güney E, Aras N, Altınel IK, Ersoy C (2010) Efficient integer programming formulations for optimum sink location and routing in heterogeneous wireless sensor networks. Comput Netw 54(11):1805–1822
78. Rajasekaran S, Pardalos P, Hsu DF (2000) Mobile networks and computing. In: DIMACS – series in discrete mathematics and theoretical computer science, vol 52. American Mathematical Society, Providence
79. Oliveira CA, Pardalos PM (2011) Mathematical aspects of network routing optimization. Volume 53 of 1931–6828. Springer, New York
80. Resende MG, Pardalos PM (2006) Handbook of optimization in telecommunications. Springer, New York. No. 1 in 78-0-387-30662-9
81. Pardalos PM, Ye Y, Boginski VL, Commander CW (2012) Sensors: theory, algorithms, and applications. Volume 61 of 1931–6828. Springer, New York
82. Aioffi WM, Mateus GR, Quintao FP (2007) Optimization issues and algorithms for wireless sensor networks with mobile sink. In: International network optimization conference, Spa
83. Lin H, Uster H (2014) Exact and heuristic algorithms for data-gathering cluster-based wireless sensor network design problem. IEEE/ACM Trans Networking 22:903–916

Particle Swarm Optimization for the Vehicle Routing Problem: A Survey and a Comparative Analysis

51

Yannis Marinakis, Magdalene Marinaki, and Athanasios Migdalas

Contents

Introduction 1564
Particle Swarm Optimization 1566
Application of Particle Swarm Optimization Algorithm in Vehicle Routing Problems 1570
Main Variants of VRP Solved by PSO 1571
Different PSO Variants 1574
Analysis of the Algorithms 1581
Conclusions 1586
Cross-References 1586
References 1586

Abstract

In the last few years, a number of books and survey papers devoted to the vehicle routing problem (VRP) or to its variants or to the methods used for the solution of one or more variants of the VRP have been published. Also, in these years, the field of swarm intelligence algorithms has had a significant growth. One of the most important swarm intelligence algorithms is the particle swarm optimization (PSO). Although the particle swarm optimization was first published in 1995, it took around 10 years in order researchers to publish papers using a PSO

Y. Marinakis (✉) · M. Marinaki
School of Production Engineering and Management, Technical University of Crete, Chania, Greece
e-mail: marinakis@ergasya.tuc.gr; magda@dssl.tuc.gr

A. Migdalas
Industrial Logistics, Luleå Technical University, Luleå, Sweden

Department of Civil Engineering, Aristotle University of Thessalonike, Thessalonike, Greece
e-mail: athmig@ltu.se; samig@civil.auth.gr

© Springer Nature Switzerland AG 2025

R. Martí et al. (eds.), *Handbook of Heuristics*,
https://doi.org/10.1007/978-3-032-00385-0_42

algorithm for the solution of variants of the VRP. However, in the last 10 years, many journal papers, conference papers, and book chapters have been published where a variant of VRP is solved using a PSO algorithm. Thus, it is significant to present a survey paper where a review and brief analysis of the most important of these papers will be given. This is the main focus of this chapter.

Keywords

Vehicle routing problem · Particle swarm optimization

Introduction

The vehicle routing problem is one of the most important problems in the field of supply chain management, of logistics, of combinatorial optimization, of transportation, and, in general, of operational research. The interest in this problem has been recently increased both from theoretical and practical aspect. There are a number of reasons for this growth. From the practical point of view, the problem is one of the most important problems in the supply chain management, and, thus, the finding of the optimal set of routes will help the decision makers to reduce the cost of the supply chain and to increase the profit. Also, in the formulation of the problem, the managers could simulate the complete network and add any of the constraints concerning the customers, the vehicles, the routes, and, also, the traffic conditions of the network and the energy consumption of the vehicles. Thus, someone could solve a realistic problem and find a near optimal set of solutions.

The reason that usually a near optimal set of solutions is found is that the problem from its origin (it was first introduced by Dantzig and Ramser in 1959 [1]) was proved to be NP-hard problem even in its simpler version, the capacitated vehicle routing problem, where the only constraint that was taken into account was the capacity of the vehicles (and later the maximum tour length constraint was added). Thus, it is impossible in real-life applications to find an optimal solution. For this reason, a number of heuristic, metaheuristic (mainly), evolutionary, and nature-inspired approaches have been proposed for the solution of the VRP and its variants. Also, exact algorithms have been proposed in order to solve the problem. They are, mainly, used for the solution of the simplest vehicle routing problems (the problems with as few as possible constraints) and for a small number of nodes.

From the theoretical point of view, there are a huge number of researchers that deal with the solution of a variant (or more variants) of the problem. These variants of the problem focus on a specific constraint, and the researchers are trying to find an algorithm that gives new best solutions in a specific set of benchmark instances in short computational time with as less as possible parameters in order to give a more general algorithm. Thus, a new researcher that he/she would like to focus to a specific variant of a vehicle routing problem, he/she could find a large number of very good papers focusing to VRP or to its variants and papers focusing to

the methods that he/she would like to implement for the solution of the problem. The most important and well-studied variants of the vehicle routing problem are the capacitated vehicle routing problem [2–6], the vehicle routing problem with time windows [7–11], the open vehicle routing problem [12], the vehicle routing problem with simultaneous pickup and deliveries [13, 14], the vehicle routing problem with backhauls [15–17], the multidepot vehicle routing problem [18–20], the stochastic vehicle routing problem [21–24], the dynamic vehicle routing problem [23, 25–27], the periodic vehicle routing problem [28], the split delivery vehicle routing problem [29], the heterogeneous fleet vehicle routing problem [30], the asymmetric vehicle routing problem [31], the vehicle routing problem with two- or three-dimensional loading constraints [32, 33], etc. Nowadays, more complicated problems have been published where more than one from the previously mentioned problems are combined in order to create a new more challenging problem. These problems are denoted from some researchers as rich vehicle routing problems [34]. However, a number of other problems have been introduced in the last years in order to cope with new needs that arise from new realistic situations of the life, like the cumulative capacitated vehicle routing problem [35], the evacuation vehicle routing problem [36], the green vehicle routing problem [37], the pollution routing problem [38], etc. Finally, there are a number of combined problems that need a solution of a vehicle routing problem, like the location routing problem [39–41], the inventory routing problem [42–44], the location inventory routing problem [45], the production routing problem [46], the production inventory distribution problem [47], and the ship routing problem [48–51]. Also, a number of publications with a real case application have been realized where the authors simulate the realistic scenario, formulate the problem (or use an existing formulation) using a combination of the existing variants or introducing a new one if it is possible, and use an existing algorithm (or propose a new one or a suitable modified variant of an existing one) to solve it.

Thus, in the last years, the EURO Working Group on Vehicle Routing and Logistics Optimization, the VeRoLog, was created (www.verolog.eu); two different series of conferences devoted to the vehicle routing problem variants and applications have been introduced, the one is an annual conference which is organized from the VeRoLog working group since 2012 and the second is a triennial workshop on freight transportation and logistics denoted as Odysseus started in Crete 2000. A number of books devoted to the vehicle routing problem have been published. The first one was published in 1988, and it was a very inspiring book for any new researcher that would like to study the vehicle routing problem or just to know about the vehicle routing problem [52]. Twenty years later, one of the two authors (Bruce Golden) of the first published book [52] published a new book [53] that summarizes the work performed in these 20 years and gives some new directions for future research. Before the publication of this book, another very successful and inspiring book was published by Paolo Toth and Daniele Vigo [14] which is the most widely read and most cited book in the field. The success of this book led the same team of authors to publish in the end of 2014 the second edition of the

book [54] which covers all the field of the vehicle routing problem with a more complete rewriting of some of the chapters of the previous edition of the book and with the new directions that have been created based on the new challenges in the field. A volume of the *Handbooks in Operations Research and Management Science* was devoted to vehicle routing problems and to more general applications, like the arc routing problem [55], and it was published in [56]. Finally, two other books devoted to the vehicle routing problem have been published in [57, 58]. Also, a large number of surveys have been published, initially, devoted to general vehicle routing problems and, nowadays, to variants of the vehicle routing problem or to methods that are applied for the solution of the vehicle routing problem or of its variants. Some of these survey papers can be found in [4–6, 34, 37, 59–76].

Thus, in this chapter, as a general review for the vehicle routing problem cannot be restricted in one chapter, we decided to focus in a specific algorithm, the particle swarm optimization (PSO) algorithm, and in the application of this algorithm for the solution of the vehicle routing problem and its main variants. This is the first survey chapter, at least to our knowledge, that is devoted to this method for the VRP. In the following sections, initially, a brief presentation of the particle swarm optimization algorithm will be given, and, then, we will present the variants of the vehicle routing problem in which a particle swarm optimization algorithm has been applied for their solution.

Particle Swarm Optimization

Particle Swarm Optimization (PSO) is a very popular global optimization method that was originally proposed by Kennedy and Eberhart as a simulation of the social behavior of social organisms such as bird flocking and fish schooling [77]. PSO uses the physical movements of the individuals in the swarm. Complete surveys, in the time that they are published, for the particle swarm optimization can be found in [78–81]. Nowadays, a complete review for the particle swarm optimization is very difficult to be performed as the range of variants and of applications of PSO covers the whole field of optimization, and, thus, only surveys in a specific subject like the one presented in this chapter (application of PSO algorithm in vehicle routing problems) can be presented without the length of the paper to increase dramatically.

In general, in a PSO algorithm, a set of solutions are used where each solution is denoted as a particle. These solutions create the swarm. In many algorithms, more than one swarms exist. Initially, the solutions are randomly initialized in the solution space. Two are the main vectors that describe a particle, the position vector (x_{ij}, where i denotes the particle ($i = 1, \cdots, N$, N is the swarm size) and j denotes the corresponding dimension of the particle ($j = 1, \cdots, d$, d is the dimension of the problem)) and the velocities vector (v_{ij}). The performance of each particle is evaluated on the predefined fitness function ($f(x)$).

Thus, each particle is randomly placed in the d-dimensional space as a candidate solution. One very simple and effective way to initialize the particles was given in

[82] where the minimum value of the solution space is denoted as $x_{\min,j}$ and the maximum value is denoted as $x_{\max,j}$. Then, the values of the solution vector for each particle are calculated from the following equation:

$$x_{ij}(0) = x_{\min,j} + r_j \left(x_{\max,j} - x_{\min,j}\right) \tag{1}$$

where r_j is a random number between (0, 1).

The velocity of the i-th particle v_{ij} is defined as the change of its position. The algorithm completes the optimization through following the personal best solution of each particle and the global best value of the whole swarm. Thus, in each iteration, except of the current position, another vector is used which is the personal best position of each particle through the iterations. The best member of the personal best position vector is the global best position of the whole swarm. Each particle adjusts its trajectory toward its own previous best position and the global best position, namely, $pbest_{ij}$ and $gbest_j$, respectively. The velocities and positions of particles are updated using the following eqs. [77]:

$$v_{ij}(t+1) = v_{ij}(t) + c_1\, rand_1 \left(pbest_{ij} - x_{ij}(t)\right) + c_2\, rand_2 \left(gbest_j - x_{ij}(t)\right) \tag{2}$$

$$x_{ij}(t+1) = x_{ij}(t) + v_{ij}(t+1) \tag{3}$$

where t is the iteration counter, c_1 and c_2 are the acceleration coefficients, and $rand_1$ and $rand_2$ are two random variables in the interval (0, 1). The values of c_1 and c_2 could be constant, or they could be adapted during the iterations. Usually in the most published papers concerning an application of PSO in a continuous or discrete problem, these values were set equal to 2. However, there is a possibility to take different values or to adjust their values during the iterations, for example, using the following eqs. [82]:

$$c_1 = c_{1,\min} + \frac{c_{1,\max} - c_{1,\min}}{iter_{\max}} \times t \tag{4}$$

$$c_2 = c_{2,\min} + \frac{c_{2,\max} - c_{2,\min}}{iter_{\max}} \times t \tag{5}$$

where $\text{iter}_{\max}$ is the maximum number of iterations and $c_{1,\min}$, $c_{1,\max}$, $c_{2,\min}$, and $c_{2,\max}$ are the minimum and maximum values that c_1 and c_2 can take, respectively. In the beginning of the procedure, the values of c_1 and c_2 are small and, then, are increasing until they reach their maximum values. By doing this, on the first iterations, there is a great freedom of movement in the particle solution space in order to find the optimum quickly.

A number of different velocity equations have been proposed during the last years. Nowadays, other researchers use one of the velocity equations proposed in

the past that have been proved to perform well (mainly, the inertia equation or the constriction equation), or, less frequently, they propose a new one which either is a variant of a previously published equation or simulates something from the nature in which the specific researcher focuses in his/her study. The most important and known velocities equations are the following:

- Inertia particle swarm optimization (IPSO) [83]:

$$v_{ij}(t+1) = w v_{ij}(t) + c_1\, rand_1 \left(pbest_{ij} - x_{ij}(t)\right) + c_2\, rand_2 \left(gbest_j - x_{ij}(t)\right) \tag{6}$$

  The difference between this variant and the one presented on Eq. (2) is the use of the inertia weight w. The inertia weight w is adapted during the iterations and is given by the following equation:

$$w = w_{\max} - \frac{w_{\max} - w_{\min}}{\text{iter}_{\max}} \times t \tag{7}$$

  where $w_{\max}$ and $w_{\min}$ are the maximum and minimum values of the inertia weight.
- Constriction particle swarm optimization [84]:

$$v_{ij}(t+1) = \chi\Big(v_{ij}(t) + c_1\, rand_1 \left(pbest_{ij} - x_{ij}(t)\right) + c_2\, rand_2 \left(gbest_j - x_{ij}(t)\right)\Big) \tag{8}$$

  where the constriction factor, χ, is used:

$$\chi = \frac{2}{|2 - c - \sqrt{c^2 - 4c}|} \text{ and } c = c_1 + c_2,\ c > 4 \tag{9}$$

- Another version of the constriction particle swarm optimization [82]:

$$v_{ij}(t+1) = \chi\Big(v_{ij}(t) + c_1\, rand_1 \left(pbest_{ij} - x_{ij}(t)\right) + c_2\, rand_2 \left(gbest_j - x_{ij}(t)\right)\Big) \tag{10}$$

  The only difference from the previous version is the use of the parameter k in the constriction factor [82]:

$$\chi = \frac{2k}{|2 - c - \sqrt{c^2 - 4c}|} \text{ and } c = c_1 + c_2,\ c > 4 \tag{11}$$

- Clerc and Kennedy [84] proposed a simpler form of the constriction factor, the **condensed form**, for the calculation of the velocities of the particles:

$$v_{ij}(t+1) = \chi\left(v_{ij}(t) + c\left(p_{mj} - x_{ij}(t)\right)\right) \tag{12}$$

where

$$\chi = \frac{2}{|2-c-\sqrt{c^2-4c}|} \text{ and } c = c_1 + c_2, c > 4 \tag{13}$$

And

$$p_{mj} = \frac{c_1 pbest_{ij} + c_2 gbest_j}{c}. \tag{14}$$

- Cognition-only particle swarm optimization [85]
  In this variant, the velocity equation is given by:

$$v_{ij}(t+1) = v_{ij}(t) + c_1\, rand_1 \left(pbest_{ij} - x_{ij}(t)\right) \tag{15}$$

  The social factor of velocities equation is not used.
- Social-only particle swarm optimization [85].
  In this variant, the velocity equation is given by:

$$v_{ij}(t+1) = v_{ij}(t) + c_2\, rand_2 \left(gbest_j - x_{ij}(t)\right) \tag{16}$$

  The cognition factor of velocities equation is not used.
- Local neighborhood topology particle swarm optimization [82]:

$$\begin{aligned} v_{ij}(t+1) = v_{ij}(t) &+ c_1\, rand_1 \left(pbest_{ij} - x_{ij}(t)\right) \\ &+ c_2\, rand_2 \left(lbest_{ij} - x_{ij}(t)\right) \end{aligned} \tag{17}$$

  where the term of *gbest* in the previous algorithms has been replaced with the term *lbest*, which means that instead of a global best population, a local best population is used.

$$lbest_{ij} \in N_i \mid f\left(lbest_{ij}\right) = \min f\left(x_{ij}\right), \forall x \in N_i \tag{18}$$

  The neighbor N_i is defined by [82]:

$$\begin{aligned} N_i = pbest_{i-n_{N_i}}(t), \ldots, pbest_{i-1}(t), pbest_i(t), \\ pbest_{i+1}(t), \ldots, pbest_{i+n_{N_i}}(t) \end{aligned} \tag{19}$$

  A particle's best position ($pbest_{ij}$) in a swarm is calculated from the equation:

$$pbest_{ij} = \begin{cases} x_{ij}(t+1), & \text{if } f\left(x_{ij}(t+1)\right) < f\left(x_{ij}(t)\right) \\ pbest_{ij}, & \text{otherwise} \end{cases} \tag{20}$$

  The optimal position of the whole swarm is calculated by the equation:

$$gbest_j \in \{pbest_{1j}, pbest_{2j}, \cdots, pbest_{Nj}\} \mid f(gbest_j) = \min\{f(pbest_{1j}), f(pbest_{2j}), \cdots, f(pbest_{Nj})\} \tag{21}$$

Most applications of PSO have concentrated on the optimization in continuous space, while a lot of work has been done to the discrete optimization problems beginning from the next two papers [83, 86]. In the following, a pseudocode of the particle swarm optimization algorithm is presented.

algorithm `Particle Swarm Optimization`
Initialization
Select the number of swarms
Select the number of particles for each swarm
Initialization of the position and velocity of each particle
Calculation of the initial cost function (fitness function) value of each particle
Keep global best particle (solution) of the whole swarm
Keep personal best of each particle
Main Phase
Do until the maximum number of iterations has not been reached:
    Calculate the velocity of each particle
    Calculate the new position of each particle
    Evaluate the new fitness function of each particle
    Update the best solution of each particle
    Update the best particle of the whole swarm
Enddo
Return the best particle (the global best solution)

Application of Particle Swarm Optimization Algorithm in Vehicle Routing Problems

In this chapter, the papers in which a variant of the particle swarm optimization algorithm is applied for the solution of a variant of the vehicle routing problem are presented. In this section, an analysis is given on how these papers have been selected. As there are a large number of papers in the literature, we will in the most important of them. A very difficult task is to define which of the papers are the "most important" ones. We use a number of criteria to show the impact of each paper in the research community. The first criterion is the impact factor of the journal or the importance of the conference in which the paper has been published. The second criterion is the number of citations (based on Google Scholar) that each paper has received through the years. Papers that have been published in the current year are not expected to have a large number of citations. However, some of them based on their computational results are included in the analysis. Finally, the third criterion is the computational results given in each paper. There are a number of

papers that have demonstrated their PSO variant giving results in one or in a small set of instances. These papers are not analyzed as the effectiveness of their variant is not sufficiently proved. Also, there are a number of papers that, although they have been published in very good journals, they present a PSO variant for the solution of a real-life application and they do not give a comparative analysis with other algorithms from the literature in benchmark instances; thus, it is very difficult to show the effectiveness of the PSO variant. These kinds of papers are not included in the analysis. Finally, there are a number of papers that they present a novel PSO procedure, they apply the method in classic sets of benchmark instances, and their computational results are competitive with the results of the most effective algorithms from the literature. This last category of papers is those that are analyzed in this chapter.

Main Variants of VRP Solved by PSO

The main variants of the vehicle routing problem that a particle swarm optimization algorithm has been applied are the following.

Capacitated Vehicle Routing Problem *Abbreviation:* ***CVRP*** *Definition:* Each vehicle must start and finish its tour at the depot. Capacity constraints of the vehicles. Maximum tour duration of each route. Not split deliveries allowed. Customers have only demands and service time [14, 53, 62, 63].

Open Vehicle Routing Problem *Abbreviation:* ***OVRP*** *Definition:* Same constraints as in the CVRP except that the vehicles do not return in the depot after the service of the customers [12].

Vehicle Routing Problem with Time Windows *Abbreviation:* ***VRPTW*** *Definition:* Same constraints as in the CVRP and in addition each customer must be serviced within a specific time window. Vehicles and depots may, also, have a time window. Minimization, initially, of the number of routes and, then, minimization of the total traveled distance [10, 14, 52, 53, 57, 87].

Vehicle Routing Problem with Simultaneously Pickup and Delivery *Abbreviation:* ***VRPSPD*** *Definition:* Same constraints as in the CVRP. The customers require not only delivery of products but, also, a simultaneous pick up of products from them. It is assumed that the delivery is performed before the pickup and that the vehicle load should never be negative or larger than the vehicle capacity [14, 52, 53, 57].

Vehicle Routing Problem with Stochastic Demands *Abbreviation:* ***VRPSD*** *Definition:* A vehicle with finite capacity leaves from the depot with full load and has to serve a set of customers whose demands are known only when the vehicle arrives to them. A route begins from the depot and visits each customer exactly once and returns to the depot. This is called an a priori tour, and it can be seen as a template for the visiting sequence of all customers [21, 24, 88].

Dynamic Vehicle Routing Problem *Abbreviation:* ***DVRP*** *Definition:* In dynamic problems, part or all of the input is unknown and is revealed dynamically during the design or execution of the routes [74].

Multidepot Vehicle Routing Problem *Abbreviation:* ***MDVRP*** *Definition:* More than one depots are used for the customers' service. There is a possibility for each customer to be clustered and served from only one depot, or the customers may be served from any of the depots using the available fleet of vehicles [20, 89].

Vehicle Routing Problem with Stochastic Travel Times *Abbreviation:* ***VRPSTT*** *Definition:* In this variant, the travel time between every pair of nodes is a random variable related to traffic jam, road maintenance, or weather conditions. The stochasticity appears in the arcs (road) of the network due to unexpected conditions [90].

Vehicle Routing Problem with Fuzzy Demands *Abbreviation:* ***VRPFD*** *Definition:* In this variant, the demands could be represented as fuzzy variables.

Periodic Vehicle Routing Problem *Abbreviation:* ***PVRP*** *Definition:* In this problem, vehicle routes must be constructed over multiple days where during each day within the planning period, a fleet of capacitated vehicles travels along routes that begin and end at a single depot [91]. The objective of the PVRP is to find a set of tours for each vehicle that minimizes total travel cost while satisfying the constraints of the problem [91].

Location Routing Problem *Abbreviation:* ***LRP*** *Definition:* In this variant, the optimal location to be used for the storage facilities have to be decided. From these locations, the vehicles will begin their routes, in a way that the total cost of the routing (distance, fuel, time, etc.) and facility location (running costs, rent or property cost, etc.) will be the minimum. At the same time, the optimal routes for the vehicles have to be found in order to satisfy the demand of the customers [40, 41, 92, 93].

Location Routing Problem with Stochastic Demands *Abbreviation:* ***LRPSD*** *Definition:* In this variant, the same constraints as in the case of the location routing problem hold except that the demands of the customers have stochastic and not deterministic values.

Team Orienteering Problem *Abbreviation:* ***TOP*** *Definition:* This is a variant of the vehicle routing problem with profits or pricing. In this problem, a set of locations is given, each one with a score. The goal is to determine a fixed number of routes, limited in length, that visit some locations and maximize the sum of the collected scores [94]. The objective of the TOP is to construct a certain number of paths starting at an origin and ending at a destination that maximize the total profit without violating predefined limits [95].

Production Routing Problem *Abbreviation:* ***PRP*** *Definition:* This variant is a hybridization of VRP with production problems. These problems are very complicated problems as they include, among others, decisions concerning the number of workers to be hired, the quantities of products to be produced using various strategies, the amount of inventories to be maintained, the assignment of the customers in different manufacturing plans or depots, and, finally, the design of the routes in order to satisfy the customers' demands.

Ship Routing Problem *Abbreviation: SRP Definition:* In this variant, the optimum routing and scheduling of different types of ships are calculated [48–51, 96].

Vehicle Routing Problem with heterogeneous fleet *Abbreviation:* ***HVRP*** *Definition:* In this variant, instead of an homogeneous fleet of vehicles, the company uses a set of vehicles with different capacities, and, thus, the fleet of vehicles is heterogeneous.

Inventory Routing Problem *Abbreviation:* ***IRP*** *Definition:* In this variant of the vehicle routing problem, a simultaneous allocation of inventories and decision of routing schedules is realized. The objective of the IRP is to minimize the total cost (the distribution and inventory costs of retailers) [42, 43].

Vehicle Routing Problem with Uncertain Demands *Abbreviation:* ***VRPUD*** *Definition:* In this variant, the demand is uncertain with unknown distribution.

Vehicle Routing and Scheduling Problem *Abbreviation:* ***VRSP*** *Definition:* In general, the goal of VRSP is to determine the set of trips that a vehicle will make during the day in order to reduce the transportation costs.

Other variants of VRP, combinations of VRP with other problems, or real-life applications of VRP that have been solved using a variant of a particle swarm optimization algorithm are the distance constraint vehicle routing problem [97], the multiple destination routing problem (MVRP) [98], the multivehicle assignment problem (MVAP) [99], the production routing problem [46], the berth allocation problem [100], the disaster relief logistics [101], the garbage collection system [102], the integrated production and distribution [103], the urban transit routing problem [104], the production and pollution routing problem [105], the reducing vehicle emissions and fuel consumption [106], the pedestrian-vehicle mixed evacuation (P-VMEP) [107], the time-dependent vehicle routing problem (TDVRP) [108], the emergency vehicle scheduling problem [109], the emergency logistics (EL) [36, 110], the evacuation vehicle routing problem (EvVRP) [111], the environmental vehicle routing problem (EnVRP) [112], the integrated production scheduling and vehicle routing [113], and the reverse logistics (RL) [114].

Table 1 presents the number of papers (journal papers, international conference papers, and book chapters, denoted as Journal, Conf, and BC in the table, respectively) in which a variant of the particle swarm optimization algorithm has been used for the solution of a variant of the vehicle routing problem. As it was expected, most of the papers concern an application of a PSO variant to the capacitated vehicle routing problem with eight journal papers published. There is no other variant of VRP with more than three journal papers. However, in the vehicle routing problem with time windows, there are a large number of conference papers and book chapters (nine in total). As we can see, a variant of PSO algorithm has been applied in 27 different variants of vehicle routing problem. In total, there are 37 journal papers and 52 international conference papers and book chapters with a PSO variant in a vehicle routing problem.

Although when the particle swarm optimization algorithm was initially proposed, there were no any applications of the algorithm for the solution of the vehicle routing problem; in the last years, a number of researchers have solved a variant of VRP using a PSO algorithm. In Table 2 journals that have published at least one paper with this topic are presented. Journals are sorted based on their impact factor for

Table 1 Number of papers using particle swarm optimization in each variant of the vehicle routing problem

Variant	Journal	Paper	Conf and BC	Paper
CVRP	8	[115–122]	7	[123–129]
VRPSPD	3	[130–132]	4	[133–136]
LRP	2	[137, 138]	2	[139, 140]
HVRP	2	[141, 142]		
VRPTW	2	[143, 144]	9	[145–153]
VRPUD			2	[154, 155]
TOP	2	[95, 156]	3	[157–159]
MVAP			1	[99]
DVRP	2	[160, 161]	3	[162–164]
OVRP	2	[165, 166]	4	[167–170]
VRPSD	1	[171]	3	[172–174]
PVRP	3	[175–177]		
RL			1	[114]
VRPFD	1	[178]	1	[179]
VRPSTT			1	[180]
MDVRP	1	[181]	4	[182–185]
DCVRP	1	[97]		
VRSP	1	[186]		
EvVRP	1	[111]	2	[36, 187]
MVRP			1	[98]
EL			1	[110]
LRPSD	1	[188]		
EnVRP			1	[112]
IRP	1	[189]		
TDVRP	1	[108]		
PRP	1	[46]		
SRP	1	[96]		

2015 (denoted as I.F. in the table) that it is presented in column three. Finally, in columns 4 and 5, the number of papers (denoted as Nr in the table) that has been published in each journal and the total number of citations of all papers based on Google Scholar on 30 May 2017 (denoted as TNrCit in the table) are presented.

Different PSO Variants

As it is mentioned earlier, the main problem of the application of a PSO algorithm for the solution of a VRP was that the PSO algorithm is suitable for continuous optimization problems and the VRP is a combinatorial optimization problem with a specific structure in the solution which needs in most cases a path representation in order to have an effective algorithm. Thus, there are a number of algorithms that use a 0/1 representation of the solution. However, these algorithms are not the most effective algorithms for the solution of this kind of problems. We mentioned that

Table 2 Journals that have published at least one PSO algorithm for the solution of a vehicle routing problem variant

Journal	Abbreviation	I.F.	Nr	TNrCit
Transportation Science	TS	3.295	1	60
Networks and Spatial Economics	NETS	3.250	1	23
Expert Systems with Application	ESWA	2.981	3	295
Applied Soft Computing	ASOC	2.857	6	201
Neurocomputing	NC	2.392	1	2
Engineering Applications of Artificial Intelligence	EAAI	2.368	1	130
Transportation Research Part E	TRE	2.279	2	57
IEEE Transactions on Systems, Man, and Cybernetics-Part C	IEEE SMC	2.171	1	74
Computers and Industrial Engineering	CIE	2.086	6	363
Journal of Intelligent Manufacturing	JIM	1.995	3	94
Computers and Operations Research	COR	1.988	2	297
Entropy	Entropy	1.743	1	20
Measurement	Measurement	1.742	1	12
International Journal of Advanced Manufacturing Technology	IJAMT	1.568	2	57
Annals of Operations Research	ANOR	1.406	1	33
Applied Mathematics and Computation	AMC	1.345	1	38
Applied Intelligence	APIN	1.215	1	8
Optimization Letters	OptL	1.019	1	5
Journal of Zhejiang University SCIENCE A	SCIENCE A	0.941	1	198
Memetic Computing	MEME	0.900	1	13
International Journal of Operational Research	IJOR	–	1	49
Procedia Engineering	PrE	–	1	8
Journal of Mathematical Modelling and Algorithms	JMMA	–	1	67
American Journal of Applied Sciences	AJAS	–	1	13
Transportation Research Journal	TRJ	–	1	4

from the first years that PSO was proposed, there were implementations of PSO [83] suitable for discrete optimization problems but not for routing type problems. Thus, the first effective application of a PSO algorithm in VRP was in 2006 [116]. Since then a number of different algorithms have been proposed that are suitable for the solution of a VRP variant. Most of them are hybridized with a local search algorithm, as almost every evolutionary algorithm if it is applied for the solution of a VRP problem. In the following, we present and analyze the most important variants of PSO that have been applied for the solution of a VRP variant. The choice was performed using the same criteria that were mentioned earlier, especially the one concerning the effectiveness of the algorithms.

SR-PSO. One of the most used representations of PSO for the solution of a VRP variant denoted as SR-PSO (SR-1 PSO or SR-2 PSO) was proposed in [115] where two representations of the particles were used for the solution of the CVRP. In the

first one, the particle consists of $n + 2m$ dimensions. The first n dimensions are related to customers (each customer is represented by one dimension). The last $2m$ dimensions are related to vehicles (each vehicle is represented by two dimensions as the reference point in the Cartesian map). The priority matrix of vehicles is constructed based on the relative distance between these points and customers' location [115]. A customer is prioritized to be served by the vehicle which has the closer distance. In the second representation, the particle consists of a $3m$-dimensional particle where it is decoded as a real number. All dimensions are related to vehicles; each vehicle is represented by three dimensions: two dimensions for the reference point and one dimension for the vehicle coverage radius [115]. The algorithms use a number of local search algorithms. A number of papers either from the same authors or from other research groups have used the same representations for solving a VRP variant.

The research group that proposed this representations have applied them for the solution of the vehicle routing problem with time windows [143], of the vehicle routing problem with simultaneously pickups and deliveries [130] (the most important publication of an application of PSO-based algorithm for solving a VRP based on the number of citations), of the multidepot VRP with pickup and deliveries [182] and [181], and of the location routing problem [137].

Other researchers that used the same encoding and decoding scheme for solving a VRP variant are the one from Hu et al. [190] that proposed a hybrid chaos-particle swarm optimization algorithm (HPSO) for solving VRPTW. In [191], a variant of the algorithm was proposed for the solution of a vehicle routing problem with uncertain demands. In [141], a variant of the algorithm was presented for a rich vehicle routing problem, the vehicle routing problem with heterogeneous fleet, mixed backhauls, and time windows (VRPHFMBTW).

Quantum PSO—QPSO. One of the first PSO implementations for the solution of the CVRP was the one published in [116]. In this PSO implementation, a quantum discrete PSO algorithm is used [192]. The algorithm is hybridized with a simulated annealing algorithm [193] for the improvement of the particles. They use a zero one encoding scheme where value equal to one means that the vehicle serves the customer and value equal to zero means that another vehicle will serve the customers. The dimension of the vector is equal to $N \times K$ where N is the number of nodes and K is the number of vehicles.

HybGenPSO. In [118], a hybridization of a genetic algorithm with a particle swarm optimization, algorithm for the solution of the CVRP was presented. In this memetic algorithm, the role of the genetic algorithm was to create the new generations, and the role of the PSO algorithm was to improve the population between two different generations. Thus, it was a memetic algorithm with the difference that usually in a memetic algorithm, each member of the population is evolved between two generations using a local search algorithm, while in this memetic algorithm, the population was evolved using a global search algorithm. This idea was inspired by the fact that in real life, each member of a population is evolved by reacting with other members of the population and not by itself. The algorithm uses the inertia equation for the velocities (Eq. 6) and was tested in the

two classic sets of benchmark instances, and a number of best-known solutions were found.

HybPSO. In [119], a hybrid particle swarm optimization algorithm was used for the solution of the capacitated vehicle routing problem. The algorithm is an improved version of the algorithm used in [138] for the solution of the location routing problem, and it uses the same three algorithms in the hybridization phase, the multiple phase neighborhood search—greedy randomized adaptive search procedure (MPNS-GRASP), the expanding neighborhood search (ENS) [194], and the path relinking (PR) [195]. The first method is used for the creation of the initial solutions, the second one is used as a local search phase, and the third one is used for the movement of the particles. The difference of the algorithm proposed in [119] from the algorithm denoted as combinatorial neighborhood topology PSO (proposed in [126]) is that the first one uses a transformation between continuous and discrete values and vice versa based on the relative position indexing [196] and it, also, uses a procedure that does not affect the solution of each particle. In the combinatorial neighborhood topology PSO, no transformation is needed at all. The algorithm proposed in [119] uses the inertia equation for the velocities (Eq. 6). The algorithm was tested in a classic set of benchmark instances, and a number of best-known solutions were found.

In [167], a new version of the hybrid particle swarm optimization algorithm that was presented in [119] for the solution of the CVRP was presented where instead of using the expanding neighborhood search algorithm for the improvement of each particle separately, the variable neighborhood search algorithm was used [197]. Another difference is that the inertia velocities equation has been replaced by the constriction velocities equation (Eq. 8).

In [171], an improved version of the hybrid particle swarm optimization algorithm that was presented in [167] for the solution of the OVRP was presented where instead of using the variable neighborhood search algorithm for the improvement of each particle separately, a simpler local search algorithm based on 2-opt and 3-opt algorithms was used. Also, in the initialization phase of the algorithm, the particles start from random values, thus avoiding the use of a more sophisticated procedure. The reason that this simpler local search algorithm and the random initial values were used was that the calculation of the objective function of the VRPSDs is much more difficult and time-consuming than the calculation of the objective function of a simpler variant of VRP and, thus, the use of a simpler local search algorithm avoids the increase of the computational time of the whole procedure. Eight different versions of velocities equation were used in order to test which one of them is the most appropriate for the selected problem.

CNTPSO. In [138], a hybrid particle swarm optimization algorithm was used for the solution of the location routing problem. The PSO algorithm was hybridized with three other effective algorithms, the multiple phase neighborhood search—greedy randomized adaptive search procedure (MPNS-GRASP) [198], the expanding neighborhood search (ENS) [194], and the path relinking [195].

The novelty of the paper was that a procedure was used where there is no transformation from continuous to discrete values and the movement of the particles

was performed using a path relinking procedure [195], where the current solution of the particle was used as starting solution and the target solution was either the local best particle or the global best particle. This was the first step of the very successful and effective topology that was denoted as combinatorial neighborhood topology and was used for the solution of the capacitated vehicle routing problem [126]. The velocities equation used was the inertia one (Eq. 6). In [126], one of the most successful versions of the PSO for the CVRP was published, denoted as combinatorial neighborhood topology PSO. This algorithm does not need a transformation from continuous to discrete values, and, thus, there is no loss of any information of good solutions that have been created in an iteration. In this algorithm, the position equation (Eq. 3) of the particles has been replaced by a path relinking strategy. The role of the velocities equation is limited to show which particle (or combination of particles) will be followed by the selected particle based on some conditions. This version of PSO is a very efficient version as it was proved by its application in the classic set of benchmark instances where better solutions were found compared to the ones found by other versions of PSO in less computational time.

In [172], an improvement of the CNTPSO was presented where a combination of this topology with an expanding neighborhood topology (ENT) is used. In this topology, there are not a standard number of local neighbors, but the number of local neighbors begins from a small number and increases using some conditions. When the number of neighbors becomes equal with the number of particles, then a global best PSO algorithm is used. The algorithm uses the same local search algorithm and the same initialization procedure as the algorithm presented in [171].

An improved version of CNTPSO with expanding neighborhood topology was presented in [188] for the solution of the location routing problem and of the location routing problem with stochastic demands. The difference of this algorithm from the initial CNTPSO is that it uses simultaneously a global neighborhood topology and the expanding neighborhood topology as it was described previously.

An adaptive version of CNTPSO is presented in [150] for the solution of the vehicle routing problem with time windows. In this version, all parameters (acceleration coefficients, iterations, local search iterations, upper and lower bounds of the velocities and of the positions, and number of particles in each swarm) are adapted during the procedure, and, thus, the algorithm works independently and without any interference from the user.

h_PSO. In [132, 133], two different algorithms for the solution of VRPSPD problem using a PSO algorithm were presented. Initially, it was presented as a conference paper [133], and its improved version was published as a journal paper in [132]. The authors presented a hybridization of PSO algorithm with a variable neighborhood search algorithm and used an annealing-based mechanism in order to maintain the diversity of particles. The inertia velocities equation was used (Eq. 6). They used a representation that began with a giant tour without taking into account the route restrictions, and, then, they used a decoding scheme to partition the giant tour into feasible routes. The procedure that they used is denoted as split procedure,

and it was first presented by Prins in the frame of an evolutionary algorithm for the solution of the CVRP [199]. The algorithms were tested in the usually used benchmark set of instances for the problem, and a number of new best solutions were found.

DAPSO. The most important application of a particle swarm optimization algorithm for the solution of a dynamic vehicle routing problem was presented in [160]. In this paper, the authors solved the DVRP with dynamic requests, and a procedure was used where a partial static VRP was solved each time a new request was received. The authors hybridized the particle swarm optimization algorithm with a variable neighborhood search algorithm for the solution of the problem (DAPSO). The representation used in this work is a simple discrete representation which expresses the route of m vehicles over the n customers to serve. The representation allows the insertion of dynamic customers in the already planned route, and it is a permutation of the customers where as a DVRP is solved, the authors keep the coordinates of the customers, the time in which each customer is served, and if a customer has been served or not. Another very interesting application of PSO algorithm in this problem was presented in [162] where a multiswarm PSO algorithm was applied, a number of benchmark instances were solved, and comparisons with algorithms from the literature were given. Finally, a third publication of the same group of authors using a PSO algorithm was presented in [163].

SPSO. One of the most important applications of a PSO algorithm in the solution of the vehicle routing problem with time windows is the one presented in [144]. In this paper, the proposed SPSO-VRPTW treated the discrete search space as an arc set of the complete graph that was defined by the nodes in the VRPTW and regarded the candidate solution as a subset of arcs. A set-based representation method was proposed to characterize the discrete search space. The search space is represented by a universal set S. The elements in S can be divided into D dimensions.

2MPSO. A two-phase particle swarm optimization algorithm for the solution of the DVRP was presented in [164], and later it was improved in [161]. They presented an algorithm where a new equation of velocities for the particles was presented where local and global search topologies were combined.

PVPSO. In [165], a PSO algorithm for the OVRP is presented. The authors used a standard PSO for the encoding and the decoding procedure where all the elements of the positions vector are sorting in descending order and, then, the first is added in the first route, the second in the route with the least residual capacity, and so on until all elements are inserted in a route. The routes, then, were improved using one-move local search. They used the inertia equation of velocities (Eq. 6), and they tested their algorithm in a small set of instances and found very good solutions.

GLNPSO. A very interesting application of a PSO in the VRPFDs was the one presented in [178]. In this problem, the variant of VRP used included, also, soft time windows constraints, and the authors considered two objective functions, the minimization of the total travel cost and the maximization of the average satisfaction level of all customers in a fuzzy environment. The authors used a global-local-

neighbor particle swarm optimization with exchangeable particles. An illustrative example that explains how the PSO algorithm had been applied in this problem was presented and analyzed in details.

PMPSO. In [117], the authors presented an approach that uses a probability matrix as the main device for particle encoding and decoding. In the decoding phase, not only the assignment of the customers in vehicles was realized, but, also, the routing of the customers was calculated. They used a number of local search algorithms (1-1 exchange, 2-opt, Or-opt) in order to improve the solution of the particles.

MOPSO. A very interesting formulation of a multiobjective competitive open vehicle routing problem with time windows was presented in [166]. In this problem, the reaching time to customers affects the sales amount. Therefore, distributors intend to service customers earlier than rivals to obtain the maximum sales. Moreover, a part of a driver's benefit is related to the amount of sales. Thus, the balance of goods carried in each vehicle is important in view of the limited vehicle capacities. They gave a complete analysis of the new problem and of the formulation, and they used a multiobjective PSO algorithm for the solution of the problem. In this multiobjective algorithm, the nondominated solutions are stored in a repository, and, then, if a new solution dominates a solution from the repository or a personal best solution of a particle, then the new solution replaces the dominated solution.

StPSO. A very interesting paper was published in [186]. In this paper, the authors solved a very complicated vehicle routing and scheduling problem with cross-docking. They use an encoding scheme to represent the scheduling in a string. Thus, all are encoded as genes in a 1-by-n string where the length is equal to the number of vehicles. Thus, the digit in the i_{th} chromosome indicates the route number to which the i_{th} vehicle is assigned [186]. A similar variant of the algorithm has been proposed in [177] for the solution of the periodic vehicle routing problem.

NPSO. In [200], a nested PSO was given for the solution of the CVRP. Initially, feasible solutions are created in clusters using sweep algorithm, and, then, a route optimization was performed inside the clusters. The PSO was used in both phases, in the first phase for reorganizing the routes and in the second phase for leading to the optimization of the routes.

DPSO. A discrete PSO algorithm is presented in [156] for the solution of the team orienteering problem. The authors hybridized their algorithm with a variable neighborhood search algorithm. Another application of DPSO for the solution of the CVRP was proposed in [120]. The authors hybridized their algorithm with an iterated local search algorithm.

DHPD. In [201], a hybridization of a PSO algorithm with a differential evolution algorithm was presented. In the paper, an indirect representation was proposed. For N customers, each individual was encoded as a real number vector with N dimensions. The integer part of each dimension or element in the vector represents the vehicle. Thus, the same integer part represents the customer in the same vehicle. The fractional part represents the sequence of the customer in the vehicle [201].

Another similar formulation of the algorithm was proposed in [169] for the solution of the OVRP. The authors presented a very interesting representation of the particles' solution where in the encoding procedure, the customers took real values and in the decoding procedure, the customers with the same integer part are assigned in the same team (vehicle or group of vehicles if the capacity of the vehicle was violated). The authors tested their algorithm using the classic set of benchmark instances, and they obtained very good results.

MODPSO. In [136], a PSO algorithm was used for the solution of the VRPSPD. The very interesting part of the algorithm was the solution representation where the m customers were divided in $m + 1$-dimensional particles. In the decoding process of the algorithm, the particles were transformed to vehicle allocation s with the sweep algorithm, and, then, the priority matrices of customers served by the same vehicle were evaluated. Based on the two matrices, the vehicle routes were constructed.

Analysis of the Algorithms

In Table 3, the most important papers that use a variant of PSO for a solution of a variant of VRP are presented. The selection of the papers has been performed based on the criteria mentioned previously. The ranking in Table 3 is based on the number of citations that each of these papers has on 30 May 2017 on Google Scholar. Also, in Table 3, papers with a small number of citations are presented, but these papers either present very effective algorithms based on their computational results or have great potential for fast increase on the number of citations due to the year of publication. In Table 3, in addition to the number of citations, it is presented which variant of the VRP is solved (column 2), with which variant of PSO (column 3), in which journal, book, and conference it is published (column 4), and the year published (column 5). The number of citations is given in the last column.

As we can see, there is only one paper with more than 200 citations [130] with 289 citations and one that it is near to 200 (paper [116] with 198 citations). Two other papers have more than 150 citations (paper [118] with 167 and paper [115] with 162 citations). Finally, there are two papers with more than 100 citations, paper [119] with 130 citations, and paper [144] with 117 citations. In total, there are six papers with more than 100 citations. Four of them present algorithms for solving the capacitated vehicle routing problem, and the other two present algorithms for solving the vehicle routing problem with simultaneous pickup and delivery. Also, two of them ([130] and [115]) are works from a specific research group that have made a significant contribution in the field. In Table 3, there are, also, other four papers from the same research group (paper [143] for the solution of the VRPTW with 49 citations, paper [182] for the solution of MDVRP with 23 citations, paper [181] for the solution of MDVRP with 8 citations, and paper [137] for the solution of LRP with 3 citations). The other research group with significant contribution in the field studied in this chapter is the one that has two publications in the six

Table 3 Total number of citations in the most important papers that use a PSO variant for solving a VRP variant

Paper	VRP variant	PSO variant	Published in	Year	Citations
Ai and Kachitvichyanukul [130]	VRPSPD	SR-PSO	COR	2009	289
Chen et al. [116]	CVRP	QPSO	SCIENCE A	2006	198
Marinakis and Marinaki [118]	CVRP	HybGenPSO	ESWA	2010	167
Ai and Kachitvichyanukul [115]	CVRP	SR-PSO	CIE	2009	162
Marinakis et al. [119]	CVRP	HybPSO	EAAI	2010	130
Goksal et al. [132]	VRPSPD	h_PSO	CIE	2013	117
Khouadjia et al. [160]	DVRP	DAPSO	ASOC	2012	78
Marinakis et al. [171]	VRPSD	HybPSO	ASOC	2013	76
Gong et al. [144]	VRPTW	SPSO	IEEE SMC	2012	74
MirHassani and Abolghasemi [165]	OVRP	PVPSO	ESWA	2011	70
Marinakis and Marinaki [138]	LRP	CNTPSO	JMMA	2008	67
Moghaddam et al. [191]	VRPUD	SR-PSO	CIE	2012	57
Xu et al. [178]	VRPTW	GLNPSO	TRE	2011	55
Ai and Kachitvichyanukul [143]	VRPTW	SR-PSO	IJOR	2009	49
Belmecheri et al. [141]	HVRP	SR-PSO	JIM	2013	42
Yao et al. [142]	HVRP	IPSO	ANOR	2016	33
Kim and Son [117]	CVRP	PMPSO	JIM	2012	31
Norouzi et al. [166]	OVRP	MOPSO	NETS	2012	23
Sombuntham and Kachitvichayanukul [182]	MDVRP	SR-PSO	IMECS 2010	2010	23
Vahdani et al. [186]	VRSP	StPSO	JIM	2012	21
Hu et al. [190]	VRPTW	SR-PSO	Entropy	2013	20
Okulewicz and Mańdziuk [164]	DVRP	2MPSO	ICAISC 2013	2013	16
Kanthavel and Prasad [200]	CVRP	NPSO	AJAS	2011	13
Muthuswamy and Lam [156]	TOP	DPSO	MEME	2011	13
Norouzi et al. [175]	PVRP	IPSO	Measurement	2015	12
Marinakis and Marinaki [126]	CVRP	CNTPSO	EvoCOP	2013	9
Qi [120]	CVRP	DPSO	PrE	2011	8
Marinakis and Marinaki [167]	OVRP	HybPSO	ANTS	2012	8
Kachitvichyanukul et al. [181]	MDVRP	SR-PSO	CIE	2015	8
Hu and Wu [201]	OVRP	DHPD	WCICA	2010	7
Marinakis and Marinaki [172]	VRPSD	CNTPSO	GECCO	2013	7
Marinakis [188]	LRPSD	CNTPSO	ASOC	2015	5
Tavakkoli Moghaddam et al. [177]	PVRP	StPSO	TRJ	2012	4
Liu and Kachitvichyanukul [137]	LRP	SR-PSO	IEAS	2013	3
Okulewicz and Mańdziuk [161]	DVRP	2MPSO	ASOC	2017	0

more significant publications in the field ([119] and [118]) and other six publications that are presented in Table 3 (paper [171] for the solution of the VRPSD with 76 citations, paper [138] for the solution of LRP with 67 citations, paper [126] for the solution of CVRP with 9 citations, paper [167] for the solution of OVRP with 8 citations, paper [172] for the solution of VRPSD with 7 citations, paper [188] for the solution of LRP and LRPSD with 5 citations). Of course, there are papers with significant impact that are presented in this table as papers with 78 citations [160], papers with 74 citations [144], or papers with 70 citations [165] which have influenced a large number of researchers.

One of the most important tables in this survey is the last one (Table 4). In this table, the computational results of the most important papers are summarized and presented analytically. In this table, they are not presented papers that the proposed algorithm was tested in one or in a small set of instances, especially when these instances are not available in the Internet. Also, they are not presented papers that the proposed PSO algorithm solves a real-life problem as we cannot compare the results and we cannot see their effectiveness. The only papers that are presented are the ones that the proposed algorithm was tested in one or more well-known set(s) of benchmark instances, and the authors presented their results in such a way that the results are comparable with the computational results of other algorithms from the literature. In Table 4 in the first column, a reference of the paper is given, in the second column the problem(s) that is (are) solved is mentioned, and in the third, fourth, and fifth columns, the number of sets of benchmark instances, the number of instances in each set (NBI), and the range in which the number of nodes of each set fluctuates are given for each paper, respectively. Finally, in the last four columns, the number of new best solutions found in the paper in the year of its publication (NBS), the number of instances in which the proposed algorithm found a solution equal to the best-known published solution (EBS), the average quality of the solutions, and the range of the qualities of the solutions are given, respectively. The quality of a solution is calculated using the following equation $\text{Quality} = \frac{(c_{\text{PSO}} - c_{\text{BKS}})}{c_{\text{BKS}}}\%$, where c_{PSO} denotes the cost of the solution found by the mentioned PSO algorithm in the corresponding paper and c_{BKS} is the cost of the best-known solution.

It was a difficult task to summarize all the results and to create this table as every researcher presents the produced results in a different way. For example, in the most important and cited paper [130], the authors present the average results in the well-known benchmark instances, and, then, they give an extensive analysis in variants of the studied problem, and it was difficult to present analytically the results of the paper. In other papers, a new set of benchmark instances is created [171], and, thus, it was difficult to give comparisons for this set of instances. However, in most of the papers that are presented in this table, we could extract the information that we would like to have in order to see the effectiveness of each one of the algorithms. In most of the algorithms presented in the table, at least a number of EBS were found, and in few of them, new best solutions were given. It is very difficult to analyze each paper separately due to space limitation, and this analysis can be found easily in each one of the papers. However, we could say that the application of PSO in VRP variants gives very interesting computational results. In most cases,

Table 4 Computational results of papers that used a variant of a PSO algorithm for the solution of a vehicle routing problem variant

Paper	VRP	Sets	NBI	Nodes	NBS	EBS	Quality	Quality range
Ai and Kachitvichyanukul [130]	VRPSPD	3	84	20–40	Average results are given			
			40	50	Average results are given			
			14	50–199	Average results are given			
Chen et al. [116]	CVRP	1	16	29–134	0	7	0.969	0.00–4.56
Marinakis and Marinaki [118]	CVRP	2	14	51–200	0	10	0.046	0.00–0.23
			20	200–483	0	1	0.60	0.00–0.91
Ai and Kachitvichyanukul[115]	CVRP	2	16	29–134	0	10	0.065	0.00–0.29
			14	51–200	0	4	0.874	0.00–2.51
Marinakis et al. [119]	CVRP	1	14	51–200	0	7	0.084	0.00–0.29
Goksal et al. [132]	VRPSPD	2	40	50	0	40	0.00	0.00
			14	50–199	0	6	0.00	0.00–3.16
Khouadjia et al. [160]	DVRP	1	21	50–199	5	0	NM|	NM
Marinakis et al. [171]	VRPSD	2	21	51–200	21	0	−1.105	−3.13 to −0.001
			40	16–60	New set of benchmark instances			
Gong et al. [144]	VRPTW	3	56	25	0	0	8.72	0.18–60.56
		29 best solutions based on number of vehicles						
			56	50	0	0	6.99	0.00–41.33
		32 best solutions based on number of vehicles						
			56	100	6	9	2.96	−12.69 to 15.70
MirHassani and Abolghasemi [165]	OVRP	1	15	32–50	0	12	0.196	0.00–2.605
Marinakis and Marinaki [138]	LRP	1	19	12–318	6	13	−0.08	−0.68 to 0.00
Moghaddam et al. [191]	VRPUD	1 (CVRP)	60	32–101	0	53	0.04	0.00–1.027

Ai and Kachitvichyanukul [143]	VRPTW	2	56	25	0	0	0.323	0.2–1.2
			56	50	0	0	1.148	0.2–7.2
Kim and Son [117]	CVRP	2	16	29–134	0	10	0.142	0.00–0.733
			14	51–200	0	6	0.712	0.00–2.95
Sombuntham and Kachitvichayanukul [182]	MDVRP	1	29	100	0	15	0.96	0–13.94
Kanthavel and Prasad [200]	CVRP	1	16	29–134	0	16	0.00	0.00–0.00
Qi [120]	CVRP	1	12	32–80	0	4	0.577	0.00–2.40
Marinakis and Marinaki [167]	OVRP	3	14	51–200	0	7	0.11	0.00–0.32
			8	51–200	0	4	0.13	0.00–0.38
			8	200–480	0	0	0.07	0.01–0.21
Marinakis and Marinaki [126]	CVRP	2	14	51–200	0	11	0.019	0.00–0.14
			20	200–483	0	1	0.37	0.00–0.81
Marinakis and Marinaki [172]	VRPSD	1	40	16–60	27	13	−0.64	−3.18 to 0.00
Liu and Kachitvichyanukul [137]	LRP	1	30	50–200	1	0	3.29	−0.02 to 16.12
Marinakis [188]	LRP	6	16	12–150	0	8	0.24	0.00–2.12
			30	20–200	0	13	0.42	0.00–2.54
			36	100–200	0	6	0.86	0.00–3.10
	LRPSD	16	12–150	15	1	−0.11	0.00–0.41	
			30	20–200	10	20	0.00	− 0.02 to 0.00
			36	100–200	32	4	−0.04	−0.08 to 0.00

the computational results are competitive with the results of the most effective algorithms from the literature. Thus, alternative propositions exist for someone that he/she would like to apply a PSO algorithm for the solution of a VRP variant.

Conclusions

In this chapter, an analytical review of the papers proposing and applying a PSO algorithm for the solution of VRP variants was presented. In total, around 100 papers were found. In some of them, new best solutions in VRP variants were presented, in other papers the authors presented the solutions that have been found that are very good and competitive solutions and in some instances equal to the best-known solutions but not new best solutions in the VRP variant solved, in other papers the authors presented and solved a new variant of the VRP with a PSO algorithm, and, finally, there are some papers that gave only illustrative examples of how the authors worked in specific instances of a VRP variant. In general, this chapter gives to the research community a basis that someone could use if he/she would like to apply a PSO algorithm for solving a specific variant of the VRP as he/she could find what has already been published and with what results and he/she could see how the researchers cope with the problems that may arise from the application of a PSO algorithm to the VRP or to its variants.

Cross-References

▸ Hyper-heuristics

Competing Interest Declaration The author(s) has no competing interests to declare that are relevant to the content of this manuscript.

References

1. Dantzig GB, Ramser JH (1959) The truck dispatching problem. Manag Sci 6(1):80–91
2. Christofides N (1985) Vehicle routing. In: Lawer EL, Lenstra JK, Rinnoy Kan AHG, Shmoys DB (eds) The traveling salesman problem: a guided tour of combinatorial optimization. Wiley, Chichester, pp 431–448
3. Christofides N, Mignozzi A, Toth P (1979) The vehicle routing problem. In: Christofides N (ed) Combinatorial optimization. Wiley, Chichester, pp 315–338
4. Fisher ML (1995) Vehicle routing. In: Ball MO, Magnanti TL, Momma CL, Nemhauser GL (eds) Network routing. Handbooks in operations research and management science, vol 8. North Holland, Amsterdam, pp 1–33
5. Laporte G, Semet F (2002) Classical heuristics for the capacitated VRP. In: Toth P, Vigo D (eds) The vehicle routing problem. Monographs on discrete mathematics and applications. SIAM, Philadelphia, pp 109–128

6. Laporte G, Gendreau M, Potvin J-Y, Semet F (2000) Classical and modern heuristics for the vehicle routing problem. Int Trans Oper Res 7:285–300
7. Cordeau JF, Deasulniers G, Desrosiers J, Solomon MM, Soumis F (2002) VRP with time windows. In: Toth P, Vigo D (eds) The vehicle routing problem. Monographs on discrete mathematics and applications. SIAM, Philadelphia, pp 157–193
8. Desrochers M, Lenstra JK, Savelsberg MWP, Soumis F (1988) Vehicle routing with time windows: optimization and approximation. In: Golden BL, Assad AA (eds) Vehicle routing: methods and studies. North Holland, Amsterdam, pp 65–84
9. Desrosiers J, Dumas Y, Solomon MM, Soumis F (1995) Time constraint routing and scheduling. In: Ball MO, Magnanti TL, Momma CL, Nemhauser GL (eds) Network routing, handbooks in operations research and management science, vol 8. Elsevier Science B V, Amsterdam, pp 35–140
10. Solomon MM, Desrosiers J (1988) Time window constrained routing and scheduling problems. Transp Sci 22(1):1–13
11. Solomon MM, Baker EK, Schaffer JR (1988) Vehicle routing and scheduling problems with time windows constraints. In: Golden BL, Assad AA (eds) Vehicle routing: methods and studies. North Holland, Amsterdam, pp 85–105
12. Sariklis D, Powell S (2000) A heuristic method for the open vehicle routing problem. J Oper Res Soc 51(5):564–573
13. Mosheiov G (1998) Vehicle routing with pickup and delivery: tour – partitioning heuristics. Comput Ind Eng 34:669–684
14. Toth P, Vigo D (2002) The vehicle routing problem. Monographs on discrete mathematics and applications. SIAM, Philadelphia
15. Caretto C, Baker B (2002) A GRASP interactive approach to the vehicle routing problem with backhauls. In: Ribeiro CC, Hansen P (eds) Essays and surveys on metaheuristics. Kluwer Academic Publishers, Norwell, pp 185–199
16. Casco DO, Golden BL, Wasil EA (1988) Vehicle routing with backhauls: models, algorithms, and case studies. In: Golden BL, Assad AA (eds) Vehicle routing: methods and studies. North Holland, Amsterdam, pp 127–147
17. Toth P, Vigo D (2002) VRP with backhauls. In: Toth P, Vigo D (eds) The vehicle routing problem. Monographs on discrete mathematics and applications. SIAM, Philadelphia, pp 195–224
18. Chao IM, Golden BL, Wasil E (1993) A new heuristic for the multi-depot vehicle routing problem that improves upon best-known solutions. Am J Math Manag Sci 13(3–4):371–406
19. Laporte G, Nobert Y, Taillefer S (1988) Solving a family of multi-depot vehicle routing and location routing problems. Transp Sci 22:161–172
20. Montoya-Torres JR, Franco JL, Isaza SN, Jimenez HF, Herazo-Padilla N (2015) A literature review on the vehicle routing problem with multiple depots. Comput Ind Eng 79:115–129
21. Gendreau M, Laport G, Seguin R (1996) Stochastic vehicle routing. Eur J Oper Res 88:3–12
22. Jaillet P, Odoni AR (1988) The probabilistic vehicle routing problem. In: Golden BL, Assad AA (eds) Vehicle routing: methods and studies. North Holland, Amsterdam, pp 293–318
23. Powell WB, Jaillet P, Odoni A (1995) Stochastic and dynamic networks and routing. In: Ball MO, Magnanti TL, Momma CL, Nemhauser GL (eds) Network routing. Handbooks in operations research and management science, vol 8. Elsevier Science B V, Amsterdam, pp 141–295
24. Stewart WR, Golden BL (1983) Stochastic vehicle routing: a comprehensive approach. Eur J Oper Res 14:371–385
25. Gendreau M, Potvin JY (1998) Dynamic vehicle routing and dispatching. In: Crainic TG, Laporte G (eds) Fleet management and logistics. Kluwer Academic Publishers, Boston, pp 115–125
26. Psaraftis HN (1988) Dynamic vehicle routing problems. In: Golden BL, Assad AA (eds) Vehicle routing: methods and studies. North Holland, Amsterdam, pp 223–248
27. Psaraftis HN (1995) Dynamic vehicle routing: status and prospects. Ann Oper Res 61: 143–164

28. Angelelli E, Speranza MG (2002) The periodic vehicle routing problem with intermediate facilities. Eur J Oper Res 137(2):233–247
29. Archetti C, Speranza MG, Hertz A (2006) A tabu search algorithm for the split delivery vehicle routing problem. Transp Sci 40(1):64–73
30. Gendreau M, Laporte G, Musaraganyi C, Taillard ED (1999) A tabu search heuristic for the heterogeneous fleet vehicle routing problem. Comput Oper Res 26:1153–1173
31. Vigo D (1996) A heuristic algorithm for the asymmetric capacitated vehicle routing problem. Eur J Oper Res 89(1):108–126
32. Bortfeldt A (2012) A hybrid algorithm for the capacitated vehicle routing problem with three-dimensional loading constraints. Comput Oper Res 39(9):2248–2257
33. Gendreau M, Iori M, Laporte G, Martello S (2008) A Tabu search heuristic for the vehicle routing problem with two-dimensional loading constraints. Networks 51(1):4–18
34. Lahyani R, Khemakhem M, Semet F (2015) Rich vehicle routing problems: from a taxonomy to a definition. Eur J Oper Res 241:1–14
35. Ngueveu SU, Prins C, Calvo RW (2010) An effective memetic algorithm for the cumulative capacitated vehicle routing problem. Comput Oper Res 37(11):1877–1885
36. Yusoff M, Ariffin J, Mohamed A (2011) A multi-valued discrete particle swarm optimization for the evacuation vehicle routing problem. In: Tan Y et al (eds) ICSI 2011, Part I. LNCS, vol 6728. Springer, Berlin/Heidelberg, pp 182–193
37. Lin C, Choy KL, Ho GTS, Chung SH, Lam HY (2014) Survey of green vehicle routing problem: past and future trends. Expert Syst Appl 41:1118–1138
38. Bektas T, Laporte G (2011) The pollution-routing problem. Transp Res B Methodol 45(8):1232–1250
39. Laporte G (1988) Location routing problems. In: Golden BL, Assad AA (eds) Vehicle routing: methods and studies. North Holland, Amsterdam, pp 163–198
40. Nagy G, Salhi S (2007) Location-routing: issues, models and methods. Eur J Oper Res 177:649–672
41. Prodhon C, Prins C (2014) A survey of recent research on location-routing problems. Eur J Oper Res 238:1–17
42. Campbell A, Clarke L, Kleywegt A, Sawelsberg M (1998) The inventory routing problem. In: Crainic TG, Laporte G (eds) Fleet management and logistics. Kluwer Academic Publishers, Boston, pp 95–113
43. Campbell A, Clarke L, Sawelsberg M (2002) Inventory routing in practice. In: Toth P, Vigo D (eds) The vehicle routing problem. Monographs on discrete mathematics and applications. SIAM, Philadelphia, pp 309–330
44. Fedegruen A, Simchi-Levi D (1995) Analysis of vehicle routing and inventory routing problems. In: Ball MO, Magnanti TL, Momma CL, Nemhauser GL (eds) Network routing. Handbooks in operations research and management science, vol 8. Elsevier Science B V, Amsterdam, pp 297–373
45. Javid AA, Azad N (2010) Incorporating location, routing and inventory decisions in supply chain network design. Transport Res E Log Transp Rev 46(5):582–597
46. Adulyasak Y, Cordeau JF, Jans R (2014) Optimization-based adaptive large neighborhood search for the production routing problem. Transp Sci 48(1):20–45
47. Bard JF, Nananukul N (2009) The integrated production inventory distribution routing problem. J Sched 12(3):257–280
48. Christiansen M, Fagerholt K, Ronen D (2004) Ship routing and scheduling: status and perspectives. Transp Sci 38(1):1–18
49. Christiansen M, Fagerholt K, Nygreen B, Ronen D (2013) Ship routing and scheduling in the new millennium. Eur J Oper Res 228:467–483
50. Ronen D (1983) Cargo ships routing and scheduling: survey of models and problems. Eur J Oper Res 12:119–126
51. Ronen D (1993) Ships scheduling: the last decade. Eur J Oper Res 71(3):325–333
52. Golden BL, Assad AA (1988) Vehicle routing: methods and studies. North Holland, Amsterdam

53. Golden BL, Raghavan S, Wasil EA (eds) (2008) The vehicle routing problem: latest advances and new challenges. Operations research/computer science interfaces series, vol 43. Springer LLC, Boston
54. Toth P, Vigo D (2014) Vehicle routing: problems, methods and applications. MOS-SIAM series on optimization, 2nd edn. SIAM, Philadelphia
55. Assad AA, Golden BL (1995) Arc routing methods and applications. In: Ball MO, Magnanti TL, Momma CL, Nemhauser GL (eds) Network routing, handbooks in operations research and management science, vol 8. Elsevier Science B V, Amsterdam, pp 375–483
56. Ball MO, Magnanti TL, Momma CL, Nemhauser GL (eds) Network routing, handbooks in operations research and management science, vol 8. Elsevier Science B V, Amsterdam
57. Pereira FB, Tavares J (2008) Bio-inspired algorithms for the vehicle routing problem. Studies in computational intelligence, vol 161. Springer, Berlin/Heideberg
58. Zeimpekis VS, Tarantilis CD, Giaglis GM, Minis I (eds) (2007) Dynamic fleet management—concepts, systems, algorithms and case studies. Book series: operations research/computer science interfaces series, vol 38. Springer
59. Archetti C, Speranza MG (2008) The split delivery vehicle routing problem: a survey. In: Golden B, Raghavan S, Wasil E (eds) The vehicle routing problem: latest advances and new challenges. Springer, Boston, pp 103–122
60. Archetti C, Speranza MG, Vigo D (2014) Vehicle routing problems with profits. In: Toth P, Vigo D (eds) Vehicle routing: problems, methods, and applications. MOS-SIAM series on optimization. SIAM, Philadelphia, pp 273–298
61. Berbeglia G, Cordeau JF, Gribkovskaia I, Laporte G (2007) Static pickup and delivery problems: a classification scheme and survey. TOP 15(1):1–31
62. Bodin L, Golden B (1981) Classification in vehicle routing and scheduling. Networks 11:97–108
63. Bodin L, Golden B, Assad A, Ball M (1983) The state of the art in the routing and scheduling of vehicles and crews. Comput Oper Res 10:63–212
64. Braysy O, Gendreau M (2005) Vehicle routing problem with time windows, part I: route construction and local search algorithms. Transp Sci 39(1):104–118
65. Braysy O, Gendreau M (2005) Vehicle routing problem with time windows, Part II: metaheuristics. Transp Sci 39(1):119–139
66. Caceres-Cruz J, Arias P, Guimarans D, Riera D, Juan AA (2015) Rich vehicle routing problem: survey. ACM Comput Surv (CSUR) 47(2):32
67. Eksioglu B, Vural AV, Reisman A (2009) The vehicle routing problem: a taxonomic review. Comput Ind Eng 57(4):1472–1483
68. Gavalas D, Konstantopoulos C, Mastakas K, Pantziou G (2014) A survey on algorithmic approaches for solving tourist trip design problems. J Heuristics 20(3):291–328
69. Gendreau M, Laporte G, Potvin J-Y (1997) Vehicle routing: modern heuristics. In: Aarts EHL, Lenstra JK (eds) Local search in combinatorial optimization. Wiley, Chichester, pp 311–336
70. Gendreau M, Laporte G, Potvin J-Y (2002) Metaheuristics for the capacitated VRP. In: Toth P, Vigo D (eds) The vehicle routing problem. Monographs on discrete mathematics and applications. SIAM, Philadelphia, pp 129–154
71. Golden BL, Wasil EA, Kelly JP, Chao IM (1998) The impact of metaheuristics on solving the vehicle routing problem: algorithms, problem sets, and computational results. In: Crainic TG, Laporte G (eds) Fleet management and logistics. Kluwer Academic Publishers, Boston, pp 33–56
72. Jozefowiez N, Semet F, Talbi EG (2008) Multi-objective vehicle routing problems. Eur J Oper Res 189(2):293–309
73. Marinakis Y, Migdalas Á (2002) Heuristic solutions of vehicle routing problems in supply chain management. In: Pardalos PM, Migdalas A, Burkard R (eds) Combinatorial and global optimization. World Scientific Publishing Co, Singapore, pp 205–236
74. Pillac V, Gendreau M, Gueret C, Medaglia AL (2013) A review of dynamic vehicle routing problems. Eur J Oper Res 225:1–11

75. Tarantilis CD (2005) Solving the vehicle routing problem with adaptive memory programming methodology. Comput Oper Res 32(9):2309–2327
76. Vidal T, Crainic TG, Gendreau M, Prins C (2013) Heuristics for multi-attribute vehicle routing problems: a survey and synthesis. Eur J Oper Res 231(1):1–21
77. Kennedy J, Eberhart R (1995) Particle swarm optimization. In: Proceedings of 1995 IEEE international conference on neural networks, vol 4, pp 1942–1948
78. Banks A, Vincent J, Anyakoha C (2007) A review of particle swarm optimization. Part I: background and development. Nat Comput 6(4):467–484
79. Banks A, Vincent J, Anyakoha C (2008) A review of particle swarm optimization. Part II: hybridisation, combinatorial, multicriteria and constrained optimization, and indicative applications. Nat Comput 7:109–124
80. Clerc M (2006) Particle swarm optimization. ISTE, London
81. Poli R, Kennedy J, Blackwell T (2007) Particle swarm optimization. An overview. Swarm Intell 1:33–57
82. Engelbrecht AP (2007) Computational intelligence: an introduction, 2nd edn. Wiley, Chichester
83. Shi Y, Eberhart R (1998) A modified particle swarm optimizer. In: Proceedings of 1998 IEEE world congress on computational intelligence, pp 69–73
84. Clerc M, Kennedy J (2002) The particle swarm: explosion, stability and convergence in a multi-dimensional complex space. IEEE Trans Evol Comput 6:58–73
85. Kennedy J (1997) The particle swarm: social adaptation of knowledge. In: Proceedings of the IEEE international conference on evolutionary computation, Indianapolis, 13–16 Apr 1997, pp 303–308
86. Kennedy J, Eberhart R (1997) A discrete binary version of the particle swarm algorithm. In: Proceedings of 1997 IEEE international conference on systems, man, and cybernetics, vol 5, pp 4104–4108
87. Solomon MM (1987) Algorithms for the vehicle routing and scheduling problems with time window constraints. Oper Res 35(2):254–265
88. Bianchi L, Birattari M, Manfrin M, Mastrolilli M, Paquete L, Rossi-Doria O, Schiavinotto T (2006) Hybrid metaheuristics for the vehicle routing problem with stochastic demands. J Math Model Algorithm 5(1):91–110
89. Renaud J, Laporte G, Boctor FF (1996) A Tabu search heuristic for the multidepot vehicle routing problem. Comput Oper Res 23(3):229–235
90. Zhang T, Chaovalitwongse WA, Zhang Y (2012) Scatter search for the stochastic travel-time vehicle routing problem with simultaneous pick-ups and deliveries. Comput Oper Res 39:2277–2290
91. Francis PM, Smilowitz KR, Tzur M (2008) The period vehicle routing problem and its extensions. In: Golden B et al (eds) The vehicle routing problem: latest advances and new challenges. Springer LLC, Boston, pp 73–102
92. Daskin M (1995) Network and discrete location. Models, algorithms and applications. Wiley, New York
93. Min H, Jayaraman V, Srivastava R (1998) Combined location-routing problems: a synthesis and future research directions. Eur J Oper Res 108:1–15
94. Vansteenwegen P, Souffriau W, Berghe GV, Oudheusden DV (2009) A guided local search metaheuristic for the team orienteering problem. Eur J Oper Res 196:118–127
95. Sevkli AZ, Sevilgen FE (2012) Discrete particle swarm optimization for the team orienteering problem. Turk J Electr Eng Comput Sci 20(2):231–239
96. De A, Mamanduru VKR, Gunasekaran A, Subramanian N, Tiwari MK (2016) Composite particle algorithm for sustainable integrated dynamic ship routing and scheduling optimization. Comput Ind Eng 96:201–215
97. Tlili T, Faiz S, Krichen S (2014) A hybrid metaheuristic for the distance-constrained capacitated vehicle routing problem. In: Procedia – social and behavioral sciences, 2nd world conference on business, economics and management-WCBEM 2013, vol 109, pp 779–783

98. Zhan Z-H, Zhang J (2009) Discrete particle swarm optimization for multiple destination routing problems. In: Giacobini M et al (eds) EvoWorkshops 2009. LNCS, vol 5484. Springer, Berlin/Heidelberg, pp 117–122
99. Di-Ming A, Zhe Z, Rui Z, Feng P (2011) Research of pareto-based multi-objective optimization for multi-vehicle assignment problem based on MOPSO. In: Tan Y et al (eds) ICSI 2011, Part II. LNCS, vol 6729. Springer, Berlin/Heidelberg, pp 10–16
100. Ting C-J, Wu K-C, Chou H (2014) Particle swarm optimization algorithm for the berth allocation problem. Expert Syst Appl 41:1543–1550
101. Bozorgi-Amiri A, Jabalameli MS, Alinaghian M, Heydari M (2012) A modified particle swarm optimization for disaster relief logistics under uncertain environment. Int J Adv Manuf Technol 60(1–4):357–371
102. Kuo RJ, Zulvia FE, Suryadi K (2012) Hybrid particle swarm optimization with genetic algorithm for solving capacitated vehicle routing problem with fuzzy demand – a case study on garbage collection system. Appl Math Comput 219:2574–2588
103. Varthanan PA, Murugan N, Kumar GM (2012) A simulation based heuristic discrete particle swarm algorithm for generating integrated production-distribution plan. Appl Soft Comput 12:3034–3050
104. Kechagiopoulos PN, Beligiannis GN (2014) Solving the urban transit routing problem using a particle swarm optimization based algorithm. Appl Soft Comput 21:654–676
105. Kumar RS, Kondapaneni K, Dixit V, Goswami A, Thakur LS, Tiwari MK (2016) Multi-objective modeling of production and pollution routing problem with time window: a self-learning particle swarm optimization approach. Comput Ind Eng 99:29–40
106. Olivera AC, García-Nieto JM, Alba E (2014) Reducing vehicle emissions and fuel consumption in the city by using particle swarm optimization. Applied intelligence. Springer Science+Business Media, New York. https://doi.org/10.1007/s10489-014-0604-3
107. Zong X, Xiong S, Fang Z (2014) A conflict-congestion model for Pedestrian-vehicle mixed evacuation based on discrete particle swarm optimization algorithm. Comput Oper Res 44:1–12
108. Norouzi N, Sadegh-Amalnick M, Tavakkoli-Moghaddam R (2016) Modified particle swarm optimization in a time-dependent vehicle routing problem: minimizing fuel consumption. Opt Lett 11:121. https://doi.org/10.1007/s11590-015-0996-y
109. Gan X, Wang Y, Yu Y, Niu B (2013) An emergency vehicle scheduling problem with time utility based on particle swarm optimization. In: Huang D-S et al (eds) ICIC 2013. LNAI, vol 7996. Springer, Berlin/Heidelberg, pp 614–623
110. Zhang L, Li Y, Fei T, Chen X, Ting G (2014) Research of emergency logistics routing optimization based on particle swarm optimization. In: Patnaik S, Li X (eds) Proceedings of international conference on computer science and information technology. Advances in intelligent systems and computing, vol 255. Springer, New Delhi, pp 415–421. https://doi.org/10.1007/978-81-322-1759-6_48
111. Yusoff M, Ariffin J, Mohamed A (2015) DPSO based on a min-max approach and clamping strategy for the evacuation vehicle assignment problem. Neurocomputing 148:30–38
112. Gan X, Liu LJ, Chen JS, Niu B (2016) Comprehensive learning PSO for solving environment heterogeneous fixed fleet VRP with time windows. In: Tan Y et al (eds) ICSI 2016, Part II. LNCS, vol 9713. Springer, Cham, pp 424–432
113. Chen S-K, Wu G-H, Ti Y-W, Wang R-Z, Fang W-P, Lu C-J (2014) Hierarchical particle swarm optimization algorithm of IPSVR problem. In: Pan J-S et al (eds) Genetic and evolutionary computing. Advances in intelligent systems and computing, vol 238. Springer International Publishing, Cham, pp 231–238. https://doi.org/10.1007/978-3-319-01796-9_24
114. Peng Y (2009) Hybrid particle swarm optimization for vehicle routing problem with reverse logistics. In: Proceedings of 2009 international conference on intelligent human-machine systems and cybernetics, Hangzhou, 26–27 Aug 2009, pp 462–465
115. Ai TJ, Kachitvichyanukul V (2009) Particle swarm optimization and two solution representations for solving the capacitated vehicle routing problem. Comput Ind Eng 56:380–387

116. Chen A-L, Yang G-K, Wu Z-M (2006) Hybrid discrete particle swarm optimization algorithm for capacitated vehicle routing problem. J Zhejiang Univ Sci A 7(4):607–614
117. Kim B-I, Son S-J (2012) A probability matrix based particle swarm optimization for the capacitated vehicle routing problem. J Intell Manuf 23(4):1119–1126
118. Marinakis Y, Marinaki M (2010) A hybrid genetic—particle swarm optimization algorithm for the vehicle routing problem. Expert Syst Appl 37:1446–1455
119. Marinakis Y, Marinaki M, Dounias G (2010) A hybrid particle swarm optimization algorithm for the vehicle routing problem. Eng Appl Artif Intell 23:463–472
120. Qi C (2011) Application of improved discrete particle swarm optimization in logistics distribution routing problem. Proc Eng Adv Control Eng Inf Sci 15:3673–3677
121. Tavakoli MM, Sami A (2013) Particle swarm optimization in solving capacitated vehicle routing problem. Buletin Teknik Elektro dan Informatika (Bull Electr Eng Inf) 2(4):252–257
122. Venkatesan SR, Logendran D, Chandramohan D (2011) Optimization of capacitated vehicle routing problem using PSO. Int J Eng Sci Technol (IJEST) 3(10):7469–7469
123. Jian L (2009) Solving capacitated vehicle routing problems via genetic particle swarm optimization. In: Proceedings of 2009 third international symposium on intelligent information technology application, Nanchang, 21–22 Nov 2009, pp 528–531
124. Kou M, Ye C, Chen Z (2011) A bee evolutionary particle swarm optimization algorithm for vehicle routing problem. In: Proceedings of 2011 6th IEEE joint international information technology and artificial intelligence conference (ITAIC), Chongqing, vol 2, 20–22 Aug 2011, pp 398–401
125. Li Y, Li D, Wang D (2012) Quantum-behaved particle swarm optimization algorithm based on border mutation and chaos for vehicle routing problem. In: Tan Y, Shi Y, Ji Z (eds) ICSI 2012, Part I. LNCS, vol 7331. Springer, Berlin/Heidelberg, pp 63–73
126. Marinakis Y, Marinaki M (2013) Combinatorial neighborhood topology particle swarm optimization algorithm for the vehicle routing problem. In: Middendorf M, Blum C (eds) EvoCOP 2013. LNCS, vol 7832, pp 133–144
127. Shen H, Zhu Y, Liu T, Jin L (2009) Particle swarm optimization in solving vehicle routing problem. In: Proceedings 2009 second international conference on intelligent computation technology and automation, Changsha, 10–11 Oct 2009, pp 287–291
128. Tang H (2011) Vehicle route optimization in logistics distribution based on extension-coded particle swarm algorithm. In: Proceedings of 2011 international conference on computer science and network technology, Harbin, 24–26 Dec 2011, pp 2350–2354
129. Wang B, Wang K, Bao F, Zhang L, Shen L (2012) Mixed climbing particle swarm algorithm in the VRP. In: Proceedings of 2012 second international conference on business computing and global informatization, pp 554–557
130. Ai TJ, Kachitvichyanukul V (2009) A particle swarm optimization for the vehicle routing problem with simultaneous pickup and delivery. Comput Oper Res 36:1693–1702
131. Chen MC, Hsiao YH, Reddy RH, Tiwari MK (2016) The self-learning particle swarm optimization approach for routing pickup and delivery of multiple products with material handling in multiple cross-docks. Transp Res E 91:208–226
132. Goksal FP, Karaoglan I, Altiparmak F (2013) A hybrid discrete particle swarm optimization for vehicle routing problem with simultaneous pickup and delivery. Comput Ind Eng 65: 39–53
133. Goksal FP, Altiparmak F, Karaoglan I (2010) A hybrid particle swarm optimization for vehicle routing problem with simultaneous pickup and delivery. In: Proceedings of 2010 40th international conference on Computers and Industrial Engineering (CIE), Awaji, 25–28 July 2010, pp 1–6
134. Tang C, Wang T (2011) An improved particle swarm optimization for the vehicle routing problem with simultaneous deliveries and pick-ups. In: Shen G, Huang X (eds) CSIE 2011, Part II. CCIS, vol 153. Springer, Berlin/Heidelberg, pp 294–300
135. Wei R, Zhang T, Tang H (2010) An improved particle swarm optimization algorithm for vehicle routing problem with simultaneous pickup and delivery. In: Zhu R et al (eds) ICICA 2010, Part I. CCIS, vol 105. Springer, Berlin/Heidelberg, pp 430–436

136. Zhang N-Z, Sun G-H, Wu Y-H, Geng F-H (2009) A modified particle swarm optimization for the vehicle routing problem with simultaneous pickup and delivery. In: Proceedings of the 7th Asian control conference, Hong Kong, 27–29 Aug 2009, pp 1679–1684
137. Liu J, Kachitvichyanukul V (2013) A new solution representation for solving location routing problem via particle swarm optimization. In: Lin Y-K et al (eds) Proceedings of the institute of industrial engineers asian conference. Springer Science+Business Media, Singapore. https://doi.org/10.1007/978-981-4451-98-7_12
138. Marinakis Y, Marinaki M (2008) A particle swarm optimization algorithm with path relinking for the location routing problem. J Math Model Algorithms 7(1):59–78
139. Bashiri M, Fallahzade E (2012) A particle swarm optimization algorithm for multi-depot capacitated location-routing problem with inventory decisions in supply chain network design. In: CIE42 proceedings, Cape Town, 16–18 July 2012. CIE and SAIIE, pp 25-1–25-9
140. Peng Y, Chen Z-X (2009) Two-phase particle swarm optimization for multi-depot location-routing problem. In: Proceedings of 2009 international conference on new trends in information and service science, Beijing, 30 June 2009–2 July 2009, pp 240–245
141. Belmecheri F, Prins C, Yalaoui F, Amodeo L (2013) Particle swarm optimization algorithm for a vehicle routing problem with heterogeneous fleet, mixed backhauls, and time windows. J Intell Manuf 24(4):775–789
142. Yao B, Yu B, Hu P, Gao J (2016) An improved particle swarm optimization for carton heterogeneous vehicle routing problem with a collection depot. Ann Oper Res 242:303–320
143. Ai TJ, Kachitvichyanukul V (2009) A particle swarm optimisation for vehicle routing problem with time windows. Int J Oper Res 6(4):519–537
144. Gong Y-J, Zhang J, Liu O, Huang R-Z, Chung HS-H, Shi Y-H (2012) Optimizing the vehicle routing problem with time windows: a discrete particle swarm optimization approach. IEEE Trans Syst Man Cybern C Appl Rev 42(2):254–267
145. Brito J, Exposito A, Moreno-Pérez JA (2015) Bi-objective discrete PSO for service-oriented VRPTW (Chapter 29). In: Greiner D et al (eds) Advances in evolutionary and deterministic methods for design, optimization and control in engineering and sciences. Computational methods in applied sciences, vol 36. Springer International Publishing, Cham, pp 445–460. https://doi.org/10.1007/978-3-319-11541-2_29
146. Castro JP, Landa-Silva D, Moreno Perez JA (2009) Exploring feasible and infeasible regions in the vehicle routing problem with time windows using a multi-objective particle swarm optimization approach (Chapter 9). In: Krasnogor N et al (eds) Nature inspired cooperative strategies for optimization. SCI, vol 236. Springer, Berlin/Heidelberg, pp 103–114
147. Gan X, Kuang J, Niu B (2014) Particle swarm optimizations for multi-type vehicle routing problem with time windows. In: Huang D-S et al (eds) ICIC 2014. LNAI, vol 8589. Springer International Publishing, Cham, pp 808–815
148. Jiang W, Zhang Y, Xie J (2009) A particle swarm optimization algorithm with crossover for vehicle routing problem with time windows. In: IEEE symposium on computational intelligence in scheduling (CI-Sched '09), Nashville, 30 Mar 2009–2 Apr 2009, pp 103–106
149. Liu X, Jiang W, Xie J (2009) Vehicle routing problem with time windows: a hybrid particle swarm optimization approach. In: Proceedings of 2009 fifth international conference on natural computation, Tianjin, 14–16 Aug 2009, pp 502–506
150. Marinakis Y, Marinaki M, Migdalas A (2014) An adaptive particle swarm optimization algorithm for the vehicle routing problem with time windows. In: Proceedings of LOT 2014, logistics, optimization and transportation conference, Molde, 1–2 Nov 2014
151. Muñoz-Zavala A, Hernández-Aguirre A, Villa-Diharce E (2009) Particle evolutionary swarm multi-objective optimization for vehicle routing problem with time windows. In: Coello Coello CA et al (eds) Swarm intelligence for multi-objective problems in data mining. SCI, vol 242. Springer, Berlin/Heidelberg, pp 233–257
152. Wang Z, Li J, Fan J, Fan C (2010) Research on improved hybrid particle swarm optimization for vehicle routing problem with time windows. In: Proceedings of 2010 international conference on artificial intelligence and computational intelligence, Sanya, 23–24 Oct 2010, pp 179–183

153. Zhu Q, Qian L, Li Y, Zhu S (2006) An improved particle swarm optimization algorithm for vehicle routing problem with time windows. In: Proceedings of 2006 IEEE congress on evolutionary computation, Vancouver, 16–21 July 2006, pp 1386–1390
154. Chen J-Q, Li W-L, Murata T (2013) Particle swarm optimization for vehicle routing problem with uncertain demand. In: Proceedings of 2013 4th IEEE international conference on software engineering and service science (ICSESS), Beijing, 23–25 May 2013, pp 857–860
155. Yan F, Xu M, Yu H (2015) The vehicle routing optimization with uncertain demands and traveling time. In: Xu J et al (eds) Proceedings of the ninth international conference on management science and engineering management. Advances in intelligent systems and computing, vol 362, pp 267–274
156. Muthuswamy S, Lam SS (2011) Discrete particle swarm optimization for the team orienteering problem. Memetic Comput 3:287–303
157. Dallard H, Lam SS, Kulturel-Konak S (2007) Solving the orienteering problem using attractive and repulsive particle swarm optimization. In: Proceedings of IEEE international conference on Information Reuse and Integration (IRI 2007), Las Vegas, 13–15 Aug 2007, pp 2–17
158. Dang D-C, Guibadj RN, Moukrim A (2011) A PSO-based memetic algorithm for the team orienteering problem. In: Di Chio C et al (eds) EvoApplications 2011, Part II. LNCS, vol 6625. Springer, Berlin/Heidelberg, pp 471–480
159. Sevkli Z, Sevilgen FE (2010) Discrete particle swarm optimization for the orienteering problem. In: Proceedings of 2010 IEEE Congress on Evolutionary Computation (CEC), Barcelona, 18–23 July 2010, pp 1–8
160. Khouadjia MR, Sarasola B, Alba E, Jourdan L, Talbi E-G (2012) A comparative study between dynamic adapted PSO and VNS for the vehicle routing problem with dynamic requests. Appl Soft Comput 12:1426–1439
161. Okulewicz M, Mańdziuk J (2017) The impact of particular components of the PSO-based algorithm solving the dynamic vehicle routing problem. Appl Soft Comput 58:586–604
162. Khouadjia MR, Alba E, Jourdan L, Talbi E-G (2010) Multi-swarm optimization for dynamic combinatorial problems: a case study on dynamic vehicle routing problem. In: Dorigo M et al (eds) ANTS 2010. LNCS, vol 6234. Springer, Berlin/Heidelberg, pp 227–238
163. Khouadjia MR, Jourdan L, Talbi E-G (2010) Adaptive particle swarm for solving the dynamic vehicle routing problem. In: Proceedings of 2010 IEEE/ACS international conference on computer systems and applications (AICCSA), Hammamet, 16–19 May 2010, pp 1–8
164. Okulewicz M, Mańdziuk J (2013) Application of particle swarm optimization algorithm to dynamic vehicle routing problem. In: Rutkowski L et al (eds) ICAISC 2013, Part II. LNAI, vol 7895. Springer, Berlin/Heidelberg, pp 547–558
165. MirHassani SA, Abolghasemi N (2011) A particle swarm optimization algorithm for open vehicle routing problem. Expert Syst Appl 38:11547–11551
166. Norouzi N, Tavakkoli-Moghaddam R, Ghazanfari M, Alinaghian M, Salamatbakhsh A (2012) A new multi-objective competitive open vehicle routing problem solved by particle swarm optimization. Netw Spat Econ 12(4):609–633
167. Marinakis Y, Marinaki M (2012) A hybrid particle swarm optimization algorithm for the open vehicle routing problem. In: Dorigo M et al (eds) ANTS 2012. LNCS, vol 7461. Springer, Berlin/Heidelberg, pp 180–187
168. Perwira Redi AAN, Maghfiroh MFN, Yu VF (2013) Discrete particle swarm optimization with path-relinking for solving the open vehicle routing problem with time windows. In: Lin Y-K et al (eds) Proceedings of the institute of industrial engineers Asian conference 2013. Springer Science+Business Media, Singapore, pp 853–859. https://doi.org/10.1007/978-981-4451-98-7_102
169. Wang W, Wu B, Zhao Y, Feng D (2006) Particle swarm optimization for open vehicle routing problem. In: Huang D-S, Li K, Irwin GW (eds) ICIC 2006. LNAI, vol 4114. Springer, Berlin/Heidelberg, pp 999–1007

170. Zhen T, Zhu Y, Zhang Q (2009) A particle swarm optimization algorithm for the open vehicle routing problem. In: Proceedings of 2009 international conference on environmental science and information application technology, Wuhan, 4–5 July 2009, pp 560–563
171. Marinakis Y, Iordanidou G, Marinaki M (2013) Particle swarm optimization for the vehicle routing problem with stochastic demands. Appl Soft Comput 13(4):1693–1704
172. Marinakis Y, Marinaki M (2013) Combinatorial expanding neighborhood topology particle swarm optimization for the vehicle routing problem with stochastic demands. In: Proceedings of GECCO 2013, genetic and evolutionary computation conference, Amsterdam, 6–10 July 2013, pp 49–56
173. Peng Y, Zhu H-Y (2008) Research on vehicle routing problem with stochastic demand and PSO-DP algorithm with inver-over operator. Syst Eng Theory Pract (SETP) 28(10):76–81
174. Zhao Y, Li C, Zhang J-L, Ren X, Ren W (2011) Research on vehicle routing problem with stochastic demand based on multi-objective method. In: Huang D-S et al (eds) ICIC 2011. LNCS, vol 6838. Springer, Berlin/Heidelberg, pp. 153–161
175. Norouzi N, Sadegh-Amalnick M, Mehdi A (2015) Evaluating of the particle swarm optimization in a periodic vehicle routing problem. Measurement 62:162–169
176. Rabbani M, Manavizadeh N, Shamekhi A (2013) A particle swarm optimization method for periodic vehicle routing problem with pickup and delivery in transportation. Adv Railw Eng Int J 1(1):51–60
177. Tavakkoli Moghaddam R, Mohmmad Zohrevand A, Rafiee K (2012) Solving a new mathematical model for a periodic vehicle routing problem by particle swarm optimization. Transp Res J 1:77–87
178. Xu J, Yan F, Li S (2011) Vehicle routing optimization with soft time windows in a fuzzy random environment. Transp Res E 47:1075–1091
179. Peng Y, Chen J (2010) Vehicle routing problem with fuzzy demands and the particle swarm optimization solution. In: Proceedings of 2010 international conference on management and service science (MASS), Wuhan, 24–26 Aug 2010, pp 1–4
180. Shao Z-J, Gao S-P, Wang S-S (2009) A hybrid particle swarm optimization algorithm for vehicle routing problem with stochastic travel time. In: Cao B-Y, Zhang C-Y, Li T-F (eds) Fuzzy information and engineering. ASC, vol 54. Springer, Berlin/Heidelberg, pp 566–574
181. Kachitvichyanukul V, Sombuntham P, Kunnapapdeelert S (2015) Two solution representations for solving multi-depot vehicle routing problem with multiple pickup and delivery requests via PSO. Comput Ind Eng 89:125–136
182. Sombuntham P, Kachitvichayanukul V (2010) A particle swarm optimization algorithm for multi-depot vehicle routing problem with pickup and delivery requests. In: Proceedings of the international multiconference of engineers and computer scientists (IMECS 2010), vol III, Hong Kong, 17–19 Mar 2010. ISBN: 978-988-18210-5-8
183. Wang T-J, Wu K-J (2012) Adaptive particle swarm optimization based on population entropy for MDVRPTW. In: Proceedings of 2012 2nd international conference on computer science and network technology, Changchun, 29–31 Dec 2012, pp 753–756
184. Wang S, Wang L, Yuan H, Ge M, Niu B, Pang W, Liu Y (2009) Study on multi-depots vehicle scheduling problem and its two-phase particle swarm optimization. In: Huang D-S et al (eds) ICIC 2009. LNAI, vol 5755. Springer, Berlin/Heidelberg, pp 748–756
185. Zhang W, Ye J (2010) An improved particle swarm optimization for the multi-depot vehicle routing problem. In: Proceedings of 2010 international conference on E-business and E-government, Guangzhou, 7–9 May 2010, pp 3188–3191
186. Vahdani B, Tavakkoli-Moghaddam R, Zandieh M, Razmi J (2012) Vehicle routing scheduling using an enhanced hybrid optimization approach. J Intell Manuf 23(3):759–774
187. Yusoff M, Ariffin J, Mohamed A (2012) DPSO based on random particle priority value and decomposition procedure as a searching strategy for the evacuation vehicle routing problem. In: Huang T et al (eds) ICONIP 2012, Part III. LNCS, vol 7665. Springer, Berlin/Heidelberg, pp 678–685

188. Marinakis Y (2015) An improved particle swarm optimization algorithm for the capacitated location routing problem and for the location routing problem with stochastic demands. Appl Soft Comput 37:680–701
189. Liu SC, Lu MC, Chung CH (2016) A hybrid heuristic method for the periodic inventory routing problem. Int J Adv Manuf Technol 85:2345–2352
190. Hu W, Liang H, Peng C, Du B, Hu Q (2013) A hybrid chaos-particle swarm optimization algorithm for the vehicle routing problem with time window. Entropy 15:1247–1270. https://doi.org/10.3390/e15041247
191. Moghaddam BF, Ruiz R, Sadjadi SJ (2012) Vehicle routing problem with uncertain demands: an advanced particle swarm algorithm. Comput Ind Eng 62:306–317
192. Yang SY, Wang M, Jiao LC (2004) A quantum particle swarm optimization. In: Proceedings of the 2004 IEEE congress on evolutionary computation, vol 1, pp 320–324
193. Kirkpatrick S (1984) Optimization by simulated annealing—quantitative studies. J Stat Phys 34:975–986
194. Marinakis Y, Migdalas A, Pardalos PM (2005) Expanding neighborhood GRASP for the traveling salesman problem. Comput Optim Appl 32:231–257
195. Glover F, Laguna M, Marti R (2003) Scatter search and path relinking: advances and applications. In: Glover F, Kochenberger GA (eds) Handbook of metaheuristics. Kluwer Academic Publishers, Boston, pp 1–36
196. Lichtblau D (2002) Discrete optimization using mathematica. In: Callaos N, Ebisuzaki T, Starr B, Abe JM, Lichtblau D (eds) World multi-conference on systemics, cybernetics and informatics (SCI 2002), vol 16. International Institute of Informatics and Systemics, pp 169–174
197. Hansen P, Mladenovic N (2001) Variable neighborhood search: principles and applications. Eur J Oper Res 130:449–467
198. Marinakis Y, Migdalas A, Pardalos PM (2009) Multiple phase neighborhood search GRASP based on lagrangean relaxation and random backtracking Lin-Kernighan for the traveling salesman problem. J Comb Optim 17:134–156
199. Prins C (2004) A simple and effective evolutionary algorithm for the vehicle routing problem. Comput Oper Res 31:1985–2002
200. Kanthavel K, Prasad P (2011) Optimization of capacitated vehicle routing problem by nested particle swarm optimization. Am J Appl Sci 8(2):107–112
201. Hu F, Wu F (2010) Diploid hybrid particle swarm optimization with differential evolution for open vehicle routing problem. In: Proceedings of the 8th world congress on intelligent control and automation, Jinan, 6–9 July 2010

Scheduling Heuristics

52

Rubén Ruiz

Contents

Introduction 1598
The Importance of Scheduling 1599
Scheduling Is Hard 1600
The Relevance of Heuristic Scheduling 1600
Notation and Classification of Scheduling Problems 1601
Types of Scheduling Problems and Notation 1602
Scheduling Constraints 1603
Scheduling Objectives 1605
Basic Scheduling Heuristics 1607
Some Simple Dispatching Rules 1607
An Application to the Parallel Machine Problem 1608
Elaborated Dispatching Rules 1609
Advanced Heuristic Methods 1610
The NEH Heuristic for the Permutation Flow Shop and Makespan Criterion 1610
A Brief Introduction to the Shifting Bottleneck Heuristic for the Job Shop Problem 1612
Metaheuristics 1613
Main Concepts of Metaheuristics in Scheduling 1613
Iterated Greedy as an Example of a Simple- and High-Performing Method 1615
Conclusions 1617
Cross-References 1617
References 1617

R. Ruiz (✉)
Universitat Politécnica de València, València, Spain

AMAZON, Amazon.com, Inc., Seattle, WA, USA
e-mail: rruiz@eio.upv.es

© Springer Nature Switzerland AG 2025

R. Martí et al. (eds.), *Handbook of Heuristics*,
https://doi.org/10.1007/978-3-032-00385-0_44

Abstract

The scheduling of operations over resources is a relevant theoretical and practical problem with applications in many fields and disciplines, including the manufacturing industry. Scheduling problems are as varied as the reality they model. Additionally, some scheduling settings are among the hardest combinatorial problems there are. This is a perfect scenario for heuristic methods where high-quality robust solutions can be obtained in a short amount of time. This chapter concentrates on heuristics for production scheduling problems and summarizes the main results that range from simple rules to advanced metaheuristics. The importance of proper scheduling in practice is first highlighted, along with its difficulty and relevance. A summary of the scheduling notation is also given. Basic scheduling techniques, dispatching rules, combined rules, advanced heuristics, and an introduction to metaheuristics are also summarized in the chapter. While necessarily brief and incomplete, this chapter serves as an introductory point to those interested readers seeking to delve in the vast and rich world of scheduling heuristics. Some pointers to fruitful future research avenues are also provided. A large list of journal articles and monographs are provided as a reference for additional details and study.

Keywords

Scheduling · Dispatching rules · Heuristics · Local search · Metaheuristics

Introduction

In a wide sense, the term scheduling deals with the allocation of tasks to resources over a finite time horizon. These resources are usually limited and therefore, tasks have to share or, more precisely, compete for them. Depending on the type of scheduling problem, there might be interrelated decisions of allocation of tasks to resources, sequencing of the tasks assigned to each resource, assignment of start and finish times to each task, and so on. The objective is to find a schedule that optimizes a given criterion or criteria. As can easily be seen, this is a very broad definition. As detailed in [1, Chap. 1] and in [2, Chap. 1], scheduling is a decision-making process that is applied to a very wide range of situations in theory and in practice. As a result, resources can encompass from machines in a factory to tools or personnel in a plant or project. Resources might also model operation rooms in a hospital or cores in a computing cluster, etc. Similarly, tasks range from manufacturing operations to be performed in a factory to surgical interventions in an operation room. The applications are virtually limitless. During this chapter the focus is on one of the most studied applications of scheduling, which is manufacturing scheduling [3]. In manufacturing or shop scheduling, tasks commonly model client's orders or jobs, and resources are the machines or production lines needed to manufacture the products. While this field inside scheduling is arguably the most studied, other

monographs deal with other areas of scheduling in detail. Such is the case of Pinedo [2] for scheduling in services, Błażewicz et al. [4] for computer scheduling, Grosche [5] for scheduling problems in the airline industry, Randolph [6] for health-care scheduling, or Briskorn [7] for scheduling in sports. This short list is just an extract of the rich and varied literature on scheduling problems.

A salient characteristic of the manufacturing scheduling scheduling field is the sheer variety of real settings found in practice. It is easy to imagine that the scheduling problem found in a factory that produces cars is quite different from a factory that manufactures light bulbs to name just two random examples. Furthermore, the underlying scheduling problems are extraordinarily difficult from a mathematical perspective as most problems are of a combinatorial nature. This variety and the difficulty of the scheduling settings have resulted in a research field that is fragmented and overly specialized. More often than not, exact mathematical procedures are devised for specific scheduling settings where rigor and mathematical elegance are set as a goal in itself, possibly neglecting the general application or practical relevance of the proposed procedures. This has resulted in the well-known "research gap" between scheduling theory and practice, as recognized initially by Graves [8], Ledbetter and Cox [9], and Ford et al. [10]. All these studies pointed out the scant application of scheduling theory into practice. McKay et al. [11] and Olhager and Rapp [12] reached the same conclusions. Dudek et al. [13], MacCarthy and Liu [14], McKay and Wiers [15], McKay et al. [16], or McKay and Wiers [17] directly addressed, at different levels, this research gap. Different state-of-the-art reviews and studies about scheduling research like those of Reisman et al. [18], Ribas et al. [19], and Ruiz and Vázquez-Rodríguez [20] concluded that real scheduling problems are seldom studied. A more complete overview of these issues is detailed in Chap. 15 of [3].

The Importance of Scheduling

Without doubt, the aforementioned research gap is not to be attributed to a lack of interest in scheduling problems from industry. Good scheduling practice is of paramount importance in the manufacturing industry. Good schedules increase machine utilization, decrease manufacturing costs, and improve the fulfillment of delivery dates to customers ([1, 21, 22], among many others). Modern industries face a number of challenges that make scheduling as important as ever, to name just a few: (1) increasing product variety and/or configurations to reach a wider market, (2) reducing time to market, (3) shortening delivery dates to costumers, (4) high pressure in costs from emerging markets, (5) reducing production lot sizes, and (6) increasing frequency of orders from clients but of smaller size. All of the above are indicative of a transition from a mass manufacturing setting to a mass customization situation in a globalized economy. The optimized scheduling of hundreds, if not thousands, of production orders on a busy production shop floor is a must in order to meet these challenges.

Furthermore, good schedules help to identify production bottlenecks, streamline and stabilize production, minimize raw material shortages, and allow dealing with priorities in production orders. The conclusion is clear: nowadays factories must employ advanced scheduling tools in order to stay competitive amidst today's terribly competitive market.

Scheduling Is Hard

Sadly, scheduling models are remarkably hard to solve. In fact, most scheduling problems belong to the $\mathcal{NP}$-Hard class of computational problems [23]. This is also true for most combinatorial optimization problems in many other fields inside operations research like location, routing, planning, etc. However, it can be argued that most real scheduling problems are particularly hard and in practice they seem to be more difficult to solve than problems in other areas. Take for example the well-known capacitated vehicle routing problem (CVRP) for which extensive literature exists. According to Laporte [24] and to the most powerful exact methods recently proposed for the problem [25], instances of up to 135 clients and 7 vehicles can be solved to optimality. In comparison, for the job shop scheduling problem (to be defined later), a wait of more than 20 years was needed to have small instances of 10 jobs and 10 machines solved to optimality by Brucker et al. [26], and no significant advances have been made for the optimal solution of instances with just 20 jobs. By all accounts, 20 jobs are way below the normal number of jobs being processed at any time in a real industry where hundreds of tasks are being processed on a daily basis.

Furthermore, real problems are much more complex than regular job shops and have many side constraints and generalizations. There is little hope of solving real problems to optimality. At this moment it is important to raise the question of whether optimal solutions are required in practice or not. Scheduling models are just that models of real problems that are, in the best case, good approximations of the modeled reality. Real data is often put into estimates that are fed to the models. In these circumstances it makes little sense to solve an approximated model with estimated data to optimality. While it is important to study theoretical problems and constructs and to propose exact approaches for such settings (as the insight gained from those studies is invaluable), real problems often require a different approach. This approach comes in the form of heuristics. Scheduling heuristics provide quick and fast approximated solutions to even the most intricate scheduling models. Moreover, and as it will be shown, such solutions are very close to optimality, at least in the cases where the optimality gap can be calculated or estimated.

The Relevance of Heuristic Scheduling

So far it has been stated that scheduling problems are as varied as the manufacturing reality they model and among the hardest combinatorial problems there are. This

has resulted in a research gap between the theory and practice of scheduling. Furthermore, there are many practical issues that arise when deploying scheduling systems in manufacturing firms. The study of such scheduling systems is not as rich as the theoretical works. Some references are Framinan and Ruiz, Framinan and Ruiz [27, 28], and Framinan et al. [3, Chaps. 13 and 14 and references therein]. To cope with the variety of real settings, general and robust heuristics are preferable to ad hoc exact approaches that are often tailored to specific problems and objectives. In particular, metaheuristics are powerful heuristic templates that, with little instantiation, are capable of solving a wide array of differing problems to almost optimality in reduced CPU times. Still, the development of such methods for practical applications is challenging and requires finesse as well as a deep knowledge of the main constructs and results. Such is the focus of this chapter. Examples from the most noteworthy basic heuristic procedures to the most advanced metaheuristics will be highlighted. Our natural inclination is toward general simple methods that include little to no problem-specific knowledge at all as they are the most suited to practical applications. More general approaches to heuristic and metaheuristic algorithms for scheduling problems are given by Morton and Pentico [29], Hoos and Stützle [30, Chap. 9], and Framinan et al. [3].

The rest of this chapter is organized as follows: section "Notation and Classification of Scheduling Problems" formalizes scheduling problems and provides a succinct explanation of the common notation used in the literature. Section "Basic Scheduling Heuristics" presents dispatching rules and simple heuristics for some classical and basic scheduling problems. Two advanced scheduling heuristics are presented in section "Advanced Heuristic Methods." A brief introduction to metaheuristics is provided in section "Metaheuristics," along with the description of a successful and simple methodology. Finally, section "Conclusions" concludes this chapter and some pointers for future research are given.

Notation and Classification of Scheduling Problems

Most production scheduling models define a set N of n jobs with consecutive indexes, i.e., $N = \{1, 2, \ldots, n\}$. The resources are modeled in the form of a set M of m machines indexed as $M = \{1, 2, \ldots, m\}$. All sets are assumed to be independent from each other. Subindexes j and k are used to refer to jobs and i and l to machines. Some basic information associated with jobs and machines is as follows:

- A processing route. For a given job j, the route is an ordered list of machines that have to be visited in order for the job to be completed at the factory. If a job has to visit three machines (e.g., 2, 1, and 6), it is said that it has three tasks. Usually, and following this small example, the task in machine 1 cannot start until the previous task of the same job in machine 2 has finished.
- Processing times. These are commonly denoted as p_j or p_{ij}. This is the amount of time that a given job j needs at machine i. This time is fixed and usually it cannot

be interrupted. Machine i is busy while processing job j and cannot be occupied during this time by any other job.
- Due dates and weights. These are denoted by d_j and w_j, respectively. The due date models the time at which job j must be completed in the factory to be delivered to the customer. The weight captures the relative importance of the job. This data will be used later for the objectives to optimize.

Of course, a wealth of additional data might be needed for the jobs and machines, depending on the real problem to solve. However, and for the sake of brevity, the previous data suffices as a bare minimum. Furthermore, in the course of this chapter, it is assumed that all data is known and deterministic. Most figures are usually represented by nonnegative integers for simplicity. The literature on stochastic scheduling is vast. The interested reader is referred to two recent monographs and the references therein for more details [31, 32]. Additionally, the part on stochastic models in Pinedo [1] is an excellent starting point.

Types of Scheduling Problems and Notation

The literature on production scheduling is mostly partitioned according to how the machines are laid out on the production floor and also on the characteristics of the processing routes of jobs. With this in mind, the main scheduling problems relate to the following scenarios:

- Single machine problems. This is the simplest setting. There is a known number n of jobs, and each one has to be processed on a single machine, following a sequence. Most of the time only a vector p_j with the processing times of the jobs on this single machine is needed.
- Parallel machine problems. Picture the previous case but with m machines disposed in parallel. Jobs still have to be processed by one single machine, but in this case it has to be one out of the m available. Therefore, an assignment problem precedes that of the sequencing problem among all jobs assigned to each machine. The m parallel machines might be identical, uniform, or unrelated. In the case of identical parallel machines, the processing times for the jobs are independent of the machines. In the unrelated case, the processing time of a job j might change depending on which machine it is assigned to. Therefore, a processing time matrix p_{ij} is needed. The uniform parallel machine case is an intermediate situation where machines are faster or slower for all jobs by a factor.
- Flow shops. Here the m machines are disposed in series and all jobs must visit all machines in the same order. This means that the processing route is identical for all jobs and is usually assumed to be $1 \ldots, m$. Jobs are broken down into m tasks with known processing times p_{ij}. As stated before, any task for a given job is unable to start before the preceding task at the previous machine has finished. A flow shop is a very common production machine on production or assembly lines.

- Job shops. Here the main difference from flow shops is that each job has a potentially different processing route. Therefore not all jobs must visit all machines and nor do they necessarily have to be in the same order.
- Open shops. This is a much less common setting in which each job has to visit a given number of the m machines, but the route is open for the scheduling method to determine.
- Hybrid shop environments. Most real problems are actually a combination of the previous shop settings and parallel machines. Instead of having m machines disposed in series, there are m stages where at each stage i, there might be a single machine or m_i parallel machines, be these identical, uniform, or unrelated. This is, by far, the most complex and, at the same time, most general production setting.

Note that the previous short list is a massive simplification. Other production settings include assembly shops, manufacturing cells, flexible manufacturing systems (FMS), batch machines, and so on. In any case, the previous classification is the most employed one. Following this classification, the most common notation was proposed by Rinnooy Kan [33] and is a triplet $\alpha/\beta/\gamma$. The first field α covers the previous scheduling types with the values 1, P, Q, R, F, J, and O for single machine, identical parallel machines, uniform parallel machines, unrelated parallel machines, flow shop, job shop, and open shop problems, respectively. For more advanced layouts, like hybrid shops, there are extensions of this notation, like the one provided by Vignier et al. [34] and reviewed in Ruiz and Vázquez-Rodríguez [20]. The second field β is concerned with the constraints present in the problem, and the last field γ captures the objective function to optimize. These two last fields are succinctly summarized in the following sections.

Scheduling Constraints

As mentioned, there are as many scheduling constraints as product varieties and types of factories. Following Framinan et al. [3], the different scheduling constraints can be classified if they affect the processing of the jobs, operations, transportation, storage, or others:

- Process constraints. In this broad category, it might be mentioned all situations that affect the flow of the operations in the factory. Here one can find job/task precedence constraints ($\beta = \{chains, intree, outtree, tree\}$ depending on whether the precedence constraints form chains, convergent trees, divergent trees, or general trees, respectively). Here job/tasks might be subject to precedences, i.e., some jobs might not start before others have finished. Another frequent processing constraint is the setup times. Setup times model all nonproductive operations that have to be performed at machines after finishing jobs and before starting new ones. Cleaning, fixing, and reconfiguration are common setup operations. Setup times might be anticipatory/non-anticipatory and/or sequence

independent/dependent. As such, S_{ijk} models the amount of time that it is needed to setup machine i after processing job j and before processing job k. The literature on scheduling with all sorts and types of setup times is incredibly large. In parallel and hybrid layouts, jobs might have machine eligibility constraints, indicating that not all available machines can process certain jobs. Machines might also be subject to unavailability periods and/or breakdowns. Jobs might also have more general processing routes requiring stage/machine skipping or recirculation (revisiting one or more stages/machines more than once). This last circumstance is indicated as $\beta = recrc$. There are many more process constraints than the ones mentioned here. Consult Pinedo [1] or Framinan et al. [3, Chap. 4] for a more detailed account.

- Operations constraints. On many occasions the operation or task of a job on a given machine is more complicated. For example, tasks might be interrupted arbitrarily to be repeated or resumed later, possibly after having processed a task from a different job in between. Tasks from the same job might also be split to be processed by different parallel machines. Tasks/jobs might be subject to release dates, i.e., $\beta = r_j$ indicates that job j might not start processing at its first machine before time r_j due to, for example, unavailability of raw materials. Apart from release dates, there might be due dates, deadlines and due windows, etc. Another frequent characteristic is that jobs might not be allowed to wait in between machines ($\beta = nwt$). This is common in the processing of prepared foods in order to avoid breaking the cold chain. At the same time, there might be forced minimum waits for a job in between stages (cooling down, drying off), maximum waiting times, waiting windows, or even overlaps between consecutive tasks of a job. Similarly, processing times might be subject to all types of additional situations, like learning, deterioration or position dependency, etc.
- Transportation constraints. It is very common in real industries to have means of transporting unfinished jobs between machines. The list here is endless as one might have from simple manually operated pallet jacks for moving products from machine to machine to forklift trucks or even complex automated guided autonomous vehicles. Conveyor belts, handling robots, and other equipment are very common too. Depending on the scenario, transportation might be modeled as a simple fixed transportation time in between machines to very complex scheduling-routing problems when the transportation is an essential part of the process and transportation equipment is limited and expensive.
- Storage constraints. Similar to transportation constraints, production in course has to be stored somewhere. Again, and depending on the real situation, storage might be of no concern as space might be ample and/or the type of product being produced small and easy to store. However, in other situations the handling of unfinished products might be so challenging as to be a central constraint in the factory. Imagine, for example, the production of commercial airplanes where moving an airplane frame from one processing station to another is far from a trivial matter.

- Other constraints. Apart from machines one might have additional resources, like special tools, fixtures, or even the personnel operating machines. This personnel might have different degrees of skill. In some factories, machines might be able to perform more than one operation in parallel. Manufacturing firms might have more than one production center. Processing and/or setup times might be controllable depending on the resources (i.e., personnel) dedicated to them. There might be timing/shift constraints, safety concerns, etc. As indicated, the list of potential situations in practice is very long.

Scheduling Objectives

Finally, field γ is concerned with the scheduling objectives to optimize. In order to understand these, there is a need for some additional notation:

- C_j is the completion time of job j in the shop. It models the time at which the last task is completed at the last machine.
- L_j is a measure of the lateness of job j with respect to its due date d_j. $L_j = C_j - d_j$.
- T_j is a measure of the tardiness of job j where $T_j = \max\{L_j, 0\}$.

This is just a small extract of the many possibilities. Similarly, earliness can be defined when jobs finish before their due dates. From these definitions the two main groups of objective functions are those related with completion times and those with due dates:

- Completion time based. The most common one is the minimization of the maximum completion time among the jobs, usually referred to as makespan or $C_{\max} = \max\{C_1, C_2, \ldots, C_n\}$. Makespan is mainly related to maximizing machine utilization. The second most studied objective is the minimization of the total flow time, defined as $\sum C = \sum_{j=1}^{n} C_j$.
- Due date based. The most common is the minimization of the total tardiness of the jobs $\sum T = \sum_{j=1}^{n} T_j$. However, the weighted version or $\sum wT = \sum_{j=1}^{n} w_j T_j$ is more realistic as being tardy for some important jobs should be considered over being tardy for irrelevant orders.

There are tens, if not hundreds, of potential objectives. A much more detailed list is given in Chap. 5 of Framinan et al. [3]. Furthermore, the reality is often multiobjective. The extension of the γ notation for multiple objectives in scheduling is detailed in T'Kindt and Billaut [35].

Let us now give a simple example of the $F/prmu/C_{\max}$ problem or the permutation flow shop problem with makespan criterion. The example with the total flow time objective or $F/prmu/\sum C$ will be used. There are five jobs and four machines. The processing times are shown in Table 1.

Table 1 Processing times (p_{ij}) of a flow shop problem with five jobs and four machines

Machine (i)	Job (j)				
	1	2	3	4	5
1	29	37	95	85	65
2	30	62	59	11	55
3	27	21	70	62	67
4	2	6	82	80	57

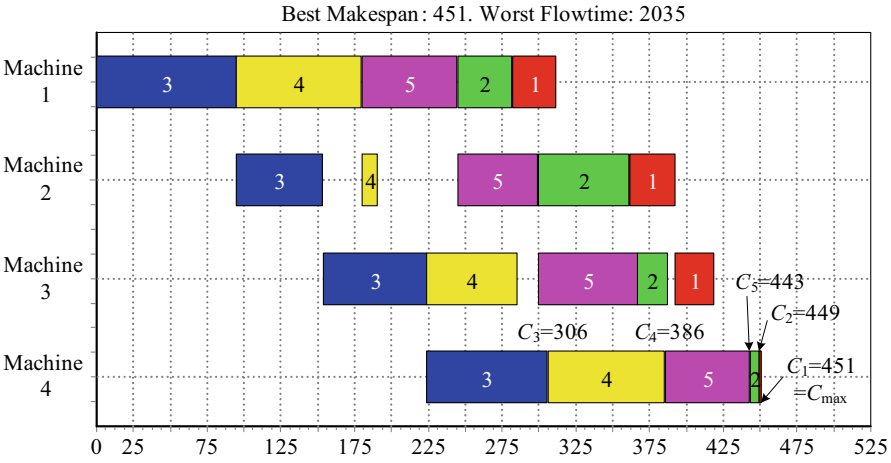


Fig. 1 Gantt chart for the permutation flow shop example with the best $C_{\max}$ which corresponds also with the worst possible $\sum C$ value

The permutation flow shop is a problem of determining a sequence for the jobs so there are $5! = 120$ possible schedules in this small example. If one calculates all of them, the best makespan value we obtain is $C_{\max*} = 451$ with the permutation or job sequence $\pi^*_{\max} = \{3, 4, 5, 2, 1\}$. The worst makespan value is $C^w_{\max} = 522$ (15.74% worse) resulting from the permutation $\pi^w_{\max} = \{1, 2, 4, 5, 3\}$. What is interesting about this example is that the best flow time value is $\sum C^* = 1{,}464$, resulting from the permutation $\pi^*_{\sum C} = \{1, 2, 4, 5, 3\}$. As one can see, $\pi^w_{\max} = \pi^*_{\sum C}$, i.e., the worst makespan solution is equal to the best flow time solution. Furthermore, the contrary is also true. For this example, the worst flow time value is $\sum C^w = 2{,}035$ (38.93% worse than the best), resulting from the permutation $\pi^w_{\sum C} = \{3, 4, 5, 2, 1\}$. As one can see, $\pi^*_{\max} = \pi^w_{\sum C}$. The conclusion is that the best possible makespan value can result in the worst possible flow time and vice versa. The two permutations in this example are depicted in Figs. 1 and 2, respectively. The completion times of all jobs in the last machine have been indicated for further examination. While this is a cherry-picked example, it is easy to find examples such as this for almost any number of jobs and machines.

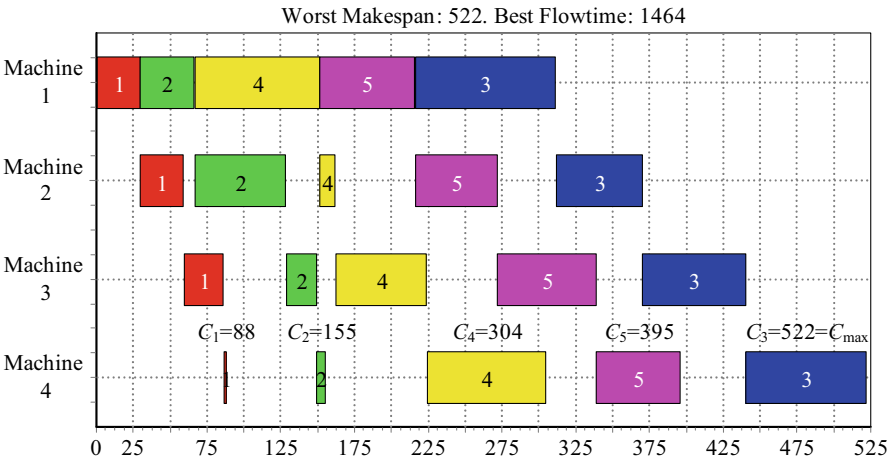


Fig. 2 Gantt chart for the permutation flow shop example with the best $\sum C$ which corresponds also with the worst possible C_{max} value

Basic Scheduling Heuristics

In the early twentieth century, modern production plants started using scheduling techniques to increase productivity. Human schedulers planned production manually without any aid other than pen and paper. The situation today has not changed much for most industries where basically spreadsheets are used to program production, but simple rules are used to obtain effective schedules. These simple rules are often referred to as list scheduling rules, dispatching rules, or priority rules. Some of them will be briefly explained. More advanced dispatching rules will be explained later.

Some Simple Dispatching Rules

Dispatching rules are usually forward heuristics where new tasks or jobs are sequenced as they arrive or as their processing becomes feasible (i.e., all preceding tasks have been already scheduled). A decision is made over a set of eligible tasks, referred to as $\wp$. Some simple dispatching rules are the following:

- First Come First Served (FCFS): Here the job or task that entered the list of eligible tasks first is scheduled, i.e., the eligible tasks are sequenced in a first in first out (FIFO) order.
- Shortest Processing Time (SPT) First: The pending eligible with the shortest processing time goes first, i.e., given m machines, the task k that satisfies the following expression is chosen: $p_{ik} = \min_{j=1}^{|\wp|} \{p_{ij}\}$. While simplistic, SPT

results in the optimal solution for the single machine total flow time problem ($1\|\sum C$). The contrary rule to SPT is the longest processing time (LPT) first rule.

- Earliest Due Date (EDD) First: This is a dispatching rule that considers due dates. Basically the pending task with the earliest due date is scheduled first. EDD is optimal for the problems $1\|T_{\max}$ and $1\|L_{\max}$.

Note that these dispatching rules are extremely fast; all of them have a computational complexity of $\mathcal{O}(n \log n)$, n being the number of jobs or tasks to be scheduled. They have many other advantages like their simplicity and ease of application. Furthermore, they are easy to understand which is something valued by human schedulers. There are literally hundreds of dispatching rules and they have been comprehensively studied. It is suffice to cite just some of the exiting reviews in Panwalkar and Iskander, Blackstone et al., Haupt, and Jayamohan and Rajendran [36–39].

An Application to the Parallel Machine Problem

In general, dispatching rules, albeit general, fast, and simple, are overly myopic and produce subpar results, especially when compared to more elaborated constructs. However, in some cases they provide strong solutions. The following example applies to the identical parallel machine problem with makespan criterion or $P//C_{\max}$. In this problem only the assignment of jobs to machines is relevant, as the sequence at each machine has no effect on the maximum completion time. There are n jobs to be scheduled in m identical parallel machines with the only input data being a vector with the processing times p_j. The $P//C_{\max}$ is $\mathcal{NP}$-Hard even for two machines [40].

Real results from an exam scheduling problem of a big company in the USA (Pearson VUE) are presented. This company has 5000 test centers in 175 countries in which all types of certifications, admissions, and exams are taken. The problem solved is the scheduling of different exams every week of a given year at each center in the USA so as to minimize the number of hours that each center is open to the public. Each center has a number of seats in which exams can be taken. Therefore, each center is modelized as an identical parallel machine problem in which the number of seats is the number of machines. The objective is to minimize the makespan as this translates directly into a minimization of the opening hours. The different exams are job types, and registrations for a given exam by users at a given week are the jobs to be scheduled at the given center in that week. All 219 centers in the USA for the 52 weeks of 2012 were solved. This resulted in a total of 11,261 different identical parallel machine problems with 1,244,063 jobs in total. The largest center had 308 machines (seats) available, and the biggest number of jobs to solve in a given week and center was 638.

The algorithm of choice is the aforementioned LPT, which for the parallel machine case is simple. Jobs are ordered from longest to shortest processing times, and the first m are assigned to the m parallel machines. Then, each job in the ordered list is assigned to the first freed machine. This LPT is known to have a worst-case performance guarantee of $\frac{4}{3} - \frac{1}{3m}$ [41]. This means that for large m values, the solution given by LPT should be at most around 33% worse than the optimal makespan. Now the results of the LPT with the solutions obtained by a state-of-the-art solver (CPLEX 12.5) are compared. The experiments were run on a computing cluster with 240 cores in total (XEON E5420 processors running at 2.5 GHz.) and 480 GBytes of RAM memory. Around 93% of problems could be solved to optimality and the average gap obtained was 0.57%. However, more than 80 h of CPU time were needed in the cluster (time limit for each problem solved was set to 300 s, which was reached in 851 cases). Comparatively speaking, the LPT resulted in a gap in respect to the MIP optimal solution or lower bound of just 0.91%. The total CPU time of the application of the LPT for all 11,261 problems was just 17.16 s on a single computer and most of this time was the processing of the input data and generation of the solution files.

The conclusion is clear. For some problems, including real ones, the application of simple dispatching rules can result in extraordinary performance in negligible CPU times.

Elaborated Dispatching Rules

Many authors have explored more intricate procedures. A clear example is the apparent tardiness cost (ATC) composite dispatching rule proposed by Vepsalainen and Morton [42]. This rule is excellently explained in Pinedo [1]. The ATC rule was presented for job shop problems with total weighted tardiness criterion but has been successfully applied to different problems as well. The rule is applied to all jobs and an index is calculated. The job with the highest index is scheduled and then the rule is calculated again for the remaining $n - 1$ jobs. In total ATC is computed $n \cdot (n + 1)/2–1$ times resulting in a computational complexity of $\mathcal{O}\left(n^2\right)$. The index is calculated for each job j, and each time a machine is free as $I_j(t) = \frac{w_j}{p_j} \cdot e^{\left(\frac{\max\left\{d_j - p_j - t, 0\right\}}{K\overline{p}}\right)}$ where $\overline{p}$ is the average of the processing times and K is a parameter that needs to be fixed. ATC was expanded for the consideration of setup times by Lee et al. [43] resulting in the apparent tardiness cost with setups or ATCS. Here the index is further complicated: $I_k\,(t,\, j) = \frac{w_k}{p_k} \cdot e^{\left(\frac{\max\{d_k - p_k - t, 0\}}{K_1\overline{p}}\right)} \cdot e^{\left(-\frac{s_{jk}}{K_2\overline{s}}\right)}$. As one can easily see, the calculations quickly become cumbersome; the rule is also harder to understand and to apply which basically defeats many of the advantages of dispatching rules. The benefits are better performance and objective function values.

Advanced Heuristic Methods

Advanced heuristics usually include problem-specific knowledge exploiting theoretical properties of scheduling problems. Alternatively, some methods aim at deriving better schedules starting from previous schedules. These latter approaches have been known as improvement heuristics. The distinction between basic or constructive heuristics and improvement heuristics is not so clear on many occasions. In any case, most of the time constructive heuristics have a natural stopping criterion, i.e., the method stops when a complete schedule or sequence has been built. Improvement heuristics don't have such a direct stopping criteria.

Most improvement heuristics and metaheuristics (to be succinctly reviewed in the next section) are strongly based on local search techniques. Local search is a heuristic exploration technique with a strong methodological background ([30], among many others). Local search requires at least the following ingredients: (1) A definition of what is a neighboring solution of a sequence; (2) a way to explore the neighbors of a solution; (3) an acceptance criterion, i.e., when a neighboring solution must replace the current one; and (4) when and how to stop the local search. Usually the main and most important issue is how to represent a solution and how to define a neighborhood from that representation. In most scheduling problems, solutions are indirectly represented as permutations of integers. The most employed neighborhoods are swap and insert where two jobs exchange their positions in the sequence in the case of swap and a job is extracted from its position and inserted into another position in the case of insert. In the following section, a very successful heuristic for the flow shop problem is detailed. It is based on the exploration of the insertion neighborhood in a constructive phase. Later another effective heuristic is briefly described but this time for the job shop problem and based on a different methodology the decomposition of the scheduling problem into smaller subproblems.

The NEH Heuristic for the Permutation Flow Shop and Makespan Criterion

The NEH heuristic of Nawaz et al. [44] is one of the most successful advanced heuristics. It was originally proposed for the permutation flow shop with makespan criterion ($F/prmu/C_{\max}$). It is powerful and capable, with hundreds of applications and references to many scheduling problems. Reviews like those of Turner and Booth [45], Taillard [46] or Ruiz and Maroto [47] recognize its capacity. The NEH exploits the fact that jobs with large total processing times must be sequenced as early as possible. The NEH consists of three steps:

1. The total processing times of the jobs in all machines are calculated: $P_j = \sum_{i=1}^{m} p_{ij}, \quad j = (1, \ldots, n)$.
2. Jobs are sorted in descending order of P_j, so that the first job is the one that has the highest time requirements. These sorted jobs are stored in a list ℓ.

3. The first two jobs of the list ℓ, referred to as $\ell_{(1)}$ and $\ell_{(2)}$, are extracted. The two possible partial sequences made up from these two jobs are constructed and their respective makespan values calculated. In other words, the two following partial sequences are tested: $\{\ell_{(1)}, \ell_{(2)}\}$ and $\{\ell_{(2)}, \ell_{(1)}\}$. From the two sequences, the one with the minimum partial $C_{\max}$ value is retained for the next iterations.
4. The main loop of the NEH starts here. At each iteration, the job at position k of the list ℓ, i.e., $\ell_{(k)}$, $k = (3, \ldots, n)$ is extracted. This job $\ell_{(k)}$ is inserted into all possible solutions of the partial incumbent sequence, and the sequence resulting in the lowest partial $C_{\max}$ value is retained for the next iteration. For example, let us suppose that the best partial sequence obtained up to a given point is $\pi_p = \{\ell_{(1)}, \ell_{(3)}, \ell_{(2)}\}$ and that $k = 4$; therefore, job $\ell_{(4)}$ has to be inserted in the four possible positions of the partial sequence: $\pi^1_{p+\ell_{(4)}} = \{\ell_{(4)}, \ell_{(1)}, \ell_{(3)}, \ell_{(2)}\}$, $\pi^2_{p+\ell_{(4)}} = \{\ell_{(1)}, \ell_{(4)}, \ell_{(3)}, \ell_{(2)}\}$, $\pi^3_{p+\ell_{(4)}} = \{\ell_{(1)}, \ell_{(3)}, \ell_{(4)}, \ell_{(2)}\}$, and $\pi^4_{p+\ell_{(4)}} = \{\ell_{(1)}, \ell_{(3)}, \ell_{(2)}, \ell_{(4)}\}$. For the next step ($k = 5$), the sequence with the lowest partial $C_{\max}$ value from the four previous possible sequences is retained.

As one can see, NEH is basically based upon insertions of jobs into a partial sequence that is constructed step after step. Step 1 has a complexity of $\mathcal{O}(nm)$. The second step is just an ordering and takes $\mathcal{O}(n \log n)$. The third step requires just two calculations of two small sequences and can be disregarded. Most CPU time is needed by step 4. In step 4 there is a loop of $n - 2$ iterations where at each step k, k insertions of job $\ell_{(k)}$ are carried out into a partial sequence that contains $k - 1$ jobs, and for each insertion one needs to calculate the $C_{\max}$ value of k jobs (including the inserted one). In total, $\frac{n(n+1)}{2} - 3$ insertions are carried out, being n of these complete sequences at the last iteration. Therefore, the computational complexity of the NEH is $\mathcal{O}\left(n^3 m\right)$. A closer look shows that when inserting the k-th job in position, let us say, j, all $C_{i,h}$, $h = \{j - 1, j - 2, \ldots, 1\}$ were already calculated in the previous insertion and there is no need to recalculate. Taillard [46] exploited this idea by which all insertions in a given step can be calculated in $O(nm)$ effectively reducing the complexity of the NEH to $O(n^2 m)$. This faster variant is referred to as NEHT or NEH with Taillard's accelerations.

The NEH was computationally tested against 25 other heuristics, including other more recent and complex heuristics in Ruiz and Maroto [47]. The experiments demonstrated that the NEH is superior to all other tested methods and at the same time much faster. This was supported by statistical analyses. In the results, NEH showed an average percentage deviation from the best solutions known in the 120 instances of Taillard [48] of just 3.33%. Although this is an average result, NEH rarely goes beyond 7%. Furthermore, NEH is actively used as a seed sequence for metaheuristic techniques.

The NEHT was also evaluated by Rad et al. [49] with surprising results. For example, in the 10 instances of the largest size (500×20) from the benchmark of Taillard [48], NEHT managed to give an average percentage deviation from the best known solutions (these solutions have been proved to be "almost optimal" with very small gaps between the highest known lower bound and lowest known upper

bound or best solution known) of just 2.24%. Furthermore, this 2.24% deviation from a very close to optimum solution was achieved in an average of just 0.0773 s. Extensions of the NEH presented in the same paper of Rad et al. [49] resulted in much better performance, reaching an average percentage deviation from best known solutions of just 1.4% in CPU times under 2 s.

With these results, it is easy to see why the NEH is such an important and widely studied heuristic. Studies about the NEH have been numerous over the years. The papers of [49–57] and more recently Fernandez-Viagas and Framinan [58] are some examples. Furthermore, the NEH does not rely on specific problem knowledge, and it is relatively easy to extend it and apply it to different scheduling problems. Such extensions are numerous in the scheduling literature, mainly over flow shop problems but also in hybrid settings and other problems.

A Brief Introduction to the Shifting Bottleneck Heuristic for the Job Shop Problem

The shifting bottleneck heuristic or SBH for short was initially proposed by Adams et al. [59] for the job shop problem and makespan minimization ($F//C_{\max}$). Many improvements on the initial SBH for the same problem were presented over the following years. Additionally, SBH has been extended by many authors to other objectives and problems. It is a very interesting method because it mimics the reality of a production floor. In real factories one production stage or machine is usually the bottleneck of the whole plant, i.e., productivity or the measured objective is hard to improve unless the scheduling or capacity of this stage is improved. SBH tries to optimize the schedule of the bottleneck machine first as the contribution of this machine to the final makespan is large. Since in a job shop there are m machines and jobs are made of many tasks with precedence relationships among them, scheduling only the bottleneck machine is not straightforward. The procedure can be simply described as follows:

1. Determine the bottleneck machine.
2. Schedule all previous tasks of the n jobs on the other machines to obtain the release dates (r_j) of the jobs on the bottleneck machine.
3. Schedule all subsequent tasks of the n jobs on the other machines to obtain the due dates (d_j) of the jobs on the bottleneck machine.
4. Schedule the bottleneck machine as a single machine problem with release dates and due dates, minimizing the maximum lateness ($1/r_j, d_j/\max L_j$).
5. Repeat all previous steps for the remaining $m - 1$ machines until all machines have a sequence.

Note that the single machine problem is solved m times. The $1/r_j, d_j/\max L_j$ is in itself a hard problem. However, efficient methods do exist for its solution. In particular, the original SBH heuristic of [59] uses the algorithm of [60]. Additional details about the SBH procedure, including examples, are given in [1] and in [3].

The performance of the SBH is outstanding, especially if one considers that the job shop problem is incredibly hard to solve. Back at the time the authors were able to solve problems of up to 50 jobs and 10 machines to optimality in some cases. As a result, the literature on the SBH is incredibly large. More details about this heuristic and many other related bottleneck decomposition methods are given in the book of [61].

Metaheuristics

While heuristics are usually devised for specific problems with a given objective function, they are, in principle, more generic. This distinction is far from clear as many heuristics are indeed general as well, like the aforementioned NEH. In any case, metaheuristics can be seen as general algorithmic templates that exist without the particular problem or application. Additionally, metaheuristics are commonly more demanding and CPU intensive than regular heuristics. The desired effect of this additional computation is a better solution quality. Other common characteristics of metaheuristics are mechanisms to escape local optimality and to reduce the effect of the myopic nature of heuristic approaches by diversification mechanisms. The field of metaheuristics is interdisciplinary and terribly prolific with literally thousands of papers being published every year in many different fields, not to mention the large number of monographs dedicated to them. Scheduling being an important problem, the application of metaheuristics to scheduling is frequent. A good starting point are the books of [30, 62].

Metaheuristics are roughly grouped into families or classes. Each family stems from a basic metaheuristic template that, with relatively few modifications, can be applied to a specific scheduling problem, and reasonable results might be obtained. To obtain very high-quality solutions, more specific operators and local search schemes are usually needed. This also results in slower methods. The main categories are simulated annealing, tabu search, genetic algorithms, ant colony optimization, scatter search, variable neighborhood search, GRASP and iterated local search, etc. In the following section, the main constructs inside most metaheuristic algorithms are described, to be followed by a brief description of a successful metaheuristic that is at the same time simple: iterated greedy.

Main Concepts of Metaheuristics in Scheduling

Most metaheuristic classes require at least the instantiation of some of the following elements:

- Representation of the solution. At the lowest level, solutions contain variables and these might be continuous, discrete, binary, etc. A representation is just how the variables are represented and arranged. In scheduling problems that require permutations and an array or list of integers, each one representing a job is

an indirect representation of the final solution. Such a solution is "scheduled" (i.e., starting and finish times for each job and/or task are calculated) in order to calculate the desired objective function. For parallel machine problems where only the assignment is relevant (e.g., the $P//C_{\max}$), each job might have an integer indicating the machine to which it is assigned to. The representation is very important for the final performance of the metaheuristic.

- Initial solution or population generation. Most metaheuristics require an initial solution to start the process. Population-based metaheuristics require a set of solutions. The easiest procedure is to randomly generate these initial solutions. Of course, this results in initial solutions of mediocre quality. The most commonly employed alternative is to use high-performance heuristics to generate initial schedules.
- Intensification and diversification mechanisms. Initial solutions are worked on through local search and/or other procedures like selection and crossover in genetic algorithms. The objective is to find new and better solutions. There are many procedures and techniques but all of these fall more or less into what is referred to as intensification. To avoid stagnation in the search process, most metaheuristics include diversification mechanisms by which existing solutions are disrupted or altered (or new solutions are generated) in order to allow the intensification procedures to work over different parts of the solution and search space.
- Acceptance criterion. New solutions generated through the intensification and diversification processes have to be examined in order to decide if they replace current incumbent solutions or not. The simplest scenario is when new solutions are accepted if and only if they improve on the current or best objective function value found so far during the search. This criterion leads to premature convergence and stagnation, and most metaheuristic templates consider more elaborated alternatives where worse solutions might be accepted, often temporarily.
- Termination criterion. Metaheuristics iterate continuously unless a user-given termination criterion is met. For most metaheuristics, optimality is not guaranteed. Furthermore, most applications of metaheuristics do not have an indication of when an optimal solution has been found in order to stop the search. Therefore, common termination criteria include a number of iterations or a consecutive number of iterations without improvements (or small improvements), elapsed CPU time, etc.

The previous operators are only a very small list where not even all of them might be needed to construct a simple method. Furthermore, elaborated metaheuristics might have many operators and procedures. Actually, in the last decade, many natural or man-made processes have inspired a real tsunami of metaheuristic methods. As a result there are algorithms that are referred to as particle swarm, imperialist competitive, honey bees, fruit flies, and even more bizarre methods like intelligent water drops or galaxy optimization. Currently, such "novel" methods are

being criticized. The papers of [63, 64] are two examples. In this chapter these criticisms are supported and very successful, but at the same time really simple metaheuristic is described.

Iterated Greedy as an Example of a Simple- and High-Performing Method

The first application of the concepts of iterated greedy (IG) to a scheduling problem was presented by [65]. The authors dealt with the permutation flow shop problem with makespan criterion ($F/prmu/C_{\max}$). Being a metaheuristic, IG starts from an initial heuristic solution. Then three phases are iteratively applied until a termination criterion is met. The first phase is a partial destruction of the current solution. Some jobs are randomly removed from the permutation. The second phase is the reconstruction of the (now) incomplete solution. All removed jobs are inserted, one by one, into all positions in the partial solution using the NEH heuristic. Each job is placed into the position resulting in the best objective function value. An optional step is a local search each time a complete solution is obtained. This local search is carried out in the insertion neighborhood until local optimality. The third last step is an acceptance criterion. A detailed pseudoalgorithm listing is given in Fig. 3.

As one can see, IG carries out intensification with the reconstruction and local search operators and diversification with the destruction. The acceptance criterion is a simplification of simulated annealing. With a constant temperature factor, worse solutions are accepted with a probability that decreases with the quality of the new solution.

The results of the IG method for the permutation flow shop problem are state-of-the-art even today. According to the results of [65], IG was able to reduce the average relative percentage deviation from the best known solutions in the 120 instances of [48] to just 0.44% in little under a minute on average. It was computationally and statistically demonstrated that IG produced better results than competing methods. Later, [66] were able to further reduce this deviation to 0.22% using a simple island model parallel version of the IG.

IG has been applied to different scheduling problems with good results. Ruiz and Stützle [67] extended the previous results with the consideration of sequence-dependent setup times. IG for no wait and blocking flow shops were studied by [68, 69], respectively. IG showed excellent performance in no-idle and mixed no-idle flow shops in [70] and [71]. Non-permutation flow shops were approached by [72]. Distributed scheduling problems have been solved with IG methods by [73, 74]. IG also gives good results in other objectives aside from makespan like tardiness in [75] or total flow time in [76], to name just a few. Multi-objective extensions of IG have proven effective for Pareto flow shops without and with setups in [77] and in [78], respectively. Parallel machine problems were successfully solved with IG methods by [79]. Finally, hybrid flow shops were studied by [80] and complex

procedure *Iterated_Greedy*

$\pi :=$ NEH_Heuristic;

$\pi :=$ Insertion_LocalSearch(π);

$\pi_b := \pi$;

while (termination criterion not satisfied) **do**

$\pi' := \pi$;

for $i := 1$ **to** *destruct* **do** % `Destruction phase`

$\pi' :=$ randomly extract a job of π' and insert it into $\pi_R^{'}$;

for $i := 1$ **to** *destruct* **do** % `Reconstruction phase`

$\pi' :=$ best permutation after inserting $\pi_R(i)$ into all possible positions of π';

$\pi'' :=$ Insertion_LocalSearch(π'); % `Local Search`

if $C_{\max}(\pi'') < C_{\max}(\pi)$ **then** $\pi := \pi''$; % `Acceptance criterion`

if $C_{\max}(\pi) < C_{\max}(\pi_b)$ **then** $\pi_b := \pi$;

elseif $\left(random \leq e^{-\frac{(C_{\max}(\pi'') - C_{\max}(\pi))}{\text{Temperature}}}\right)$ **then** $\pi := \pi''$;

endif

endwhile

return π_b

end

Fig. 3 Iterated greedy (IG) algorithm. (Adapted from [65]). *random* is a random number distributed uniformly in the range [0, 1]. Temperature and *destruct* are the two parameters of IG

realistic hybrid flow shops by [81–83]. There are many other applications of the IG methodology to other scheduling problems and also to several other combinatorial problems as well.

What is noteworthy about all previous studies is that IG algorithms are shown to produce state-of-the-art results that on many occasions are competitive or better than those produced by other much more complex metaheuristics.

There are highly successful applications of different metaheuristics for practically every formulated scheduling problem. Many are listed in [3] and the references therein.

Conclusions

In this chapter the scheduling field has been introduced, pointing out its relevance, difficulty, and hardness. Scheduling problems are varied, hard, and difficult to solve optimally. However, in many settings, obtaining approximated solutions of sufficient quality is within reach if modern heuristic methodologies are employed. The notation of scheduling problems has been introduced. A succinct explanation of simple dispatching rules is provided. An example that shows the lack of correlation between otherwise seemingly correlated objectives has been presented which again highlights the fact that scheduling problems are neither easy nor intuitive. Furthermore, in a briefly summarized application, it has been shown how a very simple dispatching rule can result in almost optimal solutions for real problems of a large size. Even though elaborated and combined dispatching rules are powerful, advanced heuristic methods are many times better. The NEH method for the flow shop and the SBH for the job shop are the clearest examples. High-performance and close-to-optimal solutions are perfectly possible in negligible CPU times. Finally, in an ever-growing field, metaheuristics propose themselves as modern heralds of flexibility, robustness, and finest solution quality. A summary of the iterated greedy method has been presented which apart from the previous qualities manages state-of-the-art results without unnecessary complexities in its design.

As for the future of scheduling heuristics, it is argued for even simpler and more robust approaches, capable of solving hard and varied scheduling problems with little to no instantiation and adaptation. Such frameworks and methodologies could prove invaluable in closing the gap between the theory and practice of scheduling.

Cross-References

▸ Iterated Greedy
▸ Iterated Local Search

Acknowledgments The author is partially supported by the Spanish Ministry of Economy and Competitiveness, under the project "SCHEYARD – Optimization of Scheduling Problems in Container Yards" (No. DPI2015-65895-R) financed by FEDER funds.

Competing Interest Declaration The author(s) has no competing interests to declare that are relevant to the content of this manuscript.

References

1. Pinedo ML (2012) Scheduling: theory, algorithms and systems, 4th edn. Springer, New York
2. Pinedo ML (2009) Planning and scheduling in manufacturing and services, 2nd edn. Springer, New York
3. Framinan JM, Leisten R, Ruiz R (2014) Manufacturing scheduling systems. An integrated view on models, methods and tools. Springer, New York

4. Błażewicz J, Ecker KH, Pesch E, Schmidt G, J W (2001) Scheduling computer and manufacturing processes, 2nd edn. Springer, Berlin
5. Grosche T (2009) Computational intelligence in integrated airline scheduling. Studies in computational intelligence, vol 173. Springer, New York
6. Randolph H (ed) (2012) Handbook of healthcare system scheduling. International series in operations research & management science, vol 168. Springer, New York
7. Briskorn D (2008) Sports leagues scheduling. Models, combinatorial properties, and optimization algorithms. Lecture notes in economics and mathematical systems, vol 603. Springer, Berlin/Heidelberg
8. Graves SC (1981) A review of production scheduling. Oper Res 29(4):646–675
9. Ledbetter WN, Cox JF (1977) Operations research in production management: an investigation of past and present utilisation. Prod Invent Manag 18(3):84–91
10. Ford FN, Bradbard DA, Ledbetter WN, Cox JF (1987) Use of operations research in production management. Prod Invent Manag 28(3):59–62
11. McKay KN, Safayeni FR, Buzacott JA (1988) Job-shop scheduling theory: what is relevant? Interfaces 4(18):84–90
12. Olhager J, Rapp B (1995) Operations research techniques in manufacturing planning and control systems. Int Trans Oper Res 2(1):29–43
13. Dudek RA, Panwalkar SS, Smith ML (1992) The lessons of flowshop scheduling research. Oper Res 40(1):7–13
14. MacCarthy BL, Liu J (1993) Addressing the gap in scheduling research: a review of optimization and heuristic methods in production scheduling. Int J Prod Res 31(1):59–79
15. McKay KN, Wiers VCS (1999) Unifying the theory and practice of production scheduling. J Manuf Syst 18(4):241–255
16. McKay KN, Pinedo ML, Webster S (2002) Practice-focused research issues for scheduling systems. Prod Oper Manag 11(2):249–258
17. McKay KN, Wiers VCS (2006) The human factor in planning and scheduling, chap 2. In: Herrmann JW (ed) Handbook of production scheduling. International series in operations research & management science, vol 89. Springer, New York, pp 23–57
18. Reisman A, Kumar A, Motwani J (1997) Flowshop scheduling/sequencing research: a statistical review of the literature, 1952–1994. IEEE Trans Eng Manag 44(3):316–329
19. Ribas I, Leisten R, Framinan JM (2010) Review and classification of hybrid flow shop scheduling problems from a production system and a solutions procedure perspective. Comput Oper Res 37(8):1439–1454
20. Ruiz R, Vázquez-Rodríguez JA (2010) The hybrid flowshop scheduling problem. Eur J Oper Res 205(1):1–18
21. Hopp WJ, Spearman ML (2011) Factory physics, 3rd edn. Waveland Press Inc., Long Grove
22. McKay KN, Wiers VCS (2004) Practical production control. A survival guide for planners and schedulers. J. Ross Publishing Inc., Boca Raton
23. Brucker P (2007) Scheduling algorithms, 5th edn. Springer, New York
24. Laporte G (2009) Fifty years of vehicle routing. Transp Sci 43(4):408–416
25. Baldacci R, Mingozzi A, Roberti R (2011) New route relaxation and pricing strategies for the vehicle routing problem. Oper Res 59(5):1269–1283
26. Brucker P, Jurisch B, Sievers B (1994) A branch and bound algorithm for the job-shop scheduling problem. Discret Appl Math 49(1–3):107–127
27. Framinan JM, Ruiz R (2010) Architecture of manufacturing scheduling systems: literature review and an integrated proposal. Eur J Oper Res 205(2):237–246
28. Framinan JM, Ruiz R (2012) Guidelines for the deployment and implementation of manufacturing scheduling systems. Int J Prod Res 50(7):1799–1812
29. Morton TE, Pentico DW (1993) Heuristic scheduling sysmtems with applications to production systems and project management. Wiley series in engineering & technology management. Wiley, Hoboken
30. Hoos HH, Stützle T (2005) Stochastic local search: foundations and applications. Morgan Kaufmann, San Francisco

31. Cai X, Wu X, Zhou X (2014) Optimal stochastic scheduling. International series in operations research & management science, vol 207. Springer, New York
32. Sarin SC, Nagarajan B, Liao L (2014) Stochastic scheduling. Expectation-variance analysis of a schedule. Cambridge University Press, Cambridge
33. Rinnooy Kan AHG (1976) Machine scheduling problems: classification, complexity and computations. Martinus Nijhoff, The Hague
34. Vignier A, Billaut JC, Proust C (1999) Les problèmes d'ordonnancement de type flow-shop hybride: État de l'art. RAIRO Recherche opérationnelle 33(2):117–183. (in French)
35. T'Kindt V, Billaut JC (2006) Multicriteria scheduling: theory, models and algorithms, 2nd edn. Springer, New York
36. Blackstone JH Jr, Phillips DT, Hogg GL (1982) A state-of-the-art survey of dispatching rules for manufactuing job shop operations. Int J Prod Res 20(1):27–45
37. Haupt R (1989) A survey or priority rule-based scheduling. OR Spectr 11(1):3–16
38. Jayamohan MS, Rajendran C (2000) New dispatching rules for shop scheduling: a step forward. Int J Prod Res 38(3):563–586
39. Panwalkar SS, Iskander W (1977) A survey of scheduling rules. Oper Res 25(1):45–61
40. Lenstra JK, Rinnooy Kan AHG, Brucker P (1977) Complexity of machine scheduling problems. Ann Discret Math 1:343–362
41. Graham RL (1969) Bounds on multiprocessing timing anomalies. SIAM J Appl Math 17(2):416–429
42. Vepsalainen APJ, Morton TE (1987) Priority rules and lead time estimation for job shop scheduling with weighted tardiness costs. Manag Sci 33(8):1036–1047
43. Lee YH, Bhaskaran K, Pinedo ML (1997) A heuristic to minimize the total weighted tardiness with sequence-dependent setups. IIE Trans 29(1):45–52
44. Nawaz M, Enscore EE Jr, Ham I (1983) A heuristic algorithm for the *m*-machine, *n*-job flow-shop sequencing problem. OMEGA Int J Manag Sci 11(1):91–95
45. Turner S, Booth D (1987) Comparison of heuristics for flow shop sequencing. OMEGA Int J Manag Sci 15(1):75–78
46. Taillard E (1990) Some efficient heuristic methods for the flow shop sequencing problem. Eur J Oper Res 47(1):67–74
47. Ruiz R, Maroto C (2005) A comprehensive review and evaluation of permutation flowshop heuristics. Eur J Oper Res 165(2):479–494
48. Taillard E (1993) Benchmarks for basic scheduling problems. Eur J Oper Res 64(2):278–285
49. Rad SF, Ruiz R, Boroojerdian N (2009) New high performing heuristics for minimizing makespan in permutation flowshops. OMEGA Int J Manag Sci 37(2):331–345
50. Chakraborty UK, Laha D (2007) An improved heuristic for permutation flowshop scheduling. Int J Inf Commun Technol 1(1):89–97
51. Dong XY, Huang HK, Chen P (2006) A more effective constructive algorithm for permutation flowshop problem. In: Intelligent data engineering and automated learning (IDEAL 2006), Lecture notes in computer science, vol 4224. Springer, Berlin/New York (Beijing Jiaotong Univ, Sch Comp & IT, Beijing 100044, Peoples R China), pp 25–32
52. Dong XY, Huang HK, Chen P (2008) An improved NEH-based heuristic for the permutation flowshop problem. Comput Oper Res 35(12):3962–3968
53. Framinan JM, Leisten R, Rajendran C (2003) Different initial sequences for the heuristic of Nawaz, Enscore and Ham to minimize makespan, idletime or flowtime in the static permutation flowshop sequencing problem. Int J Prod Res 41(1):121–148
54. Jin F, Song SJ, Wu C (2007) An improved version of the NEH algorithm and its application to large-scale flow-shop scheduling problems. IIE Trans 39(2):229–234
55. Kalczynski PJ, Kamburowski J (2007) On the NEH heuristic for minimizing the makespan in permutation flow shops. OMEGA Int J Manag Sci 35(1):53–60
56. Kalczynski PJ, Kamburowski J (2008) An improved NEH heuristic to minimize makespan in permutation flow shops. Comput Oper Res 35(9):3001–3008
57. Kalczynski PJ, Kamburowski J (2009) An empirical analysis of the optimality rate of flow shop heuristics. Eur J Oper Res 198(1):93–101

58. Fernandez-Viagas V, Framinan JM (2014) On insertion tie-breaking rules in heuristics for the permutation flowshop scheduling problem. Comput Oper Res 45:60–67
59. Adams J, Balas E, Zawack D (1988) The shifting bottleneck procedure for job shop scheduling. Manag Sci 34(3):391–401
60. Carlier J (1982) The one-machine sequencing problem. Eur J Oper Res 11(1):42–47
61. Ovacik IM, Uzsoy R (1997) Decomposition methods for complex factory scheduling problems. Springer, New York
62. El-Ghazali T (2009) Metaheuristics: from design to implementation. Wiley, New York
63. Sörensen K (2015) Metaheuristics—the metaphor exposed. Int Trans Oper Res 22(1):3–18
64. Weyland D (2010) A rigorous analysis of the harmony search algorithm: how the research community can be misled by a "novel" methodology. Int J Appl Metaheuristic Comput 1(2):50–60
65. Ruiz R, Stützle T (2007) A simple and effective iterated greedy algorithm for the permutation flowshop scheduling problem. Eur J Oper Res 177(3):2033–2049
66. Vallada E, Ruiz R (2009) Cooperative metaheuristics for the permutation flowshop scheduling problem. Eur J Oper Res 193(2):365–376
67. Ruiz R, Stützle T (2008) An iterated greedy heuristic for the sequence dependent setup times flowshop problem with makespan and weighted tardiness objectives. Eur J Oper Res 187(3):1143–1159
68. Pan QK, Wang L, Zhao BH (2008) An improved iterated Greedy algorithm for the no-wait flow shop scheduling problem with makespan criterion. Int J Adv Manuf Technol 38(7–8):778–786
69. Ribas I, Companys R, Tort-Martorell X (2011) An iterated greedy algorithm for the flowshop scheduling problem with blocking. OMEGA Int J Manag Sci 39(3):293–301
70. Ruiz R, Vallada E, Fernández-Martínez C (2009) Scheduling in flowshops with no-idle machines. In: Chakraborty UK (ed) Computational intelligence in flow shop and job shop scheduling. Studies in computational intelligence, vol 230. Springer, Berlin, pp 21–51
71. Pan QK, Ruiz R (2014) An effective iterated Greedy algorithm for the mixed no-idle flowshop scheduling problem. OMEGA Int J Manag Sci 44(1):41–50
72. Ying KC (2008) Solving non-permutation flowshop scheduling problems by an effective iterated Greedy heuristic. Int J Adv Manuf Technol 38(3–4):348–354
73. Fernandez-Viagas V, Framinan JM (2015) A bounded-search iterated greedy algorithm for the distributed permutation flowshop scheduling problem. Int J Prod Res 53(4):1111–1123
74. Lin SW, Ying KC, Huang CY (2013) Minimising makespan in distributed permutation flowshops using a modified iterated greedy algorithm. Int J Prod Res 51(16):5029–5038
75. Framinan JM, Leisten R (2008) Total tardiness minimization in permutation flow shops: a simple approach based on a variable greedy algorithm. Int J Prod Res 46(22):6479–6498
76. Pan QK, Ruiz R (2012) Local search methods for the flowshop scheduling problem with flowtime minimization. Eur J Oper Res 222(1):31–43
77. Minella G, Ruiz R, Ciavotta M (2011) Restarted iterated pareto greedy algorithm for multi-objective flowshop scheduling problems. Comput Oper Res 38(11):1521–1533
78. Ciavotta M, Minella G, Ruiz R (2013) Multi-objective sequence dependent setup times flowshop scheduling: a new algorithm and a comprehensive study. Eur J Oper Res 227(2): 301–313
79. Fanjul-Peyro L, Ruiz R (2010) Iterated greedy local search methods for unrelated parallel machine scheduling. Eur J Oper Res 207(1):55–69
80. Ying KC (2008) An iterated Greedy heuristic for multistage hybrid flowshop scheduling problems with multiprocessor tasks. IEEE Trans Evol Comput 60(6):810–817
81. Urlings T, Ruiz R (2007) Local search in complex scheduling problems. In: Stützle T, Birattari M, Hoos HH (eds) Engineering stochastic local search algorithms. Designing, implementing and analyzing effective heuristics, Lecture notes in computer science, vol 4638. Springer, Brussels, pp 202–206
82. Urlings T, Ruiz R, Sivrikaya-Şerifoğlu F (2010) Genetic algorithms for complex hybrid flexible flow line problems. Int J Met 1(1):30–54
83. Urlings T, Ruiz R, Stützle T (2010) Shifting representation search for hybrid flexible flowline problems. Eur J Oper Res 207(2):1086–1095

Search-Based Software Engineering Problems

53

Javier Yuste, Eduardo G. Pardo, and Francisco Chicano

Contents

Introduction 1622
Software Project Scheduling 1624
Optimal Feature Selection 1627
Test Case Generation 1630
Software Module Clustering 1635
Conclusion 1639
Cross-References 1640
References 1641

Abstract

The development of high-quality software systems is a complex process involving various phases and activities. As software systems grow in size and complexity, these tasks become increasingly challenging for developers, and a significant effort is needed to maintain software quality during its evolution and prevent systems from becoming costly, error-prone, and unsatisfactory. To address these issues, researchers have sought to automate various tasks within the software development process. Search-Based Software Engineering (SBSE)

J. Yuste
Universidad Rey Juan Carlos, Madrid, Spain
e-mail: javier.yuste@urjc.es

E. G. Pardo (✉)
Departamento de Informática y Estadística, Universidad Rey Juan Carlos, Madrid, Spain
e-mail: eduardo.pardo@urjc.es

F. Chicano
ITIS Software, Universidad de Málaga, Málaga, Spain
e-mail: chicano@uma.es

© Springer Nature Switzerland AG 2025

R. Martí et al. (eds.), *Handbook of Heuristics*,
https://doi.org/10.1007/978-3-032-00385-0_70

is a research area that focuses on reformulating software engineering tasks as optimization problems and designing algorithms to solve them. By automating these tasks, SBSE aims to enhance software quality, reduce the burden on developers, and lower associated costs. This chapter provides an introduction to the field of SBSE, detailing some of the most widely studied families of optimization problems in the field: software project scheduling, optimal feature selection, software test case generation, and software module clustering. The chapter concludes with some insights and references for further reading.

Keywords

Search-Based Software Engineering · Software Project Scheduling · Optimal Feature Selection · Test Case Generation · Software Module Clustering

Introduction

Software plays a crucial role in the proper functioning of modern systems, controlling a wide range of electronic devices, from ubiquitous tools like mobile phones to critical systems in aircraft and medical instruments. Over the past few decades, computer programming has revolutionized the world and is expected to continue doing so in the coming years. As software systems grow in size and complexity, navigating them becomes increasingly challenging. To address these issues, the scientific community has intensified its efforts to automatically enhance and correct software systems.

Search-Based Software Engineering (SBSE) is a research area that focuses on reformulating Software Engineering (SE) tasks as optimization problems and designing optimization algorithms to solve them. By applying optimization techniques to SE tasks, the SBSE community aims to improve the quality of software systems, correct code errors, and generally facilitate the work of software developers.

Although some preliminary works from the twentieth century can be classified within the field of SBSE, [81, 82, 92, 137, 168, 172], the term itself was first coined in 2001 by Mark Harman and Bryan F. Jones [58]. In their work, the authors described SBSE as an emerging field of SE research and expressed their expectation of a "dramatic development of the field" [58]. More than 20 years later, in 2024, the Symposium on Search-Based Software Engineering (SSBSE) celebrated its 16th edition [76]. Throughout the first 11 editions, more than 290 authors from 25 countries had already contributed to the main track of SSBSE [36]. The contributions of the SBSE community to reputed journals have also been significant.

SBSE problems are typically classified according to the phases of the software development process in which they arise. The life cycle of a software project encompasses all activities performed to evolve a system from its conception to its retirement [70]. To ensure the creation of high-quality code, software engineers

Table 1 Classification of some families of optimization problems identified in the SBSE research field according to the phase within an SDLC where they mostly arise, based on related literature [40, 59, 124, 147, 150, 152]

SDLC phase	SBSE problem	Relevant references
Project management	Predictive modeling	[41, 46, 151]
	Software project scheduling	
Analysis and requirements	Requirements selection and prioritization	[10, 146, 187]
	Next release problem	
Design	Product line architecture generation	[35, 98, 102]
	Product line architecture evolution	
	Optimized feature selection	
Integration and deployment	Service composition	[13, 148]
	Component allocation	
	Software deployment	
	Systems configuration	
Software testing	Test case generation	[47, 60, 85]
	Test case prioritization	
	Test case minimization	
	Program verification	
Software maintenance	Software module clustering	[19, 69, 124, 186]
	Automated bug fixing	
	Software refactoring	

are advised to follow a structured approach known as the Software Development Life Cycle (SDLC). An SDLC is a framework that defines, orders, and establishes transition criteria for various stages, each containing several processes and activities related to the system's life cycle [70]. Although the exact number of phases can be debated, SDLCs generally include the following phases: requirements definition, analysis and design, software development, testing, and maintenance. Implementing a defined SDLC model has been shown to enhance the quality of software products [174].

Following the structured approach of SDLC, Table 1 presents a classification of various SBSE problems according to the SDLC phase in which they arise primarily, based on the related literature [40, 59, 124, 147, 150, 152]. As observed, SBSE problems emerge in numerous SDLC phases, ranging from project management to software maintenance.

In the following sections, we will describe some of the most relevant problems in the literature. Software project scheduling is discussed in section "Software Project Scheduling"; optimized feature selection is presented in section "Optimal Feature Selection"; software test case generation is detailed in section "Test Case Generation"; and software modularity clustering is described in section "Software Module Clustering." Finally, conclusions are presented in section "Conclusion."

Software Project Scheduling

Software project management focuses on ensuring that a software product meets the elicited requirements and is delivered on time and within budget [152]. This involves identifying tasks, estimating the efforts required to complete them, identifying dependencies and bottlenecks, and assigning resources (typically, employees) to tasks. Typically, a schedule in the form of a Gantt chart is produced to detail these decisions [41]. For large projects, creating such a schedule is a complex task, and the number of possible schedules is vast [41]. The problem becomes increasingly challenging when considering additional factors such as the skills of each employee and uncertainty. Therefore, automating project scheduling with the optimization of certain objectives, such as completion times or cost reduction, can offer significant benefits to software development teams.

Software Project Scheduling (SPS) is an optimization problem that focuses on finding the optimal assignment of tasks to employees and determining the best order in which these tasks should be tackled to optimize a set of objectives, such as minimizing completion times, reducing associated costs, and minimizing overtime planning [152].

In Fig. 1, a flow chart provides an overview of SBSE approaches for the SPS problem, based on [59]. As illustrated, there are two main activities, represented by rounded rectangles filled in red: the search process, conducted by search-based optimization techniques, and the evaluation, performed by an evaluator. Additionally, there are three types of data items, represented by green polygons. The input to the optimization methods can contain various kinds of information:

- **Tasks information**: This set of inputs includes information about (i) the tasks that need to be performed; (ii) the effort estimates for each task, which may encompass cost estimates, duration estimates, or both; and (iii) the dependencies between tasks, as some tasks cannot be initiated until a specified set of tasks has been completed.
- **Staff information**: This set of inputs comprises information about (i) the staff available to work on the project; (ii) the costs associated with including each employee; and (iii) the amount of work each employee can dedicate to the project (e.g., hours per day).
- **Skills information**: This set of inputs contains information about (i) the skills of each staff member and (ii) the skills required to perform each task.

Although Fig. 1 includes tasks, staff, and skills information for completeness, not all the inputs described are always considered for SPS problems. In the literature, authors have considered different levels of detail for these problems. For instance, some authors only consider tasks and staff information [9], while others include the level of proficiency in different skills [27, 117, 160]. Additionally, some have considered differences in costs between normal and overtime work hours [27]. Regardless of the level of detail, the information provided is then used as input for search-based optimization techniques that attempt to find the optimal solution.

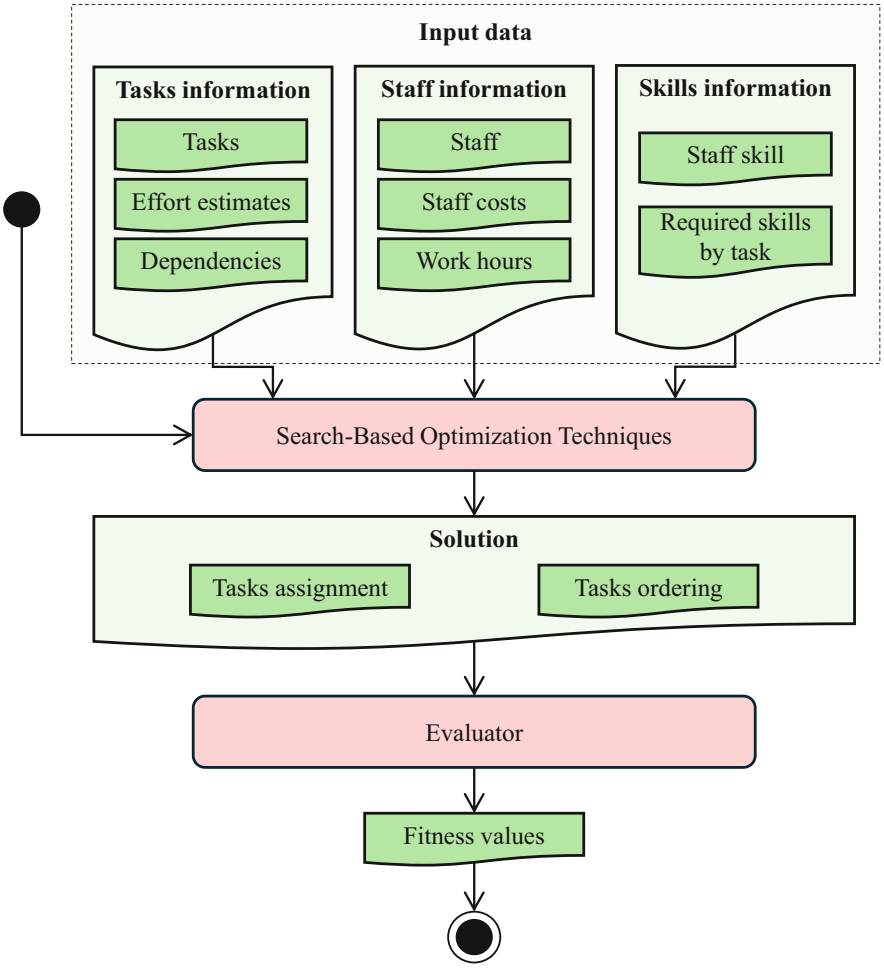


Fig. 1 Generic overview of SBSE approaches for the SPS problem, based on [59]

In this context, as illustrated in Fig. 1, a solution involves assigning personnel to tasks (tasks assignment) and determining the order in which these tasks are tackled (tasks ordering). This solution is then evaluated. Depending on the number of objective functions, the SPS can be approached in a mono-objective or multi-objective manner. A common goal is to minimize the completion time [9], meaning the shorter the project duration, the better. Similarly, minimizing the costs associated with the project is often considered [5, 27].

Moreover, the planning of software projects, like project planning in general, is subject to risks, such as product flaws, task delays, and budget reductions. Consequently, some works also consider risk [51, 90, 180]. For instance, some authors use optimistic and pessimistic estimations of the input information to design robust approaches [9]. Additionally, the challenges posed by risks during

project completion have naturally led researchers to propose dynamic approaches [128, 160]. These approaches account for various dynamic events that may arise during the project, such as the addition or removal of tasks, or changes in personnel. Thus, in addition to other objectives, the robustness and/or stability of the solutions (i.e., making small adjustments in response to dynamic events) is rewarded [160].

To solve the SPS problem, various metaheuristic schemes have been employed as the basis for the development of search algorithms. Table 2 provides a chronological summary of some of the proposals for the SPS discussed in this section, detailing the year of publication (Year), the metaheuristic approaches used (Method(s)), the quality aspects evaluated (Objective function (s)), whether a single (Single-objective) or multi-objective (Multi-objective) approach was used, and whether a dynamic approach was considered (Dynamic). As can be observed, the most prevalent approach has been the use of Genetic Algorithms (GAs) [5, 9, 26, 50, 179,

Table 2 Chronological summary of proposals for the SPS optimization problem included in this section.
Acronym key (alphabetically sorted) · **Method:** Ant Colony Optimization (ACO); Cooperative Coevolutionary Genetic Algorithm (CCGA); Evolutionary Algorithm (EA); Firefly Algorithm (FA); Genetic Algorithm (GA); Generalized Differential Evolution 3 (GDE3); Integer Linear Programming (ILP); Memetic Algorithm (MA); Multi-Objective Cellular GA (MOCell); Non-Dominated Sorting Genetic Algorithm II (NSGA-II); Pareto Archived Evolution Strategy (PAES); Particle Swarm Optimization (PSO); Simulated Annealing (SA); Stochastic Hill Climbing (SHC); Strength Pareto Evolutionary Algorithm 2 (SPEA2). **Objective function(s):** Cost (C); Duration (D); Earliest Release of Resources (E); Feasibility (F); Fragmentation (Fr); Idle time between tasks (I); Less time slots to implement a task (L); Robustness (R); Satisfaction (Sa); Stability (S)

Year	Method(s)	Objective function(s)	Single-objective	Multi-objective	Dynamic	Ref.
2004	GA	D	✓		✓	[9]
2007	GA	C+D	✓			[5]
2009	GA	C+D+S	✓			[50]
↓	SPEA2	C+D+S		✓		[51]
2010	GA	C+D+S	✓		✓	[179]
2011	NSGA-II, SPEA2, PAES, MOCell, GDE3	C+D+F		✓		[31]
↓	GA, SHC, SA	D+Fr		✓		[41]
2012	NSGA-II, SPEA2, PAES, MOCell	C+D		✓		[32]
↓	ACO	C	✓			[27]
2013	EA, GA	C+D	✓			[117]
↓	GA	C	✓			[180]
2016	GA	D	✓			[26]
2018	MA	C+D+S+R+Sa		✓	✓	[161]
2019	FA	C+D+S+R		✓	✓	[29]
2020	CCGA	C+D+Sa+R		✓		[162]
↓	PSO	C+D+S+R		✓	✓	[162]
2023	GA	C+D		✓	✓	[128]
2024	ILP	C+D+I+E+L	✓		✓	[110]

180]. While many researchers propose different variations of the general scheme or specific crossover and mutation operators, it is also common to see standard GAs such as NSGA-II or SPEA2 [31, 32, 51].

In addition, some authors have proposed heuristics based on other schemes. For example, Chen et al. [27] introduced a method based on Ant Colony Optimization. Minku et al. [117], along with an evolutionary algorithm, proposed a method based on local search. Nigar et al. [128] proposed two different heuristic methods to build initial solutions as part of a multi-objective evolutionary algorithm scheme: One heuristic focuses on minimizing completion time, while the other focuses on minimizing project costs.

Other works include proposals based on simulated annealing and hill climbing methods [41], a firework algorithm [29], a Q-learning-based memetic algorithm [161], a cooperative coevolutionary GA [162], and a particle swarm optimization approach [164]. Recently, an integer linear program formulation was proposed as an alternative approach to the problem [110].

The SPS problem is one of the most extensively studied problems in the SBSE literature. There are numerous formulations of the problem, each with different input data, objectives, and constraints, as well as a variety of approaches. For a more comprehensive overview of the literature on this problem, we refer the reader to other relevant sources [46, 151, 152, 170].

Optimal Feature Selection

A Software Product Line (SPL) is defined as "*a set of software-intensive systems that share a common, managed set of features satisfying the specific needs of a particular market segment or mission and that are developed from a common set of core assets in a prescribed way*" [33]. In essence, an SPL is a family of software products based on a common architecture, incorporating common and variable elements [130]. Maintaining an SPL facilitates systematic reuse of software components in a family of similar products, resulting in significant improvements in cost, quality, time to market, and developer productivity [169].

To develop an SPL, a Product Line Architecture (PLA) must be designed. This architecture specifies the common and variable elements of the product family and their relationships [88]. Essentially, a PLA encompasses the core design model of an SPL [35]. Consequently, the quality of a PLA is crucial for the success of an SPL [140]. However, designing a high-quality PLA is time-consuming, costly, and prone to errors [112]. Additionally, several conflicting metrics measure the quality of PLAs, making optimization problems related to PLAs often multi-objective in nature.

With respect to PLA design, there are two primary categories of problems [88]. The first category focuses on PLA generation, which involves identifying PLA components and configuring the architecture. The second category addresses PLA evolution, including the refactoring and remodularization of existing PLAs, with an emphasis on improving these architectures. Most studies in the literature focus on PLA refactoring and remodularization [88].

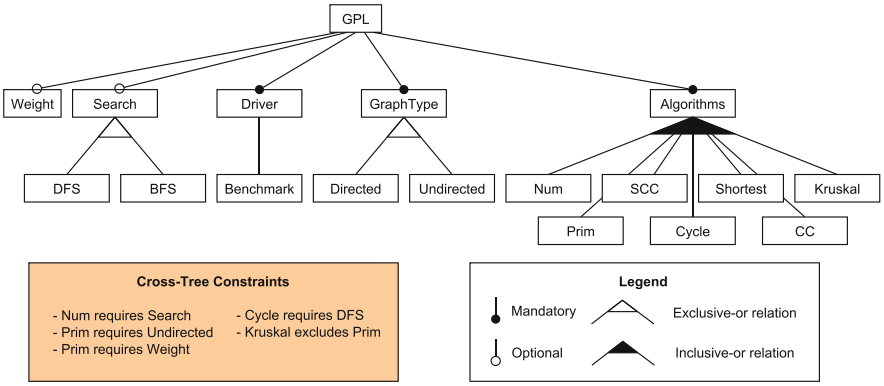


Fig. 2 A sample feature model for a Graph Product Line (figure adapted from [101])

Feature models (FMs) have become the de facto standard for modeling features of an SPL [101]. A feature is a behavior of a software product that is visible to the end user and is of interest to some stakeholders [156]. FMs model features in a tree-like structure, where nodes represent features and edges represent relations between features. By considering the relations between nodes, an FM denotes the set of feature combinations that the products of an SPL can have [83]. Figure 2 represents the FM for a Graph Product Line (GPL) [100]. The GPL is a case study SPL that has been widely used in the literature [101]. In the GPL, a product is a collection of algorithms that can be applied to a graph. In an FM, each node, except the root, represents a feature. A feature can only be included in a product if its parent feature is included as well. Following the example presented, "DFS" could not be included if "Search" was not included. There are four types of relations that can appear in an FM. A *mandatory feature* has to be selected whenever its parent is selected. In the example, the graph type needs to be defined. An *optional feature* might be selected if its parent is selected, but it is not mandatory. In the example, a graph may or may not have weights. An *exclusive-or relation* indicates that only one of the features in the group can be selected. In the example, a graph can be either "Directed" or "Undirected," but not both. An *inclusive-or relation* indicates that any number of the features in the group can be selected, as long as at least one is selected. In the example, more than one algorithm can be selected. Finally, there might also be cross-tree constraints in the FM. These constraints impose certain restrictions in the selection of features within the model. For instance, in the GPL depicted, "Prim" algorithm can only be selected if the graph is "Undirected" and has weights. Notice that even from such a simple FM, there are 73 possible combinations [101]. As SPLs become larger, the number of possible combinations becomes enormous. Large product lines, such as the Linux kernel, include more than 6000 features [156]. Indeed, the problem of optimal feature selection has been shown to be NP-hard [173].

For the optimal selection of features in PLAs, various quality attributes are frequently considered. These attributes may include correctness (i.e., valid feature

selection), richness of features (the more features, the better), reuse of previously used features, avoidance of known defects, and minimization of costs.

Regarding the availability of instances, the Software Product Lines Online Tools (SPLOT) project at the University of Waterloo represents one of the largest efforts to provide feature models of varying sizes [115]. SPLOT instances have been used to obtain numerous results concerning SPLs [156].

As is common in many SBSE problems, most algorithmic approaches propose the use of GAs [14, 34, 35, 87, 165]. Table 3 presents a chronological summary of works studying the problem of optimal features selection. Most considered objectives related to the correctness, cost, the number of known defects, use

Table 3 Chronological summary of proposals for the optimal feature selection optimization problem included in this section

Acronym key (alphabetically sorted) · **Method:** Consistency-preserving configuration operators (CPCOs); Differential Evolution (DE); guided improvement algorithm (GIA); indicator-based evolutionary algorithm (IBEA); IBEA and DE (IBED); Indicator optimization and rules Violation controlling Evolutionary Algorithm (IVEA); multi-objective bee algorithm for feature subset selection in software product lines (MOBAFS); multi-objective cellular GA (MOCell); multi-objective cross-generational elitist selection, heterogeneous recombination, cataclysmic mutation (MOCHC); multi-objective evolutionary algorithm based on decomposition (MOEA/D); non-dominated sorting genetic algorithm II (NSGA-II); SATVaEA with improvement strategies (MySPLC); Pareto archived evolution strategy (PAES); probabilistic models-based reproduction operators, adaptation of weight vectors and decomposition-based framework (PMAD); SAT solvers and IBEA (SATIBEA); SAT solvers and VaEA (SATVaEA); ShrInk Prioritize (SIP); strength Pareto evolutionary algorithm 2 (SPEA2); vector angle–based evolutionary algorithm (VaEA) **Objective function(s):** Correctness (Cr); cost (C); defects (D); memory (M); novelty of features (NF); richness of features (R); time (T)

(*) Specific objectives are considered for different instances

Year	Method(s)	Objective Functions	Multi-objective	Ref.
2013	IBEA, NSGA-II	Cr+R+NF+D+C	✓	[156]
2014	IBEA, GIA	(*)	✓	[129]
2015	IBEA, NSGA-II, SPEA2, IVEA	Cr+R+NF+D+C	✓	[97]
↓	SATIBEA	Cr+R+NF+D+C	✓	[62]
	NSGA-II, IBEA, MOCell, PAES, SPEA2, MOCHC	Cr+R+NF+D+C	✓	[138]
2016	IBED, IBEA, NSGA-II, MOCell	Cr+R+NF+D+C	✓	[181]
↓	SIP, SPEA2, MOEA/D	Cr+R+NF+D+C (*)	✓	[63]
2018	SATIBEA	Cr+R+NF+D+C	✓	[153]
↓	IBEA, SATIBEA, IVEA-II	Cr+R+NF+D+C	✓	[98]
	VaEA, SATVaEA, SATIBEA	R+NF+D+C (*)	✓	[176]
2020	SATVaEA	R+NF+D+C (*)	✓	[177]
2021	PMAD, SATVaEA, NSGA-II	Cr+NF+D+M+T	✓	[178]
2022	MySPLC	R+C+NF+D	✓	[65]
2023	IBEA+CPCOs, SATIBEA	(*)	✓	[64]
↓	MOBAFS, SATIBEA	C+NF+D	✓	[55]

of features not used before, and richness of features. As it can be observed, many authors have explored the performance of widely used algorithms such as NSGA-II, SPEA2, and IBEA. Moreover, several operators specifically tailored for the problem have been proposed and integrated into these well-known algorithms [64, 65, 178]. Additionally, some authors have proposed hybridizations between IBEA and constraint satisfaction solvers [62], satisfiability modulo theory solvers [53], and differential evolution [181]. Given the high number of constraints in large feature models (FMs), some authors proposed using correct (i.e., valid) configurations as seeds in the initial population of solutions, which yielded good results [153, 156]. Hierons et al. [63] proposed a method named ShrInk Prioritise, which included new objectives and was integrated into other algorithms, including NSGA-II, IBEA, and MOEA/D. There are also approaches that propose the use of exact methods to solve the problem of optimal feature selection [129].

Finally, to support the methods available in the literature for PLA optimization, a tool was needed to allow users to select different approaches and visualize the proposed solutions. To this end, a tool named OPLA (optimization for PLAs) was presented at the 7th SSBSE in 2015 [44]. This tool receives a class diagram representing a PLA, performs an optimization process, and allows the visualization of changes through a graphical interface.

As discussed in this section, there are numerous works studying optimization problems regarding SPLs, ranging from PLA design to feature selection, each with different input data, objectives, constraints, and approaches. For a more comprehensive overview on this subject, we refer the reader to other relevant sources [88].

Test Case Generation

Software testing is a crucial undertaking for ensuring the quality and reliability of software products [171]. It involves the "*set of activities conducted to facilitate discovery and evaluation of properties of test items*" (i.e., work products to be tested) [73]. The primary goals of software testing are to (i) detect defects, (ii) gather information about a test item, and (iii) create confidence in the correct functioning of the software system. Depending on the type of test item under evaluation, tests can be divided into different levels, such as unit testing, integration testing, and system testing, depending on granularity [73].

Unit testing focuses on testing software units, which are atomic-level software components that can be subjected to standalone testing, isolated from the rest of the system [70]. It is a widely adopted practice because tests are usually simple, fast, and effective in verifying software components at a low level [152]. Integration and system testing focus on higher levels, testing the correct interaction between different components or the observable behaviors of the system. By performing unit testing first, one can then test compound behaviors between different software components to obtain comprehensive tests for complex applications [56]. However, creating tests is a process that requires time and effort. It also requires deep knowledge of the system, experience in designing tests, and programming capabilities.

In this context, test automation and optimization tools are invaluable assets for creating tests quickly and comprehensively, leading to improved software quality and reduced development costs. Consequently, Search-Based Software Testing (SBST) has emerged as a cost-effective approach to address the impracticality of exhaustive manual testing [39].

Various problems are studied within the field of test optimization, which can be primarily classified into test case generation, test case selection, test case prioritization, and test suite minimization. In test generation, the objective is to create test cases that cover all components of the code [39]. Test selection focuses on identifying subsets of test cases that are sufficient to meet certain requirements. Test case prioritization involves ranking test cases to guide the testing process effectively. Finally, test case minimization aims to identify and remove redundant or ineffective test cases.

Although challenges remain, the success of search algorithms is evident in the emergence of SBST tools such as EvoSuite [49], Randoop [133], or TestSpark [155]. These frameworks employ various strategies, including Local Search, Novelty Search, SPEA2, NSGA-II, and GAs in general. Perhaps the most notable example of the success of SBST approaches is Sapienz [111], a tool introduced in 2016 that was later integrated into Facebook's software development process [6]. This integration spurred further interest in SBSE tools [80].

In the context of SBST, unit test generation is one of the most studied problems, leading to a dedicated survey as early as 2004 [114]. The first work on search-based techniques for testing, focused on generating tests that executed different paths within the code, was published in 1976 [116].

To address the test case generation problem, various formulations have been proposed in the literature. In general, the objective is to generate a set of test cases that cover a set of test targets [135]. That is, a set of test cases is a solution, and the best solution is the one that tests all the targets.

Figure 3 presents an example method named "Dominance," which evaluates the dominance relation between two objective vectors in a multi-objective optimization context. Given two vectors (*vector1* and *vector2*) and the number of objectives (n), the method returns the following:

- A negative number (-1) if *vector1* dominates *vector2* ($vector1 \succ vector2$)
- A positive number (1) if *vector2* dominates *vector1* ($vector1 \prec vector2$)
- Or zero (0) if neither dominates the other ($vector1 \not\succ vector2$ and $vector1 \not\prec vector2$)

All objectives are assumed to be minimized. The pseudocode of the method is shown on the left, and the corresponding activity diagram is shown on the right. The line numbers in the pseudocode correspond to the numbers in the activity diagram. As can be observed, different statements are executed depending on the input. If *vector1* dominates *vector2*, then lines 12 and 19 are not executed. If neither vector dominates the other, lines 16–20 are not executed. A possible goal of the test case generation problem might be to generate a set of test cases that execute all lines, with each statement being a test target of the system under testing. Greater coverage

Algorithm 1 Dominance

```
1:  procedure DOMINANCE(vector1, vector2, n)
2:      firstBetter ← false
3:      secondBetter ← false
4:      i ← 0
5:      while i < n do
6:          if vector1[i] < vector2[i] then
7:              firstBetter ← true
8:          else if vector1[i] > vector2[i] then
9:              secondBetter ← true
10:         end if
11:         if firstBetter & secondBetter then
12:             return 0
13:         end if
14:         i ← i + 1
15:     end while
16:     if firstBetter then
17:         return -1
18:     else
19:         return 1
20:     end if
21: end procedure
```

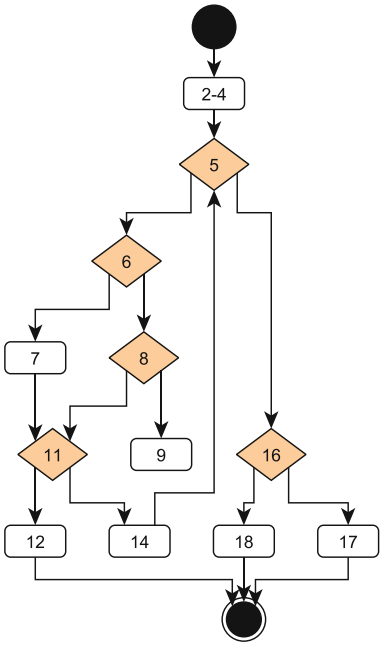


Fig. 3 Pseudocode for identifying dominance relations between two objective vectors (left) and corresponding activity diagram (right). Line numbers in the activity diagram match those in the pseudocode

of targets indicates a better solution. This might be the simplest formulation of the problem. However, depending on the definition of the test targets, other variants can be identified:

- **Branch Coverage**: To reach a given code statement, it is sufficient to execute the branch where that statement is located. Therefore, a natural alternative to statement coverage is to try to generate a set of tests that execute all the conditional statement branches in the code. That is, each branch of the code execution flow represents a different test target. The optimal solution is a set of test cases that covers all branches of the code, therefore achieving complete coverage, since all code statements are executed [135].
- **Mutation Testing**: A test suite without oracles (e.g., assert statements on the outputs of a class under test) merely checks whether some test items are executed [48]. Therefore, they are only sensitive to errors like segmentation faults or runtime exceptions. For most errors, the absence of an automated oracle implies manual review. In this context, mutation testing has received significant attention in the literature. A mutant is a variant of the original artifact that has been modified to mimic real faults. A test suite is said to cover or kill a mutant if the observable state or return values differ between the original artifact and the mutant. If the test leads the mutant to a state that differs from the original artifact, the mutant is said to be weakly killed [48]. If the test case contains an assertion

that evaluates to false when executed on the mutant and true when executed on the original class, the mutant is said to be strongly killed by the test case [48]. The goal is to find a set of test cases that kills all mutants. To guide the search, the objective function also considers distances toward state infection (i.e., a difference in the execution state immediately after the mutated statement) and toward propagation of infection (i.e., the possibility of observing a different state in the execution trace of the statements following the mutated one) [135].

In the aforementioned approaches, the objective functions evaluate the test suite by summing all distances across test cases simultaneously. However, a multi-objective approach may be more suitable in this context. In [135], the authors proposed a many-objective formulation, where the objectives are the individual distances from each test target. The solution is a set of test cases, each evaluated based on the coverage of individual test targets rather than total coverage. This formulation incentivizes the creation of test cases for previously uncovered targets. This generic formulation can then be specialized for branch, statement, or mutation coverage by redefining the set of test targets and the distance measures.

For the many-objective formulation, a method named Dynamic Many-Objective Sorting Algorithm (DynaMOSA) was proposed in 2018 [135]. This method extends the general scheme of evolutionary algorithms by using a preference criterion to rank solutions, prioritizing those closer to uncovered targets. DynaMOSA has become the reference method for the test generation problem and is currently the default algorithm used in the EvoSuite tool [49]. Nevertheless, there is still room for improvement, and open issues remain to be solved. Consequently, researchers continue to work on the test case generation problem and related issues, such as test case prioritization and minimization.

Table 4 presents a chronological summary of works regarding test case generation included in this section, detailing the year of publication (Year), the metaheuristic approaches used (Method(s)), the quality aspects evaluated (Objective Function(s)), and whether a single (Single-objective) or multi-objective (Multi-objective) approach was used. As in most areas of SBSE, the majority of proposals for SBST problems are based on nature-inspired metaheuristics [89]. The most studied frameworks in this context are genetic algorithms (GAs) [15,23,96,103,188]. In addition to GAs, Betts et al. [20] proposed a surrogate-based optimization method focusing on autonomous vehicle software. Others have proposed using a GA-based fuzzing method to find security vulnerabilities in industrial control systems [175] or using genetic programming for test generation [43]. Moreover, some authors have investigated hybridizations of GAs with artificial neural networks [61]. As is often the case, the efficiency of standard GAs, such as NSGA-II, NSGA-III, SPEA2, and IBEA, has also been investigated [52,54,78,113].

In contrast to GAs, some authors have proposed methods based on alternative schemes. For example, some have proposed methods based on ant colony optimization to reduce the test suite of software product lines [21, 22, 157]. Other works have focused on adapting cuckoo search algorithms to software test case generation [4, 159] or firefly algorithms for mutation testing [158, 163] and test case prioritization [86]. Similarly, simulated annealing has been investigated for

Table 4 Chronological summary of proposals for software testing optimization included in this section

Acronym key (alphabetically sorted) · **Method:** Ant Colony Optimization (ACO); Artificial Bee Colony (ABC); Artificial Neural Network with GA Case Suite Prioritization (ANNGCSP); Construct, Merge, Solve and Adapt (CMSA); Cuckoo Search (CS); Differential Evolution (DE); Dynamic Many-Objective Sorting Algorithm (DynaMOSA); Firefly Algorithm (FA); Genetic Algorithm (GA); Genetic Programming (GP); Indicator-Based Evolutionary Algorithm (IBEA); Interactive evolutionary test generation with readability assessment (InterEvo-TR); Memetic Algorithm (MA); Model Inference Search Heuristic (MISH); Many-Objective Sorting Algorithm (MOSA); Multipath GA (MPGA); Non-dominated sorting genetic algorithm II (NSGA-II); Particle Swarm Optimization (PSO); Surrogate-Based Optimization (SBO); Strength Pareto Evolutionary Algorithm 2 (SPEA2). · **Objective function(s):** Abstract States (AS); Approximation Level + Branch Distance (ALBD); Average Execution Frequency (AEF); Coverage (C); Dissimilarity-based fitness function (D); Fault Identification Capability (FI); Interprocedural Fitness Function (IFF); Lateral Deviation (LD); Lower than Median (LM); Mutation Coverage (M); Size (S); Testing Cost (TC); Weighted State visit frequencies (WS)

Year	Method(s)	Objective function(s)	Single-objective	Multi-objective	Ref.
2015	NSGA-II, SPEA2, IBEA	S+M		✓	[113]
↓	ACO	C	✓		[157]
	GA, PSO	C	✓		[91]
2016	GA, SBO	LD	✓		[20]
↓	ABC, PSO, DE, FA	C, D, AL+BD	✓		[154]
2017	MPGA	C+D	✓		[188]
↓	ABC	C	✓		[3]
	GA	AL+BD			[15]
	DynaMOSA	C		✓	[135]
2018	GA	C	✓		[96]
↓	GP	AS			[43]
2019	GA	C	✓		[23]
↓	PSO	C	✓		[37]
	CMSA	S	✓		[45]
	MA	C+S	✓		[16]
2022	InterEvo-TR	C	✓		[39]
↓	NSGA-II, MOSA, DynaMOSA	C		✓	[132]
	DynaMOSA	IFF		✓	[131]
2023	GA	F1-score	✓		[11]
2024	GA	M		✓	[134]
↓	MISH	LM, WS	✓		[25]

generating test suites [57]. Additionally, particle swarm optimization methods [37, 91, 105] and artificial bee colony algorithms [3, 166] have also been studied for test case generation. Ferrer et al. [45] proposed a hybrid matheuristic called *Construct, Merge, Solve & Adapt*, which was compared with four other methods for testing software product lines. Other works have proposed approaches that include differential evolution [154], memetic algorithms [16], or interactive approaches for

SBST problems [39]. Finally, it is worth noting that the large language models have emerged as promising techniques for many code-related tasks, including the generation of code tests, and some authors are already exploring this line of research [2].

As highlighted in this section, SBST is an active area of research and one of the most extensively studied problems by the SBSE community since its inception. Consequently, the introduction provided here only scratches the surface. For more in-depth information on the subject, we refer the reader to the following relevant sources [18, 89, 136, 152, 171].

Software Module Clustering

According to Lehman's laws of software evolution, formulated between 1974 and 1996 [94], software systems often deteriorate over time due to evolving needs and continued changes, becoming prone to errors (Lehman distinguishes programs into three categories: S, P, and E [95]. Lehman's laws of software evolution apply to E-programs, which are written to perform real-world activities, as opposed to S- and P-programs, which are written to perform specific tasks and do not evolve over time). Among the eight laws proposed by Lehman, four are directly related to the importance and necessity of software maintenance to maintain software quality [94]. According to the ISO (https://www.iso.org/home.html), product quality is defined by eight attributes: functional suitability, reliability, performance efficiency, usability, security, compatibility, maintainability, and portability [72]. Among these attributes, maintainability is defined as the "*degree of effectiveness and efficiency with which a product or system can be modified by the intended maintainers*," including corrections, improvements, or adaptations [72].

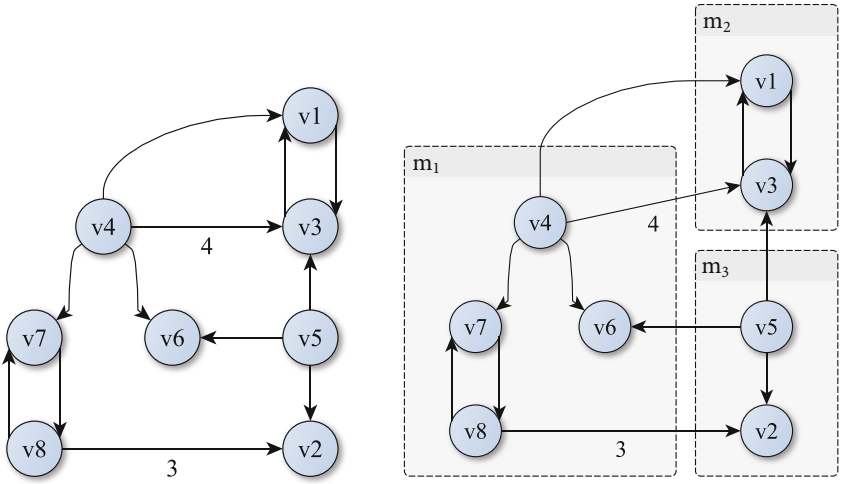
Software maintenance is primarily concerned with providing cost-effective support to a system, ensuring its continued ability to provide services [70]. This support encompasses various corrective, adaptive, perfective, and preventive actions [17]. Despite its critical role, software maintenance is often perceived as a less significant phase, leading to a lack of recognition, motivation, and support [38, 99]. However, it is the most expensive phase of the SDLC, with studies indicating that up to 80% of the total costs are allocated to software maintenance [28]. Notably, a substantial portion of the effort in this phase is dedicated to understanding the existing software [125].

One of the most significant SBSE problems encountered during the software maintenance phase is the Software Modularity Clustering Problem (SMCP). The objective of SMCP is to reorganize a software project to enhance its modularity. According to the ISO, modularity is defined as the set of "*software attributes that provide a structure of highly independent components*" [71]. In an optimal modular structure, components within the same module are strongly related (high cohesion) and weakly connected to components in other modules (low coupling). There are various interpretations of what constitutes a software component, such as a file, a class, or a package. Although the terms "module" and "component"

are sometimes used interchangeably, it is important to note that their definitions are not standardized [71]. In this context, we will use the term "component" to refer to individual elements (files and classes) and "module" to refer to collections of components (packages or folders). Therefore, the primary goal of SMCP is to reorganize the components of a software project into modules to facilitate the understanding of software projects and thereby reduce maintenance costs.

In the context of the SMCP, software projects are typically represented using a graph structure known as a Module Dependency Graph (MDG) [69]. An MDG is a specific type of artifact dependency graph that illustrates the dependencies between components within a graph structure. There are other types of graph that model different aspects of software projects, which are not discussed here [69]. Formally, an MDG is defined as a directed weighted graph, where the vertices correspond to the components of the source code, the edges denote the dependencies between these components, and the weights of the edges indicate the strength of these dependencies.

Figure 4a illustrates the MDG of a hypothetical software project comprising 10 distinct components. Component one, represented by $v1$, has two dependencies (e.g., method calls, inheritance, etc.) on component two, represented by $v2$. Consequently, the edge connecting $v1$ to $v2$ has an associated weight of two. Similarly, component four ($v4$) depends on components one ($v1$), three ($v3$), six ($v6$), and seven ($v7$), among others.



(a) Example MDG with eight vertices.

(b) A possible solution for the MDG of Figure 4a with three modules.

Fig. 4 Example of an MDG (**a**) and a solution for that instance (**b**). Vertices represent the components of the source code, edges represent the dependencies between those components, and weights represent the strength of the dependencies. For the sake of simplicity, only weights with a value greater than one are depicted

Given an MDG, a solution for the SMCP is represented by a clustering of the graph's vertices. Specifically, a solution is a set of non-empty, disjoint subsets of vertices. Figure 4b presents a possible solution for the MDG of Fig. 4a. As illustrated, the components are grouped into three distinct modules: m_1, m_2, and m_3. Specifically, $m_1 = v4, v6, v7, v8$, $m_2 = v1, v2, v3$, and $m_3 = v5, v9, v10$.

In the SMCP, variants exist that differ in how they evaluate the quality of modular organizations. Table 5 provides a chronological summary of articles that study the SMCP. For each work listed, the year of publication, the metaheuristic framework, and the objective function or functions studied are reported. In addition, the table specifies if the problem was studied in a single- (S.O.) or multi-objective (M.O.) approach. As can be observed, the first approach to the SMCP was proposed by Mancoridis et al. in 1998 [108]. These authors introduced the Modularization Quality (MQ) objective function, which calculates the value of modularity as a trade-off between coupling and cohesion. This metric was later extended into two variants to accelerate its computation and improve results: BasicMQ [122] and TurboMQ [118]. As indicated in the table, MQ has become the most widely studied problem in the SMCP literature. However, alternative variants have also been proposed [1, 7, 66, 75, 127, 141, 145].

Furthermore, some authors have proposed multi-objective approaches for the SMCP, which appear to be more suitable in this context [186]. The first to study the SMCP as a multi-objective optimization problem were Praditwong et al. [143], who introduced two approaches: the Maximizing Cluster Approach (MCA) and the Equal-Size Cluster Approach (ECA). Over time, other multi-objective variants have been proposed [68, 77, 123, 149], including extensions of MCA and ECA [30].

Regardless of the metric used to evaluate the quality of solutions, a variety of metaheuristics have been proposed for the SMCP. It is important to note that since the SMCP is proven to be NP-complete [24], exact methods are not suitable except for very small software projects [118]. Instead, approximate search-based metaheuristics are more appropriate [36, 59, 150].

In this context, population-based methods have traditionally been favored in the literature for the SMCP. These include methods based on GAs [42, 79, 104, 107, 127, 143], firefly algorithm [106], estimation of distribution algorithm [77, 167], harmony search [7], multi-agents [66, 67], hyper-heuristics [93], artificial bee colony [8, 30], interactive methods [149], particle swarm optimization [144, 145], and NSGA-III [123].

In contrast, some authors have noted the absence of efficient single-solution metaheuristics such as iterated local search or variable neighborhood descent in the literature [150]. To address this gap, several trajectory-based metaheuristics have been proposed as effective alternatives to traditional population-based approaches, achieving competitive results. Early proposals in this area were based on hill climbing [84, 104, 108, 109, 119, 122] or simulated annealing [1, 120, 121]. More recently, algorithms based on metaheuristics such as large neighborhood search [126], variable neighborhood search [184, 185], multi-objective variable neighborhood search [183, 186], and greedy randomized adaptive search procedure combined with variable neighborhood descent [182] have been proposed.

Table 5 Chronological summary of proposals for the SMCP.
Acronym key (alphabetically sorted) · **Method:** Estimation of Distribution Algorithms (EDA); Evolutionary Call-Dependency Graph Modularization (E-CDGM); Firefly Algorithm (FA); General Variable Neighborhood Search (GVNS); Genetic Algorithm (GA); Graph-Based Modularization Algorithm (GMA); Greedy Randomized Adaptive Search Procedure (GRASP); Grid-Based Large-Scale Many-Objective PSO (GLMPSO); Harmony Search (HS); Hill Climbing (HC); HC based on semantic dependency graph (HCS); Hybrid GA (HGA); Harmony Search (HS); Imperialist Competitive Algorithm (ICA); Interactive Evolutionary Approach (IEA); Iterated Local Search (ILS); Large Neighborhood Search (LNS); Many-Objective ABC (MaABC); Multi-Agent Evolutionary Algorithm (MAEA); Multi-Objective Hyper-Heuristic Evolutionary Algorithm (MHypEA); Next Ascent HC (NAHC); Non-Dominated Sorting Genetic Algorithm III (NSGA-III); Pareto Envelop-Based Selection Algorithm (PESA); Particle Swarm Optimization (PSO); Simulated Annealing (SA); Steepest Ascent HC (SAHC); Two-Archive ABC (TA-ABC); Variable Neighborhood Descent (VND). · **Objective function(s):** Cohesion, Coupling, Package Count index, and package Size index (CCCS); Dependency Quality and Connection Quality (DQCQ); Entropy-Based Objective Function (EOF); Equal-Size Cluster Approach (ECA); Extended ECA (E-ECA); Extended MCA (E-MCA); Function of Complexity Balance (FCB); Interactive Fitness Function (IFF); Linear Compound Criteria (LCC); Maximizing Cluster Approach (MCA); Modularization Quality (MQ); MQ, Non-Extreme Distribution, Coupling, and Cohesion (MQNCC); Multifactor Module Clustering (MFMC); Multi-Objective Fitness Function (MOF); Similarity-Based MQ (SM); Structure of packages, Semantics coherence, and History of changes (SSH)

Year	Method(s)	Objective function(s)	Single-objective	Multi-objective	Ref.
1998	HC	MQ	✓		[108]
1999	HC	MQ	✓		[109]
↓	GA	MQ	✓		[42]
2001	NAHC, SAHC	MQ	✓		[122]
2002	SAHC	MQ	✓		[119]
2005	HC, GAHC	MQ	✓		[104]
2006	GA, HC	MQ	✓		[120]
2008	SA	MQ	✓		[121]
2009	SA	DQCQ	✓		[1]
↓	HGA	MQ	✓		[107]
2011	GA	MQ	✓		[142]
↓	GA	MCA, ECA		✓	[143]
2014	ILS	MQ	✓		[139]
↓	FA	MQ	✓		[106]
2015	NSGA-III	SSH		✓	[123]
2016	HC,GA,MAEA	SM	✓		[66]
↓	E-CDGM	MQ	✓		[74]
	GA	MQ	✓		[79]
	EDA	MQ	✓		[167]
	MHypEA	MCA, ECA		✓	[93]
2017	MAEA	MQ	✓		[67]
↓	HS	CCCS	✓		[7]
	HC	MQ	✓		[84]
	HC	MFMC		✓	[68]

(continued)

Table 5 (continued)

Year	Method(s)	Objective function(s)	Single-objective	Multi-objective	Ref.
2018	PSO	MQNCC	✓		[145]
↓	LNS	MQ	✓		[126]
	IEC	IFF		✓	[149]
	TA-ABC	MCA, ECA		✓	[8]
	MaABC	E-MCA, E-ECA		✓	[30]
2019	GA	EOF	✓		[75]
↓	EDA	MOF		✓	[77]
2020	HGA	FCB	✓		[127]
2021	GMA	LCC	✓		[141]
2022	GRASP-VND	MQ	✓		[182]
↓	VND	FCB	✓		[184]
	GA	MCA, ECA		✓	[12]
	GLMPSO	E-MCA, E-ECA		✓	[144]
	MO-VND	MCA, ECA		✓	[183]
2024	GVNS	FCB	✓		[185]
↓	MO-GVNS	MCA, ECA		✓	[186]

As can be observed, software maintenance is a phase that demands significant effort to uphold high-quality code and provide effective support. The SMCP is an actively researched problem, with ongoing advancements needed to integrate the proposed approaches into industry workflows. For further information on this area, we refer the reader to relevant sources [150, 186].

Conclusion

Software is a critical asset for the proper functioning of modern societies. High-quality software leads to efficient processes, user satisfaction, and reduced economic costs. However, software development is inherently complex, making software engineering an essential discipline. Given the importance of software engineering and the complexity of the tasks involved in any software development process, optimizing these tasks is highly desirable. Search-Based Software Engineering (SBSE) is a research area that formulates software engineering tasks as optimization problems and designs and implements optimization techniques to solve them. Due to the complexity of the tasks and the vast number of potential solutions in real-world software projects, heuristic procedures are considered the most promising approaches.

In this chapter, we have reviewed four families of the most widely studied optimization problems in the area: software project scheduling, optimized feature selection, test case generation, and software module clustering. However, SBSE encompasses many problems that are actively studied across all phases of the software development life cycle. Therefore, the aforementioned families represent

only a subset of the research field. Although the field is rapidly maturing, many open issues and challenges remain to be investigated. The practical nature of the problems studied in SBSE offers a complex yet appealing scenario for the study of optimization techniques. Specifically, SBSE problems provide a suitable context for exploring multi- or many-objective optimization techniques to address conflicting quality metrics or objectives. Additionally, interactive approaches that involve software developers in the optimization process offer promising directions.

Finally, as highlighted in this chapter, most heuristic approaches in this area are based on nature-inspired metaheuristics, such as genetic algorithms, ant colony optimization methods, and artificial bee colony algorithms. Recent advances have also been made using trajectory-based metaheuristics, such as the greedy randomized adaptive search procedure and variable neighborhood search. While these metaheuristic frameworks show promise for the problems at hand, further work is needed, particularly in multi- or many-objective contexts.

Cross-References

- ▸ Ant Colony Optimization: A Component-Wise Overview
- ▸ Evolutionary Algorithms
- ▸ Genetic Algorithms
- ▸ GRASP
- ▸ Iterated Local Search
- ▸ Memetic Algorithms
- ▸ Multiobjective Optimization
- ▸ Multi-start Methods
- ▸ Particle Swarm Optimization
- ▸ Scheduling Heuristics
- ▸ The Hybrid Metaheuristic CMSA
- ▸ Theory of Local Search
- ▸ Variable Neighborhood Descent
- ▸ Variable Neighborhood Search

Acknowledgments This research has been partially supported by grants PID2021-125709OA-C22 and PID2021-126605NB-I00, funded by MCIN/AEI/10.13039/501100011033 and by "ERDF A way of making Europe"; grant CIAICO/2021/224 funded by Generalitat Valenciana; grant M2988 funded by "Proyectos Impulso de la Universidad Rey Juan Carlos 2022"; Cátedra de Innovación y Digitalización Empresarial entre Universidad Rey Juan Carlos y Second Episode (Ref. ID MCA06); the thematic network RED2022-134647-T; the SOFIA project (MICIU/AEI/10.13039/501100011033, grant PLEC2023-010266); Comunidad Autónoma de Madrid, CIRMA-CM (Ref. TEC-2024/COM-404); and Ministerio para la Transformación Digital y de la Función Pública, Concesión TSI-100930-2023-3 (MCA07).

Competing Interest Declaration The author(s) has no competing interests to declare that are relevant to the content of this manuscript.

References

1. Abdeen H, Ducasse S, Sahraoui H, Alloui I (2009) Automatic package coupling and cycle minimization. In: 2009 16th working conference on reverse engineering, pp. 103–112. IEEE
2. Abdullin A, Derakhshanfar P, Panichella A (2025) Test wars: A comparative study of SBST, symbolic execution, and LLM-based approaches to unit test generation. arXiv preprint arXiv:2501.10200
3. Aghdam ZK, Arasteh B (2017) An efficient method to generate test data for software structural testing using artificial bee colony optimization algorithm. Int J Softw Eng Knowl Eng 27(06):951–966
4. Ahmed BS, Abdulsamad TS, Potrus MY (2015) Achievement of minimized combinatorial test suite for configuration-aware software functional testing using the cuckoo search algorithm. Inf Softw Technol 66:13–29
5. Alba E, Chicano JF (2007) Software project management with gas. Inf Sci 177(11):2380–2401
6. Alshahwan N, Gao X, Harman M, Jia Y, Mao K, Mols A, Tei T, Zorin I (2018) Deploying search based software engineering with Sapienz at Facebook. In: Search-based software engineering: 10th international symposium, SSBSE 2018, Montpellier, September 8–9, 2018, Proceedings 10, pp. 3–45. Springer
7. Amarjeet, Chhabra JK (2017) Harmony search based remodularization for object-oriented software systems. Comput Lang Syst Struct 47:153–169
8. Amarjeet, Chhabra JK (2018) TA-ABC: two-archive artificial bee colony for multi-objective software module clustering problem. J Intell Syst 27(4):619–641
9. Antoniol G, Di Penta M, Harman M (2004) A robust search-based approach to project management in the presence of abandonment, rework, error and uncertainty. In: 10th international symposium on software metrics, 2004. Proceedings. pp. 172–183. IEEE
10. Anwar R, Bashir MB (2023) A systematic literature review of AI-based software requirements prioritization techniques. IEEE Access 11:143815–143860
11. Applis L, Panichella A, Marang R (2023) Searching for quality: genetic algorithms and metamorphic testing for software engineering ML. In: Proceedings of the genetic and evolutionary computation conference, pp 1490–1498 (2023)
12. Arasteh B, Fatolahzadeh A, Kiani F (2022) SAVALAN: multi objective and homogeneous method for software modules clustering. J Softw Evolut Proc 34(1):e2408
13. Arcuri A, Fraser G (2011) On parameter tuning in search based software engineering. In: Search based software engineering: third international symposium, SSBSE 2011, Szeged, Hungary, September 10–12, 2011. Proceedings 3, pp 33–47. Springer
14. Assunção WK, Vergilio SR, Lopez-Herrejon RE (2020) Automatic extraction of product line architecture and feature models from UML class diagram variants. Inf Softw Technol 117:106198
15. Bahaweres RB, Zawawi K, Khairani D, Hakiem N (2017) Analysis of statement branch and loop coverage in software testing with genetic algorithm. In: 2017 4th international conference on electrical engineering, computer science and informatics (EECSI), pp 1–6. IEEE
16. Bahrampour A, Rafe V (2021) Using memetic algorithm for robustness testing of contract-based software models. Artif Intell Rev 54:877–915
17. Bakota T, Hegedus P, Ladanyi G, Kortvelyesi P, Ferenc R, Gyimothy T (2012) A cost model based on software maintainability. In: IEEE international conference on software maintenance, ICSM, pp 316–325
18. Barr ET, Harman M, McMinn P, Shahbaz M, Yoo S (2014) The oracle problem in software testing: a survey. IEEE Trans Softw Eng 41(5):507–525
19. Bavota G, Di Penta M, Oliveto R (2014) Search based software maintenance: methods and tools. In: Evolving software systems. Springer, Berlin, pp 103–137

20. Betts KM, Petty MD (2016) Automated search-based robustness testing for autonomous vehicle software. Modell Simul Eng 2016(1):5309348
21. Bharathi M, Sangeetha V (2018) Weighted rank ant colony metaheuristics optimization based test suite reduction in combinatorial testing for improving software quality. In: 2018 second international conference on intelligent computing and control systems (ICICCS), pp 525–534. IEEE
22. Biswas S, Kaiser MS, Mamun S (2015) Applying ant colony optimization in software testing to generate prioritized optimal path and test data. In: 2015 international conference on electrical engineering and information communication technology (ICEEICT), pp 1–6. IEEE
23. Boopathi M, Sujatha R, Kumar CS, Narasimman S, Rajan A (2019) Markov approach for quantifying the software code coverage using genetic algorithm in software testing. Int J Bio-Inspired Comput 14(1):27–45
24. Brandes U, Delling D, Gaertler M, Gorke R, Hoefer M, Nikoloski Z, Wagner D (2007) On modularity clustering. IEEE Trans Knowl Data Eng 20(2):172–188
25. Cao C, Panichella A, Verwer S (2024) Automated test-case generation for rest APIs using model inference search heuristic. arXiv preprint arXiv:2412.03420 (2024)
26. Chand S, Singh HK, Ray T (2016) Finding robust solutions for resource constrained project scheduling problems involving uncertainties. In: 2016 IEEE congress on evolutionary computation (CEC), pp 225–232. IEEE
27. Chen WN, Zhang J (2012) Ant colony optimization for software project scheduling and staffing with an event-based scheduler. IEEE Trans Softw Eng 39(1):1–17
28. Chen C, Alfayez R, Srisopha K, Boehm B, Shi L (2017) Why is it important to measure maintainability and what are the best ways to do it? In: 2017 IEEE/ACM 39th international conference on software engineering companion (ICSE-C), pp 377–378. IEEE
29. Cheng J, Ji J, Guo YN, Ji J (2019) Dynamic multiobjective software project scheduling optimization method based on firework algorithm. Math Probl Eng 2019(1): 8405961
30. Chhabra JK (2018) Many-objective artificial bee colony algorithm for large-scale software module clustering problem. Soft Comput 22(19):6341–6361
31. Chicano F, Luna F, Nebro AJ, Alba E (2011) Using multi-objective metaheuristics to solve the software project scheduling problem. In: Proceedings of the 13th annual conference on genetic and evolutionary computation, pp 1915–1922
32. Chicano F, Cervantes A, Luna F, Recio G (2012) A novel multiobjective formulation of the robust software project scheduling problem. In: Applications of evolutionary computation: evoapplications 2012: EvoCOMNET, EvoCOMPLEX, EvoFIN, EvoGAMES, EvoHOT, EvoIASP, EvoNUM, EvoPAR, EvoRISK, EvoSTIM, and EvoSTOC, Málaga, April 11–13, 2012, Proceedings, pp 497–507. Springer
33. Clements P, Northrop LM (2001) Software product lines: practices and patterns. Addison-Wesley Longman, Boston
34. Colanzi TE, Vergilio SR (2016) A feature-driven crossover operator for multi-objective and evolutionary optimization of product line architectures. J Syst Softwe 121:126–143
35. Colanzi TE, Vergilio SR, Gimenes IM, Oizumi WN (2014) A search-based approach for software product line design. In: Proceedings of the 18th international software product line conference-volume 1, pp 237–241 (2014)
36. Colanzi TE, Assunção WK, Vergilio SR, Farah PR, Guizzo G (2020) The symposium on search-based software engineering: past, present and future. Inf Softw Technol 127:106372
37. Damia A, Esnaashari M, Parvizimosaed M (2021) Automatic web-based software structural testing using an adaptive particle swarm optimization algorithm for test data generation. In: 2021 7th international conference on web research (ICWR), pp 282–286. IEEE
38. Dekleva S (1992) Delphi study of software maintenance problems. In: Proceedings conference on software maintenance 1992, pp 10–11. IEEE Computer Society
39. Delgado-Pérez P, Ramírez A, Valle-Gómez KJ, Medina-Bulo I, Romero JR (2022) InterEvo-TR: Interactive evolutionary test generation with readability assessment. IEEE Trans Softw Eng 49(4):2580–2596

40. Di Penta M (2012) SBSE meets software maintenance: Achievements and open problems. In: International symposium on search based software engineering, pp 27–28. Springer
41. Di Penta M, Harman M, Antoniol G (2011) The use of search-based optimization techniques to schedule and staff software projects: an approach and an empirical study. Softw Pract Exper 41(5):495–519
42. Doval D, Mancoridis S, Mitchell BS (1999) Automatic clustering of software systems using a genetic algorithm. In: STEP'99, Proceedings ninth international workshop software technology and engineering practice, pp 73–81. IEEE
43. Esparcia-Alcázar AI, Almenar F, Vos TE, Rueda U (2018) Using genetic programming to evolve action selection rules in traversal-based automated software testing: results obtained with the testar tool. Memetic Comput 10:257–265
44. Féderle ÉL, Ferreira TDN, Colanzi TE, Vergilio SR (2015) Optimizing software product line architectures with OPLA-tool. In: Search-based software engineering: 7th international symposium, SSBSE 2015, Bergamo, September 5–7, 2015, Proceedings 7, pp. 325–331. Springer
45. Ferrer J, Chicano F, Ortega-Toro JA (2021) CMSA algorithm for solving the prioritized pairwise test data generation problem in software product lines. J Heurist 27:229–249
46. Ferrucci F, Harman M, Sarro F (2014) Search-based software project management. In: Softw project management in a changing world. Springer, Berlin, pp 373–399
47. Formica F, Fan T, Menghi C (2023) Search-based software testing driven by automatically generated and manually defined fitness functions. ACM Trans Softw Eng Methodol 33(2): 1–37. ACM New York, NY
48. Fraser G, Arcuri A (2015) Achieving scalable mutation-based generation of whole test suites. Empir Softw Eng 20:783–812
49. Fraser G, Zeller A (2010) Mutation-driven generation of unit tests and oracles. In: Proceedings of the 19th international symposium on software testing and analysis, pp 147–158
50. Ge Y (2009) Software project rescheduling with genetic algorithms. In: 2009 international conference on artificial intelligence and computational intelligence, vol 1, pp 439–443. IEEE
51. Gueorguiev S, Harman M, Antoniol G (2009) Software project planning for robustness and completion time in the presence of uncertainty using multi objective search based software engineering. In: Proceedings of the 11th annual conference on genetic and evolutionary computation, pp 1673–1680
52. Guizzo G, Califano F, Sarro F, Ferrucci F, Harman M (2023) Inferring test models from user bug reports using multi-objective search. Empir Softw Eng 28(4):95
53. Guo J, Liang JH, Shi K, Yang D, Zhang J, Czarnecki K, Ganesh V, Yu H (2019) SMTIBEA: a hybrid multi-objective optimization algorithm for configuring large constrained software product lines. Softw Syst Model 18:1447–1466
54. Gupta N, Sharma A, Pachariya MK (2022) Multi-objective test suite optimization for detection and localization of software faults. J King Saud Unive-Comput Inf Sci 34(6):2897–2909
55. Hajizadeh N, Jahanbazi P, Akbari R (2023) A method for feature subset selection in software product lines. Int J Softw Innov 11(1):1–22
56. Hamill P (2004) Unit test frameworks: tools for high-quality software development. O'Reilly Media, Sebastopol (2004)
57. Haraty RA, Mansour N, Zeitunlian H (2018) Metaheuristic algorithm for state-based software testing. Appl Artif Intell 32(2):197–213
58. Harman M, Jones BF (2001) Search-based software engineering. Inf Softw Technol 43(14):833–839
59. Harman M, Mansouri SA, Zhang Y (2012) Search-based software engineering: trends, techniques and applications. ACM Comput Surveys 45(1):1–61
60. Harman M, Jia Y, Zhang Y (2015) Achievements, open problems and challenges for search based software testing. In: 2015 IEEE 8th international conference on software testing, verification and validation (ICST), pp 1–12. IEEE

61. Hema Shankari K, Mathivilasini S, Arasu D, Suseendran G (2021) Genetic algorithm based on test suite prioritization for software testing in neural network. In: Proceedings of first international conference on mathematical modeling and computational science: ICMMCS 2020, pp. 409–416. Springer
62. Henard C, Papadakis M, Harman M, Le Traon Y (2015) Combining multi-objective search and constraint solving for configuring large software product lines. In: 2015 IEEE/ACM 37th IEEE international conference on software engineering, vol 1, pp 517–528. IEEE
63. Hierons RM, Li M, Liu X, Segura S, Zheng W (2016) SIP: optimal product selection from feature models using many-objective evolutionary optimization. ACM Trans Softw Eng Methodol 25(2):1–39
64. Horcas JM, Strüber D, Burdusel A, Martinez J, Zschaler S (2022) We're not gonna break it! consistency-preserving operators for efficient product line configuration. IEEE Trans Softw Eng 49(3):1102–1117
65. Hou Y, Ouyang D, Tian X, Zhang L (2022) Evolutionary many-objective satisfiability solver for configuring software product lines. Applied Intell 52:1–24
66. Huang J, Liu J (2016) A similarity-based modularization quality measure for software module clustering problems. Inf Sci 342:96–110
67. Huang J, Liu J, Yao X (2017) A multi-agent evolutionary algorithm for software module clustering problems. Soft Comput 21(12):3415–3428
68. Hwa J, Yoo S, Seo YS, Bae DH (2017) Search-based approaches for software module clustering based on multiple relationship factors. Int J Softw Eng Knowl Eng 27(07):1033–1062
69. Isazadeh A, Izadkhah H, Elgedawy I (2017) Source code modularization: theory and techniques. Springer, Berlin
70. ISO/IEC/IEEE 12207:2017 (2017) Systems and software engineering — Software life cycle processes. Tech. rep., International Organization for Standardization, Geneva, CH
71. ISO/IEC/IEEE 24765:2017 (2017) Systems and software engineering — Vocabulary. Tech. rep., International Organization for Standardization, Geneva, CH
72. ISO/IEC 25010:2011 (2011) Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models. Tech. rep., International Organization for Standardization, Geneva, CH
73. ISO/IEC/IEEE 29119-1:2022 (2022) software and systems engineering — software testing — part 1: General concepts. Tech. rep., International Organization for Standardization, Geneva, CH
74. Izadkhah H, Elgedawy I, Isazadeh A (2016) E-CDGM: an evolutionary call-dependency graph modularization approach for software systems. Cyber Inf Technol 16(3):70–90
75. Izadkhah H, Tajgardan M (2019) Information theoretic objective function for genetic software clustering. In: Multidisciplinary digital publishing institute proceedings. vol 46, p 18
76. Jahangirova G, Khomh F (2024) Search-Based Software Engineering: 16th International Symposium, SSBSE 2024, Porto de Galinhas, July 15, 2024, Proceedings, vol 14767. Springer Nature
77. Jalali NS, Izadkhah H, Lotfi S (2019) Multi-objective search-based software modularization: structural and non-structural features. Soft Comput 23(21):11141–11165
78. Jamil MA, Alhindi A, Arif M, Nour MK, Abubakar NSA, Aljabri TF (2019) Multiobjective evolutionary algorithms NSGA-II and NSGA-III for software product lines testing optimization. In: 2019 IEEE 6th international conference on engineering technologies and applied sciences (ICETAS), pp 1–5. IEEE
79. Jeet K, Dhir R (2016) Software module clustering using hybrid socio-evolutionary algorithms. Int J Inf Eng Electr Busin 8(4):43
80. Jia Y, Mao K, Harman M (2018) Finding and fixing software bugs automatically with SapFix and Sapienz. https://engineering.fb.com/2018/09/13/developer-tools/finding-and-fixing-software-bugs-automatically-with-sapfix-and-sapienz/. Accessed on September, 2024
81. Jones BF, Sthamer HH, Eyres DE (1996) Automatic structural testing using genetic algorithms. Softw Eng J 11(5):299–306

82. Jones BF, Eyres DE, Sthamer HH (1998) A strategy for using genetic algorithms to automate branch and fault-based testing. Comput J 41(2):98–107
83. Kang K, Cohen S, Hess J, Novak WE, Peterson AS, et al (1990) Feature-oriented domain analysis (FODA) feasibility study
84. Kargar M, Isazadeh A, Izadkhah H: Semantic-based software clustering using hill climbing. In: 2017 international symposium on computer science and software engineering conference (CSSE), pp 55–60. IEEE
85. Khari M, Kumar P (2019) An extensive evaluation of search-based software testing: a review. Soft Comput 23:1933–1946
86. Khatibsyarbini M, Isa MA, Jawawi DN, Hamed HNA, Suffian MDM (2019) Test case prioritization using firefly algorithm for software testing. IEEE Access 7:132360–132373
87. Khoshnevis S (2023) A search-based identification of variable microservices for enterprise SAAS. Front Comput Sci 17(3):173208
88. Khoshnevis S, Ardestani OH (2024) Search-based approaches to optimizing software product line architectures: a systematic literature review. Inf Softw Technol 170:107446
89. Khoshniat N, Jamarani A, Ahmadzadeh A, Haghi Kashani M, Mahdipour E (2024) Nature-inspired metaheuristic methods in software testing. Soft Comput 28(2):1503–1544
90. Kiper JD, Feather MS, Richardson J (2007) Optimizing the V&V process for critical systems. In: Proceedings of the 9th annual conference on genetic and evolutionary computation, pp 1139–1139
91. Koleejan C, Xue B, Zhang M (2015) Code coverage optimisation in genetic algorithms and particle swarm optimisation for automatic software test data generation. In: 2015 IEEE congress on evolutionary computation (CEC), pp 1204–1211. IEEE
92. Koza JR (1994) Genetic programming as a means for programming computers by natural selection. Stat Comput 4:87–112
93. Kumari AC, Srinivas K (2016) Hyper-heuristic approach for multi-objective software module clustering. J Syst Softw 117:384–401
94. Lehman MM (1996) Laws of software evolution revisited. In: European workshop on software process technology, pp 108–124. Springer
95. Lehman MM (1980) Programs, life cycles, and laws of software evolution. Proc IEEE 68(9):1060–1076
96. Li X, Wong WE, Gao R, Hu L, Hosono S (2018) Genetic algorithm-based test generation for software product line with the integration of fault localization techniques. Empir Softw Eng 23:1–51
97. Lian X, Zhang L (2015) Optimized feature selection towards functional and non-functional requirements in software product lines. In: 2015 IEEE 22nd international conference on software analysis, evolution, and reengineering (SANER), pp. 191–200. IEEE
98. Lian X, Zhang L, Jiang J, Goss W (2018) An approach for optimized feature selection in large-scale software product lines. J Syst Softw 137:636–651
99. Lientz BP, Swanson EB (1981) Problems in application software maintenance. Commun ACM 24(11):763–769
100. Lopez-Herrejon RE, Batory D (2001) A standard problem for evaluating product-line methodologies. In: International symposium on generative and component-based software engineering, pp 10–24. Springer
101. Lopez-Herrejon RE, Chicano F, Ferrer J, Egyed A, Alba E (2013) Multi-objective optimal test suite computation for software product line pairwise testing. In: 2013 IEEE international conference on software maintenance, pp 404–407. IEEE
102. Lopez-Herrejon RE, Linsbauer L, Egyed A (2015) A systematic mapping study of search-based software engineering for software product lines. Inf Softw Technol 61: 33–51
103. Mahajan S, Joshi SD, Khanaa V (2015) Component-based software system test case prioritization with genetic algorithm decoding technique using java platform. In: 2015 international conference on computing communication control and automation, pp 847–851. IEEE

104. Mahdavi K (2005) A clustering genetic algorithm for software modularisation with a multiple hill climbing approach. Ph.D. Thesis, Brunel University
105. Mahmoud T, Ahmed BS (2015) An efficient strategy for covering array construction with fuzzy logic-based adaptive swarm optimization for software testing use. Expert Syst Appl 42(22):8753–8765
106. Mamaghani AS, Hajizadeh M (2014) Software modularization using the modified firefly algorithm. In: 2014 8th. Malaysian software engineering conference (MySEC), pp 321–324. IEEE
107. Mamaghani AS, Meybodi MR (2009) Clustering of software systems using new hybrid algorithms. In: 2009 ninth IEEE international conference on computer and information technology, vol 1, pp 20–25. IEEE
108. Mancoridis S, Mitchell BS, Rorres C, Chen YF, Gansner ER (1998) Using automatic clustering to produce high-level system organizations of source code. In: 6th international workshop on program comprehension (IWPC'98), pp 45–52. IEEE
109. Mancoridis S, Mitchell BS, Chen Y, Gansner ER (1999) Bunch: a clustering tool for the recovery and maintenance of software system structures. In: Proceedings IEEE international conference on software maintenance-1999 (ICSM'99). 'Software Maintenance for Business Change' (Cat. No. 99CB36360), pp 50–59. IEEE
110. Manousakis K, Savva G, Papadouri N, Mavrovouniotis M, Christofides A, Kolokotroni N, Ellinas G (2024) A practical approach for resource-constrained project scheduling. IEEE Access 12:12976–12991
111. Mao K, Harman M, Jia Y (2016) Sapienz: multi-objective automated testing for android applications. In: Proceedings of the 25th international symposium on software testing and analysis, pp 94–105
112. Mariani T, Colanzi TE, Vergilio SR (2016) Preserving architectural styles in the search based design of software product line architectures. J Syst Softw 115:157–173
113. Matnei Filho RA, Vergilio SR (2015) A mutation and multi-objective test data generation approach for feature testing of software product lines. In: 2015 29th Brazilian symposium on software engineering, pp 21–30. IEEE
114. McMinn P (2004) Search-based software test data generation: a survey. Softw Test Verif Reliab 14(2):105–156
115. Mendonca M, Branco M, Cowan D (2009) SPLOT: software product lines online tools. In: Proceedings of the 24th ACM SIGPLAN conference companion on object oriented programming systems languages and applications, pp 761–762
116. Miller W, Spooner DL (1976) Automatic generation of floating-point test data. IEEE Trans Softw Eng 3:223–226
117. Minku LL, Sudholt D, Yao X (2013) Improved evolutionary algorithm design for the project scheduling problem based on runtime analysis. IEEE Trans Softw Eng 40(1):83–102
118. Mitchell BS (2002) A heuristic search approach to solving the software clustering problem. Ph.D. Thesis, Drexel University, aAI3039424
119. Mitchell BS, Mancoridis S (2002) Using heuristic search techniques to extract design abstractions from source code. In: Proceedings of the 4th annual conference on genetic and evolutionary computation, pp 1375–1382
120. Mitchell BS, Mancoridis S (2006) On the automatic modularization of software systems using the bunch tool. IEEE Trans Softw Eng 32(3):193–208
121. Mitchell BS, Mancoridis S (2008) On the evaluation of the Bunch search-based software modularization algorithm. Soft Comput 12(1):77–93
122. Mitchell B, Traverso M, Mancoridis S (2001) An architecture for distributing the computation of software clustering algorithms. In: Proceedings working IEEE/IFIP conference on software architecture, pp 181–190. IEEE
123. Mkaouer W, Kessentini M, Shaout A, Koligheu P, Bechikh S, Deb K, Ouni A (2015) Many-objective software remodularization using NSGA-III. ACM Trans Softw Eng Methodol 24(3):1–45
124. Mohan M, Greer D (2018) A survey of search-based refactoring for software maintenance. J Softw Eng Res Develop 6(1):1–52

125. Molnar AJ, Motogna S (2021) A study of maintainability in evolving open-source software. In: Evaluation of novel approaches to software engineering: 15th international conference, ENASE 2020, Prague, May 5–6, 2020, Revised selected papers 15, pp 261–282. Springer
126. Monçores MC, Alvim ACF, Barros MO (2018) Large neighborhood search applied to the software module clustering problem. Comput Oper Res 91:92–111
127. Mu L, Sugumaran V, Wang F (2020) A hybrid genetic algorithm for software architecture re-modularization. Inf Syst Front 22(5):1133–1161
128. Nigar N, Shahzad MK, Islam S, Oki O, Lukose JM (2023) Multi-objective dynamic software project scheduling: a novel approach to handle employee's addition. IEEE Access 11:39792–39806
129. Olaechea R, Rayside D, Guo J, Czarnecki K (2014) Comparison of exact and approximate multi-objective optimization for software product lines. In: Proceedings of the 18th international software product line conference-volume 1, pp 92–101
130. Oliveira Jr E (2023) UML-based software product line engineering with smarty. Springer Nature, Berlin
131. Olsthoorn M, van Deursen A, Panichella A (2022) Guiding automated test case generation for transaction-reverting statements in smart contracts. In: 2022 IEEE international conference on software maintenance and evolution (ICSME), pp 163–174. IEEE
132. Olsthoorn M, Stallenberg D, Van Deursen A, Panichella A (2022) SynTest-Solidity: automated test case generation and fuzzing for smart contracts. In: Proceedings of the ACM/IEEE 44th international conference on software engineering: companion proceedings, pp. 202–206 (2022)
133. Pacheco C, Ernst MD (2007) Randoop: feedback-directed random testing for java. In: Companion to the 22nd ACM SIGPLAN conference on object-oriented programming systems and applications companion, pp 815–816
134. Panichella A, Olsthoorn M (2024) Higher fault detection through novel density estimators in unit test generation. In: International symposium on search based software engineering, pp 18–32. Springer
135. Panichella A, Kifetew FM, Tonella P (2017) Automated test case generation as a many-objective optimisation problem with dynamic selection of the targets. IEEE Trans Softw Eng 44(2):122–158
136. Papadakis M, Kintis M, Zhang J, Jia Y, Le Traon Y, Harman M (2019) Mutation testing advances: an analysis and survey. In: Advances in computers, vol 112, pp 275–378. Elsevier
137. Pargas RP, Harrold MJ, Peck RR (1999) Test-data generation using genetic algorithms. Softw Test Verif Reliab 9(4):263–282
138. Pascual GG, Lopez-Herrejon RE, Pinto M, Fuentes L, Egyed A (2015) Applying multiobjective evolutionary algorithms to dynamic software product lines for reconfiguring mobile applications. J Syst Softw 103:392–411
139. Pinto AF, de Faria Alvim AC, de Oliveira Barros M (2014) ILS for the Software Module Clustering Problem. In: XLVI Simpósio Brasileiro de Pesquisa Operacional. Salvador:[sn], pp 1972–1983
140. Pohl K, Böckle G, Linden F (2005)Software product line engineering: foundations, principles, and techniques. Springer, Berlin
141. Pourasghar B, Izadkhah H, Isazadeh A, Lotfi S (2021) A graph-based clustering algorithm for software systems modularization. Inf Softw Technol 133:106469
142. Praditwong K (2011) Solving software module clustering problem by evolutionary algorithms. In: 2011 eighth international joint conference on computer science and software engineering (JCSSE), pp 154–159. IEEE
143. Praditwong K, Harman M, Yao X (2011) Software module clustering as a multi-objective search problem. IEEE Trans Softw Eng 37(2):264–282
144. Prajapati A (2022) Software module clustering using grid-based large-scale many-objective particle swarm optimization. Soft Comput 26:1–22
145. Prajapati A, Chhabra JK (2018) A particle swarm optimization-based heuristic for software module clustering problem. Arab J Sci Eng 43(12):7083–7094

146. Rahimi I, Gandomi AH, Nikoo MR, Chen F (2023) A comparative study on evolutionary multi-objective algorithms for next release problem. Appl Soft Comput 144:110472
147. Räihä O (2010) A survey on search-based software design. Comput Sci Rev 4(4):203–249
148. Ramírez A, Parejo JA, Romero JR, Segura S, Ruiz-Cortés A (2017) Evolutionary composition of QoS-aware web services: a many-objective perspective. Expert Syst Appl 72:357–370
149. Ramirez A, Romero JR, Ventura S (2018) Interactive multi-objective evolutionary optimization of software architectures. Inf Sci 463:92–109
150. Ramirez A, Romero JR, Ventura S (2019) A survey of many-objective optimisation in search-based software engineering. J Syst Softw 149:382–395
151. Rezende AV, Silva L, Britto A, Amaral R (2019) Software project scheduling problem in the context of search-based software engineering: a systematic review. J Syst Softw 155:43–56
152. Romero JR, Medina-Bulo I, Chicano F (2023) Optimising the software development process with artificial intelligence. Springer, Berlin
153. Saber T, Brevet D, Botterweck G, Ventresque A (2018) Is seeding a good strategy in multi-objective feature selection when feature models evolve? Inf Softw Technol 95:266–280
154. Sahin O, Akay B (2016) Comparisons of metaheuristic algorithms and fitness functions on software test data generation. Appl Soft Comput 49:1202–1214
155. Sapozhnikov A, Olsthoorn M, Panichella A, Kovalenko V, Derakhshanfar P (2024) TestSpark: IntelliJ idea's ultimate test generation companion. In: Proceedings of the 2024 IEEE/ACM 46th international conference on software engineering: companion proceedings, pp 30–34
156. Sayyad AS, Ingram J, Menzies T, Ammar H (2013) Scalable product line configuration: a straw to break the camel's back. In: 2013 28th IEEE/ACM international conference on automated software engineering (ASE), pp 465–474. IEEE
157. Sayyari F, Emadi S (2015) Automated generation of software testing path based on ant colony. In: 2015 International congress on technology, communication and knowledge (ICTCK), pp 435–440. IEEE(2015)
158. Sharma R, Saha A (2020) Identification of critical test paths using firefly algorithm for object oriented software. J Interdiscipl Math 23(1):191–203
159. Sharma S, Rizvi S, Sharma V (2019) A framework for optimization of software test cases generation using cuckoo search algorithm. In: 2019 9th international conference on cloud computing, data science & engineering (confluence), pp 282–286. IEEE
160. Shen X, Minku LL, Bahsoon R, Yao X (2015) Dynamic software project scheduling through a proactive-rescheduling method. IEEE Trans Softw Eng 42(7):658–686
161. Shen XN, Minku LL, Marturi N, Guo YN, Han Y (2018) A q-learning-based memetic algorithm for multi-objective dynamic software project scheduling. Inf Sci 428:1–29
162. Shen X, Guo Y, Li A (2020) Cooperative coevolution with an improved resource allocation for large-scale multi-objective software project scheduling. Appl Soft Comput 88:106059
163. Shomali N, Arasteh B (2020) Mutation reduction in software mutation testing using firefly optimization algorithm. Data Technol Appl 54(4):461–480
164. Silva GFD, Silva L, Britto A (2020) Dynamic software project scheduling problem with PSO and dynamic strategies based on memory. In: Intelligent systems: 9th Brazilian conference, BRACIS 2020, Rio Grande, October 20–23, 2020, proceedings, part I 9, pp 79–94. Springer
165. Silva DFD, Okada LF, Colanzi TE, Assunção WK (2020) Enhancing search-based product line design with crossover operators. In: Proceedings of the 2020 genetic and evolutionary computation conference, pp 1250–1258
166. Singhal A, Bansal A, Kumar A (2021) Meta-heuristic algorithm to generate optimised test cases for aspect-oriented software systems. Int J Adv Intell Parad 18(2):134–153
167. Tajgardan M, Izadkhah H, Lotfi S (2016) Software systems clustering using estimation of distribution approach. J Appl Comput Sci Methods 8:99–113
168. Tracey N, Clark J, Mander K (1998) Automated program flaw finding using simulated annealing. In: Proceedings of the 1998 ACM SIGSOFT international symposium on software testing and analysis, pp 73–81
169. Van der Linden FJ, Schmid K, Rommes E (2007) Software product lines in action: the best industrial practice in product line engineering. Springer Science & Business Media, Cham

170. Vega-Velázquez MÁ, García-Nájera A, Cervantes H (2018) A survey on the software project scheduling problem. Int J Prod Econ 202:145–161
171. Wang J, Huang Y, Chen C, Liu Z, Wang S, Wang Q (2024) Software testing with large language models: survey, landscape, and vision. IEEE Trans Softw Eng 50:911–936
172. Wegener J, Grimm K, Grochtmann M, Sthamer H, Jones B (1996) Systematic testing of real-time systems. In: 4th international conference on software testing analysis and review (EuroSTAR 96)
173. White J, Dougherty B, Schmidt DC (2009) Selecting highly optimal architectural feature sets with filtered cartesian flattening. J Syst Softw 82(8):1268–1284
174. Wong WY, Yu SW, Too CW (2018) A systematic approach to software quality assurance: the relationship of project activities within project life cycle and system development life cycle. In: 2018 IEEE conference on systems, process and control (ICSPC), pp 123–128. IEEE
175. Wu B, Yun L, Jin X, Liu B, Wei G (2016) Study on the fuzzing test method for industrial supervisory control configuration software based on genetic algorithm. In: 2016 11th international conference on reliability, maintainability and safety (ICRMS), pp 1–6. IEEE
176. Xiang Y, Zhou Y, Zheng Z, Li M (2018) Configuring software product lines by combining many-objective optimization and sat solvers. ACM Trans Softw Eng Methodol 26(4):1–46
177. Xiang Y, Yang X, Zhou Y, Zheng Z, Li M, Huang H (2020) Going deeper with optimal software products selection using many-objective optimization and satisfiability solvers. Empir Softw Eng 25:591–626
178. Xiang Y, Peng X, Xia X, Meng X, Li S, Huang H (2021) An investigation of decomposition-based metaheuristics for resource-constrained multi-objective feature selection in software product lines. In: Evolutionary multi-criterion optimization: 11th international conference, EMO 2021, Shenzhen, March 28–31, 2021, proceedings 11, pp 659–671. Springer
179. Xiao J, Osterweil LJ, Wang Q, Li M (2010) Dynamic resource scheduling in disruption-prone software development environments. In: Fundamental approaches to software engineering: 13th international conference, FASE 2010, held as part of the joint european conferences on theory and practice of software, ETAPS 2010, Paphos, March 20–28, 2010. Proceedings 13, pp 107–122. Springer
180. Xiao J, Osterweil LJ, Chen J, Wang Q, Li M (2013) Search based risk mitigation planning in project portfolio management. In: Proceedings of the 2013 international conference on software and system process, pp 146–155
181. Xue Y, Zhong J, Tan TH, Liu Y, Cai W, Chen M, Sun J (2016) IBED: combining IBEA and DE for optimal feature selection in software product line engineering. Appl Soft Comput 49:1215–1231
182. Yuste J, Duarte A, Pardo EG (2022) An efficient heuristic algorithm for software module clustering optimization. J Syst Softwe 190:111349
183. Yuste J, Pardo EG, Duarte A (2022) Multi-objective variable neighborhood search for improving software modularity. In: International conference on variable neighborhood search, pp 58–68. Springer
184. Yuste J, Pardo EG, Duarte A (2022) Variable neighborhood descent for software quality optimization. In: Metaheuristics international conference, pp 531–536. Springer
185. Yuste J, Pardo EG, Duarte A (2024) General variable neighborhood search for the optimization of software quality. Comput Oper Res 165:106584
186. Yuste J, Pardo EG, Duarte A, Hao JK (2024) Multi-objective general variable neighborhood search for software maintainability optimization. Eng Appl Artif Intell 133:108593
187. Zhang Y, Harman M, Mansouri SA (2007) The multi-objective next release problem. In: Proceedings of the 9th annual conference on genetic and evolutionary computation, pp 1129–1137
188. Zhu E, Yao C, Ma Z, Liu F (2017) Study of an improved genetic algorithm for multiple paths automatic software test case generation. In: Advances in swarm intelligence: 8th international conference, ICSI 2017, Fukuoka, July 27–August 1, 2017, proceedings, part I 8, pp 402–408. Springer

Selected String Problems

54

Christian Blum and Paola Festa

Contents

Introduction to String Problems 1652
Creating Diagnostic Probes for Bacterial Infections 1652
Primer Design 1653
Discovering Potential Drug Targets 1653
Motif Search 1653
Notation 1654
The Closest String Problem (CSP) 1654
The Close to Most String Problem (CTMSP) 1659
The Farthest String Problem (FSP) 1659
The Far from Most String Problem (FFMSP) 1660
A Simple ILP-Based Heuristic 1663
Experimental Evaluation 1663
Conclusions 1666
Cross-References 1670
References 1670

Abstract

This chapter overviews some string selection and comparison problems, with special emphasis on the optimization and operational research perspective. It also proposes a simple and efficient ILP-based heuristic that can be used for any of the considered problems.

C. Blum (✉)
Artificial Intelligence Research Institute (IIIA-CSIC), Bellaterra, Spain
e-mail: christian.blum@iiia.csic.es

P. Festa
Department of Mathematics and Applications "R. Caccioppoli", University of Napoli FEDERICO II, Naples, Italy
e-mail: paola.festa@unina.it

© Springer Nature Switzerland AG 2025

R. Martí et al. (eds.), *Handbook of Heuristics*,
https://doi.org/10.1007/978-3-032-00385-0_58

Keywords

String selection · String comparison · Optimization problem · Integer linear programming · Heuristic

Introduction to String Problems

Among string selection and comparison problems, there is a class of problems known as *string consensus*, where a finite set of strings is given and one is interested in finding their consensus, i.e., a new string that agrees as much as possible with all the given strings. In other words, the objective is to determine a string, called consensus, because it represents—in some way—all the given strings. The idea of representation and of being in consensus can be related to several different objectives listed in the following:

i. The consensus is a new string whose total distance from all given strings is minimum (*closest string problem*).
ii. The consensus is a new string close to most of the given strings (*close to most string problem*).
iii. The consensus is a new string whose total distance from all given strings is maximum (*farthest string problem*).
iv. The consensus is a new string far from most of the given strings (*far from most string problem*).

Computational intractability of the general string consensus problem was first proved in 1997 by Frances and Litman [10] and in 1999 by Sim and Park [34].

As a constring of the linear coding of DNA and proteins, many molecular biology problems have been formulated as computational optimization problems involving strings and strings. Biological applications of computing distance/proximity among strings occur mainly in two varieties. Some require that a region of similarity be discovered, while other applications use the reverse complement of the region, such as designing probes or primers. In the following, some relevant biological applications are outlined.

Creating Diagnostic Probes for Bacterial Infections

Probes are strands of either DNA or RNA that have been modified (i.e., made either radioactive or fluorescent) so that their presence can be easily detected. One possible application of string problems arises in creating diagnostic probes for bacterial infections [18, 25]. In this scenario, given a set of DNA strings from a group of closely related pathogenic bacteria, the task is to find a substring that occurs in each of the bacterial strings (as close as possible) without occurring in the host's DNA.

Probes are then designed to hybridize to these target strings, so that the detection of their presence indicates that at least one bacterial species is likely to be present in the host.

Primer Design

Primers are short strings of nucleotides designed such that they hybridize to a given DNA string or to all of a given set of DNA strings with the aim of providing a starting point for DNA strand synthesis by polymerase chain reaction (PCR). The hybridization of primers depends on several conditions, including some thermodynamic rules, but it is largely influenced by the number of mismatching positions among the given strings, and this number should be as small as possible [12, 13, 21].

Discovering Potential Drug Targets

Another biological application of string selection and comparison problems is related to discovering potential drug targets. Given a set of strings of orthologous genes from a group of closely related pathogens and a host (such as a human, crop, or livestock), the goal is to find a string fragment that is more conserved in all or most of the pathogens' strings but not as conserved in the host. The information encoded by this fragment can then be used for novel antibiotic development or to create a drug that harms several pathogens with minimal effect on the host. All these applications reduce to the task of finding a pattern that, with some error, occurs in one set of strings (closest string problem) and/or does not occur in another set (farthest string problem). The far from most string problem can help to identify a string fragment that distinguishes the pathogens from the host, so the potential exists to create a drug that harms several but not all pathogens [12, 13, 21].

Motif Search

A motif is a string that occurs *approximately preserved* as a substring in some/-several of the DNA strings of a given set. Approximately preserved means that the motif occurs with changes in at most t positions for a fixed nonnegative integer t. The importance of a motif lies in its characteristic of being a candidate for substrings of noncoding parts of the DNA string that have functions related to, e.g., gene expression [12, 13].

For most consensus problems, Hamming distance is used instead of other alternative measures, such as, for example, the editing distance. The biological reasons justifying this choice are very well described and motivated by Lanctot et al. in [18–20] and can be summarized claiming that the "edit distance is more suitable

to measure the *amount* of change that has happened, whereas the Hamming distance is more suitable to measure the *effect* of that change."

The remainder of this chapter is organized as follows. The next section lists notation and definitions used throughout the chapter. The following four sections are devoted to the closest string, the close to most string, the farthest string, and the far from most string problem, respectively. All these problems are mathematically formulated and their properties are analyzed. The most popular solution techniques for them are surveyed, along with the computational results obtained and analyzed in the literature. The second last section reports computational results that demonstrate empirically the efficiency of the state-of-the-art algorithms. Concluding remarks and future directions are discussed in the last section.

Notation

Throughout this chapter, the following notation and definitions will be used.

- An *alphabet* $\Sigma = \{c_1, c_2, \ldots, c_k\}$ is a finite set of elements, called *characters*.
- $s^i = (s^i_1, s^i_2, \ldots, s^i_m)$ denotes a string of m characters (that is, of length m) over alphabet Σ, i.e., $s^i_j \in \Sigma,\ j = 1, 2, \ldots, m$.
- Given two strings s^i and s^l on Σ such that $|s^i| = |s^l|$, $d_H(s^i, s^l)$ denotes their Hamming distance and is given by

$$d_H(s^i, s^l) = \sum_{j=1}^{|s^i|} \Phi(s^i_j, s^l_j), \tag{1}$$

  where s^i_j and s^l_j denote the character at position j in string s^i and in string s^l, respectively, and $\Phi : \Sigma \times \Sigma \to \{0, 1\}$ is a predicate function such that

$$\Phi(a, b) = \begin{cases} 0, \text{ if } a = b; \\ 1, \text{ otherwise.} \end{cases}$$

- For all consensus problems, each string s of length m over Σ is a valid solution.

The Closest String Problem (CSP)

Given a finite set of strings Ω on Σ, the problem is to find a *center string* $s^* \in \Sigma^m$ such that the Hamming distance between s^* and all strings in Ω is minimal; in other words, s^* is a string to which a minimal value d corresponds such that

$$d_H(s^*, s^i) \leq d, \quad \forall\, s^i \in \Omega.$$

The closest string problem can be formulated as an integer linear program (ILP). In fact, let $\Sigma_k \subseteq \Sigma$ be the set of characters appearing at position k in any of the strings from Ω.

For each $k = 1, 2, \ldots, m$ and $j \in V_k$, let us define the following binary variables:

$$x_{ck} = \begin{cases} 1, & \text{if character } c \in \Sigma_k \text{ is used at position } k \text{ of the solution;} \\ 0, & \text{otherwise.} \end{cases}$$

Then, the CSP admits the following ILP formulation:

$$\min d \tag{2}$$

subject to:

$$\sum_{c \in \Sigma_k} x_{ck} = 1 \quad \text{for } k = 1, 2 \ldots, m \tag{3}$$

$$m - \sum_{k=1}^{m} x_{s_k^i k} \leq d \quad \text{for } i = 1, 2 \ldots, n \tag{4}$$

$$d \in \mathbb{N}^+, \tag{5}$$

$$x_{ck} \in \{0, 1\}, \qquad \text{for } k = 1, 2, \ldots, m, \ \forall\, c \in \Sigma_k. \tag{6}$$

Equalities (3) guarantee that only one character is selected for each position $k \in \{1, 2, \ldots, m\}$. Inequalities (4) impose that, if a character in a string s^i is not in the solution defined by the x-variables, then that character will contribute to increasing the Hamming distance from solution x to s^i. Finally, (5) forces d to assume a nonnegative integer value and (6) define the decision variables.

This problem was first studied in the area of coding theory [33] and has been independently proved computationally intractable in [10] and in [19, 20].

In 2004, Meneses et al. [31] used a linear relaxation of the above-described mathematical model to design a branch and bound algorithm. At each iteration, the next node in the branching tree to be explored is the one with the smallest linear relaxation objective function value (also known as the *best bound first strategy*). Once the next node is selected and an optimal fractional solution $x'_{ck} \in [0, 1]$ obtained, $k = 1, \ldots, m$, $\forall\, c \in \Sigma_k$, for the linear relaxation of the corresponding subproblem, the algorithm branches on the fractional variable x_{ck} with maximum value of x'_{ck}. The bounding phase is very important in any branch and bound algorithm: the better the computed bound, the smaller the number of nodes that need to be explored and that can therefore be pruned. For the bounding phase, Meneses et al. computed an initial bound selecting one of the given input strings and modifying it until a local optimal solution is found. The authors have empirically shown that their branch and bound algorithm is able to solve in reasonable running times small-size instances with 10–30 strings each of which is 300–800 characters long.

Further exact methods proposed for the CSP are fixed-parameter algorithms [14, 24, 36] that are applicable only when the maximum Hamming distance among all pairs of strings is small. In fact, since these algorithms are designed to solve the decision version of the problem, it is necessary to apply them multiple times in order to find an optimal solution that minimizes the Hamming distance.

The first approximation algorithm for the CSP was proposed in [20] with a worst-case performance ratio of 2. It is a simple algorithm that constructs an approximate feasible solution in a purely random fashion. Starting from an empty solution, the algorithm randomly selects the next element to be added to the solution under construction.

Better performance approximation algorithms proposed in the literature are based on the linear programming relaxation of the previously described ILP model. The basic idea consists of solving the linear programming relaxation of the ILP model and using the result of the relaxed problem to find an approximate solution to the original problem. Following this line, [20] also proposed a $\frac{4}{3}(1+\epsilon)$-approximation algorithm (for any small $\epsilon > 0$) that uses the randomized rounding technique for obtaining an integer 0-1 solution from the continuous solution for the relaxed problem. The randomized rounding technique works by defining the value of a Boolean variable $x \in \{0, 1\}$ to be $x = 1$ with a certain probability y, where y is the value of the continuous variable corresponding to x in the relaxation of the original integer programming problem. In 1999, [21] used the rounding idea to design a polynomial time approximation scheme (PTAS). A PTAS is a special type of approximation algorithm that, for each $\epsilon > 1$, yields a performance guarantee of ϵ in polynomial time. Thus, this can be viewed as a way of getting solutions with guaranteed performance for any desired threshold greater than one. The PTAS proposed in [21] is also based on randomized rounding, which is refined to check results for a large (but polynomially bounded) number of subsets of indices. However, since many iterations involve the solution of a linear relaxation of an ILP model, the algorithm becomes impractical for any instance with large strings. To efficiently deal with real-world scenarios and/or medium- to large-size problem instances, several heuristic and metaheuristic algorithms have been proposed in the last few years.

In 2005, [22] designed a genetic algorithm and a simulated annealing algorithm, both in their sequential and their parallel versions. Genetic algorithms (GAs) are population-based metaheuristics that have been applied to find optimal or near-optimal solutions to combinatorial optimization problems [11, 16]. They implement the concept of *survival of the fittest*, making an analogy between a solution and an *individual* in a *population*. Each individual has a corresponding *chromosome* that encodes the solution. A chromosome consists of a string of *genes*. Each gene can take on a value, called an *allele*, from some alphabet. A chromosome has an associated *fitness level* which is correlated to the corresponding objective function value of the solution it encodes. Over a number of iterations, called *generations*, GAs evolve a population of chromosomes. This is usually accomplished by simulating the process of natural selection through mating and mutation. For the CSP, starting from an initial randomly generated population, at each generation $0 \leq t \leq \texttt{number} - \texttt{generations}$ of the Liu et al.'s GA, a population $P(t)$ of

popsize strings of length m is evolved and the fitness function to be maximized is defined as the difference $m - d_{\max}$, where $d_{\max}$ is the largest Hamming distance between an individual of the population $P(t)$ and any string in Ω. The production of offspring in a GA is done through the process of mating or crossover and Liu et al. used a multipoint crossover (MPX). In more detail, at a generic generation t, two parental individuals x and y in $P(t)$ are randomly chosen according to a probability that is proportional to their fitness. Then, iteratively until the offspring is not complete, x and y exchange parts between two randomly picked points. The resulting offspring have a new order of the strings, one part from the *first parent* and the other part from the *second parent*. Afterward, a mutation of any individual in the current population $P(t)$ is executed with some given probability. During this phase, two positions are randomly chosen and exchanged in the individual.

In their paper, Liu et al. proposed also a simulated annealing (SA) algorithm for the CSP. Originally proposed in [17], in the optimization and computer science research communities, simulated annealing is commonly said to be the "oldest" among the metaheuristics and surely one of the first techniques that had an explicit strategy to escape from local minima. Its fundamental idea is to allow moves resulting in solutions of worse quality in terms of objective function value than the current solution (uphill moves) in order to escape from local minima. The origin of simulated annealing and the choice of the acceptance criterion of a better quality solution lies in the physical annealing process, which can be modeled by methods based on Monte Carlo techniques. One of the early Monte Carlo techniques for simulating the evolution of a solid in a heat bath to thermal equilibrium is due to [28], who in 1953 designed a method that iteratively (until a stopping criterion is met) generates a string of states of the solid in the following way. At a generic iteration k, given a current state i of the solid (i.e., a current solution x):

- E_i is the energy of the solid in state i (objective function value $f(x)$).
- A subsequent state j (solution $\overline{x}$) is generated with energy E_j (objective function value $f(\overline{x})$) by applying a *perturbation mechanism* such as displacement of a single particle ($\overline{x}$ is a solution "close" to x).
- If $E_j - E_i < 0$ (i.e., $\overline{x}$ is a better quality solution), j ($\overline{x}$) is accepted; otherwise, j ($\overline{x}$) is accepted with a probability given by

$$\exp\left(-\frac{E_j - E_i}{k_B T_k}\right) \qquad \left[\exp\left(-\frac{f(\overline{x}) - f(x)}{k_B \cdot T_k}\right)\right],$$

  where T_k is the heat bath temperature and k_B is the Boltzmann constant.

As the number of performed iterations increases, the current temperature T_k is decreased resulting in a smaller probability of accepting not improving solutions. For the CSP, Liu et al.'s SA sets the initial temperature T_0 to $\frac{m}{2}$. The current temperature T_k is reduced every 100 iterations according to the "geometric cooling schedule," that is, $T_{k+100} = \gamma \cdot T_k$, where $\gamma = 0.9$. The stopping criterion is to reach a current temperature less than or equal to 0.001. Despite the interesting ideas

proposed in [22], the experimental analysis involves only small instances with up to 40 strings of length 40.

For the special case of $|\Omega| = 3$ and $|\Sigma| = 2$, [23] designed an exact approach called distance first algorithm (DFA), whose basic idea is to let the Hamming distance $d_H(s^*, s^i)$, $i = 1, 2, 3$, be as close as possible. The algorithm decreases the distance between the string that is farthest from the other two strings and solution s^* while increasing the distance between the string that is closest to the other two strings and solution s^*. For the general case, the authors proposed a polynomial-time heuristic resulting from a combination of local search strategies inspired by [31] and an approximation algorithm called largest distance decreasing algorithm (LDDA), which is based on similar ideas as DFA.

More recently, [35] proposed a novel heuristic (TA) based on the Lagrangian relaxation of the ILP model of the problem that allows the decomposition of the problem into subproblems, each corresponding to a position of the strings. The proposed algorithm combines a Lagrangian multiplier adjustment procedure to obtain feasibility and a tabu search as a local improvement procedure. In [6], Della Croce and Salassa described three relaxation-based procedures. One procedure (RA) rounds up the result of continuous relaxation, while the other two approaches (BCPA and ECPA) fix a subset of the integer variables in the continuous solution at the current value and let the solver run on the remaining (sub)problem. The authors also observed that all relaxation-based algorithms have been tested on *rectangular instances*, i.e., with $n \ll m$, and that the instances such that $n \geq m$ are harder to solve due to the higher number of constraints imposed by the strings, which enlarges the portion of non-integer components in the continuous solution of the problem. In an attempt to overcome this drawback, [5] designed a multi-start relaxation-based algorithm (called the selective fixing algorithm) that for a predetermined number of iterations takes a feasible solution as input and iteratively selects variables to be fixed at their initial value until the number of free variables is small enough that the remaining subproblem can be efficiently solved to optimality by an ILP solver. The new solution found by the solver can then be used as the initial solution for the next iteration. The authors have experimentally shown that their algorithm is much more robust compared to the state-of-the-art competitors and can solve a wider set of instances of different types, including those with $n \geq m$.

Xu and Perkins [37] proposed a WFC-CSP, a novel heuristic algorithm inspired by wave function collapse (WFC) techniques to solve the CSP. WFC is a constraint-based algorithm that was developed by game developer Maxim Gumin in [15] for procedural content generation. As summarized by [30], "the key idea is an extension of standard constraint solvers with a minimal entropy heuristic that randomly directs the solver's search in a way that follows a user-specified frequency distribution without compromising the efficiency of the search procedure." In case that a conflict is reached, Gumin's algorithm globally restarts instead of backtracking locally. Experimental results conducted by [37] show that WFC-CSP efficiently solved small-sized and large-sized CSP instances across different configurations.

The Close to Most String Problem (CTMSP)

The closest string problem can be seen as a special case of the so-called close to most string problem (CTMSP), which consists in determining a string close to most of the strings in the input set Ω. This can be formalized by saying that, given a threshold t, a string s^* must be found maximizing the variable l such that

$$d_H(s^*, s^i) \leq t, \text{ for } s^i \in P \subseteq \Sigma \text{ and } |P| = l.$$

$$\textbf{max} \sum_{i=1}^{n} y_i \tag{7}$$

subject to:

$$\sum_{c \in \Sigma} x_{ck} = 1 \quad \text{for } k = 1, \ldots, m \tag{8}$$

$$\sum_{k=1}^{m} x_{s_k^i k} \geq m \cdot y_i - t \quad \text{for } i = 1, \ldots, n \tag{9}$$

$$x_{ck}, y_i \in \{0, 1\}$$

Constraints (8) ensure that for each position k of a possible solution exactly one character from Σ_k is chosen. Constraints (9) ensure that y_i can only be set to 1 if and only if the number of differences between $s^i \in \Omega$ and the possible solution (as defined by the setting of the variables x_{ck}) is less or equal than t. Remember, in this context, that s_k^i denotes the character at position k in $s^i \in \Omega$.

Despite its similarity with the CSP, the CTMSP has not been widely studied. In [4] it was proved that this problem has no polynomial-time approximation scheme (**PTAS**) unless **NP** has randomized polynomial-time algorithms.

The Farthest String Problem (FSP)

Given a finite set of strings Ω over alphabet Σ, a problem complementary to the CSP is that of finding a string $s^* \in \Sigma^m$ farthest from the strings in Ω. This type of problem can be useful in situations such as finding a genetic string that cannot be associated with a given number of species.

Like the CSP, the farthest string problem can be formulated mathematically in the form of an ILP, where both decision variables and constraints have an interpretation which is contrary to the one in the CSP:

$$\max d \tag{10}$$

subject to:

$$\sum_{c \in \Sigma_k} x_{ck} = 1 \quad \text{for } k = 1, 2 \ldots, m \tag{11}$$

$$m - \sum_{k=1}^{m} x_{s_k^i k} \geq d \quad \text{for } i = 1, 2 \ldots, n \tag{12}$$

$$d \in \mathbb{N}^+, \tag{13}$$

$$x_{ck} \in \{0, 1\}, \qquad \text{for } k = 1, 2, \ldots, m, \ \forall\, c \in \Sigma_k. \tag{14}$$

The computational intractability of the FSP was demonstrated in [20], where it was proved that the problem remains intractable even for the simplest case where the alphabet has only two characters.

Despite its inherent computational intractability, it has be shown in [20] that there is a PTAS for the FSP. The algorithm is based on the randomized rounding of the relaxed solution of the above-reported ILP and uses the randomized rounding technique together with probabilistic inequalities to determine the maximum error possible in the solution computed by the algorithm. Note that the mathematical formulation of FSP is quite similar to the one used for the CSP, with only a change in the optimization objective and the inequality sign in the constraint

$$m - \sum_{j=1}^{m} x_{s_j^i j} \geq d, \qquad i = 1, 2, \ldots, n.$$

Thus, to solve the problem using an ILP formulation, one can use similar techniques to those employed for solving the CSP. In 2011 [38] and more recently in 2015 [39] proposed a few integer programming models for some variants of the farthest string problem and the closest string problem. The number of variables and constraints is substantially less compared with state-of-the-art integer linear programming models and the solution of the linear programming relaxation contains only a small proportion of noninteger values, which considerably simplifies both a subsequent rounding process and a branch and bound procedure.

The Far from Most String Problem (FFMSP)

A problem closely related to the farthest string problem is the far from most string problem (FFMSP). It consists in determining a string far from most of the strings in the input set Ω. This can be formalized by saying that, given a threshold t, a string s^* must be found maximizing the variable l such that

$$d_H(s^*, s^i) \geq t, \text{ for } s^i \in P \subseteq \Sigma \text{ and } |P| = l.$$

In [1], the FFMSP has been mathematically formulated as an ILP. In fact, by defining a Boolean variable x_{ck} for each position k ($k = 1, 2 \ldots, m$) of a possible solution and for each character $c \in \Sigma_k$ and a Boolean variable y_i ($i = 1, 2 \ldots, n$) for each of the n input strings provided in set Ω, the FFMSP can stated as the following ILP:

$$\textbf{max} \sum_{i=1}^{n} y_i \tag{15}$$

subject to:

$$\sum_{c \in \Sigma_k} x_{ck} = 1 \quad \text{for } k = 1, 2 \ldots, m \tag{16}$$

$$\sum_{k=1}^{m} x_{s_k^i k} \leq m - t \cdot y_i \quad \text{for } i = 1, 2 \ldots, n \tag{17}$$

$$x_{ck}, y_i \in \{0, 1\}$$

Constraints (16) ensure that for each position k of a possible solution exactly one character from Σ_k is chosen. Constraints (17) ensure that y_i can only be set to 1 if and only if the number of differences between $s^i \in \Omega$ and the possible solution (as defined by the setting of the variables x_{ck}) is greater or equal than t. Remember, in this context, that s_k^i denotes the character at position k in $s^i \in \Omega$.

Despite the similarity, it can be shown [20] that the FFMSP is much harder to approximate than the FSP, due to the approximation preserving reduction to FFMSP from the independent set problem, a classical and computationally intractable combinatorial optimization problem. In particular, [20] demonstrated that for strings over an alphabet Σ with $|\Sigma| \geq 3$, approximating the FFMSP within a polynomial factor is **NP**-hard.

The first attempt in the direction of the design of heuristic methods to efficiently solve the FFMSP was done in [26, 27], who proposed a heuristic algorithm consisting of a simple greedy construction followed by an iterative improvement phase. Later, [9] designed a simple GRASP, recently improved in [29]. Mousavi et al. noticed that the search landscape of the FFMSP is characterized by many solutions having the same objective value. Consequently, local search is likely to visit many sub-optimal local maxima. To efficiently escape from these local maxima, Mousavi et al. devised a new hybrid heuristic evaluation function and used it in conjunction with the objective function when evaluating neighbor solutions during the local search phase in the GRASP framework.

Ferone et al. [7, 8] designed the following pure and hybrid multi-start iterative heuristics:

- A pure GRASP, inspired by [9]
- A GRASP that uses forward path-relinking for intensification
- A pure VNS
- A VNS that uses forward path-relinking for intensification
- A GRASP that uses VNS to implement the local search phase
- A GRASP that uses VNS to implement the local search phase and forward path-relinking for intensification.

The algorithms were tested on several random instances and the results showed that the hybrid GRASP with VNS and forward path-relinking always found much better quality solutions compared with the other algorithms, but clearly with higher running times as compared to the pure GRASP and the hybrid GRASP with forward path-relinking. The best objective function values found by GRASP and its hybrids were when the construction phase was more greedy than random. The integration of forward path-relinking as an intensification procedure in the pure metaheuristics was beneficial in terms of solution quality. A further investigation, studying the empirical distributions of the random variable *time-to-target-solution-value*, revealed that, given any fixed amount of computing time, GRASP with forward path-relinking has an empirically higher probability than all competitors of finding a target solution.

In [1], besides the first linear integer programming formulation for the FFMSP described above (15)–(17), a hybrid ant colony optimization approach has been proposed. This hybrid approach consists of two phases. A first phase applies ant colony optimization until the convergence of the pheromone values is reached. After this first phase, the algorithm possibly applies a second phase in which the hybridization with a mathematical programming solver takes place. Both the linear integer programming formulation and the hybrid ant colony algorithm have been compared to the most performing hybrid GRASP with path-relinking and computational results on a large set of randomly generated test instances have indicated that the hybrid ACO is very competitive.

Blum and Pinacho-Davidson [2] proposed the application of a recently introduced version of ant colony optimization—negative learning ant colony optimization—to the FFMSP. The proposed algorithm makes use of negative learning in addition to the standard positive learning mechanism in order to achieve better guidance for the exploration of the search space. The outcomes of the experimental experience showed that this new algorithm is especially successful for instances with specific characteristics and that none of the existing state-of-the-art methods is best for all problem instances. What achieves excellent performance on average on each type of instance is an algorithm afterwards designed by [32]. Here, the authors proposed a LEARN_CMSA, i.e., a hybrid method that combines a population-based algorithm with a CMSA (construct, merge, solve, and adapt). CMSA in its pure version does not make use of any explicit learning mechanism. Instead, in LEARN_CMSA explicit learning has been introduced to CMSA, so achieving superior performance when compared to current state-of-the-art solvers.

A Simple ILP-Based Heuristic

In the following, we present the results of a quite simple ILP-based heuristic that can be applied to all four problems described before: the closest string problem (CSP), the close to most string problem (CTMSP), the farthest string problem (FSP), and the far from most string problem (FFMSP). The heuristic is based on the ILP models of the four problems. It works as follows. Given a fixed computation time limit (t_{limit}), maximally half of this computation time is given to an ILP solver for tackling the mixed integer linear problem (MILP) that is obtained by relaxing the x_{ck}-variables involved in all four ILP models. The used ILP solver returns the best solution found in the given computation time. Note that this solution may or may not correspond to the optimal MILP solution. The fractional values of the x_{ck}-variables after termination of the solver are henceforth denoted by x'_{ck}. In the second phase of the heuristic, the corresponding ILP models are solved in the remaining computation time, with the following additional constraints:

$$x_{ck} = 1 \quad \text{for } k = 1, \ldots, m, \; c \in \Sigma_k, \; x'_{ck} = 1. \tag{18}$$

In other words, whenever a variable x_{ck} in the best-found MILP solution has a value of 1, this value is fixed for the solution of the ILP model.

Experimental Evaluation

In the case of all four problems, the heuristic described above is compared with the application of the ILP solver to the original ILP models. Moreover, a simple greedy algorithm was applied. This greedy algorithm generates a solution by selecting for each position of the solution string the letter which, in the case of the CSP and the CTMSP, has the least number of appearances at this position in the set of input strings and which, in the case of the FSP and the FFMSP, has the highest number of appearances at this position in the set of input strings. The greedy algorithm is henceforth denoted by GREEDY. As (M)ILP solver, we used IBM ILOG CPLEX V22.1. The experimental results were obtained on a high-performance computing cluster equipped with machines with Intel® Xeon® 5670 CPUs having 12 cores of 2.933 GHz and at least 32 GB of RAM. Moreover, CPLEX was configured for single-threaded execution. Depending on the problems, we used two different computation time limits: $t_{\text{limit}} = 200\,\text{s}$ and $t_{\text{limit}} = 3600\,\text{s}$ per problem instance. The different applications of CPLEX, respectively the ILP-based heuristic, are named accordingly: CPLEX-200, CPLEX-3600, HEURISTIC-200, and HEURISTIC-3600.

All the algorithms described above were applied, in the context of all four problems, to a set of benchmark instances that was originally introduced in [7] for the FFMSP. This set consists of random instances of different sizes. More specifically, the number of input strings (n) is in {100, 200}, and the length of the

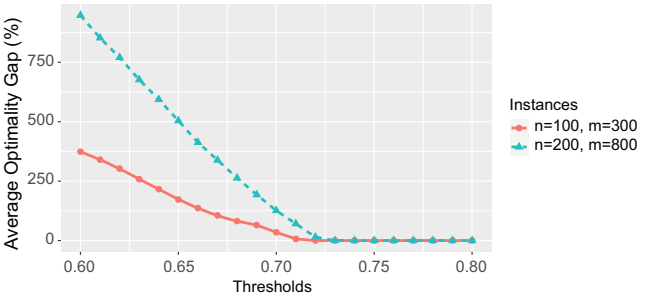
input strings (m) is in $\{300, 600, 800\}$. In all cases, the alphabet size is four, that is, $|\Sigma| = 4$. For each combination of n and m, the set consists of 100 random instances. This makes a total of 600 instances. Note that, in the context of the CTMSP and the FFMSP, a value for parameter t must be specified before running the algorithm(s). However, a sensible choice of t is not trivial. For example, in the context of the CTMSP, the lower the value of t, the easier it should be, for example, for CPLEX to solve the problem to optimality. To be able to choose meaningful values for t, the following experiments were executed. In the case of the CTMSP, CPLEX was applied to each problem instance with a CPU time limit of 3600 s for each value of $t \in \{0.6, 0.61, \ldots, 0.79, 0.8\,\text{m}\}$. In the case of the FFMSP, CPLEX was applied to each problem instance with a CPU time limit of 3600 s for each value of $t \in \{0.7, 0.71, \ldots, 0.89, 0.9\,\text{m}\}$. The corresponding optimality gaps (averaged over 100 problem instances) and the number of instances (out of 100) that were solved to optimality are graphically presented in Fig. 1. The results reveal that, in the case of the CTMSP, the problem becomes difficult for approx. $t \leq 0.72$ m. In the case of the FFMSP, the problem becomes difficult for approx. $t \geq 0.78$ m. Therefore, the following values for t were chosen for the final experimental evaluation: $t \in \{0.65, 0.7, 0.75\,\text{m}\}$ in the case of the CTMSP and $t \in \{0.75, 0.8, 0.85\,\text{m}\}$ in the case of the FFMSP.

The numerical results for the CSP are shown in Table 1. They are presented as averages over the 100 instances for each combination of n (the number of input strings) and m (the length of the input strings). For all three algorithms we provide the values of the best found solutions (averaged over 100 problem instances) and the computation time at which these solutions were found. In the case of CPLEX-200 the average optimality gap is additionally provided. The results show that HEURISTIC-200 significantly outperforms GREEDY and—with the exception of case ($n = 200, m = 800$)—produces slightly better results than CPLEX-200. Similar observations can be made concerning the results for the FSP provided in Table 2.

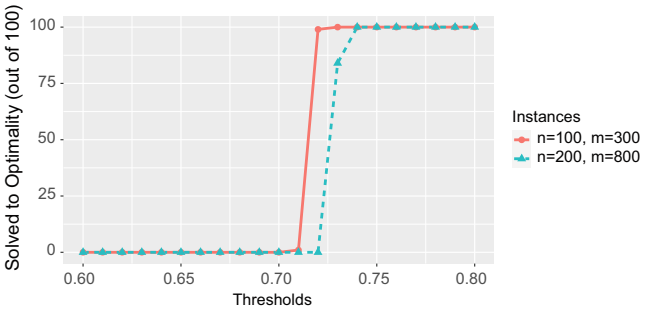
In the case of the CTMSP, both the ILP-based heuristic and CPLEX were applied with computation time limits of 200 and 3600 CPU seconds. Therefore, Table 3 contains results for HEURISTIC-200, HEURISTIC-3600, CPLEX-200, and CPLEX-3600. The following observations can be made:

Table 1 Numerical results for the CSP

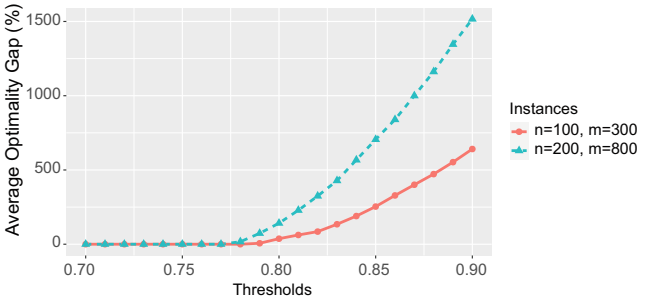
		GREEDY		HEURISTIC-200		CPLEX-200		
n	m	Value	Time	Value	Time	Value	Time	Gap
100	300	228.55	<0.009	**213.35**	10.97	213.53	11.46	0.90
100	600	446.91	<0.009	**422.88**	7.29	423.09	6.43	0.48
100	800	590.08	<0.009	**562.37**	11.23	562.60	6.08	0.37
200	300	235.02	<0.009	**219.98**	11.42	220.05	16.08	1.39
200	600	457.19	<0.009	**434.32**	12.36	434.33	24.59	0.77
200	800	604.49	<0.009	576.98	10.95	**576.94**	25.01	0.58



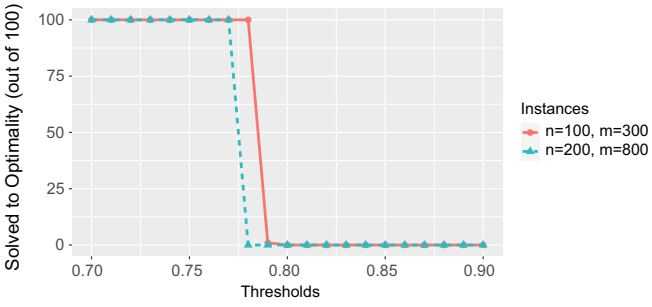
(a) CTMSP: average optimality gaps (%)



(b) CTMSP: # of solved instanced (of 100)



(c) FFMSP: average optimality gaps (%)



(d) FFMSP: # of solved instanced (of 100)

Fig. 1 Justification for the choice of parameter t in the context of the CTMSP and the FFMSP. Data shown in these graphics refer to the application of CPLEX to all problem instances. (**a**) CTMSP: average optimality gaps (%). (**b**) CTMSP: # of solved instanced (of 100). (**c**) FFMSP: average optimality gaps (%). (**d**) FFMSP: # of solved instanced (of 100)

Table 2 Numerical results for the FSP

		GREEDY		HEURISTIC-200		CPLEX-200		
n	m	Value	Time	Value	Time	Value	Time	Gap
100	300	221.67	<0.009	**236.53**	9.77	236.46	20.91	0.66
100	600	455.15	<0.009	**476.55**	8.75	476.33	6.92	0.39
100	800	611.61	<0.009	**636.56**	8.53	636.39	12.80	0.28
200	300	215.74	<0.009	**230.08**	9.97	230.07	14.92	1.27
200	600	443.88	<0.009	465.62	12.49	**465.66**	17.27	0.66
200	800	596.41	<0.009	622.75	12.22	**622.79**	19.44	0.49

- Both CPLEX and the ILP-based heuristic greatly outperform GREEDY.
- When the problem is rather easy—that is, for a setting of $t = 0.75$ m—CPLEX has usually slight advantages over the ILP-based heuristic. This is the case especially for the instances with a larger number of input strings ($n = 200$).
- With growing problem difficulty, the ILP-based heuristic starts to outperform CPLEX. In particular, for a setting of $t = 0.65$ m, the differences in the qualities of the obtained solutions between the ILP-based heuristic and CPLEX are significant.

Not surprisingly, the same observations can be made in the context of the FFMSP, for which the results are provided in Table 4. As the considered benchmark instances were originally used for the FFMSP, we are able to compare them to current state-of-the-art results for this problem (see [3]). This comparison is graphically presented in Fig. 2 for all instances concerning the interesting cases $t = 0.8$ m and $t = 0.85$ m. The results show that, for $t = 0.8$ m, HEURISTIC-3600 is generally outperformed by the other state-of-the-art methods, especially for what concerns ACO_NEG. However, note that when the instance size (in terms of the number of input strings and their length) grows, HEURISTIC-3600 starts to produce better results than some of the competitors. In the case of $t = 0.85$ m, which results in more difficult instances than $t = 0.8$ m, HEURISTIC-3600 is clearly able to compete with the current state-of-the-art methods.

Conclusions

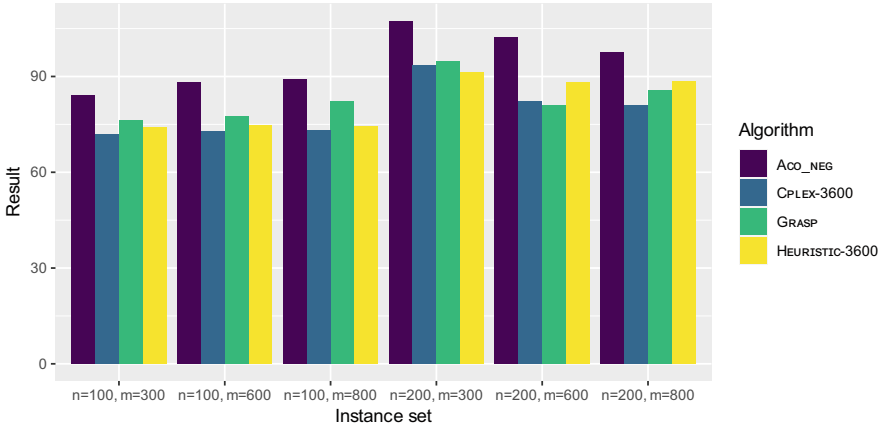
The goal of this chapter was to provide an overview of some string selection and comparison problems, with special emphasis on the optimization and operational research perspective. Besides mathematical models that can be used to find exact solutions only up to a certain instance size, there are many approximate techniques, proposed by researchers from several heterogenous communities, which are more or less efficient, depending on the input data and the type of information they make use of. Not surprisingly, generally there is no single best approach that wins in every

Table 3 Numerical results for the CTMSP

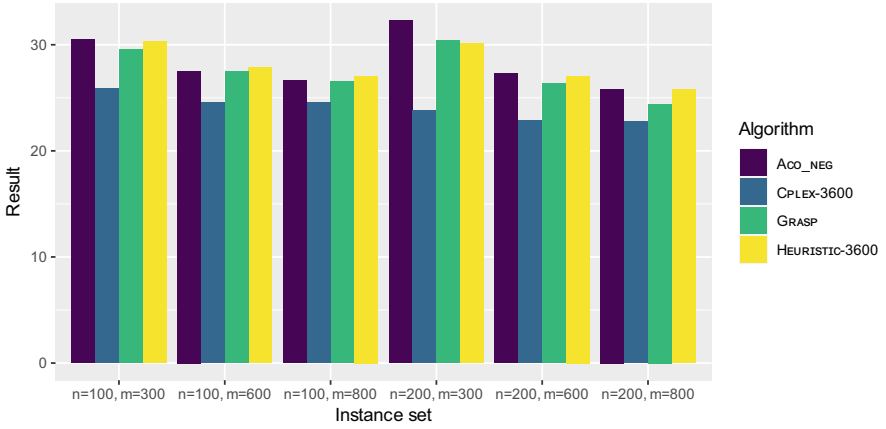
			GREEDY		HEURISTIC-200		HEURISTIC-3600		CPLEX-200			CPLEX-3600		
n	m	t	Value	Time	Value	Time	Value	Time	Value	Time	Gap	Value	Time	Gap
100	300	0.65 m	4.18	<0.009	32.36	138.07	**34.42**	2248.60	28.05	91.38	201.11%	30.48	1944.27	172.57%
100	300	0.7 m	55.94	<0.009	72.67	120.22	**75.27**	1724.74	70.53	57.72	40.72%	73.20	1050.94	34.75%
100	300	0.75 m	98.30	<0.009	99.97	1.62	99.97	1.44	**100.00**	0.67	0.00%	**100.00**	0.73	0.00%
100	600	0.65 m	0.70	<0.009	30.48	136.0	**32.1**	2153.73	26.94	98.54	215.53%	28.67	1818.08	193.08%
100	600	0.7 m	56.05	<0.009	72.96	68.67	**75.55**	900.44	71.00	85.55	40.57%	74.17	1260.83	33.83%
100	600	0.75 m	99.74	<0.009	99.99	1.92	**100.00**	2.56	**100.00**	1.06	0.00%	**100.00**	1.16	0.00%
100	800	0.65 m	0.18	<0.009	29.79	130.02	**31.21**	2156.36	26.74	107.49	218.69%	28.38	1949.53	197.10%
100	800	0.7 m	56.84	<0.009	73.13	50.44	75.85	721.35	72.41	82.38	38.21%	**75.94**	1429.90	31.24%
100	800	0.75 m	99.94	<0.009	**100.00**	1.37	**100.00**	1.24	**100.00**	1.40	0.00%	**100.00**	1.52	0.00%
200	300	0.65 m	2.16	<0.009	30.86	137.83	**35.04**	2457.94	26.91	104.40	499.89%	29.12	2262.42	449.10%
200	300	0.7 m	65.51	<0.009	86.18	150.25	**95.23**	2659.39	87.60	124.02	115.71%	92.33	1835.97	103.68%
200	300	0.75 m	186.07	<0.009	194.59	35.32	196.51	570.83	**200.00**	3.40	0.00%	**200.00**	3.65	0.00%
200	600	0.65 m	0.05	<0.009	29.17	134.38	**31.62**	2129.23	24.75	121.01	556.99%	26.50	2301.02	508.69%
200	600	0.7 m	50.04	<0.009	76.76	147.90	**89.37**	2519.44	73.82	146.50	157.65%	85.60	1007.69	121.38%
200	600	0.75 m	196.29	<0.009	197.65	34.95	198.9	410.30	**200.00**	3.55	0.00%	**200.00**	3.82	0.00%
200	800	0.65 m	0.03	<0.009	27.81	132.54	**30.32**	2010.34	24.96	113.12	551.47%	26.71	2536.50	504.25%
200	800	0.7 m	43.27	<0.009	70.52	148.24	**89.14**	2589.50	67.14	117.44	184.19%	83.97	1235.72	126.13%
200	800	0.75 m	198.40	<0.009	198.09	32.22	199.35	398.72	**200.00**	4.70	0.00%	**200.00**	5.06	0.00%

Table 4 Numerical results for the FFMSP

			GREEDY		HEURISTIC-200		HEURISTIC-3600		CPLEX-200			CPLEX-3600		
n	m	t	Value	Time	Value	Time	Value	Time	Value	Time	Gap	Value	Time	Gap
100	300	0.75 m	98.40	<0.009	99.99	0.76	**100.00**	7.77	**100.00**	0.70	0.00%	**100.00**	0.70	0.00%
100	300	0.8 m	54.67	<0.009	71.78	137.89	**74.24**	2423.77	69.74	91.25	42.09%	71.84	1338.33	37.25%
100	300	0.85 m	1.71	<0.009	28.98	109.66	**30.34**	1814.25	24.62	91.65	276.34%	25.88	1108.83	253.47%
100	600	0.75 m	99.82	<0.009	**100.00**	0.70	**100.00**	0.65	**100.00**	1.02	0.00%	**100.00**	0.93	0.00%
100	600	0.8 m	53.52	<0.009	72.02	110.81	**74.58**	1714.49	69.01	107.13	44.25%	72.74	971.38	36.12%
100	600	0.85 m	0.10	<0.009	26.96	104.86	**27.86**	1804.16	23.18	96.67	302.11%	24.58	1179.52	275.10%
100	800	0.75 m	99.99	<0.009	**100.00**	0.75	**100.00**	0.69	**100.00**	1.29	0.00%	**100.00**	1.19	0.00%
100	800	0.8 m	53.42	<0.009	72.25	115.45	**74.39**	1455.15	68.89	105.46	44.92%	73.22	1089.16	35.26%
100	800	0.85 m	0.04	<0.009	26.3	103.67	**27.05**	1801.38	23.68	105.20	294.25%	24.53	911.98	277.38%
200	300	0.75 m	188.58	<0.009	195.28	34.54	196.88	624.15	**200.00**	2.16	0.00%	**200.00**	2.00	0.00%
200	300	0.8 m	60.63	<0.009	80.03	163.36	91.37	2799.12	85.13	108.89	129.72%	**93.62**	2610.21	107.68%
200	300	0.85 m	0.61	<0.009	27.18	104.80	**30.08**	1821.05	22.08	86.56	733.48%	23.78	1544.73	667.92%
200	600	0.75 m	196.81	<0.009	198.28	33.31	199.3	383.97	**200.00**	3.08	0.00%	**200.00**	2.89	0.00%
200	600	0.8 m	43.29	<0.009	75.02	154.66	**88.09**	2706.11	72.16	160.02	172.55%	82.36	1268.41	137.29%
200	600	0.85 m	0.00	<0.009	25.64	103.61	**27.05**	1803.62	21.77	104.15	745.54%	22.86	1418.47	701.52%
200	800	0.75 m	198.75	<0.009	199.15	25.56	199.72	241.00	**200.00**	4.01	0.00%	**200.00**	4.06	0.00%
200	800	0.8 m	36.03	<0.009	72.37	145.35	**88.46**	2629.60	62.94	131.51	212.28%	80.92	1544.68	141.69%
200	800	0.85 m	0.00	<0.009	24.25	101.53	**25.79**	1800.10	21.92	93.80	739.53%	22.75	966.42	705.56%



(a) Results for all instances with $t = 0.8$ m.



(b) Results for all instances with $t = 0.85m$.

Fig. 2 Graphical representation of the comparison between HEURISTIC-3600 and some of the current FFMSP state-of-the-art methods (GRASP and ACO_NEG) for $t = 0.8$ m (see (**a**)) and $t = 0.85$ m (see (**b**))

aspect. Therefore, we proposed a simple ILP-based heuristic that can be used for any of the four considered problems. We have shown that this heuristic outperforms both a general greedy algorithm and the application of an ILP-solver (CPLEX) to the original ILP models. In the case of the far from most string problem, we were even able to show that this simple heuristic is able to produce state-of-the-art results for instances that are intrinsically difficult to solve.

Cross-References

- Genetic Algorithms
- GRASP
- Variable Neighborhood Descent
- Variable Neighborhood Search

Acknowledgments C. Blum was supported by two grants funded by MCIN/AEI/10.13039/50110 0011033: PID2022-136787NB-I00 and TED2021-129319B-I00. Moreover, C. Blum would like to acknowledge the support of the Spanish National Research Council (CSIC).

References

1. Blum C, Festa P (2014) A hybrid ant colony optimization algorithm for the far from most string problem. In: Proceedings of the 14th European conference on evolutionary computation in combinatorial optimisation (EvoCOP2014). Lecture notes in computer science, vol 8600, pp 1–12
2. Blum C, Pinacho-Davidson P (2023) Application of negative learning ant colony optimization to the far from most string problem. In: Proceedings of the 23rd European conference on evolutionary computation in combinatorial optimisation, EvoCOP 2023, held as part of EvoStar 2023. Lecture notes in computer science, vol 13987, pp 82–97
3. Blum C, Pinacho-Davidson P (2023) Application of negative learning ant colony optimization to the far from most string problem. In: Evolutionary computation in combinatorial optimization: 23rd European conference, EvoCOP 2023, held as part of EvoStar 2023, Brno, Czech Republic, 12–14 Apr 2023, proceedings. Springer, Berlin/Heidelberg, pp 82–97
4. Boucher C, Landau G, Levy A, Pritchard D, Weimann O (2013) On approximating string selection problems with outliers. Theor Comput Sci 498:107–114
5. Croce FD, Garraffa M (2014) The selective fixing algorithm for the closest string problem. Comput Oper Res 41:24–30
6. Croce FD, Salassa F (2012) Improved LP-based algorithms for the closest string problem. Comput Oper Res 39:746–749
7. Ferone D, Festa P, Resende M (2013) Hybrid metaheuristics for the far from most string problem. In: Proceedings of 8th international workshop on hybrid metaheuristics. Lecture notes in computer science, vol 7919, pp 174–188
8. Ferone D, Festa P, Resende M (2016) Hybridizations of GRASP with path relinking for the far from most string problem. Int Trans Oper Res 23(3):481–506
9. Festa P (2007) On some optimization problems in molecular biology. Math Biosci 207(2):219–234
10. Frances M, Litman A (1997) On covering problems of codes. Theory Comput Syst 30(2):113–119
11. Goldberg DE (1989) Genetic algorithms in search, optimization, and machine learning. Addison-Wesley, Reading, Massachusetts
12. Gramm J (2003) Fixed-parameter algorithms for the consensus analysis of genomic data. PhD thesis
13. Gramm J, Hüffner F, Niedermeier R (2002) Closest strings, primer design, and motif search. In: Sixth annual international conference on computational molecular biology, pp 74–75
14. Gramm J, Niedermeier R, Rossmanith P (2003) Fixed-parameter algorithms for closest string and related problems. Algorithmica 37:25–42
15. Gumin M (2016) Wave function collapse. vol GitHub [Online]. Available: https://github.com/mxgmn/WaveFunctionCollapse

16. Holland JH (1975) Adaptation in natural and artificial systems. MIT Press, Cambridge, Massachusetts
17. Kirkpatrick S (1984) Optimization by simulated annealing: quantitative studies. J Stat Phys 34(5–6):975–986
18. Lanctot J (2004) Some string problems in computational biology. PhD thesis
19. Lanctot J, Li M, Ma B, Wang S, Zhang L (1999) Distinguishing string selection problems. In: Proceedings of the annual ACM-SIAM symposium on discrete algorithms (SODA), pp 633–642
20. Lanctot J, Li M, Ma B, Wang S, Zhang L (2003) Distinguishing string selection problems. Inf Comput 185(1):41–55
21. Li M, Ma B, Wang L (1999) Finding similar regions in many strings. In: ACM symposium on theory of computing (STOC'99), pp 473–482
22. Liu X, He H, Sýkora O (2005) Parallel genetic algorithm and parallel simulated annealing algorithm for the closest string problem. In: Proceedings of ADMA 2005. Lecture notes in artificial intelligence, vol 3584, pp 591–597
23. Liu X, Liu S, Hao Z, Mauch H (2011) Exact algorithm and heuristic for the closest string problem. Comput Oper Res 38(11):1513–1520
24. Ma B, Sun X (2008) More efficient algorithms for closest string and substring problems. Lecture notes in computer science. Springer-Verlag, Berlin, Heidelberg, vol 4955, pp 396–409
25. Macario A, de Macario EC (eds) (1990) Gene probes for bacteria. Academic, San Diego
26. Meneses C, Oliveira C, Pardalos P (2005) Optimization techniques for string selection and comparison problems in genomics. IEEE Eng Med Biol Mag 24(3):81–87
27. Meneses C, Pardalos P, Resende M, Vazacopoulos A (2005) Modeling and solving string selection problems. In: Proceedings of the 2005 international symposium on mathematical and computational biology (BIOMAT 2005), pp 55–65
28. Metropolis N, Rosenbluth A, Rosenbluth M, Teller A, Teller E (1953) Equation of state calculations by fast computing machines. J Chem Phys 21(6):1087–1092
29. Mousavi S, Babaie M, Montazerian M (2012) An improved heuristic for the far from most strings problem. J Heurist 18:239–262
30. Newgas A (2021) Tessera: a practical system for extended wavefunctioncollapse. In: Proceedings of the 16th international conference on found. Digit. Games (FDG), pp 1–7
31. Pardalos P, Oliveira C, Lu Z, Meneses C (2004) Optimal solutions for the closest string problem via integer programming. INFORMS J Comput 16:419–429
32. Pinacho-Davidson P, Blum C, Pinninghoff M, Contreras R (2024) Extension of CMSA with a learning mechanism: application to the far from most string problem. Int J Comput Intell Syst 1(11):Article number 109
33. Roman S (1992) Coding and information theory. Graduate texts in mathematics, vol 134. Springer, New York
34. Sim J, Park K (1999) The consensus string problem for a metric is NP-complete. In: Proceedings of the annual australasian workshop on combinatorial algorithms (AWOCA), pp 107–113
35. Tanaka S (2012) A heuristic algorithm based on lagrangian relaxation for the closest string problem. Comput Oper Res 39:709–717
36. Wang L, Zhu B (2009) Efficient algorithms for the closest string and distinguishing string selection problems. Lecture notes in computer science. Springer, Berlin, vol 5598, pp 261–270
37. Xu S, Perkins D (2022) A heuristic solution to the closest string problem using wave function collapse techniques. IEEE Access 10:115869–115883
38. Zörnig P (2011) Improved optimization modelling for the closest string and related problems. Appl Math Model 35(12):5609–5617
39. Zörnig P (2015) Reduced-size integer linear programming models for string selection problems: application to the farthest string problem. J Comput Biol 22(8):729–742

Solving Large-Scale Vehicle Routing Problems

55

Luca Accorsi and Daniele Vigo

Contents

Introduction 1674
Scalability 1676
Problem Definition and Notation 1676
Decomposition 1677
Speeding Up Local Search 1678
  Acceleration Techniques 1679
  Pruning Techniques 1683
Additional Design and Implementation Challenges 1686
  Memory Usage 1686
  Solution Management 1687
  Initial Solution Construction 1688
  Resources Processing 1689
  More Limitation Strategies 1691
Conclusions 1692
Cross-References 1692
References 1693

Abstract

Vehicle routing problems (VRPs) are challenging combinatorial optimization problems that, given their significant impact on distribution logistics, continue to attract considerable research efforts to develop efficient and effective solution

L. Accorsi (✉)
Google, Munich, Germany

D. Vigo
Department of Electrical, Electronic and Information Engineering "G. Marconi", University of Bologna, Bologna, Italy

CIRI ICT, University of Bologna, Bologna, Italy
e-mail: daniele.vigo@unibo.it

© Springer Nature Switzerland AG 2025

R. Martí et al. (eds.), *Handbook of Heuristics*,
https://doi.org/10.1007/978-3-032-00385-0_80

methods. As the instance size grows, the combinatorial nature of these problems makes traditional approaches unable to find adequate solutions within a reasonable computing time. Large problem instances typically require the use of specialized scalability techniques that, coupled with the raw computing power of modern machines, make their heuristic resolution possible. This chapter reviews some of the most effective techniques for designing heuristics for large-scale VRPs.

Keywords

Vehicle routing problems · Large-scale instances · Scalability techniques · Heuristics

Introduction

Vehicle routing represents a family of fundamental combinatorial optimization problems. In essence, a vehicle routing problem (VRP) requires the identification of some routes to serve a set of geographically distributed customers so that the overall cost of the solution is minimal and all relevant constraints are satisfied [51].

Since the seminal proposal of the truck dispatching problem by [18], VRPs have attracted the attention of many researchers, given the challenges they pose and their significant impact on the efficiency of distribution logistics. Due to their practical relevance, various VRPs have been defined to properly model complex and realistic scenarios [56]. The ability to solve increasingly challenging problems that more closely model company requirements is thus of utmost importance from both a scientific and an application point of view.

A crucial factor in determining the empirical difficulty of a VRP is also the scale of problem instances that should be solved in practical applications. In fact, the larger the instance, the greater the search space, and thus the more challenging its effective exploration.

Currently, research on large VRPs is mostly related to the capacitated vehicle routing problem (CVRP), as it is the canonical variant of the VRP family from which more complicated problem variants are usually derived. The CVRP will thus be the main focus of this chapter. However, most of the techniques described in the following sections are largely problem independent and can, with additional implementation effort, be extended to more constrained variants.

The definition of an instance's scale is clearly problem dependent. For a CVRP, instances with several hundreds of customers are usually classified in the literature as *large scale* [52]. Larger instances, containing several thousands of customers, are generally called *very large-scale* instances [6, 33, 59]. Finally, instances with hundreds of thousands of customers are usually designated as *extremely large scale* [4]. In the remainder of the chapter, we will often not distinguish among the different scale nuances but rather just use the large-scale term with a broad meaning.

Recently, there has been renewed interest in expanding the frontier of VRPs by devising techniques to solve larger problem instances. Some applications, such as

those arising in waste collection and parcel delivery (and collection) problems, in fact, require the solution of such large-scale VRPs. A relevant example is given by [33], who proposed a set of CVRP instances with up to twelve thousand customers, some of which were derived from real-life waste collection problems in Finland. Similarly, [59] proposed a CVRP instance dataset, which has up to three thousand customers, provided by a logistics company, which contains the coordinates of customer locations within four major Greek cities. Additionally, [6] proposed a dataset of large-scale CVRP instances based on real-world parcel delivery problems in Belgium, where the largest instance includes thirty thousand customers. Finally, in [12], Cavaliere et al. extended the Belgium instances to model a VRP with simultaneous pickups and deliveries, as this is especially relevant for e-commerce applications in which the rapid increase in deliveries may co-occur with the collection of a large amount of returned goods. A complementary goal is pursued by [4], where the authors tackle extremely large-scale CVRP instances with up to one million customers (see Fig. 1) as a pure research exercise to promote insightful ideas possibly applicable and beneficial to smaller-sized instances.

Research on VRPs falls mainly into one of two categories: exact algorithms or heuristics. Despite recent advances in exact methods [17, 40, 41], large-scale instances are still the domain of heuristic approaches. The variance in the computing time of an exact method is, in fact, considerably high, and instances encountered in real-life settings should usually be solved quickly and with predictable computing time. This chapter thus overviews the most relevant techniques found in the literature that enable the scalability of heuristics for VRPs.

This chapter is organized as follows. In section "Scalability," we emphasize the importance of scalable solution approaches. Section "Problem Definition and Notation" provides a formal description of the CVRP, which is our reference problem, and introduces the common notation used throughout the other sections. In section "Decomposition," we review decomposition strategies, which are among

Fig. 1 Instance Emilia-Romagna proposed in [4] containing 900k customers

the fundamental tools for managing instances as their size grows. Section "Speeding Up Local Search" introduces local search acceleration and pruning techniques, as they have been shown to be essential components of many successful algorithms for large-scale VRPs. In section "Additional Design and Implementation Challenges," we discuss additional design choices and implementation details regarding scaling to large VRPs. Lastly, we conclude with section "Conclusions," where we provide some final remarks.

Scalability

The evaluation of heuristics is governed by several principles as discussed, for example, in Chap. 4 of [51]. The quality of the solutions found and the required computing time are among the most commonly used criteria. However, a solution approach may also have other desirable and often conflicting properties, such as being conceptually simple or flexible enough when it comes to extending it to other problem variants.

Another desirable property is the *scalability*, defined as a heuristic's capability to effectively find good-quality solutions in relatively short computing time across a wide range of instance sizes, up to large-scale ones.

This chapter reviews techniques that make VRP heuristics scalable. These techniques enable algorithms to approach large-scale instances by encouraging the usage of memory-efficient data structures and computationally efficient procedures. In addition, they can often be tuned to achieve the desired trade-off between computing time and search exhaustiveness, which is frequently correlated to solution quality.

The literature shows how techniques originally devised for tackling large-scale instances can often also be beneficial when solving smaller-scale ones, concurring in finding improved quality solutions [22, 45]. Designing methods explicitly targeting large-scale instances would thus often simply enlarge the set of approachable instances, possibly without (or with just modest) performance worsening on smaller-scale ones.

However, these benefits may come with additional design and implementation challenges. In particular, implementing some of these techniques may not be as straightforward as implementing their naive and less-scalable counterpart. Moreover, their adoption often gives rise to additional design choices, which may increase the overall complexity of the approach by requiring careful validation and possibly tuning of extra parameters.

Problem Definition and Notation

As mentioned in section "Introduction," we use the CVRP as a reference problem to describe the most relevant scalability techniques in the following sections.

The CVRP is defined on an undirected and complete graph $G = (V, E)$. The set of vertices is $V = \{0\} \cup V_c$, where 0 denotes the depot and $V_c = \{1, 2, \ldots, N\}$ is a set of N customers. We denote by E the set of edges. Each edge $(i, j) \in E$ is

associated with a cost c_{ij}, typically defined as the Euclidean distance between i and j. We assume that the costs are symmetric and satisfy the triangle inequality, that is, $c_{ij} \leq c_{ik} + c_{kj} \forall, i, j, k \in V$.

For a subset $V' \subseteq V$, we denote by $\mathcal{N}_i^k(V')$ the ordered list of the k nearest-neighbor vertices $j \in V'$ of vertex i with respect to costs c_{ij}. Whenever $k = |V'|$, we refer to the ordered list of neighbors with $\mathcal{N}_i(V')$.

Each customer $i \in V_c$ requires an integer quantity $q_i > 0$ of homogeneous goods from the depot, and for the depot, we have $q_0 = 0$.

The customers are served by an unlimited fleet of homogeneous vehicles, each with capacity Q, which is based at the depot. Recalling that a Hamiltonian circuit is a closed cycle that visits a set of customers exactly once, a CVRP solution S is made up of a number $|S|$ of Hamiltonian circuits, called routes, starting at the depot, visiting a subset of customers, and returning to the depot. Given a vertex $i \in V$, we denote by π_i and σ_i the predecessor and successor of i, respectively, in a given solution.

A solution is feasible if all customers are visited exactly once and the load of all routes in S does not exceed the capacity Q of the vehicles. The cost of a solution S is given by the sum of the edges' costs defining its routes and is denoted as $c(S)$. The goal of CVRP is to find a feasible solution with the minimum cost.

Decomposition

Decomposition is of fundamental importance for modern heuristics approaching large-scale VRP instances. In addition, it can also concur to find improved solutions when it is used to solve smaller instances [22]. We refer to [45] for a thorough analysis of decomposition techniques applied to VRPs.

The concept of decomposition is rooted in the *divide-and-conquer* principle. First, the original (large-scale) problem is split into several subproblems. The resulting subproblems are typically tailored to fit the resolution algorithm, but in general, they are considerably smaller than the original one so that they can be solved much more efficiently, possibly without the need for specialized scalability techniques. Once the subproblem solutions are available, they are collected and composed to obtain an overall solution to the original problem.

Therefore, decomposition strategies make resolution algorithms disregard the size dimension, enabling them to work on several smaller and more tractable problems. However, their usage also brings additional design questions that must be properly addressed to obtain an overall effective algorithm.

The first decision relates to the nature of the resulting subproblems. In a *homogeneous* decomposition, subproblems have the same characteristics of the original problem. This happens for VRPs created considering a fraction of the original customers. These subproblems can be solved using the same approach that theoretically could solve the original large problem. On the other hand, in a *heterogeneous* decomposition, the subproblems require the application of a specialized resolution algorithm, as they may differ significantly from the original problem. For example, consider a multilevel optimization approach with subproblems defined by

merging some of the original vertices into macro-vertices [13, 58]. In this scenario, the cost of an edge involving a macro-vertex clearly depends on the traversal order of vertices composing it, and an algorithm for the original problem cannot be directly applied.

As the original problem is decomposed into several subproblems, their dependencies must be defined. When the subproblems are *independent*, solving one of them does not affect the resolution of others. This, for example, happens when they are created from a partitioning of the original problem, and thus the cost and feasibility of a subproblem solution are unrelated to the resolution process of other subproblems. An independent decomposition is often the most desirable, as it easily enables the usage of parallel approaches by providing a straightforward way to combine the resulting subproblem solutions into a solution for the original problem.

Subproblems are instead *dependent* when some customers are assigned simultaneously to several of them [24]. In addition, dependencies also arise when the VRP has constraints on the whole or on a subset of customers, such as workload balancing between routes, levels of service for subsets of customers [11], consistency constraints [23], or a limit on the maximum number of usable vehicles. Ruin-and-recreate approaches [47] also perform a form of flexible instance decomposition with dependent subproblems [3, 14, 42]. In fact, the ruin step typically destroys a limited portion of the solution while keeping the remaining unchanged. Then, the recreate step repairs the ruined portion, often without the rigid boundaries that may be imposed by a classic partitioning decomposition. The working areas of subsequent ruin-and-recreate applications may overlap, thus enforcing dependencies between resolutions.

Finally, the definition of subproblems can be guided by several criteria. The routes of a high-quality solution are often used as a reference for partitioning customers [45]. This approach is closely related to the partial optimization metaheuristic under special intensification conditions [44, 49], which suggests optimizing subparts of a solution a posteriori, once a solution is available. Finding an appropriate way to decompose a problem a priori is, in fact, not a trivial task. Alternatively, customers frequently served together in high-quality solutions can be assigned to the same subproblems [36]. Generally, the vertices selected to belong to the same subproblem should be somehow related. For example, they may be geographically close, have similar service time windows, or have been observed together in good-quality solutions.

Speeding Up Local Search

Local search is among the most effective, yet computationally expensive techniques to improve the quality of solutions to combinatorial optimization problems [1]. It is a general heuristic approach that, given a starting solution S, explores a set of neighbor solutions S' obtained by performing a move that produces a local change to S. Virtually all successful metaheuristic approaches for VRPs and other problems have local search as their main building block.

The set $\mathcal{N}$ of solutions S' reachable from the current solution S is called the *neighborhood* of S for the local search procedure. Most of the effort in running a local search usually involves scanning the neighborhood. The neighbor solutions are evaluated for their feasibility and cost, and according to the selected search strategy, a feasible neighbor solution S' is selected to replace S. Typically, S' is either the first or the best-improving neighbor in the neighborhood. The process is then repeated until no improving solution is found, which implies that S is a *local optimum* for the neighborhood.

The number of neighbors in a neighborhood determines the computational complexity of its exploration. However, it should be noted that checking the feasibility or computing the cost of a given neighbor solution may not always be possible in constant time, thus adding complexity to the overall search process; see also section "Resources Processing."

There are some widely used VRP neighborhoods that have proven to be effective in practice. An example is the k-opt neighborhood that replaces k edges of the solution with k other edges [43]. The complexity of fully examining the k-opt neighborhood is $\mathcal{O}(|V|^k)$; thus, in practice, the special case with $k = 2$ is often used for intra- and inter-route optimization, given its quadratic computational complexity. The CROSS-exchange is another commonly used local search neighborhood [48]. It consists of replacing a contiguous sequence of n vertices with another one of length m that may belong to the same route or a different one. The computational complexity for given n and m is quadratic. The overall complexity to search the neighborhood for all possible values of n and m is therefore $\mathcal{O}(nm|V|^2)$. Finally, the ejection chain is another extensively used local search operator with exponential complexity [20]. It consists of performing a sequence of relocations, i.e., a special case of CROSS-exchange with $n = 1$ and $m = 0$, typically involving several routes, to improve the quality of the solution while seeking its feasibility.

Due to its effectiveness in improving the quality of solutions, several techniques have been developed to speed up local search executions. On the one hand, local search *acceleration techniques* can be used to explore the neighborhoods more efficiently without compromising the exhaustiveness of the search. On the other hand, local search *pruning techniques* can arbitrarily reduce the cardinality of a neighborhood, reaching the desired trade-off between speedup and quality by heuristically skipping the exploration of what are considered unpromising search areas. Finally, these techniques can be combined to obtain powerful tools for the design of scalable algorithms.

Acceleration Techniques

Some methods have been proposed to speed up local search execution by redesigning the exploration of neighborhoods compared to a naive for-loop implementation. These techniques retain the original search exhaustiveness, but better exploit the locality of local search changes to minimize the number of evaluations of neighbor

solutions. Thus, they are potentially beneficial not only to large-scale instances, but to instances of any size.

Sequential Search

Sequential search was originally independently proposed by [29] for the graph partitioning problem and by [15, 34] for the traveling salesman problem. It was then generalized and formalized by [28] for the CVRP and later applied to more constrained VRPs in [26].

Unlike traditional lexicographic search, sequential search efficiently explores local search neighborhoods by systematically decomposing moves into basic blocks called *partial moves*. This decomposition allows for early pruning of unpromising moves based on *gains* associated with partial moves.

The key aspect of sequential search is the decomposition of a local search move m into a sequence of k partial moves $p_k \circ \ldots \circ p_2 \circ p_1$. The neighbor solution S^k is then obtained by applying these partial moves to a starting solution S^0, i.e., $m(S^0) = p_k(\ldots(p_2(p_1(S^0)))) = S^k$. At stage $1 \leq i \leq k$, applying the partial move p_i to the solution S^{i-1} generates a not necessarily feasible solution S^i. In general, a move's decomposition is not unique; several decompositions of the same move exist and may differ in the number of stages and the structure of the partial moves.

For several traditional VRPs, the gain $g(m, S)$ of a move m applied to a solution S to obtain a neighbor solution S' only depends on the added and deleted edges and is defined as $g(m, S) = c(S) - c(S')$. Similarly, given a decomposition of m into $p_k \circ \ldots \circ p_2 \circ p_1$, it is possible to associate a partial gain $g(p_i, S)$, $1 \leq i \leq k$ with the partial moves composing move m in such a way that these partial gains depend only on the starting solution S.

Traditional VRP neighborhoods allow for the definition of *cost-independent* and *cyclic-independent* decompositions. In a cost-independent decomposition, the gain of a move m is given by the sum of the gains of partial moves $p_i, 1 \leq i \leq k$ composing m, that is, $g(m, S) = \sum_{i=1}^{k} g(p_i, S)$. A cyclic-independent decomposition is such that a move $m = p_k \circ \ldots \circ p_2 \circ p_1$, applied to a solution S, generates the same neighbor solution S' for every cyclic permutation of partial moves $p_1, p_2, \ldots, p_k$.

It is widely recognized that if a sequence of numbers has a positive sum, there is a cyclic permutation of these numbers such that every partial sum is positive [34]. Consequently, all improving moves described by a cost- and cyclic-independent decomposition can be constructed as a sequence of partial moves, where the sum of the partial gains is positive for all subsequences. Thus, when searching for an improving neighbor of a solution S with a move $m = p_k \circ \ldots \circ p_2 \circ p_1$, at stage 1, only partial moves p_1 such that $g(p_1, S) > 0$ need to be considered, while at a general stage ℓ, the search can be restricted to those partial moves p_ℓ such that $g(p_\ell, S) + \sum_{i=1}^{\ell-1} g(p_i, S) > 0$. Considering just this subset of moves may dramatically reduce the possibilities when looking for an improving neighbor solution by scanning only a fraction of the entire neighborhood.

The partial moves p_{ij}^{add} and p_{ij}^{rem} are the building blocks of moves that describe the VRP neighborhoods. More precisely, p_{ij}^{add} is the partial move that adds the edge (i, j) to the solution, while p_{ij}^{rem} removes it.

For example, consider a starting solution S and a swap move m_{ij}^{swap}, a special case of the CROSS-exchange operator, replacing vertex i with vertex j in S. A possible cost- and cyclic-independent decomposition of m_{ij}^{swap} is $p_{ij} \circ p_{ji} = p_{ji} \circ p_{ij}$ where p_{ab} is a compound partial move defined as $p_{ab} = p_{\pi_a a}^{rem} \circ p_{a\sigma_a}^{rem} \circ p_{\pi_b a}^{add} \circ p_{a\sigma_b}^{add}$ with π_t and σ_t denoting the predecessor and successor of the vertex t in solution S, respectively. The partial gain of p_{ab} is defined as $g(p_{ab}, S) = c_{\pi_a a} + c_{a\sigma_a} - c_{\pi_b a} - c_{a\sigma_b}$. When searching for an improving swap move, we can thus restrict the focus to first partial moves p_{ab} with a positive gain. First, all $i \in V$ are considered as initial candidate vertices to swap. Next, fixed i, promising vertices $j \in V \setminus \{i\}$ can be identified by considering neighbor vertices $t \in \mathcal{N}_i(V \setminus \{i\})$. To this end, we define the contribution associated with the choice of vertex i as $F = c_{\pi_i i} + c_{i\sigma_i}$ and the constant B as $B = F/2 = (c_{\pi_i i} + c_{i\sigma_i})/2$. The gain $g(p_{ij}, S)$ can thus be rewritten as $g(p_{ij}, S) = 2B - c_{\pi_j i} - c_{i\sigma_j}$. Looking for a first partial move with a positive gain $g(p_{ij}, S) > 0$ is equivalent to $(c_{\pi_j i} - B) + (c_{i\sigma_j} - B) < 0$, which implies $c_{\pi_j i} < B$ when $t = \pi_j$ or $c_{i\sigma_j} < B$ when $t = \sigma_j$. A single neighbor vertex $t \in \mathcal{N}_i(V \setminus \{i\})$ then identifies two swap moves, see Fig. 2. One in which $t = \pi_j$ and another one in which $t = \sigma_j$. The actual swapped vertex j is then selected accordingly, allowing the swap move gain and feasibility to be evaluated. The definition of constant B combined with the availability of the sorted list of neighbors $\mathcal{N}_i(V \setminus \{i\})$ allows the pruning of the search for a starting vertex i as soon as $c_{it} \geq B$. Moreover, the list of neighbors can heuristically be restricted to include only a fraction of vertices to further speed up the search process.

According to the instance and the local search operator under examination, [28] report a speed increase ranging from 5 to more than 10,000 when the neighborhoods are explored using sequential search compared to a traditional lexicographic search. Finally, this technique is also successfully used in [6] to solve very large-scale instances.

Static Move Descriptors

Static move descriptors (SMDs), originally introduced by [59], enable the efficient exploration of local search neighborhoods by exploiting the locality of changes induced by local search moves. In fact, a local search move application performs a very limited change, leaving most of the solution unaffected, especially when dealing with large-scale instances.

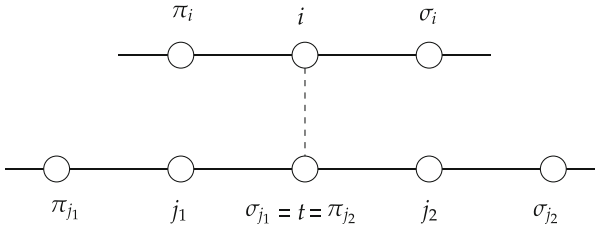


Fig. 2 Vertex t, neighbor of i, identifies the moves in which i is swapped with j_1 or with j_2

An SMD is a data structure that describes a unique local search move, generating a neighbor solution S' from a source solution S, and the associated change in the value of the objective function $\delta(S, S') = c(S) - c(S')$, called the δ-tag. The neighborhood exploration is redesigned around these data structures as follows. First, in the *initialization* stage, the δ-tag of the available SMDs is computed with respect to the current solution S. Then, a sequence of *search*, *execution*, and *update* stages is performed until no more improving solutions can be found in the neighborhood.

The search stage looks for a feasible and improving SMD, that is, an SMD describing a local search move that generates a feasible and improving neighbor solution. When such an SMD cannot be found, a local optimum is reached, and the neighborhood exploration is concluded. The local search move described by the SMD found during the search stage is then applied during the execution stage. Because the move application causes a local change to the solution, most of the SMDs will still hold a valid δ-tag even though the solution has been partially changed. The δ-tag of a constant and neighborhood-dependent number of SMDs affected by the change is finally updated during the update stage. See Fig. 3 for a possible pseudocode.

Improving SMDs can be efficiently retrieved using a heap data structure in which they are organized according to their δ-tag. The heap also ensures good performance during the update stage. The feasibility of an SMD is usually not included in its description; thus whenever an improving SMD is retrieved, its feasibility has to be checked before its possible execution. To obtain a best-improvement exploration strategy, all the most improving but infeasible SMDs are removed from the heap until a feasible one is retrieved. Then, after the move application, previously removed SMDs are reinserted into the heap, as they may now describe a feasible change. An alternative approach, proposed in [8], which assumes storing SMDs

```
begin APPLY(S)
1   H ← INITIALIZE(S);
2   L ← ∅;
3   while H is not empty do
4       Get the most improving move m from H;
5       if m does not produce a feasible change to S then
6           L ← L ∪ {m};
7           continue;
8       end-if;
9       Determine the set A of vertices affected by the application of m to S;
10      S ← EXECUTE(S, m);
10      Insert moves m ∈ L back into H;
11      L ← ∅;
12      UPDATE(H, A);
13  end-while;
end APPLY.
```

Fig. 3 Pseudocode of an SMD-based local search operator

into a binary heap implemented with a data structure using contiguous memory, consists of linearly scanning the heap to find the first feasible SMD. Given that in a binary heap entries are roughly sorted according to the used key, this results in a rough-best-improvement strategy.

Note that only improving SMDs can be stored in the heap to further speed up the interactions with the data structure. In fact, the computational complexity of its operations depends on the number of stored entries.

Compound neighborhoods can also be described in the framework. Extending the SMD definition to include a neighborhood identifier is sufficient to allow efficient exploration of several neighborhoods at once. However, note that the update stage would need to consider all the different local search operators included, making the process more expensive.

Consider again the swap local search operator already mentioned in section "Sequential Search." Given a starting solution S, the SMDs that describe a swap move are defined by the two swapped vertices $i, j \in V$ and the associated δ-tag. The execution of a move that swaps two generic vertices k and ℓ requires the update of the SMDs identified by the vertices i or j having one of the values in the set $\{\pi_k, k, \sigma_k, \pi_\ell, \ell, \sigma_\ell\}$, as some of the edges incident on these vertices have changed due to the swap. In addition to being stored in a heap, SMDs can also be indexed according to their defining vertices to quickly retrieve those requiring an update.

The SMD framework has been successfully used in [59] and [3] to solve very large-scale instances, and in [4] to deal with extremely large-scale instances.

Pruning Techniques

Contrary to acceleration techniques that preserve the original search space, pruning techniques heuristically trade the exhaustiveness of its exploration for faster execution. Despite restricting the search, because of the complex interactions among the algorithm components, they experimentally prove to be particularly effective in speeding up the overall algorithm execution without excessively harming the solution quality, especially in iterative approaches.

Granular Neighborhoods

As the size of the instance increases, fully exploring a local search neighborhood can become extremely time-consuming. In addition, only a small fraction of the explored neighbors will turn out to be improving.

A *granular neighborhood* [50] only includes a subset of the original neighbor solutions. In particular, the idea is to limit neighborhood exploration to the most promising subset of neighbors, ignoring the others. Indeed, a granular neighborhood can be seen as an effective implementation of *candidate-list* strategies [21].

These promising neighbor solutions are obtained by applying a *move generator* to a source solution S. A move generator $(i, j) \in \vec{E}$ is an arc used to uniquely identify a move for a local search operator, with $\vec{E}$ being the directed version

of the set of edges E. Indeed, even though the underlying graph is undirected, (i, j) and (j, i) can identify different local search moves. Finally, the set T of all move generators can be used to define a granular neighborhood, thus reducing the complexity of the neighborhood exploration to $\mathcal{O}(|T|)$.

The move generators are identified using *sparsification rules*. A common approach is to define, for a move generator (i, j), a local search change that inserts the edge (i, j) into the solution. For example, a swap move described by a move generator (i, j) would indeed swap vertices i and π_j. Note then the slight asymmetry of the move induced by the twin move generator (j, i).

Typical sparsification rules proxy the quality of neighbor solutions on the candidate edge (i, j) cost c_{ij}. In [2, 50], arcs are chosen as move generators if their cost is below a given cost threshold t, that is, $T = \{(i, j) \in \vec{E} : c_{ij} < t\}$. If the cost does not fully capture all problem characteristics, e.g., when there are time windows, the reduced costs obtained from a linear relaxation can be used for the same purpose [46].

An alternative to a cost-based approach consists in using a nearest-neighbor rule that guarantees a minimum number n_{gs} of the most promising move generators for every vertex [3]. Thus, $T = \cup_{i \in V}\{(i, j), (j, i) \in \vec{E} : j \in \mathcal{N}_i^{n_{gs}}(V \setminus \{i\})\}$. A cost-based rule may, in fact, not be the best model for scenarios where the standard deviation among edge costs is high, e.g., in clustered instances, leaving several vertices without local search moves directly involving them.

A *sparsification factor* $\gamma \in [0, 1]$ can be used to further filter the set of move generators and identify a fraction of the most promising ones. These *active* move generators are the only ones considered during local search procedures, i.e., $T^\gamma = \{(i, j) \in T : idx_{(i,j)} \leq \gamma \cdot |T|\}$, where $1 \leq idx_{(i,j)} \leq |T|$ is the position of move generator (i, j) in the list of all move generators sorted according to the chosen selection criteria, for example, the associated edge cost. The sparsification factor γ can be adjusted according to the algorithm's search trajectory, defining a so-called *dynamic* granular neighborhood. In particular, γ is increased when the local search struggles to find improving neighbor solutions. Instead, it is reduced to speed up exploration when several improving neighbors are often found. Typically, all move generators are identified once in a preprocessing step and stored in a list that is sorted according to the likelihood of identifying an improving solution, making changes to the fraction of currently active ones very efficient.

Varying a single sparsification factor globally increases and reduces the move generators for the whole instance. An alternative approach consists of using a *vertex-wise* sparsification factor $\gamma_i \in [0, 1], i \in V$ to achieve a more fine-grained management of the move generators [3]. The resulting sparsification vector $\bar{\gamma} = [\gamma_0, \gamma_1, \ldots, \gamma_{|V|}]$ allows for more precise control over the move generators, possibly tailoring the local search differently on different solution areas, at the cost of more complex management of such parameters, i.e., $T^{\bar{\gamma}} = \cup_{i \in V}\{(i, j) \in T_i : idx_{(i,j)} \leq \gamma_i \cdot |T_i|\}$, with T_i being the set of move generators related to the vertex i.

In the original proposal, [50] suggested including in the set of move generators all arcs incident to the depot. As noted in [7], whether this is always effective is still an open question. However, [3] report that including all such move generators may

not be appropriate when solving very large-scale instances. Their usage, even when dynamically managed, considerably slows down the local search execution without relevant improvements to the quality of final solutions.

Granular neighborhoods can be easily combined with the acceleration techniques described in section "Acceleration Techniques." In particular, the list of neighbors of vertex i used in the sequential search can be replaced by active move generators (i, j). Moreover, in the static move descriptor framework, only SMDs associated with active move generators can be initialized and later updated as a result of the application of an improved move [3, 59].

Limited Reoptimization

A key factor of successful implementations of VRP heuristics is to track instance or solution elements that are considered highly relevant to the current algorithm's state and focus the optimization procedures around them, possibly ignoring the other elements.

The first approaches in this line date back to the 1970s on research related to the traveling salesman problem (TSP). Lin and Kernighan [34] design the notorious Lin-Kernighan algorithm for the TSP, whose core is an effective search method for tentatively performing possibly long sequences of tour flips, starting from a given vertex, leading to an improved final tour. Initially, all vertices are marked as candidate starting points for searching such sequences. If the search initiated from vertex v is unsuccessful, v is unmarked, and the approach continues with another marked vertex. If, on the other hand, the search is successful, then it is possible that previously unmarked vertices may now allow for successful searches. Thus, all vertices are marked again before continuing with the next search. Applegate et al. [5] report that marking all vertices again was appropriate for the small problems instances faced by [34], having at most 110 vertices, but becomes extremely time-consuming for larger ones. Indeed, the likelihood that unmarked vertices far away from improving sequences of flips may lead to successful searches experimentally shown to be extremely low. Thus, a variation is proposed, based on the observation of [9] in the context of 2-opt, in which only vertices at the end of the flips in improving sequences are marked again. This generally results in slightly worse final solutions but greatly enhances the overall running time.

Martin et al. [37] apply a similar approach to solve the TSP in what we would classify today as an iterated local search metaheuristic [35]. In their approach, local search applications, performed using 3-opt and Lin-Kernighan, interleaved with perturbations, based on the double-bridge 4-opt move. In particular, only vertices sufficiently close to those involved in the double-bridge move are marked again as promising starting points for subsequent local search applications.

The approach is formalized in [39] through a local search framework with searches starting from a limited set of promising vertices. This set is initialized by a user-defined limitation strategy and possibly updated in light of move execution results. After initialization, an improving move is searched for each vertex v in the set. If such a move is found, the customers associated with it are optionally added to the set. On the other hand, the vertex v is optionally removed from the set if an

improving move cannot be found. Several limitation and update strategies are tested for the CVRP by using a memetic algorithm [38]. These strategies include limiting the search to endpoints of edges changed during crossover and mutation, restricting to vertices belonging to routes in children having at least a change compared to the parents, or only considering vertices from which an improving move is found.

Similarly to [37], [3] use a technique called *selective vertex caching* in an iterated local search metaheuristic where the perturbation is performed in a ruin-and-recreate fashion. The idea is to mainly re-optimize the area disrupted by the ruin-and-recreate application. To this end, a cache tracks the vertices involved in the disruption process. Then, subsequent local search procedures start only from the vertices in such a cache. In addition, whenever a change is applied to the solution, for example, due to the execution of an improving move, related vertices are added to the cache. However, instead of removing vertices that do not lead to improving changes, the cache is limited in size and managed with a least recently changed eviction policy that constantly keeps the optimization focused on more recently successfully changed areas. The cache is combined with the static move descriptor framework described in section "Static Move Descriptors" by initializing only SMDs identified by vertices belonging to the cache at the beginning of the neighborhood exploration.

Additional Design and Implementation Challenges

In this section, we briefly overview some additional technical challenges that arise when working with very large-scale instances, and we provide practical solutions described in the literature, for implementing scalable VRP heuristics. A common theme consists of using efficient data structures with a small memory footprint. Particular care should also be given to routine operations such as copying solutions, identifying neighbor nodes, or retrieving edge costs. Indeed, as the instance size grows and the optimization procedure becomes more efficient, these operations may turn out to contribute significantly to the overall computing time, while being accessory but still critical algorithm components. Overall, careful and reasonably good implementations are considerably more important here than when approaching smaller instances, as trivial inefficiencies are greatly amplified.

Memory Usage

Despite the great availability of random access memory (RAM) in modern computers, dealing with very large-scale VRPs may still require extra care in this regard, because of the super-linear growth of the memory use with respect to the instance size. The cost matrix and the list of neighbors of every vertex are typically among the most memory-consuming data structures, as their dimension grows quadratically with the instance size.

In [33], the authors developed their approach using a machine with the widely adopted 32-bit architecture, having a total addressable memory of 4 GB. To tackle very large-scale instances, they then proposed storing both the distance

(corresponding to the cost) and the travel time for each directed pair of vertices in a highly compressed format. First, they encoded both distance and travel time in a single word (2 bytes), thus halving the memory requirements. In addition, they noted that the travel time could be replaced with the average speed between two vertices, using fewer bits, and still achieving sufficient precision. This leaves more bits available for encoding the distance. Moreover, distances are not stored directly, but rather using twisty road curve factors. This representation allows storing for each pair of vertices their twisty road curve factor and average speed in a very compact way using 8 or 16 bits according to the required precision. Given that for common processors, memory accesses are much more expensive than executing calculations, original distances and travel times can be recomputed whenever required without significant performance penalties.

By analyzing near-optimal CVRP solutions with a gap smaller than 1%, [6] find that, for most instances, 95% of the vertices are connected to the 20 closest neighbors. Moreover, for 99% of the vertices, the two adjacent vertices in the route belong to the nearest 50 neighbors. There are, however, exceptions requiring long edges, for example, in instances with high variance in the demands or showing clusters of vertices. For the above reasons, the authors only store edge costs between a vertex and its 100 nearest-neighbor customers. The neighbors of a vertex are determined by sorting, for every vertex i, a list of tuples (j, c_{ij}) with $i, j \in V$. The selected number of tuples is then stored in a hash map for efficient cost retrieval. Whenever a cost is needed, the hash map is checked first, and if the value is not readily available, it is computed on demand.

In contrast to [6], in [4] all edge costs are always computed on demand. Their experiments show that, for costs corresponding to simple Euclidean distances, additional caching does not seem to be beneficial. In addition, every solution keeps track of the costs of edges composing it, allowing constant-time access to part of the costs required during algorithm procedures. Furthermore, given that local search procedures are designed according to the granular neighborhood principles described in section "Granular Neighborhoods," a local search move identified by a move generator (i, j) will always try to insert the associated edge into the solution. Thus, similarly to [25] for the TSP, the edge cost c_{ij} can be associated with the available move generators (i, j) for an additional constant-time retrieval. Finally, as in [6], a limited number of neighbors is computed and stored for every vertex. However, to avoid expensive sorting operations, neighbors are efficiently identified using a k-d tree data structure built on top of instance vertex coordinates [10]. Note that in [12], the authors identify neighbors using a radix sort and show that it can be an appropriate alternative to a more traditional comparison-based sorting approach for instances with several thousand customers.

Solution Management

A core component of a VRP heuristic is the representation and manipulation of the solution. Such an object should, in general, provide efficient ways to operate on its content according to the procedures performed by the algorithm.

A commonly used representation for large-scale VRPs is the so-called *next array* [33]. This representation combines the advantages of constant node insertion and removal given by linked lists with the data locality of contiguous memory storage. In particular, the solution is stored in a contiguous array with a size equal to the number of vertices, complemented by a separate auxiliary *start array* storing the first customer of each route. An optional *prev array* can also be used to represent a doubly linked list, if convenient.

Another popular representation is the *giant tour* [26, 28, 54, 57]. A giant tour encodes a VRP as a TSP with optional depot copies delimiting VRP routes. As an example, [28] suggest using two arrays with a length of the number of customers plus depot copies. The *node* array gives for each giant tour position the vertex that occupies it, while the *pos* array gives, for each node, its position in the giant tour. Route delimiters can be optimally placed using a dynamic programming-based linear split procedure based on dynamic programming [53], which makes this representation particularly useful in evolutionary algorithms, as it facilitates the combination of solutions with crossover procedures.

A common operation in metaheuristics involves copying solutions. For example, a copy is made to update the best found solution as soon as a candidate neighbor solution is found to be better, or to update the working solution in iterative approaches using specific acceptance criteria. According to the algorithm optimization pattern, it may not be worthwhile to perform a complete solution copy. In fact, despite being a linear-time operation, it may become extremely expensive when working with solutions of very large-scale instances. For this reason, in [4], the copy operation is replaced by an incremental synchronization between the best and the current working solutions. Initially, two identical solutions are created. Any change to the current working solution is stored in the so-called *do-* and *undo-lists*. The undo-list allows for the restoration of the working solution at any previous point in time by undoing part of the stored changes, for example, to backtrack to a better working solution. Similarly, the do-list can be used to update the best solution by applying all changes separating the best from the current working solution, for example, when the latter becomes better than the former. Synchronization between solutions is no longer a linear procedure, but it is more dependent on the algorithm optimization pattern. This mechanism has been shown to be effective in approaches for very large-scale instances that iteratively focus their optimization on localized and limited areas.

Initial Solution Construction

An initial solution is the starting point where further optimization procedures are applied. The main idea behind most approaches is to efficiently perform the preprocessing required by the construction algorithm, possibly trading its exhaustiveness with faster execution time and smaller memory requirements.

In [33], the initial solution is created using a cheapest insertion heuristic, where routes are created sequentially until all customers are routed. A new route is

initialized with a seed customer, and the unrouted customers are then added one at a time. To limit computational effort, only unrouted customers with the smallest insertion cost are considered for insertion on the current route. The next best customer to insert is identified with priority queues containing a limited and constant number of the smallest insertion costs and associated customers.

Another popular approach is the savings algorithm proposed by [16]. This construction heuristic computes an entry for every pair of customers, representing the savings that could be obtained by connecting the respective customers. These savings are then sorted and considered for application starting from the most improving one. The memory occupation as well as the savings computation grows quadratically with the instance size. Moreover, expensive sorting of all entries is required to apply the most promising savings first, making the algorithm unsuitable for very large-scale instances. Therefore, [6] propose a variation that only calculates a linear number of savings. In particular, for a vertex i, only savings connecting i to a limited set of k nearest neighbors $j \in \mathcal{N}^k(V \setminus \{i\})$ are considered. Observing that in most high-quality solutions, vertices are typically connected to other geographically close vertices, the authors propose $k = 100$ and show that this choice allows the construction of good-quality solutions in considerably short computing time and with small memory requirements. The same approach is successfully used in [12] in combination with resource extension functions [19, 27], to build an initial solution to a more constrained VRP, and in [4] to solve extremely large-scale instances.

Finally, optimizing the initial solution construction step should be weighed against the overall solution approach, as the subsequent improvement phases are typically considerably more computationally demanding than this initial step.

Resources Processing

For many constrained VRPs, the feasibility of candidate changes can be checked in constant time, provided that additional data are available. The *segment resource extension function* framework, proposed by [27], is a popular approach to formally describe how to treat common constraints. It combines ideas from resource extension functions proposed in [19] and information handling techniques introduced in [30], providing an excellent compromise between flexibility of use and efficiency of computation.

In this framework, the additional data are referred to as *resources*. Resources are typically associated with instance vertices and are computed on the basis of the position of the vertex in its route. The feasibility checks are then performed in constant time by combining the resources associated with sequences of vertices, also called *segment resources*, according to the candidate change.

For example, consider the CROSS-exchange move depicted in Fig. 4, swapping the sequences $(i_1, \ldots, i_n)$ and $(j_1, \ldots, j_m)$ between routes r_i and r_j. The feasibility assessment of such a move for route r_i can be performed by combining segment resources associated with the sequences $(0, \ldots, \pi_{i_1})$, $(j_1, \ldots, j_m)$, and $(\sigma_{i_n}, \ldots, 0)$. Similar reasoning applies to the route r_j.

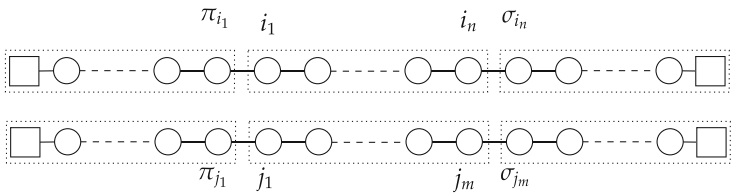


Fig. 4 Segments for the evaluation of a CROSS-exchange move swapping sequences $(i_1, \ldots, i_n)$ and $(j_1, \ldots, j_m)$ from routes r_i and r_j, respectively. The square denotes the depot

Segment resources must be computed and updated as the underlying solution changes. The two main opposite approaches consist in precomputing these resources or computing them on demand whenever required. The selection of one approach over the other depends on the balance between the number of moves evaluated and those leading to actual solution changes. Indeed, the higher the number of moves tested without application, the more efficient is having segment resources precomputed and readily available. On the other hand, having precomputed segment resources for a solution that constantly changes may incur in expensive update operations.

Irnich [26] propose a hierarchical approach that trades preprocessing effort with an increased number of combinations of segment resources required to evaluate a candidate change. In particular, this is proposed for constrained VRPs described by a giant tour representation; see also section "Solution Management." Instead of precomputing all segment resources between every pair of nodes, which would require both a quadratic procedure and memory occupation, the authors define a number of seed points to divide the vertices of the giant tour uniformly into sections. The segment resources are then computed only for every pair of nodes within a section, delimited by two seed points, and between all pairs of seed points. Using this approach, any arbitrary segment resource associated with a sequence from vertex i to j can be obtained by combining at most three already computed resources. In particular, if i and j lie in the same section, the associated segment resource is already available. If i and j are in two consecutive sections divided by a seed point s, then the desired segment resource can be computed combining resources associated with the sequences $(i, \ldots, s)$ and $(s, \ldots, j)$. Finally, when there are at least two seed points between i and j, with s_1 the first seed point after i, and s_2 the last seed point before j, the required segment resource can be calculated by combining the resources associated with $(i, \ldots, s_1)$, $(s_1, \ldots, s_2)$ and $(s_2, \ldots, j)$. This approach can be generalized to a hierarchy of segments by introducing multiple levels of seed points, trading a lighter preprocessing for an increased number of combinations. As an example, [32] successfully used a two-level approach to solve a VRP with time windows.

In [12], segment resources are used to efficiently perform constant-time feasibility checks for a VRP with simultaneous pickup and deliveries. For each route, segment resources are computed for sequences of increasing size starting from the depot, as well as from every customer to the depot. Initially, this is done for the entire

solution, and then whenever a route is changed, the associated resources are updated. Following the approach used in [55], the authors performed experiments in which all segments up to size three are precomputed for the current solution. The choice of such a size is dictated by having local search procedures that can swap at most three consecutive vertices. However, this additional preprocessing was not found to be beneficial because the maximum segment size was small enough to allow a quick computation of such resources. Moreover, for very large-scale instances, having this information associated with a solution greatly increases the cost of copy operations if such preprocessed resources are preserved. This is especially problematic for approaches that optimize a very limited portion of the solution at every iteration but still require a full solution copy to keep track of the current solution, as the generated neighbor where local search is applied may later be discarded due to the neighbor acceptance criterion [3, 12].

In [4], cumulative customer demands are used to efficiently perform inter-route 2-opt feasibility checks for extremely large-scale CVRP solutions. As in [12], segment resources are calculated for every route for the sequences from the depot to each customer and from each customer to the depot. However, these resources are computed on demand only when required and cached until there is a change to the associated route. In this case, as mentioned in section "Solution Management," the algorithm never makes solution copies. Instead, whenever a neighbor is rejected by the acceptance criterion, the current working solution is restored by undoing previously applied changes. Cached segment resources affected by this process are marked as no longer valid and recomputed when needed.

More Limitation Strategies

The recurring theme throughout the chapter is avoiding brute-force naive implementations by carefully limitating the options explored during the optimization process while still identifying good-quality solutions. So far, we have described what we consider to be the most relevant scenarios in which this idea has been applied. However, there are more areas where it can be successfully applied.

Consider, for example, a ruin-and-recreate approach. In very large-scale instances, evaluating all candidate reinsertions for unrouted customers removed during the ruin step can be extremely time-consuming. In practice, the likelihood of finding good insertion positions far away from the location of the removed customer is extremely low. A common approach, closely related to candidate lists and granular neighborhood concepts, consists of only considering a subset of promising insertion positions. For example, in [12], reinsertion of unrouted customers is limited only to positions before and after a fixed number of nearest neighbors. Similarly, in [4], all the positions in routes close to an unrouted customer are considered, although this may include distant positions for certain route shapes.

When performing local search procedures, it is often convenient to limit the exploration to reasonable sizes. As an example, [6] limit the maximum lengths of sequences exchanged during a CROSS-exchange procedure to three. In fact,

larger values seldom provide substantial improvement to the quality of solutions but greatly increase the overall running time. Similar findings have been reported for the length of ejection chains [3,6]. Exploring very long sequences of relocations is rarely useful and is extremely computationally expensive.

Sometimes, random sampling can be an alternative to complete evaluation. For example, [4] use a simulated annealing acceptance criterion to guide the algorithm's search trajectory [31]. To handle instances with significantly different edge costs, the simulated annealing temperature is defined to be proportional to the average edge costs of the instance. However, given the size of the extremely large-scale instances considered, and the underlying full graph representation, the exact temperature computation cannot be performed in a reasonable time. Instead, the temperature value is estimated randomly sampling a fixed number of instance edges. Experimental results show that, overall, this does not have a significant impact on the search trajectory of the algorithm.

Conclusions

This chapter reviews some of the most effective techniques that enable VRP heuristics to scale up to extremely large-scale problem instances. The underlying main idea is the usage of efficient procedures and data structures with acceptable memory requirements. Often, these techniques can be tuned to achieve the desired compromise between search exhaustiveness, execution speed, and memory usage. Local search plays a major role in most successful heuristics for VRPs. Therefore, acceleration and pruning techniques become essential components for successfully exploring neighborhoods that would otherwise be out of reach. Finally, routine operations should also be designed with extreme care to craft an overall efficient algorithm.

Cross-References

- A History of Metaheuristics
- Genetic Algorithms
- Guided Local Search
- Heuristics in Vehicle Routing
- Iterated Local Search
- POPMUSIC
- Tabu Search
- Theory of Local Search
- Variable Neighborhood Descent
- Variable Neighborhood Search

Acknowledgments We kindly thank Francesco Cavaliere for reading the manuscript and suggesting improvements and corrections. The research of Daniele Vigo is partially supported by the U.S. Air Force Office of Scientific Research (Grant FA8655-21-1-7046), by H2020 project TUPLES and by the European Union – NextGenerationEU under the National Recovery and Resilience Plan (PNRR) – Mission 4 – Component 2 – Investment 1.1, Notice Prin 2022 – (DD N. 104 2/2/2022) – Project Time-dependent optimization for sustainable transportation – Proposal 20223MHHA8_002 – CUP J53D23003560006.

Competing Interest Declaration The author(s) has no competing interests to declare that are relevant to the content of this manuscript.

References

1. Aarts E, Lenstra JK (eds) (2003) Local search in combinatorial optimization. Princeton University Press
2. Accorsi L, Vigo D (2020) A hybrid metaheuristic for single truck and trailer routing problems. In: Transp Sci 54(5):1351–1371. https://doi.org/10.1287/trsc.2019.0943
3. Accorsi L, Vigo D (2021) A fast and scalable heuristic for the solution of large-scale capacitated vehicle routing problems. In: Transp Sci 55(4):832–856
4. Accorsi L, Vigo D (2024) Routing one million customers in a handful of minutes. Comput Oper Res 164:106562. ISSN: 0305-0548
5. Applegate DL et al (1999) Finding tours in the TSP
6. Arnold F, Gendreau M, Sörensen K (2019) Efficiently solving very large-scale routing problems. Comput Oper Res 107:32–42. ISSN: 0305-0548
7. Becker C et al (2023) In-depth analysis of granular local search for capacitated vehicle routing. Discrete Appl Math 329:61–86. ISSN: 0166-218X
8. Beek O et al (2018) An efficient implementation of a static move descriptor-based local search heuristic. Comput Oper Res 94:1–10. ISSN: 0305-0548
9. Bentley JL (1990) Experiments on traveling salesman heuristics. In: ACM-SIAM symposium on discrete algorithms
10. Bentley JL (1990) K-d trees for semidynamic point sets. In: Proceedings of the sixth annual symposium on computational geometry, pp 187–197
11. Bulho˜es T et al (2018) The vehicle routing problem with service level constraints. Eur J Oper Res 265(2):544–558. ISSN: 0377-2217
12. Cavaliere F et al (2024) An efficient heuristic for very large-scale vehicle routing problems with simultaneous pickup and delivery. Transp Res Part E: Logist Transp Rev 186:103550. ISSN: 1366-5545
13. Chevalier C, Safro I (2009) Comparison of coarsening schemes for multilevel graph partitioning. In: Stützle T (eds) Learning and intelligent optimization. Springer, Berlin/Heidelberg, pp 191–205. ISBN: 978-3-642-11169-3
14. Christiaens J, Berghe GV (2020) Slack induction by string removals for vehicle routing problems. Transp Sci 54(2):417–433. https://doi.org/10.1287/trsc.2019.0914
15. Christofides N, Eilon S (1972) Algorithms for large-scale travelling salesman problems. Oper Res Q (1970–1977) 23(4):511–518. ISSN: 00303623
16. Clarke G, Wright JW (1964) Scheduling of vehicles from a central depot to a number of delivery points. Oper Res 12(4):568–581. https://doi.org/10.1287/opre.12.4.568
17. Costa L, Contardo C, Desaulniers G (2019) Exact branch-price-and-cut algorithms for vehicle routing. Transp Sci 53(4):946–985. https://doi.org/10.1287/trsc.2018.0878
18. Dantzig GB, Ramser JH (1959) The truck dispatching problem. Manag Sci 6(1):80–91. https://doi.org/10.1287/mnsc.6.1.80

19. Desaulniers G et al (1998) A unified framework for deterministic time constrained vehicle routing and crew scheduling problems. In: Crainic TG, Laporte G (eds) Fleet management and logistics. Springer, Boston, pp 57–93. ISBN: 978-1-4615-5755-5
20. Glover F (1996) Ejection chains, reference structures and alternating path methods for traveling salesman problems. Discrete Appl Math 65(1):223–253. First International Colloquium on Graphs and Optimization. ISSN: 0166-218X
21. Glover F, Laguna M (1997) In: Glover F, Laguna M (eds) Tabu search. Springer, New York
22. Goeke D, Gschwind T, Schneider M (2019) Upper and lower bounds for the vehicle-routing problem with private fleet and common carrier. Discrete Appl Math 264:43–61. Combinatorial Optimization: between Practice and Theory. ISSN: 0166-218X
23. Groër C, Golden B, Wasil E (2009) The consistent vehicle routing problem. Manuf Serv Oper Manag 11(4):630–643. https://doi.org/10.1287/msom.1080.0243
24. Groër C, Golden B, Wasil E (2011) A parallel algorithm for the vehicle routing problem. INFORMS J Comput 23(2):315–330. https://doi.org/10.1287/ijoc.1100.0402
25. Helsgaun K (2000) An effective implementation of the Lin–Kernighan traveling salesman heuristic. Eur J Oper Res 126(1):106–130. ISSN: 0377-2217
26. Irnich S (2008) A unified modeling and solution framework for vehicle routing and local search-based metaheuristics. INFORMS J Comput 20(2):270–287. https://doi.org/10.1287/ijoc.1070.0239
27. Irnich S (2008) Resource extension functions: properties, inversion, and generalization to segments. OR Spectrum 30:113–148. ISSN: 0305-0548
28. Irnich S, Funke B, Grünert T (2006) Sequential search and its application to vehicle-routing problems. Comput Oper Res 33(8):2405–2429. ISSN: 0305-0548
29. Kernighan BW, Lin S (1970) An efficient heuristic procedure for partitioning graphs. Bell Syst Tech J 49(2):291–307
30. Kindervater GAP, Savelsbergh MWP (2003) 10. Vehicle routing: handling edge exchanges. In: Emile A, Lenstra JK (eds) Local search in combinatorial optimization. Princeton University Press, Princeton, pp 337–360. https://doi.org/10.1515/9780691187563-013
31. Kirkpatrick S, Gelatt CD, Vecchi MP (1983) Optimization by simulated annealing. Science 220(4598):671–680. ISSN: 0036-8075. https://science.sciencemag.org/content/220/4598/671.full.pdf
32. Kool W et al (2022) Hybrid genetic search for the vehicle routing problem with time windows: a high-performance implementation. In: 12th DIMACS implementation challenge workshop. DIMACS. Rutgers University, Piscataway
33. Kytöjoki J et al (2007) An efficient variable neighborhood search heuristic for very large scale vehicle routing problems. Comput Oper Res 34(9):2743–2757. ISSN: 0305-0548
34. Lin S, Kernighan BW (1973) An effective heuristic algorithm for the traveling-salesman problem. Oper Res 21(2):498–516. https://doi.org/10.1287/opre.21.2.498
35. Lourenço HR, Martin OC, Stützle T (2003) Iterated local search. In: Handbook of metaheuristics. Springer, Boston, pp 320–353. ISBN: 978-0-306-48056-0
36. de Holanda Maia MR, Plastino A, dos Santos Souza U (2024) An improved hybrid genetic search with data mining for the CVRP. Networks 83(4):692–711. https://doi.org/10.1002/net.22213
37. Martin OC, Otto SW, Felten EW (1991) Large-step Markov chains for the traveling salesman problem. Complex Syst 5:299–326
38. Moscato P (2000) On evolution, search, optimization, genetic algorithms and martial arts – towards memetic algorithms. In: Caltech concurrent computation program
39. Nagata Y, Bräysy O (2008) Efficient local search limitation strategies for vehicle routing problems. In: van Hemert J, Cotta C (eds) Evolutionary computation in combinatorial optimization. Springer, Berlin/Heidelberg, pp 48–60. ISBN: 978-3-540-78604-7
40. Pecin D et al (2016) Improved branch-cut-and-price for capacitated vehicle routing. Math Program Comput 9:61–100
41. Pessoa A et al (2020) A generic exact solver for vehicle routing and related problems. Math Program 183(1):483–523. ISSN: 1436-4646

42. Pisinger D, Ropke S (2007) A general heuristic for vehicle routing problems. Comput Oper Res 34(8):2403–2435. ISSN: 0305-0548
43. Potvin J-Y, Lapalme G, Rousseau J-M (1989) A generalized K-opt exchange procedure for the MTSP. INFOR: Inf Syst Oper Res 27(4):474–481. https://doi.org/10.1080/03155986.1989.11732113
44. Queiroga E, Sadykov R, Uchoa E (2021) A POPMUSIC matheuristic for the capacitated vehicle routing problem. Comput Oper Res 136:105475. ISSN: 0305-0548
45. Santini A et al (2023) Decomposition strategies for vehicle routing heuristics. INFORMS J Comput 35(3):543–559. https://doi.org/10.1287/ijoc.2023.1288
46. Schneider M, Schwahn F, Vigo D (2017) Designing granular solution methods for routing problems with time windows. Eur J Oper Res 263(2):493–509
47. Schrimpf G et al (2000) Record breaking optimization results using the ruin and recreate principle. J Comput Phys 159(2):139–171. ISSN: 0021-9991
48. Taillard É et al (1997) A tabu search heuristic for the vehicle routing problem with soft time windows. Transp Sci 31(2):170–186. https://doi.org/10.1287/trsc.31.2.170
49. Taillard ÉD, Voss S (2002) Popmusic – partial optimization metaheuristic under special intensification conditions. In: Essays and surveys in metaheuristics. Springer, Boston, pp 613–629. ISBN: 978-1-4615-1507-4
50. Toth P, Vigo D (2003) The granular tabu search and its application to the vehicle-routing problem. INFORMS J Comput 15(4):333–346
51. Toth P, Vigo D (2014) In: Toth P, Vigo D (eds) Vehicle routing. Society for Industrial and Applied Mathematics, Philadelphia. https://doi.org/10.1137/1.9781611973594
52. Uchoa E et al (2017) New benchmark instances for the capacitated vehicle routing problem. Eur J Oper Res 257(3):845–858. ISSN: 0377-2217
53. Vidal T (2016) Technical note: split algorithm in O(n) for the capacitated vehicle routing problem. Comput Oper Res 69:40–47. ISSN: 0305-0548
54. Vidal T (2022) Hybrid genetic search for the CVRP: open-source implementation and SWAP* neighborhood. Comput Oper Res 140(C). ISSN: 0305-0548
55. Vidal T et al (2013) A hybrid genetic algorithm with adaptive diversity management for a large class of vehicle routing problems with time-windows. Comput Oper Res 40(1):475–489. ISSN: 0305-0548
56. Vidal T et al (2013) Heuristics for multi-attribute vehicle routing problems: a survey and synthesis. Eur J Oper Res 231(1):1–21. ISSN: 0377-2217
57. Vidal T et al (2014) A unified solution framework for multi-attribute vehicle routing problems. Eur J Oper Res 234(3):658–673
58. Walshaw C (2002) A multilevel approach to the travelling salesman problem. Oper Res 50(5):862–877. https://doi.org/10.1287/opre.50.5.862.373
59. Zachariadis EE, Kiranoudis CT (2010) A strategy for reducing the computational complexity of local search-based methods for the vehicle routing problem. Comput Oper Res 37(12):2089–2105. ISSN: 0305-0548

Supply Chain Management

56

Helena Ramalhinho Lourenço and Martín Gómez Ravetti

Contents

Introduction 1698
What Is Supply Chain Management? 1699
Decision-Making Problems in SCM 1701
Metaheuristic Algorithms for SCM Problems 1706
Current and Future Research Trends in SCM 1708
Conclusions 1710
Cross-References 1710
References 1711

Abstract

Supply chain management (SCM) is related to the management of all activities along a network of organizations to provide a good or a service to final customers. The efficiency of these activities can have a great impact on customer's satisfaction and cost reduction. However, SCM is not just the sum of activities along the supply chain but, instead, it must consider the organization, supervision, and control of all activities in the chain from an integrated and collaborative perspective aiming to provide a competitive advantage. From this point of view, an increase in the dimension and complexity of the decision problems involved is expected, as several actors with different goals must be considered to administrate efficiently the activities within the supply chain.

H. R. Lourenço (✉)
Universitat Pompeu Fabra, Barcelona, Spain
e-mail: helena.ramalhinho@upf.edu

M. G. Ravetti
Universidade Federal de Minas Gerais, Belo Horizonte, Brazil
e-mail: martin.ravetti@dep.ufmg.br

© Springer Nature Switzerland AG 2025

R. Martí et al. (eds.), *Handbook of Heuristics*,
https://doi.org/10.1007/978-3-032-00385-0_54

This chapter briefly reviews the main concepts of SCM, identifying relevant decision and optimization problems and discussing possible solution approaches. Heuristics and metaheuristics are two of the best optimization tools to be used in solving and providing business insights for the SCM problems. This chapter also describes some successful metaheuristic approaches to SCM and it examines future research trends. A large number of applications of metaheuristics to SCM integrating new subjects, such as open and big data, smart cities, and online decision-making, just to mention a few, are foreseen.

Keywords

Supply chain management · Optimization problems · Metaheuristics

Introduction

Supply chain management (SCM) consists in integrating processes by linking major business functions from a particular company as well as across companies with business relationships. SCM is more than just summed up of a series of interconnected activities among business; it consist of integrated and collaborative processes within a clear business model that leads to a more cohesive and efficient performance of companies and better customer services. Being able to optimize the activities and processes along a supply chain is strategical for most businesses.

The activities and processes in the supply chain management include topics like location and network design, sourcing and inventory strategies, manufacturing and resource strategies, sustainability and green strategies, and also transportation and distribution strategies. At the operational level, we can find different topics as for example customer service management, demand management, returns management etc. To efficiently manage these areas, in many situations large-scale complex combinatorial optimization problems must be solved in a very effective way to be able to answer the increasingly demanding market.

Companies like INDITEX, AMAZON, DELL, etc. (just to mention some) have used and applied the concept of supply chain management to improve their processes and customer service with great success. These companies manage all activities along the supply chain in a collaborative way with the final objective to serve efficiently the final customer.

This chapter argues that metaheuristics can be an outstanding tool to help in the SCM decision-making. On the one hand, problems associated with SCM are becoming more complex and bigger causing greater impact on business performance. On the other hand, metaheuristics have attributes (as accuracy, speed, simplicity, and flexibility [1]) that can help dealing with problems that have arisen in SCM.

The objective of the chapter is to emphasize previous and potential metaheuristic's applications to SCM problems. We also revised some of the most relevant work in the literature that combines and apply metaheuristics to problems in the area of supply chain management.

The chapter is organized as follows: The first two sections describe main concepts and decision-making problems in SCM. Next, relevant applications of metaheuristics to different decision-making problems in SCM are presented and discussed. Relevant current and future topics in SCM where heuristics and metaheuristics can have a significant impact on the management of a supply chain are also examined. Finally, conclusions are drawn.

What Is Supply Chain Management?

Supply chain management has been a very popular topic in the last decade not only in business but also in academia. However, it is still possible to find some ambiguity in the concept and definition of SCM [2]. In this section, the most relevant definitions are presented and discussed to clarify SCM concepts. This definition will help us to identify the optimization problems within SCM and consequently the applications of Metaheuristics to this area.

A supply chain is a set of organizations, activities, people, resources, and products that interact to provide a service or a good to a customer, from the raw material initial stages to the end users and backward when reverse logistics is relevant. Therefore, it is a network of elements interacting with each other with the objective of satisfying customers while minimizing costs. The management of all activities along the supply chain is known as supply chain management. These activities include logistics, manufacturing and operations, sourcing, distribution, sales and marketing, accounting and finance, product design, and information technology activities among others that can be specific for each type of industry. The supply chain is related to the flow of products and goods along the chain, but information flow also plays a relevant role in its efficient management. Another related matter in SCM is the collaboration and integration of the activities in the chain since decisions in one element affect directly the whole supply chain. Thus, decision based on suboptimization approaches may lead to enormous inefficiencies in the chain. Consequently, the decision-making problems in SCM are usually more complex and have greater dimensions than other decision problems in traditional individual businesses.

In the literature, several definitions can be found. However, in this chapter, only two of the most commonly mentioned are analyzed.

The Council of Supply Chain Management Professionals (CSCMP) defines SCM as: "Supply Chain Management encompasses the planning and management of all activities involved in sourcing and procurement, conversion, and all logistics management activities. Importantly, it also includes coordination and collaboration with channel partners, which can be suppliers, intermediaries, third-party service providers, and customers. In essence, supply chain management integrates supply and demand management within and across companies" [2].

Simchi-Levi et al. [3] describes SCM as: "... a set of approaches utilized to efficiently integrate suppliers, manufacturers, warehouses, and stores, so that merchandise is produced and distributed in the right quantities, to the right locations,

and at the right time, to minimize system-wide costs while satisfying service level requirements."

Janvier-James [4] presents a discussion on the lack of consensus in the definitions of SCM, a list of definitions found in the literature, and review on the theory and practice of SCM. In Chap. 2 of [5], it can be found a history of SCM and also a discussion on its definition.

When focusing on the common aspects of these definitions, it is possible to see that a supply chain can be understood as a set of interconnected entities and activities which are concerned with the plan, coordination, and control of materials, parts, and finished goods from suppliers to customers. An essential aspect of a successful management of these activities is the integration, cooperation, and coordination of the decision-making processes as well as in the sharing of information throughout the entire supply chain. Integrated decisions in SCM are related to cross functional planning. SCM coordination attempts to maintain an efficient and smooth flow of products and services. An effective SCM is not just a sum of management individual activities, but it must take into account the effect of their interrelation, seeking collaborative actions among the chain. But the importance of concepts as performance and quality service cannot be ignored. The ideas around integration and collaboration arise to improve the customer service and the performance by taking advantages of the synergies among the elements of a supply chain and to have a common objective and align the targets to satisfy the final customer.

In the literature, there is still some uncertainty about the definitions of SCM and Logistics Management (LM). Although, these two areas have a serious overlap; LM is a functional area of business while SCM crosses the businesses borders including relationships and interactions from different entities, as providers and customers for example. The CSCMP defines Logistics Management as follows: "Logistics management is that part of SCM that plans, implements, and controls the efficient, effective forward and reverses flow and storage of goods, services and related information between the point of origin and the point of consumption to meet customers' requirements" [2].

Other similar definition can be found in Johnson et al. [6]: "Logistics define the entire process of materials and products moving into, through, and out of a firm. Inbound logistics covers the movement of materials received by the suppliers. Material management describes the movements of materials and components within a firm. Physical distribution refers to the movement of goods outwards from the end of the assembly line to customers. Finally, SCM is a larger concept than logistics, dealing with managing both the flow of materials and the relationships among channel intermediaries from the point of origin of raw materials through to the final consumer."

Supply chain management gives a particular emphasis on the integration of the entire supply chain, including the logistics, marketing, operations and other activities within the organization, and with suppliers, suppliers of suppliers, customers, and customers of customers, etc. Therefore, in this chapter, we focus on cross functional aspects of the business collaboration and integrated activities as well as the decisions in SCM.

Decision-Making Problems in SCM

The decision-making in SCM is very complex as it may involve a large number of elements and processes, different businesses, or economic functions within each business. This section presents some general and relevant decision-making problems in SCM and relates them to optimization problems. Depending on the segment of the economy or industry sector, the SCM may have specific characteristics, sharing common difficulties. However, this chapter focuses on problems dealing with integration and collaboration issues along the supply chain.

The decision-making problems may be categorized into two groups: planning (long-term decision-making) and execution (tactical and short-term decision-making).

In planning, the most relevant decisions relate to the following areas: network design, sourcing and inventory strategies, manufacturing and resource strategies, sustainability and green strategies, transportation and distribution strategies, information strategies, and global supply chain strategies [7–10].

In execution, it is possible to identify several processes related to supply, manufacturing, logistics and distribution, and post-sale [11]. The problems that arise from these processes share cross functional issues along the chain.

How should the supply chain efficiency be evaluated? The main criteria mentioned in the literature are speed, reliability, cost, and customer satisfaction [4]. Other authors suggest resources measures (cost), output measures (customer related), or flexibility measures [10]. Therefore, the decision problems that can occur in SCM are in general related to finding solutions that perform well for the mentioned criterions.

Next, some examples of decision-making problems in a supply chain are described, emphasizing the ones where metaheuristics have been or can be applied. Please note that there is no intention to make an exhaustive list of problems but exemplify some important problems in SCM where heuristics and metaheuristics can be successfully applied. Some of these problems are already well known in the operations research literature; however very few of them have considered the integration and coordination aspect of SCM.

In SCM planning, two of the most relevant subjects are network design and facility location problems. Location problems have been extensively studied in operations research, and there is a large amount of applications of heuristics and metaheuristics to this area. However, the problems in this chapter on *location and network design* are problems from an integrated and collaborative perspective within a supply chain. For a survey on managerial issues in supply chain and network design, see [11]. The authors describe the main issues in network design, as determining its size, number and locations of the facilities, integration with tactical decisions as distribution, transportation, and inventory policies, as well as fulfilling customers' demand. They also evaluate supply chain network design where competition among supply chains is taking into account in the design phase. For the optimization point of view, the authors mention interesting objectives that should be considered in the network design problem as social, environmental,

economical, agility, and uncertainty considerations. Melo et al. in [12] presented a literature review of facility location models within a SCM context. One interesting aspect of this work is the identification of the key features that these models should capture in the strategic planning of the supply chain. They consider not only the common objectives and constraints of standard location problems but also its integration with other decision aspects of the chains as capacity, production, inventory, transportation, procurement, and routing. All of them received little attention in the literature. Other problems still very relevant in network design problems are globalization and internationalization of the supply chain, financial and taxation factors, risk factors, and multiobjective models (due to the consideration of multiple actors).

The *sourcing and inventory strategy* in SCM refers to several elements, for example, inventory positioning in the network, inventory objectives balancing cost and product availability, pull versus push systems, and inventory policies as VMI (vendor management inventory) or restock policies. Inventory and sourcing represent a large percentage of the supply chain's cost, thus the optimization of these elements is of critical importance. Notice that some decisions related to sourcing and inventory must be taken considering the whole supply chain including the production, transportation, and distribution decisions. These problems are well studied in the literature [13–17]. However, less attention has been done to strategic problems considering a collaborative and integrated point of view. There are few studied problems in inventory and sourcing from SCM perspective, the inventory-routing problem [18, 19] and the vendor-managed inventory [20] are a few examples.

The *manufacturing and resource strategies* are associated with the process of producing a good or a service. Most problems and relevant decisions are related to the production planning and scheduling, global production assignment, postponement strategies, resource utilization, and resource investments. Also, related issues as market forecasts and product design make a great impact on the manufacturing and resource strategies. Once again, many problems can be defined within this subject; however, only recently the supply chain perspective has been considered when defining business strategies for manufacturing and resource use. This broad strategy leads to larger and complex problems if compared with the traditional single business perspective. Production planning within a supply chain can be a very challenging problem, as its solution may affect several elements of the chain, horizontally and vertically. For example, job scheduling has been mostly applied to deal with operational decision problems, but its integration with the production planning decision [21] and among different production plants is not commonly considered. Their integration with the availability of raw materials, the global forecasting, and distribution decisions can also be of great interest in the SCM. An overview of production and scheduling problems in the supply chain can be found on [22, 23].

Sustainability SCM must be taken into consideration from product design and production to transportation and waste management. Good decisions in *sustainability and green strategies* have a significant impact on the economics and market value

of the business. For a literature review on the implications and interrelationships of SCM, sustainability, and lean management, we refer to [24]. The environmental and waste costs are often quite significative. Thus it should be taken into account in almost all SCM decisions mentioned in this chapter, for example, the impact of carbon dioxide emissions in transportation strategies. The design of green and lower environmental impact routes is attracting many researchers [25]. The sustainability or green strategies and returns management should also be considered in network design strategies, the latter is known as closed-loop network design, for example, Devika et al. in [26] apply metaheuristics to solve a similar type of problem successfully.

Transportation and distribution strategies are key elements in SCM. The connections among the elements are based on transportation activities or information interchange. Transportation elements not only permit to connect components in the chain but also it represents in many cases a substantial overall cost to the organizations in the chain. Therefore, managers need to have a good understanding of the transportation system and develop good transportation strategies. To the development and design of these strategies, several elements must be taken into account, i.e., transportation costs and performance, transit time and variability, loss and damage costs, transportation mode, international issues, transportation outsourcing, and collaborative transportation, just to mention a few. From the metaheuristic literature, a potential application is the study of the impact of transportation decisions in the SCM.

Today's information technologies allow companies to collect vast amounts of data related to several activities in the supply chain: from demand data to stock levels data, from the location of the transportation vehicle data to order delivery status, etc. Information strategies should be carefully considered since the impact on the business' performance is significant [27]. In reality, it is difficult to discuss SCM without mentioning information technologies. The capability of integration, collaboration, and coordination along a supply chain will not be possible without the information flow along the network. It should also be easy to understand that a good use of this information may lead to improvements in the decision-making and consequently on the overall performance of the business and customer satisfaction [3]. SCM strategic decisions on information are related to where and which data should be collected, accessed, storage, analyzed, and shared with other supply chain elements [3]. Notice that these decisions can affect all optimization problems, since all of them have the need for high-quality input data [28]. Other significant problems raised in this activity are related to how to deal with the enormous amount of available data. Feature selection and classification problems, as well as other data mining difficulties, must be formulated and solved to improve the information quality. Thus, metaheuristic techniques are usually an appropriate method to solve such large problems in a reasonable amount of time, avoiding suboptimizations, poor data aggregation, and other simplifications procedures. The availability of huge quantities of data will probably lead to a widespread use of metaheuristics and better decisions on SCM.

Several businesses have a worldwide presence, so global supply chain strategy is a relevant topic in SCM. Companies frequently produce globally but with some local personalization to improve sales numbers. So, strategic decisions related to marketing, financial, logistics, distribution, production, and postponement within a global supply chain can represent high complexity but at the same time make a great impact on the performance of the elements of the supply chain. Some of these SCM strategic decisions are, for example, where to produce or store the goods or how to distribute globally produced products and always considering: global market characteristics, technological aspects, global costs, and economic and political scenes [29]. The consideration of all these issues usually leads to very complex problems and eventually with nonlinear objective functions, where again, the use of metaheuristics can be extremely useful.

The characteristics of metaheuristics [1] make them one of the best tools to evaluate different scenarios during strategic decision-making processes. Managers face complex, integrated, and stochastic problems, and being able to obtain relevant insights by using optimization methods certainly provides them with a competitive advantage.

In the SCM execution area, the well-established supply chain processes as defined in Croxton et al. [30] are considered. These processes are:

- Customer relationship management
- Customer service management
- Demand management
- Order fulfillment
- Manufacturing flow management
- Supplier relationship management
- Product development and commercialization
- Returns management

Many other processes are suggested in the literature. However, they are quite similar since all include functional processes in supply, manufacturing, logistics and distribution, and post-sales.

Next, some examples of decision problems that occur in the abovementioned processes are briefly described. It is important to remember that the list of problems is not exhaustive, as the goal is to mention relevant examples with a focus on the use of metaheuristics.

Customer relationship management (CRM) is a well-established area in SCM and closely related to marketing. The objective of CRM is to identify target markets and implement marketing programs with key customers. Most of the literature has a focus on the marketing area. However, recently, there is an increase number of applications of operations research and metaheuristics to marketing and CRM processes. The main reason of this interest is due the increased complexity and dimensions of the decision problems, as for example: identification and segmentation of target customers and optimization of product portfolio, recommendation systems, and characteristics of the potential clients.

Customer service management (CSM) is the coordination structure for managing the business' relations with current and future clients. It includes the management of the sales department and information providers and it must deal with the communication among all departments and companies within the supply chain that provide products or services to customers. This process has a strong component of information systems, and different processes, from inventory problems, assignment of the corresponding customer support or representative, to optimize the sales team's organization. Most examples of applications of metaheuristics in CSM are found in inventory management, as [31, 32], and also in staff scheduling [33].

The main objective of the demand management process is to meet the customer's demand and balance their requirements with the capacities of the organizations in the chain. This is a crucial process, and one of its essential elements is the synchronization between the procurement, production, and distribution function to satisfy customer's demand and orders. Forecasting plays a major role in this process, as well as its integration into the resource planning and allocation [34].

The *order fulfillment* process provides for timely and accurate delivery of customer orders. One of the most relevant decisions in this process is the well-known *capacitated vehicle routing problem* (CVRP) [35]. The CVRP has been extensively studied in operations research area and is one of the problems with the largest number of applications of metaheuristic algorithms [36–39]. But from the SCM perspective, it is important to go beyond the classical CVRP and extend this problem to take into consideration more aspects within the supply chain. For example, the collaborative CVRP, that is, where several distribution companies collaborate to deliver their products [40, 41]. Another example is the CVRP with environmental and reverse logistics considerations [42, 43]. An important area of research is also the consideration of inventory constraints and costs in the routing decisions [18] or rich routing problems as in [44, 45], where metaheuristics have already been successfully applied.

The *manufacturing flow management* process is related to the production of the goods and services to serve the customers, including the moving of products through the plants and businesses. Within an SCM context, the production planning must take into account providers and customers, as well as the production capacity of the organization, differing significantly from the usual individual standpoint. Aspects as the production flexibility, batch size, cycle time, postponement, make to order, make to stock, customer order forecast, production scheduling, etc. must be considered when setting the production flow. Examples of applications of metaheuristics in this process can be found on [46, 47].

The *supplier relationship management* is associated with the relationships upstream in the supply chain and can be seen as a minor image of the CRM. Some of the important decision problems in this process are the selection of the right suppliers, cross docking systems, inventory coordination, and planning between providers and customers. The *product development and commercialization* process integrates all necessary activities to develop new products or services as their introduction to the market. The decision problems in this process are related to activities in the CRM and fulfillment; therefore issues such as market

and promotion planning, sales force training scheduling, make or buy decisions, inventory decisions, transportation planning, or commercialization planning are relevant. One example is the application of genetic algorithms to design different product structures considering different product parts, production processes, and green strategies [48].

The process of *returns management* is related to the reverse logistics and closed-loop supply chain processes. Over the last decade, business and academia have been increasing their attention to this issue due to three main reasons: impact on the environment and laws forcing the right disposal of products, recognition of the sustainable issues by the society, and social responsibility by the businesses. The forward logistics activities are already difficult to plan and optimize. The returns activities have an increased difficulty due to the stochastic aspect of the "demand." For example, if a company uses reused/recycled components in their production, the collection of these components can vary significantly along the time horizon, leading to an increased complexity of the production planning. Some applications of metaheuristics to design closed-loop supply chains have been addressed by [49].

The metaheuristics have already being applied to SCM at the operational level as it will be discussed in more detail in the next section. However, the need to consider integration and coordination aspects still exists, and future applications of metaheuristics to SCM must examine these issues as well as its computational efficiency and flexibility.

Metaheuristic Algorithms for SCM Problems

In the last years, there have been several applications of heuristics and metaheuristics to SCM decision problems. The reasons are quite clear, the increase in complexity due to the horizontal and vertical integration and the need for quick and flexible algorithms granted to metaheuristics, a special place in the methodologies applied to SCM.

In this section, we review relevant publications of heuristic algorithms applied to SCM problems. Note that the literature review is not exhaustive and readers are also referred to a recent survey on this topic [50].

In [51], the problem of distribution planning of a network of manufacturing plants, depots, and customers is analyzed. A heuristic based on an evolutionary algorithm approach (▸ Chap. 22, "Genetic Algorithms") is proposed where the solution consists in deciding which depots should be considered and how the product should be distributed from depots to clients minimizing fixed and delivery costs. The approach considers a bi-level structure; the upper level decides how the customers are supplied from the depots. The lower level decides on the manufacturing process.

The problem of coordinating the production schedule and the transportation of orders is analyzed in [52]. The problem aims to establish the best allocation of orders specifying, at the same time, their production sequence and completion times with the objective of minimizing the total cost of the supply chain. Two genetic algorithms (▸ Chap. 22, "Genetic Algorithms") are proposed with different

integration strategies, where the best approach allocate more CPU time in solving the scheduling problem.

Considering a seven-layer recovery network, including primary customers, collection/redistribution centers, recovery, recycling and disposal centers, and secondary customers, the problem of recovering products is analyzed in [53]. A mixed integer linear programming model is proposed to determine the collection and recycling centers for the logistic of recovered products, minimizing the total cost. To deal with real-sized instances, the authors propose heuristic approach based on Tabu search (▸ Chap. 31, "Tabu Search"), to design and select the best recovery option while minimizing fixed and variable costs.

Considering the same type of problem, but on a multiobjective approach, Devika et al. [26] analyzes a general closed-loop supply chain network with six echelons. They consider the minimization of total costs, the minimization of environmental impacts, and the maximization of social benefits. Considering two metaheuristics, adapted imperialist competitive algorithm (AICA) and variable neighborhood search (VNS) algorithm (▸ Chap. 18, "Variable Neighborhood Search"), three hybridization approaches are designed and tested against benchmark algorithms and on a case study. Other examples of quantitative approaches to green SCM can be found on [54–57].

There are few studies considering the transportation and distribution decisions at the strategic level. Lourenço and Ribeiro [58] explore the impact of considering logistics and marketing issues in the design of distribution strategies. Schmid et al. in [44] describe basic models for extensions of vehicle routing problems in the context of SCM. These extensions consider other elements in the supply chain as scheduling, packaging, batching, intermodality, etc. They also describe an important extension where inventory is considered. The inventory and routing interconnectivity is a well-known area in the operations research literature; see for example [18, 59, 60].

Considering a two-echelon supply chain, Cardona-Vales et al. [61] work on a bi-objective optimization problem, minimizing the cost for opening warehouses and the expected value of transportation costs and the maximum traveling time through the whole supply chain. Locations for warehouses and the assignment between warehouses and distribution centers have to be decided by the actual value of the demand. To deal with this stochastic problem, a hybrid heuristic based on Tabu search and GRASP (▸ Chap. 23, "GRASP") is designed to consider a multiobjective approach.

The design and evaluation of cooperative purchasing strategies for healthcare supply chains are evaluated in [62]. The authors propose hybrid heuristic algorithms based on *variable neighborhood search* and *Tabu search*. Considering a set of hospitals, they want to determine the best cooperation arrangements, to foresee global and individual savings and organize their joint supply chain to take advantage from cooperation.

Regarding *customer relationship management,* several examples of applications of metaheuristics can be found on market basket analysis [63, 64], product portfolio optimization [65], store operation within CRM systems [66], or data mining appli-

cations in CRM [67]. The CRM process is receiving more attention from the SCM point of view; however, we could observe few publications using metaheuristics. We identify that there is great potential to apply these techniques to complex problems in CRM.

Although it is possible to see an increased number of metaheuristic applications to SCM, there are still several problems to be mathematically formulated. As a consequence, there is a lack of proper benchmarks and test beds leading to a deficiency in algorithm comparison and analysis.

Current and Future Research Trends in SCM

Supply chain management will continue to be a relevant topic in the following years, and the decision-making problems will be growing in quantity, dimension, and complexity. In this section, current and future research trends in metaheuristics applied to SCM are discussed. Many of them have not been yet extensively considered in the metaheuristic research literature.

Main topics:

- Greater integration and collaboration, horizontal and vertical, along with the supply chain
- Uncertainty decision-making in SCM
- Multiobjective approaches
- E-commerce and SCM
- Online decision-making in SCM
- Open and big data in SCM
- Smart cities and SCM

One of the main characteristics of SCM is the relevance of integration and collaboration between the elements of the chain. If this integration is not taken into account, there is not a truly SCM setting. However, most current works do not always consider these aspects. We conjecture that the main problem is that incorporating objectives and constraints to model the integration and collaboration of processes lead to very complex and in many cases nonlinear problems. Still, models and algorithms in the SCM decision-making must consider these aspects to obtain a real impact and add value to the business in general.

As previously mentioned, an increase in the dimension and complexity of the problem is expected. Thus, metaheuristics are certainly a clear option to deal with this new problem profile, especially when accurate solutions are needed in fast time. High-performance computing and hybrid methods are clear options to deal with this increased size and complexity. From the application point of view, cloud computing algorithms seem to be a good option for medium-sized companies willing to have access to low-cost distributed algorithms.

Efficient decisions in SCM must take into account an important element: *uncertainty*. Uncertainty is present all over the supply chain: uncertainty demand, travel

times, cost, resource availability, etc. The uncertainty parameters make it difficult to optimize the system-wide costs and performance in SCM. The consideration of scenario strategies, robust methods to deal with the parameter's uncertainty, will play a major role in helping the decision-making. For example, a new line of research called simheuristics [68, 69] extends metaheuristic capabilities through simulation for solving problems with uncertainty parameters.

The need for integration of different elements of the supply chain often leads to optimization problems where many different objective functions must be assessed. Therefore, metaheuristics once again present themselves as an excellent option to efficiently deal with multiobjective problems in a complex environment.

E-commerce is already a reality, and with this new form of commerce had appeared new decision problems [70]. The e-commerce will be shortly a central part in many businesses, and this involves not only the retailing companies but also many other elements integrated into a supply chain, as production and logistics companies. The need to a fast response in e-commerce is also a reality, so the decision-making in a supply chain presents a different set of problems than the traditional ones.

The availability of technology and online data, as well as the need for flexibility and immediate responses, pushes the decision-making processes from an offline to an online perspective. In this new perspective, we do not count with the whole instance data but rather a sequence of input portions that must be solved as they arrive. Thus, online algorithms must take into account the current state of the system and react as a new portion of information arrives. Once again, the trade-off of computer efficiency and solution quality has to be carefully analyzed, but due to the complexity of the problems and the size of real scenarios, metaheuristic approaches are again promising in this new context.

Open email accounts, Internet search browsers, and mobile phones are just some of the new ways to capture online consumer's habit information. This incredible big amount of data represents an excellent opportunity to find commercial advantages. The problem is now how to extract good-quality information from these vast databases that usually cannot fit in regular servers and lacks proper structure [71]. The fact that these databases are relatively open in today's business increases the potential of the impact of this information in the SCM. Due to the characteristics of metaheuristics, these methods have an enormous potential to be applied in SCM decisions involving big and open data.

Although there are several different definitions of a smart city, the main concept behind the idea is to consider a city as an integrated system. The available information from its infrastructure and dynamical systems are continuously analyzed for the improvement of all the systems as energy, transport, healthcare, buildings, etc. [72]. The large amount of sensors and available data have a great impact on urban supply chain management. The availability of the resources of smart cities can provide substantial improvements in mobility, sustainability, distribution, and many other SCM activities. Notice that in a smart city environment, the decision problems in SCM will become more complex, with a need for a faster and online response. For example, retailing companies can identify the closest transportation vehicles and make collaborative distribution decisions online.

These issues are only some of the actual and future relevant topics that must be addressed in SCM. Metaheuristics, as one of the available optimization tools, can provide better quality insights to these important issues in a faster time, improving significantly the decision making.

Conclusions

An efficient supply chain management is of critical importance in today's businesses; if we take into account the market's globalization and the increasing demand for flexibility and quality, a good SCM will be vital for the survival of any company.

This chapter reviews the main concepts and decision problems of SCM. The challenges for obtaining relevant business insights and decisions are associated with methods in the area of heuristics and metaheuristics.

Decision problems that have arisen in SCM are becoming more complex and larger; the need for integrating all the elements of the supply chain is an enormous challenge, as it relates to different objective functions, online decisions, big databases, and uncertainty data. Thus, this chapter argues that heuristics and metaheuristics are the most promising tools to solve the decision problems in SCM. The attributes of metaheuristics, as well as the successful cases mentioned in this work, support the argument that these methods can address the main challenges of the actual and future SCM problems.

A larger number of applications of heuristics in metaheuristics to problems in SCM are foreseeing shortly. The vast amount of available data, the need for more integration and coordination, the need for online solutions, the consideration of uncertainty, and many other new constraints will produce a significant amount of complex decision problems with the need for a fast and efficient solution method. The use of metaheuristics may provide a significant contribution in SCM from a theoretical and practical perspective.

Cross-References

▸ Genetic Algorithms
▸ GRASP
▸ Tabu Search
▸ Variable Neighborhood Search

Acknowledgments H.R. has been supported by the Spanish Ministry of Economy and Competitiveness (TRA2013-48180-C3-P). M.G.R. acknowledges supports from the National Council for Scientific and Technological Development (CNPq) and FAPEMIG.

Competing Interest Declaration The author(s) has no competing interests to declare that are relevant to the content of this manuscript.

References

1. Cordeau JF, Gendreau M, Laporte G et al (2002) A guide to vehicle routing heuristics. J Oper Res Soc 53:512–522. https://doi.org/10.1057/palgrave/jors/2601319
2. CSCMP Supply Chain Management. https://cscmp.org/about-us/supply-chain-management-definitions
3. Simchi-Levi D, Kaminsky P, Simchi-Levi E (2003) Designing and managing the supply chain: concepts, strategies and case studies. McGraw-Hill/Irwin, Boston
4. Janvier-James AM (2011) A new introduction to supply chains and supply chain management: definitions and theories perspective. Int Bus Res 5:194–207. https://doi.org/10.5539/ibr.v5n1p194
5. Ullrich CA (2014) Issues in supply chain scheduling and contracting. Springer Fachmedien Wiesbad, Wiesbaden. https://doi.org/10.1007/978-3-658-03769-7
6. Johnson JC, Wood DF, Wardlow DL, Murphy Paul R (1999) Contemporary logistics. Prentice-Hall, Upper Saddle River
7. Akdogan AA, Demirtas O (2014) Managerial role in strategic supply chain management. Procedia Soc Behav Sci 150:1020–1029. https://doi.org/10.1016/j.sbspro.2014.09.114
8. Lambert DM, Cooper MC (2000) Issues in supply chain management. Ind Mark Manag 29:65–83. https://doi.org/10.1016/S0019-8501(99)00113-3
9. Chopra S, Meindl P (2016) Supply chain management, 6th edn. Pearson Education, Boston
10. Chen IJ, Paulraj A (2004) Understanding supply chain management: critical research and a theoretical framework. Int J Prod Res 42:131–163. https://doi.org/10.1080/00207540310001602865
11. Farahani RZ, Rezapour S, Drezner T, Fallah S (2014) Competitive supply chain network design: an overview of classifications, models, solution techniques and applications. Omega 45:92–118. https://doi.org/10.1016/j.omega.2013.08.006
12. Melo MT, Nickel S, Saldanha-da-Gama F (2009) Facility location and supply chain management – a review. Eur J Oper Res 196:401–412. https://doi.org/10.1016/j.ejor.2008.05.007
13. Gosling J, Purvis L, Naim MM (2010) Supply chain flexibility as a determinant of supplier selection. Int J Prod Econ 128:11–21. https://doi.org/10.1016/j.ijpe.2009.08.029
14. Tachizawa EM, Gimenez C (2010) Supply flexibility strategies in Spanish firms: results from a survey. Int J Prod Econ 124:214–224. https://doi.org/10.1016/j.ijpe.2009.11.020
15. Chikán A (2011) Managers' view of a new inventory paradigm. Int J Prod Econ 133:54–59. https://doi.org/10.1016/j.ijpe.2010.09.009
16. Morton T, Pentico D (1993) Heuristic scheduling systems. Wiley, New York
17. Chan L, Shen Z, Simchi-levi D, Swann JL (2004) Coordination of pricing and inventory decisions: a survey and classification. In: Simchi-Levi D, Wu SD, Shen ZM (eds) Handbook of quantitative supply chain analysis modeling in the E-business era. Kluwer Academic, Boston, pp 335–392
18. Coelho LC, Cordeau J, Laporte G (2014) Thirty years of inventory routing. Transp Sci 48:1–19. https://doi.org/10.1287/trsc.2013.0472
19. Andersson H, Hoff A, Christiansen M et al (2010) Industrial aspects and literature survey: combined inventory management and routing. Comput Oper Res 37:1515–1536. https://doi.org/10.1016/j.cor.2009.11.009
20. Diabat A (2014) Hybrid algorithm for a vendor managed inventory system in a two-echelon supply chain. Eur J Oper Res 238:114–121. https://doi.org/10.1016/j.ejor.2014.02.061
21. Mateus G, Ravetti MG, de Souza M, Valeriano T (2010) Capacitated lot sizing and sequence dependent setup scheduling: an iterative approach for integration. J Sched 13:245–259. https://doi.org/10.1007/s10951-009-0156-2
22. Kreipl S, Dickersbach JT (2008) Scheduling coordination problems in supply chain planning. Ann Oper Res 161:103–122. https://doi.org/10.1007/s10479-007-0293-y
23. Kreipl S, Pinedo M (2009) Planning and scheduling in supply chains: an overview of issues in practice. Prod Oper Manag 13:77–92. https://doi.org/10.1111/j.1937-5956.2004.tb00146.x

24. Martínez-Jurado PJ, Moyano-Fuentes J (2013) Lean management, supply chain management and sustainability: a literature review. J Clean Prod 85:134–150. https://doi.org/10.1016/j.jclepro.2013.09.042
25. Demir E, Bektaş T, Laporte G (2014) A review of recent research on green road freight transportation. Eur J Oper Res 237:775–793. https://doi.org/10.1016/j.ejor.2013.12.033
26. Devika K, Jafarian A, Nourbakhsh V (2014) Designing a sustainable closed-loop supply chain network based on triple bottom line approach: a comparison of metaheuristics hybridization techniques. Eur J Oper Res 235:594–615. https://doi.org/10.1016/j.ejor.2013.12.032
27. Yücesan E (2007) Impact of information technology on supply chain management. In: Jung H, Jeong B, Chen FF (eds) Trends in supply chain design and management SE—6. Springer, London, pp 127–148
28. Hazen BT, Boone CA, Ezell JD, Jones-Farmer LA (2014) Data quality for data science, predictive analytics, and big data in supply chain management: an introduction to the problem and suggestions for research and applications. Int J Prod Econ 154:72–80. https://doi.org/10.1016/j.ijpe.2014.04.018
29. Kot S (2014) Principles of global supply chain management. Faculty of Management, Czestochowa University of Technology
30. Croxton KL, García-Dastugue SJ, Lambert DM, Rogers DS (2001) The supply chain management processes. Int J Logist Manag 12:13–36
31. Sánchez D, Amodeo L, Prins C (2009) Meta-heuristic approaches for multi-objective simulation-based optimization in supply chain inventory management. Lect Notes Comput Sci 5484:798–807. https://doi.org/10.1007/978-1-84996-119-6_9
32. Duan Q, Liao TW (2013) A new age-based replenishment policy for supply chain inventory optimization of highly perishable products. Int J Prod Econ 145:658–671. https://doi.org/10.1016/j.ijpe.2013.05.020
33. Zolfaghari S, Quan V, El-Bouri A, Khashayardoust M (2010) Application of a genetic algorithm to staff scheduling in retail sector. Int J Ind Syst Eng 5:20–47. https://doi.org/10.1504/IJISE.2010.029755
34. Wang KJ, Chen MJ (2009) Cooperative capacity planning and resource allocation by mutual outsourcing using ant algorithm in a decentralized supply chain. Expert Syst Appl 36:2831–2842. https://doi.org/10.1016/j.eswa.2008.01.089
35. Laporte G (2009) Fifty years of vehicle routing. Transp Sci 43:408–416. https://doi.org/10.1287/trsc.1090.0301
36. Yeun LC, Ismail WANR, Omar K, Zirour M (2008) Vehicle routing problem: models and solutions. J Qual Meas Anal 4:205–218
37. Cordeau JF, Gendreau M, Hertz A et al (2005) New heuristics for the vehicle routing problem. In: Langevin A, Riopel D (eds) Logistics systems: design and optimization. Kluwer Academic, Boston, pp 279–297
38. Kumar SN, Panneerselvam R (2012) A survey on the vehicle routing problem and its variants. Intell Inf Manag 4:66–74. https://doi.org/10.4236/iim.2012.43010
39. Vidal T, Crainic TG, Gendreau M, Prins C (2013) Heuristics for multi-attribute vehicle routing problems: a survey and synthesis. Eur J Oper Res 231:1–21. https://doi.org/10.1016/j.ejor.2013.02.053
40. Pérez-Bernabeu E, Juan AA, Faulin J, Barros B (2015) Horizontal cooperation in road transportation: a multi-depot case illustrating savings in distances and pollutant emissions. Int Trans Oper Res 22:585–606. https://doi.org/10.1111/itor.12130
41. Juan AA, Faulin J, Pérez-Bernabeu E, Jozefowiez N (2014) Horizontal cooperation in vehicle routing problems with backhauling and environmental criteria. Procedia Soc Behav Sci 111:1133–1141. https://doi.org/10.1016/j.sbspro.2014.01.148
42. Lin C, Choy KL, Ho GTS et al (2014) Survey of green vehicle routing problem: past and future trends. Expert Syst Appl 41:1118–1138. https://doi.org/10.1016/j.eswa.2013.07.107
43. Erdoğan S, Miller-Hooks E (2012) A green vehicle routing problem. Transp Res Part E Logist Transp Rev 48:100–114. https://doi.org/10.1016/j.tre.2011.08.001

44. Schmid V, Doerner KF, Laporte G (2013) Rich routing problems arising in supply chain management. Eur J Oper Res 224:435–448. https://doi.org/10.1016/j.ejor.2012.08.014
45. Pisinger D, Ropke S (2007) A general heuristic for vehicle routing problems. Comput Oper Res 34:2403–2435. https://doi.org/10.1016/j.cor.2005.09.012
46. Liu S, Papageorgiou LG (2013) Multiobjective optimisation of production, distribution and capacity planning of global supply chains in the process industry. OMEGA Int J Manag Sci 41:369–381. https://doi.org/10.1016/j.omega.2012.03.007
47. Afshin Mansouri S, Gallear D, Askariazad MH (2012) Decision support for build-to-order supply chain management through multiobjective optimization. Int J Prod Econ 135:24–36. https://doi.org/10.1016/j.ijpe.2010.11.016
48. Chu CH, Luh YP, Li TC, Chen H (2009) Economical green product design based on simplified computer-aided product structure variation. Comput Ind 60:485–500. https://doi.org/10.1016/j.compind.2009.02.003
49. Vieira P, Vieira SM, Sousa JMC et al (2014) Designing closed-loop supply chains with nonlinear dimensioning factors using ant colony optimization. Soft Comput 19:2245. https://doi.org/10.1007/s00500-014-1405-7
50. Griffis SE, Bell JE, Closs DJ (2012) Metaheuristics in logistics and supply chain management. J Bus Logist 33:90–106. https://doi.org/10.1111/j.0000-0000.2012.01042.x
51. Calvete HI, Galé C, Iranzo JA (2014) Planning of a decentralized distribution network using bilevel optimization. Omega 49:30–41. https://doi.org/10.1016/j.omega.2014.05.004
52. Delavar MR, Hajiaghaei-Keshteli M, Molla-Alizadeh-Zavardehi S (2010) Genetic algorithms for coordinated scheduling of production and air transportation. Expert Syst Appl 37:8255–8266. https://doi.org/10.1016/j.eswa.2010.05.060
53. Eskandarpour M, Masehian E, Soltani R, Khosrojerdi A (2014) A reverse logistics network for recovery systems and a robust metaheuristic solution approach. Int J Adv Manuf Technol 74:1393–1406. https://doi.org/10.1007/s00170-014-6045-7
54. Boukherroub T, Ruiz A, Guinet A, Fondrevelle J (2015) An integrated approach for sustainable supply chain planning. Comput Oper Res 54:180–194. https://doi.org/10.1016/j.cor.2014.09.002
55. Brandenburg M, Govindan K, Sarkis J, Seuring S (2014) Quantitative models for sustainable supply chain management: developments and directions. Eur J Oper Res 233:299–312. https://doi.org/10.1016/j.ejor.2013.09.032
56. Ahi P, Searcy C (2013) A comparative literature analysis of definitions for green and sustainable supply chain management. J Clean Prod 52:329–341. https://doi.org/10.1016/j.jclepro.2013.02.018
57. Seuring S (2013) A review of modeling approaches for sustainable supply chain management. Decis Support Syst 54:1513–1520. https://doi.org/10.1016/j.dss.2012.05.053
58. Lourenço H, Ribeiro R (2012) Strategies for an integrated distribution problem strategies for an integrated distribution problem. In: Montoya-Torres JR, Juan AA, Huaccho L et al (eds) Hybrid algorithms for service, computing and manufacturing systems: routing and scheduling solutions. IGI Global Books, Hershey, pp 98–121
59. Ribeiro R, Lourenço HR (2005) A new model and heuristic for a multi-period inventory-routing problem. Decision science institute of international conference, Barcelona, pp 805–815
60. Caceres-Cruz J, Arias P, Guimarans D et al (2014) Rich vehicle routing problem: survey. ACM Comput Surv 47:1–28. https://doi.org/10.1145/2666003
61. Cardona-Valdés Y, Álvarez A, Pacheco J (2014) Metaheuristic procedure for a bi-objective supply chain design problem with uncertainty. Transp Res B Methodol 60:66–84. https://doi.org/10.1016/j.trb.2013.11.010
62. Rego N, Claro J, Pinho de Sousa J (2014) A hybrid approach for integrated healthcare cooperative purchasing and supply chain configuration. Health Care Manag Sci 17:303–320. https://doi.org/10.1007/s10729-013-9262-y
63. Cavique L (2007) A scalable algorithm for the market basket analysis. J Retail Consum Serv 14:400–407. https://doi.org/10.1109/EPIA.2005.341294

64. Martins P, Ladrón A, Ramalhinho H (2014) Maximum cut-clique problem: ILS heuristics and a data analysis application. Int Trans Oper Res. https://doi.org/10.1111/itor.12120
65. Sadeghia A, Alem-Tabriza A, Zandieha M (2011) Product portfolio planning: a metaheuristic-based simulated annealing algorithm. Int J Prod Res 49:2327–2350. https://doi.org/10.1080/00207540903329338
66. April J, Glover F, Kelly J, Laguna M (2001) Simulation/optimization using "Real-World" applications. In: Proceedings of 2001 winter simulation conference, Arlington, pp 134–138
67. Ngai EWT, Xiu L, Chau DCK (2009) Application of data mining techniques in customer relationship management: a literature review and classification. Expert Syst Appl 36:2592–2602. https://doi.org/10.1016/j.eswa.2008.02.021
68. Grasas A, Juan AA, Lourenço HR (2014) SimILS: a simulation-based extension of the iterated local search metaheuristic for stochastic combinatorial optimization. J Simul 10:69. https://doi.org/10.1057/jos.2014.25
69. Juan AA, Faulin J, Grasman SE et al (2015, in press) A review of Simheuristics: extending metaheuristics to deal with stochastic optimization problems. Oper Res Perspect 2:62–72. https://doi.org/10.1016/j.orp.2015.03.001
70. Giménez C, Lourenço HR (2004) E-supply chain management: review, implications and directions for future research. EUROMA INSEAD, Fontainebleau, pp 1021–1031
71. Davenport TH (2014) Big data at work dispelling the myths, uncovering the opportunities. Harvard Business School Press, Boston
72. Nam T, Pardo TA (2011) Conceptualizing smart city with dimensions of technology, people, and institutions. In: Proceedings of the 12th annual international digital government research conference on digital government innovation in challenging times – dg.o '11, p 282. https://doi.org/10.1145/2037556.2037602

The Maximum Clique and Vertex Coloring

57

Oleksandra Yezerska and Sergiy Butenko

Contents

Introduction 1716
Construction Heuristics 1717
Local Search 1720
Metaheuristics 1721
  Local Search-Based Methods 1722
  Population-Based Methods 1727
Heuristics Based on Continuous Formulations 1733
Other Heuristics 1735
Computational Results 1735
Conclusion 1738
Cross-References 1738
References 1738

Abstract

In this chapter, we review heuristic approaches for two classical and closely related problems of finding a maximum clique and an optimal vertex coloring. Both problems have a wide variety of practical applications, and due to their computational intractability, a significant effort has been focused on developing heuristic methods. This chapter discusses construction heuristics, local search strategies, and metaheuristics designed and/or adapted for the maximum clique and vertex coloring problems.

O. Yezerska · S. Butenko (✉)
Department of Industrial and Systems Engineering, Texas A&M University, College Station, TX, USA
e-mail: yaleksa@tamu.edu; butenko@tamu.edu

© Springer Nature Switzerland AG 2025

R. Martí et al. (eds.), *Handbook of Heuristics*,
https://doi.org/10.1007/978-3-032-00385-0_47

Keywords

Maximum clique · Vertex coloring · Chromatic number

Introduction

Given a simple, undirected graph $G = (V, E)$, a *clique* is defined as a subset $C \subseteq V$ of pairwise adjacent vertices. A graph is called *complete* if the set of its vertices forms a clique. A *maximal clique* is a clique that is not a subset of a larger clique. The *maximum clique problem* asks for a clique of the greatest possible cardinality in G. The size of such a clique is known as the *clique number* of G, denoted by $\omega(G)$.

An *independent set* (or *stable set*) is a subset $I \subseteq V$ of vertices with no edge between them. It is easy to see that a *clique* in G is an *independent set* in the *complement graph* of G, $\bar{G} = (V, \bar{E})$, where $\bar{E} = \binom{V}{2} \backslash E$, and vice versa. Finding a maximum clique in G is equivalent to finding a maximum independent set in $\bar{G}$. The cardinality of a maximum independent set in G is called the *independence* or *stability number* of G, denoted $\alpha(G)$, and $\alpha(\bar{G}) = \omega(G)$.

A *proper vertex coloring* (also referred to as a coloring for simplicity) is an assignment of a color (or a label) to each vertex in a graph such that no adjacent vertex has the same color. More formally, given a set of colors $\{1, 2, \ldots, k\}$, a *proper k-coloring* is a mapping $f : V \to \{1, 2, \ldots, k\}$, where $f(v_i) \neq f(v_j)\ \forall \{i, j\} \in E$. Then the vertices with the same value of f belong to the same *color class*. The *chromatic number* of a graph G, denoted $\chi(G)$, is the minimum number of colors necessary to color G. An *optimal coloring* of a graph is a proper coloring that uses exactly $\chi(G)$ colors.

It is easy to see that the vertices that belong to the same color class form an independent set, and the vertices of a clique all belong to different color classes. Then it follows that for a given graph G:

$$\chi(G) \geq |V|/\alpha(G),$$

and

$$\chi(G) \geq \omega(G).$$

The maximum clique and vertex coloring problems often arise in similar applications, where they play complementary roles. For example, let the vertices in the graph represent elements of a system and the edges between them—the incompatibility between those elements. Then a maximum clique will represent a set of mutually incompatible elements of the largest size in the system, whereas the color classes will correspond to sets of mutually compatible elements.

Both problems have numerous real-world applications across diverse domains. Some of the earliest and best-known applications of the maximum clique problem

are in computer vision, experimental design, information retrieval, coding theory, fault diagnosis, social network analysis, and computational biochemistry and genomics, among other areas [36, 158, 191]. As for the coloring, it is commonly used for scheduling [131, 147], timetabling [182, 183, 187], frequency assignment [78, 185], circuit board testing [80], register allocation [42, 46], and various problems in transportation [39, 186]. A discussion of both clique and vertex coloring applications in telecommunications is presented in [15].

The maximum clique and vertex coloring problems are both well-known NP-hard problems [79, 124] that are hard to approximate within a factor of $n^{1-\epsilon}$ for any $\epsilon > 0$, where $n = |V|$ [9, 10, 63, 64, 101, 196]. Such intractability results, along with the practical interest, have led to a considerable effort in devising numerous heuristic approaches. Almost all known types of heuristic methods have been applied to these problems. Some of the early surveys on heuristic methods for the vertex coloring problem can be found in [159, 184] and more recent in [45, 75, 77, 139, 161], among which [45, 75] review just the local search methods and [77] focuses on the recent approaches. As for the maximum clique problem, the first extensive surveys date back to the 1990s [25, 158], while the more recent one [191] focuses mostly on local search techniques. Other brief discussions on heuristic approaches for the maximum clique problem can be found in [158, 161]. It should be noted that most of these surveys also discuss exact algorithms, many of which take advantage of heuristics to enhance their performance.

The remaining sections of this chapter are organized as follows. The most common construction heuristics for the maximum clique and vertex coloring problems are discussed in section "Construction Heuristics." In section "Local Search," we briefly outline the local search strategies used for both problems. Section "Metaheuristics" covers various metaheuristic approaches. It is followed by sections "Heuristics Based on Continuous Formulations," "Other Heuristics," "Computational Results," and "Conclusion," respectively.

Construction Heuristics

Construction heuristics are used to build an initial feasible solution. The most intuitive and common construction heuristics are *sequential* ones, which recursively update the current, partial (and not necessarily feasible) solution one step at a time. Based on how the next step is selected, most of these heuristics can be classified as either greedy, random, or a combination of both. Greedy selection rules often aim to ensure the highest level of flexibility (i.e., the largest possible "candidate set") in the future steps.

The most common sequential greedy heuristics for the maximum clique problem are *best in* and *worst out*, as named by Kopf and Ruhe [128]. Best-in algorithms are those that start with an empty set as an initial solution and iteratively add a vertex with the largest degree among the candidate vertices, whereas worst out are the ones that start with a whole graph and recursively delete vertices with the lowest degree

until the remaining graph is complete. The best-in heuristic applied for finding a maximal independent set yields a solution of size at least $\frac{|V|}{\delta+1}$, where $\delta = \frac{2|E|}{|V|}$ is the average degree of a vertex [62]. The best-in heuristic was also shown to be "provably best" for the maximum clique problem in [120]. A heuristic is called provably best if no other polynomial-time algorithm can guarantee a better solution whenever one exists (assuming $\mathcal{P} \neq \mathcal{NP}$).

As for the vertex coloring problem, any heuristic that finds a coloring using at most $\Delta(G)$ colors (referred to as Brooks' coloring), where $\Delta(G)$ is the maximum vertex degree in G, is provably best [120]. According to Brooks' theorem [27], a simple, connected graph G that is neither complete nor an odd cycle satisfies $\chi(G) \leq \Delta(G)$. Brooks' coloring can be obtained using several heuristics [89, 95, 121, 123, 134, 171].

Many popular heuristics for the vertex coloring problem are based on sequential construction approaches. Sequential construction heuristics usually iteratively assign each vertex with the smallest feasible color such that no conflicts with already colored vertices occur. The order in which vertices are colored is either static and determined beforehand (preorder) or dynamic and updated on the fly. It is easy to see that the quality of the solution depends entirely on how the vertices are ordered and that there exists such vertex ordering that produces an optimal coloring [184]. Hence, considerable attention has been devoted to studying and designing various ordering schemes.

Two examples of preorder sequential coloring heuristics are the *largest first* (LF) [182] and the *smallest last* (SL) [146]. LF sorts the vertices in decreasing order of their degree, and the ordering $(v_1, \ldots, v_n)$, produced in such manner, results in the coloring with at most $1 + \max_{1 \geq j \geq n} d_j(v_j)$ colors, where $d_j(v_j)$ is the degree of vertex v_j in the induced subgraph $G_j = G[\{v_1, \ldots, v_j\}]$. SL is similar to LF, but it sorts the vertices in the reverse order. First, a vertex of the minimum degree in graph G is selected and labeled v_n. Then, iteratively for each $j = n-1, \ldots, 1$, a vertex of the minimum degree in G_j is selected and labeled v_j.

A notable sequential coloring approach that does not preorder vertices is the DSATUR [26] heuristic. A decision on what vertex to color next is based on the value of the vertex *saturation degree*—a number of the differently colored neighbors of this vertex. A vertex with the maximum saturation degree is selected. Another example of a heuristic that does not preorder vertices is the *recursive largest first* (RLF) [131]. This heuristic generates a color class one at a time and does not proceed to another color class until no more vertices can be assigned to the current class. This assignment proceeds as follows: When a new color class is generated, set U_1 contains all uncolored vertices, and set U_2 is empty. Iteratively, a vertex $v \in U_1$ is selected, assigned to the current color class, and removed from U_1. If v has any neighbors in U_1, they are removed from U_1 and placed in U_2. The vertex assignment proceeds, while U_1 is not empty. The very first vertex to be assigned to the new color class is the one with the maximum degree in $G[U_1]$, and all the rest are the ones with the maximum degree in $G[U_2]$. When U_1 becomes empty, the next color class is generated, and the process repeats.

In another study [23], a maximal independent set is iteratively extracted from the set of uncolored vertices, and all the vertices forming this independent set are assigned the same color. The process repeats until the whole graph is colored. Similar technique is used in the algorithm called XRLF [119], which is a version of RLF [131]. Several independent sets are first constructed, and then the one whose removal yields a residual graph with the smallest edge density is selected as a new color class. This process is repeated until a threshold number of uncolored vertices are left, which are then colored using an exhaustive search. Independent set extraction approach has been used to construct initial colorings within various metaheuristic frameworks [43, 70, 77, 103, 150]. In addition, a few enhanced approaches based on independent set extraction were proposed. Namely, in [188], the authors suggest to preprocess large graphs by iteratively extracting a maximal number of pairwise disjoint independent sets of the same size. Such strategy was shown to assign more vertices to the same number of color classes than that with a one-at-a-time independent set extraction. This yields smaller in size residual graphs of uncolored vertices, which are easier to color. In [100, 190], the authors observe that an independent set extracted may not necessarily define a color class in an optimal coloring and that removing such sets during preprocessing may prevent one from reaching an optimal solution. To mitigate this drawback, the authors of [100, 190] suggested a framework that reconsiders a certain number of the removed independent sets and allows some of the vertices forming those sets to change a color. Different strategies on how many and which extracted sets to reexamine were studied, and the experiments on some large and hard graph instances reported in [100, 190] demonstrate an encouraging performance of the proposed approach.

To improve the quality of solutions generated by the greedy sequential methods, different techniques ranging from randomization, multiple restarts, and local search (section "Local Search") to sophisticated metaheuristic frameworks (section "Metaheuristics") can be used. We will cover here a few simple strategies and discuss the rest in the later sections.

For the maximum clique problem, Jagota and Sanchis [114] proposed several enhancements to a basic greedy sequential heuristic, which include randomization, multiple restarts, and an adaptive mechanism. A randomization is embodied by a probabilistic greedy vertex selection rule. Three kinds of adaptation are studied: an update of the initial state (AI), an update of the probability distribution used by a selection rule (AW), and no adaptation at all (NA). The corresponding updates take place at each restart and are based on the information obtained in the previous restart. NA heuristic was shown to perform poorly compared to AI and AW. Grosso et al. [91] used similar techniques and developed a two-phase heuristic, called a *deep adaptive greedy search* (DAGS). In the first phase of DAGS, the modified variant of a sequential greedy heuristic is applied for all the vertices in the graph. The modification consists of allowing to swap already added vertices with the better candidates. The second phase is used for diversification purposes. It is a restart-adaptive greedy heuristic performed on a set of vertices that appear less frequently

in the cliques obtained during the first phase. Each vertex is assigned a weight associated with its quality as a potential candidate for a solution. Such weight is updated at each restart by means of a learning-like mechanism.

For the vertex coloring problem, an improvement procedure called an *iterative greedy* (IG) heuristic was proposed in [50, 51]. It is based on the observation that given a feasible coloring and an ordering in which vertices belonging to the same color class are grouped together, applying a sequential greedy heuristic to such an ordering will produce a solution at least as good as the previous one.

Local Search

In this section, we outline common local search strategies for both the maximum clique and vertex coloring problems and discuss their defining features.

Local search is an approach that iteratively refines a current solution by examining its local neighborhood and moving to higher-quality neighbors until no further improvement is possible. A solution's local neighborhood is defined by a *neighborhood function*, which maps a feasible solution to a set of "close" feasible solutions (neighbors). Local search methods vary in how they define this neighborhood function, the choice of search strategy, and the objective or evaluation function used to measure solution quality.

As suggested in [191], local search strategies for the maximum clique problem can be categorized into *legal* and *k-fixed penalty* ones. A legal strategy is a traditional local search, where a search space consists of legal (feasible) solutions, cliques; an evaluation function is the size of a clique; and a neighborhood function is a list of vertex operations to be performed to move from one clique to another. The most common neighborhood functions are (a, b)-interchanges, where a is the number of vertices removed from the current solution and b is the number of added vertices, with a usually smaller than b to ensure that such interchange increases the size of the solution by $(b - a)$.

A k-fixed penalty strategy, on the other hand, can be thought of as a technique that solves the decision version of the problem that answers the question "is there a clique of size k?" This type of a local search deals with infeasible solutions, sets that are not necessarily cliques. An evaluation function is defined as the number of edges in the set. By moving to sets with a larger number of edges, a local search attempts to eventually reach a feasible solution—a clique. For this type of a local search, a (1,1)-interchange is usually used, where a vertex to be removed is usually the one with the smallest number of neighbors in the solution and a vertex to be added is the one adjacent to the largest number of solution members. Such a technique does not guarantee finding a feasible solution; therefore, it is usually used as a part of a more sophisticated local search framework (section "Metaheuristics").

For the vertex coloring problem, the local search strategies can also be categorized into legal and k-fixed ones. Moreover, as discussed in [75], they can be

further grouped into *legal*, *k-fixed partial legal*, *penalty*, and *k-fixed penalty* ones. Here, the legal and the k-fixed penalty strategies, similarly to the local search strategies for the maximum clique problem, are the local searches on either feasible (proper) colorings or infeasible k-coloring, respectively. For the legal strategy, the local neighborhood is explored in an attempt of finding a feasible coloring with a fewer number of colors. An infeasible coloring is such that it contains at least one pair of adjacent vertices assigned the same color. An edge connecting a pair of such vertices is referred to as *conflicting*. For this strategy, a local search aims at performing moves resulting in the minimization of the number of conflicting edges in the solution.

Under a k-fixed partial legal strategy, the search space contains partial feasible k-colorings. A partial feasible k-coloring corresponds to a coloring of a subset of vertices $V' \subset V$ that uses k colors and is feasible with respect to already colored vertices in V'. The rest of the vertices in $V \setminus V'$ are not yet colored. The local neighborhood is explored in an attempt to find a complete feasible k-coloring. For a pure penalty strategy, the search space contains infeasible colorings, and the number of colors used is unfixed. The local search under this strategy tries to detect a feasible coloring with the minimum number of colors used. The quality function usually combines both the number of conflicting edges and the number of colors used.

The most common neighborhood functions for coloring are also interchanges [146]. They are performed by moving vertices from one color class to another, in an attempt of emptying one of the classes and, thus, reducing the number of colors used. Under the penalty and k-fixed penalty strategies, such interchanges are allowed to result in infeasible solutions.

Local search approaches also vary based on the rules of determining when to move to a better solution. By the *best improvement strategy*, the entire local neighborhood is explored, and the best neighbor is selected as a new solution, whereas the *first improvement strategy* updates the current solution with the first found neighbor of a better quality. However, with either of these strategies, a local search can get trapped in local optima of poor quality. To overcome this drawback and explore more regions of a search space, different techniques, e.g., allowing non-improving moves and recording the search process to direct the future exploration, have been devised and will be discussed in the following section.

Metaheuristics

In this section, we will review the most common metaheuristics for the maximum clique and vertex coloring problems. The metaheuristics can be classified with respect to various criteria [22]. Based on the number of solutions maintained at the same time, the algorithms can be divided into single-point search ones, encompassing most local search-based heuristics (section "Local Search-Based Methods"), and population-based ones (section "Population-Based Methods"), which deal with a pool of solutions at a time and perform evolution-like search processes.

Local Search-Based Methods

The most successful techniques to enhance local search have proven to be the ones that allow non-improving moves in the process of local neighborhood exploration. One of them, *simulated annealing* (section "Simulated Annealing"), allows moving to a worse solution with a certain probability, while another one, *tabu search* (section "Tabu Search"), records the moves and forbids (tabu) their future usage for a certain number of iterations to prevent cycling. Other successful local search-based methods have been devised and will be discussed in section "Other Local Search-Based Methods."

Simulated Annealing

Simulated annealing (SA) method is a randomized neighborhood search introduced by Kirkpatrick et al. [126] and covered in a separate chapter of this book. SA is analogous to a physical annealing—a process of heating up a solid material and slowly cooling it down in order to obtain a low-energy configuration. The main idea of SA is to iteratively select random candidate solution in the local neighborhood and, if it is of better quality, move to the candidate solution; otherwise perform the move with a certain probability of acceptance. Such probability is usually defined as $\exp(-\Delta)/T$, where Δ is a difference between the objective function values of the new and the current solutions and T is a parameter called the *temperature*. As the heuristic progresses, the probability is being decreased, ensuring fewer random walks and directing the search toward improvement. The rate at which the probability decreases is usually referred to as a "cooling schedule." The cooling schedule that follows the logarithmic law guarantees the convergence of the algorithm to a global optimum [22]; however, it may result in an exponential running time. Hence, faster cooling schedules are usually used in practice.

The most influential papers devoted to application of SA to the maximum clique problem date back in the late 1980s and mid-1990s and are all covered in [25]. Since, to the best of our knowledge, no novel SA methods for the maximum clique problem have been designed since then, we will follow [25] and briefly review here the same studies.

An application of SA to the maximum clique (independent set) problem was first described by Aarts and Korst in 1988 [1]. Without providing any experimental results, they proposed a *penalty function* method (k-fixed penalty strategy), for which a possible solution can be any set, not necessarily independent, and the quality of the solution is determined by the function $f(V') = |V'| - \lambda|E'|$, where V' and E' are the vertex and edge sets of the solution and λ is a certain weighting factor. A few years later, Jerrum [116] has theoretically proven a poor performance of SA application to the maximum clique problem. In particular, they studied a variant of SA with the temperature parameter fixed and a solution space consisting of legal cliques (legal strategy). Feo et al. [68] implemented SA for the maximum independent set problem utilizing the penalty method and observed that it was inferior to the greedy randomized adaptive search procedure (GRASP)

they proposed. However, the computational experiments presented by Homer and Peinado [108, 118] in the Second DIMACS Implementation Challenge showed that SA can be very effective. In their work, Homer and Peinado used the idea of penalty method described by Aarts and Korst and implemented it for the maximum clique problem. The results of the experiments on large graphs, reported in [108, 118], were quite promising, and SA was ranked one of the best heuristics in the Second DIMACS Implementation Challenge.

One of the first attempts to apply SA to the vertex coloring problem dates back to 1987, when Chams et al. [43] considered the following variant of the method. The neighborhood structure consists of k-colorings that are not necessarily legal (k-fixed penalty strategy), and the objective function to be minimized is defined by the number of conflicting edges. According to the experimental results reported in [43], the pure SA did not perform as strongly as when it was combined with other methods. In particular, when SA was combined with RLF [26], where RLF was used to generate color classes and SA to color the rest of the uncolored vertices, the performance of such combined method dominated all other methods tested in the paper (DSATUR and RLF). In 1991, Johnson et al. [119] conducted a detailed study of different schemes of SA adopted to the vertex coloring problem. The first scheme tested is a penalty function method, where the neighborhood structure is defined by infeasible solutions (penalty strategy), i.e., colorings with conflicting edges. The number of colors used in a particular solution is not fixed. Let C_i denote a color class, let E_i be an edge set in the subgraph induced by C_i ($i = 1, \ldots, k$; $1 \leq k \leq |V|$), and let $\Pi = (C_1, \ldots, C_k)$ be a solution. Then the cost (penalty) function associated with Π is defined as follows:

$$f(\Pi) = \sum_{i=1}^{k} 2|C_i| \cdot |E_i| - \sum_{i=1}^{k} |C_i|^2.$$

The first component of $f(\Pi)$ favors independent sets, while the second one favors large color classes. Minimization of $f(\Pi)$ results in eliminating the conflicting edges and reaching a legal coloring and also, as the authors argued, in potentially reducing the number of color classes as a side effect. The second scheme is by Morgenstern and Shapiro [151], and it utilizes the so-called *Kempe chains*, which are defined in [119] as the connected components of the subgraph induced by the union of two disjoint independent sets. In this method, the same cost function as in the penalty function method is retained, but the search space contains legal k-colorings. The new candidate solution is generated as follows. Two color classes C_i and C_j ($i \neq j$) are picked at random, and a Kempe chain defined by a set $H \subset C_i \cup C_j$ is constructed. Next C_i is replaced by $C_i \Delta H$ and C_j by $C_j \Delta H$, where $X \Delta Y$ denotes a symmetric difference $(X \backslash Y) \cup (Y \backslash X)$ between X and Y. The third scheme uses the same strategy as in [43], where the number k of colors is fixed and the objective is to minimize the number of conflicting edges in an attempt to find a feasible k-coloring. According to the results of the experiments reported by Johnson et al., none of the three methods was found dominant over the other.

A simplified version of SA with the temperature parameter remaining fixed was used by Morgenstern [118, 150] for the Second DIMACS Implementation Challenge. In this chapter, a new neighborhood structure, called *impasse neighborhood*, was defined. It turned out to be very effective, and many approaches have used it thereafter. This neighborhood structure operates with a fixed number k of colors and aims to improve a partial coloring, in which not all vertices are colored, to a complete coloring with k colors. A feasible solution is a partition of the set of vertices into $k + 1$ subsets $\{V_1, \ldots, V_k, V_{k+1}\}$, where $V_1, \ldots, V_k$ are independent sets representing a partial k-coloring and V_{k+1} is the set of uncolored vertices that needs to be eventually emptied in order to obtain a proper k-coloring of all vertices. A neighbor of a solution is obtained by moving a vertex v from V_{k+1} to one of the k independent sets, say V_i, and then moving to V_{k+1} all vertices from V_i that are adjacent to v.

Tabu Search

Tabu search was developed by Glover [85] and, independently, by Hansen and Jaumard [97]. See ▸ Chap. 31, "Tabu Search," for more details. It is a modified local search which allows performing moves yielding worse quality solutions, thus diversifying the search path. The algorithm uses the so-called *tabu lists* to store certain information about the previously visited solutions to forbid future local search moves that can result in cycling. The length of the tabu lists, called *tabu tenures*, allows to control the level of diversification and intensification of the search. Sometimes the tabu restriction is relaxed if a solution meets certain *aspiration-level* condition.

In 1989, Friden et al. [72] used tabu search ideas to construct a heuristic called STABULUS for finding stable (independent) sets. STABULUS is based on the k-fixed penalty local search strategy (see section "Local Search"), where the search space consists of the sets of size k, and the algorithm tries to minimize the number of edges in the set. To find an independent set of the largest cardinality, STABULUS is applied iteratively, incrementing k each time an independent set of size k is detected. Three tabu lists were used in STABULUS, one for storing visited solutions, and the other two contained added and deleted vertices.

A few years later, Gendreau et al. [82] proposed two simple variants of tabu search for the maximum clique problem, deterministic and probabilistic, where in the former, at each iteration, the entire set of the solution neighbors is explored and, in the latter, the sampling of the neighborhood is applied. In both variants the legal local search strategy was exploited, where the search space contains the feasible cliques and the objective is to find a clique of the maximum size. In the deterministic approach, it is allowed to remove only one vertex per iteration, while in the probabilistic it is a few at a time. Later these heuristics were enhanced with the diversification strategies [174], such as greedy restart and continuous diversification. The first one is a very simple approach of restarting the algorithm with the new solution after a certain number of iterations without improvement have been performed. A new solution is constructed from vertices that have not been visited yet or have been visited less frequently. The continuous diversification

works in "on-the-fly" mode. It guides the search path by evaluating each vertex addition/removal using the information on how frequently the vertex has been added to the solution and how long it has stayed in the solution. Both diversification techniques and their combination have proven to be beneficial if used along with the simple basic search. The more efficient and sophisticated the search is, the less improvement the diversification is able to add to it. It also did not appear to work well with the probabilistic version of the tabu search heuristic. The heuristics were tested on DIMACS instances, and the results were reported in [175].

A similar idea of prohibiting the future moves in order to prevent cycling was used in reactive local search (RLS) [17]. However, contrary to the traditional tabu search, where certain parameters (tabu tenure, depth of the search, etc.) are determined by the user through numerous preliminary experiments and are usually sensitive to certain specific structures of the instances, RLS determines all of this information dynamically within its execution. The length of tabu lists is adjusted based on previously stored information on how often the algorithm cycles, etc. Also, the algorithm is equipped with a memory-influenced restart procedure to provide additional long-term diversification. RLS showed excellent performance on DIMACS instances and is considered one of the best local search-based heuristics for the maximum clique problem.

Further efforts of applying tabu search to the maximum clique (independent set) problem include works by Wu et al. [117, 189, 192]. In [189], the authors developed an adaptive multistart tabu search approach that uses a k-fixed penalty local search strategy. The exploration of a search space is performed by moves that swap a vertex from the current solution with a vertex from the solution-constrained neighborhood. The algorithm employs a restart strategy, under which a new solution is constructed from the vertices that were less frequently used throughout the algorithm, that way visiting unexplored search space regions. The algorithm showed excellent performance when compared against several other state-of-the-art algorithms. The study of the relationship between the number of restarts and the quality of the evaluation function performed by the authors suggested that for structured instances, more frequent restarts yield better results, while the reverse holds for random graphs.

For the vertex coloring problem, tabu search techniques were first applied by Hertz and de Werra [103]. In their proposed algorithm, named TABUCOL, the search space consists of the complete k-colorings, where k is fixed. Some colorings might have conflicting edges, and, hence, the algorithm's goal is to move to the solutions with a smaller number of conflicting edges by changing the color assignment of the endpoints of conflicting edges. The tabu lists store these color assignments. The algorithm uses the same local search strategy as in [43] but yields better results than those obtained by SA in [43].

Blöchliger and Zufferey [21] designed several variations of the tabu search framework for the vertex coloring problem. The main ingredients they used were the partial k-coloring local search strategy and reactive tabu tenure, adjusted based on the fluctuations of the objective function.

Tabu search has also been applied to both the maximum clique and vertex coloring problems as a part of more complex metaheuristics, such as evolutionary

algorithms, yielding the so-called *hybrid algorithms* (discussed in section "Population-Based Methods").

Other Local Search-Based Methods

The greedy randomized adaptive search procedure (GRASP) [66, 67], described in the ▸ Chap. 23, "GRASP," is a simple metaheuristic that combines a constructive heuristic, a local search, and a restart policy. Its construction part uses both greedy and random selection rules. More precisely, the initial solution is built by randomly selecting a vertex from a so-called *restricted candidate list* (RCL), which consists of a certain number of the best candidates. This number ranges between 1 and the size of the candidate list, where the closer it is to 1, the greedier the selection rule is, and the closer it is to the candidate list size, the more randomness is involved. Every time a solution is constructed, an attempt to improve it is made using an appropriate local search technique (see section "Local Search"). The algorithm restarts a certain number of times, and the best-found solution is recorded. GRASP has been successfully applied to the maximum clique (maximum independent set) [2, 68, 169] and the vertex coloring problems [129].

Variable neighborhood search (VNS) [98, 149] is a metaheuristic that systematically explores different neighborhood structures of the same problem in the search for a better solution. This method is introduced in the ▸ Chap. 32, "Variable Neighborhood Search." VNS approach was applied to the maximum clique problem in [99]. For the vertex coloring, an algorithm based on VNS was proposed in [11]. It uses more than ten different neighborhoods and utilizes tabu search techniques. An extension of VNS, called *variable space search* (VSS), was proposed for the vertex coloring problem by Hertz et al. in [105]. In VSS, when the search is trapped in a local optimum, the entire search space, including the neighborhood searched and the objective function, is changed. The proposed algorithm, called VSS-Col, explores three different search spaces for the vertex coloring problem and yields excellent results. An *iterated local search* (see the ▸ Chap. 26, "Iterated Local Search" in this handbook for more information), which upon reaching a local optimum perturbs it so that the local search can keep going, was applied to the vertex coloring problem in [44, 156]. In [44], the authors analyzed the performance of different types of solution perturbations.

An algorithm based on *variable depth search* for the maximum clique problem, based on *k-opt* local search (KLS), was proposed by Katayama et al. [125]. The k-opt search is essentially an (a, b)-interchange with $a > b$ and $a + b = k$, where a vertices are removed from the current clique and b vertices from the local neighborhood are added to the solution yielding a clique of a larger size. The value k is determined dynamically throughout the execution of the algorithm. The reported results on selected DIMACS instances show the algorithm's competitiveness with RLS [17].

Pullan and Hoos [167] introduced a dynamic local search (DLS) algorithm for the maximum clique problem. DLS is based on alternating between a clique expansion phase, during which vertices are added to the current clique, and plateau search, during which vertices of the current clique are swapped with vertices outside of

the current clique. The selection of vertices is based on the penalties assigned to them dynamically during the search in a similar way used in DAGS [91]. When no more expansion or swapping is possible, a perturbation mechanism is applied to overcome search stagnation. The algorithm achieves substantial improvements over some state-of-the-art algorithms on many DIMACS instances. However, the algorithm has a disadvantage of being sensitive to the penalty delay parameter, which controls the frequency at which vertex penalties are decreased and needs to be determined beforehand by preliminary experiments and thorough calibration. This issue has been addressed in [166], where a modified algorithm, called a Phased Local Search (PLS), was proposed. Similarly to DLS [167], the algorithm is a combination of a clique expansion and a plateau search. Moreover, it operates three sub-algorithms, which differ in their vertex selection rule: random selection, random selection within vertex degree, and random selection within vertex penalties. In the latter one, the parameter responsible for the frequency of penalty decreases does not have to be fixed and defined externally as in DLS but is modified adaptively. PLS has shown to have a comparable and sometimes improved performance to DLS. Grosso et al. [92] performed a detailed computational study of the main components of DLS, such as usage of vertex penalties as a selection rule, plateau search, etc., in an attempt to design simple and efficient iterated local search techniques for the maximum clique problem. Another study that analyzes the key ingredients of RLS and DLS was performed by Battiti and Mascia in [16] and yielded several enhanced modifications of the mentioned algorithms.

Pullan et al. [168] introduced a parallelized hyper-heuristic algorithm for the maximum clique problem, called a cooperating local search (CLS). Hyper-heuristics [32] are heuristic search methods that manage the low-level heuristics. In particular, CLS is a methodology that controls the four PLS-based heuristics and dynamically reconfigures their allocation to cores, based on information retrieved from a trial in order to ensure the appropriateness of selected heuristic for a particular instance. In [5], Andrade et al. introduce efficient implementations of $(1, 2)$ and $(2, 3)$-interchange legal strategy local searches for the maximum independent set problem. These two local searches are integrated into an iterated local search metaheuristic, which according to the results reported in [5] yield great results, especially on large and difficult instances. Benlic and Hao [18] devised an algorithm, called breakout local search (BLS). BLS is an iterated local search, which, when a local optimum is discovered, applies a certain diversification strategy to perturb the current solution, so it can be used as a starting point to explore another region of the search space. The magnitude of perturbation is adapted dynamically according to the current search state. The results of the experiments showed that BLS competes favorably with RLS [17], VNS [99], and PLS [166].

Population-Based Methods

As opposed to local search algorithms, where one solution is maintained at each iteration, the population-based algorithms deal with several solutions (a *population*)

at a time. The new population is produced by certain interactions between the members of a current population. Namely, the population generation techniques in *evolutionary* and *genetic algorithms* (section "Evolutionary and Genetic Algorithms") are based on imitating the mechanisms of evolution, whereas in *ant colony algorithms* (section "Ant Colony Optimization"), the solution construction is inspired by the behavior of ants seeking paths to food sources.

Evolutionary and Genetic Algorithms

Evolutionary and genetic algorithms [86, 107] (see also the corresponding chapters in this volume) are inspired by the evolution and natural selection and are based on the following evolution principles: "the fittest survives," "two fit parents will have even fitter offsprings," and mutation. In the genetic algorithms, these principles are embodied with the help of respective mechanisms: *reproduction*, *crossover*, and *mutation* operators. At each iteration (in evolutionary terminology—*generation*) of an algorithm, the population of solutions (*individuals*) is subjected to all or some of those operators. Reproduction operator selects the "fittest" solutions, where the fitness level is represented by a *fitness function* (or objective function in optimization terms). Then, with the help of the crossover operator, the "offspring" solutions are produced as a result of merging one or several parts of one parent solution with one or several parts of another parent solution. The mutation operator changes a small part of a solution to provide randomization.

For the maximum clique problem, the most intuitive way to encode a solution is to use a binary array of the size equal to the cardinality of the given graph's vertex set, where an entry contains 1, if the corresponding vertex is included in the current solution, and 0, otherwise. The definition of the solution fitness function can vary. A fairly simple fitness function assesses the size of a vertex subset if it is a clique, or equals 0, otherwise. It can also be a little more complex. For example, it can combine several terms, such as the density of the induced subgraph and its size. It can also include a term to penalize the infeasible solutions.

The early work on adapting genetic algorithms to the maximum clique problem is covered in great detail in [25]. We will review in the following the same studies and also discuss several more recent advancements.

One of the earliest studies on the effectiveness of genetic algorithms at solving the maximum clique problem was done by Carter and Park [41]. The authors showed that the pure genetic algorithm performs poorly and designed several modifications, including an annealed fitness function and the modified crossover operators, to improve its performance. However, their results were still not satisfactory and led to a conclusion that genetic algorithms have to be significantly customized to be able to perform well and that they are very computationally expensive. Their later studies in [160] report that genetic algorithms are ineffective for solving combinatorial optimization problems and are inferior to other heuristics, such as simulated annealing. However, in another study [12], Back and Khuri showed quite the contrary results. They designed a genetic algorithm called GENEsYs with a fitness function which included a component for penalizing infeasible solutions. The performance of GENEsYs for the maximum independent set problem on graphs

with up to 200 vertices was promising and suggested that with the right fitness function, genetic algorithms are capable of yielding satisfactory results. In another approach, Murthy et al. [154] introduced a new crossover mechanism, called a *partial copy crossover*, and suggested to split a mutation operator into *deletion* and *addition* operators. The results presented in the paper are based on experiments with small graphs (up to 50 vertices), making it difficult to evaluate the algorithm's effectiveness.

In [71], Foster and Soule developed two variations of genetic algorithms for the maximum clique problem. One of them uses a different problem-encoding approach, referred to as a *grouping representation*, which instead of encoding each solution as a binary string uses an array that stores information about all the solutions in the current population. Such encoding scheme showed to increase effectiveness of the crossover operation. The other variant of the algorithm uses the time-weighted fitness function, which showed to improve the algorithm's performance. Hifi [106] adopted a genetic algorithm for solving the maximum weighted independent set problem. The necessary algorithm modifications included an introduction of a so-called *two-fusion crossover operator* and a replacement of a mutation operator by a *heuristic-feasibility operator*. The two-fusion crossover operator takes into account both the structure and the fitness of two parent solutions and produces two children, while the heuristic-feasibility operator transforms infeasible solutions into feasible ones. The main contribution of the designed approach is that it is easily parallelizable.

Aggarwal et al. [4] designed an optimized crossover mechanism (inspired by a heuristic CLIQMERGE proposed in [13] by Ballas and Niehaus) and applied it to the maximum independent set problem. The new crossover operator exploits the structure of the solution rather than its encoding. More precisely, when a crossover operator is applied to two solutions from the population, one of the new solutions is generated in such a way that it has the best fitness function value, while the other one is constructed in a way that ensures the diversity of the search space. Promising results of the proposed mechanism inspired authors of CLIQMERGE, Balas and Niehaus, to examine this strategy even further [14]. The authors also considered a different way of population replacement called a *steady-state replacement* and a different selection method. The results of the experiments reported in [14] were even more encouraging than those by Aggarwal et al. in [4].

Since most of pure genetic algorithms have shown to be not very effective for solving hard combinatorial optimization problems, they are usually enhanced by incorporating different techniques. Bui and Eppley [29] designed a hybrid strategy with vertex preordering phase and a local optimization at each iteration of the genetic algorithm. The authors claim that the performance of their hybrid algorithm is comparable to a continuous-based approach by Gibbons et al. [83] but inferior to tabu search- and simulated annealing-based approaches. Fleurent et al. [69] used tabu search and other techniques as alternative mutation operators. The performance of their approach was satisfactory with respect to the quality of the obtained solutions but rather disappointing with respect to the computational effort. Marchiori [143] developed an approach that combines a genetic algorithm

and a greedy heuristic, referred to as HGA. At each iteration of genetic algorithm, the greedy heuristic is applied to each member of the population to extract maximal cliques. That way, the genetic part of the approach provides exploration of the search space, while the greedy heuristic provides exploitation. Despite its simplicity, the approach has shown very good results [143, 144]. Zhang et al. [195] developed an approach called EA/G which uses similar techniques as HGA but also incorporates a guided mutation operator. The results of the experiments indicated that the algorithm was superior to HGA. Singh and Gupta [170] designed a framework which consists of two phases: generation of cliques by a steady-state genetic algorithm and extending them to maximal cliques by a heuristic which combines a randomized sequential greedy approach and the exact algorithm of Carraghan and Pardalos [40]. The results of the experiments show that the algorithm outperformed three other evolutionary algorithms, of Balas and Niehaus [14], Marchiori [144], and Fenet and Solnon [65], but its performance is inferior to that of one of the most effective local search-based techniques (RLS) of Battiti and Protasi [17].

Guturu and Dantu [94] designed an approach that combines an impatient evolutionary algorithm and a probabilistic tabu search. Brunato and Battiti [28] presented a hybrid algorithm similar to EA/G by Zhang et al. in [195]. In the proposed algorithm, which they refer to as R-EVO, the new solutions are initialized according to an estimated distribution which is based on knowledge extracted from the previous generations of the population. The solutions are then evolved through a simplified version of RLS technique.

Despite such a significant effort in studying and designing different variations of evolutionary and genetic algorithms, these approaches are still considered to be far less effective for the maximum clique problem than the local search-based ones. Perhaps, their inability to compete with the simple local search techniques lies in the fact that there is no intuitive relationship between the crossover operator and a discovery of improved cliques, and no such a meaningful operator has been devised yet [191].

For the vertex coloring problem, the solutions are usually encoded either as permutations of the vertices and referred to as *order-based encoding* or as color partitions and referred to as *direct encoding*. The direct encoding can further be categorized into the *assignment based* and *partition based*. Under the former one, a solution is represented as an array, where each entry is associated with a certain vertex and contains an index of the class the corresponding vertex belongs to. The latter one, on the other hand, represents a solution as a partition of the vertices into the color classes. Even though more satisfactory results have been achieved using direct encoding techniques, the order-based encoding has a powerful advantage of always corresponding to a feasible solution. Hence, it has been somewhat exploited as well [52, 61, 153].

One of the first attempts to apply evolutionary algorithms for vertex coloring problem is by Davis [52]. The author used order-based encoding, and a solution was evaluated using a greedy sequential heuristic, which colors the vertices in the order defined by the permutation. The algorithm yielded unsatisfactory results, as was also shown in [70]. All of the later efforts of applying evolutionary algorithms

to the vertex coloring problem involved the incorporation of other techniques, such as local search. The pioneers of this direction were Costa et al. [48] and Fleurent and Ferland [70], replacing the mutation operator by a simple descent method in the former and tabu search in the latter, respectively. In both papers the uniform crossover operator or its slightly enhanced modification [70] is used. The algorithms yielded slightly better results than their pure counterparts; however, the importance of designing better crossover operators was concluded in [70].

In later developments, the focus shifted to designing more advanced crossover operators tailored to the specifics of the problem of interest. More precisely, Dorne and Hao [56] used a specialized crossover based on the notion of the union of independent sets, which, with the combination of tabu search, led to a simple yet very powerful algorithm. Galinier and Hao [74] devised a new class of crossover mechanisms based on the partition of vertices into color classes rather than assignment of colors to vertices. The crossover operator called *greedy partition crossover* (GPX) transmits the subsets of color classes from parent solutions to the offspring solution. The power of this operator lies in its ability to retain some of each parent's structure in the offspring solution. The hybrid algorithm that uses this crossover operator and a tabu search was tested on large and difficult DIMACS instances and showed to be one of the best performing compared to other coloring algorithms [74]. A few years later, Glass and Prügel-Bennett [84] examined the hybrid algorithm of Galinier and Hao [74] by replacing the tabu search part of the algorithm with a steepest descent. The results of the experiments showed that the algorithm still remained powerful, which led to a conclusion that its success is due to the crossover operator. Hamiez and Hao [96] presented a *scatter search algorithm* devised for the vertex coloring problem. A scatter search is a subclass of evolutionary approaches, where replacements of the population are based not only on the improvement of the fitness function but also on the improvement of the population diversity; see the corresponding chapter in this handbook for more detail. The algorithm proposed in [96] was able to obtain results similar to the best-known approaches, but it was more expensive computationally.

Galinier et al. [76] proposed an *adaptive memory algorithm*, called AMACOL, where portions of solutions (color classes) are stored in a central memory and are then used to build new solutions. The algorithm uses a tabu search and yields results comparable to those obtained by an algorithm proposed in [74]. Malaguti et al. [140] designed a two-phase metaheuristic algorithm, called MMT, for the vertex coloring problem. The first phase is a combination of an evolutionary algorithm with a crossover similar to GPX from [74] and a tabu search with an impasse neighborhood, proposed in [150]. The second phase of MMT is a post-optimization procedure based on the set covering formulation of the problem. The algorithm yields promising results.

Ant Colony Optimization

Ant colony optimization, discussed in more detail in a separate chapter of this book, was originally introduced by Dorigo [55] and was inspired by the way the ants use pheromone to communicate with each other and search for better paths to a source

of food. As they search for food, each ant leaves a pheromone trail. All subsequent ants will follow the path with stronger pheromone. Since the pheromone evaporates over time, it will last longer on better (shorter) paths. This way, these paths will be followed by more ants and will subsequently have more pheromone laid. Eventually all the ants will follow one single best path. The main idea of ant colony optimization algorithms is to mimic this process. Each ant is thought of as a constructive heuristic, which builds a solution step by step using a greedy force and a *trail* information on the history of the search obtained from other ants.

First attempt to investigate ant colony capabilities to tackle the maximum clique problem was by Fenet and Solnon [65]. They introduced an algorithm, called Ant-Clique, that generates maximal cliques by repeatedly adding vertices to partial solutions, where each vertex is chosen according to the probability that depends on the level of pheromone. The pheromone deposition is proportional to the quality of previously constructed cliques. The performance of the algorithm was compared to that of [144] and showed that on average Ant-Clique is capable of reaching better solutions on the majority of tested instances. Youseff and Elliman [193] introduced an algorithm that combines the capabilities of ant colony optimization and local search techniques with prohibition rules. The algorithm was compared to the genetic local search of [144] and has shown to be somewhat competitive. Incorporation of local search techniques into the ant colony algorithm has also been addressed in [173] and has shown to improve the solution process.

One of the first ant colony algorithms for the vertex coloring problem was designed by Costa and Hertz [47]. Their algorithm, called ANTCOL, considers each ant as a constructive heuristic derived from DSATUR [26] and RLF [131]. Even though the algorithm was not superior of the best graph coloring heuristics of that time, its performance was still promising enough to encourage more research in this direction. Dowsland and Thompson further applied the same algorithm for examination scheduling [57] and later enhanced it by strengthening the construction phase and incorporating it with a tabu search improvement phase [58].

Hertz and Zuffrey [104] argued that an algorithm, where each ant has only a minor role, is still competitive with the other ant algorithms. Namely, in their proposed algorithm, each single ant does not build an entire solution but only colors a single vertex. A similar idea was explored by Bui et al. [30]. In their algorithm, instead of coloring the entire graph, each ant colors only a portion of the graph. The performance of the algorithm seemed to be rather encouraging. Plumetta et al. [163] proposed an ant local search algorithm, where each ant is considered as a local search, rather than a constructive heuristic. The algorithm showed to be competitive with other evolutionary methods available at that time.

Zufferey [197] conducted an analysis of importance of different ant roles, ranging from insignificant ones, used to make a minor decision, to more crucial ones, by performing a refined local search. It was shown experimentally that the ant algorithms are more efficient when ants have strong roles like local search procedures.

Heuristics Based on Continuous Formulations

A remarkable result by Motzkin and Straus [152], which provides an elegant connection between maximum cliques and a certain quadratic program, has been extensively explored to devise efficient heuristics for the maximum clique problem. Some of the early methods are discussed in detail in [25].

Given a graph $G = (V, E)$, let A_G be the adjacency matrix of G. Consider the following quadratic problem:

$$\max_{x \in \Delta^n} g(x) = \sum_{\{i,j\} \in E} 2x_i x_j,$$

where $\Delta^n = \left\{ x \in R^{|V|} \mid \sum_{i \in V} x_i = 1, x_i \geq 0, i \in V \right\}$ is the standard simplex in $R^{|V|}$.

Motzkin and Straus [152] showed that the size of a maximum clique in G is equal to:

$$\omega(G) = \frac{1}{1 - g(x^*)},$$

where x^* is a global maximizer of the above problem.

One perceived drawback associated with the Motzkin-Straus quadratic program is that its solution does not always correspond to a clique in the graph; hence, some postprocessing is required to obtain clique's vertices based on a solution to the continuous problem. Bomze [24] proposed a regularization of the Motzkin-Straus formulation by adding $\frac{1}{2} x^T I x$ to the objective, where I is the corresponding identity matrix. He has shown that every local maximum of the modified formulation corresponds to a maximal clique in the graph as follows: C is a maximal clique of G if and only if x^*, such that $x_i^* = 1/|C|$ if $x_i \in C$ and $x_i^* = 0$ otherwise, is a local maximum of the regularized Motzkin-Straus formulation. Hungerford and Rinaldi [110] established more general conditions on the regularization term that guarantee the one-to-one correspondence between the local maxima of the regularized Motzkin-Straus formulation and maximal cliques in the graph.

One of the early approaches based on the Motzkin-Straus result was an iterative clique retrieval procedure [157], which turned out to be of very high computational cost and was only able to solve instances on less than 75 vertices. Gibbons et al. [83] proposed to modify the Motzkin-Straus formulation so that it becomes a problem of optimizing a quadratic function over a sphere. Even though such problem is polynomially solvable, it, however, yields approximate solutions that need to be rounded. Similar approaches, with a few advances, can be found in [33, 34]. Simple heuristics based on formulations of the maximum independent set problem as maximization of polynomial functions over a unit hypercube were studied in [3].

Gruzdeva [93] proposed to add a non-convex quadratic constraint represented by the difference of two convex functions (DC contraints) [109] to a continuous formulation of the maximum clique problem. The author showed that the proposed approach is competitive with other methods based on continuous formulations.

Massaro et al. [145] transformed the Motzkin-Straus formulation of the maximum clique problem into its corresponding linear complementary problem (LCP) [49]. To deal with the inherent degeneracy of the derived LCP, the authors designed a variation of a classical Lemke's method [132] with an effective "look-ahead" pivot rule.

Butenko et al. [38] proposed variable objective search, a metaheuristic framework that performs a local search with respect to different alternative formulations of the same combinatorial optimization problems. To test their method, authors considered the maximum independent set problem and its equivalent non-convex programming formulations discussed in [3].

Recently, the Motzkin-Straus formulation was extended to a hierarchy of higher-order standard polynomial programming formulations in [35]. Namely, the authors of [35] showed that for all $k \in \{2, \ldots, \omega\}$, where $\omega \equiv \omega(G)$:

$$\binom{\omega}{k}\left(\frac{1}{\omega}\right)^k = \max_{x \in \Delta^{|V|}} \sum_{C \in \mathcal{C}_k} \prod_{i \in C} x_i,$$

where $\mathcal{C}_k$ is the set of all cliques of cardinality k in G. The above formulation is referred to as (P^k). They have shown that a local maximizer of (P^{k+1}) is also a local maximizer of (P^k), which establishes the hierarchical structure of the formulations. In particular, all local maximizers of (P^ω) are global. The support of a local maximizer of (P^k) was proved to correspond to a so-called strongly part-maximal multipartite clique in G (see [35] for the definition), which can be easily converted into a clique. To obtain the direct correspondence of local maximizers to maximal cliques, the authors proposed regularized versions of (P^k) in [138]. The numerical experiments exploring the performance of both the original and regularized formulations for $k \in \{1, \ldots, 5\}$ in conjunction with a first-order solver showed that the results improve with the increase in k and that the non-regularized formulations appear to perform slightly better than their regularized counterparts (since regularizations introduce poor-quality local maxima).

Another notable result—the "sandwich theorem" [127]— has inspired several heuristic algorithms for both the maximum clique (independent set) [31, 59] and vertex coloring problems [60, 88, 122]. For a given graph G, the sandwich theorem states that the following relationship holds:

$$\omega(G) \leq \theta(\bar{G}) \leq \chi(G),$$

where $\theta(\bar{G})$ is the Lovász theta, which can be computed in polynomial time [135].

In [31], the authors study rank-one and rank-two formulations of the semidefinite programming (SDP) formulation of the Lovász theta number. Based on the obtained further continuous formulations, they designed heuristics for the maximum

independent set problem. As for the vertex coloring problem, the proposed solution algorithms also used a semidefinite programming (SDP) formulation of the Lovász theta and were capable of obtaining near-optimal solutions for problems of medium sizes [60].

Other Heuristics

Many other interesting heuristic approaches have been devised to tackle the maximum clique and the vertex coloring problems. Balas and Niehaus [13] developed a heuristic, called CLIQMERGE, that generates large cliques by repeatedly finding a maximum clique in the subgraph induced by the union of two cliques. Such procedure is performed by finding the bipartite matching of the complement subgraph. A heuristic for the maximum independent set problem based on the operation of *edge projection*, which is a specialization of Lovász and Plummer's clique projection [136], has been designed in [142]. Goldberg and Rivenburgh [87] used a *restricted backtracking* for detecting cliques in graphs. DNA computing was applied to the maximum clique problem in [155, 194].

Neural network approach has been very popular in tackling the maximum clique problem since the mid-1980s. A detailed discussion on neural network-based methods applied to the maximum clique problem developed before 1999 is presented in [25]. The interested readers are also referred to [90, 111, 112, 115]. The recent success of deep learning applications has motivated the development of heuristics based on graph neural networks. However, their performance currently lags behind established approaches [6, 148].

For the vertex coloring problem, the neural network approach has also been applied in [19, 20, 81, 113, 162, 176, 177]. An algorithm which inherits ideas from quantum mechanics was proposed in [179]. Heuristics based on Douglas-Rachford algorithm were developed in [7, 8].

Since many large-scale real-world networks are sparse, with most vertices having a low degree, heuristics specifically designed for such networks effectively exploit these structural properties. These methods typically integrate scale-reduction or kernelization techniques with other heuristic approaches to enhance efficiency and solution quality [2, 130, 180, 181].

Computational Results

In this section, we summarize best computational results achieved by heuristic algorithms on selected hard benchmark instances for the problems of interest. We disregard running times and focus on the quality of the reported solutions. More specifically, we report only the best-found solutions for the instances with no known optima and list the methods that were able to attain these solutions.

The results for the maximum clique instances are presented in Table 1. First three columns contain information about the graph instance (name, the number of vertices $|V|$, and edges $|E|$). In the next columns, we report the size of the largest

detected clique (ω_{lb}) and list references of the methods that were able to attain such solution. Note that the first two graph instances in Table 1 are from the Second DIMACS Implementation Challenge [53], the next three instances are from the Tenth DIMACS Implementation Challenge [54], and the rest of them are from the so-called CODE family [172], which is a collection of challenging graph instances arising from the coding theory.

The computational results related to the coloring problem are contained in Table 2. Here, the first three columns are the same as in Table 1. The fourth and fifth columns contain the size of the lower (χ_{lb}) and upper (χ_{ub}) bounds on the chromatic number. The majority of the results related to the lower bound size is by [102]. The last column in Table 2 lists the references to the algorithms which have achieved χ_{ub}. The first 27 instances presented in this table are from the Second DIMACS Implementation Challenge [53]; the next 12 are from Stanford Large Network Dataset Collection [133], referred to as the SNAP; and the last 8 are from the Tenth DIMACS Implementation Challenge [54].

Table 1 Best-known solutions to the currently unsolved instances of the maximum clique problem

Instance	$\|V\|$	$\|E\|$	ω_{lb}	ω_{lb} obtained by
Hamming10-4	1,024	434,176	40	[17, 18, 94, 99, 117, 166, 167, 189, 192]
c500.9	500	112,332	57	[17, 18, 94, 99, 117, 166–168, 189, 192]
c1000.9	1,000	450,079	68	[5, 17, 18, 94, 99, 117, 166–168, 189, 192]
c2000.9	2,000	1,799,532	80	[18, 117, 189, 192]
1dc.2048	2,048	58,367	172	[5, 37, 117]
1et.2048	2,048	22,528	316	[5, 37, 117]
1tc.2048	2,048	18,944	352	[5, 37, 117]
1zc.1024	1,024	33,280	112	[5, 37, 117]
1zc.2048	2,048	78,848	198	[5, 37, 117]
1zc.4096	4,096	184,320	379	[5, 37, 117]

Table 2 Approximate solutions to the vertex coloring problem

Instance	$\|V\|$	$\|E\|$	χ_{lb}	χ_{ub}	χ_{ub} obtained by
DSJC250.5	250	15,668	26	28	[44, 73, 74, 76, 88, 137, 140, 150, 165, 179]
DSJC500.1	500	12,458	9	12	[21, 44, 56, 73, 76, 88, 105, 137, 140, 150, 163–165, 179]
DSJC500.5	500	62,624	43	48	[21, 56, 74, 76, 105, 137, 140, 150, 163–165, 178, 179]
DSJC500.9	500	1,124,367	123	126	[44, 44, 56, 76, 105, 137, 163–165, 178, 179]
DSJC1000.1	1,000	49,629	10	20	[21, 56, 74, 76, 100, 105, 137, 140, 163–165, 178, 179, 188]
DSJC1000.5	1,000	249,826	73	83	[56, 74, 100, 137, 140, 150, 164, 178, 179, 188]
DSJC1000.9	1,000	449,449	216	222	[100, 178, 179, 188]

(continued)

Table 2 (continued)

Instance	$\|V\|$	$\|E\|$	χ_{lb}	χ_{ub}	χ_{ub} obtained by
r1000.1c	1000	485090	96	98	[21, 73, 100, 137, 140, 150, 164, 165, 179]
latin_square_10	900	307,350	90	97	[178]
1-Insertions_5	202	1,227	4	6	[76, 141]
1-Insertions_6	607	6,337	4	7	[76, 141]
2-Insertions_4	149	541	4	5	[44, 76, 141]
2-Insertions_5	597	3,936	3	6	[76, 141]
3-Insertions_4	281	1,046	3	4	[44]
3-Insertions_5	1,406	9,695	3	6	[44, 141]
4-Insertions_4	475	1,795	3	5	[76, 141]
4-FullIns_5	4,146	77,305	7	9	[44, 141]
5-FullIns_4	1,085	11,395	8	9	[141]
wap01	2,368	110,871	41	42	[44, 100]
wap02	2,464	111,742	40	41	[100]
wap03	4,730	286,722	40	44	[100]
wap04	5,231	294,902	40	42	[100]
wap07	1,809	103,368	40	41	[100]
wap08	1,870	104,176	40	42	[100, 141]
c2000.5	2,000	999,836	99	145	[100]
c2000.9	2,000	1,799,532	80	408	[100]
c4000.5	4,000	4,000,268	107	259	[100]
Wiki-Vote	7,115	100,762	19	24	[180]
p2p-Gnutella04	10,876	39,994	4	6	[180]
p2p-Gnutella25	22,687	54,705	4	6	[180]
p2p-Gnutella24	26,518	65,369	4	6	[180]
Cit-HepTh	27,770	352,285	23	25	[180]
Cit-HepPh	34,546	420,877	19	21	[180]
p2p-Gnutella30	36,682	88,328	4	6	[180]
p2p-Gnutella31	62,586	147,892	4	6	[180]
soc-Epinions1	75,879	405,740	25	30	[180]
Email-EuAll	265,214	364,481	18	20	[180]
WikiTalk	2,394,385	4,659,565	31	51	[180]
cit-Patents	3,774,768	16,518,947	11	12	[180]
kron_g500-simple-logn16	65,536	2,456,071	136	155	[180]
G_n_pin_pout	100,000	501,198	4	8	[180]
smallworld	100,000	499,998	6	8	[180]
wave	156,317	1,059,331	6	9	[180]
audikw1	943,695	38,354,076	36	44	[180]
ldoor	952,203	22,785,136	21	35	[180]
333SP	3,712,815	11,108,633	4	5	[180]
cage15	5,154,859	47,022,346	6	13	[180]

More detailed discussions on computational performance of various heuristics can be found in the surveys [77, 139, 191].

Conclusion

In this chapter, we discussed various heuristics for the maximum clique and the vertex coloring problems that were developed in the past several decades. Local search-based metaheuristics have demonstrated greater effectiveness compared to population-based approaches. Nevertheless, the latter continue to attract significant research interest. When combined with efficient local search techniques, population-based methods can achieve competitive results and offer the additional advantage of easy parallelization. Meanwhile, heuristics inspired by continuous optimization present a promising avenue for future research. In particular, refining continuous formulations by eliminating poor-quality local optima would facilitate the rapid computation of high-quality solutions using standard gradient-based solvers. Significant efforts are being devoted to the development of artificial intelligence and quantum computing-based methods. However, their effectiveness remains to be established.

Cross-References

- GRASP
- Tabu Search
- Variable Neighborhood Search

Acknowledgments We thank the anonymous reviewers and the book editors, whose suggestions helped to improve the original version of the manuscript. Partial support of the National Science Foundation under grant CMMI-1538493 and Air Force Office of Scientific Research under grant FA9550-23-1-0300 is also gratefully acknowledged.

Competing Interest Declaration The author(s) has no competing interests to declare that are relevant to the content of this manuscript.

References

1. Aarts E, Korst J (1988) Simulated annealing and Boltzmann machines. Wiley, Chichester
2. Abello J, Pardalos PM, Resende MGC (1999) On maximum clique problems in very large graphs. In: Abello J, Vitter J (eds) External memory algorithms and visualization. American Mathematical Society, Boston, pp 119–130
3. Abello J, Butenko S, Pardalos PM, Resende MGC (2001) Finding independent sets in a graph using continuous multivariable polynomial formulations. J Glob Optim 21(2):111–137
4. Aggarwal CC, Orlin JB, Tai RP (1997) Optimized crossover for the independent set problem. Oper Res 45(2):226–234

5. Andrade DV, Resende MGC, Werneck RF (2012) Fast local search for the maximum independent set problem. J Heuristics 18(4):525–547
6. Angelini MC, Ricci-Tersenghi F (2023) Modern graph neural networks do worse than classical greedy algorithms in solving combinatorial optimization problems like maximum independent set. Nat Mach Intell 5:29–31
7. Aragón Artacho FJ, Campoy R (2018) Solving graph coloring problems with the Douglas-Rachford algorithm. Set-Valued Var Anal 26:277–304
8. Aragón Artacho FJ, Campoy R, Elser V (2020) An enhanced formulation for solving graph coloring problems with the Douglas-Rachford algorithm. J Glob Optim 77:383–403
9. Arora S, Safra S (1998) Probabilistic checking of proofs: a new characterization of NP. J ACM (JACM) 45(1):70–122
10. Arora S, Lund C, Motwani R, Sudan M, Szegedy M (1998) Proof verification and the hardness of approximation problems. J ACM (JACM) 45(3):501–555
11. Avanthay C, Hertz A, Zufferey N (2003) A variable neighborhood search for graph coloring. Eur J Oper Res 151(2):379–388
12. Back T, Khuri S (1994) An evolutionary heuristic for the maximum independent set problem. In: Proceedings of the first IEEE conference on evolutionary computation, pp 531–535
13. Balas E, Niehaus W (1996) Finding large cliques in arbitrary graphs by bipartite matching. DIMACS Ser Discret Math Theor Comput Sci 26:29–52
14. Balas E, Niehaus W (1998) Optimized crossover-based genetic algorithms for the maximum cardinality and maximum weight clique problems. J Heuristics 4(2):107–122
15. Balasundaram B, Butenko S (2006) Graph domination, coloring and cliques in telecommunications. In: Handbook of optimization in telecommunications. Springer, Berlin, pp 865–890
16. Battiti R, Mascia F (2010) Reactive and dynamic local search for max-clique: engineering effective building blocks. Comput Oper Res 37(3):534–542
17. Battiti R, Protasi M (2001) Reactive local search for the maximum clique problem. Algorithmica 29(4):610–637
18. Benlic U, Hao JK (2013) Breakout local search for maximum clique problems. Comput Oper Res 40(1):192–206
19. Berger MO (1994) k-coloring vertices using a neural network with convergence to valid solutions. In: Proceedings of IEEE international conference on neural networks, vol 7, pp 4514–4517
20. Blas AD, Jagota A, Hughey R (2002) Energy function-based approaches to graph coloring. IEEE Trans Neural Netw 13(1):81–91
21. Blöchliger I, Zufferey N (2008) A graph coloring heuristic using partial solutions and a reactive tabu scheme. Comput Oper Res 35(3):960–975
22. Blum C, Roli A (2003) Metaheuristics in combinatorial optimization: overview and conceptual comparison. ACM Comput Surv (CSUR) 35(3):268–308
23. Bollobás B, Thomason A (1985) Random graphs of small order. North-Holland Math Stud 118:47–97
24. Bomze IM (1997) Evolution towards the maximum clique. J Glob Optim 10:143–164
25. Bomze IM, Budinich M, Pardalos PM, Pelillo M (1999) The maximum clique problem. In: Handbook of combinatorial optimization. Springer, Boston, pp 1–74
26. Brélaz D (1979) New methods to color the vertices of a graph. Commun ACM 22(4):251–256
27. Brooks RL (1941) On colouring the nodes of a network. In: Mathematical proceedings of the Cambridge philosophical society, vol 37. Cambridge University Press, Cambridge, pp 194–197
28. Brunato M, Battiti R (2011) R-EVO: a reactive evolutionary algorithm for the maximum clique problem. IEEE Trans Evol Comput 15(6):770–782
29. Bui TN, Eppley PH (1995) A hybrid genetic algorithm for the maximum clique problem. In: Proceedings of the 6th international conference on genetic algorithms. Morgan Kaufmann Publishers Inc., pp 478–484
30. Bui TN, Nguyen TH, Patel CM, Phan KAT (2008) An ant-based algorithm for coloring graphs. Discret Appl Math 156(2):190–200

31. Burer S, Monteiro RD, Zhang Y (2002) Maximum stable set formulations and heuristics based on continuous optimization. Math Program 94(1):137–166
32. Burke E, Kendall G, Newall J, Hart E, Ross P, Schulenburg S (2003) Hyper-heuristics: an emerging direction in modern search technology. Int Ser Oper Res Manag Sci 57:457–474
33. Busygin S (2006) A new trust region technique for the maximum weight clique problem. Discret Appl Math 154(15):2080–2096
34. Busygin S, Butenko S, Pardalos PM (2002) A heuristic for the maximum independent set problem based on optimization of a quadratic over a sphere. J Comb Optim 6(3):287–297
35. Butenko S, Makovenko M, Pardalos M (2022) A hierarchy of standard polynomial programming formulations for the maximum clique problem. SIAM J Opt 32(3):2102–2128
36. Butenko S, Wilhelm WE (2006) Clique-detection models in computational biochemistry and genomics. Eur J Oper Res 173(1):1–17
37. Butenko S, Pardalos PM, Sergienko IV, Shylo V, Stetsyuk P (2009) Estimating the size of correcting codes using extremal graph problems. In: Pearce C, Hunt E (eds) Optimization: structure and applications. Springer, New York, pp 227–243
38. Butenko S, Yezerska O, Balasundaram B (2013) Variable objective search. J Heuristics 19(4):697–709
39. Caprara A, Kroon L, Monaci M, Peeters M, Toth P (2007) Passenger railway optimization. Handb Oper Res Manag Sci 14:129–187
40. Carraghan R, Pardalos PM (1990) An exact algorithm for the maximum clique problem. Oper Res Lett 9(6):375–382
41. Carter R, Park K (1993) How good are genetic algorithms at finding large cliques: an experimental study. Technical report, Computer Science Department, Boston University
42. Chaitin GJ, Auslander MA, Chandra AK, Cocke J, Hopkins ME, Markstein PW (1981) Register allocation via coloring. Comput Lang 6(1):47–57
43. Chams M, Hertz A, de Werra D (1987) Some experiments with simulated annealing for coloring graphs. Eur J Oper Res 32(2):260–266
44. Chiarandini M, Stützle T et al (2002) An application of iterated local search to graph coloring problem. In: Proceedings of the computational symposium on graph coloring and its generalizations, pp 112–125
45. Chiarandini M, Dumitrescu I, Stützle T (2007) Stochastic local search algorithms for the graph colouring problem. In: Handbook of approximation algorithms and metaheuristics. Chapman & Hall/CRC, Boca Raton, p 63-1
46. Chow FC, Hennessy JL (1990) The priority-based coloring approach to register allocation. ACM Trans Program Lang Syst (TOPLAS) 12(4):501–536
47. Costa D, Hertz A (1997) Ants can colour graphs. J Oper Res Soc 48(3):295–305
48. Costa D, Hertz A, Dubuis C (1995) Embedding a sequential procedure within an evolutionary algorithm for coloring problems in graphs. J Heuristics 1(1):105–128
49. Cottle RW, Pang JS, Stone RE (1992) The linear complementarity problem. SIAM, Philadelphia
50. Culberson JC (1992) Iterated greedy graph coloring and the difficulty landscape. Technical report. TK 92-07, Department of Computing Science, University of Alberta
51. Culberson JC, Luo F (1996) Exploring the k-colorable landscape with iterated greedy. In: Johnson DS, Trick MA (eds) Cliques, coloring, and satisfiability: second DIMACS implementation challenge. American Mathematical Society, Providence, pp 245–284
52. Davis L (1991) Order-based genetic algorithms and the graph coloring problem. In: Handbook of genetic algorithms. Van Nostrand Reinhold, New York, pp 72–90
53. DIMACS (1993) NP hard problems: maximum clique, graph coloring, and satisfiability. The second DIMACS implementation challenge. http://dimacs.rutgers.edu/Challenges/. Accessed 10 Jan 2018
54. DIMACS (2012) Algorithm implementation challenge: graph partitioning and graph clustering. The tenth DIMACS implementation challenge. http://dimacs.rutgers.edu/Challenges/. Accessed 10 Jan 2018

55. Dorigo M, Maniezzo V, Colorni A (1996) Ant system: optimization by a colony of cooperating agents. IEEE Trans Syst Man Cybern B Cybern 26(1):29–41
56. Dorne R, Hao JK (1998) A new genetic local search algorithm for graph coloring. In: Parallel problem solving from nature. Springer, Berlin/Heidelberg, pp 745–754
57. Dowsland KA, Thompson JM (2005) Ant colony optimization for the examination scheduling problem. J Oper Res Soc 56(4):426–438
58. Dowsland KA, Thompson JM (2008) An improved ant colony optimisation heuristic for graph colouring. Discret Appl Math 156(3):313–324
59. Dukanovic I, Rendl F (2007) Semidefinite programming relaxations for graph coloring and maximal clique problems. Math Program 109(2–3):345–365
60. Dukanovic I, Rendl F (2008) A semidefinite programming-based heuristic for graph coloring. Discret Appl Math 156(2):180–189
61. Eiben ÁE, Van Der Hauw JK, van Hemert JI (1998) Graph coloring with adaptive evolutionary algorithms. J Heuristics 4(1):25–46
62. Erdös P (1970) On the graph theorem of Turán. Mat Lapok 21(249–251):10
63. Feige U, Kilian J (1996) Zero knowledge and the chromatic number. In: Proceedings of eleventh annual IEEE conference on computational complexity, pp 278–287
64. Feige U, Kilian J (1998) Zero knowledge and the chromatic number. J Comput Syst Sci 57:187–199
65. Fenet S, Solnon C (2003) Searching for maximum cliques with ant colony optimization. In: Applications of evolutionary computing. Springer, Berlin, pp 236–245
66. Feo TA, Resende MGC (1989) A probabilistic heuristic for a computationally difficult set covering problem. Oper Res Lett 8:67–71
67. Feo TA, Resende MGC (1995) Greedy randomized adaptive search procedures. J Glob Optim 6:109–133
68. Feo TA, Resende MGC, Smith SH (1994) A greedy randomized adaptive search procedure for maximum independent set. Oper Res 42(5):860–878
69. Ferland J, Fleurent C (1996) Object-oriented implementation of heuristic search methods for graph coloring, maximum clique and satisfiability. In: Johnson DS, Trick MA (eds) Cliques, coloring, and satisfiability: second DIMACS implementation challenge. American Mathematical Society, Providence, pp 619–652
70. Fleurent C, Ferland JA (1996) Genetic and hybrid algorithms for graph coloring. Ann Oper Res 63(3):437–461
71. Foster JA, Soule T (1995) Using genetic algorithms to find maximum cliques. Technical report. LAL95-12, Department of Computer Science, University of Idaho
72. Friden C, Hertz A, de Werra D (1989) STABULUS: a technique for finding stable sets in large graphs with tabu search. Computing 42:35–44
73. Funabiki N, Higashino T (2000) A minimal-state processing search algorithm for graph coloring problems. IEICE Trans Fundam Electron Commun Comput Sci 83(7):1420–1430
74. Galinier P, Hao JK (1999) Hybrid evolutionary algorithms for graph coloring. J Comb Optim 3(4):379–397
75. Galinier P, Hertz A (2006) A survey of local search methods for graph coloring. Comput Oper Res 33(9):2547–2562
76. Galinier P, Hertz A, Zufferey N (2008) An adaptive memory algorithm for the k-coloring problem. Discret Appl Math 156(2):267–279
77. Galinier P, Hamiez JP, Hao JK, Porumbel D (2013) Recent advances in graph vertex coloring. In: Handbook of optimization. Springer, Berlin/Heidelberg, pp 505–528
78. Gamst A (1986) Some lower bounds for a class of frequency assignment problems. IEEE Trans Veh Technol 35(1):8–14
79. Garey MR, Johnson DS (1979) Computers and intractability: a guide to the theory of NP-completeness. W.H. Freeman and Company, New York
80. Garey MR, Johnson DS, So H (1976) An application of graph coloring to printed circuit testing. IEEE Trans Circuits Syst 23(10):591–599

81. Gassen DW, Carothers JD (1993) Graph color minimization using neural networks. In: Proceedings of IEEE international joint conference on neural networks, vol 2, pp 1541–1544
82. Gendreau M, Soriano P, Salvail L (1993) Solving the maximum clique problem using a tabu search approach. Ann Oper Res 41(4):385–403
83. Gibbons LE, Hearn DW, Pardalos PM (1996) A continuous based heuristic for the maximum clique problem. In: Johnson DS, Trick MA (eds) Cliques, coloring, and satisfiability: second DIMACS implementation challenge. American Mathematical Society, Providence, pp 103–124
84. Glass CA, Prügel-Bennett A (2003) Genetic algorithm for graph coloring: exploration of Galinier and Hao's algorithm. J Comb Optim 7(3):229–236
85. Glover F (1989) Tabu search. Part I. ORSA J Comput 1(3):190–206
86. Goldberg DE (1989) Genetic algorithms in search, optimization, and machine learning. Addison-Wesley, Reading
87. Goldberg MK, Rivenburgh RD (1996) Constructing cliques using restricted backtracking. In: Johnson DS, Trick MA (eds) Cliques, coloring, and satisfiability: second DIMACS implementation challenge. American Mathematical Society, Providence, pp 285–307
88. Govorčin J, Gvozdenović N, Povh J (2013) New heuristics for the vertex coloring problem based on semidefinite programming. Cent Eur J Oper Res 21(1):13–25
89. Grable DA, Panconesi A (1998) Fast distributed algorithms for Brooks-Vizing colourings. In: Proceedings of the ninth annual ACM-SIAM symposium on discrete algorithms. Society for Industrial and Applied Mathematics, pp 473–480
90. Grossman T (1996) Applying the inn model to the maximum clique problem. In: Johnson DS, Trick MA (eds) Cliques, coloring, and satisfiability: second DIMACS implementation challenge. American Mathematical Society, Providence, pp 125–146
91. Grosso A, Locatelli M, Della Croce F (2004) Combining swaps and node weights in an adaptive greedy approach for the maximum clique problem. J Heuristics 10(2):135–152
92. Grosso A, Locatelli M, Pullan W (2008) Simple ingredients leading to very efficient heuristics for the maximum clique problem. J Heuristics 14(6):587–612
93. Gruzdeva TV (2013) On a continuous approach for the maximum weighted clique problem. J Glob Optim 56(3):971–981
94. Guturu P, Dantu R (2008) An impatient evolutionary algorithm with probabilistic tabu search for unified solution of some np-hard problems in graph and set theory via clique finding. IEEE Trans Syst Man Cybern B Cybern 38(3):645–666
95. Hajnal P, Szemerédi E (1990) Brooks coloring in parallel. SIAM J Discret Math 3(1):74–80
96. Hamiez JP, Hao JK (2002) Scatter search for graph coloring. In: Artificial evolution. Springer, Berlin/Heidelberg, pp 168–179
97. Hansen P, Jaumard B (1990) Algorithms for the maximum satisfiability problem. Computing 44(4):279–303
98. Hansen P, Mladenović N (1999) An introduction to variable neighborhood search. In: Voss S et al (eds) Meta-heuristics: advances and trends in local search paradigms for optimization. Springer, Boston, pp 433–458
99. Hansen P, Mladenović N, Urošević D (2004) Variable neighborhood search for the maximum clique. Discret Appl Math 145(1):117–125
100. Hao JK, Wu Q (2012) Improving the extraction and expansion method for large graph coloring. Discret Appl Math 160(16):2397–2407
101. Håstad J (1999) Clique is hard to approximate within $n^{1-\epsilon}$. Acta Math 182:105–142
102. Held S, Cook W, Sewell EC (2012) Maximum-weight stable sets and safe lower bounds for graph coloring. Math Program Comput 4(4):363–381
103. Hertz A, de Werra D (1987) Using tabu search techniques for graph coloring. Computing 39(4):345–351
104. Hertz A, Zufferey N (2006) A new ant algorithm for graph coloring. In: Workshop on nature inspired cooperative strategies for optimization, NICSO, pp 51–60
105. Hertz A, Plumettaz M, Zufferey N (2008) Variable space search for graph coloring. Discret Appl Math 156(13):2551–2560

106. Hifi M (1997) A genetic algorithm-based heuristic for solving the weighted maximum independent set and some equivalent problems. J Oper Res Soc 48(6):612–622
107. Holland JH (1992) Adaptation in natural and artificial systems: an introductory analysis with applications to biology, control, and artificial intelligence. MIT Press, Cambridge
108. Homer S, Peinado M (1996) Experiments with polynomial-time clique approximation algorithms on very large graphs. DIMACS Ser Discret Math Theor Comput Sci 26:147–168
109. Horst R, Thoai NV (1999) Dc programming: overview. J Optim Theory Appl 103(1):1–43
110. Hungerford JT, Rinaldi F (2019) A general regularized continuous formulation for the maximum clique problem. Math Oper Res 44(4):1161–1173
111. Jagota A (1992) Efficiently approximating MAX-CLIQUE in a Hopfield-style network. In: International joint conference on neural networks, vol 2, pp 248–253
112. Jagota A (1995) Approximating maximum clique with a Hopfield network. IEEE Trans Neural Netw 6(3):724–735
113. Jagota A (1996) An adaptive, multiple restarts neural network algorithm for graph coloring. Eur J Oper Res 93(2):257–270
114. Jagota A, Sanchis LA (2001) Adaptive, restart, randomized greedy heuristics for maximum clique. J Heuristics 7(6):565–585
115. Jagota A, Sanchis L, Ganesan R (1996) Approximately solving maximum clique using neural networks and related heuristics. DIMACS Ser Discret Math Theor Comput Sci 26:169–204
116. Jerrum M (1992) Large cliques elude the metropolis process. Random Struct Algorithm 3(4):347–359
117. Jin Y, Hao JK (2015) General swap-based multiple neighborhood tabu search for the maximum independent set problem. Eng Appl Artif Intell 37:20–33
118. Johnson DS, Trick MA (eds) (1996) Cliques, coloring, and satisfiability: second DIMACS implementation challenge. American Mathematical Society, Providence
119. Johnson DS, Aragon CR, McGeoch LA, Schevon C (1991) Optimization by simulated annealing: an experimental evaluation; part II, graph coloring and number partitioning. Oper Res 39(3):378–406
120. Kahruman-Anderoglu S, Buchanan A, Butenko S, Prokopyev O (2016) On provably best construction heuristics for hard combinatorial optimization problems. Networks 67:238–245. https://doi.org/10.1002/net.21620
121. Karchmer M, Naor J (1988) A fast parallel algorithm to color a graph with Δ colors. J Algorithm 9(1):83–91
122. Karger D, Motwani R, Sudan M (1998) Approximate graph coloring by semidefinite programming. J ACM (JACM) 45(2):246–265
123. Karloff HJ (1989) An NC algorithm for Brooks' theorem. Theor Comput Sci 68(1):89–103
124. Karp RM (1972) Reducibility among combinatorial problems. In: Miller RE, Thatcher JW (eds) Complexity of computer computations. Plenum, New York, pp 85–103
125. Katayama K, Hamamoto A, Narihisa H (2005) An effective local search for the maximum clique problem. Inf Process Lett 95(5):503–511
126. Kirkpatrick S, Gelatt CD, Vecchi M (1983) Optimization by simulated annealing. Science 220(4598):671–680
127. Knuth DE (1994) The sandwich theorem. Electron J Comb 1:1–48. http://www.combinatorics.org/Volume_1/Abstracts/v1i1a1.html. Accessed 10 Jan 2018
128. Kopf R, Ruhe G (1987) A computational study of the weighted independent set problem for general graphs. Found Control Eng 12(4):167–180
129. Laguna M, Martí R (2001) A GRASP for coloring sparse graphs. Comput Optim Appl 19(2):165–178
130. Lamm S, Sanders P, Schulz C, Strash D, Werneck RF (2017) Finding near-optimal independent sets at scale. J. Heuristics 23(4):207–229
131. Leighton FT (1979) A graph coloring algorithm for large scheduling problems. J Res Natl Bur Stand 84(6):489–506
132. Lemke CE (1965) Bimatrix equilibrium points and mathematical programming. Manag Sci 11(7):681–689

133. Leskovec J, Krevl A (2014) SNAP datasets: Stanford large network dataset collection. http://snap.stanford.edu/data. Accessed 10 Jan 2018
134. Lovász L (1975) Three short proofs in graph theory. J Comb Theory Ser B 19(3):269–271
135. Lovász L (1979) On the Shannon capacity of a graph. IEEE Trans Inf Theory 25(1):1–7
136. Lovász L, Plummer MD (2009) Matching theory. American Mathematical Society, Providence
137. Lü Z, Hao JK (2010) A memetic algorithm for graph coloring. Eur J Oper Res 203(1): 241–250
138. Makovenko M, Butenko S, Pardalos M (2025) Regularized polynomial formulations for the maximum clique problem. Comput Optim Appl (2025). https://doi.org/10.1007/s10589-025-00674-z
139. Malaguti E, Toth P (2010) A survey on vertex coloring problems. Int Trans Oper Res 17(1): 1–34
140. Malaguti E, Monaci M, Toth P (2008) A metaheuristic approach for the vertex coloring problem. INFORMS J Comput 20(2):302–316
141. Malaguti E, Monaci M, Toth P (2011) An exact approach for the vertex coloring problem. Discret Optim 8(2):174–190
142. Mannino C, Sassano A (1996) Edge projection and the maximum cardinality stable set problem. DIMACS Ser Discret Math Theor Comput Sci 26:205–219
143. Marchiori E (1998) A simple heuristic based genetic algorithm for the maximum clique problem. In: Proceedings of the ACM symposium on applied computing, vol 27, pp 366–373
144. Marchiori E (2002) Genetic, iterated and multistart local search for the maximum clique problem. In: Applications of evolutionary computing. Springer, Berlin/Heidelberg, pp 112–121
145. Massaro A, Pelillo M, Bomze IM (2002) A complementary pivoting approach to the maximum weight clique problem. SIAM J Optim 12(4):928–948
146. Matula DW, Marble G, Isaacson JD (1972) Graph coloring algorithms. In: Read R (ed) Graph theory and computing. Academic, New York, pp 109–122
147. Mehta NK (1981) The application of a graph coloring method to an examination scheduling problem. Interfaces 11(5):57–65
148. Min Y, Wenkel F, Perlmutter M, Wolf G (2022) Can hybrid geometric scattering networks help solve the maximum clique problem? Adv Neural Inf Process Syst 35:22713–22724
149. Mladenović N, Hansen P (1997) Variable neighborhood search. Comput Oper Res 24(11):1097–1100
150. Morgenstern C (1996) Distributed coloration neighborhood search. Discret Math Theor Comput Sci 26:335–358
151. Morgenstern CA, Shapiro HD (1986) Chromatic number approximation using simulated annealing. Technical report, Department of Computer Science, University of New Mexico
152. Motzkin TS, Straus EG (1965) Maxima for graphs and a new proof of a theorem of Turán. Can J Math 17(4):533–540
153. Mumford CL (2006) New order-based crossovers for the graph coloring problem. In: Parallel problem solving from nature. Springer, Berlin, pp 880–889
154. Murthy AS, Parthasarathy G, Sastry V (1994) Clique finding – a genetic approach. In: Proceedings of the first IEEE conference on evolutionary computation, pp 18–21
155. Ouyang Q, Kaplan PD, Liu S, Libchaber A (1997) DNA solution of the maximal clique problem. Science 278(5337):446–449
156. Paquete L, Stützle T (2002) An experimental investigation of iterated local search for coloring graphs. In: Applications of evolutionary computing. Springer, Berlin/Heidelberg, pp 122–131
157. Pardalos PM, Phillips A (1990) A global optimization approach for solving the maximum clique problem. Int J Comput Math 33(3–4):209–216
158. Pardalos PM, Xue J (1994) The maximum clique problem. J Glob Optim 4(3):301–328
159. Pardalos PM, Mavridou T, Xue J (1999) The graph coloring problem: a bibliographic survey. In: Handbook of combinatorial optimization. Springer, Boston, pp 1077–1141

160. Park K, Carter B (1995) On the effectiveness of genetic search in combinatorial optimization. In: Proceedings of the ACM symposium on applied computing, pp 329–336
161. Pattillo J, Butenko S (2011) Clique, independent set, and graph coloring. In: Encyclopedia of operations research and management science. Wiley, Hoboken, pp 3150–3163
162. Philipsen W, Stok L (1991) Graph coloring using neural networks. In: IEEE international symposium on circuits and systems, pp 1597–1600
163. Plumettaz M, Schindl D, Zufferey N (2010) Ant local search and its efficient adaptation to graph colouring. J Oper Res Soc 61(5):819–826
164. Porumbel DC, Hao JK, Kuntz P (2010) An evolutionary approach with diversity guarantee and well-informed grouping recombination for graph coloring. Comput Oper Res 37(10):1822–1832
165. Porumbel DC, Hao JK, Kuntz P (2010) A search space "cartography" for guiding graph coloring heuristics. Comput Oper Res 37(4):769–778
166. Pullan W (2006) Phased local search for the maximum clique problem. J Comb Optim 12(3):303–323
167. Pullan W, Hoos HH (2006) Dynamic local search for the maximum clique problem. J Artif Intell Res 25:159–185
168. Pullan W, Mascia F, Brunato M (2011) Cooperating local search for the maximum clique problem. J Heuristics 17(2):181–199
169. Resende MGC, Feo TA, Smith SH (1998) Algorithm 787: Fortran subroutines for approximate solution of maximum independent set problems using grasp. ACM Trans Math Softw (TOMS) 24(4):386–394
170. Singh A, Gupta AK (2006) A hybrid heuristic for the maximum clique problem. J Heuristics 12(1–2):5–22
171. Skulrattanakulchai S (2006) Δ-list vertex coloring in linear time. Inf Process Lett 98(3): 101–106
172. Sloane N (2000) Challenge problems: independent sets in graphs. https://oeis.org/A265032/a265032.html. Accessed 10 Jan 2018
173. Solnon C, Fenet S (2006) A study of ACO capabilities for solving the maximum clique problem. J Heuristics 12(3):155–180
174. Soriano P, Gendreau M (1996) Diversification strategies in tabu search algorithms for the maximum clique problem. Ann Oper Res 63(2):189–207
175. Soriano P, Gendreau M (1996) Tabu search algorithms for the maximum clique problem. In: Johnson DS, Trick MA (eds) Cliques, coloring, and satisfiability: second DIMACS implementation challenge. American Mathematical Society, Providence, pp 221–242
176. Takefuji Y, Lee KC (1991) Artificial neural networks for four-coloring map problems and k-colorability problems. IEEE Trans Circuits Syst 38(3):326–333
177. Talaván PM, Yáñez J (2008) The graph coloring problem: a neuronal network approach. Eur J Oper Res 191(1):100–111
178. Titiloye O, Crispin A (2011) Graph coloring with a distributed hybrid quantum annealing algorithm. In: Agent and multi-agent systems: technologies and applications. Springer, Berlin/Heidelberg, pp 553–562
179. Titiloye O, Crispin A (2011) Quantum annealing of the graph coloring problem. Discret Optim 8(2):376–384
180. Verma A, Buchanan A, Butenko S (2015) Solving the maximum clique and vertex coloring problems on very large sparse networks. INFORMS J Comput 27(1):164–177
181. Walteros JL, Buchanan A (2020) Why is maximum clique often easy in practice? Oper Res 68(6):1866–1895
182. Welsh DJ, Powell MB (1967) An upper bound for the chromatic number of a graph and its application to timetabling problems. Comput J 10(1):85–86
183. de Werra D (1985) An introduction to timetabling. Eur J Oper Res 19(2):151–162
184. de Werra D (1990) Heuristics for graph coloring. In: Computational graph theory. Springer, Berlin, pp 191–208

185. de Werra D, Gay Y (1994) Chromatic scheduling and frequency assignment. Discret Appl Math 49(1):165–174
186. Woo TK, Su SY, Newman-Wolfe R (1991) Resource allocation in a dynamically partitionable bus network using a graph coloring algorithm. IEEE Trans Commun 39(12):1794–1801
187. Wood D (1969) A technique for colouring a graph applicable to large scale timetabling problems. Comput J 12(4):317–319
188. Wu Q, Hao JK (2012) Coloring large graphs based on independent set extraction. Comput Oper Res 39(2):283–290
189. Wu Q, Hao JK (2013) An adaptive multistart tabu search approach to solve the maximum clique problem. J Comb Optim 26(1):86–108
190. Wu Q, Hao JK (2013) An extraction and expansion approach for graph coloring. Asia Pac J Oper Res 30(05):1350018
191. Wu Q, Hao JK (2015) A review on algorithms for maximum clique problems. Eur J Oper Res 242(3):693–709
192. Wu Q, Hao JK, Glover F (2012) Multi-neighborhood tabu search for the maximum weight clique problem. Ann Oper Res 196:611–634
193. Youssef SM, Elliman DG (2004) Reactive prohibition-based ant colony optimization (RPACO): a new parallel architecture for constrained clique sub-graphs. In: 16th IEEE international conference on tools with artificial intelligence, pp 63–70
194. Zhang BT, Shin SY (1999) Code optimization for DNA computing of maximal cliques. In: Advances in soft computing. Springer, Heidelberg, pp 588–599
195. Zhang Q, Sun J, Tsang E (2005) An evolutionary algorithm with guided mutation for the maximum clique problem. IEEE Trans Evol Comput 9(2):192–200
196. Zuckerman D (2007) Linear degree extractors and the inapproximability of max clique and chromatic number. Theory Comput 3:103–128
197. Zufferey N (2012) Optimization by ant algorithms: possible roles for an individual ant. Optim Lett 6(5):963–973

The Multiplant Lot Sizing Problem with Multiple Periods and Items

58

Mariá C. V. Nascimento, Horacio H. Yanasse, Desiree M. Carvalho, and Silvio Alexandre de Araujo

Contents

Introduction . . . 1748
The Lot Sizing Problem . . . 1749
On the Complexity . . . 1751
The Multiplant, Multi-item Lot Sizing Problem . . . 1752
Network Flow-Based Formulation . . . 1755
Computational Experiments . . . 1758
Setup of the Experiments . . . 1758
Results of the Experiments . . . 1759
Extensions for the Capacitated Problem . . . 1761
Extended Classical Formulation for the MPCLSP-PFT . . . 1761
Facility Location Reformulation for the MPCLSP-PFT . . . 1762
Computational Experiments for the MPCLSP-PFT . . . 1765
Results of the Experiments . . . 1766
Conclusions . . . 1767
Cross-References . . . 1768
References . . . 1768

M. C. V. Nascimento (✉)
Instituto Tecnológico de Aeronáutica (ITA), São José dos Campos, SP, Brazil
e-mail: maria.nascimento@gp.ita.br

H. H. Yanasse · D. M. Carvalho
Instituto de Ciência e Tecnologia, Universidade Federal de São Paulo - UNIFESP, São José dos Campos, SP, Brazil
e-mail: horacio.yanasse@unifesp.br; dmcarvalho@unifesp.br

S. A. de Araujo
Universidade Estadual Paulista-UNESP, São José do Rio Preto, SP, Brazil
e-mail: silvio.araujo@unesp.br

© Springer Nature Switzerland AG 2025

R. Martí et al. (eds.), *Handbook of Heuristics*,
https://doi.org/10.1007/978-3-032-00385-0_41

Abstract

The lot sizing problem consists in determining lot sizes and their scheduling to meet the required demands. This chapter focuses on a multiplant lot sizing problem with a planning with multiple items, each plant with their own demands and multiple periods. In spite of not been widely investigated, their theoretical and practical importance are supported by applications from private and public sectors. The first part of this chapter pays special attention to the multiplant uncapacitated lot sizing problem (MPULSP). In particular, a novel network flow formulation is introduced, and some computational experiments were carried out to assess the performance of commercial solvers in solving a number of large instance problems. Moreover, a comparative analysis is performed by considering two other formulations for the MPULSP found in the literature. The second part of this chapter turns its attention to the multiplant capacitated lot sizing problem (MPCLSP) and some extensions that add flexibility to the production process. Mathematical formulations are presented and evaluated through computational results.

Keywords

Lot sizing · Mathematical formulations · Comparative analysis

Introduction

Lot sizing plays an important role in improving the profits of the companies in addition to other goal effects, as, for example, the environmental impacts. Many features and requirements of the companies influence and impose additional constraints and limitations on a lot sizing problem. As a consequence, a number of variants on this subject, each of them suitable for particular applications, can be found in the literature.

Buschkühl et al. [3] and Karimi et al. [11] defined, in a comprehensive literature review, the primary characteristics and existing variants of lot sizing problems. Nevertheless, Karimi et al. [11] do not mention lot sizing problems involving more than one plant, best known as multiplant lot sizing problems (MPLSP). Buschkühl et al. [3] briefly discuss the MPLSP, but because of the general scope of their review in dynamic lot sizing problems, they do not go into detail on this problem. Highly relevant in certain industry sectors such as mattress companies [17], the MPLSP has received a reasonable amount of attention in the last two decades even though a very small number of solution methods have been studied to approach it. This chapter, therefore, focuses on the MPLSP presenting the state-of-the-art solution methods and mathematical models.

There are two cases of the MPLSP in the literature: the capacitated case (MPCLSP) [17] and the uncapacitated case (MPULSP) [16]. The studies present in [4,5,17,20] approach two mathematical formulations for the capacitated case. To approach the uncapacitated case, it is enough to delete the capacity constraints of

the capacitated models. Both problems have been proved to be NP-hard [16]. The solution methods of the literature mostly focus on the MPCLSP [5, 14, 17, 18].

The Lot Sizing Problem

Lot sizing is a research topic widely studied due to its importance for the production planning. According to Karimi et al. [11], lot sizing belongs to the medium-term decision-making, when it is necessary to determine the timing and how much of the items to produce to meet the demands and minimize the total costs. This tactical planning is responsible for the productivity of the company, where the decisions must consider the requirements and limitations of the manufacturing process.

Many variants of the lot sizing problem and solution methods to solve them can be found in the literature. Among the numerous features that define the different variations of the problem, it is highlighted the finite planning horizon divided into periods, single or multiple production levels, single or multiple items, single or multiple machines in each plant, single or multiple plants, uncapacitated or capacitated, stochastic or deterministic demand, with or without setup time, and limited or unlimited inventory capacity or not. These characteristics define the primary traits of the classical lot sizing problems.

The next section further describes these features loosely based on [11].

Demands

The demand of an item is the required amount to produce in the desired production planning. This amount comprises the estimated value according to the history of the productive process or the quantity of the order of clients. The types of demands are stationary (static or dynamic). Stationary demands mean they do not vary over time. The values of dynamic demands may change over time [23].

Apart from being stationary or dynamic, the demands can be deterministic or nondeterministic. The determinism on demands means that they are known beforehand, and most of the existing studies in the literature consider this type of demand. The nondeterministic or stochastic demands depend on a probability function or an information subject to uncertainty.

Planning Horizon

Scheduling the production up to a certain time in the future is a relevant characteristic of lot sizing problems. In general, one defines this planning as finite, infinite, and rolling. The main characteristics regarding these types of planning horizon are enumerated as follows.

- The time interval of a finite planning horizon is limited.
- There is no time limit imposed in an infinite planning horizon. Demands, in this case, are generally stationary.
- The rolling planning horizon is a strategy that projects the actual planning horizon that has been scheduled to part of or to the entire next interval of time. This planning aims at keeping running the production of a firm based on past planning.

The lot sizing problems with rolling planning horizon are hard to evaluate as discussed in [8].

Items
The production system can have a single or multiple types of items to produce. In single-item systems, only one type of item is considered as the final product. Multi-item systems involve the production of more than one type of item as the final product.

Multi-item production may be an item independent problem. In problems with item dependency, an item can only be manufactured after the production of another item. Dependencies on items are usually found in problems with more than one level.

Levels
The dependence between tasks performed for the transformation of the raw material into a final product is modeled by levels in the production system.

- Single level: In a single-level production system, the transformation of the raw material into the final product (item) is direct. This means that there are no intermediate steps, that is, the production of items is independent.
- Multilevel: The production of some final product (item) depends on the operations performed at different levels of the production system. In other words, the production of an item depends on the availability of items produced at previous levels.

Plants
A production system must have a facility (plant) where the items are produced. There are problems in which there is more than one production facility. In these multiplant lot sizing problems, each plant has its own demand and capacity, and lot transfers between plants are possible subject to the transportation costs.

Machines
A production facility may have a single or multiple machines to produce items. Multiple parallel machines produce the same items. The problem can be uncapacitated and capacitated when the machine production is limited by an upper bound and is located at the same facility.

Setup Structure
A setup cost is incurred to produce an item type, and a setup time is required to prepare the machine. These two components characterize the machine setup structure in a production system, which can be classified as simple or complex. In the simple setup, the time and cost of preparation are not influenced by previous periods' production. In the complex setup, both time and cost of setup may be dependent on the sequencing of tasks and decisions taken in previous periods, as follows:

- Setup carry-over: If the preparation of the machine is ready at the end of a period for the production of a certain type of item, there is no need to re-prepare the machine in the beginning of the following period, if the same type of item is the first to be produced.
- Setup crossover: This type of setup enables a setup starting at the end of a period be interrupted and resumed in the beginning of the next period.
- Setup by similarity: The preparation is carried out taking into consideration the group of items to be produced. Similar items can be produced without additional preparation if they are produced in the sequence.
- Sequence-dependent setup: The preparation is dependent on the sequence determined for the production of the items.

Inventory and Backlogging
When the amount of production of an item exceeds the demand for the item in the period produced, the production stock may be subject to costs, and it may be restricted to the capacity of the storage warehouse, among other factors [17, 18]. In another case, a delay (backlogging) in the supply of demand is allowed, subject to some penalty costs.

Some additional characteristics for the MPLSP are presented as follows.

Flexibility
In the standard lot sizing problem with multiple plants, each item can be produced at any plant, incurring setup costs and time before production begins. This scenario assumes complete plant flexibility. However, in practice, installing plants with full flexibility can be prohibitively expensive, particularly if the products vary significantly. Consequently, it may be more practical to implement limited flexibility, where each plant is capable of producing only specific types of items.

Clients
The MPLSP involves producing various items across multiple plants. In a network of geographically dispersed plants and clients, distributing these items can be managed through direct shipments from the plants to the clients. This requires careful consideration of transportation costs to optimize the overall distribution efficiency.

On the Complexity

Wagner and Whitin [23] proved that the uncapacitated lot sizing problem with multiple items (or a single item), setup time, and deterministic demand can be solved in polynomial time by a dynamic programming algorithm. Bitran and Yanasse [1] proved that the capacitated lot sizing problem is NP-hard. Maes et al. [13] proved that to decide whether an instance of the capacitated lot sizing problem with setup times is feasible is NP-complete. The existing studies include new problems, reviews, and surveys and they present methods, their computational complexity, and

mathematical formulations [2,3,8,10,11,15]. The computational complexity of the different variants of the lot sizing problem depends on their features, such as limited capacity and existence of setup times.

Among the plethora of variants and possibilities to approach this problem, this chapter limits the attention to the description of the multiplant lot sizing problem with setup time and unlimited inventory.

Different from most uncapacitated lot sizing problems that, even with setup times have polynomial time solution, the MPULSP has been proved NP-hard [16].

The Multiplant, Multi-item Lot Sizing Problem

The multiplant lot sizing problem (MPLSP), where all plants produce the same items and have limited capacity, is known as multiplant capacitated lot sizing problem (MPCLSP). For this problem, the literature on mathematical formulations presents two different mixed integer programs. First, in this chapter, the formulation proposed in [16] is presented.

Let one define the following parameters of the problem.

n: number of items
m: number of plants
p: number of periods
cs_{ij}: setup cost for producing item i at plant j
c_{ij}: unitary production cost of item i at plant j
h_{ij}: unitary inventory cost of item i at plant j
r_{jk}: unitary transportation cost of items from plant j to plant k
d_{ijt}: demand of item i at plant j in period t
b_{ij}: processing time of item i at plant j
f_{ij}: setup time for preparing the machine for producing item i at plant j
C_{jt}: capacity of production at plant j in period t

To formulate the MPCLSP, Sambasivan and Schimidt [17] considered the following variables:

x_{ijt}: the quantity of item i produced at plant j in period t
y_{ijt}: binary setup variable that receives 1 if item i is produced at plant j in period t, and 0, otherwise
I_{ijt}: variable that controls the inventory of item i at plant j at the end of period t
w_{ijkt}: variable that specifies the amount of item i transferred from plant j to plant k in period t

$$\min \sum_{i=1}^{n} \sum_{j=1}^{m} \sum_{t=1}^{p} (cs_{ij} y_{ijt} + c_{ij} x_{ijt} + h_{ij} I_{ijt} + \sum_{k \neq j} r_{jk} w_{ijkt}) \tag{1}$$

subject to

$$x_{ijt} + I_{ij,t-1} - I_{ijt} - \sum_{k \neq j} w_{ijkt} + \sum_{l \neq j} w_{iljt} = d_{ijt} \qquad \forall i, j, t \qquad (2)$$

$$\sum_{i=1}^{n} (b_{ij} x_{ijt} + f_{ij} y_{ijt}) \leq C_{jt} \qquad \forall j, t \qquad (3)$$

$$x_{ijt} \leq L y_{ijt} \qquad \forall i, j, t \qquad (4)$$

$$y_{ijt} \in \{0, 1\}, x_{ijt} \geq 0, I_{ijt} \geq 0, w_{ijkt} \geq 0 \qquad \forall i, j, t; j \neq k \qquad (5)$$

The objective function (1) is the minimization of the sum of setup, production, inventory, and transfer costs. Constraints (2) balance inventory production transference and demand of items at each plant in each period t. Constraints (3) limit the time production capacity of plant-period (j, t). Constraints (4) ensure the production of item i at plant j in period t can occur only if the corresponding setup is performed. Finally, constraints (5) define the domain of the variables.

Silva [19] proposed a mathematical formulation for the MPULSP based on the study found in [9]. Carvalho and Nascimento [4] refined this integer program and the formulation, presented next.

This formulation possesses the following parameters:

$$\bar{c}_{ijtku} = \begin{cases} 0, \text{ if } u < t, \\ c_{ij} + \chi_{ijtku}, \text{ otherwise} \end{cases}$$

where

χ_{ijtku} is the inventory and transfer costs for producing one unit of item i at plant j in period t to meet the unitary demand of item i at plant k in period u. It is given by

$$\chi_{ijtku} = \min\{(u-t)h_{ij} + r_{jk}, \min_{1 \leq v \leq p} \{(u-t)h_{iv} + r_{jv} + r_{vk}\}, (u-t)h_{ik} + r_{jk}\} \qquad (6)$$

$\bar{c}_{ijtku}$ is the cost of producing one unity of item i at plant j in period t plus the least inventory and transportation costs of this unity of i from plant j to plant k from period u to t. In addition to y_{ijt} defined as previously, the following variables are used:

x_{ijtku} is the production of item i at plant j in period t to meet the demand of item i at plant k in period u.

The model of [4] for the MPCLSP is

$$\min \sum_{i=1}^{n}\sum_{j=1}^{m}\sum_{t=1}^{p}\sum_{k=1}^{m}\sum_{u=1}^{p}(\overline{c}_{ijtku}x_{ijtku}) + \sum_{i=1}^{n}\sum_{j=1}^{m}\sum_{t=1}^{p}(cs_{ij}y_{ijt}) \tag{7}$$

subject to

$$\sum_{j=1}^{m}\sum_{t=1}^{u} x_{ijtku} = d_{iku} \qquad \forall(i,k,u) \tag{8}$$

$$\sum_{i=1}^{n}\left(f_{ij}y_{ijt} + \sum_{k=1}^{m}\sum_{u=t}^{p} b_{ij}x_{ijtku}\right) \leq C_{jt} \qquad \forall(j,t) \tag{9}$$

$$x_{ijtku} \leq \min\left\{d_{iku}; \left\lfloor \frac{C_{jt} - f_{ij}}{b_{ij}} \right\rfloor\right\} y_{ijt} \qquad \forall(i,j,t,k,u) \tag{10}$$

$$x_{ijtku} \in \mathbb{N} \qquad \forall(i,j,t,k,u) \tag{11}$$

$$y_{ijt} \in \{0,1\} \qquad \forall(i,j,t) \tag{12}$$

The objective function aims at minimizing the total costs: production, setup, inventory, and transference between plants. Constraints (8) ensure that the demands of all plants are met; constraints (9) ensure the production capacity at each plant in every period is not violated. In constraints (10), if item i is produced at plant j in period t, then y_{ijt} assumes value 1, that is, a corresponding setup cost must be added to the total cost. Constraints (11) and (12), respectively, define the domain of the decision variables.

It is noteworthy that the number of variables of this formulation asymptotically grows faster with the size than the variables of the model of [17]. Consequently, there is a major influence on the performance of exact solution methods in solving the problems regarding different sizes of instances.

As some applications involving MPLSP have a large number of items, heuristics are solution methods that play an important role in this problem, as discussed in the next section.

Solution Methods

In the literature, one can find a few heuristic methods specially designed for solving both MPULSPand MPCLSP. They are a simple heuristic based on lot transfers [17], a Lagrangian heuristic [18], a Greedy Randomized Search Procedure (GRASP) with path relinking [14], and Lagrangian heuristics [4].

The Lagrangian heuristic proposed in [18] aims at finding a heuristic solution for the relaxed uncapacitated problem and posteriorly transfers lots of production of items as an attempt of achieving a feasible solution for the MPCLSP. The authors take into consideration the mathematical formulation proposed in [17] to develop their method. They evaluate the performance of the Lagrangian heuristic by using

artificial instances they generated. As a result of the experiments, the Lagrangian heuristic systematically achieved better results than the first heuristic proposed to solve the MPCLSP, found in [17].

Later, Nascimento et al. [14] besides proposing a novel hybrid metaheuristic set forth a new set of instances with a more diversified structure and classified them as indicated in the section of experiments of this paper. In their experiments using the set of instances introduced in [18], they show that the hybrid metaheuristic, GPheur, had a better performance than the Lagrangian heuristic [18]. In order to better investigate the performance of their metaheuristic, they carry out experiments with the set of instances they suggested. For these instances, GPheur could not find optimal solutions for the hard instances, even though the heuristic method achieved better results than CPLEX v. 7.5.

More recently, Carvalho and Nascimento [4] proposed a pair of Lagrangian heuristics that make use of CPLEX v.12.6 to find the optimal solution of the uncapacitated problem, by considering the model introduced in [19]. One is a Lagrangian heuristic, named by the authors as Lag, and the other is a hybrid version, referred to as LaPRe. For finding feasible solutions for the capacitated problem, both feasibility strategies, adapted for tackling the MPCLSP from [21] and the local search phase of the GRASP from [14], were embedded in the strategy. To enhance the quality of the solutions, the path relinking, also proposed in [14], was used to compose the hybrid Lagrangian heuristic. In experiments, the authors show that both LaPRe and Lag significantly outperformed GPheur achieving feasible solutions for the whole set of instances. However, for smaller instances, CPLEX v. 12.6 outperformed the Lagrangian heuristics on most of the classes of instances. On the other hand, when considering large instances, CPLEX could not find good solutions for all instances in a limit of 1800 seconds established by the authors to run each instance of the experiment, whereas the Lagrangian heuristics presented better lower and upper bounds.

Network Flow-Based Formulation

Because of the importance of the multiplant lot sizing problems, this chapter gives a new insight to the MPULSP. For this, it presents a network flow-based formulation for the MPULSP, different from the formulations proposed in the literature.

In unlimited capacity lot sizing problems, the production of items is independent. Therefore, the formulation will disregard item indices, without loss of generality.

In a network flow, consider the following parameters:

cs_j: setup cost of an item at plant j
d_{jt}: demand in plant j in each period t
h_j: inventory cost at each plant j
c_j: production cost of an item at plant j
r_{jk}: transfer cost between plants j and k
M: sum of the demands of every plant-period

Therefore, if a machine is ready to produce an item, the cost in producing at plant j in period t to meet the demand at plant k in period u is

$$\chi'_{jtku} = c_j + \min\{(u-t)h_j + r_{jk}, \min_{1\le v\le p}\{(u-t)h_v + r_{jv} + r_{vk}\}, (u-t)h_k + r_{jk}\}$$

In the most general case, other plants can be considered in the transfer of the produced item. To calculate the costs of the arcs, it is necessary to determine them by solving the minimum cost problem between the pair of vertices of the network. Consider a toy example with two plants and three periods as illustrated in Fig. 1.

In Fig. 1, node 0 is the source of the network. It receives the sum of the demands of all periods in each plant. The nodes represented as yellow rectangles are the setup nodes, whereas round blue nodes indicate the demands to be met. The labels of yellow nodes, s_{kt}, indicate the setup at plant k in period t. The labels of round nodes, kt, represent the planning in the plant k at period t. A sink node is associated with each blue node, to represent the addressing of the corresponding demand. Arcs with heads incident to demand nodes are only those with tails in setup nodes or with tails from the immediately previous period at the same plant. These conditions ensure the machine to be prepared for producing that item. Aggregated to such arcs are the aforementioned costs. Moreover, arcs with head in setup nodes have tails in the production or source nodes.

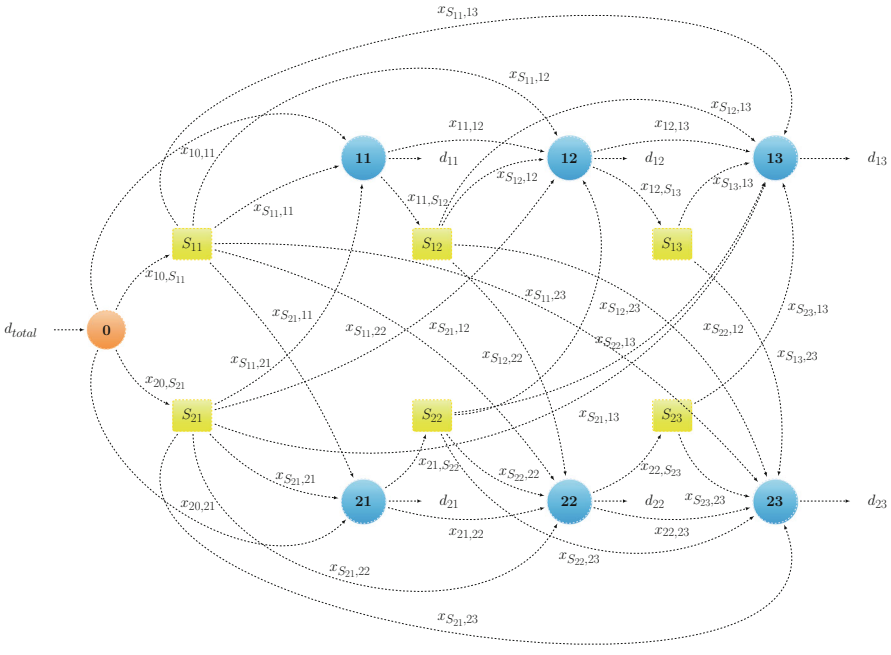


Fig. 1 Example with two plants and three periods

To formally introduce the network flow-based problem, referred to here as NYC, consider the following variables:

y_{jt}	is a binary setup variable that receives 1 if there is setup to produce demands at plant j in period t, and 0, otherwise.
$x_{jt-1,jt}$	is an arc that carries demands from plant j not produced up to period $t-1$.
$x_{jt-1,s_{jt}}$	is the amount of demand not met up to period $t-1$ to be produced at plant j in period t.
$x_{s_{jt},ku}$	is the amount of demand produced at plant j in period t to meet the demand of plant k in period u.

The domain of the indexes is the following: $j, k \in \{1, \ldots m\}$; $t \in \{1, \ldots, p\}$; $u \in \{t, \ldots, p\}$. The general formulation is, thus, the following:

$$\min \sum_{j=1}^{m}\sum_{k=1}^{m}\sum_{t=1}^{p}\sum_{u=t}^{p} \chi'_{jtku} x_{s_{jt},ku} + \sum_{j=1}^{m}\sum_{t=1}^{p} cs_j y_{jt} \tag{13}$$

subject to

$$\sum_{j=1}^{m}(x_{j0,s_{j1}} + x_{j0,j1}) = \sum_{j=1}^{m}\sum_{t=1}^{p} d_{jt} \tag{14}$$

$$x_{jt-1,s_{jt}} = \sum_{k=1}^{m}\sum_{u=t}^{p} x_{s_{jt},ku} \qquad \forall j, t \tag{15}$$

$$x_{jt,s_{j,t+1}} + x_{jt,jt+1} - x_{jt-1,jt} = 0 \qquad \forall j, t = 1, \ldots, p-1 \tag{16}$$

$$\sum_{j=1}^{m}\sum_{t=1}^{u} x_{s_{jt},ku} = d_{ku} \qquad \forall k, u \tag{17}$$

$$x_{jt-1,s_{jt}} \leq M y_{jt} \qquad \forall j, t \tag{18}$$

$$y_{jt} \in \{0, 1\} \qquad \forall j, t \tag{19}$$

$$x_{s_{jt},ku}, x_{jt-1,s_{jt}}, x_{jt-1,jt} \geq 0 \qquad \forall j, k, t \leq u \tag{20}$$

Constraints (14) define the set of source nodes assigning to them the sum of the demands of the plants. Constraints (15) refer to the flow conservation constraints of setup nodes. Constraints (16) ensure the flow conservation of production nodes. Constraints (17) ensure that the demands of all plants are met. Constraints (18) ensure the setup variables to be 1 if there is production in period t, whereas constraints (19) and (20) define the domains of the variables of the model.

In the next section, computational experiments were carried out to evaluate the performance of CPLEX v.12.6 with the three models discussed in this chapter.

Computational Experiments

For the analysis of the proposed network flow-based model, it was performed an experiment comparing its results with those achieved by CPLEX v.12.6 considering the formulations presented in [17] and [19] refined and introduced in [4].

All algorithms were implemented in C++ language and the experiments carried out in an Ubuntu Server 14.04, Intel core i7 with 3.4 GHz, CPU 500 MB and 16 GB DDR RAM.

For the experiment, it was employed a set of instances generated as suggested in [14]. It is a set of eight classes of datasets with 160 instances. The division into classes was due to three main characteristics of the instance: capacity, setup cost, and setup time. Accordingly, the instances were classified with tight (T) or normal (N) capacity. Considering $\beta = \sum_{t=1}^{p} \sum_{i=1}^{n} \frac{(d_{ijt} b_{ij} + f_{ij})}{p}$, instances with normal and tight capacities have their parameter C_{jt} set as $1 \times \beta$ and $0.9 \times \beta$, respectively.

Regarding the setup costs, instances were defined with low (L) or high (H). In the former, cs_{ij} was chosen randomly within interval [5.0; 95.0]. In the latter, instances were chosen randomly within interval [50.0; 950.0].

To define each f_{ij}, it was picked randomly a value within interval [10.0; 50.0] when considering an instance with low setup time (L) and from interval [15.0; 75.0] if it was a high setup time (H) instance.

The remainder parameters, c_{ijt}, h_{ij}, r_{jk}, b_{ij}, and d_{ijt}, were set selecting at random values from the respective intervals: [1.5; 2.5], [0.2; 0.4], [0.2; 0.4], [1.0; 5.0], and [0, 180.0].

Therefore, a class of instances of the type NHL has instances with normal capacity, high setup cost, and low setup time. In the first set of instances, generated in [14], each instance corresponds to a problem with 12 periods, 2 plants, and 100, 200, 300, or 400 items. For each combination, the set has 5 problems, totaling 20 instances in each class.

Setup of the Experiments

To describe the results of the experiments and computationally compare the models, the performance profiles of [7] will be employed. This profile consists in a graphic that displays the relation between the metric to be evaluated, in this case, the time to solve every instance, and the performance of each algorithm in solving the instances, being the reference the best solution times found among all algorithms. For this, consider the ratio in Eq. (21).

$$\rho_{a,\tau} = \frac{1}{|\mathscr{S}|} \{s \in \mathscr{S} : r_{sa} \leq \tau\} \tag{21}$$

The factor τ in Eq. (21) means specifically the factor that multiplied by the best solution found considering all algorithms results in a lower bound for the solution

found by algorithm a for instance s. Therefore, r_{sa} can be calculated as indicated in Eq. (22).

$$r_{sa} = \frac{t_{sa}}{\min\{t_{sa'} : a' \in \mathscr{A}\}} \tag{22}$$

In Eq. (22), t_{sa} corresponds to the metric used for evaluating the solutions, which must be minimized. Therefore, r_{sa} is the performance of algorithm a to solve instance s in comparison to every algorithm. Consequently, $\rho_{a,\tau}$ is the percentage of solutions of the set of instances solved by algorithm a that respects the bound τ.

Results of the Experiments

In this experiment, the network flow-based formulation, named NYC, the model presented in [17], SS02, and the formulation adapted in [4], named CN16, were compared. As mentioned before, the time to achieve the optimal solution is the object of assessment. For this, the first performance profiles displayed in Fig. 2

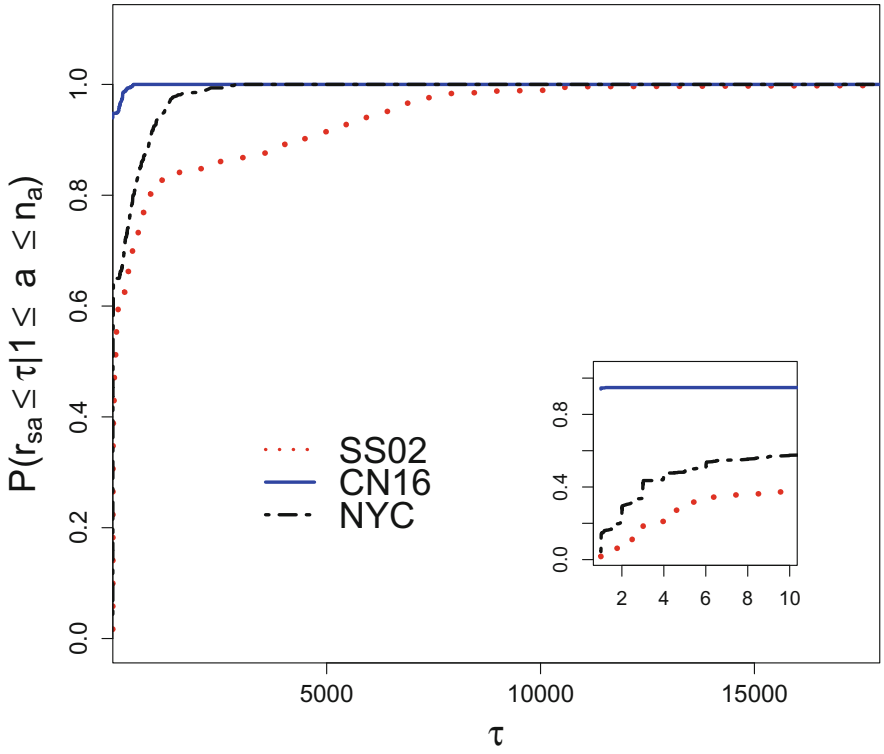


Fig. 2 This figure shows the performance profiles considering the two existing models in the literature, SS02 and CN16, and the formulation proposed in this chapter, NYC

assess only the small-sized instances (those with a number of items lesser than or equal to 50). In this case, it was possible to consider the time elapsed considering SS02, since it was comparable with NYC.

According to the results of the experiment, CN16 presents a clear advantage in performance in comparison to the other formulations. NYC achieves better results than SS02 even though their performance was very close. However, the difference between the performance profiles of NYC and SS02 becomes more evident with the increase in the number of items.

Figure 3 plots the performance profiles considering the larger instances (with a number of items larger than or equal to 100). This evaluation only takes into consideration the results achieved using models CN16 and NYC since to run instances modeled as SS02, it was very time-consuming.

Although the performance of NYC was poorer than CN16, it is noteworthy that their difference in performance fell in relation to the smaller instances. The value of τ for NYC to have at least half of the instances with results comparable to CN16 is 6. On the other hand, for larger instances it was approximately 2.

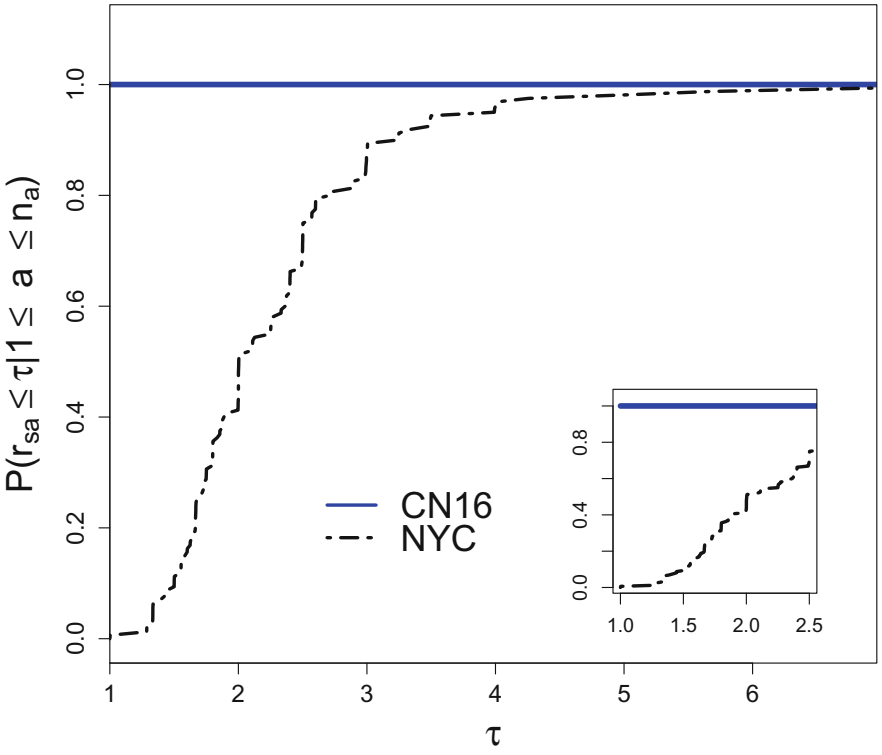


Fig. 3 This figure shows the performance profiles considering the existing model in the literature CN16 and the formulation proposed in this chapter, NYC

Extensions for the Capacitated Problem

In [6], the authors address the Multiplant Capacitated Lot Sizing Problem with Process Flexibility and Transportation Costs (MPCLSP-PFT), which extends the model (1)–(5). Specifically, they examine a lot sizing problem incorporating limited process flexibility, where each plant is capable of producing only certain types of items. Furthermore, the model includes a network of plants and clients, taking into account the transportation costs from each plant and each client. Below, one of the mathematical models proposed by Carvalho et al. [6] is presented, which is based on the classical formulation (1)–(5).

Extended Classical Formulation for the MPCLSP-PFT

Initially, it is important to consider additional parameters and decision variables based on the classical formulation of the multiplant lot sizing problem as proposed by [16].

Additional Parameters

c:	number of clients
d_{itk}:	demand of client k for item i in period t
sd_{itu}:	sum of the demands of item i, from period t to period u
b_{ij}:	processing time of item i at plant j
f_{ij}:	setup time for producing item i at plant j
bc_{itk}:	unit backlog cost of item i for client k in period t
tc_{ijk}:	unit transportation cost of item i from plant j to client k
fc_{ij}:	flexibility investment cost for item i at plant j
$Fmax$:	maximum budget to invest in flexibility

Additional Decision Variables

tr_{ijkt}:	amount of item i to be transported from plant j to client k in period t
B_{itk}:	amount of backlog for item i and client k in period t
z_{ij}:	binary variable that assumes value 1, if plant j is configured to produce item i, and 0, otherwise

The capacitated lot sizing problem with multiple plants, process flexibility, and transportation costs (MPCLSP-PFT) is modeled as follows:

$$(M1): \quad \min \sum_{i=1}^{n}\sum_{j=1}^{m}\sum_{t=1}^{p}\left[\left(cs_{ij}y_{ijt}+c_{ij}x_{ijt}+h_{ij}I_{ijt}\right)+\sum_{k=1}^{c}\left(tc_{ijk}tr_{ijkt}\right)\right]$$
$$+\sum_{i=1}^{n}\sum_{t=1}^{p}\sum_{k=1}^{c}\left(bc_{itk}b_{itk}\right) \tag{23}$$

subject to

$$I_{ijt} = x_{ijt} + I_{ij,t-1} - \sum_{k=1}^{c} tr_{ijkt} \qquad \forall\ (i, j, t) \tag{24}$$

$$d_{itk} = \sum_{j=1}^{m} tr_{ijkt} + B_{itk} - B_{i,t-1,k} \qquad \forall\ (i, t, k) \tag{25}$$

$$x_{ijt} \leq \min\{sd_{i1p}, (C_{jt} - f_{ij})/b_{ij}\} y_{ijt} \qquad \forall\ (i, j, t) \tag{26}$$

$$\sum_{i=1}^{n} \left(b_{ij} x_{ijt} + f_{ij} y_{ijt}\right) \leq C_{jt} \qquad \forall\ \ (j, t) \tag{27}$$

$$y_{ijt} \leq z_{ij} \qquad \forall\ (i, j, t) \tag{28}$$

$$\sum_{i=1}^{n} \sum_{j=1}^{m} f c_{ij} z_{ij} \leq Fmax \tag{29}$$

$$y_{ijt} \in \{0, 1\},\ z_{ij} \in \{0, 1\},\ x_{ijt} \geq 0,\ tr_{ijkt} \geq 0 \qquad \forall\ (i, j, t, k) \tag{30}$$

$$I_{ijt} \geq 0,\ I_{ij0} = 0,\ I_{ijp} = 0,\ B_{itk} \geq 0,\ B_{i0k} = 0 \qquad \forall\ (i, j, t, k) \tag{31}$$

The objective function (23) minimizes the total of setup, production, inventory, backlog, and transportation costs. Constraints (24) ensure flow balance at the plants, while constraints (25) maintain flow balance at the client locations. At the client sites, early fulfillment is not allowed, meaning no inventory is permitted; however, backlogging is allowed when demand cannot be met on time. Constraints (26) require that a plant setup operation is performed in order to produce a specific item. Capacity constraints (27) take into account production and setup at each plant for each period. Constraints (28) ensure that a setup cannot be executed if the plant is not configured to produce a specific item in a specific period. Constraint (29) imposes a limit on the budget invested in flexibility. Finally, constraints (30) and (31) define the domains of the decision variables. In addition to the formulation (23)–(31), the authors in [6] proposed a reformulation based on the facility location problem, along with an additional formulation that combines elements of both the original and the facility location-based formulations.

Facility Location Reformulation for the MPCLSP-PFT

In Krarup and Bilde [12], the authors propose a facility location reformulation for the classical version of lot sizing problem. In the present chapter, such reformulation for the MPCLSP-PFT is adapted.

Additional Parameters

$Cost_{ijtuk}$ represents the unit cost for production, inventory, and transportation of item i, produced at plant j during period t, to meet the demand of client k in period u, and is calculated as follows: $Cost_{ijtuk} = c_{ij} + tc_{ijk} + (u-t)h_{ij}$, for all $(i, j, t, u \geq t, k)$.

$Costb_{ijtuk}$ represents the unit cost for production and transportation of item i, produced as backlog at plant j in period t, to fulfill the demand of client k in period u, and is determined as follows: $Costb_{ijtuk} = c_{ij} + tc_{ijk} + \sum_{l=u}^{t-1} bc_{ilk}$, for all $(i, j, t, u < t, k)$.

Additional Decision Variables

x'_{ijtuk}: amount of item i to be produced at plant j in period t to satisfy the demand of client k in period u

B'_{ipuk}: amount of item i backlogged at the end of the planning horizon (i.e., period p), necessary to fulfill the unmet demand of client k in period u

The facility location reformulation for the MPCLSP-PFT problem is modeled as follows:

$$(M2): \quad \min \quad \sum_{i=1}^{n}\sum_{j=1}^{m}\sum_{t=1}^{p}(cs_{ij}y_{ijt}) + \sum_{i=1}^{n}\sum_{u=1}^{p}\sum_{k=1}^{c}\left[\left(\sum_{t=u}^{p} bc_{itk}\right)B'_{ipuk}\right] + \sum_{i=1}^{n}\sum_{j=1}^{m}\sum_{t=1}^{p}\sum_{k=1}^{c}\left[\left(\sum_{u=t}^{p}(Cost_{ijtuk}x'_{ijtuk})\right) + \left(\sum_{u=1}^{t-1}(Costb_{ijtuk}x'_{ijtuk})\right)\right] \tag{32}$$

subject to

$$\sum_{j=1}^{m}\sum_{t=1}^{p} x'_{ijtuk} + B'_{ipuk} = d_{iuk} \qquad \forall\ (i, u, k) \tag{33}$$

$$x'_{ijtuk} \leq \min\{d_{iuk}, (C_{jt} - f_{ij})/b_{ij}\}y_{ijt} \qquad \forall\ (i, j, t, u, k) \tag{34}$$

$$\sum_{i=1}^{n}\left(\sum_{u=1}^{p}\sum_{k=1}^{c} b_{ij}x'_{ijtuk} + f_{ij}y_{ijt}\right) \leq Cap_{jt} \qquad \forall\ (j, t) \tag{35}$$

$$y_{ijt} \leq z_{ij} \qquad \forall\ (i, j, t) \tag{36}$$

$$\sum_{i=1}^{n}\sum_{j=1}^{m} fc_{ij}z_{ij} \leq Fmax \tag{37}$$

$$y_{ijt} \in \{0, 1\},\ z_{ij} \in \{0, 1\} \qquad \forall\ (i, j, t) \tag{38}$$

$$x'_{ijtuk} \geq 0,\ B'_{ipuk} \geq 0 \qquad \forall\ (i, j, t, u, k) \tag{39}$$

Objective function (32) minimizes the total costs, including setup, production, inventory, backlog, and transportation. Constraints (33) model the flow balance at both the plants and the clients, where no inventory is permitted, but backlogging is allowed. In these constraints, the additional variable B'_{ipuk} is used to account for the possibility that part of the demand may remain unmet by any production at the end of the planning horizon in cases where backlogging occurs. Constraints (34) ensure that if an item is produced at a specific plant, a setup must occur. Constraints (35) limit the available capacity for production and setup in each plant and period. For each plant j, item i, and period t, constraints (36) enforce that a setup cannot be performed unless the plant j is configured to produce the specific item i. Constraints (37) restrict the total budget allocated to flexibility investments. Lastly, the domains of the decision variables are defined by constraints (38) and (39).

In [6] the authors analyzed the lower bounds obtained with both the formulations $M1$ and $M2$ to determine the best approach for solving the problem. Their analysis revealed that for problems with tight capacity, $M1$ provides better bounds, while $M2$ achieves better bounds for problems with normal capacity. Based on these findings, they proposed a third formulation, ($M3$) to the MPCLSP-PFT, which consistently produces lower bounds that are equal to or better than those obtained with formulations $M1$ and $M2$. The formulation $M3$ consists in minimizing the objective function value $\zeta \in \mathbb{R}$, subject to constraints (24)–(31), (33)–(35), and (39)–(42).

$$\zeta \geq \sum_{i=1}^{n}\sum_{j=1}^{m}\sum_{t=1}^{p}\left[\left(cs_{ij}y_{ijt} + c_{ij}x_{ijt} + h_i s_{ijt}\right) + \sum_{k=1}^{c}\left(tc_{ijk}tr_{ijkt}\right)\right] + \sum_{i=1}^{n}\sum_{t=1}^{p}\sum_{k=1}^{c}(bc_{itk}B_{itk}) \tag{40}$$

$$\zeta \geq \sum_{i=1}^{n}\sum_{j=1}^{m}\sum_{t=1}^{p}(cs_{ij}y_{ijt}) + \sum_{i=1}^{n}\sum_{u=1}^{p}\sum_{k=1}^{c}(bc_{ipk}B'_{ipuk}) + \sum_{i=1}^{n}\sum_{j=1}^{m}\sum_{t=1}^{p}\sum_{k=1}^{c}\left[\left(\sum_{u=t}^{p}(\text{Cost}_{ijtuk}x'_{ijtuk})\right) + \left(\sum_{u=1}^{t-1}(\text{Cost}b_{ijtuk}x'_{ijtuk})\right)\right] \tag{41}$$

$$\sum_{u=1}^{p}\sum_{k=1}^{c}x'_{ijtuk} = x_{ijt} \qquad \forall\ (i, j, t) \tag{42}$$

Constraints (40) and (41) impose a lower bound on the value of the objective function, ensuring that ζ is at least as large as the objective function values of $M1$ and $M2$, respectively. For each item, plant, and period tuple, constraints (42) ensure consistency between the two types of production decision variables.

Computational Experiments for the MPCLSP-PFT

This section compares the proposed mathematical models using a set of instances provided by Carvalho et al. [6]. The experiments were conducted on a computer equipped with two Intel Core i5-9300H processors running at 2.4 GHz and 16 GB of DDR3 RAM. Each instance was solved using a single thread with a time limit of 3600 seconds, utilizing the commercial solver CPLEX v. 12.10.

The instances used involve four and five plants (f); five and eight items (n); five periods (p); and four and six clients (c). The parameter $Fmax$, appearing in constraints (29), represents the total budget available for investment in flexibility. It is assumed that $fc_{ij} = 1$, meaning the total budget corresponds to the maximum number of production-plant links. Observe that a flexibility level of 100% means that all plants can produce all items, that is, there is total flexibility. In computational tests, the value considered for the parameter $Fmax$ is 75% and will be represented by $Flex3$ in the next tables, meaning it is considered that 75% of the total possible links ($n \times m$) can be used.

The available capacity levels per period, as defined in constraints (27), vary based on a factor μ, which is multiplied by the total average capacity $ACap$, which is calculated as $\left(\left(\sum_{i\in\mathscr{I}}\sum_{k\in\mathscr{K}}\sum_{t\in\mathscr{T}} d_{ikt}b_i + f_i\right)/(m \times p)\right)$. Values $\mu = 0.6, 0.9, 1.3$ are considered to encompass a broad range of problems, ensuring that the solutions exhibit varying levels of backlog.

To generate the transportation costs (tc_{ijk}), a pair of coordinates, bounded between 0 and 1, is uniformly assigned to each plant and client within a square on the Cartesian plane. The costs are then calculated using the Euclidean distance between each plant and client. Specifically, for each item i, the transportation cost tc_{ijk} is obtained by multiplying the Euclidean distance between plant j and client k by a scalar value $\beta = 0.3$.

Table 1 outlines the general parameters defined for the MPCLSP-PFT, which are based on a subset of instances proposed in [22].

Table 1 Parameter definitions designed for the MPCLSP-PFT. Each row presents a set or a value used to define their parameters where U[a, b] represents the uniform distribution

Parameters	All classes of instances
d_{itk}	U[0, 180]
f_{ij}	U[10, 50]
b_{ij}	1
cs_{ij}	U[400, 1000]
h_{ij}	U[1, 5]
bc_{jtk}	300
bc_{jpk}	600
c_{ij}	0
fc_{ij}	1

Results of the Experiments

By comparing the upper bounds and the gaps obtained by each model, it is possible to observe that the values are quite similar. Specifically, the results are:

- Model $M1$: upper bound = 1,285,413 and gap = 0.2%
- Model $M2$: upper bound = 1,285,667 and gap = 0.5%
- Model $M3$: upper bound = 1,285,720 and gap = 0.2%

Figure 4 illustrates the performance profile of each model in terms of computational time. The results of the experiment indicate that model $M2$ demonstrates a clear advantage in performance compared to model $M3$. Moreover, $M2$ outperforms $M1$, although their performance metrics are closely aligned.

The performance profile in Fig. 5 requires a different analysis, as higher lower bound values indicate better model performance. From Fig. 5, it is evident that model $M3$ achieves the highest LP lower bounds, demonstrating a clear performance advantage over model $M1$. Additionally, $M3$ outperforms $M2$, although their performance metrics are closely aligned.

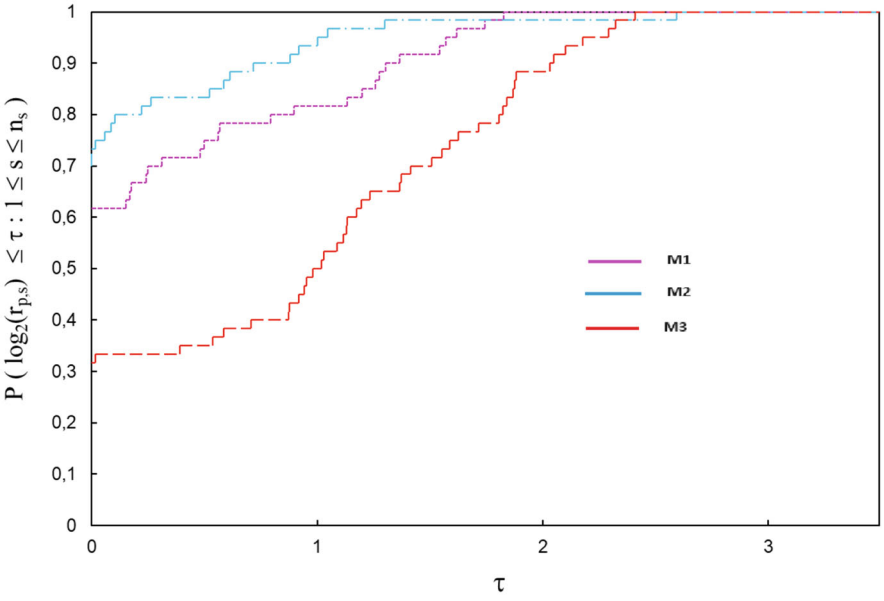


Fig. 4 This figure shows the performance profiles considering the three existing models analyzing their computational time

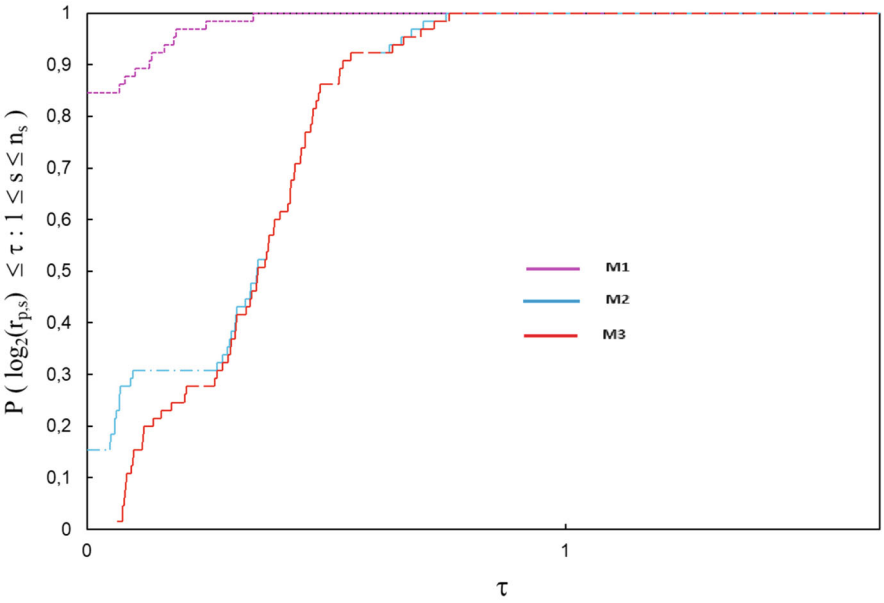


Fig. 5 This figure shows the performance profiles considering the three existing models analyzing their Linear relaxation lower bounds

Conclusions

Lot sizing plays a fundamental role in production planning. Considering the planning horizon phased into periods, this problem consists in making decisions with regard to plant, period, item, and amount of items to produce to meet all demands without delay. In multiplant capacitated lot sizing problem, it is possible to schedule items on multiple machines/plants enabled by transferring items to plants where they are demanded. This is the subject of this study about which is presented a brief literature review, primarily focusing on the existing solution methods and approaches to its solution.

A few efficient solution methods were specially designed to tackle the multiplant capacitated lot sizing problem (MPCLSP). The hard nature of this problem, for which not even the uncapacitated variant has a polynomial algorithm to find optimal solutions, may be the reason behind the low interest in developing frameworks. Nevertheless, two Lagrangian heuristics recently proposed achieved good results mainly on large-scale instances and instances with high setup costs. For smaller instances and those with low setup costs, CPLEX significantly outperformed them.

Even though being NP-hard, few formulations were specially designed for the multiplant uncapacitated lot sizing problem (MPULSP). The relevance in studying the MPULSP, besides approaching production lines that do not have capacity constraints, is to enable a study of strategies that relax capacity constraints to browse the search space for feasible solutions for the MPCLSP. This chapter introduces a network flow-based formulation for the MPULSP and evaluates a recently proposed mathematical formulation for the MPCLSP by relaxing the capacity constraints to contrast their performance in solving instances according to solver CPLEX. Moreover, the firstly proposed formulation for the MPULSP was investigated in the computational experiments. For such, experiments with instances found in the literature were carried out by evaluating the time to solution, assessed with Dolan-Moré performance profiles.

Finally, the updated version of this chapter, based on [6], focuses on the multiplant capacitated lot sizing problem (MPCLSP) and its extensions. Unlike the standard problem with parallel resources, these extensions introduce a limited amount of flexibility, allowing each plant to produce only certain types of items. Additionally, the model considers a network of clients dispersed around different plants. Three mathematical formulations are presented and evaluated through computational results.

Cross-References

▸ GRASP

Acknowledgments The authors gratefully acknowledge the support of the following Brazilian funding agencies: the National Council for Scientific and Technological Development—CNPq (Conselho Nacional de Desenvolvimento Científico e Tecnológico) (grant numbers 304144/2019-3, 405702/2021-3, 302998/2022-5, 402240/2023-5, 403735/2021-1, 402592/2024-7, 309385/2021-0), the São Paulo Research Foundation—FAPESP (Fundação de Amparo à Pesquisa do Estado de São Paulo) (grant numbers 2013/07375-0, 2022/05803-3 and 2024/01409-4), and the National Council for the Improvement of Higher Education—CAPES (Coordenação de Aperfeiçoamento de Pessoal de Nível Superior).

Competing Interest Declaration The author(s) has no competing interests to declare that are relevant to the content of this manuscript.

References

1. Bitran G, Yanasse H (1982) Computational complexity of the capacitated lot size problem. Manag Sci 28:1174–1186
2. Brahimi N, Dauzere-Peres S, Najid NM, Nordli A (2006) Single item lot sizing problems. Eur J Oper Res 168:1–16
3. Buschkühl L, Sahling F, Helber S, Tempelmeier H (2010) Dynamic capacitated lot-sizing problems: a classification and review of solution approaches. Or Spectrum 32(2):231–261

4. Carvalho DM, Nascimento MCV (2016) Lagrangian heuristics for the capacitated multi-plant lot sizing problem with multiple periods and items. Comput Oper Res 71:137–148
5. Carvalho DM, Nascimento MCV (2018) A kernel search to the multi-plant capacitated lot sizing problem with setup carry-over. Comput Oper Res 100:43–53. https://doi.org/10.1016/j.cor.2018.07.008
6. Carvalho DM, Jans R, de Araujo SA, Fiorotto DJ (2024) A heuristic approach for the integrated production–transportation problem with process flexibility. Comput Oper Res 163:106502. https://doi.org/10.1016/j.cor.2023.106502. https://www.sciencedirect.com/science/article/pii/S0305054823003660
7. Dolan ED, Moré JJ (2002) Benchmarking optimization software with performance profiles. Math Program Ser A 91:201–213
8. Drexl A, Kimms A (1997) Lot sizing and scheduling - survey and extensions. Eur J Oper Res 99:221–235
9. Eppen GD, Martin RK (1987) Solving multi-item capacitated lot-sizing problems using variable redefinition. Oper Res 35(6):832–848
10. Jans R, Degraeve Z (2008) Modeling industrial lot sizing problems: a review. Int J Prod Res 46:1619–1943
11. Karimi B, Ghomi SMTF, Wilson JM (2003) The capacitated lot sizing problem: a review of models and algorithms. OMEGA 31:365–378
12. Krarup J, Bilde O (1977) Plant location, set covering and Economic lot size: an O(mn)-algorithm for structured problems. In: Collatz L, Meinardus G, Wetterling W (eds) Numerische Methoden bei Optimierungsaufgaben Band 3. International Series of Numerical Mathematics, vol 36. Springer, Basel, pp 155–180
13. Maes J, McClain JO, Wassenhove LNV (1991) Multilevel capacitated lot sizing complexity and lp-based heuristics. Eur J Oper Res 53:131–148
14. Nascimento MCV, Resende MCG, Toledo FMB (2010) GRASP with path-relinking for the multi-plant capacitated lot sizing problem. Eur J Oper Res 200:747–754
15. Quadt D, Kuhn H (2008) Capacitated lot-sizing with extensions: a review. 4OR 6(1):61–83.https://doi.org/10.1007/s10288-007-0057-1
16. Sambasivan M (1994) Uncapacitated and capacitated lot sizing for multi-plant, multi-item, multi-period problems with inter-plant transfer. PhD thesis, University of Alabama, Tuscaloosa
17. Sambasivan M, Schimidt CP (2002) A heuristic procedure for solving multi-plant, multi-item, multi-period capacited lot-sizing problems. Asia Pacific J Oper Res 19:87–105
18. Sambasivan M, Yahya S (2005) A Lagrangean-based heuristic for multi-plant, multi-item, multi-period capacitated lot-sizing problems with inter-plant transfers. Comput Oper Res 32:537–555
19. Silva DH (2013) Métodos híbridos para o problem de dimensionamento de lotes com múltiplas plantas. Master's thesis, Universidade de São Paulo - São Carlos
20. Silva DH, Toledo FMB (2012) Dimensionamento de lotes com múltiplas plantas: comparação entre dois modelos. In: Simpósio Brasileiro de Pesquisa Operacional, pp 1638–1646
21. Toledo FMB, Armentano VA (2006) A Lagrangean-based heuristic for the capacitated lot-sizing problem in parallel machines. Eur J Oper Res 175:1070–1083
22. Trigeiro WW, Thomas LJ, McClain JO (1989) Capacitated lot sizing with setup times. Manag Sci 35:353–366
23. Wagner HM, Whitin TM (1958) Dynamic version of the economic lot size model. Manag Sci 5:89–96

The OGA Company: The Role of Heuristics in Logistic Consulting Services

59

Arturo López-Damas, Adolfo García, Reyes Maíllo, Alfredo G. Hernández-Díaz, Juan Carlos Rubio, and Rafael Martí

Contents

Introduction . . . 1772
Case Study . . . 1773
Routing . . . 1774
The Hexaly Template . . . 1776
A Practical Case in the Beverage Industry . . . 1778
Computational Experiments . . . 1781
Comparison with Existing Solutions . . . 1786
Conclusions . . . 1790
Cross-References . . . 1791
References . . . 1791

Abstract

OGA is a Spanish consulting company based in Seville that applies optimization to help companies improve their business processes. Specializing in creating business value through the application of critical process optimization techniques, OGA mainly employs metaheuristic techniques to deliver high-quality solutions to practical problems faced by companies. From scheduling

A. López-Damas · A. García · R. Maíllo · J. C. Rubio
OGA, Sevilla, Spain
e-mail: arturo.lopezdamas@oga.ai; adolfo.garcia@oga.ai; reyes.maillo@oga.ai; juancarlos.rubio@oga.ai

A. G. Hernández-Díaz (✉)
Universidad Pablo de Olavide, Sevilla, Spain
e-mail: alfredo.garcia@oga.ai; agarher@upo.es

R. Martí
Operations Research and Statistics Department, School of Mathematics, University of Valencia, Valencia, Spain
e-mail: rafael.marti@uv.es

© Springer Nature Switzerland AG 2025

R. Martí et al. (eds.), *Handbook of Heuristics*,
https://doi.org/10.1007/978-3-032-00385-0_72

to routing, the company approaches logistic problems in the supply chain to improve the operations of their final customer. In this chapter, we show the complexities of real-world optimization problems in an example of the beverage industry and how OGA efficiently solves them by applying metaheuristic technologies in a financially sustainable manner.

Keywords

OGA · Metaheuristics · Logistic sector · Business processes · Real-world optimization problems

Introduction

OGA is a consulting company specialized in the development of algorithms, models, and integral solutions for the optimization of critical business processes. It operates in various logistics sectors of the supply chain, where it tackles complex operational planning, fleet optimization, optimal locations of logistics centers or nodes, and integration of transport or distribution routes.

Management and business optimization consulting services assess the strengths and weaknesses of a company's supply chain, crafting optimal strategies and implementation plans from management, business processes, and IT viewpoints tailored to clients within the manufacturing and distribution sectors.

One crucial aspect of the role as consultants involves bridging the gap between clients and the latest research advancements from universities and research institutes in both Operations Research and Artificial Intelligence. Through effective communication with academia and industry alike, OGA delivers tailored solutions to meet the clients' real-world business requirements. As is well known in the sector, one of the main differences between the research developed in academia and published in journals, and the development and research performed in consultant firms, is the collaboration with clients who are looking for suitable solutions, even when the problem is initially vague, which may involve assisting in problem definition itself.

One of the specific characteristics of implementing heuristic algorithms in the business environment is the need to make the process economically efficient. Consulting firms must balance meeting client requirements with implementing solutions in short development times. Ideally speaking, consulting firms seek generic developments that serve different users, perhaps with minor customizations, to adapt predeveloped software to create new products. However, what often happens is that the end user has many requirements that hinder, if not prevent, the reusability of the code. In this chapter, we describe the in-between solution applied in OGA to reach a trade-off between customization and efficiency. In particular, in some projects, the company develops applications in which part of the solution process is based on an in-house implementation of heuristics, and the other part is

run with a commercial library also based on heuristics, which in this case is Hexaly (formerly known as Local Solver).

As advertised on their website, Hexaly is a well-known optimization solver for Routing, Scheduling, and Packing, among other models (www.hexaly.com). Its user community is growing, and there are currently around 1,000 developers who leveraged the Hexaly platform to build around 400 optimization apps for 20,000 users. It provides preestablished templates that the programmer may adapt to create a customized model for the specific problem at hand.

In this chapter, we describe a commercial application of OGA to customers in supply chain logistics, based on this paradigm of combining their own heuristic algorithms with Hexaly templates. In particular, section "Case Study" describes a case study based on a routing model. In section "Routing," we describe the vehicle routing model, and in section "The Hexaly Template" its implementation in Hexaly. Section "A Practical Case in the Beverage Industry" is devoted to the practical problem solved by OGA for an international company based in Spain. The chapter finishes with some computational experiments, shown in section "Computational Experiments," to show in practical terms the quality of the solutions obtained with the built application, and the associated conclusions.

Case Study

Logistics-focused companies dedicate their efforts to various aspects, including the transportation of goods and ancillary services such as warehouse administration and inventory oversight. Although transportation is the main focus within this sector, effective stock control and effective product storage management are equally vital. Consequently, decision support systems play a key role in facilitating this management process.

Smart mobility refers to efficient transportation of people or goods. The growth of cities and the importance of mobility within the urban area have led to research on ways to minimize traffic, its impact on the environment, and increase the efficiency of travel. In this section, we describe an application developed by the OGA team in the context of the supply chain. Specifically, we consider a routing problem since it represents an important family of combinatorial optimization problems and therefore illustrates representative problems in consulting companies.

In the logistics consulting industry, clients have varying needs when it comes to the speed of obtaining solutions. Some clients operate with flexible timelines and are open to receiving solutions within a broader time frame. They may prioritize thorough analysis and strategic planning over immediate responses. However, clients face a significant challenge that requires fast turnaround times. For these clients, receiving solutions in seconds or in a few minutes is critical due to the urgency of their operational demands or market pressures.

This disparity in client needs presents a considerable challenge for the logistics industry and specifically for consulting firms like OGA. On the one hand, the ability to deliver solutions quickly requires robust systems, streamlined processes, and advanced technology to ensure that responses are not only fast but also accurate and effective. However, accommodating clients with more flexible timelines still requires a high level of service and precision, although the pressure to act immediately is less intense.

Routing

The Capacitated Vehicle Routing Problem (CVRP) can be mathematically formulated as an integer optimization problem with binary variables from the following input elements in terms of graphs:

- $G = (V, E)$: a complete undirected graph where V represents the set of nodes (including the depot and customer locations) and E represents the set of edges between nodes
- c_{ij}: the cost (distance, time, or monetary value) of traveling from node i to node j
- d_i: the demand of customer i
- Q: the capacity of each vehicle
- K: the number of available vehicles

The CVRP aims to minimize the total distance traveled. It also ensures that each customer's demand is met and that vehicle capacity is not exceeded. The mathematical formulation of the CVRP can be given as follows by introducing binary decision variables x_{ij} indicating whether a vehicle travels from node i to node j, i.e., $x_{ij} = 1$ if vehicle travels from i to j, and $x_{ij} = 0$ otherwise.

$$\text{minimize} \quad \sum_{i \in V} \sum_{j \in V, j \neq i} c_{ij} \cdot x_{ij}$$

subject to:

1. Each customer is visited exactly once:

$$\sum_{j \in V, j \neq i} x_{ij} = 1, \quad \forall i \in V \setminus \{0\}.$$

2. Each vehicle must leave and return to the depot:

$$\sum_{j \in V} x_{0j} = K$$

$$\sum_{j \in V} x_{j0} = K.$$

3. Capacity constraint at each customer node:

$$\sum_{j \in V, j \neq i} d_j \cdot x_{ji} \leq Q, \quad \forall i \in V \setminus \{0\}.$$

4. Flow conservation at each node (except the depot):

$$\sum_{j \in V, j \neq i} x_{ij} - \sum_{j \in V, j \neq i} x_{ji} = 0, \quad \forall i \in V \setminus \{0\}.$$

5. Binary restrictions on decision variables:

$$x_{ij} \in \{0, 1\}, \quad \forall i, j \in V, i \neq j.$$

The objective function minimizes the total distance traveled by all vehicles, subject to constraints that ensure that each customer is visited exactly once, vehicles leave and return to the depot, capacity constraints are satisfied, and flow conservation is maintained at each node.

CVRP is known to be difficult to solve (NP hard) [3]. Over the years, routing has evolved into a relevant research area, which might include real-life characteristics such as multiple criteria, soft constraints, or stochastic travel times, just to name a few. Additionally, when we move from the academic to the business context, optimization problems become even more complex to solve. In particular, the first issue we face is the instance size, which in many cases makes the model impractical. The second problem is usually the existence of numerous additional constraints, due to the specific characteristics of the client.

Generally speaking, we can identify five categories to classify routing problems: characteristics of the network, vehicles, facility, demand, and objective:

- *Network*: *Directed*, *undirected*, or *mixed*.
- *Vehicles* are typically identical, but we can also find a *heterogeneous fleet*.
- *Multiple or mobile depots* are characteristics related to *facilities*, in which we consider several depots or intermediate facilities.
- The *demand* can be deterministic or stochastic, reflecting the latter practical situations in waste collection or reverse logistics with uncertainty in the data.
- *Objective*: Single or *multiple objectives*, including: service and labor cost, overtime and penalty costs, duration and duration of routes, etc.

One of the most popular extensions of the VRP involves a diverse fleet comprising various types of trucks with differing capacities and costs, reflecting the situation of many transportation companies. This variant, called Heterogeneous or simply Mixed Fleet VRP, has received less attention compared to the standard VRP due to its increased complexity. Yet another significant variation is the VRP that incorporates Time Windows (VRPTW), where customers can be attended to only within designated time frames, necessitating the scheduling of vehicle trips.

Vehicles can also be *multicompartment*, having the required edges a demand for different commodities that must be kept segregated during the transportation action.

In recent years, increased focus has been directed toward more intricate variants of the VRP, often referred to as "rich" VRPs, addressing real-world distribution challenges. These variants commonly feature multiple depots, the need for multiple vehicle trips, diverse vehicle types, and additional operational complexities such as loading constraints.

In spite of the practical need of solving very large VRP instances due to the operations of large companies, algorithm development in the academic context has concentrated on tackling routing problems involving only a few hundred customers at most. The rationale behind this focus remains ambiguous. One possible explanation could be attributed to the prevalent size of the commonly employed benchmark instances. The standard benchmark set in VRP was introduced by Golden et al. [4], with instances featuring up to 483 customers, while more contemporary benchmarks, like the one established by Uchoa et al. [8], include instances with a maximum of 1000 customers.

Special mention deserves FILO, probably the best heuristic published so far for the CVRP. It is based on the Iterated Local Search (ILS) paradigm and incorporates various existing acceleration techniques along with novel strategies for localized and controlled optimization tailored to the current instance and solution. Using a Simulated Annealing neighbor acceptance criterion, FILO ensures continuous diversification, exploring different regions of the search space to avoid premature convergence. Accorsi and Vigo [1] have recently published a benchmark test in extensively studied instances to show that FILO is highly competitive with state-of-the-art algorithms in terms of computing time and solution quality. The paper also provides open-source guidelines, algorithm source code, and a library of reusable components to promote further research and reproducibility.

Figure 1 shows a partial representation of an OGA-solved instance for a client in the beverage business. In particular, it depicts part of the map of a city in Spain where the company has to provide service to 20,000 clients. This is a very large instance of a special case of the VRP, so we cannot directly apply existing solvers, and we have to design and implement a solving method to address its particularities through additional constraints.

The Hexaly Template

Hexaly optimizer provides solutions for the most common routing problems, namely the traveling salesman problem (TSP), the capacitated vehicle routing problem (CVRP), and the capacitated vehicle routing problem with time windows (CVRPTW). Templates for these models are available at www.hexaly.com with sample data and ready-to-use code in LSP, C++, Java, .NET, and Python. It reaches a 1.2% average gap after 60 seconds of running time in the CVRP instances tested [8].

The CVRP problem is modeled with a *list variable* per truck in the fleet, and the coverage of all customers is enforced by a partition constraint. The list operator

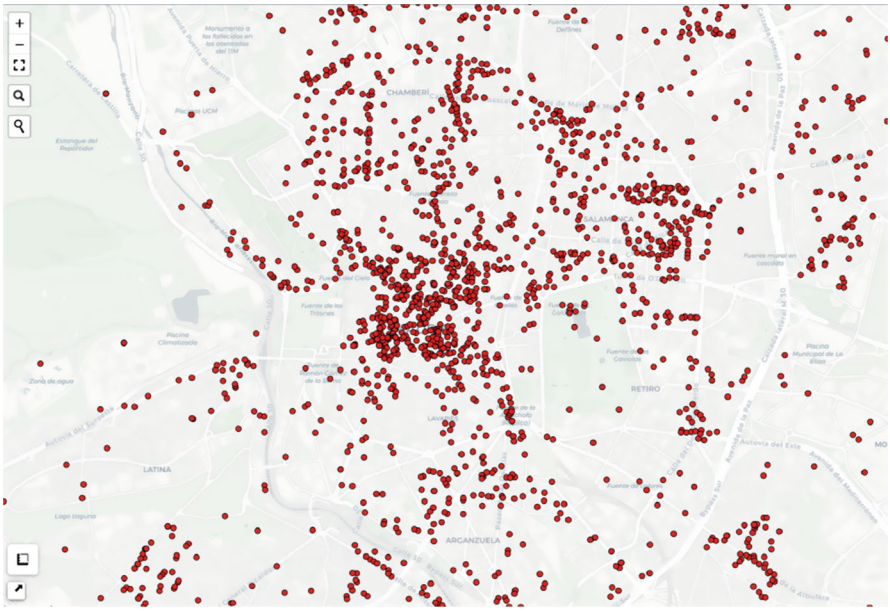


Fig. 1 Example of clients in a map

allows us to define a decision variable whose value is a collection of integers within $[0, n-1]$ where n is the operator's argument. The following template, provided by the company, illustrates how easy it is to create a model in Python to solve a standard CVRP. It shows that we only need to call the minimize function after including the constraints. Hexaly automatically performs the local search-based optimization and returns the best solution found. We refer the reader to the website of the company at www.hexaly.com for a detailed description of the following code.

```
function model() {
    // Sequence of customers visited by each truck
    customersSequences[k in 1..nbTrucks]
    <- list(nbCustomers);

    // All customers must be visited by the trucks
    constraint partition[k in 1..nbTrucks]
    (customersSequences[k]);

    for [k in 1..nbTrucks] {
        local sequence <- customersSequences[k];
        local c <- count(sequence);

        // Sum of demands along the route must not
        exceed the truck capacity
```

```
        constraint sum(sequence, j => demands[j])
        <= truckCapacity;

        // Distance traveled by the truck
        routeDistances[k] <- sum(1..c-1, i =>
        distanceMatrix[sequence[i - 1]][sequence[i]])
             + (c > 0 ? (distanceWarehouse
                [sequence[0]] + distanceWarehouse
                    [sequence[c - 1]]) : 0);
    }

    // Minimize the total distance traveled by all
    trucks
    minimize sum[k in 1..nbTrucks](routeDistances[k]);
}
```

As mentioned above, our solving methodology is based on a combination of in-house development and Hexaly post-optimization. We apply a sort of multistart method in which the randomized construction is performed with a specific algorithm, and the typical local search post-optimization is done with a customized Hexaly model from the basic code above in which we add all the customers' requirements.

A Practical Case in the Beverage Industry

One of the most difficult decision problems confronting a distribution manager in the soft-drink industry on a day-to-day basis concerns the routing, scheduling, and staffing of delivery vehicles. The Operations Research journal published in 1987 a practical paper devoted to this topic [3], in which the authors stated that given the large volume of soft-drink sales, the expenses associated with the distribution activities are enormous, and recommended, even in these early times, the use of computer software packages to optimize its cost. The paper describes the particularities of the routing problem in this context, such as determine driver territories or create compact routes.

In the context of logistics consulting services provided by companies like OGA, managing and maintaining distance and time matrices is a critical and often challenging aspect. These matrices serve as foundational tools for optimizing routes, managing supply chains, and making informed decisions. The generation of these matrices involves calculating the distances and travel times between various points in a network, which can be complex due to the need for accuracy and comprehensiveness.

For many consulting firms, the initial creation of these matrices can be a significant "pain point." It requires integrating data from various sources, including geographic information systems (GIS), traffic databases, and historical data to

ensure precision. This process often involves considerable effort and experience to develop an initial matrix that accurately reflects real-world conditions. Once established, maintaining these matrices becomes another challenge. Traffic patterns, road conditions, and other factors can change frequently, making it crucial to update the matrices regularly to retain their relevance and accuracy. Failure to do so can lead to outdated information that negatively affects decision-making and logistics efficiency.

In our particular problem, we apply a multistart scheme in which we call the Hexaly solver at certain stages, to improve specific aspects of the solution. The algorithm has the following six main steps:

1. **Preprocessing**. Determining the search strategy based on the overall load compared to the capacity of the truck. Computing the target values for the objectives to build a goal programming model based on the load balance, to obtain similar truck loads, compactness of the route shape, and route overlapping. Group orders and determining special routes with high priority.
2. **Construction**. Fleet allocation based on delivery zones and order according to priority. Calls to the Hexaly solver for each zone, providing a pool of orders and a pool of vehicles.
3. **Construction postprocessing**. Attempts to add orders left for the second tour to their nearest route. If it meets the constraints, it is added to the route and removed from the second tour.
4. **Improvement**. Cross-checking. The polygon representing each route is checked whether it intersects with other polygons to evaluate their overlapping. If so, they are sent together to the Hexaly solver for a post-optimization. We have empirically found that the company prefers nonoverlapping routes among other reasons because it reduces conflicts between drivers.
5. **Improvement postprocessing**. Attempts to add orders left for the second tour to their nearest route in the first tour after cross-checking. If it meets the constraints, it is added to the route and removed from the second tour.
6. **Solution completion**. Add the remaining elements to the second tour.

As shown in the algorithm above, Hexaly is called in both the construction and improvement to optimize different objective functions. We have empirically found that it is more efficient than developing code to optimize these criteria.

Figure 2 shows the complete flow diagram of our complete heuristic. It starts by determining whether the overall load is high or low compared to the truck capacity (Step 1), since two different strategies are applied to the trucks in each case. Similarly the complete preprocessing takes place in steps 1 to 4 of the flowchart (depicted in orange in the diagram). The following blocks in the flowchart correspond to the steps 2 to 6 in the algorithm, namely construction (2 blocks), construction postprocessing (1 block), improvement (crosscheck routes block), improvement postprocessing, and finally solution completion (second tour).

Note that the user interacts with the main front-end screen to set the different options of the software. Figure 3 shows it.

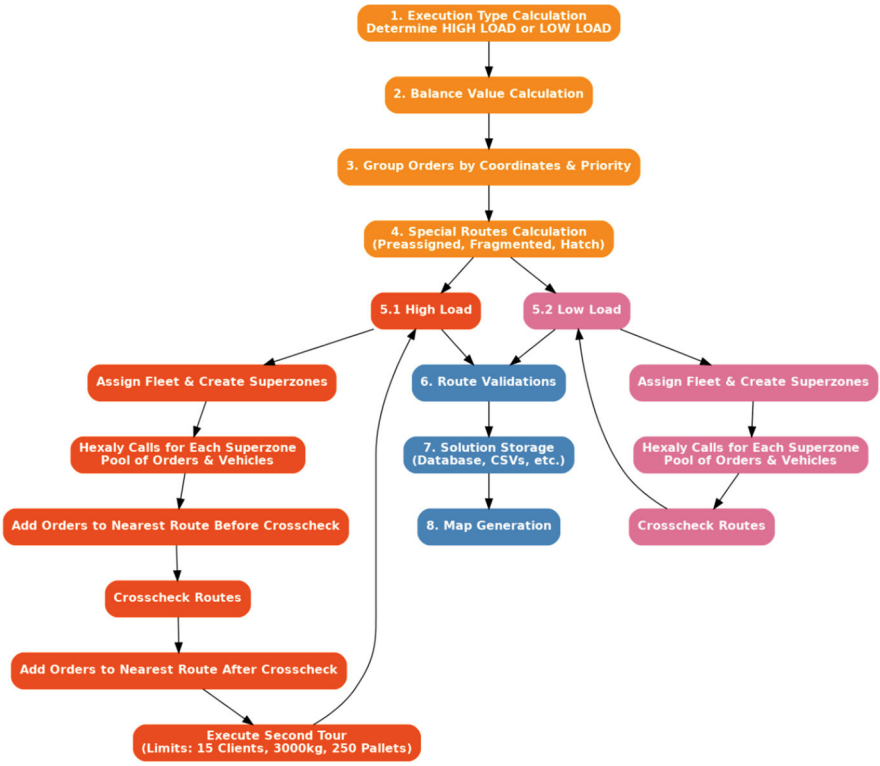


Fig. 2 Flowchart of the entire algorithm

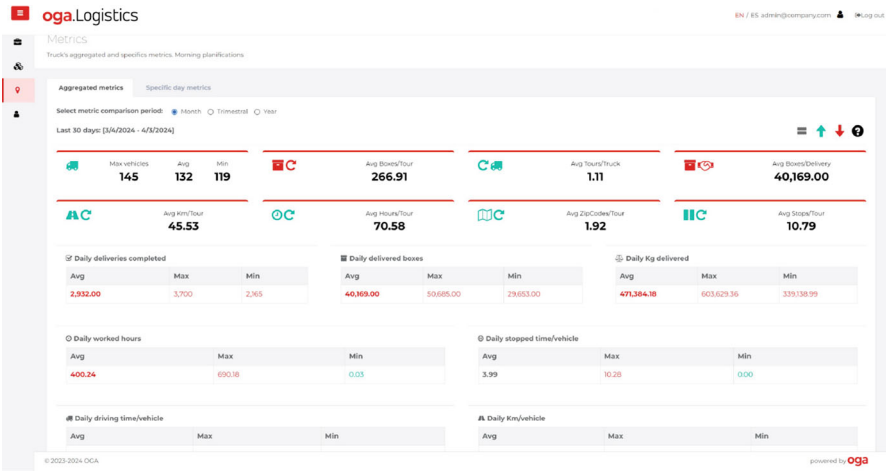


Fig. 3 Main screen of the OGA.Logistics application

Computational Experiments

In this section, we describe the solutions obtained by OGA in this problem. Note that due to confidentiality, we cannot disclose all the details of the instances or the solutions.

The customer has about 33,000 sales points, and they are visited from Monday to Friday in two shifts (an average of 6,600 points daily). Aggregated demand varies according to the day of the week and the time of year (between 300,000 kg and 1.5 million kg). The fleet used varies according to the day, but once known, they must all be used. They use approximately 105 to 140 trucks of 6200 kg and up to 15 vans of a capacity of 1500 kg. On low-load days, all trucks make only one trip, and the vans make two trips per shift. On high-load days, trucks can make up to two trips per shift (the minimization of second trips is added as an objective) and vans a maximum of three trips. Once the above data are known, the model is run until no further significant improvement is achieved and returns the high-quality solution obtained. This usually happens after about 2 to 3 minutes of execution. As a point of comparison, the customer previously required five people to plan routes manually for about 4 hours.

We now illustrate the performance of the algorithm with the example partially represented in Fig. 1. OGA has to solve this example periodically, and in each instance the number of clients usually ranges from 10,000 to 20,000 approximately. The challenge is that the fleet of vehicles remains the same in all the cases: 150 trucks.

A typical simplification in this type of problem is to group clients in delivery zones and restrict each truck to serve clients only within one zone. Figure 4 shows in different colors the delivery zones in our case study. It is clear that this simplification may lead to suboptimal solutions since in optimization terms we are considerably restricting the solution space. On the other hand, in technical terms, this leads to a simplified computation of the distances because within a delivery zone we may simply use the Euclidean distance as a proxy to the travel distance based on streets and traffic, which would be very inaccurate if a truck may serve clients in the entire map. Our constructive method is partially based on these zones, but it allows us to cross the boundary of the delivery zones in an effort to reach better solutions. Figure 5 shows five constructed routes in our example.

It must be noted that, to represent the solutions in a clear way, we avoid the inclusion of the first arc in the route from the depot as well as the last arc to the depot. Figure 6 shows in (a) what we represent (without these two arcs with the depot) and in (b) the complete routes that include them.

The compactness and overlap of the solution are probably the most difficult characteristics to achieve. This is related to the visual attractiveness of the routes [7], which, as mentioned above, is a standard criterion in practical routing problems. Figure 7 shows in (a) two routes with overlap areas that the company wants to avoid due to the agreement with the drivers. In (b), we show a good solution in terms of overlap. Furthermore, it should be noted that in beverage delivery in urban areas,

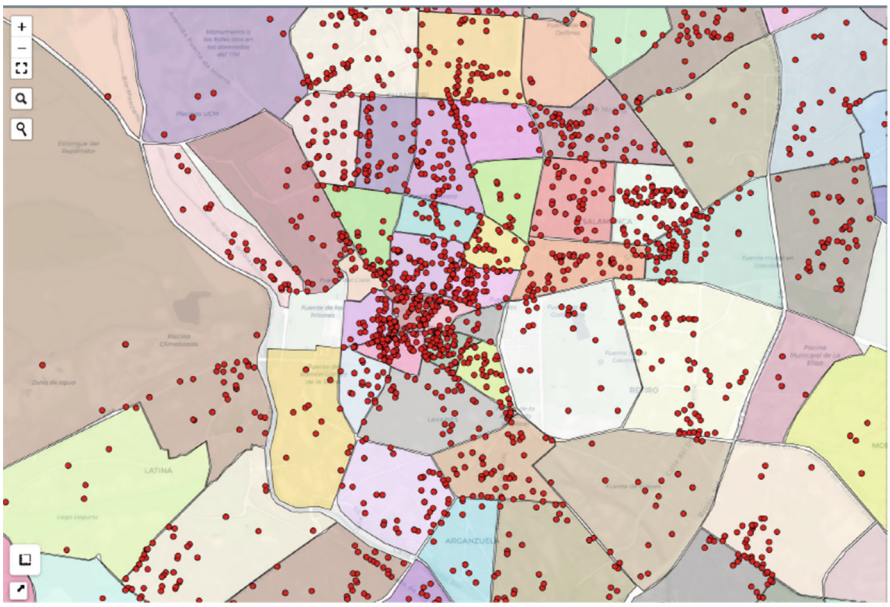


Fig. 4 Example of delivery zones

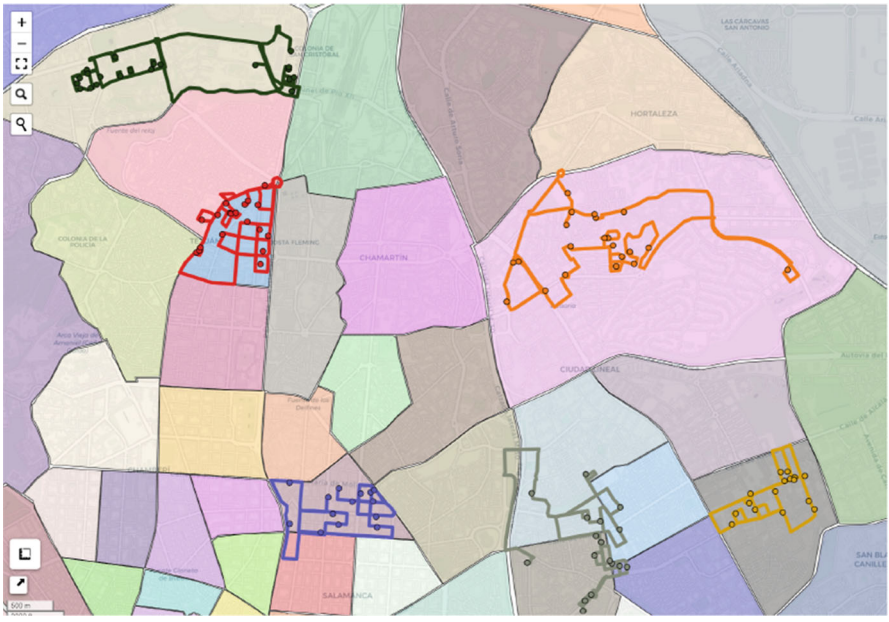


Fig. 5 Some routes partially based on delivery zones

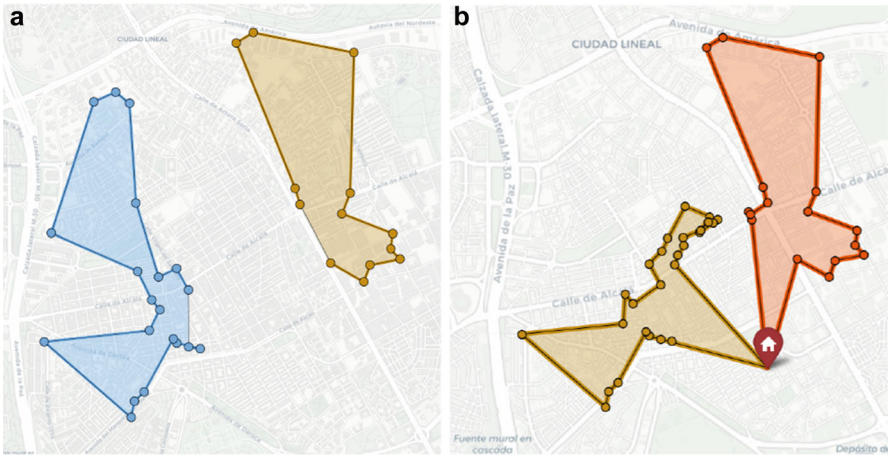


Fig. 6 Routes representation in a simplified way. (**a**) Depicted. (**b**) Complete

Fig. 7 Overlapping routes. (**a**) Overlapping. (**b**) Optimized

trucks park in a spot and from there provide service to several customers with a handcart. This is why a compact route is better than a straight-line route, since grouping customers around a parking spot simplifies the driver task.

Figure 7a clearly shows that routes 3 and 4 present and overlapping. They can be qualified as not friendly by the drivers since they both cover a similar territory. Alternatively, Fig. 7b organizes the customers in different nonoverlapping routes that are considered more friendly by the drivers, since they feel that no one interferes with their assigned territory.

Table 1 Low-demand comparison

COMPUTATIONAL RESULTS		Monday Nov-2023 Low Load	Thursday Nov 2023 Low Load	Friday Nov 2023 Low Load
Total NumTrucks		119	130	141
Total NumVans		0	15	15
Extra NumVans Trips		0	0	0
Delivered Orders		2165	3912	4851
Delivered Weight (kg)		339138.99	625764.3	840786.95
Balanced Objective Value in NumTrucks (BOV)		250	400	560
Vans < 800 kg	Total	0	0	0
	Min	–	–	–
	Mean	–	–	–
	Max	–	–	–
Truck Boxes $< 75\%$ BOV	Ref. Limit	188	300	420
	Total	8	15	20
	Min	156	277	109
	Mean	166	291	368
	Max	174	299	419
Truck Boxes $> 125\%$ BOV	Ref. Limit	314	500	700
	Total	6	0	0
	Min	323	–	–
	Mean	335	–	–
	Max	344	–	–

We complement the figures above with the following tables in which we report on the solutions obtained with our algorithm on different days of the week in 2023 (to compare the robustness of our method). In particular, Table 1 shows the computational results in three days in which the load of the trucks (the demand) is relatively low compared to their total capacity. Table 2 shows the results for two days with a high load on trucks (high demand). In particular, these two tables report the following statistics:

- **Total #Trucks**: Number of trucks included in the planning. Note that in this problem we have to use all the trucks.
- **Total #Vans**: Number of vans included in the planning.
- **Extra #Vans Trips**: Number of trips on the van above the maximum allowed that is required to provide the service.
- **Delivered Orders**: Number of orders to be delivered.
- **Delivered Weight (kg)**: Amount of weight of the total order.

Table 2 High-demand comparison

COMPUTATIONAL RESULTS		Tuesday Sept 2023 High Load	Wednesday Nov 2023 High Load
Total Trucks		138	141
Total Vans		17	17
Extra Vans Trips		4	0
Delivered Orders		4939	4375
Delivered Weight		1094715.78	999679.14
Mean Trucks Boxes		–	–
Vans < 800 kg	Total	1	4
	Min	658	44
	Mean	658	429.5
	Max	658	771.2
Trucks First Trip < 4000 kg	Total	0	4
	Min	–	60.6
	Mean	–	900.27
	Max	–	2326.57
Trucks First Trip < 5000 kg	Total	4	14
	Min	4711	60.6
	Mean	4823	3472.06
	Max	4983	4933.64
Trucks First Trip < 5500 kg	Total	18	37
	Min	4711	60.6
	Mean	5097	4560.71
	Max	5497	5482.02
Trucks Second Tips < 1500 kg	Total	4	5
	Min	127	290.4
	Mean	623	724.24
	Max	1477	1276.02

- **Balanced Objective Value** in #Trucks (BOV): Target value in the number of load boxes that we want to achieve in the solution corresponding to the low-demand days.
- **Vans < 800 kg**: Number of vans with a load lower than 800 kg. These vans are considered underused and, therefore, a good solution should have a low value of this statistic.
- **Truck Boxes $< 75\%$ BOV**: Number of trucks with a load in the number of boxes lower than 75% of the target value BOV.
- **Truck Boxes $> 125\%$ BOV**: Number of trucks with a load in the number of boxes higher than 125% of the target value BOV.
- **Trucks First Trip < 4000 kg**: Number of trucks that are below 4000 kg on the first trip on high-demand days.

- **Trucks First Trip $<$ 5000 kg**: Number of trucks that are below 5000 kg on the first trip on high-demand days.
- **Trucks First Trip $<$ 5500 kg**: Number of trucks that are below 5500 kg on the first trip on high-demand days.
- **Trucks Second Trip $<$ 1500 kg**: Number of trucks that are less than 1500 kg on the second trip on days of high demand.

Table 1 shows that the algorithm obtains high-quality results. This can be seen by analyzing the statistics reported in this figure. The first indicator is in *Extra Vans* = 0, which shows that no additional van was required. The second is in the wide row $Vans < 800$ kg and the symbols N/A represent that no van is underused. Furthermore, the number of trucks with boxes less than 75% of the target value is low. In particular, for Monday it is 8 out of 119 trucks, for Tuesday 15 out of 130, and for Friday 20 out of 141. Similarly, the number of trucks with overload, as measured by the number of boxes over 125% of the target value, is also low (6, 0, and 0 for the three days considered, respectively).

In Table 2 we can observe similar results in the case of high-demand days. In particular, here we look for maximum use of the vehicles in terms of their capacity because these days the demand is high and we cannot afford the underused of the vehicles. The statistics show that the number of underused vehicles is very low compared to the total number of vehicles. A close examination of the few cases in which some vehicles carry a low load discloses that it is due to isolated points that make it very difficult, if not impossible, to complete the load of the vehicle in an efficient route.

Comparison with Existing Solutions

We now illustrate the relative improvement achieved with the OGA solution compared to the current solution applied by the company.

To represent a larger area of the map, we avoid the inclusion of lines in the company solution (see Fig. 8) and only depict the customer's points. Points in the same color indicate that they belong to the same route. Since OGA's solution is clearer, we can include the lines in the associated map (see Fig. 9).

To analyze the issues of the current solution of the company, in Fig. 10 we circle some of the areas that we want to highlight. In the red circles of the figure, we identify a large number of overlappings among the polygons representing the routes. We can easily see that in Fig. 9 there are no overlaps. It is important to note that in both cases the routes are balanced, meaning that they have a similar load, which makes the optimization process (to avoid overlapping) challenging. Additionally, we can see in the optimized OGA solution the reduction in the number of zones that each route visits, thus increasing their compactness.

We conclude our experimentation with another example to compare the initial solution (the one implemented by the company using their own means) in Fig. 11,

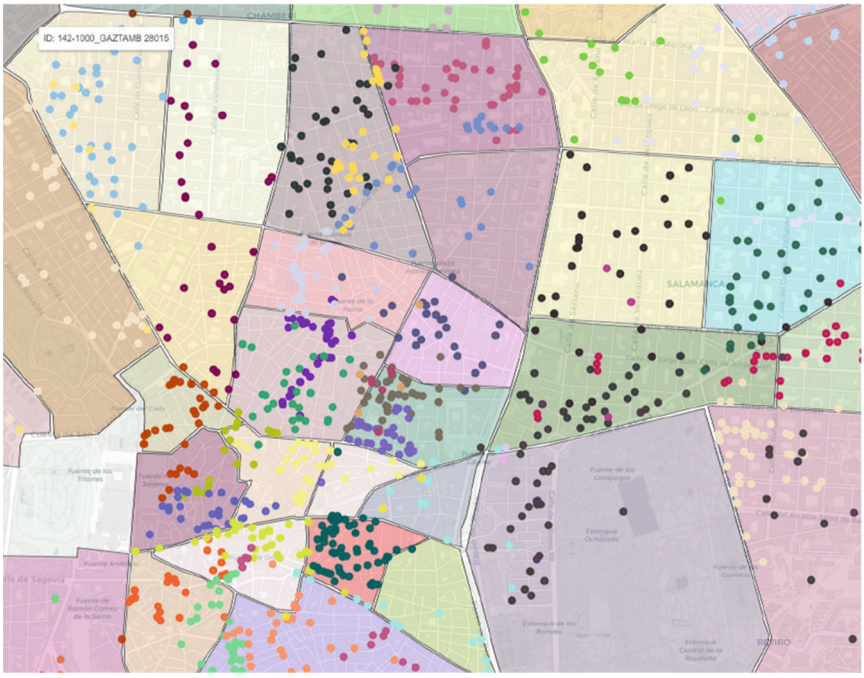


Fig. 8 Current solution applied by the company

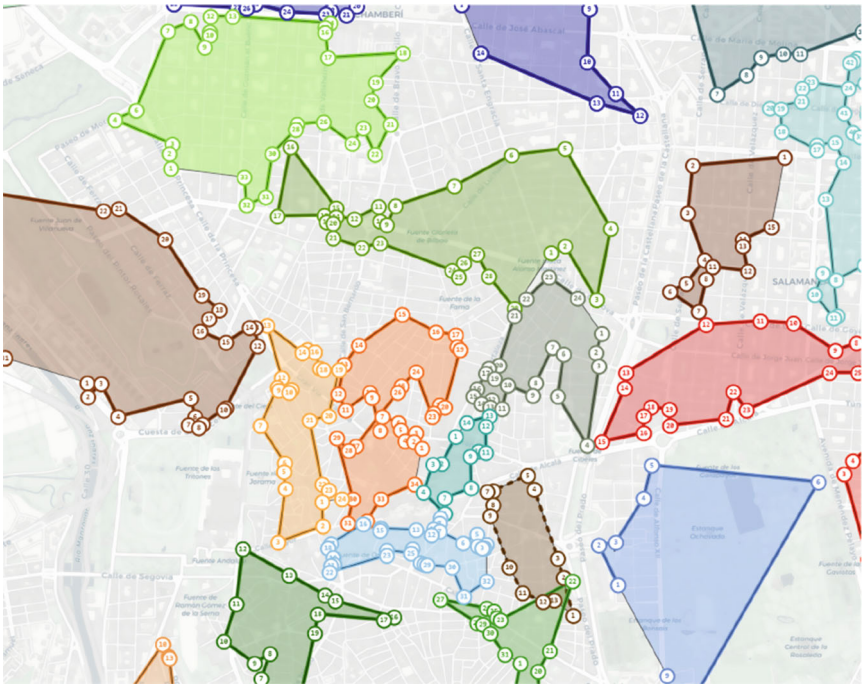


Fig. 9 Optimized solution obtained by OGA

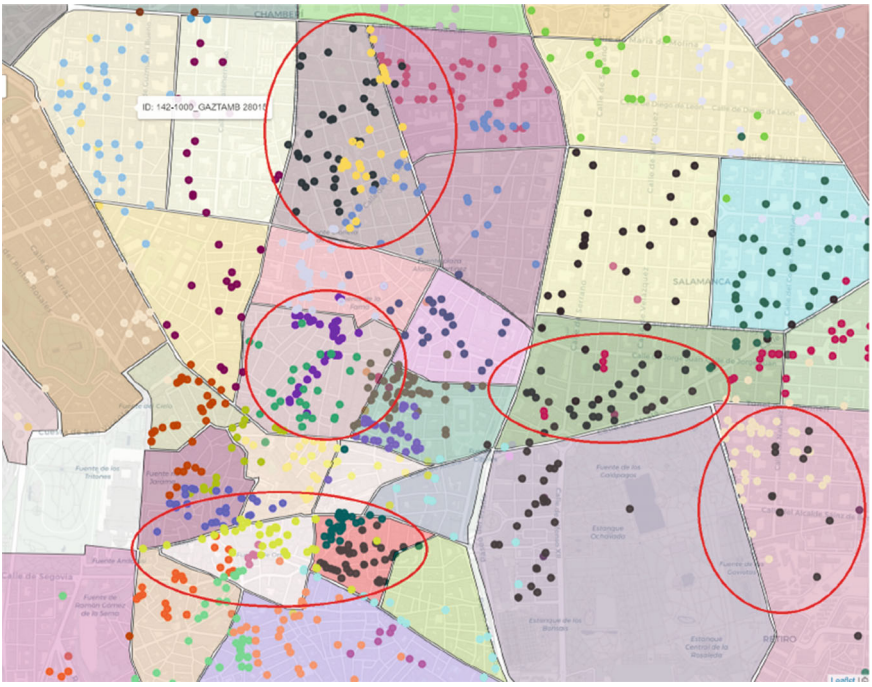


Fig. 10 Issues identified in current solution

Fig. 11 Solution created by the company

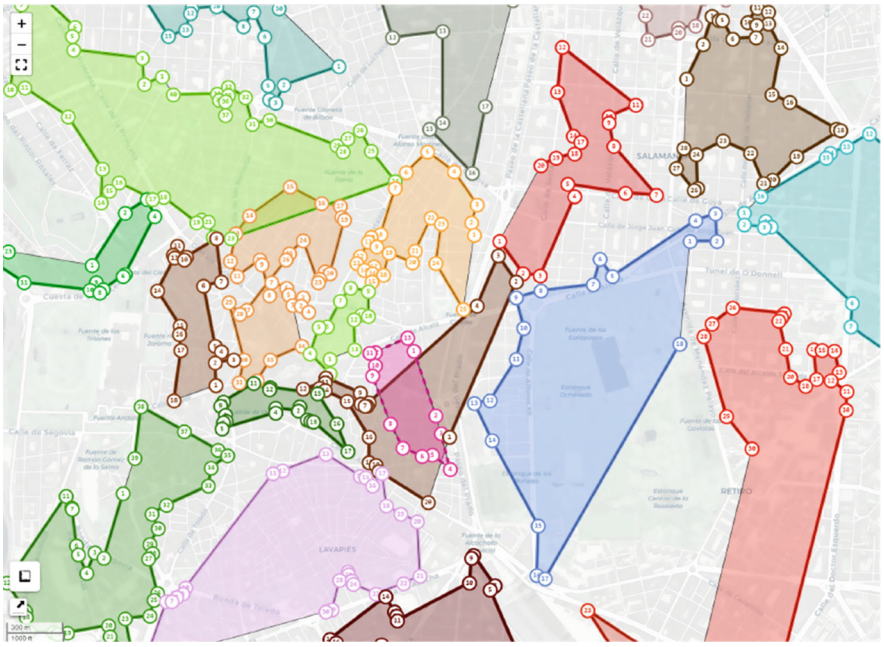


Fig. 12 Solution provided by OGA

Table 3 Numerical comparison between the company and OGA solutions

	Routes	Load	Pallets	Customers	Ldev.	Pdev.	LowLoad	LowPallets
Company	134	4113.8	438.5	17.8	33.6%	38.5%	36	50
OGA	133	3807.9	352.1	18.1	29.8%	36.8%	33	37

and the efficient solution in Fig. 12 provided by OGA with the application of the algorithm described in this chapter.

Figures 11 and 12 confirm the superiority of our proposal, based on metaheuristic technologies, compared with the initial solutions implemented by the company. We complement the visual analysis of the solutions with a quantitative comparison of the example shown in these figures. Table 3 compares the two solutions according to the following statistics:

- Routes. Number of routes in the solution
- Load. Average number of kilograms in the load of each truck
- Pallets. Average number of pallets in each truck
- Customers. Average number of customers served by each truck on each route
- Ldev. Average percentage deviation of the load of each truck with respect to the reference value of 4000 Kg set by the company
- Pdev. Average percentage deviation of the pallets of each truck with respect to the reference value of 400 pallets by truck set by the company

- LowLoad. Number of trucks (routes) with a load lower than the reference value of 4000 Kg set by the company
- LowPallets. Number of trucks (routes) with a number of pallets lower than the reference value of 400 pallets set by the company

The statistics reported in Table 3 clearly show the superiority of the OGA proposal over the previous solution implemented by the company. It must also be noted that it was obtained with the algorithm partially described in this chapter in a few minutes, while the solution developed by the company was manually created and required a few hours of the specialized engineers.

Conclusions

In this chapter, we have explored the application of metaheuristic techniques by OGA, a Spanish consulting company, to solve a complex real-world optimization problem in the beverage distribution business. Through our examination, several key conclusions have emerged:

- **Effective Application of Metaheuristics**. OGA has demonstrated that metaheuristic methodologies can be effectively employed to address intricate optimization problems in different domains, such as routing. The flexibility and robustness of these techniques enable OGA to tackle various logistical challenges, thus optimizing supply chain operations for their clients.
- **Broad Range of Techniques**. In line with other top consulting companies that apply metaheuristics, OGA utilizes a broad spectrum of methodologies, including Genetic Algorithms, GRASP, or tabu search among others. The selection of appropriate techniques based on the specific characteristics of each problem provides a powerful approach to solving real problems.
- **Financial Sustainability**. One of the standout aspects of OGA's approach is their ability to apply metaheuristic solutions in a financially sustainable manner. The combination of in-house development and a commercial solver such as Hexaly allows fast solutions to be created in a cost-effective way.
- **Handling Real-World Complexities**. The real-world problems addressed by OGA often come with multiple constraints and dynamic variables, making them highly complex. We illustrate with an example from the distribution business how a heuristic algorithm can provide high-quality solutions to these problems.
- **Contribution to Business Value**. By improving logistics and supply chain operations, OGA creates substantial business value for its clients. Optimization of these critical processes not only streamlines operations, but also enhances overall competitiveness and service delivery, leading to tangible improvements in business performance.

In conclusion, OGA's application of metaheuristic techniques showcases the profound impact these methodologies have on solving real-world optimization problems. Their success provides a valuable blueprint for other companies seeking to leverage similar technologies to enhance their business processes and achieve financial sustainability.

Cross-References

▸ GRASP
▸ Heuristics in Vehicle Routing
▸ Theory of Local Search

Acknowledgments This research has been partially supported by OGA under contract with the University of Valencia and by the European Regional Development Fund (MCIN/AIE/10.13039/501100011033 – ERDF. A way of making Europe) with codes PID2024-160226OB-C21, PID2021-125709OB-C21, and PID2022-139543OB-C41.

Competing Interest Declaration The author(s) has no competing interests to declare that are relevant to the content of this manuscript.

References

1. Accorsi L, Vigo D (2021) A fast and scalable heuristic for the solution of large-scale capacitated vehicle routing problems. Transp Sci 55(4):832–856
2. Arnold F, Gendreau M, Sörensen K (2019) Efficiently solving very large-scale routing problems. Comput Oper Res 107:32–42
3. Golden BL, Wasil EA (1987) OR practice–computerized vehicle routing in the soft drink industry. Eur Oper Res 35(1):6–17
4. Golden BL, Wasil EA, Kelly JP, Chao I-M (1998) The impact of metaheuristics on solving the vehicle routing problem: algorithms, problem sets, and computational results. In: Fleet management and logistics. Springer, Boston, pp 33–56
5. Panadero J, Keenan P, Juan A, Martí R, McGarraghy S (2021) A strategic oscillation simheuristic for the time capacitated arc routing problem with stochastic demands. Comput Oper Res 133:105377
6. Ribeiro-Kramer R, Subramanian A, Vaz-Penna P (2016) Asymmetric vehicle routing problem with heterogeneous limited fleet: a case study in a beverage industry. Gest Prod 23(1):165–176
7. Rocha D, Aloise D, Aloise DJ, Contarde C (2022) Visual attractiveness in vehicle routing via bi-objective optimization. Comput Oper Res 137:105507
8. Uchoa E, Pecin D, Pessoa A, Poggi M, Vidal T, Subramanian A (2017) New benchmark instances for the capacitated vehicle routing problem. Eur J Oper Res 257(3):845–858

Trees and Forests

60

Andréa Cynthia Santos, Christophe Duhamel, and Rafael Andrade

Contents

Introduction . . . 1794
Basic Features for Trees and Forests . . . 1797
Data Structures for Trees . . . 1800
Encodings for Trees . . . 1801
A Lagrangian Heuristic for DCMST . . . 1803
Problem Formulation . . . 1803
A Subgradient Procedure for DCMST Problem . . . 1805
A Greedy Heuristic for DCST . . . 1806
Improving DCSTs with a Local Search . . . 1808
A Lagrangian Heuristic for DCMST . . . 1809
Evolutionary Heuristic for a Generalization of BDMST . . . 1811
Problem Definition . . . 1812
An NSGA-II for Bi-MDCST . . . 1813
Conclusion . . . 1816
Cross-References . . . 1817
References . . . 1817

A. C. Santos (✉)
LITIS, ISEL, Le Havre Normandie University, Le Havre Cedex, France
e-mail: andrea-cynthia.duhamel@univ-lehavre.fr

C. Duhamel
LITIS, Le Havre Normandie University, UFR Sciences et Techniques, Le Havre Cedex, France
e-mail: christophe.duhamel@univ-lehavre.fr

R. Andrade
Departamento de Estatística e Matemática Aplicada, Centro de Ciências, Universidade Federal do Ceará, Fortaleza, Brazil
e-mail: rafael.andrade@ufc.br; rca@lia.ufc.br

© Springer Nature Switzerland AG 2025

R. Martí et al. (eds.), *Handbook of Heuristics*,
https://doi.org/10.1007/978-3-032-00385-0_49

Abstract

Trees and forests have been a fascinating research topic in Operations Research (OR)/Management Science (MS) throughout the years because they are involved in numerous difficult problems, have interesting theoretical properties, and cover a large number of practical applications. A tree is a finite undirected connected simple graph with no cycles, while a set of independent trees is called a forest. A spanning tree is a tree covering all nodes of a graph. In this chapter, key components for solving difficult tree and forest problems, as well as insights to develop efficient heuristics relying on such structures, are surveyed. They are usually combined to obtain very efficient metaheuristics, hybrid methods, and matheuristics. Some emerging topics and trends in trees and forests are pointed out. This is followed by two case studies: A Lagrangian-based heuristic for the minimum degree-constrained spanning tree problem and an evolutionary algorithm for a generalization of the bounded-diameter minimum spanning tree problem. Both problems find applications in network design, telecommunication, and transportation fields, among others.

Keywords

BDMST · DCMST · Forest · Heuristics · Lagrangian heuristic · Tree · Bi-objective heuristic

Introduction

Trees and forests have been a fascinating topic in Operations Research (OR)/Management Science (MS) throughout the years because they are the core of difficult problems, have interesting theoretical properties, and cover a large number of practical applications. Their interest remains intact due to the technical and scientific challenges and the rich diversity of applications using such structures. For instance, several applications have been recently addressed: people tracking is modeled as a minimum cost arborescence problem by [1]; cluster-based topologies with connecting requirements are defined as a minimum spanning tree (MST) to improve wireless sensor network lifetime in [2]; peer-to-peer distributed interactions are studied by [3] as a spanning tree, where end-to-end delays are minimized; reliable telecommunication networks have been investigated by [4], in which redundancy is added to spanning trees by introducing k-cliques ($k \geq 2$); and difficult MST problems have been investigated including uncertain data [5], new variants for optical multicast network design [6], or even multiobjective MST versions [7, 8].

Let $G = (V, E)$ be a finite, undirected, connected, and simple graph with a set V of vertices and a set E of edges, where $n = \mid V \mid$ and $m = \mid E \mid$. A tree $\mathcal{T} = \left(V', E'\right)$ is a connected subgraph of G with no cycles, such that $V' \subseteq V$ and $E' \subset E$. Whenever $V' = V$, the corresponding tree defines a spanning tree of G. An arborescence is a special directed tree with a root node r. Moreover, a forest is a set of disjoint trees

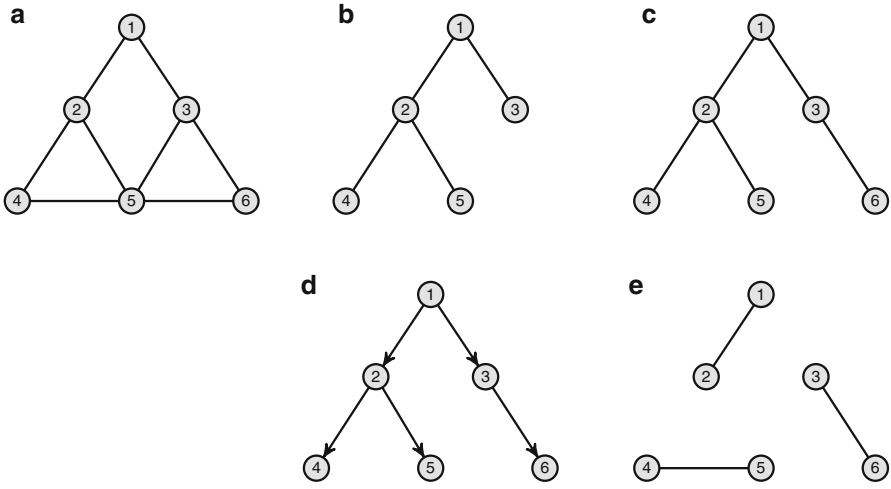


Fig. 1 Examples of tree, spanning tree, arborescence, and forest. (**a**) Graph G', (**b**) Tree, (**c**) Spanning tree, (**d**) Arborescence, and (**e**) Forest

$\mathcal{F} = \{\mathcal{T}_1, \mathcal{T}_2, \ldots, \mathcal{T}_l\}$. An example of graph G' is given in Fig. 1a, followed by examples of a tree, a spanning tree, an arborescence, and a forest of G', respectively, in Fig. 1b–e where $r = 1$. In this chapter, it is assumed familiarity with basic graph definitions such as connected graphs, connected components, subgraphs, paths, cycles, edges incident to nodes, node degree, etc.

The basic minimum spanning tree (MST) problem is defined in a weighted graph G, where a cost $c_{ij} \geq 0$ is associated with every edge $[i, j] \in E$. It aims at finding a spanning tree $\mathcal{T}$ of G such that its total cost is minimized. The number of possible MSTs for a graph is very high with up to n^{n-2} for complete graphs [9]. However, polynomial-time algorithms such as Prim's, Kruskal's, Boruvka's, and cycle elimination algorithm [10] are available to compute an MST. The general idea of Prim's algorithm is to extend an MST from an initial node $i \in V$. At each step it includes the cheapest-weight edge $[i, j] \in E$ such that one of its extremities belongs to the tree, while the other one does not. Thus, the connected component is extended until all nodes $i \in V$ belong to the solution. Kruskal's algorithm performs forest merging as follows. Initially, each vertex is a tree. Then, at each iteration, the cheapest edge connecting two forests is selected, and the two forests are merged, whenever it does not form a cycle. Boruvka's algorithm is quite similar to Kruskal's algorithm, except that, at each iteration, the cheapest edge connecting each forest to another one is selected. Then the merges are done accordingly. Both Kruskal's and Boruvka's algorithms can be developed with asymptotic complexity of $O(m \log n)$, as well as Prim's algorithm using binary heaps. Moreover, Prim's algorithm complexity can be improved to $O(m + n \log n)$ by using Fibonacci heaps; see [10] for details. The cycle elimination algorithm scans the graph using a depth-first search (DFS) strategy. The edge $[i, j]$ is added to the tree whenever a vertex

$j \in V$ not yet visited is found from the current node i of the DFS. However, if j has already been visited, it means a cycle has been found. In this case, the highest-weight edge from the found cycle is removed from the incumbent partial solution. DFS complexity is $O(m + n)$. Thus, considering a complete graph, the cycle elimination algorithm runs in $O(mn)$ since there are $O(m - n - 1)$ possible back edges and a cycle scan is done in $O(n)$. Several NP-hard problems rely on an MST with additional constraints on nodes, on edges, or even on the MST general structure. Thus, algorithms that compute an MST are usually adapted for such NP-hard extensions in order to define basic heuristics and obtain initial feasible solutions.

A large number of difficult problems rely on trees and forests. Studies [11–14] provide a collection of such problems, along with their complexity analysis. An example where constraints on the tree structure make the problem difficult is the Steiner Tree problem [15–18]. Given a subset $N \subset V$ of Steiner nodes (also referred as terminal nodes), the Steiner Tree problem consists in determining a particular minimum tree covering N. Also, setting constraints on the nodes or on the edges (paths) of an MST can transform the problem into an NP-hard problem. For instance, the degree-constrained minimum spanning tree (DCMST) problem [19–21] consists in finding an MST such that each vertex has a degree not larger than a maximum given value $k \in \mathbb{N}^*$. The bounded-diameter minimum spanning tree (BDMST) problem [7, 22, 23] is an example of a difficult problem where constraints are imposed on the MST diameter, i.e., the diameter of a tree is the number of edges in the longest path among any pair of vertices. Thus, the BDMST aims at finding an MST where the unique path between any pair of nodes does not exceed a given number of edges. Basic problems can also become difficult due to the nature of the data. For instance, the Minimum Arborescence problem [24] is defined in a digraph with real cost values associated with each arc. The objective is to find a minimum cost arborescence. Thus, for problems involving trees and forests, adding constraints or even changing few parameters or data can turn a P problem into an NP-complete problem.

Key components for solving difficult tree and forest problems, as well as insights to develop efficient heuristics relying on such structures, are surveyed here. They are usually combined to obtain very efficient metaheuristics, hybrid methods, and matheuristics. Section "Basic Features for Trees and Forests" is dedicated to a number of basic heuristics, local searches, etc. Then, two applications are described as examples, respectively, in sections "A Lagrangian Heuristic for DCMST" and "Evolutionary Heuristic for a Generalization of BDMST." Two different strategies to handle the difficult constraints are presented in these sections. The strategy used for DCMST consists of removing the difficult constraints and adding them into the objective function, following a Lagrangian relaxation. Another strategy is considered for the BDMST, where the difficult constraints are addressed as a new criterion and a bi-objective genetic algorithm is used.

Basic Features for Trees and Forests

Several heuristics, local searches, operators, different encodings, and perturbations are available in the literature to build trees and forests. They are very often combined to produce sophisticated methods. The most common heuristic structures are classified as shown in Fig. 2. The idea of this classification is not to cover all available heuristics for trees and forests, but to introduce the most used strategies having a strong potential to derive other heuristics for various difficult problems. There are two major classes of heuristics: constructive and improvement heuristics. The former includes edges or nodes at each iteration to obtain a feasible solution. The latter starts with an initial solution (e.g., a spanning tree), not necessarily feasible, and try to improve it in order to obtain better feasible solutions. Constructive and improvement heuristics are presented below, in such a way they can be adapted and applied to different difficult problems on trees and forests, where nodes, edges, or structural additional constraints are considered.

Constructive heuristics generate a feasible solution iteratively, using mainly two strategies: tree expansion and tree fusion. In the tree expansion strategy, one node is added at a time by means of an edge $e = [i, j] \in E$, where one extremity of e belongs to the solution under construction and the other extremity does not. Thus the partial incumbent solution is always a connected component. Tree expansion heuristics are usually implemented using Prim's algorithm. Examples of tree expansion heuristics are the One Time Tree (OTT) [25] and randomized greedy heuristic (RGH) [26] developed for the BDMST. Fusion heuristics start from initial disconnected forests and try to connect them by adding an edge $e = [i, j] \in E$ at a time. Thus a partial solution contains several connected components and Kruskal's algorithm can be adapted and applied [27], see section "A Greedy Heuristic for DCST." The way new edges/vertices are selected for inclusion in partial solutions (e.g., constructive heuristics) or for removal from a solution (e.g., improvement heuristics), after evaluating the objective function, has a strong impact on the final

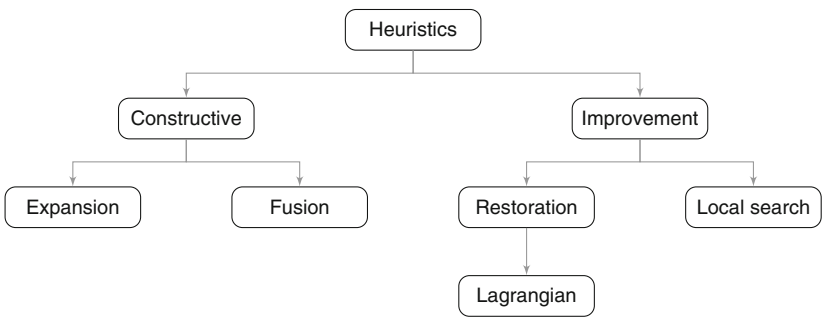


Fig. 2 A classification of heuristics for trees

solution quality. The most common ways to select edges/vertices are greedy, semi-greedy, and random. They can also be handled considering a single criterion or multi-criteria. Without loss of generality and considering a minimization function, a greedy criterion selects the edge/vertex insertion that leads to the smallest increase in the objective function. One way to handle a semi-greedy criterion is to build a list of good candidates to enter the solution, not necessarily the best ones, and make a random choice from this list. Another idea frequently found in the literature is to consider a normalized ratio between the cost and the impact on difficult constraints, e.g., a ratio between the edge cost and the additional increase in the diameter for the BDMST. Various criteria can also be addressed using a priority order instead of a ratio. For instance, one can use a second optimization criterion to select one edge, e.g., the impact on difficult constraints, whenever several edges with the smallest costs occur.

Improvement heuristics start from a solution, not necessarily feasible, and try to obtain a feasible solution by exchanging edges in the tree (resp. in a forest) with edges outside the solution. They can be roughly classified as restoration and local search heuristics. The restoration heuristics try to transform solutions in feasible ones by minimizing violations. A classical example of restoration heuristics is Lagrangian heuristics which have appeared with the pioneering works of [28, 29]. They have been broadly applied to a number of difficult problems relying on trees and forests [30–32]. The general idea is to relax difficult constraints and transfer them to the objective function, keeping a simple problem that is refined until a good solution to the overall problem is obtained. Consider a generic NP-hard optimization problem given by $Z = \{\min cx: Ax \geq b, Bx \geq d, x \in \{0, 1\}^t\}$, where A and B are matrix with rational coefficients and with dimensions $r \times t$ and $s \times t$, respectively; b, c, and d are rational vectors, and $x \in \{0, 1\}^t$ are integer variables. Let $\{\min cx: Bx \geq d, x \in \{0, 1\}^t\}$ be an easy problem that can be solved in polynomial time, and $Ax \geq b$ be the difficult constraints. Thus, Lagrangian multipliers $\lambda \in \mathbb{R}^r_+$ are associated with the difficult constraints and added into the objective function. The Lagrangian problem (LP) is given by $LP = \{\min cx + \lambda(b - Ax): Bx \geq d, x \in \{0, 1\}^t\}$ which is a lower bound on Z. The best lower bound is given by $LP(\lambda^*) = \max\{LP(\lambda), \lambda \geq 0\}$, called dual Lagrangian problem. The dual Lagrangian problem can be solved by using, for example, the subgradient methods of [33] or the volume algorithm of [34] or even by computing the Lagrange multipliers approximately using the multiplier adjustment method [35]. Theoretical results to determine the conditions stating when a Lagrangian relaxation can be better than a linear relaxation are provided by [36]. Obviously, the mathematical formulation and the decomposition will strongly impact the quality of solutions produced using Lagrangian relaxation. This kind of restoration heuristic has an interesting property: at each iteration dual and primal solutions can be built.

Local searches are very sophisticated improvement heuristics which contribute to producing very good local optima (eventually global optima). The most common local search moves, broadly applied to trees, are the *edge drop* move and the *edge insertion* move [23, 37]. Given an initial feasible solution for a difficult problem

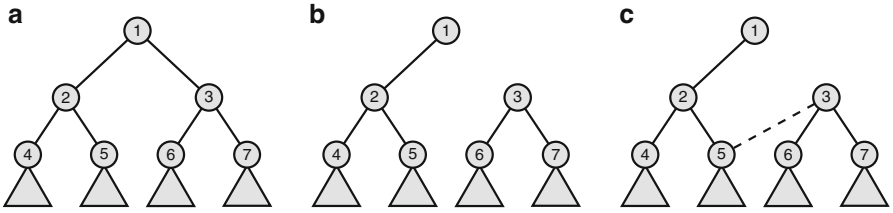


Fig. 3 Example of an *edge drop* move. (**a**) Tree, (**b**) Disconnecting 1 and 3, and (**c**) Reconnecting 3 and 5

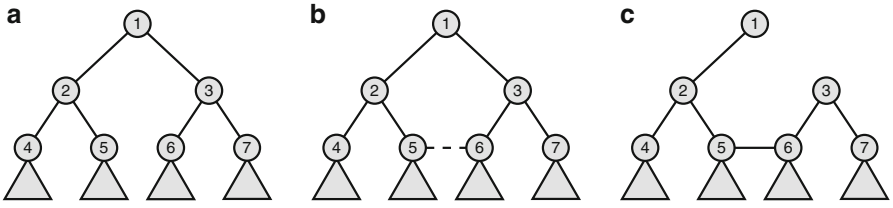


Fig. 4 Example of an *edge insertion* move. (**a**) Tree, (**b**) Connecting 5 and 6, and (**c**) Disconnecting 1 and 3

relying on a tree, the *edge drop* move consists of removing an edge from the tree. This produces two distinct connected components which are reconnected using an edge not in the tree. The greedy and semi-greedy criteria and multi-criteria can be used to choose the edge to enter the tree. Figure 3 illustrates an example, where Fig. 3a is a feasible initial solution. Then edge [1, 3] is removed as shown in Fig. 3b, generating two separate connected components. Consider [3, 5] as the edge which produces the best improvement possible. Thus it enters the solution following a greedy strategy as shown in Fig. 3c.

On the other hand, in the *edge insertion* move, an edge is first inserted in the tree. By definition of a tree, it necessarily results in a cycle. Then, an edge may be removed from the cycle. Figure 4 depicts an example for this move. A feasible solution is presented in Fig. 4a. In the sequel, edge [5, 6] is introduced and the cycle {5, 6, 3, 1, 2, 5} is formed as shown in Fig. 4b. Suppose that removing edge [1, 3] will produce the highest gain in the objective function. Thus this edge is dropped following a greedy strategy as depicted in Fig. 4c.

A number of variations on the *edge drop* and *edge insertion* moves are found in the literature. These moves are also referred as 1-opt since one edge in the incumbent solution is replaced by another one that does not belong to the solution. When applied to trees, two strategies are available to perform such a 1-opt move: either first dropping an edge or first inserting an edge. This may lead to different final solutions. The k-opt generalizes the 1-opt move by replacing k edges from an incumbent solution. This move is formalized in section "A Lagrangian Heuristic for DCMST."

Data Structures for Trees

Choosing the adequate data structure for representing and manipulating a solution is a critical task since it directly impacts the complexity of basic operations. They include both building and modifying a solution. Two mapping functions are associated with the data structures: How to store the information from a given tree (encoding) and how to obtain a tree out of the information (decoding). In this section, a basic direct representation is presented as well as several data to handle additional constraints on the trees. In section "Encodings for Trees," other representations are presented, along with the way to obtain the associated tree. Several insights are given regarding their use in evolutionary algorithms.

Let $G = (V, E)$ and $s \in V$ be, respectively, a graph defined as before and an arbitrary root vertex [10]. The following data structure can be used for directed and undirected trees, depending on the root node which can be artificial or not. The predecessor j of a vertex i in a tree is the first vertex in the path from i to s. By definition of a tree, j is unique and only depends on the choice of s. A successor j of a vertex i is a vertex whose predecessor is i. One of the simplest ways to store the information of a tree in a fixed-size data structure is to use the predecessors. This direct representation keeps the predecessors in a vertex-indexed vector *pred*. Thus $pred(i)$ is the predecessor of vertex i or, alternatively, the edge between i and $pred(i)$. The root has no predecessors; then $pred(s) = \emptyset$. Such a structure allows an $O(1)$ access to each predecessor and to each edge connecting the subtree rooted at i with the rest of the tree. It also allows a scan of all edges of a tree in $O(n)$.

Additional redundant data may be useful to perform tree manipulation and to allow efficient checking operations. One such data is the list of all the direct successors of a vertex. It can be explicitly defined as a list of vertices for each vertex, as well as be implicitly stored by adding two attributes to each vertex i: its first successor $succ(i)$ and $next(i)$, the next successor for $pred(i)$. For instance, if the direct successors of vertex a are $\{b, c, d\}$, the first successor of a is $b = succ(a)$. The successors of a are as follows: $c = next(b)$, $d = next(c)$ and $next(d) = \emptyset$. This encoding shares similarities with the forward star structure presented in [38] for graphs, and the implicit list can be made bidirectional by adding an attribute $pred(i)$ to each vertex which stores the previous successor in the list of $pred(i)$. This way, $O(1)$ operations such as successor insertion or deletion are ensured. The number of direct successors $nb_succ(i)$ can be kept as well.

Successor and predecessor structures allow managing several operations such as finding a cycle for *edge insertion* moves, finding the cheapest-weight costs for *edge drop* moves, and, with a few additional information, addressing difficult constraints like degree and diameter constraints. For instance, the $depth(i)$ of a vertex i is equal to the number of edges in the unique path from a given root vertex s to i. It can be used for checking hop and diameter constraints.

Scanning a cycle created by adding an edge $[i, j] \in E$ into a spanning tree can be done efficiently using the predecessor structure. First, each node in the path $i \rightarrow s$ is set as visited. The same is done in the path $j \rightarrow s$, stopping as soon as a given

vertex k has already been visited. Thus k belongs to both paths $i \rightarrow s$ and $j \rightarrow s$. Getting the edge with the highest cost can be done by keeping the edge with the highest cost found when scanning the path $i \rightarrow s$ (resp. $j \rightarrow s$) at each visited node. A modified DFS or breadth-first search (BFS) starting at i and stopping as soon as j is found could also be used. However, this would require more sophisticated tree data structures to obtain $O(n)$ complexity as for the predecessor vector. This way, the *edge insertion* move can be performed in $O(n)$.

Another useful data can be defined to quickly check if a vertex belongs to a subtree. Let $first(i)$ and $last(i)$ be two indexes associated with each vertex i. A DFS is applied on the tree, starting from s. Then $first(i)$ stores the DFS vertex counter the first time i is visited and $last(i)$ gets the DFS vertex counter when the DFS leaves the subtree of i. Thus, vertex j belongs to the subtree of vertex i if and only if $first(j) \in [first(i), last(i)]$. Besides, the number of vertices in the subtree of i, i included, is $size(i) = last(i) - first(i) + 1$. This information can be used to check for special cases in *edge insertion* moves, if one extremity belongs to the subtree of the other one. The same idea can be used to check if a tree is feasible in the capacitated MST [12, 37]. A demand D_i is associated with each vertex i and the sum of the demands in any subtree must not exceed a capacity Q. A demand counter is used instead of a vertex counter in the DFS.

The *edge drop* move requires reconnecting two trees disconnected after an edge $[i, j]$ has been removed. Without loss of generality, suppose i is the predecessor of j, i.e., $i = pred(j)$. First, all the vertices are set as unvisited. Then, all vertices in the subtree of j (included) are set as visited by performing a DFS or a BFS, using the list of successors presented before. Getting the lowest cost edge reconnecting the two trees can be done in $O(m)$ by finding the smallest edge in E with one extremity visited and the other one unvisited. This last operation is the most expensive and the *edge drop* move is in $O(m)$.

As mentioned before, the depth of a vertex from the root in a tree can be used to check hop and diameter constraints. For the root vertex, $depth(s) = 0$. A simple DFS or BFS starting from s is sufficient to compute the depth of each vertex in $O(n)$. They may require an artificial vertex 0 playing the role of center, $depth(0) = 0$. The artificial vertex connects the central vertex whenever D is even and one of the extremities for a central edge if D is odd. Let $L = D/2$ and $L = (D - 1)/2$ be the maximum depth allowed for a node, respectively, when D is even resp. odd. Thus, whenever an artificial vertex is used, a diameter limit D on the tree corresponds to a depth limit $L + 1$ for each vertex from vertex 0. Checking the degree is simpler; one may just add the number of successors and predecessors for each vertex.

Encodings for Trees

The tree representation given in the section "Data Structures for Trees" can be used for most constructive heuristics and for moves in local searches and metaheuristics. However, evolutionary algorithms (EAs) usually require additional properties on

the representation, and several other tree encodings have been proposed. In EAs, operations are mostly done on encodings (the chromosomes) rather than on the solutions. These usually consist of crossovers and mutations in order to make the set of encodings (the population) evolve. Selection is done to obtain the best elements. It requires the evaluation of each element through the construction of the associated solution, named decoding.

Several properties on the encoding/decoding, i.e., the mapping between encodings and solutions, are needed to fully benefit from the design of EAs: (i) the time and space complexities for those operations should be as low as possible, (ii) any encoding should correspond to a feasible solution, (iii) the decoding should be unbiased, (iv) there should be at least one encoding leading to each optimal solution, (v) any offspring obtained from crossover and mutation should be valid, and (vi) the offsprings should share as much similarities as possible with their parents.

According to those criteria, the predecessor structure presented before offers a $O(n)$ complexity for encoding/decoding. However, a random vector of predecessors is very unlikely to be feasible. Classical crossovers and mutations also generate infeasible offsprings and a repair operator is required. On the other hand, there always exists one encoding for each optimal solution and the feasible offsprings inherit a lot of similarities from their parents.

Another classical representation is the Boolean edge-indexed vector. The status of each edge (used/unused) is stored in the solution. Thus it follows the binary encoding paradigm suggested in the early versions of genetic algorithms (GAs). However, a random vector is highly unlikely to be feasible and crossovers and mutations seldom produce feasible offsprings. Thus the repair operator is mandatory, unless defining dedicated crossovers and mutations.

Prüfer sequences have also been used to encode trees. They were introduced in [39] to prove the Cayley's theorem [9] and there is a bijection between the set of spanning trees and the set of Prüfer sequences. They are basically repetition vectors of size $n - 2$ in which each vertex i appears $degree(i) - 1$ times. Encoding and decoding can be done in $O(n \log n)$. There is no need for repair operator since every sequence corresponds to a feasible spanning tree. However, while appealing, this representation suffers from several drawbacks [40], especially with respect to property (vi) since a small change in the sequence might lead to a completely different solution.

Random keys [41] have also been used and are quite popular. A random key is a weight in [0, 1]. For trees, one random key is associated with each edge, leading to a real-valued vector of size m. The decoding first consists of sorting the edges according to their random key and then applying a Kruskal-based constructive heuristic. Thus, it is done in $O(m \log m)$. It always leads to a feasible spanning tree and does not require a repair operator.

All the encodings suffer from shortcomings with respect to the properties (i)–(vi) mentioned before [42]. Moreover, they may not be able to handle sparse graph or additional constraints on the tree structure. Aside from random keys, a good alternative seems to be edge-set encoding [42]: The edges of the tree are directly stored in a variable-length vector. Thus, this is an explicit variable-size encoding

whose encoding/decoding can be done in $O(n)$. It is less biased toward special trees than random keys. It has been shown to be quite effective for solving NP-hard extensions of MST, provided dedicated crossovers and mutations are used.

A Lagrangian Heuristic for DCMST

The use of Lagrangian relaxation in heuristic procedures has been proved to be quite a powerful tool for solving certain problems, especially DCMST. A Lagrangian heuristic (LH) involves a small number of ingredients that can be implemented efficiently: (i) determining Lagrange multipliers and evaluating Lagrange subproblems, (ii) obtaining feasible solutions from the Lagrange multipliers, and (iii) improving feasible solutions by a local search. The main advantages of an LH are that it is simple to implement, solutions of good quality can be obtained in a reasonable computational time, and lower bounds (LB) and upper bounds (UB) are available at each iteration. In addition, an LH can be used to obtain starting incumbent solutions in more complex exact algorithms.

Let $G = (V, E)$ be a graph defined as before. Costs $c_e \in \mathbb{R}^+$ are associated with each edge $e \in E$, and a maximum degree $d_v \in \mathbb{N}^*$ is associated with each node $v \in V$. The DCMST problem consists in finding a minimal cost spanning tree $\mathcal{T}$ of G such that the degree constraints are ensured for each node.

Figure 5a illustrates an example of a graph G, in which edge costs are reported in the middle of each edge and the maximum node degree constraints for spanning trees of G are given in brackets near each node. An MST of G, with cost equal to 5, is given in Fig. 5b. One may note that this MST violates the degree constraint in node 1. A feasible solution for the degree-constrained spanning tree (DCST) for G is provided in Fig. 5c, with cost equal to 9.

Problem Formulation

In order to present a mathematical model for DCMST problem, let $E(S) \subseteq E$ be the set of edges with both extremities in $S \subseteq V$. Let $\delta(v) \subseteq E$ be the set of edges adjacent

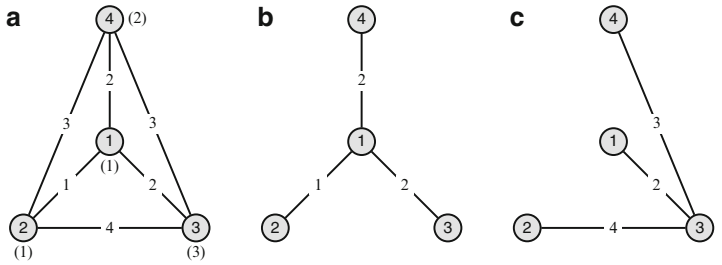


Fig. 5 Example of a tree with degree constraints. (**a**) A graph G, (**b**) MST of G, and (**c**) A DCST of G

to $v \in V$. Binary variables x_e, for all $e \in E$, represent the characteristic vector for a spanning tree of G, where $x_e = 1$ if an edge e belongs to the solution, and $x_e = 0$ otherwise. The mathematical formulation is given from (1) to (4).

$$(P) \quad \min_{x \in \{0,1\}^m} \quad \sum_{e \in E} c_e x_e \tag{1}$$

$$s.t. \quad \sum_{e \in E} x_e = n - 1, \tag{2}$$

$$\sum_{e \in E(S)} x_e \leq |S| - 1, S \subset V, \tag{3}$$

$$\sum_{e \in \delta(v)} x_e \leq d_v, \forall \ v \in V. \tag{4}$$

An MST is defined from (1) to (3) and constraints (4) control the degree of each vertex. These last constraints are the difficult ones. Thus they are relaxed in the LH presented next. Note that when $d_v = 2$, for all $v \in V$, the problem is reduced to finding a Hamiltonian path of minimum cost in G. Thus the DCMST is NP-hard [12].

The role of Lagrange penalties or multipliers is to take into account violations on some relaxed constraints. Given a tree $\mathcal{T}$ and for every node $v \in V$, the amount of node violation is the difference $\sum_{e \in \delta(v)} x_e - d_v$ between its maximum degree d_v and its degree in $\mathcal{T}$. In a DCST only non-positive node violations are allowed. Positive node violation is penalized by associating a non-negative penalty λ_i with each violated node i. In this case, every edge incident to i has its cost increased by λ_i. When both end nodes i and j of an edge $[i, j] \in E$ have positive penalties λ_i and λ_j, respectively, the resulting Lagrange cost becomes $c_{ij} + \lambda_i + \lambda_j$.

Consider the MST in Fig. 5b. It violates the degree constraint of node 1 (three edges are adjacent to this node and the maximum allowed is one). Figure 6 illustrates an example of MSTs computed with edge costs modified by Lagrange multipliers, where spanning trees of minimum cost are defined by solid lines in each graph. If a penalty $\lambda_1 = 10$ is set and $\lambda_i = 0$ for $i \in V \backslash \{1\}$ and an MST is computed for the graph with modified edge costs, the spanning tree in Fig. 6a is obtained. The degree constraint of node 2 is violated. Alternatively, if $\lambda_1 = 10$, $\lambda_2 = 2$, and $\lambda_i = 0$ for $i \in V \backslash \{1, 2\}$, an MST on the modified graph is in Fig. 6b. It is also feasible for the original graph G in Fig. 5a.

Thus, the idea is to determine the "best" set of Lagrange multipliers leading to an MST on the graph with modified edge costs whose Lagrangian solution value is equal to the value of an optimal DCMST. To do so, the Lagrangian problem (P_λ) is defined by associating Lagrange multipliers $\lambda \in \mathbb{R}^n_+$ with constraints (4) and bringing them to the objective function (1).

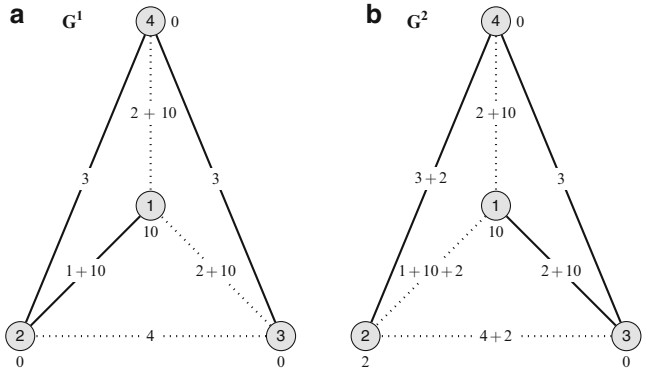


Fig. 6 Example of graphs with edge cost modified by Lagrange multipliers. (**a**) MST with $\lambda_1 = 10$. (**b**) MST with $\lambda_1 = 10$, $\lambda_2 = 2$

$$(P_\lambda) \quad \min_{x\in\{0,1\}^m} \sum_{[i,j]\in E} \left(c_{ij} + \lambda_i + \lambda_j\right) x_{ij} - \sum_{i\in V} \lambda_i d_i \qquad (5)$$
$$s.t. \qquad (2) - (3).$$

For any $\lambda \geq 0$, the solution value of (P_λ) is a lower bound on the optimal solution of (P). Consequently, the solution of the problem (D) is the best lower bound one can get for the solution of (P). Problem (D) is the Lagrangian dual problem of (P).

$$(D) \quad \max_{\lambda\geq 0} \quad \min_{x\in\{0,1\}^m} \sum_{[i,j]\in E} \left(c_{ij} + \lambda_i + \lambda_j\right) x_{ij} - \sum_{i\in V} \lambda_i d_i \qquad (6)$$
$$s.t. \qquad (2) - (3).$$

The MST on the graph G^2 in Fig. 6b has a cost $(3 + 5 + 12) - (10 \times 1 + 2 \times 1) = 8$. This is a lower bound on the optimal DCMST value of the graph in Fig. 5a. It is optimal since that solution is feasible for G and its original cost in G is 8.

A Subgradient Procedure for DCMST Problem

The classical subgradient (SG) method of Held et al. [43] is used to compute Lagrange multipliers λ iteratively for problem (P_λ). At each iteration k of the method, the idea is to find a direction s^k and a step size t_k to move from λ^k to a new set of multipliers λ^{k+1}:

$$\lambda^{k+1} = \max\left\{0, \lambda^k + t_k s^k\right\} \qquad (7)$$

s^k is the subgradient of $\left(P_{\lambda^k}\right)$ with respect to the Lagrangian solution x^k. Its coordinates are given by

$$s_i^k = \sum_{e \in \delta(i)} x_e^k - d_i, \quad \forall i \in V \tag{8}$$

Determining the step size t_k requires an incumbent UB on the optimal solution value of (P), the Lagrangian value LB^k referred to x^k, and the norm of the direction s^k. It is defined as

$$t_k = \frac{\alpha \left(UB - LB^k\right)}{\left\|s^k\right\|^2} \tag{9}$$

where α is a scaling factor. Initially $\alpha = 2$ and it is reduced (usually $\alpha \leftarrow \alpha/2$) after some iterations with no improvement on the best LB. Thus, convergence of the SG algorithm is ensured. The number of iterations can be limited to a maximum value determined according to the characteristic of the problem instances being solved. If the Lagrangian solution is feasible for (P) and its Lagrangian value is equal to UB, it is optimal and the algorithm stops. Further details on the SG algorithm for DCMST problem can be found in [30].

As an example, the SG algorithm is applied to the graph G in Fig. 5a. Initially $\lambda^0 = 0$ and the resulting graph $G(\lambda^0)$ has the same edge costs as in G. The Lagrangian solution $\mathcal{T}^0$ for $G(\lambda^0)$ is the MST of G (Fig. 5b), and its Lagrangian cost is $\mathrm{LB}^0 = 5$. The degree of nodes 1, 2, 3, and 4 in $\mathcal{T}^0$ are 3, 1, 1, and 1, respectively. The initial subgradient direction is $(s^0)^t = (2, 0, -2, -1)$. Figure 8 presents graphs $G(\lambda^k)$ of each SG iteration k. The Lagrange edge costs are displayed near each edge and Lagrange multipliers near each node. Moreover, Lagrangian solutions $\mathcal{T}^k$ of $G(\lambda^k)$ are defined by solid lines. Lagrange multipliers are computed according to Fig. 5 as follows: suppose the incumbent $\mathrm{UB} = 9$ is given by the DCST in Fig. 5c, as computed by heuristic in section "A Greedy Heuristic for DCST." The new set of Lagrange multipliers is $(\lambda^1)^t = (16/9, 0, 0, 0)$. The corresponding graph $G(\lambda^1)$ leads to the Lagrangian solution $\mathcal{T}^1$ in Fig. 7, in solid lines. The new direction s^1 is carried forward in Fig. 8, column $\mathcal{T}^1$. The SG algorithm iterates until iteration 4, where the Lagrangian solution $\mathcal{T}^4$ is a feasible DCST for G. The original cost of $\mathcal{T}^4$ in G is 8 and its Lagrangian cost is $\mathrm{LB}^4 = 596/75$. As all edges of G have integer costs, the optimal solution value must be integer. Therefore, the optimal solution value UB^* is such that $\lceil 596/75 \rceil \leq \mathrm{UB}^* \leq 8$. Thus $\mathcal{T}^4$ is optimal and the algorithm stops.

A Greedy Heuristic for DCST

Kruskal's algorithm can be adapted [30] to compute DCSTs on a graph $G = (V, E)$. It consists of (i) ordering the edges in E by non-decreasing edge costs; (ii) creating a forest with $|V|$ trivial trees $C_i = \{i\}$, for all $i \in V$; and (iii) connecting all disjoint trees (forests) by including one edge at a time, using the ordered list until a spanning tree is obtained. A vertex is saturated if its degree matches the maximum degree

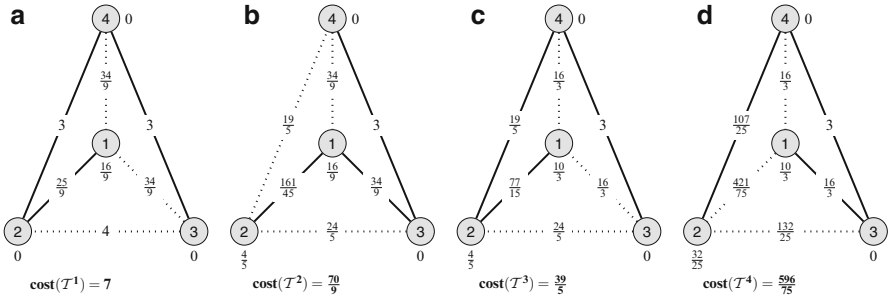


Fig. 7 Example of iterations for the SG method. (**a**) $\mathbf{G}(\boldsymbol{\lambda}^1)$, (**b**) $\mathbf{G}(\boldsymbol{\lambda}^2)$, (**c**) $\mathbf{G}(\boldsymbol{\lambda}^3)$, and (**d**) $\mathbf{G}(\boldsymbol{\lambda}^4)$

$$\mathcal{T}^1 \quad \lambda^1 = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \end{pmatrix} + \frac{2(9-5)}{9} \begin{pmatrix} 2 \\ 0 \\ -2 \\ -1 \end{pmatrix} \overset{\lambda \geq 0}{=} \begin{pmatrix} \frac{16}{9} \\ 0 \\ 0 \\ 0 \end{pmatrix} \quad s^1 = \begin{pmatrix} 0 \\ 1 \\ -2 \\ 0 \end{pmatrix}$$

$$\mathcal{T}^2 \quad \lambda^2 = \begin{pmatrix} \frac{16}{9} \\ 0 \\ 0 \\ 0 \end{pmatrix} + \frac{2(9-7)}{5} \begin{pmatrix} 0 \\ 1 \\ -2 \\ 0 \end{pmatrix} \overset{\lambda \geq 0}{=} \begin{pmatrix} \frac{16}{9} \\ \frac{4}{5} \\ 0 \\ 0 \end{pmatrix} \quad s^2 = \begin{pmatrix} 1 \\ 0 \\ -1 \\ -1 \end{pmatrix}$$

$$\mathcal{T}^3 \quad \lambda^3 = \begin{pmatrix} \frac{16}{9} \\ \frac{4}{5} \\ 0 \\ 0 \end{pmatrix} + \frac{2(9-\frac{70}{9})}{3} \begin{pmatrix} 1 \\ 0 \\ -1 \\ -1 \end{pmatrix} \overset{\lambda \geq 0}{=} \begin{pmatrix} \frac{10}{3} \\ \frac{4}{5} \\ 0 \\ 0 \end{pmatrix} \quad s^3 = \begin{pmatrix} 0 \\ 1 \\ -2 \\ 0 \end{pmatrix}$$

$$\mathcal{T}^4 \quad \lambda^4 = \begin{pmatrix} \frac{10}{3} \\ \frac{4}{5} \\ 0 \\ 0 \end{pmatrix} + \frac{2(9-\frac{39}{5})}{5} \begin{pmatrix} 0 \\ 1 \\ -2 \\ 0 \end{pmatrix} \overset{\lambda \geq 0}{=} \begin{pmatrix} \frac{10}{3} \\ \frac{32}{25} \\ 0 \\ 0 \end{pmatrix} \quad s^4 = \begin{pmatrix} 0 \\ 0 \\ -1 \\ 0 \end{pmatrix}$$

Fig. 8 Updating Lagrange multipliers during the SG iterations shown in Fig. 7

allowed. A tree is saturated if all its vertices are saturated. Then, an edge $[u, v] \in E$ is inserted in the solution if both u and v are not saturated and if they do not belong to the same tree. Moreover, merging the two trees using $[u, v]$ must not lead to a saturated tree, except if the resulting tree spans V.

Consider the graph G in Fig. 5a and assume that the ordered list of edges is {[1, 2], [1, 4], [1, 3], [2, 4], [3, 4], [2, 3]}. Initially, each vertex is a trivial tree. The first edge to be considered is [1, 2]. It is discarded as its inclusion would result in a saturated tree. In the sequel, edge [1, 4] can be added since it does not saturate vertices 1 and 4 nor create a cycle. Node 1 is saturated; thus, the third edge [1, 3] is rejected. The fourth edge [2, 4] is also not accepted because it would result in

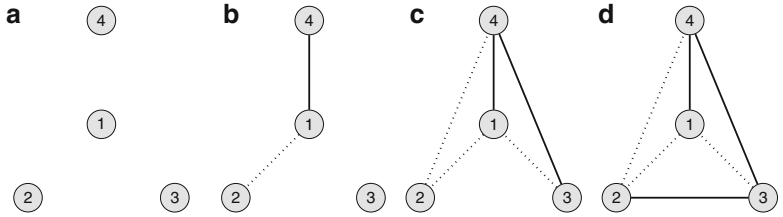


Fig. 9 Example of Kruskal-based heuristic for DCST. (**a**) Initial, (**b**) First edge, (**c**) Second edge, and (**d**) Third edge

a saturated tree. The fifth edge [3, 4] is included, *idem* for the sixth edge [2, 3]. Then the algorithm stops with a feasible DCST. Note that this heuristic stops with a feasible solution for complete graphs only. Figure 9 presents this example with the ordered list of edges defined above. Solid lines correspond to edges included, while dotted lines correspond to discarded edges.

This heuristic can also be applied to graphs with modified Lagrange edge costs during the SG and compute its cost on the graph G. Nevertheless, doing this every SG iteration is time consuming. Finally, this heuristic can be used only when a Lagrangian solution improves the best incumbent LB on the optimal solution of (P).

Improving DCSTs with a Local Search

Given a graph $G = (V, E)$ and a spanning tree $\mathcal{T}$ of G, let $E(\mathcal{T})$ be the set of edges in $\mathcal{T}$. Moreover, consider a given subset $\{e_1, e_2, \cdots, e_k\} \subset E(\mathcal{T})$ of the edges belonging to $\mathcal{T}$. If there is a subset of edges $\left\{\hat{e}_1, \hat{e}_2, \cdots, \hat{e}_k\right\} \subset E \backslash E(\mathcal{T})$ such that $E(\mathcal{T}) \setminus \{e_1, e_2, \cdots, e_k\} \cup \left\{\hat{e}_1, \hat{e}_2, \cdots, \hat{e}_k\right\}$ induces a spanning tree $\hat{\mathcal{T}}$ of G, then $\hat{\mathcal{T}}$ is said to belong to a k-neighborhood of $\mathcal{T}$, denoted by $\hat{\mathcal{T}} \in N_k(\mathcal{T})$, and that $\mathcal{T} \in N_k\left(\hat{\mathcal{T}}\right)$.

Consider $k+1$ connected components $C_1, C_2, \cdots, C_{k+1}$ obtained after removing k edges $\{e_1, e_2, \cdots, e_k\} \subset E(\mathcal{T})$ from a given spanning tree $\mathcal{T}$. If $\{\overline{e}_1, \overline{e}_2, \cdots, \overline{e}_k\} \subset E \backslash E(\mathcal{T})$ is a minimum cost subset of k edges to reconnect the $k+1$ components $C_1, C_2, \cdots, C_{k+1}$ forming a new spanning tree, then $\mathcal{T} \cup \{\overline{e}_1, \overline{e}_2, \cdots, \overline{e}_k\} \setminus \{e_1, e_2, \cdots, e_k\}$ is called a *k -opt edge exchange*. Such a move corresponds to applying k times an *edge drop* or an *edge insertion* move, as seen in section "Basic Features for Trees and Forests."

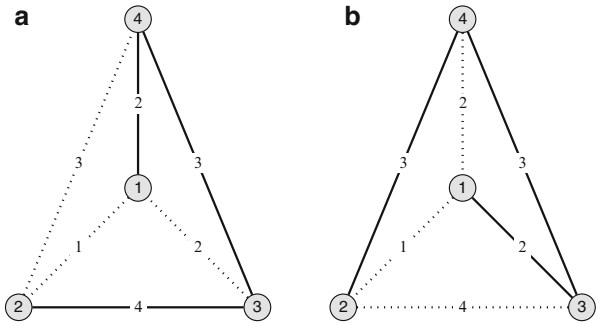


Fig. 10 Example of a *2-opt edge exchange* move. (**a**) Tree $\mathcal{T}$, (**b**) Tree $\mathcal{T}^1$

Figure 10 illustrates an example of a *2-opt edge exchange*. Tree $\mathcal{T}^1$ in Fig. 10b is obtained from $\mathcal{T}$ in Fig. 10a by removing the edges [1, 4], [2, 3] and adding the edges [1, 3], [2, 4] while keeping the feasibility of the resulting tree.

Determining the best *k-opt edge exchange* move among all $\binom{|E(\mathcal{T})|}{k}$ combinations of k edges of $E(\mathcal{T})$ is very time consuming. A reasonable trade-off between solution quality and processing time is to limit k to 3. A greedy local search then iteratively performs *k-opt edge exchange* moves, until reaching a local optimum.

A Lagrangian Heuristic for DCMST

Using the components presented before, Algorithm 1 is a Lagrangian heuristic for DCMST problem. Given a graph $G = (V, E)$, a feasible starting DCST $\mathcal{T}^*$ is computed by the heuristic in section "A Greedy Heuristic for DCST." A limit *MaxIter* is set to the number of SG iterations. Basically, the subgradient method (section "A Subgradient Procedure for DCMST Problem") is used to update Lagrange multipliers until the stopping criterion (number of iterations or optimal solution) is met. At each iteration k, the classical Kruskal's algorithm [27] computes a Lagrangian solution $\mathcal{T}^k$ on the graph $G(\lambda^k)$: $\mathcal{T}^k \backslash \text{MST}\left(G\left(\lambda^k\right)\right)$. The Lagrangian solution cost of $\mathcal{T}^k$ (LagrCost $\left(\mathcal{T}^k\right)$) gives a lower bound LB^k on the optimal solution value. If it improves LB, the new value LB^k is updated. Eventually, $\mathcal{T}^k$ is a DCST. In this case, a local search procedure (section "Improving DCSTs with a Local Search") is called to improve $\mathcal{T}^k$ if the cost Cost $\left(\mathcal{T}^k\right)$ of $\mathcal{T}^k$ in G improves UB: $\mathcal{T}^* \backslash \text{LocalSearch}\left(\mathcal{T}^k\right)$.

Algorithm 1 Lagrangian heuristic [30]

Data: a graph $G = (V, E)$, a DCST $\mathcal{T}^*$, a maximum number of iterations $MaxIter$;
Result: a DCST of G;
initialization: $k \leftarrow 0$; $\lambda^0 \leftarrow \mathbf{0}$; $T^0 \leftarrow MST(G(\lambda^0))$; LB$\leftarrow LagrCost(\mathcal{T}^0)$; UB$\leftarrow Cost(\mathcal{T}^*)$; **while** *(LB<UB* **and** $k < MaxIter$*)* **do**
  compute λ^{k+1} by using Equation (7);
  set $\mathcal{T}^k \leftarrow MST(G(\lambda^k))$ and LB$^k \leftarrow LagrCost(\mathcal{T}^k)$;
  if *(*LB$^k >$ *LB)* **then**
    LB$\leftarrow$LB^k;
  end
  if *(*$\mathcal{T}^k$ *is a DCST* **and** $Cost(\mathcal{T}^k) <$*UB)* **then**
    $\mathcal{T}^* \leftarrow LocalSearch(\mathcal{T}^k)$;
    UB$\leftarrow Cost(\mathcal{T}^*)$;
  end
  $k \leftarrow k + 1$;
end
return $\mathcal{T}^*$;

As mentioned before, halving α after some SG iterations without any improvement on LB guarantees the convergence of the subgradient procedure. Also, one can work with a reduced instance $G' = (V, E')$ of the problem, by considering only a subset $E' \subset E$ of ordered edges required to obtain a feasible DCST for the problem, by using the modified Kruskal's algorithm from section "A Greedy Heuristic for DCST." Moreover, more sophisticated local searches and even metaheuristics can be used instead of the local search presented in section "Improving DCSTs with a Local Search." For instance, variable neighborhood search (VNS), dynamic variable neighborhood descent (VND), and absorption function [44] have been used in [19]. As the k-opt local search procedure is based on a greedy criterion, the Lagrangian heuristic may not escape local optima. To overcome this issue, the authors in [19] suggest to integrate a VNS and a dynamic VND, as well as absorption functions of [44].

The LH presented here has been applied to a number of benchmark instances from the literature: Euclidean And-inst, Hamiltonian Ham-inst, Shrd [45], M-graph and R-graph instances, and the new Dr-inst and De-inst instances [21]. In addition, the solutions obtained by LH have been used as cutoff to speed up exact methods [19, 21] for DCMST problems.

Results obtained by LH have provided new average gaps of up to 0.188% and 6.230% for And-inst and Ham-inst instances, respectively. Moreover, it has solved to optimality the SHRD, M-graph, and R-graph sets. After introducing the VNS, optimality was reached for 23 out of 25 And-inst instances, 4 out of 15 Ham-inst

instances, and 8 out of 18 De-inst instances. Furthermore, optimal solutions were obtained for all Dr-inst instances. Thus, LH has been shown to be a powerful approach for DCMST problems.

Evolutionary Heuristic for a Generalization of BDMST

In this section, another way to handle difficult constraints is described for a generalization of the BDMST. The difficult constraints of the problem are defined on the paths of the tree rather than on the nodes as for DCMST. BDMST is defined in a connected and undirected graph $G = (V, E)$, where costs $c_{ij} \geq 0$ are associated with each edge $[i, j] \in E$. Consider $\mathcal{T}$ as a spanning tree of G. By MST definition, there is a unique path $\mathcal{P}_{ij}$ in $\mathcal{T}$ between any pair of nodes $i, j \in V$. Thus, given ρ_{ij} the number of edges in the path $\mathcal{P}_{ij}$, the diameter D of $\mathcal{T}$ is defined as $D = \max\{\rho_{ij} : \forall\, i, j \in V, i \neq j\}$. The BDMST consists in finding an MST such that its diameter does not exceed a given $k \in \mathbb{N}^*$, i.e., $D \leq k$. This problem belongs to the NP-hard class whenever $3 < k < n - 1$ [12].

An example is shown in Fig. 11 for $k = 2$, considering the graph given in Fig. 11a. The MST is presented in Fig. 11b. It violates the diameter since $D = 3$. The optimal BDMST solution is provided in Fig. 11c, where the longest path has two edges ($D = 2$).

Several works are found in the literature for BDMST such as mathematical formulations [46–48], exact methods [22, 49, 50], and heuristics [23, 26, 51]. Yet, optimality has not been proved for an important benchmark of instances proposed by [26], containing complete graphs whose size varies from 50 to 1000 vertices. Thus, an original strategy is to investigate the search space of this problem by using bi-objective approaches to compute the Pareto front. The idea is to drop the difficult constraints, i.e., diameter, and to consider them as a new criterion. Then a bi-objective optimization problem is solved. Thus the total tree cost and the diameter are minimized simultaneously. The problem remains difficult since the goal is to determine a Pareto front. However, the multiobjective strategy is interesting since every spanning tree is feasible and efficient algorithms are available to compute

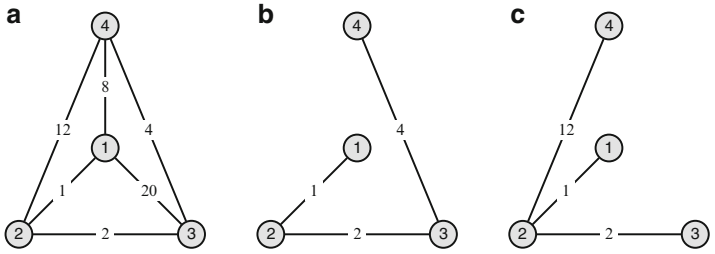


Fig. 11 Example of a tree with diameter constraints. (**a**) A graph G, (**b**) MST of G, and (**c**) A BDMST of G

them. Moreover, in recent years, several advances have been done in solving multiobjective (bi-objective) problems using heuristics and metaheuristics. In addition, bi-objective heuristics do not depend on a good mathematical formulation. Another key point is that it is possible to bound the values for each criteria for problems relying on trees such as BDMST.

Bi-objective optimization for MSTs has already been investigated considering two cost objective functions. For example, a branch-and-bound was proposed in [8] and a two-phase enumeration was introduced in [52, 53]. Bi-objective metaheuristics are also used for two cost objective functions as in [54, 55]. A multiobjective evolutionary algorithm (MOEA) for the network design problem is presented in [55] to minimize the infrastructure cost and the maintenance cost, while a multiobjective greedy randomized adaptive search procedure (GRASP) is applied in [54] to find MST with two costs. Using this idea to handle difficult constraints seems to be more recent, in particular for the BDMST. Works [7, 56, 57] are dedicated to multiobjective strategies for the BDMST, referred as the bi-objective minimum diameter-cost spanning tree (bi-MDCST) problem. One of the most efficient multiobjective heuristics developed for bi-MDCST is the non-dominated sorting genetic algorithm (NSGA-II), presented in the sequel.

Problem Definition

Without loss of generality, applying bi-objective heuristics for a problem $\mathcal{P}$ like the bi-MDCST implies the minimization of two objective functions $\{\min f_1(x), \min f_2(x) \ | x \in \mathcal{X}\}$. $\mathcal{X}$ is the feasible solution space of $\mathcal{P}$ and $f(x)$ is the vector of objective functions. For the bi-MDCST, $\{f_1(x), f_2(x)\}$ denote the cost and the diameter, respectively. Then a set of compromise solutions with respect to the two objective functions has to be obtained instead of a single solution as for the classical BDMST.

The concept of dominance is used to define the set of compromise solutions, usually called Pareto front. Considering two bi-MDCST solutions x and y, x *dominates* y if and only if the following conditions hold:

$$\begin{cases} f_k(x) < f_k(y) \ \exists k \in \{1, 2\} \\ f_k(x) \leq f_k(y) \ \forall k \in \{1, 2\} \end{cases} \text{and} \tag{10}$$

A Pareto solution x^* is optimal whenever it is not dominated by any solution in $\mathcal{X}$. The Pareto-optimal front is composed of the Pareto-optimal solutions (non-dominated). The bi-MDCST seeks a set of Pareto-optimal spanning trees $\mathcal{T}$ of G where f_1 and f_2 are simultaneously minimized. Figure 12 presents three solutions for bi-MDCST, considering the graph of Fig. 11a. The cost and the diameter for $\mathcal{T}_1$, $\mathcal{T}_2$, and $\mathcal{T}_3$ are, respectively, $\{29, 2\}$, $\{14, 3\}$, and $\{24, 2\}$. Solution $\mathcal{T}_1$ is dominated by $\mathcal{T}_3$ due to the cost. However, solutions $\mathcal{T}_2$ and $\mathcal{T}_3$ are not dominated by each other since $\mathcal{T}_2$ has a smaller cost than $\mathcal{T}_3$, while $\mathcal{T}_3$ has a smaller diameter than $\mathcal{T}_2$. One may

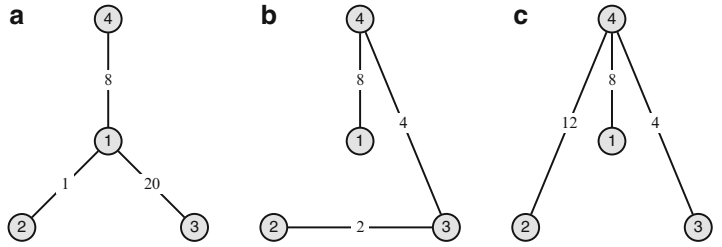


Fig. 12 Example of solutions for bi-MDCST. (**a**) Solution $\mathcal{T}_1$, (**b**) Solution $\mathcal{T}_2$, and (**c**) Solution $\mathcal{T}_3$

note that solutions $\mathcal{T}_2$ and $\mathcal{T}_3$ are not Pareto-optimal because they are, respectively, dominated by the MST depicted in Fig. 11b of value {7,3} and the solution shown in Fig. 11c of values {15,2}.

For bi-MDCST, an obvious LB on the cost can be obtained by computing an MST on the graph. The diameter of this solution also provides an UB on the diameter. In addition, for complete graphs, the star is also a trivial LB on the diameter. The minimal cost of a star in the graph can provide an UB for the cost. Spanning trees of diameters $D \in \{2, 3\}$ can be computed in polynomial time as shown in [7]. Those LB on the diameter may not necessarily exist on general graphs. However, checking if a graph contains at least one such spanning tree can be done in polynomial time.

An NSGA-II for Bi-MDCST

The NSGA-II has been proposed by [58] and is one of the metaheuristics available to compute Pareto fronts for multiobjective optimization problems. This method has been applied to a number of multiobjective optimization problems for the past ten years. Moreover, it uses simple and efficient operators to manage the Pareto front convergence. Such operators are added to a classical genetic algorithm (GA), which is very popular in the scientific literature. The two special multiobjective operators are the *ranking* and the *crowding distance*. These two operators are computed for each solution and are responsible to manage the Pareto front convergence. Both the *ranking* and the *crowding distance* are associated with each solution. Given a target solution $\mathcal{T}$, the *ranking* contains the number of solutions that dominate $\mathcal{T}$. Thus, the lower the ranking, the better the solution. Fronts are then defined by solutions of same ranking. Given a front of M solutions, the *crowding distance* w is computed following Eq. (11), where $sc(j)$ and $pr(j)$ are, respectively, the value which precedes j in w and the value which follows j in w. For any (nondegenerate) front with more than one solution, ${z_m}^{\max} \neq {z_m}^{\min}$. The bi-MDCST involves two objectives $m = 1, 2$; thus, ${z_m}^{\max}$ and ${z_m}^{\min}$ are the maximum and the minimum objective function values, which need to be properly normalized. The crowding distance is a kind of Manhattan distance between two solutions from the same front. Then, the higher the crowding distance is, the more distant the two solutions are. As a consequence, solutions with larger crowding distance are preferred since they belong to less covered areas.

$$w_j = \sum_{m=1}^{M} \left(\frac{z_m^{sc(j)} - z_m^{pr(j)}}{z_m^{\max} - z_m^{\min}} \right) \tag{11}$$

The algorithm performs the following steps: (1) generate the initial population (set of solutions), (2) compute and order the population using the ranking and crowding distance, (3) select the first half elements of the population, and (4) generate the remaining elements using genetic operators and go to step (2). These steps are repeated until stopping criteria are met and are detailed below. In addition, a local search can be applied to each new solution in the population, i.e., before moving to step 2. In the vocabulary of GA, the use of a local search within a GA is referred as a memetic algorithm (MA). The idea of generating all $2n$ solutions and ordering them before cutting n solutions avoids cutting good solutions.

Given a graph $G = (V, E)$, a solution for bi-MDSCT (also referred as chromosome in the GA vocabulary) can be encoded using a vector of size n containing the direct predecessor of a vertex in the tree. A population can be generated using any heuristics (greedy, randomized, etc.) or even randomly generated spanning trees (step 1). The work [48] suggests an initial population of size $2n$ which contains (i) two specific solutions, an MST of G and a spanning tree of diameter $D = 2$ or $D = 3$, if it applies; (ii) half of the population obtained randomly; and (iii) the remaining solutions computed using a randomized version of Prim's algorithm (i.e., the vertex entering the solution is randomly selected from a list of good candidates).

As mentioned above, the *ranking* and the *crowding distance* are computed for each solution in the initial population (step 2). Then, these operators will be responsible for the selection of solutions, following a multiobjective evolutionary approach rather than directly considering their cost and diameter. The comparison between two solutions $\mathcal{T}_1$ and $\mathcal{T}_2$ with respect to their *ranking* r_1 and r_2 and their *crowding distance* w_1 and w_2 is as follows:

$$(r_1 < r_2) \text{ or } ((r_1 = r_2) \text{ and } (w_1 > w_2)) \tag{12}$$

$\mathcal{T}_1$ is said to be better than $\mathcal{T}_2$ whenever the condition (12) holds. In the following (step 2), the population is ordered. Now, the algorithm proceeds to the selection (step 3) by keeping the n best solutions for the next iteration. The n new solutions to enter the population can be generated by the crossover proposed by [59] (step 3). One advantage of this crossover is that feasibility of new solution is ensured, even for sparse graphs. The way to select the parents in the crossover can follow a classical elitist strategy, i.e., one parent from the elite set and the other one randomly selected from the remaining population. The crossover proposed by [59] works as follows: given two solutions (parents) $\mathcal{T}_1$ and $\mathcal{T}_2$, a support graph G' containing all edges from $\mathcal{T}_1$ and $\mathcal{T}_2$ is built. Then, an MST is computed on G' to obtain the new solution. Figure 13 illustrates this crossover, considering the graph G shown

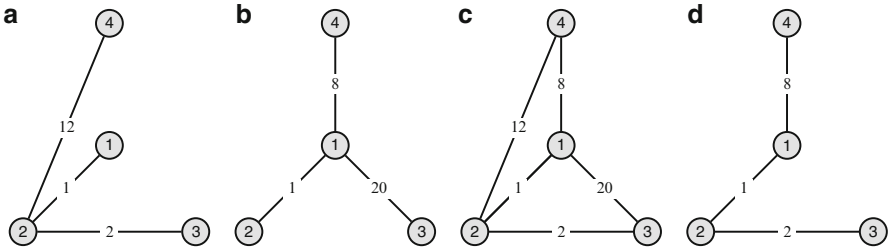


Fig. 13 Example of the crossover operator. (**a**) Solution $\mathcal{T}_1$, (**b**) Solution $\mathcal{T}_2$, (**c**) Graph G', and (**d**) MST of G'

in Fig. 11a. The two parents $\mathcal{T}_1$ and $\mathcal{T}_2$ are, respectively, given in Fig. 13a and b. The resulting support graph G' is presented in Fig. 13c and the new individual is provided in Fig. 13d, for which $Cost = 11$ and $D = 3$.

The local search iteratively performs a 2-opt move until no improvement can be done on the current solution. The 2-opt move uses the *edge drop* move previously presented. A specific feature of the 2-opt move for bi-MDCST suggested by [7] is that a move is accepted if and only if the diameter does not change and the cost is reduced. This means the local search will reach a local optima having the same diameter as the current solution.

The main insights obtained in [7, 56] by applying the NSGA-II for bi-MDSCT are summarized below. Three main sets of instances available in the literature for the BDMST were used in the experiments. They were proposed by [23, 26, 46], respectively. In addition, two test sets were proposed by [7] containing 9 and 11 instances, named Hamiltonian cycle and Hamiltonian path, respectively. Results produced by NSGA-II were compared with available optimal values for the BDMST from [22, 48]. The NSGA-II for bi-MDCST manages to find some optimal diameter values of the BDMST and it is very close in the other cases. The computational time does not exceed the computational time spent by dedicated and sophisticated methods for the BDMST. The Pareto-optimal fronts for the test sets used in [7, 23].

Some interesting characteristics have been observed: (i) a significant difference in tree cost occurs in the Pareto-optimal front between solutions with $D = 2$ and $D = 3$; (ii) since the instances from [23] have edges with similar cost, there are several MSTs with different diameters; (iii) some diameters are not interesting because the best corresponding solutions are dominated; and (iv) for the majority of instances from the sets mentioned above, up to 15 target diameters exist. The multiobjective approach has shown to be effective. It works consistently well and can provide useful information about the search space and the Pareto front for bi-MDCST problems.

Conclusion

Trees and forests are very rich topics, both in terms of theoretical and practical issues. Integrating good ingredients in sophisticated heuristics is the first step to produce good, competitive heuristics and even find new better results for NP-hard problems relying on these structures. In this context, some basic components for trees and forests are summarized in this chapter and two ways of addressing difficult constraints (degree and diameter) are presented, with the hope it will inspire new and fruitful research on these topics.

In terms of constructive heuristics, two basic ways have been addressed in the literature: tree expansion and tree fusion. Concerning the improvement heuristics, two main classes appear: restoration and local search. Some of these strategies use tree properties such as the local searches based on k-opt moves. In fact, by definition of a tree, removing an edge from a tree results in two connected components and inserting an edge in a tree obviously creates a cycle. Thus even an 1-opt can be considered two ways, either by first removing an edge or by first adding an edge. Even if they basically are 1-opt moves, these may impact the way the search space is explored.

Some key points are also provided in order to represent a tree in terms of data structures. Encoding and decoding a tree can be done directly by means of predecessor vectors indexed on the vertex or edge sets. It can also be done indirectly by using random keys. Additional data structures are also required to handle difficult constraints such as degrees and diameter. The choice of data structures will mainly impact the algorithm complexity. As a consequence, careful choices can save running time.

Two different strategies to address difficult constraints have been presented. The first, more classical, relies on a Lagrangian relaxation in which the constraints are relaxed and integrated into the objective function using the Lagrangian penalties. The second way consists of dropping the constraints, considering them as a new criterion. The resulting bi-objective problem is then solved by multiobjective approaches. The former presents the advantages of being simple to implement; it obtains solutions of good quality in a reasonable computational time, and lower and upper bounds are available at each iteration. A drawback is that it strongly depends on the mathematical formulation and the chosen decomposition. The latter is far less investigated in the literature, in particular for addressing difficult constraints. It has the following advantages: it provides additional information about the search space, and the computational time to obtain the Pareto front is very close to the time spent applying sophisticated dedicated methods. Moreover, limits in the search space can be computed and dominance rules can speed up the inspection of the solutions. A drawback is that the problem remains difficult since it may involve solving several NP-hard problems. However, facing NP-hard problems from a multiobjective perspective remains an interesting direction for further research.

Cross-References

- ▸ Evolutionary Algorithms
- ▸ Multiobjective Optimization
- ▸ Variable Neighborhood Descent
- ▸ Variable Neighborhood Search

Competing Interest Declaration The author(s) has no competing interests to declare that are relevant to the content of this manuscript.

References

1. Henschel R, Leal-Taixé L, Rosenhahn B (2014) Efficient multiple people tracking using minimum cost arborescences. In: Jiang X, Hornegger J, Koch R (eds) Pattern recognition lecture notes in computer science. Springer, Cham, pp 265–276
2. Santos AC, Duhamel C, Belisário LS, Guedes LM (2012) Strategies for designing energy-efficient clusters-based WSN topologies. J Heuristics 18(4):657–675
3. Li Y, Yu J, Tao D (2014) Genetic algorithm for spanning tree construction in P2P distributed interactive applications. Neurocomputing 140:185–192
4. Shangin RE, Pardalos PM (2014) Heuristics for minimum spanning *k*-tree problem. Procedia Comput Sci 31:1074–1083
5. Kasperski A, Zieliński P (2011) On the approximability of robust spanning tree problems. Theor Comput Sci 412(4–5):365–374
6. Silva RMA, Silva DM, Resende MGC, Mateus GR, Gonçalves JF, Festa P (2014) An edge-swap heuristic for generating spanning trees with minimum number of branch vertices. Optim Lett 8(4):1225–1243
7. Santos AC, Lima DR, Aloise DJ (2014) Modeling and solving the bi-objective minimum diameter-cost spanning tree problem. J Glob Optim 60(2):195–216
8. Sourd F, Spanjaard O (2008) A multiobjective branch-and-bound framework: application to the biobjective spanning tree problem. INFORMS J Comput 20(3):472–484
9. Cayley A (1889) A theorem on trees. Q J Pure Appl Math 23:376–378
10. Cormen TH, Leiserson CE, Rivest R, Stein C (2009) Introduction to algorithms, 3rd edn. The MIT Press, Cambridge
11. Camerini PM, Galbiati G, Maffioli F (1980) Complexity of spanning tree problems: part I. Eur J Oper Res 5(5):346–352
12. Garey MR, Johnson DS (1979) Computers and intractability: a guide to the theory of NP-Completeness. W.H. Freeman, New York
13. Ho J-M, Lee DT, Chang C-H, Wong K (1991) Minimum diameter spanning trees and related problems. SIAM J Comput 20(5):987–997
14. Magnanti TL, Wolsey LA (1995) Optimal trees. In: Monma CL, Ball MO, Magnanti TL, Nemhauser GL (eds) Network models. Volume 7 of handbooks in operations research and management science. Elsevier, Amsterdam, pp 503–615
15. Álvarez-Miranda E, Ljubić I, Toth P (2013) Exact approaches for solving robust prize-collecting Steiner tree problems. Eur J Oper Res 229(3):599–612
16. Gouveia L, Leitner M, Ljubić I (2012) On the hop constrained Steiner tree problem with multiple root nodes. In: Ridha Mahjoub A, Markakis V, Milis I, Paschos Vangelis T (eds) Combinatorial optimization. Volume 7422 of lecture notes in computer science. Springer, Berlin/Heidelberg, pp 201–212

17. Hwang FK, Richards DS, Winter P (1992) The Steiner tree problem. Annals of discrete mathematics, vol 53. Elsevier, Amsterdam
18. Leitner M, Ljubic I, Sinnl M (2015) A computational study of exact approaches for the bi-objective prize-collecting Steiner tree problem. INFORMS J Comput 27(1):118–134
19. Andrade R, Freitas AT (2013) Disjunctive combinatorial branch in a subgradient tree algorithm for the DCMST problem with VNS-Lagrangian bounds. Electron Notes Discrete Math 41:5–12
20. Cerrone C, Cerulli R, Raiconi A (2014) Relations, models and a memetic approach for three degree-dependent spanning tree problems. Eur J Oper Res 232(3):442–453
21. da Cunha AS, Lucena A (2007) Lower and upper bounds for the degree-constrained minimum spanning tree problem. Networks 50(1):55–66
22. Gouveia L, Simonetti L, Uchoa E (2011) Modeling hop-constrained and diameter-constrained minimum spanning tree problems as Steiner tree problems over layered graphs. Math Program 128:123–148
23. Lucena A, Ribeiro C, Santos AC (2010) A hybrid heuristic for the diameter constrained minimum spanning tree problem. J Glob Optim 46:363–381
24. Duhamel C, Gouveia L, Moura P, Souza M (2008) Models and heuristics for a minimum arborescence problem. Networks 51(1):34–47
25. Achuthan NR, Caccetta L, Caccetta PA, Geelen JF (1994) Computational methods for the diameter restricted minimum weight spanning tree problem. Australas J Comb 10:51–71
26. Raidl GR, Julstrom BA (2003) Greedy heuristics and an evolutionary algorithm for the bounded-diameter minimum spanning tree problem. In: Proceedings of the 18th ACM symposium on applied computing, Melbourne, pp 747–752
27. Kruskal JB (1956) On the shortest spanning subtree of a graph and the traveling salesman problem. Am Math Soc 7:48–50
28. Held M, Karp RM (1970) The travelling-salesman problem and minimum spanning trees. Oper Res 18:1138–1162
29. Held M, Karp RM (1971) The travelling-salesman problem and minimum spanning trees: part II. Math Program 1:6–25
30. Andrade R, Lucena A, Maculan N (2006) Using Lagrangian dual information to generate degree constrained spanning trees. Discret Appl Math 154(5):703–717
31. da Cunha AS, Lucena A, Maculan N, Resende MGC (2009) A relax-and-cut algorithm for the prize-collecting Steiner problem in graphs. Discret Appl Math 157(6):1198–1217
32. Martinez LC, da Cunha AS (2012) A parallel Lagrangian relaxation algorithm for the min-degree constrained minimum spanning tree problem. In: Mahjoub AR, Markakis V, Milis I, Paschos V (eds) Combinatorial optimization. Volume 7422 of lecture notes in computer science. Springer, Berlin/Heidelberg, pp 237–248
33. Fisher ML (1981) The Lagrangian relaxation method for solving integer programming problems. Manag Sci 27:1–18
34. Barahona F, Anbil R (2000) The volume algorithm: producing primal solutions with the subgradient method. Math Program 87:385–399
35. Beasley JE (1993) Lagrangean relaxation. In: Reeves C (ed) Modern heuristic techniques for combinatorial problems. Wiley, New York, pp 243–303
36. Guignard M (2003) Lagrangean relaxation. In: Cerdá MAL, Jurado IG (eds) Trabajos de Investigación Operativa, vol 11, chapter 2. Sociedad de Estadística e Investigatión Operativa, Madrid, pp 151–228
37. de Souza MC, Duhamel C, Ribeiro CC (2003) A GRASP heuristic for the capacitated minimum spanning tree problem using a memory-based local search strategy. In: Resende M, de Sousa J (eds) Metaheuristics: computer decision-making. Kluwer, Boston, pp 627–658
38. Ahuja RK, Magnanti TL, Orlin JB (1993) Network flows – theory, algorithms and applications. Prentice Hall, Upper Saddle River
39. Prüfer H (1918) Neuer beweis eines satzes über permutationen. Archiv für Mathematik und Physik 27:141–144

40. Gottlieb J, Julstrom BA, Raidl GR, Rothlauf F (2001) Prüfer numbers: a poor representation of spanning trees for evolutionary search. In: Spector L, Goodman ED et al (eds) Proceedings of the genetic and evolutionary computation conference (GECCO-2001). Morgan Kaufmann, San Francisco, pp 343–350
41. Bean JC (1994) Genetic algorithms and random keys for sequencing and optimization. ORSA J Comput 6(2):154–160
42. Raidl GR, Julstrom BA (2003) Edge sets: an effective evolutionary coding of spanning trees. IEEE Trans Evol Comput 7(3):225–239
43. Held MH, Wolfe P, Crowder HD (1974) Validation of subgradient optimization. Math Program 6:62–88
44. Souza MC, Martins P (2008) Skewed VNS enclosing second order algorithm for the degree constrained minimum spanning tree problem. Eur J Oper Res 191(3):677–690
45. Krishnamoorthy M, Ernst AT, Sharaiha YM (2001) Comparison of algorithms for the degree constrained minimum spanning tree. J Heuristics 7(6):587–611
46. Gouveia L, Magnanti TL (2003) Network flow models for designing diameter-constrained minimum-spanning and Steiner trees. Networks 41:159–173
47. Gruber M, Raidl GR (2005) A new 0-1 ILP approach for the bounded diameter minimum spanning tree problem. In: Hansen P, Mladenovic N, Pérez JAM, Batista BM, MorenoVega JM (eds) The 2nd international network optimization conference, Lisbon, vol 1, pp 178–185
48. Santos AC, Lucena A, Ribeiro CC (2004) Solving diameter constrained minimum spanning tree problem in dense graphs. Lect Notes Comput Sci 3059:458–467
49. Noronha TF, Ribeiro CC, Santos AC (2010) Solving diameter-constrained minimum spanning tree problems by constraint programming. Int Trans Oper Res 17(5):653–665
50. Noronha TF, Santos AC, Ribeiro CC (2008) Constraint programming for the diameter constrained minimum spanning tree problem. Electron Notes Discrete Math 30:93–98
51. Requejo C, Santos E (2009) Greedy heuristics for the diameter-constrained minimum spanning tree problem. J Math Sci 161:930–943
52. Ramos RM, Alonso S, Sicilia J, Gonzalez C (1998) The problem of the optimal biobjective spanning tree. Eur J Oper Res 111(3):617–628
53. Steiner S, Radzik T (2008) Computing all efficient solutions of the biobjective minimum spanning tree problem. Comput Oper Res 35(1):198–211
54. Arroyo JEC, Vieira PS, Vianna DS (2008) A GRASP algorithm for the multi-criteria minimum spanning tree problem. Ann Oper Res 159:125–133
55. Zhou G, Gen M (1999) Genetic algorithm approach on multi-criteria minimum spanning tree problem. Eur J Oper Res 114:141–152
56. de Sousa EG, Santos AC, Aloise DJ (2015) An exact method for solving the bi-objective minimum diameter-cost spanning tree problem. RAIRO Oper Rech 49:143–160
57. Saha S, Kumar R (2011) Bounded-diameter MST instances with hybridization of multi-objective ea. Int J Comput Appl 18(4):17–25
58. Deb K, Pratap A, Agarwal S, Meyarivan T (2002) A fast and elitist multiobjective genetic algorithm: NSGA-II. IEEE Trans Evol Comput 6(2):182–197
59. Carrano EG, Fonseca CM, Takahashi RHC, Pimenta LCA, Neto OM (2007) A preliminary comparison of tree encoding schemes for evolutionary algorithms. In: IEEE international conference on systems, man and cybernetics. ISIC, Montreal, pp 1969–1974

World's Best Universities and Personalized Rankings

61

Mario Inostroza-Ponta, Natalie Jane de Vries, and Pablo Moscato

Contents

Introduction 1822
Materials and Methods 1824
Construction of a Dominance Graph and Transitive Reduction 1826
Symbolic Regression Analysis to Predict Levels Using All Variables 1828
Results 1829
University Ranking Results 2007–2009 1830
Transitive Dominance Example Results 1833
Sensitivity Analysis of Mobility Over the Period 2010–2012 1834
Dominance Graph Computation of 2007–2012 with Symbolic Regression Analysis 1837
The Case of the Alumni Attribute 1845

Supplementary Information The online version contains supplementary material available at ▶ https://doi.org/10.1007/978-3-032-00385-0_60.

M. Inostroza-Ponta (✉)
Departamento de Ingeniería Informática, Universidad de Santiago, Santiago, Chile
e-mail: mario.inostroza@usach.cl

N. J. de Vries
School of Electrical Engineering and Computing, Faculty of Engineering and Built Environment, The University of Newcastle, Callaghan, NSW, Australia
e-mail: natalie.devries@newcastle.edu.au

P. Moscato
School of Information and Physical Sciences, University of Newcastle, Newcastle, NSW, Australia

School of Electrical Engineering and Computing, Faculty of Engineering and Built Environment, The University of Newcastle, Callaghan, NSW, Australia
e-mail: pablo.moscato@newcastle.edu.au

© Springer Nature Switzerland AG 2025

R. Martí et al. (eds.), *Handbook of Heuristics*,
https://doi.org/10.1007/978-3-032-00385-0_60

The Case of the Performance Per Capita Attribute (Size) 1847
Discussion 1849
A Transparent and User-Centric Ranking Method 1849
Implications for Research Policy Makers 1850
Overview of Existing Multi-objective Ranking Methods 1851
Conclusion 1852
Cross-References 1853
References 1853

Abstract

This chapter presents a heuristic for a multi-objective ranking problem using a dataset of international interest as an example of its application, namely, the ranking of the world's top educational institutions. The problem of ranking academic institutions is a subject of keen interest for administrators, consumers, and research policy makers. From a mathematical perspective, the proposed heuristic addresses the need for more transparent models and associated methods related to the problem of identifying sound relative rankings of objects with multiple attributes. The low complexity of the method allows software implementations that scale well for thousands of objects as well as permitting reasonable visualization. It is shown that a simple and multi-objective-aware ranking system can easily be implemented, which naturally leads to intuitive research policies resulting from varying scenarios presented within. The only assumption that this method relies on is the ability to sort the candidate objects according to each given attribute. Thus the attributes could be numerical or ordinal in nature. This helps to avoid the selection of an ad hoc single score based on an arbitrary assignment of attributes' weights as other heuristics do. To illustrate the use of this proposed methodology, results are presented and obtained using the dataset on the ranking of world universities (of the years 2007–2012), by academic performance, published annually by ARWU.

Keywords

Analytics · Digital humanities · Pareto optimality · Ranking · Symbolic regression

Introduction

Ranking multi-attributed objects is ubiquitous in the twenty-first century. This is not an overstatement. Every time people use a web search engine and query it using a particular keyword or phrase, a ranked list of webpages appears on our screen. This ranking is automatic, in most cases deterministic, and is well defined in mathematical terms. Not only the speed but the quality of the ranking may decide the fate of the company that has created that search engine. Every time people read a newspaper, or watch television, the news stories provided have been edited, and

the core subject has been ranked and selected. Today, our governments want to decide on spending based on the objective rankings of institutions, their quality, performance, and delivery of service.

Specifically, the ranking of educational institutions has received increasing interest from scholars, policy developers, and strategic decision-makers worldwide. Our society, regardless of nationality, enjoys (and in some cases needs) to rank tennis players, sport teams, health systems, supercomputers, economical performance, scientists' achievements, most and least "liveable" countries and cities, restaurants, waiting lists for surgery and transplantation [1], targets in structure-based virtual screening of three-dimensional protein libraries [2], genes in microarray experiments [3, 4] or in association studies [5], athletic training education programs [6], pedestrian crash zones [7], stressfulness of joints and joint motions in ergonomics applications [8], movie stars, dental esthetics [9]), and even US presidents.

The academic ranking of world universities follows the general trend of globalization of the economy. Researchers agree that ranking global universities is attracting increasing attention as it is a topic of high interest to many different stakeholders; see, for instance, [10–13], among others. If you look at universities as service providers and if their potential students are free to choose among a variety of them, it is natural to pay attention to rankings to help them select the best tailored to their interests. This has become of even greater importance in recent years due to the increased mobility of students, researchers, and staff [14]. Furthermore, strategic decision-making within institutions, and policy development at a larger scale, relies on the provision of useful information regarding an institution's quality, excellence, and global ranking position.

Needless to say, ranking global universities is a quest of great impact and social significance [15]; however, the methodologies proposed are naive at best. It could be said that the problem is inherently *ill posed*. First, outcomes of ranking activities should be user centric. A student who has to choose where to go for graduate school studies may be more interested in the quality of their specific disciplines that constitute the core set of knowledge she is looking for and may dismiss an institution that has Nobel Prize winners lecturing using chalk and a blackboard (see [16] for a discussion on how to define quality). Managers and strategic decision-makers need to have access to information about how their institution benchmarks against their "competitors" in order to successfully and optimally allocate their resources and funds. Furthermore, it is important to remember that different stakeholders have different expectations about quality [11] and different expectations of an educational institution in general. In essence, it is important to highlight that ranking systems should be developed allowing the various users of these systems to easily add or remove an attribute of the objects in order to suit their specific needs and interests.

The request of [17] *"...there is still a dearth of peer-reviewed scientific publications on international ranking methods. Raw data and several key details about the methodology still remain unavailable to public scrutiny."* [17] easily resonates with this study. In addition to this, it recognized that the trustworthiness of many ranking systems is sensitive to the conceptual framework (its indicators) as well as the modeling choices such as weighting of indicators [14]. In order to fill this methodological gap, the issue of academic rankings of universities is addressed as an

illustrative test case. This chapter thus aims to differ from Ioannidis et al. who have well articulated their case for the importance of a common international consensus on the validity of the measured attributes for each institution. The methodology presented here moves away from the traditional comparison of one-dimensional lists between ranked objects.

In this chapter other metrics that can be computed are included as well as a very simple heuristic that, using raw data, allows users to select/deselect attributes to better target their interests. This approach can "tailor" a ranking system according to the user's individual preferences. Therefore, this system naturally provides a user-centric experience, while it is still based on quantitative nonsubjective information. This said, while this chapter is focussed on the largest online dataset of universities' performance, its final aim is to exemplify on its use to address the multi-objective ranking problem from a general methodological standpoint. In this method, the issue of sensitivity due to arbitrarily assigning of weights to indicators is addressed and the problems that arise in doing so, rather than using raw data.

The chapter is organized as follows. Section "Materials and Methods" illustrates where the data was obtained from, a brief description of the data, and an outline of the methodology used. Section "Results" has several subsections in order to display the results. Results for the computation of the dominance graph for the period 2007–2009 are displayed in section "University Ranking Results 2007–2009" followed by results describing transitive dominance as well as the implementation of symbolic regression modeling (section "Transitive Dominance Example Results"), mobility of universities in ranking (section "Sensitivity Analysis of Mobility over the Period 2010–2012"), and case examples of various universities, and, finally, a brief discussion on the attribute *Alumni* is included in section "The Case of the Alumni Attribute". Section "Discussion" provides a discussion of the results and includes the limitations thereof and recommendations for possible future areas of research.

Materials and Methods

To illustrate the proposed analysis methods for ranking, and to ensure reproducibility and interpretability of the results, the online publicly available information provided by the Academic Ranking of World Universities (ARWU) is used in this study. This is an annual publication and the data that is used for their ranking is provided by the Institute for Higher Education, Shanghai Jiao Tong University, since 2003.

The ARWU from the Shanghai Jiao Tong University is based on the performance of universities according to six categories (attributes). These attributes are displayed in Table 1 with their respective weighting given by the Shanghai Jiao Tong University and their explanations. The six attributes are *Alumni*, *Award*, *HiCi*, *N&S*, *SCI* (or PUB), and *Size* (or PCP). It is important to note that for institutions specializing in humanities and the social sciences, such as the London School of Economics, in the ARWU ranking systems, the attribute of N&S is not considered, and weighting is distributed to other indicators [11]. This borderline between

Table 1 Explanation of the institute for higher education, Shanghai Jiao Tong University ranking system. Note that the attribute "Size" is also referred to as "Per Capita Performance" (PCP) and the attribute "SCI" is also referred to as "PUB" in various articles and resources

Attribute	Weighting	Explanation
Alumni	10%	Number of alumni of the institution winning Nobel Prizes and Fields Medals
Award	20%	Number of staff of the institution winning Nobel Prize and Field Medals categories
HiCi	20%	Number of highly cited researchers in 21 broad subjects
N&S	20%	Number of articles published in Nature or Science
SCI	20%	Articles in Science Citation Index Expanded, Social Science Citation Index, and Arts and Humanities Citation Index
Size	10%	Academic performance with respect to the size of the institution

institutions, however, is somewhat arbitrary. Therefore, it has to be recognized that for some institutions only four attributes (with arbitrary weighting) remain for consideration and that is a natural aspect of its organization on a reduced number of fields. In each category a university will receive a value between 0 and 100. The university with the best performance in a particular category receives the top value of a 100, and the rest of the universities will receive a proportional value against this measure. Further details and explanation of these attributes and the scoring procedure can be found on the Shangai Jiao Tong University ARWU website (http://www.shanghairanking.com) or in Liu et al. [11].

The raw data of this ranking process for the years 2007–2012 is used. From the top 500 universities in each year, only those that are present in all years since 2007 are included. This means that in total, the scores of 444 universities are used in the method. With this information, three datasets for analysis are created; one for the years 2007–2009, another for 2010–2012, and a combined dataset including the attributes over the whole six-year period. The first two are an array U of 444 rows and 18 columns, and each value in the array represents the score that the institution has in each of the six categories in the three years. The combined array has, of course, 36 columns.

The purpose of this chapter is not to argue the accurateness of this information, but rather utilize this data for an application of the method proposed. The reader should recognize that the use of this dataset has been challenged in terms of its reproducibility by Razvan [18] and also recognize the work by Docampo [19] who also alert of some problems in the ARWU's processes. However, as highlighted by many authors, after recognition of these issues and also noting the criticisms revolving about the ARWU ranking, this contribution addresses two of them. In particular, in section "Results", the *Alumni* attribute and the attribute related to *Size* (PCP) deserve two separate experiments to understand their contribution. On this point, it can be argued that, to ensure transparency, an international standard on the universities reporting on performance on specific attributes should be made available and that it is best, for reproducibility, to work with raw data. This said, this information is taken bona fide, as correct and complete.

From this set of attributes, other types of ranking systems could create other attributes or "meta-features" by combining or multiplying multiple attributes of the dataset. Typically, aggregates should be avoided whenever possible, and normalizations (e.g., dividing total number of publications by the total number of full-time employees) can easily be derived should this be required or desired by the user. Having said this, these standards may not make the dataset used in this study fully satisfactory (aggregates exist, normalization by full-time employees equivalents are not made, other problems have been identified in Ioannidis et al. [17]), but this dataset is one of the world's best, regarding its coverage, and serves to illustrate our proposed methodology.

Construction of a Dominance Graph and Transitive Reduction

The method proposed in this chapter uses concepts inspired by transitive dominance to build a directed acyclic graph (DAG) ($G(V, A)$). A university x dominates university y, if and only if, for each attribute i the values of the universities for this attribute satisfy $x_i \geq y_i$, $\forall\, i : [1..m] \wedge \exists\, j \in [1..m] \,/\, x_j > y_j$. This means that the performance of university x is at least better or equal in all attributes than university y, with at least one attribute at having a higher value. This definition creates a DAG $G(V, A)$ where each vertex represents a university, and if there is an arc $a_{xy} \in A$, it means that university x dominates university y. Within this chapter it will be shown that this DAG naturally induces a hierarchy of universities, with universities at Level 1 being those that are *not* dominated by any other university in the set. Here it is implied that the notation $x_i \geq y_i$ is general in meaning and is valid for both numerical and ordinal attributes. It is assumed that the attributes' measures are selected in such a way that a higher value on them implies a higher standing. This is a strength of this method as it can deal with datasets of mixed attribute types.

The resulting graph of this procedure would be hard to visualize in two dimensions as there are 444 nodes and it is very dense. Therefore, in order to simplify the DAG, without losing the hierarchy created, a method similar to, but less restrictive than, *transitive reduction* [20] is used. As Bang-Jensen and Gutin [20] describe, a transitive reduction of a directed graph (digraph) D is a spanning subdigraph H of D with no *transitively irrelevant* arcs, and it is called *minimum equivalent subdigraph*. In transitive reduction, "reachability" between nodes is not affected as only superfluous arcs are removed [21]. The minimum equivalent subdigraph of an acyclic digraph can be found in polynomial time and is unique.

However, in this instance, another algorithm that reduces the number of arcs further is used so that it is less restrictive in maintaining the reachability condition. A simplified visualization of this procedure is presented in Fig. 1 and is mathematically outlined here:

1. Identify the set of all universities which are nondominated; this set will be called the set of "Level 1" universities;

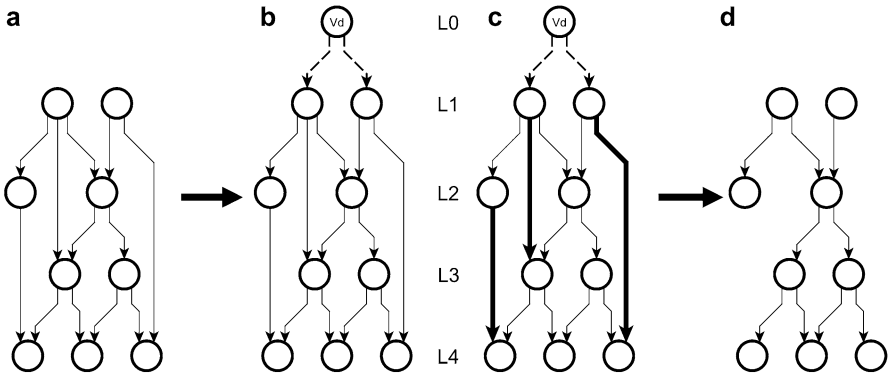


Fig. 1 Construction of the dominance graph. Each vertex represents a university, and there is an arc from each pair of universities (x, y) whenever x dominates y, according to the dominance definition given in the text. This method starts by first identifying the set of universities that are not dominated by any other university in the set. To illustrate on the algorithm, a set of figures are produced. Figure **(a)** shows the initial status. By assigning the nondominated set of universities to a top level, it would be possible to identify levels for all the other universities in the set. Figures **(b)**–**(d)** illustrate on the procedure to compute this and the resulting reduced graph as some arcs are removed. In **(b)** a dummy vertex v_d is added, which corresponds to a hypothetical "dream university" which is nondominated, which would be at "Level 0." From the dummy vertex to each of the nondominated universities, an arc is added (as shown in **(b)**). Then a level of a university is recursively defined as the maximum number of arcs that need to be transversed from v_d to it. Figure **(c)** shows arcs that connect universities for which the difference levels are more than one. These arcs are shown in boldface. Since visualizing such a dense directed graph would be unclear, these are removed as it would not affect the level of any given university. As a result, a reduced DAG is produced as shown in **(d)**, and through visualization it is shown how its results now uncover the underlying layer structure of **(a)**

2. Add a dummy vertex v_d, which would correspond to a hypothetical nondominated university by all universities in the set (the Level 0 or "dream university"), and draw an arc from v_d to each of the universities in Level 1 (Fig. 1b);
3. The level of a university (represented by a vertex v_i) is then equal to the number of arcs that need to be traversed in the longest directed path from the "dream university" (vertex v_d) to v_i;
4. For each arc $a_{xy} \in A$ if $|Level(x) - Level(y)| > 1$, then $A = A - \{a_{xy}\}$ (see, for instance, the bold arcs in Fig. 1c are deleted from the DAG Fig. 1d).

The resulting graph from this process has the same hierarchical properties as the original one. Based on this final reduced graph, university x *"closely dominates"* university y if $a_{xy} \in A$ after the reduction.

This heuristic ranking methodology in terms of levels comes from a clear algorithmic procedure and is nonparametric, and it does not require an ad hoc definition of weights to be applied to each attribute. It is based on established grounds in multi-objective optimization and clearly reflects all pairwise comparable

dominance relationships in the data. Furthermore, this method accepts numerical as well as categorical attributes allowing the inclusion of more variables in ranking activities.

Following the above process, in order to investigate "global" dominance relationships, the number of institutions that each university is transitively dominated by in the reduced graph will be calculated. This will provide an overall picture of where each institution is placed in comparison to all institutions included in the set. As an extra decision-making tool, it will provide institution leaders with the necessary information to compare and benchmark against other similar institutions. Examples of this are provided in section "Results".

Symbolic Regression Analysis to Predict Levels Using All Variables

In order to go one step further and investigate which variables are important in terms of "predicting" which level (rank) a university is going to be a part of, symbolic regression analysis is used in this chapter. By doing this, it clarifies whether the results found are consistent when using variables of each year for six consecutive iterations of the symbolic regression analysis. Symbolic regression is a data-driven method to find a structure in data. Unlike linear regression methods, symbolic regression not only finds the best values for a set of coefficients, it also finds the structure of the model that best fits the target variable [22] (including but not necessarily limited to linear models). In order to do this, a powerful software named *Eureqa* [23] is used which is based on evolutionary computation techniques to search for the best model. Eureqa runs an evolutionary search procedure to find the solution that fits the data best with the lowest possible level of complexity as is consistent with other symbolic regression methods [24].

In Eureqa, the "best models" are those that are observed in a Pareto optimality curve which imply a trade-off between their complexity and their fit. Complexity meaning the number of functions and "building blocks" used by *Eureqa* to fit a model to the target variable. The user selects a fitness function (for instance, absolute error or mean squared error) that guides Eureqa in selecting the best models. Users also have the option to select their preferred "building blocks" (for instance, arithmetic operations like multiplication, subtraction, and addition or the introduction of a constant). In a previous publication, the authors of Ref. [25] have shown the usefulness of symbolic regression and *Eureqa* in finding "functional constructs" from online consumer behavior data. Furthermore, the data-driven nature of symbolic regression is suitable to the context of this study as the aim is to make inferences on the data based on the information present in the data alone without a priori parameters or attribute weights.

In this study in particular, the aim is to find those variables that are of importance when "predicting" which level an institution is in as well as finding out whether the assignment to levels of all universities can be consistently predicted with variables from each of the years. To do this, the "level" is set as the target variable in *Eureqa*,

while it tries to fit a model to predict between Levels 1–11. The level ranking dataset used for this procedure is the ranking obtain from running the dominance graph on the whole dataset, i.e., years 2007–2012. Following this, six searches are conducted in *Eureqa* in which every search uses only one year of data. The reason is to find out whether the models are consistent in predicting one outcome of the dominance graph using variables from several years. The variables in the dataset that are most frequently used in the models are highlighted, and these are identified as the key variables in predicting "level membership." Furthermore, the best simple linear function found by *Eureqa* is used for further analysis which is consistent with a previous publication using this software [25]. In this manner it is possible to inspect the variables that are found to predict the target variable, in this case, ranking level in detail.

In this study, the following "building blocks" were selected for the *Eureqa* analysis, the use of a "constant," "integer constant," "input variable," "addition," "subtraction," and "multiplication." For all of the six years, the six searches are run two times, once with the error metric as *R-squared goodness of fit* and once with *squared error Akaike information criterion* [AIC]. The *squared error* with AIC is selected as AIC provides a means for model selection. AIC deals with the *goodness of fit* of the model and the complexity of the model which is in line with the other selection criteria that are used based on the Pareto optimality curve.

Results

The results based on the construction of the directed dominance graphs are presented here. The total dataset for the years of 2007–2012 was analyzed and split into two datasets representing the periods 2007–2009 and 2010–2012. In Figs. 2 and 3 the hierarchical layout of the graph created for the three-year periods of 2007–2009 and 2010–2012, respectively, can be seen.

Resulting from the ranking procedure, as outlined in the methodology, the graph of 2007–2009 has 12 levels with a different number of universities in each level in the first period. In the second period, the graph has 13 levels, with Level 13 only containing one university (University of Jyvaskyla, Finland). Furthermore, colors are used to highlight the correlation with the Shanghai Jiao Tong University study. Red corresponds to those institutions that have been considered at the top 10 according to the Shanghai Jiao Tong University study (in the last year included) with the remaining top 50 in orange, and the remaining top 100 are colored in green. Also included in this section are the results of the calculation of transitive dominance of institutions, the symbolic regression analysis testing, the volatility of ranking over a longer period of time, and the removal of the attribute Alumni as well as the attribute PCP (*Size*) in the ranking method and the effects of the removal of each of these attributes on the outcome. In the representation of results, various institutions are highlighted as examples in order to communicate the findings of this study.

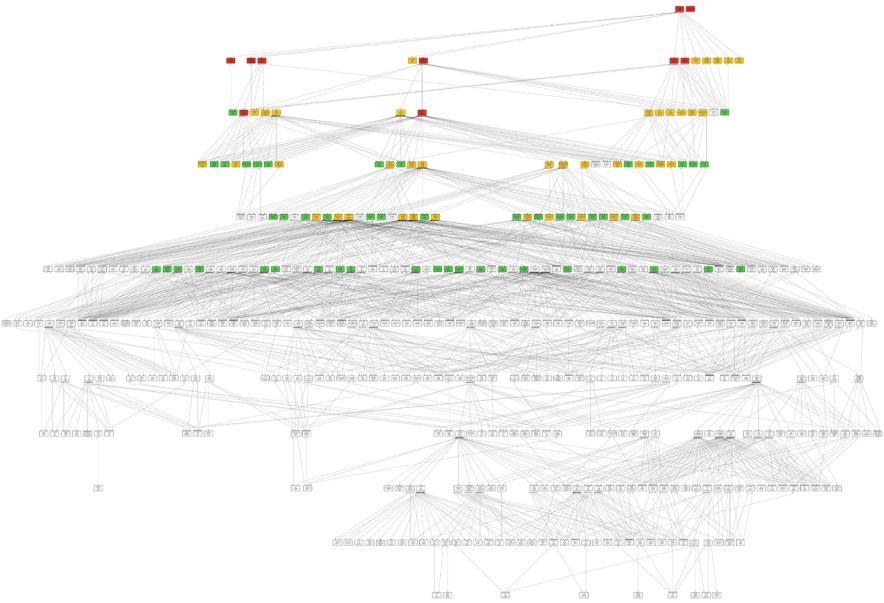


Fig. 2 The rankingof 444 world universities according to our proposed methodology using the same raw data provided by the Institute of Higher Education, Shanghai Jiao Tong University, for the years 2007, 2008, and 2009. Universities are arranged in *levels*, and a university is in a level if there exists another university that *closely dominates* it and is in the immediately higher level (the mathematical definition of this concept is outlined in the Methods Section). This graph provides important information for decision-making. In order to improve the overall ranking level, the university strategic decision-makers should pay attention to the best effective way of reaching the next level by improving in those attributes that would allow it to rank in the next level in the hierarchy. In addition, the university should also closely monitor the set of universities in the inferior level as they are the in direct competition for a position at the same level in the hierarchy. For instance, Yale University and Cornell University are two Level 3 universities that are both closely dominated by University of California at Berkeley. At the same time, these two institutions have a large number of institutions that they closely dominate at Level 4 (NB: A high-resolution version of this image can be found in the Supplementary material)

University Ranking Results 2007–2009

When looking at Level 1 in Fig. 2, some information that is clearly revealed by the method becomes apparent. The "Level 1" universities correspond to those which are not dominated by any other university in terms of the attributes considered in the method. In this graph, these are Harvard University and the California Institute of Technology (Caltech). While Harvard closely dominates several others, Caltech does not closely dominate any other institution of Level 2. Level 2 institutions are Stanford, Columbia University, the University of Tokyo, the University of California at Los Angeles, the University of Michigan at Ann Arbor, the University of Pennsylvania, the University of Washington, the University of California at

Berkeley, the University of Toronto, the University of Cambridge, Massachusetts Institute of Technology, and Princeton University.

The reason for Caltech's position in Level 1 can be explained not by Caltech's total productivity but rather its productivity regarding its size. Here it is noted that instead of normalizing raw data on the institution's performance by the number of full-time academics (or their equivalent full-time aggregated data), the Institute of Higher Education, Shanghai Jiao Tong University proposed the addition of an attribute called *"Academic performance with respect to the size of an institution"* or "Performance Per Capita" (PCP). It is important to remark that Caltech is a special case in terms of size. As pointed out by Baty in early 2014 [26], Caltech may be called not just "small," but *"tiny"* with its 300 professorial faculty, about 600 research scholars, and, at the last count, only 1204 graduate students and just 977 undergraduates. This translates to an almost 3:1 student to faculty ratio. As a result of the introduction of the "Performance Per Capita" (PCP) attribute, which relates to the size of the institution in the ranking method, Caltech (with its "tiny" size) is "singled out" at the top due to its maximum value of this attribute without any close domination of Level 2 institutions. This reflects the consensus that the lack of normalization of the other attributes would bias ranking systems to benefit larger institutions. For instance, Rockefeller University is another small-sized institution that is very productive and is at Level 3, closely dominated by the Massachusetts Institute of Technology at Level 2.

Other Level 3 institutions include the Ecole Normale Superioure of Paris, the University of Chicago, Cornell University, Yale University, the University of Oxford, the University of California of San Diego, Kyoto University, Duke University, John Hopkins University, the University of Minnesota at Twin Cities, the University of Wisconsin at Madison, the University of Sao Paulo, and the University of Pittsburg. The University of Sao Paulo (Brazil) consolidates its position as a world Level 3 institution due to its relatively large SCI performance score.

At Level 4, the international mix of institutions is more clear. Level 4 includes the Hebrew University of Jerusalem (Israel), two German universities (the Technical University Munich and the University of Munich), three UK institutions (the Imperial College of Science, Technology, and Medicine, London School of Hygiene and Tropical Medicine, and the University College London), three from France (Pierre and Marie Curie University Paris, the University of Paris-Sud, and the University of Montpellier), the University of Oslo (Norway), the University of Basel (Switzerland), Moscow State University (Russia), the University of British Columbia (Canada), Osaka University (Japan), and a large group of USA-based universities, Carnegie Mellon University, the University of Colorado at Boulder, the University of California San Francisco, Pennsylvania State University—University Park, the University of Illinois at Urbana-Champaign, the University of Texas Southwestern Medical Center at Dallas, Northwestern University, the University of California Davis, The Ohio State University Columbus, Case Western Reserve University, the University of Florida, Washington University in St. Louis, and the University of Rochester.

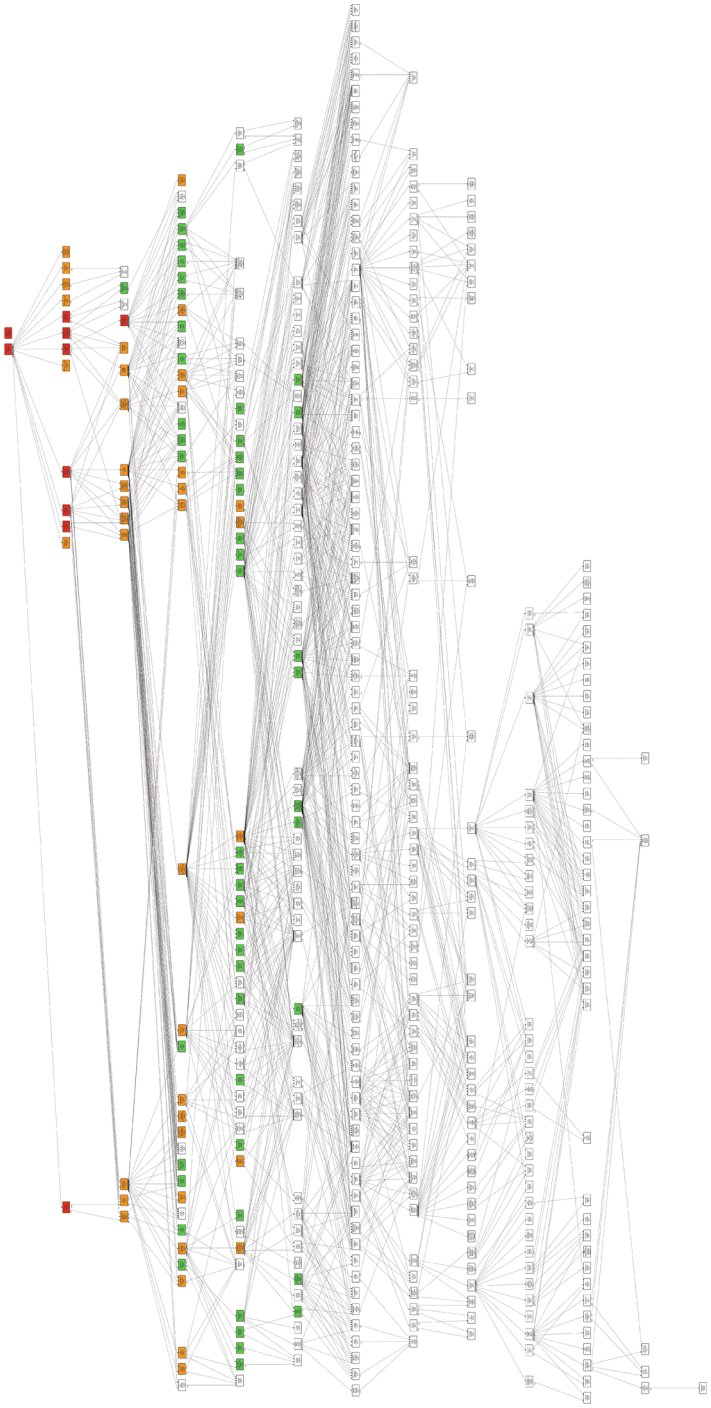


Fig. 3 The rankingof 444 world universities according to our proposed methodology using the same raw data provided by the Institute of Higher Education, Shanghai Jiao Tong University, for the years 2010, 2011, and 2012. The California Institute of Technology and Harvard University continue to be the single institutions at Level 1. At Level 2, there is almost the same group of American institutions (and The Universities of Tokyo and Cambridge).

Lastly, the Ecole Polytechnique Federale de Lausanne (EPFL) Institute in Switzerland is also situated on Level 4. This institution is highlighted because Ioannidis et al. in [17] have pointed out how the EPFL of Lausanne was a missing institution in the top lists of Shanghai Jiao Tong University ranking system in previous years, yet the methodology in this chapter has the EPFL institution in a relatively high position. The EPFL is located at Level 4 as shown in Fig. 2. Furthermore, as is to be expected, as you move down the levels further, the number of institutions in each level increases, and the variety of origins becomes more apparent which can be examined in Fig. 2. A full presentation of institutions at all remaining levels can be found in Fig. 2.

Transitive Dominance Example Results

As stated, the number of institutions that each university is transitively dominated by and which ones it dominates has been computed, for instance, looking at the three UK-based institutions at Level 4, the Imperial College of Science, Technology, and Medicine, London School of Hygiene and Tropical Medicine, and the University College London. The Imperial College is transitively dominated by only seven universities in the world, while it transitively dominates 337 institutions. This is an impressive number for a Level 4 university, very close to the 346 institutions dominated by Caltech at Level 1. The University College London (UCL) numbers are equally impressive with only five institutions that transitively dominate it and with 377 transitively dominated by UCL. In contrast, the London School of Hygiene and Tropical Medicine is transitively dominated by only 12 institutions (consistent with the overall trend of expected dominance for an institution at this high level), but it does not dominate any other institution.

The same numbers of the three French institutions at Level 4 are Pierre and Marie Curie University Paris (9 and 209), the University of Paris-Sud (9 and 117), and the

Fig. 3 (continued) However, the University of Pennsylvania now is at Level 3, and John Hopkins University and the University of Oxford now climb a level and now are at Level 2. If a single city of interest is to be highlighted, the universities in Hong Kong show a clear upward trend with the City University of Hong Kong climbing (climbing from Level 10 to Level 7), the Chinese University of Hong Kong and the University of Hong Kong (both raised from Level 8 to Level 6), and the Hong Kong Polytechnic University (from Level 10 to Level 8). Other Asian institutions raised three levels (Sun Yat-sen University and Hanyang University, from a Level 11 to Level 8). In Europe the remarkable uphill move of the Italy's Scuola Normale Superiore di Pisa that raised from Level 6 to Level 3 can be noted, as well as Sweden's Stockholm School of Economics (from Level 7 to Level 4) and Austria's Medical University of Innsbruck that from the bottom Level 12 is now at Level 7. However, over the three-year subsequent period, only 10% of the institutions raised two or more levels (and only 4.05% decreased it by a similar gap) indicating that as a metric is less volatile than other metrics to evaluate institutional performance (NB: A high-resolution version of this image can be found in the Supplementary material)

University of Montpellier 2 (13 and 45). This analysis provides some clarity on the peculiarities of some small-sized but intensively research-oriented institutions.

Sensitivity Analysis of Mobility Over the Period 2010–2012

Research policy makers can intuitively evaluate the chances that a university can climb the level structure by way of improving in different attributes. A university can climb up one level only if it improves on those categories in which it is being closely dominated by other institutions (given that those institutions do not move themselves). The inbound degree and the "weight" of each arc implicitly convey how likely it is for a university to improve its level. It is intuitive that the lower the inbound degree and the lower the weights, the easier it is for a university to climb levels up in the overall ranking.

There is a clear linear correlation trend between changes in levels and changes in the transitive dominance of a given university. In order to inspect this, a comparison is made between the change in level position of the 2010–2012 dominance graph and the 2007–2009 dominance graph. Firstly, several universities are highlighted as examples of level changes, followed by universities' changes in level over these two periods when the attribute PCP is removed from the dominance graph computation. This will show those changes that occur based on only the other attributes that remain.

Figure 4 shows five universities that provide different examples of changes in levels. Firstly, in Fig. 4a, the Netherland's Radboud University Nijmegen is shown. In the Figure it is noticeable that the attribute "Alumni" is nonexistent in the years 2007–2010 and appears for the years 2011–2012. For the case of the Radboud University Nijmegen, this steep increase is due to the fact that in 2010, two researchers affiliated to the University obtained the Nobel Prize in Physics for their research in the properties of graphene [27]. Examining these types of changes further, results show that the Radboud University Nijmegen shows an impressive improvement (from 58 institutions dominating it to only 12). Next in Fig. 4b, the USA-based Thomas Jefferson University attributes are shown which explain its level improvement to Level 6 from a previous Level 10 position and has only 41 universities dominating it from a previous 162 total. The Figure shows that between the period 2007–2009 and 2010–2012, the Thomas Jefferson University increased significantly in the attribute related to size (PCP). The Medical University of Innsbruck that was at the lowest level in 2007–2009 (Level 12) jumped to Level 7 in 2010–2012 also experienced the largest change in transitive dominance (from 222 to 88) and is shown in Fig. 4c. This is a change of approximately 60% in the number of institutions that dominate it. In this figure it shows that the Medical University of Innsbruck has a steep increase in the value for the attribute related to size. As stated, this attribute measures "Per Capita Performance" (PCP) in terms of research. Therefore, it could be that either the University reduced in size at this point in time or that it has significantly increased their research output with the same institutional size or even that some part of its research output was not previously included.

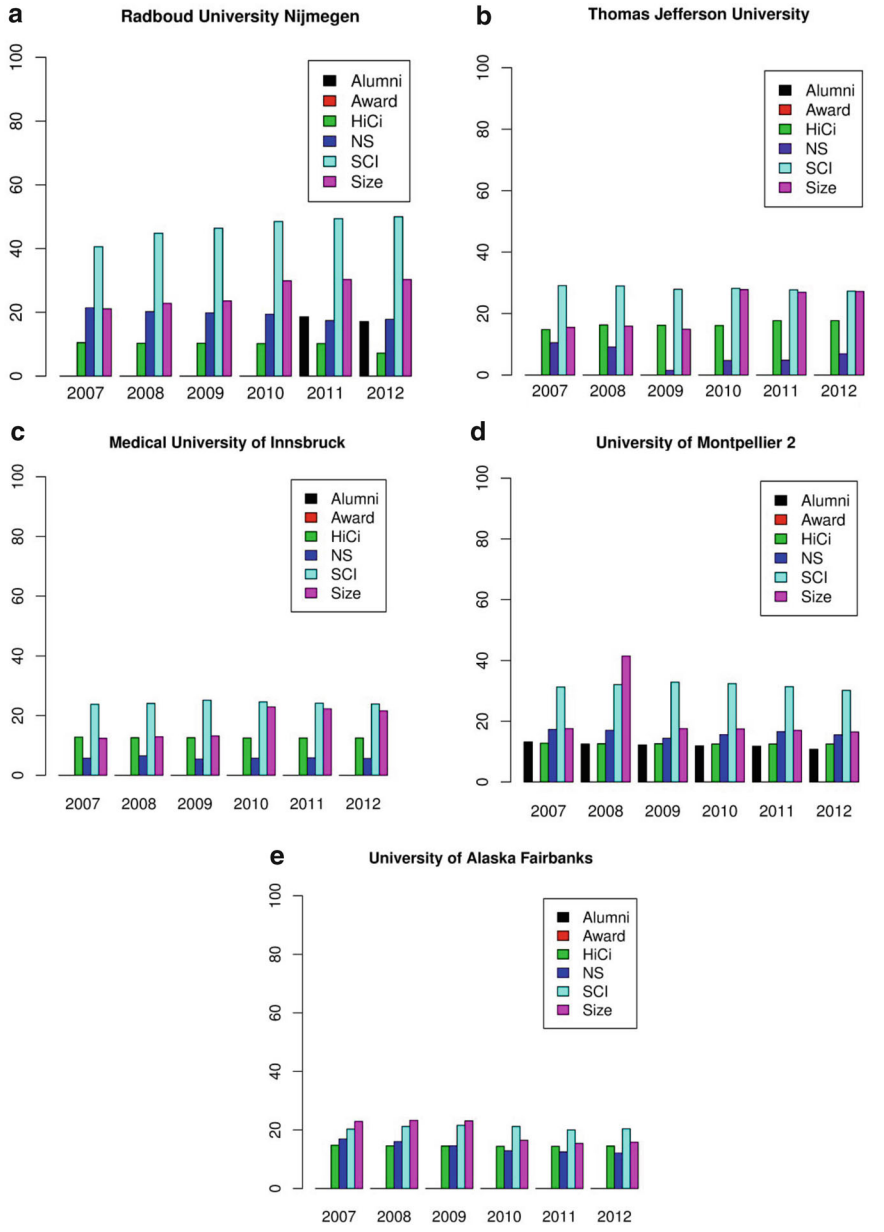


Fig. 4 Attribute values of universities that have changed levels over time—Figure **(a)** shows Radboud University which has climbed levels due to their increase in Alumni score. In figure **(b)**, Thomas Jefferson University provides an example of a university that has improved its level position due to an increase in Performance Per Capita (PCP), likely due to the institution

Furthermore, Fig. 4d shows the French University of Montpellier 2 and its peculiar values for the attribute "Size" (PCP). When comparing the two periods of time, the University of Montpellier 2 experiences a large drop in their level position from Level 4 to Level 8. In the figure it is evident that the University of Montpellier 2 had a steep decline in their score for the attribute "Size" (PCP). It may be argued that this could be due to an abnormality in one of the values of "PCP" (for the year 2008) which is conspicuously high. As a consequence, this may have affected the position of this university in the level for years 2010–2012. Finally, Fig. 4e shows the University of Alaska Fairbanks. This university is also an example of a university falling in its level position. The University of Alaska at Fairbanks is now at Level 10 from a previous position at Level 7. In the figure it can be seen that this drop is explained by the decrease in the attribute of N&S as well as a large decline in PCP.

Considering that many universities' drop in level can be attributed to their change in PCP, the results of the two periods without the PCP variable are compared. This will show those universities that still change in level position without considering their Performance Per Capita and may shed light on which other attributes are important to level position changes. Figure 5 shows four universities which provide examples of this. In Fig. 5a, the UK-based University of Bath is shown. The University of Bath declined in its position over the years due to a decrease in the value for N&S attribute. The USA-based Northeastern University and the Australian University of Tasmania are shown in Fig. 5b, c, respectively, and show the reason for their level improvement. For both of these universities, their level improvement can be attributed to their increase in value for the N&S attribute in the new three-year period. This means that it is likely that each of these institutions shifted resources, and academics allocated their time differently in order to publish more in the journals Nature and Science in the second period (2010–2012) than in the first period (2007–2009). Lastly, Fig. 5d shows the City University of Hong Kong which experienced a drop in their position due to the decline in citations in N&S. Interestingly, although their value for "HiCi" improves in the newest three-year period, the drop in N&S is enough to shift this university's level position.

A complete list of the results of universities' mobility over the two time periods, for those universities that fell or increased more than two levels, is shown in Table 2.

Fig. 4 (continued) changing in size. The Medical University of Innsbruck also provides an interesting example in figure **(c)** as it has the largest climb in level (from Level 12 to Level 7). It is possible to see for this university that all the categories are kept mostly the same, with the exception of category PCP, related to size. Figure **(d)** shows the university with the largest change in its level, University of Montpellier 2, which dropped from Level 4 to Level 8 when comparing the two periods that are investigated in this study. The profile of the university shows an unusual value for the PCP category in year 2008. This is probably due to an error in the data collected or published. Finally, in figure **(e)**, the University of Alaska Fairbanks drops from Level 7 to 10 which is explained by the drop in the category N&S, as well as the drop in PCP (Size)

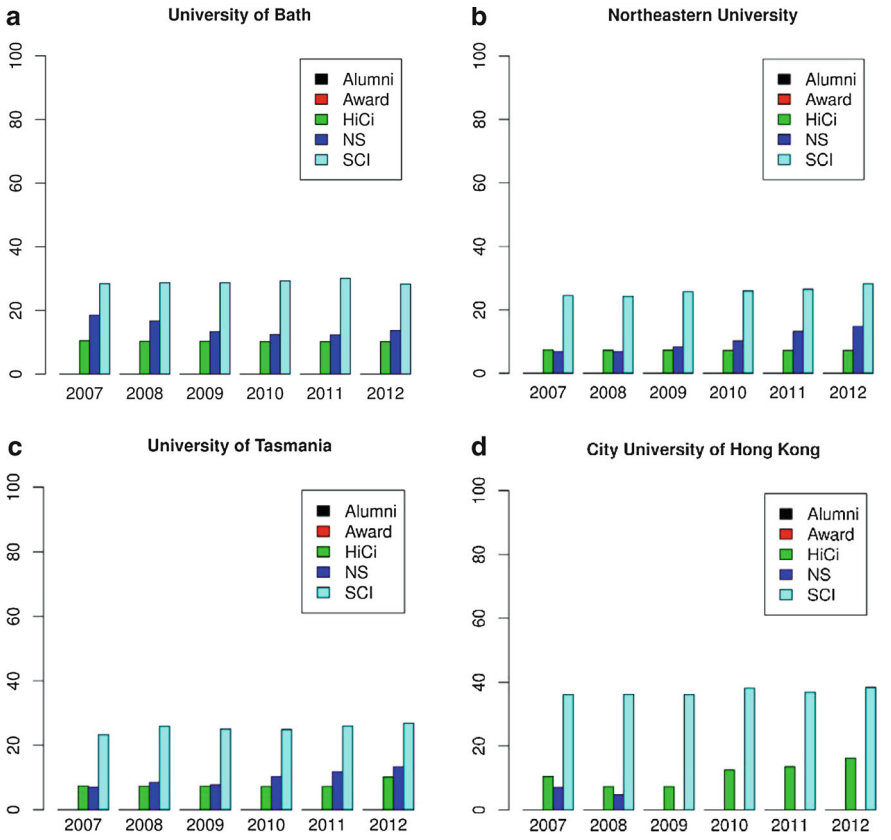


Fig. 5 Examples of universities that still moved levels in between the two periods of time when Performance Per Capita (PCP) was moved—This figure shows four universities that dropped or improved in their level position in the dominance graphs of 2007–2009 and 2010–2012 without PCP included. Figure **(a)** shows the University of Bath, which drops two levels between 2007–2009 and 2010–2012 due to their decrease in the value for the attribute N&S. Figure **(b)** shows Northeastern University which is the university that shows the largest improvement in level position in this comparison. As can be seen in this figure, their improvement can be attributed to their high values for N&S and SCI. Next, in figure **(c)**, the University of Tasmania shows that their rise from Level 16 to 13 is mainly due to their increase in the category N&S. Finally, figure **(d)**, the City University of Hong Kong jumps from Level 13 to 10 although they have a value of zero for the attribute N&S after 2009. The figure shows their level improvement is solely due to the increase in Highly Cited work HiCi

Dominance Graph Computation of 2007–2012 with Symbolic Regression Analysis

After the periods 2007–2009 and 2010–2012 were examined separately, the results of the dominance graph for the whole period are examined. The resulting dominance graph is displayed in Fig. 6.

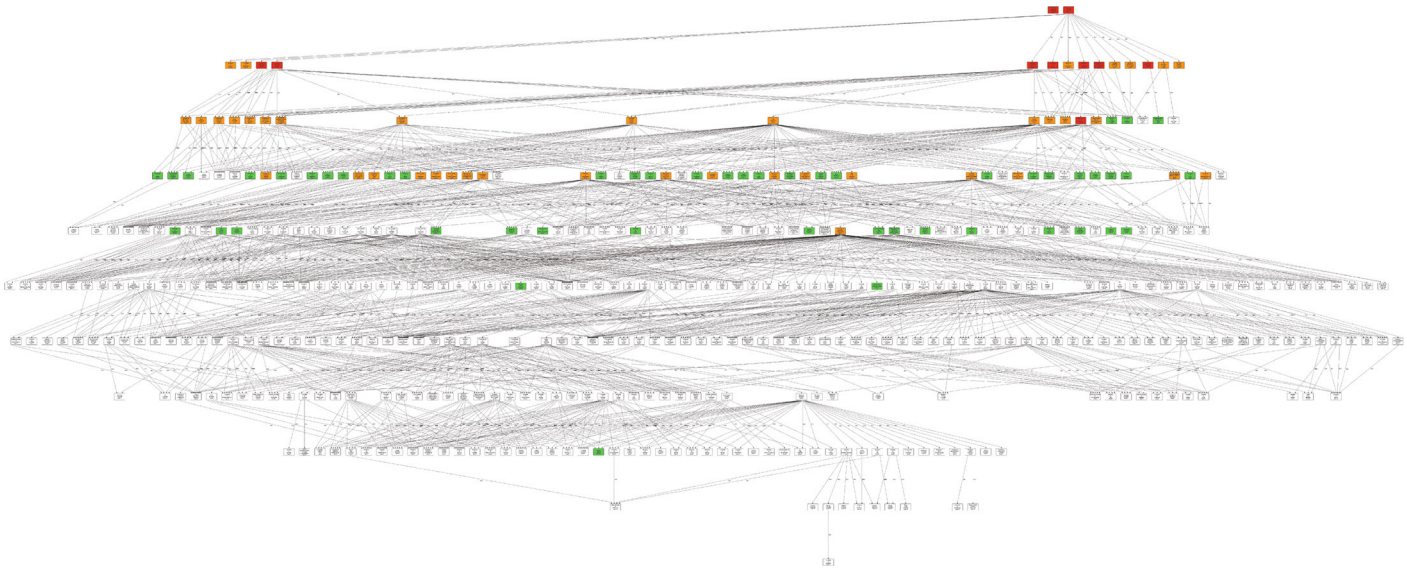


Fig. 6 Results of the dominance graph for the whole period 2007–2012. This graph contains a total of 11 levels, and consistent to the other results, Harvard University is still in Level 1 and directly dominates multiple universities in Level 2. The California Institute of Technology is again the only other institution on Level 1 but does not directly dominate and institutions in Level 2. Again, the coloring is consistent to the other dominance graph figures to highlight the ARWU ranking (NB: A high resolution quality of this image is available in Supplementary material)

Table 2 Universities that change more than two levels in the Pareto dominance graph in the two periods of years considered

University	Level		Change
	2007–2009	2010–2012	
Medical University of Innsbruck	12	7	− 5
Sao Paulo State University	11	7	− 4
Thomas Jefferson University	10	6	− 4
City University of Hong Kong	10	7	− 3
Hanyang University	11	8	− 3
Northeastern University	12	9	− 3
Scuola Normale Superiore di Pisa	6	3	− 3
Stockholm School of Economics	7	4	− 3
Sun Yat-sen University	11	8	− 3
University of Alaska Fairbanks	7	10	3
University of Montpellier 2	4	8	4

Examining the results of universities' position in the dominance graph over a total of 6 years allows the further investigation of the implications for institutions. This is because this graph compiles all the scores for all attributes for each year. With more information available in one ranking outcome like this, more inferences can be made for institutions, strategic planning activities, and research policy making. As stated in section "Materials and Methods", symbolic regression analysis is used on the 2007–2012 ranking results to find those variables that are important in predicting a universities' rank as well as test for consistencies using data from each year to predict the ranking of the 2007–2012 outcome. Using *Eureqa*, the aim is to find a linear function using the existing variables in the dataset to predict the level each university is a "member" of. Two separate error metrics are used to optimize the R^2 goodness of fit (GoF) and the squared error [AIC]. In each experiment the objective was to fit a model to predict the levels of the dominance graph shown in Fig. 6 using, in turn, the variables from each of the years 2007–2012. Results for the experiments optimizing R^2 GoF are shown in Table 3. Results optimizing the squared error [AIC] are shown in Table 4. As can be seen in these tables, all models contain the same variables, Alumni, PUB (research output), and PCP (related to size), for one exception which includes N&S (publications in Nature and Science) instead of PUB. However, it is noted that both PUB and N&S relate to a universities' research output and could therefore be seen as interchangeable as different institutions have higher research outputs in varying disciplines.

Furthermore, all models have highly similar coefficients. All constants at the start of each model range between 9.59 and 10.91 followed by highly similar coefficient attached to the Alumni attribute (ranged between 0.02 and 0.04). Following this, all models, except for that one using 2011 data and the Square Error [AIC], contain the attribute PUB with a coefficient ranging between 0.05 and 0.06. For the exception, N&S is included with a coefficient of also 0.05. This shows that for almost all models, PUB has a greater weighting in predicting universities' levels than the

Table 3 Results of the symbolic regression analysis predicting "level membership" or position. In this table the best linear models found using the R-squared goodness-of-fit measure are presented including the respective error metric. The experiment was run six times to include variables from each of the years separately to find consistencies in the variables used by *Eureqa*. As can be seen, all models are very similar; include the same three variables, Alumni, PUB (related to number of publications), and PCP (related to size); and even have almost identical coefficients

Year of variables used	Best linear model for level prediction	R^2 GoF
2007	Level = 10.40 – 0.02(Alumni) − 0.05(PUB) − 0.10(PCP)	0.71
2008	Level = 10.39 – 0.03(Alumni) − 0.05(PUB) − 0.09(PCP)	0.69
2009	Level = 10.41 – 0.02(Alumni) − 0.06(PUB) − 0.09(PCP)	0.71
2010	Level = 10.91 – 0.02(Alumni) − 0.06(PUB) − 0.10(PCP)	0.71
2011	Level = 10.90 – 0.02(Alumni) − 0.06(PUB) − 0.10(PCP)	0.71
2012	Level = 10.87 – 0.02(Alumni) − 0.06(PUB) − 0.10(PCP)	0.71

Table 4 Results of the symbolic regression analysis predicting "level membership." In this table the best linear models found using the squared error [AIC] measure are presented including the respective error metric. As in Table 3, the experiment was run six times to include variables from each of the years separately to find consistencies in the variables used by *Eureqa*

Year of variables used	Best linear model for level prediction	Mean squared error
2007	Level = 10.56 – 0.02(Alumni) − 0.05(PUB) − 0.10(PCP)	1.04
2008	Level = 10.40 – 0.03(Alumni) − 0.05(PUB) − 0.09(PCP)	1.11
2009	Level = 10.75 – 0.02(Alumni) − 0.06(PUB) − 0.10(PCP)	1.03
2010	Level = 10.87 – 0.03(Alumni) − 0.06(PUB) − 0.10(PCP)	1.04
2011	Level = 9.59 – 0.04(Alumni) − 0.05(N&S) − 0.06(PUB)	1.21
2012	Level = 10.76 – 0.03(Alumni) − 0.06(PUB) − 0.10(PCP)	1.05

Alumni attribute does. Finally, for all models other than the 2011 model in Table 4, PCP is the last included attribute with the heaviest weighted coefficient between 0.09 and 0.10. Again, the exception is the model using squared error [AIC] using 2011 data which includes PUB as its last variable with a coefficient of 0.06 (which is similar to the coefficient attached to PUB for all other models). Besides this one exception, these models show a high level of consistency across which variables are used to predict "level membership" for universities. These results indicate that PCP (size) has a big impact on predicting which level a university is part of in the dominance graph of the ARWU dataset. Furthermore, it can be observed that PUB has a stronger impact on determining which level a university is on than Alumni and that these three attributes combined predict the levels of the 2007–2012 dominance graph to reasonable error values (as shown in Tables 3 and 4). Finally, these symbolic regression results could assist a university in determining strategies for improvement in the ARWU ranking as they provide the strongest impacting attributes on which level a university is on.

To further illustrate the usefulness of the proposed ranking method in this chapter in detail, various examples are outlined from the 2007–2012 dominance graph. The universities selected to provide case examples are the University of Copenhagen, the

University of Amsterdam, and the Australian National University (ANU) as these institutions provide varying interesting cases.

Case Example of the University of Copenhagen

In the total period of 2007–2012, the University of Copenhagen was positioned on Level 4 of the dominance graph. As shown in Fig. 7, the University of Copenhagen is only closely dominated by one university, Yale, while it closely dominates 30 other institutions in Level 5. The fact that only Yale directly dominates the University of Copenhagen in our ranking result is a satisfactory achievement for the University of Copenhagen as this means the "inbound degree" of arcs for Copenhagen is low. However, it must be outlined that as the weight of the arc between Yale and Copenhagen equals to 36, it means that Yale dominates Copenhagen in every attribute for every year ($6 \times 6 = 36$). This shows that the weight of each arc is important in the dominance graph and provides information to the institutions wishing to directly benchmark with other institutions having very similar, though superior, profiles.

Case Example of the University of Amsterdam

In the total period of 2007–2012, the University of Amsterdam (UvA) was positioned on Level 5 of the dominance graph. It is one of the 30 universities that is closely dominated by the University of Copenhagen. Those universities that dominate the UvA and those that the UvA closely dominates are shown in Fig. 8 and create a cone-like shape that the UvA can analyze further for benchmarking purposes. As can be seen in the figure, the UvA is only closely dominated by Northwestern University, the University of Copenhagen, and the University of Illinois at Urbana-Champaign. The University of Amsterdam closely dominates nine universities at Level 6. Five of these universities are from Asia, Fudan University in China, Nanyang Technological University in Singapore, the University of Hong Kong, and the City University of Hong Kong. One such university is from Brazil, State University of Campinas; two universities from Europe, University of Erlangen-Nuremberg in Germany and the Swedish University of Agricultural Sciences; and, finally, one university from Israel, Yeshiva University.

Analyzing the relationship of UvA and the universities it is closely dominated by provides higher education decision-makers and research policy makers with further information about where to exert their resources more heavily. For instance, in the example of the UvA, it is shown how "close" UvA is to those universities that dominate it and possible areas of concentration for UvA to gradually improve its rank according to this system. For this, the scores of attributes only for the year 2012 are used as this is the most recent year in this dataset and therefore provide the most up-to-date answers. The attribute with the smallest difference in score for UvA and one of the universities is dominated by is "PCP" (Performance Per Capita). The UvA has a score in 2012 of 25.8, while the Northwestern University has a score of 26.4 making it only a 0.6 difference in score. This means that if the UvA improves in this item by just a mere 0.6 of a point score, it can change the domination status of one of the three institutions at a higher level that closely dominate it.

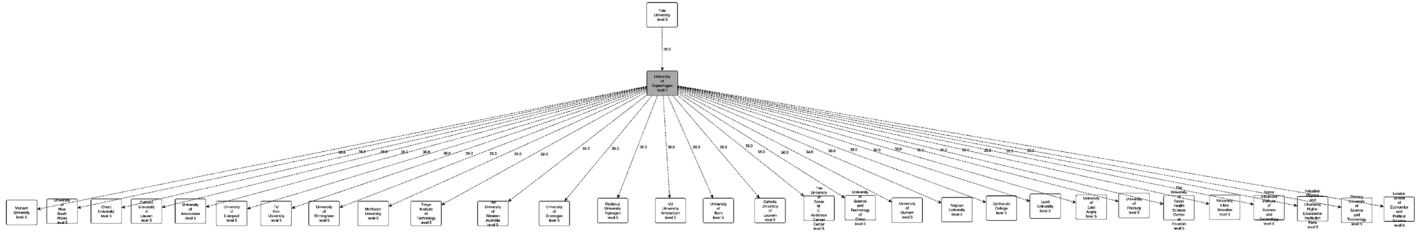


Fig. 7 The position of the University of Copenhagen at Level 4 in the dominance graph of the period 2007–2012. The University of Copenhagen provides an interesting example as it is only directly dominated by one university in Level -, Yale, and it dominates a total of 30 universities directly

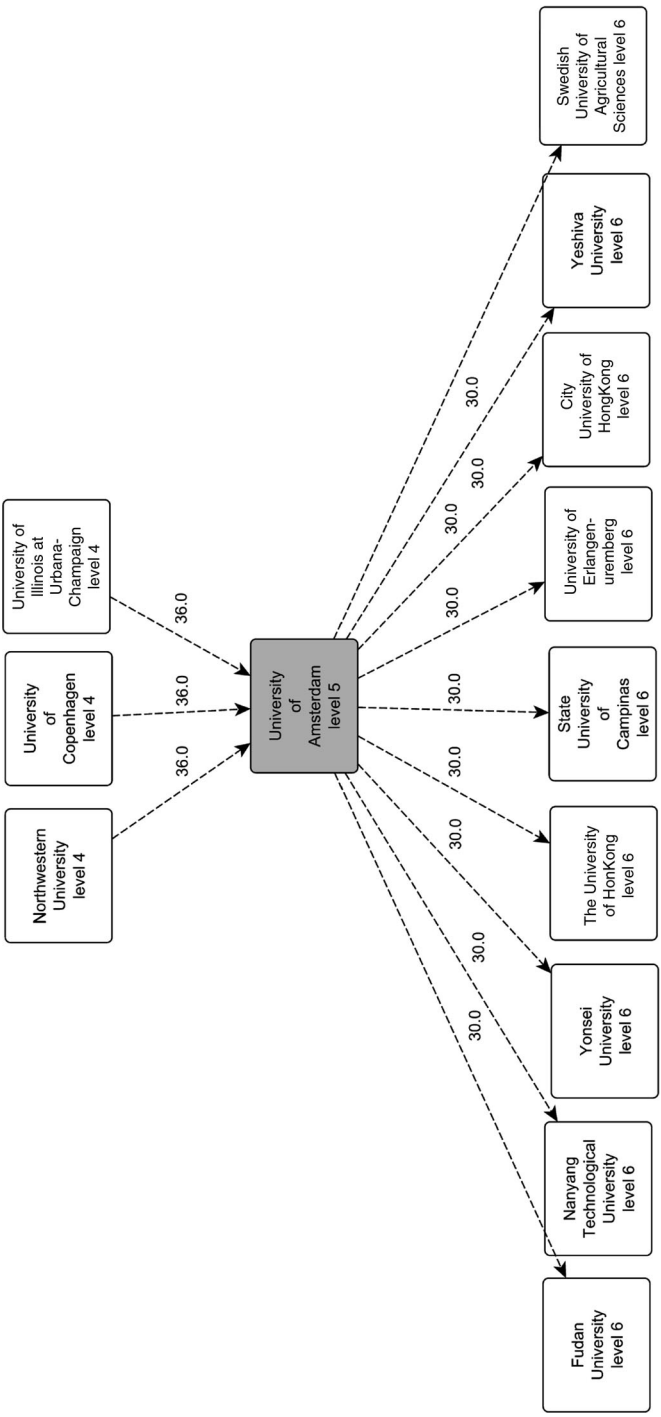


Fig. 8 The position of the University of Amsterdam at Level 5 in the dominance graph of the period 2007–2012. It shows that the University of Amsterdam is directly dominated by three universities at Level 4, Northwestern University, the University of Copenhagen, and the University of Illinois at Urbana-Champaign. In turn, the University of Amsterdam dominates nine other universities directly in Level 6 as shown in the figure

Furthermore, the University of Illinois at Urbana-Champaign has a score of 27.8 for PCP meaning that UvA is only 2% points lower. The University of Copenhagen has a score of 32.3 for PCP making it 6.5 points higher than UvA. The second closest attribute score for UvA and its dominating universities is in "PUB." UvA's score in this attribute in 2012 is 52.8 while the University of Illinois at Urbana-Champaign has a score of 56.1 meaning there is only a 3.3 point difference. Northwestern University has a score of 58.1 (5.3 point difference with UvA) and the University of Copenhagen has a score of 58.6 (5.8 point difference with UvA). These results tell us that the UvA is closest to its dominating institutions in these attributes. This can inform management at UvA to focus attention and resources on increasing publications in certain journals and in doing so improve their Performance Per Capita (PCP) instead of investing heavily in other attributes in which the differences are pronounced.

Case Example of the Australian National University

In the analysis using the total period of 2007–2012, the Australian National University (ANU) was positioned on Level 4 of the dominance graph which is the same level as the University of Copenhagen (see Fig. 9). ANU is closely dominated by six universities at Level 3, while it, in turn, closely dominates five universities at Level 5. The universities that dominate ANU are all from the USA or the UK, University of California San Diego; Yale University; University College London; the Imperial College of Science, Technology, and Medicine; Cornell University; and the University of Chicago. The universities that ANU closely dominates are a more international mix, the University of Bern in Switzerland, Oregon State University from the USA, the University of Durham and the University of East Anglia in the UK, and Korea Advanced Institute of Science and Technology.

ANU is taken as an example of how to investigate the relationship with those universities a given institution closely dominates. In the case of ANU, there are five. In this scenario it is important to investigate which universities would be close in certain attributes or in some cases almost equal. Once the university of interest is no longer better in all attributes, it will no longer closely dominate those universities. Again, only the scores from 2012 will be investigated to provide a scenario as up to date as possible. In the case of ANU, three of the closest scores are in the attribute "Alumni" which relates to the alumni of the institution that obtain a Nobel Prize. In most cases, this is extremely difficult to alter, and a discussion on this attribute is provided in the following section. The next attributes in which universities closely "compete" with ANU are the attributes N&S and PCP.

The closest university in terms of the attribute N&S is the University of Bern followed by Oregon State University. These universities have scores less than 4 points below that of ANU making them competitors for ANU in this system. In terms of PCP, the Korea Advanced Institute of Science and Technology and the University of Bern again come close to ANU. The reason these attributes are outlined rather than Alumni is because these attributes are more in the control of the university than the number of Nobel Prize winners they "produce." For instance, ANU could encourage and enable its researchers to publish articles in Nature

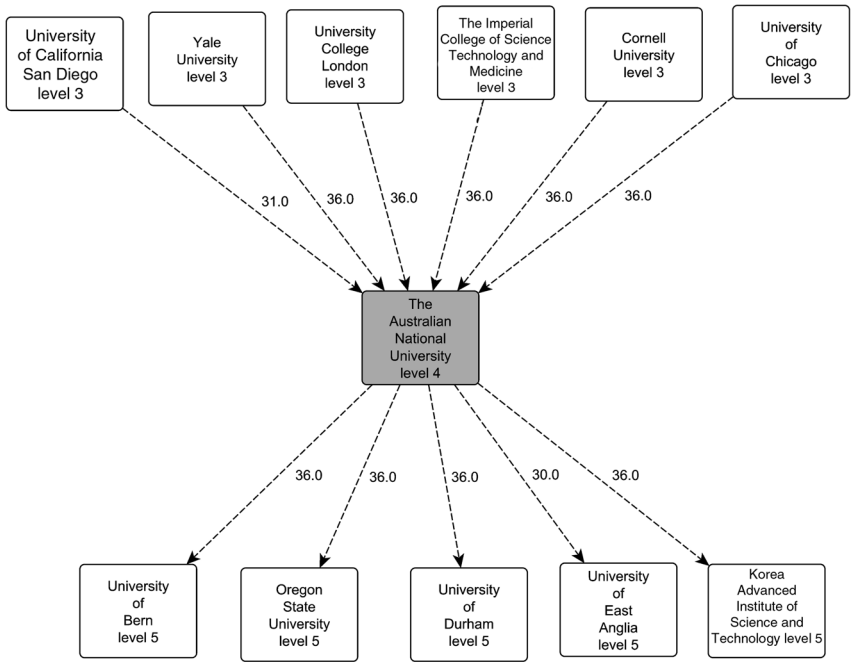


Fig. 9 The position of the National University of Australia at Level 4 in the dominance graph of the period 2007–2012. It is closely dominated by six universities at Level 3, while it, in turn, closely dominates five universities at Level 5

and Science rather than other journals in the field. Furthermore, when improving research output in terms of quality and volume without growing in size, it would be likely that Performance Per Capita would increase. These results show us the attributes that ANU needs to focus on in order to keep their competitive advantage and higher position in the dominance graph due to the universities they closely dominate.

The Case of the Alumni Attribute

In the ARWU ranking, the attribute measuring "Alumni," "the total number of the alumni of an institution winning Nobel Prizes and Fields Medals," has been extensively scrutinized and criticized for not measuring actual research or teaching excellence [28, 29]. As Ioannidis et al. [17] state, there is no doubt that research excellence *could* indirectly be measured by the Nobel and Field Medal faculty members working at a given institution. However, there is no justification to assume the existence of any clear correlation with the quality of teaching and research as a whole of the institution. Perhaps, it is only a measure of maturity, sheer size, number of graduates, financial support, and capital investment on a particular discipline

(only certain disciplines can give Nobel Awards or Field Medals). In addition, it is unclear why an event with such a low probability of occurrence (very few individuals receive Nobel Prizes or Field Medals) has such a large impact on the activities measured over populations of individuals. Furthermore, Ioannidis et al. have well argued that, in many cases, the affiliation of the Nobel and Field Medals differs at the time they actually conducted the award-winning work and the time that they were awarded the prize [17].

To prove that an attribute in the ranking process may be a spurious one, another type of sensitivity analysis is conducted as part of this chapter's proposed heuristic method. The attribute *Alumni* is eliminated and its effects on the ranking of the universities and their levels over the whole period 2007–2012 are subsequently investigated. It was previously observed that institutions such as the Netherland's Radboud University Nijmegen may have obtained dramatic changes in overall level position due to the attribution of Alumni alone (as can be seen in Fig. 4a shown by the black bar) which provides yet another reason for the comparison of these two outcomes.

According to the definition, the category Alumni represents *"the total number of the alumni of an institution winning Nobel Prizes and Fields Medals. Alumni are defined as those who obtain bachelor, Master's or doctoral degrees from the institution"*. In order to analyze the effect of this category in the ranking analysis, the dominance graph without the Alumni category is recalculated. Thus, each of the 444 universities performance is only observed in the remaining five scores in six consecutive years.

The resulting dominance graph on this dataset has 11 levels which is the same number of levels as the result of 2007–2012 period with all attributes included. This means that any changes in levels by a university is significant and provides useful information. However, here only those universities that increased or dropped by two or more levels are presented for the sake of brevity.

From the 444 universities, 196 universities (44.14%) do not change in level when the attribute Alumni is taken out of the dominance graph algorithm. There are 226 universities (50.9%) that change in only one level and only 21 universities (4.73%) that fall two or three levels. Furthermore, there is only one university (City University of New York City College) that falls four levels, indicating the robustness of this scheme. This points out the irrelevance of the Alumni attribute for the presented heuristic ranking method. This may please those opposed to Alumni being included as a proxy for measuring quality.

However, it is interesting to highlight how the performance of individual institutions may be affected by this choice. Looking at the City University of New York City College, without its impressive legacy of Nobel Prize winners, falls from Level 5 to Level 9. It was clearly in a higher level due to its high performance in the Alumni attribute (with a score of more than 37 in the three years), but the score in the rest of the categories is no higher than 18.

The three universities that fall more than three levels are Eotvos Lorand University in Hungary which fell from Level 7 to Level 10, University of Chile which fell from Level 8 to Level 11, and Saint Petersburg State University from Russia which

fell from Level 6 to Level 9. It is clear that these institutions, without their successful score in the Alumni attribute, fall down to lower levels as their scores reflect much lower results in the other categories. It is also interesting to note that the Radboud University Nijmegen in the Netherlands, which, as pointed out, increased in levels over the two separated periods (2007–2009 and 2010–2012), actually falls from Level 5 to Level 6 over the whole period (2007–2012) when the attribute Alumni is not considered. This strengthens the argument that their climb in levels over the six-year period is merely due to their Alumni winning the Nobel prize in Physics rather than a total improvement in "academic excellence" supporting those who argue the superfluous nature of the Alumni attribute in measuring "academic excellence." The results of all universities which dropped or gained two or more levels with the removal of the Alumni attribute are shown in Table 5.

The Case of the Performance Per Capita Attribute (Size)

In the ARWU ranking, the only way in which the "size" of an institution is taken into account is through the only per capita measure: Performance Per Capita (PCP). In the ARWU ranking, PCP is defined by as "*The weighted scores of the above five indicators divided by the number of full-time equivalent academic staff. If the*

Table 5 Universities that change more than two levels in the Pareto dominance graph in the period 2007–2012 when the attribute Alumni is not considered. There are another 226 universities that drop one level

University	Levels dropped
City University of New York City College	− 4
Eotvos Lorand University	− 3
University of Chile	− 3
Saint Petersburg State University	− 3
Case Western Reserve University	− 2
University of Nebraska Lincoln	− 2
University of Tuebingen	− 2
Queen Mary U. of London	− 2
Technical University of Berlin	− 2
The University of Connecticut Storrs	− 2
University of Cape Town	− 2
Brigham Young University	− 2
Ecole Polytechnique	− 2
Technical University Darmstadt	− 2
The University of Montana Missoula	− 2
University of Oulu	− 2
Moscow State University	− 2
Complutense University of Madrid	− 2
University of the Witwatersrand	− 2
University of Warsaw	− 2
Istanbul University	− 2
Technical University of Braunschweig	− 2

number of academic staff for institutions of a country cannot be obtained, the weighted scores of the above five indicators is used." [30]. Since this attribute is the only variable in ARWU's ranking method which takes an institution's size into account, it is an interesting image to see the effect of its removal on our methodology.

The sample of 444 universities over the total period of 2007–2012 has been used for the dominance graph algorithm in order to compare its outcome to the result of the 2007–2012 results for all attributes. There are more changes to the outcome when PCP is removed than when Alumni was removed. Firstly, the resulting dominance graph has 12 levels which means that some universities may change in levels due to a total reorganization of the graph. This means that all universities which changed more than two levels are significant; however, this presents a total of 124 universities; therefore, again for the sake of brevity, only those universities which changed in more than three levels are reported here. In these results, only 138 universities (31.08%) did not change a level, 182 universities (40.99%) changed in only one level, 84 universities (18.92%) changed 2 levels, and exactly 40 universities (9%) changed three or more levels.

In the case of deleting the PCP attribute from the method, the most extreme result is the Scuola Normale Superiore di Pisa in Italy, which drops an impressive 8 levels from Level 3 to Level 11. This incredible drop is likely due to its extremely small size in terms of staff and students. The Scuola Normale Superiore (SNS) has only approximately 150 undergraduate students, 120 postgraduates, and only 190 doctoral students as it only takes approximately 60 new students annually through its highly selective entry exam. However, it is important to note that the SNS in Italy is an entirely special case. In order to attend this institution, students sit a rigorous entry exam and must obtain certain levels of grades to remain enrolled in SNS. Furthermore, what makes SNS a special case is that its undergraduate students follow courses at the public university, the University of Pisa (Level 6 in the results without PCP), while living on the SNS campuses. This means that its research academics are able to devote their time to research and that they only have high-caliber students. This explains their impressive result in terms of Performance Per Capita and the incredible drop when PCP is removed from the method.

Furthermore, a small Swiss Institution the Swiss Federal Institute of Technology (EPFL) located in Lausanne has previously been highlighted. Another small Swiss institution the Swiss Federal Institute of Technology Zurich (ETHZ) is highlighted for a different reason. ETHZ was located on Level 3 in the result when all attributes were included in the method; however, with PCP removed, ETHZ fell to Level 4. The removal of the PCP attribute also had a negative effect for EPFL which fell from Level 4 to Level 7. This shows that for many small institutions, the attribute which takes Size into account is important for their position on the ranking levels.

The last small institution that is discussed here provides a particularly interesting case, Princeton University. Between the result for 2007–2012 with all attributes included and the result with PCP removed, Princeton remains on Level 2. In both results, Princeton University is only closely dominated by Harvard. The only change is in the universities Princeton closely dominates as it dominates the Scuola Normale

Superiore di Pisa just discussed and Ecole Superieure Paris in the result of all attributes but does not closely dominate any institution when PCP is removed from the method. This is because both these institutions drop in levels when PCP is removed leaving no Level 3 institution closely dominated by Princeton. From this result for Princeton University, it can stated that even though it is a small institution when compared to the likes of Harvard, it "holds its ground" even when the only per capita measure is removed unlike the other small institutions that have been discussed.

Discussion

This section elaborates on the findings of the proposed heuristic ranking method and reflects on the contribution of this chapter from a mathematical standpoint as well as a research policy making standpoint which is the application in this chapter to highlight the usefulness of this method. Furthermore, a brief overview of other existing multi-objective ranking methods is provided as a means for comparing the method presented in this chapter. This brief review is then used to highlight several benefits of this chapter's methodology as well as define any future research possibilities to improve the method. Firstly, the two main contributions of this chapter are described in further detail which are a mathematical contribution in terms of a transparent, user-centric heuristic ranking method through the use of a dominance graph and the provision of implications for decision-makers, managers, and research policy makers at educational institutions and governmental offices.

A Transparent and User-Centric Ranking Method

As stated, many authors argue that current ranking systems which include arbitrarily assigned weights and measurement approaches are generally naive, biased, flawed, or a combination of all.

This chapter has attempted to provide a different approach to the ranking of objects with multiple attributes through the use of the global universities ranking example. Needless to say, the methodology is applicable to other types of problems and would allow users of this method to naturally deal with attributes that are numerical and/or ordinal in nature. For example, scores or attributes that have ordinal values in categories could be taken into consideration for this method allowing a much wider adoption of this method for different ranking needs that include a variety of attributes or objects. The aim of discussing particular case examples was to provide an illustration of the user-centric approach of this method which allows alternative ranking outcomes to be investigated according to the user's interests and needs. It is for these reasons that the simple method used in this chapter is easily transferable to other situations.

One of the main benefits of the heuristic ranking method proposed here is that each user with differing interests can be satisfied by selecting which of the

attributes must be included in the ranking. As it has been pointed out, the position of several universities changed with the deletion of an attribute. This means that this system allows users to look at the world ranking of universities based on attributes that matter to them and exclude those that do not. Naturally, this would also apply to other ranking and ordering exercises in different situations where users may have varying interests. Such instances could include the ranking of medical institutions for possible medical treatments, companies or organizations for prospective employment, or even the analysis of countries or cities with attributes relating to "livability."

Furthermore, the approach to ranking of global universities in this study provides a more transparent method as it uses raw data rather than accumulated, normalized, or weighted data. As it is the weighting of the data in the ARWU ranking of world universities that concerns a large number of authors [25], it is argued that the proposed heuristic hierarchical method presented in this chapter satisfies a missing need for transparently ranking universities based on raw data available publicly. As it has been explained, although current university ranking methods seem to be flawed and subject to data inaccuracy, scholars and educational institution leaders agree that these activities are "here to stay" [52]. Hence, utilizing a methodology which is transparent and user centric may be more appropriate for the world "need" of global university rankings and other ranking activities.

Implications for Research Policy Makers

As stated before, well-known annual rankings of global universities, such as the ARWU, are often criticized due to their level of volatility [52]. The advantage of the methodology proposed in this chapter is that it organizes global universities in levels based on a dominance graph rather than a "string list." This means that the real nature of slow and steady university growth and improvement is reflected in this method rather than highly volatile changes. As such, this methodological approach correlates well with the subjective assumption that large institutions, like universities, when compared with other institutions of similar size and prestige, do not experience high levels of volatility in terms of their status over periods spanning just a few years. This level-based scheme induced by a dominance graph shows low volatility over consecutive periods of three years and therefore reflects the *real-life* nature of large educational institution growth.

The ability of an educational institution to compare and benchmark itself within the "ladders" of the hierarchy of the presented layers is what actually brings university leaders a clear path for improvement as outlined in the case of the University of Amsterdam in section "Case Example of the University of Amsterdam". Or conversely, a clear image of close competitors in levels below that institution can be examined, as highlighted with the example of the Australian National University in section "Case Example of the Australian National University". In these examples it was easily and clearly identified that these universities' nearest objectives (subset of

the top layer that dominate that university) and those attributes they should consider to "protect their standing" (the ones a university closely dominates in the layer immediately below). This knowledge is of considerable value to universities and its research policy makers as it aides strategic decisions regarding the distribution of resources and funding. Although many different approaches to find the optimal allocation of resources by universities exist, for example, such as in [31], since global ranking systems are here to stay, ways of improving on these systems will become more valuable.

Furthermore, it has been found that changes in resource allocation have an impact on the type of activities academics concentrate on [31]. This means that those universities in sections "Case Example of the University of Amsterdam" and "Case Example of the Australian National University", who needed to improve in areas such as publications in Nature and Science, can base their decisions for allocations of resources on numbers that will actually help them in the global standing of universities.

Overview of Existing Multi-objective Ranking Methods

The problem of multicriteria ranking has been active in the decision-makers research community for several years. In most cases, authors aim to produce a total order of the alternatives, or as it has previously been referred to in this chapter, a "string list" of object number one to the bottom ranked object. It is possible to classify the methods presented in two approaches: the first one, and most common, is the use of different ways of aggregating the criteria (features) such that different ways of combining allow to obtained better rankings. Examples of the above can be found in [32–41]. It is arguable whether a specific way of combining or weighting criteria is better than other. The second approach to solving this problem is by modeling it as a multiobjective optimization problem and to solve it by using, for example, evolutionary algorithms. Examples of the above can be found in [42–46]. This approach has not been fully exploited, and recent advances in evolutionary algorithms and other techniques to solve multi-objective optimization can be applied. It is also well known that the use of specific knowledge of the problem in the solution leads to good performing methods. In the literature, several applications of domain-specific multicriteria ranking methods can also be found, which incorporate characteristics of the problem at hand in order to combine the different criteria to make the decision. Examples of the above can be found in [47–56].

Independently of the approach used, all methods aim to produce more reliable rankings and benefit decision-makers. The approach presented in this chapter differs from the ones presented above in that it neither aggregates criteria or produces a total ranking of the alternatives. The method uses multiple criteria (features) in the way they are delivered and only compares values of alternatives into each criteria. This simple method allows the user to scale to several alternatives or objects to be ranked

and produces groups or layers of alternatives that are distinguishable from the others, but that between them there is no single alternative that is better than other, according to the criteria considered. In other words, the results of the dominance graph heuristic approach "ranking" provides the user with more information for comparison as the different layers can be compared, as well as the outcomes within the layers.

Conclusion

This chapter has contributed a simple method for hierarchically ranking a set of objects according to multiple characteristics and with multiple objectives in mind. Several extensions and improvements can be made to this method in future efforts. Firstly, in this study only one dataset is used to present the methodology, those produced by the annual Shanghai ranking, to illustrate on the use of our methodology and its implications for research policy. However, even in the case of ranking global universities, other datasets and information sources are available. For instance, the Times Higher Education [57] ranks universities annually, utilizing a different approach. Future research could investigate the combination of information by the use of attributes that come from the different ranking organizations.

Secondly, as stated multiple times, the use of a dominance graph in order to investigate ranking problems is transferrable to different situations. For instance, other researchers could apply this method and test it on other datasets, potentially including a combination of numerical and categorical (ordinal only) data, without the need of an ad hoc weighting scheme, which is very useful under different circumstances. Any research problem in which the objective is to rank or order based on dominance using multiple variables could implement this method as part of the process.

In order to highlight the flexibility of the method presented in this chapter, some extremely different examples are explained. A different use of this method could be for the increased interest in dealing with protected area zoning for conservation as a multi-objective optimization problem [58]. For many years, when multiple geographic areas are examined, the objective is to create a priority ranking of these natural areas based on a variety of variables such as climax condition, educational suitability, species significance, community representation, and human impact [59]. Furthermore, as Smith and Theberge [60] explain, when evaluating natural areas using measurements for a series of criteria, it involves deciding which criteria are important and most significant based on their measurements. The use of a dominance graph would allow alteration and the inclusion or exclusion of certain criteria for each natural area individually and would thus be useful to many different instances across various natural circumstances. For instance, another circumstance for such analysis is the prioritization of farmland preservation for multiple objectives [61] in which the purpose is to preserve it from urbanization.

Another area of obvious application is that of sports [62–65] and other ranking games in academia, even if they can turn futile when using the wrong assumptions

[66]. We are also aware that, after the publication of this paper, a number of other alternative techniques will soon follow that may explore the directed acyclic graph construction hereby presented. We also envision that some researchers may try to exploit the transitive relations of dominance to build up different ranking scores even for objects at the same level. Given that this method is utilized in a global ranking of educational institutions could possibly be transferable to an area so different such as natural area conservation zoning or sports-related ranking activities, it becomes easy to imagine the wide use of dominance graphs in many other ranking activities.

In conclusion, this study adds to the literature a diverse and dynamic heuristic method for ranking multi-variable problems that is transparent and not biased by arbitrary weightings unilaterally and perpetually selected by the ranking organization. We have outlined the utility of this method in the case of ranking global universities using open, publicly available data and, in doing so, provided a novel methodology to approach the ranking of universities globally. This system allows for clearer benchmarking of universities which are at a similar standing. Overall it helps to provide assistance to research policy and decision-makers while at the same time enables a global standing of the universities in the world's landscape of diversified academic offers.

Cross-References

▸ Multiobjective Optimization

Acknowledgments Pablo Moscato acknowledges support from the Australian Research Council Future Fellowship FT120100060 and Australian Research Council Discovery Projects DP120102576 and DP140104183.

Competing Interest Declaration The author(s) has no competing interests to declare that are relevant to the content of this manuscript.

References

1. Agnew T, Whitlock R, Neutze J, Kerr A (1994) Waiting lists for coronary artery surgery: can they be better organised? N Z Med J 107(979):211–215
2. Kellenberger E, Foata N, Rognan D (2008) Ranking targets in structure-based virtual screening of three-dimensional protein libraries: methods and problems. J Chem Inf Model 48(5):1014–1025. https://doi.org/10.1021/ci800023x. http://pubs.acs.org/doi/abs/10.1021/ci800023x
3. Braga-Neto U, Hashimoto R, Dougherty ER, Nguyen DV, Carroll RJ (2004) Is cross-validation better than resubstitution for ranking genes? Bioinformatics 20(2):253–258. https://doi.org/10.1093/bioinformatics/btg399. http://bioinformatics.oxfordjournals.org/content/20/2/253.abstract
4. Li W, Suh YJ, Zhang J (2006) Does logarithm transformation of microarray data affect ranking order of differentially expressed genes? In: 28th annual international conference of the IEEE Engineering in medicine and biology society, EMBS'06, vol Supplement, pp 6593–6596. https://doi.org/10.1109/IEMBS.2006.260896

5. Sun J, Kuo PH, Riley BP, Kendler KS, Zhao Z (2008) Candidate genes for schizophrenia: a survey of association studies and gene ranking. Am J Med Genet B Neuropsychiatr Genet 147B(7):1173–1181. https://doi.org/10.1002/ajmg.b.30743
6. Voll CA, Goodwin JE, Pitney WA (1999) Athletic training education programs: to rank or not to rank? J Athl Train 34(1):48–52. http://search.proquest.com/docview/206648692?accountid=45394
7. Pulugurtha SS, Krishnakumar VK, Nambisan SS (2007) New methods to identify and rank high pedestrian crash zones: an illustration. Accid Anal Prev 39(4):800–811. https://doi.org/10.1016/j.aap.2006.12.001. http://www.sciencedirect.com/science/article/pii/S000145750600217X
8. Kee D, Karwowski W (2003) Ranking systems for evaluation of joint and joint motion stressfulness based on perceived discomforts. Appl Ergon 34(2):167–176. https://doi.org/10.1016/S0003-6870(02)00141-2 . http://www.sciencedirect.com/science/article/pii/S0003687002001412
9. Burden DJ (1995) The ranking of dental aesthetics. J Orthod 22(3):259–261. http://jorthod.maneyjournals.org/content/22/3/259.abstract
10. Buela-Casal G, Gutirrez-Martnez O, Bermdez-Snchez M, Vadillo-Muoz O (2007) Comparative study of international academic rankings of universities. Scientometrics 71(3):349–365. https://doi.org/10.1007/s11192-007-1653-8
11. Liu NC, Cheng Y (2005) The academic ranking of world universities. High Educ Eur 30(2):127–136. https://doi.org/10.1080/03797720500260116
12. Marginson S (2007) Global university rankings: implications in general and for Australia. J High Educ Policy Manag 29:131–142. https://doi.org/10.1080/13600800701351660
13. Merisotis J, Sadlak J (2005) Higher education rankings: evolution, acceptance, and dialogue. High Educ Eur 30:97–101. https://doi.org/10.1080/03797720500260124. s
14. Saisana M, d'Hombres B, Saltelli A (2011) Rickety numbers: volatility of university rankings and policy implications. Res Policy 40(1):165–177. https://doi.org/10.1016/j.respol.2010.09.003. http://www.sciencedirect.com/science/article/pii/S0048733310001812
15. Thakur M (2007) The impact of ranking systems on higher education and its stakeholders. J Inst Res 13(1):83–96
16. Harvey L, Green D (1993) Defining quality. Assess Eval High Educ 18(1):9–34. https://doi.org/10.1080/0260293930180102. http://www.tandfonline.com/doi/abs/10.1080/0260293930180102
17. Ioannidis J, Patsopoulos N, Kavvoura F, Tatsioni A, Evangelou E, Kouri I, Contopoulos-Ioannidis D, Liberopoulos G (2007) International ranking systems for universities and institutions: a critical appraisal. BMC Med 5(1):30. https://doi.org/10.1186/1741-7015-5-30. http://www.biomedcentral.com/1741-7015/5/30
18. Razvan FV (2007) Irreproducibility of the results of the Shanghai academic ranking of world universities. Scientometrics 72(1):25–32. https://doi.org/10.1007/s11192-007-1712-1
19. Docampo D (2013) Reproducibility of the Shanghai academic ranking of world universities results. Scientometrics 94(2):567–587. https://doi.org/10.1007/s11192-012-0801-y. http://link.springer.com/article/10.1007/s11192-012-0801-y
20. Bang-Jensen J, Gutin G (2001) Digraphs: theory, algorithms and applications. Springer, London
21. Moyles DM, Thompson GL (1969) An algorithm for finding a minimum equivalent graph of a digraph. J ACM 16(3):455–460. https://doi.org/10.1145/321526.321534. http://doi.acm.org/10.1145/321526.321534
22. Smits G, Kotanchek M (2005) Pareto-front exploitation in symbolic regression. In: O'Reilly UM, Yu T, Riolo R, Worzel B (eds) Genetic programming theory and practice II. Genetic programming, vol 8. Springer, pp 283–299. https://doi.org/10.1007/0-387-23254-0_17
23. Schmidt M, Lipson H (2014) Eureqa (version 0.98 beta) [software]. http://www.nutonian.com/research/reference/
24. Vladislavleva EJ, Smits GF, den Hertog D (2009) Order of nonlinearity as a complexity measure for models generated by symbolic regression via Pareto genetic programming.

IEEE Trans Evol Comput 13(2):333–349. https://doi.org/10.1109/TEVC.2008.926486. http://dl.acm.org/citation.cfm?id=1650365
25. de Vries NJ, Carlson J, Moscato P (2014) A data-driven approach to reverse engineering customer engagement models: towards functional constructs. PLoS One 9(7):e102,768. https://doi.org/10.1371/journal.pone.0102768
26. Baty P (2014) Caltech: secrets of the worlds number one university: how does a tiny institution create such outsized impact? http://www.timeshighereducation.co.uk/features/caltech-secrets-of-the-worlds-number-one-university/2011008.fullarticle
27. Nijmegen RU (2011) Nobel prize in physics 2010. http://www.ru.nl/english/research/researchers/nobel-prize/
28. Jeremic V, Bulajic M, Martic M, Radojicic Z (2011) A fresh approach to evaluating the academic ranking of world universities. Scientometrics 87(3):587–596. https://doi.org/10.1007/s11192-011-0361-6
29. Oddershede J (2013) What rankings don't tell you about university excellence. http://theconversation.com/what-rankings-dont-tell-you-about-university-excellence-18704
30. ARWU (2013) ARWU methodology: definition of indicators. http://www.shanghairanking.com/ARWU-Methodology-2013.html
31. Liefner I (2003) Funding, resource allocation, and performance in higher education systems. High Educ 46(4):469–489. https://doi.org/10.1023/A:1027381906977
32. Carrillo VM, Taboada HA (2012) A general iterative procedure of the non-numerical ranking preferences method for multiple objective decision making. In: Proceedings of the complex adaptive systems 2012 conference. Washington, DC, pp 135–139. https://doi.org/10.1016/j.procs.2012.09.043
33. Chen Y, Kilgour DM, Hipel KW (2011) An extreme-distance approach to multiple criteria ranking. Math Comput Model 53(5–6):646–658. https://doi.org/10.1016/j.mcm.2010.10.001
34. Gerani S, Zhai C, Crestani F (2012) Score transformation in linear combination for multi-criteria relevance ranking. In: Advances in information retrieval – proceedings of 34th European conference on IR research, ECIR 2012, Barcelona, pp 256–267
35. Greco S, Mousseau V, Slowinski R (2008) Ordinal regression revisited: multiple criteria ranking using a set of additive value functions. Eur J Oper Res 191(2):416–436. https://doi.org/10.1016/j.ejor.2007.08.013
36. Hinloopen E, Nijkamp P, Rietveld P (2004) Integration of ordinal and cardinal information in multi-criteria ranking with imperfect compensation. Eur J Oper Res 158(2):317–338. https://doi.org/10.1016/j.ejor.2003.06.007
37. Kadzinski M, Tervonen T (2013) Robust multi-criteria ranking with additive value models and holistic pair-wise preference statements. Eur J Oper Res 228(1):169–180. https://doi.org/10.1016/j.ejor.2013.01.022
38. Kadzinski M, Greco S, Slowinski R (2012) Selection of a representative value function in robust multiple criteria ranking and choice. Eur J Oper Res 217(3):541–553. https://doi.org/10.1016/j.ejor.2011.09.032
39. Toma I, Roman D, Fensel D, Sapkota B, Gómez JM (2007) A multi-criteria service ranking approach based on non-functional properties rules evaluation. In: Proceedings of fifth international conference on service-oriented computing – ICSOC 2007, Vienna, pp 435–441. https://doi.org/10.1007/978-3-540-74974-5_40
40. Wu J, Liang L (2012) A multiple criteria ranking method based on game cross-evaluation approach. Annals OR 197(1):191–200. https://doi.org/10.1007/s10479-010-0817-8
41. Xu X (2001) The SIR method: a superiority and inferiority ranking method for multiple criteria decision making. Eur J Oper Res 131(3):587–602. https://doi.org/10.1016/S0377-2217(00)00101-6
42. Fernandez E, Leyva JC (2002) A method based on multiobjective optimization for deriving a ranking from a fuzzy preference relation. Eur J Oper Res 154(1):110–124. https://doi.org/10.1016/S0377-2217(02)00705-1
43. Hughes EJ (2001) Evolutionary multi-objective ranking with uncertainty and noise. In: Evolutionary multi-criterion optimization, proceedings of first international conference, EMO 2001, Zurich, pp 329–343. https://doi.org/10.1007/3-540-44719-9_23

44. López JCL, Aguilera-Contreras MA (2005) A multiobjective evolutionary algorithm for deriving final ranking from a fuzzy outranking relation. In: Evolutionary multi-criterion optimization, proceedings of third international conference, EMO 2005, Guanajuato, pp 235–249. https://doi.org/10.1007/978-3-540-31880-4_17
45. López JCL, Chavira DAG, Noriega JJS (2014) A multiobjective genetic algorithm based on NSGA II for deriving final ranking from a medium-sized fuzzy outranking relation. In: 2014 IEEE symposium on computational intelligence in multi-criteria decision-making, MCDM 2014, Orlando, pp 24–31. https://doi.org/10.1109/MCDM.2014.7007184
46. di Pierro F, Khu S, Savic DA (2007) An investigation on preference order ranking scheme for multiobjective evolutionary optimization. IEEE Trans Evol Comput 11(1):17–45. https://doi.org/10.1109/TEVC.2006.876362
47. Dadelo S, Turskis Z, Zavadskas EK, Dadeliene R (2014) Multi-criteria assessment and ranking system of sport team formation based on objective-measured values of criteria set. Expert Syst Appl 41(14):6106–6113. https://doi.org/10.1016/j.eswa.2014.03.036
48. Dalal O, Sengamedu SH, Sanyal S (2012) Multi-objective ranking of comments on web. In: Proceedings of the 21st World Wide Web conference, WWW 2012, Lyon, pp 419–428. https://doi.org/10.1145/2187836.2187894. http://doi.acm.org/10.1145/2187836.2187894
49. Lerche DB, Brüggemann R, Sørensen PB, Carlsen L, Nielsen OJ (2002) A comparison of partial order technique with three methods of multi-criteria analysis for ranking of chemical substances. J Chem Inf Comput Sci 42(5):1086–1098. https://doi.org/10.1021/ci010268p
50. Malekmohammadi B, Zahraie B, Kerachian R (2011) Ranking solutions of multi-objective reservoir operation optimization models using multi-criteria decision analysis. Expert Syst Appl 38(6):7851–7863. https://doi.org/10.1016/j.eswa.2010.12.119
51. Nguyen LT, Yee WG, Liew R, Frieder O (2010) Experiences with using SVM-based learning for multi-objective ranking. In: Proceedings of the 19th ACM conference on information and knowledge management, CIKM 2010, Toronto, pp 1917–1920. https://doi.org/10.1145/1871437.1871763. http://doi.acm.org/10.1145/1871437.1871763
52. Nwamadi O, Zhu X, Nandi AK (2012) Multi-criteria ranking based greedy algorithm for physical resource block allocation in multi-carrier wireless communication systems. Signal Process 92(11):2706–2717. https://doi.org/10.1016/j.sigpro.2012.04.020
53. Rad A, Naderi B, Soltani M (2011) Clustering and ranking university majors using data mining and AHP algorithms: a case study in Iran. Expert Syst Appl 38(1):755–763. https://doi.org/10.1016/j.eswa.2010.07.029
54. Reba MNM, Rosli AZ, Makhfuz MA, Sabarudin NS, Roslan NH (2013) Determination of sustainable land potential based on priority ranking: multi-criteria analysis (MCA) technique. In: Computational science and its applications – ICCSA 2013 – proceedings of 13th international conference, Ho Chi Minh City, part VI, pp 212–218. https://doi.org/10.1109/ICCSA.2013.44
55. Schall D (2014) A multi-criteria ranking framework for partner selection in scientific collaboration environments. Decis Support Syst 59:1–14. https://doi.org/10.1016/j.dss.2013.10.001
56. Shi Z, Hao F (2013) A strategy of multi-criteria decision-making task ranking in social-networks. J Supercomput 66(1):556–571. https://doi.org/10.1007/s11227-013-0934-7
57. THE (2013) Times higher education: world university rankings. http://www.timeshighereducation.co.uk/world-university-rankings/
58. Geneletti D, van Duren I (2008) Protected area zoning for conservation and use: a combination of spatial multicriteria and multiobjective evaluation. Landsc Urban Plan 85(2):97–110. https://doi.org/10.1016/j.landurbplan.2007.10.004. http://www.sciencedirect.com/science/article/pii/S0169204607002496
59. Gehlbach FR (1975) Investigation, evaluation, and priority ranking of natural areas. Biol Conserv 8(2):79–88. https://doi.org/10.1016/0006-3207(75)90033-6. http://www.sciencedirect.com/science/article/pii/0006320775900336
60. Smith PGR, Theberge JB (1987) Evaluating natural areas using multiple criteria: theory and practice. Environ Manag 11(4):447–460. https://doi.org/10.1007/BF01867653. http://link.springer.com/article/10.1007/BF01867653

61. Machado EA, Stoms DM, Davis FW, Kreitler J (2006) Prioritizing farmland preservation cost-effectively for multiple objectives. J Soil Water Conserv 61(5):250–258. http://www.jswconline.org/content/61/5/250
62. DeOliveira E, Callum R (2004) Who's the best? Data envelopment analysis and ranking players in the national football league. In: Economics, management and optimization in sports, 1st edn. Springer, Berlin/Heidelberg, pp 15–30. https://doi.org/10.1007/978-3-540-24734-0
63. Hu ZH, Zhou JX, Zhang MJ, Zhao Y (2015) Methods for ranking college sports coaches based on data envelopment analysis and pagerank. Expert Syst 32(6):652–673. https://doi.org/10.1111/exsy.12108, eXSY-Jun-14-124.R1
64. Karminsky A, Polozov A (2016) Evolution of ideas about rating and ranking in sports. Springer International Publishing, Cham, pp 187–200. https://doi.org/10.1007/978-3-319-39261-5_7
65. Radicchi F (2011) Who is the best player ever? A complex network analysis of the history of professional tennis. PLoS One 6(2):1–7. https://doi.org/10.1371/journal.pone.0017249
66. Adler NJ, Harzing AW (2009) When knowledge wins: transcending the sense and nonsense of academic rankings. Acad Manag Learn Edu 8(1):72–95. https://doi.org/10.5465/AMLE.2009.37012181. http://amle.aom.org/content/8/1/72.abstract, http://amle.aom.org/content/8/1/72.full.pdf+html

Index

A
Ab initio quantum mechanics, 163
Acceleration constants, 856
Acceleration techniques, 1679
  sequential search, 1680–1681
  SMDs, 1681–1683
Acceptance criterion, 752, 757, 780, 781, 783, 784, 786–791, 793, 798
Acceptance iteration, 757
Accumulated information, 99
ACO, *see* Action selection model, 711, 713
Active guided evolution strategies (AGES), 443
Active search, 84
Ad hoc heuristics, 186
Ad Hoc strategies, 501–503
Adapt-CMSA, 373, 376–379, 382, 384
Adapted local search, 45
Adaptive ant colony optimization, 1405
Adaptive BRKGA (A-BRKGA), 931
Adaptive control of p_m, 646
Adaptive heuristic information, 551
Adaptive heuristics, 755
Adaptive large neighborhood search (ALNS), 12–14, 1328, 1398–1400
Adaptive local minima, 215
Adaptive memory algorithm, 1731
Adaptive metaheuristics, 10, 19
  adaptive large neighborhood search, 12–14
  AI, 20
  greedy randomized adaptive search procedure, 11
  reactive search, 11
  simple adaptive mechanisms, 11
Adaptive multi-mining structure, 48
Adaptive multi-start (AMS) heuristics, 215
Adaptive reasoning technique (ART), 217
Adaptive sampling method, 109
Adaptive starting solutions, 215
Adaptive stochastic gradient descent (ASGD), 1452, 1461–1464
Adaptive variable neighborhood search (AVNS), 1397
Adaptive VND, 528
ADATE, 18
Additive drift theorem, 1157–1159
adiabatic, 283
Advanced heuristic methods, 1609–1610
Advanced memetic algorithms, 819–820
Advanced traffic information system, 1204
AIMAll program, 168
Algebraic and logical constraints, 421
Algebraic constraints, 396
Algorithm-based parallelization strategies, 1096–1099
Algorithmic approach, 1020–1023
Algorithmic logic, 1096
Alternate optima, 1012–1014, 1028
Alumni attribute, 1846
AMACOL, 1731
Amazon Flex, 1386
Amino acids, 1297–1299, 1301, 1303, 1305
Analogy principle, 997
Analysis, 408
Analytic hierarchy process (AHP), 348
Annealing, 283–284, 999
Ant colonization algorithms, 17
Ant colony optimization (ACO), 126, 242, 444, 544–549, 572, 573, 765, 1001, 1098, 1111, 1286, 1287, 1340, 1405, 1471, 1482, 1731–1732
  ACO algorithmic framework, 548–560
  algorithms, 559
  and exact methods, 570–571
  and surrogate models, 571
  and tree search methods, 569–570
  continuous optimization problems, 567
  default parameter settings, 562

© Springer Nature Switzerland AG 2025
R. Martí et al. (eds.), *Handbook of Heuristics*,
https://doi.org/10.1007/978-3-032-00385-0

Ant colony optimization (ACO) (*cont.*)
dynamic problems, 568–569
empirical performance, 1485
for Max-Mean Model, 1286
global pheromone update, 553–556
global updating, 1485
initialization of pheromones, 557
as instantiations of ACO metaheuristic, 559–560
local pheromone update, 557–558
local updating, 1484
local search, 559
main source, 1482
multi-objective problems, 567–568
parameter adaptation, 571–572
pheromone limits, 558–559
pheromone reinitialization, 557
pheromone trails, 549–551, 1482
pheromone update schedule, 556
pseudocode of procedure, 1484
QAP, configuration for, 564–566
solution construction, 551–553
Ant colony system (ACS), 545, 552, 557, 558, 560, 561, 1286
Anticipatory routing problem, 1186
Apparent tardiness cost (ATC), 1609
Application programming interface (API), 931, 1284
Applications, 603
in machine learning, 593–594
in optimization, 592–593
ApplyLocalSearch(s_{tmp})procedure, 711, 712
Arborescence, 1795
Arc routing problem with stochastic demands (ARPSD), 353
Arithmetic invariant, 418
Artificial fitness levels (AFL), 1144, 1156–1157
for elitist populations, 1147
level-based analysis of non-elitist populations, 1152–1156
lower bounds, 1150–1152
upper bounds, 1145–1150
Artificial immune systems (AIS), 241
Artificial intelligence (AI), 149, 445–446, 960, 979, 998
adaptive metaheuristics, 20
AI-driven parameter tuning, 20
AI-enabled multi-level metaheuristics, 21
control flow, 20
Artificial neural networks (ANNs), 72, 151, 165, 701, 1035–1038
Artificial pheromone trails, 544
Aspiration criteria, 1200
ASSIGN neighborhood, 200
Assignment problems, 435
Associated reduced network, 1474
Asynchronous cooperation, 1109–1114, 1123
Atlas-based segmentation, 1460–1461
Atomic local frame (ALF), 168–170
Attribute-based decomposition of mathematical structure, 1115
Attributive memory, 944
Australian National University, 1844
AutoGraphiX (AGX), 983
Automated machine learning (AutoML), 1037, 1046
Automated parameter control, 700
Automatic design of algorithms, 695, 696
Automatic programming problem, 980
Automatic vehicle location (AVL), 1204
Autonomous systems (AS), 1529
Autoregressive construction methods, 80–81
Average diversity convergence, 1311
Average fitness convergence, 1311
Average job delay, 1430
Average percentage difference (APD), 60
Avogadro's number, 267

B

Back propagation (BP), 151, 152
Backtracking search, 320, 331
Bacterial foraging optimization algorithm (BFOA), 1457
Bandwidth minimization problem (BMP), 1338–1342
Barabási-Albert model, 375, 382, 383
Bare bones PSO (BBPSO), 875
Baseline construction algorithm, 748
Basic tabu search, 6
Basic variable neighbourhood search, 6
Basic VND (B-VND), 527, 529
Basic VNS (BVNS), 961, 964–967, 972–974, 1021, 1022
Bayesian network, 590, 591, 593–595
Bayesian Network Classifiers, 700
Bayesian optimization, 20
Bayesian optimization algorithm (BOA), 131, 590, 591
Bayesian statistics, 165
Bayes' theorem, 165
Bean's algorithm, 923
Berth allocation problem (BAP), 913–915, 1415
comparative analysis, 1419–1421
continuous BAP, 1416
discrete BAP, 1416

general heuristic framework, 1418
hybrid BAP, 1416
operational, 1416
strategic, 1416
tactical, 1416
Berth allocation problem under time-dependent limitations (BAP-TL), 914
Best bound first strategy, 1655
Best-fill (BF) algorithm, 1232
Best improvement strategy, 1721
Best-known solutions (BKS), 1431
Best-worst ant system (BWAS), 545, 557
Bias functions, 672–673, 927
Biased random-key genetic algorithm (BRKGA), 26, 29–32, 923, 925, 928, 932, 1033, 1049–1051, 1284, 1401, 1506
A-BRKGA, 931
APIs for, 931
BRKGA-CNN proposal, 1057–1064
BRKGA-DFNN proposal, 1051–1057
BRKGA-GNN proposal, 1064–1071
BRKGA-QL, 931
classical, 923
double elitism, 923
flowchart detailing BRKGA process, 924
implementation, 924–926
multi-start variant, 924
optional procedures of framework, 926–927
parameters and recommended ranges, 930
pseudo-code, 924
traditional, 929
Biased randomized GRASP (BR-GRASP), 672
Biased randomized version, 1404
Biased random-key genetic programming (BRKGP), 26, 28, 29, 36–38
Binary problems
neighborhoods for, 516–518
Binary PSO (BPSO), 307, 874
Binary valued quantum inspired genetic algorithm, 297
Binomial random variable, 1137
Bin packing, 693, 695, 699, 700, 707, 717, 719, 730, 1241
Bi-objective heuristics, 1812
Bi-objective minimum diameter-cost spanning tree (bi-MDCST), 1812
NSGA-II for, 1813–1815
solutions for, 1813
Bi-objective optimization, 1812
Bioinformatics, 592
Biological analogy, 108
Biomedicine, 592
Bipartite graph, 1497
Bivariate dependencies, 589
Bivariate marginal distribution algorithm (BMDA), 589, 590
Blackboard, 1105
Black box optimization, 694
Black-box optimization benchmark (BBOB), 111–113
Black-box optimization problem, 865
Black-box optimizers, 1002
Black widow-optimized convolutional long short-term memory network (BWO-Conv-LSTM), 154–156
classification, 158
feature extraction and selection, 158
model and methodology, 156–158
optimization of hyperparameters, BWO, 158–160
oversampling, 157
pre-processing, 158
results, 160–162
Blind search, 171
BLX-α, 645
Boltzmann factor, 269, 291
Bonferroni correction method, 727
Boolean circuit, 493, 494
Boolean expressions, 141
Boolean optimization problem (BOOP), 18
Boolean parameter, 370
Boolean variable, 1661
Bootstrap, 220, 223, 225, 226
Bortfeldt and Gehring's solutions, 1239
Boruvka's algorithm, 1795
Bottom-left (BL) algorithm, 1231, 1232
Bottom-left-descending (BLD) algorithm, 1236
Bottom-left-fill (BLF) algorithm, 1231, 1232
Bottom-left placement rule, 1248, 1250, 1251
Bottom-left-right (BLR), 1237
Bounded-diameter minimum spanning tree (BDMST), 1796, 1811, 1812, 1815
Box configuration, 192
Box-Jenkins dataset, 983
BrainWeb, 1458
Branch-and-bound (BB) algorithm, 1551, 1655
Branch-and-price algorithm, 1390
Branch-and-price/column generation-based approaches, 1368–1369
Branch coverage, 1632
Breadth-first search (BFS), 1801
Breakout local search (BLS), 1727
BRKGA-CNN, 1057, 1072, 1076
comparative models, 1073
datasets, 1072–1073
decoder, 1061–1062

BRKGA-CNN (*cont.*)
experimental configurations, 1073
fitness evaluation, 1062–1064
representation, 1057–1058
representation of individuals, 1058–1059
BRKGA-DFNN, 1051, 1071, 1075–1076
comparative models, 1072
datasets, 1071
decoder, 1054–1056
experimental configurations, 1072
fitness evaluation, 1055
initialization of individuals, 1054
representation of feedforward neural networks, 1051–1054
BRKGA-GNN, 1077
activation functions, 1066
aggregation strategies, 1065
attention mechanisms, 1065
comparative models, 1074
datasets, 1074
decoder, 1068–1070
experimental configurations, 1074
fitness evaluation, 1070–1071
representation of graph neural networks, 1064–1066
representation of individuals, 1066–1068
training hyperparameters, 1066
BRKGACuda 2.0, 932
Brooks' theorem, 1718
Broyden–Fletcher–Goldfarb–Shanno method, 1452
Bucket elimination (BE), 823

C
Candidate list (CL), 552, 943, 1281, 1287, 1683
Candidate swap, 1404
Canonical 1-factorization, 1498
Canonical particle swarm, 856
Capacitated clustering problem (CCP), 1273, 1285
Capacitated vehicle routing problem (CVRP), 49–55, 83, 905, 910–911, 1014, 1354, 1355, 1357, 1359–1362, 1364, 1368–1371, 1374–1377, 1571, 1600, 1674–1677, 1680, 1686, 1687, 1691, 1705, 1774–1777
Capacitated vehicle routing problem with time windows (CVRPTW), 1776
Capacity constraint, 1775
Car sequencing, 410–413, 423
Cardinality-based (CB) mechanism, 667
4-Cardinality tree problem, 966
Cartesian coordinate, 169
Cellular GA (cGA), 647
Central processing units (CPUs), 1098, 1099
Central-memory asynchronous cooperation strategies, 1113
Central-memory mechanisms, 1116
Central-memory-based cooperative search, 1112
Centralized-memory mechanisms, 1105
Ceschia and Schaerf's movement, 1235
Chao1984 method, 221, 225
Chao&Bunge method, 221, 223, 225–228
Chao&Lee methods, 221
Chao&Lee2 method, 223, 225, 226, 228
CHARMM package, 1301
CHC algorithm, 644
Chernoff bounds, 1143, 1144
CHeSC 2011 competition, 730
Christofides' algorithm, 478
Chromatic index, 1497
Chromatic number, 1716, 1736
Chromosomes, 31, 629
Circle method, 1495, 1496, 1498–1500, 1512
Circle packing problems (CPP), 971
Circuit design, 1327
City distribution centers (CDC), 1182
City logistics
computation of feasibility, 1205–1207
definition, 1177
DINS, 1208–1210
DTS, 1211–1213
environmental concerns and sensitivities, 1177
extensions, 1213–1215
feasibility, 1210
fleet management and ICT applications, 1183–1187
heuristic approach, 1208
operational decisions, 1194
organizational aspects and decision-making, 1180
PDVRPTW, 1194–1203
profitability, 1210
real-time management, 1203–1204
relevance, 1177–1178
spatial restrictions, 1177
strategic decisions, 1187
systems approach, 1178–1180
traffic infrastructure, 1177
two-echelon location routing, 1193–1194
two-echelon single-source location problems, 1187–1193
City terminals (CT), 1182
C language, 188, 196

Classic greedy search, 394
Classic IR algorithms
  gradient descent, 1452
  ICP, 1451–1452
Classical constructive heuristics, 1356
Classical evolution strategies, 92–97
Classical force field (CFF), 163
Classical heuristics, 1356
  classical constructive heuristics, 1356
  classical improvement heuristics, 1357–1358
Classical mechanics, 262, 264
Classical memetic algorithm, 813–814
Classical OCs, 1391
Clique, 1716, 1717, 1719–1730, 1732–1735, 1738
  definition, 1716
  maximum clique (*see* Maximum clique)
  number, 1716
Closed under scaling, 482
Closest string problem (CSP), 1652, 1654–1658, 1663, 1664
Close to most string problem (CTMSP), 1652, 1659, 1663–1665, 1667
Clustering, 591, 594, 911
Clustering adaptive multi-start (CAMS) method, 216
Clustering problem, 1546
Coarsening, 1113
CODE family, 1736
Coevolutionary algorithms, 137
Cognition-only particle swarm optimization, 1569
Cognitive term, 856
Collaborative hybrid metaheuristics, 648
Collaborative relay hybrid GAs, 650
Collaborative teamwork hybrid GAs, 649
Collective hill climbing, 91
Collegial, 1094
Colorful chordless lantern, 1508
Coloring, 1716–1721, 1723–1726, 1730–1732, 1734, 1735, 1738
Column generation (CG), 1553
Combination strategies, 604
Combinatorial constraints, 422
Combinatorial optimization, 139, 213, 215–218, 364, 365, 367, 370, 602, 607, 622, 693, 695, 707, 734, 961, 1012–1014, 1028
Combinatorial optimization problems (COPs), 72, 219, 221, 546, 547, 664, 1495
Combinators, 391
Combining optimizers with mutual information trees (COMIT), 589, 594
COMET, 391
Commercial solver (CPLEX), 1396, 1397, 1400–1403, 1405
Commercial solver (Gurobi), 1405
Communication graph, 1105
Commutator, 272, 274
Compact genetic algorithm (CGA), 131, 589
Compaction problem, 1258
Complement graph, 1716
Complementary heuristics, 812
Complete bipartite graph, 1497
Complexity penalty components, 167
Complexity theory, 993
Comprehensive learning PSO (CLPSO), 880–881
Compressor station, 1474
Compressor unit, 1483
Compstatin, 1302
Computational complexity, 489–493
Computational complexity of stochastic search algorithms, 1134–1135
Computer programs, 133
Configuration, 5–10, 14, 16, 19–21
Conflict minimization, 1438
Congestion, 1323, 1325, 1329
`ConstructSolution`, 213
Conservative force, 265
Consistent variable, 216, 673
Constrained GAs, 654
Constrained optimization problem, 436, 460, 870–872
Constraint-based local search, 390–393, 424
  basics, 395
  car sequencing, 410–413
  constraint model, 400
  control primitives, 402, 403
  differentiation, 398, 420–423
  empirical results, 423–424
  evaluations and violations, 396–398
  extensibility, 392
  feasible solution, 401
  gradient, 398
  implementation, 416–423
  invariants, 416–420
  legal neighbors, 402
  model, 392, 395–401
  neighbor selection, 402
  neighborhood, 401
  neighborhood selectors, 403
  objective functions, 399
  optimal solution, 401
  performance, 392
  programs, 401–405
  progressive party, 406–410

Constraint-based local search (*cont.*)
queens problem, 393, 394
randomized choosers, 404
rich language, 391
rich search, 391
scene allocation, 413–415
search procedure, 401
separation, 391
variable violations, 399
versatility, 391
Constraint conversion, 397
Constraint-handling, 245
Constraint interface, 399
Constraint programming (CP), 390, 413, 444, 570
Constraints, 395
Constriction coefficient, 856
Constriction particle swarm optimization, 1568
Construction, 79, 431, 750, 1717–1720, 1779
autoregressive, 80–81
non-autoregressive, 81–82
post-processing, 1779
VNEP, 1537–1538
WSP, 1532–1533
Constructive algorithms, 748
Constructive-based metaheuristics, 513
Constructive heuristics, 13, 43, 548, 1230, 1247–1253, 1356, 1496, 1797, 1801, 1816
adaptive, 548
item-driven algorithms, 1231–1232
space-driven algorithms, 1232–1234
static, 548
Constructive mechanisms, 766
Constructive randomized method, 610
Constructive search, 763, 766
Construct, merge, solver & adapt (CMSA), 364, 384, 1662
general framework, 364
key features, 365
maximum independent set, 370–383
reinforcement learning, 368–370
self-adaptive CMSA, 367–368
standard CMSA, 366
timeline of CMSA applications, 370–371
variants, 365–367
Container, 1412
heuristic framework to store and retrieve, 1437
relocation problems, 1436–1437, 1439
storage, 1433–1437
target positions, 1437–1439
transshipment flows of, 1414
Container storage, 1433
computational analysis, 1439
container relocation problems, 1436–1437, 1439
container target positions, 1437–1439
yard crane scheduling problem, 1434–1435
Context information, 1105
Context vector, 879
Continuous BAP, 1416
Continuous global optimization, 977–979
Continuous GRASP (C-GRASP), 676–678
Continuous optimization, 515–516, 567, 961
Contour matching, 132
Control flow, 4, 16, 20
Conventional weighted aggregation (CWA), 888
Convolutional neural networks (CNN), 73, 155, 156, 158, 159, 163, 1033
Cooling schedule, 291
Cooperating local search (CLS), 1727
Cooperation mechanisms, 1103
Cooperative PSO (CPSO), 879–880
Cooperative search, 1103–1107, 1114–1122
Correlated mutations, 99
Correlation coefficients, 1275
Corridor method (CM), 915
Cost explicit, 483
Cost-independent decomposition, 1680
Cost perturbations, 671–672
Counting invariant, 419
Covariance function, 164, 166
Covariance-Hessian relation, 105
Covariance matrix, 102, 105, 166
Covariance matrix adaptation (CMA-ES), 130
Covariance matrix adaptation evolution strategy (CMA-ES), 91, 92, 101–103, 111, 118
Coverage problem, 1545
CPLEX, 188, 197, 198, 975, 976, 1420, 1663, 1666, 1669
CPLEX v.12.6, 1755, 1757, 1758
Critical difference (CD) plots
Barabási-Albert graphs, 382
Erdös-Rényi graphs, 381
Watts-Strogatz graphs, 381
Crowding distance, 238, 1814
Crowdshipping, 1387, 1392, 1406
compensation strategies, 1390
dynamic and stochastic information integration, 1389–1390
mathematical formulation, 1387–1389
transshipment points integration, 1391
Crowdsourced delivery scheduling problem, 1404

Crowdsourcing multi-depot vehicle routing problem with time windows, 1405
Crowdsourcing task allocation, 1403
Cumulative distribution functions (CDF), 324
Cumulative global step-size adaptation, 101
Curie temperature, 283
Customer classification genetic algorithm (CCGA), 1404
Customer relationship management (CRM), 1704
Customer service management (CSM), 1705
Cutting and packing (C&P) problems, 688, 695, 733, 734, 1220, 1221, 1263
  additional characteristics and constraints, 1227–1230
  dimensionality, 1221–1222
  geometry, 1222–1225
  irregular problems (*see* Irregular problems)
  problem types, 1225–1229
  cutwidth graph minimization problem (CWP), 973
  cutwidth minimization problem (CMP), 521, 1325–1329
  cyber-attacks, 155
  cycle detection mechanism, 11
  cycle elimination algorithm, 1796
  cycle time simulator (CTS), 348
  cyclic bandwidth problem (CBP), 1341
  cyclic bandwidth sum problem (CBSP), 1332
  cyclic cutwidth minimization problem (CCMP), 1328
  cyclic topology, 1474, 1481
  cyclic VND (C-VND), 527, 529
  cyclic-independent decomposition, 1680
Cut-values, 1326

D
Darwinian principle, 923
Darwin's theory of evolution, 922
Data mining, 42, 49, 68, 149, 150
  DM-MSH and MDM-MSH strategies, 43–45
  DM-supported HGS for CVRP, 49–55
  DM-supported MS-ILS for HFVRP, 55–63
  DM-supported MS-ILS for MS-CFLP-CI, 63–65
  MineReduce strategy, 46–48
Data mining-based feature selection procedure, 695
Data-fit, 167
Decision factor (DF) heuristic, 1038
Decision support system (DSS), 347, 1184, 1214, 1215
Declarative semantics of invariant, 417
DecodeHead procedure, 33
Decoder, 30, 922
DecodeTail procedure, 34
Decoding of a chromosome, 32, 33
Decoding procedure, 922, 923, 928, 929
Decoherence, 272, 279
Decomposition, 1091–1093, 1364–1367, 1677–1678
Deep adaptive greedy search (DAGS), 1719
Deep feedforward neural networks (DFNNs), 1033
Deep learning (DL), 151, 152, 155, 156, 162, 174, 654
  BRKGA in, 1051–1071
  construction methods, 79–82
  deep-learned construction and evolutionary neural networks, 1044–1045
  deep neural networks, 73–74
  deep parameter control, 76–79
  improvement policies, 79–84
  graph neural networks, 74–75
  improvement methods, 82–83
  metaheuristics, 1045–1046
  pointer networks, 75
  swarm and evolutionary computing approaches, 1040
  transductive methods, 84
  transformer architecture, 75–76
Deep learning heuristic tree search (DLTS), 1440
Deep neural networks (DNNs), 72–74, 85, 1032, 1034, 1035, 1077
  advantages, 72
  basic form, 73
  neuroevolution, 1042
  optimization, 1039
  survey on evolutionary construction, 1043–1044
Deep parameter control, 76
  dynamically configurable parameters, 78
  learning control policies, 77
  representing search state, 78–79
Deep reinforcement learning (DRL), 1489
Define Secondary Structure of Proteins (DSSP), 1305
Deflection technique, 870

Degree-constrained minimum spanning tree (DCMST), 1796, 1803
Lagrangian heuristic for, 1809–1811
problem formulation, 1803–1805
subgradient procedure for DCMST problem, 1805–1808
Degree-constrained spanning tree (DCST), 1803
greedy heuristic for, 1806–1808
Kruskal-based heuristic for, 1808
with local search, 1808–1809
Delta potential well (DPW), 876
Denial of service (DoS) attacks, 154
Density control problem (DCP), 1546
Depth-first search (DFS), 1795, 1801
Derandomization, 92, 98
Derandomized ES algorithms, 99
Derandomized evolution strategies, 98
first level of derandomization, 99–101
indirect selection, 98
realization of parameter variation, 98
second level of derandomization, 101–103
Derandomized mutative step-size control, 99
Derandomized self-adaptation, 91, 97
Derandomized step-size control, 97
Design of experiments (DOE), 758
Destination quay crane/stack, 1427
Destructive heuristics, 13
Deterministic control of p_m, 645
Differentiability, 416
Differentiable architecture search (DARTS), 1042
Differential evolution (DE), 127, 131, 1455
Differentiation, 398, 420–423
Diffusion, 1107, 1109
Digital quantum computers, 275
Dijkstra's algorithm, 997
Dilation, 1323, 1338
Dimensionality, 1221–1222
Direct encoding, 1730
Direct exchanges of information, 1105
Directed acyclic graphs (DAGs), 74, 1826
Directed profitable rural postman problem (DPRPP), 1367
Discharging job, 1427
Discrete BAP, 1416
Discrete (dominant) recombination, 95
Discrete EDAs, 589
Discrete parameters, 78
Dispatching rules, 1601, 1617
elaborated, 1609
simple, 1607–1608
Dispersion measures, 1278
Disruptive selection, 642
Distance first algorithm (DFA), 1658
Distance-constrained capacitated vehicle routing problem (DCVRP), 200
Distance matrix, 1275
Distributed algorithms, 1544
Distributed GAs (DGAs), 646, 647
Distributed guided local search (Dis-GLS), 444
Diverse subset, 1276
Diversification, 412
Diversification generator method (DGM), 605, 609–610, 613
Diversification phase, 1200
Diversification techniques, 641
control parameters, 645–646
crossover operator, 644–645
raw diversity, 641–642
spatial separation, 646–647
strategies to maintain diversity, 642–644
Diversification, refining and tight-refining (DRT), 916
Diversity as objective (DAO), 1317
Diversity measures, 1278–1280
Diversity models, 1272
Diversity preservation, 1299–1300
Divide-and-conquer principle, 1677
DM-supported HGS, for capacitated VRP, 49
computational results, 51–55
data mining application, 50–51
problem definition, 49–50
DM-supported MS-ILS, for heterogeneous fleet VRP, 55
computational results, 60–63
data mining application, 57–58
problem definition, 57
DM-supported MS-ILS, for multi-source capacitated FLP with customer incompatibilities, 63–64
computational results, 65
data mining application, 64
problem definition, 64
Domain-specific heuristics, 1342
Dominance graph, 1837
Dotted-board-model, 1259, 1260
Double round-robin tournament (DRR), 1495
Drift analysis, 1157, 1166
additive drift theorem, 1157–1159
multiplicative drift theorem, 1159–1162
negative-drift theorem, 1164–1166
variable drift theorem, 1162–1164
DSATUR, 1718, 1723, 1732
Dual Lagrangian problem, 1798
Dual representation simulated annealing algorithm (DRSA), 1341

Dynamic algorithm configuration (DAC), 77, 79
Dynamic and stochastic vehicle routing problem, 1398
Dynamic berth allocation problem (DBAP), 914, 1415, 1417, 1419, 1421
  recent heuristics developments, 1421–1422
  tabu search with path relinking, 1420
Dynamic fine-grained, 1097
Dynamic granular neighborhood, 1684
Dynamic graphs, 1342
Dynamic insertion heuristic (DINS), 1208–1210
Dynamic local search (DLS), 1726, 1727
Dynamic Many-Objective Sorting Algorithm (DynaMOSA), 1633
Dynamic optimization, 137, 654, 698
Dynamic problems, 568–569
Dynamic programming (DP), 1477, 1478, 1485
Dynamic tabu search (DTS), 1211–1213
Dynamic vehicle routing problem (DVRP), 1185, 1571
Dynamic weighted aggregation (DWA), 888
Dynamical systems analysis, 105

E

(1 + 1) EA, 1137, 1163
Earliest due date (EDD), 1418, 1608
Earliest due date greedy (EDD-G), 1418
E-commerce, 1386, 1709
Edge coloring, 1497, 1498, 1500
Edge drop, 1798–1801, 1815
Edge insertion, 1798–1801
Edge projection, 1735
Edgeworth-Pareto optimality, 233
Efficient active search, 84
Eigen-decomposition, 103
Elementary Shortest Path Problem with Resource Constraints (ESPPRC), 1369
Elite-based GLS (EB-GLS), 445
Elite set, 601, 922
Elite solutions, 673
Elitist ant system (EAS), 545
Elitist steady-state genetic algorithms, 1138
Elitist truncation selection, 1136
Embedding, 1323, 1325, 1338
Empirical analysis, 471
Empirical cumulative distribution function (ECDF), 111, 112
Empowerment scheduling, 455
Encoding and chromosome structure, 31
Energy, 264, 592
Engineering applications, 242
Entanglement, 271, 272, 278, 279
Equal-Size Cluster Approach (ECA), 1637
Equity models, 1271–1273
Erdös-Rényi graphs, 381, 383
Erdös-Rényi model, 375
Ergodicity, 269
Eshelman's CHC algorithm, 641
Estimate distribution algorithm (EDA), 443, 459
Estimation of Bayesian network algorithm (EBNA), 590, 594
Estimation of Distribution Algorithm and Tabu Search (EDA-TS), 1287
Estimation of distribution algorithms (EDAs), 127, 131, 307, 584, 586, 595
  discrete, 589
  general scheme, 587–590
  initialization process, 588
  joint probability distribution, 589–590
  machine learning, 593–594
  optimization, 592–593
  practical issues, 591–592
  pseudocode, 588
  sampling of new candidate solutions, 590
  selection of individuals, 589
Estimation of multivariate normal algorithm (EMNA), 590
Euclidean distance, 1277, 1765
Euclidean metric, 1276
Euclidean TSP, 471, 472, 501
Eulerian graph, 478
Events, 413
Evolutionary algorithms (EAs), 8, 26, 90, 91, 234, 235, 629, 812, 817, 852, 853, 1001, 1033, 1132, 1135–1139, 1728–1731, 1800–1802
  alternative representations, 133
  co-evolution gene expression, 141
  coevolutionary algorithms, 137
  coevolving solutions, 137, 138
  combinatorial optimization, 139
  common variants, 132
  differential evolution (DE), 131, 132
  dynamic optimisation, 137
  estimation of distribution algorithms, 131
  evolution strategies and evolutionary programming, 130
  genetic algorithms, 128, 129
  hybridisation with local search, 134
  methodology, 138
  multimodal optimisation, 135, 136
  multiobjective optimisation, 135, 137
  niching and co-evolution, 140, 141

Evolutionary algorithms (EAs) (*cont.*)
open source frameworks, 139, 140
parameters, 138
Parkinson's disease diagnosis, 142
performance comparisons, 132
software tools, 139
Evolutionary algorithms for inverse protein folding problem
algorithm design, 1307
algorithm results, 1308–1311
diversity measure, 1306–1307
diversity preservation, 1299–1300
experimental setup, 1308
multimodal optimization and niching, 1302–1303
primary and secondary structure validation results, 1312–1314
problem description, 1303
protein design, 1300–1302
protein samples, 1308–1309
quantile constraint, 1308
removal of doubles, 1307
secondary structure, 1305–1306
sequence identity, 1304
tertiary structure validation results, 1314–1316
Evolutionary biology, 1014
Evolutionary computation (EC), 443, 1040
Evolutionary deep learning (EDL), 1047–1048
Evolutionary local search (ELS), 218
Evolutionary methods, 601
Evolutionary multi-objective optimization (EMO), 232
Evolutionary multitask optimization, 1401
Evolutionary multitasking algorithm (EMA), 245, 653, 1401
Evolutionary neural architecture search (ENAS), 1048–1049
Evolutionary neural networks (ENNs), 1043–1045
Evolutionary optimization, 600
Evolutionary population-based metaheuristic, 814
Evolutionary programming (EP), 130, 999
Evolution strategies (ESs), 90, 91, 94, 111, 117, 127, 130–132, 139, 443, 458
accessibility of points by mutation, 107
causal representation, 106
classical, 92–97
contemporary, 91
derandomized, 98–103
experimental settings, 112
non-standard, 106–111
results, 112–117
scalability of mutation, 107
theoretical results, 104–106
unbiasedness of mutation, 107
unimodal mutation distributions, 107
Exchange, 522
Expanding neighborhood search (ENS), 1577
Explainable AI (XAI), 1079
Explicit memory, 944
Explicit search-space decomposition, 1092, 1099–1102
Explicit search-space multiple-control decomposition L/SE/pC/S strategy, 1101
Explicit search-space single-control decomposition L/SE/1C/S/ strategy, 1101
Exponential bias, 672
Exponentially weighted moving average, 102
Expression evaluation, 396
Extended compact genetic algorithm (EcGA), 590

F

1-factorization, 1497
Fair Maximum-Minimum Diversity (FDM), 1281
Far from most string problem (FFMSP), 1652, 1660–1665, 1668, 1669
Farthest string problem (FSP), 1652, 1659–1660, 1663, 1666
Fast local search (FLS), 437–440, 443
Feasible set, 961
Feasible solution, 961
Feature models (FMs), 1628, 1630
Feature-based techniques, medical IR
differential evolution, 1455
genetic algorithm, 1455
particle swarm optimization, 1455–1456
scatter search, 1456
Feature subset selection, 593
Feedforward neural networks (FNNs), 1038–1039
FEREBUS, 154, 163–165, 167
FFLUX, 163
Field theory, 263
Filled functions, 868
"Flip" move, 945
Finance, 351
Financial sustainability, 1790
Fine-tuning process, 84
First come first served (FCFS), 1418, 1428, 1607
First come, first served greedy (FCFS-G), 1418

First improvement strategy, 1721
First law of thermodynamics, 268
First level of derandomization, 99–101
First repetition time, 220
Fisher information, 105
Fitness approximation, 244
Fitness-based partition, 1144, 1145
Fitness Euclidean-distance ratio (FER), 882
Fitness Euclidean-distance ratio PSO (FERPSO), 882
Fitness function, 35, 1135
Fitness uniform selection scheme (FUSS), 642
Fleet utilization, 1430
Flexibility optimization, 1438
Flexible memory, 1000
Flow shop (FS), 717, 720, 1602
Flowshop scheduling problem, 222
Force, 265
Forced optimal covariance adaptive learning (FOCAL), 109
Forests, 1794–1798, 1806, 1816
Formulation change function, 971
Formulation space search (FSS), 961, 971
Forward insertion heuristic, 1397
Forward time slack, 1397
Frequency-based memory, 948, 949, 1200
Friedman test, 727
Frustration, 284
Fuel consumption cost, 1473
Fully informed PSO (FIPS), 876
Functional parallelism, 1092
Functions, 27
Functions of unitation, 1139, 1144, 1158
Fuzzy genetic algorithm (FGA), 1328
Fuzzy logic system, 344, 1328

G

GAMAS, 647
Games and education, 705
Gantt chart, 1624
Gap unitation block, 1140
Gas transmission network problems, 1472
Gate operations planning, 1415
Gaussian distribution, 95, 165
Gaussian mixture noise model, 164
Gaussian noise, 887
Gaussian process (GP), 152, 165
Gaussian process regression (GPR), 150–152, 154, 163–167, 170–173
Gaussian-like likelihood, 164
Gbest model, 864
Gene expression programming (GEP), 26
Generalized assignment problem, 459
Generalized max-mean dispersion problem, 1272
Generalized opposition-based PSO (GOPSO), 886
Generalized partial team swap (GPTS), 1510
General-purpose heuristic solvers, 1006
General relativity, 263
General variable neighborhood search (GVNS), 533, 961, 966, 1332, 1335
Generational genetic algorithm, 639, 1136
Generational loop, 93
Generational with elitism, 639
Generation hyper-heuristics, 15
Generation set adaptation (GSA), 100
Generic local search procedure, 669
Generic memetic algorithm, 813
GENEsYs, 1728
GENET, 429, 445
Genetic algorithms (GAs), 9, 127–129, 142, 443, 458, 459, 600, 601, 628, 629, 655, 695, 703, 810, 812, 821, 922, 979, 1049, 1111, 1237–1239, 1288, 1289, 1300, 1301, 1307–1311, 1317, 1400, 1403, 1405, 1455–1457, 1505–1506, 1633, 1656, 1657, 1728–1731, 1802
  applications, 652–655
  construction, 630–641
  crossover operator, 635–636
  diverse solutions, 632–633
  diversification techniques, 641–647
  evaluation, 633
  evolution model and replacement strategy, 638–640
  genetic operators, 634–635
  heuristic solutions, 633
  hybrid GAs, 648–649, 651
  initial population, 632–633
  mutation operator, 637–638
  random solutions, 632
  representation, 631–632
  selection operator, 634
  stop condition, 640
Genetic operators, 634–635
Genetic programming (GP), 15, 26, 27, 127, 133, 139, 142, 690, 698, 701, 702, 706, 979, 980
Genetic repair effect, 105
Genotype-phenotype mapping, 133
Genotypes, 629
Geometric information, 1258
Geometry, 1222–1225
Gibbs-King algorithm, 1337
Global convergence, 105
Global distance test (GDT), 1316

Global exchanges of information, 1107
Global learning rates, 97
Global minimum, 962
Global optimization, 212–215
Global optimization problem (GOP), 515, 977, 978
Global pheromone update, 553–556
Global Positioning System (GPS), 1545, 1552
Global PSO model, 864
Global step-size, 100, 101
Gomory-Hu tree method, 1330
Goods distribution centers, 1182
Google Scholar, 602
Gradient-based learning algorithms, 151
Gradient descent, 1452
Gradients, 398
Granular neighborhoods, 1683–1685
Granular tabu search (GTS), 1403
Graph, 1322, 1323, 1332, 1335, 1340
Graphical processing units (GPU), 1096
Graph layout problems, 1322, 1342
  BMP, 1338–1342
  CMP, 1325–1329
  MinLA, 1329–1332
  SumCut/profile problem, 1336–1338
  VSP, 1333–1336
Graph model for single round-robin tournaments, 1498–1500
Graph neural networks (GNNs), 74–75, 1033
Graph Product Line (GPL), 1628
Graph theory, 1497–1498
Greedy construction heuristics, 748–749
Greedy heuristics, 997
Greedy methods, 762, 763
Greedy partition crossover (GPX), 1731
Greedy randomized adaptive search procedure (GRASP), 11, 49, 548, 609, 610, 612–616, 618–622, 664, 665, 746, 749, 752, 764, 794, 1001, 1119, 1241, 1280, 1281, 1283–1285, 1287, 1289, 1290, 1327, 1396, 1404, 1503–1504, 1661, 1662, 1722, 1726
  bias functions, 672–673
  construction phase, 666–668
  continuous GRASP, 676–678
  cost perturbations, 671–672
  hybridizations, 673–676
  industrial applications, 665
  local search phase, 669–670
  operations research problems, 665
  pseudo-code, 665, 667
  reactive GRASP, 670–671
Greedy selection rule, 997
Green vehicle routing problem with passing-by couriers, 1397
Grey wolf optimizer (GWO), 165
Grid data set, 1329
Grouping representation, 1729
Guaranteed convergence PSO (GCPSO), 874
Guided fast local search (GFLS), 438, 440–442, 453, 461
Guided genetic algorithm (GGA), 443, 459, 460
Guided local search (GLS), 429–431, 438, 461
  and artificial intelligence, 445–446
  assignment problems, 435, 459–460
  augmented objective function and move evaluations, 433–434
  commercial packages, 461
  constrained optimization problem, 436, 460
  construction method, 431
  extensions, 442
  hybrids, 443–444
  implementation guideline, 431–434
  improvement method, 431
  indicator functions and feature penalization, 433
  for multi-objective optimization, 446–450
  packing and placement problems, 436–437
  parameter λ, 433
  penalization scheme, 461
  possible features for common applications, 434–437
  principles, 429
  pseudo-code, 431
  resource allocation and scheduling problems, 460
  resource allocation problems, 436
  routing/scheduling problems, 435, 457–459
  stopping criterion, 434
  TSP, 450–453
  variations, 444–445
  WSP, 453–457
Guided Pareto local search (GPLS), 447–449
Guided tabu search (GTS), 444
Guided VNS (GVNS), 443
Guiding solutions, 955
Guillotine cutting pattern, 1230

H

Hamiltonian, 265, 266, 269, 273, 274, 281, 284, 291, 292, 294
Hamiltonian circuit, 1677
Hamiltonian cycle (HC), 474, 476, 477, 787, 1497

Hamming distance, 494, 644, 1654, 1658
Harmonic oscillator, 877
Harwell-Boeing data set, 1329
Harwell-Boeing sparse matrix collection, 1341
Head-tail representation, 31, 38
Healthcare, 350
Heaviest *k*–sub-graph (HSP) problem, 1284
Heterogeneous block, 1234
Heterogeneous decomposition, 1677
Heterogeneous fleet vehicle routing problem (HFVRP), 55–63
Heuristic scheduling
  advanced heuristic methods, 1609–1610
  application to parallel machine problem, 1608–1609
  basic, 1607
  elaborated dispatching rules, 1609
  iterated greedy, 1615–1616
  NEH heuristic for permutation flow shop and makespan criterion, 1610–1612
  relevance, 1600–1601
  shifting bottleneck heuristic for job shop problem, 1612–1613
  simple dispatching rules, 1607–1608
Heuristics, 152, 179, 187, 194, 196, 198, 391, 810, 1088, 1414, 1416, 1418–1422, 1428, 1430, 1431, 1439, 1440, 1496, 1600, 1601, 1607, 1610, 1611, 1613, 1614, 1617, 1675–1677, 1685, 1692, 1735, 1796–1798, 1801, 1808, 1809, 1811, 1812, 1814, 1816
  algorithm, 152
  approach, 1208
  construction, 1717–1720
  on continuous formulations, 1733–1735
  driven NN design, 1037
  feasibility operator, 1729
  information, 544, 550
  local search, 781–783
  metaheuristics (*see* Metaheuristics)
  methods, 1551–1552
  MIP-based, 179–182
  search, 428, 434, 461, 957
Hexaly template, 1776–1778
Hierarchical structures, 1105
High data-exchange frequency, 1110
Hilbert space, 270
Hill-climbing method, 1331
History of metaheuristics, 993
  early period, 994, 996–998
  framework-centric period, 994, 1002–1004
  metaphor-centric period, 1004
  method-centric period, 994, 998–1002
  pre-theoretical period, 994–996
  scientific period, 994, 1005–1007
Home-away swap (HAS), 1507
Homogeneous decomposition, 1677
Homogenous block, 1234
Host-based IDS (HIDS), 155
Hub-and-spoke telecommunication network, 351
Humanities, 1824
Human problem-solving behavior, 998
HybGenPSO, 1576–1577
HybPSO, 1577
Hybrid adaptive large neighborhood search (HALNS), 1398, 1399
Hybrid algorithms, 1726
Hybrid BAP, 1416
Hybrid genetic algorithm, 345, 648
  collaborative relay, 650
  collaborative teamwork, 649
  integrative relay, 651
  integrative teamwork, 650–651
Hybrid genetic search (HGS), 49–55, 1360
Hybrid heuristics, 1403, 1555
Hybrid heuristics with data mining
  DM-supported HGS for CVRP, 49–55
  DM-supported MS-ILS for HFVRP, 55–63
  DM-supported MS-ILS for MS-CFLP-CI, 63–65
Hybridizations, 673–676
Hybridized metaheuristics, 811
Hybrid metaheuristics, 648, 1002
Hybrid PSO, 884–885
Hybrid shop environments, 1603
Hybrid SLS methods, 793–794
HyFlex framework, 707
HyFlex state-of-the-art hyper-heuristics, 730
Hyper-heuristic approach, 14–16, 18, 134, 1341
  action selection model, 713–716
  automated parameter control, 700, 701
  automatic design of algorithms, 695–698
  dynamic and uncertain environments, 698
  educational timetabling applications of, 705
  experimental protocol, 716, 718
  games and education, 705
  HyFlex framework, 707
  HyFlex state-of-the-art hyper-heuristics, 730, 731
  iterated local search with adaptive heuristic selection, 708, 710, 712, 713

Hyper-heuristic approach (*cont.*)
machine learning methodologies, 699–700
for metaheuristic configuration, 16–18
recent advances in field of, 691
recent advances on the methodology of, 692–695
scheduling problems, 701–704
Hyperparameter optimization (HPO), 111, 164, 1033, 1034, 1039, 1048, 1077–1079
Hypervolume, 239

I
ILP formulation, 1655
ILP model, 1656, 1658
Image registration (IR), 1446
atlas-based segmentation of brain deep nuclei, 1460–1461
characteristics, 1447
classic IR algorithms, 1451–1452
experimental study, 1458
metaheuristics-based image registration, 1452–1457
optimization procedure, 1451
problem statement, 1447
registration of brain magnetic resonance images, 1458–1459
similarity metric, 1449–1451
transformation model, 1448–1449
Implicit path-relinking, 927
Implicit search-space multiple-control decomposition L/SI/pC/S strategy, 1103
Implicit SSD, 1093, 1102–1103
Improvement heuristics, 1367–1368, 1797, 1798, 1816
Improvement method (IM), 605, 612
Improvement post-processing, 1779
Incorporation of user's preferences, 244
Incremental computation, 419
Incremental graph, 417
Independence number, 1716
Independent multi-search, 1093, 1102–1103
Independent search, 1093
Independent set, 1716, 1722, 1724–1729, 1734
Indicator decoder, 929
Indicator-based evolutionary algorithm (IBEA), 239
Indirect exchanges of information, 1105
Indirect memory-based communication graph, 1106
Individual step-sizes, 100
Inertia particle swarm optimization (IPSO), 1568
Inertia weight, 856, 859, 860
Inference, 1014
Information and communications technologies (ICT), 1184, 1203, 1214
Information geometry optimization (IGO), 105, 106
Information-sharing cooperation mechanism, 1104
Initialization, 94
Initiating solution, 955
Input minimization, 1225–1227, 1229
InsideOpt solver, 1006
Integer linear program (ILP), 365, 366, 372, 374, 375, 377, 384, 1544
Integer problems, neighborhoods for, 518–521
Integrated clustering and routing problem (ICRP), 1555
Integrated Production and Transportation Scheduling problem with Crowdsourced Delivery (IPTS-CD), 1400
Integrative hybrid metaheuristics, 649
Integrative relay hybrid GAs, 651
Integrative teamwork hybrid GAs, 650–651
Intensification, 408
Intensification-based tabu search, 1288
Intensity-based techniques
bacterial foraging optimization algorithm, 1457
genetic algorithm, 1456–1457
scatter search, 1457
Inter-element relationship, 1276
Inter-route neighborhood, 1400
Interconnection communication graphs, 1106
Intermediate recombination, 95
Internal vehicle scheduling problem (IVSP), 1426–1433
Inter shift, 1404
Inter swap, 1404
Intra shift, 1404
Intra swap, 1404
Intrusion detection system (IDS), 155, 156
Invariants, 390, 416–420
Inventory routing problem (IRP), 346, 1370, 1573
Inverse folding problem (IFP), 1299, 1300, 1302, 1303, 1307, 1316
Inverse reinforcement learning theory, 699, 734
Inversion, 1023
IPOP-CMA-ES, 108, 112–114
Irregular problems, 1247
constructive heuristics, 1247–1253
local search heuristics, 1254–1256
matheuristics, 1257–1260
metaheuristics, 1259–1263
Ising spin glasses, 284

Island model, 217, 926
Isomorphic edge colorings, 1497
Isomorphic 1-factorizations, 1497
Isotropic Gaussian distributions, 97
Isotropic self adaptation, 95
I-TASSER, 1302, 1312, 1314, 1315, 1317
Item-driven algorithms, 1231–1232
Iterated greedy (IG), 747, 768–770, 1615–1616
  acceptance criterion, 752
  algorithmic outline of, 749, 753
  construction, 751–752
  destruction, 750–751
  flow shop scheduling, 756–761
  for routing problems, 768
  for scheduling problems, 768
  framework, 749–753
  greedy construction heuristics, 748–749
  historical development, 761–763
  local search, 752–753, 765–766
  PFSP, 755–756
  principle of, 749
  repeated (greedy) construction algorithms, 764–765
  for scheduling problems, 767
  SCP, 754–755
  tree search algorithms, 766–767
  TSP, 753–754
Iterated local search (ILS), 49, 695, 696, 707–710, 732, 752, 753, 780–786, 800, 801, 926, 1360–1361, 1392, 1504, 1726, 1776
  advantage, 783
  algorithmic outline, 783
  algorithms, 787–791
  applications, 795–800
  for other problems, 799–800
  for routing problems, 796–797
  for scheduling problems, 797–798
  framework, 783–786
  historical development of, 791–792
  local search heuristics, 781–783
  relationship of, 792–795
Iteration charts, 62
Iterative closest point (ICP), 1447, 1451–1452, 1464
Iterative construction heuristic, 763
Iterative flattening, 763
Iterative greedy (IG) heuristic, 1720
Iterative hold-out cross-validation (IHOCV), 164, 165, 167, 172, 173
Iterative improvement, 470–473, 477, 480, 483–485, 489, 495, 499–501, 503, 504, 507, 605
Iterative process, 1050

J
Jackknife, 220, 223, 225
Jakobs's implementation, 1231
Job shops, 1603
Joint probability distribution, 585, 586, 588, 589, 592
  bivariate dependencies, 589
  multivariate dependencies, 590
  no dependencies, 589
Juvan-Mohar method, 1330

K
k-change neighborhood function, 472, 477, 480, 481
Kempe chains, 1723
Kendall tau distance, 1017, 1023
Kernel search, 916
Kernighan-Lin neighborhood function, 496
k-fixed penalty strategy, 1720, 1722
Knapsack capacity, 47
Knapsack problem, 630
  crossover operator, 635
  mutation operator, 637
  representation, 631
Knowledge-augmented recombination operator, 817
Knowledge discovery, 1355, 1356, 1371–1372, 1377
Knowledge-guided local search (KGLS), 458
k-opt heuristics, 1202
k-partition problem, 1274
Kronecker delta, 166
Kruskal's algorithm, 1795, 1797, 1806
k–th neighborhood, 519

L
Labeling, 1323
Lagrangean approaches, 600
Lagrangean relaxation, 600
Lagrange edge costs, 1806
Lagrange multipliers, 1807
Lagrangian heuristic (LH), 1190, 1754, 1798, 1803
  for DCMST, 1809–1811
  main advantages, 1803
Lagrangian multipliers, 1190
Lagrangian problem (LP), 1798
Lagrangian relaxation, 1658
Lamarckian approach, 819
Landside, 1413
Laplacian matrix, 1330

Large neighborhood search (LNS), 12, 82–83, 181, 198, 443, 763, 905, 906, 1361–1362, 1398–1400, 1405
Large-scale problems, 243
Large-scale VRP instances, *see* Vehicle routing problems (VRPs)
Large-stop Markov chains (LSMC) algorithm, 791
Largest distance decreasing algorithm (LDDA), 1658
Last-mile delivery with crowdshipping, 1387, 1392, 1406
  compensation strategies, 1390
  dynamic and stochastic information integration, 1389–1390
  mathematical formulation, 1387–1389
  transshipment points integration, 1391
Latin hypercube sampling, 633
Lbest model, 864
L-blocking, 1509
LEADINGONES function, 584, 585, 587
Learn-CMSA, 368
Learnheuristics, 344
Learning by analogy, 996
Learning classifier systems (LCS), 127
Learning factorized distribution algorithm (LFDA), 590
Learning rule optimization (LRO), 1033, 1034, 1048, 1077–1079
Left vertices, 1333
Legal neighbors, 402
Lin-Kernighan algorithm (LK), 457
Lin-Kernighan heuristic, 787, 788, 791
Linear arrangement, 1323, 1327, 1329–1332
Linear bias, 672
Linear block, 1140
Linear combinatorial optimization problem, 482–488
Linear complementary problem (LCP), 1734
Linear genetic programming (LGP), 26
Linear layout problems
  BMP, 1338–1342
  CMP, 1325–1329
  MinLA, 1329–1332
  SumCut, 1336–1338
  VSP, 1333–1336
Linear ordering problem (LOP), 221, 947, 1013, 1015–1017, 1024, 1026, 1027, 1323, 1330
  algorithmic approach, 1020–1023
  mathematical models, 1017–1020
Linear program (LP), 179, 1555
Linear programming problem, 1272
Linear programming relaxation, 1656
Linear relaxation, 1655
Liu and Teng's implementation, 1231
Load, 1323, 1329
Loading job, 1427
Local branching, 180–181, 916
Localization problem, 1545
Local minimum, 962
Local neighborhood topology particle swarm optimization, 1569
Local optima estimation, 228
Local pheromone update, 557
Local PSO model, 864
`LocalSearch` procedure, 213
Local search, 153, 428, 429, 455, 470–473, 478, 480, 488–491, 493, 495, 496, 499, 504–508, 536, 605, 669–670, 677, 746, 748, 750, 756, 758, 759, 763, 765–766, 768, 770, 815, 816, 928, 1495, 1496, 1506, 1610, 1613–1615, 1678–1679, 1798
  acceleration techniques, 1679–1683
  algorithms, 390, 559
  category, 711
  construction method, 450
  heuristics, 781–783
  hybridization, 752–753
  improvement method, 451, 456
  methods, 82–83, 523–526
  and neighborhoods, 1500–1501
  parameterization, 818
  problems, 489, 507
  procedure, 785–786
  pruning techniques, 1683–1686
  results, 759–761
  role of, 818–819
  solution representation, 450, 456
  TSP, 450–451
Local search based methods, 1722
  BLS, 1727
  CLS, 1727
  DLS, 1726, 1727
  GRASP, 1726
  iterated local search, 1726
  PLS, 1727
  simulated annealing method, 1722
  simulated annealing method, 1724
  tabu search, 1724–1726
  VNS, 1726
  VSS, 1726
Local search heuristics, 1234, 1254, 1500
  heuristics based on searching over sequences, 1254
  heuristics operating over complete layouts, 1255–1256

procedures based on ordered lists of items, 1234–1235
procedures based on the layout of solutions, 1235–1236
VNEP, 1538
WSP, 1533–1534
Location models, 1182, 1187
Location problems, 607
Location routing problem (LRP), 907–909, 1572
Location routing problem with stochastic demands (LRPSD), 1572
Log bias, 672
Logical model, 336
Logistics, 346–348, 1176, 1440
Logistics consulting industry, 1773
Logistics consulting services, 1778
Logistics management, 1700
Logistics-focused companies, 1773
LOLIB benchmark, 222
Long short-term memory (LSTM), 74, 155, 156, 158, 159
Long-term memory, 948–950
Long-term TS, 1286
Longest continuous segment (LCS), 1316
Longest processing time (LPT), 1608, 1609
Lot sizing problem, 1749
clients, 1751
complexity, 1751
demands, 1749
flexibility, 1751
inventory and backlogging, 1751
items, 1750
machines, 1750
multi-level, 1750
planning horizon, 1749
plants, 1750
setup structure, 1750–1751
single-level, 1750
Lot transfers, 1754
Lovász theta, 1734
Low-level 1/A/1C/S/SPSS parallelization strategy, 1097
Low-level heuristics, 15, 16
Low-level perturbative heuristics, 15
Lower bounds (LB), 1803, 1806

M

Machine learning (ML), 149, 150, 155, 344, 355, 593–594, 654, 1007, 1342, 1372–1373, 1414
Machine learning force fields (MLFFs), 163
Machine learning methodologies, 699–700
Manufacturing flow management, 1705
Many-objective optimization, 240
Maritime container terminals, 1412, 1413, 1440
areas of, 1413
berth allocation, 1415–1422
container storage, 1433–1440
goal of, 1412
internal vehicle scheduling, 1426–1432
landslide, 1413
optimization problems, 1414–1415
quay crane scheduling problem, 1422–1426
seaside, 1413
suitable performance of, 1413
yard, 1413, 1434
Markov chain, 105, 1149
Markov process, 93, 1164
Markov property, 93
Markov's inequality, 1143
Master-worker parallel programming model, 1096
Matching, 1497
Mathematical programming, 178, 992
Mathematical relaxation procedures, 600
Matheuristics, 570, 648, 1245–1247, 1257–1260, 1363–1364, 1370–1371
branch-and-price/column generation-based approaches, 1368–1369
decomposition approaches, 1364–1367
improvement heuristics, 1367–1368
Max mean dispersion problem, 1272
Max Pooling (MP) layers, 1061
Max-circuit/flip, 494
Max-cut problem (MCP), 517, 525
Max-min ant system (MMAS), 545, 557, 558, 560
Max-min diversity problem (MMDP), 1271, 1272, 1281, 1282
MAXDICUT problem, 330
Maximal first repetition time, 220
Maximally diverse grouping problem (MDGP), 519–521, 1274, 1285
Maximal spaces, 1233
Maximizing Cluster Approach (MCA), 1637
Maximum clique
construction heuristics, 1717–1720
local search, 1720–1721
metaheuristics (*see* Metaheuristics)
Maximum clique problem (MCP), 1289
Maximum cut problem, 320

Maximum diversity problem (MDP), 607–609, 954, 1271, 1272, 1284
diversification generator method, 609–610
improvement method, 612
reference set initialization, 610
reference set update method, 611
solution combination method, 611
subset generation method, 611
Maximum flow problem (MFP), 1526–1527
Maximum independent set (MIS), CMSA, 370–372
benchmark set, 375
definition of solution components, 372
experimental evaluation, 374–383
ILP solver parameters, 375
parameter tuning, 376–377
probabilistic construction of MIS solutions, 372–374
results, 377–383
sub-instance solving, 374
Maximum likelihood principle, 101, 102
Maximum minsum dispersion problem, 1273
Maximum satisfiability (Max-SAT), 707
Mean absolute error (MAE), 36, 171
Mean square error (MSE), 1449, 1452, 1459, 1460, 1462, 1463
Means-end-analysis, 996
Mechanics
classical, 262, 264
quantum, 264, 269
relativistic quantum, 264
statistical, 262, 269
Medical imaging, 1446, 1456
Medium-term decision making, 1749
Memetic algorithms (MAs), 127, 134, 138, 650, 810–812, 825, 833, 834, 1505
academic papers, 825
advanced, 819–820
applications, 824–832
bioinformatics and biomedicine, 826
classical, 813–814
combinatorial optimization, 826
data analysis, 826
engineering, 826
finances and investments, 827
fleet management and VRPs, 827
for network alignment, 820–822
future-generation, 832–834
local search, role of, 818–819
manufacturing and supply chain optimization, 828
memetic speciation, 825–832
network design and optimization, 827
population-based components, 816–818
real-world applications of optimization, 829
restart procedure, 818
structure, 812–820
telecommunications, 830
warehouse management, 830
weighted constraint satisfaction problems, 822–824
Memetic PSO (MPSO), 885–886
Memory-based variable neighborhood search (MVNS), 1397
Memory matrix, 101
Memory usage, 1686–1687
Meta-heuristics, 390, 391, 431, 434, 442, 443, 446, 458, 699, 703, 734, 1088, 1089
Meta-learning, 20
Metaheuristic algorithm, 5
Metaheuristic-based components, 651
Metaheuristic community, 1004
Metaheuristic for randomized priority search (Meta-RaPS), 217
Metaheuristic optimization of hyperparameters, 163–165
ALF feature calculation, 169–170
fundamentals of GPR, 165–167
generation of dataset, 168–169
investigation, metaheuristic optimizers, 167–168
optimization mechanisms, 170
optimization of $\mathcal{L}(\mathbf{v}|\mathbf{U}, \boldsymbol{\phi})$ cost function, 171, 172
optimization of RMSE($\boldsymbol{\phi}$) cost function, 173
results, 170–173
Metaheuristics, 4, 15, 42, 76, 152–154, 218, 232, 242, 603, 605, 609, 622, 756, 762, 765, 780, 781, 783, 792, 800, 810–812, 814, 817, 929, 942, 960, 962, 975, 984, 993, 999, 1007, 1193, 1208, 1244, 1259–1263, 1280, 1291, 1356, 1358, 1363, 1391–1392, 1422, 1431, 1440, 1471, 1478, 1482, 1485–1489, 1601, 1610, 1613, 1616, 1617, 1688, 1701–1710, 1790, 1813
acceptance criterion, 1614
ACO, 242
adaptive, 6, 10–14
AIS, 241
configuration, 8–10
definition, 513, 994
early period, 994, 996–998
evolutionary algorithms, 1400–1402
framework-centric period, 994, 1002–1004
genetic algorithms, 1237–1239, 1400–1402

GRASP algorithms, 1241
HGS, 1360
hybrid approaches, 1402–1404
hybridization and state-of-the-art frameworks, 1359–1362
ILS, 1360–1361, 1392
initial solution/population generation, 1614
intensification and diversification mechanisms, 1614
large neighborhood search, 1398–1400
LNS, 1361–1362
local search based methods, *see* Local search based methods
Max-Mean model, 1285–1287
Max-Min model, 1280–1282
Max-MinSum model, 1287–1288
Max-Sum model, 1282–1285
metaphor-centric period, 1004
method-centric period, 994, 998–1002
Min-Diff model, 1288–1290
multi-level, 7
multi-start algorithms, 1236
path relinking algorithms, 1241–1242
population based methods, 1358, 1727–1732
pre-theoretical period, 994–996
PSO, 241
representation of solution, 1613
scientific period, 994, 1005–1007
simulated annealing algorithms, 1239
tabu search algorithms, 1240–1241, 1400
termination criterion, 1614
trajectory-based, 1358
tree search algorithms, 1242–1244
unified methods, 1362
VNS, 1396–1398
Metaheuristics-based image registration
early evolutionary image registration methods, 1453–1454
feature-based techniques, 1454–1456
intensity-based techniques, 1456–1457
suitability, 1452–1453
Metaheuristic search (MHS), 152
Metaheuristics, for sports scheduling
applications, 1515–1516
genetic algorithms, 1505–1506
GRASP, 1503–1504
ILS, 1504
simulated annealing, 1502
tabu search, 1502–1503
VNS, 1504–1505
Metaheuristics International Conference (MIC), 1001
Metaheuristic technologies, 1789
Metric TSP, 474, 476, 478–480, 482, 501
Metropolis(–Hastings) algorithm, 288–290
m-farthest models, 1027
m-farthest-sum, 1020
MI ESs (MIESs), 106
Micro GAs (μGA), 651
Min-circuit/flip, 494
Min-conflict, 405
MineReduce strategy, 46–48
Minimization problem, 665
Minimum cost flow problem (MCFP), 1527
Minimum differential dispersion problem, 1273
Minimum equivalent subdigraph, 1826
Minimum fuel consumption problem (MFCP), 1470–1473, 1475, 1478, 1482, 1487, 1488
Minimum linear arrangement (MinLA) problem, 1329–1332
Minimum spanning tree (MST), 473, 1794–1796, 1801, 1803–1806, 1811–1815
Mirrored orthogonal sampling, 110
Mixed algorithm portfolio, 331–332
Mixed integer linear problem (MILP), 1013, 1016, 1368, 1663
Mixed integer programming (MIP), 390, 413
local branching, 180–181
polishing algorithm, 181–182
prepack optimization problem, 192
proximity search, 182
relaxation-induced neighborhood search, 181
vehicle routing, 200
Mixed variable neighborhood descent, 533–536
Mixed-integer linear programming (MILP), 974–976
Mixed-integer (MI) optimization, 106
Mixed-integer nonlinear programming (MINLP) model, 193, 195, 1471, 1474
ant colony optimization, 1481–1485
steady-state MINLP models, 1477
Mixed-integer search spaces, 106
Mobile WSN, 1542
Model, 391, 392, 395–401
Modeling language, 1006
Modern heuristics, 1000
Modern synthesis, 94
Modified DFS, 1801
Modularization Quality (MQ), 1637
Module Dependency Graph (MDG), 1636

Molecular dynamics (MD) simulations, 163
Monte Carlo algorithms, 321
Monte Carlo method, 212, 214
Monte Carlo simulation, 345, 350, 351
Monte Carlo Tree search, 700, 734
Motzkin-Straus formulation, 1733, 1734
Move, 943
Multi modal registration, 1450
Multi-armed bandit (MAB) problem, 715
Multi-compartment vehicle routing problem (MCVRP), 673
Multi-criteria decision-making (MCDM), 347
Multi-depot periodic VRP (MDPVRP), 1119
Multi-depot vehicle routing problem (MD-VRP), 1417
Multi-depot VRP (MDVRP), 1119, 1121
Multi-directional evolutionary algorithm (MDEA), 1401
Multi-layer perceptrons (MLPs), 244
Multi-level cooperation, 1112, 1113
Multi-level metaheuristics, 19
  AI-enabled multi-level metaheuristics, 21
  and hyper-heuristics, 14–19
Multi-meme algorithms, 832
Multi-modal optimization problems, 108
Multi-objective ACO (MOACO), 568
Multi-objective cost functions, 151
Multi-objective evolutionary algorithm (MOEA), 135–137, 232, 234–236, 244, 449, 1043
  diversity maintenance mechanism, 235
  MOEA/D, 238
  NSGA-II, 237
  PAES, 237
  PESA, 238
  recent algorithmic trends, 239–240
  selection mechanism, 235
  SPEA2, 236
Multi-objective genetic algorithm (MOGA), 1299, 1308
Multi-objective NAS (MOO-NAS), 1043, 1048
Multi-objective optimization (MOO), 697–698, 1045, 1342, 1477, 1487, 1488
  GLS for, 446–450
  NSGA-II for, 1487–1488
Multi-objective particle swarm optimization algorithm, 698
Multi-objective problems, 567–568
Multi-plant capacitated lot sizing problem (MPCLSP), 1748, 1749, 1752, 1754, 1755, 1767, 1768
Multi-plant capacitated lot sizing problem with process flexibility and transportation costs (MPCLSP-PFT), 1761
  computational experiments, 1765–1766
  extended classical formulation, 1761–1762
  facility location reformulation, 1762–1764
  results of experiments, 1766
Multi-plant lot sizing problem (MPLSP), 1748, 1752–1754
  computational experiments, 1758–1760
  network flow-based formulation, 1755–1757
  results of experiments, 1759–1760
  set up of experiments, 1758–1759
  solution methods, 1754–1755
Multi-plant uncapacitated lot sizing problem (MPULSP), 1748, 1752–1755, 1768
Multi-point-crossover (MPX), 1657
Multi-source capacitated facility location problem with customer incompatibilities (MS-CFLP-CI), 63–65
Multi-start adaptive threshold accepting algorithm (MS-TA), 218
Multi-start algorithm, 213, 214, 218, 219
Multi-start methods
  average rankings of the methods according to neighborhood, 225
  average rankings of the methods according to sample size, 226
  average rankings of the methods according to the type of problem, 225
  average relative errors and standard deviations, 226, 227
  Bootstrap method, 220
  Chao1984 method, 221
  Chao&Bunge method, 221
  Chao&Lee methods, 221
  combinatorial optimization, 215–218
  estimating the number of local optima, 218, 219
  first repetition time, 220
  global optimization, 214, 215
  Jackknife method, 220
  maximal first repetition time, 220
  pseudo-code for, 213
  Schnabel census, 220
Multicommodity capacitated network design problem (MCND), 1102
Multicommodity flow problems, 1527–1528
Multidepot vehicle routing problem (MDVRP), 1572
Multilayer neural network, 73
Multimedia network, 1543

Multimodal GAs, 654
Multimodal optimization, 135, 591
Multinomial sampling, 221
Multiobjective CMA-ES, 110
Multiobjective evolutionary algorithm (MOEA), 1812
Multiobjective evolutionary algorithm based on decomposition (MOEA/D), 238, 449
Multiobjective GAs, 653
Multiobjective metaheuristics, 242, 243
Multiobjective optimization, 591
Multiobjective optimization problems (MOPs), 233
Multiobjective particle swarm optimizers (MOPSOs), 241, 887–889
Multiple bin-size bin packing problem, 1242
Multiple heuristics, 819
Multiple levels of decomposition, 1090
Multiple local search, 820
Multiple metaheuristic optimizers, 163
Multiple optima
  college rankings and fairness, 1024–1025
  premier league, 1026
Multiple phase neighborhood search–greedy randomized adaptive search procedure (MPNS-GRASP), 1577
Multiplicative drift theorem, 1159–1162
Multipoint search selection perturbation, 15
Multistart heuristic (MSH), 43
  elite set generation phase, 43
  hybrid phase, 43
Multistart techniques, 868
Multivariate dependencies, 590
Multivariate Gaussian distribution, 95
Mutant vector, 132
Mutation, 817
  operator, 93, 95
  parameters, 95
  probability, 645
  strength, 98
  testing, 1632
Mutative self-adaptation, 91
Mutative step-size control, 97, 98
Mutual information maximization for input clustering (MIMIC), 589, 594
Mutual information (MI), 1450

N

Natural ESs, 105
Natural gas pipeline network optimization, metaheuristics ant colony optimization, 1481–1485
  basic model, description of, 1473–1474
  classical approaches, 1477–1478
  network topology, 1474–1477
  NSGA-II for multi-objective optimization, 1487–1488
  particle swarm optimization for non-isothermal systems, 1486
  problem description and modeling framework, 1472–1477
  simulated annealing for integrated preventive maintenance, 1487
  simulated annealing for time-dependent systems, 1486–1487
  tabu search, 1478–1482
Natural gas transmission system, 1472
Natural gradient method, 105
Needle-in-a-haystack function, 1139
Negative assortative mating, 644
Negative-drift theorem, 1164–1166
Neighbor selection, 402
Neighborhood, 391, 401, 855, 863–865, 1506
  for binary problems, 516–518
  change, 962
  connectivity of tournament space, 1511–1514
  considerations, 1510–1511
  for continuous optimization problems, 515–516
  evaluation procedure, 1096
  function, 471–474, 476, 477, 479–489, 493–496, 500, 501, 507, 1720
  GPTS, 1510
  HAS, 1507
  for integer problems, 518–521
  OCs, 1391
  for permutation problems, 520–523
  PRS, 1508, 1509
  PTS, 1508, 1509
  radius, 864
  RS, 1507–1508
  size, 856
  structure, 669
  topology, 864
  TS, 1507
Nested variable neighborhood descent, 530–533
Net mass flow rate, 1473
Network Alignment, 820–822
Network-based IDS (NIDS), 155
Network flow-based formulation, 1755–1757
Network intrusions, 154
Network optimization
  basic network problems, 1525
  MCFP, 1527
  MFP, 1526–1527

Network optimization (*cont.*)
multicommodity flow problems, 1527–1528
PPSPP, 1525
preliminaries, 1525
VNEP, 1534–1538
WSP, 1528–1534
Network topology, 1474
cyclic topology, 1474
linear topology, 1474
on solution algorithms, 1475
pipeline network topologies, 1476
tree topology, 1474
Network traffic data, 154
Network virtualization, 1534
Neural architecture search (NAS), 1033–1035, 1037–1039, 1077
comprehensive survey, 1042–1043
evolutionary techniques, 1036
technical view, 1040–1041
Neural attention, 75
Neural-GLS, 445
Neural large neighborhood search (NLNS) methods, 83
Neural network (NN), 1001, 1034, 1735
automated, evolutionary design, 1041
evolutionary design, 1046–1047
Niche-radius, 108
Niching, 105, 107, 135, 141
Niching PSO, 882–883
Niching technique, 135
No free lunch theorem (NFLT), 132, 811, 834
Node selection criteria, 205–206
Node value operator, 980
Noisy intermediate-scale quantum (NISQ), 280
Non-approximability, 480–481
Non-autoregressive approaches, 81–82
Nondominated Sorting Genetic Algorithm (NSGA), 237
NSGA-II, 237, 1487–1488, 1813–1815
NSGA-III, 238
Non-guillotine cutting pattern, 1230
Nonlinear continuous functions, 515
Nonlinear equality constraint, 1472
Nonlinear nonconvex set, 1472
Nonlinear programming (NLP) model, 1471, 1474
steady-State NLP models, 1477
tabu search, 1478–1482
Non-revisiting GA, 643
Nonsequential dynamic programming (NDP), 1477–1479, 1481, 1482
Non-standard evolution strategies, 106–107
algorithm configurators and hyperparameter optimization, 111
mirrored orthogonal sampling, 110
multi-criterion and constraint-handling ESs, 109–110
niching and multi-population ESs, 107–108
noise-handling and robust optimization with ESs, 108–109
Non-stepwise reconfigurable techniques, 77
NP-complete, 1751
NP-completeness proof, 473–476
NP-hard, 1325, 1334, 1338
NP-hard combinatorial optimization problems, 337
NP-hard optimization problem, 152
Numbering, 1323, 1337
Numerical measures, 1279
Nurse rostering, 703

O

Objective function, 399, 961, 1020, 1023, 1472, 1473
Objective function adjustment (OA) algorithm, 445
Objective function value (OFV), 1485
Objective functions, 399
Objective interface, 400
OBS, 353
Occasional couriers (OCs), 1386–1391, 1396, 1398, 1399, 1402, 1405
Odd cycle, 1497
Off-line parameter initialization, 8
Offspring population, 1135
OGA, 1772–1774, 1790, 1791
beverage industry, practical case in, 1778–1779
broad range of techniques, 1790
computational experiments, 1781–1790
contribution to business value, 1790
current solution, 1787, 1788
delivery zones, 1782
effective application of metaheuristics, 1790
financial sustainability, 1790
handling real-world complexities, 1790
Hexaly template, 1776–1778
high demand comparison, 1785
low demand comparison, 1784
main screen, 1780
optimized solution, 1787

overlapping routes, 1783
routing, 1774–1776, 1783
solution, 1788, 1789
One-way constraints, 416
Online or off-line learning, 689
Online parameter control, 931
Open shops, 1603
Open vehicle routing problem (OVRP), 1571
OpenMP, 931
Operational semantics of invariant, 417
Operations constraints, 1604
Operations research (OR) methods, 72
Opposition-based PSO, 886
Optimal bandwidth, 1339
Optimal change rate, 98
Optimal coloring, 1716
Optimal computing budget allocation (OCBA), 887
Optimal restart strategies, 323–326
Optimal selection difference, 98
Optimal solution, 962, 1495
Optimal solution set, 1027
Optimization, 4, 512, 922, 951, 960, 961, 972, 973, 975, 983, 992, 995, 998
Optimization for PLAs (OPLA), 1630
Optimization methods, 1495–1496
Optimization problems, 1496, 1546, 1547, 1551–1553, 1556, 1699, 1701, 1703, 1709
Optimization procedure, 1451
Optimized feature selection, 1627, 1630
OptQuest/Multi-start or OQMS, 215
Order based encoding, 1730
Order crossover, 636
Order fulfillment, 1705
Ordered 1-factorization, 1498
Organic speciation, 107
Oriented edge coloring, 1500
Oscillating assignment, 942
Output maximization, 1225, 1226, 1228

P

Packing, 688, 693, 695, 696, 699, 700, 707, 733, 734
Parallel algorithms, 1342
Parallel Distributed Graphical Programming (PDGP), 26
Parallelism, 244
Parallelization, 592
Parallel machine problems, 1602
Parallel meta-heuristic search, 1089–1091
algorithm-based parallelization strategies, 1096–1099
asynchronous cooperation, 1109–1114
cooperative search, 1103–1107
decomposition strategies, 1091–1093
explicit SSD, 1099–1102
implicit SSD, 1102–1103
knowledge creation in cooperative search, 1114–1122
performance measures, 1095–1096
search strategies, 1093–1095
synchronous cooperation, 1107–1109
Parallel optimization, 323
Parallel speedup, 327
Parameter adaptation, 571–572
Parameter values, 980
Parameterized flexibility optimization, 1438
Parent population, 1135
Pareto archived evolution strategy (PAES), 110, 237
Pareto-based MOEAs, 240
Pareto dominance, 110, 235, 888
Pareto envelope-based selection algorithm (PESA), 238
Pareto front (PF), 234, 237, 240, 446, 447, 888, 1811
Pareto local optimum set, 447–449
Pareto local search (PLS), 447
Pareto optimal front, 136, 1815
Pareto optimality, 233, 1828, 1829
Pareto optimal set, 234, 236, 240
Pareto optimal solutions, 888
Pareto ranking, 235, 238
Pareto set, 109
Parkinson's disease, 142
Partial differential equations (PDEs), 1472, 1474, 1477
Partial OPtimization Metaheuristic Under Special Intensification Conditions (POPMUSIC), 904, 917
adaptation for location-routing, 907–910
BAP, 913–915
clustering, 911
CM, 915
CVRP, 910–911
empirical complexity, 909–910
initial solution, 907–909
max weight stable set problems, 912–913
part and sub-problem definitions, 909
seed-part selection, 909
template, 904–906
TSP, 911
Partial round swap (PRS), 1508, 1509
Partial Solver Group, 1120
Partial team swap (PTS), 1508, 1509
Partially Matched Crossover (PMX), 821

Particle swarm optimization (PSO), 126, 241, 307, 852–854, 889, 1281, 1340, 1455–1456, 1486, 1566–1570
alleviating local minimizers, 868–869
analysis of algorithms, 1581–1586
application, 1570–1571
bare bones PSO, 875
basic model, 854–857
binary PSO, 874
CNTPSO, 1577–1578
comprehensive learning PSO, 880–881
convergence and parameter setting, 857–873
cooperative PSO, 879–880
DAPSO, 1579
detecting several minimizers, 869–870
DHPD, 1580–1581
DPSO, 1580
early precursors, 858
enhanced and specialized PSO variants, 874–889
for MMDP, 1282
for non-isothermal systems, 1486
fully informed PSO, 876
GLNPSO, 1579–1580
guaranteed convergence PSO, 874
HybGenPSO, 1576–1577
HybPSO, 1577
hybrid PSO, 884–885
h_PSO, 1578–1579
in noisy environments, 887
initialization, stopping conditions and boundary violation, 865–867
main variants of VRP, 1571–1574
memetic PSO, 885–886
MODPSO, 1581
MOPSO, 1580
2MPSO, 1579
multiobjective PSO, 887–889
neighborhood, 863–865
niching PSO, 882–883
NPSO, 1580
opposition-based PSO, 886
penalty functions for constrained optimization problems, 870–872
performance-enhancing techniques, 867–873
PMPSO, 1580
PVPSO, 1579
QPSO, 1576
quantum PSO, 876
solving discrete problems, 872–873
SR-PSO, 1575–1576, 1579
stability analysis, 860–863
standard PSO, 883
StPSO, 1580
TRIBES, 881–882
unified PSO, 877–879
velocity clamping and inertia weight, 859–860
Particles, 854
Partitional clustering, 593
Partition-based clustering, 591
Path length control, 102–103
Path relinking (PR), 674–676, 927, 955–957, 996, 1000, 1098, 1241–1242, 1577
p-control (pC), 1094
Penalty function method, 1722
Penalty/incentive function, 949
Perfect matching, 1497
Performance bound, 485–488
Performance Per Capita (PCP), 1831, 1847, 1848
Performance ratio, 472, 477, 478, 480, 481, 508
Periodic construction-deconstruction, 763
Periodic vehicle routing problem (PVRP), 1119, 1572
Permutation-based problems, 593
Permutation flow shop scheduling problem (PFSP), 221–226, 755, 756, 789–791
Permutation problems, 873
neighborhoods for, 520–523
Permutations, 589, 591, 593
Personal protective equipment (PPE), 347
Personnel scheduling (PS), 717, 721
Perturbation, 784–787, 1657
Perturbative local search method, 765–766, 782
Phased local search (PLS), 489, 504, 506, 507, 1727
completeness, 493, 494
reduction, 490, 496, 504–507
Phenotype, 134
Pheromone
deposition, 554
evaporation, 554
initialization, 557
limits, 558–559
reinitialization, 557, 560
trails, 544, 550, 559
update, 553–556
Phylogenetic uncertainty, 1015
Pickup and delivery vehicle routing problem with time windows (PDVRPTW), 1194–1203

Pickup-delivery customer pair shift operator, 1201
Pipeline network topologies, 1476
Pipeline resistance parameter, 1474
Pipe VND (P-VND), 527, 529
Placement (deployment) problem, 1545
Planning horizon, 1749
Plateau block, 1140
Plateau unitation block, 1141
Pointer networks, 75
Point-to-point shortest path problem (PPSPP), 1525
Poisson bracket, 266
Polishing algorithm, 181–182
Pólya's principle, 997
Polygon method, 1498
Polynomial approximation functions, 1474
Polynomial bias, 672
Polynomially searchable neighborhood function, 474, 476, 480–486, 488
Polynomial Parity Argument for Directed graphs (PPAD), 508
Polynomial Parity Argument (PPA), 508
Polynomial Pigeonhole Principle (PPP), 508
Polynomial-time algorithm, 481–483, 489, 1795
Polynomial time approximation scheme (PTAS), 1656, 1660
Polynomial-time Local Search (PLS), 489–496, 498–501, 504–508
Population based methods
  ant colony optimization, 1731–1732
  evolutionary and genetic algorithms, 1728–1731
Population-based algorithm, 1153
Population-based approach, 601
Population-based components, 816–818
Population-based ESs, 91
Population-based incremental learning (PBIL), 131, 589, 592, 594
Population-based metaheuristics, 513, 1358
Population-based MHS algorithms, 153
Population-based optimization algorithms, 1302
Population-based SLS methods, 795
Population of individuals, 629
Positive Not-All-Equal Max-3Sat (POS NAE MAX-3SAT), 495
Potential energy surface (PES), 168
Predator prey evolution strategy, 110
Prefix notation, 27
Pre-marshalling problem, 1436
Prepack optimization problem, 192
Pre-processing, 1779
Pressure variables, 1483
Primal-dual algorithm, 671
Primal-dual VNS (PD-VNS), 961, 969, 970
Primitive set, 27
Prim's algorithm, 1795
Principal algorithms, 128
Principal components analysis (PCA), 101
Probabilistic analysis, 471
Probabilistic graphical model, 584
Probabilistic rule, 551
Probabilistic Tabu Search, 999
Probability function, 1483
Probability matching (PM), 714
Problem flexibility, 1014
Problem independent state features, 78
Problem-specific algorithms, 992
Problem-specific heuristics, 993
Process constraints, 1603–1604
Processing route, 1601
Processing times, 1601–1602
Product development and commercialization, 1705
Product Line Architecture (PLA), 1627, 1628, 1630
Production routing problem (PRP), 1572
Productivity, 1749
Professional fleet (PF), 1387
Profitable synergy, 648
Program-Expression (PE), 27, 29, 32
Progressive party, 406–410
Protein
  design, 1300–1302
  engineering, 1296
  samples, 1308–1309
  structure, 1298, 1300, 1302
Proximate optimality principle (POP), 442, 916
Proximity search, 182
P-rule, 27–29, 35
Pruning techniques, 1683
  granular neighborhoods, 1683–1685
  limited reoptimization, 1685–1686
Pseudo-Boolean function, 1138
Pseudo-code
  of DM-MSH, 44
  of MDM-MSH, 44, 45
  of MR-MSH, 48
  of pattern-based randomized CW solution generation, 52
Pseudo-random proportional rule, 552
Pseudocode, 1632

Q
Quadratic assignment problem (QAP), 221, 444, 545, 547, 550, 556, 560, 561, 567, 571, 572, 781, 782, 788–789
Quadratic unconstrained binary optimization (QUBO) model, 1007
Quality of service (QoS), 1543
Quantile constraint (QC), 1308
Quantum adiabatic computing (AQC), 281
Quantum advantage, 276
Quantum annealing (QA), 282
Quantum binary PSO, 307
Quantum chromosome, 297–299
Quantum computer hardware, 279
Quantum computing, 1007
Quantum field theory (QFT), 264
Quantum gate, 276
Quantum information, 278
Quantum inspired evolutionary algorithms, 296
Quantum mechanics, 163, 264, 269
Quantum mutation, 303
Quantum observation step, 300
Quantum PSO (QPSO), 876, 1576
Quantum spin glass, 285
Quantum supremacy, 276
Quantum swarm, 307
Quantum tunnelling, 273, 285, 292
Quasi-memory vector, 100
Quasi-Newton method (QN), 1452
Quay crane assignment, 1415
Quay crane scheduling problem (QCSP), 1422–1426
Qubit, 274, 275, 278, 297–299
Quenching, 283

R
Radial basis function (RBF), 166, 244
Radio link frequency assignment problem (RLFAP), 459
Random, 1418, 1438
Random access memory (RAM), 1686
Random bias, 672
Random greedy method (R-G), 1418
Random insertion heuristic (RIH), 754
Random key, 922, 1506
Random-key genetic algorithm (RKGA), 29, 30, 922, 932
  BRKGA (*see* Biased random-key genetic algorithm (BRKGA))
  customizing, 928–929
  evolutionary process, 922
Random-key interval, 929
Random-key optimizer (RKO), 678
Random local search (RLS), 1137
Random permutation, 196, 197
Random vector, 100
Random VND, 528
Randomised search heuristics, *see* Stochastic search algorithms
Randomized rounding technique, 1656
Rank-based ant system (RAS), 545
Ranking, 1822, 1830–1834, 1839, 1840, 1846, 1847
Rastrigin function, 105
Reactive GRASP, 670–672
Reactive local search (RLS), 1725
Reactive search, 11
Real-parameter optimization, 631
  crossover operator, 636
  mutation operator, 638
  representation, 632
Real-time fleet management system, 1185
Real-time management, 1203–1204
Real-valued quantum inspired evolutionary algorithm, 303
Reallocation model, 206
Recency, frequency, monetary (RFM) model, 1404
Recency-based memory, 944
Recombination operator, 93, 94
Rectangular problems (2D and 3D), 1230
  constructive heuristics, 1230–1234
  local search heuristics, 1234–1236
  matheuristics, 1245–1247
  metaheuristics, 1236–1245
Recurrent neural networks (RNNs), 74, 155
Recursive largest first (RLF), 1718, 1719, 1723, 1732
Reduced network, 1474, 1475
Reduced VNS (RVNS), 961, 963–964, 1335
ReEvo, 446
Reference points, 601
Reference set (RefSet), 601
Reference set creation sub–routine (RSCsr), 605
Reference set initialization, 610
Reference set update method (RSUM), 605, 611
Reformulation descent (RD), 971
Regression, 150
Regret algorithms, 997
Reinforcement learning (RL), 20
  based NAS methods, 1036
  CMSA, 368–370
Relapse probability, 1149

Relative improvement (RI), 1481, 1482
Relative percentage deviation (RPD), 562, 565, 759
Relativistic quantum mechanics, 264
Relativity
  general, 263
  special, 263
Relaxation-based algorithms, 1658
Relaxation-induced neighborhood search (RINS), 181
Relay nodes (RN), 1552
Relocation movement, 1436
Repeated (greedy) construction algorithms, 764–765
Repeated construction, 749
Rerouting algorithm (RRA), 1204–1208
Residence counts, 948
Resource allocation problems, 436
Resources processing, 1689–1691
Restart distribution, 320, 323, 324, 326
Restart period (or a restart parameter), 324
Restart strategy, 322
  mixed algorithm portfolio, 331, 332
  optimal restart strategies, 323, 324
  restart distribution, 323
  single algorithm portfolio, 326–329
  uniform restart strategy, 322
  universal restart strategy, 322
Restoration heuristics, 1798
Restricted candidate list (RCL), 12, 667, 677, 1281, 1287, 1404, 1726
Restricted Hamiltonian Cycle (RHC), 474
Restricted tournament selection (RTS), 643
Retrieval movement, 1436
Returns management, 1706
Reusable min-conflict procedure, 406
Reverse Cuthill-McKee (RCM) algorithm, 1337
Riemannian manifold, 105
Right vertices, 1333
Roadie, 1386
Robust optimization, 108
Root-mean-square deviation (RMSD), 1316
Root-mean-square error (RMSE), 164, 982
Rotated frame, 103
Round swap (RS), 1507–1508
Round-robin tournaments, 1494, 1495, 1498, 1506–1514
Route assignment and pricing procedure, 1399
Routing problems, 429, 435, 443–445, 451, 457, 458, 768, 796–797, 1355–1360, 1362, 1364–1371, 1373, 1375–1377, 1545
Ruin-and-recreate, 762
Runtime analysis, 1142, 1166, 1167
Rural postman problem (RPP), 457

S

Sample average approximation (SAA), 346
Sandwich theorem, 1734
SAT domain, 722
Saw-tooth GA, 642
Scalability, 1676
Scaling factor, 875, 882
Scatter search (SS), 600–603, 622, 942, 1098, 1456, 1731
  computational experiments, 612–622
  diversification generation method, 605
  framework, 601
  Google Scholar, 602
  improvement method, 605
  maximum diversity problem (*see* Maximum diversity problem (MDP))
  methodology, 601, 602
  reference set update method, 605
  search methods, 613–615
  search parameters, 618–620
  search profiles, 616–617
  solution combination method, 607
  subset generation method, 607
  template, 602, 605
Scene allocation, 413–415, 424
Schedule length, 561
Scheduling constraints, 1603
  operations constraints, 1604
  process constraints, 1603–1604
  storage constraints, 1604
  transportation constraints, 1604
Scheduling, 591–593, 701–704, 767–768, 797–798, 1598–1600
  advanced heuristic methods, 1609–1610
  application to parallel machine problem, 1608–1609
  basic, 1607
  elaborated dispatching rules, 1609
  importance, 1599–1600
  iterated greedy, 1615–1616
  metaheuristics, 1613–1615
  notation and classification, 1601–1602
  objectives, 1605–1607
  NEH heuristic for permutation flow shop and makespan criterion, 1610–1612
  relevance, 1600–1601
  shifting bottleneck heuristic for job shop problem, 1612–1613
  simple dispatching rules, 1607–1608
  types and notation, 1602–1603

Schema theory, 1133
Schnabel census, 220
Schrödinger equation, 270
Scientific applications, 243
Scientific communities, 1007
Scientific study, 996
Scopus-indexed journal, 337, 339
Scoring rule, 1232
SC-values, 1336
Search, 391
Search agents, 854
Search-based software engineering (SBSE), 1622, 1623, 1639
  optimized feature selection, 1627–1630
  software module clustering, 1635–1639
  SPS, 1624–1627
  test case generation, 1630–1635
Search-based Software Testing (SBST), 1631
Search coordinator (SC), 1116, 1118
Search heuristics, 76
Search methods, 613–615
Search parameters, 618–620
Search profiles, 616–617
Search robustness, 1095
Search space, 1501
Search space separation (SSD), 1092
  explicit, 1092, 1099–1102
  implicit, 1093, 1102–1103
Search strategies
  asynchronous, 1094
  communications and learning, 1094
  control, 1094
  knowledge, 1094
  synchronous, 1094
Seaside, 1413
Second DIMACS Implementation Challenge, 1736
Second law of thermodynamics, 268
Second level of derandomization, 101–102
  evolution path, 102
  path length control, 102–103
  (μ_W, λ) Rank-μ CMA, 103
Second neighborhood, 407
Second-order differential equation, 862
Second-order Hessian information, 109
Secondary structure
  definition, 1305
  estimation, 1305–1306
Segment resource extension function, 1689
Selected string problems, *see* String problems
Selection difference, 98
Selection operator, 93, 634
Selection, extraction, recombination, and reallocation (SERR) matheuristic, 204, 208, 209
Self-adaptation, 95
Self-adaptive CMSA, 367–368
Self-adaptive control of p_m, 646
Self-adaptive mutation, 97
Semidefinite programming (SDP), 1734, 1735
Sensitivity analysis, 1834–1836
Sep-values, 1333
Sequence identity, 1303–1304
Sequential search, 1680–1681
Sequential variable neighborhood descent, 526–530
Set covering problem (SCP), 217, 754–755, 762
Shannon's information entropy, 886
Shift operator, 1201
Shifting bottleneck heuristic (SBH), 1612–1613
Ship routing problem (SRP), 1572
Short-term memory, 943–944
Short-term TS, 1286
Shortest processing time (SPT), 1418, 1607
Shortest processing time greedy (SPT-G), 1418
Shortest-empty-travel-time-first (SETTF), 1429
Simheuristics, 337, 338, 356, 648
  computational results, 352–355
  extensions, 343–344
  finance, applications in, 351
  healthcare, applications in, 350
  logistics, applications in, 346–348
  methodology, 341–343
  open challenges, 354–355
  production, applications in, 348–350
  stochastic optimization problems, 339–341
  transportation, applications in, 345–346
Similarity metric, 1449–1451
Simple adaptive mechanisms, 11
Simple genetic algorithm (SGA), 1166
Simple graph, 1497
Simple iterated greedy
  with local search step, 760–761
  without local search, 759–760
Simple SLS methods, 793
Simple variable neighborhood search, 6
Simulated annealing (SA), 267, 289, 608, 746, 748, 752, 755, 756, 761, 762, 999, 1001, 1239, 1331, 1398, 1402, 1404, 1405, 1486, 1502, 1657, 1722–1724, 1729
  for integrated preventive maintenance, 1487
  for time-dependent systems, 1486–1487

Simulated quantum annealing, 292
Simulation-based decision support system, 347
Simulation methods, 336
Simulation-optimization approaches, 337
Single algorithm portfolio, 326–330
Single compressor unit, 1473
Single machine problems, 1602
Single machine total weighted tardiness problem (SMTWTP), 797
Single round-robin tournament (SRR), 1495, 1498–1500
Single-echelon system, 1182
Single-level decomposition, 1090
Skewed general variable neighborhood search (SGVNS), 1332
Skewed VNS (SVNS), 961, 967–968
Small data set, 1328
Smallest position value (SPV), 873
Smart mobility, 1773
SMD-based local search operator, 1682
S metric selection Evolutionary Multi-Objective Algorithm (SMS-EMOA), 239
Social-only particle swarm optimization, 1569
Social term, 856
Software Development Life Cycle (SDLC), 1623
Software Engineering (SE), 1622
Software maintenance, 1635
Software Modularity Clustering Problem (SMCP), 1635–1638
Software Product Lines Online Tools (SPLOT), 1629
Software Product Line (SPL), 1627–1630
Software Project Scheduling (SPS), 1624–1627
  skills information, 1624
  staff information, 1624
  tasks information, 1624
Solution combination method (SCM), 607, 611
Solution completion, 1779
Solution construction, 551–553
Solution quality, 472
Solution robustness, 1012, 1014
Solution space, 961
Source code, 1006
Source quay crane/stack, 1427
Space-driven algorithms, 1232–1234
Space management, in 3D
  maximal spaces, 1233
  partitions, 1233
Spanning tree, 1795
Sparsification factor, 1684
Sparsification rules, 1684
Special relativity, 263
Speciation-based PSO (SPSO), 882
Spectral sequencing method, 1330
Speedup ratio, 331, 332
Spin glasses, 284
Split delivery vehicle routing problem (SDVRP), 1368
Square well, 877
Squeaky wheel optimization (SWO), 764, 1244
Stability analysis, 860–863
Stability number, 1716
Stable set, 1716
STABULUS, 1724
Stacking problem, 1436
Standard bit mutation (SBM) operator, 1135, 1138
Standard normal distributed random number, 95
Standard PSO (SPSO), 883
Static BAP, 1415
Static heuristic information, 551, 553
Static move descriptors (SMDs), 1681–1683
Static VND, 528
Statistical learning, 101, 105
statistical mechanics, 262, 269
Statistical tests, 1328
Steady-state assumption, 1474
Steady-state genetic algorithm, 640, 643, 651, 1135
Steady-state MINLP models, 1477
Steady-state NLP models, 1477
Steady-state replacement, 1729
Steiner nodes, 1796
Steiner Tree problem, 1796
STELLA software, 163
Step-size control, 95, 96
Stochastic gradient descent (SGD), 1033, 1039, 1042, 1044–1046
Stochastic local search (SLS), 43, 746, 748, 762, 763, 766, 768, 770, 783, 786, 792
  hybrid, 793–794
  population-based, 795
  simple, 793
Stochastic optimization problems, 336, 339–341
Stochastic problems, 569
Stochastic process, 1158
Stochastic search algorithms, 1133
  AFL, 1144–1157
  computational complexity, 1134–1135
  drift analysis, 1157–1166
  evolutionary algorithms, 1135–1139
  tail inequalities, 1142–1144
  test functions, 1139–1142

Stochastic search methods, 705
Stochastic universal selection, 634
Stopping criterion, 1200
Storage constraints, 1604, 1605
Storage movement, 1436
Strategic oscillation (SO), 217, 950–955, 1003, 1285
Strategy parameters, 98
Strength Pareto Evolutionary Algorithm (SPEA), 236
Strength Pareto Evolutionary Algorithm 2 (SPEA2), 236
Stretching technique, 869
String problems
  CSP, 1654–1658
  CTMSP, 1659
  diagnostic probes for bacterial infections, 1652–1653
  discovering potential drug targets, 1653
  experimental evaluation, 1663–1666
  FFMSP, 1660–1662
  FSP, 1659–1660
  motif search, 1653
  notation, 1654
  primer design, 1653
  simple ILP-based heuristic, 1663
Strings consensus, 1652
Strong ergodicity, 269
Strongly determined variables, 216, 673, 942
Strongly ergodicity, 269
SubKP fills, 1237
Subset generation method (SGM), 607, 611
Successive augmentation (SAG), 1331
SumCut/profile problem, 1336–1338
Summation invariant, 418
Superlinear parallel speedup, 327
Superlinear speedup, 326, 327
Supermarket chain grocery delivery crowdshipping problem, 1397
Superposition of states, 271
Supervised classification, 594
Supervised machine learning, 149, 174
  aspects and challenges, 151–152
  BWO-Conv-LSTM for hybrid intrusion detection, 154–162
  classification, 150
  local minima problem, solution to, 152
  metaheuristic optimization of hyperparameters of GPR model, 163–173
  regression, 150
Supplier relationship management, 1705
Supply chain management (SCM), 1698
  decision-making problems in, 1701–1706
  definition, 1699
  future research, 1708–1710
  manufacturing and resource strategies, 1702
  metaheuristics algorithms for problems, 1706–1708
  order fulfillment, 1705
  sourcing and inventory strategy in, 1702
  sustainability and green strategies, 1702
  transportation and distribution strategies, 1703
Surrogate models, 571
Sustainability and green strategies, 1702
Suzuki-Trotter formula, 274, 293
Swap operator, 980, 981
Swarm, 854
  algorithm, 1197–1199
  diversity, 860, 861
  explosion effect, 858
  intelligence (SI), 853, 1033, 1040
Symbolic regression (SR), 36, 133, 1828, 1829
Symmetric TSP, 477, 478, 501, 502
Symplectic structure, 272
Symposium on Search-Based Software Engineering (SSBSE), 1622
Synchronous cooperation, 1107–1109
Syntax trees (ST), 27
Synthesis, 409
Synthesized search, 409

T

Tabu, 410
Tabu list, 999, 1199, 1480, 1724
Tabu memory, 1000
Tabu search (TS), 6, 429, 442, 444, 457, 458, 461, 609, 942, 999, 1000, 1114, 1115, 1197, 1199–1202, 1281, 1283, 1286, 1400, 1402, 1405, 1471, 1478–1479, 1482, 1502–1503, 1724–1726, 1729–1732
  algorithms, 1240–1241
  empirical performance, 1481–1482
  for Max-Sum Model, 1284
  initial set of flows, 1479
  long-term memory, 948–950
  method, 321, 322
  neighborhood definition, 1479
  optimal pressure values, 1480
  path relinking, 955–957

pseudocode of procedure, 1480
short-term memory, 943–944
stopping criterion, 1481
strategic oscillation, 950–955
tabu list, 1480
tenure, 1200, 1724
Tabu search algorithm with dynamic neighborhood size (TSDNS), 1288
Tabu search with strategic oscillation (TS-SO), 1285
TABUCOL, 1725
Tail inequalities, 1142–1144
Talbi's taxonomy for metaheuristics, 45
Target vector, 132
Team orienteering problem (TOP), 457, 1572
Team swap (TS), 1507
Tenth DIMACS implementation challenge, 1736
Terminals, 27
Terrestrial network, 1543
Test bed, 197
Test case generation, 1630–1635
Test functions, 1139–1142
Testing protocols, 1006
Theoretical Aspects of Local Search, 472
Thermodynamic equilibrium, 267
Thermodynamics, 262, 267
Threshold accepting (TA), 218, 443
Tightly PLS-reducible, 505
Time complexity, 471, 472, 488, 489, 499, 500, 508
Time series forecasting (TSF) problem, 982
Time series prediction methodologies, 701
Time-dependent partial differential equation, 1474
Time-to-target (TTT) plot, 63
TimeFold, 1006
Timetabling, 688, 691, 693, 696, 704, 705, 734
TOPOS, 1251–1253, 1262
Total score (TS), 1316
Tournament, 634
Trace, 274
Traffic engineering, 1524
Trajectory-based algorithms, 153
Trajectory-based metaheuristics, 513, 1358
Transductive methods, 79, 84
Transfer node, 1404
replacement, 1404
replacement with swap, 1404
Transferability, 20
Transformation model, 1448–1449
Transient analysis, 1472
Transient models, 1477
Transient optimization, 1472
Transition counts, 948
Transition graph complexity problem, 504
Transition probabilities, 1150
Transitive dominance, 1833
Transitive reduction, 1826
Transmission network, 1472, 1473
Transportation, 345–346
constraints, 1604
network, 1470, 1471, 1477, 1488
Transshipment flows, 1414
Transshipment of freights, 1412
Transverse Ising spin glass (TISG) model, 285
Traveling salesman problem (TSP), 75, 200, 289, 365, 457, 471, 472, 474, 476–480, 482, 500–502, 507, 545–548, 550, 556, 560–562, 567, 571, 572, 591, 593, 630, 717, 723, 749, 753–754, 781, 787–788, 905, 911, 1357, 1546, 1547, 1554, 1555, 1776
crossover operator, 635
GLS, 450–453
mutation operator, 637
representation, 631
Trees, 1795
classification of heuristics for, 1797
data structures, 1799–1801
with degree constraints, 1803
encodings, 1801–1803
expansion, 1797, 1816
fusion, 1797, 1816
search algorithms, 766–767, 1242–1244
structured population, 819
topology, 1474
Trial vector, 132
Triangulation, 1016
TRIBES algorithm, 881–882
Trotter dimension, 293
TSpack, 1240
Tuning, 700, 717
Two-dimensional bandwidth minimization problem (TDBMP), 1341
Two-dimensional cutting pattern, 1222
Two-echelon city dispatching problem, 1401
Two-echelon location routing, 1193–1194
Two-echelon single-source location problems, 1187–1193
Two-echelon vehicle routing problem, 1399, 1404
Two-fusion crossover operator, 1729
Two-phase tabu search intensification algorithm (TPITS), 1288
Two-stage simulated annealing (TSSA), 1331
Type-II maximum likelihood (ML-II), 164, 165, 167, 171, 172

U
Unauthorized network traffic activities, 154
Uncapacitated single allocation *p*-hub median problem (USApHMP), 534
Uncertainty principle, 273
Underground network, 1543
Underwater network, 1543
Unified methods, 1362
Unified PSO (UPSO), 877–879
Unified tabu search (UTS), 1199, 1211, 1212
Uniform crossover, 1135, 1136
Uniform Graph Partitioning (UGP), 494, 495
Uniform restart strategy, 322
Union VND (U-VND), 527, 529
Unitation, 1139
United Nations Conference on Trade And Development (UNCTAD), 1412
Univariate Gaussian distribution, 589
Univariate marginal distribution algorithm (UMDA), 589, 594
Universal restart strategy, 322
University of Amsterdam, 1841–1844
University of Copenhagen, 1841
Upper bounds (UB), 1803, 1806
Upper confidence-bound (UCB) algorithm, 715
Urban logistics, 1177

V
Value-based (VB) mechanism, 667
Variable depth search, 1726
Variable drift theorem, 1162–1164
Variable formulation search (VFS), 961, 972–974, 1327
Variable neighborhood decomposition search (VNDS), 916, 961, 969
Variable neighborhood descent (VND), 218, 514, 526, 536, 674, 961–963, 966, 974, 979, 983, 1332, 1396, 1397, 1399, 1404
  mixed, 533–536
  nested, 514, 530–533
  sequential, 514, 526–530
Variable neighborhood formulation space search, 970
Variable neighborhood programming (VNP), 979–983
  neighborhood structures, 980, 981
  objective function, 982
  parameters adjustment for forecasting problem, 982
  shaking, 982
  solution representation, 980
Variable neighborhood search (VNS), 443, 514, 526, 674, 794, 960, 961, 1021, 1022, 1027, 1099, 1290, 1327, 1328, 1338, 1390, 1396–1398, 1405, 1504–1505, 1726
  accept procedure with p secondary formulations, 972
  bandwith minimization problem, 973
  basic schemes, 961
  BVNS, 964–967
  coherence, 984
  continuous global optimization, 977–979
  discovery science, 983–984
  effectiveness, 985
  efficiency, 985
  generality, 985
  GVNS, 966–967
  innovation, 985
  interactivity, 985
  metric dimension problem, 973
  MILP, 974–976
  minimal doubly resolving set problem, 973
  multiplicity, 985
  PD-VNS, 969–970
  precision, 984
  robustness, 985
  RVNS, 963–964
  simplicity, 984
  SVNS, 967–968
  user-friendliness, 985
  variable neighborhood formulation space search, 970
  variable neighborhood programming, 979–983
  VFS, 972
  VND, 962–963
  VNDS, 969
Variable neighborhood search based on the metropolis criterion (VNSMC), 1404
Variable objective search, 1734
Variable space search (VSS), 1726
Vector evaluated PSO (VEPSO), 888
Vehicle productivity, 1430
Vehicle routing, 591, 593
  classical heuristics, 1356–1358
  instances, 1375
  knowledge discovery, 1371–1372
  large-scale instances, 1373–1375
  machine learning, 1372–1373
  matheuristics, 1363–1371
  metaheuristics, 1358–1363
  publicly available implementations, 1376
Vehicle routing and scheduling problem (VRSP), 1573

Vehicle routing problems (VRPs), 336, 456, 568, 724, 796, 827, 906, 1100, 1102, 1115, 1186, 1197, 1198, 1388, 1391, 1398, 1399, 1404, 1426, 1674, 1775
  additional design and implementation challenges, 1686–1692
  decomposition, 1677–1678
  initial solution construction, 1688
  limitation strategies, 1691–1692
  memory usage, 1686–1687
  problem definition and notation, 1676
  resources processing, 1689–1691
  scalability, 1676
  solution management, 1687–1688
  speeding up local search, 1678–1686
Vehicle routing problem with fuzzy demands (VRPFD), 1572
Vehicle routing problem with heterogeneous capacity, time windows, and occasional drivers (VRPHTO), 1401
Vehicle routing problem with occasional drivers (VRPOD), 1387, 1388, 1392, 1397, 1399, 1401, 1402
Vehicle routing problem with occasional drivers and time windows with multiple deliveries (VRPODTWmd), 1396, 1404
Vehicle routing problem with simultaneously pickup and delivery (VRPSPD), 1571
Vehicle routing problem with stochastic demands (VRPSD), 1571
Vehicle routing problem with stochastic travel times (VRPSTT), 1572
Vehicle routing problem with time windows (VRPTW), 1186, 1199, 1571
Vehicle routing problem with uncertain demands (VRPUD), 1573
Vehicle scheduling problem (VSP), 1430
Velocity, 855
Velocity clamping, 859–861
VeRoLog, 1565
Vertex coloring, 1716–1721, 1723–1726, 1730–1732, 1734, 1735, 1738
  approximate solutions, 1736
  construction heuristics, 1717–1720
  local search, 1720–1721
  metaheuristics (*see* Metaheuristics)
  proper, 1716
Vertex separation problem (VSP), 1333–1336
Very large-scale instances, 1674
Very large-scale integration (VLSI), 1322
Violation degree, 396
Violations, 396
Virtual network embedded problem (VNEP), 1534–1535, 1538
  construction heuristics, 1537–1538
  heuristic approaches, 1536–1537
  local search heuristics, 1538
  single VNEP variant, 1535–1536
VND with dynamic order (Dynamic VND), 527–528
VND with the static order of neighborhoods (Static VND), 527
VRP with time window (VRPTW), 1112, 1117
VSS, *see* Variable space search (VSS)

W
Wake effect, 183
WalkSAT, 390
Watts-Strogatz graphs, 381, 383
Watts-Strogatz model, 375
Wave function collapse (WFC), 1658
Wavefunction, 270
Weak ergodicity, 269
Web of Science, 603
Weighted Constraint Satisfaction Problems (WCSPs), 822–824
Weighted intermediate recombination, 103
Weight setting problem (WSP), 1529, 1538
  construction heuristics, 1532–1533
  heuristic approaches, 1531–1532
  local search heuristics, 1533–1534
  telecommunication networks, 1529–1531
Wilcoxon test, 1309
Wilcoxon-signed rank test, 726
Wind farm layout optimization problem, 182–189
Wireless sensor networks (WSN), 1014, 1542, 1544, 1557
  classical optimization problems, 1547
  classification, 1543
  clustering and mobile sink, 1549–1551
  coverage and density control, 1547–1548
  density control, clustering, routing, and sink mobility, 1553–1555
  fundamental requirement, 1545
  heterogeneous, 1543
  heuristic methods, 1551–1552
  homogeneous, 1543
  integrated problems, 1555–1556
  intercluster routes, 1546
  intra-cluster routes, 1546

Wireless sensor networks (WSN) (*cont.*)
optimization models, 1547
routing/connectivity, 1548–1549
sensor location, placement and coverage, 1552–1553
Workforce scheduling, 455
Workforce scheduling problem (WSP), 453–457, 460
Worst-case analysis, 471

X
XRLF algorithm, 1719

Y
Yard, 1413, 1433, 1434
Yard crane scheduling problem (YCSP), 1434–1435